

ADSP-2159x/ADSP-SC59x SHARC+ Processor

Hardware Reference

Revision 0.1, September 2020

Part Number
82-100147-01



Notices

Copyright Information

© 2020 Analog Devices, Inc., ALL RIGHTS RESERVED. This document may not be reproduced in any form without prior, express written consent from Analog Devices, Inc.

Printed in the USA.

Disclaimer

Analog Devices, Inc. reserves the right to change this product without prior notice. Information furnished by Analog Devices is believed to be accurate and reliable. However, no responsibility is assumed by Analog Devices for its use; nor for any infringement of patents or other rights of third parties which may result from its use. No license is granted by implication or otherwise under the patent rights of Analog Devices, Inc.

Trademark and Service Mark Notice

The Analog Devices logo, Blackfin, CrossCore, EngineerZone, EZ-Board, EZ-KIT Lite, EZ-Extender, SHARC, SHARC+, Blackfin+, A²B and VisualDSP++ are registered trademarks of Analog Devices, Inc.

EZ-KIT Mini and SigmaStudio are trademarks of Analog Devices, Inc.

All other brand and product names are trademarks or service marks of their respective owners.

Contents

Clock Generation Unit (CGU)

CGU Features	1-1
CGU Functional Description.....	1-1
ADSP-2159x CGU Register List	1-2
ADSP-2159x CGU Interrupt List	1-3
ADSP-2159x CGU Trigger List.....	1-3
CGU Definitions.....	1-3
CGU PLL Block Diagram	1-4
CLKOUT Selections	1-5
CGU Operating Modes	1-6
CGU Power-up Sequence	1-6
CGU Event Control	1-6
Oscillator Watchdog	1-7
CGU Programming Model	1-8
Configuring CGU Modes	1-9
Changing the Clock Frequencies	1-9
Changing the PLL Clock Frequency.....	1-9
Changing the CCLKn or SYSCLK Frequency Without Modifying the PLLCLK Frequency	1-10
Changing the OCLK Frequency.....	1-11
Selecting SCLK1_0 Source.....	1-11
Changing SCLK1_0 Frequencies.....	1-11
Aligning All Clocks	1-12
Shutting Off CCLKn from Another Master	1-12
Valid Clock Multiplier Settings	1-13
PLL Bypass and PLL Disable	1-13
ADSP-2159x Specific Information	1-14
ADSP-2159x CGU Register Descriptions	1-14
Core Clock Buffer Disable Register	1-15

Core Clock Buffer Status Register	1-16
CLKOUT Select Register	1-17
Control Register	1-19
Clocks Divisor Register	1-21
DIV Register Extension	1-24
Oscillator Watchdog Register	1-25
PLL Control Register	1-27
Revision ID Register	1-29
System Clock Buffer Disable Register	1-30
System Clock Buffer Status Register	1-32
Status Register	1-34
Time Stamp Counter 32 LSB Register	1-37
Time Stamp Counter 32 MSB Register	1-38
Time Stamp Control Register	1-39
Time Stamp Counter Initial 32 LSB Value Register	1-40
Time Stamp Counter Initial MSB Value Register	1-41

Clock Distribution Unit (CDU)

CDU Features.....	2-1
CDU Functional Description.....	2-1
CDU Block Diagram.....	2-1
CDU Definitions.....	2-3
CDU Clock Configuration Options	2-3
CDU Programming Model.....	2-4
Changing the PLL and Clock Frequency	2-4
Changing the Clock Frequency.....	2-5
ADSP-2159x CDU Register Descriptions	2-5
CDU Configuration	2-7
CLKIN Select	2-8
CDU Revision ID	2-9

CDU Status	2–10
Dynamic Power Management (DPM)	
DPM Features.....	3–1
DPM Functional Description	3–1
ADSP-2159x DPM Register List.....	3–1
DPM Definitions	3–1
DPM Operating Modes	3–2
Reset State	3–2
Full-on Mode.....	3–2
DPM Event Control	3–3
DPM Programming Model.....	3–3
ADSP-2159x DPM Register Descriptions	3–4
Control Register	3–5
Peripherals Disable Register 0	3–6
Peripherals Disable Register 1	3–8
Revision ID	3–10
Status Register	3–11
Reset Control Unit (RCU)	
RCU Features	4–1
RCU Functional Description	4–2
ADSP-2159x RCU Register List.....	4–2
ADSP-2159x RCU Trigger List	4–3
RCU Definitions	4–3
RCU Architectural Concepts	4–4
RCU Status and Error Signals.....	4–5
ADSP-2159x RCU Register Descriptions	4–5
Boot Code Register	4–6
Core Reset Outputs Control Register	4–9
Core Reset Outputs Status Register	4–10

Control Register	4-11
Message Register	4-13
Message Clear Bits Register	4-17
Message Set Bits Register	4-18
System Interface Disable Register	4-19
System Interface Status Register	4-20
System Reset Request Status Register	4-21
Status Register	4-22
Software Vector Register 0	4-24
Software Vector Register 1	4-25
Software Vector Register 2	4-26
SVECT Lock Register	4-27

System Event Controller (SEC) and Generic Interrupt Controller (GIC)

SEC Features.....	5-1
SEC Functional Description	5-2
ADSP-2159x SEC Register List.....	5-2
ADSP-2159x SEC Interrupt List	5-3
ADSP-2159x SEC Trigger List	5-3
Combined SEC and GIC Interrupt List	5-4
SEC Definitions	5-16
SEC System Event Aggregator (SEA)	5-17
SEC Block Diagram.....	5-17
SEC Fault Interface (SFI)	5-18
SEC Core Interface (SCI)	5-19
SEC Source Interface (SSI)	5-20
SEC Architectural Concepts	5-20
System Interrupt Acknowledge.....	5-20
System Interrupt Groups.....	5-21
System Interrupt Flow.....	5-21
System Interrupt Priorities	5-22

SEC Error.....	5-22
SEC Programming Model.....	5-22
Programming Concepts.....	5-23
Programming Examples.....	5-23
Fault Management Interface Programming Model	5-23
Configuring a System Source to Interrupt a Core.....	5-24
Core/SEC Handshake Requirements to Ensure Proper Interrupt Handling	5-25
Configuring a System Source as a Fault	5-26
Configuring the WDOG Expiry Event to Issue a System Reset.....	5-26
SEC Programming Restrictions	5-26
ADSP-2159x SEC Register Descriptions	5-27
SCI Active Register n	5-29
SCI Control Register n	5-30
SCI Group Mask Register n	5-32
SCI Priority Level Register n	5-34
SCI Priority Mask Register n	5-36
Core Pending Register n	5-37
SCI Source ID Register n	5-38
SCI Status Register n	5-39
Global End Register	5-41
Fault COP Period Register	5-42
Fault COP Period Current Register	5-43
Fault Control Register	5-44
Fault Delay Register	5-47
Fault Delay Current Register	5-48
Fault End Register	5-49
Fault Source ID Register	5-50
Fault System Reset Delay Register	5-51
Fault System Reset Delay Current Register	5-52
Fault Status Register	5-53

Global Control Register	5-55
Global Status Register	5-56
Global Raise Register	5-58
Source Control Register n	5-59
Source Status Register n	5-62
GIC Overview	5-63
GIC Functional Description	5-64
GIC Block Diagram	5-65
ADSP-2159x GICDST Register Descriptions	5-65
GIC Port 0 Enable	5-66
Software Generated Interrupt Priority Register	5-67
Shared Peripheral Interrupt Priority Register	5-68
Software Generated Interrupt Active Register	5-69
Software Generated Interrupt Control Register	5-70
Software Generated Interrupt Clear-Pending Register	5-72
Software Generated Interrupt Pending Set Register	5-73
Software Generated Interrupt Security Register	5-74
Shared Peripheral Interrupt Register	5-75
Shared Peripheral Interrupt Active Register	5-76
Shared Peripheral Interrupt Configuration Register	5-77
Shared Peripheral Interrupt Enable Clear Register	5-78
Shared Peripheral Interrupt Enable Set Register	5-79
Shared Peripheral Interrupt Pending Clear Register	5-80
Shared Peripheral Interrupt Pending Set Register	5-81
Shared Peripheral Interrupt Security Register	5-82
Shared Peripheral Interrupt Processor Targets Register	5-83
ADSP-2159x GICCPU Register Descriptions	5-83
Aliased Binary Point Register (ICCABPR)	5-84
Binary Point Register (ICCBPR)	5-85
CPU Interface Control Register (ICCICR)	5-86

End of Interrupt Register (ICCEOIR)	5–87
Highest Pending Interrupt Register (ICCHPIR)	5–88
Interrupt Acknowledge Register (ICCIAR)	5–89
Priority Mask Register (ICCIPMR)	5–90
Running Priority Register (ICCRPR)	5–91

Trigger Routing Unit (TRU)

TRU Features	6–1
TRU Functional Description	6–1
ADSP-2159x TRU Register List.....	6–2
ADSP-2159x TRU Interrupt List	6–2
ADSP-2159x TRU Trigger List	6–3
TRU Definitions	6–3
TRU Block Diagram	6–4
TRU Architectural Concepts	6–4
TRU Programming Model.....	6–4
Programming Concepts.....	6–5
Programming Examples.....	6–5
Configuring a Simple Trigger Sequence.....	6–5
TRU Event Control	6–5
TRU Status and Error Signals.....	6–5
ADSP-2159x TRU Register Descriptions	6–5
Error Address Register	6–7
Global Control Register	6–8
Master Trigger Register	6–10
Slave Select Register	6–11
Status Information Register	6–12

L2 System Memory

L2 System Memory Features	7–1
L2 System Memory Functional Description.....	7–1

ADSP-2159x L2CTL Register List	7-1
ADSP-2159x L2CTL Interrupt List	7-3
ADSP-2159x L2CTL Trigger List	7-3
L2 System Memory Block Diagram	7-3
L2 System Memory Architectural Concepts.....	7-4
Access Characteristics	7-4
Read/Write Latency and Throughput	7-4
L2 Memory Controller Block Diagram (Instance)	7-5
Arbitration and Priority.....	7-5
Data Integrity.....	7-7
ECC Algorithm.....	7-7
ECC Hardware Control	7-9
ECC Error Management	7-9
Memory Refresh.....	7-10
Power Modes.....	7-10
L2 System Memory Event Control.....	7-10
ECC Error Interrupt.....	7-10
ADSP-2159x L2CTL Register Descriptions	7-11
Control Register	7-13
Error Type 0 Address Register	7-16
Error Type 1 Address Register	7-17
Error Type 2 Address Register	7-18
Error Type 3 Address Register	7-19
Error Type 4 Address Register	7-20
ECC Error Address 0 Register	7-21
ECC Error Address 1 Register	7-22
ECC Error Address 2 Register	7-23
ECC Error Address 3 Register	7-24
ECC Error Address 4 Register	7-25
ECC Error Address 5 Register	7-26

ECC Error Address 6 Register	7-27
ECC Error Address 7 Register	7-28
ECC Error Address 8 Register	7-29
Error Type 0 Register	7-30
Error Type 1 Register	7-31
Error Type 2 Register	7-32
Error Type 3 Register	7-33
Error Type 4 Register	7-34
Initialization Register	7-35
Initialization Status Register	7-37
Power Control Register	7-39
Revision ID Register	7-41
Read Priority Count Register	7-42
Read Priority Count Register	7-43
Scrub Start Address Register	7-44
Scrub Count Register	7-45
Scrub Control Register	7-46
Status Register	7-48
L2 Port Error Status Register	7-51
Write Priority Count Register	7-52
Write Priority Count Register	7-53

Dynamic Memory Controller (DMC)

DMC Features	8-1
Feature Exclusions	8-2
DMC Functional Description	8-2
ADSP-2159x DMC Register List	8-3
ADSP-2159x DMC Register List	8-3
Protocol Controller.....	8-4
Efficiency Controller	8-4

Page-Based Scheduling	8-4
Same Master Transaction Scheduling	8-5
DMC Read Data Buffer	8-5
Closed Page Per Bank.....	8-5
SCB ID-Based Priority	8-5
Delaying up to Eight Auto-Refresh Commands.....	8-6
Page and Bank Interleaving	8-6
System Crossbar Slave Interface.....	8-6
Read/Write Command and Data Buffers.....	8-7
Peripheral Bus Slave Interface	8-7
Architectural Concepts	8-7
Controller On Die Termination (ODT).....	8-8
Mode Register Set and Extended Mode Register Set Command.....	8-8
DDR3 Reset Functionality.....	8-8
DDR3 SDRAM Organization.....	8-8
DMC Clocking	8-9
DMC DMA	8-9
DMC Operating Modes	8-10
Self-Refresh Mode	8-10
DMC Event Control.....	8-10
DMC Programming Model	8-10
PHY DLL Calibration	8-11
DDR3 ZQ Calibration Short CMD	8-11
DDR3 ZQ Calibration Long CMD.....	8-11
On Die Termination (DDR3).....	8-12
Leveling Techniques	8-12
Initializing the DMC	8-13
Resetting the DMC Lane	8-13
Performing ZQ Calibration	8-13
Programming the DMC Controller	8-14
ADSP-2159x DMC Register Descriptions	8-15

Configuration Register	8-17
Control Register	8-19
DLL Control Register	8-23
Data Calibration Address Register	8-24
Data Calibration Data 0 Register	8-25
Data Calibration Data 1 Register	8-26
Efficiency Control Register	8-27
Shadow EMR3 Register	8-31
Shadow MR0 Register (DDR3)	8-32
Shadow MR1 Register (DDR3)	8-34
Shadow MR2 Register (DDR3)	8-37
Mask (Mode Register Shadow) Register	8-39
Priority ID Register 1	8-41
Priority ID Register 2	8-42
Priority ID Mask Register 1	8-43
Priority ID Mask Register 2	8-44
DMC Read Data Buffer ID Register 1	8-45
DMC Read Data Buffer ID Register 2	8-46
DMC Read Data Buffer Mask Register 1	8-47
DMC Read Data Buffer Mask Register 2	8-48
Status Register	8-49
Timing 0 Register	8-52
Timing 1 Register	8-54
Timing 2 Register	8-55
ADSP-2159x DMC Register Descriptions	8-56
DDR CA Lane Control Register	8-57
Data Lane 0 Control Register 0	8-59
Data Lane 0 Control Register 1	8-61
Data Lane 1 Control Register 0	8-62
Data Lane 1 Control Register 1	8-64

DDR ROOT Module Control Register	8–65
DDR Calibration Control Register 0	8–67
DDR Calibration Control Register 1	8–68
DDR Calibration Control Register 2	8–69

One-Time Programmable Memory Controller (OTPC)

OTPC Features	9–1
OTPC Functional Description	9–1
ADSP-2159x OTPC Register List	9–1
ADSP-2159x OTPC Interrupt List	9–2
Error Correction	9–2
OTPC Event Control	9–2
OTPC Interrupt Signals	9–2
OTPC Status and Error Signals	9–2
OTP API Overview	9–2
OTP Programming	9–3
OTP Program	9–3
OTP Reading	9–4
OTP Get Field	9–4
OTP Counters	9–5
Lock API	9–5
ADSP-2159x OTPC Register Descriptions	9–6
OTP Security State Register	9–7
OTP Status Register	9–8

System Memory Protection Unit (SMPU)

SMPU Features	10–1
SMPU Functional Description	10–2
ADSP-2159x SMPU Register List	10–3
SMPU Interrupts	10–4
Memory Writes	10–4

Memory Reads	10-4
ID Comparison	10-4
Memory Region.....	10-8
SMPU Definitions	10-9
SMPU Block Diagram.....	10-10
SMPU Architectural Concepts	10-10
SMPU Operating Modes	10-11
SMPU Interrupt Signals	10-11
SMPU Status and Error Signals	10-12
ADSP-2159x SMPU Register Descriptions	10-12
Bus Error Address Register	10-14
Bus Error Details Register	10-15
SMPU Control Register	10-16
Exclusive Access IDn Address	10-18
Exclusive Access Status	10-19
Interrupt Address Register	10-20
Interrupt Details Register	10-21
Region n Address Register	10-22
Region n Control Register	10-23
SMPU Revision ID Register	10-26
Region n ID A Register	10-27
Region n ID B Register	10-28
Region n ID Mask A Register	10-29
Region n ID Mask B Register	10-30
SMPU Control Secure Accesses Register	10-31
Region n Control Secure Accesses Register	10-33
SMPU Status Register	10-34

General-Purpose Ports (PORT)

PORT Features	11-2
---------------------	------

PORT Functional Description.....	11-2
ADSP-2159x PORT Register List	11-3
ADSP-2159x PORT Trigger List.....	11-3
ADSP-2159x PINT Register List	11-4
ADSP-2159x PINT Interrupt List	11-4
ADSP-2159x PINT Trigger List.....	11-5
ADSP-2159x PADS Register List	11-5
PORT Architectural Concepts.....	11-7
Internal Interfaces	11-7
External Interfaces.....	11-7
GPIO Pin Function.....	11-7
Input Mode.....	11-7
Output Mode	11-7
Trigger Toggle Mode.....	11-8
Open-Drain Mode	11-8
Port Multiplexing Control.....	11-8
PORT Event Control.....	11-9
PORT Interrupt Signals	11-9
PORT Programming Model	11-11
ADSP-2159x PORT Register Descriptions	11-14
Port x GPIO Data Register	11-16
Port x GPIO Data Clear Register	11-20
Port x GPIO Data Set Register	11-24
Port x GPIO Output Toggle Register	11-27
Port x GPIO Direction Register	11-30
Port x GPIO Direction Clear Register	11-34
Port x GPIO Direction Set Register	11-38
Port x Function Enable Register	11-41
Port x Function Enable Clear Register	11-44
Port x Function Enable Set Register	11-47

Port x GPIO Input Enable Register	11-50
Port x GPIO Input Enable Clear Register	11-53
Port x GPIO Input Enable Set Register	11-56
Port x GPIO Lock Register	11-59
Port x Multiplexer Control Register	11-61
Port x GPIO Polarity Invert Register	11-63
Port x GPIO Polarity Invert Clear Register	11-67
Port x GPIO Polarity Invert Set Register	11-70
Port x GPIO Trigger Toggle Register	11-73
ADSP-2159x PINT Register Descriptions	11-74
PINT Assign Register	11-76
PINT Edge Clear Register	11-78
PINT Edge Set Register	11-81
PINT Invert Clear Register	11-84
PINT Invert Set Register	11-87
PINT Latch Register	11-90
PINT Mask Clear Register	11-94
PINT Mask Set Register	11-97
PINT Pin State Register	11-100
PINT Request Register	11-104
ADSP-2159x PADS Register Descriptions	11-108
DAI0 1 to 10 pins DS control	11-110
DAI0 11 to 20 pins DS control	11-113
DAI0 Port Input Enable Control Register	11-116
DAI1 1 to 10 pins DS control	11-117
DAI1 11 to 20 pins DS control	11-120
DAI1 Port Input Enable Control Register	11-123
DAIx Pull-Down Enable	11-124
DAIx Pull-Up Enable	11-125
Non-GPIO Drive Strength Register	11-126

Peripheral PAD Configuration0 Register	11-128
Peripheral Configuration1 Register	11-130
PORTA 0 to 7 pins DS control	11-131
PORTA 8 - 15 pins DS control	11-134
PORTB 0 to 7 pins DS control	11-137
PORTB 8 - 15 pins DS control	11-140
PORTC 0 to 7 pins DS control	11-143
PORTC 8 - 15 pins DS control	11-146
PORTD 0 to 7 pins DS control	11-147
PORTD 8 to 15 pins DS control	11-148
PORTE 0 to 7 pins DS control	11-149
PORTE 8 to 15 pins DS control	11-150
PORTF 0 to 7 pins DS control	11-151
PORTF 8 to 15 pins DS control	11-152
PORTG 0 to 7 pins DS control	11-153
PORTG 8 to 15 pins DS control	11-154
PORTH 0 to 7 pins DS control	11-155
PORTH 8 to 15 pins DS control	11-156
PORTI 0 to 7 pins DS control	11-157
PORTx Pull-Down Enable	11-158
PORTx Pull-Up Enable	11-159

Thermal Monitoring Unit (TMU)

TMU Features	12-1
TMU Functional Description	12-1
ADSP-2159x TMU Register List.....	12-1
ADSP-2159x TMU Interrupt List	12-2
ADSP-2159x TMU Trigger List	12-2
TMU Definitions	12-2
TMU Block Diagram	12-3

TMU Architectural Concepts	12-3
TMU and HADC	12-4
TMU Event Control	12-5
Status and Error Signals.....	12-5
TMU Programming Guidelines	12-5
ADSP-2159x TMU Register Descriptions	12-6
Alert High Limit Register	12-7
Alert Low Limit Register	12-8
Averaging Register	12-9
Temperature conversion blank register	12-10
TMU Control Register	12-11
Fault High Limit Register	12-12
Fault Low Limit Register	12-13
Gain Value Register	12-14
Interrupt Mask Register	12-15
Offset Register	12-16
Temperature Refresh Counter	12-17
Status Register	12-18
Temperature Value Register	12-19
 Housekeeping ADC (HADC)	
HADC Features	13-1
HADC Functional Description.....	13-2
ADSP-2159x HADC Register List	13-2
ADSP-2159x HADC Interrupt List	13-2
ADSP-2159x HADC Trigger List.....	13-3
HADC Definitions.....	13-3
HADC Block Diagram	13-4
HADC Signal Descriptions	13-5
HADC Architectural Concepts.....	13-5

Converter Operation	13-5
Auto-Scan.....	13-6
Channel Sequence Programming.....	13-6
ADC Transfer Function.....	13-7
Results.....	13-7
HADC Operating Modes	13-7
HADC Event Control.....	13-8
HADC Programming Model	13-8
ADSP-2159x HADC Register Descriptions	13-8
Channel Mask Register	13-10
Control Register	13-11
Channel Data Registers	13-13
Interrupt Mask Register	13-14
Status Register	13-15
 Watchdog Timer (WDOG)	
WDOG Features.....	14-1
WDOG Functional Description	14-2
ADSP-2159x WDOG Register List.....	14-3
ADSP-2159x WDOG Interrupt List	14-3
WDOG Block Diagram.....	14-3
Internal Interface	14-4
External Interface	14-4
ADSP-2159x WDOG Register Descriptions	14-4
Count Register	14-5
Control Register	14-6
Watchdog Timer Status Register	14-7
Watchdog Timer Window Register	14-9
 Link Port (LP)	
LP Features	15-1

LP Functional Description.....	15-1
ADSP-2159x LP Register List	15-2
ADSP-2159x LP Interrupt List	15-2
ADSP-2159x LP Trigger List.....	15-2
ADSP-2159x LP DMA Channel List.....	15-3
Block Diagram.....	15-3
External Connections	15-4
Internal Blocks	15-4
Architectural Concepts	15-4
Link Port Protocol.....	15-4
FIFO Buffers	15-7
Handshake for Link Port Enable Process	15-9
Clocking	15-10
Multi-Processor Connectivity	15-10
LP Operating Modes	15-12
LP Data Transfer Modes.....	15-12
Core Data Transfers	15-12
DMA Data Transfers.....	15-12
LP Event Control.....	15-13
Interrupt Signals	15-13
Enabling Link Port Interrupts	15-13
Status and Error Signals.....	15-14
LP Programming Model	15-14
Setting Up a DMA Transmit Operation	15-14
Setting Up a DMA Receive Operation.....	15-15
Setting Up a Core Transmit Operation	15-16
Setting Up a Core Receive Operation	15-16
ADSP-2159x LP Register Descriptions	15-17
Control Register	15-18
Clock Divider Value Register	15-20

Receive Buffer Register	15–21
Status Register	15–22
Transmit Buffer Register	15–24
Shadow Input Transmit Buffer Register	15–25
Shadow Output Transmit Buffer Register	15–26

Serial Peripheral Interface (SPI)

SPI Features.....	16–1
SPI Functional Description.....	16–2
ADSP-2159x SPI Register List	16–2
ADSP-2159x SPI Interrupt List	16–3
ADSP-2159x SPI Trigger List.....	16–4
ADSP-2159x SPI DMA Channel List	16–5
SPI Block Diagram	16–5
Transfer Protocol	16–5
Clock Considerations	16–7
Controlling Delay Between Frames	16–7
Flow Control	16–9
Slave Select Operation	16–10
Beginning and Ending a Non-DMA SPI Transfer	16–11
Transmit Operation in Non-DMA Mode	16–12
Receive Operation in Non-DMA Mode	16–12
Dual I/O Mode	16–12
Quad I/O Mode (SPI2 only)	16–13
Fast Mode	16–14
Memory-Mapped Mode (SPI2 only).....	16–15
Memory-Mapped Description of Operation.....	16–16
Memory-Mapped Architectural Concepts.....	16–17
Memory-Mapped Read Accesses.....	16–19
Memory-Mapped High-Performance Features.....	16–22
Merged Read Accesses	16–22

Wrap Around Accesses	16–22
Execute-In-Place (XIP, SPI2 only)	16–23
Memory-Mapped Mode Error Status Bits	16–24
Memory-Mapped Programming Guidelines	16–25
SPI Interrupt Signals	16–27
Data Interrupts.....	16–27
Status Interrupts.....	16–28
Error Conditions	16–28
SPI Programming Concepts.....	16–29
Master Operation in Non-DMA Modes	16–30
Slave Operation in Non-DMA Modes	16–30
Configuring DMA Master Mode.....	16–31
Configuring DMA Slave Mode Operation.....	16–32
ADSP-2159x SPI Register Descriptions	16–33
Clock Rate Register	16–35
Control Register	16–36
Delay Register	16–42
Masked Interrupt Condition Register	16–43
Masked Interrupt Clear Register	16–45
Interrupt Mask Register	16–48
Interrupt Mask Clear Register	16–50
Interrupt Mask Set Register	16–53
Memory Mapped Read Header	16–56
SPI Memory Top Address	16–60
Receive FIFO Data Register	16–61
Received Word Count Register	16–62
Received Word Count Reload Register	16–63
Receive Control Register	16–64
Slave Select Register	16–67
Status Register	16–70

Transmit FIFO Data Register	16-75
Transmitted Word Count Register	16-76
Transmitted Word Count Reload Register	16-77
Transmit Control Register	16-78

Octal Serial Peripheral Interface (OSPI)

OSPI Features.....	17-1
OSPI Functional Description	17-2
ADSP-2159x OSPI Register List	17-2
ADSP-2159x OSPI Interrupt List	17-3
OSPI Block Diagram.....	17-3
Architectural Concepts	17-4
Direct Access Controller (DAC)	17-5
Software Triggered Instruction Generator (STIG)	17-6
Servicing STIG Request	17-7
Arbitration Between DAC and STIG Access	17-7
Auto Polling for DAC Write Access	17-7
SPI Command Translation.....	17-8
Hold and Reset Control	17-9
Flash Instruction Type Support	17-9
Dual Data Rate (DDR) Operations	17-10
Data Sampling	17-13
PHY Module Architecture.....	17-14
PHY Pipeline Mode	17-17
Programming Concepts	17-17
Configuring OSPI after Reset.....	17-17
Programming Dummy Cycles.....	17-18
Configuring OSPI for Optimal Use.....	17-18
Configuring OSPI for DAC Read Operation.....	17-19
Configuring OSPI for DAC Write Operation.....	17-19
Issuing STIG Command	17-20

Entering XIP mode.....	17–22
Using PHY mode	17–23
ADSP-2159x OSPI Register Descriptions	17–26
Octal SPI Control Register	17–28
Device Delay Register	17–33
Device Read Instruction Control Register	17–34
Device Write Instruction Control Register	17–36
Device Size Control Register	17–38
DLL Observable Register (Lower)	17–39
DLL Observable Register (Upper)	17–40
Flash Command Address Register	17–41
Flash Command Control Register	17–42
Flash Command Control Memory Register	17–44
Flash Command Read Data Register (Lower)	17–45
Flash Command Read Data Register (Upper)	17–46
Flash Command Write Data Register (Lower)	17–47
Flash Command Write Data Register (Upper)	17–48
Interrupt Mask Register	17–49
Interrupt Status Register	17–50
Lower Write Protection Register	17–51
Mode Bit Control Register	17–52
Module ID Register	17–53
Polling Expiration Register	17–54
Opcode Extension Register (Lower)	17–55
Opcode Extension Register (Upper)	17–56
PHY Control Register	17–57
PHY DLL Master Control Register	17–58
Polling Flash Status Register	17–59
Read Data Capture Register	17–60
Remap Address Register	17–61

Upper Write Protection Register	17–62
Write Protection Control Register	17–63
Write Completion Control Register	17–64

Universal Asynchronous Receiver/Transmitter (UART)

UART Features	18–1
UART Functional Description	18–2
ADSP-2159x UART Register List	18–3
ADSP-2159x UART Interrupt List	18–3
ADSP-2159x UART Trigger List	18–4
ADSP-2159x UART DMA Channel List	18–5
UART Block Diagram	18–5
UART Architectural Concepts	18–6
Internal Interface	18–6
External Interface	18–6
Hardware Flow Control	18–7
Bit Rate Generation	18–7
Autobaud Detection	18–8
UART Debug Features	18–10
UART Operating Modes	18–10
UART Mode	18–10
IrDA SIR Mode	18–11
Multi-Drop Bus Mode	18–11
UART Data Transfer Modes	18–13
UART Mode Transmit Operation (Core)	18–13
UART Mode LIN Break Command	18–13
UART Mode Receive Operation (Core)	18–14
IrDA Transmit Operation	18–15
IrDA Receive Operation	18–15
MDB Transmit Operation	18–16
MDB Receive Operation	18–17

DMA Mode.....	18–17
Mixing DMA and Core Modes.....	18–18
Setting Up Hardware Flow Control.....	18–18
UART Event Control.....	18–18
Interrupt Masks.....	18–19
Interrupt Servicing	18–19
Transmit Interrupts	18–20
Receive Interrupts.....	18–21
Status Interrupts.....	18–22
Multi-Drop Bus Events.....	18–23
UART Programming Model	18–23
Detecting Autobaud	18–23
Using Common Initialization Steps.....	18–24
Using Core Transfers	18–24
Using DMA Transfers.....	18–24
Using Interrupts	18–24
Setting Up Hardware Flow Control.....	18–24
ADSP-2159x UART Register Descriptions	18–25
Clock Rate Register	18–26
Control Register	18–27
Interrupt Mask Register	18–33
Interrupt Mask Clear Register	18–37
Interrupt Mask Set Register	18–39
Receive Buffer Register	18–41
Receive Shift Register	18–42
Receive Counter Register	18–43
Scratch Register	18–44
Status Register	18–45
Transmit Address/Insert Pulse Register	18–50
Transmit Hold Register	18–51

Transmit Shift Register	18–52
Transmit Counter Register	18–53

Enhanced Parallel Peripheral Interface (EPPI)

EPPI Features	19–1
EPPI Functional Description	19–2
ADSP-2159x EPPI Register List.....	19–3
ADSP-2159x EPPI Interrupt List	19–3
ADSP-2159x EPPI Trigger List.....	19–4
RGB Data Formats	19–4
Data Clipping.....	19–4
Data Mirroring.....	19–5
Windowing.....	19–6
Preamble, Blanking and Stripping Support.....	19–6
EPPI Definitions	19–7
EPPI Block Diagram	19–8
EPPI Architectural Concepts	19–8
Reset Operation	19–8
Frame Sync Polarity and Sampling Edge.....	19–9
Direct Memory Access (DMA)	19–9
EPPI Clock	19–10
EPPI Operating Modes.....	19–11
ITU-R 656 Modes.....	19–11
ITU-R 656 Background	19–11
ITU-R 656 Input Modes.....	19–15
Entire Field	19–15
Active Video.....	19–15
Vertical Blanking Interval (VBI).....	19–15
ITU-R 656 Output in General-Purpose Transmit Modes.....	19–16
Frame Synchronization in ITU-R 656 Modes.....	19–17
General-Purpose EPPI Modes	19–18

General-Purpose 0 Frame Sync Mode.....	19–18
General-Purpose 1 Frame Sync Mode.....	19–18
General-Purpose 2 Frame Sync Mode.....	19–19
Data Enable in General-Purpose 2 Frame Sync Transmit Mode	19–19
General-Purpose 3 Frame Sync Mode.....	19–19
Supported Data Formats	19–20
Receive Data Formats	19–20
Transmit Data Formats	19–22
Data Transfer Modes	19–23
Data Packing for Receive Modes	19–23
Data Packing for Transmit Modes	19–24
Sign-Extended and Zero-Filled Data	19–24
Split Receive Modes	19–24
Split Transmit Modes	19–25
Clock Gating.....	19–25
Support for Delayed Start of EPPI Frame Syncs	19–25
Ignoring Premature External Frame Syncs for Data Consistency	19–26
EPPI Event Control	19–26
EPPI Status, Error, and Interrupt Signals	19–27
Frame and Line Track Errors	19–27
Line Track Errors	19–27
Frame Track Errors.....	19–27
Preamble Error Not Corrected Error	19–28
EPPI Programming Model	19–28
Receiving ITU-R 656 Frames	19–28
Transmitting ITU-R 656 Frames in GP Transmit Modes	19–28
Configuring Transfers in GP 0 FS Mode	19–29
Configuring Transfers in GP 1 FS Mode	19–29
Configuring Transfers in GP 2 FS Mode	19–30
Configuring Transfers in GP 3 FS Mode	19–31
Configuring the EPPI to Use the Windowing Feature	19–31

EPPI Mode Configuration.....	19-32
Configuring 8-Bit Receive Mode	19-32
Configuring 10/12/14-Bit Receive Modes	19-33
Configuring 16-Bit Receive Mode	19-35
Configuring 18-Bit Receive Mode	19-36
Configuring 8-Bit Split Receive Mode.....	19-37
Configuring 10/12/14/16-Bit Split Receive Mode with SPLTWDR=0	19-40
Configuring 16-Bit Split Receive Mode with SPLTWDR=1.....	19-41
Configuring 8-Bit Transmit Mode.....	19-42
Configuring 10/12/14-Bit Transmit Modes	19-43
Configuring 16-Bit Transmit Mode.....	19-43
Configuring 18-Bit Transmit Mode.....	19-44
Configuring 8-Bit Split Transmit Mode	19-44
Configuring 10/12/14/16-Bit Transmit Mode with SPLTWDR=0	19-47
Configuring 16-Bit Split Transmit Mode with SPLTWDR=1	19-49
EPPI Programming Concepts.....	19-50
ADSP-2159x EPPI Register Descriptions	19-50
Clock Divide Register	19-52
Control Register	19-53
Control Register 2 Register	19-61
Clipping Register for EVEN (Luma) Data Register	19-62
Lines Per Frame Register	19-63
Frame Sync 1 Delay Value Register	19-64
FS1 Period Register / EPPI Active Samples Per Line Register	19-65
FS1 Width Register / EPPI Horizontal Blanking Samples Per Line Register	19-66
Frame Sync 2 Delay Value Register	19-67
FS2 Period Register / EPPI Active Lines Per Field Register	19-68
FS2 Width Register / EPPI Lines Of Vertical Blanking Register	19-69
Horizontal Transfer Count Register	19-70
Horizontal Delay Count Register	19-71
Interrupt Mask Register	19-72

Samples Per Line Register	19-74
Clipping Register for ODD (Chroma) Data Register	19-75
Status Register	19-76
Vertical Transfer Count Register	19-79
Vertical Delay Count Register	19-80

General-Purpose Timer (TIMER)

GP Timer Features.....	20-1
ADSP-2159x TIMER Register List	20-2
ADSP-2159x TIMER Interrupt List	20-2
ADSP-2159x TIMER Trigger List.....	20-3
Timer Block Diagram.....	20-5
Internal Interface	20-5
Internal Timer Connections	20-6
External Interface	20-6
GP Timer Operating Modes	20-6
General Operation.....	20-7
Period, Width and Delay Register Interaction	20-7
Single-Pulse PWMOUT Mode	20-8
Continuous PWMOUT Mode	20-9
Width Capture (WIDCAP) Mode	20-10
Width Capture Mode Overflow.....	20-13
Windowed Watchdog (WATCHDOG) Modes	20-15
Windowed Watchdog Width Mode	20-15
Windowed Watchdog Period Mode.....	20-17
Pin Interrupt (PININT) Mode.....	20-19
External Clock (EXTCLK) Mode	20-19
GP Timer Programming Concepts	20-20
Setting Up Constantly Changing Timer Conditions	20-20
Configuring, Enabling, and Disabling One or More Timers.....	20-21
Configuring Timer Data and Status Interrupts	20-21

Configuring the Timer as a Trigger Slave.....	20–21
Ordered Trigger Toggle Mode	20–22
Using the Timer Broadcast Feature.....	20–22
Timer Illegal States	20–23
Continuous PWMOUT Mode	20–23
Single Pulse PWMOUT Mode.....	20–24
WIDCAP Mode	20–25
EXTCLK Mode	20–25
WATCHDOG Events.....	20–26
ADSP-2159x TIMER Register Descriptions	20–27
Broadcast Delay Register	20–28
Broadcast Period Register	20–29
Broadcast Width Register	20–30
Data Interrupt Latch Register	20–31
Data Interrupt Mask Register	20–32
Error Type Status Register	20–33
Run Register	20–37
Run Clear Register	20–38
Run Set Register	20–39
Status Interrupt Latch Register	20–40
Status Interrupt Mask Register	20–41
Stop Configuration Register	20–42
Stop Configuration Clear Register	20–43
Stop Configuration Set Register	20–44
Timer n Configuration Register	20–45
Timer n Counter Register	20–50
Timer n Delay Register	20–51
Timer n Period Register	20–52
Timer n Width Register	20–53
Trigger Slave Enable Register	20–54

Trigger Master Mask Register	20–55
General-Purpose Counter (CNT)	
GP Counter Features	21–1
GP Counter Functional Description	21–1
ADSP-2159x CNT Register List	21–2
ADSP-2159x CNT Interrupt List	21–2
ADSP-2159x CNT Trigger List.....	21–3
GP Counter Operating Modes.....	21–3
Quadrature Encoder Mode	21–3
Binary Encoder Mode.....	21–4
Up/Down Counter Mode	21–4
Direction Counter Mode	21–5
Timed Direction Mode.....	21–5
GP Counter Programming Model	21–5
GP Counter General Programming Flow	21–5
GP Counter Mode Configuration.....	21–5
Configuring GP Counter Push-Button Operation	21–6
Configuring Zero-Marker-Zeros-Counter Mode	21–6
Configuring Zero-Marker-Error Mode	21–6
Configuring Zero-Once Mode.....	21–6
Configuring Boundary Auto-Extend Mode	21–7
Configuring Boundary Capture Mode.....	21–7
Configuring Boundary Compare and Boundary Zero Modes	21–7
Configuring GP Counter Push-Button Operation	21–8
GP Counter Programming Concepts.....	21–8
CNT Input Noise Filtering	21–8
Capturing Counter Interval and CNT_CNTR Read Timing	21–9
Capturing Time Interval Between Successive Counter Events	21–10
GP Counter Event Control	21–11
Illegal Gray and Binary Code Events	21–11

Up/Down Count Events	21-11
Zero-Count Events	21-12
Overflow Events	21-12
Boundary Match Events	21-12
Zero Marker Events	21-12
ADSP-2159x CNT Register Descriptions	21-12
Configuration Register	21-14
Command Register	21-17
Counter Register	21-20
Debounce Register	21-21
Interrupt Mask Register	21-23
Maximum Count Register	21-26
Minimum Count Register	21-27
Status Register	21-28
 Media Local Bus (MLB)	
Features.....	22-1
MLB Definitions	22-2
Clocking	22-3
Functional Description	22-3
ADSP-2159x MLB Register List.....	22-4
ADSP-2159x MLB Interrupt List	22-5
MediaLB Protocol	22-5
MLB Architectural Concepts	22-6
MediaLB Block Diagram	22-6
MediaLB Interface	22-7
Routing Fabric.....	22-8
Data Buffer RAM.....	22-8
Channel Table RAM	22-8
Address Mapping	22-8
Channel Allocation Table.....	22-9

Channel Set Up.....	22–10
Channel Descriptor Tables.....	22–11
AHB Descriptor Table (ADT).....	22–15
Interrupt Interface Block.....	22–19
Operating Modes.....	22–20
Isochronous Data Exchange.....	22–20
Asynchronous and Control Data Exchange.....	22–21
Synchronous Data Exchange.....	22–22
Data Transfer.....	22–22
DMA.....	22–23
Programming Model.....	22–23
Channel Initialization.....	22–23
Configure the Hardware.....	22–23
Program the CAT and the CDT.....	22–23
Program the ADT.....	22–24
Service.....	22–25
Servicing the DMA Channel Interrupts.....	22–26
Servicing the MediaLB Status Interrupts.....	22–26
Polling for MediaLB System Commands.....	22–27
ADSP-2159x MLB Register Descriptions	22–27
Peripheral Channel Mask 0 Register	22–29
Peripheral Channel Mask 1 Register	22–30
Peripheral Channel Status 0 Register	22–31
Peripheral Channel Status 1 Register	22–32
Bus Control Register	22–33
MediaLB Control 0 Register	22–35
Control 1 Register	22–37
MLB Global Control Register	22–38
HBI Channel Busy 0 Register	22–39
HBI Channel Busy 1 Register	22–40

HBi Channel Error 0 Register	22-41
HBi Channel Error 1 Register	22-42
HBi Channel Mask 0 Register	22-43
HBi Channel Mask 1 Register	22-44
HBi Control Register	22-45
Memory Interface Address Register	22-46
Memory Interface Control Register	22-47
Memory Interface Control Data 0 Register	22-48
Memory Interface Control Data 1 Register	22-49
Memory Interface Control Data 2 Register	22-50
Memory Interface Control Data 3 Register	22-51
Memory Interface Control Data Write Enable 0 Register	22-52
Memory Interface Control Data Write Enable 1 Register	22-53
Memory Interface Control Data Write Enable 2 Register	22-54
Memory Interface Control Data Write Enable 3 Register	22-55
Interrupt Enable Register	22-56
Channel Status 0 Register	22-59
Channel Status 1 Register	22-60
System Data Register	22-61
System Status Register	22-62
MediaLB 6-pin Control 0 Register	22-64

Two-Wire Interface (TWI)

TWI Features.....	23-1
TWI Functional Description	23-2
ADSP-2159x TWI Register List.....	23-2
ADSP-2159x TWI Interrupt List	23-3
TWI Block Diagram.....	23-3
External Interface	23-3
Serial Clock Signal (SCL)	23-4
Serial Data Signal (SDA).....	23-4

Internal Interface.....	23-5
TWI Architectural Concepts	23-5
TWI Protocol.....	23-5
Clock Generation and Synchronization	23-6
Bus Arbitration	23-6
Start and Stop Conditions	23-7
General Call Support.....	23-7
Fast Mode	23-8
TWI Operating Modes	23-8
Repeated Start	23-8
Transmit Receive Repeated Start.....	23-8
Receive Transmit Repeated Start.....	23-9
Clock Stretching	23-10
Clock Stretching During FIFO Underflow	23-10
Clock Stretching During FIFO Overflow	23-11
Clock Stretching During Repeated Start.....	23-11
TWI Programming Model.....	23-12
General Setup	23-12
Slave Mode	23-13
Master Mode Program Flow	23-14
Master Mode Clock Setup	23-15
Master Mode Transmit	23-15
Master Mode Receive.....	23-16
ADSP-2159x TWI Register Descriptions	23-17
SCL Clock Divider Register	23-18
Control Register	23-19
FIFO Control Register	23-21
FIFO Status Register	23-23
Interrupt Mask Register	23-24
Interrupt Status Register	23-26

Master Mode Address Register	23–29
Master Mode Control Registers	23–30
Master Mode Status Register	23–33
Rx Data Double-Byte Register	23–36
Rx Data Single-Byte Register	23–37
Slave Mode Address Register	23–38
Slave Mode Control Register	23–39
Slave Mode Status Register	23–41
Tx Data Double-Byte Register	23–42
Tx Data Single-Byte Register	23–43

Ethernet Media Access Controller (EMAC)

EMAC Features.....	24–1
EMAC Functional Description	24–3
ADSP-2159x EMAC Register List.....	24–4
ADSP-2159x EMAC Interrupt List	24–10
ADSP-2159x EMAC Trigger List	24–11
EMAC Definitions	24–11
EMAC Block Diagram and Interfaces.....	24–12
EMAC CORE Subblocks	24–14
EMAC PHY Interface	24–16
RGMII Board Design Recommendations.....	24–18
Clock Sources	24–20
EMAC Architectural Concepts	24–21
EMAC Feature Summary	24–21
EMAC System Crossbar Interface (EMAC SCB).....	24–22
Priority of SCB Requests	24–23
SCB Interface Programming Options	24–23
DMA Bursts Using the SCB Interface	24–25
SCB Bus Transaction Status	24–26
Fatal Bus Error	24–26

DMA Controller (EMAC DMA)	24-26
DMA Related Registers	24-27
DMA Descriptors	24-28
OWN Bit (Ownership) Semaphore	24-45
Application Data Buffer Alignment.....	24-46
Buffer Size Calculations	24-46
EMAC FIFO Layer (EMAC MFL)	24-46
FIFO Layer Transmit Path	24-47
FIFO Layer Receive Path.....	24-48
EMAC CORE	24-49
EMAC CORE Transmission Engine	24-51
Source Address, VLAN, and CRC Insertion, Replacement, or Deletion	24-55
EMAC CORE Reception Engine	24-57
EMAC Station Management Interface (SMI)	24-67
MDC Clock Frequency	24-68
SMI Write Operation	24-69
SMI Read Operation.....	24-70
EMAC Management Counters (MMC)	24-70
MMC Receive Interrupt Register	24-72
MMC Transmit Interrupt Register	24-72
MMC Receive Checksum Offload Interrupt Register	24-72
EMAC Precision Time Protocol (PTP) Engine	24-72
IEEE1588 and the PTP Engine	24-72
Block Diagram	24-76
PTP Module Clock.....	24-77
Time Stamp Module	24-79
System Time	24-86
Target Time Trigger (Alarm)	24-89
Pulse-Per-Second (PPS).....	24-89
PTP Interrupts.....	24-92
Audio Video Data Transmission	24-92
Transmit Path Functions	24-93

Receive Path Functions	24-94
DMA Arbiter	24-95
Slot Number Function	24-96
Interrupts.....	24-96
Credit-Based Shaper Algorithm Functions	24-96
Energy-Efficient Ethernet.....	24-98
Transmit Path Functions.....	24-98
LPI Timers.....	24-99
EMAC Event Control	24-100
EMAC Interrupt Signals.....	24-101
PHYINT Interrupt Signal	24-102
EMAC Programming Model.....	24-102
EMAC Programming Steps	24-102
DMA Initialization.....	24-102
EMAC CORE Initialization	24-104
Performing Normal Transmit and Receive Operations	24-104
Stopping and Starting Transfers	24-105
Interrupts and Interrupt Service Routines	24-105
Enabling Checksum for Transmit and Receive	24-106
Programming the System Time Module.....	24-107
Programming the PTP for Frame Detection and Time Stamping.....	24-108
Programming for Auxiliary Time Stamps	24-108
Programming Fixed Pulse-Per-Second Output	24-109
Programming Flexible Pulse-Per-Second Output.....	24-109
EMAC Programming Concepts	24-110
IEEE 802.3 Ethernet Packet Structure.....	24-110
Frame Size Statistics for Application Software.....	24-111
Software Visualization of Programmable Packet Size	24-111
Ethernet Packet Structure in C	24-111
DMA Descriptor Implementation in C	24-112
PTP Header Structure in C	24-112

ADSP-2159x EMAC Register Descriptions	24-113
MAC Address 0 High Register	24-120
MAC Address 0 Low Register	24-121
MAC Address 1 High Register	24-122
MAC Address 1 Low Register	24-123
Debug Register	24-124
DMA SCB Bus Mode Register	24-127
DMA SCB Status Register	24-129
DMA Bus Mode Register	24-130
DMA Interrupt Enable Register	24-133
DMA Missed Frame Register	24-136
DMA Operation Mode Register	24-137
DMA Rx Buffer Current Register	24-141
DMA Rx Descriptor List Address Register	24-142
DMA Rx Descriptor Current Register	24-143
DMA Rx Interrupt Watch Dog Register	24-144
DMA Rx Poll Demand register	24-145
DMA Status Register	24-146
DMA Tx Buffer Current Register	24-151
DMA Tx Descriptor List Address Register	24-152
DMA Tx Descriptor Current Register	24-153
DMA Tx Poll Demand Register	24-154
DMA Bus Mode Register	24-155
Channel 1 Credit Shaping Control Register	24-158
Channel 1 Average Traffic Transmitted Register	24-159
Channel 1 High Credit Value Register	24-160
Channel 1 Idle Slope Credit Value Register	24-161
Channel 1 Low Credit Value Register	24-162
Channel 1 Control Bits for Slot Function Register	24-163
Channel 1 Send Slope Credit Value Register	24-164

DMA Interrupt Enable Register	24–165
DMA Missed Frame Register	24–168
DMA Operation Mode Register	24–169
DMA Rx Buffer Current Register	24–173
DMA Rx Descriptor List Address Register	24–174
DMA Rx Descriptor Current Register	24–175
DMA Rx Interrupt Watch Dog Register	24–176
DMA Rx Poll Demand Register	24–177
DMA Status Register	24–178
DMA Tx Buffer Current Register	24–183
DMA Tx Descriptor List Address Register	24–184
DMA Tx Descriptor Current Register	24–185
DMA Tx Poll Demand Register	24–186
DMA Bus Mode Register	24–187
Channel 2 Credit Shaping Control Register	24–190
Channel 2 Avg Traffic Transmitted Status Register	24–191
Channel 2 High Credit Value Register	24–192
Channel 2 Idle Slope Credit Value Register	24–193
Channel 2 Low Credit Value Register	24–194
Channel 2 Control Bits for Slot Function Register	24–195
Channel 2 Send Slope Credit Value Register	24–196
DMA Interrupt Enable Register	24–197
DMA Missed Frame Register	24–200
DMA Operation Mode Register	24–201
DMA Rx Buffer Current Register	24–205
DMA Rx Descriptor List Address Register	24–206
DMA Rx Descriptor Current Register	24–207
DMA Rx Interrupt Watch Dog Register	24–208
DMA Rx Poll Demand register	24–209
DMA Status Register	24–210

DMA Tx Buffer Current Register	24–215
DMA Tx Descriptor List Address Register	24–216
DMA Tx Descriptor Current Register	24–217
DMA Tx Poll Demand Register	24–218
Flow Control Register	24–219
RGMII Control and Status Register	24–221
Hash Table High Register	24–222
Hash Table Low Register	24–223
Interrupt Mask Register	24–224
MMC IPC Rx Interrupt Mask Register	24–225
MMC IPC Rx Interrupt Register	24–231
Interrupt Status Register	24–236
Layer3 and Layer4 Control Register	24–238
Layer 3 Address0 Register	24–240
Layer 3 Address1 Register	24–241
Layer 3 Address2 Register	24–242
Layer 3 Address3 Register	24–243
Layer 4 Address Register	24–244
Low Power Idle Control and Status Register	24–245
Low Power Idle Timeout Register	24–247
MAC Configuration Register	24–248
MAC Rx Frame Filter Register	24–253
AV MAC Control Register	24–256
MMC Control Register	24–258
MMC Rx Interrupt Mask Register	24–260
MMC Rx Interrupt Register	24–264
MMC TX Interrupt Mask Register	24–268
MMC Tx Interrupt Register	24–272
Rx 1024- to Max-Byte Frames (Good/Bad) Register	24–276
Rx 128- to 255-Byte Frames (Good/Bad) Register	24–277

Rx 256- to 511-Byte Frames (Good/Bad) Register	24–278
Rx 512- to 1023-Byte Frames (Good/Bad) Register	24–279
Rx 64-Byte Frames (Good/Bad) Register	24–280
Rx 65- to 127-Byte Frames (Good/Bad) Register	24–281
Rx alignment Error Register	24–282
Rx Broadcast Frames (Good) Register	24–283
Rx CRC Error Register	24–284
Rx Good Control Frames Register	24–285
Rx FIFO Overflow Register	24–286
Rx Frame Count (Good/Bad) Register	24–287
Rx ICMP Error Frames Register	24–288
Rx ICMP Error Octets Register	24–289
Rx ICMP Good Frames Register	24–290
Rx ICMP Good Octets Register	24–291
Rx IPv4 Datagrams Fragmented Frames Register	24–292
Rx IPv4 Datagrams Fragmented Octets Register	24–293
Rx IPv4 Datagrams (Good) Register	24–294
Rx IPv4 Datagrams Good Octets Register	24–295
Rx IPv4 Datagrams Header Errors Register	24–296
Rx IPv4 Datagrams Header Errors Register	24–297
Rx IPv4 Datagrams No Payload Frame Register	24–298
Rx IPv4 Datagrams No Payload Octets Register	24–299
Rx IPv4 UDP Disabled Frames Register	24–300
Rx IPv4 UDP Disabled Octets Register	24–301
Rx IPv6 Datagrams Good Frames Register	24–302
Rx IPv6 Good Octets Register	24–303
Rx IPv6 Datagrams Header Error Frames Register	24–304
Rx IPv6 Header Errors Register	24–305
Rx IPv6 Datagrams No Payload Frames Register	24–306
Rx IPv6 No Payload Octets Register	24–307

Rx Jab Error Register	24–308
Rx Length Error Register	24–309
Rx Multicast Frames (Good) Register	24–310
Rx Octet Count (Good) Register	24–311
Rx Octet Count (Good/Bad) Register	24–312
Rx Out Of Range Type Register	24–313
Rx Oversize (Good) Register	24–314
Rx Pause Frames Register	24–315
Rx Error Frames Received Register	24–316
Rx Runt Error Register	24–317
Rx TCP Error Frames Register	24–318
Rx TCP Error Octets Register	24–319
Rx TCP Good Frames Register	24–320
Rx TCP Good Octets Register	24–321
Rx Unicast Frames (Good) Register	24–322
Rx UDP Error Frames Register	24–323
Rx UDP Error Octets Register	24–324
Rx UDP Good Frames Register	24–325
Rx UDP Good Octets Register	24–326
Rx Undersize (Good) Register	24–327
Rx VLAN Frames (Good/Bad) Register	24–328
Rx Watch Dog Error Register	24–329
SMI Address Register	24–330
SMI Data Register	24–332
Time Stamp Addend Register	24–333
Time Stamp Auxiliary TS Nano Seconds Register	24–334
Time Stamp Auxiliary TM Seconds Register	24–335
Time Stamp Control Register	24–336
Time Stamp High Second Register	24–342
Time Stamp Nanoseconds Register	24–343

Time Stamp Nanoseconds Update Register	24–344
Time Stamp PPS Interval Register	24–345
Time Stamp Target Time Nanoseconds Register	24–346
Time Stamp Target Time Seconds Register	24–347
PPS Width Register	24–348
PPS 1 Interval Register	24–349
PPS 1 Target Time Nanoseconds Register	24–350
PPS 1 Target Time Seconds Register	24–351
PPS 1 Width Register	24–352
PPS 2 Interval Register	24–353
PPS 2 Target Time Nanoseconds Register	24–354
PPS 2 Target Time Seconds Register	24–355
PPS 2 Width Register	24–356
PPS 3 Interval Register	24–357
PPS 3 Target Time Nanoseconds Register	24–358
PPS 3 Target Time Seconds Register	24–359
PPS 3 Width Register	24–360
PPS Control Register	24–361
Time Stamp Low Seconds Register	24–364
Time Stamp Seconds Update Register	24–365
Time Stamp Status Register	24–366
Time Stamp Sub Second Increment Register	24–369
Tx 1024- to Max-Byte Frames (Good/Bad) Register	24–370
Tx 128- to 255-Byte Frames (Good/Bad) Register	24–371
Tx 256- to 511-Byte Frames (Good/Bad) Register	24–372
Tx 512- to 1023-Byte Frames (Good/Bad) Register	24–373
Tx 64-Byte Frames (Good/Bad) Register	24–374
Tx 65- to 127-Byte Frames (Good/Bad) Register	24–375
Tx Broadcast Frames (Good) Register	24–376
Tx Broadcast Frames (Good/Bad) Register	24–377

Tx Carrier Error Register	24–378
Tx Deferred Register	24–379
Tx Excess Collision Register	24–380
Tx Excess Deferral Register	24–381
Tx Frame Count (Good) Register	24–382
Tx Frame Count (Good/Bad) Register	24–383
Tx Late Collision Register	24–384
Tx Multicast Frames (Good) Register	24–385
Tx Multicast Frames (Good/Bad) Register	24–386
Tx Multiple Collision (Good) Register	24–387
Tx Octet Count (Good) Register	24–388
Tx OCT Count (Good/Bad) Register	24–389
Number of Tx Frames (Good) greater than maxsize	24–390
Tx Pause Frame Register	24–391
Tx Single Collision (Good) Register	24–392
Tx Unicast Frames (Good/Bad) Register	24–393
Tx Underflow Error Register	24–394
Tx VLAN Frames (Good) Register	24–395
VLAN Tag Register	24–396
VLAN Hash Table Register	24–398
VLAN Tag Inclusion or Replacement Register	24–399
Watchdog Timeout Register	24–400

Digital Audio Interface (DAI)

SRU Features	25–1
Functional Description	25–2
ADSP-2159x DAI Register List.....	25–2
ADSP-2159x DAI Interrupt List	25–5
ADSP-2159x DAI Trigger List	25–5
DAI Block Diagram.....	25–6

DAI Signal Naming Conventions	25-7
I/O Pin Buffers.....	25-7
Pin Buffer Signals.....	25-8
Pin Buffer Input Signal	25-8
Pin Buffer Enable Signal	25-8
Pin Buffer Functions	25-8
Pin Buffers as Signal Input.....	25-8
Pin Buffers As Signal Output.....	25-9
DAI Pin Buffer Status	25-9
DAIn Peripherals.....	25-10
Output Signals With Pin Buffer Enable Control.....	25-10
Output Signals Without Pin Buffer Enable Control.....	25-10
Signal Routing Units (SRUs)	25-10
Cross Mode Connections	25-10
Signal Routing Matrix by Groups.....	25-12
DAI Group Routing.....	25-13
Rules for SRU Connections.....	25-14
Miscellaneous Buffers and Functions.....	25-15
DAI Routing Unit (DRU)	25-15
DAI Routing Capabilities.....	25-16
DAI Default Routing.....	25-19
Unused DAI Connections.....	25-20
DAI Operating Modes	25-20
DAI Pin Buffer Polarity.....	25-21
DAI Miscellaneous Buffer Polarity	25-21
DAI System Interrupt Controller (SIC)	25-21
Signal Routing Unit Effect Latency.....	25-24
DAI Programming Model.....	25-24
Debug Features	25-24
DAI Sources Overview.....	25-24
DAI0 Group A – Clock Routing Signals.....	25-25

DAI0 Group B – Serial Data Source Signals	25–29
DAI0 Group C – Frame Sync Source Signals	25–30
DAI0 Group D – Pin Signal Assignment Source Signals	25–31
DAI0 Group E – Miscellaneous Source Signals	25–34
DAI0 Group F – Pin Output Enable Source Signals	25–37
DAI1 Group A – Clock Routing Source Signals	25–38
DAI1 Group B – Serial Data Source Signals	25–39
DAI1 Group C – Frame Sync Source Signals	25–41
DAI1 Group D – Pin Signal Assignment Source Signals	25–42
DAI1 Group E – Miscellaneous Source Signals	25–45
DAI1 Group F – Pin Output Enable Source Signals	25–46
DAI Destination Registers Overview	25–48
SPORT Grouping with DAI.....	25–55
Global PCG Enable	25–55
ADSP-2159x DAI Register Descriptions	25–56
Clock Routing Control Register 0	25–59
Clock Routing Control Register 1	25–61
Clock Routing Control Register 2	25–63
Clock Routing Control Register 3	25–64
Clock Routing Control Register 4	25–65
Clock Routing Control Register 5	25–66
Serial Data Routing Control Register 0	25–68
Serial Data Routing Control Register 1	25–69
Serial Data Routing Control Register 2	25–70
Serial Data Routing Control Register 3	25–71
Serial Data Routing Control Register 4	25–72
Serial Data Routing Control Register 5	25–73
Serial Data Routing Control Register 6	25–74
Extended Clock Routing Control Register 0	25–75
Extended Clock Routing Control Register 1	25–77

Extended Clock Routing Control Register 2	25–79
Extended Clock Routing Control Register 3	25–80
Extended Clock Routing Control Register 4	25–81
Extended Clock Routing Control Register 5	25–83
Extended Serial Data Routing Control Register 0	25–85
Extended Serial Data Routing Control Register 1	25–86
Extended Serial Data Routing Control Register 2	25–87
Extended Serial Data Routing Control Register 3	25–88
Extended Serial Data Routing Control Register 4	25–89
Extended Serial Data Routing Control Register 5	25–90
Extended Serial Data Routing Control Register 6	25–91
Extended Frame Sync Routing Control Register 0	25–92
Extended Frame Sync Routing Control Register 1	25–94
Extended Frame Sync Routing Control Register 2	25–96
Extended Frame Sync Routing Control Register 4	25–97
Extended Miscellaneous Control Register 0	25–98
Extended Miscellaneous Control Register 1	25–100
Extended Miscellaneous Control Register 2	25–101
Extended Pin Buffer Enable Register 0	25–102
Extended Pin Buffer Enable Register 1	25–103
Extended Pin Buffer Enable Register 2	25–104
Extended Pin Buffer Enable Register 3	25–105
Extended Pin Buffer Assignment Register 0	25–106
Extended Pin Buffer Assignment Register 1	25–107
Extended Pin Buffer Assignment Register 2	25–108
Extended Pin Buffer Assignment Register 3	25–109
Extended Pin Buffer Assignment Register 4	25–110
Frame Sync Routing Control Register 0	25–111
Frame Sync Routing Control Register 1	25–113
Frame Sync Routing Control Register 2	25–115

Frame Sync Routing Control Register 4	25–116
Global SPORT Interrupt Grouping Register	25–117
Global PCG Enable Control Register	25–120
Global SPORT Enable Register	25–123
Falling-Edge Interrupt Mask Register	25–126
Core Interrupt Priority Assignment Register	25–129
Rising-Edge Interrupt Mask Register	25–132
High Priority Interrupt Latch Register	25–135
Shadow High Priority Interrupt Latch Register	25–138
Low Priority Interrupt Latch Register	25–142
Shadow Low Priority Interrupt Latch Register	25–145
Miscellaneous Control Register 0	25–149
Miscellaneous Control Register 1	25–151
Miscellaneous Control Register 1	25–153
Pin Buffer Enable Register 0	25–154
Pin Buffer Enable Register 1	25–155
Pin Buffer Enable Register 2	25–156
Pin Buffer Enable Register 3	25–157
Pin Buffer Assignment Register 0	25–158
Pin Buffer Assignment Register 1	25–159
Pin Buffer Assignment Register 2	25–160
Pin Buffer Assignment Register 3	25–161
Pin Buffer Assignment Register 4	25–162
Pin Status Register	25–164

Serial Port (SPORT)

Features.....	26–1
Signal Descriptions	26–3
SRU Programming	26–6
Functional Description	26–6

ADSP-2159x SPORT Register List	26-6
ADSP-2159x SPORT Interrupt List	26-7
ADSP-2159x SPORT Trigger List.....	26-9
ADSP-2159x SPORT DMA Channel List.....	26-10
Block Diagram.....	26-11
Architectural Concepts	26-12
Multiplexer Logic	26-13
Data Types and Companding	26-16
Companding as a Function.....	26-17
Transmit Path.....	26-17
Receive Path	26-19
Operating Modes and Options	26-19
Serial Word Length.....	26-21
Clock Sample and Drive Edges	26-21
Frame Sync Options	26-23
Data-Dependent versus Data-Independent Frame Syncs	26-23
Support for Edge-Detected and Level-Sensitive Frame Syncs.....	26-23
Early versus Late Frame Syncs	26-24
Framed versus Unframed Frame Syncs	26-25
Frame Sync Polarity.....	26-26
Premature Frame Sync Error Detection	26-26
Mode Selection	26-27
Standard DSP Serial Mode.....	26-27
Stereo Modes.....	26-28
I ² S Mode.....	26-29
Left-Justified Mode	26-30
Right-Justified Mode.....	26-31
Multichannel (TDM) Mode.....	26-33
Packed I ² S Mode.....	26-36
Gated Clock Mode	26-37
Data Transfers and Interrupts	26-38

Data Buffers	26–38
Data Buffer Status	26–40
Single-Word (Core) Transfers	26–40
DMA Transfers.....	26–41
Data Transfer Interrupt	26–42
Error Detection (Status) Interrupt.....	26–43
Grouping of SPORTs.....	26–44
SPORT Programming Model	26–46
Initializing Core-Driven (Non-MCM) Transfers.....	26–46
Initializing Multichannel Transfers	26–48
Using DMA for SPORT Transfers.....	26–49
Using Companding as a Function.....	26–49
Programming Global SPORT Groups	26–50
Disabling Global SPORT Groups.....	26–51
ADSP-2159x SPORT Register Descriptions	26–51
Half SPORT 'A' Multichannel 0-31 Select Register	26–53
Half SPORT 'B' Multichannel 0-31 Select Register	26–54
Half SPORT 'A' Multichannel 32-63 Select Register	26–55
Half SPORT 'B' Multichannel 32-63 Select Register	26–56
Half SPORT 'A' Multichannel 64-95 Select Register	26–57
Half SPORT 'B' Multichannel 64-95 Select Register	26–58
Half SPORT 'A' Multichannel 96-127 Select Register	26–59
Half SPORT 'B' Multichannel 96-127 Select Register	26–60
Half SPORT 'A' Control 2 Register	26–61
Half SPORT 'B' Control 2 Register	26–62
Half SPORT 'A' Control Register	26–63
Half SPORT 'B' Control Register	26–71
Half SPORT 'A' Divisor Register	26–80
Half SPORT 'B' Divisor Register	26–81
Half SPORT 'A' Error Register	26–82

Half SPORT 'B' Error Register	26–84
Half SPORT 'A' Multichannel Control Register	26–86
Half SPORT 'B' Multichannel Control Register	26–88
Half SPORT 'A' Multichannel Status Register	26–90
Half SPORT 'B' Multichannel Status Register	26–91
Half SPORT 'A' Rx Buffer (Primary) Register	26–92
Half SPORT 'B' Rx Buffer (Primary) Register	26–93
Half SPORT 'A' Rx Buffer (Secondary) Register	26–94
Half SPORT 'B' Rx Buffer (Secondary) Register	26–95
Half SPORT 'A' Tx Buffer (Primary) Register	26–96
Half SPORT 'B' Tx Buffer (Primary) Register	26–97
Half SPORT 'A' Tx Buffer (Secondary) Register	26–98
Half SPORT 'B' Tx Buffer (Secondary) Register	26–99

Precision Clock Generators (PCG)

Features.....	27–1
Functional Description	27–2
ADSP-2159x PCG Register List.....	27–2
Internal Interface	27–3
Serial Clock	27–3
Frame Sync	27–4
Frame Sync Output	27–4
Divider Mode Selection	27–4
Phase Shift	27–4
Pulse Width	27–5
Default Pulse Width.....	27–5
Input Clock Source Considerations	27–6
Timing Example for I ² S Mode	27–6
Cross Mode Connections	27–6
Operating Modes	27–6
Normal Mode.....	27–6

Bypass Mode	27-7
One-Shot Mode.....	27-7
Audio System Example	27-8
Hardware Trigger Control	27-10
Clock Configuration Examples	27-10
Global PCG Enable	27-11
PCG Event Control	27-11
External Event Trigger	27-11
External Event Trigger Delay	27-11
Programming Model.....	27-12
Frame Sync Phase Setting	27-12
External Event Trigger	27-12
Debug Features	27-13
ADSP-2159x PCG Register Descriptions	27-13
Precision Clock A Control 0 Register	27-15
Precision Clock A Control 1 Register	27-16
Precision Clock B Control 0 Register	27-17
Precision Clock B Control 1 Register	27-18
Precision Clock C Control 0 Register	27-19
Precision Clock C Control 1 Register	27-20
Precision Clock D Control 0 Register	27-21
Precision Clock D Control 1 Register	27-22
Precision Clock E Control 0 Register	27-23
Precision Clock E Control 1 Register	27-24
Precision Clock F Control 0 Register	27-25
Precision Clock F Control 1 Register	27-26
Precision Clock G Control 0 Register	27-27
Precision Clock G Control 1 Register	27-28
Precision Clock H Control 0 Register	27-29
Precision Clock H Control 1 Register	27-30

Precision Clock Pulse Width Control 1 Register	27–31
Precision Clock Pulse Width Control 2 Register	27–33
Precision Clock Pulse Width Control 3 Register	27–35
Precision Clock Pulse Width Control 4 Register	27–37
Precision Clock Frame Sync Synchronization 1 Register	27–39
Precision Clock Frame Sync Synchronization 2 Register	27–42
Precision Clock Frame Sync Synchronization 3 Register	27–45
Precision Clock Frame Sync Synchronization 4 Register	27–48

Asynchronous Sample Rate Converter (ASRC)

Features.....	28–1
Functional Description	28–2
ADSP-2159x ASRC Register List	28–2
ASRC Interrupt List	28–2
ASRC Block Diagram	28–2
SRU Programming	28–3
Clocking.....	28–3
I/O Ports	28–3
De-Emphasis Filter	28–4
Mute Control	28–4
SRC Core	28–4
RAM FIFO	28–4
Digital Servo Loop	28–5
FIR Filter	28–5
Sample Rate Sensing	28–5
Digital Filter Group Delay	28–5
Data Format	28–6
Operating Modes.....	28–6
TDM Input Mode.....	28–6
TDM Output Mode.....	28–7
Matched-Phase Mode	28–7

Bypass Mode	28–8
De-Emphasis Mode	28–8
Dithering Mode.....	28–8
Muting Modes.....	28–9
Soft Mute	28–9
Hard Mute	28–9
Auto Mute.....	28–9
Interrupts.....	28–10
Sources	28–10
SRC Mute Out.....	28–10
Masking.....	28–10
Service	28–10
Programming Model.....	28–10
Debug Features	28–10
ADSP-2159x ASRC Register Descriptions	28–11
Control Register for ASRC 0 and 1	28–12
Control Register for ASRC 2 and 3	28–17
Mute Register	28–22
Ratio Register for ASRC 0 and 1	28–23
Ratio Register for ASRC 2 and 3	28–25

Sony/Philips Digital Interface (S/PDIF)

Features.....	29–1
ADSP-2159x SPDIF Register List	29–1
SRU Programming	29–3
S/PDIF Interrupt List.....	29–3
Clocking	29–3
S/PDIF Transmitter.....	29–3
Functional Description	29–4
Input Data Formats.....	29–5
Operating Modes.....	29–6

Full Serial Mode	29–6
Standalone Mode.....	29–6
Data Output Mode	29–7
S/PDIF Receiver	29–7
Functional Description	29–8
Clock Recovery.....	29–8
TDM Clock Output.....	29–9
Output Data Format	29–9
Channel Status	29–9
Operating Modes.....	29–10
Compressed or Non-linear Audio Data	29–10
Emphasized Audio Data.....	29–10
Single-Channel Double-Frequency Mode.....	29–10
Clock Recovery Modes	29–10
Interrupts.....	29–11
Sources	29–11
Transmit Block Start	29–11
Receiver Status	29–11
Receiver Error.....	29–11
Masking.....	29–12
Service	29–12
Programming Model.....	29–12
Programming the Transmitter	29–12
Programming the Receiver.....	29–12
Interrupted Data Streams on the Receiver	29–13
Debug Features	29–13
Loopback Routing	29–13
ADSP-2159x SPDIF Register Descriptions	29–13
Receive Control	29–15
Receive Status Register	29–17
Receive Status A0 Register	29–20

Receive Status B0 Register	29–21
Receive Status A1 Register	29–22
Receive Status B1 Register	29–23
Transmit Control Register	29–24
Transmit Status A0 Register	29–27
Transmit Status A1 Register	29–28
Transmit Status A2 Register	29–29
Transmit Status A3 Register	29–30
Transmit Status A4 Register	29–31
Transmit Status A5 Register	29–32
Transmit Status B0 Register	29–33
Transmit Status B1 Register	29–34
Transmit Status B2 Register	29–35
Transmit Status B3 Register	29–36
Transmit Status B4 Register	29–37
Transmit Status B5 Register	29–38
Transmit User Buffer A0 Register	29–39
Transmit User Buffer A1 Register	29–40
Transmit User Buffer A2 Register	29–41
Transmit User Buffer A3 Register	29–42
Transmit User Buffer A4 Register	29–43
Transmit User Buffer A5 Register	29–44
Transmit User Buffer B0 Register	29–45
Transmit User Buffer B1 Register	29–46
Transmit User Buffer B2 Register	29–47
Transmit User Buffer B3 Register	29–48
Transmit User Buffer B4 Register	29–49
Transmit User Buffer B5 Register	29–50
User Bit Update Register	29–51

Direct Memory Access (DMA)

DMA Channel Features	30-1
DMA Channel Functional Description	30-3
ADSP-2159x DMA Register List.....	30-3
ADSP-2159x DMA Channel List	30-4
DMA Definitions	30-6
Block Diagram.....	30-7
Architectural Concepts	30-9
DMA Channel SCB Interface.....	30-9
SCB Interface Signals	30-9
SCB Burst Transfers	30-10
Data Address Alignment	30-10
Descriptor Set Address Alignment	30-11
Peripheral Control Commands.....	30-11
Idle Command.....	30-12
Request-Data Command.....	30-12
Request-Data Urgent Command.....	30-12
Peripheral-Control Command Restrictions	30-12
DMA Channel Peripheral DMA Bus.....	30-12
Memory DMA and Triggering	30-13
Medium Band Width DMA Channel MMR Access Bus	30-15
DMA Channel Operation Flow.....	30-15
Startup Flow	30-16
Refresh Flow	30-17
Work Unit Transition Flow	30-18
Transfer Termination and Shutdown Flow	30-21
DMA Channel Errors.....	30-22
Status and Debug Errors	30-22
DMA Configuration Register Errors	30-23
Illegal Register Write During Run.....	30-23
Address Alignment Error.....	30-24

Memory Access Error	30-24
Trigger Overrun Error	30-24
Bandwidth-Monitor Error.....	30-24
Control Interface Error	30-24
DMA Operating Modes.....	30-24
Register-Based Flow Modes	30-25
Stop Mode.....	30-25
Autobuffer Mode.....	30-25
Descriptor-Based Flow Modes	30-25
Descriptor-Array Mode	30-26
Descriptor-List Mode	30-27
Descriptor-On-Demand Modes.....	30-27
Data Transfer Modes	30-28
Two-Dimensional DMA.....	30-28
DMA Channel Event Control.....	30-29
Event Signals	30-30
Work Unit State Events	30-31
Peripheral Interrupt Request Events	30-31
Peripheral Data Request Events	30-31
DMA Channel Triggers	30-32
Issuing Triggers	30-32
Waiting For Triggers	30-32
DMA Channel Programming Model	30-33
Mode Configuration.....	30-34
Register-Based Linear-Buffer Stop Flow Mode	30-34
Register-Based Autobuffer Flow Mode	30-35
Descriptor-Array Flow Mode.....	30-36
Descriptor-List Flow Mode	30-37
Register-Based Memory-to-Memory Transfer in Stop Flow Mode.....	30-38
Programming Concepts	30-39
Synchronization of Software and DMA.....	30-40

Interrupt and Trigger Event-Based Synchronization.....	30–40
Register Polling Based Synchronization.....	30–40
Descriptor Queues	30–41
Queues Using Event Generation for Every Descriptor Set.....	30–41
Queues Using Minimal Events	30–42
ADSP-2159x DMA Register Descriptions	30–43
Start Address of Current Buffer Register	30–45
Current Address Register	30–46
Bandwidth Limit Count Register	30–47
Bandwidth Limit Count Current Register	30–48
Bandwidth Monitor Count Register	30–49
Bandwidth Monitor Count Current Register	30–50
Configuration Register	30–51
Current Descriptor Pointer Register	30–59
Pointer to Next Initial Descriptor Register	30–60
Previous Initial Descriptor Pointer Register	30–61
Status Register	30–62
Inner Loop Count Start Value Register	30–65
Current Count (1D) or Intra-row XCNT (2D) Register	30–66
Inner Loop Address Increment Register	30–67
Outer Loop Count Start Value (2D only) Register	30–68
Current Row Count (2D only) Register	30–69
Outer Loop Address Increment (2D only) Register	30–70
Extended Memory DMA (EMDMA)	
EMDMA Features	31–1
EMDMA Functional Description	31–1
ADSP-2159x EMDMA Register List.....	31–2
ADSP-2159x EMDMA Interrupt List	31–2
ADSP-2159x EMDMA Trigger List	31–3
DMA Addressing.....	31–3

DMA Burst Transfers	31-3
Transfer Control Block (TCB) Memory Storage	31-3
Chain Assignment	31-4
Starting Chain Loading	31-4
Buffered Chain Loading Register	31-4
TCB Storage	31-4
EMDMA Operating Modes	31-7
Standard DMA	31-8
Circular Buffered DMA	31-8
Chained DMA Mode	31-9
Data Direction On-the-Fly	31-9
Write Back Circular Index Pointer	31-10
Scatter/Gather DMA	31-10
Pre Modified Read/Write Index	31-12
Delay Line DMA	31-13
ADSP-2159x EMDMA Register Descriptions	31-15
External Base Address Register	31-17
Circular Buffer Length Register	31-18
Chain Pointer Register	31-19
Internal Count Register	31-20
External Count Register	31-21
External Memory DMA Control Register	31-22
Internal Index Register	31-26
External Index Register	31-27
Internal Modifier Register	31-28
External Modifier Register	31-29
Delay Line Tap Count Register	31-30
Tap List Pointer Register	31-31

Cyclic Redundancy Check (CRC)

CRC Features.....	32-1
CRC Functional Description	32-2
ADSP-2159x CRC Register List.....	32-2
ADSP-2159x CRC Interrupt List	32-3
CRC Definitions	32-3
CRC Block Diagram.....	32-4
Peripheral DMA Bus	32-5
MMR Access Bus.....	32-5
Mirror Block.....	32-6
Data FIFO	32-6
DMA Request Generator.....	32-6
CRC Engine	32-6
Compare Logic	32-6
CRC Architectural Concepts.....	32-6
Look-up Table	32-7
Data Mirroring.....	32-7
FIFO Status and Data Requests.....	32-8
CRC Operating Modes	32-9
Data Transfer Modes	32-9
Memory Scan Compute-and-Compare Mode.....	32-10
Memory Scan Data Verify	32-10
Memory Transfer Compute-and-Compare Mode	32-10
Memory Transfer Data Fill Mode.....	32-11
CRC Event Control	32-11
Interrupt Signals.....	32-11
CRC Programming Model.....	32-12
CRC Mode Configuration.....	32-12
Look-up Table Generation	32-12
Core Driven Memory Scan Compute-and-Compare Mode	32-13
DMA Driven Memory Scan Compute-and-Compare Mode.....	32-14

Core Driven Memory Scan Data Verify Mode	32–16
DMA Driven Memory Scan Data Verify Mode	32–17
Core Driven Memory Transfer Compute-and-Compare Mode	32–19
DMA Driven Memory Transfer Compute-and-Compare Mode	32–20
DMA Driven Memory Transfer Data Fill Mode	32–22
ADSP-2159x CRC Register Descriptions	32–23
Data Compare Register	32–25
Control Register	32–26
Data Word Count Register	32–29
Data Count Capture Register	32–30
Data Word Count Reload Register	32–31
Data FIFO Register	32–32
Fill Value Register	32–33
Interrupt Enable Register	32–34
Interrupt Enable Clear Register	32–35
Interrupt Enable Set Register	32–36
Polynomial Register	32–37
CRC Current Result Register	32–38
CRC Final Result Register	32–39
Status Register	32–40
 System Security	
Security Features	33–1
Security Functional Description	33–2
Security Mode Configuration	33–4
Status and Error Signals	33–4
 System Protection Unit (SPU)	
SPU Features	34–1
SPU Functional Description	34–1
ADSP-2159x SPU Register List	34–1

ADSP-2159x SPU Interrupt List	34-2
Peripheral Register Write Protection.....	34-2
SPU Block Diagram.....	34-5
SPU Architectural Concepts	34-5
SPU Event Control	34-5
SPU Programming Model	34-6
SPU Mode Configuration.....	34-7
Locking Write-Protect Registers	34-7
Protecting a Peripheral	34-7
Configuring Security Privileges of a Peripheral	34-8
ADSP-2159x SPU Register Descriptions	34-8
Control Register	34-9
Secure Check Register	34-10
Secure Control Register	34-11
Secure Core Registers	34-13
Secure Peripheral Register	34-14
Status Register	34-15
Write Protect Register n	34-16
Write-Protect, Secure Peripheral, and Secure Core Registers	34-16
ADSP-2159x Specific Information.....	34-23
 Security Packet Engine (PKTE)	
PKTE Features.....	35-1
PKTE Functional Description	35-1
ADSP-2159x PKTE Register List.....	35-1
ADSP-2159x PKTE Interrupt List	35-3
PKTE Definitions	35-4
Cipher Module	35-5
Hash Module.....	35-5
Pseudo-Random Number Generator	35-6

Packet Engine Processing Details.....	35–8
Crypto Padding.....	35–8
Pad Generation and Insertion	35–9
Pad Types.....	35–9
Pad Length.....	35–10
Pad Verification and Consumption	35–12
Crypto and Hash Algorithms	35–14
IV Processing.....	35–17
ARC4 Processing.....	35–18
Hash State Loading	35–19
Sequence Number Processing.....	35–19
Sequence Number Processing in Extended SSL/TLS.....	35–19
Sequence Number Processing in DTLS.....	35–20
PKTE Block Diagram.....	35–23
PKTE Architectural Concepts	35–24
Packet Engine.....	35–24
Input/Output FIFO Buffers	35–24
Parallel Operations	35–24
DMA Controller	35–24
Interrupt Controller	35–25
Clock Controller	35–25
PKTE Operating Modes.....	35–25
Autonomous Ring Mode (ARM)	35–26
Target Command Mode (TCM)	35–27
Direct Host Mode (DHM)	35–27
PKTE Event Control	35–27
PKTE Interrupt Signals.....	35–27
PKTE Programming Model.....	35–30
PKTE Mode Configuration.....	35–33
PKTE Programming Concepts	35–33
Packet Engine Descriptor	35–33

Descriptor Processing.....	35–34
Descriptor Ownership.....	35–36
SA Record and State Record Structure.....	35–36
SA Record Structure	35–37
SA State Structure	35–39
ARC4 State Structure	35–39
Configuring Operations in the PKTE	35–40
Basic Operations and Decoding	35–40
Error Code Description	35–41
Extended Error Codes	35–42
Number Format	35–44
PKTE Programming Examples	35–45
Calculating SHA in Direct Host Mode.....	35–45
Performing AES Decryption in Direct Host Mode	35–46
ADSP-2159x PKTE Register Descriptions	35–47
Packet Engine ARC4 State Record Address	35–50
Starting Entry of 256-byte ARC4 State Buffer	35–51
Packet Engine Buffer Pointer Register	35–52
Packet Engine Buffer Threshold Register	35–53
Packet Engine Command Descriptor Ring Base Address	35–55
Packet Engine Command Descriptor Count Register	35–56
Packet Engine Command Descriptor Count Increment Register	35–57
Packet Engine Configuration Register	35–58
PE Clock Control Register	35–61
PKTE Continue Register	35–63
Packet Engine Control Register	35–64
Starting Entry of 256-byte Data Input/Output Buffer	35–69
Packet Engine Destination Address	35–70
Packet Engine DMA Configuration Register	35–71
Packet Engine Endian Configuration Register	35–73

Packet Engine Halt Control Register	35-75
Packet Engine Halt Status Register	35-77
Interrupt Mask Disable Register	35-80
Interrupt Mask Enable Register	35-82
Interrupt Masked Status Register	35-84
Packet Engine Input Buffer Count Register	35-86
Packet Engine Input Buffer Count Increment Register	35-87
Interrupt Configuration Register	35-88
Interrupt Clear Register	35-89
Interrupt Enable Register	35-91
Interrupt Unmasked Status Register	35-93
Packet Engine Length Register	35-95
Packet Engine Output Buffer Count Register	35-97
Packet Engine Output Buffer Count Decrement Register	35-98
Packet Engine Result Descriptor Ring Base Address	35-99
Packet Engine Result Descriptor Count Registers	35-100
Packet Engine Result Descriptor Count Decrement Registers	35-101
Packet Engine Ring Configuration	35-102
Packet Engine Ring Pointer Status	35-103
Packet Engine Ring Status	35-104
Packet Engine Ring Threshold Registers	35-105
Packet Engine SA Address	35-107
ARC4 i and j Pointer Register	35-108
SA Command 0	35-109
SA Command 1	35-114
SA Inner Hash Digest Registers	35-118
SA Key Registers	35-119
SA Initialization Vector Register	35-120
SA Outer Hash Digest Registers	35-121
SA Ready Indicator	35-122

SA Sequence Number Register	35–123
SA Sequence Number Mask Registers	35–124
SA SPI Register	35–125
Packet Engine Source Address	35–126
Packet Engine Status Register	35–127
Packet Engine State Record Address	35–131
State Hash Byte Count Registers	35–132
State Inner Digest Registers	35–133
State Initialization Vector Registers	35–134
Packet Engine User ID	35–135

Public Key Accelerator (PKA)

PKA Features	36–1
PKA Functional Description	36–1
ADSP-2159x PKA Register List	36–2
PKA Definitions	36–2
PKA Architectural Concepts	36–3
PKA Block Diagram	36–3
PKCP Vector Operations	36–4
Modular Exponentiation Operations	36–6
Modular Inversion	36–13
Modular Inversion with an Even Modulus	36–14
Modular Inversion with a Prime Modulus	36–14
ECC Operations	36–14
ADSP-2159x PKA Register Descriptions	36–20
PKA Vector_A Length	36–21
PKA Vector_A Address	36–22
PKA Vector_B Length	36–23
PKA Vector_B Address	36–24
PKA Compare Result	36–25

PKA Vector_C Address	36–26
PKA Most-Significant-Word of Divide Remainder	36–27
PKA Vector_D Address	36–28
PKA Function	36–29
Start of PKA RAM space	36–32
PKA Most-Significant-Word of Result Vector	36–33
PKA Bit Shift Value	36–34

Public Key Interrupt Controller (PKIC)

PKIC Functional Description	37–1
ADSP-2159x PKIC Register List.....	37–1
ADSP-2159x PKIC Interrupt List	37–2
PKIC Programming Model.....	37–2
PKIC Programming Concepts	37–2
ADSP-2159x PKIC Register Descriptions	37–3
Acknowledge Register	37–4
Enable Clear Register	37–5
Enable Control Register	37–6
Enable Set Register	37–7
Enabled Status Register	37–8
Polarity Control Register	37–9
Raw Status Register	37–10
Type Control Register	37–11

True Random Number Generator (TRNG)

TRNG Features	38–1
TRNG Functional Description	38–1
ADSP-2159x TRNG Register List.....	38–1
Random Number Generation	38–2
Locking Detection and Prevention.....	38–3
Run Testing.....	38–4

Monobit Testing.....	38-4
Poker Testing.....	38-5
Data for Tests	38-6
X9.31 Postprocessing.....	38-6
TRNG Block Diagram	38-6
TRNG Architectural Concepts.....	38-7
TRNG Operating Modes.....	38-8
TRNG Data Transfer Modes	38-9
TRNG Event Control.....	38-9
TRNG Interrupt Signals.....	38-9
ADSP-2159x TRNG Register Descriptions	38-10
TRNG Alarm Counter Register	38-12
TRNG Alarm Mask Register	38-14
TRNG Alarm Stop Register	38-15
TRNG Block Count Register	38-16
TRNG Configuration Register	38-17
Counter Register	38-19
TRNG Control Register	38-20
TRNG FRO De-tune Register	38-23
TRNG FRO Enable Register	38-24
TRNG Input Registers	38-25
TRNG Interrupt Acknowledge Register	38-26
Post-Process Key Registers	38-28
TRNG LFSR Access Register	38-29
TRNG LFSR Access Register	38-30
TRNG LFSR Access Register	38-31
TRNG Monobit Test Result Register	38-32
TRNG Output Registers	38-33
TRNG Poker Test Result Registers	38-34
TRNG Run Count Registers	38-35

TRNG Run Test State and Result Registers	38–36
TRNG Status Register	38–38
TRNG Test Register	38–40
TRNG Post-Process "V" Value Registers	38–43

FIR Accelerator (FIR)

Features.....	39–1
Clocking	39–1
Functional Description	39–2
ADSP-2159x FIR Register List.....	39–3
ADSP-2159x FIR Interrupt List	39–3
ADSP-2159x FIR Trigger List	39–4
Compute Block	39–4
Partial Sum Register	39–5
Delay Line Memory.....	39–5
Coefficient Memory	39–5
Prefetch Data Buffer.....	39–5
Processing Output.....	39–6
System Memory Storage	39–7
Coefficients and Input Buffer Storage	39–7
Single Rate Input Filtering	39–7
Decimation	39–7
Interpolation	39–8
Operating Modes.....	39–8
Single Rate Processing.....	39–8
Single Iteration.....	39–8
Floating-point Multi-Iteration.....	39–8
Window Processing	39–9
Multi-Rate Processing	39–9
Decimation.....	39–9
Interpolation	39–9

Floating-Point Data Format.....	39–10
Fixed-Point Data Format	39–10
Auto Configuration Mode (ACM)	39–10
Data Transfer	39–11
Chain Assignment	39–11
DMA Access	39–13
Burst Access Support	39–13
Data and Coefficient Load in Parallel	39–13
Accelerator TCB	39–13
Chain Pointer DMA	39–13
Programming Model.....	39–15
Legacy Mode	39–15
Auto Configuration Mode (ACM)	39–16
Debug Mode	39–17
Write to Local Memory	39–17
Read from Local Memory.....	39–18
Single-Step Mode	39–18
Computing FIR Output, Tap Length Greater than 4096.....	39–18
Debug Features	39–19
Local Memory Access	39–19
Single-Step Mode	39–20
Emulation Considerations	39–20
Interrupts.....	39–20
Sources	39–20
Window Complete	39–20
All Channels Complete	39–21
MAC Status	39–21
Service	39–21
ADSP-2159x FIR Register Descriptions	39–21
FIR Chain Pointer Register	39–23

FIR Coefficient Count Register	39–24
FIR Coefficient Index Register	39–25
FIR Coefficient Modifier Register	39–26
FIR Global Control Register	39–27
FIR Channel Control Register	39–31
Debug Address Register	39–33
FIR Debug Control Register	39–34
FIR Debug Data Read Register	39–35
FIR Debug Data Write Register	39–36
FIR DMA Status Register	39–37
FIR Input Data Base Register	39–39
FIR Input Data Count Register	39–40
FIR Input Data Index Register	39–41
FIR Input Data Modifier Register	39–42
FIR MAC Status Register	39–43
FIR Output Data Base Register	39–46
FIR Output Data Count Register	39–47
FIR Output Data Index Register	39–48
FIR Output Data Modifier Register	39–49
software control register 1	39–50
software control register 2	39–51
Secondary global control register	39–52

IIR Accelerator (IIR)

Features.....	40–1
Clocking	40–1
Functional Description	40–2
ADSP-2159x IIR Register List	40–3
ADSP-2159x IIR Interrupt List	40–4
ADSP-2159x IIR Trigger List.....	40–5

Multiply and Accumulate (MAC) Unit	40-6
Input Data and Biquad State	40-6
Coefficient Memory	40-6
Internal Memory Storage.....	40-6
Coefficient Memory Storage.....	40-6
Operating Modes.....	40-7
Window Processing Mode	40-7
40-Bit Floating-Point Mode	40-7
Save Biquad State Mode	40-8
Auto Configuration Mode (ACM)	40-8
Data Transfers.....	40-9
IIR Accelerator TCB.....	40-9
DMA Access.....	40-11
Burst Mode Access	40-11
Chain Pointer DMA	40-11
Interrupts.....	40-12
Sources	40-13
Window Complete	40-13
All Channels Complete	40-13
MAC Status	40-14
Programming Model.....	40-14
ADSP-2159x IIR Register Descriptions	40-17
Chain Pointer Register	40-19
Coefficient Buffer Index Register	40-20
Coefficient Buffer Length Register	40-21
Coefficient Index Modifier Register	40-22
Global Control Register	40-23
Channel Control Register	40-27
IIR Debug Address Register	40-29
IIR Debug Control Register	40-30

IIR Debug Read Data High Register	40-31
IIR Debug Read Data Low Register	40-32
IIR Debug Write Data High Register	40-33
IIR Debug Write Data Low Register	40-34
DMA Status Register	40-35
Input Buffer Base Register	40-37
Input Data Index Register	40-38
Input Data Buffer Length Register	40-39
Input Data Index Modifier Register	40-40
MAC Status Register	40-41
Output Buffer Base Register	40-42
Output Data Buffer Index Register	40-43
IIR Output Data Buffer Length Register	40-44
IIR Output Data Index Modifier Register	40-45
Software Control Register1	40-46
Software Control Register2	40-47
Secondary Global Control Register	40-48

Boot ROM and Booting the Processor

SRAM Requirements	41-1
Preboot Operations.....	41-2
Start-up Sequence.....	41-3
Core Reset Sequencing	41-3
Core 1 Start-up	41-4
Core 2 Start-up	41-6
Fault Configuration.....	41-6
L2 Controller Configuration	41-7
L2 Memory Initialization	41-7
Idle On Entry	41-7
SPU Configuration.....	41-8
SMPU Configuration	41-8

Secure Debug Key Processing	41-9
CGU Configuration	41-10
Releasing All Cores from Reset	41-12
L1 Memory Initialization	41-12
Default Entry Point	41-13
NO-BOOT Processing	41-13
<u>SYS_RESOUT</u> Signal	41-14
DMC Configuration	41-14
Bypassing the Boot Process	41-15
Boot Mode Disable	41-15
Boot Command Customization	41-16
Boot Mode Specific SPU Configuration	41-16
Executing the Boot Mode	41-17
Boot Modes	41-17
No-Boot Mode	41-18
SPI Master Boot Mode	41-18
SPI Slave Boot Mode	41-24
Link Port Slave Boot Mode	41-27
UART Slave Boot Mode	41-29
OSPI Master Boot Mode	41-32
Boot Loader Stream	41-40
Block Types	41-43
Normal Block	41-44
First Block	41-44
Final Block	41-44
Indirect Block	41-45
Ignore Block	41-45
Fill Block	41-46
Init Block	41-46
Callback Block	41-48

Save Block	41-49
Single-Block Boot Streams	41-49
Direct Code Execution	41-50
Multi-Application Boot Streams	41-51
CRC32 Protection	41-53
Secure Boot.....	41-53
Terminology	41-55
Signing for Secure Boot Images	41-56
Secure Boot Image Types.....	41-56
Secure Boot Image Format.....	41-57
Secure Boot Image Attributes	41-60
Secure Boot Streams	41-60
Secure Debug Access.....	41-63
Boot ROM Errors and Failures	41-64
Boot ROM Programming Model	41-67
Boot Mode Driver API	41-67
Error Handler	41-69
Page Mode	41-69
Boot Hook Function	41-70
enum ROM_HOOK_CALL_CAUSE	41-70
Boot Return Feature	41-70
Boot Termination and Application Execution	41-71
Boot ROM OTP Customizations	41-71
API Reference	41-72
adi_rom_Boot()	41-72
adi_rom_BootKernel().....	41-75
adi_rom_Crc32Init()	41-75
adi_rom_Crc32Poly()	41-76
adi_rom_GetAddress().....	41-76
adi_rom_MemCompare()	41-77

adi_rom_MemCopy()	41-77
adi_rom_MemCrc()	41-78
adi_rom_MemDma()	41-79
adi_rom_MemFill()	41-82
adi_rom_PeriphDma()	41-82
adi_rom_otp_cfg()	41-83
adi_rom_otp_get()	41-84
adi_rom_otp_lock()	41-84
adi_rom_otp_fa_enable()	41-84
adi_rom_otp_pgm()	41-85
callback()	41-85
initcode()	41-87
adi_rom_idle_loop()	41-88
adi_rom_CguInit()	41-88
adi_rom_DmcPhyCalibration()	41-88
adi_rom_DmcInit()	41-89
adi_rom_ShaInit()	41-89
adi_rom_Sha()	41-90
Data Structures	41-93
struct ADI_ROM_BOOT_BUFFER	41-93
struct ADI_ROM_BOOT_CONFIG	41-93
struct ADI_ROM_BOOT_HEADER	41-102
struct ADI_ROM_BOOT_INTER_BUFFER	41-103
struct ADI_ROM_BOOT_INTER_BUFFERS	41-104
struct ADI_ROM_BOOT_LINKPORT	41-104
struct ADI_ROM_BOOT_MODES	41-105
struct ADI_ROM_BOOT_REGISTRY	41-106
struct ADI_ROM_BOOT_SPI	41-107
struct ADI_ROM_BOOT_OSPI	41-109
struct ADI_ROM_BOOT_UART	41-111

struct ADI_ROM_OTP_BOOT_CFG	41-112
struct ADI_ROM_OTP_BOOT_CGU_INFO	41-115
struct ADI_ROM_OTP_BOOT_CMD_INFO	41-118
struct ADI_ROM_OTP_BOOT_INFO	41-119
struct ADI_ROM_OTP_DMC_CONFIG.....	41-120
struct ROM_BOOT_DMA_INSTANCE.....	41-121
struct ROM_BOOT_MDMA	41-122
struct ROM_BOOT_MDMA_REGS	41-122
struct ROM_DMA_MDMA_CONFIG	41-123
struct ROM_DMA_PDMA_CONFIG	41-124
struct otp_data	41-126
Enumerations	41-129
enum ADI_ROM_BOOT_KEY_TYPE	41-129
enum ADI_ROM_BOOT_TYPE	41-129
enum OTPCMD	41-130
enum ROM_BOOT_MDMA_CRC_SUPPORT.....	41-132
enum ROM_BOOT_RESULT	41-133
enum ROM_CORE_ID	41-137
enum ROM_DMA_DONE_DETECT_METHOD.....	41-138
enum ROM_DMA_MDMA_ID.....	41-139
enum ROM_DMA_MDMA_OPERATION.....	41-139
enum ROM_DMA_PDMA_OPERATION	41-140
enum ROM_GETADDR_VALUE	41-141
enum ROM_HOOK_CALL_CAUSE	41-141
enum ROM_SB_IMAGE_TYPE.....	41-142
enum ROM_SPI_PROTOCOL	41-143

System Crossbars (SCB)

SCB Features	42-1
SCB Functional Description	42-1

ADSP-2159x SCB0 Register List.....	42-2
ADSP-2159x SCB1 Register List.....	42-7
ADSP-2159x SCB3 Register List.....	42-7
ADSP-2159x SCB4 Register List.....	42-7
ADSP-2159x SCB5 Register List.....	42-8
SCB Architectural Concepts	42-8
SCB Block Diagram	42-9
SCB Block Diagram	42-10
System Crossbars	42-20
SCB Bus Master IDs	42-21
SCB Programming Model.....	42-23
SCB Programming Concepts.....	42-25
QoS Programming	42-29
ADSP-2159x SCB0 Register Descriptions	42-30
CRC0 Channel 0 Read Quality of Service Register	42-36
CRC0 Channel 0 Write Quality of Service Register	42-37
CRC0 Channel 1 Read Quality of Service Register	42-38
CRC0 Channel 1 Write Quality of Service Register	42-39
CRC1 Channel 0 Read Quality of Service Register	42-40
CRC1 Channel 0 Write Quality of Service Register	42-41
CRC1 Channel 1 Read Quality of Service Register	42-42
CRC1 Channel 1 Write Quality of Service Register	42-43
CRC2 Channel 0 Read Quality of Service Register	42-44
CRC2 Channel 0 Write Quality of Service Register	42-45
CRC2 Channel 1 Read Quality of Service Register	42-46
CRC2 Channel 1 Write Quality of Service Register	42-47
CRC3 Channel 0 Read Quality of Service Register	42-48
CRC3 Channel 0 Write Quality of Service Register	42-49
CRC3 Channel 1 Read Quality of Service Register	42-50
CRC3 Channel 1 Write Quality of Service Register	42-51

CRYPTO Read Quality of Service Register	42-52
CRYPTO Write Quality of Service Register	42-53
DBG Read Quality of Service Register	42-54
DBG Write Quality of Service Register	42-55
DLDMA0 Channel 0 Read Quality of Service Register	42-56
DLDMA0 Channel 0 Write Quality of Service Register	42-57
DLDMA0 Channel 1 Read Quality of Service Register	42-58
DLDMA0 Channel 1 Write Quality of Service Register	42-59
DLDMA1 Channel 0 Read Quality of Service Register	42-60
DLDMA1 Channel 0 Write Quality of Service Register	42-61
DLDMA1 Channel 1 Read Quality of Service Register	42-62
DLDMA1 Channel 1 Write Quality of Service Register	42-63
EMAC Read Quality of Service Register	42-64
EMAC Write Quality of Service Register	42-65
ETR Read Quality of Service Register	42-66
ETR Write Quality of Service Register	42-67
GIGE Read Quality of Service Register	42-68
GIGE Write Quality of Service Register	42-69
HSMDMA1 Channel 0 Read Quality of Service Register	42-70
HSMDMA1 Channel 0 Write Quality of Service Register	42-71
HSMDMA1 Channel 1 Read Quality of Service Register	42-72
HSMDMA1 Channel 1 Write Quality of Service Register	42-73
HSMDMA Channel 0 Read Quality of Service Register	42-74
HSMDMA Channel 0 Write Quality of Service Register	42-75
HSMDMA Channel 1 Read Quality of Service Register	42-76
HSMDMA Channel 1 Write Quality of Service Register	42-77
LP0 Read Quality of Service Register	42-78
LP0 Write Quality of Service Register	42-79
LP1 Read Quality of Service Register	42-80
LP1 Write Quality of Service Register	42-81

MLB Read Quality of Service Register	42–82
MLB Write Quality of Service Register	42–83
MSMDMA1 Channel 0 Read Quality of Service Register	42–84
MSMDMA1 Channel 0 Write Quality of Service Register	42–85
MSMDMA1 Channel 1 Read Quality of Service Register	42–86
MSMDMA1 Channel 1 Write Quality of Service Register	42–87
MSMDMA Channel 0 Read Quality of Service Register	42–88
MSMDMA Channel 0 Write Quality of Service Register	42–89
MSMDMA Channel 1 Read Quality of Service Register	42–90
MSMDMA Channel 1 Write Quality of Service Registers	42–91
ARM_L2CC M0 Read Quality of Service Register	42–92
ARM_L2CC M0 Write Quality of Service Register	42–93
ARM_L2CC M1 Read Quality of Service Register	42–94
ARM_L2CC M1 Write Quality of Service Register	42–95
PPI F0 Read Quality of Service Register	42–96
PPI F0 Write Quality of Service Register	42–97
PPI F1 Read Quality of Service Register	42–98
PPI F1 Write Quality of Service Register	42–99
LP Fabric (CLKO8) Synchronization Mode Register	42–100
SH0 DPORT Read Quality of Service Register	42–101
SH0 DPORT Write Quality of Service Register	42–102
SH0 FIR Channel 0 Read Quality of Service Register	42–103
SH0 FIR Channel 0 Write Quality of Service Register	42–104
SH0 FIR Channel 1 Read Quality of Service Register	42–105
SH0 FIR Channel 1 Write Quality of Service Register	42–106
SH0 IIR Channel 0 Read Quality of Service Register	42–107
SH0 IIR Channel 0 Write Quality of Service Register	42–108
SH0 IIR Channel 1 Read Quality of Service Register	42–109
SH0 IIR Channel 1 Write Quality of Service Register	42–110
SH0 LPORT Read Quality of Service Register	42–111

SH0 LPORT Write Quality of Service Register	42-112
SH0 MMR Read Quality of Service Register	42-113
SH0 MMR Write Quality of Service Register	42-114
SH1 DPORT Read Quality of Service Register	42-115
SH1 DPORT Write Quality of Service Register	42-116
SH1 FIR Channel 0 Read Quality of Service Register	42-117
SH1 FIR Channel 0 Write Quality of Service Register	42-118
SH1 FIR Channel 1 Read Quality of Service Register	42-119
SH1 FIR Channel 1 Write Quality of Service Register	42-120
SH1 IIR Channel 0 Read Quality of Service Register	42-121
SH1 IIR Channel 0 Write Quality of Service Register	42-122
SH1 IIR Channel 1 Read Quality of Service Register	42-123
SH1 IIR Channel 1 Write Quality of Service Register	42-124
SH1 IPORT Read Quality of Service Register	42-125
SH1 IPORT Write Quality of Service Register	42-126
SH1 MMR Read Quality of Service Register	42-127
SH1 MMR Write Quality of Service Register	42-128
SP0A Read Quality of Service Register	42-129
SP0A Write Quality of Service Register	42-130
SP0B Read Quality of Service Register	42-131
SP0B Write Quality of Service Register	42-132
SP1A Read Quality of Service Register	42-133
SP1A Write Quality of Service Register	42-134
SP1B Read Quality of Service Register	42-135
SP1B Write Quality of Service Register	42-136
SP2A Read Quality of Service Register	42-137
SP2A Write Quality of Service Register	42-138
SP2B Read Quality of Service Register	42-139
SP2B Write Quality of Service Register	42-140
SP3A Read Quality of Service Register	42-141

SP3A Write Quality of Service Register	42-142
SP3B Read Quality of Service Register	42-143
SP3B Write Quality of Service Register	42-144
SP4A Read Quality of Service Register	42-145
SP4A Write Quality of Service Register	42-146
SP4B Read Quality of Service Register	42-147
SP4B Write Quality of Service Register	42-148
SP5A Read Quality of Service Register	42-149
SP5A Write Quality of Service Register	42-150
SP5B Read Quality of Service Register	42-151
SP5B Write Quality of Service Register	42-152
SP6A Read Quality of Service Register	42-153
SP6A Write Quality of Service Registers	42-154
SP6B Read Quality of Service Register	42-155
SP6B Write Quality of Service Register	42-156
SP7A Read Quality of Service Register	42-157
SP7A Write Quality of Service Register	42-158
SP7B Read Quality of Service Register	42-159
SP7B Write Quality of Service Register	42-160
SPI0 RX Read Quality of Service Register	42-161
SPI0 RX Write Quality of Service Register	42-162
SPI0 TX Read Quality of Service Register	42-163
SPI0 TX Write Quality of Service Register	42-164
SPI1 RX Read Quality of Service Register	42-165
SPI1 RX Write Quality of Service Register	42-166
SPI1 TX Read Quality of Service Register	42-167
SPI1 TX Write Quality of Service Register	42-168
SPI2 RX Read Quality of Service Register	42-169
SPI2 RX Write Quality of Service Register	42-170
SPI2 TX Read Quality of Service Register	42-171

SPI2 TX Write Quality of Service Register	42–172
SPI3 RX Read Quality of Service Register	42–173
SPI3 RX Write Quality of Service Register	42–174
SPI3 TX Read Quality of Service Register	42–175
SPI3 TX Write Quality of Service Register	42–176
UART0 RX Read Quality of Service Register	42–177
UART0 RX Write Quality of Service Register	42–178
UART0 TX Read Quality of Service Register	42–179
UART0 TX Write Quality of Service Register	42–180
UART1 RX Read Quality of Service Register	42–181
UART1 RX Write Quality of Service Register	42–182
UART1 TX Read Quality of Service Register	42–183
UART1 TX Write Quality of Service Registers	42–184
UART2 RX Read Quality of Service Register	42–185
UART2 RX Write Quality of Service Register	42–186
UART2 TX Read Quality of Service Register	42–187
UART2 TX Write Quality of Service Register	42–188
UART3 RX Read Quality of Service Register	42–189
UART3 RX Write Quality of Service Register	42–190
UART3 TX Read Quality of Service Register	42–191
UART3 TX Write Quality of Service Register	42–192
USB0 Read Quality of Service Register	42–193
USB0 Write Quality of Service Register	42–194
ADSP-2159x SCB1 Register Descriptions	42–194
DMC Fabric (CLK03) Synchronization Mode Register	42–195
ADSP-2159x SCB3 Register Descriptions	42–195
DMC MMRG Fabric (CLK03) Synchronization Mode Register	42–196
DMC MMRG Fabric (CLK04) Synchronization Mode Register	42–197
DMC MMRG Fabric (CLK08) Synchronization Mode Register	42–198
ADSP-2159x SCB4 Register Descriptions	42–198

Fabric Acc Mmr Ib.read Qos	42-200
Fabric Acc Mmr Ib.write Qos	42-201
Fabric S1port Ib.read Qos	42-202
Fabric S1port Ib.write Qos	42-203
Fabric S2port Ib.read Qos	42-204
Fabric S2port Ib.write Qos	42-205
Fir Ch0 Ib.read Qos	42-206
Fir Ch0 Ib.write Qos	42-207
Fir Ch1 Ib.read Qos	42-208
Fir Ch1 Ib.write Qos	42-209
Iir0 Ch0 Ib.read Qos	42-210
Iir0 Ch0 Ib.write Qos	42-211
Iir0 Ch1 Ib.read Qos	42-212
Iir0 Ch1 Ib.write Qos	42-213
Iir1 Ch0 Ib.read Qos	42-214
Iir1 Ch0 Ib.write Qos	42-215
Iir1 Ch1 Ib.read Qos	42-216
Iir1 Ch1 Ib.write Qos	42-217
Iir2 Ch0 Ib.read Qos	42-218
Iir2 Ch0 Ib.write Qos	42-219
Iir2 Ch1 Ib.read Qos	42-220
Iir2 Ch1 Ib.write Qos	42-221
Iir3 Ch0 Ib.read Qos	42-222
Iir3 Ch0 Ib.write Qos	42-223
Iir3 Ch1 Ib.read Qos	42-224
Iir3 Ch1 Ib.write Qos	42-225
Sharc Dport.read Qos	42-226
Sharc Dport.write Qos	42-227
ADSP-2159x SCB5 Register Descriptions	42-227
SPI2/OSPI Memory Map Address Remap Register	42-228

System Watchpoint Unit (SWU)

SWU Features.....	43-1
SWU Functional Description.....	43-1
ADSP-2159x SWU Register List	43-1
ADSP-2159x SWU Interrupt List	43-2
ADSP-2159x SWU Trigger List	43-3
SWU Definitions.....	43-4
SWU Architectural Concepts.....	43-4
SWU-to-SCB Interface.....	43-4
SWU Block Diagram.....	43-4
SCB Interface Block	43-5
MMR Interface Block	43-5
SWU Operating Modes	43-5
Bandwidth Mode.....	43-5
Watchpoint Mode.....	43-5
Match Block	43-5
Scaling.....	43-6
SWU Event Control	43-6
SWU Interrupts.....	43-6
SWU Status and Errors.....	43-7
Triggers	43-7
SWU Programming Model	43-7
SWU Mode Configuration	43-8
Configuring the SWU for Bandwidth Mode	43-8
Configuring the SWU for Watchpoint Mode	43-9
ADSP-2159x SWU Register Descriptions	43-10
Count Register n	43-11
Control Register n	43-12
Current Register n	43-17
Global Control Register	43-18

Global Status Register	43–19
Bandwidth History Register n	43–23
ID Register n	43–24
Lower Address Register n	43–25
Target Register n	43–26
Upper Address Register n	43–27

Memory Error Protection Unit (MEPU)

Memory Error Controller (MEC)	44–1
MEC Features	44–1
Parity Controller (PCTL)	44–2
PCTL Features.....	44–2
MEC Functional Description.....	44–2
ADSP-2159x MEC Register List	44–2
ADSP-2159x MEC Interrupt List	44–3
ADSP-2159x MEC Trigger List.....	44–4
MEPU Block Diagram	44–5
MEC Block Diagram.....	44–5
PCTL Block Diagram.....	44–6
MECx Input and Output Block Diagram	44–6
MEC Architectural Concepts.....	44–7
MEC Configuration	44–7
PCTL Integration	44–9
ADSP-2159x MEC Register Descriptions	44–9
Component ID0 Register	44–11
Component ID1 Register	44–12
Component ID2 Register	44–13
Component ID3 Register	44–14
Clear Register	44–15
ECC Error Control Register	44–16

ECC Error Interrupt Mask Register	44-17
ECC Error Status Register	44-18
ECC Error Interrupt Request Global Control Register	44-19
ECC Error Interrupt Request Global Status Register	44-20
Parity Error Interrupt Request Global Control Register	44-21
Parity Error Interrupt Request Global Status Register	44-22
Parity Error Control Register	44-23
Parity Error Control Register	44-25
Parity Error Interrupt Mask Register	44-26
Parity Error Interrupt Mask Register	44-28
Parity Error Status Register	44-29
Parity Error Status Register	44-31
Peripheral ID0 Register	44-32
Peripheral ID1 Register	44-33
Peripheral ID2 Register	44-34
Peripheral ID3 Register	44-35
Peripheral ID4 Register	44-36
Peripheral ID5 Register	44-37
Peripheral ID6 Register	44-38
Peripheral ID7 Register	44-39

System Debug and Trace Unit (DBG)

DBG Features	45-1
DBG Functional Description	45-2
ADSP-2159x CSPFT Register List	45-2
ADSP-2159x TAPC Register List	45-3
DBG Block Diagram	45-3
DBG Definitions	45-4
Test Access Port Controller (TAPC)	45-5
Embedded Trace Macrocell (ETM)	45-6

Debug Access Ports.....	45–6
Trace Unit	45–6
Programmable Flow Trace (CSPFT).....	45–6
System Trace Module (STM)	45–7
Embedded Cross Trigger (ECT)	45–7
CTI Debug Trigger Tables.....	45–9
ADSP-2159x CSPFT Register Descriptions	45–11
Address Comparator Access Type Register	45–13
Address Comparator Value Register	45–14
Authentication Status Register	45–15
Configuration Code Extension Register	45–16
Component ID0 Register	45–17
Component ID1 Register	45–18
Component ID2 Register	45–19
Component ID3 Register	45–20
Context ID Comparator Mask Register	45–21
Context ID Comparator Value	45–22
Claim Tag Clear Register	45–23
Claim Tag Set Register	45–24
Counter Enable Event Register	45–25
Counter Reload Event Register	45–28
Counter Reload Value Register	45–31
Counter Value Register	45–32
Main Control Register	45–33
Device Type Identifier Register	45–35
External Output Event Register	45–36
Hardware Feature Register	45–39
Lock Access Register	45–41
Lock Status Register	45–42
Peripheral ID0 Register	45–43

Peripheral ID1 Register	45–44
Peripheral ID2 Register	45–45
Peripheral ID3 Register	45–46
Peripheral ID4 Register	45–47
Status Register	45–48
Synchronization Frequency Register	45–49
TraceEnable Control Register	45–50
TraceEnable Event Register	45–51
CoreSight Trace ID Register	45–54
Trigger Event Register	45–55
TraceEnable Start/Stop Control Register	45–58
ADSP-2159x TAPC Register Descriptions	45–59
Debug Control Register	45–61
IDCODE Register	45–63
Secure Debug Key 0 Register	45–64
Secure Debug Key 1 Register	45–65
Secure Debug Key 2 Register	45–66
Secure Debug Key 3 Register	45–67
Secure Debug Key Control Register	45–68
Secure Debug Key Status Register	45–69
USERCODE Register	45–70

ADSP-2159x Register List

Preface

Thank you for purchasing and developing systems using an ADSP-2159x SHARC+[®] processor from Analog Devices, Inc.

Purpose of This Manual

The *ADSP-2159x SHARC+ Processor Hardware Reference* provides architectural information about the ADSP-2159x processors. This hardware reference provides the main architectural information about these processors. The architectural descriptions cover functional blocks, buses, and ports, including all features and processes that they support.

For timing, electrical, and package specifications, refer to the *SHARC+ Dual-Core DSP with Arm Cortex-A5 ADSP-21591/593/594/ADSP-SC592/594 Data Sheet*.

Intended Audience

Technical or Customer Support

You can reach customer and technical support for processors from Analog Devices in the following ways:

- Post your questions in the processors and DSP support community at *EngineerZone*[®]:

<http://ez.analog.com/community/dsp>

- Submit your questions to technical support at *Connect with ADI Specialists*:

<http://www.analog.com/support>

- E-mail your questions about software/hardware development tools to:

processor.tools.support@analog.com

- E-mail your questions about processors and DSPs to:

processor.support@analog.com (world wide support)

processor.china@analog.com (China support)

- Contact your Analog Devices sales office or authorized distributor. Locate one at:

<http://www.analog.com/adi-sales>

- Send questions by mail to:

Analog Devices, Inc.

Three Technology Way

P.O. Box 9106

Norwood, MA 02062-9106 USA

Product Information

Product information can be obtained from the Analog Devices Web site.

Analog Devices Web Site

The Analog Devices Web site, <http://www.analog.com>, provides information about a broad range of products—analog integrated circuits, amplifiers, converters, and digital signal processors.

To access a complete technical library for each processor family, go to: http://www.analog.com/processors/technical_library The manuals selection opens a list of current manuals related to the product as well as a link to the previous revisions of the manuals. When locating your manual title, note a possible errata check mark next to the title that leads to the current correction report against the manual.

Also note, [MyAnalog.com](http://www.analog.com) is a free feature of the Analog Devices Web site that allows customization of a Web page to display only the latest information about products you are interested in. You can choose to receive weekly e-mail notifications containing updates to the Web pages that meet your interests, including documentation errata against all manuals. [MyAnalog.com](http://www.analog.com) provides access to books, application notes, data sheets, code examples, and more.

Visit [MyAnalog.com](http://www.analog.com) to sign up. If you are a registered user, just log on. Your user name is your e-mail address.

EngineerZone

[EngineerZone](http://www.analog.com) is a technical support forum from Analog Devices. It allows you direct access to ADI technical support engineers. You can search FAQs and technical information to get quick answers to your embedded processing and DSP design questions.

Use EngineerZone to connect with other DSP developers who face similar design challenges. You can also use this open forum to share knowledge and collaborate with the ADI support team and your peers. Visit <http://ez.analog.com> to sign up.

Supported Processors

The following is the list of Analog Devices, Inc. processors.

Blackfin+[®] (ADSP-BF7xx) Processors

The name *Blackfin+* refers to the enhanced fixed-point Blackfin core architecture featured by the ADSP-BF70x processor product line, which is a family of 16-bit embedded processors.

Blackfin® (ADSP-BF6xx/BF5xx) Processors

The name *Blackfin* refers to the fixed-point core architecture featured on the following processors: ADSP-BF5xx and ADSP-BF6xx.

SHARC® (ADSP-21xxx) Processors

The name *SHARC* refers to the high-performance, 32-bit, floating-point core architecture featured on the following processors: ADSP-2116x, ADSP-2126x, ADSP-213xx, and ADSP-214xx. These processors can be used in speech, sound, graphics, and imaging applications.

SHARC+® (ADSP-SC5xx, ADSP-215xx) Processors

The name *SHARC+* refers to the enhanced high-performance, 32-bit, floating-point core architecture featured on the following processors: ADSP-215xx/ADSP-SC5xx. The connected SHARC+ ADSP-SC5xx processors also contain an Arm® Cortex®-A5 core. These products can be used in speech, sound, graphics, and imaging applications.

The following is the list of Analog Devices, Inc. processors supported by the IAR Embedded WorkBench® development tools. For information about the IAR Embedded WorkBench product and software download, go to <http://www.iar.com/en/Products/IAR-Embedded-Workbench>.

Notation Conventions

Text conventions used in this manual are identified and described as follows. Additional conventions, which apply only to specific chapters, may appear throughout this document.

Example	Description
<i>File > Close</i>	Titles in reference sections indicate the location of an item within the CrossCore Embedded Studio IDE's menu system (for example, the <i>Close</i> command appears on the <i>File</i> menu).
{this that}	Alternative required items in syntax descriptions appear within curly brackets and separated by vertical bars; read the example as <i>this</i> or <i>that</i> . One or the other is required.
[this that]	Optional items in syntax descriptions appear within brackets and separated by vertical bars; read the example as an optional <i>this</i> or <i>that</i> .
[this, ...]	Optional item lists in syntax descriptions appear within brackets delimited by commas and terminated with an ellipse; read the example as an optional comma-separated list of <i>this</i> .
.SECTION	Commands, directives, keywords, and feature names are in text with Letter Gothic font.
<i>filename</i>	Non-keyword placeholders appear in text with italic style format.

Example	Description
NOTE:	<i>NOTE:</i> For correct operation, ... A note provides supplementary information on a related topic. In the online version of this book, the word <i>NOTE:</i> appears instead of this symbol.
CAUTION:	<i>CAUTION:</i> Incorrect device operation may result if ... <i>CAUTION:</i> Device damage may result if ... A caution identifies conditions or inappropriate usage of the product that could lead to undesirable results or product damage. In the online version of this book, the word <i>CAUTION:</i> appears instead of this symbol.
ATTENTION:	<i>ATTENTION:</i> Injury to device users may result if ... A warning identifies conditions or inappropriate usage of the product that could lead to conditions that are potentially hazardous for devices users. In the online version of this book, the word <i>ATTENTION:</i> appears instead of this symbol.
Registers/Bits	All registers and bits in this manual are linked (clickable) to their respective descriptions in the "Register Descriptions" of each chapter.
Miscellaneous Conventions	Interrupt and internal signals are shown in all caps with no other formatting. For example the SPDIFn_RX or SCLK signal or the PKTE0_IRQ interrupt. An overbar denotes an active-low signal as in $\overline{\text{SYS_FAULT}}$.

Register Documentation Conventions

Register diagrams use the following conventions:

- The descriptive name of the register appears at the top with the short form of the name.
- If a bit has a short name, the short name appears first in the bit description, followed by the long name.
- The reset value appears in binary in the individual bits and in hexadecimal to the left of the register.
- Bits marked *X* have an unknown reset value. Consequently, the reset value of registers that contain such bits is undefined or dependent on pin values at reset.
- Shaded bits are reserved

NOTE: To ensure upward compatibility with future implementations, write back the value that is read for reserved bits in a register, unless otherwise specified.

Register description tables use the following conventions:

- Each bit's or bit field's access type appears beneath the bit number in the table in the form (read-access/write-access). The access types include:
 - R = read, RC = read clear, RS = read set, R0 = read zero, R1 = read one, Rx = read undefined
 - W = write, NW = no write, W1C = write one to clear, W1S = write one to set, W0C = write zero to clear, W0S = write zero to set, WS = write to set, WC = write to clear, W1A = write one action

- Many bit and bit field descriptions include enumerations, identifying bit values and related functionality. Unless otherwise indicated (with a prefix), these enumerations are decimal values.

1 Clock Generation Unit (CGU)

The Clock Generation Unit (CGU) includes the phase locked loop (PLL) and the PLL control unit (PCU). The PLL generates a master clock that runs at a frequency that is a multiple of the CLKIN input clock frequency. The PCU divides down the master clock to generate various system clocks and synchronization signals.

CGU Features

The CGU module supports the following features:

- Provides smooth transitions from the current clock condition to a new condition with PLL logic and executes the changes to clocks due to register programming.
- Provides PLL and clock domain status reporting for event management.
- Supports the capability to bypass the PLL for power savings.
- Software controlled dynamic power management allows control of the core clock frequency (f_{CLK}).
- Controls clock gating of the core and system clocks.

NOTE: For more information about processor-specific CGU features, see the processor data sheet.

CGU Functional Description

The CGU generates all on-chip clocks and synchronization signals based on the programmed PLL multiplication factor and dividers. The CGU provides the following functionality.

Change the PLL Clock Frequency

The CGU allows programs to change the PLL clock frequency by writing new values to bits in the control register. Any time the PLL rellocks, the CGU aligns all core and system clocks.

Change Other Clock Frequencies

The CGU allows programs to change the CCLKn, SYSCLK, SCLKn, DCLK, and OCLK frequencies by writing values to the [CGU_DIV](#) register. Any time the clock frequency is changed, the OCLK, CCLKn, SYSCLK, DCLK, and SCLKn clocks exit the frequency change sequence aligned.

Perform Clock Alignment

The CGU can align all clocks by writing to the [CGU_DIV](#) register. This function aligns all PLL-based clocks.

For more information on these functions, see the [CGU Programming Model](#) section.

ADSP-2159x CGU Register List

The clock generation unit (CGU) includes the phase locked loop (PLL) and the PLL control unit (PCU). The PLL generates a clock, running at a frequency that is a multiple of the CLKIN input clock's frequency. The CGU also generates all on-chip clocks and synchronization signals. The PCU permits application software control of the PLL's operation. A set of registers govern CGU operations. For more information on CGU functionality, see the CGU register descriptions.

Table 1-1: ADSP-2159x CGU Register List

Name	Description
CGU_CCBF_DIS	Core Clock Buffer Disable Register
CGU_CCBF_STAT	Core Clock Buffer Status Register
CGU_CLKOUTSEL	CLKOUT Select Register
CGU_CTL	Control Register
CGU_DIV	Clocks Divisor Register
CGU_DIVEX	DIV Register Extension
CGU_OSCWDCTL	Oscillator Watchdog Register
CGU_PLLCTL	PLL Control Register
CGU_REVID	Revision ID Register
CGU_SCBF_DIS	System Clock Buffer Disable Register
CGU_SCBF_STAT	System Clock Buffer Status Register
CGU_STAT	Status Register
CGU_TSCOUNT0	Time Stamp Counter 32 LSB Register
CGU_TSCOUNT1	Time Stamp Counter 32 MSB Register
CGU_TSCTL	Time Stamp Control Register
CGU_TSVALUE0	Time Stamp Counter Initial 32 LSB Value Register
CGU_TSVALUE1	Time Stamp Counter Initial MSB Value Register

ADSP-2159x CGU Interrupt List

Table 1-2: ADSP-2159x CGU Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
1	CGU0_EVT	CGU0 Event	Edge	
2	CGU1_EVT	CGU1 Event	Edge	

ADSP-2159x CGU Trigger List

Table 1-3: ADSP-2159x CGU Trigger List Masters

Trigger ID	Name	Description	Sensitivity
1	CGU0_EVT	CGU0 Event	Edge
2	CGU1_EVT	CGU1 Event	Edge

Table 1-4: ADSP-2159x CGU Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

CGU Definitions

DPM

The Dynamic Power Management module (DPM) works with the CGU to provide flexible power dissipation modes for the processor.

PCU

The PLL control unit (PCU) in the CGU controls PLL operations. The MMR registers of the CGU are implemented in the PCU.

PLL

The phase-locked loop (PLL) operates within the CGU.

RCU

The reset control unit (RCU) provides input to the CGU to manage clocks during processor reset.

CDU

The clock distribution unit distributes the clocks from the CGU to different clock domains

CGU

The clock generation unit (CGU) is comprised of the PLL and PCU. The CGU generates the clocks listed in the *Clock Descriptions* table.

Table 1-5: Clock Descriptions

Clock	Description
CCLK0_0	CCLK0 derived from CGU0
CCLK1_0	CCLK1 derived from CGU0
SYSCLK_0	SYSCLK derived from CGU0
SCLK0_0	SCLK0 derived from CGU0
SCLK1_0	SCLK1 derived from CGU0
DCLK_0	DCLK derived from CGU0
OCLK_0	OCLK derived from CGU0
CCLK0_1	CCLK0 derived from CGU1
SYSCLK_1	SYSCLK derived from CGU1
SCLK0_1	SCLK0 derived from CGU1
SCLK1_1	SCLK1 derived from CGU1
DCLK_1	DCLK derived from CGU1
OCLK_1	OCLK derived from CGU`

CGU PLL Block Diagram

The *CGU PLL Block Diagram* provides a top-level block diagram of the phase locked loop (PLL). The main blocks of the PLL are the phase/frequency detector (PFD), the charge pump, the loop filter and the voltage controlled oscillator (VCO). The VCO multiplies the SYS_CLKIN0 input to a higher frequency.

Additional configuration options are configured using the Clock Distribution Unit.

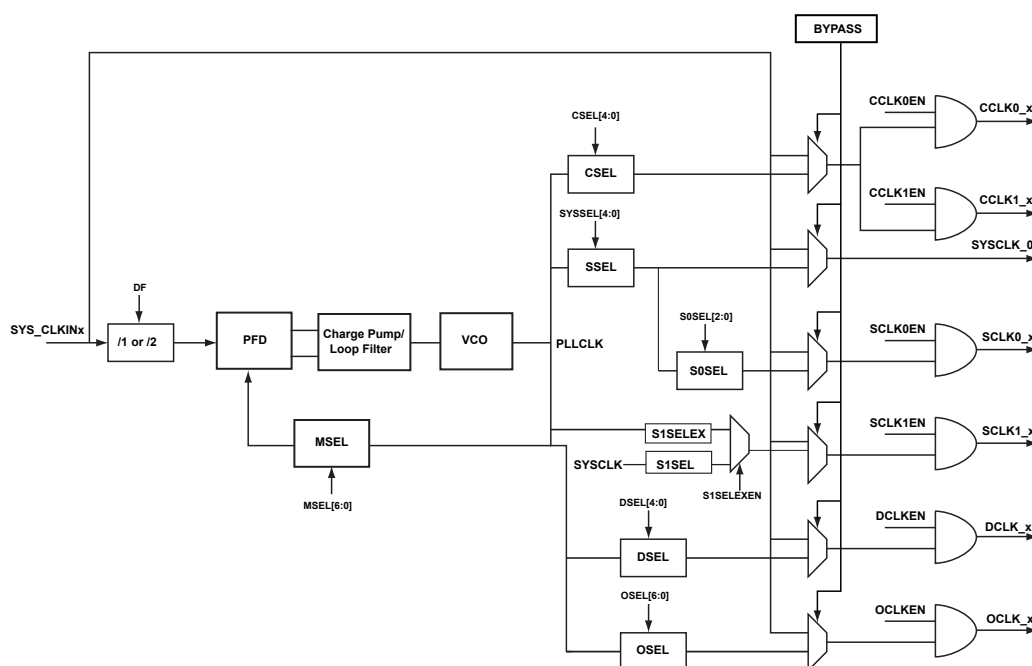


Figure 1-1: CGU PLL Block Diagram

CLKOUT Selections

The *SYS_CLKOUT Generation* figure is a conceptual representation of the CLKOUT module. Different clocks that originate from the CGU blocks are available on the SYS_CLKOUT output pin. The selection of the clock output on the SYS_CLKOUT pin is controlled by the CGU_CLKOUTSEL.CLKOUTSEL bit field. Most of the clock frequencies are divided with a fixed divider value, for example CGU0_SYSCLK0 has a fixed divider of 8 for the CLKOUT pin.

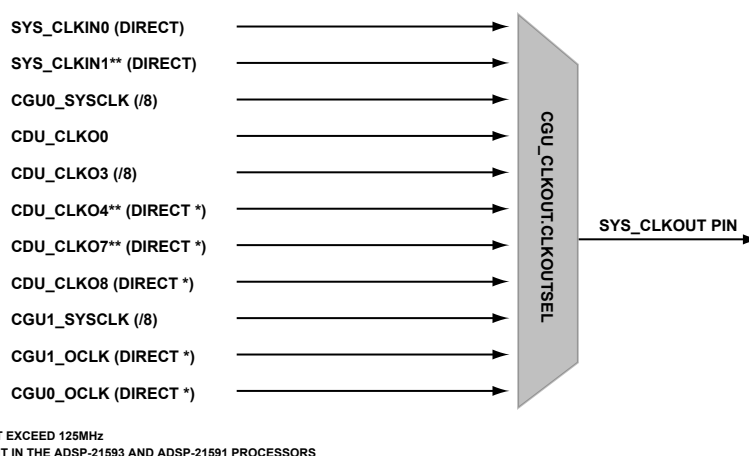


Figure 1-2: SYS_CLKOUT Generation

The BMODE pin affects the CLKOUT pins after a hardware reset is deasserted.

BMODE= 0 — When a hardware reset is deasserted, CGU0_OCLK/PLLCLK0 is selected to come out of the CLKOUT pin. The default value of OSEL and MSEL is the same. This functionality enables the user to verify PLL functionality by checking that the CLKOUT period is identical to CLKIN period.. While programming all values of the CGU_CLKOUTSEL.CLKOUTSEL bit field is possible, changing it to 0 (SYS_CLKIN) is not possible until and unless a non-zero value is first programmed to the the CGU_CLKOUTSEL.CLKOUTSEL bit field.

BMODE = (non zero) — When a hardware reset is deasserted, SYS_CLKIN is selected by default.

CGU Operating Modes

The CGU does not have configurable operating modes, but CGU operations affect the operating modes of other modules. Some CGU operation issues that affect operation of other modules include the following:

- The PLL of the CGU operates in either normal mode (CGU clock divisors applied) or bypass mode (CGU PLL is bypassed and clock divisors ignored).
- The SCB uses the CGU for clock synchronization across clock domains. For more information, see [System Crossbars \(SCB\)](#).
- The DPM uses the CGU for clock management as power state transitions occur. For more information, see the Dynamic Power Management (DPM) chapter.
- The CGU uses clock gating control to obtain flexible low-power modes.

CGU Power-up Sequence

See the product data sheet for exact power-up requirements. The processor is configured to come up in clock bypass mode. The programs is required to configure full speed clocks and safety monitors. CLKIN0 and all supplies should be stable before the SYS_HWRST signal is deasserted.

CGU Event Control

The CGU generates an event or error for several different reasons. Events and errors are described in the following sections.

Events

After a frequency change, a CGU event indicates that the PLL has locked and clocks are synchronized. If a core was idled while changing frequencies, the CGU can use an event interrupt to break the core idle. While in active mode, a CGU event indicates that the PLL has locked.

CGU Errors

A CGU error occurs under following conditions:

- A write access to the [CGU_DIV/CGU_DIVEX](#) registers trigger an alignment sequence while the PLL is locked and is still aligning the clocks.

The `CGU_STAT.WDIVERR` bit state indicates this error. If this error occurs, clear the `CGU_STAT.WDIVERR` bit and rewrite the desired values to the `CGU_DIV/CGU_DIVEX` registers.

- A change to the `CGU_DIV/CGU_DIVEX` registers occur while the PLL is locked and is still aligning the clocks.

The `CGU_STAT.WDIVERR` bit state indicates this error. If this error occurs, clear the `CGU_STAT.WDIVERR` bit and rewrite the desired values to the `CGU_DIV/CGU_DIVEX` registers.

- A write access to the `CGU_CTL.DF` bit field occurs or a write access to the `CGU_CTL.MSEL` bit field occurs while the PLL is locking.

The `CGU_STAT.WDFMSERR` bit state indicates this error. If this error occurs, wait until the PLL has finished locking, clear the error, and rewrite the desired value change.

The CGU monitors changes to the following fields:

- SYSCLK Divisor – `CGU_DIV.SYSSEL`

CGU Generated Bus Errors

The CGU generates a bus error when a read or write transaction to an unused address within the CGU address range is attempted. It also generates a bus error when a misaligned access is made to a CGU register. In addition to the bus error, the `CGU_STAT.ADDRERR` bit is set. If a write to a write-protected CGU register is attempted, the CGU generates a bus error. In addition, the `CGU_STAT.LWERR` bit is set.

Oscillator Watchdog

The oscillator watchdog detects the absence of input clock transitions and provides a fault warning through the `SYS_FAULT` pin. To detect harmonic or subharmonic crystal oscillator behavior, the watchdog (under programmable control) also detects and reports input oscillator frequencies above and below the specified limits. Use an internal asynchronous, local 1 MHz oscillator combined with a series of programmable counters for this detection. Set the `CGU_OSCWDCTL.MONDIS` bit and clear the `CGU_OSCWDCTL.FAULTEN` bit to optionally disable all the input clock monitor and fault detection functions.

Set the `CGU_OSCWDCTL.HODEN` bit to enable harmonic oscillation detection. The CGU uses the `CGU_OSCWDCTL.HODF` bit field to indicate the desired lower fail limit (in MHz) for the harmonic oscillation detection. The upper limit is always twice the lower limit.

The *HODF Settings for Different Input Clock Frequencies* table shows an example of the `CGU_OSCWDCTL.HODF` bit settings for different input clock frequencies.

Table 1-6: HODF Settings for Different Input Clock Frequencies

<code>CGU_OSCWDCTL.HODF[5:0]</code>	Subharmonic Frequency (MHz)	Nom. Lower Fail Limit (MHz)	Input Clock Frequency (MHz)	Nom. Upper Fail Limit (MHz)	Second Harmonic Frequency (MHz)
14	10	14	20	28	40

Table 1-6: HODF Settings for Different Input Clock Frequencies (Continued)

CGU_OSCWDCTL.HODF[5:0]	Subharmonic Frequency (MHz)	Nom. Lower Fail Limit (MHz)	Input Clock Frequency (MHz)	Nom. Upper Fail Limit (MHz)	Second Harmonic Frequency (MHz)
21	15	21	30	42	60

The CGU uses the CGU_OSCWDCTL.BOUF (Bad Oscillator Upper Frequency limit) asynchronous control bit field to indicate the desired upper fail limit for the bad oscillation detection. Set the CGU_OSCWDCTL.BOUEN bit to enable upper-limit bad oscillation detection. A bad oscillation detection condition signals a fault before any processor operations occur. This detection occurs (even in bypass mode) whenever a clock frequency exceeds the specifications.

The CGU_OSCWDCTL.BOUF = 0 operation starts with a target of 32 MHz and each additional LSB increases the frequency test limit by 2 MHz. For example:

Target Upper Frequency Limit = CGU_OSCWDCTL.BOUF × 2 MHz + 32 MHz

The CGU_STAT.OSCWDSTATFC status bits indicate the nature of the fault.

The *Fault Map* table shows the fault values.

Table 1-7: Fault Map

CGU_STAT.OSCWDSTATFC Bitfield Values	Fault Type
0	No Fault
1	No Input Clock
2	Subharmonic CLKIN
3	Harmonic CLKIN
4	No AUX_CLK
5	CLKIN > Upper Freq Limit (BOUF)
6	Reserved
7	Multiple Limit Faults

There is a priority to the faults given in the case of multiple fault errors. The highest priority is given to No Input clock followed by No AUX_CLK. The other three fault cases share the lowest priority. Multiple limit faults are asserted if more than one type of subharmonic CLKIN, harmonic CLKIN, or BOUF faults are observed.

NOTE: All the CGU_STAT.OSCWDSTATFC faults other than the absence of AUX_CLK (for example, CGU_STAT.OSCWDSTATFC = 4) are not reliable and used for debug only.

Program and enable the OSCWDOG to match the actual crystal, before bringing the PLL out of bypass.

CGU Programming Model

The programming model for the CGU involves the various mode configuration techniques.

Configuring CGU Modes

Use the following procedures to configure the clocks and PLL.

NOTE: The program needs to clear the `CGU_STAT.CLKSALGN` bit before changing clocks. The following sequence is executed once, inside the application, after coming out of reset.

```
*pREG_CGU0_PLLCTL |= BITM_CGU_PLLCTL_PLLBPCL; // come out of bypass and enter
Full ON
while( (pADI_CGU0 ->STAT & 0xF) != 0x5 ) { } // poll
// now clocks are running with hardware default divisors.
// now program can change frequencies If desired the program can put the PLL
again into bypass.
```

Changing the Clock Frequencies

Applications change clock frequencies in two ways. The first way is modifying the PLL multiplication value by writing to the `CGU_CTL` register and the second is modifying the clock dividers by writing to the `CGU_DIV` register. Both actions have different implications even if the frequencies of the final clock are the same. Write accesses to change the `CGU_CTL.DF` or `CGU_CTL.MSEL` bit fields while the PLL is locking set the `CGU_STAT.WDFMSERR` error bit. The `CGU_STAT.WDIVERR` error bit is set when one of following accesses is attempted while the PLL is locked, but still aligning the clocks:

- A write access to the `CGU_DIV/CGU_DIVEX` to trigger an alignment sequence.
- A write access to the `CGU_DIV/CGU_DIVEX` to change the `CGU_DIV.CSEL/`, `CGU_DIV.SYSSEL`, `CGU_DIV.S0SEL`, `CGU_DIV.S1SEL`, or `CGU_DIV.DSEL/CGU_DIVEX.S1SELEX` bits.

Read-after-write accesses to these registers return the new value, even if the frequency of the clock change is still in-progress.

Modifying the PLL multiplier requires the PLL to relock. Once the PLL locks, the CGU synchronizes the clocks. Changes to the `CGU_CTL.DF` or `CGU_CTL.MSEL` bit field result in bypassing the PLL. By setting the `CGU_CTL.WFI` bit, programs force the PLL to wait for the core to return to the idle or reset state before the frequency changes. If necessary, clear the `CGU_DIV.UPDT` bit to avoid multiple clock alignment sequences. If the `CGU_DIV` register is not updated, the CGU uses the current values to determine the frequencies of the clock. It is the programs responsibility to guarantee that the new `CGU_CTL.DF` or `CGU_CTL.MSEL` and `CGU_DIV/CGU_DIVEX` combinations are legal.

Changing the PLL Clock Frequency

To change the phase-locked loop clock (*PLLCLK*) frequency, write new values to the `CGU_CTL.MSEL` field or `CGU_CTL.DF` field. Any time the PLL relocks, all core and system clocks are aligned.

1. Read `CGU_STAT` register and verify that:
 - a. The `CGU_STAT.PLEN` bit =1 (PLL enabled)
 - b. The `CGU_STAT.PLOCK` bit =1 (PLL is not locking)

- c. The `CGU_STAT.CLKSALGN` bit =0 (clocks aligned)
2. Write the desired values to the clock divisor select fields of the `CGU_DIV/CGU_DIVEX` register with the `CGU_DIV.UPDT` bit =0.
3. Write the desired values to the `CGU_CTL.DF` and `CGU_CTL.MSEL` fields.
 - a. To change the PLL frequency while the core is idle, write to the `CGU_CTL` register with the `CGU_CTL.WFI` bit =1.
 - b. To change the PLL frequency while the core is active, write to the `CGU_CTL` register with the `CGU_CTL.WFI` bit =0.

This sequence performs these actions:

1. Updates the corresponding CGU registers.
2. Bypasses the PLL.
3. Makes the PLL lock to the new values in the `CGU_CTL.MSEL` or `CGU_CTL.DF` fields.
4. Changes the clock frequencies.
5. Exits the PLL bypass with all clocks aligned.

When exiting the PLL bypass state, a CGU event occurs.

The `CGU_STAT` register exits this sequence with the `CGU_STAT.PLEN` bit =1, the `CGU_STAT.PLOCK` bit =1, the `CGU_STAT.PLLBP` bit =0, and the `CGU_STAT.CLKSALGN` bit =0. Poll the `CGU_STAT.PLOCK` bit, `CGU_STAT.PLLBP` bit, and `CGU_STAT.CLKSALGN` bit to discover when the PLL is locked and the clocks are aligned.

Changing the frequency of the PLL is allowed while the PLL is bypassed. In this case the new PLLCLK frequency is not used until the PLL is no longer bypassed.

Changing the CCLKn or SYSCLK Frequency Without Modifying the PLLCLK Frequency

To change the clock frequencies, write new values to `CGU_DIV.CSEL` or `CGU_DIV.SYSSEL` bits. The frequency change occurs only when the PLL is not bypassed. Any time the CCLKn or SYSCLK clock frequencies are changed, they exit the frequency change sequence aligned.

1. Read the `CGU_STAT` register to verify that the `CGU_STAT.CLKSALGN` bit =0 (clocks aligned).
2. Write the desired `CGU_DIV.CSEL`, `CGU_DIV.SYSSEL`, and `CGU_DIV.OSEL` bitfield values with the `CGU_DIV.UPDT` bit = 1 and/or, write to the `CGU_DIVEX.S1SELEX` field. This write updates the `CGU_DIV` register, changes the SCLKn and SYSCLK frequencies, and aligns the clocks. When the clocks are aligned, a CGU event occurs.

The `CGU_STAT` register exits this sequence with the `CGU_STAT.CLKSALGN` bit =0. Poll the `CGU_STAT.CLKSALGN` bit to discover when the clocks are aligned. Any write attempt to change the

CGU_DIV.S0SEL or CGU_DIV.S1SEL bit fields while CGU_STAT.CLKSALGN bit =1 (clocks alignment in progress) triggers an MMR access bus error and the CGU_DIV register is not modified.

Also, any write attempt to change the CGU_DIVEX.S1SELEX field while the CGU_STAT.CLKSALGN bit =1 (clocks alignment in progress) triggers an MMR access bus error and the CGU_DIVEX register is not modified.

Programming the SYSCLK frequency to a higher value than CCLKn also triggers an MMR access bus error and the CGU_DIV/CGU_DIVEX register is not modified.

Writing to the CGU_DIV/CGU_DIVEX register is allowed while the processor is in active (PLL bypassed) mode. In this case the effect of the write is visible only after the transition to full-on (PLL not bypassed) mode.

Accessing the DDR memory while changing the SYSCLK frequency is not supported and can have unpredictable results.

Changing the OCLK Frequency

To change the OCLK clock frequency, write a new CGU_DIV.OSEL bit value. Any time the OCLK clock frequency is changed, the OCLK, CCLKn, SYSCLK, and SCLKn clocks exit the frequency change sequence aligned.

1. Read the CGU_STAT register to verify that the CGU_STAT.CLKSALGN bit =0 (clocks aligned).
2. Write the desired CGU_DIV.OSEL value with the CGU_DIV.UPDT bit =1. This write updates the CGU_DIV register, changes the OCLK frequency, and aligns all clocks except OCLK.

The CGU_STAT register exits this sequence with the CGU_STAT.CLKSALGN bit =0. Poll the CGU_STAT.CLKSALGN bit to discover when the clocks are aligned. Any write attempt to change the CGU_DIV.DSEL field while the CGU_STAT.CLKSALGN bit =1 (clock alignment in progress) triggers an MMR access bus error and the CGU_DIV/CGU_DIVEX is not modified. When the clocks are aligned, a CGU event occurs.

Writing to the CGU_DIV.OSEL bit field is allowed while the processor is in active (PLL bypassed) mode. In this case the effect of the write is visible only after the transition to full-on (PLL not bypassed) mode.

Selecting SCLK1_0 Source

SCLK1_0 sources the SPDIF receiver clock. This can be sourced from PLL clock or SYSCLK0_0. Programming the CGU_DIVEX enables PLL clock as source (bypassing SYSCLK0_0 that continues to source other peripherals).

When CGU_DIVEX is enabled, the PLL clock is divided from CGU_DIVEX register value.

Changing SCLK1_0 Frequencies

Complete the following steps to change the SCLK0 or SCLK1 frequency.

1. Read the CGU_STAT register. Verify that CGU_STAT.CLKSALGN = 0 (Clocks aligned).

2. Read the [CGU_CTL](#) register. Verify that the `CGU_CTL.S1SELEXEN` bit is set as needed. (These bits can only be modified when the PLL is bypassed).
3. Read the [CGU_STAT](#) register. Verify that the PLL is not bypassed. (If it is bypassed, see [PLL Bypass and PLL Disable](#) topic.)
4. Read the [CGU_DIV](#) register. Verify that the `CGU_DIV.UPDT` bit is set.
5. Write the desired `CGU_DIVEX.S1SELEX` value to the [CGU_DIVEX](#) registers .

Aligning All Clocks

To align the clocks, write 1 to the `CGU_DIV.ALGN` bit. The frequency can also be changed, if necessary. The clocks aligned include:

- CCLK_n
- SYSCLK
- SCLK_n
- DCLK
- OCLK

1. Read the [CGU_STAT](#) register to verify that `CGU_STAT.CLKSALGN` bit =0 (clocks aligned).
2. Write 1 to the `CGU_DIV.ALGN` bit. All other fields can change.

ADDITIONAL INFORMATION: This write does not alter the [CGU_DIV/CGU_DIVEX](#) register unless one of the clock-select fields is modified. When the clocks are aligned, a CGU event occurs.

The [CGU_STAT](#) register exits this sequence with the `CGU_STAT.CLKSALGN` bit =0. Poll the `CGU_STAT.CLKSALGN` bit to discover when the clocks are aligned. Any write to the [CGU_DIV/CGU_DIVEX](#) register intended to align clocks or to change a clock select field while the `CGU_STAT.CLKSALGN` bit =1 (clocks alignment in progress) triggers an MMR access bus error. In this case, the [CGU_DIV/CGU_DIVEX](#) register is not modified.

Writing 1 to the `CGU_DIV.ALGN` bit has no effect while the processor is in active (PLL bypassed) mode.

Shutting Off CCLK_n from Another Master

CCLK_n can be shut off to save power when it is not in use. In the ADSP-2156x processor, this master is a memory DMA.

1. Disable interrupts to core n.
2. Set the `RCU_SIDIS.SI[n]` bit to disable the interfaces of core n in order to:
 - a. Stop DMA accesses to its L1.

- b. Stop memory accesses to core 0.
 - c. Stop accesses to MMRs.
 3. Test the `RCU_SISTAT.SI[n]` bit to detect when accesses to core n have been disabled and all the pending transactions have completed.
 4. Set the `CGU_CCBF_DIS.CCBF0` bit to disable the CCLKn buffer.
 5. Check the `CGU_CCBF_STAT.CCBF0` bit.
- If the `CGU_CCBF_STAT.CCBF0` bit is set, continue.

Reenable CCLKn From Another Master

1. Clear the `CGU_CCBF_DIS.CCBF0` to enable CCLKn.
2. Check the `CGU_CCBF_STAT.CCBF0` bit.
 - a. If the `CGU_CCBF_DIS.CCBF0` bit is cleared, continue.
3. Clear the `RCU_SIDIS.SI[n]` bit. The core deasserts its acknowledge signal in response to the `RCU_SYSRST0` signal. This operation clears the `RCU_SISTAT.SI[n]` bit.

Valid Clock Multiplier Settings

Processor operations depend on valid settings in the `CGU_CTL` and `CGU_DIV` registers. These registers control the clock multiplier and divisor values. Set these registers such that the minimum and maximum clocks specified in the data sheet are not violated. All other clock specifications in the data sheet must also be adhered to for correct operation of the processor.

PLL Bypass and PLL Disable

Writing 1 to the `CGU_PLLCTL.PLLBPST` bit tells the PLL to apply `OSC_CLKIN` clock to CCLK, SYCLK, SCLK0, SCLK1, DCLK (PLL Bypass), and OCLK outputs. Writing 1 to the `CGU_PLLCTL.PLLBPCL` bit tells the PLL to exit its PLL Bypass state and make all output clocks align and transition to their programmed frequencies.

The PLL can be disabled by clearing the `CGU_PLLCTL.PLLEN` bit while in the bypass state. If necessary, clock buffers can be disabled.

CCLK0 clock can be disabled or enabled by writing 1 or 0 to the corresponding bit in the `CGU_CCBF_DIS` register.

To determine which core clock buffers were disabled since the last read, software reads the `CGU_CCBF_STAT` register. The SCLK0, SCLK1, DCLK and OCLK clocks can be disabled or enabled by writing 1 or 0 to the corresponding bit in the `CGU_SCBF_DIS` register. Software cannot disable SYCLK.

ADSP-2159x Specific Information

The processor has two system crystal oscillators and two system CGU units to provide the clocks to the system. However, the ADSP-21593 and ADSP-21591 processors have one system crystal oscillator. Both of the CGUs come up in bypass mode out of reset.

CGU0 is the main CGU which provides most of the clocks including CCLK0, SYSCLK (SYSCLK_0), SCLK0 (SCLK0_0), and SCLK1 (SCLK1_0) to the system buses, infrastructure, and most of the peripherals. The rest of the clock outputs from the two CGUs can be routed to a specific peripheral in the system. For details, refer to the [Clock Distribution Unit \(CDU\)](#).

NOTE: The frequency ratio of the core clock to SYSCLK is 2:1.

The frequency ratios of SYSCLK to SCLK0 are 6:1, 4:1 or 2:1.

The processor does not support any other frequency ratio.

ADSP-2159x CGU Register Descriptions

Clock Generation Unit (CGU) contains the following registers.

Table 1-8: ADSP-2159x CGU Register List

Name	Description
CGU_CCBF_DIS	Core Clock Buffer Disable Register
CGU_CCBF_STAT	Core Clock Buffer Status Register
CGU_CLKOUTSEL	CLKOUT Select Register
CGU_CTL	Control Register
CGU_DIV	Clocks Divisor Register
CGU_DIVEX	DIV Register Extension
CGU_OSCWDCTL	Oscillator Watchdog Register
CGU_PLLCTL	PLL Control Register
CGU_REVID	Revision ID Register
CGU_SCBF_DIS	System Clock Buffer Disable Register
CGU_SCBF_STAT	System Clock Buffer Status Register
CGU_STAT	Status Register
CGU_TSCOUNT0	Time Stamp Counter 32 LSB Register
CGU_TSCOUNT1	Time Stamp Counter 32 MSB Register
CGU_TSCTL	Time Stamp Control Register
CGU_TSVALUE0	Time Stamp Counter Initial 32 LSB Value Register
CGU_TSVALUE1	Time Stamp Counter Initial MSB Value Register

Core Clock Buffer Disable Register

The `CGU_CCBF_DIS` register controls each core's clock buffer to determine if the CCLK is enabled.

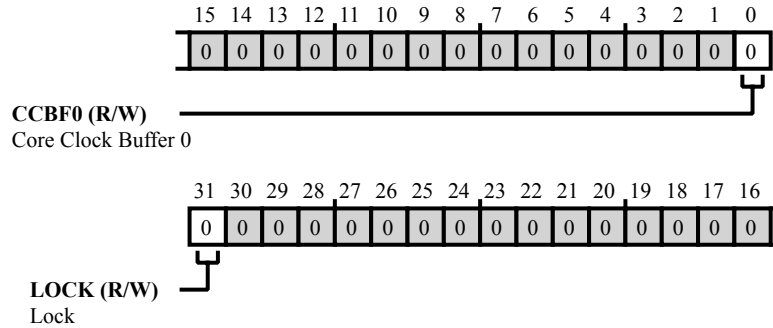


Figure 1-3: `CGU_CCBF_DIS` Register Diagram

Table 1-9: `CGU_CCBF_DIS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If set (=1) the <code>CGU_CCBF_DIS</code> .LOCK bit locks the <code>CGU_CCBF_DIS</code> register.
		0 Unlock register
		1 Lock register
0 (R/W)	CCBF0	Core Clock Buffer 0. The <code>CGU_CCBF_DIS</code> .CCBF0 bit enables (=0) or disables (=1) CCLK0s buffer.
		0 Enable buffer
		1 Disable buffer

Core Clock Buffer Status Register

The `CGU_CCBF_STAT` register shows which core clock buffer(s) are disabled. For example clearing the `CGU_CCBF_DIS.CCBF0` bit clears the `CGU_CCBF_STAT.CCBF0` bit after a number of cycles. To guarantee that the correct value is read, this register should be read twice and the second result used.

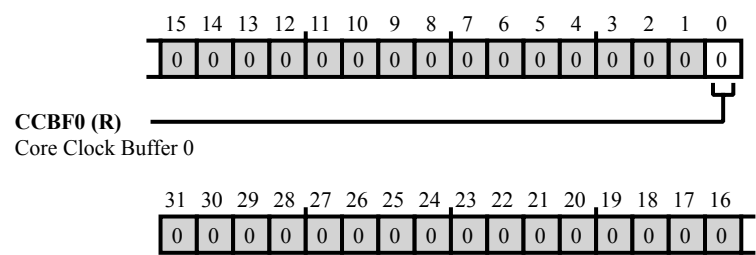


Figure 1-4: `CGU_CCBF_STAT` Register Diagram

Table 1-10: `CGU_CCBF_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	CCBF0	Core Clock Buffer 0.
		The <code>CGU_CCBF_STAT.CCBF0</code> bit reports the status of the <code>CGU_CCBF_DIS.CCBF0</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled

CLKOUT Select Register

The `CGU_CLKOUTSEL` selects the signal that the CGU drives through the CLKOUT multiplexer. Also, this register selects the divisor for the USBCLK output.

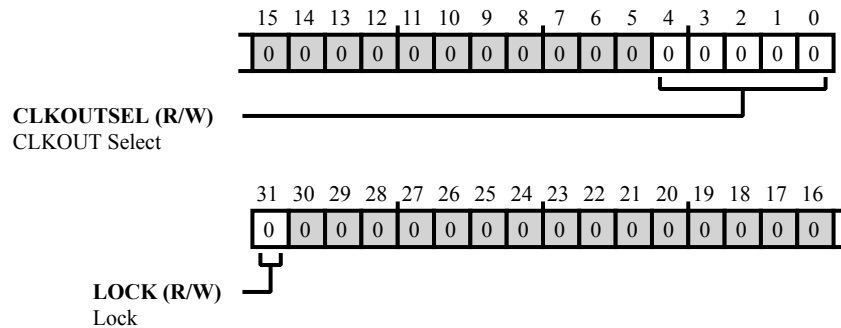


Figure 1-5: CGU_CLKOUTSEL Register Diagram

Table 1-11: CGU_CLKOUTSEL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>CGU_CLKOUTSEL.LOCK</code> bit is set, the <code>CGU_CLKOUTSEL</code> register is read only (locked).
		0 Unlock
		1 Lock
4:0 (R/W)	CLKOUTSEL	CLKOUT Select. The <code>CGU_CLKOUTSEL.CLKOUTSEL</code> selects the signal that the CGU drives through the CLKOUT pin multiplexer.
		0 CLKIN0
		1 CLKIN1
		2 PLLTOP0.SYSCLKO
		3 PLLTOP0.CCLK0
		4 CDU.CLKO3 for DDR
		5 CDU.CLKO4 for CAN
		6 CDU.CLKO7 for EMAC
		7 CDU.CLKO8 for LP
		8 PLLTOP1.SYSCLKO
		9 PLLTOP1.OCLK

Table 1-11: CGU_CLKOUTSEL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		10	PLLTOP0.OCLK
		11	Reserved
		12	Reserved
		13	Reserved
		14	PLLTOP0.SCLK1
		15	Reserved
		16	PLLTOP0.SCLK0
		17	Reserved
		18	Reserved
		19	GND

Control Register

The `CGU_CTL` controls the clock generation divisors for `SYS_CLKIN` and the PLL. Read after write accesses to the `CGU_CTL` register returns the new value even if the clock's frequency change is still in progress.

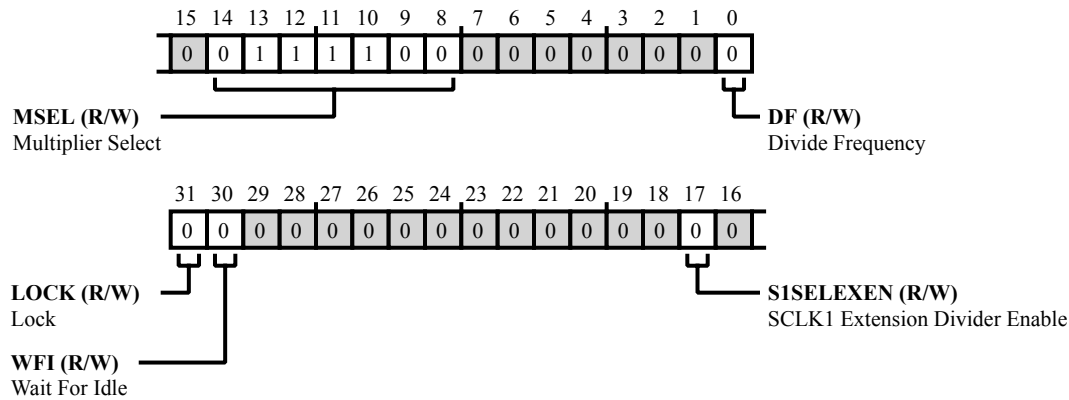


Figure 1-6: CGU_CTL Register Diagram

Table 1-12: CGU_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>CGU_CTL.LOCK</code> bit is set, the <code>CGU_CTL</code> register is read only (locked).
		0 Unlock 1 Lock
30 (R/W)	WFI	Wait For Idle.
		Modifying the PLL multiplier requires the PLL to re-lock and once the PLL locks, clocks have to be synchronized. Changes to the <code>CGU_CTL.MSEL</code> and the <code>CGU_CTL.DF</code> bit values results in bypassing the PLL.
		The <code>CGU_CTL.WFI</code> bit forces the PLL to wait for all processor cores to be in an idle or reset state before changing frequencies as a result of changes to the <code>CGU_CTL.MSEL</code> or <code>CGU_CTL.DF</code> bits. Write accesses to the <code>CGU_CTL</code> to change the <code>CGU_CTL.DF</code> or <code>CGU_CTL.MSEL</code> bit values while the PLL is locking sets the <code>CGU_STAT.WDFMSERR</code> bit.
		0 Update Immediately 1 Wait for Idle

Table 1-12: CGU_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	S1SELEXEN	SCLK1 Extension Divider Enable. The <code>CGU_CTL.S1SELEXEN</code> directs the CGU to select S1SELEX instead of S1SEL. This bit can only be updated when the PLL is bypassed.
		0 Selects SYSCLK to SCLK1 Divider
		1 Selects PLLCLK to SCLK1 Divider
14:8 (R/W)	MSEL	Multiplier Select. The <code>CGU_CTL.MSEL</code> bit field selects the multiplier in the PLLCLK equation: $\text{PLLCLK frequency} = (\text{SYS_CLKIN frequency} / (\text{DF} + 1)) * \text{MSEL}$ Where the value of MSEL is between 1 and 127.
		0 MSEL = 128
		1-127 MSEL = 1 to 127
0 (R/W)	DF	Divide Frequency. The <code>CGU_CTL.DF</code> bit selects whether or not the CLKIN input is divided by two before being passed to the PLL.
		0 Pass OSC_CLKIN to PLL
		1 Pass OSC_CLKIN/2 to PLL

Clocks Divisor Register

The `CGU_DIV` register controls clock divisors for core clocks, system clocks, external (off core) memory clocks, and output clock. Read after write accesses to the `CGU_DIV` register returns the new value even if the clock's frequency change is still in progress.

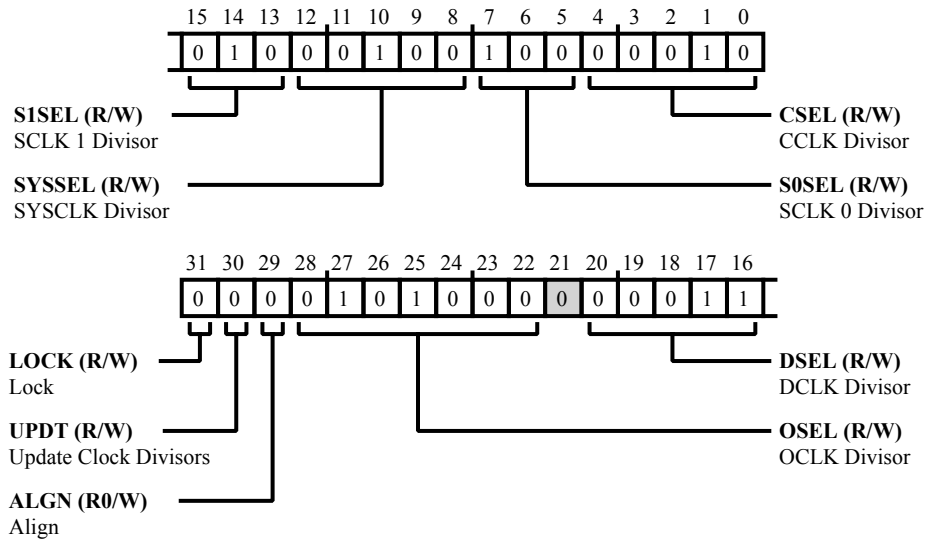


Figure 1-7: CGU_DIV Register Diagram

Table 1-13: CGU_DIV Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>CGU_DIV.LOCK</code> bit is set, the <code>CGU0_DIV</code> and <code>CGU0_DIVEX</code> registers are read only (locked).
		0 Unlock
		1 Lock
30 (R/W)	UPDT	Update Clock Divisors.
		The <code>CGU_DIV.UPDT</code> controls whether the CGU drives new <code>CGU_DIV.CSEL</code> , <code>CGU_DIV.SYSEL</code> , <code>CGU_DIV.S0SEL</code> , <code>CGU_DIV.S1SEL</code> , <code>CGU_DIV.DSEL</code> , and <code>CGU_DIV.OSEL</code> values to PLL after <code>CGU_DIV</code> register update.
		0 No PLL Update
		1 Drive Updated SEL Values to PLL

Table 1-13: CGU_DIV Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R0/W)	ALGN	Align. The CGU_DIV.ALGN directs the CGU to align the PLL-based clocks. The divisor selections (CGU_DIV.CSEL, CGU_DIV.SYSSEL, CGU_DIV.S0SEL, CGU_DIV.S1SEL, CGU_DIV.DSEL, and/or CGU_DIV.OSEL) do not have to change.
		0 No Action
		1 Align PLL Clocks
28:22 (R/W)	OSEL	OCLK Divisor. The CGU_DIV.OSEL selects the divisor in the OCLK equation: $\text{OCLK frequency} = (\text{SYS_CLKIN frequency} / (\text{DF}+1)) * \text{MSEL} / \text{CGU_DIV.OSEL}$ Where the value of CGU_DIV.OSEL is between 1 and 127.
		0 OSEL = 128
		1-127 OSEL = 1 to 127
20:16 (R/W)	DSEL	DCLK Divisor. The CGU_DIV.DSEL selects the divisor in the DCLK equation: $\text{DCLK frequency} = (\text{SYS_CLKIN frequency} / (\text{DF}+1)) \text{MSEL} / \text{CGU_DIV.DSEL}$ Where the value of CGU_DIV.DSEL is between 1 and 31.
		0 DSEL = 32
		1-31 DSEL = 1 to 31
15:13 (R/W)	S1SEL	SCLK 1 Divisor. The CGU_DIV.S1SEL selects the divisor in the SCLK1 equation: $\text{SCLK1 frequency} = (\text{SYSCLK frequency}) / \text{CGU_DIV.S1SEL}$ Where the value of CGU_DIV.S1SEL is between 1 and 7.
		0 S1SEL = 8
		1-7 S1SEL = 1 to 7
12:8 (R/W)	SYSSEL	SYSCLK Divisor. The CGU_DIV.SYSSEL selects the divisor in the SYSCLK equation: $\text{SYSCLK frequency} = (\text{SYS_CLKIN frequency} / (\text{DF}+1)) \text{MSEL} / \text{CGU_DIV.SYSSEL}$ Where the value of CGU_DIV.SYSSEL is between 1 and 31.
		0 SYSSEL = 32
		1-31 SYSSEL = 1 to 31

Table 1-13: CGU_DIV Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:5 (R/W)	S0SEL	SCLK 0 Divisor. The <code>CGU_DIV.S0SEL</code> selects the divisor in the SCLK0 equation: $\text{SCLK0 frequency} = (\text{SYSCLK frequency}) / \text{CGU_DIV.S0SEL}$ Where the value of <code>CGU_DIV.S0SEL</code> is between 1 and 7.
		0 S0SEL = 8
		1-7 S0SEL = 1 to 7
4:0 (R/W)	CSEL	CCLK Divisor. The <code>CGU_DIV.CSEL</code> bit field selects the divisor in the CCLK equation: $\text{CCLK frequency} = (\text{SYS_CLKIN frequency} / (\text{DF}+1)) * \text{MSEL} / \text{CGU_DIV.CSEL}$ Where the value of <code>CGU_DIV.CSEL</code> is between 1 and 31.
		0 CSEL = 32
		1-31 CSEL= 1 to 31

DIV Register Extension

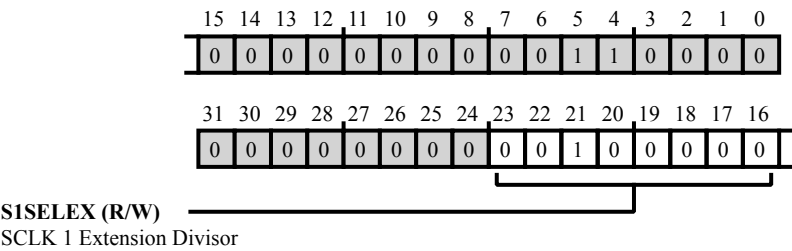


Figure 1-8: CGU_DIVEX Register Diagram

Table 1-14: CGU_DIVEX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	S1SELEX	SCLK 1 Extension Divisor. The CGU_DIVEX.S1SELEX selects the divisor in the SCLK1EX equation: SCLK1EX frequency = (PLLCLK frequency) / CGU_DIVEX.S1SELEX Where the value of CGU_DIVEX.S1SELEX is between 1 and 255.
		0 S1SELEX = 256
		1-255 S1SELEX = 1 to 255

Oscillator Watchdog Register

The `CGU_OSCWDCTL` register configures the CGU to allow the detection of the absence of input clock transitions and provides a fault warning via the `SYS_FAULT` pin. The `CGU_OSCWDCTL` register also detects and reports input oscillator frequencies above and below specified limits, in order to specifically detect harmonic or sub-harmonic crystal oscillator behavior. This detection is achieved by using an internal asynchronous, local 1 MHz oscillator combined with a series of programmable counters.

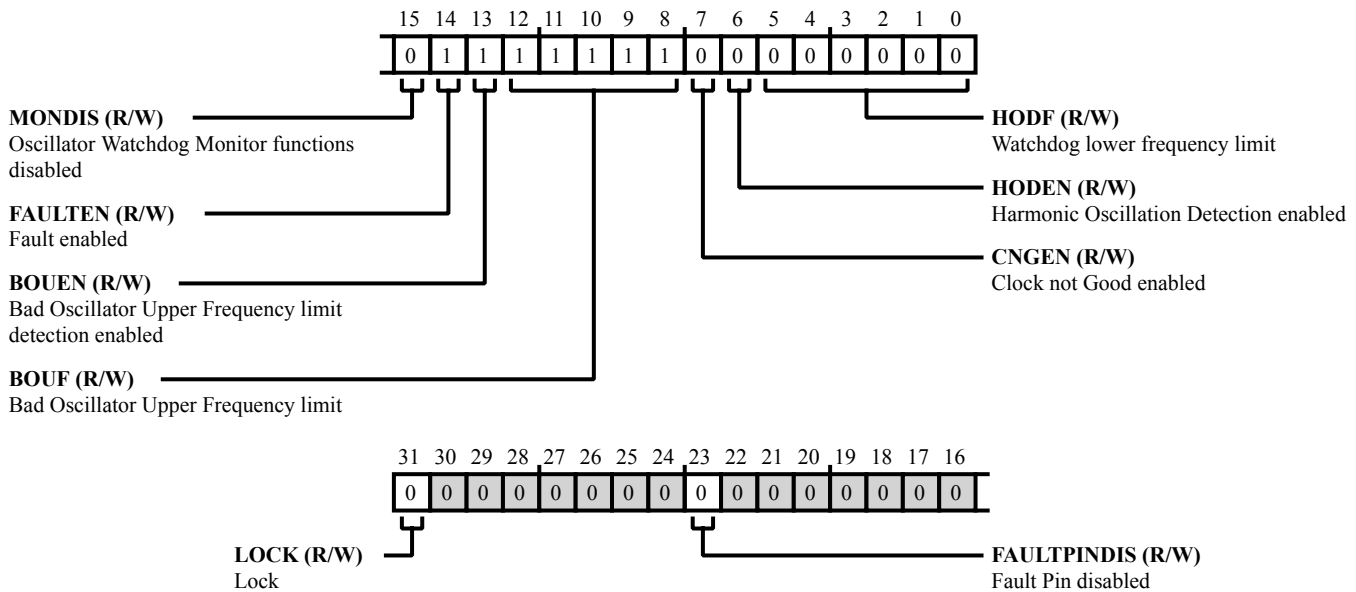


Figure 1-9: `CGU_OSCWDCTL` Register Diagram

Table 1-15: `CGU_OSCWDCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
23 (R/W)	FAULTPINDIS	Fault Pin disabled. The <code>CGU_OSCWDCTL.FAULTPINDIS</code> bit disables pin fault detection.
15 (R/W)	MONDIS	Oscillator Watchdog Monitor functions disabled. The <code>CGU_OSCWDCTL.MONDIS</code> bit disables all the input clock monitor and fault detection functions.
14 (R/W)	FAULTEN	Fault enabled. The <code>CGU_OSCWDCTL.FAULTEN</code> bit enables fault detection.
13 (R/W)	BOUEN	Bad Oscillator Upper Frequency limit detection enabled. The <code>CGU_OSCWDCTL.BOUEN</code> bit enables upper limit bad oscillation detection.

Table 1-15: CGU_OSCWDCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12:8 (R/W)	BOUF	Bad Oscillator Upper Frequency limit. The CGU_OSCWDCTL.BOUF bits indicate the desired upper fail limit for the bad oscillation detection.
7 (R/W)	CNGEN	Clock not Good enabled. The CGU_OSCWDCTL.CNGEN bit enables the detection of an oscillator watchdog clock fault.
6 (R/W)	HODEN	Harmonic Oscillation Detection enabled. The CGU_OSCWDCTL.HODEN bit enables harmonic oscillation detection.
5:0 (R/W)	HODF	Watchdog lower frequency limit. The CGU_OSCWDCTL.HODF bit field is used to indicate the desired lower fail limit for the harmonic oscillation detection in MHz.

PLL Control Register

The `CGU_PLLCTL` register contains bits that enable and disable the PLL as well as control its function.

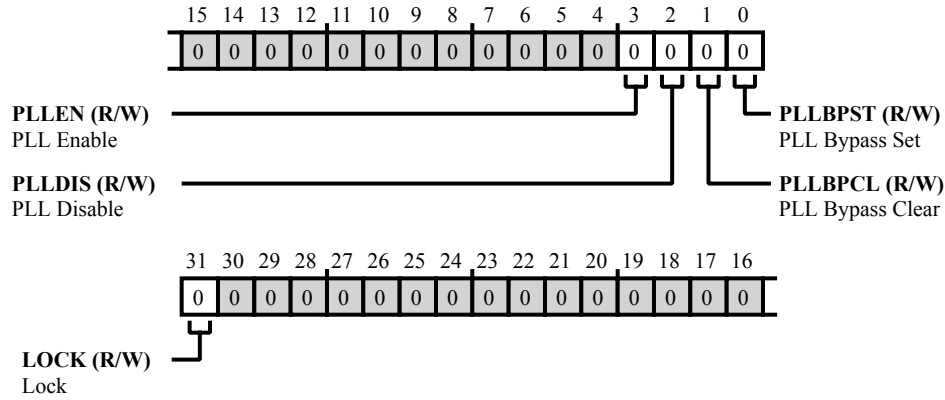


Figure 1-10: `CGU_PLLCTL` Register Diagram

Table 1-16: `CGU_PLLCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. Setting (=1) the <code>CGU_PLLCTL . LOCK</code> bit locks access to the <code>CGU_PLLCTL</code> register.
		0 Unlock register
		1 Lock register
3 (R/W)	PLLEN	PLL Enable. Setting (=1) the <code>CGU_PLLCTL . PLLEN</code> bit enables the PLL. Check the <code>CGU_STAT . PLLEN</code> bit to verify that the action is complete.
		0 No action
		1 Enable PLL
2 (R/W)	PLLDIS	PLL Disable. Setting (=1) the <code>CGU_PLLCTL . PLLDIS</code> bit disables the PLL. Check the <code>CGU_STAT . PLLEN</code> bit to verify that the action is complete.
		0 No action
		1 Disable PLL
1 (R/W)	PLLBPCL	PLL Bypass Clear. Setting (=1) the <code>CGU_PLLCTL . PLLBPCL</code> bit takes the PLL out of bypass mode. Check the <code>CGU_STAT . PLLBP</code> bit to verify that the action is complete.
		0 No action
		1 Exit bypass mode

Table 1-16: CGU_PLLCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	PLLBPST	PLL Bypass Set. Setting (=1) the CGU_PLLCTL.PLLBPST bit bypasses the PLL and all the clocks run on CLKIN. Check the CGU_STAT.PLLBP bit to verify that the action is complete.
		0 Use PLL
		1 Bypass PLL

Revision ID Register

The `CGU_REVID` register reports the version of the CGU.

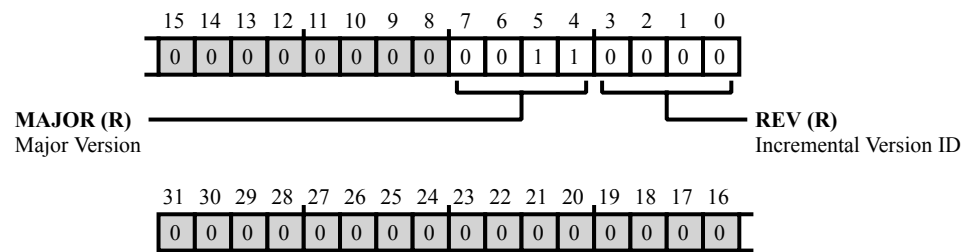


Figure 1-11: CGU_REVID Register Diagram

Table 1-17: CGU_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major Version.
3:0 (R/NW)	REV	Incremental Version ID.

System Clock Buffer Disable Register

The `CGU_SCBF_DIS` register controls each system's clock buffer to determine if the SCLKn buffer is enabled.

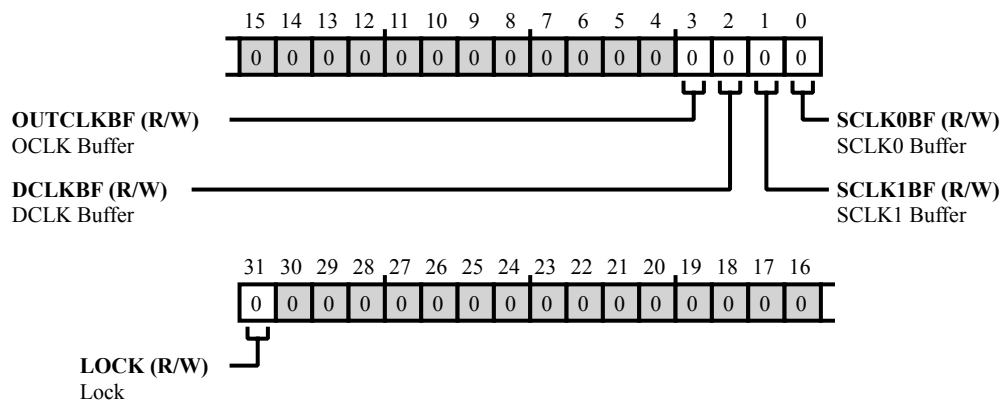


Figure 1-12: `CGU_SCBF_DIS` Register Diagram

Table 1-18: `CGU_SCBF_DIS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. The <code>CGU_SCBF_DIS</code> . <code>LOCK</code> bit allows writes to the <code>CGU_SCBF_DIS</code> register when cleared (=0) or blocks writes if set (=1) and the <code>SPU_CTL</code> . <code>GLCK</code> bit is set.
		0 Unlock register
		1 Lock register
3 (R/W)	OUTCLKBF	OCLK Buffer. The <code>CGU_SCBF_DIS</code> . <code>OUTCLKBF</code> bit enables (=0, default) or disables (=1) OCLKs buffer.
		0 Enable buffer
		1 Disable buffer
2 (R/W)	DCLKBF	DCLK Buffer. The <code>CGU_SCBF_DIS</code> . <code>DCLKBF</code> bit enables (=0, default) or disables (=1) DCLKs buffer.
		0 Enable buffer
		1 Disable buffer

Table 1-18: CGU_SCBF_DIS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R/W)	SCLK1BF	SCLK1 Buffer. The CGU_SCBF_DIS.SCLK1BF bit enables (=0, default) or disables (=1) SCLK1s buffer.	
		0	Enable buffer
		1	Disable buffer
0 (R/W)	SCLK0BF	SCLK0 Buffer. The CGU_SCBF_DIS.SCLK0BF bit enables (=0, default) or disables (=1) SCLK0s buffer.	
		0	Enable buffer
		1	Disable buffer

System Clock Buffer Status Register

The `CGU_SCBF_STAT` register shows which system clock buffer(s) are disabled. For example clearing the `CGU_CCBF_DIS.CCBF0` bit clears the `CGU_SCBF_STAT.SCLK0BF` bit after a number of cycles. To guarantee that the correct value is read, this register should be read twice and the second result used.

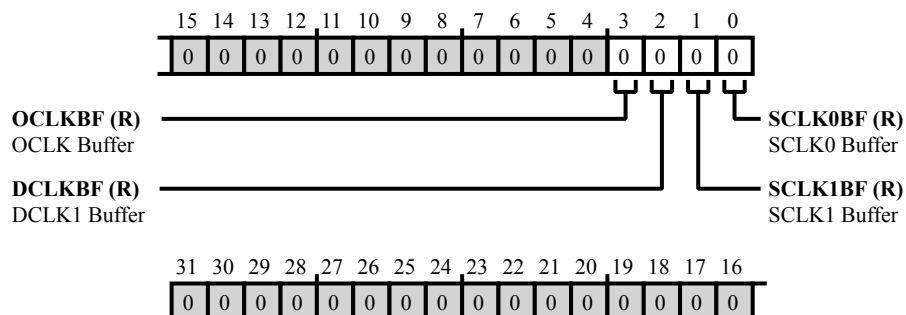


Figure 1-13: `CGU_SCBF_STAT` Register Diagram

Table 1-19: `CGU_SCBF_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/NW)	OCLKBF	OCLK Buffer. The <code>CGU_SCBF_STAT.OCLKBF</code> bit reports the status of the <code>CGU_SCBF_DIS.OUTCLKBF</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled
2 (R/NW)	DCLKBF	DCLK1 Buffer. The <code>CGU_SCBF_STAT.DCLKBF</code> bit reports the status of the <code>CGU_SCBF_DIS.DCLKBF</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled
1 (R/NW)	SCLK1BF	SCLK1 Buffer. The <code>CGU_SCBF_STAT.SCLK1BF</code> bit reports the status of the <code>CGU_SCBF_DIS.SCLK1BF</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled

Table 1-19: CGU_SCBF_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	SCLK0BF	SCLK0 Buffer. The <code>CGU_SCBF_STAT.SCLK0BF</code> bit reports the status of the <code>CGU_SCBF_DIS.SCLK0BF</code> bit where 0 = enabled and 1 = disabled.
		0 Enabled
		1 Disabled

Status Register

The `CGU_STAT` register reflects the PLL status and errors detected during the PLL configuration. This register may be cleared asynchronously by a reset signal from the RCU module. All bits---except those defined as W1C (write-1-to-clear)---are read only.

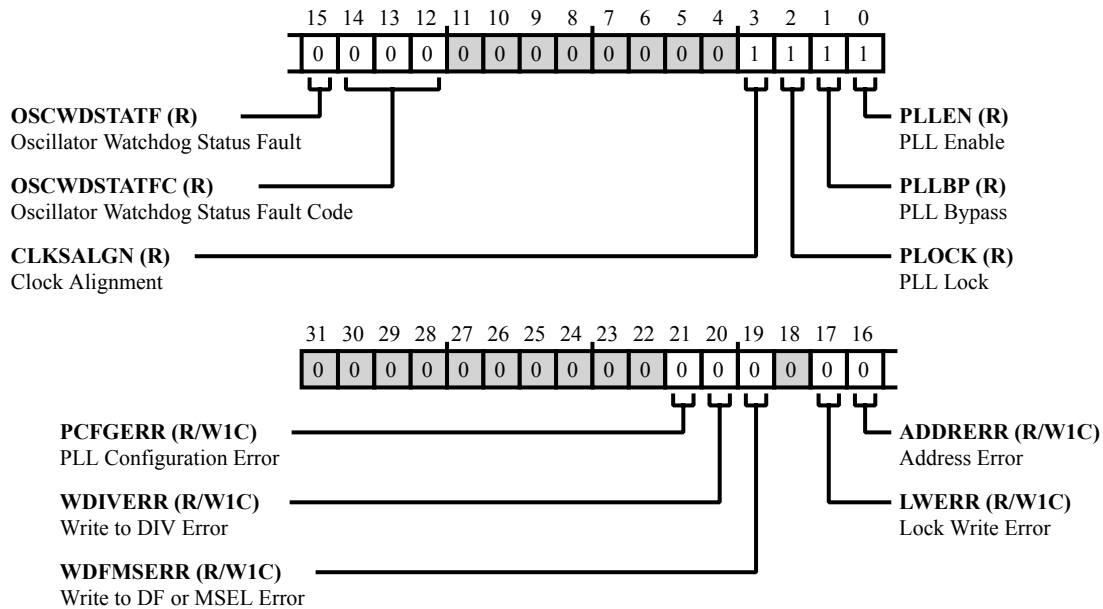


Figure 1-14: CGU_STAT Register Diagram

Table 1-20: CGU_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W1C)	PCFGERR	PLL Configuration Error. If the <code>CGU_PLLCTL.PLLBPST</code> and the <code>CGU_PLLCTL.PLLBPCL</code> bits are set (=1) simultaneously or the <code>CGU_PLLCTL.PLLDIS</code> bit was set (=1) in full-on mode or while trying to enter full-on mode (<code>CGU_PLLCTL.PLLBPCL</code> =1), the <code>CGU_STAT.PCFGERR</code> bit triggers the bus error.
		0 No Error
		1 Configuration Error

Table 1-20: CGU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W1C)	WDIVERR	Write to DIV Error. The CGU_STAT.WDIVERR bit indicates a write access to the CGU_DIV register (to trigger an alignment sequence or to change the CGU_DIV.CSEL, CGU_DIV.SYSSEL, CGU_DIV.S0SEL, CGU_DIV.S1SEL, or CGU_DIV.DSEL bit values) while the PLL is locked, but still aligning the clocks. Read after write accesses to the CGU_STAT and CGU_DIV registers return the new value even if the clock frequency change is still in progress.
		0 No Error
		1 Write DIV Error
19 (R/W1C)	WDFMSERR	Write to DF or MSEL Error. The CGU_STAT.WDFMSERR bit indicates a write access to the CGU_CTL register to change the CGU_CTL.DF or CGU_CTL.MSEL bit values while the PLL is locking.
		0 No Error
		1 Write DF/MSEL Error
17 (R/W1C)	LWERR	Lock Write Error. The CGU_STAT.LWERR bit indicates an attempt to write to write-protected (locked) CGU registers. The CGU issues a bus error for this condition.
		0 No Error
		1 Lock Write Error
16 (R/W1C)	ADDRERR	Address Error. The CGU_STAT.ADDRERR bit indicates an attempt to make a read or write access to unimplemented addresses or accesses are non-aligned. The CGU issues a bus error for this condition.
		0 No Error
		1 Address Error
15 (R/NW)	OSCWDSTATF	Oscillator Watchdog Status Fault. The CGU_STAT.OSCWDSTATF bit indicates a fault in the oscillator watchdog (CGU's OSC_WDSTAT[1:0]) input pins.
		0 No Fault
		1 Fault
14:12 (R/NW)	OSCWDSTATFC	Oscillator Watchdog Status Fault Code. The CGU_STAT.OSCWDSTATFC bit field indicates the nature of the fault in the oscillator watchdog (CGU's OSC_WDSTAT[1:0]) input pins.
		0 No Fault
		1 No Input Clock

Table 1-20: CGU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		2	Subharmonic CLKIN
		3	Harmonic CLKIN
		4	No AUX_CLK
		5	CLKIN > Upper Frequency Limit (BOUF)
		6	Reserved
		7	Multiple Limit Faults
3 (R/NW)	CLKSALGN	Clock Alignment. The CGU_STAT.CLKSALGN bit indicates whether a clock alignment sequence is in progress. This bit is set when clocks alignment is required by changes to CGU_DIV.CSEL, CGU_DIV.S0SEL, CGU_DIV.S1SEL, CGU_DIV.DSEL, or CGU_DIV.OSEL. The CGU_STAT.CLKSALGN bit is cleared when clocks are aligned. Note that (after a PLL frequency change in active state) the CGU_STAT.CLKSALGN bit may indicate that clocks are not aligned even though the clocks are aligned (all clocks are aligned and running at CLKIN frequency).	
		0	Clocks are Aligned
		1	Clocks not Aligned (alignment in progress)
2 (R/NW)	PLOCK	PLL Lock. The CGU_STAT.PLOCK bit indicates whether the PLL is locked. This bit is set when the PLL locks (PLL lock counter end-of-count). The CGU_STAT.PLOCK bit is cleared when requested PLL frequency change (for PLL reset, PLL disable-to-enable transition, or a change to the CGU_CTL.MSEL or CGU_CTL.DF values) is in progress.	
		0	PLL not Locked (PLL frequency change in progress)
		1	PLL Locked
1 (R/NW)	PLLBP	PLL Bypass. The CGU_STAT.PLLBP bit indicates whether the PLL is bypassed. The default value for the CGU_STAT.PLLBP bit is determined by the bypass strap pin.	
		0	PLL not Bypassed
		1	PLL Bypassed
0 (R/NW)	PLLEN	PLL Enable. The CGU_STAT.PLLEN bit indicates whether the PLL is enabled.	
		0	Disabled
		1	Enabled

Time Stamp Counter 32 LSB Register

The `CGU_TSCOUNT0` register address is used to read the CoreSight time stamp counter LSB 32-bit (bits [31:0]) value.

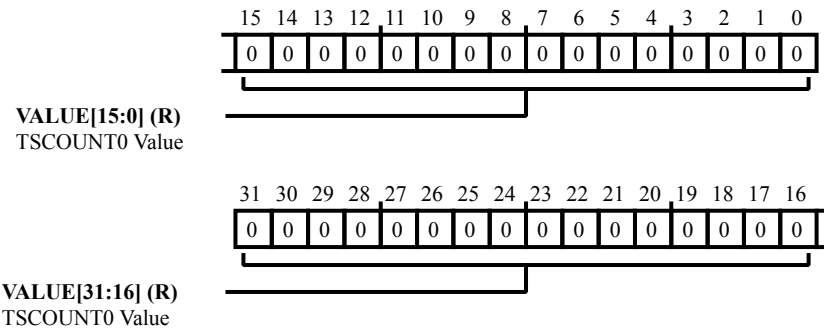


Figure 1-15: CGU_TSCOUNT0 Register Diagram

Table 1-21: CGU_TSCOUNT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	TSCOUNT0 Value. The <code>CGU_TSCOUNT0.VALUE</code> bit field holds the time stamp counter 32 LSBs.

Time Stamp Counter 32 MSB Register

The `CGU_TSCOUNT1` register address is used to read the CoreSight time stamp counter MSB 32-bit (bits [63:32]) value.

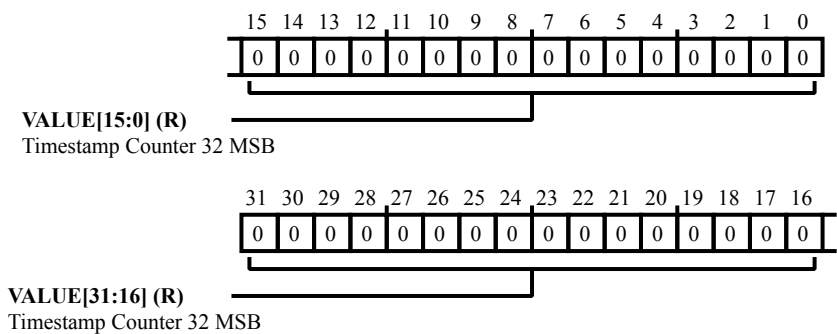


Figure 1-16: CGU_TSCOUNT1 Register Diagram

Table 1-22: CGU_TSCOUNT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Timestamp Counter 32 MSB. The <code>CGU_TSCOUNT1.VALUE</code> bit field holds the time stamp counter 32 MSBs.

Time Stamp Control Register

The `CGU_TSCTL` register controls the operation of the CoreSight time stamp counter.

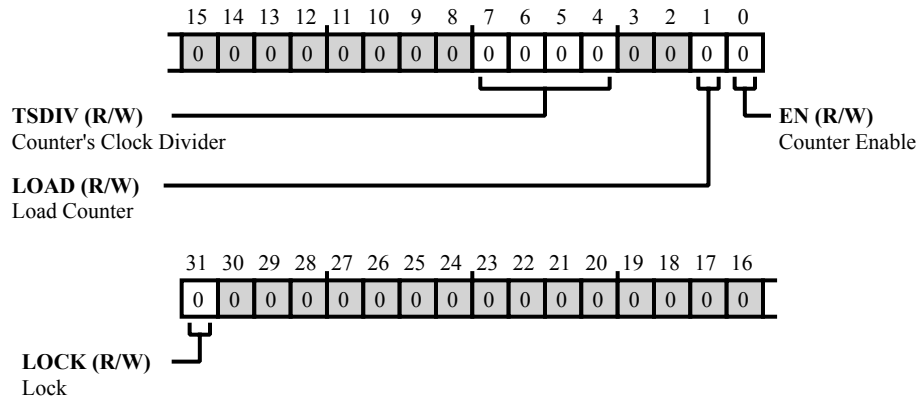


Figure 1-17: `CGU_TSCTL` Register Diagram

Table 1-23: `CGU_TSCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. Setting the <code>CGU_TSCTL.LOCK</code> bit locks this register.
		0 Unlock
		1 Lock
7:4 (R/W)	TSDIV	Counter's Clock Divider. The <code>CGU_TSCTL.TSDIV</code> bit field divides <code>SYSCCLK</code> by 2^{TSDIV} .
		0-15 Divides <code>SYSCCLK</code> by 2^{TSDIV}
1 (R/W)	LOAD	Load Counter. Writing one to the <code>CGU_TSCTL.LOAD</code> bit causes CoreSight time stamp counter to be loaded from the <code>CGU_TSVALUE0</code> and <code>CGU_TSVALUE1</code> registers.
		0 Always read as "0"
0 (R/W)	EN	Counter Enable. The <code>CGU_TSCTL.EN</code> bit enables or disables the CoreSight time stamp counter.
		0 Counter Disabled
		1 Counter Enabled

Time Stamp Counter Initial 32 LSB Value Register

The `CGU_TSVALUE0` register holds the least significant bits (bits [31:0]) value that is initially loaded to the CoreSight time stamp counter.

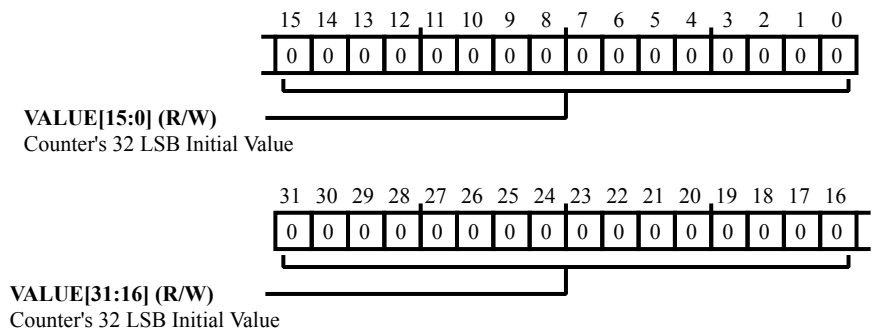


Figure 1-18: CGU_TSVALUE0 Register Diagram

Table 1-24: CGU_TSVALUE0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Counter's 32 LSB Initial Value. The <code>CGU_TSVALUE0.VALUE</code> bit field holds the LSBs value that is initially loaded to the CoreSight time stamp counter.

Time Stamp Counter Initial MSB Value Register

The `CGU_TSVALUE1` register holds the most significant bits (bits [63:32]) value that is initially loaded to the CoreSight time stamp counter.

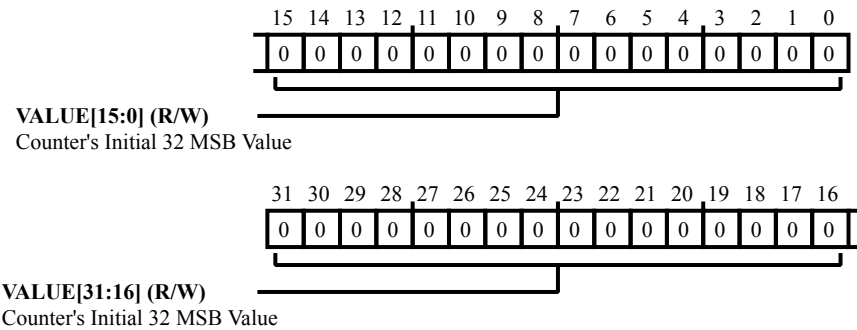


Figure 1-19: CGU_TSVALUE1 Register Diagram

Table 1-25: CGU_TSVALUE1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Counter's Initial 32 MSB Value. The <code>CGU_TSVALUE1.VALUE</code> bit field holds the MSBs value that is initially loaded to the CoreSight time stamp counter.

2 Clock Distribution Unit (CDU)

The Clock Distribution Unit (CDU) consists of an array of multiplexors that select clocks originated from up to four different clock sources. These sources are different clocks that are generated from the CGUs. The multiplexors are configured by software. All clocks multiplexors have up to four different sources. Unused input clocks are grounded internally and never selected. The output clock signal for each multiplexor is assigned to one or more destinations within the processor.

CDU Features

The CDU modules supports the following features:

- Generation of up to 13 output clocks
- Output clock buffers that can be disabled by software
- Each multiplexor has four input clocks
- Multiplexors that are configured by writing to configuration registers ([CDU_CFG\[n\]](#))
- Clocks originated in CGU0 that are selected by default
- A CDU status register ([CDU_STAT](#)) that indicates a configuration change is in-progress

CDU Functional Description

The CDU functions as a set of software-configurable multiplexors that select clocks from different sources.

CDU Block Diagram

The *CDU Block Diagram* figure shows the functional blocks within the CDU. As shown in the figure, the CDU takes different clocks generated by the CGU blocks and provides flexibility to route any clock from possible options to the output CDU clocks. The output clocks from the CDU are connected to specific targets such as the DDR module. All of the targets are clocked through the CDU output clocks rather than being directly clocked by CGU clocks.

This configuration provides the flexibility to meet the specific clock requirements of the different modules in the system (such as the core, DDR, or SPI module) without compromising the clocking of other modules. Such a flexibility is not possible with a single CGU in the system. With two CGUs:

- Each CGU has its own set of clock multipliers and dividers, providing a greater number of orthogonal clocks than possible with a single CGU.
- The `CDU_CLKINSEL.CGU1` bit determines the input for CGU1.

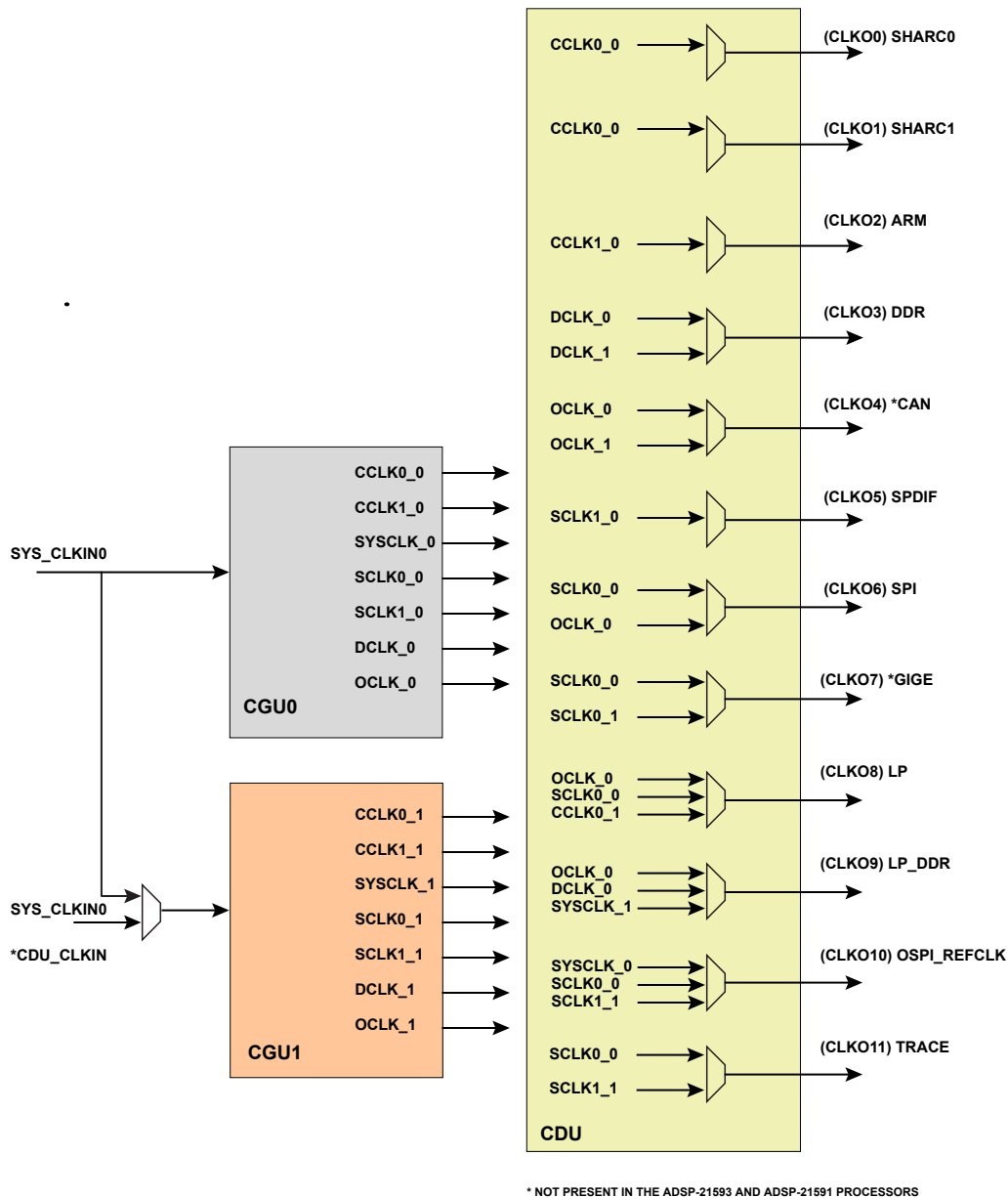


Figure 2-1: CGU/CDU Block Diagram

The DDR is clocked from the DCLK (DCLK_0 and DCLK_1) from CGU0 or CGU1. This configuration provides the flexibility to program the DDR with frequencies orthogonal to the core clock frequencies.

CDU Definitions

The *Clock Descriptions* table provides a brief description of the clocks supported by the processor.

Table 2-1: Clock Descriptions

Clock	Description
CCLK0_0	CCLK0 derived from CGU0
CCLK1_0	CCLK1 derived from CGU0
SYSCLK_0	SYSCLK derived from CGU0
SCLK0_0	SCLK0 derived from CGU0
SCLK1_0	SCLK1 derived from CGU0
DCLK_0	DCLK derived from CGU0
OCLK_0	OCLK derived from CGU0
CCLK0_1	CCLK0 derived from CGU1
SYSCLK_1	SYSCLK derived from CGU1
SCLK0_1	SCLK0 derived from CGU1
SCLK1_1	SCLK1 derived from CGU1
DCLK_1	DCLK derived from CGU1
OCLK_1	OCLK derived from CGU1
CDU_CLKOn	Clocks that come out of the CDU module and go to different blocks

CDU Clock Configuration Options

The *CDU Targets* table provides information about clock source and destination options.

Table 2-2: CDU Targets

CDU0 Input				CDU0 Output	Target
IN0_CLKOn	IN1_CLKOn	IN2_CLKOn	IN3_CLKOn		
CCLK0_0	N/A	N/A	N/A	CLKO0	SHARC0 and corresponding accerators
CCLK0_0	N/A	N/A	N/A	CLKO1	SHARC1 and corresponding accerators
CCLK1_0	N/A	N/A	N/A	CLKO2	ARM
DCLK_0	DCLK_1	N/A	N/A	CLKO3	DDR

Table 2-2: CDU Targets (Continued)

CDU0 Input				CDU0 Output	Target
IN0_CLKOn	IN1_CLKOn	IN2_CLKOn	IN3_CLKOn		
OCLK_0	OCLK_1	N/A	N/A	CLKO4**1	CAN
SCLK1_0	N/A	N/A	N/A	CLKO5	SPDIF
SCLK0_0	OCLK_0	N/A	N/A	CLKO6	SPI
SCLK0_0	SCLK0_1	N/A	N/A	CLKO7*	GIGE
OCLK_0	SCLK0_0	CCLK0_1	N/A	CLKO8	LP
OCLK_0	DCLK_0	SYCLK_1	N/A	CLKO9	LP_DDR
SYCLK_0	SCLK0_0	SCLK1_1	N/A	CLKO10	OSPI_REFCLK
SCLK0_0	SCLK1_1	N/A	N/A	CLKO12	TRACE

*1 Clock is not present in the ADSP-21591 and ADSP-21593 processors

CDU Programming Model

The `CDU_CFG[n]` registers are a system configuration resource. These registers are accessed by a system configuration routine that also handles the configuration of other system modules. Writes to a `CDU_CFG[n]` register must occur when there is not a `CDU_CLKOn` configuration change in progress. If a CGUn inputs are selected, that CGUn configuration must be completed first.

Changing the PLL and Clock Frequency

- Read the `CGU_STAT` register. Verify that:
 - `CGU_STAT.PLLEN = 1` (enabled)
 - `CGU_STAT.PLOCK = 1` (PLL is not locking)
 - `CGU_STAT.CLKSALGN = 0` (clocks aligned)
- Write the desired values into the `CGU_DIV.CSEL`, `CGU_DIV.S0SEL`, `CGU_DIV.SYSEL`, `CGU_DIV.S1SEL`, `CGU_DIV.DSEL` and `CGU_DIV.OSEL` bits with the `CGU_DIV.UPDT` bit cleared (= 0).
- Write the desired values to the `CGU_CTL.DF` and `CGU_CTL.MSEL` bits.
 - To change the PLL frequency while all cores are idle, set the `CGU_CTL.WFI` bit (=1).
 - To change the PLL frequency while the cores are active, clear the `CGU_CTL.WFI` bit (=0).
- Read the `CGU_STAT` register. Verify that:
 - `CGU_STAT.PLLEN = 1` (enabled)

- `CGU_STAT.PLOCK = 1` (not locking)
 - `CGU_STAT.CLKSALGN = 0` (clocks aligned)
5. If clocks switch from CGUm clocks to CGUn input clocks, read the `CGU_SCBF_STAT` and `CGU_CCBF_STAT` registers. The `CGU_CCBF_STAT[1:0]` bit field corresponds to the CCLK1 and CCLK0 clock buffers, respectively. The `CGU_SCBF_STAT[3:0]` bit field corresponds to the OUTCLK, DCLK, SCLK1, and SCLK0 clock buffers, respectively.
 6. Read the `CDU_STAT` register. Verify that the `CDU_STAT.CLK00` (no CLKOn configuration change in progress).
 7. Write to the `CDU_CLKINSEL` register to select the CGU's CLKIN input clock.
 8. Write to the `CDU_CFG[n].SEL` bit to select the clock source (the `CDU_CFG[n].EN` bit should =1.)
 9. Read the `CDU_CFG[n]` register. Verify that the `CDU_CLKINSEL` has the programmed value.

CLKOn is reconfigured.

Changing the Clock Frequency

Use the following procedure to change a clock frequency.

1. Read the `CGU_STAT` register to verify that the `CGU_STAT.CLKSALGN` bit =0 (clocks aligned).
2. Write the desired values into the `CGU_DIV.CSEL`, `CGU_DIV.S0SEL`, `CGU_DIV.SYSSEL`, `CGU_DIV.S1SEL`, `CGU_DIV.DSEL` and `CGU_DIV.OSEL` bits with the `CGU_DIV.UPDT` bit = 1.
3. Read the `CGU_DIV` registers to verify that the `CGU_DIV.CSEL`, `CGU_DIV.S0SEL`, `CGU_DIV.SYSSEL`, `CGU_DIV.S1SEL`, `CGU_DIV.DSEL` and `CGU_DIV.OSEL` bit values are correct.
4. Read the `CGU_STAT` register to verify that the `CGU_STAT.CLKSALGN = 0` (clocks aligned).
5. If clocks switch from CGUm clocks to CGUn input clocks, read the `CGU_SCBF_STAT` and `CGU_CCBF_STAT` registers. The `CGU_CCBF_STAT[1:0]` bit field corresponds to the CCLK1 and CCLK0 clock buffers, respectively. The `CGU_SCBF_STAT[3:0]` bit field corresponds to the OUTCLK, DCLK, SCLK1, and SCLK0 clock buffers, respectively.
6. Read the `CDU_STAT` register.
7. Write to the `CDU_CFG[n].SEL` bit to select the clock source (`CDU_CFG[n].EN = 1`).
8. Read the `CDU_CFG[n]` register. Verify that the `CDU_CLKINSEL` has the programmed value.
9. Verify that the `CDU_STAT.CLK00 = 0`.

ADSP-2159x CDU Register Descriptions

Clock Distribution Unit (CDU) contains the following registers.

Table 2-3: ADSP-2159x CDU Register List

Name	Description
CDU_CFG[n]	CDU Configuration
CDU_CLKINSEL	CLKIN Select
CDU_REVID	CDU Revision ID
CDU_STAT	CDU Status

CDU Configuration

The `CDU_CFG[n]` registers control the configuration of the clock multiplexors. `CDU0_CFG[n]` corresponds to output clock `CDU_CLKO[n]`.

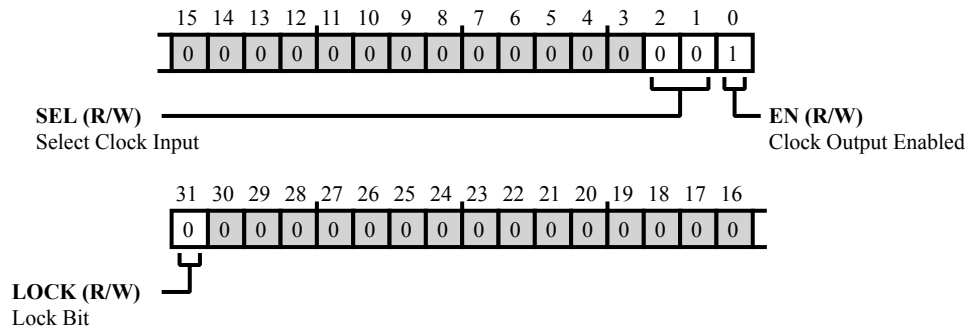


Figure 2-2: `CDU_CFG[n]` Register Diagram

Table 2-4: `CDU_CFG[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit.
2:1 (R/W)	SEL	Select Clock Input.
		0 IN0_CLKOn Selected
		1 IN1_CLKOn Selected
		2 IN2_CLKOn Selected
		3 IN3_CLKOn Selected
0 (R/W)	EN	Clock Output Enabled. The <code>CDU_CFG[n]</code> .EN bit enables clock output.

CLKIN Select

The `CDU_CLKINSEL` register controls the configuration of the CLKIN multiplexors. One bit is assigned to each CGU in the system. This bit selects either CLKIN0 or CLKINn CGUn inputs.

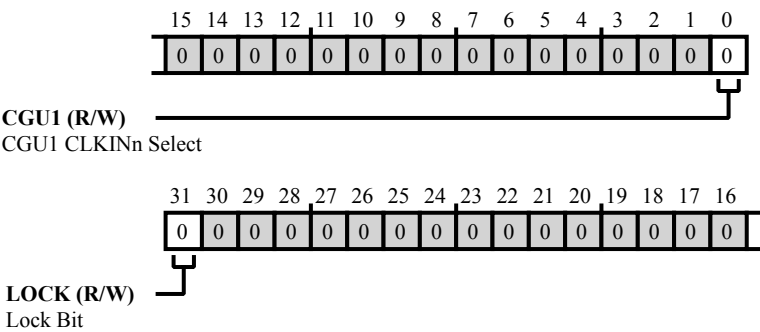


Figure 2-3: CDU_CLKINSEL Register Diagram

Table 2-5: CDU_CLKINSEL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit.
0 (R/W)	CGU1	CGU1 CLKINn Select. The <code>CDU_CLKINSEL.CGU1</code> bit drives <code>CDU_CLKIN_SEL[0]</code> to CGU1.
		0 Selects CLKIN0
		1 Selects CLKIN1

CDU Revision ID

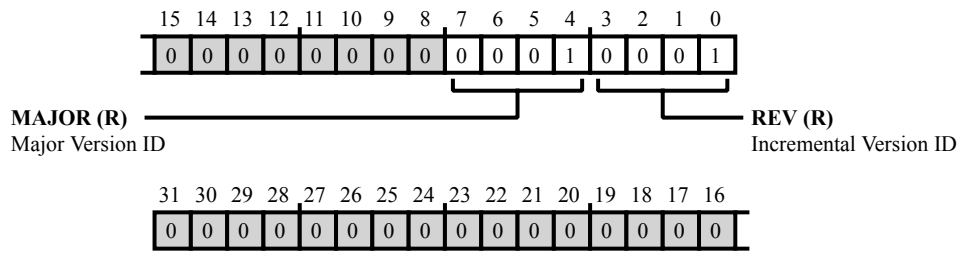


Figure 2-4: CDU_REVID Register Diagram

Table 2-6: CDU_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major Version ID.
3:0 (R/NW)	REV	Incremental Version ID.

CDU Status

The `CDU_STAT` register reflects the status a change in the configuration of the clock muxes.

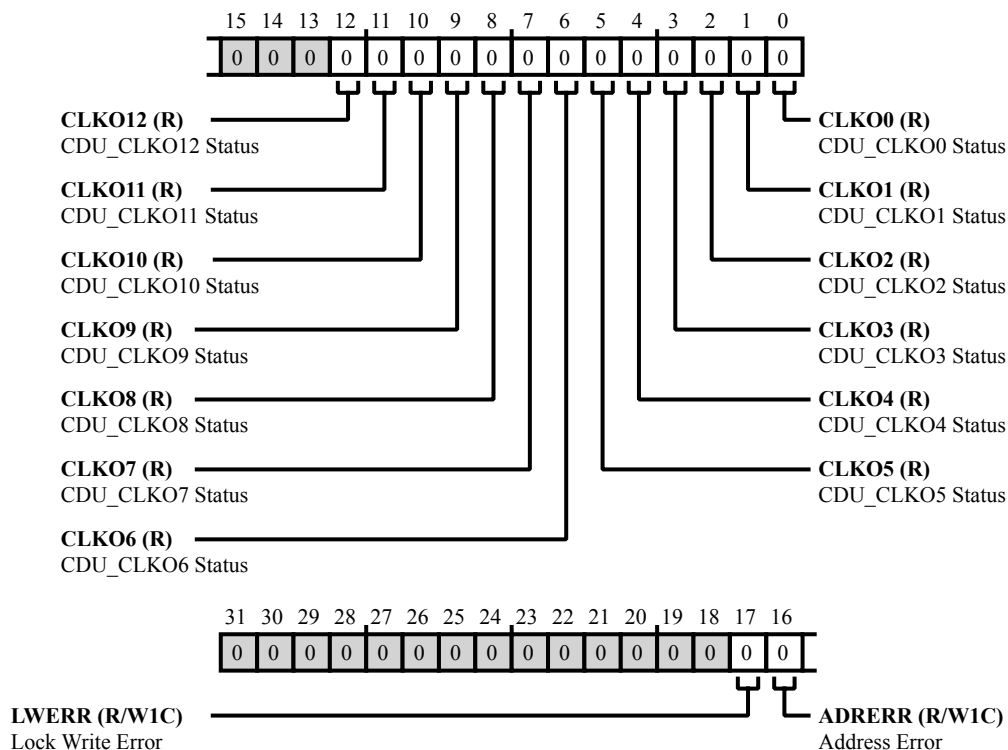


Figure 2-5: CDU_STAT Register Diagram

Table 2-7: CDU_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W1C)	LWERR	Lock Write Error. The <code>CDU_STAT.LWERR</code> bit indicates a lock error where write transactions try to access write protected registers.
		0 No Lock Write Error
		1 Lock Write Error
16 (R/W1C)	ADDRERR	Address Error. The <code>CDU_STAT.ADDRERR</code> bit indicates an address error where read or write transactions try to access unimplemented addresses or accesses are non-aligned.
		0 No Address Error
		1 Address Error

Table 2-7: CDU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/NW)	CLKO12	CDU_CLKO12 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
11 (R/NW)	CLKO11	CDU_CLKO11 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
10 (R/NW)	CLKO10	CDU_CLKO10 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
9 (R/NW)	CLKO9	CDU_CLKO9 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
8 (R/NW)	CLKO8	CDU_CLKO8 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
7 (R/NW)	CLKO7	CDU_CLKO7 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
6 (R/NW)	CLKO6	CDU_CLKO6 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
5 (R/NW)	CLKO5	CDU_CLKO5 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
4 (R/NW)	CLKO4	CDU_CLKO4 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
3 (R/NW)	CLKO3	CDU_CLKO3 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress

Table 2-7: CDU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/NW)	CLKO2	CDU_CLKO2 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
1 (R/NW)	CLKO1	CDU_CLKO1 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress
0 (R/NW)	CLKO0	CDU_CLKO0 Status.	
		0	No Configuration Change in Progress
		1	Configuration Change in Progress

3 Dynamic Power Management (DPM)

The dynamic power management (DPM) unit of the processor controls transitions between different power-saving modes.

DPM Features

The DPM allows programs to control the power mode of the processor as follows.

- Permits operation of multiple, external wake-up sources

DPM Functional Description

The DPM can be programmed to transition between power modes.

ADSP-2159x DPM Register List

A set of registers govern DPM operations. For more information on DPM functionality, see the DPM register descriptions.

Table 3-1: ADSP-2159x DPM Register List

Name	Description
DPM_CTL	Control Register
DPM_PER_DIS0	Peripherals Disable Register 0
DPM_PER_DIS1	Peripherals Disable Register 1
DPM_REVID	Revision ID
DPM_STAT	Status Register

DPM Definitions

To make the best use of the DPM, it is useful to understand the following terms.

CGU

Acronym for the clock generation unit (CGU), which is comprised of the PLL and PCU

DPM

Acronym for the dynamic power management (DPM) controller.

Full-on mode

The normal operating mode in which all clock domains are derived from the PLL.

PCU

Acronym for the PLL control unit (PCU).

PLL

Acronym for the phase-locked loop (PLL).

RCU

Acronym for the reset control unit (RCU).

DPM Operating Modes

The DPM includes several operating modes. The modes are:

- Reset
- Full-on

Reset State

Reset is the initial state of the processor and is the result of a hardware or software triggered event. The DPM itself does not trigger entering reset. The external `SYS_HWRST` pin or the RCU triggers entering reset. The DPM responds to reset by transitioning to its default state.

From Reset, the DPM always transitions to PLL Bypassed state.

Full-on Mode

Full-on mode is the default state of the DPM after Reset.

In full-on mode, the processor can reach its maximum clock rate and power dissipation can be at its highest.

DPM Event Control

The DPM event is triggered when an enabled wake-up is asserted. The DPM generates bus errors when a misaligned access to a register occurs. It also generates errors when an attempt is made to access unused DPM address space or a write-protected register.

DPM Events

The DPM event interrupt is triggered when any bit in the [DPM_STAT](#) register is set, indicating that an enabled wake-up was asserted. The DPM event interrupt stays active until the user clears any bits that are set in the [DPM_STAT](#) register.

DPM Errors

The DPM generates a bus error when a read or write transaction is attempted to an unused address within the DPM address range. It also generates a bus error when a misaligned access is made to a DPM register. In addition to the bus error, the DPM sets the `DPM_STAT.ADDRERR` bit.

If a write to a write-protected DPM register is attempted, the DPM generates a bus error. In addition, the DPM sets the `DPM_STAT.LWERR` bit.

DPM Programming Model

The *DPM_PER_DIS0 Register Mapping* and *DPM_PER_DIS01 Register Mapping* tables show the module clocks and the corresponding peripheral. The [DPM_PER_DIS0](#) and [DPM_PER_DIS0](#) registers are used to shut off the clock to each peripheral if it is not required by the application.

Table 3-2: DPM0_PER_DIS0 Register Mapping

Peripheral Name	Gated Module Clocks	Type of sync on PER-DIS bit	Effect latency (In cycles of module clocks)	DPM0_PER_DISn bit
FIR0/FFT	cclk	Not Required	1	0
IIR0	cclk	Not Required	1	1
DAI0	sclk0	Single Flop	2	5
	sclk1	Single Flop	2	
	sysclk	Not Required	1	
DAI1	sclk0	Single Flop	2	6
	sclk1	Single Flop	2	
	sysclk	Not Required	1	
MLB0	sysclk	Not Required	1	9
DLMDMA0	sysclk	Not Required	1	13
DLMDMA1	sysclk	Not Required	1	14

Table 3-2: DPM0_PER_DIS0 Register Mapping (Continued)

Peripheral Name	Gated Module Clocks	Type of sync on PER-DIS bit	Effect latency (In cycles of module clocks)	DPM0_PER_DISn bit
CRYPTO ACCELERATOR-0 (EIP-150/PKP)	sysclk	Not Required	1	16
CRYPTO ACCELERATOR-1 (EIP-93/SPE)	sysclk	Not Required	1	17
SMPU-SPIF	sysclk	Not Required	1	19
SMPU-L2CTL-CL2-0	sysclk	Not Required	1	20
SMPU-L2CTL-DL2-0	sysclk	Not Required	1	21
SMPU-DMC0	sysclk	Not Required	1	27

Table 3-3: DPM0_PER_DIS1 Register Mapping

Peripheral Name	Gated Module Clocks	Type of sync on PER-DIS bit	Effect latency (In cycles of module clocks)	DPM0_PER_DISn bit
TWI0	sclk0	Single Flop	2	0
TWI1	sclk0	Single Flop	2	1
TWI2	sclk0	Single Flop	2	2
TWI3	sclk0	Single Flop	2	3
TWI4	sclk0	Single Flop	2	4
TWI5	sclk0	Single Flop	2	5
OSPI	sysclk	Not Required	1	27

ADSP-2159x DPM Register Descriptions

Dynamic Power Management (DPM) contains the following registers.

Table 3-4: ADSP-2159x DPM Register List

Name	Description
DPM_CTL	Control Register
DPM_PER_DIS0	Peripherals Disable Register 0
DPM_PER_DIS1	Peripherals Disable Register 1
DPM_REVID	Revision ID
DPM_STAT	Status Register

Control Register

The [DPM_CTL](#) register controls sleep modes selections and PLL operations of the DPM. A write protect feature permits locking out changes to this register.

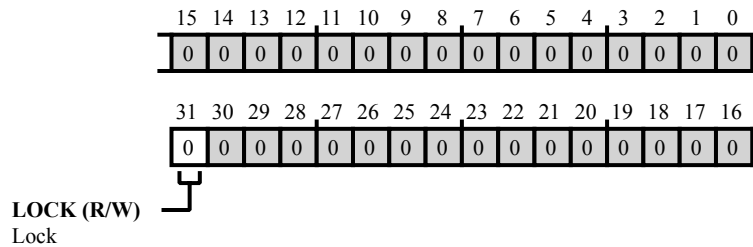


Figure 3-1: DPM_CTL Register Diagram

Table 3-5: DPM_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (SPU_CTL.GLCK bit =1) and the DPM_CTL.LOCK bit is set, the DPM_CTL register is read only (locked).
		0 Unlock
		1 Lock

Peripherals Disable Register 0

The `DPM_PER_DIS0` register is used to shut off the clocks to peripherals that are not used in an application in order to save clock switching power.

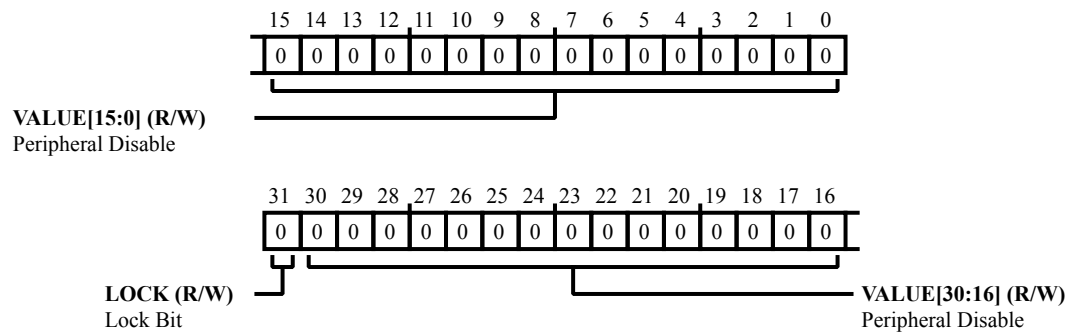


Figure 3-2: DPM_PER_DIS0 Register Diagram

Table 3-6: DPM_PER_DIS0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit.
30:0 (R/W)	VALUE	Peripheral Disable. The <code>DPM_PER_DIS0.VALUE</code> bits are used to shut of clocks in individual peripherals.
	0	SH0_FIR0
	1	SH0_IIR0
	2	SH0_IIR1
	3	SH0_IIR2
	4	SH0_IIR3
	5	DAI0
	6	DAI1
	7	Reserved
	8	Reserved
	9	MLB0
	10	EMAC0 (GigE)
	11	EMAC1(10/100)
	12	Reserved
	13	EMDMA0/CH1

Table 3-6: DPM_PER_DIS0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		14 EMDMA0/CH0
		15 Reserved
		16 CRYPTO ACCELERATOR-0 (EIP-150/PKP)
		17 CRYPTO ACCELERATOR-1 (EIP-150/PKP)
		18 CRYPTO ACCELERATOR-0 (EIP-150/PKP)
		19 SMPU-SPI2
		20 SMPU-L2CTL-CL2-0
		21 SMPU-L2CTL-DL2-0
		22 SMPU-L2CTL-CL2-1
		23 SMPU-L2CTL-DL2-1
		24 SMPU-L2CTL-CL2-2
		25 Reserved
		26 Reserved
		27 SMPU-DMC0
		28 Reserved
		29 CAN0
		30 CAN1

Peripherals Disable Register 1

The `DPM_PER_DIS1` register is used to shut off the clocks to peripherals that are not used in an application to save clock switching power.

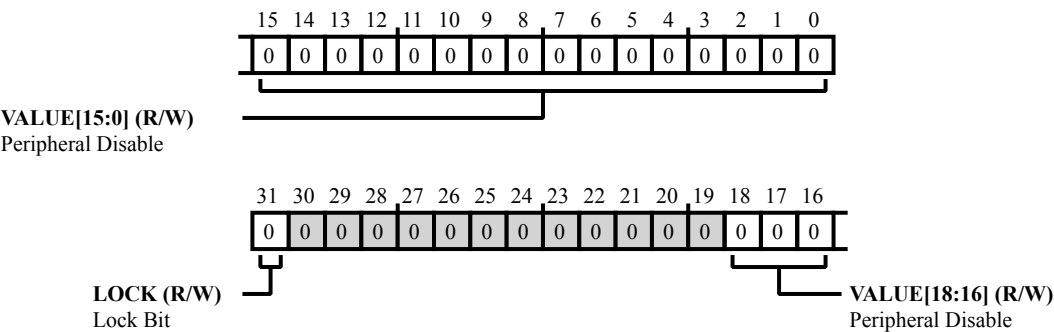


Figure 3-3: DPM_PER_DIS1 Register Diagram

Table 3-7: DPM_PER_DIS1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit.
18:0 (R/W)	VALUE	Peripheral Disable. The <code>DPM_PER_DIS1.VALUE</code> bits are used to shut of clocks in individual peripherals.
		0 TWI0
		1 TWI1
		2 TWI2
		3 TWI3
		4 TWI4
		5 TWI5
		6 OSPI_sysclk
		7 USBULPI
		8 C0
		9 SH1_FIR0
		10 SH1_IIR0
		11 SH1_IIR1
		12 SH1_IIR2
		13 SH1_IIR3

Table 3-7: DPM_PER_DIS1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		14	OSPI_CLK010
		15	PDM
		16	Reserved
		17	Reserved
		18	Reserved

Revision ID

The `DPM_REVID` register provides the revision of the DPM module.

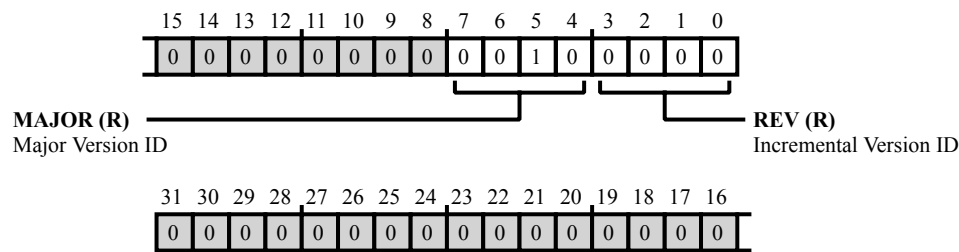


Figure 3-4: DPM_REVID Register Diagram

Table 3-8: DPM_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major Version ID.
3:0 (R/NW)	REV	Incremental Version ID.

Status Register

The `DPM_STAT` register contains bits that report the state of the module and various errors.

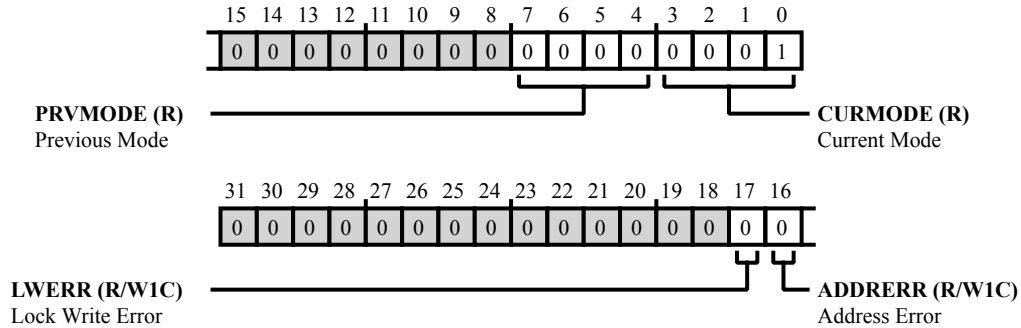


Figure 3-5: DPM_STAT Register Diagram

Table 3-9: DPM_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W1C)	LWERR	Lock Write Error. The <code>DPM_STAT.LWERR</code> bit indicates that a write transaction attempted an access to a write protected register. Triggers the <code>DPMLV_PSLVERR</code> interrupt.
		0 Inactive
		1 Active
16 (R/W1C)	ADDRERR	Address Error. The <code>DPM_STAT.ADDRERR</code> bit indicates that a read or write transaction attempted an access to an unimplemented address or a write transaction attempted an access to a read only register or accesses are non aligned. Triggers the <code>DPMLV_PSLVERR</code> interrupt.
		0 Inactive
		1 Active
7:4 (R/NW)	PRVMODE	Previous Mode. The <code>DPM_STAT.PRVMODE</code> bit field indicates the previous mode of the the module.
		0 Reset
		1 Full-On
		2 Reserved
		3 Reserved
		4 Reserved
		5-15 Reserved

Table 3-9: DPM_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/NW)	CURMODE	Current Mode. The DPM_STAT . CURMODE bit field indicates the current mode of the the module.
		0 Reserved
		1 Full-On
		2 Reserved
		3 Reserved
		4-15 Reserved

4 Reset Control Unit (RCU)

Reset is the initial state of the processor at power-on or the run-time state of any core, as controlled by another core in the device via the RCU or as a result of a hardware or software triggered event. In this state, all control registers are set to their default values and functional units are idle. Exiting a full system hardware reset results with only the host core being released from reset and ready to boot (Core 0 on the ADSP-SC57x devices, Core 1 on the ADSP-215xx devices).

The following table shows the cores that are released from reset. Exiting a Core n only reset starts with this Core n being ready to execute the code from the software vector registers ([RCU_SVECT0](#) , [RCU_SVECT2](#)).

Additional information on reset and booting can be found in the *Boot ROM and Booting the Processor* chapter.

Product	Core Released from Reset by RCU
ADSP-SC57x	Core 0
ADSP-2157x	Core 1

The Reset Control Unit (RCU) controls how all the functional units enter and exit reset. Differences in functional requirements and clocking constraints define how reset signals are generated. Programs must guarantee that none of the reset functions puts the system into an undefined state or causes resources to stall. This functionality is important when only one of the cores is reset (programs must ensure that there is no pending system activity involving the core that is being reset). While core resets and software system resets are controlled directly in the RCU, hardware resets can come from the TRU, SEC, or CGU Oscillator Watchdog.

RCU Features

The RCU module supports the following features:

- Hardware reset through the [SYS_HWRST](#) pin
- Software system reset through the RCU control ([RCU_CTL](#)) register
- Hardware system reset through:
 - TRU module
 - SEC module (System Fault Unit)

- A clock not good reset state (safe state of chip under reset) from the Oscillator Watchdog.
- Core reset through RCU Core Reset Output ([RCU_CRCTL](#)) register

RCU Functional Description

This section provides information on the function of RCU module.

Hardware reset using SYS_HWRST pin

Asserting the [SYS_HWRST](#) pin resets all functional units, except the real time clock (RTC) (if present).

Hardware reset through RCU

The RCU can perform a full system reset which can be initiated through hardware blocks like the SEC, the TRU, and the oscillator watchdog.

Software reset using RCU registers

Setting the [RCU_STAT](#).[SWRST](#) bit issues a software reset for all system units except the RTC module and [RCU_BCODE](#), [RCU_CRCTL](#) and [RCU_STAT](#) registers.

Core reset RCU registers

A core can be individually reset by software, or by setting the [RCU_CRCTL](#).[CR\[n\]](#) bit.

ADSP-2159x RCU Register List

The Reset Control Unit (RCU) controls how all the functional units in the processor enter and exit Reset. Differences in functional requirements and clocking constraints exist (units in different clock domains have to enter reset asynchronously, but units exit reset in a deterministic way), and these differences define how reset signals are generated. Reset signals propagate through all functional units asynchronously. For more information on RCU functionality, see the RCU register descriptions.

Table 4-1: ADSP-2159x RCU Register List

Name	Description
RCU_BCODE	Boot Code Register
RCU_CRCTL	Core Reset Outputs Control Register
RCU_CRSTAT	Core Reset Outputs Status Register
RCU_CTL	Control Register
RCU_MSG	Message Register
RCU_MSG_CLR	Message Clear Bits Register

Table 4-1: ADSP-2159x RCU Register List (Continued)

Name	Description
RCU_MSG_SET	Message Set Bits Register
RCU_SIDIS	System Interface Disable Register
RCU_SISTAT	System Interface Status Register
RCU_SRRQSTAT	System Reset Request Status Register
RCU_STAT	Status Register
RCU_SVECT0	Software Vector Register 0
RCU_SVECT1	Software Vector Register 1
RCU_SVECT2	Software Vector Register 2
RCU_SVECT_LCK	SVECT Lock Register

ADSP-2159x RCU Trigger List

Table 4-2: ADSP-2159x RCU Trigger List Masters

Trigger ID	Name	Description	Sensitivity
None			

Table 4-3: ADSP-2159x RCU Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
50	RCU0_SYSRST0	RCU0 System Reset Slave 0	Pulse
51	RCU0_SYSRST1	RCU0 System Reset Slave 1	Pulse

RCU Definitions

To make the best use of the RCU, it is useful to understand the terms in this section.

The target or source defines the following are types of resets.

Hardware Reset (by target)

All functional units except a small subsection of debug interfaces are set to their default states. State is lost in all non-volatile storage.

System Reset (by target)

All functional units except the RCU, flash interface, and debug are set to their default states.

Core n Only Reset (by target)

Affects Core n only. The system software must guarantee that a bus master cannot access the core in reset state.

Hardware Reset (by source)

The `SYS_HWRST` input signal is asserted active (pulled low).

System Reset (by source)

Software can trigger the reset by writing to the `RCU_CTL` register or by another functional unit such as the TRU or any of the generic reset inputs.

RCU Architectural Concepts

To understand the architecture of the RCU, it is important to consider the reset sources and how differing resets affect the functional units of the processor.

The RCU provides the hardware that controls how all the functional units enter and exit reset. Differences in functional requirements and clocking constraints define how reset signals are generated. For example, units in different clock domains must enter reset asynchronously but exit reset in a deterministic way.

The program must guarantee that none of the reset functions put the system in an undefined state or cause resources to stall. This functionality is important when only one of the cores is reset. The program must guarantee that there is no pending system activity involving Core n before it is reset. For example, there must be no pending transactions to core 0 when the core 0 is reset and vice-versa.

The *RCU Reset Sources* table defines how reset sources affect the different functional units.

Table 4-4: RCU Reset Sources

Reset Source	Reset Type	Affected Functional Units
<code>SYS_HWRST</code> pin assertion	Hardware Reset	All functional units, except RTC (if present)
<code>SYSCLK</code> clock domain system reset by the Fault Unit (FMU) or TRU slave	System Reset	All functional units, except: • RTC (if present) • <code>RCU_STAT</code> • <code>RCU_BCODE</code> • the units on the <code>VDDEXT</code> power domain
<code>RCU_CTL.SYSRST</code> bit set (software triggered reset)	System Reset	All functional units, except: • RTC (if present) • <code>RCU_STAT</code> • <code>RCU_BCODE</code> • the units on the <code>VDDEXT</code> power domain

Table 4-4: RCU Reset Sources (Continued)

Reset Source	Reset Type	Affected Functional Units
RCU_CRCTL.CR[n] bit set (software triggered reset)	Core Only Reset	Core n only , for ($2 \geq n \geq 0$)

RCU Status and Error Signals

The `RCU_STAT` register reflects status and error information. There are three kinds of errors that can occur in the RCU. The reset out error is triggered when `RSTOUT` is both asserted and deasserted at the same time. The lock write error occurs if an attempt is made to write a lock RCU register. The address error occurs if a read-only register is written to or if an attempt is made to a reserved address within the RCU MMR address range.

ADSP-2159x RCU Register Descriptions

Reset Control Unit (RCU) contains the following registers.

Table 4-5: ADSP-2159x RCU Register List

Name	Description
<code>RCU_BCODE</code>	Boot Code Register
<code>RCU_CRCTL</code>	Core Reset Outputs Control Register
<code>RCU_CRSTAT</code>	Core Reset Outputs Status Register
<code>RCU_CTL</code>	Control Register
<code>RCU_MSG</code>	Message Register
<code>RCU_MSG_CLR</code>	Message Clear Bits Register
<code>RCU_MSG_SET</code>	Message Set Bits Register
<code>RCU_SIDIS</code>	System Interface Disable Register
<code>RCU_SISTAT</code>	System Interface Status Register
<code>RCU_SRRQSTAT</code>	System Reset Request Status Register
<code>RCU_STAT</code>	Status Register
<code>RCU_SVECT0</code>	Software Vector Register 0
<code>RCU_SVECT1</code>	Software Vector Register 1
<code>RCU_SVECT2</code>	Software Vector Register 2
<code>RCU_SVECT_LCK</code>	SVECT Lock Register

Boot Code Register

The `RCU_BCODE` register can be used to determine if and how core boots. This register is set to its default values by RESET.

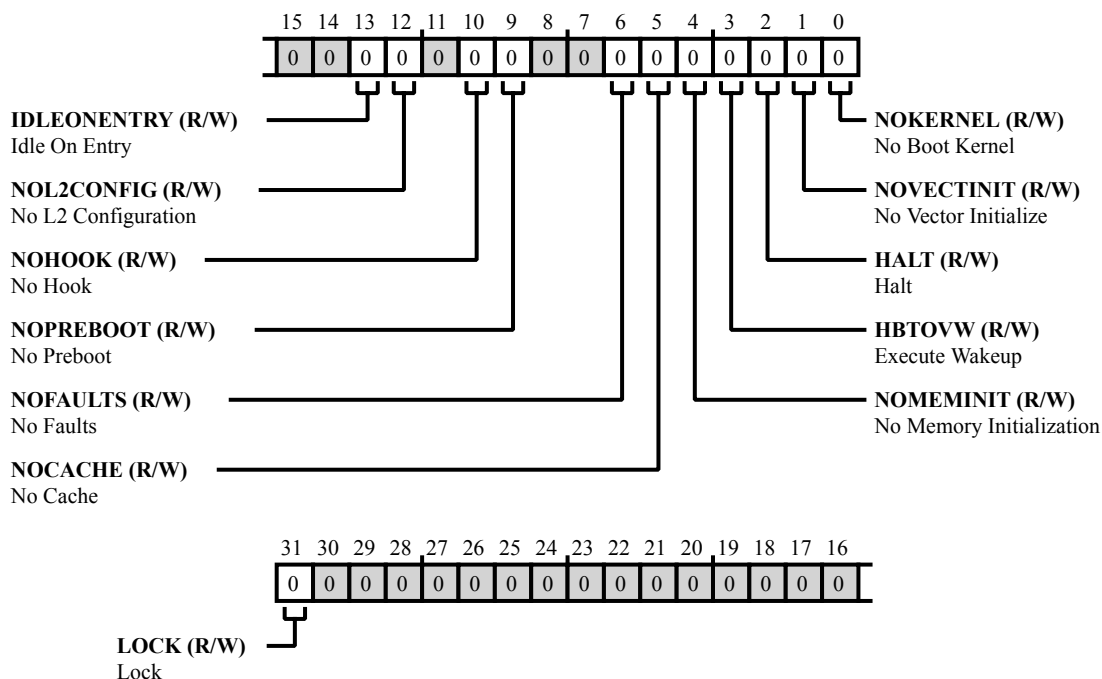


Figure 4-1: RCU_BCODE Register Diagram

Table 4-6: RCU_BCODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>RCU_BCODE.LOCK</code> bit is set, the <code>RCU_BCODE</code> register is read only (locked).
		0 Unlock
		1 Lock
13 (R/W)	IDLEONENTRY	Idle On Entry. The <code>RCU_BCODE.IDLEONENTRY</code> bit configures the RCU to enter the idle state at startup.
		0 Do not enter idle state
		1 Enter idle state

Table 4-6: RCU_BCODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	NOL2CONFIG	No L2 Configuration. The RCU_BCODE.NOL2CONFIG bit configures the RCU to not perform the L2 memory configuration.
		0 Configure L2 memory
		1 Do not configure L2 memory
10 (R/W)	NOHOOK	No Hook. The RCU_BCODE.NOHOOK bit configures the RCU to not perform the hook routine.
		0 Perform hook routine
		1 Do not perform hook routine
9 (R/W)	NOPREBOOT	No Preboot. The RCU_BCODE.NOPREBOOT bit configures the RCU to not perform the customer preboot routine.
		0 Perform preboot
		1 Do not perform preboot
6 (R/W)	NOFAULTS	No Faults. The RCU_BCODE.NOFAULTS bit configures the RCU to not perform fault initialization.
		0 Perform fault initialization
		1 Do not perform fault initialization
5 (R/W)	NOCACHE	No Cache. The RCU_BCODE.NOCACHE bit configures the RCU to not perform a cache initialization and to not enable the cache.
		0 Enable and initialize cache
		1 Do not initialize or enable cache
4 (R/W)	NOMEMINIT	No Memory Initialization. The RCU_BCODE.NOMEMINIT bit configures the RCU to not perform a memory initialization.
		0 Perform memory initialization
		1 Do not perform memory initialization

Table 4-6: RCU_BCODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	HBTOWV	Execute Wakeup. The RCU_BCODE.HBTOWV bit configures the RCU to execute a wakeup.
		0 Do not wakeup
		1 Execute wakeup
2 (R/W)	HALT	Halt. The RCU_BCODE.HALT bit configures the RCU to execute the no boot routine.
		0 Do not execute routine
		1 Execute routine
1 (R/W)	NOVECTINIT	No Vector Initialize. The RCU_BCODE.NOVECTINIT bit configures the RCU to not vector to the application.
		0 Vector
		1 Do not vector
0 (R/W)	NOKERNEL	No Boot Kernel. The RCU_BCODE.NOKERNEL bit configures the RCU to not execute the boot kernel.
		0 Execute boot kernel
		1 Do not execute boot kernel

Core Reset Outputs Control Register

The RCU core reset control n registers (`RCU_CRCTL`) include a lock bit (`RCU_CRCTL.LOCK`) and a core reset bit (`RCU_CRCTL.CR[n]`) for each core reset signal on the product.

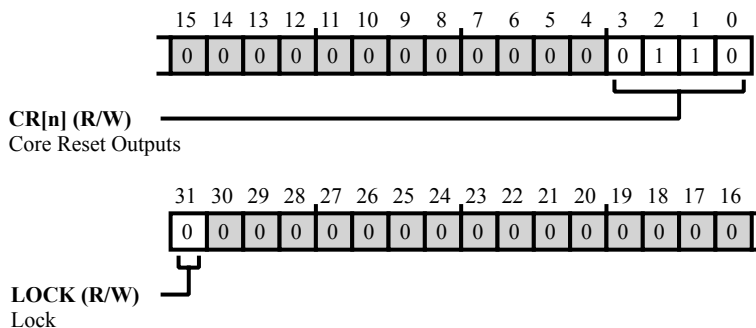


Figure 4-2: RCU_CRCTL Register Diagram

Table 4-7: RCU_CRCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>RCU_CRCTL.LOCK</code> bit is set, the <code>RCU_CRCTL</code> register is read only (locked).
		0 Unlock
		1 Lock
3:0 (R/W)	CR[n]	Core Reset Outputs.
		The <code>RCU_CRCTL.CR[n]</code> bits control <code>CRES[1:0]</code> core reset signals. The <code>RCU_CRES[n]</code> signals can be individually controlled. They are reset to their default value by a hard reset or a system reset. For each <code>RCU_CRES[n]</code> , the selected <code>RCU0_CRMSKi[n]</code> bit is cleared.
		0 RCU_CRES[3:0] Deasserted
		1 RCU_CRES[0] ARM A5 reset control bit asserted
		2 RCU_CRES[1] SHARC0 reset control bit asserted
		4 RCU_CRES[2] SHARC1 reset control bit asserted
		8 RCU_CRES[3] DEBUG reset control bit asserted
		15 RCU_CRES[3:0] Asserted

Core Reset Outputs Status Register

The RCU core reset status register ([RCU_CRSTAT](#)) contains status bits, indicating which core reset signals have been asserted.

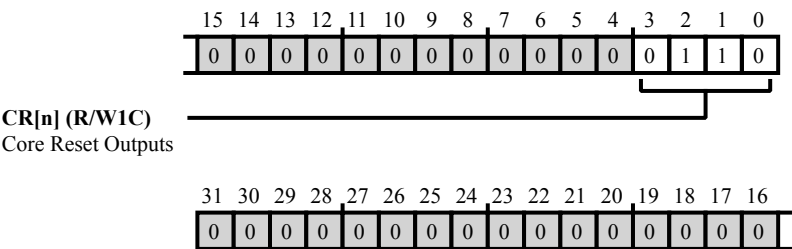


Figure 4-3: RCU_CRSTAT Register Diagram

Table 4-8: RCU_CRSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W1C)	CR[n]	Core Reset Outputs. The RCU_CRSTAT.CR[n] bits indicate which cores have been reset since the last time the bit was cleared. Bits masked by CORE_DISABLE_MASK[15:0] are permanently disabled and the corresponding CR bits set. CR bits are sticky, they need to be cleared by software.
		0 RCU_CRES[1:0] deasserted. CR[n] corresponds to RCU_CRES[n].
		1 RCU_CRES[0] ARM A5 reset control bit asserted
		2 RCU_CRES[1] SHARC0 reset control bit asserted
		4 RCU_CRES[2] SHARC1 reset control bit asserted
		8 RCU_CRES[3] DEBUG reset control bit asserted
		15 RCU_CRES[3:0] were asserted since the last time bits were cleared. CR[n] corresponds to RCU_CRES[n].

Control Register

The RCU control register (`RCU_CTL`) provides a register lock, enables for the core and system reset requests inputs and control for the Reset Output pin.

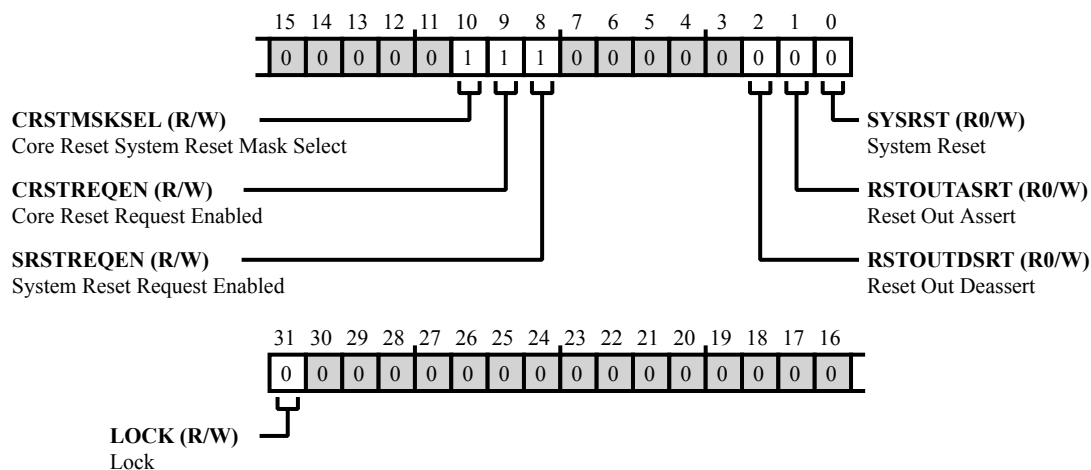


Figure 4-4: RCU_CTL Register Diagram

Table 4-9: RCU_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>RCU_CTL.LOCK</code> bit is set, the <code>RCU_CTL</code> register is read only (locked). This bit is cleared by a hard reset or any system reset event.
		0 Unlock 1 Lock
10 (R/W)	CRSTMSKSEL	Core Reset System Reset Mask Select. The <code>RCU_CTL.CRSTMSKSEL</code> bit selects the core reset system reset mask. This bit is cleared by a hard reset.
9 (R/W)	CRSTREQEN	Core Reset Request Enabled.
		The <code>RCU_CTL.CRSTREQEN</code> bit controls whether the <code>SYSCLK</code> domain source(s) of reset is/are enabled to reset the core(s) when asserted. This bit is cleared by hard reset or any system reset event.
		0 Disabled 1 Enabled

Table 4-9: RCU_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	SRSTREQEN	System Reset Request Enabled. The RCU_CTL.SRSTREQEN bit controls whether the SYSCLK domain sources of reset are enabled to do a system reset when asserted. This bit is cleared by a hard reset.
		0 Disabled
		1 Enabled
2 (R0/W)	RSTOUTDSRT	Reset Out Deassert. The RCU_CTL.RSTOUTDSRT bit controls the deassertion of the system reset pin.
		0 No Action
		1 Deassert RSTOUT
1 (R0/W)	RSTOUTASRT	Reset Out Assert. The RCU_CTL.RSTOUTASRT bit controls assertion of the system reset pin.
		0 No Action
		1 Assert RSTOUT
0 (R0/W)	SYSRST	System Reset. The RCU_CTL.SYSRST bit provides reset for all system units.
		0 No Action
		1 System Reset

Message Register

The `RCU_MSG` is a general-purpose register. It is intended to provide flexibility for Boot ROM code and to pass predefined variables to the debugger. Please see the Booting chapter for product-specific details.

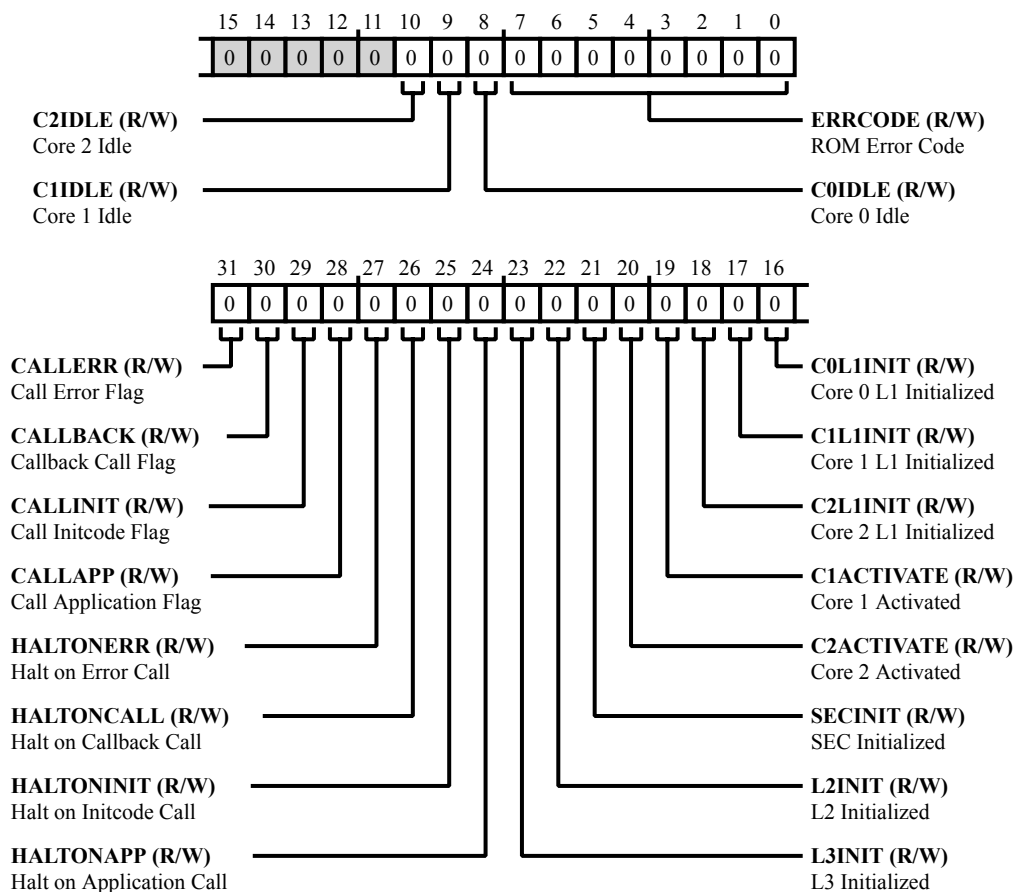


Figure 4-5: RCU_MSG Register Diagram

Table 4-10: RCU_MSG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CALLERR	Call Error Flag. The <code>RCU_MSG.CALLERR</code> bit indicates that a flag has been set by the boot code prior to an error call.
		0 Flag not set
		1 Flag set

Table 4-10: RCU_MSG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W)	CALLBACK	Callback Call Flag. The RCU_MSG.CALLBACK bit indicates that a flag has been set by the boot code prior to a callback call.
		0 Flag not set
		1 Flag set
29 (R/W)	CALLINIT	Call Initcode Flag. The RCU_MSG.CALLINIT bit indicates that a flag has been set by the boot code prior to an initcode call.
		0 Flag not set
		1 Flag set
28 (R/W)	CALLAPP	Call Application Flag. The RCU_MSG.CALLAPP bit indicates that a flag has been set by the boot code prior to an application call.
		0 Flag not set
		1 Flag set
27 (R/W)	HALTONERR	Halt on Error Call. The RCU_MSG.HALTONERR bit generates an emulation exception prior to an error call.
		0 Do not generate exception
		1 Generate exception
26 (R/W)	HALTONCALL	Halt on Callback Call. The RCU_MSG.HALTONCALL bit generates an emulation exception prior to a callback call.
		0 Do not generate exception
		1 Generate exception
25 (R/W)	HALTONINIT	Halt on Initcode Call. The RCU_MSG.HALTONINIT bit generates an emulation exception prior to an initcode call.
		0 Do not generate exception
		1 Generate exception

Table 4-10: RCU_MSG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/W)	HALTONAPP	Halt on Application Call. The RCU_MSG.HALTONAPP bit generates an emulation exception prior to an application call.
		0 Do not generate exception
		1 Generate exception
23 (R/W)	L3INIT	L3 Initialized. The RCU_MSG.L3INIT bit indicates that the L3 resource is initialized.
		0 Resource not initialized
		1 Resource initialized
22 (R/W)	L2INIT	L2 Initialized. The RCU_MSG.L2INIT bit indicates that the L2 resource is initialized.
		0 Resource not initialized
		1 Resource initialized
21 (R/W)	SECINIT	SEC Initialized. The RCU_MSG.SECINIT bit is used by tools for initialization of the SEC.
20 (R/W)	C2ACTIVATE	Core 2 Activated. The RCU_MSG.C2ACTIVATE bit is used by tools for activation of Core 2.
19 (R/W)	C1ACTIVATE	Core 1 Activated. The RCU_MSG.C1ACTIVATE bit is used by tools for activation of Core 1.
18 (R/W)	C2L1INIT	Core 2 L1 Initialized. The RCU_MSG.C2L1INIT bit indicates that the core 2 L1 resource is initialized.
17 (R/W)	C1L1INIT	Core 1 L1 Initialized. The RCU_MSG.C1L1INIT bit indicates that the core 1 L1 resource is initialized.
		0 Resource not initialized
		1 Resource initialized
16 (R/W)	C0L1INIT	Core 0 L1 Initialized. The RCU_MSG.C0L1INIT bit indicates that the core 0 L1 resource is initialized.
		0 Resource not initialized
		1 Resource initialized
10 (R/W)	C2IDLE	Core 2 Idle. The RCU_MSG.C2IDLE bit indicates that core 2 is in a safe idle state in ROM.

Table 4-10: RCU_MSG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	C1IDLE	Core 1 Idle. The <code>RCU_MSG.C1IDLE</code> bit indicates that core 1 is in a safe idle state in ROM.
8 (R/W)	C0IDLE	Core 0 Idle. The <code>RCU_MSG.C0IDLE</code> bit indicates that core 0 is in a safe idle state in ROM.
7:0 (R/W)	ERRCODE	ROM Error Code. The <code>RCU_MSG.ERRCODE</code> bit indicates the error code of the ROM. It is valid only when in the error handler.

Message Clear Bits Register

The `RCU_MSG_CLR` register is used to clear bits in `RCU_MSG` register. Reading this register returns 0x00000000.

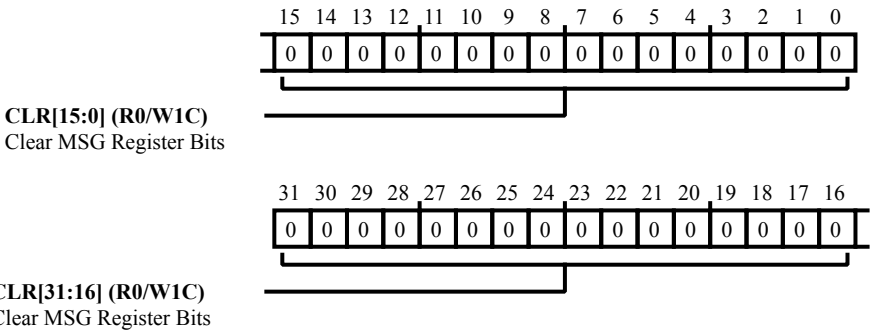


Figure 4-6: RCU_MSG_CLR Register Diagram

Table 4-11: RCU_MSG_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W1C)	CLR	Clear MSG Register Bits. The <code>RCU_MSG_CLR</code> .CLR bit resets MSG bit n.

Message Set Bits Register

The `RCU_MSG_SET` register is used to set bits in `RCU_MSG` register. Reading this register returns 0x00000000.

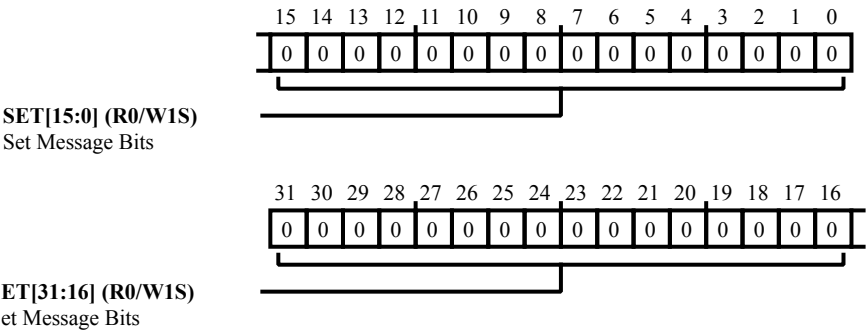


Figure 4-7: RCU_MSG_SET Register Diagram

Table 4-12: RCU_MSG_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W1S)	SET	Set Message Bits. The <code>RCU_MSG_SET.SET</code> bit sets MSG bit n.

System Interface Disable Register

The RCU system interface disable register ([RCU_SIDIS](#)) lets the RCU assert a system interface disable request to functional units in the processor. This register is set to its default values by a hard reset or any system reset event. For information on mapping between [RCU_SIDIS](#) bits and functional units, see the RCU functional description.

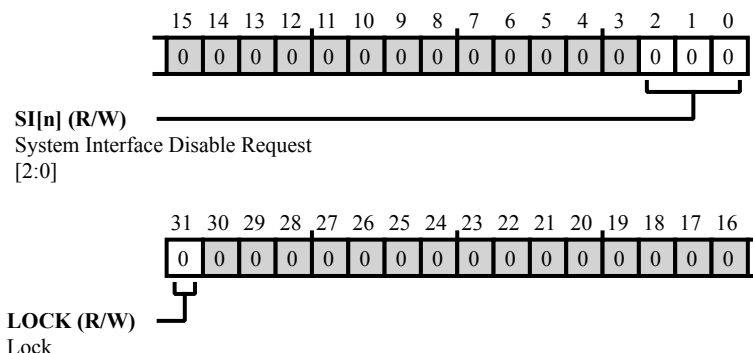


Figure 4-8: RCU_SIDIS Register Diagram

Table 4-13: RCU_SIDIS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>RCU_SIDIS.LOCK</code> bit is set, the RCU_SIDIS register is read only (locked).
		0 Unlock 1 Lock
2:0 (R/W)	SI[n]	System Interface Disable Request [2:0].
		Each <code>RCU_SIDIS.SI[n]</code> bit corresponds to a functional unit in the processor that supports the system interface disable request-acknowledge protocol.
		0 <code>RCU_SI_DISABLE_REQ[2:0]</code> deasserted
		1 <code>RCU_SI_DISABLE_REQ[0]</code> system interface disable request to SHARC0 asserted
		2 <code>RCU_SI_DISABLE_REQ[1]</code> system interface disable request to SHARC1 asserted
		4 Reserved 7 <code>RCU_SI_DISABLE_REQ[2:0]</code> asserted

System Interface Status Register

The RCU system interface status register ([RCU_SISTAT](#)) indicates whether a functional unit has or has not acknowledged an RCU unit disable request.

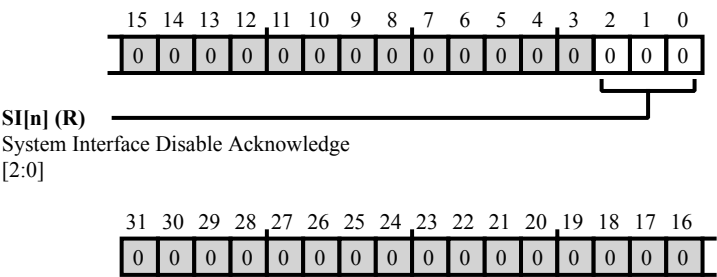


Figure 4-9: RCU_SISTAT Register Diagram

Table 4-14: RCU_SISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/NW)	SI[n]	System Interface Disable Acknowledge [2:0]. The <code>RCU_SISTAT.SI[n]</code> bit indicates whether a functional unit has or has not acknowledged an RCU unit disable request.
		0 No Acknowledge
		1 <code>RCU_SI_DISABLE_ACK[0]</code> system interface disable acknowledge from SHARC0 asserted
		2 <code>RCU_SI_DISABLE_ACK[1]</code> system interface disable acknowledge from SHARC1 asserted
		4 Reserved
		7 <code>SI_DISABLE_ACK[2:0]</code> asserted

System Reset Request Status Register

The RCU system reset request status register ([RCU_SRRQSTAT](#)) contains status bits, indicating which system reset request input triggered a system reset. This register is set to its default values by a hard reset.

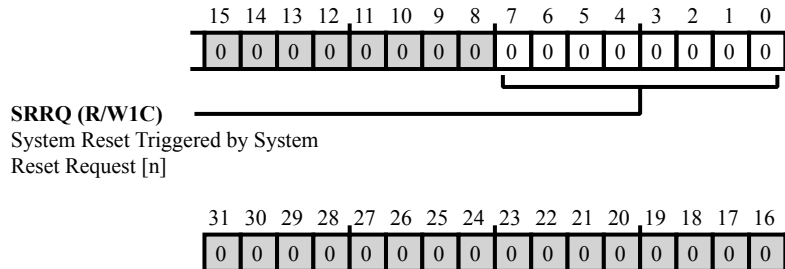


Figure 4-10: RCU_SRRQSTAT Register Diagram

Table 4-15: RCU_SRRQSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W1C)	SRRQ	System Reset Triggered by System Reset Request [n]. The RCU_SRRQSTAT . SRRQ bits are set by the assertion of the corresponding system reset request input and deasserted by writing "1" to the bit. The RCU_SRRQSTAT register is cleared by a hard reset.
		1 RCU_SRRQ[0], System Reset Request from SEC asserted
		2 RCU_SRRQ[1], System Reset Request from TRGS_RCU0_SYSRST0 asserted
		4 RCU_SRRQ[2], System Reset Request from TRGS_RCU0_SYSRST1 asserted
		8 RCU_SRRQ[3], System Reset Request from CTI3 asserted

Status Register

The RCU status register (`RCU_STAT`) contains status bits for all RCU reset sources, reset status, and boot mode inputs. Status bits for reset sources are sticky and can cleared by software. Error status bits are cleared by any reset event.

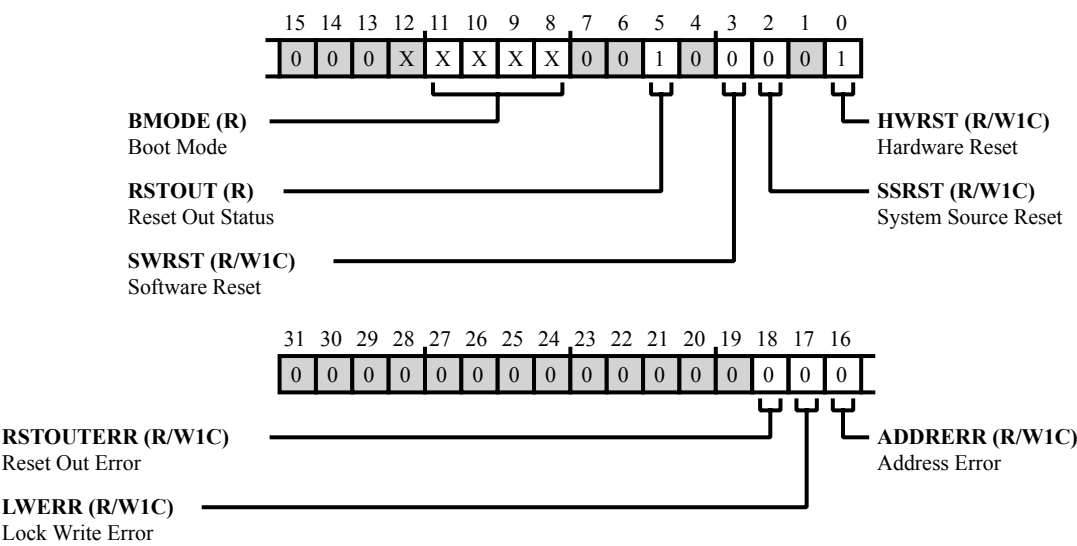


Figure 4-11: RCU_STAT Register Diagram

Table 4-16: RCU_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W1C)	RSTOUTERR	Reset Out Error. The <code>RCU_STAT.RSTOUTERR</code> bit indicates (if set) that a write attempted to set the <code>RCU_CTL.RSTOUTASRT</code> and <code>RCU_CTL.RSTOUTDSRT</code> simultaneously. This condition triggers a bus error.
		0 No Error
		1 Error Occurred
17 (R/W1C)	LWERR	Lock Write Error. The <code>RCU_STAT.LWERR</code> bit indicates (when set) there was an attempted write to an RCU register while the <code>RCU_CTL.LOCK</code> bit was set and the global lock bit is enabled (<code>SPU_CTL.GLCK</code> bit =1). This status bit is sticky; write-1-to-clear
		0 No Error
		1 Error Occurred

Table 4-16: RCU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W1C)	ADDRERR	Address Error. The RCU_STAT.ADDRERR bit indicates that the RCU generated an address error. This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
11:8 (R/NW)	BMODE	Boot Mode. The RCU_STAT.BMODE bits indicate the input on the boot mode pins.
5 (R/NW)	RSTOUT	Reset Out Status. The RCU_STAT.RSTOUT bit indicates the assertion status of the system reset pin.
		0 RSTOUT Deasserted
		1 RSTOUT Asserted
3 (R/W1C)	SWRST	Software Reset. The RCU_STAT.SWRST bit indicates that a system reset (which was triggered by software) has occurred since the last time a hardware reset occurred or since the RCU_STAT.SWRST bit was cleared by software.
		0 Inactive
		1 Reset Occurred
2 (R/W1C)	SSRST	System Source Reset. The RCU_STAT.SSRST bit indicates that a system reset triggered by hardware in the system clock domain, clock A domain, or clock B domain has occurred since the last time a hardware reset occurred or since the RCU_STAT.SSRST bit was cleared by software.
		0 Inactive
		1 Reset Occurred
0 (R/W1C)	HWRST	Hardware Reset. The RCU_STAT.HWRST bit indicates that a hardware reset has occurred.
		0 Inactive
		1 Reset Occurred

Software Vector Register 0

The `RCU_SVECT0` register contains the default location of the first instruction to execute after a reset.

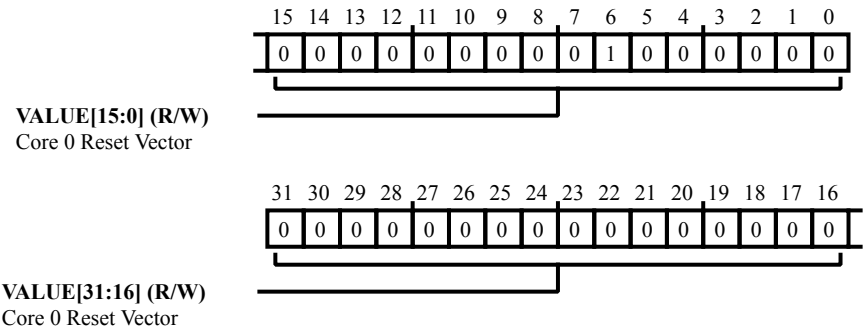


Figure 4-12: RCU_SVECT0 Register Diagram

Table 4-17: RCU_SVECT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Core 0 Reset Vector. The <code>RCU_SVECT0.VALUE</code> bit field contains the default location of the first instruction to execute after a reset.

Software Vector Register 1

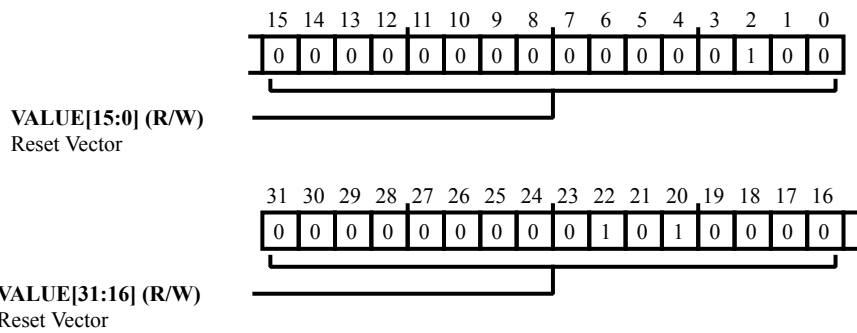


Figure 4-13: RCU_SVECT1 Register Diagram

Table 4-18: RCU_SVECT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Reset Vector.

Software Vector Register 2

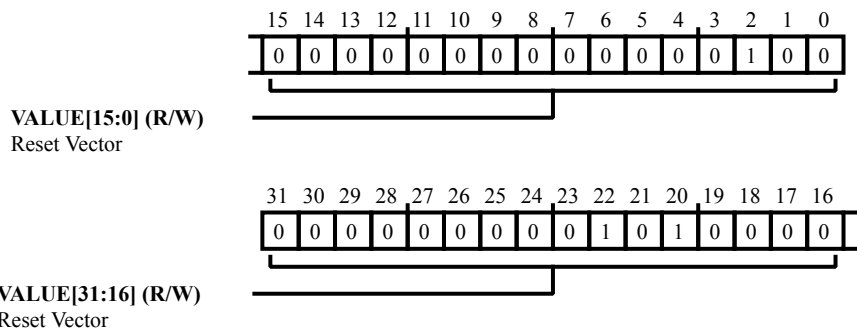


Figure 4-14: RCU_SVECT2 Register Diagram

Table 4-19: RCU_SVECT2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Reset Vector.

SVECT Lock Register

The RCU software vector lock register ([RCU_SVECT_LCK](#)) provides a register lock and software vector n enable bits for each processor core on the product. This register is set to its default values by a hard reset or any system reset event.

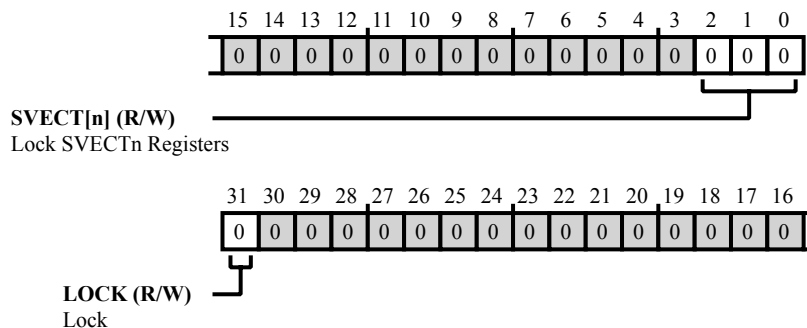


Figure 4-15: RCU_SVECT_LCK Register Diagram

Table 4-20: RCU_SVECT_LCK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>RCU_SVECT_LCK.LOCK</code> bit is set, the RCU_SVECT_LCK register is read only (locked).
		0 Unlock
		1 Lock
2:0 (R/W)	SVECT[n]	Lock SVECTn Registers. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>RCU_SVECT_LCK.SVECT[n]</code> bit is set, the SVECT registers are read only (locked).

5 System Event Controller (SEC) and Generic Interrupt Controller (GIC)

There are two interrupt controllers—a generic interrupt controller (GIC) for the Arm core and the system event controller (SEC) for the SHARC cores.

System event management is the responsibility of the system event controller (SEC). The SEC manages the configuration of all system event sources. The SEC also manages the propagation of system events to all connected cores and the system fault interface.

All of the peripheral interrupts are routed using a single SEC interrupt to the desired core. The SEC allows programmability of the peripheral interrupt's priority, supporting up to 256 priority levels that are arbitrated within the SEC itself. The SEC also allows these interrupts to be grouped and masked by priority level and provides the flexibility to choose which core(s) the interrupt is routed to.

The SEC also supports self-nesting of interrupts, which is required when sharing a single interrupt request to an individual core, as this allows for a higher-priority peripheral interrupt to be passed to the core while it is currently servicing a lower-priority peripheral interrupt. For more information, refer to “Self-Nesting Mode for System Event Controller Interrupt (SECI)” in the *SHARC+ Core Programming Reference*.

For more information about the Arm GIC, visit the Arm Information Center.

SEC Features

The following list describes the system event controller features.

- Comprehensive system event source management including interrupt enable, fault enable, priority, core mapping, and source grouping.
- Fault management including fault action configuration, timeout, external indication, and system reset.
- Determinism where all system events have the same propagation delay and provide unique identification of a specific system event source.
- Distributed programming model where each system event source control and all status fields are independent of all others.

- Slave control port which provides access to all SEC registers for configuration, status, and interrupt or fault service model.
- Global locking supports a register level protection model to prevent writes to “locked” registers.

SEC Functional Description

The following sections provide a functional description of the SEC.

The *SEC/GIC Interrupt Signal Flow* figure shows an overview of the interrupt systems.

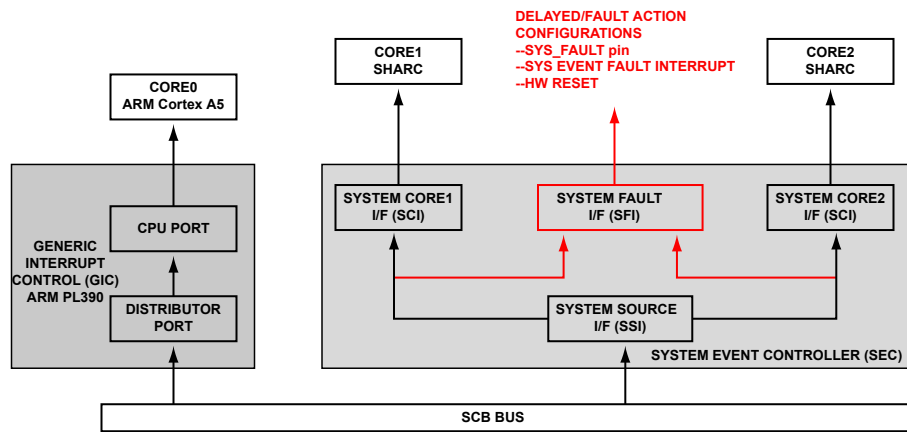


Figure 5-1: SEC/GIC Interrupt Signal Flow

ADSP-2159x SEC Register List

The System Event Controller (SEC) manages the system fault sources, including control features such as enable/disable, priority, and active/pending source status. For more information on SEC functionality, see the SEC register descriptions.

Table 5-1: ADSP-2159x SEC Register List

Name	Description
SEC_CACT[n]	SCI Active Register n
SEC_CCTL[n]	SCI Control Register n
SEC_CGMSK[n]	SCI Group Mask Register n
SEC_CPLVL[n]	SCI Priority Level Register n
SEC_CPMSK[n]	SCI Priority Mask Register n
SEC_CPND[n]	Core Pending Register n
SEC_CSID[n]	SCI Source ID Register n
SEC_CSTAT[n]	SCI Status Register n
SEC_END	Global End Register

Table 5-1: ADSP-2159x SEC Register List (Continued)

Name	Description
SEC_FCOPP	Fault COP Period Register
SEC_FCOPP_CUR	Fault COP Period Current Register
SEC_FCTL	Fault Control Register
SEC_FDLY	Fault Delay Register
SEC_FDLY_CUR	Fault Delay Current Register
SEC_FEND	Fault End Register
SEC_FSID	Fault Source ID Register
SEC_FSRDLY	Fault System Reset Delay Register
SEC_FSRDLY_CUR	Fault System Reset Delay Current Register
SEC_FSTAT	Fault Status Register
SEC_GCTL	Global Control Register
SEC_GSTAT	Global Status Register
SEC_RAISE	Global Raise Register
SEC_SCTL[n]	Source Control Register n
SEC_SSTAT[n]	Source Status Register n

ADSP-2159x SEC Interrupt List

Table 5-2: ADSP-2159x SEC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
0	SEC0_ERR	SEC0 Error	Level	

ADSP-2159x SEC Trigger List

Table 5-3: ADSP-2159x SEC Trigger List Masters

Trigger ID	Name	Description	Sensitivity
81	SEC0_FAULT	SEC0 Fault	Edge

Table 5-4: ADSP-2159x SEC Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

Combined SEC and GIC Interrupt List

The *Combined SEC and GIC Interrupt List* table provides a complete list of interrupts supported by the SHARC+ processor and Arm core. Note that the DAI has its own system interrupt controllers. For more information see the [DAI System Interrupt Controller \(SIC\)](#).

Table 5-5: Combined SEC and GIC Interrupt List

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
NA	NA	NA	GIC	GIC0_SOFT00	Arm Software Interrupt 0	0
NA	NA	NA	GIC	GIC0_SOFT01	Arm Software Interrupt 1	1
NA	NA	NA	GIC	GIC0_SOFT02	Arm Software Interrupt 2	2
NA	NA	NA	GIC	GIC0_SOFT03	Arm Software Interrupt 3	3
NA	NA	NA	GIC	GIC0_SOFT04	Arm Software Interrupt 4	4
NA	NA	NA	GIC	GIC0_SOFT05	Arm Software Interrupt 5	5
NA	NA	NA	GIC	GIC0_SOFT06	Arm Software Interrupt 6	6
NA	NA	NA	GIC	GIC0_SOFT07	Arm Software Interrupt 7	7
NA	NA	NA	NA	NA	Reserved	8-31
0	0	NA	SEC	SEC0_ERR	SEC0 Error	32
1	1	NA	CGU	CGU0_EVT	CGU0 Event	33
2	2	NA	CGU	CGU1_EVT	CGU1 Event	34
3	3	NA	WDOG	WDOG0_EXP	WDOG0 Expiration	35
4	4	NA	WDOG	WDOG1_EXP	WDOG1 Expiration	36
5	5	NA	WDOG	WDOG2_EXP	WDOG2 Expiration	37
6	6	NA	OTPC	OTPC0_ERR	OTPC0 Dual bit Error Interrupt	38
7	7	NA	TMU	TMU0_FAULT	TMU0 Fault Event	39
8	8	NA	TMU	TMU0_ALERT	TMU0 Alert Event	40
9	9	NA	TAPC	TAPC0_KEYFAIL	Test/User Key Fail Interrupt	41
10	10	NA	L2CTL	L2CTL0_ECC_ERR	L2CTL0 ECC Error	42
		Reserved				Reserved
12	12	NA	L2CTL	L2CTL0_EVT	L2CTL0 Scrub/Initialization Done	44
13	13	NA	MEC	MEC1_EEIRQ0	MEC1 L2CTL0/CAN ECC Error to Core 1	45
		Reserved				Reserved

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
15	15	NA	MEC	MEC1_PEIRQ0	MEC1 Core 0 Parity Error Interrupt to Core 1	47
16	16	NA	MEC	MEC1_PEIRQ1	MEC1 Core 1 Parity Error Interrupt to Core 1	48
17	17	NA	MEC	MEC1_PEIRQ2	MEC1 Core 2 Parity Error Interrupt to Core 1	49
18	18	NA	MEC	MEC1_PEIRQ3	MEC1 System Peripheral Parity Error Interrupt to Core 1	50
19	19	NA	MEC	MEC0_EEIRQ0	Reserved	51
		Reserved				Reserved
21	21	NA	MEC	MEC0_PEIRQ0	Reserved	53
22	22	NA	MEC	MEC0_PEIRQ1	Reserved	54
23	23	NA	MEC	MEC0_PEIRQ2	Reserved	55
24	24	NA	MEC	MEC0_PEIRQ3	Reserved	56
25	25	NA	MEC	MEC2_EEIRQ0	MEC2 L2CTL0/CAN ECC Error to Core 2	57
		Reserved				Reserved
27	27	NA	MEC	MEC2_PEIRQ0	MEC2 Core 0 Parity Error Interrupt to Core 2	59
28	28	NA	MEC	MEC2_PEIRQ1	MEC2 Core 1 Parity Error Interrupt to Core 2	60
29	29	NA	MEC	MEC2_PEIRQ2	MEC2 Core 2 Parity Error Interrupt to Core 2	61
30	30	NA	MEC	MEC2_PEIRQ3	MEC2 System Peripheral Parity Error Interrupt to Core 2	62
31	31	NA	CORE	C0_L2CC	CORE0 L2CC Interrupt	63
32	32	NA	CORE	C0_PMUIRQ	Reserved	64
33	33	NA	CORE	C0_INITDONE	C0 Memory Initialization Done	65
34	34	NA	CORE	C1_IRQ0	CORE1 Data Read Interrupt	66
35	35	NA	CORE	C1_IRQ1	CORE1 Data Write Interrupt	67
36	36	NA	CORE	C1_IRQ2	CORE1 Instruction Read Interrupt	68
37	37	NA	SYSTEM	SOFT8	Software-driven Interrupt 8	69
38	38	NA	CORE	C2_IRQ0	CORE2 Data Read Interrupt	70

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
39	39	NA	CORE	C2_IRQ1	CORE2 Data Write Interrupt	71
40	40	NA	CORE	C2_IRQ2	CORE2 Instruction Read Interrupt	72
41	41	NA	SYSTEM	SOFT9	Software-driven Interrupt 9	73
42	42	NA	CORE	CORE1_DS_SLV_ACC_INTR	Core 1 Slave Access request, if core is in Deep Sleep	74
43	43	NA	CORE	CORE2_DS_SLV_ACC_INTR	Core2 Slave Access request, if core is in Deep Sleep	75
44	44	NA	DAI	DAI0_IRQH	DAI0 High Priority Interrupt	76
45	45	NA	DAI	DAI1_IRQH	DAI1 High Priority Interrupt	77
46	46	NA	DAI	DAI0_IRQL	DAI0 Low Priority Interrupt	78
47	47	NA	DAI	DAI1_IRQL	DAI1 Low Priority Interrupt	79
48	48	NA	TIMER	TIMER0_TMR0	TIMER0 Timer 0	80
49	49	NA	TIMER	TIMER0_TMR1	TIMER0 Timer 1	81
50	50	NA	TIMER	TIMER0_TMR2	TIMER0 Timer 2	82
51	51	NA	TIMER	TIMER0_TMR3	TIMER0 Timer 3	83
52	52	NA	TIMER	TIMER0_TMR4	TIMER0 Timer 4	84
53	53	NA	TIMER	TIMER0_TMR5	TIMER0 Timer 5	85
54	54	NA	TIMER	TIMER0_TMR6	TIMER0 Timer 6	86
55	55	NA	TIMER	TIMER0_TMR7	TIMER0 Timer 7	87
56	56	NA	TIMER	TIMER0_TMR8	TIMER0 Timer 8	88
57	57	NA	TIMER	TIMER0_TMR9	TIMER0 Timer 9	89
58	58	NA	TIMER	TIMER0_TMR10	TIMER0 Timer 10	90
59	59	NA	TIMER	TIMER0_TMR11	TIMER0 Timer 11	91
60	60	NA	TIMER	TIMER0_TMR12	TIMER0 Timer 12	92
61	61	NA	TIMER	TIMER0_TMR13	TIMER0 Timer 13	93
62	62	NA	TIMER	TIMER0_TMR14	TIMER0 Timer 14	94
63	63	NA	TIMER	TIMER0_TMR15	TIMER0 Timer 15	95
64	64	NA	TIMER	TIMER0_STAT	TIMER0 Status	96
65	65	NA	PINT	PINT0_BLOCK	PINT0 Pin Interrupt Block	97
66	66	NA	PINT	PINT1_BLOCK	PINT1 Pin Interrupt Block	98

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
67	67	NA	PINT	PINT2_BLOCK	PINT2 Pin Interrupt Block	99
68	68	NA	PINT	PINT3_BLOCK	PINT3 Pin Interrupt Block	100
69	69	NA	PINT	PINT4_BLOCK	PINT4 Pin Interrupt Block	101
70	70	NA	PINT	PINT5_BLOCK	PINT5 Pin Interrupt Block	102
71	71	NA	PINT	PINT6_BLOCK	PINT6 Pin Interrupt Block	103
72	72	NA	PINT	PINT7_BLOCK	PINT7 Pin Interrupt Block	104
73	73	NA	SYSTEM	SOFT0	Software-driven Interrupt 0	105
74	74	NA	SYSTEM	SOFT1	Software-driven Interrupt 1	106
75	75	NA	SYSTEM	SOFT2	Software-driven Interrupt 2	107
76	76	NA	SYSTEM	SOFT3	Software-driven Interrupt 3	108
77	77	NA	SYSTEM	SOFT4	Software-driven Interrupt 4	109
78	78	NA	SYSTEM	SOFT5	Software-driven Interrupt 5	110
79	79	NA	SYSTEM	SOFT6	Software-driven Interrupt 6	111
80	80	NA	SYSTEM	SOFT7	Software-driven Interrupt 7	112
81	81	NA	SPORT	SPORT0_A_DMA	SPORT0 Channel A DMA	113
82	82	NA	SPORT	SPORT0_A_STAT	SPORT0 Channel A Status	114
83	83	NA	SPORT	SPORT0_B_DMA	SPORT0 Channel B DMA	115
84	84	NA	SPORT	SPORT0_B_STAT	SPORT0 Channel B Status	116
85	85	NA	SPORT	SPORT1_A_DMA	SPORT1 Channel A DMA	117
86	86	NA	SPORT	SPORT1_A_STAT	SPORT1 Channel A Status	118
87	87	NA	SPORT	SPORT1_B_DMA	SPORT1 Channel B DMA	119
88	88	NA	SPORT	SPORT1_B_STAT	SPORT1 Channel B Status	120
89	89	NA	SPORT	SPORT2_A_DMA	SPORT2 Channel A DMA	121
90	90	NA	SPORT	SPORT2_A_STAT	SPORT2 Channel A Status	122
91	91	NA	SPORT	SPORT2_B_DMA	SPORT2 Channel B DMA	123
92	92	NA	SPORT	SPORT2_B_STAT	SPORT2 Channel B Status	124
93	93	NA	SPORT	SPORT3_A_DMA	SPORT3 Channel A DMA	125
94	94	NA	SPORT	SPORT3_A_STAT	SPORT3 Channel A Status	126
95	95	NA	SPORT	SPORT3_B_DMA	SPORT3 Channel B DMA	127
96	96	NA	SPORT	SPORT3_B_STAT	SPORT3 Channel B Status	128

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
97	97	NA	SPORT	SPORT4_A_DMA	SPORT4 Channel A DMA	129
98	98	NA	SPORT	SPORT4_A_STAT	SPORT4 Channel A Status	130
99	99	NA	SPORT	SPORT4_B_DMA	SPORT4 Channel B DMA	131
100	100	NA	SPORT	SPORT4_B_STAT	SPORT4 Channel B Status	132
101	101	NA	SPORT	SPORT5_A_DMA	SPORT5 Channel A DMA	133
102	102	NA	SPORT	SPORT5_A_STAT	SPORT5 Channel A Status	134
103	103	NA	SPORT	SPORT5_B_DMA	SPORT5 Channel B DMA	135
104	104	NA	SPORT	SPORT5_B_STAT	SPORT5 Channel B Status	136
105	105	NA	SPORT	SPORT6_A_DMA	SPORT6 Channel A DMA	137
106	106	NA	SPORT	SPORT6_A_STAT	SPORT6 Channel A Status	138
107	107	NA	SPORT	SPORT6_B_DMA	SPORT6 Channel B DMA	139
108	108	NA	SPORT	SPORT6_B_STAT	SPORT6 Channel B Status	140
109	109	NA	SPORT	SPORT7_A_DMA	SPORT7 Channel A DMA	141
110	110	NA	SPORT	SPORT7_A_STAT	SPORT7 Channel A Status	142
111	111	NA	SPORT	SPORT7_B_DMA	SPORT7 Channel B DMA	143
112	112	NA	SPORT	SPORT7_B_STAT	SPORT7 Channel B Status	144
113	113	NA	SPORT	GLOB- AL_SPORT_INT0_DM A	SPORT DMA GROUP0 interrupt	145
114	114	NA	SPORT	GLOB- AL_SPORT_INT1_DM A	SPORT DMA GROUP1 interrupt	146
115	115	NA	SPORT	GLOB- AL_SPORT_INT2_DM A	SPORT DMA GROUP2 interrupt	147
116	116	NA	SPORT	GLOB- AL_SPORT_INT3_DM A	SPORT DMA GROUP3 interrupt	148
117	117	NA	LP	LP0_DMA	LP0 DMA Data	149
118	118	NA	LP	LP0_STAT	LP0 Status	150
119	119	NA	LP	LP1_DMA	LP1 DMA Data	151
120	120	NA	LP	LP1_STAT	LP1 Status	152

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
121	121	NA	SPI	SPI0_TXDMA	SPI0 TX DMA Channel	153
122	122	NA	SPI	SPI0_RXDMA	SPI0 RX DMA Channel	154
123	123	NA	SPI	SPI0_STAT	SPI0 Status	155
124	124	NA	SPI	SPI0_ERR	SPI0 Error	156
125	125	NA	SPI	SPI1_TXDMA	SPI1 TX DMA Channel	157
126	126	NA	SPI	SPI1_RXDMA	SPI1 RX DMA Channel	158
127	127	NA	SPI	SPI1_STAT	SPI1 Status	159
128	128	NA	SPI	SPI1_ERR	SPI1 Error	160
129	129	NA	SPI	SPI2_TXDMA	SPI2 TX DMA Channel	161
130	130	NA	SPI	SPI2_RXDMA	SPI2 RX DMA Channel	162
131	131	NA	SPI	SPI2_STAT	SPI2 Status	163
132	132	NA	SPI	SPI2_ERR	SPI2 Error	164
133	133	NA	SPI	SPI3_TXDMA	SPI3TX DMA Channel	165
134	134	NA	SPI	SPI3_RXDMA	SPI3 RX DMA Channel	166
135	135	NA	SPI	SPI3_STAT	SPI3 Status	167
136	136	NA	SPI	SPI3_ERR	SPI3 Error	168
137	137	NA	OSPI	OSPI0_IRQ	OSPI Interrupt request	169
138	138	NA	UART	UART0_TXDMA	UART0 Transmit DMA	170
139	139	NA	UART	UART0_RXDMA	UART0 Receive DMA	171
140	140	NA	UART	UART0_STAT	UART0 Status	172
141	141	NA	UART	UART1_TXDMA	UART1 Transmit DMA	173
142	142	NA	UART	UART1_RXDMA	UART1 Receive DMA	174
143	143	NA	UART	UART1_STAT	UART1 Status	175
144	144	NA	UART	UART2_TXDMA	UART2 Transmit DMA	176
145	145	NA	UART	UART2_RXDMA	UART2 Receive DMA	177
146	146	NA	UART	UART2_STAT	UART2 Status	178
147	147	NA	UART	UART3_TXDMA	UART3 Transmit DMA	179
148	148	NA	UART	UART3_RXDMA	UART3 Receive DMA	180
149	149	NA	UART	UART3_STAT	UART3 Status	181
150	150	NA	TWI	TWI0_DATA	TWI0 Data Interrupt	182

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
151	151	NA	TWI	TWI1_DATA	TWI1 Data Interrupt	183
152	152	NA	TWI	TWI2_DATA	TWI2 Data Interrupt	184
153	153	NA	TWI	TWI3_DATA	TWI3 Data Interrupt	185
154	154	NA	TWI	TWI4_DATA	TWI4 Data Interrupt	186
155	155	NA	TWI	TWI5_DATA	TWI5 Data Interrupt	187
156	156	NA	CNT	CNT0_STAT	CNT0 Status	188
157	157	NA	CTI	ECT_C0_EVT	Reserved	189
158	158	NA	CTI	ECT_C1_EVT	Core 1 CTI Event (CTI1)	190
159	159	NA	CTI	ECT_C2_EVT	Core 2 CTI Event (CTI2)	191
160	160	NA	PKIC	PKIC0_IRQ	Public Key Interrupt (PKA, TRNG, SL)	192
161	161	NA	PKTE	PKTE0_IRQ	Security Packet Engine Interrupt	193
162	162	NA	TRU	TRU0_SLV0	Reserved	194
163	163	NA	TRU	TRU0_SLV1	Reserved	195
164	164	NA	TRU	TRU0_SLV2	Reserved	196
165	165	NA	TRU	TRU0_SLV3	Reserved	197
166	166	NA	FIR	C1_FIR0_DMA	Core 1 FIR0 DMA	198
167	167	NA	FIR	C1_FIR0_STAT	Core 1 FIR0 Status	199
168	168	NA	IIR	C1_IIR0_DMA	Core 1 IIR0 DMA	200
169	169	NA	IIR	C1_IIR0_STAT	Core 1 IIR0 Status	201
170	170	NA	IIR	C1_IIR1_DMA	Core 1 IIR1 DMA	202
171	171	NA	IIR	C1_IIR1_STAT	Core 1 IIR1 Status	203
172	172	NA	IIR	C1_IIR2_DMA	Core 1 IIR2 DMA	204
173	173	NA	IIR	C1_IIR2_STAT	Core 1 IIR2 Status	205
174	174	NA	IIR	C1_IIR3_DMA	Core 1 IIR3 DMA	206
175	175	NA	IIR	C1_IIR3_STAT	Core 1 IIR3 Status	207
176	176	NA	FIR	C2_FIR0_DMA	Core 2 FIR0 DMA	208
177	177	NA	FIR	C2_FIR0_STAT	Core 2 FIR0 Status	209
178	178	NA	IIR	C2_IIR0_DMA	Core 2 IIR0 DMA	210
179	179	NA	IIR	C2_IIR0_STAT	Core 2 IIR0 Status	211

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
180	180	NA	IIR	C2_IIR1_DMA	Core 2 IIR1 DMA	212
181	181	NA	IIR	C2_IIR1_STAT	Core 2 IIR1 Status	213
182	182	NA	IIR	C2_IIR2_DMA	Core 2 IIR2 DMA	214
183	183	NA	IIR	C2_IIR2_STAT	Core 2 IIR2 Status	215
184	184	NA	IIR	C2_IIR3_DMA	Core 2 IIR3 DMA	216
185	185	NA	IIR	C2_IIR3_STAT	Core 2 IIR3 Status	217
186	186	NA	HADC	HADC0_EVT	HADC0 Interrupt	218
187	187	NA	MLB	MLB0_INT0	MLB0 AHB interrupt 0	219
188	188	NA	MLB	MLB0_INT1	MLB0 AHB Interrupt 1	220
189	189	NA	MLB	MLB0_STAT	MLB0 Status	221
190	190	NA	CRC	CRC0_ERR	CRC0 Error	222
191	191	NA	CRC	CRC1_ERR	CRC1 Error	223
192	192	NA	CRC	MDMA0_SRC	MDMA Source 0 (Enh BW DMA)/CRC0 In	224
193	193	NA	CRC	MDMA0_DST	MDMA Dest 0 (Enh BW DMA)/CRC0 Out	225
194	194	NA	CRC	MDMA1_SRC	MDMA Source 1 (Enh BW DMA)/CRC1 In	226
195	195	NA	CRC	MDMA1_DST	MDMA Dest 1 (Enh BW DMA)/CRC1 Out	227
196	196	NA	CRC	CRC0_DCNTEXP	CRC0 Datacount expiration	228
197	197	NA	CRC	CRC1_DCNTEXP	CRC1 Datacount expiration	229
198	198	NA	CRC	CRC2_ERR	CRC2 Error	230
199	199	NA	CRC	CRC3_ERR	CRC3 Error	231
200	200	NA	CRC	MDMA4_SRC	MDMA Source 4 (Enh BW DMA)/CRC2 In	232
201	201	NA	CRC	MDMA4_DST	MDMA Dest 4 (Enh BW DMA)/CRC2 Out	233
202	202	NA	CRC	MDMA5_SRC	MDMA Source 5 (Enh BW DMA)/CRC3 In	234
203	203	NA	CRC	MDMA5_DST	MDMA Dest 5 (Enh BW DMA)/CRC3 Out	235

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
204	204	NA	CRC	CRC2_DCNTEXP	CRC2 Datacount expiration	236
205	205	NA	CRC	CRC3_DCNTEXP	CRC3 Datacount expiration	237
206	206	NA	MDMA	MDMA2_SRC	Enh BW DMA Channel 0	238
207	207	NA	MDMA	MDMA2_DST	Enh BW DMA Channel 1	239
208	208	NA	MDMA	MDMA3_SRC	Max BW DMA Channel 0	240
209	209	NA	MDMA	MDMA3_DST	Max BW DMA Channel 1	241
210	210	NA	EMDMA	EMDMA0_DONE	EMDMA0 DMA Done	242
211	211	NA	EMDMA	EMDMA1_DONE	EMDMA1 DMA Done	243
212	212	NA	MDMA	MDMA7_SRC	Max BW DMA 1 Channel 0	244
213	213	NA	MDMA	MDMA7_DST	Max BW DMA 1 Channel 1	245
214	214	NA	MDMA	MDMA6_SRC	Enh BW DMA 1 Channel 0	246
215	215	NA	MDMA	MDMA6_DST	Enh BW DMA 1 Channel 1	247
216	216	NA	SPU	SPU0_INT	SPU0 Event	248
217	217	NA	SMPU	SMPU0_AGGR_INT	SMPU Aggregated Event	249
218	218	NA	EPPI	EPPI0_CH0_DMA	EPPI0 DMA Channel 0	250
219	219	NA	EPPI	EPPI0_CH1_DMA	EPPI0 DMA Channel 1	251
220	220	NA	EPPI	EPPI0_STAT	EPPI0 Status	252
221	221	NA	EMAC	EMAC0_STAT	EMAC0 Status	253
222	222	NA	EMAC	EMAC0_PWR	EMAC0 Power	254
223	223	NA	EMAC	EMAC0_DMA0	EMAC0 DMA0	255
224	224	NA	EMAC	EMAC0_DMA1	EMAC0 DMA1	256
225	225	NA	EMAC	EMAC0_DMA2	EMAC0 DMA2	257
226	226	NA	EMAC	EMAC0_MAC	EMAC0 MAC	258
227	227	NA	EMAC	EMAC1_STAT	EMAC1 Status	259
228	228	NA	EMAC	EMAC1_PWR	EMAC1 Power	260
229	229	NA	EMAC	EMAC1_DMA	EMAC1 DMA	261
230	230	NA	EMAC	EMAC1_MAC	EMAC1 MAC	262
231	231	NA	CAN	CAN0_WU_IRQ	CAN0 wakeup interrupts	263
232	232	NA	RE-SERVED	RESERVED0	Reserved	264

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
233	233	NA	CAN	CAN0_IRQ	CAN0 interrupts	265
234	234	NA	RE-SERVED	RESERVED1	Reserved	266
235	235	NA	CAN	CAN0_MSG_IRQ	CAN0 message receive/transmit interrupt	267
236	236	NA	CAN	CAN1_WU_IRQ	CAN1 wakeup interrupts	268
237	237	NA	RE-SERVED	RESERVED2	Reserved	269
238	238	NA	CAN	CAN1_IRQ	CAN1 interrupts	270
239	239	NA	RE-SERVED	RESERVED3	Reserved	271
240	240	NA	CAN	CAN1_MSG_IRQ	CAN1 message receive/transmit interrupt	272
241	241	NA	USB	USB0_INT	USB0 Interrupt	273
242	242	NA	TRU	TRU0_SLV4	TRU0 Interrupt 4	274
243	243	NA	TRU	TRU0_SLV5	TRU0 Interrupt 5	275
244	244	NA	TRU	TRU0_SLV6	TRU0 Interrupt 6	276
245	245	NA	TRU	TRU0_SLV7	TRU0 Interrupt 7	277
246	246	NA	TRU	TRU0_SLV8	TRU0 Interrupt 8	278
247	247	NA	TRU	TRU0_SLV9	TRU0 Interrupt 9	279
248	248	NA	TRU	TRU0_SLV10	TRU0 Interrupt 10	280
249	249	NA	TRU	TRU0_SLV11	TRU0 Interrupt 11	281
		Reserved				Reserved
251	251	0	FIR	C1_FIR0_BUS_ERR	Core 1 FIR0 bus error	283
251	252	1	IIR	C1_IIR0_BUS_ERR	Core 1 IIR0 bus error	284
251	253	2	IIR	C1_IIR1_BUS_ERR	Core 1 IIR1 bus error	285
251	254	3	IIR	C1_IIR2_BUS_ERR	Core 1 IIR2 bus error	286
251	255	4	IIR	C1_IIR3_BUS_ERR	Core 1 IIR3 bus error	287
251	256	5	FIR	C2_FIR0_BUS_ERR	Core 2 FIR0 bus error	288
251	257	6	IIR	C2_IIR0_BUS_ERR	Core 2 IIR0 bus error	289
251	258	7	IIR	C2_IIR1_BUS_ERR	Core 2 IIR1 bus error	290

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
251	259	8	IIR	C2_IIR2_BUS_ERR	Core 2 IIR2 bus error	291
251	260	9	IIR	C2_IIR3_BUS_ERR	Core 2 IIR3 bus error	292
252	261	0	SPI	SPI0_TXDMA_ERR	SPI0 TX DMA Channel Error	293
252	262	1	SPI	SPI0_RXDMA_ERR	SPI0 RX DMA Channel Error	294
252	263	2	SPI	SPI1_TXDMA_ERR	SPI1 TX DMA Channel Error	295
252	264	3	SPI	SPI1_RXDMA_ERR	SPI1 RX DMA Channel Error	296
252	265	4	SPI	SPI2_TXDMA_ERR	SPI2 TX DMA Channel Error	297
252	266	5	SPI	SPI2_RXDMA_ERR	SPI2 RX DMA Channel Error	298
252	267	6	SPI	SPI3_TXDMA_ERR	SPI3 TX DMA Channel Error	299
252	268	7	SPI	SPI3_RXDMA_ERR	SPI3 RX DMA Channel Error	300
252	269	8	UART	UART0_TXDMA_ERR	UART0 Transmit DMA Error	301
252	270	9	UART	UART0_RXDMA_ERR	UART0 Receive DMA Error	302
252	271	10	UART	UART1_TXDMA_ERR	UART1 Transmit DMA Error	303
252	272	11	UART	UART1_RXDMA_ERR	UART1 Receive DMA Error	304
252	273	12	UART	UART2_TXDMA_ERR	UART2 Transmit DMA Error	305
252	274	13	UART	UART2_RXDMA_ERR	UART2 Receive DMA Error	306
252	275	14	UART	UART3_TXDMA_ERR	UART3 Transmit DMA Error	307
252	276	15	UART	UART3_RXDMA_ERR	UART3 Receive DMA Error	308
252	277	16	LP	LP0_DMA_ERR	LP0 DMA Data Error	309
252	278	17	LP	LP1_DMA_ERR	LP1 DMA Data Error	310
252	279	18	EPPI	EP-PI0_CH0_DMA_ERR	EPPI0 DMA Channel 0 Error	311
252	280	19	EPPI	EP-PI0_CH1_DMA_ERR	EPPI0 DMA Channel 1 Error	312
253	281	0	CRC	MDMA0_SRC_ERR	Enh BW Source 0 DMA Channel Error	313
253	282	1	CRC	MDMA0_DST_ERR	Enh BW Dest 0 DMA Channel Error	314
253	283	2	CRC	MDMA1_SRC_ERR	Enh BW Source 1 DMA Channel Error	315
253	284	3	CRC	MDMA1_DST_ERR	Enh BW Dest 1 DMA Channel Error	316

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
253	285	4	MDMA	MDMA2_SRC_ERR	Enh BW DMA Channel 0 Error	317
253	286	5	MDMA	MDMA2_DST_ERR	Enh BW DMA Channel 1 Error	318
253	287	6	MDMA	MDMA3_SRC_ERR	Max BW DMA Channel 0 Error	319
253	288	7	MDMA	MDMA3_DST_ERR	Max BW DMA Channel 1 Error	320
253	289	8	CRC	MDMA4_SRC_ERR	Enh BW Source 4 DMA Channel Error	321
253	290	9	CRC	MDMA4_DST_ERR	Enh BW Dest 4 DMA Channel Error	322
253	291	10	CRC	MDMA5_SRC_ERR	Enh BW Source 5 DMA Channel Error	323
253	292	11	CRC	MDMA5_DST_ERR	Enh BW Dest 5 DMA Channel Error	324
253	293	12	MDMA	MDMA6_SRC_ERR	Enh BW DMA Channel 4 Error	325
253	294	13	MDMA	MDMA6_DST_ERR	Enh BW DMA Channel 5 Error	326
253	295	14	MDMA	MDMA7_SRC_ERR	Max BW DMA Channel 4 Error	327
253	296	15	MDMA	MDMA7_DST_ERR	Max BW DMA Channel 5 Error	328
254	297	0	SPORT	SPORT0_A_DMA_ERR	SPORT0 Channel A DMA Error	329
254	298	1	SPORT	SPORT0_B_DMA_ERR	SPORT0 Channel B DMA Error	330
254	299	2	SPORT	SPORT1_A_DMA_ERR	SPORT1 Channel A DMA Error	331
254	300	3	SPORT	SPORT1_B_DMA_ERR	SPORT1 Channel B DMA Error	332
254	301	4	SPORT	SPORT2_A_DMA_ERR	SPORT2 Channel A DMA Error	333
254	302	5	SPORT	SPORT2_B_DMA_ERR	SPORT2 Channel B DMA Error	334
254	303	6	SPORT	SPORT3_A_DMA_ERR	SPORT3 Channel A DMA Error	335
254	304	7	SPORT	SPORT3_B_DMA_ERR	SPORT3 Channel B DMA Error	336
254	305	8	SPORT	SPORT4_A_DMA_ERR	SPORT4 Channel A DMA Error	337
254	306	9	SPORT	SPORT4_B_DMA_ERR	SPORT4 Channel B DMA Error	338
254	307	10	SPORT	SPORT5_A_DMA_ERR	SPORT5 Channel A DMA Error	339
254	308	11	SPORT	SPORT5_B_DMA_ERR	SPORT5 Channel B DMA Error	340
254	309	12	SPORT	SPORT6_A_DMA_ERR	SPORT6 Channel A DMA Error	341
254	310	13	SPORT	SPORT6_B_DMA_ERR	SPORT6 Channel B DMA Error	342
254	311	14	SPORT	SPORT7_A_DMA_ERR	SPORT7 Channel A DMA Error	343

Table 5-5: Combined SEC and GIC Interrupt List (Continued)

SEC ID	Interrupt Number SHARC+	SEA ID	Module	SEC/GIC Interrupt Name	SEC/GIC Interrupt Description	Interrupt Number Arm
254	312	15	SPORT	SPORT7_B_DMA_ERR	SPORT7 Channel B DMA Error	344
255	313	0	SWU	SWU0_EVT	SWU0 Event SMC	345
255	314	1	SWU	SWU1_EVT	SWU1 Event CL2_0	346
255	315	2	SWU	SWU2_EVT	SWU2 Event DL2_0	347
255	316	3	SWU	SWU7_EVT	SWU7 Event SHARC0 S1	348
255	317	4	SWU	SWU8_EVT	SWU8 Event SHARC0 S2	349
255	318	5	SWU	SWU9_EVT	SWU9 Event SHARC1 S1	350
255	319	6	SWU	SWU10_EVT	SWU10 Event SHARC2 S2	351
255	320	7	SWU	SWU11_EVT	SWU11 Event SMMR	352
255	321	8	SWU	SWU12_EVT	SWU12 Event SPI2/OSPI	353
255	322	9	SWU	SWU13_EVT	SWU13 Event DMC0	354
255	323	10	SWU	SWU3_EVT	SWU3 Event CL2_1	355
255	324	11	SWU	SWU4_EVT	SWU4 Event DL2_1	356
255	325	12	SWU	SWU5_EVT	SWU5 Event CL2_2	357

SEC Definitions

The event controller uses the following definitions.

System Events

System source indications including interrupts and faults.

System Source

Point of origin of system event.

SID (Identification, unique)

Source numeric identifier for each system source connected to the SEC.

SSI

SEC source interface, system event source control, and status subblock of the SEC.

SCI

SEC core interface, core interface subblock of the SEC

SPR

SEC prioritizer determines the highest priority pending interrupt and the highest priority active interrupt. The SPR provides these interrupts in the appropriate registers of the SCI for the priority and nesting model of the SCI.

SFI

SEC Fault Interface, fault management subblock of the SEC.

SEA

SEC system event aggregator generates an aggregate request to the SEC based on the active events at a given time.

SEC System Event Aggregator (SEA)

System event sources request event service from the SEC. In order to reduce the number of required SSI channels in the SEC, the System Event Aggregator (SEA) acts as an intermediate aggregator between the source and the SEC. All unique system events have dedicated inputs to the SEA. The SEA generates an aggregate request to the SEC based on which events are active at a given time.

To determine the active event for a given SEC ID, read the `SEC_SSTAT[n].CHID` bit field. The *Combined SEC and GIC Interrupt List* table shows five SEAs connected to SEC ID 251-255. All the sources grouped over the SEA are error events for a set of peripherals like the FIR-IIR, SPI-UART-LP-EPPI, MDMA, SPORT, and SWU. An error event from a peripheral is only expected when that peripheral is enabled. To mask SEA events from sources, disable the source peripheral conditions. There are no registers to mask events from the SEA, but there is a provision to mask the complete group output at the SEC.

Events are handled with a fixed priority order, on a first-come, first-served basis. If multiple events corresponding to the same SEC ID are active at the same time, the priority level is order dependent with the lower SEA IDs having the higher priority. The SEA events are level-based, so they remain high until the events are cleared at the source ensuring no event is missed. There is no support for nesting in a group since they all have same the SEC ID. An SEA event must be handled completely before addressing the next event.

SEC Block Diagram

The *SEC Block Diagram* shows the event management architecture.

System sources connect to the SEC through the SSI. Each core has a dedicated SCI. The SFI provides fault action connections to the rest of the system.

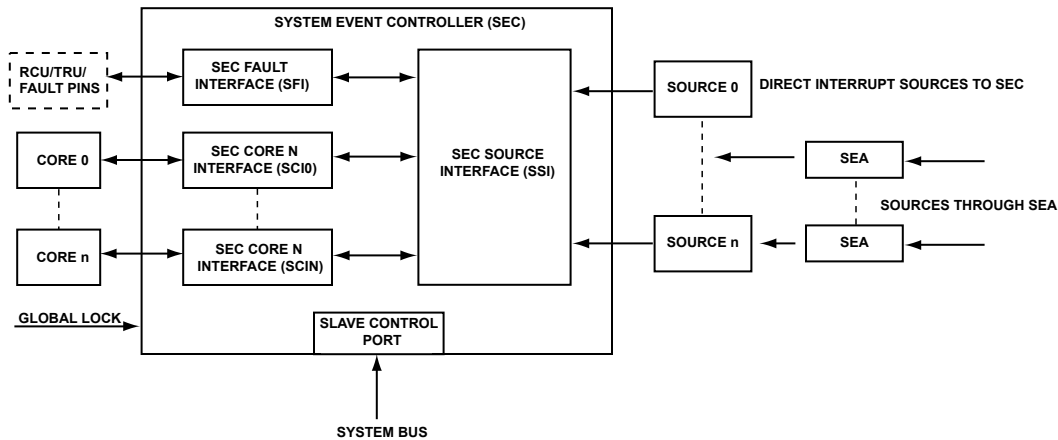


Figure 5-2: SEC Block Diagram

As shown in the figure, the SEC has three blocks for event management. The SFI monitors and manages any fault event triggered from various fault input sources. System interrupt sources are routed to the SFI through SEC source interface (SSI).

SEC Fault Interface (SFI)

The SFI manages fault events and associated actions. The fault management support provided in the SEC helps satisfy the safety requirements of various applications. The SSI provides the highest priority pending source that is enabled as a fault. The SFI captures this value and enables a countdown, and once the countdown expires, takes the prescribed action.

Fault actions which can be configured, as shown in *SFI Block Diagram*, include

- Trigger Output
- System Reset
- Fault Output
 - Computer Operating Properly (COP) mode
 - Fault Mode

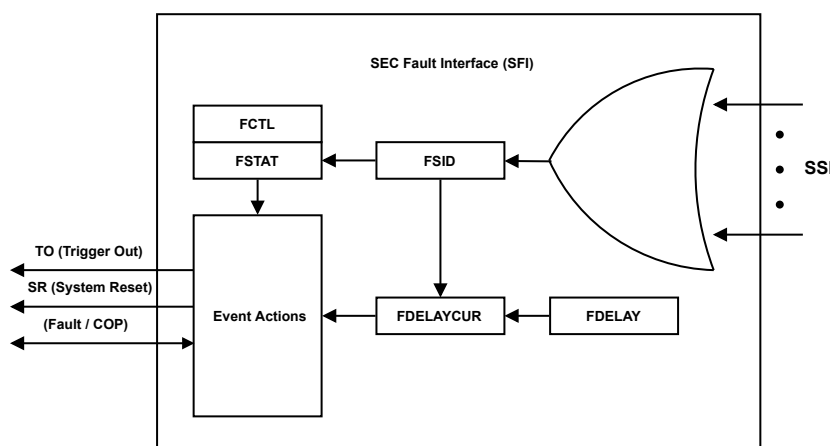


Figure 5-3: SFI Block Diagram

Fault Management

System sources can be enabled as fault sources in the `SEC_SCTL[n]` register. When a source enabled as a fault moves to pending, it is forwarded to the SFI as a fault indication. The pending bit (`SEC_FSTAT.PND`) indicates a source has signaled a fault assertion but it has not yet triggered the event actions (if delay is enabled). The SEC fault interface sets the `SEC_FSTAT.PND` bit when the fault source ID register (`SEC_FSID`) is updated on assertion of a fault source input. The system source pending triggers a fault pending and after a programmable delay the fault moves to active. Event actions then execute if appropriate action is not taken by the core. The `SEC_FSTAT.ACT` bit indicates that the SEC has received a fault source input, the delay has expired, and the fault actions are enabled.

The `SEC_FSTAT.NPND` bit indicates if one or more sources have signaled a fault assertion, but the input has not yet triggered the fault pending detection in the SEC fault interface. The SEC sets the `SEC_FSTAT.NPND` bit when the fault interface detects assertion of any enabled fault source input, while either the `SEC_FSTAT.PND` or `SEC_FSTAT.ACT` bits are set. The SEC clears the `SEC_FSTAT.NPND` bit when there are no fault sources waiting.

A fault indication from an external device can also be detected on sampling the fault signals. When a fault is detected the `SEC_FSTAT.ACT` and `SEC_FSID.FEXT` bits are set. The assertion of either signal results in a fault input detection.

The `SEC_FEND` register receives a fault end indication from the core. The core writes the SID of the fault to the `SEC_FEND` register. If the SID matches the value in the `SEC_FSID` register, the `SEC_FSTAT.PND` and `SEC_FSTAT.ACT` bits are cleared.

SEC Core Interface (SCI)

The SCI manages communication between the corresponding core and the SEC. The SEC prioritizer (SPR) of the SCI receives pending, active, and priority information from the SSI for each system event source assigned to this SCI. The SPR determines the highest-priority pending system event and the SCI determines whether it propagates to the core. The SCI maintains the coherency for the system event service model implemented on the connected core.

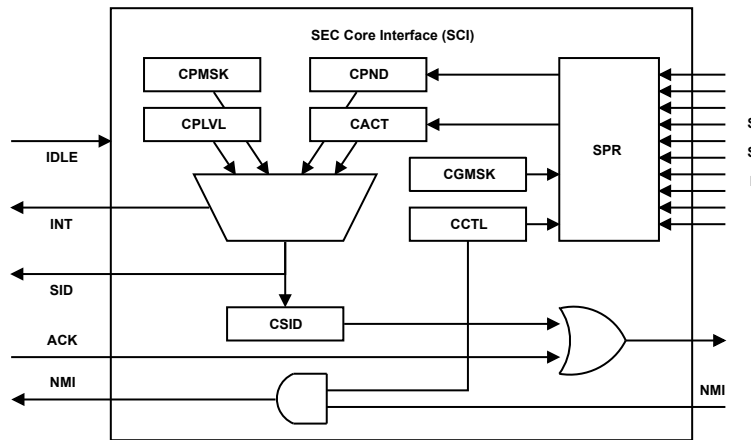


Figure 5-4: SCI Overview Block Diagram

SEC Source Interface (SSI)

The SSI manages all of the system event sources. It maintains the status of each source in the corresponding `SEC_SSTAT[n]` register. The corresponding `SEC_SCTL[n]` register manages the control of each source. A pending and enabled event passes its indication and priority to the SCI to which it is assigned for further processing.

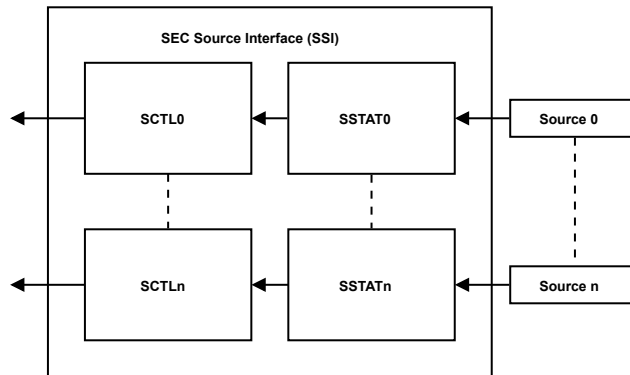


Figure 5-5: SSI Overview Block Diagram

SEC Architectural Concepts

The following sections describe SEC architectural features.

System Interrupt Acknowledge

A system interrupt acknowledge occurs when the core provides an indication that it has acquired the SID of the interrupt last issued by the SEC. The SEC core interface option allows generation by:

- A slave port write to the `SEC_CSID[n]` register.
- The assertion of an input acknowledge signal (the connected core generates the signal).

System Interrupt Groups

System sources can be assigned to groups using the `SEC_SCTL[n].GRP` bit field. Source groups allow fast context switching for system interrupts at each SCI. The `SEC_CGMSK[n]` register allows quick masking of interrupt groups of unlimited size with a single write operation.

System Interrupt Flow

An enabled and asserted system interrupt source is latched at the SSI and routed to the appropriate SCI based on the core target select (`SEC_SCTL[n].CTG`) bit field setting. The SEC priority ordering determines the highest priority pending system interrupt and the SCI updates the `SEC_CPND[n].SID` and `SEC_CACT[n].PRIO` bit field values. The SCI compares the `SEC_CPND[n]` register value against the highest priority active source in the `SEC_CACT[n]` register).

The priority level register (`SEC_CPLVL[n]`) determines how many of the MSBs the SEC uses in the comparison. The priority mask register (`SEC_CPMSK[n]`) and the group mask register (`SEC_CGMSK[n]`) determines which pending interrupt sources participate. If the `SEC_CPND[n]` register value is a higher priority (lower value) than the priority of the `SEC_CACT[n]` register from the comparison based on the `SEC_CPLVL[n]` register, the system interrupt output is asserted. The source ID register (`SEC_CSID[n]`) is updated with the `SEC_CPND[n].SID` bit field value and forwarded to the connected core.

After the core provides an interrupt acknowledgment, the interrupt source is active, until the SEC completes interrupt service with a write to the `SEC_END.SID` bit field with the same value. Note the following:

- Interrupt acknowledgement occurs with an MMR write of the `SEC_CSID[n]` register or the core version of the `SEC_CSID[n]` register.
- Interrupt active status indication is `SEC_SSTAT[n].ACT==1`.

The following sequence shows the example flow for a single interrupt.

1. The SEC compares the `SEC_CPND[n]` register value to the `SEC_CACT[n]` register value. If the interrupt in the `SEC_CPND[n]` register is higher priority, continue.
2. The SEC copies the `SEC_CPND[n]` register value to the `SEC_CSID[n]` register and asserts the interrupt signal.
3. The core reads the `SEC_CSID[n]` register (or core version).
4. The core writes to the `SEC_CSID[n]` register (or core version, asserts the acknowledge signal).
5. The SEC deasserts the interrupt signal and clears the `SEC_SSTAT[n].PND` bit and sets the `SEC_SSTAT[n].ACT` bit of the source going active.
6. The core writes the `SEC_CSID[n]` of the active interrupt to the `SEC_END` register.
7. The SEC clears the `SEC_SSTAT[n].ACT` bit of the source being ended.

The following sequence shows the example flow for interrupt nesting where interrupt A is a lower priority and occurs earlier than interrupt B.

1. The SEC compares the `SEC_CPND[n]` (A) register value to the `SEC_CACT[n]` register and if the interrupt in the `SEC_CPND[n]` register is a higher priority, continue.
2. The SEC copies `SEC_CPND[n]` (A) register to the `SEC_CSID[n]` register and asserts the interrupt signal.
3. The core reads the `SEC_CSID[n]` (A) register (or core version).
4. The core writes to the `SEC_CSID[n]` register (or core version, asserts the acknowledge signal).
5. The SEC deasserts the INT signal and clears the `SEC_SSTAT[n].PND` bit and sets the `SEC_SSTAT[n].ACT` bit of the source (A) going active.
6. The SEC compares the `SEC_CPND[n]` (B) register value to the `SEC_CACT[n]` (A) register value. If the `SEC_CACT[n]` (B) register value is a higher priority, continue.
7. The SEC copies the `SEC_CPND[n]` (B) register value to `SEC_CSID[n]` register and asserts the interrupt signal.
8. The core reads the `SEC_CSID[n]` (B) register (or core version).
9. The core writes to the `SEC_CSID[n]` register (or core version, asserts the acknowledge signal).
10. The SEC deasserts the INT signal and clears the `SEC_SSTAT[n].PND` bit and sets the `SEC_SSTAT[n].ACT` bit of the source (B) going active.
11. The core writes the `SEC_CSID[n]` of the active interrupt (B) to the `SEC_END` register.
12. The SEC clears the `SEC_SSTAT[n].ACT` bit of the source (B) being ended.
13. The core writes the `SEC_CSID[n]` of the active interrupt (A) to the `SEC_END` register.
14. The SEC clears the `SEC_SSTAT[n].ACT` bit of the source (A) being ended.

System Interrupt Priorities

Each system interrupt source has its own programmable priority level which is configured using the `SEC_SCTL[n].PRIO` bit field. The SCI evaluates the priority of all pending sources to determine the source of the highest-priority pending system interrupt for forwarding to the attached core. If more than one source of the pending system interrupt has the same priority setting, the SCI chooses the one with the lowest SID. For example, if SID 0, SID 1, and SID 2 are all pending and have the same priority setting, the SCI chooses SID 0 as the highest-priority source.

SEC Error

The processor includes an SEC error (`SEC_GSTAT.ERR`) as a source input to the SEC to allow for handling the error as an interrupt or fault.

SEC Programming Model

Implementing a system interrupt service model using the SEC requires, at a minimum:

- Proper configuration of a system interrupt source (for example a peripheral or DMA)
- A core interrupt or event service model

The core must be configured for response to system interrupts from the SEC. The SEC must be configured to enable and map the system interrupt source to the correct SCI and to forward interrupts to the connected core.

The system interrupt source must be configured to generate interrupt assertions. Alternatively, the processor can use software triggering for interrupt assertion. Software driven interrupts are generated by writing the source ID of the interrupt to be triggered to the `SEC_RAISE` register.

Programming Concepts

The following list provides the basic programming concepts necessary for configuring the SEC.

- Configuring an SSI as a system interrupt for a specific core.
- Configuring an SCI to provide system interrupts to the connected core (See [Configuring a System Source to Interrupt a Core](#)).
- Configuring an SSI as a system fault (See [Configuring a System Source as a Fault](#)).
- Configuring the SFI to manage system faults.

Programming Examples

This section provides example programming tasks that are typical for SEC usage.

Fault Management Interface Programming Model

The SFI interface can be programmed to manage fault events from system sources and associated actions such as issuing a system reset when watchdog expiration event occurs.

1. Set the `SEC_GCTL.EN` bit to enable the SEC.
2. Write to the `SEC_FCTL` register to configure specific fault actions.
 - Trigger Output. Set the `SEC_FCTL.TOEN` bit for the SEC to produce trigger outputs when a fault becomes active. The `SEC_FCTL.TES` bit can be programmed to select the event that directs the SEC to assert trigger output when a fault is pending or active. Configure slaves for SEC fault trigger master output.

NOTE: If the `SEC_FCTL.TOEN` and or the `SEC_FCTL.TES` bits =1 (Trigger Output Enabled and Trigger on Fault Pending), an external fault (if enabled by the `SEC_FCTL.FIEN` bit) will not issue a trigger since Fault Pending is bypassed for external faults.

- System Reset. The Reset Control Unit (RCU) controls how the functional units enter and exit reset. Configure the `RCU_CTL.SRSTREQEN` bit. This bit controls whether the sources of reset are enabled to perform a system reset. To issue a system reset request when a fault becomes active, set the `SEC_FCTL.SREN` bit. The SEC fault system reset delay register (`SEC_FSRDLY`) can be programmed for the delay, if required, from a fault becoming active to system reset request assertion.

- Fault Output. This configuration allows the SEC to indicate the fault status based on the `SEC_FCTL.CMS` bit configuration.
 - Computer Operating Normally (COP) mode. To configure fault output for COP mode, set the `SEC_FCTL.FOEN` bit to enable fault output. Set the `SEC_FCTL.CMS` bit to select COP mode to toggle the fault pin when no fault is active. Program the `SEC_FCOPP` period register with a desired width value for the COP toggled output pin.
 - Fault mode. Set the `SEC_FCTL.FOEN` bit to enable fault output. The `SEC_FCTL.CMS` bit should be set to Fault mode to toggle the fault pin when a fault is active.
- 3. If required, program the Fault Input to sample fault inputs from external devices on fault pins. Configure the `SEC_FCTL.FIEN` bit to enable the SEC to sample a fault input from an external device.

ADDITIONAL INFORMATION: The `SEC_FCTL.FIEN` bit should be set only while the `SEC_FCTL.EN` bit is low. If the `SEC_FCTL.EN` bit is already high and the `SEC_FCTL.FIEN` bit needs to be set, the `SEC_FCTL.EN` bit should be cleared first. Fault input can only be enabled when Fault mode is selected by the `SEC_FCTL.CMS` bit.
- 4. Program the required fault delay to the `SEC_FDLY.COUNT` bit field if a delay between fault source assertion and the fault response is required.
- 5. Configure the `SEC_FCTL` register to enable the SEC.

ADDITIONAL INFORMATION: The `SEC_FCTL.EN` bit should be set only while the `SEC_FSTAT.ACT` bit is low.
- 6. Write to the control register of a specific source register using the `SEC_SCTL[n]` register to enable the source as a fault.

Configuring a System Source to Interrupt a Core

To configure a system source to interrupt a core, the SEC itself must be enabled with the source interface (SSI) and core interface (SCI) properly initialized. Specifically, the SCI must be set up to accept interrupt signaling from the SEC and pass them to the specified core, and the SSI must properly enable each of the peripheral interrupt sources to generate interrupt signals and optionally define a priority scheme that overrides the default priority settings. In summary:

1. Write to the `SEC_GCTL` register to enable the SEC.
2. Write to the appropriate SCI `SEC_CCTL[n]` register to enable SEC interrupts to be sent to that core.
3. Write to the appropriate SSI `SEC_SCTL[n]` register to enable that peripheral as an interrupt source and to set the core target field to map the source to the desired SCI.
4. (Optional) By default, all the SEC interrupts are grouped as a single priority level, so passing of peripheral interrupt requests from the SEC is based solely on the default enumerated source ID. By programming the `SEC_CPLVL[n].PLVL` register, interrupt sources can be grouped into priority levels within the SEC such that arbitration is first performed by source ID within a grouped priority level before proceeding to the next

priority level, thus providing the flexibility to have lower-priority interrupt sources considered before higher-priority sources.

ADDITIONAL INFORMATION: The `SEC_CPMSK[n]` and `SEC_CGMSK[n]` registers must also can be programmed to mask the interrupts based on the customized levels and grouping.

Core/SEC Handshake Requirements to Ensure Proper Interrupt Handling

A specific handshake with the SEC is required to handle interrupts associated with an individual core. The handshake ensures that nested interrupts are properly tracked and that new peripheral interrupts being raised within the SEC are either passed immediately to the core or held off and queued within the SEC for later servicing.

Use the following procedure to write a custom dispatcher inside the Interrupt Service Routine. Note that the core needs to read and acknowledge the `SEC_CSID[n]` register by writing the same value. The core must also write to the `SEC_END` register after the ISR execution completes.

1. Read the `SEC_CSID[n]` register to obtain the source ID of the peripheral interrupt request.
2. Write the read value back to the `SEC_CSID[n]` register to send the acknowledge signal to the SEC that the core has accepted and begun processing the interrupt request.
3. Execute the actual ISR (typically a call to a specific handler function from a look-up table based on the peripheral source ID). Write to the `SEC_GCTL` register to enable the SEC.
4. Write the value of the `SEC_CSID[n]` register of the active interrupt (read in step 1 above) to the `SEC_END` register to signal to the SEC that the interrupt has now been serviced.
5. Return from interrupt.

This procedure allows a higher-priority interrupt raised by the SEC to be serviced by the core after step 2. The SEC knows what it passed to the core because of the write to the `SEC_CSID[n]` register. After the core acknowledges that write, the SEC knows whether or not newly raised peripheral interrupts are a higher priority than the highest-priority interrupt currently being processed by the core.

- If higher priority, the SEC pushes the current `SEC_CSID[n]` value to an internal stack, writes the new `SEC_CSID[n]` value, and asserts a new SEC interrupt request.
- If lower priority, the SEC queues the interrupt until the core writes to the `SEC_END` register with the source ID of the higher-priority interrupt, confirming that it was fully processed.

At this point the `SEC_CSID[n]` value is popped from the internal stack and any pending peripheral interrupt requests are arbitrated before the SEC writes the new `SEC_CSID[n]` value and asserts a new interrupt request. At the same time the core self-nests the latched SEC interrupt requests as needed. When a higher-priority interrupt is presented to the core the write to the `SEC_END` register in the SEC handler epilog code guarantees that each nested level has the required handshake to signal to the SEC block that each individual source ID interrupt request is fully serviced. See the *SHARC+ Core Programming Reference* for more details about SEC handler code.

Configuring a System Source as a Fault

Use the following procedure configure a system source as a fault.

1. Write to the `SEC_GCTL` register to enable the SEC.
2. Write to the `SEC_FCTL` register to configure specific fault actions.
3. Write to the `SEC_FDLY` bit field to specify fault delay.
4. Write to the control register of a specific source to enable the source as a fault.

Configuring the WDOG Expiry Event to Issue a System Reset

Use the following procedure to configure the WDOG timer to issue a system reset.

1. Configure the `SEC_GCTL` register to enable the SEC.

ADDITIONAL INFORMATION:

```
*pREG_SEC0_GCTL = BITM_SEC_GCTL_EN;
```

2. Configure the `SEC_FCTL` register to choose the Fault response mode. In the following code example, the system reset is issued.

ADDITIONAL INFORMATION:

```
*pREG_RCU0_CTL |= BITM_RCU_CTL_SRSTREQEN;
*pREG_SEC0_FCTL |= BITM_SEC_FCTL_SREN;
```

3. Configure the `SEC_SCTL[n].SEN` and `SEC_SCTL[n].FEN` bits in the registers to determine how the fault source is handled. To configure the WDOG as the fault source, program the register. The program can configure any interrupt as the fault source by programming the corresponding register.

ADDITIONAL INFORMATION:

```
*pREG_SEC0_SCTL3 = BITM_SEC_SCTL_FEN|BITM_SEC_SCTL_SEN;
```

ADDITIONAL INFORMATION: The SEC ID corresponding to WDOG0 is 3, as indicated in the [Combined SEC and GIC Interrupt List](#).

4. Write to the enable bit.

ADDITIONAL INFORMATION:

```
*pREG_SEC0_FCTL |= BITM_SEC_FCTL_EN;
```

SEC Programming Restrictions

Setting the `SEC_FCTL.EN` bit while the `SEC_FSTAT.ACT` bit is high can result in unpredictable behavior. To avoid this issue, set the `SEC_FCTL.EN` bit while the `SEC_FSTAT.ACT` bit is low. The `SEC_FSTAT.ACT` bit is only set when the `SEC_FCTL.EN` bit is high. Therefore, the problem can only occur if the `SEC_FCTL.EN` bit transitions from 1 to 0 and then to 1 again.

Writing to `SEC_FEND` to end a fault with both the `SEC_FCTL.FOEN` bit and the `SEC_FCTL.FIEN` bit set can result in erroneous external fault detection. If this operation (ending a fault) and configuration (fault input and fault output enabled) are required by the application, clear the `SEC_FCTL.FOEN` bit prior to writing to `SEC_FEND`. The recommended sequence for ending a fault with the `SEC_FCTL.FIEN` or `SEC_FCTL.FOEN==1` is as follows:

1. Clear the `SEC_FCTL.FOEN` bit.
2. Write to the `SEC_FEND` register.
3. Set the `SEC_FCTL.FOEN` bit.

ADSP-2159x SEC Register Descriptions

System Event Controller (SEC) contains the following registers.

Table 5-6: ADSP-2159x SEC Register List

Name	Description
<code>SEC_CACT[n]</code>	SCI Active Register n
<code>SEC_CCTL[n]</code>	SCI Control Register n
<code>SEC_CGMSK[n]</code>	SCI Group Mask Register n
<code>SEC_CPLVL[n]</code>	SCI Priority Level Register n
<code>SEC_CPMSK[n]</code>	SCI Priority Mask Register n
<code>SEC_CPND[n]</code>	Core Pending Register n
<code>SEC_CSID[n]</code>	SCI Source ID Register n
<code>SEC_CSTAT[n]</code>	SCI Status Register n
<code>SEC_END</code>	Global End Register
<code>SEC_FCOPP</code>	Fault COP Period Register
<code>SEC_FCOPP_CUR</code>	Fault COP Period Current Register
<code>SEC_FCTL</code>	Fault Control Register
<code>SEC_FDLY</code>	Fault Delay Register
<code>SEC_FDLY_CUR</code>	Fault Delay Current Register
<code>SEC_FEND</code>	Fault End Register
<code>SEC_FSID</code>	Fault Source ID Register
<code>SEC_FSRDLY</code>	Fault System Reset Delay Register
<code>SEC_FSRDLY_CUR</code>	Fault System Reset Delay Current Register
<code>SEC_FSTAT</code>	Fault Status Register
<code>SEC_GCTL</code>	Global Control Register

Table 5-6: ADSP-2159x SEC Register List (Continued)

Name	Description
SEC_GSTAT	Global Status Register
SEC_RAISE	Global Raise Register
SEC_SCTL[n]	Source Control Register n
SEC_SSTAT[n]	Source Status Register n

SCI Active Register n

The SEC SCI active interrupt register (`SEC_CACT[n]`) contains the source ID and priority of the highest priority active interrupt detected by the SEC prioritizer.

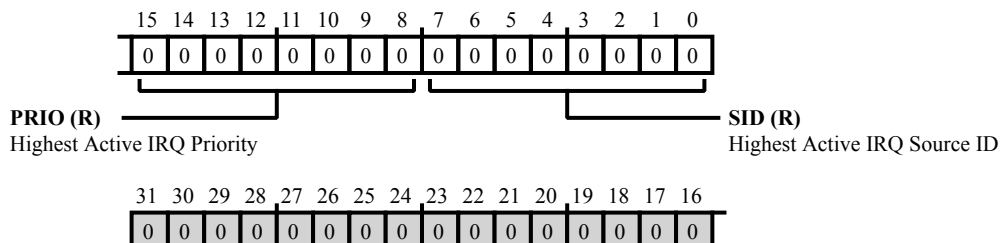


Figure 5-6: SEC_CACT[n] Register Diagram

Table 5-7: SEC_CACT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/NW)	PRIO	Highest Active IRQ Priority. The <code>SEC_CACT[n].PRIO</code> indicates the priority value of the highest priority active interrupt for core n.
7:0 (R/NW)	SID	Highest Active IRQ Source ID. The <code>SEC_CACT[n].SID</code> identifies the source ID value of the highest priority active interrupt for core n.

SCI Control Register n

The SEC control register (`SEC_CCTL[n]`) contains SCI control bits for all system sources.

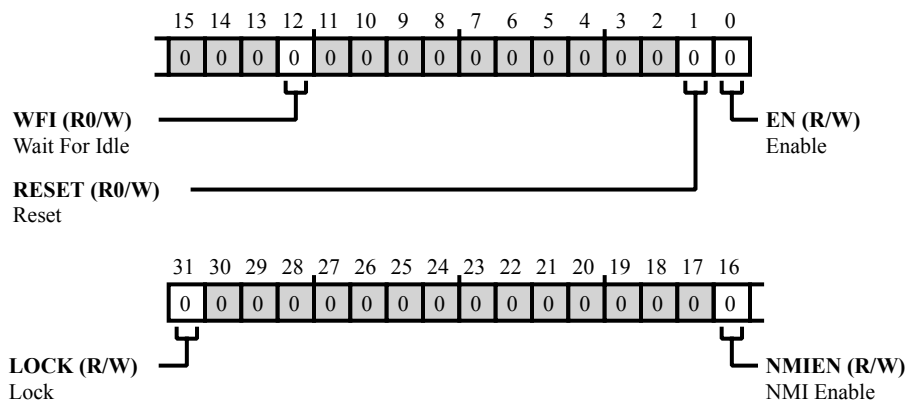


Figure 5-7: SEC_CCTL[n] Register Diagram

Table 5-8: SEC_CCTL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>SEC_CCTL[n].LOCK</code> bit is enabled, the <code>SEC_CCTL[n]</code> register is read only.
		0 Unlock
		1 Lock
16 (R/W)	NMIEN	NMI Enable.
		The <code>SEC_CCTL[n].NMIEN</code> bit controls NMI propagation to the core. When the <code>SEC_CCTL[n].NMIEN</code> bit is enabled, the SCI allows NMIs to propagate to the core for servicing.
		0 Disable
		1 Enable
12 (R0/W)	WFI	Wait For Idle.
		When set, the <code>SEC_CCTL[n].WFI</code> bit forces the SCI to wait for indication of core idle before the SCI resumes activity.
		0 No Action
		1 Wait for Idle

Table 5-8: SEC_CCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R0/W)	RESET	Reset. When set, the SEC_CCTL[n].RESET bit resets all SCI registers to their default values.
		0 No Action
		1 Reset
0 (R/W)	EN	Enable. The SEC_CCTL[n].EN bit controls operation of the SCI. Clearing the SEC_CCTL[n].EN bit halts the execution of the SCI without resetting status registers. (The INT signal to a core is not affected.) Setting the SEC_CCTL[n].EN bit enables the SCI to begin or resume operation with the current configuration and status.
		0 Disable
		1 Enable

SCI Group Mask Register n

The SEC SCI group mask register ([SEC_CGMSK\[n\]](#)) contains selections for a group mask, an ungroup mask, and a register lock. This register contains the system interrupt group masks for the connected core. The core uses the [SEC_CGMSK\[n\].UGRP](#) and [SEC_CGMSK\[n\].GRP](#) fields to mask (disable) interrupts from the specified groups.

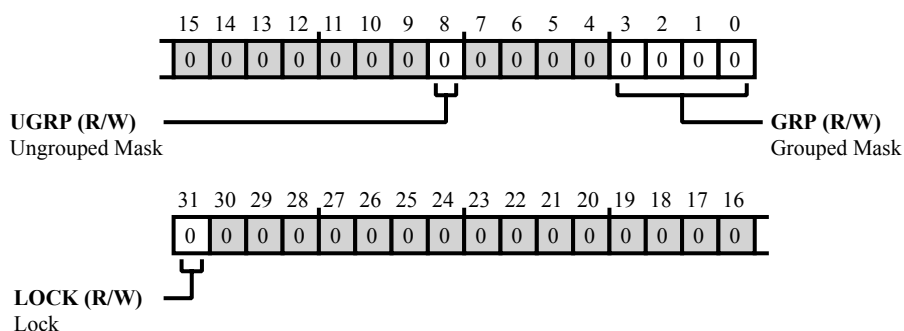


Figure 5-8: SEC_CGMSK[n] Register Diagram

Table 5-9: SEC_CGMSK[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (SPU_CTL.GLCK bit =1) and the SEC_CGMSK[n].LOCK bit is enabled, the SEC_CGMSK[n] register is read only.
		0 Unlock
		1 Lock
8 (R/W)	UGRP	Ungrouped Mask. The SEC_CGMSK[n].UGRP bit masks interrupts (if set) for the ungrouped interrupt sources for core n.
		0 Unmask Ungrouped Sources
		1 Mask Ungrouped Sources
3:0 (R/W)	GRP	Grouped Mask. The SEC_CGMSK[n].GRP field selects a group of interrupt sources to mask for core n. (For more information about interrupt source groups, see the SEC_SCTL[n] register description.)
		0 No groups masked
		1 Mask group 0
		2 Mask group 1
		3 Mask groups 0, 1

Table 5-9: SEC_CGMSK[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		4 Mask group 2
		5 Mask groups 0, 2
		6 Mask groups 1, 2
		7 Mask groups 0, 1, 2
		8 Mask group 3
		9 Mask groups 0, 3
		10 Mask groups 1, 3
		11 Mask groups 0, 1, 3
		12 Mask groups 2, 3
		13 Mask groups 0, 2, 3
		14 Mask groups 1, 2, 3
		15 Mask groups 0, 1, 2, 3

SCI Priority Level Register n

The SEC SCI priority level register ([SEC_CPLVL\[n\]](#)) contains selections for priority levels and a register lock. This register is used to divide the total number of priority levels into sub-levels. The sub-level priority resolution provides the tie breaker for simultaneously pending interrupts assigned to the same level.

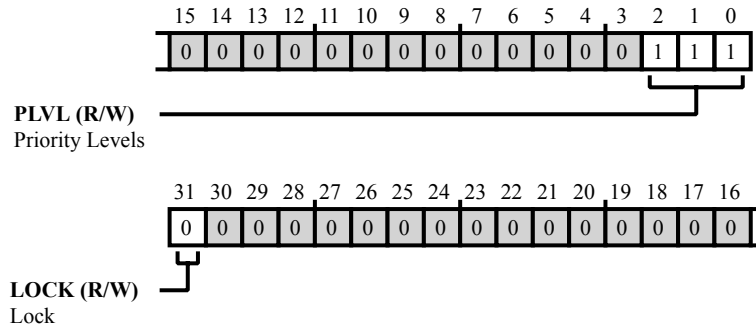


Figure 5-9: SEC_CPLVL[n] Register Diagram

Table 5-10: SEC_CPLVL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock is enabled (<code>SPU_CTL . GLCK</code> bit =1) and the <code>SEC_CPLVL[n] . LOCK</code> bit is enabled, the SEC_CPLVL[n] register is read only.
		0 Unlock 1 Lock
2:0 (R/W)	PLVL	Priority Levels.
		The <code>SEC_CPLVL[n] . PLVL</code> field serves to divide the total number of interrupt priority levels into sub-levels. The sub-level priority resolution provides the tie breaker for simultaneously pending interrupts assigned to the same interrupt level. The sub-level priority value specifies the number of MSBs (minus 1) designated to interrupt levels, while the remaining LSBs are designated for sub-level specification.
		For example, if the <code>SEC_CPLVL[n] . PLVL</code> field is set to two, the result is four priority levels are specified, because only the two MSBs are used for preemption evaluation. The remaining bits of the priority setting are used for sub-level prioritization.
		0 1 MSBs (2 priority levels)
		1 2 MSBs (4 priority levels)
		2 3 MSBs (8 priority levels)
		3 4 MSBs (16 priority levels)
		4 5 MSBs (32 priority levels)
		5 6 MSBs (64 priority levels)

Table 5-10: SEC_CPLVL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		6	7 MSBs (128 priority levels)
		7	8 MSBs (256 priority levels)

SCI Priority Mask Register n

The SEC SCI priority mask register (`SEC_CPMSK[n]`) contains the SCI priority mask for core n and includes a register lock.

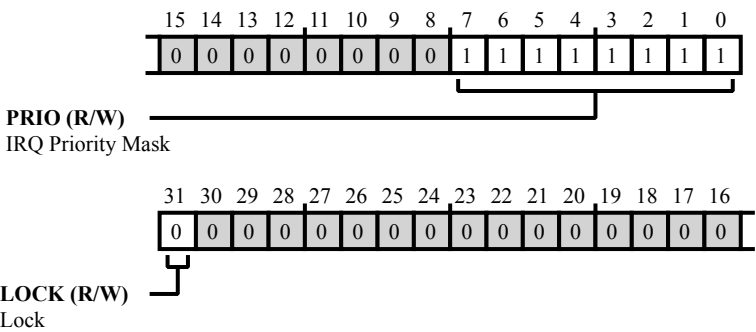


Figure 5-10: SEC_CPMSK[n] Register Diagram

Table 5-11: SEC_CPMSK[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>SEC_CPMSK[n].LOCK</code> bit is enabled, the <code>SEC_CPMSK[n]</code> register is read only.
		0 Unlock
		1 Lock
7:0 (R/W)	PRIO	IRQ Priority Mask. The <code>SEC_CPMSK[n].PRIO</code> contains the system interrupt priority mask for core n. The core uses the <code>SEC_CPMSK[n].PRIO</code> field to mask (block) interrupts below the specified level.
		0 Priority level 0 (highest)
		1-254
		255 Priority level 255 (lowest)

Core Pending Register n

The SCI pending interrupt register (`SEC_CPND[n]`) contains the source ID and priority of the highest priority pending interrupt detected by the SEC prioritizer.

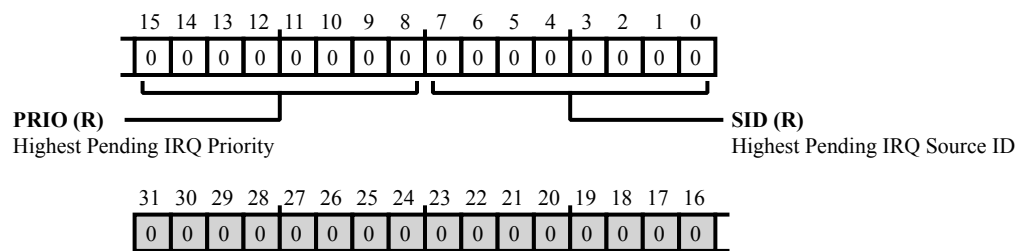


Figure 5-11: SEC_CPND[n] Register Diagram

Table 5-12: SEC_CPND[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/NW)	PRIO	Highest Pending IRQ Priority. The <code>SEC_CPND[n] . PRIO</code> indicates the priority value of the highest priority pending interrupt for core n.
7:0 (R/NW)	SID	Highest Pending IRQ Source ID. The <code>SEC_CPND[n] . SID</code> identifies the source ID value of the highest priority pending interrupt for core n.

SCI Source ID Register n

The SCI source ID register (`SEC_CSID[n]`) contains the source ID of the interrupt last issued to core n. The `SEC_CSID[n]` register value is loaded by the SCI when a system interrupt indication is sent to core n. The SCI does not change the `SEC_CSID[n]` until after the interface receives an interrupt acknowledge from core n. Writing to the `SEC_CSID[n]` register generates an interrupt acknowledge, but does not update the value in the register.

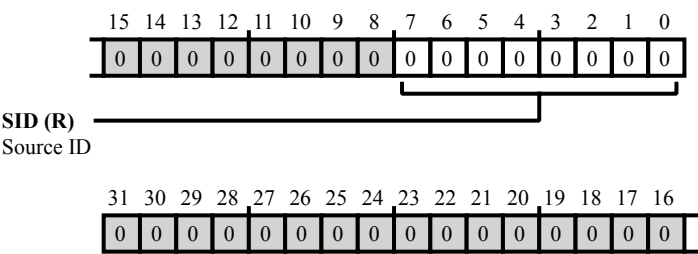


Figure 5-12: SEC_CSID[n] Register Diagram

Table 5-13: SEC_CSID[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	SID	Source ID. The <code>SEC_CSID[n] . SID</code> bit is the source ID of the interrupt last issued to core n.

SCI Status Register n

The SCI status register ([SEC_CSTAT\[n\]](#)) contains status bits, indicating the operational status of the SCI.

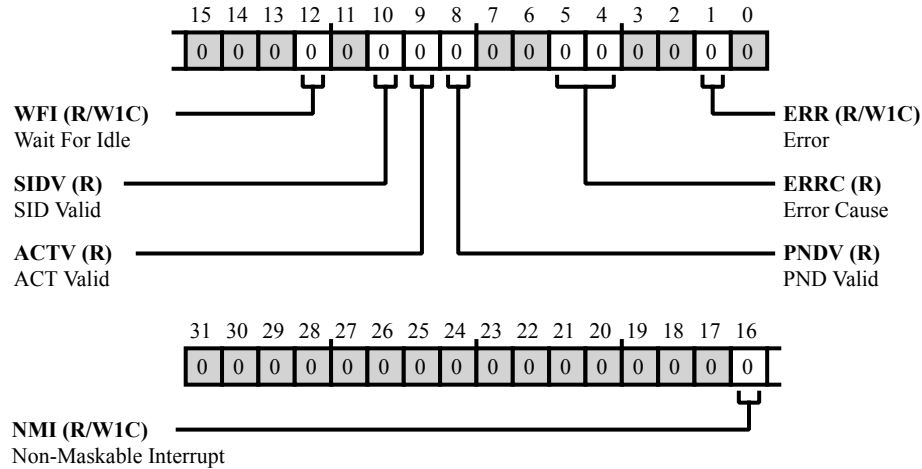


Figure 5-13: SEC_CSTAT[n] Register Diagram

Table 5-14: SEC_CSTAT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W1C)	NMI	Non-Maskable Interrupt. The SEC_CSTAT[n].NMI bit indicates whether an NMI has occurred since the bit was last cleared.
		0 No NMI Occurred
		1 NMI Occurred
12 (R/W1C)	WFI	Wait For Idle. The SEC_CSTAT[n].WFI bit indicates (if set) that the SCI is temporarily disabled, pending a core idle indication. This bit is set when SEC_CCTL[n].WFI is set.
		0 Not Waiting
		1 Waiting
10 (R/NW)	SIDV	SID Valid. The SEC_CSTAT[n].SIDV bit indicates (if set) that the current value in the SEC_CSID[n] register is valid. The SCI sets the SEC_CSTAT[n].SIDV bit when the updating the SEC_CSID[n] register with a new value. The SEC_CSTAT[n].SIDV bit is cleared when the SEC_CSID[n] register is written. This status indication may be used to extract all pending interrupts in a single interrupt service routine.
		0 Invalid
		1 Valid

Table 5-14: SEC_CSTAT[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/NW)	ACTV	ACT Valid. The SEC_CSTAT[n].ACTV bit indicates (if set) that the current value in the SEC_CACT[n] register is valid. The SCI sets the SEC_CSTAT[n].ACTV bit when updating the SEC_CACT[n] registers with a new value. The SEC_CSTAT[n].ACTV bit is cleared when the SEC_CSID[n] register is written.
		0 Invalid
		1 Valid
8 (R/NW)	PNDV	PND Valid. The SEC_CSTAT[n].PNDV bit indicates (if set) that the current value in the SEC_CPND[n] register is valid. The SCI sets the SEC_CSTAT[n].PNDV bit when updating the SEC_CPND[n] register with a new value. The SEC_CSTAT[n].PNDV bit is cleared when the SEC_CSID[n] register is written.
		0 Invalid
		1 Valid
5:4 (R/NW)	ERRC	Error Cause. The SEC_CSTAT[n].ERRC bits are updated on assertion of the SEC_CSTAT[n].ERR bit to indicate the SCI error type. SEC_CSTAT[n].ERRC is only updated on the assertion of SEC_CSTAT[n].ERR. Subsequent errors while SEC_CSTAT[n].ERR is asserted do not update SEC_CSTAT[n].ERRC.
		0 Reserved
		1 Acknowledge Error. SCI has received an acknowledge without a pending, unacknowledged interrupt present.
		2 Reserved
		3 Reserved
1 (R/W1C)	ERR	Error. The SEC_CSTAT[n].ERR bit indicates that an error has occurred in the SCI. When SEC_CSTAT[n].ERR is set, the SCI updates the SEC_CSTAT[n].ERRC field to the value of the corresponding error cause.
		0 No Error
		1 Error Occurred

Global End Register

The SEC global end register (`SEC_END`) contains a source ID interrupt service end field (`SEC_END.SID`). When a core has finished servicing an interrupt, the core writes the `SEC_END.SID` field in the `SEC_END` register. This write causes the SEC to clear the `SEC_SSTAT[n].ACT` bit in the `SEC_SSTAT[n]` register of the corresponding interrupt.

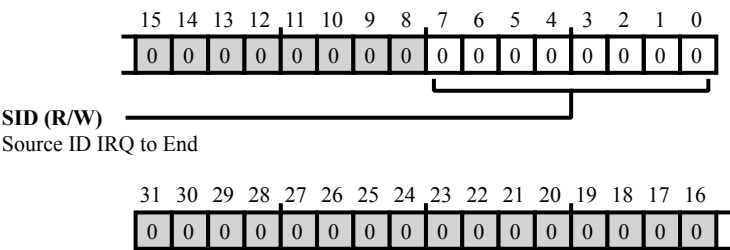


Figure 5-14: SEC_END Register Diagram

Table 5-15: SEC_END Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	SID	Source ID IRQ to End. The SEC_END.SID bit field contains the source ID interrupt service end value.

Fault COP Period Register

The SEC fault COP period register ([SEC_FCOPP](#)) contains the width value (count in (SEC) clock cycles) for the high and low phase of the computer operating properly (COP) toggled output on the COP pin. Note that the actual high/low phase value is the `SEC_FCOPP.COUNT` programmed value plus 1.

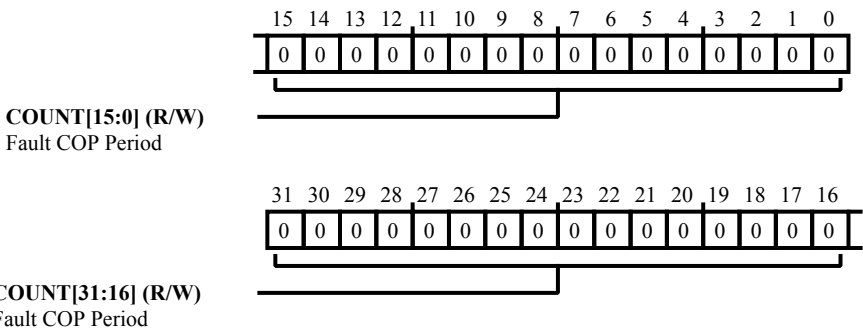


Figure 5-15: SEC_FCOPP Register Diagram

Table 5-16: SEC_FCOPP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	COUNT	Fault COP Period. The <code>SEC_FCOPP.COUNT</code> bit field is the width value for the high and low phase of the computer operating properly (COP) toggled output on the COP pin.

Fault COP Period Current Register

The SEC fault COP period current register (`SEC_FCOPP_CUR`) contains the active count (in (SEC) clock periods) for the current phase (high or low) of the computer operating properly (COP) toggled output on the COP pin. The SEC loads the `SEC_FCOPP_CUR` register from the `SEC_FCOPP` register when the `SEC_FCOPP_CUR.COUNT` field is cleared and the SEC is in COP mode (`SEC_FCTL.CMS` bit =1). The SEC decrements the `SEC_FCOPP_CUR` count each (SEC) clock cycle while `SEC_FCTL.CMS` is set and the `SEC_FSTAT.ACT` bit is not set.

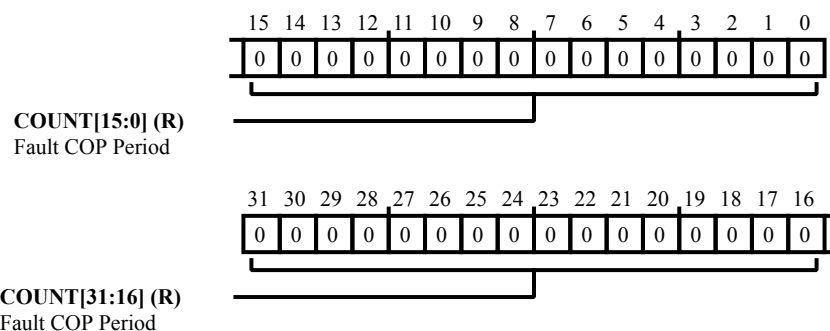


Figure 5-16: SEC_FCOPP_CUR Register Diagram

Table 5-17: SEC_FCOPP_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	COUNT	Fault COP Period. The <code>SEC_FCOPP_CUR.COUNT</code> bit field is the active count for the current phase (high or low) of the computer operating properly (COP) toggled output on the COP pin.

Fault Control Register

The SEC fault control register ([SEC_FCTL](#)) contains fault control bits for all SEC channels. This register controls the operation of the System Fault Management Interface (SFI).

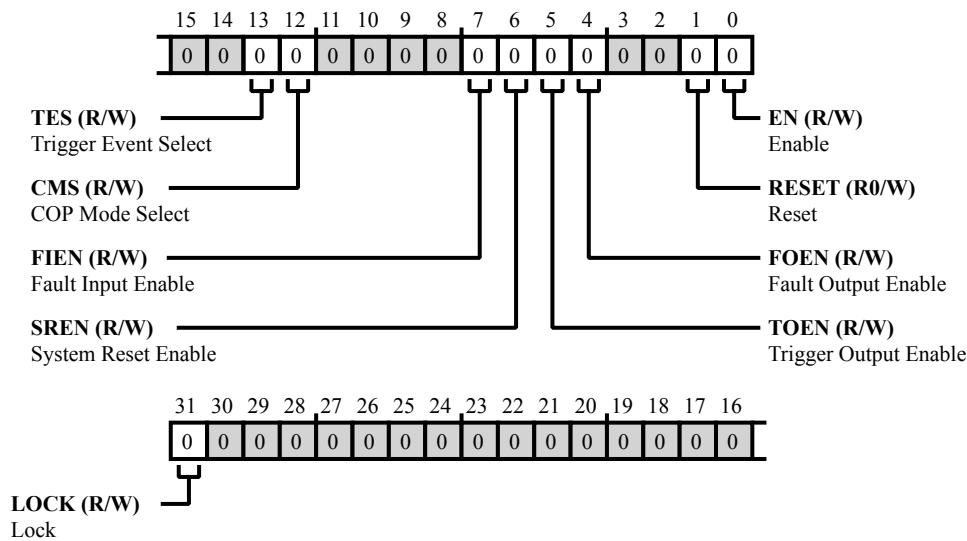


Figure 5-17: SEC_FCTL Register Diagram

Table 5-18: SEC_FCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (SPU_CTL.GLCK bit =1) and the SEC_FCTL.LOCK bit is enabled, the SEC_FCTL register is read only.
		0 UnLock
		1 Lock
13 (R/W)	TES	Trigger Event Select. The SEC_FCTL.TES bit selects the event that directs the SEC to assert trigger output. In fault pending mode, the SEC asserts trigger output when a fault is pending. In fault active mode, the SEC asserts trigger output when a fault is active.
		0 Fault Active Mode
		1 Fault Pending Mode

Table 5-18: SEC_FCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	CMS	COP Mode Select. The SEC_FCTL.CMS selects the SEC mode for handling fault input. In COP mode, the SEC toggles the COP pin to indicate that no fault is active and ceases toggling the pin to indicate that a fault is active. In fault mode, the SEC deasserts the fault pin (=0) and fault_b pin (=1) when no fault is active and asserts the fault pin (=1) and fault_b pin (=0) when a fault is active. Not all processors feature both the fault and fault_b pins. Refer to the product data sheet for details.
		0 Fault Mode
		1 COP Mode
7 (R/W)	FIEN	Fault Input Enable. The SEC_FCTL.FIEN bit enables the SEC to sample fault input. If SEC_FCTL.FIEN is set (=1), a fault indication from an external device sets the SEC_FSTAT.ACT bit and SEC_FSID.FEXT bit.
		0 Disable
		1 Enable
6 (R/W)	SREN	System Reset Enable. The SEC_FCTL.SREN bit enables the SEC to issue a system reset request when a fault becomes active.
		0 Disable
		1 Enable
5 (R/W)	TOEN	Trigger Output Enable. The SEC_FCTL.TOEN bit enables the SEC to produce trigger output when a fault becomes active.
		0 Disable
		1 Enable
4 (R/W)	FOEN	Fault Output Enable. The SEC_FCTL.FOEN bit enables the SEC to indicate fault status, according to the SEC_FCTL.CMS bit configuration.
		0 Disable
		1 Enable
1 (R0/W)	RESET	Reset. Setting the SEC_FCTL.RESET bit resets ALL SEC registers to their default values.
		0 No Action
		1 Reset

Table 5-18: SEC_FCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Enable. The SEC_FCTL.EN bit controls the operational state of the SEC. Clearing the SEC_FCTL.EN bit halts the execution of the SEC without resetting status registers. Setting the SEC_FCTL.EN bit enables the SEC to begin or resume operation with the current configuration and status.
		0 Disable
		1 Enable

Fault Delay Register

The SEC fault delay register ([SEC_FDLY](#)) contains the number (`SEC_FDLY.COUNT` field) of (SEC) clock periods to delay from fault pending to fault active, when actions are enabled.

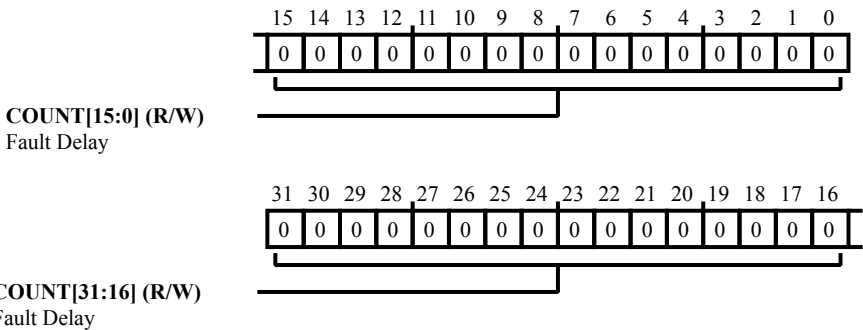


Figure 5-18: SEC_FDLY Register Diagram

Table 5-19: SEC_FDLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	COUNT	Fault Delay. The SEC_FDLY.COUNT bit field is the number of (SEC) clock periods to delay from fault pending to fault active, when actions are enabled.

Fault Delay Current Register

The SEC fault delay current register (`SEC_FDLY_CUR`) contains the active count (`SEC_FDLY_CUR.COUNT` field) in (SEC) clock periods for the delay from fault pending to fault active, when actions are enabled. The count is loaded from the `SEC_FDLY` register when a fault becomes pending (`SEC_FSTAT.PND` bit is set). The SEC decrements the value in `SEC_FDLY_CUR` each (SEC) clock cycle while the `SEC_FSTAT.PND` bit is set.

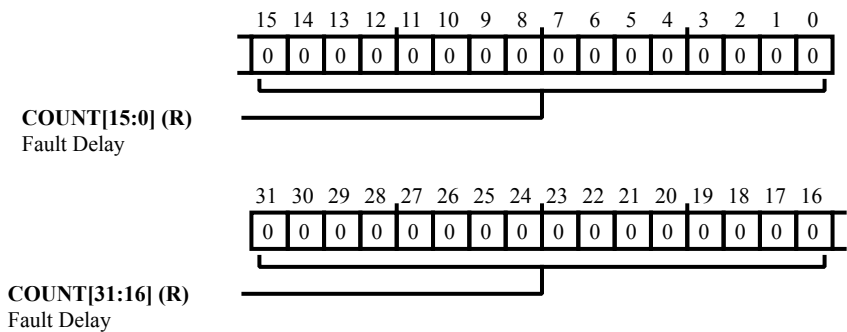


Figure 5-19: SEC_FDLY_CUR Register Diagram

Table 5-20: SEC_FDLY_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	COUNT	Fault Delay. The <code>SEC_FDLY_CUR.COUNT</code> bit field is the active count in (SEC) clock periods for the delay from fault pending to fault active, when actions are enabled.

Fault End Register

The SEC fault end register (**SEC_FEND**) contains fault source ID and internal/external fields. This register receives fault end indication from a core.

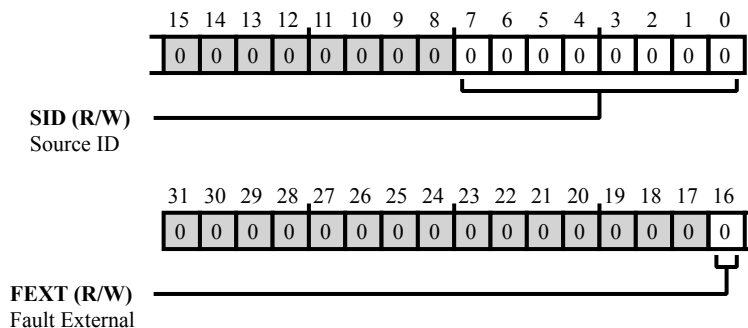


Figure 5-20: SEC_FEND Register Diagram

Table 5-21: SEC_FEND Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	FEXT	Fault External. Setting the SEC_FEND.FEXT bit, when the SEC_FEND.SID field is cleared, clears an active fault from an external source.
		0 Fault Internal
		1 Fault External
7:0 (R/W)	SID	Source ID. The SEC_FEND.SID identifies a fault to be ended as indicated to the SEC by the core. The core loads the SEC_FEND.SID field value. If the SEC_FEND.SID value matches the SEC_FSID.SID value, the SEC_FSTAT.PND bit and SEC_FSTAT.ACT bit are cleared.

Fault Source ID Register

The SEC fault source ID register ([SEC_FSID](#)) contains a fault source ID and internal/external fields.

NOTE:These bits are not reset by system reset so that a fault that automatically triggers a system reset to avoid a fault may be analyzed after the reset occurs.

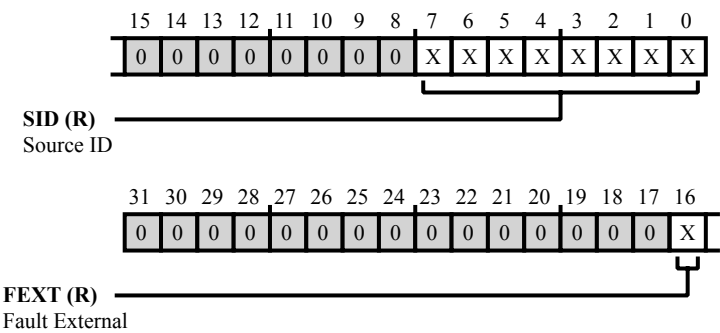


Figure 5-21: SEC_FSID Register Diagram

Table 5-22: SEC_FSID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/NW)	FEXT	Fault External. The SEC_FSID.FEXT bit indicates that the last active fault was asserted by an external device. The SEC sets the SEC_FSID.FEXT bit when the SEC_FSTAT.ACT bit is set by the fault input pins. The SEC_FSID.FEXT bit is cleared when the SEC_FSTAT.ACT bit is set by an internal fault or when the external fault is ended. When the SEC_FSID.FEXT bit is set, the SEC_FSID.SID is cleared.
		0 Fault Internal
		1 Fault External
7:0 (R/NW)	SID	Source ID. The SEC_FSID.SID identifies the fault assertion detected by the SEC fault interface. The SEC loads the SEC_FSID.SID field value when a system fault indication is asserted. The SEC fault interface does not change the SEC_FSID.SID value until the fault is no longer pending or active, as indicated by the SEC_FSTAT.PND bit and SEC_FSTAT.ACT bit being cleared in the SEC_FSTAT register.

Fault System Reset Delay Register

The SEC fault system reset delay register ([SEC_FSRDLY](#)) contains the number (`SEC_FSRDLY.COUNT` field) of (SEC) clock periods for the delay from a fault becoming active to system reset request assertion, if enabled.

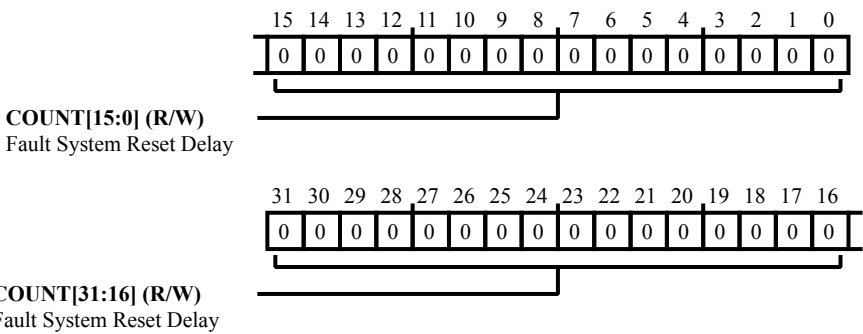


Figure 5-22: SEC_FSRDLY Register Diagram

Table 5-23: SEC_FSRDLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	COUNT	Fault System Reset Delay. The <code>SEC_FSRDLY.COUNT</code> bit field is the number of (SEC) clock periods for the delay from a fault becoming active to system reset request assertion.

Fault System Reset Delay Current Register

The SEC fault system reset delay current register (`SEC_FSRDLY_CUR`) contains the active count (`SEC_FSRDLY_CUR.COUNT` field) in (SEC) clock periods for the delay from fault active to system reset assertion, if enabled. The count is loaded from the `SEC_FSRDLY` register when a fault becomes active (`SEC_FSTAT.ACT` bit is set). The SEC decrements the value in `SEC_FSRDLY_CUR` each (SEC) clock cycle while the `SEC_FSTAT.ACT` bit is set.

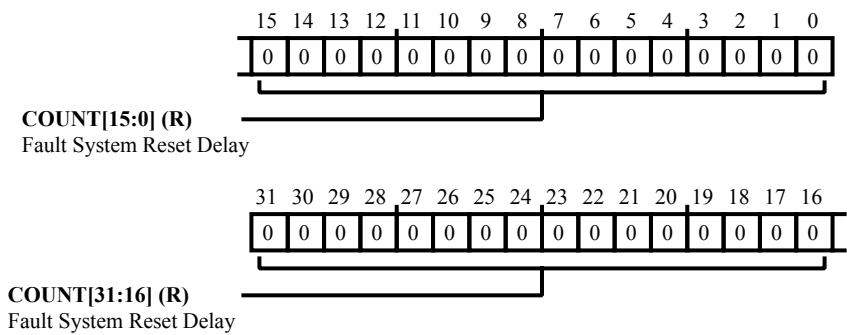


Figure 5-23: SEC_FSRDLY_CUR Register Diagram

Table 5-24: SEC_FSRDLY_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	COUNT	Fault System Reset Delay. The <code>SEC_FSRDLY_CUR.COUNT</code> bit field is the active count in (SEC) clock periods for the delay from fault active to system reset assertion.

Fault Status Register

The SEC fault status register ([SEC_FSTAT](#)) indicates the operational status of the SFI.

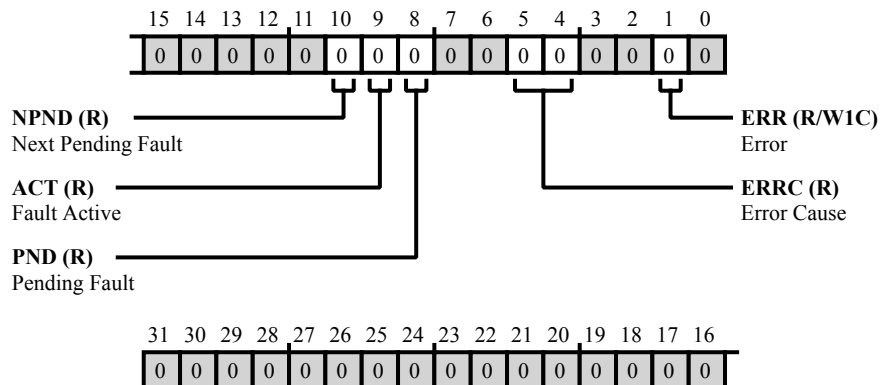


Figure 5-24: SEC_FSTAT Register Diagram

Table 5-25: SEC_FSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/NW)	NPND	Next Pending Fault. The SEC_FSTAT.NPND bit indicates that one or more sources have signaled fault assertion, but the input has not yet triggered the fault pending detection in the SEC fault interface. The SEC sets the SEC_FSTAT.NPND bit when the fault interface detects assertion of any enabled fault source input, while either the SEC_FSTAT.PND or SEC_FSTAT.ACT bits are set. The SEC clears the SEC_FSTAT.NPND bit when there are no fault sources waiting.
		0 Not Pending
		1 Pending
9 (R/NW)	ACT	Fault Active. The SEC_FSTAT.ACT bit indicates that the SEC has received a fault source input, the current fault delay count (in the SEC_FDLY_CUR register) has expired, and the fault actions are enabled. The SEC also sets the SEC_FSTAT.ACT bit on fault input detection if the SEC_FCTL.FIEN bit is set. The SEC_FSTAT.ACT bit is cleared by writing the ID value of the asserted fault from SEC_FSID register to the SEC_FEND register.
		0 No Fault
		1 Active Fault

Table 5-25: SEC_FSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	PND	Pending Fault. The SEC_FSTAT.PND bit indicates a fault source has signaled a fault assertion to the SEC, but the SEC has not yet triggered the event actions due to the delay selected with the SEC_FDLY register. The SEC fault interface sets the SEC_FSTAT.PND bit when the SEC_FSID is updated on assertion of a fault source input. The SEC_FSTAT.PND bit is only set when the SEC_FSTAT.ACT bit is cleared. The SEC updates the SEC_FSID register with the SID value when the SEC_FSTAT.PND bit is set. The SEC_FSTAT.PND bit is cleared <i>either</i> by the SEC fault interface when the current delay count in the SEC_FDLY_CUR register expires <i>or</i> by writing the SEC_FSID.SID field value (which indicates the ID of the asserted fault) to the SEC_FEND register.
		0 Not Pending
		1 Pending
5:4 (R/NW)	ERRC	Error Cause. When the SEC_FSTAT.ERR bit is asserted, the SEC updates SEC_FSTAT.ERRC field to convey the interrupt source error type. When the error type is source overflow, the status indicates that a source signal assertion occurred or an SEC raise operation was attempted while pending was already set. The source overflow is detected when the source is set for edge only. When the error type is end error, the status indicates that an end was received for a source while the SEC_FSTAT.ACT bit was not set.
		0 Source Overflow Error
		1 Reserved
		2 End Error
		3 Reserved
1 (R/W1C)	ERR	Error. The SEC_FSTAT.ERR bit indicates an SEC fault interface error. When SEC_FSTAT.ERR is set, the SEC updates the SEC_FSTAT.ERRC field to indicate the corresponding error cause. When multiple errors occur, the SEC_FSTAT register captures the status for the first error and does not capture subsequent errors until the status is cleared.
		0 No Error
		1 Error Occurred

Global Control Register

The SEC global control register ([SEC_GCTL](#)) provides register locking, reset, and enable for the SEC module.

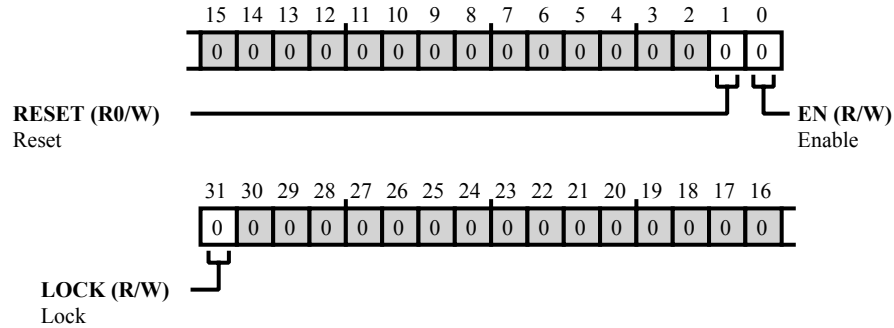


Figure 5-25: SEC_GCTL Register Diagram

Table 5-26: SEC_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (SPU_CTL.GLCK bit =1) and the SEC_GCTL.LOCK bit is enabled, the SEC_GCTL register is read only.
		0 Unlock
		1 Lock
1 (R0/W)	RESET	Reset. The SEC_GCTL.RESET bit is write-1-action and triggers a soft reset to all SEC registers.
		0 No Action
		1 Reset
0 (R/W)	EN	Enable. The SEC_GCTL.EN bit is read/write and must be set for the SEC to begin/resume SEC operation with the current configuration and status. Clearing the SEC_GCTL.EN bit halts the execution of the SFI and all SCIs. All SSIs remain active, along with all error detection, without resetting status registers.
		0 Disable
		1 Enable

Global Status Register

The SEC global status register ([SEC_GSTAT](#)) contains global status bits for the SEC.

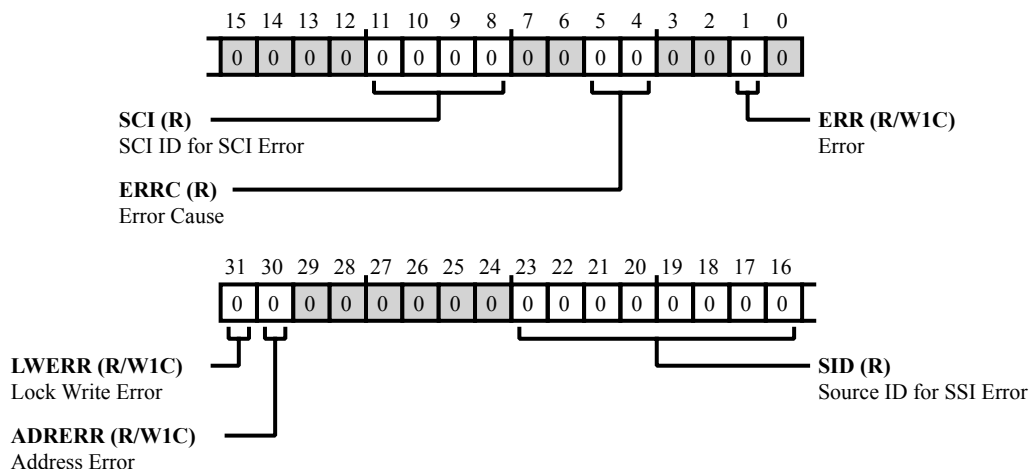


Figure 5-26: SEC_GSTAT Register Diagram

Table 5-27: SEC_GSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Lock Write Error. The SEC_GSTAT.LWERR bit indicates (when set) there was an attempted write to an SEC register while the SEC_GCTL.LOCK bit was set and while the global lock bit was enabled (SPU_CTL.GLCK bit =1). This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
30 (R/W1C)	ADREERR	Address Error. The SEC_GSTAT.ADREERR bit indicates that the SEC generated and address error. This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
23:16 (R/NW)	SID	Source ID for SSI Error. The SEC_GSTAT.SID bits indicate the source ID that generated the last SSI error conveyed in the SEC_GSTAT.ERRC field.
11:8 (R/NW)	SCI	SCI ID for SCI Error. The SEC_GSTAT.SCI bits indicate the number for the specific SCI that generated the last SCI error conveyed in the SEC_GSTAT.ERRC field.

Table 5-27: SEC_GSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5:4 (R/NW)	ERRC	Error Cause. When the SEC updates the SEC_GSTAT.ERR bit, the SEC updates the SEC_GSTAT.ERRC bits to indicate the error type. Note that for SCI errors, the error status represents an OR of all the errors from each SCI. Note that for SSI errors, the error status indicates an error is active for any SSI input. This error is an OR of all the interrupt source errors.
		0 SFI Error
		1 SCI Error
		2 SSI Error
		3 Reserved
1 (R/W1C)	ERR	Error. The SEC_GSTAT.ERR bit indicates an error has occurred in the SEC. When the SEC asserts this bit (=1), the SEC updates the SEC_GSTAT.ERRC field to indicate the corresponding error cause. Even if multiple errors occur, only the first error is captured on assertion of this bit. This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred

Global Raise Register

The SEC global raise register ([SEC_RAISE](#)) contains a source ID event set-to-pending field (`SEC_RAISE.SID`). When a source ID value is written to this field, the SEC raises the source's event status to pending.

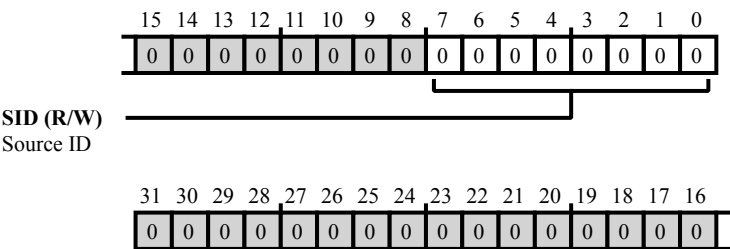


Figure 5-27: SEC_RAISE Register Diagram

Table 5-28: SEC_RAISE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	SID	Source ID. The <code>SEC_RAISE.SID</code> bit field is the source ID of event that is set to pending status.

Source Control Register n

The SEC source control register (`SEC_SCTL[n]`) contains control bits to configure the SEC event sources. This register controls the configuration of the corresponding SEC event source.

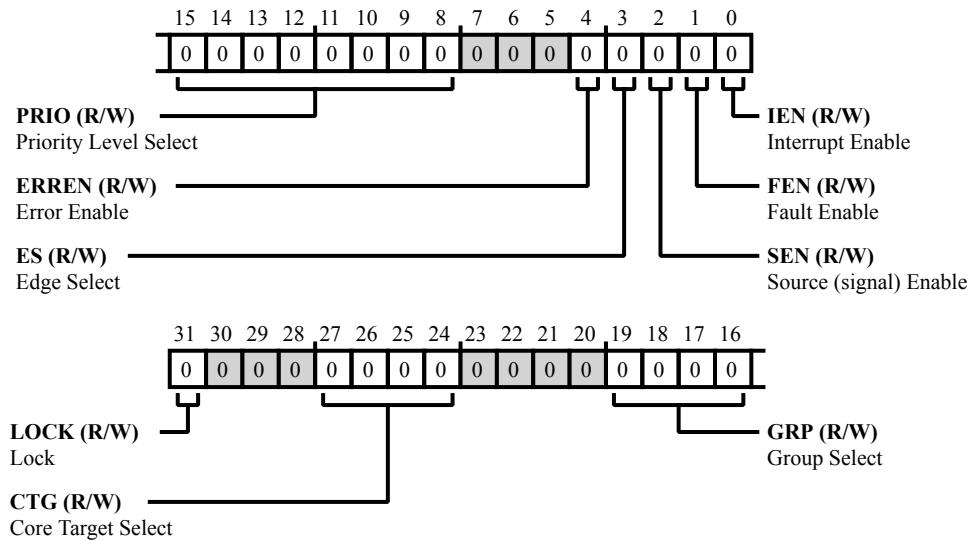


Figure 5-28: SEC_SCTL[n] Register Diagram

Table 5-29: SEC_SCTL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>SEC_SCTL[n].LOCK</code> bit is enabled, the <code>SEC_SCTL[n]</code> register is read only.
		0 Unlock
		1 Lock
27:24 (R/W)	CTG	Core Target Select. The <code>SEC_SCTL[n].CTG</code> bits selects the specific SEC core interface to which the interrupt is mapped. Each system interrupt is mapped uniquely to one specific SEC core interface and (as a result) to a specific core.

Table 5-29: SEC_SCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration				
19:16 (R/W)	GRP	Group Select. The SEC_SCTL[n] .GRP bits each select a specific group for the interrupt. Each system interrupt can be assigned to any combination of groups supported by the SEC_SCTL[n] .GRP field. For example, consider the situation where SEC_SCTL[n] .GRP0 represents interrupt group 0, SEC_SCTL[n] .GRP1 represents interrupt group 1, and so on. One group might be used for all enabled interrupts (for example, group 0) and an additional group might be used for all wakeup interrupts (for example, group 1). This approach supports a model of all interrupts and just the wakeup subset. Before going to idle or sleep, all non-wakeup interrupts can be masked off to allow only wakeup interrupts to be enabled for service. Selecting no group (all SEC_SCTL[n] .GRP bits = 0) places the interrupt source in the category of "un-grouped".				
15:8 (R/W)	PRI0	Priority Level Select. The SEC_SCTL[n] .PRI0 bits sets the relative priority for an interrupt request. A pending interrupt request forwards its SEC_SCTL[n] .PRI0 value to the SEC core interface.				
4 (R/W)	ERREN	Error Enable. The SEC_SCTL[n] .ERREN bit permits the SEC_SSTAT[n] .ERR status bit to be set on error detection. If SEC_SCTL[n] .ERREN is cleared, no errors are detected. <table><tr><td>0</td><td>Disable</td></tr><tr><td>1</td><td>Enable</td></tr></table>	0	Disable	1	Enable
0	Disable					
1	Enable					
3 (R/W)	ES	Edge Select. The SEC_SCTL[n] .ES bit selects the operational and sensitivity mode of the SEC source interface input. <table><tr><td>0</td><td>Level Sensitive</td></tr><tr><td>1</td><td>Edge Sensitive</td></tr></table>	0	Level Sensitive	1	Edge Sensitive
0	Level Sensitive					
1	Edge Sensitive					
2 (R/W)	SEN	Source (signal) Enable. The SEC_SCTL[n] .SEN bit controls whether the system event source input signal may affect the pending status of the source. Clearing the SEC_SCTL[n] .SEN bit disables the source input signal from affecting the pending status. Setting SEC_SCTL[n] .SEN enables the source input signal to affect the pending status. <table><tr><td>0</td><td>Disable</td></tr><tr><td>1</td><td>Enable</td></tr></table>	0	Disable	1	Enable
0	Disable					
1	Enable					

Table 5-29: SEC_SCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	FEN	Fault Enable. The SEC_SCTL[n] . FEN bit controls whether the SEC may forward an interrupt request to the SEC fault interface as a fault source. This bit does not affect the ability of an interrupt source to set an interrupt as pending. The SEC_SCTL[n] . FEN bit only affects whether the pending request may be forwarded to the SEC fault interface.
		0 Disable
		1 Enable
0 (R/W)	IEN	Interrupt Enable. The SEC_SCTL[n] . IEN bit controls whether the SEC may forward an interrupt request to a core for servicing. This bit does not affect the ability of an interrupt source to set an interrupt as pending.
		0 Disable
		1 Enable

Source Status Register n

The SEC event source status register (`SEC_SSTAT[n]`) contains bits indicating the status of the corresponding event source n. An event source may be: pending, active, active and pending, or neither pending nor active.

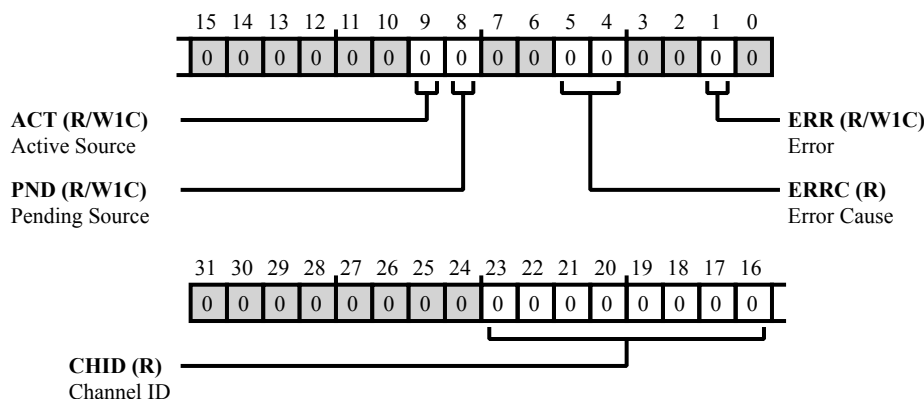


Figure 5-29: SEC_SSTAT[n] Register Diagram

Table 5-30: SEC_SSTAT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/NW)	CHID	Channel ID. The <code>SEC_SSTAT[n].CHID</code> bits indicate the ID of the specific source (from a set of sources sharing one SEC source interface input) that asserted the SEC source interface input. An SEC source interface input may support multiple system sources, in which case the assertion must be qualified by an identifier to determine the channel that generated the assertion. The <code>SEC_SSTAT[n].CHID</code> field provides this value in the form of a numeric reference that is mapped to a specific interrupt source. The prioritization for simultaneously asserted sources is according to ID, with 0 being the highest priority. The <code>SEC_SSTAT[n].CHID</code> is captured when the SEC source interface input is acknowledged.
9 (R/W1C)	ACT	Active Source. The <code>SEC_SSTAT[n].ACT</code> bit indicates the source has been accepted by a core for servicing, but the service is not yet complete. An <code>SEC_SSTAT[n].ACT</code> bit is set by the SEC when the specific system interrupt is acknowledged by the core through the SEC core interface. An <code>SEC_SSTAT[n].ACT</code> bit is cleared by the SEC when the core provides interrupt service end indication for the specific system interrupt through the SEC core interface.
		0 Not Active
		1 Active

Table 5-30: SEC_SSTAT[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	PND	Pending Source. The SEC_SSTAT[n] . PND bit indicates the source has signaled an event request, but the event request has not been (or is not currently being) serviced. A SEC_SSTAT[n] . PND bit is set by the SEC on detection of an assertion of the corresponding system source input. A SEC_SSTAT[n] . PND bit is cleared by the SEC when the specific system event is acknowledged by the core through the SEC core interface or by a W1C operation.
		0 Not Pending
		1 Pending
5:4 (R/NW)	ERRC	Error Cause. When the SEC_SSTAT[n] . ERR bit is asserted, the SEC updates SEC_SSTAT[n] . ERRC field to convey the interrupt source error type. When the error type is source overflow, the status indicates that a source signal assertion occurred or an SEC raise operation was attempted while pending was already set. The source overflow is detected when the source is set for edge only. When the error type is end error, the status indicates that an end was received for a source while the SEC_SSTAT[n] . ACT bit was not set.
		0 Source Overflow Error
		1 Reserved
		2 End Error
		3 Reserved
1 (R/W1C)	ERR	Error. The SEC_SSTAT[n] . ERR bit indicates an error for a specific system interrupt source. When the SEC_SSTAT[n] . ERR bit is set, the SEC updates the SEC_SSTAT[n] . ERRC field to the value of the corresponding error cause. Even if multiple errors occur, only the first error is captured on assertion of the SEC_SSTAT[n] . ERR bit.
		0 No Error
		1 Error Occurred

GIC Overview

The generic interrupt controller (GIC) provides an interface to the uniprocessor Cortex A5 core in the processor and collects up to 325 interrupt requests from all processor system sources. In addition, the GIC also supports eight software generated interrupts that are internal to the Cortex A5 core and not connected to the SECs. All GIC interrupts are also connected to the SEC in the same order.

Each interrupt can be configured as a normal or a secure interrupt. Software force registers and software priority masking are also supported. The "Register Descriptions" section in this chapter provide brief descriptions of these Arm-based registers. For complete information refer to the *Arm® Generic Interrupt Controller Architecture version 1.0 Architecture Specification*.

GIC Functional Description

The GIC splits logically into a GICPORT0 (distributor block) and one GICPORT1 (CPU interface blocks).

General Interrupt Controller Port0 (GIC Distributer)

The distributor block provides a programming interface to perform the following tasks.

- Globally enable the forwarding of interrupts to the CPU interfaces
- Enable or disable each interrupt
- Set the priority level of each interrupt
- Set the target processor list of each interrupt
- Set each peripheral interrupt to be level-sensitive or edge-triggered
- Set each interrupt as either Group 0 or Group 1
- Forward an SGI to one or more target processors

In addition, the Distributor provides:

- Visibility of the state of each interrupt
- A mechanism for software to set or clear the pending state of a peripheral interrupt.

General Interrupt Controller Port1 (GIC CPU)

GICPORT1 (CPU interface) block performs priority masking and preemption handling for a connected processor in the system. GICPORT1 supports 8 SGIs (software generated interrupts) and 262 SPIs (shared peripheral interrupts).

Each CPU interface provides a programming interface to perform the following tasks.

- Enable signaling of interrupt requests to the processor
- Acknowledge interrupts
- Indicate that interrupt processing is complete
- Set interrupt priority masks for the processor
- Define the preemption policy for the processor
- Determine the highest priority pending interrupt for the processor

GIC Block Diagram

The *GIC Block Diagram* shows the event management architecture.

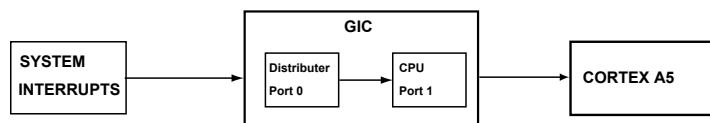


Figure 5-30: GIC Block Diagram

ADSP-2159x GICDST Register Descriptions

GIC Distributor Port (GICDST) contains the following registers.

Table 5-31: ADSP-2159x GICDST Register List

Name	Description
GICDST_EN	GIC Port 0 Enable
GICDST_SGI_PRIO[n]	Software Generated Interrupt Priority Register
GICDST_SPI_PRIO[n]	Shared Peripheral Interrupt Priority Register
GICDST_SGI_ACTIVE	Software Generated Interrupt Active Register
GICDST_SGI_CTL	Software Generated Interrupt Control Register
GICDST_SGI_PND_CLR	Software Generated Interrupt Clear-Pending Register
GICDST_SGI_PND_SET	Software Generated Interrupt Pending Set Register
GICDST_SGI_SECURITY	Software Generated Interrupt Security Register
GICDST_SPI[n]	Shared Peripheral Interrupt Register
GICDST_SPI_ACTIVE[n]	Shared Peripheral Interrupt Active Register
GICDST_SPI_CFG[n]	Shared Peripheral Interrupt Configuration Register
GICDST_SPI_EN_CLR[n]	Shared Peripheral Interrupt Enable Clear Register
GICDST_SPI_EN_SET[n]	Shared Peripheral Interrupt Enable Set Register
GICDST_SPI_PND_CLR[n]	Shared Peripheral Interrupt Pending Clear Register
GICDST_SPI_PND_SET[n]	Shared Peripheral Interrupt Pending Set Register
GICDST_SPI_SECURITY[n]	Shared Peripheral Interrupt Security Register
GICDST_SPI_TRGT[n]	Shared Peripheral Interrupt Processor Targets Register

GIC Port 0 Enable

The `GICDST_EN` register enables global monitoring of the peripheral interrupt signals and forwarding pending interrupts to the CPU interfaces.

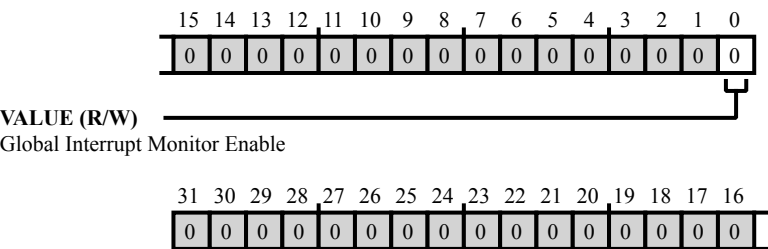


Figure 5-31: GICDST_EN Register Diagram

Table 5-32: GICDST_EN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	VALUE	Global Interrupt Monitor Enable. The <code>GICDST_EN.VALUE</code> bit field enables global monitoring of the peripheral interrupt signals and forwarding pending interrupts to the CPU interfaces.

Software Generated Interrupt Priority Register

The `GICDST_SGI_PRIO[n]` register provides the 8-bit priority field for each interrupt supported by the GIC. This field stores the priority of the corresponding interrupt.

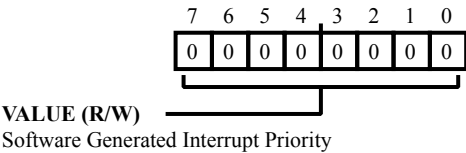


Figure 5-32: GICDST_SGI_PRIO[n] Register Diagram

Table 5-33: GICDST_SGI_PRIO[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	Software Generated Interrupt Priority. The <code>GICDST_SGI_PRIO[n].VALUE</code> bit field contains the 8-bit priority field for each interrupt supported by the GIC. This field stores the priority of the corresponding interrupt.

Shared Peripheral Interrupt Priority Register

The `GICDST_SPI_PRIO[n]` registers provide an 8-bit priority field for each interrupt supported by the GIC. This field stores the priority of the corresponding interrupt.

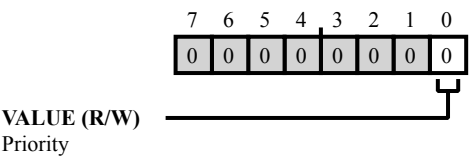


Figure 5-33: GICDST_SPI_PRIO[n] Register Diagram

Table 5-34: GICDST_SPI_PRIO[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	VALUE	Priority. The <code>GICDST_SPI_PRIO[n].VALUE</code> bit field stores the priority of the corresponding interrupt (byte offset 3 to Byte offset 0).

Software Generated Interrupt Active Register

The `GICDST_SGI_ACTIVE` registers provide a Set-active bit for each interrupt that the GIC supports. Writing to a Set-active bit Activates the corresponding interrupt. These registers are used when preserving and restoring GIC state.

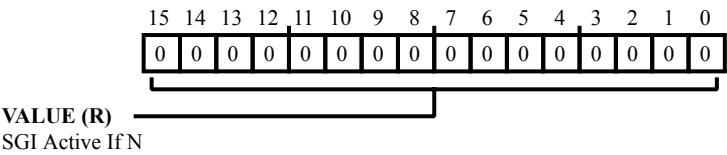


Figure 5-34: GICDST_SGI_ACTIVE Register Diagram

Table 5-35: GICDST_SGI_ACTIVE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	SGI Active If N. The <code>GICDST_SGI_ACTIVE.VALUE</code> bit field provides a Set-active bit for each interrupt that the GIC supports.

Software Generated Interrupt Control Register

The `GICDST_SGI_CTL` register controls the generation of SGIs. It is implementation defined whether this register has any effect when the forwarding of interrupts by Distributor is disabled by the `GICD_CTLR` settings.

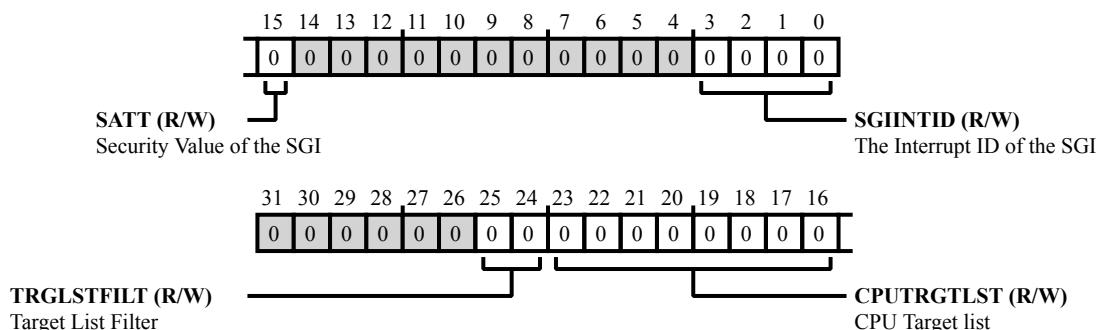


Figure 5-35: GICDST_SGI_CTL Register Diagram

Table 5-36: GICDST_SGI_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:24 (R/W)	TRGLSTFILT	Target List Filter. The <code>GICDST_SGI_CTL.TRGLSTFILT</code> bit field determines how the distributor must process the requested SGI.
		0 Forward the interrupt to the CPU interfaces specified in the <code>CPUTargetList</code> field
		1 Forward the interrupt to all CPU interfaces except that of the processor that requested the interrupt
		2 Forward the interrupt only to the CPU interface of the processor that requested the interrupt
		3 Reserved
23:16 (R/W)	CPUTRGTLST	CPU Target list. When the <code>GICDST_SGI_CTL.CPUTRGTLST</code> bit field TargetList Filter = 0b00, defines the CPU interfaces to which the Distributor must forward the interrupt. Each bit of the <code>GICDST_SGI_CTL.CPUTRGTLST</code> bit field refers to the corresponding CPU interface, for example <code>CPUTargetList[0]</code> corresponds to CPU interface 0. Setting a bit to 1 indicates that the interrupt must be forwarded to the corresponding interface. If this field is 0x00 when TargetListFilter is 0b00, the Distributor does not forward the interrupt to any CPU interface.

Table 5-36: GICDST_SGI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SATT	Security Value of the SGI. The GICDST_SGI_CTL.SATT bit is implemented only if the GIC includes the Security Extensions. This field is writable only by a Secure access. Any Non-secure write to the GICD_SGIR generates an SGI only if the specified SGI is programmed as Group 1, regardless of the value of bit[15] of the write.
		0 Forward the SGI specified in the SGIINTID field to a specified CPU interface only if the SGI is configured as Group 0 on that interface.
		1 Forward the SGI specified in the SGIINTID field to a specified CPU interfaces only if the SGI is configured as Group 1 on that interface.
3:0 (R/W)	SGIINTID	The Interrupt ID of the SGI.

Software Generated Interrupt Clear-Pending Register

The `GICDST_SGI_PND_CLR` register provides a clear pending bit for each interrupt supported by the GIC. Writing 1 to a clear-pending bit clears the pending status of the corresponding peripheral interrupt. Reading a bit identifies whether the interrupt is pending.

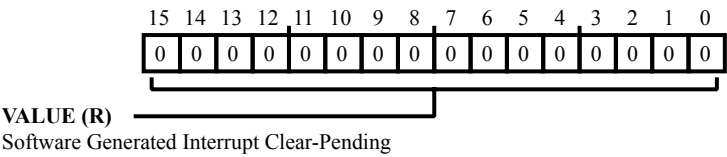


Figure 5-36: GICDST_SGI_PND_CLR Register Diagram

Table 5-37: GICDST_SGI_PND_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	Software Generated Interrupt Clear-Pending. Writing 1 to a clear-pending bit in the <code>GICDST_SGI_PND_CLR.VALUE</code> bit field clears the pending status of the corresponding peripheral interrupt. Reading a bit identifies whether the interrupt is pending.

Software Generated Interrupt Pending Set Register

The `GICDST_SGI_PND_SET` register provides a set-pending bit for each interrupt supported by the GIC.

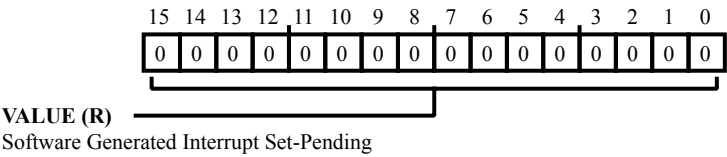


Figure 5-37: GICDST_SGI_PND_SET Register Diagram

Table 5-38: GICDST_SGI_PND_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	Software Generated Interrupt Set-Pending. Writing 1 to a Set-pending bit in the <code>GICDST_SGI_PND_SET.VALUE</code> bit field sets the status of the corresponding peripheral interrupt to pending. Reading a bit identifies whether the interrupt is pending.

Software Generated Interrupt Security Register

The `GICDST_SGI_SECURITY` registers provide a status bit for each interrupt supported by the GIC. Each bit controls whether the corresponding interrupt is in Group 0 or Group 1. Typically, when used with a processor that implements the ARM Security Extensions, Group 0 interrupts are Secure interrupts, and Group 1 interrupts are Non-secure interrupts,

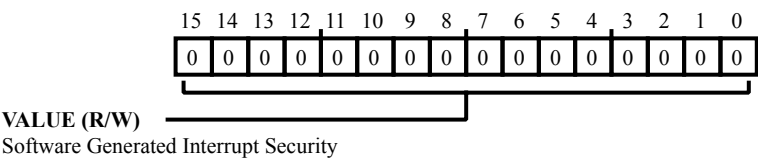


Figure 5-38: GICDST_SGI_SECURITY Register Diagram

Table 5-39: GICDST_SGI_SECURITY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Software Generated Interrupt Security. Each bit in the <code>GICDST_SGI_SECURITY.VALUE</code> bit field controls whether the corresponding interrupt is in Group 0 or Group 1.

Shared Peripheral Interrupt Register

The `GICDST_SPI[n]` register contains bits that provide the status of the SPI[987:0] inputs.

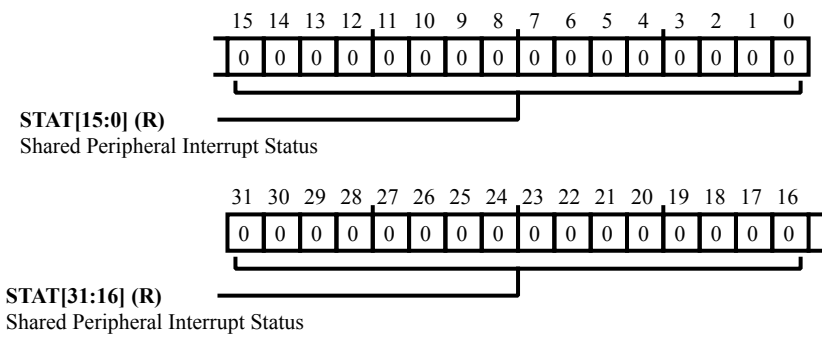


Figure 5-39: GICDST_SPI[n] Register Diagram

Table 5-40: GICDST_SPI[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	STAT	Shared Peripheral Interrupt Status. The <code>GICDST_SPI[n].STAT</code> bit field returns the status of the SPI[987:0] inputs on the distributor where bit [x] = 0 SPI[x] is low and bit [x] = 1 SPI[x] is high.

Shared Peripheral Interrupt Active Register

The `GICDST_SPI_ACTIVE[n]` register provides an active bit for each interrupt supported by the GIC.

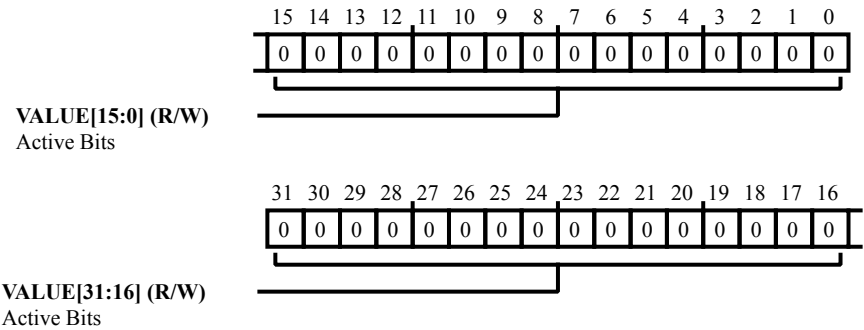


Figure 5-40: `GICDST_SPI_ACTIVE[n]` Register Diagram

Table 5-41: `GICDST_SPI_ACTIVE[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Active Bits. The <code>GICDST_SPI_ACTIVE[n].VALUE</code> bit field contains an Active bit for each interrupt supported by the GIC. Reading an active bit identifies whether the corresponding interrupt is active (=1) or not active (=0).

Shared Peripheral Interrupt Configuration Register

The `GICDST_SPI_CFG[n]` register provides a 2-bit `Int_config` field for each interrupt supported by the GIC.

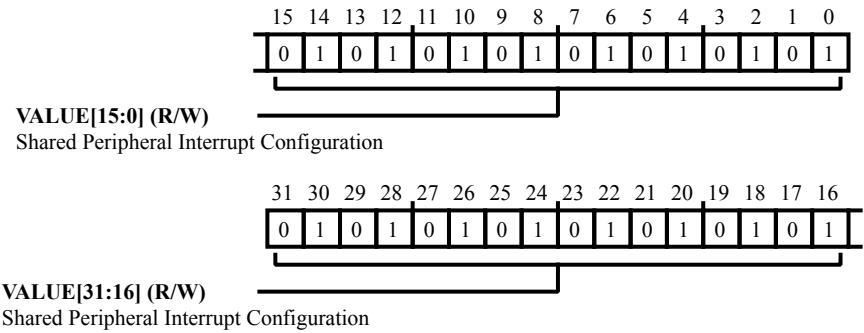


Figure 5-41: GICDST_SPI_CFG[n] Register Diagram

Table 5-42: GICDST_SPI_CFG[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Shared Peripheral Interrupt Configuration. The <code>GICDST_SPI_CFG[n].VALUE</code> bit field identifies whether the corresponding interrupt is: edge-triggered or level-sensitive handled using the 1-N model or using the N-N model

Shared Peripheral Interrupt Enable Clear Register

The `GICDST_SPI_EN_CLR[n]` register provides a clear-enable bit for each interrupt supported by the GIC.

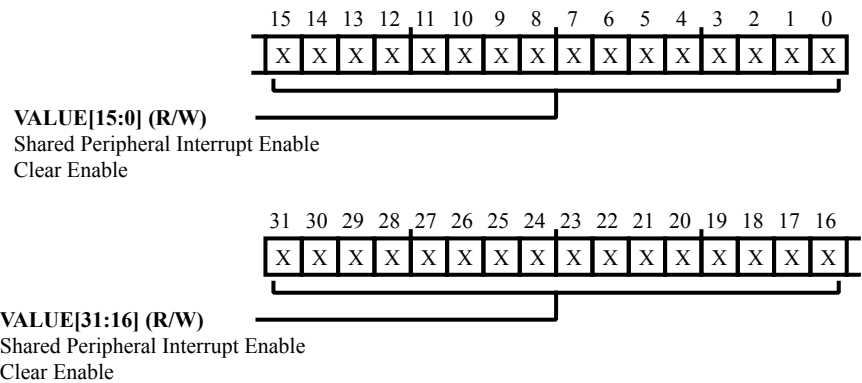


Figure 5-42: `GICDST_SPI_EN_CLR[n]` Register Diagram

Table 5-43: `GICDST_SPI_EN_CLR[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Shared Peripheral Interrupt Enable Clear Enable. Writing 1 to a <code>GICDST_SPI_EN_CLR[n].VALUE</code> bit disables forwarding of the corresponding interrupt to the CPU interfaces. Reading a bit identifies whether the interrupt is enabled.

Shared Peripheral Interrupt Enable Set Register

The `GICDST_SPI_EN_SET[n]` register provides a set-enable bit for each interrupt supported by the GIC.

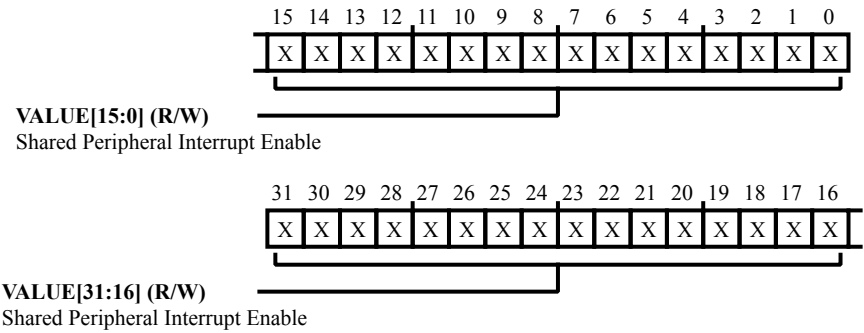


Figure 5-43: `GICDST_SPI_EN_SET[n]` Register Diagram

Table 5-44: `GICDST_SPI_EN_SET[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Shared Peripheral Interrupt Enable. Writing 1 to a <code>GICDST_SPI_EN_SET[n].VALUE</code> bit enables forwarding of the corresponding interrupt to the CPU interfaces. Reading a bit identifies whether the interrupt is enabled.

Shared Peripheral Interrupt Pending Clear Register

The `GICDST_SPI_PND_CLR[n]` register provides a clear-pending bit for each interrupt supported by the GIC.

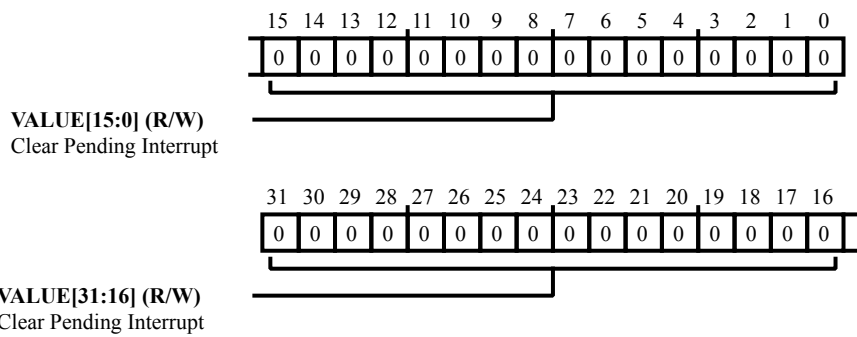


Figure 5-44: GICDST_SPI_PND_CLR[n] Register Diagram

Table 5-45: GICDST_SPI_PND_CLR[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Clear Pending Interrupt. Writing 1 to a <code>GICDST_SPI_PND_CLR[n].VALUE</code> bit clears the pending status of the corresponding peripheral interrupt. Reading a bit identifies whether the interrupt is pending.

Shared Peripheral Interrupt Pending Set Register

The `GICDST_SPI_PND_SET[n]` register provides a set-pending bit for each interrupt supported by the GIC.

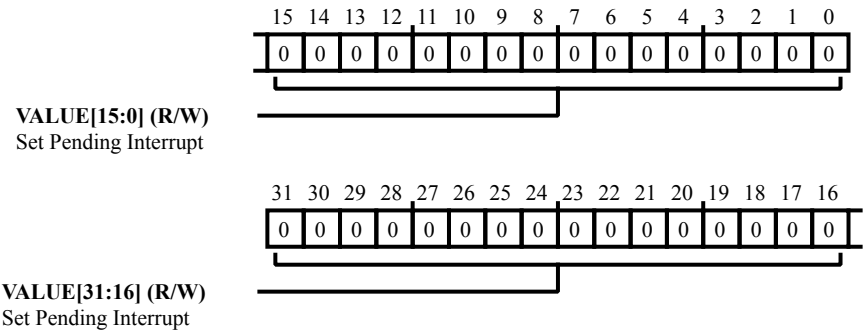


Figure 5-45: `GICDST_SPI_PND_SET[n]` Register Diagram

Table 5-46: `GICDST_SPI_PND_SET[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Set Pending Interrupt. Writing 1 to a <code>GICDST_SPI_PND_SET[n].VALUE</code> bit sets the status of the corresponding peripheral interrupt to pending. Reading a bit identifies whether the interrupt is pending.

Shared Peripheral Interrupt Security Register

The `GICDST_SPI_SECURITY[n]` register provides a security status bit for each interrupt supported by the GIC.

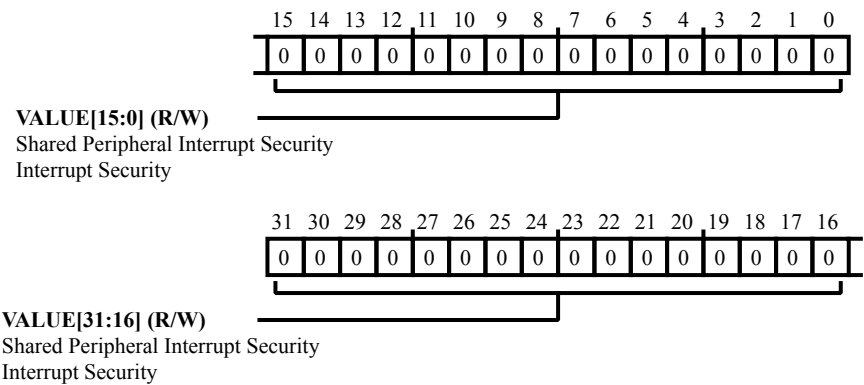


Figure 5-46: GICDST_SPI_SECURITY[n] Register Diagram

Table 5-47: GICDST_SPI_SECURITY[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Shared Peripheral Interrupt Security Interrupt Security. The <code>GICDST_SPI_SECURITY[n].VALUE</code> bits control the security status of the corresponding interrupt.

Shared Peripheral Interrupt Processor Targets Register

The `GICDST_SPI_TRGT[n]` register provides an 8-bit CPU targets field for each interrupt supported by the GIC.

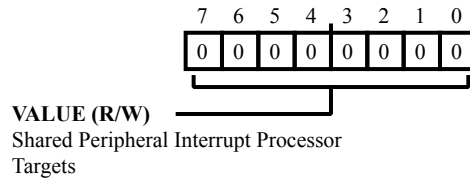


Figure 5-47: GICDST_SPI_TRGT[n] Register Diagram

Table 5-48: GICDST_SPI_TRGT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	Shared Peripheral Interrupt Processor Targets. The <code>GICDST_SPI_TRGT[n].VALUE</code> bit field stores the list of processors that the interrupt is sent to if it is asserted.

ADSP-2159x GICCPU Register Descriptions

GIC CPU Port (GICCPU) contains the following registers.

Table 5-49: ADSP-2159x GICCPU Register List

Name	Description
<code>GICCPU_BIN_PT_ALIAS</code>	Aliased Binary Point Register (ICCBPR)
<code>GICCPU_BIN_PT</code>	Binary Point Register (ICCBPR)
<code>GICCPU_CTL</code>	CPU Interface Control Register (ICCICR)
<code>GICCPU_EOI</code>	End of Interrupt Register (ICCEOIR)
<code>GICCPU_PND_HI</code>	Highest Pending Interrupt Register (ICCHPIR)
<code>GICCPU_INT_ACK</code>	Interrupt Acknowledge Register (ICCIAR)
<code>GICCPU_PRIO_MSK</code>	Priority Mask Register (ICCIPMR)
<code>GICCPU_RUN_PRIO</code>	Running Priority Register (ICCRPR)

Aliased Binary Point Register (ICCABPR)

The `GICCPU_BIN_PT_ALIAS` register Provides an alias for the non secure `GICCPU_BIN_PT` register.

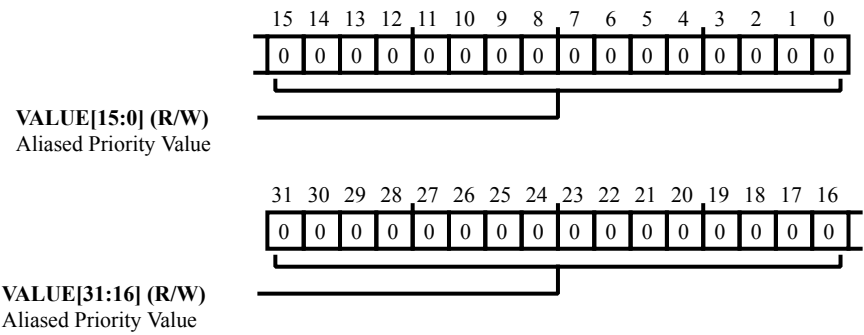


Figure 5-48: GICCPU_BIN_PT_ALIAS Register Diagram

Table 5-50: GICCPU_BIN_PT_ALIAS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Aliased Priority Value.

Binary Point Register (ICCBPR)

The `GICCPU_BIN_PT` register defines the point at which the priority value fields split into two parts, the group priority field and the sub-priority field. The group priority field is used to determine interrupt preemption.

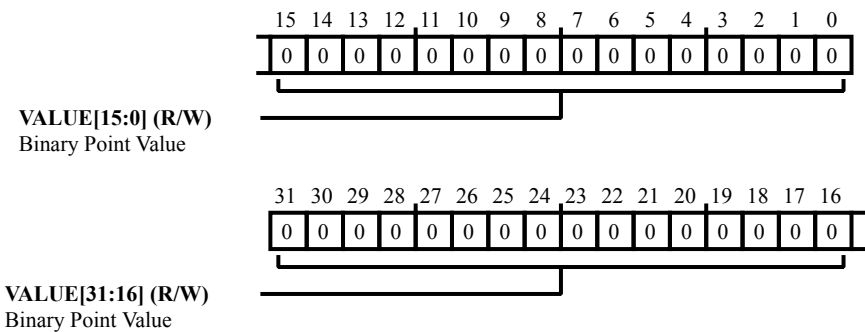


Figure 5-49: GICCPU_BIN_PT Register Diagram

Table 5-51: GICCPU_BIN_PT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Binary Point Value. The <code>GICCPU_BIN_PT.VALUE</code> bit field defines the point at which the priority value fields split into two parts.

CPU Interface Control Register (ICCICR)

The `GICCPU_CTL` register enables the signaling of interrupts to the target processors. In a GIC that implements the Security Extensions, provides additional global controls for handling Secure interrupts.

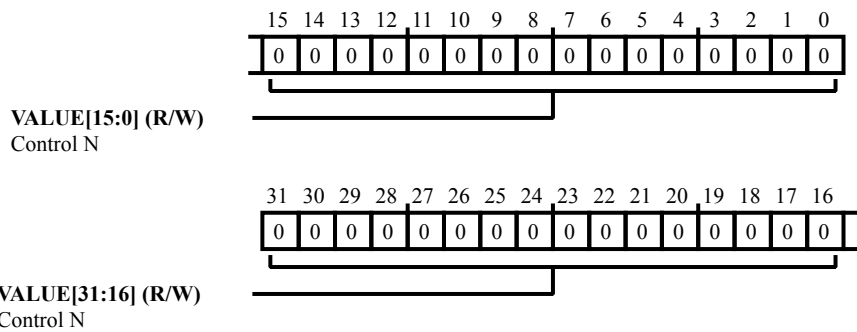


Figure 5-50: GICCPU_CTL Register Diagram

Table 5-52: GICCPU_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Control N.

End of Interrupt Register (ICCEOIR)

A processor writes to the `GICCPU_EOI` register to inform the CPU interface that it has completed its interrupt service routine for the specified interrupt.

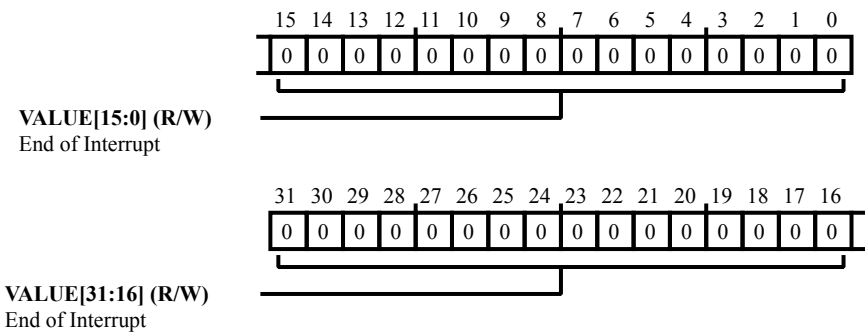


Figure 5-51: GICCPU_EOI Register Diagram

Table 5-53: GICCPU_EOI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	End of Interrupt. The <code>GICCPU_EOI.VALUE</code> bit field indicates to the CPU interface that it has completed its interrupt service routine for the specified interrupt.

Highest Pending Interrupt Register (ICCHPIR)

The `GICCPU_PND_HI` register indicates the Interrupt ID, and processor ID if appropriate, of the pending interrupt with the highest priority on the CPU interface.

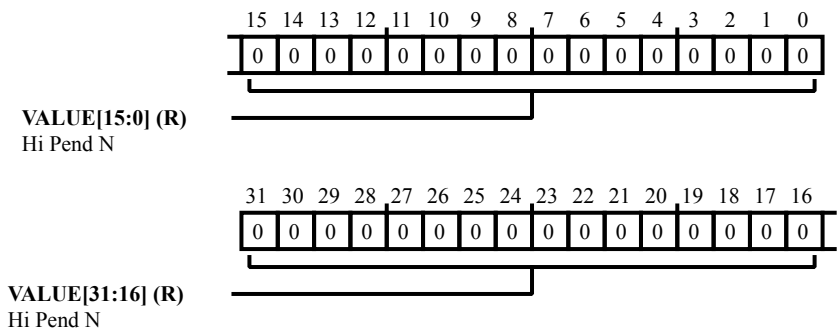


Figure 5-52: GICCPU_PND_HI Register Diagram

Table 5-54: GICCPU_PND_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Hi Pend N. Highest Pending Interrupt Register (ICCHPIR)

Interrupt Acknowledge Register (ICCIAR)

The processor reads the `GICCPU_INT_ACK` register to obtain the interrupt ID of the signaled interrupt. This read acts as an acknowledge for the interrupt.

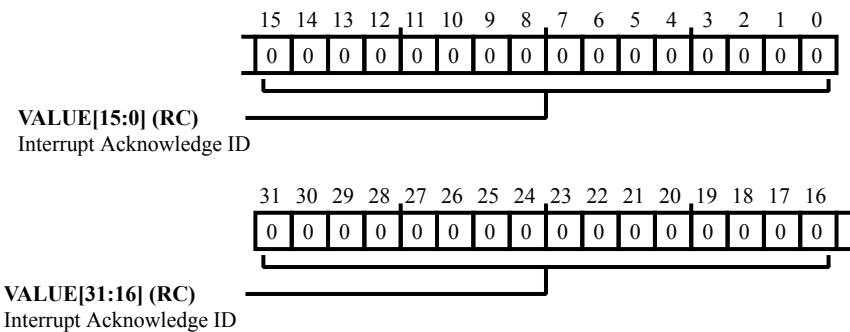


Figure 5-53: GICCPU_INT_ACK Register Diagram

Table 5-55: GICCPU_INT_ACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RC/NW)	VALUE	Interrupt Acknowledge ID. The <code>GICCPU_INT_ACK.VALUE</code> bit field contains the interrupt ID of the signaled interrupt.

Priority Mask Register (ICCIPMR)

The `GICCPU_PRIO_MSK` register provides an interrupt priority filter. Only interrupts with higher priority than the value in this register can be signaled to the processor.

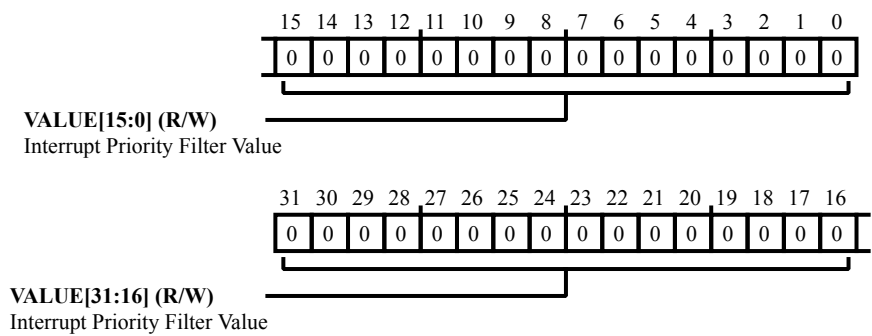


Figure 5-54: GICCPU_PRIO_MSK Register Diagram

Table 5-56: GICCPU_PRIO_MSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Interrupt Priority Filter Value. The <code>GICCPU_PRIO_MSK.VALUE</code> bit field contains the interrupt priority filter value.

Running Priority Register (ICCRPR)

The `GICCPU_RUN_PRIO` register indicates the priority of the highest priority interrupt that is active on the CPU interface.

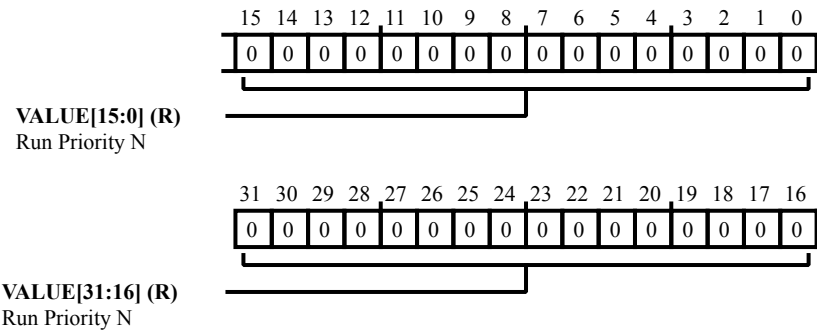


Figure 5-55: GICCPU_RUN_PRIO Register Diagram

Table 5-57: GICCPU_RUN_PRIO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Run Priority N. The <code>GICCPU_RUN_PRIO.VALUE</code> bit field contains the priority value of the highest priority interrupt that is active on the CPU interface. If there is no active interrupt on the CPU interface, and the GIC implements 8-bit priority fields, a read of this register returns the value 0xFF, corresponding to the lowest possible interrupt priority. If the GIC implements priority fields of less than 8 bits, the read might return the register reset value of 0xFF, or might return a value corresponding to the lowest possible interrupt priority. Software cannot determine the number of implemented priority bits from a read of this register.

6 Trigger Routing Unit (TRU)

The TRU provides system-level sequence control without core intervention. The TRU maps trigger masters (generators of triggers) to trigger slaves (receivers of triggers). Slave endpoints can be configured to respond to triggers in various ways. Multiple TRUs may be provided in a multiprocessor system to create a trigger network. Common applications enabled by the TRU include:

- Automatically triggering the start of a DMA sequence after a sequence from another DMA channel completes
- Software triggering
- Synchronization of concurrent activities

TRU Features

The TRU supports the following features:

- Automatically triggering the start of a DMA sequence after a sequence from another DMA channel completes. Once a DMA channel completes data transfer, it can act as a Trigger Master and signal an internal trigger pulse to the programmed Trigger Slave which can also be another DMA channel. The Slave Trigger connected to the DMA channel kicks off the DMA transfer automatically. None of this requires core intervention once the initialization is done.
- Software triggers. The best use of triggers is to minimize core intervention. It is also possible to initiate a trigger pulse to a Trigger Slave, in the software.
- Synchronization of concurrent activities. A single Trigger Master can initiate a trigger pulse to multiple Trigger Slaves so that several system level activities can be synchronized on an internally or externally generated event.
- Configuration protection through register-level lock bits and global lock indication

TRU Functional Description

The following sections provide a description of the TRU.

ADSP-2159x TRU Register List

The Trigger Routing Unit (TRU) provides simple sequence control of distributed modules without the penalties associated with core intervention (for example, interrupt overhead). The TRU receives trigger inputs from all master trigger inputs (MTI) and the TRU master trigger register ([TRU_MTR](#)). Based on these inputs, the TRU logic generates trigger outputs that initiate slave operations in the processor core and peripherals. A set of registers governs TRU operations. For more information on TRU functionality, see the TRU register descriptions.

Table 6-1: ADSP-2159x TRU Register List

Name	Description
TRU_ERRADDR	Error Address Register
TRU_GCTL	Global Control Register
TRU_MTR	Master Trigger Register
TRU_SSR[n]	Slave Select Register
TRU_STAT	Status Information Register

ADSP-2159x TRU Interrupt List

Table 6-2: ADSP-2159x TRU Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
162	TRU0_SLV0	TRU0 Interrupt 0 - Core 0	Edge	
163	TRU0_SLV1	TRU0 Interrupt 1 - Core 0	Edge	
164	TRU0_SLV2	TRU0 Interrupt 2 - Core 0	Edge	
165	TRU0_SLV3	TRU0 Interrupt 3 - Core 0	Edge	
242	TRU0_SLV4	TRU0 Interrupt 4 - Core 1	Edge	
243	TRU0_SLV5	TRU0 Interrupt 5 - Core 1	Edge	
244	TRU0_SLV6	TRU0 Interrupt 6 - Core 1	Edge	
245	TRU0_SLV7	TRU0 Interrupt 7 - Core 1	Edge	
246	TRU0_SLV8	TRU0 Interrupt 8 - Core 2	Edge	
247	TRU0_SLV9	TRU0 Interrupt 9 - Core 2	Edge	
248	TRU0_SLV10	TRU0 Interrupt 10 - Core 2	Edge	
249	TRU0_SLV11	TRU0 Interrupt 11 - Core 2	Edge	

ADSP-2159x TRU Trigger List

Table 6-3: ADSP-2159x TRU Trigger List Masters

Trigger ID	Name	Description	Sensitivity
None			

Table 6-4: ADSP-2159x TRU Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
157	TRU0_SLV0	TRU0 Interrupt Request 0	Pulse
158	TRU0_SLV1	TRU0 Interrupt Request 1	Pulse
159	TRU0_SLV2	TRU0 Interrupt Request 2	Pulse
160	TRU0_SLV3	TRU0 Interrupt Request 3	Pulse
161	TRU0_SLV4	TRU0 Interrupt Request 4	Pulse
162	TRU0_SLV5	TRU0 Interrupt Request 5	Pulse
163	TRU0_SLV6	TRU0 Interrupt Request 6	Pulse
164	TRU0_SLV7	TRU0 Interrupt Request 7	Pulse
165	TRU0_SLV8	TRU0 Interrupt Request 8	Pulse
166	TRU0_SLV9	TRU0 Interrupt Request 9	Pulse
167	TRU0_SLV10	TRU0 Interrupt Request 10	Pulse
168	TRU0_SLV11	TRU0 Interrupt Request 11	Pulse

TRU Definitions

The following definitions are helpful when using the TRU module.

Trigger Master

A trigger master is any system module that provides trigger event indication to the TRU. Trigger master modules define trigger events and conditions for assertion.

Trigger Master ID

Trigger masters are assigned a unique numeric ID according to their physical connection to the TRU. Trigger master ID 0 is reserved and defined as null.

Trigger Slave

A trigger slave is any system module that receives a trigger event indication from the TRU. Trigger slave modules define a trigger event response.

NOTE: For peripherals configured as a slave receiver: if the corresponding peripheral DMA is kept waiting for trigger from a master, it can receive stale or garbage data. The peripheral receiver begins shifting in data as soon as it is enabled and it does not wait for the trigger master. Upon receiving the trigger, this data is moved into the DMA FIFO. Therefore, this sequence can result in the slave receiver getting unusable data.

TRU Block Diagram

Trigger masters and the Master Trigger Register (MTR) generate trigger assertions. Each trigger slave has a dedicated Slave Select Register (SSR) that specifies the unique trigger master from which it receives the trigger indication.

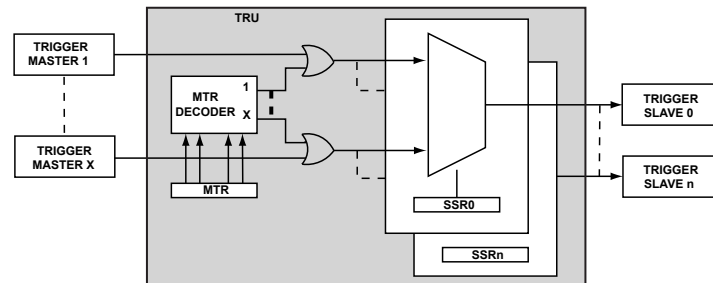


Figure 6-1: TRU Block Diagram

TRU Architectural Concepts

The TRU supports a simple trigger-in/trigger-out model for modules that comply with the triggering functional model. The TRU is the controller of the trigger system. Trigger outputs from trigger masters are mapped to trigger inputs of trigger slaves through a set of programmable registers ([TRU_SSR\[n\]](#)).

System modules are trigger master only, trigger slave only, or trigger master and trigger slave.

All of the trigger input and output signals are connected to a trigger routing unit (TRU) which manages the connections of triggers between modules.

In multi-processor systems, multiple TRU units are provided. These TRUs are networked together. Generic Trigger Ports (GTPs) are provided to forward trigger events from one TRU unit to another, forming a pathway from trigger masters to trigger slaves wherever they might lie in the system.

TRU Programming Model

Implementing sequence control using the TRU requires, at a minimum, proper configuration of a trigger slave, a trigger master, and the TRU module itself. The only requirement for the configuration procedure is that the trigger master is configured and enabled as the last step.

Complete the following other steps:

- Configure the trigger slave for response to triggers.
- Configure the TRU to map the trigger master to the trigger slave through the [TRU_SSR\[n\]](#) registers.
- Configure the trigger master to generate trigger assertions.

- Alternatively, use software triggering for trigger assertion. Writing the trigger master ID to the MTR register generates software triggers.

Programming Concepts

The following concepts aid in programming the TRU.

- *Trigger Sequence Configuration.* A simple sequence consists of one trigger master and one trigger slave. More complex trigger sequences consist of several trigger slaves functioning as trigger slave and trigger master. Additionally, trigger sequences can loopback to the original master forming a perpetual sequence.
- *Software Triggering.* Writing a trigger master ID to the MTR generates a trigger within the TRU from the trigger master ID specified.
- *Synchronization.* The TRU can be used to coarsely synchronize events by mapping multiple trigger slaves to the same trigger master or by generating multiple trigger master assertions simultaneously through the MTR.
- *Configuration Protection.* The `TRU_SSR[n].LOCK` bit and the `TRU_GCTL.LOCK` bit enable register level write-protection when the global lock is asserted in the SPU.

Programming Examples

The following examples shows the steps to create a single trigger.

Configuring a Simple Trigger Sequence

The following example shows the steps to create a simple trigger.

1. Write to the `TRU_GCTL` register to enable the TRU.
2. Write to the `TRU_SSR[n]` register of a specific trigger slave to assign it to a specific trigger master.
3. Enable the trigger slave to wait for and accept a trigger.
4. Enable the trigger master to generate a trigger.

TRU Event Control

The TRU is a major part of event control solutions. It is the center of the trigger functional model and can extend to support the interrupt and fault management models as well.

TRU Status and Error Signals

The TRU does not have dedicated status and error output signals other than the MMR interface. Slave errors are reported to the master over the standard peripheral bus protocol.

ADSP-2159x TRU Register Descriptions

Trigger Routing Unit (TRU) contains the following registers.

Table 6-5: ADSP-2159x TRU Register List

Name	Description
TRU_ERRADDR	Error Address Register
TRU_GCTL	Global Control Register
TRU_MTR	Master Trigger Register
TRU_SSR[n]	Slave Select Register
TRU_STAT	Status Information Register

Error Address Register

The TRU error address register ([TRU_ERRADDR](#)) holds the address from the memory-mapped register access generating an access error of TRU registers.

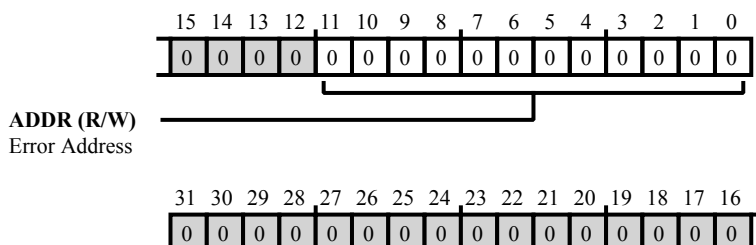


Figure 6-2: TRU_ERRADDR Register Diagram

Table 6-6: TRU_ERRADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11:0 (R/W)	ADDR	Error Address. The <code>TRU_ERRADDR.ADDR</code> holds the address from the memory-mapped register access generating an access error of TRU registers. These errors occur on access to the TRU_SSR[n] or TRU_MTR registers when these registers are locked or on access to an invalid address. See the TRU_SSR[n] and TRU_MTR register descriptions for more information about locking. The TRU_ERRADDR register holds the address of the first error to occur. In the event of multiple errors occurring, the TRU_ERRADDR register contains the address of the first error. To re-enable the TRU_ERRADDR register for update, both status bits (<code>TRU_STAT.LWERR</code> and <code>TRU_STAT.ADDRERR</code>) in the TRU_STAT register must be cleared.

Global Control Register

The TRU global control register ([TRU_GCTL](#)) provides register locking, TRU reset, and TRU enable.

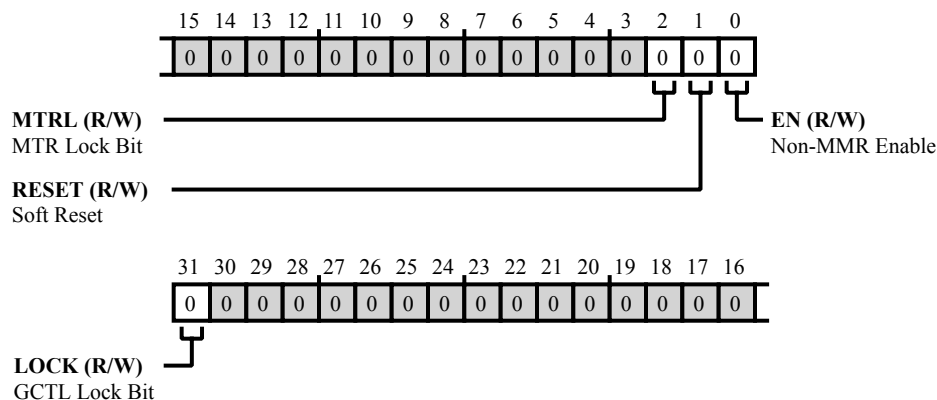


Figure 6-3: TRU_GCTL Register Diagram

Table 6-7: TRU_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	GCTL Lock Bit. If the global lock is enabled (SPU_CTL.GLCK bit =1) and the TRU_GCTL.LOCK bit is enabled, the TRU_GCTL register is read only.
		0 Read write
		1 Read only
2 (R/W)	MTRL	MTR Lock Bit. If the global lock is enabled (SPU_CTL.GLCK bit =1) and the TRU_GCTL.MTRL bit is enabled, the TRU_MTR register is read only.
		0 Read write
		1 Read only
1 (R/W)	RESET	Soft Reset. The TRU_GCTL.RESET bit is write-1-action and triggers a soft reset to all TRU registers.
		0 No action
		1 Soft reset

Table 6-7: TRU_GCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Non-MMR Enable. The TRU_GCTL.EN bit is read/write and must be set for the TRU to propagate trigger events. All TRU register read/write operations continue to operate independent of the TRU_GCTL.EN bit.
		0 No trigger events
		1 Propagate trigger events

Master Trigger Register

The TRU master trigger register (**TRU_MTR**) permits trigger generation through software by writing a trigger master ID value to one of the four fields in the **TRU_MTR** register. If the global lock is enabled (**SPU_CTL.GLCK** bit =1) and the **TRU_GCTL.LOCK** bit is set, the **TRU_MTR** register is read only. Note this register is primarily used for debug to trigger a TRU output

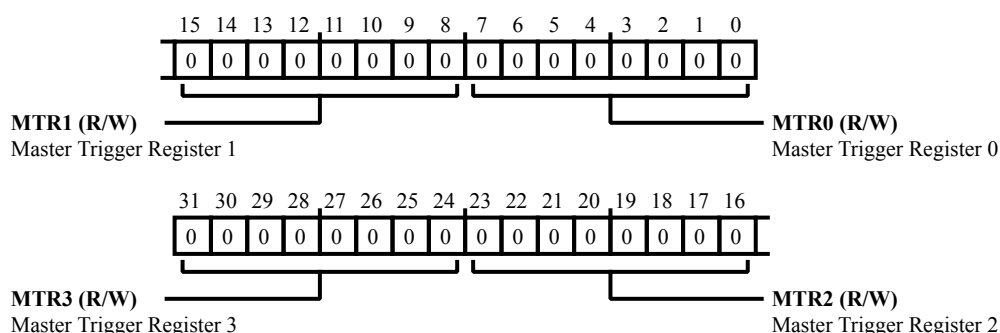


Figure 6-4: TRU_MTR Register Diagram

Table 6-8: TRU_MTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	MTR3	Master Trigger Register 3. The TRU_MTR.MTR3 bit field is the trigger master ID value for master 3.
		0 No master specified
		1-182 Range of valid masters
23:16 (R/W)	MTR2	Master Trigger Register 2. The TRU_MTR.MTR2 bit field is the trigger master ID value for master 2.
		0 No master specified
		1-182 Range of valid masters
15:8 (R/W)	MTR1	Master Trigger Register 1. The TRU_MTR.MTR1 bit field is the trigger master ID value for master 1.
		0 No master specified
		1-182 Range of valid masters
7:0 (R/W)	MTR0	Master Trigger Register 0. The TRU_MTR.MTR0 bit field is the trigger master ID value for master 0.
		0 No master specified
		1-182 Range of valid masters

Slave Select Register

The TRU slave select registers ([TRU_SSR\[n\]](#)) each provide slave selection and register locking.

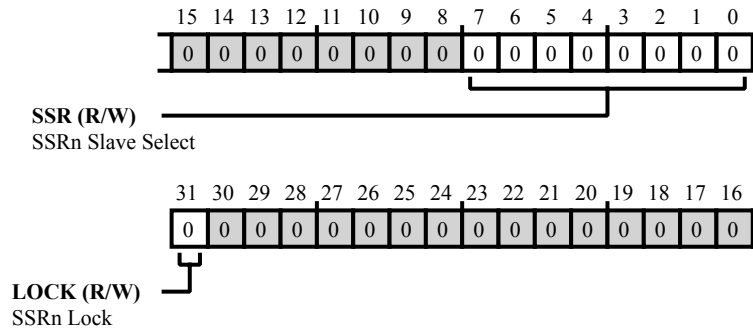


Figure 6-5: TRU_SSR[n] Register Diagram

Table 6-9: TRU_SSR[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	SSRn Lock. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>TRU_SSR[n].LOCK</code> bit is enabled, the <code>TRU_SSR[n]</code> register is read only.
		0 Unlock register
		1 Lock register
7:0 (R/W)	SSR	SSRn Slave Select. The <code>TRU_SSR[n]</code> register selects the trigger master ID to which the trigger slave responds. For example, when a <code>TRU_SSR[n]</code> register is set to respond to trigger master ID n, a trigger that is generated by trigger master ID n results in a trigger out to the slave.
		0 No master specified
		1-182 Range of valid masters

Status Information Register

The TRU status register (`TRU_STAT`) contains the status of `TRU_MTR` and `TRU_SSR[n]` register writes and status of bus read/write errors.

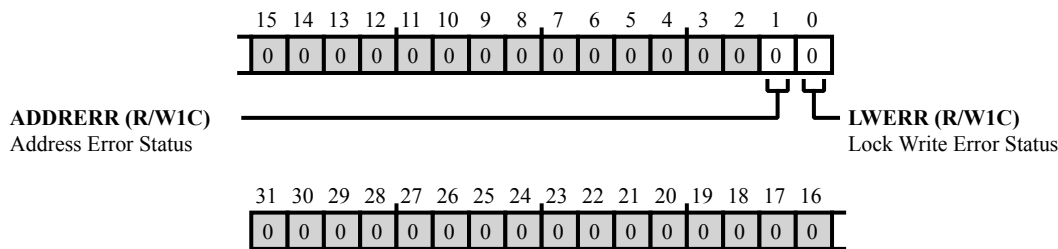


Figure 6-6: TRU_STAT Register Diagram

Table 6-10: TRU_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W1C)	ADDRERR	Address Error Status. The <code>TRU_STAT.ADDRERR</code> bit is set when an invalid address is provided for an MMR access while the TRU is selected. Writing a one to this bit clears the error indication. The <code>TRU_ERRADDR</code> register also is updated when an address error occurs during an MMR access while the TRU is selected.
		0 No error
		1 Error occurred
0 (R/W1C)	LWERR	Lock Write Error Status. If <code>TRU_STAT.LWERR</code> is set, a lock write error has occurred. Writing a one to this bit clears the error indication.
		0 No error
		1 Error occurred

7 L2 System Memory

L2 system memories have significant bandwidth for core accesses, but it is important to note that L2 responds slower to core accesses than L1 memories. L2 SRAM is the ideal storage for multiple processor cores to share data and instruction resources, such as semaphores, shared buffers, and code libraries. Due to sophisticated data integrity protection and write protection, L2 SRAM is also ideal for data and instructions critical for safe operation of the application.

L2 System Memory Features

The L2 system memory features include:

- Operation at SYSCLK frequency
- ECC protection of SRAM area
- ECC memory refresh

There is one instance of L2 system memory: L2CTL0.

- L2CTL0 contains 2M byte of RAM grouped into eight banks, 256K bytes each and 192K bytes of boot ROM.
- SHARC core data ports can access L2 memory through direct path as well as through Arm L2 cache.

L2 System Memory Functional Description

The L2 system memory manages all of the L2 SRAM and ROM memory banks. The system memory interface arbitrates competing accesses, write protection, and ensures SRAM data integrity. The L2 system memory domain is a unified instruction and data memory. It can hold any mixture of code and data required by the system design.

The following sections provide a functional description of the L2 system memory.

ADSP-2159x L2CTL Register List

Table 7-1: ADSP-2159x L2CTL Register List

Name	Description
L2CTL_CTL	Control Register
L2CTL_EADDR0	Error Type 0 Address Register
L2CTL_EADDR1	Error Type 1 Address Register
L2CTL_EADDR2	Error Type 2 Address Register
L2CTL_EADDR3	Error Type 3 Address Register
L2CTL_EADDR4	Error Type 4 Address Register
L2CTL_ERRADDR0	ECC Error Address 0 Register
L2CTL_ERRADDR1	ECC Error Address 1 Register
L2CTL_ERRADDR2	ECC Error Address 2 Register
L2CTL_ERRADDR3	ECC Error Address 3 Register
L2CTL_ERRADDR4	ECC Error Address 4 Register
L2CTL_ERRADDR5	ECC Error Address 5 Register
L2CTL_ERRADDR6	ECC Error Address 6 Register
L2CTL_ERRADDR7	ECC Error Address 7 Register
L2CTL_ERRADDR8	ECC Error Address 8 Register
L2CTL_ET0	Error Type 0 Register
L2CTL_ET1	Error Type 1 Register
L2CTL_ET2	Error Type 2 Register
L2CTL_ET3	Error Type 3 Register
L2CTL_ET4	Error Type 4 Register
L2CTL_INIT	Initialization Register
L2CTL_ISTAT	Initialization Status Register
L2CTL_PCTL	Power Control Register
L2CTL_REVID	Revision ID Register
L2CTL_RPCR0	Read Priority Count Register
L2CTL_RPCR1	Read Priority Count Register
L2CTL_SADR	Scrub Start Address Register
L2CTL_SCNT	Scrub Count Register
L2CTL_SCTL	Scrub Control Register
L2CTL_STAT	Status Register
L2CTL_STAT_1	L2 Port Error Status Register

Table 7-1: ADSP-2159x L2CTL Register List (Continued)

Name	Description
L2CTL_WPCR0	Write Priority Count Register
L2CTL_WPCR1	Write Priority Count Register

ADSP-2159x L2CTL Interrupt List

Table 7-2: ADSP-2159x L2CTL Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
10	L2CTL0_ECC_ERR	L2CTL0 ECC Error	Level	
11	Reserved	Reserved	Reserved	Reserved
12	L2CTL0_EVT	L2CTL0 Scrub/Initialization Done	Level	

ADSP-2159x L2CTL Trigger List

Table 7-3: ADSP-2159x L2CTL Trigger List Masters

Trigger ID	Name	Description	Sensitivity
47	L2CTL0_EVT	L2CTL0 L2 Memory Event	Level

Table 7-4: ADSP-2159x L2CTL Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

L2 System Memory Block Diagram

The *ADSP-SC59x Complete L2 System Memory Block Diagram* shows the complete L2 system memory, including the memory block instance L2CTL0. The L2CTL0 block contains boot ROM code for Arm and eight banks of L2 RAM containing 256 Kbytes each. The Arm reset ISR (location 0x00000000) is mapped to this block.

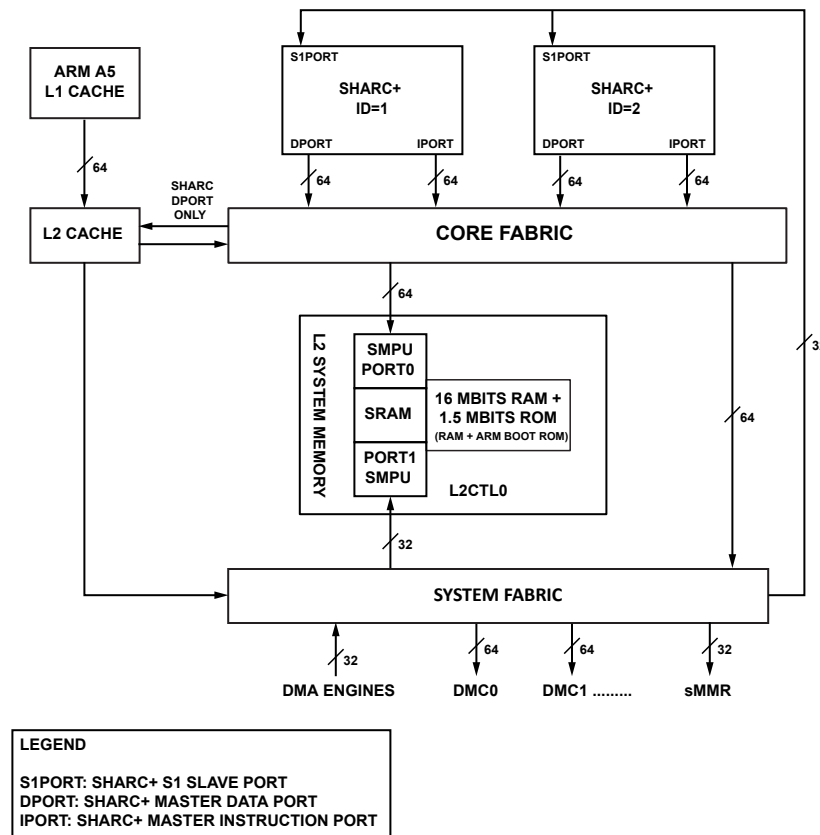


Figure 7-1: ADSP-SC59x Complete L2 System Block Diagram

L2 System Memory Architectural Concepts

The following sections describe architecture features of the L2 system memory.

- [Read/Write Latency and Throughput](#)
- [Arbitration and Priority](#)

Access Characteristics

The L2 system memory interface converts all 8-bit, 16-bit, 32-bit, and 64-bit accesses to 128-bit accesses. Additionally, it converts 8-bit, 16-bit, 32-bit, and 64-bit bursts to an equivalent internal 128-bit access. For example, the L2 system memory interface converts a 128-bit address-aligned burst of 8-bit accesses of burst length 16 to a single 128-bit access.

Read/Write Latency and Throughput

The L2 memory design is optimized for burst accesses at the crossbar interface. The L2 system memory buffers and converts write data of 8/16/32/64-bit to an equivalent 128-bit access. This conversion creates modulo-32-bit writes if the starting addresses are 32-bit aligned. A single 8-bit or 16-bit access, or a non-32-bit address-aligned 8-bit or 16-bit burst access to an ECC-enabled bank creates an extra latency of two SYSCLK cycles. No extra latency is seen if the ECC is disabled.

NOTE: Continuous 8/16-bit core access to an ECC-enabled L2 bank is not recommended from a throughput perspective.

L2 Memory Controller Block Diagram (Instance)

As shown in the *L2 System Memory Block Diagram*, the L2 controller has five ports that interface to system cross-bars. Port 0, port 1, and port 2 are 128-bit interfaces dedicated to core and MDMA3 traffic. Port 3 and port 4 are 128-bit interfaces that connect through DMA. For L2 SRAM, all ports (0/1/2/3/4) have a read channel and a write channel; for L2 ROM, all ports (0/1/2/3/4) have read channels only. The SRAM is organized in multiple banks; each bank has 256K Bytes. The 192KB ROM is divided into three banks of 64KB.

Within each bank, data is organized into 16384 words, with each word comprising 128 bits of data and 28 bits of ECC checksum. ROM memory is not protected by the ECC scheme. When the L2 controller accesses RAM and ROM cells, it always reads and writes whole 128-bit words. Despite this, the L2 controller supports 8-, 16-, 32-, and 64-bit reads and writes from cores and system by applying respective data masks.

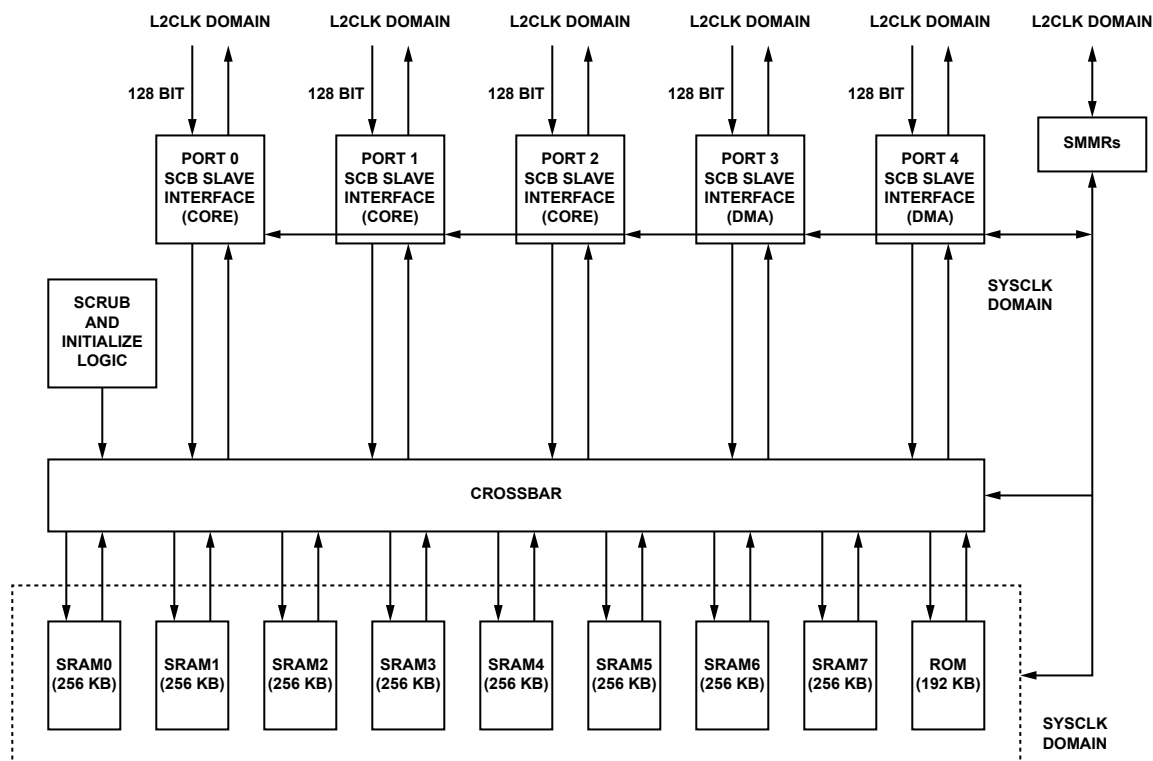


Figure 7-2: L2 System Memory Block Diagram

Arbitration and Priority

Each bank of L2 RAM or ROM has an arbiter which receives requests from the five crossbar ports.

Each arbiter follows a fixed priority scheme for giving grants when more than one channel requests the same bank. The arbiter also supports priority elevation through urgent priority requests.

NOTE: Attempting a write access to both L2 ROM spaces returns an error.

The *Fixed Priority* table shows the priority for fixed priority mode (with urgent priority disabled) for each SCB channel.

If two cores (or the 64-bit Max BW DMA) simultaneously try to access L2 for the same instance (both read or both write), even to different banks, software allows only one master access at a time. One access port can support one read and write at the same time. However, if one core issues a write and the other issues a read, then access can proceed simultaneously. There is no extra latency inside L2, as long as the accesses are to different banks (assuming pending DMA traffic is also to a non-conflicting bank).

When a core and DMA both access the same bank via the same port (both read or both writes), the best access rate that DMA can achieve is one 128-bit access in every three SYSCLK cycles during the conflict period. This access rate is achieved by programming the read priority count register (`L2CTL_RPCR0.RPC0`) bit and the write priority count register (`L2CTL_WPCR0.WPC0`) to 0, while programming the `L2CTL_RPCR0.RPC1` and the `L2CTL_WPCR0.WPC1` bits to 1.

The arbiters also support priority elevation for a particular channel that has been starved of grants for many SYSCLK cycles. If a channel does not get a grant for N cycles after its request, then that channel can elevate the priority of its request by issuing an urgent priority request. This request causes that particular channel to become the highest priority master for the next grant cycle (pipelined arbitration for urgent priority). The number of cycles N , after which the priority is elevated, can be programmed for each channel separately using the (`L2CTL_RPCR0`) and (`L2CTL_WPCR0`) registers.

Programming the bits in the (`L2CTL_RPCR0`) and (`L2CTL_WPCR0`) registers appropriately achieves the best grant rate for DMA. This grant rate of one in three SYSCLK cycles during the conflict period is achievable under the following conflict conditions:

- An access conflict between the core and DMA to the same memory bank in the fixed priority arbitration scheme with core activity always prioritized over DMA activity
- An access conflict within the pipelined implementation of urgent priority

To disable urgent priority requests, set the `L2CTL_CTL.DISURP` bit. This bit disables the urgent priority requests for all port channels. Each channel can also be prevented from raising the urgent priority request through the priority count register for the specific channel. However, there is no support for disabling urgent priority for a specific memory bank arbiter.

The *Fixed Priority With Priority Elevation* table provides the various priority levels for the L2 system memory.

Table 7-5: Fixed Priority With Priority Elevation

Channel	Priority Level
L2 Scrub/Initialization Request	21 (highest)
Port 0 Read Channel Urgent Request	20
Port 0 Write Channel Urgent Request	19
Port 1 Read Channel Urgent Request	18

Table 7-5: Fixed Priority With Priority Elevation (Continued)

Channel	Priority Level
Port 1 Write Channel Urgent Request	17
Port 2 Read Channel Urgent Request	16
Port 2 Write Channel Urgent Request	15
Port 3 Read Channel Urgent Request	14
Port 3 Write Channel Urgent Request	13
Port 4 Read Channel Urgent Request	12
Port 4 Write Channel Urgent Request	11
Port 0 Read Channel Normal Request	10
Port 0 Write Channel Normal Request	9
Port 1 Read Channel Normal Request	8
Port 1 Write Channel Normal Request	7
Port 2 Read Channel Normal Request	6
Port 2 Write Channel Normal Request	5
Port 3 Read Channel Normal Request	4
Port 3 Write Channel Normal Request	3
Port 4 Read Channel Normal Request	2
Port 4 Write Channel Normal Request	1 (lowest)

Data Integrity

The following sections provide information on how the L2 system memory ensures data integrity.

ECC Algorithm

Hsaio encoding calculates the ECC syndrome. A 7-bit syndrome is generated during write operation and stored as a 7-bit parity field along with the 32 data bits. Each data bit contributes to three parity bits according. Each parity bit represents the XOR value of 13 or 14 data bits according to the following mapping:

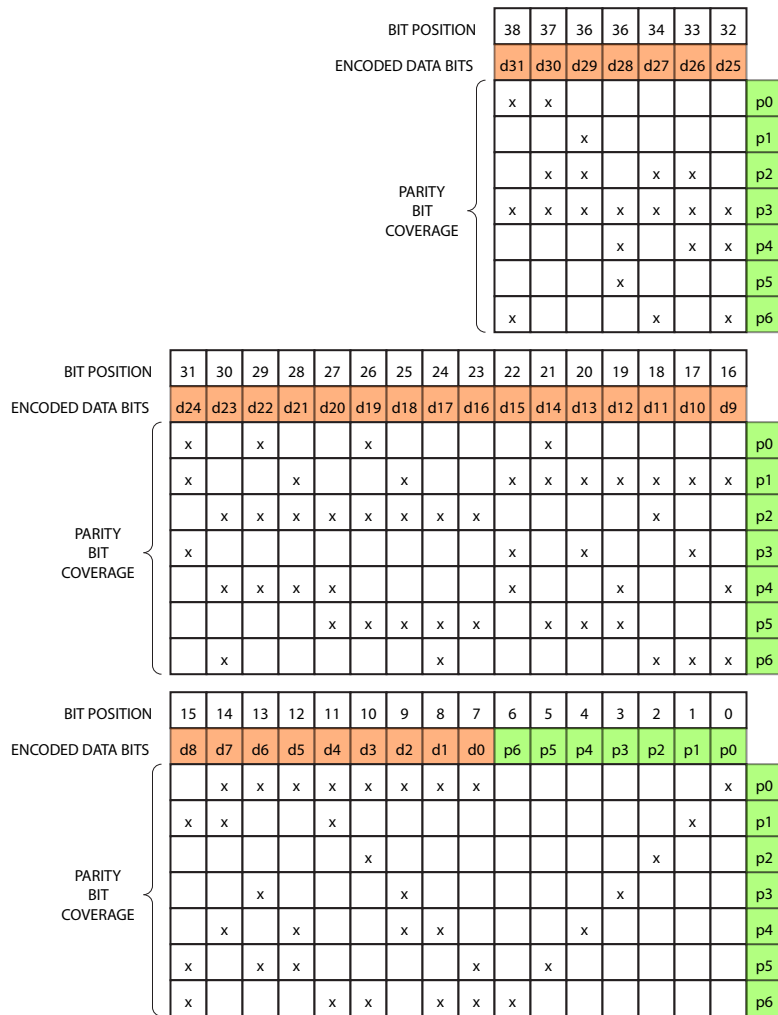


Figure 7-3: Hsaio Parity Bit Mapping

During read operation, the parity bits become part of the syndrome equation. The new syndrome bits are now the XOR values of the 13 or 14 data bits plus the respective stored parity bit. If any of the seven syndrome bits is set, an error situation is detected. An OR gate cross of the 7 bits reports the error, without specifying the type of the error.

If a single parity bit failed, the new 7-bit syndrome has 1 bit that is set. If a single data bit failed, the new syndrome has 3 bits that are set, because all three related parity bits fail. So, an XOR gate cross of all seven syndrome bits detects a single-bit error, indicating that an odd number of syndrome bits is set.

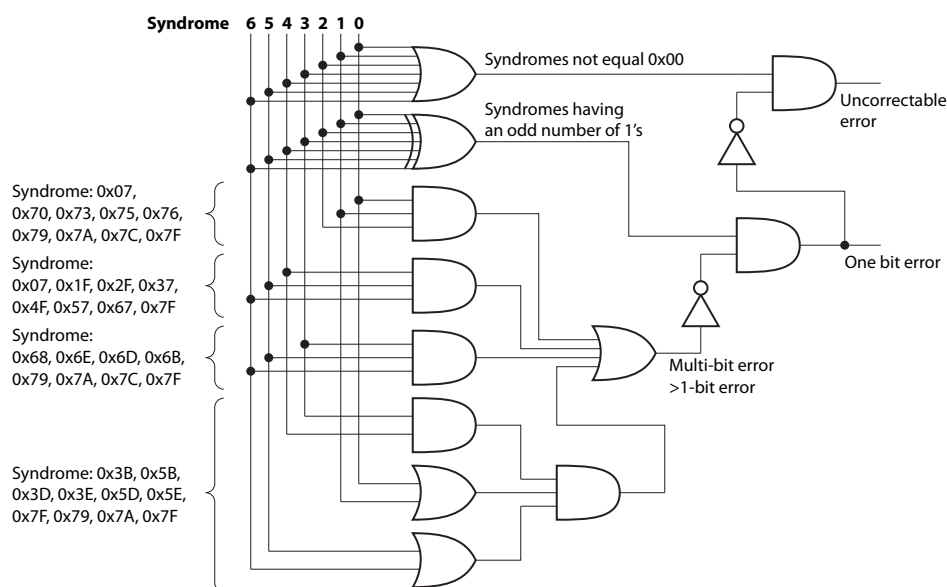


Figure 7-4: Hsiao Error Reports

The XOR gate detects single-bit errors and does not flag any dual-bit error. But, the gate does flag 50% of the other multi-bit errors undesirably. Extra logic is implemented to increase the detection rate of multi-bit errors to 68.7% as shown in the figure.

If a single-bit error is detected, the failing bit can be localized and corrected. If all three syndrome bits corresponding to a specific data bit are 1, a data error is assumed. The respective data bit is toggled on its way to the system bus.

ECC Hardware Control

After reset, ECC protection is enabled. The boot code initializes all L2 SRAM data and checksum cells. ECC protection adds some cycle penalty when 8-bit and 16-bit values write L2 memory. Disable ECC protection for individual SRAM banks by setting the `L2CTL_CTL.BK0EDIS` through `L2CTL_CTL.BK7EDIS` disable bits. Due to caching mechanisms of the processor cores and data bursting of the DMA channels, 8-bit and 16-bit write accesses are rather uncommon. Typically, only two-dimensional DMA operations or uncached 8-bit and 16-bit store instruction can trigger these writes.

ECC Error Management

The L2 system memory flags 2-bit and multi-bit errors to the system by:

- Raising the `ECC_ERR` interrupt
- Reporting a read error to the system bus
- Setting the sticky `L2CTL_STAT.ECCERR7-L2CTL_STAT.ECCERR0` status flag
- Latching the address of the failing operation into the respective `L2CTL_ERRADDR7-L2CTL_ERRADDR0` register.

There is one error status bit and one error address register per L2 SRAM bank.

Typically, ECC_ERR events are declared as system faults in the system event controller (SEC). Whether these faults are reported, the interrupt service routine can consult the `L2CTL_STAT` register and the `L2CTL_ERRADDR0` through `L2CTL_ERRADDR7` registers to determine whether:

- The data at the failing L2 address was critical enough to require an immediate reboot of the system
- The data at the failing L2 address was less critical or can be restored

The `L2CTL_STAT.ECCERR0` through `L2CTL_STAT.ECCERR7` flags are cleared with a W1C operation.

Memory Refresh

If data in L2 SRAM contains single-bit errors, the data is corrected on its way to the system buses. The corrected value is not written back to the SRAM location. To prevent any risk of accumulation of single-bit errors over time and to minimize likelihood of multi-bit errors, the L2 system memory provides a special memory refresh mechanism.

Memory refresh operation is meaningless when the `L2CTL_CTL.BK0EDIS` through `L2CTL_CTL.BK7EDIS` disable bits are set.

Power Modes

Each L2 memory bank supports low-power modes.

If the bank is not in use, put each L2 system memory bank into the following low-power modes to save power.

Deep Sleep Mode

Set the `L2CTL_PCTL.BK7DS` through `L2CTL_PCTL.BK0DS` control bits to enter this mode. This mode preserves the memory contents.

Shut Down Mode

Set the `L2CTL_PCTL.BK7SD` through `L2CTL_PCTL.BK0SD` control bits to enter this mode. The memory banks do not retain the data (existing data is lost).

NOTE: It may take up to 11 SYSCLK cycles to deactivate the power mode after writing to the `L2CTL_PCTL` register. Therefore, L2 access must be issued after this delay.

Access to a memory bank in shut down or deep sleep mode may result in an unpredictable behavior. Ensure that initialization/refresh and memory access to such banks are not issued.

There is no deep sleep or shut down mode for ROM memories.

L2 System Memory Event Control

The following sections describe event control features of the L2 system memory, such as error response.

ECC Error Interrupt

A bus error is signaled under any of the following conditions.

- A write access to ROM address space
- A read/write access to reserved address space
- An ECC multi-bit error in an ECC-enabled bank. A non-modulo, 32-bit write to an ECC-enabled bank can also potentially create a bus error response due to an ECC multi-bit error. This response is because the L2 system memory implements a 32-bit ECC, and therefore a non-modulo, 32-bit write results in a read. This read can create multi-bit errors even if the memory was initialized.

Bus error notifications are stored in the [L2CTL_STAT](#) register, and the addresses that generated the error on a given port are stored in the [L2CTL_EADDR0/L2CTL_EADDR1](#) register of that port. The details of the error are stored in the [L2CTL_ET0/L2CTL_ET1](#) register of the port.

ADSP-2159x L2CTL Register Descriptions

Some description. (L2CTL) contains the following registers.

Table 7-6: ADSP-2159x L2CTL Register List

Name	Description
L2CTL_CTL	Control Register
L2CTL_EADDR0	Error Type 0 Address Register
L2CTL_EADDR1	Error Type 1 Address Register
L2CTL_EADDR2	Error Type 2 Address Register
L2CTL_EADDR3	Error Type 3 Address Register
L2CTL_EADDR4	Error Type 4 Address Register
L2CTL_ERRADDR0	ECC Error Address 0 Register
L2CTL_ERRADDR1	ECC Error Address 1 Register
L2CTL_ERRADDR2	ECC Error Address 2 Register
L2CTL_ERRADDR3	ECC Error Address 3 Register
L2CTL_ERRADDR4	ECC Error Address 4 Register
L2CTL_ERRADDR5	ECC Error Address 5 Register
L2CTL_ERRADDR6	ECC Error Address 6 Register
L2CTL_ERRADDR7	ECC Error Address 7 Register
L2CTL_ERRADDR8	ECC Error Address 8 Register
L2CTL_ET0	Error Type 0 Register
L2CTL_ET1	Error Type 1 Register
L2CTL_ET2	Error Type 2 Register
L2CTL_ET3	Error Type 3 Register

Table 7-6: ADSP-2159x L2CTL Register List (Continued)

Name	Description
L2CTL_ET4	Error Type 4 Register
L2CTL_INIT	Initialization Register
L2CTL_ISTAT	Initialization Status Register
L2CTL_PCTL	Power Control Register
L2CTL_REVID	Revision ID Register
L2CTL_RPCR0	Read Priority Count Register
L2CTL_RPCR1	Read Priority Count Register
L2CTL_SADR	Scrub Start Address Register
L2CTL_SCNT	Scrub Count Register
L2CTL_SCTL	Scrub Control Register
L2CTL_STAT	Status Register
L2CTL_STAT_1	L2 Port Error Status Register
L2CTL_WPCR0	Write Priority Count Register
L2CTL_WPCR1	Write Priority Count Register

Control Register

The `L2CTL_CTL` register includes a write protection bit, enables L2 banks, and selects mapping of banks (as ECC RAM or data RAM).

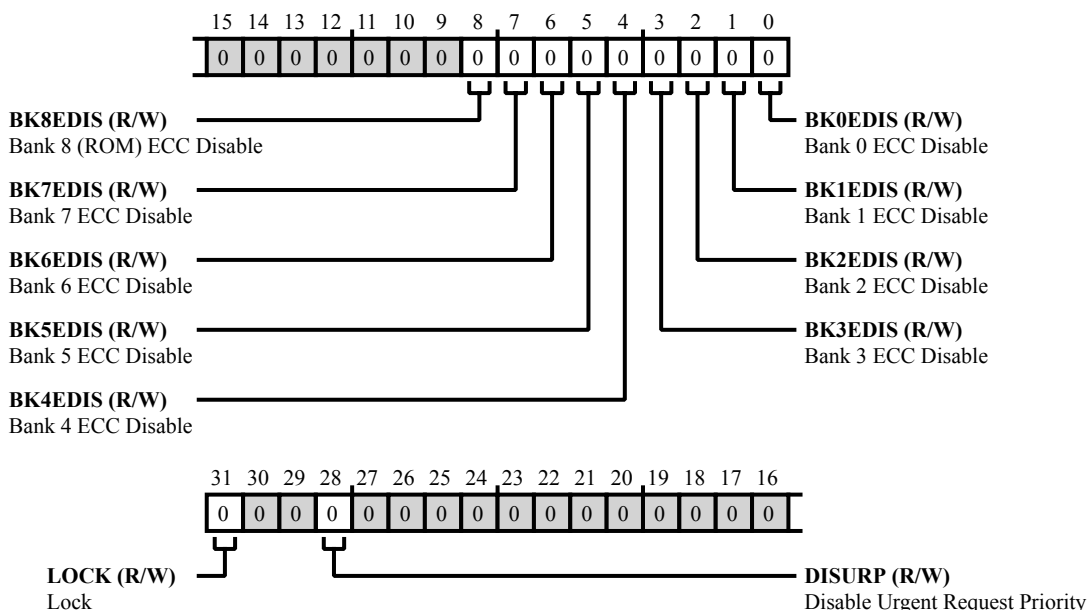


Figure 7-5: L2CTL_CTL Register Diagram

Table 7-7: L2CTL_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>L2CTL_CTL.LOCK</code> bit is set, the <code>L2CTL_CTL</code> register is read only (locked).
		0 Unlock
		1 Lock
28 (R/W)	DISURP	Disable Urgent Request Priority. The <code>L2CTL_CTL.DISURP</code> disables urgent request priority mode for all L2 banks.
		0 Enable URP
		1 Disable URP
8 (R/W)	BK8EDIS	Bank 8 (ROM) ECC Disable. The <code>L2CTL_CTL.BK8EDIS</code> bit disables L2 bank 8 (ROM) ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC

Table 7-7: L2CTL_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	BK7EDIS	Bank 7 ECC Disable. The L2CTL_CTL.BK7EDIS bit disables L2 bank 7 ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC
6 (R/W)	BK6EDIS	Bank 6 ECC Disable. The L2CTL_CTL.BK6EDIS bit disables L2 bank 6 ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC
5 (R/W)	BK5EDIS	Bank 5 ECC Disable. The L2CTL_CTL.BK5EDIS bit disables L2 bank 5 ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC
4 (R/W)	BK4EDIS	Bank 4 ECC Disable. The L2CTL_CTL.BK4EDIS bit disables L2 bank 4 ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC
3 (R/W)	BK3EDIS	Bank 3 ECC Disable. The L2CTL_CTL.BK3EDIS bit disables L2 bank 3 ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC
2 (R/W)	BK2EDIS	Bank 2 ECC Disable. The L2CTL_CTL.BK2EDIS bit disables L2 bank 2 ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC

Table 7-7: L2CTL_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	BK1EDIS	Bank 1 ECC Disable. The L2CTL_CTL.BK1EDIS bit disables L2 bank 1 ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC
0 (R/W)	BK0EDIS	Bank 0 ECC Disable. The L2CTL_CTL.BK0EDIS bit disables L2 bank 0 ECC operation. ECC operation should be enabled or disable before the start of L2 bank access.
		0 Enable ECC
		1 Disable ECC

Error Type 0 Address Register

The `L2CTL_EADDR0` register holds the address that created an access error on the L2 port 0 bus interface. This register is be updated only if the corresponding error status bit (`L2CTL_STAT.ERR0`) is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_EADDR0` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the register captures the write access error address.

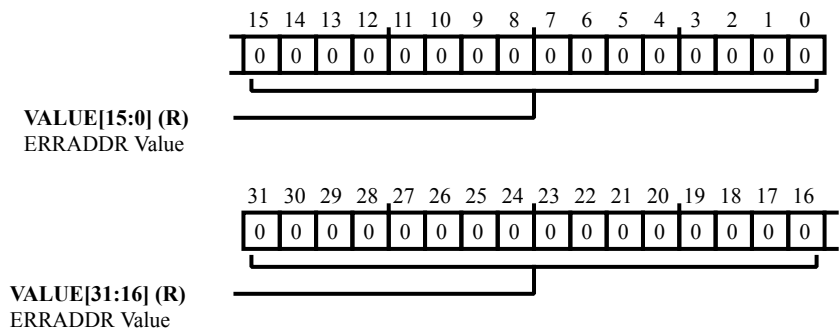


Figure 7-6: L2CTL_EADDR0 Register Diagram

Table 7-8: L2CTL_EADDR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_EADDR0.VALUE</code> bits hold the address causing the bus error.

Error Type 1 Address Register

The `L2CTL_EADDR1` register holds the address that created an access error on the L2 port 1 bus interface. This register is be updated only if the corresponding error status bit (`L2CTL_STAT.ERR1`) is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_EADDR1` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the register captures the write access error address.

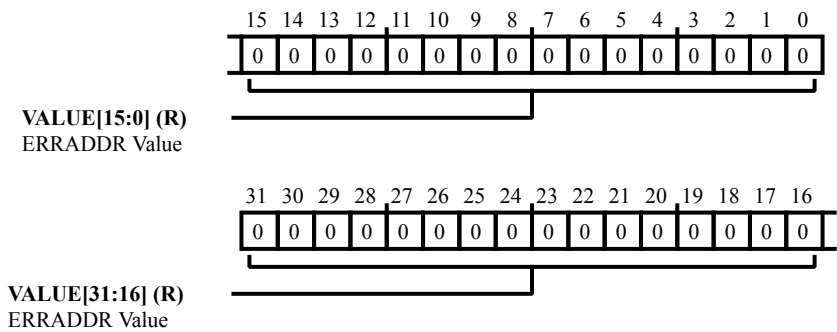


Figure 7-7: L2CTL_EADDR1 Register Diagram

Table 7-9: L2CTL_EADDR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_EADDR1.VALUE</code> bits hold the address causing the bus error.

Error Type 2 Address Register

The `L2CTL_EADDR2` register holds the address that created an access error on the L2 port 2 bus interface (cores). This register is be updated only if the corresponding error status bit (`L2CTL_STAT_1.ERR2`) is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_EADDR2` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the register captures the write access error address.

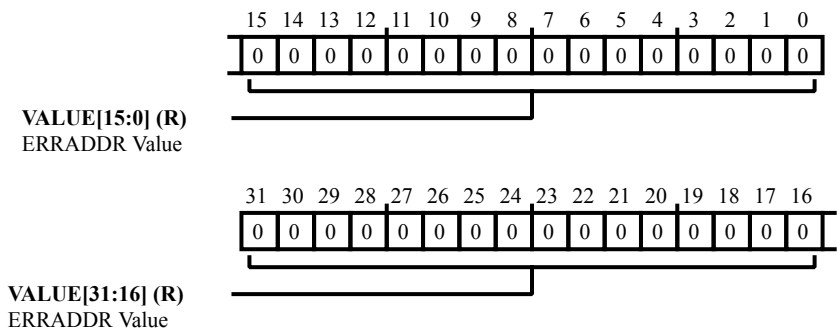


Figure 7-8: L2CTL_EADDR2 Register Diagram

Table 7-10: L2CTL_EADDR2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_EADDR2.VALUE</code> bits hold the address causing the bus error.

Error Type 3 Address Register

The `L2CTL_EADDR3` register holds the address that created an access error on the L2 port 3 bus interface (DMA). This register is be updated only if the corresponding error status bit (`L2CTL_STAT_1.ERR3`) is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_EADDR3` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the register captures the write access error address.

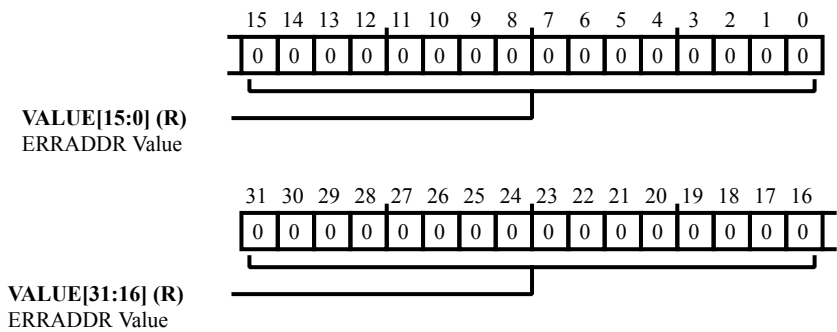


Figure 7-9: L2CTL_EADDR3 Register Diagram

Table 7-11: L2CTL_EADDR3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_EADDR3.VALUE</code> bits hold the address causing the bus error.

Error Type 4 Address Register

The `L2CTL_EADDR4` register holds the address that created an access error on the L2 port 4 bus interface (DMA). This register is be updated only if the corresponding error status bit (`L2CTL_STAT_1.ERR4`) is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_EADDR4` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the register captures the write access error address.

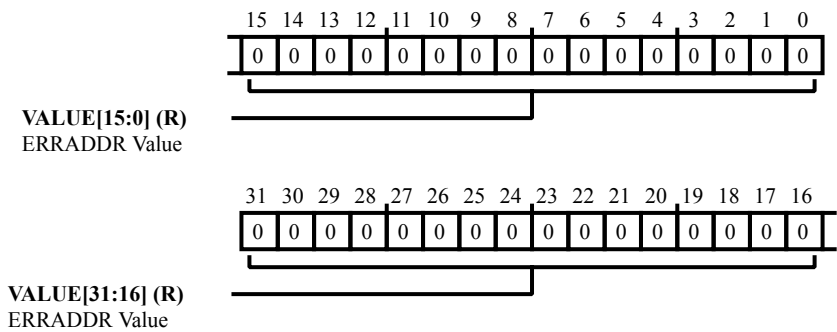


Figure 7-10: L2CTL_EADDR4 Register Diagram

Table 7-12: L2CTL_EADDR4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_EADDR4.VALUE</code> bits hold the address causing the bus error.

ECC Error Address 0 Register

The `L2CTL_ERRADDR0` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR0`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

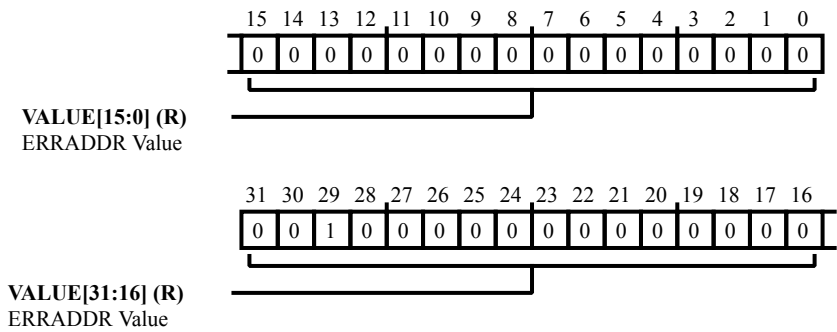


Figure 7-11: L2CTL_ERRADDR0 Register Diagram

Table 7-13: L2CTL_ERRADDR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR0.VALUE</code> bits hold the address containing the ECC double-bit error.

ECC Error Address 1 Register

The `L2CTL_ERRADDR1` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR1`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

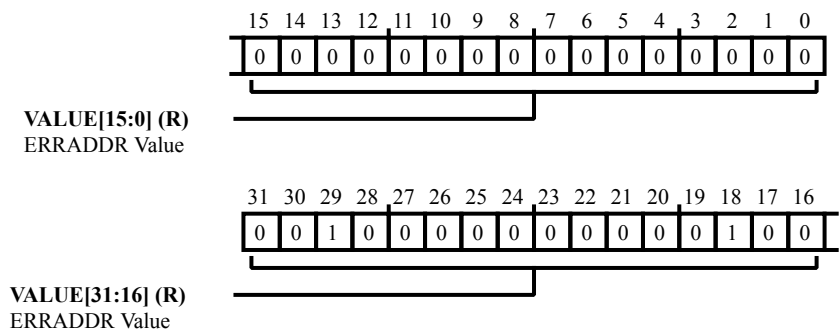


Figure 7-12: L2CTL_ERRADDR1 Register Diagram

Table 7-14: L2CTL_ERRADDR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR1.VALUE</code> bits hold the address containing the ECC double-bit error.

ECC Error Address 2 Register

The `L2CTL_ERRADDR2` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR2`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

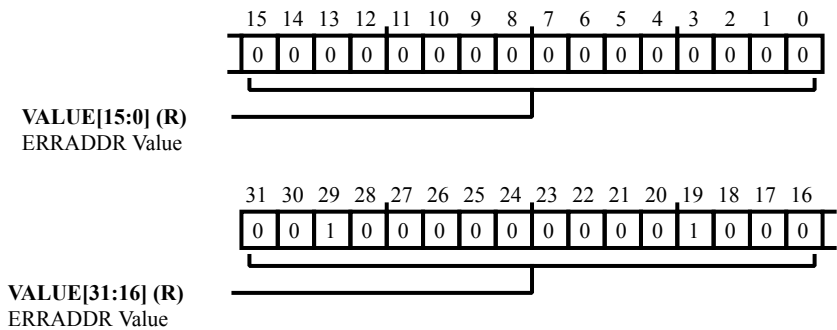


Figure 7-13: L2CTL_ERRADDR2 Register Diagram

Table 7-15: L2CTL_ERRADDR2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR2.VALUE</code> bits hold the address containing the ECC double-bit error.

ECC Error Address 3 Register

The `L2CTL_ERRADDR3` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR3`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

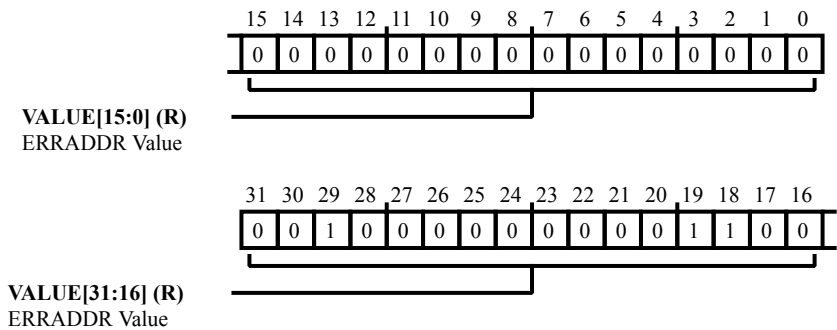


Figure 7-14: L2CTL_ERRADDR3 Register Diagram

Table 7-16: L2CTL_ERRADDR3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR3.VALUE</code> bits hold the address containing the ECC double-bit error.

ECC Error Address 4 Register

The `L2CTL_ERRADDR4` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR4`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

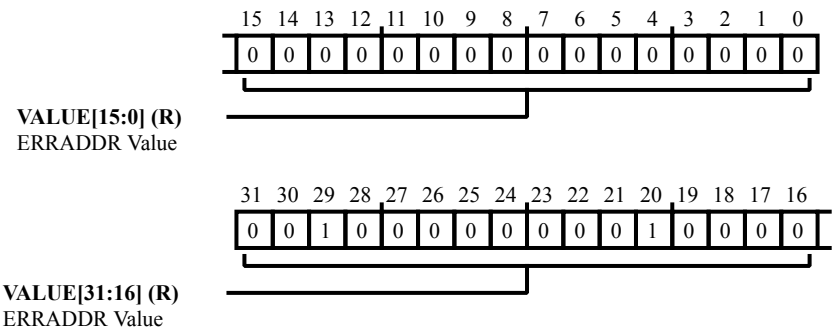


Figure 7-15: L2CTL_ERRADDR4 Register Diagram

Table 7-17: L2CTL_ERRADDR4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR4.VALUE</code> bits hold the address containing the ECC double-bit error.

ECC Error Address 5 Register

The `L2CTL_ERRADDR5` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR5`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

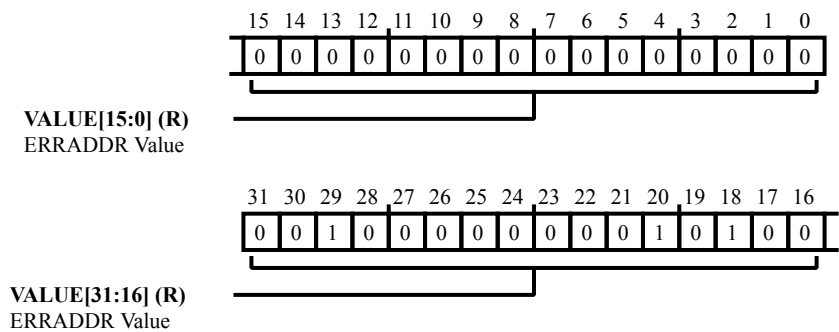


Figure 7-16: L2CTL_ERRADDR5 Register Diagram

Table 7-18: L2CTL_ERRADDR5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR5.VALUE</code> bits hold the address containing the ECC double-bit error.

ECC Error Address 6 Register

The `L2CTL_ERRADDR6` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR6`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

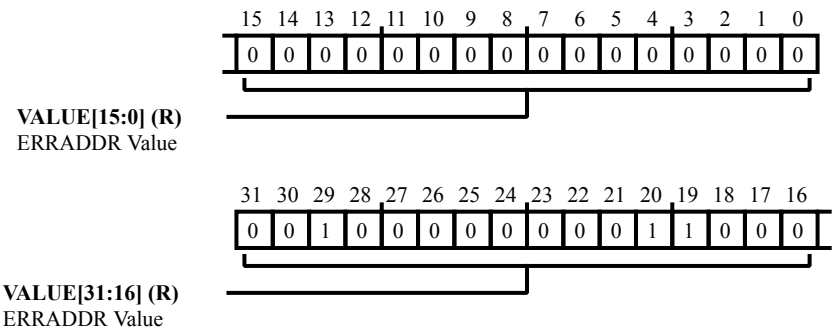


Figure 7-17: L2CTL_ERRADDR6 Register Diagram

Table 7-19: L2CTL_ERRADDR6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR6.VALUE</code> bits hold the address containing the ECC double-bit error.

ECC Error Address 7 Register

The `L2CTL_ERRADDR7` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR7`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

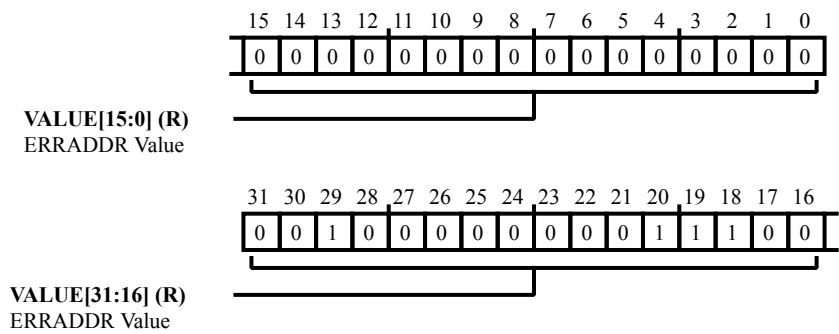


Figure 7-18: L2CTL_ERRADDR7 Register Diagram

Table 7-20: L2CTL_ERRADDR7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR7.VALUE</code> bits hold the address containing the ECC double-bit error.

ECC Error Address 8 Register

The `L2CTL_ERRADDR8` register holds the address containing an ECC multi-bit error for the corresponding bank. The L2CTL updates this register only if the bank’s status bit (`L2CTL_STAT.ECCERR8`) is cleared. After the bank’s status bit is set for an error, further errors in the same bank are not detected until a W1C clears the status bit.

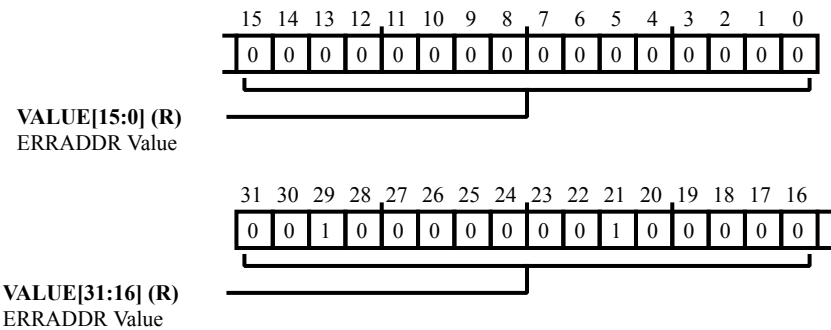


Figure 7-19: L2CTL_ERRADDR8 Register Diagram

Table 7-21: L2CTL_ERRADDR8 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	ERRADDR Value. The <code>L2CTL_ERRADDR8.VALUE</code> bits hold the address containing the ECC double-bit error.

Error Type 0 Register

The `L2CTL_ET0` register holds information about the error transaction that has occurred on the bus for the corresponding L2 bus port 0. This register is updated only if the corresponding error status bit `L2CTL_STAT.ERR0` is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_ET0` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the `L2CTL_ET0` captures the write access error, keeping in sync with the error address register (`L2CTL_EADDR0`).

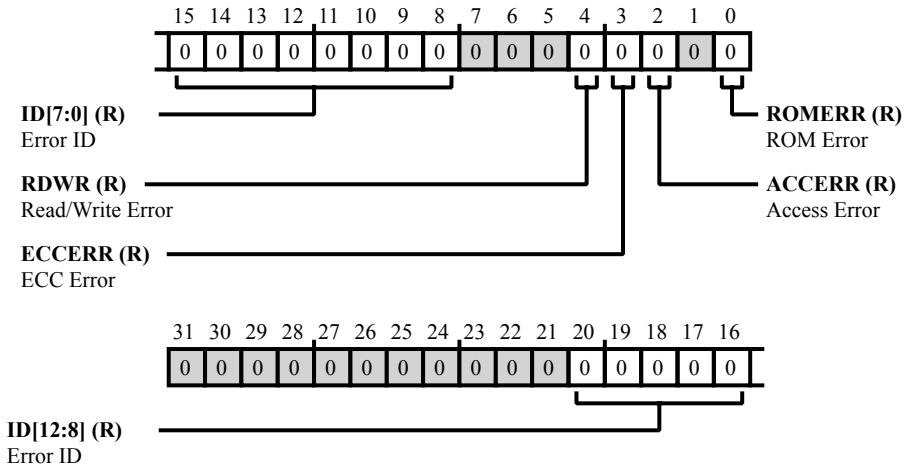


Figure 7-20: L2CTL_ET0 Register Diagram

Table 7-22: L2CTL_ET0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:8 (R/NW)	ID	Error ID. The <code>L2CTL_ET0.ID</code> bits hold the bus master ID of the access that caused an error.
4 (R/NW)	RDWR	Read/Write Error. The <code>L2CTL_ET0.RDWR</code> bit indicates whether a read or write access caused an error.
		0 Read Access created Error
		1 Write Access created Error
3 (R/NW)	ECCERR	ECC Error. The <code>L2CTL_ET0.ECCERR</code> bit indicates whether the access had an ECC double-bit error.
2 (R/NW)	ACCERR	Access Error. The <code>L2CTL_ET0.ACCERR</code> bit indicates whether the access went to a restricted bank.
0 (R/NW)	ROMERR	ROM Error. The <code>L2CTL_ET0.ROMERR</code> bit indicates whether a write access went to a ROM area.

Error Type 1 Register

The `L2CTL_ET1` register holds information about the error transaction that has occurred on the bus for the corresponding L2 bus port 1. This register is updated only if the corresponding error status bit `L2CTL_STAT.ERR1` is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_ET1` register until a `W1C` clears the corresponding status bit. If read and write access errors occur simultaneously, the `L2CTL_ET1` captures the write access error, keeping in sync with the error address register (`L2CTL_EADDR1`).

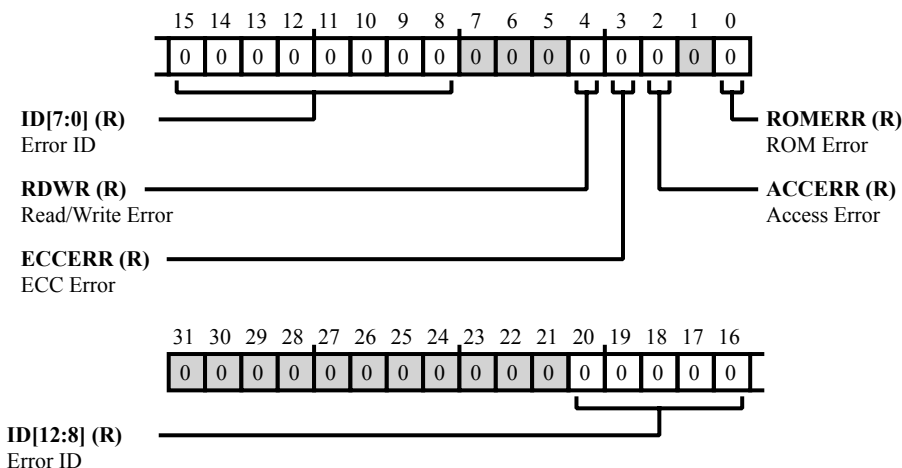


Figure 7-21: L2CTL_ET1 Register Diagram

Table 7-23: L2CTL_ET1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:8 (R/NW)	ID	Error ID. The <code>L2CTL_ET1.ID</code> bits hold the bus master ID of the access that caused an error.
4 (R/NW)	RDWR	Read/Write Error. The <code>L2CTL_ET1.RDWR</code> bit indicates whether a read or write access caused an error.
		0 Read access created error
		1 Write access created error
3 (R/NW)	ECCERR	ECC Error. If the <code>L2CTL_ET1.ECCERR</code> bit =1, the access had an ECC double-bit error.
2 (R/NW)	ACCERR	Access Error. If the <code>L2CTL_ET1.ACCERR</code> bit =1, the access went to a restricted bank.
0 (R/NW)	ROMERR	ROM Error. If the <code>L2CTL_ET1.ROMERR</code> bit =1, a write access went to a ROM area.

Error Type 2 Register

The `L2CTL_ET2` register holds information about the error transaction that has occurred on the bus for the corresponding L2 bus port 2 (cores). This register is updated only if the corresponding error status bit `L2CTL_STAT_1.ERR2` is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_ET2` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the `L2CTL_ET2` captures the write access error, keeping in sync with the error address register (`L2CTL_EADDR2`).

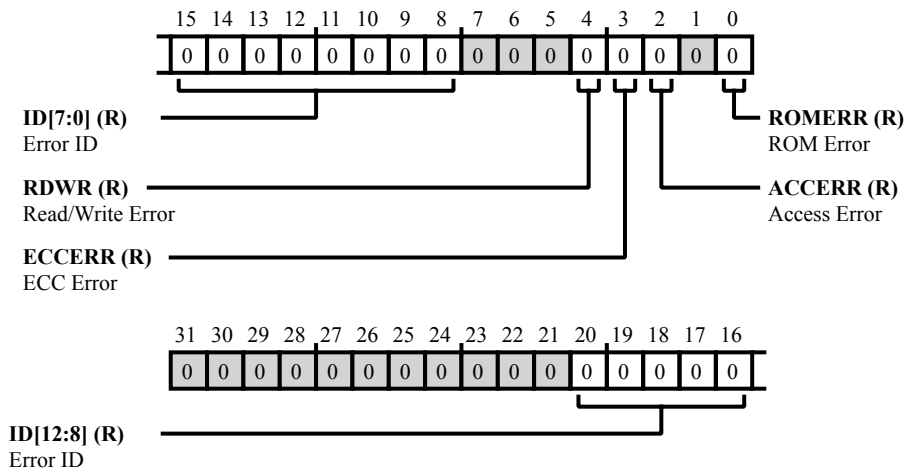


Figure 7-22: L2CTL_ET2 Register Diagram

Table 7-24: L2CTL_ET2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:8 (R/NW)	ID	Error ID. The <code>L2CTL_ET2.ID</code> bits hold the bus master ID of the access that caused an error.
4 (R/NW)	RDWR	Read/Write Error. The <code>L2CTL_ET2.RDWR</code> bit indicates whether a read or write access caused an error.
3 (R/NW)	ECCERR	ECC Error. The <code>L2CTL_ET2.ECCERR</code> bit indicates whether the access had an ECC double-bit error.
2 (R/NW)	ACCERR	Access Error. If the <code>L2CTL_ET2.ACCERR</code> bit =1, the access went to a restricted bank.
0 (R/NW)	ROMERR	ROM Error. If the <code>L2CTL_ET2.ROMERR</code> bit =1, a write access went to a ROM area.

Error Type 3 Register

The `L2CTL_ET3` register holds information about the error transaction that has occurred on the bus for the corresponding L2 bus port 3 (DMA). This register is updated only if the corresponding error status bit `L2CTL_STAT_1.ERR3` is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_ET3` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the `L2CTL_ET3` captures the write access error, keeping in sync with the error address register (`L2CTL_EADDR3`).

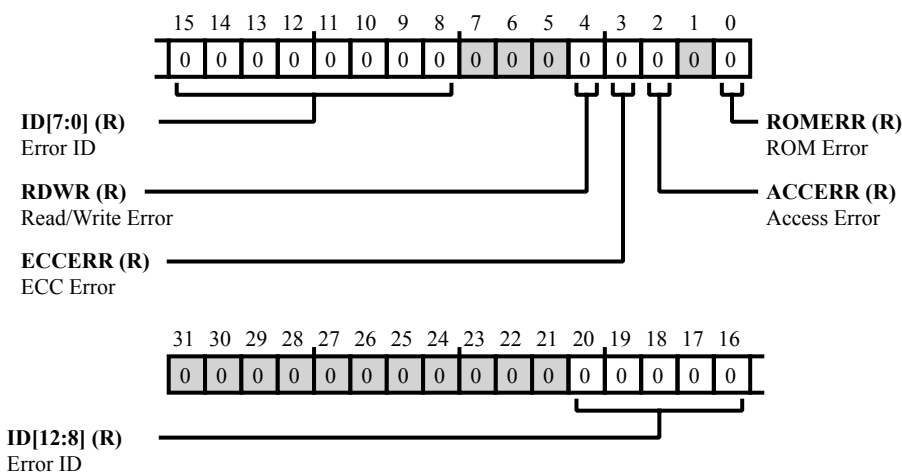


Figure 7-23: L2CTL_ET3 Register Diagram

Table 7-25: L2CTL_ET3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:8 (R/NW)	ID	Error ID. The <code>L2CTL_ET3.ID</code> bits hold the bus master ID of the access that caused an error.
4 (R/NW)	RDWR	Read/Write Error. The <code>L2CTL_ET3.RDWR</code> bit indicates whether a read or write access caused an error.
3 (R/NW)	ECCERR	ECC Error. The <code>L2CTL_ET3.ECCERR</code> bit indicates whether the access had an ECC double-bit error.
2 (R/NW)	ACCERR	Access Error. If the <code>L2CTL_ET3.ACCERR</code> bit =1, the access went to a restricted bank.
0 (R/NW)	ROMERR	ROM Error. If the <code>L2CTL_ET3.ROMERR</code> bit =1, a write access went to a ROM area.

Error Type 4 Register

The `L2CTL_ET4` register holds information about the error transaction that has occurred on the bus for the corresponding L2 bus port 4 (DMA). This register is updated only if the corresponding error status bit `L2CTL_STAT_1.ERR4` is cleared. After the status bit is set for an error, further errors do not update the `L2CTL_ET4` register until a W1C clears the corresponding status bit. If read and write access errors occur simultaneously, the `L2CTL_ET4` captures the write access error, keeping in sync with the error address register (`L2CTL_EADDR4`).

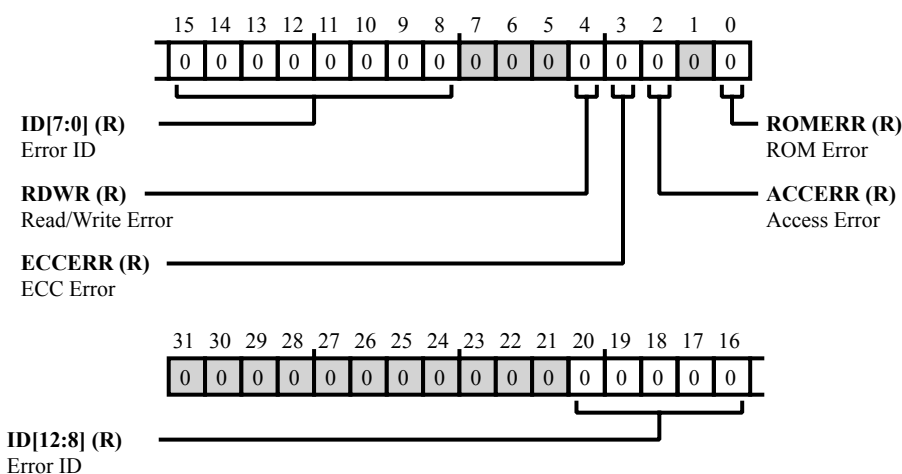


Figure 7-24: L2CTL_ET4 Register Diagram

Table 7-26: L2CTL_ET4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:8 (R/NW)	ID	Error ID. The <code>L2CTL_ET4.ID</code> bits hold the bus master ID of the access that caused an error.
4 (R/NW)	RDWR	Read/Write Error. The <code>L2CTL_ET4.RDWR</code> bit indicates whether a read or write access caused an error.
3 (R/NW)	ECCERR	ECC Error. The <code>L2CTL_ET4.ECCERR</code> bit indicates whether the access had an ECC double-bit error.
2 (R/NW)	ACCERR	Access Error. If the <code>L2CTL_ET4.ACCERR</code> bit =1, the access went to a restricted bank.
0 (R/NW)	ROMERR	ROM Error. If the <code>L2CTL_ET4.ROMERR</code> bit =1, a write access went to a ROM area.

Initialization Register

The `L2CTL_INIT` register initializes memory banks with 128'b0 and ECC bits corresponding to 128'b0. Any writes to the bits in this register while initialization is occurring to any of the banks is ignored. All bits are W1A (Write 1 for Action).

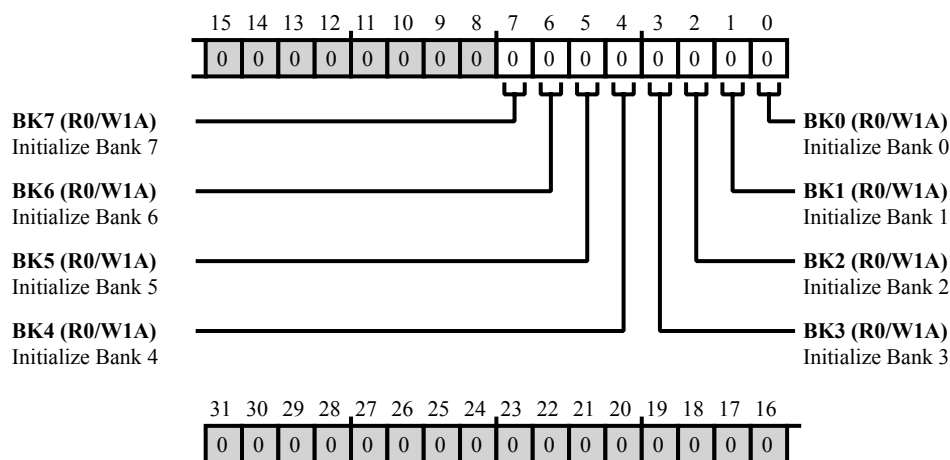


Figure 7-25: L2CTL_INIT Register Diagram

Table 7-27: L2CTL_INIT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R0/W1A)	BK7	Initialize Bank 7. The write-1 to the <code>L2CTL_INIT.BK7</code> bit initializes bank 7 with 128b0 and ECC bits corresponding to 128b0. Any write to this bit during the initialization of the banks is ignored.
6 (R0/W1A)	BK6	Initialize Bank 6. The write-1 to the <code>L2CTL_INIT.BK6</code> bit initializes bank 6 with 128b0 and ECC bits corresponding to 128b0. Any write to this bit during the initialization of the banks is ignored.
5 (R0/W1A)	BK5	Initialize Bank 5. The write -1 to the <code>L2CTL_INIT.BK5</code> bit initializes bank 5 with 128b0 and ECC bits corresponding to 128b0. Any write to this bit during the initialization of the banks is ignored.
4 (R0/W1A)	BK4	Initialize Bank 4. The write-1 to the <code>L2CTL_INIT.BK4</code> bit initializes bank 4 with 128b0 and ECC bits corresponding to 128b0. Any write to this bit during the initialization of the banks is ignored.

Table 7-27: L2CTL_INIT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R0/W1A)	BK3	Initialize Bank 3. The write-1 to the L2CTL_INIT.BK3 bit initializes bank 3 with 128b0 and ECC bits corresponding to 128b0. Any write to this bit during the initialization of the banks is ignored.
2 (R0/W1A)	BK2	Initialize Bank 2. The write-1 to the L2CTL_INIT.BK2 bit initializes bank 2 with 128b0 and ECC bits corresponding to 128b0. Any write to this bit during the initialization of the banks is ignored.
1 (R0/W1A)	BK1	Initialize Bank 1. The write-1 to the L2CTL_INIT.BK1 bit initializes bank 1 with 128b0 and ECC bits corresponding to 128b0. Any write to this bit during the initialization of the banks is ignored.
0 (R0/W1A)	BK0	Initialize Bank 0. The write-1 to the L2CTL_INIT.BK0 bit initializes bank 0 with 128b0 and ECC bits corresponding to 128b0. Any write to this bit during the initialization of the banks is ignored.

Initialization Status Register

The `L2CTL_ISTAT` register holds the status of the RAM bank initialization. If set, the corresponding bank is initialized.

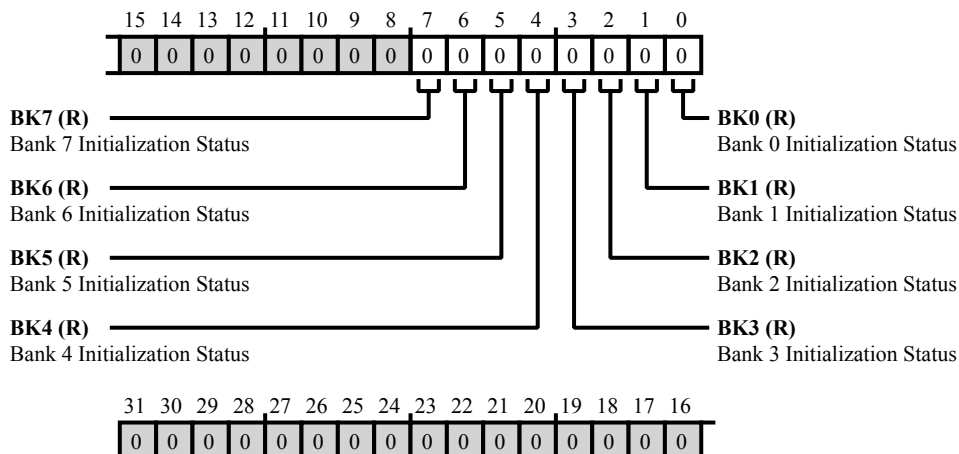


Figure 7-26: L2CTL_ISTAT Register Diagram

Table 7-28: L2CTL_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	BK7	Bank 7 Initialization Status. The <code>L2CTL_ISTAT.BK7</code> bits hold the bank 7 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
6 (R/NW)	BK6	Bank 6 Initialization Status. The <code>L2CTL_ISTAT.BK6</code> bits hold the bank 6 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
5 (R/NW)	BK5	Bank 5 Initialization Status. The <code>L2CTL_ISTAT.BK5</code> bits hold the bank 5 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
4 (R/NW)	BK4	Bank 4 Initialization Status. The <code>L2CTL_ISTAT.BK4</code> bits hold the bank 4 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
3 (R/NW)	BK3	Bank 3 Initialization Status. The <code>L2CTL_ISTAT.BK3</code> bits hold the bank 3 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.
2 (R/NW)	BK2	Bank 2 Initialization Status. The <code>L2CTL_ISTAT.BK2</code> bits hold the bank 2 initialization status. A W1A on a <code>BKxINIT</code> bit clears this bit and the bit is set when initialization completes.

Table 7-28: L2CTL_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	BK1	Bank 1 Initialization Status. The L2CTL_ISTAT.BK1 bits hold the bank 1 initialization status. A W1A on a BKxINIT bit clears this bit and the bit is set when initialization completes.
0 (R/NW)	BK0	Bank 0 Initialization Status. The L2CTL_ISTAT.BK0 bits hold the bank 0 initialization status. A W1A on a BKxINIT bit clears this bit and the bit is set when initialization completes.

Power Control Register

The `L2CTL_PCTL` register has the various control settings for selectively enabling the deep sleep and shut down power saving features.

NOTE: The corresponding L2 bank should not be accessed if the power control feature is enabled for that bank. An access to a bank in deep sleep/shut down may create unpredictable behavior.

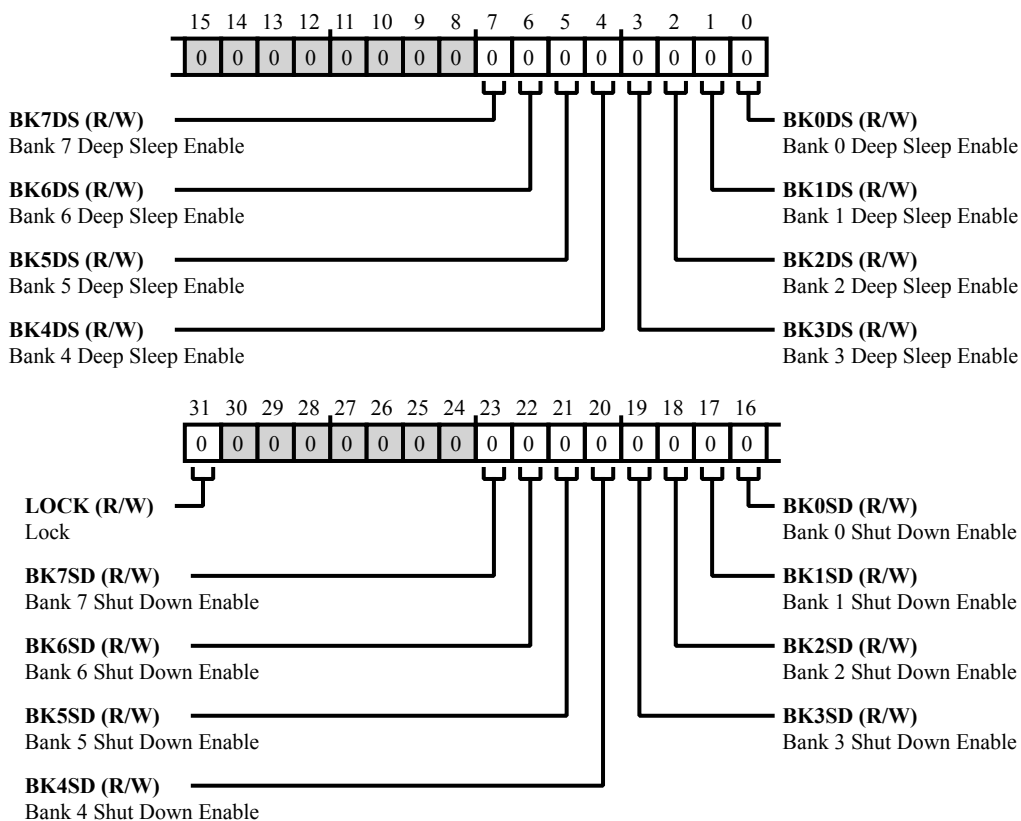


Figure 7-27: L2CTL_PCTL Register Diagram

Table 7-29: L2CTL_PCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>L2CTL_PCTL.LOCK</code> bit is set, the <code>L2CTL_PCTL</code> register is read only (locked).
		0 Unlock 1 Lock
23 (R/W)	BK7SD	Bank 7 Shut Down Enable. The <code>L2CTL_PCTL.BK7SD</code> bits enables bank 7 shut down.

Table 7-29: L2CTL_PCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	BK6SD	Bank 6 Shut Down Enable. The L2CTL_PCTL.BK6SD bits enables bank 6 shut down.
21 (R/W)	BK5SD	Bank 5 Shut Down Enable. The L2CTL_PCTL.BK5SD bits enables bank 5 shut down.
20 (R/W)	BK4SD	Bank 4 Shut Down Enable. The L2CTL_PCTL.BK4SD bits enables bank 4 shut down.
19 (R/W)	BK3SD	Bank 3 Shut Down Enable. The L2CTL_PCTL.BK3SD bits enables bank 3 shut down.
18 (R/W)	BK2SD	Bank 2 Shut Down Enable. The L2CTL_PCTL.BK2SD bits enables bank 2 shut down.
17 (R/W)	BK1SD	Bank 1 Shut Down Enable. The L2CTL_PCTL.BK1SD bits enables bank 1 shut down.
16 (R/W)	BK0SD	Bank 0 Shut Down Enable. The L2CTL_PCTL.BK0SD bits enables bank 0 shut down.
7 (R/W)	BK7DS	Bank 7 Deep Sleep Enable. The L2CTL_PCTL.BK7DS bits enables bank 7 deep sleep.
6 (R/W)	BK6DS	Bank 6 Deep Sleep Enable. The L2CTL_PCTL.BK6DS bits enables bank 6 deep sleep.
5 (R/W)	BK5DS	Bank 5 Deep Sleep Enable. The L2CTL_PCTL.BK5DS bits enables bank 5 deep sleep.
4 (R/W)	BK4DS	Bank 4 Deep Sleep Enable. The L2CTL_PCTL.BK4DS bits enables bank 4 deep sleep.
3 (R/W)	BK3DS	Bank 3 Deep Sleep Enable. The L2CTL_PCTL.BK3DS bits enables bank 3 deep sleep.
2 (R/W)	BK2DS	Bank 2 Deep Sleep Enable. The L2CTL_PCTL.BK2DS bits enables bank 2 deep sleep.
1 (R/W)	BK1DS	Bank 1 Deep Sleep Enable. The L2CTL_PCTL.BK1DS bits enables bank 1 deep sleep.
0 (R/W)	BK0DS	Bank 0 Deep Sleep Enable. The L2CTL_PCTL.BK0DS bits enables bank 0 deep sleep.

Revision ID Register

The `L2CTL_REVID` register provides the L2 Revision ID.

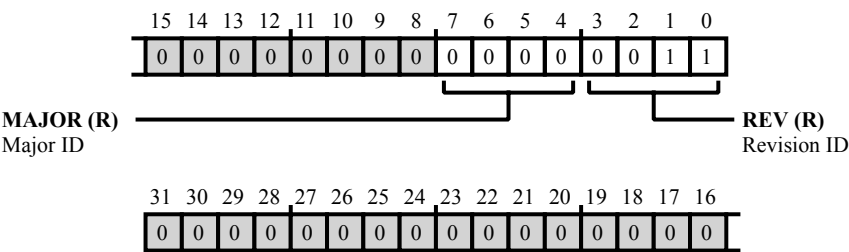


Figure 7-28: L2CTL_REVID Register Diagram

Table 7-30: L2CTL_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major ID. The <code>L2CTL_REVID.MAJOR</code> bit indicates L2 Major ID.
3:0 (R/NW)	REV	Revision ID. The <code>L2CTL_REVID.REV</code> bit indicates the L2 Revision ID.

Read Priority Count Register

The `L2CTL_RPCR0` register stores the count value to be used for priority elevation for bus read channels of Port0 to Port3. If a bus channel is not granted access from the bank arbiter, the channel waits for the programmed number (stored in `RPCx`) of `SYSCLK` cycles, before the request is elevated to a high priority request. If a priority count value is programmed as zero for a channel, that channel does not raise the urgent priority request. Default value wait cycle for each channel is 15..

This is a read/write register, but a new value in the corresponding field must be written only when there are no outstanding transactions on the corresponding bus read channel. A best practice is to program this register before initiating an L2 access.

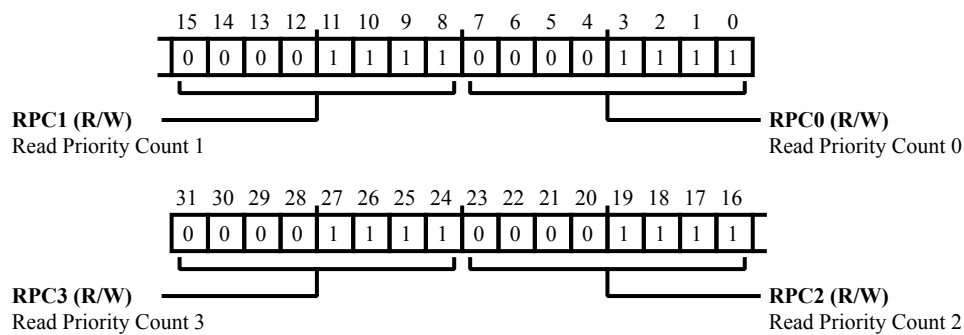


Figure 7-29: L2CTL_RPCR0 Register Diagram

Table 7-31: L2CTL_RPCR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	RPC3	Read Priority Count 3. The <code>L2CTL_RPCR0.RPC3</code> bits hold the priority count for L2 bus read channel 3.
23:16 (R/W)	RPC2	Read Priority Count 2. The <code>L2CTL_RPCR0.RPC2</code> bits hold the priority count for L2 bus read channel 2.
15:8 (R/W)	RPC1	Read Priority Count 1. The <code>L2CTL_RPCR0.RPC1</code> bits hold the priority count for L2 bus read channel 1.
7:0 (R/W)	RPC0	Read Priority Count 0. The <code>L2CTL_RPCR0.RPC0</code> bits hold the priority count for L2 bus read channel 0.

Read Priority Count Register

The `L2CTL_RPCR1` register stores the count value to be used for priority elevation for bus read channel 4. If a bus channel is not granted access from the bank arbiter, the channel waits for the programmed number (stored in `RPC4`) of `SYSCCLK` cycles, before the request is elevated to a high priority request. If a priority count value is programmed as zero for a channel, that channel does not raise the urgent priority request. Default value of `RPC4` is 15.

This is a read/write register, but a new value in the corresponding field must be written only when there are no outstanding transactions on the corresponding bus read channel. A best practice is to program this register before initiating an L2 access.

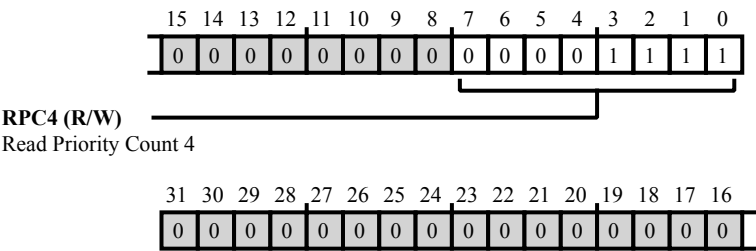


Figure 7-30: L2CTL_RPCR1 Register Diagram

Table 7-32: L2CTL_RPCR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	RPC4	Read Priority Count 4. The <code>L2CTL_RPCR1</code> . <code>RPC4</code> bits hold the priority count for L2 bus read channel 4.

Scrub Start Address Register

The `L2CTL_SADR` register stores the scrub start address value. Writes to this register can be prevented by setting the `L2CTL_SCTL.LOCK` bit and the global lock.

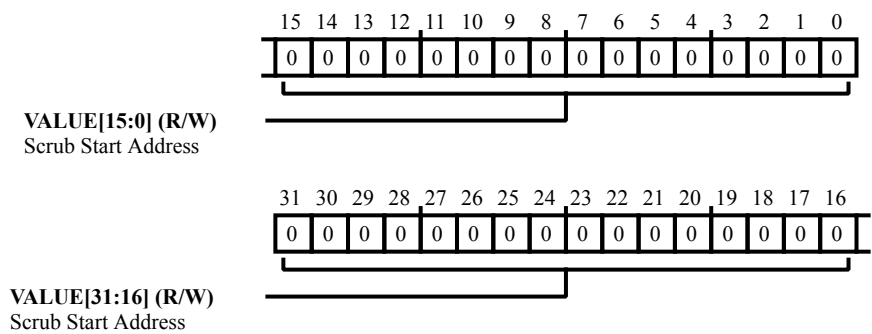


Figure 7-31: L2CTL_SADR Register Diagram

Table 7-33: L2CTL_SADR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Scrub Start Address. The <code>L2CTL_SADR.VALUE</code> bits hold the scrub start address. The writes to this register can be prevented by setting the <code>L2_SCTL.LOCK</code> and the global lock.

Scrub Count Register

The `L2CTL_SCNT` register determines the number of 128-bit locations scrubbed starting from the start address (`L2CTL_SADR` register). Writes to the `L2CTL_SCNT` register can be prevented by setting the `L2CTL_CTL.LOCK` bit and the global lock.

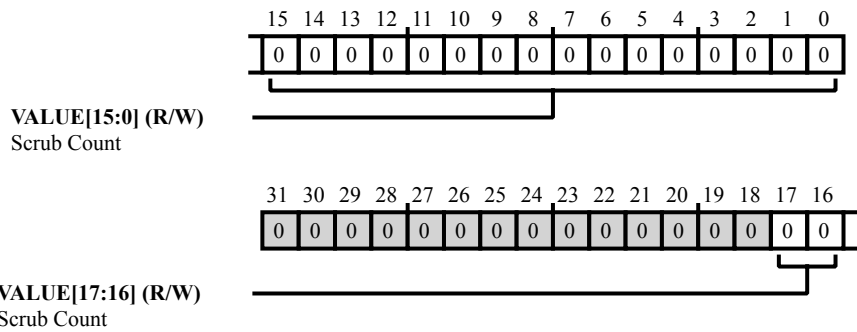


Figure 7-32: L2CTL_SCNT Register Diagram

Table 7-34: L2CTL_SCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17:0 (R/W)	VALUE	Scrub Count. The <code>L2CTL_SCNT.VALUE</code> bits determines the number of 128-bit locations scrubbed starting from the start address. Ensure value programmed is less than the total addressable 128-bit L2 RAM locations available.

Scrub Control Register

The `L2CTL_SCTL` register holds the automatic scrub related controls. This is a read/write register. Memory scrub can be performed by programming the `L2CTL_SADR` (scrub start address) register with the start address of the scrub and the `L2CTL_SCNT` register with the total number of 128-bit addresses to be scrubbed starting from the `L2CTL_SADR` register. The number of cycles between each scrub can be programmed with the `L2CTL_SCTL.SRT` bit.

During the scrub the controller issues a read followed by write back if there is a single bit ECC error. Once the L2 controller completes scrubbing the memory region mentioned using the `L2CTL_SADR` and the `L2CTL_SCNT` registers, an interrupt is generated and scrubbing re-starts from the start unless the scrub enable bit is disabled in the control register. If scrub enable is cleared before completing the address range used in the `L2CTL_SADR` and `L2CTL_SCNT` registers, the scrub stops after completing any already issued scrub access.

The scrub read/writes always start from the full 128-bit equivalent of the address written into the `L2CTL_SADR` register. A 128-bit value is always read and the 128-bit value is written back. The scrub access has the highest priority. Programs can configure 8-, 16-, or 32-bit addresses in the `L2CTL_SADR` register but the lower 3 bits are treated as don't care because the internal memory array is always accessed in a 128-bit mode.

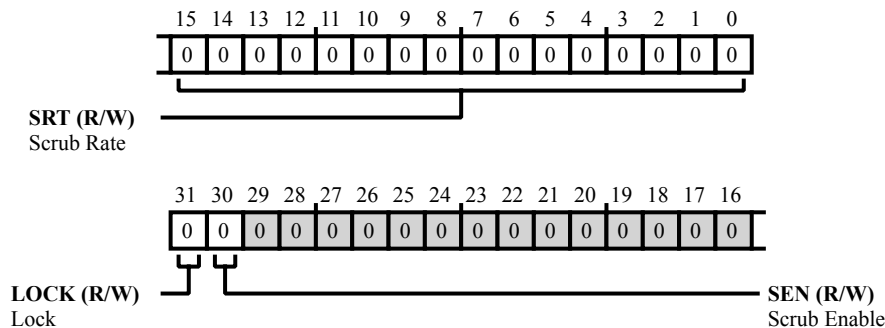


Figure 7-33: L2CTL_SCTL Register Diagram

Table 7-35: L2CTL_SCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock.
		If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>L2CTL_SCTL.LOCK</code> bit is set, the <code>L2CTL_SCTL</code> register is read only (locked).
		0 Unlock
		1 Lock
30 (R/W)	SEN	Scrub Enable. The <code>L2CTL_SCTL.SEN</code> bits enable automatic scrub.

Table 7-35: L2CTL_SCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	SRT	Scrub Rate. The L2CTL_SCTL.SRT bits determines the number of clock cycles that elapsed between two automatic scrubs.

Status Register

The `L2CTL_STAT` register indicates ECC error status, refresh register status, and bus error status of Port0 and Port1.

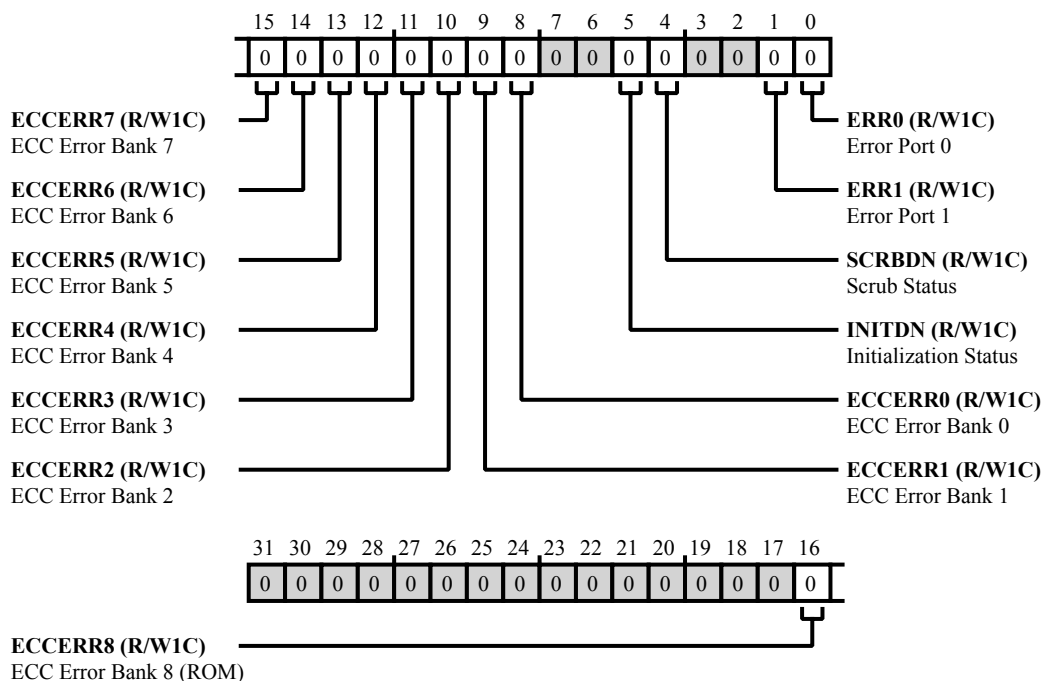


Figure 7-34: L2CTL_STAT Register Diagram

Table 7-36: L2CTL_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W1C)	ECCERR8	ECC Error Bank 8 (ROM). The <code>L2CTL_STAT.ECCERR8</code> bit indicates that an ECC double-bit error occurred inside L2 bank 8 (ROM).
		0 No Status
		1 ECC Double Bit Error
15 (R/W1C)	ECCERR7	ECC Error Bank 7. The <code>L2CTL_STAT.ECCERR7</code> bit indicates that an ECC double-bit error occurred inside L2 bank 7.
		0 No Status
		1 ECC Double Bit Error

Table 7-36: L2CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W1C)	ECCERR6	ECC Error Bank 6. The L2CTL_STAT.ECCERR6 bit indicates that an ECC double-bit error occurred inside L2 bank 6.
		0 No Status
		1 ECC Double Bit Error
13 (R/W1C)	ECCERR5	ECC Error Bank 5. The L2CTL_STAT.ECCERR5 bit indicates that an ECC double-bit error occurred inside L2 bank 5.
		0 No Status
		1 ECC Double Bit Error
12 (R/W1C)	ECCERR4	ECC Error Bank 4. The L2CTL_STAT.ECCERR4 bit indicates that an ECC double-bit error occurred inside L2 bank 4.
		0 No Status
		1 ECC Double Bit Error
11 (R/W1C)	ECCERR3	ECC Error Bank 3. The L2CTL_STAT.ECCERR3 bit indicates that an ECC double-bit error occurred inside L2 bank 3.
		0 No Status
		1 ECC Double Bit Error
10 (R/W1C)	ECCERR2	ECC Error Bank 2. The L2CTL_STAT.ECCERR2 bit indicates that an ECC double-bit error occurred inside L2 bank 2.
		0 No Status
		1 ECC Double Bit Error
9 (R/W1C)	ECCERR1	ECC Error Bank 1. The L2CTL_STAT.ECCERR1 bit indicates that an ECC double-bit error occurred inside L2 bank 1.
		0 No Status
		1 ECC Double Bit Error

Table 7-36: L2CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	ECCERR0	ECC Error Bank 0. The L2CTL_STAT.ECCERR0 bit indicates that an ECC double-bit error occurred inside L2 bank 0.
		0 No Status
		1 ECC Double Bit Error
5 (R/W1C)	INITDN	Initialization Status. The L2CTL_STAT.INITDN bit indicates whether initialization has been completed.
		0 Initialization Not Complete
		1 Initialization Completed
4 (R/W1C)	SCRBDN	Scrub Status. The L2CTL_STAT.SCRBDN bit indicates whether a round of memory scrub has completed.
		0 Scrub Not Complete
		1 Scrub Completed
1 (R/W1C)	ERR1	Error Port 1. The L2CTL_STAT.ERR1 indicates whether the L2CTL has detected a bus access error on L2s bus port 1.
		0 No Error
		1 Bus Access Error
0 (R/W1C)	ERR0	Error Port 0. The L2CTL_STAT.ERR0 indicates whether the L2CTL has detected a bus access error on L2s bus port 0.
		0 No Error
		1 Bus Access Error

L2 Port Error Status Register

The `L2CTL_STAT_1` register stores the AXI Error Status from Port2 onwards. The register bits are sticky - once set it has to be explicitly reset by writing a 1 to that particular bit.

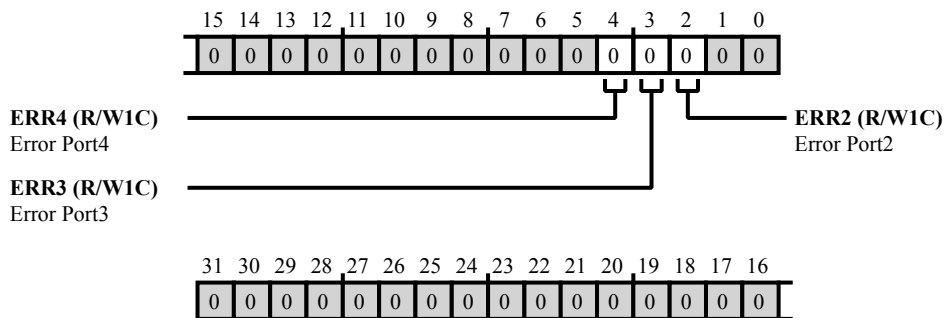


Figure 7-35: L2CTL_STAT_1 Register Diagram

Table 7-37: L2CTL_STAT_1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W1C)	ERR4	Error Port4. The <code>L2CTL_STAT_1</code> .ERR4 indicates whether the L2CTL has detected a bus access error on L2s bus port 4.
3 (R/W1C)	ERR3	Error Port3. The <code>L2CTL_STAT_1</code> .ERR3 indicates whether the L2CTL has detected a bus access error on L2s bus port 3.
2 (R/W1C)	ERR2	Error Port2. The <code>L2CTL_STAT_1</code> .ERR2 indicates whether the L2CTL has detected a bus access error on L2s bus port 2.

Write Priority Count Register

The `L2CTL_WPCR0` register stores the count value to be used for priority elevation for bus write channels. If a bus channel is not granted access from the bank arbiter, the channel waits for the programmed number of SYSCLK cycles, before the request is elevated to a high priority request. If a priority count value is programmed as zero for a channel, that channel does not raise the urgent priority request.

This is a read/write register, but a new value in the corresponding field must be written only when there are no outstanding transactions on the corresponding bus write channel. A best practice is to program this register before initiating an L2 access.

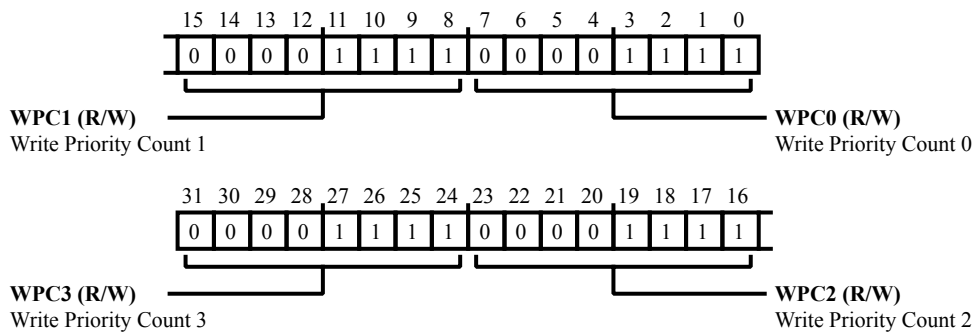


Figure 7-36: L2CTL_WPCR0 Register Diagram

Table 7-38: L2CTL_WPCR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	WPC3	Write Priority Count 3. The <code>L2CTL_WPCR0.WPC3</code> bits hold the priority count for L2 bus write channel 3.
23:16 (R/W)	WPC2	Write Priority Count 2. The <code>L2CTL_WPCR0.WPC2</code> bits hold the priority count for L2 bus write channel 2.
15:8 (R/W)	WPC1	Write Priority Count 1. The <code>L2CTL_WPCR0.WPC1</code> bits hold the priority count for L2 bus write channel 1.
7:0 (R/W)	WPC0	Write Priority Count 0. The <code>L2CTL_WPCR0.WPC0</code> bits hold the priority count for L2 bus write channel 0.

Write Priority Count Register

The `L2CTL_WPCR1` register stores the count value to be used for priority elevation for bus write channels. If a bus channel is not granted access from the bank arbiter, the channel waits for the programmed number of `SYSCLK` cycles, before the request is elevated to a high priority request. If a priority count value is programmed as zero for a channel, that channel does not raise the urgent priority request.

This is a read/write register, but a new value in the corresponding field must be written only when there are no outstanding transactions on the corresponding bus write channel. A best practice is to program this register before initiating an L2 access.

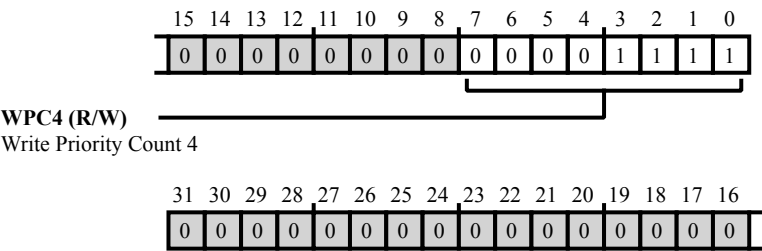


Figure 7-37: L2CTL_WPCR1 Register Diagram

Table 7-39: L2CTL_WPCR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	WPC4	Write Priority Count 4. The <code>L2CTL_WPCR1.WPC4</code> bits hold the priority count for L2 bus write channel 4.

8 Dynamic Memory Controller (DMC)

The dynamic memory controller (DMC) provides a glueless interface between DDR3 SDRAMs and the system crossbar interface (SCB). The DMC enables execution of instructions from, as well as transfer of data to and from, DDR3 SDRAM respectively.

NOTE: The term DDR3 is referred to generically as DDR SDRAM in the rest of this chapter unless otherwise noted.

The DMC is partitioned in a manner that allows reconfiguration and maintainability. The memory access protocol state machine along with JEDEC standard specific logic is embedded in the *protocol controller*. An access and operation reordering mechanism is incorporated as an *efficiency controller*. An SCB slave interface is provided to interface with the on-chip interconnect. This interface results in an efficient slave implementation owing to its out-of-order transaction capabilities. The control and status registers present in the DMC can be accessed using the MMR access bus.

The DMC supports access to the external memory by core and DMA accesses.

DMC Features

The DMC includes a protocol controller that supports:

- JESD79-3E compatible double data rate DDR3 SDRAM devices

The features of the dynamic memory controller are:

- Provides 16-bit data only interface to SDRAM devices
- Supports a single external rank (one chip select)
- Provides page hit detection that supports multiple column accesses to the same row
- User-specified active, precharge, and refresh commands.
- Programmable SDRAM access timing parameters
- Enables automatic refresh generation with programmable refresh intervals
- Self-refresh mode to reduce system power consumption
- Efficient transaction processing to improve throughput and bandwidth using:

- Software programmable SCB IDs to allow SCB ID-based priority
- The ability to postpone up to eight auto-refresh commands
- Software selectable closed page scheme on a per bank basis
- Simple transaction scheduling mechanism to reduce read write turnaround frequency on the memory bus
- Accesses with the same SCB ID are scheduled back-to-back to take advantage of same page access in SDRAM
- Caching of SDRAM read data burst for specific masters to reduce the latency for same burst accesses.

The DDR3 features are:

- 512 Mb to 8 Gb device sizes
- Burst length BL = 8
- Support for additive latency
- Support for programmable (and ZQ calibration) ODT and drive impedance
- Support for read leveling
- Support for write leveling

Feature Exclusions

The DMC exclusions are as follows:

For DDR3:

- 4-bit and 8-bit wide DDR3 DRAM memories are not supported
- Burst interleaved accesses are not supported
- Both burst chop-BC4 and BC4-on-the-fly are not supported
- Auto refresh pull-in is not supported
- DLL off mode is not supported

DMC Functional Description

The dynamic memory controller consists of master and slave interfaces, a protocol controller, and an efficiency controller. The following sections describe the function of these interfaces and controllers.

ADSP-2159x DMC Register List

The Dynamic Memory Controller module (DMC) provides an interface to external double-data-rate SDRAM. This interface supports various DDR standards (see chapter descriptions). A set of registers governs DMC controller operations. For more information on DMC controller functionality, see the DMC Controller Register Descriptions.

Table 8-1: ADSP-2159x DMC Register List

Name	Description
DMC_CFG	Configuration Register
DMC_CTL	Control Register
DMC_DLLCTL	DLL Control Register
DMC_DT_CALIB_ADDR	Data Calibration Address Register
DMC_DT_DATA_CALIB_DATA0	Data Calibration Data 0 Register
DMC_DT_DATA_CALIB_DATA1	Data Calibration Data 1 Register
DMC_EFFCTL	Efficiency Control Register
DMC_EM3	Shadow EM3 Register
DMC_MR	Shadow MR0 Register (DDR3)
DMC_MR1	Shadow MR1 Register (DDR3)
DMC_MR2	Shadow MR2 Register (DDR3)
DMC_MSK	Mask (Mode Register Shadow) Register
DMC_PRIO	Priority ID Register 1
DMC_PRIO2	Priority ID Register 2
DMC_PRIOMSK	Priority ID Mask Register 1
DMC_PRIOMSK2	Priority ID Mask Register 2
DMC_RDDATABUFID1	DMC Read Data Buffer ID Register 1
DMC_RDDATABUFID2	DMC Read Data Buffer ID Register 2
DMC_RDDATABUFMSK1	DMC Read Data Buffer Mask Register 1
DMC_RDDATABUFMSK2	DMC Read Data Buffer Mask Register 2
DMC_STAT	Status Register
DMC_TR0	Timing 0 Register
DMC_TR1	Timing 1 Register
DMC_TR2	Timing 2 Register

ADSP-2159x DMC Register List

DMCPHY (DMC) contains the following registers.

Table 8-2: ADSP-2159x DMC Register List

Name	Description
DMC_DDR_CA_CTL	DDR CA Lane Control Register
DMC_DDR_LANE0_CTL0	Data Lane 0 Control Register 0
DMC_DDR_LANE0_CTL1	Data Lane 0 Control Register 1
DMC_DDR_LANE1_CTL0	Data Lane 1 Control Register 0
DMC_DDR_LANE1_CTL1	Data Lane 1 Control Register 1
DMC_DDR_ROOT_CTL	DDR ROOT Module Control Register
DMC_DDR_ZQ_CTL0	DDR Calibration Control Register 0
DMC_DDR_ZQ_CTL1	DDR Calibration Control Register 1
DMC_DDR_ZQ_CTL2	DDR Calibration Control Register 2

Protocol Controller

The DDR SDRAM protocol controller translates memory access requests from the SCB (system crossbar) interface to JEDEC protocol-specific transactions used by DDR SDRAM devices.

The protocol controller ensures that the various timing parameters are met before reading and writing the SDRAM. The controller also performs the SDRAM initialization sequence as mandated by the standard. The protocol controller can:

- Issue reads and writes
- Precharge a row in a bank
- Activate a row in a bank
- Put the SDRAM devices in self-refresh and power-down modes

The protocol controller takes mode register writes from the MMR interface and translates them into mode register writes to SDRAM. Writing into the mode register is restricted through a mask register.

Efficiency Controller

The efficiency controller controls the ordering of transfers buffered in the read and write command buffers. It attempts to order transfers to optimize the available memory bandwidth. The DMC uses a number of schemes, described in the following sections, to increase the throughput.

Page-Based Scheduling

The DMC parses each write and read transaction that it buffered and gets the information of the row (page) and bank address. The protocol controller maintains the information about the pages that are opened in each bank. The efficiency controller uses the information about the opened pages while scheduling the buffered transactions. The transactions to the opened pages are given higher priority than the other outstanding transactions.

Same Master Transaction Scheduling

The DMC also stores the ID of each transaction that it buffered. In most of the cases, the transactions related to a master result in page hits from the locality of reference rule. The efficiency controller uses the ID information of the transactions while scheduling. When the page-based scheduling of the buffered transactions is complete, same master transaction scheduling is triggered. If multiple transactions from a master are received, the efficiency controller schedules the transactions back-to-back.

DMC Read Data Buffer

The DMC read data buffer contains a data buffer and an address buffer. The depth of the data buffer is equal to the burst length that is programmed in SDRAM. The address buffer holds the corresponding SDRAM burst address. When an SDRAM write address from any master matches an address in the DMC read data buffer, the DMC invalidates the related data in the read buffer. When the [DMC_RDDATABUFMSK1](#) or [DMC_RDDATABUFMSK2](#) register is programmed with a value other than zero, the DMC read data buffer operation is enabled. The set of masters whose data is buffered and retrieved are programmed in the [DMC_RDDATABUFID1](#) or [DMC_RDDATABUFID2](#) registers. The DMC can use the [DMC_RDDATABUFMSK1](#) and [DMC_RDDATABUFMSK2](#) ID registers to select a set of masters similar to the programming of the [DMC_PRIOMSK](#) and [DMC_PRIOMSK2](#) registers.

See the [SCB ID-Based Priority](#) section for details.

Closed Page Per Bank

The [DMC_EFFCTL](#) register provides per-bank granularity for closing pages. The software can determine that most accesses to a given bank in memory always result in a missed page. In this case, set the [PREC_BANK](#) bit corresponding to the required bank to close the row after every transfer. This proactive step can result in reduced thrashing and increases memory throughput.

SCB ID-Based Priority

The primary goal of the dynamic memory controller is to improve sustainable memory system bandwidth so that the service time for the average request can be reduced. However, to service critical requests from any master in the system, the DMC provides a mechanism to elevate priority of a given access. The DMC priority ID registers ([DMC_PRIO](#) and [DMC_PRIO2](#)) can be programmed with up to two SCB IDs with elevated priority.

After every access in a snapshot, the command buffers are searched to determine whether an ID of a command matches with the ID programmed in the [DMC_PRIO](#) and [DMC_PRIO2](#) registers. The priority SCB ID access is sent before the subsequent access in the snapshot if:

- A match occurs, and
- The direction of the access (for example write) is the same as the direction of the snapshot (write)

There is an alternative to providing priority to a specific SCB ID. If a number of IDs from the same master require priority, program the DMC priority mask ID registers ([DMC_PRIOMSK](#) and [DMC_PRIOMSK2](#)) so that the corresponding bits are 0. The DMC uses a combination of the [DMC_PRIO](#) and [DMC_PRIO2](#) registers and the [DMC_PRIOMSK/DMC_PRIOMSK2](#) registers to elevate the priority of a select few or all IDs that belong to a master. By default, none of the IDs are prioritized. The following are a few possibilities:

- The `DMC_PRIOMSK` field is set to 0x00000000. If a single ID (7234) needs priority, set the `DMC_PRIOMSK` field to 0xFFFFFFFF and set the `DMC_PRIO` field to 7234.
- If the `DMC_PRIOMSK` field is set to 0xFFFFFFFFE, the SCB IDs 7234 and 7235 are given priority.
- If the `DMC_PRIOMSK` field is set to 0xFFFFFFFEC, the SCB IDs 7234, 7235, 7236, and 7237 are given priority.
- If two transactions with priority, one read and the other a write, are outstanding, the priority transaction that does not change the direction of the DMC access gets priority. The other priority transaction is handled at the beginning of the next snapshot. For example, if a write snapshot is in-progress, the write priority transaction is sent. The read priority transaction is sent at the beginning of the next read snapshot.

NOTE: Use SCB ID-based priority judiciously because it can significantly reduce the throughput of the DMC.

Delaying up to Eight Auto-Refresh Commands

The DMC uses this method to ensure that auto-refresh does not interfere with any critical data transfers. Up to eight auto-refresh commands can accumulate in the DMC. The exact number of auto-refresh commands can be programmed using the `DMC_EFFCTL.NUMREF` bit.

After the first refresh command is accumulated, the DMC constantly looks for an opportunity to schedule a refresh command. When the SCB read and write command buffers become empty for the programmed number of clock cycles (`DMC_EFFCTL.IDLECYC` bit field), the accumulated number of refresh commands are sent back-to-back to the DRAM. (The empty state of the SCB command buffers implies that no access is outstanding.)

After every refresh, the SCB command buffers are checked to ensure that they stay empty. If the SCB command buffers are always full, once the programmed number of refresh commands accumulates, the refresh operation is elevated to urgent priority. One refresh command is sent immediately. After this process, the DMC continues to wait for an opportunity to send out refresh commands. If self-refresh mode is enabled, all pending refresh commands are given out only after that DMC enters into self-refresh mode.

Page and Bank Interleaving

Page and bank interleaving allow consecutive row accesses to fall into the same bank (bank interleaving) or into a different bank (page interleaving). The DMC uses bank interleaving by default (`DMC_CTL.ADDRMODE` bit =0). If the `DMC_CTL.ADDRMODE` bit =1, the DMC uses page interleaving. Page misses in one addressing mode result in hits in the other addressing mode.

System Crossbar Slave Interface

The DMC uses the system crossbar slave interface to move all data. The system crossbar interface accepts interleaved write transactions and sends out-of-order responses. The read and write interfaces consist of buffers for address, data, and control information transferred to or from the system crossbar bus.

The system crossbar interface transactions are sent to the SDRAM only after the SDRAM has been initialized. However, if transactions arrive before or during initialization, they accumulate in the system crossbar interface and are sent out to the protocol controller once the initialization completes.

To increase throughput, the system crossbar write-response is sent out as soon as the final DDR burst is scheduled for transfer into the SDRAM. However, if an auto-refresh is needed, the scheduled write data is sent only after the auto-refresh. A delay can occur. The delay is a maximum of 64 clock cycles from the moment the write response is sent on the SCB to the write operation of the data into SDRAM.

The system crossbar interface performs the following operations:

- Buffers read and write command requests from the system crossbar bus
- Processes the requests by converting them to protocol controller user-interface transfers
- Sends and receives data to or from the protocol controller
- Creates a suitable read/write response and sends read data back to the system crossbar bus

The system crossbar slave interface supports the following:

- All burst lengths (1–16)
- Incremental and wrap bursts
- Data transfer sizes of 8-bit, 16-bit, or 32-bit
- Arrival of write data before write address
- Generation of error responses which include:
 - Any access to an unimplemented region of the external memory space
 - Any access when the SDRAM is in self-refresh mode/power-down mode
 - Any access when the direct command interface is in operation

Read/Write Command and Data Buffers

The system crossbar interface consists of a four-deep read command buffer and a four-deep write command buffer. Up to four write commands and four read commands can be waiting for access to the SDRAM. The system crossbar write buffer is 32 deep. It can support write data interleaving of two. The system crossbar read buffer is 32 deep.

Peripheral Bus Slave Interface

The peripheral bus slave interface connects the dynamic memory controller to the peripheral bus and provides a host controller with access to the registers. The peripheral bus slave interface supports the following features:

- Read and write word accesses
- 32-bit data bus

Architectural Concepts

The following sections provide information on the architecture of the interface.

Controller On Die Termination (ODT)

The controller ODT is enabled with the granularity of a byte lane. The description of this feature can be obtained in the description of the corresponding PHY registers. Controller ODT involves extra overhead in terms of power consumption during reads.

The DMC implements dynamic on die termination at processor pads. When controller ODT is enabled, the termination resistors in the pads are turned on when the controller reads data from the DRAM. These resistors are turned off when the controller writes to the DRAM.

Mode Register Set and Extended Mode Register Set Command

The load mode register command initializes the SDRAM operation parameters. The DMC supports the mode register set and extended mode register set commands. The controller automatically issues the mode register set command during power-on initialization and also when the `DMC_MR` register is written with the `DMC_MSK.MR` bit. The mode register set command is sent after the ongoing data transfer completes.

DDR3 Reset Functionality

DDR3 contains an additional pin corresponding to reset functionality. Reset is part of the initialization sequence but it can be performed asynchronously when needed. The reset procedure is similar to the steps involved in the initialization except the initial part of power-up.

To perform reset on the DDR3 module:

1. Check to ensure the module is in the idle state by polling the `DMC_STAT.IDLE` bit (0x0008).
2. Set the `DMC_CTL.RESET` bit (0x0004).
3. Monitor the `DMC_STAT.RESETDONE` bit for the completion of the reset function.

Do not perform any transactions during a module reset. Wait for the `DMC_STAT.RESETDONE` signal.

DDR3 SDRAM Organization

The DMC supports DDR3 SDRAM memory modules ranging from 512 Mb to 8 Gb. The following tables list the address translation mechanism from the user interface to DDR3 memory interface. The controller also supports two types of addressing modes: bank interleaving (`DMC_CTL.ADDRMODE = 1`) and page interleaving (`DMC_CTL.ADDRMODE = 0`).

Bank Interleaving

The *DDR3 Bank Interleaving* table shows DDR3 bank interleaving.

Table 8-3: DDR3 Bank Interleaving

SDRAM size	Bank address bits	Row address bits	Column address bits
512 Mb	25:24	23:11	10:1
1 Gb	26:24	23:11	10:1
2 Gb	27:25	24:11	10:1

Table 8-3: DDR3 Bank Interleaving (Continued)

SDRAM size	Bank address bits	Row address bits	Column address bits
8 Gb	29:27	26:11	10:1

Page Interleaving

The *DDR3 Page Interleaving* table shows DDR3 page interleaving.

Table 8-4: DDR3 Page Interleaving

SDRAM size	Row address bits	Bank address bits	Column address bits
512 Mb	25:13	12:11	10:1
1 Gb	26:14	13:11	10:1
2 Gb	27:14	13:11	10:1
4 Gb	28:14	13:11	10:1
8 Gb	29:14	13:11	10:1

DMC Clocking

The DMC uses a divided-down version of the *PLLCLK* (PLL clock) to generate an internal clock for clocking the DMC block and interface. The specific value of the *DCLK* frequency is programmed in the *CGU_DIV* register.

For information on the maximum clock frequency supported for specific modes, refer to the processor data sheet.

NOTE: For details on DMC clocking, see the [Clock Generation Unit \(CGU\)](#) and [Clock Generation Unit \(CGU\)](#) chapters.

DMC DMA

The DMC supports DMA-based transfers to and from external DDR SDRAM memory and internal memory.

The DMC DMA controller, part of the distributed DMA engines (DDE) that are dispersed through the infrastructure, connects to the system crossbar fabric.

The DMC uses two DDEs for memory-to-memory DMA (MDMA). One channel is the source channel, and the second, the destination channel.

DMA transfers on the processor are descriptor-based or register-based. Register-based DMA allows the processor to program DMA control registers directly to initiate a DMA transfer. On completion, the control registers can be automatically updated with their original setup values for continuous transfer, if needed. Descriptor-based DMA transfers require a set of parameters stored within memory to initiate a DMA sequence. This transfer allows the chaining together of multiple DMA sequences. In descriptor-based DMA operations, a DMA channel can be programmed to set up and start another DMA transfer automatically after the current sequence completes.

Enhanced DMA operations (such as delay line DMA, scatter or gather DMA) are also supported to or from the DMC module.

DMC Operating Modes

DDR3 Mode

The DMC module supports JESD79-3E compatible double data rate DDR3 SDRAM. To configure this mode of operation, first set (=1) the `DMC_CTL.DDR3EN` bit

Self-Refresh Mode

For low-power consumption, the SDRAM can be put in self-refresh mode. When no data activity occurs, the DMC can put the SDRAM in self refresh to save power. The `DMC_STAT.IDLE` bit indicates the activity on the DMC. If this bit is set, there is no activity in the DMC.

Enable self-refresh mode by writing the `DMC_CTL.SRREQ` bit. The DMC stays in a self-refresh state as long as this bit is asserted. The `DMC_STAT.SRACK` bit indicates when the SDRAM enters self-refresh mode.

When the DMC is in self-refresh mode, the DMC generates an SCB error when any data accesses (read or write requests) is requested.

The DMC can be brought out of self-refresh mode by clearing the `DMC_CTL.SRREQ` bit again. The controller clears the `DMC_STAT.SRACK` bit after the self-refresh operation completes.

DMC Event Control

The DMC has no related interrupt or trigger event information.

DMC Programming Model

The dynamic memory controller contains five groups of memory-mapped registers. The DMC uses the MMR access bus to connect to these registers.

- Control and status registers. These registers control the various operation modes of the dynamic memory controller and provide status.
- Timing parameter registers. The value programmed in these registers depends on the speed grade of the SDRAM device used.
- Mode register mirror registers. These shadow registers are copies of the mode registers residing in the SDRAM device.
- PHY control and status registers. The DMC uses these registers to control the operation of the PHY.
- PAD control registers. The DMC uses these registers to control the various aspects of the I/O pads.

The DMC control registers contain sensitive timing parameters and settings for the DDR SDRAM. These registers are programmed with values that are in the operating range of the DDR used.

Writing to reserved fields or writing any reserved values in register bits can cause the dynamic memory controller to function erroneously.

PHY DLL Calibration

The PHY DLL calibration is performed as part of the SDRAM power-up initialization. It calibrates data against the DQS and CLK signal. However, running DLL calibration after self-refresh or at an arbitrary time is required in certain cases.

The DMC allows PHY DLL calibration to start by setting the `DMC_CTL.DLLCAL` bit. The `DMC_STAT.DLLCALDONE` bit can be used to monitor the progress of the calibration. Once calibration is over, this bit is set. Once the calibration procedure is started by writing to the `DMC_CAL_PADCTL0.CALSTRT` bit, the full calibration takes 300 DCLK cycles to complete.

NOTE: DLL calibration can be initiated only when the DMC is idle (`DMC_STAT.IDLE=1`).

DDR3 ZQ Calibration Short CMD

The ZQ calibration short command is generally used to correct small variations in ZQ (~0.5%). To perform ZQ calibration, the controller is first checked for its idle state. Once the idle bit is obtained, a ZQCS command can be issued by setting the `DMC_STAT.ZQCSDONE` bit (0x0004). The `DMC_STAT.ZQCSDONE` bit (0x0008) can be used to monitor the calibration sequence. When this bit is 0, it indicates that the calibration is ongoing. When it is 1, it indicates that the calibration is done. As an example, a GP timer can be used to periodically trigger a ZQCS command to address tiny variations.

NOTE: The ZQ calibration function is essential for normal operation of DDR3. With the reference of the external resistance ($240\ \Omega \pm 1\%$) connected to the `DMC_RZQ` pin, DDR3 calibrates the R_{on} and R_{tt} values of the ZQ pin against temperature and voltage variations.

DDR3 ZQ Calibration Long CMD

Several DDR3 impedance calibrations are implemented for optimal signal integrity. The long ZQ calibration is used after power-up and the short ZQ calibration is used periodically during normal operation to compensate for voltage and temperature drift. These calibration sequences improve connectivity between the SDRAM pads and the PCB trace. The `DMC_RZQ` pin on the SDRAM is connected to an external precision resistor that adjusts the output driver impedance R_{tt} and ODT values to match the trace impedance. The connection reduces impedance discontinuity and minimizes signal reflections.

The command has two variants named as ZQ calibration long (ZQCL) and ZQ calibration short (ZQCS). The ZQCL command is issued during initialization and after self-refresh exit command. It can be issued later depending on the system environment.

The DMC pads can be autocalibrated to the required driver impedance R_{tt} using an external resistance RZQ and the On Die Termination (ODT) value using the corresponding bits (`DMC_CAL_PADCTL2`). The autocalibration logic translates these values into a corresponding drive strength control inside the PHY and then routed to the PADS. Autocalibration starts as soon as the bit is programmed (set the DCLK at the required frequency before

setting this `DMC_CAL_PADCTL0.CALSTRT` bit). Autocalibration expects the program to select two different member sets of pads (address/command pads versus CLK/Data/DQS/DM pads).

On Die Termination (DDR3)

The DMC supports dynamic On Die Termination (ODT) at the pads. When the controller ODT is set, the termination resistors in the pads are turned on when the controller reads data from the DRAM. These resistors are turned off when the controller is writing to the DRAM. Controller ODT is enabled with the granularity of a byte lane. The description of this feature can be obtained in the description of the corresponding PHY registers.

DDR3 SDRAM provides extended ODT mode.

- *Asynchronous ODT*: ODT timing in the slow exit power-down mode
- *Dynamic ODT*: Function that can dynamically switch the ODT resistance during a write operation without an MRS command. It improves signal quality during a write operation.

Leveling Techniques

The DMC controller supports read and write leveling.

Write Leveling

It is difficult to meet the specifications with the on-board parasitic, trace length variability along with high frequency. The DMC controller supports leveling to compensate the skews between clock and strobe. This skew compensation is done independently for every byte lane. Write leveling is performed by the DDR3 memory controller as a part of initialization by programming the `DMC_MR1.WL` bit.

To dynamically perform write leveling:

1. Ensure that the DMC is idle.
2. Program the `DMC_MR1.WL` bit.
3. Enable the `DMC_MSK` register for the `DMC_MR1` register.
4. Wait for 2000 DMC clock cycles to complete write leveling.

Read Leveling

Read leveling is useful for calibrating read data timing. It is compensated for imbalanced loading on read path. Read leveling can be performed on LSB or byte lane. It is performed as a part of initialization sequence by setting the `DMC_EMR3.MPR` bit.

To dynamically perform read leveling:

1. Ensure that the DMC is idle.
2. Program the `DMC_EMR3.MPR` bit.

3. Enable DMC mask register `DMC_EMR3` register.
4. Wait for 2000 DMC clock cycles to complete read leveling.

Read Leveling during DQS Strobe Gating

To ensure robust read timing, read leveling must be done individually for all data bits. Some DDR3 memory devices do not drive read leveling patterns on all data pins. The devices send pattern only information on the LSBs of each byte lane. In these cases, the `DMC_EMR3` register cannot be used for read leveling. Instead, use the `DMC_CTL.RL_DQS` bit for read leveling.

Read leveling during DQS strobe gating is dynamically supported except during initialization.

Initializing the DMC

Complete the following procedures to initialize the DMC module.

- [Resetting the DMC Lane](#)
- [Performing ZQ Calibration](#)
- [Programming the DMC Controller](#)

Resetting the DMC Lane

If there is a change in the DMC clock frequency, the DDR lane must be reset using the following procedure. Once the lane is reset, wait 9000 DDR clock cycles for the DDR DLL to lock before restarting the DDR.

1. Set the `DMC_DDR_LANE0_CTL0.CB_RSTDLL` and `DMC_DDR_LANE1_CTL0.CB_RSTDLL` bits.
2. Change the DMC clock frequency.
3. Clear the `DMC_DDR_LANE0_CTL0.CB_RSTDLL` and `DMC_DDR_LANE1_CTL0.CB_RSTDLL` bits.

Performing ZQ Calibration

Perform the following ZQ calibration procedure for the proper impedance matching.

1. Program the `DMC_DDR_ZQ_CTL0.IMPWRADD` bit field with the drive strength of the address and command signals. A 100 Ω drive strength is recommended for the drive strength of address and command signals. When programming a drive strength of 100 Ω , write 0x64 into this bit field.
2. Program the `DMC_DDR_ZQ_CTL0.IMPWRDQ` bit field with the drive strength of the DQ, DQS, DM and clock signals. A 100 Ω drive strength is recommended for the drive strength of address and command signals. When programming a drive strength of 100 Ω , write 0x64 into this bit field.
3. Program the `DMC_DDR_ZQ_CTL0.IMPRTT` bit field with the adjusted On Die Termination (ODT) for the Data and DQS signals for the read operation. Due to a correction factor, program this field to 80% of the equivalent ODT.

$DMC_DDR_ZQ_CTL0.IMPRTT \text{ value} = ODT * 2 * 0.8$

For example, if a 50Ω terminating resistance is required on the data pads to match the trace impedance to the board impedance, there will be two 50Ω resistance data pads in parallel. The value is programmed to $100 \times 0.8 = 80$.

4. Write 0 to the [DMC_DDR_ZQ_CTL1](#) register.
5. Program 0x70000000 to the [DMC_DDR_ZQ_CTL2](#) register.
6. Program 0x00000000 to the [DMC_DDR_CA_CTL](#) register.
7. Program 0x00000000 to the [DMC_DDR_ROOT_CTL](#) register.
8. Program 0x00010000 to the [DMC_DDR_ROOT_CTL](#) register.
9. Wait for 8000 DMC clock cycles.
10. Program 0x0C000001 to the [DMC_DDR_CA_CTL](#) register.
11. Wait for 8000 DMC clock cycles.
12. Program 0x00000000 to the [DMC_DDR_CA_CTL](#) register.
13. Program 0x00000000 to the [DMC_DDR_ROOT_CTL](#) register.

Programming the DMC Controller

Perform the following procedure to program the DMC controller.

1. Program the [DMC_CFG](#) register with the size of the DDR memory device. Program the [DMC_CFG.IFWID](#) and [DMC_CFG.SDRWID](#) bit fields to 16-bits wide.
2. Program the [DMC_TR0](#), [DMC_TR1](#), [DMC_TR2](#) registers with the timing parameters as required by the DDR memory device.
3. Program the [DMC_MR](#) and [DMC_MR1](#) registers with the appropriate values. A memory ODT of 120Ω is recommended.
4. Program the [DMC_MR2](#) register. If read leveling is required, program the [DMC_EMR3](#) register.
5. Program the [DMC_DLLCTL](#) register. Program the [DMC_DLLCTL.DATA CYC](#) bit field with 15ns/ddr clock period in ns. Program the [DMC_DLLCTL.DLLCALRDCNT](#) bit field with 240.
6. Wait for 2000 DMC clock cycles.
7. Set the [DMC_DDR_CA_CTL.SW_REFRESH](#) bit, keeping the other bits unchanged.
8. Wait for five DMC clock cycles.
9. Set the [DMC_DDR_ROOT_CTL.SW_REFRESH](#) bit. Program the [DMC_DDR_ROOT_CTL.PIPE_OFSTDCYCLE](#) bit field with 2.
10. Program the [DMC_CTL](#) register.

- `DMC_CTL.RDTOWR = 5`
 - Set the `DMC_CTL.DDR3EN` bit.
 - Set the `DMC_CTL.INIT` bit.
11. If write leveling is enabled, wait for 600 DMC clock cycles or until the `DMC_MR1.WL` bit auto clears.
 12. If read leveling is enabled wait for 2000 DMC clock cycles or until the `DMC_EMR3.MPR` bit auto clears.
 13. If read leveling during data strobe gating is enabled, wait for 600 DMC clock cycles or until the `DMC_CTL.RL_DQS` bit auto clears.
 14. Wait for 722000 DMC clock cycles to complete DMC initialization or until the `DMC_STAT.INITDONE` bit is set.
 15. Set the `DMC_DDR_LANE0_CTL1.COMP_DCYCLE` and `DMC_DDR_LANE1_CTL1.COMP_DCYCLE` bits, without affecting other bits.
 16. Wait for 10 DMC clock cycles.
 17. Clear the `DMC_DDR_LANE0_CTL1.COMP_DCYCLE` and `DMC_DDR_LANE1_CTL1.COMP_DCYCLE` bits, without affecting other bits.
 18. Set and then clear the `DMC_DDR_LANE0_CTL0.CB_RSTDAT` bit, without affecting other bits.
 19. Set and then clear `DMC_DDR_LANE1_CTL0.CB_RSTDAT` bit, without affecting other bits.
 20. Wait for 2500 DMC clock cycles.
 21. Read the `DMC_STAT.PHYRDPHASE` bit field. Program this value into the `DMC_DLLCTL.DATACYC` bit field, without affecting other bits in the [DMC_DLLCTL](#) register.
 22. Reprogram bits: `DMC_CTL.DDR3EN` and `DMC_CTL.RDTOWR` (if operation frequency is above 666MHz).

ADSP-2159x DMC Register Descriptions

Dynamic Memory Controller (DMC) contains the following registers.

Table 8-5: ADSP-2159x DMC Register List

Name	Description
DMC_CFG	Configuration Register
DMC_CTL	Control Register
DMC_DLLCTL	DLL Control Register
DMC_DT_CALIB_ADDR	Data Calibration Address Register
DMC_DT_DATA_CALIB_DATA0	Data Calibration Data 0 Register
DMC_DT_DATA_CALIB_DATA1	Data Calibration Data 1 Register

Table 8-5: ADSP-2159x DMC Register List (Continued)

Name	Description
DMC_EFFCTL	Efficiency Control Register
DMC_EM3	Shadow EM3 Register
DMC_MR	Shadow MR0 Register (DDR3)
DMC_MR1	Shadow MR1 Register (DDR3)
DMC_MR2	Shadow MR2 Register (DDR3)
DMC_MSK	Mask (Mode Register Shadow) Register
DMC_PRIO	Priority ID Register 1
DMC_PRIO2	Priority ID Register 2
DMC_PRIOMSK	Priority ID Mask Register 1
DMC_PRIOMSK2	Priority ID Mask Register 2
DMC_RDDATABUFID1	DMC Read Data Buffer ID Register 1
DMC_RDDATABUFID2	DMC Read Data Buffer ID Register 2
DMC_RDDATABUFMSK1	DMC Read Data Buffer Mask Register 1
DMC_RDDATABUFMSK2	DMC Read Data Buffer Mask Register 2
DMC_STAT	Status Register
DMC_TR0	Timing 0 Register
DMC_TR1	Timing 1 Register
DMC_TR2	Timing 2 Register

Configuration Register

The `DMC_CFG` register selects SDRAM device specific parameters and selects the SDRAM interface width.

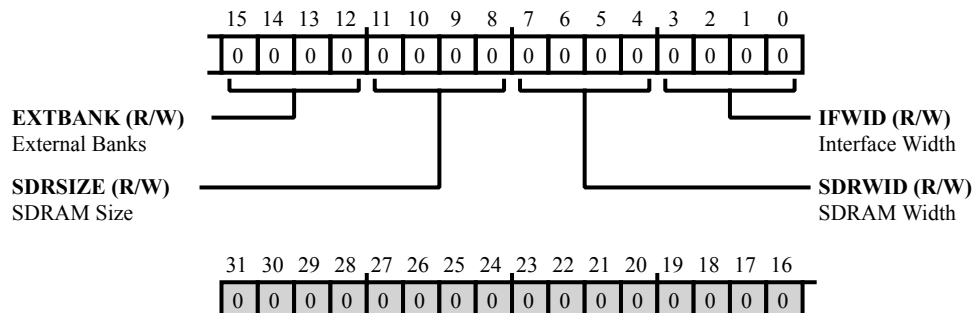


Figure 8-1: DMC_CFG Register Diagram

Table 8-6: DMC_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:12 (R/W)	EXTBANK	External Banks. The <code>DMC_CFG.EXTBANK</code> bits select the number of external banks connected to the DMC. Note that all values other than those shown are reserved.
		0 1 External Bank
		1-15 Reserved
11:8 (R/W)	SDRSIZE	SDRAM Size. The <code>DMC_CFG.SDRSIZE</code> bits select the size of individual SDRAM connected to the DMC. Note that all values other than those shown are reserved.
		3 512M Bit SDRAM
		4 1G Bit SDRAM
		5 2G Bit SDRAM
		6 4G Bit SDRAM
		7 8G Bit SDRAM
7:4 (R/W)	SDRWID	SDRAM Width. The <code>DMC_CFG.SDRWID</code> bits select the width of the individual SDRAM connected to the DMC. Note that all values other than those shown are reserved.
		0-1 Reserved
		2 16-Bit Wide SDRAM
		3-15 Reserved

Table 8-6: DMC_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
3:0 (R/W)	IFWID	Interface Width. The DMC_CFG.IFWID bits select the width of the interface between the DMC and SDRAM. Note that all values other than those shown are reserved.	
		0-1	Reserved
		2	16-Bit Wide Interface. All other values are reserved. This field specifies the interface width between the controller and the SDRAM.
		3-15	Reserved

Control Register

The `DMC_CTL` register controls DMC modes, DLL calibration, and DRAM initialization.

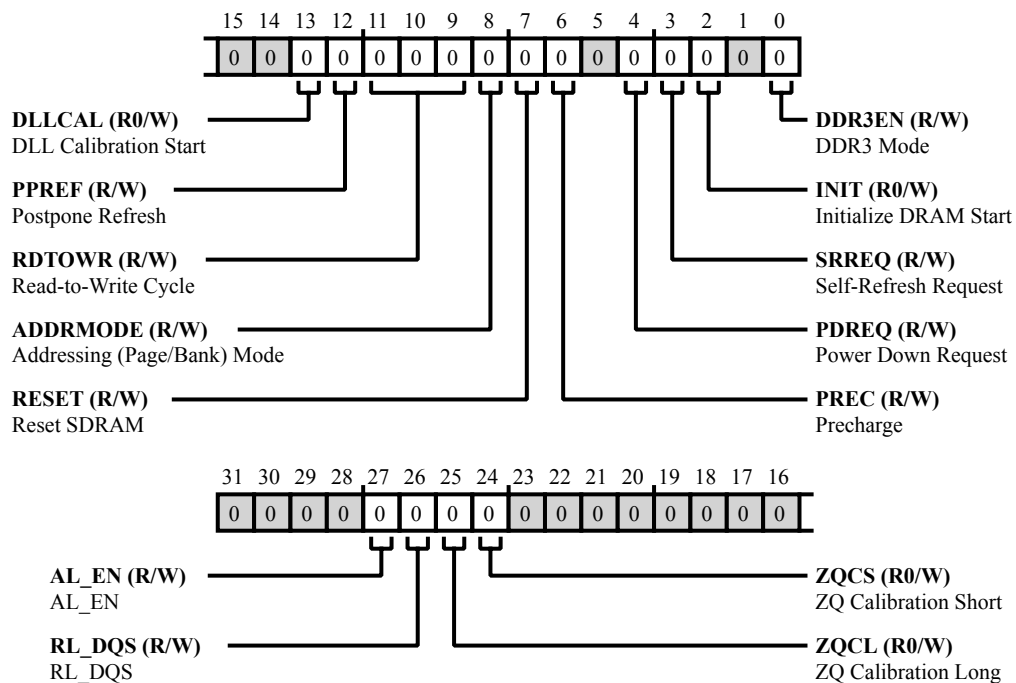


Figure 8-2: DMC_CTL Register Diagram

Table 8-7: DMC_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27 (R/W)	AL_EN	AL_EN. When set, enables 800MHz operation
		0 Disables greater than 667MHz operation
		1 Enables greater than 667MHz operation
26 (R/W)	RL_DQS	RL_DQS. When set, enables Read leveling during DQS Gating Training. This bit auto-clears on read leveling completion
		0 Enables Read leveling during DQS Gating Training
		1 Disables Read leveling during DQS Gating training

Table 8-7: DMC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R0/W)	ZQCL	ZQ Calibration Long. The DMC_CTL . ZQCL bit starts the ZQ calibration long sequence. Note that this bit always reads as 0.
		0 No effect
		1 Triggers ZQ calibration long sequence
24 (R0/W)	ZQCS	ZQ Calibration Short. The DMC_CTL . ZQCS bit starts the ZQ calibration short sequence. Note that this bit always reads as 0.
		0 No effect
		1 Triggers ZQ calibration short sequence
13 (R0/W)	DLLCAL	DLL Calibration Start. The DMC_CTL . DLLCAL bit starts the PHY DLL calibration sequence. Note that this bit always reads as 0.
		0 No effect
		1 Start PHY DLL calibration
12 (R/W)	PPREF	Postpone Refresh. The DMC_CTL . PPREF bit enables postponing the DMCs sending of auto-refresh commands. When enabled, the DMC accumulates refresh commands. The DMC_EFFECTL . NUMREF field selects the number of refresh commands that the DMC may accumulate. When disabled, the DMC_TR1 . TREF field selects the interval for auto-refresh command distribution. A maximum of eight auto-refresh commands can be accumulated in DDR3 mode.
		0 Disable Postpone Refresh
		1 Enable Postpone Refresh
11:9 (R/W)	RDTOWR	Read-to-Write Cycle. The DMC_CTL . RDTOWR bits select the number of cycles that the DMC adds when a write operation follows a read operation. For proper operation, it should be programmed with the value of 010.
		0 1 Cycle Added from JEDEC Spec Value
		1 2 Cycles Added from JEDEC Spec Value
		2 3 Cycles Added from JEDEC Spec Value
		3 4 Cycles Added from JEDEC Spec Value
		4 5 Cycles Added from JEDEC Spec Value
		5 6 Cycles Added from JEDEC Spec Value

Table 8-7: DMC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		6	7 Cycles Added from JEDEC Spec Value
		7	8 Cycles Added from JEDEC Spec Value
8 (R/W)	ADDRMODE	Addressing (Page/Bank) Mode. The DMC_CTL.ADDRMODE bit selects whether the DMC uses page or bank interleaving for addressing. When using page interleaving, the bank address bits follow the most significant column address bits. When using bank interleaving, the bank address bits follow the most significant row address bits.	
		0	Bank Interleaving
		1	Page Interleaving
7 (R/W)	RESET	Reset SDRAM. The DMC_CTL.RESET bit starts the reset sequence. Note that this bit always reads as 0.	
		0	No effect
		1	Starts reset sequence
6 (R/W)	PREC	Precharge. The DMC_CTL.PREC bit enables precharge, which closes DRAM rows immediately after access. When disabled, all accesses result in the respective DRAM rows remaining open, until the DMC needs to close them.	
		0	No Effect
		1	Enable Precharge
4 (R/W)	PDREQ	Power Down Request. The DMC_CTL.PDREQ bit enables power-down mode. When the DMC is in power-down mode, any data accesses cause the DMC to generate a bus error. The DRAM remains in power-down mode as long as this bit is 1.	
		0	Disable Power-Down
		1	Enable Power-Down
3 (R/W)	SRREQ	Self-Refresh Request. The DMC_CTL.SRREQ bit enables self-refresh mode. When the DMC is in self-refresh mode, any data accesses cause the DMC to generate a bus error. The DRAM remains in self-refresh mode as long as this bit is 1.	
		0	Disable Self-Refresh
		1	Enable Self-Refresh

Table 8-7: DMC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R0/W)	INIT	Initialize DRAM Start. The DMC_CTL . INIT bit starts the power up DRAM initialization sequence and DLL calibration sequence. Note that this bit always reads as 0.
		0 No Effect
		1 Start DRAM Initialization
0 (R/W)	DDR3EN	DDR3 Mode. The DMC_CTL . DDR3EN bit selects whether the DMC operates in DDR3 mode.
		0 Reserved
		1 Enable DDR3 mode

DLL Control Register

The `DMC_DLLCTL` register holds the programmable parameters associated with the DLLs within the DMC PHY.

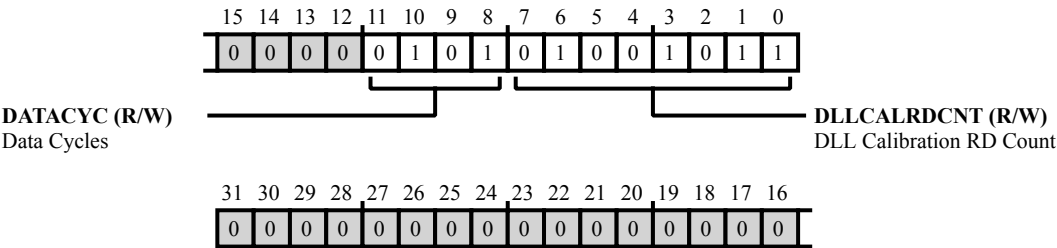


Figure 8-3: DMC_DLLCTL Register Diagram

Table 8-8: DMC_DLLCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11:8 (R/W)	DATA CYC	Data Cycles. The <code>DMC_DLLCTL.DATA CYC</code> bits select the latency after which the DMC reads data from the PHY. This field must be written with the value (9). All other values are reserved. Taking round trip delay into account, the DLL indicates whether a latency of 2 cycles is supported by means of status bits.
7:0 (R/W)	DLL CAL RD CNT	DLL Calibration RD Count. The <code>DMC_DLLCTL.DLL CAL RD CNT</code> field selects the number of read operations that the PHY uses for DLL calibration.

Data Calibration Address Register

The `DMC_DT_CALIB_ADDR` register provides the address used for the data calibration for read and write. During the DMC PHY DLL calibration, a particular set of locations in the DRAM is written and a series of reads are performed back to back to calibrate the PHY. The DMC PHY needs prior information about the data that would be read during the PHY DLL calibration. The controller performs one burst write operation to the address programmed in `DMC_DT_CALIB_ADDR` (0x0090).

Note: While the exact address chosen does not matter much during memory initialization, if calibration of the PHY is performed when the DRAM contains valid data, care needs to be taken to ensure that this address points to an unused address. Else, this operation will modify application data stored at the address selected.

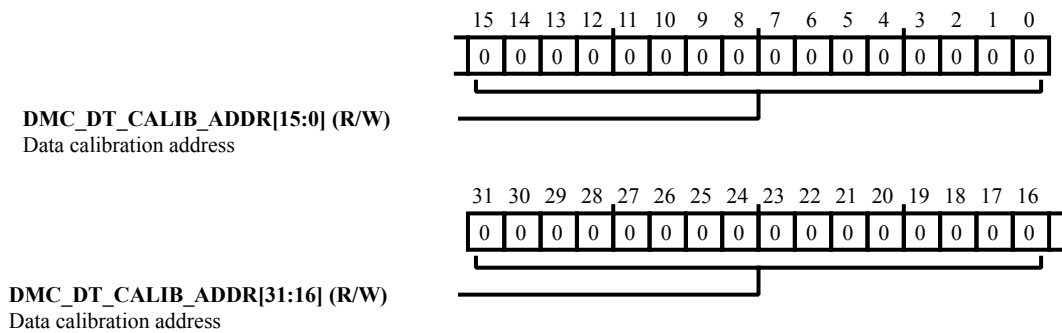


Figure 8-4: DMC_DT_CALIB_ADDR Register Diagram

Table 8-9: DMC_DT_CALIB_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DMC_DT_CALIB_ADDR	Data calibration address. The <code>DMC_DT_CALIB_ADDR.DMC_DT_CALIB_ADDR</code> bit field contains the address to be programmed for the data calibration for read and write.

Data Calibration Data 0 Register

The `DMC_DT_DATA_CALIB_DATA0` register contains the first 32-bit data used for the write during the data calibration.

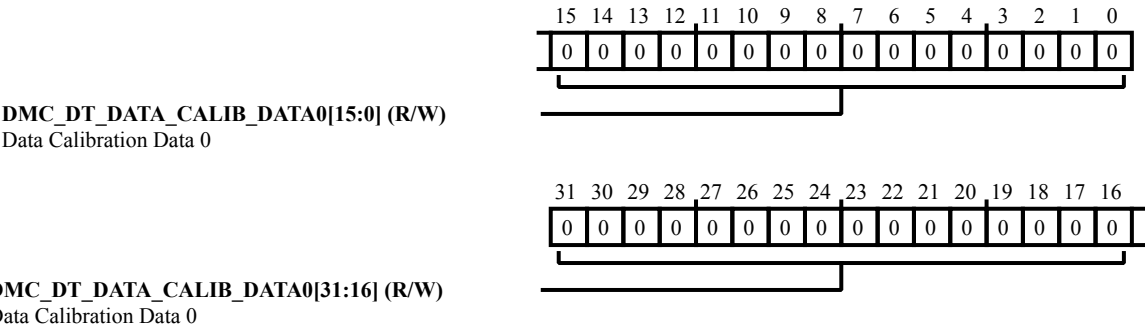


Figure 8-5: DMC_DT_DATA_CALIB_DATA0 Register Diagram

Table 8-10: DMC_DT_DATA_CALIB_DATA0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DMC_DT_DATA_CAL- IB_DATA0	Data Calibration Data 0. The DMC_DT_DATA_CALIB_DATA0.DMC_DT_DATA_CALIB_DATA0 bit field contains the first 32 bit data used for the write during the data calibration.

Data Calibration Data 1 Register

The `DMC_DT_DATA_CALIB_DATA1` register contains the second 32-bit data used for the write during the data calibration.

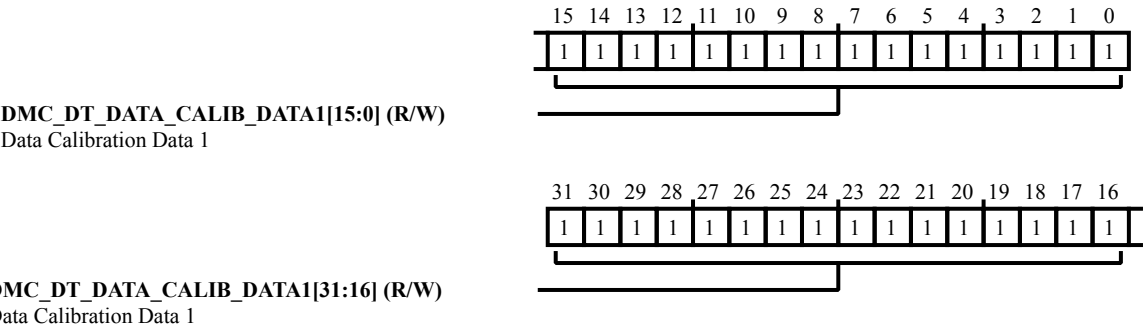


Figure 8-6: `DMC_DT_DATA_CALIB_DATA1` Register Diagram

Table 8-11: `DMC_DT_DATA_CALIB_DATA1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	<code>DMC_DT_DATA_CAL- IB_DATA1</code>	Data Calibration Data 1. The <code>DMC_DT_DATA_CALIB_DATA1.DMC_DT_DATA_CALIB_DATA1</code> bit field contains the second 32 bit data used for the write during the data calibration.

Efficiency Control Register

The `DMC_EFFCTL` register control DMC features that improve throughput efficiency. These include features such as auto-refresh management, precharge options, and write data options.

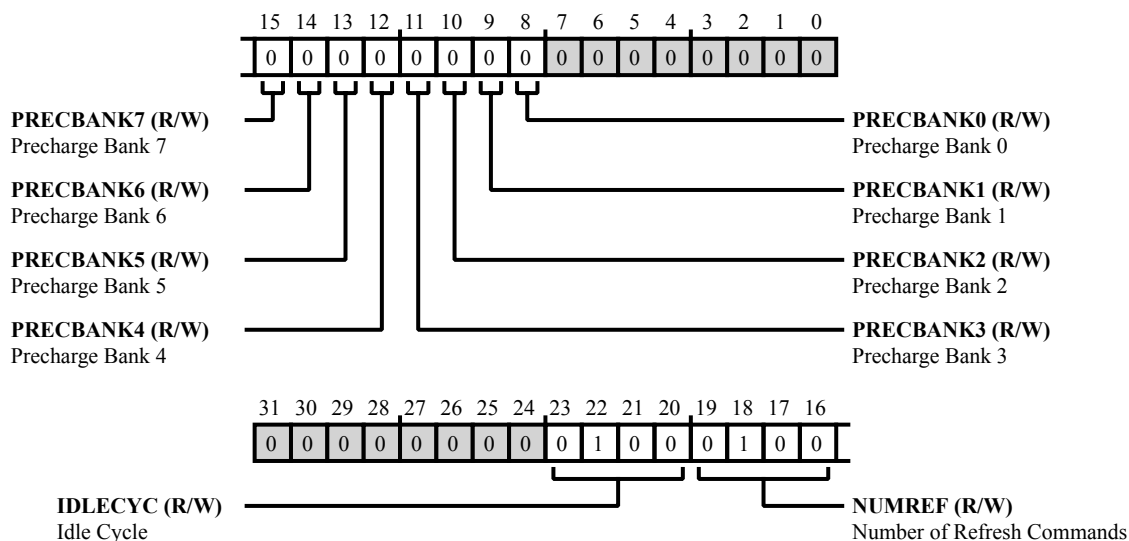


Figure 8-7: DMC_EFFCTL Register Diagram

Table 8-12: DMC_EFFCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:20 (R/W)	IDLECYC	Idle Cycle. The <code>DMC_EFFCTL.IDLECYC</code> bits select the number of cycles after which the DMC issues any accumulated auto-refresh commands if postpone refresh is enabled (<code>DMC_CTL.PPREF = 1</code>). When <code>DMC_EFFCTL.IDLECYC</code> is set to 0, the DMC ignores the <code>DMC_CTL.PPREF</code> selection and does not accumulate/postpone periodic auto-refresh commands. Note 1: By default, accumulated auto-refresh commands are issued after counting four idle cycles. Note 2: This value is ignored if <code>DMC_CTL.PPREF</code> is not set. Note 3: Setting this value to 0000 overrides the "postpone refresh" feature and does not accumulate/postpone periodic auto refreshes.
		0-15 0 to 15 Idle Cycles to Postpone Refresh Commands

Table 8-12: DMC_EFFCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19:16 (R/W)	NUMREF	Number of Refresh Commands. The DMC_EFFCTL.NUMREF bits select the number of auto-refresh commands that the DMC can accumulate if postpone refresh is enabled (DMC_CTL.PPREF =1). In DDR3 mode, the DMC may accumulate up to eight auto-refresh commands. Note 1: By default, accumulated auto-refresh commands are issued after counting four idle cycles. Note 2: This value is ignored if DMC_CTL.PPREF is not set.
		0 No Refresh Commands Accumulate
		1 1 Refresh Command May Accumulate
		2 2 Refresh Commands May Accumulate
		3 3 Refresh Commands May Accumulate
		4 4 Refresh Commands May Accumulate
		5 5 Refresh Commands May Accumulate
		6 6 Refresh Commands May Accumulate
		7 7 Refresh Commands May Accumulate
		8 8 Refresh Commands May Accumulate
15 (R/W)	PRECBANK7	Precharge Bank 7. The DMC_EFFCTL.PRECBANK7 bit enables precharge (closes the page) of bank 7 after each transfer if the DMC precharge feature is enabled (DMC_CTL.PREC =1). Note: The (DMC_CTL.PREC) takes precedence over value in this register. If (DMC_CTL.PREC =1) then all banks are precharged.
		0 Disable Precharge Bank 7
		1 Enable Precharge Bank 7
14 (R/W)	PRECBANK6	Precharge Bank 6. The DMC_EFFCTL.PRECBANK6 bit enables precharge (closes the page) of bank 6 after each transfer if the DMC precharge feature is enabled (DMC_CTL.PREC =1). Note: The (DMC_CTL.PREC) takes precedence over value in this register. If (DMC_CTL.PREC =1) then all banks are precharged.
		0 Disable Precharge Bank 6
		1 Enable Precharge Bank 6

Table 8-12: DMC_EFFCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PRECBANK5	Precharge Bank 5. The DMC_EFFCTL.PRECBANK5 bit enables precharge (closes the page) of bank 5 after each transfer if the DMC precharge feature is enabled (DMC_CTL.PREC =1). Note: The (DMC_CTL.PREC) takes precedence over value in this register. If (DMC_CTL.PREC =1) then all banks are precharged.
		0 Disable Precharge Bank 5
		1 Enable Precharge Bank 5
12 (R/W)	PRECBANK4	Precharge Bank 4. The DMC_EFFCTL.PRECBANK4 bit enables precharge (closes the page) of bank 4 after each transfer if the DMC precharge feature is enabled (DMC_CTL.PREC =1). Note: The (DMC_CTL.PREC) takes precedence over value in this register. If (DMC_CTL.PREC =1) then all banks are precharged.
		0 Disable Precharge Bank 4
		1 Enable Precharge Bank 4
11 (R/W)	PRECBANK3	Precharge Bank 3. The DMC_EFFCTL.PRECBANK3 bit enables precharge (closes the page) of bank 3 after each transfer if the DMC precharge feature is enabled (DMC_CTL.PREC =1). Note: The (DMC_CTL.PREC) takes precedence over value in this register. If (DMC_CTL.PREC =1) then all banks are precharged.
		0 Disable Precharge Bank 3
		1 Enable Precharge Bank 3
10 (R/W)	PRECBANK2	Precharge Bank 2. The DMC_EFFCTL.PRECBANK2 bit enables precharge (closes the page) of bank 2 after each transfer if the DMC precharge feature is enabled (DMC_CTL.PREC =1). Note: The (DMC_CTL.PREC) takes precedence over value in this register. If (DMC_CTL.PREC =1) then all banks are precharged.
		0 Disable Precharge Bank 2
		1 Enable Precharge Bank 2
9 (R/W)	PRECBANK1	Precharge Bank 1. The DMC_EFFCTL.PRECBANK1 bit enables precharge (closes the page) of bank 1 after each transfer if the DMC precharge feature is enabled (DMC_CTL.PREC =1). Note: The (DMC_CTL.PREC) takes precedence over value in this register. If (DMC_CTL.PREC =1) then all banks are precharged.
		0 Disable Precharge Bank 1
		1 Enable Precharge Bank 1

Table 8-12: DMC_EFFCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	PRECBANK0	Precharge Bank 0. The DMC_EFFCTL.PRECBANK0 bit enables precharge (closes the page) of bank 0 after each transfer if the DMC precharge feature is enabled (DMC_CTL.PREC =1). Note: The (DMC_CTL.PREC) takes precedence over value in this register. If (DMC_CTL.PREC =1) then all banks are precharged.
		0 Disable Precharge Bank 0
		1 Enable Precharge Bank 0

Shadow EMR3 Register

The `DMC_EMR3` register in the DMC shadows the EMR3 register in the SDRAM when the DMC is in DDR3 mode (`DMC_CTL.DDR3EN = 1`). If unmasked by the corresponding bit in the shadow mask register (`DMC_MSK.EMR3 = 1`), a write to `DMC_EMR3` triggers an extended “mode register set” command on the memory interface. If masked, a write to `DMC_EMR3` only updates the register in the DMC, not the register in the SDRAM.

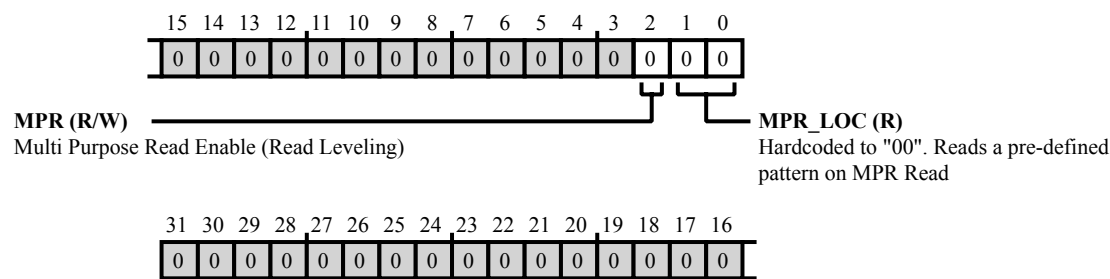


Figure 8-8: DMC_EMR3 Register Diagram

Table 8-13: DMC_EMR3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	MPR	Multi Purpose Read Enable (Read Leveling). The <code>DMC_EMR3.MPR</code> bit enables high temperature self-refresh rate.
		0 Disable
		1 Enable
1:0 (R/NW)	MPR_LOC	Hardcoded to "00". Reads a pre-defined pattern on MPR Read.

Shadow MR0 Register (DDR3)

The **DMC_MR** register in the DMC shadows the MR register in the SDRAM when the DMC is DDR3 mode (**DMC_CTL.DDR3EN** = 1). If unmasked by the corresponding bit in the shadow mask register (**DMC_MSK.MR** = 1), a write to **DMC_MR** triggers a “mode register set” command on the memory interface. If masked, a write to **DMC_MR** only updates the register in the DMC, not the register in the SDRAM.

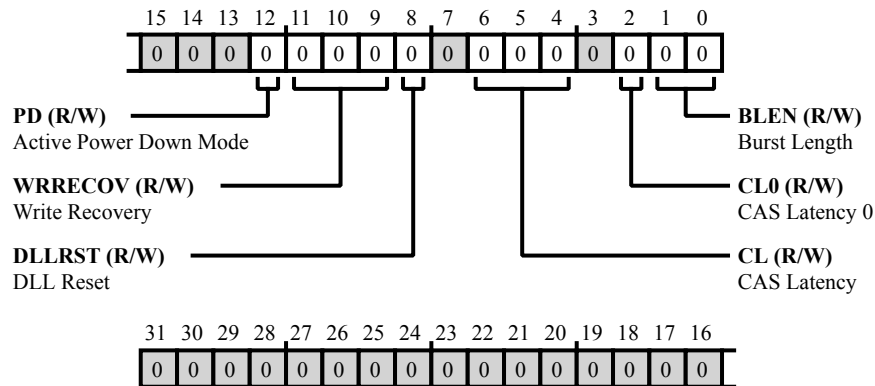


Figure 8-9: DMC_MR Register Diagram

Table 8-14: DMC_MR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	PD	Active Power Down Mode. The DMC_MR.PD bit selects the active power-down mode. Note that this parameter applies only for DDR3 mode. For more information about this mode, see the data sheet for the SDRAM being used in your system.
		0 Fast Exit (normal)
		1 Slow Exit (low power)
11:9 (R/W)	WRRECOV	Write Recovery. The DMC_MR.WRRECOV bit selects the write recovery time in terms of clock cycles (t_{CK}). Note that this parameter applies only for DDR3 mode. For more information about this mode, see the data sheet for the SDRAM being used in your system.
		0 16 clock cycles for DDR3
		1 5 clock cycles for DDR3
		2 6 clock cycles for DDR3
		3 7 clock cycles for DDR3
		4 8 clock cycles for DDR3
		5 10 clock cycles for DDR3
		6 12 clock cycles for DDR3

Table 8-14: DMC_MR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		7	14 clock cycles for DDR3
8 (R/W)	DLLRST	DLL Reset. The DMC_MR.DLLRST bit initiates a DLL reset on the SDRAM. Note that this parameter applies only for DDR3 mode. For more information about this operation, see the data sheet for the SDRAM being used in your system.	
		0	Normal Operation
		1	Reset DLL
6:4 (R/W)	CL	CAS Latency. The DMC_MR.CL bit field select latency from the assertion of a read/write signal to the SDRAM until the first valid data on the output from the SDRAM in terms of clock cycles. For more information about this operation, see the data sheet for the SDRAM being used in your system. For DDR3 only, Bit[2] of this register must be used along with [6:4]. Following CAS values are seen. "0010": 5 "0100": 6 "0110": 7 "1000": 8 "1010": 9 "1100": 10 "1110": 11 "0001": 12 "0011": 13 "0101": 14 All other combinations are reserved.	
2 (R/W)	CL0	CAS Latency 0. The DMC_MR.CL0 bit is applicable for DDR3 only and is used in conjunction with the DMC_MR.CL bits.	
1:0 (R/W)	BLEN	Burst Length. The DMC_MR.BLEN bits select burst length for transfers. For more information about this operation, see the data sheet for the SDRAM being used in your system. Note that values other than those shown are not supported.	
		0	8-Bit Burst Length - DDR3 only
		2	Reserved
		3	Reserved

Shadow MR1 Register (DDR3)

The `DMC_MR1` register is a mirror of the DDR3 SDRAM Mode register 1. This register is used only when the DMC is operating in DDR3 mode. A write to this register triggers an extended "mode register 1 set" command on the memory interface provided the corresponding mask bit is set in the mask register. Else, only the mirror register is updated.

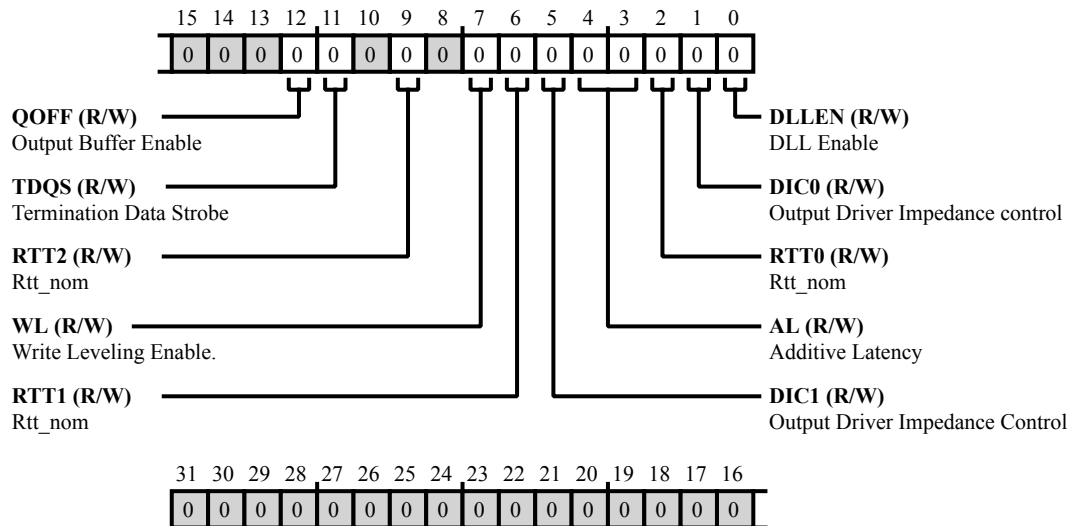


Figure 8-10: DMC_MR1 Register Diagram

Table 8-15: DMC_MR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	QOFF	Output Buffer Enable. The <code>DMC_MR1.QOFF</code> bit enables the SDRAM output pins. For more information about this operation, see the data sheet for the SDRAM being used in your system.
		0 Output buffer enabled
		1 Output buffer disabled
11 (R/W)	TDQS	Termination Data Strobe. The <code>DMC_MR1.TDQS</code> bit provides additional termination resistance outputs that may be useful in some system configurations. The <code>DMC_MR1.TDQS</code> bit is not supported in x4 or x16 configurations. When enabled via the mode register, the same termination resistance function is applied to the TDQS/TDQS# pins that is applied to the DQS/DQS# pins.
		0 Enable
		1 Disable

Table 8-15: DMC_MR1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration				
9 (R/W)	RTT2	Rtt_nom. The DMC_MR1 . RTT2 bit is used in conjunction with the DMC_MR1 . RTT0 and DMC_MR1 . RTT1 bits. (9 6 2) 0 0 0 Rtt_Nom disabled 0 0 1 RZQ/4 0 1 0 RZQ/2 0 1 1 RZQ/6 1 0 0 RZQ/12 (reserved if Rtt_Nom is used during writes) 1 0 1 RZQ/8 (reserved if Rtt_Nom is used during writes) 1 1 0 Reserved 1 1 1 Reserved				
7 (R/W)	WL	Write Leveling Enable.. The DMC_MR1 . WL bit enables the SDRAM output pins. For more information about this operation, see the data sheet for the SDRAM being used in your system. <table><tr><td>0</td><td>Disable</td></tr><tr><td>1</td><td>Enable</td></tr></table>	0	Disable	1	Enable
0	Disable					
1	Enable					
6 (R/W)	RTT1	Rtt_nom. The DMC_MR1 . RTT1 bit combines with the DMC_MR1 . RTT0 bit to set the termination resistance. See the DMC_MR1 . RTT2 and DMC_MR1 . RTT0 bit description for more information.				
5 (R/W)	DIC1	Output Driver Impedance Control. The DMC_MR1 . DIC1 bit is used in conjunction with the DMC_MR1 . DIC0 bit. (5, 1) 0 0 RZQ/6 0 1 RZQ/7 1 0 Reserved 1 1 Reserved				

Table 8-15: DMC_MR1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:3 (R/W)	AL	Additive Latency. The DMC_MR1.AL bits select a number of added latency time for CAS operations in terms of clock cycles (t_{CK}). For more information about this operation, see the data sheet for the SDRAM being used in your system.
		0 AL disabled
		1 CL-1
		2 CL-2
		3 Reserved
2 (R/W)	RTT0	Rtt_nom. The DMC_MR1.RTT0 bit combines with the DMC_MR1.RTT1 and DMC_MR1.RTT1 bits to set the termination resistance. See the DMC_MR1.RTT1 and DMC_MR1.RTT2 bit descriptions for more information.
1 (R/W)	DIC0	Output Driver Impedance control. The DMC_MR1.DIC0 bit is used with the DMC_MR1.DIC1 bit.
0 (R/W)	DLLEN	DLL Enable. The DMC_MR1.DLLEN bit enables the DLL in the SDRAM. For more information about this operation, see the data sheet for the SDRAM being used in your system.
		0 Enable
		1 Disable

Shadow MR2 Register (DDR3)

The `DMC_MR2` register mirrors DDR3 SDRAM device Mode register 2 when the controller is operating in DDR3 mode. A write to this register triggers an extended "mode register set" command on the memory interface provided the corresponding mask bit is set in the mask register. Else, only the mirror register is updated.

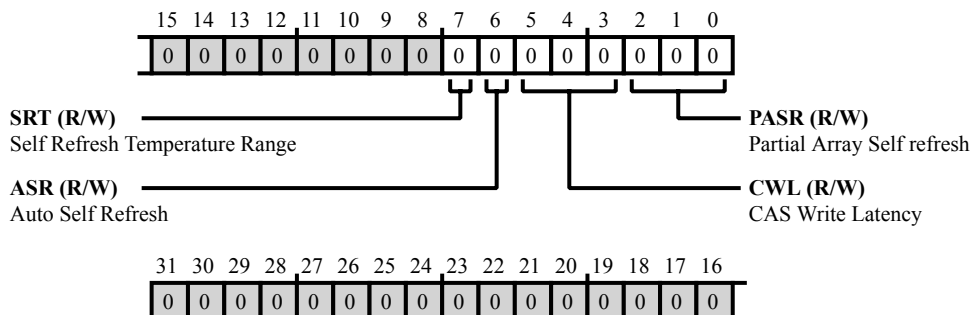


Figure 8-11: DMC_MR2 Register Diagram

Table 8-16: DMC_MR2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	SRT	Self Refresh Temperature Range. The <code>DMC_MR2</code> . SRT bit enables high temperature self-refresh rate.
		0 Disable
		1 Enable
6 (R/W)	ASR	Auto Self Refresh.
		0 Manual SR Reference (SRT)
		1 ASR enable (Optional)
5:3 (R/W)	CWL	CAS Write Latency.
		0 5 clock cycles
		1 6 clock cycles
		2 7 clock cycles
		3 8 clock cycles
		4 9 clock cycles
		5 10 clock cycles
		6 11 clock cycles
		7 12 clock cycles

Table 8-16: DMC_MR2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	PASR	Partial Array Self refresh. The DMC_MR2 . PASR bits select the amount of memory to be refreshed during self refresh. For more information about this operation, see the data sheet for the SDRAM being used in your system.
		0 4 banks: full array, 8 banks: full array
		1 4 banks: Half Array (BA[1:0]=00&01), 8 banks: Half Array (BA[2:0] = 000, 001, 010, &011)
		2 4 banks: Quarter Array (BA[1:0]=00), 8 banks: Quarter Array (BA[2:0] = 000&001)
		3 4 banks: not defined, 8 banks: 1/8th array (BA[2:0] = 000)
		4 4 banks: 3/4 Array (BA[1:0]=01, 10&11), 8 banks: 3/4 Array (BA[2:0] = 010, 011, 100, 101, 110, &111)
		5 4 banks: Half Array (BA[1:0]=10&11), 8 banks: Half Array (BA[2:0] = 100, 101, 110, &111)
		6 4 banks: Quarter Array (BA[1:0]=11), 8 banks: Quarter Array (BA[2:0] = 110 & 111)
		7 4 banks: not defined, 8 banks: 1/8th array (BA[2:0] = 111)

Mask (Mode Register Shadow) Register

The `DMC_MSK` register permits masking (disabling) writes to the MR and EMRn registers in the SDRAM in DDR3 Mode. When masked, writes to these registers go instead to shadow copies of these registers (`DMC_MR`, `DMC_MR1`, `DMC_MR2`), which are maintained within the DMC. When a shadow register's corresponding bit is unmasked (enabled), the DMC generates the MRS or EMRS command to transfer the contents of the shadow register (in the DMC) to the actual register (in the SDRAM).

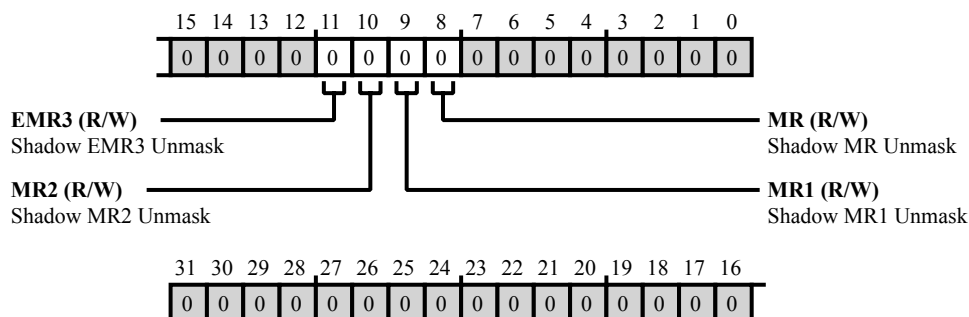


Figure 8-12: DMC_MSK Register Diagram

Table 8-17: DMC_MSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	EMR3	Shadow EMR3 Unmask. The <code>DMC_MSK.EMR3</code> bit masks or unmasks writes to the EMR3 register in the SDRAM. When masked, writes to this register instead go to the EMR3 register. When unmasked, the DMC writes the EMR3 value to the EMR3 register in the SDRAM. After completing the write, the DMC clears this bit.
		0 Mask (Disable) Write to EMR3
		1 Unmask (Enable) Write to EMR3
10 (R/W)	MR2	Shadow MR2 Unmask. The <code>DMC_MSK.MR2</code> bit masks or unmasks writes to the MR2 register (in DDR3 mode) in the SDRAM. When masked, writes to this register instead go to the <code>DMC_MR2</code> register. When unmasked, the DMC writes the <code>DMC_MR2</code> value to the MR2 register (in DDR3 mode) in the SDRAM. After completing the write, the DMC clears this bit.
		0 Mask (Disable) Write to EMR2
		1 Unmask (Enable) Write to EMR2

Table 8-17: DMC_MSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	MR1	Shadow MR1 Unmask. The DMC_MSK.MR1 bit masks or unmasks writes to the MR1 register in the DDR3 SDRAM. When masked, writes to this register instead go to the DMC_MR1 register. When unmasked, the DMC writes the DMC_MR1 value to the MR1 register in the SDRAM. After completing the write, the DMC clears this bit. Note that this bit is valid only for DDR3 Mode of operation.
		0 Mask (Disable) Write to EMR1
		1 Unmask (Enable) Write to EMR1
8 (R/W)	MR	Shadow MR Unmask. The DMC_MSK.MR bit masks or unmasks writes to the MR register in the SDRAM. When masked, writes to this register instead go to the DMC_MR register. When unmasked, the DMC writes the DMC_MR value to the MR register in the SDRAM. After completing the write, the DMC clears this bit.
		0 Mask (Disable) Write to MR
		1 Unmask (Enable) Write to MR

Priority ID Register 1

The `DMC_PRIO` register allows transactions from selected masters that generate specific SCB IDs to obtain higher priority than the transactions proceeding in the usual fashion. The contents of the register are masked with the contents of the `DMC_PRIOMSK` register to obtain a single SCB ID or a range of IDs that get elevated priority.

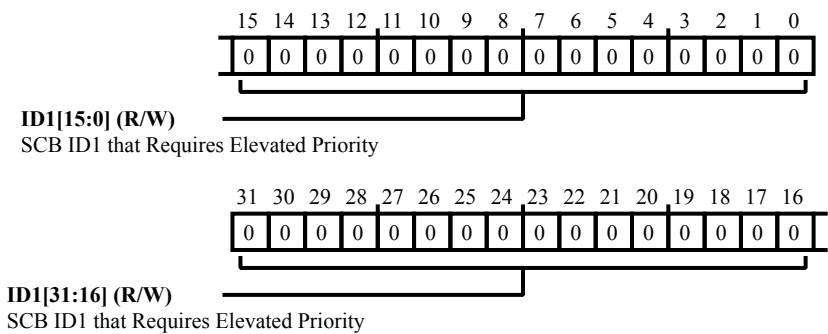


Figure 8-13: DMC_PRIO Register Diagram

Table 8-18: DMC_PRIO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ID1	SCB ID1 that Requires Elevated Priority.

Priority ID Register 2

The `DMC_PRIO2` register is another register which allows transactions from selected masters that generate specific SCB IDs to obtain higher priority than the transactions proceeding in the usual fashion. The contents of the register are masked with the contents of the `DMC_PRIOMSK2` register to obtain a single SCB ID or a range of IDs that get elevated priority.

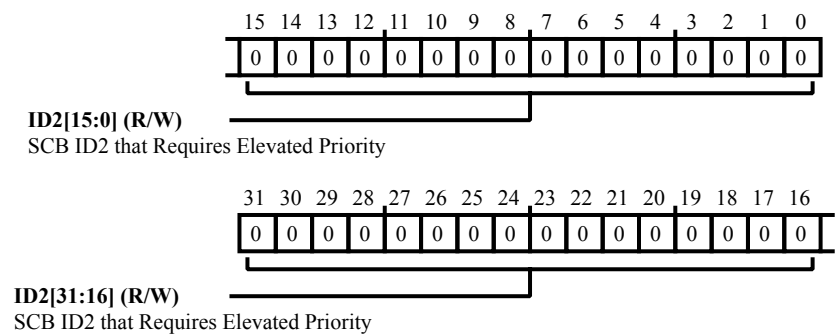


Figure 8-14: `DMC_PRIO2` Register Diagram

Table 8-19: `DMC_PRIO2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ID2	SCB ID2 that Requires Elevated Priority.

Priority ID Mask Register 1

The `DMC_PRIOMSK` register masks the respective ID bits in the `DMC_PRIOMSK` register. This masking provides for elevating the access priority of either a single ID or a range of IDs.

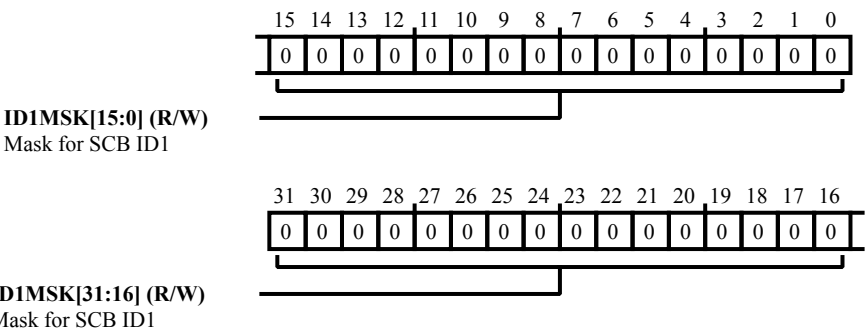


Figure 8-15: DMC_PRIOMSK Register Diagram

Table 8-20: DMC_PRIOMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ID1MSK	Mask for SCB ID1.

Priority ID Mask Register 2

The `DMC_PRIOMSK2` register bits mask the respective ID bits in the `DMC_PRIO2` register. This masking provides for elevating the access priority of either a single ID or a range of IDs.

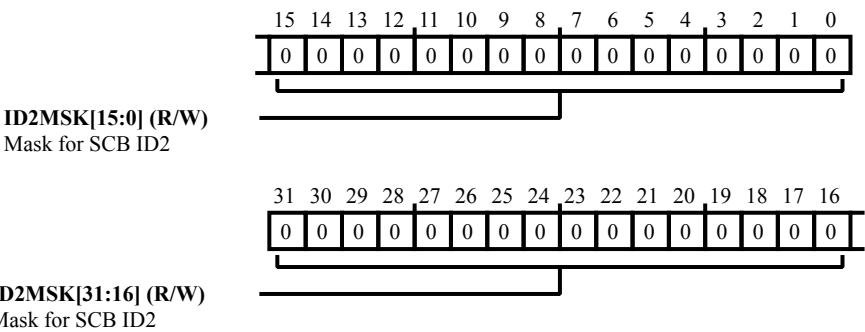


Figure 8-16: `DMC_PRIOMSK2` Register Diagram

Table 8-21: `DMC_PRIOMSK2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ID2MSK	Mask for SCB ID2.

DMC Read Data Buffer ID Register 1

The [DMC_RDDATABUFID1](#) register allows read transactions from selected masters to make use of DMC read data buffer. The contents of the register are masked with the contents of the [DMC_RDDATABUFMSK1](#) register to obtain a single SCB ID or a range of IDs that get elevated priority.

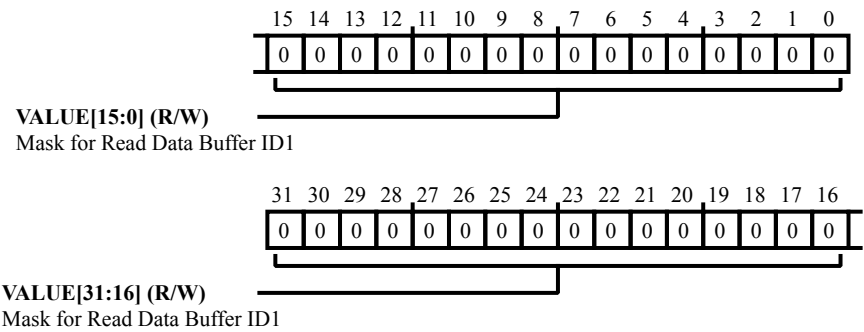


Figure 8-17: DMC_RDDATABUFID1 Register Diagram

Table 8-22: DMC_RDDATABUFID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Mask for Read Data Buffer ID1.

DMC Read Data Buffer ID Register 2

The [DMC_RDDATABUFID2](#) register allows read transactions from selected masters to make use of DMC read data buffer. The contents of the register are masked with the contents of the [DMC_RDDATABUFMSK2](#) register to obtain a single SCB ID or a range of IDs that get elevated priority.

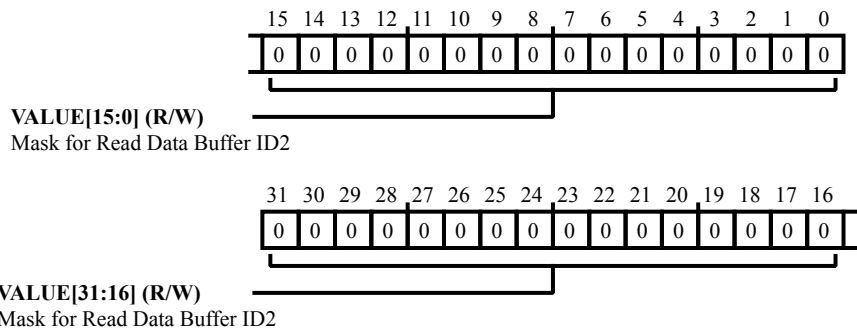


Figure 8-18: DMC_RDDATABUFID2 Register Diagram

Table 8-23: DMC_RDDATABUFID2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Mask for Read Data Buffer ID2.

DMC Read Data Buffer Mask Register 1

The `DMC_RDDATABUFMSK1` register bits mask the respective ID bits in the DMC Priority Mask ID register.

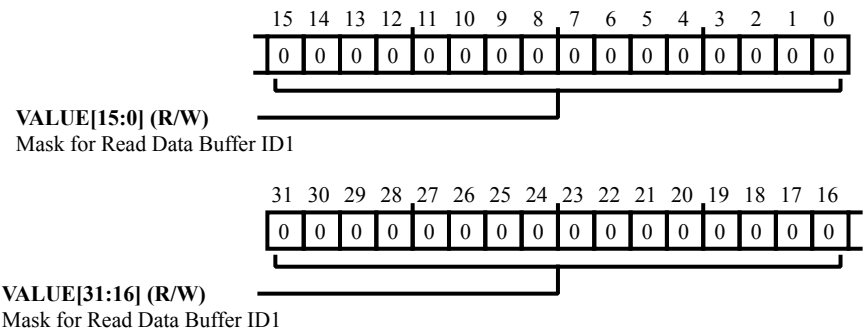


Figure 8-19: DMC_RDDATABUFMSK1 Register Diagram

Table 8-24: DMC_RDDATABUFMSK1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Mask for Read Data Buffer ID1.

DMC Read Data Buffer Mask Register 2

The `DMC_RDDATABUFMSK2` register bits mask the respective ID bits in the DMC Priority Mask ID register.

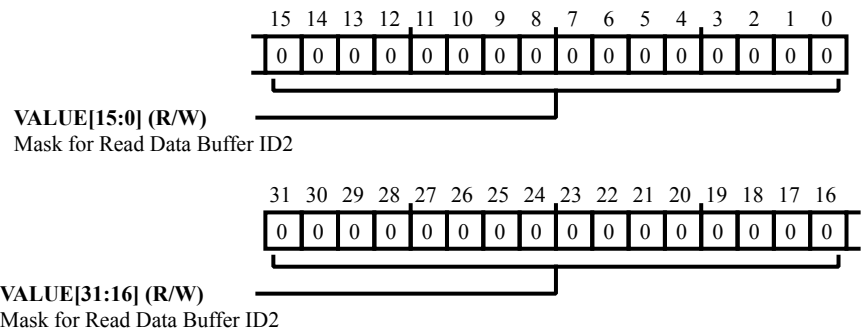


Figure 8-20: `DMC_RDDATABUFMSK2` Register Diagram

Table 8-25: `DMC_RDDATABUFMSK2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Mask for Read Data Buffer ID2.

Status Register

The `DMC_STAT` register indicates status for modes selected with the `DMC_CTL` register and indicates status DMC operations.

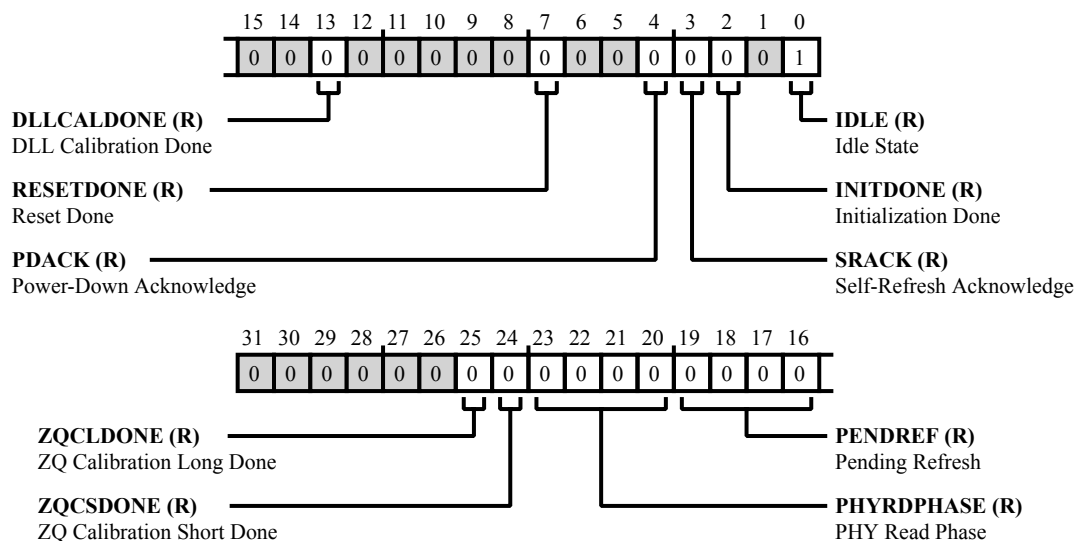


Figure 8-21: DMC_STAT Register Diagram

Table 8-26: DMC_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/NW)	ZQCLDONE	ZQ Calibration Long Done. The <code>DMC_STAT.ZQCLDONE</code> bit checks if the ZQ calibration long sub routine is done.
		0 ZQ Calibration long is ongoing
		1 ZQ Calibration long is done
24 (R/NW)	ZQCSDONE	ZQ Calibration Short Done. The <code>DMC_STAT.ZQCSDONE</code> bit checks if the ZQ calibration short sub routine is done.
		0 ZQ Calibration short is ongoing
		1 ZQ Calibration Short is done
23:20 (R/NW)	PHYRDPHASE	PHY Read Phase. The <code>DMC_STAT.PHYRDPHASE</code> bits indicate the latency after which the DMC may read from the PHY. Taking round trip delay into account, the DLL indicates the exact number of clock cycles after which the controller needs to read data. Values other than those shown are reserved.
		2 2 Clock Cycles Latency

Table 8-26: DMC_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		3	3 Clock Cycles Latency
		4	4 Clock Cycles Latency
		5	5 Clock Cycles Latency
		6	6 Clock Cycles Latency
		7	7 Clock Cycles Latency
		8	8 Clock Cycles Latency
		9	9 Clock Cycles Latency
		10	10 Clock Cycles Latency
		11	11 Clock Cycles Latency
		12	12 Clock Cycles Latency
		13	13 Clock Cycles Latency
		14	14 Clock Cycles Latency
		15	15 Clock Cycles Latency
19:16 (R/NW)	PENDREF	Pending Refresh. The DMC_STAT . PENDREF bits indicate the number of pending auto-refresh commands whose value can be from "0000" to "0111".	
13 (R/NW)	DLLCALDONE	DLL Calibration Done. The DMC_STAT . DLLCALDONE indicates that the PHY DLL calibration sequence is complete.	
		0	No Status
		1	Completed PHY DLL Calibration
7 (R/NW)	RESETDONE	Reset Done. The DMC_STAT . RESETDONE bit indicates that the reset sequence is complete.	
		0	SDRAM Reset is ongoing
		1	SDRAM Reset is done
4 (R/NW)	PDACK	Power-Down Acknowledge. The DMC_STAT . PDACK bit indicates that power-down mode is active.	
		0	Not in Power-Down Mode
		1	Power-Down Mode Active

Table 8-26: DMC_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/NW)	SRACK	Self-Refresh Acknowledge. The DMC_STAT . SRACK bit indicates that self-refresh mode is active.
		0 Not in Self-Refresh Mode
		1 Self-Refresh Mode Active
2 (R/NW)	INITDONE	Initialization Done. The DMC_STAT . INITDONE bit indicates that the initialization sequence is complete.
		0 No Status
		1 Initialize Done
0 (R/NW)	IDLE	Idle State. The DMC_STAT . IDLE bit indicates whether the DMC is idle or busy.
		0 Busy
		1 Idle

Timing 0 Register

The `DMC_TR0` register selects timing parameters for DMC operation to corresponding with parameters of the SDRAM device that is used in the system. The timing registers must be programmed to match the device for correct operation of the SDRAM and must be programmed before initializing the SDRAM. Note that all values for bit fields in `DMC_TR0` are in increments of clock cycle time (t_{CK}).

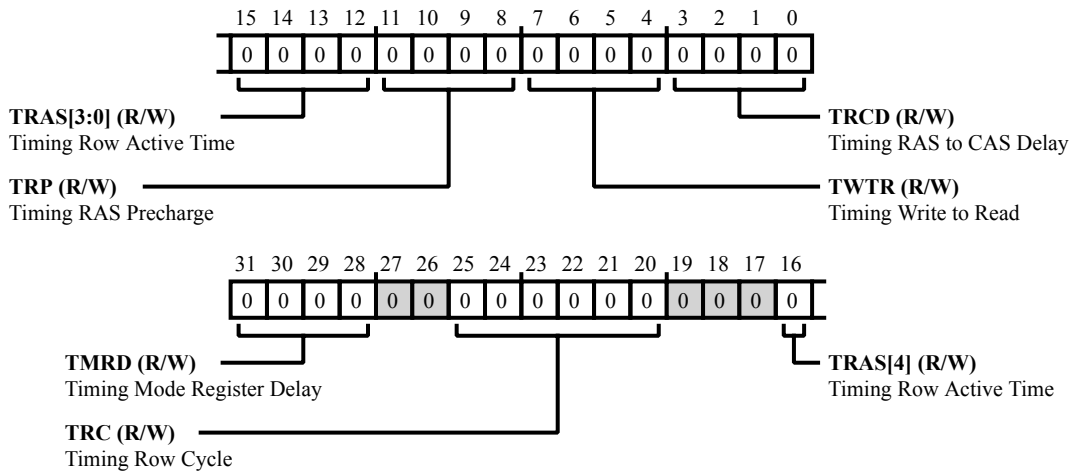


Figure 8-22: DMC_TR0 Register Diagram

Table 8-27: DMC_TR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:28 (R/W)	TMRD	Timing Mode Register Delay. The <code>DMC_TR0</code> . TMRD field selects the set-to-active timing parameter (t_{MRD}), which is the number of clock cycles that occur after the mode registers in the SDRAM are set and before the next command is issued.
25:20 (R/W)	TRC	Timing Row Cycle. The <code>DMC_TR0</code> . TRC field selects the active-to-active time (t_{RC}), which is the minimum number of clock cycles that occur from an active command to the next active command in the same bank.
16:12 (R/W)	TRAS	Timing Row Active Time. The <code>DMC_TR0</code> . TRAS field selects the active-to-precharge time (t_{RAS}), which is the number of clock cycles that occur from an active command until a precharge command is allowed.
11:8 (R/W)	TRP	Timing RAS Precharge. The <code>DMC_TR0</code> . TRP field selects the precharge-to-active time (t_{RP}), which is the number of clock cycles that occur while the SDRAM recovers from a precharge command and becomes ready to accept the next active command.

Table 8-27: DMC_TR0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/W)	TWTR	Timing Write to Read. The DMC_TR0 . TWTR field selects the write-to-read delay time (t_{WTR}), which is the number of clock cycles that occur from the last write data to the next read command.
3:0 (R/W)	TRCD	Timing RAS to CAS Delay. The DMC_TR0 . TRCD field selects the RAS to CAS delay time (t_{RCD}), which is the number of clock cycles that occur from an active command to a read/write assertion.

Timing 1 Register

The `DMC_TR1` register selects timing parameters for DMC operation to corresponding with parameters of the SDRAM device that is used in the system. The timing registers must be programmed to match the device for correct operation of the SDRAM and must be programmed before initializing the SDRAM. Note that all values for bit fields in `DMC_TR1` are in increments of clock cycle time (t_{CK}).

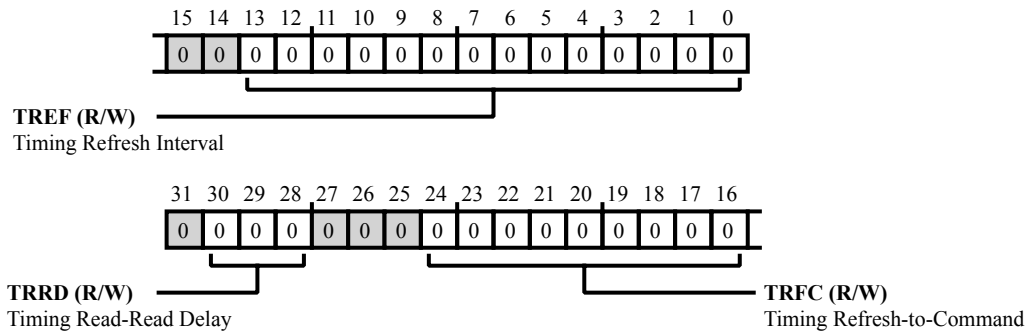


Figure 8-23: DMC_TR1 Register Diagram

Table 8-28: DMC_TR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30:28 (R/W)	TRRD	Timing Read-Read Delay. The <code>DMC_TR1 . TRRD</code> field selects the active-to-active time (t_{RRD}), which is the minimum number of clock cycles occurring from a bank x active command to a bank y active command.
24:16 (R/W)	TRFC	Timing Refresh-to-Command. The <code>DMC_TR1 . TRFC</code> field selects the refresh-to-active command delay (t_{RFC}), which is the number of clock cycles required for the SDRAM to recover from a refresh signal to be ready to take the next command. It is also the number of clock cycles needed for the SDRAM to recover from executing one active command and ready to accept the next active command.
13:0 (R/W)	TREF	Timing Refresh Interval. The <code>DMC_TR1 . TREF</code> field selects the refresh interval time (t_{REF}), which is the number of clock cycles occurring from one refresh command to the next refresh command. The actual timing of issuing a precharge command may be delayed by if the SDRAM is processing a normal access. However, the delay is not accumulative so there is no need to shorten the refresh interval to account for the memory access time. The non-accumulative refresh delay typically increases memory bandwidth by a few percentage points.

Timing 2 Register

The `DMC_TR2` register selects timing parameters for DMC operation to corresponding with parameters of the SDRAM device that is used in the system. The timing registers must be programmed to match the device for correct operation of the SDRAM and before initializing the SDRAM.

Note that all values for bit fields in `DMC_TR2` are in increments of clock cycle time (t_{CK}).

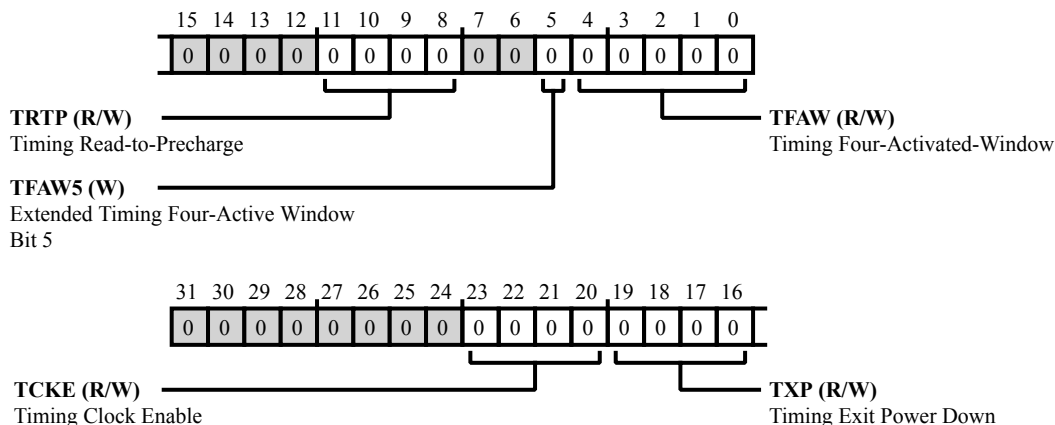


Figure 8-24: DMC_TR2 Register Diagram

Table 8-29: DMC_TR2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:20 (R/W)	TCKE	Timing Clock Enable. The <code>DMC_TR2.TCKE</code> field selects the CKE minimum pulsewidth (t_{CKE}).
19:16 (R/W)	TXP	Timing Exit Power Down. The <code>DMC_TR2.TXP</code> field selects the exit power down to next valid command time (t_{XP}).
11:8 (R/W)	TRTP	Timing Read-to-Precharge. The <code>DMC_TR2.TRTP</code> field selects the internal read to precharge time (t_{RTP}). If the resulting value is less than 2, the register needs to be programmed with two. Note: Minimum t_{RTP} that needs to be programmed is 2.
5 (RX/W)	TFAW5	Extended Timing Four-Active Window Bit 5. The <code>DMC_TR2.TFAW5</code> bit is the extended bit for FAW timing for 800 MHz operation for values greater than 31. It is only applicable when <code>DMC_CTL.AL_EN = 1</code> . When <code>DMC_CTL.AL_EN = 0</code> , the <code>DMC_TR2.TFAW5</code> bit is reserved. When <code>DMC_CTL.AL_EN = 1</code> , the total <code>FAW[5:0] = {TFAW5, TFAW[4:0]}</code> . NOTE: This bit is write-only. Read as zero.

Table 8-29: DMC_TR2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	TFAW	Timing Four-Activated-Window. The DMC_TR2 . TFAW field selects the four-banks-activated window time (t_{FAW}). No more than four SDRAM banks should be activated within this window.

ADSP-2159x DMC Register Descriptions

1600MHz DMCPHY (DDR3/DDR3L) (DMC) contains the following registers.

Table 8-30: ADSP-2159x DMC Register List

Name	Description
DMC_DDR_CA_CTL	DDR CA Lane Control Register
DMC_DDR_LANE0_CTL0	Data Lane 0 Control Register 0
DMC_DDR_LANE0_CTL1	Data Lane 0 Control Register 1
DMC_DDR_LANE1_CTL0	Data Lane 1 Control Register 0
DMC_DDR_LANE1_CTL1	Data Lane 1 Control Register 1
DMC_DDR_ROOT_CTL	DDR ROOT Module Control Register
DMC_DDR_ZQ_CTL0	DDR Calibration Control Register 0
DMC_DDR_ZQ_CTL1	DDR Calibration Control Register 1
DMC_DDR_ZQ_CTL2	DDR Calibration Control Register 2

DDR CA Lane Control Register

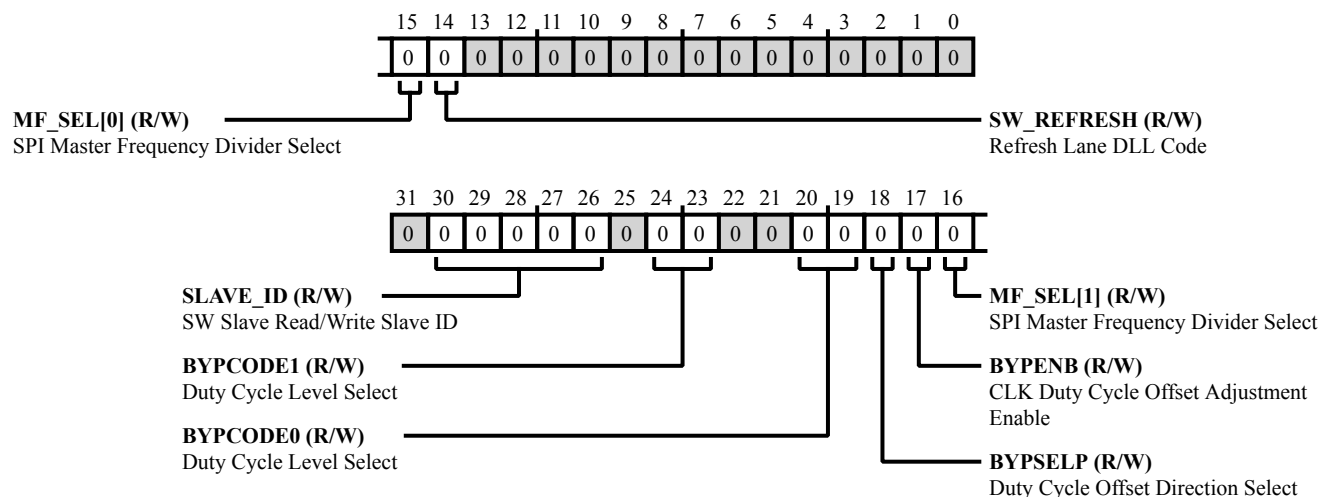


Figure 8-25: DMC_DDR_CA_CTL Register Diagram

Table 8-31: DMC_DDR_CA_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30:26 (R/W)	SLAVE_ID	SW Slave Read/Write Slave ID. The DMC_DDR_CA_CTL.SLAVE_ID bit field is the slave ID for SW slave read or write transactions.
24:23 (R/W)	BYPCODE1	Duty Cycle Level Select. The DMC_DDR_CA_CTL.BYPCODE1 bit field combines with BYPCODE0 to select one of four levels of duty cycle adjustment.
		0
		0
		1
		2
20:19 (R/W)	BYPCODE0	Duty Cycle Level Select. The DMC_DDR_CA_CTL.BYPCODE0 bit field combines with BYPCODE1 to select one of four levels of duty cycle adjustment.
		0
		0
		1
		2

Table 8-31: DMC_DDR_CA_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	BYPSELP	Duty Cycle Offset Direction Select. The DMC_DDR_CA_CTL.BYPSELP bit selects the direction of the duty cycle offset.
		0 Negative
		1 Positive
17 (R/W)	BYPENB	CLK Duty Cycle Offset Adjustment Enable. The DMC_DDR_CA_CTL.BYPENB bit enables the CLK duty cycle offset adjustment.
		0 Disable
		1 Enable
16:15 (R/W)	MF_SEL	SPI Master Frequency Divider Select. The DMC_DDR_CA_CTL.MF_SEL bit selects the SPI master frequency divider for root to lane transfers.
		0 Divide by 2
		1 Divide by 4
		2 Divide by 8
		3 Divide by 16
14 (R/W)	SW_REFRESH	Refresh Lane DLL Code.

Data Lane 0 Control Register 0

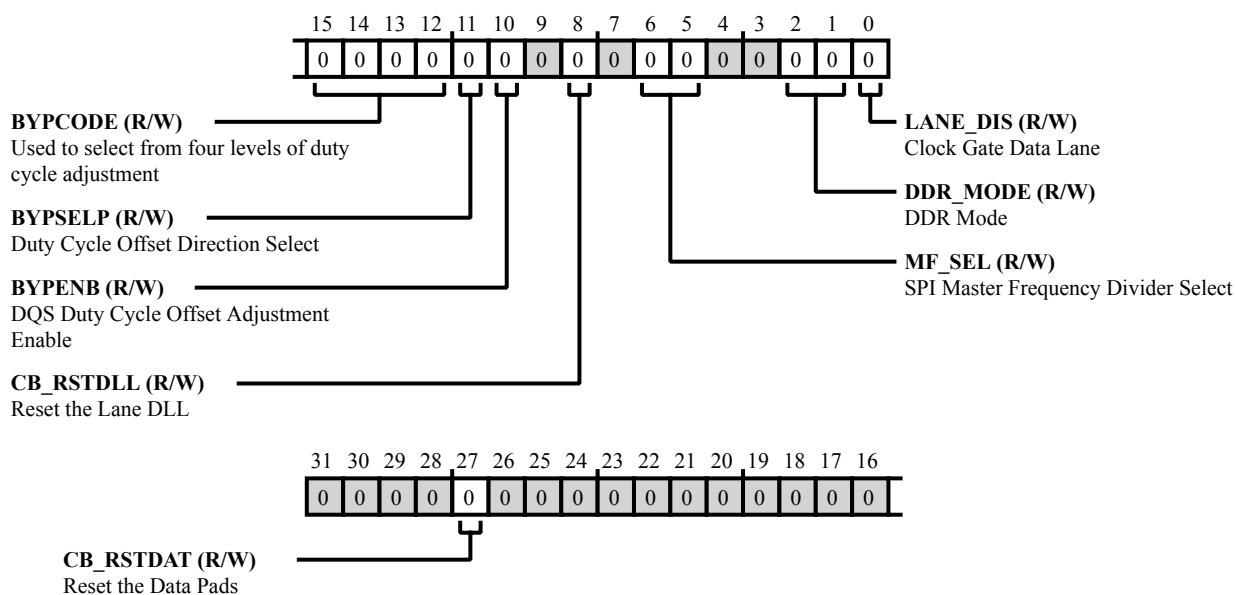


Figure 8-26: DMC_DDR_LANE0_CTL0 Register Diagram

Table 8-32: DMC_DDR_LANE0_CTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27 (R/W)	CB_RSTDAT	Reset the Data Pads.
15:12 (R/W)	BYPCODE	Used to select from four levels of duty cycle adjustment.
		1
		2
		4
		8
11 (R/W)	BYPSEL	Duty Cycle Offset Direction Select. The DMC_DDR_LANE0_CTL0.BYPSEL bit selects the direction of the duty cycle offset.
		0 Negative
		1 Positive

Table 8-32: DMC_DDR_LANE0_CTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	BYPENB	DQS Duty Cycle Offset Adjustment Enable. The DMC_DDR_LANE0_CTL0 .BYPENB bit enables the DQS duty cycle offset adjustment.
		0 Disable
		1 Enable
8 (R/W)	CB_RSTDLL	Reset the Lane DLL.
6:5 (R/W)	MF_SEL	SPI Master Frequency Divider Select. The DMC_DDR_LANE0_CTL0 .MF_SEL bit selects the SPI master frequency divider for root to lane transfers.
		0 Divide by 2
		1 Divide by 4
		2 Divide by 8
		3 Divide by 16
2:1 (R/W)	DDR_MODE	DDR Mode.
		0 DDR3/3L Mode
0 (R/W)	LANE_DIS	Clock Gate Data Lane.
		0 Lane is Enabled
		1 Lane is Clock Gated

Data Lane 0 Control Register 1

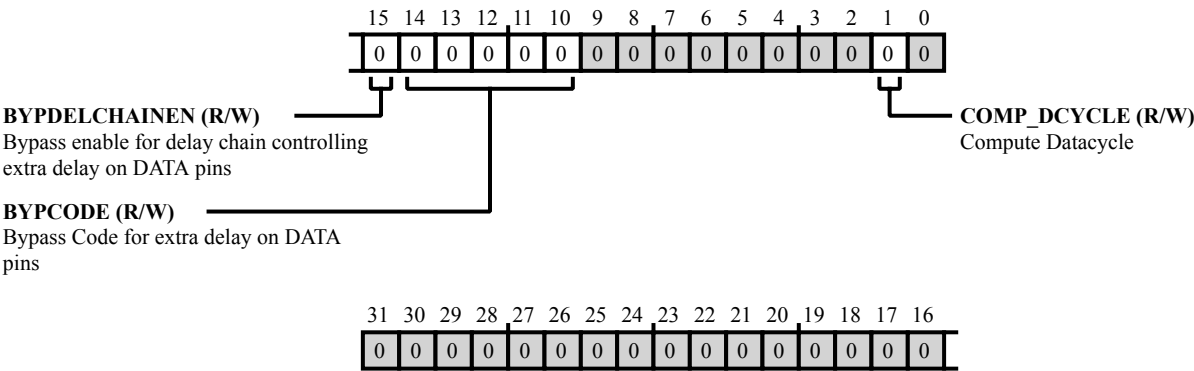


Figure 8-27: DMC_DDR_LANE0_CTL1 Register Diagram

Table 8-33: DMC_DDR_LANE0_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	BYPDELCHAINEN	Bypass enable for delay chain controlling extra delay on DATA pins.
14:10 (R/W)	BYPCODE	Bypass Code for extra delay on DATA pins.
1 (R/W)	COMP_DCYCLE	Compute Datacycle.

Data Lane 1 Control Register 0

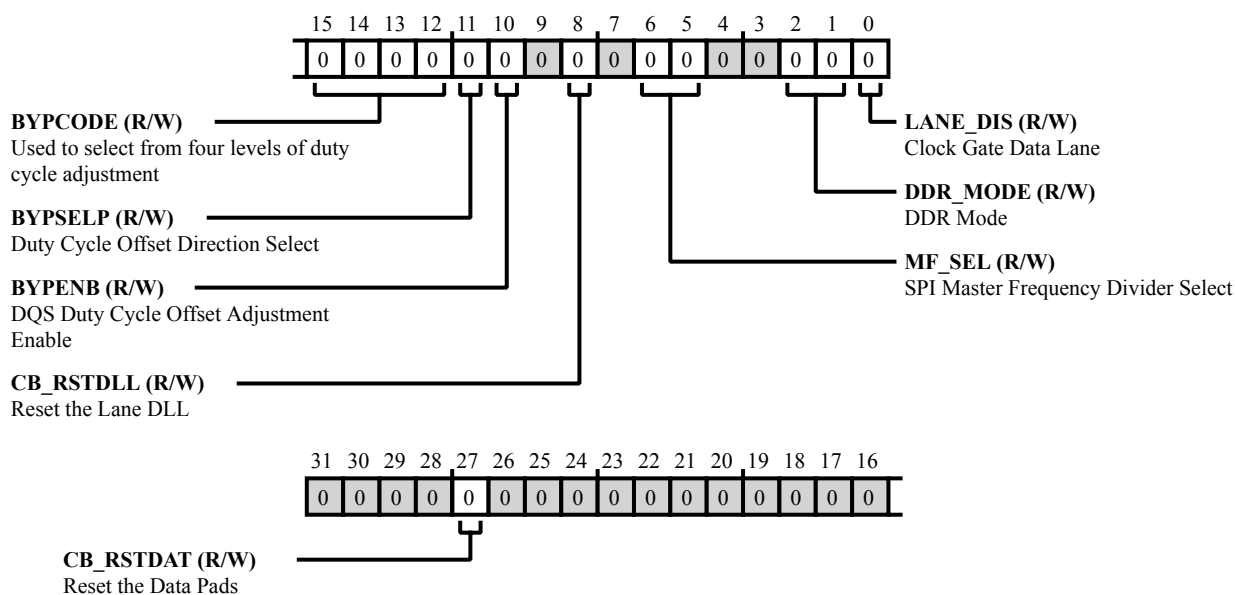


Figure 8-28: DMC_DDR_LANE1_CTL0 Register Diagram

Table 8-34: DMC_DDR_LANE1_CTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27 (R/W)	CB_RSTDAT	Reset the Data Pads.
15:12 (R/W)	BYPCODE	Used to select from four levels of duty cycle adjustment.
		1
		2
		4
		8
11 (R/W)	BYPSEL	Duty Cycle Offset Direction Select. The DMC_DDR_LANE1_CTL0.BYPSEL bit selects the direction of the duty cycle offset.
		0 Negative
		1 Positive

Table 8-34: DMC_DDR_LANE1_CTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	BYPENB	DQS Duty Cycle Offset Adjustment Enable. The DMC_DDR_LANE1_CTL0 .BYPENB bit enables the DQS duty cycle offset adjustment.
		0 Disable
		1 Enable
8 (R/W)	CB_RSTDLL	Reset the Lane DLL.
6:5 (R/W)	MF_SEL	SPI Master Frequency Divider Select. The DMC_DDR_LANE1_CTL0 .MF_SEL bit selects the SPI master frequency divider for root to lane transfers.
		0 Divide by 2
		1 Divide by 4
		2 Divide by 8
		3 Divide by 16
2:1 (R/W)	DDR_MODE	DDR Mode.
		0 DDR3/3L Mode
0 (R/W)	LANE_DIS	Clock Gate Data Lane.
		0 Lane is Enabled
		1 Lane is Clock Gated

Data Lane 1 Control Register 1

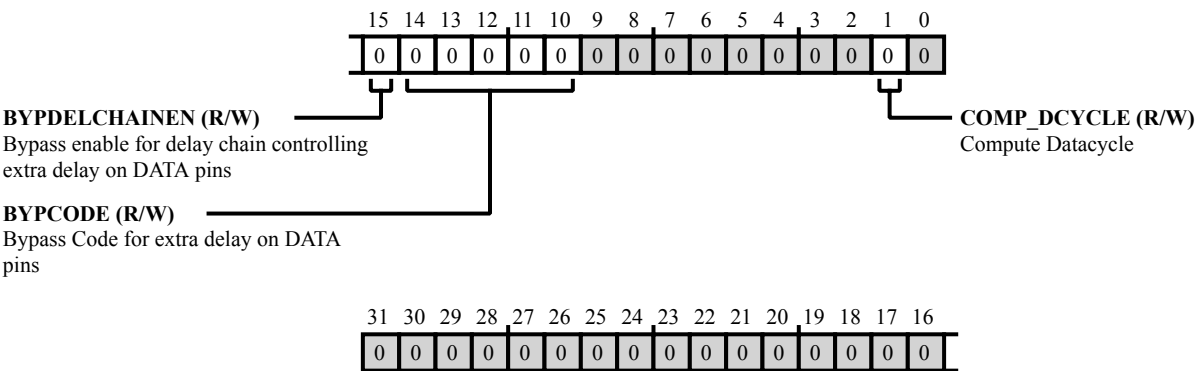


Figure 8-29: DMC_DDR_LANE1_CTL1 Register Diagram

Table 8-35: DMC_DDR_LANE1_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	BYPDELCHAINEN	Bypass enable for delay chain controlling extra delay on DATA pins.
14:10 (R/W)	BYPCODE	Bypass Code for extra delay on DATA pins.
1 (R/W)	COMP_DCYCLE	Compute Datacycle.

DDR ROOT Module Control Register

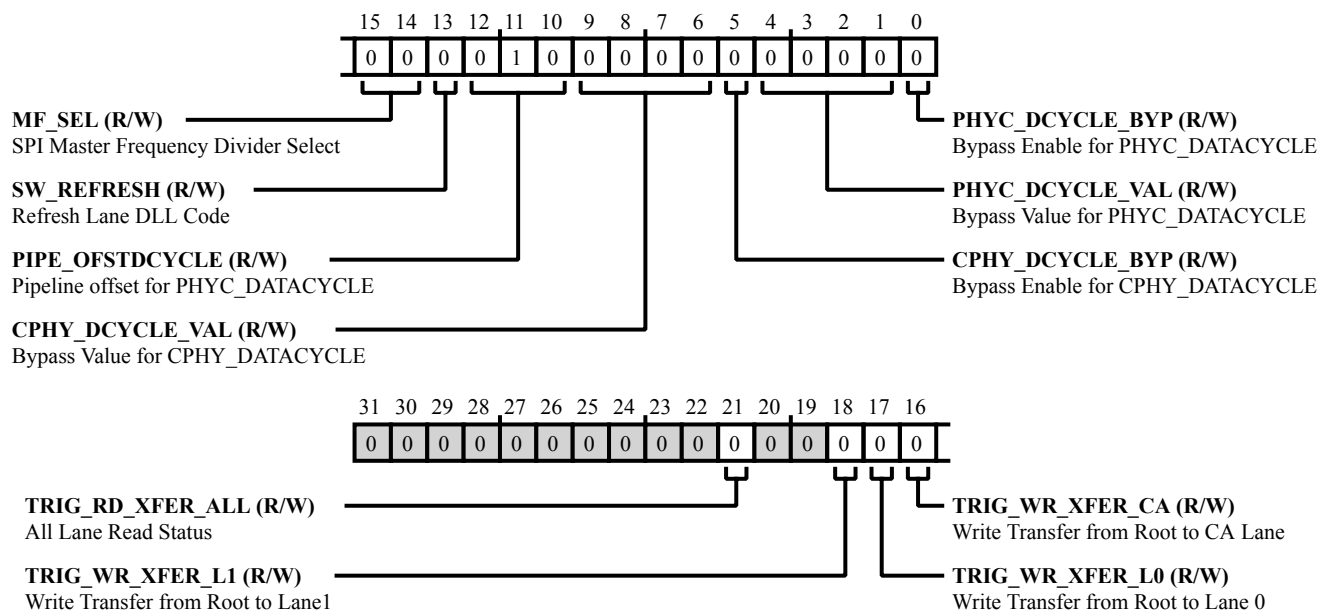


Figure 8-30: DMC_DDR_ROOT_CTL Register Diagram

Table 8-36: DMC_DDR_ROOT_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	TRIG_RD_XFER_ALL	All Lane Read Status. Software trigger to read the status data from all lanes.
18 (R/W)	TRIG_WR_XFER_L1	Write Transfer from Root to Lane1. Software trigger for a write transfer from the root to lane 1.
17 (R/W)	TRIG_WR_XFER_L0	Write Transfer from Root to Lane 0. Software trigger for a write transfer from the root to lane 0.
16 (R/W)	TRIG_WR_XFER_CA	Write Transfer from Root to CA Lane. Software trigger for a write transfer from the root to CA lane.
15:14 (R/W)	MF_SEL	SPI Master Frequency Divider Select. The DMC_DDR_ROOT_CTL.MF_SEL bit selects the SPI master frequency divider for root to lane transfers.
		0 Divide by 2
		1 Divide by 4
		2 Divide by 8
		3 Divide by 16

Table 8-36: DMC_DDR_ROOT_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	SW_REFRESH	Refresh Lane DLL Code.
12:10 (R/W)	PIPE_OFSTDCYCLE	Pipeline offset for PHYC_DATACYCLE.
9:6 (R/W)	CPHY_DCYCLE_VAL	Bypass Value for CPHY_DATACYCLE.
5 (R/W)	CPHY_DCYCLE_BYP	Bypass Enable for CPHY_DATACYCLE.
4:1 (R/W)	PHYC_DCYCLE_VAL	Bypass Value for PHYC_DATACYCLE.
0 (R/W)	PHYC_DCYCLE_BYP	Bypass Enable for PHYC_DATACYCLE.

DDR Calibration Control Register 0

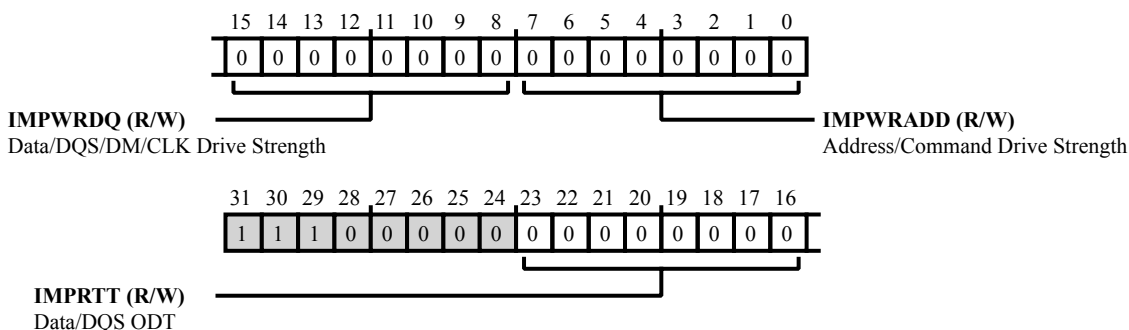


Figure 8-31: DMC_DDR_ZQ_CTL0 Register Diagram

Table 8-37: DMC_DDR_ZQ_CTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	IMPRTT	Data/DQS ODT. Desired ODT in ohms for Data and DQS pads.
15:8 (R/W)	IMPWRDQ	Data/DQS/DM/CLK Drive Strength. Desired drive strength in ohms for Data, DQS, DM, and clock pads. A drive strength of 100 ohms is recommended.
7:0 (R/W)	IMPWRADD	Address/Command Drive Strength. Desired drive strength in ohms for address and command pads. A drive strength of 100 ohms is recommended.

DDR Calibration Control Register 1

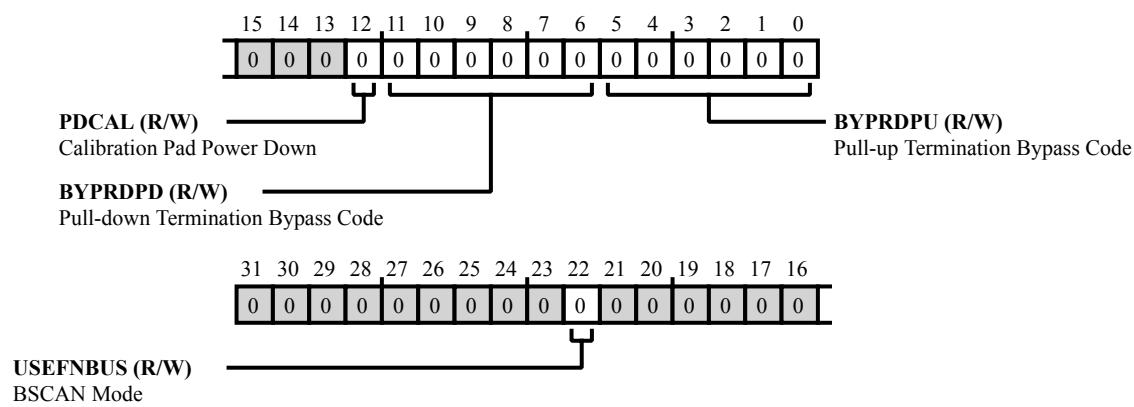


Figure 8-32: DMC_DDR_ZQ_CTL1 Register Diagram

Table 8-38: DMC_DDR_ZQ_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	USEFNBUS	BSCAN Mode. The bit controls whether to use bypass codes or functional codes in BSCAN mode.
12 (R/W)	PDCAL	Calibration Pad Power Down. Software bypass to power down the calibration pad.
11:6 (R/W)	BYPRDPD	Pull-down Termination Bypass Code.
5:0 (R/W)	BYPRDPU	Pull-up Termination Bypass Code.

DDR Calibration Control Register 2

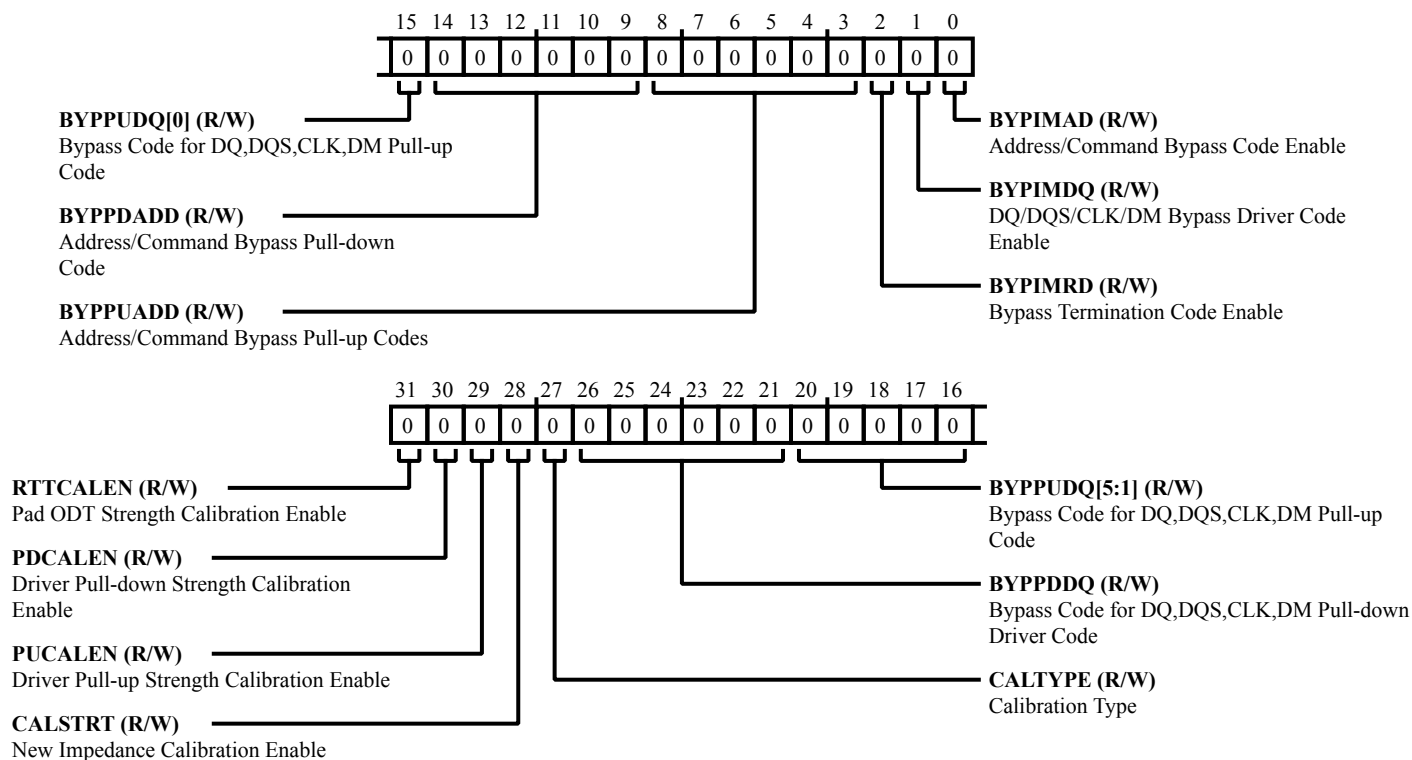


Figure 8-33: DMC_DDR_ZQ_CTL2 Register Diagram

Table 8-39: DMC_DDR_ZQ_CTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	RTTCALEN	Pad ODT Strength Calibration Enable.
30 (R/W)	PDCALEN	Driver Pull-down Strength Calibration Enable.
29 (R/W)	PUCALEN	Driver Pull-up Strength Calibration Enable.
28 (R/W)	CALSTRT	New Impedance Calibration Enable.
27 (R/W)	CALTYPE	Calibration Type. Short or Long Calibration
26:21 (R/W)	BYPPDDQ	Bypass Code for DQ,DQS,CLK,DM Pull-down Driver Code.

Table 8-39: DMC_DDR_ZQ_CTL2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20:15 (R/W)	BYPPUDQ	Bypass Code for DQ,DQS,CLK,DM Pull-up Code.
14:9 (R/W)	BYPPDADD	Address/Command Bypass Pull-down Code.
8:3 (R/W)	BYPPUADD	Address/Command Bypass Pull-up Codes. Bypass pull-up codes for address and command pads
2 (R/W)	BYPIMRD	Bypass Termination Code Enable.
1 (R/W)	BYPIMDQ	DQ/DQS/CLK/DM Bypass Driver Code Enable.
0 (R/W)	BYPIPAD	Address/Command Bypass Code Enable. Bypass code enable for address and command pads.

9 One-Time Programmable Memory Controller (OTPC)

This chapter describes the operation of the OTP controller. The OTP module is a complete system integrating an OTP memory core with a programming controller, charge pump, and voltage regulator. A built-in Hamming Code Error Correction (ECC), and a fully implemented double-redundant program or read scheme protect the OTP data.

OTP memory access is through the [OTP API Overview](#) provided by the ROM.

CAUTION: OTP memory does not support burst transfers, which are required to support cache line fills. As such, OTP memory should not be made cacheable. If it is, the OTP controller returns an error when a read access is attempted.

OTPC Features

The OTP memory and controller have the following features:

- Built-in redundant read mode
- Built-in integrated power supply
- Built-in Hamming Code Error Correction (ECC)
- Full word serial (single bit at a time) programming with internal VPP

OTPC Functional Description

ADSP-2159x OTPC Register List

The One-Time-Programmable Memory controller (OTPC) supports programming the OTP memory. A set of registers governs OTPC operations. For more information on OTPC functionality, see the OTPC register descriptions.

Table 9-1: ADSP-2159x OTPC Register List

Name	Description
OTPC_SECU_STATE	OTP Security State Register

Table 9-1: ADSP-2159x OTPC Register List (Continued)

Name	Description
OTPC_STAT	OTP Status Register

ADSP-2159x OTPC Interrupt List

Table 9-2: ADSP-2159x OTPC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
6	OTPC0_ERR	OTPC0 Dual-bit Error	Level	

Error Correction

The OTP memory features a Hamming error correction implementation. Signal bit errors are automatically corrected, and dual-bit errors are detected. Refer to [OTPC Interrupt Signals](#).

ECC is always enabled. ECC applies to each 16-bit segment. Because of this functionality, each 16-bit location can only be written to once. Writing to a 16-bit location a second time results in unexpected behavior.

OTPC Event Control

The following sections provide information on OTP events and error management.

OTPC Interrupt Signals

When making 32-bit accesses to OTP memory, a double-bit error in any 16-bit segment triggers the OTPC_INT interrupt. The OTPC also has the OTPC dual bit error (OTPC0_ERR) with the SEA ID of 6. See the [System Event Controller \(SEC\)](#) and [Generic Interrupt Controller \(GIC\)](#) chapter for more information.

OTPC Status and Error Signals

The OTP controller does not produce error signals.

OTP API Overview

The ROM provides a set of functions to facilitate OTP field access. The OTP memory is broken up into a set of specialized fields that are described in this section. The API removes the requirement of understanding the details of the layout or OTP access procedures.

All OTP accesses are made through the provided API.

OTP Programming

The OTP programming API provides a simple access, abstracting particulars of the OTP controller.

Any fields that contain zero or null pointers are skipped.

All addresses are assumed to be byte addresses unless otherwise noted.

A list of APIs follows:

<code>bool adi_rom_otp_pgm(otp_data* data);</code>	OTP Program
<code>bool adi_rom_lock();</code>	Lock API

OTP Program

Program OTP memory using a struct containing the following predefined data fields.

Name	OTP Program	-
PP Define	FUNC_ROM_OTPPGM	
Prototype	<code>bool adi_rom_otp_pgm(otp_data* data);</code>	-
Argument	data	struct containing data to program OTP with
Return Value	bool	true for programming success
Stack Requirements	valid stack	-

```
bool res = adi_rom_otp_pgm(data);
```

The following type of struct is available for programming. Refer to the ROM header file for the exact definition

```
typedef struct {
    void      *reserved2;
    uint32_t  ospi_read_data_capture:16;
    uint32_t  (*huk)[ROM_OTP_SZ_huk];
    uint32_t  (*pvt_128key0)[ROM_OTP_SZ_pvt_128key0];
    uint32_t  (*pvt_128key1)[ROM_OTP_SZ_pvt_128key1];
    uint32_t  (*pvt_128key2)[ROM_OTP_SZ_pvt_128key2];
    uint32_t  (*pvt_128key3)[ROM_OTP_SZ_pvt_128key3];
    uint32_t  (*ek)[ROM_OTP_SZ_ek];
    uint32_t  (*secure_emu_key)[ROM_OTP_SZ_secure_emu_key];
    uint32_t  emu_key_disable:16;
    uint32_t  (*public_key0)[ROM_OTP_SZ_public_key0];
    uint32_t  (*public_key1)[ROM_OTP_SZ_public_key1];
    uint32_t  (*boot_info)[ROM_OTP_SZ_boot_info];
    uint8_t   antiroll_nv_cntr;
    uint32_t  (*gpl)[ROM_OTP_SZ_gpl];
    uint32_t  bootModeDisable:8;
    uint32_t  (*preboot_ddr_cfg)[ROM_OTP_SZ_preboot_ddr_cfg];
}
```

```
uint32_t (*stageID) [ROM_OTP_SZ_stageID];
} otp_data;
```

NOTE: Make OTP memory a non-cacheable region if the core needs access to it.

OTP Reading

This API provides a unified source for retrieving OTP data fields.

All addresses are assumed to be byte addresses, unless otherwise noted.

A list of APIs follow:

<code>bool adi_rom_otp_get(OTPCMD cmd, uint32_t* data);</code>	OTP Get Field
---	-------------------------------

OTP Get Field

Retrieves indicated data from OTP memory.

Name	OTP Get Field	
Prototype	<code>bool adi_rom_otp_get(OTPCMD cmd, uint32_t* data);</code>	
Argument	cmd	Indicates what data to fetch, based on theOTPCMD enum.
Argument	data	memory location to write the data to
Return Value	bool	true for a successful read
Stack Requirements	valid stack	

```
bool res = adi_rom_otp_get(otpcmd_info,data);
```

The data specified by the OTPCMD enum parameter is fetched from OTP memory and placed in the location specified by data. The OTPCMD enum contains entries for each field defined in OTP memory, for the most current list, refer to the OTP header file.

An example of the enum style follows:

```
typedef enum {
    otpcmd_reserved0 = 0,
    otpcmd_reserved2,
    otpcmd_ospi_read_data_capture, /*!< OSPI Read Data Capture Register */
    otpcmd_huk, /*!< Hardware Unique Key */
    otpcmd_pvt_128key0, /*!< Customer Privatekey 0 128bits */
    otpcmd_pvt_128key1, /*!< Customer Privatekey 1 128bits */
    otpcmd_pvt_128key2, /*!< Customer Privatekey 2 128bits */
    otpcmd_pvt_128key3, /*!< Customer Privatekey 3 128bits */
    otpcmd_ek, /*!< Endorsement Key */
```



```
otpcmd_secure_emu_key,      /*!< Secure Emulation Key */
```

OTP Counters

The OTPC module implements a counter API to allow easy reading or writing of the counter without dealing with the complexities of rewriting OTP memory sections that are ECC protected.

The OTPC module provides two functional APIs for counters. These APIs are not extra; the module uses the same `get` and `pgm` APIs. The APIs are functionally unique in the way that they set and retrieve data as counters in OTP memory.

The API uses a different method to count bits because each bit in OTP memory can only be set =1 once, and the ECC protects each 16-bit unit. This functionality essentially means that each 16-bit unit can only be written to once. Therefore, a counter that can count 0–31 requires 32×16 bits of memory.

The API receives and returns the value of the counter as a `uint8_t` binary number. Writing a value less than the current value of the counter or greater than the maximum value results in an error.

To implement this functionality, the driver counts by shifting 1's from the left, treating each block as 1 bit. A three-bit counter is encoded as follows.

bit 2	bit 1	bit 0	Value
0000	0000	0000	0
0001	0000	0000	1
0001	0001	0000	2
0001	0001	0001	3

Lock API

This API locks the device.

Name	Lock API	-
PP Define	<code>FUNC_ROM_LOCK</code>	-
Prototype	<code>bool adi_rom_lock();</code>	-
Return Value	<code>bool</code>	true for success
Stack Requirements	valid stack	-

```
bool res = adi_rom_lock();
```

Calling this function locks the device, making it a secure. Once locked, the `OTPC_SECU_STATE` register indicates that the part is locked, and access is limited. For more information, refer to the security documentation regarding a locked device.

NOTE: Locked Status. The `OTPC_SECU_STATE` register is updated only after the part is rebooted. After calling the lock function, the register still indicates that the part is open.

ADSP-2159x OTPC Register Descriptions

OTP Memory Controller (OTPC) contains the following registers.

Table 9-3: ADSP-2159x OTPC Register List

Name	Description
<code>OTPC_SECU_STATE</code>	OTP Security State Register
<code>OTPC_STAT</code>	OTP Status Register

OTP Security State Register

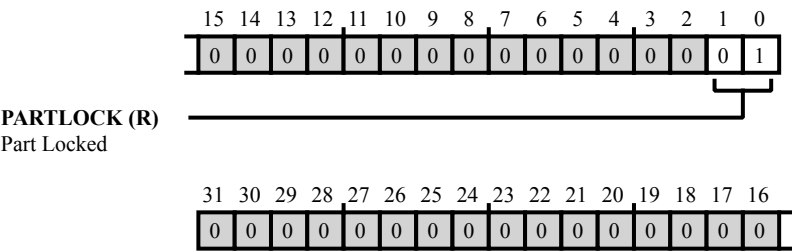


Figure 9-1: OTPC_SECU_STATE Register Diagram

Table 9-4: OTPC_SECU_STATE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/NW)	PARTLOCK	Part Locked. The <code>OTPC_SECU_STATE.PARTLOCK</code> bit indicates a locked part.
		0 OPEN part
		1 Locked part
		2 Reserved

OTPC Status Register

The `OTPC_STAT` register bits indicate errors and flag status and control the protection bits.

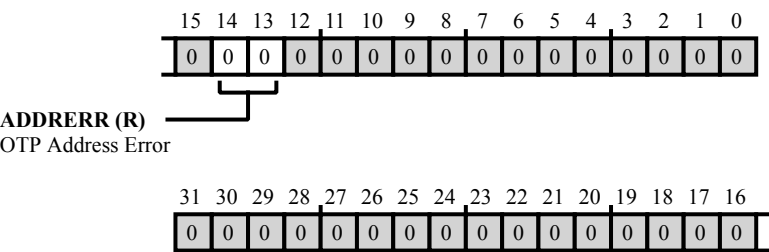


Figure 9-2: `OTPC_STAT` Register Diagram

Table 9-5: `OTPC_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14:13 (R/NW)	ADDRERR	OTP Address Error. The <code>OTPC_STAT.ADDRERR</code> bit field indicates errors which occur when the OTP programming address is out of range or tries to access protected space.
		0 No error - proper OTP address
		1 OTP address out of range
		2 8-bit OTP address
		3 Protected OTP address

10 System Memory Protection Unit (SMPU)

The SMPU provides a flexible way of protecting memory regions against read or write access from any or all masters in the system. In addition, it can guard against memory access depending on security privileges of the system master.

SMPU Features

The system memory protection unit has the following features.

- After reset, the default state of the system is fully open. The SMPUs admit any access to memory spaces by any master.
- Each SMPU instance can be configured to monitor multiple regions. Each can be individually enabled.
- Each region can be configured with its own protection settings.
- Provides general read or write protection.
- Read and write transactions are restricted or allowed depending on the transaction ID.

On the ADSP-2156x processor, four SMPU instances are available to protect the L2, external memory (DMC) interface.

Table 10-1: SMPU Instances

Module	SMPU Instance
Core_L2	2
DMA_L2	3
DMC0	9
SPI Flash Address Space	11

All the SMPU instances can be configured up to eight regions.

- Up to eight outstanding read or write transactions supported on SMPU instances for Core_L2 and DMC0. Up to four outstanding read or write transactions supported on the SMPU instances for DMA_L2 and SPI flash address space.

SMPU Functional Description

The following sections provide details on the function of the SMPU module. If the region security settings allow transactions to go through, the ID in the ID-based region protection settings can still filter the transactions.

For the memory that an SMPU protects, programs can configure region-based settings with the `SMPU_RCTL[n]` registers. (There can be multiple SMPUs in a system.) The `SMPU_RCTL[n]` registers define the ID-based protection for memory regions.

If the target address does not reside in any configured memory region, the transaction permission resorts back to the global configuration setting.

Protection Units

Each SMPU provides two protection units, A and B for ID-based matching in the region-based memory protection. This feature provides a degree of flexibility for the user to match against multiple IDs.

Instruction Fetches

When the core executes instructions from memory, this operation is also considered a memory transaction. If the SMPU is configured to protect a memory region from read accesses that contain instructions, the core cannot fetch and execute these instructions.

Using Cache

When the processor uses both cache and the SMPU, there are a few issues to be aware of. If the SMPU is configured to protect a memory region from write accesses, instruction fetches from a core are still possible since instructions are not updated and replaced during run time.

NOTE: The debugger typically replaces an instruction with a breakpoint instruction for software breakpoints. If a memory region is protected against write accesses, software breakpoints are not possible unless the SMPU is configured with the appropriate system master ID of the debugger. The configuration allows it to perform a write-access.

In the case where memory is used for data, a read access or cache fill is not possible if the memory is blocked from read accesses. If read accesses are allowed but write accesses are disallowed, then there is an issue with coherency. The cache is filled but when the cache is updated and must be written back to the SMPU protected memory, the write-access is blocked.

In general, exercise caution when using both the SMPU and cache.

Speculative Reads

If speculative reads are enabled (`SMPU_CTL.RSDIS = 0`), the SMPU forwards the read transaction directly to the memory before checking the protection setting corresponding to the addressed memory region. This functionality saves one clock cycle in the clock domain of the SMPU. The SMPU checks the protection setting while the read

transaction occurs with the memory. If the protection setting dictates that the target memory address is blocked, the SMPU blocks the read to the master.

If speculative reads are disabled (`SMPU_CTL.RSDIS = 1`), the SMPU checks the protection settings first and forwards the transaction to memory only if it passes the configured protection settings. This functionality incurs a one-cycle latency per read.

NOTE: Reads affect certain memory operations such as automatic clearing of the memory (that is, FIFOs). When the SMPU protects this type of memory, disable read speculation since the blocking can occur without the read transaction reaching the target memory.

ADSP-2159x SMPU Register List

The System Memory Protection Unit (SMPU) provides selective protection of the processor's memory resources. The SMPU includes a set of processor events that can be monitored during program execution. A set of registers governs SMPU operations. For more information on SMPU functionality, see the SMPU register descriptions.

Table 10-2: ADSP-2159x SMPU Register List

Name	Description
<code>SMPU_BADDR</code>	Bus Error Address Register
<code>SMPU_BDTLS</code>	Bus Error Details Register
<code>SMPU_CTL</code>	SMPU Control Register
<code>SMPU_EXACADD[n]</code>	Exclusive Access IDn Address
<code>SMPU_EXACSTAT[n]</code>	Exclusive Access Status
<code>SMPU_IADDR</code>	Interrupt Address Register
<code>SMPU_IDTLS</code>	Interrupt Details Register
<code>SMPU_RADDR[n]</code>	Region n Address Register
<code>SMPU_RCTL[n]</code>	Region n Control Register
<code>SMPU_REVID</code>	SMPU Revision ID Register
<code>SMPU_RIDA[n]</code>	Region n ID A Register
<code>SMPU_RIDB[n]</code>	Region n ID B Register
<code>SMPU_RIDMSKA[n]</code>	Region n ID Mask A Register
<code>SMPU_RIDMSKB[n]</code>	Region n ID Mask B Register
<code>SMPU_SECURECTL</code>	SMPU Control Secure Accesses Register
<code>SMPU_SECURERCTL[n]</code>	Region n Control Secure Accesses Register
<code>SMPU_STAT</code>	SMPU Status Register

SMPU Interrupts

The SMPU has one interrupt with the SEC ID = 217. See the [System Event Controller \(SEC\) and Generic Interrupt Controller \(GIC\)](#) chapter for complete information on interrupt generation and use.

Memory Writes

A write transaction to address n is prevented when the following is true.

Address n is in memory region m and memory region m is write-protected ($\text{SMPU_RCTL}[n].\text{WPROTEN} = 1$) and ID is not a match. (See [ID Comparison](#)). The block occurs because the memory region is configured for write-protection and the ID comparison does not result in a match. If an ID comparison results in a match, the write transaction is allowed through.

Memory Reads

A read transaction from address n is prevented when the following is true:

- Address n is in memory region and memory region m is read-protected ($\text{SMPU_RCTL}[n].\text{RPROTEN} = 1$) and
- ID is not a match

The block occurs because the memory region is configured for read-protection and the ID comparison does not result in a match. If the ID comparison results in a match, the read transaction is permitted. (See [ID Comparison](#)).

ID Comparison

ID comparison automatically occurs during region-based memory protection. ID matches allow the transaction to bypass the configured memory protection for that region. The following sections describe the calculation of a write ID match and read ID match.

Write Transaction

The state of the following values determines the ID value that is compared with the ID of an incoming write transaction:

- The $\text{SMPU_RCTL}[n].\text{WIDCINV}$ bit
- The $\text{SMPU_RIDA}[n].\text{ID}$ and $\text{SMPU_RIDB}[n].\text{ID}$ bit fields
- The $\text{SMPU_RIDMSKA}[n].\text{MSK}$ and $\text{SMPU_RIDMSKB}[n].\text{MSK}$ bit fields

Write IDA match = $((\text{ID of incoming write transaction AND } \text{SMPU_RIDMSKA}[n].\text{MSK}) == (\text{SMPU_RIDA}[n].\text{ID AND } \text{SMPU_RIDMSKA}[n].\text{MSK}))$

Write IDB match = $((\text{ID of incoming write transaction AND } \text{SMPU_RIDMSKB}[n].\text{MSK}) == (\text{SMPU_RIDB}[n].\text{ID AND } \text{SMPU_RIDMSKB}[n].\text{MSK}))$

Write ID match = $(\text{Write IDA match OR Write IDB match}) \text{ XOR } \text{SMPU_RCTL}[n].\text{WIDCINV}$ bit

Read Transaction

The state of the following values determines the ID value that is compared with the ID of an incoming read transaction:

- The `SMPU_RCTL[n].RIDCINV` bit
- The `SMPU_RIDA[n].ID` and `SMPU_RIDB[n].ID` bit fields
- The `SMPU_RIDMSKA[n].MSK` and `SMPU_RIDMSKB[n].MSK` bit fields

Read IDA match = ((ID of incoming read transaction AND `SMPU_RIDMSKA[n].MSK`) == (`SMPU_RIDA[n].ID` AND `SMPU_RIDMSKA[n].MSK`))

Read IDB match = ((ID of incoming read transaction AND `SMPU_RIDMSKB[n].MSK`) == (`SMPU_RIDB[n].ID` AND `SMPU_RIDMSKB[n].MSK`))

Read ID match = (Read IDA match OR Read IDB match) XOR `SMPU_RCTL[n].RIDCINV`

In the two cases described above, the incoming transaction (either write or read) ID is AND'ed with the configured mask value in protection unit A. It is then compared to the value of the configured ID value which is also AND'ed with the configured mask value in protection unit A. The mask provides a method to allow a group of IDs to match. This process is also performed for protection unit B. The two outcomes (from A and B) are then OR'ed together.

Depending on the setting of the `SMPU_RCTL[n].RIDCINV` or the `SMPU_RCTL[n].WIDCINV` bits, the ID match comparison is inverted or not. The final result after applying the inversion, `SMPU_RCTL[n].RIDCINV`, or `SMPU_RCTL[n].WIDCINV`, determines whether the transaction bypasses the protection.

Usage

The masks, `SMPU_RIDMSKA[n]` and `SMPU_RIDMSKB[n]`, are AND'ed with both the incoming transaction ID and the configured ID in `SMPU_RIDA[n].ID` and `SMPU_RIDB[n].ID`, respectively. By default the masks are zero. If ID-based region protection is enabled by setting the `SMPU_RCTL[n].WPROTEN` or `SMPU_RCTL[n].RPROTEN` bit fields and the masks are not set, the ID comparison essentially compares zeros. The comparison allows all transactions to bypass (if the region-based security setting is also configured in a way to allow transactions to go through for the region). To have the ID-based region protection to function, the mask registers and ID registers must also be set.

System IDs

The *System Master IDs* table provides the IDs for the system masters. An x means that the bit can be a 0 or a 1. There are multiple IDs associated with that particular system master.

Table 10-3: System Master IDs

ASIB Name	IID	SIID	ID
SP0A	0	0	13'b0000x00000000

Table 10-3: System Master IDs (Continued)

ASIB Name	IID	SIID	ID
SP0B	1	0	13'b0000x00010000
SP1A	2	0	13'b0000x00100000
SP1B	3	0	13'b0000x00110000
SP2A	4	0	13'b0000x01000000
SP2B	5	0	13'b0000x01010000
SP3A	6	0	13'b0000x01100000
SP3B	7	0	13'b0000x01110000
CRC0_CH0	3	1	13'b0000x00110001
CRC0_CH1	2	1	13'b0000x00100001
MLB	4	2	13'b0000001000010
UART0_TX	0	3	13'b0000x00000011
UART0_RX	4	3	13'b0000x01000011
UART2_TX	3	3	13'b0000x00110011
SPI0TX	0	4	13'b0000x00000100
SPI0RX	1	4	13'b0000x00010100
SPI1TX	2	4	13'b0000x00100100
SPI1RX	3	4	13'b0000x00110100
SPI2TX	5	4	13'b0000x01010100
SPI2RX	4	4	13'b0000x01000100
LP0	0	5	13'b0000x00000101
UART1_TX	1	3	13'b0000x00010011
UART1_RX	2	3	13'b0000x00100011
LP1	1	5	13'b0000x00010101
CRYPTO	5	6	13'b0000001010110
SH0_FIR_CH0	0	7	13'b0xxxx00000111
SH0_FIR_CH1	1	7	13'b0xxxx00010111
DLDMA0_CH0	0	6	13'b0000000000110
DLDMA0_CH1	1	6	13'b0000000010110
DLDMA1_CH0	2	6	13'b0000000100110
DLDMA1_CH1	3	6	13'b0000000110110
MSMDMA_CH0	0	8	13'b0000x00001000

Table 10-3: System Master IDs (Continued)

ASIB Name	IID	SIID	ID
MSMDMA_CH1	1	8	13'b0000x00011000
DBG	4	8	13'b0000001001000
ETR	4	6	13'b0000001000110
CRC1_CH0	1	1	13'b0000x00010001
CRC1_CH1	0	1	13'b0000x00000001
UART2_RX	5	3	13'b0000x01010011
SH0_DPORT	0	9	13'b0xxxx00001001
SH0_IPORT	4	7	13'b0000001000111
HSMDMA_CH0	0	10	13'b0000x00001010
HSMDMA_CH1	1	10	13'b0000x00011010
SP4A	0	11	13'b0000x00001011
SP4B	1	11	13'b0000x00011011
SP5A	2	11	13'b0000x00101011
SP5B	3	11	13'b0000x00111011
SP6A	4	11	13'b0000x01001011
SP6B	5	11	13'b0000x01011011
SP7A	6	11	13'b0000x01101011
SP7B	7	11	13'b0000x01111011
SH0_MMR	0	12	13'b0xxxx00001100
UART3_TX	6	3	13'b0000x01100011
UART3_RX	7	3	13'b0000x01110011
SPI3TX	6	4	13'b0000x01100100
SPI3RX	7	4	13'b0000x01110100
CRC2_CH0	1	13	13'b0000x00011101
CRC2_CH1	0	13	13'b0000x00001101
CRC3_CH0	3	13	13'b0000x00111101
CRC3_CH1	2	13	13'b0000x00101101
MSMDMA1_CH0	3	8	13'b0000x00111000
MSMDMA1_CH1	2	8	13'b0000x00101000
HSMDMA1_CH0	2	10	13'b0000x00101010
HSMDMA1_CH1	3	10	13'b0000x00111010

Table 10-3: System Master IDs (Continued)

ASIB Name	IID	SIID	ID
PPI_F0	8	3	13'b0000x10000011
PPI_F1	9	3	13'b0000x10010011
USB0	2	14	13'b00000000101110
SH1_IPORT	5	7	13'b00000001010111
SH1_DPORT	1	9	13'b0xxxx00011001
SH1_MMR	2	12	13'b0xxxx00101100
PL310_M0	2	9	13'bxxxxx00101001
PL310_M1	3	9	13'bxxxxx00111001
GIGE	0	15	13'b0xxxx00001111
EMAC	1	15	13'b0xxxx00011111
SH1_FIR_CH0	2	7	13'b0xxxx00100111
SH1_FIR_CH1	3	7	13'b0xxxx00110111
SH0_IIR_CH0	4	9	13'b0xxxx01001001
SH0_IIR_CH1	5	9	13'b0xxxx01011001
SH1_IIR_CH0	6	9	13'b0xxxx01101001
SH1_IIR_CH1	7	9	13'b0xxxx01111001

Memory Region

Memory regions can start at address 0x00000000 or at any address that is a multiple of its size. The *Supported Memory Region Size and Alignment* table shows the memory region sizes that the processor supports and the alignment of the memory region. (X values are do-not-care).

SMPU supports a maximum of eight regions.

Table 10-4: Supported Memory Region Size and Alignment

Size	SMPU_RCTLn.SIZE	Address	Possible Values for N
4KB	0b00000	0xXXXXX000	-
8KB	0b00001	0xXXXXN000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
16KB	0b00010	0xXXXXN000	0x0, 0x4, 0x8, 0xC
32KB	0b00011	0xXXXXN000	0x0, 0x8
64KB	0b00100	0xXXXX0000	-
128KB	0b00101	0xXXXN0000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE

Table 10-4: Supported Memory Region Size and Alignment (Continued)

Size	SMPU_RCTLn.SIZE	Address	Possible Values for N
256KB	0b00110	0xXXXN0000	0x0, 0x4, 0x8, 0xC
512KB	0b00111	0xXXXN0000	0x0, 0x8
1MB	0b01000	0xXXX00000	-
2MB	0b01001	0xXXN00000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
4MB	0b01010	0xXXN00000	0x0, 0x4, 0x8, 0xC
8MB	0b01011	0xXXN00000	0x0, 0x8
16MB	0b01100	0xXX000000	-
32MB	0b01101	0xXN000000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
64MB	0b01110	0xXN000000	0x0, 0x4, 0x8, 0xC
128MB	0b01111	0xXN000000	0x0, 0x8
256MB	0b10000	0xX0000000	-
512MB	0b10001	0xN0000000	0x0, 0x2, 0x4, 0x8, 0xA, 0xC, 0xE
1GB	0b10010	0xN0000000	0x0, 0x4, 0x8, 0xC
2GB	0b10011	0xN0000000	0x0, 0x8
4GB	0b10100	0x00000000	-

For the case where the region size is selected as 4 GB, the region address must be at address 0x00000000.

NOTE: If a memory region address is not aligned to its size, the memory region start address protected by the SMPU is the configured address with the corresponding least significant bits masked. For example, if the size is configured for 16 KB (SMPU_RCTL[n].SIZE = 0b00010), and the base address is configured for SMPU_RADDR[n].BADDR = 0x00005018, the actual base address used by the SMPU is 0x00004000. When SMPU_RADDR[n].BADDR is read back, the program reads 0x00005000. This functionality is because only bits [11:0] are reserved as 0's. Programs must use care when setting the base address as it is not always the true base address.

SMPU Definitions

To make the best use of the SMPU, it is useful to understand the terms in this section.

Global Protection

Guarding of the entire memory space for the particular SMPU instantiation.

Region-Based Protection

Guarding individual segments of memory inside the memory space for the particular SMPU instantiation.

ID Match

A successful comparison of the ID associated with the incoming transaction and the ID and MASK configured in the SMPU.

SMPU Block Diagram

The *SMPU Top-Level Block Diagram* shows the SMPU block.

As seen in the diagram, the SMPU sits between the memory port (SCB master port) and the SCB fabric (SCB slave port). It acts as a gateway analyzing the transaction requests. It either rejects the transaction request or allows access based on the user-programmed configuration of the SMPU.

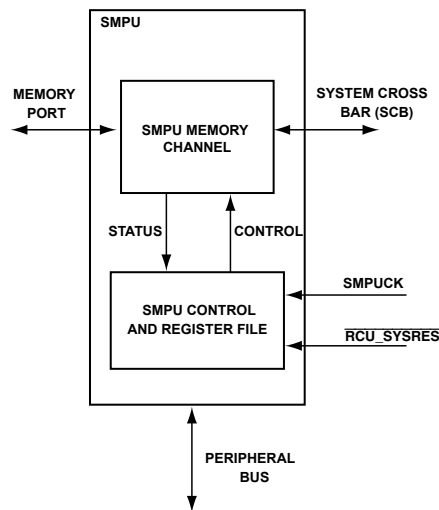


Figure 10-1: SMPU Top-Level Block Diagram

SMPU Architectural Concepts

The following sections provide brief descriptions of the architecture of the SMPU module.

Default Setting

At reset, the default state of the system is fully open. The SMPUs admit any access to memory spaces.

Latency

The SMPU adds latency to all the transactions to the memory except reads when read speculation is enabled (`SMPU_CTL.RSDIS = 0`). In this case, read accesses are always forwarded to the memory and read responses are generated according to the SMPU settings. If read speculation is disabled (`SMPU_CTL.RSDIS = 1`), reads are blocked if they cause a security or protection violation. The SMPU generates the SCB read response that corresponds to a blocked transaction.

If read speculation is enabled, the SMPU adds 1 clock cycle latency to the read transaction. If read speculation is disabled, the SMPU adds 2 clock cycles latency to the read transaction.

SMPU Operating Modes

The SMPU does not have any strict modes of operation. However, it can be configured for region-based protection where a master with a particular ID can be blocked or allowed based on settings in the `SMPU_RCTL[n]` register.

Region-based protection is programmed with registers:

- `SMPU_RCTL[n]`
- `SMPU_RADDR[n]`
- `SMPU_RIDA[n]`
- `SMPU_RIDMSKA[n]`
- `SMPU_RIDB[n]`
- `SMPU_RIDMSKB[n]`

SMPU Interrupt Signals

There is one interrupt signal associated with the SMPU. If interrupts are enabled, the `SMPU_STAT.IRQ` bit is set. The `SMPU_IRQ` signal is asserted when the SMPU detects a memory access violation. The target address triggering the interrupt is found in the `SMPU_IADDR` register. The `SMPU_IDTLS` register provides further details about the cause of the interrupt.

Write errors are prioritized over read errors.

Protection violations (an ID-based violation) can trigger the SMPU interrupt, and can be enabled independently. The protection violation interrupt is enabled by setting the `SMPU_CTL.PINTEN` bit.

The SMPU interrupt is asserted for any of the following conditions:

If a second memory access violation occurs while the `SMPU_STAT.IRQ` bit is set, the `SMPU_STAT.IOVR` (interrupt overrun) bit is set. The `SMPU_IADDR` and the `SMPU_IDTLS` registers are not updated until the `SMPU_STAT.IRQ` bit is cleared. Any information on the subsequent interrupt is lost. Once the `SMPU_STAT.IRQ` bit and the `SMPU_STAT.IOVR` bit are cleared, any new memory access violations can trigger an interrupt and its details can be captured.

NOTE: When a blocked access occurs, the SMPU triggers an interrupt when interrupt generation is enabled. The SMPU can also be configured to generate a bus error that propagates back to the system master. The system master can also trigger an interrupt due to this bus error.

NOTE: On the processor, each SMPU instance has an interrupt. All of the SMPU interrupts are OR'ed and mapped to a single SMPU interrupt on the SEC. While servicing the SMPU interrupt, check all of the `SMPU_STAT` registers to determine which triggered the interrupt. The interrupt service routine clears the `SMPU_STAT.IRQ` bit of the all of the `SMPU_STAT` registers for which the interrupt is triggered.

SMPU Status and Error Signals

If bus errors are enabled (`SMPU_CTL.PBEDIS = 0`), the SMPU generates and returns a bus error to the master initiating the blocked access. This bit also sets the `SMPU_STAT.BERR` bit. The `SMPU_BADDR` and `SMPU_BDTLS` registers can be read to get the address and details of the transaction that caused the SMPU to generate the error.

Write errors are prioritized over read errors.

A bus error status is returned to the system master if:

- an ID-based violation happened and the `SMPU_CTL.PBEDIS` bit = 0

If a second memory access violation occurs while the `SMPU_STAT.BERR` bit is set, the `SMPU_STAT.BEOVR` bit (bus error overrun) is set. The `SMPU_BADDR` and the `SMPU_BDTLS` registers are not updated until the `SMPU_STAT.IRQ` bit is cleared. The information about the transaction that caused the `SMPU_STAT.BEOVR` bit to be set is lost.

NOTE: If both the protection violation interrupt is not enabled (`SMPU_CTL.PINTEN = 0`) and the protection bus error is disabled (`SMPU_CTL.PBEDIS = 1`), the SMPU blocks invalid transactions. However, it does not provide any status or interrupt information indicating that a transaction is blocked.

ADSP-2159x SMPU Register Descriptions

The System Memory Protection Unit (SMPU) contains the following registers.

Table 10-5: ADSP-2159x SMPU Register List

Name	Description
<code>SMPU_BADDR</code>	Bus Error Address Register
<code>SMPU_BDTLS</code>	Bus Error Details Register
<code>SMPU_CTL</code>	SMPU Control Register
<code>SMPU_EXACADD[n]</code>	Exclusive Access IDn Address
<code>SMPU_EXACSTAT[n]</code>	Exclusive Access Status
<code>SMPU_IADDR</code>	Interrupt Address Register
<code>SMPU_IDTLS</code>	Interrupt Details Register
<code>SMPU_RADDR[n]</code>	Region n Address Register
<code>SMPU_RCTL[n]</code>	Region n Control Register
<code>SMPU_REVID</code>	SMPU Revision ID Register
<code>SMPU_RIDA[n]</code>	Region n ID A Register
<code>SMPU_RIDB[n]</code>	Region n ID B Register
<code>SMPU_RIDMSKA[n]</code>	Region n ID Mask A Register

Table 10-5: ADSP-2159x SMPU Register List (Continued)

Name	Description
SMPU_RIDMSKB[n]	Region n ID Mask B Register
SMPU_SECURECTL	SMPU Control Secure Accesses Register
SMPU_SECURERCTL[n]	Region n Control Secure Accesses Register
SMPU_STAT	SMPU Status Register

Bus Error Address Register

Programs read the `SMPU_BADDR` and the `SMPU_BDTLS` registers to determine the cause of a bus error. Write errors are prioritized over read errors.

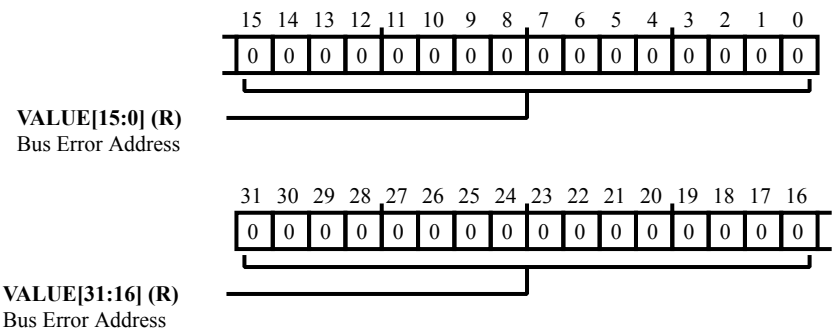


Figure 10-2: `SMPU_BADDR` Register Diagram

Table 10-6: `SMPU_BADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Bus Error Address. The <code>SMPU_BADDR.VALUE</code> bit field contains the address of the bus error.

Bus Error Details Register

The `SMPU_BDTLS` register indicates the ID of the bus error transaction, whether the transaction that caused the last bus error was a read, a write, secure or non-secure.

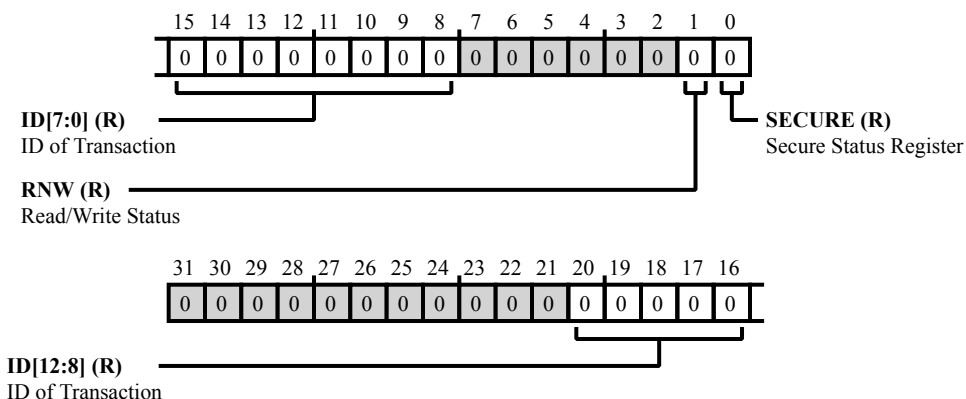


Figure 10-3: SMPU_BDTLS Register Diagram

Table 10-7: SMPU_BDTLS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:8 (R/NW)	ID	ID of Transaction. The <code>SMPU_BDTLS</code> .ID bit field provides the ID of the transaction that caused the bad address error.
1 (R/NW)	RNW	Read/Write Status. The <code>SMPU_BDTLS</code> .RNW bit indicates whether the last transaction that caused the bad address error was a read or write.
		0 Transaction that caused last bus error was a write
		1 Transaction that caused last bus error was a read
0 (R/NW)	SECURE	Secure Status Register. The <code>SMPU_BDTLS</code> .SECURE bit indicates whether the last transaction that caused the bad address error was secure or non-secure.
		0 Transaction that caused last bus error was non-secure
		1 Transaction that caused last bus error was secure

SMPU Control Register

The `SMPU_CTL` register provides access to the locking control, error interrupts and SMPU violations.

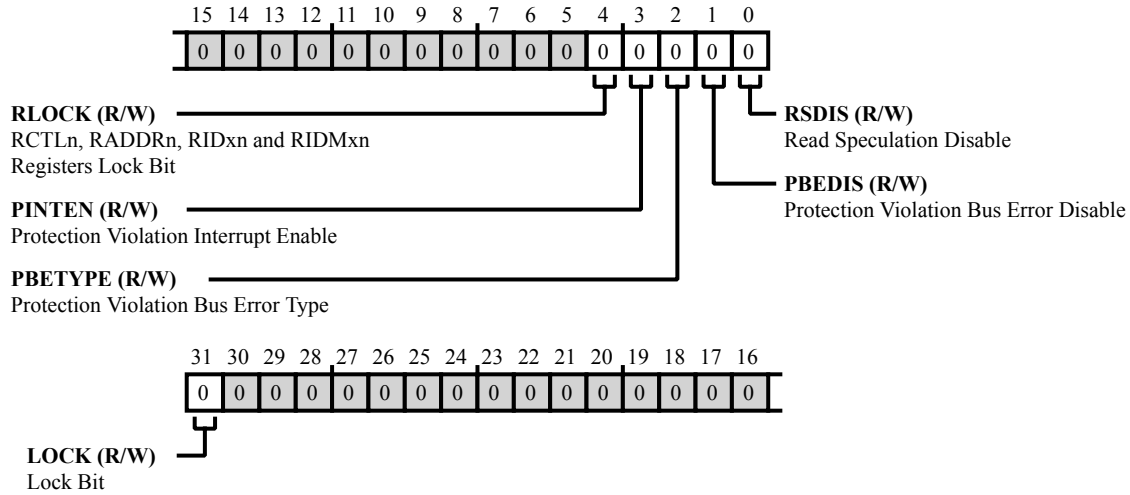


Figure 10-4: SMPU_CTL Register Diagram

Table 10-8: SMPU_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit. When the <code>SMPU_CTL</code> .LOCK bit is set and the global lock signal is asserted from the SPU, the <code>SMPU_CTL</code> register is write-protected. Write-protection is disabled only when the global lock signal becomes deasserted again.
		0 CTL Global Lock Disable. The <code>SMPU_CTL</code> register is not write-protected.
		1 CTL Global Lock Enable. The <code>SMPU_CTL</code> register is write-protected.
4 (R/W)	RLOCK	RCTLn, RADDRn, RIDxn and RIDMxn Registers Lock Bit. When the <code>SMPU_CTL</code> .RLOCK bit is set, all the registers associated with region-based control (<code>SMPU_RCTL[n]</code> , <code>SMPU_RADDR[n]</code> , <code>SMPU_RIDA[n]</code> , <code>SMPU_RIDB[n]</code> , <code>SMPU_RIDMSKA[n]</code> and <code>SMPU_RIDMSKB[n]</code>) are write-protected when the global lock signal is active from the SPU. Write access is allowed again when the global lock signal is deasserted.
		0 Region Registers Write-Protect Enable. All region registers are not write-protected.
		1 Region Registers Write-Protect Disable. All region registers are write-protected.

Table 10-8: SMPU_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	PINTEN	Protection Violation Interrupt Enable. The SMPU_CTL.PINTEN bit controls whether or not an interrupt is generated when a protection violation occurs.
		0 Protection Violation IRQ Disable. The protection violation interrupt is disabled.
		1 Protection Violation IRQ Enable. The protection violation interrupt is enabled.
2 (R/W)	PBETYPE	Protection Violation Bus Error Type. The SMPU_CTL.PBETYPE bit controls whether a protection violation produces a decode error or a slave error.
		0 Decode Error Type. Decode error for transactions that violate the configured protection.
		1 Slave Error Type. Slave Error for transactions which violate the configured protection
1 (R/W)	PBEDIS	Protection Violation Bus Error Disable. If set, the SMPU_CTL.PBEDIS bit blocks protection violations, but does not cause a bus error.
		0 Bus Error Generation Enable. Transactions which violate the configured protection are blocked and cause a bus error.
		1 Bus Error Generation Disable. Transactions which violate the configured protection are blocked but do not cause a bus error.
0 (R/W)	RSDIS	Read Speculation Disable. The SMPU_CTL.RSDIS bit controls whether or not the read addresses are checked before being sent to the slave.
		0 Read Speculation Enable. Read addresses are sent to the slave without checking.
		1 Read Speculation Disable. Read addresses are checked before being sent to the slave.

Exclusive Access IDn Address

The `SMPU_EXACADD[n]` register provides the address ID of an exclusive access.

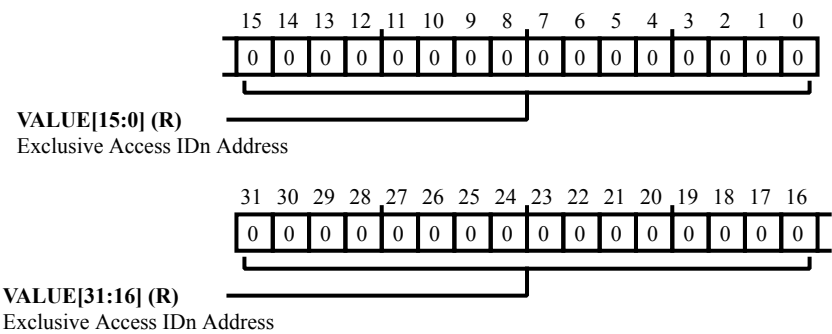


Figure 10-5: `SMPU_EXACADD[n]` Register Diagram

Table 10-9: `SMPU_EXACADD[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Exclusive Access IDn Address.

Exclusive Access Status

The `SMPU_EXACSTAT[n]` register provides the exclusive access ID, read size and read length as well as the indication that the access was valid.

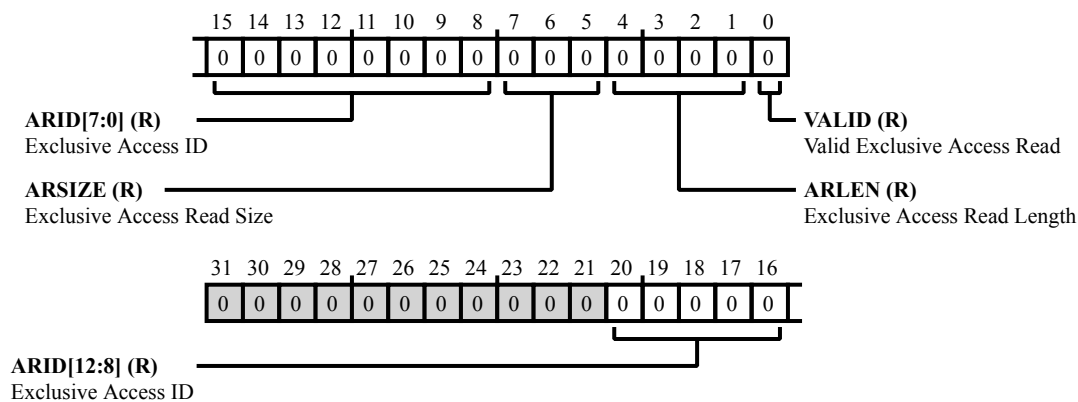


Figure 10-6: SMPU_EXACSTAT[n] Register Diagram

Table 10-10: SMPU_EXACSTAT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:8 (R/NW)	ARID	Exclusive Access ID.
7:5 (R/NW)	ARSIZE	Exclusive Access Read Size.
4:1 (R/NW)	ARLEN	Exclusive Access Read Length.
0 (R/NW)	VALID	Valid Exclusive Access Read.

Interrupt Address Register

The `SMPU_IADDR` register indicates an attempt to make a read or write access to unimplemented addresses or accesses are non-aligned. The SMPU issues a bus error for this condition.

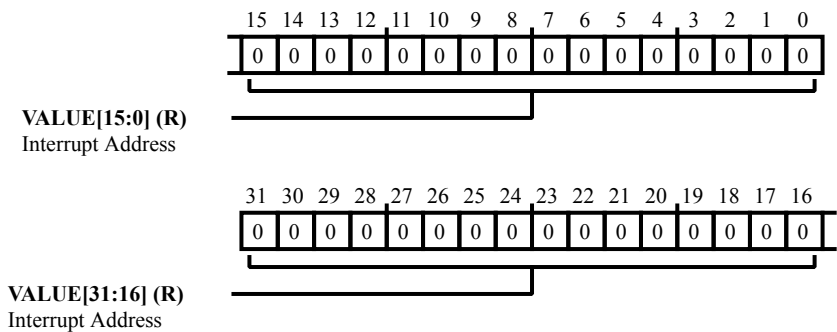


Figure 10-7: SMPU_IADDR Register Diagram

Table 10-11: SMPU_IADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Interrupt Address. The <code>SMPU_IADDR.VALUE</code> bit field is the address where an attempt to access an unimplemented address or a non-aligned access has occurred.

Interrupt Details Register

The `SMPU_IDTLS` register provides the ID of the last signaled interrupt, whether the interrupt was caused by a read or write, and whether the transaction that caused the last signaled interrupt was secure.

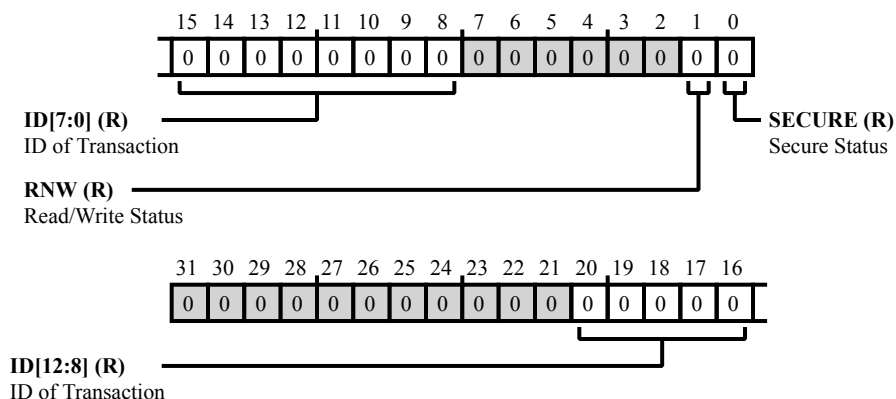


Figure 10-8: SMPU_IDTLS Register Diagram

Table 10-12: SMPU_IDTLS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:8 (R/NW)	ID	ID of Transaction. The <code>SMPU_IDTLS.ID</code> bit field provides the ID of the transaction that caused the interrupt.
1 (R/NW)	RNW	Read/Write Status. The <code>SMPU_IDTLS.RNW</code> bit indicates whether the last transaction that caused the interrupt was a read or write.
		0 Transaction that caused last signaled interrupt was a write
		1 Transaction that caused last signaled interrupt was a read
0 (R/NW)	SECURE	Secure Status. The <code>SMPU_IDTLS.SECURE</code> bit indicates whether the last transaction that caused the interrupt was secure or non-secure.
		0 Transaction that caused last signaled interrupt was non-secure
		1 Transaction that caused last signaled interrupt was secure

Region n Address Register

The `SMPU_RADDR[n]` register is used to define the base address for a memory region to be protected.

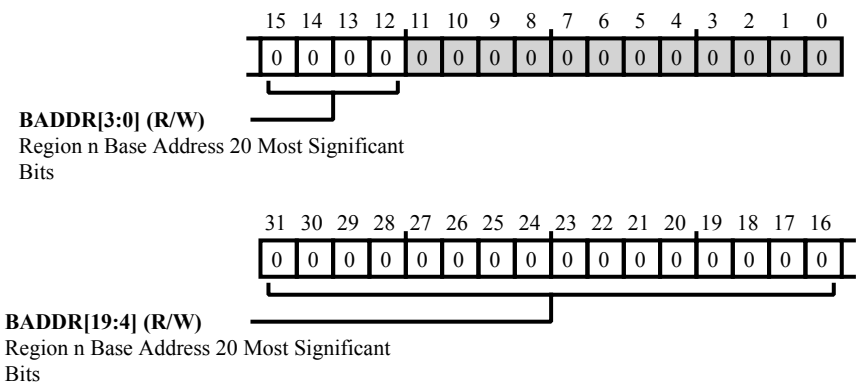


Figure 10-9: SMPU_RADDR[n] Register Diagram

Table 10-13: SMPU_RADDR[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:12 (R/W)	BADDR	Region n Base Address 20 Most Significant Bits. The <code>SMPU_RADDR[n].BADDR</code> bit field defines the base address for a memory region to be protected.

Region n Control Register

The `SMPU_RCTL[n]` register is used to define the level of protection for a region of memory. The protection of a region is controlled and defined by this register and the `SMPU_RADDR[n]`, `SMPU_RIDA[n]`, `SMPU_RIDB[n]`, `SMPU_RIDMSKA[n]`, and `SMPU_RIDMSKB[n]` registers.

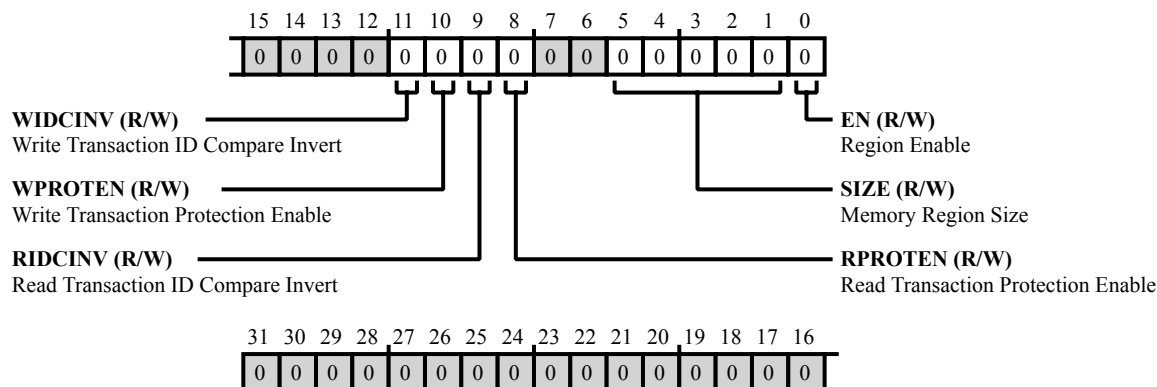


Figure 10-10: `SMPU_RCTL[n]` Register Diagram

Table 10-14: `SMPU_RCTL[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	WIDCINV	Write Transaction ID Compare Invert. The <code>SMPU_RCTL[n].WIDCINV</code> bit inverts the write ID match result.
		0 Write transaction ID comparison result not inverted
		1 Write transaction ID comparison result inverted
10 (R/W)	WPROTEN	Write Transaction Protection Enable. The <code>SMPU_RCTL[n].WPROTEN</code> bit enables protection against ID-based write transactions for the memory region.
		0 Write transaction ID-based protection disabled
		1 Write transaction ID-based protection enabled
9 (R/W)	RIDCINV	Read Transaction ID Compare Invert. When the <code>SMPU_RCTL[n].RIDCINV</code> bit is set, the read ID match result is inverted.
		0 Read transaction ID comparison result not inverted
		1 Read transaction ID comparison result inverted

Table 10-14: SMPU_RCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	RPROTEN	Read Transaction Protection Enable. The SMPU_RCTL[n] . RPROTEN bit enable bit to turn on protection against ID-based read transactions for the memory region.
		0 Read transaction ID-based protection disabled
		1 Read transaction ID-based protection enabled
5:1 (R/W)	SIZE	Memory Region Size. The SMPU_RCTL[n] . SIZE bit defines the size of the memory region to be protected.
		0 4 KB
		1 8 KB
		2 16 KB
		3 32 KB
		4 64 KB
		5 128 KB
		6 256 KB
		7 512 KB
		8 1 MB
		9 2 MB
		10 4 MB
		11 8 MB
		12 16 MB
		13 32 MB
		14 64 MB
		15 128 MB
		16 256 MB
		17 512 MB
		18 1 GB
		19 2 GB
		20 4 GB
		21-31 Reserved

Table 10-14: SMPU_RCTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Region Enable. The SMPU_RCTL[n].EN bit enables the protection of a region.
		0 Disabled
		1 Enabled

SMPU Revision ID Register

The `SMPU_REVID` register provides the major and minor revision numbers of this module.

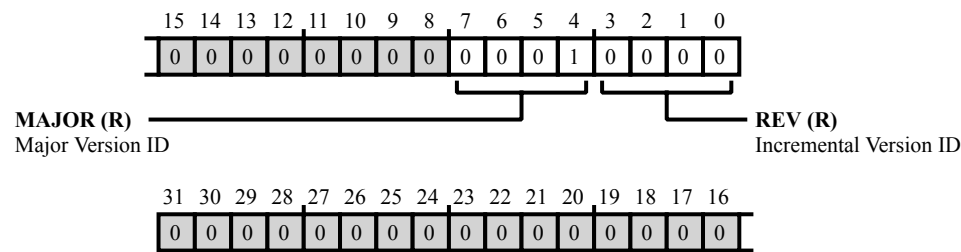


Figure 10-11: SMPU_REVID Register Diagram

Table 10-15: SMPU_REVID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	MAJOR	Major Version ID.
3:0 (R/NW)	REV	Incremental Version ID.

Region n ID A Register

The `SMPU_RIDA[n]` register is used for ID comparison 'A'. This comparison is performed after a mask is applied to both the transaction ID (from either the read or write IDs) and the register value. An ID match means that the ID is the exception to the rule and the read or write is allowed even if the region is read or write-protected. For more detail, refer to the ID Comparison section.

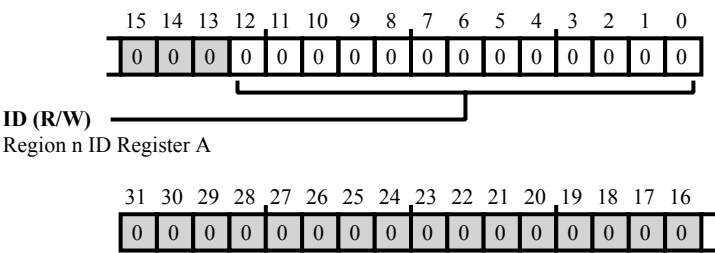


Figure 10-12: `SMPU_RIDA[n]` Register Diagram

Table 10-16: `SMPU_RIDA[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12:0 (R/W)	ID	Region n ID Register A. The <code>SMPU_RIDA[n] . ID</code> bit field, combined with the mask provides the means to bypass the configured memory protection for a region.

Region n ID B Register

The `SMPU_RIDB[n]` register is used for ID comparison 'B'. This comparison is performed after a mask is applied to both the transaction ID (from either the read or write IDs) and the register value. An ID match means that the ID is the exception to the rule and the read or write is allowed even if the region is read or write-protected. For more details, refer to the ID Comparison section.

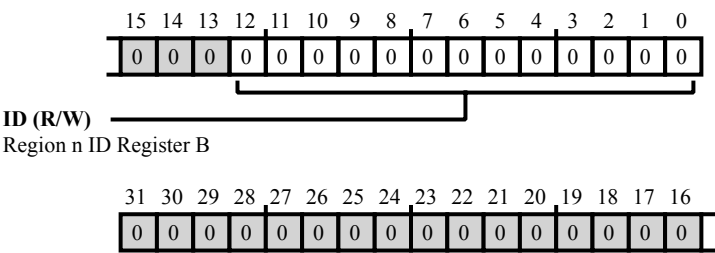


Figure 10-13: `SMPU_RIDB[n]` Register Diagram

Table 10-17: `SMPU_RIDB[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12:0 (R/W)	ID	Region n ID Register B. The <code>SMPU_RIDB[n] . ID</code> bit field, combined with the mask provides the means to bypass the configured memory protection for a region.

Region n ID Mask A Register

The `SMPU_RIDMSKA[n]` register is used for ID comparison 'A'. The mask allows or disallows certain IDs from affecting the final result of the ID match. For more details, refer to the ID Comparison section.

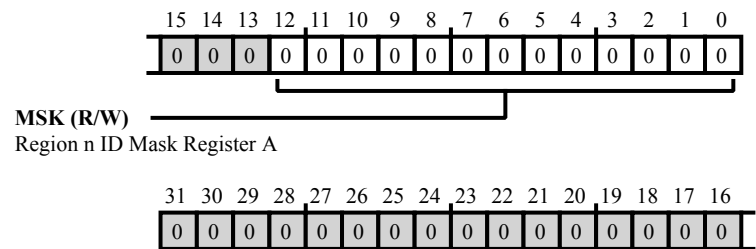


Figure 10-14: `SMPU_RIDMSKA[n]` Register Diagram

Table 10-18: `SMPU_RIDMSKA[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12:0 (R/W)	MSK	Region n ID Mask Register A. The <code>SMPU_RIDMSKA[n].MSK</code> bit field, combined with the incoming transaction, provides the means to bypass the configured memory protection for a region.

Region n ID Mask B Register

The `SMPU_RIDMSKB[n]` register is used for ID comparison 'B'. The mask allows or disallows certain IDs from affecting the final result of the ID match. For more details, refer to the ID Comparison section.

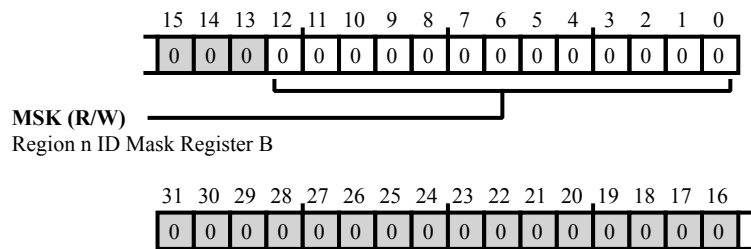


Figure 10-15: `SMPU_RIDMSKB[n]` Register Diagram

Table 10-19: `SMPU_RIDMSKB[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12:0 (R/W)	MSK	Region n ID Mask Register B. The <code>SMPU_RIDMSKB[n].MSK</code> bit field, combined with the incoming transaction provides the means to bypass the configured memory protection for a region.

SMPU Control Secure Accesses Register

The `SMPU_SECURECTL` register provides the bits required to set up the security settings for the processor. These settings includes error generation and read/write security.

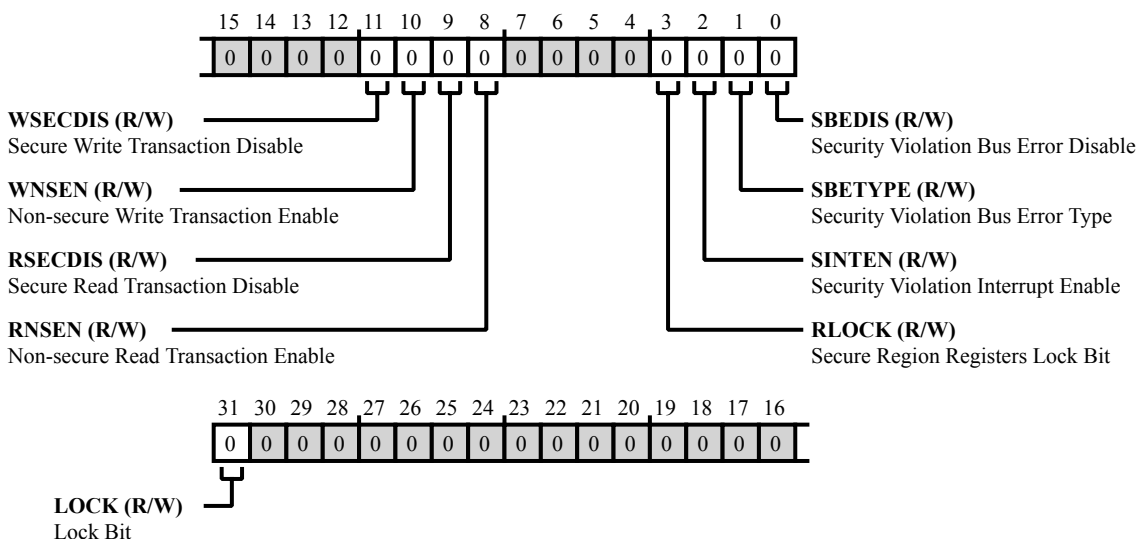


Figure 10-16: SMPU_SECURECTL Register Diagram

Table 10-20: SMPU_SECURECTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock Bit. When the <code>SMPU_SECURECTL.LOCK</code> bit is set and the global lock signal is asserted from the SPU, the <code>SMPU_SECURECTL</code> register is write-protected. Write-protection is disabled only when the global lock signal becomes deasserted again.
		0 <code>SMPU_SECURECTL</code> is not write-protected
		1 <code>SMPU_SECURECTL</code> is write-protected
11 (R/W)	WSECDIS	Secure Write Transaction Disable. The <code>SMPU_SECURECTL.WSECDIS</code> bit disables secure write transactions.
		0 Enable secure write transactions
		1 Disable secure write transactions
10 (R/W)	WNSEN	Non-secure Write Transaction Enable. The <code>SMPU_SECURECTL.WNSEN</code> bit enables non-secure write transactions.
		0 Disable non-secure writes
		1 Enable non-secure writes

Table 10-20: SMPU_SECURECTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	RSECDIS	Secure Read Transaction Disable. The SMPU_SECURECTL.RSECDIS bit disables secure read transactions.
		0 Enable secure read transactions
		1 Disable secure read transactions
8 (R/W)	RNSEN	Non-secure Read Transaction Enable. The SMPU_SECURECTL.RNSEN bit enables non-secure read transactions.
		0 Disable non-secure read transactions
		1 Enable non-secure read transactions
3 (R/W)	RLOCK	Secure Region Registers Lock Bit. When the SMPU_SECURECTL.RLOCK bit is set, the secure region control registers, SMPU_SECURECTL[n] , are write-protected when the global lock signal is active from the SPU. When the global lock signal is deasserted, write access is allowed again.
		0 Disable write-protection on secure region registers
		1 Enable write-protection on secure region registers
2 (R/W)	SINTEN	Security Violation Interrupt Enable. The SMPU_SECURECTL.SINTEN bit enables interrupt generation when a security violation occurs.
		0 Disable security settings violation interrupt
		1 Enable security settings violation interrupt
1 (R/W)	SBETYPE	Security Violation Bus Error Type. The SMPU_SECURECTL.SBETYPE bit controls whether a decode error or a slave error is returned when a security violation occurs.
		0 Return a decode error error which violates the security settings
		1 Return a slave error which violates the security settings
0 (R/W)	SBEDIS	Security Violation Bus Error Disable. The SMPU_SECURECTL.SBEDIS bit controls whether or not a bus error is caused when a security violation occurs.
		0 Enable bus error
		1 Disable bus error

Region n Control Secure Accesses Register

The `SMPU_SECURERCTL[n]` register contains bits that configure read/write security for a specific region.

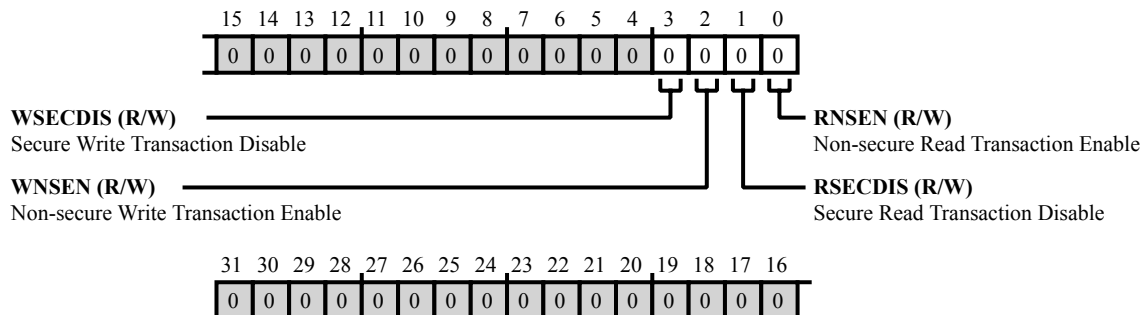


Figure 10-17: `SMPU_SECURERCTL[n]` Register Diagram

Table 10-21: `SMPU_SECURERCTL[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	WSECDIS	Secure Write Transaction Disable. The <code>SMPU_SECURERCTL[n].WSECDIS</code> bit disables secure write transactions for the memory region.
		0 Enable secure write transactions to this region
		1 Disable secure write transactions to this region
2 (R/W)	WNSSEN	Non-secure Write Transaction Enable. This <code>SMPU_SECURERCTL[n].WNSSEN</code> bit enables non-secure write transactions for the memory region.
		0 Disable non-secure write transactions to this region
		1 Enable non-secure write transactions to this region
1 (R/W)	RSECDIS	Secure Read Transaction Disable. The <code>SMPU_SECURERCTL[n].RSECDIS</code> bit disables secure read transactions for the memory region.
		0 Enable secure read transactions to this region
		1 Disable secure read transactions to this region
0 (R/W)	RNSSEN	Non-secure Read Transaction Enable. The <code>SMPU_SECURERCTL[n].RNSSEN</code> bit enables non-secure read transactions for the memory region.
		0 Disable non-secure read transactions to this region
		1 Enable non-secure read transactions to this region

SMPU Status Register

The `SMPU_STAT` register provides the state of the SMPU and indicates various errors. All bits in this register are write 1 to clear.

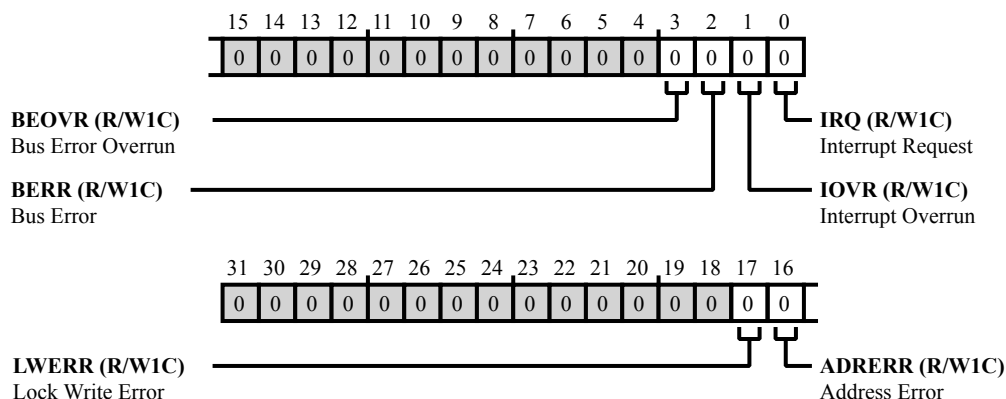


Figure 10-18: SMPU_STAT Register Diagram

Table 10-22: SMPU_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W1C)	LWERR	Lock Write Error. The <code>SMPU_STAT.LWERR</code> bit is set when <code>SMPU_CTL.LOCK</code> bit =1, the global lock signal is asserted from the SPU and a read or write attempt was made to the <code>SMPU_CTL</code> MMR.
		0 No Lock Write Error
		1 Lock Write Error
16 (R/W1C)	ADRERR	Address Error. The <code>SMPU_STAT.ADRERR</code> bit is set when the SMPU MMR is accessed as an un-aligned address, or when a read-only MMR is written to.
		0 No Address Error
		1 Address Error
3 (R/W1C)	BEOVR	Bus Error Overrun. The <code>SMPU_STAT.BEOVR</code> bit indicates that another bus error had occurred. Any new information about the most recent violation which caused the bus error is not captured.
		0 No Bus Error overrun
		1 Bus Error overrun has occurred

Table 10-22: SMPU_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W1C)	BERR	Bus Error. This SMPU_STAT . BERR bit indicates if a bus error was generated.
		0 No Bus Error since this bit has been cleared
		1 Bus Error has been generated
1 (R/W1C)	IOVR	Interrupt Overrun. The SMPU_STAT . IOVR bit indicates if another violation occurred while the previous violation interrupt was not finished being serviced. Information about the most recent violation is then not captured.
		0 No Interrupt overrun
		1 Interrupt overrun has occurred
0 (R/W1C)	IRQ	Interrupt Request. The SMPU_STAT . IRQ bit provides an indication that an interrupt has been generated.
		0 No Interrupt since this bit has been cleared
		1 Interrupt has been generated

11 General-Purpose Ports (PORT)

This section describes general-purpose ports, pin multiplexing, general-purpose input/output (GPIO) functionality, and pin interrupts. The general-purpose ports provide the following three functions:

- Pin multiplexing scheme
- GPIO functionality
- Pin interrupt requests

NOTE: In this chapter, the naming convention for registers and bits omits the alphabetic group enumeration to refer to any and all of the ports. For example, `PORT_FER` represents registers `PORTA_FER`, `PORTB_FER`, and so on. Likewise `PORT_FER.PX1` represents bits `PA1`, `PB1`, and so on.

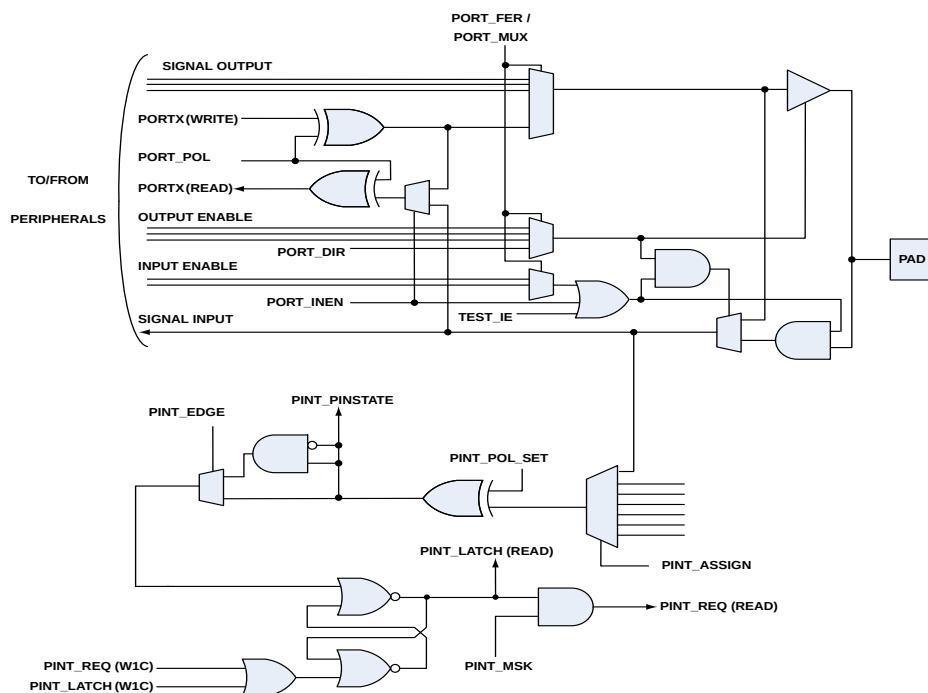


Figure 11-1: Simplified GPIO and Pin Interrupt Signal Flow

PORT Features

The PORTs include the following features:

- Input mode, output mode, and open-drain mode of GPIO operation
- Port multiplexing controlled on a pin-by-pin basis
- No external glue hardware required for unused pins
- All port pins provide interrupt request functionality
- Byte-wide pin-to-interrupt request assignment

PORT Functional Description

The number of ports and each's composition are defined in the processor datasheet. Each port has a dedicated set of MMR registers that control pin functions and operates in general-purpose I/O (GPIO) mode by default, as controlled by the port-specific `PORT_FER` register. Each bit in this register, as well as the other PORT MMRs, represents a specific GPIO pin on the specified port.

Input Mode, Output Mode, and Open-Drain Mode of GPIO Operation

At reset, every GPIO pin defaults to input mode with the input drivers disabled. To enable any GPIO input driver, set the bits corresponding to the individual pins in the appropriate input enable register (`PORT_INEN`).

The GPIO output drivers are enabled by setting the corresponding bits in the direction registers (`PORT_DIR`).

The PORT can use every GPIO in open-drain mode by clearing the respective bit in the `PORT_DATA` register or setting the respective bit in the `PORT_DATA_CLR` register. Then, set the corresponding bit in the `PORT_INEN` register. Read from the `PORT_DATA` register to obtain the status from the pin.

Port Multiplexing Controlled on Pin-by-Pin Basis

Each port has two dedicated MMRs that control the port multiplexing, the 16-bit function enable (`PORT_FER`) registers and the 32-bit port multiplexing (`PORT_MUX`) registers.

All Port Pins Provide Interrupt Functionality

Pin interrupts are completely decoupled from GPIO functionality. Pins are connected to the system event controller (SEC) via the PINTx modules, each of which is configurable in terms of which port pins are sensed for interrupt generation.

ADSP-2159x PORT Register List

The PORT module (PORT) regulates the use of the multiplexable processor pins. Every port pin can operate in general-purpose I/O (GPIO) mode or as an alternate function. This GPIO operation is the default after processor reset and is controlled by a set of registers that control GPIO functionality. Every bit in these registers represents a certain GPIO pin of a specific port. For more information on PORT functionality, see the PORT register descriptions.

Table 11-1: ADSP-2159x PORT Register List

Name	Description
PORT_DATA	Port x GPIO Data Register
PORT_DATA_CLR	Port x GPIO Data Clear Register
PORT_DATA_SET	Port x GPIO Data Set Register
PORT_DATA_TGL	Port x GPIO Output Toggle Register
PORT_DIR	Port x GPIO Direction Register
PORT_DIR_CLR	Port x GPIO Direction Clear Register
PORT_DIR_SET	Port x GPIO Direction Set Register
PORT_FER	Port x Function Enable Register
PORT_FER_CLR	Port x Function Enable Clear Register
PORT_FER_SET	Port x Function Enable Set Register
PORT_INEN	Port x GPIO Input Enable Register
PORT_INEN_CLR	Port x GPIO Input Enable Clear Register
PORT_INEN_SET	Port x GPIO Input Enable Set Register
PORT_LOCK	Port x GPIO Lock Register
PORT_MUX	Port x Multiplexer Control Register
PORT_POL	Port x GPIO Polarity Invert Register
PORT_POL_CLR	Port x GPIO Polarity Invert Clear Register
PORT_POL_SET	Port x GPIO Polarity Invert Set Register
PORT_TRIG_TGL	Port x GPIO Trigger Toggle Register

ADSP-2159x PORT Trigger List

Table 11-2: ADSP-2159x PORT Trigger List Masters

Trigger ID	Name	Description	Sensitivity
None			

Table 11-3: ADSP-2159x PORT Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
41	PORTA_TOGGLE	PORTA Port Toggle Trigger	Pulse
42	PORTB_TOGGLE	PORTB Port Toggle Trigger	Pulse
43	PORTC_TOGGLE	PORTC Port Toggle Trigger	Pulse
44	PORTD_TOGGLE	PORTD Port Toggle Trigger	Pulse
45	PORTE_TOGGLE	PORTE Port Toggle Trigger	Pulse
46	PORTF_TOGGLE	PORTF Port Toggle Trigger	Pulse
47	PORTG_TOGGLE	PORTG Port Toggle Trigger	Pulse
48	PORTH_TOGGLE	PORTH Port Toggle Trigger	Pulse
49	PORTI_TOGGLE	PORTI Port Toggle Trigger	Pulse

ADSP-2159x PINT Register List

The Pin Interrupt module (PINT) controls the pin-to-interrupt assignment in a byte-wide manner. The pin-interrupt assignment registers do not consist of 32 individual bits. They consist of four control bytes, each functioning as a multiplexer control. For more information, see the PINT register descriptions.

All PINT registers are 32 bits wide and can be accessed by 32-bit load/store instructions. They also support 16-bit operation where the upper 16 bits are ignored and the application uses the lower 16 bits only. Consequently, all PINT registers support 32-bit accesses as well as 16-bit accesses for the lower half words. Applications may use faster 16-bit accesses as long as they do not require functionality of upper register halves.

Table 11-4: ADSP-2159x PINT Register List

Name	Description
PINT_ASSIGN	PINT Assign Register
PINT_EDGE_CLR	PINT Edge Clear Register
PINT_EDGE_SET	PINT Edge Set Register
PINT_INV_CLR	PINT Invert Clear Register
PINT_INV_SET	PINT Invert Set Register
PINT_LATCH	PINT Latch Register
PINT_MSK_CLR	PINT Mask Clear Register
PINT_MSK_SET	PINT Mask Set Register
PINT_PINSTATE	PINT Pin State Register
PINT_REQ	PINT Request Register

ADSP-2159x PINT Interrupt List

Table 11-5: ADSP-2159x PINT Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
65	PINT0_BLOCK	PINT0 Pin Interrupt Block 0	Level	
66	PINT1_BLOCK	PINT1 Pin Interrupt Block 1	Level	
67	PINT2_BLOCK	PINT2 Pin Interrupt Block 2	Level	
68	PINT3_BLOCK	PINT3 Pin Interrupt Block 3	Level	
69	PINT4_BLOCK	PINT4 Pin Interrupt Block 4	Level	
70	PINT5_BLOCK	PINT5 Pin Interrupt Block 5	Level	
71	PINT6_BLOCK	PINT6 Pin Interrupt Block 6	Level	
72	PINT7_BLOCK	PINT7 Pin Interrupt Block 7	Level	

ADSP-2159x PINT Trigger List

Table 11-6: ADSP-2159x PINT Trigger List Masters

Trigger ID	Name	Description	Sensitivity
73	PINT0_BLOCK	PINT0 Pin Interrupt Block	Level
74	PINT1_BLOCK	PINT1 Pin Interrupt Block	Level
75	PINT2_BLOCK	PINT2 Pin Interrupt Block	Level
76	PINT3_BLOCK	PINT3 Pin Interrupt Block	Level
77	PINT4_BLOCK	PINT4 Pin Interrupt Block	Level
78	PINT5_BLOCK	PINT5 Pin Interrupt Block	Level
79	PINT6_BLOCK	PINT6 Pin Interrupt Block	Level
80	PINT7_BLOCK	PINT7 Pin Interrupt Block	Level

Table 11-7: ADSP-2159x PINT Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

ADSP-2159x PADS Register List

The PADS controls signal hysteresis and other system interface signal features for a number of module interfaces.

Table 11-8: ADSP-2159x PADS Register List

Name	Description
PADS_DAI0_0_DS	DAI0 1 to 10 pins DS control

Table 11-8: ADSP-2159x PADS Register List (Continued)

Name	Description
PADS_DAI0_1_DS	DAI0 11 to 20 pins DS control
PADS_DAI0_IE	DAI0 Port Input Enable Control Register
PADS_DAI1_0_DS	DAI1 1 to 10 pins DS control
PADS_DAI1_1_DS	DAI1 11 to 20 pins DS control
PADS_DAI1_IE	DAI1 Port Input Enable Control Register
PADS_DAI[n]_PDE	DAIx Pull-Down Enable
PADS_DAI[n]_PUE	DAIx Pull-Up Enable
PADS_NONPORTS_DS	Non-GPIO Drive Strength Register
PADS_PCFG0	Peripheral PAD Configuration0 Register
PADS_PCFG1	Peripheral Configuration1 Register
PADS_PORTA0_DS	PORTA 0 to 7 pins DS control
PADS_PORTA1_DS	PORTA 8 - 15 pins DS control
PADS_PORTB0_DS	PORTB 0 to 7 pins DS control
PADS_PORTB1_DS	PORTB 8 - 15 pins DS control
PADS_PORTC0_DS	PORTC 0 to 7 pins DS control
PADS_PORTC1_DS	PORTC 8 - 15 pins DS control
PADS_PORTD0_DS	PORTD 0 to 7 pins DS control
PADS_PORTD1_DS	PORTD 8 to 15 pins DS control
PADS_PORTE0_DS	PORTE 0 to 7 pins DS control
PADS_PORTE1_DS	PORTE 8 to 15 pins DS control
PADS_PORTF0_DS	PORTF 0 to 7 pins DS control
PADS_PORTF1_DS	PORTF 8 to 15 pins DS control
PADS_PORTG0_DS	PORTG 0 to 7 pins DS control
PADS_PORTG1_DS	PORTG 8 to 15 pins DS control
PADS_PORTH0_DS	PORTH 0 to 7 pins DS control
PADS_PORTH1_DS	PORTH 8 to 15 pins DS control
PADS_PORTI0_DS	PORTI 0 to 7 pins DS control
PADS_PORT[n]_PDE	PORTx Pull-Down Enable
PADS_PORT[n]_PUE	PORTx Pull-Up Enable

PORT Architectural Concepts

These sections describe in more detail how the PORT module connects externally to pins and internally to the MMR bus. Ports are named alphabetically beginning with A.

- [Internal Interfaces](#)
- [External Interfaces](#)
- [GPIO Pin Function](#)
- [Port Multiplexing Control](#)

Internal Interfaces

All of the pin multiplexing, GPIO, and pin interrupt control block MMRs can be accessed through the MMR bus. There is no DMA support. Each of the pin interrupt (PINTx) modules has its own dedicated interrupt request output signal that connects directly to the system event controller (SEC).

External Interfaces

The pin multiplexing hardware can be seen as a layer between the on-chip peripherals and the silicon pads connecting to the physical pins/balls or the package, as controlled by the PORT unit.

GPIO Pin Function

By default, the PORT sets every GPIO pin to input mode. The input drivers are not enabled, which avoids the need for unnecessary current sinks and external termination resistors on unused pins.

Input Mode

The default mode of every GPIO pin after reset is input mode, but the input drivers are not enabled. To enable GPIO input drivers, set the bits corresponding to the PORT pins in the appropriate input enable register ([PORT_INEN](#)). When enabled, a read from the [PORT_DATA](#) register returns the logical state of the input pins. However, the input signal does not overwrite the state of the internal flip-flop used for providing output to the same pin. Only software can alter the state. If the input driver is enabled, a write to the [PORT_DATA](#) register can alter the state of the flip-flop, but the change cannot be read back.

Output Mode

Any GPIO pin can be configured for output mode. The GPIO output drivers are enabled by setting the bits corresponding to the PORT pins in the appropriate direction register. The PORT implements direction registers as a pair of write-1-to-set (W1S) and write-1-to-clear (W1C) MMRs called [PORT_DIR_SET](#) and [PORT_DIR_CLR](#), respectively. As such, software can alter the direction of the signal flow on individual GPIO pins without impacting other GPIOs on the same port.

Both the [PORT_DIR_SET](#) and [PORT_DIR_CLR](#) registers return the same value when read, and a logical 1 indicates an enabled output. The [PORT_DATA](#) registers control the state of output pins. A logical 0 drives the output low while a logical 1 drives the output high.

While writes to the [PORT_DATA](#) register can alter all of the GPIOs on a specific port at once, there are also a pair of W1S and W1C MMRs called [PORT_DATA_SET](#) and [PORT_DATA_CLR](#), respectively. These registers enable

the manipulation of individual GPIO outputs. The state of the outputs can be obtained by reading the `PORT_DATA` registers. Because the state of the GPIO output can be controlled before the output driver is enabled, set or clear the internal flip-flop first by programming this register to avoid volatile levels on the output pin.

Trigger Toggle Mode

Any GPIO pin that has been configured for output mode can be toggled using a trigger input from the Trigger Routing Unit (TRU). To enable this functionality for a particular GPIO, set the appropriate bit in the `PORT_TRIG_TGL` register. Any subsequent trigger for the designated port causes all GPIO outputs set in the `PORT_TRIG_TGL` register to toggle.

To avoid unpredictable behavior, do not set, clear, or toggle the `PORT_DATA`, `PORT_DATA_SET`, `PORT_DATA_CLR`, or `PORT_DATA_TGL` registers when the GPIO output has the corresponding `PORT_TRIG_TGL` bit set. To change the state of the GPIO output using one of these registers, first clear the corresponding bit in the `PORT_TRIG_TGL` register.

Open-Drain Mode

Every GPIO can also be used in open-drain mode. First, either clear the respective bit in the `PORT_DATA` register or set the respective bit in the `PORT_DATA_CLR` register. Then, set the appropriate bit in the `PORT_INEN` register. Read from the `PORT_DATA` register to return the status from the pin rather than the state of the internal flip-flop.

By toggling the output driver through the `PORT_DIR_SET` and `PORT_DIR_CLR` register pair, the output signal can be pulled low or three-stated, as required. The polarity of the driven signal can be inverted when the internal flip-flop is set. When using a GPIO port in open-drain mode, take care to not exceed the V_{IH} operating condition associated with the respective pins.

Port Multiplexing Control

To configure pins properly, consult the processor datasheet to determine which bits in the `PORT_FER` and `PORT_MUX` register map to the pin of interest, and then set these registers appropriately for the desired function.

After reset, all port pins default to GPIO input mode with their output and input drivers disabled. As a result, all unused port pins can be left unconnected. Disabled pins appear as high-impedance to external circuits.

Each port has two dedicated MMRs that control the port multiplexing, the 16-bit function enable (`PORT_FER`) registers and the 32-bit port multiplexing (`PORT_MUX`) registers.

The function enable register specifies whether the pin is used as a GPIO pin or allocated for use by a specific peripheral, but it does not specify what the peripheral function is. Each bit in the 16-bit `PORT_FER` register corresponds to an individual port pin. For example, if bit 1 (PA1) of the `PORTA_FER` register is cleared, the PA_01 pin is configured as a GPIO. When set, one of the available peripheral functions becomes active on the PA_01 pin instead.

Pairs of bits in the `PORT_MUX` register control the multiplexing between the peripheral functions available to an individual pin, as some PORT pins provide up to four possible peripheral functions.

Refer to the Signal Muxing table in the datasheet for the specific `PORT_MUX` settings.

PORT Event Control

The following sections describe event generation in the PORT module.

PORT Interrupt Signals

The pin interrupts are decoupled from GPIO functionality, providing the following advantages.

- Flexible mapping scheme enables pins from up to four different ports to be grouped into one common interrupt scheme.
- Interrupt requests work on input and output pins regardless of whether the pin is functioning as a GPIO or a peripheral.

The processor has a number of interrupt channels dedicated to pin interrupts, managed by a set of pin interrupt (PINTx) blocks. Each PINTx block can sense up to 32 GPIO pins, as described in the following list and figure.

- PINT0 can sense pin activity on PORTA and PORTB
- PINT1 can sense pin activity on PORTB and PORTC
- PINT2 can sense pin activity on PORTC and PORTD
- PINT3 can sense pin activity on PORTD and PORTE
- PINT4 can sense pin activity on PORTE and PORTF
- PINT5 can sense pin activity on PORTF and PORTG
- PINT6 can sense pin activity on PORTG and PORTH
- PINT7 can sense pin activity on PORTH and PORTI

Note that PORTC to PORTI are not available on the low pin count package.

The processor supports both 32-bit and 16-bit peripheral bus accesses to PINTx registers.

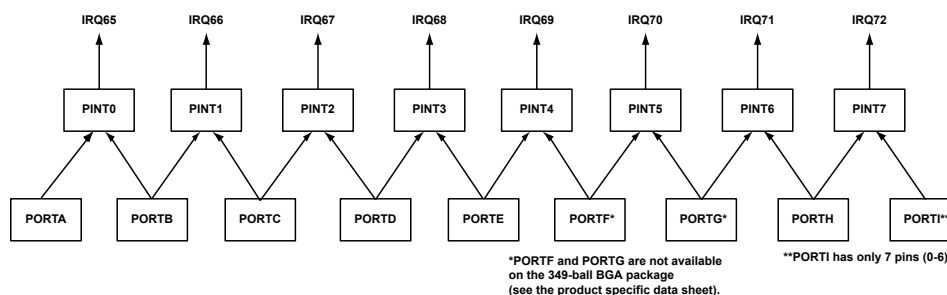


Figure 11-2: GPIO to PINTx Assignment

Pins connect to the PINTx module and then to the system event controller (SEC), as shown in the *PINTx Block Diagram*.

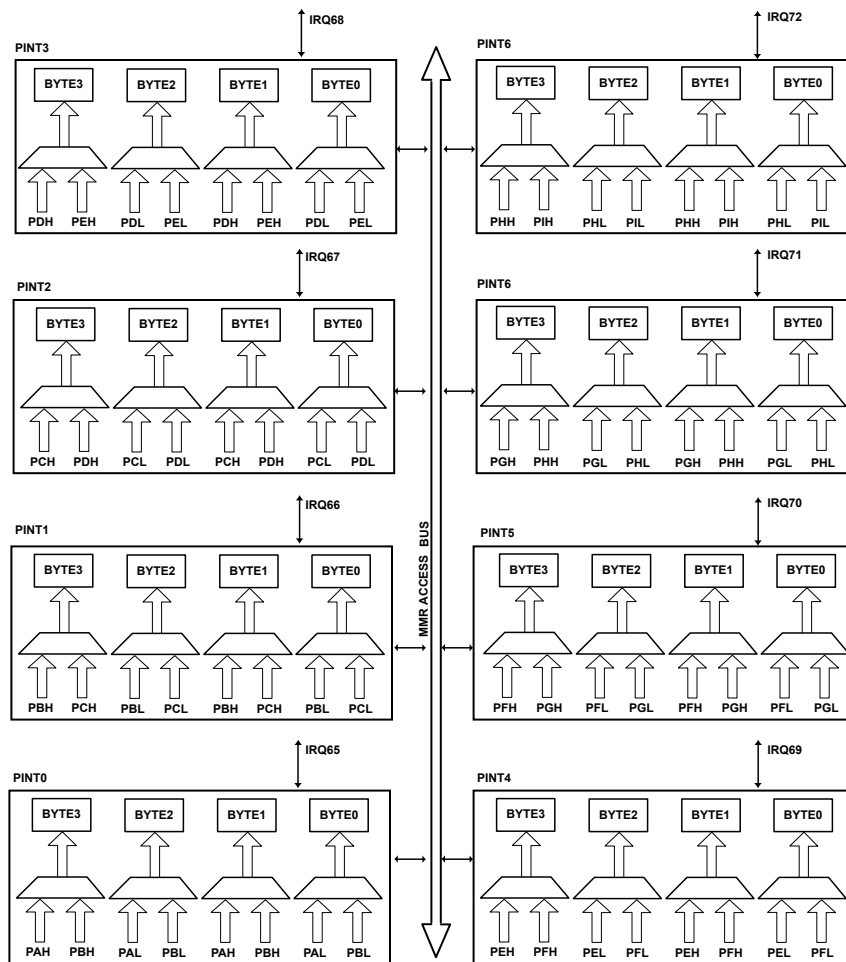


Figure 11-3: PINTx Block Diagram

As shown in the *PINTx Block Diagram*, each port is subdivided into two 8-pin half ports, upper (P×H) and lower (P×L). The `PINT_ASSIGN` registers control the 8-bit multiplexers associated with these half ports, where the lower half units (eight pins) can be forwarded to either byte 0 or byte 2 of the PINTx blocks, and the upper half units (eight pins) can be forwarded to either byte 1 or byte 3 of the PINTx blocks.

When a half port is assigned to a byte in any PINTx block, the state of the eight pins appears in the `PINT_PINSTATE` register, regardless of whether the pin is enabled for GPIO or peripheral functions (input or output). When neither the input nor output drivers of the pin are enabled, the pin state is read as zero. The `PINT_PINSTATE` register reports the inverted state of the pin when the `PINT_INV_SET` register activates the signal inverter. The inverter can be enabled on an individual bit-by-bit basis. Each bit in the `PINT_INV_SET/PINT_INV_CLR` register pair represents a pin signal.

By default, PORT interrupt request generation is level-sensitive, and an interrupt request occurs when the enabled pin is sensed as active high. Use the `PINT_EDGE_SET` register to change the interrupt request generation scheme to instead be edge-sensitive (rising edge generates the interrupt request). Use the `PINT_INV_SET` register to invert the polarity such that the PINTx block generates the interrupt request on active-low signals or falling edges.

The PINTx modules also assist when both signal edges must generate unique interrupt requests. If two different interrupt requests are required, the `PINT_ASSIGN` registers can route a single input signal to two different PINTx blocks, where one block inverts the signal in the `PINT_INV_SET` register and the other one does not. In this fashion, a unique software routine is associated with the hardware PINTx block that is generating the unique interrupt request for each signal edge. When both signal edges can be serviced by the same interrupt request, each half port can be routed to two separate bytes within a single PINTx block using the `PINT_ASSIGN` register, and then one of the half ports needs to have the inversion enabled in the `PINT_INV_SET` register. The servicing software routine can then detect from the `PINT_LATCH` register whether a falling, rising, or both edges have occurred.

Regardless of whether level-sensitive or edge-sensitive mode is used, the hardware always latches an interrupt request. Latched signals can be read from the `PINT_LATCH` registers. Only a software or hardware reset clears the latches. To clear the latch, a W1C operation must be performed to the `PINT_REQ` or `PINT_LATCH` register. When in level-sensitive mode, the interrupt request remains asserted if the pin state does not change by the time the interrupt service routine exits.

Because every PINTx block groups up to 32 pin signals, the `PINT_MSK_SET/ PINT_MSK_CLR` register pair can control which of the signals can request an interrupt at the system level. Software can interrogate the `PINT_REQ` register for signaling pins. The `PINT_REQ` bits represent a logical AND between the mask and the latch. When any of these bits is set, the block output interrupt request is asserted.

PORT Programming Model

The *GPIO Programming Model Flow (Part 1)*, *GPIO Programming Model Flow (Part 2)*, and *GPIO Programming Model Flow (Part 3)* figures show the programming model for the general-purpose ports. This programming includes the GPIO input and output operation, open-drain mode, and the pin interrupt PINTx modules.

NOTE: These process flow diagrams connect where call-out letters appear. For example, call-out A in the *GPIO Programming Model Flow (Part 1)* diagram connects to call-out A in the *GPIO Programming Model Flow (Part 2)* diagram.

The following flowcharts describe the processes for setting up pins for various functions. Begin the process from the start label in the *GPIO Programming Model Flow (Part 1)* figure. The first decision (GPIO or peripheral) determines how to program the `PORT_FER` register. Set the bit(s) corresponding to the pin(s) to 1 to enable peripheral functionality or to 0 to enable GPIO mode. For more information on setting up for peripheral functions, refer to [Port Multiplexing Control](#).

If the pin is to be a GPIO pin, a subsequent series of decisions must be made that will impact how the `PORT_DATA`, `PORT_POL`, `PORT_DIR`, and `PORT_INEN` configuration registers must be programmed. Depending on the type of GPIO pin desired, some configurations do not apply and can have different meanings. The following paragraphs briefly describe the function of the different settings for each of the pin functions in the input, output, and open-drain GPIO modes. It is a best practice to use the SET or CLR versions of the PORT registers, where applicable, to effect changes on a pin-by-pin basis rather than on the full port.

For output mode, first clear the `PORT_DATA` register to set all the pins low. Then write the `PORT_DIR` register to define the direction of each pin (set the bits associated with the desired output pins to 1). In output mode, the other registers are not significant. The *GPIO Programming Model Flow (Part 1)* chart shows this flow starting at label 2.

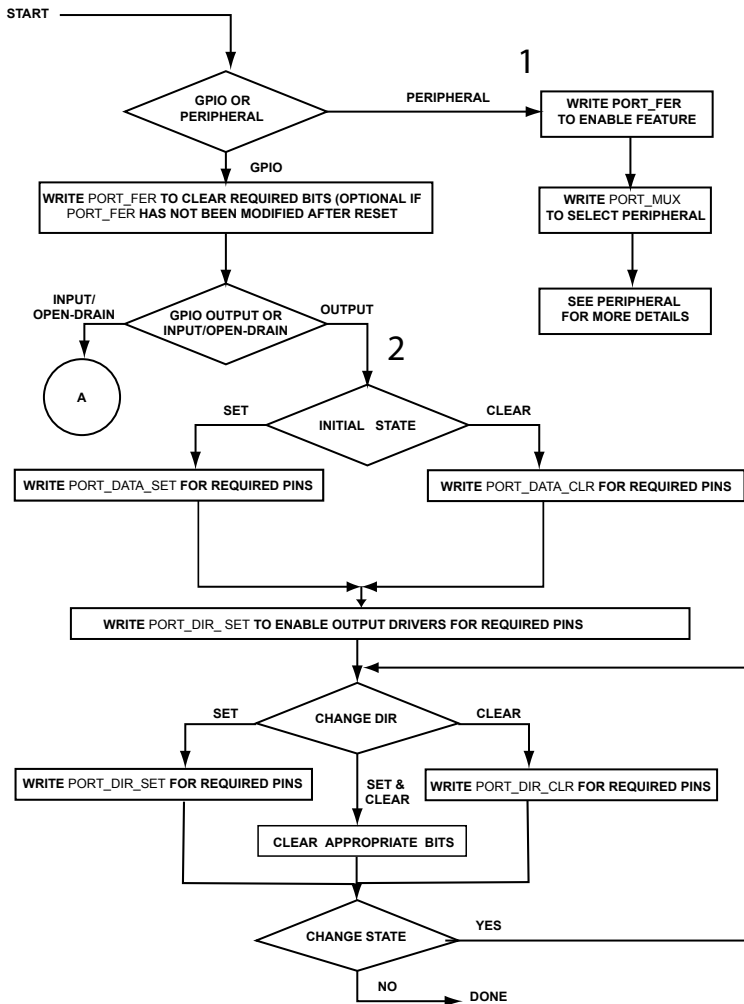


Figure 11-4: GPIO Programming Model Flow (Part 1)

For input mode, first decide the polarity for each pin using the `PORT_POL` register. Program the `PORT_DIR` register to define the appropriate pins as inputs (write a 0 to the bit location associated with the pin). If interrupt requests are desired, configure the PINT module as shown in the *GPIO Programming Model Flow (Part 3)* figure starting at label B. Finally, write the `PORT_INEN` register to enable the associated input drivers. The *GPIO Programming Model Flow (Part 2)* chart shows this entire flow starting at label 3.

For open-drain mode, set all pins low by clearing the `PORT_DATA` register. Then, use the `PORT_INEN` register to enable the appropriate input drivers. Set the `PORT_DIR` register in this mode to indicate whether the pin is in an active state or not (active being 0). The *GPIO Programming Model Flow (Part 2)* chart shows this flow starting at label 4.

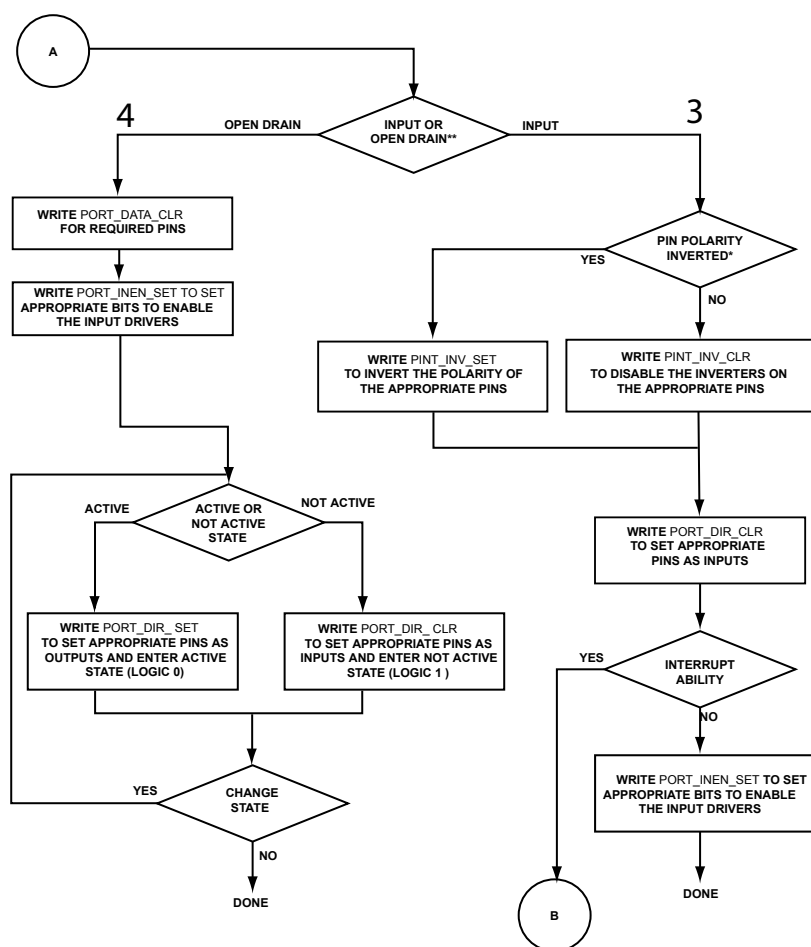


Figure 11-5: GPIO Programming Model Flow (Part 2)

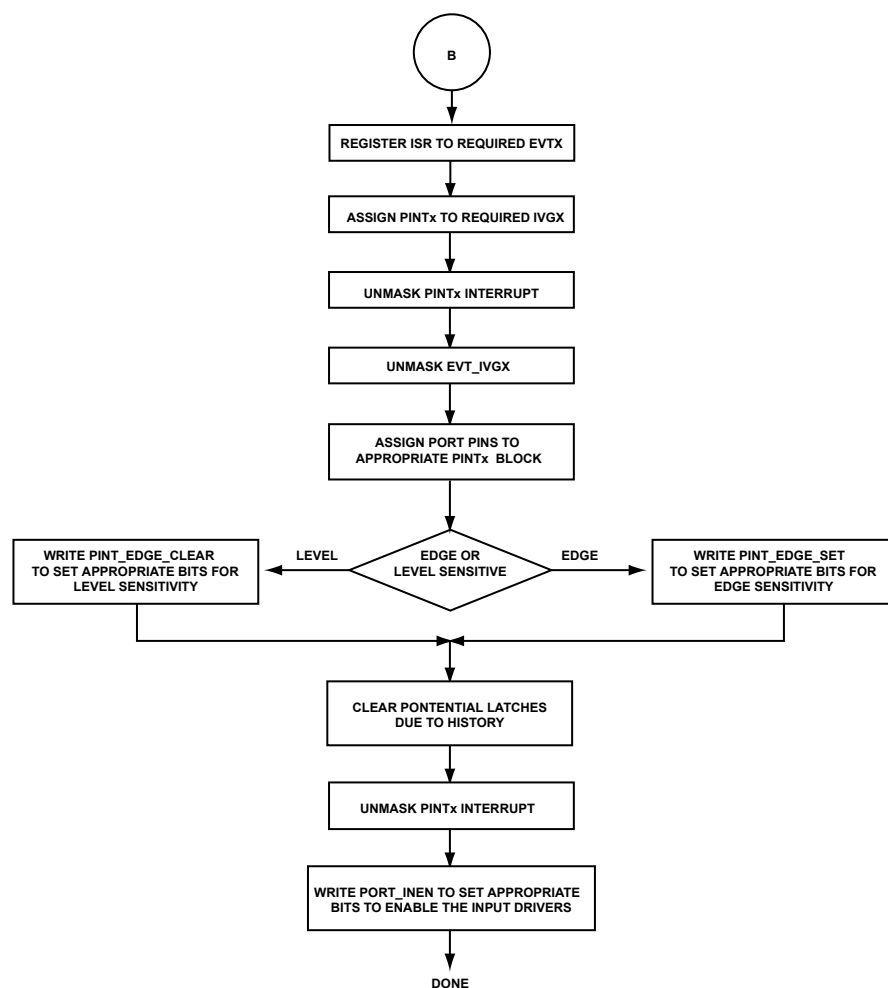


Figure 11-6: GPIO Programming Model Flow (Part 3)

ADSP-2159x PORT Register Descriptions

The General-Purpose Input/Output Port (PORT) contains the following registers.

Table 11-9: ADSP-2159x PORT Register List

Name	Description
PORT_DATA	Port x GPIO Data Register
PORT_DATA_CLR	Port x GPIO Data Clear Register
PORT_DATA_SET	Port x GPIO Data Set Register
PORT_DATA_TGL	Port x GPIO Output Toggle Register
PORT_DIR	Port x GPIO Direction Register
PORT_DIR_CLR	Port x GPIO Direction Clear Register

Table 11-9: ADSP-2159x PORT Register List (Continued)

Name	Description
PORT_DIR_SET	Port x GPIO Direction Set Register
PORT_FER	Port x Function Enable Register
PORT_FER_CLR	Port x Function Enable Clear Register
PORT_FER_SET	Port x Function Enable Set Register
PORT_INEN	Port x GPIO Input Enable Register
PORT_INEN_CLR	Port x GPIO Input Enable Clear Register
PORT_INEN_SET	Port x GPIO Input Enable Set Register
PORT_LOCK	Port x GPIO Lock Register
PORT_MUX	Port x Multiplexer Control Register
PORT_POL	Port x GPIO Polarity Invert Register
PORT_POL_CLR	Port x GPIO Polarity Invert Clear Register
PORT_POL_SET	Port x GPIO Polarity Invert Set Register
PORT_TRIG_TGL	Port x GPIO Trigger Toggle Register

Port x GPIO Data Register

The operation of the `PORT_DATA` register depends on whether the bit/pin is in output mode or input mode. In both modes, a set bit in the `PORT_DATA` register corresponds to a signal high on a GPIO pin. A cleared bit in the `PORT_DATA` register corresponds to a signal low on a GPIO pin.

The `PORT_DATA`, `PORT_DATA_SET`, and `PORT_DATA_CLR` registers control the state of GPIO pins in output mode. To enable output mode (and output drivers), use the `PORT_DIR_SET` and `PORT_DIR_CLR` registers.

Writes to the `PORT_DATA` register affect the state of all pins of the port that are in output mode. To set or clear specific pins without impacting other pins of the port, use the `PORT_DATA_SET` and `PORT_DATA_CLR` registers.

When the GPIO pins are in input mode (input driver is enabled with the `PORT_INEN` register), reads from the `PORT_DATA`, `PORT_DATA_SET`, and `PORT_DATA_CLR` registers return the state of the respective GPIO pins.

Note that when the input driver is not enabled, reads from the `PORT_DATA`, `PORT_DATA_SET`, and `PORT_DATA_CLR` registers return the value previously written to the registers.

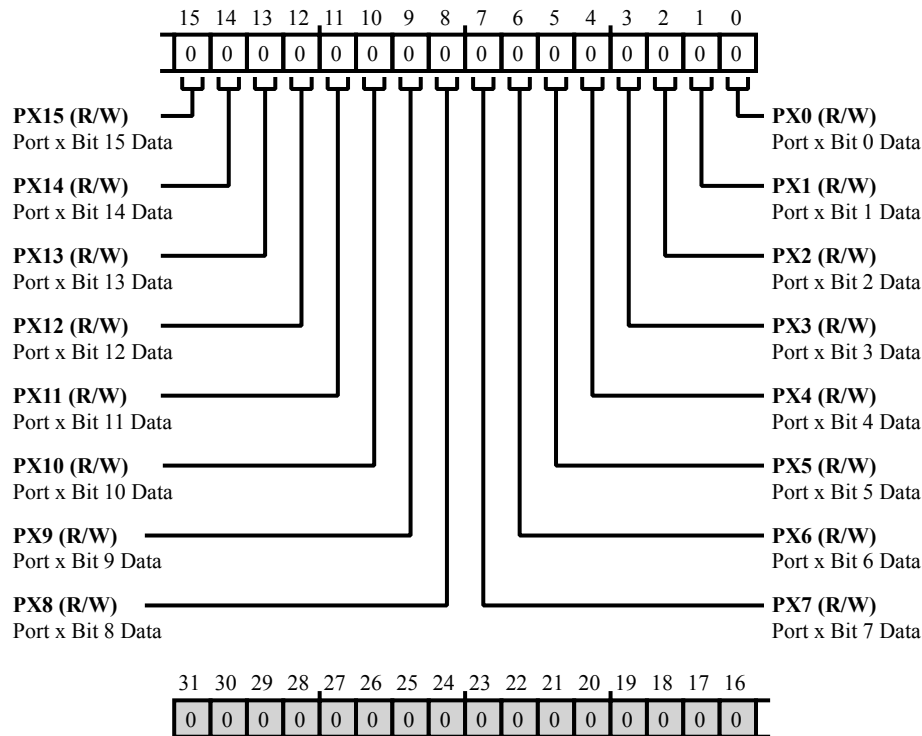


Figure 11-7: `PORT_DATA` Register Diagram

Table 11-10: PORT_DATA Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Data. The PORT_DATA.PX15 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
14 (R/W)	PX14	Port x Bit 14 Data. The PORT_DATA.PX14 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
13 (R/W)	PX13	Port x Bit 13 Data. The PORT_DATA.PX13 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
12 (R/W)	PX12	Port x Bit 12 Data. The PORT_DATA.PX12 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
11 (R/W)	PX11	Port x Bit 11 Data. The PORT_DATA.PX11 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
10 (R/W)	PX10	Port x Bit 10 Data. The PORT_DATA.PX10 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
9 (R/W)	PX9	Port x Bit 9 Data. The PORT_DATA.PX9 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High

Table 11-10: PORT_DATA Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	PX8	Port x Bit 8 Data. The PORT_DATA.PX8 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
7 (R/W)	PX7	Port x Bit 7 Data. The PORT_DATA.PX7 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
6 (R/W)	PX6	Port x Bit 6 Data. The PORT_DATA.PX6 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
5 (R/W)	PX5	Port x Bit 5 Data. The PORT_DATA.PX5 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
4 (R/W)	PX4	Port x Bit 4 Data. The PORT_DATA.PX4 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
3 (R/W)	PX3	Port x Bit 3 Data. The PORT_DATA.PX3 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
2 (R/W)	PX2	Port x Bit 2 Data. The PORT_DATA.PX2 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High

Table 11-10: PORT_DATA Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	PX1	Port x Bit 1 Data. The PORT_DATA.PX1 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High
0 (R/W)	PX0	Port x Bit 0 Data. The PORT_DATA.PX0 bit indicates a signal on a GPIO pin.
		0 Signal Low
		1 Signal High

Port x GPIO Data Clear Register

The `PORT_DATA_CLR` register operates differently for port bits/pins, depending on whether the bit/pin is output mode or input mode. For more information, see the `PORT_DATA` register description.

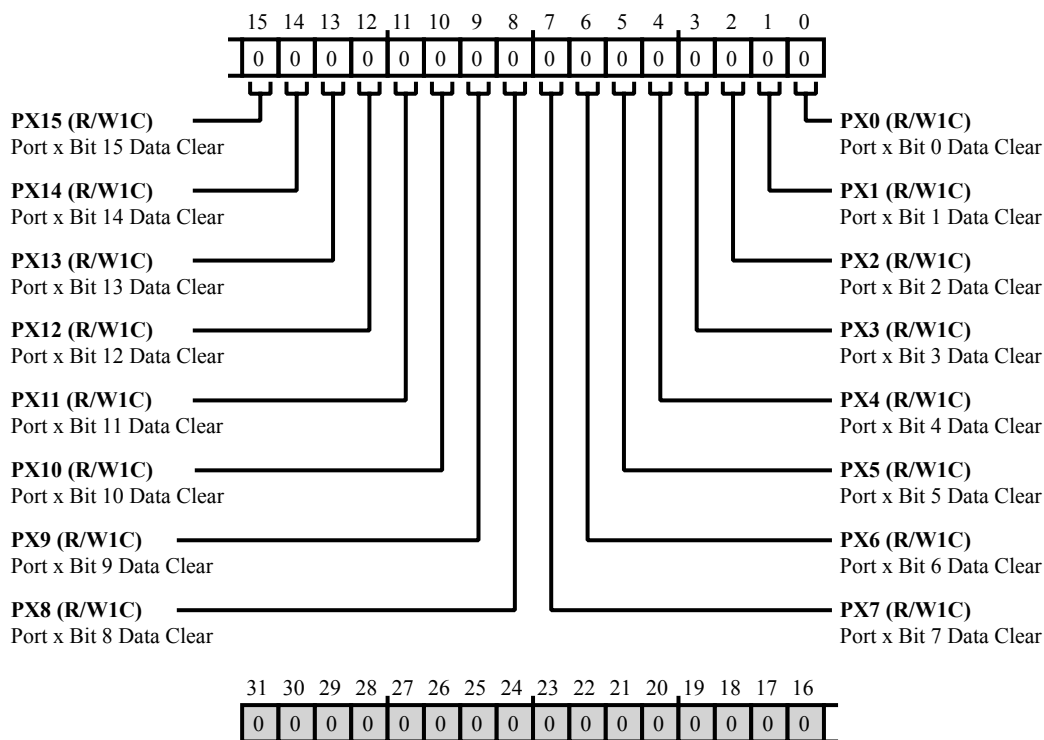


Figure 11-8: `PORT_DATA_CLR` Register Diagram

Table 11-11: `PORT_DATA_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Data Clear. The <code>PORT_DATA_CLR</code> . PX15 bit clears the pin without impacting other pins of the port.
		0 No Effect
		1 Clear Bit. Write 1 for signal low in output mode.
14 (R/W1C)	PX14	Port x Bit 14 Data Clear. The <code>PORT_DATA_CLR</code> . PX14 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.

Table 11-11: PORT_DATA_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1C)	PX13	Port x Bit 13 Data Clear. The PORT_DATA_CLR.PX13 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
12 (R/W1C)	PX12	Port x Bit 12 Data Clear. The PORT_DATA_CLR.PX12 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
11 (R/W1C)	PX11	Port x Bit 11 Data Clear. The PORT_DATA_CLR.PX11 bit clears the pin without impacting other pins of the port.
		0 No Effect
		1 Clear Bit. Write 1 for signal low in output mode.
10 (R/W1C)	PX10	Port x Bit 10 Data Clear. The PORT_DATA_CLR.PX10 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
9 (R/W1C)	PX9	Port x Bit 9 Data Clear. The PORT_DATA_CLR.PX9 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
8 (R/W1C)	PX8	Port x Bit 8 Data Clear. The PORT_DATA_CLR.PX8 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.

Table 11-11: PORT_DATA_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	PX7	Port x Bit 7 Data Clear. The PORT_DATA_CLR.PX7 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
6 (R/W1C)	PX6	Port x Bit 6 Data Clear. The PORT_DATA_CLR.PX6 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
5 (R/W1C)	PX5	Port x Bit 5 Data Clear. The PORT_DATA_CLR.PX5 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
4 (R/W1C)	PX4	Port x Bit 4 Data Clear. The PORT_DATA_CLR.PX4 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
3 (R/W1C)	PX3	Port x Bit 3 Data Clear. The PORT_DATA_CLR.PX3 bit clears the pin without impacting other pins of the port.
		0 No Effect. Write 0 has no effect in output mode.
		1 Clear Bit. Write 1 for signal low in output mode.
2 (R/W1C)	PX2	Port x Bit 2 Data Clear. The PORT_DATA_CLR.PX2 bit clears the pin without impacting other pins of the port.
		0 No Effect Write 0 has no effect in output mode.
		1 Clear Bit Write 1 for signal low in output mode.

Table 11-11: PORT_DATA_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R/W1C)	PX1	Port x Bit 1 Data Clear. The PORT_DATA_CLR.PX1 bit clears the pin without impacting other pins of the port.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Clear Bit. Write 1 for signal low in output mode.
0 (R/W1C)	PX0	Port x Bit 0 Data Clear. The PORT_DATA_CLR.PX0 bit clears the pin without impacting other pins of the port.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Clear Bit. Write 1 for signal low in output mode.

Port x GPIO Data Set Register

The `PORT_DATA_SET` register operates differently for port bits/pins, depending on whether the bit/pin is output mode or input mode. For more information, see the `PORT_DATA` register description.

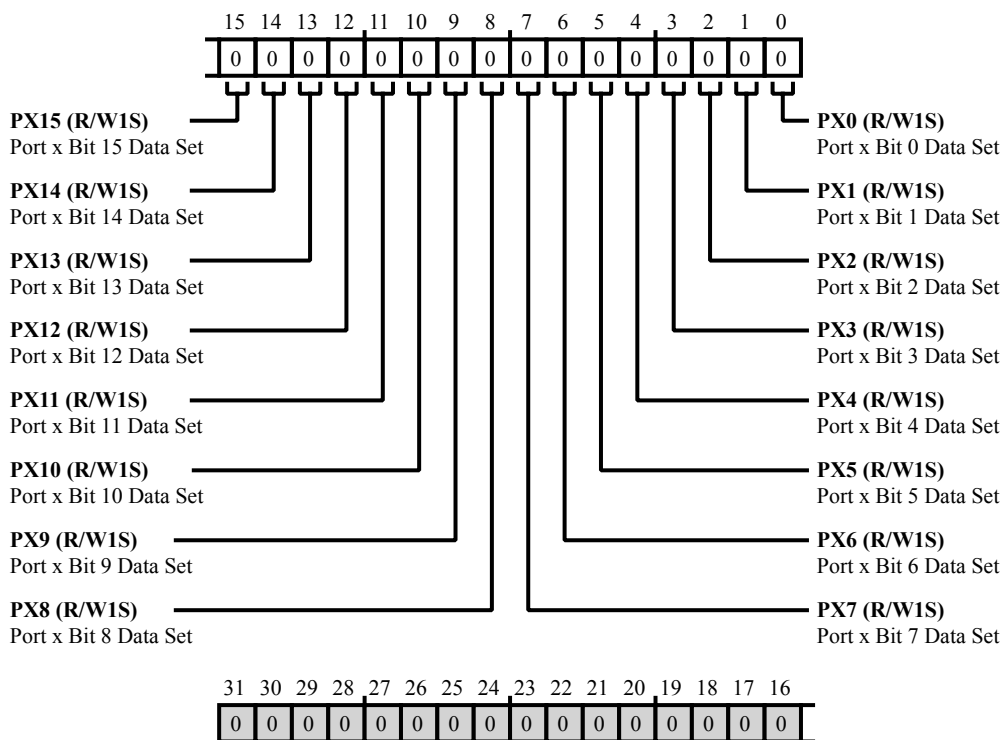


Figure 11-9: PORT_DATA_SET Register Diagram

Table 11-12: PORT_DATA_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
15 (R/W1S)	PX15	Port x Bit 15 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
14 (R/W1S)	PX14	Port x Bit 14 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
13 (R/W1S)	PX13	Port x Bit 13 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.

Table 11-12: PORT_DATA_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/W1S)	PX12	Port x Bit 12 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
11 (R/W1S)	PX11	Port x Bit 11 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit.
10 (R/W1S)	PX10	Port x Bit 10 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
9 (R/W1S)	PX9	Port x Bit 9 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
8 (R/W1S)	PX8	Port x Bit 8 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
7 (R/W1S)	PX7	Port x Bit 7 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
6 (R/W1S)	PX6	Port x Bit 6 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
5 (R/W1S)	PX5	Port x Bit 5 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
4 (R/W1S)	PX4	Port x Bit 4 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
3 (R/W1S)	PX3	Port x Bit 3 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.

Table 11-12: PORT_DATA_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W1S)	PX2	Port x Bit 2 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
1 (R/W1S)	PX1	Port x Bit 1 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.
0 (R/W1S)	PX0	Port x Bit 0 Data Set.	
		0	No Effect. Write 0 has no effect in output mode.
		1	Set Bit. Write 1 for signal high in output mode.

Port x GPIO Output Toggle Register

The `PORT_DATA_TGL` register permits toggling the state of output GPIO pins. Setting bits in the `PORT_DATA_TGL` register affects the state of specific pins without impacting other pins of the port.

Reading the `PORT_DATA_TGL` returns the state of the `PORT_DATA` register output pin state, but does not return the input pin/signal state.

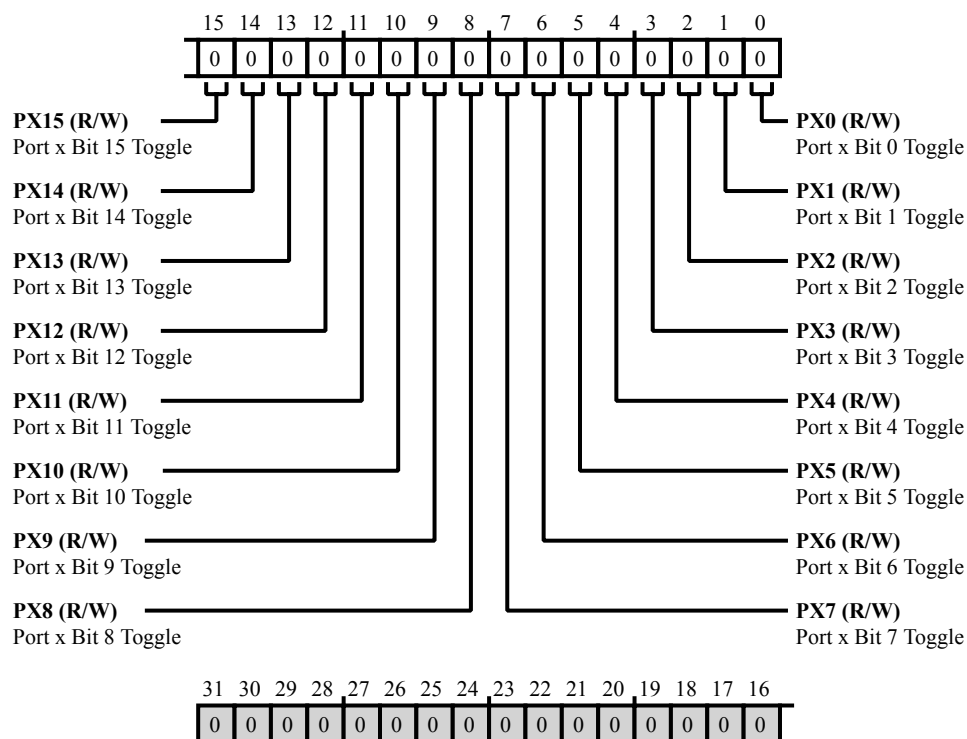


Figure 11-10: `PORT_DATA_TGL` Register Diagram

Table 11-13: `PORT_DATA_TGL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Toggle. The <code>PORT_DATA_TGL</code> , PX15 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit.
14 (R/W)	PX14	Port x Bit 14 Toggle. The <code>PORT_DATA_TGL</code> , PX14 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit

Table 11-13: PORT_DATA_TGL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PX13	Port x Bit 13 Toggle. The PORT_DATA_TGL.PX13 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
12 (R/W)	PX12	Port x Bit 12 Toggle. The PORT_DATA_TGL.PX12 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
11 (R/W)	PX11	Port x Bit 11 Toggle. The PORT_DATA_TGL.PX11 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
10 (R/W)	PX10	Port x Bit 10 Toggle. The PORT_DATA_TGL.PX10 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
9 (R/W)	PX9	Port x Bit 9 Toggle. The PORT_DATA_TGL.PX9 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
8 (R/W)	PX8	Port x Bit 8 Toggle. The PORT_DATA_TGL.PX8 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
7 (R/W)	PX7	Port x Bit 7 Toggle. The PORT_DATA_TGL.PX7 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit

Table 11-13: PORT_DATA_TGL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	PX6	Port x Bit 6 Toggle. The PORT_DATA_TGL.PX6 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
5 (R/W)	PX5	Port x Bit 5 Toggle. The PORT_DATA_TGL.PX5 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
4 (R/W)	PX4	Port x Bit 4 Toggle. The PORT_DATA_TGL.PX4 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
3 (R/W)	PX3	Port x Bit 3 Toggle. The PORT_DATA_TGL.PX3 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
2 (R/W)	PX2	Port x Bit 2 Toggle. The PORT_DATA_TGL.PX2 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
1 (R/W)	PX1	Port x Bit 1 Toggle. The PORT_DATA_TGL.PX1 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit
0 (R/W)	PX0	Port x Bit 0 Toggle. The PORT_DATA_TGL.PX0 bit toggles the output GPIO bit/pin state.
		0 No Effect
		1 Toggle Bit

Port x GPIO Direction Register

The `PORT_DIR`, `PORT_DIR_SET`, and `PORT_DIR_CLR` registers select output or input mode for GPIO pins and enable output drivers. Use the `PORT_INEN`, `PORT_INEN_SET`, and `PORT_INEN_CLR` registers to enable or disable input drivers.

Writes to the `PORT_DIR` register affect the state of all pins of the port. To select a direction for specific pins without impacting other pins of the port, use the `PORT_DIR_SET` and `PORT_DIR_CLR` registers.

Setting a bit in the `PORT_DIR` register enables output mode on the corresponding a GPIO pin. Clearing a bit in the `PORT_DIR` register disables output mode on the corresponding GPIO pin.

Input Mode - The default mode of every GPIO pin after reset is the input mode, but the input drivers are not enabled. To enable GPIO input drivers, set the corresponding bits in the `PORT_INEN` register. When enabled, a read from the `PORT_DATA` register returns the logical state of the input pin. The input signal does not overwrite the state of the bit used for the output case. That state can only be altered by software. If the input driver is enabled, a write to the `PORT_DATA` register can alter the state of the bit, but the change cannot be read back.

Output Mode - Any GPIO pin can be configured for output mode. The GPIO output drivers are enabled by setting the corresponding bits in the `PORT_DIR`, `PORT_DIR_SET`, or `PORT_DIR_CLR` registers. By using the `PORT_DIR_SET` and `PORT_DIR_CLR` registers, the direction of the signal flow of individual GPIO pins can be altered by separate software threads without mutually impacting other GPIOs on the same port. Both registers return the same value when read. Because the state of the GPIO output can already be controlled before the output driver is enabled, it is recommended to first set or clear the bit (using the `PORT_DATA`, `PORT_DATA_SET`, or `PORT_DATA_CLR` registers) to avoid any volatile levels on the output.

Open-Drain Mode - Every GPIO can also be used in open-drain mode. To accomplish this, first, clear the respective bit in the `PORT_DATA` or `PORT_DATA_CLR` register. Then, set the one bit in the `PORT_INEN` register. Reads from the `PORT_DATA` register then return the status from the pin and do not return the state of the internal flip-flop. By toggling the output driver through the `PORT_DIR_SET` and `PORT_DIR_CLR` register pair, the output signal can be pulled low or three-stated as required. Note that the polarity of the driven signal can be inverted when the internal flip-flop is set instead. When a GPIO port is used in open-drain mode, take care to not exceed the V_{IH} operating condition associated with the respective pin.

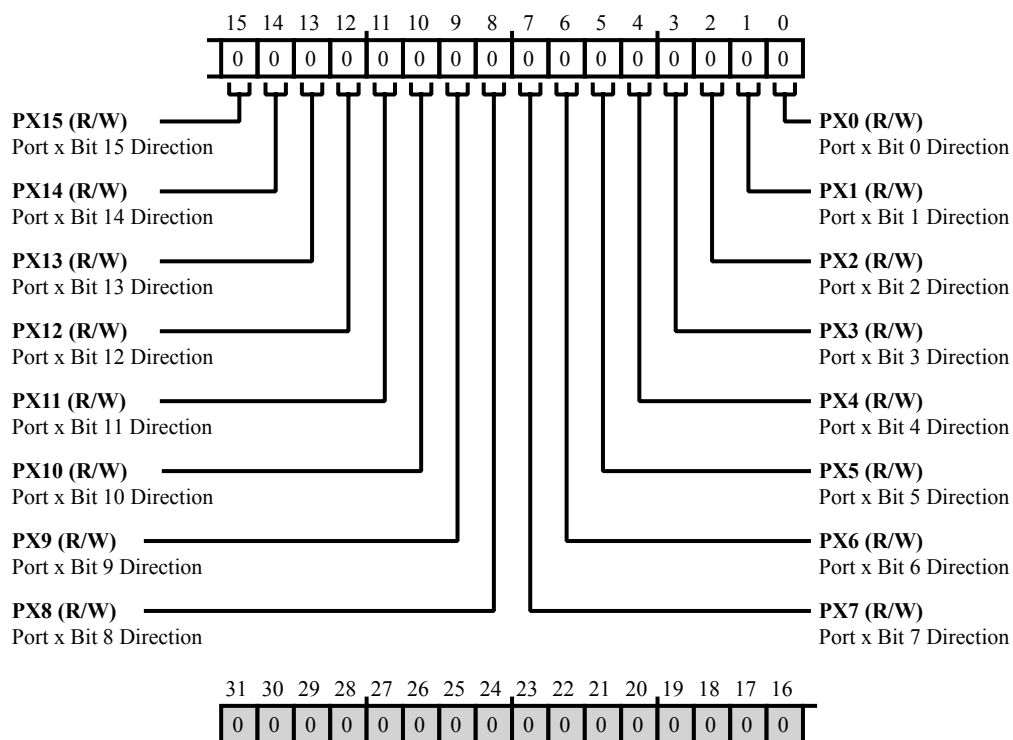


Figure 11-11: PORT_DIR Register Diagram

Table 11-14: PORT_DIR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Direction.
		0 Input mode. The output driver is disabled.
		1 Output mode. The output driver is enabled.
14 (R/W)	PX14	Port x Bit 14 Direction.
		0 Input mode. The output driver is disabled.
		1 Output mode. The output driver is enabled.
13 (R/W)	PX13	Port x Bit 13 Direction.
		0 Input mode. The output driver is disabled.
		1 Output mode. The output driver is enabled.
12 (R/W)	PX12	Port x Bit 12 Direction.
		0 Input mode. The output driver is disabled.
		1 Output mode. The output driver is enabled.

Table 11-14: PORT_DIR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
11 (R/W)	PX11	Port x Bit 11 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
10 (R/W)	PX10	Port x Bit 10 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
9 (R/W)	PX9	Port x Bit 9 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
8 (R/W)	PX8	Port x Bit 8 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
7 (R/W)	PX7	Port x Bit 7 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
6 (R/W)	PX6	Port x Bit 6 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
5 (R/W)	PX5	Port x Bit 5 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
4 (R/W)	PX4	Port x Bit 4 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
3 (R/W)	PX3	Port x Bit 3 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
2 (R/W)	PX2	Port x Bit 2 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.

Table 11-14: PORT_DIR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R/W)	PX1	Port x Bit 1 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.
0 (R/W)	PX0	Port x Bit 0 Direction.	
		0	Input mode. The output driver is disabled.
		1	Output mode. The output driver is enabled.

Port x GPIO Direction Clear Register

The `PORT_DIR_CLR` register disables output mode and disables the output drivers for GPIO pins. For more information, see the `PORT_DIR` register description.

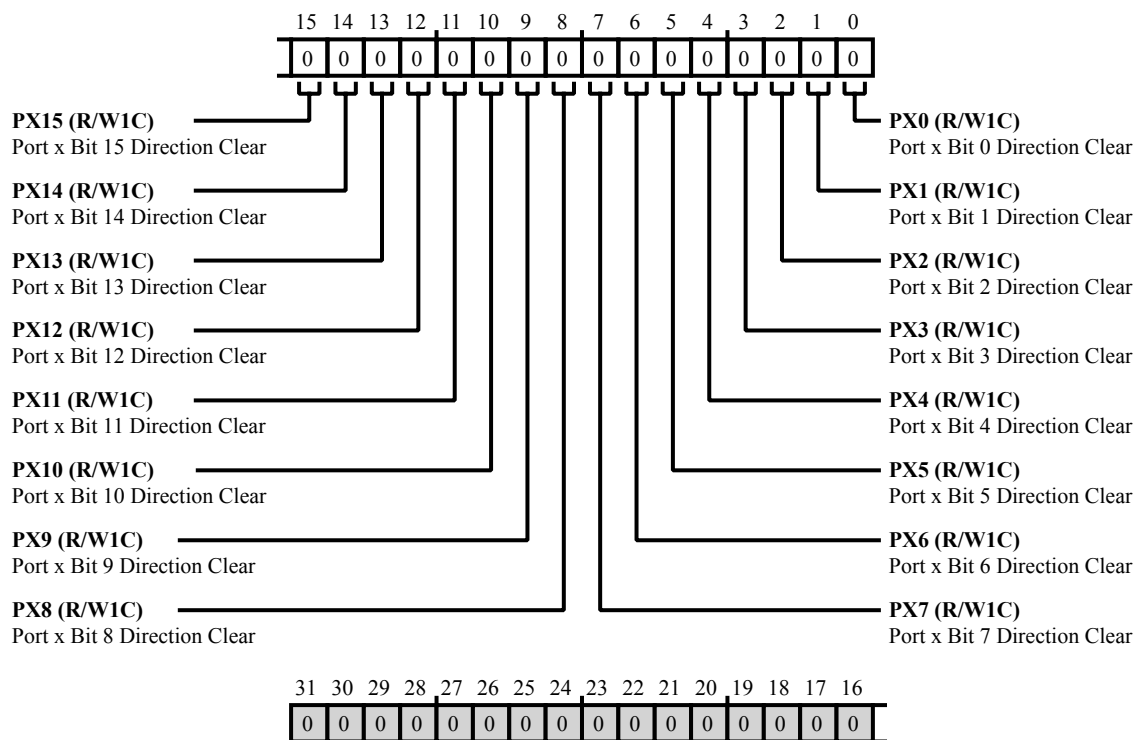


Figure 11-12: `PORT_DIR_CLR` Register Diagram

Table 11-15: `PORT_DIR_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Direction Clear. The <code>PORT_DIR_CLR</code> . PX15 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
14 (R/W1C)	PX14	Port x Bit 14 Direction Clear. The <code>PORT_DIR_CLR</code> . PX14 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver

Table 11-15: PORT_DIR_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1C)	PX13	Port x Bit 13 Direction Clear. The PORT_DIR_CLR.PX13 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
12 (R/W1C)	PX12	Port x Bit 12 Direction Clear. The PORT_DIR_CLR.PX12 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
11 (R/W1C)	PX11	Port x Bit 11 Direction Clear. The PORT_DIR_CLR.PX11 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
10 (R/W1C)	PX10	Port x Bit 10 Direction Clear. The PORT_DIR_CLR.PX10 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
9 (R/W1C)	PX9	Port x Bit 9 Direction Clear. The PORT_DIR_CLR.PX9 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver
8 (R/W1C)	PX8	Port x Bit 8 Direction Clear. The PORT_DIR_CLR.PX8 bit disables output mode and the output drivers for port x.
		0 No Effect
		1 Disable output mode/driver

Table 11-15: PORT_DIR_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
7 (R/W1C)	PX7	Port x Bit 7 Direction Clear. The PORT_DIR_CLR.PX7 bit disables output mode and the output drivers for port x.	
		0	No Effect
		1	Disable output mode/driver
6 (R/W1C)	PX6	Port x Bit 6 Direction Clear. The PORT_DIR_CLR.PX6 bit disables output mode and the output drivers for port x.	
		0	No Effect
		1	Disable output mode/driver
5 (R/W1C)	PX5	Port x Bit 5 Direction Clear. The PORT_DIR_CLR.PX5 bit disables output mode and the output drivers for port x.	
		0	No Effect
		1	Disable output mode/driver
4 (R/W1C)	PX4	Port x Bit 4 Direction Clear. The PORT_DIR_CLR.PX4 bit disables output mode and the output drivers for port x.	
		0	No Effect
		1	Disable output mode/driver
3 (R/W1C)	PX3	Port x Bit 3 Direction Clear. The PORT_DIR_CLR.PX3 bit disables output mode and the output drivers for port x.	
		0	No Effect
		1	Disable output mode/driver
2 (R/W1C)	PX2	Port x Bit 2 Direction Clear. The PORT_DIR_CLR.PX2 bit disables output mode and the output drivers for port x.	
		0	No Effect
		1	Disable output mode/driver

Table 11-15: PORT_DIR_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R/W1C)	PX1	Port x Bit 1 Direction Clear. The PORT_DIR_CLR.PX1 bit disables output mode and the output drivers for port x.	
		0	No Effect
		1	Disable output mode/driver
0 (R/W1C)	PX0	Port x Bit 0 Direction Clear. The PORT_DIR_CLR.PX0 bit disables output mode and the output drivers for port x.	
		0	No Effect
		1	Disable output mode/driver

Port x GPIO Direction Set Register

The `PORT_DIR_SET` register enables output mode and output drivers for GPIO pins. For more information, see the `PORT_DIR` register description.

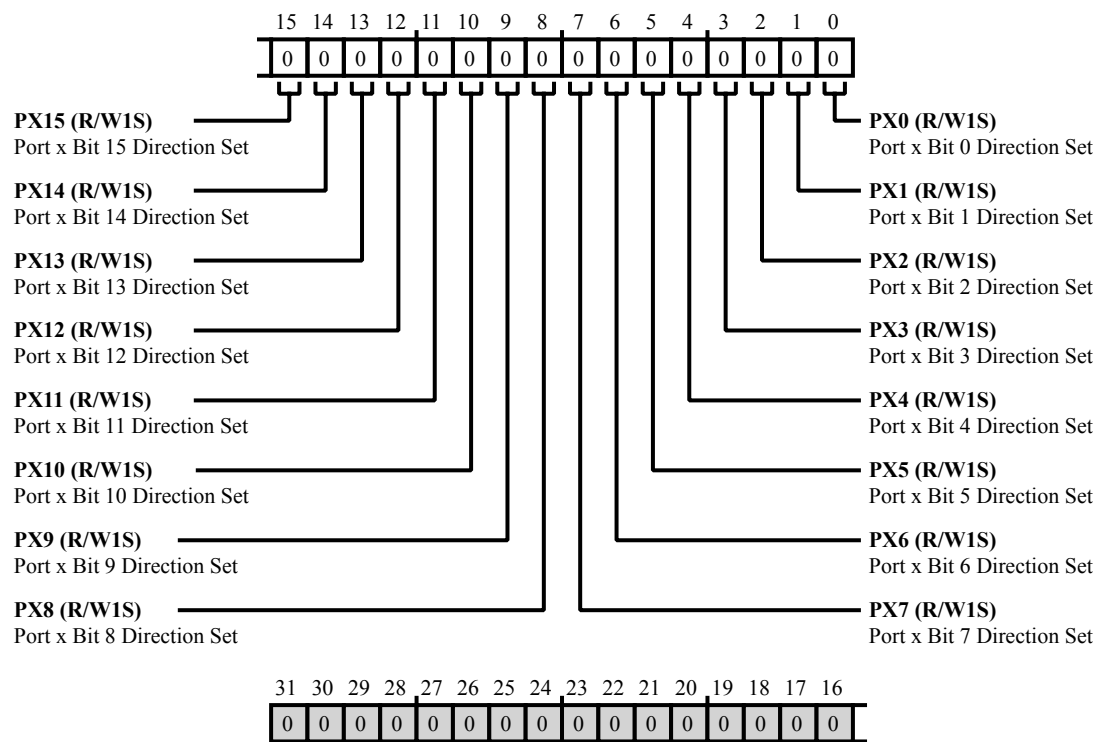


Figure 11-13: `PORT_DIR_SET` Register Diagram

Table 11-16: `PORT_DIR_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Direction Set. The <code>PORT_DIR_SET</code> . PX15 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver
14 (R/W1S)	PX14	Port x Bit 14 Direction Set. The <code>PORT_DIR_SET</code> . PX14 bit enables the output mode/driver for port x.
		0 No Effect
		1 Enable output mode/driver

Table 11-16: PORT_DIR_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
13 (R/W1S)	PX13	Port x Bit 13 Direction Set. The PORT_DIR_SET.PX13 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
12 (R/W1S)	PX12	Port x Bit 12 Direction Set. The PORT_DIR_SET.PX12 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
11 (R/W1S)	PX11	Port x Bit 11 Direction Set. The PORT_DIR_SET.PX11 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
10 (R/W1S)	PX10	Port x Bit 10 Direction Set. The PORT_DIR_SET.PX10 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
9 (R/W1S)	PX9	Port x Bit 9 Direction Set. The PORT_DIR_SET.PX9 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
8 (R/W1S)	PX8	Port x Bit 8 Direction Set. The PORT_DIR_SET.PX8 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
7 (R/W1S)	PX7	Port x Bit 7 Direction Set. The PORT_DIR_SET.PX7 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver

Table 11-16: PORT_DIR_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
6 (R/W1S)	PX6	Port x Bit 6 Direction Set. The PORT_DIR_SET . PX6 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
5 (R/W1S)	PX5	Port x Bit 5 Direction Set. The PORT_DIR_SET . PX5 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
4 (R/W1S)	PX4	Port x Bit 4 Direction Set. The PORT_DIR_SET . PX4 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
3 (R/W1S)	PX3	Port x Bit 3 Direction Set. The PORT_DIR_SET . PX3 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
2 (R/W1S)	PX2	Port x Bit 2 Direction Set. The PORT_DIR_SET . PX2 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
1 (R/W1S)	PX1	Port x Bit 1 Direction Set. The PORT_DIR_SET . PX1 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver
0 (R/W1S)	PX0	Port x Bit 0 Direction Set. The PORT_DIR_SET . PX0 bit enables the output mode/driver for port x.	
		0	No Effect
		1	Enable output mode/driver

Port x Function Enable Register

The `PORT_FER` register bits indicate each port bit's operating mode: general purpose I/O mode or peripheral mode. After reset, all pins default to GPIO mode. Setting a bit in the `PORT_FER` registers enables a peripheral module to take ownership of the pin. The function enable bits impact output control only. Regardless of the setting of the function enable bits, both GPIO and peripherals can still sense the pin input. After a function is enabled, it is up to the `PORT_MUX` registers as to which peripheral takes control.

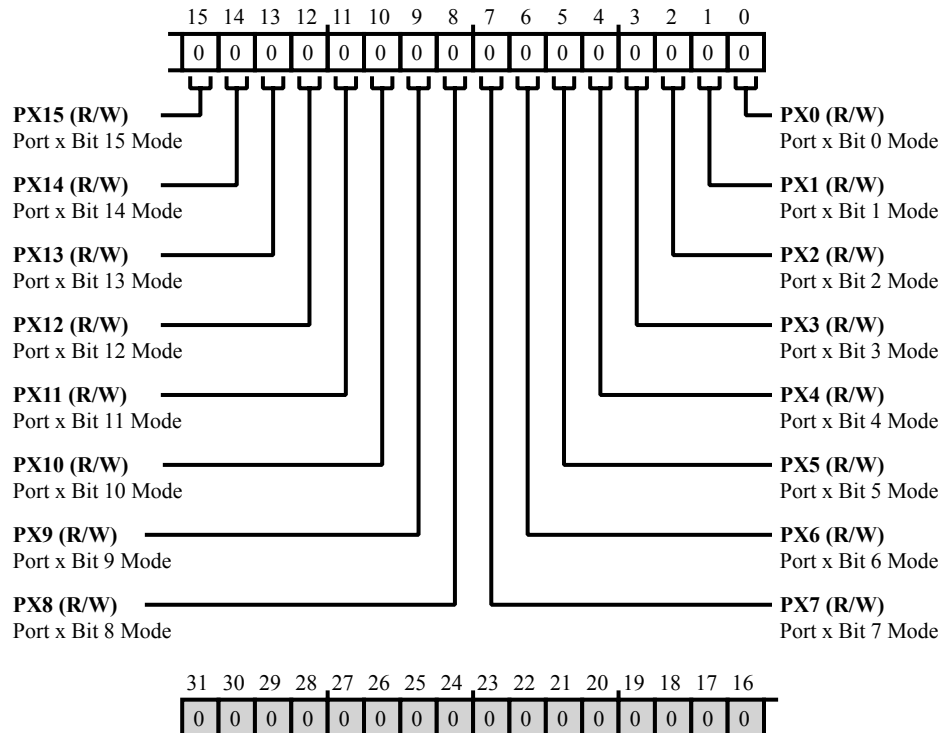


Figure 11-14: PORT_FER Register Diagram

Table 11-17: PORT_FER Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Mode. The <code>PORT_FER.PX15</code> bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
14 (R/W)	PX14	Port x Bit 14 Mode. The <code>PORT_FER.PX14</code> bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode

Table 11-17: PORT_FER Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PX13	Port x Bit 13 Mode. The PORT_FER.PX13 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
12 (R/W)	PX12	Port x Bit 12 Mode. The PORT_FER.PX12 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
11 (R/W)	PX11	Port x Bit 11 Mode. The PORT_FER.PX11 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
10 (R/W)	PX10	Port x Bit 10 Mode. The PORT_FER.PX10 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
9 (R/W)	PX9	Port x Bit 9 Mode. The PORT_FER.PX9 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
8 (R/W)	PX8	Port x Bit 8 Mode. The PORT_FER.PX8 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
7 (R/W)	PX7	Port x Bit 7 Mode. The PORT_FER.PX7 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode

Table 11-17: PORT_FER Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	PX6	Port x Bit 6 Mode. The PORT_FER.PX6 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
5 (R/W)	PX5	Port x Bit 5 Mode. The PORT_FER.PX5 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
4 (R/W)	PX4	Port x Bit 4 Mode. The PORT_FER.PX4 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
3 (R/W)	PX3	Port x Bit 3 Mode. The PORT_FER.PX3 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
2 (R/W)	PX2	Port x Bit 2 Mode. The PORT_FER.PX2 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
1 (R/W)	PX1	Port x Bit 1 Mode. The PORT_FER.PX1 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode
0 (R/W)	PX0	Port x Bit 0 Mode. The PORT_FER.PX0 bit indicates the operating mode for port x.
		0 GPIO Mode
		1 Peripheral Mode

Port x Function Enable Clear Register

The `PORT_FER_CLR` register permits enabling GPIO mode for each bit and corresponding GPIO pin. Writing 1 to a bit in `PORT_FER_CLR` enables GPIO mode for the corresponding pin.

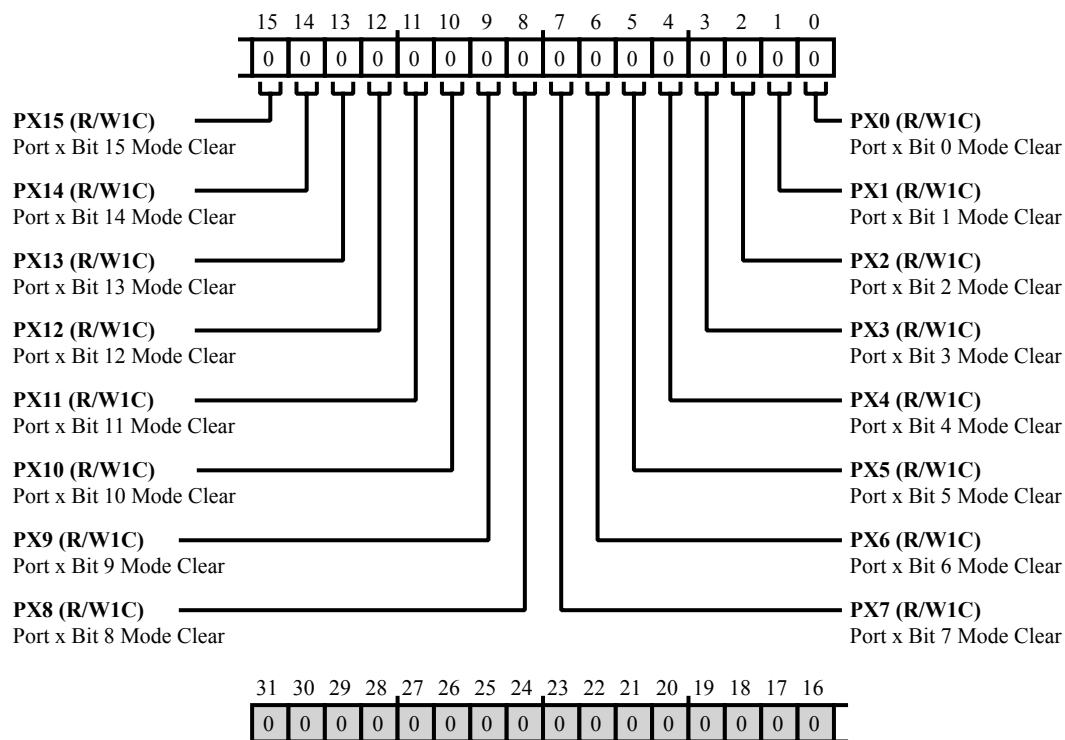


Figure 11-15: `PORT_FER_CLR` Register Diagram

Table 11-18: `PORT_FER_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Mode Clear. The <code>PORT_FER_CLR</code> . PX15 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode
14 (R/W1C)	PX14	Port x Bit 14 Mode Clear. The <code>PORT_FER_CLR</code> . PX14 bit enables GPIO mode.
		0 No Effect
		1 Set Bit for GPIO Mode

Table 11-18: PORT_FER_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
13 (R/W1C)	PX13	Port x Bit 13 Mode Clear. The PORT_FER_CLR.PX13 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
12 (R/W1C)	PX12	Port x Bit 12 Mode Clear. The PORT_FER_CLR.PX12 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
11 (R/W1C)	PX11	Port x Bit 11 Mode Clear. The PORT_FER_CLR.PX11 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
10 (R/W1C)	PX10	Port x Bit 10 Mode Clear. The PORT_FER_CLR.PX10 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
9 (R/W1C)	PX9	Port x Bit 9 Mode Clear. The PORT_FER_CLR.PX9 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
8 (R/W1C)	PX8	Port x Bit 8 Mode Clear. The PORT_FER_CLR.PX8 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
7 (R/W1C)	PX7	Port x Bit 7 Mode Clear. The PORT_FER_CLR.PX7 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode

Table 11-18: PORT_FER_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
6 (R/W1C)	PX6	Port x Bit 6 Mode Clear. The PORT_FER_CLR.PX6 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
5 (R/W1C)	PX5	Port x Bit 5 Mode Clear. The PORT_FER_CLR.PX5 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
4 (R/W1C)	PX4	Port x Bit 4 Mode Clear. The PORT_FER_CLR.PX4 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
3 (R/W1C)	PX3	Port x Bit 3 Mode Clear. The PORT_FER_CLR.PX3 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
2 (R/W1C)	PX2	Port x Bit 2 Mode Clear. The PORT_FER_CLR.PX2 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
1 (R/W1C)	PX1	Port x Bit 1 Mode Clear. The PORT_FER_CLR.PX1 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode
0 (R/W1C)	PX0	Port x Bit 0 Mode Clear. The PORT_FER_CLR.PX0 bit enables GPIO mode.	
		0	No Effect
		1	Set Bit for GPIO Mode

Port x Function Enable Set Register

The `PORT_FER_SET` register permits enabling peripheral mode for each bit and corresponding GPIO pin. Writing 1 to a bit in `PORT_FER_SET` enables peripheral mode for the corresponding pin.

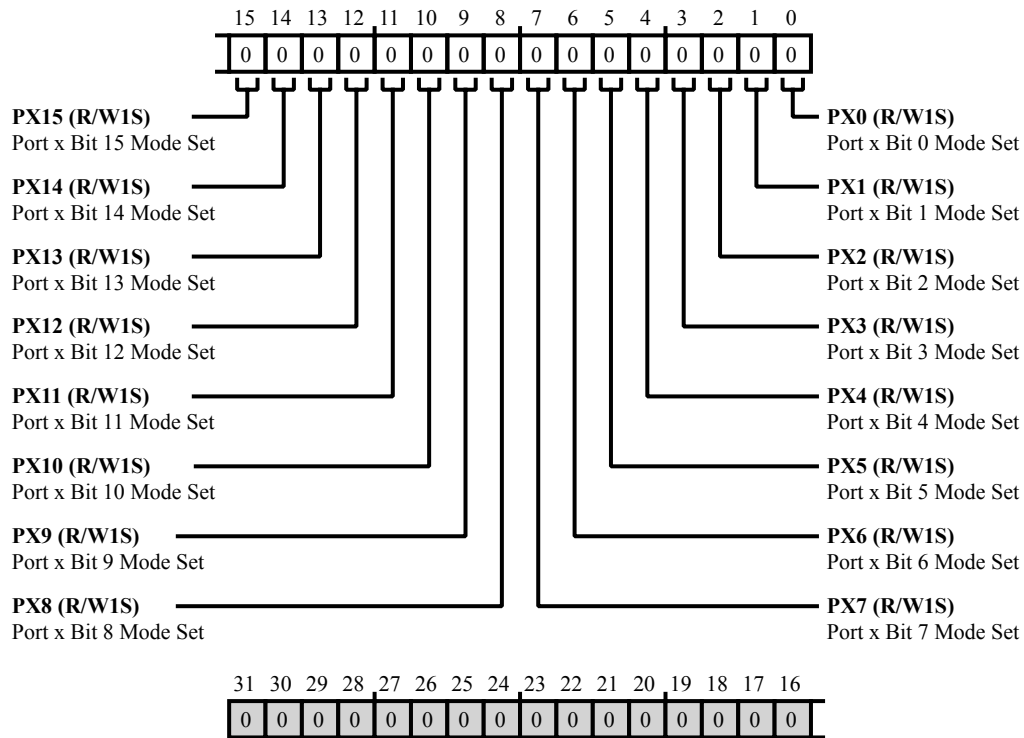


Figure 11-16: `PORT_FER_SET` Register Diagram

Table 11-19: `PORT_FER_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Mode Set. The <code>PORT_FER_SET</code> . PX15 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
14 (R/W1S)	PX14	Port x Bit 14 Mode Set. The <code>PORT_FER_SET</code> . PX14 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode

Table 11-19: PORT_FER_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
13 (R/W1S)	PX13	Port x Bit 13 Mode Set. The PORT_FER_SET . PX13 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
12 (R/W1S)	PX12	Port x Bit 12 Mode Set. The PORT_FER_SET . PX12 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
11 (R/W1S)	PX11	Port x Bit 11 Mode Set. The PORT_FER_SET . PX11 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
10 (R/W1S)	PX10	Port x Bit 10 Mode Set. The PORT_FER_SET . PX10 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
9 (R/W1S)	PX9	Port x Bit 9 Mode Set. The PORT_FER_SET . PX9 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
8 (R/W1S)	PX8	Port x Bit 8 Mode Set. The PORT_FER_SET . PX8 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode
7 (R/W1S)	PX7	Port x Bit 7 Mode Set. The PORT_FER_SET . PX7 bit enables peripheral mode.	
		0	No Effect
		1	Set Bit for Peripheral Mode

Table 11-19: PORT_FER_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1S)	PX6	Port x Bit 6 Mode Set. The PORT_FER_SET . PX6 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
5 (R/W1S)	PX5	Port x Bit 5 Mode Set. The PORT_FER_SET . PX5 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
4 (R/W1S)	PX4	Port x Bit 4 Mode Set. The PORT_FER_SET . PX4 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
3 (R/W1S)	PX3	Port x Bit 3 Mode Set. The PORT_FER_SET . PX3 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
2 (R/W1S)	PX2	Port x Bit 2 Mode Set. The PORT_FER_SET . PX2 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
1 (R/W1S)	PX1	Port x Bit 1 Mode Set. The PORT_FER_SET . PX1 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode
0 (R/W1S)	PX0	Port x Bit 0 Mode Set. The PORT_FER_SET . PX0 bit enables peripheral mode.
		0 No Effect
		1 Set Bit for Peripheral Mode

Port x GPIO Input Enable Register

The `PORT_INEN`, `PORT_INEN_SET`, and `PORT_INEN_CLR` registers enable or disable input drivers, which are required for using a GPIO pin in input mode.

Writes to the `PORT_INEN` register affect the input drivers for all pins of the port. To set or clear specific pin drivers without impacting other pin drivers of the port, use the `PORT_INEN_SET` and `PORT_INEN_CLR` registers.

If the input is enabled, reads from the `PORT_DATA`, `PORT_DATA_SET`, or `PORT_DATA_CLR` registers return the state of the pins. However, the state of the output is not overwritten by the input. It is altered by software writes only. Input and output drivers can be enabled at the same time. In this case, a read of the data register returns the true value of the data register and not the pin state.

For more information, see the `PORT_DATA` register description and the `PORT_DIR` register description.

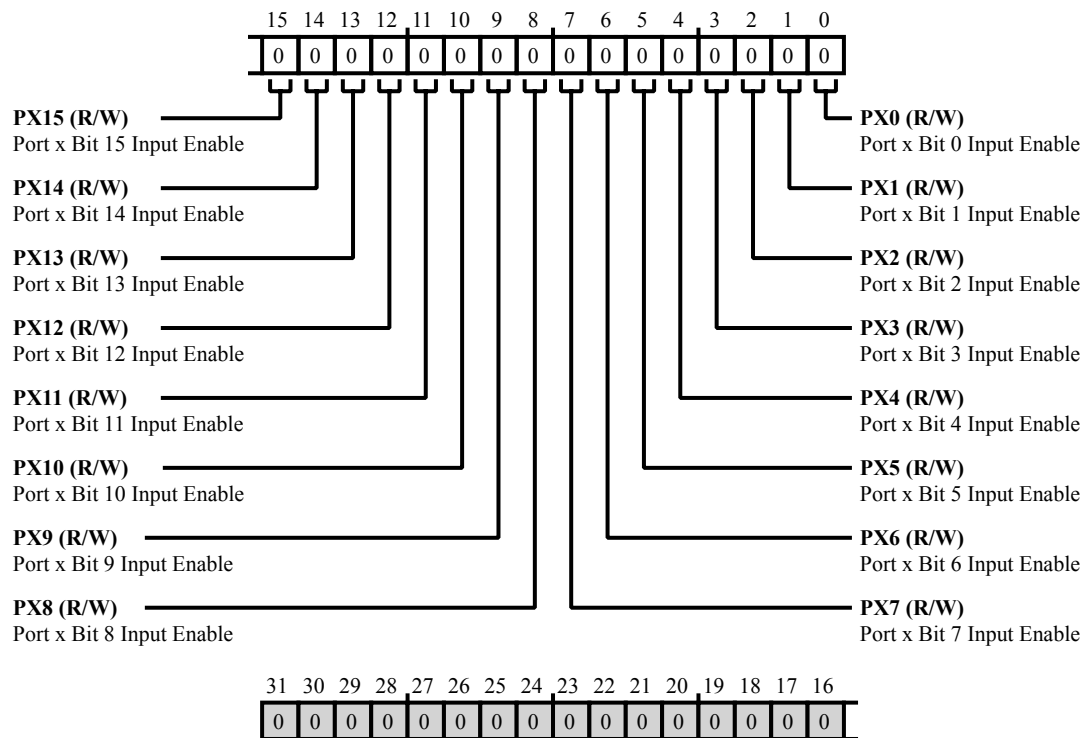


Figure 11-17: PORT_INEN Register Diagram

Table 11-20: PORT_INEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Input Enable.
		0 Disable Input Driver
		1 Enable Input Driver

Table 11-20: PORT_INEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
14 (R/W)	PX14	Port x Bit 14 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
13 (R/W)	PX13	Port x Bit 13 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
12 (R/W)	PX12	Port x Bit 12 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
11 (R/W)	PX11	Port x Bit 11 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
10 (R/W)	PX10	Port x Bit 10 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
9 (R/W)	PX9	Port x Bit 9 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
8 (R/W)	PX8	Port x Bit 8 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
7 (R/W)	PX7	Port x Bit 7 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
6 (R/W)	PX6	Port x Bit 6 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
5 (R/W)	PX5	Port x Bit 5 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver

Table 11-20: PORT_INEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
4 (R/W)	PX4	Port x Bit 4 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
3 (R/W)	PX3	Port x Bit 3 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
2 (R/W)	PX2	Port x Bit 2 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
1 (R/W)	PX1	Port x Bit 1 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver
0 (R/W)	PX0	Port x Bit 0 Input Enable.	
		0	Disable Input Driver
		1	Enable Input Driver

Port x GPIO Input Enable Clear Register

The `PORT_INEN_CLR` register disables the input drivers for GPIO pins. For more information, see the `PORT_INEN` register description.

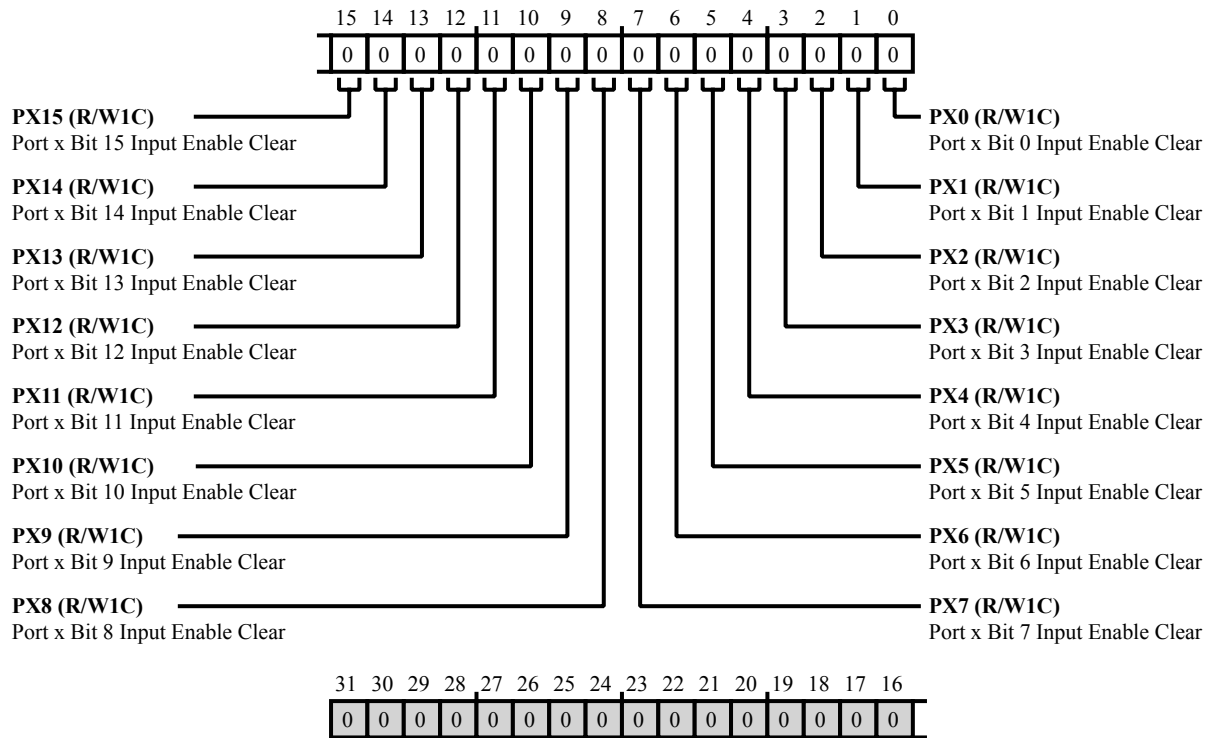


Figure 11-18: `PORT_INEN_CLR` Register Diagram

Table 11-21: `PORT_INEN_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Input Enable Clear.
		0 No Effect
		1 Clear Bit. Set to disable the input driver.
14 (R/W1C)	PX14	Port x Bit 14 Input Enable Clear.
		0 No Effect
		1 Clear Bit. Set to disable the input driver.
13 (R/W1C)	PX13	Port x Bit 13 Input Enable Clear.
		0 No Effect
		1 Clear Bit. Set to disable the input driver.

Table 11-21: PORT_INEN_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/W1C)	PX12	Port x Bit 12 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
11 (R/W1C)	PX11	Port x Bit 11 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
10 (R/W1C)	PX10	Port x Bit 10 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
9 (R/W1C)	PX9	Port x Bit 9 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
8 (R/W1C)	PX8	Port x Bit 8 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
7 (R/W1C)	PX7	Port x Bit 7 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
6 (R/W1C)	PX6	Port x Bit 6 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
5 (R/W1C)	PX5	Port x Bit 5 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
4 (R/W1C)	PX4	Port x Bit 4 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
3 (R/W1C)	PX3	Port x Bit 3 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.

Table 11-21: PORT_INEN_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W1C)	PX2	Port x Bit 2 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
1 (R/W1C)	PX1	Port x Bit 1 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.
0 (R/W1C)	PX0	Port x Bit 0 Input Enable Clear.	
		0	No Effect
		1	Clear Bit. Set to disable the input driver.

Port x GPIO Input Enable Set Register

The `PORT_INEN_SET` register enables input drivers for GPIO pins. For more information, see the `PORT_INEN` register description.

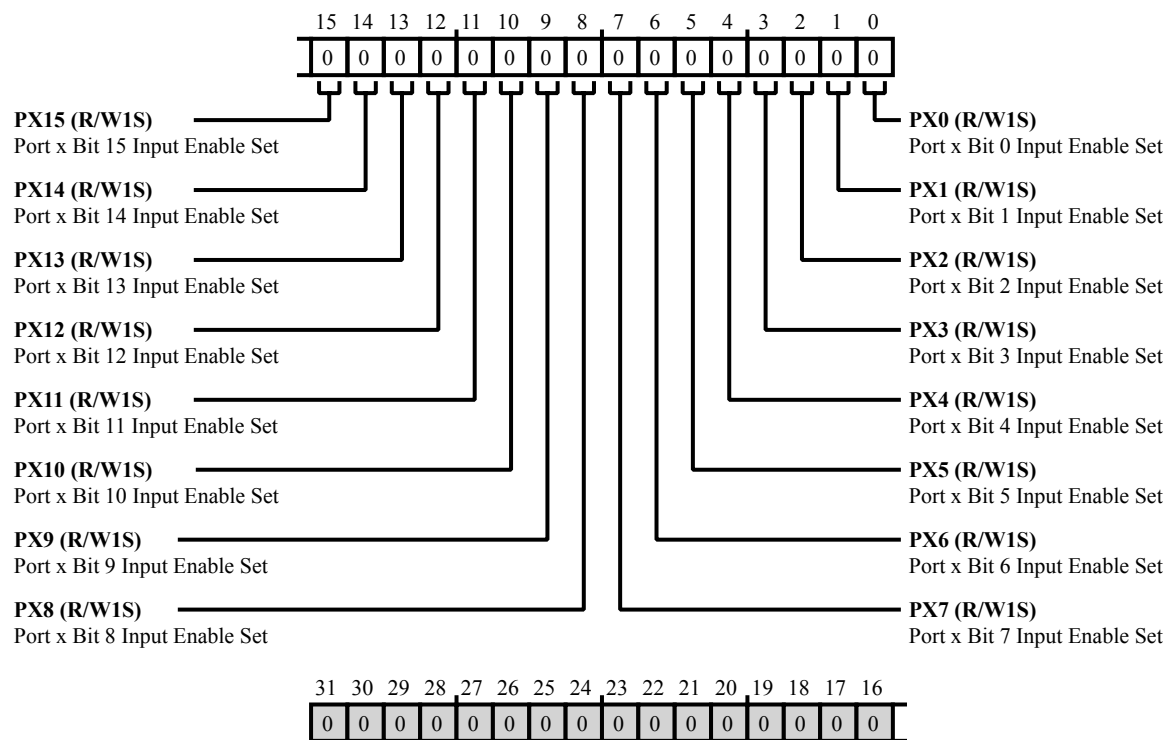


Figure 11-19: `PORT_INEN_SET` Register Diagram

Table 11-22: `PORT_INEN_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Input Enable Set.
		0 No Effect
		1 Set Bit. Set to enable the input driver.
14 (R/W1S)	PX14	Port x Bit 14 Input Enable Set.
		0 No Effect
		1 Set Bit. Set to enable the input driver.
13 (R/W1S)	PX13	Port x Bit 13 Input Enable Set.
		0 No Effect
		1 Set Bit. Set to enable the input driver.

Table 11-22: PORT_INEN_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/W1S)	PX12	Port x Bit 12 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
11 (R/W1S)	PX11	Port x Bit 11 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
10 (R/W1S)	PX10	Port x Bit 10 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
9 (R/W1S)	PX9	Port x Bit 9 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
8 (R/W1S)	PX8	Port x Bit 8 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
7 (R/W1S)	PX7	Port x Bit 7 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
6 (R/W1S)	PX6	Port x Bit 6 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
5 (R/W1S)	PX5	Port x Bit 5 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
4 (R/W1S)	PX4	Port x Bit 4 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
3 (R/W1S)	PX3	Port x Bit 3 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.

Table 11-22: PORT_INEN_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W1S)	PX2	Port x Bit 2 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
1 (R/W1S)	PX1	Port x Bit 1 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.
0 (R/W1S)	PX0	Port x Bit 0 Input Enable Set.	
		0	No Effect
		1	Set Bit. Set to enable the input driver.

Port x GPIO Lock Register

The `PORT_LOCK` register enables (unlocks) or disables (locks) write access selectively for the PORT control registers.

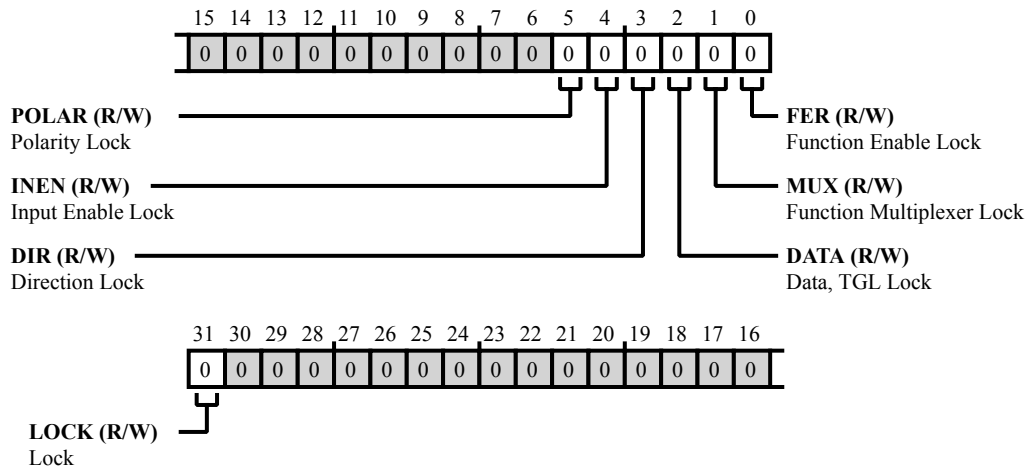


Figure 11-20: PORT_LOCK Register Diagram

Table 11-23: PORT_LOCK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Lock. If the global lock bit is set (<code>SPU_CTL.GLCK</code> bit =1) and the <code>PORT_LOCK.LOCK</code> bit is set, the <code>PORT_LOCK</code> register is read only (locked).
		0 Unlock
		1 Lock
5 (R/W)	POLAR	Polarity Lock. The <code>PORT_LOCK.POLAR</code> disables write access to the <code>PORT_POL</code> , <code>PORT_POL_SET</code> , and <code>PORT_POL_CLR</code> registers.
		0 Unlock POL
		1 Lock POL
4 (R/W)	INEN	Input Enable Lock. The <code>PORT_LOCK.INEN</code> disables write access to the <code>PORT_INEN</code> , <code>PORT_INEN_SET</code> , and <code>PORT_INEN_CLR</code> registers.
		0 Unlock INEN
		1 Lock INEN

Table 11-23: PORT_LOCK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	DIR	Direction Lock. The PORT_LOCK.DIR disables write access to the PORT_DIR, PORT_DIR_SET, PORT_DIR_CLR registers.
		0 Unlock DIR
		1 Lock DIR
2 (R/W)	DATA	Data, TGL Lock. The PORT_LOCK.DATA disables write access to the PORT_DATA, PORT_DATA_SET, PORT_DATA_CLR, and PORT_DATA_TGL registers.
		0 Unlock DATA
		1 Lock DATA
1 (R/W)	MUX	Function Multiplexer Lock. The PORT_LOCK.MUX disables write accesses to the PORT_MUX register.
		0 Unlock MUX
		1 Lock MUX
0 (R/W)	FER	Function Enable Lock. The PORT_LOCK.FER disables write access to the PORT_FER, PORT_FER_SET, and PORT_FER_CLR registers.
		0 Unlock FER
		1 Lock FER

Port x Multiplexer Control Register

When a pin is in peripheral mode (not GPIO mode), the `PORT_MUX` register controls which peripheral takes ownership of a pin. Ports may have multiple, different peripheral functions. Two bits are required to describe every multiplexer on an individual pin-by-pin scheme. For example, bit 0 and bit 1 of the `PORT_MUX` register control the multiplexer of pin 0, bit 2 and bit 3 of `PORT_MUX` control the multiplexer of pin 1, and so on. The value of any `PORT_MUX` bit has no effect on the port pins when the associated bit in the `PORT_FER` register is 0 (selects GPIO mode). Even if a port has only one function, the `PORT_MUX` register is still present. For single function ports (no multiplexing is needed), leave the `PORT_MUX` bits at 0 (default). For all `PORT_MUX` bit fields: 00 = default/reset peripheral option, 01 = first alternate peripheral option, 10 = second alternate peripheral option, and 11 = third alternate peripheral option.

See the processor data sheet for details regarding the peripheral options associated with each port.

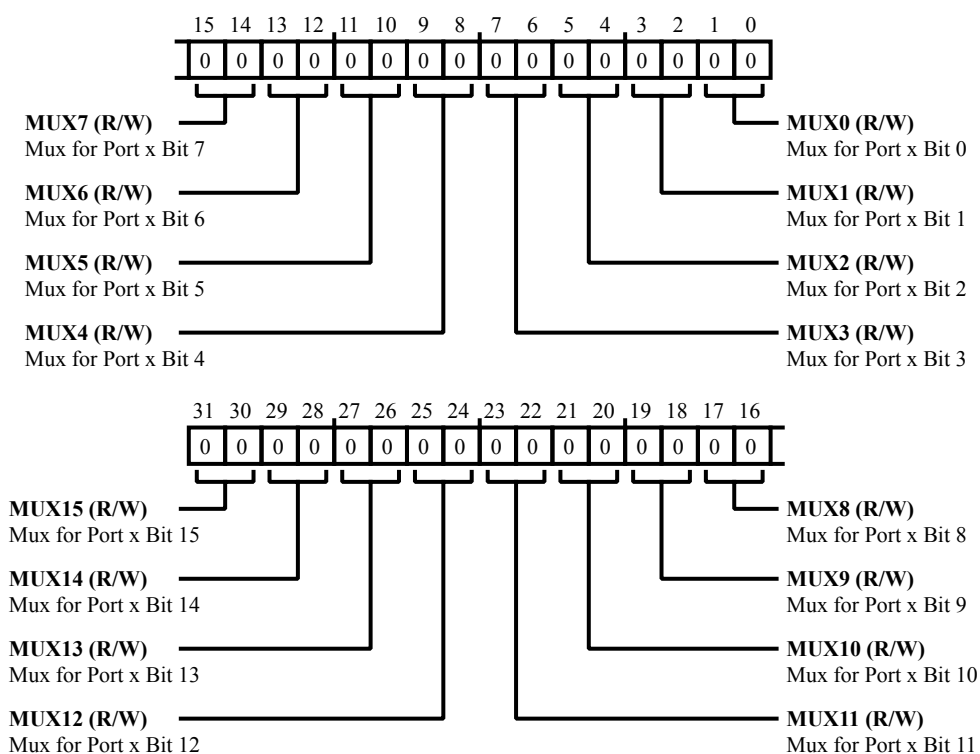


Figure 11-21: `PORT_MUX` Register Diagram

Table 11-24: `PORT_MUX` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:30 (R/W)	MUX15	Mux for Port x Bit 15. The <code>PORT_MUX.MUX15</code> bit provides multiplexer control for port x bit 15.

Table 11-24: PORT_MUX Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29:28 (R/W)	MUX14	Mux for Port x Bit 14. The PORT_MUX.MUX14 bit provides multiplexer control for port x bit 14.
27:26 (R/W)	MUX13	Mux for Port x Bit 13. The PORT_MUX.MUX13 bit provides multiplexer control for port x bit 13.
25:24 (R/W)	MUX12	Mux for Port x Bit 12. The PORT_MUX.MUX12 bit provides multiplexer control for port x bit 12.
23:22 (R/W)	MUX11	Mux for Port x Bit 11. The PORT_MUX.MUX11 bit provides multiplexer control for port x bit 11.
21:20 (R/W)	MUX10	Mux for Port x Bit 10. The PORT_MUX.MUX10 bit provides multiplexer control for port x bit 10.
19:18 (R/W)	MUX9	Mux for Port x Bit 9. The PORT_MUX.MUX9 bit provides multiplexer control for port x bit 9.
17:16 (R/W)	MUX8	Mux for Port x Bit 8. The PORT_MUX.MUX8 bit provides multiplexer control for port x bit 8.
15:14 (R/W)	MUX7	Mux for Port x Bit 7. The PORT_MUX.MUX7 bit provides multiplexer control for port x bit 7.
13:12 (R/W)	MUX6	Mux for Port x Bit 6. The PORT_MUX.MUX6 bit provides multiplexer control for port x bit 6.
11:10 (R/W)	MUX5	Mux for Port x Bit 5. The PORT_MUX.MUX5 bit provides multiplexer control for port x bit 5.
9:8 (R/W)	MUX4	Mux for Port x Bit 4. The PORT_MUX.MUX4 bit provides multiplexer control for port x bit 4.
7:6 (R/W)	MUX3	Mux for Port x Bit 3. The PORT_MUX.MUX3 bit provides multiplexer control for port x bit 3.
5:4 (R/W)	MUX2	Mux for Port x Bit 2. The PORT_MUX.MUX2 bit provides multiplexer control for port x bit 2.
3:2 (R/W)	MUX1	Mux for Port x Bit 1. The PORT_MUX.MUX1 bit provides multiplexer control for port x bit 1.
1:0 (R/W)	MUX0	Mux for Port x Bit 0. The PORT_MUX.MUX0 bit provides multiplexer control for port x bit 0.

Port x GPIO Polarity Invert Register

The `PORT_POL`, `PORT_POL_SET`, and `PORT_POL_CLR` registers enable or disable inverting polarity of GPIO signals. To invert polarity of peripheral signals, use the inversion selection programming in the signal's corresponding module.

Writes to the `PORT_POL` register affect the polarity inversion selection of all pins of the port. To enable or disable polarity inversion for specific pins without impacting other pins of the port, use the `PORT_POL_SET` and `PORT_POL_CLR` registers.

Setting a bit in the `PORT_POL` register enables polarity inversion on the corresponding inversion GPIO pin, making the pin active-low or falling-edge sensitive. Clearing a bit in the `PORT_POL` register disables polarity (default state) on the corresponding GPIO pin, making it active-high or rising-edge sensitive.

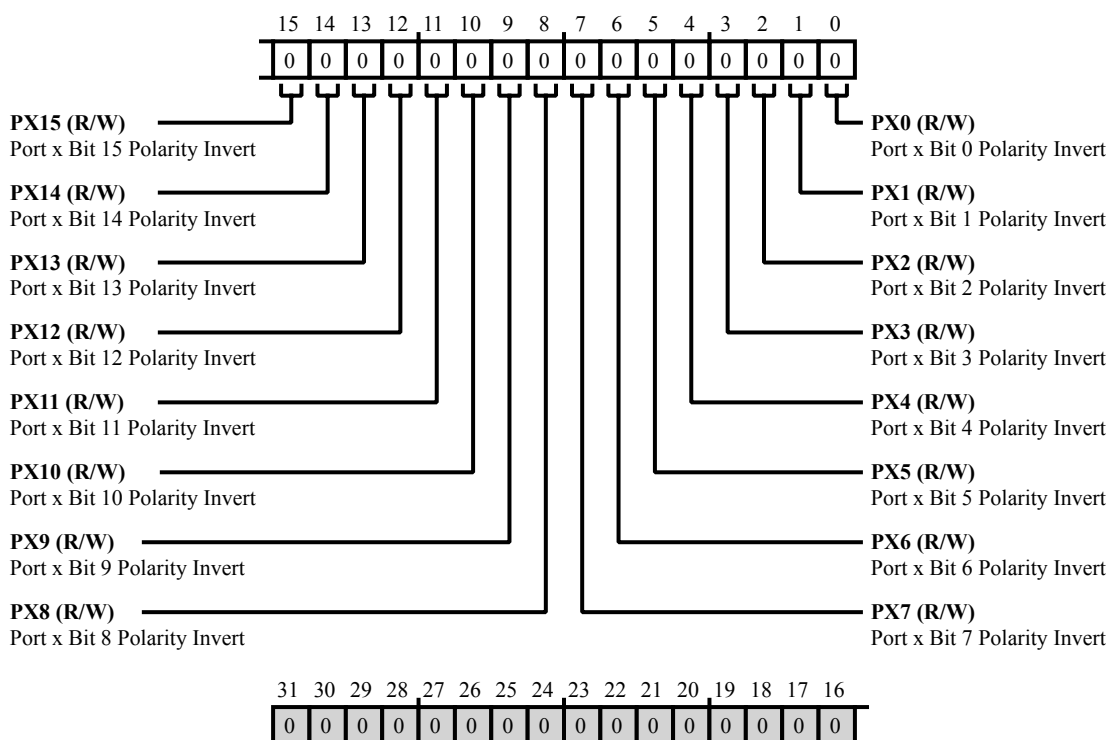


Figure 11-22: `PORT_POL` Register Diagram

Table 11-25: `PORT_POL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	Port x Bit 15 Polarity Invert. The <code>PORT_POL.PX15</code> bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.

Table 11-25: PORT_POL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	PX14	Port x Bit 14 Polarity Invert. The PORT_POL.PX14 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
13 (R/W)	PX13	Port x Bit 13 Polarity Invert. The PORT_POL.PX13 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
12 (R/W)	PX12	Port x Bit 12 Polarity Invert. The PORT_POL.PX12 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
11 (R/W)	PX11	Port x Bit 11 Polarity Invert. The PORT_POL.PX11 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
10 (R/W)	PX10	Port x Bit 10 Polarity Invert. The PORT_POL.PX10 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
9 (R/W)	PX9	Port x Bit 9 Polarity Invert. The PORT_POL.PX9 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.
8 (R/W)	PX8	Port x Bit 8 Polarity Invert. The PORT_POL.PX8 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.

Table 11-25: PORT_POL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
7 (R/W)	PX7	Port x Bit 7 Polarity Invert. The PORT_POL.PX7 bit enables polarity inversion.	
		0	No Invert. GPIO is active high or rising edge sensitive.
		1	Invert. GPIO is active low or falling edge sensitive.
6 (R/W)	PX6	Port x Bit 6 Polarity Invert. The PORT_POL.PX6 bit enables polarity inversion.	
		0	No Invert. GPIO is active high or rising edge sensitive.
		1	Invert. GPIO is active low or falling edge sensitive.
5 (R/W)	PX5	Port x Bit 5 Polarity Invert. The PORT_POL.PX5 bit enables polarity inversion.	
		0	No Invert. GPIO is active high or rising edge sensitive.
		1	Invert. GPIO is active low or falling edge sensitive.
4 (R/W)	PX4	Port x Bit 4 Polarity Invert. The PORT_POL.PX4 bit enables polarity inversion.	
		0	No Invert. GPIO is active high or rising edge sensitive.
		1	Invert. GPIO is active low or falling edge sensitive.
3 (R/W)	PX3	Port x Bit 3 Polarity Invert. The PORT_POL.PX3 bit enables polarity inversion.	
		0	No Invert. GPIO is active high or rising edge sensitive.
		1	Invert. GPIO is active low or falling edge sensitive.
2 (R/W)	PX2	Port x Bit 2 Polarity Invert. The PORT_POL.PX2 bit enables polarity inversion.	
		0	No Invert. GPIO is active high or rising edge sensitive.
		1	Invert. GPIO is active low or falling edge sensitive.
1 (R/W)	PX1	Port x Bit 1 Polarity Invert. The PORT_POL.PX1 bit enables polarity inversion.	
		0	No Invert. GPIO is active high or rising edge sensitive.
		1	Invert. GPIO is active low or falling edge sensitive.

Table 11-25: PORT_POL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	PX0	Port x Bit 0 Polarity Invert. The PORT_POL.PX0 bit enables polarity inversion.
		0 No Invert. GPIO is active high or rising edge sensitive.
		1 Invert. GPIO is active low or falling edge sensitive.

Port x GPIO Polarity Invert Clear Register

The `PORT_POL_CLR` register disables polarity inversion for GPIO pins. For more information, see the `PORT_POL` register description.

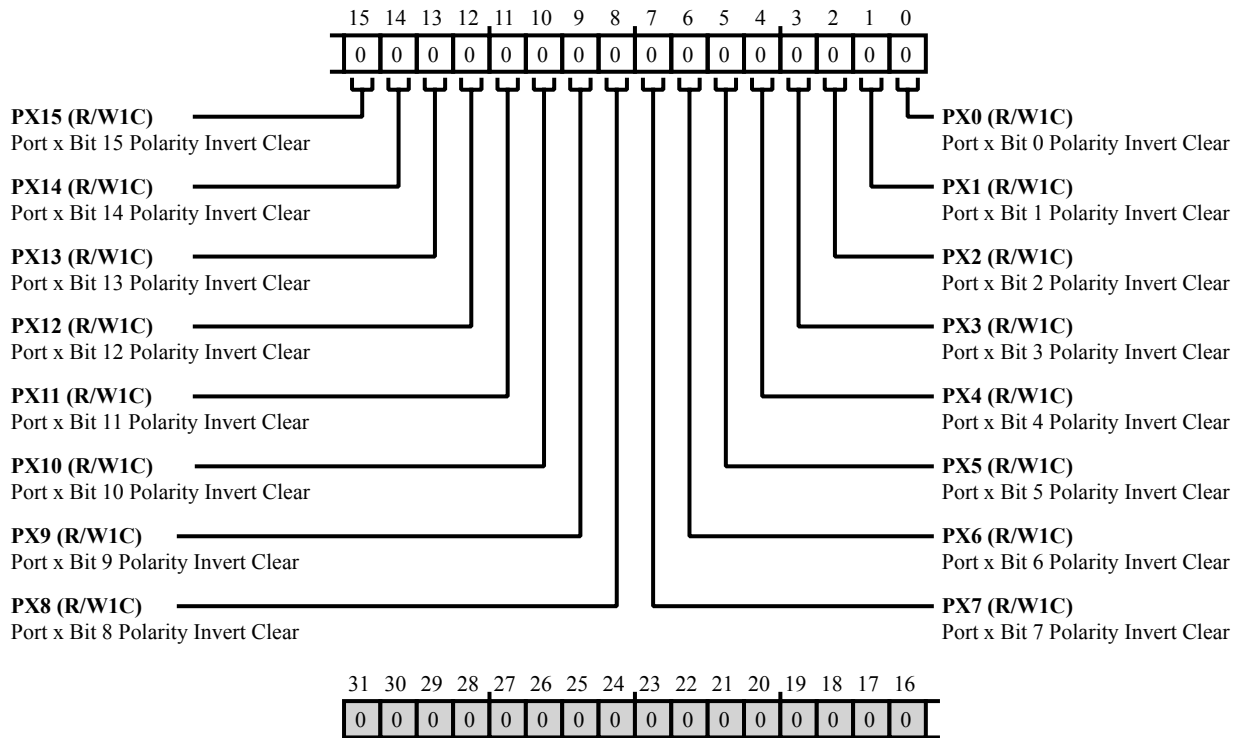


Figure 11-23: `PORT_POL_CLR` Register Diagram

Table 11-26: `PORT_POL_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PX15	Port x Bit 15 Polarity Invert Clear.
		0 No Effect
		1 Clear Bit. Set to disable GPIO pin polarity invert.
14 (R/W1C)	PX14	Port x Bit 14 Polarity Invert Clear.
		0 No Effect
		1 Clear Bit. Set to disable GPIO pin polarity invert.
13 (R/W1C)	PX13	Port x Bit 13 Polarity Invert Clear.
		0 No Effect
		1 Clear Bit. Set to disable GPIO pin polarity invert.

Table 11-26: PORT_POL_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
12 (R/W1C)	PX12	Port x Bit 12 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
11 (R/W1C)	PX11	Port x Bit 11 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
10 (R/W1C)	PX10	Port x Bit 10 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
9 (R/W1C)	PX9	Port x Bit 9 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
8 (R/W1C)	PX8	Port x Bit 8 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
7 (R/W1C)	PX7	Port x Bit 7 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
6 (R/W1C)	PX6	Port x Bit 6 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
5 (R/W1C)	PX5	Port x Bit 5 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
4 (R/W1C)	PX4	Port x Bit 4 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
3 (R/W1C)	PX3	Port x Bit 3 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.

Table 11-26: PORT_POL_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W1C)	PX2	Port x Bit 2 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
1 (R/W1C)	PX1	Port x Bit 1 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.
0 (R/W1C)	PX0	Port x Bit 0 Polarity Invert Clear.	
		0	No Effect
		1	Clear Bit. Set to disable GPIO pin polarity invert.

Port x GPIO Polarity Invert Set Register

The `PORT_POL_SET` register enables polarity inversion for GPIO pins. For more information, see the `PORT_POL` register description.

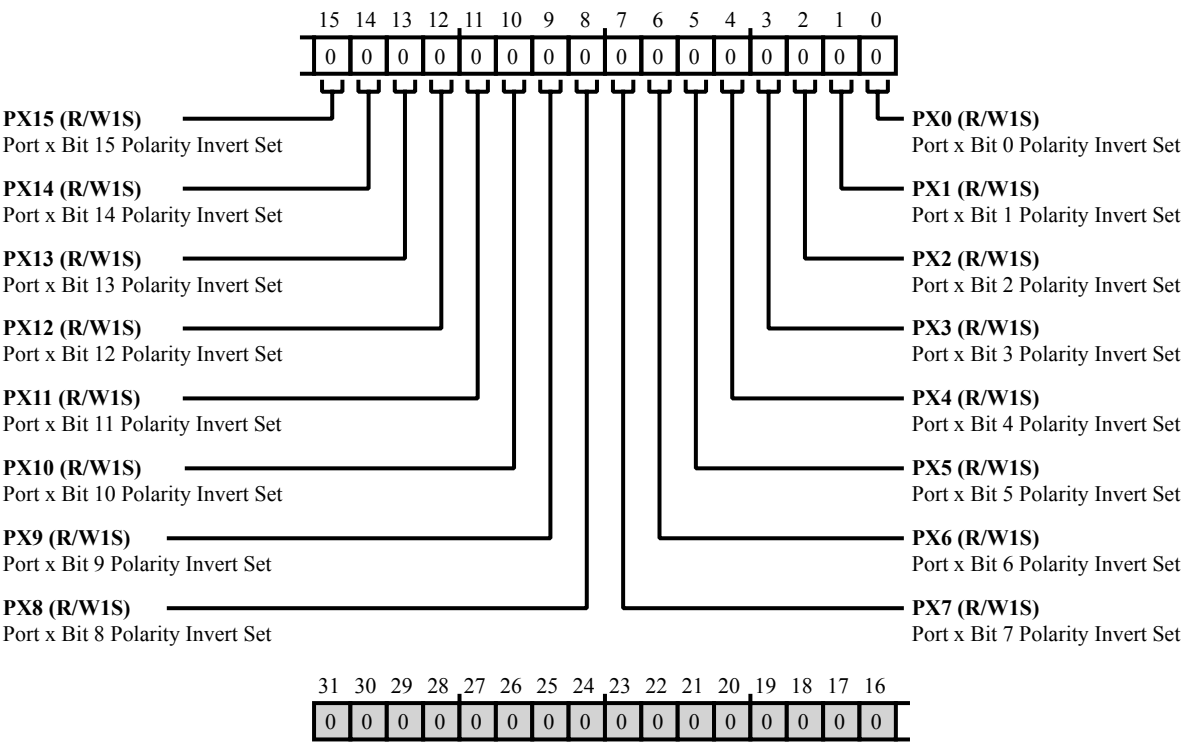


Figure 11-24: `PORT_POL_SET` Register Diagram

Table 11-27: `PORT_POL_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PX15	Port x Bit 15 Polarity Invert Set. The <code>PORT_POL_SET</code> . <code>PX15</code> bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
14 (R/W1S)	PX14	Port x Bit 14 Polarity Invert Set. The <code>PORT_POL_SET</code> . <code>PX14</code> bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.

Table 11-27: PORT_POL_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1S)	PX13	Port x Bit 13 Polarity Invert Set. The PORT_POL_SET . PX13 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
12 (R/W1S)	PX12	Port x Bit 12 Polarity Invert Set. The PORT_POL_SET . PX12 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
11 (R/W1S)	PX11	Port x Bit 11 Polarity Invert Set. The PORT_POL_SET . PX11 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
10 (R/W1S)	PX10	Port x Bit 10 Polarity Invert Set. The PORT_POL_SET . PX10 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
9 (R/W1S)	PX9	Port x Bit 9 Polarity Invert Set. The PORT_POL_SET . PX9 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
8 (R/W1S)	PX8	Port x Bit 8 Polarity Invert Set. The PORT_POL_SET . PX8 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
7 (R/W1S)	PX7	Port x Bit 7 Polarity Invert Set. The PORT_POL_SET . PX7 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.

Table 11-27: PORT_POL_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1S)	PX6	Port x Bit 6 Polarity Invert Set. The PORT_POL_SET . PX6 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
5 (R/W1S)	PX5	Port x Bit 5 Polarity Invert Set. The PORT_POL_SET . PX5 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
4 (R/W1S)	PX4	Port x Bit 4 Polarity Invert Set. The PORT_POL_SET . PX4 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
3 (R/W1S)	PX3	Port x Bit 3 Polarity Invert Set. The PORT_POL_SET . PX3 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
2 (R/W1S)	PX2	Port x Bit 2 Polarity Invert Set. The PORT_POL_SET . PX2 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
1 (R/W1S)	PX1	Port x Bit 1 Polarity Invert Set. The PORT_POL_SET . PX1 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.
0 (R/W1S)	PX0	Port x Bit 0 Polarity Invert Set. The PORT_POL_SET . PX0 bit enables pin polarity inversion.
		0 No Effect
		1 Set Bit. Set to enable GPIO pin polarity invert.

Port x GPIO Trigger Toggle Register

The `PORT_TRIG_TGL` register permits toggling the state of output GPIO pins in response to a trigger from the TRU for the corresponding port. Setting bits in the `PORT_TRIG_TGL` register enables triggers to toggle the state of those specific pins without impacting other pins of the port.

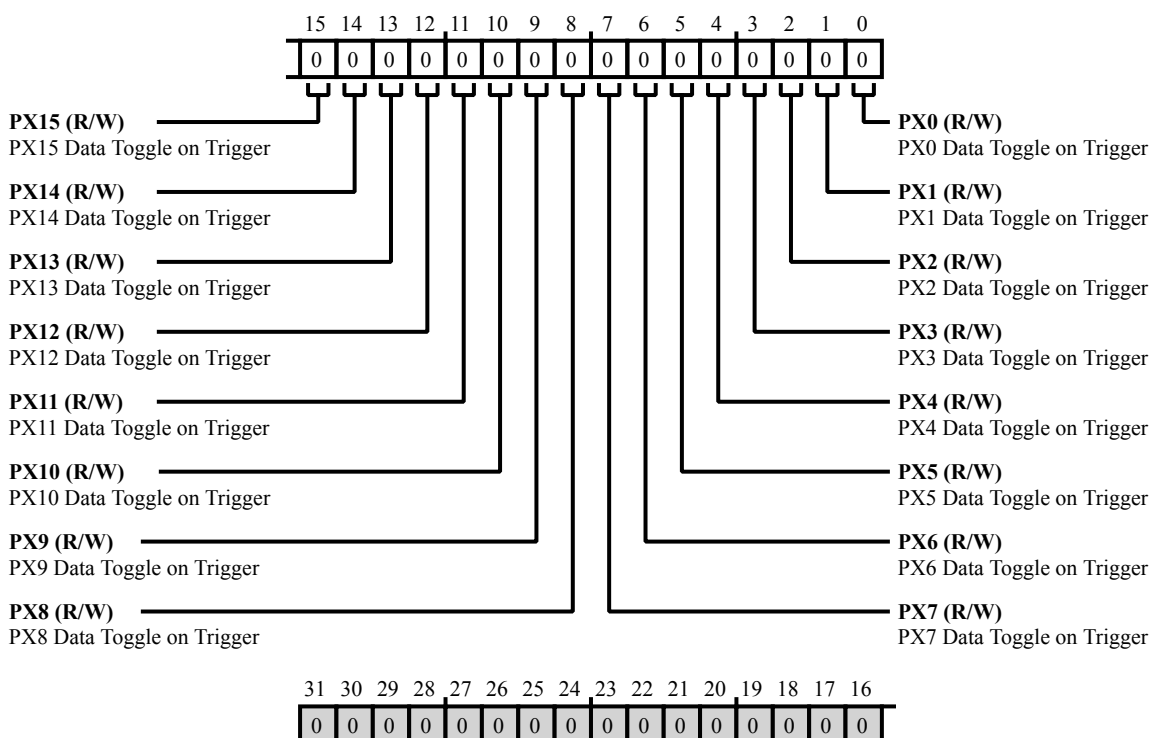


Figure 11-25: PORT_TRIG_TGL Register Diagram

Table 11-28: PORT_TRIG_TGL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	PX15	PX15 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX15 bit enables triggers to toggle the state of the pin.
14 (R/W)	PX14	PX14 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX14 bit enables triggers to toggle the state of the pin.
13 (R/W)	PX13	PX13 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX13 bit enables triggers to toggle the state of the pin.
12 (R/W)	PX12	PX12 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX12 bit enables triggers to toggle the state of the pin.
11 (R/W)	PX11	PX11 Data Toggle on Trigger. The <code>PORT_TRIG_TGL</code> . PX11 bit enables triggers to toggle the state of the pin.

Table 11-28: PORT_TRIG_TGL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	PX10	PX10 Data Toggle on Trigger. The PORT_TRIG_TGL.PX10 bit enables triggers to toggle the state of the pin.
9 (R/W)	PX9	PX9 Data Toggle on Trigger. The PORT_TRIG_TGL.PX9 bit enables triggers to toggle the state of the pin.
8 (R/W)	PX8	PX8 Data Toggle on Trigger. The PORT_TRIG_TGL.PX8 bit enables triggers to toggle the state of the pin.
7 (R/W)	PX7	PX7 Data Toggle on Trigger. The PORT_TRIG_TGL.PX7 bit enables triggers to toggle the state of the pin.
6 (R/W)	PX6	PX6 Data Toggle on Trigger. The PORT_TRIG_TGL.PX6 bit enables triggers to toggle the state of the pin.
5 (R/W)	PX5	PX5 Data Toggle on Trigger. The PORT_TRIG_TGL.PX5 bit enables triggers to toggle the state of the pin.
4 (R/W)	PX4	PX4 Data Toggle on Trigger. The PORT_TRIG_TGL.PX4 bit enables triggers to toggle the state of the pin.
3 (R/W)	PX3	PX3 Data Toggle on Trigger. The PORT_TRIG_TGL.PX3 bit enables triggers to toggle the state of the pin.
2 (R/W)	PX2	PX2 Data Toggle on Trigger. The PORT_TRIG_TGL.PX2 bit enables triggers to toggle the state of the pin.
1 (R/W)	PX1	PX1 Data Toggle on Trigger. The PORT_TRIG_TGL.PX1 bit enables triggers to toggle the state of the pin.
0 (R/W)	PX0	PX0 Data Toggle on Trigger. The PORT_TRIG_TGL.PX0 bit enables triggers to toggle the state of the pin.

ADSP-2159x PINT Register Descriptions

The Pin Interrupt module (PINT) contains the following registers.

Table 11-29: ADSP-2159x PINT Register List

Name	Description
PINT_ASSIGN	PINT Assign Register
PINT_EDGE_CLR	PINT Edge Clear Register
PINT_EDGE_SET	PINT Edge Set Register
PINT_INV_CLR	PINT Invert Clear Register

Table 11-29: ADSP-2159x PINT Register List (Continued)

Name	Description
PINT_INV_SET	PINT Invert Set Register
PINT_LATCH	PINT Latch Register
PINT_MSK_CLR	PINT Mask Clear Register
PINT_MSK_SET	PINT Mask Set Register
PINT_PINSTATE	PINT Pin State Register
PINT_REQ	PINT Request Register

PINT Assign Register

The `PINT_ASSIGN` register controls the pin-to-interrupt request assignment in a byte-wide manner. This register consists of four control bytes that each function as a multiplexer control.

The PINT ports are subdivided into 8-bit half ports, resulting in lower and upper half 8-bit units. Using the multiplexers controlled by the `PINT_ASSIGN` register, the lower half units of eight pins can be forwarded to either byte 0 or byte 2 of either associated PINT block. The upper half units can be forwarded to either byte 1 or byte 3 of the PINT block, without further restrictions.

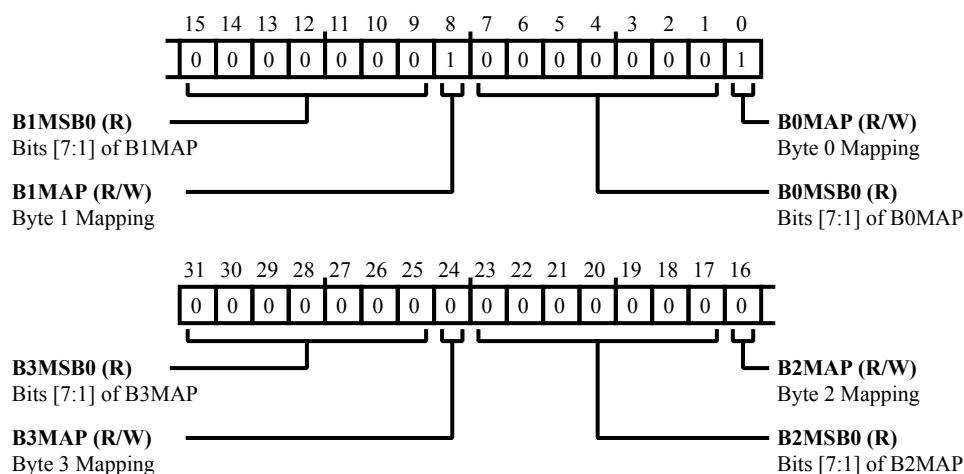


Figure 11-26: PINT_ASSIGN Register Diagram

Table 11-30: PINT_ASSIGN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:25 (R/NW)	B3MSB0	Bits [7:1] of B3MAP.
24 (R/W)	B3MAP	Byte 3 Mapping.
		0 B3MAP_PAH. Byte 3 = PA.H
		1 B3MAP_PBH. Byte 3 = PB.H
23:17 (R/NW)	B2MSB0	Bits [7:1] of B2MAP.
16 (R/W)	B2MAP	Byte 2 Mapping.
		0 B2MAP_PAL. Byte 2 = PA.L
		1 B2MAP_PBL. Byte 2 = PB.L
15:9 (R/NW)	B1MSB0	Bits [7:1] of B1MAP.

Table 11-30: PINT_ASSIGN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8 (R/W)	B1MAP	Byte 1 Mapping.	
		0	B1MAP_PAH. Byte 1 = PA.H
		1	B1MAP_PBH. Byte 1 = PB.H
7:1 (R/NW)	B0MSB0	Bits [7:1] of B0MAP.	
0 (R/W)	B0MAP	Byte 0 Mapping.	
		0	B0MAP_PAL. Byte 0 = PA.L
		1	B0MAP_PBL. Byte 0 = PB.L

PINT Edge Clear Register

The `PINT_EDGE_CLR` register permits selecting level-sensitive interrupts. Writing 1 to a bit in `PINT_EDGE_CLR` enables level sensitivity for the corresponding pin interrupt.

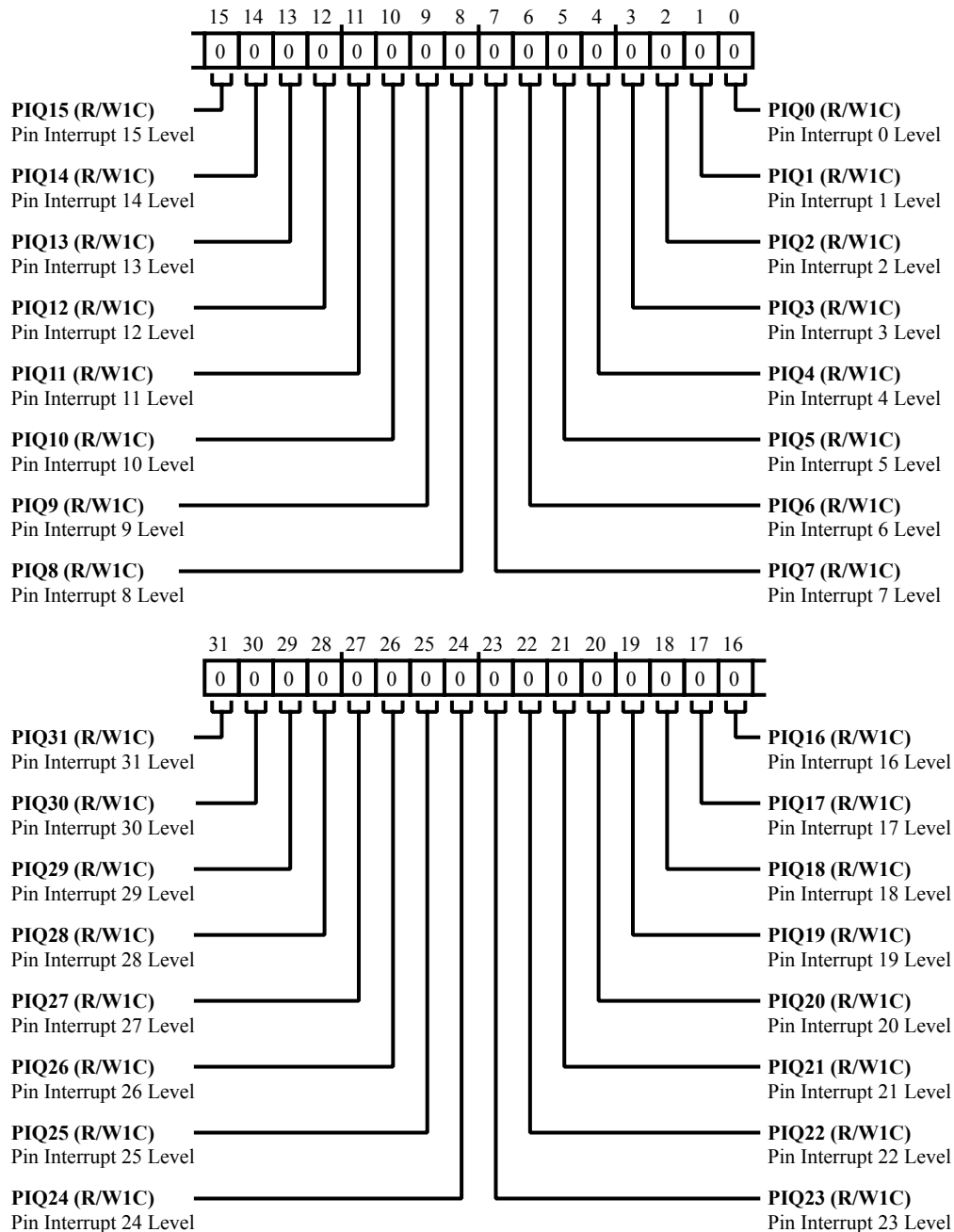


Figure 11-27: `PINT_EDGE_CLR` Register Diagram

Table 11-31: PINT_EDGE_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 Level. Set the PINT_EDGE_CLR.PIQ31 bit to enable level sensitivity.
30 (R/W1C)	PIQ30	Pin Interrupt 30 Level. Set the PINT_EDGE_CLR.PIQ30 bit to enable level sensitivity.
29 (R/W1C)	PIQ29	Pin Interrupt 29 Level. Set the PINT_EDGE_CLR.PIQ29 bit to enable level sensitivity.
28 (R/W1C)	PIQ28	Pin Interrupt 28 Level. Set the PINT_EDGE_CLR.PIQ28 bit to enable level sensitivity.
27 (R/W1C)	PIQ27	Pin Interrupt 27 Level. Set the PINT_EDGE_CLR.PIQ27 bit to enable level sensitivity.
26 (R/W1C)	PIQ26	Pin Interrupt 26 Level. Set the PINT_EDGE_CLR.PIQ26 bit to enable level sensitivity.
25 (R/W1C)	PIQ25	Pin Interrupt 25 Level. Set the PINT_EDGE_CLR.PIQ25 bit to enable level sensitivity.
24 (R/W1C)	PIQ24	Pin Interrupt 24 Level. Set the PINT_EDGE_CLR.PIQ24 bit to enable level sensitivity.
23 (R/W1C)	PIQ23	Pin Interrupt 23 Level. Set the PINT_EDGE_CLR.PIQ23 bit to enable level sensitivity.
22 (R/W1C)	PIQ22	Pin Interrupt 22 Level. Set the PINT_EDGE_CLR.PIQ22 bit to enable level sensitivity.
21 (R/W1C)	PIQ21	Pin Interrupt 21 Level. Set the PINT_EDGE_CLR.PIQ21 bit to enable level sensitivity.
20 (R/W1C)	PIQ20	Pin Interrupt 20 Level. Set the PINT_EDGE_CLR.PIQ20 bit to enable level sensitivity.
19 (R/W1C)	PIQ19	Pin Interrupt 19 Level. Set the PINT_EDGE_CLR.PIQ19 bit to enable level sensitivity.
18 (R/W1C)	PIQ18	Pin Interrupt 18 Level. Set the PINT_EDGE_CLR.PIQ18 bit to enable level sensitivity.
17 (R/W1C)	PIQ17	Pin Interrupt 17 Level. Set the PINT_EDGE_CLR.PIQ17 bit to enable level sensitivity.
16 (R/W1C)	PIQ16	Pin Interrupt 16 Level. Set the PINT_EDGE_CLR.PIQ16 bit to enable level sensitivity.

Table 11-31: PINT_EDGE_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PIQ15	Pin Interrupt 15 Level. Set the PINT_EDGE_CLR.PIQ15 bit to enable level sensitivity.
14 (R/W1C)	PIQ14	Pin Interrupt 14 Level. Set the PINT_EDGE_CLR.PIQ14 bit to enable level sensitivity.
13 (R/W1C)	PIQ13	Pin Interrupt 13 Level. Set the PINT_EDGE_CLR.PIQ13 bit to enable level sensitivity.
12 (R/W1C)	PIQ12	Pin Interrupt 12 Level. Set the PINT_EDGE_CLR.PIQ12 bit to enable level sensitivity.
11 (R/W1C)	PIQ11	Pin Interrupt 11 Level. Set the PINT_EDGE_CLR.PIQ11 bit to enable level sensitivity.
10 (R/W1C)	PIQ10	Pin Interrupt 10 Level. Set the PINT_EDGE_CLR.PIQ10 bit to enable level sensitivity.
9 (R/W1C)	PIQ9	Pin Interrupt 9 Level. Set the PINT_EDGE_CLR.PIQ9 bit to enable level sensitivity.
8 (R/W1C)	PIQ8	Pin Interrupt 8 Level. Set the PINT_EDGE_CLR.PIQ8 bit to enable level sensitivity.
7 (R/W1C)	PIQ7	Pin Interrupt 7 Level. Set the PINT_EDGE_CLR.PIQ7 bit to enable level sensitivity.
6 (R/W1C)	PIQ6	Pin Interrupt 6 Level. Set the PINT_EDGE_CLR.PIQ6 bit to enable level sensitivity.
5 (R/W1C)	PIQ5	Pin Interrupt 5 Level. Set the PINT_EDGE_CLR.PIQ5 bit to enable level sensitivity.
4 (R/W1C)	PIQ4	Pin Interrupt 4 Level. Set the PINT_EDGE_CLR.PIQ4 bit to enable level sensitivity.
3 (R/W1C)	PIQ3	Pin Interrupt 3 Level. Set the PINT_EDGE_CLR.PIQ3 bit to enable level sensitivity.
2 (R/W1C)	PIQ2	Pin Interrupt 2 Level. Set the PINT_EDGE_CLR.PIQ2 bit to enable level sensitivity.
1 (R/W1C)	PIQ1	Pin Interrupt 1 Level. Set the PINT_EDGE_CLR.PIQ1 bit to enable level sensitivity.
0 (R/W1C)	PIQ0	Pin Interrupt 0 Level. Set the PINT_EDGE_CLR.PIQ0 bit to enable level sensitivity.

PINT Edge Set Register

The `PINT_EDGE_SET` register permits selecting edge-sensitive interrupts. Writing 1 to a bit in `PINT_EDGE_SET` enables edge sensitivity for the corresponding pin interrupt.

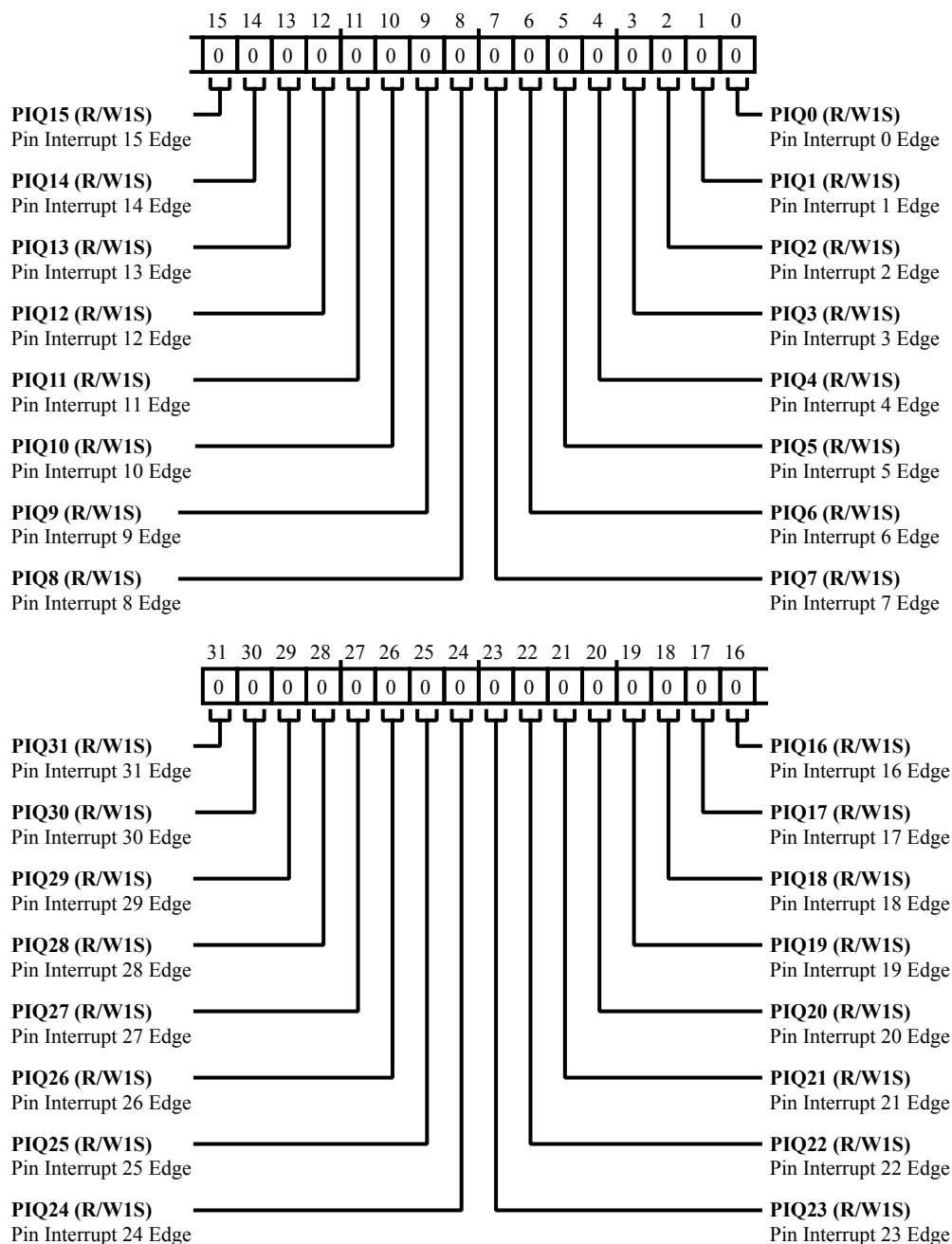


Figure 11-28: `PINT_EDGE_SET` Register Diagram

Table 11-32: PINT_EDGE_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	PIQ31	Pin Interrupt 31 Edge. Set the PINT_EDGE_SET.PIQ31 bit to enable edge sensitivity.
30 (R/W1S)	PIQ30	Pin Interrupt 30 Edge. Set the PINT_EDGE_SET.PIQ30 bit to enable edge sensitivity.
29 (R/W1S)	PIQ29	Pin Interrupt 29 Edge. Set the PINT_EDGE_SET.PIQ29 bit to enable edge sensitivity.
28 (R/W1S)	PIQ28	Pin Interrupt 28 Edge. Set the PINT_EDGE_SET.PIQ28 bit to enable edge sensitivity.
27 (R/W1S)	PIQ27	Pin Interrupt 27 Edge. Set the PINT_EDGE_SET.PIQ27 bit to enable edge sensitivity.
26 (R/W1S)	PIQ26	Pin Interrupt 26 Edge. Set the PINT_EDGE_SET.PIQ26 bit to enable edge sensitivity.
25 (R/W1S)	PIQ25	Pin Interrupt 25 Edge. Set the PINT_EDGE_SET.PIQ25 bit to enable edge sensitivity.
24 (R/W1S)	PIQ24	Pin Interrupt 24 Edge. Set the PINT_EDGE_SET.PIQ24 bit to enable edge sensitivity.
23 (R/W1S)	PIQ23	Pin Interrupt 23 Edge. Set the PINT_EDGE_SET.PIQ23 bit to enable edge sensitivity.
22 (R/W1S)	PIQ22	Pin Interrupt 22 Edge. Set the PINT_EDGE_SET.PIQ22 bit to enable edge sensitivity.
21 (R/W1S)	PIQ21	Pin Interrupt 21 Edge. Set the PINT_EDGE_SET.PIQ21 bit to enable edge sensitivity.
20 (R/W1S)	PIQ20	Pin Interrupt 20 Edge. Set the PINT_EDGE_SET.PIQ20 bit to enable edge sensitivity.
19 (R/W1S)	PIQ19	Pin Interrupt 19 Edge. Set the PINT_EDGE_SET.PIQ19 bit to enable edge sensitivity.
18 (R/W1S)	PIQ18	Pin Interrupt 18 Edge. Set the PINT_EDGE_SET.PIQ18 bit to enable edge sensitivity.
17 (R/W1S)	PIQ17	Pin Interrupt 17 Edge. Set the PINT_EDGE_SET.PIQ17 bit to enable edge sensitivity.
16 (R/W1S)	PIQ16	Pin Interrupt 16 Edge. Set the PINT_EDGE_SET.PIQ16 bit to enable edge sensitivity.

Table 11-32: PINT_EDGE_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PIQ15	Pin Interrupt 15 Edge. Set the PINT_EDGE_SET.PIQ15 bit to enable edge sensitivity.
14 (R/W1S)	PIQ14	Pin Interrupt 14 Edge. Set the PINT_EDGE_SET.PIQ14 bit to enable edge sensitivity.
13 (R/W1S)	PIQ13	Pin Interrupt 13 Edge. Set the PINT_EDGE_SET.PIQ13 bit to enable edge sensitivity.
12 (R/W1S)	PIQ12	Pin Interrupt 12 Edge. Set the PINT_EDGE_SET.PIQ12 bit to enable edge sensitivity.
11 (R/W1S)	PIQ11	Pin Interrupt 11 Edge. Set the PINT_EDGE_SET.PIQ11 bit to enable edge sensitivity.
10 (R/W1S)	PIQ10	Pin Interrupt 10 Edge. Set the PINT_EDGE_SET.PIQ10 bit to enable edge sensitivity.
9 (R/W1S)	PIQ9	Pin Interrupt 9 Edge. Set the PINT_EDGE_SET.PIQ9 bit to enable edge sensitivity.
8 (R/W1S)	PIQ8	Pin Interrupt 8 Edge. Set the PINT_EDGE_SET.PIQ8 bit to enable edge sensitivity.
7 (R/W1S)	PIQ7	Pin Interrupt 7 Edge. Set the PINT_EDGE_SET.PIQ7 bit to enable edge sensitivity.
6 (R/W1S)	PIQ6	Pin Interrupt 6 Edge. Set the PINT_EDGE_SET.PIQ6 bit to enable edge sensitivity.
5 (R/W1S)	PIQ5	Pin Interrupt 5 Edge. Set the PINT_EDGE_SET.PIQ5 bit to enable edge sensitivity.
4 (R/W1S)	PIQ4	Pin Interrupt 4 Edge. Set the PINT_EDGE_SET.PIQ4 bit to enable edge sensitivity.
3 (R/W1S)	PIQ3	Pin Interrupt 3 Edge. Set the PINT_EDGE_SET.PIQ3 bit to enable edge sensitivity.
2 (R/W1S)	PIQ2	Pin Interrupt 2 Edge. Set the PINT_EDGE_SET.PIQ2 bit to enable edge sensitivity.
1 (R/W1S)	PIQ1	Pin Interrupt 1 Edge. Set the PINT_EDGE_SET.PIQ1 bit to enable edge sensitivity.
0 (R/W1S)	PIQ0	Pin Interrupt 0 Edge. Set the PINT_EDGE_SET.PIQ0 bit to enable edge sensitivity.

PINT Invert Clear Register

The `PINT_INV_CLR` register disables inverting input polarity. Writing 1 to a bit in `PINT_INV_CLR` disables an inverter for input on the corresponding pin.

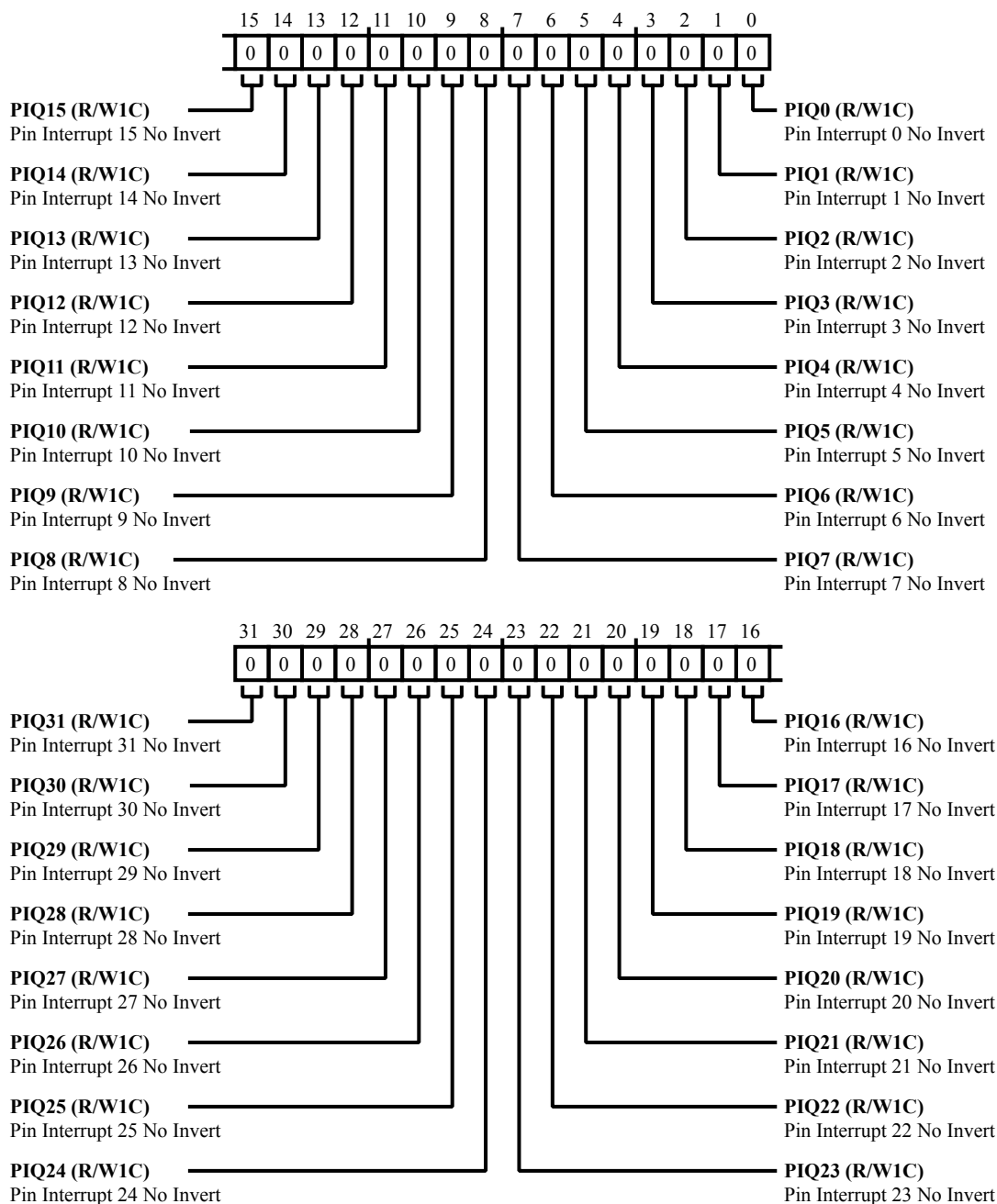


Figure 11-29: PINT_INV_CLR Register Diagram

Table 11-33: PINT_INV_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 No Invert. Set the PINT_INV_CLR.PIQ31 bit to disable inverted input.
30 (R/W1C)	PIQ30	Pin Interrupt 30 No Invert. Set the PINT_INV_CLR.PIQ30 bit to disable inverted input.
29 (R/W1C)	PIQ29	Pin Interrupt 29 No Invert. Set the PINT_INV_CLR.PIQ29 bit to disable inverted input.
28 (R/W1C)	PIQ28	Pin Interrupt 28 No Invert. Set the PINT_INV_CLR.PIQ28 bit to disable inverted input.
27 (R/W1C)	PIQ27	Pin Interrupt 27 No Invert. Set the PINT_INV_CLR.PIQ27 bit to disable inverted input.
26 (R/W1C)	PIQ26	Pin Interrupt 26 No Invert. Set the PINT_INV_CLR.PIQ26 bit to disable inverted input.
25 (R/W1C)	PIQ25	Pin Interrupt 25 No Invert. Set the PINT_INV_CLR.PIQ25 bit to disable inverted input.
24 (R/W1C)	PIQ24	Pin Interrupt 24 No Invert. Set the PINT_INV_CLR.PIQ24 bit to disable inverted input.
23 (R/W1C)	PIQ23	Pin Interrupt 23 No Invert. Set the PINT_INV_CLR.PIQ23 bit to disable inverted input.
22 (R/W1C)	PIQ22	Pin Interrupt 22 No Invert. Set the PINT_INV_CLR.PIQ22 bit to disable inverted input.
21 (R/W1C)	PIQ21	Pin Interrupt 21 No Invert. Set the PINT_INV_CLR.PIQ21 bit to disable inverted input.
20 (R/W1C)	PIQ20	Pin Interrupt 20 No Invert. Set the PINT_INV_CLR.PIQ20 bit to disable inverted input.
19 (R/W1C)	PIQ19	Pin Interrupt 19 No Invert. Set the PINT_INV_CLR.PIQ19 bit to disable inverted input.
18 (R/W1C)	PIQ18	Pin Interrupt 18 No Invert. Set the PINT_INV_CLR.PIQ18 bit to disable inverted input.
17 (R/W1C)	PIQ17	Pin Interrupt 17 No Invert. Set the PINT_INV_CLR.PIQ17 bit to disable inverted input.
16 (R/W1C)	PIQ16	Pin Interrupt 16 No Invert. Set the PINT_INV_CLR.PIQ16 bit to disable inverted input.

Table 11-33: PINT_INV_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PIQ15	Pin Interrupt 15 No Invert. Set the PINT_INV_CLR.PIQ15 bit to disable inverted input.
14 (R/W1C)	PIQ14	Pin Interrupt 14 No Invert. Set the PINT_INV_CLR.PIQ14 bit to disable inverted input.
13 (R/W1C)	PIQ13	Pin Interrupt 13 No Invert. Set the PINT_INV_CLR.PIQ13 bit to disable inverted input.
12 (R/W1C)	PIQ12	Pin Interrupt 12 No Invert. Set the PINT_INV_CLR.PIQ12 bit to disable inverted input.
11 (R/W1C)	PIQ11	Pin Interrupt 11 No Invert. Set the PINT_INV_CLR.PIQ11 bit to disable inverted input.
10 (R/W1C)	PIQ10	Pin Interrupt 10 No Invert. Set the PINT_INV_CLR.PIQ10 bit to disable inverted input.
9 (R/W1C)	PIQ9	Pin Interrupt 9 No Invert. Set the PINT_INV_CLR.PIQ9 bit to disable inverted input.
8 (R/W1C)	PIQ8	Pin Interrupt 8 No Invert. Set the PINT_INV_CLR.PIQ8 bit to disable inverted input.
7 (R/W1C)	PIQ7	Pin Interrupt 7 No Invert. Set the PINT_INV_CLR.PIQ7 bit to disable inverted input.
6 (R/W1C)	PIQ6	Pin Interrupt 6 No Invert. Set the PINT_INV_CLR.PIQ6 bit to disable inverted input.
5 (R/W1C)	PIQ5	Pin Interrupt 5 No Invert. Set the PINT_INV_CLR.PIQ5 bit to disable inverted input.
4 (R/W1C)	PIQ4	Pin Interrupt 4 No Invert. Set the PINT_INV_CLR.PIQ4 bit to disable inverted input.
3 (R/W1C)	PIQ3	Pin Interrupt 3 No Invert. Set the PINT_INV_CLR.PIQ3 bit to disable inverted input.
2 (R/W1C)	PIQ2	Pin Interrupt 2 No Invert. Set the PINT_INV_CLR.PIQ2 bit to disable inverted input.
1 (R/W1C)	PIQ1	Pin Interrupt 1 No Invert. Set the PINT_INV_CLR.PIQ1 bit to disable inverted input.
0 (R/W1C)	PIQ0	Pin Interrupt 0 No Invert. Set the PINT_INV_CLR.PIQ0 bit to disable inverted input.

PINT Invert Set Register

The `PINT_INV_SET` register enables inverting input polarity. Writing 1 to a bit in `PINT_INV_SET` enables an inverter for input on the corresponding pin.

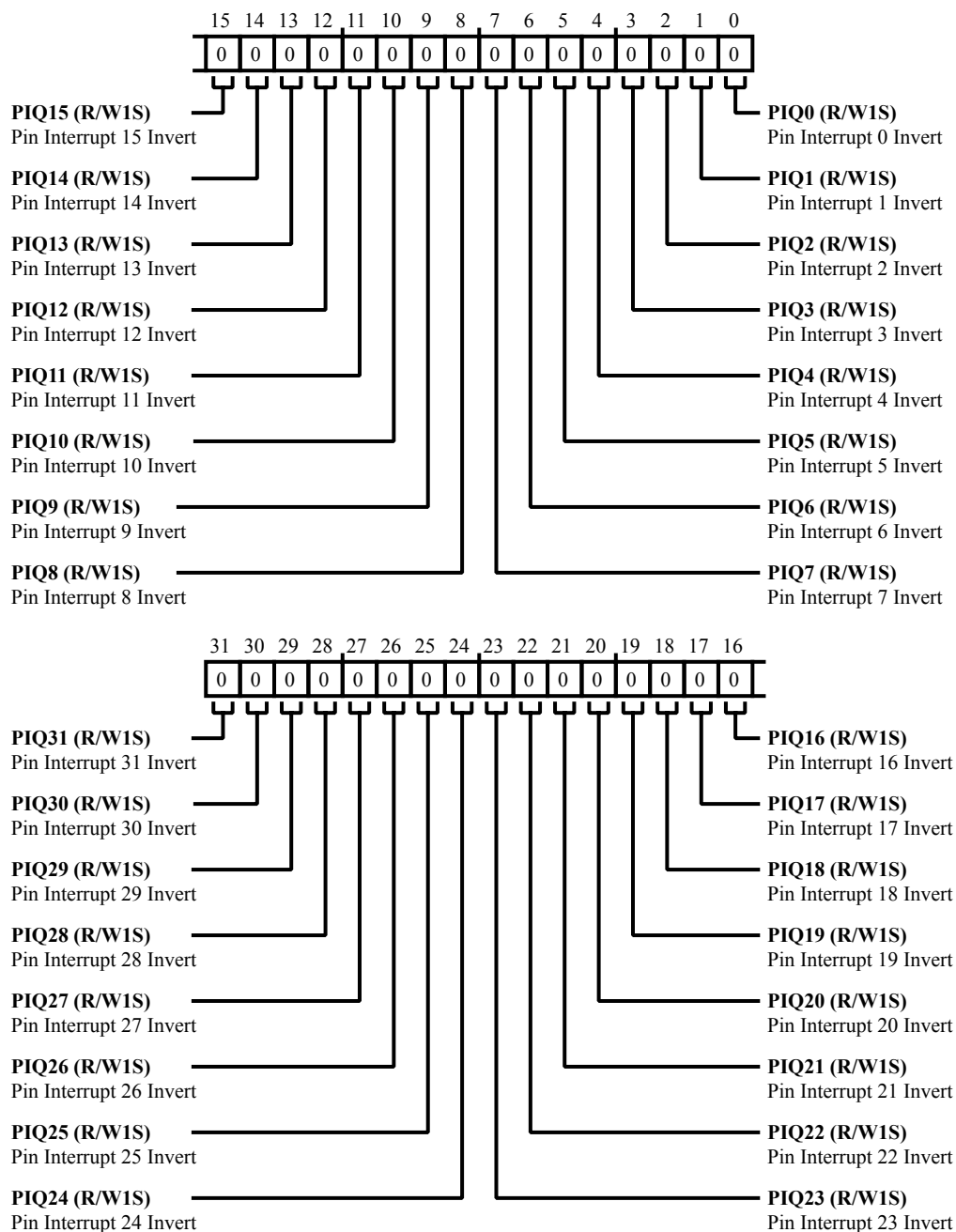


Figure 11-30: PINT_INV_SET Register Diagram

Table 11-34: PINT_INV_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	PIQ31	Pin Interrupt 31 Invert. Set the PINT_INV_SET.PIQ31 bit to enable inverted input.
30 (R/W1S)	PIQ30	Pin Interrupt 30 Invert. Set the PINT_INV_SET.PIQ30 bit to enable inverted input.
29 (R/W1S)	PIQ29	Pin Interrupt 29 Invert. Set the PINT_INV_SET.PIQ29 bit to enable inverted input.
28 (R/W1S)	PIQ28	Pin Interrupt 28 Invert. Set the PINT_INV_SET.PIQ28 bit to enable inverted input.
27 (R/W1S)	PIQ27	Pin Interrupt 27 Invert. Set the PINT_INV_SET.PIQ27 bit to enable inverted input.
26 (R/W1S)	PIQ26	Pin Interrupt 26 Invert. Set the PINT_INV_SET.PIQ26 bit to enable inverted input.
25 (R/W1S)	PIQ25	Pin Interrupt 25 Invert. Set the PINT_INV_SET.PIQ25 bit to enable inverted input.
24 (R/W1S)	PIQ24	Pin Interrupt 24 Invert. Set the PINT_INV_SET.PIQ24 bit to enable inverted input.
23 (R/W1S)	PIQ23	Pin Interrupt 23 Invert. Set the PINT_INV_SET.PIQ23 bit to enable inverted input.
22 (R/W1S)	PIQ22	Pin Interrupt 22 Invert. Set the PINT_INV_SET.PIQ22 bit to enable inverted input.
21 (R/W1S)	PIQ21	Pin Interrupt 21 Invert. Set the PINT_INV_SET.PIQ21 bit to enable inverted input.
20 (R/W1S)	PIQ20	Pin Interrupt 20 Invert. Set the PINT_INV_SET.PIQ20 bit to enable inverted input.
19 (R/W1S)	PIQ19	Pin Interrupt 19 Invert. Set the PINT_INV_SET.PIQ19 bit to enable inverted input.
18 (R/W1S)	PIQ18	Pin Interrupt 18 Invert. Set the PINT_INV_SET.PIQ18 bit to enable inverted input.
17 (R/W1S)	PIQ17	Pin Interrupt 17 Invert. Set the PINT_INV_SET.PIQ17 bit to enable inverted input.
16 (R/W1S)	PIQ16	Pin Interrupt 16 Invert. Set the PINT_INV_SET.PIQ16 bit to enable inverted input.

Table 11-34: PINT_INV_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PIQ15	Pin Interrupt 15 Invert. Set the PINT_INV_SET.PIQ15 bit to enable inverted input.
14 (R/W1S)	PIQ14	Pin Interrupt 14 Invert. Set the PINT_INV_SET.PIQ14 bit to enable inverted input.
13 (R/W1S)	PIQ13	Pin Interrupt 13 Invert. Set the PINT_INV_SET.PIQ13 bit to enable inverted input.
12 (R/W1S)	PIQ12	Pin Interrupt 12 Invert. Set the PINT_INV_SET.PIQ12 bit to enable inverted input.
11 (R/W1S)	PIQ11	Pin Interrupt 11 Invert. Set the PINT_INV_SET.PIQ11 bit to enable inverted input.
10 (R/W1S)	PIQ10	Pin Interrupt 10 Invert. Set the PINT_INV_SET.PIQ10 bit to enable inverted input.
9 (R/W1S)	PIQ9	Pin Interrupt 9 Invert. Set the PINT_INV_SET.PIQ9 bit to enable inverted input.
8 (R/W1S)	PIQ8	Pin Interrupt 8 Invert. Set the PINT_INV_SET.PIQ8 bit to enable inverted input.
7 (R/W1S)	PIQ7	Pin Interrupt 7 Invert. Set the PINT_INV_SET.PIQ7 bit to enable inverted input.
6 (R/W1S)	PIQ6	Pin Interrupt 6 Invert. Set the PINT_INV_SET.PIQ6 bit to enable inverted input.
5 (R/W1S)	PIQ5	Pin Interrupt 5 Invert. Set the PINT_INV_SET.PIQ5 bit to enable inverted input.
4 (R/W1S)	PIQ4	Pin Interrupt 4 Invert. Set the PINT_INV_SET.PIQ4 bit to enable inverted input.
3 (R/W1S)	PIQ3	Pin Interrupt 3 Invert. Set the PINT_INV_SET.PIQ3 bit to enable inverted input.
2 (R/W1S)	PIQ2	Pin Interrupt 2 Invert. Set the PINT_INV_SET.PIQ2 bit to enable inverted input.
1 (R/W1S)	PIQ1	Pin Interrupt 1 Invert. Set the PINT_INV_SET.PIQ1 bit to enable inverted input.
0 (R/W1S)	PIQ0	Pin Interrupt 0 Invert. Set the PINT_INV_SET.PIQ0 bit to enable inverted input.

PINT Latch Register

The `PINT_LATCH` register indicates the interrupt latch status for pin interrupts. When set, an interrupt request is latched. When cleared, there is no interrupt request latched.

Both the `PINT_REQ` and `PINT_LATCH` registers indicate whether an interrupt request is latched on the respective pin. The `PINT_LATCH` register is a latch that operates regardless of the interrupt masks. Bits of the `PINT_REQ` register depend on the mask register. The `PINT_REQ` register is a logical AND of the `PINT_LATCH` register and the interrupt mask.

Having two separate registers here enables the user to interrogate certain pins in polling mode while others work in interrupt mode. The `PINT_LATCH` registers can be used for edge detection or pin activity detection.

Both registers have W1C behavior. Writing a 1 to either register clears the respective bits in both registers. For interrupt operation, the user may prefer to W1C the `PINT_REQ` register (address still loaded in Px pointer). In polling mode, it might be cleaner to W1C the `PINT_LATCH` register.

Whether in edge-sensitive mode or level-sensitive mode, `PINT_LATCH` bits are never cleared by hardware except at system reset. Even in level-sensitive mode, the `PINT_LATCH` register functions as latch.

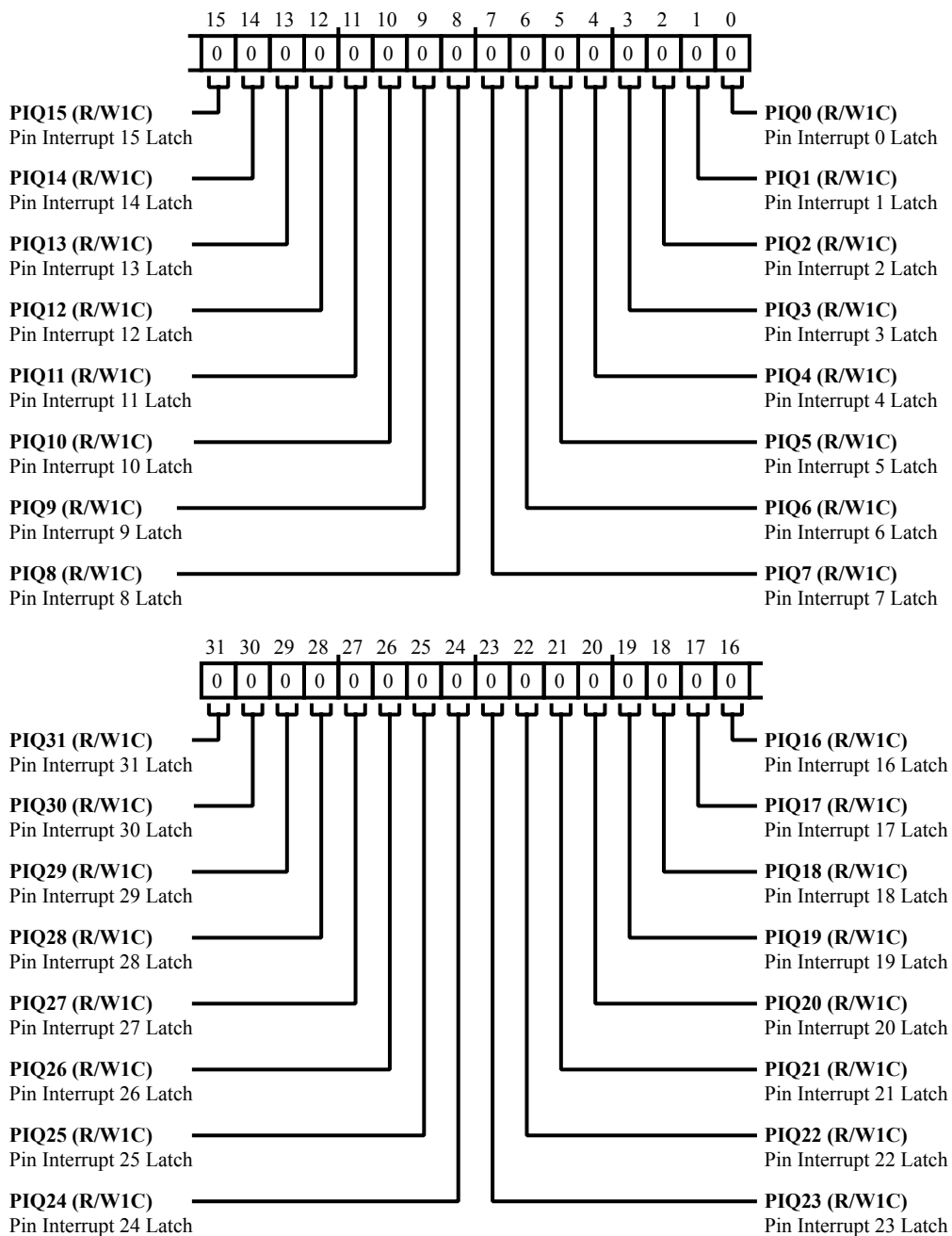


Figure 11-31: PINT_LATCH Register Diagram

Table 11-35: PINT_LATCH Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 Latch. If the PINT_LATCH.PIQ31 bit is set, the request is latched.

Table 11-35: PINT_LATCH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W1C)	PIQ30	Pin Interrupt 30 Latch. If the PINT_LATCH.PIQ30 bit is set, the request is latched.
29 (R/W1C)	PIQ29	Pin Interrupt 29 Latch. If the PINT_LATCH.PIQ29 bit is set, the request is latched.
28 (R/W1C)	PIQ28	Pin Interrupt 28 Latch. If the PINT_LATCH.PIQ28 bit is set, the request is latched.
27 (R/W1C)	PIQ27	Pin Interrupt 27 Latch. If the PINT_LATCH.PIQ27 bit is set, the request is latched.
26 (R/W1C)	PIQ26	Pin Interrupt 26 Latch. If the PINT_LATCH.PIQ26 bit is set, the request is latched.
25 (R/W1C)	PIQ25	Pin Interrupt 25 Latch. If the PINT_LATCH.PIQ25 bit is set, the request is latched.
24 (R/W1C)	PIQ24	Pin Interrupt 24 Latch. If the PINT_LATCH.PIQ24 bit is set, the request is latched.
23 (R/W1C)	PIQ23	Pin Interrupt 23 Latch. If the PINT_LATCH.PIQ23 bit is set, the request is latched.
22 (R/W1C)	PIQ22	Pin Interrupt 22 Latch. If the PINT_LATCH.PIQ22 bit is set, the request is latched.
21 (R/W1C)	PIQ21	Pin Interrupt 21 Latch. If the PINT_LATCH.PIQ21 bit is set, the request is latched.
20 (R/W1C)	PIQ20	Pin Interrupt 20 Latch. If the PINT_LATCH.PIQ20 bit is set, the request is latched.
19 (R/W1C)	PIQ19	Pin Interrupt 19 Latch. If the PINT_LATCH.PIQ19 bit is set, the request is latched.
18 (R/W1C)	PIQ18	Pin Interrupt 18 Latch. If the PINT_LATCH.PIQ18 bit is set, the request is latched.
17 (R/W1C)	PIQ17	Pin Interrupt 17 Latch. If the PINT_LATCH.PIQ17 bit is set, the request is latched.
16 (R/W1C)	PIQ16	Pin Interrupt 16 Latch. If the PINT_LATCH.PIQ16 bit is set, the request is latched.
15 (R/W1C)	PIQ15	Pin Interrupt 15 Latch. If the PINT_LATCH.PIQ15 bit is set, the request is latched.

Table 11-35: PINT_LATCH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W1C)	PIQ14	Pin Interrupt 14 Latch. If the <code>PINT_LATCH.PIQ14</code> bit is set, the request is latched.
13 (R/W1C)	PIQ13	Pin Interrupt 13 Latch. If the <code>PINT_LATCH.PIQ13</code> bit is set, the request is latched.
12 (R/W1C)	PIQ12	Pin Interrupt 12 Latch. If the <code>PINT_LATCH.PIQ12</code> bit is set, the request is latched.
11 (R/W1C)	PIQ11	Pin Interrupt 11 Latch. If the <code>PINT_LATCH.PIQ11</code> bit is set, the request is latched.
10 (R/W1C)	PIQ10	Pin Interrupt 10 Latch. If the <code>PINT_LATCH.PIQ10</code> bit is set, the request is latched.
9 (R/W1C)	PIQ9	Pin Interrupt 9 Latch. If the <code>PINT_LATCH.PIQ9</code> bit is set, the request is latched.
8 (R/W1C)	PIQ8	Pin Interrupt 8 Latch. If the <code>PINT_LATCH.PIQ8</code> bit is set, the request is latched.
7 (R/W1C)	PIQ7	Pin Interrupt 7 Latch. If the <code>PINT_LATCH.PIQ7</code> bit is set, the request is latched.
6 (R/W1C)	PIQ6	Pin Interrupt 6 Latch. If the <code>PINT_LATCH.PIQ6</code> bit is set, the request is latched.
5 (R/W1C)	PIQ5	Pin Interrupt 5 Latch. If the <code>PINT_LATCH.PIQ5</code> bit is set, the request is latched.
4 (R/W1C)	PIQ4	Pin Interrupt 4 Latch. If the <code>PINT_LATCH.PIQ4</code> bit is set, the request is latched.
3 (R/W1C)	PIQ3	Pin Interrupt 3 Latch. If the <code>PINT_LATCH.PIQ3</code> bit is set, the request is latched.
2 (R/W1C)	PIQ2	Pin Interrupt 2 Latch. If the <code>PINT_LATCH.PIQ2</code> bit is set, the request is latched.
1 (R/W1C)	PIQ1	Pin Interrupt 1 Latch. If the <code>PINT_LATCH.PIQ1</code> bit is set, the request is latched.
0 (R/W1C)	PIQ0	Pin Interrupt 0 Latch. If the <code>PINT_LATCH.PIQ0</code> bit is set, the request is latched.

PINT Mask Clear Register

The `PINT_MSK_CLR` register permits masking (disabling) of interrupt requests. Writing 1 to a bit in `PINT_MSK_CLR` masks the corresponding pin interrupt.

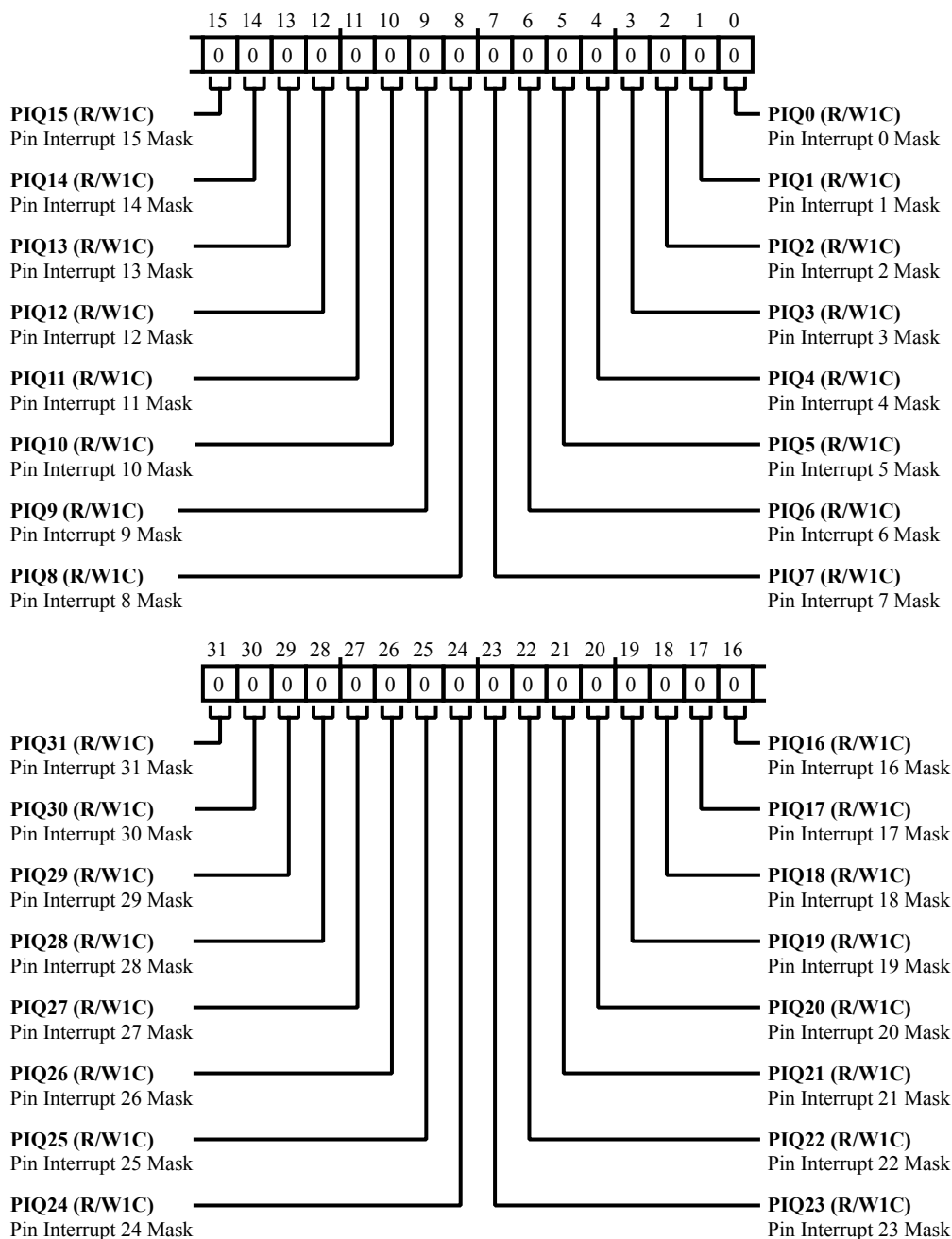


Figure 11-32: PINT_MSK_CLR Register Diagram

Table 11-36: PINT_MSK_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 Mask. Set the PINT_MSK_CLR.PIQ31 bit to disable the interrupt.
30 (R/W1C)	PIQ30	Pin Interrupt 30 Mask. Set the PINT_MSK_CLR.PIQ30 bit to disable the interrupt.
29 (R/W1C)	PIQ29	Pin Interrupt 29 Mask. Set the PINT_MSK_CLR.PIQ29 bit to disable the interrupt.
28 (R/W1C)	PIQ28	Pin Interrupt 28 Mask. Set the PINT_MSK_CLR.PIQ28 bit to disable the interrupt.
27 (R/W1C)	PIQ27	Pin Interrupt 27 Mask. Set the PINT_MSK_CLR.PIQ27 bit to disable the interrupt.
26 (R/W1C)	PIQ26	Pin Interrupt 26 Mask. Set the PINT_MSK_CLR.PIQ26 bit to disable the interrupt.
25 (R/W1C)	PIQ25	Pin Interrupt 25 Mask. Set the PINT_MSK_CLR.PIQ25 bit to disable the interrupt.
24 (R/W1C)	PIQ24	Pin Interrupt 24 Mask. Set the PINT_MSK_CLR.PIQ24 bit to disable the interrupt.
23 (R/W1C)	PIQ23	Pin Interrupt 23 Mask. Set the PINT_MSK_CLR.PIQ23 bit to disable the interrupt.
22 (R/W1C)	PIQ22	Pin Interrupt 22 Mask. Set the PINT_MSK_CLR.PIQ22 bit to disable the interrupt.
21 (R/W1C)	PIQ21	Pin Interrupt 21 Mask. Set the PINT_MSK_CLR.PIQ21 bit to disable the interrupt.
20 (R/W1C)	PIQ20	Pin Interrupt 20 Mask. Set the PINT_MSK_CLR.PIQ20 bit to disable the interrupt.
19 (R/W1C)	PIQ19	Pin Interrupt 19 Mask. Set the PINT_MSK_CLR.PIQ19 bit to disable the interrupt.
18 (R/W1C)	PIQ18	Pin Interrupt 18 Mask. Set the PINT_MSK_CLR.PIQ18 bit to disable the interrupt.
17 (R/W1C)	PIQ17	Pin Interrupt 17 Mask. Set the PINT_MSK_CLR.PIQ17 bit to disable the interrupt.
16 (R/W1C)	PIQ16	Pin Interrupt 16 Mask. Set the PINT_MSK_CLR.PIQ16 bit to disable the interrupt.

Table 11-36: PINT_MSK_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	PIQ15	Pin Interrupt 15 Mask. Set the PINT_MSK_CLR.PIQ15 bit to disable the interrupt.
14 (R/W1C)	PIQ14	Pin Interrupt 14 Mask. Set the PINT_MSK_CLR.PIQ14 bit to disable the interrupt.
13 (R/W1C)	PIQ13	Pin Interrupt 13 Mask. Set the PINT_MSK_CLR.PIQ13 bit to disable the interrupt.
12 (R/W1C)	PIQ12	Pin Interrupt 12 Mask. Set the PINT_MSK_CLR.PIQ12 bit to disable the interrupt.
11 (R/W1C)	PIQ11	Pin Interrupt 11 Mask. Set the PINT_MSK_CLR.PIQ11 bit to disable the interrupt.
10 (R/W1C)	PIQ10	Pin Interrupt 10 Mask. Set the PINT_MSK_CLR.PIQ10 bit to disable the interrupt.
9 (R/W1C)	PIQ9	Pin Interrupt 9 Mask. Set the PINT_MSK_CLR.PIQ9 bit to disable the interrupt.
8 (R/W1C)	PIQ8	Pin Interrupt 8 Mask. Set the PINT_MSK_CLR.PIQ8 bit to disable the interrupt.
7 (R/W1C)	PIQ7	Pin Interrupt 7 Mask. Set the PINT_MSK_CLR.PIQ7 bit to disable the interrupt.
6 (R/W1C)	PIQ6	Pin Interrupt 6 Mask. Set the PINT_MSK_CLR.PIQ6 bit to disable the interrupt.
5 (R/W1C)	PIQ5	Pin Interrupt 5 Mask. Set the PINT_MSK_CLR.PIQ5 bit to disable the interrupt.
4 (R/W1C)	PIQ4	Pin Interrupt 4 Mask. Set the PINT_MSK_CLR.PIQ4 bit to disable the interrupt.
3 (R/W1C)	PIQ3	Pin Interrupt 3 Mask. Set the PINT_MSK_CLR.PIQ3 bit to disable the interrupt.
2 (R/W1C)	PIQ2	Pin Interrupt 2 Mask. Set the PINT_MSK_CLR.PIQ2 bit to disable the interrupt.
1 (R/W1C)	PIQ1	Pin Interrupt 1 Mask. Set the PINT_MSK_CLR.PIQ1 bit to disable the interrupt.
0 (R/W1C)	PIQ0	Pin Interrupt 0 Mask. Set the PINT_MSK_CLR.PIQ0 bit to disable the interrupt.

PINT Mask Set Register

The `PINT_MSK_SET` register permits unmasking (enabling) of interrupt requests. Writing 1 to a bit in `PINT_MSK_SET` unmask the corresponding pin interrupt.

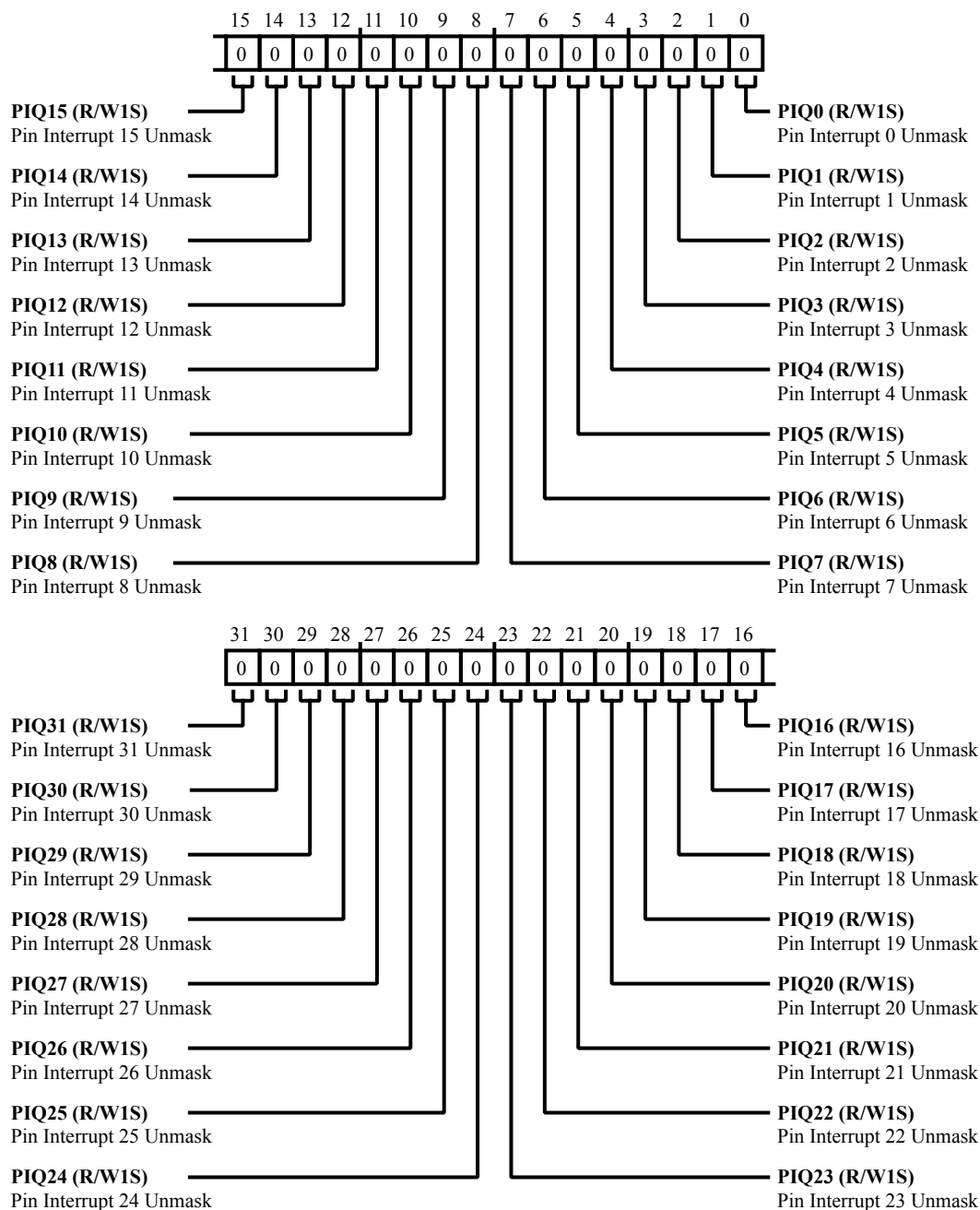


Figure 11-33: `PINT_MSK_SET` Register Diagram

Table 11-37: PINT_MSK_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	PIQ31	Pin Interrupt 31 Unmask. Set the <code>PINT_MSK_SET.PIQ31</code> bit to enable the interrupt.
30 (R/W1S)	PIQ30	Pin Interrupt 30 Unmask. Set the <code>PINT_MSK_SET.PIQ30</code> bit to enable the interrupt.
29 (R/W1S)	PIQ29	Pin Interrupt 29 Unmask. Set the <code>PINT_MSK_SET.PIQ29</code> bit to enable the interrupt.
28 (R/W1S)	PIQ28	Pin Interrupt 28 Unmask. Set the <code>PINT_MSK_SET.PIQ28</code> bit to enable the interrupt.
27 (R/W1S)	PIQ27	Pin Interrupt 27 Unmask. Set the <code>PINT_MSK_SET.PIQ27</code> bit to enable the interrupt.
26 (R/W1S)	PIQ26	Pin Interrupt 26 Unmask. Set the <code>PINT_MSK_SET.PIQ26</code> bit to enable the interrupt.
25 (R/W1S)	PIQ25	Pin Interrupt 25 Unmask. Set the <code>PINT_MSK_SET.PIQ25</code> bit to enable the interrupt.
24 (R/W1S)	PIQ24	Pin Interrupt 24 Unmask. Set the <code>PINT_MSK_SET.PIQ24</code> bit to enable the interrupt.
23 (R/W1S)	PIQ23	Pin Interrupt 23 Unmask. Set the <code>PINT_MSK_SET.PIQ23</code> bit to enable the interrupt.
22 (R/W1S)	PIQ22	Pin Interrupt 22 Unmask. Set the <code>PINT_MSK_SET.PIQ22</code> bit to enable the interrupt.
21 (R/W1S)	PIQ21	Pin Interrupt 21 Unmask. Set the <code>PINT_MSK_SET.PIQ21</code> bit to enable the interrupt.
20 (R/W1S)	PIQ20	Pin Interrupt 20 Unmask. Set the <code>PINT_MSK_SET.PIQ20</code> bit to enable the interrupt.
19 (R/W1S)	PIQ19	Pin Interrupt 19 Unmask. Set the <code>PINT_MSK_SET.PIQ19</code> bit to enable the interrupt.
18 (R/W1S)	PIQ18	Pin Interrupt 18 Unmask. Set the <code>PINT_MSK_SET.PIQ18</code> bit to enable the interrupt.
17 (R/W1S)	PIQ17	Pin Interrupt 17 Unmask. Set the <code>PINT_MSK_SET.PIQ17</code> bit to enable the interrupt.
16 (R/W1S)	PIQ16	Pin Interrupt 16 Unmask. Set the <code>PINT_MSK_SET.PIQ16</code> bit to enable the interrupt.

Table 11-37: PINT_MSK_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1S)	PIQ15	Pin Interrupt 15 Unmask. Set the PINT_MSK_SET.PIQ15 bit to enable the interrupt.
14 (R/W1S)	PIQ14	Pin Interrupt 14 Unmask. Set the PINT_MSK_SET.PIQ14 bit to enable the interrupt.
13 (R/W1S)	PIQ13	Pin Interrupt 13 Unmask. Set the PINT_MSK_SET.PIQ13 bit to enable the interrupt.
12 (R/W1S)	PIQ12	Pin Interrupt 12 Unmask. Set the PINT_MSK_SET.PIQ12 bit to enable the interrupt.
11 (R/W1S)	PIQ11	Pin Interrupt 11 Unmask. Set the PINT_MSK_SET.PIQ11 bit to enable the interrupt.
10 (R/W1S)	PIQ10	Pin Interrupt 10 Unmask. Set the PINT_MSK_SET.PIQ10 bit to enable the interrupt.
9 (R/W1S)	PIQ9	Pin Interrupt 9 Unmask. Set the PINT_MSK_SET.PIQ9 bit to enable the interrupt.
8 (R/W1S)	PIQ8	Pin Interrupt 8 Unmask. Set the PINT_MSK_SET.PIQ8 bit to enable the interrupt.
7 (R/W1S)	PIQ7	Pin Interrupt 7 Unmask. Set the PINT_MSK_SET.PIQ7 bit to enable the interrupt.
6 (R/W1S)	PIQ6	Pin Interrupt 6 Unmask. Set the PINT_MSK_SET.PIQ6 bit to enable the interrupt.
5 (R/W1S)	PIQ5	Pin Interrupt 5 Unmask. Set the PINT_MSK_SET.PIQ5 bit to enable the interrupt.
4 (R/W1S)	PIQ4	Pin Interrupt 4 Unmask. Set the PINT_MSK_SET.PIQ4 bit to enable the interrupt.
3 (R/W1S)	PIQ3	Pin Interrupt 3 Unmask. Set the PINT_MSK_SET.PIQ3 bit to enable the interrupt.
2 (R/W1S)	PIQ2	Pin Interrupt 2 Unmask. Set the PINT_MSK_SET.PIQ2 bit to enable the interrupt.
1 (R/W1S)	PIQ1	Pin Interrupt 1 Unmask. Set the PINT_MSK_SET.PIQ1 bit to enable the interrupt.
0 (R/W1S)	PIQ0	Pin Interrupt 0 Unmask. Set the PINT_MSK_SET.PIQ0 bit to enable the interrupt.

PINT Pin State Register

When a half port is assigned to a byte in any PINT block, the state of the eight pins (regardless of GPIO or function, input or output) can be seen in the `PINT_PINSTATE` register. While neither input nor output drivers of the pin are enabled, reads of the pin state in `PINT_PINSTATE` return zero. The `PINT_PINSTATE` register reports the inverted state of the pin if the signal inverter is activated by the `PINT_INV_SET` register. The inverter can be enabled on an individual bit-by-bit basis. Every bit in the `PINT_INV_SET` and `PINT_INV_CLR` register pair represents a pin signal.

The pin interrupt pin state registers enable the service routine to read the current state of the pin without reading from GPIO space. If there was an edge-sensitive interrupt, the service routine can check whether the state of the pin is still high or turned low.

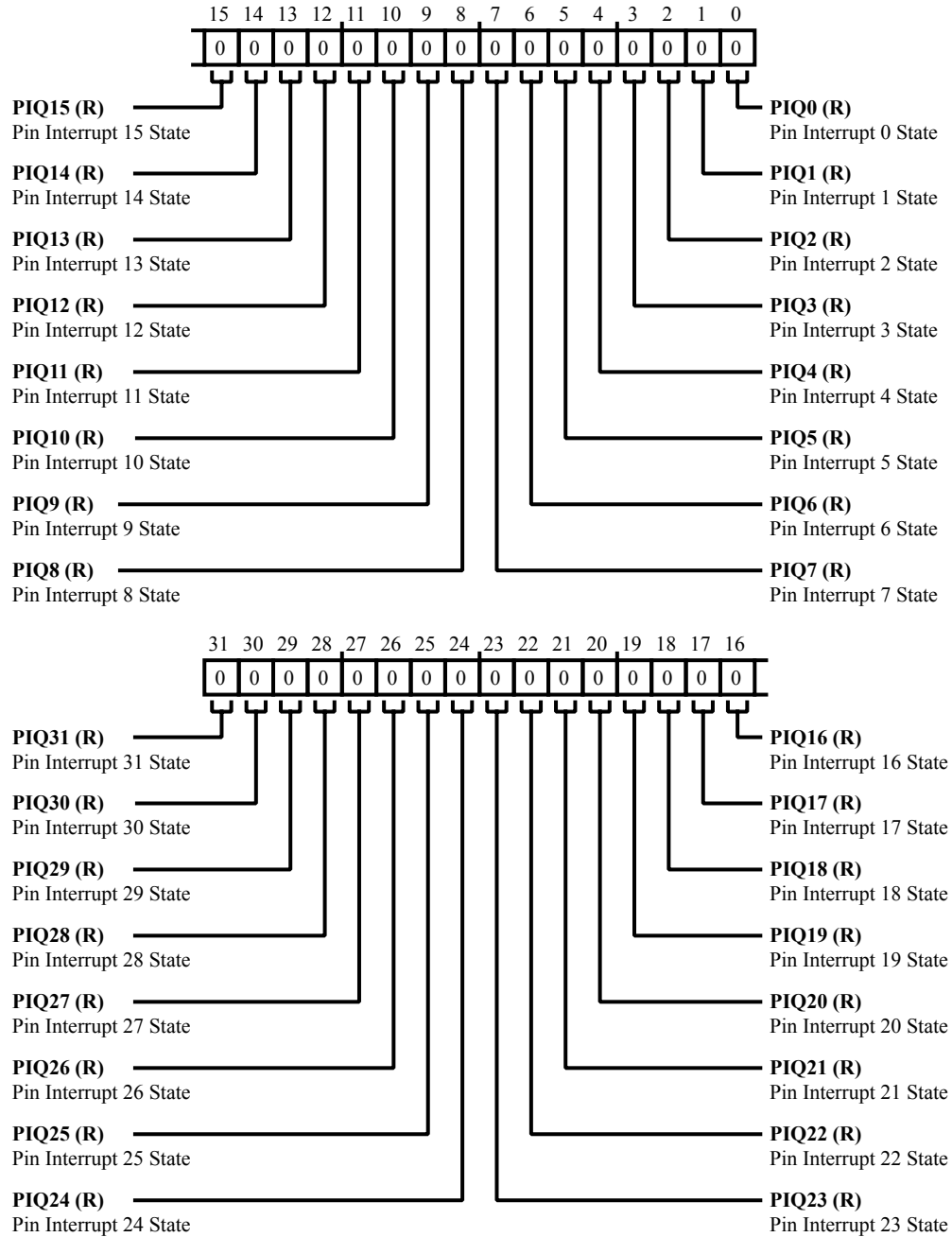


Figure 11-34: PINT_PINSTATE Register Diagram

Table 11-38: PINT_PINSTATE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	PIQ31	Pin Interrupt 31 State. A read of the <code>PINT_PINSTATE.PIQ31</code> bit returns the pin state.

Table 11-38: PINT_PINSTATE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/NW)	PIQ30	Pin Interrupt 30 State. A read of the PINT_PINSTATE.PIQ30 bit returns the pin state.
29 (R/NW)	PIQ29	Pin Interrupt 29 State. A read of the PINT_PINSTATE.PIQ29 bit returns the pin state.
28 (R/NW)	PIQ28	Pin Interrupt 28 State. A read of the PINT_PINSTATE.PIQ28 bit returns the pin state.
27 (R/NW)	PIQ27	Pin Interrupt 27 State. A read of the PINT_PINSTATE.PIQ27 bit returns the pin state.
26 (R/NW)	PIQ26	Pin Interrupt 26 State. A read of the PINT_PINSTATE.PIQ26 bit returns the pin state.
25 (R/NW)	PIQ25	Pin Interrupt 25 State. A read of the PINT_PINSTATE.PIQ25 bit returns the pin state.
24 (R/NW)	PIQ24	Pin Interrupt 24 State. A read of the PINT_PINSTATE.PIQ24 bit returns the pin state.
23 (R/NW)	PIQ23	Pin Interrupt 23 State. A read of the PINT_PINSTATE.PIQ23 bit returns the pin state.
22 (R/NW)	PIQ22	Pin Interrupt 22 State. A read of the PINT_PINSTATE.PIQ22 bit returns the pin state.
21 (R/NW)	PIQ21	Pin Interrupt 21 State. A read of the PINT_PINSTATE.PIQ21 bit returns the pin state.
20 (R/NW)	PIQ20	Pin Interrupt 20 State. A read of the PINT_PINSTATE.PIQ20 bit returns the pin state.
19 (R/NW)	PIQ19	Pin Interrupt 19 State. A read of the PINT_PINSTATE.PIQ19 bit returns the pin state.
18 (R/NW)	PIQ18	Pin Interrupt 18 State. A read of the PINT_PINSTATE.PIQ18 bit returns the pin state.
17 (R/NW)	PIQ17	Pin Interrupt 17 State. A read of the PINT_PINSTATE.PIQ17 bit returns the pin state.
16 (R/NW)	PIQ16	Pin Interrupt 16 State. A read of the PINT_PINSTATE.PIQ16 bit returns the pin state.
15 (R/NW)	PIQ15	Pin Interrupt 15 State. A read of the PINT_PINSTATE.PIQ15 bit returns the pin state.

Table 11-38: PINT_PINSTATE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/NW)	PIQ14	Pin Interrupt 14 State. A read of the <code>PINT_PINSTATE.PIQ14</code> bit returns the pin state.
13 (R/NW)	PIQ13	Pin Interrupt 13 State. A read of the <code>PINT_PINSTATE.PIQ13</code> bit returns the pin state.
12 (R/NW)	PIQ12	Pin Interrupt 12 State. A read of the <code>PINT_PINSTATE.PIQ12</code> bit returns the pin state.
11 (R/NW)	PIQ11	Pin Interrupt 11 State. A read of the <code>PINT_PINSTATE.PIQ11</code> bit returns the pin state.
10 (R/NW)	PIQ10	Pin Interrupt 10 State. A read of the <code>PINT_PINSTATE.PIQ10</code> bit returns the pin state.
9 (R/NW)	PIQ9	Pin Interrupt 9 State. A read of the <code>PINT_PINSTATE.PIQ9</code> bit returns the pin state.
8 (R/NW)	PIQ8	Pin Interrupt 8 State. A read of the <code>PINT_PINSTATE.PIQ8</code> bit returns the pin state.
7 (R/NW)	PIQ7	Pin Interrupt 7 State. A read of the <code>PINT_PINSTATE.PIQ7</code> bit returns the pin state.
6 (R/NW)	PIQ6	Pin Interrupt 6 State. A read of the <code>PINT_PINSTATE.PIQ6</code> bit returns the pin state.
5 (R/NW)	PIQ5	Pin Interrupt 5 State. A read of the <code>PINT_PINSTATE.PIQ5</code> bit returns the pin state.
4 (R/NW)	PIQ4	Pin Interrupt 4 State. A read of the <code>PINT_PINSTATE.PIQ4</code> bit returns the pin state.
3 (R/NW)	PIQ3	Pin Interrupt 3 State. A read of the <code>PINT_PINSTATE.PIQ3</code> bit returns the pin state.
2 (R/NW)	PIQ2	Pin Interrupt 2 State. A read of the <code>PINT_PINSTATE.PIQ2</code> bit returns the pin state.
1 (R/NW)	PIQ1	Pin Interrupt 1 State. A read of the <code>PINT_PINSTATE.PIQ1</code> bit returns the pin state.
0 (R/NW)	PIQ0	Pin Interrupt 0 State. A read of the <code>PINT_PINSTATE.PIQ0</code> bit returns the pin state.

PINT Request Register

The `PINT_REQ` register indicates the interrupt request status for pin interrupts. When set, an interrupt request is pending. When cleared, there is no interrupt request pending.

Both the `PINT_REQ` and `PINT_LATCH` registers indicate whether an interrupt request is latched on the respective pin. The `PINT_LATCH` register is a latch that operates regardless of the interrupt masks. Bits of the `PINT_REQ` register depend on the mask register. The `PINT_REQ` register is a logical AND of the `PINT_LATCH` register and the interrupt mask.

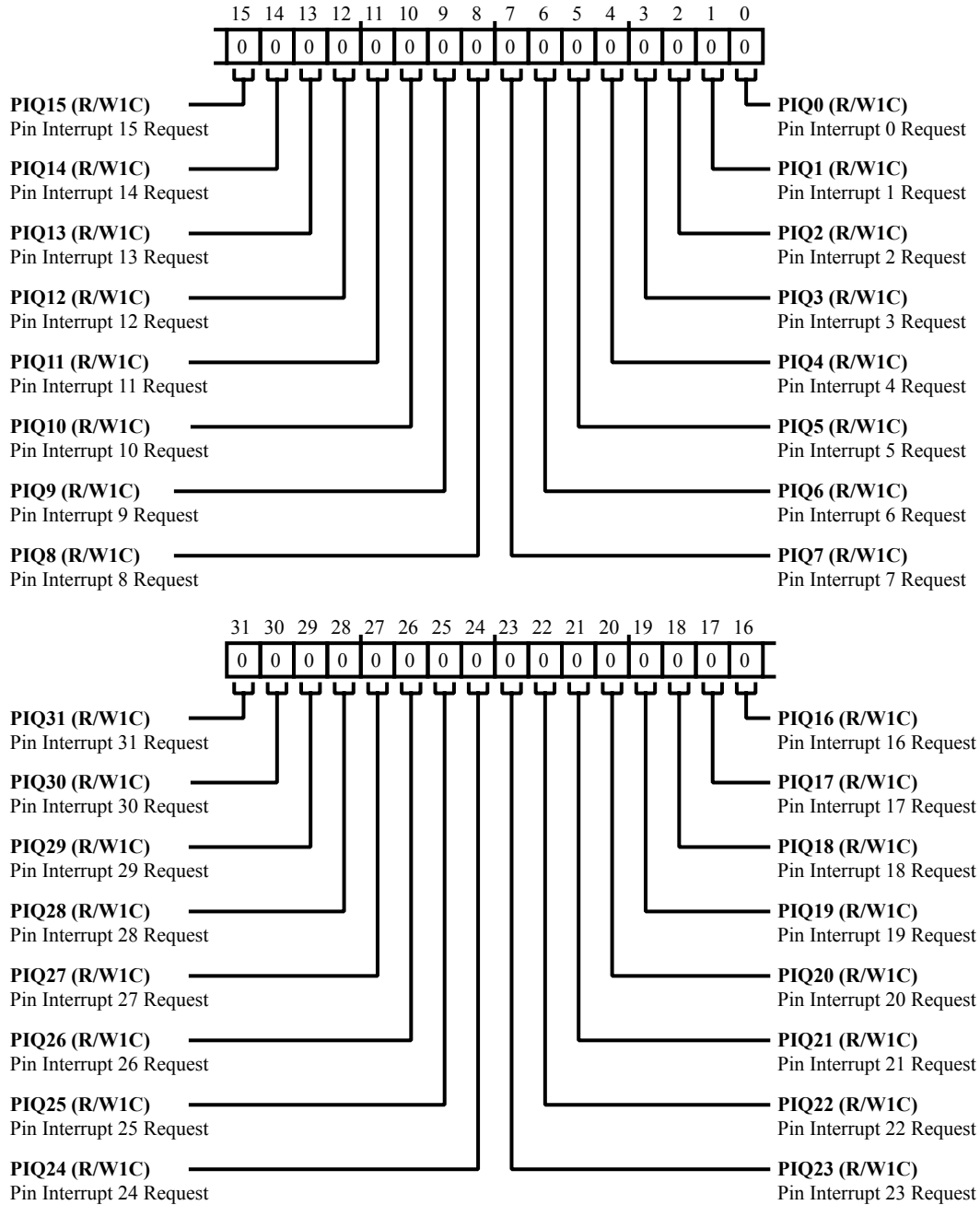


Figure 11-35: PINT_REQ Register Diagram

Table 11-39: PINT_REQ Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	PIQ31	Pin Interrupt 31 Request. If the PINT_REQ.PIQ31 bit is set, a request is pending.

Table 11-39: PINT_REQ Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W1C)	PIQ30	Pin Interrupt 30 Request. If the PINT_REQ.PIQ30 bit is set, a request is pending.
29 (R/W1C)	PIQ29	Pin Interrupt 29 Request. If the PINT_REQ.PIQ29 bit is set, a request is pending.
28 (R/W1C)	PIQ28	Pin Interrupt 28 Request. If the PINT_REQ.PIQ28 bit is set, a request is pending.
27 (R/W1C)	PIQ27	Pin Interrupt 27 Request. If the PINT_REQ.PIQ27 bit is set, a request is pending.
26 (R/W1C)	PIQ26	Pin Interrupt 26 Request. If the PINT_REQ.PIQ26 bit is set, a request is pending.
25 (R/W1C)	PIQ25	Pin Interrupt 25 Request. If the PINT_REQ.PIQ25 bit is set, a request is pending.
24 (R/W1C)	PIQ24	Pin Interrupt 24 Request. If the PINT_REQ.PIQ24 bit is set, a request is pending.
23 (R/W1C)	PIQ23	Pin Interrupt 23 Request. If the PINT_REQ.PIQ23 bit is set, a request is pending.
22 (R/W1C)	PIQ22	Pin Interrupt 22 Request. If the PINT_REQ.PIQ22 bit is set, a request is pending.
21 (R/W1C)	PIQ21	Pin Interrupt 21 Request. If the PINT_REQ.PIQ21 bit is set, a request is pending.
20 (R/W1C)	PIQ20	Pin Interrupt 20 Request. If the PINT_REQ.PIQ20 bit is set, a request is pending.
19 (R/W1C)	PIQ19	Pin Interrupt 19 Request. If the PINT_REQ.PIQ19 bit is set, a request is pending.
18 (R/W1C)	PIQ18	Pin Interrupt 18 Request. If the PINT_REQ.PIQ18 bit is set, a request is pending.
17 (R/W1C)	PIQ17	Pin Interrupt 17 Request. If the PINT_REQ.PIQ17 bit is set, a request is pending.
16 (R/W1C)	PIQ16	Pin Interrupt 16 Request. If the PINT_REQ.PIQ16 bit is set, a request is pending.
15 (R/W1C)	PIQ15	Pin Interrupt 15 Request. If the PINT_REQ.PIQ15 bit is set, a request is pending.

Table 11-39: PINT_REQ Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W1C)	PIQ14	Pin Interrupt 14 Request. If the PINT_REQ.PIQ14 bit is set, a request is pending.
13 (R/W1C)	PIQ13	Pin Interrupt 13 Request. If the PINT_REQ.PIQ13 bit is set, a request is pending.
12 (R/W1C)	PIQ12	Pin Interrupt 12 Request. If the PINT_REQ.PIQ12 bit is set, a request is pending.
11 (R/W1C)	PIQ11	Pin Interrupt 11 Request. If the PINT_REQ.PIQ11 bit is set, a request is pending.
10 (R/W1C)	PIQ10	Pin Interrupt 10 Request. If the PINT_REQ.PIQ10 bit is set, a request is pending.
9 (R/W1C)	PIQ9	Pin Interrupt 9 Request. If the PINT_REQ.PIQ9 bit is set, a request is pending.
8 (R/W1C)	PIQ8	Pin Interrupt 8 Request. If the PINT_REQ.PIQ8 bit is set, a request is pending.
7 (R/W1C)	PIQ7	Pin Interrupt 7 Request. If the PINT_REQ.PIQ7 bit is set, a request is pending.
6 (R/W1C)	PIQ6	Pin Interrupt 6 Request. If the PINT_REQ.PIQ6 bit is set, a request is pending.
5 (R/W1C)	PIQ5	Pin Interrupt 5 Request. If the PINT_REQ.PIQ5 bit is set, a request is pending.
4 (R/W1C)	PIQ4	Pin Interrupt 4 Request. If the PINT_REQ.PIQ4 bit is set, a request is pending.
3 (R/W1C)	PIQ3	Pin Interrupt 3 Request. If the PINT_REQ.PIQ3 bit is set, a request is pending.
2 (R/W1C)	PIQ2	Pin Interrupt 2 Request. If the PINT_REQ.PIQ2 bit is set, a request is pending.
1 (R/W1C)	PIQ1	Pin Interrupt 1 Request. If the PINT_REQ.PIQ1 bit is set, a request is pending.
0 (R/W1C)	PIQ0	Pin Interrupt 0 Request. If the PINT_REQ.PIQ0 bit is set, a request is pending.

ADSP-2159x PADS Register Descriptions

Pads Controller (PADS) contains the following registers.

Table 11-40: ADSP-2159x PADS Register List

Name	Description
PADS_DAI0_0_DS	DAI0 1 to 10 pins DS control
PADS_DAI0_1_DS	DAI0 11 to 20 pins DS control
PADS_DAI0_IE	DAI0 Port Input Enable Control Register
PADS_DAI1_0_DS	DAI1 1 to 10 pins DS control
PADS_DAI1_1_DS	DAI1 11 to 20 pins DS control
PADS_DAI1_IE	DAI1 Port Input Enable Control Register
PADS_DAI[n]_PDE	DAIx Pull-Down Enable
PADS_DAI[n]_PUE	DAIx Pull-Up Enable
PADS_NONPORTS_DS	Non-GPIO Drive Strength Register
PADS_PCFG0	Peripheral PAD Configuration0 Register
PADS_PCFG1	Peripheral Configuration1 Register
PADS_PORTA0_DS	PORTA 0 to 7 pins DS control
PADS_PORTA1_DS	PORTA 8 - 15 pins DS control
PADS_PORTB0_DS	PORTB 0 to 7 pins DS control
PADS_PORTB1_DS	PORTB 8 - 15 pins DS control
PADS_PORTC0_DS	PORTC 0 to 7 pins DS control
PADS_PORTC1_DS	PORTC 8 - 15 pins DS control
PADS_PORTD0_DS	PORTD 0 to 7 pins DS control
PADS_PORTD1_DS	PORTD 8 to 15 pins DS control
PADS_PORTE0_DS	PORTE 0 to 7 pins DS control
PADS_PORTE1_DS	PORTE 8 to 15 pins DS control
PADS_PORTF0_DS	PORTF 0 to 7 pins DS control
PADS_PORTF1_DS	PORTF 8 to 15 pins DS control
PADS_PORTG0_DS	PORTG 0 to 7 pins DS control
PADS_PORTG1_DS	PORTG 8 to 15 pins DS control
PADS_PORTH0_DS	PORTH 0 to 7 pins DS control
PADS_PORTH1_DS	PORTH 8 to 15 pins DS control
PADS_PORTI0_DS	PORTI 0 to 7 pins DS control
PADS_PORT[n]_PDE	PORTx Pull-Down Enable

Table 11-40: ADSP-2159x PADS Register List (Continued)

Name	Description
<code>PADS_PORT[n]_PUE</code>	PORTx Pull-Up Enable

DAI0 1 to 10 pins DS control

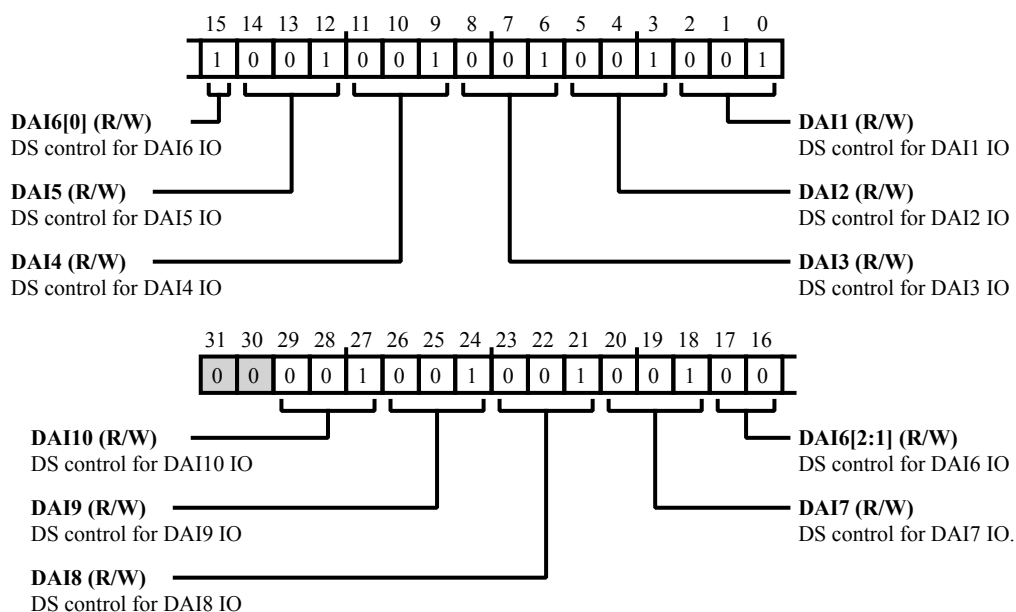


Figure 11-36: PADS_DAI0_0_DS Register Diagram

Table 11-41: PADS_DAI0_0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	DAI10	DS control for DAI10 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
26:24 (R/W)	DAI9	DS control for DAI9 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-41: PADS_DAI0_0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
23:21 (R/W)	DAI8	DS control for DAI8 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	DAI7	DS control for DAI7 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
17:15 (R/W)	DAI6	DS control for DAI6 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	DAI5	DS control for DAI5 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	DAI4	DS control for DAI4 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-41: PADS_DAI0_0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:6 (R/W)	DAI3	DS control for DAI3 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	DAI2	DS control for DAI2 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	DAI1	DS control for DAI1 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

DAI0 11 to 20 pins DS control

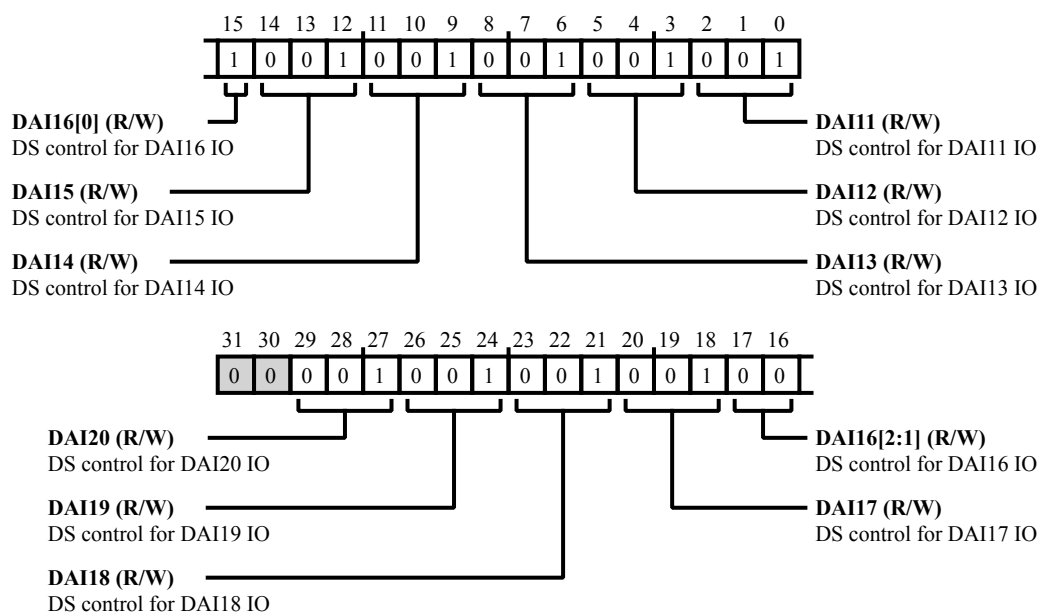


Figure 11-37: PADS_DAI0_1_DS Register Diagram

Table 11-42: PADS_DAI0_1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	DAI20	DS control for DAI20 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
26:24 (R/W)	DAI19	DS control for DAI19 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-42: PADS_DAI0_1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
23:21 (R/W)	DAI18	DS control for DAI18 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	DAI17	DS control for DAI17 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
17:15 (R/W)	DAI16	DS control for DAI16 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	DAI15	DS control for DAI15 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	DAI14	DS control for DAI14 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-42: PADS_DAI0_1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:6 (R/W)	DAI13	DS control for DAI13 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	DAI12	DS control for DAI12 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	DAI11	DS control for DAI11 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

DAI0 Port Input Enable Control Register

The `PADS_DAI0_IE` register configures input enable control of the DAI0 (20 pins) pads. If =0 implies input buffer disable and if =1 implies enable.

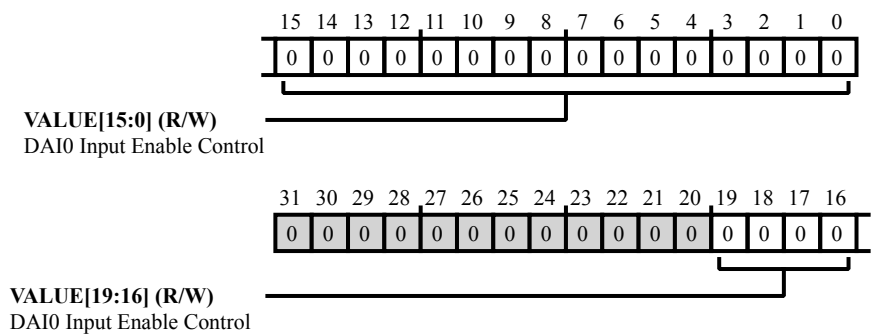


Figure 11-38: PADS_DAI0_IE Register Diagram

Table 11-43: PADS_DAI0_IE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	VALUE	DAI0 Input Enable Control.

DAI1 1 to 10 pins DS control

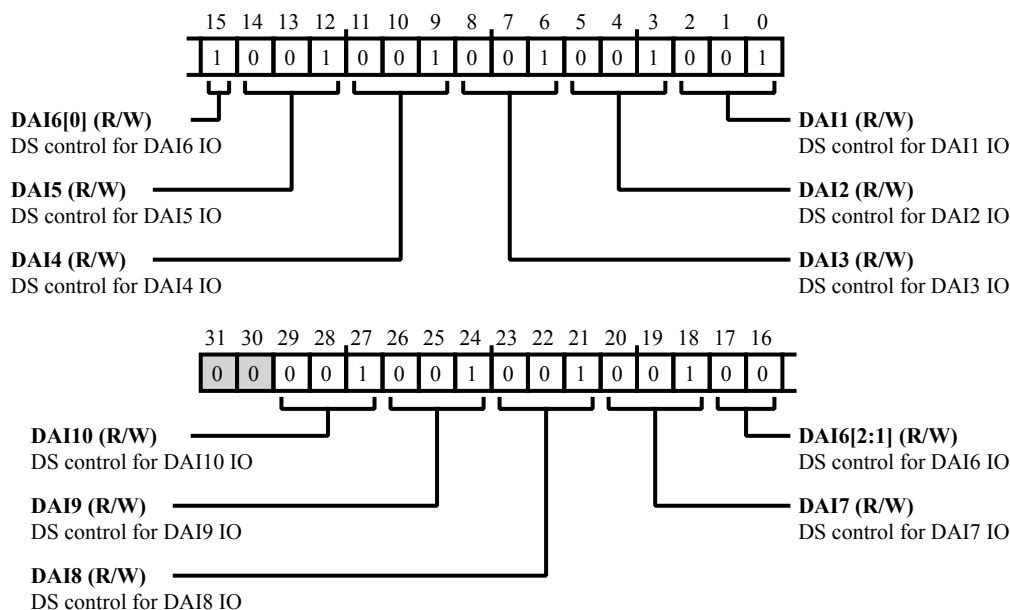


Figure 11-39: PADS_DAI1_0_DS Register Diagram

Table 11-44: PADS_DAI1_0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	DAI10	DS control for DAI10 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
26:24 (R/W)	DAI9	DS control for DAI9 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-44: PADS_DAI1_0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
23:21 (R/W)	DAI8	DS control for DAI8 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	DAI7	DS control for DAI7 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
17:15 (R/W)	DAI6	DS control for DAI6 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	DAI5	DS control for DAI5 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	DAI4	DS control for DAI4 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-44: PADS_DAI1_0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:6 (R/W)	DAI3	DS control for DAI3 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	DAI2	DS control for DAI2 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	DAI1	DS control for DAI1 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

DAI1 11 to 20 pins DS control

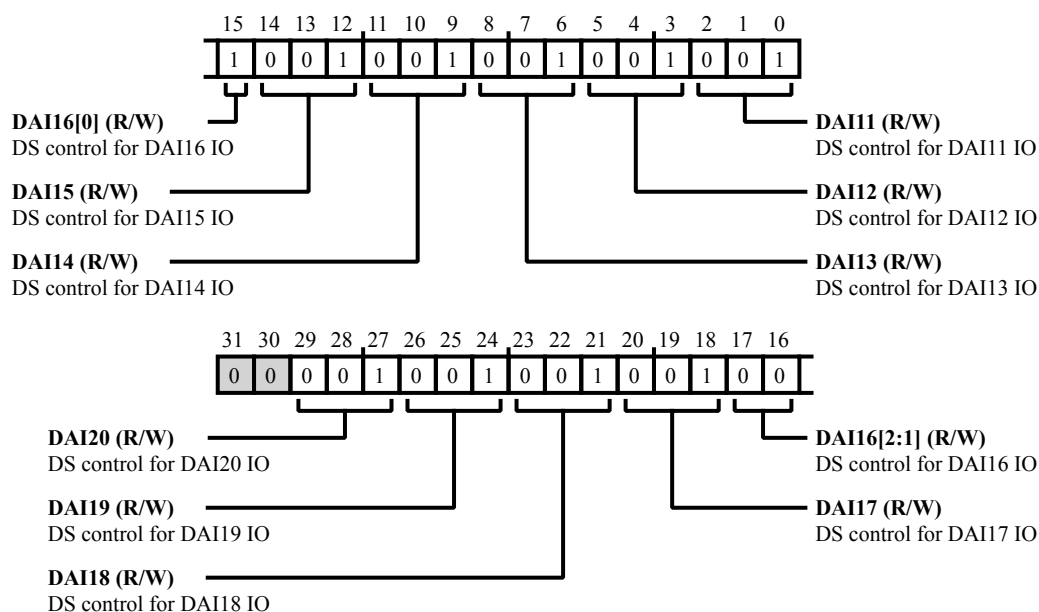


Figure 11-40: PADS_DAI1_1_DS Register Diagram

Table 11-45: PADS_DAI1_1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:27 (R/W)	DAI20	DS control for DAI20 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
26:24 (R/W)	DAI19	DS control for DAI19 IO.
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-45: PADS_DAI1_1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
23:21 (R/W)	DAI18	DS control for DAI18 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	DAI17	DS control for DAI17 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
17:15 (R/W)	DAI16	DS control for DAI16 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	DAI15	DS control for DAI15 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	DAI14	DS control for DAI14 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-45: PADS_DAI1_1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:6 (R/W)	DAI13	DS control for DAI13 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	DAI12	DS control for DAI12 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	DAI11	DS control for DAI11 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

DAI1 Port Input Enable Control Register

The `PADS_DAI1_IE` register configures input enable control of the DAI1 (20 pins) pads. If =0 implies input buffer disable and if =1 implies enable.

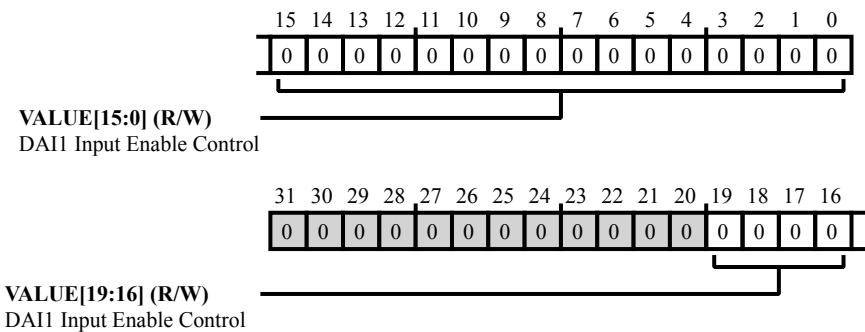


Figure 11-41: PADS_DAI1_IE Register Diagram

Table 11-46: PADS_DAI1_IE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	VALUE	DAI1 Input Enable Control.

DAIx Pull-Down Enable

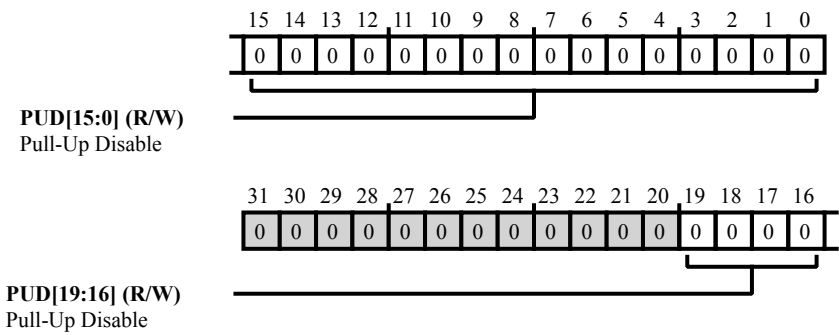


Figure 11-42: PADS_DAI[n]_PDE Register Diagram

Table 11-47: PADS_DAI[n]_PDE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	PUD	Pull-Up Disable.

DAIx Pull-Up Enable

The `PADS_DAI[n]_PUE` register enables the pull-up resistor for DAI pins.

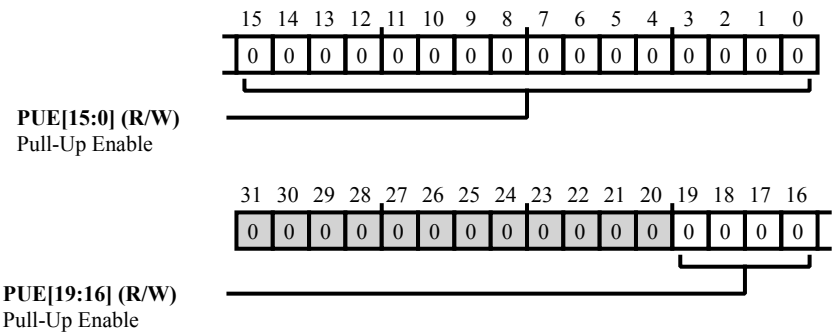


Figure 11-43: PADS_DAI[n]_PUE Register Diagram

Table 11-48: PADS_DAI[n]_PUE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	PUE	Pull-Up Enable.

Non-GPIO Drive Strength Register

The `PADS_NONPORTS_DS` register is for ADI internal usage only. The `PADS_NONPORTS_DS` register sets the drive strength and tolerance for the non-GPIO pins. The drive strength can be controlled only for pin groups, classified by the interface type.

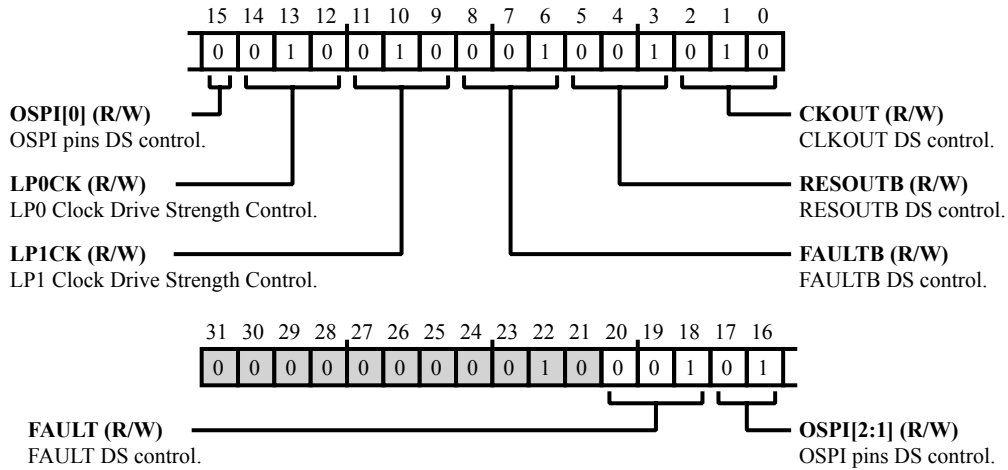


Figure 11-44: PADS_NONPORTS_DS Register Diagram

Table 11-49: PADS_NONPORTS_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:18 (R/W)	FAULT	FAULT DS control..
17:15 (R/W)	OSPI	OSPI pins DS control..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	LP0CK	LP0 Clock Drive Strength Control..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-49: PADS_NONPORTS_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
11:9 (R/W)	LP1CK	LP1 Clock Drive Strength Control..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
8:6 (R/W)	FAULTB	FAULTB DS control..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	RESOUTB	RESOUTB DS control..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
2:0 (R/W)	CKOUT	CLKOUT DS control..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Peripheral PAD Configuration0 Register

The `PADS_PCFG0` register provides several configuration options for the pads and multiplexing for peripherals.

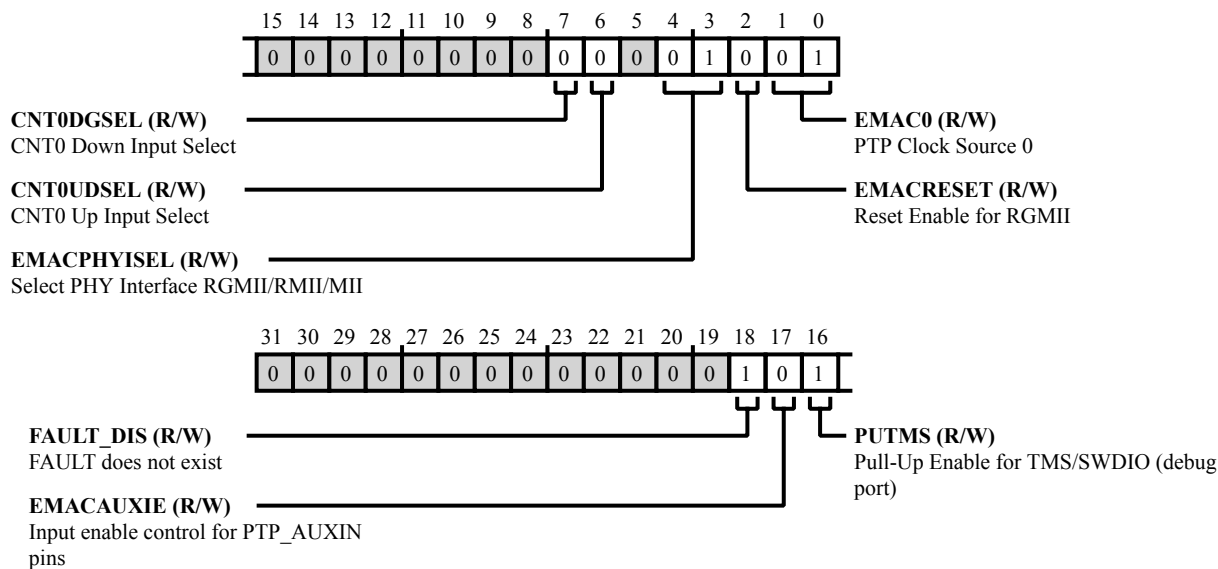


Figure 11-45: PADS_PCFG0 Register Diagram

Table 11-50: PADS_PCFG0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	FAULT_DIS	FAULT does not exist.
		0 Disable input
		1 Enable input
17 (R/W)	EMACCAUXIE	Input enable control for PTP_AUXIN pins.
		0 Disable input
		1 Enable input
16 (R/W)	PUTMS	Pull-Up Enable for TMS/SWDIO (debug port).
		0 Disable pull-up
		1 Enable pull-up
7 (R/W)	CNT0DGSEL	CNT0 Down Input Select. The <code>PADS_PCFG0</code> bit selects the input for the CNT0 counter down and gate.
		0 GPIO port pins
		1 Trigger connection

Table 11-50: PADS_PCFG0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	CNT0UDSEL	CNT0 Up Input Select. The <code>PADS_PCFG0</code> bit selects the input for the CNT0 counter up and direction.
		0 GPIO port pins
		1 Trigger connection
4:3 (R/W)	EMACPHYSEL	Select PHY Interface RGMII/RMII/MII.
		0 MII Interface
		1 RGMII Interface
		2 RMII Interface
		3 Reserved
2 (R/W)	EMACRESET	Reset Enable for RGMII. The <code>PADS_PCFG0.EMACRESET</code> bit asserts the reset on the RGMII interface. To select the PHY interface (RGMII or RMII), set the EMACPHYSEL bit as required and then set <code>PADS_PCFG0.EMACRESET</code> .
		0 RGMII reset is asserted
		1 RGMII reset reset is deasserted
1:0 (R/W)	EMAC0	PTP Clock Source 0. The <code>PADS_PCFG0.EMAC0</code> selects the clock source for the PTP Block in EMAC0.
		0 EMAC0_RMII CLK
		1 SCLK
		2 External Clock
		3 SCLK

Peripheral Configuration1 Register

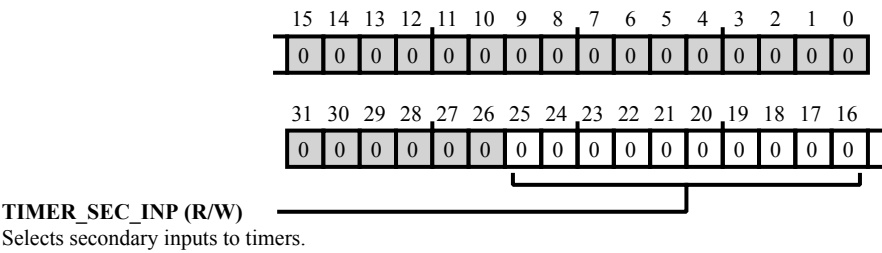


Figure 11-46: PADS_PCFG1 Register Diagram

Table 11-51: PADS_PCFG1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:16 (R/W)	TIMER_SEC_INP	Selects secondary inputs to timers.. Selects secondary inputs to timers. '0' on each bit selects {PB_02,PA_06, PA_02,PA_08,PA_06,PA_11,PB_00,PB_11,PA_14,PA_07,PB_01}, '1' on each bit selects {PE_03,PG_09,PG10,PI_05,PF15,PG04,PD_10,PD_04,PD_08,PF_05} respectively on {TIMER0_ACLK4, TIMER0_ACLK3,TIMER0_ACLK2,TIMER0_ACLK1,TIMER0_ACI4,TIMER0_ACI3,TIMER0_ACI2,TIMER0_ACI1,TIMER0_ACI0,TIMER0_CLK}

PORTA 0 to 7 pins DS control

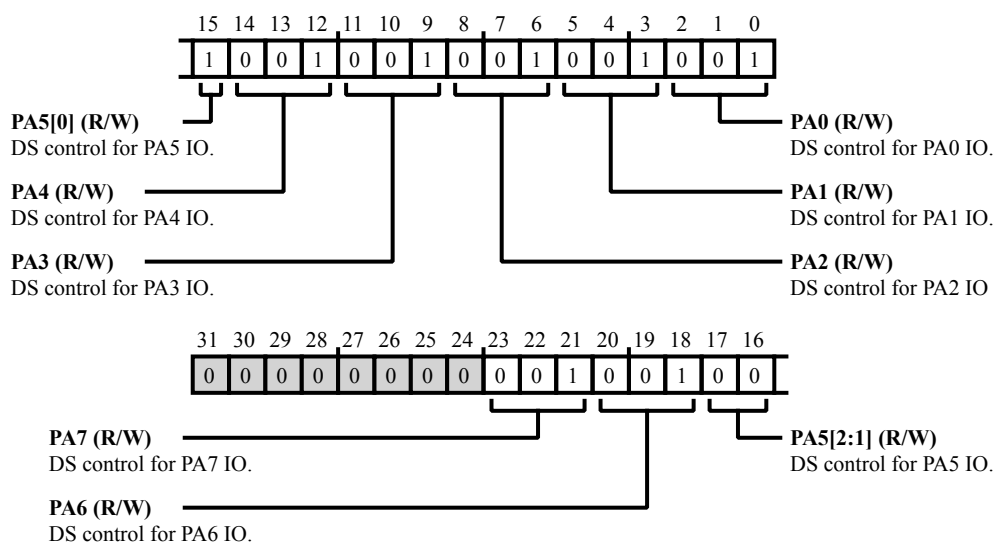


Figure 11-47: PADS_PORTA0_DS Register Diagram

Table 11-52: PADS_PORTA0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PA7	DS control for PA7 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	PA6	DS control for PA6 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-52: PADS_PORTA0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
17:15 (R/W)	PA5	DS control for PA5 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	PA4	DS control for PA4 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	PA3	DS control for PA3 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
8:6 (R/W)	PA2	DS control for PA2 IO.	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	PA1	DS control for PA1 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-52: PADS_PORTA0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2:0 (R/W)	PA0	DS control for PA0 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

PORTA 8 - 15 pins DS control

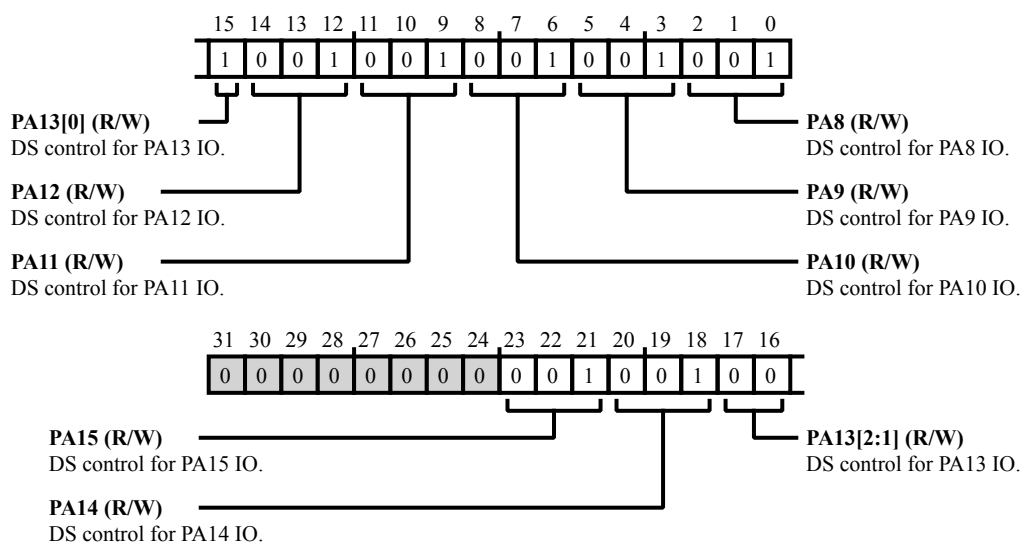


Figure 11-48: PADS_PORTA1_DS Register Diagram

Table 11-53: PADS_PORTA1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PA15	DS control for PA15 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	PA14	DS control for PA14 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-53: PADS_PORTA1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
17:15 (R/W)	PA13	DS control for PA13 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	PA12	DS control for PA12 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	PA11	DS control for PA11 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
8:6 (R/W)	PA10	DS control for PA10 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	PA9	DS control for PA9 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-53: PADS_PORTA1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2:0 (R/W)	PA8	DS control for PA8 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

PORTB 0 to 7 pins DS control

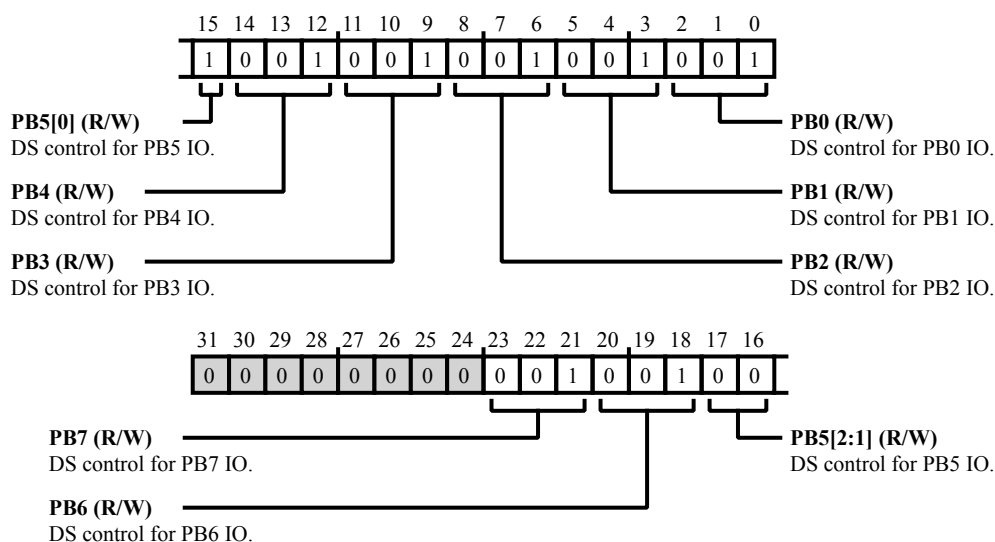


Figure 11-49: PADS_PORTB0_DS Register Diagram

Table 11-54: PADS_PORTB0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PB7	DS control for PB7 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	PB6	DS control for PB6 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-54: PADS_PORTB0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
17:15 (R/W)	PB5	DS control for PB5 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	PB4	DS control for PB4 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	PB3	DS control for PB3 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
8:6 (R/W)	PB2	DS control for PB2 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	PB1	DS control for PB1 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-54: PADS_PORTB0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2:0 (R/W)	PB0	DS control for PB0 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

PORTB 8 - 15 pins DS control

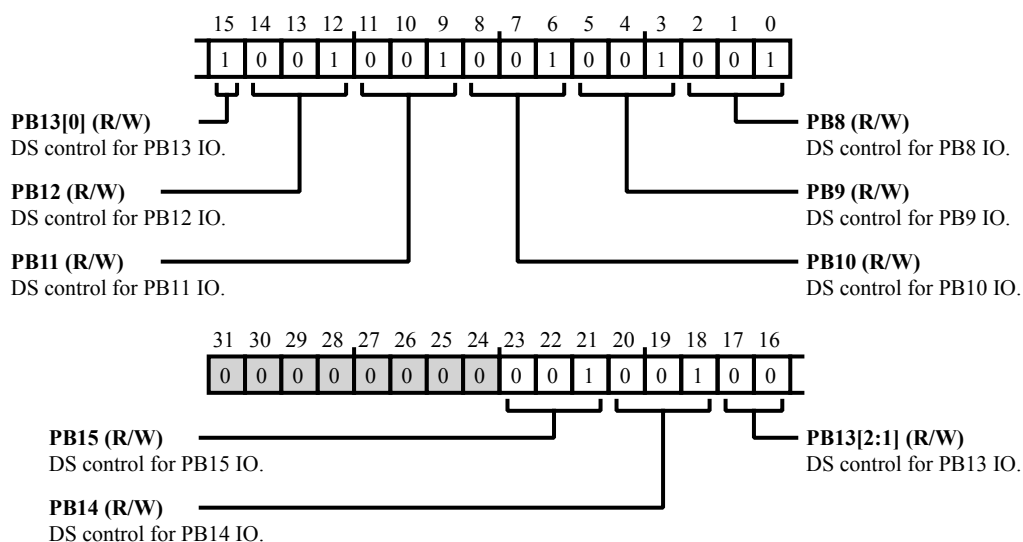


Figure 11-50: PADS_PORTB1_DS Register Diagram

Table 11-55: PADS_PORTB1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PB15	DS control for PB15 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	PB14	DS control for PB14 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-55: PADS_PORTB1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
17:15 (R/W)	PB13	DS control for PB13 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	PB12	DS control for PB12 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	PB11	DS control for PB11 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
8:6 (R/W)	PB10	DS control for PB10 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	PB9	DS control for PB9 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-55: PADS_PORTB1_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2:0 (R/W)	PB8	DS control for PB8 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

PORTC 0 to 7 pins DS control

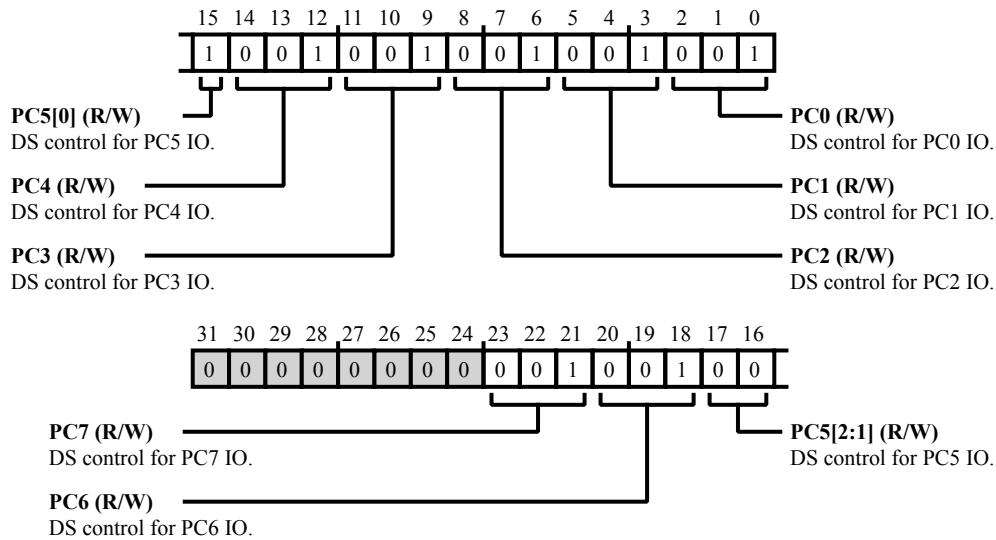


Figure 11-51: PADS_PORTC0_DS Register Diagram

Table 11-56: PADS_PORTC0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PC7	DS control for PC7 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
20:18 (R/W)	PC6	DS control for PC6 IO..
		1 Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2 Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-56: PADS_PORTC0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
17:15 (R/W)	PC5	DS control for PC5 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
14:12 (R/W)	PC4	DS control for PC4 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
11:9 (R/W)	PC3	DS control for PC3 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
8:6 (R/W)	PC2	DS control for PC2 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.
5:3 (R/W)	PC1	DS control for PC1 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010.To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

Table 11-56: PADS_PORTC0_DS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2:0 (R/W)	PC0	DS control for PC0 IO..	
		1	Drive strength 001. To be used for operating frequency less than or equal to 62.5MHz. Drive Strength 000/1xx are invalid
		2	Drive strength 010. To be used for operating frequency more than 62.5MHz. Drive Strength 000/1xx are invalid.

PORTC 8 - 15 pins DS control

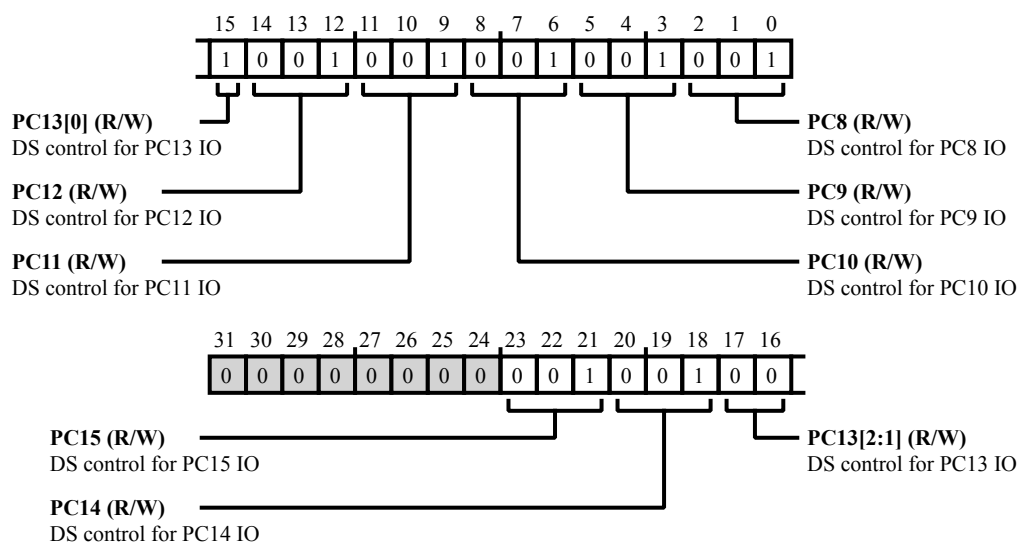


Figure 11-52: PADS_PORTC1_DS Register Diagram

Table 11-57: PADS_PORTC1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PC15	DS control for PC15 IO.
20:18 (R/W)	PC14	DS control for PC14 IO.
17:15 (R/W)	PC13	DS control for PC13 IO.
14:12 (R/W)	PC12	DS control for PC12 IO.
11:9 (R/W)	PC11	DS control for PC11 IO.
8:6 (R/W)	PC10	DS control for PC10 IO.
5:3 (R/W)	PC9	DS control for PC9 IO.
2:0 (R/W)	PC8	DS control for PC8 IO.

PORTD 0 to 7 pins DS control

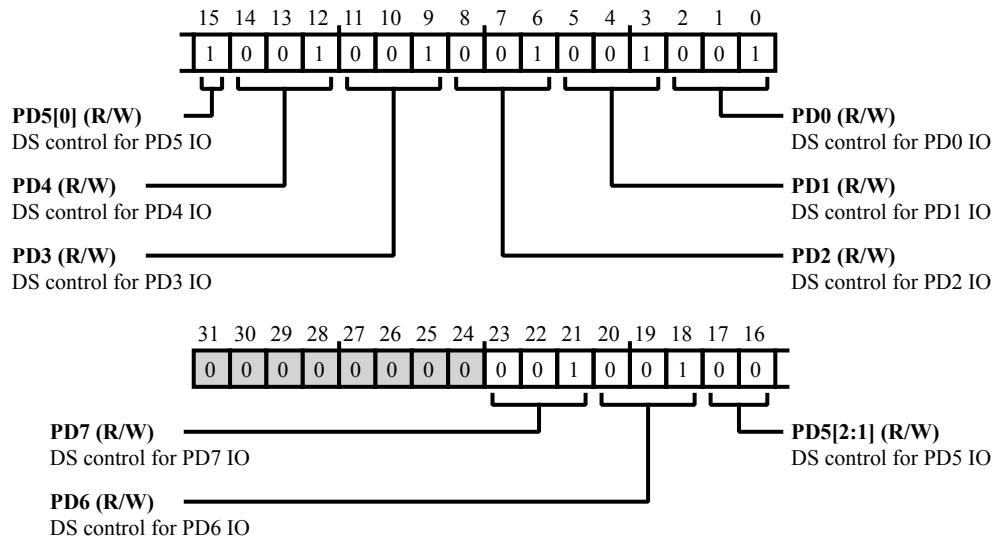


Figure 11-53: PADS_PORTD0_DS Register Diagram

Table 11-58: PADS_PORTD0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PD7	DS control for PD7 IO.
20:18 (R/W)	PD6	DS control for PD6 IO.
17:15 (R/W)	PD5	DS control for PD5 IO.
14:12 (R/W)	PD4	DS control for PD4 IO.
11:9 (R/W)	PD3	DS control for PD3 IO.
8:6 (R/W)	PD2	DS control for PD2 IO.
5:3 (R/W)	PD1	DS control for PD1 IO.
2:0 (R/W)	PD0	DS control for PD0 IO.

PORTD 8 to 15 pins DS control

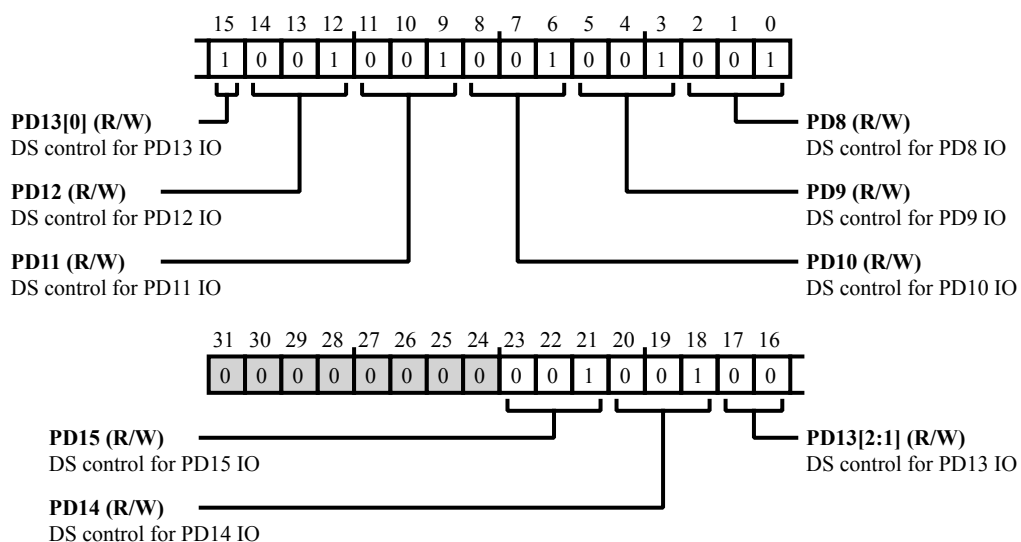


Figure 11-54: PADS_PORTD1_DS Register Diagram

Table 11-59: PADS_PORTD1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PD15	DS control for PD15 IO.
20:18 (R/W)	PD14	DS control for PD14 IO.
17:15 (R/W)	PD13	DS control for PD13 IO.
14:12 (R/W)	PD12	DS control for PD12 IO.
11:9 (R/W)	PD11	DS control for PD11 IO.
8:6 (R/W)	PD10	DS control for PD10 IO.
5:3 (R/W)	PD9	DS control for PD9 IO.
2:0 (R/W)	PD8	DS control for PD8 IO.

PORTE 0 to 7 pins DS control

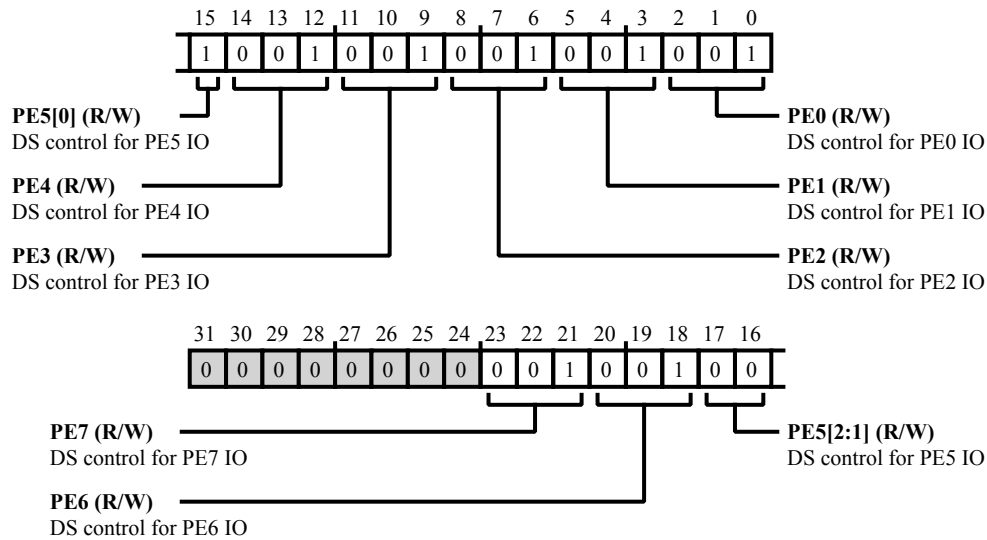


Figure 11-55: PADS_PORTE0_DS Register Diagram

Table 11-60: PADS_PORTE0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PE7	DS control for PE7 IO.
20:18 (R/W)	PE6	DS control for PE6 IO.
17:15 (R/W)	PE5	DS control for PE5 IO.
14:12 (R/W)	PE4	DS control for PE4 IO.
11:9 (R/W)	PE3	DS control for PE3 IO.
8:6 (R/W)	PE2	DS control for PE2 IO.
5:3 (R/W)	PE1	DS control for PE1 IO.
2:0 (R/W)	PE0	DS control for PE0 IO.

PORTE 8 to 15 pins DS control

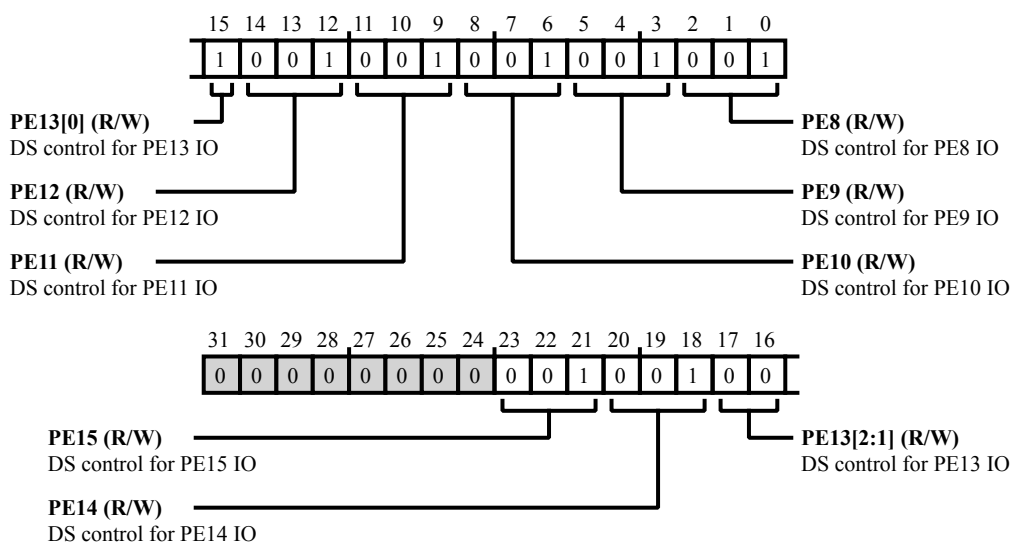


Figure 11-56: PADS_PORTE1_DS Register Diagram

Table 11-61: PADS_PORTE1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PE15	DS control for PE15 IO.
20:18 (R/W)	PE14	DS control for PE14 IO.
17:15 (R/W)	PE13	DS control for PE13 IO.
14:12 (R/W)	PE12	DS control for PE12 IO.
11:9 (R/W)	PE11	DS control for PE11 IO.
8:6 (R/W)	PE10	DS control for PE10 IO.
5:3 (R/W)	PE9	DS control for PE9 IO.
2:0 (R/W)	PE8	DS control for PE8 IO.

PORTF 0 to 7 pins DS control

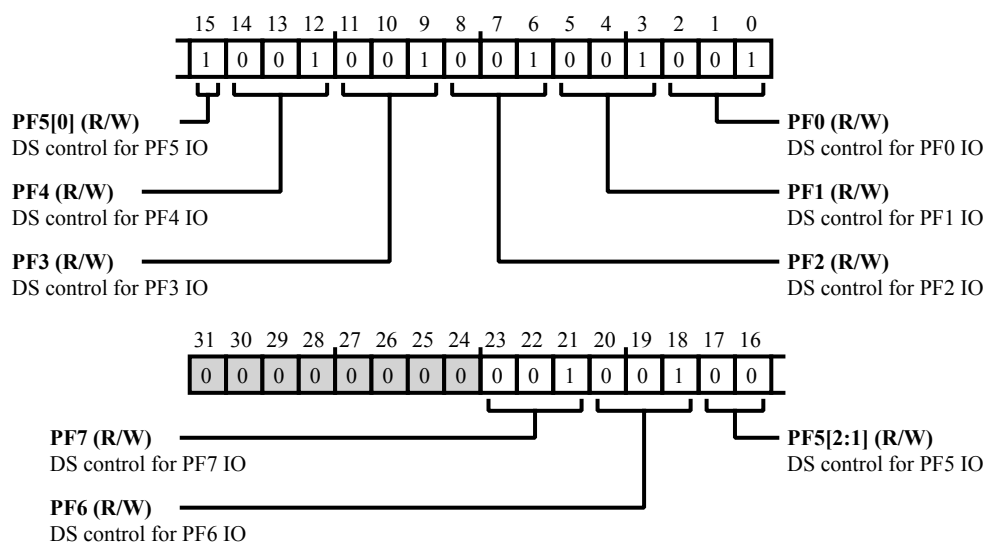


Figure 11-57: PADS_PORTF0_DS Register Diagram

Table 11-62: PADS_PORTF0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PF7	DS control for PF7 IO.
20:18 (R/W)	PF6	DS control for PF6 IO.
17:15 (R/W)	PF5	DS control for PF5 IO.
14:12 (R/W)	PF4	DS control for PF4 IO.
11:9 (R/W)	PF3	DS control for PF3 IO.
8:6 (R/W)	PF2	DS control for PF2 IO.
5:3 (R/W)	PF1	DS control for PF1 IO.
2:0 (R/W)	PF0	DS control for PF0 IO.

PORTF 8 to 15 pins DS control

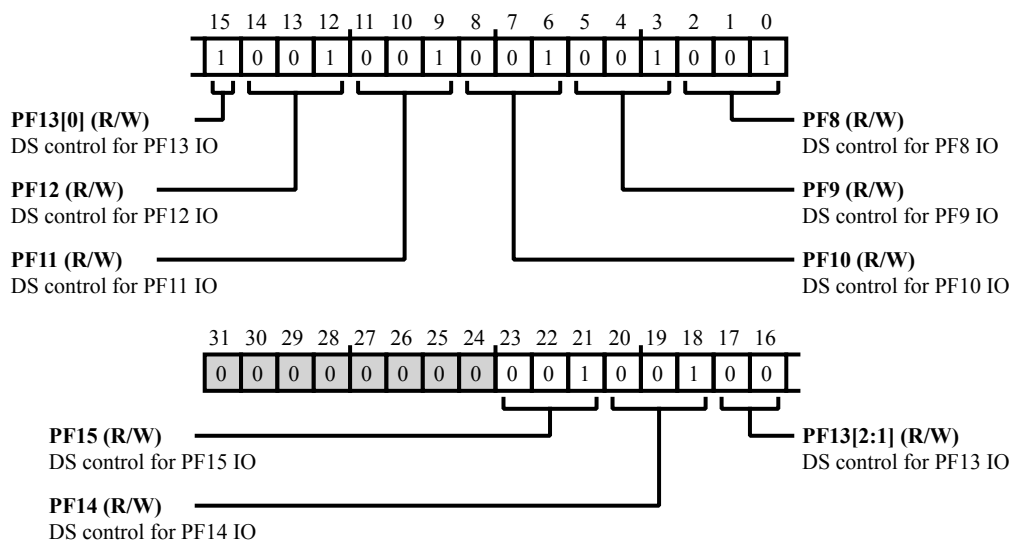


Figure 11-58: PADS_PORTF1_DS Register Diagram

Table 11-63: PADS_PORTF1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PF15	DS control for PF15 IO.
20:18 (R/W)	PF14	DS control for PF14 IO.
17:15 (R/W)	PF13	DS control for PF13 IO.
14:12 (R/W)	PF12	DS control for PF12 IO.
11:9 (R/W)	PF11	DS control for PF11 IO.
8:6 (R/W)	PF10	DS control for PF10 IO.
5:3 (R/W)	PF9	DS control for PF9 IO.
2:0 (R/W)	PF8	DS control for PF8 IO.

PORTG 0 to 7 pins DS control

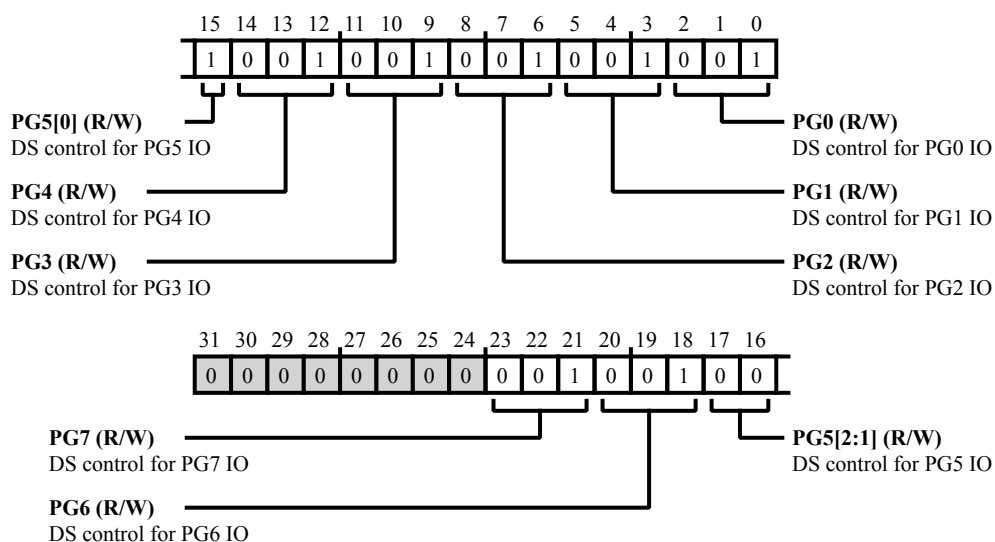


Figure 11-59: PADS_PORTG0_DS Register Diagram

Table 11-64: PADS_PORTG0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PG7	DS control for PG7 IO.
20:18 (R/W)	PG6	DS control for PG6 IO.
17:15 (R/W)	PG5	DS control for PG5 IO.
14:12 (R/W)	PG4	DS control for PG4 IO.
11:9 (R/W)	PG3	DS control for PG3 IO.
8:6 (R/W)	PG2	DS control for PG2 IO.
5:3 (R/W)	PG1	DS control for PG1 IO.
2:0 (R/W)	PG0	DS control for PG0 IO.

PORTG 8 to 15 pins DS control

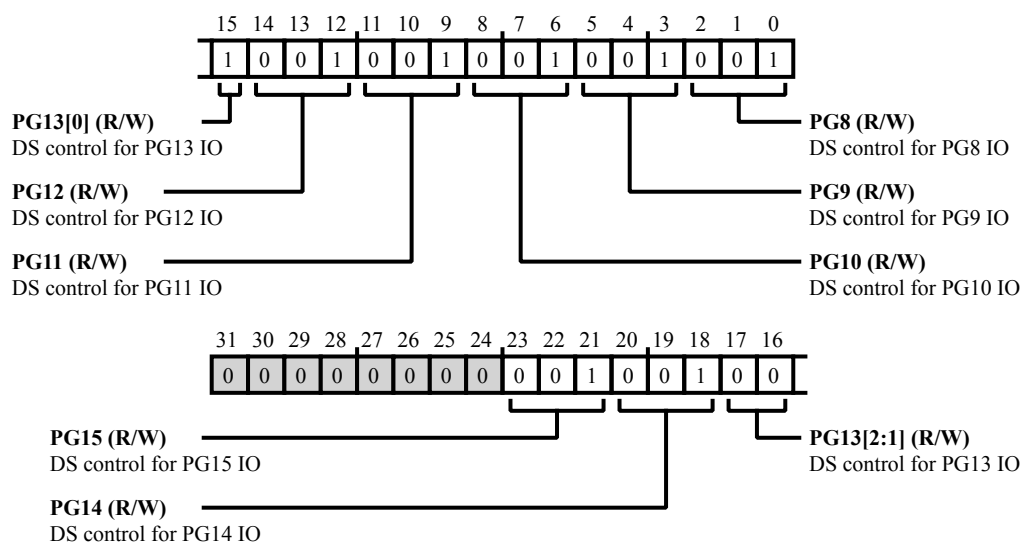


Figure 11-60: PADS_PORTG1_DS Register Diagram

Table 11-65: PADS_PORTG1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PG15	DS control for PG15 IO.
20:18 (R/W)	PG14	DS control for PG14 IO.
17:15 (R/W)	PG13	DS control for PG13 IO.
14:12 (R/W)	PG12	DS control for PG12 IO.
11:9 (R/W)	PG11	DS control for PG11 IO.
8:6 (R/W)	PG10	DS control for PG10 IO.
5:3 (R/W)	PG9	DS control for PG9 IO.
2:0 (R/W)	PG8	DS control for PG8 IO.

PORTH 0 to 7 pins DS control

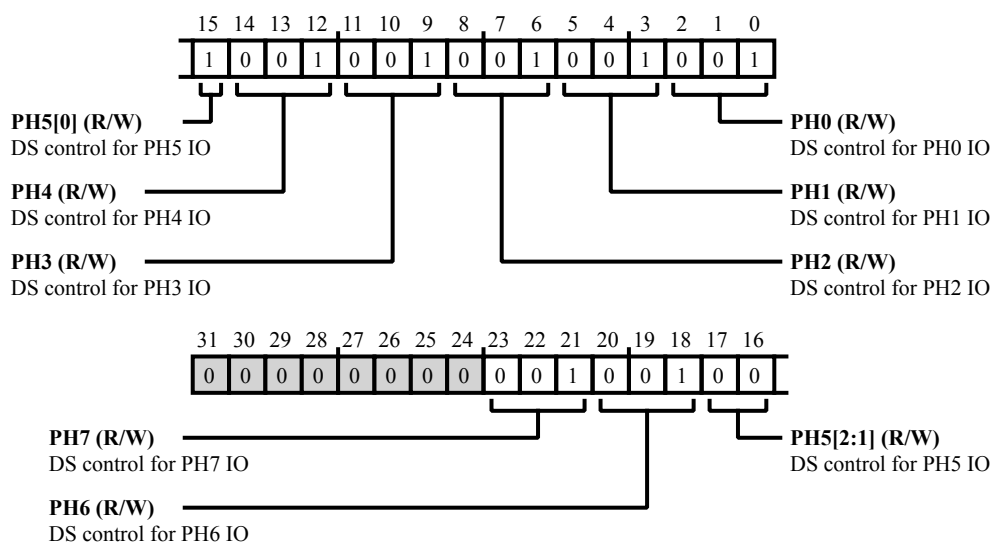


Figure 11-61: PADS_PORTH0_DS Register Diagram

Table 11-66: PADS_PORTH0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PH7	DS control for PH7 IO.
20:18 (R/W)	PH6	DS control for PH6 IO.
17:15 (R/W)	PH5	DS control for PH5 IO.
14:12 (R/W)	PH4	DS control for PH4 IO.
11:9 (R/W)	PH3	DS control for PH3 IO.
8:6 (R/W)	PH2	DS control for PH2 IO.
5:3 (R/W)	PH1	DS control for PH1 IO.
2:0 (R/W)	PH0	DS control for PH0 IO.

PORTH 8 to 15 pins DS control

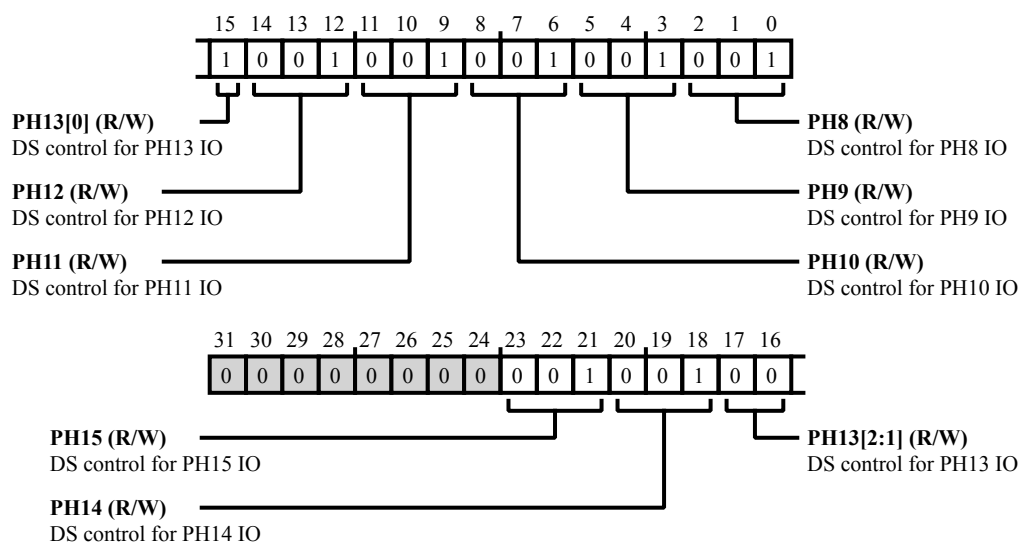


Figure 11-62: PADS_PORTH1_DS Register Diagram

Table 11-67: PADS_PORTH1_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PH15	DS control for PH15 IO.
20:18 (R/W)	PH14	DS control for PH14 IO.
17:15 (R/W)	PH13	DS control for PH13 IO.
14:12 (R/W)	PH12	DS control for PH12 IO.
11:9 (R/W)	PH11	DS control for PH11 IO.
8:6 (R/W)	PH10	DS control for PH10 IO.
5:3 (R/W)	PH9	DS control for PH9 IO.
2:0 (R/W)	PH8	DS control for PH8 IO.

PORTI 0 to 7 pins DS control

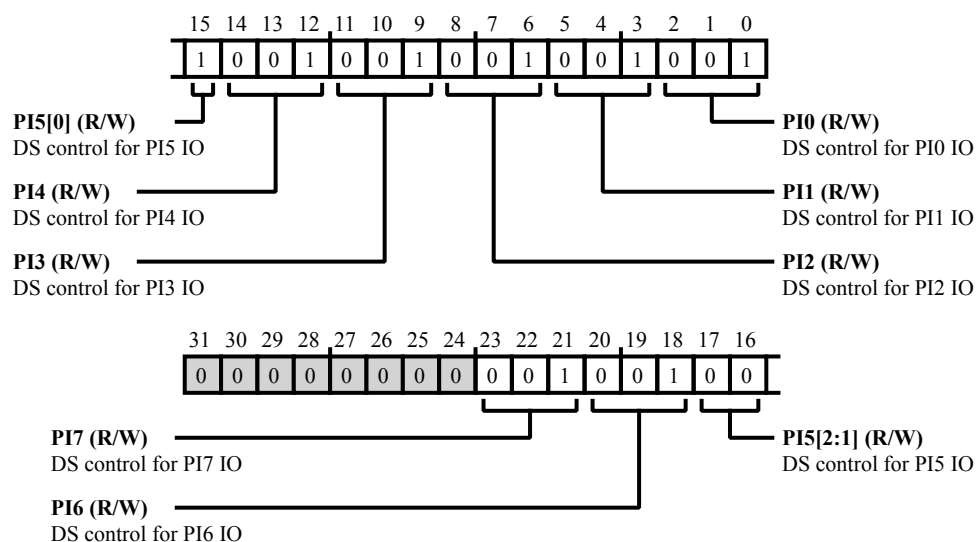


Figure 11-63: PADS_PORTI0_DS Register Diagram

Table 11-68: PADS_PORTI0_DS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:21 (R/W)	PI7	DS control for PI7 IO.
20:18 (R/W)	PI6	DS control for PI6 IO.
17:15 (R/W)	PI5	DS control for PI5 IO.
14:12 (R/W)	PI4	DS control for PI4 IO.
11:9 (R/W)	PI3	DS control for PI3 IO.
8:6 (R/W)	PI2	DS control for PI2 IO.
5:3 (R/W)	PI1	DS control for PI1 IO.
2:0 (R/W)	PI0	DS control for PI0 IO.

PORTx Pull-Down Enable

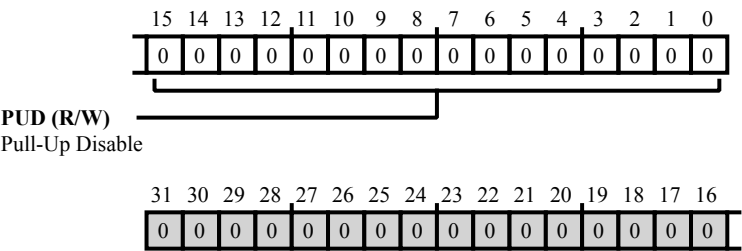


Figure 11-64: PADS_PORT[n]_PDE Register Diagram

Table 11-69: PADS_PORT[n]_PDE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	PUD	Pull-Up Disable.

PORTx Pull-Up Enable

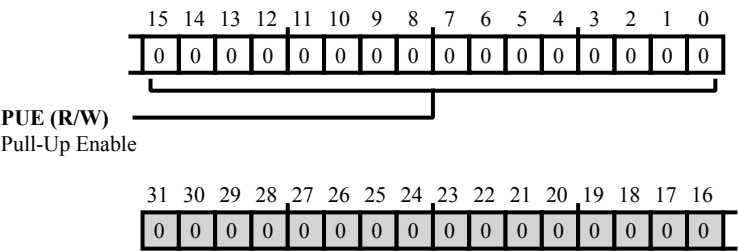


Figure 11-65: PADS_PORT[n]_PUE Register Diagram

Table 11-70: PADS_PORT[n]_PUE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	PUE	Pull-Up Enable.

12 Thermal Monitoring Unit (TMU)

The TMU provides on-chip temperature measurement which is important in applications that have substantial power consumption. The TMU is integrated into the processor die and digital infrastructure using an MMR-based system access to measure the die temperature variations in real-time.

TMU Features

The TMU supports the following features:

- On-chip temperature sensing
- Programmable over-temperature and under-temperature limits
- Programmable conversion rate
- Averaging feature available
- Temperature gain and offset correction options
- Uses dedicated channel of HADC for conversion
- Programmable blanking and refresh period for TMU conversion

TMU Functional Description

Following sections provide the functional description of TMU.

ADSP-2159x TMU Register List

Thermal monitoring unit

Table 12-1: ADSP-2159x TMU Register List

Name	Description
TMU_ALERT_LIM_HI	Alert High Limit Register
TMU_ALERT_LIM_LO	Alert Low Limit Register
TMU_AVG	Averaging Register

Table 12-1: ADSP-2159x TMU Register List (Continued)

Name	Description
TMU_CNV_BLANK	Temperature conversion blank register
TMU_CTL	TMU Control Register
TMU_FLT_LIM_HI	Fault High Limit Register
TMU_FLT_LIM_LO	Fault Low Limit Register
TMU_GAIN	Gain Value Register
TMU_IMSK	Interrupt Mask Register
TMU_OFFSET	Offset Register
TMU_REFR_CNTR	Temperature Refresh Counter
TMU_STAT	Status Register
TMU_TEMP	Temperature Value Register

ADSP-2159x TMU Interrupt List

Table 12-2: ADSP-2159x TMU Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
7	TMU0_FAULT	TMU0 Fault		
8	TMU0_ALERT	TMU0 Fault		

ADSP-2159x TMU Trigger List

Table 12-3: ADSP-2159x TMU Trigger List Masters

Trigger ID	Name	Description	Sensitivity
158	TMU0_FAULT	TMU0 TM0 Fault Event	
159	TMU0_ALERT	TMU0 TM0 Alert Event	

Table 12-4: ADSP-2159x TMU Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

TMU Definitions

The following definitions are helpful when using the TMU module.

Thermal Diode

A special type of diode whose electrical properties change with the temperature in a defined way.

DTM

Dynamic thermal management is a set of techniques that adapt the run-time behavior of a processor to achieve the highest performance under thermal constraints.

TMU Block Diagram

The *Thermal Monitoring Unit Block Diagram* shows the main block inside TMU.

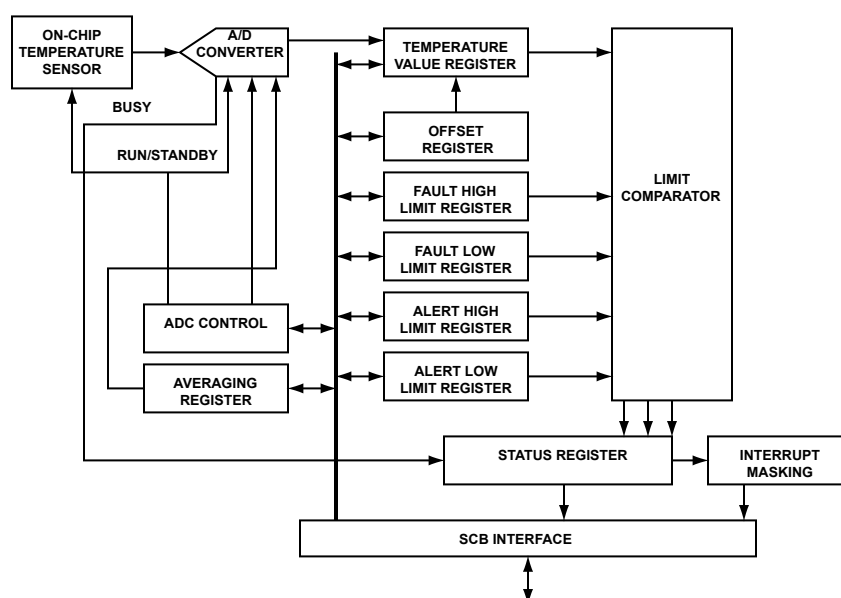


Figure 12-1: Thermal Monitoring Unit Block Diagram

TMU Architectural Concepts

The following sections provide information on the architecture and system integration of the TMU.

System Integration

Dynamic thermal management (DTM) techniques adapt the run-time behavior of a processor to achieve the highest performance under thermal constraints. One of the most important aspects of DTM is to capture the run-time variations in the temperature caused by power consumption variations due to workload changes.

The accuracy of thermal measurements directly affects the efficiency of thermal management as well as the performance of the processor. Temperature estimations lower or higher than the actual temperature may cause late or early activation of DTM techniques.

- Late activation of DTM can result in degraded reliability because the temperature may exceed the designated thresholds.
- Early activation of DTM can have significant impact on performance.

The TMU is an analog thermal sensor that consists of a temperature-sensing diode, a calibrated reference current source, and a current comparator. The sensor placement error is one of the most important sources of inaccuracy in values obtained from thermal sensors.

The TMU module provides the temperature monitoring capability to the chip and implements thermal management in the end system. It provides many features which ensure the minimum load on the software and also minimal or no external components for a flexible temperature monitoring system.

Digital Thermometer

The TMU functions as a digital thermometer with an MMR-based system access. The on-die temperature value is measured and digitized through an A/D converter. The temperature value is stored and updated periodically in an MMR register. The TMU can be configured to generate an interrupt as it crosses the upper temperature limit (`TMU_FLT_LIM_HI` register). A thermal event is also routed to the interrupt port of the sensor. The event is then routed to the SEC, to ensure that core intervention is not required in the event of overheating.

Temperature Sensor Averaging

The temperature sensor averaging feature enhances the accuracy of the temperature measurements. To enable this feature, the `TMU_AVG` bits must be enabled in the control register after power-up. In this mode, the averaging reduces the effect of noise on the temperature result. The temperature is measured each time a conversion is performed. A moving average method is used to determine the result in the temperature value register. The total time to measure a temperature channel is typically 1 ms.

TMU and HADC

The TMU module uses one of the ADC channels to digitize the temperature reading. Therefore, the HADC module is shared by the TMU. If the TMU is not enabled, the HADC operates at its full throughput. TMU can be enabled periodically or permanently. In periodic enable mode, it is enabled once in a specific refresh period. After enabling, the TMU measures the temperature for a specific blanking period. During the blanking period, the TMU uses the ADC for temperature measurement, so that the ADC is not available for data conversion.

The typical value of blanking period is 1×10^6 system clock (SCLK) cycles

The typical value of refresh period is 21×10^6 SCLK cycles

So, for every 21×10^6 cycles, 1×10^6 cycles are used for temperature measurement. Program these values for blanking period and refresh period through the TMU registers.

When TMU is enabled, the HADC module is not available for external input conversion. This state is indicated by the `HADC_STAT.TMUHADC_BUSY` status bit. All the requests for HADC conversion are ignored when `HADC_STAT.TMUHADC_BUSY` bit is high.

In the periodic enable mode, the ongoing HADC conversion is completed activating the TMU to measure temperature as shown. If HADC is in autoscan mode, the TMU is activated after completing the ongoing sequence. If HADC is in fixed-conversion mode, all the fixed conversions are completed and followed by TMU activation.

TMU Event Control

The TMU generates different events depending on the state of the TMU temperature measurement and the different thresholds set in thresh hold registers. These events are reported in the `TMU_STAT` register as shown below. It can generate an event for each of following conditions.

- The fault high limit is configured in the `TMU_FLT_LIM_HI` register. The interrupt is generated when the temperature value is greater than or equal to this value.
- The alert high limit is configured in the `TMU_ALRT_LIM_HI` register. The interrupt is generated when the temperature value is greater than or equal to this value.
- The fault low limit is configured in the `TMU_FLT_LIM_LO` register. The `TMU_STAT.FLTLO` status bit is set when the temperature value is less than or equal to this value.
- The alert low limit is configured in the `TMU_ALRT_LIM_LO` register. The `TMU_STAT.FLTLO` status bit is set when the temperature value is less than or equal to this value.

Interrupts and status conditions can be masked (disabled) or unmasked (enabled) by setting and clearing bits in the `TMU_IMSK` register.

NOTE: Only `TMU_FLT_LIM_HI` and `TMU_ALRT_LIM_HI` generate an interrupt to core. Other events only result in status change in the `TMU_STAT` register.

Status and Error Signals

When the measured temperature value exceeds the high or low limits that are configured in the `TMU_FLT_LIM_HI`/`TMU_FLT_LIM_LO` and the `TMU_ALRT_LIM_HI`/`TMU_ALRT_LIM_LO` registers, corresponding thermal events are triggered.

The fault and alert events are sent to the core through the status register (`TMU_STAT`). Both the `TMU_STAT.FLTHI` and `TMU_STAT.ALRTHI` bits are sticky bits and are cleared by a W1C operation by the core.

The alert and fault registers can be programmed through the MMR bus interface as shown in the TMU block diagram. A write into the `TMU_FLT_LIM_HI`/`TMU_FLT_LIM_LO` and `TMU_ALRT_LIM_HI`/`TMU_ALRT_LIM_LO` registers or the `TMU_TEMP` register is not allowed when the events are being triggered.

TMU Programming Guidelines

The following section provides basic programming information for the TMU module.

To get the best performance and accuracy from the TMU, initialize the `TMU_GAIN`, `TMU_OFFSET` and `TMU_AVG` registers before enabling the module. Contact Analog Devices, Inc for the current best values to use.

After these registers are programmed, the rest of the TMU initialization can take place. This includes setting up the fault and alert limits and then enabling the TMU.

ADSP-2159x TMU Register Descriptions

Thermal monitoring unit (TMU) contains the following registers.

Table 12-5: ADSP-2159x TMU Register List

Name	Description
<code>TMU_ALERT_LIM_HI</code>	Alert High Limit Register
<code>TMU_ALERT_LIM_LO</code>	Alert Low Limit Register
<code>TMU_AVG</code>	Averaging Register
<code>TMU_CNV_BLANK</code>	Temperature conversion blank register
<code>TMU_CTL</code>	TMU Control Register
<code>TMU_FLT_LIM_HI</code>	Fault High Limit Register
<code>TMU_FLT_LIM_LO</code>	Fault Low Limit Register
<code>TMU_GAIN</code>	Gain Value Register
<code>TMU_IMSK</code>	Interrupt Mask Register
<code>TMU_OFFSET</code>	Offset Register
<code>TMU_REFR_CNTR</code>	Temperature Refresh Counter
<code>TMU_STAT</code>	Status Register
<code>TMU_TEMP</code>	Temperature Value Register

Alert High Limit Register

The `TMU_ALERT_LIM_HI` register sets the temperature alert high limit as an integer value. The value is stored in two's complement format. Asserts `TMU_STAT.ALRTHI` if the `TMU_TEMP` value is greater than or equal to `TMU_ALERT_LIM_HI`. The `TMU_ALERT_LIM_HI` value should be programmed for value greater than 8'h3C (60 degC).

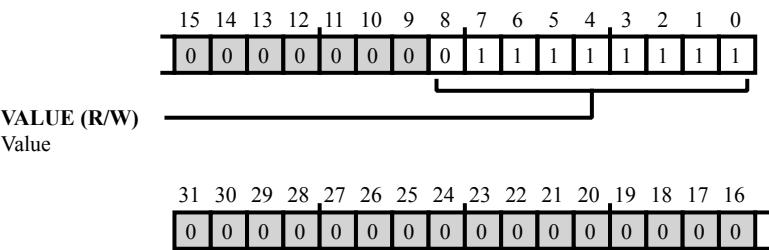


Figure 12-2: TMU_ALERT_LIM_HI Register Diagram

Table 12-6: TMU_ALERT_LIM_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/W)	VALUE	Value. The <code>TMU_ALERT_LIM_HI.VALUE</code> bit field configures the alert temperature high limit as an integer value stored in two's complement format. If the temperature value is greater than or equal to the high-limit the <code>TMU_STAT.ALRTHI</code> bit is set.

Alert Low Limit Register

The `TMU_ALERT_LIM_LO` register configures the alert temperature low limit.

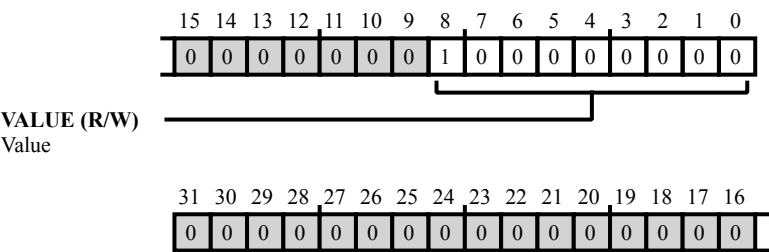


Figure 12-3: `TMU_ALERT_LIM_LO` Register Diagram

Table 12-7: `TMU_ALERT_LIM_LO` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/W)	VALUE	Value. The <code>TMU_ALERT_LIM_LO.VALUE</code> bit field configures the alert temperature low limit as an integer value stored in two's complement format. If the temperature value is less than or equal to the low-limit the <code>TMU_STAT.ALRTLO</code> bit is set.

Averaging Register

The `TMU_AVG` register enables averaging on the TMU.

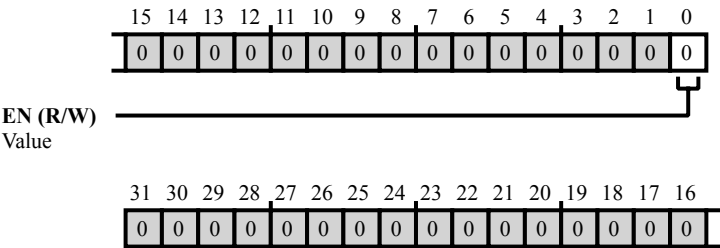


Figure 12-4: `TMU_AVG` Register Diagram

Table 12-8: `TMU_AVG` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Value. The <code>TMU_AVG.EN</code> bit enables averaging on the TMU. Averaging is done using the formula $(7 \times \text{previous_avg_value} + \text{current_value})/8$. Initially the <code>current_value</code> is taken as <code>previous_avg_value</code> .
		0 No averaging
		1 Enable averaging

Temperature conversion blank register

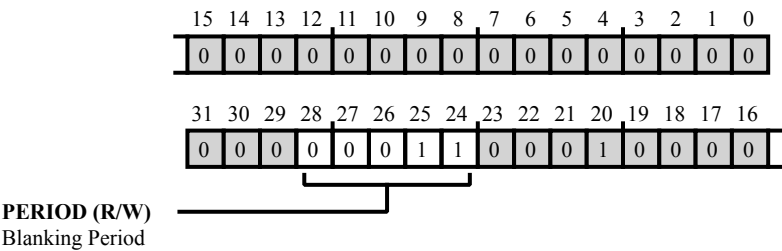


Figure 12-5: TMU_CNv_BLANK Register Diagram

Table 12-9: TMU_CNv_BLANK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:24 (R/W)	PERIOD	Blanking Period. The TMU_CNv_BLANK.PERIOD specifies the blanking period. TMU uses the HADC for temperature conversion during this period. The conversion time is calculated as: (CNV_BLANK+1)*50k SCLK cycles. So, the minimum conversion time is 50k SCLK cycles. Default conversion time is 20*10^4 SCLK cycles. For this duration, the system cannot use HADC for data conversion and HADC does take input for data-conversion. The status is indicated by TMUHADC_BUSY bit of the HADC block.

TMU Control Register

The `TMU_CTL` register contains bits that allow programs to configure and enable the TMU.

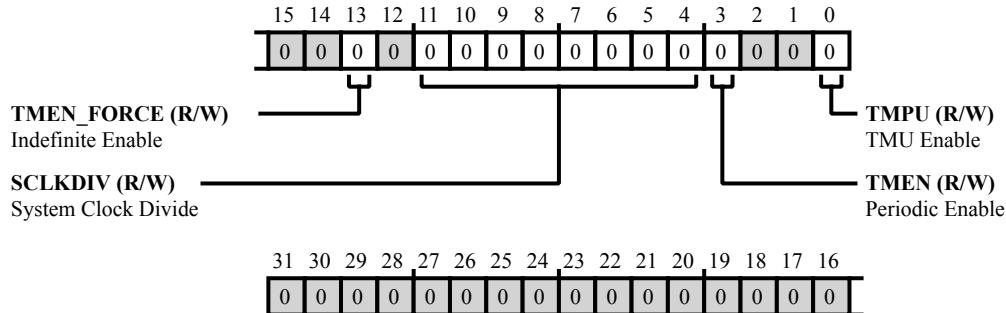


Figure 12-6: TMU_CTL Register Diagram

Table 12-10: TMU_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	TMEN_FORCE	Indefinite Enable. Asserting the <code>TMU_CTL.TMEN_FORCE</code> bit enables the TMU indefinitely. HADC is always used by the TMU, so the HADC cannot be used to convert channel inputs.
11:4 (R/W)	SCLKDIV	System Clock Divide. The <code>TMU_CTL.SCLKDIV</code> bit selects the division ratio for the system clock (SCLK). SCLK is divided by $(21+4*SCLKDIV)$. Thus, by default the SCLK is divided by 21.
3 (R/W)	TMEN	Periodic Enable. Asserting the <code>TMU_CTL.TMEN</code> bit enables the TMU periodically. TMU is enabled once in every refresh period defined by the <code>TMU_REFR_CNTR</code> register. And in each refresh period, TMU measures temperature for the blanking period (Tblank). It is defined by CNV_BLANK bit.
		0 Disable TMU
		1 Enable TMU periodically
0 (R/W)	TMPU	TMU Enable. The <code>TMU_CTL.TMPU</code> bit enables the TMU. By default, the module is in power-down mode. The peripheral interface is active even in power-down mode (the TMU registers can be read/written).
		0 Power Down the analog circuitry of TMU
		1 Power Up the analog circuitry of TMU

Fault High Limit Register

The `TMU_FLT_LIM_HI` register sets the temperature fault high limit as an integer value. The value is stored in two's complement format. Asserts `TMU_STAT.FLTHI` if the `TMU_TEMP` value is greater than or equal to `TMU_FLT_LIM_HI`. The `TMU_FLT_LIM_HI` value should be programmed for value greater than 8'h3C (60 degC).

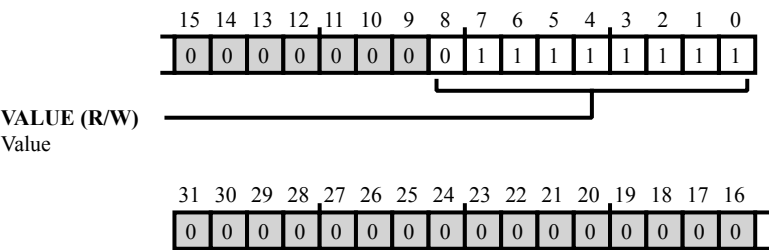


Figure 12-7: TMU_FLT_LIM_HI Register Diagram

Table 12-11: TMU_FLT_LIM_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/W)	VALUE	Value. The <code>TMU_FLT_LIM_HI.VALUE</code> bit field sets the temperature high limit as an integer in two's complement format. If the limit value is greater than or equal to the high-limit the <code>TMU_STAT.FLTHI</code> bit is set.

Fault Low Limit Register

The `TMU_FLT_LIM_LO` register configures the fault temperature low limit.

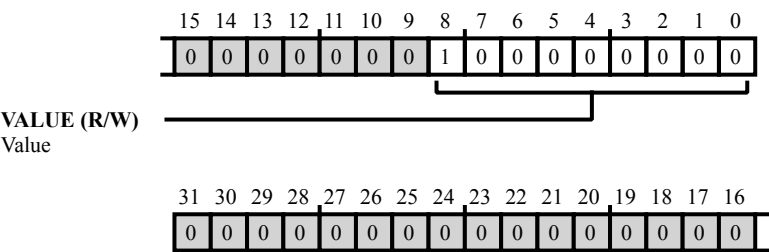


Figure 12-8: `TMU_FLT_LIM_LO` Register Diagram

Table 12-12: `TMU_FLT_LIM_LO` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/W)	VALUE	Value. The <code>TMU_FLT_LIM_LO.VALUE</code> bit field configures the fault temperature low limit as an integer which is stored in two's complement format. If the temperature value is less than or equal to the low-limit, the <code>TMU_STAT.FLTLO</code> bit is set.

Gain Value Register

The `TMU_GAIN` register is used to configure the gain value in two's complement format. This value is used to correct the gain error in the `TMU_TEMP` register.

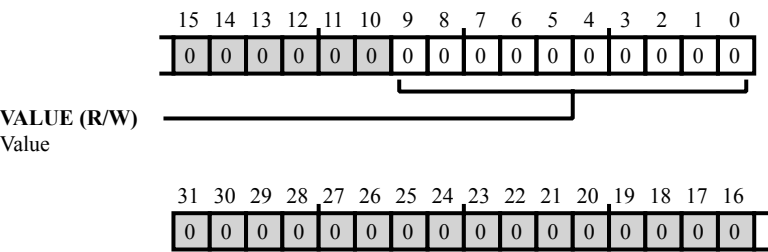


Figure 12-9: TMU_GAIN Register Diagram

Table 12-13: TMU_GAIN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/W)	VALUE	Value. The <code>TMU_GAIN.VALUE</code> bit field configures the gain value in two's complement format. This value is used to correct the gain error in the <code>TMU_TEMP</code> register.

Interrupt Mask Register

The `TMU_IMSK` register provides bit that are used to mask and unmask the interrupts associated with the TMU module.

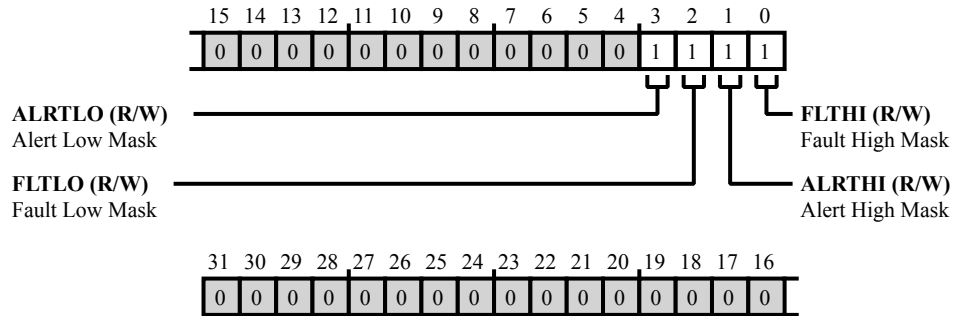


Figure 12-10: TMU_IMSK Register Diagram

Table 12-14: TMU_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	ALRTLO	Alert Low Mask. The <code>TMU_IMSK.ALRTLO</code> bit masks or unmasks the alert low status change (<code>TMU_STAT.ALRTLO</code>).
		0 Unmask alert low status change
		1 Mask alert low status change
2 (R/W)	FLTLO	Fault Low Mask. The <code>TMU_IMSK.FLTLO</code> bit masks or unmasks the fault low status change (<code>TMU_STAT.FLTLO</code>).
		0 Unmask fault low status change
		1 Mask fault low status change
1 (R/W)	ALRTHI	Alert High Mask. The <code>TMU_IMSK.ALRTHI</code> bit masks or unmasks the alert high status change (<code>TMU_STAT.ALRTHI</code>).
		0 Unmask alert high status change
		1 Mask alert high status change
0 (R/W)	FLTHI	Fault High Mask. The <code>TMU_IMSK.FLTHI</code> bit masks or unmasks the fault high interrupt (<code>TMU_STAT.FLTHI</code>).
		0 Unmask fault high interrupt
		1 Mask fault high interrupt

Offset Register

The value programmed in the `TMU_OFFSET` register is used to correct the offset error in `TMU_TEMP` register.

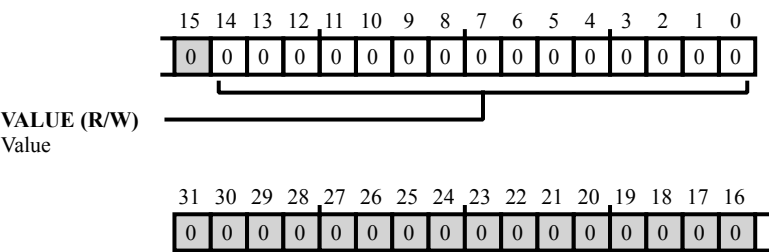


Figure 12-11: TMU_OFFSET Register Diagram

Table 12-15: TMU_OFFSET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14:0 (R/W)	VALUE	Value. The <code>TMU_OFFSET.VALUE</code> bit field provides the offset value which is used to correct the offset error in the <code>TMU_TEMP</code> register. This value is in a two's complement fixed point Q7.7 format i.e 8 MSB's represent integer part in 2's complement format. and remaining 7 LSB's represent decimal part. Offset will be applied after multiplying gain.

Temperature Refresh Counter

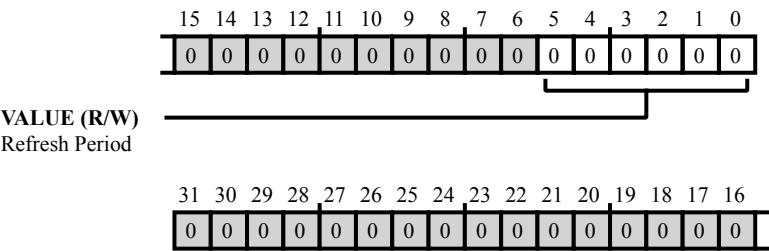


Figure 12-12: TMU_REFR_CNTR Register Diagram

Table 12-16: TMU_REFR_CNTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5:0 (R/W)	VALUE	Refresh Period. The TMU_REFR_CNTR.VALUE bit field defines the period to refresh the temperature value. Temperature is refreshed for every (VALUE+1)*(21+4*SCLKDIV)*1M system-clock cycles. SCLKDIV is defined in TMU_CTL register. So, by default, the period is 21 million system-clock cycles.

Status Register

The `TMU_STAT` register bits indicate when an error or fault is detected.

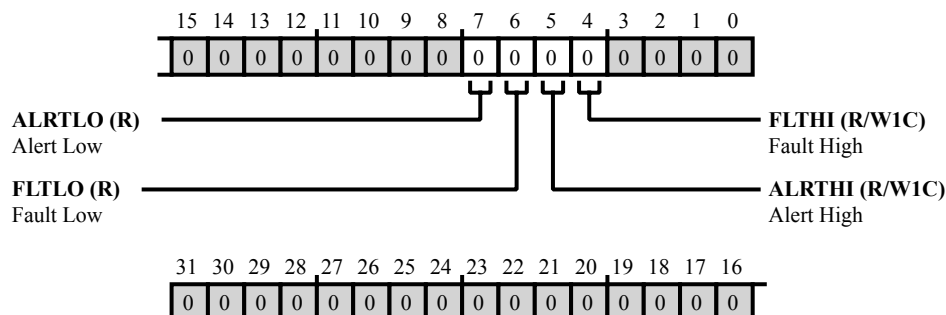


Figure 12-13: `TMU_STAT` Register Diagram

Table 12-17: `TMU_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	ALRTLO	Alert Low. The <code>TMU_STAT.ALRTLO</code> bit is set when the temperature value is less than or equal to the setting in the <code>TMU_ALRT_LIM_LO</code> register. The bit assertion is masked if <code>TMU_IMSK.ALRTLO</code> is HIGH.
6 (R/NW)	FLTLO	Fault Low. The <code>TMU_STAT.FLTLO</code> bit is set when the temperature value is less than or equal to the setting in the <code>TMU_FLT_LIM_LO</code> register. The bit assertion is masked if <code>TMU_IMSK.FLTLO</code> is HIGH.
5 (R/W1C)	ALRTHI	Alert High. The <code>TMU_STAT.ALRTHI</code> bit is set when the temperature value is greater than or equal to the setting in the <code>TMU_ALRT_LIM_HI</code> register. The temperature value register is set to 16'h2C00 (effective temperature value of 44 degrees C) when a W1C operation is done on this bit. The bit assertion is masked if <code>TMU_IMSK.ALRTHI</code> is HIGH.
4 (R/W1C)	FLTHI	Fault High. The <code>TMU_STAT.FLTHI</code> bit is set when the temperature value is greater than or equal to the setting in the <code>TMU_FLT_LIM_HI</code> register. The temperature value register is set to 16'h2C00 (effective temperature value of 44) when a W1C is done on this bit. The bit assertion is masked if <code>TMU_IMSK.FLTHI</code> is HIGH.

Temperature Value Register

The `TMU_TEMP` register provides the temperature value from the A/D converter and the status.

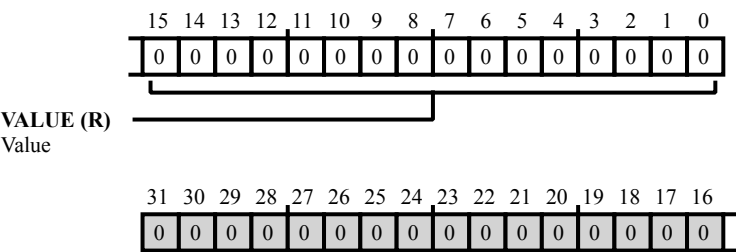


Figure 12-14: `TMU_TEMP` Register Diagram

Table 12-18: `TMU_TEMP` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	Value. The <code>TMU_TEMP.VALUE</code> bit field is the temperature value from A/D converter. This value is stored in two's complement fixed point Q7.8 format where the 8 MSB's represent the integer part in 2's complement, and the remaining 8 LSB's represent the decimal part.

13 Housekeeping ADC (HADC)

The Housekeeping ADC is a 12-bit (with 10-bit accuracy), successive approximation ADC. It operates from single supply and features throughput rates up to 1 MSPS. The HADC can be used for the collection of housekeeping parameters like voltages, temperatures in the system or for any general-purpose use as well.

NOTE: HADC is used by the TMU for temperature monitor, so it may be unavailable during temperature conversion. For details, refer to the [TMU and HADC](#) section.

HADC Features

The HADC supports following features:

- 12-bit ADC core with built-in sample and hold
- ENOB = 10 bit
- 8 input channels which can be extended to 15 channels by putting an external channel multiplexer
- Throughput rates up to 1 MSPS
- Single ended operation
- External reference nominal at 1.8 V
- Analog input 0 V to 1.8 V.
- Selectable ADC clock frequency through a pre-scaler
- Conversion type adaptable to each application: allows single or continuous conversion with option of auto-scan
- Number of channels: 4 for BGA and 2 for LQFP
- Auto sequencing capability provides up to 8 *auto-conversions* in a single session. Each conversion can be programmed to select any of the available input channels.

HADC Functional Description

The HADC provides the analog to digital conversion capability for general-purpose housekeeping tasks, such as voltage and temperature monitoring. The core of HADC is a 12-bit SAR ADC, providing multiple analog input channels.

The HADC has the following functionality:

Fixed and continuous conversion modes

ADC converts the input channel sequence for a fixed number of times or continuously converts an input channel sequence.

Auto scanning

All the input channels can be sampled in a sequential manner.

Channel sequence programming

The sequence of a channel can be selected by programming the channel mask register. If the bit corresponding to the channel is programmed to zero, that channel is included in the auto-scan chain.

ADSP-2159x HADC Register List

The Housekeeping ADC (HADC) provides a general purpose, multi-channel successive approximation A-to-D converter. A set of registers governs HADC operations. For more information on HADC functionality, see the HADC register descriptions.

Table 13-1: ADSP-2159x HADC Register List

Name	Description
HADC_CHAN_MSK	Channel Mask Register
HADC_CTL	Control Register
HADC_DATA[nn]	Channel Data Registers
HADC_IMSK	Interrupt Mask Register
HADC_STAT	Status Register

ADSP-2159x HADC Interrupt List

Table 13-2: ADSP-2159x HADC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
186	HADC0_EVT	HADC0 Event	Edge	

ADSP-2159x HADC Trigger List

Table 13-3: ADSP-2159x HADC Trigger List Masters

Trigger ID	Name	Description	Sensitivity
38	HADC0_EOC	HADC0 HADC0 End of Conversion	Edge

Table 13-4: ADSP-2159x HADC Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

HADC Definitions

The following definitions are helpful when using the HADC module.

Auto-scan

Auto-scan is a feature which allows the multiple channels to be scanned and converted in sequence one after the other.

HADC Wakeup Time

It is the time required by the module after coming out of power down before it can start converting.

Signal-to-Noise and Distortion Ratio (SINAD)

The measured ratio of signal-to-noise and distortion at the output of the ADC. The signal is the rms amplitude of the fundamental. Noise is the sum of all non-fundamental signals up to half the sampling frequency ($f_s/2$), excluding dc. The ratio depends on the number of quantization levels in the digitization process; the more levels, the smaller the quantization noise. The theoretical signal-to-noise and distortion ratio for an ideal N-bit converter with a sine wave input is given by:

$$\text{Signal-to-(Noise + Distortion)} = (6.02 N + 1.76) \text{ dB}$$

Total Harmonic Distortion (THD)

The ratio of the rms sum to the harmonics to the fundamental.

Peak Harmonic or Spurious Noise

The ratio of the rms value of the next largest component in the ADC output spectrum (up to $f_s/2$ and excluding dc) to the rms value of the fundamental. Typically, the value of this specification is determined by the largest harmonic in the spectrum, but for ADCs where the harmonics are buried in the noise floor, it is a noise peak.

Integral Nonlinearity

The maximum deviation from a straight line passing through the endpoints of the ADC transfer function. The endpoints are zero scale, a point 1 LSB below the first code transition, and full scale, a point 1 LSB above the last code transition.

Differential Nonlinearity

The difference between the measured and the ideal 1 LSB change between any two adjacent codes in the ADC.

Offset Error

The deviation of the first code transition (00...000) to (00...001) from the ideal—that is, $\text{GND1} + 1 \text{ LSB}$.

Offset Error Match

The difference in offset error between any two channels.

Gain Error

The deviation of the last code transition (111...110) to (111...111) from the ideal (that is, $\text{REFIN} - 1 \text{ LSB}$) after the offset error has been adjusted out.

Gain Error Matching

The difference in gain error between any two channels.

Power Supply Rejection Ratio (PSRR)

PSRR is defined as the ratio of the power in the ADC output at full-scale frequency, f , to the power of a 100 mV p-p sine wave applied to the ADC VDD supply of frequency, f_s . The frequency of the input varies from 5 kHz to 25 MHz. $\text{PSRR (dB)} = 10 \log(P_f/P_{f_s})$ where: P_f is the power at frequency, f , in the ADC output. P_{f_s} is the power at frequency, f_s , in the ADC output.

HADC Block Diagram

The *HADC Block Diagram* figure shows the functional blocks within the HADC and the interface to the processor core and the peripherals. The HADC has an internal 8 channel multiplexer that is controlled by a programmable sequencer which selects the desired channel or sequence of channels.

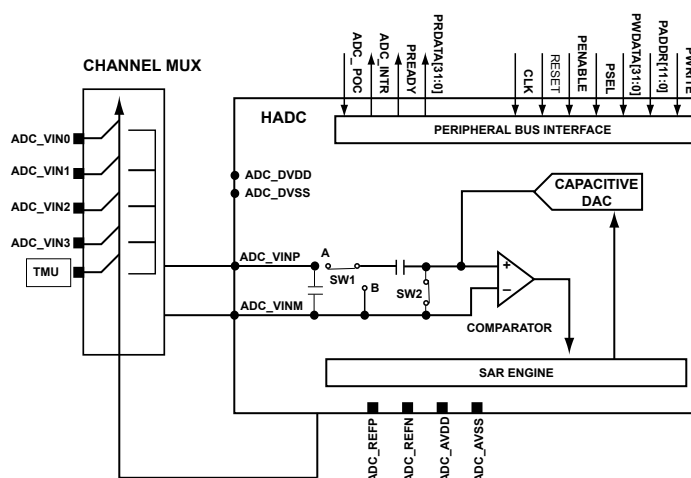


Figure 13-1: HADC Block Diagram

HADC Signal Descriptions

The *HADC Signal Descriptions* table provides descriptions of the signals used by the HADC.

Table 13-5: HADC Signal Descriptions

Signal Name	Signal Description
AVDD	ADC I/O supply
AVSS	I/O ground for analog blocks
VREFP	External reference for ADC
VREFN	Ground reference for ADC
VINn	Analog input at channel n

HADC Architectural Concepts

The HADC is based on a 12-bit SAR ADC that provides a simple register-based access model to obtain the results of conversion. The digital front end of the HADC provides a set of registers to configure the mode of operation, sampling frequency, and input channel selection control. The ADC supports multiple input analog channels which can be individually selected or deselected for conversion. The results of each analog channel are stored in a register. The core can access the register to read the conversion results once the conversion is complete. The HADC also provides the interrupts on completion of each channel conversion to avoid polling by the core. The following sections provide more details about the architecture of the HADC.

Converter Operation

The housekeeping ADC is a 12-bit successive approximation ADC based around a capacitive DAC. The *ADC Acquisition Phase* figure and the *ADC Conversion Phase* figure show simplified schematics of the ADC. The ADC is comprised of control logic, SAR, and a capacitive DAC. The components are used to add and subtract fixed amounts of charge from the sampling capacitor to bring the comparator back into a balanced condition. The *ADC Conversion Phase* figure shows the ADC during its acquisition phase. SW2 is closed and SW1 is in Position A. The

comparator is held in a balanced condition and the sampling capacitor acquires the signal on the selected V_{IN} channel.

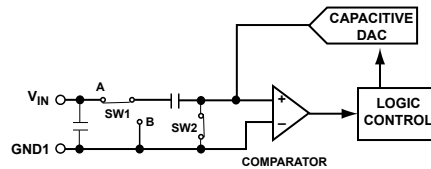


Figure 13-2: ADC Acquisition Phase

When the ADC starts a conversion (see the *ADC Conversion Phase* figure), SW2 opens and SW1 moves to Position B, causing the comparator to become unbalanced. The control logic and the capacitive DAC are used to add and subtract fixed amounts of charge to bring the comparator back into a balanced condition. When the comparator is rebalanced, the conversion is complete. The control logic generates the ADC output code.

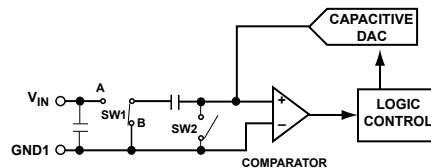


Figure 13-3: ADC Conversion Phase

Auto-Scan

The HADC features auto-scan mode where all the input channels can be sampled in a sequential manner. The number of channels enabled in auto-scan mode can be selected by programming the `HADC_CHAN_MSK` register. If the bit corresponding to the channel is set high, that particular channel is masked, and is not included in the auto-scan chain. In this way programs can sample all, none, or a selected set of channels by writing a high or a low for the individual channel. Auto sequencing allows the system to convert the same channel multiple times, allowing programs to perform oversampling algorithms.

For example, if the `HADC_CHAN_MSK` register bits [3:0] are set to 1101, then channel 0, channel 2 and channel 3 are not included in the auto-scan chain. Whether the conversion is a single or fixed number or continuous depends on the status of `HADC_CTL.CONT` bit. If this bit is low, the `HADC_CTL.FIXEDCNV` bits determine the number of sequence conversions.

The maximum number of fixed sequence conversions is 15. By default, the first eight channels of the HADC are enabled and extended channels are masked. An extended channel is the increased number of channels when a single channel of ADC is externally multiplexed, effectively increasing the total number of available channels. The total number of channels supported is 15. The program must configure the `HADC_CHAN_MSK` register to enable any desired channel.

Channel Sequence Programming

The sequence of a channel can be selected by programming the `HADC_CHAN_MSK` register. If the bit corresponding to the channel is programmed to zero, that channel is included in the auto-scan chain. If the program must get the conversion results from a particular channel, then the bit corresponding to that channel should be zero.

Channels 8-15 are virtual channels, in that they can connect an external multiplexer and increase the effective number of channels. As the output of the external multiplexer is routed to channel 7, this channel is not available (or converted) separately when the virtual channels are not masked in the `HADC_CHAN_MSK` register. The auto-scan section has more details.

ADC Transfer Function

The output coding is straight binary for the analog input channel conversion. The designed code transitions occur at successive LSB values (that is, 1 LSB, 2 LSBs, and so forth). The LSB size is $V_{REF}/4096$ for the HADC. The *ADC Transfer Function* figure shows the ideal transfer characteristic for straight binary coding.

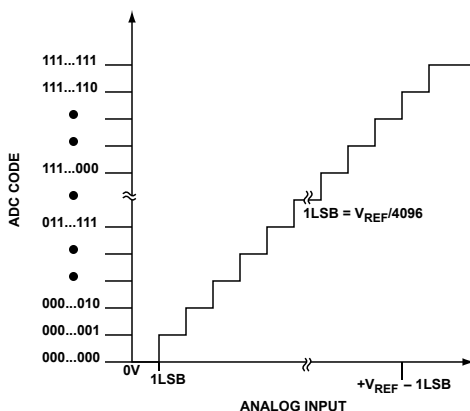


Figure 13-4: ADC Transfer Function

Results

The HADC takes 20 cycles of f_{SAMPLE} for one channel conversion. (The value of the `HADC_CTL.FDIV` bit field determines f_{SAMPLE}). The time taken to complete one sequence depends on the number of channels in the auto-scan chain. There is a latency of 1 cycle from the time the channel is selected internally to sample to the time the data is ready. After the end of each channel conversion, the data is written into the corresponding data register. An interrupt is generated (if the interrupt mask is not enabled) to signal that the data is ready for that particular channel.

HADC Operating Modes

The HADC has two modes of operation described in the following sections.

Fixed Conversion Mode

In this mode, the ADC converts the input channel sequence for a fixed number of times. The frequency is configured in the `HADC_CTL.FIXEDCNV` bit field. To use this mode, clear the `HADC_CTL.CONT` bit.

Continuous Conversion Mode

In this mode ADC continuously converts an input channel sequence, as long as `HADC_CTL.STARTCNV` bit is held high. To use this mode, set the `HADC_CTL.CONT` bit.

HADC Event Control

The HADC generates different events depending on the state of the ADC and the status of channel conversions. It can generate an event for each of the following conditions:

- When ADC is ready for conversion
- At the end of sequence conversion
- At the end of each individual channel conversion

Each of these events can generate an interrupt. To generate an interrupt on any desired event, clear the respective bit in the [HADC_IMSK](#) register.

HADC Programming Model

Following sections provide some guidelines for HADC programming.

Powering Up the HADC

To power-up the HADC, program the following bits in the [HADC_CTL](#) register.

- Deassert the `HADC_CTL.PD` bit (HADC power down)
- Set the `HADC_CTL.NRST` bit (Reset)
- Set the `HADC_CTL.ENLS` bit (Enable level shifters)

After deasserting `HADC_CTL.PD`, the HADC requires a finite wake-up time (t_{WAKEUP}) before it can start converting. The HADC requires only two f_{SAMPLE} clocks from the assertion of the `HADC_CTL.NRST` bit before the module is ready to convert. (`HADC_CTL.PD` is low). Poll the `HADC_STAT.RDY` bit. A 1 on this bit indicates that the HADC is ready to convert data.

Enabling the HADC

Setting the `HADC_CTL.STARTCNV` bit enables the HADC. When this bit is kept high, the HADC can work in either continuous or fixed conversion mode. After the `HADC_CTL.STARTCNV` bit is set =1, the [HADC_CHAN_MSK](#) can still be re-programmed, but the new sequence only comes into effect after the current sequence conversion is complete.

ADSP-2159x HADC Register Descriptions

Housekeeping ADC (HADC) contains the following registers.

Table 13-6: ADSP-2159x HADC Register List

Name	Description
HADC_CHAN_MSK	Channel Mask Register

Table 13-6: ADSP-2159x HADC Register List (Continued)

Name	Description
HADC_CTL	Control Register
HADC_DATA[nn]	Channel Data Registers
HADC_IMSK	Interrupt Mask Register
HADC_STAT	Status Register

Channel Mask Register

The `HADC_CHAN_MSK` register provides bits that mask each channel. The LSB corresponds to channel 0 , the second LSB to channel 1 and so on. If a mask bit is set, the corresponding channel is not converted. By default, channels 0-7 are not masked and the extended channels 8-15 are masked.

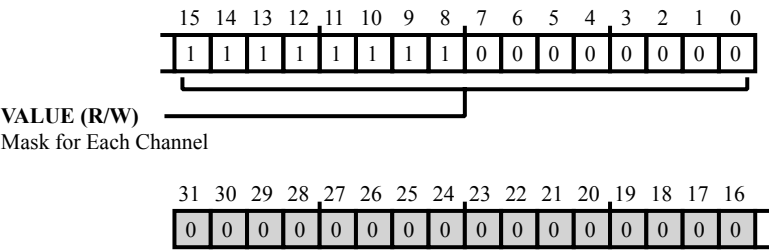


Figure 13-5: HADC_CHAN_MSK Register Diagram

Table 13-7: HADC_CHAN_MSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Mask for Each Channel. The <code>HADC_CHAN_MSK.VALUE</code> bit field is the mask bit for each channel. The first MSB corresponds to channel 15, the second MSB to channel 14 and so on. If the mask is set for a particular channel, that channel is not converted. By default, channels 0-7 are not masked and the extended channels 8-15 are masked. The first MSB corresponds to channel 7, the second MSB to channel 6 and so on. If the mask is set for a particular channel, that channel is not converted. By default, channels 0-7 are not masked.

Control Register

The `HADC_CTL` register contains control bits that configure various module settings start or reset the module.

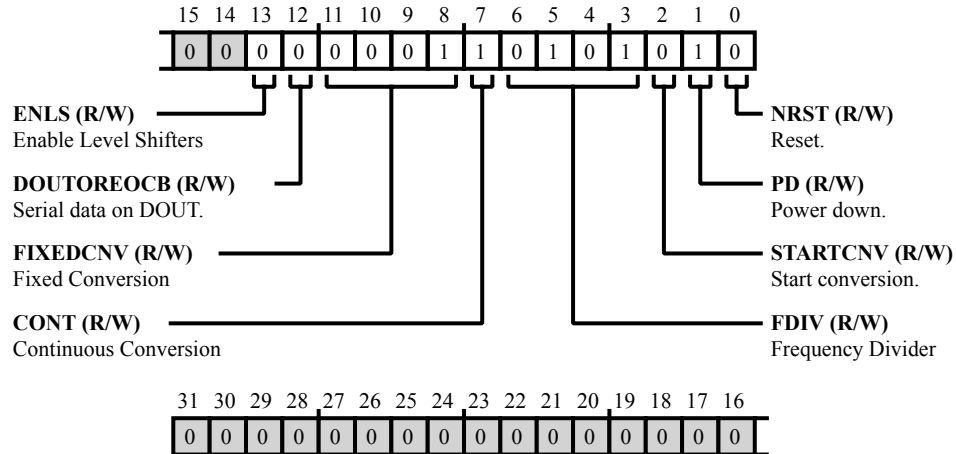


Figure 13-6: HADC_CTL Register Diagram

Table 13-8: HADC_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	ENLS	Enable Level Shifters. Setting the <code>HADC_CTL.ENLS</code> bit enables the level shifters, allowing the HADC analog side (which works in the <code>VDD_EXT</code> domain) to work with the digital core and interface is (in the <code>VDD_INT</code> domain).
		0 Disable down level shifters
		1 Enable down level shifters
12 (R/W)	DOUTOREOCB	Serial data on DOUT.. If the <code>HADC_CTL.DOUTOREOCB</code> bit =1, serial data arrives on the <code>EOC_DOUT</code> pin. If this bit =0 (default) it acts as an EOC only if the external multiplexer is connected.
		0 Reserved
		1 Reserved
11:8 (R/W)	FIXEDCNV	Fixed Conversion. The <code>HADC_CTL.FIXEDCNV</code> bit configures the number of conversions = <code>FIXEDCNV</code> . This value determines how many times a sequence is converted when the HADC is in fixed conversion mode. This only applies when the <code>HADC_CTL.CONT</code> bit =0. Before changing the <code>HADC_CTL.FIXEDCNV</code> bit, clear the <code>HADC_CTL.NRST</code> bit (=0).

Table 13-8: HADC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	CONT	Continuous Conversion. When the HADC_CTL.CONT bit =0, the ADC converts a sequence for a fixed number of times. This number is configured using the HADC_CTL.FIXEDCNV bit field. When the HADC_CTL.CONT bit =1, the ADC continuously converts a given sequence, provided the HADC_CTL.STARTCNV is held high.
		0 ADC converts sequence for fixed number of times
		1 ADC continuously converts given sequence
6:3 (R/W)	FDIV	Frequency Divider. The HADC_CTL.FDIV bit field configures the $f_{\text{SAMPLE}} = f_{\text{CLK}} / (\text{FDIV} + 1)$. Select f_{CLK} and HADC_CTL.FDIV values so that f_{SAMPLE} is in range of 50 kHz to 22.5 MHz. The minimum value for HADC_CTL.FDIV is 1. Before changing the HADC_CTL.FDIV bits, clear the HADC_CTL.NRST bit.
2 (R/W)	STARTCNV	Start conversion.. The HADC_CTL.STARTCNV bit needs to be set for the ADC to start converting data. If the ADC is running in non continuous mode, it is reset by hardware after the desired number of conversions is completed.
		0 No action
		1 Start converting
1 (R/W)	PD	Power down.. The HADC_CTL.PD bit powers down the analog circuitry of the ADC. After this bit returns to 0 a finite power-up time is required before the ADC can start converting data.
		0 No action
		1 Power down the analog circuitry of the ADC
0 (R/W)	NRST	Reset.. The HADC_CTL.NRST bit resets the ADC.
		0 Reset the ADC
		1 No action

Channel Data Registers

The `HADC_DATA[nn]` registers NN ranges from 0-14. Each corresponding to an ADC channel. `ADC_DATA_0` corresponds to channel 0, `ADC_DATA_1` to channel 1 and so on.

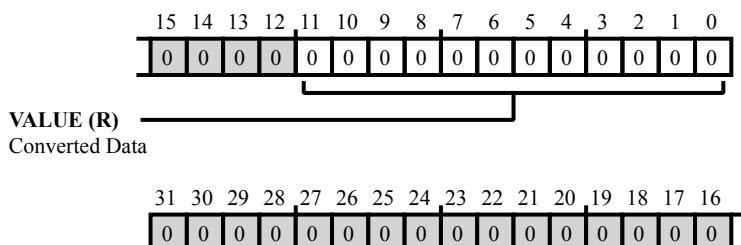


Figure 13-7: `HADC_DATA[nn]` Register Diagram

Table 13-9: `HADC_DATA[nn]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11:0 (R/NW)	VALUE	Converted Data. The <code>HADC_DATA[nn].VALUE</code> bit field contains the digital code for the sampled analog value. Each channel has its own data register.

Interrupt Mask Register

The `HADC_IMSK` register masks (disables) or unmasks (enables) the interrupts as programmed.

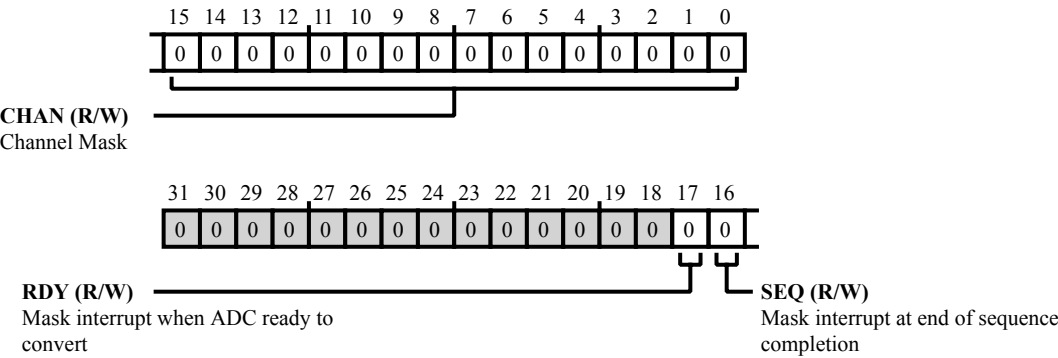


Figure 13-8: HADC_IMSK Register Diagram

Table 13-10: HADC_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	RDY	Mask interrupt when ADC ready to convert. The <code>HADC_IMSK.RDY</code> bit masks the interrupt generated when ADC is ready to convert.
16 (R/W)	SEQ	Mask interrupt at end of sequence completion. The <code>HADC_IMSK.SEQ</code> bit masks the interrupt which is generated at the end of sequence completion.
		0 Interrupt is unmasked
		1 Interrupt is masked
15:0 (R/W)	CHAN	Channel Mask. The <code>HADC_IMSK.CHAN</code> bit field provides the interrupt mask bit for each channel. N ranges from 0-15. The MSB corresponds to channel 15, the second MSB to channel 14 and so on. If the bit is SET, interrupt is masked for corresponding channel.

Status Register

The `HADC_STAT` register contains bits that provide status information on the HADC module.

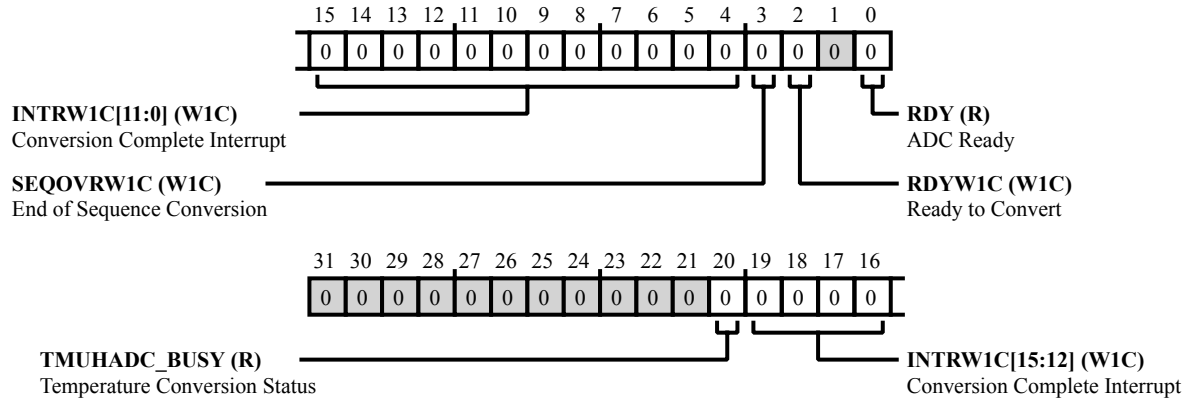


Figure 13-9: HADC_STAT Register Diagram

Table 13-11: HADC_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/NW)	TMUHADC_BUSY	Temperature Conversion Status. The <code>HADC_STAT.TMUHADC_BUSY</code> bit, if high, indicates to the system that temperature conversion is ongoing. All ADC conversion requests are ignored.
19:4 (RX/W1C)	INTRW1C	Conversion Complete Interrupt. The <code>HADC_STAT.INTRW1C</code> bit field indicates when the corresponding ADC channel completes conversion. N ranges from 0-15 and the MSB corresponds to channel 15, the second MSB to channel 14 and so on. N ranges from 0-7 and the MSB corresponds to channel 7, the second MSB to channel 6 and so on. These bits are sticky and are W1C.
3 (RX/W1C)	SEQOVRW1C	End of Sequence Conversion. The <code>HADC_STAT.SEQOVRW1C</code> bit indicates the end of a sequence conversion and is a sticky status bit which is W1C.
2 (RX/W1C)	RDYW1C	Ready to Convert. The <code>HADC_STAT.RDYW1C</code> bit is the stick version of the <code>HADC_STAT.RDY</code> bit.
0 (R/NW)	RDY	ADC Ready. The <code>HADC_STAT.RDY</code> bit is set (=1) when the ADC is ready to convert data.

14 Watchdog Timer (WDOG)

The processor has a 32-bit watchdog timer that can be used to verify system reliability by generating an event to the processor core when the watchdog expires before software updates it.

WDOG Features

The watchdog timer has the following features:

- Programmable 32-bit watchdog count value
- 8-bit disable bit pattern
- General-purpose core event generation

The watchdog timer can supervise system software stability by periodically reloading it to prevent expiration of the downward-counting timer (such that the count never becomes 0). When used in this fashion, an expiring timer can indicate the system software is not running normally.

Expiration of the WDOG counter generates a general-purpose interrupt, which can be used in a variety of ways:

- as an interrupt vector sent through the System Event Controller (SEC) to the core to be serviced by a handler function, providing full software control of device resources (for example, GPIO management and reset control).
- as a fault condition through the SEC to provide hardware-automated:
 - signalling of the fault condition on external pins to the system,
 - system reset requests to the Reset Control Unit (RCU), and/or
 - trigger outputs (SEC0_FAULT trigger master) through the Trigger Routing Unit (TRU) to initiate activities in a variety of potential trigger slaves (for example, GPIO control).

To facilitate debugging, the watchdog timer does not decrement (even if enabled) when the processor is in emulation mode.

WDOG Functional Description

When enabled, the 32-bit watchdog timer counts downward every SCLK0_0 cycle. If it reaches zero, the watchdog expiration event is generated, which can be used in many ways.

To start the watchdog timer:

1. Program the watchdog timeout period (in SCLK0_0 cycles) in the [WDOG_CNT](#) register. With the watchdog disabled, this write also preloads the [WDOG_STAT](#) register.
2. Enable the watchdog timer by writing any value other than 0xAD to the [WDOG_CTL.WDEN](#) field.

Once the watchdog is enabled, writes to the [WDOG_CNT](#) register are ignored. The counter begins decrementing, and the current counter value can be read from the 32-bit [WDOG_STAT](#) register at any time.

To prevent the counter from expiring, software must "kick" the watchdog by writing any value to the [WDOG_STAT](#) register while the current count is non-zero. While the value written is irrelevant and ignored, this action resets the current counter in the [WDOG_STAT](#) register to the programmed [WDOG_CNT](#) value, and decrementing continues. The internal counter continues decrementing until it reaches zero, at which point the expiration event is generated, and the [WDOG_CTL.WDRO](#) rollover bit is set.

Watchdog operation continues in this manner unless disabled by explicitly writing 0xAD to the [WDOG_CTL.WDEN](#) field.

The watchdog expiration event can be used in a variety of ways, as listed below.

- The watchdog expiration event itself is one of numerous interrupt sources that is managed by the System Event Controller. Like other peripheral sources, this event can be used to cause a vector to a handler function that executes based on interrupt priority.
- The watchdog expiration event can be used to initiate automated hardware response through the SEC Fault Interface (SFI).

For this, the WDOG expiry has to be configured as the fault source in the SEC. Then the response to the WDOG expiry can be set to any of the below:

- Signalling through the external fault pin.
- System reset
- Trigger outputs to numerous potential trigger slaves.

For further details on how watchdog expiration event can be used with the SFI, see [Configuring the WDOG Expiry Event to Issue a System Reset](#).

ADSP-2159x WDOG Register List

The Watchdog Timer unit (WDOG) provides a software-based watchdog timer that can improve system reliability by generating an event to the processor core if the watchdog expires before being updated by software. A set of registers governs WDOG operations. For more information on WDOG functionality, see the WDOG register descriptions.

Table 14-1: ADSP-2159x WDOG Register List

Name	Description
WDOG_CNT	Count Register
WDOG_CTL	Control Register
WDOG_STAT	Watchdog Timer Status Register
WDOG_WIN	Watchdog Timer Window Register

ADSP-2159x WDOG Interrupt List

Table 14-2: ADSP-2159x WDOG Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
3	WDOG0_EXP	WDOG0 Expiration	Level	
4	WDOG1_EXP	WDOG1 Expiration	Level	
5	WDOG2_EXP	WDOG2 Expiration	Level	

WDOG Block Diagram

The *Watchdog Timer Block Diagram* figure shows the detailed block diagram for the watchdog timer.

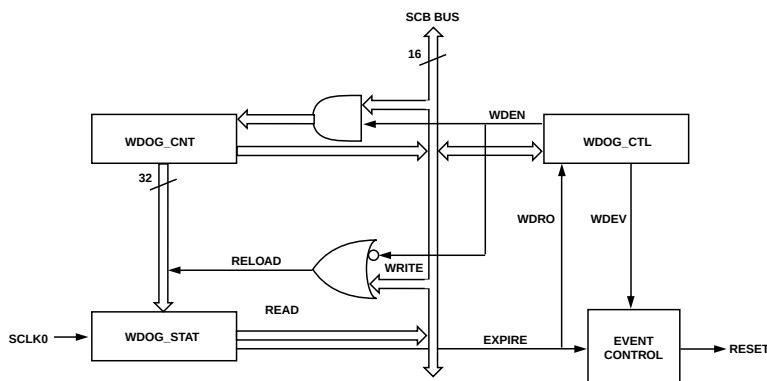


Figure 14-1: Watchdog Timer Block Diagram

Internal Interface

The system clock (SCLK0) clocks the watchdog timer. The registers are accessed through the 16-bit peripheral MMR access bus. 32-bit read/write operations always access the 32-bit [WDOG_CNT](#) and [WDOG_STAT](#) registers. Hardware ensures that those accesses are atomic. When the counter expires, the WDOG expiration event is generated.

External Interface

The watchdog timer does not directly interact with any external pins.

ADSP-2159x WDOG Register Descriptions

Watchdog Timer Unit (WDOG) contains the following registers.

Table 14-3: ADSP-2159x WDOG Register List

Name	Description
WDOG_CNT	Count Register
WDOG_CTL	Control Register
WDOG_STAT	Watchdog Timer Status Register
WDOG_WIN	Watchdog Timer Window Register

Count Register

The `WDOG_CNT` register holds the programmable, unsigned count value. A valid write to this register also pre-loads the WDOG counter. For added safety, the `WDOG_CNT` register can be updated only when the WDOG timer is disabled. A write to the `WDOG_CNT` register while the timer is enabled does not modify the contents of this register. This register must be accessed with 32-bit read/writes only.

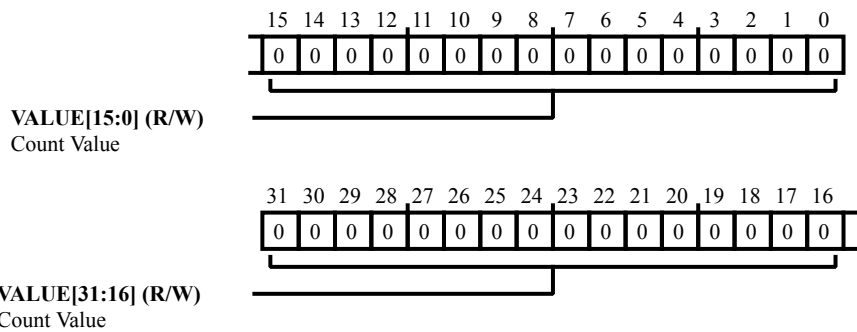


Figure 14-2: WDOG_CNT Register Diagram

Table 14-4: WDOG_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Count Value. The <code>WDOG_CNT.VALUE</code> bit field holds the programmable, unsigned count value.

Control Register

The `WDOG_CTL` register controls the watchdog timer. This register supports enabling/disabling the watchdog timer and supports checking the timer rollover status. Note that when the processor is in emulation mode, the watchdog timer counter will not decrement even if it is enabled.

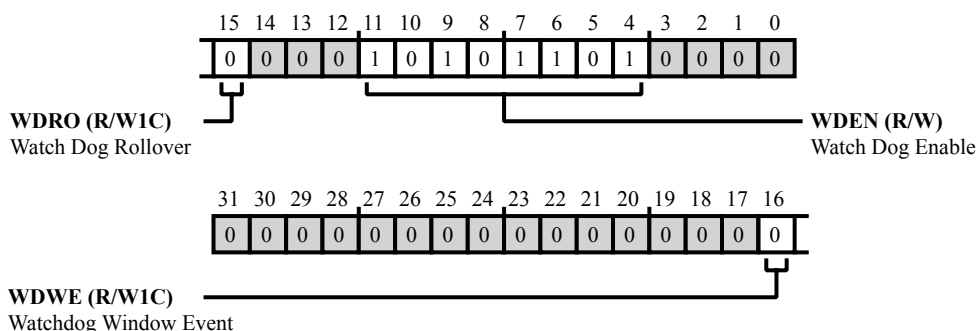


Figure 14-3: WDOG_CTL Register Diagram

Table 14-5: WDOG_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W1C)	WDWE	Watchdog Window Event. Software can determine whether the timer has been serviced unexpectedly early by interrogating the <code>WDOG_CTL.WDWE</code> status bit. This is a sticky bit that is set whenever the watchdog is serviced and the <code>WDOG_STAT</code> value is greater than the <code>WDOG_WIN</code> value. It is cleared only by disabling the watchdog timer and by writing a 1 to the bit.
		0 Window Event has not occurred
		1 Window Event has occurred
15 (R/W1C)	WDRO	Watch Dog Rollover. Software can determine whether the timer has rolled over by interrogating the <code>WDOG_CTL.WDRO</code> status bit. This is a sticky bit that is set whenever the watch dog timer count reaches 0 and cleared only by disabling the watch dog timer and then writing a 1 to the bit.
		0 WDT Has Not Expired
		1 WDT Has Expired
11:4 (R/W)	WDEN	Watch Dog Enable. The <code>WDOG_CTL.WDEN</code> field is used to enable and disable the watchdog timer. Writing any value other than the disable value into this field enables the watchdog timer. This multi-bit disable key minimizes the chance of inadvertently disabling the watchdog timer.
		173 Counter Disabled. All other values mean that the counter is enabled.

Watchdog Timer Status Register

The `WDOG_STAT` register contains the current count value of the watchdog timer. Reads of this register return the current count value. When the watchdog timer is enabled, the `WDOG_STAT` register is decremented by 1 on each `SCLK0` cycle. When the count value reaches 0, the watchdog timer stops counting, and the expiry event is generated. The `WDOG_STAT` register is a 32-bit unsigned system MMR that must be accessed with 32-bit reads and writes.

Values cannot be stored directly in this register but are instead copied from the `WDOG_CNT` register. This copy process can happen in two ways:

- While the watchdog timer is disabled, writing the `WDOG_CNT` register pre-loads the `WDOG_STAT` register.
- While the watchdog timer is enabled, writing the `WDOG_STAT` register loads it with the value in the `WDOG_CNT` register.
- While the watchdog timer is disabled, writing to the `WDOG_STAT` register also reloads it with the value in the `WDOG_CNT` register.

When the processor executes a write (of an arbitrary value) to the `WDOG_STAT` register, the value in the `WDOG_CNT` register is copied into the `WDOG_STAT` register. Typically, software sets the value of `WDOG_CNT` at initialization, then periodically writes to the `WDOG_STAT` register before the watchdog timer expires. This reloads the watchdog timer with the value from `WDOG_CNT` and prevents generation of the expiry event.

If the program does not reload the counter before `SCLK0` x count register cycles, an expiry event is generated, and the `WDOG_CTL.WDRO` bit is set. When this happens, the counter stops decrementing and remains at zero. If the counter is enabled with a zero loaded to the counter, the `WDOG_CTL.WDRO` bit is set immediately and the counter remains at zero and does not decrement.

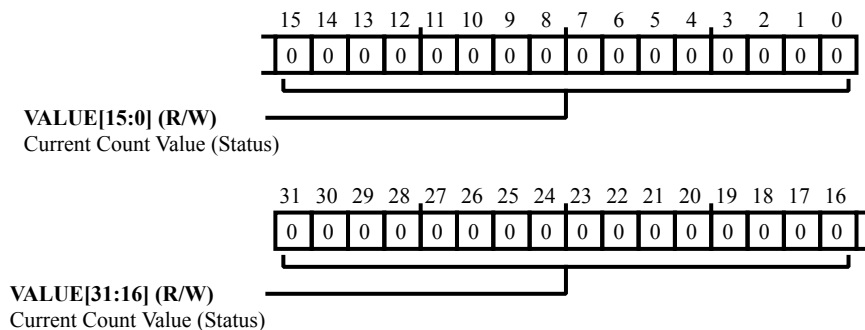


Figure 14-4: `WDOG_STAT` Register Diagram

Table 14-6: WDOG_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Current Count Value (Status). The <code>WDOG_STAT.VALUE</code> bit field contains the current count value of the watchdog timer.

Watchdog Timer Window Register

Watchdog window register holds the unsigned window value programmed. The window register can be programmed while the WDOG is disabled; any write to the register when WDOG is enabled doesn't alter the contents of the register. When the WDOG is enabled and the core services the WDOG by doing a write to `WDOG_STAT` register while the `WDOG_STAT` value is greater than `WDOG_WIN`, the WDWE event is generated and also the counter stops decrementing.

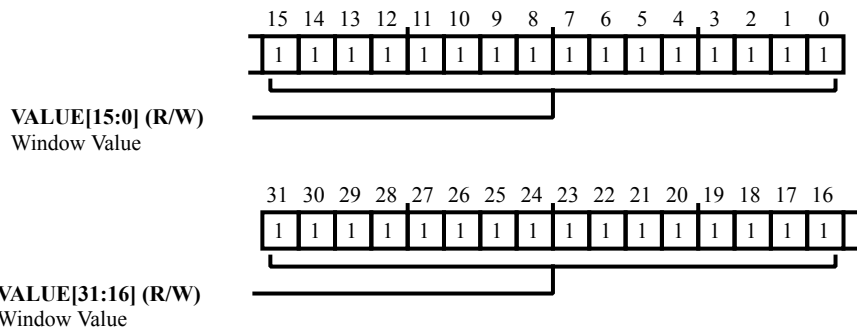


Figure 14-5: WDOG_WIN Register Diagram

Table 14-7: WDOG_WIN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Window Value. The WDOG_WIN.VALUE bit field contains the unsigned window value.

15 Link Port (LP)

Link ports allow the processor to connect to other processors or peripheral link ports using a simple communication protocol for high-speed parallel data transfer. This peripheral allows various I/O peripheral interconnection schemes to I/O peripheral devices as well as co-processing and multiprocessing schemes.

The link ports of the processor support 8-bit wide data transfers. The link port pins are multiplexed in the GPIO ports. For information on processor multiplexing, see the data sheet for the specific processor.

Link ports can operate independently and simultaneously, allowing glueless high-speed connectivity of up to four external processors.

NOTE: Reference to SCLK in this chapter can be considered as OCLK0_0.

LP Features

All link ports are identical in their design and have the following common features.

- Bidirectional ports with eight data signals (LP_D0 - LP_D7), an acknowledge signal (LP_ACK), and a clock signal (LP_CLK).
- Provide high-speed, point-to-point data transfers to other processors, allowing different types of interconnections between multiple processors.
- Pack data into 32-bit words. The processor can directly read this data or transfer it through DMA to or from on-chip memory.
- Support for data buffering through a 2-deep FIFO for transmit and a 4-deep FIFO for receive.
- Programmable clock and acknowledge based handshake mechanism for efficient communication.
- A dedicated DMA channel.
- Support for data transfer in 2-bit mode (non-DDR only) and 4-bit mode (DDR/non-DDR)

LP Functional Description

This section provides a description of the link port, including a list of its registers and a functional block diagram.

ADSP-2159x LP Register List

The Link Port LP is an 8-bit wide parallel port that can connect to another processor's LP or another LP-compatible device. This port allows a variety of interconnection schemes to I/O peripheral devices as well as co-processing and multiprocessing schemes. A set of registers governs LP operations. For more information on LP functionality, see the LP register descriptions.

Table 15-1: ADSP-2159x LP Register List

Name	Description
LP_CTL	Control Register
LP_DIV	Clock Divider Value Register
LP_RX	Receive Buffer Register
LP_STAT	Status Register
LP_TX	Transmit Buffer Register
LP_TXIN_SHDW	Shadow Input Transmit Buffer Register
LP_TXOUT_SHDW	Shadow Output Transmit Buffer Register

ADSP-2159x LP Interrupt List

Table 15-2: ADSP-2159x LP Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
117	LP0_DMA	LP0 DMA Channel		30
118	LP0_STAT	LP0 Status		
119	LP1_DMA	LP1 DMA Channel		36
120	LP1_STAT	LP1 Status		
277	LP0_DMA_ERR	LP0 DMA Data Error		
278	LP1_DMA_ERR	LP1 DMA Data Error		

ADSP-2159x LP Trigger List

Table 15-3: ADSP-2159x LP Trigger List Masters

Trigger ID	Name	Description	Sensitivity
48	LP0_DMA	LP0 DMA Channel	
49	LP1_DMA	LP1 DMA Channel	

Table 15-4: ADSP-2159x LP Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
31	LP0_DMA	LP0 DMA Channel	Pulse
32	LP1_DMA	LP1 DMA Channel	Pulse

ADSP-2159x LP DMA Channel List

Table 15-5: ADSP-2159x LP DMA Channel List

DMA ID	DMA Channel Name	Description
DMA30	LP0_DMA	LP0 DMA Channel
DMA36	LP1_DMA	LP1 DMA Channel

Block Diagram

The *Link Port Block Diagram* shows the block diagram of a link port.

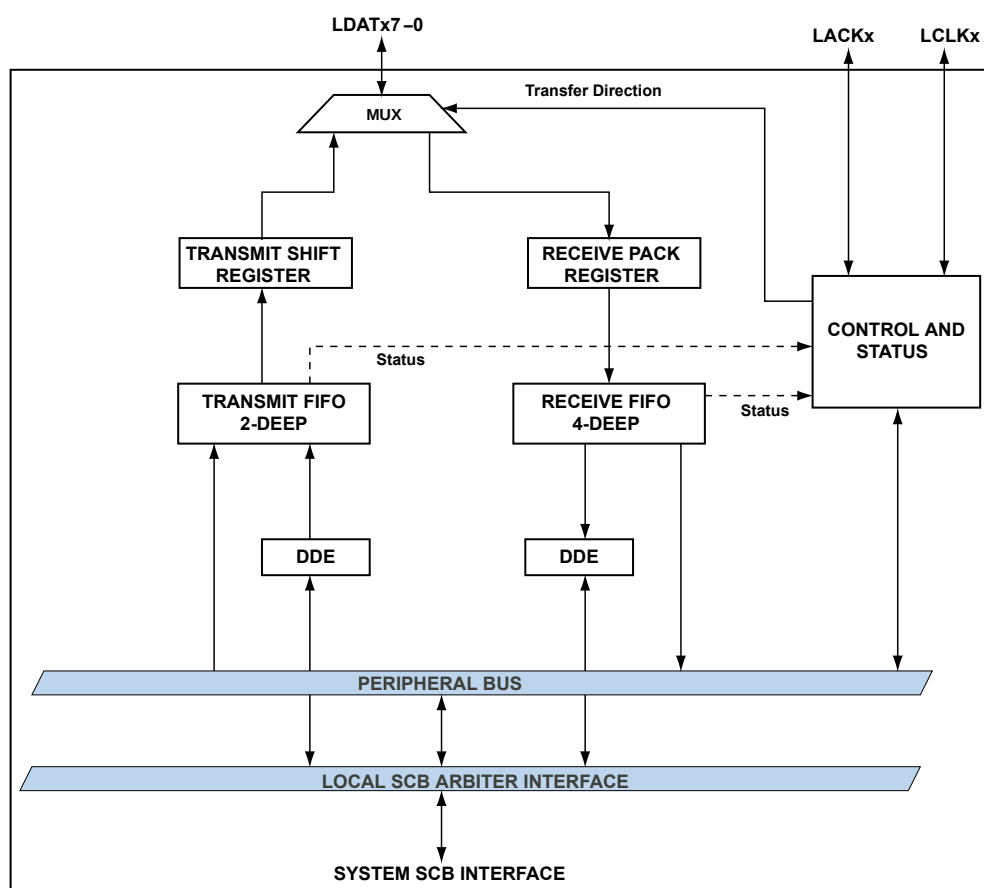


Figure 15-1: Link Port Block Diagram

External Connections

As shown in the *Link Port Pin Connections* figure, a link port has eight data lines (LP_D0 – LP_D7), a clock line (LP_CLK), and an acknowledge line (LP_ACK). A link port can act as either a transmitter or a receiver but not both at the same time.

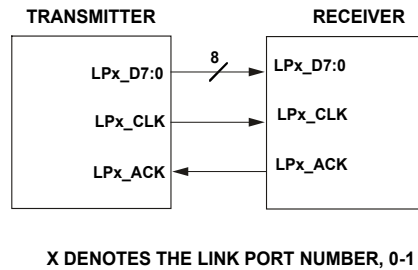


Figure 15-2: Link Port Pin Connections

Use external pull-downs for the LP_CLK and LP_ACK pins so that the link port can enable the transmitter and receiver, irrespective of the state of the other.

Internal Blocks

As shown in the block diagram, the link ports have independent modules for transmit and receive. If enabled as transmitter, the link port uses a 2 deep 32-bit FIFO. If enabled as a receiver, the port uses a 4 deep 32-bit FIFO. The core MMR access bus interfaces with these FIFOs. The distributed DMA engines (DDE) use the system cross bar (SCB) interface to access the FIFO. The link port uses the LP_CTL.TRAN bit to determine whether the module is enabled for transmit or receive operation.

Architectural Concepts

The following sections describe the architectural concepts of the link port.

- [Link Port Protocol](#)
- [FIFO Buffers](#)
- [Handshake for Link Port Enable Process](#)
- [Clocking](#)
- [Multi-Processor Connectivity](#)

Link Port Protocol

A link port transmitted word consists of 4 bytes and the communication proceeds as follows.

1. The transmitter asserts the link port clock (LP_CLK) with each byte of data. The receiver uses the falling edge of LP_CLK driven by the transmitter to latch the byte.
2. When the receiver is ready to accept another word in the receive buffer it asserts the acknowledge signal, LP_ACK.

3. The transmitter samples LP_ACK driven by the receiver at the beginning of each word transmission. If LP_ACK is deasserted, then the transmitter does not transmit the next word.
4. The transmitter leaves LP_CLK high and continues to drive the first byte of the next word until LP_ACK is asserted.
5. When this assertion occurs, the transmitter drives LP_CLK low. The transmission of the next word starts. If the transmit buffer is empty, LP_CLK remains low until the buffer refills, regardless of the state of LP_ACK.

The LP_ACK signal can deassert when it anticipates that the buffer could fill. The receiver reasserts the LP_ACK signal as soon as the internal DMA grant signal has occurred or the core reads the receive buffer. Either of these actions frees a buffer location.

NOTE: The LP_ACK signal inhibits transmission of the next word and not of the current byte.

The LP_ACK signal provides a handshake between the receiver and transmitter in the following configurations.

- When configured as a transmitter, the port drives both the data and the clock while LP_ACK is three-stated. In this mode, LP_CLK is always synchronous with OCLK0_0.
- When configured as a receiver, the link port drives the acknowledge signal and the data and clock lines are three-stated. In this case, the external LP_CLK signal can either be synchronous or asynchronous with OCLK0_0.
- When the link port is disabled, the data, clock, and acknowledge signals are three-stated.

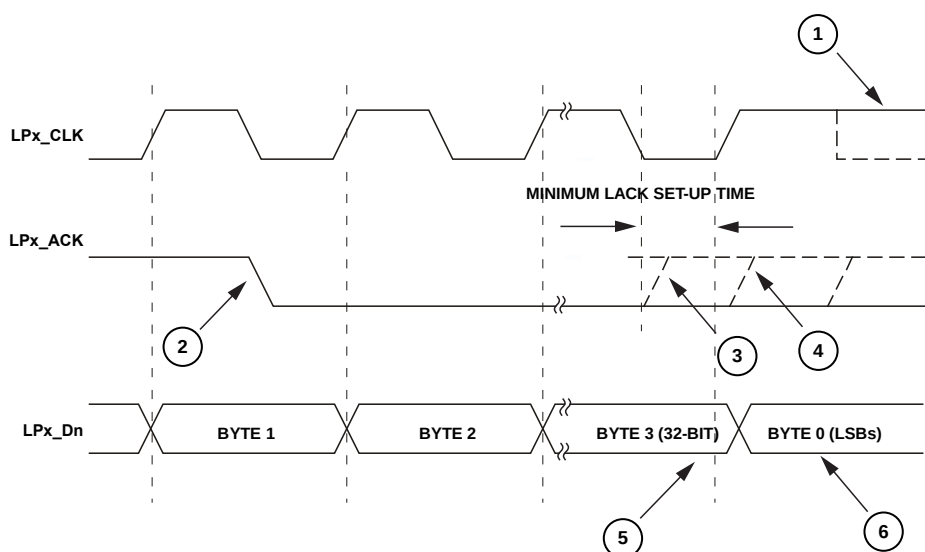


Figure 15-3: Link Port Communication and Handshake Waveform

The following list describes the stages shown in the *Link Port Communication and Handshake Waveform* figure.

1. LP_CLK stays high at byte 0 when LP_ACK is sampled low on the previous LP_CLK rising edge. LP_CLK high indicates a stall.
2. The LP_ACK signal can deassert after byte 0.

3. The LP_ACK signal reasserts as soon as the link buffer is not full (depending on Rx FIFO conditions).
4. The transmitter samples LP_ACK to determine whether to transmit the next word.
5. The receiver accepts the remaining word even if LP_ACK is deasserted. The transmitter does not send the following word.
6. Transmission of data for next word is held until LP_ACK is asserted.

The transmitter samples the LP_ACK signal. If the signal is high, the transmitter gives out the falling edges of LP_CLK for data sampling. The LP_ACK signal is first sampled at the rising edge of OCLK0_0. One more OCLK0_0 stage synchronizes the signal further. This synchronized signal is given to the subsequent logic. The LP_CLK falling edge is aligned with OCLK0_0 falling edge in a 1:1 clock ratio mode and with the OCLK0_0 rising edge for the rest of the clock ratios. The following figures explain how the synchronization is maintained between the LP_ACK and LP_CLK signals.

In the following figure, synchronizing time is guaranteed to be 1.5 OCLK0_0 cycles.

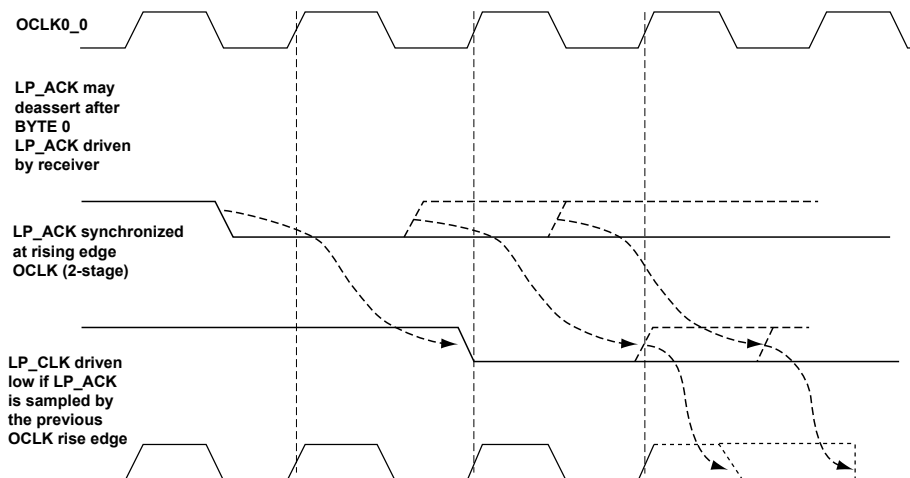


Figure 15-4: LP_ACK Synchronization for OCLK0_0:LP_CLK=1:1

In the following figure, synchronizing time is guaranteed to be 2 OCLK0_0 cycles.

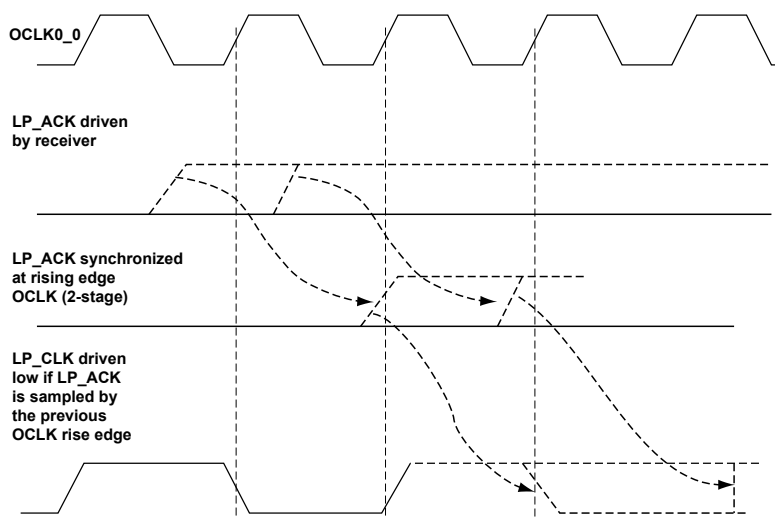


Figure 15-5: LP_ACK Synchronization for OCLK0_0: LP_CLK=1:2, 1:4 and Up

The link port uses the value programmed in the `LP_DIV` register at the transmitter to determine the frequency of the link port clock (LP_CLK). However, the signal appearing on the LP_CLK pin is also dependent on the status of the LP_ACK pin driven by the receiver. The *Relationship Between Internal Link Port Clock and Link Port Clock at the Pins* figure shows this relationship.

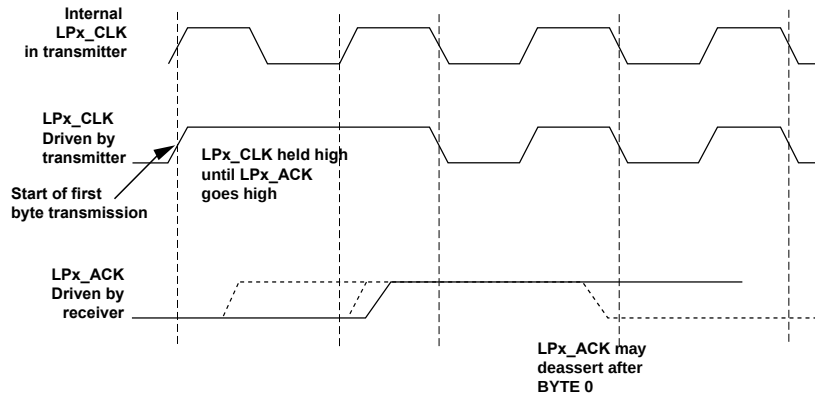


Figure 15-6: Relationship Between Internal Link Port Clock and Link Port Clock at the Pins

FIFO Buffers

When a link port is configured as transmitter, the link port uses a 2-deep FIFO buffer. A shift register unpacks the single 32-bit word to four 8-bit data bytes. As the FIFO has space for more data, the link port makes a new DMA request. If the FIFO becomes empty, the LP_CLK signal is deasserted. The core can access FIFO through the `LP_TX` register.

The core or DMA makes three writes (2-stage FIFO and 1 shift register) to the transmit buffer before it signals a full condition. The link port uses the `LP_STAT.FFST` bit field to reflect the status of the FIFO but not the shift register full or empty condition. However, the program can poll the `LP_STAT.LPBS` bit to discover whether the link

port is driving data from the shift register to the pins. The `LP_STAT.LPBS` bit is also set when receiver has held off transmission by driving `LP_ACK` low.

NOTE: When the 2-deep FIFO and the output shift-register overflow, any further write to the link port buffer overwrites the input stage of the FIFO.

NOTE: The core can also read the transmit FIFO through the data `LP_TX` register.

NOTE: If the transmitter is disabled while performing writes to the transmit FIFO, a FIFO full condition is signaled after two writes.

The transmit buffer registers have shadow registers. Using these shadow registers, both stages of the 2-deep FIFO can be read without updating the status registers. The `LP_TXIN_SHDW` register corresponds to the input stage of the FIFO. The `LP_TXOUT_SHDW` register corresponds to the output stage of the FIFO as shown in the *Transmit FIFO path* figure.

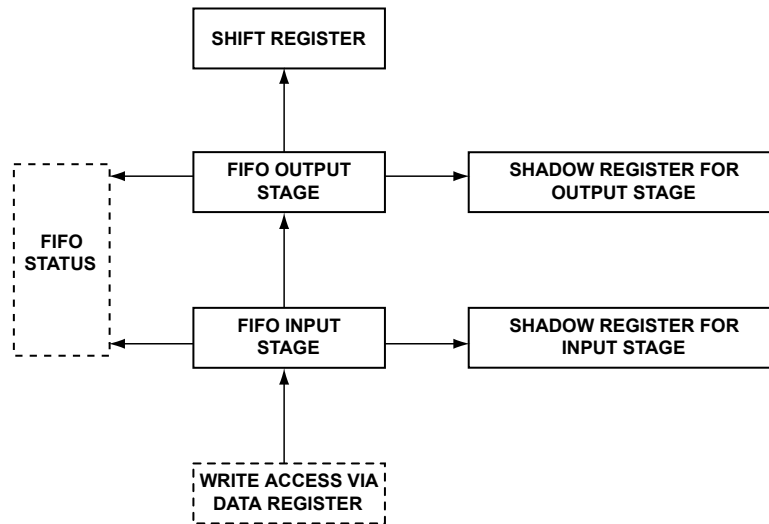


Figure 15-7: Transmit FIFO path

When a link port is configured as receiver, data transfers to the core or DMA from the full 4-deep receive FIFO. An internal packing register packs data to 32 bits. Four reads can occur from the receive buffer by the core or DMA before it signals an empty condition. The link port uses the `LP_STAT.FFST` bits to reflect the status of the 4-deep read buffer FIFO. The core can access this FIFO through the `LP_RX` register.

NOTE: When receive FIFO overflows (`LP_STAT.ROVF` bit=1), any further data from the transmitter is lost. Only the data retained in the receive FIFO can be retrieved further.

The receiver drives the `LP_ACK` output signal low, after the first byte of data for the next-to-last empty slot (in the 4-deep FIFO) is received. This functionality prevents data loss due to the transmitter starting transmission of the next word before the `LP_ACK` signal reaches the transmitter. (The timing is due to the larger delay in synchronization.) This functionality guarantees that even after allowing for the extra synchronization cycle in the transmitter and receiver, there is no overflow in the receive FIFO. The *LACK Generation Based on Receive FIFO Status* figure

shows how FIFO slots influence the acknowledge signal generation. The grayed sections show received data. The white sections show empty locations where the decision to pull LP_ACK high is taken.

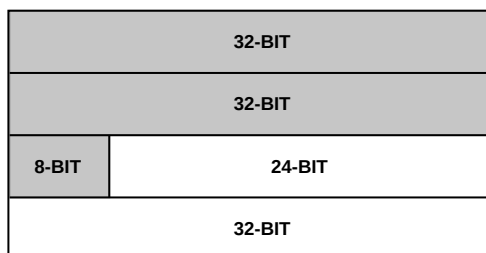


Figure 15-8: LACK Generation Based on Receive FIFO Status

NOTE: The link port uses a 4-deep receive FIFO only under a worst case situation, as mentioned. In all other cases, respond as if the FIFO has only a 3-deep stage. The LP_ACK signal is pulled high before the last stage of the FIFO.

The link port has memory-mapped buffers for both receive and transmit operations. A JTAG-based emulator can read the FIFO which can cause unexpected problems in data transfers. This activity can only happen during an emulation event (typically hitting a breakpoint or single-stepping). The emulator issues core reads through JTAG. To work around this issue, see the tools documentation for more information.

Handshake for Link Port Enable Process

In a link port-based system, the transmitter and the receiver can be enabled at different times. Use external pull-downs for the LP_CLK and LP_ACK signals.

If the receiver is enabled before the transmitter, the external pull-down holds the LP_CLK signal of the transmitter low. The receiver is held off. The receiver can wait for a rising edge on the LP_CLK signal to assert its receive service request interrupt. This rising edge occurs only when transmitter starts driving the first data on to the bus, after the application enables it.

If the transmitter is enabled before the receiver, the external pull-down holds the LP_ACK signal of the receiver low. Transmission is held off. Refer to the *Enable the Transmitter Before the Receiver* figure. The transmitter can wait for a rising edge on the LP_ACK signal to assert its transmit service request interrupt. This rising edge is asserted as soon as the receiver is enabled after the hardware drives the LP_ACK high.

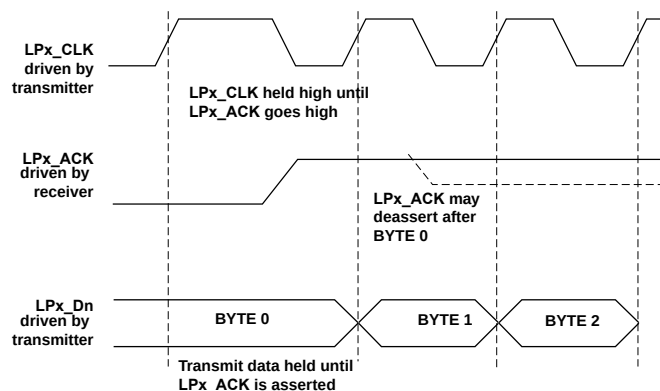


Figure 15-9: Enable the Transmitter Before the Receiver

NOTE: Service request interrupts or status are asserted only when the link port (receiver or transmitter) is disabled.

Clocking

The link port clock (LP_CLK) is derived from the internal system clock (OCLK0_0). The link port clock to system clock ratio can be configured in the [LP_DIV](#) register. This value applies to the transmitter only. The receiver can operate at any asynchronous frequency up to the maximum frequency, independent of the ratio programmed. The following formula describes the relationship between the frequency of the link port clock, the OCLK0_0 frequency, and the [LP_DIV](#) value.

- $f_{LP_CLK} = f_{OCLK0_0}$ or $f_{LP_CLK_MAX}$ if $DIV = 0$
- $f_{LP_CLK} = f_{OCLK0_0} / (2 \times DIV)$ if $DIV > 0$

Where: f_{LP_CLK} = link clock frequency, $f_{LP_CLK_MAX}$ = link clock maximum frequency, and f_{OCLK0_0} = system clock frequency.

While programming the [LP_DIV](#) register to select the clock ratio, ensure that the LP_CLK frequency does not exceed the maximum frequency supported for the device. For supported frequencies, see the product-specific data sheet.

NOTE: The Drive Strength (DS) value must be changed to b(010) for the LPCLK pin to operate the link port at the maximum frequency of 125 MHz. Refer the [PADS_PORTB0_DS](#) and [PADS_PORTC0_DS](#) register bit descriptions for details.

Multi-Processor Connectivity

Link ports can operate independently, allowing glueless connection with external processors. Link ports have dedicated DMA channels, allowing independent data transfers. The following group of figures shows some examples of different bus connection topology that can be used in multi-processor system design. The inter-connection methods are not limited to these examples.

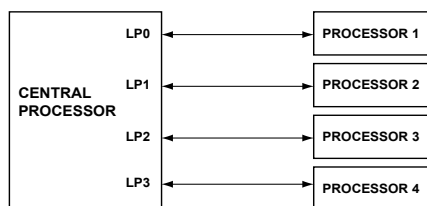


Figure 15-10: Central Processor-Based Model

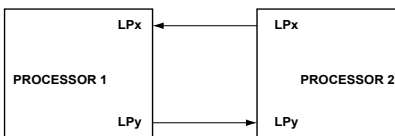


Figure 15-11: Link Port Full-Duplex Transfer Model

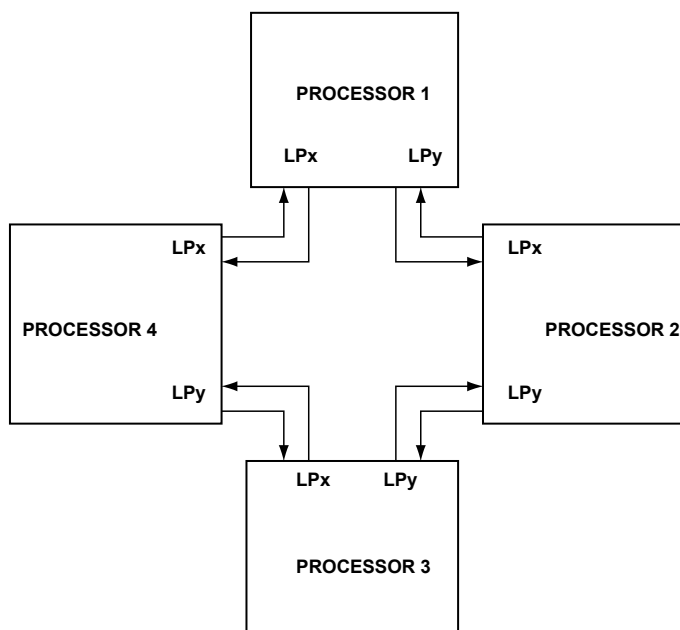


Figure 15-12: Link Port Ring Model

The link port protocol does not include built-in support for multiple masters. However, there can be situations where multiple devices try to become the bus master at the same time. Multi-master conflicts can be resolved using token passing. In token passing, the token is a software flag that passes between processors.

At reset, the token is set to reside in the link port of one device, making it the master and the transmitter. When a receiver (slave) wants to become the master, it can assert its `LP_ACK` signal to get the attention of the master. The master knows, through the software protocol, whether to respond with actual data or whether the token is requested. If the master wishes to give up the token, it can send back a user-defined token release word and thereafter clear its token flag.

Simultaneously, the slave sets its token and can thereafter transmit. The token release word can be any user-defined value. Because the transmitter and receiver expect a code word, this word does not need to be exclusive of normal

data transmission. If the master wishes to give up the token, it can send back a user-defined token release word and thereafter clear its token flag. Simultaneously, the slave examines the data sent back and if it is the token release word, the slave sets its token, and can thereafter transmit.

The link port protocol includes handshake mechanism to inform the other end of transfer (transmit or receive) of an enable instance. However, it does not support handshakes to inform a disable instance, while a chunk of data transfers. The application must assume the disabled state of the other end, and take appropriate action.

For example, in a multi-processing environment, a receiver did not read its full FIFO for an extended time due to internal bus arbitrations. The transmitter can require software or a peripheral timer-based timeout to inform the application that the `LP_ACK` signal is low for an extended time period.

LP Operating Modes

The link port does not have particular modes of operation, as the peripheral is based on a simple protocol. The following sections explain the data transfer modes, using the core and using DMA.

- [Core Data Transfers](#)
- [DMA Data Transfers](#)

LP Data Transfer Modes

This section describes link port DMA and core data transfers.

Core Data Transfers

If DMA is disabled for a link port buffer, the processor core can write or read internal FIFO buffers as a memory-mapped register through the MMR access bus. In order to avoid FIFO overflow or underflow, the core can access the FIFO registers in one of the two following ways.

1. Access link port registers using an interrupt service routine (ISR) mapped to the data request interrupt of the link port. The interrupt request remains high only if the FIFO is accessible (if the FIFO is not full in transmit mode and not empty in receive mode).
2. Poll the FIFO status bits of the `LP_STAT` register. Write to the transmit FIFO if not full or read from the receive FIFO if not empty.

DMA Data Transfers

Dedicated DMA channels are available for each link port. DMA-related activity is explained in the following steps.

1. Data Receive – Once the DMA channel and link port module are configured and enabled, the external device begins writing data to the FIFO through the data pins of the link port. The FIFO detects this activity and in turn sends a DMA request. After the request is granted, the DMA transfer progresses until the FIFO is empty.
2. Data Transmit – Once the DMA channel and link port module are configured and enabled, setting the `LP_CTL.EN` bit automatically asserts a DMA request when the transmit FIFO is empty. After the request is granted, DMA fills the FIFO. The external device begins reading data from the FIFO through the data pins of

the link port. The FIFO detects that there is room in the buffer and asserts another DMA request, continuing the process.

LP Event Control

This section describes how the link port uses interrupts and status signals.

Interrupt Signals

Each link port has two dedicated interrupt lines registered with the system event controller—a data request interrupt and a status interrupt. Data request interrupts are asserted based on FIFO conditions for data transfer. Status interrupts are asserted when a service request status or an overflow status is set. The following list explains each of these interrupts.

- **Data Request Interrupt.** Asserted if the FIFO is not full in transmission mode and the FIFO is not empty in reception mode. This functionality serves as a core triggered interrupt in non-DMA mode and as the DMA interrupt request in DMA mode. Generation of this interrupt is based on the `LP_STAT.FFST` (status bit of the link port buffer).
- **Link Port Transmit Service Request Interrupt (LTRQ).** Allow a disabled link port to generate an interrupt when an external access is attempted. When a link port is configured as transmitter, the transmit service request interrupt is enabled by setting the `LP_CTL.TRQMSK` bit. When set, an external receiver can indicate to the disabled transmitter that it must receive data through the connected link port. The receiver does so by driving a high level on the `LP_ACK` line. When the `LP_ACK` of the disabled transmitter link port is detected high, a `LP_STAT.LTRQ` interrupt is generated. The transmitter can enable itself for data transfer with the receiver. The link port needs a pull-down on `LP_ACK` for this feature to function properly.
- **Link Port Receive Service Request Interrupt (LRRQ).** When a link port is configured as receiver, this interrupt is enabled by setting the `LP_CTL.RRQMSK` bit. When set, an external transmitter can indicate to the disabled receiver that it must receive data through the connected link port. The transmitter does so by driving out the first data. When the `LP_CLK` of the disabled receiver link port is detected high, a `LP_STAT.LRRQ` interrupt is generated. The receiver can further enable itself for data transfer with the transmitter. The link port needs a pull-down on the `LP_CLK` signal for this feature to function properly.
- **Link Port Receive Overflow Interrupt (LPOVF).** Generated when the receiver FIFO overflows and is enabled by setting the `LP_CTL.ROVFMSK` bit. This interrupt can happen if the transmitter continues to transmit data even though the receiver has deasserted `LP_ACK` signal causing the receive FIFO to overflow.

Enabling Link Port Interrupts

A data request interrupt is fed to the system event controller directly and can be controlled separately from the application.

To mask the interrupt, set the mask bits in `LP_CTL` register corresponding to service interrupts and the overflow interrupt. These interrupts are OR'ed and fed to the SIC as a single `LP_STAT` interrupt. These interrupts are latched and stored in the associated bits of `LP_STAT` register. If an `LP_STAT` interrupt occurs, in the ISR, programs

can read the [LP_STAT](#) register bits to determine the type of interrupt. These bits are write-one-to-clear (W1C); writing one to the bit resets the bit and disables the corresponding interrupt.

Status and Error Signals

This section explains the various status signals in the [LP_STAT](#) register.

- **Transfer Status signals.** The link port uses the bus status bit ([LP_STAT.LPBS](#)) to give the status of the bus condition (busy or idle), when the link port is configured as transmitter. The [LP_STAT.LPBS](#) is high if the link port drives data into the link port pins. Programs can poll this bit after polling the [LP_STAT.FFST](#) bit to disable the link port safely.

The link buffer status ([LP_STAT.FFST](#)) field directly indicates the status of the FIFO (including empty or full conditions) during data transfer. Software can poll this field in the [LP_STAT](#) register before writing to the FIFO (in case of transmission) or reading from the FIFO (in case of reception). The [LP_STAT.FFST](#) bit is automatically cleared when the link port is disabled.

- **Transfer Request Status signals.** The link port uses the receive request status ([LP_STAT.LRRQ](#)) bit to indicate that an external receiver wants to receive data (in case the link port is a disabled transmitter). The link port uses the transmit request status ([LP_STAT.LTRQ](#)) bit to indicate that an external transmitter wants to send data (in case the link port is a disabled receiver). Software can poll these bits to enable the transmitter or receiver accordingly.
- **Error Status signals.** In receive mode 32-bit data is received in four chunks of 8-bit data. This data is then packed to a single 32-bit data before loading the FIFO. The link buffer error status ([LP_STAT.LPACK](#)) bit is high during this packing process and goes low after packing.

The link port overflow status ([LP_STAT.ROVF](#)) bit is set when the receive FIFO overflows. This event can occur if the transmitter continues to transmit data even though the receiver has deasserted [LP_ACK](#) causing the receiver FIFO to overflow.

LP Programming Model

The following sections provide information on configuring the operating mode and enabling the link ports.

- [Setting Up a DMA Transmit Operation](#)
- [Setting Up a DMA Receive Operation](#)
- [Setting Up a Core Transmit Operation](#)
- [Setting Up a Core Receive Operation](#)

Setting Up a DMA Transmit Operation

This following procedure describes the typical steps for configuring the link ports in DMA transmit mode.

1. Enable the link port pins in the GPIO port mux using the appropriate [PORT_FER](#) and [PORT_MUX](#) registers.

2. Configure the drive strength value to b(010) for the link port clock pins to operate the link port at maximum frequency (`PADS_PORTB0_DS` and `PADS_PORTC0_DS` registers).
3. Install interrupt handlers for DMA and for transfer status (service request interrupt).
4. Configure the link port to transmit by setting the `LP_CTL` bit and enable the transmit request interrupt mask by setting the `LP_CTL.TRQMSK` bit.
5. Program the link port clock divider by writing a value to the `LP_DIV` register.
6. If using DMA stop mode or auto buffer mode, program the appropriate DMA registers.
ADDITIONAL INFORMATION: An example configuration is: `DMA_ADDRSTART`, `DMA_XCNT`, `DMA_XMOD`, and `DMA_CFG` registers (Stop/Auto, `DMA_CFG.PSIZE` =1, `DMA_CFG.MSIZE` =4, interrupt generation and memory read).
7. Wait for the link port receiver (connected externally) to be enabled. The application can wait for the transmit service request interrupt to assert.
8. Clear the transmit service request interrupt status by writing 1 to the `LP_STAT.LTRQ` bit.
9. Enable DMA by setting the `DMA_CFG.EN` bit.
10. Enable the link port by setting the `LP_CTL.EN` bit.
11. Wait for DMA to assert a transfer completion interrupt.
12. Clear the DMA interrupt source by writing 1 to the `DMA_STAT.IRQDONE` bit.

Setting Up a DMA Receive Operation

This section describes the typical steps for using the link ports in DMA receive mode.

1. Enable the link port pins in GPIO port mux using the appropriate `PORT_FER` and `PORT_MUX` registers.
2. Install interrupt handlers for DMA and for transfer status (service request interrupt).
3. Configure the link port for reception (clear the `LP_CTL.TRAN` bit) and enable the receive request interrupt mask by setting the `LP_CTL.RRQMSK` bit.
4. If using DMA stop mode or auto buffer mode, program the DMA registers.

ADDITIONAL INFORMATION: An example configuration is: `DMA_ADDRSTART`, `DMA_XCNT`, `DMA_XMOD`, and `DMA_CFG` registers (Stop/Auto, `DMA_CFG.PSIZE`=1, `DMA_CFG.MSIZE`=4, interrupt generation and memory write).

5. If using DMA array mode or list mode, create DMA configuration data structures filled with components.

ADDITIONAL INFORMATION: An example configuration is: `DMA_ADDRSTART`, `DMA_XCNT`, `DMA_XMOD`, and `DMA_CFG` registers (Array/List, `DMA_CFG.PSIZE` =1, `DMA_CFG.MSIZE` =4, interrupt generation, memory write and fetch =4/5) and `DMA_DSCPTR_NXT` register (if list mode). Further, program

DMA configuration register (Array/List, `DMA_CFG.PSIZE = 1`, `DMA_CFG.MSIZE = 4`, Memory Write and Fetch = 4/5) and program the `DMA_DSCPTR_NXT` register (if list mode).

6. Wait for the link port transmitter (connected externally) to be enabled with subsequent data transmission. The application can wait for the receive service request interrupt to assert.
7. Clear the receive service request interrupt status by writing 1 to the `LP_STAT.LRRQ` bit.
8. Enable DMA by setting the `DMA_CFG.EN` bit.
9. Enable the link port by setting the `LP_CTL.EN` bit.
10. Wait for DMA to assert the transfer complete interrupt.
11. Clear the DMA interrupt source by writing 1 to the `DMA_STAT.IRQDONE` bit of the DMA status register.

Setting Up a Core Transmit Operation

This section describes the typical steps for using the link ports in processor core based transmission.

1. Enable the link port pins in the GPIO port mux using the appropriate `PORT_FER` and `PORT_MUX` registers.
2. Configure the drive strength value to b(010) for the link port clock pins to operate the link port at maximum frequency (`PADS_PORTB0_DS` and `PADS_PORTC0_DS` registers).
3. Install interrupt handlers for data transfer and for transfer status (service request interrupt). The interrupt handlers for data transfer are the same source or ID as the DMA interrupt line in the SEC.
4. Configure the link port for transmission by setting the `LP_CTL.TRAN` bit) and enable the transmit request interrupt mask by setting the `LP_CTL.TRQMSK` bit).
5. Program the link port clock divider by writing a value in to the `LP_DIV` register.
6. Wait for the link port receiver (connected externally) to be enabled. The application can wait for a transmit service request interrupt to assert.
7. Clear the transmit service request interrupt status by writing 1 to the `LP_STAT.LTRQ` bit.
8. Enable the link port by setting the `LP_CTL.EN` bit.
9. The data request interrupt is asserted whenever there is free space in the FIFO. The application can write to the `LP_TX` register based on the FIFO conditions (half or empty) reflected in the `LP_STAT.FFST` bit field.

Setting Up a Core Receive Operation

This section describes the typical steps for using the link ports in processor core-based reception.

1. Enable the link port pins in the GPIO port mux using the appropriate `PORT_FER` and `PORT_MUX` registers.
2. Install interrupt handlers for data transfer and for transfer status (service request interrupt). The interrupt handlers for data transfer are the same source or ID as the DMA interrupt line in the SEC).

3. Configure link port for reception (clear `LP_CTL.TRAN` bit). Enable the receive request interrupt mask bit (set `LP_CTL.RRQMSK`).
4. Wait for the link port transmit (connected externally) to be enabled with subsequent transmission of data. The application can wait for receive service request interrupt to be asserted.
5. Clear the receive service request interrupt status by writing 1 to the `LP_STAT.LRRQ` bit.
6. Enable the link port by setting the `LP_CTL.EN` bit.
7. The data request interrupt is asserted whenever there is free space in the FIFO. The application can read from the `LP_RX` register based on the FIFO conditions (1 or 2 or 3 data available) which is reflected in the `LP_STAT.FFST` bit field.

ADSP-2159x LP Register Descriptions

Link Port (LP) contains the following registers.

Table 15-6: ADSP-2159x LP Register List

Name	Description
<code>LP_CTL</code>	Control Register
<code>LP_DIV</code>	Clock Divider Value Register
<code>LP_RX</code>	Receive Buffer Register
<code>LP_STAT</code>	Status Register
<code>LP_TX</code>	Transmit Buffer Register
<code>LP_TXIN_SHDW</code>	Shadow Input Transmit Buffer Register
<code>LP_TXOUT_SHDW</code>	Shadow Output Transmit Buffer Register

Control Register

The LP_CTL register provides LP interrupt masking, selection of transfer direction, and link port enable.

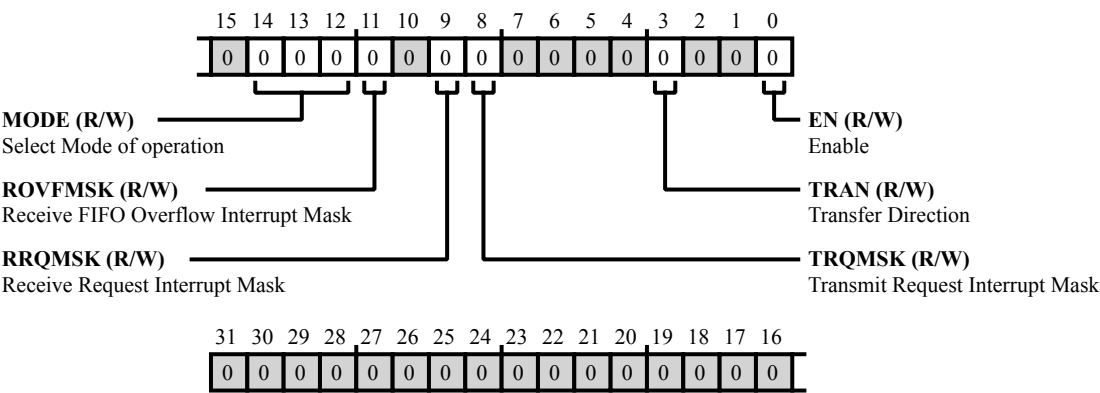


Figure 15-13: LP_CTL Register Diagram

Table 15-7: LP_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14:12 (R/W)	MODE	Select Mode of operation. select each mode of operations - SDR (2bit and 4 bit) , DDR (2 bit, 4bit and 8bit)
		0 8BIT Single Data Rate Mode selected (D0, D1, D2, D3, D4, D5, D6 and D7 data pins used), Default Mode
		1 4BIT Single Data Rate Mode selected (D0, D1, D2 and D3 data pins used)
		2 2BIT Single Data Rate Mode selected (D0 and D1 data pins used)
		4 Reserved
		5 4BIT Double Data Rate Mode selected (D0, D1, D2 and D3 data pins used)
		6 2BIT Double Data Rate Mode selected (D0 and D1 data pins used)
11 (R/W)	ROVFMSK	Receive FIFO Overflow Interrupt Mask. Receive FIFO Overflow Interrupt Mask
		0 Mask Disable Receive FIFO Overflow Interrupt
		1 Unmask Enable Receive FIFO Overflow Interrupt

Table 15-7: LP_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	RRQMSK	Receive Request Interrupt Mask. Link Port Receive Request Mask
		0 Mask Disable Receive Request interrupt.
		1 Unmask Enable Receive Request interrupt.
8 (R/W)	TRQMSK	Transmit Request Interrupt Mask. Link Port Transmit Request Mask
		0 Mask Disable Transmit Request interrupt.
		1 Unmask Enable Transmit Request interrupt.
3 (R/W)	TRAN	Transfer Direction. The LP_CTL . TRAN bit selects the transfer direction as transmit (if set) or receive (if cleared) for link buffer.
		0 Receive Direction transfer is receive
		1 Transmit Direction transfer is transmit
0 (R/W)	EN	Enable. The LP_CTL . EN enables or disables the link port. When the processor disables the port (LP_CTL . EN transitions from high to low), the processor clears the corresponding LP_STAT bits.
		0 Disable Disable linkport
		1 Enable linkport Enable linkport

Clock Divider Value Register

The `LP_DIV` register selects the divisor for ratio between the internal LP clock (LCLK) and system clock (CLKO8). This programming is applicable only for the transmitter. The receiver can operate at any asynchronous frequency up to the maximum frequency independent of the ratio programmed.

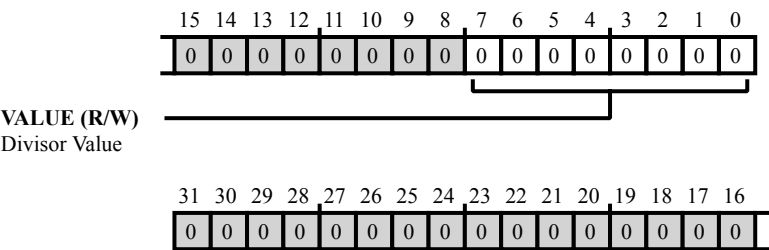


Figure 15-14: LP_DIV Register Diagram

Table 15-8: LP_DIV Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	Divisor Value. The <code>LP_DIV.VALUE</code> bits select the clock divider (relating the LP' internally generated clock (LCLK) to the system clock (CLKO8). The <code>LP_DIV.VALUE</code> should be programmed prior to LP enable. For <code>LP_DIV.VALUE</code> = 0, LCLK = CLKO8 For <code>LP_DIV.VALUE</code> = xxxxxxxx, LCLK = CLKO8 / (2 x DIV)

Receive Buffer Register

The `LP_RX` register buffers the receive data flow through the LP. The receive buffer is a four-location deep FIFO. In the receive buffer, data is transferred to the core or DMA from the receive FIFO where an internal register does the packing. This packing register is not software accessible. For more information on LP buffer features and operations, see the LP functional description.

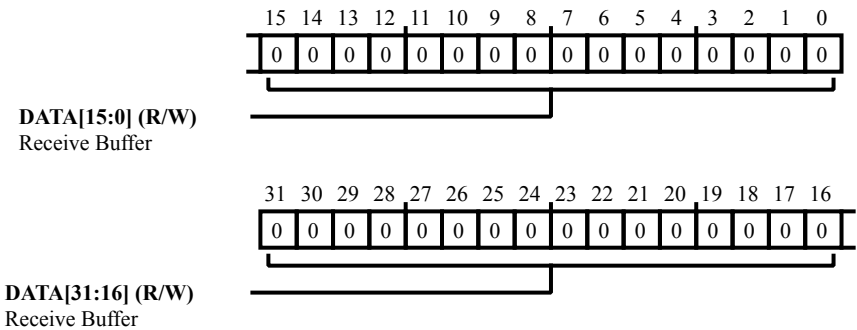


Figure 15-15: LP_RX Register Diagram

Table 15-9: LP_RX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	Receive Buffer.

Status Register

The `LP_STAT` register provides status information on link port interrupts, FIFO, buses, and receive/transmit requests.

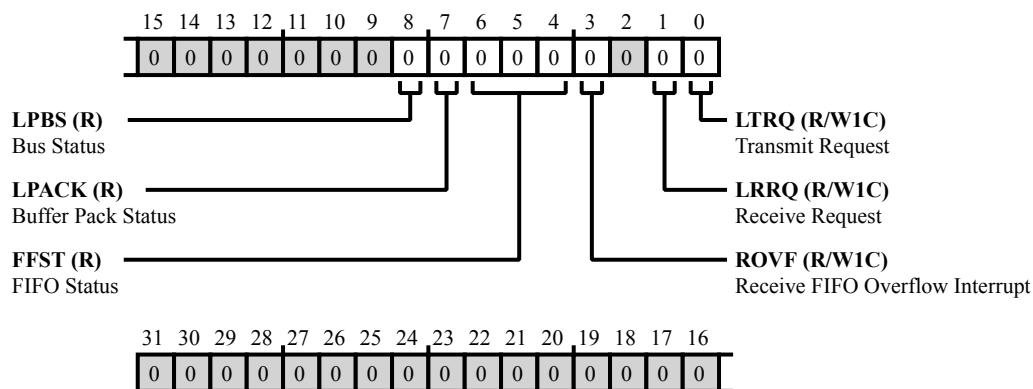


Figure 15-16: LP_STAT Register Diagram

Table 15-10: LP_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	LPBS	Bus Status. The <code>LP_STAT.LPBS</code> bit indicates the LPDAT bus status. <code>LP_STAT.LPBS</code> is kept high if data is being driven by the link port into the <code>LP_D[n]</code> pins.
		0 Bus is Idle Link Port Bus is idle
		1 Bus Busy Link Port Bus is busy
7 (R/NW)	LPACK	Buffer Pack Status. The <code>LP_STAT.LPACK</code> bit indicates packing status. In receive mode, 32-bit data is received in 4 blocks of 8-bit data. Then, the data is packed to get a single 32-bit data before loading the FIFO. The <code>LP_STAT.LPACK</code> bit is high during this packing process and goes low after packing. In transmit mode, 32-bit data in the FIFO is unpacked to 4 blocks of 8-bit data before sending. The <code>LP_STAT.LPACK</code> is high during unpacking.
		0 Packing Complete Packing done
		1 Packing Incomplete Packing is in progress
6:4 (R/NW)	FFST	FIFO Status. The <code>LP_STAT.FFST</code> bits indicate the FIFO status. These bits are cleared when the LP is disabled.
		0 TX - Empty; RX - Empty Link buffer (TX OR RX) empty

Table 15-10: LP_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		1 TX - Reserved ; RX - Has 1 data word RX has 1 word of data. TX reserved
		2 TX - Reserved; RX - Has 2 data words RX has 2 word of data. TX reserved.
		3 TX - Reserved; RX - Has 3 data words RX has 3 word of data. TX reserved.
		4 TX - One Word; RX -Has 4 data words RX has 4 word of data. TX 1 word of data.
		5 Reserved
		6 TX - FIFO Full; RX - Reserved RX reserved. TX reserved.
		7 Reserved
3 (R/W1C)	ROVF	Receive FIFO Overflow Interrupt. This interrupt is generated when the receiver FIFO overflows. This overflow may happen if the transmitter continues to transmit data even though the receiver has de-asserted the LP_ACK pin.
1 (R/W1C)	LRRQ	Receive Request. The LP generates this interrupt when the LP_CLK pin of a disabled link port (the receiver) is forced high by another link port (the transmitter).
0 (R/W1C)	LTRQ	Transmit Request. The LP generates this interrupt when the LP_ACK pin of a disabled link port (the transmitter) is forced high by another link port (the receiver).

Transmit Buffer Register

The `LP_TX` register buffers the transmit data flow through the LP. The transmit buffer is two words deep. In the transmit buffer, the input stage of the FIFO is used to accept core data or DMA data from internal memory, and the data is transferred to the link port interface from the output stage of the FIFO. The output stage performs the unpacking in the transmit buffer. The least significant byte is transmitted first. As each word is unpacked and transmitted, the next location in FIFO becomes available and a new DMA request is made if DMA is enabled. If the register becomes empty, the LP asserts the `LP_CLK` signal. For more information on LP buffer features and operations, see the LP functional description.

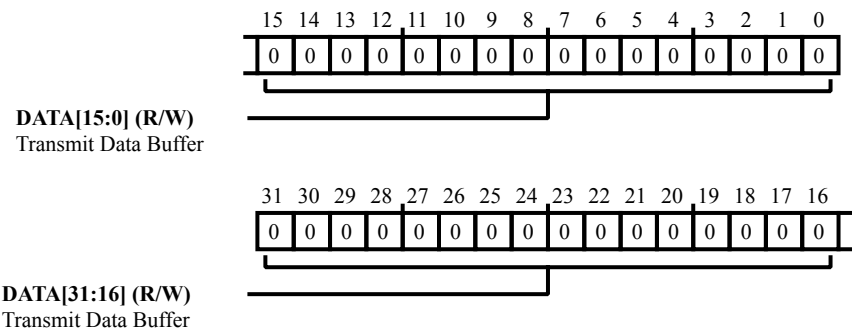


Figure 15-17: LP_TX Register Diagram

Table 15-11: LP_TX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	Transmit Data Buffer.

Shadow Input Transmit Buffer Register

The `LP_TXIN_SHDW` register contains the same data as the input stage of the transmit buffer. Read of this shadow transmit buffer does not update the `LP_STAT` register.

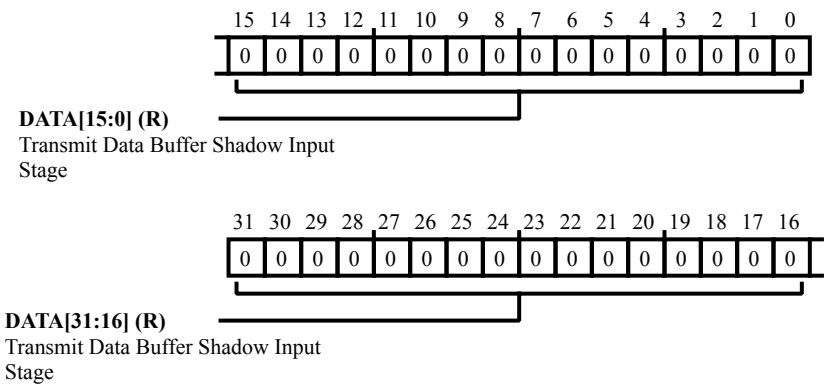


Figure 15-18: LP_TXIN_SHDW Register Diagram

Table 15-12: LP_TXIN_SHDW Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	DATA	Transmit Data Buffer Shadow Input Stage.

Shadow Output Transmit Buffer Register

The `LP_TXOUT_SHDW` register contains the same data as the output stage of the transmit buffer. Read of this shadow transmit buffer does not update the `LP_STAT` register.

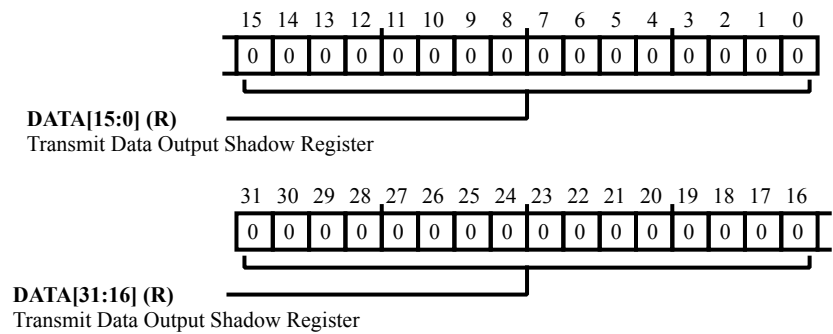


Figure 15-19: LP_TXOUT_SHDW Register Diagram

Table 15-13: LP_TXOUT_SHDW Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	DATA	Transmit Data Output Shadow Register.

16 Serial Peripheral Interface (SPI)

The serial peripheral interface is an industry-standard synchronous serial link that supports communication with multiple SPI-compatible devices. The baseline SPI peripheral is a synchronous, four-wire interface consisting of two data pins, one device select pin, and a gated clock pin. The two data pins allow full-duplex operation to other SPI-compatible devices. Two extra (optional) data pins are provided on specific SPIs to support quad SPI operation. Enhanced modes of operation such as flow control, fast mode, and dual-I/O mode (DIOM) are also supported. In addition, a direct memory access (DMA) mode allows for transferring several words with minimal CPU interaction.

With a range of configurable options, the SPI ports provide a glueless hardware interface with other SPI-compatible devices in master mode, slave mode, and multimaster environments. The SPI peripheral includes programmable baud rates, clock phase, and clock polarity. The peripheral can operate in a multimaster environment by interfacing with several other devices, acting as either a master device or a slave device. In a multimaster environment, the SPI peripheral uses open-drain outputs to avoid data bus contention. The flow control features enable slow slave devices to interface with fast master devices by providing an SPI ready pin which flexibly controls the transfers.

NOTE: SPI peripherals (SPI0, SPI1, and SPI3) on the processor operate in the SCLK0 domain. SPI2 operates in the SCLK1 domain. For more details on SCLK programming, refer the *Clock Generation Unit (CGU)* chapter.

SPI Features

The SPI module supports the following features:

- Full-duplex, synchronous serial interface
- 8, 16-bit and 32-bit word sizes
- Programmable baud rate, clock phase, and polarity
- Programmable interframe latency
- Flow control
- Support for Fast and DIOM modes
- SPI1, SPI2 and SPI3 support quad mode. Memory-mapped mode is supported by SPI2 only.
- Independent receive and transmit DMA channels

- Burst transfer mode for non-DMA write accesses

SPI Functional Description

This section provides information on the function of the SPI module.

Shift register functionality

The SPI is essentially a shift register that serially transmits and receives data bits to or from other SPI devices. During an SPI transfer, data is simultaneously transmitted (shifted out serially) and received (shifted in serially). A serial clock line synchronizes shifting and sampling of the information on the two serial data lines.

Master slave functionality

During a data transfer, one SPI system acts as the link master which controls the data flow. The other system acts as the slave, which has data shifted into and out of it by the master. Different devices can take turn being masters, and one master can simultaneously shift data into multiple slaves (broadcast mode). However, only one slave can drive its output to write data back to the master at any given time. This rule must be enforced in the broadcast mode. Several slaves can be selected to receive data from the master in this mode. But only one slave can be enabled to send data back to the master.

Enhanced operating modes

SPI supports enhanced modes of operation like fast mode, DIOM, and Quad-SPI, and optional flow control. In fast mode, received data is sampled on the transmit edge instead of the standard receive edge, thus enabling a full-cycle path for the received data. In DIOM, both MOSI and MISO are configured as input or output pins, and 2 bits are shifted in or out on each receive or transmit edge. In Quad-SPI mode, SPI_D3:0 are configured as input or output pins and 4 bits are shifted in or out on each receive or transmit edge. A slower slave can use flow control to stall a faster master device.

Single and multi-master use

The SPI can be used in a single master as well as multi-master environment. The `SPI_MOSI`, `SPI_MISO`, and the `SPI_CLK` signals are all tied together in both configurations. SPI transmission and reception can be enabled simultaneously or individually, depending on `SPI_RXCTL` and `SPI_TXCTL` settings. In broadcast mode, several slaves can be enabled to receive, but only one slave must be in transmit mode and driving the `SPI_MISO` line.

ADSP-2159x SPI Register List

The Serial Peripheral Interface (SPI) provides a full-duplex, synchronous serial interface, which supports both master/slave modes and multi-master environments. The SPI's baud rate and clock phase/polarities are programmable, and it has integrated DMA channels for both transmit and receive data streams. A set of registers governs SPI operations. For more information on SPI functionality, see the SPI register descriptions.

Table 16-1: ADSP-2159x SPI Register List

Name	Description
SPI_CLK	Clock Rate Register
SPI_CTL	Control Register
SPI_DLY	Delay Register
SPI_ILAT	Masked Interrupt Condition Register
SPI_ILAT_CLR	Masked Interrupt Clear Register
SPI_IMSK	Interrupt Mask Register
SPI_IMSK_CLR	Interrupt Mask Clear Register
SPI_IMSK_SET	Interrupt Mask Set Register
SPI_MMRDH	Memory Mapped Read Header
SPI_MMTOP	SPI Memory Top Address
SPI_RFIFO	Receive FIFO Data Register
SPI_RWC	Received Word Count Register
SPI_RWCR	Received Word Count Reload Register
SPI_RXCTL	Receive Control Register
SPI_SLVSEL	Slave Select Register
SPI_STAT	Status Register
SPI_TFIFO	Transmit FIFO Data Register
SPI_TWC	Transmitted Word Count Register
SPI_TWCR	Transmitted Word Count Reload Register
SPI_TXCTL	Transmit Control Register

ADSP-2159x SPI Interrupt List

Table 16-2: ADSP-2159x SPI Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
121	SPI0_TXDMA	SPI0 TX DMA Channel	Level	22
122	SPI0_RXDMA	SPI0 RX DMA Channel	Level	23
123	SPI0_STAT	SPI0 Status	Level	
124	SPI0_ERR	SPI0 Error	Level	
125	SPI1_TXDMA	SPI1 TX DMA Channel	Level	24
126	SPI1_RXDMA	SPI1 RX DMA Channel	Level	25

Table 16-2: ADSP-2159x SPI Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
127	SPI1_STAT	SPI1 Status	Level	
128	SPI1_ERR	SPI1 Error	Level	
129	SPI2_TXDMA	SPI2 TX DMA Channel	Level	26
130	SPI2_RXDMA	SPI2 RX DMA Channel	Level	27
131	SPI2_STAT	SPI2 Status	Level	
132	SPI2_ERR	SPI2 Error	Level	
133	SPI3_TXDMA	SPI3 TX DMA Channel	Level	55
134	SPI3_RXDMA	SPI3 RX DMA Channel	Level	56
135	SPI3_STAT	SPI3 Status	Level	
136	SPI3_ERR	SPI3 Error	Level	
261	SPI0_TXDMA_ERR	SPI0 TX DMA Channel Error	Level	
262	SPI0_RXDMA_ERR	SPI0 RX DMA Channel Error	Level	
263	SPI1_TXDMA_ERR	SPI1 TX DMA Channel Error	Level	
264	SPI1_RXDMA_ERR	SPI1 RX DMA Channel Error	Level	
265	SPI2_TXDMA_ERR	SPI2 TX DMA Channel Error	Level	
266	SPI2_RXDMA_ERR	SPI2 RX DMA Channel Error	Level	
267	SPI3_TXDMA_ERR	SPI3 TX DMA Channel Error	Level	
268	SPI3_RXDMA_ERR	SPI3 RX DMA Channel Error	Level	

ADSP-2159x SPI Trigger List

Table 16-3: ADSP-2159x SPI Trigger List Masters

Trigger ID	Name	Description	Sensitivity
82	SPI0_TXDMA	SPI0 TX DMA Channel	Edge
83	SPI0_RXDMA	SPI0 RX DMA Channel	Edge
84	SPI1_TXDMA	SPI1 TX DMA Channel	Edge
85	SPI1_RXDMA	SPI1 RX DMA Channel	Edge
86	SPI3_TXDMA	SPI3 TX DMA Channel	Edge
87	SPI3_RXDMA	SPI3 RX DMA Channel	Edge
88	SPI2_TXDMA	SPI2 TX DMA Channel	Edge
89	SPI2_RXDMA	SPI2 RX DMA Channel	Edge

Table 16-4: ADSP-2159x SPI Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
52	SPI0_TXDMA	SPI0 TX DMA Channel	Pulse
53	SPI0_RXDMA	SPI0 RX DMA Channel	Pulse
54	SPI1_TXDMA	SPI1 TX DMA Channel	Pulse
55	SPI1_RXDMA	SPI1 RX DMA Channel	Pulse
56	SPI3_TXDMA	SPI3 TX DMA Channel	Pulse
57	SPI3_RXDMA	SPI3 RX DMA Channel	Pulse
58	SPI2_TXDMA	SPI2 TX DMA Channel	Pulse
59	SPI2_RXDMA	SPI2 RX DMA Channel	Pulse

ADSP-2159x SPI DMA Channel List

Table 16-5: ADSP-2159x SPI DMA Channel List

DMA ID	DMA Channel Name	Description
DMA22	SPI0_TXDMA	SPI0 TX DMA Channel
DMA23	SPI0_RXDMA	SPI0 RX DMA Channel
DMA24	SPI1_TXDMA	SPI1 TX DMA Channel
DMA25	SPI1_RXDMA	SPI1 RX DMA Channel
DMA26	SPI2_TXDMA	SPI2 TX DMA Channel
DMA27	SPI2_RXDMA	SPI2 RX DMA Channel
DMA55	SPI3_TXDMA	SPI3 TX DMA Channel
DMA56	SPI3_RXDMA	SPI3 RX DMA Channel

SPI Block Diagram

The *SPI Controller Block Diagram* illustrates the blocks of the SPI module. The module is comprised of three primary parts:

- SPI core contains the receive and transmit FIFOs and their associated shift registers
- Control blocks contain the synchronizer and logic to control the data flow through the data pipelines
- Register block

Transfer Protocol

The SPI module implements two channels that are independent of each other. The SPI module uses the [SPI_RXCTL](#) and [SPI_TXCTL](#) dedicated control registers to control these channels. Except in dual and quad modes, SPI can enable and use both channels simultaneously.

The SPI protocol supports four different combinations of serial clock phase and polarity. These combinations are selected through the `SPI_CTL.CPOL` and `SPI_CTL.CPHA` bits.

The *SPI Transfer Protocol* figures demonstrate the two basic transfer formats as defined by the `CPHA` bit. Two waveforms are shown for `SPI_CLK`; one for `SPI_CTL.CPOL=0` and the other for `SPI_CTL.CPOL=1`. The diagrams can be interpreted as master or slave timing diagrams since the `SPI_CLK`, `SPI_MISO`, and `SPI_MOSI` pins are directly connected between the master and the slave. The `SPI_MISO` signal is the output from the slave (slave transmission), and the `SPI_MOSI` signal is the output from the master (master transmission). The master generates the `SPI_CLK` signal. The `SPI_SS` signal is the slave device select input to the slave from the master. The diagrams represent an 8-bit transfer (`SPI_CTL.SIZE=0`) with the MSB first (`SPI_CTL.LSBF=0`). Any combination of the `SPI_CTL.SIZE` and `SPI_CTL.LSBF` bits is permissible. For example, a 16-bit transfer with the LSB first is another possible configuration.

The clock polarity and the clock phase could be identical for the master device and the slave device involved in the communication link. The transfer format from the master can be changed between transfers to adjust for various requirements of a slave device.

The SPI module uses the `SPI_CTL.ASSEL` bit to determine when the SPI hardware or software control the `SPI_SEL[n]` line. When `SPI_CTL.ASSEL=1`, the slave select line must be set to the polarity set in the `SPI_CTL.SELST` field between each serial transfer. The actual behavior of `SPI_SEL[n]` also depends on the parameters programmed into the `SPI_DLY` register. The SPI hardware logic automatically controls this functionality. When `SPI_CTL.ASSEL=0`, `SPI_SEL[n]` can either remain active between successive transfers or be inactive. The software must control this activity through manipulation of the `SPI_SLVSEL` register.

The *SPI Transfer Protocol* pair of figures illustrates the case when `SPI_CTL.ASSEL = 1` and the `SPI_SEL[n]` line is inactive between frames. If `ASSEL = 0`, the `SPI_SEL[n]` line can remain active between frames; however, the first bit is only driven when an active transition of `SPI_CLK` occurs.

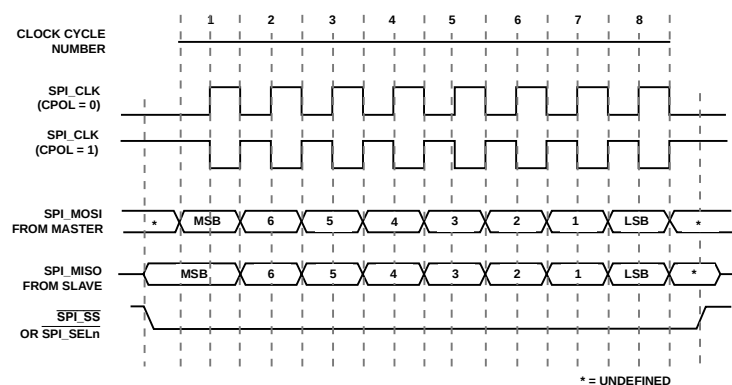


Figure 16-1: SPI Transfer Protocol for `CPHA=0`

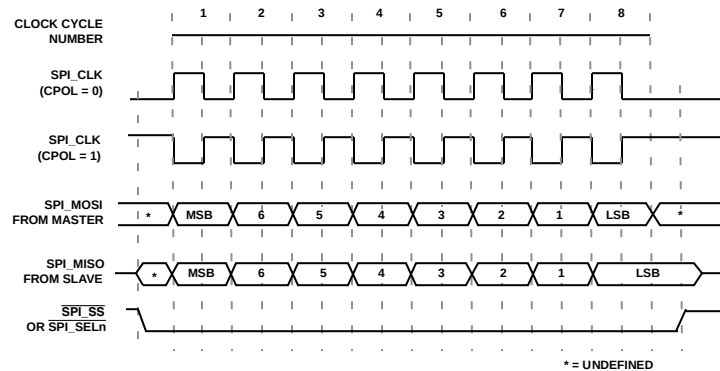


Figure 16-2: SPI Transfer Protocol for CPHA=1

Clock Considerations

The `SPI_CLK` signal is a gated clock that is only active during data transfers, for the time of the transferred word. In normal mode, the number of active edges is equal to the number of bits to be transmitted or received. In dual-I/O mode, it is half of the number of bits to be transmitted or received, and in quad-SPI mode it is one-fourth of the number. The clock rate can be derived using both even and odd dividers from `CLKO6`.

For master devices, the SPI uses the `SPI_CLK` register value to determine the clock rate, whereas this value is ignored for slave devices.

When the SPI controller is a master, `SPI_CLK` is an output signal. Conversely, when the SPI controller is a slave, `SPI_CLK` is an input signal. Slave devices ignore the SPI clock when the slave select input is driven inactive. The SPI uses the `SPI_CLK` signal to shift out and shift in the data driven onto the `SPI_MISO` and `SPI_MOSI` lines. The data is always shifted out on one edge of the clock (the active edge) and sampled on the opposite edge of the clock (the sampling edge). Clock polarity and clock phase relative to data are programmable through the `SPI_CTL` register and define the transfer format.

Controlling Delay Between Frames

The *SPI Timing with Lead and Lag Programming (Independent of SPI_CTL.CPHA Setting)* figure illustrates SPI timing using the `SPI_DLY.LEADX` and `SPI_DLY.LAGX` programming. The SPI uses the `SPI_DLY.LAGX` bits to control the timing between the slave select (`SPI_SS`) signal assertion and the first `SPI_CLK` edge. The SPI uses the `SPI_DLY.LEADX` bits to control the timing between the last `SPI_CLK` edge and deassertion of the `SPI_SS` signal. The lead and lag timing can be extended by a 1 `SPI_CLK` duration to ease timing restrictions on the slave device.

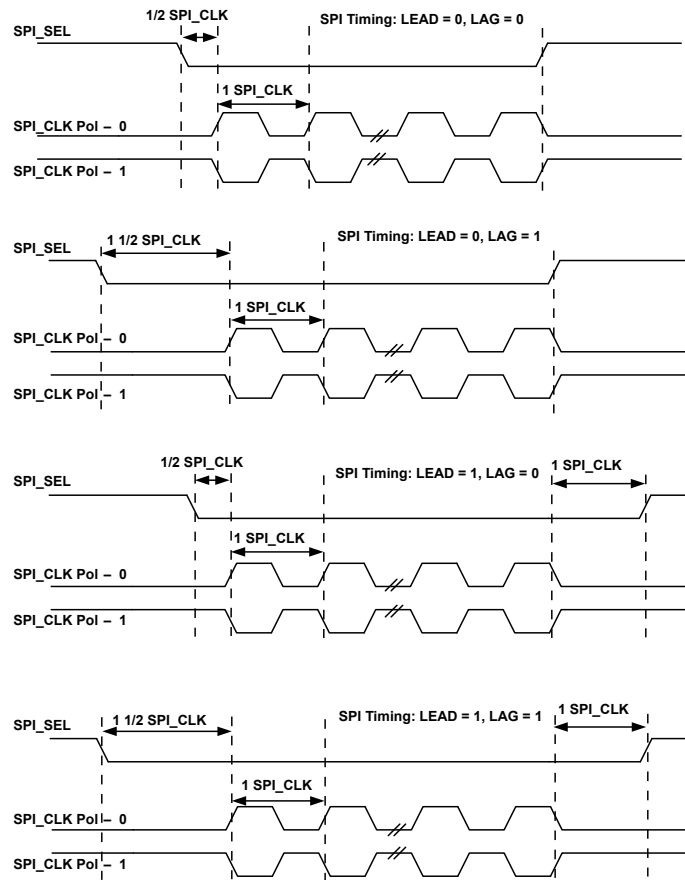


Figure 16-3: SPI Timing with Lead and Lag Programming (Independent of SPI_CTL.CPHA Setting)

The *SPI Timing with SPI_DLY.STOP Programming (Independent of SPI_CTL.CPHA Setting)* figure illustrates SPI timing with STOP programming. The SPI module uses this timing to insert multiples of SPI_CLK period delays between transfers. The SPI_SS line is deasserted for the duration specified in the SPI_DLY.STOP bit field, assuming the SPI_CTL.SELST bit is configured for deassertion between transfers.

If the SPI_DLY.STOP bit = 0, the master operates in a *continuous mode*. This mode causes an immediate start of the second word after the last bit is transferred from the first word. During this mode of operation, the slave select line is continuously asserted.

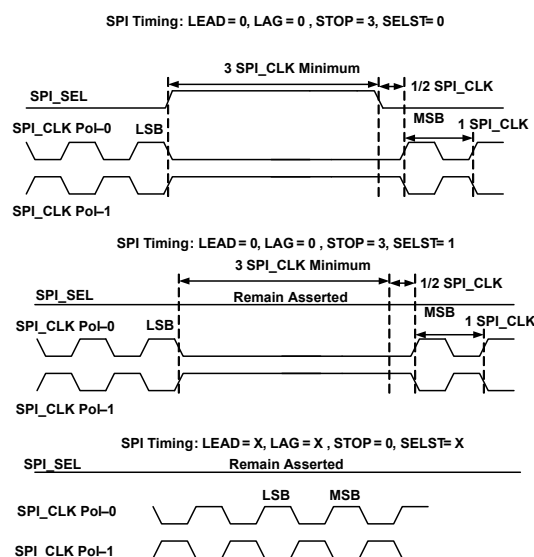


Figure 16-4: SPI Timing with SPI_DLY.STOP Programming (Independent of SPI_CTL.CPHA Setting)

When the `SPI_DLY.STOP` bit = 0 and initial conditions for a transfer are not met, the interface pauses before the next transfer. During this pause, the SPI uses the `SPI_CTL.SELST` bit to determine the state of the slave select pin. The SPI uses the `SPI_DLY.LEADX` and `SPI_DLY.LAGX` bits to determine the timing between `SPI_CLK` edges and the slave select line.

Flow Control

In master mode, the slave device must drive the `SPI_RDY` pin. The pin acts as an input signal. The slave can deassert the `SPI_RDY` pin to stop the master from initiating any new transfer. If `SPI_RDY` is deasserted in the middle of a transfer, the current transfer continues, and the next transfer will not start unless the slave asserts the `SPI_RDY` signal. Whenever the slave deasserts `SPI_RDY` and stalls the master, the SPI controller goes into a waiting state, and the `SPI_STAT.FCS` bit is set. When the slave asserts `SPI_RDY`, the SPI controller resumes operation, and the `SPI_STAT.FCS` bit is cleared.

In slave mode, the `SPI_RDY` pin acts as an output signal. Flow control can be configured on either the TX channel or the RX channel. The SPI uses the `SPI_CTL.FCCH` bit to control this configuration. If flow control is configured on the TX channel, as the `SPI_TFIFO` status nears the empty condition, the `SPI_RDY` pin is deasserted. If flow control is configured on the RX channel, as the `SPI_RFIFO` status nears the full condition, the `SPI_RDY` pin is deasserted. The SPI uses the `SPI_CTL.FCWM` bits to control the FIFO status at which `SPI_RDY` deassertion takes place. Flow control in slave mode is purely based on the FIFO status and does not depend on the word counters.

The *SPI Flow Control Timing in Master Mode* figure illustrates this timing.

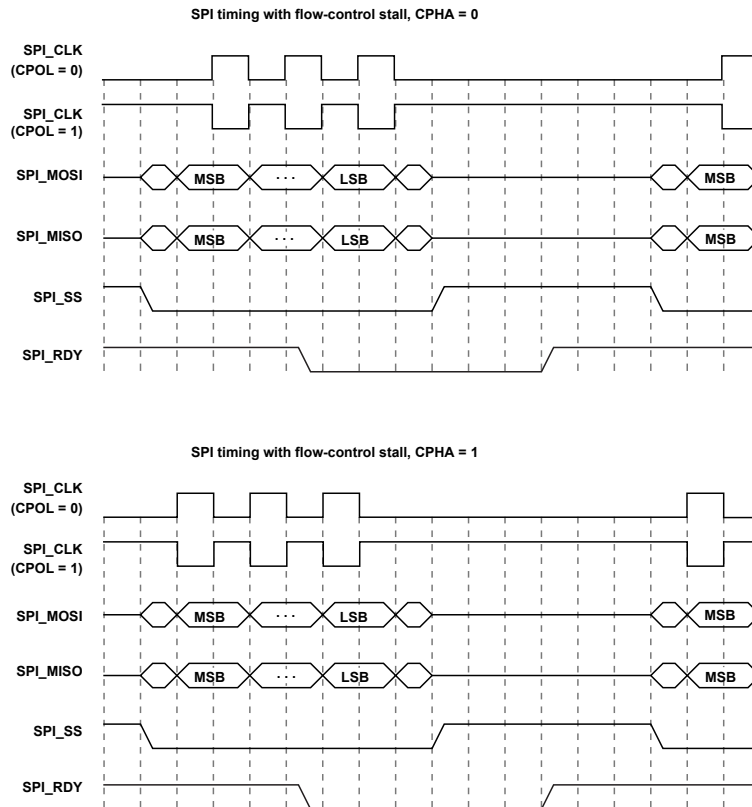


Figure 16-5: SPI Flow Control Timing in Master Mode.

Slave Select Operation

If the SPI is in slave mode, $\overline{\text{SPI_SS}}$ acts as the slave select input. When SPI is enabled as a master, $\overline{\text{SPI_SS}}$ can serve as an error detection input for the SPI in a multi-master environment. The SPI_CTL.PSSE bit enables this feature. When $\text{SPI_CTL.PSSE}=1$, the $\overline{\text{SPI_SS}}$ input is the master mode error input. Otherwise, $\overline{\text{SPI_SS}}$ is ignored.

The $\overline{\text{SPI_SS}}$ signal is an active-low signal. The master asserts the signal during the transfer. The signal can be deasserted or remain asserted between transfers. When $\overline{\text{SPI_SS}}$ is deasserted, SPI_CLK and inputs are ignored, and outputs are three-stated.

The slave select bits ($\text{SPI_SLVSEL.SSEL1} - \text{SPI_SLVSEL.SSEL7}$) are used in a multiple-slave SPI environment. For example, if there are eight SPI devices in the system including a processor master, the master processor can support the SPI mode transactions across the other seven devices. This configuration requires only one master processor in this multi-slave environment.

For example, assume that the SPI of the processor is the master. The $\text{SPI_SLVSEL.SSEL1} - \text{SPI_SLVSEL.SSEL7}$ bits on the processor can be connected to the slave select pin of each slave device. In this configuration, the slave select bits can be used in three ways. In cases 1 and 2, the processor is the master and the seven microcontrollers or peripherals with SPI interfaces are slaves. The processor can do one of the following:

1. Transmit to all seven SPI devices at the same time in a broadcast mode. Here, all slave select bits are set.

2. Receive and transmit from one SPI device by enabling only one slave SPI device at a time.
3. If all the slaves are also processors, then the requester can receive data from only one processor at a time. (The functionality is enabled by clearing the `SPI_CTL.EMISO` bit in the six other slave processors.) The requestor can transmit broadcast data to all seven at the same time. This MISO enabling feature is available in some other microcontrollers. Therefore, it is possible to use the MISO enabling feature with any other SPI device that includes this functionality.

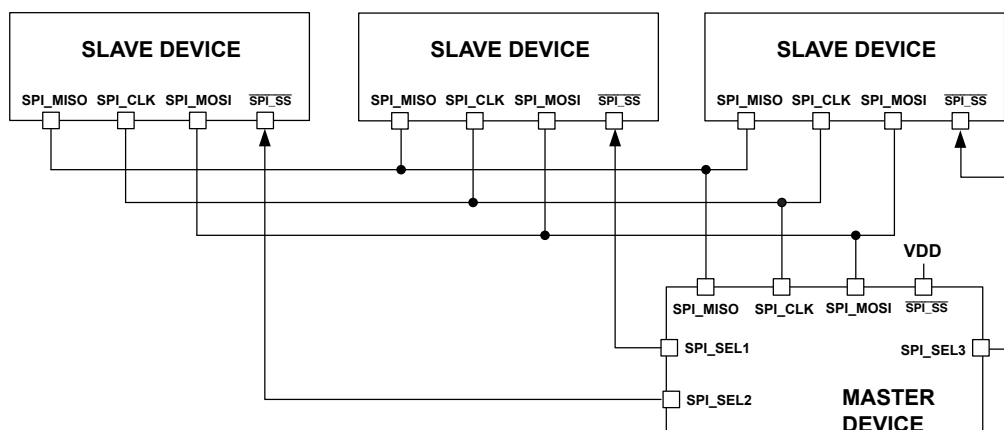


Figure 16-6: Single-Master, Multiple-Slave Configuration

Beginning and Ending a Non-DMA SPI Transfer

The start and finish of a non-DMA SPI transfer depend on the following settings.

1. Whether the device is configured as a master or a slave.
2. The state of the `SPI_CTL.ASSEL` bit, which selects between hardware and software control over `SPI_SLVSEL`.

When `SPI_CTL.CPHA=0`, the enabled slave select outputs are driven active. However, the `SPI_CLK` signal remains inactive for the first half of the first cycle of `SPI_CLK`. For a slave with `SPI_CTL.CPHA=0`, the transfer starts as soon as the `SPI_SS` input goes low.

When `SPI_CTL.CPHA=1`, a transfer starts with the first active edge of `SPI_CLK` for both slave and master devices. For a master device, a transfer is complete after it sends the last data and simultaneously receives the last data bit. A transfer for a slave device ends after the last sampling edge of `SPI_CLK`. If `SPI_CTL.ASSEL=1`, the hardware maintains responsibility for toggling `SPI_SS` between frames. If `SPI_CTL.ASSEL=0`, software controls the `SPI_SS` line and can keep it active between frames.

The `SPI_STAT.RFE` bit defines when the receive buffer can be read, indicating that `SPI_RFIFO` is not empty. The `SPI_STAT.TFF` bit defines when the transmit buffer can be written, indicating that the `SPI_TFIFO` is not full. The end of a single word transfer occurs when the `SPI_STAT.RFE` bit is cleared. The status indicates that a new word has been received and written into the receive FIFO. The `SPI_STAT.RFE` bit remains cleared as long as the receive FIFO has valid data.

To maintain software compatibility with other SPI devices, the `SPI_STAT.SPIF` bit is also available for polling. This bit can have a slightly different behavior from other commercially available devices.

In master mode with the `SPI_CTL.ASSEL` bit cleared, software manually asserts the required slave select signal before starting the transaction. After all data transfers, software typically releases the slave select line.

When the receive or transmit word counters are enabled in the `SPI_TXCTL` or `SPI_RXCTL` registers, the SPI generates a finish interrupt at the end of the transfer. It signals the end of all transfers related to that transaction.

Transmit Operation in Non-DMA Mode

The transmit operation in non-DMA mode is enabled through the `SPI_TXCTL.TEN` bit. It can be enabled independently from the receive operation, and the transmit channel can become the initiating channel based on the `SPI_TXCTL.TTI` bit setting.

Transmit underrun is not possible in this mode, as no new transfer initiates unless the transmit FIFO is empty (in the case that `SPI_TXCTL.TTI = 1`). A receive overflow is detected when data from a new frame transfer replaces older data in a full receive FIFO. This event can occur if `SPI_TXCTL.TTI = 1` and the receive channel is enabled in a non-initiating capacity.

An SPI transmit interrupt request is signaled once the transmit channel has been enabled and the transmit FIFO is not full. The SPI uses the `SPI_TXCTL.TDR` bit setting to control the frequency of the interrupt request.

Receive Operation in Non-DMA Mode

The receive operation in non-DMA mode is enabled through the `SPI_RXCTL.REN` bit. It can be enabled independently from the transmit operation, and the receive channel can become the initiating channel based on the `SPI_RXCTL.RTI` bit setting.

Receive overflow is not possible in this mode, as no new transfer initiates when the receive FIFO is full (in the case of `SPI_RXCTL.RTI = 1`). A transmit underrun can occur (`SPI_TXCTL.TDU` bit) when no valid data is in the `SPI_TFIFO` register when a transfer is initiated. This event can occur if `SPI_RXCTL.RTI = 1` and the transmit channel is enabled in a non-initiating capacity.

An SPI receive interrupt request is signaled once the receive channel has been enabled and there is data waiting to be read. The SPI uses the `SPI_RXCTL.RDR` bit setting to control the frequency of the interrupt request.

Dual I/O Mode

In Dual I/O mode, the `SPI_MISO` and `SPI_MOSI` pins are configured to operate in the same direction which doubles bandwidth. The SPI uses the `SPI_CTL.SOSI` bit to determine the order of bits on the pins. When set, the processor sends the first bit on the `SPI_MOSI` pin and the second bit on the `SPI_MISO` pin. If the `SPI_CTL.SOSI` bit is cleared, the order is reversed. Since dual I/O mode uses both pins to transmit or receive data, only one channel can be enabled, either transmit or receive. Flow control through the `SPI_RDY` pin is supported. Interrupt request generation is unaffected by dual I/O mode. However, the interrupt service interval is reduced, since the individual transfer latency is halved.

Changing to quad SPI mode must be done when the SPI is in a quiescent state.

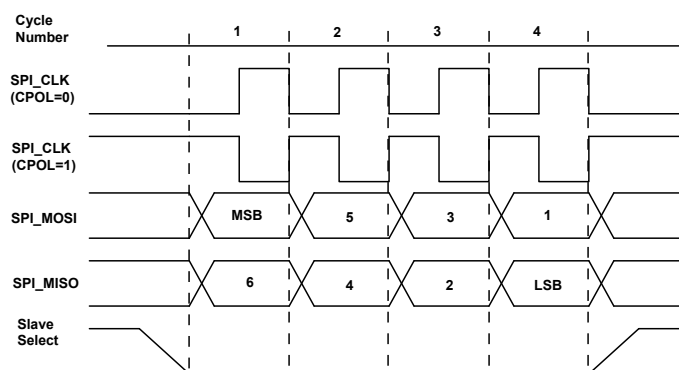


Figure 16-7: Dual I/O Mode Transfer Protocol for CPHA=0, SOSI=1, 8-Bit Transfer, LSBF=0.

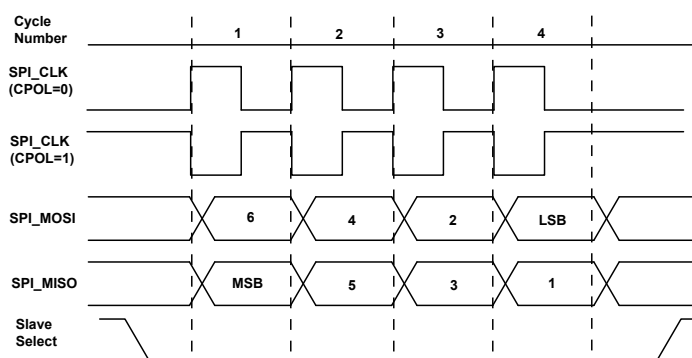


Figure 16-8: Dual I/O Mode Transfer Protocol for CPHA=1, SOSI=0, 8-Bit Transfer, LSBF=0.

Quad I/O Mode (SPI2 only)

In quad SPI mode, the `SPI_MISO` and `SPI_MOSI` pins, in tandem with the `SPI_D2` and `SPI_D3` pins, are configured to operate in the same direction. The SPI uses the `SPI_CTL.SOSI` bit to determine the order of bits on the pins. When set, the processor sends:

- The first bit on the `SPI_MOSI` pin
- The second bit on the `SPI_MISO` pin
- The third bit on the `SPI_D2` pin
- The fourth bit on the `SPI_D3` pin

If the `SPI_CTL.SOSI` bit is cleared, the order is reversed. Since quad SPI mode uses all four pins to transmit or receive data, only one channel can be enabled as either transmit or receive. Flow control through the `SPI_RDY` pin is supported. Interrupt generation is unaffected by quad SPI mode.

Changing to quad SPI mode must be done when the SPI is in a quiescent state.

While using dual or quad I/O mode for communicating with flash devices, program the `SPI_CTL.CPHA` and the `SPI_CTL.CPOL` bits = 1. This programming avoids bus contention during read operations, because the flash device starts driving out the bits immediately after dummy cycles in read header.

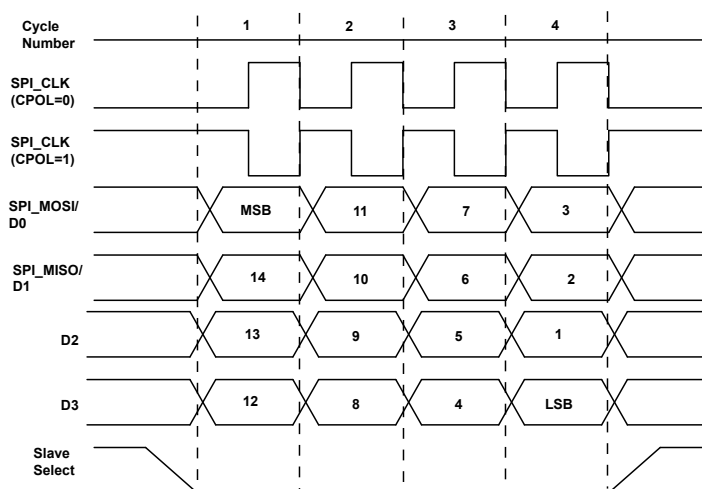


Figure 16-9: Quad Mode Timing for CPHA=0, SOSI=1, 16-Bit Transfer, LSBF=0.

NOTE: The SPI does support quad SPI 8-bit transfer in slave continuous mode of operation with an SCLK:SPI_CLK ratio of less than 1:2. A minimum of 2 SCLK cycles is required between transfers in 8-bit quad SPI slave mode with an SCLK:SPI_CLK ratio of less than 1:2.

Fast Mode

Fast mode is similar to the normal mode of operation when transmitting. When receiving, data is sampled at the next transmit edge allowing a full cycle of timing in the receive direction. This mode is valid in master mode operation only. When the SPI operates in fast mode, the slave drives the data for one full cycle.

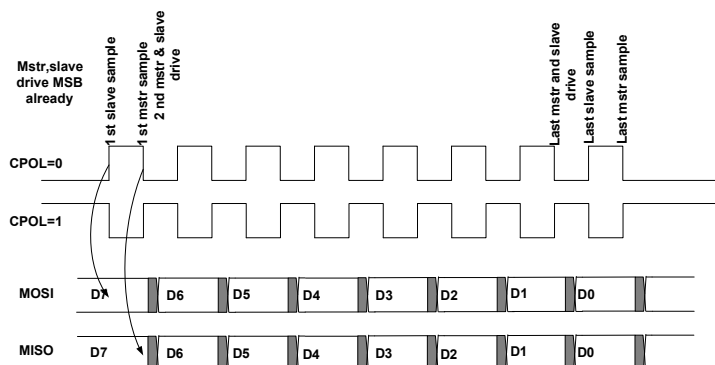


Figure 16-10: SPI Transfer Protocol in Fast Mode for SPI_CTL.CPHA = 0

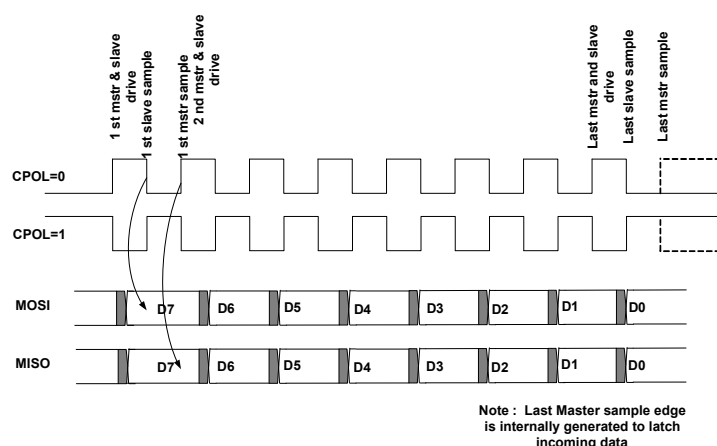


Figure 16-11: SPI Transfer Protocol in Fast Mode for SPI_CTL.CPHA = 1

Memory-Mapped Mode (SPI2 only)

The SPI supports direct memory-mapped read accesses from a SPI memory device, enabled by setting the SPI_CTL.MMSE bit. This mode allows for direct execution of instructions from a SPI memory device without the need for a low-level software driver, as hardware handles all overhead tasks (for example, transmission of the read header, pin turnaround timing, and receive data sizing). The SPI features configurable options in the memory-mapped read header register (SPI_MMRDH) to provide compatibility with a wide range of SPI memory devices.

In non-memory-mapped mode, the software is responsible for providing the command and required dummy words for the read response, whereas this is all handled by hardware when the SPI is in memory-mapped mode. The memory of the SPI device is accessible directly through reads of the processor address space. The read accesses can be code or data accesses in core mode or when using memory DMA (MDMA). These accesses allow code to execute directly from SPI memory devices (true eXecute-In-Place operations), and the contents can be cached to improve performance. It is not necessary to access the SPI data buffer registers nor poll status bits; however, the hardware does not support peripheral DMA accesses nor write operations to the SPI memory space.

The *Types of Operations* table is a comparison of the permitted operations in the non-memory-mapped and memory-mapped modes supported by the SPI controller.

Table 16-6: Types of Operations

SPI Operation	Non-Memory-Mapped Mode	Memory-Mapped Mode
Core data write	Yes	No
Core data read	Yes	Yes
Code fetch: Execute-In-Place (XIP)	No	Yes
Read/Write accesses using SPI Peripheral DMA	Yes	No
Read/Write accesses by other peripheral DMA channels	No	No
MDMA read*	No	Yes

Table 16-6: Types of Operations (Continued)

SPI Operation	Non-Memory-Mapped Mode	Memory-Mapped Mode
MDMA write	No	No

NOTE: MDMA read* - Only standard bandwidth MDMA (MDMA0 and MDMA1) and Enhanced Bandwidth MDMA (MDMA2) can be used for accessing flash address space in memory mapped mode. The maximum bandwidth MDMA (MDMA3) cannot access flash address space.

Memory-Mapped Description of Operation

Memory-mapped mode is enabled by setting the `SPI_CTL.MMSE` bit. When enabled, the SPI (if ready) accepts the read requests through a dedicated on-chip slave interface. The memory subsystem master drives this dedicated interface through the SCB fabric.

In a typical scenario, the memory subsystem master issues read requests to the fabric, and the fabric routes these requests to the slave port of the SPI peripheral. The master describes the read access using a number of parameters such as starting address, transfer size, and burst type. The SPI responds to this read access request when it is ready for a new transfer. It loads the opcode, a specified number of address bytes, and an optional mode byte into the transmit FIFO. The SPI memory state machine begins when both the transmit and receive channels of the SPI are enabled with:

- the transmit transfer initiation bit is set (`SPI_TXCTL.TTI=1`), *and*
- the receive initiation bit is cleared (`SPI_RXCTL.RTI=0`)

The SPI memory read sequence starts with the assertion of `SPI_SEL1`. If the SPI memory state machine is in the reset state, it looks for a command. The SPI hardware then sends the specific 8-bit read command (which can be optionally skipped), followed by the SPI memory read address. After this, a dummy period is inserted, in which a mode byte is optionally sent and the pins are held or three-stated during the dummy clocking period.

NOTE: This read header is transmitted over the SPI standard protocol pins (`SPI_CLK`, `SPI_MOSI`, `SPI_MISO`, `SPI_SEL1`) or over the extended SPI protocol pins (`SPI_CLK`, `SPI_MOSI`, `SPI_MISO`, `SPI_D2`, `SPI_D3`, `SPI_SEL1`), based on the `SPI_MMRDH.CMDPINS`, `SPI_MMRDH.ADRPINS`, and `SPI_CTL.MIOM` bit settings. SPI memory devices usually support communication in MSB-first mode. In dual mode, the SPI typically uses `SPI_MISO` as IO1 and `SPI_MOSI` as IO0. In quad mode, the SPI typically uses `SPI_D3` pin as IO3, `SPI_D2` as IO2, `SPI_MISO` as IO1, and `SPI_MOSI` as IO0.

When all I/O data pins are three-stated, the SPI continues clocking the SPI memory device, which drives out the data bits at the addressed location, until all bytes are received. The SPI hardware reads the data as configured by the `SPI_CTL.MIOM` bit setting. Upon reception of the last byte, the SPI typically deasserts `SPI_SEL1` to prepare for the next requested read header.

Application code must ensure that the opcode sent is consistent with multiple I/O programming and that the parameters specified in the memory-mapped read header register are consistent with flash read access timing.

The *SPI Memory-Mapped Register Operations Flow* diagram shows how the fields of the `SPI_MMRDH` register determine the read header while initiating transfers in memory-mapped mode.

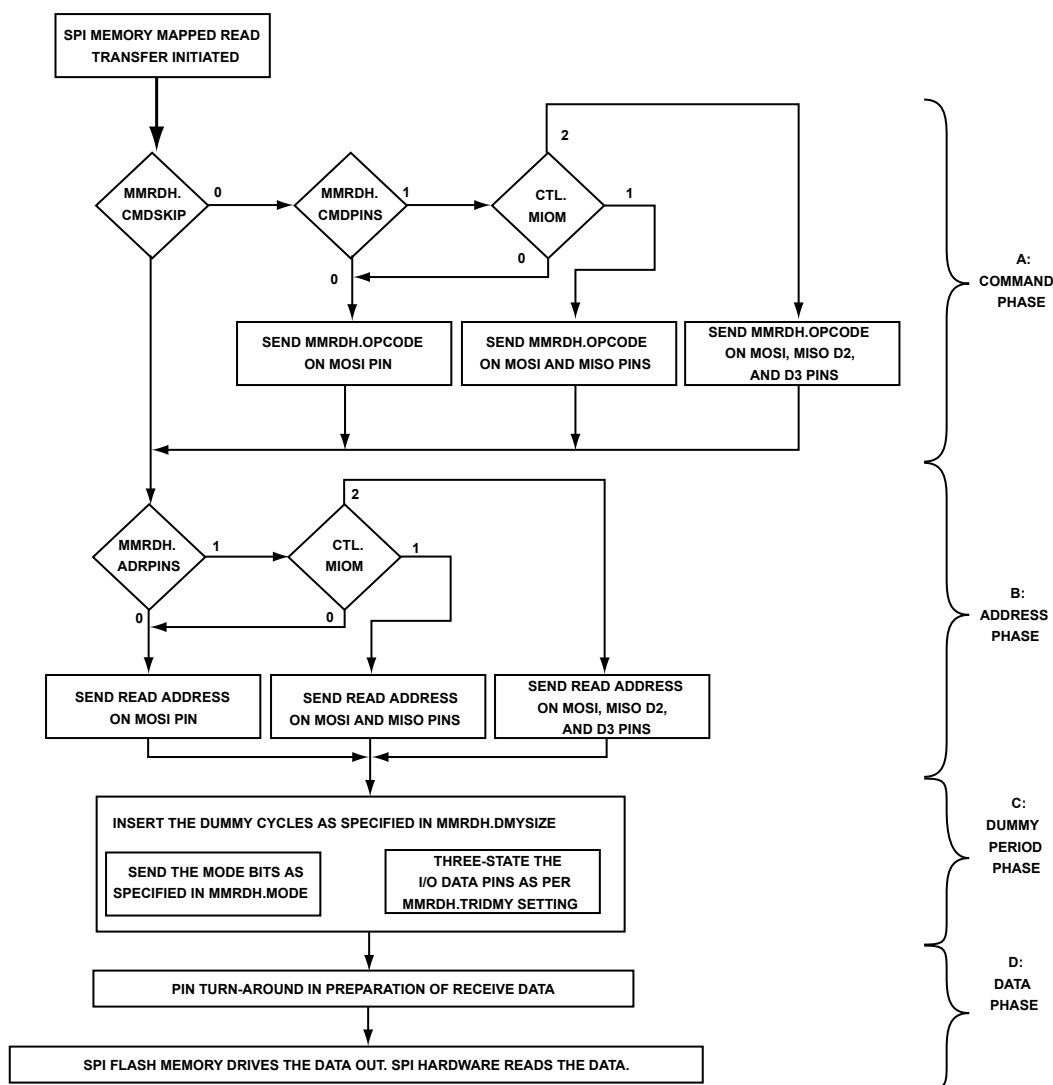


Figure 16-12: SPI Memory-Mapped Register Operations Flow

Memory-Mapped Architectural Concepts

In memory-mapped mode, the SPI accepts read requests through a dedicated on-chip slave interface. The SPI (if ready) accepts these requests and begins the process of assembling the read header based on access attributes described in both the `SPI_MMRDH` register and the internal bus request. After the read header transmission is complete, a pin turnaround period is timed and the receiver is enabled. The SPI continues clocking the SPI memory device until all bytes are received.

The SPI memory-mapped hardware accommodates various memory devices with different read timing. The capabilities include extra mode bits, flexible dummy period timing, and three-state control, as configured in the `SPI_MMRDH` register.

The *Memory-Mapped Protocol* figure shows the protocol for the SPI controller in memory-mapped mode.

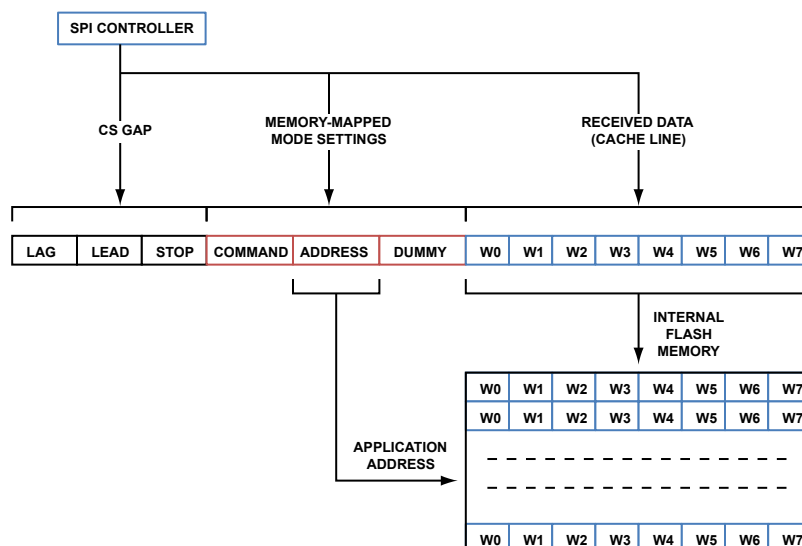


Figure 16-13: Memory-Mapped Protocol

As shown in the figure, the COMMAND field (`SPI_MMRDH.OPCODE`) is transmitted upon assertion of the `SPI_SEL[n]` signal. The SPI memory interprets this 8-bit value as a read command. Any 8-bit read opcode whose timing is compliant with the processor SPI and features provided by memory-mapped hardware is allowed, the most common being:

- Standard Read (0x03)
- Fast Read (0x0B)
- Fast Read Dual Output (0x3B)
- Fast Read Dual I/O (0x6B)
- Fast Read Quad Output (0xBB)
- Fast Read Quad I/O (0xEB)

NOTE: The SPI hardware does not validate the content of the `SPI_MMRDH.OPCODE` field prior to transmitting.

DMYSIZE (Number of Dummy Bytes)

When operating at a high clock frequency in multi-IO modes, most flash devices require some dummy clocks after the address bits. These dummy clock cycles allow the internal circuits of the device extra time for setting up the initial address. These bits specify the number of bytes separating address transmission and read data return.

The number of dummy cycles required varies per manufacturer, the read command used, and the SPI access time. The SPI hardware allows dummy cycles to be programmed in bytes in the `SPI_MMRDH.DMYSIZE` field, the value of which is a function of the number of pins used to transmit the address (`SPI_MMRDH.ADRPINS`), as shown in the *Pins Used to Transmit the Address (ADRPINS)* table.

Table 16-7: Pins Used to Transmit the Address (ADRPINS)

SPI_MMRDH. DMYSIZE	Dummy clock cycles		
	(SPI_MMRDH.ADRPINS=0, SPI_CTL.MIOM=x) Dummy bytes elapse over 1-pin	(SPI_MMRDH.ADRPINS=1, SPI_CTL.MIOM=1) Dummy bytes elapse over 2- pins	(SPI_MMRDH.ADRPINS=1, MIOM=2) Dummy bytes elapse over 4- pins
000	0	0	0
001	8	4	2
010	16	8	4
011	24	12	6
100	32	16	8
101	40	20	10
110	48	24	12
111	56	28	14

This dummy clocking period allows the mode bits to be sent, the pins to be three-stated, and the pins to be turned around in preparation for the receive data.

Memory-Mapped Read Accesses

The SPI hardware supports the most commonly used read operations.

- Two standard SPI reads (read and read fast), which use the unidirectional SPI_MOSI and SPI_MISO pins in addition to SPI_SEL[n] and SPI_CLK
- Four extended SPI multiple I/O reads: dual output, quad output, dual I/O, and quad I/O reads

The *SPI Read Operations* table and *SPI Flash Fast Read Sequence* figures summarize the types of read operations. Program each read operation in a way that is compatible with the description given in the SPI flash data sheet.

Table 16-8: SPI Read Operations

Operation	Read Command (Opcode)	CMDPIN	ADRPIN	DMYSIZE	Three-state	Multiple I/O Mode	Data Pins
Read	0x03	1	1	Zero	No	No	1
Fast Read	0x0B	1	1	Non-Zero	Yes	No	1
Dual Output Read	0x3B	1	1	Non-Zero	Yes	Yes(IO0-1)	2
Quad Output Read	0x6B	1	1	Non-Zero	Yes	Yes(IO0-3)	4
Dual I/O Read	0xBB	1, 2	2	Non-Zero	Yes	Yes (IO0-1)	2
Quad I/O Read	0xEB	1, 4	4	Non-Zero	Yes	Yes (IO0-3)	4

Some memory devices also support word quad I/O read (0xE7) and octal quad I/O read (0xE3) operations. These operations require fewer dummy cycles than normal quad I/O read operations.

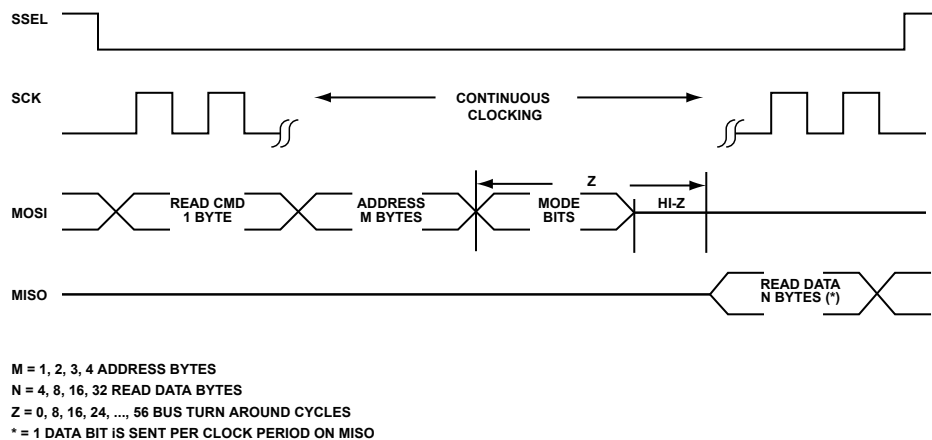


Figure 16-14: SPI Flash Fast Read Sequence

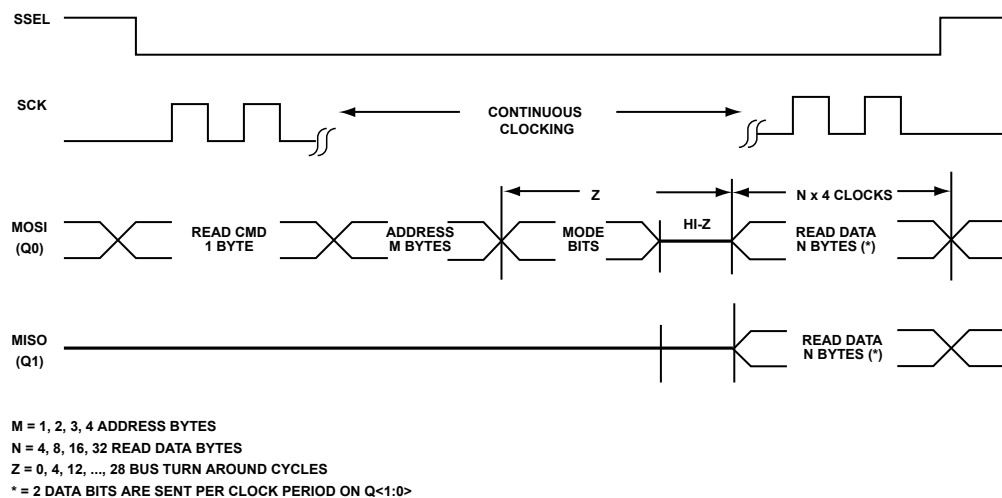


Figure 16-15: SPI Flash Fast Read (Dual Output) Sequence

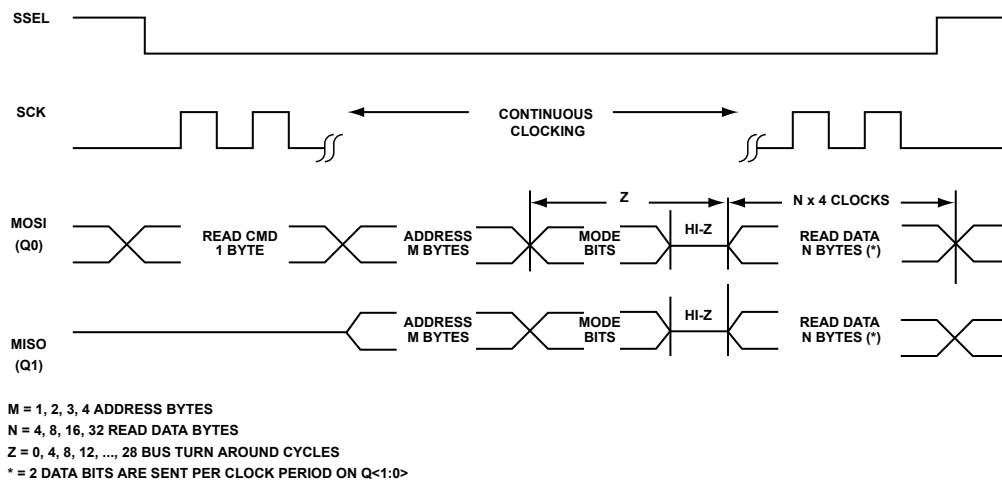


Figure 16-16: SPI Flash Fast Read (Dual I/O) Sequence

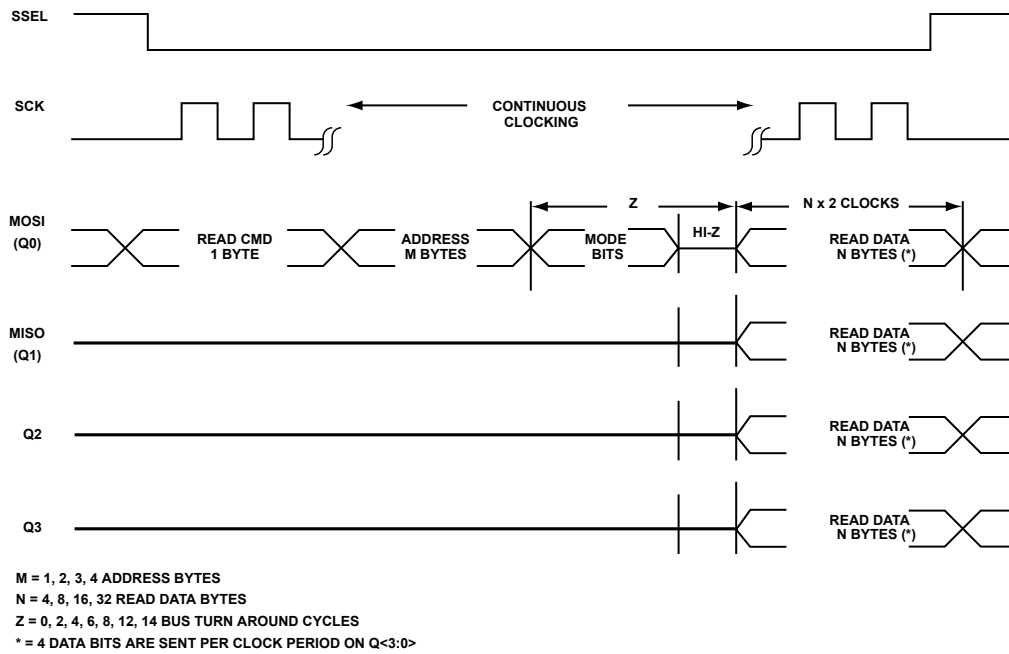


Figure 16-17: SPI Flash Quad Output Read Sequence

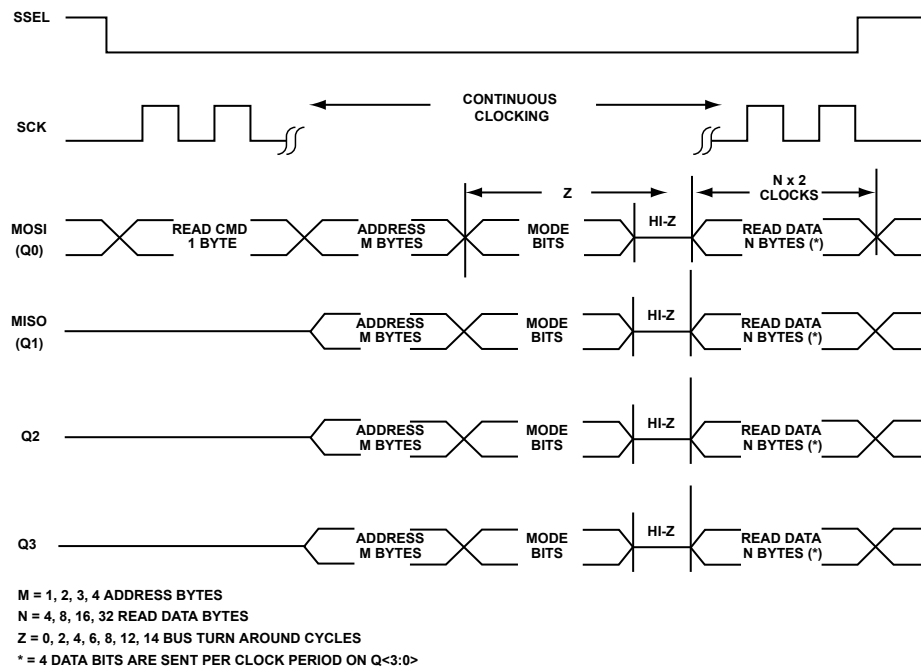


Figure 16-18: SPI Flash Quad I/O Read Sequence

SPI memory-mapped reads can be made cacheable in the core's internal memory by properly configuring the region as cacheable memory without bypass (see the related core's cache configuration documentation for details). In the figures, the number of read data bytes (N) is based on the following:

- For an instruction fetch by core (when in XIP mode); the number of instruction bytes to be fetched depends on the cache line size of the cache.

- For a data fetch by the core (data read), the number of data bytes to be fetched depends on the cache line size of the cache.

Although the minimum size of a memory-mapped data read transfer is 4 bytes, applications can fetch a single byte or a 2-byte data. (For example, it can fetch an unsigned char or short access in C code). In this case, only the required bytes are provided to the core and the other bytes are cached.

The on-chip memory subsystem master provides a starting address for the burst and the SPI hardware issues this address as part of the read header. The address provided is N-byte aligned. For example, to read the 30th byte from SPI memory, then the typical address to provide is:

- 28 (0x0000_001C) for a 32-bit cache line
- 24 (0x0000_0018) for a 64-bit cache line
- 16 (0x0000_0010) for a 128-bit cache line
- 0 (0x0000_0000) for a 256-bit cache line

The read data is returned to the memory subsystem in the order provided by the SPI memory. There can be considerable delay for the expected data provided to the master.

To minimize this delay, the wrap feature can be used where the memory subsystem provides the address of the critical word.

- For MDMA reads, the number of read data bytes (N) is always equal to 4 bytes. The MDMA read does not depend on the cache setting. For MDMA reads, limit the `DMA_CFG.MSIZE` field to 1, 2 or 4 bytes. The address provided by the memory subsystem master to the SPI hardware is always 4 byte-aligned.

Memory-Mapped High-Performance Features

In addition to automating the SPI memory read accesses, the memory-mapped hardware also provides some features to improve SPI memory fetches and increase the system performance. The following sections describe these features.

Merged Read Accesses

It is common for the memory subsystem to fetch two or more cache lines from consecutive addresses (the address sequencing is linear without any jumps). To take advantage of this situation, the SPI memory-mapped hardware provides a feature called merging. Enable merging by setting the `SPI_MMRDH.MERGE` bit.

When enabled, the hardware compares the address of an incoming read request to the address of a request the SPI memory is actively servicing. It can decide to merge two accesses when the address for the second access is incremental. For example, if the first address of a 32-byte cache line fetch is 0x0000_0000 and the second fetch is to address 0x0000_0020, then these two accesses can be merged. Merging increases efficiency and overall fetch bandwidth by eliminating the read header for those accesses which only require continuation of the SPI clock.

Wrap Around Accesses

Many SPI flash memory devices support wrapping which is used to enhance critical word fetching of cache lines. In this mode, the SPI device automatically wraps the read address to the base of a cache line once the end of the cache line is reached.

Wrap around accesses are enabled by setting the `SPI_MMRDH.WRAP` bit.

Some flash devices require programs to send a *Set Wrap* command to place the device in wrap mode. Other flash devices provide a configuration register which must be programmed to set the flash in wrap mode. Since the SPI memory-mapped hardware does not support any write operations to flash, perform this step in non-memory-mapped mode (`SPI_CTL.MMSE=0`) by accessing the SPI registers.

Data access is limited to 8-byte, 16-byte, or 32-byte sections of flash page in wrap mode. The Arm core uses the Wrap 4 access (64-bit data) for L1 cache. The Arm core uses Wrap 4 and Wrap 8 accesses (64-bit data) for L2 cache. The cores use the Wrap 8 accesses for unaligned accesses. During the read request to the SPI memory-mapped hardware, the memory subsystem master of the processor provides the address of a critical word instead of the line base. The read-data starts at the address specified in the instruction. Once it reaches the end boundary of the 8, 16, or 32-byte section, the output automatically wraps around to the beginning boundary to the line base address. The data fetch continues. It is not necessary to deassert the SPI `SPI_SEL[n]` signal or resend the read header to wrap to the cache line base when servicing misaligned cache fill requests.

The *Byte Sequence in Wrap Modes* table shows byte sequences in various wrap modes.

Table 16-9: Byte Sequence in Wrap Modes

Starting Address	8-Byte Wrap (cache_line = 8 byte)
0	0-1-2- . . . -6-7
1	1-2- 3-. . . -7-0
7	7-0-1- . . . -5-6
15	15-8-9- . . . -13-14
31	31-24-25-. . . -29-30

The burst with wrap feature allows applications to fetch a critical address quickly. Applications then fill the cache afterwards within a fixed length (8/16/32-byte) of data without issuing multiple read commands. Certain applications can benefit from this feature to improve cache fill efficiency and overall performance of system code execution.

NOTE: Do not use the merge and wrap feature together. Using wrap bursts can unintentionally disable merging (merging cannot occur for unaligned wrapping bursts). A wrap burst can start fetching data words in the middle of the cache line and cannot be merged with the next access.

Execute-In-Place (XIP, SPI2 only)

Execute-In-Place, most commonly known as XIP, allows software code to execute directly from an SPI flash device rather than downloading the code and executing it out of RAM. XIP, also known as Command Skip mode, is a general term and can be applied to fetching data as well.

There is a difference between XIP mode and standard mode. In XIP mode, after the SPI memory device is selected (`CS# =LOW`), the memory device does not decode the first input byte as command code. Instead, it expects the read header to directly start with address bytes. In standard mode, the memory decodes the first input byte it receives as a command code.

The XIP mode dramatically reduces random access time for applications that require fast code execution without shadowing the memory content on a RAM. The SPI memory-mapped hardware provides a control bit, `SPI_MMRDH.CMDSKIP` to skip the command from read header.

Some SPI memory devices require configuration of their control register to enable the XIP mode of operation, using the non-memory-mapped mode of the processor SPI. Typically, during the dummy cycle period, the mode bits are used to confirm the XIP operation and the `SPI_MMRDH.MODE` field must be set appropriately. A dummy memory-mapped access may be needed before setting the `SPI_MMRDH.CMDSKIP` bit in order to set the SPI memory device in Command Skip mode.

For more details about how to configure SPI memories into XIP mode, refer to the device data sheet.

NOTE: When configuring the flash to XIP mode from the SHARC+ core, ensure that the routine that configures flash to XIP is not routed through the L2CC. This is accomplished by first configuring the flash to XIP mode, then enabling the L2CC from the core.

Memory-Mapped Mode Error Status Bits

The SPI memory-mapped hardware provides bits in the `SPI_STAT` register to report errors. It provides these bits for notification only and their state has no effect on SPI operations. The status register bits are sticky. A W1C (write-1-to-clear) operation clears the bits.

- Memory-Mapped Write Error (`SPI_STAT.MMWE`). This bit is set (=1) if an attempt is made to write to address space that is reserved for memory-mapped SPI memory. The SPI memory-mapped hardware does not support automated write access to SPI memory space.
- Memory-Mapped Read Error (`SPI_STAT.MMRE`). This bit is set (=1) if an attempt is made to read address space reserved for memory-mapped SPI memory while memory mapping is disabled (`SPI_CTL.MMSE = 0`).
- Memory-Mapped Access Error (`SPI_STAT.MMAE`). This bit is set (=1) if an attempt is made to access either the TX or RX FIFO while memory-mapped access of SPI memory is enabled. In this case, attempts to communicate with the SPI device using legacy methods are blocked and receive fabric reports an error. Legacy methods include any direct access made to the TX and RX FIFOs, whether by DMA or processor MMR.
- Memory-Mapped Write Error Mask (`SPI_CTL.MMWEM`) bit specifies whether an error response is returned to the fabric on write attempts to address space that is reserved for memory-mapped SPI memory reads. Regardless of whether a write error response is masked using this bit, the memory-mapped write error (`SPI_STAT.MMWE`) sticky notification bit is still set.

NOTE: Unlike other bits in the `SPI_STAT` register, these memory-mapped mode error bits do not have associated bits in the SPI interrupt mask (`SPI_IMSK`) and SPI interrupt condition (`SPI_ILAT`) registers.

The memory-mapped top register (`SPI_MMTOP`) is used to specify the upper limit of the SPI memory address. The memory-mapped accesses to SPI memory addresses equal to or above this range are considered illegal. The accesses are blocked and a bus error response is generated.

This register is useful to block the invalid SPI memory address accesses. Some SPI memory vendors do not clearly specify (guarantee) that overrange address bits are ignored (address spaces can be wrapped).

Memory-Mapped Programming Guidelines

Setting the `SPI_CTL.MMSE` bit enables SPI memory-mapped mode. When enabled, the SPI interface is forced to be consistent with SPI memory requirements regardless the settings of certain control bits. The following tables specify typical settings for configuring the SPI in memory-mapped mode:

Table 16-10: SPI Control (SPI_CTL) Register

Bits	Typical values to set	Description	Comments
<code>SPI_CTL.MSTR</code>	1	Master mode enable	
<code>SPI_CTL.PSSE</code>	0	Protected slave select enable	
<code>SPI_CTL.ODM</code>	0	Open-drain mode enable	
<code>SPI_CTL.CPHASPI_CTL.CPOL</code>	0–0 or 1–1	SPI mode of communication	Flash dependent, usually SPI flash supports mode-0 (<code>CPHA=CPOL=0</code>) and mode-3 (<code>CPHA=CPOL=1</code>)
<code>SPI_CTL.ASSEL</code>	1	Hardware slave select pin control	
<code>SPI_CTL.SELST</code>	1	Slave select asserted between transfers	
<code>SPI_CTL.EMISO</code>	1	MISO pin enable	
<code>SPI_CTL.SIZE</code>	2	32-bit transfer size	
<code>SPI_CTL.LSBF</code>	0	MSB bit first mode	Flash dependent, usually SPI flash communicates in MSB bit first mode
<code>SPI_CTL.FCEN</code> <code>SPI_CTL.FCCH</code> <code>SPI_CTL.FCPL</code> <code>SPI_CTL.FCWM</code>	0	Hardware flow control related bits	
<code>SPI_CTL.FMODE</code>	1	Fast mode enable	Typically set to 1 for full cycle timing, 0 only works at low speed
<code>SPI_CTL.SOSI</code>	0	Treat <code>SPI_MOSI</code> pin as IO0 pin.	

Table 16-11: SPI Receive Control Register

Bits	Typical values to set	Description
<code>SPI_RXCTL.REN</code>	1	Receive channel enable
<code>SPI_RXCTL.RTI</code>	0	Receive transfer initiation disable
<code>SPI_RXCTL.RWCEN</code>	0	Receive word counter disable

Table 16-11: SPI Receive Control Register (Continued)

Bits	Typical values to set	Description
SPI_RXCTL.RDR	0	Receive data request disable
SPI_RXCTL.RDO	0	Discard incoming data if RFIFO is full
SPI_RXCTL.RRWM	0	Receive FIFO regular watermark
SPI_RXCTL.RUWM	0	Receive FIFO urgent watermark disable

Table 16-12: SPI Transmit Control Register

Bits	Typical values to set	Description
SPI_TXCTL.TEN	1	Transmit channel enable
SPI_TXCTL.TTI	1	Transmit transfer initiation disable
SPI_TXCTL.TWCEN	0	Transmit word counter disable
SPI_TXCTL.TDR	0	Transmit data request disable
SPI_TXCTL.TDU	0	Send last word when TFIFO is empty
SPI_TXCTL.TRWM	0	Transmit FIFO regular watermark
SPI_TXCTL.TUWM	0	Transmit FIFO urgent watermark disable

Table 16-13: SPI DLY Control Register

Bits	Typical values to set	Description	Comments See Flash data sheet for CS (for example, SSEL) timing specs
SPI_DLY.LAGX	1	Extended lag timing	
SPI_DLY.LEADX	1	Extended lead timing	
SPI_DLY.STOP	3	Stop bit between the transfers	Can be set to 1 at lower SPI clock frequencies.

The multiple I/O mode (SPI_CTL.MIOM) bits are partially ignored:

- The command (opcode) is transmitted using either just one or the number of pins specified by the SPI_CTL.MIOM bits, depending on SPI_MMRDH.CMDPINS bit setting.
- The address is then transmitted using either just one or the number of pins specified by the SPI_CTL.MIOM bits, depending on SPI_MMRDH.ADRPINS bit setting.
- The data is always read with the number of pins specified by the SPI_CTL.MIOM bits.

NOTE: Set the SPI module enable bits SPI_CTL.EN last after configuring all registers.

Use the following programming guidelines for memory-mapped mode:

- The SPI memory-mapped hardware does not check the flash status before initiating the access. It assumes that SPI memory is always able to respond to a read access. Before enabling memory-mapped mode (for example, setting the `SPI_CTL.MMSE` bit) ensure that SPI flash is ready for a read access. When using non-memory-mapped mode, a write-complete status can be examined prior to enabling the SPI in memory-mapped mode. (See the write in progress bit in the SPI flash memory status register.) Also, immediately after initial power-up, SPI memory devices can be inaccessible for a vendor-specified period.
- When SPI is enabled in memory-mapped mode, attempts to communicate with the SPI device using legacy methods are blocked. Legacy methods include any direct access made to the transmit or receive FIFOs, whether initiated by DMA or by a processor MMR access.
- To use some of the features offered by SPI memory devices, programs can first configure the SPI memory device by setting its control word or sending some commands. Since SPI memory-mapped hardware does not allow any type of SPI write operations, configure the SPI in non-memory-mapped mode prior to enabling memory-mapped mode.
- The memory-mapped hardware does not interpret the opcode. It does not check the validity of the timing that is specified in the `SPI_MMRDH` register for a particular opcode. Programs must set the fields of the `SPI_MMRDH` register to be consistent with the read-type selected.
- When the core requests the data or code fetch, the memory-mapped transfer depends on cache settings. The cache configuration register in the SPI memory device must be appropriately configured before enabling memory-mapped mode. Some of the high performance modes like merge, wrap, and transfer size depend on cache parameters.
- SPI memory-mapped MDMA reads do not support wrapping. For MDMA reads, limit the `DMA_CFG.MSIZE` field to 1 byte, 2 bytes or 4 bytes.
- There is not always tool support to change the SPI memory-mapped hardware setting or cache settings on-the-fly. Changing these settings can optimize the performance of code that accesses SPI memory in memory-mapped mode. It is expected that the SPI memory, SPI peripheral, and cache are programmed to one specific set of control settings for the whole application. Profiling or benchmarking of the actual application can be done to find the setting that works best.

SPI Interrupt Signals

The SPI controller supports three types of interrupt request signals that correspond to data, status, and error conditions.

Data Interrupts

The SPI peripheral supports two data interrupt channels – receive and transmit. These interrupt signals are multiplexed into the DMA request lines. Since the peripheral interfaces with separate read and write interfaces with DMA, the read and write data interrupts are independent. When the DMA channels are not used, the interrupts are routed directly to the system event controller. The interrupts occupy the same vector locations as the corresponding DMA channels.

Each of the data interrupt requests can be individually controlled. Program the `SPI_RXCTL.RDR` and `SPI_TXCTL.TDR` bit fields for receive and transmit, respectively. When receive is enabled, the RX interrupt request is issued whenever there is data available in the receive datapath for reading. (The event occurs according to the `SPI_RXCTL.RDR` bit setting.) When transmit is enabled, the TX interrupt request is issued whenever the transmit datapath can be written. (The event occurs according to the `SPI_TXCTL.TDR` setting.) DMA data interrupts are compatible with second-generation DMA to incorporate urgent data requests and transfer finish interrupt requests apart from the usual data request interrupts. Transmit interrupt requests operate independently from the word counter-value in the `SPI_TWC` register.

Status Interrupts

The SPI controller supports several status interrupt requests to indicate different conditions of the receiver and transmitter. All status interrupt requests can be masked. Status interrupt requests are signaled directly through a single SPI status IRQ line. The line cannot be combined with the SPI error IRQ line for some processors. The *SPI Status Interrupts* table describes the status interrupt requests that are available for the SPI controller.

Table 16-14: SPI Status Interrupts

SPI_STAT Bit	Description
<code>SPI_STAT.RUWM</code>	Receive FIFO urgent watermark interrupt request. Issued when the level of the RFIFO breaches the watermark set in the <code>SPI_RXCTL.RUWM</code> field. It is cleared when the level of the RFIFO reaches the watermark set in the <code>SPI_RXCTL.RRWM</code> field. If the RX channel is configured in DMA mode, <code>SPI_RXCTL.RUWM</code> is multiplexed with the data request.
<code>SPI_STAT.TUWM</code>	Transmit FIFO urgent watermark interrupt request. Issued when the level of the TFIFO breaches the watermark set using the <code>SPI_TXCTL.TUWM</code> bit. It is cleared when the level of the TFIFO reaches the watermark set in the <code>SPI_TXCTL.TRWM</code> field. If the TX channel is configured in DMA mode, <code>SPI_STAT.TUWM</code> is multiplexed with the data request.
<code>SPI_STAT.TS</code>	Transmit start interrupt request. Issued when the start of a transmit burst is detected by loading of the <code>SPI_TWC</code> register with the contents of the <code>SPI_TWCR</code> register.
<code>SPI_STAT.RS</code>	Receive start interrupt request. Issued when the start of a receive burst is detected by the loading of <code>SPI_RWC</code> with the contents of <code>SPI_RWCR</code> .
<code>SPI_STAT.TF</code>	Transmit finish interrupt request. Issued when a transmit burst completes (<code>SPI_TWC</code> decrements to zero).
<code>SPI_STAT.RF</code>	Receive finish interrupt request. Issued when a receive burst completes (<code>SPI_RWC</code> decrements to zero).

Error Conditions

The SPI controller supports interrupt requests upon several different error conditions. All interrupt requests are maskable. The individual error indications combine into a single SPI error IRQ signal, which can be multiplexed on some processors with the aggregated SPI status IRQ signal. The *SPI Error Interrupts* table details the possible error indications.

Error conditions arise depending on which of the channels (transmit or receive) are enabled. If a channel is disabled, all errors related to it are ignored. When both channels are enabled, errors from both channels are enabled.

Table 16-15: SPI Error Interrupts

Bit	Description
<code>SPI_STAT.MF</code>	Mode fault. Signaled when another device also tries to be a master in a multi-master system and drives the <code>SPI_SS</code> input low. This error is signaled in master mode operation.
<code>SPI_STAT.TUR</code>	Transmission error. Signaled when an underflow condition occurs on the transmit channel. This event occurs when a new transfer starts but <code>SPI_TFIFO</code> is empty. This error does not occur in master transmit initiating mode since <code>SPI_TFIFO Not Empty</code> is one of the conditions for transfer initiation.
<code>SPI_STAT.ROR</code>	Reception error. Signaled when an overflow condition occurs on the receive channel. This event occurs when a new data word is received, but the <code>SPI_RFIFO</code> is full. This error condition does not occur in master receive initiating mode since <code>SPI_RFIFO Not Full</code> is one of the conditions for transfer initiation.
<code>SPI_STAT.TC</code>	Transmit collision error. Signaled when loading data to the transmit shift register happens near the first transmitting edge of <code>SPI_CLK</code> . In slave mode of operation, the SPI controller is unaware of when the next transfer starts. Loading of data to the transmit shift register can happen just after the transmitting edge. This event results in the setup time not being met for the first bit transmitted. The transmitted data is corrupt. In <code>SPI_CTL.CPHA 1</code> mode, the first <code>SPI_CLK</code> edge is taken as the first transmitting edge. If <code>SPI_CTL.CPHA = 0</code> , then the last <code>SPI_CLK</code> edge of the last transmission (<code>SPI_CTL.SELST = 1</code>) or slave select deassertion (<code>SPI_CTL.SELST = 0</code>) is taken as the first transmitting edge. This error is signaled only in the slave mode of operation. In master mode of operation, loading of data happens before the first transmitting edge of <code>SPI_CLK</code> .

SPI Programming Concepts

The following sections provide general programming guidelines and procedures.

Programming Guidelines

It is acceptable to program `SPI_RXCTL` and `SPI_TXCTL` registers after programming the `SPI_CTL` register. However, program the initiating mode register and its counter-register, if enabled, after the non-initiating mode register. For example, if transmit is the initiating mode and receive is the non-initiating mode, then program the `SPI_RXCTL` and `SPI_RWC` registers before the `SPI_TXCTL` and `SPI_TWC` registers. If enabling both transmit and receive in initiating mode, enable the `SPI_CTL` register after programming both the `SPI_RXCTL` and `SPI_TXCTL` registers.

These programming guidelines prevent SPI from starting a transfer when SPI registers are not fully programmed. Other ways of programming are also allowed as long as the initiating conditions prevent the start of communication until after programming of SPI registers is complete.

Avoid data corruption when changing the SPI module configuration. Do not change the configuration during a data transfer. Additionally, change the clock polarity only when no slave is selected. However, an exception to this rule exists. When an SPI communication link consists of a single master and slave, `SPI_CTL.ASSEL = 0`. The slave select input of the slave is permanently tied low. In this case, the slave is always selected. Avoid data corruption by enabling the slave only after both the master and slave devices are configured.

The module supports 8, 16-bit and 32-bit word sizes. To ensure correct operation, configure both the master and slave with the same word size.

Master Operation in Non-DMA Modes

This section describes the operation of the SPI as a master in non-DMA mode.

1. Write to the `SPI_SLVSEL` register, setting one or more of the SPI select enable bits. This operation ensures that the desired slaves are properly deselected while the master is configured.
2. The `SPI_RXCTL.RTI` and `SPI_TXCTL.TTI` bits determine the SPI initiating mode. The initiating mode defines the primary transfer channel, and also the initiating condition for the transfer.
3. Write to the `SPI_CLK`, `SPI_CTL`, `SPI_RXCTL`, and `SPI_TXCTL` registers. This operation enables the device as a master and configures the SPI system. It specifies the transfer modes and channels, appropriate word length, transfer format, baud rate, and other control information.

ADDITIONAL INFORMATION: If `SPI_RXCTL.RTI` is enabled and `SPI_TXCTL.TTI` is not, write to the `SPI_RXCTL` register after writing into `SPI_CTL`, `SPI_TXCTL`, and `SPI_TFIFO` registers to prevent a transmit underrun for the first transfer.

4. If `SPI_CTL.ASSEL=0`, activate the desired slaves by clearing one or more of the `SPI_SLVSEL` flag bits. Otherwise, the SPI hardware performs slave activation.
5. The SPI controller then generates the programmed clock pulses on `SPI_CLK` and simultaneously shifts data out of `SPI_MOSI` while shifting data in from `SPI_MISO`. Before a shift, the shift register is loaded with the contents of the `SPI_TFIFO` register. At the end of the transfer, the contents of the shift register are loaded into `SPI_RFIFO`.
6. Whenever the initiating conditions are satisfied, the SPI continues to send and receive words. If the transmit buffer remains empty or the receive buffer remains full, the device operates according to the states of the `SPI_TXCTL.TDU` and `SPI_RXCTL.RDO` bits.
7. It is possible to program a secondary channel in addition to the initiating channel. This feature allows usage of available channel resources for receives or transmits simultaneously with the initiating channel.

Slave Operation in Non-DMA Modes

When a device is enabled as a slave in a non-DMA mode, a transition of the `SPI_SS` select signal to the active state (low) triggers the the start of a transfer. Or, the first active edge of `SPI_CLK` triggers the start, depending on the state of `SPI_CTL.CPHA` bit. The interface operates in the following manner.

1. The core writes to the `SPI_CTL`, `SPI_RXCTL`, and `SPI_TXCTL` registers. The operation defines the mode of the serial link to be the same as the mode setup in the SPI master.
2. To prepare for the data transfer, the core writes data to be transmitted into `SPI_TFIFO`.
3. Once the `SPI_SS` falling edge is detected, the slave starts sending data on active `SPI_CLK` edges and sampling data on inactive `SPI_CLK` edges.
4. Reception or transmission continues until `SPI_SS` is released or until the slave has received the proper number of clock cycles.

5. The slave device continues to receive or transmit with each new falling edge transition on $\overline{\text{SPI_SS}}$ or active SPI_CLK edge. If the transmit buffer remains empty or the receive buffer remains full, the device operates according to the states of the SPI_TXCTL.TDU and SPI_RXCTL.RDO bits.

Configuring DMA Master Mode

The SPI interface supports a write DMA channel and a read DMA channel. It can use these functions individually or in a lock-step manner in duplex mode ($\text{SPI_TXCTL.TTI} = \text{SPI_RXCTL.RTI} = 1$).

1. Write to the appropriate DMA registers to enable the SPI DMA channel and to configure the necessary work units, access direction, word count, and so on.
2. Write to the [SPI_SLVSEL](#) register, setting one or more of the SPI flag select bits.
3. Write to the SPI_CLK and [SPI_CTL](#) registers, enabling the device as a master and configuring the SPI system by specifying the appropriate word length, transfer format, baud rate, and so forth.
4. Write to [SPI_RXCTL](#) to configure SPI master receive mode, or write to [SPI_TXCTL](#) to configure SPI master transmit mode.
5. Finally, write to the SPI_RXCTL.REN bit to enable the receive channel, or write to SPI_TXCTL.TEN to enable the transmit channel.
6. If the SPI_RXCTL.RTI bit is enabled, a receive transfer is initiated upon enabling SPI_CTL.EN bit. If the receive word counter is enabled (SPI_RXCTL.RWCEN), then the [SPI_RWC](#) register must be non-zero for a transfer to initiate.

ADDITIONAL INFORMATION: If enabling both receive and transmit DMA channels, but not enabling SPI_TXCTL.TTI , write to the [SPI_RXCTL](#) register after writing the [SPI_CTL](#) and [SPI_TXCTL](#) registers. In this way, a transmit underrun can be prevented for the first transfer. Subsequent transfers are initiated as the SPI reads data from the receive shift register and writes to the SPI receive FIFO. The SPI then requests a write from DMA to memory. Upon a DMA grant, the DMA engine reads a word from the SPI receive FIFO and writes to memory. New requests continue to be initiated as long as the receive FIFO does not fill up, when [SPI_RWC](#) does not become zero while $\text{SPI_RXCTL.RWCEN} = 1$.

7. If SPI_TXCTL.TTI is enabled, the SPI controller requests DMA reads from memory as long as there is space for more data in the transmit pipe. Upon a DMA grant, the DMA engine reads a word from memory and writes to the transmit FIFO. As long as transmit data is available in the FIFO, and the [SPI_TWC](#) register is non-zero when $\text{SPI_TXCTL.TWCEN} = 1$, the SPI continues to initiate transfers until disabled.
8. If both the SPI_TXCTL.TTI and SPI_RXCTL.RTI bits are enabled, the SPI controller requests a DMA read from memory. However, there must be space for more data in the transmit pipe and the number of words written into the SPI must be less than [SPI_TWC](#) if $\text{SPI_TXCTL.TWCEN} = 1$. Upon a DMA grant, the DMA engine reads a word from memory and writes to the transmit FIFO.

ADDITIONAL INFORMATION: As the SPI writes data from the transmit FIFO into the transmit shift register, it initiates a transfer on the SPI link. Data received from the transfer is moved from the SPI receive shift register to the receive FIFO. The SPI controller requests a write from DMA to memory. Upon a DMA grant,

the DMA engine reads a word from the receive FIFO and writes to memory. Transfer continues to be initiated as long as both receives and transmits can accommodate new data.

9. If the receive pipe fills up due to unavailability of DMA grants, the transmit pipe stalls until the pipe is drained. If the transmit pipe fills up, the SPI stops requesting for DMA writes. If the value in `SPI_RWC` expires, further write-requests to DMA stop. However, data already written into the transmit FIFO is sent, and read requests to DMA continue until the receive data is read from the receive FIFO.
10. The SPI then generates the programmed clock pulses on `SPI_CLK` and simultaneously shifts data out of `SPI_MOSI` while shifting data in from `SPI_MISO`. For receive transfers, the value in the shift register is loaded into the `SPI_RFIFO` register at the end of the transfer. For transmit transfers, the value in the `SPI_TFIFO` register is loaded into the shift register at the start of the transfer.

Configuring DMA Slave Mode Operation

This mode occurs when the SPI is enabled as a slave and the DMA engine is configured to transmit or receive data. A transition of the `SPI_SS` signal to the active-low state triggers the start of a transfer. Or, the first active edge of `SPI_CLK` triggers the start of a transfer, depending on the state of the `SPI_CTL.CPHA` bit. The following steps illustrate the SPI receive or transmit DMA sequence in an SPI slave (in response to a master command). The SPI supports a receive DMA channel and a transmit DMA channel.

1. Write to the appropriate DMA registers to enable the SPI DMA channel and configure the necessary work units, access direction, word count, and so on.
2. Write to the `SPI_CTL`, `SPI_RXCTL`, and `SPI_TXCTL` registers to define the mode of the serial link to be the same as the mode configured in the SPI master.
3. If the receive channel is enabled (`SPI_RXCTL.REN` is asserted), the following actions occur:
 - a. Once the slave select input is active, the slave starts receiving and transmitting data on active `SPI_CLK` edges.
 - b. The value in the shift register is loaded into the `SPI_RFIFO` register at the end of the transfer.
 - c. Once `SPI_RFIFO` has valid data, it requests a write from DMA to memory.
 - d. Upon a DMA grant, the DMA engine reads a word from the receive FIFO and writes to memory.
 - e. As long as there is data in the receive FIFO, the SPI slave continues to request a DMA write to memory. The DMA engine continues to read a word from the FIFO and writes to memory. The SPI slave continues receiving words on active `SPI_CLK` edges as long as the `SPI_SS` input is active.
 - f. If the data collected in the receive pipe breaches the set level, and the DMA engine cannot keep up with the receive rate, the slave can deassert the `SPI_RDY` signal. This signaling throttles the master. The receive pipe level is set according to the `SPI_CTL.FCWM` field. The signal is deasserted as the DMA drains the receive FIFO. Alternatively, the SPI can use the `SPI_RXCTL.RDO` bit to decide when the incoming data is discarded or overwritten into the receive FIFO (when `SPI_CTL.FCEN` is inactive).
4. If the transmit channel is enabled (`SPI_TXCTL.TEN` is asserted), the following actions occur:

- a. The SPI requests a DMA read from memory.
 - b. Upon a DMA grant, the DMA engine reads a word from memory and writes to the transmit FIFO.
 - c. The SPI then reads DMA data from the transmit FIFO and writes to the transmit shift register, awaiting the start of the next transfer.
 - d. Once the slave select input is active, the slave starts receiving and transmitting data on active `SPI_CLK` edges.
 - e. As long as there is room in the transmit FIFO, the SPI slave continues to request a DMA read from memory. The DMA engine continues to read a word from memory and write to the transmit FIFO. The SPI slave continues transmitting words on active `SPI_CLK` edges as long as the `SPI_SS` input is active.
 - f. If the number of outstanding data entries in the transmit pipe breaches the level set and the DMA cannot keep up with the transmit rate, the slave deasserts the `SPI_RDY` signal. This signaling throttles the master. The transmit pipe level is set according to the `SPI_CTL.FCWM` field. The signal is deasserted as the DMA fills the transmit FIFO. Alternately, the `SPI_TXCTL.TDU` bit decides the state of the transmit data (when `SPI_CTL.FCEN` is deasserted).
5. If both receive and transmit channels are enabled, the following actions occur after the actions for each channel. Transfers continue as long as both receive and transmit channels can accommodate new data.
- a. If the receive pipe fills up due to the unavailability of DMA grant, the SPI interface stalls the master by asserting the `SPI_RDY` pin. This signal is deasserted as the DMA drains the receive FIFO. Alternately, the SPI uses the `SPI_RXCTL.RDO` bit to decide when the incoming data is discarded or overwritten in the receive FIFO (when `SPI_CTL.FCEN` is deasserted).
 - b. If the transmit pipe fills up, the SPI stops requesting DMA writes until the pipe clears.
 - c. If there is an underflow problem in the transmit pipe, the slave stalls the master by deasserting `SPI_RDY` while the DMA fills the transmit FIFO. Alternately, the SPI uses the `SPI_TXCTL.TDU` bit to decide the state of the transmit data (when `SPI_CTL.FCEN` is deasserted).

ADSP-2159x SPI Register Descriptions

Serial Peripheral Interface (SPI) contains the following registers.

Table 16-16: ADSP-2159x SPI Register List

Name	Description
<code>SPI_CLK</code>	Clock Rate Register
<code>SPI_CTL</code>	Control Register
<code>SPI_DLY</code>	Delay Register
<code>SPI_ILAT</code>	Masked Interrupt Condition Register
<code>SPI_ILAT_CLR</code>	Masked Interrupt Clear Register

Table 16-16: ADSP-2159x SPI Register List (Continued)

Name	Description
SPI_IMSK	Interrupt Mask Register
SPI_IMSK_CLR	Interrupt Mask Clear Register
SPI_IMSK_SET	Interrupt Mask Set Register
SPI_MMRDH	Memory Mapped Read Header
SPI_MMTOP	SPI Memory Top Address
SPI_RFIFO	Receive FIFO Data Register
SPI_RWC	Received Word Count Register
SPI_RWCR	Received Word Count Reload Register
SPI_RXCTL	Receive Control Register
SPI_SLVSEL	Slave Select Register
SPI_STAT	Status Register
SPI_TFIFO	Transmit FIFO Data Register
SPI_TWC	Transmitted Word Count Register
SPI_TWCR	Transmitted Word Count Reload Register
SPI_TXCTL	Transmit Control Register

Clock Rate Register

The `SPI_CLK` register selects the baud rate for SPI data transfers, relating this rate to the SPI serial clock (SPI clock) and the system clock (CLKO6).

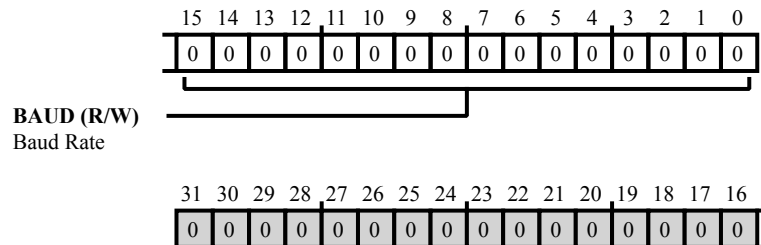


Figure 16-19: SPI_CLK Register Diagram

Table 16-17: SPI_CLK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	BAUD	Baud Rate. The <code>SPI_CLK.BAUD</code> bits set the SPI baud rate according to the formula: $BAUD = (CLKO6 / SPI\ Clock) - 1$

Control Register

The `SPI_CTL` register enables the SPI and configures settings for operating modes, communication protocols, and buffer operations.

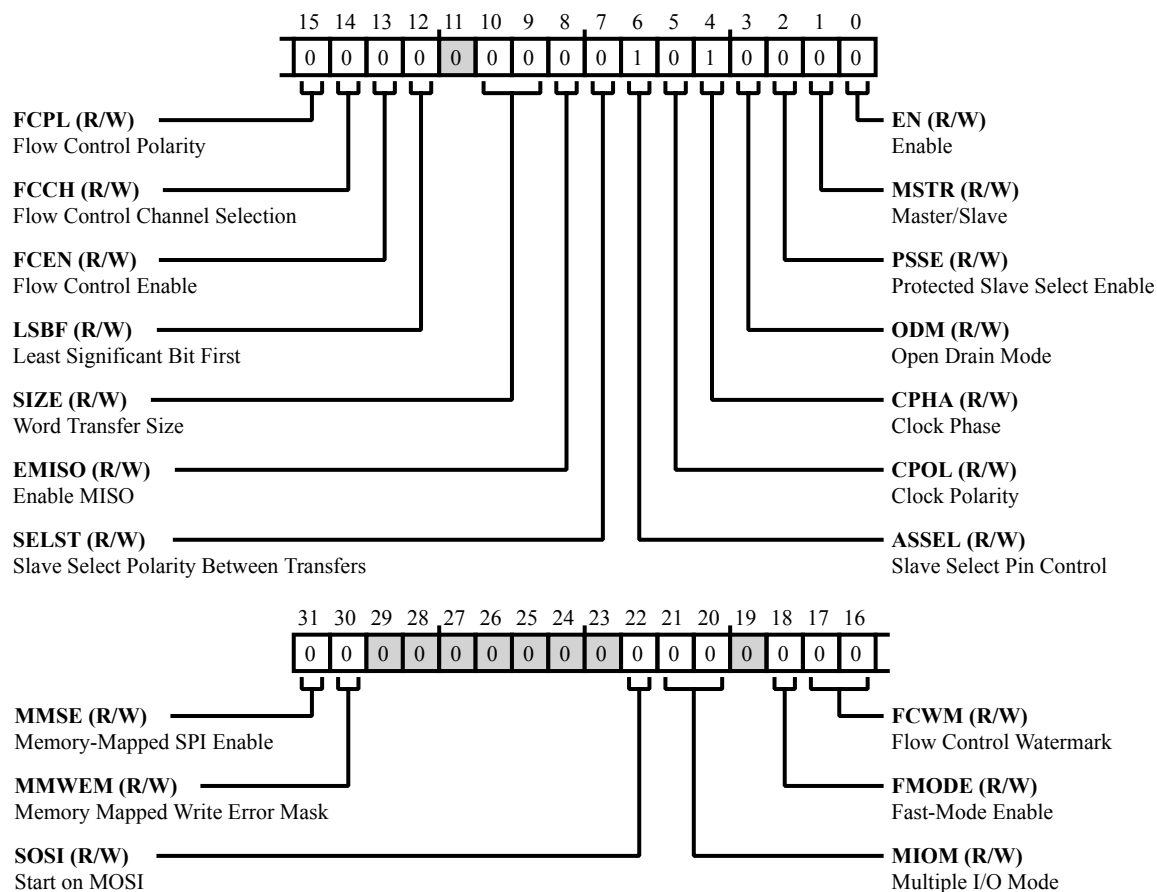


Figure 16-20: SPI_CTL Register Diagram

Table 16-18: SPI_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	MMSE	Memory-Mapped SPI Enable. When the <code>SPI_CTL.MMSE</code> bit is asserted, communication to an SPI memory device is automated such that the memory it contains is accessible directly through the read of processor address space assigned to it. (As far as the SPI peripheral is concerned, this includes all read accesses received by the SPI peripherals system crossbar slave port.) Note that when memory-mapped access of SPI memory is enabled, attempts to communicate with the SPI device using legacy methods are blocked and receive fabric error responses are generated. Legacy methods include any direct access made to the Tx and Rx FIFOs, whether initiated by DMA or processor MMR access.
		0 Hardware automated access of memory-mapped SPI memory disabled.
		1 Hardware-automated access of memory-mapped SPI memory enabled.
30 (R/W)	MMWEM	Memory Mapped Write Error Mask. The <code>SPI_CTL.MMWEM</code> bit specifies whether an error response is returned to the fabric upon write attempts to address space reserved for memory-mapped reads of SPI memory.
		0 Write error response returned upon write attempts to memory-mapped SPI memory
		1 Write error response masked (not returned) upon write attempts to memory-mapped SPI memory
22 (R/W)	SOSI	Start on MOSI. The <code>SPI_CTL.SOSI</code> bit is valid only when <code>SPI_CTL.MIOM</code> is enabled for either DIOM or QIOM, and this bit selects the starting pin and the bit placement on pins for these modes. In DIOM, by default, (<code>SPI_CTL.SOSI = 0</code>) SPI sends the first bit on the <code>SPI_MISO</code> pin and the second bit on the <code>SPI_MOSI</code> pin. In QIOM, by default, the SPI sends the first bit on the <code>SPI_D3</code> pin, the second bit on the <code>SPI_D2</code> pin, the third bit on the <code>SPI_MISO</code> pin and the fourth bit on the <code>SPI_MOSI</code> pin. This order can be reversed by setting the <code>SPI_CTL.SOSI</code> bit. When this bit is set, the SPI sends the first bit on the <code>SPI_MOSI</code> pin. The first bit referred to here depends on the <code>SPI_CTL.LSBF</code> bit setting (MSB bit or LSB bit).
		0 Start on MISO (DIOM) or start on SPI_D3
		1 Start on MOSI

Table 16-18: SPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	MIOM	Multiple I/O Mode. The <code>SPI_CTL.MIOM</code> bits enable SPI operation in dual I/O mode (DIOM) or quad I/O mode (QIOM). These bits can only be changed when the SPI is disabled (<code>SPI_CTL.EN = 0</code>).
		0 No MIOM (disabled)
		1 DIOM operation
		2 QIOM operation
		3 Reserved
18 (R/W)	FMODE	Fast-Mode Enable. The <code>SPI_CTL.FMODE</code> bit enables fast mode operation for SPI receive transfers. SPI transmit operations in fast mode are the same as normal mode.
		0 Disable
		1 Enable
17:16 (R/W)	FCWM	Flow Control Watermark. The <code>SPI_CTL.FCWM</code> bits select the watermark level of the transmit channel (<code>SPI_TFIFO</code> buffer) or receive channel (<code>SPI_RFIFO</code> buffer) that triggers flow control operation. These bits are applicable only when the SPI is a slave (<code>SPI_CTL.MSTR = 0</code>) and flow control is enabled (<code>SPI_CTL.FCEN = 1</code>). When the watermark condition is met, the SPI slave deasserts the <code>SPI_RDY</code> pin.
		0 TFIFO empty or RFIFO full
		1 TFIFO 75% or more empty, or RFIFO 75% or more full
		2 TFIFO 50% or more empty, or RFIFO 50% or more full
		3 Reserved
15 (R/W)	FCPL	Flow Control Polarity. The <code>SPI_CTL.FCPL</code> bit selects flow control polarity for the <code>SPI_RDY</code> pin when flow control is enabled. When the <code>SPI_RDY</code> pin is active, the SPI is indicating it is ready for data transfer.
		0 Active-low RDY
		1 Active-high RDY

Table 16-18: SPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	FCCH	Flow Control Channel Selection. The <code>SPI_CTL.FCCH</code> bit selects whether the SPI applies flow control to the transmit channel (<code>SPI_TFIFO</code> buffer) or receive channel (<code>SPI_RFIFO</code> buffer). This bit is applicable only when the SPI is a slave and flow control is enabled.
		0 Flow control on RX buffer
		1 Flow control on TX buffer
13 (R/W)	FCEN	Flow Control Enable. The <code>SPI_CTL.FCEN</code> bit enables SPI flow control operation, which permits slow slave devices to interface with fast master devices. This bit controls the operation of the <code>SPI_RDY</code> pin. Note that options for flow control operation are available using the <code>SPI_CTL.FCCH</code> , <code>SPI_CTL.FCPL</code> , and <code>SPI_CTL.FCWM</code> bits.
		0 Disable
		1 Enable
12 (R/W)	LSBF	Least Significant Bit First. The <code>SPI_CTL.LSBF</code> bit selects whether the SPI transmits/receives data as LSB first (little endian) or MSB first (big endian). This bit can only be changed when the SPI is disabled.
		0 MSB sent/received first (big endian)
		1 LSB sent/received first (little endian)
10:9 (R/W)	SIZE	Word Transfer Size. The <code>SPI_CTL.SIZE</code> bits select the SPI transfer word size as 8, 16 or 32 bits. To ensure correct operation, both the master and slave must be configured with the same word size. This bit can only be changed when the SPI is disabled (<code>SPI_CTL.EN = 0</code>).
		0 8-bit word
		1 16-bit word
		2 32-bit word
		3 Reserved
8 (R/W)	EMISO	Enable MISO. The <code>SPI_CTL.EMISO</code> bit enables master-in-slave-out (MISO) mode. This SPI mode is applicable only when the SPI is a slave.
		0 Disable
		1 Enable

Table 16-18: SPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	SELST	Slave Select Polarity Between Transfers. The SPI_CTL.SELST bit selects the state (polarity) for the $\overline{\text{SPI_SEL[n]}}$ pin between SPI transfers when the SPI is a master and hardware slave select assertion is enabled (SPI_CTL.ASSEL = 1). In slave mode, this bit affects the detection of both transmit collision (SPI_STAT.TC and underrun (SPI_STAT.TUR) errors.
		0 Deassert slave select (high)
		1 Assert slave select (low)
6 (R/W)	ASSEL	Slave Select Pin Control. The SPI_CTL.ASSEL bit selects whether the SPI hardware sets the $\overline{\text{SPI_SEL[n]}}$ pin output value (ignoring the slave select SPI_SLVSEL.SSEL1 - SPI_SLVSEL.SSEL7 bits) or whether software control of the slave select bits set the $\overline{\text{SPI_SEL[n]}}$ pin output value. This feature is applicable only when the SPI is a master. When hardware control is enabled, the $\overline{\text{SPI_SEL[n]}}$ pin output is asserted during the transfers, and the pin polarity between transfers is selected by the SPI_CTL.SELST bit. When software control is enabled, the $\overline{\text{SPI_SEL[n]}}$ pin output value is set through software control of the slave select bits, and as such, the pin may either remain asserted (low) or be deasserted between transfers.
		0 Software slave select control
		1 Hardware slave select control
5 (R/W)	CPOL	Clock Polarity. The SPI_CTL.CPOL bit selects whether the SPI uses an active-low or active-high signal for the SPI clock (SPI_CLK). This bit works with the SPI_CTL.CPHA bit to select combinations of clock phase and polarity for the SPI_CLK pin. This bit can only be changed when the SPI is disabled.
		0 Active-high SPI CLK
		1 Active-low SPI CLK
4 (R/W)	CPHA	Clock Phase. The SPI_CTL.CPHA bit selects whether the SPI starts toggling the signal for the SPI clock (SPI_CLK) from the start of the first data bit or from the middle of the first data bit. The SPI_CTL.CPHA bit works with the SPI_CTL.CPOL bit to select combinations of clock phase and polarity for the SPI_CLK pin. This bit can only be changed when the SPI is disabled.
		0 SPI CLK toggles from middle
		1 SPI CLK toggles from start

Table 16-18: SPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	ODM	Open Drain Mode. The SPI_CTL.ODM bit configures the data output pins (SPI_MOSI and SPI_MISO) to behave as open drain outputs, which prevents contention and possible damage to pin drivers in multi-master or multi-slave SPI systems. When SPI_CTL.ODM is enabled and the SPI is a master, the SPI three-states the SPI_MOSI pin when the data driven out on MOSI is a logic-high. The SPI does not three-state the SPI_MOSI pin when the driven data is a logic-low. When SPI_CTL.ODM is enabled and the SPI is a slave, the SPI three-states the SPI_MISO pin when the data driven out on SPI_MISO is a logic-high. Note that an external pull-up resistor is required on both the SPI_MOSI and SPI_MISO pins when SPI_CTL.ODM is enabled.
		0 Disable
		1 Enable
2 (R/W)	PSSE	Protected Slave Select Enable. The SPI_CTL.PSSE bit enables the <u>SPI_SS</u> pin to provide error detection input in a multi-master environment when the SPI is in master mode. If some other device in the system asserts the <u>SPI_SS</u> pin while SPI is enabled as master (and SPI_CTL.PSSE is enabled), this condition causes a mode fault error.
		0 Disable
		1 Enable
1 (R/W)	MSTR	Master/Slave. The SPI_CTL.MSTR bit toggles the SPI between master mode and slave mode. This bit can only be changed when the SPI is disabled.
		0 Slave
		1 Master
0 (R/W)	EN	Enable. The SPI_CTL.EN bit enables SPI operation.
		0 Disable SPI module
		1 Enable

Delay Register

The `SPI_DLY` register selects a transfer delay and the lead/lag timing between slave select signals and SPI clock edge assertion/deassertion.

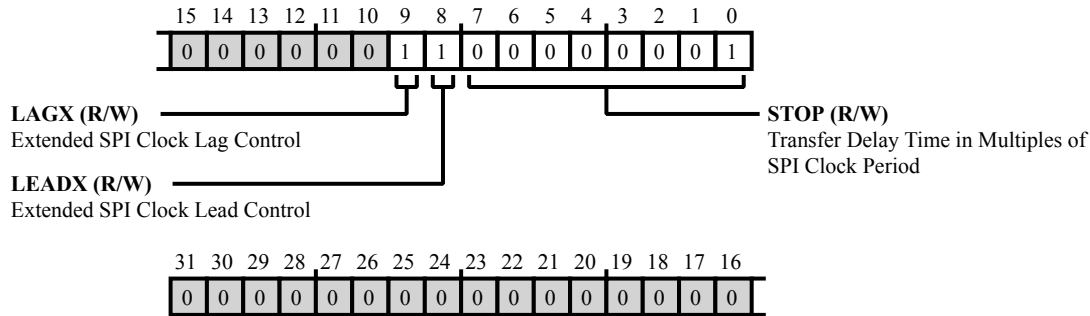


Figure 16-21: SPI_DLY Register Diagram

Table 16-19: SPI_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	LAGX	Extended SPI Clock Lag Control. The <code>SPI_DLY.LAGX</code> bit enables insertion of a 1-SPI_CLK cycle lag (extend lag) in the timing between the slave select (<code>SPI_SEL[n]</code>) assertion and first SPI clock edge.
		0 Disable
		1 Enable
8 (R/W)	LEADX	Extended SPI Clock Lead Control. The <code>SPI_DLY.LEADX</code> bit enables insertion of a 1-SPI_CLK cycle lead (extend lead) in the timing between the slave select (<code>SPI_SEL[n]</code>) deassertion and last SPI clock edge.
		0 Disable
		1 Enable
7:0 (R/W)	STOP	Transfer Delay Time in Multiples of SPI Clock Period. The <code>SPI_DLY.STOP</code> bits select a delay (number of stop bits in multiples of SPI clock duration) at the end of each SPI transfer. The default delay is the minimum value required to comply with the SPI protocol (1-bit duration). The <code>SPI_DLY.STOP</code> bits can be programmed with smaller delay values, resulting in continuous operation (for example, stop bits =0).

Masked Interrupt Condition Register

The `SPI_ILAT` register latches interrupts, queuing the interrupt requests for service. When a condition is indicated by a bit in the `SPI_STAT` register and the corresponding interrupt request is unmasked in `SPI_IMSK`, the SPI latches the interrupt request bit in `SPI_ILAT`.

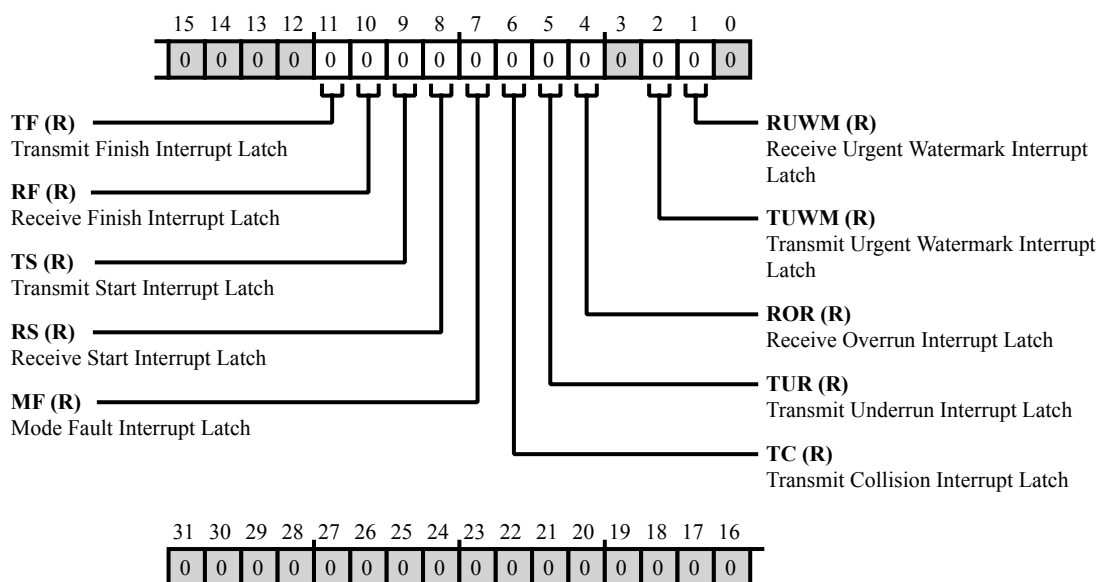


Figure 16-22: SPI_ILAT Register Diagram

Table 16-20: SPI_ILAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	TF	Transmit Finish Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
10 (R/NW)	RF	Receive Finish Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
9 (R/NW)	TS	Transmit Start Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
8 (R/NW)	RS	Receive Start Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request

Table 16-20: SPI_ILAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	MF	Mode Fault Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
6 (R/NW)	TC	Transmit Collision Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
5 (R/NW)	TUR	Transmit Underrun Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
4 (R/NW)	ROR	Receive Overrun Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
2 (R/NW)	TUWM	Transmit Urgent Watermark Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request
1 (R/NW)	RUWM	Receive Urgent Watermark Interrupt Latch.
		0 No interrupt request
		1 Latched interrupt request

Masked Interrupt Clear Register

The `SPI_ILAT_CLR` register permits clearing individual mask bits in the `SPI_ILAT` register without affecting other bits in the register. Use write-1-to-clear on a bit in the `SPI_ILAT_CLR` register to clear the corresponding bit in the `SPI_ILAT` register.

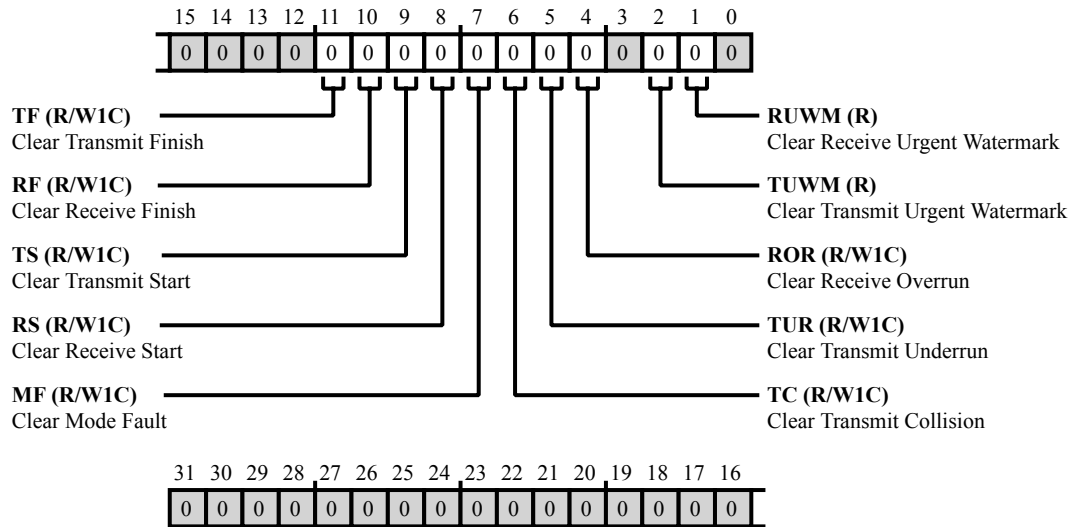


Figure 16-23: `SPI_ILAT_CLR` Register Diagram

Table 16-21: `SPI_ILAT_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W1C)	TF	Clear Transmit Finish. The <code>SPI_ILAT_CLR.TF</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
10 (R/W1C)	RF	Clear Receive Finish. The <code>SPI_ILAT_CLR.RF</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
9 (R/W1C)	TS	Clear Transmit Start. The <code>SPI_ILAT_CLR.TS</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit

Table 16-21: SPI_ILAT_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	RS	Clear Receive Start. The <code>SPI_ILAT_CLR.RS</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
7 (R/W1C)	MF	Clear Mode Fault. The <code>SPI_ILAT_CLR.MF</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
6 (R/W1C)	TC	Clear Transmit Collision. The <code>SPI_ILAT_CLR.TC</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
5 (R/W1C)	TUR	Clear Transmit Underrun. The <code>SPI_ILAT_CLR.TUR</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
4 (R/W1C)	ROR	Clear Receive Overrun. The <code>SPI_ILAT_CLR.ROR</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit
2 (R/NW)	TUWM	Clear Transmit Urgent Watermark. The <code>SPI_ILAT_CLR.TUWM</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit

Table 16-21: SPI_ILAT_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	RUWM	Clear Receive Urgent Watermark. The <code>SPI_ILAT_CLR.RUWM</code> bit clears the corresponding mask bit in the <code>SPI_ILAT</code> register.
		0 No effect
		1 Clear mask bit

Interrupt Mask Register

The `SPI_IMSK` register unmask (enables) or mask (disables) SPI interrupt requests. When a condition is indicated by a bit in the `SPI_STAT` register and the corresponding interrupt request is unmasked in `SPI_IMSK`, the SPI latches the interrupt request bit in the `SPI_ILAT` register, queuing the interrupt request for service.

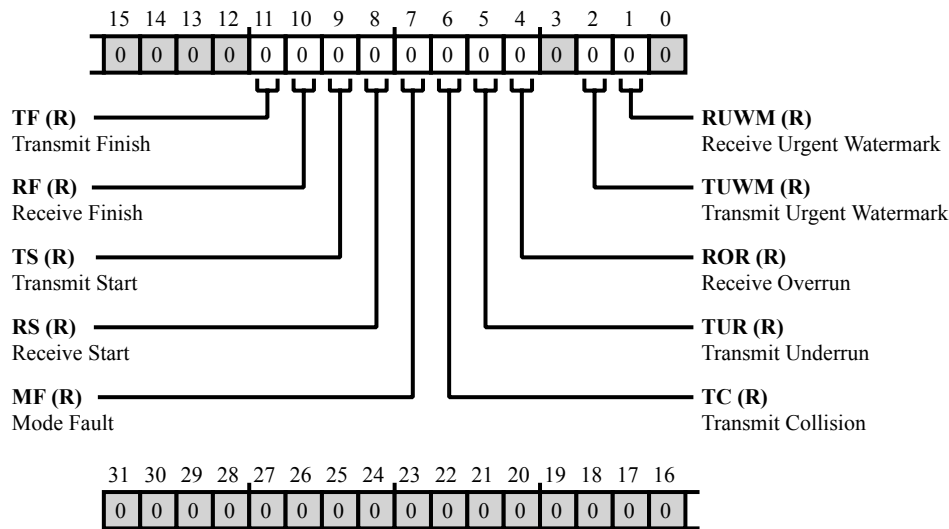


Figure 16-24: SPI_IMSK Register Diagram

Table 16-22: SPI_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	TF	Transmit Finish. The <code>SPI_IMSK.TF</code> bit unmask (enables) or mask (disables) the TF interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
10 (R/NW)	RF	Receive Finish. The <code>SPI_IMSK.RF</code> bit unmask (enables) or mask (disables) the RF interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
9 (R/NW)	TS	Transmit Start. The <code>SPI_IMSK.TS</code> bit unmask (enables) or mask (disables) the TS interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request

Table 16-22: SPI_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	RS	Receive Start. The <code>SPI_IMSK.RS</code> bit unmask (enables) or masks (disables) the RS interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
7 (R/NW)	MF	Mode Fault. The <code>SPI_IMSK.MF</code> bit unmask (enables) or masks (disables) the MF interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
6 (R/NW)	TC	Transmit Collision. The <code>SPI_IMSK.TC</code> bit unmask (enables) or masks (disables) the TC interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
5 (R/NW)	TUR	Transmit Underrun. The <code>SPI_IMSK.TUR</code> bit unmask (enables) or masks (disables) the TUR interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
4 (R/NW)	ROR	Receive Overrun. The <code>SPI_IMSK.ROR</code> bit unmask (enables) or masks (disables) the ROR interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
2 (R/NW)	TUWM	Transmit Urgent Watermark. The <code>SPI_IMSK.TUWM</code> bit unmask (enables) or masks (disables) the TUWM interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request
1 (R/NW)	RUWM	Receive Urgent Watermark. The <code>SPI_IMSK.RUWM</code> bit unmask (enables) or masks (disables) the RUWM interrupt.
		0 Disable (mask) interrupt request
		1 Enable (unmask) interrupt request

Interrupt Mask Clear Register

The `SPI_IMSK_CLR` register permits clearing individual mask bits in the `SPI_IMSK` register without affecting other bits in the register. Use write-1-to-clear on a bit in the `SPI_IMSK_CLR` register to clear the corresponding bit in the `SPI_IMSK` register.

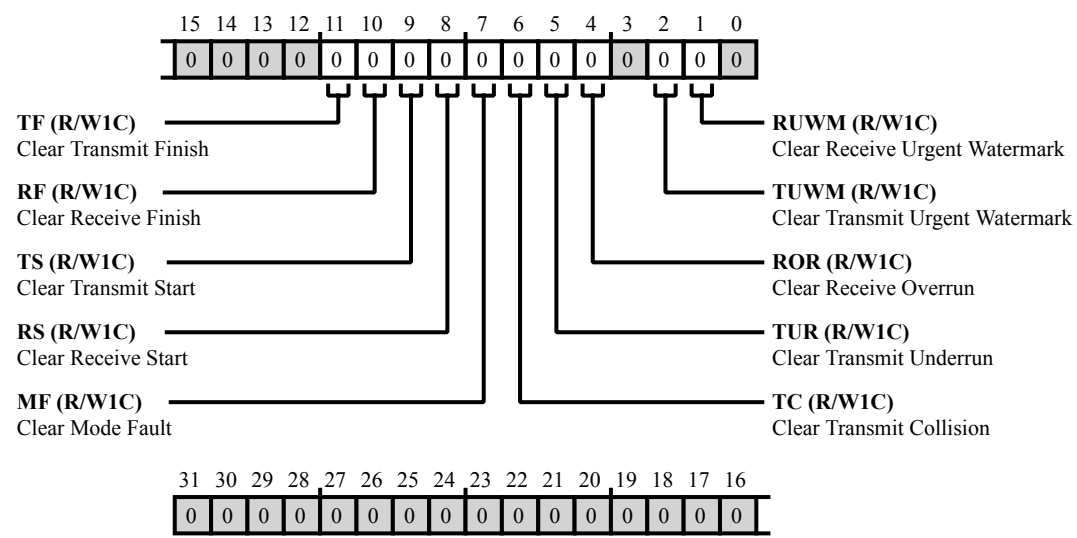


Figure 16-25: SPI_IMSK_CLR Register Diagram

Table 16-23: SPI_IMSK_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W1C)	TF	Clear Transmit Finish. The <code>SPI_IMSK_CLR.TF</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
10 (R/W1C)	RF	Clear Receive Finish. The <code>SPI_IMSK_CLR.RF</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
9 (R/W1C)	TS	Clear Transmit Start. The <code>SPI_IMSK_CLR.TS</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit

Table 16-23: SPI_IMSK_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	RS	Clear Receive Start. The <code>SPI_IMSK_CLR.RS</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
7 (R/W1C)	MF	Clear Mode Fault. The <code>SPI_IMSK_CLR.MF</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
6 (R/W1C)	TC	Clear Transmit Collision. The <code>SPI_IMSK_CLR.TC</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
5 (R/W1C)	TUR	Clear Transmit Underrun. The <code>SPI_IMSK_CLR.TUR</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
4 (R/W1C)	ROR	Clear Receive Overrun. The <code>SPI_IMSK_CLR.ROR</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit
2 (R/W1C)	TUWM	Clear Transmit Urgent Watermark. The <code>SPI_IMSK_CLR.TUWM</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Clear mask bit

Table 16-23: SPI_IMSK_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R/W1C)	RUWM	Clear Receive Urgent Watermark. The <code>SPI_IMSK_CLR.RUWM</code> bit clears the corresponding mask bit in the <code>SPI_IMSK</code> register.	
		0	No effect
		1	Clear mask bit

Interrupt Mask Set Register

The `SPI_IMSK_SET` register permits setting individual mask bits in the `SPI_IMSK` register without affecting other bits in the register. Use write-1-to-set on a bit in the `SPI_IMSK_SET` register to set the corresponding bit in the `SPI_IMSK` register.

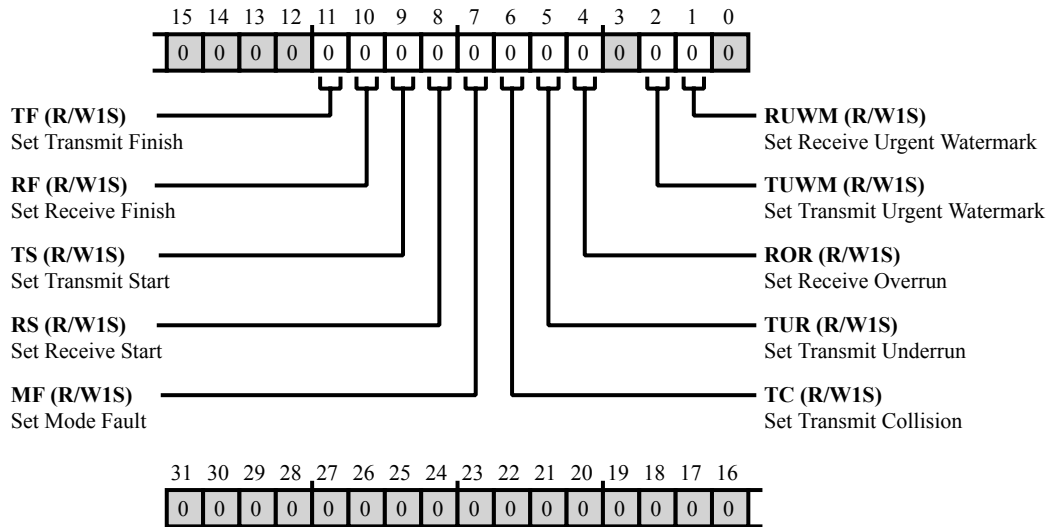


Figure 16-26: SPI_IMSK_SET Register Diagram

Table 16-24: SPI_IMSK_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W1S)	TF	Set Transmit Finish. The <code>SPI_IMSK_SET.TF</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
10 (R/W1S)	RF	Set Receive Finish. The <code>SPI_IMSK_SET.RF</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
9 (R/W1S)	TS	Set Transmit Start. The <code>SPI_IMSK_SET.TS</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit

Table 16-24: SPI_IMSK_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1S)	RS	Set Receive Start. The <code>SPI_IMSK_SET.RS</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
7 (R/W1S)	MF	Set Mode Fault. The <code>SPI_IMSK_SET.MF</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
6 (R/W1S)	TC	Set Transmit Collision. The <code>SPI_IMSK_SET.TC</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
5 (R/W1S)	TUR	Set Transmit Underrun. The <code>SPI_IMSK_SET.TUR</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
4 (R/W1S)	ROR	Set Receive Overrun. The <code>SPI_IMSK_SET.ROR</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit
2 (R/W1S)	TUWM	Set Transmit Urgent Watermark. The <code>SPI_IMSK_SET.TUWM</code> bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit

Table 16-24: SPI_IMSK_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W1S)	RUWM	Set Receive Urgent Watermark. The <code>SPI_IMSK_SET</code> .RUWM bit sets the corresponding mask bit in the <code>SPI_IMSK</code> register.
		0 No effect
		1 Set mask bit

Memory Mapped Read Header

The `SPI_MMRDH` register enables the use of memory-mapped mode. This mode allows direct memory-mapped read accesses of an SPI memory device and is primarily used to directly execute instructions from an SPI FLASH memory without using a low-level software driver. All overhead tasks such as transmission of the read header, pin turnaround timing and receive data sizing are handled in hardware.

The memory-mapped access mode is enabled by setting the `SPI_CTL.MMSE` bit. The features within the `SPI_MMRDH` register include a command skip mode, variable length byte addressing, and independent multi-pin support for command transmission, address transmission and data reception. In addition, the command opcode and mode bytes are fully programmable.

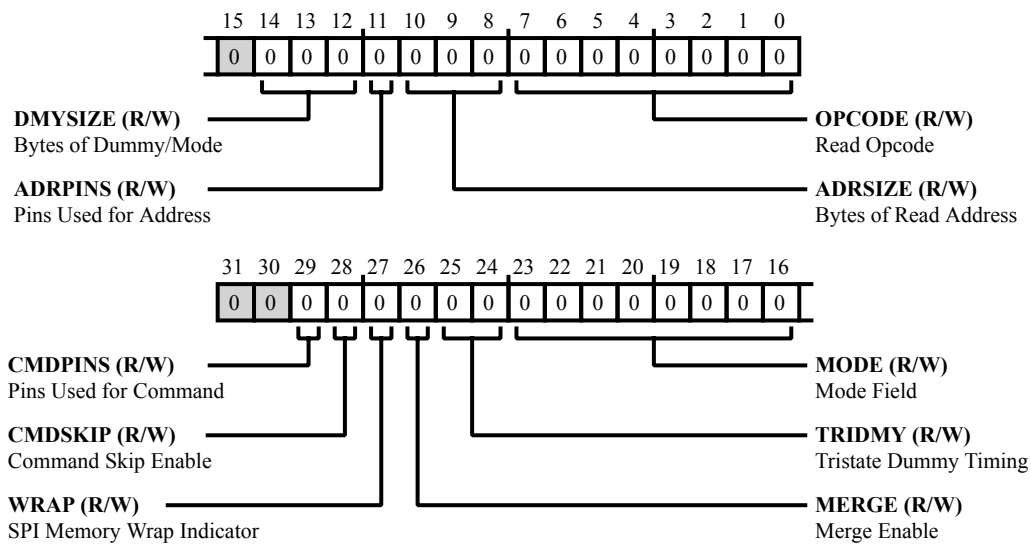


Figure 16-27: SPI_MMRDH Register Diagram

Table 16-25: SPI_MMRDH Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	CMDPINS	Pins Used for Command.
		The <code>SPI_MMRDH.CMDPINS</code> bit specifies the number of pins to be used for command transmission. This bit must be set consistent with the expectations established by the read opcode. Hardware does not interpret <code>SPI_MMRDH.OPCODE</code> , but rather relies on this bit to specify behavior. When cleared, it overrides the <code>SPI_CTL.MIOM</code> bits. When set, it uses bits specified by the <code>SPI_CTL.MIOM</code> bit setting.
		0 Use only one pin: MOSI (overrides <code>SPI_CTL.MIOM</code> bits)
		1 Use pins specified by <code>SPI_CTL.MIOM</code> bits

Table 16-25: SPI_MMRDH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	CMDSKIP	Command Skip Enable. The <code>SPI_MMRDH.CMDSKIP</code> bit enables command skip mode where the address is sent first and the <code>OPCODE</code> field is not sent (<code>SPI_MMRDH.CMDSKIP</code> bit =1). This mode is useful for supporting XIP (Execute-In-Place) operation where only the address is sent and the same read command is assumed. The SPI flash device must be primed with an initial read command, before the <code>SPI_MMRDH.CMDSKIP</code> bit is set.
		0 <code>OPCODE</code> field is sent first followed by address
		1 <code>OPCODE</code> field is not sent; address is sent first
27 (R/W)	WRAP	SPI Memory Wrap Indicator. The <code>SPI_MMRDH.WRAP</code> bit must be set by software if software places a connected SPI memory device into a 8-byte, 16-byte or 32-byte wrap mode based on the <code>ILINE</code> and <code>DLINE</code> field setting of the cache configuration register address wrap mode. Software achieves this by transmitting a vendor specified command to the SPI memory device while the <code>SPI_CTL.MMSE</code> bit =0. If the <code>SPI_MMRDH.WRAP</code> bit =1, the SPI does not need to deassert the SPI slave select signal and resend the read header in order to wrap to the cache line base when servicing misaligned cache fill requests. Although this improves cache fill efficiency, it requires that the SPI deassert the SPI slave select pin and resend the read header whenever a DMA burst requests crosses 32 byte alignments. Setting this bit improves cache throughput but decreases DMA throughput.
		0 SPI Memory auto increments address purely sequentially
		1 SPI Memory auto increments address but wraps within 32 Byte lines
26 (R/W)	MERGE	Merge Enable. When the <code>SPI_MMRDH.MERGE</code> bit is set, SPI hardware combines the two successive transfers. This increases the throughput rate when accessing a large number of sequential memory locations. For more information refer to the Merged Read Accesses section.

Table 16-25: SPI_MMRDH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
25:24 (R/W)	TRIDMY	Tristate Dummy Timing. The <code>SPI_MMRDH.TRIDMY</code> bits specify whether and when output pins are three-stated during the interval of time specified by the <code>SPI_MMRDH.DMYSIZE</code> bits. Output pins potentially three-stated include all pins which were used to transmit the address
		0 Tristate outputs immediately
		1 Tristate outputs after 4 bits of dummy/mode are transmitted
		2 Tristate outputs after 8 bits of dummy/mode are transmitted
		3 Never tristate outputs (previously specified output state is held)
23:16 (R/W)	MODE	Mode Field. These bits specify up to a leading byte to be transmitted during the interval of time specified by the <code>SPI_MMRDH.DMYSIZE</code> bit field. This first byte, or a portion of it, is interpreted as mode bits when certain opcodes are used in conjunction with certain SPI memory devices. Mode bits are sent using the same number of pins which were used to transmit the address. Once sent, output pins will be held in their final resultant state until the conclusion of all dummy byte periods, unless three-stating the outputs is specified first by the <code>SPI_MMRDH.TRIDMY</code> bits.
14:12 (R/W)	DMYSIZE	Bytes of Dummy/Mode. The <code>SPI_MMRDH.DMYSIZE</code> bit field specifies the number of bytes separating address transmission and read data return. Dummy bytes elapse assuming dummy bits are transmitted using the same number of pins which were used to transmit address.
		0 0 Bytes
		1 1 Bytes
		2 2 Bytes
		3 3 Bytes
		4 4 Bytes
		5 5 Bytes
		6 6 Bytes
		7 7 Bytes

Table 16-25: SPI_MMRDH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	ADRPINS	Pins Used for Address. The <code>SPI_MMRDH.ADRPINS</code> bit specifies the number of pins to be used for address transmission. This bit must be set consistent with expectations established by read opcode. Hardware does not interpret the <code>SPI_MMRDH.OPCODE</code> , but rather relies on this bit to specify behavior.
		0 Use only one pin: MOSI (overrides <code>SPI_CTL.MIOM</code> bits)
		1 Use pins specified by <code>SPI_CTL.MIOM</code> bits
10:8 (R/W)	ADRSIZE	Bytes of Read Address. The <code>SPI_MMRDH.ADRSIZE</code> bit field defines the number of bytes used to specify the read address. The read address is sent immediately following the transmission of opcode. Unlike opcode bits, address bits may be sent using either one or multiple pins. The number of pins is selected using the <code>SPI_MMRDH.ADRPINS</code> bit. The address sent to a connected SPI memory device is an echo of the read address received by the SPI peripheral slave port. The least significant bytes of address are sent when the entire address is not sent.
		0 1 Byte
		1 1 Byte
		2 2 Bytes
		3 3 Bytes
		4 4 Bytes
7:0 (R/W)	OPCODE	Read Opcode. The <code>SPI_MMRDH.OPCODE</code> bit field specifies the initial bits transmitted in response to a read request of SPI memory. Although any opcode may be sent, values 0x03, 0x0B, 0x3B, 0x6B, 0xBB, and 0xEB are likely to be the most commonly used. <code>SPI_MMRDH.OPCODE</code> is sent by the SPI without interpretation; the states of these bits have no effect beyond specifying what is initially shifted across the SPI interface.

SPI Memory Top Address

The `SPI_MMTOP` register specifies the top populated address of a connected SPI memory device.

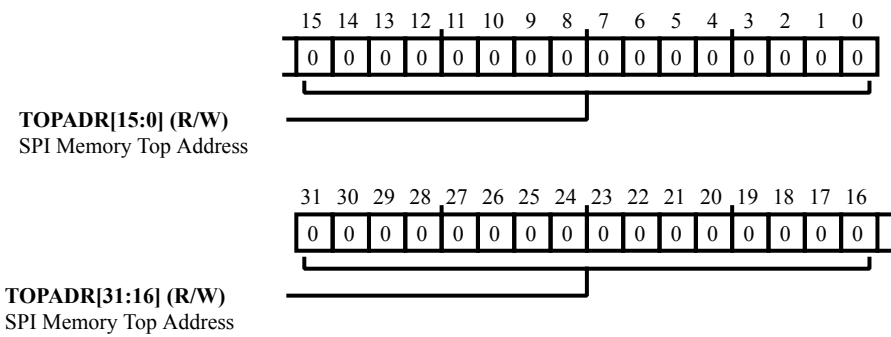


Figure 16-28: SPI_MMTOP Register Diagram

Table 16-26: SPI_MMTOP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	TOPADR	SPI Memory Top Address. The <code>SPI_MMTOP.TOPADR</code> bit field specifies the top populated address of a connected SPI memory device. Attempts to access SPI memory are not blocked if this address is exceeded and an error is generated as part of the read response.

Receive FIFO Data Register

The `SPI_RFIFO` register has an interface to the receive shift register in the SPI and has an interface to the processor's data buses. The top level of the buffer is visible to programs as the 32-bit `SPI_RFIFO` register, but the size (number of word locations) of the receive FIFO is actually flexible with transfer word size. The size of the receive FIFO is 8 if the word size is 8-bit, or the size is 4 if the word size is 16-bit, or the size is 2 if the word size is 32-bit.

Both masters and slaves may stop or stall receive transfers based on FIFO status. When the receive FIFO is full, the SPI master stops initiating new transfers on the SPI if `SPI_RXCTL.RTI` is enabled. A slave may stall the SPI interface when the content of the FIFO crosses the selected watermark. If data reception continues after `SPI_RFIFO` is full, the data in the receive FIFO is invalid. The SPI indicates this condition with receive overrun (`SPI_STAT.ROR`) error. This condition is possible when `SPI_RXCTL.RTI = 0` and `SPI_RXCTL.REN = 1` for a master, or for a slave that does not exercise flow control.

Note that the receive FIFO is reset (cleared) when the SPI is disabled after being enabled.

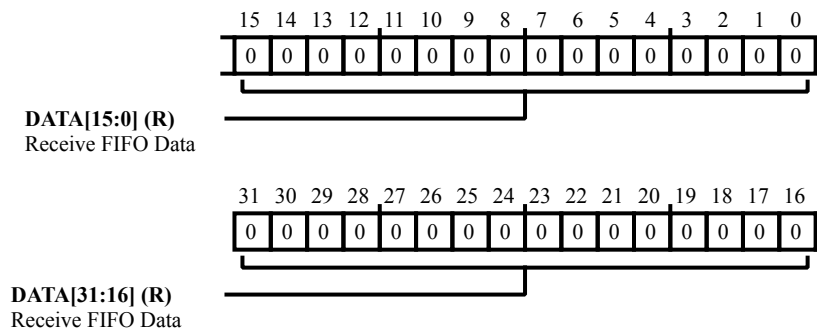


Figure 16-29: SPI_RFIFO Register Diagram

Table 16-27: SPI_RFIFO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	DATA	Receive FIFO Data. The <code>SPI_RFIFO.DATA</code> bit field contains the FIFO receive data.

Received Word Count Register

The `SPI_RWC` register holds a count of the number of words remaining to be received by the SPI. To start the decrement of the word count in `SPI_RWC`, enable the receive word counter (`SPI_RXCTL.RWCEN = 1`). The SPI uses the word count to control the duration of transfers and to signal the completion of a burst of transfers with the receive finish interrupt (`SPI_ILAT.RF`). In DMA mode, the SPI uses the `SPI_RWC` register to ensure that the number of frames received during a DMA transfer is equal to the number of words programmed in the DMA channel controller. The values programmed into the `SPI_RWC` registers should match the word count in the DMA configuration. The `SPI_RWC` register maintains the number of frames to be received in a transfer. The `SPI_RWC` should only be changed when the counter is disabled.

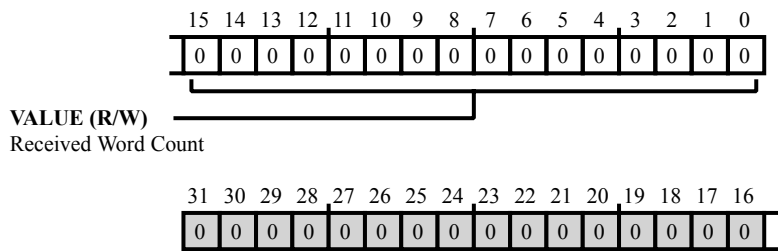


Figure 16-30: SPI_RWC Register Diagram

Table 16-28: SPI_RWC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Received Word Count. The <code>SPI_RWC.VALUE</code> bits hold the receive transfer word count.

Received Word Count Reload Register

The `SPI_RWCR` register holds the receive word count value that the SPI loads into the `SPI_RWC` register when the transfer count decrements to zero. To prevent the SPI from reloading the counter, use zero for the reload count value. The `SPI_RWCR` register should only be changed when the counter is disabled.

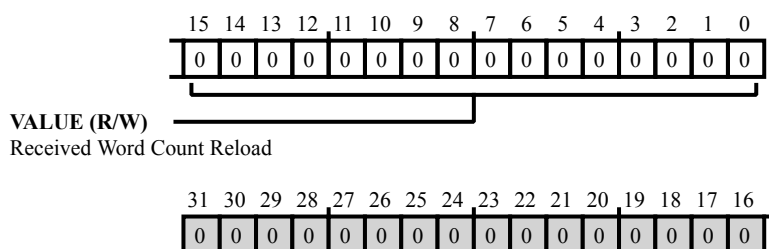


Figure 16-31: SPI_RWCR Register Diagram

Table 16-29: SPI_RWCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Received Word Count Reload. The <code>SPI_RWCR.VALUE</code> bits hold the receive transfer word count reload value.

Receive Control Register

The `SPI_RXCTL` register enables the SPI receive channel, initiates receive transfers, and configures `SPI_RFIFO` buffer watermark settings.

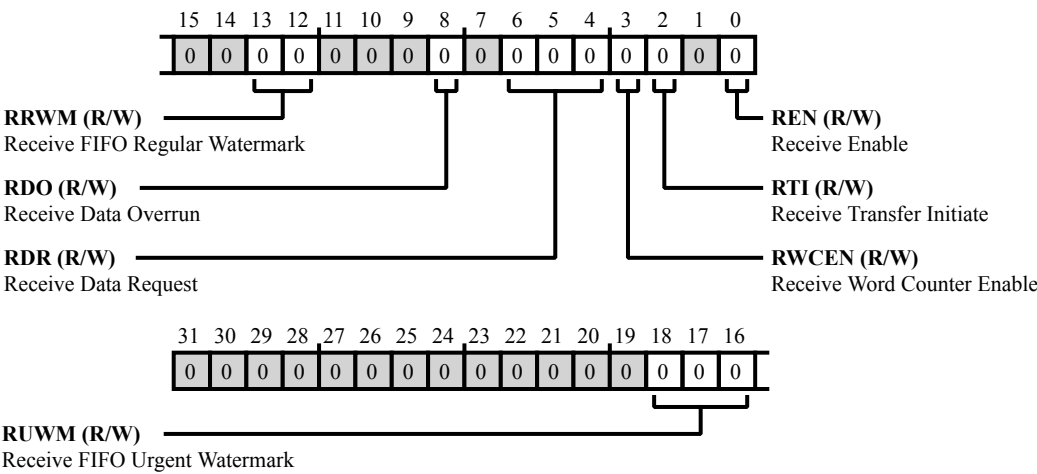


Figure 16-32: SPI_RXCTL Register Diagram

Table 16-30: SPI_RXCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:16 (R/W)	RUWM	Receive FIFO Urgent Watermark. The <code>SPI_RXCTL.RUWM</code> bits select the receive FIFO (<code>SPI_RFIFO</code>) watermark level for urgent data bus requests. The SPI also uses this watermark level for generation of the <code>SPI_ILAT.RUWM</code> interrupt. When an urgent <code>SPI_RFIFO</code> watermark is enabled with <code>SPI_RXCTL.RUWM</code> , the <code>SPI_RXCTL.RRWM</code> selection is used as the deassertion condition for any <code>SPI_ILAT.RUWM</code> interrupts that are latched.
		0 Disabled
		1 25% full RFIFO
		2 50% full RFIFO
		3 75% full RFIFO
		4 Full RFIFO
		5 Reserved
		6 Reserved
		7 Reserved

Table 16-30: SPI_RXCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13:12 (R/W)	RRWM	Receive FIFO Regular Watermark. The SPI_RXCTL.RRWM bits select the receive FIFO (SPI_RFIFO) watermark level for regular data bus requests. When an urgent SPI_RFIFO watermark is enabled with SPI_RXCTL.RUWM, the SPI_RXCTL.RRWM selection is used as the deassertion condition for any SPI_ILAT.RUWM interrupts that are latched.
		0 Empty RFIFO
		1 RFIFO less than 25% full
		2 RFIFO less than 50% full
		3 RFIFO less than 75% full
8 (R/W)	RDO	Receive Data Overrun. The SPI_RXCTL.RDO bit selects handling for receive data requests when the receive buffer (SPI_RFIFO) is full. If enabled and SPI_RFIFO is full, the SPI overwrites old data in the buffer with incoming data. If disabled and SPI_RFIFO is full, the SPI keeps old data in the buffer and discards incoming data.
		0 Discard incoming data if SPI_RFIFO is full
		1 Overwrite old data if SPI_RFIFO is full
6:4 (R/W)	RDR	Receive Data Request. The SPI_RXCTL.RDR bits select receive FIFO (SPI_RFIFO) watermark conditions that direct the SPI to generate a receive data request.
		0 Disabled
		1 Not empty RFIFO
		2 25% full RFIFO
		3 50% full RFIFO
		4 75% full RFIFO
		5 Full RFIFO
		6 Reserved
		7 Reserved
3 (R/W)	RWCEN	Receive Word Counter Enable. The SPI_RXCTL.RWCEN bit enables the decrement of the SPI_RWC register when the count is not zero and SPI_RXCTL.RTI is enabled. Enabling SPI_RXCTL.RWCEN prevents receive overrun errors from occurring. The SPI_RXCTL.RWCEN bit is valid only when the SPI is a master.
		0 Disable
		1 Enable

Table 16-30: SPI_RXCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	RTI	Receive Transfer Initiate. The <code>SPI_RXCTL.RTI</code> bit enables initiation of receive transfers if the receive FIFO (<code>SPI_RFIFO</code>) is not full. The bit also enables this initiation if <code>SPI_RWC</code> is not zero when <code>SPI_RXCTL.RWCEN</code> is enabled. Enabling <code>SPI_RXCTL.RTI</code> prevents receive overrun errors from occurring. The <code>SPI_RXCTL.RTI</code> bit is valid only when the SPI is a master.
		0 Disable
		1 Enable
0 (R/W)	REN	Receive Enable. The <code>SPI_RXCTL.REN</code> bit enables SPI receive channel operation.
		0 Disable
		1 Enable

Slave Select Register

The `SPI_SLVSEL` register enables the `SPI_SEL[n]` pins for output and indicates the state (high or low) of these pins when enabled.

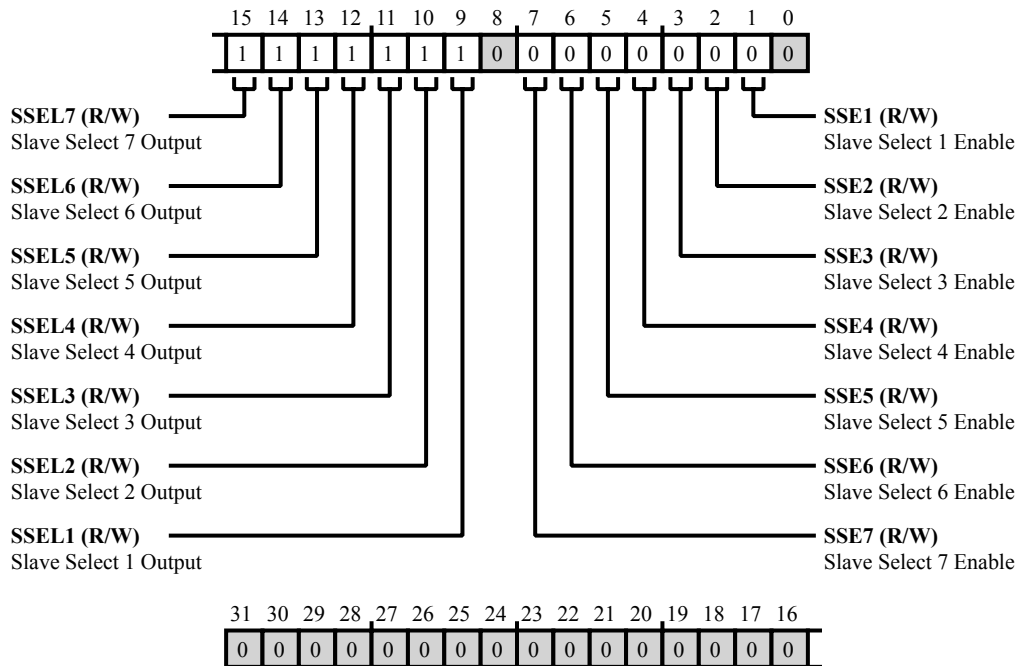


Figure 16-33: SPI_SLVSEL Register Diagram

Table 16-31: SPI_SLVSEL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SSEL7	Slave Select 7 Output. The <code>SPI_SLVSEL.SSEL7</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
14 (R/W)	SSEL6	Slave Select 6 Output. The <code>SPI_SLVSEL.SSEL6</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High

Table 16-31: SPI_SLVSEL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	SSEL5	Slave Select 5 Output. The <code>SPI_SLVSEL.SSEL5</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
12 (R/W)	SSEL4	Slave Select 4 Output. The <code>SPI_SLVSEL.SSEL4</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
11 (R/W)	SSEL3	Slave Select 3 Output. The <code>SPI_SLVSEL.SSEL3</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
10 (R/W)	SSEL2	Slave Select 2 Output. The <code>SPI_SLVSEL.SSEL2</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
9 (R/W)	SSEL1	Slave Select 1 Output. The <code>SPI_SLVSEL.SSEL1</code> bit state indicates the value driven on the related <code>SPI_SEL[n]</code> pin.
		0 Low
		1 High
7 (R/W)	SSE7	Slave Select 7 Enable. The <code>SPI_SLVSEL.SSE7</code> bit enables the related <code>SPI_SEL[n]</code> pin for output. If disabled, the SPI three-states the related <code>SPI_SEL[n]</code> pin.
		0 Disable
		1 Enable

Table 16-31: SPI_SLVSEL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	SSE6	Slave Select 6 Enable. The SPI_SLVSEL.SSE6 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
5 (R/W)	SSE5	Slave Select 5 Enable. The SPI_SLVSEL.SSE5 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
4 (R/W)	SSE4	Slave Select 4 Enable. The SPI_SLVSEL.SSE4 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
3 (R/W)	SSE3	Slave Select 3 Enable. The SPI_SLVSEL.SSE3 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
2 (R/W)	SSE2	Slave Select 2 Enable. The SPI_SLVSEL.SSE2 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable
1 (R/W)	SSE1	Slave Select 1 Enable. The SPI_SLVSEL.SSE1 bit enables the related $\overline{\text{SPI_SEL}}[n]$ pin for output. See the SPI_SLVSEL.SSE7 bit description for more information.
		0 Disable
		1 Enable

Status Register

The `SPI_STAT` register indicates SPI status including FIFO status, error conditions, and interrupt conditions. When an interrupt condition from this register is unmasked (enabled) by the corresponding bit in the `SPI_IMSK` register, the interrupt request is latched into the corresponding bit in the `SPI_ILAT` register.

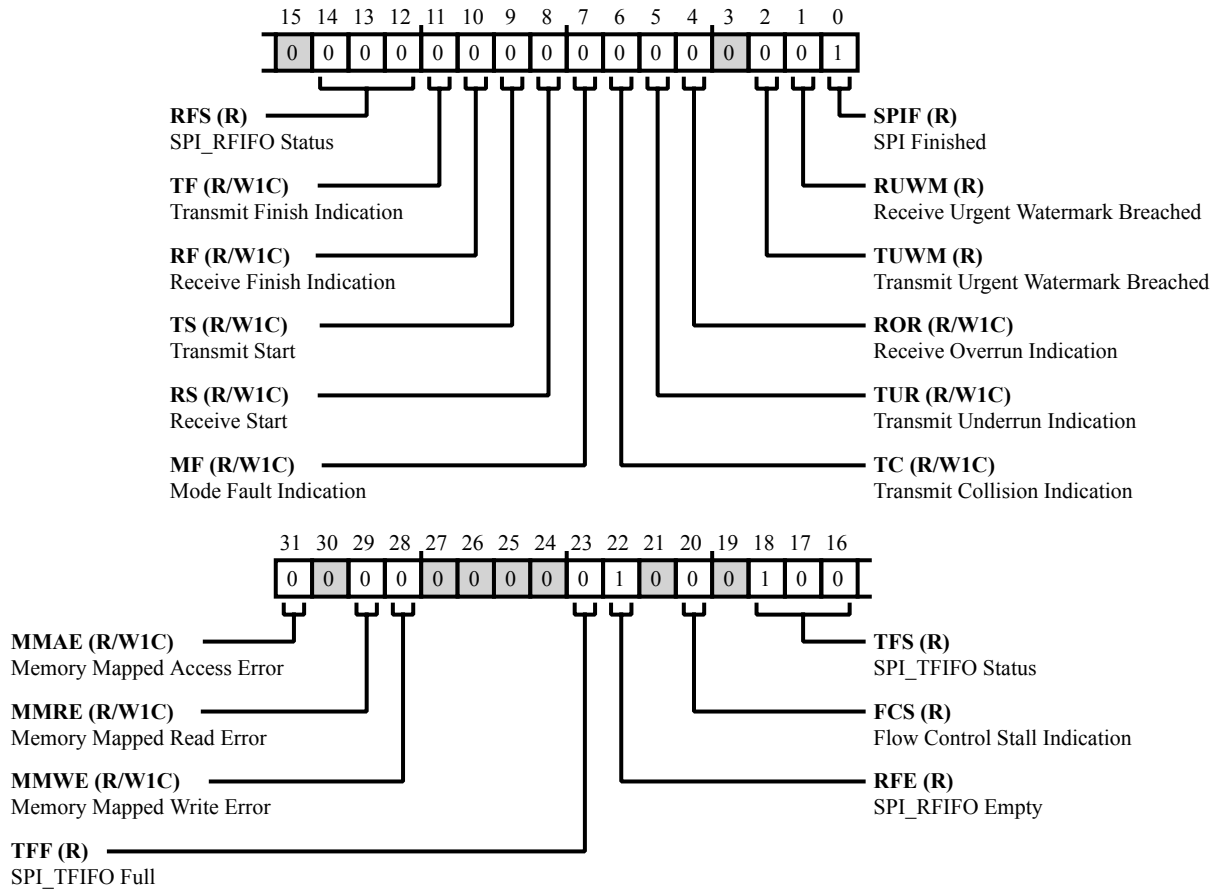


Figure 16-34: SPI_STAT Register Diagram

Table 16-32: SPI_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	MMAE	Memory Mapped Access Error. The <code>SPI_STAT.MMAE</code> bit =1 if an attempt is made to access either the Tx or Rx FIFO while memory-mapped access of SPI memory is enabled (see the <code>SPI_CTL.MMSE</code> bit). The <code>SPI_STAT.MMAE</code> bit =0 when a 1 is written to it. The <code>SPI_STAT.MMAE</code> bit is provided for software notification only. Its state has no further effect.

Table 16-32: SPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W1C)	MMRE	Memory Mapped Read Error. The <code>SPI_STAT.MMRE</code> bit =1 if an attempt is made to read address space reserved for memory-mapped SPI memory while memory mapping is disabled (see the <code>SPI_CTL.MMSE</code> bit). The <code>SPI_STAT.MMRE</code> bit =0 when a 1 is written to it. This bit is provided for software notification only. Its state has no further effect.
28 (R/W1C)	MMWE	Memory Mapped Write Error. The <code>SPI_STAT.MMWE</code> bit =1 if an attempt is made to write address space reserved for memory-mapped SPI memory. The <code>SPI_STAT.MMWE</code> bit =0 when a 1 is written to it. This bit is provided for software notification only. Its state has no further effect.
23 (R/NW)	TFF	SPI_TFIFO Full. The <code>SPI_STAT.TFF</code> bit indicates whether the <code>SPI_TFIFO</code> is full or not full.
		0 Not full Tx FIFO
		1 Full Tx FIFO
22 (R/NW)	RFE	SPI_RFIFO Empty. The <code>SPI_STAT.RFE</code> bit indicates whether the <code>SPI_RFIFO</code> is empty or not empty.
		0 Rx FIFO not empty
		1 Rx FIFO empty
20 (R/NW)	FCS	Flow Control Stall Indication. The <code>SPI_STAT.FCS</code> bit indicates whether a slave has deasserted the <code>SPI_RDY</code> pin to stall the SPI master while the slave is unable to service the SPI masters request. This bit is valid only when the SPI is a master (<code>SPI_CTL.MSTR</code> =1) and flow control is enabled (<code>SPI_CTL.FCEN</code> =1).
		0 No Stall (RDY pin asserted)
		1 Stall (RDY pin deasserted)
18:16 (R/NW)	TFS	SPI_TFIFO Status. The <code>SPI_STAT.TFS</code> bits indicate the status of the <code>SPI_TFIFO</code> . The SPI uses this status when evaluating transmit watermark conditions.
		0 Full TFIFO
		1 25% empty TFIFO
		2 50% empty TFIFO
		3 75% empty TFIFO
		4 Empty TFIFO

Table 16-32: SPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14:12 (R/NW)	RFS	SPI_RFIFO Status. The <code>SPI_STAT.RFS</code> bits indicate the status of the <code>SPI_RFIFO</code> . The SPI uses this status when evaluating receive watermark conditions.
		0 Empty RFIFO
		1 25% full RFIFO
		2 50% full RFIFO
		3 75% full RFIFO
		4 Full RFIFO
11 (R/W1C)	TF	Transmit Finish Indication. The <code>SPI_STAT.TF</code> bit indicates that the SPI has detected the finish of a transmit burst transfer (the <code>SPI_TWC</code> count decrements to zero). This condition can only occur when <code>SPI_TXCTL.TTI</code> and <code>SPI_TXCTL.TWCEN</code> are enabled.
		0 No status
		1 Transmit finish detected
10 (R/W1C)	RF	Receive Finish Indication. The <code>SPI_STAT.RF</code> bit indicates that the SPI has detected the finish of a receive burst transfer (the <code>SPI_RWC</code> count decrements to zero). This condition can only occur when <code>SPI_RXCTL.RTI</code> and <code>SPI_RXCTL.RWCEN</code> are enabled.
		0 No status
		1 Receive finish detected
9 (R/W1C)	TS	Transmit Start. The <code>SPI_STAT.TS</code> bit indicates that the SPI has detected the start of a transmit burst transfer. A transmit bursts starts with the load of <code>SPI_TWC</code> from the <code>SPI_TWCR</code> . This condition can only occur when <code>SPI_TXCTL.TTI</code> and <code>SPI_TXCTL.TWCEN</code> are enabled.
		0 No status
		1 Transmit start detected
8 (R/W1C)	RS	Receive Start. The <code>SPI_STAT.RS</code> bit indicates that the SPI has detected the start of a receive burst transfer. A receive bursts starts with the load of <code>SPI_RWC</code> from the <code>SPI_RWCR</code> . This condition can only occur when <code>SPI_RXCTL.RTI</code> and <code>SPI_RXCTL.RWCEN</code> are enabled.
		0 No status
		1 Receive start detected

Table 16-32: SPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	MF	Mode Fault Indication. The <code>SPI_STAT.MF</code> bit, when SPI is a master and <code>SPI_CTL.PSSE</code> is enabled, indicates that multiple masters have asserted slave select inputs.
		0 No status
		1 Mode fault occurred
6 (R/W1C)	TC	Transmit Collision Indication. The <code>SPI_STAT.TC</code> bit, when SPI is a slave, indicates that the load of data into the shift register has occurred too close to the first transmitting edge of the SPI clock.
		0 No status
		1 Transmit collision occurred
5 (R/W1C)	TUR	Transmit Underrun Indication. The <code>SPI_STAT.TUR</code> bit, when the transmit FIFO (<code>SPI_TFIFO</code>) is empty, indicates that the last word in the transmit FIFO has been re-sent as transmit data. Alternately, it indicates that zero has been sent as transmit data.
		0 No status
		1 Transmit underrun occurred
4 (R/W1C)	ROR	Receive Overrun Indication. The <code>SPI_STAT.ROR</code> bit, when the receive FIFO (<code>SPI_RFIFO</code>) is full, indicates that a word in the receive FIFO has been overwritten with incoming receive data. Alternately, it indicates that incoming receive data has been discarded.
		0 No status
		1 Receive overrun occurred
2 (R/NW)	TUWM	Transmit Urgent Watermark Breached. The <code>SPI_STAT.TUWM</code> bit indicates that the transmit urgent watermark (<code>SPI_TXCTL.TUWM</code>) has been reached. This condition is cleared when the transmit FIFO fills enough to reach the transmit regular watermark (<code>SPI_TXCTL.TRWM</code>).
		0 Tx regular watermark reached
		1 Tx urgent watermark breached
1 (R/NW)	RUWM	Receive Urgent Watermark Breached. The <code>SPI_STAT.RUWM</code> bit indicates that the receive urgent watermark (<code>SPI_RXCTL.RUWM</code>) has been reached. This condition is cleared when the receive FIFO empties enough to reach the receive regular watermark (<code>SPI_RXCTL.RRWM</code>).
		0 Rx regular watermark reached
		1 Rx urgent watermark breached

Table 16-32: SPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	SPIF	SPI Finished. The <code>SPI_STAT.SPIF</code> bit indicates that a single word transfer is complete.
		0 No status
		1 Completed single word transfer

Transmit FIFO Data Register

The `SPI_TFIFO` register has an interface to the transmit shift register in the SPI and has an interface to the processor's data buses. The top level of the buffer is visible to programs as the 32-bit `SPI_TFIFO` register, but the size (number of word locations) of the transmit FIFO is actually flexible with transfer word size. The size of the transmit FIFO is 8 if the word size is 8-bit, or the size is 4 if the word size is 16-bit, or the size is 2 if the word size is 32-bit.

Both masters and slaves may stop or stall transmit transfers based on FIFO status. When the transmit FIFO is empty, the SPI master stops initiating new transfers on the SPI if `SPI_TXCTL.TTI` is enabled. A slave may stall the SPI interface when the content of the FIFO crosses the selected watermark. If data transmit requests continue after `SPI_TFIFO` is empty, the data sent from the transmit FIFO is invalid, and the SPI indicates this condition with transmit underrun (`SPI_STAT.TUR`). This condition is possible when `SPI_TXCTL.TTI = 0` and `SPI_TXCTL.TEN = 1` for a master, or for a slave that does not exercise flow control.

Note that the transmit FIFO is reset (cleared) when the SPI is disabled after being enabled.

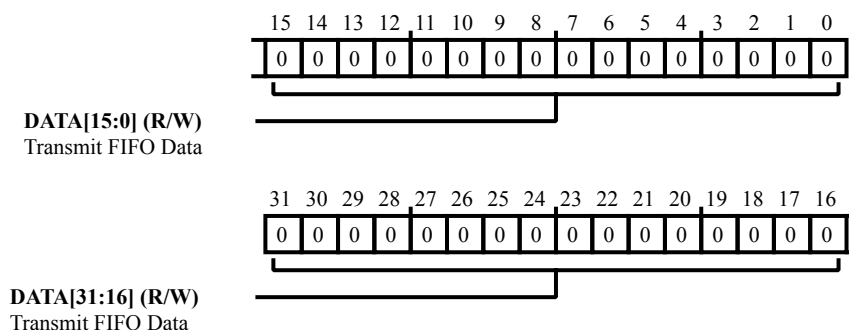


Figure 16-35: `SPI_TFIFO` Register Diagram

Table 16-33: `SPI_TFIFO` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	Transmit FIFO Data. The <code>SPI_TFIFO.DATA</code> bit field contains the FIFO transmit data.

Transmitted Word Count Register

The `SPI_TWC` register holds a count of the number of words remaining to be transmitted by the SPI. To start the decrement of the word count in `SPI_TWC`, enable the transmit word counter (`SPI_TXCTL.TWCEN = 1`). The SPI uses the word count to control the duration of transfers and to signal the completion of a burst of transfers with the transmit finish interrupt request. In DMA mode, the SPI uses the `SPI_TWC` to ensure that the number of frames transmitted during a DMA transfer is equal to the number of words programmed in the DMA channel controller. The values programmed into the `SPI_TWC` registers should match the word count in the DMA configuration. The `SPI_TWC` maintains the number of frames to be transmitted in a transfer. The `SPI_TWC` should only be changed when the counter is disabled.

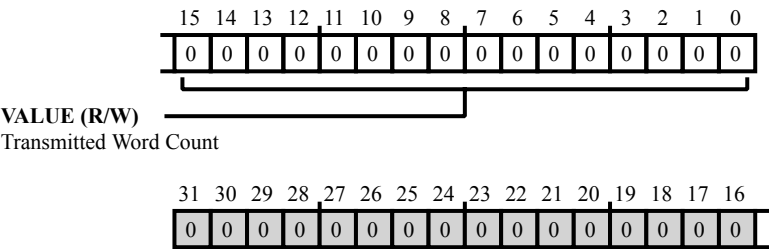


Figure 16-36: SPI_TWC Register Diagram

Table 16-34: SPI_TWC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Transmitted Word Count. The <code>SPI_TWC.VALUE</code> bits hold the transmit transfer word count.

Transmitted Word Count Reload Register

The `SPI_TWCR` register holds the transmit word count value that the SPI loads into the `SPI_TWC` register when the transfer count decrements to zero. To prevent the SPI from reloading the counter, use zero for the reload count value. The `SPI_TWCR` should only be changed when the counter is disabled.

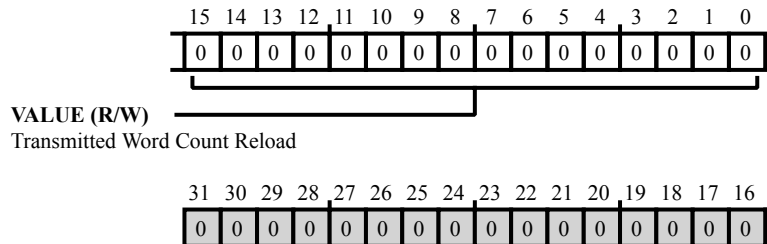


Figure 16-37: SPI_TWCR Register Diagram

Table 16-35: SPI_TWCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Transmitted Word Count Reload. The <code>SPI_TWCR.VALUE</code> bits hold the transmit transfer word count reload value.

Transmit Control Register

The `SPI_TXCTL` register enables the SPI transmit channel, initiates transmit transfers, and configures `SPI_TFIFO` buffer watermark settings.

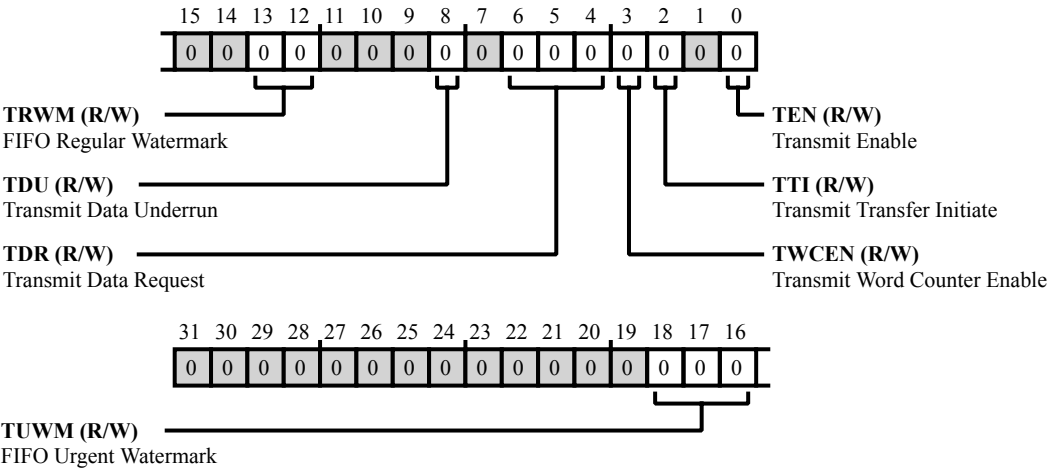


Figure 16-38: SPI_TXCTL Register Diagram

Table 16-36: SPI_TXCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:16 (R/W)	TUWM	FIFO Urgent Watermark. The <code>SPI_TXCTL.TUWM</code> bits select the transmit FIFO (<code>SPI_TFIFO</code>) watermark level for urgent data bus requests. The SPI also uses this watermark level for generation of the <code>SPI_ILAT.TUWM</code> interrupt request. When an urgent <code>SPI_TFIFO</code> watermark is enabled with <code>SPI_TXCTL.TUWM</code> , the <code>SPI_TXCTL.TRWM</code> selection is used as the deassertion condition for any <code>SPI_ILAT.TUWM</code> interrupt requests that are latched.
		0 Disabled
		1 25% empty TFIFO
		2 50% empty TFIFO
		3 75% empty TFIFO
		4 Empty TFIFO

Table 16-36: SPI_TXCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13:12 (R/W)	TRWM	FIFO Regular Watermark. The SPI_TXCTL.TRWM bits select the transmit FIFO (SPI_TFIFO) watermark level for regular data bus requests. When an urgent SPI_TFIFO watermark is enabled with SPI_TXCTL.TUWM, the SPI_TXCTL.TRWM selection is used as the deassertion condition for any SPI_ILAT.TUWM interrupt requests that are latched.
		0 Full TFIFO
		1 TFIFO less than 25% empty
		2 TFIFO less than 50% empty
		3 TFIFO less than 75% empty
8 (R/W)	TDU	Transmit Data Underrun. The SPI_TXCTL.TDU bit selects handling for transmit data requests when the transmit buffer (SPI_TFIFO) is empty. If enabled and SPI_TFIFO is empty, the SPI transmits zero as data. If disabled and SPI_TFIFO is empty, the SPI transmits the last word in the buffer as data.
		0 Send last word when SPI_TFIFO is empty
		1 Send zeros when SPI_TFIFO is empty
6:4 (R/W)	TDR	Transmit Data Request. The SPI_TXCTL.TDR bits select transmit FIFO (SPI_TFIFO) watermark conditions that direct the SPI to generate a transmit status interrupt request.
		0 Disabled
		1 Not full TFIFO
		2 25% empty TFIFO
		3 50% empty TFIFO
		4 75% empty TFIFO
		5 Empty TFIFO
3 (R/W)	TWCEN	Transmit Word Counter Enable. The SPI_TXCTL.TWCEN bit enables the decrement of the transmit word count (SPI_TWC) register when the count is not zero and SPI_TXCTL.TTI is enabled. Enabling SPI_TXCTL.TWCEN prevents transmit underrun errors from occurring. The SPI_TXCTL.TWCEN bit is valid only when the SPI is a master.
		0 Disable
		1 Enable

Table 16-36: SPI_TXCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	TTI	Transmit Transfer Initiate. The SPI_TXCTL.TTI bit enables initiation of transmit transfers if the transmit FIFO (SPI_TFIFO) is not empty. The bit also enables this initiation if SPI_TWC is not zero when SPI_TXCTL.TWCEN is enabled. Enabling SPI_TXCTL.TTI prevents transmit underrun errors from occurring. The SPI_TXCTL.TTI bit is valid only when the SPI is a master.
		0 Disable
		1 Enable
0 (R/W)	TEN	Transmit Enable. The SPI_TXCTL.TEN bit enables SPI transmit channel operation.
		0 Disable
		1 Enable

17 Octal Serial Peripheral Interface (OSPI)

The Octal Serial Peripheral Interface (OSPI) controller provides access to serial flash devices. It is designed to access flash devices with its modes enabling efficient communication, with minimum overhead on software. Standard SPI is supported along with high performance dual, quad, and octal SPI modes where data can be transferred on up to 8 data pins. Along with normal mode of operation in single transfer (STR) mode, the OSPI controller supports dual transfer rate (DTR) mode, where data, address, and commands are transferred on both edges of the serial clock. With direct access mode support, any read/write to the OSPI memory space triggers a read/write to flash memory.

Direct read accesses using core and DMA are possible, with the latter minimizing software overhead. The OSPI controller also provides Software Triggered Instruction Generation (STIG) mode, which can be used to erase the data on flash devices and access volatile and nonvolatile configuration registers, legacy flash status register, and other status/protection registers.

The OSPI provides a glueless hardware interface with SPI flash devices. The OSPI peripheral includes programmable baud rates, clock phase, clock polarity, separate dummy cycles for read and write accesses, data sampling control, programmable page size/block size, and so on. With this independent control of how many lines to use for command, address, and data allows to virtually interface any flash device with the processor through OSPI controller.

The OSPI also provides support for DQS when sampling data to improve read data capture and PHY mode support to enable high-speed data transfers.

OSPI Features

The OSPI controller supports the following features:

- Full-duplex, synchronous, serial interface
- Memory mapped direct mode operation for performing flash data transfers
- STIG mode operation to issue flash commands
- Automatic flash status polling support for flash program through direct access mode
- XIP (Execute in Place) support
- Support for single, dual, quad, or octal modes of operation
- Support for DDR mode and DTR protocol (including Octal DDR protocol)

- Up to 16-bit data transfer in each SPI clock cycle
- Programmable baud rate, clock phase, and polarity
- Programmable interrupt generation
- Programmable chip select control timing
- Programmable dummy cycles for read and write operations
- Tune data capture mechanism to improve high speed operation
- Programmable write protected regions to block system writes
- Support for DQS to sample the data to improve read data capturing
- PHY mode support to allow the high speed data transfers

OSPI Functional Description

ADSP-2159x OSPI Register List

The OSPI is a Octal SPI controller.

Table 17-1: ADSP-2159x OSPI Register List

Name	Description
OSPI_CTL	Octal SPI Control Register
OSPI_DLY	Device Delay Register
OSPI_DRICTL	Device Read Instruction Control Register
OSPI_DWICTL	Device Write Instruction Control Register
OSPI_DSCTL	Device Size Control Register
OSPI_DLLOB_LWR	DLL Observable Register (Lower)
OSPI_DLLOB_UP	DLL Observable Register (Upper)
OSPI_FCA	Flash Command Address Register
OSPI_FCCTL	Flash Command Control Register
OSPI_FCMCTL	Flash Command Control Memory Register
OSPI_FCRD_LWR	Flash Command Read Data Register (Lower)
OSPI_FCRD_UP	Flash Command Read Data Register (Upper)
OSPI_FCWD_LWR	Flash Command Write Data Register (Lower)
OSPI_FCWD_UP	Flash Command Write Data Register (Upper)
OSPI_IMSK	Interrupt Mask Register
OSPI_ISTAT	Interrupt Status Register

Table 17-1: ADSP-2159x OSPI Register List (Continued)

Name	Description
OSPI_WRPROT_LWR	Lower Write Protection Register
OSPI_MBCTL	Mode Bit Control Register
OSPI_MODID	Module ID Register
OSPI_POLLEXP	Polling Expiration Register
OSPI_OE_LWR	Opcode Extension Register (Lower)
OSPI_OE_UP	Opcode Extension Register (Upper)
OSPI_PHYCTL	PHY Control Register
OSPI_PHYMCTL	PHY DLL Master Control Register
OSPI_POLSTAT	Polling Flash Status Register
OSPI_RDC	Read Data Capture Register
OSPI_REMAPADDR	Remap Address Register
OSPI_WRPROT_UP	Upper Write Protection Register
OSPI_WRPROT_CTL	Write Protection Control Register
OSPI_WCCTL	Write Completion Control Register

ADSP-2159x OSPI Interrupt List

Table 17-2: ADSP-2159x OSPI Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
137	OSPI0_INT	OSPI0 Error	Level	

OSPI Block Diagram

The OSPI module is comprised of:

- Low level SPI protocol controller
- Internal transmit/receive FIFOs
- DAC
- STIG controller
- Register interface

The figure shows the OSPI controller block diagram.

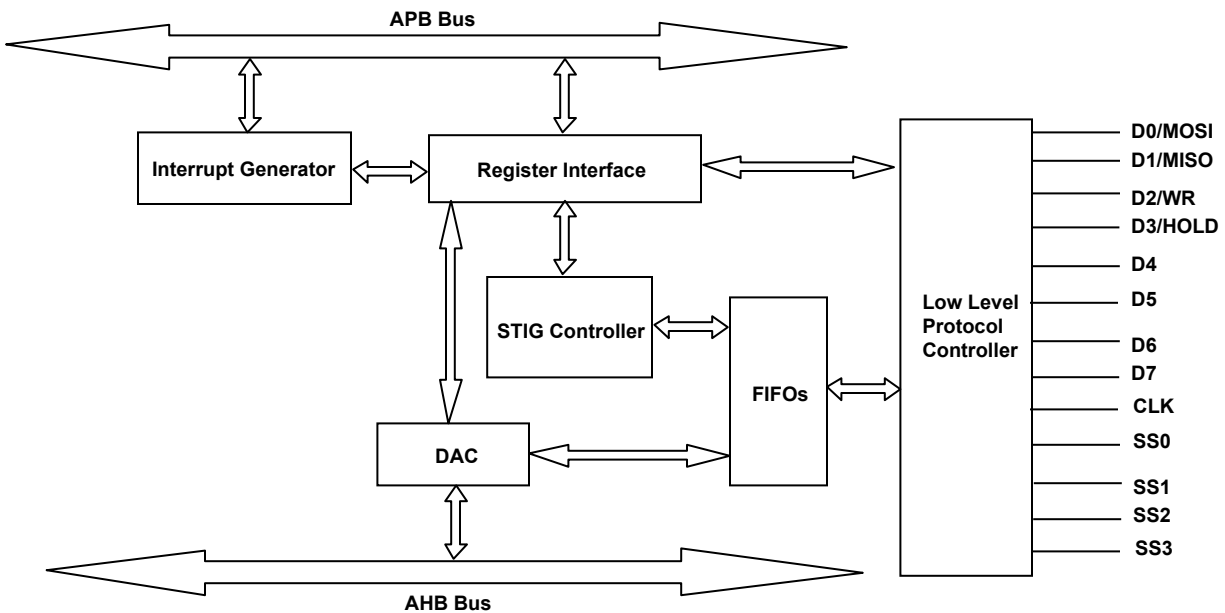


Figure 17-1: OSPI Controller Block Diagram

Architectural Concepts

There are two clock sources for the OSPI controller and one generated clock for clocking external flash device.

The SYSClk is the main system clock used to transfer data over the AHB bus between the master (core/DMA) and the OSPI controller. The SCLK0_0 clock is used to access the OSPI controller registers to perform basic configuration of the controller and interrupt handling. The SYSClk, which also acts as a reference clock, is used to serialize the data and drive the OSPI interface. The external clock driven on the OSPI_CLK pin, which is synchronous to the reference clock, is derived from the reference and divided using internal baud rate dividers. Using the reference clock, allows the core to decouple the frequency of the SPI flash device from system clocks, thereby providing a flexible clocking solution.

The OSPI controller supports two independent controllers (to support different types of flash accesses):

- Direct Access Controller (DAC)
- Software Triggered Instruction Generator (STIG)

The high-level architecture of the OSPI controller is divided into two blocks. The first is the high-level OSPI controller that interfaces with the rest of the SoC. It also queues flash transactions based on data from AHB and APB bus interfaces. The second module is the low-level SPI Protocol Controller. It is used for serializing data from high-level OSPI controller and translating them into the SPI transfer protocol where there are two possible data paths. The first path preserves backward compatibility with the previous SoCs. This path bypasses the PHY module. Use this path when there is no strict requirement for high performance (clock divider is greater or equal 4).

The second data path is through the PHY Module and is for dividers lower than 4. The configuration is set by software. The controller waits for a valid direct access from the AHB bus (which runs off SYSClk) or for a software triggered read from the APB bus. When such an event occurs, the corresponding internal controller is selected

(DAC or STIG). The flash command generator block arbitrates between accesses and forwards control into the low-level SPI module. This block works on the reference clock. Transmit data is synchronized internally to the TX FIFO and then serialized to the SPI interface. When the direction of transfer changes and the device returns data to the controller, data is read into the RX FIFO and then synchronized to the internal clocks. The SPI control logic selects the source of the data path (with or without PHY) based on the register configuration. The reference clock and the associated delayed variants are relevant for this transfer type when the PHY mode is enabled.

Direct Access Controller (DAC)

Direct access refers to the operation where AHB bus accesses targeted to the OSPI memory mapped space of processor directly triggers a read/write on flash memory. It is memory mapped and used to access and directly execute the code from external flash memory. When DAC is enabled through the `OSPI_CTL.DACEN` field, any incoming AHB access is serviced by the DAC. Direct accesses can be launched by accessing the OSPI memory address space using core or via one of the MDMA channels. In memory mapped mode, communication to a flash device is automated. The flash memory is accessible directly through reads/write of the processor address space. This access allows code to execute directly from flash devices (true XIP operations). The content can be cached for good performance.

When servicing AHB reads, the DAC sends an additional access downstream apart from the one issued by the AHB in single AHB burst. This is invisible from the system interface. It is a predicted read and ensures that the SPI core operates with maximum bandwidth. The AHB burst not defined by AMBA. It is defined as follows:

- The first access of the AHB burst is defined by:
An AHB access that is non-sequential (based on address comparison).
An AHB access that is non-sequential or sequential when the downstream modules are idle.
- The last access of the AHB burst is defined by an AHB access that is sequential and precedes a new burst.
- Size of the AHB burst is the number of AHB accesses (first access to last access).
- The number of DAC requests per AHB burst is equal to the size of the AHB burst + 1.

When AHB clock is slower than the SPI data rate, the sequential read transfer may be interrupted on the SPI side and continues after the AHB clock accepts the last data. The limiting value gets narrower as the SPI clock divider and AHB access size decrease and the number of data IOs (single, dual, quad, or octal) increase. If the system cannot meet the SPI data rate, use higher SPI clock divider. This decreases the SPI data rate, but prevents OSPI accesses to external flash to break in between and pass the *Opcode-Address-Dummy* sequence each time. This increases the overall performance by keeping the data transfer continuous.

The controller is designed to work at high data rates. This predicted read results in redundant data during mixed AHB accesses on slow system data rate. Mixed accesses implies that direct AHB sequence of single non-sequential access following few sequential accesses. The predicted read cannot be dropped because of SPI transfer interruption. It is sent as a separate SPI transaction which creates redundant data on the interface for one access. To avoid this issue, drop this data by system or introduce wait states on AHB before issuing a non-sequential direct AHB access.

For AHB writes, the DAC triggers a sequence of write commands that mimic the way reads are processed, though the number of DAC write requests are equal to the number of AHB write requests received. The AHB controller ensures that flash bursts do not break the flash page boundary. When a page boundary is detected, only the number

of byte accesses up to that boundary are forwarded. A sequential direct write request that crosses a page boundary must be detected as non-sequential, causing the controller to force the flash device to enter a self-timed page program cycle. The controller supports splitting the writes crossing the page boundaries only for word-aligned addresses.

The external AHB master supplying data for writes must guarantee write data for a particular page that can be provided to the controller in a timely fashion to avoid downstream starvation. If there is a large delay in the system issuing a sequential write, flash write cycle may be prematurely initiated, reducing the device life time.

Flash erase operations, which may be required before a page write, are triggered by software using the STIG. Once a page program cycle has been started, the OSPI controller automatically polls for write cycle to complete before allowing any AHB accesses to complete. This is achieved by holding the subsequent AHB direct accesses in wait state.

Software Triggered Instruction Generator (STIG)

DAC is primarily used to transfer data. To access the volatile and nonvolatile configuration registers, the legacy Flash SPI Status register, other status/protection registers and perform erase operations, a separate software controller is required.

The STIG is controlled using the `OSPI_FCCTL` register by setting up the command to issue to the flash device. This is a generic controller used to perform instructions that the flash device supports from the extended SPI protocol. The STIG controller command is sent to the flash device and written to the software in the `OSPI_FCCTL.OPCODE` field. This is different from the `OSPI_DRICTL.OPCODERD` and `OSPI_DWICTL.OPCODEWR` fields, as these fields do not impact the STIG operation.

Configuring instructions that are not compliant with the flash specifications can cause unpredictable behavior of the controller. Bit 0 is used to trigger the command, bit 1 used by software to poll the status of the command execution. For reads, when the command has been serviced (bit 1 toggles from 1 to 0), up to 8 bytes of read data is placed in the `OSPI_FCRD_LWR` and `OSPI_FCRD_UP` registers. For writes, data must be placed in the `OSPI_FCWD_LWR` and `OSPI_FCWD_UP` registers. The STIG completion request can be also checked by the corresponding interrupt. The interrupt indicates that the controller is ready to accept a new STIG request. The STIG completion request is not equivalent to completion on the SPI side. For example, if STIG is configured to the data command only on transmit, data is taken from the corresponding STIG register fields and put into TX FIFO. As all write bytes are known, another STIG can be queued before completing serialization of the current one.

There are few commands that require more data to read than 8 bytes (for example, READ ID command). Additional STIG memory bank is implemented to accommodate this data, if required. The STIG memory bank is controlled by the `OSPI_FCMCTL` register. If enabled, the number of bytes to read in the STIG is extended to 16 bytes, as defined in the `OSPI_FCMCTL.RDSZ` field. There are few commands (excluding read arrays that are not intended to be handled in STIG mode, but in Direct Access Mode) that return more than 8 bytes to the controller.

If the number of read bytes in the STIG, as defined in the `OSPI_FCMCTL.RDSZ` field, exceeds the memory bank depth, the remaining data overwrites the STIG memory bank locations starting from its first address. When memory bank is enabled, the `OSPI_FCRD_LWR` and `OSPI_FCRD_UP` registers keep the last 8 bytes read from the flash device by the STIG. For example, to get a single byte from the last eight bytes of a long continuous read SPI data chain, there is no need to access the STIG memory bank as data can be obtained from the `OSPI_FCRD_LWR` and

`OSPI_FCRD_UP` registers. To access more data, STIG memory bank data request must be triggered. It is controlled by the `OSPI_FCMCTL` register and works analogously to trigger the STIG. Bit 0 (`OSPI_FCMCTL.TRIGREQ`) is used to trigger the command. Bit 1 (`OSPI_FCMCTL.BNKREQ`) used by software to poll the status of the command execution. When bit 1 toggles from 1 to 0, the data byte (`OSPI_FCMCTL.BNKDATA`) from the corresponding address (`OSPI_FCMCTL.BNKADDR`) is valid. The address must be set before triggering the STIG memory bank access. Each consecutive STIG access overwrites the previous one so that the data in the bank fits into byte index fetched by the last STIG access configured to use the memory bank (first incoming byte equals first address of the memory bank, second one equals the second address and so on).

NOTE: In ADSP-21569, the STIG memory bank depth is 16 bytes.

Servicing STIG Request

The OSPI controller determines the number of bytes in the `OSPI_FCCTL` register to be sent to the flash device, when there is a STIG request. The `OSPI_FCCTL.OPCODE` field indicates the instruction to be sent and is always pushed first. If there is an address to send, the address (size is also programmed in the same register) is sent next. The address is stored in the `OSPI_FCA` register. If there are dummy cycles (size of which is also programmed in the `OSPI_FCCTL` register), they are sent next.

For write data (size of which is also programmed in the `OSPI_FCCTL` register), up to 8 bytes can be sent (as stored in the `OSPI_FCWD_LWR` and `OSPI_FCWD_UP` registers). When read data is collected from the flash device, the OSPI controller stores it in the `OSPI_FCRD_LWR` and `OSPI_FCRD_UP` registers.

Arbitration Between DAC and STIG Access

When multiple controllers are simultaneously active, a simple, fixed priority arbitration scheme is used to arbitrate between each interface and to access the external flash.

The fixed priority is defined as follows, highest priority first:

1. Direct Access Write
2. STIG Access
3. Direct Access Read

When one controller is servicing the request, the other controller is back pressured while waiting to be serviced.

Auto Polling for DAC Write Access

For DAC write accesses, which initiate flash program operation, the OSPI controller provides an option to automatically poll the flash status for program completion. When a flash write operation is triggered through DAC, the OSPI controller sends the write data up to page size defined in the `OSPI_DSCTL.PGSZ` field. Once a page boundary is reached, the DAC stops sending data to the flash device as the flash device enters the program mode. Once the device is in program mode, the OSPI controller can automatically send the flash status read commands to check the state of the flash.

The `OSPI_WCCTL` register defines the setting to poll the flash device correctly. By default, it is configured to poll bit 0 of the Device STATUS register (using opcode 0x05), which is common across all devices to indicate Write in Progress (WIP). However, in some devices (such as Micron devices, where size is > 512 MB), it is required that the

controller polls a different bit of a different device register. This means the controller must issue a different command during this polling phase. For example, in Micron N25Q devices, it is bit 7 of the FLAG STATUS register (opcode 0x70) instead of bit 0 of the STATUS register that must be polled. For this reason, the `OSPI_WCCTL` register has been provided for software to determine the bit and the opcode to use to poll for write completion, and also the number of successive valid polls that should take place.

To ensure that the bandwidth of the device is not affected by continuous read status SPI transactions, prolong the delay between the successive flash status read command by programming the `OSPI_WCCTL.REPDLY` field. This feature is implemented to free up the SPI bandwidth, if required. However, defining this delay is not always desired as the ready bit indication can come later what impacts the overall performance. This register must be setup while the controller is idle.

When the OSPI controller starts to service a STIG request, it sets the `OSPI_FCCTL.STAT` bit to indicate a command execution is in progress. When the OSPI controller is in the auto polling state, servicing a STIG request is different. Several devices are inaccessible after a program operation until the device has completed that write. Few of them have a possibility to suspend the programming page. It can be controlled by the `OSPI_POLSTAT.STAT` bit, which indicates active auto polling phase. After requesting a STIG, the OSPI controller issues an appropriate opcode to memory. While servicing a STIG (in auto polling phase), the `OSPI_FCCTL.STAT` bit remains steady and other parts of transfer such as address or dummy bits are disabled (to issue program suspend command, only opcode is needed).

SPI Command Translation

Requests issued by the DAC or STIG are translated into a sequence of byte transfers to send downstream (before serializing the flash device). These sequences depend on the requested transfer, except the following example (1-byte non-sequential read):

INSTRUCTION OPCODE > ADDRESS > MODEBYTE > DUMMY CYCLES > 1-DATA BYTE

For sequential accesses, an extra data byte per read is pushed to the flash device on the back of the above sequence, assuming that it can be done without any gap between each transferred byte.

The actual sequence sent to the flash device depends on the requested transfers (non-sequential or sequential), whether the device is configured in XIP mode or not, and the state of the `OSPI_DRICTL` and `OSPI_DWICTL` registers. For writes, the write enable latch (WEL) within the flash device must be high before a write sequence can be issued. The OSPI controller automatically issues a WEL command before triggering a write command via DAC (user need not perform this operation). To increase flexibility and performance, turn off this feature by setting the `OSPI_DWICTL.WELDIS`. The opcode for WEL is 0x06 and common between the devices. However, user can program this using the `OSPI_OE_LWR.XRDBYT/OSPI_OE_UP.FSTBYTWEL` and optionally `OSPI_OE_LWR.XWRBYT/OSPI_OE_UP.XBYTWEL` (flash device requires 2-byte command code for WEL).

When write requests from the DAC are not received and all outstanding requests have been sent, the flash device automatically starts the page program write cycle. Any incoming request at this time is held in wait state until the cycle is completed. The OSPI controller automatically polls the flash device legacy SPI status register to identify when the write cycle has completed. This is achieved by sending the Read Status Register (RDSR) opcode to the flash device and waiting until the device indicates that the write cycle is completed (the Write in Progress bit is

cleared to zero and WEL bit is cleared to zero or device ready bit is set to one). The WREN and RDSR device instructions are only sent by the controller. For other specific instruction that the user determines must be sent to the device (for example, if the device needs to be unprotected before a write command is issued), has to be separately handled by issuing flash commands via the STIG.

Hold and Reset Control

There is an option to trigger hold and reset features on the I/Os of the flash device. The hold functionality is common across the devices and takes an alternative function of the DQ3 pin (applicable when device does not operate in quad SPI or DDR modes). The transfer can be held and resumed by the `OSPI_CTL.HLD` bit. Devices with hold feature on DQ3 need another dedicated pin for hardware reset. Devices without hold feature have an alternative reset on the DQ3 pin, which makes the additional reset pin redundant. The controller supports both variants. The `OSPI_CTL.RSTCFG` field allows the user to configure the hardware reset solution implemented in the device. After configuration, it is possible to trigger hold or reset features using I/Os (`OSPI_CTL.HLD` or `OSPI_CTL.RST` bits).

NOTE: The OSPI controller does not support dedicated reset pin to control the reset of flash devices. Flash devices with reset functionality on DQ3 can be controlled directly via OSPI controller.

After hold activation, the controller is in waiting state. Any other operations must not be requested before de-asserting the `OSPI_CTL.HLD` bit. The hold feature is useful when any SPI transaction needs to be prolonged to adjust it into specific point in time. To check if the hold request is suspended, the `OSPI_CTL.IDLE` bit can be polled for. If SPI is not in the idle state, the transfer is suspended. Software must reset the `OSPI_CTL.HLD` bit before any new SPI transaction. In case hold request is set before starting any transfer, it will be in hold state after it starts.

Hardware reset must be activated when CS is high (no valid transaction is present on SPI bus). It can be checked by polling the `OSPI_CTL.IDLE` bit. If the controller is in the idle state and no other transfer requests are queued, hardware reset can be triggered. Reset feature is useful when any write, program, or erase operation needs to be cancelled. No transfer request is permitted before driving the reset back to inactive. Triggering hold or reset on the DQ3 pin at the time the device is configured to work in Quad SPI or DDR mode, overwrites transfer data on the DQ3 with 0. This behaviour is considered as a software error. Ensure that the flash device is introduced to suitable SPI Mode (by polling its Configuration Register) before triggering alternative DQ3 function.

Flash Instruction Type Support

To send correct read and write opcodes, software must initialize the `OSPI_DRICTL` and `OSPI_DWICTL` registers. These registers include fields to setup the required instruction opcodes that is intended to be used to access the flash (default is basic read and basic page program) as well as the instruction type, edge mode (DDR or SDR) and whether the instruction uses single, dual, quad, or octal pins for address and data transfers.

To ensure that the controller can operate from a reset state, the registers are reset to an opcode compatible with single I/O standard SPI devices. Though it is applicable for both reads and writes, the `OSPI_DRICTL.INSTRTYP` field appears only once (not included in the `OSPI_DWICTL` register). If software sets this to any value other than 0, the `OSPI_DRICTL.ADDRTRNSFR/OSPI_DWICTL.ADDRTRNSFR/OSPI_DRICTL.DATATRNSFR/OSPI_DWICTL.DATATRNSFR` bits become don't care. It is made available to

allow software to support less common flash instructions, where opcode, address, and data are sent on 2 or 4 data lines (opcode from several instructions are sent serially to the flash device, even for dual/quad instructions).

Dual Data Rate (DDR) Operations

The OSPI controller supports DDR mode of operation, in which some or all transfer phases happen in DDR mode, where the I/O lines are driven/sampled on both rising and falling edges of the clock.

In DDR mode, the OSPI controller supports the following modes:

- Dual Transfer Rate (DTR) mode, where command is driven in SDR mode and address/dummy cycles/data are driven in DDR mode.
- DTR protocol mode, where everything including command is driven in DDR mode.

For DTR mode, there are specific commands which are recognized by the flash as DTR commands. These are DTR Read commands which can be handled in STR Protocol (but in DTR Mode). This is achieved by setting the `OSPI_CTL.DTREN` bit to 0 and `OSPI_DRICTL.DDREN` bit to 1.

For DTR protocol mode, all normal commands can be sent in DDR fashion, provided the flash is already configured in a mode to accept the commands in DDR mode. Support for DTR protocol provides significant enhancement. This is enabled in the OSPI controller by setting the `OSPI_DRICTL.DDREN` bit to 1. It allows the device to work in DTR mode for each possible command (not only DTR read ones). It can also handle opcode phase in DTR mode, which improves the overall performance.

There are several devices (for example, Micron N25Q512A) that can handle read operations in DTR mode. They can issue and capture data on both rising and falling edges while working with a dedicated command type. This enables the controller to maintain same throughput twice the lower frequency of `SPI_CLK` for STR mode.

In MT25Q family of Micron devices, DTR protocol is implemented. It enables device to handle all commands in DTR mode. DTR Read commands detect DTR mode based on dedicated opcode. Therefore, opcode has to be sent as STR. When DTR protocol is enabled, the device does not need opcode to detect edge mode, as it can recognize based on the volatile or nonvolatile bit in the flash configuration register.

The table illustrates how software must configure the controller for selected specific read and write instruction supported by the different devices.

Table 17-3: Read Configuration Examples

Command	Opcode (Number of Lanes/ Edge Mode)	Address/ Dummy/ Mode (Number of Lanes/ Edge Mode)	Data (Number of Lanes/ Edge mode)	Instruction Type <code>OSPI_DRICTL.INSTRYP</code>	Address Transfer Type <code>OSPI_DRICTL.ADDRTRNSFR</code>	Data Transfer Type <code>OSPI_DRICTL.DATATRNSFR</code>	DDR Bit Enable <code>OSPI_DRICTL.DDREN</code>
READ	1/SDR	1/SDR	1/SDR	0	0	0	0
FAST_READ	1/SDR	1/SDR	1/SDR	0	0	0	0
DTR FAST_READ	1/SDR	1/DDR	1/DDR	0	0	0	1

Table 17-3: Read Configuration Examples (Continued)

Command	Opcode (Number of Lanes/ Edge Mode)	Address/ Dummy/ Mode (Number of Lanes/ Edge Mode)	Data (Number of Lanes/ Edge mode)	Instruction Type OSPI_ DRICTL. INSTRYP	Address Transfer Type OSPI_ DRICTL. ADDRTRNSFR	Data Transfer Type OSPI_ DRICTL. DATATRNSFR	DDR Bit Enable OSPI_ DRICTL. DDREN
DOR (Dual output Fast Read)	1/SDR	1/SDR	2/SDR	0	0	1	0
DTR DOR (Dual output Fast Read)	1/SDR	1/DDR	2/DDR	0	0	1	1
DIOR (Dual I/O Fast Read)	1/SDR	2/SDR	2/SDR	0	1	1	0
DTR DIOR (Dual I/O Fast Read)	1/SDR	2/DDR	2/DDR	0	1	1	1
QOR (Quad output Fast Read)	1/SDR	1/SDR	4/SDR	0	0	2	0
DTR QOR (Quad output Fast Read)	1/SDR	1/DDR	4/DDR	0	0	2	1
QIOR (Quad I/O Fast Read)	1/SDR	4/SDR	4/SDR	0	2	2	0
DTR QIOR (Quad I/O Fast Read)	1/SDR	4/DDR	4/DDR	0	2	2	1
OOR (Octal output Fast Read)	1/SDR	1/SDR	8/SDR	0	0	3	0
DTR OOR (Octal output Fast Read)	1/SDR	1/DDR	8/DDR	0	0	3	1
OIOR (Octal I/O Fast Read)	1/SDR	8/SDR	8/SDR	0	3	3	0
DTR OIOR (Octal I/O Fast Read)	1/SDR	8/DDR	8/DDR	0	3	3	1
DCFR (Dual Command Fast Read)	2/SDR	2/SDR	2/SDR	1	don't care	don't care	0

Table 17-3: Read Configuration Examples (Continued)

Command	Opcode (Number of Lanes/ Edge Mode)	Address/ Dummy/ Mode (Number of Lanes/ Edge Mode)	Data (Number of Lanes/ Edge mode)	Instruction Type OSPI_DRICTL. INSTRTYP	Address Transfer Type OSPI_DRICTL. ADDRTRNSFR	Data Transfer Type OSPI_DRICTL. DATATRNSFR	DDR Bit Enable OSPI_DRICTL. DDREN
DTR DCFR (Dual Command Fast Read)	2/SDR	2/DDR	2/DDR	1	don't care	don't care	1
QCFR (Quad Command Fast Read)	4/SDR	4/SDR	4/SDR	2	don't care	don't care	0
DTR QCFR (Quad Command Fast Read)	4/SDR	4/DDR	4/DDR	2	don't care	don't care	1
OCFR (Octal Command Fast Read)	8/SDR	8/SDR	8/SDR	3	don't care	don't care	0
DTR OCFR (Octal Command Fast Read)	8/SDR	8/DDR	8/DDR	3	don't care	don't care	1

NOTE: This data is applicable for 3-byte and 4-byte address variants of the commands.

In DTR protocol, all transfer phases (including opcode) take DDR edge mode independent of the command under execution. DTR protocol is enabled by the OSPI_CTL.DTREN bit. It has higher priority than the OSPI_DRICTL.DDREN bit. Therefore, if the OSPI_CTL.DTREN bit is set, irrespective of value programmed in the OSPI_DRICTL.DDREN bit, all command, address, dummy cycles, and data are transferred in DDR mode.

Table 17-4: Write Configuration Examples

Command	Opcode (Number of Lanes)	Address/ Dummy/ Mode (Number of Lanes)	Data (Number of Lanes)	Instruction OSPI_DRICTL. INSTRTYP	Address Transfer Type OSPI_DWICTL. ADDRTRNSFR	Data Transfer Type OSPI_DWICTL. DATATRNSFR
PP (Page Program)	1	1	1	0	0	0
DIFP (Dual Input Fast Program)	1	1	2	0	0	1

Table 17-4: Write Configuration Examples (Continued)

Command	Opcode (Number of Lanes)	Address/Dummy/Mode (Number of Lanes)	Data (Number of Lanes)	Instruction OSPI_DRICTL .INSTRTYP	Address Transfer Type OSPI_DWICTL .ADDRTRNSFR	Data Transfer Type OSPI_DWICTL .DATATRNSFR
DIEFP (Dual Input Extended Fast Program)	1	2	2	0	1	1
QIFP (Quad Input Fast Program)	1	1	4	0	0	2
QIEFP (Quad Input Extended Fast Program)	1	4	4	0	2	2
OIFP (Octal Input Fast Program)	1	1	8	0	0	3
OIEFP (Octal Input Extended Fast Program)	1	8	8	0	3	3
DCPP (Dual Command Page Program)	2	2	2	1	don't care	don't care
QCPP (Quad Command Page Program)	4	4	4	2	don't care	don't care
OCPP (Octal Command Page Program)	8	8	8	3	don't care	don't care

NOTE: This data is applicable for both 3-byte or 4-byte address variants of the commands.

In DTR protocol, all transfer phases (including opcode) take DDR edge mode independent of the command under execution.

Data Sampling

The OSPI provides flexibility to configure the sampling point of read data with the SPI clock to meet setup and hold timings of the flash devices when operating at higher speed. The `OSPI_RDC` register provides an option to control the sampling point. In OSPI controller, data received on the OSPI pins is sampled by internal reference clock (SYSCLK). In STR mode, by default (when the `OSPI_RDC.DLYRD` field is set to 0), the incoming data is on the first falling edge of the SYSCLK in second half of SPI clock. In DTR mode, it is on the first falling edge of the SYSCLK in second quarter of SPI clock.

The `OSPI_RDC.SMPLEDG` bit selects the edge of the SYSCLK, in which data from flash memory are sampled. This increases the sampling resolution by two times. The `OSPI_RDC.DLYRD` field controls the additional number of SYSCLK cycles that must be applied to the internal read data capture circuit. Large clock-to-out delay of the flash

memory, trace delays, and other device delays may impose an upper limit on the flash clock frequency which is less for the flash memory to operate. To compensate this, software must set this register to a value that guarantees robust data captures.

For example, if baud divider (`OSPI_CTL.BAUD` field) is programmed to 7, it results in divider of 16. Therefore, `SPI_CLK: SYSCLK = 16:1` (every 1 `SPI_CLK` cycle has 16 `SYSCLK` cycles). Assuming that the `OSPI_RDC.DLYRD` bits are programmed to X,

In STR mode of operation, received data is captured on:

- $(16/2) + X^{\text{th}}$ falling edge of `SYSCLK` if `OSPI_RDC.SMPLEDG = 0`
- $(16/2) + X^{\text{th}}$ rising edge of `SYSCLK` if `OSPI_RDC.SMPLEDG = 1`

In DTR mode of operation, received data is captured on:

- $(16/4) + X^{\text{th}}$ falling edge of `SYSCLK` if `OSPI_RDC.SMPLEDG = 0`
- $(16/4) + X^{\text{th}}$ rising edge of `SYSCLK` if `OSPI_RDC.SMPLEDG = 1`

To improve the hold timing during transfers in DTR mode, the `OSPI_RDC.DDRDLYRD` delays the transmitted data by programmable number of `SYSCLK` cycles.

NOTE: The OSPI interface for Octal DTR mode may not work at higher frequencies for flash devices that require DQS.

PHY Module Architecture

The PHY Module is divided into the following four sub-blocks:

- Data Transmitter

The data transmitter serializes the transmit data to assign complaint signal values into the master output interface. The input data size is 2-bytes in order to cover the octal DDR scenario where two bytes are sent within a single reference clock cycle. The configuration interface covers selected SPI protocols (single, dual, quad, or octal) or command types (DDR, SDR). These values are necessary to calculate the number of bits that fit into a clock cycle and consequently what set value must be set by the hardware.

- Data Receiver

The data receiver block samples data from flash device.

- Delay Line Module

The delay line module block provides an appropriate delay for the input clock to ensure correct data transmitting and sampling. This block contains two separate delay paths for the OSPI clock, one feeding the external FLASH device and the other feeding the internal PHY sampling clock used for capturing data from the device when the PHY mode is enabled. Delay line values are configurable in software.

- Clock Arbiter

The clock arbiter handles gating the clock logic for clocks forwarded to the DLL inputs.

The PHY Module Block Diagram shows the sub-blocks in the PHY module. The sections following the diagram describe the architecture of the sub-blocks in more detail.

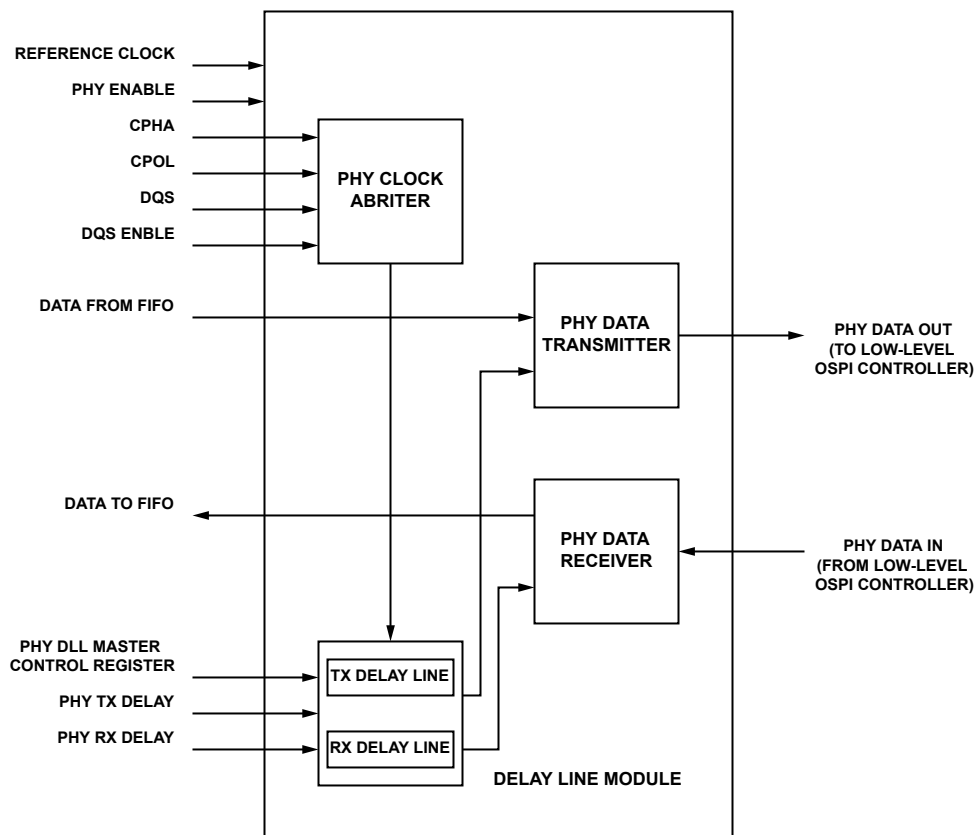


Figure 17-2: PHY Module Block Diagram

PHY Data Transmitter

The data transmitter latches queued data to transmit on both the negative and positive edges of the reference clock. Then data is chopped to fit into the output interface. With the PHY module, in the last stage of the TX data path data from both edges are connected, forwarded back to the low-level OSPI controller, and then put on the OSPI master outputs if the PHY enable bit is asserted. Otherwise, the master outputs generated by the low-level OSPI controller are selected.

PHY Data Receiver

In the data receiver module, data is sampled on both edges of the delayed reference clock (sampling clock) ensuring a wide range of the sampling window. Read data is gathered in shift registers and then merged into a byte-sized input FIFO. Data reception is programmable to work either with the delayed reference clock or with the DQS input signal from the flash device.

Delay Line Module

Due to the asynchronous nature of flash devices, it is important to address the timing requirements for capturing and receiving data between the controller and the flash. The PHY module contains logic to meet the timing requirements. The delay compensation circuit is designed with the following features:

- Programmable OSPI clock delay specified as a percentage of a clock cycle
- Programmable sampling clock delay specified as a percentage of a clock cycle
- Delay compensation circuit re-sync circuitry activated during refresh cycles to compensate for temperature and voltage drift

The delay compensation circuitry relies on a master/slave approach. The master delay line is used to determine how many delay elements constitute a complete cycle. This count, along with the programmable fractional delay settings, determines the actual number of delay elements to program into the slave delay lines. The master and slave delay lines are identical. This approach allows the memory controller to observe a clock and then delay other signals by a fixed percentage of that clock.

DLL Locking is controlled through the register interface. When the DLL is reset, the master DLL performs a locking procedure starting with the DLL start point value in the `OSPI_PHYMCTL.INITDLY` bit field and the current frequency of operation. The DLL start point value must not exceed a delay of 1 and 1/4 cycles of delay, when calculated by estimating the worst case timing through a delay element at the highest frequency the DLL will be operating at. For example, if the maximum operating frequency is 200 MHz (period of 5 ns) and the worst case delay element has a 85 ps delay, then program the DLL start point to $5 / 0.085 = 58.8 = 59$ elements = 0x3B.

With this setting, the master DLL is will correctly lock for all frequencies below 200 MHz and in any process corner. If the delay provided by the delay line is enough to cover a full clock cycle, the master DLL is in full clock mode. In this case, the DLL lock value in the `OSPI_DLLOB_LWR.LWRLOCK` bit field reports the number of delay element in one full clock cycle. The slave delay line fractional setting uses this number to determine the number of delay elements to add to the slave delay lines. For example, if the DLL lock value = 50 = 0x32, then the slave delay line percentage = 25% = 0x20, and the slave delay line delay value = $50 * 0.25 = 12.5$ elements. The DLL Module will round the calculated value into an integer number of delay elements.

If the frequency of operation is such that the delay line is not long enough to accommodate a full cycle of delay, the master DLL will automatically detect this and switch to half clock mode. In this mode, the master DLL attempts to lock when the delay in the delay line reaches a half-clock cycle. If lock is achieved in half clock mode, the slave delay lines are automatically adjusted to program a fractional delay of a full cycle. There is no need to change the slave delay settings based upon the lock mode of the master DLL. For example, if the DLL lock value = 50 = 0x32, then the slave delay line percentage = 25% = 0x20, and the slave delay line delay value = $(50 * 2) * 0.25 = 25$ elements. Do not work in half clock mode when using a loop-back sampling method or when the SPI clock is configured to SPI Mode 3 (CPHA = 1, CPOL = 1).

If the frequency of operation is such that the delay line is not long enough to capture a half-clock cycle of delay, the master DLL will indicate lock and set the number of delays to the maximum length of the delay line. This is called saturation mode. There is no need to change the slave delay settings in this mode. The slave delay settings will be fixed at the fractional delay based upon the maximum delay of the delay line times 2. For example, if the DLL lock

value = MAX = 128 = 0x80, and the slave delay line percentage = 25% = 0x20, then the slave delay line delay value = $(128 \times 2) \times 0.25 = 64$ elements.

PHY Clock Arbiter

The OSPI clock is generated only when the SPI control logic FSM indicates that an SPI transfer is ongoing. Conversely, the reference clock is continuously generated. This module generates a sampling clock that is ready to pass through the RX delay line. It is generated when read data phase of SPI transfer is detected based on information from the SPI control module. The sampling clock is optionally sourced from DQS or loop-back input.

PHY Pipeline Mode

The PHY pipeline mode is for a direct read mode of operation. The flash command generator pipelines and puts a few expected sequential accesses into the TX FIFO causing the CS to stay active since the low-level OSPI controller controls the internal TX FIFO fill level. To correctly trigger a direct read in pipeline mode, first poll the `OSPI_CTL.IDLE` bit to determine that the TX FIFO is empty. The sequential data transfer is interrupted when non-sequential access occur or too many wait states are introduced by the AHB master to keep the flash transaction continuous. Introducing wait states slows down the system data rate.

Enable this mode when following conditions are met:

- $\text{SYSCLK} > 1.1 \times \text{Reference clock}$

NOTE: In PHY mode the reference clock is taken from the CDU CLK010

- Accesses are only 32-bits in size

Programming Concepts

Software configures the OSPI controller before communicating with the flash device. The static configuration bits must be setup before the OSPI controller is enabled using the `OSPI_CTL.EN` field. To change the controller configuration, disable it before reconfiguring.

Configuring OSPI after Reset

The OSPI controller comes up with a state that is suitable to perform basic reads and writes using the DAC. Basic read (opcode 0x03) and write (opcode 0x02) instructions are supported by all target devices. The controller also wakes up with a baud rate divider setting of 32. Assuming that the SYSCLK (reference clock) operates at 500 MHz after reset, the effective SPI clock is 15.625 MHz. This must be slow enough to meet all timing requirements of all target devices without any further device programming.

If the target device does not use 3 address bytes, the `OSPI_DSCTL` register must be modified to the appropriate size. If software wants writes to the device and the number of bytes per device page is not equal to 256, the `OSPI_DSCTL` register must also be modified. The software must enable the write protect feature prior to enabling the OSPI controller. This blocks any AHB writes from taking effect. The `OSPI_WRPROT_LWR`, `OSPI_WRPROT_UP`, `OSPI_WRPROT_CTL`, and `OSPI_DSCTL.PGSZ` fields must be setup.

After reset, software can read from and write to the flash device. Enabling/disabling the controller and DAC is achieved by writing to the corresponding `OSPI_CTL` register fields. Maintain the default values of the baud rate divisor and default state of the `OSPI_CTL.CPOL/OSPI_CTL.CPHA` bits. A write data value of 0x00780081 is recommended.

Programming Dummy Cycles

When programming the dummy cycles for OSPI reads (DAC and STIG-initiated reads) from the flash device, mode data should be considered as it impacts the actual number of dummy cycles seen on the bus. If mode data is not enabled, then the number of dummy cycles is that same as those set in dummy cycles field, respectively, for STIG or DAC operation.

However, if mode data is enabled, then in addition to the programmed number of dummy cycles, a few extra cycles are needed to transmit out the mode data. The number of cycles added depends on the bus mode of the OSPI. For example, if the address phase of the command is configured to be sent on a single line in STR mode, then the mode bits consume eight cycles. If the address phase of the command is configured to be sent on four lines in DTR mode, then the mode bits consume one cycle. These extra cycles should be considered when configuring the dummy cycles for the controller so the overall dummy clock cycles match the dummy cycles provided in flash data sheet.

Configuring OSPI for Optimal Use

Software must accurately configure the controller to optimally access the flash.

To configure the controller:

1. Wait until pending STIG are completed or poll the `OSPI_CTL.IDLE` field.
2. Disable the `OSPI_CTL.DACEN` field. The OSPI controller can be completely disabled using the `OSPI_CTL.EN` field.
3. Update the `OSPI_DLY` register. This register allows the user to tweak how the chip select is driven after each flash access. This is required as each device may have different timing requirements. As the serial clock frequency is increased, these timing requirements become critical. The numbers programmed in this register are based on the period of reference clock (SYSCLK).

For example, an ATMEL device needs 50 ns minimum time before CS can be re-asserted after it has been de-asserted. By default, the controller provides a minimum of 1 SCLK period. When the device is operating at 50 MHz, the SCLK period is only 20 ns. Therefore, additional 30 ns are required. As the register defines the number of SYSCLK cycles to add, if SYCLK is running at 500 MHz (2 ns period), user must program a value of at least 15 to the `OSPI_DLY.DSRT` bits. This delay can be extended during auto-polling phase. Polling repetition delay can be defined in the `OSPI_WCCTL.REPDLY` field.

4. Update the `OSPI_DSCTL` register. The number of bytes per page is required to perform any write operation on flash. The number of bytes per device block is only required if the write protect feature is used.
5. Setup and enable the write protection registers (`OSPI_WRPROT_LWR`, `OSPI_WRPROT_UP`, and `OSPI_WRPROT_CTL`), if write protection features are required.

6. Enable the required interrupts using the `OSPI_IMSK` register.
7. Update the `OSPI_REMAPADDR` register to remap the DAC addresses to different address in flash.
8. Set the baud rate divisor in the `OSPI_CTL.BAUD` to define the required clock frequency of the target device.
9. Select the appropriate chip select signal to use through the `OSPI_CTL.BAUD` bits.
10. Update the `OSPI_RDC` register. This register delays when read data is captured and helps when read data path from the device to the controller is long and the device clock frequency is high.
11. Enable the OSPI controller and DAC using the `OSPI_CTL` register.

Configuring OSPI for DAC Read Operation

1. Configure the `OSPI_DRICTL` register:
 - a. Program the desired opcode for Read command (0xBB for Dual SPI read).
 - b. Configure the instruction type, address transfer type, and data transfer type as per the opcode.
 - c. Configure the dummy cycles for the selected command opcode depending on the flash device.
 - d. Set the Mode bit Enable if the mode data needs to be transferred during dummy cycles phase.
 - e. Set the DDR enable bit if the command works in DTR mode.
2. If the operation requires dual opcode, set the `OSPI_CTL.OPCODEEN` bit.
3. Update the second byte for read opcode in the `OSPI_OE_LWR.XRDBYT` field.
4. To operate in DTR protocol mode, set the `OSPI_CTL.DTREN` bit.

If this bit is set, the `OSPI_DRICTL.DDREN` bit has no effect and all phases of command including the opcode are done in DDR mode.

5. Update the `OSPI_DSCTL.ADDRSZ` field.
6. Update the `OSPI_MBCTL.MODE` field.
7. Enable the DAC mode using the `OSPI_CTL.DACEN` bit, if not enabled.
8. Start the read transfer by accessing the OSPI memory mapped space through core or MDMA accesses. Any access to address 0x60000000 will access the flash address 0x0 if address remapping is 0.
9. If using MDMA for the transfer, wait for the DMA to complete.

NOTE: For accesses through core, software need not check FIFO levels. All the FIFOs are internally maintained by DAC.

Configuring OSPI for DAC Write Operation

1. Configure the `OSPI_DWICTL` register:

- a. Program the desired opcode for write command like 0x02 for single SPI page program.
 - b. Configure the address transfer type and data transfer type as per the opcode.
 - c. Configure the dummy cycles for selected command opcode depending on the flash device.
2. Update the `OSPI_DRICtrl.INSTRTyp` field as per the selected opcode.
3. Clear the `OSPI_DWICtrl.WELDIS` bit to manually send the WEL (Write Enable) instruction to flash. Else, the controller automatically issues WEL to flash before sending any write command to flash.
If WEL command for flash under use is not 0x06, update the `OSPI_OE_Up.FSTBYTWEL` field.
4. If the operation requires dual opcode, set the `OSPI_Ctrl.OPCODEEN` bit.
5. Update the second byte for read opcode in the `OSPI_OE_Lwr.XWRBYT` field.
6. Update the number of address bytes the command expects in the `OSPI_DSCTL.ADDRSZ` field.
7. As the controller supports the automatic polling for write operation complete, update the `OSPI_WCCTL` register accordingly to enable the automatic polling for flash program operation complete.
 - a. Enable the automatic polling by clearing the `OSPI_WCCTL.DIS` bit.
 - b. Update Polling opcode and define polling bit index and polarity as per the flash device.
 - c. Update polling count and polling repetition delay.
 - d. Optionally, update the `OSPI_POLLEXP` register if polling expiration is enabled.
8. Enable the DAC mode using the `OSPI_Ctrl.DACEN`, if not enabled.
9. Start the read transfer by accessing the OSPI memory mapped space through core or MDMA accesses. Any access to address 0x60000000 will access the flash address 0x0 if address remapping is 0.
10. Wait for the DMA to complete, if using MDMA for the transfer.

NOTE: For accesses through core, software need not check FIFO levels. All FIFOs are internally maintained by DAC.

Issuing STIG Command

This is a typical method that software uses to access the flash device registers and perform erase operations. It can also be used to access the flash array (though only 8 data bytes can be read or written at a time), as defined in the `OSPI_FCRD_Lwr`, `OSPI_FCWD_Lwr`, `OSPI_FCRD_Up` and, `OSPI_FCWD_Up` registers.

To issue a STIG command:

1. Set the `OSPI_IMSK.STIGREQ_MSK` bit to enable the STIG command completion interrupt.
2. Update the `OSPI_FCCTL.OPCODE` and `OSPI_FCCTL.DMY` bits, as per the flash command that need to be executed via STIG.
3. Clear the `OSPI_FCCTL.STIGBNKEN` bit.

4. Set the `OSPI_FCCTL.ADDREN` bit, if the command requires the address to be driven.

If set:

- a. Update the `OSPI_FCCTL.ADDRSZ` field according to the number of address bytes expected by the command.
- b. Update the `OSPI_FCA` register with the desired flash address.

5. If command expects the data to be sent to flash, set the `OSPI_FCCTL.WREN` bit.

If set:

- a. Update the `OSPI_FCCTL.WRSZ` field according to the number of data bytes to be sent to flash.
- b. Update the `OSPI_FCWD_LWR` and `OSPI_FCWD_UP` registers with data to be sent to flash.

6. If command expects the data to be received from the flash, set the `OSPI_FCCTL.RDEN` bit.

If set, update the `OSPI_FCCTL.RDSZ` field according to the number of data bytes to expected from flash.

7. Update the `OSPI_FCCTL.MODEEN` bit, if the mode data are sent during dummy cycles.

If set, update the `OSPI_MBCTL.MODE` field with desired mode data.

8. Set the `OSPI_FCCTL.EXE` bit to execute the STIG command.
9. Set the `OSPI_ISTAT.STIGREQ` bit (or wait for interrupt if enabled) indicating that the STIG command execution is completed.
10. If command expects the data to be received from the flash, read the data from the `OSPI_FCWD_LWR` and `OSPI_FCWD_UP` registers.

NOTE: Do not set the `OSPI_FCCTL.WREN` and `OSPI_FCCTL.RDEN` bits simultaneously. A given command can either read data or send data to the flash, but not both at the same time.

With normal STIG command, only 8 bytes can be read at a time, but with support of STIG memory bank up to 16 bytes can be read in to STIG memory bank. This is controlled by the `OSPI_FCCTL.STIGBNKEN` bit and `OSPI_FCMCTL` register. Commands issued using this interface have a higher priority than all other read accesses coming from AHB, and interrupts any read commands being requested by the DAC.

To issue a STIG memory bank read command:

1. Set the `OSPI_IMSK.STIGREQ_MSK` bit to enable STIG command completion interrupt.
2. Update the `OSPI_FCCTL.OPCODE` and `OSPI_FCCTL.DMY` bits as per the flash command that need to be executed via STIG.
3. Set the `OSPI_FCCTL.STIGBNKEN` bit.
4. Set the `OSPI_FCCTL.ADDREN` bit, if the command requires the address to be driven.

If set:

- a. Update the `OSPI_FCCTL.ADDRSZ` field according to the number of address bytes expected by the command.
 - b. Update the `OSPI_FCA` register with the desired flash address.
5. Update the `OSPI_FCCTL.MODEEN` bit, if the mode data are sent during dummy cycles.
If set, update the `OSPI_MBCTL.MODE` field with the desired mode data.
6. Set the `OSPI_FCCTL.EXE` bit to execute the STIG command.
7. Set the `OSPI_ISTAT.STIGREQ` bit (or wait for interrupt if enabled) indicating that the STIG command execution is completed.

At this stage, 16 bytes of data read from the flash device are available in the STIG memory bank. To access this data, a STIG memory bank read has to be issued. Each read can read one byte from STIG memory bank at a time. To read all data, 16 reads must be issued.

To read a data byte from STIG memory bank:

1. Set the `OSPI_FCMCTL.BNKADDR` field. This can be any value from 0 to 16.
2. Set the `OSPI_FCMCTL.TRIGREQ` bit to trigger the STIG memory bank read.
3. Poll the `OSPI_FCMCTL.BNKREQ` bit to clear.
4. Read the requested data byte from the `OSPI_FCMCTL.BNKDATA`.

Entering XIP mode

The controller supports XIP operations to minimize the latency for back-to-back reads or code execution.

If the flash device comes in XIP on power up, software cannot discover the state of XIP from POR via the Flash Status Register reads. The only operation of a flash device when XIP mode is enabled, is an XIP read. In such cases, software must be aware that if flash enters XIP from POR, the `OSPI_MBCTL.MODE` and `OSPI_CTL.XIPIMM` fields must be set. This makes the controller to enter XIP mode immediately and start communicating with the flash device in XIP mode on next read issued. Therefore, it does not require the read opcode to be transferred. To exit XIP mode, this bit must be set to 0. This takes effect in the attached device only after the next read instruction is executed. Software must ensure that at least one read instruction is requested after resetting this bit before it can be sure XIP mode in the device is exited.

`OSPI_CTL.XIPRD`: If it is not known that the flash device enters XIP from POR, and XIP from POR may be supported by the attached flash device, software can attempt to exit the XIP mode by issuing an XIP exit command using a STIG command. For this, software must be aware of the mode bit requirements of that device, as XIP entry and exit changes per device.

XIP mode is supported in several flash devices. However, flash manufacturers do not have a consistent standard approach. Most of them use signature bits that are sent to the device immediately following the address bytes. Few of them (such as Micron devices) use signature bits and require a flash device configuration register write to enable XIP.

The following section describes how software ensures entry into XIP mode for the flash devices compliant with the OSPI controller.

Micron N25Q, MT25Q, and MT35X Devices

XIP mode must first be enabled by setting the corresponding field of VCR within the flash device. The VCR can be written to using the `OSPI_FCCTL` register to issue a VCR write command.

1. Disable the DAC using the `OSPI_CTL.DACEN` bit to ensure that no new AHB read accesses are sent to the flash device.
2. Use STIG mode to issue a VCR write to flash memory.
3. Set XIP mode bits in the `OSPI_MBCTL.MODE` field to `8'b00000000`.
4. Enable the OSPI controller XIP mode by setting the `OSPI_CTL.XIPRD` bit.
5. Re-enable the DAC.

Micron (Supporting Basic XIP Mode), Winbond, Spansion Devices

1. Disable the DAC using the `OSPI_CTL.DACEN` bit to ensure that no new AHB read accesses are sent to the flash device.
2. Set XIP mode bits in the `OSPI_MBCTL.MODE` to:
 - `8'b10000000`, for Micron devices supporting basic XIP mode
 - `8'b00100000`, for Winbond devices
 - `8'b10100000`, for Spansion devices
3. Enable XIP mode by setting the `OSPI_CTL.XIPRD` bit.
4. Re-enable the DAC.

Using PHY mode

The OSPI PHY mode extends the architecture of the OSPI flash controller to allow the interfacing of high-speed flash devices and ensure the reliable data transfers at high speed. For non-PHY operations, the controller assumes that OSPI transfer clock is at least 4 times lower than the reference clock to ensure correct data transactions in SDR Mode and 8 times lower for DDR transactions. This provides comfortable regulation of data synchronization but it is not effective from a dynamic power efficiency standpoint.

The PHY mode allows transfers at speeds that match the increasing reference clocks of emerging higher performance flash devices, including frequencies close to 200 MHz. The PHY module works with the OSPI Flash Controller to enable DDR or SDR transfers at the device frequencies. For example, if a given flash device operates at 200 MHz, the reference clock is also 200 MHz, eliminating the need for high speed peripheral clocks.

The reference clock for OSPI is `SYSCLK`, but to enable support for PHY mode the reference clock is selected from one of the options from CDU output clock 10 (Refer to CDU chapter). By default, it is connected to `SYSCLK`

which is suitable for non-PHY mode of operation. For PHY mode, the reference clock is either SCLK0_0 or SCLK1_1.

To configure PHY mode, first implement the procedure described in the “Configuring OSPI for Optional Use” section and then proceed with the following steps based on the DLL mode configuration:

DLL Bypass Mode

1. Enable the PHY mode by setting the `OSPI_CTL.PHYEN` bit and enable the DDR protocol with the `OSPI_CTL.DTREN` bit.

NOTE: Ensure that the device is configured to work in DDR Protocol (do not confuse with DDR commands).

2. Calibrate the software using the following sub steps:
 - a. Calculate how many delay elements are necessary to shift the reference clock period by 25% and program this value in the `OSPI_PHYCTL.TXDLY` bit field. This is the best case for DDR transfers from the set-up/hold timing standpoint. The delay may vary slightly in a real design.
 - b. Resynchronize the DLLs by setting the `OSPI_PHYCTL.RESYNC` bit. If this has already been set by a previous resynchronization, toggle the bit to 0 and then back to 1 to trigger the re-synchronization DLL logic.
 - c. Enable the PHY bypass mode by setting the `OSPI_PHYMCTL.BYPCTL` bit.
 - d. Trigger a read request from a location where the value is predictable. Depending on the device, this is the parameter page, ID, status, data from the OTP region, or data from the location of the flash array.
 - e. Check for data correctness by incrementing the value of the RX clock delay using the `OSPI_PHYCTL.RXDLY` bit field, resynchronizing the DLLs, triggering a valid read request, and then checking for the correct data and store information. Continue to check until the range boundary of the RX clock delay is achieved and then proceed to the next step.
3. Set the RX clock delay value with one from the middle of valid range based on the information in storage.
4. Resynchronize the DLLs
5. Set the device read instruction register for Octal Read DDR Configuration. Configure each transfer phase to work in Octal Mode and set the number of dummy cycles to a minimum of the number specified in the documentation of the device. More dummy cycles may be set to accommodate additional read path delays in actual systems data.
6. Enable pipeline mode by setting the `OSPI_CTL.PIPEPHYEN` bit.
7. Perform a sequential read of data consistent with the conditions indicated in the PHY Pipeline Mode section.
8. Poll for IDLE with the `OSPI_CTL.IDLE` bit and when it is asserted to high trigger the next transfer request.

DLL Master Mode

1. Enable the PHY mode by setting the `OSPI_CTL.PHYEN` bit and enable the DDR protocol with the `OSPI_CTL.DTREN` bit.

NOTE: Ensure that the device is configured to work in DDR Protocol (do not confuse with DDR commands).

2. Calibrate the software using the following sub steps:
 - a. Drive the `OSPI_CTL.RST` bit low to reset the DLL.
 - b. Calculate initial delay value for the Master DLL according to the PHY DLL Master Control Register bits[6:0] and write this value into the `OSPI_PHYMCTL.INITDLY` bit field.
 - c. Set DLL reset bit (`OSPI_CTL.RST`) back to high. The master delay Line automatically starts the locking procedure with the configuration given once the DLL module is in reset state.
 - d. Poll the `OSPI_DLLOB_LWR.LOCK` bit. When it is set, the lock is done.
 - e. Resynchronize the DLLs by setting the `OSPI_PHYCTL.RESYNC` bit. If this has already been set by a previous resynchronization, toggle the bit to 0 and then back to 1 to trigger the re-synchronization DLL logic.
 - f. Set the TX DLL delay bit field (`OSPI_PHYCTL.TXDLY`) and the RX DLL Delay bit field (`OSPI_PHYCTL.RXDLY`) to the current percentage clock offsets. Wait 20 reference clock cycles for the new configuration to propagate before triggering the next SPI transfer.
 - g. Trigger a read request from a location where the value is predictable. Depending on the device, this is the parameter page, ID, status, data from the OTP region, or data from the location of the flash array.
 - h. Check for data correctness by incrementing the value of the RX clock delay using the `OSPI_PHYCTL.RXDLY` bit field, resynchronizing the DLLs, triggering a valid read request, and then checking for the correct data and store information. Continue to check until the range boundary of the RX clock delay is achieved and then proceed to the next step.
3. Set the RX clock delay value with one from the middle of valid range based on the information in storage.
4. Resynchronize the DLLs
5. Set the device read instruction register for Octal Read DDR Configuration. Configure each transfer phase to work in Octal Mode and set the number of dummy cycles to a minimum of the number specified in the documentation of the device. More dummy cycles may be set to accommodate additional read path delays in actual systems data.
6. Enable pipeline mode by setting the `OSPI_CTL.PIPEPHYEN` bit.
7. Perform a sequential read of data consistent with the conditions indicated in the PHY Pipeline Mode section.
8. Poll for IDLE with the `OSPI_CTL.IDLE` bit and when it is asserted to high trigger the next transfer request.

ADSP-2159x OSPI Register Descriptions

Octal SPI (OSPI) contains the following registers.

Table 17-5: ADSP-2159x OSPI Register List

Name	Description
OSPI_CTL	Octal SPI Control Register
OSPI_DLY	Device Delay Register
OSPI_DRICTL	Device Read Instruction Control Register
OSPI_DWICTL	Device Write Instruction Control Register
OSPI_DSCTL	Device Size Control Register
OSPI_DLLOB_LWR	DLL Observable Register (Lower)
OSPI_DLLOB_UP	DLL Observable Register (Upper)
OSPI_FCA	Flash Command Address Register
OSPI_FCCTL	Flash Command Control Register
OSPI_FCMCTL	Flash Command Control Memory Register
OSPI_FCRD_LWR	Flash Command Read Data Register (Lower)
OSPI_FCRD_UP	Flash Command Read Data Register (Upper)
OSPI_FCWD_LWR	Flash Command Write Data Register (Lower)
OSPI_FCWD_UP	Flash Command Write Data Register (Upper)
OSPI_IMSK	Interrupt Mask Register
OSPI_ISTAT	Interrupt Status Register
OSPI_WRPROT_LWR	Lower Write Protection Register
OSPI_MBCTL	Mode Bit Control Register
OSPI_MODID	Module ID Register
OSPI_POLLEXP	Polling Expiration Register
OSPI_OE_LWR	Opcode Extension Register (Lower)
OSPI_OE_UP	Opcode Extension Register (Upper)
OSPI_PHYCTL	PHY Control Register
OSPI_PHYMCTL	PHY DLL Master Control Register
OSPI_POLSTAT	Polling Flash Status Register
OSPI_RDC	Read Data Capture Register
OSPI_REMAPADDR	Remap Address Register
OSPI_WRPROT_UP	Upper Write Protection Register
OSPI_WRPROT_CTL	Write Protection Control Register

Table 17-5: ADSP-2159x OSPI Register List (Continued)

Name	Description
OSPI_WCCTL	Write Completion Control Register

Octal SPI Control Register

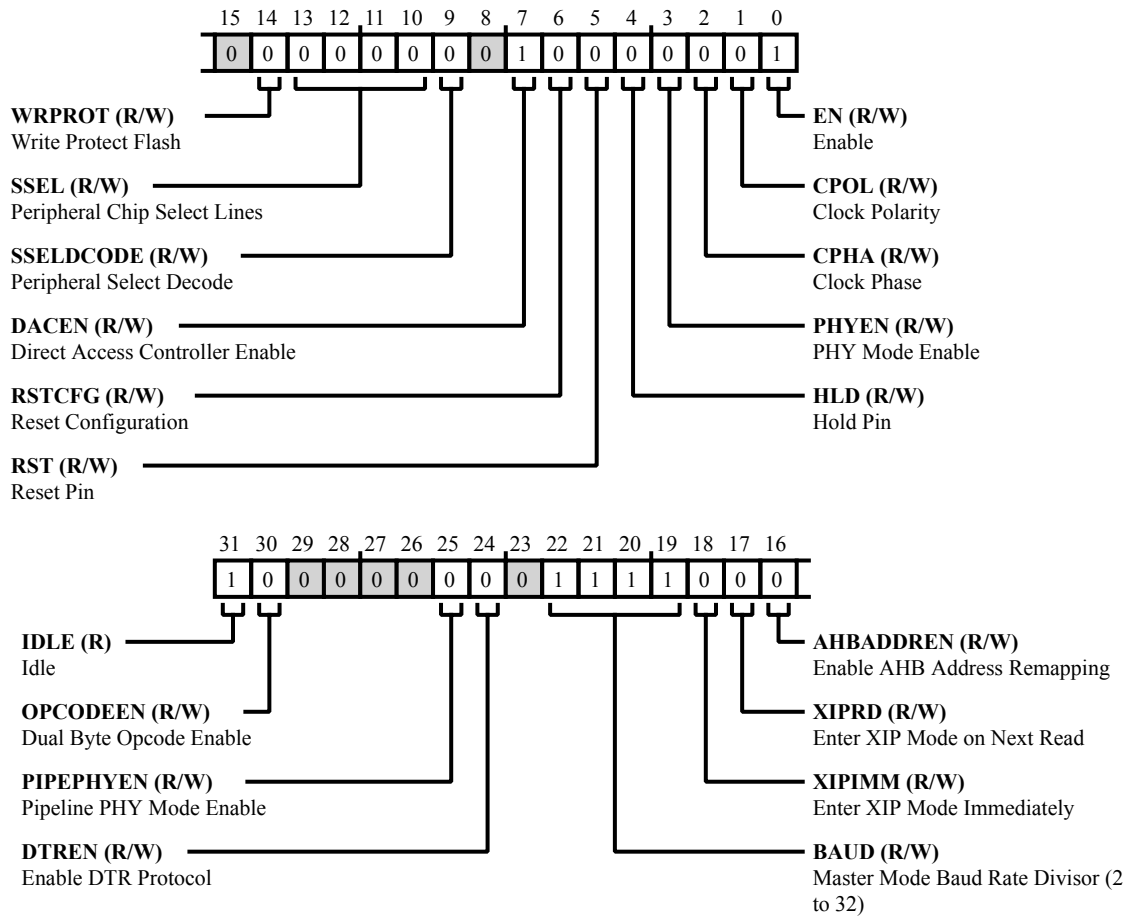


Figure 17-3: OSPI_CTL Register Diagram

Table 17-6: OSPI_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	IDLE	Idle. Serial interface and low level SPI pipeline is idle. This is a status read-only bit. Note: This is a re-timed signal. There is an inherent delay in generating this signal.
30 (R/W)	OPCODEEN	Dual Byte Opcode Enable. The <code>OSPI_CTL.OPCODEEN</code> bit is set if the target Flash device supports dual byte opcode (Macronix MX25). It is applicable for Octal I/O mode or protocol only. It should be set back to low if the device is configured to work in another SPI Mode. If enabled, the supplementing bytes are taken from the <code>OSPI_OE_LWR</code> and <code>OSPI_OE_UP</code> registers.

Table 17-6: OSPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	PIPEPHYEN	Pipeline PHY Mode Enable. The <code>OSPI_CTL.PIPEPHYEN</code> bit is relevant only for configuration with PHY module. It is asserted to 1 between consecutive PHY pipeline reads transfers and deasserted to 0 otherwise.
24 (R/W)	DTREN	Enable DTR Protocol. The <code>OSPI_CTL.DTREN</code> bit enables the DTR protocol. It must be set if the device is configured to work in DTR protocol.
22:19 (R/W)	BAUD	Master Mode Baud Rate Divisor (2 to 32). The <code>OSPI_CTL.BAUD</code> bit field selects the baud rate divisor. SPI Baud Rate = (Master Reference Clock)/ Baud Rate Divisor where, Baud Rate Divisor is: 4'b0000 = /2 4'b0001 = /4 4'b0010 = /6 4'b0011 = /8 4'b0100 = /10 ... 4'b0100 = /32
18 (R/W)	XIPIMM	Enter XIP Mode Immediately. If XIP is enabled, setting to 0 causes the controller to exit XIP mode on the next read instruction. If set to 1, operate the device in XIP mode immediately. Use this register when the external device wakes up in XIP mode (as per the contents of its non-volatile configuration register). The controller assumes the next read instruction is passed to the device as an XIP instruction, and therefore does not require the read opcode to be transferred. Note: To exit XIP mode, this bit should be set to 0. This takes effect in the attached device only after the next read instruction is executed. Software must ensure that at least one read instruction is requested after resetting this bit (to ensure that XIP mode is exited).
		0 If XIP is enabled, it causes the controller to exit XIP mode on the next read instruction
		1 Operate the device in XIP mode immediately

Table 17-6: OSPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	XIPRD	Enter XIP Mode on Next Read. If XIP is enabled, setting to 0 causes the controller to exit XIP mode on the next read instruction. If XIP is disabled, setting to 1 informs the controller that the device is ready to enter XIP on the next read instruction. The controller sends the appropriate command sequence, including mode bits to cause the device to enter XIP mode. Use this register after the controller ensures that the Flash device is configured to enter XIP mode. Note: To exit XIP mode, this bit should be set to 0. This takes effect in the attached device only after the next read instruction is executed. Software must ensure that at least one read instruction is requested after resetting this bit (to ensure that XIP mode is exited).
		0 If XIP is enabled, it cause the controller to exit XIP mode on the next read instruction
		1 If XIP is disabled, this setting informs the controller that the device is ready to enter XIP on the next read instruction
16 (R/W)	AHBADDREN	Enable AHB Address Remapping. The <code>OSPI_CTL.AHBADDREN</code> bit enables AHB address remapping (Direct Access Mode only). When set to 1, the incoming AHB address will be adapted and sent to the Flash device as (address + N), where N is the value stored in the remap address register.
14 (R/W)	WRPROT	Write Protect Flash. The <code>OSPI_CTL.WRPROT</code> bit, when set (=1), drives the Write Protect pin of the Flash. This is resynchronized to the generated memory clock, as necessary.
13:10 (R/W)	SSEL	Peripheral Chip Select Lines. The <code>OSPI_CTL.SSEL</code> bit field indicates the peripheral chip select line. If <code>OSPI_CTL.SSELDPCODE = 0</code> , <code>ss[3:0]</code> is output: <code>ss[3:0] SSEL[3:0]</code> <code>4'bxxx0.....1110.....SSEL0 selected</code> <code>4'bxx01.....1101.....SSEL1 selected</code> <code>4'bx011.....1011.....SSEL2 selected</code> <code>4'b0111.....0111.....SSEL3 selected</code> <code>4'b1111.....1111.....No peripheral selected</code> If <code>OSPI_CTL.SSELDPCODE = 1</code> , <code>ss[3:0]</code> directly drives <code>SSEL[3:0]</code> .

Table 17-6: OSPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	SSELDPCODE	Peripheral Select Decode. The OSPI_CTL.SSELDPCODE bit indicates the peripheral select decode.
		0 Only one of the 4 selects n_ss_out [3:0] is active
		1 Allows external 4:16 decode (n_ss_out = ss)
7 (R/W)	DACEN	Direct Access Controller Enable. The OSPI_CTL.DACEN bit enables the Direct Access Controller. When the Direct Access Controller and Indirect Access Controller are disabled, all AHB requested are completed with an error response.
		0 Disable Direct Access Controller once current transfer of the data word is complete
		1 Enable Direct Access Controller
6 (R/W)	RSTCFG	Reset Configuration. The OSPI_CTL.RSTCFG bit indicates the reset pin configuration.
		0 Reset Feature on DQ3 Pin of the Device
		1 Reset Feature on Dedicated Pin of the Device (Controlling the 5th bit Influences the Output)
5 (R/W)	RST	Reset Pin. The OSPI_CTL.RST bit, when set (=1), drives the Reset pin of the Flash and is reset to deactivate the Reset pin.
4 (R/W)	HLD	Hold Pin. The OSPI_CTL.HLD bit, when set (=1), drives the Hold pin of the Flash and reset to deactivate the Hold pin.
3 (R/W)	PHYEN	PHY Mode Enable. When the OSPI_CTL.PHYEN bit is enabled, the controller is informed that PHY module is to be used for handling SPI transfers. This bit is relevant only for configuration with PHY module.
2 (R/W)	CPHA	Clock Phase. The OSPI_CTL.CPHA bit selects the clock phase.
		0 SPI clock is active outside the word
		1 SPI clock is inactive outside the word
1 (R/W)	CPOL	Clock Polarity. The OSPI_CTL.CPOL bit selects clock polarity outside SPI word.
		0 OSPI clock is quiescent low
		1 OSPI clock is quiescent high

Table 17-6: OSPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Enable. The OSPI_CTL.EN bit enables OSPI operation.
		0 Disable OSPI Module
		1 Enable OSPI Module

Device Delay Register

This register introduces relative delays in the generation of the master output signals. All timings are defined in cycles of the SPI REFERENCE CLOCK/ext_clk.

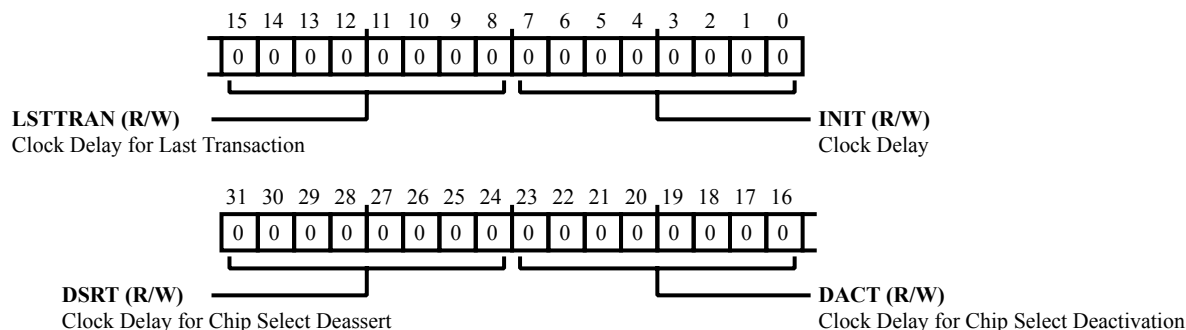


Figure 17-4: OSPI_DLY Register Diagram

Table 17-7: OSPI_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	DSRT	Clock Delay for Chip Select Deassert. Delay in master reference clocks for the length that the master mode chip select outputs are de-asserted between transactions. The minimum delay is always SCLK period to ensure the chip select is not asserted again within an SCLK period.
23:16 (R/W)	DACT	Clock Delay for Chip Select Deactivation. Delay in master reference clocks between one chip select being de-activated and the activation of another. This is used to ensure a quiet period between the selection of two different slaves and requires the transmit FIFO to be empty.
15:8 (R/W)	LSTTRAN	Clock Delay for Last Transaction. Delay in master reference clocks between last bit of current transaction and de-asserting the device chip select (n_ss_out). By default, the chip select is de-asserted on the cycle following the completion of the current transaction.
7:0 (R/W)	INIT	Clock Delay. Clock delay with n_ss_out. Delay in master reference clocks between setting n_ss_out low and first bit transfer.

Device Read Instruction Control Register

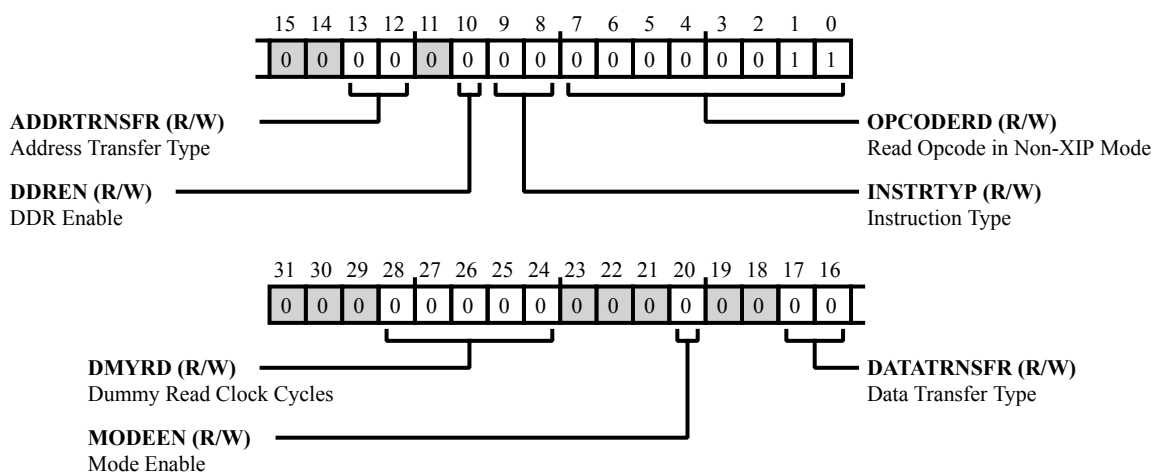


Figure 17-5: OSPI_DRICTL Register Diagram

Table 17-8: OSPI_DRICTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:24 (R/W)	DMYRD	Dummy Read Clock Cycles. Number of dummy clock cycles required by the device for a read instruction.
20 (R/W)	MODEEN	Mode Enable. This field is set to 1 to ensure that the <code>OSPI_MBCTL.MODE</code> bits are sent following the address bytes.
17:16 (R/W)	DATATRNSFR	Data Transfer Type. Data transfer type for standard SPI modes.
		0 SIO Mode. Data is shifted to the device on DQ0 only and from the device on DQ1 only.
		1 Used for Dual Input/Output instructions. For data transfers, DQ0 and DQ1 are used as both inputs and outputs.
		2 Used for Quad Input/Output instructions. For data transfers, DQ0, DQ1, DQ2, and DQ3 are used as both inputs and outputs.
		3 Used for Quad Input/Output instructions. For data transfers, DQ[7:0] are used as both inputs and outputs.

Table 17-8: OSPI_DRICTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13:12 (R/W)	ADDRTRNSFR	Address Transfer Type. Address transfer type for standard SPI modes.
		0 Addresses can be shifted to the device on DQ0 only.
		1 Addresses can be shifted to the device on DQ0 and DQ1 only.
		2 Addresses can be shifted to the device on DQ0, DQ1, DQ2, and DQ3.
		3 Addresses can be shifted to the device on DQ [7:0]
10 (R/W)	DDREN	DDR Enable. It checks if the opcode from the OSPI_DRICTL.OPCODERD field is compliant with one of the DDR read commands. Set this field to 1 when opcode from bits 7 to 0 is compliant with DDR command. Master output data is issued by the controller in DDR fashion starting of the address SPI transfer phase. It is not applicable for STIG Mode.
9:8 (R/W)	INSTRTYP	Instruction Type. Instruction Type.
		0 Use standard SPI mode. Data is shifted to the device on DQ0 only and from the device on DQ1 only.
		1 Use DIO-SPI mode. Instruction sent on DQ0 and DQ1.
		2 Use QIO-SPI mode. Instruction sent on DQ0, DQ1, DQ2, and DQ3.
		3 Use OIO-SPI mode. Instruction sent on DQ[7:0].
7:0 (R/W)	OPCODERD	Read Opcode in Non-XIP Mode. Read Opcode to use when not in XIP mode.

Device Write Instruction Control Register

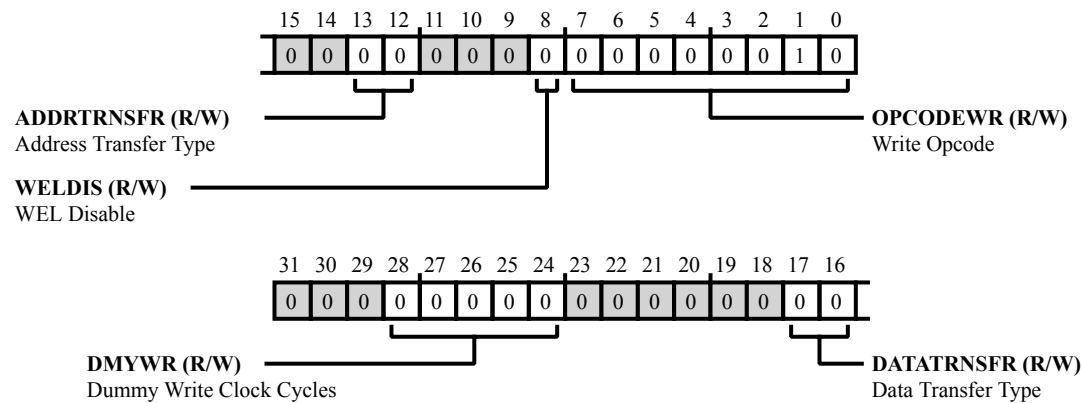


Figure 17-6: OSPI_DWICRTL Register Diagram

Table 17-9: OSPI_DWICRTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:24 (R/W)	DMYWR	Dummy Write Clock Cycles. Number of dummy clock cycles required by the device for write instruction.
17:16 (R/W)	DATATRNSFR	Data Transfer Type. Data transfer type for standard SPI modes.
		0 SIO Mode. Data is shifted to the device on DQ0 only and from the device on DQ1 only.
		1 Used for Dual Input/Output instructions. For data transfers, DQ0 and DQ1 are used as both inputs and outputs.
		2 Used for Octal Input/Output Instructions. For data transfers, DQ0, DQ1, DQ2, and DQ3 are used as both inputs and outputs.
		3 Used for Octal Input/Output instructions. For data transfers, DQ[7:0] are used as both inputs and outputs.

Table 17-9: OSPI_DWICTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13:12 (R/W)	ADDRTRNSFR	Address Transfer Type. Address transfer type for standard SPI modes.
		0 Addresses can be shifted to the device on DQ0 only.
		1 Addresses can be shifted to the device on DQ0 and DQ1.
		2 Addresses can be shifted to the device on DQ0, DQ1, DQ2, and DQ3.
		3 Addresses can be shifted to the device on DQ [7:0].
8 (R/W)	WELDIS	WEL Disable. This is to turn off automatic issuing of WEL command before write operation for DAC or INDAC.
7:0 (R/W)	OPCODEWR	Write Opcode.

Device Size Control Register

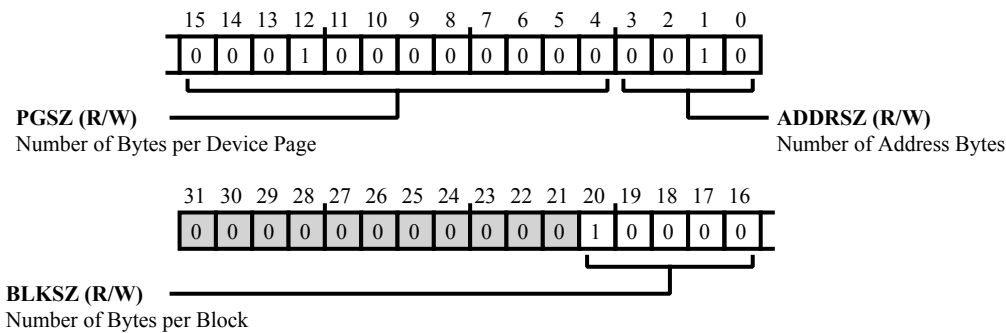


Figure 17-7: OSPI_DSCTL Register Diagram

Table 17-10: OSPI_DSCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20:16 (R/W)	BLKSZ	Number of Bytes per Block. The controller uses this field to perform write protection logic. The number of bytes per block must be a power of 2. 0: 1 byte 1: 2 bytes 3: 8 bytes ... 16: 65535 bytes and so on
15:4 (R/W)	PGSZ	Number of Bytes per Device Page. The controller uses this field to perform flash writes up to and across page boundaries.
3:0 (R/W)	ADDRSZ	Number of Address Bytes. Defines the address size for DAC mode. A value of 0 indicates 1 byte.

DLL Observable Register (Lower)

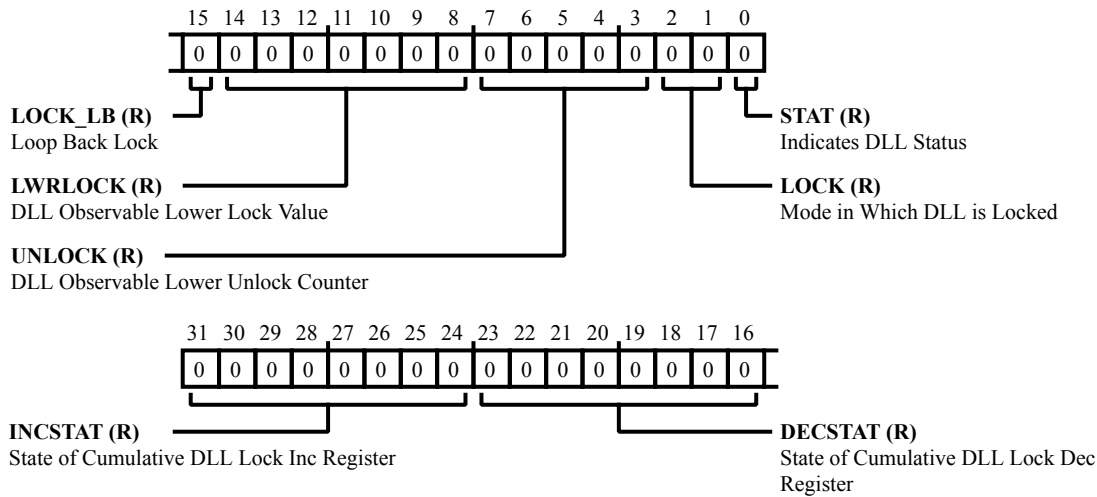


Figure 17-8: OSPI_DLLOB_LWR Register Diagram

Table 17-11: OSPI_DLLOB_LWR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	INCSTAT	State of Cumulative DLL Lock Inc Register. Holds the state of the cumulative dll_lock_inc register.
23:16 (R/NW)	DECSTAT	State of Cumulative DLL Lock Dec Register. Holds the state of the cumulative dll_lock_dec register.
15 (R/NW)	LOCK_LB	Loop Back Lock. This bit indicates that lock of loop back is done.
14:8 (R/NW)	LWRLOCK	DLL Observable Lower Lock Value. Reports the DLL encoder value from the master DLL to slave DLLs.
7:3 (R/NW)	UNLOCK	DLL Observable Lower Unlock Counter. Reports the number of increments or decrements required for the master DLL to complete the locking process.
2:1 (R/NW)	LOCK	Mode in Which DLL is Locked. Defines the mode in which the DLL has achieved the lock.
0 (R/NW)	STAT	Indicates DLL Status.

DLL Observable Register (Upper)

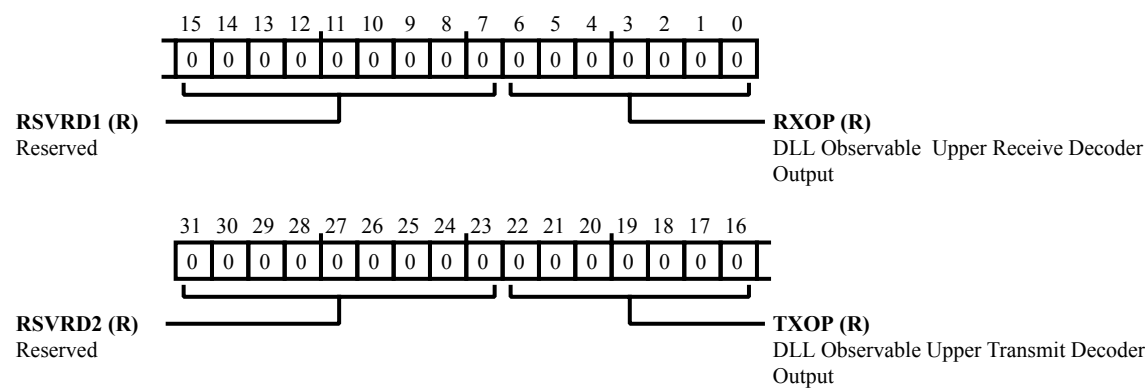


Figure 17-9: OSPI_DLLOB_UP Register Diagram

Table 17-12: OSPI_DLLOB_UP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:23 (R/NW)	RSVRD2	Reserved. Unused.
22:16 (R/NW)	TXOP	DLL Observable Upper Transmit Decoder Output. Holds the encoded value for the TX delay line for this slice.
15:7 (R/NW)	RSVRD1	Reserved. Unused.
6:0 (R/NW)	RXOP	DLL Observable Upper Receive Decoder Output. Holds the encoded value for the RX delay line for this slice.

Flash Command Address Register

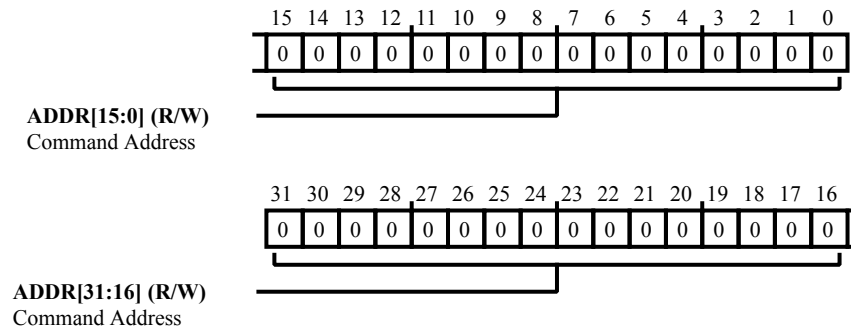


Figure 17-10: OSPI_FCA Register Diagram

Table 17-13: OSPI_FCA Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ADDR	Command Address. It is the address used by the command specified in the <code>OSPI_FCCTL.OPCODE</code> field. This must be setup before triggering the STIG command using the <code>OSPI_FCCTL.EXE</code> field.

Flash Command Control Register

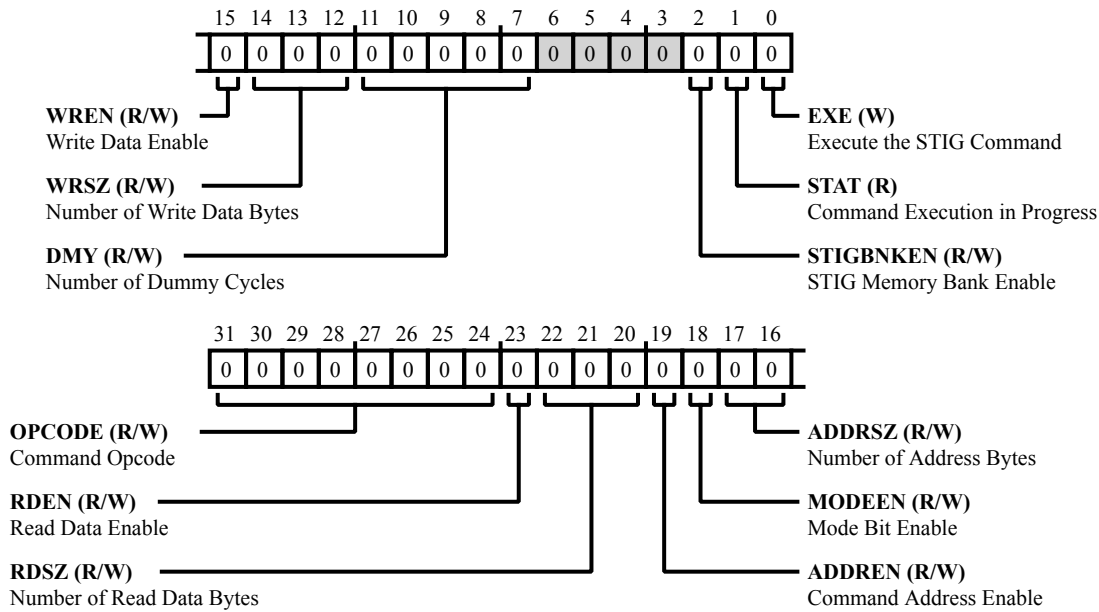


Figure 17-11: OSPI_FCCTL Register Diagram

Table 17-14: OSPI_FCCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	OPCODE	Command Opcode. This field must be set before triggering the STIG command. For example, 0x20 maps to sub sector erase. Writing to the OSPI_FCCTL.EXE launches the command. Note: Using this approach, to issue commands to the device will make use of the instruction type of the OSPI_DRICTL and OSPI_DWICTL registers.
		0 Command opcode, command address, command dummy bytes, and command data are serially transferred.
		1 Command opcode, command address, command dummy bytes, and command data are transferred in parallel using DQ0 and DQ1 pins.
		2 Command opcode, command address, command dummy bytes, and command data are transferred in parallel using DQ0, DQ1, and DQ2 pins.
		3 Reserved
23 (R/W)	RDEN	Read Data Enable. Set to 1 if the command specified in the OSPI_FCCTL.OPCODE requires read data bytes to be received from the device.

Table 17-14: OSPI_FCCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22:20 (R/W)	RDSZ	Number of Read Data Bytes. Up to 8 data bytes can be read using this command. Set this field to 0 for 1 byte and 7 for 8 bytes.
19 (R/W)	ADDREN	Command Address Enable. Set to 1 if the command specified in the <code>OSPI_FCCTL.OPCODE</code> field requires an address. This must be set before triggering the command via writing a 1 to the execute field.
18 (R/W)	MODEEN	Mode Bit Enable. Set this field to 1 to ensure the <code>OSPI_MBCTL.MODE</code> field is sent following the address bytes.
17:16 (R/W)	ADDRSZ	Number of Address Bytes. Set to the number of address bytes required (the address itself is programmed in the <code>OSPI_FCA</code> register. This must be setup before triggering the command via bit 0 of this register.
		0 1 address byte
		1 2 address bytes
		2 3 address bytes
		3 4 address bytes
15 (R/W)	WREN	Write Data Enable. Set to 1 if the command specified in the <code>OSPI_FCCTL.OPCODE</code> field requires write data bytes to be sent to the device.
14:12 (R/W)	WRSZ	Number of Write Data Bytes. Up to 8 data bytes may be written using this command. Set to 0 for 1 byte, 7 for 8 bytes.
11:7 (R/W)	DMY	Number of Dummy Cycles. Set to the number of dummy cycles required. This should be setup before triggering the command via the <code>OSPI_FCCTL.EXE</code> field.
2 (R/W)	STIGBNKEN	STIG Memory Bank Enable. This bit must be asserted high before triggering the flash memory bank transfer. It must be de-asserted before triggering any other operation. This is set before triggering the command via <code>OSPI_FCCTL.EXE</code> field.
1 (R/NW)	STAT	Command Execution in Progress.
0 (RX/W)	EXE	Execute the STIG Command.

Flash Command Control Memory Register

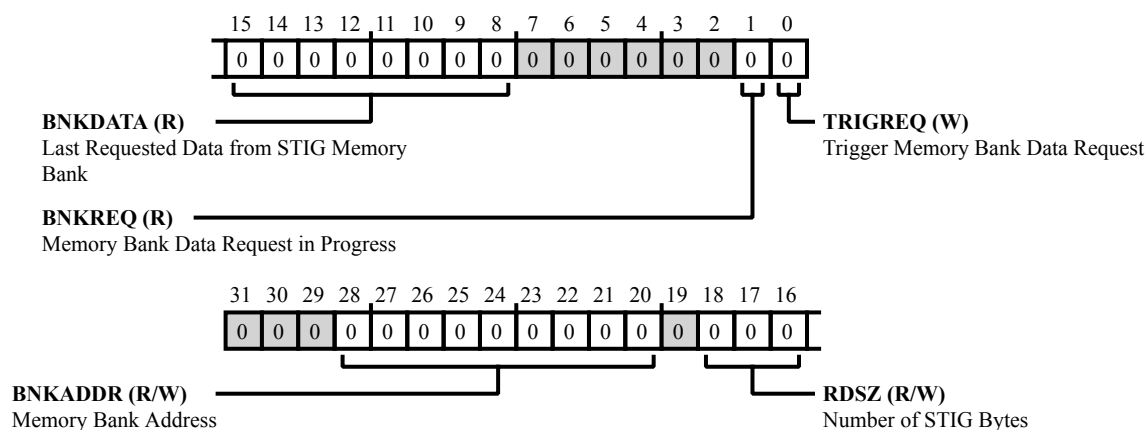


Figure 17-12: OSPI_FCMCTL Register Diagram

Table 17-15: OSPI_FCMCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:20 (R/W)	BNKADDR	Memory Bank Address. The address of the memory bank from which data is read.
18:16 (R/W)	RDSZ	Number of STIG Bytes. It defines the number of read bytes for the extended STIG.
15:8 (R/NW)	BNKDATA	Last Requested Data from STIG Memory Bank.
1 (R/NW)	BNKREQ	Memory Bank Data Request in Progress.
0 (RX/W)	TRIGREQ	Trigger Memory Bank Data Request.

Flash Command Read Data Register (Lower)

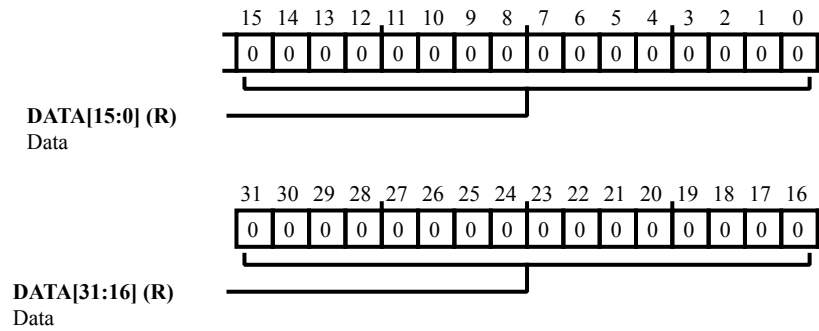


Figure 17-13: OSPI_FCRD_LWR Register Diagram

Table 17-16: OSPI_FCRD_LWR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	DATA	Data. This is the data that is returned by the flash device for any status or configuration read operation carried out by triggering the STIG event in the OSPI_FCCTL register. The register is valid when the <code>OSPI_FCCTL.STAT</code> bit is low.

Flash Command Read Data Register (Upper)

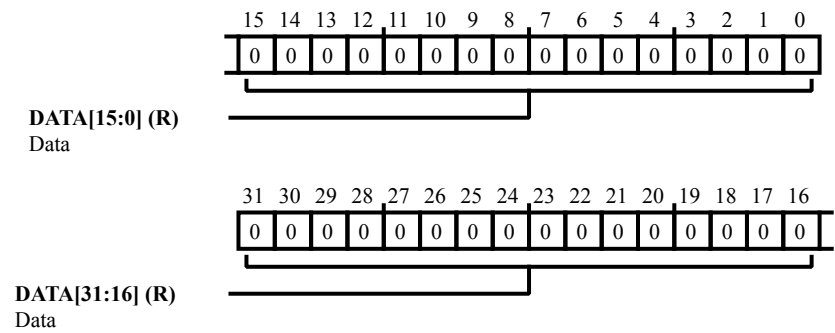


Figure 17-14: OSPI_FCRD_UP Register Diagram

Table 17-17: OSPI_FCRD_UP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	DATA	Data. This is the data that is returned by the flash device for any status or configuration read operation carried out by triggering the STIG event in the OSPI_FCCTL register. The register is valid when the <code>OSPI_FCCTL.STAT</code> bit is low

Flash Command Write Data Register (Lower)

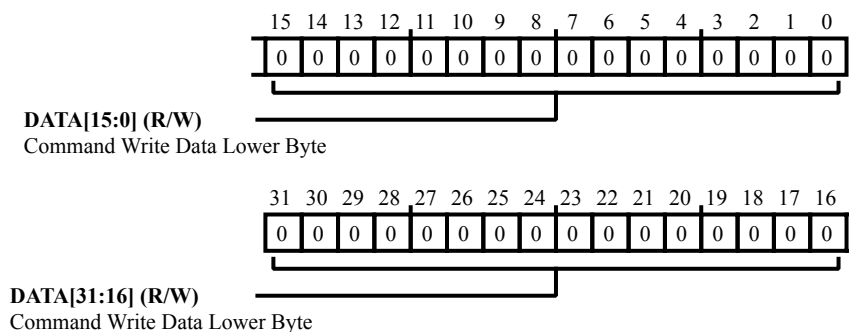


Figure 17-15: OSPI_FCWD_LWR Register Diagram

Table 17-18: OSPI_FCWD_LWR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	Command Write Data Lower Byte. It is the data written to the flash for any status or configuration write operation carried out by triggering the STIG event in the OSPI_FCCTL register. This must be setup before triggering the command using the OSPI_FCCTL.EXE field.

Flash Command Write Data Register (Upper)

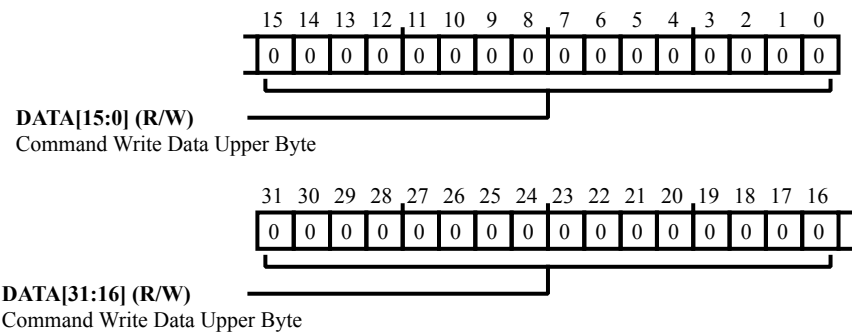


Figure 17-16: OSPI_FCWD_UP Register Diagram

Table 17-19: OSPI_FCWD_UP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	Command Write Data Upper Byte. It is the data written to the flash for any status or configuration write operation carried out by triggering the event in the OSPI_FCCTL register. This must be setup before triggering the command using the <code>OSPI_FCCTL.EXE</code> field.

Interrupt Mask Register

This register allows the user to mask/unmask particular interrupt sources. This register must be setup when the controller is idle.

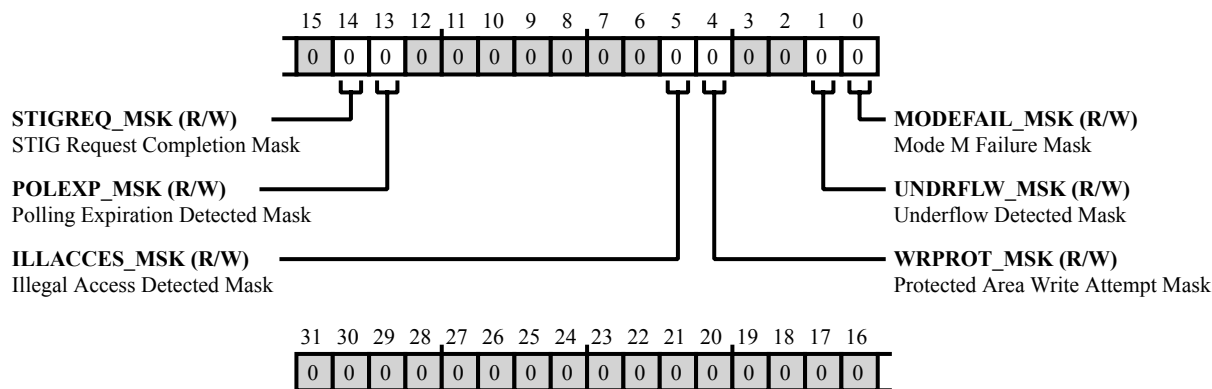


Figure 17-17: OSPI_IMSK Register Diagram

Table 17-20: OSPI_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	STIGREQ_MSK	STIG Request Completion Mask.
13 (R/W)	POLEXP_MSK	Polling Expiration Detected Mask.
5 (R/W)	ILLACCES_MSK	Illegal Access Detected Mask.
4 (R/W)	WRPROT_MSK	Protected Area Write Attempt Mask.
1 (R/W)	UNDRFLW_MSK	Underflow Detected Mask.
0 (R/W)	MODEFAIL_MSK	Mode M Failure Mask.

Interrupt Status Register

The status fields in this register are set when the described event occurs, and the interrupt is enabled in the `OSPI_IMSK` register. If any of these bit fields are set, the interrupt output is asserted high. The fields are each cleared by writing a 1 to the field.

Note: Bit fields [10:6] are valid only when legacy SPI mode is active.

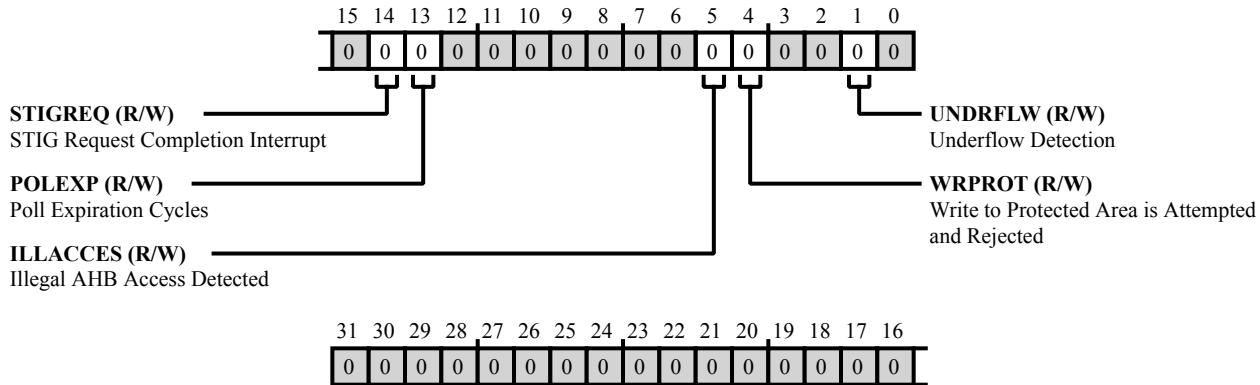


Figure 17-18: OSPI_ISTAT Register Diagram

Table 17-21: OSPI_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	STIGREQ	STIG Request Completion Interrupt. The controller is ready to get another STIG request.
13 (R/W)	POLEXP	Poll Expiration Cycles. The maximum number of programmed polls cycles is expired.
5 (R/W)	ILLACCES	Illegal AHB Access Detected. AHB wrapping bursts and the use of SPLIT/RETRY accesses cause this error interrupt to trigger.
4 (R/W)	WRPROT	Write to Protected Area is Attempted and Rejected.
1 (R/W)	UNDRFLW	Underflow Detection. Underflow is detected.
		0 No underflow is detected.
		1 underflow is detected and an attempt to transfer data is made when the small TX FIFO is empty. This may occur when AHB write data is slowly provided to keep up with the requested write operation.

Lower Write Protection Register

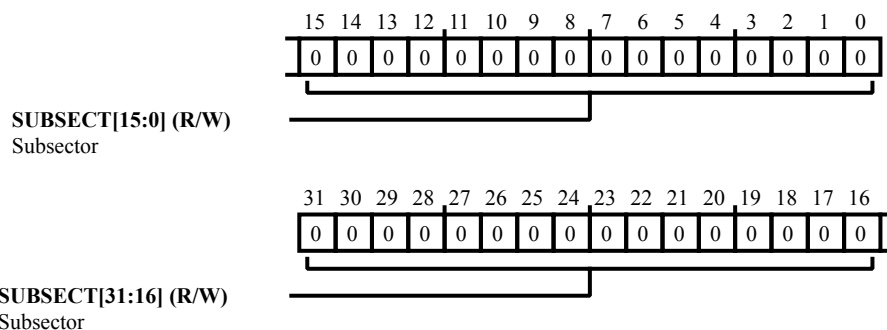


Figure 17-19: OSPI_WRPROT_LWR Register Diagram

Table 17-22: OSPI_WRPROT_LWR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	SUBSECT	Subsector. It defines the lower end of the address range to be protected. The address to be programmed must be OSPI memory map address (starting from 0x60000000) shifted by the value in the OSPI_DSCTL.BLKSZ. For example, if lower address of flash to be protected is 0x100 and the OSPI_DSCTL.BLKSZ= 4, the value to be programmed in this register must be 0x60000100 >> 4, which is 0x06000010.

Mode Bit Control Register

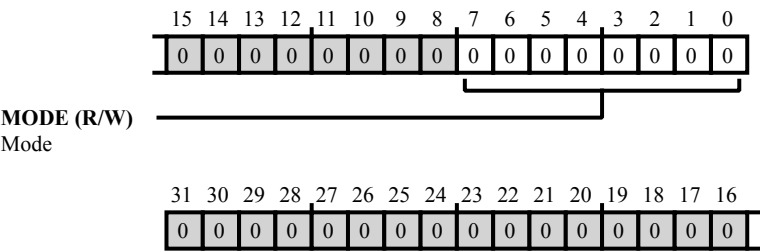


Figure 17-20: OSPI_MBCTL Register Diagram

Table 17-23: OSPI_MBCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	MODE	Mode. These bits are sent to the device following the address bytes if mode bit transmission is enabled.

Module ID Register

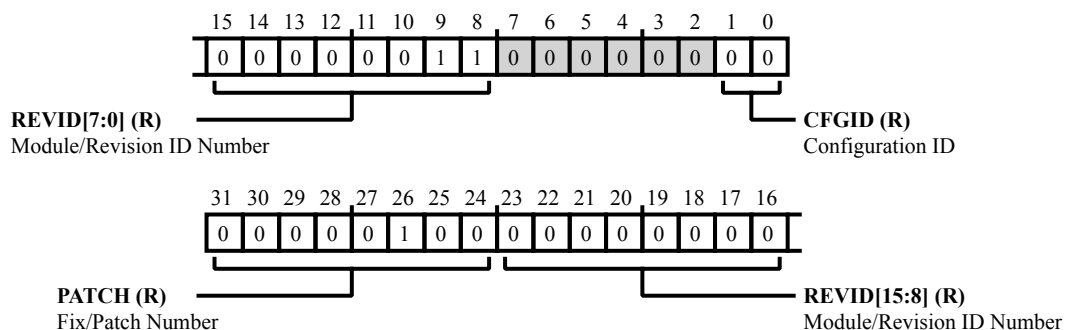


Figure 17-21: OSPI_MODID Register Diagram

Table 17-24: OSPI_MODID Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	PATCH	Fix/Patch Number. Fix/patch number related to the OSPI_MODID.REVID field.
23:8 (R/NW)	REVID	Module/Revision ID Number.
1:0 (R/NW)	CFGID	Configuration ID. Configuration ID number.
		0 Reserved
		1 Reserved
		2 Reserved
		3 Reserved

Polling Expiration Register

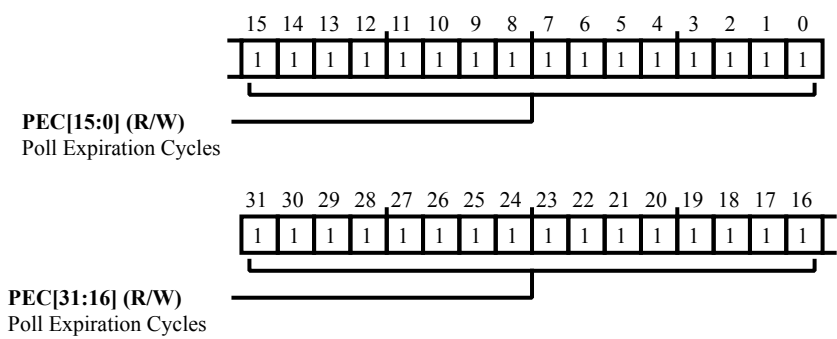


Figure 17-22: OSPI_POLLEXP Register Diagram

Table 17-25: OSPI_POLLEXP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PEC	Poll Expiration Cycles. It defines the number of polls cycles before expiration.

Opcode Extension Register (Lower)

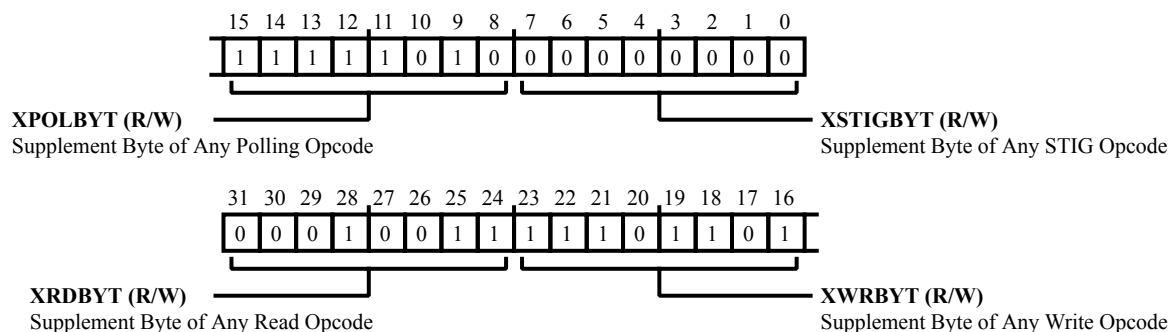


Figure 17-23: OSPI_OE_LWR Register Diagram

Table 17-26: OSPI_OE_LWR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	XRDBYT	Supplement Byte of Any Read Opcode.
23:16 (R/W)	XWRBYT	Supplement Byte of Any Write Opcode.
15:8 (R/W)	XPOLBYT	Supplement Byte of Any Polling Opcode.
7:0 (R/W)	XSTIGBYT	Supplement Byte of Any STIG Opcode.

Opcode Extension Register (Upper)

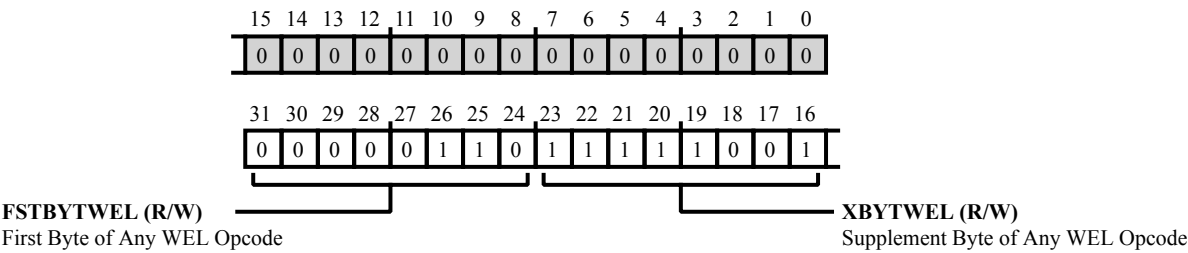


Figure 17-24: OSPI_OE_UP Register Diagram

Table 17-27: OSPI_OE_UP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	FSTBYTWEL	First Byte of Any WEL Opcode.
23:16 (R/W)	XBYTWEL	Supplement Byte of Any WEL Opcode.

PHY Control Register

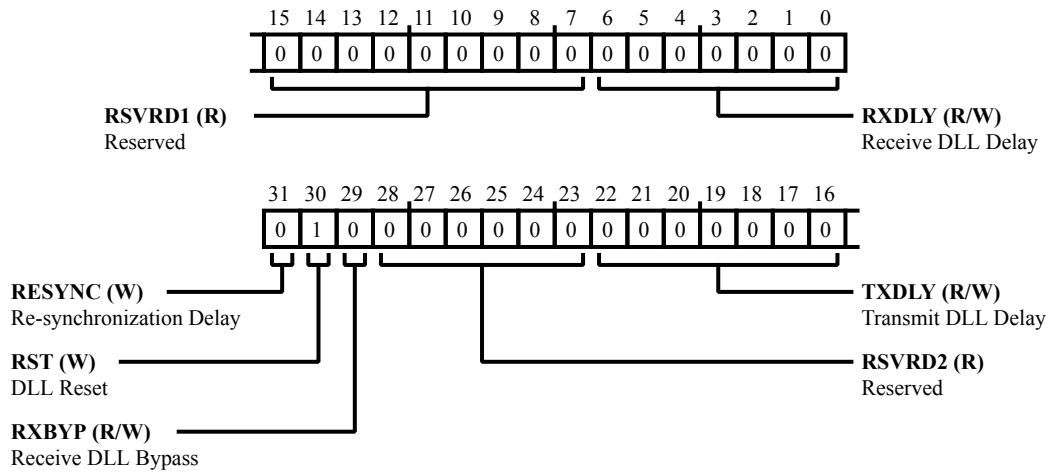


Figure 17-25: OSPI_PHYCTL Register Diagram

Table 17-28: OSPI_PHYCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (RX/W)	RESYNC	Re-synchronization Delay. This bit is used for re-synchronization delay lines to update them with values from the OSPI_PHYCTL.RXDLY and OSPI_PHYCTL.TXDLY fields.
30 (RX/W)	RST	DLL Reset. This bit is used for reset of Delay Lines by software.
29 (R/W)	RXBYP	Receive DLL Bypass. This field determines if RX DLL is bypassed.
28:23 (R/NW)	RSVRD2	Reserved. Unused.
22:16 (R/W)	TXDLY	Transmit DLL Delay. This field determines the number of delay elements to insert on data path between ref_clk and spi_clk.
15:7 (R/NW)	RSVRD1	Reserved. Unused.
6:0 (R/W)	RXDLY	Receive DLL Delay. This field determines the number of delay elements to insert on data path between ref_clk and rx_dll_clk.

PHY DLL Master Control Register

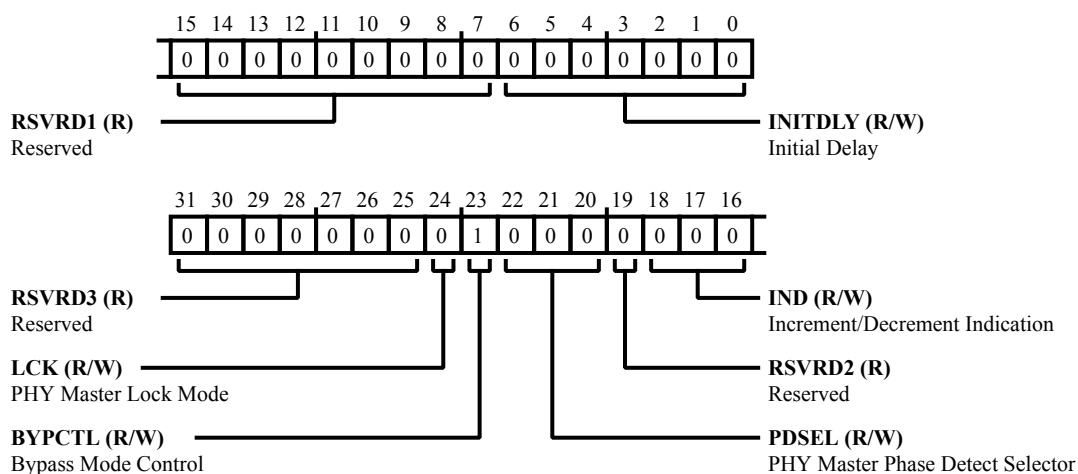


Figure 17-26: OSPI_PHYMCTL Register Diagram

Table 17-29: OSPI_PHYMCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:25 (R/NW)	RSVRD3	Reserved. Unused.
24 (R/W)	LCK	PHY Master Lock Mode. Determines if the master delay line locks on a full cycle or half cycle of delay.
23 (R/W)	BYPCTL	Bypass Mode Control. Controls the bypass mode of the master and slave DLLs
22:20 (R/W)	PDSEL	PHY Master Phase Detect Selector. Selects the number of delay elements to be inserted between the phase detect flip-flops.
19 (R/NW)	RSVRD2	Reserved. Unused.
18:16 (R/W)	IND	Increment/Decrement Indication. Indicates the number of consecutive increment or decrement indications.
15:7 (R/NW)	RSVRD1	Reserved. Unused.
6:0 (R/W)	INITDLY	Initial Delay. This value is the initial delay value for the DLL.

Polling Flash Status Register

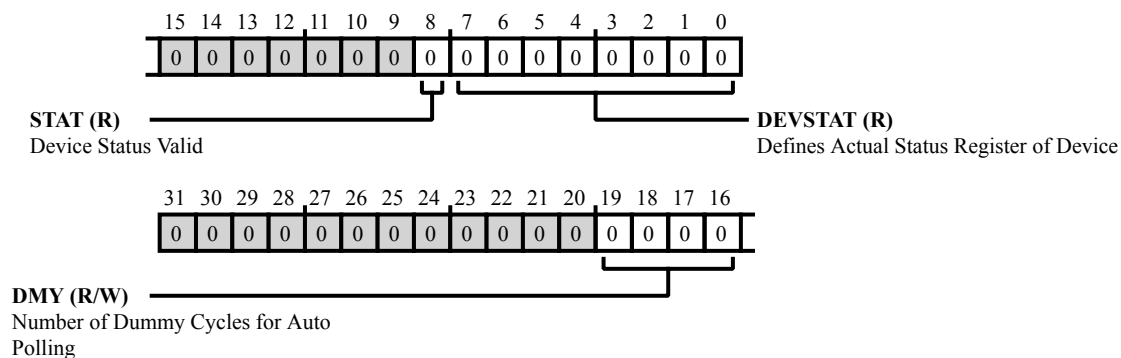


Figure 17-27: OSPI_POLSTAT Register Diagram

Table 17-30: OSPI_POLSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:16 (R/W)	DMY	Number of Dummy Cycles for Auto Polling. This field enables the user to define additional dummy cycles for read status during the auto-polling state. It may be necessary to add some cycles if the external delay of read data path shifts it outside the defined clock cycle.
8 (R/NW)	STAT	Device Status Valid. This bit is set when value in bits from 7 to 0 is valid.
7:0 (R/NW)	DEVSTAT	Defines Actual Status Register of Device.

Read Data Capture Register

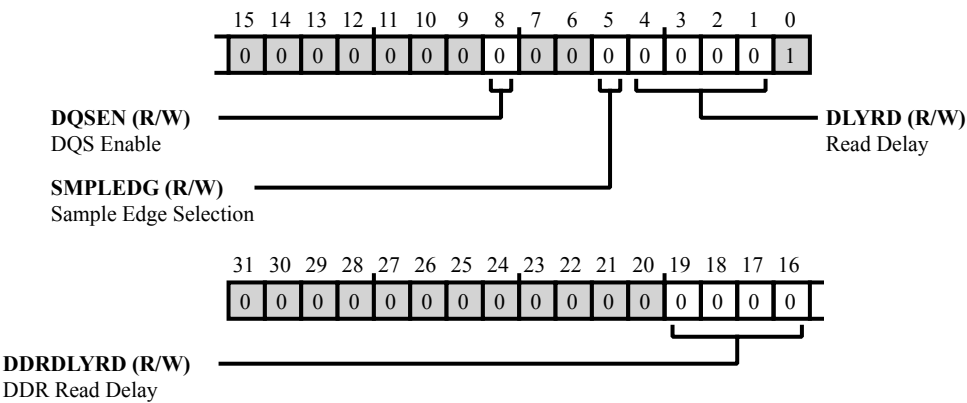


Figure 17-28: OSPI_RDC Register Diagram

Table 17-31: OSPI_RDC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
19:16 (R/W)	DDRDLYRD	DDR Read Delay. Delay the transmitted data by the number of ref_clk cycles.This field is relevant only when DDR read Command is executed. Else, it can be ignored.	
8 (R/W)	DQSEN	DQS Enable. If this bit is enabled, the signal from DQS input is driven into RX DLL and is used for data capturing in PHY Mode instead of internally generated gated ref_clk.	
5 (R/W)	SMPLEDG	Sample Edge Selection. Select the edge on which data outputs from flash memory has to be sampled.	
		0	Data outputs from Flash are sampled on falling edge of the ref_clk.
		1	Data outputs from Flash are sampled on rising edge of the ref_clk.
4:1 (R/W)	DLYRD	Read Delay. Delay the read data capturing logic by the programmed number of SYSCLK cycles.	

Remap Address Register

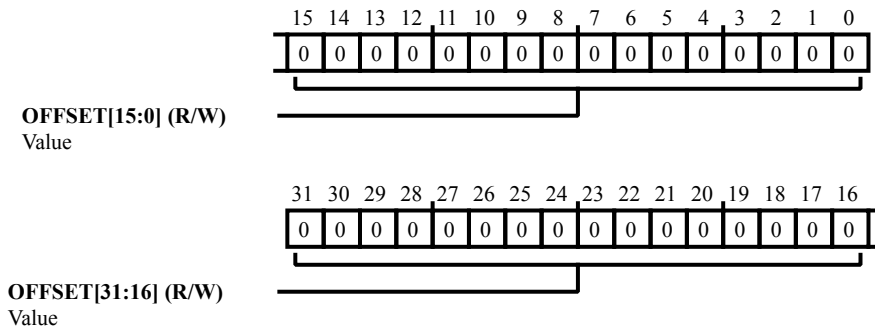


Figure 17-29: OSPI_REMAPADDR Register Diagram

Table 17-32: OSPI_REMAPADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	OFFSET	Value. Used to remap an incoming AHB address to a different address used by the Flash.

Upper Write Protection Register

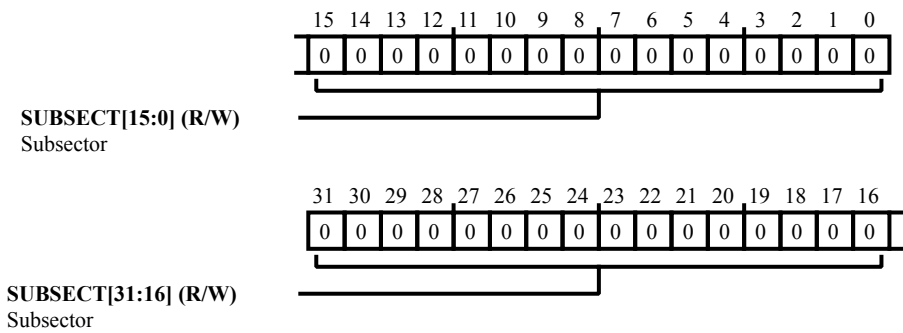


Figure 17-30: OSPI_WRPROT_UP Register Diagram

Table 17-33: OSPI_WRPROT_UP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	SUBSECT	Subsector. It defines the upper end of the address range to be protected. The address to be programmed must be OSPI memory map address (starting from 0x60000000) shifted by the value in the OSPI_DSCTL.BLKSZ. For example, if upper address of flash to be protected is 0x100 and the OSPI_DSCTL.BLKSZ= 4, the value to be programmed in this register must be 0x60000100 >> 4, which is 0x06000010.

Write Protection Control Register

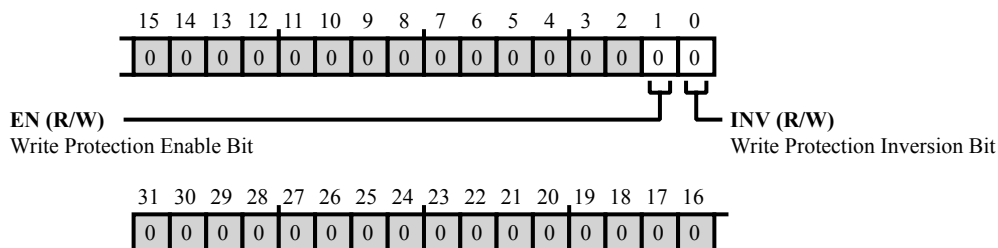


Figure 17-31: OSPI_WRPROT_CTL Register Diagram

Table 17-34: OSPI_WRPROT_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	EN	Write Protection Enable Bit. When set to 1, any AHB write access with an address within the protection region defined in the lower and upper write protection registers is rejected. An AHB error response is generated and an interrupt source triggered. When set to 0, the protection region is disabled.
0 (R/W)	INV	Write Protection Inversion Bit. When set to 1, the protection region defined in the lower and upper write protection registers is inverted meaning it is the region that the system is permitted to write to. When set to 0, the protection region defined in the lower and upper write protection registers is the region that the system is not permitted to write to.

Write Completion Control Register

This register defines how the controller polls the device following a write transfer.

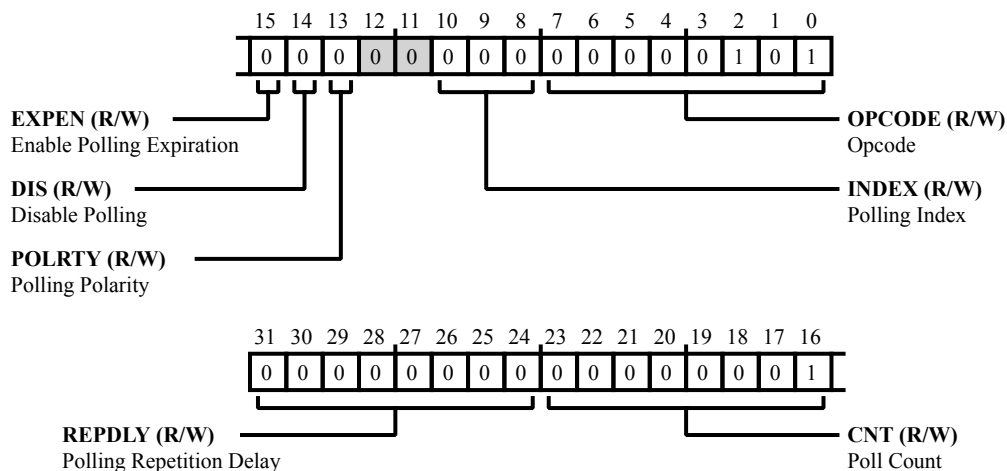


Figure 17-32: OSPI_WCCTL Register Diagram

Table 17-35: OSPI_WCCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	REPDLY	Polling Repetition Delay. Defines the additional delay required to ensure that the Chip Select is de-asserted during auto polling phase.
23:16 (R/W)	CNT	Poll Count. Defines the number of times the controller should expect to see a true result from polling in successive reads of the device register.
15 (R/W)	EXPEN	Enable Polling Expiration. It is set to 1 to enable auto polling expiration.
14 (R/W)	DIS	Disable Polling. It switches off automatic polling function.
13 (R/W)	POLRTY	Polling Polarity. Defines the polling polarity.
		0 Write transfer to the device will be complete if the polled bit is 0.
		1 Write transfer to the device will be complete if the polled bit is 1.

Table 17-35: OSPI_WCCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10:8 (R/W)	INDEX	Polling Index. Defines the bit index that must be polled. A value of 010 means that bit 2 of the returned data is polled for. A value of 111 means that bit 7 of the returned data is polled for.
7:0 (R/W)	OPCODE	Opcode. Defines the opcode that must be issued by the controller when it is automatically polling for device program completion. This command is issued followed all device write operations. By default, this polls the standard device status register using opcode 0x05.

18 Universal Asynchronous Receiver/Transmitter (UART)

The UART module is a full-duplex peripheral compatible with PC-style industry-standard UARTs. The UART converts data between serial and parallel formats. The serial communication follows an asynchronous protocol that supports various word lengths, stop bits, bit rates, and parity-generation options. Multiple events can generate interrupts.

The UART is logically compliant to EIA-232E, EIA-422, EIA-485 and LIN standards, but usually requires external transceiver devices to meet electrical requirements. In IrDA (Infrared Data Association) mode, the UART meets the half-duplex IrDA SIR (9.6/115.2 Kbps rate) protocol. In multi-drop bus mode, the UART meets the full-duplex MDB/ICP v2.0 protocol.

The UART module supports partial modem status and control functionality to allow for hardware flow control.

The UART is a DMA-capable peripheral with separate transmit and receive DMA master channels. The use of DMA requires minimal software intervention as the DMA engine moves the data. The UART can also use a programmed core mode of operation. The core mode requires software management of the data flow using either interrupts or polling.

The UART can use one of the peripheral timers for a hardware-assisted auto-baud detection mechanism. The timers are external to the UART.

NOTE: The UARTs (UART0, UART1 and UART2) operate in the SCLK0 domain.

UART Features

Each UART includes the following features.

- 5–8 data bits
- Programmable extra stop bit and programmable extra half-stop bit
- Even, odd, and sticky parity bit options
- Extra 8-stage receive FIFO with programmable threshold interrupt request
- Flexible transmit and receive interrupt request timing

- Three interrupt outputs for receive, transmit, and status
- Independent DMA operation for receive and transmit
- Programmable automatic request to send (RTS)/clear to send (CTS) hardware flow control
- False start bit detection
- SIR IrDA operation mode
- MDB/ICP v2.0 operation mode
- Internal loopback
- Improved bit rate granularity
- LIN break command/Inter-frame gap transmission support

Table 18-1: UART Specifications

Feature	Availability
<i>Protocol</i>	
Master-Capable	Yes
Slave-Capable	Yes
Transmission Simplex	Yes
Transmission Half-Duplex	Yes
Transmission Full-Duplex	Yes
<i>Access Type</i>	
Data Buffer	Yes
Core Data Access	Yes
DMA Data Access	Yes
DMA Channels	2 (per UART Port)
DMA Descriptor	Yes
Boot Capable	Yes (Slave Mode)
Local Memory	No
Clock Operation	SCLK0/16

UART Functional Description

The following sections provide details on the UARTs functionality.

ADSP-2159x UART Register List

The Universal Asynchronous Receiver/Transmitter module (UART) is a full-duplex peripheral compatible with PC-style industry-standard UARTs. The UART converts data between serial and parallel formats. The serial communication follows an asynchronous protocol that supports various word length, stop bit, parity, and interrupt generation options. A set of registers governs UART operations. For more information on UART functionality, see the UART register descriptions.

Table 18-2: ADSP-2159x UART Register List

Name	Description
UART_CLK	Clock Rate Register
UART_CTL	Control Register
UART_IMSK	Interrupt Mask Register
UART_IMSK_CLR	Interrupt Mask Clear Register
UART_IMSK_SET	Interrupt Mask Set Register
UART_RBR	Receive Buffer Register
UART_RSR	Receive Shift Register
UART_RXCNT	Receive Counter Register
UART_SCR	Scratch Register
UART_STAT	Status Register
UART_TAIP	Transmit Address/Insert Pulse Register
UART_THR	Transmit Hold Register
UART_TSR	Transmit Shift Register
UART_TXCNT	Transmit Counter Register

ADSP-2159x UART Interrupt List

Table 18-3: ADSP-2159x UART Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
138	UART0_TXDMA	UART0 Transmit DMA	Level	20
139	UART0_RXDMA	UART0 Receive DMA	Level	21
140	UART0_STAT	UART0 Status	Level	
141	UART1_TXDMA	UART1 Transmit DMA	Level	34
142	UART1_RXDMA	UART1 Receive DMA	Level	35
143	UART1_STAT	UART1 Status	Level	
144	UART2_TXDMA	UART2 Transmit DMA	Level	37

Table 18-3: ADSP-2159x UART Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
145	UART2_RXDMA	UART2 Receive DMA	Level	38
146	UART2_STAT	UART2 Status	Level	
147	UART3_TXDMA	UART3 Transmit DMA	Level	53
148	UART3_RXDMA	UART3 Receive DMA	Level	54
149	UART3_STAT	UART3 Status	Level	
269	UART0_TXDMA_ERR	UART0 Transmit DMA Error	Level	
270	UART0_RXDMA_ERR	UART0 Receive DMA Error	Level	
271	UART1_TXDMA_ERR	UART1 Transmit DMA Error	Level	
272	UART1_RXDMA_ERR	UART1 Receive DMA Error	Level	
273	UART2_TXDMA_ERR	UART2 Transmit DMA Error	Level	
274	UART2_RXDMA_ERR	UART2 Receive DMA Error	Level	
275	UART3_TXDMA_ERR	UART3 Transmit DMA Error	Level	
276	UART3_RXDMA_ERR	UART3 Receive DMA Error	Level	

ADSP-2159x UART Trigger List

Table 18-4: ADSP-2159x UART Trigger List Masters

Trigger ID	Name	Description	Sensitivity
160	UART0_TXDMA	UART0 Transmit DMA	Edge
161	UART0_RXDMA	UART0 Receive DMA	Edge
162	UART1_TXDMA	UART1 Transmit DMA	Edge
163	UART1_RXDMA	UART1 Receive DMA	Edge
164	UART2_TXDMA	UART2 Transmit DMA	Edge
165	UART2_RXDMA	UART2 Receive DMA	Edge
166	UART3_TXDMA	UART3 Transmit DMA	Edge
167	UART3_RXDMA	UART3 Receive DMA	Edge

Table 18-5: ADSP-2159x UART Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
169	UART0_TXDMA	UART0 Transmit DMA	Pulse
170	UART0_RXDMA	UART0 Receive DMA	Pulse

Table 18-5: ADSP-2159x UART Trigger List Slaves (Continued)

Trigger ID	Name	Description	Sensitivity
171	UART1_TXDMA	UART1 Transmit DMA	Pulse
172	UART1_RXDMA	UART1 Receive DMA	Pulse
173	UART2_TXDMA	UART2 Transmit DMA	Pulse
174	UART2_RXDMA	UART2 Receive DMA	Pulse
175	UART3_TXDMA	UART3 Transmit DMA	Pulse
176	UART3_RXDMA	UART3 Receive DMA	Pulse

ADSP-2159x UART DMA Channel List

Table 18-6: ADSP-2159x UART DMA Channel List

DMA ID	DMA Channel Name	Description
DMA20	UART0_TXDMA	UART0 Transmit DMA
DMA21	UART0_RXDMA	UART0 Receive DMA
DMA34	UART1_TXDMA	UART1 Transmit DMA
DMA35	UART1_RXDMA	UART1 Receive DMA
DMA37	UART2_TXDMA	UART2 Transmit DMA
DMA38	UART2_RXDMA	UART2 Receive DMA
DMA53	UART3_TXDMA	UART3 Transmit DMA
DMA54	UART3_RXDMA	UART3 Receive DMA

UART Block Diagram

The *UART Block Diagram* figure shows a simplified block diagram of one UART module and how it interconnects to the processor system.

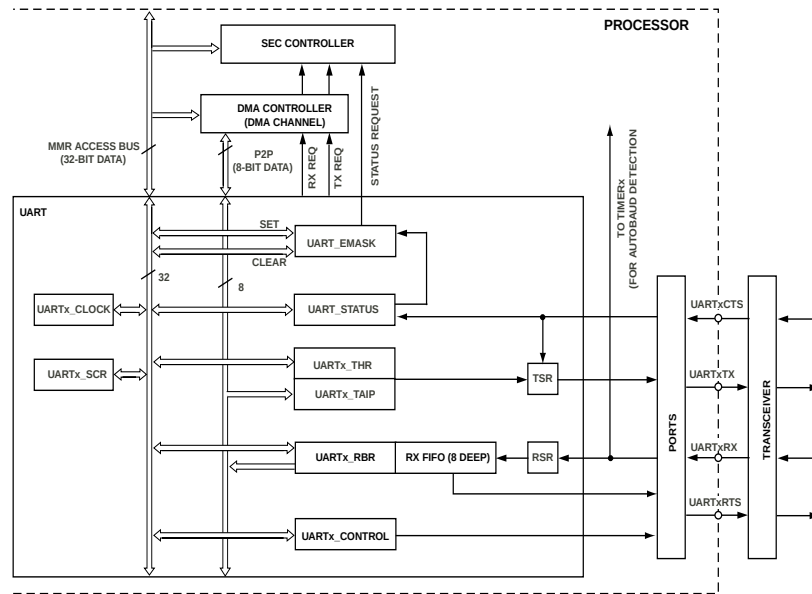


Figure 18-1: UART Block Diagram

UART Architectural Concepts

The following sections provide information about the UART architecture.

Internal Interface

The UART is a DMA-capable peripheral with support for separate transmit and receive DMA master channels. It operates in either DMA or programmed core modes. The core mode requires software management of the data flow using either interrupts or polling. The DMA method requires minimal software intervention, as the DMA engine itself moves the data. The `UART_RBR` and `UART_THR` registers also connect to one of the peripheral DMA buses.

All UART registers are 32 bits wide and the registers connect to the peripheral MMR bus. Not all MMRs can be used and unused bits are zero-filled. The UART has three interrupt outputs described as follows.

- The transmit and receive request outputs can function as DMA requests and connect to the DMA controller. Therefore, if the DMA is not enabled, the DMA controller simply forwards the request to the system event controller (SEC).
- The status interrupt output connects directly to the SEC. On many processors, the alternative capture input (`(TIMER_ACI[nn])`) of one of the GP timers also senses the `UART_RX` pin. When configured in capture mode, the processor can then use the GP timer to detect the bit rate of the received signal.

External Interface

Each UART features a `UART_RX` (receive) pin and a `UART_TX` (transmit) pin available through the general-purpose ports. These two pins usually connect to an external transceiver device that meets the electrical requirements of full-duplex or half-duplex standards. For example, EIA-232, EIA-422, 4-wire EIA-485 for full-duplex or 2-wire EIA-485, LIN for half-duplex. Additionally, the UART features a pair of clear-to-send, input pins (`UART_CTS`),

and request-to-send, output pins (`UART_RTS`) for hardware flow control. UART signals are multiplexed with other functions at the pin level.

Hardware Flow Control

To prevent the UART transmitter from sending data while the receiving counterpart is not ready, the UART features a `UART_RTS/UART_CTS` hardware flow control mechanism. The `UART_RTS` signal is an output that connects to the `UART_CTS` input of the communication partner. If data transfer is bidirectional, the figure shows the *UART Hardware Flow Control* handshake.

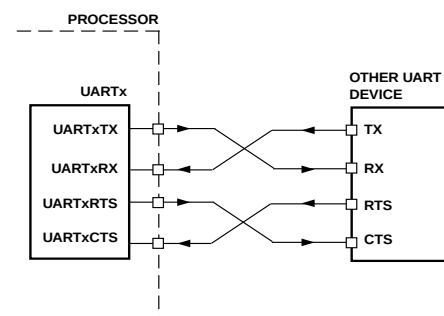


Figure 18-2: UART Hardware Flow Control

In both DMA and core mode, the receiver can deassert the `UART_RTS` signal to indicate that its receive buffer is almost full. Continued data transfers can cause an overrun error. The transmitter pauses when the `UART_CTS` input is in a deasserted state. In this state, the transmitter completes transmission of the data currently held in the transmit shift register (`UART_TSR`) but it does not continue with the data in the transmit hold register (`UART_THR`). If the `UART_CTS` pin is asserted again, the transmitter resumes and loads the content of `UART_THR` register into the `UART_TSR` register.

Bit Rate Generation

The peripheral clock (`SCLK0`) and the 16-bit divisor in the `UART_CLK` register characterize the sample clock. The UART uses the `UART_CTL.EN` bit to enable the clock. By default, every serial bit is oversampled 16 times. The bit clock is 1/16th of the sample clock. If not in IrDA mode, the bit clock can equal the sample clock if the `UART_CLK.EDBO` bit is set, so that the following equation applies:

$$\text{Bit Rate} = \text{SCLK0} / 16^{(1-\text{EDBO})} \times \text{Divisor}$$

The *UART Bit Rate Examples with 100 MHz SCLK* table provides example divide factors required to support standard baud rates at an `SCLK0` of 100 MHz.

Table 18-7: UART Bit Rate Examples with 100 MHz SCLK0

Bit Rate (bits/sec)	EDBO = 0			EDBO = 1		
	DL	Actual	% Error	DL	Actual	% Error
2400	2604	2400.15	0.006	41667	2399.98	0.001
4800	1302	4800.31	0.006	20833	4800.08	0.002

Table 18-7: UART Bit Rate Examples with 100 MHz SCLK0 (Continued)

Bit Rate (bits/sec)	EDBO = 0			EDBO = 1		
	DL	Actual	% Error	DL	Actual	% Error
9600	651	9600.61	0.006	10417	9599.69	0.003
19200	326	19171.78	0.147	5208	19201.23	0.006
38400	163	38343.56	0.147	2604	38402.46	0.006
57600	109	57339.45	0.452	1736	57603.69	0.006
115200	54	115740.74	0.469	868	115207.37	0.006
921600	7	892857.14	3.119	109	917431.19	0.452
1500000	4	1562500	4.167	67	1492537.31	0.498
3000000	2	3125000	4.167	33	3030303.03	1.01
6250000	1	6250000	0	16	6250000	0

NOTE: Properly select the SCLK0 frequencies. Even multiples of bit rate decrease the error percentage.

Setting the bit clock equal to the sample clock (UART_CLK.EDBO =1) improves the bit rate granularity and the bit clock matches with the bit rate of the communication partner. The disadvantage is that the power dissipation is higher and sample points are not always accurate. Therefore, use UART_CLK.EDBO mode only when bit rate accuracy is not acceptable in the UART_CLK.EDBO=0 mode.

The UART_CLK.EDBO=1 mode is not intended to increase the speed of operation beyond the electrical limitations of the UART transfer protocol.

Autobaud Detection

At the chip level, the UART_RX pin is typically routed to an alternate capture input (TIMER_ACI [nn]) of a general-purpose timer. When working in width capture mode, the processor uses this general-purpose timer to detect the bit rate applied to the UART_RX pin automatically by an external device. It often uses the capture capabilities of the timer to supervise the bit rate at run time. If the UART communicates with any device supplied by a weak clock oscillator that drifts over time, the processor can then readjust its UART bit rate dynamically, as required.

Often, the processor uses autobaud detection for initial bit rate negotiations where it is most likely a slave device waiting for the host to send a predefined autobaud character. This situation is common for UART booting. Do not enable the UART_CTL.EN bit while autobaud detection is in-process, to prevent the UART from starting a receive operation with incorrect bit rate matching. Alternatively, set the UART_CTL.LOOP_EN bit to disconnect the UART from its UART_RX pin.

A software routine can detect the pulse widths of serial stream bit cells. The sample base of the timer is synchronous with the UART operation (all derived from the same SCLK0). The UART uses pulse widths to calculate the bit rate divider as follows:

$$\text{Divisor} = \text{TIMER_TMR}[n]_WID / (16^{(1-\text{EDBO})} \times \text{Number of captured UART bits})$$

To increase the number of timer counts and the resolution of the captured signal, do not measure just the pulse width of a single bit. Instead, enlarge the pulse of interest over more bits. Traditionally, a NULL character (ASCII 0x00) is used in autobaud detection, as shown in the *Autobaud Detection* figure.

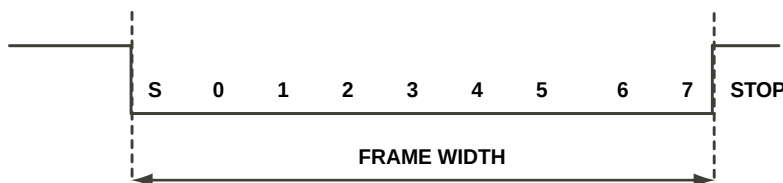


Figure 18-3: Autobaud Detection

Because the example frame encloses 8 data bits and 1 start bit, apply the following formula:

$$\text{Divisor} = \text{TIMER_TMR}[n]_WID / (16^{(1-\text{EDBO})} \times 9)$$

NOTE: For processor-specific mapping of timer alternate capture inputs to the UARTs of the processor, see "Width Capture (WIDCAP) Mode" in the "*General-Purpose Timer (TIMER)*" chapter.

Real receive signals often have asymmetrical falling and rising edges, and the sampling logic level is not exactly in the middle of the signal voltage range. At higher bit rates, such pulse-width-based autobaud detection does not always return adequate results without extra conditioning of the analog signal. Measure signal periods to work around this issue.

For example, predefine the ASCII character "@" (0x40) as the autobaud detection character and measure the period between two subsequent falling edges. As shown in the *Autobaud Detection Character 0x40* figure, measure the period between the falling edge of the start bit and the falling edge after bit 6. Since this period encloses 8 bits, apply the following formula:

$$\text{Divisor} = \text{TIMER_TMR}[n]_PER / (16^{(1-\text{EDBO})} \times 8)$$

or:

- Divisor = $\text{TIMER_TMR}[n]_PER \gg 7$, if $\text{UART_CLK}.\text{EDBO}=0$
- Divisor = $\text{TIMER_TMR}[n]_PER \gg 3$, if $\text{UART_CLK}.\text{EDBO}=1$

The *Autobaud Detection Character 0x40* figure shows the ASCII "@" (0x40) detection character.

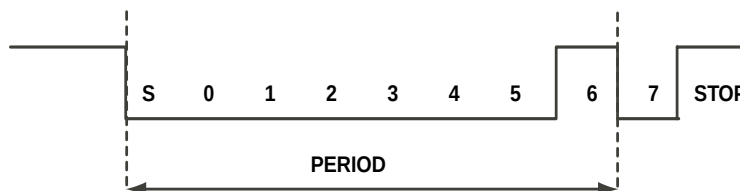


Figure 18-4: Autobaud Detection Character 0x40

UART Debug Features

The UART can automatically calculate and transmit a parity bit. The *UART Parity* table summarizes parity behavior assuming 8-bit data words (`UART_CTL.WLS=b#11`).

Table 18-8: UART Parity

PEN	STP	EPS	Data (hex)	Data (binary, LSB first)	Parity
0	x	x	x	x	None
1	0	0	0x60	0000 0110	1
1	0	0	0x57	1110 1010	0
1	0	1	0x60	0000 0110	0
1	0	1	0x57	1110 1010	1
1	1	0	x	x	1
1	1	1	x	x	0

The two force error bits, `UART_CTL.FPE` and `UART_CTL.FFE`, are intended for test purposes. They are useful for debugging software, especially in loopback mode.

The UART can be set to internal loopback mode (`UART_CTL.LOOP_EN=1`). Loopback mode disconnects the input of the receiver from the receive pin and internally redirects the transmit output to the receiver. The transmit pin remains active and continues to transmit data externally as well. Loopback mode also forces the `UART_RTS` pin to deassert, disconnects the `UART_STAT.CTS` bit from the `UART_CTS` input pin, and connects the internal version of `UART_RTS` to the `UART_STAT.CTS` bit.

Additionally, the `UART_TX` pin can be forced to zero asynchronously using the `UART_CTL.SB` bit.

UART Operating Modes

The following sections describe the main operating modes of the UART.

- [UART Mode](#)
- [IrDA SIR Mode](#)
- [Multi-Drop Bus Mode](#)

UART Mode

The UART mode follows an asynchronous serial communication protocol with these options:

- 1 start bit
- 5–8 data bits
- Address bit (available in MDB mode only)

- None, even, odd or sticky parity
- 1, 1½, or 2 stop bits (1½ stop bits valid only in 5-bit word length)

The `UART_CTL` register controls the format of received and transmitted character frames. Data is always transmitted and received with the least significant bit (LSB) first.

The *Bit Stream on a UART TX Pin Transmitting an “S” Character (0x53)* figure shows a typical physical bit stream measured on a `UART_TX` pin.

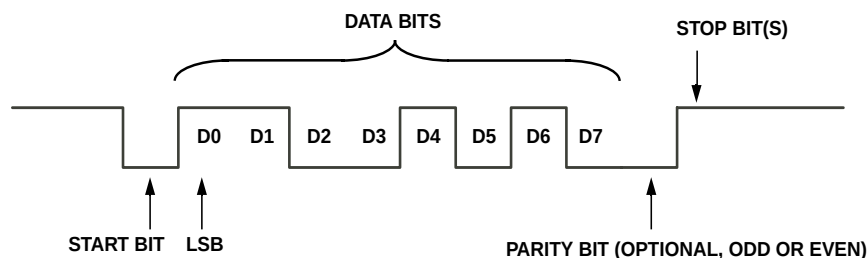


Figure 18-5: Bit Stream on a UART TX Pin Transmitting an “S” Character (0x53)

IrDA SIR Mode

The UART also supports serial data communication by way of infrared signals, according to the recommendations of the Infrared Data Association (IrDA). The physical layer known as IrDA SIR (9.6/115.2 Kbps rate) is based on return-to-zero-inverted (RZI) modulation. The UART does not support pulse position modulation.

Using the 16x data rate clock, RZI modulation is achieved by inverting and modulating the non-return-to-zero (NRZ) code normally transmitted by the UART. On the receive side, the UART uses a 16x clock to determine an IrDA pulse sample window, from which it recovers the RZI modulated NRZ code.

NOTE: The `UART_CLK.EDBO` bit is not valid in IrDA mode. Clear (=0) this bit in this mode.

Multi-Drop Bus Mode

The UART uses a protocol for point-to-point connections as well as in networks where the EIA-485 standard is representative of UART-based bus systems. The EIA-232 standard defines point-to-point connections. In such networks, node addressing is important.

In a multidrop bus (MDB) network, for example, an address bit enhances the UART frame. The address bit is inserted between the data bits and the optional parity bit. To configure the UART for MDB mode, set the mode of operation bits (`UART_CTL.MOD [5:4]`) to 01.

By convention, the address bit is transmitted low for regular data bytes. When set, it marks special address bytes that require the attention of all nodes on the network.

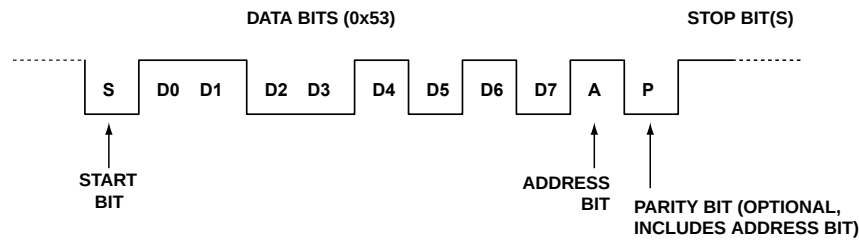


Figure 18-6: UART Frame with Address Bit

All transmit operations are processed through the transmit buffer register (`UART_THR`), so all DMA data transmissions clear the address bit. If data is written to the transmit address or insert pulse register (`UART_TAIP`) instead, the same transmit operation is initiated with the only exception that the address bit is sent high.

The UART uses the `UART_STAT.ADDR` bit of the receiver to signal whether the previously received frame had the address bit set or not. Hardware updates it every time a new frame is received. When the enable address word interrupt bit (`UART_IMSK.EAWI`) is set, the reception of an address byte triggers a special status interrupt request.

The address sticky bit (`UART_STAT.ASTKY`) is the sticky version of the `UART_STAT.ADDR` bit. Hardware sets it whenever the `UART_STAT.ADDR` bit is set. Software can clear the `UART_STAT.ASTKY` bit with a W1C operation.

In MDB mode, only address bytes progress to the receive FIFO by default. Data bytes are gated unless the `UART_STAT.ASTKY` bit is set. The receiver ignores all traffic on the UART bus. This way, the processor can go into low-power mode and interrupt activity does not load the processor every time a frame is transmitted on the UART bus. If, however, an address frame is transmitted, the receiver immediately samples all further traffic. A software routine can analyze the received data, decide whether it was of relevance for the local network node, and W1C the `UART_STAT.ASTKY` bit if it was not.

Software can overrule of the hardware address frame detection by setting the `UART_STAT.ADDR` bit and (indirectly) the `UART_STAT.ASTKY` bit with a W1S operation.

The MDB mode follows an asynchronous serial communication protocol with the following options.

- 1 start bit
- 5–8 data bits
- Address bit
- None, even, odd or sticky parity
- 1, 1½, or 2 stop bits (1½ stop bits are valid only in 5-bit word length)

NOTE: If the address bit and parity bit are both enabled, the parity check and generation includes the address bit in its parity calculation.

UART Data Transfer Modes

The UART can transfer data using both the core and DMA. Receive and transmit paths operate independently except that the bit rate and the frame format are identical for both transfer directions. Transmit and receive channels are both buffered. The `UART_THR` register buffers the transmit shift register (`UART_TSR`) and the `UART_RBR` register buffers the receive shift register (`UART_RSR`).

UART Mode Transmit Operation (Core)

In core mode, the processor core moves data to and from the UART. A write to the `UART_THR` register initiates the transmit operation. If no former operation is pending, the `UART_THR` register passes the data immediately to the `UART_TSR` register. There, it is shifted out at the bit rate characterized by the `UART_CLK` register, with start, stop, and parity bits appended as defined by the `UART_CTL` register.

The `UART_THR` register and the `UART_TSR` register can be modeled as a two-stage transmit buffer. The least significant bit (LSB) always transmits first. This bit is bit 0 of the value written to the `UART_THR` register.

UART Mode LIN Break Command

Some UART-based protocols demand synchronization methods that are not native to standard UART implementations. For example, the Local Interconnect Network (LIN) protocol requires a low-pulse of well-defined transmit length as a prologue to every multi-byte message. Its length must be at least 13 bit-times.

With previous UARTs, there were two options to implement this protocol:

- A null byte is transmitted with a temporarily lowered bit rate, or
- A software counter generates the period and the asynchronous set break (SB) mechanisms pull the transmit pin low

Since both methods have their disadvantages, the newer UART introduces a new inter-frame gap technique.

The feature is not available in MDB or IrDA operating modes. However, in standard UART mode (bits `UART_CTL.MOD[5:4] = 00`), a write to the `UART_TAIP` register initiates the transmission of an inter-frame pulse. If the transmit buffer is not empty, the UART first transmits all bytes in the queue. It only initiates with pulse generation after the last stop bit of the last byte has been shifted out.

The value written into the `UART_TAIP` register defines the nature and the duration of the transmitted pulse. Bits [6:0] control the duration in bit-times and bit [7] controls the value (duration = `UART_TAIP[6:0] / UART_CLK[15:0]`). If `UART_TAIP[7]` is set, and an active high pulse is issued, the number of stop bits is extended. If `UART_TAIP[7]` is cleared, a low pulse is generated. Invert the polarity using the `UART_CTL.FCPOL` bit. Writing a value of 13 into the `UART_TAIP` register generates the break command as required by the LIN protocol.

NOTE: If the `UART_CTL.TPOLC` bit is enabled, an inverted most-significant bit can be transmitted.

NOTE: If another transmission is pending (in the `UART_TSR` register), the `UART_TAIP` initiated pulse is queued until after all pending operations have finished and all stop bits are transmitted.

The transmission of break command/inter-frame gap precedes transmission of the number of stop bits as set in the `UART_CTL.STB` and `UART_CTL.STBH` bit fields.

The UART receiver can detect break commands through the break indicator (`UART_STAT.BI`) flag. This flag reports that an entire UART frame has been received in low state. It does not report whether the duration of the received low pulse was exact or at least 13 bit-times as LIN masters transmit. Typically, the break indicator meets LIN requirements. The processor can use GP timers to determine the pulse width more precisely, if necessary.

Each `UART_RX` pin is also routed to a GP timer through its alternate capture input (TACI). This functionality is not only useful for bit rate detection (*autobaud*) but also helps to measure the pulse widths precisely on the `UART_RX` input. Additionally, the GP timers can issue an interrupt request or a fault condition when the received pulse width is shorter than a bit time or longer than the worst-case break condition. The windowed watchdog width mode of the GP timers controls this functionality.

UART Mode Receive Operation (Core)

The receive operation uses the same data format as the transmit configuration except that one valid stop bit is always sufficient. The `UART_CTL.STB` and `UART_CTL.STBH` bits have no impact on the receiver.

The UART receiver senses the falling edges of the receive input. When it detects an edge, the receiver starts sampling the input according to settings in the `UART_CLK` register. The receiver samples the start bit (majority sampling) close to its midpoint. If sampled low, it assumes a valid start condition. Otherwise, it discards the detected falling edge.

After detection of the start bit, the received word is shifted into the `UART_RSR` register.

After the corresponding stop bit is received, the content of the `UART_RSR` register is transferred to the 8-deep receive FIFO and is accessible by reading the `UART_RBR` register.

The receive FIFOs and the `UART_RBR` register act as a 9-stage receive buffer. If the stop bit of the ninth word is received before software reads the `UART_RBR` register, an overrun error is reported. Overruns protect data in the `UART_RBR` register and the receive FIFO from being overwritten by further data until the software clears the `UART_STAT.OE` bit. However, the data in the `UART_RSR` register is immediately destroyed as soon as the overrun occurs.

The sampling clock is 16 times faster than the bit clock. The receiver oversamples every bit 16 times and makes a majority-decision based on the middle three samples. This functionality improves immunity against noise and hazards on the line. The receiver disregards spurious pulses of less than two times the sampling clock period.

Normally, the receiver samples every incoming bit at exactly the 7th, 8th and 9th sample clock. If, however, the `UART_CLK.EDBO` bit is set to 1, the receiver samples bits roughly at 7/16th, 8/16th, and 9/16th of their period. This configuration achieves better bit rate granularity and accuracy as required at high operation speeds. Hardware design must ensure that the incoming signal is stable between 6/16th and 10/16th of the nominal bit period.

Reception starts when the UART receiver detects a falling edge on the `UART_RX` input pin. The receiver attempts to see a start bit. The data is shifted into the `UART_RSR` register. After the ninth sample of the first, the receiver processes the stop bit and copies the received data to the 8-stage receive FIFO. The `UART_RSR` recovers for further data reception.

The receiver samples data bits close to their midpoint. Because the receiver clock is typically asynchronous to the data rate of the transmitter, the sampling point can drift relative to the center of the data bits. The sampling point is synchronized again with each start bit, so the error accumulates only over the length of a single word. The polarity of received data is selectable, using the `UART_CTL.RPOLC` bit.

NOTE: The receiver checks for only a single stop bit. After the third sample of the first stop bit has been received (at time 9/16th of the stop bit duration), the receiver immediately acts (status update). It then prepares for new falling edge detection (start detection).

IrDA Transmit Operation

To generate the IrDA pulse transmitted by the UART, the normal NRZ output of the transmitter is first inverted if the `UART_CTL.TPOLC` bit is configured for active-low operation. In this configuration, a zero is transmitted as a high pulse of 16 UART clock periods and a one is transmitted as a low pulse for 16 UART clock periods. Then, six UART clock periods delay the leading edge of the pulse. Similarly, eight UART clock periods truncate the trailing edge of the pulse. For a 16-cycle UART clock period, this operation results in the final representation of the original zero as a high pulse of only 3/16 clock periods. The *IrDA Transmit Pulse* figure shows how the pulse is centered around the middle of the bit time. The final IrDA pulse is fed to the off-chip infrared driver.

This modulation approach ensures a pulse width output from the UART of three cycles high out of every 16 UART clock cycles. As shown in the *IrDA Transmit Pulse* figure, the error terms associated with the bit rate generator are small and well within the tolerance of most infrared transceiver specifications.

NOTE: In IrDA mode, writes to the `UART_TAIP` register are equivalent to writes to the `UART_THR` register.

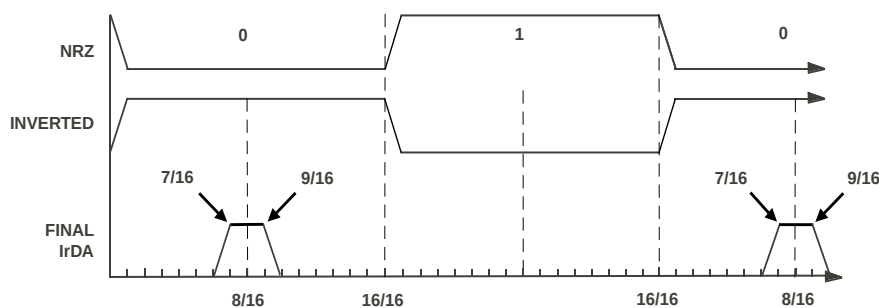


Figure 18-7: IrDA Transmit Pulse

IrDA Receive Operation

The IrDA receiver function is more complex than the transmit function. The receiver must discriminate the IrDA pulse and reject noise. The receiver looks for the IrDA pulse in a narrow window centered around the middle of the expected pulse.

Glitch filtering is accomplished by counting 16 system clocks from the time the receiver detects an initial pulse. If the pulse is absent when the counter expires, the receiver interprets it as a glitch. Otherwise, the receiver interprets it as a zero. This assessment is acceptable because glitches originating from on-chip capacitive cross-coupling typically

do not last for more than a fraction of the system clock (SCLK0) period. Appropriate shielding avoids sources outside of the chip and not part of the transmitter. The only other source of a glitch is the transmitter itself. The processor relies on the transmitter to perform within specification. If the transmitter violates the specification, unpredictable results can occur. The 4-bit counter adds an extra level of protection at a minimal cost.

NOTE: Because SCLK0 can change across systems, the longest glitch tolerated is inversely proportional to the SCLK0 frequency.

A counter that is clocked at the 16x bit-time sample clock determines the receive sampling window. The sampling window is resynchronized with each start bit by centering the sampling window around the start bit.

The polarity of receive data is selectable, using the `UART_CTL.RPOLC` bit. The *IrDA Receiver Pulse Detection* figure provides examples of each polarity type.

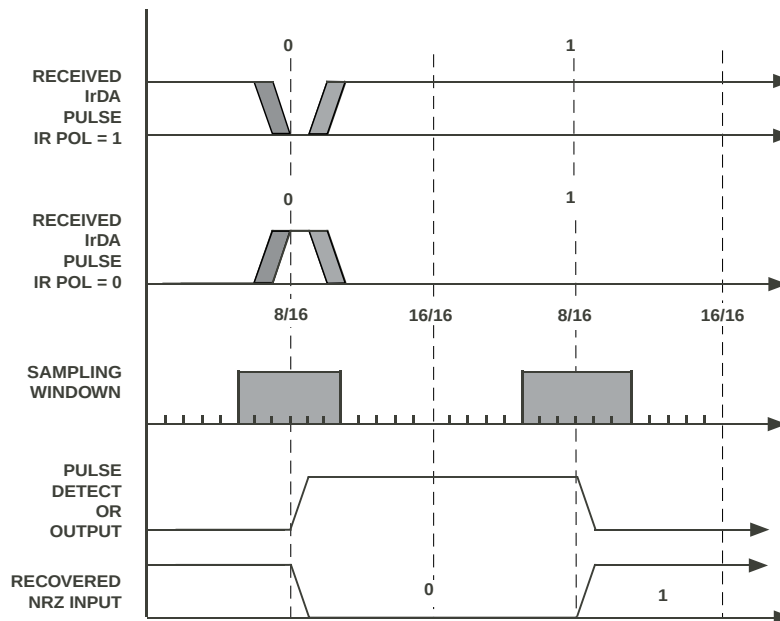


Figure 18-8: IrDA Receiver Pulse Detection

MDB Transmit Operation

In MDB mode, receive and transmit paths operate independently from each other, except for sharing bit rate and frame formats for both transfer directions.

Transmit operation is initiated by writing the `UART_THR` or `UART_TAIP` registers. A write to the `UART_THR` register transmits the written word with the appending address bit set low. A write to the `UART_TAIP` register transmits the written word with the appended address bit set high. The data is moved into the `UART_TSR` register, where it is shifted out at the bit rate programmed by the `UART_CLK` register, with start, stop, address, and parity bits appended, as required.

If DMA is enabled, the DMA engine always writes the data into the `UART_THR` register, and the written word is transmitted with the appending address bit set low.

The polarity of transmit data is selectable, using the `UART_CTL.TPOLC` bit.

MDB Receive Operation

Receive operations use the same data format as the transmit configuration, except that the number of stop bits is always assumed to be 1. After detection of the start bit, the received word is shifted into the `UART_RSR` register at the programmed bit.

Normally, the receiver samples every incoming bit at exactly the 7th, 8th and 9th sample clock. If, however, the `UART_CLK.EDBO` bit is set, the receiver samples the bits roughly at 7/16th, 8/16th, and 9/16th of their period. This configuration achieves better bit rate granularity and accuracy needed at high operation speeds. Hardware design must ensure that the incoming signal is stable between 6/16th and 10/16th of the nominal bit period.

After the appropriate number of bits (including address, parity, and stop bits) is received, the `UART_RSR` register is transferred to the receive FIFO and accessible through the `UART_RBR` register.

The polarity of receive data is selectable, using the `UART_CTL.RPOLC` bit.

DMA Mode

In DMA mode, separate receive and transmit DMA channels move data between the UART and memory. The software does not have to move data; it just has to set up the appropriate transfers either through the descriptor mechanism or through autobuffer mode.

DMA channels provide a 4-deep FIFO, resulting in total buffer capabilities of 6 words at the transmit side and 9 words at the receive side. In DMA mode, the bus activity and arbitration mechanism determine the latency. The processor loading and interrupt priorities do not determine the latency.

To enable UART DMA, first set up the system DMA control registers. Then, enable the `UART_IMSK.ERBFI` or `UART_IMSK.ETBEI` interrupts. This sequence is necessary because these interrupt request lines double as DMA request lines. With DMA enabled, once these requests are received, the DMA control unit generates a direct memory access. If DMA is not enabled, the UART interrupt is passed on to the system interrupt handling unit. The status interrupt for the UART goes directly to the system event controller (SEC), bypassing the DMA unit completely.

For transmit DMA, programs must set the `DMA_CFG.SYNC` bit. With this bit set, interrupt generation is delayed until the entire DMA FIFO is drained to the UART module. The UART transmit DMA interrupt service routine can disable the DMA or to clear the `UART_IMSK.ETBEI` control bit only when the `DMA_CFG.SYNC` bit is set. Otherwise, up to four data bytes can be lost.

When the `UART_IMSK.ETBEI` bit is set, an initial transmit DMA request is issued immediately. The program then clears the `UART_IMSK.ETBEI` bit through the DMA service routine.

In DMA transmit mode, the `UART_IMSK.ETBEI` bit enables the peripheral request to the DMA FIFO. The `DMA_CFG.EN` bit enables the strobe on the memory side. If the DMA count is less than the DMA FIFO depth, which is 4, then the DMA interrupt can be requested before the `UART_IMSK.ETBEI` bit is set. If this behavior is unwanted, set the `DMA_CFG.SYNC` bit.

Regardless of the `DMA_CFG.SYNC` setting, the DMA stream has not left the UART transmitter completely at the time the interrupt request is generated. Transmission can abort in the middle of the stream, causing data loss, when the UART clock was disabled without extra synchronization with the `UART_STAT.TEMT` bit.

The UART provides functionality to avoid resource-consuming polling of the `UART_STAT.TEMT` bit. The `UART_IMSK_SET.EDTPTI` bit enables the `UART_STAT.TEMT` bit to trigger a DMA interrupt. To delay the DMA completion interrupt until the last data word of a STOP DMA has left the UART, keep the `DMA_CFG.INT` bit cleared and set the `UART_IMSK_SET.EDTPTI` bit instead. Then, the normal DMA completion interrupt is suppressed. Later, the `UART_STAT.TEMT` event triggers a DMA interrupt after the last word of the DMA has left the UART transmit buffers. If `DMA_CFG.INT` and `UART_IMSK.EDTPTI` are set, when finishing STOP mode, the DMA requests two interrupts.

The DMA of the UART module supports 8-bit and 16-bit operation, but not 32-bit operation. It does not support sign-extension.

Mixing DMA and Core Modes

Switching from DMA mode to core operation dynamically requires some consideration, especially for transmit operations. By default, the interrupt timing of the DMA is synchronized with the memory side of the DMA FIFOs. Normally, the transmit DMA completion interrupt is generated after the last byte is copied from the memory into the DMA FIFO. The transmit DMA interrupt service routine is not yet permitted to disable the `DMA_CFG.EN` bit. The interrupt is requested when the `DMA_STAT.IRQDONE` bit is set. The `DMA_STAT.RUN` bit, however, remains set until the data has completely left the transmit DMA FIFO.

When planning to switch from a DMA to core mode, set the `DMA_CFG.SYNC` bit in the word of the last descriptor or work unit before handing over control. Then, after the interrupt request occurs, software can write new data into the `UART_THR` register as soon as the `UART_STAT.THRE` bit permits. If the `DMA_CFG.SYNC` bit cannot be set, software can poll the `DMA_STAT.RUN` bit instead. Alternatively, using the `UART_IMSK.EDTPTI` bit can avoid expensive status bit polling.

When switching from core to DMA operation, ensure that the first DMA request is issued properly. If the DMA is enabled while the UART is still transmitting, no precaution is required. If, however, the DMA is enabled after the `UART_STAT.TEMT` bit is high, pulse the `UART_IMSK.ETBEI` bit to initiate DMA transmission.

Setting Up Hardware Flow Control

The following steps show how to set up UART hardware flow control:

1. Configure automatic or manual hardware flow control for the receiver through the `UART_CTL.ARTS` bit, or the transmitter through the `UART_CTL.ACTS` bit.
2. Configure `UART_CTS` and `UART_RTS` polarity through the `UART_CTL.FCPOL` bit.

On reset, when the UART is not yet enabled and the port multiplexing has not been programmed, the `UART_RTS` pin is not driven. Some applications require a resistor to pull the `UART_RTS` signal to either state during reset.

UART Event Control

Status flags in the `UART_STAT` register are available to signal data reception, parity, and error conditions, if necessary.

DMA and Interrupt Multiplexing

See the *Direct Memory Access (DMA)* chapter for information on DMA multiplexing. Several interrupts and DMA channels in the UART can be multiplexed.

NOTE: To operate in interrupt mode without using DMA channels, set the `UART_IMSK.ELSI` bit. This configuration redirects receive and transmit requests to the status interrupt output. The status interrupt goes directly to the SEC without going through the DMA controller.

Interrupt Masks

Each UART features a set of interrupt mask registers: `UART_IMSK`, `UART_IMSK_SET`, and `UART_IMSK_CLR`. The `UART_IMSK` register supports read/write operations. Writing ones to the `UART_IMSK_SET` register enables interrupts, writing ones to the `UART_IMSK_CLR` register disables them. Reads from either register return the enabled bits. This way, different interrupt service routines can control transmit, receive, and status interrupt requests independently and easily.

The UART module uses the `UART_IMSK` registers to enable requests for system handling of empty or full states of data registers. Unless polling is used as a means of action, the `UART_IMSK.ERBFI` and `UART_IMSK.ETBEI` bits in this register are normally set.

Each UART module has three interrupt outputs. It uses one for transmission, one for reception, and one for reporting status events. The UART module routes transmit and receive requests through the DMA controller. The status request goes directly to the .

If the associated DMA channel is enabled, the request functions as a DMA request. If the DMA channel is disabled, it simply forwards the request to the SEC. A DMA channel must be associated with the UART module to enable transmit and receive interrupts. Otherwise, transmit and receive requests cannot be forwarded.

NOTE: To operate in interrupt mode without using DMA channels, set the `UART_IMSK.ELSI` bit. This configuration redirects receive and transmit requests to the status interrupt request output. The status interrupt goes directly to the SEC without going through the DMA controller.

Interrupt Servicing

Interrupt service routines (ISRs) perform UART writes and reads. Separate interrupt lines are provided for transmit, receive, and status. The `UART_IMSK` register group enables the independent interrupts individually. To enable UART transmit interrupts, set the `UART_CTL.EN` bit.

The ISRs evaluate the status bits in the `UART_STAT` register to determine the signaling interrupt source. The system event controller for the processor assigns and unmask interrupts. The ISRs must clear the interrupt latches explicitly. To reduce interrupt frequency on the receive side in core mode, use the `UART_IMSK.ERFCI` status interrupt as an alternative to the regular `UART_IMSK.ERBFI` receive interrupt. Hardware must ensure that at least two (if `UART_CTL.RFIT=0`) or four (if `UART_CTL.RFIT=1`) words are available in the receive buffer by the time the interrupt is requested.

Transmit Interrupts

The UART module uses the `UART_IMSK_SET.ETBEI` bit to enable transmit interrupt requests.

The `UART_THR` and `UART_TAIP` registers are the same physical register, and both affect the signaling of the `UART_STAT.TEMT`, `UART_STAT.TFI`, and `UART_STAT.THRE` bits similarly.

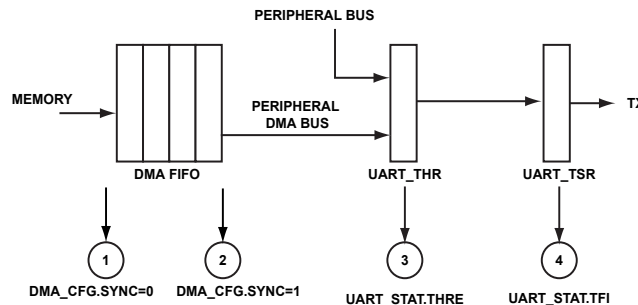


Figure 18-9: Transmit Interrupts

The UART module asserts the transmit request along with the `UART_STAT.THRE` bit, indicating that the transmit buffer is ready for new data. The `UART_STAT.THRE` bit resets to 1. When the `UART_IMSK_SET.ETBEI` bit is set, the UART module immediately issues an interrupt or DMA request. This way, no special handling of the first character is required when transmission of a string is initiated. Set the `UART_IMSK_SET.ETBEI` bit and let the interrupt service routine load the first character from memory and write it to the `UART_THR` register in the normal manner. ISRs can clear the `UART_IMSK.ETBEI` bit through the `UART_IMSK_CLR` register when the string transmission has completed.

Hardware clears the `UART_STAT.THRE` bit when new data is written to the `UART_THR` register. These write operations also clear the transmit interrupt request. However, they also initiate further transmission. If continued transmission is undesirable, the UART module can alternatively clear the transmit request through the `UART_IMSK_CLR.ETBEI` bit register. Transfers of data from the `UART_THR` register to the `UART_TSR` register reset this status flag in the `UART_STAT` register.

ISRs can interrogate the `UART_STAT.TEMT` bit to discover any ongoing transmission. The sticky counterpart of the `UART_STAT.TEMT` bit, `UART_STAT.TFI`, indicates when the transmit buffer has drained and can trigger a status interrupt. When data is pending in either one of these registers, the `UART_STAT.TEMT` flag is low. As soon as all data has left the `UART_TSR` register, the `UART_STAT.TEMT` bit goes high again and indicates that all pending transmit operations (including stop bits) have finished. Then, it is safe to disable the `UART_CTL.EN` bit or to three-state off-chip line drivers. Then, the UART module can generate an interrupt either through the status interrupt channel when the `UART_IMSK.ETFI` bit is set, or through the DMA controller when enabled by the `UART_IMSK.EDTPTI` bit.

When enabled by the `UART_IMSK.ETBEI` bit, the `UART_STAT.THRE` flag requests data along the peripheral command lines to the DMA controller (referred to as TXREQ). This signal is routed through the DMA controller. If the associated DMA channel is enabled, the TXREQ signal functions as a DMA request, otherwise the DMA

controller simply forwards it to the SEC. Alternatively the `UART_IMSK.ETXS` bit can redirect the transmit interrupts to the UART status interrupt.

With interrupts disabled, the UART module can poll the status flags to determine when data is ready to move. Because polling is processor intensive, it is not typically used in real-time signal processing environments. Since read operations from `UART_STAT` registers have no side effects, different software threads can interrogate these registers without mutual impacts.

Polling the `SEC_SSTAT[n]` register without enabling the interrupts by the `SEC_CCTL[n]` register is an alternate method of operation to consider. Software can write up to two words into the `UART_THR` register before enabling the UART clock. As soon as the `UART_CTL.EN` bit is set, the UART module sends those two words.

Receive Interrupts

The UART module uses the `UART_IMSK_SET.ERBFI` bit to enable receive interrupt requests. If set, the `UART_STAT.DR` flag requests an interrupt on the dedicated RXREQ output, indicating that new data is available in the `UART_RBR` register. This signal is routed through the DMA controller. If the associated DMA channel is enabled, the RXREQ signal functions as a DMA request; otherwise the DMA controller simply forwards it to the SEC. Alternatively, if no DMA channel is assigned to the UART, the `UART_IMSK.ERXS` bit can redirect the receive interrupts to the UART status interrupt. When software reads the `UART_RBR` register, hardware clears the `UART_STAT.DR` bit again, which, in turn, clears the receive interrupt request.

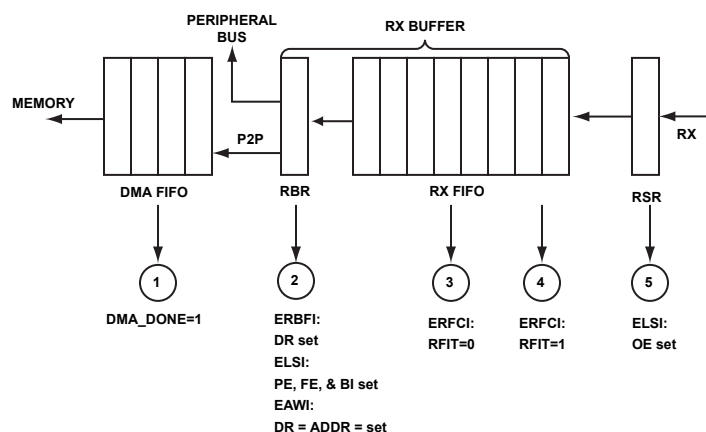


Figure 18-10: Receive Interrupts

Hardware updates the following:

- `UART_STAT.DR` bits
- `UART_STAT.ADDR` bits
- `UART_STAT.ASTKY` bits
- `UART_STAT.PE` bits
- `UART_STAT.FE` bits
- `UART_STAT.BI` bits

- [UART_RBR](#) register

The `UART_STAT.OE` bit is updated as soon as an overflow condition occurs (for example when a stop bit for a frame is received and the receive FIFO is full). When software does not read the [UART_RBR](#) register in time, the received data is protected from being overwritten by new data until software clears the `UART_STAT.OE` bit. Only the content of the [UART_RSR](#) register can be overwritten in the overrun case.

The UART module uses the `UART_STAT.RFCS` bit to monitor the state of the 8-deep receive FIFO. It uses the `UART_CTL.RFIT` bit to control the behavior of the buffer. If `UART_CTL.RFIT` is zero, the `UART_STAT.RFCS` bit is set when the receive buffer holds four or more words. If `UART_CTL.RFIT` is set, the `UART_STAT.RFCS` bit is set when the receive buffer holds seven or more words. Hardware clears the `UART_STAT.RFCS` bit when a core or DMA reads the [UART_RBR](#) register and when the buffer is flushed below the level of four (`UART_CTL.RFIT=0`) or seven (`UART_CTL.RFIT=1`). If the associated interrupt bit `UART_IMSK.ERFCI` is enabled, a status interrupt request is reported when the `UART_STAT.RFCS` bit is set.

If errors are detected during reception, an interrupt can be requested from the status interrupt output. This status interrupt request goes directly to the SEC. The bit enables status interrupt requests.

The controller detects the following error conditions, shown with their associated bits in the [UART_STAT](#) register.

- Overrun error (`UART_STAT.OE` bit)
- Parity error (`UART_STAT.PE` bit)
- Framing error or invalid stop bit (`UART_STAT.FE` bit)
- Break indicator (`UART_STAT.BI` bit)

Status Interrupts

The UART module uses status interrupt channels for the following purposes:

- Line status interrupt requests
- Flow control interrupt requests
- Receive FIFO threshold interrupt requests
- Transmission finished interrupt request

The UART module uses the `UART_IMSK.ELSI` bit to enable the line status interrupts. If set, the status interrupt request is asserted with one of the `UART_STAT.BI`, `UART_STAT.FE`, `UART_STAT.PE`, or `UART_STAT.OE` receive errors bits. A W1C operation to the [UART_STAT](#) register clears the error bits. Once all error conditions are cleared, the interrupt request deasserts.

The UART module uses the `UART_IMSK_SET.ERFCI` bit to enable the receive FIFO count interrupt. If set, a status interrupt request is generated when the `UART_STAT.RFCS` is active. The `UART_STAT.RFCS` bit indicates a receive buffer threshold level. If the `UART_CTL.RFIT` bit is cleared, software can safely read two words out of the [UART_RBR](#) register by the time the `UART_STAT.RFCS` interrupt occurs.

If the `UART_CTL.RFIT` bit is set, software can safely read four words. The interrupt request and the `UART_STAT.RFCS` bit are cleared when the `UART_RBR` is read enough of times, so that the receive buffer drains below the threshold of two (`UART_CTL.RFIT=0`) or four (`UART_CTL.RFIT=1`). Because in DMA mode a status service routine may not be permitted to read `UART_RBR`, this interrupt is only recommended in core mode. In DMA mode, use this functionality for error recovery only.

The UART module uses the `UART_IMSK_SET.EDSSI` bit to enable the flow control interrupts. If active, a status interrupt is generated when the sticky `UART_STAT.SCTS` bit register is set, indicating that the `UART_CTS` input of the transmitter been reasserted. A W1C operation to the `UART_STAT.SCTS` bit clears the interrupt request.

The UART module uses the `UART_IMSK_SET.ETFI` bit to enable the transmission finished interrupt. If active, a status interrupt request is asserted when the `UART_STAT.TFI` bit is set. The `UART_STAT.TFI` is the sticky version of the `UART_STAT.TEMT` bit, indicating that a byte that started transmission has finished. A W1C operation to the `UART_STAT.TFI` bit clears the interrupt request.

Multi-Drop Bus Events

Several status bits and interrupt features in the `UART_STAT` and `UART_IMSK` registers facilitate efficient data handling in multi-drop bus mode. These features include the address (`UART_STAT.ADDR`) bit, the address sticky (`UART_STAT.ASTKY`) bit and the enable address word interrupt (`UART_IMSK.EAWI`). One of the key features of the multi-drop bus protocol is its address bit. The address bit signifies to the slaves that the master is transmitting an address word (all read it) or a data word (only the addressed slave reads its). The UART hardware provides an efficient method of handling the situation described with the use of `UART_STAT.ASTKY` bit.

NOTE: The UART module uses the `UART_STAT.ASTKY` bit in multi-drop bus mode to indicate when an address operation for a peripheral is occurring. The `UART_STAT.ASTKY` bit is a sticky version of the `UART_STAT.ADDR` bit. Hardware sets the bit whenever the `UART_STAT.ADDR` bit is set. Only software clears it with a W1C operation. With the `UART_STAT.ASTKY` bit set, words are received irrespective of the mode bit or address bit setting. With the `UART_STAT.ASTKY` bit cleared, only address words (mode bit =1) are received and words with mode bit =0 are ignored in MDB mode. This bit does not affect reception in non-MDB modes. (Words with mode bit =0 are not moved from the `UART_RSR` register to the receive FIFO.)

UART Programming Model

The following sections provide basic procedures for configuring various UART operations.

Detecting Autobaud

To detect Autobaud:

1. Ensure that the timer is disabled.
2. Configure the following bits: `UART_CTL.MOD = 00`, `UART_CTL.LOOP_EN = 1`, `UART_CTL.WLS = 11` (8-bit data), and `UART_CTL.EN = 1`

3. Configure the following bits: `TIMER_TMR[n]_CFG.TMODE = 1101`, `TIMER_TMR[n]_CFG.OUTDIS = 1`, `TIMER_TMR[n]_CFG.IRQMODE = 10` and enable the timer.

4. Send test data through the host device and wait for the timer interrupt and disable the timer.

The bit rate can be derived from the timer period register value according to the formula provided in the [Auto-baud Detection](#) section.

Using Common Initialization Steps

When using the core or the DMA to execute transfers, the following steps are common to all UART modes.

1. All UART signals are multiplexed and compete with other functions at pin level. First, program the port registers according to the guidelines in the [PORT](#) chapter.
2. Program the [UART_CLK](#) register. Refer to [Bit Rate Generation](#) topic.
3. Program the [UART_CTL](#) register and enable the UART clock.

Using Core Transfers

Write data into the [UART_THR](#) register, when the `UART_STAT.THRE` bit is set, to initiate a core transmit operation. If the `UART_STAT.DR` bit is set, received data can be read from the [UART_RBR](#) register.

Using DMA Transfers

1. Make sure that the `UART_IMSK.ETBEI` or the `UART_IMSK.ERBFI` bits are cleared before configuring the DMA.
2. Configure the dedicated DMA channel.
3. Set the `UART_IMSK.ETBEI` or `UART_IMSK.ERBFI` bits to start the transfer.

Using Interrupts

Each UART features three interrupt signal outputs.

1. Enable individual interrupts in the system event controller (SEC).
2. Register IRQ handlers.
3. Use the interrupts mask registers to enable specific IRQ events.

Setting Up Hardware Flow Control

1. Configure automatic or manual hardware flow control for the receiver through the `UART_CTL.ARTS` bit, or the transmitter through the `UART_CTL.ACTS` bit.
2. Configure `UART_CTS` and `UART_RTS` polarity through the `UART_CTL.FCPOL` bit.

ADSP-2159x UART Register Descriptions

UART (UART) contains the following registers.

Table 18-9: ADSP-2159x UART Register List

Name	Description
UART_CLK	Clock Rate Register
UART_CTL	Control Register
UART_IMSK	Interrupt Mask Register
UART_IMSK_CLR	Interrupt Mask Clear Register
UART_IMSK_SET	Interrupt Mask Set Register
UART_RBR	Receive Buffer Register
UART_RSR	Receive Shift Register
UART_RXCNT	Receive Counter Register
UART_SCR	Scratch Register
UART_STAT	Status Register
UART_TAIP	Transmit Address/Insert Pulse Register
UART_THR	Transmit Hold Register
UART_TSR	Transmit Shift Register
UART_TXCNT	Transmit Counter Register

Clock Rate Register

The `UART_CLK` register divides the system clock (`SCLK0`) down to the bit clock.

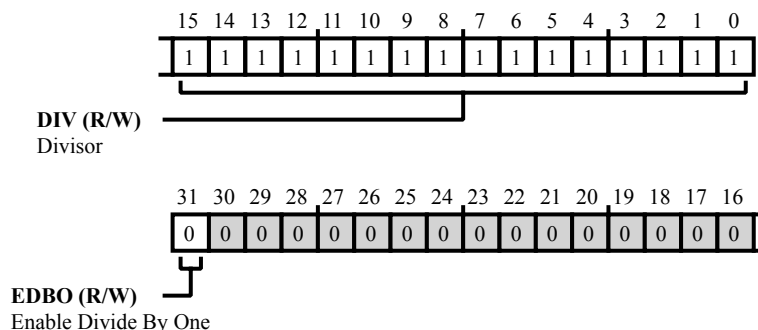


Figure 18-11: `UART_CLK` Register Diagram

Table 18-10: `UART_CLK` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	EDBO	Enable Divide By One. The <code>UART_CLK.EDBO</code> bit enables the bypassing of the divide-by-16 prescaler in bit clock generation. This functionality improves bit rate granularity, especially at high bit rates. Do not set this bit in IrDA mode. Note: Properly select the <code>SCLK</code> frequencies. Even multiples of bit rate decrease the error percentage. Setting the bit clock equal to the sample clock (<code>UART_CLK.EDBO = 1</code>) improves the bit rate granularity and the bit clock matches with the bit rate of the communication partner. The disadvantage is that the power dissipation is higher and sample points are not always accurate. Therefore, use <code>UART_CLK.EDBO = 1</code> mode only when bit rate accuracy is not acceptable in the <code>UART_CLK.EDBO = 0</code> mode. The <code>UART_CLK.EDBO = 1</code> mode is not intended to increase the speed of operation beyond the electrical limitations of the UART transfer protocol.
		0 Bit clock prescaler = 16
		1 Bit clock prescaler = 1
15:0 (R/W)	DIV	Divisor. The <code>UART_CLK.DIV</code> provides the divisor for the UART's clock bit rate calculation. The bit rate is defined by: $\text{Bit Rate} = \text{SCLK0} / (16^{(1-\text{EDBo})} \times \text{UART_CLK.DIV})$

Control Register

The `UART_CTL` register provides enable and disable control for UART and IrDA mode of operation. It also provides UART line control, permitting selection of the format of received and transmitted character frames. Modem feature control also is available from this register, including partial modem functionality to allow for hardware flow control and loopback mode.

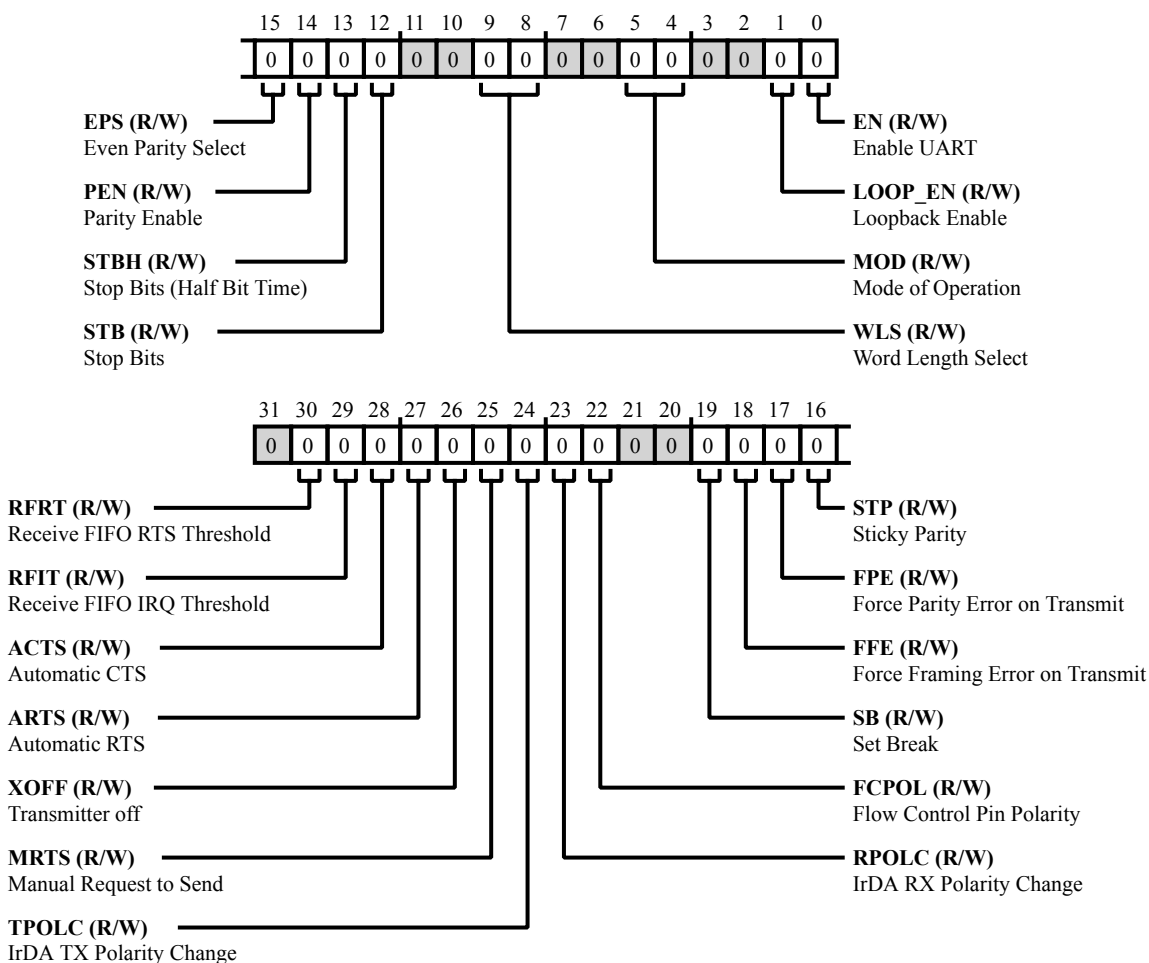


Figure 18-12: `UART_CTL` Register Diagram

Table 18-11: UART_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W)	RFRT	Receive FIFO RTS Threshold. The <code>UART_CTL.RFRT</code> bit controls <code>UART_RTS</code> pin assertion and deassertion timing. This bit is ignored if <code>UART_CTL.ARTS</code> is cleared. If set, the <code>UART_RTS</code> pin is deasserted when the receive buffer already holds seven words and an eighth start bit is detected. It is reasserted when the FIFO contains seven words or less. If cleared, the <code>UART_RTS</code> pin is deasserted when the RX buffer already holds four words and a fifth start bit is detected. The <code>UART_RTS</code> pin is reasserted when the RX buffer contains no more than 4 words.
		0 Deassert RTS if RX FIFO word count > 4; assert if <= 4
		1 Deassert RTS if RX FIFO word count > 7; assert if <= 7
29 (R/W)	RFIT	Receive FIFO IRQ Threshold. The <code>UART_CTL.RFIT</code> bit controls the timing of the <code>UART_STAT.RFCS</code> bit. If <code>UART_CTL.RFIT</code> is cleared, the receive threshold is two. If <code>UART_CTL.RFIT</code> is set, the threshold is four words in the receive buffer.
		0 Set RFCS=1 if RX FIFO count >= 4
		1 Set RFCS=1 if RX FIFO count >= 7
28 (R/W)	ACTS	Automatic CTS. The <code>UART_CTL.ACTS</code> bit must be set to enable the <code>UART_CTS</code> input pin for <code>UART_TX</code> handshaking. If enabled, the <code>UART_STAT.CTS</code> bit holds the value (if <code>UART_CTL.FCPOL</code> is set) or complement value (if <code>UART_CTL.FCPOL</code> is cleared) of the <code>UART_CTS</code> input pin. The <code>UART_STAT.CTS</code> bit can be used to determine whether the external device is ready to receive data (if <code>UART_STAT.CTS</code> is set) or whether it is busy (if <code>UART_STAT.CTS</code> is cleared). If <code>UART_CTL.ACTS</code> is cleared, the <code>UART_TX</code> handshaking protocol is disabled, and the <code>UART_TX</code> line transmits data whenever there is data to send, regardless of the value of <code>UART_CTS</code> . Software can pause ongoing transmission by setting the <code>UART_CTL.XOFF</code> bit.
		0 Disable TX handshaking protocol
		1 Enable TX handshaking protocol
27 (R/W)	ARTS	Automatic RTS. The <code>UART_CTL.ARTS</code> bit must be set to enable the <code>UART_RTS</code> input pin for <code>UART_TX</code> handshaking. If set, the hardware guarantees a minimal <code>UART_RTS</code> pin deassertion pulse width of at least the number of data bits defined by the <code>UART_CTL.WLS</code> bit field. If cleared, the <code>UART_RTS</code> pin is not generated automatically by hardware. The <code>UART_RTS</code> pin can still be manually controlled by the <code>UART_CTL.MRTS</code> bit, and software is responsible for <code>UART_RTS</code> pulse width control (if needed).
		0 Disable RX handshaking protocol.
		1 Enable RX handshaking protocol.

Table 18-11: UART_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	XOFF	Transmitter off. The <code>UART_CTL.XOFF</code> bit (if set) turns off transmission (XOFF) by preventing the content of <code>THR</code> from being continued to <code>TSR</code> . When set, this bit turns on transmission (XON). The state of the <code>UART_CTL.XOFF</code> bit is ignored if the <code>UART_CTL.ACTS</code> bit is set.
		0 Transmission ON, if <code>ACTS</code> =0
		1 Transmission OFF, if <code>ACTS</code> =0
25 (R/W)	MRTS	Manual Request to Send. The <code>UART_CTL.MRTS</code> bit controls the state of the <code>UART_RTS</code> output pin when the <code>UART_CTL.ARTS</code> bit is cleared. When <code>UART_CTL.MRTS</code> is cleared, the UART deasserts the <code>UART_RTS</code> pin, signaling to the external device that the UART is not ready to receive. When <code>UART_CTL.MRTS</code> is set, the UART asserts the <code>UART_RTS</code> pin, signaling to the external device that the UART is ready to receive.
		0 Deassert RTS pin when <code>ARTS</code> =0
		1 Assert RTS pin when <code>ARTS</code> =0
24 (R/W)	TPOLC	IrDA TX Polarity Change. The <code>UART_CTL.TPOLC</code> bit selects the active low or high polarity for IrDA communications. This bit is effective only in IrDA mode. If set, in IrDA mode, the <code>UART_TX</code> pin idles high. In UART or MDB mode, it is inverted-NRZ. If cleared, in IrDA mode, the <code>UART_TX</code> pin idles low. In UART or MDB mode, it is NRZ.
		0 Active-low TX polarity setting
		1 Active-high TX polarity setting
23 (R/W)	RPOLC	IrDA RX Polarity Change. The <code>UART_CTL.RPOLC</code> bit selects the active low or high polarity for IrDA communications. This bit is effective only in IrDA mode. If set, in IrDA mode, the <code>UART_RX</code> pin idles high. In UART or MDB mode, it is inverted-NRZ. If cleared, in IrDA mode, the <code>UART_RX</code> pin idles low. In UART or MDB mode, it is NRZ.
		0 Active-low RX polarity setting
		1 Active-high RX polarity setting

Table 18-11: UART_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	FCPOL	Flow Control Pin Polarity. The <code>UART_CTL.FCPOL</code> bit selects the polarities of the <code>UART_CTS</code> and <code>UART_RTS</code> pins. When the <code>UART_CTL.FCPOL</code> bit is cleared, the <code>UART_RTS</code> and <code>UART_CTS</code> pins are active low, and the UART is halted when the <code>UART_RTS</code> and <code>UART_CTS</code> pin state is high. When <code>UART_CTL.FCPOL</code> bit is set, the <code>UART_RTS</code> and <code>UART_CTS</code> pins are active high, and the UART is halted when the <code>UART_RTS</code> and <code>UART_CTS</code> pin state is low.
		0 Active low CTS/RTS
		1 Active high CTS/RTS
19 (R/W)	SB	Set Break. If set, the <code>UART_CTL.SB</code> bit forces the <code>UART_TX</code> pin to low asynchronously, regardless of whether or not data is currently transmitted. This bit functions even when the UART clock is disabled. Because the <code>UART_TX</code> pin normally drives high, it can be used as a flag output pin, if the UART is not used. (For example, if <code>UART_CTL.TPOLC</code> is cleared, drive <code>UART_TX</code> pin low; or if <code>UART_CTL.TPOLC</code> is set, drive <code>UART_TX</code> pin high.)
		0 No force
		1 Force TX pin to 0
18 (R/W)	FFE	Force Framing Error on Transmit. The <code>UART_CTL.FFE</code> bit is intended for test purposes. This bit is useful for debugging software, especially in loopback mode.
		0 Normal operation
		1 Force error
17 (R/W)	FPE	Force Parity Error on Transmit. The <code>UART_CTL.FPE</code> bit is intended for test purposes. This bit is useful for debugging software, especially in loopback mode.
		0 Normal operation
		1 Force parity error

Table 18-11: UART_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	STP	Sticky Parity. The <code>UART_CTL.STP</code> bit controls whether the parity is generated by hardware based on the data bits or whether it is set to a fixed value. If this bit is cleared, the hardware calculates the parity bit value based on the data bits. Then, the <code>EPS</code> bit determines whether odd or even parity mode is chosen. If this bit is set, odd parity is used. That means that the total count of logical-1 data bits including the parity bit must be an odd value. Even parity is chosen by <code>UART_CTL.STP</code> cleared and <code>UART_CTL.EPS</code> set. Then, the count of logical-1 bits must be an even value. If the <code>UART_CTL.STP</code> bit is set, hardware parity calculation is disabled. In this case, the sent and received parity equals the inverted <code>UART_CTL.EPS</code> bit.
		0 No forced parity
		1 Force (Stick) parity to defined value (if <code>PEN=1</code>)
15 (R/W)	EPS	Even Parity Select.
		0 Odd parity
		1 Even parity
14 (R/W)	PEN	Parity Enable. The <code>UART_CTL.PEN</code> bit enables parity transmission and parity check. The <code>UART_CTL.PEN</code> bit inserts one additional bit between the most significant data bit and the first stop bit. The polarity of this so-called parity bit depends on data and the <code>UART_CTL.STP</code> and <code>UART_CTL.EPS</code> control bits. Both transmitter and receiver calculate the parity value. The receiver compares the received parity bit with the expected value and issues a parity error if they do not match. If the <code>UART_CTL.PEN</code> bit is cleared, the <code>UART_CTL.STP</code> and the <code>UART_CTL.EPS</code> bits are ignored.
		0 Disable
		1 Enable parity transmit and check
13 (R/W)	STBH	Stop Bits (Half Bit Time).
		0 0 half-bit-time stop bit
		1 1 half-bit-time stop bit
12 (R/W)	STB	Stop Bits. The <code>UART_CTL.STB</code> bit controls how many stop bits are appended to transmitted data.
		0 1 stop bit
		1 2 stop bits

Table 18-11: UART_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9:8 (R/W)	WLS	Word Length Select. The <code>UART_CTL.WLS</code> field determines whether the transmitted and received UART word consists of 5, 6, 7, or 8 data bits.
		0 5-bit word
		1 6-bit word
		2 7-bit word
		3 8-bit word
5:4 (R/W)	MOD	Mode of Operation. The <code>UART_CTL.MOD</code> selects the UART operation mode (UMOD).
		0 UART mode
		1 MDB mode
		2 IrDA SIR mode
1 (R/W)	LOOP_EN	Loopback Enable. The <code>UART_CTL.LOOP_EN</code> bit enables UART loopback mode. When set, this bit disconnects the input of the receiver from the <code>UART_RX</code> pin, and internally redirects the transmit output to the receiver. The <code>UART_TX</code> pin remains active and continues to transmit data externally as well. Loopback mode also forces the <code>UART_RTS</code> pin to its deassertive state, disconnects the <code>UART_CTS</code> bit from the <code>UART_CTS</code> input pin, and directly connects the <code>UART_CTL.MRTS</code> bit to the <code>UART_STAT.CTS</code> bit. In loopback mode, setting the <code>UART_CTL.MRTS</code> bit sets the <code>UART_STAT.CTS</code> bit and enables the transmitter of the UART. Clearing the <code>UART_CTL.MRTS</code> bit clears the <code>UART_STAT.CTS</code> bit and disables the transmitter of the UART.
		0 Disable
		1 Enable
0 (R/W)	EN	Enable UART. The <code>UART_CTL.EN</code> enables the UART clocks. This bit also resets the state machine and control registers when cleared. Using this bit to disable the UART -- when not used -- reduces power consumption.
		0 Disable
		1 Enable

Interrupt Mask Register

The `UART_IMSK` register indicates the interrupt mask status (unmasked, if set, or masked, if cleared) of the UART status interrupt requests. This register is not a data register. Instead, it is controlled by the `UART_IMSK_SET` and `UART_IMSK_CLR` register pair. Writing ones to the `UART_IMSK_SET` register enables (unmasks) interrupt requests, and writing ones to the `UART_IMSK_CLR` register disables (masks) them. Reads from either register return the enabled bits.

The `UART_IMSK` register is used to enable requests for system handling of empty or full states of UART data registers. Unless polling is used as a means of action, the `UART_IMSK.ERBFI` and `UART_IMSK.ETBEI` bits are normally set. Setting this register without enabling system DMA causes the UART to notify the processor of the data inventory state using interrupts. For proper operation in this mode, system interrupts must be enabled, and appropriate interrupt handling routines must be present.

Each UART features three separate interrupt channels to handle the data transmit, data receive, and line status events independently, regardless of whether DMA is enabled or not. If no DMA channels are assigned to the UART, set the `UART_IMSK.ELSI` bit to reroute the transmit and receive interrupts to the status interrupt request output.

With system DMA enabled, the UART uses DMA to transfer data to or from the processor. Dedicated DMA channels are available for receive and transmit operations. Line error handling can be configured independently from the receive or transmit setup.

The DMA of the UART is enabled by first setting up the system DMA control registers and then enabling the `UART_IMSK.ERBFI` and `UART_IMSK.ETBEI` interrupts. This configuration is because the interrupt request lines double as DMA request lines. Depending on whether DMA is enabled or not, upon receiving these requests, the DMA control unit either generates a direct memory access or passes the UART interrupt on to the system interrupt handling unit(s). However, the error interrupt for the UART goes directly to the system interrupt handling unit(s), bypassing the DMA unit completely.

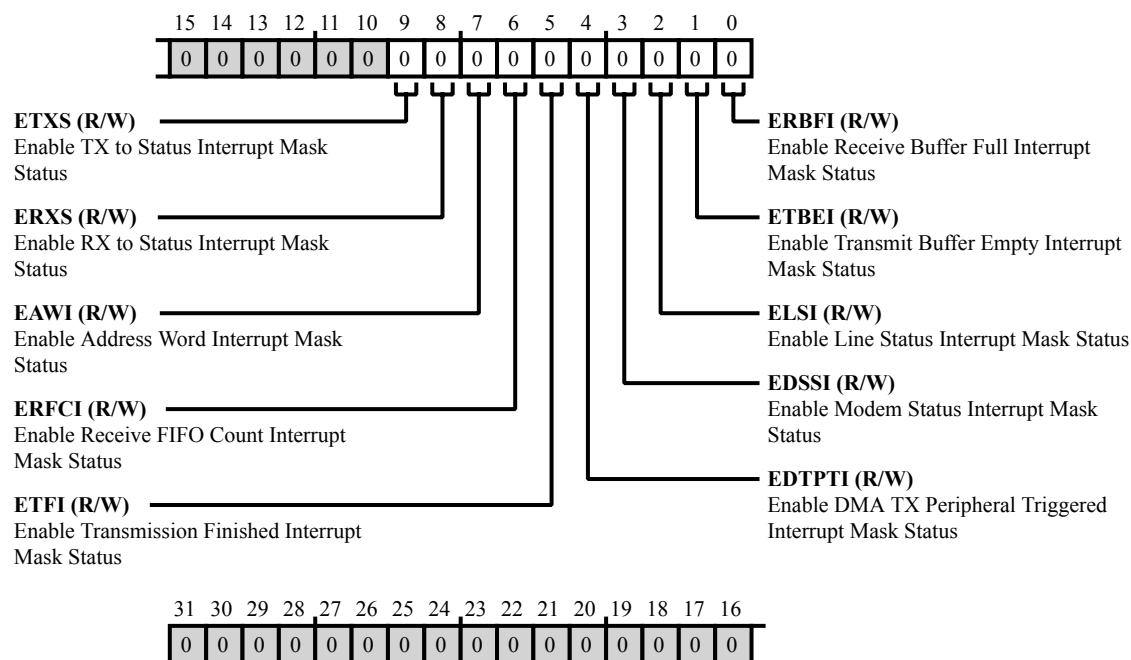


Figure 18-13: UART_IMSK Register Diagram

Table 18-12: UART_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	ETXS	Enable TX to Status Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ETXS</code> bit indicates re-direction of the TX interrupt requests to status interrupt output. If cleared, TX interrupt requests are routed to normal interrupt outputs.
		0 Interrupt is masked
		1 Interrupt is unmasked
8 (R/W)	ERXS	Enable RX to Status Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ERXS</code> bit indicates re-direction of RX interrupt requests to status interrupt output. If cleared, RX interrupt requests are routed to normal interrupt outputs.
		0 Interrupt is masked
		1 Interrupt is unmasked

Table 18-12: UART_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	EAWI	Enable Address Word Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.EAWI</code> bit indicates generation of a status interrupt request when an Address word in MDB-mode is present in the <code>UART_RBR</code> . A received word is an address word if the <code>UART_STAT.ADDR</code> bit is set.
		0 Interrupt is masked
		1 Interrupt is unmasked
6 (R/W)	ERFCI	Enable Receive FIFO Count Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ERFCI</code> bit indicates enabling of the receive buffer threshold interrupt request if signaled by the <code>UART_STAT.RFCS</code> bit. Read the <code>UART_RBR</code> register sufficient times to clear the interrupt request.
		0 Interrupt is masked
		1 Interrupt is unmasked
5 (R/W)	ETFI	Enable Transmission Finished Interrupt Mask Status. If set (interrupt unmasked) the <code>UART_IMSK.ETFI</code> bit indicates enabling of interrupt generation on the status interrupt channel when the transmit buffer register, the transmit address register, and the transmit shift register are all empty as indicated by the <code>UART_STAT.TFI</code> . The <code>UART_IMSK.ETFI</code> interrupt can be used to avoid expensive polling of the <code>UART_STAT.TEMT</code> bit, when the UART clock or line drivers should be disabled after transmission has completed. With the <code>UART_STAT.TFI</code> bit to clear the interrupt request. In DMA operation, the <code>UART_IMSK.ETFI</code> bits functionality might be preferred.
		0 Interrupt is masked
		1 Interrupt is unmasked
4 (R/W)	EDTPTI	Enable DMA TX Peripheral Triggered Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.EDTPTI</code> bit indicates enabling of the DMA completion interrupt request to be delayed until the data has left the UART completely. This bit is required for DMA transmit operation only. If set, the UART can generate a DMA interrupt request by the time the <code>UART_STAT.TEMT</code> bit goes high after the last DMA data word is transmitted. When <code>UART_IMSK.EDTPTI</code> is set, usually the <code>DMA_CFG.INT</code> field is cleared to 00 in a STOP mode DMA. This set up suppresses the normal completion interrupt request, and the <code>UART_STAT.TEMT</code> event is signaled through the DMA controller and triggers the DMA interrupt. If both (<code>DMA_CFG.INT</code> not 00 and <code>UART_IMSK.EDTPTI</code> set), two interrupts are requested at the end of a STOP mode DMA.
		0 Interrupt is masked
		1 Interrupt is unmasked

Table 18-12: UART_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	EDSSI	Enable Modem Status Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.EDSSI</code> bit indicates enabling of a modem status interrupt request on the same status interrupt channel when the <code>UART_STAT.SCTS</code> bit is set. This indicates <code>UART_CTS</code> pin re-assertion. Write-1-to-clear (W1C) the <code>UART_STAT.SCTS</code> bit to clear the interrupt request.
		0 Interrupt is masked
		1 Interrupt is unmasked
2 (R/W)	ELSI	Enable Line Status Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ELSI</code> bit indicates that redirection of TX and RX interrupt requests to the status interrupt output of the UART by OR'ing them with the <code>UART_STAT.OE</code> , <code>UART_STAT.PE</code> , <code>UART_STAT.FE</code> , and <code>UART_STAT.BI</code> interrupt requests. Set this bit when no DMA channel is associated with the UART. Enabling <code>UART_IMSK.ELSI</code> disables the RX/TX interrupt channels and negates the <code>UART_IMSK.EDTPTI</code> bit.
		0 Interrupt is masked
		1 Interrupt is unmasked
1 (R/W)	ETBEI	Enable Transmit Buffer Empty Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ETBEI</code> bit indicates generation of a TX interrupt request if the <code>UART_STAT.THRE</code> bit is set.
		0 Interrupt is masked
		1 Interrupt is unmasked
0 (R/W)	ERBFI	Enable Receive Buffer Full Interrupt Mask Status. If set (interrupt unmasked), the <code>UART_IMSK.ERBFI</code> indicates generation of an RX interrupt request if the <code>UART_STAT.DR</code> bit is set.
		0 Interrupt is masked
		1 Interrupt is unmasked

Interrupt Mask Clear Register

The `UART_IMSK` indicates interrupt mask status (unmasked if set, masked if cleared) of UART status interrupts. This register is not a data register. Instead it is controlled by the `UART_IMSK_SET` and `UART_IMSK_CLR` register pair. Writing ones to `UART_IMSK_SET` enables (unmasks) interrupt requests, and writing ones to `UART_IMSK_CLR` disables (masks) them. Reads from either register return the enabled bits. For more information, see the `UART_IMSK` register description.

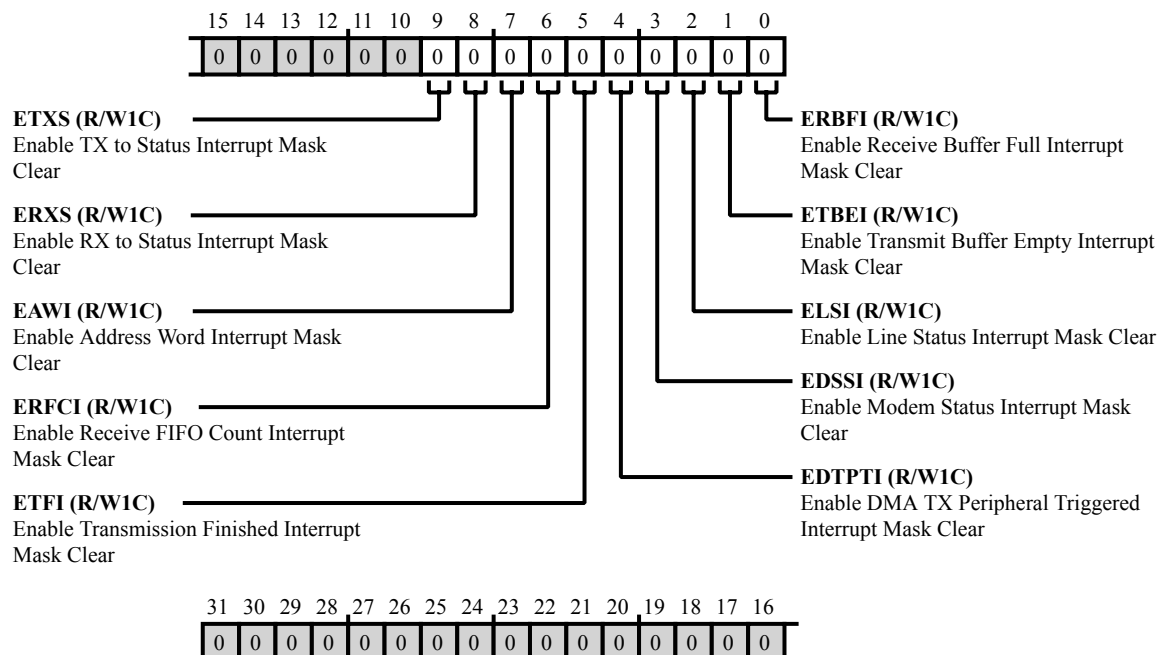


Figure 18-14: UART_IMSK_CLR Register Diagram

Table 18-13: UART_IMSK_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W1C)	ETXS	Enable TX to Status Interrupt Mask Clear.
		0 No action
		1 Mask interrupt
8 (R/W1C)	ERXS	Enable RX to Status Interrupt Mask Clear.
		0 No action
		1 Mask interrupt
7 (R/W1C)	EAWI	Enable Address Word Interrupt Mask Clear.
		0 No action
		1 Mask interrupt

Table 18-13: UART_IMSK_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
6 (R/W1C)	ERFCI	Enable Receive FIFO Count Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
5 (R/W1C)	ETFI	Enable Transmission Finished Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
4 (R/W1C)	EDTPTI	Enable DMA TX Peripheral Triggered Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
3 (R/W1C)	EDSSI	Enable Modem Status Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
2 (R/W1C)	ELSI	Enable Line Status Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
1 (R/W1C)	ETBEI	Enable Transmit Buffer Empty Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt
0 (R/W1C)	ERBFI	Enable Receive Buffer Full Interrupt Mask Clear.	
		0	No action
		1	Mask interrupt

Interrupt Mask Set Register

The `UART_IMSK` indicates interrupt request mask status (unmasked if set, masked if cleared) of UART status interrupts. This register is not a data register. Instead it is controlled by the `UART_IMSK_SET` and `UART_IMSK_CLR` register pair. Writing ones to `UART_IMSK_SET` enables (unmasks) interrupt requests, and writing ones to `UART_IMSK_CLR` disables (masks) them. Reads from either register return the enabled bits. For more information, see the `UART_IMSK` register description.

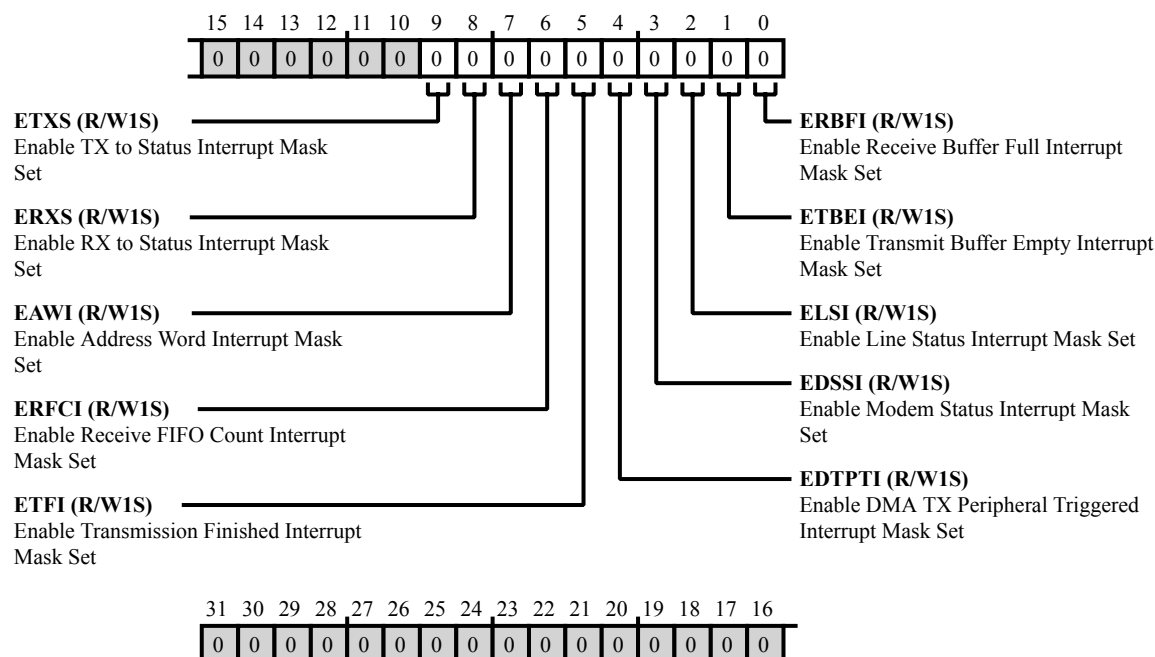


Figure 18-15: UART_IMSK_SET Register Diagram

Table 18-14: UART_IMSK_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W1S)	ETXS	Enable TX to Status Interrupt Mask Set.
		0 No action
		1 Unmask interrupt
8 (R/W1S)	ERXS	Enable RX to Status Interrupt Mask Set.
		0 No action
		1 Unmask interrupt
7 (R/W1S)	EAWI	Enable Address Word Interrupt Mask Set.
		0 No action
		1 Unmask interrupt

Table 18-14: UART_IMSK_SET Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
6 (R/W1S)	ERFCI	Enable Receive FIFO Count Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
5 (R/W1S)	ETFI	Enable Transmission Finished Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
4 (R/W1S)	EDTPTI	Enable DMA TX Peripheral Triggered Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
3 (R/W1S)	EDSSI	Enable Modem Status Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
2 (R/W1S)	ELSI	Enable Line Status Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
1 (R/W1S)	ETBEI	Enable Transmit Buffer Empty Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt
0 (R/W1S)	ERBFI	Enable Receive Buffer Full Interrupt Mask Set.	
		0	No action
		1	Unmask interrupt

Receive Buffer Register

The read-only `UART_RBR` register is the UART’s receive buffer. It is updated when there is pending data in the receive FIFO. Newly available data is signaled by the `UART_STAT.DR` bit.

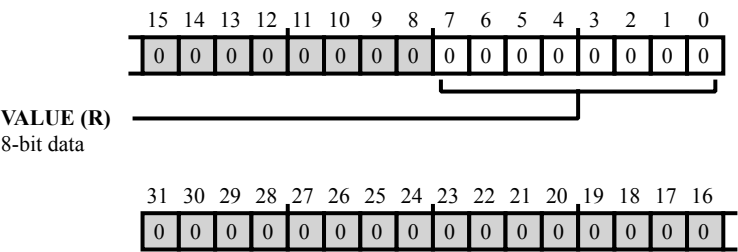


Figure 18-16: UART_RBR Register Diagram

Table 18-15: UART_RBR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	VALUE	8-bit data.

Receive Shift Register

The read only `UART_RSR` register which returns the content of the UART’s receive shift register.

The frame data is moved into this shift register after polarity inversion, if any (including the native polarity inversion in the IrDA case).

In the case of the longest frame (MDB, with parity mode, and 8 bit data word-length), the start bit may be shifted out and not available for reading at the end of the frame reception. This register is NOT reset at the start of frame. If read, in the middle of a frame reception, data corresponding the previous frame may not have entirely shifted out (for example, the read data that have been read may NOT correspond entirely to the frame being received).

Because the UART is receiving only 1 stop bit, the `UART_RSR` contains only 1 stop bit even if more than one stop bit is present in the actual transfer. This register may be considered as storing the 10 most recently received bits (taking into consideration the stop bit receive limitation above).

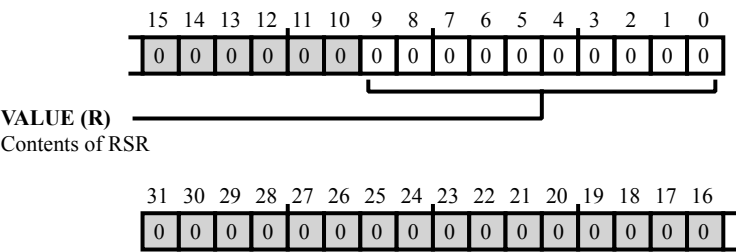


Figure 18-17: UART_RSR Register Diagram

Table 18-16: UART_RSR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/NW)	VALUE	Contents of RSR.

Receive Counter Register

The `UART_RXCNT` register returns the content of 16-bit counter in the UART receiver. This count is used for baud rate clock generation (the lower [15:0] is the count data).

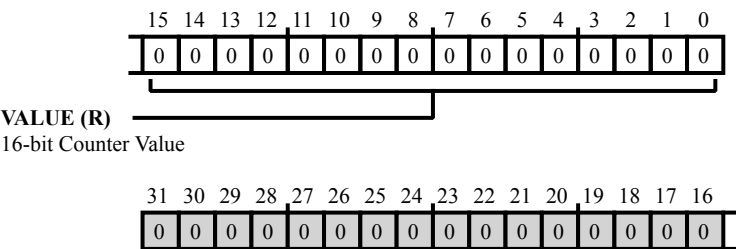


Figure 18-18: UART_RXCNT Register Diagram

Table 18-17: UART_RXCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	16-bit Counter Value.

Scratch Register

The `UART_SCR` registers contain 8-bit scratch pad data. These registers are used for general purpose data storage and do not control the UART hardware in any way.

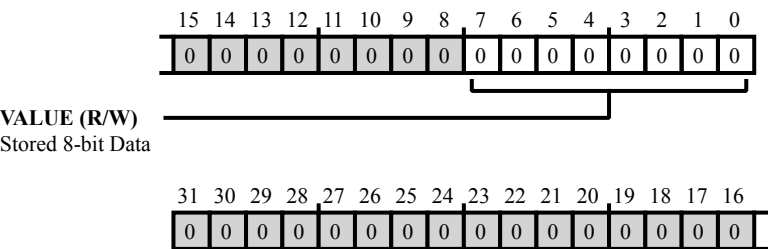


Figure 18-19: UART_SCR Register Diagram

Table 18-18: UART_SCR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	Stored 8-bit Data.

Status Register

The `UART_STAT` register contains the UART line status and UART modem status, as indicated by the current states of the UART's `UART_CTS` pin and internal receive buffers. Writes to this register can perform write-one-to-clear (W1C) operations on most status bits. Reading this register has no side effects.

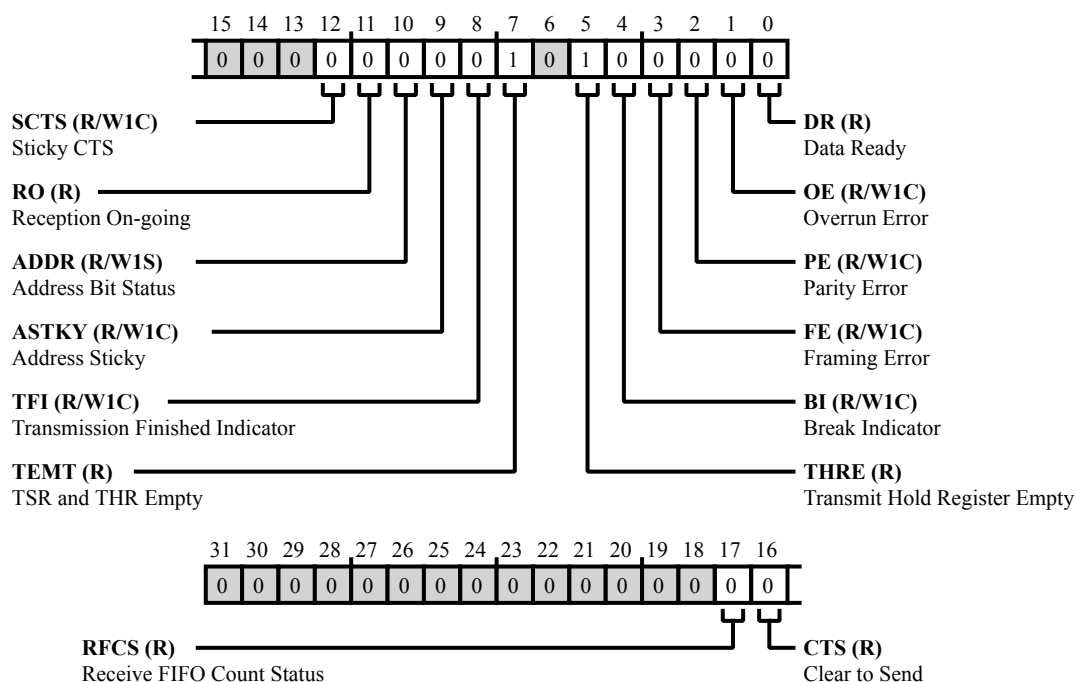


Figure 18-20: UART_STAT Register Diagram

Table 18-19: UART_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/NW)	RFCS	Receive FIFO Count Status.
		The <code>UART_STAT.RFCS</code> bit is set when the receive buffer holds more or equal entries than a certain threshold. The threshold is controlled by the <code>UART_CTL.RFIT</code> bit. If <code>UART_CTL.RFIT</code> is cleared, the threshold is four entries. If <code>UART_CTL.RFIT</code> is set, the threshold is seven entries. The <code>UART_STAT.RFCS</code> bit is cleared when the <code>UART_RBR</code> register is read sufficient times until the buffer is drained below the threshold. The <code>UART_STAT.RFCS</code> bit can trigger a status interrupt request if enabled by the <code>UART_IMSK_SET.ERFCI</code> bit.
		0 RX FIFO has less than 4 (7) entries when <code>RFIT=0</code> (1)
		1 RX FIFO has at least 4 (7) entries when <code>RFIT=0</code> (1)

Table 18-19: UART_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/NW)	CTS	Clear to Send. The <code>UART_STAT.CTS</code> bit holds the value (if <code>UART_CTL.FCPOL</code> set) or the complement value (if <code>UART_CTL.FCPOL</code> cleared) of the <code>UART_CTS</code> input pin. The <code>UART_CTL.ACTS</code> bit must be set to enable this feature. The core can read the value of the <code>UART_STAT.CTS</code> bit to determine whether the external device is ready to receive (<code>UART_STAT.CTS</code> set) or if it is busy (<code>UART_STAT.CTS</code> cleared). If <code>UART_CTL.ACTS</code> is cleared, the <code>UART_TX</code> handshaking protocol is disabled, and the UART transmits data as long as there is data to transmit, regardless of the value of <code>UART_STAT.CTS</code> . When <code>UART_CTL.ACTS</code> is cleared, the software can pause transmission temporarily by setting the <code>XOFF</code> bit. Note that in loopback mode (<code>UART_CTL.LOOP_EN</code> set), the <code>UART_STAT.CTS</code> bit is disconnected from the <code>UART_CTS</code> input pin. Instead, the bit is directly connected to the <code>UART_CTL.MRTS</code> bit.
		0 Not clear to send (External device not ready to receive)
		1 Clear to send (External device ready to receive)
12 (R/W1C)	SCTS	Sticky CTS. The <code>UART_STAT.SCTS</code> bit is a sticky bit that is set when <code>UART_STAT.CTS</code> transitions from 0 to 1. The <code>UART_STAT.SCTS</code> bit is cleared by software with a W1C operation. This bit can trigger a line status interrupt request if enabled by the <code>UART_IMSK_SET.EDSSI</code> bit.
		0 CTS has not transitioned from low to high
		1 CTS has transitioned from low to high
11 (R/NW)	RO	Reception On-going.
		0 No data reception in progress
		1 Data reception in progress
10 (R/W1S)	ADDR	Address Bit Status. The <code>UART_STAT.ADDR</code> bit is used to mirror the address bit of the word in <code>UART_RBR</code> in multi-drop bus protocol, and is enabled only in MDB mode. The <code>UART_STAT.ADDR</code> bit is updated by hardware upon detecting a received word with the address bit in <code>UART_RBR</code> set or cleared. Additionally, software can set the <code>ADDR</code> bit with a write-1-to-set (W1S) operation.
		0 Address bit is low
		1 Address bit is high

Table 18-19: UART_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W1C)	ASTKY	Address Sticky. The <code>UART_STAT.ASTKY</code> bit is used in multi-drop bus mode to indicate whether a peripheral is currently being addressed. This bit is a sticky version of the <code>UART_STAT.ADDR</code> bit and is set by hardware when setting the <code>UART_STAT.ADDR</code> bit. The <code>UART_STAT.ASTKY</code> bit can only be cleared by software with a write-one-to-clear (W1C) operation. With the <code>UART_STAT.ASTKY</code> bit set, words will be received irrespective of the <code>UART_CTL.MOD</code> bit or <code>UART_STAT.ADDR</code> bit selection. With the <code>UART_STAT.ASTKY</code> bit cleared, only address words (<code>UART_CTL.MOD</code> bit set) will be received and words with <code>UART_CTL.MOD</code> bit cleared are ignored (not moved from the RSR to the RX FIFO) in MDB mode. The <code>UART_STAT.ASTKY</code> bit does not affect reception in non-MDB modes.
		0 ADDR bit has not been set
		1 ADDR bit has been set
8 (R/W1C)	TFI	Transmission Finished Indicator. The <code>UART_STAT.TFI</code> bit is a sticky version of the <code>UART_STAT.TEMT</code> bit. While <code>UART_STAT.TEMT</code> is automatically cleared by hardware when new data is written to the <code>UART_THR</code> register, the sticky <code>UART_STAT.TFI</code> bit remains set, until it is cleared by software (W1C). The <code>UART_STAT.TFI</code> bit enables more flexible transmit interrupt request timing.
		0 TEMT did not transition from 0 to 1
		1 TEMT transition from 0 to 1
7 (R/NW)	TEMT	TSR and THR Empty. The <code>UART_STAT.TEMT</code> bit indicates that the <code>UART_THR</code> and <code>UART_TAIP</code> registers and the <code>UART_TSR</code> register are empty. In this case, the program is permitted to write to the <code>UART_THR</code> and <code>UART_TAIP</code> registers twice without losing data. The <code>UART_STAT.TEMT</code> bit can also be used as indicator that pending UART transmission is completed. At that time, it is safe to disable the <code>UART_CTL.EN</code> bit or to three-state the off-chip line driver.
		0 Not empty TSR/THR
		1 TSR/THR Empty

Table 18-19: UART_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/NW)	THRE	Transmit Hold Register Empty. The <code>UART_STAT.THRE</code> bit indicates that the UART transmit channel is ready for new data and software can write to the <code>UART_THR</code> and <code>UART_TAIP</code> registers. Writes to the <code>UART_THR</code> and <code>UART_TAIP</code> registers clear the <code>UART_STAT.THRE</code> . The bit is set again when the <code>UART_THR</code> and <code>UART_TAIP</code> registers are empty and ready to accept data.
		0 Not empty THR/TAIP
		1 Empty THR/TAIP
4 (R/W1C)	BI	Break Indicator. The <code>UART_STAT.BI</code> bit indicates that the first stop bit is sampled low and the entire <u>data word</u> , including parity bit, consists of low bits only. (This condition indicates that <code>UART_RX</code> was held low for more than the maximum word length.) The <code>UART_STAT.BI</code> bit is updated simultaneously with the <code>UART_STAT.DR</code> bit, that is, by the time the first stop bit is received or when data is loaded from the receive FIFO to the <code>UART_RBR</code> register. The bit is sticky and can be cleared by W1C operations.
		0 No break interrupt
		1 Break interrupt this indicates UARTxRX was held low(RPOLC=0) / high (RPOLC=1) for more than the maximum word length
3 (R/W1C)	FE	Framing Error. The <code>UART_STAT.FE</code> bit indicates that the first stop bit is sampled. This bit is updated simultaneously with the <code>UART_STAT.DR</code> bit, that is, by the time the first stop bit is received or when data is loaded from the receive FIFO to the <code>UART_RBR</code> register. The <code>UART_STAT.FE</code> bit is sticky and can be cleared by W1C operations. Note that invalid stop bits can be simulated by setting the <code>UART_CTL.FFE</code> bit.
		0 No error
		1 Invalid stop bit error
2 (R/W1C)	PE	Parity Error. The <code>UART_STAT.PE</code> bit indicates that the received parity bit does not match the expected value. This bit is updated simultaneously with the <code>UART_STAT.DR</code> bit, that is, by the time the first stop bit is received or when data is loaded from the receive FIFO to the <code>UART_RBR</code> register. The <code>UART_STAT.PE</code> bit is sticky and can be cleared by W1C operations. Note that invalid parity bits can be simulated by setting the <code>UART_CTL.FPE</code> bit.
		0 No parity error
		1 Parity error

Table 18-19: UART_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W1C)	OE	Overrun Error. The <code>UART_STAT.OE</code> bit indicates that further data is received while the internal receive buffer was full. This bit is set when sampling the stop bit of the sixth data word. To avoid overruns, read the <code>UART_RBR</code> register in time. In DMA receive mode, overruns are very unlikely to happen ever. After an overrun occurs, the <code>UART_RBR</code> and receive FIFO are protected from being overwritten by new data until the <code>UART_STAT.OE</code> bit is cleared by software. The content of the <code>UART_RSR</code> register is lost as soon as the overrun occurs. The <code>UART_STAT.OE</code> bit is sticky and can be cleared by W1C operations.
		0 No overrun
		1 Overrun error
0 (R/NW)	DR	Data Ready. The <code>UART_STAT.DR</code> bit indicates that data is available in the receiver and can be read from the <code>UART_RBR</code> register. The bit is set by hardware when the receiver detects the first valid stop bit. The bit is cleared by hardware when the <code>UART_RBR</code> register is read.
		0 No new data
		1 New data in RBR

Transmit Address/Insert Pulse Register

The `UART_TAIP` register and the `UART_THR` register share the same physical register, but `UART_TAIP` has different effect than the `UART_THR` register when `UART_TAIP` is written to in MDB and UART modes.

In MDB mode, data written to the `UART_TAIP` register is transmitted as an address frame (as with the `UART_CTL.MOD` bit set).

In UART mode, a write to `UART_TAIP` causes a pulse of value `UART_TAIP` [7] for a duration of `UART_TAIP` [6:0] x bit time. (There is additional inversion if the `UART_CTL.TPOLC` bit is set).

Bit time is defined by the `UART_CLK` register. The transmission of the pulse is followed by stop bit transmission as specified by the `UART_CTL.STB` and `UART_CTL.STBH` bits. This could be used for supporting line break command and inter-frame gap.

In IrDA mode, writes to `UART_TAIP` is treated the same as writes to `UART_THR`.

Accesses to the `UART_TAIP` register have the same affects as the `UART_THR` register with respect to the `UART_STAT.THRE`, `UART_STAT.TEMT`, and `UART_STAT.TFI` flags.

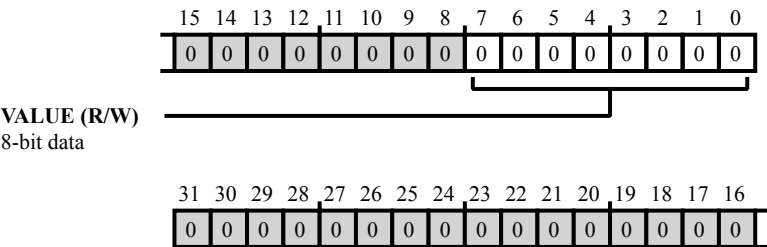


Figure 18-21: UART_TAIP Register Diagram

Table 18-20: UART_TAIP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	8-bit data.

Transmit Hold Register

The write-only `UART_THR` register is the UART’s transmit buffer. The `UART_STAT.THRE` bit indicates whether data can be written to `UART_THR`. Writes to this register automatically propagate to the internal `UART_TSR` register as soon as `UART_TSR` is ready. Then, transmit operation is initiated immediately.

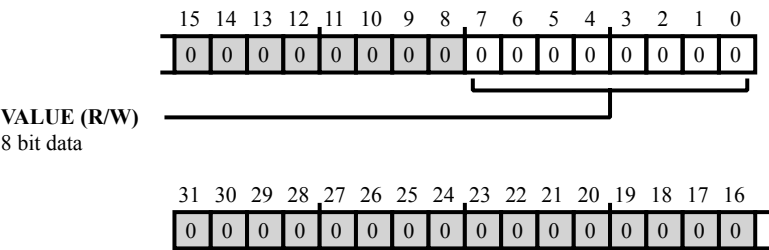


Figure 18-22: UART_THR Register Diagram

Table 18-21: UART_THR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	8 bit data.

Transmit Shift Register

The read only `UART_TSR` register which returns the content of the UART’s transmit shift register.

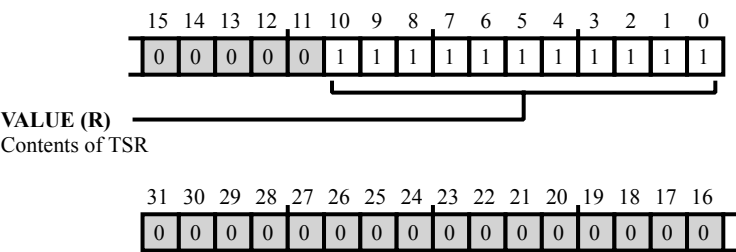


Figure 18-23: UART_TSR Register Diagram

Table 18-22: UART_TSR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/NW)	VALUE	Contents of TSR.

Transmit Counter Register

The `UART_TXCNT` read only register returns the content of 16-bit counter in the UART transmitter. This count is used for baud rate clock generation (the lower [15:0] is the count data).

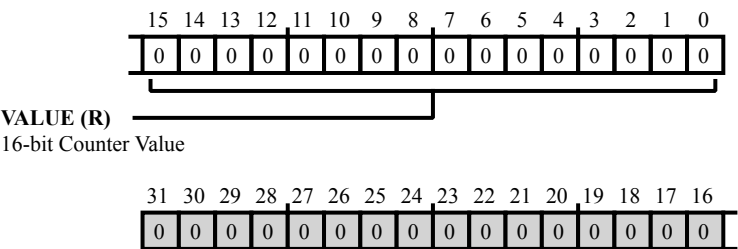


Figure 18-24: UART_TXCNT Register Diagram

Table 18-23: UART_TXCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	16-bit Counter Value.

19 Enhanced Parallel Peripheral Interface (EPPI)

The Enhanced Parallel Peripheral Interface (EPPI) is a half-duplex, bidirectional port with a dedicated clock pin and three frame sync (FS) pins. It can support direct connections to active TFT LCDs, parallel A/D and D/A converters, video encoders and decoders, image sensor modules and other general-purpose peripherals. Each EPPI has two DMA channels associated with it. Moreover, in some modes, an EPPI can use an extra DMA channel.

EPPI Features

The EPPI module supports the following features.

- Programmable data length from 8 bits to up to 24 bits per clock cycle (depending on the product model)
- Bidirectional and half-duplex port
- Internal or external clock source
- Clock gating by an external device asserting the clock gating control signal
- Various framed and non-framed operating modes, as well as internal or external frame syncs
- Various general-purpose modes with 0, 1, 2, and 3 frame sync modes for both receive and transmit
- Ignores premature external frame syncs for data consistency
- SMPTE274M and SMPTE 296M high definition format support
- ITU-656, SMPTE 296M and SMPTE 274M status word error detection and correction for ITU-656 receive modes
- ITU-656, SMPTE 296M and SMPTE 274M receive modes – active video only, vertical blanking only, and entire field
- ITU-656, SMPTE 296M and SMPTE 274M preamble and status word decode
- Optional packing and unpacking of data to or from 32 bits from or to 8, 16 bits and 24 bits. If packing or unpacking is enabled, endianness can be altered to change the order of packing or unpacking of bytes/words
- Optional sign extension or zero-fill and alternate even or odd data sample filter for receive modes

- RGB888 to RGB666 or RGB565 conversion for transmit modes
- 4:2:2 YCrCb data Tx/Rx interleaving or de-interleaving modes
- Configurable LCD data enable (DEN) output available on frame sync 3
- Delayed start of PPI frame syncs
- Data clipping and mirroring
- Horizontal and vertical windowing for general purpose 2 and 3 frame sync modes
- Preamble, blanking and stripping support
- Multiplexing dual input

EPPI Functional Description

The EPPI has the following functionality.

RGB data formats

For transmit modes, the EPPI can convert RGB888 data in memory to either RGB565 or RGB666 at the output using bits in the Control register.

Data clipping

The EPPI contains two registers to define the lower and upper limits for the Luma and Chroma components. This functionality is used for clipping data values during 8-bit, 10-bit, 12-bit or 16-bit transmit modes.

Data mirroring

A data mirroring feature is available which mirrors the EPPI data bits 15–0. This functionality is available in both transmit and receive modes.

Windowing

The EPPI supports windowing for general-purpose input modes.

Preamble, blanking and stripping support

The EPPI can embed blanking information and clip active data to be transmitted. This functionality is available for single channel data, interleaved data, and parallel data and supports data lengths equal to 16 bits, 20 bits or 24 bits.

ADSP-2159x EPPI Register List

The EPPI is a half-duplex, bidirectional parallel port. It comprises a clock pin, 3 frame sync pins, and a set of data pins. For more information on EPPI functionality, see the EPPI register descriptions.

Table 19-1: ADSP-2159x EPPI Register List

Name	Description
EPPI_CLKDIV	Clock Divide Register
EPPI_CTL	Control Register
EPPI_CTL2	Control Register 2 Register
EPPI_EVENCLIP	Clipping Register for EVEN (Luma) Data Register
EPPI_FRAME	Lines Per Frame Register
EPPI_FS1_DLY	Frame Sync 1 Delay Value Register
EPPI_FS1_PASPL	FS1 Period Register / EPPI Active Samples Per Line Register
EPPI_FS1_WLHB	FS1 Width Register / EPPI Horizontal Blanking Samples Per Line Register
EPPI_FS2_DLY	Frame Sync 2 Delay Value Register
EPPI_FS2_PALPF	FS2 Period Register / EPPI Active Lines Per Field Register
EPPI_FS2_WLVB	FS2 Width Register / EPPI Lines Of Vertical Blanking Register
EPPI_HCNT	Horizontal Transfer Count Register
EPPI_HDLY	Horizontal Delay Count Register
EPPI_IMSK	Interrupt Mask Register
EPPI_LINE	Samples Per Line Register
EPPI_ODDCLIP	Clipping Register for ODD (Chroma) Data Register
EPPI_STAT	Status Register
EPPI_VCNT	Vertical Transfer Count Register
EPPI_VDLY	Vertical Delay Count Register

ADSP-2159x EPPI Interrupt List

Table 19-2: ADSP-2159x EPPI Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
218	EPPI0_CH0_DMA	EPPI0 Channel 0 DMA	Level	28
219	EPPI0_CH1_DMA	EPPI0 Channel 1 DMA	Level	29
220	EPPI0_STAT	EPPI0 Status	Level	
279	EPPI0_CH0_DMA_ERR	EPPI0 DMA Channel 0 Error		

Table 19-2: ADSP-2159x EPPI Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
280	EPPI0_CH1_DMA_ERR	EPPI0 DMA Channel 1 Error		

ADSP-2159x EPPI Trigger List

Table 19-3: ADSP-2159x EPPI Trigger List Masters

Trigger ID	Name	Description	Sensitivity
34	EPPI0_CH0_DMA	EPPI0 Channel 0 DMA	Edge
35	EPPI0_CH1_DMA	EPPI0 Channel 1 DMA	Edge

Table 19-4: ADSP-2159x EPPI Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
19	EPPI0_CH0_DMA	EPPI0 Channel 0 DMA	Pulse
20	EPPI0_CH1_DMA	EPPI0 Channel 1 DMA	Pulse

RGB Data Formats

For transmit modes, the EPPI can convert RGB888 data in memory to RGB666 at the output when the `EPPI_CTL.RGBFMTEN` bit is set and the `EPPI_CTL.DLEN` value is equal to 18 bits. Similarly, the EPPI can convert RGB888 data in memory to RGB565 at the output when the `EPPI_CTL.RGBFMTEN` bit is set and `EPPI_CTL.DLEN` is equal to 16 bits.

This conversion is performed as follows:

- If `EPPI_CTL.PACKEN = 1`, the EPPI first unpacks according to the `EPPI_CTL.SWAPEN` bit setting, and the three 32-bit data words from the DMA are broken into four 24-bit data words to be transmitted out, as described earlier.
- If `EPPI_CTL.PACKEN = 0`, the EPPI takes the lower 24 bits of the 32-bit DMA as the data to be transmitted. Then, the EPPI truncates this 24-bit data word to the required data width. It removes the lower 2 bits of G and the lower 2 bits or 3 bits of R and B.

Data Clipping

The EPPI contains two registers to define the lower and upper limits for the Luma and Chroma components. It uses these registers for clipping data values during 8-bit, 10-bit, 12-bit or 16-bit transmit modes. All data values for odd samples which are less than the value in the `EPPI_ODDCLIP.LOWODD` bit field are replaced with the value in the `EPPI_ODDCLIP.LOWODD` field. All data values for even samples which are less than the value in the `EPPI_EVENCLIP.LOWEVEN` field are replaced with the value in the `EPPI_EVENCLIP.LOWEVEN` field.

In the same manner, all data values for odd samples which are more than the value in the `EPPI_ODDCLIP.HIGHODD` bit field are replaced with the value in the `EPPI_ODDCLIP.HIGHODD` field. All data values for even samples which are more than the values in the `EPPI_EVENCLIP.HIGHEVEN` field are replaced with the values in the `EPPI_EVENCLIP.HIGHEVEN` field.

Depending on the programmed EPPI length, only the corresponding bits (least aligned) are considered for clipping. For example, if the EPPI is programmed in 10-bit mode, bits 9:0 and bits 25:16 constitute the clipping thresholds. The higher bits are ignored. The EPPI supports 8-bit, 10-bit, 12-bit, and 16-bit clipping thresholds.

For the 4:2:2 YCrCb color space, Luma and Chroma typically have different lower and upper thresholds. Separate thresholds can be required for even and odd data samples. For monochrome (Y only) or some non-video clipping applications, the value in the `EPPI_ODDCLIP.LOWODD` field can be the same as the value in the `EPPI_EVENCLIP.LOWEVEN` field. The value in the `EPPI_ODDCLIP.HIGHODD` field can be the same as the value in the `EPPI_EVENCLIP.HIGHEVEN` field.

In GP 0 FS mode with internal blanking generation, clipping is valid only for the active video part of the transmitted data. ITU-R 656 preambles, status words, and blanking data bypass the clipping logic.

If the EPPI is programmed in 16, 20-bit or 24-bit mode with the `EPPI_CTL.SPLTWRD` bit set, the YDATA (luma data) gets the clipping threshold levels of the `EPPI_EVENCLIP` register. The CDATA (chroma data) gets the clipping threshold levels of the `EPPI_ODDCLIP` register.

The clipping registers are ignored when the `EPPI_CTL.RGBFMTEN` bit is set.

Data Mirroring

To increase the pin multiplexing options for the EPPI data pins, a data mirroring feature is available which mirrors the EPPI data bits 15:0. This feature is available in both transmit and receive modes. It is enabled by setting the `EPPI_CTL.DMIRR` bit.

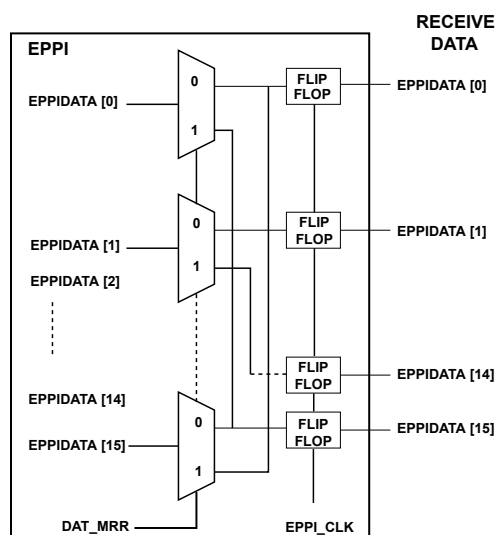


Figure 19-1: Data Mirroring Receive

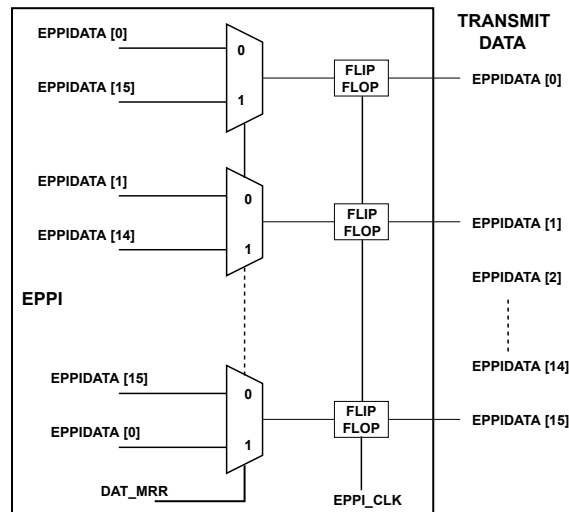


Figure 19-2: Data Mirroring Transmit

Windowing

The EPPI supports windowing for general-purpose input modes. The module can be configured to bring in a region of interest instead of the entire frame of data, which helps reduce bandwidth requirements.

Preamble, Blanking and Stripping Support

The EPPI supports embedding blanking information and clipping of active data for transmission. This functionality is available for single-channel data, interleaved data, and parallel data and supports a data length (EPPI_CTL.DLEN) of 16, 20 bits or 24 bits.

Support of preamble generation or detection and stripping of blanking information is also provided for ITU656 mode and the SMPTE 274M and 296M HD formats. The *Video Mode Comparison* table shows the SMPTE standards in comparison with the ITU656 modes. Preambles for SMPTE and ITU656 modes are identical. Extension of preamble to 12 bits is also supported.

Table 19-5: Video Mode Comparison

Video Mode	Frame Rate	Frame Resolution	Active Video Resolution	Sampling Frequency (MHz)	Remarks
ITU656 (NTSC)	30	1716x525	720x480	27.02	Y-C interleaved
ITU656 (PAL)	25	1728x625	720x576	27.00	Y-C interleaved
SMPTE 296M	30	3300x750	1280x720	74.25	Y,C separate
	60	1650x750	1280x720	74.25	Y,C separate

Table 19-5: Video Mode Comparison (Continued)

Video Mode	Frame Rate	Frame Resolution	Active Video Resolution	Sampling Frequency (MHz)	Remarks
SMPTE 274M	30	2200x1125	1920x1080	74.25	Y,C separate
	60	2200x1125	1920x1080	148.50	Y,C separate
	25	2640x1125	1920x1080	74.25	Y,C separate
	50	2640x1125	1920x1080	148.50	Y,C separate
	24	2750x1125	1920x1080	74.25	Y,C separate

See the clock operating conditions section of the data sheet for the maximum sampling frequency for this product.

EPPI Definitions

The following definitions are helpful when using the EPPI module.

ITU-R BT.-656

Description of a digital video protocol for interfaces and data stream format required to send uncompressed PAL or NTSC standard definition TV (525 or 625 lines) signals.

YUV422

YUV is a color space where luminance (Y) and chrominance (UV) components define the pixels. The suffix signifies how the chrominance components have been decimated and provide formatting information. In this case, the YUV422 format has the chrominance decimated by two, meaning only half of each chrominance component is available. Typical YUV422 formatting interleaves the luminance and chrominance (for example, U1Y1V1Y2U2Y3V2Y4).

RGB888

RGB is a color space where three color values, one red (R), one green (G) and one blue (B), define the pixels. The suffix signifies the bit widths for these color components. In this case, RGB888 means that each red, green, and blue value is 8 bits.

RGB565

RGB is a color space where three color values, one red (R), one green (G) and one blue (B) define the pixels. The suffix signifies the bit widths for these color components. In this case, RGB565 means that the red (R) and blue (B) are 5 bits each while the green (G) is 6 bits. When packed together, each RGB565 pixel can be represented in a 16-bit data word. LCD display panels commonly use this format.

SMPTE 274M

An HD standard defining the spatial resolution (image sample structure) and frame rates for 1920x1080.

SMPTE 296M

An HD standard for defining the spatial resolution (image sample structure) and frame rates for 1280x720.

EPPI Block Diagram

The *EPPI Block Diagram* figure shows the functional blocks within the EPPI.

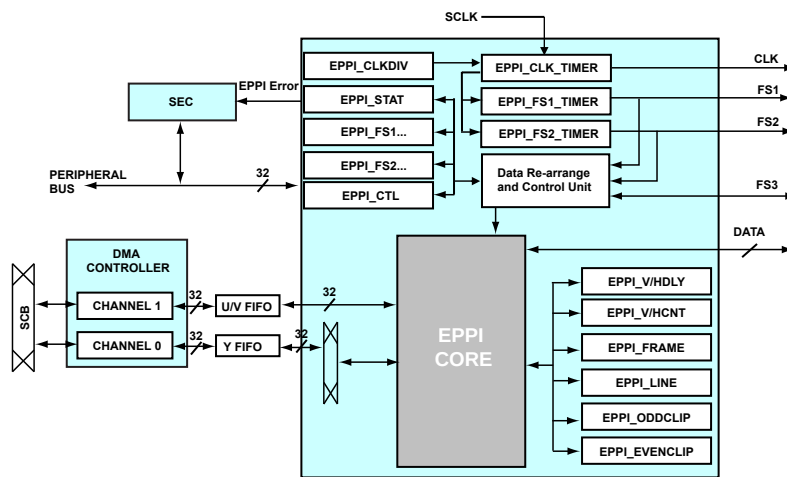


Figure 19-3: EPPI Block Diagram

EPPI Architectural Concepts

The following sections describe the architectural concepts.

- [Reset Operation](#)
- [Frame Sync Polarity and Sampling Edge](#)
- [Direct Memory Access \(DMA\)](#)
- [EPPI Clock](#)

Reset Operation

On a hardware reset, the entire EPPI is reset. All MMRs return to their default values. EPPI interrupt and DMA requests become inactive and internally generated **EPPI_CLK** and frame syncs are aborted.

In software, write 0 to the **EPPI_CTL.EN** bit to reset and reconfigure the EPPI. When disabled in this manner, only the **EPPI_STAT** register is cleared to its reset value. Interrupts and DMA requests become inactive and internally generated clock and frame syncs are aborted.

Frame Sync Polarity and Sampling Edge

The `EPPI_CTL.POLS` and `EPPI_CTL.POLC` bits provide a mechanism to select the active level of the frame syncs and the sampling or driving edge of the EPPI clock, respectively. This functionality allows the EPPI to connect to data sources and receivers with a wide array of control signal polarities. Often, the remote data source or receiver also offer configurable signal polarities. In these cases, the `EPPI_CTL.POLS` and `EPPI_CTL.POLC` bits add flexibility.

Table 19-6: Frame Sync Polarity Selections and Frame Sync Pin States

Bit Setting	Frame Sync 2	Frame Sync 1
POLS = b#00	Active high	Active high
POLS = b#01	Active high	Active low
POLS = b#10	Active low	Active high
POLS = b#11	Active low	Active low

Table 19-7: Clock Polarity Selections and Receive/Transmit Pin States

Bit Setting	Receive		Transmit	
	Sample Data	Sample/Drive Syncs	Drive Data	Sample/Drive Syncs
POLC = b#00	Falling edge	Falling edge	Rising edge	Rising edge
POLC = b#01	Falling edge	Rising edge	Rising edge	Falling edge
POLC = b#10	Rising edge	Falling edge	Falling edge	Rising edge
POLC = b#11	Rising edge	Rising edge	Falling edge	Falling edge

Direct Memory Access (DMA)

The EPPI has a native DMA controller with two channels. A local arbiter arbitrates between these channels and requests are forwarded to the system crossbar. The EPPI has one connection to the fabric.

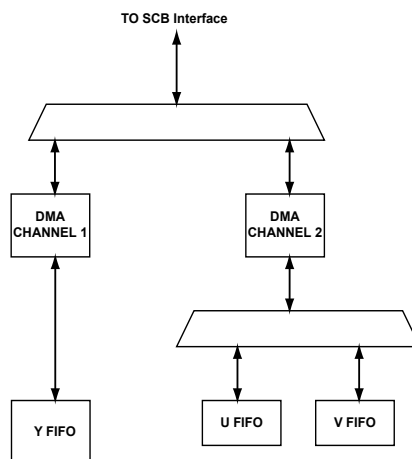


Figure 19-4: EPPI DMA Interface

The EPPI must be used with DMA. Configuring the EPPI DMA channels is a necessary step toward using the EPPI interface. The channels can be configured for either transmit or receive operation, and have a maximum throughput of $(EPPI_CLK) \times (32 \text{ bits/transfer})$. In modes where data lengths permit, packing can increase transfer bandwidth.

The DMA engine generates interrupts at the completion of a row, frame, or partial-frame transfer. The DMA engine also coordinates the source or destination point for the data that is transferred through the EPPI.

The 2D DMA capability allows the processor to be interrupted at the end of a line or after a frame of video is transferred, or if a DMA error occurs. The `DMA_XCNT` and `DMA_YCNT` registers allow for flexible data interrupt points. For example, assume the `DMA_XMOD = DMA_YMOD = 1`. If a data frame contains 320×240 bytes (240 rows of 320 bytes each), the following conditions hold.

- Setting `DMA_XCNT = 320`, `DMA_YCNT = 240`, and `DMA_CFG.INT = 1` interrupts on every row transferred, for the entire frame.
- Setting `DMA_XCNT = 320`, `DMA_YCNT = 240`, and `DMA_CFG.INT = 2` interrupts only on the completion of the frame (when 240 rows of 320 bytes have been transferred).
- Setting `DMA_XCNT = 38,400 ( $320 \times 120$ ), DMA_YCNT = 2, and DMA_CFG.INT = 1 causes an interrupt when half of the frame is transferred, and again when the whole frame is transferred.`

The following is the general procedure for setting up DMA operation with the EPPI.

1. Configure the DMA registers as appropriate for the desired DMA operating mode.
2. Enable the DMA channel for operation.
3. Configure appropriate EPPI registers.
4. Enable the EPPI by writing 1 to the `EPPI_CTL.EN` bit.

EPPI Clock

The EPPI can be supplied with an external clock, or the clock can be generated internally and supplied to external devices. For information on the maximum `PPI_CLK` specification in internal and external clock modes, see the product-specific data sheet.

When using an external `EPPI_CLK`, there can be up to two cycles latency before valid data is received or transmitted.

The internal clock can be generated from `SCLK0` when the `EPPI_CTL.ICLKGEN` bit is set. The value in the `EPPI_CLKDIV` register determines the generated clock frequency. The internally generated EPPI clock frequency is:

$$f_{PCLK} = f_{SCLK0} / (EPPI_CLKDIV + 1)$$

where:

f_{PCLK} – frequency of internally generated EPPI clock

f_{SCLK} – frequency of `SCLK0`

[EPPI_CLKDIV](#) – Clock division value programmed in the [EPPI_CLKDIV](#) register.

The *Relationship Between CLKDIV and the Ratio of SCLK0 to EPPI Clock* table gives a few examples.

Table 19-8: Relationship Between CLKDIV and the Ratio of SCLK0 to EPPI Clock

CLKDIV15–0	EPPI/SCLK0 Clock Ratio
0x0002	1:3
0x0003	1:4
0x0004	1:5
0x0005	1:6
...	...

EPPI Operating Modes

The EPPI supports various receive and transmit modes of operation which include the detection and generation of preamble data. Specifically, the EPPI supports data formats described in the specifications ITU656, SMPTE 274M and SMPTE 296M. In addition to these modes, the EPPI also supports general-purpose receive and transmit using up to three frame syncs (FS).

The control register ([EPPI_CTL](#)) includes most of the bits used for configuring operating modes. The “Register Descriptions” section of this chapter provides complete descriptions of these bits.

ITU-R 656 Modes

The EPPI supports three input modes and one output mode for ITU-R 656 framed data. This section describes these modes.

ITU-R 656 Background

In ITU-R 656 mode, the horizontal (H), vertical (V), and field (F) signals are sent as an embedded part of the video data stream. The signals are sent in a series of bytes that form a control word. ITU-R 656 was formerly known as CCIR-656.

The letter H is used to distinguish between the *start of active video* (SAV) and *end of active video* (EAV) signals. These signals indicate the beginning and end of active video data in each line. The SAV occurs on a 1-to-0 transition of H, and EAV occurs on a 0-to-1 transition of H. The space between EAV and SAV is filled with horizontal blanking data. Therefore, H = 1 during the horizontal blanking portion of the data stream, and H = 0 during the active video portion of the data stream.

The letter V is used to denote the vertical blanking portion of the data stream. A transition in V can occur only in the EAV sequence. When V = 1, the data stream contains vertical blanking data, and when V = 0, the data stream contains active video data.

The letter F is used to distinguish Field 1 from Field 2. Interlaced video has two fields in a frame of data. It requires each field to be handled uniquely, and alternate rows of each field combined to create the actual video image.

For interlaced video, $F = 0$ represents Field 1 (*Odd Field*) and $F = 1$ represents Field 2 (*Even Field*). Progressive video makes no distinction between Field 1 and Field 2, and F is always 0 for progressive video. Interlaced video requires each field to be handled uniquely because alternate rows of each field combine to create the actual video image.

An entire field of video is comprised of active video plus horizontal blanking (the space between an EAV and SAV code) and vertical blanking (the space where $V = 1$). A field of video commences on a transition of the F bit.

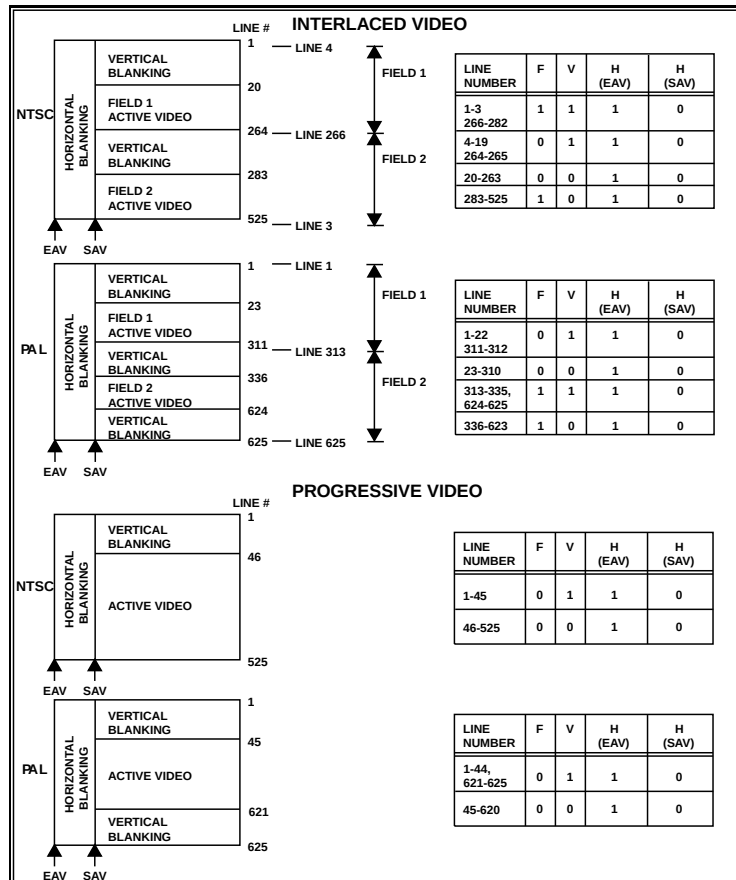


Figure 19-5: Typical Video Frame Partitioning for NTSC/PAL Systems in Interlaced and Progressive ITU-R BT.656 Systems

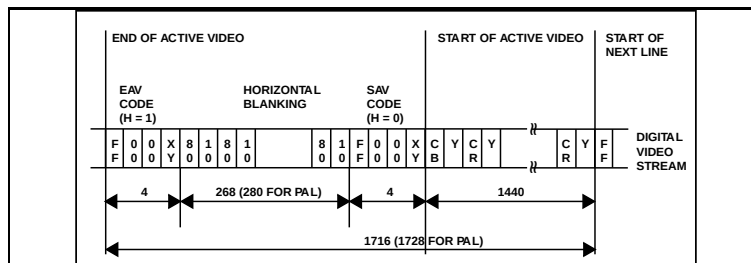


Figure 19-6: ITU-R 656 8-Bit Parallel Data Stream from NTSC (PAL) Systems

NOTE: Refer to the *Control Sequences for 8-Bit and 10-Bit ITU-R 656 Video* table. There is a defined preamble of three data elements (for example, in the case of 8-bit video: 0xFF, 0x00, 0x00), followed by the XY status word. The status word contains four protection bits for error detection and correction excluding the

F (field), V (vertical blanking), and H (horizontal blanking) bits. F and V are only allowed to change as part of EAV sequences (that is, transition from H = 0 to H = 1).

The bit definitions are as follows:

- F = 0 for field 1
- F = 1 for field 2
- V = 1 during vertical blanking
- V = 0 when not in vertical blanking
- H = 0 at SAV
- H = 1 at EAV
- $P3 = V \text{ XOR } H$
- $P2 = F \text{ XOR } H$
- $P1 = F \text{ XOR } V$
- $P0 = F \text{ XOR } V \text{ XOR } H$

P3–P0 are protection bits that enable 1-bit and 2-bit error detection, and 1-bit error correction at the receiver. The EPPI corrects the error if it detects 1-bit errors in F, V, or H. Errors in the protection bits themselves are detected but not corrected.

Table 19-9: Control Sequences for 8-Bit and 10-Bit ITU-R 656 Video

	8-Bit Data								10-Bit Data	
	D9 (MSB)	D8	D7	D6	D5	D4	D3	D2	D1	D0
Preamble	1	1	1	1	1	1	1	1	1	1
	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0
Control Byte	1	F	V	H	P3	P2	P1	P0	0	0

The `EPPI_STAT` register contains 2 bits, `EPPI_STAT.ERRDET` and `EPPI_STAT.ERRNCOR`, that are used to report the status of error detected and error not corrected, respectively.

The `EPPI_STAT.ERRDET` bit is set whenever an error is detected in the status word. However, this bit does not generate an interrupt. The `EPPI_STAT.ERRNCOR` bit is set when more than a 1-bit error is detected in the status word. An interrupt is generated when the `EPPI_STAT.ERRNCOR` bit is set. It can be cleared by clearing the `EPPI_STAT.ERRNCOR` and `EPPI_STAT.ERRDET` bits. Both bits are sticky and W1C.

In many applications, video streams other than the standard NTSC/PAL formats (for example, CIF, QCIF) can be employed. The processor interface is flexible enough to accommodate different row and field lengths. In general, as

long as the incoming video has the proper EAV/SAV codes, the EPPI can read it in. A CIF image could be formatted to be 656-compliant, where EAV and SAV values define the range of the image for each line. The V and F codes are used to delimit fields and frames.

The following sections provide descriptions of EPPI operations.

Table 19-10: Operating Modes and Generic EPPI Operation

		How to configure	Useful for	How to configure in ITU R 656 Tx Mode
ITU-R BT.656 Rx	Entire field	DIR = 0 XFRTYPE = b#01		
	Active video	DIR = 0 XFRTYPE = b#00		
	Blanking only	DIR = 0 XFRTYPE = b#10		
GP 0 FS	Tx	DIR = 1 XFRTYPE = b#11 FSCFG = b#00	Applications where periodic frame syncs are not used to frame the data	BLANKGEN = 1 DLEN = (b#000, b#001 or b#100)
	Rx	DIR = 0 XFRTYPE = b#11 FSCFG = b#00		
GP 1 FS	Tx	DIR = 1 XFRTYPE = b#11 FSCFG = b#01	Interfacing with ADCs, DACs, and other general-purpose devices	BLANKGEN = 1 DLEN = (b#000, b#001 or b#100)
	Rx	DIR = 0 XFRTYPE = b#11 FSCFG = b#01		
GP 2 FS	Tx	DIR = 1 XFRTYPE = b#11 FSCFG = b#10	Video applications that use two hardware synchronization signals, HSYNC and VSYNC	BLANKGEN = 1 DLEN = (b#000, b#001 or b#100)
	Rx	DIR = 0 XFRTYPE = b#11 FS_CFG = b#10		
GP 3 FS	Tx	DIR = 1 XFRTYPE = b#11 FSCFG = b#11	Video applications that use three hardware sync signals, HSYNC, VSYNC, and FIELD	BLANKGEN = 1 DLEN = (b#000, b#001 or b#100)
	Rx	DIR = 0 XFRTYPE = b#11 FSCFG = b#11		

ITU-R 656 Input Modes

In the ITU-R 656 input modes, the video source provides the clock or the system supplies it externally.

As shown in the *ITU-R 656 Input Submodes* figure and described in the following sections, there are three submodes supported for ITU-R 656 inputs: entire field, active video only, and vertical blanking interval only.

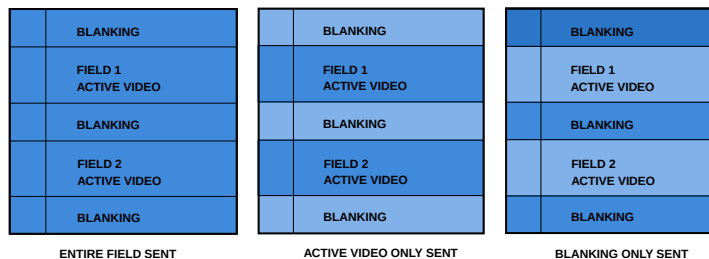


Figure 19-7: ITU-R 656 Input Submodes

Entire Field

In this mode, the EPPI reads the entire incoming bit stream. This stream includes active video as well as control byte sequences and ancillary data that can be embedded in horizontal and vertical blanking intervals.

Data transfer starts immediately after Field 1 synchronization occurs. The transfer does not include the first EAV code that contains the $F = 0$ assignment for interlaced video or the $V = 0$ assignment for progressive video.

Active Video

The EPPI uses this mode when only the active video portion of a field is of interest. The EPPI ignores (does not read in) all data between EAV and SAV, as well as all data present when $V = 1$. Furthermore, the control byte sequences are not stored to memory. The EPPI filters the sequences. After the start of Field 1 synchronizes, the EPPI ignores incoming samples until it sees an SAV.

In active video mode, programs must specify the number of total (active plus vertical blanking) lines per frame in the `EPPI_FRAME` register. Programs must specify the number of total (active plus horizontal blanking plus 8) samples per line in the `EPPI_LINE` register.

In this mode, any input data sequence that is considered part of the preamble is not sent to memory such as in 8-bit ITU mode. If `0xFF` or `0x00` appear in the input data stream, these values are considered part of the preamble. The part of the preamble can appear individually and not be tagged along with the preamble sequence `FE, 00, 00`. This functionality also applies to vertical blanking interval mode.

Vertical Blanking Interval (VBI)

In this mode, data transfer is only active while $V = 1$ is in the control byte sequence. This functionality indicates that the video source is in the midst of the vertical blanking interval (VBI), which is sometimes used for ancillary data transmission. The ITU-R 656 recommendation specifies the format for these ancillary data packets, but the EPPI is not equipped to decode the packets themselves. Software must handle this task. Horizontal blanking data is logged where it coincides with the rows of the VBI.

The VBI is split into two regions within each field. The EPPI considers these two separate regions as one contiguous space. However, frame synchronization begins at the start of Field 1, which does not necessarily correspond to the

start of vertical blanking. For instance, in 525/60 systems, the start of Field 1 ($F = 0$) corresponds to line 4 of the VBI.

In VBI mode, the program must specify the number of total (active plus vertical blanking) lines per frame in the `EPPI_FRAME` register. The program must specify the number of total (active plus horizontal blanking plus 8) samples per line in the `EPPI_LINE` register.

In this mode, any input data sequence that is considered as part of the preamble is not sent to memory such as in 8-bit ITU mode. If 0xFF or 0x00 appears in the input data stream, these values are considered part of the preamble. The part of the preamble can appear individually, and not be tagged along with the preamble sequence FF, 00, 00. This functionality applies to active video mode too.

ITU-R 656 Output in General-Purpose Transmit Modes

In GP transmit mode, the EPPI frames an ITU-R 656 output stream with the proper preambles and blanking intervals by setting the `EPPI_CTL.BLANKGEN` bit. The EPPI fetches active data from memory through the DMA channel, saving DMA bandwidth. The EPPI generates and embeds the proper preamble, status word (EAV and SAV sequences), and blanking data along with the active video from memory. Program the `EPPI_FS1_PASPL`, `EPPI_FS2_WLVB`, `EPPI_FS2_PALPF`, and `EPPI_FS1_WLHB` registers to perform the desired functions. The EPPI can also drive out the frame syncs using the `EPPI_CTL.FSCFG` bit setting.

The *16-Bit Transmit with Internal Blanking Generation* figure shows the bit stream format in 16-bit transmit modes with blanking generation (EPPI_CTL.BLANKGEN enabled). Each 16-bit data sample consists of 8-bit luma (Y) and 8-bit chroma (Cr or Cb) components. During transmission, the chroma data and blanking bytes of value 0x80 are placed on the upper half (MSBs) of the data lines. The luma data and blanking bytes of value 0x10 are placed on the lower half (LSBs) of the data lines.

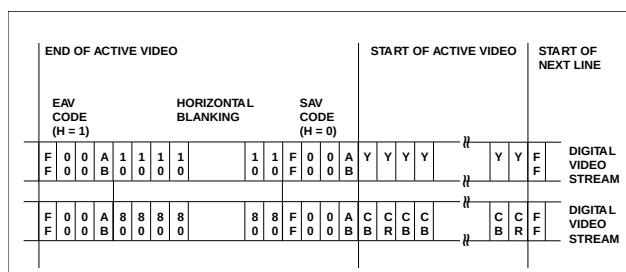


Figure 19-8: 16-Bit Transmit with Internal Blanking Generation

The *Generated Blanking Preamble Sequence* figure shows the data transmitted by the EPPI in this mode. After the EPPI is enabled, and if the EPPI FIFO is not empty, the transmission starts by sending out an EAV sequence for a vertical blanking line. For interlaced video, F starts at 1. For progressive video, F is always 0.

NOTE: Internal blanking generation functionality is valid only when the data length is 8, 10, or 16 bits and when the EPPI is in GP transmit modes. The EPPI_CTL.BLANKGEN bit generates preambles even in GP 2FS mode.

The internal blanking generation functionality of the ITU-R 656 output mode can also be bypassed by clearing the `EPPI_CTL.BLANKGEN` bit. (For example, if sending ancillary data in the blanking interval). The `EPPI_CTL.BLANKGEN` bit generates preambles even in GP 2FS mode.

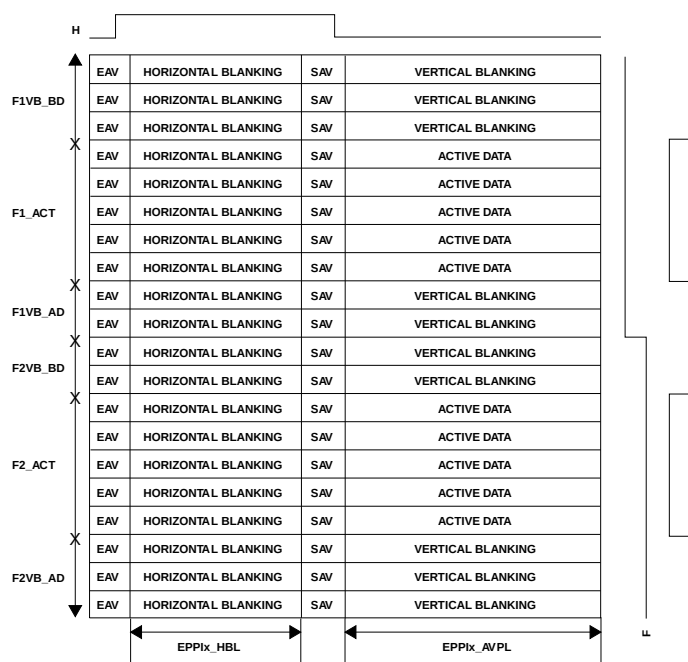


Figure 19-9: Generated Blanking Preamble Sequence

Frame Synchronization in ITU-R 656 Modes

For interlaced video, the start of frame synchronization occurs when a high-to-low transition is detected in F, the field indicator. For progressive video, the start of frame synchronization occurs when a high-to-low transition is detected in V, the vertical blanking indicator. These transitions in F and V can occur only in the EAV sequence. A start of line is detected on a low-to-high transition in H, the horizontal blanking indicator, which occurs in the EAV sequence as well.

For interlaced video, the start of frame corresponds to the start of field 1. Therefore, up to two fields can be ignored before the EPPI receives data. (For example, if field 1 started before the EPPI-to-camera channel was established). For progressive video, the start of frame corresponds to the start of active video.

Because all H and V signaling is embedded in the data stream in ITU-R 656 modes, the EPPI ignores the count registers ([EPPI_HCNT](#), [EPPI_VCNT](#)). However, the EPPI still uses the [EPPI_FRAME](#) register to check for synchronization errors. Therefore, program this MMR with the number of lines expected in each frame of video.

The EPPI monitors the number of EAV-to-SAV transitions that occur from the start of a frame until it decodes the end of frame condition. (For example, a transition from F = 1 to F = 0 for interlaced video and a transition from V = 1 to V = 0 for progressive video).

At the end of frame condition, the actual number of lines processed is compared against the value in [EPPI_FRAME](#). If there is a mismatch, a frame track error is asserted in the [EPPI_STAT](#) register. For example, if an SAV transition was missed, the current field only has NUM_ROWS – 1 rows. But, resynchronization occurs at the start of the next frame. When the EPPI receives the entire field, the field status bit is toggled in the [EPPI_STAT](#) register. This way, an interrupt service routine (ISR) can discern which field was previously read in.

General-Purpose EPPI Modes

The general-purpose (GP) EPPI modes accommodate a wide variety of data capture and transmission applications.

Each EPPI has three bidirectional frame sync pins. The EPPI internally generates frame syncs, or an external device communicating with the EPPI generates them.

GP modes differ based on the number of frame syncs used and the EPPI supports GP 0 FS—GP 3 FS modes.

All the GP modes, except 0 FS mode, support horizontal windowing. GP modes with 2 and 3 frame syncs also support vertical windowing.

For GP transmit modes with internal clock or frame syncs, the EPPI starts generating the clock or frame syncs only when the EPPI FIFO is full for the first time. For GP 0 FS transmit mode, the EPPI only starts transmitting when the EPPI FIFO is full for the first time.

General-Purpose 0 Frame Sync Mode

This mode is useful for applications where periodic frame syncs are not used to frame the data.

After the initial trigger, the EPPI receives or transmits data samples on every clock cycle. However, if the `EPPI_CTL.SKIPEN` bit is set for receive mode, the EPPI receives only alternate data samples.

The `EPPI_LINE`, `EPPI_FRAME`, `EPPI_HCNT`, `EPPI_HDLY`, `EPPI_VCNT`, and `EPPI_VDLY` registers are not valid for GP 0 FS mode. Therefore, windowing is not possible in this mode. Also, line and frame track errors are not applicable in this mode.

GP 0 FS receive mode is further divided into two submodes; internal trigger (`EPPI_CTL.FLDSEL` bit =0) and external trigger (`EPPI_CTL.FLDSEL` bit =1). The submodes are based on how the processor initiates data transmission or reception. GP 0 FS transmit mode is always internally triggered. DMA handles all subsequent data manipulation.

- *Frame synchronization in GP 0 FS external trigger mode.* When the EPPI is programmed in external trigger mode, it does not generate the `EPPI_FS1` signal and the external device must provide a trigger. The EPPI starts receiving the data as soon as an `EPPI_FS1` signal assertion is detected. After that, the DMA handles all subsequent data manipulation and any activity on `EPPI_FS1` is ignored.
- *Frame synchronization in GP 0 FS internal trigger mode.* When the EPPI is programmed in internal trigger mode, it starts receiving or transmitting data as soon as the EPPI clock is enabled and synchronized. There can be up to four PPI clock cycles of latency before valid data is received or transmitted.

General-Purpose 1 Frame Sync Mode

This mode is useful for interfacing the EPPI with analog-to-digital converters (ADCs), digital-to-analog converters (DACs), and other general-purpose devices. This mode works for both transmit and receive.

The `EPPI_FRAME`, `EPPI_VDLY`, and `EPPI_VCNT` registers have no effect in GP 1 FS mode. As a result, frame track errors and vertical windowing are not available.

General-Purpose 2 Frame Sync Mode

This mode is useful for video applications that use two hardware synchronization signals, HSYNC and VSYNC. The HSYNC signal can be connected to EPPI_FS1 and the VSYNC signal can be connected to EPPI_FS2.

Data Enable in General-Purpose 2 Frame Sync Transmit Mode

The EPPI_FS3 pin functions as a data enable (DEN) pin, when EPPI is configured in GP 2 FS transmit mode and generating the frame sync internally. The bits EPPI_CTL.MUXSEL and EPPI_CTL.CLKGATEN are not enabled. The functionality of the DEN pin is described in the following two cases.

Case 1

Blanking generation is configured using the EPPI_CTL.BLANKGEN bit. EPPI data length (EPPI_CTL.DLEN bit) is configured for 8, 10, or 16-bit transfers. The EPPI_FS3 pin asserts during the *active data* regions, aligned with EPPI_CLK according to the clock polarity (EPPI_CTL.POLC bit) settings. For this mode, the pin EPPI_FS3 is driven based on the EPPI_CTL.POLC setting. The pin EPPI_FS3 is driven out on the same EPPI clock edge that drives out data. The frame sync polarity (EPPI_CTL.POLS) setting does not apply here—EPPI_FS3 is always active high in this mode.

Case 2

Blanking generation (EPPI_CTL.BLANKGEN = 0) is disabled. Or blanking generation is enabled, but the EPPI data length (EPPI_CTL.DLEN bit) is configured for a transfer size other than 8, 10, or 16 bits. The EPPI_FS3 pin asserts at the start of the active data region on each line, aligned with EPPI_CLK according to the EPPI_CTL.POLC bit settings. For this mode, the pin EPPI_FS3 is driven based on the EPPI_CTL.POLC setting. The EPPI_FS3 signal is driven out on the same EPPI clock edge that drives out data.

The EPPI_CTL.POLS bit setting does not apply for case 2. The EPPI_FS3 signal is always active high in this mode. Once asserted, EPPI_FS3 stays asserted for the number of clock cycles per line configured in the EPPI_HCNT register, then it deasserts. This behavior on each line continues for the total number of lines programmed in the EPPI_VCNT register per frame. The behavior repeats at the start of subsequent video frames.

In case 2, if transmission of valid data is held off due to delays programmed in the EPPI_HDLY or EPPI_VDLY registers, the assertion of EPPI_FS3 is also held off. The delay is on a per-line or per-frame basis.

General-Purpose 3 Frame Sync Mode

This mode is useful for video applications that use three synchronization signals for hardware: HSYNC, VSYNC, and FIELD. The HSYNC connects to the EPPI_FS1 pin, VSYNC connects to the EPPI_FS2 pin, and FIELD connects to the EPPI_FS3 pin.

GP 3 FS mode is similar in operation to GP 2 FS mode. However, the start of frame synchronization in GP 3 FS also considers the state of the EPPI_FS3 pin. All the windowing register settings ([EPPI_FRAME](#), [EPPI_LINE](#), [EPPI_HDLY](#), [EPPI_HCNT](#), [EPPI_VDLY](#), and [EPPI_VCNT](#) registers), as well as data reception or transmission and error generation are the same as for GP 2 FS mode. In addition, for GP 3 FS mode with internal frame syncs, the EPPI_CTL.FLDSEL bit setting specifies the condition under which the transfer begins.

The EPPI generates the EPPI_FS3 signal and toggles during every assertion of EPPI_FS2 or a combination of EPPI_FS2 and EPPI_FS1. The toggle depends on the EPPI_CTL.FLDSEL bit setting. The EPPI skips an EPPI_FS2 signal when the EPPI_FS3 value is high. Because of this condition, program the EPPI_FS2 period value to half of the total number of pixels in the frame as in GP 3 FS mode. When in GP 2 FS mode, program the EPPI_FS2 period with the value equal to the number of pixels per frame.

Supported Data Formats

The following sections describe EPPI receive and transmit data formats.

Receive Data Formats

The *EPPI Receive Data Formats* table provides information about EPPI configuration for specific use models for receive data.

Table 19-11: EPPI Receive Data Formats

Input Data Width	Use Model	Splitting/Packing Options
8	NTSC/PAL data	EPPI_CTL.SPLTEO =1 EPPI_CTL.SUBSPLTODD =1 if necessary to separate chroma components
	RGB sensor	No splitting possible. EPPI_CTL.PACKEN =1 – Four EPPI words are packed to 32-bit DMA data. EPPI_CTL.PACKEN =0 – Each EPPI word is sent as 8-bit data on the 32-bit DMA bus. This transfer consumes 4 times the DMA bandwidth of the 8-bit case with EPPI_CTL.PACKEN =1;
	ADCs	Gives I (in phase) and Q (quadrature) components. EPPI_CTL.SPLTEO =1 EPPI_CTL.SUBSPLTODD =0 since there are only two components.

Table 19-11: EPPI Receive Data Formats (Continued)

Input Data Width	Use Model	Splitting/Packing Options
10	NTSC/PAL data	Each EPPI word is zero filled or sign extended to 16 bits. EPPI_CTL.SPLTEO =1. EPPI_CTL.SUBSPLTODD =1 if necessary to separate chroma components.
	RGB sensor	No splitting possible. EPPI_CTL.PACKEN =1. Two EPPI words are zero filled or sign extended to 16 bits and packed to 32-bit DMA data. EPPI_CTL.PACKEN =0. Each EPPI word can be zero filled or sign extended to 16 bits and sent as a 16-bit data on the 32-bit DMA bus. This transfer consumes double the bandwidth of the 10-bit case with EPPI_CTL.PACKEN =1;
	ADCs	Each EPPI word is zero filled or sign extended to 16 bits. EPPI_CTL.SPLTEO =1 SEPPI_CTL.SUBSPLTODD = 0 since there are only two components.
12	RGB sensor	No splitting possible. EPPI_CTL.PACKEN =1. Two EPPI words are zero filled or sign extended to 16 bits and packed to 32-bit DMA data. EPPI_CTL.PACKEN =0. Each EPPI word can be zero filled or sign extended to 16 bits and sent as a 16-bit data on the 32-bit DMA bus. This transfer consumes double the bandwidth of the 12-bit case with EPPI_CTL.PACKEN =1;
	ADCs	Each EPPI word is zero filled or sign extended to 16 bits. EPPI_CTL.SPLTEO =1 EPPI_CTL.SUBSPLTODD =0 since there are only two components.
14	ADCs	Each EPPI word is zero filled or sign extended to 16 bits. EPPI_CTL.SPLTEO =1 EPPI_CTL.SUBSPLTODD =0 since there are only two components.

Table 19-11: EPPI Receive Data Formats (Continued)

Input Data Width	Use Model	Splitting/Packing Options
16	8-bit luma/chroma pair for NTSC or HD	EPPI_CTL.SPLTEO =1, EPPI_CTL.SPLTWRD =1, EPPI_CTL.SUBSPLTODD =1 if necessary to separate chroma components.
	16-bit luma/chroma pair for NTSC or HD	EPPI_CTL.SPLTEO =1, EPPI_CTL.SPLTWRD =0, EPPI_CTL.SUBSPLTODD =1 if necessary to separate chroma components.
	RGB565 sensor	No splitting possible. EPPI_CTL.PACKEN =1. Two EPPI words are packed to a 32-bit DMA data. EPPI_CTL.RGBFMTEN is valid only in transmit modes. So, RGB565 cannot be byte aligned in memory. EPPI_CTL.PACKEN =0. Each EPPI word is sent as a 16-bit data on the 32-bit DMA bus. This transfer consumes double the bandwidth of the 16-bit case with EPPI_CTL.PACKEN =1
	8-bit ADCs I/Q pair	EPPI_CTL.SPLTEO =1, EPPI_CTL.SPLTWRD =1, EPPI_CTL.SUBSPLTODD =0.
	16-bit ADCs I/Q pair	EPPI_CTL.SPLTEO =1, EPPI_CTL.SPLTWRD =0, EPPI_CTL.SUBSPLTODD =0.

Transmit Data Formats

The *EPPI Transmit Data Formats* table provides information about EPPI configuration for specific use models for transmit data.

Table 19-12: EPPI Transmit Data Formats

Output Data Width	Use Model	Splitting/Packing Options
8	NTSC/PAL data	EPPI_CTL.SPLTEO =1 EPPI_CTL.SUBSPLTODD =1 if the chroma components (U and V) come in separate DMA words.
	Serial RGB for lower-resolution LCDs	No splitting possible. EPPI_CTL.PACKEN =1. The 32-bit DMA data is unpacked to drive four EPPI words. EPPI_CTL.PACKEN =0. The lowest 8 bits of the DMA data is driven on the EPPI data and the rest of the 24 bits are discarded. This transfer consumes 4 times the DMA bandwidth of the 8-bit case with EPPI_CTL.PACKEN =1.
10	NTSC/PAL data	EPPI_CTL.SPLTEO =1 EPPI_CTL.SUBSPLTODD =1 if the chroma components (U and V) come in separate DMA words.
	DACs	EPPI_CTL.SPLTEO =1, EPPI_CTL.SUBSPLTODD =0.

Table 19-12: EPPI Transmit Data Formats (Continued)

Output Data Width	Use Model	Splitting/Packing Options
12	DACs	EPPI_CTL.SPLTEO =1, EPPI_CTL.SUBSPLTODD =0.
14	DACs	EPPI_CTL.SPLTEO =1, EPPI_CTL.SUBSPLTODD =0.
16	8-bit luma/chroma pair for NTSC or HD	EPPI_CTL.SPLTEO =1, EPPI_CTL.SPLTWDRD =1, EPPI_CTL.SUBSPLTODD =1 if the chroma components (U and V) come in separate DMA words.
	16-bit luma/chroma pair for NTSC or HD	EPPI_CTL.SPLTEO =1, EPPI_CTL.SPLTWDRD =0, EPPI_CTL.SUBSPLTODD =1 if the chroma components (U and V) come in separate DMA words.
	RGB565 LCD	No splitting possible. EPPI_CTL.RGBFMTEN =1. Takes RGB888 data from the memory and drops the LSBs from each component to drive out RGB565 data.
	8-bit ADCs I/Q pair	EPPI_CTL.SPLTEO =1, EPPI_CTL.SPLTWDRD =1, EPPI_CTL.SUBSPLTODD = 0
	16-bit ADCs I/Q pair	EPPI_CTL.SPLTEO =1, EPPI_CTL.SPLTWDRD =0, EPPI_CTL.SUBSPLTODD =1
18	RGB666 LCD	No splitting possible. EPPI_CTL.RGBFMTEN =1. Takes RGB888 data from the memory and drops the 2 LSBs from each component to drive out RGB666 data.

Data Transfer Modes

The following sections describe EPPI data transfer modes, including receive or transmit data packing, sign extension, zero fill, receive or transmit split modes, clock gating, delayed start, and data consistency management.

Data Packing for Receive Modes

For receive modes, if the EPPI_CTL.PACKEN bit =1 and the DMA is 32 bits, the EPPI packs the incoming data into 32-bit words based on the EPPI_CTL.DLEN and EPPI_CTL.SWAPEN bit settings. When EPPI_CTL.SWAPEN =0, the EPPI puts the first data in the least significant bits and when EPPI_CTL.SWAPEN =1, the EPPI puts the first data in the most significant bits. The packing options for the EPPI_CTL.DLEN bits are as follows.

- When EPPI_CTL.DLEN =8, four 8-bit words can be packed into one 32-bit word.
- When EPPI_CTL.DLEN =16, two 16-bit words can be packed into one 32-bit word.
- For EPPI_CTL.DLEN values that are more than 8 bits but less than 16 bits, two such words are either sign-extended or zero-filled to 16 bits, and packed into one 32-bit word.
- When EPPI_CTL.DLEN =18, the EPPI sign-extends or zero-fills the 18-bit data to 24 bits and packs four 24-bit words into three 32-bit words.

- When `EPPI_CTL.DLEN = 24`, the EPPI packs four 24-bit words into three 32-bit words.

When `EPPI_CTL.PACKEN = 0`, the EPPI receives the incoming data and sends it on the bus as-is. If `EPPI_CTL.DLEN` is less than or equal to 16 bits, the DMA is a 16-bit DMA; otherwise it is a 32-bit DMA.

Data Packing for Transmit Modes

For transmit modes, if the `EPPI_CTL.DLEN` bit = 1 and the DMA is a 32-bit DMA, the EPPI unpacks the 32-bit word according to the `EPPI_CTL.DLEN` and `EPPI_CTL.SWAPEN` bit settings.

If `EPPI_CTL.SWAPEN = 1`, the EPPI transmits the most significant bits as the first data, and if `EPPI_CTL.SWAPEN = 0`, the EPPI transmits the least significant bits as the first data. The unpacking options for the `EPPI_CTL.DLEN` bits are as follows.

- When `EPPI_CTL.DLEN = 8`, the EPPI transmits one 32-bit word from memory as four 8-bit data words.
- For `EPPI_CTL.DLEN` values greater than 8 bits but less than or equal to 16 bits, the EPPI transmits one 32-bit word from memory as two 16-bit data words.
- When `EPPI_CTL.DLEN = 18` or `24`, the EPPI transmits three 32-bit words from memory as four data words.

Sign-Extended and Zero-Filled Data

The following list describes the bit settings and functionality for sign-extending and zero-filling data.

- For `EPPI_CTL.DLEN` equal to 10, 12 or 14, data is zero-filled or sign-extended to 16 bits.
- For `EPPI_CTL.DLEN` equal to 18 bits, data is zero-filled or sign-extended to 24 bits if packing is enabled, and zero-filled or sign-extended to 32 bits if packing is disabled.
- For `EPPI_CTL.DLEN` equal to 24 bits, data is zero-filled or sign-extended to 32 bits if packing is disabled.
- For `EPPI_CTL.DLEN` equal to 8 bits, data is zero-filled or sign-extended to 16 bits if packing is disabled.
- If `EPPI_CTL.SIGNEXT = 1`, then the data is sign-extended, otherwise it is zero-filled.

Split Receive Modes

The control register has three control bits for split receive modes: `EPPI_CTL.SPLTEO`, `EPPI_CTL.SUBSPLTODD`, and `EPPI_CTL.DMACFG`. Packing is not valid in split modes.

- If `EPPI_CTL.SPLTEO = 1`, the EPPI splits the incoming data stream into two substreams, an even stream, and an odd stream, and packs them separately.
- The `EPPI_CTL.SUBSPLTODD` bit is available only when `EPPI_CTL.SPLTEO = 1`. When `EPPI_CTL.SUBSPLTODD = 1`, the EPPI subsplits the odd substream, and packs the streams separately.
- The `EPPI_CTL.DMACFG` bit is also available only if `EPPI_CTL.SPLTEO = 1`. If `EPPI_CTL.DMACFG = 1`, the EPPI uses two DMA channels and if `EPPI_CTL.DMACFG = 0`, the EPPI uses only one DMA channel.

Split Transmit Modes

The EPPI_CTL register has three control bits for split transmit modes: EPPI_CTL.SPLTEO, EPPI_CTL.SUBSPLTODD, and EPPI_CTL.DMACFG. The DMA is always a 32-bit DMA. Packing is not valid in split modes.

- If EPPI_CTL.SPLTEO = 1, the EPPI receives the Luma (Y3Y2Y1Y0) and interleaved Chroma (Cr1Cb1Cr0Cb0) data as 32 bits from the DMA channel. The EPPI interleaves the data to form a 4:2:2 YCrCb data stream to transmit.
- The EPPI_CTL.SUBSPLTODD bit is available only when EPPI_CTL.SPLTEO = 1. In this case, if EPPI_CTL.SUBSPLTODD = 1, the EPPI receives the Luma (Y3Y2Y1Y0) and deinterleaved Chroma (Cb3Cb2Cb1Cb0 and Cr3Cr2Cr1Cr0). The EPPI interleaves the data to form a 4:2:2 YCrCb data stream to transmit. (The EPPI does not decimate the chroma data when formatting it into 4:2:2.)
- The EPPI_CTL.DMACFG bit is also valid only if EPPI_CTL.SPLTEO = 1. If EPPI_CTL.DMACFG = 1, the EPPI uses two DMA channels and if EPPI_CTL.DMACFG = 0, the EPPI uses only one DMA channel.

Clock Gating

In ITU-R BT.656 and GP 0/1/2 FS modes, EPPI_FS3 becomes a clock-gating input. This functionality is valid for both internally and externally sourced EPPI_CLK, in both receive and transmit modes. This clock gating signal must be synchronous with EPPI_CLK. The external device on the rising edge of EPPI_CLK must drive the clock gating signal. Its function is to hold the sync and data lines in their current state until EPPI_FS3 is driven low. There are no additional latency cycles upon coming out of clock gating mode.

If clock gating is not required, the EPPI_FS3 pin must either be tied to ground, or configured to operate as another of its multiplexed functions.

In GP 2 FS transmit mode with internally generated frame syncs, the EPPI_FS3 pin functions as a data enable signal.

Support for Delayed Start of EPPI Frame Syncs

The EPPI supports a delayed start of the EPPI_FS1 and EPPI_FS2 frame syncs. The EPPI_FS1_DLY and EPPI_FS2_DLY registers are programmable registers corresponding to EPPI_FS1 (HSYNC) and EPPI_FS2 (VSYNC).

The delay programmed in these registers applies to the first active edge of the internally generated frame sync. The delay starts from the first EPPI_CLK edge. The delay counter runs only for the first time and then shuts off until the EPPI is reenabled. (The delay counter is the period counter itself, since they do not run together.) Program the delay registers prior to the first EPPI_CLK edge (similar to the width and period registers). The *EPPI Delayed Frame Sync Generation* figure shows the functioning of EPPI_FS1 and EPPI_FS2.

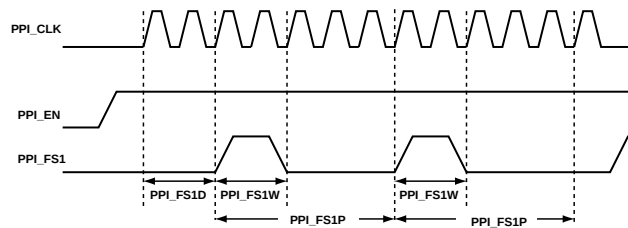


Figure 19-10: EPPI Delayed Frame Sync Generation

Ignoring Premature External Frame Syncs for Data Consistency

Once a frame has started with a VSYNC followed by an HSYNC (or both coming together), a line is tracked. When the count expires, the state machine waits at the end of line for an HSYNC to come. With the arrival of the HSYNC, the state machine starts tracking the next line, and so on.

The number of lines tracked is counted separately. Once the end of a frame is reached, the state machine waits there for the next VSYNC/HSYNC combination. The next frame starts once they are sampled. Unfortunately, every incoming FS (VSYNC or HSYNC) resets the respective counters and the tracking starts all over (even if the FS signals are premature). The result is incomplete data (or frames) to enter into memory through the PxP interface.

To correct this problem, the EPPI waits for a frame or line completion before considering any incoming FS as valid.

- Single FS mode and line tracking in dual FS mode – When a line is in progress, when HSYNC is detected prematurely, it is ignored. A line track underflow event is generated.
- Dual FS mode – If a VSYNC is received when a frame is in progress, it is ignored. A frame track underflow error (EPPI_STAT.FTERRUNDR) is generated.

Ignoring the FS ensures that once a frame starts, the amount of data that goes into the memory/PxP interface corresponds exactly to the programmed data size in a frame.

NOTE: Even if the premature FS is a valid FS, the state machine loses at most one frame and it recovers in the subsequent FS. The FS to number of data going into the memory relationship is always maintained as programmed.

When data underflow errors occur at the DMA interface, the EPPI does the following.

- If a premature line sync is detected, an LT underflow error is generated (EPPI_STAT.LTERRUNDR =1). All further line track errors are ignored until the EPPI detects the next valid line sync.
- If a premature frame sync is detected, an FT underflow error is generated (EPPI_STAT.FTERRUNDR =1). All further frame track and line track errors are ignored until the EPPI detects the next valid frame sync.

EPPI Event Control

The following sections describe how EPPI manages events.

EPPI Status, Error, and Interrupt Signals

The EPPI generates error interrupts (flagged in the `EPPI_STAT` register) when any one of the following error conditions occur.

- `EPPI_STAT.YFIFOERR` (YFIFO underflow or overflow)
- `EPPI_STAT.CFIFOERR` (CFIFO underflow or overflow)
- `EPPI_STAT.LTERROVR` (line track overflow error)
- `EPPI_STAT.LTERRUNDR` (line track underflow error)
- `EPPI_STAT.FTERROVR` (frame track overflow error)
- `EPPI_STAT.FTERRUNDR` (frame track underflow error)
- `EPPI_STAT.ERRNCOR` (ITU preamble error not corrected)

A W1C (write-1-to-clear) operation clears the error conditions. Each of the individual conditions which cause an EPPI error interrupt can be masked. The interrupt mask register (`EPPI_IMSK`) allows the masking of individual conditions which cause error interrupts.

There is only one interrupt line from each EPPI so all interrupts are internally OR'ed and sent as a single interrupt to the core. The `EPPI_STAT` register must then be read to discover specific errors. The following sections describe these errors in detail.

Frame and Line Track Errors

In external frame sync mode, the EPPI uses line track error (`EPPI_STAT.LTERROVR` and `EPPI_STAT.LTERRUNDR`) and frame track error (`EPPI_STAT.FTERROVR` and `EPPI_STAT.FTERRUNDR`) status bits to monitor the line and frame synchronization errors. The EPPI updates the bits when there is a mismatch detected in the HSYNC and VSYNC as compared to the programmed values in `EPPI_LINE` and `EPPI_FRAME` count registers.

Line Track Errors

The line track overflow (`EPPI_STAT.LTERROVR`) and underflow errors (`EPPI_STAT.LTERRUNDR`) generate a maskable interrupt as soon as the EPPI identifies them and not at the next frame sync.

- If the frame sync has not arrived when the `EPPI_LINE` counter expires, then the `EPPI_STAT.LTERROVR` error is generated.
- When the `EPPI_LINE` counter is running and a frame sync is detected, the `EPPI_STAT.LTERRUNDR` error is generated. A W1C operation clears both interrupts.

Frame Track Errors

The frame track overflow (`EPPI_STAT.FTERROVR`) and underflow errors (`EPPI_STAT.FTERRUNDR`) generate a maskable interrupt as soon as the EPPI identifies them. When the `EPPI_FRAME.VALUE` counter expires, the `EPPI_STAT.FTERROVR` error is reported before the next frame sync arrives.

When the `EPPI_FRAME` counter is running and a frame sync is detected, then an `EPPI_STAT.FTERRUNDR` is reported.

Both errors generate an error interrupt. Perform a W1C operation to clear the interrupts at their respective locations in the status register.

A premature frame sync results in a frame track under run error. But, the error is logged (register bit set) only after the subsequent blanking period (if any) elapses.

Preamble Error Not Corrected Error

The EPPI supports data embedded frame syncs in ITU and SMPTE formats. In these formats, the module can receive an erroneous preamble which is not correctable. The `EPPI_STAT.ERRNCOR` error signals when this event occurs.

EPPI Programming Model

The following sections describe programming techniques, including receiving or transmitting ITU-R 656 frames; configuring transfers in GP0, GP1, GP2, and GP3 modes; and managing EPPI mode configurations.

Receiving ITU-R 656 Frames

The EPPI supports the reception of ITU-R 656 compliant frames.

1. Configure the EPPI to receive either full ITU-R 656 frame, active video, or blanking information by configuring the `EPPI_CTL.XFRTYPE` bits.
2. In both active video mode and in VBI (vertical blanking information) mode, specify the number of total (active plus vertical blanking) lines per frame in the `EPPI_FRAME` register. Specify the number of total (active plus horizontal blanking plus 8) samples per line in the `EPPI_LINE` register.
3. Configure DMA descriptors to move the data to memory.
4. Enable DMA.
5. Enable the EPPI.
6. To program the EPPI in internal clock mode, follow the procedure above with the `EPPI_CTL.ICLKGEN` bit =0. After enabling the EPPI, add a delay of 200 SCLK0 cycles (worst case) to ensure the EPPI FIFO becomes full. Then switch to internal clock mode by setting the `EPPI_CTL.ICLKGEN` bit =1.

Depending on the EPPI configuration, either the full ITU-R 656 frame is moved to memory or only the active video or only the blanking information.

Transmitting ITU-R 656 Frames in GP Transmit Modes

The EPPI can take active video from memory and generate the proper preambles and blanking information to produce valid ITU-R 656 video frames for transmission.

1. Provide active data frame in memory.
2. Set the `EPPI_CTL.BLANKGEN` bit so the EPPI generates blanking information.
3. Configure the `EPPI_FS1_WLHB`, `EPPI_FS1_PASPL`, `EPPI_FS2_WLVB`, `EPPI_FS2_PALPF` registers accordingly.
4. Configure the rest of the EPPI settings.
5. Configure DMA to fetch active frame data from memory buffers.
6. Enable DMA.
7. Enable the EPPI.
8. To program the EPPI in internal clock mode, follow the procedure above with the `EPPI_CTL.ICLKGEN` bit =0. After enabling the EPPI, add a delay of 200 SCLK0 cycles (worst case) to ensure the EPPI FIFO becomes full. Then switch to internal clock mode by setting the `EPPI_CTL.ICLKGEN` bit =1.

The EPPI takes the active data from memory, generates the blanking information, and transmits an ITU-R 656 frame

Configuring Transfers in GP 0 FS Mode

The EPPI can be configured to not use periodic frame syncs to frame the data.

1. Configure the EPPI to operate in GP 0 FS mode by setting `EPPI_CTL.XFRTYPE = b#11` and `EPPI_CTL.FSCFG = b#00`.
2. When receiving, configure the EPPI to trigger on internally or externally by setting the `EPPI_CTL.FLDSEL` field appropriately. When transmitting, the EPPI always generates a trigger internally.
3. Configure DMA to move the data to or from memory.
4. Enable DMA.
5. Enable EPPI.
6. To program the EPPI in internal clock mode, follow the procedure above with the `EPPI_CTL.ICLKGEN` bit =0. After enabling the EPPI, add a delay of 200 SCLK0 cycles (worst case) to ensure the EPPI FIFO becomes full. Then switch to internal clock mode by setting the `EPPI_CTL.ICLKGEN` bit =1.

The DMA descriptions control the amount of data transferred. The frame syncs from the EPPI do not control the amount.

Configuring Transfers in GP 1 FS Mode

The GP 1 FS mode is useful for interfacing the EPPI with analog-to-digital converters (ADCs), digital-to-analog converters (DACs), and other general-purpose devices. This mode works for both transmit and receive.

NOTE: The `EPPI_FRAME`, `EPPI_VDLY`, and `EPPI_VCNT` registers have no effect in GP 1 FS mode. As a result, frame track errors and vertical windowing are not possible in this mode.

1. Configure GP 1 FS mode by setting the `EPPI_CTL.XFRTYPE` bit =b#11 and the `EPPI_CTL.FSCFG` bit =b#01. An external device can provide the frame syncs or the EPPI can source the frame syncs.
2. Program the `EPPI_LINE` register to contain the number clock cycles expected between two assertions of the `EPPI_FS1` signal to monitor the line track errors. Program the `EPPI_LINE` register before the `EPPI_HCNT` register.
3. Program the `EPPI_HDLY` register to contain the number of clock cycles to wait after the assertion of `EPPI_FS1`. For example, the start of frame.
4. Program the `EPPI_HCNT` register to contain the number of data samples to receive or transmit for each frame.
5. Configure DMA to move the data to or from memory.
6. Enable DMA.
7. Enable the EPPI.
8. To program the EPPI in internal clock mode, follow the procedure above with the `EPPI_CTL.ICLKGEN` bit =0. After enabling the EPPI, add a delay of 200 SCLK0 cycles (worst case) to ensure the EPPI FIFO becomes full. Then, switch to internal clock mode by setting the `EPPI_CTL.ICLKGEN` bit =1.

Data moves in or out of memory. A frame sync frames the data for every line.

Configuring Transfers in GP 2 FS Mode

GP 2 FS mode is useful for video applications that use two hardware synchronization signals, HSYNC and VSYNC. The HSYNC connects to the `EPPI_FS1` signal and VSYNC connects to the `EPPI_FS2` signal.

1. Configure the EPPI to operate in GP 2 FS mode by setting the `EPPI_CTL.XFRTYPE` bit =b#11 and the `EPPI_CTL.FSCFG` bit =b#10. An external device can provide the frame syncs or the EPPI can source the frame syncs.
2. Program the `EPPI_FRAME` register to contain the number of expected lines per frame. The value can be equal to the number of `EPPI_FS1` signal assertions expected between the start of each frame sync. The EPPI uses the value to monitor frame track errors. Program the `EPPI_FRAME` register before the `EPPI_VCNT` register.
3. Program the `EPPI_LINE` register to contain the number of clock cycles expected between two assertions of the `EPPI_FS1` signal to monitor line track errors. Program the `EPPI_LINE` register before the `EPPI_HCNT` register.
4. Program the `EPPI_HDLY` register to configure the number of clock cycles to wait after the assertion of the `EPPI_FS1` signal. (For example, the start of the line).
5. Program the `EPPI_HCNT` register to contain the number of data samples to receive or transmit for each line.

6. Program the `EPPI_VDLY` register to contain the number of lines to wait after the start of frame is detected.
7. Program the `EPPI_VCNT` register to contain the number of lines to receive or transmit.
8. If setting up the EPPI for transmit, the data enable (DEN) pin behaves according to the enabling of the blanking generation and the data length setting (DLEN). See [Data Enable in General-Purpose 2 Frame Sync Transmit Mode](#) for more details.
9. Enable DMA.
10. Enable the EPPI.
11. To program the EPPI in internal clock mode, follow the procedure above with the `EPPI_CTL.ICLKGEN` bit =0. After enabling the EPPI, add a delay of 200 SCLK0 cycles (worst case) to ensure the EPPI FIFO becomes full. Then switch to internal clock mode by setting the `EPPI_CTL.ICLKGEN` bit =1.

Data moves in or out of memory. A frame sync frames the data for every line and frame.

Configuring Transfers in GP 3 FS Mode

GP 3 FS mode is useful for video applications that use three synchronization signals for hardware: HSYNC, VSYNC, and FIELD. The HSYNC connects to `EPPI_FS1`, VSYNC connects to `EPPI_FS2`, and FIELD connects to `EPPI_FS3`.

1. Configure the EPPI to operate in GP 3 FS mode by setting the `EPPI_CTL.XFRTYPE` bit =b#11 and the `EPPI_CTL.FSCFG` bit =b#11. An external device can provide the frame syncs or the EPPI can source the frame syncs.
2. Configure the windowing registers according to steps in GP 2 FS mode.
3. Enable DMA.
4. Enable the EPPI.
5. To program the EPPI in internal clock mode, follow the procedure above with the `EPPI_CTL.ICLKGEN` bit =0. After enabling the EPPI, add a delay of 200 SCLK0 cycles (worst case) to ensure the EPPI FIFO becomes full. Then switch to internal clock mode by setting the `EPPI_CTL.ICLKGEN` bit =1.

Data moves in or out of memory. A frame sync frames the data for every line and frame. Operation and result are similar to operation in GP 2 FS mode but the EPPI also uses the `EPPI_FS3` signal.

Configuring the EPPI to Use the Windowing Feature

Windowing is a useful feature for applications where the region of interest is smaller than the active video stream (for example, sensor calibration, auto-focusing, and others). It can result in significant DMA bandwidth reduction. The EPPI supports windowing for GP input modes.

1. Program the `EPPI_FRAME` register with the number of lines the frame contains.

2. Program the `EPPI_LINE` register with the number of samples per line in the frame.
3. Program the `EPPI_VDLY` register with the number of lines to wait after the start of a new frame before starting to read or transmit data.
4. Program the `EPPI_VCNT` register with the number of lines to read in or write out after `EPPI_VDLY` number of lines from the start of the frame.
5. Program the `EPPI_HDLY` register with the number of clock cycles to delay after the assertion of `EPPI_FS1` is detected for the start of a new line.
6. Program the `EPPI_HCNT` register with the number of samples to read in or write out after `EPPI_HDLY` number of cycles have expired since the assertion of `EPPI_FS1`.

EPPI Mode Configuration

This section describes EPPI mode configurations, including support for all EPPI transmit and receive modes.

Configuring 8-Bit Receive Mode

For 8-bit non-split receive mode and if `EPPI_CTL.PACKEN = 1`, the EPPI packs 4 bytes of incoming data into a 32-bit word. Alternate even or odd samples can be skipped based on the `EPPI_CTL.SKIPEN` and `EPPI_CTL.SKIPEO` bits. The first incoming data can be placed either in the least significant bit positions or in the most significant bit positions, based on the `EPPI_CTL.SWAPEN` bit setting.

Table 19-13: 8-Bit Receive Mode with Packing Enabled

Pin Data (8 bits)	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=1 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=1 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=1 SIGNEXT=X
0x11						
0x22						
0x33						
0x44	0x4433 2211	0x1122 3344				
0x55						
0x66						
0x77			0x7755 3311		0x1133 5577	
0x88	0x8877 6655	0x5566 7788		0x8866 4422		0x2244 6688
0x99						
0xAA						
0xBB						

Table 19-13: 8-Bit Receive Mode with Packing Enabled (Continued)

Pin Data (8 bits)	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=1 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=1 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=1 SIGNEXT=X
0xCC	0xCCBB AA99	0x99AA BBCC				
0xDD						
0xEE						
0xFF			0xFFDD BB99		0x99BB DDFF	
0x00	0x00FF EEDD	0xDDE EFF00		0x00EE CCAA		0xAACC EE00

If `EPPI_CTL.PACKEN=0`, the DMA is a 16-bit DMA and the EPPI either sign-extends or zero-fills the bytes of incoming data into a 16-bit word. The `EPPI_CTL.SWAPEN` bit has no effect if `EPPI_CTL.PACKEN=0`.

Table 19-14: 8-Bit Receive Mode with Packing Disabled

Pin Data (8 bits)	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=X SIGNEXT=0	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=X SIGNEXT=1	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=X SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=X SIGNEXT=1
0x44	0x0044	0x0044	0x0044	
0x55	0x0055	0x0055		0x0055
0x66	0x0066	0x0066	0x0066	
0x77	0x0077	0x0077		0x0077
0x88	0x0088	0xFF88	0x0088	
0x99	0x0099	0xFF99		0xFF99
0xAA	0x00AA	0xFFAA	0x00AA	
0xBB	0x00BB	0xFFBB		0xFFBB

Configuring 10/12/14-Bit Receive Modes

For 10, 12, or 14-bit non-split receive modes, the EPPI first either zero-fills or sign-extends the incoming data into a 16-bit word. The action depends on the setting of the `EPPI_CTL.SIGNEXT` bit. If `EPPI_CTL.PACKEN=1`, the EPPI then packs two of these words into one 32-bit word. Alternate even or odd samples can be skipped based on the `EPPI_CTL.SKIPEN` and `EPPI_CTL.SKIPEO` bits. The first incoming data can be placed either in the least significant bit positions or in the most significant bit positions, based on the `EPPI_CTL.SWAPEN` bit setting.

Table 19-15: 10-Bit Receive Mode with Sign Extension, with Packing Enabled

Pin Data (10 bits)	MSB	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=0 SIGNEXT=1	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=1 SIGNEXT=1	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=0 SIGNEXT=1
0x111	0			
0x222	1	0xFE22 0111	0x0111 FE22	
0x333	1			0xFF33 0111
0x044	0	0x0044 FF33	0xff33 0044	
0x155	0			
0x266	1	0xFE66 0155	0x0155 FE66	
0x377	1			0xFF77 0155
0x088	0	0x0088 FF77	0xFF77 0088	

Table 19-16: 10-Bit Receive Mode with Sign Extension, with Packing Enabled

Pin Data (10 bits)	MSB	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=0 SIGNEXT=1	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=1 SIGNEXT=1	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=1 SIGNEXT=1
0x111	0			
0x222	1			
0x333	1		0x0011 FF33	
0x044	0	0x0044 FE22		0xFE22 0044
0x155	0			
0x266	1			
0x377	1		0x0155 FF77	
0x088	0	0x0088 FE66		0xFE66 0088

Table 19-17: 10-Bit Receive Mode, with Zero-Fill, with Packing Enabled

Pin Data (10 bits)	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=0 SIGNEXT=0	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=1 SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=0 SIGNEXT=0	DMA DATA SKIP_EN=1 SKIP_EO=0 SWAPEN=0 SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=1 SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=1 SIGNEXT=0
0x111						
0x222	0x0222 0111	0x0111 0222				
0x333			0x0333 0111		0x0011 0333	
0x044	0x0044 0333	0x0333 0044		0x0044 0222		0x0222 0044
0x155						
0x266	0x0266 0155	0x0155 0266				
0x377			0x0377 0155		0x0155 0377	
0x088	0x0088 0377	0x0377 0088		0x0088 0266		0x0266 0088

The *10-bit Receive Mode with Packing Disabled* table shows a 10-bit receive mode example when `EPPI_CTL.PACKEN = 0`:

Table 19-18: 10-bit Receive Mode with Packing Disabled

Pin Data (10 bits)	MSB	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=X SIGNEXT=1	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=X SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=X SIGNEXT=1	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=X SIGNEXT=0
0x111	0	0x0111	0x0111	0x0111	
0x222	1	0xFE22	0x0222		0x0222
0x333	1	0xFF33	0x0333	0xFF33	
0x044	0	0x0044	0x0444		0x0444
0x155	0	0x0155	0x0155	0x0155	
0x266	1	0xFE66	0x0266		0x0266
0x377	1	0xFF77	0x0377	0xFF77	
0x088	0	0x0088	0x0088		0x0088

Configuring 16-Bit Receive Mode

For 16-bit non-split receive mode, if `EPPI_CTL.PACKEN = 1`, the EPPI packs two 16-bit incoming data into one 32-bit word. Alternate even or odd samples can be skipped based on the `EPPI_CTL.SKIPEN` and

EPPI_CTL.SKIPED bits. The first incoming data can be placed either in the least significant bit positions or in the most significant bit positions, based on the EPPI_CTL.SWAPEN bit setting.

Table 19-19: 16-Bit Receive Mode with Packing Enabled

Pin Data (16 bits)	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=1 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=0 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=1 SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=1 SIGNEXT=X
0x1111						
0x2222	0x2222 1111	0x1111 2222				
0x3333			0x3333 1111		0x1111 3333	
0x4444	0x4444 3333	0x3333 4444		0x4444 2222		0x2222 4444
0x5555						
0x6666	0x6666 5555	0x5555 6666				
0x7777			0x7777 5555		0x5555 7777	
0x8888	0x8888 7777	0x7777 8888		0x8888 6666		0x6666 8888

Table 19-20: 16-bit Receive Mode with Packing Disabled

Pin Data (16 bits)	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=X SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=X SIGNEXT=X	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=X SIGNEXT=X
0x1111	0x1111	0x1111	
0x2222	0x2222		0x2222
0x3333	0x3333	0x3333	
0x4444	0x4444		0x4444
0x5555	0x5555	0x5555	
0x6666	0x6666		0x6666
0x7777	0x7777	0x7777	
0x8888	0x8888		0x8888

Configuring 18-Bit Receive Mode

For 18-bit non-split receive mode, if EPPI_CTL.PACKEN =0, the EPPI zero-fills or sign-extends the incoming data into a 32-bit word. If EPPI_CTL.PACKEN =1, the EPPI first zero-fills or sign-extends the incoming data to 24 bits, and then packs four such 24-bit data words into three 32-bit words. Alternate even or odd samples can be

skipped based on the `EPPI_CTL.SKIPEN` and `EPPI_CTL.SKIPEO` bits. The `EPPI_CTL.SWAPEN` bit has no effect.

Table 19-21: 18-bit Receive Mode with Packing Disabled

Pin Data (18 bits)	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=X SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=X SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=X SIGNEXT=0
0x0 6666	0x0000 6666	0x0000 6666	
0x1 7777	0x0001 7777		0x0001 7777
0x2 8888	0x0002 8888	0x0002 8888	
0x3 9999	0x0003 9999		0x0003 9999

Table 19-22: 18-bit Receive Mode with Packing Enabled

Pin Data (18 bits)	DMA DATA SKIPEN=0 SKIPEO=X SWAPEN=X SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=1 SWAPEN=X SIGNEXT=0	DMA DATA SKIPEN=1 SKIPEO=0 SWAPEN=X SIGNEXT=0
0x0 1122			
0x1 3344	0x4400 1122		
0x2 5566	0x5566 0133	0x6600 1122	
0x3 7788	0x0377 8802		0x8801 3344
0x0 99AA		0x99AA 0255	
0x1 BBCC	0xCC00 99AA		0xBBCC 0377
0x2 DDEE	0xDDEE 01BB	0x02DD EE00	
0x3 FF12	0x03FF 122D		0x03FF 1201

Configuring 8-Bit Split Receive Mode

For 8-bit split receive mode, the `EPPI_CTL.PACKEN` and `EPPI_CTL.SIGNEXT` bits are not valid. The EPPI always packs 4 bytes of data into one 32-bit word.

Table 19-23: 8-bit Split Receive Mode with SKIPEN = 0 and SWAPEN = 0

Pin Data (8 bits)	SPLTEO=1 SUBSPLTODD= 0 SWAPEN=0 SKIPEN=0 SKIPEO=X			SPLTEO=1 SUBSPLTODD= 1 SWAPEN=0 SKIPEN=0 SKIPEO=X		
	DMACFG=1		DMACFG=0	DMACFG=1		DMACFG=0
	Primary DMA Channel	Secondary DMA Channel	Primary DMA Channel	Primary DMA Channel	Secondary DMA Channel	Primary DMA Channel
V ₀						
Y ₀						
U ₀						
Y ₁						
V ₁						
Y ₂						
U ₁		U ₁ V ₁ U ₀ V ₀	U ₁ V ₁ U ₀ V ₀			
Y ₃	Y ₃ Y ₂ Y ₁ Y ₀		Y ₃ Y ₂ Y ₁ Y ₀	Y ₃ Y ₂ Y ₁ Y ₀		Y ₃ Y ₂ Y ₁ Y ₀
V ₂						
Y ₄						
U ₂						
Y ₅						
V ₃					V ₃ V ₂ V ₁ V ₀	V ₃ V ₂ V ₁ V ₀
Y ₆						
U ₃		U ₃ V ₃ U ₂ V ₂	U ₃ V ₃ U ₂ V ₂		U ₃ U ₂ U ₁ U ₀	
Y ₇	Y ₇ Y ₆ Y ₅ Y ₄		Y ₇ Y ₆ Y ₅ Y ₄	Y ₇ Y ₆ Y ₅ Y ₄		Y ₇ Y ₆ Y ₅ Y ₄
V ₄						U ₃ U ₂ U ₁ U ₀

Table 19-24: 8-bit Split Receive Mode with SKIPEN = 0 and SWAPEN = 1

Pin Data (8 bits)	SPLTEO=1 SUBSPLTODD=0 SWAPEN=1 SKIPEN=0 SKIPEO=X			SPLTEO=1 SUBSPLTODD=1 SWAPEN=1 SKIPEN=0 SKIPEO=X		
	DMACFG=1		DMACFG=0	DMACFG=1		DMACFG=0
	PRIMARY DMA CHANNEL	SECONDARY DMA CHANNEL	PRIMARY DMA CHANNEL	PRIMARY DMA CHANNEL	SECONDARY DMA CHANNEL	PRIMARY DMA CHANNEL
V ₀						
Y ₀						
U ₀						
Y ₁						
V ₁						
Y ₂						
U ₁		V ₀ U ₀ V ₁ U ₁	V ₀ U ₀ V ₁ U ₁			
Y ₃	Y ₀ Y ₁ Y ₂ Y ₃		Y ₀ Y ₁ Y ₂ Y ₃	Y ₀ Y ₁ Y ₂ Y ₃		Y ₀ Y ₁ Y ₂ Y ₃
V ₂						
Y ₄						
U ₂						
Y ₅						
V ₃					V ₀ V ₁ V ₂ V ₃	V ₀ V ₁ V ₂ V ₃
Y ₆						
U ₃		V ₂ U ₂ V ₃ U ₃	V ₂ U ₂ V ₃ U ₃		U ₀ U ₁ U ₂ U ₃	
Y ₇	Y ₄ Y ₅ Y ₆ Y ₇		Y ₄ Y ₅ Y ₆ Y ₇	Y ₄ Y ₅ Y ₆ Y ₇		Y ₄ Y ₅ Y ₆ Y ₇
V ₄						U ₀ U ₁ U ₂ U ₃

When the bits settings are EPPI_CTL.SPLTEO = 1, EPPI_CTL.SUBSPLTODD = 1 and EPPI_CTL.DMACFG = 0, the EPPI packs the second Chroma component sent over the DMA bus completely before the Luma component. However, it is intentionally held until that previous word is moved out. This functionality allows the separation of Luma and Chroma values into individual buffers when using 2D-DMA. The second Chroma component is U₀U₁U₂U₃ in the *8-bit Split Receive Mode with SKIPEN = 0 and SWAPEN = 0* and *8-bit Split Receive Mode with SKIPEN = 0 and SWAPEN = 1* tables. The Luma component is Y₄Y₅Y₆Y₇ in the *8-bit Split Receive Mode with SKIPEN = 0 and SWAPEN = 1* table.

Configuring 10/12/14/16-Bit Split Receive Mode with SPLTWRD=0

For 16-bit split receive mode, the `EPPI_CTL.PACKEN` bit is not valid. The EPPI always packs two 16-bit words into one 32-bit word. For 10, 12, or 14-bit split receive modes, the EPPI first either sign-extends or zero-fills the incoming data into a 16-bit word. The EPPI then packs two of these words into one 32-bit word to send to the DMA.

Table 19-25: 16-bit Split Receive Mode with `SPLTWRD = 0`, `SKIPEN = 0` and `SWAPEN = 0`

Pin Data (16 bits)	SPLTEO=1 SUBSPLTODD=0 SWAPEN=0 SKIPEN=0 SKIPEO=X			SPLTEO=1 SUBSPLTODD=1 SWAPEN=0 SKIPEN=0 SKIPEO=X		
	DMACFG=1		DMACFG=0	DMACFG=1		DMACFG=0
	Primary DMA Channel	Secondary DMA Channel	Primary DMA Channel	Primary DMA Channel	Secondary DMA Channel	Primary DMA Channel
V ₀						
Y ₀						
U ₀		U ₀ V ₀	U ₀ V ₀			
Y ₁	Y ₁ Y ₀		Y ₁ Y ₀	Y ₁ Y ₀		Y ₁ Y ₀
V ₁					V ₁ V ₀	V ₁ V ₀
Y ₂						
U ₁		U ₁ V ₁	U ₁ V ₁		U ₁ U ₀	
Y ₃	Y ₃ Y ₂		Y ₃ Y ₂	Y ₃ Y ₂		Y ₃ Y ₂
V ₂						U ₁ U ₀

Table 19-26: 16-bit Split Receive Mode with SPLITWRD = 0, SKIPEN = 0 and SWAPEN = 1

Pin Data (16 bits)	SPLTEO=1 SUBSPLTODD=0 SWAPEN=1 SKIPEN=0 SKIPEO=X			SPLTEO=1 SUBSPLTODD=1 SWAPEN=1 SKIPEN=0 SKIPEO=X		
	DMACFG=1		DMACFG=0	DMACFG=1		DMACFG=0
	PRIMARY DMA CHANNEL	SECONDARY DMA CHANNEL	PRIMARY DMA CHANNEL	PRIMARY DMA CHANNEL	SECONDARY DMA CHANNEL	PRIMARY DMA CHANNEL
V ₀						
Y ₀						
U ₀		V ₀ U ₀	V ₀ U ₀			
Y ₁	Y ₀ Y ₁		Y ₀ Y ₁	Y ₀ Y ₁		Y ₀ Y ₁
V ₁					V ₀ V ₁	V ₀ V ₁
Y ₂						
U ₁		V ₁ U ₁	V ₁ U ₁		U ₀ U ₁	
Y ₃	Y ₂ Y ₃		Y ₂ Y ₃	Y ₂ Y ₃		Y ₂ Y ₃
V ₂						U ₀ U ₁

Configuring 16-Bit Split Receive Mode with SPLITWRD=1

For 16-bit split receive mode, the EPPI_CTL.PACKEN bit is not valid. The EPPI always packs two 16-bit words into one 32-bit word. The EPPI_CTL.SPLITWRD bit is only valid when the EPPI_CTL.DLEN bit =16 bits.

Table 19-27: 16-bit Split Receive Mode with SPLITWRD = 1, SKIPEN = 0 and SWAPEN = 0

Pin Data (16 bits)	SPLTEO=1 SUBSPLTODD=0 SWAPEN=0 SKIPEN=0 SKIPEO=X			SPLT_EVEN_ODD=1 SUBSPLTODD=1 SWAPEN=0 SKIPEN=0 SKIPEO=X		
	DMACFG=1		DMACFG=0	DMACFG=1		DMACFG=0
	Primary DMA Channel	Secondary DMA Channel	Primary DMA Channel	Primary DMA Channel	Secondary DMA Channel	Primary DMA Channel
V ₀ Y ₀						

Table 19-27: 16-bit Split Receive Mode with SPLTWDR = 1, SKIPEN = 0 and SWAPEN = 0 (Continued)

Pin Data (16 bits)	SPLTEO=1 SUBSPLTODD=0 SWAPEN=0 SKIPEN=0 SKIPEO=X			SPLT_EVEN_ODD=1 SUBSPLTODD=1 SWAPEN=0 SKIPEN=0 SKIPEO=X		
	DMACFG=1		DMACFG=0	DMACFG=1		DMACFG=0
	Primary DMA Channel	Secondary DMA Channel	Primary DMA Channel	Primary DMA Channel	Secondary DMA Channel	Primary DMA Channel
U ₀ Y ₁						
V ₁ Y ₂						
U ₁ Y ₃	Y ₃ Y ₂ Y ₁ Y ₀	U ₁ V ₁ U ₀ V ₀	Y ₃ Y ₂ Y ₁ Y ₀	Y ₃ Y ₂ Y ₁ Y ₀		Y ₃ Y ₂ Y ₁ Y ₀
V ₂ Y ₄			U ₁ V ₁ U ₀ V ₀			
U ₂ Y ₅						
V ₃ Y ₆					V ₃ V ₂ V ₁ V ₀	V ₃ V ₂ V ₁ V ₀
U ₃ Y ₇	Y ₇ Y ₆ Y ₅ Y ₄	U ₃ V ₃ U ₂ V ₂	Y ₇ Y ₆ Y ₅ Y ₄	Y ₇ Y ₆ Y ₅ Y ₄	U ₃ U ₂ U ₁ U ₀	Y ₇ Y ₆ Y ₅ Y ₄
V ₄ Y ₈			U ₃ V ₃ U ₂ V ₂			U ₃ U ₂ U ₁ U ₀

Configuring 8-Bit Transmit Mode

For 8-bit non-split transmit mode, if the EPPI_CTL.PACKEN bit =1, the DMA is a 32-bit DMA and the EPPI unpacks the 32-bit word from memory into 4 bytes to transmit. The EPPI transmits either the MSBs or the LSBs as the first data, depending on the EPPI_CTL.SWAPEN bit setting. If EPPI_CTL.PACKEN =0, the DMA is a 16-bit DMA and the EPPI transmits the lower 8 bits. The EPPI_CTL.SWAPEN bit has no effect when EPPI_CTL.PACKEN =0.

Table 19-28: 8-bit Transmit Mode with Packing Enabled

DMA Data (32 bits)	Pin Data when SWAPEN=0	Pin Data when SWAPEN=1
0x11223344	0x44	0x11
0x55667788	0x33	0x22
	0x22	0x33
	0x11	0x44
	0x88	0x55
	0x77	0x66
	0x66	0x77

Table 19-28: 8-bit Transmit Mode with Packing Enabled (Continued)

DMA Data (32 bits)	Pin Data when SWAPEN=0	Pin Data when SWAPEN=1
	0x55	0x88

Table 19-29: Data Sent in 8-bit Transmit Mode with Packing Disabled

DMA Data (16 bits)	Pin Data SWAPEN=X
0x1234	0x34
0x2345	0x45
0x3456	0x56

Configuring 10/12/14-Bit Transmit Modes

For 10, 12, or 14-bit non-split transmit modes, if the `EPPI_CTL.PACKEN` bit =1, the DMA is a 32-bit DMA. The EPPI unpacks the 32-bit word from memory into two 16-bit data words. The EPPI then transmits the required LSBs from each data word. The EPPI transmits either the most significant word or the least significant word as the first data, depending on the `EPPI_CTL.SWAPEN` bit setting. If `EPPI_CTL.PACKEN` =0, the DMA is a 16-bit DMA and the EPPI transmits the required LSBs. The `EPPI_CTL.SWAPEN` bit has no effect when the `EPPI_CTL.PACKEN` bit =0.

Table 19-30: 10-bit Transmit Mode with Packing Enabled

DMA Data (32 bits)	Pin Data when SWAPEN=0	Pin Data when SWAPEN=1
0x1111 2222	0x222	0x111
0x3333 4444	0x111	0x222
	0x044	0x333
	0x333	0x044

Table 19-31: 10-bit Transmit Mode with Packing Disabled

DMA Data (16 bits)	Pin Data SWAPEN=X
0x1234	0x234
0x2345	0x345
0x3456	0x056
0x4567	0x167

Configuring 16-Bit Transmit Mode

For 16-bit non-split transmit mode, if the `EPPI_CTL.PACKEN` bit =1, the DMA is a 32-bit DMA. The EPPI unpacks the 32-bit word from memory into two 16-bit data words to transmit. The EPPI transmits either the MSBs or the LSBs as the first data, depending on the `EPPI_CTL.SWAPEN` bit setting. If the `EPPI_CTL.PACKEN` bit =0, the DMA is a 16-bit DMA and the EPPI transmits the data as is. The `EPPI_CTL.SWAPEN` has no effect when `EPPI_CTL.PACKEN` bit =0.

Table 19-32: 16-bit Transmit Mode with Packing Enabled

DMA Data (32 bits)	Pin Data when SWAPEN=0	Pin Data when SWAPEN=1
0x1111 2222	0x2222	0x1111
0x3333 4444	0x1111	0x2222
	0x4444	0x3333
	0x3333	0x4444

Table 19-33: 16-bit Transmit Mode with Packing Disabled

DMA Data (16 bits)	Pin Data SWAPEN=X
0x1234	0x1234
0x2345	0x2345
0x3456	0x3456

Configuring 18-Bit Transmit Mode

For 18-bit transmit mode, if the `EPPI_CTL.PACKEN` bit =1, the DMA is a 32-bit DMA and the EPPI unpacks the 32-bit word from memory. In this mode, when `EPPI_CTL.RGBFMTEN` is set, the least significant 2 bits of R, G, and B are dropped.

Table 19-34: 18-bit Transmit Mode with Packing Enabled

DMA Data	Pin Data (18-bits)	
	RGBFMTEN=0	RGBFMTEN=1
0x0123 4567	0x3 4567	0x0 8459
0x89AB CDEF	0x1 EF01	0x3 3EC0
0x0123 4567	0x3 89AB	0x1 98AA
	0x1 2345	0x0 0211

Table 19-35: 18-bit Transmit Mode with Packing Disabled

DMA Data	Pin Data (18-bits)	
	RGBFMTEN=0	RGBFMTEN=1
0x0123 4567	0x3 4567	0x0 8459
0x89AB CDEF	0x3 CDEF	0x2 ACFB
0x0123 4567	0x3 4567	0x0 8459

Configuring 8-Bit Split Transmit Mode

For 8-bit split transmit mode, the `EPPI_CTL.PACKEN` bit is not valid. The EPPI always unpacks the 32-bit DMA data into 4 bytes to transmit.

Table 19-36: 8-bit Split Transmit Mode with SPLTEO=1, SUBSPLTODD=0 and SWAPEN=0

DMACFG=1			DMACFG=0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (8 bits)	DMA0 DATA (32 bits)	Pin Data (8 bits)
Y ₃ Y ₂ Y ₁ Y ₀	U ₁ V ₁ U ₀ V ₀	V ₀	U ₁ V ₁ U ₀ V ₀	V ₀
Y ₇ Y ₆ Y ₅ Y ₄	U ₃ V ₃ U ₂ V ₂	Y ₀	Y ₃ Y ₂ Y ₁ Y ₀	Y ₀
		U ₀	U ₃ V ₃ U ₂ V ₂	U ₀
		Y ₁	Y ₇ Y ₆ Y ₅ Y ₄	Y ₁
		V ₁		V ₁
		Y ₂		Y ₂
		U ₁		U ₁
		Y ₃		Y ₃
		V ₂		V ₂
		Y ₄		Y ₄
		U ₂		U ₂
		Y ₅		Y ₅
		V ₃		V ₃
		Y ₆		Y ₆
		U ₃		U ₃
		Y ₇		Y ₇

Table 19-37: 8-bit Split Transmit Mode with SPLTEO=1, SUBSPLTODD=1 and SWAPEN=0

DMACFG=1			DMACFG=0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (8 bits)	DMA0 DATA (32 bits)	Pin Data (8 bits)
Y ₃ Y ₂ Y ₁ Y ₀	V ₃ V ₂ V ₁ V ₀	V ₀	V ₃ V ₂ V ₁ V ₀	V ₀
Y ₇ Y ₆ Y ₅ Y ₄	U ₃ U ₂ U ₁ U ₀	Y ₀	Y ₃ Y ₂ Y ₁ Y ₀	Y ₀
	V ₇ V ₆ V ₅ V ₄	U ₀	U ₃ U ₂ U ₁ U ₀	U ₀
	U ₇ U ₆ U ₅ U ₄	Y ₁	Y ₇ Y ₆ Y ₅ Y ₄	Y ₁
		V ₁		V ₁
		Y ₂		Y ₂
		U ₁		U ₁
		Y ₃		Y ₃
		V ₂		V ₂

Table 19-37: 8-bit Split Transmit Mode with SPLTEO=1, SUBSPLTODD=1 and SWAPEN=0 (Continued)

DMACFG=1			DMACFG=0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (8 bits)	DMA0 DATA (32 bits)	Pin Data (8 bits)
		Y ₄		Y ₄
		U ₂		U ₂
		Y ₅		Y ₅
		V ₃		V ₃
		Y ₆		Y ₆
		U ₃		U ₃
		Y ₇		Y ₇

Table 19-38: 8-bit Split Transmit Mode with SPLTEO=1, SUBSPLTODD=0 and SWAPEN=1

DMACFG=1			DMACFG=0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (8 bits)	DMA0 DATA (32 bits)	Pin Data (8 bits)
Y ₃ Y ₂ Y ₁ Y ₀	U ₁ V ₁ U ₀ V ₀	U ₁	U ₁ V ₁ U ₀ V ₀	U ₁
Y ₇ Y ₆ Y ₅ Y ₄	U ₃ V ₃ U ₂ V ₂	Y ₃	Y ₃ Y ₂ Y ₁ Y ₀	Y ₃
		V ₁	U ₃ V ₃ U ₂ V ₂	V ₁
		Y ₂	Y ₇ Y ₆ Y ₅ Y ₄	Y ₂
		U ₀		U ₀
		Y ₁		Y ₁
		V ₀		V ₀
		Y ₀		Y ₀
		U ₃		U ₃
		Y ₇		Y ₇
		V ₃		V ₃
		Y ₆		Y ₆
		U ₂		U ₂
		Y ₅		Y ₅
		V ₂		V ₃
		Y ₄		Y ₄

Table 19-39: 8-bit Split Transmit Mode with SPLTEO=1, SUBSPLTODD=1, and SWAPEN=1

DMACFG=1			DMACFG=0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (8 bits)	DMA0 DATA (32 bits)	Pin Data (8 bits)
Y ₃ Y ₂ Y ₁ Y ₀	V ₃ V ₂ V ₁ V ₀	V ₃	V ₃ V ₂ V ₁ V ₀	V ₃
Y ₇ Y ₆ Y ₅ Y ₄	U ₃ U ₂ U ₁ U ₀	Y ₃	Y ₃ Y ₂ Y ₁ Y ₀	Y ₃
	V ₇ V ₆ V ₅ V ₄	U ₃	U ₃ V ₃ U ₂ V ₂	U ₃
	U ₇ U ₆ U ₅ U ₄	Y ₂	Y ₇ Y ₆ Y ₅ Y ₄	Y ₂
		V ₂		V ₂
		Y ₁		Y ₁
		U ₂		U ₂
		Y ₀		Y ₀
		V ₁		V ₁
		Y ₇		Y ₇
		U ₁		U ₁
		Y ₆		Y ₆
		V ₀		V ₀
		Y ₅		Y ₅
		U ₀		U ₀
		Y ₄		Y ₄

Configuring 10/12/14/16-Bit Transmit Mode with SPLTWRD=0

For 16-bit split transmit mode, the EPPI_CTL.PACKEN bit is not valid. The EPPI always unpacks the 32-bit DMA data into two 16-bit words to transmit. For 10, 12, or 14-bit split transmit modes, the EPPI first unpacks the data in the same way as for 16-bit transmit mode. But, the EPPI transmits only the required number of LSBs.

Table 19-40: 16-bit Split Transmit Mode with SPLTEO = 1, SUBSPLTODD = 0, and SWAPEN = 0

DMACFG = 1			DMACFG = 0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (16 bits)	DMA0 DATA (32 bits)	Pin Data (16 bits)
Y ₁ Y ₀	U ₀ V ₀	V ₀	U ₀ V ₀	V ₀
Y ₃ Y ₂	U ₁ V ₁	Y ₀	Y ₁ Y ₀	Y ₀
		U ₀	U ₁ V ₁	U ₀
		Y ₁	Y ₃ Y ₂	Y ₁
		V ₁		V ₁

Table 19-40: 16-bit Split Transmit Mode with SPLTEO = 1, SUBSPLTODD = 0, and SWAPEN = 0 (Continued)

DMACFG = 1			DMACFG = 0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (16 bits)	DMA0 DATA (32 bits)	Pin Data (16 bits)
		Y ₂		Y ₂
		U ₁		U ₁
		Y ₃		Y ₃

Table 19-41: 16-bit Split Transmit Mode with SPLTEO = 1, SUBSPLTODD = 1, and SWAPEN = 0

DMACFG = 1			DMACFG = 0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (16 bits)	DMA0 DATA (32 bits)	Pin Data (16 bits)
Y ₁ Y ₀	V ₁ V ₀	V ₀	V ₁ V ₀	V ₀
Y ₃ Y ₂	U ₁ U ₀	Y ₀	Y ₁ Y ₀	Y ₀
	V ₃ V ₂	U ₀	U ₁ U ₀	U ₀
	U ₃ U ₂	Y ₁	Y ₃ Y ₂	Y ₁
		V ₁		V ₁
		Y ₂		Y ₂
		U ₁		U ₁
		Y ₃		Y ₃

Table 19-42: 16-bit Split Transmit Mode with SPLTEO = 1, SUBSPLTODD = 0, and SWAPEN = 1

DMACFG = 1			DMACFG = 0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (16 bits)	DMA0 DATA (32 bits)	Pin Data (16 bits)
Y ₁ Y ₀	V ₀ U ₀	V ₀	V ₀ U ₀	V ₀
Y ₃ Y ₂	V ₁ U ₁	Y ₁	Y ₁ Y ₀	Y ₁
		U ₀	V ₁ U ₁	U ₀
		Y ₀	Y ₃ Y ₂	Y ₀
		V ₁		V ₁
		Y ₃		Y ₃
		U ₁		U ₁
		Y ₂		Y ₂

Table 19-43: 16-bit Split Transmit Mode with SPLTEO = 1, SUBSPLTODD = 1, and SWAPEN = 1

DMACFG = 1			DMACFG = 0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (16 bits)	DMA0 DATA (32 bits)	Pin Data (16 bits)
Y ₁ Y ₀	V ₁ V ₀	V ₁	V ₁ V ₀	V ₁
Y ₃ Y ₂	U ₁ U ₀	Y ₁	Y ₁ Y ₀	Y ₁
	V ₃ V ₂	U ₁	U ₁ U ₀	U ₁
	U ₃ U ₂	Y ₀		Y ₀
		V ₀		V ₀
		Y ₃		Y ₁
		U ₀		U ₀
		Y ₂		Y ₂

Configuring 16-Bit Split Transmit Mode with SPLTWRD=1

For 16-bit split transmit mode, the EPPI_CTL.PACKEN bit is not valid. The EPPI always unpacks the 32-bit DMA data into two 16-bit words to transmit. The EPPI_CTL.SPLTWRD bit is only valid when the EPPI_CTL.DLEN bit =16 bits.

Table 19-44: 16-bit Split Transmit Mode with SPLTWRD = 1, SUBSPLTODD = 0, and SWAPEN = 0

DMACFG = 1			DMACFG = 0	
DMA0 DATA (32 bits)	DMA1 DATA (32 bits)	Pin Data (16 bits)	DMA0 DATA (32 bits)	Pin Data (16 bits)
Y ₃ Y ₂ Y ₁ Y ₀	U ₁ V ₁ U ₀ V ₀	V ₀ Y ₀	U ₁ V ₁ U ₀ V ₀	V ₀ Y ₀
Y ₇ Y ₆ Y ₅ Y ₄	U ₃ V ₃ U ₂ V ₂	U ₀ Y ₁	Y ₃ Y ₂ Y ₁ Y ₀	U ₀ Y ₁
		V ₁ Y ₂	U ₃ V ₃ U ₂ V ₂	V ₁ Y ₂
		U ₁ Y ₃	Y ₇ Y ₆ Y ₅ Y ₄	U ₁ Y ₃
		V ₂ Y ₄		V ₂ Y ₄
		U ₂ Y ₅		U ₂ Y ₅
		V ₃ Y ₆		V ₃ Y ₆
		U ₃ Y ₇		U ₃ Y ₇

Table 19-45: 16-bit Split Transmit Mode with SPLTWRD = 1, SUBSPLTODD = 1, and SWAPEN = 0

DMACFG = 1			DMACFG = 0	
PRIMARY DMA DATA (32 bits)	SECONDARY DMA DATA (32 bits)	Pin Data (16 bits)	DMA0 DATA (32 bits)	Pin Data (16 bits)
Y ₃ Y ₂ Y ₁ Y ₀	V ₃ V ₂ V ₁ V ₀	V ₀ Y ₀	V ₃ V ₂ V ₁ V ₀	V ₀ Y ₀

Table 19-45: 16-bit Split Transmit Mode with SPLTWDR = 1, SUBSPLTODD = 1, and SWAPEN = 0 (Continued)

DMACFG = 1			DMACFG = 0	
PRIMARY DMA DATA (32 bits)	SECONDARY DMA DATA (32 bits)	Pin Data (16 bits)	DMA0 DATA (32 bits)	Pin Data (16 bits)
Y ₇ Y ₆ Y ₅ Y ₄	U ₃ U ₂ U ₁ U ₀	U ₀ Y ₁	Y ₃ Y ₂ Y ₁ Y ₀	U ₀ Y ₁
	V ₇ V ₆ V ₅ V ₄	V ₁ Y ₂	U ₃ U ₂ U ₁ U ₀	V ₁ Y ₂
	U ₇ U ₆ U ₅ U ₄	U ₁ Y ₃	Y ₇ Y ₆ Y ₅ Y ₄	U ₁ Y ₃
		V ₂ Y ₄		V ₂ Y ₄
		U ₂ Y ₅		U ₂ Y ₅
		V ₃ Y ₆		V ₃ Y ₆
		U ₃ Y ₇		U ₃ Y ₇

EPPI Programming Concepts

This section provides information on SMPTE programming.

SMPTE Modes Programming

The programming model of SMPTE modes is similar to ITU Modes. All programming modes pertaining to ITU modes like XFRTYPE, FSCFG, FLDSEL, and BLANKGEN hold true for SMPTE modes as well. The only difference is that since ITU modes use Y-Cr/Cb interleaved data and SMPTE use parallel Y-Cr/Cb data, SPLTWDR could be set while operating in SMPTE modes. The *Programming Modes for SMPTE Formats* table describes the programming modes for different SMPTE formats.

Table 19-46: Programming Modes for SMPTE Formats

SMPTE Format	SMPTE Channel Width	EPPI Input Bit Width	EPPI Mode	Remarks
296M	8	16 Cr/Cb - [15:8] Y - [7:0]	DLEN = 16 bits SPLTWDR = 1	SIGNEXT not supported
	8	16 Cr/Cb - [15:8] Y - [7:0]	DLEN = 16 bits SPLTWDR = 1	SIGNEXT not supported

ADSP-2159x EPPI Register Descriptions

Enhanced Parallel Peripheral Interface (EPPI) contains the following registers.

Table 19-47: ADSP-2159x EPPI Register List

Name	Description
EPPI_CLKDIV	Clock Divide Register
EPPI_CTL	Control Register
EPPI_CTL2	Control Register 2 Register
EPPI_EVENCLIP	Clipping Register for EVEN (Luma) Data Register
EPPI_FRAME	Lines Per Frame Register
EPPI_FS1_DLY	Frame Sync 1 Delay Value Register
EPPI_FS1_PASPL	FS1 Period Register / EPPI Active Samples Per Line Register
EPPI_FS1_WLHB	FS1 Width Register / EPPI Horizontal Blanking Samples Per Line Register
EPPI_FS2_DLY	Frame Sync 2 Delay Value Register
EPPI_FS2_PALPF	FS2 Period Register / EPPI Active Lines Per Field Register
EPPI_FS2_WLVB	FS2 Width Register / EPPI Lines Of Vertical Blanking Register
EPPI_HCNT	Horizontal Transfer Count Register
EPPI_HDLY	Horizontal Delay Count Register
EPPI_IMSK	Interrupt Mask Register
EPPI_LINE	Samples Per Line Register
EPPI_ODDCLIP	Clipping Register for ODD (Chroma) Data Register
EPPI_STAT	Status Register
EPPI_VCNT	Vertical Transfer Count Register
EPPI_VDLY	Vertical Delay Count Register

Clock Divide Register

The `EPPI_CLKDIV` register provides the divisor for EPPI internal clock generation. The generated clock frequency is given by following formula:

$$\text{EPPI_CLK} = (\text{SCLK0}) / (\text{EPPI_CLKDIV} + 1)$$

Note that a value of 0xFFFF is invalid for the `EPPI_CLKDIV` register.

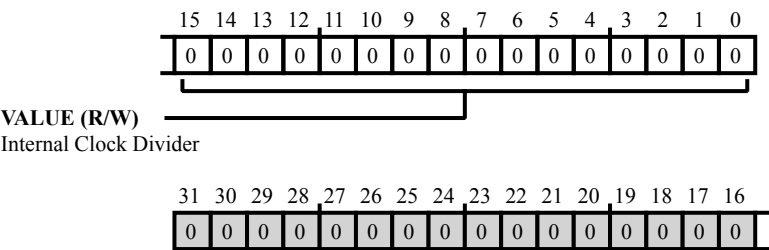


Figure 19-11: EPPI_CLKDIV Register Diagram

Table 19-48: EPPI_CLKDIV Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Internal Clock Divider.

Control Register

The `EPPI_CTL` register configures the EPPI for operating mode, control signal polarities, and data width of the port.

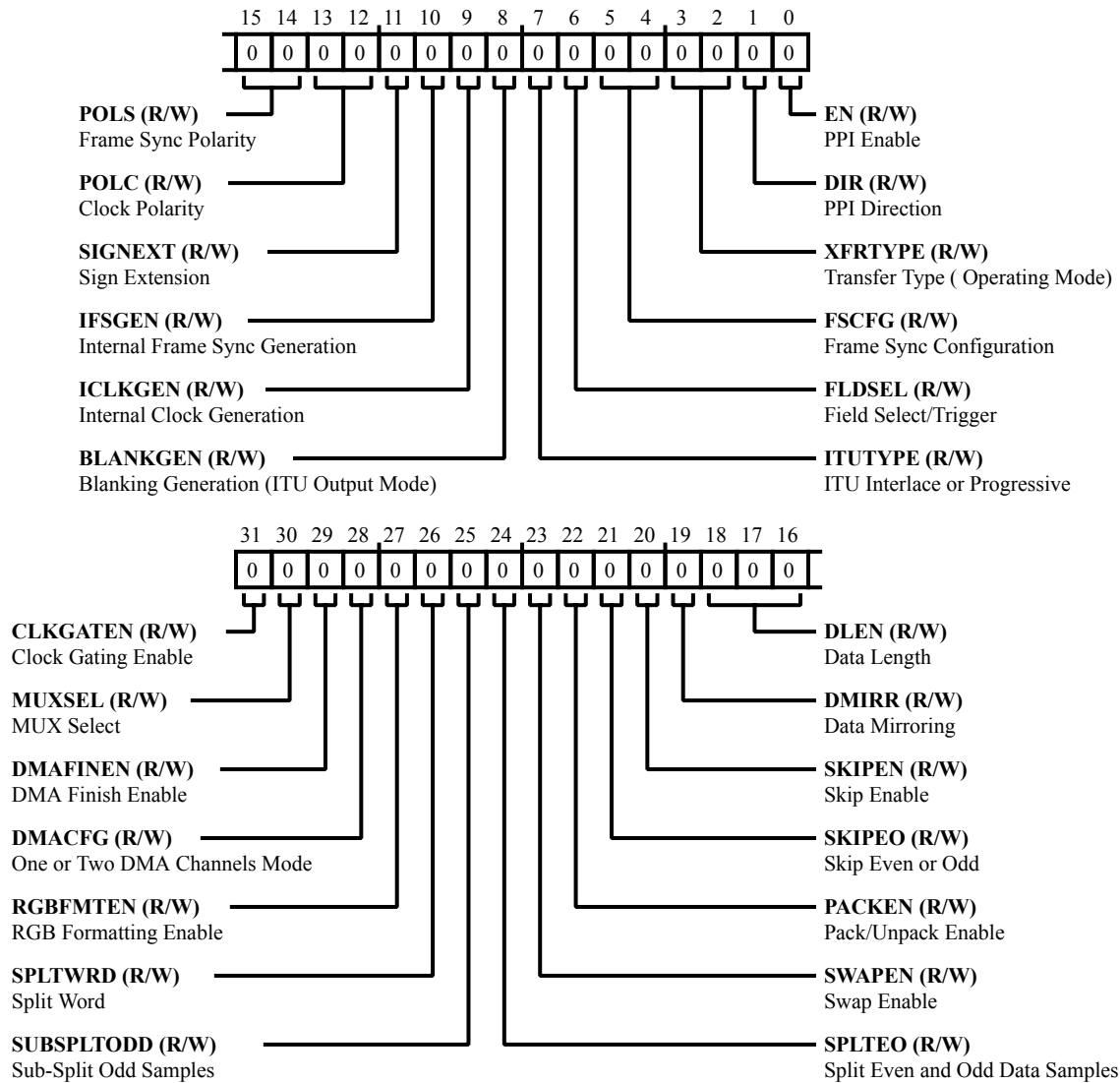


Figure 19-12: EPPI_CTL Register Diagram

Table 19-49: EPPI_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKGATEN	Clock Gating Enable. The EPPI_CTL.CLKGATEN bit enables using the EPPI_FS3 pin as a clock gating pin. When EPPI_CTL.CLKGATEN is set, the EPPI_FS3 pin acts as a clock gating signal, and both the internal and external clock are gated. Note that the EPPI_FS3 pin gating signal is active low, and the EPPI_CTL.CLKGATEN selection is ignored if EPPI_CTL.MUXSEL is set or EPPI_CTL.FSCFG equals 0x3.
		0 Disable
		1 Enable
30 (R/W)	MUXSEL	MUX Select. The EPPI_CTL.MUXSEL bit enables multiplexing of a primary and alternate camera using the EPPI main data and clock lines. For more information on this feature, see the EPPI functional description.
		0 Normal Operation
		1 Multiplexed Operation
29 (R/W)	DMAFINEN	DMA Finish Enable. The EPPI_CTL.DMAFINEN bit selects whether or not the EPPI sends a finish command (010) through the DDE COMMAND line soon after a frame/line is received completely.
		0 No Finish Command
		1 Enable Send Finish Command
28 (R/W)	DMACFG	One or Two DMA Channels Mode. The EPPI_CTL.DMACFG bit is valid only if EPPI_CTL.SPLTEO is set. If EPPI_CTL.DMACFG is set, the EPPI uses two DMA channels. And, if EPPI_CTL.DMACFG is cleared, the EPPI uses only one DMA channel.
		0 PPI Uses One DMA Channel
		1 PPI Uses Two DMA Channels
27 (R/W)	RGBFMTEN	RGB Formatting Enable. For 16- or 18-bit transmit modes only, the EPPI_CTL.RGBFMTEN bit enables conversion of RGB888 from memory into RGB666 output data (18-bit transmit) or enables conversion of RGB888 from memory into RGB565 output data (16-bit transmit). Note that EPPI_CTL.SPLTEO and EPPI_CTL.RGBFMTEN should never be set simultaneously.
		0 Disable RGB Formatted Output
		1 Enable RGB Formatted Output

Table 19-49: EPPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration												
26 (R/W)	SPLTWRD	Split Word. The EPPI_CTL.SPLTWRD bit selects split word data placement when the data length (EPPI_CTL.DLEN) selects 16-, 20-, or 24-bit data words. For all other EPPI_CTL.SPLTWRD values, the set or clear selections for EPPI_CTL.SPLTWRD produce the same result (act as though EPPI_CTL.SPLTWRD is cleared). For EPPI_CTL.SPLTWRD set, the EPPI_CTL.DLEN values below result in the following combinations of split words: <table><tr><td>DLEN</td><td>Cr/Cb data</td><td>Y data</td></tr><tr><td>16</td><td>PPI_DATA[15:8]</td><td>PPI_DATA[7:0]</td></tr><tr><td>20</td><td>PPI_DATA[19:10]</td><td>PPI_DATA[9:0]</td></tr><tr><td>24</td><td>PPI_DATA[23:12]</td><td>PPI_DATA[11:0]</td></tr></table>	DLEN	Cr/Cb data	Y data	16	PPI_DATA[15:8]	PPI_DATA[7:0]	20	PPI_DATA[19:10]	PPI_DATA[9:0]	24	PPI_DATA[23:12]	PPI_DATA[11:0]
		DLEN	Cr/Cb data	Y data										
		16	PPI_DATA[15:8]	PPI_DATA[7:0]										
		20	PPI_DATA[19:10]	PPI_DATA[9:0]										
24	PPI_DATA[23:12]	PPI_DATA[11:0]												
0	PPI_DATA has (DLEN-1) bits of Y or Cr or Cb													
1	PPI_DATA Contains 2 Elements per Word													
25 (R/W)	SUBSPLTODD	Sub-Split Odd Samples. The EPPI_CTL.SUBSPLTODD bit is valid only if EPPI_CTL.SPLTEO is set. If EPPI_CTL.SUBSPLTODD is set, the EPPI sub-splits the odd sub-stream, and packs them separately.												
		0	Disable											
		1	Enable											
24 (R/W)	SPLTEO	Split Even and Odd Data Samples. If EPPI_CTL.SPLTEO is set, the EPPI splits the incoming data stream into two sub-streams, an even stream and an odd stream, and packs them separately.												
		0	Do Not Split Samples											
		1	Split Even/Odd Samples											
23 (R/W)	SWAPEN	Swap Enable. The EPPI_CTL.SWAPEN selects whether or not to swap the order of the first data (most-significant bits versus least-significant bits) of the DMA word. For receive modes, the EPPI puts the first data in the most significant bits (if set) or puts the first data in the least significant bits (if cleared) of the DMA word. For transmit modes, the EPPI transmits the most significant bits in the DMA word as the first data (if set) or transmits the least significant bits in the DMA word as the first data (if cleared).												
		0	Disable											
		1	Enable											

Table 19-49: EPPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration																								
22 (R/W)	PACKEN	Pack/Unpack Enable. The EPPI_CTL . PACKEN select whether or not packing is enabled (for receive modes) and unpacking is enabled (for transmit modes). When this bit is set, EPPI transfer DMA is 32-bits wide. When this bit is cleared and the EPPI_CTL . DLEN is less than or equal to 16 bits, EPPI transfer DMA is 16-bits wide. For receive modes, if this bit is set, then the EPPI packs the incoming data into 32-bit words. If this bit is cleared, then the EPPI does not do any packing. For transmit modes, if this bit is set, then the EPPI always unpacks the 32-bit data from DMA. If this bit is not set, the EPPI does not do any unpacking.																								
		0 Disable																								
		1 Enable																								
21 (R/W)	SKIPEO	Skip Even or Odd. The EPPI_CTL . SKIPEO bit selects whether even (if set) or odd (if cleared) samples are skipped if sample skipping is enabled (EPPI_CTL . SKIPEN is set). This feature only is useful for receive modes.																								
		0 Skip Odd Samples																								
		1 Skip Even Samples																								
20 (R/W)	SKIPEN	Skip Enable. The EPPI_CTL . SKIPEN bit enables skipping alternate samples. This feature only is useful for receive modes.																								
		0 No Samples Skipping																								
		1 Skip Alternate Samples																								
19 (R/W)	DMIRR	Data Mirroring. The EPPI_CTL . DMIRR field enables mirroring (bit reversing) of the data coming in or going out on the EPPI data pins. <table><tr><td>Pin</td><td>PPI Data</td><td>PPI Data</td></tr><tr><td>Data</td><td>(DAT_MRR=0)</td><td>(DAT_MRR=1)</td></tr><tr><td colspan="3">-----</td></tr><tr><td>15</td><td>15</td><td>0</td></tr><tr><td>14</td><td>14</td><td>1</td></tr><tr><td>....</td><td>....</td><td>....</td></tr><tr><td>1</td><td>1</td><td>14</td></tr><tr><td>0</td><td>0</td><td>15</td></tr></table>	Pin	PPI Data	PPI Data	Data	(DAT_MRR=0)	(DAT_MRR=1)	-----			15	15	0	14	14	1				1	1	14	0	0	15
		Pin	PPI Data	PPI Data																						
		Data	(DAT_MRR=0)	(DAT_MRR=1)																						

15	15	0																								
14	14	1																								
....																										
1	1	14																								
0	0	15																								
0 No Data Mirroring																										
1 Data Mirroring																										
18:16 (R/W)	DLEN	Data Length. The EPPI_CTL . DLEN bits select the data length for the EPPI. Note that the 20 bits data length selection is valid only for SMPTE modes (EPPI_CTL . SPLTWRD set).																								

Table 19-49: EPPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		0	8 bits
		1	10 bits
		2	12 bits
		3	14 bits
		4	16 bits
		5	18 bits
		6	20 bits
		7	24 bits
15:14 (R/W)	POLS	Frame Sync Polarity. The EPPI_CTL.POLS selects whether the frame syncs' polarity is active low versus active high.	
		0	FS1 and FS2 are active high
		1	FS1 is active low. FS2 is active high
		2	FS1 is active high. FS2 is active low
		3	FS1 and FS2 are active low
13:12 (R/W)	POLC	Clock Polarity. The EPPI_CTL.POLC selects the rising versus falling edge for sampling data and sampling/driving syncs.	
		0	Clock/Sync Polarity Mode 0. For receive mode: Sample data on falling edge and sample/drive syncs on falling edge. For transmit mode: Drive data on rising edge and sample/drive syncs on rising edge.
		1	Clock/Sync Polarity Mode 1. For receive mode: Sample data on falling edge and sample/drive syncs on rising edge. For transmit mode: Drive data on rising edge and sample/drive syncs on falling edge.
		2	Clock/Sync Polarity Mode 2. For receive mode: Sample data on rising edge and sample/drive syncs on falling edge. For transmit mode: Drive data on falling edge and sample/drive syncs on rising edge.
		3	Clock/Sync Polarity Mode 3. For receive mode: Sample data on rising edge and sample/drive syncs on rising edge. For transmit mode: Drive data on falling edge and sample/drive syncs on falling edge.

Table 19-49: EPPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	SIGNEXT	Sign Extension. The EPPI_CTL.SIGNEXT select whether (for receive modes when EPPI_CTL.DLEN selecting 16 bit data length) the data is sign extended or zero filled. Not that EPPI_CTL.SPLTWRD is removed from this shared bit.
		0 Zero Filled
		1 Sign Extended
10 (R/W)	IFSGEN	Internal Frame Sync Generation. The EPPI_CTL.IFSGEN bit selects whether the frame syncs are generated internally or are supplied by an external device.
		0 External Frame Sync
		1 Internal Frame Sync
9 (R/W)	ICLKGEN	Internal Clock Generation. The EPPI_CTL.ICLKGEN bit selects whether the EPPI_CLK is generated internally or is supplied by an external device.
		0 External Clock
		1 Internal Clock
8 (R/W)	BLANKGEN	Blanking Generation (ITU Output Mode). The EPPI_CTL.BLANKGEN enables ITU output with internal blanking. In GP 8, 10 transmit bit modes (when EPPI_CTL.SPLTWRD is cleared) and 16-, 20-, and 24-bit transmit modes (when EPPI_CTL.SPLTWRD is set), EPPI_CTL.BLANKGEN selects whether or not the EPPI generates blanking and generates preamble and insertion with active data from memory.
		0 Disable
		1 Enable
7 (R/W)	ITUTYPE	ITU Interlace or Progressive. The EPPI_CTL.ITUTYPE selects interlaced or progressive operation for ITU656 mode. This selection is valid for both TX and RX modes.
		0 Interlaced
		1 Progressive

Table 19-49: EPPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W)	FLDSEL	Field Select/Trigger. The EPPI_CTL.FLDSEL bits configure the EPPI field and trigger selection. These are valid for GP modes (EPPI_CTL.XFRTYPE = 0x3) and ITU656 active video mode (EPPI_CTL.XFRTYPE cleared).
		0 Field Mode 0. Read field 1 (for ITU656 active video mode). Set internal trigger (for GP RX mode). FS3 is toggled on FS2 assertion followed by FS1 assertion (when the EPPI_CTL.FSCFG bit selects sync mode 3 and the EPPI_CTL.IFSGEN bit selects internal frame sync).
		1 Field Mode 1 Read field 1 and field 2 (ITU656 active video mode). Set external trigger (GP RX mode). FS3 is toggled on FS2 assertion (when the EPPI_CTL.FSCFG bit selects sync mode 3 and the EPPI_CTL.IFSGEN bit selects internal frame sync).
5:4 (R/W)	FSCFG	Frame Sync Configuration. The EPPI_CTL.FSCFG bits configure the EPPI frame syncs. These are valid only for GP modes (EPPI_CTL.XFRTYPE = 0x3). The output of the frames syncs also depends on whether the EPPI transfer direction is transmit and the EPPI is in ITU output mode (EPPI_CTL.BLANKGEN is set).
		0 Sync Mode 0. FS0 driven in GP mode. FS0 not driven in ITU output mode.
		1 Sync Mode 1. FS1 driven in GP mode. HSYNC driven on FS1 in ITU output mode.
		2 Sync Mode 2. FS2 driven in GP mode. HSYNC driven on FS1 and VSYNC driven on FS1 in ITU output mode.
		3 Sync Mode 3. FS3 driven in GP mode. HSYNC driven on FS1, VSYNC driven on FS2, and FIELD driven on FS3 in ITU output mode.

Table 19-49: EPPI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3:2 (R/W)	XFRTYPE	Transfer Type (Operating Mode). The EPPI_CTL.XFRTYPE bits select the EPPI operating mode. In receive mode (EPPI_CTL.DIR cleared), the EPPI modes include ITU656 active video only mode, ITU656 entire field mode, ITU656 vertical blanking only mode, and non-ITU656 mode (GP mode). For transmit mode (EPPI_CTL.DIR set), the EPPI_CTL.XFRTYPE bits have no effect, and the EPPI (in transmit mode) is always in GP mode.
		0 ITU656 Active Video Only Mode
		1 ITU656 Entire Field Mode
		2 ITU656 Vertical Blanking Only Mode
		3 Non-ITU656 Mode (GP Mode)
1 (R/W)	DIR	PPI Direction. The EPPI_CTL.DIR bit selects whether the EPPI is in receive mode (if cleared) or in transmit mode (if set).
		0 Receive Mode
		1 Transmit Mode
0 (R/W)	EN	PPI Enable. The EPPI_CTL.EN bit enables or disables the EPPI.
		0 Disable
		1 Enable

Control Register 2 Register

The `EPPI_CTL2` register controls HSYNC finish signal generation.

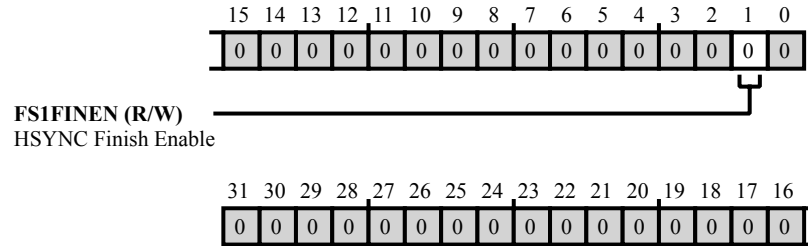


Figure 19-13: EPPI_CTL2 Register Diagram

Table 19-50: EPPI_CTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	FS1FINEN	HSYNC Finish Enable. The <code>EPPI_CTL2.FS1FINEN</code> bit selects whether (if set) the EPPI sends a finish command (010) through the DDE COMMAND line soon after a LINE is received completely or (if cleared) the EPPI sends a finish command (010) through the DDE COMMAND line soon after a FRAME is received completely. Note that the <code>EPPI_CTL.DMAFINEN</code> bit must be set for the EPPI to generate either of the finish commands.
		0 Finish sent after frame RX done. PPI sends a finish command (010) through the DDE COMMAND line soon after a FRAME is received completely
		1 Finish sent after frame/line RX done. PPI sends a finish command (010) through the DDE COMMAND line soon after a frame/line is received completely.

Clipping Register for EVEN (Luma) Data Register

The `EPPI_EVENCLIP` register selects the clipping threshold for luma data, which provides clipping of individual video components.

The high even and low even spaces in `EPPI_EVENCLIP` are 16-bits wide and (depending on the `EPPI_CTL.DLEN` bit selection) only the corresponding video component bits are considered for clipping.

For example, if the EPPI is programmed in 10-bit mode, bits [9:0] and bits [25:16] constitute the clipping thresholds. The higher bits are (in this case) ignored.

Using this method, 8-, 10-, 12- and 16-bit clipping thresholds can be set.

Note that when the EPPI is programmed in 16-, 20-, or 24-bit mode with the `EPPI_CTL.SPLTWRD` bit set, the luma data gets the clipping threshold levels of the `EPPI_EVENCLIP` register, and the chroma data gets the clipping threshold levels of the `EPPI_ODDCLIP` register.

Also, note that the `EPPI_EVENCLIP` and `EPPI_ODDCLIP` registers are ignored when the `EPPI_CTL.RGBFMTEN` bit is set.

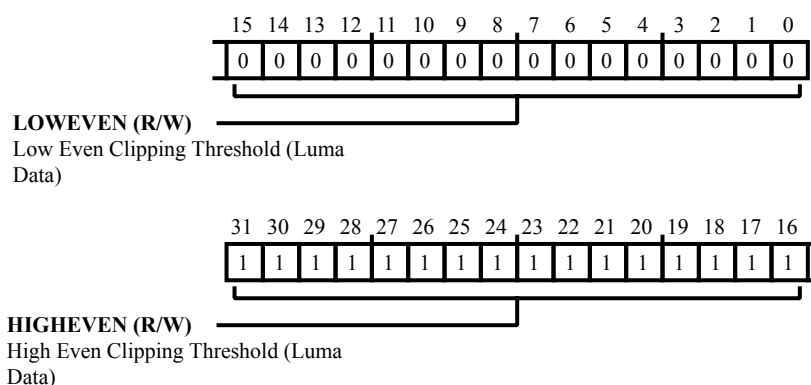


Figure 19-14: EPPI_EVENCLIP Register Diagram

Table 19-51: EPPI_EVENCLIP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	HIGHEVEN	High Even Clipping Threshold (Luma Data). The <code>EPPI_EVENCLIP.HIGHEVEN</code> bit field selects the clipping threshold for luma data. The high even spaces are 16-bits wide and (depending on the <code>EPPI_CTL.DLEN</code> selection) only the corresponding video component bits are considered for clipping.
15:0 (R/W)	LOWEVEN	Low Even Clipping Threshold (Luma Data). The <code>EPPI_EVENCLIP.LOWEVEN</code> bit field selects the clipping threshold for luma data. The low even spaces are 16-bits wide and (depending on the <code>EPPI_CTL.DLEN</code> selection) only the corresponding video component bits are considered for clipping.

Lines Per Frame Register

The EPPI_FRAME register tracks the frame track overflow and underflow errors. This register should be programmed with the number of lines expected per frame. Any write to the EPPI_FRAME register will also write the same value to the EPPI_VCNT register. But, any write to the EPPI_VCNT register does not affect the EPPI_FRAME register value. So the EPPI_FRAME register should be programmed before the EPPI_VCNT register.

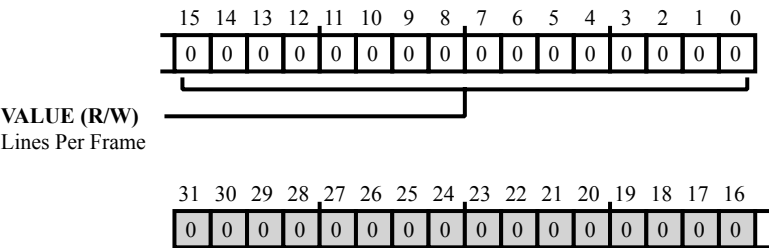


Figure 19-15: EPPI_FRAME Register Diagram

Table 19-52: EPPI_FRAME Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Lines Per Frame. The EPPI_FRAME.VALUE holds the number of lines expected per frame of data.

Frame Sync 1 Delay Value Register

The `EPPI_FS1_DLY` register selects the delay count (based on the period of the `EPPI_CLK` clock) between the first rising edge of `EPPI_CLK` after the EPPI is enabled and the first active edge of the associated frame sync when the internal frame sync is used.

Note that if the `EPPI_FS1_DLY` or `EPPI_FS2_DLY` registers are programmed with value 0, the EPPI operates as though 0 value is 1, and the first frame sync transition occurs after the completion of one period value of the respective counters.

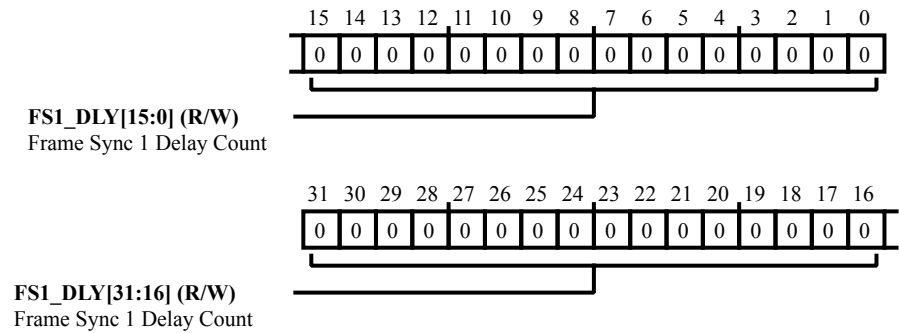


Figure 19-16: EPPI_FS1_DLY Register Diagram

Table 19-53: EPPI_FS1_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	FS1_DLY	Frame Sync 1 Delay Count. The <code>EPPI_FS1_DLY.FS1_DLY</code> bit field selects the delay count.

FS1 Period Register / EPPI Active Samples Per Line Register

The `EPPI_FS1_PASPL` register content varies depending on whether the EPPI is in GP1/2/3 FS modes or in GP transmit mode.

In GP 1, 2, or 3 FS modes, the `EPPI_FS1_PASPL` register is used for the generation of frame sync 1. The register contains the period required for `EPPI_FS1` based on the `EPPI_CLK` clock.

In GP transmit mode with the `EPPI_CTL.BLANKGEN` bit set, this register contains the number of samples of active video or vertical blanking samples per line. When used for blanking generation, only the lower 16 bits are valid.

Note that a value of 0 for this register is illegal. If programmed as 0, the EPPI regards the `EPPI_FS1_PASPL` register as containing 1.

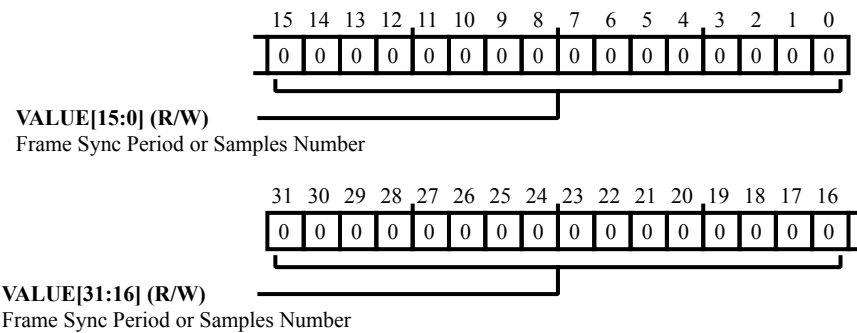


Figure 19-17: `EPPI_FS1_PASPL` Register Diagram

Table 19-54: `EPPI_FS1_PASPL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Frame Sync Period or Samples Number. In GP 1, 2, or 3 FS modes, the <code>EPPI_FS1_PASPL.VALUE</code> bit field is used for the generation of frame sync 1 and contains the period required for <code>EPPI_FS1</code> based on the <code>EPPI_CLK</code> clock. In GP transmit mode with the <code>EPPI_CTL.BLANKGEN</code> bit set, this bit field contains the number of samples of active video or vertical blanking samples per line.

FS1 Width Register / EPPI Horizontal Blanking Samples Per Line Register

The `EPPI_FS1_WLHB` register's content varies depending on whether the EPPI is in GP1/2/3 FS modes or in GP transmit mode.

In GP 1, 2 or 3 FS modes, `EPPI_FS1_WLHB` is used for the generation of frame sync 1. The register contains the width required for `EPPI_FS1` based on the `EPPI_CLK` clock.

In GP transmit mode with the `EPPI_CTL.BLANKGEN` bit set, this register contains the number of samples of horizontal blanking per line. When used for blanking generation, only the lower 16 bits are valid.

Note that a value of 0 for the `EPPI_FS1_WLHB` register is illegal. If programmed as 0, the EPPI regards the `EPPI_FS1_WLHB` register as containing 1.

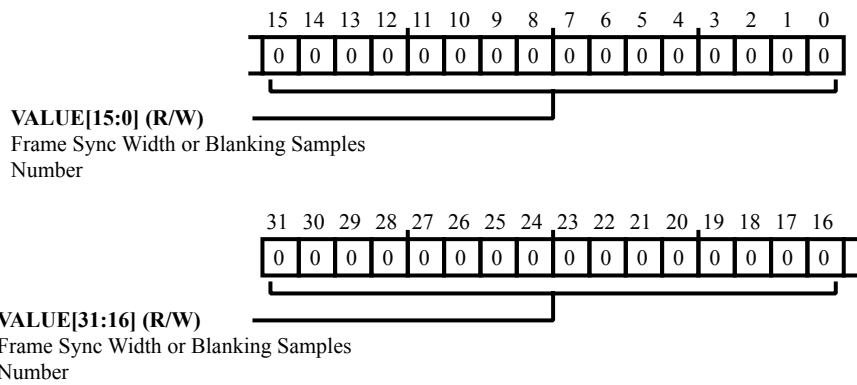


Figure 19-18: EPPI_FS1_WLHB Register Diagram

Table 19-55: EPPI_FS1_WLHB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Frame Sync Width or Blanking Samples Number. The <code>EPPI_FS1_WLHB.VALUE</code> bit field content varies depending on whether the EPPI is in GP1/2/3 FS modes or in GP transmit mode. In GP 1, 2 or 3 FS modes, the <code>EPPI_FS1_WLHB.VALUE</code> bit field is used for the generation of frame sync 1. The register contains the width required for <code>EPPI_FS1</code> based on the <code>EPPI_CLK</code> clock. In GP transmit mode with <code>EPPI_CTL.BLANKGEN</code> set, this bit field contains the number of samples of horizontal blanking per line.

Frame Sync 2 Delay Value Register

The `EPPI_FS2_DLY` register selects the delay count (based on the period of the `EPPI_CLK` clock) between the first rising edge of `EPPI_CLK` after EPPI enabled and the first active edge of the associated frame sync when the internal frame sync is used.

Note that if the `EPPI_FS1_DLY` or `EPPI_FS2_DLY` registers are programmed with the value 0, the EPPI operates as though 0 value is 1, and the first frame sync transition occurs after the completion of one period value of the respective counters.

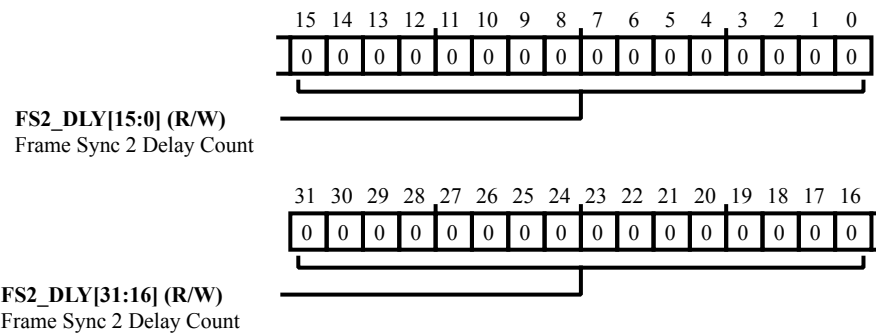


Figure 19-19: EPPI_FS2_DLY Register Diagram

Table 19-56: EPPI_FS2_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	FS2_DLY	Frame Sync 2 Delay Count. The <code>EPPI_FS2_DLY.FS2_DLY</code> bit field selects the delay count.

FS2 Period Register / EPPI Active Lines Per Field Register

The `EPPI_FS2_PALPF` register content varies depending on whether the EPPI is in GP2/3 FS modes or in GP transmit mode.

In GP 2 or 3 FS modes, `EPPI_FS2_PALPF` is used for the generation of frame sync 2. This register contains the period required for `EPPI_FS2` based on the `EPPI_CLK` clock.

In GP transmit mode with the `EPPI_CTL.BLANKGEN` bit set, this register contains the number of lines of active video per field.

Note that a value of 0 for the `EPPI_FS2_PALPF.F1ACT` or `EPPI_FS2_PALPF.F2ACT` bits is illegal. If either is programmed as 0, the EPPI regards the 0 value fields as containing 1.

Also, note that for progressive video, the `EPPI_FS2_PALPF.F2ACT` bit is ignored.

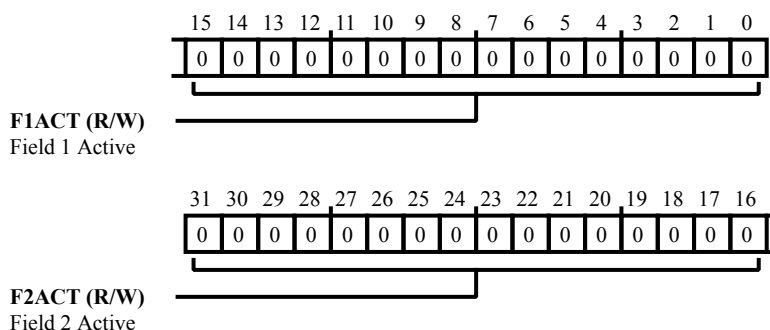


Figure 19-20: `EPPI_FS2_PALPF` Register Diagram

Table 19-57: `EPPI_FS2_PALPF` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	F2ACT	Field 2 Active. The <code>EPPI_FS2_PALPF.F2ACT</code> bit field contains the number of lines of active data in field 2.
15:0 (R/W)	F1ACT	Field 1 Active. The <code>EPPI_FS2_PALPF.F1ACT</code> bit field contains the number of lines of active data in field 1.

FS2 Width Register / EPPI Lines Of Vertical Blanking Register

The `EPPI_FS2_WLVB` register content varies depending on whether the EPPI is in GP2/3 FS modes or in GP transmit mode.

In GP 2 or 3 FS modes, the `EPPI_FS2_WLVB` register is used for the generation of frame sync 2. The register contains the width required for `EPPI_FS2` based on the `EPPI_CLK` clock.

In GP transmit mode with the `EPPI_CTL.BLANKGEN` bit set, this register contains the number or lines of vertical blanking.

Note that for progressive video, the `EPPI_FS2_WLVB.F2VBBD` and `EPPI_FS2_WLVB.F2VBAD` bits are ignored.

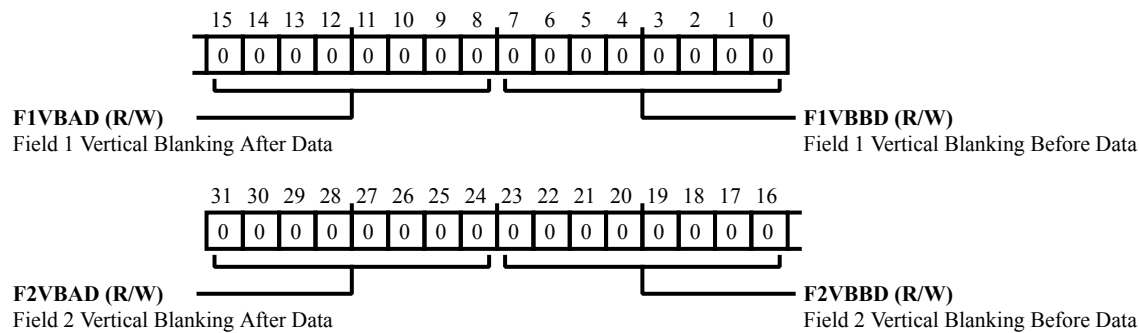


Figure 19-21: `EPPI_FS2_WLVB` Register Diagram

Table 19-58: `EPPI_FS2_WLVB` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	F2VBAD	Field 2 Vertical Blanking After Data. The <code>EPPI_FS2_WLVB.F2VBAD</code> bit field contains the number of lines of vertical blanking after field 2.
23:16 (R/W)	F2VBBD	Field 2 Vertical Blanking Before Data. The <code>EPPI_FS2_WLVB.F2VBBD</code> bit field contains the number of lines of vertical blanking before field 2.
15:8 (R/W)	F1VBAD	Field 1 Vertical Blanking After Data. The <code>EPPI_FS2_WLVB.F1VBAD</code> bit field contains the number of lines of vertical blanking after field 1.
7:0 (R/W)	F1VBBD	Field 1 Vertical Blanking Before Data. The <code>EPPI_FS2_WLVB.F1VBBD</code> bit field contains the number of lines of vertical blanking before field 1.

Horizontal Transfer Count Register

The `EPPI_HCNT` register holds the number of samples to read in or write out per line, after `EPPI_HDLY` number of cycles have expired since the assertion of `EPPI_FS1`. Any write to the `EPPI_LINE` register modifies the `EPPI_HCNT` register. But, any write to the `EPPI_HCNT` register does not affect the `EPPI_LINE` register value. So the `EPPI_HCNT` register should be programmed after the `EPPI_LINE` register.

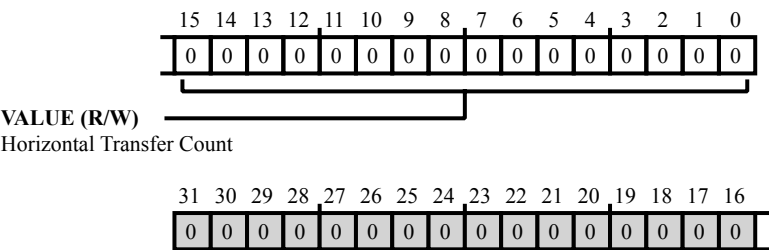


Figure 19-22: EPPI_HCNT Register Diagram

Table 19-59: EPPI_HCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Horizontal Transfer Count. The <code>EPPI_HCNT.VALUE</code> holds the number of samples to read in or write out per line, after <code>EPPI_HDLY</code> number of cycles have expired since the last assertion of <code>EPPI_FS1</code> .

Horizontal Delay Count Register

The `EPPI_HDLY` register contains the number of clock cycles to delay after the assertion of `EPPI_FS1` is detected before starting to read or write data.

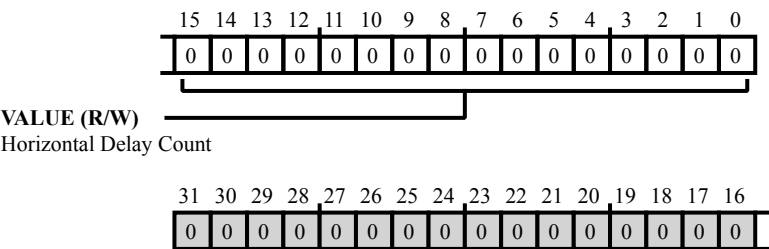


Figure 19-23: EPPI_HDLY Register Diagram

Table 19-60: EPPI_HDLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Horizontal Delay Count. The <code>EPPI_HDLY.VALUE</code> holds the number of <code>EPPI_CLK</code> cycles to delay after as- sertion of <code>EPPI_FS1</code> before starting to read or write data.

Interrupt Mask Register

The **EPPI_IMSK** register permits the masking (if associated bit is set) of EPPI error interrupts for YFIFO underflow or overflow, CFIFO underflow or overflow, line track overflow error, line track underflow error, frame track overflow error, frame track underflow error, and ERR_NCOR (ITU preamble error not corrected). These conditions are flagged in the **EPPI_STAT** register and cleared by write-1-to-clear.

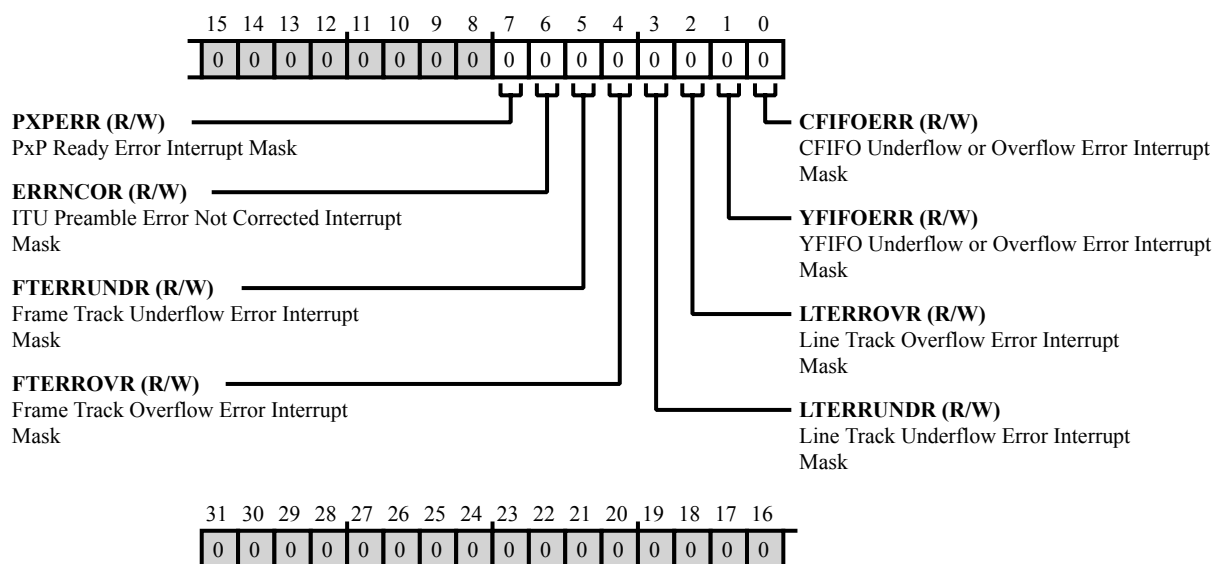


Figure 19-24: EPPI_IMSK Register Diagram

Table 19-61: EPPI_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	PXPERR	PxP Ready Error Interrupt Mask.
		0 Unmask Interrupt
		1 Mask Interrupt
6 (R/W)	ERRNCOR	ITU Preamble Error Not Corrected Interrupt Mask.
		0 Unmask Interrupt
		1 Mask Interrupt
5 (R/W)	FTERRUNDR	Frame Track Underflow Error Interrupt Mask.
		0 Unmask Interrupt
		1 Mask Interrupt
4 (R/W)	FTERROVR	Frame Track Overflow Error Interrupt Mask.
		0 Unmask Interrupt
		1 Mask Interrupt

Table 19-61: EPPI_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
3 (R/W)	LTERRUNDR	Line Track Underflow Error Interrupt Mask.	
		0	Unmask Interrupt
		1	Mask Interrupt
2 (R/W)	LTERROVR	Line Track Overflow Error Interrupt Mask.	
		0	Unmask Interrupt
		1	Mask Interrupt
1 (R/W)	YFIFOERR	YFIFO Underflow or Overflow Error Interrupt Mask.	
		0	Unmask Interrupt
		1	Mask Interrupt
0 (R/W)	CFIFOERR	CFIFO Underflow or Overflow Error Interrupt Mask.	
		0	Unmask Interrupt
		1	Mask Interrupt

Samples Per Line Register

The `EPPI_LINE` register tracks the line track overflow and underflow errors. This register should be programmed with the number of samples expected per line. Any write to the `EPPI_LINE` register will also write the same value to the `EPPI_HCNT` register. However, any write to the `EPPI_HCNT` register does not affect the `EPPI_LINE` register value. So the `EPPI_LINE` register should be programmed before the `EPPI_HCNT` register.

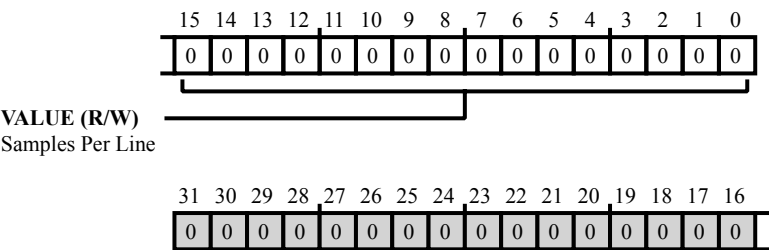


Figure 19-25: EPPI_LINE Register Diagram

Table 19-62: EPPI_LINE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Samples Per Line. The <code>EPPI_LINE.VALUE</code> holds the number of samples expected per line.

Clipping Register for ODD (Chroma) Data Register

The `EPPI_ODDCLIP` register selects the clipping threshold for chroma data, which provides clipping of individual video components.

The high odd and low odd spaces in `EPPI_ODDCLIP` are 16-bits wide and (depending on the `EPPI_CTL.DLEN` bit selection) only the corresponding video component bits are considered for clipping.

For example, if the EPPI is programmed in 10-bit mode, bits [9:0] and bits [25:16] constitute the clipping thresholds. The higher bits are (in this case) ignored.

Using the this method, 8-, 10-, 12- and 16-bit clipping thresholds can be set.

Note that when the EPPI is programmed in 16-, 20-, or 24-bit mode with the `EPPI_CTL.SPLTWRD` bit set, the luma data gets the clipping threshold levels of the `EPPI_EVENCLIP` register, and the chroma data gets the clipping threshold levels of the `EPPI_ODDCLIP` register.

Also, note that the `EPPI_EVENCLIP` and `EPPI_ODDCLIP` registers are ignored when the `EPPI_CTL.RGBFMTEN` bit is set.

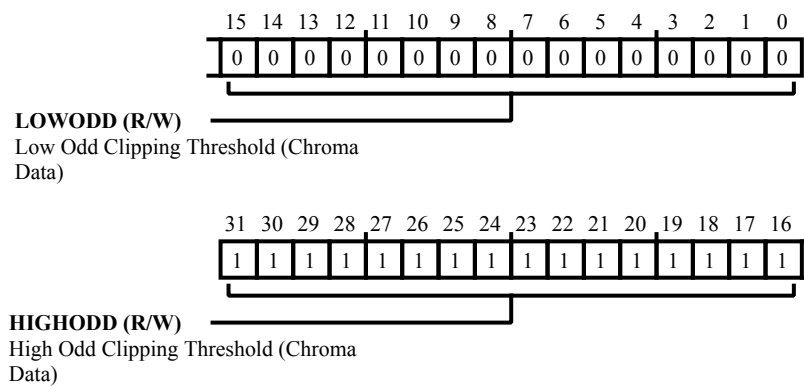


Figure 19-26: EPPI_ODDCLIP Register Diagram

Table 19-63: EPPI_ODDCLIP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	HIGHODD	High Odd Clipping Threshold (Chroma Data). The <code>EPPI_ODDCLIP.HIGHODD</code> bit field selects the clipping threshold for luma data. The high odd spaces are 16-bits wide and (depending on the <code>EPPI_CTL.DLEN</code> selection) only the corresponding video component bits are considered for clipping.
15:0 (R/W)	LOWODD	Low Odd Clipping Threshold (Chroma Data). The <code>EPPI_ODDCLIP.LOWODD</code> bit field selects the clipping threshold for luma data. The low add spaces are 16-bits wide and (depending on the <code>EPPI_CTL.DLEN</code> selection) only the corresponding video component bits are considered for clipping.

Status Register

The `EPPI_STAT` register contains bits that provide information about the current operating state of the EPPI.

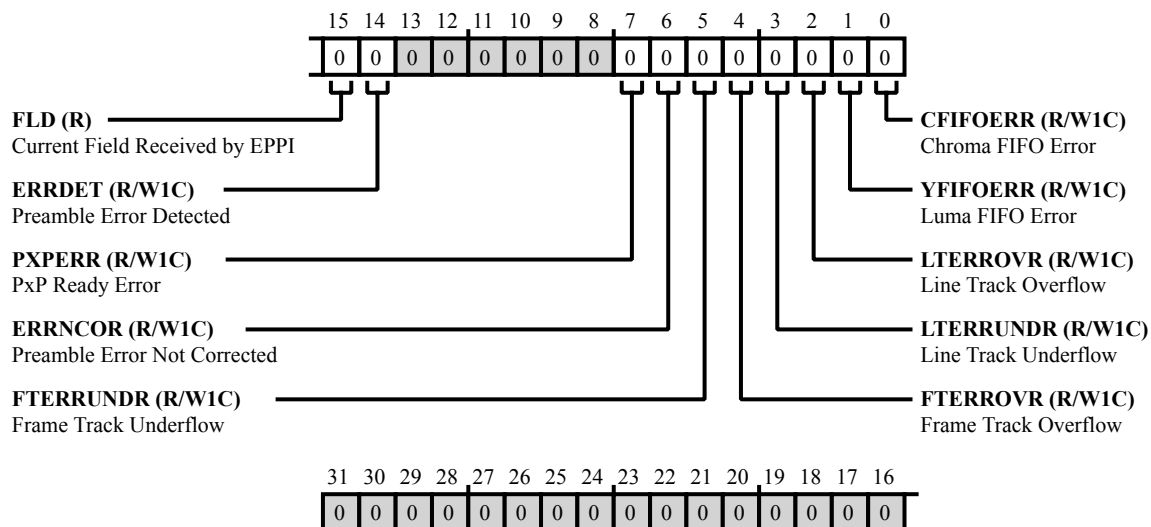


Figure 19-27: EPPI_STAT Register Diagram

Table 19-64: EPPI_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/NW)	FLD	Current Field Received by EPPI. The <code>EPPI_STAT.FLD</code> bit indicates whether the current field being received by the PPI is field 1 (if clear) or field 2 (if set).
		0 Field 1
		1 Field 2
14 (R/W1C)	ERRDET	Preamble Error Detected. The <code>EPPI_STAT.ERRDET</code> bit is useful only in ITU receive modes and indicates if an error has been detected in the status word of EAV or SAV sequences (if set) or not (if clear).
		0 No Preamble Error Detected
		1 Preamble Error Detected
7 (R/W1C)	PXPERR	PxP Ready Error. The <code>EPPI_STAT.PXPERR</code> bit is valid only in the RX mode. This bit indicates whether the incoming PPI data overflows the PxP interface (if set) or not (if clear). This bit is sticky and must be cleared by software by writing 1 to it.

Table 19-64: EPPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1C)	ERRNCOR	Preamble Error Not Corrected. The <code>EPPI_STAT.ERRNCOR</code> bit is useful only in the ITU receive modes and indicates if an error in the status word of EAV or SAV sequences can not be cleared (if set) or not (if clear). This bit is sticky and must be cleared by software by writing 1 to it.
		0 No uncorrected preamble error has occurred
		1 Preamble error detected but not corrected
5 (R/W1C)	FTERRUNDR	Frame Track Underflow. The <code>EPPI_STAT.FTERRUNDR</code> bit indicates whether a frame track underflow error has occurred (if set) or not (if clear). This bit is sticky and must be cleared by software by writing 1 to it.
		0 No Error Detected
		1 Error Occurred
4 (R/W1C)	FTERROVR	Frame Track Overflow. The <code>EPPI_STAT.FTERROVR</code> bit indicates whether a frame track overflow error has occurred (if set) or not (if clear). This bit is sticky and must be cleared by software by writing 1 to it.
		0 No Error Detected
		1 Error Occurred
3 (R/W1C)	LTERRUNDR	Line Track Underflow. The <code>EPPI_STAT.LTERRUNDR</code> bit indicates whether a line track underflow error has occurred (if set) or not (if clear). This bit is sticky and must be cleared by software by writing 1 to it.
		0 No Error Detected
		1 Error Occurred
2 (R/W1C)	LTERROVR	Line Track Overflow. The <code>EPPI_STAT.LTERROVR</code> bit indicates whether a line track overflow error has occurred (if set) or not (if clear). This bit is sticky and must be cleared by software by writing 1 to it.
		0 No Error Detected
		1 Error Occurred

Table 19-64: EPPI_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W1C)	YFIFOERR	Luma FIFO Error. For RX modes, the EPPI_STAT.YFIFOERR bit indicates whether the Luma FIFO has overflowed (if set) or not (if clear). For TX modes, this bit indicates whether the Luma FIFO has underflowed (if set) or not (if clear). This bit is sticky and must be cleared by software by writing 1 to it.
		0 No Error Detected
		1 Error Occurred
0 (R/W1C)	CFIFOERR	Chroma FIFO Error. For RX modes, the EPPI_STAT.CFIFOERR bit indicates whether the chroma FIFO has overflowed (if set) or not (if clear). For TX modes, this bit indicates whether the chroma FIFO has underflowed (if set) or not (if clear). This bit is sticky and must be cleared by software by writing 1 to it.
		0 No Error Detected
		1 Error Occurred

Vertical Transfer Count Register

The `EPPI_VCNT` register holds the number of lines to read in or write out, after `EPPI_VDLY` number of lines from the start of frame. Any write to the `EPPI_FRAME` register modifies the `EPPI_VCNT` register. However, any write to the `EPPI_VCNT` register does not affect the `EPPI_FRAME` register value. So the `EPPI_VCNT` register should be programmed after the `EPPI_FRAME` register.

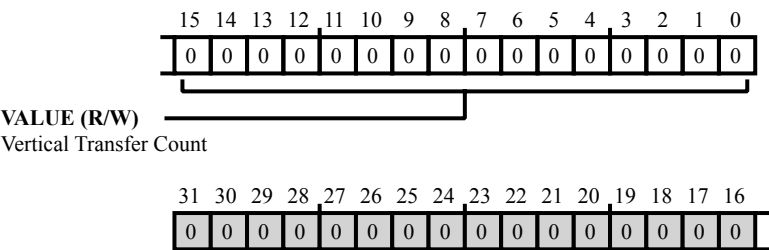


Figure 19-28: EPPI_VCNT Register Diagram

Table 19-65: EPPI_VCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Vertical Transfer Count. The <code>EPPI_VCNT.VALUE</code> holds the number of lines to read in or write out, after <code>EPPI_VDLY</code> number of lines from the start of frame.

Vertical Delay Count Register

The `EPPI_VDLY` register contains the number of lines to wait after the start of a new frame before starting to read/transmit data.

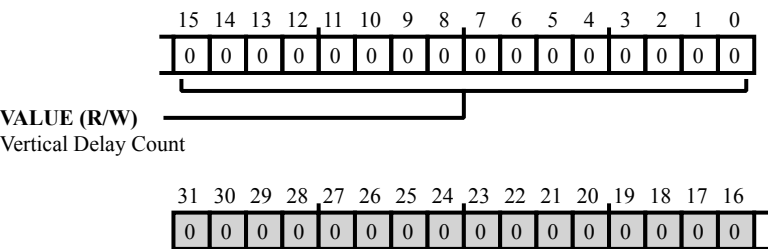


Figure 19-29: EPPI_VDLY Register Diagram

Table 19-66: EPPI_VDLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Vertical Delay Count. The <code>EPPI_VDLY.VALUE</code> holds the number of lines to wait after the start of a new frame before starting to read/transmit data.

20 General-Purpose Timer (TIMER)

The general-purpose timer (GP Timer) module serves as a collection of system timers that support various system-level functions. These functions include:

- Synchronized PWM waveform output capability
- External signal capture
- External event count
- General time-base functionality

Additionally, interrupt requests can be generated upon completion of timer events. Moreover, GP timers can act both as trigger masters and trigger slaves.

GP Timer Features

Each timer can be individually configured in any of these modes:

- Pin interrupt capture mode
- Windowed watchdog mode
- Pulse-width count and capture (WDTH_CAP) mode
- External Event (EXT_CLK) mode
- Pulse-width modulation (PWM_OUT) mode

Other features include:

- Synchronous operation
- Consistent management of period and pulse width values
- Autobaud detection for UART module (where available)
- Graceful bit pattern termination when stopping
- Support for center-aligned PWM patterns
- Error detection on implausible pattern values

- All read and write accesses to 32-bit registers are atomic
- Every timer has its dedicated interrupt request output

ADSP-2159x TIMER Register List

The General-Purpose Timer block (TIMER) provides timers that may be used for external event capture and measurement, system timing, and PWM waveform generation. A set of registers governs TIMER operations. For more information on TIMER functionality, see the TIMER register descriptions.

Table 20-1: ADSP-2159x TIMER Register List

Name	Description
TIMER_BCAST_DLY	Broadcast Delay Register
TIMER_BCAST_PER	Broadcast Period Register
TIMER_BCAST_WID	Broadcast Width Register
TIMER_DATA_ILAT	Data Interrupt Latch Register
TIMER_DATA_IMSK	Data Interrupt Mask Register
TIMER_ERR_TYPE	Error Type Status Register
TIMER_RUN	Run Register
TIMER_RUN_CLR	Run Clear Register
TIMER_RUN_SET	Run Set Register
TIMER_STAT_ILAT	Status Interrupt Latch Register
TIMER_STAT_IMSK	Status Interrupt Mask Register
TIMER_STOP_CFG	Stop Configuration Register
TIMER_STOP_CFG_CLR	Stop Configuration Clear Register
TIMER_STOP_CFG_SET	Stop Configuration Set Register
TIMER_TMR[n]_CFG	Timer n Configuration Register
TIMER_TMR[n]_CNT	Timer n Counter Register
TIMER_TMR[n]_DLY	Timer n Delay Register
TIMER_TMR[n]_PER	Timer n Period Register
TIMER_TMR[n]_WID	Timer n Width Register
TIMER_TRG_IE	Trigger Slave Enable Register
TIMER_TRG_MSK	Trigger Master Mask Register

ADSP-2159x TIMER Interrupt List

Table 20-2: ADSP-2159x TIMER Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
48	TIMER0_TMR00	TIMER0 Timer 0	Level	
49	TIMER0_TMR01	TIMER0 Timer 1	Level	
50	TIMER0_TMR02	TIMER0 Timer 2	Level	
51	TIMER0_TMR03	TIMER0 Timer 3	Level	
52	TIMER0_TMR04	TIMER0 Timer 4	Level	
53	TIMER0_TMR05	TIMER0 Timer 5	Level	
54	TIMER0_TMR06	TIMER0 Timer 6	Level	
55	TIMER0_TMR07	TIMER0 Timer 7	Level	
56	TIMER0_TMR08	TIMER0 Timer 8	Level	
57	TIMER0_TMR09	TIMER0 Timer 9	Level	
58	TIMER0_TMR10	TIMER0 Timer 10	Level	
59	TIMER0_TMR11	TIMER0 Timer 11	Level	
60	TIMER0_TMR12	TIMER0 Timer 12	Level	
61	TIMER0_TMR13	TIMER0 Timer 13	Level	
62	TIMER0_TMR14	TIMER0 Timer 14	Level	
63	TIMER0_TMR15	TIMER0 Timer 15	Level	
64	TIMER0_STAT	TIMER0 Status	Level	

ADSP-2159x TIMER Trigger List

Table 20-3: ADSP-2159x TIMER Trigger List Masters

Trigger ID	Name	Description	Sensitivity
142	TIMER0_TMR00_MST	TIMER0 Timer 0	Edge
143	TIMER0_TMR01_MST	TIMER0 Timer 1	Edge
144	TIMER0_TMR02_MST	TIMER0 Timer 2	Edge
145	TIMER0_TMR03_MST	TIMER0 Timer 3	Edge
146	TIMER0_TMR04_MST	TIMER0 Timer 4	Edge
147	TIMER0_TMR05_MST	TIMER0 Timer 5	Edge
148	TIMER0_TMR06_MST	TIMER0 Timer 6	Edge
149	TIMER0_TMR07_MST	TIMER0 Timer 7	Edge
150	TIMER0_TMR08_MST	TIMER0 Timer 8	Edge

Table 20-3: ADSP-2159x TIMER Trigger List Masters (Continued)

Trigger ID	Name	Description	Sensitivity
151	TIMER0_TMR09_MST	TIMER0 Timer 9	Edge
152	TIMER0_TMR10_MST	TIMER0 Timer 10	Edge
153	TIMER0_TMR11_MST	TIMER0 Timer 11	Edge
154	TIMER0_TMR12_MST	TIMER0 Timer 12	Edge
155	TIMER0_TMR13_MST	TIMER0 Timer 13	Edge
156	TIMER0_TMR14_MST	TIMER0 Timer 14	Edge
157	TIMER0_TMR15_MST	TIMER0 Timer 15	Edge

Table 20-4: ADSP-2159x TIMER Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
125	TIMER0_TMR00_SLV0	TIMER0 Timer 0	Pulse
126	TIMER0_TMR00_SLV1	TIMER0 Timer 0	Pulse
127	TIMER0_TMR01_SLV0	TIMER0 Timer 1	Pulse
128	TIMER0_TMR01_SLV1	TIMER0 Timer 1	Pulse
129	TIMER0_TMR02_SLV0	TIMER0 Timer 2	Pulse
130	TIMER0_TMR02_SLV1	TIMER0 Timer 2	Pulse
131	TIMER0_TMR03_SLV0	TIMER0 Timer 3	Pulse
132	TIMER0_TMR03_SLV1	TIMER0 Timer 3	Pulse
133	TIMER0_TMR04_SLV0	TIMER0 Timer 4	Pulse
134	TIMER0_TMR04_SLV1	TIMER0 Timer 4	Pulse
135	TIMER0_TMR05_SLV0	TIMER0 Timer 5	Pulse
136	TIMER0_TMR05_SLV1	TIMER0 Timer 5	Pulse
137	TIMER0_TMR06_SLV0	TIMER0 Timer 6	Pulse
138	TIMER0_TMR06_SLV1	TIMER0 Timer 6	Pulse
139	TIMER0_TMR07_SLV0	TIMER0 Timer 7	Pulse
140	TIMER0_TMR07_SLV1	TIMER0 Timer 7	Pulse
141	TIMER0_TMR08_SLV0	TIMER0 Timer 8	Pulse
142	TIMER0_TMR08_SLV1	TIMER0 Timer 8	Pulse
143	TIMER0_TMR09_SLV0	TIMER0 Timer 9	Pulse
144	TIMER0_TMR09_SLV1	TIMER0 Timer 9	Pulse
145	TIMER0_TMR10_SLV0	TIMER0 Timer 10	Pulse

Table 20-4: ADSP-2159x TIMER Trigger List Slaves (Continued)

Trigger ID	Name	Description	Sensitivity
146	TIMER0_TMR10_SLV1	TIMER0 Timer 10	Pulse
147	TIMER0_TMR11_SLV0	TIMER0 Timer 11	Pulse
148	TIMER0_TMR11_SLV1	TIMER0 Timer 11	Pulse
149	TIMER0_TMR12_SLV0	TIMER0 Timer 12	Pulse
150	TIMER0_TMR12_SLV1	TIMER0 Timer 12	Pulse
151	TIMER0_TMR13_SLV0	TIMER0 Timer 13	Pulse
152	TIMER0_TMR13_SLV1	TIMER0 Timer 13	Pulse
153	TIMER0_TMR14_SLV0	TIMER0 Timer 14	Pulse
154	TIMER0_TMR14_SLV1	TIMER0 Timer 14	Pulse
155	TIMER0_TMR15_SLV0	TIMER0 Timer 15	Pulse
156	TIMER0_TMR15_SLV1	TIMER0 Timer 15	Pulse

Timer Block Diagram

The Timer block diagram figure shows all of the possible clock sources.

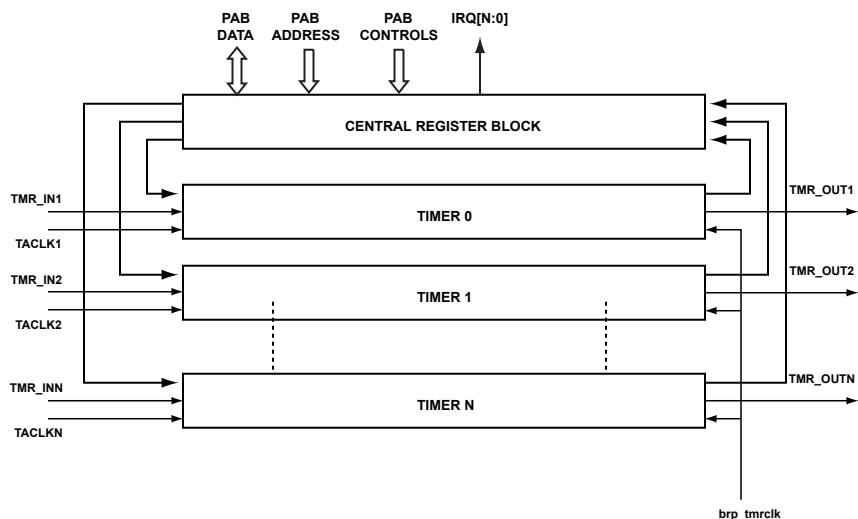


Figure 20-1: Timer Block Diagram

Internal Interface

The processor core always accesses the timer registers through the MMR access bus. Hardware ensures that all read and write operations from and to 32-bit timer registers are atomic. Every timer has a dedicated data interrupt request. There is also one common timer status and error interrupt request output that connects to the system event controller. Whenever a data interrupt request is generated, a data trigger master pulse is also driven out, if enabled. Each timer has an individual trigger input line, and each timer can be either started or stopped as a trigger slave.

In total, the GP timer module can have up to $(N + 1)$ interrupt request output lines and N data trigger lines.

Internal Timer Connections

The Timers support alternate inputs for the clock/capture (see [External Interface](#)). Some signals have internal default alternate connections according to the *Timer Signal Routing* table.

Table 20-5: Timer Signal Routing

Timer Signal	Connection
TM0_ACLK0	SYS_CLKIN0
TM0_ACI5	DAI0_PB04
TM0_ACLK5	DAI0_PB03
TM0_ACI6	DAI1_PB04
TM0_ACLK6	DAI1_PB03
TM0_ACI07	CNT0_TO
TM0_ACLK7	SYS_CLKIN1
TM0_ACI8	DAI0_PB06
TM0_ACLK8	DAI0_PB05
TM0_ACI9	DAI1_PB06
TM0_ACLK9	DAI1_PB05
TM0_ACI14	DAI0 Group C
TM0_ACI15	DAI1 Group C

External Interface

Each GP timer module can support up to 16 individual timers. However, most processors have less than this number. The exact number of timers available on a given processor is available in the data sheet for the processor.

Every timer has one main input/output signal ($TIMER_TMR[nn]$) and, usually, one auxiliary input pin, used as an alternate capture input ($TIMER_ACI[nn]$). Each timer can either run with a time base of SCLK0 or can reference an external clock on one of two ($TIMER_ACLK[nn]$) pins. The TMR_ALT_CLK0 signal maps to individual alternate clock ($TIMER_ACLK[nn]$) pins for one or more timers. For instance, a TM_ACLK3 pin would provide an alternate site to supply an external signal that would serve as reference clock for TMR3. Likewise, the TMR_ALT_CLK1 signal from each timer unit connects together internally to provide a single global timer clock pin ($TIMER_CLK$) for the GP timer module. It is used as an additional time base.

GP Timer Operating Modes

The following sections provide information on the various operating modes of the GP timer.

General Operation

The core of every timer is a 32-bit counter that can be interrogated through the read-only `TIMER_TMR[n]_CNT` register. Once the module enables a timer, it loads the timer `TIMER_TMR[n]_CNT` register with a starting value.

A timer can operate in one of several different modes, configured through the `TIMER_TMR[n]_CFG` register for that timer. These modes are: PWMOUT, EXTCLK, WIDCAP, WATCHDOG, PININT, and IDLE. The *Timer Mode Descriptions* table summarizes the modes.

Table 20-6: Timer Mode Descriptions

Timer Mode	Description
PWMOUT	Generates single or continuous PWM waveforms with programmable pulse width, period, and delay
EXTCLK	Counts edges of an externally applied waveform
WIDCAP	Captures pulse width or period of an externally applied waveform
WATCHDOG	Monitors pulse width or period of an external signal and compares against a window of acceptable values, optionally generating an interrupt when it falls inside or outside of that window
PININT	Can generate an interrupt request on an active edge applied to a timer pin
IDLE	Idle; no activity

Period, Width and Delay Register Interaction

When the timer is started, writes to the buffer registers are immediately copied through to the double-buffered period, pulse width, and delay registers. These values are then ready for use in the first timer period. When a timer is already running, software can write new values to the `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` registers. The written values are buffered and do not update into the registers until the end of the current period. (The update occurs when the value in the `TIMER_TMR[n]_CNT` register equals the value in the `TIMER_TMR[n]_PER` register.)

If new values are not written to these registers, the value from the previous period is reused. Writes to these registers are atomic; it is not possible for the high word to be written without the low word also being written. Values written to the period, pulse width, and delay registers are always stored in the buffer registers. Reads from the same register always return the current, active value of period, pulse width, or delay value. Written values are not readback until they become active.

The usage of the `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` registers varies, depending on the mode of the timer specified by the `TIMER_TMR[n]_CFG.TMODE` bits. See the *Usage of the Period, Width, and Delay Registers in Different Timer Modes* table for more information.

Table 20-7: Usage of the Period, Width, and Delay Registers in Different Timer Modes

Timer Mode	<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_DLY</code>
IDLE	Not writable	Not writable	Not writable

Table 20-7: Usage of the Period, Width, and Delay Registers in Different Timer Modes (Continued)

Timer Mode	<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_DLY</code>
WATCHDOG	Update on-the-fly. New value takes effect either at timer start or when an asserting edge on the input signal is sensed.	Read-only. Retains value of last measured width or period of the input signal.	Update on-the-fly. New value takes effect either at timer start or when an asserting edge on the input signal is sensed.
WIDCAP	Read-only. Period value captured at the appropriate time and updated from its buffer register simultaneously with the Width register.	Read-only. Width value captured at the appropriate time and updated from its buffer register simultaneously with the Period register.	Not used
PWMOUT	Update on-the-fly. New value takes effect either at timer start or at the end of the current period. A write followed by immediate read returns the current operating values.	Update on-the-fly. New value takes effect either at timer start or at the end of the current period. A write followed by immediate read returns the current operating values.	Update on-the-fly. New value takes effect either at timer start or at the end of the current period. A write followed by immediate read returns the current operating values.
EXTCLK	Can be updated on-the-fly.	Not used	Not used
PININT	Not used	Not used	Not used

If any of the period, pulse width, and delay registers are not used, then programs cannot write into that register. For example, in WIDCAP mode, the delay registers are not used. So, the program is not allowed to write any value to the `TIMER_TMR[n]_DLY` register. To prevent undesired operation, program the `TIMER_TMR[n]_CFG.TMODE` bits before programming the period, width, or delay registers.

If a program changes the `TIMER_TMR[n]_CFG.TMODE` bits from a status register to writable register (for example in PWMOUT mode), hardware does not clear these registers. These values are automatically overwritten by new values specified by software.

In PWMOUT mode with small periods, there may not be enough time between updates from the buffer registers to write these registers. The next period can use one old value and one new value. To prevent (width + pulse delay) > period errors, write the width and delay registers before the period register when decreasing the values. Write the period register before the width and delay registers when increasing the value.

Single-Pulse PWMOUT Mode

In single-pulse PWMOUT mode, the timer generates a single pulse on the `TIMER_TMR[nn]` pin. This mode is frequently used to implement a precise delay, often with generating an output trigger. The timer module uses the value in the `TIMER_TMR[n]_DLY` register to control the assertion of a pulse. The value in the `TIMER_TMR[n]_WID` register defines the pulse width. The `TIMER_TMR[n]_PER` is not used and cannot be written in this mode. After completion of the pulse, the timer is automatically stopped, and optionally generates an interrupt. The timer uses the `TIMER_TMR[n]_CFG.PULSEHI` bit to control pulse polarity.

The timer can be configured to generate a data interrupt request after satisfying various conditions specified by the `TIMER_TMR[n]_CFG.IRQMODE` bits.

It is not necessary to clear the relevant `TIMER_RUN` bit to stop the timer cleanly. At the end of the pulse, the timer stops automatically and the corresponding `TIMER_RUN` bit is cleared. To generate multiple discrete pulses (as opposed to a continuous PWM waveform), write a 1 to the appropriate `TIMER_RUN` bit, and wait for the timer to stop. Then, write another 1 to the same `TIMER_RUN` bit.

Continuous PWMOUT Mode

In continuous PWMOUT mode, the timer generates a repetitive pulse with a well-defined period, duty cycle, and pulse position. The `TIMER_TMR[n]_DLY`, `TIMER_TMR[n]_PER`, and `TIMER_TMR[n]_WID` registers are programmed with the values of the required PWM pulse. After the timer is started, the counter counts towards the value programmed in the `TIMER_TMR[n]_PER` register. Initially, the `TIMER_TMR[nn]` pin remains in a deasserted state. The pin toggles to an asserted state when the value in the `TIMER_TMR[n]_CNT` register equals the value in the `TIMER_TMR[n]_DLY` register.

The timer can control the assertion sense of the `TIMER_TMR[nn]` pin with the `TIMER_TMR[n]_CFG.PULSEHI` bit. The `TIMER_TMR[nn]` pin holds this value for the number of clock cycles specified in the `TIMER_TMR[n]_WID` register. Then, the pin deasserts and holds this value until the completion of the programmed period. The same waveform is generated repeatedly until the timer is disabled.

The timer can be configured to generate a data interrupt request after satisfying any of various conditions specified by the `TIMER_TMR[n]_CFG.IRQMODE` bits.

It is important to guarantee that the programmed period is greater than or equal to the sum of width and delay. Similarly, delay must be less than period. Violating either of these criteria results in an unpredictable waveform on the `TIMER_TMR[nn]` pin until the situation is rectified by writing proper values to these registers.

The maximum frequency possible to generate on the `TIMER_TMR[nn]` pin is achieved by setting `TIMER_TMR[n]_PER` to 2 and `TIMER_TMR[n]_WID` to 1. This operation makes the `TIMER_TMR[nn]` pin toggle each `SCLK0` clock cycle (assuming the timer is configured to clock internally), producing a duty cycle of 50%.

When the `TIMER_STOP_CFG.TMR[nn]` bit of a timer is 0, the timer treats a stop operation as a stop-is-pending condition. When terminated with this setting, the timer automatically completes the current waveform and then stops cleanly, remaining in a deasserted state. This functionality prevents truncation of the current pulse and unwanted PWM patterns at the `TIMER_TMR[nn]` pin. The processor can determine when the timer stops running by polling the corresponding `TIMER_RUN.TMR[nn]` bit until it reads 0 or by waiting for the last interrupt (if enabled).

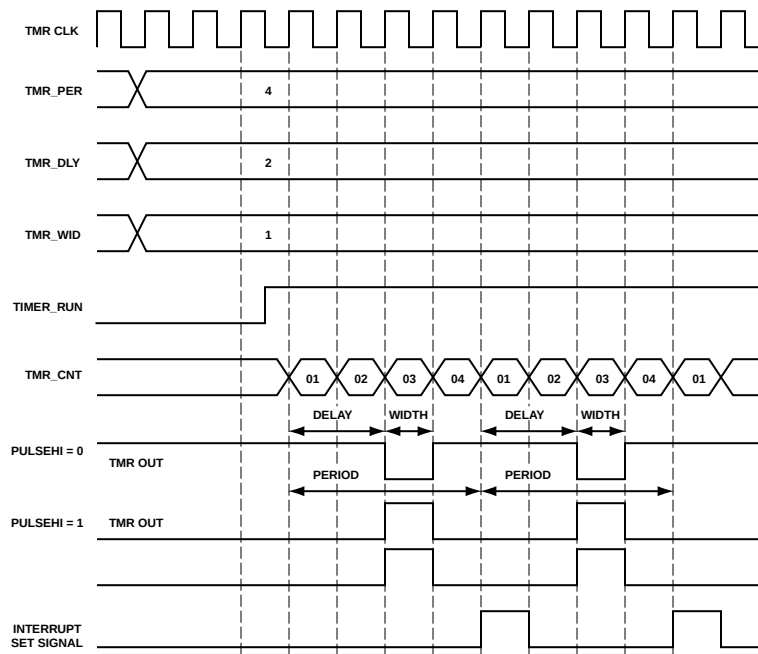


Figure 20-2: Signal Generation in Continuous PWMOUT Mode

The `TIMER_TMR[n]_CFG` register cannot be reconfigured until after the timer stops and the `TIMER_RUN` register reads 0.

Programs can force a timer to stop immediately in PWMOUT mode by writing a 1 to the `TIMER_STOP_CFG` register followed by writing a 1 to the `TIMER_RUN_CLR` register. (Or, a program can stop a timer by writing a 0 to the appropriate `TIMER_RUN.TMR[nn]` bit.) This operation stops the timer whether the pending stop is waiting for the end of the current period or the end of the current pulse width. The timer can use this feature to regain immediate control of a timer during an error recovery sequence.

Use this feature carefully, as it can corrupt the PWM pattern generated at the `TIMER_TMR[nn]` pin, though after such a stop the pin deasserts automatically. Each timer samples its `TIMER_RUN.TMR[nn]` bit at the end of each period. It stops cleanly at the end of the first period after the `TIMER_RUN.TMR[nn]` bit is low. A timer that is disabled and then restarted (before the end of the current period), continues to run as if nothing happened. Typically, the program disables a PWMOUT timer and then waits for it to stop itself.

Width Capture (WIDCAP) Mode

The timer uses WIDCAP mode, often called capture mode, to measure pulse widths on the `TIMER_TMR[nn]` or (`TIMER_ACI[nn]`) inputs. The polarity (active high or low) of the input signal can be selected with the `TIMER_TMR[n]_CFG.PULSEHI` bit. The *Timer Signal Flow in Width Capture Mode* figure shows the control signal flow for WIDCAP_CAP mode.

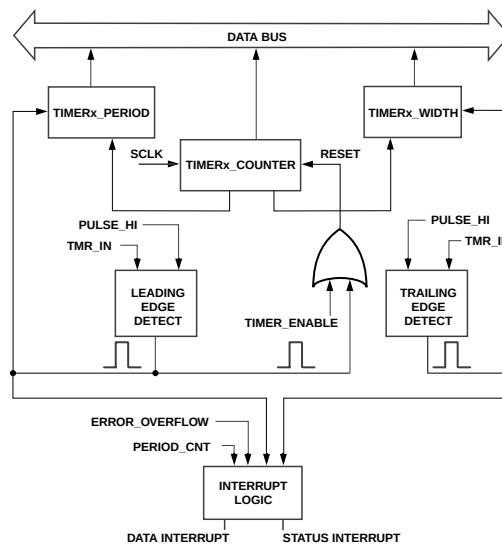


Figure 20-3: Timer Signal Flow in Width Capture Mode

NOTE: SCLK in the *Timer Signal Flow in Width Capture Mode* figure is SCLK0.

In this mode, the timer uses the `TIMER_TMR[n]_CFG.TINSEL` bit to select between the `TIMER_TMR[nn]` or (`TIMER_ACI[nn]`) input. The internally clocked timer is used to determine the period and pulse width of the externally applied rectangular waveforms.

When a timer is enabled in this mode, the timer resets the count in its `TIMER_TMR[n]_CNT` register to 0x0000 0001. It does not start counting until it detects a leading edge on the selected input pin.

When the timer detects the first leading edge, it starts incrementing. When it detects a trailing edge of a waveform, it captures the current 32-bit value of its `TIMER_TMR[n]_CNT` register into its width buffer register. At the next leading edge, the timer transfers the current 32-bit value of its `TIMER_TMR[n]_CNT` register into its period buffer register. The `TIMER_TMR[n]_CNT` register is reset to 0x0000 0001 again, and the timer continues counting and capturing until it is disabled.

In this mode, programs can measure both the pulse width and the pulse period of a waveform. The timer does not use the `TIMER_TMR[n]_DLY` register in this mode. The timer uses the `TIMER_TMR[n]_CFG.PULSEHI` bit to control the definition of leading edge and trailing edge of the `TIMER_TMR[nn] / (TIMER_ACI[nn])` pin.

In WIDCAP mode, the following events always occur at the same time as one unit:

1. The `TIMER_TMR[n]_PER` register is updated from the period buffer register.
2. The `TIMER_TMR[n]_WID` register is updated from the width buffer register.
3. The `TIMER_DATA_ILAT.TMR[nn]` bit is set (if enabled).
4. A timer data trigger pulse is generated (if enabled).

The `TIMER_TMR[n]_CFG.TMODE` bit 0 controls the point in time at which this set of events is executed. Taken together, these four events are called a measurement report. The `TIMER_STAT_ILAT` register is not set at a

measurement report. A measurement report occurs, at most, once per input signal period. The current `TIMER_TMR[n]_CNT` value is always copied to the width buffer and period buffer registers at the trailing and leading edges of the input signal, respectively. But, these values are not visible to software. A measurement report event samples the captured values into visible registers and sets the timer interrupt request to signal that the `TIMER_TMR[n]_PER` and the `TIMER_TMR[n]_WID` registers are ready to be read.

When the `TIMER_TMR[n]_CFG.TMODE` bit = `b#1011`, the measurement report occurs just after the width buffer register captures its value at a falling edge. Then, the `TIMER_TMR[n]_WID` register reports the pulse width measured in the pulse that has ended, but the `TIMER_TMR[n]_PER` register reports the pulse period measured at the end of the previous period. If only the first trailing edge has occurred, then the first period value has not yet been measured at the first measurement report. So, the period value is not valid. A read of the `TIMER_TMR[n]_PER` value in this case returns 0. See the *Example of Width Capture Deasserted Mode (TMODE=b#1011)* figure for more information.

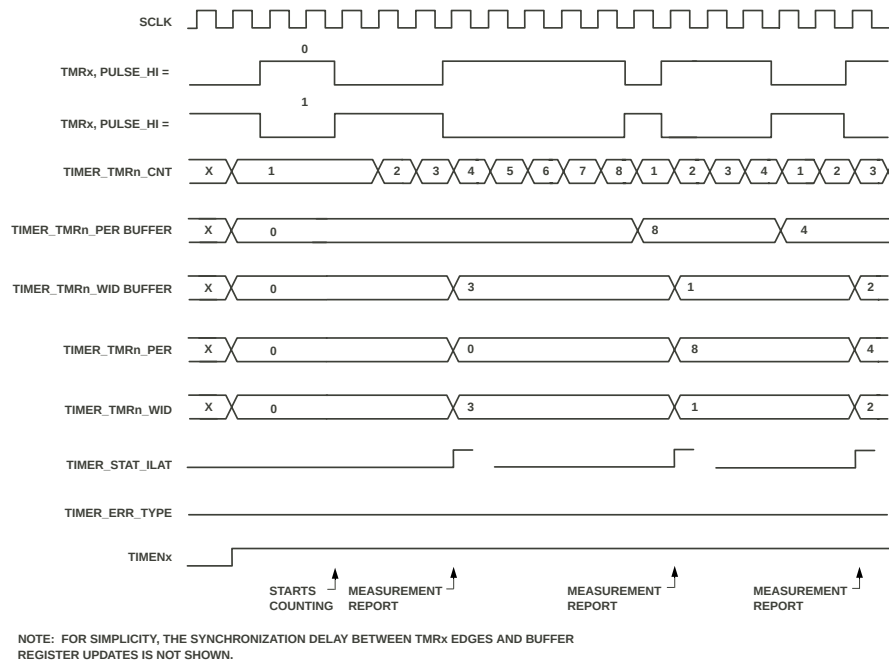


Figure 20-4: Example of Width Capture Deasserted Mode (TMODE=b#1011)

NOTE: SCLK in the *Example of Width Capture Deasserted Mode (TMODE=b#1011)* figure is SCLK0.

When the `TIMER_TMR[n]_CFG.TMODE` bit = `b#1010`, the measurement report occurs just after the period buffer register captures its value at a leading edge. Then, the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers report the pulse period and pulse width measured in the period that has ended. Refer to the *Example of Width Capture Asserted Mode (TMODE=b#1010)* figure for more information.

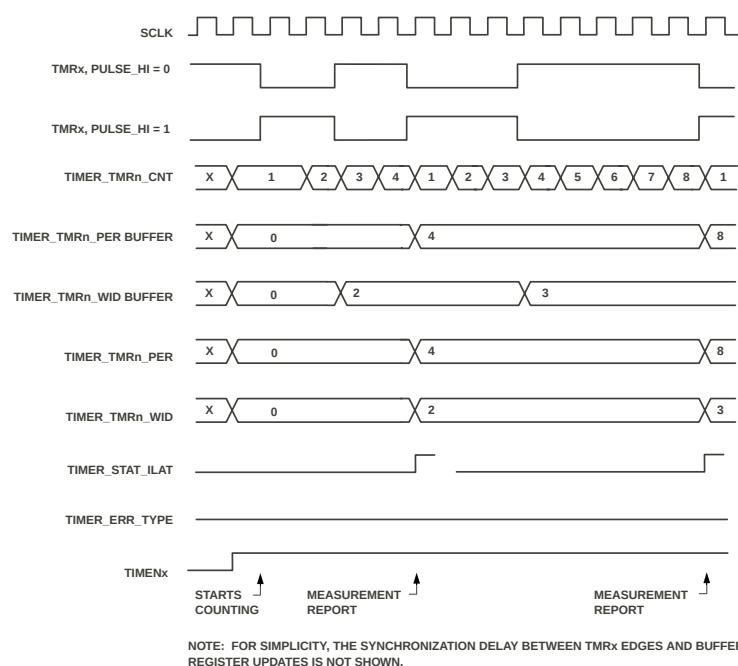


Figure 20-5: Example of Width Capture Asserted Mode (TMODE=b#1010)

NOTE: SCLK in the *Example of Width Capture Asserted Mode (TMODE=b#1010)* figure is SCLK0.

To measure the pulse width of a waveform that has only one leading edge and one trailing edge, set `TIMER_TMR[n]_CFG.TMODE = b#1011`. If `TIMER_TMR[n]_CFG.TMODE = b#1010` for this case, no period value is captured in the period buffer register. Instead, the timer generates an error report interrupt request (if enabled) when the `TIMER_TMR[n]_CNT` range is exceeded and the counter wraps around. In this case, both the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers read 0 (because no measurement report occurred to copy the value captured in the width buffer register to the `TIMER_TMR[n]_WID` register).

If using the `TIMER_TMR[n]_CFG.TMODE` bit =b#1010 mode to measure the width of a single pulse, programs can disable the timer after taking the interrupt that ends the measurement interval. If desired, restart the timer as appropriate in preparation for another measurement. This procedure prevents the timer from free-running after the width measurement and logging errors generated by the timer count overflowing.

Width Capture Mode Overflow

A timer status interrupt request (when enabled) is generated when the `TIMER_TMR[n]_CNT` register wraps around from 0xFFFF FFFF to 0 in the absence of a leading edge. At that point, the `TIMER_STAT_ILAT` bit is set and the `TIMER_ERR_TYPE` bits change to indicate a count overflow due to a period greater than the range of the counter. This indication is referred to as an error report. A data interrupt request in WIDCAP mode indicates that a new measurement is ready to be read (a measurement report). Similarly, an interrupt request on the timer status interrupt line (shared interrupt request for all timers) indicates an overflow error when generated in this mode.

The `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers are never updated at the time an overflow error is signaled. If the timer overflows and the `TIMER_TMR[n]_CFG.TMODE` bit =b#1010, the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers are not updated. If the timer overflows and the

`TIMER_TMR[n]_CFG.TMODE` bit = `b#1011`, the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers are updated only if a trailing edge is detected at a previous measurement report.

Software can count the number of error reports between measurement report interrupt requests to measure input signal periods longer than `0xFFFF FFFF`. Each error report interrupt request adds a full 2^{32} SCLK0 counts to the total for the period, but the width is ambiguous. Ensure that if software monitors only the status interrupt request, then status interrupt requests from all other timers are masked.

Refer to the *Example Timing for Width Capture Followed by Period Overflow (TMR_CFG.TMODE=b#1010)* figure. The period is `0x1 0000 0004`, but the pulse width could be either `0x0 0000 0002` or `0x1 0000 0002`.

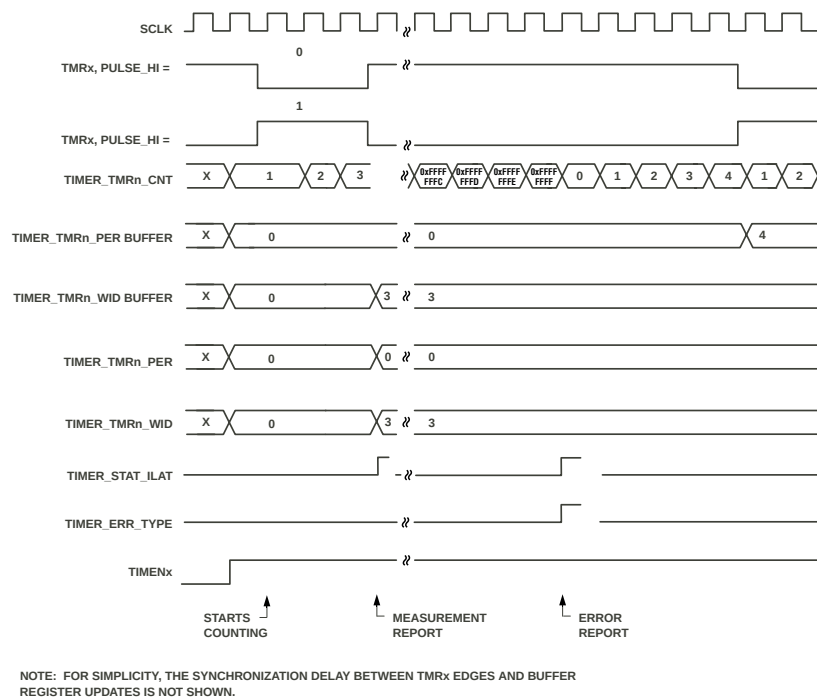


Figure 20-6: Example Timing for Width Capture Followed by Period Overflow (TMR_CFG.TMODE=b#1010)

NOTE: SCLK in the *Example Timing for Width Capture Followed by Period Overflow* figure is SCLK0.

The waveform applied to the `TIMER_TMR[nn]` (or `(TIMER_ACI[nn])`) pin is not required to have a 50% duty cycle. The minimum input low time is little more than one SCLK0 period. The minimum input high time is a little more than one SCLK0 period. (Refer to the product data sheet for details.) The maximum `TIMER_TMR[nn]` input frequency is less than $SCLK0/2$, with a 50% duty cycle. Under these conditions, the WIDCAP mode timer measures: period = 2 and pulse width = 1.

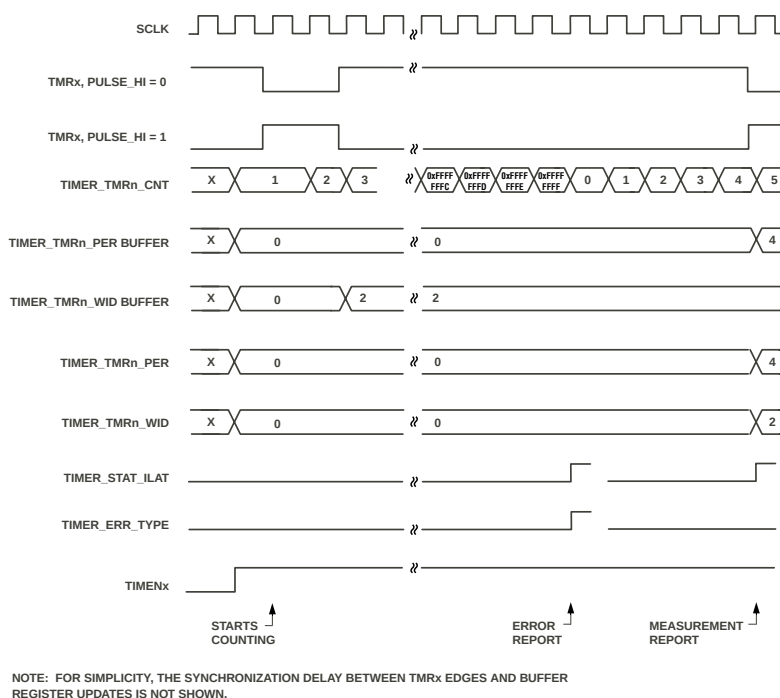


Figure 20-7: Example Timing for Width Capture Followed by Period Overflow (TMR_CFG.TMODE=b#1011)

NOTE: SCLK in the *Example Timing for Width Capture Followed by Period Overflow* figure is SCLK0.

Windowed Watchdog (WATCHDOG) Modes

In windowed watchdog (WATCHDOG) modes, a timer can take inputs from one of the `TIMER_TMR[nn]` pins or the `TIMER_ACI[nn]` pins. With this mode, the timer can monitor pulse width (width watchdog mode) or pulse period (period watchdog mode) on the input line. It also compares the measured value against a minimum required value and maximum allowed value and generates an interrupt request appropriately. The timer uses the `TIMER_TMR[n]_CFG.PULSEHI` bit to select polarity of the input signal.

The waveform applied to the input pin in watchdog mode is not required to have a 50% duty cycle. The minimum input pulse low time, high time, and total period specifications are available in the product data sheet.

Windowed Watchdog Width Mode

In windowed watchdog width mode, the timer counter monitors the pulse width of an input signal on one of the `TIMER_TMR[nn]` pins or one of the alternate clock pins (`TIMER_ACLK[nn]`). Program the minimum pulse width (p_{MIN}) in the `TIMER_TMR[n]_DLY` register and the maximum pulse width (p_{MAX}) in the `TIMER_TMR[n]_PER` register. Both values are programmed in terms of number of clock cycles (SCLK0 or alternate clock). The timer can generate an interrupt if the deasserting pulse edge occurs:

- Inside the window ($p_{MIN} < \text{pulse width} \leq p_{MAX}$), or
- Outside the window ($\text{pulse width} \leq p_{MIN}$ or $\text{pulse width} > p_{MAX}$)

After enabling the timer in this mode, it always starts counting at the asserting edge of the input signal. Any pulse that is already active when the timer is enabled is ignored.

With the `TIMER_TMR[n]_CFG.IRQMODE` bit =b#11, the timer generates an interrupt if the timed pulse width exceeds p_{MAX} , or if the pulse width is less than p_{MIN} . After attaining p_{MAX} , the pulse stays at an active level, and the counter keeps on counting until it sees a deasserting edge. When the input pulse is not active, the counter holds its current value until it again sees an asserting edge, or it restarts. An interrupt can also be generated for when the pulse occurs within the specified window condition, by setting `TIMER_TMR[n]_CFG.IRQMODE` =b#10.

In this mode, a trailing edge on the input pin triggers capturing of pulse width into the `TIMER_TMR[n]_WID` register. During the inactive portion of the input signal, the internal counter does not increment. The *Watchdog Width Mode Timing* figure shows the signal flow in this mode.

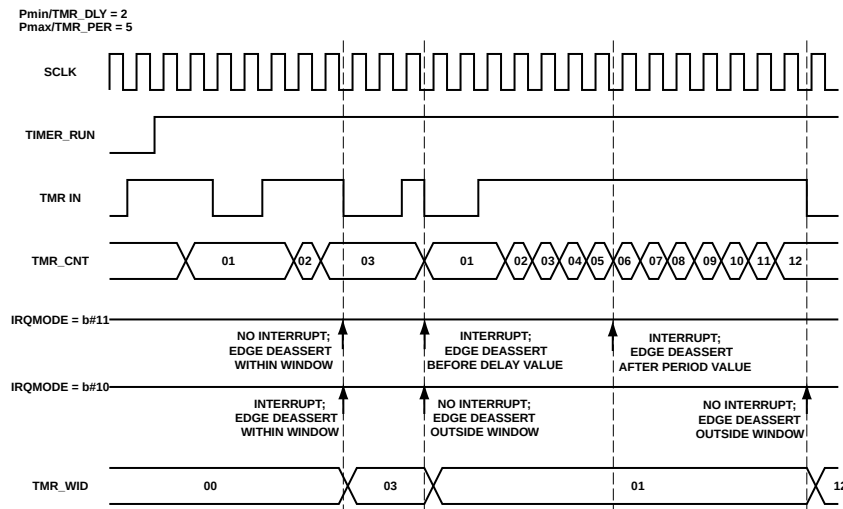


Figure 20-8: Watchdog Width Mode Timing

NOTE: SCLK in the *Watchdog Width Mode Timing* figure is SCLK0.

To check only the upper limit on pulse width (p_{MAX} but not p_{MIN}), then program p_{MIN} as 0 or 1. In such a case, it is better to use `TIMER_TMR[n]_CFG.IRQMODE` =b#11. With `TIMER_TMR[n]_CFG.IRQMODE` = b#10, a pulse width of 1 clock cycle results in an interrupt. For details, see the *Windowed Watchdog Width Mode Interpretation* table.

Table 20-8: Windowed Watchdog Width Mode Interpretation

Timer Delay	Timer Period	Incoming Pulse Width	IRQMODE= b#10	IRQMODE= b#11	Error Interrupt?
0 or 1	Anything ≥ 1	PW = 1	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		$PW \leq TMR_PER$	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		$PW > TMR_PER$	No Interrupt	Interrupt when pulse width exceeds Pmax (Period Register) Value	No Error Interrupt
> 1 but $\leq$ (Period -1)	Anything > 1	$PW \leq TMR_DLY$	No Interrupt	Interrupt at deasserting edge of input signal	No Error Interrupt
		$TMR_DLY < PW \leq TMR_PER$	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		$PW > TMR_PER$	No Interrupt	Interrupt when pulse width exceeds Pmax (Period Register) Value	No Error Interrupt
$\geq$ Period	-	$PW \leq TMR_PER$	Undefined	Undefined	No Error Interrupt
-	-	$PW > TMR_PER$	Undefined	Undefined	b#11 Error Type
-	0	-	Undefined	Undefined	b#10 Error Type

Windowed Watchdog Period Mode

In this mode, the timer monitors the number of clock cycles between two consecutive rising or falling edges of an input signal on one of the `TIMER_TMR[nn]` or `TIMER_ACI[nn]` pins. Program the required minimum number of clock cycles (t_{MIN}) in the `TIMER_TMR[n]_DLY` register and the required maximum allowed number of clock cycles (t_{MAX}) in the `TIMER_TMR[n]_PER` register. Both values are programmed in terms of number of clock cycles (SCLK0) or alternate time clock (`TIMER_ACLK[nn]`). The timer can generate an interrupt when two consecutive occurrences of an active edge are:

- Within a specified window ($t_{MIN} < \text{Pulse Period} \leq t_{MAX}$), or
- Outside a specified window ($\text{pulse width} \leq t_{MIN}$ or $t_{MAX} < \text{pulse width}$)

When the `TIMER_TMR[n]_CFG.IRQMODE` bit = b#11 and the pulse period $> t_{MAX}$ or is $\leq t_{MIN}$, the timer generates an interrupt (if unmasked). After attaining the t_{MAX} value, the counter keeps on counting until it sees an active edge on the input line. An interrupt can also be generated for when the pulse occurs within the specified

window condition, by setting `TIMER_TMR[n]_CFG.IRQMODE = b#10`. Refer to the *Watchdog Period Mode Timing* figure for timer functionality in period watchdog mode.

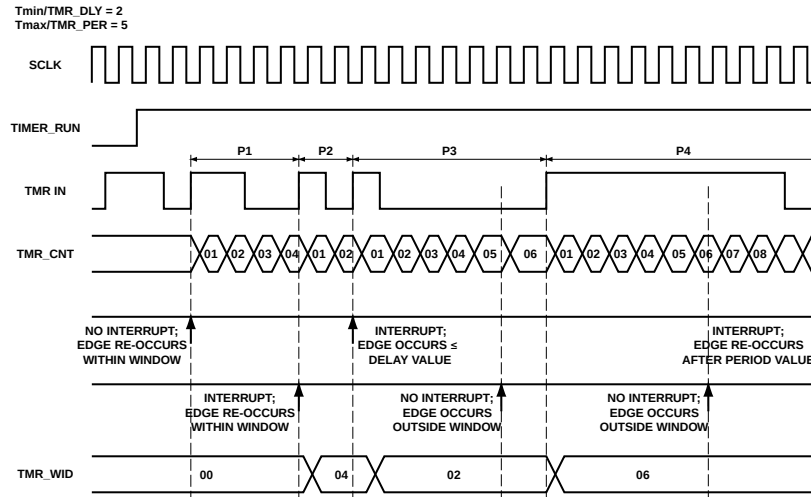


Figure 20-9: Watchdog Period Mode Timing

NOTE: SCLK in the *Watchdog Period Mode Timing* figure is SCLK0.

To check only the upper limit on period (the t_{MAX} value, not the t_{MIN} value), program t_{MIN} as 0 or 1. For details, refer to the *Windowed Watchdog Period Mode Interpretation* table.

Table 20-9: Windowed Watchdog Period Mode Interpretation

Timer Delay	Timer Period	Incoming Pulse Width	IRQMODE=b#10	IRQMODE =b#11	Error Interrupt?
0 or 1	Anything ≥ 2	Pulse Period $\leq$ TMR_PER	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		Pulse Period $>$ TMR_PER	No Interrupt	Interrupt when pulse period crosses Pmax (Period Register) value	No Error Interrupt
> 1 but $\leq$ Period -1	Anything ≥ 2	Pulse Period $\leq$ TMR_DLY	No Interrupt	Interrupt at deasserting edge of input signal	No Error Interrupt
		TMR_DLY $<$ Pulse Period $\leq$ TMR_PER	Interrupt at deasserting edge of input signal	No Interrupt	No Error Interrupt
		Pulse Period $>$ TMR_PER	No Interrupt	Interrupt when pulse width exceeds Pmax (Period Register) value	No Error Interrupt

Table 20-9: Windowed Watchdog Period Mode Interpretation (Continued)

Timer Delay	Timer Period	Incoming Pulse Width	IRQMODE=b#10	IRQMODE =b#11	Error Interrupt?
$\geq$ Period	-	Pulse Period < TMR_PER	Undefined	Undefined	No Error Interrupt
		Pulse Period $\geq$ TMR_PER	Undefined	Undefined	b#11 Error Type
-	0 or 1	-	Undefined	Undefined	b#10 Error Type

Pin Interrupt (PININT) Mode

In PININT mode, any active edges on either the `TIMER_TMR[n]` pin or the `(TIMER_ACI[nn])` pin can cause an edge-based interrupt, if enabled. (The timer uses the `TIMER_TMR[n]_CFG.TINSEL` register to select the pin). The event on the input pin can set the `TIMER_DATA_ILAT.TMR[nn]` bit and issue a system interrupt request. Program the `TIMER_TMR[n]_CFG.PULSEHI` bit to change active edge polarity.

Since the interrupt request is generated in the SCLK0 clock domain, the width of the input signal must be more than one SCLK0 period. Along with generating the interrupt request, the timer also generates a trigger pulse (configured using the `TIMER_TRG_MSK` register). Due to the configuration of polarity, glitches can cause the generation of an undesired interrupt request at the input. To avoid this problem, programs must ensure that interrupt requests are unmasked only after configuring the desired polarity.

External Clock (EXTCLK) Mode

The timer uses EXTCLK mode, sometimes referred to as the counter mode, to count external events (signal edges), on either the `TIMER_TMR[n]` or `(TIMER_ACI[nn])` input pin. The timer works as a counter clocked by an external source (the signal at the pin), which can be asynchronous to SCLK0. The current count in the `TIMER_TMR[n]_CNT` register represents the number of leading-edge events detected. The `TIMER_TMR[n]_PER` register is programmed with the value of the maximum timer external count before stopping or issuing an interrupt request or trigger.

The `TIMER_TMR[n]_CFG.PULSEHI` bit determines the polarity of the leading edge on the input pin. The timer uses the `TIMER_TMR[n]_CFG.TINSEL` bit to select whether the event is counted on the `TIMER_TMR[nn]` or on the `(TIMER_ACI[nn])` pin. The `TIMER_STAT_ILAT.TMR[nn]` and `TIMER_ERR_TYPE` bits are set if *one* of these conditions is met:

- `TIMER_TMR[n]_CNT` wraps around from 0xFFFF FFFF to 0
- The period = 0 at startup
- `TIMER_TMR[n]_CNT` register rolls over (from count = period to count = 0x1)

The `TIMER_TMR[n]_WID` and `TIMER_TMR[n]_DLY` registers are unused in this mode and must not be written.

The *EXTCLK Mode Control Flow* figure shows a flow diagram for EXTCLK mode.

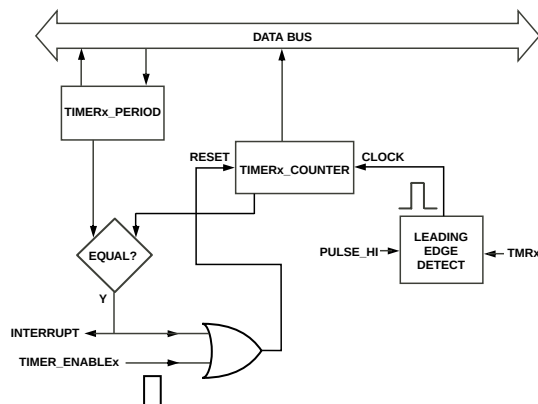


Figure 20-10: EXTCLK Mode Control Flow

The waveform applied to the input pin is not required to have a 50% duty cycle. The minimum input pulse low time, high time, and total period specifications are available in the product data sheet. Program the period to any value from 1 to $(2^{32} - 1)$, inclusive.

After the timer has started, it resets the `TIMER_TMR[n]_CNT` register to 0x0 and then waits for the first leading edge on the input pin. This edge causes `TIMER_TMR[n]_CNT` to be incremented to the value 0x1, and every subsequent leading edge increments it by one. After the `TIMER_TMR[n]_CNT` register reaches the value programmed in the `TIMER_TMR[n]_PER` register, the corresponding `TIMER_DATA_ILAT` bit is set, and an interrupt and trigger are both generated (if enabled). The next leading-edge reloads the `TIMER_TMR[n]_CNT` register with 0x1, and the timer continues counting until it is disabled.

GP Timer Programming Concepts

Using the features, operating modes, and event control for the GP timer to their greatest potential requires an understanding of some GP timer-related concepts.

Setting Up Constantly Changing Timer Conditions

This task shows how to use different period, pulse width, and delay settings for each of the first three timer periods after the timer starts.

1. Program the first set of `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` register values.
2. Enable the timer using the `TIMER_RUN` register.
3. Immediately program the second set of `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` register values, as needed.
4. Wait for the first timer interrupt request.
5. Program the third set of `TIMER_TMR[n]_PER`, `TIMER_TMR[n]_WID`, and `TIMER_TMR[n]_DLY` register values.

Each new setting is then programmed when the preceding timer interrupt request is received.

Configuring, Enabling, and Disabling One or More Timers

1. Configure the relevant timers for the operating mode and other properties using the `TIMER_TMR[n]_CFG` register.
2. Write a 1 to the representative `TIMER_RUN.TMR[nn]` bit. Or, use the `TIMER_RUN_SET` register to avoid disturbing the settings of other timers that are not going through configuration.

The timer is enabled and operating.

3. To stop one or more timers, first program the `TIMER_STOP_CFG` register to determine whether to stop immediately or gracefully upon receiving a stop command.

ADDITIONAL INFORMATION: PWMOUT modes are the only modes where a timer can be configured for graceful termination.

4. Write a 0 to the representative `TIMER_RUN.TMR[nn]` bits to stop the timer according to their `TIMER_STOP_CFG` settings. Alternately, write a 1 to the appropriate `TIMER_RUN_CLR.TMR[nn]` bits to avoid disturbing the settings of other timers that are not terminating.

The timers stop.

Configuring Timer Data and Status Interrupts

1. Configure the `TIMER_TMR[n]_CFG.IRQMODE` bit field with the desired interrupt properties.
2. Unmask the interrupt source.
3. Set the `TIMER_TMR[n]_CFG.IRQMODE` field but leave the interrupt masked at the system level to poll the `TIMER_DATA_ILAT.TMR[nn]` bit of the timer without generating an interrupt.
4. Use the `TIMER_STAT_IMSK` register to generate interrupt requests by overflow or error conditions (incorrect programming values). The timer uses the `TIMER_STAT_ILAT.TMR[nn]` bits to report interrupt errors, when the timer status interrupt source is unmasked.
5. To poll the `TIMER_STAT_ILAT.TMR[nn]` bit of the timer without generating an interrupt, unmask the corresponding bit in the `TIMER_STAT_IMSK` register, but leave the interrupt masked at the system level.

Configuring the Timer as a Trigger Slave

The timer can be configured to either start or stop or toggle between these two states on the input trigger pulse depending on the configuration of the `TIMER_TMR[n]_CFG.TGLTRIG` and `TIMER_TMR[n]_CFG.SLAVETRIG` bits.

- If `TIMER_TMR[n]_CFG.TGLTRIG` bit =0 and `TIMER_TMR[n]_CFG.SLAVETRIG` bit =1 then the trigger pulse starts timer, if it is stopped.

- If `TIMER_TMR[n]_CFG.TGLTRIG` bit =0 and `TIMER_TMR[n]_CFG.SLAVETRIG` bit =0 then the trigger pulse stops timer, if it is running.
- When `TIMER_TMR[n]_CFG.TGLTRIG` bit =0, the trigger pulse has no effect when the timer is already in the requested state.

If `TIMER_TMR[n]_CFG.TGLTRIG` bit is 1, the trigger pulse starts the timer if it is stopped, or stops the timer if it is running. The `TIMER_TMR[n]_CFG.SLAVETRIG` bit has no effect on trigger mechanism. In continuous PWMOUT mode, the timer stops gracefully or abruptly depending on the stop mechanism programmed in the `TIMER_STOP_CFG_CLR` register. In other modes, the timer stops immediately.

Ordered Trigger Toggle Mode

The timer can be configured to either start or stop the timer on the input trigger pulse depending on the configuration of the `TIMER_TMR[n]_CFG.TGLTRIG` and `TIMER_TMR[n]_CFG.ORDTGLTRIG` bits.

- If `TIMER_TMR[n]_CFG.TGLTRIG` bit is 1 and `TIMER_TMR[n]_CFG.ORDTGLTRIG` bit is 0, then the input trigger pulse on trigger slave 0 or trigger slave 1 starts the timer if it is stopped, or stops the timer if it is running.
- If `TIMER_TMR[n]_CFG.TGLTRIG` bit is 1 and `TIMER_TMR[n]_CFG.ORDTGLTRIG` bit is 1, then the input trigger pulse on trigger slave 0 starts timer if it is stopped. If the timer is running, it stops when the input trigger pulse on trigger slave 1 is detected.

Trigger slave 0 cannot toggle the timer from the run to halt state. Trigger slave 1 cannot toggle the timer from the halt to run state.

NOTE: The timer state is not toggled by writes to the `TIMER_RUN`, `TIMER_RUN_SET`, and `TIMER_RUN_CLR` registers in this mode.

Using the Timer Broadcast Feature

The broadcast feature provides a means to update period, width, and delay registers simultaneously across more than one timer.

1. Enable the appropriate broadcast bits (`TIMER_TMR[n]_CFG.BPEREN`, `TIMER_TMR[n]_CFG.BWIDEN` are `TIMER_TMR[n]_CFG.BDLYEN`) for the timers involved in the broadcast. The use of these bits depends on which broadcast registers the timer uses (`TIMER_BCAST_PER`, `TIMER_BCAST_WID`, or `TIMER_BCAST_DLY`).
2. Program the `TIMER_BCAST_PER` register (for example), to broadcast the period setting across the multiple timers enabled.

The enabled timers load their `TIMER_TMR[n]_PER` registers with the value specified in the `TIMER_BCAST_PER` register.

3. Repeat Step 2 as needed for the `TIMER_BCAST_WID` and `TIMER_BCAST_DLY` register settings.

Timer Illegal States

The following sections use these definitions:

- **Startup.** The first clock period during which the timer counter is running after the timer is started by writing the `TIMER_RUN` register.
- **Rollover.** The time when the current count in `TIMER_TMR[n]_CNT` matches the value in `TIMER_TMR[n]_PER` and the counter is reloaded with the value 1.
- **Overflow.** The timer counter was incremented instead of doing a rollover when it was holding the maximum count value of 0xFFFF FFFF. The counter does not have a large enough range to express the next greater value and so it erroneously loads a new value of 0x0000 0000.
- **Unchanged.** No new error.

When the `TIMER_ERR_TYPE` register is designated unchanged, it displays the previously reported error code orb# 00 when there has been no error since this timer was enabled.

When the `TIMER_STAT_ILAT` register is unchanged, it reads 0 when there has been no error or overflow since this timer was enabled. Or, it reads 0 if software has performed a W1C to clear any previous error. If software has not acknowledged a previous error, the `TIMER_STAT_ILAT` register reads 1. Software can read the `TIMER_STAT_ILAT` register to check for errors. If a particular bit of a timer is set in this register, software can then read the `TIMER_ERR_TYPE` register for more information. Once detected, software can W1C the appropriate `TIMER_STAT_ILAT` bit to acknowledge the error.

Read the following tables as:

- In mode ___ at event __,
- if `TIMER_TMR[n]_PER` is ___ and `TIMER_TMR[n]_WID` is ___ and `TIMER_TMR[n]_DLY` is __,
- then `TIMER_ERR_TYPE` is ___ and `TIMER_STAT_ILAT` is ___.

Startup error conditions do not prevent the timer from starting. Similarly, overflow and rollover error conditions do not stop the timer. Illegal cases can cause unwanted behavior of the `TIMER_TMR[nn]` pins.

NOTE: For PININT mode, the timer does not use error functionality.

Continuous PWMOUT Mode

Table 20-10: Startup Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
≤ 1	Anything other than period[8]	Anything	Anything	b#10	Set

Table 20-10: Startup Event (Continued)

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
≥ 2	Anything including 0, excluding TMR_PER value	Anything including 0	$\leq \text{PERIOD}$	Unchanged	Unchanged
	Anything including 0	Anything including 0	$> \text{PERIOD}$	Unchanged[9] (Detected at rollover)	Unchanged (Detected at rollover)
	Anything	Anything	$> 2^{32} - 1$	b#11	Set
	=Period	=0	=Period	No error	Unchanged (Detected at rollover)

Table 20-11: Rollover Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
≥ 1	Anything	Anything	Anything	b#10[timer running at SCLK0] b#11 [timer running at ALT_CLKx]	Set
≥ 2	Anything including 0, excluding TMR_PER value	Anything including 0	$\leq \text{PERIOD}$	Unchanged	Unchanged
	Anything including 0, excluding TMR_PER value	Anything > 0	$> \text{PERIOD}$	b#11	Set
	Anything	Anything	$> 2^{32} - 1$	b#11	Set
	= Period[10]	=0	=Period	b#11	Set
	>Period	=0	>Period	Unchanged	Unchanged

Table 20-12: Overflow Event (On TMR_PER Register Programming Error Only)

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
Anything	Anything	Anything	Anything	b#01	Set

Single Pulse PWMOUT Mode

For single pulse PWMOUT mode, there are no rollover events.

Table 20-13: Startup Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_STAT_ILAT</code> (if enabled)	<code>TIMER_STAT_ILAT</code> (if enabled)
N/A	Anything	$== 0$	Anything	b#11[11]	Set
N/A	Anything including 0	≥ 1	$> 2^{32} - 1$	Unchanged	Unchanged
N/A	Anything including 0	≥ 1	$> 2^{32} - 1$	b#11	Set

Table 20-14: Overflow Event (On another error, such as $DELAY + WIDTH \geq 2^{32} - 1$)

	<code>TIMER_TMR[n]_DLY</code>		<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>		<code>TIMER_STAT_ILAT</code> (if enabled)
Anything	Anything	Anything	Anything	b#01	Set

WIDCAP Mode

For WIDCAP mode, the `TIMER_TMR[n]_PER` and `TIMER_TMR[n]_WID` registers are read-only and the `TIMER_TMR[n]_DLY` register is not used. Therefore, no startup or rollover errors are possible.

Table 20-15: Overflow Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
Anything	N/A	Anything	N/A	b#01	Set

EXTCLK Mode

Table 20-16: Startup Event

<code>TIMER_TMR[n]_PER</code>	<code>TIMER_TMR[n]_DLY</code>	<code>TIMER_TMR[n]_WID</code>	<code>TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY</code>	<code>TIMER_ERR_TYPE</code>	<code>TIMER_STAT_ILAT</code> (if enabled)
$= 0$	N/A	N/A	N/A	b#01	Set
≥ 1	N/A	N/A	N/A	Unchanged	Unchanged

Table 20-17: Rollover Event

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
=0	N/A	N/A	N/A	b#01	Set
≥1	N/A	N/A	N/A	Unchanged	Unchanged

Table 20-18: Overflow Event (On TMR_PER Register = 0 Only)

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
Anything	N/A	N/A	N/A	b#01	Set

WATCHDOG Events

Table 20-19: Startup Event

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
≤ Allowed MIN[12]	Anything < PERIOD	N/A	N/A	b#01	Set
> Allowed MIN	Anything < PERIOD	N/A	N/A	Unchanged	Unchanged
> Allowed MIN	Anything ≥ PERIOD	Refer to WATCHDOG Mode tables			

Table 20-20: Rollover Event

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
≤ Allowed MIN[10]	Anything < PERIOD	N/A	N/A	b#01	Set
> Allowed MIN	Anything	N/A	N/A	Unchanged	Unchanged
> Allowed MIN	Anything ≥ PERIOD	Refer to WATCHDOG Mode tables			

Table 20-21: Overflow Event

TIMER_TMR[n]_PER	TIMER_TMR[n]_DLY	TIMER_TMR[n]_WID	TIMER_TMR[n]_WID + TIMER_TMR[n]_DLY	TIMER_ERR_TYPE	TIMER_STAT_ILAT (if enabled)
Anything	Anything	N/A	N/A	b#01	Set

ADSP-2159x TIMER Register Descriptions

General-Purpose Timer Block (TIMER) contains the following registers.

Table 20-22: ADSP-2159x TIMER Register List

Name	Description
TIMER_BCAST_DLY	Broadcast Delay Register
TIMER_BCAST_PER	Broadcast Period Register
TIMER_BCAST_WID	Broadcast Width Register
TIMER_DATA_ILAT	Data Interrupt Latch Register
TIMER_DATA_IMSK	Data Interrupt Mask Register
TIMER_ERR_TYPE	Error Type Status Register
TIMER_RUN	Run Register
TIMER_RUN_CLR	Run Clear Register
TIMER_RUN_SET	Run Set Register
TIMER_STAT_ILAT	Status Interrupt Latch Register
TIMER_STAT_IMSK	Status Interrupt Mask Register
TIMER_STOP_CFG	Stop Configuration Register
TIMER_STOP_CFG_CLR	Stop Configuration Clear Register
TIMER_STOP_CFG_SET	Stop Configuration Set Register
TIMER_TMR[n]_CFG	Timer n Configuration Register
TIMER_TMR[n]_CNT	Timer n Counter Register
TIMER_TMR[n]_DLY	Timer n Delay Register
TIMER_TMR[n]_PER	Timer n Period Register
TIMER_TMR[n]_WID	Timer n Width Register
TIMER_TRG_IE	Trigger Slave Enable Register
TIMER_TRG_MSK	Trigger Master Mask Register

Broadcast Delay Register

For timers with `TIMER_TMR[n]_CFG.BDLYEN` enabled, a write to the `TIMER_BCAST_DLY` register concurrently updates the delay (`TIMER_TMR[n]_DLY`) registers of only those timers. A read of the `TIMER_BCAST_DLY` register returns `0x00000000`, and no bus error is generated. To read back a written value, read that TMR's `TIMER_TMR[n]_DLY` register.

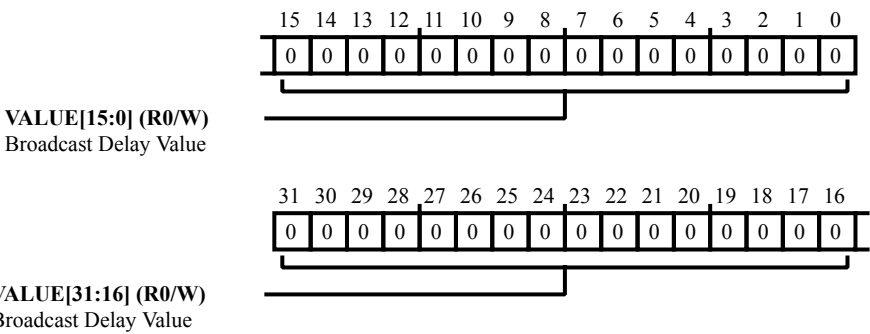


Figure 20-11: TIMER_BCAST_DLY Register Diagram

Table 20-23: TIMER_BCAST_DLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W)	VALUE	Broadcast Delay Value. A write to the <code>TIMER_BCAST_DLY.VALUE</code> bit field concurrently updates the delay (<code>TIMER_TMR[n]_DLY</code>) registers of only those timers. A read of the <code>TIMER_BCAST_DLY.VALUE</code> bit field returns <code>0x0000 0000</code> , and no bus error is generated.

Broadcast Period Register

For timers with `TIMER_TMR[n]_CFG.BPEREN` enabled, a write to the `TIMER_BCAST_PER` register concurrently updates the period (`TIMER_TMR[n]_PER`) registers of only those timers. A read of `TIMER_BCAST_PER` returns 0x00000000, and no bus error is generated. To read back a written value, read that TMR's `TIMER_TMR[n]_PER` register.

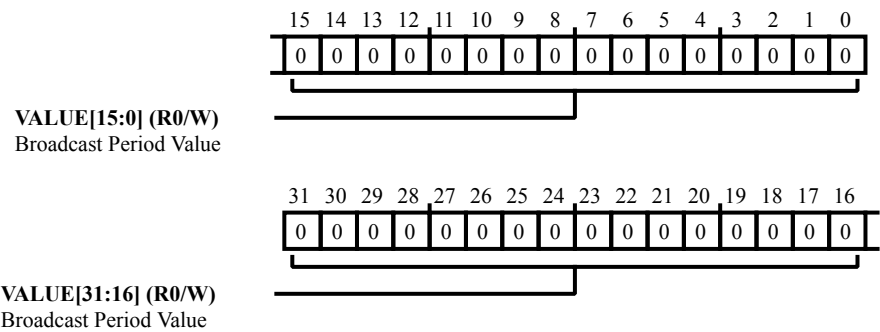


Figure 20-12: TIMER_BCAST_PER Register Diagram

Table 20-24: TIMER_BCAST_PER Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W)	VALUE	Broadcast Period Value. A write to the <code>TIMER_BCAST_PER.VALUE</code> bit field concurrently updates the period (<code>TIMER_TMR[n]_PER</code>) registers of only those timers. A read of the <code>TIMER_BCAST_PER.VALUE</code> bit fields returns 0x0000 0000, and no bus error is generated.

Broadcast Width Register

For timers with `TIMER_TMR[n]_CFG.BWIDEN` enabled, a write to the `TIMER_BCAST_WID` register concurrently updates the width (`TIMER_TMR[n]_WID`) registers of only those timers. A read of the `TIMER_BCAST_WID` register returns `0x00000000`, and no bus error is generated. To read back a written value, read that TMR's `TIMER_TMR[n]_WID` register.

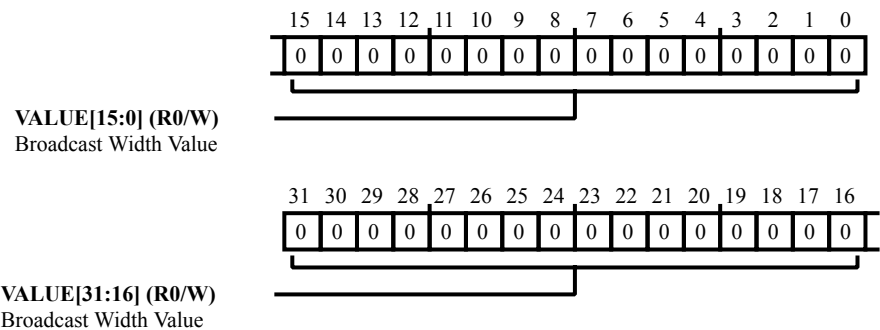


Figure 20-13: `TIMER_BCAST_WID` Register Diagram

Table 20-25: `TIMER_BCAST_WID` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R0/W)	VALUE	Broadcast Width Value. A write to the <code>TIMER_BCAST_WID.VALUE</code> bit field concurrently updates the width (<code>TIMER_TMR[n]_WID</code>) registers of only those timers. A read of the <code>TIMER_BCAST_WID.VALUE</code> bit field returns <code>0x0000 0000</code> , and no bus error is generated.

Data Interrupt Latch Register

The `TIMER_DATA_ILAT` holds the latched interrupt status for interrupt requests that have been unmasked (enabled) by the `TIMER_DATA_IMSK` register and generated according to the conditions selected by the `TIMER_TMR[n]_CFG.IRQMODE` bits. If a bit in `TIMER_DATA_ILAT` is already set and the corresponding interrupt is masked in `TIMER_DATA_IMSK`, the latch holds its old value, leaving the interrupt request asserted until it is reset by software with a W1C operation.

Note that interrupt service routines (ISRs) should clear the appropriate bits in `TIMER_DATA_ILAT` before returning from the ISR, to ensure that the interrupt is not re-issued. To make sure that no timer event is missed, the latch should be reset at the very beginning of the ISR when in EXTCLK mode.

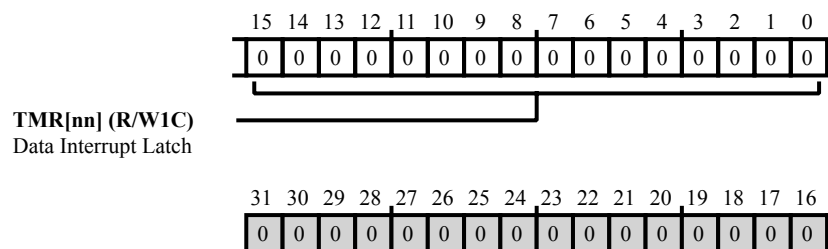


Figure 20-14: `TIMER_DATA_ILAT` Register Diagram

Table 20-26: `TIMER_DATA_ILAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W1C)	<code>TMR[nn]</code>	Data Interrupt Latch. For all <code>TIMER_DATA_ILAT.TMR[nn]</code> bits, status of =0 indicates no interrupt is latched, and status of =1 indicates a latched interrupt (indicating an unmasked interrupt request from a timer with a condition matching the one selected with corresponding <code>TIMER_TMR[n]_CFG.IRQMODE</code> bit has occurred).

Data Interrupt Mask Register

Each timer may generate a unique processor data interrupt request signal. The `TIMER_DATA_IMSK` register contains an interrupt mask for these requests, masking (disabling) or unmasking (enabling) the interrupts as programmed. The reset value of the `TIMER_DATA_IMSK` register is 0xFFFF, masking these interrupts after reset.

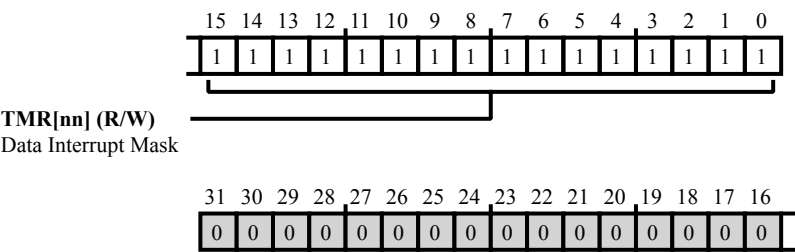


Figure 20-15: `TIMER_DATA_IMSK` Register Diagram

Table 20-27: `TIMER_DATA_IMSK` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	TMR[nn]	Data Interrupt Mask. For all <code>TIMER_DATA_IMSK.TMR[nn]</code> bits, write =0 unmask (enables) the corresponding data interrupt request, and write =1 masks (disables) the corresponding data interrupt request.

Error Type Status Register

The `TIMER_ERR_TYPE` register contains error type status bits for each timer. These bits indicate the type of error (if any) in a running timer. This register is read-only. These status bits are cleared at reset and when a particular timer is enabled.

Each time an error request interrupt is latched in the `TIMER_STAT_ILAT` register, the corresponding `TERRx` bits in the `TIMER_ERR_TYPE` register are loaded with a code that identifies the type of error that was detected. This status value is held until the next error or until a particular timer is restarted. No bus error is generated if a write is performed on the `TIMER_ERR_TYPE` register.

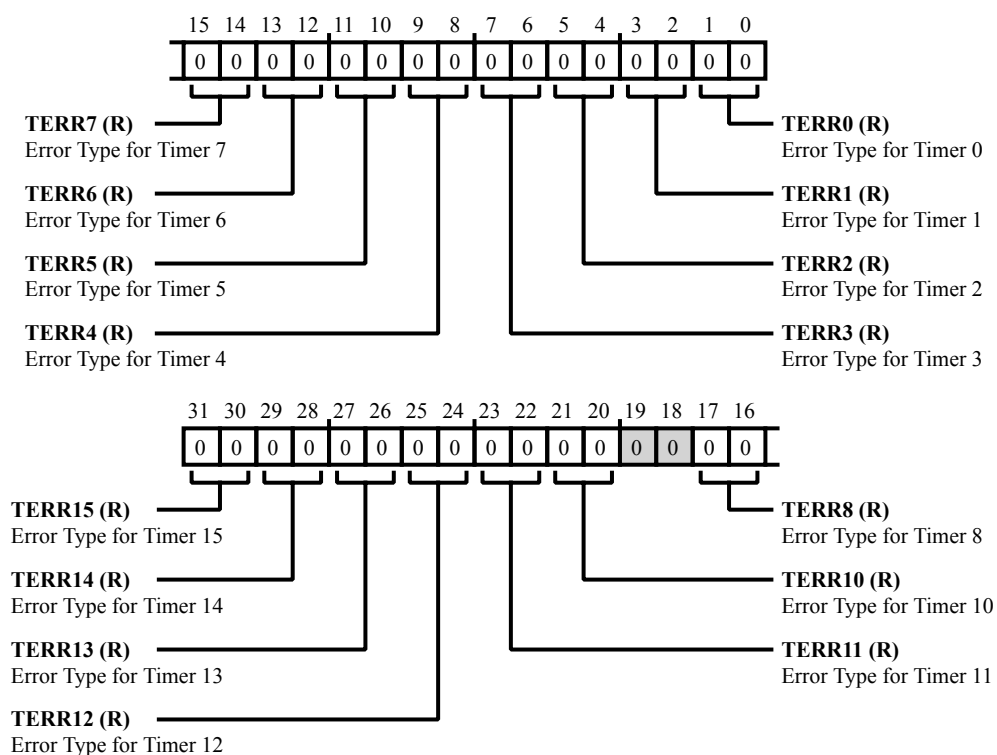


Figure 20-16: `TIMER_ERR_TYPE` Register Diagram

Table 20-28: `TIMER_ERR_TYPE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:30 (R/NW)	TERR15	Error Type for Timer 15.
		0 No Error
		1 Counter Overflow Error
		2 PER Register Programming Error
		3 WID or DLY Register Programming Error

Table 20-28: TIMER_ERR_TYPE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
29:28 (R/NW)	TERR14	Error Type for Timer 14.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
27:26 (R/NW)	TERR13	Error Type for Timer 13.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
25:24 (R/NW)	TERR12	Error Type for Timer 12.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
23:22 (R/NW)	TERR11	Error Type for Timer 11.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
21:20 (R/NW)	TERR10	Error Type for Timer 10.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
17:16 (R/NW)	TERR8	Error Type for Timer 8.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error

Table 20-28: TIMER_ERR_TYPE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
15:14 (R/NW)	TERR7	Error Type for Timer 7.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
13:12 (R/NW)	TERR6	Error Type for Timer 6.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
11:10 (R/NW)	TERR5	Error Type for Timer 5.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
9:8 (R/NW)	TERR4	Error Type for Timer 4.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
7:6 (R/NW)	TERR3	Error Type for Timer 3.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
5:4 (R/NW)	TERR2	Error Type for Timer 2.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error

Table 20-28: TIMER_ERR_TYPE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
3:2 (R/NW)	TERR1	Error Type for Timer 1.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error
1:0 (R/NW)	TERR0	Error Type for Timer 0.	
		0	No Error
		1	Counter Overflow Error
		2	PER Register Programming Error
		3	WID or DLY Register Programming Error

Run Register

The `TIMER_RUN` allows all timers to be enabled simultaneously, permitting them to run synchronously. For each timer, there is a single start/stop control bit. Writing a 1 to this bit starts the corresponding timer; writing a 0 stops the timer with mechanism specified in the timer stop configuration `TIMER_STOP_CFG` register.

The start/stop control bits can be set/reset individually or in any combination. While starting or stopping one particular timer directly with this register, software must perform a read-modify write, so the bits corresponding to other timers remain unchanged. To avoid this need, software can use the `TIMER_RUN_CLR` register.

Reading the `TIMER_RUN` register shows the start status for the corresponding timer. A 1 indicates that the timer is running.

If a timer is in run state (corresponding run bit is =1), a software write of 1 in this bit does not have any effect on the timer state. The write does not result in restarting the timer.

Note that the `TIMER_RUN` register is not used in PININT mode. PININT mode starts as soon as the `TIMER_TMR[n]_CFG.TMODE` bits are set to 111.

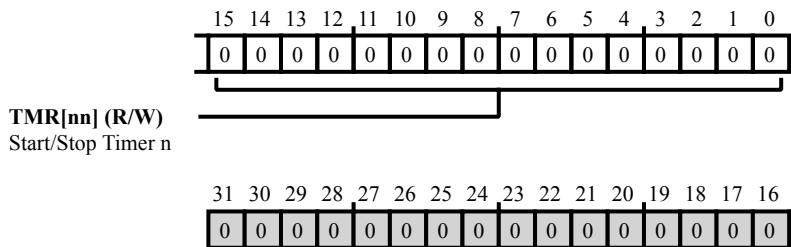


Figure 20-17: `TIMER_RUN` Register Diagram

Table 20-29: `TIMER_RUN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	<code>TMR[nn]</code>	Start/Stop Timer n. For all <code>TIMER_RUN.TMR[nn]</code> bits, write =0 for stop, and write =1 for start. Read =1 when timer is running.

Run Clear Register

The `TIMER_RUN_CLR` register is an alias register, providing a mechanism to clear a specific start/stop bit in the `TIMER_RUN` register without affecting other bits in `TIMER_RUN`. To stop a particular timer, software must write a 1 into the corresponding `TIMER_RUN_CLR` bit. Writing a 0 has no effect. Because `TIMER_RUN_CLR` is a write-only register, the result of any write to this register must be checked by reading the `TIMER_RUN` register. A read of the `TIMER_RUN_CLR` returns 0x0000.

Note that the stopping mechanism of a timer may be abrupt or graceful (after completion of current waveform period) depending on the selection in the `TIMER_STOP_CFG` register.

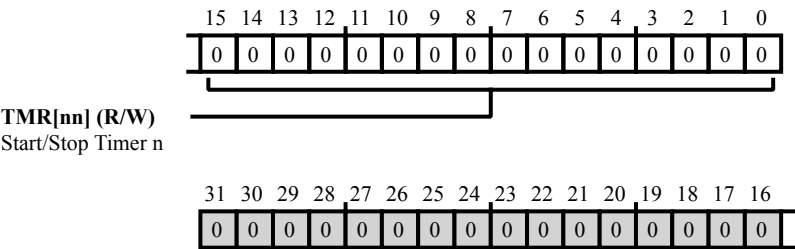


Figure 20-18: `TIMER_RUN_CLR` Register Diagram

Table 20-30: `TIMER_RUN_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R0/W1C)	TMR[nn]	RUN Clear Alias. For all <code>TIMER_RUN_CLR.TMR[nn]</code> bits, write =0 has no effect, and write =1 for stop (clearing the corresponding in start/stop bit in the <code>TIMER_RUN</code> register). Using <code>TIMER_RUN_CLR</code> to clear start/stop bits permits stopping specific timers without influencing run status of other timers.

Run Set Register

The `TIMER_RUN_SET` register is an alias register, providing a mechanism to set a specific start/stop bit in the `TIMER_RUN` register without affecting other bits in `TIMER_RUN`. To start a particular timer, software must write a 1 into the corresponding `TIMER_RUN_SET` bit. Writing a zero has no effect. For an example, to start timer 3 without affecting any other timer, write 0x0008 into `TIMER_RUN_SET`. Because `TIMER_RUN_SET` is a write-only register, the result of any write to this register must be checked by reading the `TIMER_RUN` register. A read of the `TIMER_RUN_SET` returns 0x0000.

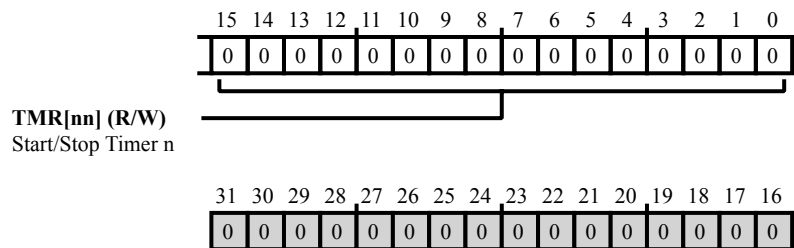


Figure 20-19: `TIMER_RUN_SET` Register Diagram

Table 20-31: `TIMER_RUN_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R0/W1S)	<code>TMR[nn]</code>	RUN Set Alias. For all <code>TIMER_RUN_SET.TMR[nn]</code> bits, write =0 has no effect, and write =1 for start (setting the corresponding start/stop bit in the <code>TIMER_RUN</code> register). Using <code>TIMER_RUN_SET</code> to set start/stop bits permits starting specific timers without influencing the run status of other timers.

Status Interrupt Latch Register

The `TIMER_STAT_ILAT` holds the latched interrupt status for error interrupt requests, indicating a timer overflow condition or indicating that prohibited programming has occurred for a timer. These interrupt status bits are sticky and are W1C. The bits in the `TIMER_STAT_ILAT` register provide information regarding each timer interrupt request source.

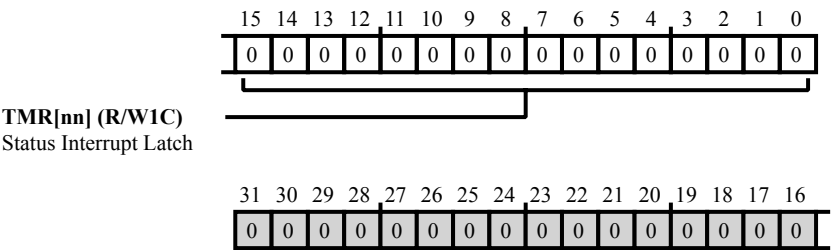


Figure 20-20: `TIMER_STAT_ILAT` Register Diagram

Table 20-32: `TIMER_STAT_ILAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W1C)	TMR[nn]	Status Interrupt Latch. For all <code>TIMER_STAT_ILAT.TMR[nn]</code> bits, status of 0 indicates no error interrupt request is latched, and status of 1 indicates a timer counter overflow or programming error interrupt request is latched.

Status Interrupt Mask Register

While each timer may generate a status interrupt request, these requests are OR'ed to generate a single status interrupt signal to the system event controller. The `TIMER_STAT_IMSK` register contains an interrupt mask for these requests, masking (disabling) or unmasking (enabling) the interrupts as programmed. The reset value of the `TIMER_STAT_IMSK` register is 0xFFFF, masking these interrupts after reset.

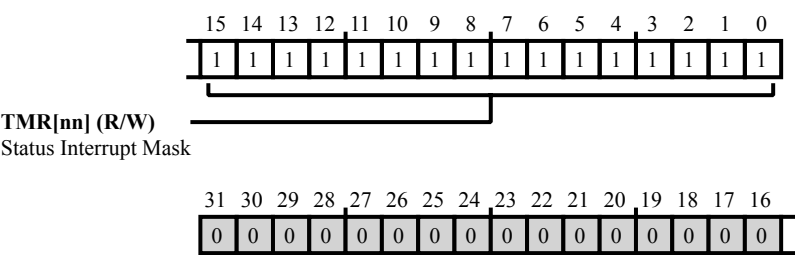


Figure 20-21: `TIMER_STAT_IMSK` Register Diagram

Table 20-33: `TIMER_STAT_IMSK` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	TMR[nn]	Status Interrupt Mask. For all <code>TIMER_STAT_IMSK.TMR[nn]</code> bits, write =0 unmask (enables) the corresponding status interrupt request, and write =1 masks (disables) the corresponding status interrupt request.

Stop Configuration Register

The `TIMER_STOP_CFG` register selects the stop mode for each timer. Timers may be stopped abruptly (immediate halt - all modes) or gracefully in PWMOUT modes (single pulse and continuous). The halt is achieved through either a write =0 to the corresponding bit in `TIMER_RUN` or a write =1 to the corresponding bit in `TIMER_RUN_CLR`. A read of `TIMER_STOP_CFG` returns the last value written.

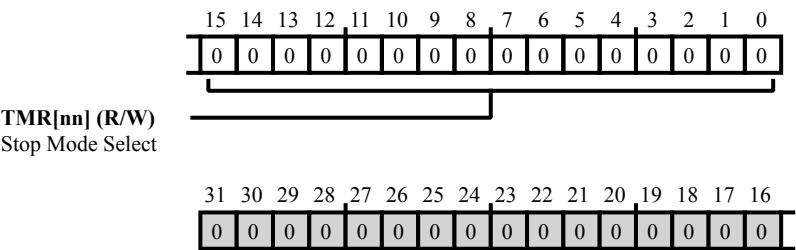


Figure 20-22: TIMER_STOP_CFG Register Diagram

Table 20-34: TIMER_STOP_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	TMR[nn]	Stop Mode Select. For all <code>TIMER_STOP_CFG.TMR[nn]</code> bits, write =0 for graceful termination (PWMOUT modes only), and write =1 for abrupt (immediate halt) on stop.

Stop Configuration Clear Register

This is an alias register, providing a mechanism to clear a specific bit in the `TIMER_STOP_CFG` register without affecting other bits in `TIMER_STOP_CFG`. To clear a bit in `TIMER_STOP_CFG`, software must write a 1 to the corresponding bit of `TIMER_STOP_CFG_CLR` register. Writing a zero has no effect. Because the `TIMER_STOP_CFG_CLR` register is a write-only register, the result of any write to this register must be checked by reading the `TIMER_STOP_CFG` register. A read of the `TIMER_STOP_CFG_CLR` register returns 0x0000.

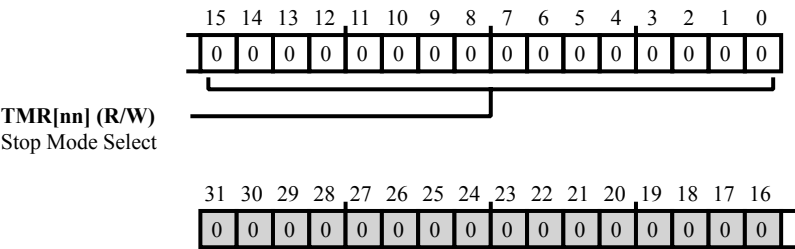


Figure 20-23: TIMER_STOP_CFG_CLR Register Diagram

Table 20-35: TIMER_STOP_CFG_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R0/W1C)	TMR[nn]	STOP_CFG Clear Alias. For all <code>TIMER_STOP_CFG_CLR.TMR[nn]</code> bits, write =0 has no effect, and write =1 for graceful stop in PWMOUT modes (clearing the corresponding stop mode select bit in the <code>TIMER_STOP_CFG</code> register). Using <code>TIMER_STOP_CFG_CLR</code> to clear stop mode bits permits configuring specific timers without influencing the stop mode configuration of other timers.

Stop Configuration Set Register

This is an alias register, providing a mechanism to set a specific bit in the `TIMER_STOP_CFG` register without affecting other bits in `TIMER_STOP_CFG`. To set a bit in the `TIMER_STOP_CFG` register, software must write a 1 to the corresponding bit of the `TIMER_STOP_CFG_SET` register. Writing a zero has no effect. Because the `TIMER_STOP_CFG_SET` register is a write-only register, the result of any write to this register must be checked by reading the `TIMER_STOP_CFG` register. A read of the `TIMER_STOP_CFG_SET` register returns 0x0000.

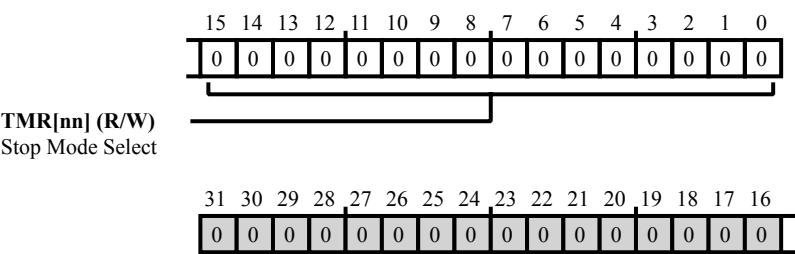


Figure 20-24: TIMER_STOP_CFG_SET Register Diagram

Table 20-36: TIMER_STOP_CFG_SET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R0/W1S)	TMR[nn]	STOP_CFG Set Alias. For all <code>TIMER_STOP_CFG_SET.TMR[nn]</code> bits, write =0 has no effect, and write =1 for abrupt stop (setting the corresponding stop mode select bit in the <code>TIMER_STOP_CFG</code> register). Using <code>TIMER_STOP_CFG_SET</code> to set stop mode bits permits configuring specific timers without influencing the stop mode configuration of other timers.

Timer n Configuration Register

Each timer has a `TIMER_TMR[n]_CFG` register that specifies its operating mode. Only write to a `TIMER_TMR[n]_CFG` register when the corresponding timer is not running.

After disabling a timer operating in PWMOUT mode, verify that the timer has stopped running by checking the start/stop status of the timer in the `TIMER_RUN` register before writing to the timer's `TIMER_TMR[n]_CFG` register.

Note that a timer's `TIMER_TMR[n]_CFG` register may be read at any time.

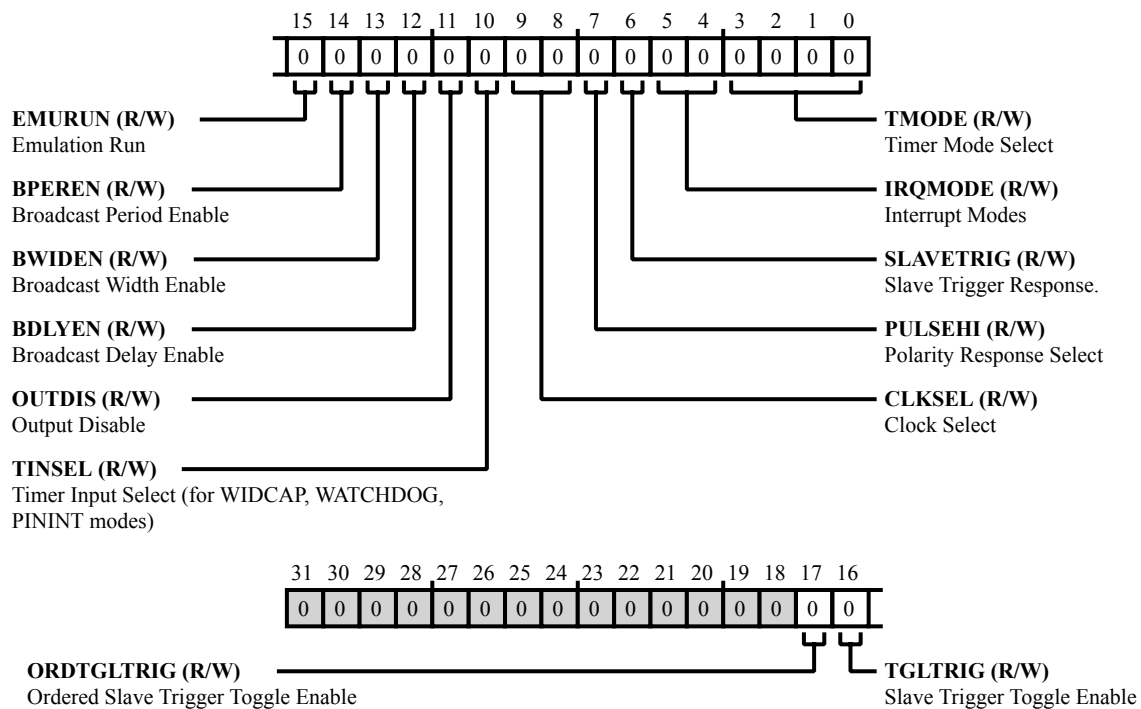


Figure 20-25: `TIMER_TMR[n]_CFG` Register Diagram

Table 20-37: TIMER_TMR[n]_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	ORDTGLTRIG	Ordered Slave Trigger Toggle Enable. The <code>TIMER_TMR[n]_CFG.ORDTGLTRIG</code> bit controls ordered slave trigger toggle mode. The timer(slave) state is not toggled by writes to the <code>TIMER_RUN</code> , <code>TIMER_RUN_SET</code> and <code>TIMER_RUN_CLR</code> registers when this bit is enabled.
		0 No effect
		1 Trigger pulse on Trigger Slave 0 starts the timer if it in halt state and trigger pulse in Trigger Slave 1 stops the timer if it is running
16 (R/W)	TGLTRIG	Slave Trigger Toggle Enable. The <code>TIMER_TMR[n]_CFG.TGLTRIG</code> bit stops the timer if it is running and starts the timer if it is halted (in the stop state). If the <code>TIMER_TMR[n]_CFG.TGLTRIG</code> bit is set, then the setting of the <code>TIMER_TMR[n]_CFG.SLAVETRIG</code> bit is ignored.
		0 Slave Trigger Response Depends on SLAVETRIG Bit Setting
		1 Slave Trigger Toggles Timer State
15 (R/W)	EMURUN	Emulation Run. The <code>TIMER_TMR[n]_CFG.EMURUN</code> bit causes the timer to run (count) during emulation.
		0 Stop Timer During Emulation
		1 Run Timer During Emulation
14 (R/W)	BPEREN	Broadcast Period Enable. The <code>TIMER_TMR[n]_CFG.BPEREN</code> bit enables updates to the <code>TIMER_TMR[n]_PER</code> register simultaneously across more than one timer.
		0 Disable Broadcast to PER Register
		1 Enable Broadcast to PER Register
13 (R/W)	BWIDEN	Broadcast Width Enable. The <code>TIMER_TMR[n]_CFG.BWIDEN</code> bit enables updates to the <code>TIMER_TMR[n]_WID</code> register simultaneously across more than one timer.
		0 Disable Broadcast to WID Register
		1 Enable Broadcast to WID Register

Table 20-37: TIMER_TMR[n]_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	BDLYEN	Broadcast Delay Enable. The <code>TIMER_TMR[n]_CFG.BDLYEN</code> bit enables updates to the <code>TIMER_TMR[n]_DLY</code> register simultaneously across more than one timer.
		0 Disable Broadcast to DLY Register
		1 Enable Broadcast to DLY Register
11 (R/W)	OUTDIS	Output Disable. The <code>TIMER_TMR[n]_CFG.OUTDIS</code> bit enables or disables the timer pin output buffer.
		0 Enable TMR Pin Output Buffer
		1 Disable TMR Pin Output Buffer
10 (R/W)	TINSEL	Timer Input Select (for WIDCAP, WATCHDOG, PININT modes).
		0 Use TMR Pin Input
		1 Use TMR Alternate Capture Input
9:8 (R/W)	CLKSEL	Clock Select. The <code>TIMER_TMR[n]_CFG.CLKSEL</code> bit field selects the TIMER clock to use.
		0 Use SCLK0
		1 Use TMR_ALT_CLK0 as TMR Clock
		3 Use TMR_ALT_CLK1 as TMR Clock
7 (R/W)	PULSEHI	Polarity Response Select. The <code>TIMER_TMR[n]_CFG.PULSEHI</code> bit defines specific behaviors of the timer based on the operating mode. For more information, see the specific operating mode in the Programming Concepts section.
		0 Negative Response or Pulse. A Negative Edge Response or Negative Action Pulse on the TMR pin.
		1 Positive Response or Pulse. A Positive Edge Response or Positive Action Pulse on the TMR pin.
6 (R/W)	SLAVETRIG	Slave Trigger Response.. The <code>TIMER_TMR[n]_CFG.SLAVETRIG</code> bit controls the trigger response. The trigger pulse has no effect (to stop or start the timer) if the timer is already in the requested state.
		0 Pulse Stops Timer if it is Running
		1 Pulse Starts Timer if it is Stopped

Table 20-37: TIMER_TMR[n]_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5:4 (R/W)	IRQMODE	Interrupt Modes. The <code>TIMER_TMR[n]_CFG.IRQMODE</code> bit field selects the interrupt request mode. Note that any mismatched combination of the <code>TIMER_TMR[n]_CFG.IRQMODE</code> and the <code>TIMER_TMR[n]_CFG.TMODE</code> bits results in no interrupt being generated. In WIDCAP modes, the position of the interrupt is controlled with the <code>TIMER_TMR[n]_CFG.TMODE</code> bit, and the <code>TIMER_TMR[n]_CFG.IRQMODE</code> bit is ignored. Whenever an interrupt is generated, a trigger master pulse is also generated, if enabled in the <code>TIMER_TRG_MSK</code> register.
		0 Active Edge Mode. The timer generates an interrupt at every active edge. The active edge polarity depends on the state of the <code>TIMER_TMR[n]_CFG.PULSEHI</code> bit. Valid for PININT mode only.
		1 Delay Expired Mode. The timer generates an interrupt when the <code>TIMER_TMR[n]_CNT</code> value reaches the value in the <code>TIMER_TMR[n]_DLY</code> register. This mode is valid for all PWMOUT modes.
		2 Width Plus Delay Expired Mode. The timer generates an interrupt when the <code>TIMER_TMR[n]_CNT</code> value reaches the value in the <code>TIMER_TMR[n]_WID</code> register plus the value in the <code>TIMER_TMR[n]_DLY</code> register. (PWMOUT modes only)
		3 Period Expired Mode. The timer generates an interrupt when the <code>TIMER_TMR[n]_CNT</code> value reaches the value in the <code>TIMER_TMR[n]_PER</code> register. (Continuous PWMOUT and EXTCLK modes only)
3:0 (R/W)	TMODE	Timer Mode Select. The <code>TIMER_TMR[n]_CFG.TMODE</code> bit field selects the operating mode of each timer.
		0-7 Idle Mode
		8 Period Watchdog Mode
		9 Width Watchdog Mode

Table 20-37: TIMER_TMR[n]_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		10	Measurement Report at Asserting Edge of Waveform
		11	Measurement Report at Deasserting Edge of Waveform
		12	Continuous PWMOUT Mode
		13	Single Pulse PWMOUT Mode
		14	EXTCLK Mode
		15	PININT (pin interrupt) Mode

Timer n Counter Register

The `TIMER_TMR[n]_CNT` register holds the current timer count. After enabling, the count is re-initialized to either 0x0 or 0x1, depending on the configuration and mode. The `TIMER_TMR[n]_CNT` register is read-only and may be read at any time (whether the timer is running or stopped). Reading the `TIMER_TMR[n]_CNT` register returns an atomic 32-bit value.

Depending on the timer operation mode, the counter increment can be clocked by a number of sources, including SCLK0, the TMR or alternate capture input pins, ACLK[n]. The counter retains its value after the timer is disabled.

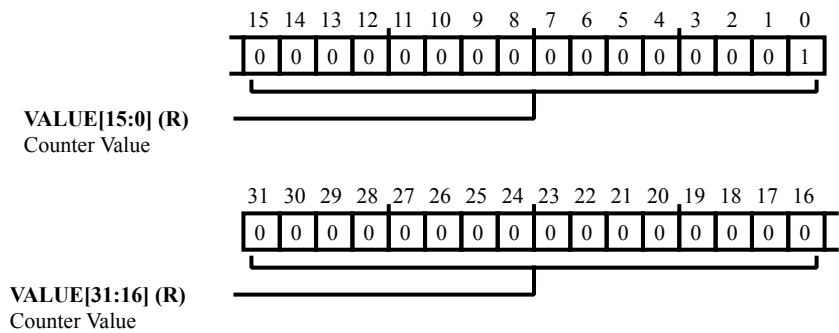


Figure 20-26: TIMER_TMR[n]_CNT Register Diagram

Table 20-38: TIMER_TMR[n]_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Counter Value. The <code>TIMER_TMR[n]_CNT.VALUE</code> bit field holds the current timer count.

Timer n Delay Register

The `TIMER_TMR[n]_DLY` register holds the delay value for the corresponding timer. This register's use is based on the selected timer mode.

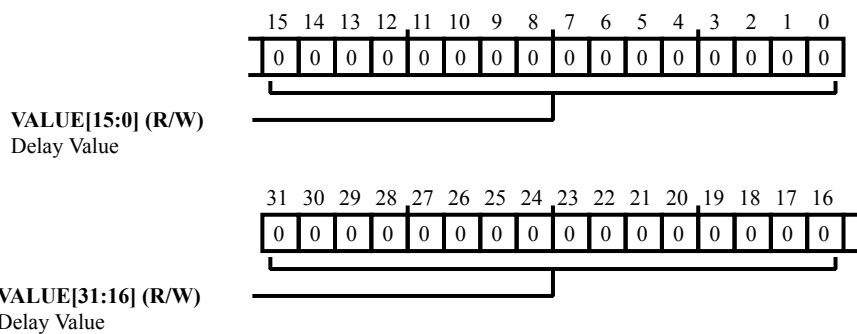


Figure 20-27: `TIMER_TMR[n]_DLY` Register Diagram

Table 20-39: `TIMER_TMR[n]_DLY` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Delay Value. The <code>TIMER_TMR[n]_DLY.VALUE</code> bit field holds the delay value for the corresponding timer.

Timer n Period Register

The `TIMER_TMR[n]_PER` register holds the period value for the corresponding timer. This register's use is based on the selected timer mode.

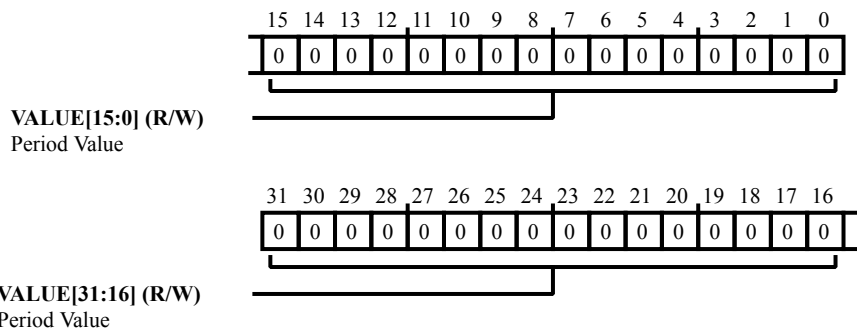


Figure 20-28: `TIMER_TMR[n]_PER` Register Diagram

Table 20-40: `TIMER_TMR[n]_PER` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Period Value. The <code>TIMER_TMR[n]_PER.VALUE</code> bit field holds the period value for the corresponding timer.

Timer n Width Register

The `TIMER_TMR[n]_WID` register holds the width value for the corresponding timer. This register's use is based on the selected timer mode.

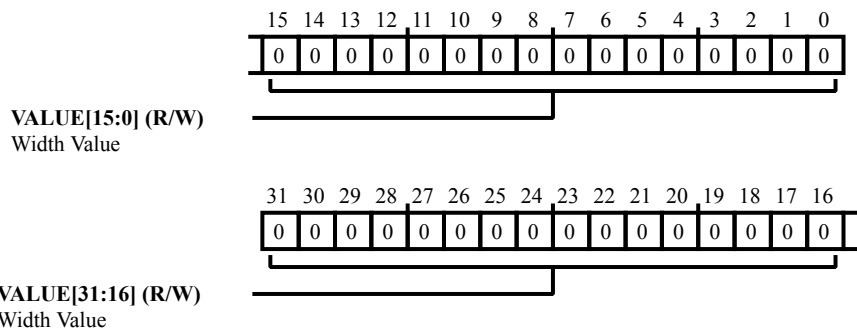


Figure 20-29: `TIMER_TMR[n]_WID` Register Diagram

Table 20-41: `TIMER_TMR[n]_WID` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Width Value. The <code>TIMER_TMR[n]_WID.VALUE</code> bit field holds the width value for the corresponding timer.

Trigger Slave Enable Register

As a trigger slave, each timer can generate a unique data trigger pulse signal. The `TIMER_TRG_IE` contains trigger input enable bits for these signals, disabling or enabling the triggers as programmed. The reset value of the `TIMER_TRG_IE` register is 0xFFFF, masking these triggers after reset.

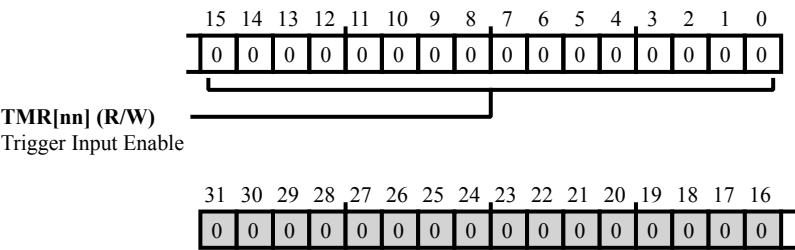


Figure 20-30: `TIMER_TRG_IE` Register Diagram

Table 20-42: `TIMER_TRG_IE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	TMR[nn]	Trigger Input Enable. For all <code>TIMER_TRG_IE.TMR[nn]</code> bits, write =0 disables the corresponding trigger input, and write =1 enables the corresponding trigger input.

Trigger Master Mask Register

As a trigger master, each timer can generate a unique data trigger pulse signal. The `TIMER_TRG_MSK` register contains a trigger mask for these outputs, masking (disabling) or unmasking (enabling) the triggers as programmed. The reset value of the `TIMER_TRG_MSK` register is 0xFFFF, masking these triggers after reset.

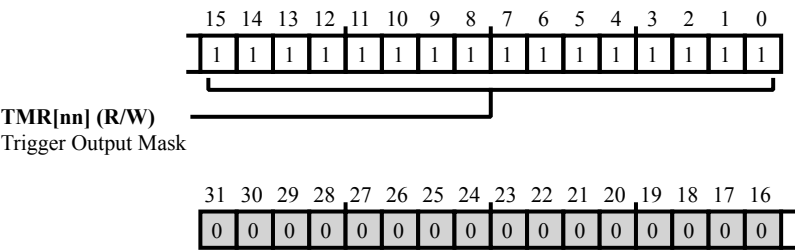


Figure 20-31: TIMER_TRG_MSK Register Diagram

Table 20-43: TIMER_TRG_MSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	TMR[nn]	Trigger Output Mask. For all <code>TIMER_TRG_MSK.TMR[nn]</code> bits, write =0 unmask (enables) the corresponding data trigger output, and write =1 masks (disables) the corresponding data trigger output.

21 General-Purpose Counter (CNT)

The GP counter converts pulses from incremental position encoders into data that is representative of the actual position of the pulse. This conversion is done by integrating (counting) pulses on one or two inputs. Since integration provides relative position, some devices also feature a zero-position input (zero marker). The GP counter can use the zero position input feature to establish a reference point for verifying that the acquired position does not drift over time. In addition, the GP counter can use the incremental position information to determine speed, if the time intervals are measured.

The GP counter provides flexible ways to establish position information. When used with the GP timer block, the GP counter can allow for the acquisition of coherent position or time stamp information that enables speed calculation.

GP Counter Features

The GP counter includes the following features:

- 32-bit up or down counter
- Quadrature encode mode (Gray code)
- Binary encoder mode
- Alternative frequency-direction mode
- Timed direction and up or down counting modes
- Zero marker or push-button support
- Capture event timing in association with GP Timer
- Boundary comparison and boundary setting features

GP Counter Functional Description

The *GP Counter Block Diagram* shows a block diagram of the GP counter. The CNT_UD and CNT_DG pins accept various forms of incremental inputs. The 32-bit counter processes the inputs. The GP counter uses the CNT_ZM pin to sense the pressing of a push button.

NOTE: When enabled, the GP counter requires 3 SCLK0 cycles of initialization before recognizing valid toggles on its input pins.

The three input pins can be filtered (debounced) before the GP counter evaluates them.

The GP counter features a flexible boundary comparison. In all of the operating modes, the counter can be compared to an upper and lower limit. It takes various actions when reaching these limits.

The GP counter has a flexible input selection. Apart from accepting inputs from the CNT0_UD and CNT0_DG and PORT pins, the counter can be configured to accept trigger inputs by setting PADS_PCFG0.CNT0UDSEL and PADS_PCFG0.CNT0DGSEL bits. Refer to the [PADS_PCFG0](#) register description in General-Purpose Ports (PORT) chapter for details.

The module can optionally generate an interrupt request to the system through its IRQ line. On many processors, a GP timer module can use an output to generate time stamps on certain events.

ADSP-2159x CNT Register List

The GP Counter (CNT) provides support for manually controlled rotary controllers, such as the volume wheel on a radio device. This unit also supports industrial encoders.

The CNT converts pulses from incremental position encoders into data that is representative of the actual position. To complete this task, the CNT integrates (counts) pulses on one or two inputs. Because integration provides relative position, a zero position input (zero marker) is usually provided that establishes a reference point, verifying that the acquired position does not drift over time. The incremental position information may also be used to determine speed, if the relevant time intervals are measured. The CNT provides flexible ways to establish position information. When used in conjunction with the General-purpose Timer (TIMER), the CNT allows acquisition of coherent position/time stamp information, enabling speed calculation.

A set of registers govern CNT operations. For more information on CNT functionality, see the CNT register descriptions.

Table 21-1: ADSP-2159x CNT Register List

Name	Description
CNT_CFG	Configuration Register
CNT_CMD	Command Register
CNT_CNTR	Counter Register
CNT_DEBNCE	Debounce Register
CNT_IMSK	Interrupt Mask Register
CNT_MAX	Maximum Count Register
CNT_MIN	Minimum Count Register
CNT_STAT	Status Register

ADSP-2159x CNT Interrupt List

Table 21-2: ADSP-2159x CNT Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
156	CNT0_STAT	CNT0 Status	Level	

ADSP-2159x CNT Trigger List

Table 21-3: ADSP-2159x CNT Trigger List Masters

Trigger ID	Name	Description	Sensitivity
3	CNT0_STAT	CNT0 Status	Level
4	CNT0_UD	CNT0 CNT0 Count Up and Direction	Level
5	CNT0_DG	CNT0 CNT0 Count Down and Gate	Level
6	CNT0_TO	CNT0 CNT0 Output to Timer Block	Level

Table 21-4: ADSP-2159x CNT Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
0	CNT0_UD	CNT0 CNT0 Count Up and Direction	Pulse
1	CNT0_DG	CNT0 CNT0 Count Down and Gate	Pulse

GP Counter Operating Modes

The GP counter has the following five modes of operation.

1. Quadrature Encoder
2. Binary Encoder
3. Up/Down Counter
4. Direction Counter
5. Timed Direction

With the exception of the timed direction mode, the GP counter can operate with the GP timer block to capture additional timing information (time stamps) associated with events detected by this block.

Quadrature Encoder Mode

In this mode, the CNT_UD and CNT_DG inputs expect a quadrature-encoded signal that is interpreted as a two-bit Gray code. The order of transitions of the CNT_UD and CNT_DG inputs determines whether the counter increments or decrements. The CNT_CNTR register contains the number of transitions that have occurred as shown in the *Quadrature Events and Counting Mechanism* table. Optionally, an interrupt is generated when both inputs

change within one SCLK0 cycle. Gray coding prohibits such transitions. Therefore, the `CNT_CNTR` register remains unchanged, and an error condition is signaled.

Table 21-5: Quadrature Events and Counting Mechanism

CNT_COUNTER Register Value	-4	-3	-2	-1	0	+1	+2	+3	+4
CDG, CUD Inputs	00	01	11	10	00	01	11	10	00

It is possible to reverse the count direction of the Gray-coded signal by enabling the polarity inverter of either the `CNT_UD` pin or the `CNT_DG` pin. Inverting both pins does not alter the behavior. The GP counter can enable this feature with the `CNT_CFG.CDGINV` and `CNT_CFG.CUDINV` bits.

As an example, the `CNT_DG` and `CNT_UD` inputs are 00 and the next transition is to 01. These inputs normally change the counter in increments as shown in the table. If the `CNT_UD` polarity is inverted, this condition generates a received input of 01 followed by 00. The results is a decrement of the counter, altering the behavior of the connected hardware.

Binary Encoder Mode

This mode is almost identical to quadrature encoder mode, with the exception that the `CNT_UD`: `CNT_DG` inputs expect a binary-encoded signal. The order of transitions of the `CNT_UD` and `CNT_DG` inputs determines whether the counter increments or decrements. The `CNT_CNTR` register contains the number of transitions that have occurred as shown in the *Binary Events and Counting Mechanism* table. Optionally, an interrupt is generated when the detected code steps by more than 1 (in binary arithmetic) within one SCLK0 cycle. Such transitions are erroneous. Therefore, the `CNT_CNTR` register remains unchanged, and an error condition is signaled.

Table 21-6: Binary Events and Counting Mechanism

CNT_COUNTER Register Value	-4	-3	-2	-1	0	+1	+2	+3	+4
CDG:CUD Inputs	00	01	10	11	00	01	10	11	00

Reversing the `CNT_UD` and `CNT_DG` pin polarity has a different effect in binary encoder mode than for the quadrature encoder mode. Inverting the polarity of the `CNT_UD` pin only, or inverting both the `CNT_UD` and `CNT_DG` pins, results in reversing the count direction.

Up/Down Counter Mode

In this mode, the counter increments or decrements at every active edge of the input pins. The GP counter uses the `CNT_CFG.CUDINV` bit to select an active edge and has the following results.

- If the GP counter module detects an active edge at the `CNT_UD` input, the counter increments.
- If the GP counter module detects an active edge at the `CNT_DG` input, the counter decrements.
- If simultaneous edges occur on the `CNT_DG` and `CNT_UD` pins, the counter remains unchanged, and both up-count and down-count events are signaled in the `CNT_STAT` register.

Direction Counter Mode

In this mode, the counter is incremented or decremented at every active edge of the `CNT_DG` input pin. The state of the `CNT_UD` input determines whether the counter increments or decrements. The GP counter uses the `CNT_CFG.CUDINV` bit to select the polarity.

If the GP counter detects an active edge at the `CNT_DG` input, the counter value changes by one in the selected direction.

Timed Direction Mode

In this mode, the counter is incremented or decremented at each `SCLK0` cycle. The state of the `CNT_UD` input determines whether the counter increments or decrements. The GP counter uses the `CNT_CFG.CUDINV` bit to select the polarity. The `CNT_DG` pin can be used to gate the clock. The GP counter uses the `CNT_CFG.CDGINV` bit to select the polarity.

GP Counter Programming Model

The following sections provide information for programming the interface.

GP Counter General Programming Flow

The following are general guidelines for configuring and enabling the GP counter.

1. Initialize (but do not enable) the GP counter for the desired mode and settings through the `CNT_CFG` register.
2. Usually, events of interest are processed using interrupts rather than by polling status bits. In this case, clear all status bits and activate the interrupt generation requests with the `CNT_IMSK` register.
3. Configure interrupts at the system level to insure desired interrupt signaling to the system.
4. If timing information is required, set up the relevant GP Timer in width capture mode.
5. Finally, enable interrupt requests and the GP Counter itself using the `CNT_IMSK` and `CNT_CFG` registers, respectively.

GP Counter Mode Configuration

The GP counter can use the `CNT_ZM` input pin to sense the zero marker output of a rotary device or to detect the pressing of a push button. There are four programming schemes, which are functional in all counter modes:

- Push-button mode
- Zero-marker-zeros-counter mode
- Zero-marker-error mode
- Zero-once mode

Configuring GP Counter Push-Button Operation

Use the following procedure to configure push-button operation:

1. Set `CNT_IMSK.CZM` to enable (unmask) the zero marker interrupt.
2. Select the active edge polarity through the `CNT_CFG.CZMINV` bit.
3. Proceed with any other desired configuration steps and enable the peripheral.

An active edge at the `CNT_ZM` input sets the `CNT_IMSK.CZM` bit.

Configuring Zero-Marker-Zeros-Counter Mode

The following provides information on configuring zero-marker-zeros-counter mode for the GP counter.

1. Set `CNT_IMSK.CZMZ` to enable `CNT_CNTR`. The zero marker interrupt zeroes the counter.
2. Set `CNT_CFG.ZMZC` to enable ZMZC mode.
3. Select the active edge polarity through the `CNT_CFG.CZMINV` bit.
4. Proceed with any other desired configuration steps and enable the peripheral.

This configuration causes an active level at the `CNT_ZM` pin to clear the `CNT_CNTR` register and keep it cleared until the `CNT_ZM` pin is deactivated. In addition, the `CNT_STAT.CZMZ` bit is set.

Configuring Zero-Marker-Error Mode

The GP counter uses this mode to detect discrepancies between counter-value and the zero marker output of certain rotary encoder devices.

1. Set the `CNT_STAT.CZME` bit to enable this mode.
2. Select the active edge of the `CNT_ZM` pin through the `CNT_CFG.CZMINV` bit.
3. Proceed with any other desired configuration steps and enable the peripheral.

When the GP counter detects an active edge at the `CNT_ZM` input pin, it compares the four LSBs of the `CNT_CNTR` register to zero. If they are not zero, the GP counter uses `CNT_STAT.CZME` bit to signal a mismatch.

Configuring Zero-Once Mode

The GP counter uses this mode to perform an initial reset of the counter-value when it detects an active zero marker. After that, the zero marker is ignored (the counter is no longer reset).

1. Set the `CNT_CMD.W1ZMONCE` bit to enable this mode.
2. Select the active edge of the `CNT_ZM` pin through the `CNT_CFG.CZMINV` bit.
3. Ensure that at least one of the following bits is enabled: `CNT_IMSK.CZM`, `CNT_IMSK.CZME`, `CNT_IMSK.CZMZ`.

4. Proceed with any other desired configuration steps and enable the peripheral.

The `CNT_CNTR` register and the `CNT_CMD.W1ZMONCE` bit are cleared on the next active edge of the `CNT_ZM` pin. Now the `CNT_CMD.W1ZMONCE` bit can be read to check whether the event has already occurred.

Configuring Boundary Auto-Extend Mode

In this mode, hardware modifies the boundary registers (`CNT_MIN` and `CNT_MAX`) whenever the `CNT_CNTR` value reaches either of them. The GP counter uses this mode to monitor the widest angle a thumb wheel even if the software did not generate interrupts.

1. Initialize `CNT_CNTR` with the desired value.
2. Set both `CNT_MIN` and `CNT_MAX` to this same value.
3. Configure the `CNT_CFG.BNDMODE` field for auto extend mode.
4. Proceed with any other desired configuration steps and enable the peripheral.

The `CNT_MAX` register is loaded with the current `CNT_CNTR` value when the latter increments beyond the `CNT_MAX` value. Similarly, the `CNT_MIN` register is loaded with the `CNT_CNTR` value when the latter decrements below the `CNT_MIN` value. The `CNT_STAT.MAXC` and `CNT_STAT.MINC` status bits are set when the `CNT_CNTR` value matches the respective boundary register value.

Configuring Boundary Capture Mode

In this mode, the `CNT_CNTR` value is latched into the `CNT_MIN` register at one detected edge of the `CNT_ZM` input pin, and latched into the `CNT_MAX` boundary register at the opposite edge.

1. To capture the `CNT_ZM` pin rising edge into `CNT_MIN` and the falling edge into `CNT_MAX`, program `CNT_CFG.CZMINV` for active high polarity. Conversely, to capture the `CNT_ZM` pin falling edge into `CNT_MIN` and the rising edge into `CNT_MAX`, program `CNT_CFG.CZMINV` for active low polarity.
2. Program the `CNT_IMSK.MAXC` and `CNT_IMSK.MINC` interrupt mask bits according to interrupt generation requirements.
3. Configure the `CNT_CFG.BNDMODE` field for boundary capture mode.
4. Proceed with any other desired configuration steps and enable the peripheral.

The `CNT_STAT.MAXC` and `CNT_STAT.MINC` status bits report the capture event, depending on how interrupt masks are configured.

Configuring Boundary Compare and Boundary Zero Modes

In these modes, the two boundary registers (`CNT_MAX` and `CNT_MIN`) are compared to the value in the `CNT_CNTR` register.

1. Program `CNT_MAX` and `CNT_MIN` registers with appropriate upper and lower range values.

2. Program the `CNT_IMSK.MAXC` and `CNT_IMSK.MINC` interrupt mask bits according to interrupt generation requirements.
3. Configure the `CNT_CFG.BNDMODE` field for boundary compare mode.
4. Proceed with any other desired configuration steps and enable the peripheral.

If after incrementing, `CNT_CNTR = CNT_MAX`, then the `CNT_STAT.MAXC` bit is set. Similarly if after decrementing, `CNT_CNTR = CNT_MIN`, then the `CNT_STAT.MINC` bit is set.

Additionally, for boundary zero mode, the counter-value in `CNT_CNTR` is set to zero. The `CNT_STAT.MAXC` and `CNT_STAT.MINC` bits are not set when software updates the `CNT_MAX` or `CNT_MIN` registers.

Configuring GP Counter Push-Button Operation

Use the following procedure to configure push-button operation:

1. Set `CNT_IMSK.CZM` to enable (unmask) the zero marker interrupt.
2. Select the active edge polarity through the `CNT_CFG.CZMINV` bit.
3. Proceed with any other desired configuration steps and enable the peripheral.

An active edge at the `CNT_ZM` input sets the `CNT_IMSK.CZM` bit.

GP Counter Programming Concepts

Using the features, operating modes, and event control for the GP counter to their greatest potential requires an understanding of some GP counter-related concepts. Some key aspects to consider are input noise filtering and capturing timing information.

CNT Input Noise Filtering

In all modes, the three input pins can be filtered to present clean signals to the GP counter logic. The GP counter uses the `CNT_CFG.DEBEN` bit to enable or disable this filtering. The *Programmable Noise Filtering* figure shows the filtering operation for the `CNT_UD` pin.

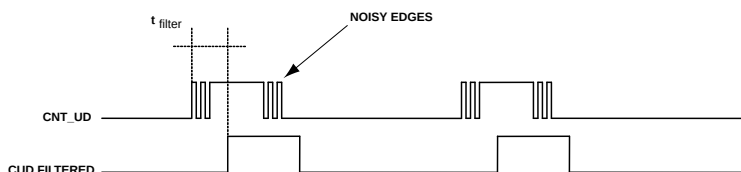


Figure 21-1: Programmable Noise Filtering

The CNT module implements the filtering mechanism using counters for each GP counter pin, where each counter is initialized from the `CNT_DEBNCE.DPRESCALE` field. When a transition is detected on a pin, the corresponding counter starts counting up to the programmed number of `SCLK0` cycles. The state of the pin is latched after time t_{filter} and passed on to the GP counter logic.

The following formula determines the time t_{filter} , given SCLK0 and the CNT_DEBNCE.DPRESCALE value, where lower values of CNT_DEBNCE.DPRESCALE result in shorter debounce delays:

$$t_{\text{filter}} = 128 \times (2^{\text{DPRESCALE}} \times \text{SCLK0})$$

Capturing Counter Interval and CNT_CNTR Read Timing

When the count speed is low, it is often useful to capture the time elapsed since the last count event. Program the `TIMER TMR[n] CFG` register of the associated GP timer in a width capture mode with the following bit settings.

- `TIMER_TMR[n]_CFG.PULSEHI = 0`
- `TIMER_TMR[n]_CFG.TMODE = b#1011`
- `TIMER_TMR[n]_CFG.TINSEL = 1`

The *Capture Intervals* figure shows and the following list describe the operation of the GP counter and the GP timer in this mode.

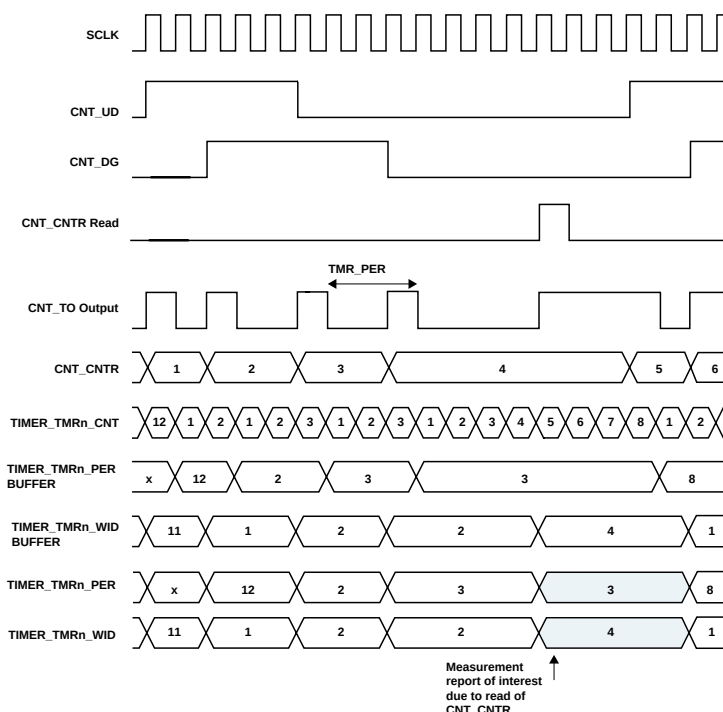


Figure 21-2: Capture Intervals

NOTE: SCLK in the *Capture Intervals* figure is SCLK0.

1. The CNT_TO signal generates a pulse every time a count event occurs. In addition, when the processor reads the CNT_CNTR register, the CNT_TO signal presents a pulse which is extended (high) until the next count event.
2. The GP timer updates its TIMER_TMR[n]_PER register with the period (measured from falling edge to falling edge, because TIMER_TMR[n]_CFG.PULSEHI = 0) of the CNT_TO signal.

3. The `TIMER_TMR[n]_WID` register is updated with the pulse width (the portion where `CNT_TO` is low, again because `TIMER_TMR[n]_CFG.PULSEHI = 0`).
4. Both registers are updated at every rising edge of the `CNT_TO` signal (because `TIMER_TMR[n]_CFG.TMODE = b#011`).

The `TIMER_TMR[n]_PER` register contains the period between the last two count events. The `TIMER_TMR[n]_WID` register contains the time since the last count event and the read of the `CNT_CNTR` register, both measured in `SCLK0` cycles.

Read the `CNT_CNTR` register to latch the two time measurements, providing a coherent triplet of information to calculate speed and position.

NOTE: Speed restrictions apply to the use of the `CNT_TO` signal. Therefore, programs must not operate at count event rates that are high. For instance, if `CNT_CNTR` is incremented or decremented every `SCLK0` cycle (timed direction mode), the `CNT_TO` signal is not valid.

Capturing Time Interval Between Successive Counter Events

When the required timing information is the interval between successive count events, program the associated timer in a width capture mode. Set the `TIMER_TMR[n]_CFG` bit of `TIMER_TMR[n]_CFG.PULSEHI = 1`, `TIMER_TMR[n]_CFG.TMODE = b#1010` and `TIMER_TMR[n]_CFG.TINSEL = 1`. Typically, this information is sufficient if the speed of GP counter events does not to reach low values.

The *Period Register Timing* figure shows the operation of the GP counter and the GP timer in this mode.

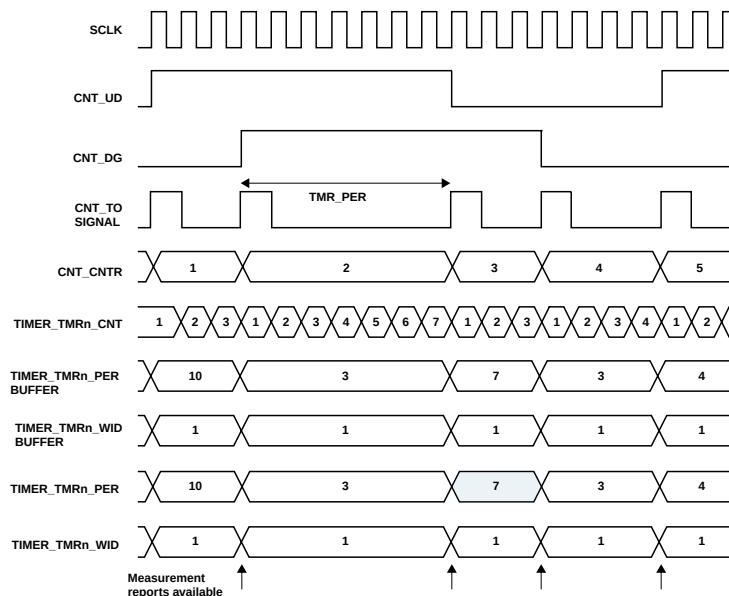


Figure 21-3: Period Register Timing

NOTE: SCLK in the *Period Register Timing* figure is `SCLK0`.

The `CNT_TO` signal generates a pulse every time a count event occurs. The GP timer updates its `TIMER_TMR[n]_PER` register with the period (measured from rising edge to rising edge) of the `CNT_TO` signal. The `TIMER_TMR[n]_PER` register is updated at every rising edge of the `CNT_TO` signal and contains the number of `SCLK0` cycles that have elapsed since the previous rising edge.

Incidentally, the `TIMER_TMR[n]_WID` register is also updated at the same time, but is generally of no interest in this mode of operation. If no reads of the `CNT_CNTR` register occur between counter events, the `TIMER_TMR[n]_WID` register only contains the width of the `CNT_TO` pulse. If a read of `CNT_CNTR` has occurred between events, the `TIMER_TMR[n]_WID` register contains the time between the read of `CNT_CNTR` and the next event.

This mode can also be used with `TIMER_TMR[n]_CFG.PULSEHI = 0`. In this case, the period of `CNT_TO` is measured between falling edges. It results in the same values as in the previous case, only the latching occurs one `SCLK0` cycle later.

GP Counter Event Control

Eleven events can be signaled to the processor using status information and optional interrupt requests. The GP counter uses the respective bits in the `CNT_IMSK` register to enable the interrupt requests. It uses dedicated bits in the `CNT_STAT` register to report events. When an interrupt request from the GP counter is serviced, the application software is responsible for correct interpretation of the events. It is recommended to logically AND the content of the `CNT_IMSK` and `CNT_STAT` registers to identify pending interrupt requests.

Perform a write-one-to-clear (W1C) operation to the `CNT_STAT` register to clear the interrupt requests. Hardware does not clear the status bits automatically, unless the counter module is disabled.

The following sections describe the events associated with the GP counter.

Illegal Gray and Binary Code Events

When illegal transitions occur in quadrature encoder or binary encoder modes, the `CNT_STAT.IC` bit is set. If enabled by the `CNT_STAT.IC` bit, the counter module generates an interrupt request. Set the `CNT_STAT.IC` bit only in the quadrature encoder or binary encoder modes.

Up/Down Count Events

The GP counter uses the `CNT_STAT.UC` bit to indicate whether the counter has been incremented. Similarly, the `CNT_STAT.DC` bit reports decrements. The two events are independent. For instance, if the counter increments by one and then decrements by two, both bits remain set, even though the resulting counter-value shows a decrement by one.

In up/down counter mode, hardware can detect simultaneous active edges on the `CNT_UD` and `CNT_DG` inputs. In that case, the `CNT_CNTR` remains unchanged, but both the `CNT_STAT.UC` and `CNT_STAT.DC` bits are set. Interrupt requests for these events can be enabled through the `CNT_IMSK.UC` and `CNT_IMSK.DC` bits. Use this feature carefully when the counter is clocked at high rates. This suggestion is especially critical when the counter operates in `DIR_TMR` mode, as interrupts are generated every `SCLK0` cycle.

These events can also be used for more push buttons, when GP counter features are unnecessary. When up/down counter mode is enabled, the GP counter can use these count events to report interrupts from push buttons that connect to the CNT_UD and CNT_DG inputs.

Zero-Count Events

The CNT_STAT.CZERO status bit indicates that the CNT_CNTR has reached a value equal to 0x0000 0000 after an increment or decrement. This bit is not set when the counter value is set to zero by a write to CNT_CNTR or by setting the CNT_CMD.W1LCNTZERO bit. If enabled by the CNT_IMSK.CZERO bit, the GP counter module generates an interrupt request.

Overflow Events

There are two status bits that indicate whether the signed counter-register has overflowed from a positive to a negative value or conversely. The CNT_STAT.COV31 bit reports that the 32-bit CNT_CNTR register has either incremented from 0x7FFF FFFF to 0x8000 0000, or decremented from 0x8000 0000 to 0x7FFF FFFF.

If enabled by the CNT_IMSK.COV31 bit, an interrupt request is generated. Similarly, in applications where only the lower 16 bits of the counter are of interest, the CNT_STAT.COV15 status bit reports counter transitions from 0xFFFF 7FFF to 0xFFFF 8000, or from 0xFFFF 8000 to 0xFFFF 7FFF. If enabled by the CNT_IMSK.COV15 bit, an interrupt request is generated.

Boundary Match Events

The CNT_STAT.MINC and CNT_STAT.MAXC status bits report boundary events as described in [Configuring Boundary Capture Mode](#). These bits are not set if the software updates the CNT_CNTR, CNT_MAX, or CNT_MIN registers or writes to the CNT_CMD register. The CNT_IMSK.MINC and CNT_IMSK.MAXC bits enable interrupt request generation on boundary events.

Zero Marker Events

The CNT_STAT.CZM, CNT_STAT.CZME, and CNT_STAT.CZMZ bits are associated with zero marker events, as described in [Configuring GP Counter Push-Button Operation](#). Each of these events can optionally generate an interrupt request, when enabled by the corresponding CNT_IMSK.CZM, CNT_IMSK.CZME and CNT_IMSK.CZMZ bits.

ADSP-2159x CNT Register Descriptions

CNT (CNT) contains the following registers.

Table 21-7: ADSP-2159x CNT Register List

Name	Description
CNT_CFG	Configuration Register
CNT_CMD	Command Register

Table 21-7: ADSP-2159x CNT Register List (Continued)

Name	Description
CNT_CNTR	Counter Register
CNT_DEBNCE	Debounce Register
CNT_IMSK	Interrupt Mask Register
CNT_MAX	Maximum Count Register
CNT_MIN	Minimum Count Register
CNT_STAT	Status Register

Configuration Register

The `CNT_CFG` register configures counter modes, configures input pins, and enables the CNT.

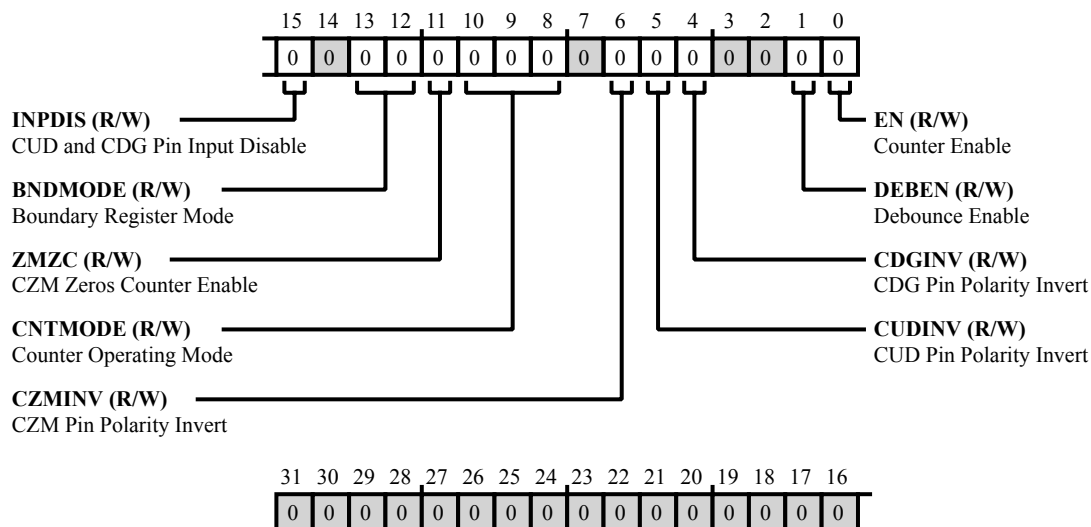


Figure 21-4: CNT_CFG Register Diagram

Table 21-8: CNT_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	INPDIS	CUD and CDG Pin Input Disable. The <code>CNT_CFG</code> . <code>INPDIS</code> disables or enables the <code>CNT_UD</code> input pin and the <code>CNT_DG</code> pin.
		0 Enable
		1 Pin Input Disable
13:12 (R/W)	BNDMODE	Boundary Register Mode. The <code>CNT_CFG</code> . <code>BNDMODE</code> bit field selects the mode for the <code>CNT_MIN</code> and <code>CNT_MAX</code> boundary registers.
		0 BND_COMP. Boundary Compare Mode
		1 BND_ZERO. Boundary Zero Mode
		2 BND_CAPT. Boundary Capture Mode
		3 BND_AEXT. Boundary Auto-extend Mode

Table 21-8: CNT_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	ZMZC	CZM Zeros Counter Enable. The CNT_CFG . ZMZC bit enables or disables level sensitive - active CNT_ZM pin operation to zero the CNT_CNTR register.
		0 Disable
		1 Enable
10:8 (R/W)	CNTMODE	Counter Operating Mode. The CNT_CFG . CNTMODE bit field selects the operating mode for the CNT_UD input pin and the CNT_DG pin.
		0 QUAD_ENC. Quadrature Encoder Mode
		1 BIN_ENC. Binary Encoder Mode
		2 UD_CNT. Rotary Counter Mode
		4 DIR_CNT. Direction Counter Mode
		5 DIR_TMR. Direction Timer Mode
6 (R/W)	CZMINV	CZM Pin Polarity Invert. The CNT_CFG . CZMINV bit selects the polarity for the CNT_ZM pin. This polarity must be configured before the counter is enabled. It must not change on-the-fly while the counter is enabled.
		0 Active High, Rising Edge
		1 Active Low, Falling Edge
5 (R/W)	CUDINV	CUD Pin Polarity Invert. The CNT_CFG . CUDINV bit selects the polarity for the CNT_UD pin. This polarity must be configured before the counter is enabled. It must not change on-the-fly while the counter is enabled.
		0 Active High, Rising Edge
		1 Active Low, Falling Edge
4 (R/W)	CDGINV	CDG Pin Polarity Invert. The CNT_CFG . CDGINV bit selects the polarity for the CNT_DG pin. This polarity must be configured before the counter is enabled. It must not change on-the-fly while the counter is enabled.
		0 Active High, Rising Edge
		1 Active Low, Falling Edge

Table 21-8: CNT_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	DEBEN	Debounce Enable. The CNT_CFG.DEBEN bit enables or disables CNT input debounce filtering operation selected with the CNT_DEBNCE register.
		0 Disable
		1 Enable
0 (R/W)	EN	Counter Enable. The CNT_CFG.EN bit enables or disables CNT operation.
		0 Counter Disable
		1 Counter Enable

Command Register

The `CNT_CMD` register configures the CNT, enabling operations such as zeroing a counter register and copying or swapping boundary registers. These actions are taken by setting the appropriate bit.

Read operations from this register do not return meaningful values, with the exception of the `CNT_CMD.W1ZMONCE` bit, where a set bit indicates that the bit has been set by software before, but a zero marker event has not yet been detected on the `CNT_ZM` pin yet. For more information, see the CNT functional description.

The `CNT_CNTR`, `CNT_MIN`, and `CNT_MAX` registers can be initialized to zero by setting the `CNT_CMD.W1LCNTZERO`, `CNT_CMD.W1LMINZERO`, and `CNT_CMD.W1LMAXZERO` bits. In addition to clearing registers, the `CNT_CMD` register permits modifying the `CNT_MIN` and `CNT_MAX` boundary registers in a number of ways. The current counter value in the `CNT_CNTR` register can be captured and loaded into either of the two boundary registers to create new boundary limits. This operation is performed by setting the `CNT_CMD.W1LMAXCNT` and `CNT_CMD.W1LMINCNT` bits. Alternatively, the counter can be loaded from `CNT_MAX` or `CNT_MIN` using the `CNT_CMD.W1LCNTMAX` and `CNT_CMD.W1LCNTMIN` bits. It is also possible to transfer the current `CNT_MAX` value into `CNT_MIN` (or conversely) through the `CNT_CMD.W1LMINMAX` and `CNT_CMD.W1LMAXMIN` bits.

Another counter operation is the ability to only have the zero marker clear the `CNT_CNTR` register once. For more information, see the CNT functional description.

It is possible for multiple actions to be performed simultaneously by setting multiple bits in the `CNT_CMD` register. However, there are restrictions. The bits associated with each command have been grouped together such that all bits that involve a write to the `CNT_CNTR`, `CNT_MAX`, or `CNT_MIN` registers are located within bits 4-bit groups of the `CNT_CMD` register.

Note that a maximum of three commands can be issued at any one time, excluding the `CNT_CMD.W1ZMONCE` command. Also, note that `CNT_CMD.W1LCNTMIN`, `CNT_CMD.W1LCNTMAX`, and `CNT_CMD.W1LCNTZERO` bits have to be used exclusively. Never set more than one of them at the same time.

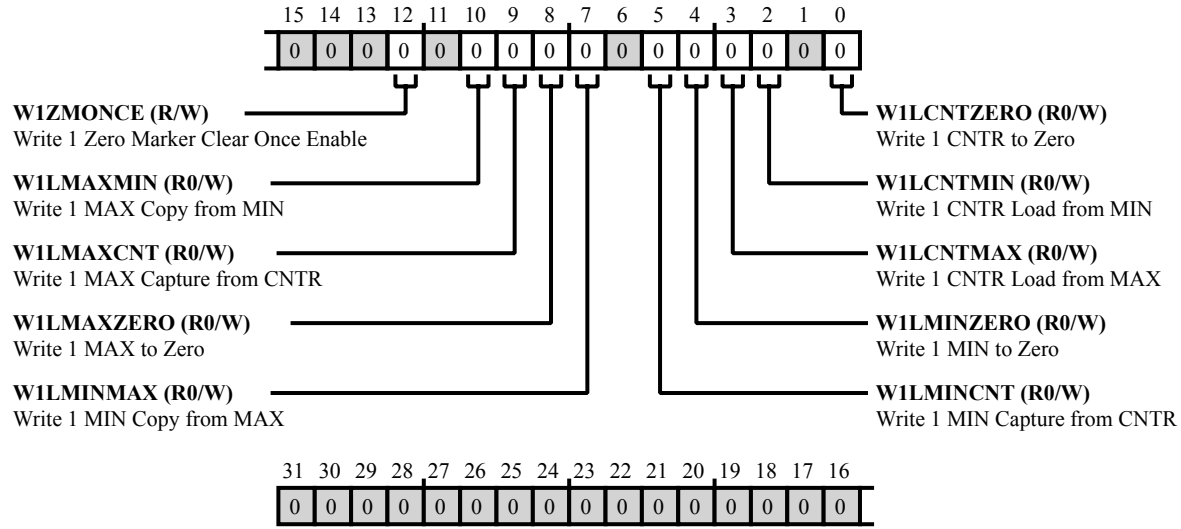


Figure 21-5: CNT_CMD Register Diagram

Table 21-9: CNT_CMD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	W1ZMONCE	Write 1 Zero Marker Clear Once Enable. The CNT_CMD.W1ZMONCE enables a single zero marker clear of the CNT_CNTR register. Reading a 1 in this bit indicates that the bit has been set by software before, but no zero marker event has been detected on the CNT_ZM pin yet.
10 (R0/W)	W1LMAXMIN	Write 1 MAX Copy from MIN. The CNT_CMD.W1LMAXMIN bit transfers the current CNT_MIN register value into CNT_MAX register.
9 (R0/W)	W1LMAXCNT	Write 1 MAX Capture from CNTR. The CNT_CMD.W1LMAXCNT bit loads the current value in the CNT_CNTR register into the CNT_MAX register to create a new boundary limit.
8 (R0/W)	W1LMAXZERO	Write 1 MAX to Zero. Writing a 1 to the CNT_CMD.W1LMAXZERO bit clears the CNT_MAX register.
7 (R0/W)	W1LMINMAX	Write 1 MIN Copy from MAX. The CNT_CMD.W1LMINMAX bit transfers the current CNT_MAX register value into CNT_MIN register.
5 (R0/W)	W1LMINCNT	Write 1 MIN Capture from CNTR. The CNT_CMD.W1LMINCNT bit loads the current value in the CNT_CNTR register into the CNT_MIN register to create a new boundary limit.
4 (R0/W)	W1LMINZERO	Write 1 MIN to Zero. Writing a 1 to the CNT_CMD.W1LMINZERO bit clears the CNT_MIN register.

Table 21-9: CNT_CMD Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R0/W)	W1LCNTMAX	Write 1 CNTR Load from MAX. The CNT_CMD.W1LCNTMAX bit loads the current value in the CNT_MAX register into the CNT_CNTR register to create a new boundary limit.
2 (R0/W)	W1LCNTMIN	Write 1 CNTR Load from MIN. The CNT_CMD.W1LCNTMIN bit loads the current value in the CNT_MIN register into the CNT_CNTR register to create a new boundary limit.
0 (R0/W)	W1LCNTZERO	Write 1 CNTR to Zero. Writing a 1 to the CNT_CMD.W1LCNTZERO bit clears the CNT_CNTR register.

Counter Register

The `CNT_CNTR` register holds the 32-bit, two's-complement count value. It can be read and written at any time. Hardware ensures that reads and write are atomic, by providing respective shadow registers. This register can be accessed with either 32-bit or 16-bit operations. This allows use of the CNT as a 16-bit counter if sufficient for the application.

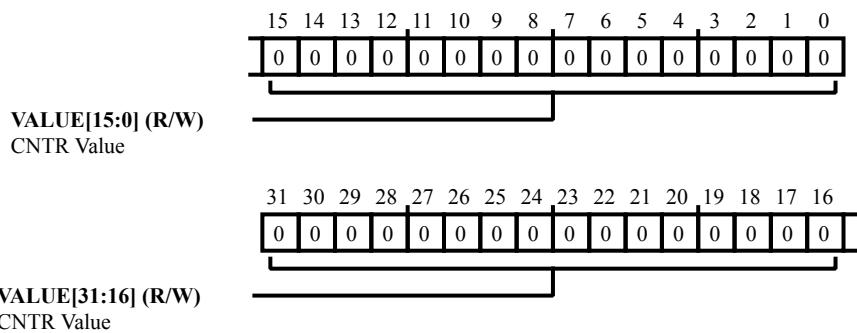


Figure 21-6: CNT_CNTR Register Diagram

Table 21-10: CNT_CNTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	CNTR Value. The <code>CNT_CNTR.VALUE</code> bit field holds the 32-bit, two's-complement count value.

Debounce Register

The `CNT_DEBNCE` register selects the noise filtering characteristic of the three input pins according to the formula:

$$t_{\text{filter}} = 128 \times (2^{\text{DPRESCALE}} / \text{SCLK0})$$

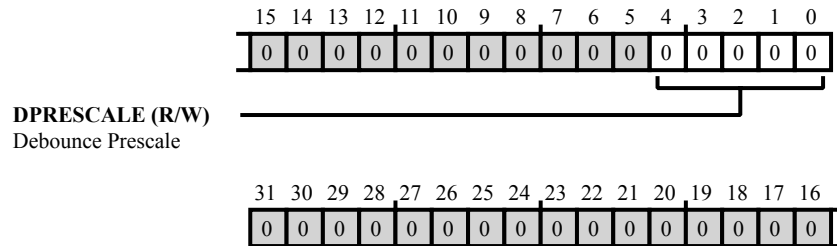


Figure 21-7: CNT_DEBNCE Register Diagram

Table 21-11: CNT_DEBNCE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	DPRESCALE	Debounce Prescale. The <code>CNT_DEBNCE.DPRESCALE</code> selects the desired number of input filtering cycles (and resulting input debounce time) in multiples of <code>SCLK0</code> .
		0 1x cycles = 128 <code>SCLK0</code> cycles
		1 2x cycles
		2 4x cycles
		3 8x cycles
		4 16x cycles
		5 32x cycles
		6 64x cycles
		7 128x cycles
		8 256x cycles
		9 512x cycles
		10 1024x cycles
		11 2048x cycles
		12 4096x cycles
		13 8192x cycles
		14 16384x cycles
		15 32768x cycles
		16 65536x cycles

Table 21-11: CNT_DEBNCE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		17	131072x cycles
		18	Reserved from this value. The values 10010 - 11111 are reserved.
		31	Reserved until this value

Interrupt Mask Register

The `CNT_IMSK` register supports enabling (unmasking) interrupt request generation from each of the CNT events.

All bits in `CNT_IMSK` either disable/mask an interrupt request (if bit cleared) or enable/unmask an interrupt request (if bit set).

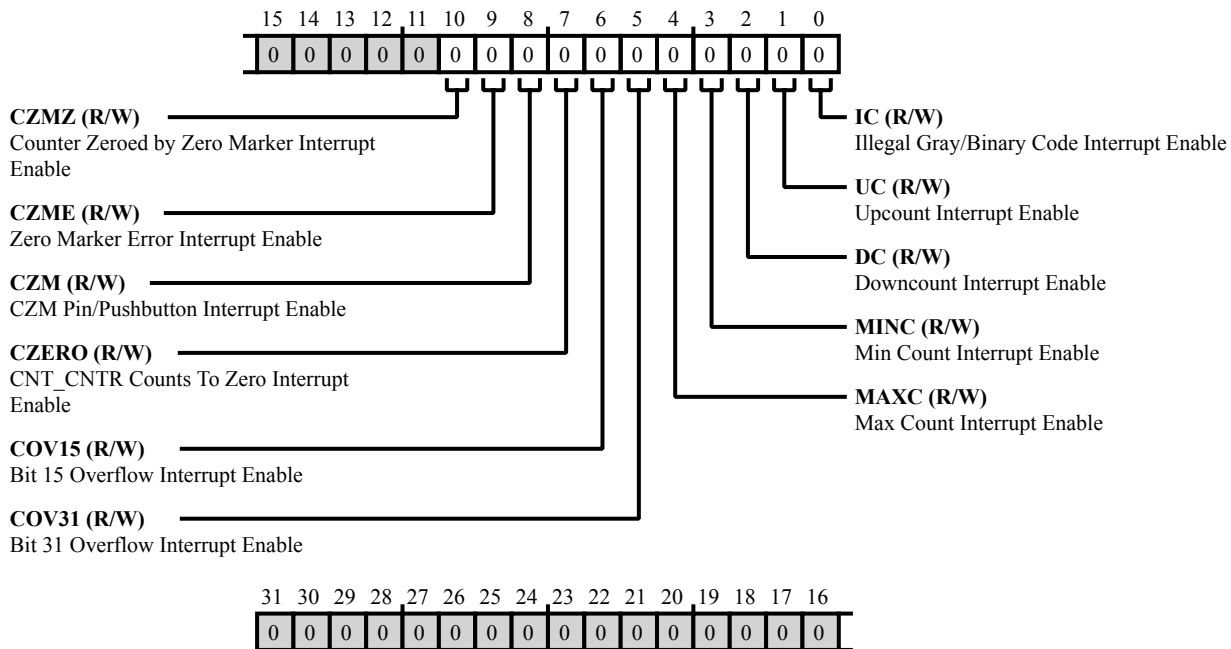


Figure 21-8: CNT_IMSK Register Diagram

Table 21-12: CNT_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	CZMZ	Counter Zeroed by Zero Marker Interrupt Enable. The <code>CNT_IMSK.CZMZ</code> bit enables (unmasks) the counter zeroed by zero marker interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
9 (R/W)	CZME	Zero Marker Error Interrupt Enable. The <code>CNT_IMSK.CZME</code> bit enables (unmasks) the zero marker error interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt

Table 21-12: CNT_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	CZM	CZM Pin/Pushbutton Interrupt Enable. The CNT_IMSK.CZM bit enables (unmasks) the CZM pin/pushbutton interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
7 (R/W)	CZERO	CNT_CNTR Counts To Zero Interrupt Enable. The CNT_IMSK.CZERO bit enables (unmasks) the counts to zero interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
6 (R/W)	COV15	Bit 15 Overflow Interrupt Enable. The CNT_IMSK.COV15 bit enables (unmasks) the bit 15 overflow interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
5 (R/W)	COV31	Bit 31 Overflow Interrupt Enable. The CNT_IMSK.COV31 bit enables (unmasks) the bit 31 overflow interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
4 (R/W)	MAXC	Max Count Interrupt Enable. The CNT_IMSK.MAXC bit enables (unmasks) the max count interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
3 (R/W)	MINC	Min Count Interrupt Enable. The CNT_IMSK.MINC bit enables (unmasks) the min count interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
2 (R/W)	DC	Downcount Interrupt Enable. The CNT_IMSK.DC bit enables (unmasks) the down count interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt

Table 21-12: CNT_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	UC	Upcount Interrupt Enable. The CNT_IMSK. UC bit enables (unmasks) the up count interrupt request.
		0 Mask Interrupt
		1 Unmask Interrupt
0 (R/W)	IC	Illegal Gray/Binary Code Interrupt Enable. The CNT_IMSK. IC bit enables (unmasks) the illegal Gray/Binary Code interrupt request and should only be used in these modes.
		0 Mask Interrupt
		1 Unmask Interrupt

Maximum Count Register

The `CNT_MAX` register holds the 32-bit, two's-complement, higher boundary value. It can be read and written at any time. Hardware ensures that reads and write are atomic, by providing respective shadow registers. This register can be accessed with either 32-bit or 16-bit operations. This allows for using the CNT as a 16-bit counter if sufficient for the application.

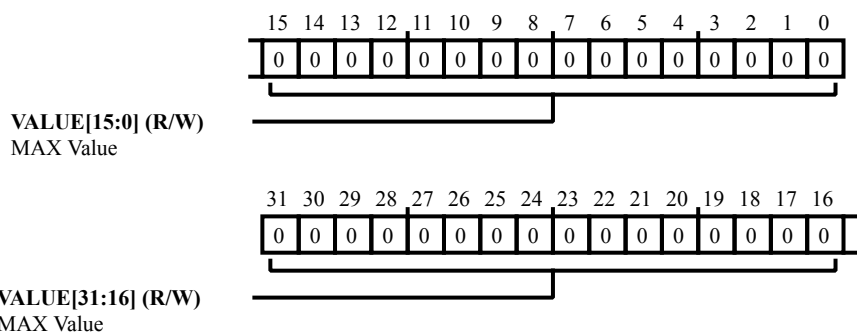


Figure 21-9: CNT_MAX Register Diagram

Table 21-13: CNT_MAX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	MAX Value. The <code>CNT_MAX.VALUE</code> bit field holds the 32-bit, two's-complement, higher boundary value.

Minimum Count Register

The `CNT_MIN` register holds the 32-bit, two's-complement, lower boundary value. It can be read and written at any time. Hardware ensures that reads and write are atomic, by providing respective shadow registers. This register can be accessed with either 32-bit or 16-bit operations. This allows for using the CNT as a 16-bit counter if sufficient for the application.

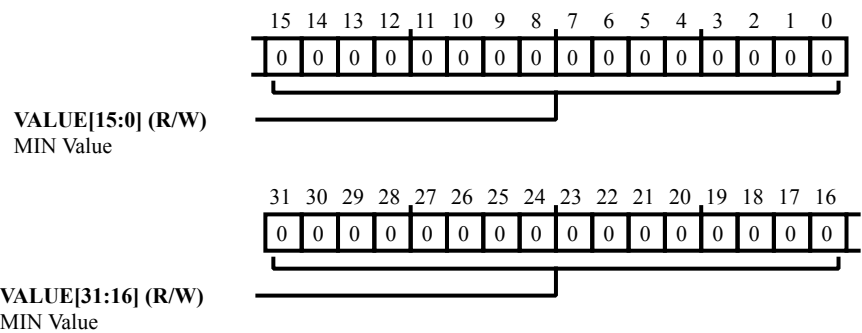


Figure 21-10: CNT_MIN Register Diagram

Table 21-14: CNT_MIN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	MIN Value. The <code>CNT_MIN.VALUE</code> bit field holds the 32-bit, two's-complement, lower boundary value.

Status Register

The **CNT_STAT** register provides status information for each of the CNT events as configured in the **CNT_IMSK** register. When a CNT event is detected, the corresponding bit in this register is set. It remains set until either software writes a 1 to the bit (write-1-to-clear) or the CNT is disabled.

All bits in the **CNT_STAT** register indicate either no interrupt request pending (if bit cleared) or an interrupt request pending (if bit set).

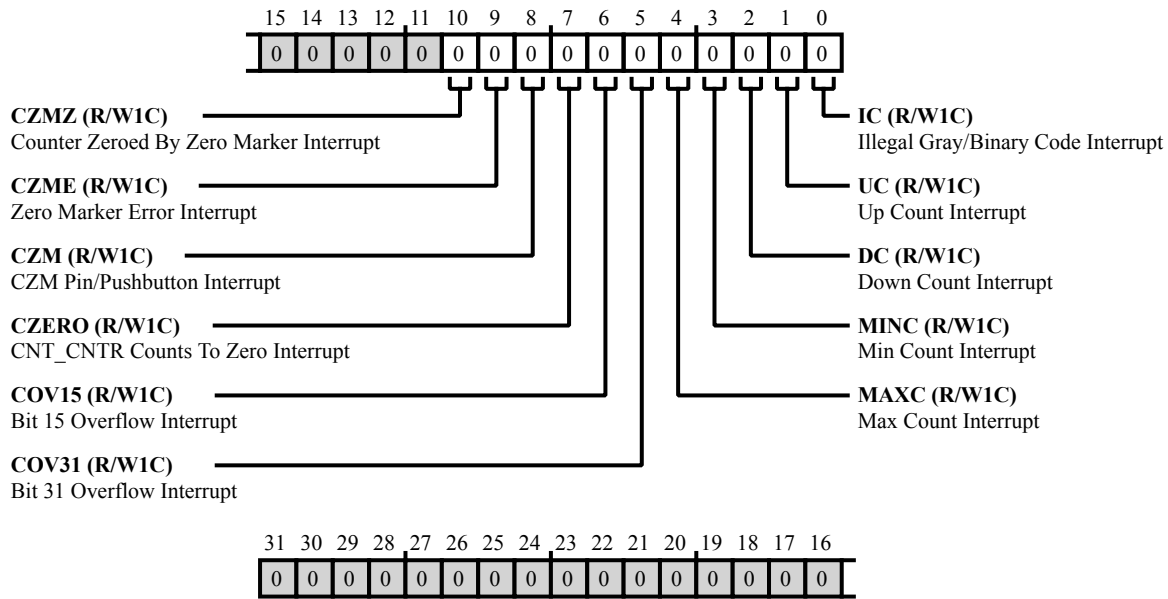


Figure 21-11: CNT_STAT Register Diagram

Table 21-15: CNT_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W1C)	CZMZ	Counter Zeroed By Zero Marker Interrupt. The CNT_STAT.CZMZ bit indicates a zero marker error. If the CNT_CFG.ZMZC bit =1, this interrupt request is generated when the CZMII latch reports a significant edge on the CZM input. Once cleared by software the CNT_STAT.CZM bit is not set again when the CZM input remains active without pulsing.
		0 No error
		1 Error occurred

Table 21-15: CNT_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W1C)	CZME	Zero Marker Error Interrupt. The CNT_STAT.CZME bit behaves similarly to the CNT_STAT.CZM bit, with the exception that CNT_STAT.CZME is not set on the CZM edge when the lower four bits of the CNT_CNTR are not zero. In many applications this indicates an error condition, as the zero marker might be out of sync with the counter.
		0 No error
		1 Error occurred
8 (R/W1C)	CZM	CZM Pin/Pushbutton Interrupt. The CNT_STAT.CZM bit indicates a CZM pin/pushbutton error. This interrupt request is generated when a significant edge is seen on the CZM pin, regardless what mode the counter is operating in. This is often used to sense push buttons (especially with the debouncing circuit enabled).
		0 No error
		1 Error occurred
7 (R/W1C)	CZERO	CNT_CNTR Counts To Zero Interrupt. The CNT_STAT.CZERO bit indicates a counts to zero error. This error is generated when the CNT_CNTR register has incremented or decremented toward 0x0000.0000. The latch is not set when software writes to the CNT_CNTR register directly or when the counter is zeroed by writes to the CNT_CMD register.
		0 No error
		1 Error occurred
6 (R/W1C)	COV15	Bit 15 Overflow Interrupt. The CNT_STAT.COV15 bit indicates a bit 15 overflow error. This error is generated when the 16-bit twos-complement CNT_CNTR register has incremented from 0xxxxx.7FFF to 0xxxxx.8000 or decremented from 0xxxxx.8000 to 0xxxxx.7FFE.
		0 No error
		1 Error occurred
5 (R/W1C)	COV31	Bit 31 Overflow Interrupt. The CNT_STAT.COV31 bit indicates a bit 31 overflow error. This error is generated when the 32-bit twos-complement CNT_CNTR register has incremented from 0x7FFF.FFFF to 0x8000.0000 or decremented from 0x8000.0000 to 0x7FFF.FFFF.
		0 No error
		1 Error occurred

Table 21-15: CNT_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W1C)	MAXC	Max Count Interrupt. The CNT_STAT.MAXC bit indicates a max count error. This interrupt is used in boundary compare (BND_COMP) mode. If after incrementing the CNT_CNTR register equals CNT_MAX, the CNT_STAT.MAXC bit is set.
		0 No error
		1 Error occurred
3 (R/W1C)	MINC	Min Count Interrupt. The CNT_STAT.MINC bit indicates a minimum count error. This interrupt is used in boundary compare (BND_COMP) mode. If, after decrementing, the CNT_CNTR register equals CNT_MIN, the CNT_STAT.MINC bit is set.
		0 No error
		1 Error occurred
2 (R/W1C)	DC	Down Count Interrupt. The CNT_STAT.DC bit indicates a down count error. This interrupt is generated when the CNT_CNTR register decrements.
		0 No error
		1 Error occurred
1 (R/W1C)	UC	Up Count Interrupt. The CNT_STAT.UC bit indicates an up count interrupt. This interrupt is generated when the CNT_CNTR register increments.
0 (R/W1C)	IC	Illegal Gray/Binary Code Interrupt. The CNT_STAT.IC bit indicates a illegal Gray/Binary Code interrupt and should only be used in these modes. In normal operation those codes can increment or decrement the CNT_CNTR register by one at a time. If the sensed inputs instruct the counter to increment or decrement by two, the CNT_STAT.IC bit is set. Hardware sets the CNT_STAT.IC bit in QUAD_ENC and BIN_ENC encoder modes only.
		0 No error
		1 Error occurred

22 Media Local Bus (MLB)

Media Local Bus (MediaLB[®]) is an on-PCB or inter-chip communication bus, which allows an application to access the MOST network data. Media Local Bus supports all the MOST network data transport methods including synchronous stream data, asynchronous packet data, control message data and isochronous data. The MLB topology supports communication among the MLB controller and MLB devices, where the MLB controller is the interface between the MLB devices and the MOST network.

The MLB module serves as an interface between the MediaLB and the processor, implementing the requirements of the physical layer and the link layer outlined in the MediaLB specification. It supports up to 64 logical channels with up to 468 bytes of data per MediaLB frame. Transmit and receive data can be transferred between MediaLB and on-chip memory with DMA block transfers.

The MLB supports the MOST25, MOST50 and MOST150 standards and this document assumes familiarity with these standards. For more information, refer to the Media Local Bus specification version 4.2.

- Copyright 1998-2016 Microchip Technology Inc. All rights reserved. Portions of this chapter are included with permission from Microchip Technology, Inc.

Features

The objective of MLB is to map all the MOST Network data types (transport methods) into a single low-cost, scalable, and standardized hardware interface between a MediaLB controller and at least one other MediaLB Device. The adoption of MediaLB simplifies the hardware interface, reduces the pin count, and facilitates the design of modular reusable hardware. From a software development perspective, the use of MediaLB relieves the system developer from the complexity of the MOST network, which simplifies software development and enables the design of reusable software for different applications. This simplified, standardized interface shortens time to market and makes software maintenance effortless.

- Compliant to Media Local Bus specification version 4.2
- Support MOST25, MOST50, MOST150 standards
- Support 3-pin and 6-pin mode
 - 6-pin mode supports a data rate of $2048 \times \text{FS}$
 - 3-pin mode support various data rates of $256 \times \text{FS}$, $512 \times \text{FS}$, $1024 \times \text{FS}$

- Support 64 logic channels
- Dedicated PLL for clock recovery and phase alignment
- Dedicated pins for 6-pin mode (LVDS)
- Shared pins for 3-pin mode
- Dedicated internal RAM for data buffering and channel table
- Recovered clock (after division) available out on a shared pin for DAI

NOTE: Only one MLB interface (3-pin mode or 6-pin mode) is active at any given time.

MLB Definitions

The following are standard MLB and MOST terms that are used in this chapter.

AGU

Address Generation Units. To access a particular HBI channel the HC must first configure one of two HBI address generation units. AGUs can be configured by writing the HBI command registers, HCMD0 and HCMD1.

CAT

Channel Allocation Table. The Channel Allocation Table (CAT) is comprised of 16 CTR entries. Each 16-bit CAT entry represents a logical connection to or from a transmit/receive device (for example MediaLB or HBI channel).

CDT

Channel Descriptor Table.

FCE

Flow Control Enable bit. The FCE bit is used by MediaLB isochronous Rx channels only.

HBI

Host Bus Interface. The HBI block provides 16-bit parallel slave access to all MOST channels and data types for the external Host Controller (HC). The HBI supports up to 64 independent channels.

HC

Host Controller (external).

HCMDx

HBI Command registers.

HSTSx

HBI status registers.

MFE

Multi-Frame per sub-buffer enable bit. The MFE bit is used by MediaLB synchronous channels only.

PML

Packet Message Length.

Clocking

The MLB controller provides an external clock pin—the media local bus clock. The MLB controller generates the clock. It is synchronized to the MOST network and provides the timing for the entire MLB interface at $FS = 48$ kHz.

Functional Description

The *MediaLB Block Diagram* figure shows the MLB high-level architecture. The MLB core serves as an interface between the MediaLB and the processor, implementing the requirements of the physical layer and the link layer outlined in the MediaLB specification. The MLB core has the following responsibilities.

- Transmit commands and data when functioning as the transmitting device associated with a *Channel Address*
- Receive data and transmit Rx status responses when functioning as the receiving device associated *Channel Address*
- MLB lock detection
- System channel command handling

The MediaLB device can function as either a MediaLB 3-pin interface (single-ended) or MediaLB 6-pin interface (differential) but only one interface can be active at a time. The MediaLB interfaces are capable of exchanging data at speeds up to $1024 \times Fs$ in 3-pin mode or $2048 \times Fs$ in 6-pin mode.

A set of physical channels for exchanging data over the MediaLB bus is supported. These physical channels (4 bytes in length, or a quadlet) can be grouped into logical channels, where each logical channel is referenced using a channel address and represents a uni-directional datapath between a specific MediaLB device transmitting the data and the MediaLB device(s) receiving the data. The MediaLB 6-pin interface provides support for up to 468 bytes of data per frame. The logical channels, configured by system software, can be any combination of channel types (synchronous, asynchronous, isochronous, or control) and direction (transmit or receive).

ADSP-2159x MLB Register List

The MediaLB Device Interface Macro 2 (MediaLB DIM 2), also referred to as OS62420, implements the required functionality of a Media Local Bus (MediaLB) device. This logic serves as an interface between the inter-chip MediaLB bus and a customer IC, implementing the physical- and link-layer requirements outlined in the MediaLB Specification.

Table 22-1: ADSP-2159x MLB Register List

Name	Description
MLB_ACMR0	Peripheral Channel Mask 0 Register
MLB_ACMR1	Peripheral Channel Mask 1 Register
MLB_ACSR0	Peripheral Channel Status 0 Register
MLB_ACSR1	Peripheral Channel Status 1 Register
MLB_ACTL	Bus Control Register
MLB_CTL0	MediaLB Control 0 Register
MLB_CTL1	Control 1 Register
MLB_GCTL	MLB Global Control Register
MLB_HCBR0	HBI Channel Busy 0 Register
MLB_HCBR1	HBI Channel Busy 1 Register
MLB_HCER0	HBI Channel Error 0 Register
MLB_HCER1	HBI Channel Error 1 Register
MLB_HCMR0	HBI Channel Mask 0 Register
MLB_HCMR1	HBI Channel Mask 1 Register
MLB_HCTL	HBI Control Register
MLB_MADR	Memory Interface Address Register
MLB_MCTL	Memory Interface Control Register
MLB_MDAT0	Memory Interface Control Data 0 Register
MLB_MDAT1	Memory Interface Control Data 1 Register
MLB_MDAT2	Memory Interface Control Data 2 Register
MLB_MDAT3	Memory Interface Control Data 3 Register
MLB_MDWE0	Memory Interface Control Data Write Enable 0 Register
MLB_MDWE1	Memory Interface Control Data Write Enable 1 Register
MLB_MDWE2	Memory Interface Control Data Write Enable 2 Register
MLB_MDWE3	Memory Interface Control Data Write Enable 3 Register
MLB_MIEN	Interrupt Enable Register

Table 22-1: ADSP-2159x MLB Register List (Continued)

Name	Description
MLB_MS0	Channel Status 0 Register
MLB_MS1	Channel Status 1 Register
MLB_MSD	System Data Register
MLB_MSS	System Status Register
MLB_PCTL0	MediaLB 6-pin Control 0 Register

ADSP-2159x MLB Interrupt List

Table 22-2: ADSP-2159x MLB Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
187	MLB0_INT0	MLB0 Interrupt 0		
188	MLB0_INT1	MLB0 Interrupt 1		
189	MLB0_STAT	MLB0 Status		

MediaLB Protocol

The MediaLB topology supports communication among all MediaLB devices, including the MediaLB controller. The bus interface consists of a uni-directional line for clock (MLBC), a bidirectional line for signal information (MLBS), and a bidirectional line for data transfer (MLBD). The MediaLB topology supports one controller connected to one or more devices, where the controller is the interface between the MediaLB devices and the MOST network.

The MediaLB controller includes MediaLB device functionality, and also generates the MediaLB clock (MLBC) that is synchronized to the MOST Network. This generated clock provides the timing for the entire MediaLB interface. The MLBS line is a multiplexed signal which carries channel addresses generated by the MediaLB controller, as well as command and RxStatus bytes from MediaLB devices. The MLBD line is driven by the transmitting MediaLB device and is received by all other MediaLB devices, including the MediaLB controller. The MLBD line carries the actual data (synchronous, asynchronous, control, or isochronous).

Once per MOST network frame, the MLB controller generates a unique frame sync pattern on the MLB_SIG line. The end of the frame sync pattern defines the byte boundary and the channel boundary for the MLB_SIG and MLB_DAT lines of all MLB devices.

The MLB controller manages the arbitration for all the channels on the MLB and grants bandwidth for all the MLB devices. An MLB *physical channel* is defined as four bytes wide, or a quadlet. Physical channels can be grouped into multiple quadlets (which do not have to be consecutive) to form an MLB *logical channel*, which is defined by a unique channel address.

As shown in *MLB Data Structure*, the MLB controller initiates communication by sending out a channel address on the MLB_SIG line for each physical channel. The channel address indicates which MLB device is transmitting and which MLB devices are receiving in the following physical channel. Therefore, four bytes after the controller outputs the channel address on the MLB_SIG line, the transmitting device outputs a command byte command on the MLB_SIG line and outputs the respective data on the MLB_DAT line, concurrently. The MLB command byte contains the type of data currently being transmitted (for example synchronous, asynchronous or control).

The MLB device receiving the channel data outputs a status byte, RxStatus, on the MLB_SIG line immediately after the transmitting device outputs the command byte. The status response can indicate that the receiving device is busy and cannot receive the data at present, or the device is ready to receive the data. Since synchronous stream data is sent in a broadcast fashion, receiving devices cannot return a busy status and should not drive RxStatus onto the MLB_SIG line.

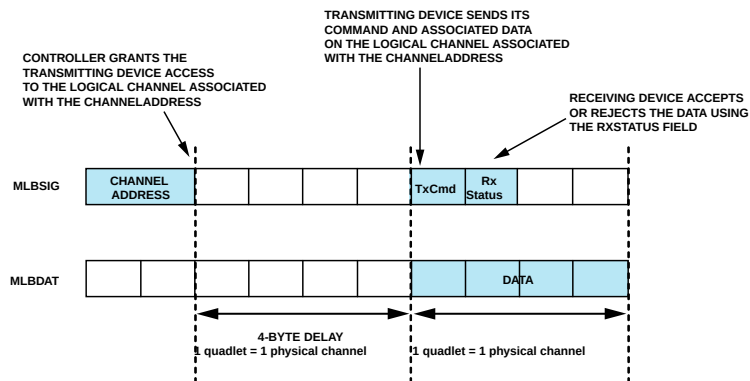


Figure 22-1: MLB Data Structure

MLB Architectural Concepts

The following sections provide information about the MLB architecture.

MediaLB Block Diagram

The *MediaLB Block Diagram* shows the various blocks within the interface its connections to the processor.

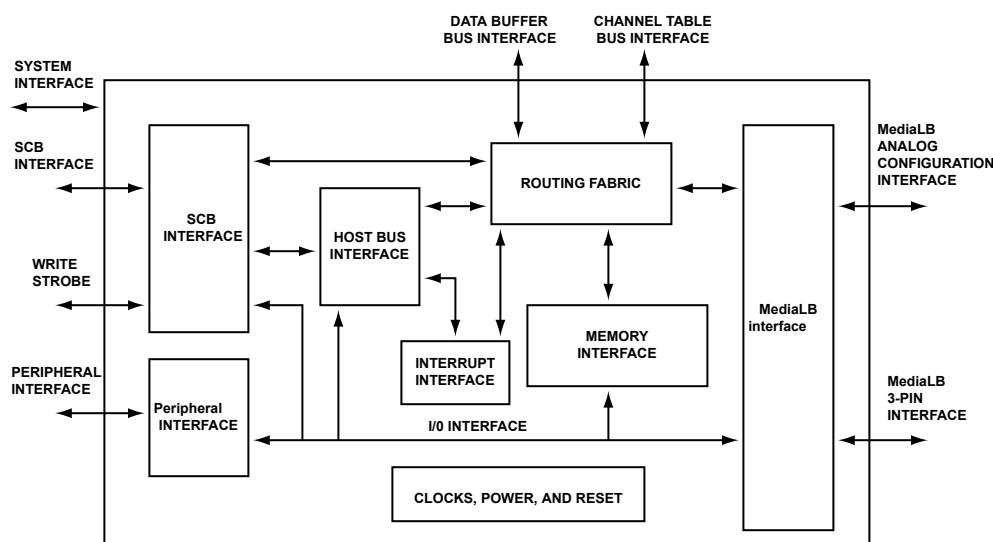


Figure 22-2: MediaLB Block Diagram

MediaLB Interface

The Media Local Bus (MediaLB) block supports both a MediaLB 3-pin interface and MediaLB 6-pin interface; however, only one MediaLB interface can be active at any given time. Both MediaLB interfaces provide real-time access to all network data types - synchronous, asynchronous, control, and isochronous data.

- MediaLB 3-pin interface – Supports the MediaLB protocol for single-ended 3-pin mode, with a maximum data rate of 1024 FS (49.152 MHz at FS = 48 kHz).
- MediaLB 6-pin interface – Supports the MediaLB protocol for high-speed differential 6-pin mode, with a maximum data

maximum data rate of 2048 FS (98.304 MHz at FS = 48 kHz).

MediaLB Channel Address to Logical Channel Mapping

The MediaLB channel addresses are mapped to the logical channels as shown in the *MediaLB Channel Address to Logical Channel Mapping* table.

Table 22-3: MediaLB Channel Address to Logical Channel Mapping

Channel Address	Logical Address
0x0002	1
0x0004	2
0x0006	3
...	...
0x007C	62
0x007E	63

Table 22-3: MediaLB Channel Address to Logical Channel Mapping (Continued)

Channel Address	Logical Address
0x01FE	0 (Logical channel 0 is the system channel and is reserved)

Routing Fabric

The Routing Fabric (RF) block manages the flow of data between the MediaLB port and the HBI port. It manages accessing the Channel Table RAM (CTR) and Data Buffer RAM (DBR), which are explained in the following sub-sections. The routing fabric uses channel descriptors (stored in the CTR) to manage access to dynamic buffers in the DBR.

Data Buffer RAM

The Data Buffer RAM (DBR) is an 8-bit x 16k entries deep, single-port synchronous SRAM and provides dynamic circular buffering between the transmit and receive devices. The size and location of each data buffer is defined by software in the Channel Descriptor Table (CDT), which is located in the Channel Table RAM (CTR), described in following sections.

Channel Table RAM

The Channel Table RAM (CTR) is a 128-bit x 144-entry table that allows system software to dynamically configure channel routing and allocate data buffers in the DBR. The CTR is logically divided into three tables:

- [Channel Descriptor Tables](#)
- [AHB Descriptor Table \(ADT\)](#)
- [Channel Allocation Table](#)

Address Mapping

The *CTR Address Mapping* table shows the address mapping for the CTR.

Table 22-4: CTR Address Mapping

Label	Address	Bits [127:96]	Bits [95:64]	Bits [63:32]	Bits [31:0]
Channel Descriptor Table (CDT)					
CDT	0x00	CDT0[127:0], CL = 0			
	0x01	CDT1[127:0], CL = 1			
	0x02	CDT2[127:0], CL = 2			
	...				
	0x3D	CDT61[127:0], CL = 61			
	0x3E	CDT62[127:0], CL = 62			
	0x3F	CDT63[127:0], CL = 63			
AHB Descriptor Table (ADT)					

Table 22-4: CTR Address Mapping (Continued)

Label	Address	Bits [127:96]		Bits [95:64]		Bits [63:32]		Bits [31:0]	
ADT	0x40	ADT0[127:0], CL = 0							
	0x41	ADT1[127:0], CL = 1							
	0x42	ADT2[127:0], CL = 2							
	...								
	0x7D	ADT61[127:0], CL = 61							
	0x7E	ADT62[127:0], CL = 62							
	0x7F	ADT63[127:0], CL = 63							
Channel Allocation Table (CAT)									
CAT for Medi-aLB	0x80	CAT7	CAT6	CAT5	CAT4	CAT3	CAT2	CAT1	CAT0
	...	...	...	...	...	...	...	...	...
	0x87	CAT63	CAT62	CAT61	CAT60	CAT59	CAT58	CAT57	CAT56
CAT for HBI* ¹	0x88	CAT71	CAT70	CAT69	CAT68	CAT67	CAT66	CAT65	CAT64
	...	...	...	...	...	...	...	...	...
	0x8F	CAT127	CAT126	CAT125	CAT124	CAT123	CAT122	CAT121	CAT120

^{*1} A fixed relationship exists between ADT entries and HBI CAT entries. When using HBI channel 0 (CAT64), program ADT0. When using HBI channel 1 (CAT65), program ADT1, and so on.

Channel Allocation Table

The *Channel Allocation Table (CAT)* table is comprised of 16 CTR entries (addresses 0x80 0x8F) as shown in the *CTR Entry Map* table. Each 16-bit CAT entry represents a logical connection to or from a transmit or receive device. (for example, MediaLB channel). All entries are indexed according to a fixed physical address assigned to every RX/TX channel as shown in the *CAT Entry Formats* table. The value stored in a CAT entry includes a 6-bit connection label, which provides a pointer to the CDT. To complete a logical channel and form a routing connection, system software must assign the same connection label to both the RX and TX channels.

Table 22-5: CAT Entry Map

Peripheral	TX Channels	RX Channels	CAT Start Index	CAT End Index	Entries
MediaLB	0 to 64	64 TX Channels	0	63	64
HBI	0 to 64	64 TX Channels	64	127	64

The format of a full CAT entry is shown in the *CAT Entry Formats* table, with field descriptions described in the *CAT Field Definitions* table. All reserved bits of a CAT entry field should be written as zero.

Table 22-6: CAT Entry Formats

Channel Type	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Isochronous	rsvd	FCE	rsvd	RNW	CE	CT[2:0] = 011			rsvd	CL[5:0]						
Asynchronous	rsvd		MT	RNW	CE	CT[2:0] = 010			rsvd	CL[5:0]						
Control	rsvd		MT	RNW	CE	CT[2:0] = 001			rsvd	CL[5:0]						
Synchronous	rsvd	MFE	MT	RNW	CE	CT[2:0] = 000			rsvd	CL[5:0]						

Table 22-7: CAT Field Definitions

Field	Description
CL[5:0]	Connection Label (offset into CDT)
CT[2:0]	Channel Type (others) 111 = Reserved 110 = Reserved 101 = Reserved 100 = Reserved 011 = Isochronous 010 = Asynchronous 001 = Control 000 = Synchronous
CE	Channel Enable. 0 = Disabled, 1 = Enabled
RNW	Read Not Write. 0 = Write, 1 = Read
MT	Mute Enable. 0 = Disabled When set for synchronous channels, the MT bit forces RX channels to write zeros into the channel data buffer, and TX channels to output zeros on the physical interface. When set for asynchronous and control channels, the MT bit causes DMA to halt at a packet boundary. Not valid for isochronous channels.
FCE	Flow Control Enable. 0 = Disabled, 1 = Enabled The FCE bit is used by MediaLB isochronous RX channels only.
MFE	Multi-Frame per sub-buffer enable. 0 = Disabled, 1 = Enabled The MFE bit is used by MediaLB synchronous channels only.
rsvd	Reserved. Software writes a 0 to all reserved bits when the entry is initialized. These bits are read-only after initialization.

Channel Set Up

Data direction is in reference to the DBR. The data direction of CAT entries corresponding to the same channel is reversed for the HBI CAT and the MediaLB CAT.

- For a Tx channel (from the HC to the MediaLB interface):
 - HBI CAT entry: RNW = 0 (write)

- MediaLB CAT entry: RNW = 1 (read)
- Conversely, for an Rx channel (data from MediaLB to HC):
 - HBI CAT entry: RNW = 1 (read)
 - MediaLB CAT entry: RNW = 0 (write)

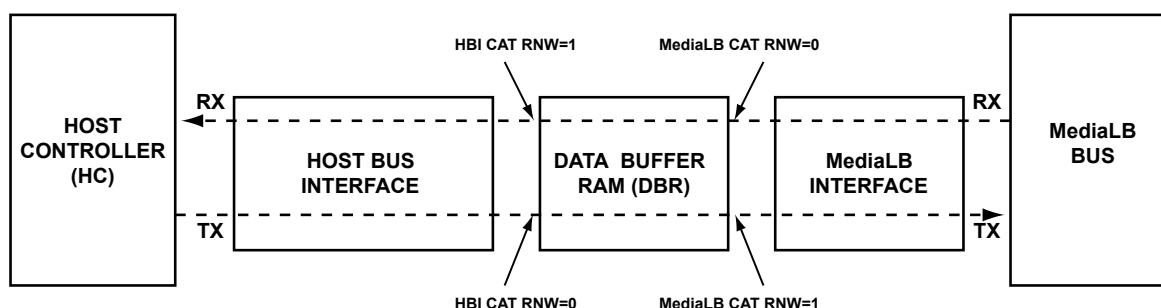


Figure 22-3: DBR Directional Relationship

Channel Descriptor Tables

The Channel Descriptor Table (CDT) is comprised of 64 CTR entries (addresses 0x00 - 0x3F), as shown in the [Table 22-4 CTR Address Mapping](#) table. Each 128-bit CDT entry (also referred to as a channel descriptor) is referenced by a connection label and contains information about a data buffer in the DBR (for example buffer size, address pointers). The format of each CDT entry is dependent on the channel type (synchronous, isochronous, asynchronous, or control).

NOTE: All reserved channel descriptor bits must be written to 0 by software when initialized.

Synchronous Channel Descriptors

The format and field definitions for a synchronous CDT entry are shown in the *Synchronous CDT Entry Format* and *Synchronous CDT Entry Field Definitions* tables.

Table 22-8: Synchronous CDT Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	WSBC[1:0]		Reserved													
16	RSBC[1:0]		Reserved													
32	Reserved															
48	Reserved															
64	WSTS[3:0]				WPTR[11:0]											
80	RSTS[3:0]				RPTR[11:0]											
96	Reserved				BD[11:0]											
112	Reserved		BA[13:0]													

Table 22-9: Synchronous CDT Entry Field Definitions

Field	Description	Details	Access
BA	Buffer Base Address	Can start at any byte in the 16k DBR	RW
BD	Buffer Depth	BD = size of buffer in bytes 1. Buffer end address = BA + BD. BD = 4 m bpf 1 where: m = frames per sub-buffer (for MFE = 0, m = 1) bpf = bytes per frame.	RW
RPTR	Read Pointer	Software initializes to 0, hardware updates. Counts the read address offset within a buffer. DMA read address = BA + RPTR.	RWU
WPTR	Write Pointer	Software initializes to 0, hardware updates. Counts the write address offset within a buffer. DMA read address = BA + WPTR.	RWU
RSBC	Read Sub-buffer Counter	Software initializes to 0, hardware updates. Counts the read sub-buffer offset. DMA uses for pointer management.	RWU
WSBC	Write Sub-buffer Counter	Software initializes to 0, hardware updates. Counts the write sub-buffer offset. DMA uses for pointer management.	RWU
RSTS	Read Status	Software initializes to 0, hardware updates. RSTS States: xxx0 = normal operation (no mute) xxx1 = normal operation (mute) xx0x = idle	RWU
WSTS	Write Status	Software initializes to 0, hardware updates. WSTS States: xxx0 = normal operation (no mute) xxx1 = normal operation (mute) xx0x = idle 1xxx = command protocol error	RWU
Reserved. Software writes a 0 to all these bits when the entry is initialized. Reserved bits are RO after initialization.			RWU

Isochronous Channel Descriptors

The format and field definitions for a synchronous CDT entry are shown in the *Isochronous Entry Formats* and *Isochronous CDT Entry Field Definitions* tables.

Table 22-10: Isochronous Entry Formats

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Reserved															
16	Reserved															
32	Reserved								BS[8:0]							
48	Reserved															
64	WSTS[3:0]				WPTR[12:0]											
80	RSTS[3:0]				RPTR[12:0]											
96	Reserved				BD[12:0]											
112	BF	Rsvd	BA[13:0]													

Table 22-11: Isochronous CDT Entry Field Definitions

Field	Description	Details	Access
BA	Buffer Base Address	Can start at any byte in the 16k DBR.	RW
BD	Buffer Depth	BD = size of buffer in bytes 1. Buffer end address = BA + BD. Isochronous buffers must be large enough to hold at least 3 blocks (packets) of data. BD Must be an integer multiple of blocks.	RW
BF	Buffer Full	Software initializes to 0, hardware updates. DMA write hardware sets BF when the buffer is full. DMA read hardware clears BF when the buffer is empty. BF is valid only when buffer is full or empty, otherwise ignore.	RWU
BS	Block Size	BS defines when to begin the DMA to the data buffer. BS = buffer block size in bytes 1. For RX channels, the DMA writes start when the number of empty bytes in the data buffer the block size. For TX channels, the DMA reads start when the number of valid bytes in the data buffer the block size.	RWU
RPTR	Read Pointer	Software initializes to 0, hardware updates. Counts the read address offset within a buffer. DMA read address = BA + RPTR.	RWU
WPTR	Write Pointer	Software initializes to 0, hardware updates. Counts the write address offset within a buffer. DMA write address = BA + WPTR.	RWU

Table 22-11: Isochronous CDT Entry Field Definitions (Continued)

Field	Description	Details	Access
RSTS	Read Status	Software initializes to 0, hardware updates. RSTS States: xx1 = active xx0 = idle	RWU
WSTS	Write Status	Software initializes to 0, hardware updates. WSTS States: xxx0 = active xxx1 = idle xx0x = command protocol error 1xxx = buffer overflow (FCE = 0 only)	RWU
Reserved. Software writes a 0 to all these bits when the entry is initialized. Reserved bits are RO after initialization.			RWU

Asynchronous/Control Channel Descriptors

The format and field definitions for an Asynchronous/Control CDT entry are shown in the *Asynchronous/Control CDT Entry Format* and *Asynchronous/Control CDT Entry Field Definitions* tables.

Table 22-12: Asynchronous/Control CDT Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	WPC[4:0]					Reserved										
16	RPC[4:0]					Reserved										
32	Rsvd	WPC[7:5]			Reserved											
48	Rsvd	RPC[7:5]			Reserved											
64	WSTS[3:0]				WPTR[11:0]											
80	RSTS[3:0]				RPTR[11:0]											
96	RSTS[4]	WSTS[4]	Rsvd		BD[11:0]											
112	Rsvd		BA[13:0]													

Table 22-13: Asynchronous/Control CDT Entry Field Definitions

Field	Description	Details	Access
BA	Buffer Base Address	Can start at any byte in the 16k DBR.	RW
BD	Buffer Depth	BD = size of buffer in bytes 1. Buffer end address = BA + BD. BD = max packet length 1.	RW

Table 22-13: Asynchronous/Control CDT Entry Field Definitions (Continued)

Field	Description	Details	Access
RPC	Read Packet Count	Software initializes to 0, hardware updates. Used with RPC, RPTR and WPTR to determine if the buffer is empty or full.	RWU
WPC	Write Packet Count	Software initializes to 0, hardware updates. Used with RPC, RPTR and WPTR to determine if the buffer is empty or full.	RWU
RPTR	Read Pointer	Software initializes to 0, hardware updates. Counts the read address offset within a buffer. DMA read address = BA + RPTR.	RWU
WPTR	Write Pointer	Software initializes to 0, hardware updates. Counts the write address offset within a buffer. DMA read address = BA + WPTR.	RWU
RSTS	Read Status	Software initializes to 0, hardware updates. RSTS States: x0x00 = idle xx1xx = <i>ReceiverProtocolError</i> response received from RX device 1xxxx = <i>ReceiverBreak</i> command received from RX device	RWU
WSTS	Write Status	Software initializes to 0, hardware updates. Status States (only valid for DMA pointers associated with the MLB block, not HBI block pointers): xxx0 = idle xxx1 = command protocol error detected xx0x = <i>AsyncBreak/ControlBreak</i> command received from TX device	RWU
Reserved. Software writes a 0 to all these bits when the entry is initialized. Reserved bits are RO after initialization.			RWU

AHB Descriptor Table (ADT)

The AHB block manages data exchange between local channel data buffers within MLB module and the system memory buffer. To support system memory buffering, a ping-pong memory structure is implemented on a per channel basis using 128-bit descriptors for AHB Descriptor Table (ADT) entries. The [Table 22-4 CTR Address Mapping](#) table provides a complete address map of the CTR, including the location of the ADT.

Each logical channel is assigned a separate 128-bit descriptor, defining the data buffers in the system memory used by the DMA interface for that channel. The descriptors are stored at fixed addresses in the CTR as described in previous section. The *ADT Field Definitions* table provides an overview of field definitions for ADT entries.

Table 22-14: ADT Field Definitions

Field	No. of Bits	Description	Access
CE	1	Channel Enable. 0 = Disabled	RW, U
LE	1	Endianness Select. 0 = Big Endian, 1 = Little Endian	RW
PG	1	Page pointer. Software initializes to 0, hardware writes thereafter. 0 = Ping buffer, 1 = Pong buffer	RW, U
RDY1	1	Buffer Ready bit for ping buffer page. 0 = Not ready, 1 = Ready	RW
RDY2	1	Buffer Ready bit for pong buffer page. 0 = Not ready, 1 = Ready	RW
DNE1	1	Buffer Done bit for ping buffer page. 0 = Not done, 1 = Done	R, U, c0
DNE2	1	Buffer Done bit for pong buffer page. 0 = Not done, 1 = Done	R, U, c0
ERR1	1	Error Response detected for ping buffer page. 0 = No error, 1 = Error	R, U, c0
ERR2	1	Error Response detected for pong buffer page. 0 = No error, 1 = Error	R, U, c0
PS1	1	Packet Start bit for ping buffer page. 0 = No packet start, 1 = Packet start Reserved for synchronous and isochronous channels.	RW, U both TX and RX
PS2	1	Packet Start bit for pong buffer page. 0 = No packet start, 1 = Packet start Reserved for synchronous and isochronous channels.	RW, U both TX and RX
MEP1	1	Most Ethernet Packet indicator for ping buffer page. 0 = Not MEP, 1 = MEP. MEP1 only valid for the first page of a segmented buffer. Reserved for control synchronous and isochronous channels.	Rsvd for TX, R, U, c0 for RX
MEP2	1	Most Ethernet Packet indicator for pong buffer page. 0 = Not MEP, 1 = MEP. MEP2 only valid for the first page of a segmented buffer. Reserved for control synchronous and isochronous channels.	Rsvd for TX, R, U, c0 for RX
BD1	11 to 13	Buffer Depth for ping buffer page. 11 or 12 bits for asynchronous and control channels. 13 bits for synchronous and isochronous channels.	RW
BD2	11 to 13	Buffer Depth for pong buffer page. 11 or 12 bits for asynchronous and control channels. 13 bits for synchronous and isochronous channels.	RW
BA1	32	Buffer Base Address for ping buffer page	RW
BA2	32	Buffer Base Address for pong buffer page.	RW
Reserved	varies	Reserved. Software writes a 0 to all these bits when the entry is initialized. Reserved bits are RO after initialization.	RW, U

The *Ping-Pong System Memory Structure* figure shows that this system memory structure is similar for all channel types and shows the relationship between the BAn, BDn, and PG descriptor fields.

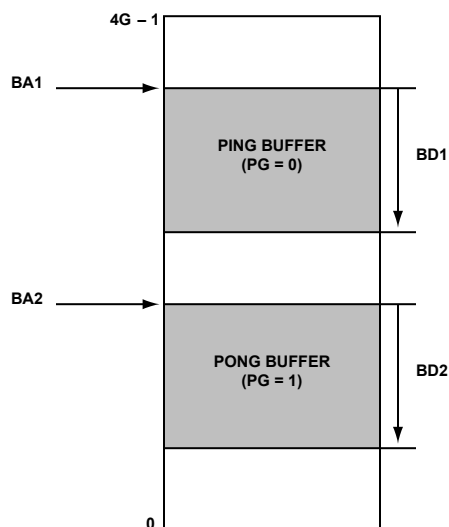


Figure 22-4: Ping-Pong System Memory Structure

Each ADT entry (also referred to as a *Channel Descriptor*) holds a 32-bit BAn field which defines the start of each ping or pong buffer within system memory. The BDn field is used to indicate the size for the respective ping or pong page. The maximum size is 2k entries for asynchronous and control channels and 8k entries for isochronous and synchronous channels.

Synchronous Channel Descriptors

The synchronous buffering scheme allows each ping or pong buffer to contain a single frame or a multiple number of frames. For this reason, the synchronous buffer depth (BDn) must be defined in terms of an integer number (n), frames per sub-buffer (m) and bytes per frame (bpf) of data (for example $BDn = n \ m \ bpf \ 1$). The *Synchronous ADT Entry Format* table shows the format for a synchronous ADT entry. The field definitions are defined in the *ADT Field Definitions* table. Each synchronous channel buffer can be up to 8k-bytes deep.

Table 22-15: Synchronous ADT Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	CE	LE	PG	Reserved												
16	Reserved															
32	RDY1	DNE1	ERR1	BD1[12:0]												
48	RDY2	DNE2	ERR2	BD2[12:0]												
64	BA1[15:0]															
80	BA1[31:16]															
96	BA2[15:0]															
112	BA2[31:16]															

Isochronous Channel Descriptors

The isochronous buffering scheme allows each ping or pong buffer to contain a single block or a multiple number of blocks. For this reason, the isochronous buffer depth (BD_n) must be defined in terms of an integer number (n) and block size (BS) (for example BD_n = n (BS + 1) 1).

The *Isochronous ADT Entry Format* table shows the format for an isochronous ADT entry. The field definitions are defined in the *ADT Field Definitions* table. Each isochronous channel buffer can be up to 8k-bytes deep.

Table 22-16: Isochronous ADT Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	CE	LE	PG	Reserved												
16	Reserved															
32	RDY1	DNE1	ERR1	BD1[12:0]												
48	RDY2	DNE2	ERR2	BD2[12:0]												
64	BA1[15:0]															
80	BA1[31:16]															
96	BA2[15:0]															
112	BA2[31:16]															

Asynchronous and Control Channel Descriptors

Every asynchronous and control packet adheres to the Port Message Protocol (PMP), which designates the first two bytes of each packet as the packet length (PML). Each packet must be no more than 2048-bytes (PML 2048).

Software must set the buffer ready bit (RDY_n) for each buffer as it programs the DMA. As hardware processes each buffer, it sets the done bit (DNE_n) and generates an interrupt to inform HC. When hardware finishes processing a buffer, it can begin processing another buffer if RDY_n is set. The application is responsible for setting up and configuring the channel buffer descriptor prior to every DMA access on the channel.

Two packet buffering modes are supported by hardware for programming the DMA, single-packet mode (MLB_ACTL.MPB = 0) and multiple-packet mode (MLB_ACTL.MPB = 1). The MPB is written prior to enabling the channel DMA.

Single Packet Mode. The single-packet mode asynchronous and control buffering scheme supports a maximum of one packet per buffer (for example, ping or pong). Both non-segmented and segmented data packets are allowed while using single-packet mode. Non-segmented packets are exchanged when only one buffer (for example, ping or pong) is needed for packet transfer. Segmented packets are exchanged when a single packet is too long for one buffer and the packet must span multiple buffers.

The *Single-Packet Asynchronous and Control Entry Format* table shows the format for single-packet mode asynchronous and control ADT entries. The field definitions are defined in the *ADT Field Definitions* table.

Table 22-17: Single-Packet Asynchronous and Control Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	CE	LE	PG	Reserved												
16	Reserved															
32	RDY1	DNE1	ERR1	PS1	MEP1	BD1[10:0]										
48	RDY2	DNE2	ERR2	PS2	MEP2	BD2[10:0]										
64	BA1[15:0]															
80	BA1[31:16]															
96	BA2[15:0]															
112	BA2[31:16]															

Multiple Packet Mode. The multiple-packet mode asynchronous and control buffering scheme supports more than one packet per system memory buffer, as shown in the *Asynchronous/Control CDT Entry Format* table. Multiple-packet mode reduces the interrupt rate for packet channels at the cost of increasing buffering and latency.

For TX packet channels in multiple-packet mode, software sets the packet start bit (PS_n) for every buffer. Setting PS_n informs hardware that the first two bytes of the buffer contains the port message length (PML) of the first packet. After the first packet, hardware keeps track of where packets start and end within the current buffer. Software should not write to PS_n while the buffer is active (RDY_n = 1 and DNE_n = 0). For TX packet channels, the buffer is done (DNE_n = 1) when the last byte of the last packet in the buffer is read from system memory. Software should set the buffer depth to contain the exact number of complete packets for that buffer. Segmented buffers are not supported for TX packet channels in multiple-packet mode.

NOTE: The PS1 and PS2 bits are only valid for TX channels. Set PS1 and PS2 = 1 at the start of the buffer.

Table 22-18: Multiple-Packet Asynchronous and Control Entry Format

Bit Offset	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	CE	LE	PG	Reserved												
16	Reserved															
32	RDY1	DNE1	ERR1	PS1	BD1[11:0]											
48	RDY2	DNE2	ERR2	PS2	BD2[11:0]											
64	BA1[15:0]															
80	BA1[31:16]															
96	BA2[15:0]															
112	BA2[31:16]															

Interrupt Interface Block

The interrupt interface raises an interrupt when specific changes to HBI channel descriptors occur, including:

- For asynchronous and control read/write channels:
 - a packet is available to read in the channel buffer, or
 - sufficient empty space is available in the channel buffer to accept a requested packet write
- For isochronous read/write channels:
 - the number of valid bytes in the channel buffer exceeds the block size, or
 - the number of empty bytes in the channel buffer exceeds the block size

Operating Modes

The following sections describe the operating modes of the MLB interface. The channel type selection enables the logical channels to operate in synchronous, asynchronous, isochronous, or control channels.

NOTE: The logical channels can be any combination of channel type (for example synchronous, asynchronous, or control) and direction (transmit or receive).

Isochronous Data Exchange

An isochronous HBI channel is initially opened and synchronized with `HCMD0.CMD[2:0] = 010`. For isochronous channels, no further synchronization is required from the HBI perspective; however, an optional resynchronization command is available for HC flexibility. Setting `HCMD0.CMD[2:0] = 011` reinitializes the address pointer within the data buffer, ensuring that subsequent data exchange with the channel is aligned at an isochronous packet boundary. When the HC must close an isochronous channel before it has read or written an entire data packet, setting `HCMD0.CMD[2:0] = 000` reopens the channel without synchronizing the address pointer in the buffer. This action allows reading and writing to continue where the HC previously stopped.

For isochronous data transmission, the *Exchanging Isochronous Data on an HBI Channel* figure shows the flow control.

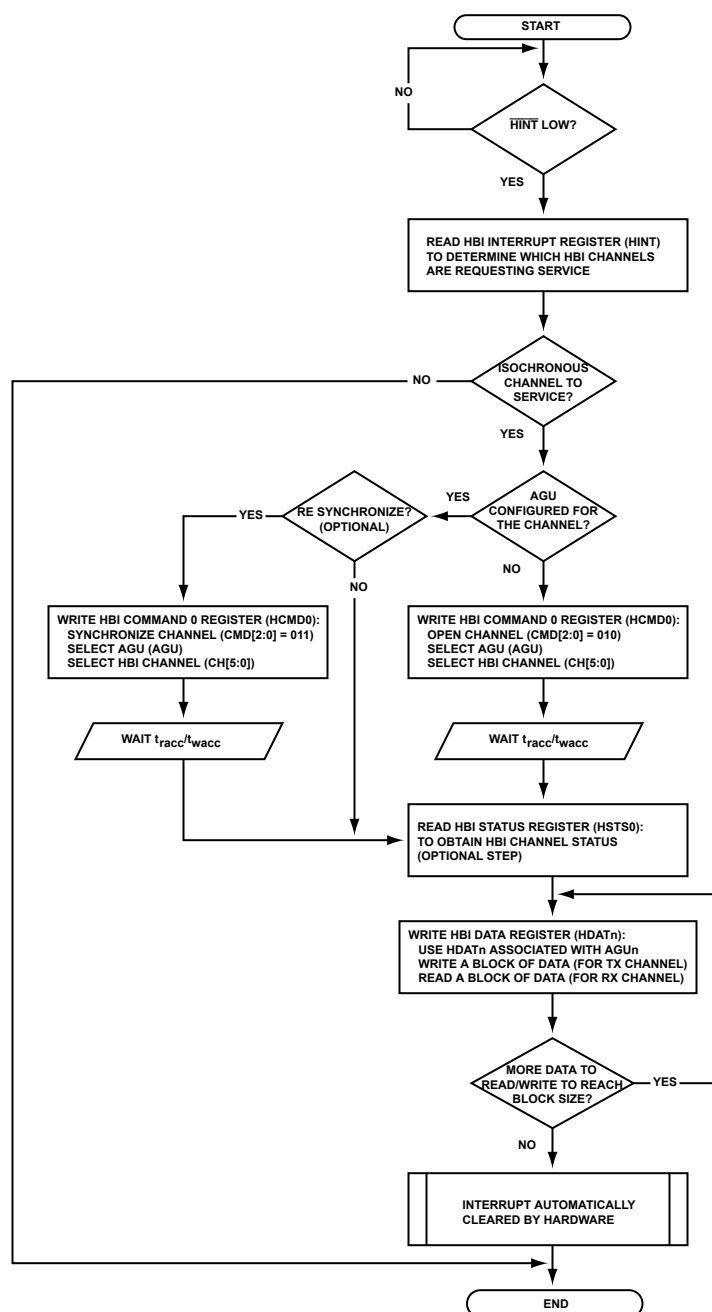


Figure 22-5: Exchanging Isochronous Data on an HBI Channel

Asynchronous and Control Data Exchange

An asynchronous or control HBI channel is initially opened using `HCMD0.CMD[2:0] = 010`. Occasionally, the HC may need to close a packet channel before it has completed writing or reading the current packet of data (for example, the HC needs to use this AGU to service another HBI channel). In this case, the HC can reopen the previous channel with `HCMD0.CMD[2:0] = 000`. This situation allows the HC to continue writing or reading a packet from

the point it left off. In this situation, the PML is already known and the packet length is not reread or rewritten by the HC.

During the reading of a packet, if the HC sets HCMD0.CMD[2:0]= 010 for the channel before the last byte of the packet is read, an internal hardware pointer is reset to the beginning of the packet buffer. This situation requires that the HC reread the PML (from HSTS1) and reread the packet data (from HDATn) from the beginning. In the same manner, if the HC resets HCMD0.CMD[2:0]= 010 for the channel before the last byte of a packet is written, an internal hardware pointer is reset to the beginning of the packet buffer. This situation requires that the HC rewrite the PML (to HCMD1) and rewrite the packet data (to HDATn) from the beginning. Any previous packet data in the buffer is overwritten.

Frame synchronization is not supported for asynchronous channels.

Synchronous Data Exchange

The MLB core provides two modes of operation; standard and multi-frame per sub-buffer which provide flexibility for implementing synchronous channels. Channels configured for standard mode require less buffer space, but have higher interrupt rates and more stringent latency requirements. Channels setup for multi-frame per sub-buffer mode require more buffer space, but have lower interrupt rates and less stringent latency requirements.

To set up a channel in multi-frame per sub-buffer mode:

1. Program the MLB_CTL0 . FCNT bit field to select the number of frames per sub-buffer.
2. Program the CAT to enable multi-frame sub-buffering (MFE= 1) for each particular channel.
3. Set the buffer depth in the CDT: $BD = 4 \times m \times bpf + 1$ where: m = frames per sub-buffer, bpf = bytes per frame.
4. Repeat for additional synchronous channels

A sample synchronous data buffer is shown in the *Synchronous Data Buffer Structure* figure. Each data buffer contains four sub-buffers and each sub-buffer contains space for 1 to 64 frames of data, determined by the MLB_CTL0 . FCNT bits.

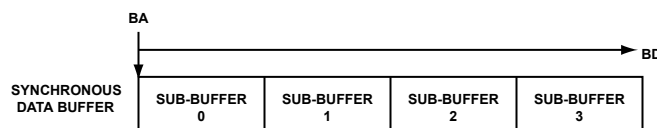


Figure 22-6: Synchronous Data Buffer Structure

Data Transfer

Two modes of operation are supported for transferring channel data between the MLB and internal memory. DMA allows the multi-channel DMA engine to manage data transfers without core intervention. Core driven mode (I/O mode) allows software to manage the transfer of data between MLB and internal memory.

NOTE: All hardware channels must use the same data transfer method. Mixed mode operation where hardware channels operate in both I/O mode and DMA mode is not supported.

DMA

The processor supports DMA mode which uses only INCR8, INCR4 and SINGLE beat bursts. Program the `MLB_ACTL.DMAMODE = 1` to use DMA mode.

Programming Model

This section provides general guidelines for programming the MediaLB interface.

Channel Initialization

The software flow required to initialize a channel must be performed in order to ensure proper operation.

Configure the Hardware

1. Initialize the CTR and registers.
 - a. Set all the CTR (CAT, CDT, and ADT) bits to 0.
 - b. Set all bits of all registers to 0.
2. Configure the MediaLB interface.
 - a. Select 3-pin or 6-pin MediaLB operation: `MLB_CTL0.PEN = 0` (3-pin), `MLB_CTL0.PEN = 1` (6-pin).
 - b. Select MediaLB clock speed via `MLB_CTL0.CLK`.
 - c. Set MediaLB enable via `MLB_CTL0.EN`.
3. Configure the HBI interface.
 - a. Set `MLB_HCMR0` and `MLB_HCMR1 = 0xFFFFFFFF` to activate all channels.
 - b. Set the HBI enable bit: `MLB_HCTL.EN = 1`.

Program the CAT and the CDT

1. Initialize all bits of the CAT to 0.
2. Select a logical channel: $N = 1 - 63$.
3. Program the CDT for channel N.
 - a. Set the 14-bit base address (BA)
 - b. Set the 12-bit or 13-bit buffer depth (BD): BD = buffer depth in bytes 1
 - For synchronous channels: $(BD + 1) = 4 \text{ frames per sub-buffer (m) bytes-per-frame (bpf)}$
 - For isochronous channels: $(BD + 1) \bmod (BS + 1) = 0$

- For asynchronous channels: $(BD + 1)$ max packet length (1024 for a MOST Data Packet (MDP))
- 1536 for a MOST Ethernet Packet (MEP)
- For control channels: $(BD + 1)$ max packet length (64)

4. Program the CAT for the inbound DMA.

- For TX channels (to MediaLB) HBI is the inbound DMA
- For RX channels (from MediaLB) MediaLB is the inbound DMA
- Set the channel direction: $RNW = 0$
- Set the channel type: $CT[2:0] = 010$ (asynchronous), 001 (control), 011 (isochronous), or 000 (synchronous)
- Set the connection label: $CL[5:0] = N$
- If $CT[2:0] = 000$ (synchronous), set the mute bit ($MT = 1$)
- Set the channel enable: $CE = 1$
- Set all other bits of the CAT to 0

5. Program the CAT for the outbound DMA.

- For TX channels (to MediaLB) HBI is the outbound DMA
- For RX channels (from MediaLB) MediaLB is the outbound DMA
- Set the channel direction: $RNW = 1$
- Set the channel type: $CT[2:0] = 010$ (asynchronous), 001 (control), 011 (isochronous), or 000 (synchronous)
- Set the channel label: $CL[5:0] = N$
- If $CT[2:0] = 000$ (synchronous), set the mute bit ($MT = 1$)
- Set the channel enable: $CE = 1$
- Set all other bits of the CAT to 0

6. Repeat steps 2 through 5 to initialize all logical channels.

Program the ADT

- Initialize all bits of the ADT to 0
- Select a logical channel: $N = 1 - 63$
- Program the AMBA AHB block ping page for channel N
 - Set the 32-bit base address (BA1)

- b. Set the 11-bit buffer depth (BD1): $BD1 = \text{buffer depth in bytes} - 1$
 - For synchronous channels: $(BD1 + 1) = n \text{ frames per sub-buffer (m) bytes-per-frame (bpf)}$
 - For isochronous channels: $(BD1 + 1) \bmod (BS + 1) = 0$
 - For asynchronous channels: $5 (BD1 + 1) \leq 4096 \text{ (max packet length)}$
 - For control channels: $5 (BD1 + 1) \leq 4096 \text{ (max packet length)}$
 - c. For asynchronous and control Tx channels set the packet start bit (PS1) iff the page contains the start of the packet
 - d. Clear the page done bit (DNE1)
 - e. Clear the error bit (ERR1)
 - f. Set the page ready bit (RDY1)
4. Program the AMBA AHB block pong page for channel N
 - a. Set the 32-bit base address (BA2)
 - b. Set the 11-bit buffer depth (BD2): $BD2 = \text{buffer depth in bytes} - 1$
 - For synchronous channels: $(BD2 + 1) = n \text{ frames per sub-buffer (m) bytes-per-frame (bpf)}$
 - For isochronous channels: $(BD2 + 1) \bmod (BS + 1) = 0$
 - For asynchronous channels: $5 (BD2 + 1) \leq 4096 \text{ (max packet length)}$
 - For control channels: $5 (BD2 + 1) \leq 4096 \text{ (max packet length)}$
 - c. For asynchronous and control TX channels set the packet start bit (PS2) iff the page contains the start of the packet
 - d. Clear the page done bit (DNE2)
 - e. Clear the error bit (ERR2)
 - f. Set the page ready bit (RDY2)
 5. Select Big Endian (LE = 0) or Little Endian (LE = 1)
 6. Select the active page: PG = 0 (ping), PG = 1 (pong)
 7. Set the channel enable (CE) bit for all active logical channels
 8. Repeat steps 2 through 7 for all active logical channels.

Service

After initialization, each channel will require periodic servicing. Use the procedures in the following sections to service DMA and MLB interrupts and to poll for MLB system commands.

Servicing the DMA Channel Interrupts

1. Program the [MLB_ACMR0/MLB_ACMR1](#) registers to enable interrupts from all active DMA channels.
2. Select the status clear method. `MLB_ACTL.SCE = 0` (hardware clears on read), `MLB_ACTL.SCE = 1` (software writes a 1 to clear).
3. Select 1 or 2 interrupt signals. Configure the `MLB_ACTL.SMX` bit = 0 (one interrupt for channels 0 through 31 and another interrupt for channels 32 through 63 on). Configure the `MLB_ACTL.SMX` bit = 1 (single interrupt for all channels).
4. Wait for an interrupt.
5. Read the [MLB_ACSR0/MLB_ACSR1](#) registers to determine which channel or channels are causing the interrupt.
6. If the `MLB_ACTL.SCE` bit = 1, write the results of step 5 back to the [MLB_ACSR0](#) and [MLB_ACSR1](#) registers to clear the interrupt.
7. Select a logical channel ($N = 0 - 63$) with an interrupt to service.
8. Read the ADT entry for channel N to:
 - a. Determine the active page (ping or pong) via the PG bit
 - b. Determine which page(s) are done via the DNEN bits
 - c. Determine which channels encountered an AHB error via the ERRN bit
 - d. Determine which asynchronous and control Rx channel pages contain a packet start via the PSN bit (extract the PML)
9. Repeat steps 6 through 8 for all channels with pending interrupts
10. Repeat steps 4 through 9 while there are active channels.

Servicing the MediaLB Status Interrupts

1. Select the MediaLB channel status register ([MLB_MS0/MLB_MS1](#)) to be cleared by software, writing a 0 to the appropriate bits.
2. Program the [MLB_MIEN](#) register to enable protocol error interrupts for all active MediaLB channels. (`MLB_MIEN.CTXPE = 1`, `MLB_MIEN.CRXPE = 1`, `MLB_MIEN.ATXPE = 1`, `MLB_MIEN.ARXPE = 1`, `MLB_MIEN.SYNCPE = 1`, and `MLB_MIEN.ISOCPE = 1`).
3. Wait for an interrupt on the [MLB_INT0/MLB_INT1](#) signal.
4. Read the [MLB_MS0/MLB_MS1](#) registers to determine which channel(s) are causing the interrupt.
5. Read the RSTS/WSTS of the appropriate CDT(s) to determine the interrupt type.

6. Clear the RSTS/WSTS errors to ensure that the current status of channel operations is reflected in the register:

Option	Description
For synchronous RX channels	WSTS[3] = 0
For synchronous TX channels	RSTS[3] = 0
For isochronous RX channels	WSTS[2:1] = 00
For isochronous TX channels	RSTS[2:1] = 00
For asynchronous and control RX channels	WSTS[4] = 0 and WSTS[2] = 0
For asynchronous and control TX channels	RSTS[4] = 0 and RSTS[2] = 0

Polling for MediaLB System Commands

The MediaLB System status ([MLB_MSS](#)) register is used to detect a system command received from the MediaLB controller. The processor's peripheral automatically sends the appropriate system response to the MediaLB controller. The procedure for the application is:

1. The application periodically polls the [MLB_MSS](#) register.
2. Clear by writing a 0 to the appropriate bit in the [MLB_MSS](#) register after the application finishes the service.
3. If `MLB_MSS.SWSYSCMD = 1`, read the [MLB_MSD](#) register to receive the system data sent from MediaLB controller.

ADSP-2159x MLB Register Descriptions

MediaLB Device Interface Macro 2 (MLB) contains the following registers.

Table 22-19: ADSP-2159x MLB Register List

Name	Description
MLB_ACMR0	Peripheral Channel Mask 0 Register
MLB_ACMR1	Peripheral Channel Mask 1 Register
MLB_ACSR0	Peripheral Channel Status 0 Register
MLB_ACSR1	Peripheral Channel Status 1 Register
MLB_ACTL	Bus Control Register
MLB_CTL0	MediaLB Control 0 Register
MLB_CTL1	Control 1 Register
MLB_GCTL	MLB Global Control Register
MLB_HCBR0	HBI Channel Busy 0 Register
MLB_HCBR1	HBI Channel Busy 1 Register

Table 22-19: ADSP-2159x MLB Register List (Continued)

Name	Description
MLB_HCER0	HBI Channel Error 0 Register
MLB_HCER1	HBI Channel Error 1 Register
MLB_HCMR0	HBI Channel Mask 0 Register
MLB_HCMR1	HBI Channel Mask 1 Register
MLB_HCTL	HBI Control Register
MLB_MADR	Memory Interface Address Register
MLB_MCTL	Memory Interface Control Register
MLB_MDAT0	Memory Interface Control Data 0 Register
MLB_MDAT1	Memory Interface Control Data 1 Register
MLB_MDAT2	Memory Interface Control Data 2 Register
MLB_MDAT3	Memory Interface Control Data 3 Register
MLB_MDWE0	Memory Interface Control Data Write Enable 0 Register
MLB_MDWE1	Memory Interface Control Data Write Enable 1 Register
MLB_MDWE2	Memory Interface Control Data Write Enable 2 Register
MLB_MDWE3	Memory Interface Control Data Write Enable 3 Register
MLB_MIEN	Interrupt Enable Register
MLB_MS0	Channel Status 0 Register
MLB_MS1	Channel Status 1 Register
MLB_MSD	System Data Register
MLB_MSS	System Status Register
MLB_PCTL0	MediaLB 6-pin Control 0 Register

Peripheral Channel Mask 0 Register

The `MLB_ACMR0` register allows control over which channel(s) generate interrupts on `MLB_INT[1:0]`. All of the bits in this register default to 0 (masked) so the HC must initially write to the `MLB_ACMR0` register to enable interrupts. Each bit of this register is read/write accessible.

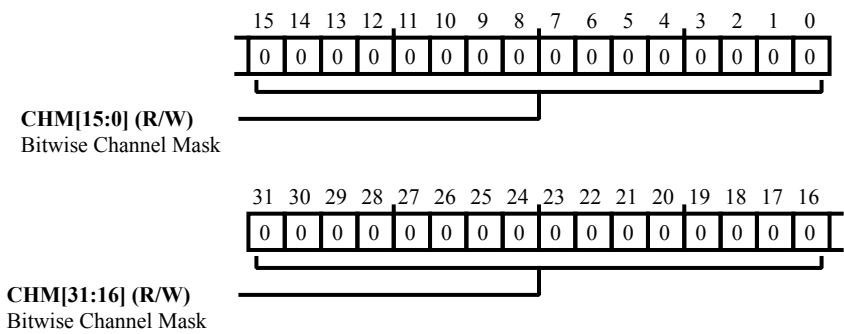


Figure 22-7: MLB_ACMR0 Register Diagram

Table 22-20: MLB_ACMR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	CHM	Bitwise Channel Mask. The <code>MLB_ACMR0.CHM</code> bit field masks or unmasks channels 31-0.
		0 Mask interrupt for Channel 31:0 (bitwise; all channels shown)
		4294967295 Unmask interrupt for Channel 31:0 (bitwise; all channels shown)

Peripheral Channel Mask 1 Register

The [MLB_ACMR1](#) register allows control over which channel(s) generate interrupts on MLB_INT[1:0]. All of the bits in this register default to 0 (masked) so the HC must initially write to the [MLB_ACMR1](#) register to enable interrupts. Each bit of this register is read/write accessible.

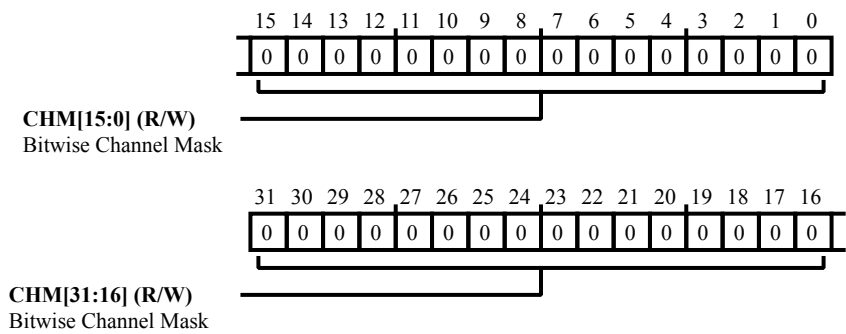


Figure 22-8: MLB_ACMR1 Register Diagram

Table 22-21: MLB_ACMR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	CHM	Bitwise Channel Mask. The <code>MLB_ACMR1.CHM</code> bit field masks or unmasks channels 63:32.
		0 Mask interrupt for Channel 63:32 (bitwise; all channels shown)
		4294967295 Unmask interrupt for Channel 63:32 (bitwise; all channels shown)

Peripheral Channel Status 0 Register

The **MLB_ACSR0** register contains interrupt bits for each of the 64 physical channels. When a bit in this register is set, it indicates that the corresponding physical channel has an interrupt pending.

A peripheral interrupt is triggered when either **DNE_n** or **ERR_n** is set within the Bus Channel Descriptor. The HC is notified of the channel interrupt via **MLB_INT[1:0]**. When an interrupt occurs in **ACCUSER** (for channels 31 to 0) **MLB_INT[0]** is set. When an interrupt occurs in **MLB_ACSR1** (for channels 63 to 32) **MLB_INT[1]** is set.

Interrupts in the **MLB_ACSR0** and **MLB_ACSR1** registers can be optionally multiplexed onto a single interrupt signal, **MLB_INT[0]**, if the **MLB_ACTL.SMX** bit = 1.

If the **MLB_ACTL.SCE** bit = 0, hardware automatically clears the interrupt bit(s) after the HC reads the peripheral channel status registers. Alternatively, if the **MLB_ACTL.SCE** bit = 1, software must write a 1 to the appropriate bit(s) of the peripheral channel status registers to clear the interrupt(s).

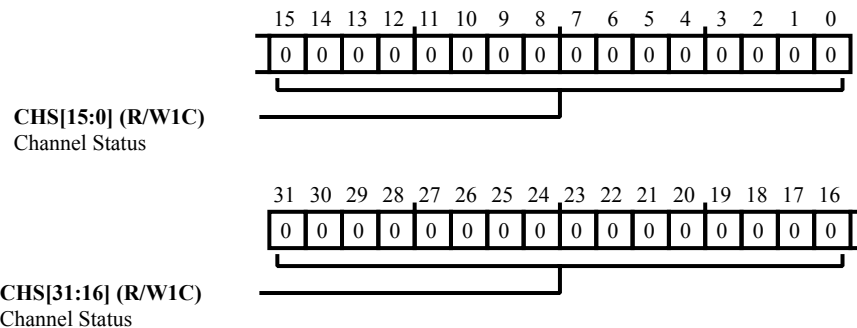


Figure 22-9: MLB_ACSR0 Register Diagram

Table 22-22: MLB_ACSR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W1C)	CHS	Channel Status. The MLB_ACSR0.CHS bit field indicates channel status for channels 31-0.
		0 No interrupt on Channel 31:0 (bitwise; all channels shown)
		4294967295 Interrupt on Channel 31:0 (bitwise; all channels shown)

Peripheral Channel Status 1 Register

The `MLB_ACSR1` register contains interrupt bits for each of the 64 physical channels. When a bit in this register is set, it indicates that the corresponding physical channel has an interrupt pending.

A peripheral interrupt is triggered when either `DNEn` or `ERRn` is set within the Bus Channel Descriptor. The HC is notified of the channel interrupt via `MLB_INT[1:0]`. When an interrupt occurs in `ACCUSER` (for channels 31 to 0) `MLB_INT[0]` is set. When an interrupt occurs in `MLB_ACSR1` (for channels 63 to 32) `MLB_INT[1]` is set.

Interrupts in the `MLB_ACSR1` and `MLB_ACSR0` registers can be optionally multiplexed onto a single interrupt signal, `MLB_INT[0]`, if the `MLB_ACTL.SMX` bit = 1.

If the `MLB_ACTL.SCE` bit = 0, hardware automatically clears the interrupt bit(s) after the HC reads the peripheral channel status registers. Alternatively, if the `MLB_ACTL.SCE` bit = 1, software must write a 1 to the appropriate bit(s) of peripheral channel status registers to clear the interrupt(s).

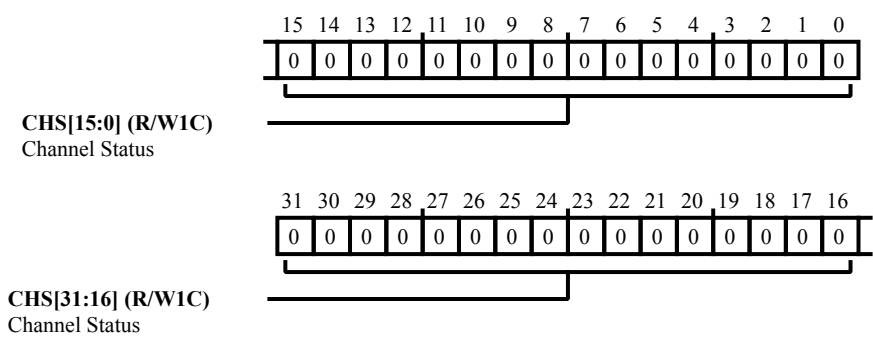


Figure 22-10: MLB_ACSR1 Register Diagram

Table 22-23: MLB_ACSR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W1C)	CHS	Channel Status. The <code>MLB_ACSR1.CHS</code> bit field indicates channel status for channels 63-32.
		0 No interrupt on Channel 63:32 (bitwise; all channels shown)
		4294967295 Interrupt on Channel 63:32 (bitwise; all channels shown)

Bus Control Register

The `MLB_ACTL` register is written by the HC to configure the block for channel interrupts. This register contains bits to select the packet buffering mode and the DMA mode. This register also contains bits that are used to multiplex channel interrupts onto a single interrupt signal and to select the method of clearing channel interrupts.

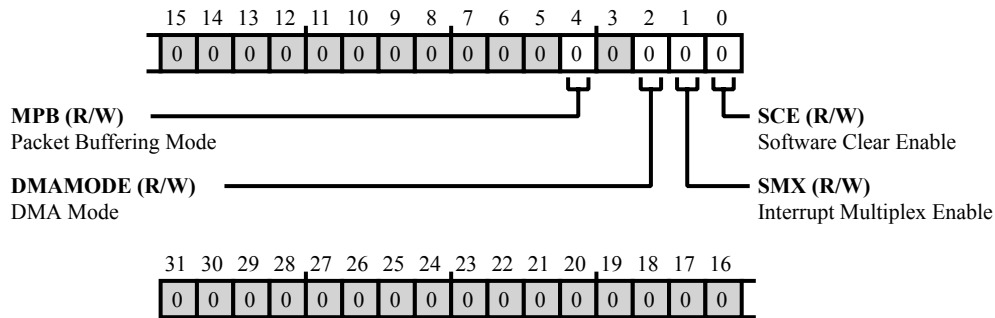


Figure 22-11: MLB_ACTL Register Diagram

Table 22-24: MLB_ACTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	MPB	Packet Buffering Mode. The <code>MLB_ACTL.MPB</code> bit selects whether the buffering mode is single-packet or multiple-packet.
		0 single-packet mode
		1 multiple-packet mode
2 (R/W)	DMAMODE	DMA Mode. The <code>MLB_ACTL.DMAMODE</code> bit selects between DMA mode 1 and 0. DMA Mode 0 uses incrementing bursts of an unspecified length. This allows the block to perform single beat transfers as well as an incrementing (INCR) burst of unspecified length up to the maximum specified burst length (8 beats). DMA Mode 1 uses only INCR8, INCR4, and SINGLE beat bursts. The <code>hburst[2:0]</code> signal encodes the burst transfer used for a DMA operation.
		0 DMA Mode 0
		1 DMA Mode 1

Table 22-24: MLB_ACTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	SMX	Interrupt Multiplex Enable. The MLB_ACTL.SMX bit selects whether ACSR0 generates an interrupt on MLB_INT[0] and ACSR1 generates an interrupt on MLB_INT[1] or ACSR0 and ACSR1 generate an interrupt on MLB_INT[0] only.
		0 ACSR0 generates an interrupt on MLB_INT[0]; ACSR1 generates an interrupt on MLB_INT[1]
		1 ACSR0 and ACSR1 generate an interrupts on MLB_INT[0] only
0 (R/W)	SCE	Software Clear Enable. The MLB_ACTL.SCE bit selects whether hardware clears the interrupt after a ACSRn register read or software clears the interrupt.
		0 Hardware clears interrupt after a ACSRn register read
		1 Software clears interrupt

MediaLB Control 0 Register

The `MLB_CTL0` register contains bit that enable the module and provide module status. Note that the maximum speed for ML6-pin mode is 2048 x FS.

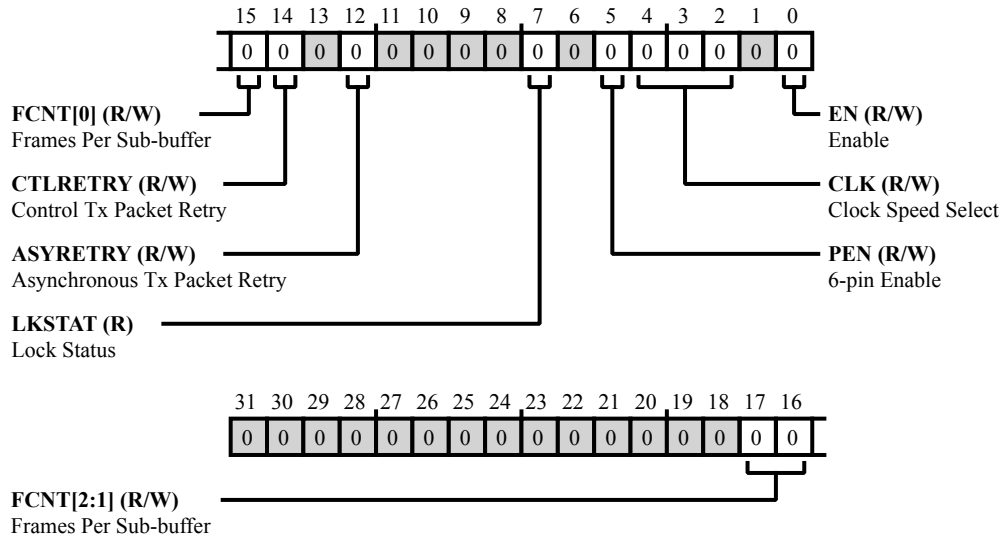


Figure 22-12: MLB_CTL0 Register Diagram

Table 22-25: MLB_CTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17:15 (R/W)	FCNT	Frames Per Sub-buffer. The <code>MLB_CTL0 . FCNT</code> bit field configures the frames per sub-buffer on synchronous channels.
		0 1 Frame per sub-buffer (Operation is the same as Standard mode)
		1 2 frames per sub-buffer
		2 4 frames per sub-buffer
		3 8 frames per sub-buffer
		4 16 frames per sub-buffer
		5 32 frames per sub-buffer
		6 64 frames per sub-buffer
14 (R/W)	CTLRETRY	Control Tx Packet Retry. When the <code>MLB_CTL0 . CTLRETRY</code> bit is set, a control packet that is flagged with a Break or Protocol error by the receiver is retransmitted. When cleared, a control packet that is flagged with a Break or Protocol error by the receiver is skipped.

Table 22-25: MLB_CTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	ASYRETRY	Asynchronous Tx Packet Retry. When the MLB_CTL0.ASYRETRY bit is set, an asynchronous packet that is flagged with a Break or Protocol error by the receiver is retransmitted. When cleared, an asynchronous packet that is flagged with a Break or ProtocolError by the receiver is skipped.
7 (R/NW)	LKSTAT	Lock Status. When the MLB_CTL0.LKSTAT bit is set (=0), the MediaLB block is synchronized to the incoming MediaLB frame with the following conditions. - If MLB_CTL1.LOCK =0 (unlocked), MLB_CTL1.LOCK is set after a FRAME-SYNC is detected at the same position for three consecutive frames. - If MLB_CTL1.LOCK =1 (locked), MLB_CTL1.LOCK is cleared after not receiving a FRAMESYNC at the expected time for two consecutive frames. In this case FRAMESYNC patterns occurring at locations other than the expected one are ignored.
5 (R/W)	PEN	6-pin Enable. The MLB_CTL0.PEN bit configures the MLB for 6-pin or 3-pin mode.
		0 3-pin interface enabled
		1 6-pin interface enabled
4:2 (R/W)	CLK	Clock Speed Select. The MLB_CTL0.CLK bit field sets the clock speed.
		0 256 x Fs (MLB_CTL.PEN = 0)
		1 512 x Fs (MLB_CTL.PEN = 0)
		2 1024 x Fs (MLB_CTL.PEN = 0)
		3 2048 x Fs (MLB_CTL.PEN = 1)
		4 Reserved
		5 Reserved
		6 Reserved
		7 Reserved
0 (R/W)	EN	Enable. When the MLB_CTL0.EN bit is set (=1), MediaLB clock, signal, and data are received and transmitted on the appropriate MediaLB pins.

Control 1 Register

The `MLB_CTL1` register contains bits that provide lock status and control system commands.

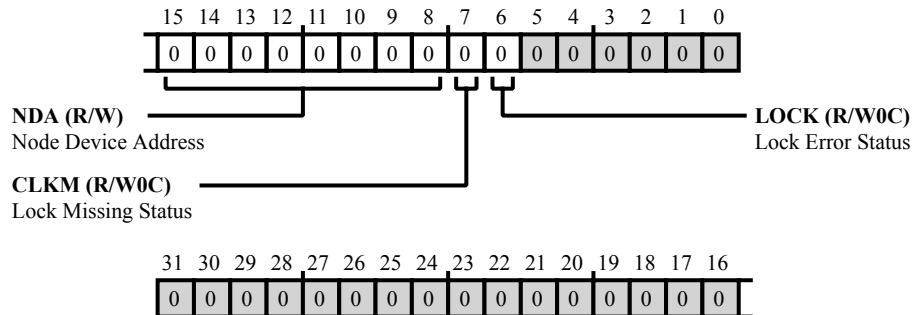


Figure 22-13: MLB_CTL1 Register Diagram

Table 22-26: MLB_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	NDA	Node Device Address. The <code>MLB_CTL1 . NDA</code> bit field is used for system commands directed to individual MediaLB nodes.
7 (R/W0C)	CLKM	Lock Missing Status. The <code>MLB_CTL1 . CLKM</code> bit is set when the MediaLB clock is not toggling at the pin. This bit is cleared by software.
6 (R/W0C)	LOCK	Lock Error Status. The <code>MLB_CTL1 . LOCK</code> bit is set when the MediaLB is unlocked. This bit is cleared by software.

MLB Global Control Register

The `MLB_GCTL` register contains bits that manage the MLB clock.

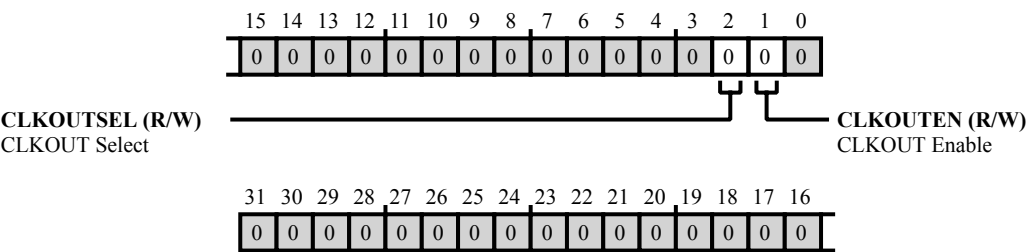


Figure 22-14: MLB_GCTL Register Diagram

Table 22-27: MLB_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	CLKOUTSEL	CLKOUT Select. The <code>MLB_GCTL.CLKOUTSEL</code> bit selects either the MLB 3 pin clock or the 6 pin clock as <code>MLBCLKOUT</code> .
		0 MLB 3 pin clock is selected for <code>MLBCLKOUT</code>
		1 MLB 6 pin clock is selected for <code>MLBCLKOUT</code>
1 (R/W)	CLKOUTEN	CLKOUT Enable.

HBI Channel Busy 0 Register

The HC can determine which channel(s) are busy by reading the `MLB_HCBR0` register. An HBI channel is busy if:

- it is currently loaded into one of the two AGUs
- the channel is enabled, `CE = 1` from the Channel Allocation Table
- the DMA is active

When an HBI channel is busy, hardware may write back its local copy of the channel descriptor at any time. System software should not write a CDT descriptor for a channel that is busy. Only two HBI channels can be busy at any given time. Each bit of this register is read-only.

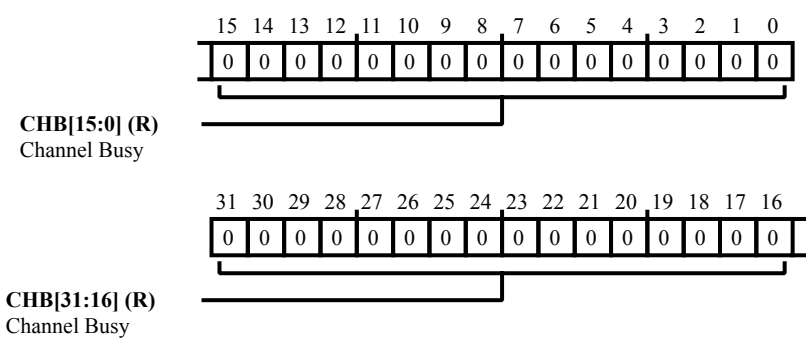


Figure 22-15: `MLB_HCBR0` Register Diagram

Table 22-28: `MLB_HCBR0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CHB	Channel Busy. The <code>MLB_HCBR0.CHB</code> bit field contains the bitwise channel busy bit for channels 31:0. When a bit is cleared (=0) the channel is idle. When a bit is set (=1) the channel is busy.

HBI Channel Busy 1 Register

The HC can determine which channel(s) are busy by reading the [MLB_HCBR1](#) register. An HBI channel is busy if:

- it is currently loaded into one of the two AGUs
- the channel is enabled, CE = 1 from the Channel Allocation Table
- the DMA is active

When an HBI channel is busy, hardware may write back its local copy of the channel descriptor at any time. System software should not write a CDT descriptor for a channel that is busy. Only two HBI channels can be busy at any given time. Each bit of this register is read-only.

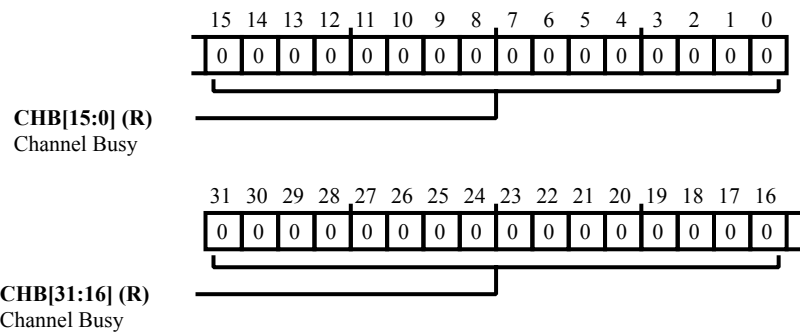


Figure 22-16: MLB_HCBR1 Register Diagram

Table 22-29: MLB_HCBR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CHB	Channel Busy. The MLB_HCBR1 . CHB bit field contains the bitwise channel busy bit for channels 63:32. When a bit is cleared (=0) the channel is idle. When a bit is set (=1) the channel is busy.

HBI Channel Error 0 Register

The `MLB_HCER0` register indicates which channels (channels 31:0) have encountered fatal errors.

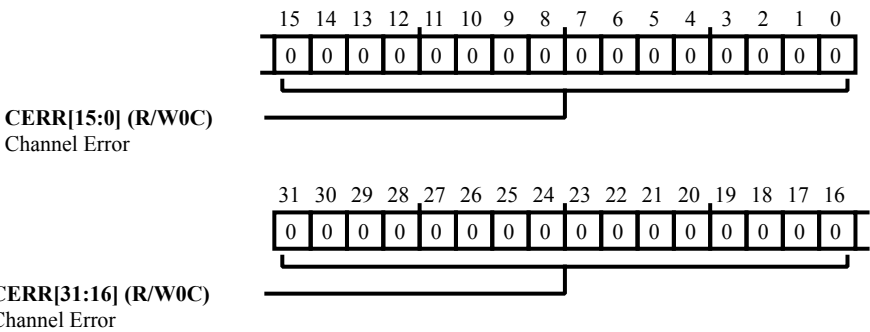


Figure 22-17: MLB_HCER0 Register Diagram

Table 22-30: MLB_HCER0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W0C)	CERR	Channel Error. The <code>MLB_HCER0.CERR</code> bit field reports bitwise errors for channels 31:0.

HBI Channel Error 1 Register

The `MLB_HCER1` register indicates which channel(s) have encountered fatal errors.

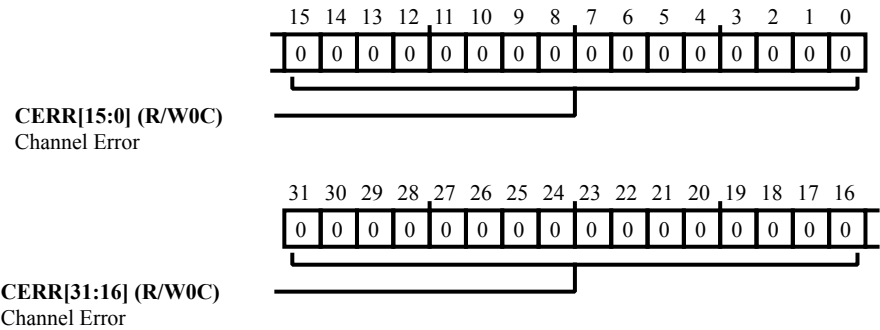


Figure 22-18: `MLB_HCER1` Register Diagram

Table 22-31: `MLB_HCER1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W0C)	CERR	Channel Error. The <code>MLB_HCER1</code> . CERR bite field reports bitwise errors for channels 63:32.

HBI Channel Mask 0 Register

The `MLB_HCMR0` register controls which channels (for channels 31:0) are able to generate an HBI interrupt.

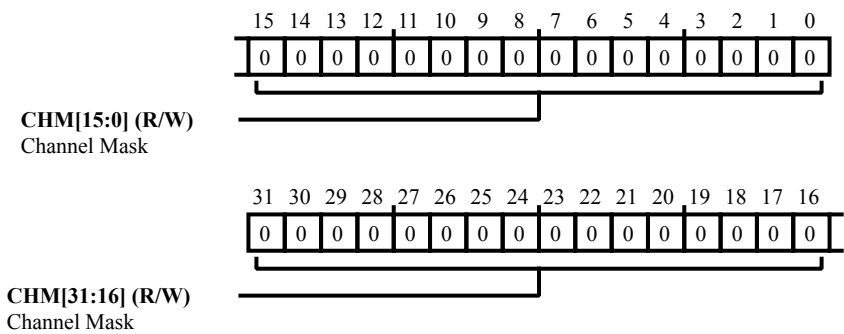


Figure 22-19: `MLB_HCMR0` Register Diagram

Table 22-32: `MLB_HCMR0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	CHM	Channel Mask. The <code>MLB_HCMR0.CHM</code> bit field contains the bitwise channel mask bit for channels 31:0. When a bit is cleared (=0) the channel is masked. When set (=1) the channel is unmasked.

HBI Channel Mask 1 Register

The `MLB_HCMR1` register controls which channels (for channels 63:32) are able to generate an HBI interrupt.

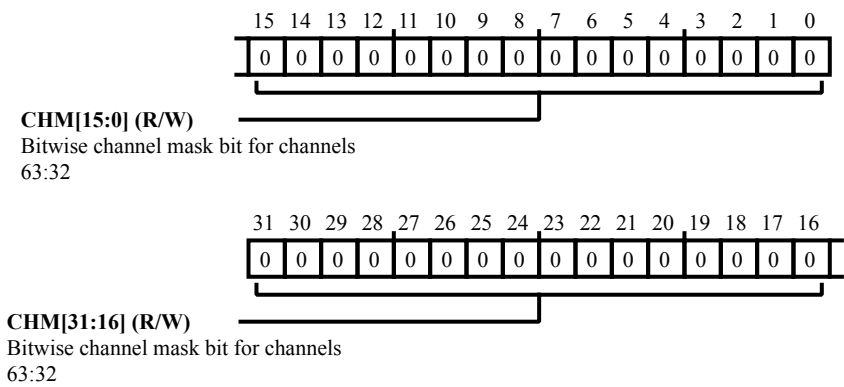


Figure 22-20: `MLB_HCMR1` Register Diagram

Table 22-33: `MLB_HCMR1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	CHM	Bitwise channel mask bit for channels 63:32. The <code>MLB_HCMR1.CHM</code> bit field contains the bitwise channel mask bit for channels 63:32. When a bit is cleared (=0) the channel is masked. When set (=1) the channel is unmasked.

HBI Control Register

The `MLB_HCTL` register controls and monitors general operation of the HBI block through the Address Generation Units) by reading and writing the register through the I/O interface. Each bit of this register is read/write.

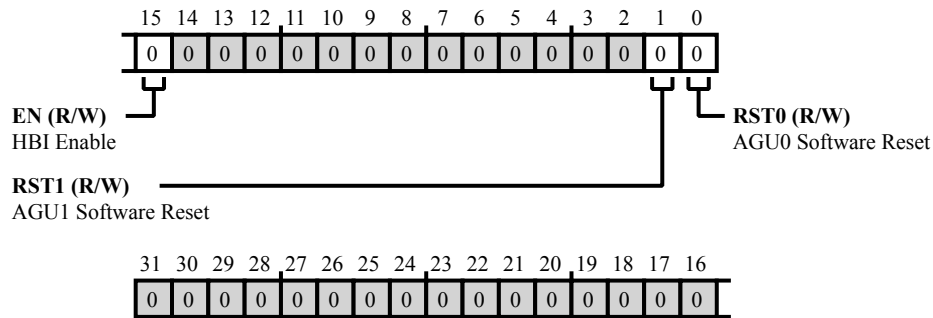


Figure 22-21: MLB_HCTL Register Diagram

Table 22-34: MLB_HCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	EN	HBI Enable. Setting the <code>MLB_HCTL.EN</code> bit enables the HBI.
1 (R/W)	RST1	AGU1 Software Reset. Setting the <code>MLB_HCTL.RST1</code> bit resets AGU1.
0 (R/W)	RST0	AGU0 Software Reset. Setting the <code>MLB_HCTL.RST0</code> bit resets AGU0.

Memory Interface Address Register

The `MLB_MADR` register contains bit fields that contain the Channel Table RAM (CTR) or Data Buffer RAM (DBR) addresses. It also contains bits that set target location and read-not-write parameters.

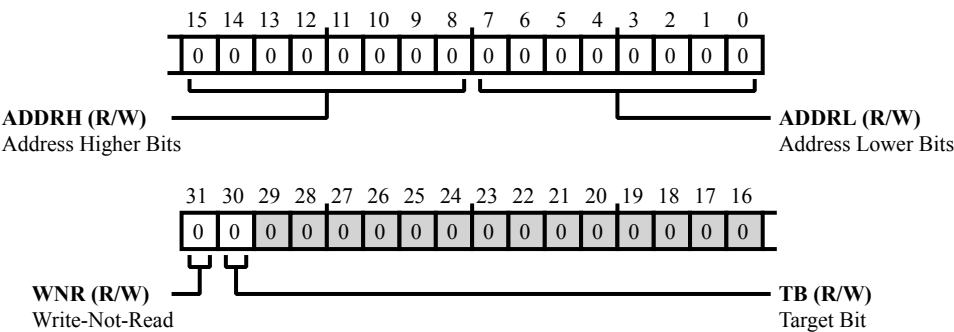


Figure 22-22: `MLB_MADR` Register Diagram

Table 22-35: `MLB_MADR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	WNR	Write-Not-Read. The <code>MLB_MADR.WNR</code> bit selects Write-Not-Read.
		0 Read
		1 Write
30 (R/W)	TB	Target Bit. The <code>MLB_MADR.TB</code> bit sets the target location.
		0 Channel Table RAM (CTR)
		1 Data Buffer RAM (DBR)
15:8 (R/W)	ADDRH	Address Higher Bits. The <code>MLB_MADR.ADDRH</code> bit field contains the DBR address of the 8-bit entry (bits [13:8]).
7:0 (R/W)	ADDRL	Address Lower Bits. The <code>MLB_MADR.ADDRL</code> bit field contains the CTR address of the 128-bit entry or the DBR address of the 8-bit entry (bits [7:0]).

Memory Interface Control Register

The `MLB_MCTL` register contains a bit that indicates that the data transfer is complete.

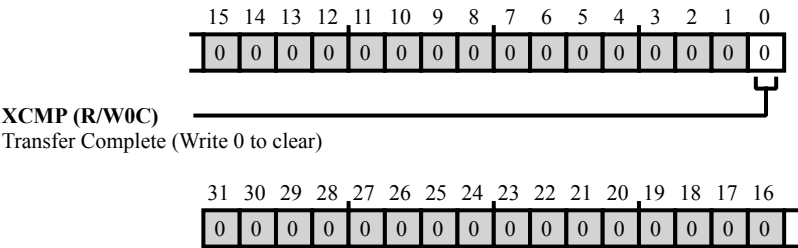


Figure 22-23: `MLB_MCTL` Register Diagram

Table 22-36: `MLB_MCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W0C)	XCMP	Transfer Complete (Write 0 to clear). The <code>MLB_MCTL.XCMP</code> bit indicates that the data transfer is complete.

Memory Interface Control Data 0 Register

The `MLB_MDAT0` register contains Channel Table RAM (CTR) data (bits [31:0] of 128-bit entry) or Data Buffer RAM (DBR) data (bits [7:0] of 8-bit entry).

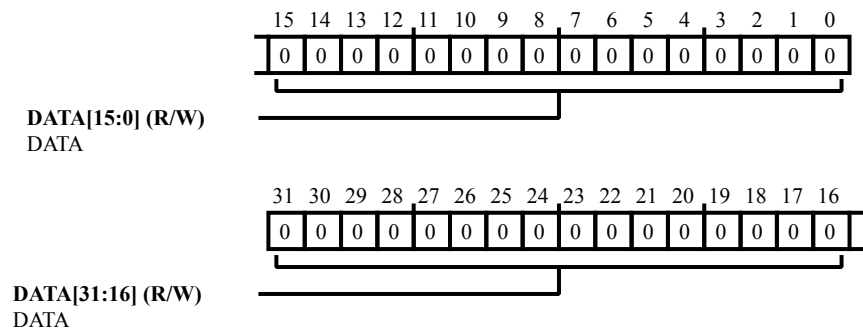


Figure 22-24: MLB_MDAT0 Register Diagram

Table 22-37: MLB_MDAT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	DATA. The <code>MLB_MDAT0</code> . <code>DATA</code> bit field contains CTR data or DBR data.

Memory Interface Control Data 1 Register

The `MLB_MDAT1` register contains Channel Table RAM (CTR) data (bits [63:32] of 128-bit entry).

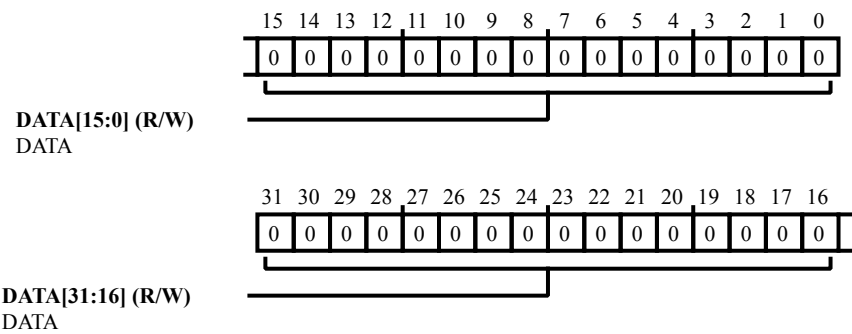


Figure 22-25: `MLB_MDAT1` Register Diagram

Table 22-38: `MLB_MDAT1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	DATA. The <code>MLB_MDAT1</code> . DATA bit field contains CTR data.

Memory Interface Control Data 2 Register

The `MLB_MDAT2` register contains Channel Table RAM (CTR) data (bits [95:64] of 128-bit entry).

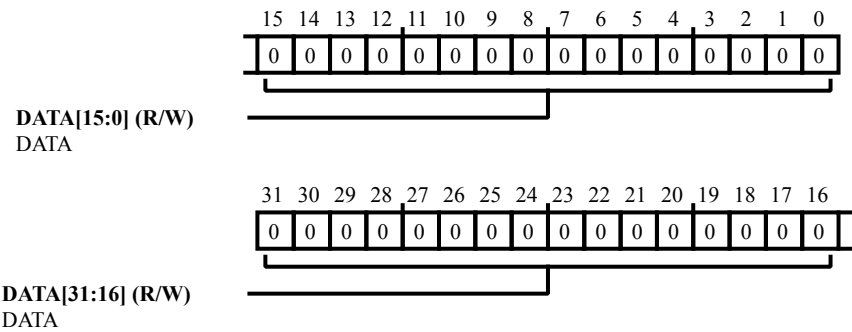


Figure 22-26: MLB_MDAT2 Register Diagram

Table 22-39: MLB_MDAT2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	DATA. The MLB_MDAT2 . DATA bit field contains CTR data.

Memory Interface Control Data 3 Register

The `MLB_MDAT3` register contains Channel Table RAM (CTR) data (bits [127:96] of 128-bit entry).

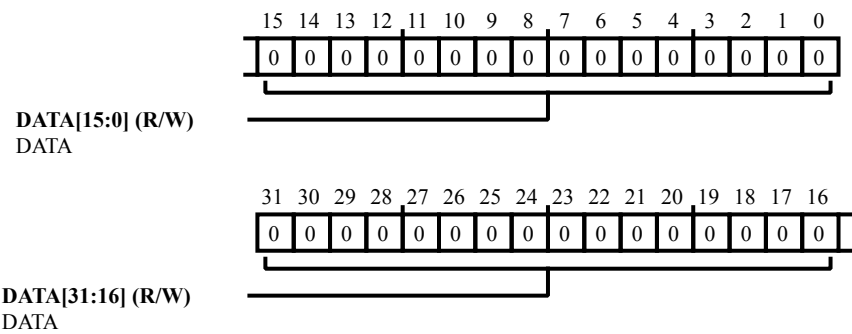


Figure 22-27: `MLB_MDAT3` Register Diagram

Table 22-40: `MLB_MDAT3` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	DATA	DATA. The <code>MLB_MDAT3</code> . DATA bit field contains CTR data.

Memory Interface Control Data Write Enable 0 Register

The `MLB_MDWE0` register contains the bitwise write enable for Channel Table RAM (CTR) data bits [31:0]. When cleared (=0) the bit is disabled, when set (=1) the bit is enabled.

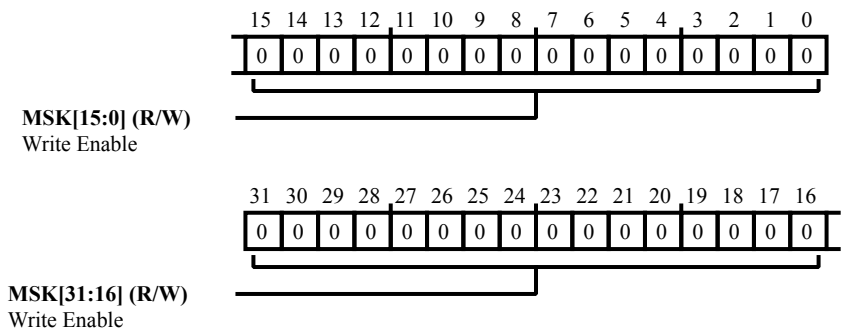


Figure 22-28: MLB_MDWE0 Register Diagram

Table 22-41: MLB_MDWE0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MSK	Write Enable. The <code>MLB_MDWE0</code> .MSK bit field contains the bitwise write enable for CTR data.

Memory Interface Control Data Write Enable 1 Register

The `MLB_MDWE1` register contains the bitwise write enable for Channel Table RAM (CTR) data bits [63:32]. When cleared (=0), the bit is disabled. When set (=1), the bit is enabled.

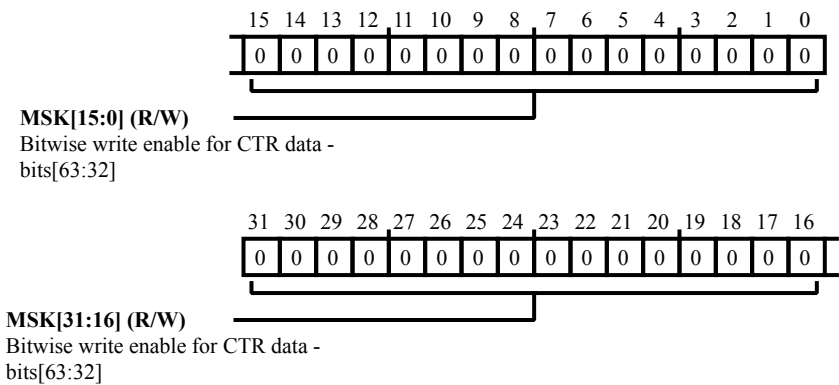


Figure 22-29: MLB_MDWE1 Register Diagram

Table 22-42: MLB_MDWE1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MSK	Bitwise write enable for CTR data - bits[63:32]. The <code>MLB_MDWE1</code> . MSK bit field contains the bitwise write enable for CTR data.

Memory Interface Control Data Write Enable 2 Register

The `MLB_MDWE2` register contains the bitwise write enable for Channel Table RAM (CTR) data bits [95:64]. When cleared (=0), the bit is disabled. When set (=1), the bit is enabled.

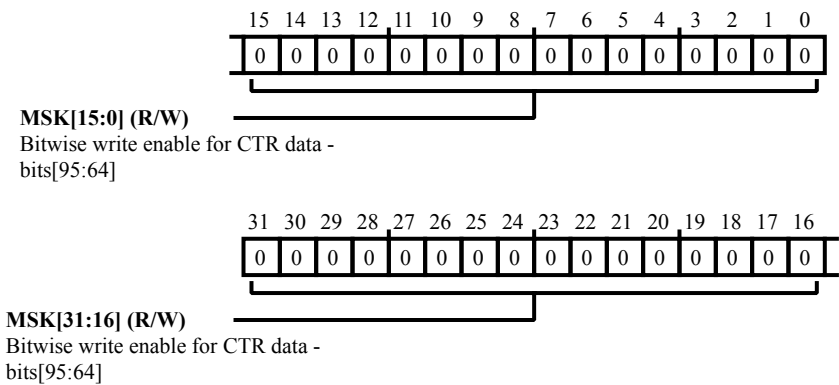


Figure 22-30: MLB_MDWE2 Register Diagram

Table 22-43: MLB_MDWE2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MSK	Bitwise write enable for CTR data - bits[95:64]. The <code>MLB_MDWE2</code> . MSK bit field contains the bitwise write enable for CTR data.

Memory Interface Control Data Write Enable 3 Register

The `MLB_MDWE3` register contains the bitwise write enable for Channel Table RAM (CTR) data bits [127:96]. When cleared (=0), the bit is disabled. When set (=1), the bit is enabled.

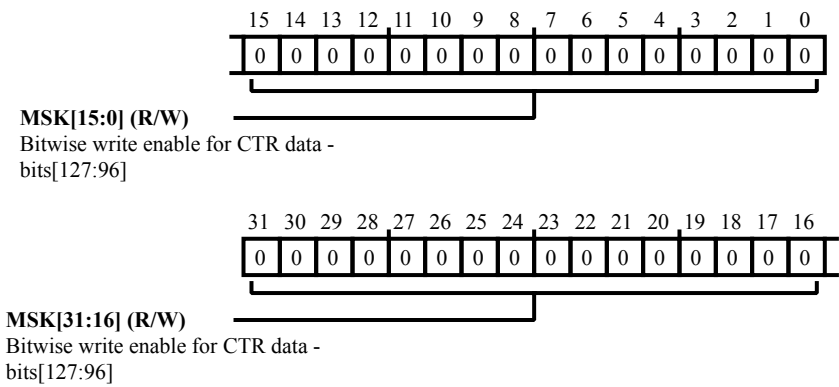


Figure 22-31: MLB_MDWE3 Register Diagram

Table 22-44: MLB_MDWE3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MSK	Bitwise write enable for CTR data - bits[127:96]. The <code>MLB_MDWE3.MSK</code> bit field contains the bitwise write enable for CTR data.

Interrupt Enable Register

The `MLB_MIEN` register is used to enable various interrupt conditions.

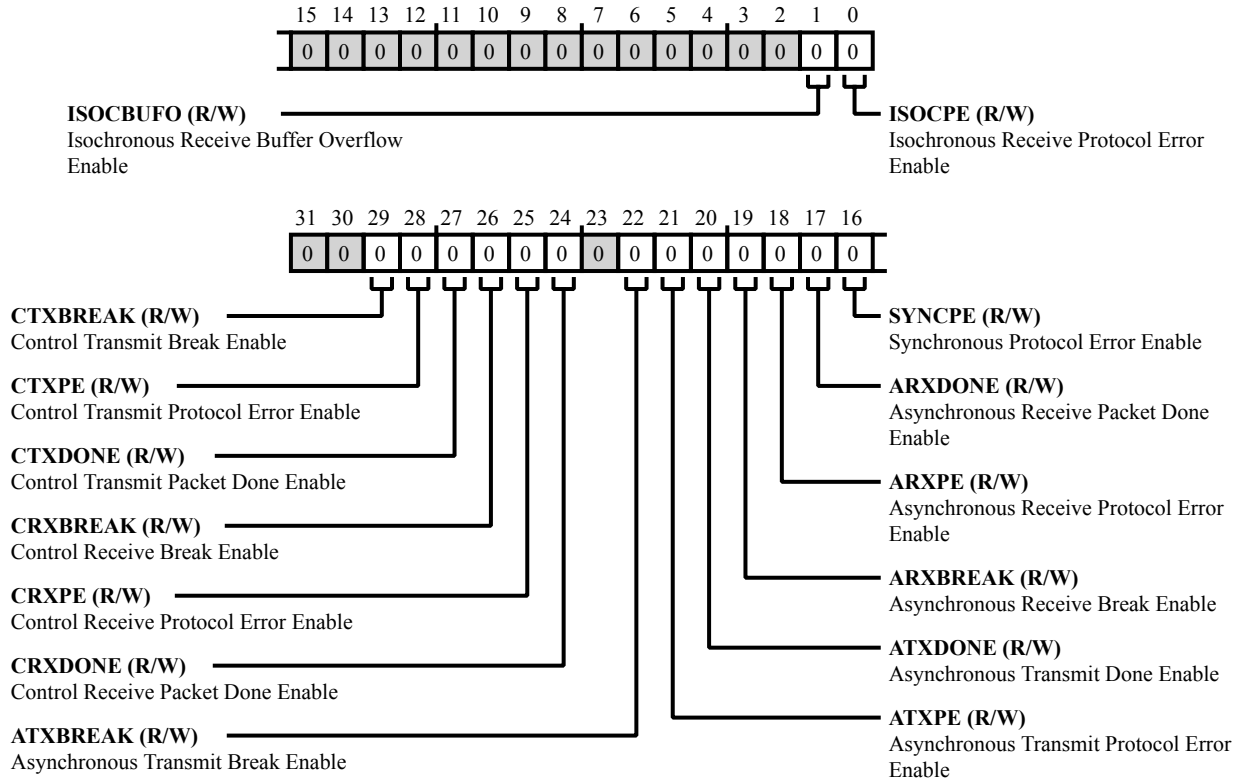


Figure 22-32: `MLB_MIEN` Register Diagram

Table 22-45: `MLB_MIEN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	CTXBREAK	Control Transmit Break Enable. When the <code>MLB_MIEN.CTXBREAK</code> bit is set, a ReceiverBreak response received from the receiver on a control Tx channel causes the appropriate channel bit in the <code>MLB_MS0</code> or <code>MLB_MS1</code> registers to be set.
28 (R/W)	CTXPE	Control Transmit Protocol Error Enable. When the <code>MLB_MIEN.CTXPE</code> bit is set, a ProtocolError generated by the receiver on a control Tx channel causes the appropriate channel bit in the <code>MLB_MS0</code> or <code>MLB_MS1</code> registers to be set.
27 (R/W)	CTXDONE	Control Transmit Packet Done Enable. When the <code>MLB_MIEN.CTXDONE</code> bit is set, a packet transmitted with no errors on a control Tx channel causes the appropriate channel bit in the <code>MLB_MS0</code> or <code>MLB_MS1</code> registers to be set.

Table 22-45: MLB_MIEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	CRXBREAK	Control Receive Break Enable. When the MLB_MIEN.CRXBREAK bit is set, a ControlBreak command received from the transmitter on a control Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
25 (R/W)	CRXPE	Control Receive Protocol Error Enable. When the MLB_MIEN.CRXPE bit is set, a ProtocolError detected on a control Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
24 (R/W)	CRXDONE	Control Receive Packet Done Enable. When the MLB_MIEN.CRXDONE bit is set, a packet received with no errors on a control Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
22 (R/W)	ATXBREAK	Asynchronous Transmit Break Enable. When the MLB_MIEN.ATXBREAK bit is set, a ReceiverBreak response received from the receiver on an asynchronous Tx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
21 (R/W)	ATXPE	Asynchronous Transmit Protocol Error Enable. When the MLB_MIEN.ATXPE bit is set, a ProtocolError generated by the receiver on an asynchronous Tx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
20 (R/W)	ATXDONE	Asynchronous Transmit Done Enable. When the MLB_MIEN.ATXDONE bit is set, a packet transmitted with no errors on an asynchronous Tx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
19 (R/W)	ARXBREAK	Asynchronous Receive Break Enable. When the MLB_MIEN.ARXBREAK bit is set, a AsyncBreak command received from the transmitter on an asynchronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
18 (R/W)	ARXPE	Asynchronous Receive Protocol Error Enable. When the MLB_MIEN.ARXPE bit is set, a protocol error detected on an asynchronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.
17 (R/W)	ARXDONE	Asynchronous Receive Packet Done Enable. When the MLB_MIEN.ARXDONE bit is set, a packet received with no errors on an asynchronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.

Table 22-45: MLB_MIEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	SYNCPE	Synchronous Protocol Error Enable. When the MLB_MIEN . SYNCPE bit is set, a protocol error detected on a synchronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set. This occurs only when isochronous flow control is disabled.
1 (R/W)	ISOCBUFO	Isochronous Receive Buffer Overflow Enable. When the MLB_MIEN . ISOCBUFO bit is set, a buffer overflow on an isochronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set. This occurs only when isochronous flow control is disabled.
0 (R/W)	ISOCPE	Isochronous Receive Protocol Error Enable. When the MLB_MIEN . ISOCPE bit is set, a protocol error detected on an isochronous Rx channel causes the appropriate channel bit in the MLB_MS0 or MLB_MS1 registers to be set.

Channel Status 0 Register

The `MLB_MS0` register indicates the channel status for MediaLB channels 31 to 0. Channel status bits are set by hardware and cleared by software. Status is only set if the appropriate bits in the `MLB_MIEN` register are set.

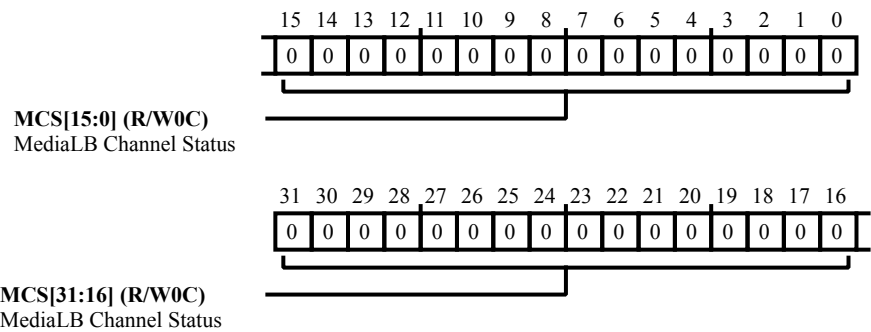


Figure 22-33: `MLB_MS0` Register Diagram

Table 22-46: `MLB_MS0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W0C)	MCS	MediaLB Channel Status. The <code>MLB_MS0.MCS</code> bit field indicates the MediaLB channel status for channels 31 to 0.

Channel Status 1 Register

The `MLB_MS1` register indicates the channel status for MediaLB channels 32 to 63. Channel status bits are set by hardware and cleared by software. Status is only set if the appropriate bits in the `MLB_MIEN` register are set.

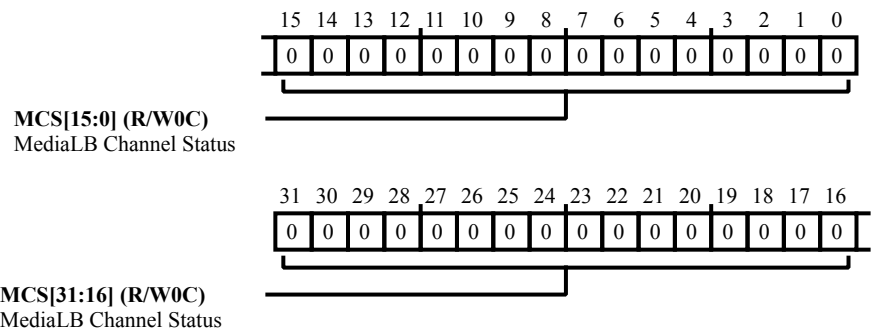


Figure 22-34: `MLB_MS1` Register Diagram

Table 22-47: `MLB_MS1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W0C)	MCS	MediaLB Channel Status. The <code>MLB_MS1</code> .MCS bit field indicates the MediaLB channel status for channels 63 to 32.

System Data Register

The `MLB_MSD` register allows system software to receive control information from the MLB controller. The `MLB_MSD` register is updated once per frame by the hardware during the MLB system channel.

The `MLB_MSD` register is loaded with the data from the `MLBDAT_IN` signal during the system channel quadlet. System software must read this register before the start of the next MLB frame to prevent the current data from being lost.

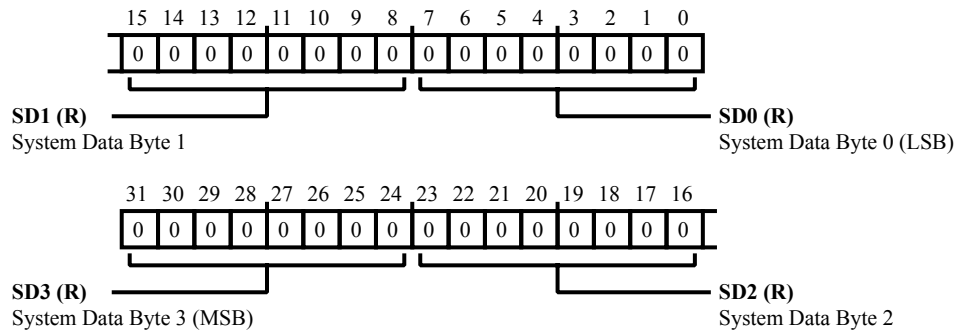


Figure 22-35: MLB_MSD Register Diagram

Table 22-48: MLB_MSD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	SD3	System Data Byte 3 (MSB). The <code>MLB_MSD.SD3</code> bits are updated with MediaLB Data [31:24] when a MediaLB software system command is received in the system quadlet. If the <code>MLB_MSS.SWSYSCMD</code> bit is already set, then SD3 is not updated.
23:16 (R/NW)	SD2	System Data Byte 2. The <code>MLB_MSD.SD2</code> bits are updated with MediaLB Data [23:16] when a MediaLB software system command is received in the system quadlet. If the <code>MLB_MSS.SWSYSCMD</code> bit is already set, then SD2 is not updated.
15:8 (R/NW)	SD1	System Data Byte 1. The <code>MLB_MSD.SD1</code> bits are updated with MediaLB Data [15:8] when a MediaLB software system command is received in the system quadlet. If the <code>MLB_MSS.SWSYSCMD</code> bit is already set, then SD1 is not updated.
7:0 (R/NW)	SD0	System Data Byte 0 (LSB). The <code>MLB_MSD.SD0</code> bits are updated with MediaLB Data [7:0] when a MediaLB software system command is received in the system quadlet. If the <code>MLB_MSS.SWSYSCMD</code> bit is already set, then SD0 is not updated.

System Status Register

The `MLB_MSS` register allows system software to monitor and control the status of the MLB network. The register is updated once per frame by hardware during the MLB system channel. The bits of this register are not valid until the processor is locked to the MLB interface (except for the bits associated with MLB lock and unlock, SDMU and SDML). System software must service events before the start of the next MLB frame to prevent the current frame status from being lost.

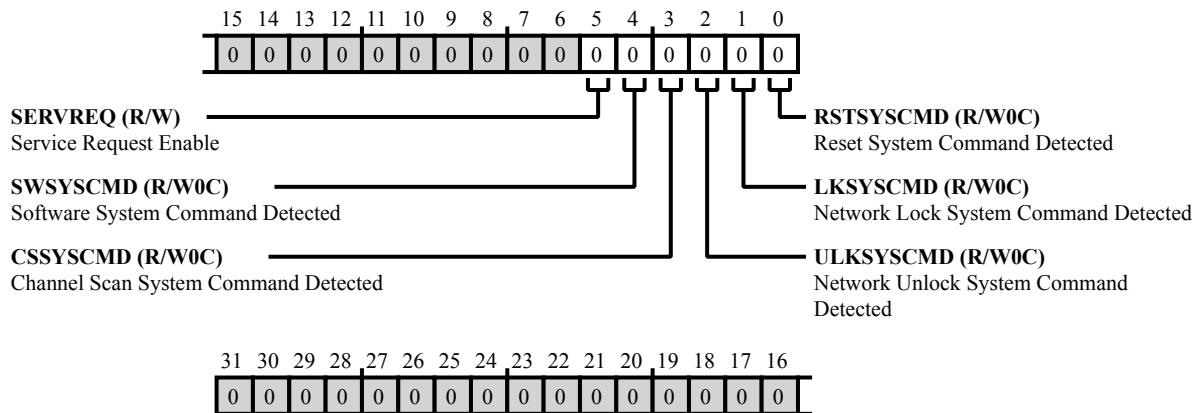


Figure 22-36: MLB_MSS Register Diagram

Table 22-49: MLB_MSS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SERVREQ	Service Request Enable. When the <code>MLB_MSS.SERVREQ</code> bit set, the MediaLB block responds with a device present, request service system response if a matching channel scan system command is detected. When cleared, the MediaLB block responds with a device present system response.
4 (R/W0C)	SWSYSCMD	Software System Command Detected. The <code>MLB_MSS.SWSYSCMD</code> bit indicates that a software system command was detected (in the system quadlet). The <code>MLB_MSS.SWSYSCMD</code> bit is set by hardware, cleared by software. Data is stored in the <code>MLB_MSD</code> register for this command.
3 (R/W0C)	CSSYSCMD	Channel Scan System Command Detected. The <code>MLB_MSS.CSSYSCMD</code> bit indicates that a channel scan system command was detected (in the system quadlet). The <code>MLB_MSS.CSSYSCMD</code> bit is set by hardware, cleared by software. If the node address specified in the data quadlet matches the value in the <code>MLB_CTL1.NDA</code> bit field, the device responds either device present or device present, request service system response in the next system quadlet.

Table 22-49: MLB_MSS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W0C)	ULKSYSCMD	Network Unlock System Command Detected. The <code>MLB_MSS.ULKSYSCMD</code> bit indicates a network unlock system command was detected (in the system quadlet). The <code>MLB_MSS.ULKSYSCMD</code> bit is by hardware, cleared by software.
1 (R/W0C)	LKSYSCMD	Network Lock System Command Detected. The <code>MLB_MSS.LKSYSCMD</code> bit indicates a network lock system command was detected (in the system quadlet). The <code>MLB_MSS.LKSYSCMD</code> bit is set by hardware and cleared by software.
0 (R/W0C)	RSTSYSCMD	Reset System Command Detected. The <code>MLB_MSS.RSTSYSCMD</code> bit indicates a reset system command was detected (in the system quadlet). The <code>MLB_MSS.RSTSYSCMD</code> bit is set by hardware, cleared by software.

MediaLB 6-pin Control 0 Register

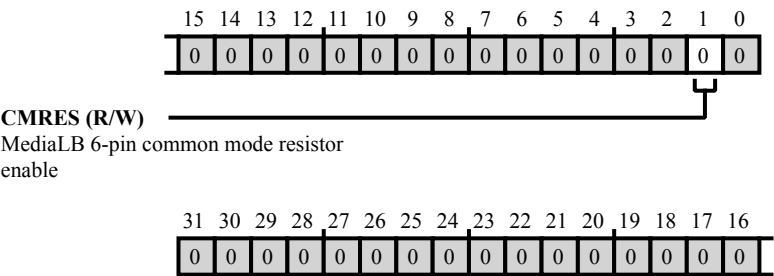


Figure 22-37: MLB_PCTL0 Register Diagram

Table 22-50: MLB_PCTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	CMRES	MediaLB 6-pin common mode resistor enable. This signal is no longer used.

23 Two-Wire Interface (TWI)

The processor has a two-wire interface (TWI), that provides a simple exchange method of control data between multiple devices. The TWI module is compatible with the widely used I²C bus standard. Additionally, the TWI module is fully compatible with serial camera control bus (SCCB) functionality for easier control of various CMOS camera sensor devices.

The TWI module offers the capabilities of simultaneous master and slave operation and support for both 7-bit addressing and multimedia data arbitration. The TWI interface uses two pins for transferring clock (TWI_SCL) and data (TWI_SDA) and supports the protocol at speeds up to 400K bits/sec. The TWI interface pins are compatible with 3.3V logic levels.

To preserve processor bandwidth, the TWI module can be set up with transfer-initiated interrupts to only service FIFO buffer data reads and writes. Protocol-related interrupts are optional. The TWI externally moves 8-bit data while maintaining compliance with the I²C bus protocol.

NOTE: All TWI instances (TWI0-5) operate in the SCLK0 domain.

TWI Features

The TWI is fully compatible with the widely used I²C bus standard.

The TWI controller includes the following features.

- Simultaneous master and slave operation on multiple device systems
- Support for multi-master bus arbitration
- 7-bit addressing
- 100K bits/second and 400K bits/second data rates
- General call address support
- Master clock synchronization and support for clock low extension
- Separate multiple-byte receive and transmit FIFOs
- Low interrupt rate

- Individual override control of data and clock lines in the event of bus lock-up
- Input filter for spike suppression
- Serial camera control bus support as specified in the *OmniVision Serial Camera Control Bus (SCCB) Functional Specification*
- Programmable drive/tolerance of TWI pins. Refer to the [PADS_PCFG0](#) register description in General-Purpose Ports (PORT) chapter for details.

TWI Functional Description

The TWI interface is a shift register that serially transmits and receives data bits. It moves data 1 bit at a time at the SCL rate, to and from other TWI devices. The SCL signal synchronizes the shifting and sampling of the data on the serial data pin.

ADSP-2159x TWI Register List

The Two-Wire Interface controller TWI allows a device to interface to an inter-IC bus as specified by the Philips I²C Bus Specification version 2.1, dated January 2000. A set of registers governs TWI operations. For more information on TWI functionality, see the TWI register descriptions.

Table 23-1: ADSP-2159x TWI Register List

Name	Description
TWI_CLKDIV	SCL Clock Divider Register
TWI_CTL	Control Register
TWI_FIFCTL	FIFO Control Register
TWI_FIFOSTAT	FIFO Status Register
TWI_IMSK	Interrupt Mask Register
TWI_ISTAT	Interrupt Status Register
TWI_MSTRADDR	Master Mode Address Register
TWI_MSTRCTL	Master Mode Control Registers
TWI_MSTRSTAT	Master Mode Status Register
TWI_RXDATA16	Rx Data Double-Byte Register
TWI_RXDATA8	Rx Data Single-Byte Register
TWI_SLVADDR	Slave Mode Address Register
TWI_SLVCTL	Slave Mode Control Register
TWI_SLVSTAT	Slave Mode Status Register
TWI_TXDATA16	Tx Data Double-Byte Register
TWI_TXDATA8	Tx Data Single-Byte Register

ADSP-2159x TWI Interrupt List

Table 23-2: ADSP-2159x TWI Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
150	TWI0_DATA	TWI0 Data Interrupt	Level	
151	TWI1_DATA	TWI1 Data Interrupt	Level	
152	TWI2_DATA	TWI2 Data Interrupt	Level	
153	TWI3_DATA	TWI3 Data Interrupt	Level	
154	TWI4_DATA	TWI4 Data Interrupt	Level	
155	TWI5_DATA	TWI5 Data Interrupt	Level	

TWI Block Diagram

The *TWI Block Diagram* figure shows the basic blocks of the TWI interface.

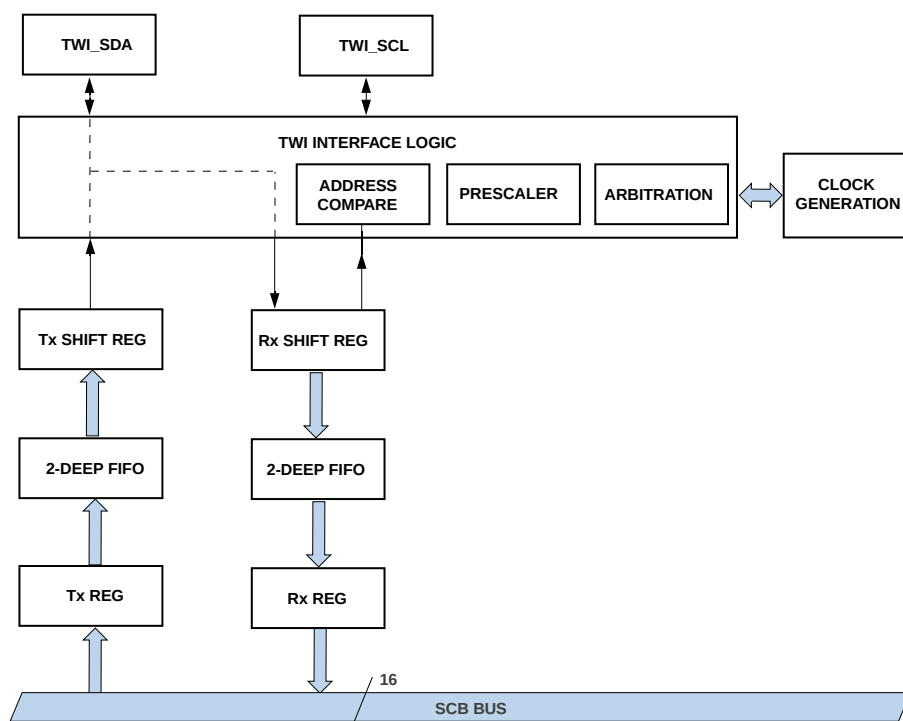


Figure 23-1: TWI Block Diagram

External Interface

The TWI_SDA (serial data) and TWI_SCL (serial clock) signals are open drain and require pull-up resistors. These bidirectional signals externally interface the TWI controller to the I²C bus and no other external connections or logic are necessary.

Serial Clock Signal (SCL)

The serial clock signal (`TWI_SCL`) is an input in slave mode. In master mode, the TWI controller must set this signal to the desired frequency.

The TWI controller supports the standard mode of operation (up to 100 kHz) or fast mode (up to 400 kHz). The TWI control register (`TWI_CTL`) sets the `TWI_CTL.PRESCALE` value which sets the relationship between the system clock (`SCLK0`) and the internally timed events of the TWI controller. The internal time reference is derived from `SCLK0` using a prescaled value. The prescale value is the number of `SCLK0` periods used in the generation of one internal time reference. Set the value of prescale to create an internal time reference with a period of 10 MHz. It is represented as a 7-bit binary value as follows.

$$\text{PRESCALE} = f_{\text{SCLK0}}/10\text{MHz}$$

NOTE: It is not always possible to achieve 10-MHz accuracy. In such cases, it is safe to round up the `PRESCALE` value to the next highest integer. For example, if `SCLK0` is 100 MHz, the `PRESCALE` value is calculated as $100\text{ MHz}/10\text{ MHz} = 10$. A prescale value of 14 in this case ensures that all timing requirements are met.

During master mode operation, the TWI module uses the `TWI_CLKDIV` register values to create the minimum `TWI_CLKDIV.CLKHI` and `TWI_CLKDIV.CLKLO` durations of the `TWI_SCL` signal. The `TWI_CLKDIV.CLKHI` field specifies the minimum number of 10-MHz time reference periods the `TWI_SCL` waits before a new clock low period begins, assuming a single master. (The 10-MHz time reference periods are represented as an 8-bit binary value). The TWI uses the `TWI_CLKDIV.CLKLO` field to specify the minimum number of internal time reference periods (represented as an 8-bit binary value); the `TWI_SCL` signal is held low.

Serial clock frequencies can vary from 400 kHz to less than 20 kHz. The resolution of the clock generated is 1/10 MHz or 100 ns. The following equation describes the frequency.

$$\text{CLKDIV} = \text{TWI_SCL period}/10\text{ MHz time reference.}$$

For example, for an `TWI_SCL` of 400 kHz (period = $1/400\text{ kHz} = 2500\text{ ns}$) and an internal time reference of 10 MHz (period = 100 ns), the following equation applies:

$$\text{CLKDIV} = 2500\text{ ns}/100\text{ ns} = 25$$

Therefore, a `TWI_SCL` signal with a 30% duty cycle has `TWI_CLKDIV.CLKLO`=17 and `TWI_CLKDIV.CLKHI`=8. Adding `TWI_CLKDIV.CLKLO` and `TWI_CLKDIV.CLKHI` equals `CLKDIV`.

NOTE: The `TWI_CLKDIV.CLKHI` and `TWI_CLKDIV.CLKLO` fields are not intended to guarantee a certain frequency. Rather, they guarantee a certain minimum high and low duration for the `TWI_SCL` signal. Slew rate controls falling edges. The *RC* time constant governs the rising edges. The pull-up resistor and the `TWI_SCL` capacitance form the time constant. See the “Register Descriptions” section for more details.

Serial Data Signal (SDA)

The TWI transmits and receives serial data, depending on the direction of the transfer, on the bidirectional serial data signal (`SDA`).

Internal Interface

The peripheral bus interface supports the transfer of 16-bit wide data. The processor uses the interface in the support of register and FIFO buffer reads and writes. The TWI internal interface is comprised of the blocks described as follows.

Register block. Contains all control and status bits and reflects what can be written or read as outlined by the programming model. Each function block updates their corresponding status bits.

FIFO buffer. Configured as a 1-byte-wide, 2-deep transmit FIFO buffer and a 1-byte-wide, 2-deep receive FIFO buffer.

Transmit shift register. Serially shifts its data out externally off chip. The output can be controlled for generation of acknowledgments or it can be manually overwritten.

Receive shift register. Receives its data serially from off chip. The receive shift register is 1 byte wide and data received can either be transferred to the FIFO buffer or used in an address comparison.

Address compare block. Supports address comparison in the event the TWI controller module is accessed as a slave.

Prescaler block. Must be programmed to generate a 10-MHz time reference relative to the system clock. The block uses this time base for filtering of data and timing events specified by the electrical data sheet (See the Philips specification). The block uses the time base to generate the TWI_SCL clock as well.

Clock generation module. Generates an external TWI_SCL clock when in master mode. It includes the logic necessary for synchronization in a multi-master clock configuration and clock stretching when configured in slave mode.

NOTE: The TWI does not support DMA based operation.

TWI Architectural Concepts

The TWI controller follows the transfer protocol of the Philips I²C Bus specification version 2.1 dated January 2000.

NOTE: The TWI unit does not support DMA-based operation.

TWI Protocol

The *Data Transfer* figure shows a simple complete transfer.

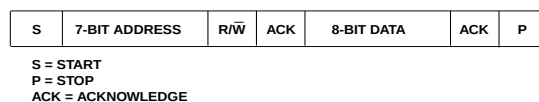
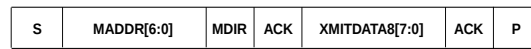


Figure 23-2: Data Transfer

The TWI controller register contents maps to a basic transfer. The *Data Transfer with Bit Illustration* figure details the same transfer from the *Data Transfer* figure noting the corresponding TWI controller bit names. In this illustration, the TWI controller successfully transmits 1 byte of data. The slave has acknowledged both address and data.



S = START
P = STOP
ACK = ACKNOWLEDGE

Figure 23-3: Data Transfer with Bit Illustration

Clock Generation and Synchronization

The TWI controller implementation only issues a clock during master mode operation and only at the time a transfer initiates. If arbitration for the bus is lost, the serial clock output immediately three-states. If multiple clocks attempt to drive the serial clock line, the TWI controller synchronizes its clock with the other remaining clocks. The *Clock Synchronization* figure shows this functionality.

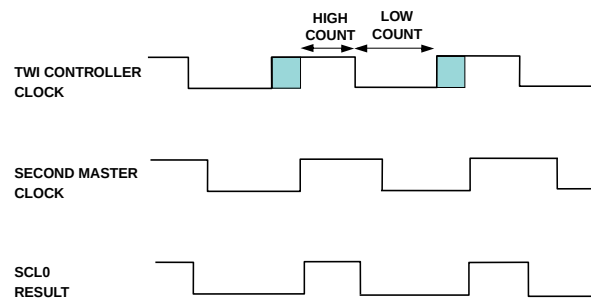


Figure 23-4: Clock Synchronization

The TWI controller serial clock (TWI_SCL) output follows these rules:

- Once the clock high (TWI_CLKDIV.CLKHI) count is complete, the serial clock output is driven low and the clock low (TWI_CLKDIV.CLKLO) count begins.
- Once the clock low count is complete, the serial clock line is three-stated. This state allows the external pull-up resistor to pull the TWI_SCL signal high. The clock synchronization logic enters into a delay mode (shaded area) until the TWI_SCL signal is detected at logic 1 level. Now, the clock high count begins.

Bus Arbitration

The TWI controller initiates a master mode transmission only when the bus is idle. If the bus is idle and two masters initiate a transfer, arbitration for the bus begins. The *Bus Arbitration* figure shows the arbitration.

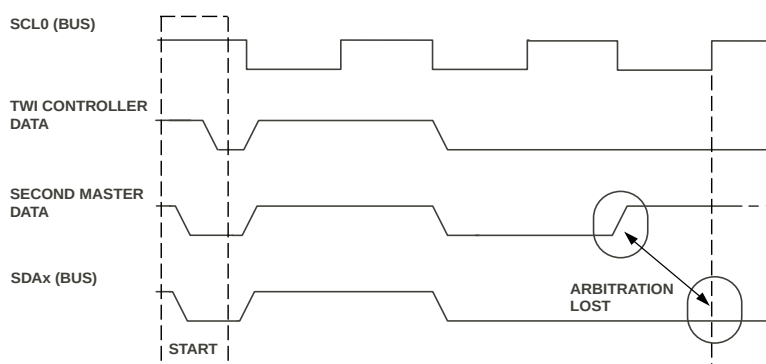


Figure 23-5: Bus Arbitration

The TWI controller monitors the serial data bus (SDA) while the `TWI_SCL` signal is high. If the `TWI_SDA` signal is determined to be an active logic 0 level while the data of the TWI controller is a logic 1 level, the TWI controller has lost arbitration. It stops generating the clock and data signals. Arbitration is not only performed at the serial clock edges, but also during the entire time the `TWI_SCL` signal is high.

Start and Stop Conditions

Start and stop conditions involve serial data transitions while the serial clock is a logic 1 level. The TWI controller generates and recognizes these transitions. Typically, start and stop conditions occur at the beginning and at the conclusion of a transmission, except repeated start combined transfers. The *Start and Stop Conditions* figure shows the transitions.

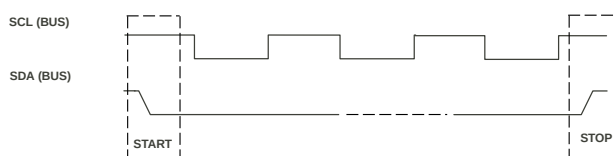


Figure 23-6: Start and Stop Conditions

The TWI special case start and stop conditions of the TWI controller include the following.

- Controller addressed as a slave-receiver. If the master asserts a stop condition during the data phase of a transfer, the TWI controller concludes the transfer (`TWI_ISTAT . SCOMP`).
- Controller addressed as a slave-transmitter. If the master asserts a stop condition during the data phase of a transfer, the TWI controller concludes the transfer (`TWI_ISTAT . SCOMP`) and indicates a slave transfer error (`TWI_ISTAT . SERR`).
- Controller as a master-transmitter or master-receiver. If the stop bit (`TWI_MSTRCTL . STOP`) is set during an active master transfer, the TWI controller issues a stop condition as soon as possible avoiding any error conditions. The TWI controller operates as if data transfer count had been reached.

General Call Support

The TWI controller always decodes and acknowledges a general call address if:

- The TWI controller is enabled as a slave

- General call is enabled

The `TWI_SLVCTL.GEN` bit configures general call addressing (0x00) only when the TWI controller is a slave-receiver.

If the data associated with the transfer is (NAK) not acknowledged, the `TWI_SLVCTL.NAK` bit can be set. If the TWI controller issues a general call as a master-transmitter, set the appropriate address (`TWI_MSTRADDR` register) and transfer direction (`TWI_MSTRCTL.DIR` bit) and load the transmit FIFO data.

NOTE: The byte following the general call address usually defines the slaves response to the call. The interpretation of the command in the second byte is based on the value of its LSB. For a TWI slave device, the bytes received after the general call address are considered data.

Fast Mode

Fast mode essentially uses the same mechanics as the standard mode of operation. Fast mode affects electrical specifications and timing. When fast mode is enabled, (FAST) timing is modified to meet the following electrical requirements.

- Serial data rise times before arbitration evaluation (t_r)
- Stop condition set-up time from serial clock to serial data (t_{SUSTO})
- Bus free time between a stop and start condition (t_{BUF})

TWI Operating Modes

The TWI has two modes of operation: repeated start and clock stretching. The following sections describe the operating modes.

Repeated Start

A repeated start condition is the absence of a stop condition between two transfers. The two transfers can be of any direction type. Examples include a transmit followed by a receive, or a receive followed by a transmit. The following sections guide the programmer in developing a service routine.

Transmit Receive Repeated Start

The *Repeated Start Followed by Data Receive* figure shows a repeated start followed by a data receive sequence. The shading in the figure indicates that the slave has control of the bus.

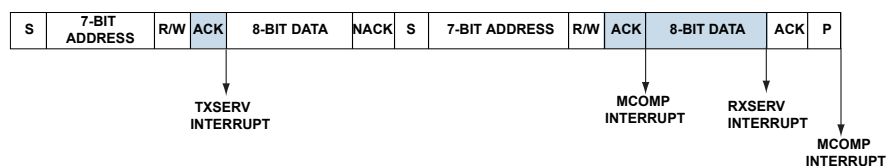


Figure 23-7: Repeated Start Followed by Data Receive

The tasks performed at each interrupt are:

- Transmit FIFO service (TWI_I_STAT.TXSERV) interrupt request. This interrupt is generated due to a FIFO access. Since this byte is the last of this transfer, the TWI uses the TWI_FIFOSTAT register to indicate that the transmit FIFO is empty. When read, TWI_MSTRCTL.DCNT bit field=0. Set the TWI_MSTRCTL.RSTART bit to indicate a repeated start and set the TWI_MSTRCTL.DIR bit if the following transfer is a data receive.
- Master transfer complete (TWI_I_STAT.MCOMP) interrupt. This interrupt request is generated when all data transfers (TWI_MSTRCTL.DCNT bit field=0). If no errors occur, a start condition initiates. Clear the TWI_MSTRCTL.RSTART bit and program the TWI_MSTRCTL.DCNT bits with the desired number of bytes to receive.
- Receive FIFO service (TWI_I_STAT.RXSERV) interrupt. This interrupt request is generated due to the arrival of a byte in the receive FIFO. Simple data handling is the only requirement.
- Master transfer complete (TWI_I_STAT.MCOMP) interrupt. This interrupt request is generated when the transfer completes.

Receive Transmit Repeated Start

The *Repeated Start Data Receive Followed by Data Transmit* figure illustrates a repeated start data receive followed by a data transmit sequence. The shading in the figure indicates that the slave has control of the bus.

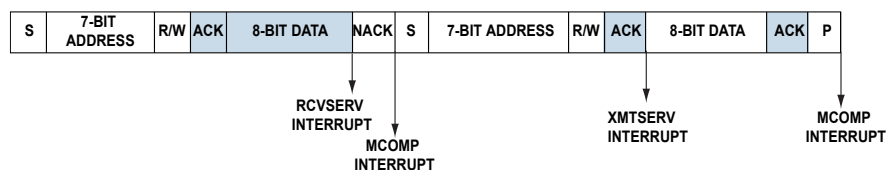


Figure 23-8: Repeated Start Data Receive Followed by Data Transmit

The tasks performed at each interrupt are:

- Receive FIFO service (TWI_I_STAT.RXSERV) interrupt. This interrupt request is generated due to the arrival of a data byte in the receive FIFO. Set the TWI_MSTRCTL.RSTART bit to indicate a repeated start and clear the TWI_MSTRCTL.DIR bit if the following transfer is a data transmit.
- Master transfer complete (TWI_I_STAT.MCOMP) interrupt. This interrupt request has occurred due to the completion of the data receive transfer. If no errors occur, a start condition initiates. Clear the TWI_MSTRCTL.RSTART bit and program the TWI_MSTRCTL.DCNT bits with the desired number of bytes to transmit.
- Transmit FIFO service (TWI_I_STAT.TXSERV) interrupt. This interrupt request is generated due to a FIFO access. Simple data handling is the only requirement.
- Master transfer complete (TWI_I_STAT.MCOMP) interrupt. This interrupt request is generated when the transfer completes.

NOTE: There is no timing constraint to meet the conditions—program the bits as required. Refer to [Clock Stretching During Repeated Start](#) section for more on how the controller stretches the clock during repeated start transfers.

Clock Stretching

Clock stretching is an added function of the TWI controller in master mode operation. This behavior uses self-induced stretching of the I²C clock while waiting to service interrupts. Hardware initiates stretching automatically. No programming is necessary. The TWI controller as a master supports three modes of clock stretching:

- [Clock Stretching During FIFO Underflow](#)
- [Clock Stretching During FIFO Overflow](#)
- [Clock Stretching During Repeated Start](#)

Clock Stretching During FIFO Underflow

During a master mode transmit, an interrupt request occurs the instant the transmit FIFO becomes empty. The most recent byte begins transmission. If the `TWI_ISTAT.TXSERV` interrupt request is not serviced, the concluding acknowledge phase of the transfer stretches.

Stretching of the clock continues until new data bytes are written to the transmit FIFO ([TWI_TXDATA8](#) or [TWI_TXDATA16](#) registers). No other action is required to release the clock and continue the transmission. This behavior continues until the transmission completes (`TWI_MSTRCTL.DCNT=0`). The transmission concludes (`TWI_ISTAT.MCOMP`). The *Clock Stretching during FIFO Underflow* figure and table show the stretching.

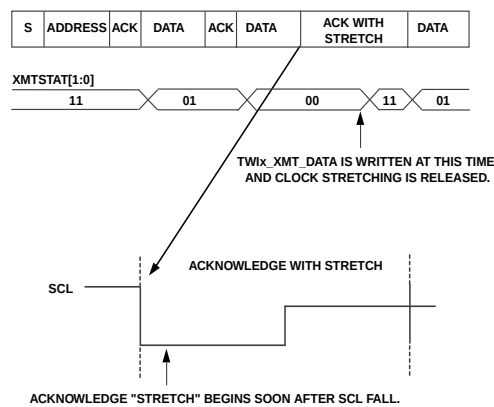


Figure 23-9: Clock Stretching during FIFO Underflow

TWI Controller	Processor
Interrupt: XMTSERV – Transmit FIFO buffer is empty.	Acknowledge: Clear the interrupt request source bits. Write to the transmit FIFO buffer.
...	...
Interrupt: MCOMP – Master transmit complete (DCNT= 0x00).	Acknowledge: Clear the interrupt request source bits.

Clock Stretching During FIFO Overflow

During a master mode receive operation, an interrupt occurs at the instant the receive FIFO becomes full. It is during the acknowledge phase of this received byte that clock stretching begins. The TWI module makes no attempt to initiate the reception of another byte. Stretching of the clock continues until the data bytes previously received are read from the receive FIFO buffer (`TWI_RXDATA8` or `TWI_RXDATA16` registers). No other action is required to release the clock and continue the reception of data. This behavior continues until the reception is complete (`TWI_MSTRCTL.DCNT=0`). Reception concludes (`TWI_ISTAT.MCOMP`). The *Clock Stretching During FIFO Overflow* figure and table show the clock stretching.

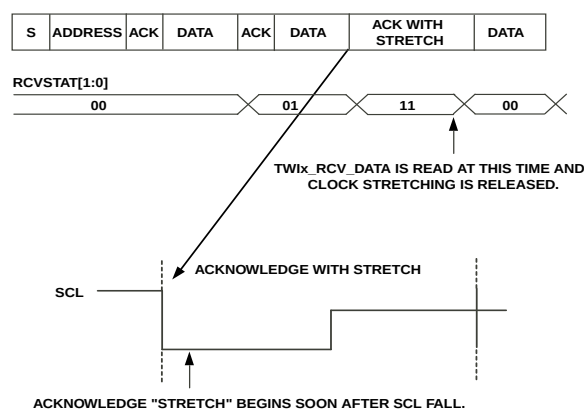


Figure 23-10: Clock Stretching During FIFO Overflow

TWI Controller	Processor
Interrupt: RCVSERV – Receive FIFO buffer is full.	Acknowledge: Clear the interrupt request source bits. Read the receive FIFO buffer.
...	...
Acknowledge: Clear the interrupt source bits.	Interrupt: MCOMP – Master receive complete.

Clock Stretching During Repeated Start

The repeated start feature in I²C protocol requires a transition between two subsequent transfers. With the use of clock stretching, the task of managing transitions becomes simpler and common to all transfer types.

Once an initial TWI master transfer completes (transmit or receive), the clock initiates a stretch during the repeated start phase between transfers. Concurrent with this event, the initial transfer generates a `TWI_ISTAT.MCOMP` interrupt to signify the initial transfer has completed (`TWI_MSTRCTL.DCNT=0`). This initial transfer is handled without any special bit setting sequences or timing.

The clock stretching logic described applies here. With no system-related timing constraints, the subsequent transfer (receive or transmit) is set up and activated. This sequence can repeat as many times as required to string a series of repeated start transfers together. The *Clock Stretching during Repeated Start Condition* figure and table show the clock stretching.

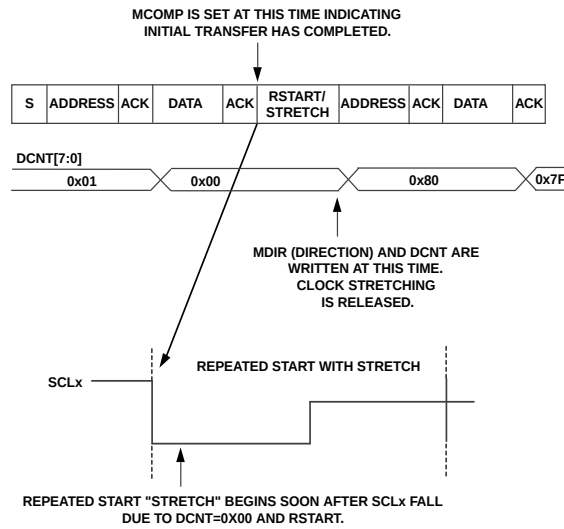


Figure 23-11: Clock Stretching during Repeated Start Condition

TWI Controller	Processor
Interrupt: MCOMP – Initial transmit has completed and DCNT = 0x00. Note: transfer in progress, RSTART previously set.	Acknowledge: Clear the interrupt request source bits. Write to TWIx_MASTER_CTL, setting MDIR (receive), clearing RSTART, and setting new DCNT value (nonzero).
Interrupt: RCVSERV – Receive FIFO is full.	Acknowledge: Clear the interrupt request source bits. Read the receive FIFO buffer.
...	...
Interrupt: MCOMP – Master receive complete	Acknowledge: Clear the interrupt request source bits.

TWI Programming Model

The topics in this section provide information on the basic programming steps required to set up and run the two wire interface.

General Setup

General setup refers to register writes that are required for both slave mode and master mode operations.

Perform general setup before setting either the master or slave enable bits.

1. Program the `TWI_CTL.EN` bit to enable the TWI controller and set the prescale value (`TWI_CTL.PRESCALE` bit).
2. Program the prescale value to the binary representation of $f_{SCLK0}/10$ MHz. Round up all values to the next whole number.
3. Set the `TWI_CTL.EN` bit to enable the controller.

Once the TWI controller is enabled, a bus busy condition can be detected. This condition clears after t_{BUF} has expired, assuming no additional bus activity has been detected.

Slave Mode

When enabled, slave mode operation supports both receive and transmit data transfers.

It is not possible to enable only one data transfer direction and not acknowledge (NAK) the other. The following setup reflects this functionality.

1. Program the `TWI_SLVADDR` register. The TWI uses the appropriate 7 bits in determining a match during the address phase of the transfer.
2. Program the `TWI_TXDATA8.VALUE` or `TWI_TXDATA16` registers. These values are the initial data values for transmission when the slave is addressed and transmission is needed. This step is optional. If no data is written when the slave is addressed and transmission is needed, the serial clock (`TWI_SCL`) stretches. An interrupt is generated until data is written to the transmit FIFO.
3. Program the `TWI_IMSK` register. There are enable-bits associated with the desired interrupt sources. For example, programming the value 0x000F results in an interrupt request output to the processor, when the TWI module detects a valid address match. An interrupt request also occurs when a valid slave transfer completes or has an error, or a subsequent transfer has begun and the previous transfer has not been serviced.
4. Program the `TWI_SLVCTL` register. This step prepares and enables slave mode operation. For example, programming the value 0x0005 enables slave mode operation and requires 7-bit addressing. It indicates that data in the transmit FIFO buffer is for slave mode transmission.

The *Slave Mode Interaction* table and *TWI Slave Mode Program Flow* diagram represent the interaction between the TWI controller and the processor using this example.

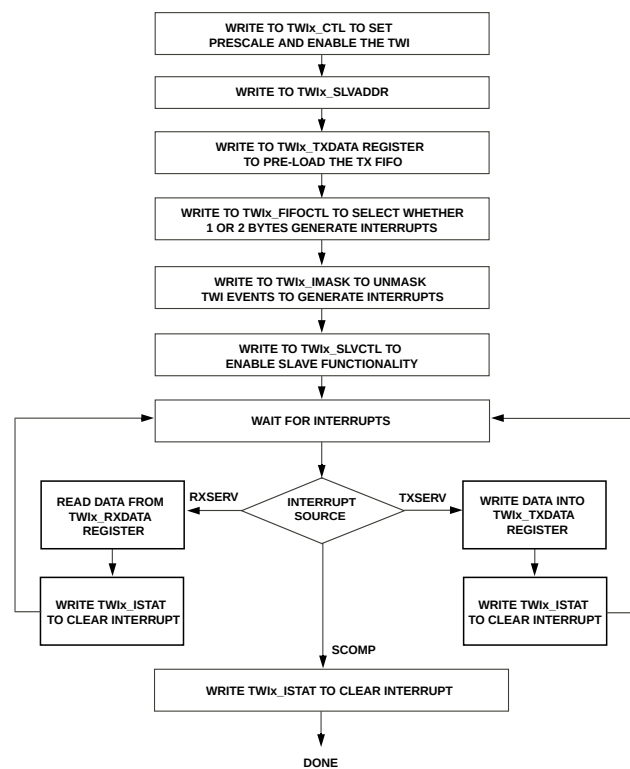


Figure 23-12: TWI Slave Mode Program Flow

Table 23-3: Slave Mode Interaction

TWI Controller	Processor
Interrupt: SINIT – Slave transfer in progress.	Acknowledge: Clear the interrupt source bits.
Interrupt: RCVSERV – Receive buffer is full.	Acknowledge: Clear the interrupt source bits. Read TWIx_FIFO_STAT. Read the receive FIFO buffer.
...	...
Interrupt: SCOMP – Slave transfer complete.	Acknowledge: Clear the interrupt source bits. Read the receive FIFO buffer.

Master Mode Program Flow

The *Master Mode Program Flow* figure shows the program for the TWI in master mode.

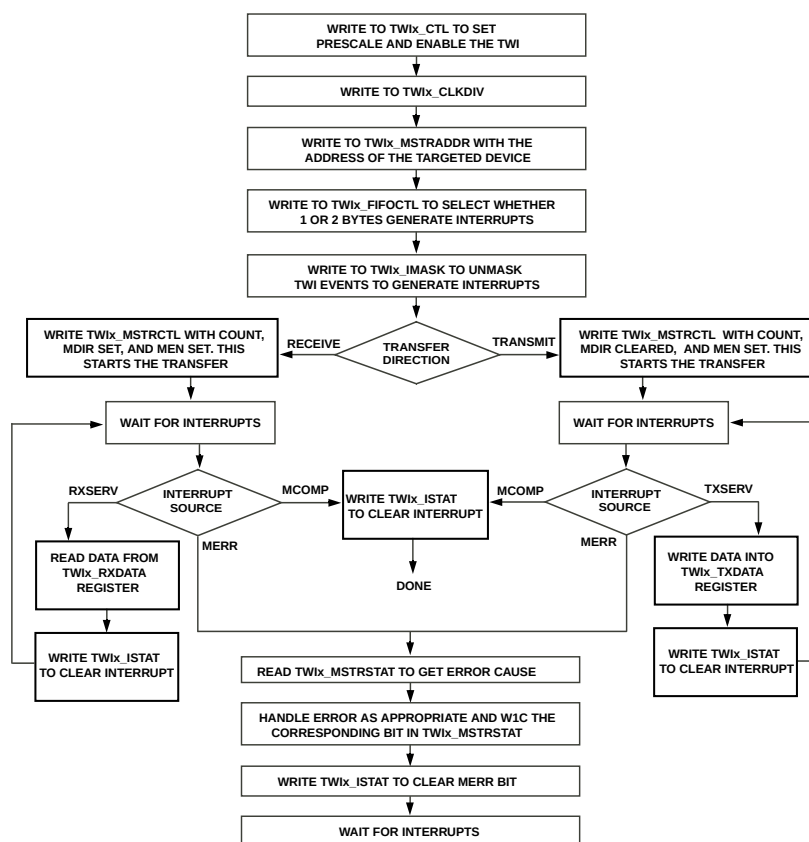


Figure 23-13: Master Mode Program Flow

Master Mode Clock Setup

Master mode operation is set up and executed on a per-transfer basis.

An example of programming steps for a receive and for a transmit is given separately in following sections. The programming step for clock setup listed here is common to both transfer types.

1. Program the `TWI_CLKDIV` register to define the minimum high and minimum low duration for the clock.

The `TWI_CLKDIV.CLKHI` and `TWI_CLKDIV.CLKLO` fields do not guarantee a certain frequency. Rather, they guarantee a certain minimum high and low duration for `TWI_SCL`. The slew rate controls falling edges. The RC time constant formed by the pull-up resistor and the SCL capacitance govern rising edges. See the “Register Descriptions” section for more details.

Master Mode Transmit

Follow these programming steps for a single master mode transmission:

1. Program the `TWI_MSTRADDR` register. This step defines the address transmitted during the address phase of the transfer.

2. Program the `TWI_TXDATA8` or `TWI_TXDATA16` register. This step configures the initial data transmitted. It is an error to complete the address phase of the transfer and not have data available in the transmit FIFO buffer.
3. Program the `TWI_FIFCTL` register. The programming indicates if the transmit FIFO buffer interrupt requests occur with each byte transmitted (8-bits) or with every 2 bytes transmitted (16-bits).
4. Program the `TWI_IMSK` register. This step enables the bits associated with the desired interrupt request sources. For example, programming the value 0x0030 results in an interrupt output to the processor when the master transfer completes, and the master transfer has an error.
5. Program the `TWI_MSTRCTL` register. This step prepares and enables master mode operation. For example, programming the value 0x0201: enables master mode operation, generates a 7-bit address, sets the direction to master-transmit, uses standard mode timing, and transmits 8 data bytes before generating a stop condition.

The *Master Mode Transmit Setup Interaction* table represents the interaction between the TWI controller and the processor using this example.

Table 23-4: Master Mode Transmit Setup Interaction

TWI Controller	Processor
Interrupt: XMTSERV – Transmit buffer is empty.	Acknowledge: Clear the interrupt request source bits. Write to the transmit FIFO buffer.
...	...
Interrupt: MCOMP – Master transfer complete.	Acknowledge: Clear the interrupt request source bits.

Master Mode Receive

Follow these programming steps for a single master mode receive.

1. Program the `TWI_MSTRADDR` register. This step defines the address transmitted during the address phase of the transfer.
2. Program the `TWI_FIFCTL` register. This step indicates if the receive FIFO buffer interrupt requests occur with each byte received (8-bits) or with every 2 bytes received (16-bits).
3. Program the `TWI_IMSK` register. This step configures the enable bits associated with the desired interrupt sources. For example, programming the value 0x0030 results in an interrupt request output to the processor when the master transfer completes, and the master transfer has an error.
4. Program the `TWI_MSTRCTL` register. This step prepares and enables master mode operation. For example, programming the value 0x0205: enables master mode operation, generates a 7-bit address, sets the direction to master-receive, uses standard mode timing, and receives 8 data bytes before generating a stop condition.

The *Master Mode Receive Setup Interaction* table shows the interaction between the TWI controller and the processor using this example.

Table 23-5: Master Mode Receive Setup Interaction

TWI Controller	Processor
Interrupt: RCVSERV – Receive buffer is full.	Acknowledge: Clear the interrupt request source bits. Read the receive FIFO buffer.
...	...
Interrupt: MCOMP – Master transfer complete.	Acknowledge: Clear the interrupt request source bits. Read the receive FIFO buffer.

NOTE: After the `TWI_MSTRCTL.DCNT` bit decrements to zero, the TWI master device sends a NAK to indicate to the slave transmitter to release the bus. This operation allows the master to send the stop signal to terminate the transfer.

ADSP-2159x TWI Register Descriptions

Two-Wire Interface (TWI) contains the following registers.

Table 23-6: ADSP-2159x TWI Register List

Name	Description
<code>TWI_CLKDIV</code>	SCL Clock Divider Register
<code>TWI_CTL</code>	Control Register
<code>TWI_FIFCTL</code>	FIFO Control Register
<code>TWI_FIFOSTAT</code>	FIFO Status Register
<code>TWI_IMSK</code>	Interrupt Mask Register
<code>TWI_ISTAT</code>	Interrupt Status Register
<code>TWI_MSTRADDR</code>	Master Mode Address Register
<code>TWI_MSTRCTL</code>	Master Mode Control Registers
<code>TWI_MSTRSTAT</code>	Master Mode Status Register
<code>TWI_RXDATA16</code>	Rx Data Double-Byte Register
<code>TWI_RXDATA8</code>	Rx Data Single-Byte Register
<code>TWI_SLVADDR</code>	Slave Mode Address Register
<code>TWI_SLVCTL</code>	Slave Mode Control Register
<code>TWI_SLVSTAT</code>	Slave Mode Status Register
<code>TWI_TXDATA16</code>	Tx Data Double-Byte Register
<code>TWI_TXDATA8</code>	Tx Data Single-Byte Register

SCL Clock Divider Register

During master mode operation, the `TWI_CLKDIV` holds values, which the TWI uses to create the high and low durations of the serial clock (SCL). The clock signal SCL is an output in master mode and an input in slave mode. The values in the `TWI_CLKDIV.CLKLO` and `TWI_CLKDIV.CLKHI` fields add up to the `CLKDIV` value the following equation.

$$\text{CLKDIV} = \text{TWI SCL period} / 10 \text{ MHz time reference}$$

Serial clock frequencies can vary from 400 KHz to less than 20 KHz. The resolution of the clock generated is 1/10 MHz or 100 ns. For example, for an SCL of 400 KHz (period = 1/400 KHz = 2500 ns) and an internal time reference of 10 MHz (period = 100 ns):

$$\text{CLKDIV} = 2500 \text{ ns} / 100 \text{ ns} = 25$$

For an SCL with a 30% duty cycle, use `TWI_CLKDIV.CLKLO` = 17 and `TWI_CLKDIV.CLKHI` = 8.

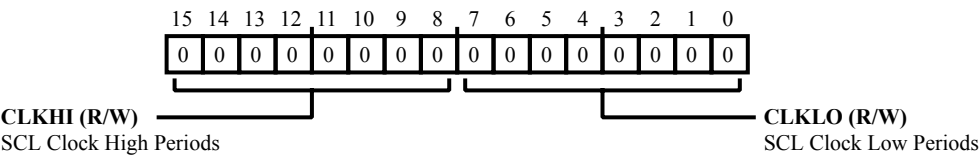


Figure 23-14: TWI_CLKDIV Register Diagram

Table 23-7: TWI_CLKDIV Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	CLKHI	SCL Clock High Periods. The <code>TWI_CLKDIV.CLKHI</code> specifies the number of 10 MHz time reference periods the serial clock (SCL) waits before a new clock low period begins, assuming a single master.
7:0 (R/W)	CLKLO	SCL Clock Low Periods. The <code>TWI_CLKDIV.CLKLO</code> specifies the number of internal time reference periods the serial clock (SCL) is held low.

Control Register

The `TWI_CTL` enables the TWI, establishes a relationship between the system clock (`SCLK0`) and the TWI controller's internally timed events, and enables SCCB compatibility.

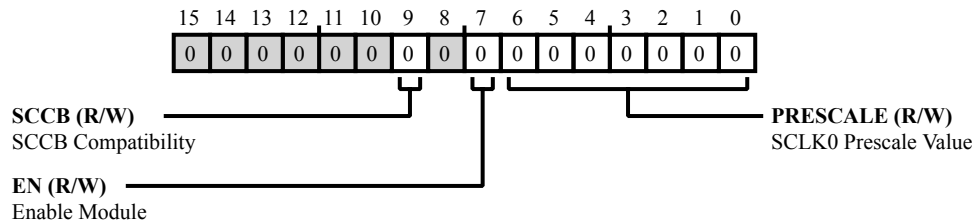


Figure 23-15: TWI_CTL Register Diagram

Table 23-8: TWI_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	SCCB	SCCB Compatibility. The <code>TWI_CTL.SCCB</code> enables SCCB compatible operation for the TWI. SCCB compatibility is an optional feature and should not be used in an I ² C bus system. When this feature is enabled, all slave asserted acknowledgement bits are ignored by this master. This feature is valid only during transfers where the TWI is mastering an SCCB bus. Slave mode transfers should be avoided when this feature is enabled because the TWI controller always generates an acknowledge in slave mode.
		0 Disable SCCB compatibility. When disabled, master transfers are not SCCB compatible.
		1 Enable SCCB compatibility. When enabled, master transfers are SCCB compatible. All slave-asserted acknowledgment bits are ignored by this master.
7 (R/W)	EN	Enable Module. The <code>TWI_CTL.EN</code> enables TWI controller operation for either master and/or slave mode of operation. It is recommended that this bit be set at the time <code>TWI_CTL.PRESCALE</code> is initialized and remain set. This method guarantees accurate operation of bus busy detection logic.
		0 Disable
		1 Enable

Table 23-8: TWI_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	PRESCALE	SCLK0 Prescale Value. The <code>TWI_CTL.PRESCALE</code> holds the pre-scaled value for the TWI internal time reference. This reference is derived from SCLK0 according to the formula: $\text{TWI_CTL.PRESCALE} = f_{\text{SCLK0}}/10\text{MHz}$ The <code>TWI_CTL.PRESCALE</code> specifies the number of system clock (SCLK0) periods used in the generation of one internal time reference. The value of <code>TWI_CTL.PRESCALE</code> must be set to create an internal time reference with a period of 10 MHz. It is represented as a 7-bit binary value.

FIFO Control Register

The `TWI_FIFOCTL` control bits affect only the FIFO and are not tied in any way with master or slave mode operation.

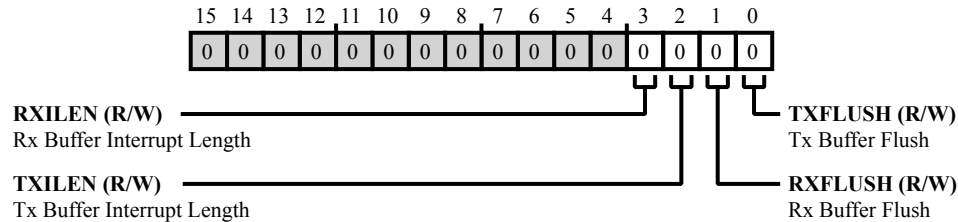


Figure 23-16: `TWI_FIFOCTL` Register Diagram

Table 23-9: `TWI_FIFOCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	RXILEN	Rx Buffer Interrupt Length. The <code>TWI_FIFOCTL.RXILEN</code> determines the rate at which receive buffer interrupts are to be generated. Interrupts may be generated with each byte received or after two bytes are received. Interrupt status is available in <code>TWI_FIFOSTAT.RXSTAT</code> .
		0 RXSERVI on 1 or 2 Bytes in FIFO
		1 RXSERVI on 2 Bytes in FIFO
2 (R/W)	TXILEN	Tx Buffer Interrupt Length. The <code>TWI_FIFOCTL.TXILEN</code> determines the rate at which transmit buffer interrupts are to be generated. Interrupts may be generated with each byte transmitted or after two bytes are transmitted. Interrupt status is available in <code>TWI_FIFOSTAT.TXSTAT</code> .
		0 TXSERVI on 1 Byte of FIFO Empty
		1 TXSERVI on 2 Bytes of FIFO Empty
1 (R/W)	RXFLUSH	Rx Buffer Flush. The <code>TWI_FIFOCTL.RXFLUSH</code> directs the TWI to flush the contents of the receive buffer and update <code>TWI_FIFOSTAT.RXSTAT</code> to indicate the buffer is empty. This state is held until this bit is cleared. During an active receive, the receive buffer in this state responds to the receive logic as if it is full.
		0 Normal Operation of Rx Buffer
		1 Flush Rx Buffer

Table 23-9: TWI_FIFOCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	TXFLUSH	Tx Buffer Flush. The <code>TWI_FIFOCTL.TXFLUSH</code> directs the TWI to flush the contents of the transmit buffer and update <code>TWI_FIFOSTAT.TXSTAT</code> to indicate the buffer is empty. This state is held until this bit is cleared. During an active transmit, the transmit buffer in this state responds to the transmit logic as if it is empty.
		0 Normal Operation of Tx Buffer
		1 Flush Tx Buffer

FIFO Status Register

The `TWI_FIFOSTAT` fields indicate the state of the FIFO buffers' receive and transmit contents. The FIFO buffers do not discriminate between master data and slave data. By using the status and control bits provided, the FIFO can be managed to allow simultaneous master and slave operation.

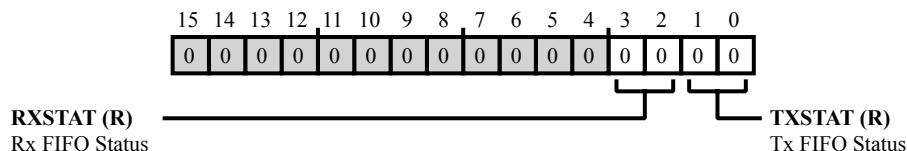


Figure 23-17: TWI_FIFOSTAT Register Diagram

Table 23-10: TWI_FIFOSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:2 (R/NW)	RXSTAT	Rx FIFO Status. The read-only <code>TWI_FIFOSTAT.RXSTAT</code> indicates the number of valid data bytes in the receive FIFO buffer. The status is updated with each FIFO buffer read using the peripheral data bus or write access by the receive shift register. Simultaneous accesses are allowed.
		0 Empty. The FIFO is empty.
		1 Contains 1 Byte. The FIFO contains one byte of data. A single byte peripheral read of the FIFO is allowed.
		2 Reserved
		3 Full. The FIFO is full and contains two bytes of data. Either a single or double byte peripheral read of the FIFO is allowed.
1:0 (R/NW)	TXSTAT	Tx FIFO Status. The read-only <code>TWI_FIFOSTAT.TXSTAT</code> field indicates the number of valid data bytes in the FIFO buffer. The status is updated with each FIFO buffer write using the peripheral data bus or read access by the transmit shift register. Simultaneous accesses are allowed.
		0 Empty. The FIFO is empty. Either a single or double byte peripheral write of the FIFO is allowed.
		1 Contains 1 Byte. The FIFO contains one byte of data. A single byte peripheral write of the FIFO is allowed.
		2 Reserved
		3 Full. The FIFO is full and contains two bytes of data.

Interrupt Mask Register

The **TWI_IMSK** enables interrupt sources to assert the interrupt output. Each mask bit corresponds with one interrupt request source bit in **TWI_ISTAT**. Reading and writing **TWI_IMSK** does not affect the contents of the **TWI_ISTAT**.

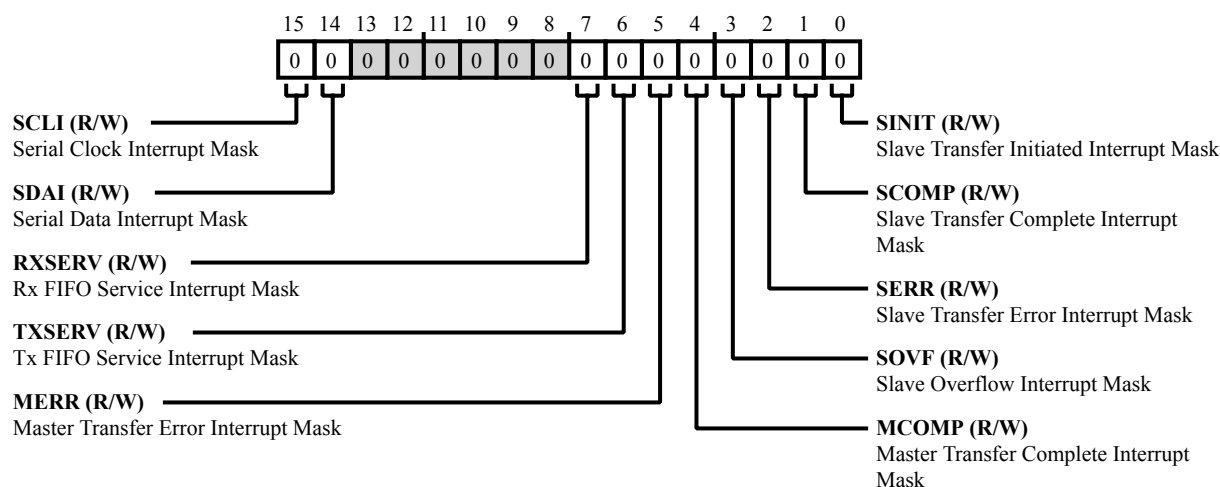


Figure 23-18: TWI_IMSK Register Diagram

Table 23-11: TWI_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SCLI	Serial Clock Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
14 (R/W)	SDAI	Serial Data Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
7 (R/W)	RXSERV	Rx FIFO Service Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
6 (R/W)	TXSERV	Tx FIFO Service Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt

Table 23-11: TWI_IMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	MERR	Master Transfer Error Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
4 (R/W)	MCOMP	Master Transfer Complete Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
3 (R/W)	SOVF	Slave Overflow Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
2 (R/W)	SERR	Slave Transfer Error Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
1 (R/W)	SCOMP	Slave Transfer Complete Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt
0 (R/W)	SINIT	Slave Transfer Initiated Interrupt Mask.
		0 Mask (Disable) Interrupt
		1 Unmask (Enable) Interrupt

Interrupt Status Register

The `TWI_ISTAT` contains information about functional areas requiring servicing. Many of the bits serve as an indicator to further read and service various status registers. After servicing the interrupt source associated with a bit, the user must clear that interrupt source bit by writing a 1 to it.

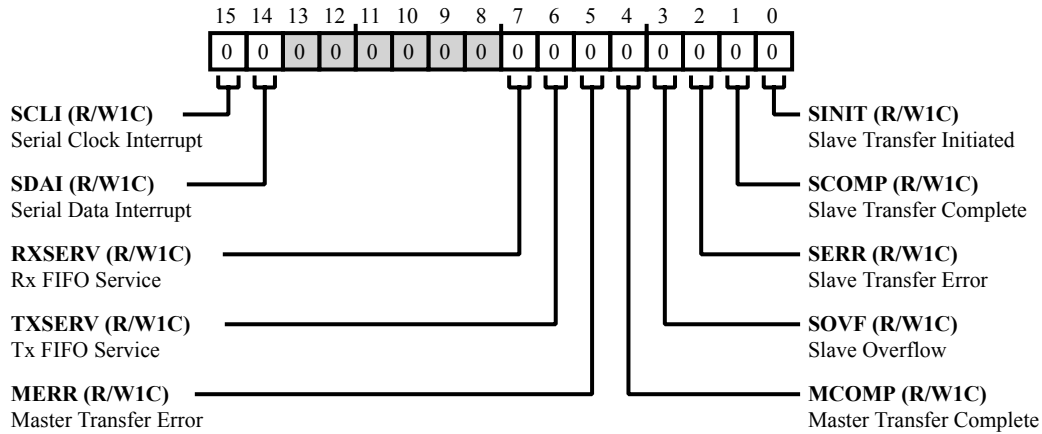


Figure 23-19: TWI_ISTAT Register Diagram

Table 23-12: TWI_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	SCLI	Serial Clock Interrupt. If the TWI is enabled (<code>TWI_CTL.EN</code>), SCLI is set on a high-to-low transition of the serial clock pin (<code>SCLx</code>). Normally, this bit is not required for I ² C bus transfers. It will be initially set on an I ² C transfer and does not require clearing.
		0 No Interrupt. No transition was detected on the <code>SCLx</code> pin.
		1 Interrupt Detected. A high-to-low transition was detected on the <code>SCLx</code> pin. This bit is W1C.
14 (R/W1C)	SDAI	Serial Data Interrupt. If the TWI is enabled (<code>TWI_CTL.EN</code>), SDAI is set on a high-to-low transition of the serial data pin (<code>SDAx</code>). Normally, this bit is not required for I ² C bus transfers. It will be initially set on an I ² C transfer and does not require clearing.
		0 No Interrupt. No transition was detected on the <code>SDAx</code> pin.
		1 Interrupt Detected. A high-to-low transition was detected on the <code>SDAx</code> pin. This bit is W1C.

Table 23-12: TWI_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W1C)	RXSERV	Rx FIFO Service. If <code>TWI_FIFOCTL.RXILEN = 0</code> , the <code>TWI_ISTAT.RXSERV</code> is set each time the <code>TWI_FIFOSTAT.RXSTAT</code> field is updated to either 01 or 11. If <code>TWI_FIFOCTL.RXILEN = 1</code> , the <code>TWI_ISTAT.RXSERV</code> is set each time <code>TWI_FIFOSTAT.RXSTAT</code> is updated to 11.
		0 No Interrupt. The FIFO does not require servicing, or the <code>TWI_FIFOSTAT.RXSTAT</code> field has not changed since this bit was last cleared.
		1 Interrupt Detected. The receive FIFO buffer has one or two 8-bit words of data available to be read.
6 (R/W1C)	TXSERV	Tx FIFO Service. If <code>TWI_FIFOCTL.TXILEN = 0</code> , the <code>TWI_ISTAT.TXSERV</code> is set each time the <code>TWI_FIFOSTAT.TXSTAT</code> field is updated to either 01 or 00. If <code>TWI_FIFOCTL.TXILEN = 1</code> , the <code>TWI_ISTAT.TXSERV</code> is set each time <code>TWI_FIFOSTAT.TXSTAT</code> is updated to 00.
		0 No Interrupt. FIFO does not require servicing, or the <code>TWI_FIFOSTAT.TXSTAT</code> field has not changed since this bit was last cleared.
		1 Interrupt Detected. The transmit FIFO buffer has one or two 8-bit locations available to be written.
5 (R/W1C)	MERR	Master Transfer Error. The <code>TWI_ISTAT.MERR</code> indicates that a master error has occurred. The conditions surrounding the error are indicated by the master status register (TWI_MSTRSTAT).
		0 No Interrupt
		1 Interrupt Detected
4 (R/W1C)	MCOMP	Master Transfer Complete. The <code>TWI_ISTAT.MCOMP</code> indicates that the initiated master transfer has completed. In the absence of a repeat start, the bus has been released.
		0 No Interrupt
		1 Interrupt Detected
3 (R/W1C)	SOVF	Slave Overflow. The <code>TWI_ISTAT.SOVF</code> indicates that the <code>TWI_ISTAT.SCOMP</code> bit was set at the time a subsequent transfer has acknowledged an address phase. The transfer continues, however, it may be difficult to delineate data of one transfer from another.
		0 No Interrupt
		1 Interrupt Detected

Table 23-12: TWI_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W1C)	SERR	Slave Transfer Error. The <code>TWI_ISTAT.SERR</code> indicates that a slave error has occurred. A restart or stop condition has occurred during the data receive phase of a transfer.
		0 No Interrupt
		1 Interrupt Detected
1 (R/W1C)	SCOMP	Slave Transfer Complete. The <code>TWI_ISTAT.SCOMP</code> indicates that the transfer is complete and either a stop, or a restart was detected.
		0 No Interrupt
		1 Interrupt Detected
0 (R/W1C)	SINIT	Slave Transfer Initiated. The <code>TWI_ISTAT.SINIT</code> indicates whether or not a slave transfer is in progress.
		0 No Interrupt. A transfer is not in progress, or an address match has not occurred since the last time this bit was cleared.
		1 Interrupt Detected. The slave has detected an address match, and a transfer has been initiated.

Master Mode Address Register

During the addressing phase of a transfer, the TWI controller, with its master enabled, transmits the contents of `TWI_MSTRADDR`. When programming this register, omit the read/write bit. That is, only the upper 7 bits that make up the slave address should be written to this register. For example, if the slave address is `b#1010000X`, where `X` is the read/write bit, the `TWI_MSTRADDR` is programmed with `b#1010000`, which corresponds to `0x50`. When sending out the address on the bus, the TWI controller appends the read/write bit as appropriate based on the state of the `TWI_MSTRCTL.DIR` bit.

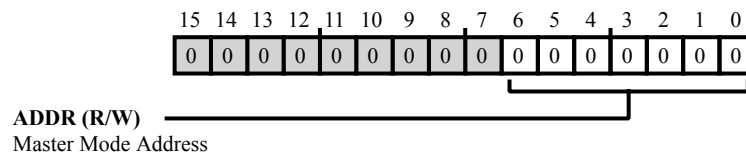


Figure 23-20: TWI_MSTRADDR Register Diagram

Table 23-13: TWI_MSTRADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	ADDR	Master Mode Address.

Master Mode Control Registers

The `TWI_MSTRCTL` controls the logic associated with master mode operation. Bits in this register do not affect slave mode operation and should not be modified to control slave mode functionality.

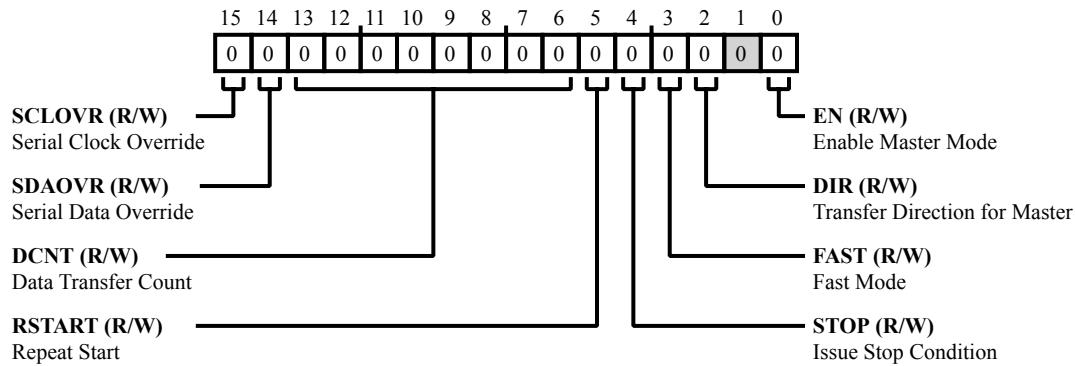


Figure 23-21: `TWI_MSTRCTL` Register Diagram

Table 23-14: `TWI_MSTRCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SCLOVR	Serial Clock Override. The <code>TWI_MSTRCTL.SCLOVR</code> provides direct control of the serial clock line when required. Normal master and slave mode operation should not require override operation. When <code>TWI_MSTRCTL.SCLOVR</code> is set, the TWI overrides normal serial clock output, driving it to an active 0 level and overriding all other logic. This state is held until this bit is cleared. When <code>TWI_MSTRCTL.SCLOVR</code> is cleared, the TWI permits normal serial clock operation under the control of master mode clock generation and slave mode clock stretching logic.
		0 Permit Normal SCL Operation
		1 Override Normal SCL Operation
14 (R/W)	SDAOVR	Serial Data Override. The <code>TWI_MSTRCTL.SDAOVR</code> provides direct control of the serial data line when required. Normal master and slave mode operation should not require override operation. When <code>TWI_MSTRCTL.SDAOVR</code> is set, the TWI overrides normal serial data operation under the control of the transmit shift register and acknowledge logic, driving serial data output to an active 0 level and overriding all other logic. This state is held until this bit is cleared. When <code>TWI_MSTRCTL.SDAOVR</code> is cleared, the TWI permits normal serial data operation.
		0 Permit Normal SDA Operation
		1 Override Normal SDA Operation

Table 23-14: TWI_MSTRCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration				
13:6 (R/W)	DCNT	Data Transfer Count. The <code>TWI_MSTRCTL.DCNT</code> indicates the number of data bytes to transfer. As each data word is transferred, the TWI decrements this counter. When <code>TWI_MSTRCTL.DCNT</code> decrements to 0, a stop condition is generated. Setting <code>TWI_MSTRCTL.DCNT</code> to 0xFF disables the counter. In this transfer mode, data continues to be transferred until it is concluded by setting the <code>TWI_MSTRCTL.STOP</code> bit. In the event a master transmit is aborted due to a slave data NAK, the value of <code>TWI_MSTRCTL.DCNT</code> equals the number of bytes not sent. The byte which was NAK'ed by the slave is counted as a sent byte.				
5 (R/W)	RSTART	Repeat Start. The <code>TWI_MSTRCTL.RSTART</code> enables the TWI to issue a repeat start condition at the conclusion of the current transfer (<code>TWI_MSTRCTL.DCNT =0</code>) and begin the next transfer. The current transfer concludes with updates to the appropriate status and interrupt bits. If errors occurred during the previous transfer, a repeat start does not occur. In the absence of any errors, master enable (<code>TWI_MSTRCTL.EN</code>) does not self clear on a repeat start. <table><tr><td>0</td><td>Disable Repeat Start</td></tr><tr><td>1</td><td>Enable Repeat Start</td></tr></table>	0	Disable Repeat Start	1	Enable Repeat Start
0	Disable Repeat Start					
1	Enable Repeat Start					
4 (R/W)	STOP	Issue Stop Condition. The <code>TWI_MSTRCTL.STOP</code> directs the TWI to issue a stop condition. The transfer concludes as soon as possible avoiding any error conditions (as if data transfer count had been reached). At that time, the <code>TWI_IMSK</code> is updated along with any associated status bits. <table><tr><td>0</td><td>Permit Normal Operation</td></tr><tr><td>1</td><td>Issue Stop</td></tr></table>	0	Permit Normal Operation	1	Issue Stop
0	Permit Normal Operation					
1	Issue Stop					
3 (R/W)	FAST	Fast Mode. The <code>TWI_MSTRCTL.FAST</code> selects whether the TWI operates in fast mode or standard mode. In fast mode, the TWI uses timing specifications for transfers at up to 400K bits/s. In standard mode, the TWI uses timing specifications for transfers at up to 100K bits/s. <table><tr><td>0</td><td>Select Standard Mode</td></tr><tr><td>1</td><td>Select Fast Mode</td></tr></table>	0	Select Standard Mode	1	Select Fast Mode
0	Select Standard Mode					
1	Select Fast Mode					
2 (R/W)	DIR	Transfer Direction for Master. The <code>TWI_MSTRCTL.DIR</code> selects the transfer direction for the TWI as master initiated receive or transmit. <table><tr><td>0</td><td>Master Transmit</td></tr><tr><td>1</td><td>Master Receive</td></tr></table>	0	Master Transmit	1	Master Receive
0	Master Transmit					
1	Master Receive					

Table 23-14: TWI_MSTRCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Enable Master Mode. The <code>TWI_MSTRCTL.EN</code> enables master mode functionality. A start condition is generated if the bus is idle. This bit self clears at the completion of a transfer (after <code>TWI_MSTRCTL.DCNT</code> decrements to zero), including transfers terminated due to errors. If disabled (=0) during operation, the transfer is aborted, and all logic associated with master mode transfers are reset. Serial data and serial clock (SDA, SCL) are no longer driven. Write-1-to-clear status bits are not affected.
		0 Disable
		1 Enable

Master Mode Status Register

The `TWI_MSTRSTAT` holds information during master mode transfers and at their conclusion. Generally, master mode status bits are not directly associated with the generation of interrupt requests, but these bits offer information on the current transfer. Slave mode operation does not affect master mode status bits.

Note that while `TWI_MSTRSTAT.SCLSEN` is set (this condition could be due to having no pull-up resistor on `TWI_SCL` or another agent is driving `TWI_SCL` low), the acknowledge bits (`TWI_MSTRSTAT.ANAK` and `TWI_MSTRSTAT.DNAK`) do not update. This result occurs because the acknowledge conditions are sampled during the high phase of `TWI_SCL`.

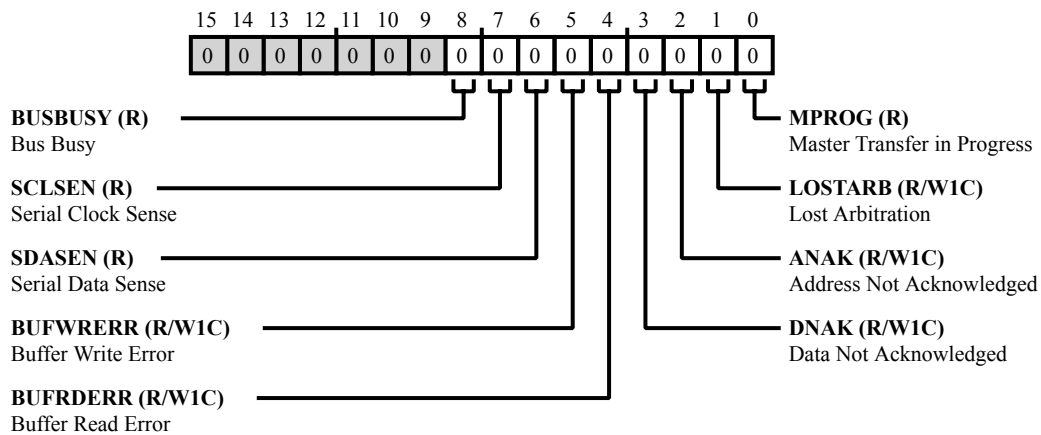


Figure 23-22: `TWI_MSTRSTAT` Register Diagram

Table 23-15: `TWI_MSTRSTAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	<code>BUSBUSY</code>	Bus Busy. The <code>TWI_MSTRSTAT.BUSBUSY</code> indicates whether the bus is currently busy or free. This indication is not limited to only this device but is for all devices. On a start condition, the setting of the register value is delayed due to the input filtering. On a stop condition the clearing of the register value occurs after <code>tBUF</code> .
		0 Bus Free. The bus is free. The clock and data bus signals have been inactive for the appropriate bus free time.
		1 Bus Busy. The bus is busy. Clock or data activity has been detected.

Table 23-15: TWI_MSTRSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	SCLSEN	Serial Clock Sense. The <code>TWI_MSTRSTAT.SCLSEN</code> indicates the active or inactive state of the serial clock. Use this status bit when direct sensing of the serial clock line is required. The register value is delayed due to the input filter (nominally 50 ns). Normal master and slave mode operation should not require this feature.
		0 SCL Inactive "One". An inactive "one" is being sensed on the serial clock.
		1 SCL Active "Zero". An active "zero" is being sensed on the serial clock. The source of the active driver is not known and can be internal or external.
6 (R/NW)	SDASEN	Serial Data Sense. The <code>TWI_MSTRSTAT.SDASEN</code> indicates the active or inactive status of the serial data. Use this status bit when direct sensing of the serial data line is required. The register value is delayed due to the input filter (nominally 50 ns). Normal master and slave mode operation should not require this feature.
		0 SDA Inactive "One". An inactive "one" is currently being sensed on the serial data line.
		1 SDA Active "Zero". An active "zero" is currently being sensed on the serial data line. The source of the active driver is not known and can be internal or external.
5 (R/W1C)	BUFWRERR	Buffer Write Error. The <code>TWI_MSTRSTAT.BUFWRERR</code> indicates whether the current master transfer was aborted due to a receive buffer write error. The receive buffer and receive shift register were both full at the same time. This bit is W1C.
		0 No Status
		1 Buffer Write Error
4 (R/W1C)	BUFRDERR	Buffer Read Error. The <code>TWI_MSTRSTAT.BUFRDERR</code> indicates whether the current master transfer was aborted due to the detection of a NAK during data transmission. This bit is W1C.
		0 No Status
		1 Buffer Read Error
3 (R/W1C)	DNAK	Data Not Acknowledged. The <code>TWI_MSTRSTAT.DNAK</code> indicates whether the current master transfer was aborted due to the detection of a NAK during data transmission. This bit is W1C.
		0 No Status
		1 Data NAK

Table 23-15: TWI_MSTRSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W1C)	ANAK	Address Not Acknowledged. The <code>TWI_MSTRSTAT.ANAK</code> indicates whether the current master transfer was aborted due to the detection of a NAK during the address phase of the transfer. This bit is W1C.
		0 No Status
		1 Address NAK
1 (R/W1C)	LOSTARB	Lost Arbitration. The <code>TWI_MSTRSTAT.LOSTARB</code> indicates whether the current transfer was aborted due to the loss of arbitration with another master. This bit is W1C.
		0 No Status
		1 Lost Arbitration
0 (R/NW)	MPROG	Master Transfer in Progress. The <code>TWI_MSTRSTAT.MPROG</code> indicates whether or not a master transfer is in progress. If clear (<code>TWI_MSTRSTAT.MPROG = 0</code>), currently no transfer is taking place. This can occur after a transfer is complete or while an enabled master is waiting for an idle bus.
		0 No Status
		1 Master Transfer in Progress

Rx Data Double-Byte Register

The `TWI_RXDATA16` holds a 16-bit data value read from the FIFO buffer. To reduce interrupt output rates and peripheral bus access times, a double byte receive data access can be performed. Two data bytes can be read, effectively emptying the receive FIFO buffer with a single access.

The data is read in little endian byte order, where byte 0 is the first byte received and byte 1 is the second byte received. With each access, the receive status (`TWI_FIFOSTAT.RXSTAT`) field is updated to indicate it is empty. If an access is performed while the FIFO buffer is not full, the read data is unknown and the existing FIFO buffer data and its status remains unchanged.

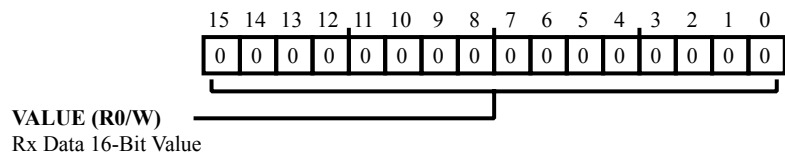


Figure 23-23: TWI_RXDATA16 Register Diagram

Table 23-16: TWI_RXDATA16 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R0/W)	VALUE	Rx Data 16-Bit Value.

Rx Data Single-Byte Register

The `TWI_RXDATA8` holds an 8-bit data value read from the FIFO buffer. Receive data is read from the corresponding receive buffer in a first-in first-out order. Although peripheral bus reads are 16 bits, a read access to `TWI_RXDATA8` accesses only one transmit data byte from the FIFO buffer. With each access, the receive status (`TWI_FIFOSTAT.RXSTAT`) field is updated. If an access is performed while the FIFO buffer is empty, the data is unknown and the FIFO buffer status remains indicating it is empty.

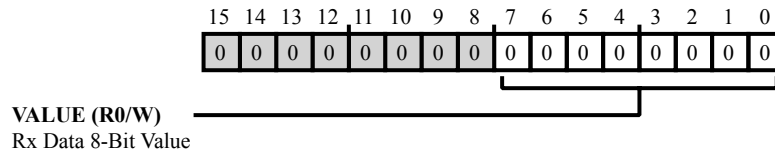


Figure 23-24: `TWI_RXDATA8` Register Diagram

Table 23-17: `TWI_RXDATA8` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R0/W)	VALUE	Rx Data 8-Bit Value.

Slave Mode Address Register

The `TWI_SLVADDR` holds the slave mode address, which is the valid address to which the slave-enabled TWI controller responds. The TWI controller compares this value with the received address during the addressing phase of a transfer.

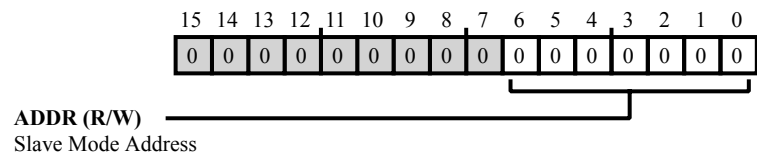


Figure 23-25: TWI_SLVADDR Register Diagram

Table 23-18: TWI_SLVADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	ADDR	Slave Mode Address.

Slave Mode Control Register

The `TWI_SLVCTL` controls the logic associated with slave mode operation. Settings in this register do not affect master mode operation and should not be modified to control master mode functionality.

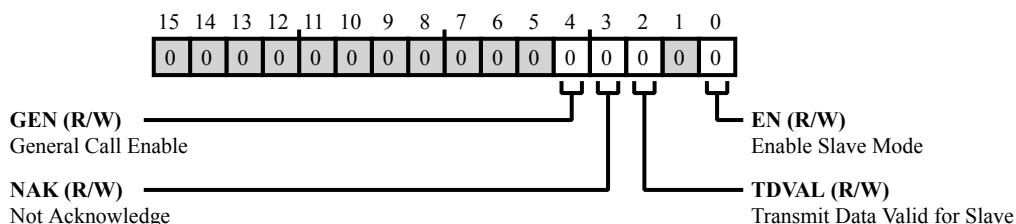


Figure 23-26: TWI_SLVCTL Register Diagram

Table 23-19: TWI_SLVCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	GEN	General Call Enable. The <code>TWI_SLVCTL.GEN</code> enables general call address matching. When enabled, a general call slave receive transfer is accepted. All status and interrupt source bits associated with transfers are updated. Note that general call address detection is available only when slave mode is enabled.
		0 Disable General Call Matching
		1 Enable General Call Matching
3 (R/W)	NAK	Not Acknowledge. The <code>TWI_SLVCTL.NAK</code> directs the TWI to generate a NAK (if set) or an ACK (if cleared) at the conclusion of data transfer for slave receive. For NAK, the slave is still considered to be addressed at the conclusion of transfer.
		0 Generate ACK
		1 Generate NAK
2 (R/W)	TDVAL	Transmit Data Valid for Slave. The <code>TWI_SLVCTL.TDVAL</code> selects whether the data in the transmit FIFO is available (valid) for slave transmission (<code>TWI_SLVCTL.TDVAL</code> set). If the FIFO data is not available (invalid) for slave transmission (<code>TWI_SLVCTL.TDVAL</code> cleared), the data in the transmit FIFO is for master mode transmits, and the data is not allowed to be used during a slave transmit; the transmit FIFO is treated as if it is empty.
		0 Data Invalid for Slave Tx
		1 Data Valid for Slave Tx

Table 23-19: TWI_SLVCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	Enable Slave Mode. The <code>TWI_SLVCTL.EN</code> enables slave operation. Enabling slave and master modes of operation concurrently is allowed. If disabled, no attempt is made to identify a valid address. If <code>TWI_SLVCTL.EN</code> is cleared during a valid transfer, clock stretching ceases, the serial data line is released, and the current byte is not acknowledged.
		0 Disable
		1 Enable

Slave Mode Status Register

During and at the conclusion of register slave mode transfers, the `TWI_SLVSTAT` holds information on the current transfer. Generally slave mode status bits are not associated with the generation of interrupt requests. Master mode operation does not affect slave mode status bits.

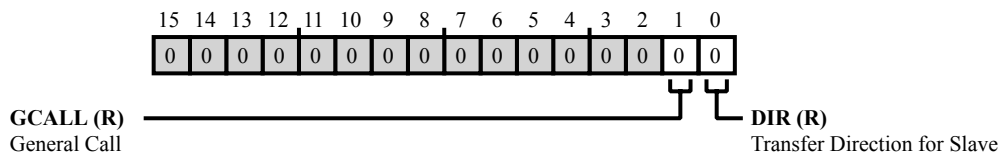


Figure 23-27: TWI_SLVSTAT Register Diagram

Table 23-20: TWI_SLVSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	GCALL	General Call. The <code>TWI_SLVSTAT.GCALL</code> indicates whether or not--at the time of addressing--the address was determined to be a general call. This bit self clears if slave mode is disabled (<code>TWI_SLVCTL.EN = 0</code>).
		0 Not a General Call Address
		1 General Call Address
0 (R/NW)	DIR	Transfer Direction for Slave. The <code>TWI_SLVSTAT.DIR</code> indicates whether--at the time of addressing--the transfer direction was determined to be slave transmit or receive. This bit self clears if slave mode is disabled (<code>TWI_SLVCTL.EN = 0</code>).
		0 Slave Receive
		1 Slave Transmit

Tx Data Double-Byte Register

The `TWI_TXDATA16` register holds a 16-bit data value written into the FIFO buffer. To reduce interrupt latency output rates and peripheral bus access times, a double byte transfer data access can be done. Two data bytes can be written, effectively filling the transmit FIFO buffer with a single access.

The data is written in little endian byte order, where byte 0 is the first byte to be transferred and byte 1 is the second byte to be transferred. With each access, the transmit status (`TWI_FIFOSTAT.TXSTAT`) field is updated. If an access is performed while the FIFO buffer is not empty, the write is ignored and the existing FIFO buffer data and its status remains unchanged. This register when read back returns zero.

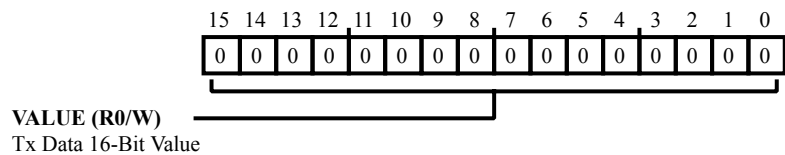


Figure 23-28: TWI_TXDATA16 Register Diagram

Table 23-21: TWI_TXDATA16 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R0/W)	VALUE	Tx Data 16-Bit Value.

Tx Data Single-Byte Register

The `TWI_TXDATA8` register holds an 8-bit data value written into the FIFO buffer. Transmit data is entered into the corresponding transmit buffer in a first-in first-out order. For 16-bit peripheral bus writes, a write access to this register adds only one transmit data byte to the FIFO buffer. With each access, the transmit status (`TWI_FIFOSTAT.TXSTAT`) field is updated. If an access is performed while the FIFO buffer is full, the write is ignored and the existing FIFO buffer data and its status remains unchanged. This register returns zero when read back.

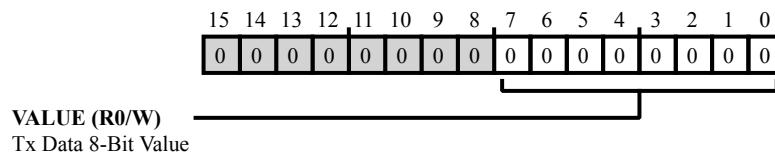


Figure 23-29: TWI_TXDATA8 Register Diagram

Table 23-22: TWI_TXDATA8 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R0/W)	VALUE	Tx Data 8-Bit Value.

24 Ethernet Media Access Controller (EMAC)

The EMAC peripheral in the processor enables network connectivity to applications through an Ethernet interface. The module is fully compliant to the following standards:

NOTE: On the processor, EMAC0 is 10/100/1000 BaseT-compliant and supports AVB (Audio Video Bridging) standards.

- Carrier Sense Multiple Access With Collision Detection (CSMA/CD) Access Method and Physical Layer Specifications, Standard 802.3-2005, Institute of Electrical and Electronics Engineers (IEEE).
- Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems, Standard 1588–2008, Institute of Electrical and Electronics Engineers (IEEE).
- Reduced Media Independent Interface Specification, Revision 1.2, RMII Consortium.
- Reduced Gigabit Media Independent Interface (RGMI), Revision 2.6, HP/Marvell.
- Timing and Synchronization for Time-Sensitive Applications in Bridged LANs, Standard 802.1AS-2011, Institute of Electrical and Electronics Engineers (IEEE).
- Forwarding and Queuing Enhancements for Time-Sensitive Streams, Standard 802.1Qav-2009, Institute of Electrical and Electronics Engineers (IEEE).
- The EMAC supports IEEE 803.3az-2010 standard for Energy Efficient Ethernet. This feature enables the Media Access Control (MAC) sublayer along with a family of physical layers to operate in the Low-Power Idle (LPI) mode.

NOTE: Copyright © 2010 Synopsys, Inc.; portions of this chapter are included with permission from Synopsys, Inc.

EMAC Features

The Ethernet features include the following:

- Supports 10/100 Mbps data transfer rates with external PHY interfaced through RMII
- Full-duplex and half-duplex support for Ethernet
- Dedicated DMA controller with independent read write channels

- Supports dual-buffer (ring) or linked-list (chained) descriptor chaining
- Direct interface with the system crossbar bus
- Provides support for CSMA/CD protocol for half-duplex operation
- IEEE 802.3x flow control for full-duplex and half-duplex
- Automatic network monitoring statistics with management counters
- Flexible address filtering options for uni-cast, multi-cast, and broadcast addresses
- Support for promiscuous mode in reception
- Supports IEEE 802.1Q VLAN tag detection
- VLAN tag-based frame filtering - Perfect match and hash-based filtering
- Supports programmable Inter-frame Gap (IFG)
- Checksum offload engine for checking IPv4 header checksum and TCP/UDP/ICMP checksum encapsulated in IPv4 or IPv6 datagrams
- Station management interface for PHY device configuration and management
- Automatic CRC and pad generation controllable on a per-frame basis
- CRC replacement, source address field insertion or replacement, and VLAN insertion, replacement, and deletion in transmitted frames with per-frame control
- Layer 3 and layer 4-based frame filtering - TCP or UDP over IPv4 or IPv6
- Supports Gigabit data transfer rates with external PHY interfaced through RGMII
- Ethernet frame timestamping as described in IEEE 1588–2002 and IEEE 1588–2008. The transmit or receive status of each frame include 64-bit timestamps
- Hardware assisted time stamping capable of up to 8-ns resolution
- Automatic detection of PTP messages through Ethernet, IPv4, and IPv6 packets
- Four programmable PPS outputs that physically represent PTP system time
- Auxiliary snapshot to timestamp external events
- Four auxiliary input pins available
- Two separate channels (channel 1 and channel 2) or queues for transmission and reception of time-sensitive traffic. Channel 0 is available by default and carries the legacy best-effort Ethernet traffic on the transmit side.
- IEEE 802.1-Qav specified credit-based shaper (CBS) algorithm for channel 1 and channel 2
- Slot number function to schedule the data fetching by DMA from the system memory for channel 1 and channel 2

- Separate DMA, Tx FIFO, and Rx FIFO (MTL) for each additional channel
- Programmable control to route received VLAN tagged non-AV packets to channels or queues
- Standard IEEE 802.3az-2010 for Energy Efficient Ethernet
- EMAC0 supports the following FIFO sizes: 2048 bytes for transmit FIFO and also for receive FIFO
- Supports MII/RMII/RGMII interfaces for external PHY interface

EMAC Functional Description

This section provides information on the function of Ethernet MAC peripheral.

Hardware for the Media Access Control protocol

This function allows applications to support TCP/IP based network communication. At the system end, the module supports direct connection with the system crossbar bus for memory or MMR transactions. It supports RMII (Reduced Media Independent Interface), RGMII (Reduced Gigabit Media Independent Interface), MII (Media Independent Interface), and SMI (Station Management Interface) for interfacing with the external PHY chip.

Dedicated DMA Controller with independent read/write channels

Performs both data and status transfers between the application and the media independent interfaces. Internal transmit and receive FIFOs are used to buffer and regulate the frames. Dedicated interrupt lines connect the EMAC interrupt sources to the System Event Controller (SEC).

MAC Management Counters (MMC) block

An extended set of registers that collect various statistics compliant with IEEE 802.3 definitions regarding the operation of the interface. The registers are updated for each new transmitted or received frame when the condition to update the counter is met. The EMAC provides a set of such counters, along with extended usage control.

PTP (Precision Time Protocol) engine

Provides hardware assistance for the implementation of the IEEE 1588 version 1 and version 2 standards, which allows time synchronization between systems.

Audio Video (AV) functionality

Enables transmission of time-sensitive traffic over bridged local area networks (LANs). The EMAC provides hardware support for IEEE 802.1-Qav specified credit-based shaper (CBS) algorithm. In addition, slot number function allows scheduling of fetching of data by DMA from system memory.

ADSP-2159x EMAC Register List

The Ethernet MAC (EMAC) module provides an Ethernet interface to the processor. The interface is compliant to IEEE Standard 802.3-2005. A set of registers govern EMAC operations. For more information on EMAC functionality, see the EMAC register descriptions.

Table 24-1: ADSP-2159x EMAC Register List

Name	Description
EMAC_ADDR0_HI	MAC Address 0 High Register
EMAC_ADDR0_LO	MAC Address 0 Low Register
EMAC_ADDR1_HI	MAC Address 1 High Register
EMAC_ADDR1_LO	MAC Address 1 Low Register
EMAC_DBG	Debug Register
EMAC_DMA0_BMODE	DMA SCB Bus Mode Register
EMAC_DMA0_BMSTAT	DMA SCB Status Register
EMAC_DMA0_BUSMODE	DMA Bus Mode Register
EMAC_DMA0_IEN	DMA Interrupt Enable Register
EMAC_DMA0_MISS_FRM	DMA Missed Frame Register
EMAC_DMA0_OPMODE	DMA Operation Mode Register
EMAC_DMA0_RXBUF_CUR	DMA Rx Buffer Current Register
EMAC_DMA0_RXDSC_ADDR	DMA Rx Descriptor List Address Register
EMAC_DMA0_RXDSC_CUR	DMA Rx Descriptor Current Register
EMAC_DMA0_RXIWDG	DMA Rx Interrupt Watch Dog Register
EMAC_DMA0_RXPOLL	DMA Rx Poll Demand register
EMAC_DMA0_STAT	DMA Status Register
EMAC_DMA0_TXBUF_CUR	DMA Tx Buffer Current Register
EMAC_DMA0_TXDSC_ADDR	DMA Tx Descriptor List Address Register
EMAC_DMA0_TXDSC_CUR	DMA Tx Descriptor Current Register
EMAC_DMA0_TXPOLL	DMA Tx Poll Demand Register
EMAC_DMA1_BUSMODE	DMA Bus Mode Register
EMAC_DMA1_CHCBSCTL	Channel 1 Credit Shaping Control Register
EMAC_DMA1_CHCBSSTAT	Channel 1 Average Traffic Transmitted Register
EMAC_DMA1_CHHIC	Channel 1 High Credit Value Register
EMAC_DMA1_CHISC	Channel 1 Idle Slope Credit Value Register
EMAC_DMA1_CHLOC	Channel 1 Low Credit Value Register

Table 24-1: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_DMA1_CHSFCS	Channel 1 Control Bits for Slot Function Register
EMAC_DMA1_CHSSC	Channel 1 Send Slope Credit Value Register
EMAC_DMA1_IEN	DMA Interrupt Enable Register
EMAC_DMA1_MISS_FRM	DMA Missed Frame Register
EMAC_DMA1_OPMODE	DMA Operation Mode Register
EMAC_DMA1_RXBUF_CUR	DMA Rx Buffer Current Register
EMAC_DMA1_RXDSC_ADDR	DMA Rx Descriptor List Address Register
EMAC_DMA1_RXDSC_CUR	DMA Rx Descriptor Current Register
EMAC_DMA1_RXIWDG	DMA Rx Interrupt Watch Dog Register
EMAC_DMA1_RXPOLL	DMA Rx Poll Demand Register
EMAC_DMA1_STAT	DMA Status Register
EMAC_DMA1_TXBUF_CUR	DMA Tx Buffer Current Register
EMAC_DMA1_TXDSC_ADDR	DMA Tx Descriptor List Address Register
EMAC_DMA1_TXDSC_CUR	DMA Tx Descriptor Current Register
EMAC_DMA1_TXPOLL	DMA Tx Poll Demand Register
EMAC_DMA2_BUSMODE	DMA Bus Mode Register
EMAC_DMA2_CHCBSCTL	Channel 2 Credit Shaping Control Register
EMAC_DMA2_CHCBSSTAT	Channel 2 Avg Traffic Transmitted Status Register
EMAC_DMA2_CHHIC	Channel 2 High Credit Value Register
EMAC_DMA2_CHISC	Channel 2 Idle Slope Credit Value Register
EMAC_DMA2_CHLOC	Channel 2 Low Credit Value Register
EMAC_DMA2_CHSFCS	Channel 2 Control Bits for Slot Function Register
EMAC_DMA2_CHSSC	Channel 2 Send Slope Credit Value Register
EMAC_DMA2_IEN	DMA Interrupt Enable Register
EMAC_DMA2_MISS_FRM	DMA Missed Frame Register
EMAC_DMA2_OPMODE	DMA Operation Mode Register
EMAC_DMA2_RXBUF_CUR	DMA Rx Buffer Current Register
EMAC_DMA2_RXDSC_ADDR	DMA Rx Descriptor List Address Register
EMAC_DMA2_RXDSC_CUR	DMA Rx Descriptor Current Register
EMAC_DMA2_RXIWDG	DMA Rx Interrupt Watch Dog Register
EMAC_DMA2_RXPOLL	DMA Rx Poll Demand register

Table 24-1: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_DMA2_STAT	DMA Status Register
EMAC_DMA2_TXBUF_CUR	DMA Tx Buffer Current Register
EMAC_DMA2_TXDSC_ADDR	DMA Tx Descriptor List Address Register
EMAC_DMA2_TXDSC_CUR	DMA Tx Descriptor Current Register
EMAC_DMA2_TXPOLL	DMA Tx Poll Demand Register
EMAC_FLOWCTL	FLow Control Register
EMAC_GIGE_CTLSTAT	RGMII Control and Status Register
EMAC_HASHTBL_HI	Hash Table High Register
EMAC_HASHTBL_LO	Hash Table Low Register
EMAC_IMSK	Interrupt Mask Register
EMAC_IPC_RXIMSK	MMC IPC Rx Interrupt Mask Register
EMAC_IPC_RXINT	MMC IPC Rx Interrupt Register
EMAC_ISTAT	Interrupt Status Register
EMAC_L3L4_CTL	Layer3 and Layer4 Control Register
EMAC_L3_ADDR0	Layer 3 Address0 Register
EMAC_L3_ADDR1	Layer 3 Address1 Register
EMAC_L3_ADDR2	Layer 3 Address2 Register
EMAC_L3_ADDR3	Layer 3 Address3 Register
EMAC_L4_ADDR	Layer 4 Address Register
EMAC_LPI_CTLSTAT	Low Power Idle Control and Status Register
EMAC_LPI_TMRCTL	Low Power Idle Timeout Register
EMAC_MACCFG	MAC Configuration Register
EMAC_MACFRMFILT	MAC Rx Frame Filter Register
EMAC_MAC_AVCTL	AV MAC Control Register
EMAC_MMC_CTL	MMC Control Register
EMAC_MMC_RXIMSK	MMC Rx Interrupt Mask Register
EMAC_MMC_RXINT	MMC Rx Interrupt Register
EMAC_MMC_TXIMSK	MMC TX Interrupt Mask Register
EMAC_MMC_TXINT	MMC Tx Interrupt Register
EMAC_RX1024TOMAX_GB	Rx 1024- to Max-Byte Frames (Good/Bad) Register
EMAC_RX128TO255_GB	Rx 128- to 255-Byte Frames (Good/Bad) Register

Table 24-1: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_RX256TO511_GB	Rx 256- to 511-Byte Frames (Good/Bad) Register
EMAC_RX512TO1023_GB	Rx 512- to 1023-Byte Frames (Good/Bad) Register
EMAC_RX64_GB	Rx 64-Byte Frames (Good/Bad) Register
EMAC_RX65TO127_GB	Rx 65- to 127-Byte Frames (Good/Bad) Register
EMAC_RXALIGN_ERR	Rx alignment Error Register
EMAC_RXBCASTFRM_G	Rx Broadcast Frames (Good) Register
EMAC_RXCRC_ERR	Rx CRC Error Register
EMAC_RXCTLFrm_G	Rx Good Control Frames Register
EMAC_RXFIFO_OVF	Rx FIFO Overflow Register
EMAC_RXFRMCNT_GB	Rx Frame Count (Good/Bad) Register
EMAC_RXICMP_ERR_FRM	Rx ICMP Error Frames Register
EMAC_RXICMP_ERR_OCT	Rx ICMP Error Octets Register
EMAC_RXICMP_GD_FRM	Rx ICMP Good Frames Register
EMAC_RXICMP_GD_OCT	Rx ICMP Good Octets Register
EMAC_RXIPV4_FRAG_FRM	Rx IPv4 Datagrams Fragmented Frames Register
EMAC_RXIPV4_FRAG_OCT	Rx IPv4 Datagrams Fragmented Octets Register
EMAC_RXIPV4_GD_FRM	Rx IPv4 Datagrams (Good) Register
EMAC_RXIPV4_GD_OCT	Rx IPv4 Datagrams Good Octets Register
EMAC_RXIPV4_HDR_ERR_FRM	Rx IPv4 Datagrams Header Errors Register
EMAC_RXIPV4_HDR_ERR_OCT	Rx IPv4 Datagrams Header Errors Register
EMAC_RXIPV4_NOPAY_FRM	Rx IPv4 Datagrams No Payload Frame Register
EMAC_RXIPV4_NOPAY_OCT	Rx IPv4 Datagrams No Payload Octets Register
EMAC_RXIPV4_UDSBL_FRM	Rx IPv4 UDP Disabled Frames Register
EMAC_RXIPV4_UDSBL_OCT	Rx IPv4 UDP Disabled Octets Register
EMAC_RXIPV6_GD_FRM	Rx IPv6 Datagrams Good Frames Register
EMAC_RXIPV6_GD_OCT	Rx IPv6 Good Octets Register
EMAC_RXIPV6_HDR_ERR_FRM	Rx IPv6 Datagrams Header Error Frames Register
EMAC_RXIPV6_HDR_ERR_OCT	Rx IPv6 Header Errors Register
EMAC_RXIPV6_NOPAY_FRM	Rx IPv6 Datagrams No Payload Frames Register
EMAC_RXIPV6_NOPAY_OCT	Rx IPv6 No Payload Octets Register
EMAC_RXJAB_ERR	Rx Jab Error Register

Table 24-1: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_RXLEN_ERR	Rx Length Error Register
EMAC_RXMCASTFRM_G	Rx Multicast Frames (Good) Register
EMAC_RXOCTCNT_G	Rx Octet Count (Good) Register
EMAC_RXOCTCNT_GB	Rx Octet Count (Good/Bad) Register
EMAC_RXOORTYPE	Rx Out Of Range Type Register
EMAC_RXOSIZE_G	Rx Oversize (Good) Register
EMAC_RXPAUSEFRM	Rx Pause Frames Register
EMAC_RXRCV_ERR	Rx Error Frames Received Register
EMAC_RXRUNT_ERR	Rx Runt Error Register
EMAC_RXTCP_ERR_FRM	Rx TCP Error Frames Register
EMAC_RXTCP_ERR_OCT	Rx TCP Error Octets Register
EMAC_RXTCP_GD_FRM	Rx TCP Good Frames Register
EMAC_RXTCP_GD_OCT	Rx TCP Good Octets Register
EMAC_RXUCASTFRM_G	Rx Unicast Frames (Good) Register
EMAC_RXUDP_ERR_FRM	Rx UDP Error Frames Register
EMAC_RXUDP_ERR_OCT	Rx UDP Error Octets Register
EMAC_RXUDP_GD_FRM	Rx UDP Good Frames Register
EMAC_RXUDP_GD_OCT	Rx UDP Good Octets Register
EMAC_RXUSIZE_G	Rx Undersize (Good) Register
EMAC_RXVLANFRM_GB	Rx VLAN Frames (Good/Bad) Register
EMAC_RXWDOG_ERR	Rx Watch Dog Error Register
EMAC_SMI_ADDR	SMI Address Register
EMAC_SMI_DATA	SMI Data Register
EMAC_TM_ADDEND	Time Stamp Addend Register
EMAC_TM_AUXSTMP_NSEC	Time Stamp Auxiliary TS Nano Seconds Register
EMAC_TM_AUXSTMP_SEC	Time Stamp Auxiliary TM Seconds Register
EMAC_TM_CTL	Time Stamp Control Register
EMAC_TM_HISEC	Time Stamp High Second Register
EMAC_TM_NSEC	Time Stamp Nanoseconds Register
EMAC_TM_NSECUPDT	Time Stamp Nanoseconds Update Register
EMAC_TM_PPS0INTVL	Time Stamp PPS Interval Register

Table 24-1: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_TM_PPS0NTGTM	Time Stamp Target Time Nanoseconds Register
EMAC_TM_PPS0TGTM	Time Stamp Target Time Seconds Register
EMAC_TM_PPS0WIDTH	PPS Width Register
EMAC_TM_PPS1INTVL	PPS 1 Interval Register
EMAC_TM_PPS1NTGTM	PPS 1 Target Time Nanoseconds Register
EMAC_TM_PPS1TGTM	PPS 1 Target Time Seconds Register
EMAC_TM_PPS1WIDTH	PPS 1 Width Register
EMAC_TM_PPS2INTVL	PPS 2 Interval Register
EMAC_TM_PPS2NTGTM	PPS 2 Target Time Nanoseconds Register
EMAC_TM_PPS2TGTM	PPS 2 Target Time Seconds Register
EMAC_TM_PPS2WIDTH	PPS 2 Width Register
EMAC_TM_PPS3INTVL	PPS 3 Interval Register
EMAC_TM_PPS3NTGTM	PPS 3 Target Time Nanoseconds Register
EMAC_TM_PPS3TGTM	PPS 3 Target Time Seconds Register
EMAC_TM_PPS3WIDTH	PPS 3 Width Register
EMAC_TM_PPSCTL	PPS Control Register
EMAC_TM_SEC	Time Stamp Low Seconds Register
EMAC_TM_SECUPDT	Time Stamp Seconds Update Register
EMAC_TM_STMPSTAT	Time Stamp Status Register
EMAC_TM_SUBSEC	Time Stamp Sub Second Increment Register
EMAC_TX1024TOMAX_GB	Tx 1024- to Max-Byte Frames (Good/Bad) Register
EMAC_TX128TO255_GB	Tx 128- to 255-Byte Frames (Good/Bad) Register
EMAC_TX256TO511_GB	Tx 256- to 511-Byte Frames (Good/Bad) Register
EMAC_TX512TO1023_GB	Tx 512- to 1023-Byte Frames (Good/Bad) Register
EMAC_TX64_GB	Tx 64-Byte Frames (Good/Bad) Register
EMAC_TX65TO127_GB	Tx 65- to 127-Byte Frames (Good/Bad) Register
EMAC_TXBCASTFRM_G	Tx Broadcast Frames (Good) Register
EMAC_TXBCASTFRM_GB	Tx Broadcast Frames (Good/Bad) Register
EMAC_TXCARR_ERR	Tx Carrier Error Register
EMAC_TXDEFERRED	Tx Deferred Register
EMAC_TXEXCESSCOL	Tx Excess Collision Register

Table 24-1: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_TXEXCESSDEF	Tx Excess Deferral Register
EMAC_TXFRMCNT_G	Tx Frame Count (Good) Register
EMAC_TXFRMCNT_GB	Tx Frame Count (Good/Bad) Register
EMAC_TXLATECOL	Tx Late Collision Register
EMAC_TXMCASTFRM_G	Tx Multicast Frames (Good) Register
EMAC_TXMCASTFRM_GB	Tx Multicast Frames (Good/Bad) Register
EMAC_TXMULTCOL_G	Tx Multiple Collision (Good) Register
EMAC_TXOCTCNT_G	Tx Octet Count (Good) Register
EMAC_TXOCTCNT_GB	Tx OCT Count (Good/Bad) Register
EMAC_TXOVRSIZE_G	Number of Tx Frames (Good) greater than maxsize
EMAC_TXPAUSEFRM	Tx Pause Frame Register
EMAC_TXSNGCOL_G	Tx Single Collision (Good) Register
EMAC_TXUCASTFRM_GB	Tx Unicast Frames (Good/Bad) Register
EMAC_TXUNDR_ERR	Tx Underflow Error Register
EMAC_TXVLANFRM_G	Tx VLAN Frames (Good) Register
EMAC_VLANTAG	VLAN Tag Register
EMAC_VLAN_HSHTBL	VLAN Hash Table Register
EMAC_VLAN_INCL	VLAN Tag Inclusion or Replacement Register
EMAC_WDOG_TIMEOUT	Watchdog Timeout Register

ADSP-2159x EMAC Interrupt List

Table 24-2: ADSP-2159x EMAC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
221	EMAC0_STAT	EMAC0 Status	None	
222	EMAC0_PWR	EMAC0 Power	None	
223	EMAC0_DMA0	EMAC0 DMA0	None	
224	EMAC0_DMA1	EMAC0 DMA1	None	
225	EMAC0_DMA2	EMAC0 DMA2	None	
226	EMAC0_MAC	EMAC0 MAC	None	
227	EMAC1_STAT	EMAC1 Status	None	

Table 24-2: ADSP-2159x EMAC Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
228	EMAC1_PWR	EMAC1 Power	None	
229	EMAC1_DMA0	EMAC1 DMA	None	
230	EMAC1_MAC	EMAC1 MAC	None	

ADSP-2159x EMAC Trigger List

Table 24-3: ADSP-2159x EMAC Trigger List Masters

Trigger ID	Name	Description	Sensitivity
30	EMAC0_STAT	EMAC0 Status	None
31	EMAC1_STAT	EMAC1 Status	None

Table 24-4: ADSP-2159x EMAC Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

EMAC Definitions

The following definitions are helpful for using the EMAC.

EMAC SCB

System Crossbar Interface of EMAC

EMAC DMA

DMA Controller of EMAC

EMAC MFL

MAC FIFO Layer inside EMAC

EMAC CORE

CORE layer inside EMAC which performs the actual Ethernet operations, including interface with PHY through the reduced media interface(s).

MMC

MAC Management Counter

SMI

Station Management Interface that controls PHY through MDIO and MDC signals.

RMII

Reduced Media Independent Interface

MAC

Media Access Control

PTP

Precision Time Protocol

EMAC Block Diagram and Interfaces

The *EMAC Simplified Block Diagram* illustrates the overall functional architecture of the Ethernet MAC peripheral. The EMAC module is comprised of four major layers: EMAC SCB, EMAC DMA, EMAC MFL, and EMAC CORE. Each of these layers (subblocks) is explained in depth in their respective sections in this chapter.

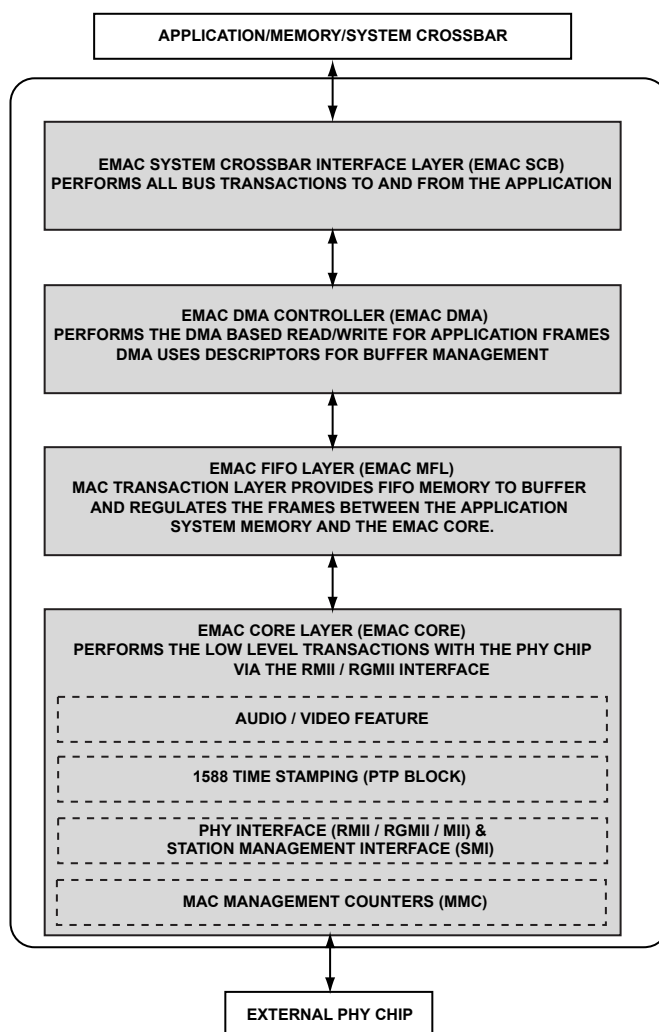


Figure 24-1: EMAC Simplified Block Diagram

A more comprehensive block diagram is shown in the *EMAC Complete Block Diagram*. It includes most of the important blocks inside the EMAC. The EMAC is connected to processor memory and the system crossbar through the System Crossbar Bus Interface (SCB) and System Peripheral Bus Interface (SPB). These connections are which are part of the SCB layer. The SPB interface is connected to all modules that require MMR programming.

The DMA controller performs application data transfer frame by frame, through well-defined descriptor structures. A FIFO layer acts as a buffer between the DMA controller and EMAC CORE.

The EMAC CORE is the most important block because it contains subblocks to support IEEE802.3 based communication with external network interfaces of 10/100/1000 Mbps speeds. It includes the PTP subblock, which assists applications requiring time synchronization; the AV Feature subblock, which enables transmission of time-sensitive traffic over bridged LANs; and a MMC subblock, which generates packet transfer statistics.

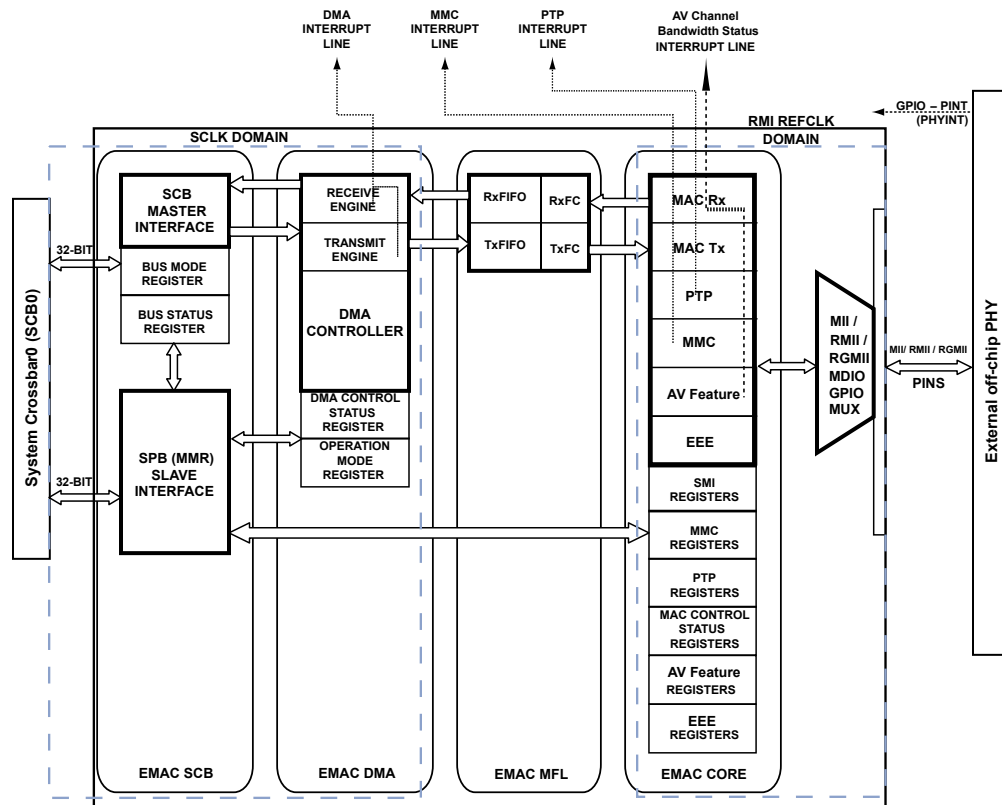


Figure 24-2: EMAC Complete Block Diagram

EMAC CORE Subblocks

The *Core Transmit Engine Subblocks* table summarizes the core transmit engine subblocks and their functions. Refer to the EMAC CORE section for further explanation of each of these subblocks.

Table 24-5: CORE Transmit Engine Subblocks

CORE Transmit Engine Sub Block	Function
Transmit Bus Interface	Interface to the FIFO.
Transmit Frame Controller	Appends Zero-PAD data, if required, for short frames. Appends CRC for frame checksum from the CRC generator.
Transmit Protocol Engine	Generates preamble and SFD, as per 802.3 protocol. Generates jam pattern in half-duplex mode, for collisions. Jabber timeout, for excessively large frames. Flow control for half-duplex mode (back pressure). Generates transmit frame status.
Transmit Scheduler	Maintains the inter-frame gap between two transmitted frames. Follows the truncated binary exponential back-off algorithm for half-duplex mode.

Table 24-5: CORE Transmit Engine Subblocks (Continued)

CORE Transmit Engine Sub Block	Function
Transmit CRC Generator	Generate CRC for the frame checksum field of the Ethernet frame.
Transmit Flow Control	Receives the pause frame, appends the calculated CRC, and sends the frame to the protocol engine module.
Transmit Checksum Offload Engine	Supports checksum calculation and insertion in the transmit path, for IPV4/TCP/UDP/ICMP packets.

The *Core Receive Engine Subblocks* table summarizes the core receive engine subblocks and their function. Refer to the EMAC CORE section for more information on each of these subblocks.

Table 24-6: Core Receive Engine Subblocks

CORE Receive Engine Subblock	Functionality Overview
Receive Protocol Engine	Strips the incoming preamble and SFD. Checks for correct length or type field. Performs internal loopback, if necessary. Generates receive status. Supports watchdog of received frames. Supports jumbo frames.
Receive CRC Module	Checks for CRC error, by comparing with FCS.
Receive Frame Controller Module	Packs incoming 8-bit input stream to 32-bit data internally. Performs frame filtering, for uni-cast, multi-cast, and broadcast frames. Attaches the calculated IP checksum input from checksum offload engine. Updates the receive status to bus interface.
Receive Flow Control Module	Detects the receiving pause frame and pauses the frame transmission for the delay specified within the received pause frame. Works in full duplex mode.
Receive IP Checksum Offload Engine	Calculates IPv4 header checksums and verify against the received IPv4 header checksums. Identifies a TCP, UDP, or ICMP payload in the received IP data-grams.
Receive Bus Interface Unit Module	Interface to the FIFO.
Address Filtering Module	Filters destination and source address based on uni-cast, multi-cast, and broadcast frames. Provides CRC hash filtering.

EMAC PHY Interface

The EMAC can interface to the PHY through the RMII interface standard. The *RMII Pins* table shows the RMII pins available in the EMAC, in terms of their generic names. Refer to the data sheet for exact pin names.

Table 24-7: RMII Pins

Sl. No.	Generic Signal Name (IEEE Standards)	RMII Pin Functionality
1.	TXD0	RMII transmit data pin D0 (di-bit lower)
2.	TXD1	RMII transmit data pin D1 (di-bit higher)
3.	RXD0	RMII receive data pin D0 (di-bit lower)
4.	RXD1	RMII receive data pin D1 (di-bit higher)
5.	RMII CLK	RMII common clock (for TX and RX), also called reference clock
6.	TXEN	RMII transmit enable pin (TX valid)
7.	CRS	RMII carrier sense / receive data valid
8.	MDC	Serial management clock driven by EMAC
9.	MDIO	Serial management bidirectional data

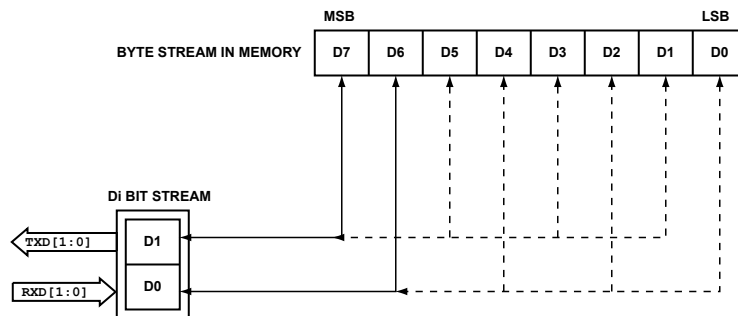


Figure 24-3: RMII Di-bit Data Transfer

Extended EMAC PHY Interface

The EMAC can interface to the PHY through the RGMII interface standard. The *RGMII Pins* table shows the RGMII pins available in the EMAC, in terms of their generic names. Refer to the data sheet for exact pin names.

Table 24-8: RGMII Pins

Sl. No.	Generic Signal Name (IEEE Standards)	RGMII Pin Functionality
1.	TXD3-0	RGMII transmit data pins D3-0
2.	RXD3-0	RGMII receive data pins D3-0
3.	TXCLK	RGMII transmit reference clock driven by EMAC
4.	RXCLK	RGMII receive reference clock driven by PHY
5.	TXCTL	RGMII transmit enable pin (TX valid)
6.	RXCTL	RGMII receive data valid

Table 24-8: RGMII Pins (Continued)

Sl. No.	Generic Signal Name (IEEE Standards)	RGMII Pin Functionality
7.	MDC	Serial management clock driven by EMAC
8.	MDIO	Serial management bidirectional data

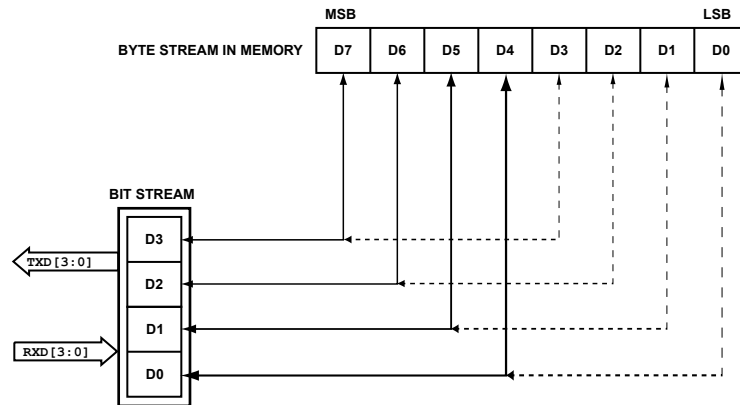


Figure 24-4: RGMII Data Bit Transfer

EMAC PHY Interface

The EMAC can interface to the PHY through the MII interface standard. The *MII Pins* table shows the MII pins available in the EMAC, in terms of their generic names. Refer to the data sheet for exact pin names.

Table 24-9: MII Pins

Sl. No.	Generic Signal Name (IEEE Standards)	MII Pin Functionality
1.	TXCLK	MII transmit clock
2.	TXD0-3	MII transmit data pins 0-3
3.	TXEN	MII transmit enable
4.	RXCLK	MII receive clock
5.	RXD0-3	MII receive data pins 0-3
6.	RXDV	MII receive data valid
7.	RXER	MII receive error
8.	CRS	MII carrier sense
9.	COL	MII collision detect

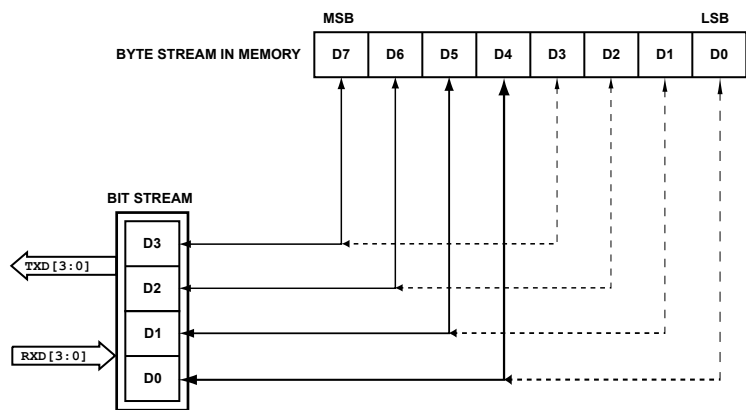


Figure 24-5: MII Data Bit Transfer

PHY Interface Selection

The EMAC0 supports both 10/100 Mbps data-transfer rates with external PHY interfaced through RMII and MII and 10/100/1000 Mbps data-transfer rates with external PHY interfaced through RGMII.

Select the external PHY interface for EMAC0 using the `PADS_PCFG0` register, as shown in the following table.

<code>PADS_PCFG0.EMACPHYISEL</code>	0 = MII Interface 1 = RGMII Interface 2 = RMII Interface
<code>PADS_PCFG0.EMACRESET</code>	0 = reset is asserted 1 = reset is deasserted To select PHY interface, set <code>PADS_PCFG0.EMACPHYISEL</code> bit as required and then set <code>PADS_PCFG0.EMACRESET</code> .

RGMII Board Design Recommendations

Use the following guidelines during when performing board design when using the RGMII interface.

MAC to PHY (Transmit)

The Ethernet MAC transmits data to the Ethernet PHY. The Ethernet MAC sends data with $tskewT$ (the timing of TXC at the MAC) that meets the RGMII specification ($tskewT = -500\text{ ps to }+500\text{ ps}$ skew window for transmitter to drive data). The RGMII specification requires that at the PHY end, $tskewR$ is sampled at 1.0 to 2.6 ns. According to the RGMII standard, clocks must be routed such that an additional trace delay of greater than 1.5 ns and less than 2 ns is added to the associated clock signal.

To meet this standard without adding trace delays, most of the PHYs in the industry already include delay logic that can compensate for this on-board delay. These PHY types can manage a $tskewR$ of $\pm 500\text{ ps}$ (the skew for TXC data sampling seen inside the PHY).

The PHY or the on-board delay must delay the clock signal by 1.5 to 2.0 ns so that $tskewR$ is sampled at 1.0 to 2.6 ns. The *MAC to PHY Delay Diagram (Transmitting Data)* figure shows where the delays must occur.

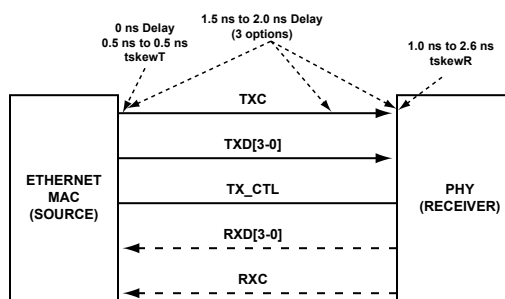


Figure 24-6: MAC to PHY Delay Diagram (Transmitting Data)

PHY to MAC (Receive)

The Ethernet MAC receives data from the Ethernet PHY. Just as in the transmit case, a trace delay of greater than 1.5 ns and less than 2 ns must be added to the associated clock signal, as required by the RGMII specification. Also similar to the transmit case, most of the PHYs in the industry already include delay logic that can compensate for the RXC clock as well.

As shown in the *MAC to PHY Delay Diagram (Receiving Data)* figure, a board trace or the PHY can be used to generate the required 1.5–2.0 ns delay to RXC and the 1.0–2.6 ns tskeW_R.

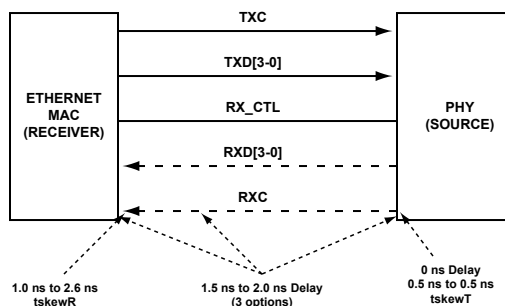


Figure 24-7: MAC to PHY Delay Diagram (Receiving Data)

There following are two options for board design.

- The on-board delay must delay the clock signal by 1.5 to 2.0 ns. In this case, no additional delay must be introduced by the PHY.
- Most of the PHYs in the industry already include a mode that can introduce a delay logic. If this mode of the PHY is used then the trace lengths on the board need to be matched exactly.

The second option is recommended since it is easier to implement the delay by using the mode in the PHY rather than during the board design. This is the approach followed in the ADI EZ-Kits.

NOTE: Refer to the product specific datasheet for exact processor timing.

For the trace length recommendations for the EMAC signals, please refer to the [SYNOPSIS?] datasheet.

For the exact requirements as recommended by the RGMII protocol, please refer to the RGMII specification.

Clock Sources

The Ethernet MAC is clocked internally from SCLK0. Check the processor data sheet for the valid frequency range of the appropriate SCLK0 signal for Ethernet operation.

Source a 50-MHz clock externally to operate the EMAC in RMII mode. This clock is the same for both transmit and receive. The MDC station management clock is derived from the SCLK0 and driven from the MAC to the PHY, when accessing any PHY registers. .

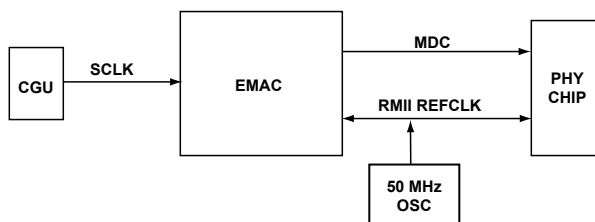


Figure 24-8: EMAC Clock Sources for RMII PHY interface

EMAC0 Clock Sources

EMAC0 supports RGMII, RMII and MII interfaces. The external PHY sources a 2.5 MHz or 25 MHz clock (for 10/100 or gigabit Ethernet respectively) to operate the EMAC RXCLK in RGMII mode. The RGMII TXCLK is driven from CLK07 of the CDU (Clock Distribution Unit) and needs to be configured to 125 MHz regardless of the EMAC0 speeds (10/100/1000 Mbit/s). The `EMAC_MACCFG.PS` and `EMAC_MACCFG.FES` bits are used to divide the clocks down.

The following tables show the clock sources for the RGMII, MII and RMII interfaces.

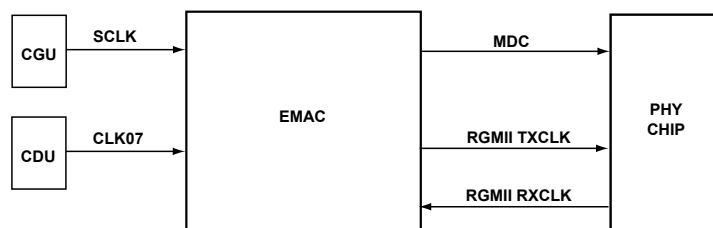


Figure 24-9: EMAC Clock Sources - RGMII PHY Interface

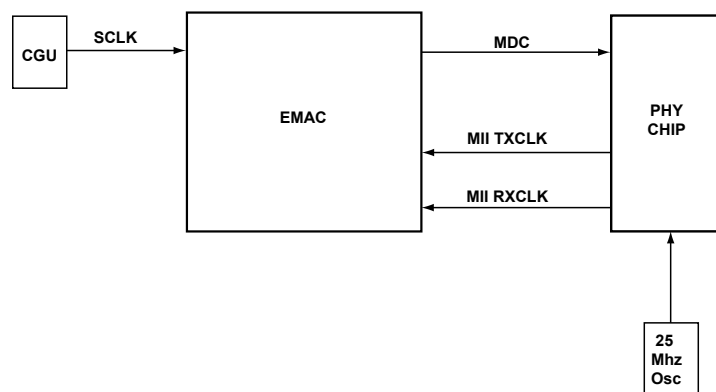


Figure 24-10: EMAC Clock Sources - MII PHY Interface

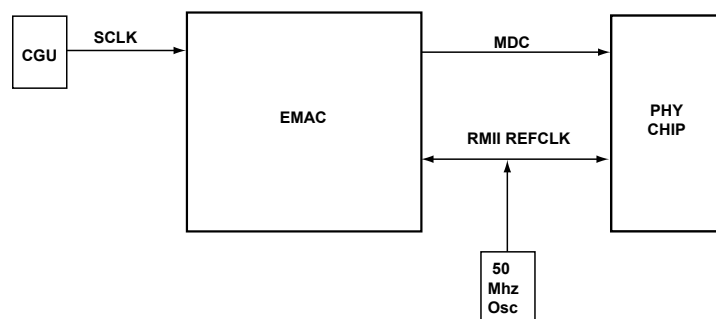


Figure 24-11: EMAC Clock Sources - RMII PHY Interface

EMAC Architectural Concepts

This section explains different architectural concepts relevant to EMAC peripheral, such as EMAC SCB, EMAC DMA, EMAC MFL, EMAC CORE, and others.

EMAC Feature Summary

The *EMAC Feature Summary* table provides a summary of the features that are available on EMAC0 .

Table 24-10: EMAC Feature Summary

Feature	Value (EMAC0)
Speed of Operation	10/100/1000 Mbps
EMAC_VER.UVER hardcoded value	0x10
PHY Interface	RGMII or RMII or MII
Receive FIFO Size (in Bytes)	2048
Transmit FIFO Size (in Bytes)	2048
Energy Efficient Ethernet (EEE)	Yes
PTP (IEEE 1588)	Yes

Table 24-10: EMAC Feature Summary (Continued)

Feature	Value (EMAC0)
AV Feature	Yes
No of TX Channels	3
No of RX Channels	3

EMAC System Crossbar Interface (EMAC SCB)

The EMAC SCB bus interface provides the bus connectivity to support highly effective throughput of data traffic. System bus use is maximized by allowing simultaneous read and write transfers initiated from different DMA channels. The EMAC controller connects directly to the SCB0 crossbar. The following interfaces are available with the design.

- A 32-bit SCB master interface for reading and writing to and from the application memory.
- A 32-bit SPB slave interface for register programming.

Refer to the “System Crossbars (SCB)” chapter for more information on how the crossbar operates. This chapter details only the EMAC-specific information.

Table 24-11: EMAC-SCB Interface Data Transfer Specifications with Crossbar

Specification Term	Comments
1 beat in SCB	SINGLE burst
BLN4 bursts	4 beats in SCB
BLN8 bursts	8 beats in SCB
BLN16 bursts	16 beats in SCB
Bus size	32-bit fixed bus size; equals 1 beat
INCR bursts	Incrementing Bursts
INCR ALIGNED bursts	Incrementing aligned bursts
UNDEF bursts	Undefined burst length
PBL	Programmable Burst Length for DMA

The *EMAC DMA Read/Write channels with System Crossbar* figure shows DMA write channel and read channel datapaths and their connection to the system crossbar.

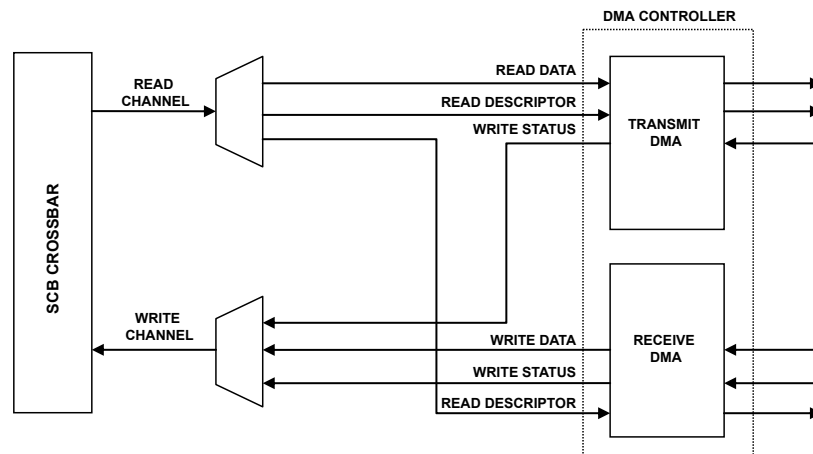


Figure 24-12: EMAC DMA Read/Write channels with System Crossbar

NOTE: Transmit descriptor read and receive descriptor write-back (status update) operations can occur simultaneously. However, transmit descriptor read and write-back operations cannot occur simultaneously. Transmit DMA (or receive DMA) does not initiate the next transfer unless the previous one is complete.

Priority of SCB Requests

The descriptor transfers have higher priority than the data transfers. For example, if there are two bus requests, such as a receive descriptor read and a transmit data read, the receive descriptor read has a higher priority. The next receive data write (subsequent to the receive descriptor read) does not depend on the completion of the transmit data-read transfer.

If there are requests for descriptor reads from both DMA channels, they are serviced based on a first-come first-serve. Receive DMA has higher priority if the descriptor-read requests are generated from both the DMA channels in the same clock cycle. Similarly, in the write channel, descriptor writes from DMA have higher priority than the data-write transfers for the receive DMA.

SCB Interface Programming Options

The SCB bus interface supports the following programmable options for the EMAC module. These options are available using the `EMAC_DMA0_BMODE` register with the `EMAC_DMA0_BUSMODE` register. These programming options apply to DMA1 and DMA2 as well.

- **Outstanding transactions.** The EMAC-SCB supports up to four outstanding read/write requests on the SCB bus. Software can control these requests by programming the `EMAC_DMA0_BMODE.WROSRLMT` and `EMAC_DMA0_BMODE.RDOSRLMT` bits. Maximum outstanding requests = $\text{EMAC_DMA0_BMODE.WROSRLMT} + 1$ (or) $\text{EMAC_DMA0_BMODE.RDOSRLMT} + 1$.
- **Allowed burst sizes.** The allowed burst sizes are 4 (`EMAC_DMA0_BMODE.BLEN4`), 8 (`EMAC_DMA0_BMODE.BLEN8`), 16 (`EMAC_DMA0_BMODE.BLEN16`) and the SINGLE burst. The EMAC-SCB uses only those burst sizes configured by the program (through the `EMAC_DMA0_BMODE` register) for data transfer through the SCB bus. However, SINGLE burst is available by default, when the `EMAC_DMA0_BMODE.UNDEF` bit is cleared. Data transfers are restricted to the maximum burst size from this list of programmed burst sizes.

- **Burst splitting and burst selection.** The EMAC-SCB splits the DMA requests into multiple bursts on the SCB system bus. Splitting is based on DMA count and software controllable burst enable bits (shown in the allowed burst sizes) as well as burst types (INCR and INCR_ALIGNED). Burst types are also controllable through the software. SINGLE burst is enabled when the `EMAC_DMA0_BMODE.UNDEF` bit is not set. Burst length select priority is in the sequence: UNDEF, 16, 8, and 4.
- **INCR burst type**
 - If the `EMAC_DMA0_BMODE.UNDEF` bit is set, the EMAC-SCB always chooses the maximum allowed burst length based on the `EMAC_DMA0_BMODE.BLEN16`, `EMAC_DMA0_BMODE.BLEN8`, `EMAC_DMA0_BMODE.BLEN4` bits. When the DMA requests are not multiples of the maximum allowed burst length, the SCB can choose a burst-length of any value less than the maximum enabled. (All lesser burst-length enables are redundant). For example, when length bits are enabled and the DMA requests a burst of 42 beats, the SCB splits it into three bursts of 16, 16 and 10 beats respectively.
 - If `EMAC_DMA0_BMODE.UNDEF` is not enabled, then the burst length is based on the priority of the enabled bits in the following order `EMAC_DMA0_BMODE.BLEN16`, `EMAC_DMA0_BMODE.BLEN8`, `EMAC_DMA0_BMODE.BLEN4`. When the DMA requests a burst transfer, the SCB interface splits the requested bursts into multiple transfers using only the enabled burst lengths. This splitting can occur when the requested burst is not a multiple of the maximum enabled burst. If it cannot choose any of the enabled burst lengths, then it selects the burst length as 1.

For example, the DMA requests a burst transfer of 42 beats, the SCB interface splits it into multiple bursts of size 16, 16, 8, 1 and 1 beats respectively. (In this case, the allowed burst sizes are enabled and the sequence is in decreasing burst sizes).

- **INCR_ALIGNED burst type.** When the address-aligned burst-type is enabled (`EMAC_DMA0_BMODE.AAL`), the SCB interface splits the DMA requested bursts. The "INCR Burst Type" section explains burst splitting conditions further. Each burst-size aligns to the least significant bits of the start address. The SCB interface initially generates smaller bursts so that the remaining transfers move with the maximum (enabled) fixed burst lengths.

For example, in the same setting as explained earlier for `EMAC_DMA0_BMODE.UNDEF` set, the DMA requests a burst size of 42 beats at the start address of 0x000003A4. (`EMAC_DMA0_BMODE.BLEN16`, `EMAC_DMA0_BMODE.BLEN8`, and `EMAC_DMA0_BMODE.BLEN4` are enabled). The SCB starts the first transfer with size 3 such that the address of the next burst is aligned (0x000003B0) for a burst of 16. Therefore, the sequence of bursts is 3, 16, 16, and 7, respectively.

When `EMAC_DMA0_BMODE.UNDEF` is not set, then (having a start address of 0x000003A4 with 42 beats), the sequence of burst transfers is 1, 1, 1, 16, 16, 4, and 3 respectively. The sequence of smaller bursts at the beginning is used to align the address to the next higher enabled burst-lengths programmed in the register.

- **Burst operations for DMA transactions.** The `EMAC_DMA0_BMODE.PBL` (programmable burst length) field indicates the maximum number of beats to transfer in one DMA transaction. This value is also the maximum used in a single block read/write. It is shown in the following table.

- For example, if `EMAC_DMA0_BUSMODE.PBL=32` and if `EMAC_DMA0_BMODE.BLEN16` is enabled, the DMA automatically splits 32 bursts in to 2 x 16 bursts. If `EMAC_DMA0_BUSMODE.PBL=8`, and if `EMAC_DMA0_BMODE.BLEN16` and `EMAC_DMA0_BMODE.BLEN8` are enabled, the maximum burst is limited to `EMAC_DMA0_BMODE.BLEN8`. If the `EMAC_DMA0_BUSMODE.PBL8` bit is set, the programmed `EMAC_DMA0_BUSMODE.PBL` value is multiplied by 8 times internally. However, the result cannot be more than the maximum limits specified.
- Set the `EMAC_DMA0_BUSMODE.USP` bit to make the receive DMA burst length configuration independent of the transmit DMA configuration. When this bit is set, the EMAC uses the `EMAC_DMA0_BUSMODE.RPBL` bits to define the burst length of receive DMA. If the `EMAC_DMA0_BUSMODE.USP` bit is not set, the `EMAC_DMA0_BUSMODE.RPBL` bits are used for both transmit and receive. Programs must ensure that the PBL maximum limit is not violated.
- The receive and transmit descriptors are always accessed in the maximum burst-size for the 16-bytes to be read (PBL-max limit is $(\text{TX or RX FIFO size}/2)/4$ words. (PBL maximum for transmit and receive limits burst-size).

Table 24-12: DMA PBL Max Limits

Burst Limit Max Term	Definition
PBL-maximum limit	$(\text{FIFO size}/2)/4$ words
PBL-maximum limit (transmit)	$2048 \text{ bytes}/2 / 4 = 256$ words
PBL-maximum limit (receive)	$2048 \text{ bytes}/2 / 4 = 256$ words

DMA Bursts Using the SCB Interface

The transmit DMA initiates a data transfer when sufficient space to accommodate the configured burst is available in the transmit FIFO. Or, the transmit DMA initiates a data transfer when the number of bytes until the end of frame is less than the configured burst-length. The DMA indicates the start address and the number of transfers required to the SCB master interface. When the SCB interface is configured for fixed-length burst, then it transfers data using the best combination of INCR4/8/16 and 1 beat transaction.

The receive DMA initiates a data transfer when sufficient data to accommodate the configured burst is available in the MTL receive FIFO. Or, the receive DMA initiates a data transfer when the end of frame is detected in the receive FIFO. For example, when the amount is less than the configured burst-length. The DMA indicates the start address and the number of transfers required to the SCB master interface. When the SCB interface is configured for fixed-length burst, then it transfers data using the best combination of INCR 4, 8, 16 or 1 beat transaction. If the end-of frame is reached before the fixed-burst ends on the SCB interface, then dummy transfers are performed to complete the fixed-burst. Otherwise (if `EMAC_DMA0_BUSMODE.FB` is reset), it transfers data using INCR (undefined length) transactions.

When the SCB interface is configured for address-aligned beats, both DMA engines ensure that the first burst transfer is less than or equal to the configured PBL size. (The address-aligned beats configuration uses the `EMAC_DMA0_BUSMODE.AAL` bit). Therefore, all subsequent beats start at an address that is aligned to the configured PBL.

SCB Bus Transaction Status

The SCB uses the `EMAC_DMA0_BMSTAT.BUSRD` and `EMAC_DMA0_BMSTAT.BUSWR` bits to indicate whether the channel is active or not.

Fatal Bus Error

The EMAC SCB asserts the error interrupt (`EMAC_DMA0_STAT.FBI`) when the corresponding fatal bus error interrupt is enabled in the DMA interrupt enable register. The application must reset the core to restart the DMA.

DMA Controller (EMAC DMA)

The EMAC has a built-in DMA controller that performs reads and writes of application data and descriptors through the SCB master interface.

The DMA controller has independent transmit and receive engines, and a CSR (control and status register) space. The transmit engine transfers data from system memory to a FIFO, while the receive engine transfers data from the FIFO to the system memory. The controller uses a descriptor chain-based transfer mechanism to move data efficiently from source to destination with minimal processor core intervention. The DMA is specially designed for packet-oriented data transfers such as Ethernet frames. The controller can be programmed to interrupt the application for situations such as frame transmit and receive transfer completion, and other normal or abnormal conditions.

The DMA and the application device driver communicate through two internal data structures:

1. DMA control and status registers (CSR).
2. Data buffers and descriptor lists. Descriptor lists operate in ring mode and chain mode, as shown in the *EMAC DMA Descriptor Models* figure.

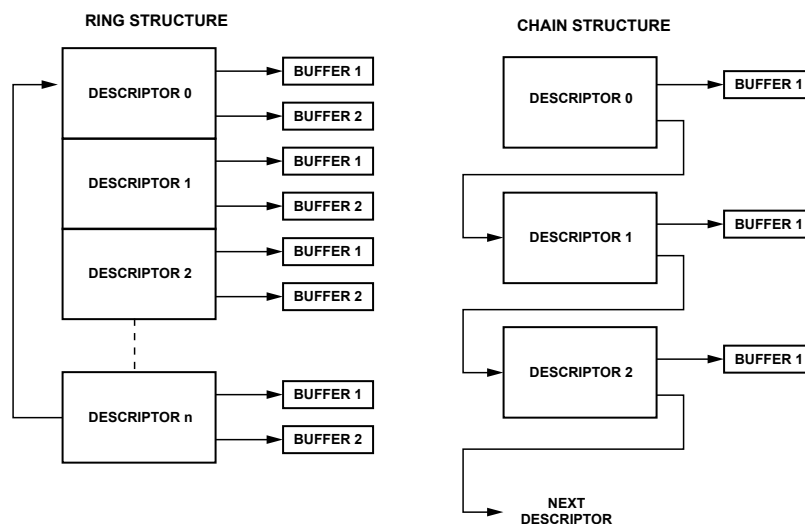


Figure 24-13: EMAC DMA Descriptor Models

Descriptors that reside in the application memory act as pointers to receive and transmit buffers. Descriptors have the following extra attributes.

- There are two descriptor lists, one for receive, and one for transmit. The base address of each list is written into the address registers of the receive and transmit descriptor lists respectively.
- A descriptor list is forward linked (either implicitly or explicitly). The last descriptor can point back to the first entry to create a ring structure.
- Explicit chaining of descriptors is accomplished by setting the second address chained in both receive and transmit descriptors.
- The descriptor lists reside in the address space of the application memory.
- Each descriptor can point to a maximum of two buffers. This attribute enables two buffers, physically addressed, rather than contiguous buffers in memory.

A data buffer resides in the physical memory space of the application. It consists of an entire frame or part of a frame, but cannot exceed a single frame. Buffers can contain only data. The descriptor maintains buffer status. *Data chaining* refers to frames that span multiple data buffers. However, a single descriptor cannot span multiple frames. The DMA skips to the next frame buffer when the end-of-frame is detected. Data chaining is enabled or disabled.

NOTE: It is possible to define a skip length (in terms of $N \times 32$ -bit words) between two subsequent descriptors, when using ring mode. Program the `EMAC_DMA0_BUSMODE.DSL/EMAC_DMA1_BUSMODE.DSL/EMAC_DMA2_BUSMODE.DSL` field to enable this attribute. With this option available, programs are not always restricted to a contiguous memory location in ring mode.

DMA Related Registers

The *Summary of DMA Related Registers* table provides a summary of DMA registers relative to their function. Refer to the “Register Descriptions” sections for complete bit descriptions of each of these registers.

Table 24-13: Summary of DMA Related Registers

Register Name	Description
Bus Mode ^{*1}	Establishes the bus operating modes for the DMA based on the SCB master interface.
Transmit Poll Demand	Enables the transmit DMA to check whether the DMA owns the current descriptor. The transmit poll demand command wakes up the TxDMA when it is in suspend mode. The TxDMA can go into suspend mode because of an underflow error in a transmitted frame or because of the unavailability of descriptors owned by transmit DMA. Issue this command anytime and the TxDMA resets this command once it starts refetching the current descriptor from host memory.
Receive Poll Demand	Enables the receive DMA to check for new descriptors. This command wakes up the RxDMA from the SUSPEND state. The RxDMA can go into SUSPEND state only because of the unavailability of descriptors owned by it.
Receive Descriptor List Address	Points to the start of the receive descriptor list. The descriptor lists reside in the application memory space and must be word-aligned (32-bit data bus). The DMA internally converts the descriptor list to a bus width aligned address by making the corresponding LSBs low.

Table 24-13: Summary of DMA Related Registers (Continued)

Register Name	Description
Transmit Descriptor List Address	Points to the start of the transmit descriptor list. The descriptor lists reside in the application memory space and must be word-aligned (for 32-bit data bus). The DMA internally converts it to bus width aligned address by making the corresponding LSB to low.
DMA Status	Contains all the status bits that the DMA reports to the application. The software driver reads this register during an interrupt service routine or during polling. Most of the fields in this register interrupt the host.
Operation Mode	Establishes the transmit and receive operating modes and commands. The operation mode register is the last control register written as part of DMA initialization.
Interrupt Enable	Enables the interrupts reported by DMA status register. After a hardware or software reset, all interrupts are disabled.
Missed Frame and Buffer Overflow Counter	The DMA maintains two counters to track the number of missed frames during reception. This register reports the current value of the counter, which is used for diagnostic purposes.
Receive Interrupt Watchdog Timer	When written with non-zero value, enables the watchdog timer for receive interrupt (RI) in the DMA status register.
SCB Bus Mode	Controls the SCB interface master behavior. It controls the burst splitting and the number of outstanding requests.
SCB Status	Provides the active status of the SCB interface read and write channels.
Current Host Transmit Descriptor	Points to the start address of the current transmit descriptor read by the DMA.
Current Host Receive Descriptor	Points to the start address of the current receive descriptor read by the DMA.
Current Host Transmit Buffer Address	Points to the current transmit buffer address the DMA is reading.
Current Host Receive Buffer Address	Points to the current receive buffer address the DMA is reading.

*1 Do not write to the `EMAC_DMA0_BUSMODE.DSL/EMAC_DMA1_BUSMODE.DSL/EMAC_DMA2_BUSMODE.DSL` registers until the first write updates. Otherwise, the second write operation does not update properly. For correct operation, the delay between two writes to the same register location must be at least 8 cycles of 50 MHz RMII REFCLK.

Table 24-14: DMA Registers with Consecutive Writes

Registers with Implications for Consecutive Writes
DMA Bus Mode

DMA Descriptors

The DMA module in the Ethernet subsystem transfers data based on a linked list of descriptors. The descriptor addresses must be aligned to the 32-bit bus width. The descriptors can be either 4 x 32-bit words (16 bytes) or 8 x 32-bit words (32 bytes). Configure the controller for the appropriate word length using the DMA bus mode register. The descriptor words are numbered from 0 to 7 for both the transmit and receive engine.

Typical factors for deciding the descriptor word size are as follows:

- When the time stamping or receive checksum engines are not enabled, the extended descriptors are not required. The software can use descriptors with the default size of 16 bytes (4 words).
- When the time stamping feature is enabled, the software must allocate 32 bytes (8 words) of memory for every descriptor. (The time stamping feature is used with the IEEE 1588 PTP engine).
- When only the receive checksum offload is enabled (time stamping disabled), software must allocate 32 bytes (8 words) of memory for every descriptor. However, only word 4 of the extended words (descriptors 4–7) contains the required status information. Treat the rest of the extended words as reserved or dummy.

Transmit Descriptor

The *Transmit Descriptor Words* figure shows the transmit descriptor structure in memory. The application software must program the TDES0 control bits during descriptor initialization. When the DMA updates the descriptor, it writes back all the control bits except the OWN bit (which it clears) and updates the status bits. The following tables give the contents of the transmitter descriptor word 0 (TDES0) through word 7 (TDES7).

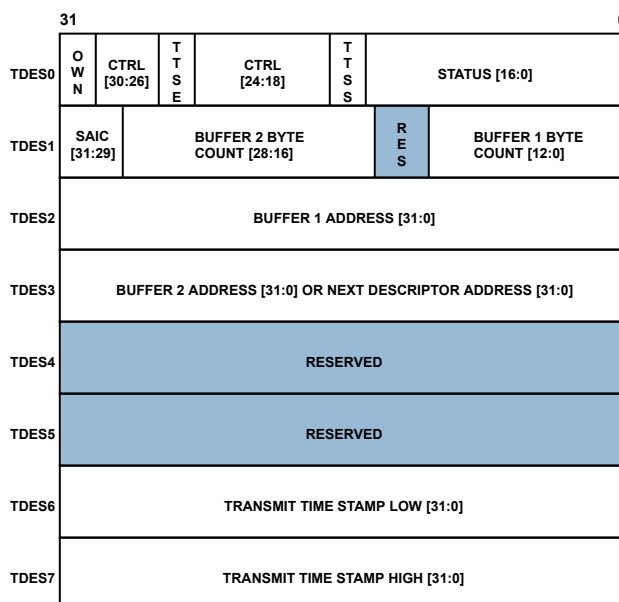


Figure 24-14: Transmit Descriptor Words

Table 24-15: Transmit Descriptor Fields (TDES0)

Bit	Name	Description
31	OWN	Ownership. When set, this bit indicates that the DMA owns the descriptor. When this bit is reset, it indicates that the application owns the descriptor. The DMA clears this bit either when it completes the frame transmission or when the buffers allocated in the descriptor are read completely. The ownership bit of the first descriptor of the frame must be set after all subsequent descriptors belonging to the same frame have been set. This configuration avoids a possible race condition between fetching a descriptor and the driver setting an ownership bit.
30	IC	Interrupt on Completion. When set, this bit sets the transmit interrupt (DMA status register [0]) after the present frame is transmitted.

Table 24-15: Transmit Descriptor Fields (TDES0) (Continued)

Bit	Name	Description
29	LS	Last Segment. When set, this bit indicates that the buffer contains the last segment of the frame.
28	FS	First Segment. When set, this bit indicates that the buffer contains the first segment of a frame.
27	DC	Disable CRC. When this bit is set, the EMAC does not append a cyclic redundancy check (CRC) to the end of the transmitted frame. This functionality is valid only when the first segment (TDES0[28]) is set.
26	DP	Disable Pad. When set, the EMAC does not automatically add padding to a frame shorter than 64 bytes. When this bit is reset, the DMA automatically adds padding and CRC to a frame shorter than 64 bytes. The CRC field is added despite the state of the DC (TDES0[27]) bit. This functionality is valid only when the first segment (TDES0[28]) is set.
25	TTSE	Transmit Time Stamp Enable. When set, this bit enables IEEE1588 hardware time stamping for the transmit frame referenced by the descriptor. This field is valid only when the first segment control bit (TDES0[28]) is set.
24	CRCR	CRC Replacement Control. When set, the EMAC replaces the last four bytes of the transmitted packet with recalculated CRC bytes. The host should ensure that the CRC bytes are present in the frame being transferred from the transmit buffer. This bit is valid when the first segment bit (TDES0[28]) and the disable CRC bit (TDES0[27]) are set.
24	Reserved	
23:22	CIC	Checksum Insertion Control. These bits control the checksum calculation and insertion. Bit encodings are as follows: 00 = Checksum Insertion disabled. 01 = Only IP header checksum calculation and insertion are enabled. 10 = IP header checksum and payload checksum calculation and insertion are enabled, but pseudo-header checksum is not calculated in hardware. 11 = IP header checksum and payload checksum calculation and insertion are enabled, and pseudo-header checksum is calculated in hardware.
21	TER	Transmit End of Ring. When set, this bit indicates that the descriptor list reached its final descriptor. The DMA returns to the base address of the list, creating a descriptor ring.
20	TCH	Second Address Chained. When set, this bit indicates that the second address in the descriptor is the next descriptor address rather than the second buffer address. When TDES0[20] bit is set, TBS2 (TDES1[28:16]) is a <i>do-not-care</i> value. TDES0[21] takes precedence over TDES0[20].
19:18	Reserved	

Table 24-15: Transmit Descriptor Fields (TDES0) (Continued)

Bit	Name	Description
19:18	VLIC	VLAN Insertion Control. When set, these bits request the MAC to perform VLAN tagging or untagging before transmitting the frames. If the frame is modified for VLAN tags, the MAC automatically recalculates and replaces the CRC bytes. Bit encodings are as follows. 00 = Do not add a VLAN tag. 01 = Remove the VLAN tag from the frames before transmission. This option should be used only with the VLAN frames. 10 = Insert a VLAN tag with the tag value programmed in the VLAN tag inclusion or replacement EMAC_VLAN_INCL register. 11 = Replace the VLAN tag in frames with the tag value programmed in the VLAN tag inclusion or replacement EMAC_VLAN_INCL register. This option should be used only with the VLAN frames. These bits are valid when the first segment bit (TDES0[28]) is set.
17	TTSS	Transmit Time Stamp Status. This bit is a status bit to indicate that a time stamp is captured for the described transmit frame. When this bit is set, TDES2 and TDES3 have a time stamp value captured for the transmit frame. This field is only valid when the last segment control bit of the descriptor (TDES0[29]) is set.
16	IHE	IP Header Error. When set, this bit indicates that the EMAC transmitter detected an error in the IP datagram header. The transmitter checks the header length in the IPv4 packet against the number of header bytes received from the application. It indicates an error status when there is a mismatch. For IPv6 frames, a header error is reported if the main header length is not 40 bytes. Furthermore, the Ethernet length or type field value for an IPv4 or IPv6 frame must match the IP header version received with the packet. For IPv4 frames, an error status is also indicated if the header length field has a value less than 0x5.
15	ES	Error Summary. Indicates the logical OR of the following bits: TDES0[14] = Jabber Timeout TDES0[13] = Frame Flush TDES0[11] = Loss of Carrier TDES0[10] = No Carrier TDES0[9] = Late Collision TDES0[8] = Excessive Collision TDES0[2] = Excessive Deferral TDES0[1] = Underflow Error TDES0[16] = IP Header Error TDES0[12] = IP Payload Error
14	JT	Jabber Timeout. When set, this bit indicates that the EMAC transmitter has experienced a jabber timeout. This bit is only set when the EMAC_MACCFG.JB bit is not set.
13	FF	Frame Flushed. When set, this bit indicates that the DMA or MFL flushed the frame due to a software flush command given by the CPU.

Table 24-15: Transmit Descriptor Fields (TDES0) (Continued)

Bit	Name	Description
12	IPE	IP Payload Error. When set, this bit indicates that EMAC transmitter detected an error in the TCP, UDP, or ICMP IP datagram payload. The transmitter checks the payload length received in the IPv4 or IPv6 header against the actual number of TCP, UDP, or ICMP packet bytes received from the application. It issues an error status when there is a mismatch.
11	LC	Loss of Carrier. When set, this bit indicates a loss of carrier occurred during frame transmission. This functionality is valid only for the frames transmitted without collision when the EMAC operates in half-duplex mode. Loss of Carrier. When set, this bit indicates that the EMAC aborted the frame transmission because of a collision occurring after the collision window (64 byte-times, including preamble, in RMII mode; 512 byte-times, including preamble and carrier extension, in RGMII mode). This bit is not valid if the underflow error bit is set.
10	NC	No Carrier. When set, this bit indicates that the carrier sense signal from the PHY did not assert during transmission.
9	LC	Late Collision. When set, this bit indicates that the EMAC aborted the frame transmission because of a collision occurring after the collision window (64 byte-times, including preamble). This bit is not valid if the underflow error bit is set. Late Collision. When set, this bit indicates that the EMAC aborted the frame transmission because of a collision occurring after the collision window (64 byte-times, including preamble, in RMII mode; 512 byte-times, including preamble and carrier extension, in RGMII mode). This bit is not valid if the underflow error bit is set.
8	EC	Excessive Collision. When set, this bit indicates that the EMAC aborted the transmission after 16 successive collisions, while attempting to transmit the current frame. If the <code>EMAC_MACCFG.DR</code> disable retry bit is set, this bit is set after the first collision, and the transmission of the frame aborts.
7	VF	VLAN Frame. When set, this bit indicates that the transmitted frame was a VLAN-type frame.
6:3	CC	Collision Count. When set, these bit indicate that the EMAC aborted the frame transmission due to a collision occurring after the collision window (64 byte-times, including preamble). This bit is not valid if the Underflow Error bit is set. This field is updated only in half-duplex mode.
6:3	SLOTNUM	Slot Number Control Bits. In AV mode, these bits indicate the slot interval in which the data should be fetched from the corresponding buffers, addressed by TDES2 or TDES3. When the transmit descriptor is fetched, the DMA compares the slot number value in this field with the slot function control and status register (RSN). It fetches the data from the buffers only if there is a match in values. These bits are valid only for AV channels (not channel 0).
2	ED	Excessive Deferral. When set, this bit indicates that the transmission has ended because of excessive deferral when the <code>EMAC_MACCFG.DC</code> deferral check bit is set high. Excessive deferral is over 24,288-bit times. (155,680-bits times in 1,000-Mbps mode or when jumbo frame is enabled).
1	UF	Underflow Error. When set, this bit indicates that the EMAC aborted the frame because data arrived late from the application memory. Underflow error indicates that the DMA encountered

Table 24-15: Transmit Descriptor Fields (TDES0) (Continued)

Bit	Name	Description
		an empty transmit buffer while transmitting the frame. The transmission process enters the suspended state and sets both transmit underflow (EMAC_DMA0_STAT.UNF) and transmit interrupt (EMAC_DMA0_STAT.TI) bits.
0	DB	Deferred Bit. When set, this bit indicates that the EMAC defers before transmission because of the presence of carrier. This bit is valid only in half-duplex mode.

Table 24-16: Transmit Descriptor Word 1 (TDES1)

Bit	Name	Description
31-29	Reserved	
31-29	SAIC	Source Address Insertion Control. Request the MAC to add or replace the source address field in the Ethernet frame with the value given in the MAC address register 0 or MAC address register 1. If the source address field is modified in a frame, the MAC automatically recalculates and replaces the CRC bytes. SAIC[2] chooses between MAC address register 0 and MAC address register 1 for source address insertion or replacement. The following list describes SAIC[1:0]. 00 = Do not include the source address. 01 = Include or insert the source address. For reliable transmission, the application must provide frames without source addresses. 10 = Replace the source address. For reliable transmission, the application must provide frames with source addresses. 11 = Reserved. This field is valid only when the first segment bit (TDES0[28]) is set.
28-16	TBS2	Transmit Buffer 2 Size. These bits indicate the second data buffer size in bytes. This field is not valid if TDES0[20] is set.
15-13	Reserved	
12-0	TBS1	Transmit Buffer 1 Size. These bits indicate the first data buffer byte size, in bytes. If this field is 0, the DMA ignores this buffer and uses buffer 2 or the next descriptor, depending on the value of TCH (TDES0[20]).

Table 24-17: Transmit Descriptor 2 (TDES2)

Bit	Name	Description
31-0	Buffer 1 Address Pointer	These bits indicate the physical address of buffer 1. There is no limitation on the buffer address alignment

Table 24-18: Transmit Descriptor 3 (TDES3)

Bit	Name	Description
31-0	Buffer 2 Address Pointer (Next Descriptor Address)	Indicates the physical address of buffer 2 when DMA uses a descriptor ring structure. If the second address chained (TDES1[24]) bit is set, this address contains the pointer to the physical memory where the next descriptor is present. The buffer address pointer must align to the bus width only when TDES1[24] is set. LSBs are ignored internally.

Table 24-19: Transmit Descriptor 6 (TDES6)

Bit	Name	Description
31–0	TTSL	Transmit Frame Time Stamp Low. The DMA updates this field with the least significant 32 bits of the time stamp captured for the corresponding transmit frame. This field has the time stamp only if the last segment bit (LS) in the descriptor is set and time stamp status (TTSS) bit is set.

Table 24-20: Transmit Descriptor 7 (TDES7)

Bit	Name	Description
31–0	TTSH	Transmit Frame Time Stamp High. The DMA updates this field with the most significant 32 bits of the time stamp captured for the corresponding receive frame. This field has the time stamp only if the last segment bit (LS) in the descriptor is set and time stamp status (TTSS) bit is set.

DMA Transmit Process

The following sections describe how the transmission process works for direct memory access on the EMAC controller.

- [Default \(Non-OSF\) Mode](#)
- [OSF Mode Enabled](#)
- [Transmit Frame Processing](#)
- [Transmit Polling Suspended](#)

Default (Non-OSF) Mode

The following sequence describes the default process for DMA transmit works. The sequence applies DMA1 and DMA2 as well.

1. The application sets up the transmit descriptor (using TDES0- TDES3) and sets the OWN bit (TDES0) after setting up the corresponding data buffers with Ethernet frame data.
2. Once the `EMAC_DMA0_OPMODE.ST` bit is set, the DMA enters the run state.
3. While in the run state, the DMA polls the transmit descriptor list for frames requiring transmission. After polling starts, it continues in either sequential descriptor ring order or chained order. If the DMA detects a descriptor flagged as owned by the application, or if an error condition occurs, transmission suspends. Both the transmit buffer unavailable (`EMAC_DMA0_STAT.TU`) and normal interrupt summary (`EMAC_DMA0_STAT.NIS`) bits are set. The transmit engine proceeds to Step 9.
4. If the acquired descriptor is flagged as owned by DMA (`TDES0[31] = 1#b1`), the DMA decodes the transmit data buffer address from the acquired descriptor.
5. The DMA fetches the transmit data from the application memory and transfers the data to the MFL for transmission.

6. If an Ethernet frame is stored over data buffers in multiple descriptors, the DMA closes the intermediate descriptor and fetches the next descriptor. Steps 3, 4, and 5 repeat until the end-of-Ethernet-frame data transfers to the MFL.
7. Frame transmission completes. If IEEE 1588 time stamping was enabled for the frame, the time stamp value obtained from MFL is written to the transmit descriptor (TDES2 and TDES3) that contains the end-of-frame buffer. (The transmit status indicates if IEEE 1588 time stamping enables). The status information is then written to this transmit descriptor (TDES0). Because the OWN bit is cleared during this step, the application now owns this descriptor. If time stamping was not enabled for this frame, the DMA does not alter the contents of TDES2 and TDES3.
8. Transmit interrupt (`EMAC_DMA0_STAT.TI`) is set after completing transmission of a frame. The frame has interrupt on completion (`TDES1 [31]`) set in its last descriptor. The DMA engine then returns to Step 3.
9. In the suspend state, the DMA tries to reacquire the descriptor (and returns to Step 3) when it receives a transmit poll demand and the `EMAC_DMA0_STAT.UNF` bit is cleared.

NOTE: If the `EMAC_DMA0_OPMODE.OSF` bit is not set, the actual inter frame gap (IFG) is more than the value programmed in the `EMAC_MACCFG` register.

OSF Mode Enabled

While in the run state, the transmit process can simultaneously acquire two frames without closing the status descriptor of the first (if the `EMAC_DMA0_OPMODE.OSF` bit is set). As the transmit process finishes transferring the first frame, it immediately polls the transmit descriptor list for the second frame. If the second frame is valid, the transmit process transfers this frame before writing the status information of the first frame.

In OSF mode, the run state transmit DMA operates in the following sequence.

1. The DMA operates as described in steps 1–6 of [Default \(Non-OSF\) Mode](#).
2. Without closing the previous last descriptor of the frame, the DMA fetches the next descriptor.
3. If the DMA owns the acquired descriptor, the DMA decodes the transmit buffer address in this descriptor. If the DMA does not own the descriptor, the DMA goes into suspend mode and skips to Step 7.
4. The DMA fetches the transmit frame from the application memory and transfers the frame to the MFL until the end-of-frame data is transferred. It closes the intermediate descriptors when this frame splits across multiple descriptors.
5. The DMA waits for the frame transmission status and time stamp of the previous frame. Once the status is available, the DMA writes the time stamp to TDES2 and TDES3, if the time stamp was captured (as indicated by a status bit). The DMA then writes the status, with a cleared OWN bit, to the corresponding TDES0, thus closing the descriptor. If time stamping was not enabled for the previous frame, the DMA does not alter the contents of TDES2 and TDES3.
6. If enabled, the transmit interrupt is set; the DMA fetches the next descriptor, and then proceeds to Step 3 (when status is normal). If the previous transmission status shows an underflow error, the DMA goes into suspend mode (Step 7).

7. In suspend mode, if a pending status and time stamp are received from the MFL, the DMA writes the time stamp (if enabled for the current frame) to TDES2 and TDES3. The DMA then writes the status to the corresponding TDES0. It then sets relevant interrupts and returns to suspend mode.
8. After receiving a transmit poll demand (`EMAC_DMA0_TXPOLL`), the DMA can exit suspend mode and enter the run state. (Go to Step 1 or Step 2 depending on pending status)

NOTE: If the `EMAC_DMA0_OPMODE.OSF` bit is set, the DMA fetches the next descriptor in advance of closing the current descriptor. Therefore, the descriptor chain must have more than two different descriptors for proper operation.

NOTE: If the `EMAC_DMA0_OPMODE.OSF` bit is set, the DMA starts fetching the second frame immediately after completing the transfer of the first frame to the FIFO. It does not wait for the status to update. In the meantime, the MFL receives the second frame into the FIFO while transmitting the first frame. The difference in cycles is not seen for the first descriptor, because the time taken for the complete descriptor processing remains the same whether `EMAC_DMA0_OPMODE.OSF` is set or not. The difference appears only for the following descriptor because its processing began earlier.

Transmit Frame Processing

The transmit DMA engine expects that the data buffers contain complete Ethernet frames, excluding: preamble, pad bytes, and FCS fields. The destination address, source address, and type or length fields contain valid data. If the transmit descriptor indicates that the EMAC CORE must disable CRC or PAD insertion, the buffer must have complete Ethernet frames (excluding preamble), including the CRC bytes.

Frames can be data-chained and can span several buffers. Frames must be delimited by the first descriptor (TDES0[28]) and the last descriptor (TDES0[29]), respectively.

As transmission starts, the first descriptor must have (TDES0[28]) set. Frame data transfers from the application buffer to the transmit FIFO. Concurrently, if last descriptor (TDES0[29]) of the current frame clears, the transmit process attempts to acquire the next descriptor. The transmit process expects this descriptor to have TDES0[28] clear. If TDES1[29] is clear, it indicates an intermediary buffer. If TDES0[29] is set, it indicates the last buffer of the frame.

After the last buffer of the frame has been transmitted, the DMA writes back the final status information. The DMA writes to the transmit descriptor 0 (TDES0) word of the descriptor that has the last segment set in transmit descriptor 0 (TDES0[29]). Now, if interrupt-on-completion (TDES0[30]) is set, the transmit interrupt (DMA_STAT [0]) is set, the next descriptor is fetched, and the process repeats.

Actual frame transmission begins after the MFL transmit FIFO has reached either a programmable transmit threshold (`EMAC_DMA0_OPMODE.TTC`), or a full frame is contained in the FIFO. There is also an option for store-and-forward mode (`EMAC_DMA0_OPMODE.TSF`). Descriptors are released (OWN bit TDES0 [31] clears) when the DMA finishes transferring the frame.

Transmit Polling Suspended

Either of the following conditions suspends transmit polling:

1. The DMA detects a descriptor owned by the application (TDES0 [31] = 0).

2. A frame transmission aborts when a transmit error due to underflow is detected. The appropriate transmit descriptor 0 (TDES0) bit is set.

If the second condition occurs, both of the abnormal interrupt summary ([15]) and transmit underflow bits ([5]) are set. The information is written to transmit descriptor 0, causing the suspension. If the DMA goes into a SUSPEND state due to the first condition, then both `EMAC_DMA0_STAT.NIS` and `EMAC_DMA0_STAT.TU` are set.

In both cases, the position in the transmit list is retained. The retained position is that of the descriptor following the last descriptor closed by the DMA.

The driver must explicitly issue a transmit poll demand command after rectifying the suspension cause. If the first condition occurs, the driver must give descriptor ownership to the DMA and then issue a poll demand command to resume the transfer.

Receive Descriptor

The *Receive Descriptor Words* figure shows the structure of the receive descriptor. It can have 32 bytes of descriptor data (8 DWORDs) when advanced time stamping or checksum is enabled.

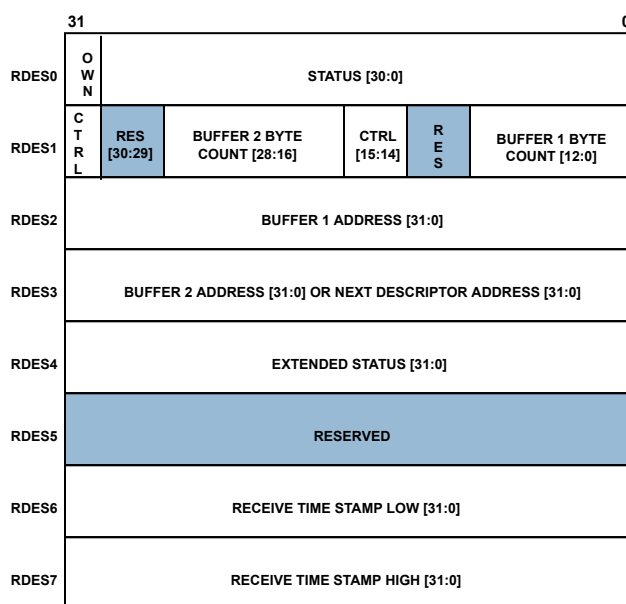


Figure 24-15: Receive Descriptor Words

Table 24-21: Receive Descriptor Fields (RDES0)

Bit	Name	Description
31	OWN	Ownership. When set, this bit indicates that the DMA of the EMAC subsystem owns the descriptor. When this bit is reset, this bit indicates that the application owns the descriptor. The DMA clears this bit either when it completes the frame reception or when the buffers that are associated with this descriptor are full.
30	AFM	Destination Address Filter Fail. When set, this bit indicates a frame that failed in the DA filter in the EMAC CORE.

Table 24-21: Receive Descriptor Fields (RDES0) (Continued)

Bit	Name	Description
29–16	FL	Frame Length. These bits indicate the byte length of the received frame that transferred to application memory (including CRC). This field is valid when last descriptor (RDES0[8]) is set and either the descriptor error (RDES0[14]) or overflow error bits are reset. This field is valid when last descriptor (RDES0[8]) is set. When the last descriptor and error summary bits are not set, this field indicates the accumulated number of bytes that transferred for the current frame.
15	ES	Error Summary. Indicates the logical OR of the following bits. RDES0[1] = CRC Error RDES0[3] = RGMII Receive Error RDES0[4] = Watchdog Timeout RDES0[6] = Late Collision RDES0[7] = Time Stamp Available RDES4[4:3] = IP Header/Payload Error RDES0[11] = Overflow Error RDES0[14] = Descriptor Error. This field is valid only when the last descriptor (RDES0[8]) is set.
14	DE	Descriptor Error. When set, this bit indicates a frame truncation caused by a frame that does not fit within the current descriptor buffers. The DMA does not own the next descriptor. The frame is truncated. This field is valid only when the last descriptor (RDES0[8]) is set.
13	Reserved	
13	SAF	Source Address Filter Fail. When set, this bit indicates that the SA field of frame failed the SA filter in the EMAC Core.
12	LE	Length Error. When set, this bit indicates that the actual length of the frame received and that the length or type field does not match. This bit is valid only when the frame type (RDES0[5]) bit is reset.
11	OE	Overflow Error. When set, this bit indicates that the received frame is damaged due to buffer overflow in MFL.
10	VLAN	VLAN Tag. When set, this bit indicates that the frame pointed to by this descriptor is a VLAN frame tagged by the EMAC CORE.
9	FS	First Descriptor. When set, this bit indicates that this descriptor contains the first buffer of the frame. If the size of the first buffer is 0, the second buffer contains the beginning of the frame. If the size of the second buffer is also 0, the next descriptor contains the beginning of the frame.
8	LS	Last Descriptor. When set, this bit indicates that the buffers pointed to by this descriptor are the last buffers of the frame
7	Time Stamp Available	When set, this bit indicates that a snapshot of the time stamp is written in descriptor words 6 (RDES6) and 7 (RDES7). This functionality is valid only when the last descriptor bit (RDES0[8]) is set
6	LC	Late Collision. When set, this bit indicates that a late collision has occurred while receiving the frame in half-duplex mode.

Table 24-21: Receive Descriptor Fields (RDES0) (Continued)

Bit	Name	Description
5	FT	Frame Type. When set, this bit indicates that the receive frame is an Ethernet-type frame (the LT field is greater than or equal to 16'h0600). When this bit is reset, it indicates that the received frame is an IEEE802.3 frame. This bit is not valid for runt frames less than 14 bytes.
4	RWT	Receive Watchdog Timeout. When set, this bit indicates that the receive watchdog timer has expired while receiving the current frame. The current frame is truncated after the watchdog timeout.
3	Reserved	
3	RE	Receive Error. When set, this bit indicates that the RGMII PHY sent RGMII RXERR on RXCTL pin during frame reception. This error also includes the carrier extension error in RGMII and half-duplex mode.
2	DE	Dribble Bit Error. When set, this bit indicates that the received frame has a non-integer multiple of bytes (odd nibbles).
1	CE	CRC Error. When set, this bit indicates that a Cyclic Redundancy Check (CRC) error occurred on the received frame. This field is valid only when the last descriptor (RDES0[8]) is set.
0	Extended Status Available	When set, this bit indicates that the extended status is available in descriptor word 4 (RDES4). This functionality is valid only when the last descriptor bit (RDES0[8]) is set.

Table 24-22: Receive Descriptor Fields 1 (RDES1)

Bit	Name	Description
31	DIC	Disable Interrupt on Completion. When set, this bit prevents setting the EMAC_DMA0_STAT.RI bit of the status register for the received frame ending in the buffer indicated by this descriptor. This activity, in turn, disables the assertion of the interrupt to the application due to RI for that frame.
30–29	Reserved	
28–16	RBS2	Receive Buffer 2 Size. These bits indicate the second data buffer size, in bytes. The buffer size must be a multiple of 4 (32-bit bus), even if the value of RDES3 (buffer2 address pointer) does not align to bus width. If the buffer size is not an appropriate multiple of 4, 8, or 16, the resulting behavior is undefined. This field is not valid if RDES1[14] is set.
15	RER	Receive End of Ring. When set, this bit indicates that the descriptor list reached its final descriptor. The DMA returns to the base address of the list, creating a descriptor ring.
14	RCH	Second Address Chained. When set, this bit indicates that the second address in the descriptor is the next descriptor address rather than the second buffer address. When this bit is set, RBS2 (RDES1[28:16]) is a <i>do-not-care</i> value. RDES1[15] takes precedence over RDES1[14].
13	Reserved	
12–0	RBS1	Receive Buffer 1 Size. Indicates the size of the first data buffer in bytes. The buffer size must be a multiple of 4 (32-bit bus), even if the value of RDES2 (buffer1 address pointer) is not aligned. When the buffer size is not a multiple of 4, the resulting behavior is undefined. If this field is 0, the DMA ignores this buffer and uses buffer 2 or next descriptor depending on the value of RCH (Bit 14).

Table 24-23: Receive Descriptor Fields 2 (RDES2)

Bit	Name	Description
31–0	Buffer 1 Address Pointer	These bits indicate the physical address of buffer 1. There are no limitations on the buffer address alignment except for the following condition: The DMA uses the configured value for its address generation when the RDES2 value stores the start of frame. The DMA performs a write operation with the RDES2[1:0] bits as 0 during the transfer of the start of frame. However, the frame data shifts per the address pointer of the actual buffer. The DMA ignores RDES2[1:0] when the address pointer is to a buffer where the middle or last part of the frame is stored. (RDES2[1:0] corresponds to a bus width of 32).

Table 24-24: Receive Descriptor Fields 3 (RDES3)

Bit	Name	Description
31–0	Buffer 2 Address Pointer (Next Descriptor Address)	These bits indicate the physical address of buffer 2 when DMA uses a descriptor ring structure. If the second address chained (RDES1[24]) bit is set, this address contains the pointer to the physical memory where the next descriptor is present. If RDES1[24] is set, the buffer (next descriptor) address pointer must be bus width-aligned (RDES3[1:0] = 0, corresponding to a bus width of 32. LSBs are ignored internally.) However, when RDES1[24] is reset, there are no limitations on the RDES3 value, except for the following condition: The DMA uses the configured value for its buffer address generation when the RDES3 value stores the start of frame. The DMA ignores RDES3[1:0] when the address pointer is to a buffer where the middle or last part of the frame is stored. (RDES3[1:0] corresponds to a bus width of 32.)

Table 24-25: Receive Descriptor Fields 4 (RDES4)

Bit	Name	Description
31-26	Reserved	
25	Layer 4 Filter Match	When set, this bit indicates that the received frame matches layer 4 filter fields. This status is given only when one of the following conditions is true: - Layer 3 fields are not enabled and all enabled layer 4 fields match. - All enabled layer 3 and layer 4 filter fields match.
24	Layer 3 Filter Match	When set, this bit indicates that the received frame matches layer 3 IP address fields. This status is given only when one of the following conditions is true: - All enabled layer 3 fields match and all enabled layer 4 fields are bypassed. - All enabled filter fields match.
23-21	Reserved	
20-18	VLAN Tag Priority	These bits give the VLAN tag's user value in the received packet. These bits are valid only when the RDES4[16] and RDES4[17] are set.
17	AV Tagged Packet Received	When set, this bit indicates that an AV tagged packet is received. Otherwise, this bit indicates that an untagged AV packet is received. This bit is valid when RDES4[16] is set.
16	AV Packet Received	When set, this bit indicates that an AV packet is received.
15	Reserved	

Table 24-25: Receive Descriptor Fields 4 (RDES4) (Continued)

Bit	Name	Description
14	Timestamp Dropped	When set, this bit indicates that the time stamp is captured for this frame but dropped in the MFL RxFIFO because of overflow.
13	PTP Version	When set, this bit indicates that the received PTP message has the IEEE 1588 version 2 format. When reset, it has the version 1 format. This description is valid only if the message type (RDES4[11:8]) is non-zero.
12	PTP Frame Type	When set, this bit indicates that the PTP message transfers directly over Ethernet. When this bit is not set and the message type is non-zero, it indicates that the PTP message transfers over UDP-IPv4 or UDP-IPv6. Bits 6 and 7 have the information on IPv4 or IPv6.
11–8	Message Type	These bits are encoded to give the type of the message received. 0000 = No PTP message received 0001 = SYNC (all clock types) 0010 = Follow_Up (all clock types) 0011 = Delay_Req (all clock types) 0100 = Delay_Resp (all clock types) 0101 = Pdelay_Req (in peer-to-peer transparent clock) or Announce (in ordinary or boundary clock) 0110 = Pdelay_Resp (in peer-to-peer transparent clock) or Management (in ordinary or boundary clock) 0111 = Pdelay_Resp_Follow_Up (in peer-to-peer transparent clock) or Signaling (for ordinary or boundary clock) 1xxx - Reserved
7	IPv6 Packet Received	When set, this bit indicates that the received packet is an IPv6 packet.
6	IPv4 Packet Received	When set, this bit indicates that the received packet is an IPv4 packet.
5	IP Checksum Bypassed	When set, this bit indicates that the checksum offload engine is bypassed.
4	IP Payload Error	When set, this bit indicates that the 16-bit IP payload checksum that the core calculated does not match the corresponding checksum field in the received segment. (For example: the TCP, UDP, or ICMP checksum). The bit is also set when the TCP, UDP, or ICMP segment length does not match the payload length value in the IP header field.
3	IP Header Error	When set, this bit indicates that the 16-bit IPv4 header checksum calculated by the core does not match the received checksum bytes. Or, it indicates that the IP datagram version is not consistent with the Ethernet type value.

Table 24-25: Receive Descriptor Fields 4 (RDES4) (Continued)

Bit	Name	Description
2-0	IP Payload Type	These bits indicate the type of payload encapsulated in the IP datagram processed by the receive checksum offload engine (COE). The COE also sets these bits to 2'b00 when it does not process the payload of the IP datagram. It does not process the payload because of an IP header error or fragmented IP. 000 = Unknown or did not process IP payload 001 = UDP 010 = TCP 011 = ICMP 1xx = Reserved

Table 24-26: Extended Receive Descriptor Fields 4 (RDES4)

Bit	Name	Description
31-26	Reserved	
25	Layer 4 Filter Match	When set, this bit indicates that the received frame matches layer 4 filter fields. This status is given only when one of the following conditions is true: • Layer 3 fields are not enabled and all enabled layer 4 fields match. • All enabled layer 3 and layer 4 filter fields match.
24	Layer 3 Filter Match	When set, this bit indicates that the received frame matches layer 3 IP address fields. This status is given only when one of the following conditions is true: • All enabled layer 3 fields match and all enabled layer 4 fields are bypassed. • All enabled filter fields match.
23-21	Reserved	
20-18	VLAN Tag Priority	These bits give the VLAN tag's user value in the received packet. These bits are valid only when the RDES4[16] and RDES4[17] are set.
17	AV Tagged Packet Received	When set, this bit indicates that an AV tagged packet is received. Otherwise, this bit indicates that an untagged AV packet is received. This bit is valid when RDES4[16] is set.
16	AV Packet Received	When set, this bit indicates that an AV packet is received.
15	Reserved	
14	Timestamp Dropped	When set, this bit indicates that the timestamp was captured for this frame but got dropped in the MFL Rx FIFO because of overflow.

Table 24-27: Receive Descriptor Fields 6 (RDES6)

Bit	Name	Description
31–0	RTSL	Receive Frame Time Stamp Low. The DMA updates this field with the least significant 32 bits of the time stamp captured for the corresponding receive frame. The DMA updates this field only for the last descriptor of the receive frame. The status bit (RDES0[8]) indicates the last descriptor.

Table 24-28: Receive Descriptor Fields 7 (RDES7)

Bit	Name	Description
31–0	RTSH	Receive Frame Time Stamp High. The DMA updates this field with the most significant 32 bits of the time stamp captured for the corresponding receive frame. The DMA updates this field only for the last descriptor of the receive frame. The status bit (RDES0[8]) indicates the last descriptor.

EMAC DMA Receive Process

The following sections describe how the receive process for direct memory access works on the EMAC controller.

- [Receive Frame Processing](#)
- [Receive Descriptor Acquisition](#)
- [Receive Process Suspended](#)

The reception process for DMA works as follows:

1. The application sets up receive descriptors (RDES0–RDES3) and sets the OWN bit (RDES0 [31]).
2. Once the `EMAC_DMA0_OPMODE.SR` bit is set, the DMA enters the run state. While in the run state, the DMA attempts to acquire free descriptors by polling the receive descriptor list. If the fetched descriptor is not free (the application owns the descriptor), the DMA enters the suspend state and jumps to Step 9.
3. The DMA decodes the receive data buffer address from the acquired descriptors.
4. Incoming frames are processed and placed in the data buffers of the acquired descriptor.
5. When the buffer is full or the frame transfer is complete, the receive engine fetches the next descriptor.
6. If the current frame transfer is complete, the DMA proceeds to Step 7. If IEEE 1588 time stamping is enabled, the DMA writes the time stamp (if available) to the current descriptor. If the DMA does not own the next fetched descriptor and the frame transfer is not complete, the DMA sets the descriptor error bit in the RDES0. (The bit is set unless flushing is disabled.) The DMA closes the current descriptor (clears the OWN bit). The DMA marks it as intermediate by clearing the last segment (LS) bit in the RDES0 value (marks it as last descriptor if flushing is not disabled). The DMA then proceeds to Step 8. If the DMA owns the next descriptor but the current frame transfer is not complete, the DMA closes the current descriptor as intermediate and reverts to Step 4.
7. If IEEE 1588 time stamping is enabled, the DMA writes the time stamp (if available) to RDES2 and RDES3 of the current descriptor. The DMA then takes the status of the receive frame from the MFL and writes the status word to RDES0 of the current descriptor. The OWN bit is cleared and the last segment bit is set.

8. The receive engine checks the OWN bit of the latest descriptor. If the host owns the descriptor (OWN bit is 0), the `EMAC_DMA0_STAT.RU` bit is set. The DMA receive engine enters the suspended state (Step 9). If the DMA owns the descriptor, the engine returns to Step 4 and awaits the next frame.
9. Before the receive engine enters the suspend state, partial frames are flushed from the receive FIFO (programs control flushing using the `EMAC_DMA0_OPMODE.DFF` bit).
10. The receive DMA exits the suspend state when a receive poll demand is given or the start of next frame is available from the receive FIFO of the MFL. The engine proceeds to Step 2 and refetches the next descriptor.

Receive Frame Processing

The EMAC transfers the received frames to the application memory only when:

- the frame passes the address filter subblock, and
- the frame size is greater than or equal to configurable threshold bytes set for the receive FIFO of MFL, or
- the complete frame is written to the FIFO in store-and-forward mode.

If the frame fails the address filtering, the EMAC block drops the frame (unless the `EMAC_MACFRMFILT.RA` bit is set). Frames that are shorter than 64 bytes, because of collision or premature termination, can be purged from the receive FIFO.

After receiving 64 bytes (configurable threshold), the MFL block requests that the DMA block begin transferring the frame data to the receive buffer pointed to by the current descriptor. The DMA sets first descriptor (`RDES0 [9]`) after the SCB becomes ready to receive the data (if DMA is not fetching transmit data from the application). The descriptors release when the OWN (`RDES [31]`) bit is reset to 0. The bit is reset either as the data buffer fills up or as the last segment of the frame is transferred to the receive buffer. If the frame is contained in a single descriptor, both the last descriptor (`RDES [8]`) and the first descriptor (`RDES [9]`) are set.

The DMA fetches the next descriptor, sets the last descriptor (`RDES [8]`) bit, and releases the `RDES0` status bits in the previous frame descriptor. Then, the DMA sets the `EMAC_DMA0_STAT.RI` bit. The same process repeats unless the DMA encounters a descriptor the application owns. If this encounter occurs, the receive process sets the `EMAC_DMA0_STAT.RU` bit and then enters the suspend state. The position in the receive list is retained.

Receive Descriptor Acquisition

The receive engine always attempts to acquire an extra descriptor in anticipation of an incoming frame. Descriptor acquisition is attempted when any of the following conditions is satisfied:

- The `EMAC_DMA0_OPMODE.SR` bit is set immediately after being placed in the run state.
- The data buffer of current descriptor is full before the frame ends for the current transfer.
- The controller completes frame reception, but the current receive descriptor is not yet closed.
- The receive process suspends because of an application-owned buffer (`RDES0 [31] = 0`) and a new frame is received.
- A receive poll demand issues.

Receive Process Suspended

If a new receive frame arrives while the receive process is in the suspend state, the DMA refetches the current descriptor in the application memory. If the DMA now owns the descriptor, the receive process reenters the run state and starts frame reception. If the application still owns the descriptor, by default, the DMA discards the current frame at the top of the receive FIFO and increments the missed frame counter. If more than one frame is stored in the receive FIFO, the process repeats.

Avoid the discarding or flushing of the frame at the top of the receive FIFO by setting the `EMAC_DMA0_OPMODE.DFF` bit. In such conditions, the receive process sets the receive buffer unavailable status and returns to the suspend state.

OWN Bit (Ownership) Semaphore

Usage or ownership of the transmit or receive descriptor between the application and EMAC is mutually exclusive. While the EMAC accesses the descriptor, the application cannot modify it. Conversely, while the host updates the descriptor, the EMAC cannot use the content of the descriptor. This functionality is implemented through the OWN bit in the transmit or receive descriptor, acting as a semaphore to prevent multiple, simultaneous access to the descriptors.

The following example is based on a usage case of 4 WORDs enabled for descriptors. A chain structure configuration is assumed. (The `EMAC_DMA0_BUSMODE.ATDS` bit is not set). However, the explanation of the OWN bit semaphore remains consistent irrespective of any particular mode of operation.

1. Transmit OWN Bit:

- TDES0 – TDES3 words implement the transmit descriptors. TDES0 [31] is defined as the OWN bit. When TDES0 [31] is set to 0, this bit indicates that the descriptor is available for the application to update. The application sets up the descriptors, including the buffer addresses, by updating TDES0 through TDES3.
- To release ownership of the descriptor to the EMAC, the application sets the transmit OWN bit, TDES0 [31], to 1. TDES0 [31] = 1 indicates that the descriptor is ready for the EMAC to use. DMA reads the descriptors, then fetches the data for transmission from the buffer locations pointed to by the transmit descriptors (TDES2 and TDES3). When either the last data buffer is empty or the end-of-frame is reached, DMA clears the TDES0 [31] bit to 0. Now, the transmit descriptor releases to the application for updates.

2. Receive OWN Bit:

- RDES0 – RDES3 words implement the receive descriptors. RDES0 [31] is defined as the OWN bit. When RDES0 [31] is set to 0, this bit indicates that the descriptor is available for the application to update. The application sets up the descriptors, including the buffer locations for writing the received data, by updating RDES0 through RDES3. To give ownership of the descriptor to the EMAC, the host sets the receive OWN bit, RDES0 [31], to 1.
- RDES0 [31] = 1 indicates that the descriptor is ready for use by the EMAC. DMA reads the descriptors, then writes the received data to the buffers with locations pointed to by the receive descriptors (RDES2

and RDES3). When either the last data buffer is full or the end-of-frame is reached, DMA clears the RDES0 [31] bit to 0. Now, the receive descriptor releases to the application for updates

Application Data Buffer Alignment

The transmit and receive data buffers do not have any restrictions on the start address alignment. The start address for the buffers aligns to any of the 4 bytes. However, the DMA always initiates transfers with the address aligned to the bus width with dummy data for the byte lanes not required. This alignment typically happens during the transfer of the beginning or end of an Ethernet frame.

Example for Buffer Read

If the transmit buffer address is 0x0002 and 15 bytes must transfer, the DMA reads 5 full words (5 x 32-bit data) from address 0x0000. However, when transferring data to the EMAC transmit FIFO, the extra bytes (the first 2 bytes) are dropped or ignored. Similarly, the last 3 bytes of the last transfer are also ignored. The DMA always transfers a full 32-bit data to the transmit FIFO, unless it is the end-of-frame.

Example for Buffer Write

If the receive buffer address is 0x0002 and 15 bytes of a received frame must transfer, the DMA writes 5 full words (5 x 32-bit data) to address 0x0000. However, the first 2 bytes of first transfer and the last 3 bytes of the third transfer have dummy data.

Buffer Size Calculations

The DMA engines do not update the size fields in the transmit and receive descriptors alone. The DMA updates only the status fields (RDES0 and TDES0) of the descriptors. The driver must perform the size calculations. The transmit DMA transfers the exact number of bytes (indicated by buffer size field of TDES1) towards the EMAC CORE. If a descriptor is marked as first (FS bit of TDES1 is set), then the DMA marks the first transfer from the buffer as the start of frame. If a descriptor is marked as last (LS bit of TDES1), the DMA marks the last transfer from that data buffer as the end-of frame to the EMAC.

The receive DMA transfers data to a buffer until the buffer is full or the end-of frame is received from the MFL. If a descriptor is not marked as last (LS bit of RDES0), then the buffers of the descriptor are full. The amount of valid data in a buffer is its buffer size field minus the data buffer pointer offset, when the FS bit of that descriptor is set. The offset is zero when the data buffer pointer aligns to the data bus width. If a descriptor is marked as last, then the buffer cannot be full (as indicated by the buffer size in RDES1). To compute the amount of valid data in this final buffer, the driver must:

- Read the frame length (FL bits of RDES0 [29:16]), and
- Subtract the sum of the buffer sizes of the preceding buffers in the frame

The receive DMA always transfers the start of next frame with a new descriptor.

EMAC FIFO Layer (EMAC MFL)

The MAC FIFO layer provides FIFO memory to buffer and regulates the frames between the application system memory and the EMAC CORE. It also allows the transfer of data between the application clock domain and the

EMAC clock domains. The MFL layer has transfer controllers for each direction, called the transmit controller (TxFIFO) and the receive controller (RxFIFO). The datapath for both directions is 32-bit wide and each controller has a dedicated FIFO.

The EMAC0 transmit FIFO size is fixed to 2048 bytes. The EMAC0 receive FIFO size is fixed to 2048 bytes.

FIFO Layer Transmit Path

The DMA engine controls all transactions for the transmit path with the application. Ethernet frames read from the system memory are pushed into the FIFO by the DMA. The frame is then popped out and transferred to the EMAC CORE when triggered. When the end-of-frame transfers, the status of the transmission is taken from the EMAC CORE and transferred back to the DMA. The FIFO fill level is indicated to the DMA so that it can initiate a data fetch in required bursts from the system memory through the SCB interface.

When the DMA enables the `EMAC_DMA0_OPMODE.OSF` bit, the MFL receives the second frame into the FIFO while transmitting the first frame. When the first frame has transferred, the status is sent to DMA. If the DMA has already completed sending the second packet to the MFL, it waits for the status of the first packet before proceeding to the next frame.

The following are the modes of operation for FIFO transactions.

1. Threshold mode. When the number of bytes in the FIFO crosses the configured threshold level, the data is ready to be popped out and forwarded to the EMAC CORE. The data is also ready when the end-of-frame is written before the threshold is crossed. The DMA uses the `TTC` bits of the DMA bus mode register to configure the threshold level.
2. Store-and-Forward mode. In this mode, the MFL pops the frame towards the EMAC CORE after a complete frame is stored in the FIFO. If the TX FIFO size is smaller than the Ethernet frame for transmission (such as a jumbo frame), then the frame forwards in two cases. The TX FIFO is almost full or the requested FIFO does not have space to accommodate the requested burst-length. Therefore, the FIFO read controller never stalls in store-and-forward mode even if the Ethernet frame length is bigger than the TX FIFO depth.

The FIFO threshold in the store-and-forward mode is given by the following formula.

$\text{DataWidth} = 32 \text{ bits}$ and $\text{PBL} = \text{Burst Length}$ programmed through the `DMA_BUSMODE` register.

NOTE: To avoid occurrences of a TX underflow event when using the store-and-forward mode, or in other words, to ensure that the entire frame is stored in the FIFO before the MFL pops the frame towards the EMAC CORE for transmission, ensure that the FIFO threshold (calculated with the above formula) is greater than the packet size. The PBL needs to be programmed accordingly.

Transmit FIFO and Half-Duplex Retransmissions

While a frame transfers from the FIFO, a collision event can occur on the EMAC line interface in half-duplex mode. The EMAC then indicates a retry attempt to the MFL. The EMAC gives the status before the end-of-frame transfers from MFL. Then, the MFL enables the retransmission by popping out the frame again from the FIFO.

After more than 96 bytes pop out of FIFO, the FIFO controller frees up that space. The controller makes it available to the DMA to push in more data. Retransmission is not possible after this threshold is crossed or when the EMAC CORE indicates a late-collision event.

Transmit FIFO Flush Operation

The EMAC provides control to the software to flush the transmit FIFO in the MFL layer using the `EMAC_DMA0_OPMODE.FTF` bit. The flush operation is immediate. The MFL clears the transmit FIFO and the corresponding pointers to the initial state. It clears the FIFO and pointers even if it is in the middle of transferring a frame to the EMAC CORE. The data that the MAC transmitter has already accepted is not flushed. The data is scheduled for transmission and results in underflow. The transmit FIFO does not complete the transfer of the rest of the frame. As in all underflow conditions, a runt frame is transmitted and observed on the line. The status of the frame is marked with both underflow and frame flush events (TDES0 bits 13 and 1).

The MFL also stops accepting any data from the application (DMA) during the flush operation. The MFL generates and transfers transmit status words to the application for the number of frames flushed inside the MFL (including partial frames). Frames that completely flush in the MFL have the status bit for frame flush (TDES0 bit 13) set. The MFL completes the flush operation when the application (DMA) accepts all of the status words for the frames that flushed. The MFL then clears the transmit FIFO flush control register bit. The MFL starts accepting new frames from the application (DMA).

FIFO Layer Receive Path

The receive controller operates in the following sequence:

1. When the EMAC CORE receives a frame, it pushes in data with the frame start and end indicators. The MFL accepts the data and pushes it into the FIFO.
2. The receive controller takes the data out of the FIFO and sends it to the DMA.
 - Threshold mode (default). This mode is configured using `EMAC_DMA0_OPMODE.RTC`. When the FIFO receives 64 bytes or a full packet of data, the receive controller pops out the data and indicates its availability to the DMA. Some error frames cannot be dropped, because the error status is received at the end-of-frame. By this time, the start of that frame has already been read out of the FIFO.
 - Rx FIFO Store-and-Forward mode. This mode is configured using `EMAC_DMA0_OPMODE.RSF`. A frame is read out only after being written completely into the receive FIFO. In this mode, all error frames are dropped such that only valid frames are read out and forwarded to the application. Error frames are dropped when the EMAC CORE is configured for this feature.
3. After the end-of-frame transfers, the status word from the EMAC CORE is also the pushed FIFO. When the status of a partial frame due to overflow is given out, the frame length field in the status word is not valid.

Receive FIFO Multi-Frame Handling

Since the status is available immediately following the data, the MFL stores any number of frames into the FIFO, as long as it is not full.

Receive FIFO Error Handling

If the MFL Rx FIFO is full before it receives the end of frame data from the EMAC, the DMA declares an overflow condition. The whole frame (including the status word) drops. The overflow counter in the DMA (overflow counter-register) increments. This activity occurs even if the `EMAC_DMA0_OPMODE.FEF` bit is set. If the start address of such a frame has already transferred, the rest of the frame drops. A dummy end of frame is written to the FIFO along with the status word. The status indicates a partial frame due to overflow. In such frames, the frame length field is invalid.

The MFL receive control logic can filter error and undersized frames using the `EMAC_DMA0_OPMODE.FEF` and `EMAC_DMA0_OPMODE.FUF` bits. If the start address of the frame has already transferred to the Rx FIFO read controller, that frame is not filtered. The start address of the frame transfers to the read controller after the frame crosses the receive threshold (set by the `EMAC_DMA0_OPMODE.RTC` bits).

If the MFL receive FIFO is configured for store-and-forward mode, it can filter and drop all error frames.

EMAC CORE

The EMAC CORE is the lowest block in the EMAC peripheral and it performs all operations with the external world (PHY chip). It has independent transmit and receive modules. The modules interact with the EMAC FIFO layer at one end and interacts with the PHY chip through the RMII interface at the other end. Both modules have several subblocks which are discussed in subsequent sections.

Transmission is initiated when the MFL (FIFO layer) pushes in data with the start of frame. The CORE then transmits to the reduced media-independent interface. After the end of frame transfers out, the CORE gives the status of the transmission back to the MFL. The MFL forwards the transmission to the application through DMA.

A receive operation initiates when the EMAC detects an SFD on the RMII/RGMII. The CORE strips the preamble and SFD before proceeding to process the frame. The header fields are checked for the filtering and the FCS field used to verify the CRC for the frame. The frame drops in the core when it fails the address filter.

NOTE: The term *CORE* (written in capitals) refers to the internal block of Ethernet peripheral. Do not confuse the term with the *processor core*.

Table 24-29: EMAC CORE Related Registers

Register Name	Description
MAC Configuration ^{*1}	Establishes receive and transmit operating modes including: • Watchdog, Jabber, and Jumbo frame sizes • Inter Frame Gap • Speed Control – 10/100/1000 Mbps • Full or Half Duplex • Loopback Mode • Checksum Offload • Enabling Tx or Rx Engines

Table 24-29: EMAC CORE Related Registers (Continued)

Register Name	Description
MAC Frame Filter	Contains the filter controls for receiving frames. Some of the controls from this register go to the address check block of the MAC, which performs the first level of address filtering. The second level of filtering is performed on the incoming frame, based on other controls such as pass bad frames and pass control frames.
Hash Table High/Low ¹	A 64-bit hash table is used for group address filtering. For hash filtering, the contents of the destination address in the incoming frame passes through the CRC logic. The upper 6 bits of the CRC register index the contents of the hash table.
SMI Address ¹	Controls the management cycles to the external PHY through the Station Management interface. The register also includes a field to program the frequency of MDC.
SMI Data ¹	Stores write data for the PHY register at the address specified in SMI Address register. This register also stores read data from the PHY register at the address specified by SMI address register.
Flow Control ¹	Controls the generation and reception of the control (pause command) frames by the flow control module of the EMAC. The fields of the control frame are selected as specified in the 802.3x specification. The EMAC uses the pause time value from this register in the pause time field of the control frame. The host must make sure that the activate bit is cleared before writing to the register.
VLAN Tag ¹	Contains the IEEE 802.1Q VLAN tag to identify the VLAN frames. The MAC compares the 13th and 14th bytes of the receiving frame (length or type) with 16.h8100. The following 2 bytes are compared with the VLAN tag. If a match occurs, it sets the received VLAN bit in the receive frame status. The legal length of the frame increases from 1518 bytes to 1522 bytes.
Debug	Provides the status of all main modules of the transmit and receive datapaths and the FIFOs. An all-zero status indicates that the MAC core is in idle state (and FIFOs are empty) and no activity exists in the datapaths.
Interrupt Status	The contents of this register identify the events in the EMAC-CORE that can generate MMC and PTP-related interrupts.
Interrupt Mask	Enables the program to mask the interrupt signal because of the corresponding PTP event in the interrupt status register.
MAC Address0 High/Low ¹	Holds the upper or lower 16 bits of the MAC address of the station. The first DA byte that is received on the RMII interface corresponds to the LS byte (bits [7:0]) of the MAC address low register. For example, if 0x112233445566 is received (0x11 is the first byte) on the RMII as the destination address, then the macaddress0 register [47:0] is compared with 0x665544332211.
Operation Mode ¹	

*1 There must not be any further writes to these registers until the first write updates. Otherwise, the second write operation is not updated properly. For correct operation, the delay between two writes to the same register location must be at least 8 cycles of 50MHz RMII REFCLK.

NOTE: Refer to the “Register Description” section for the detailed bit-level explanation of the registers.

EMAC CORE Transmission Engine

The following modules constitute the transmission function (transmission engine components) of the EMAC:

- [Transmit Bus Interface Module \(TBU\)](#)
- [Transmit Frame Controller Module \(TFC\)](#)
- [Transmit Checksum Offload Engine \(TCOE\)](#)
- [Transmit Protocol Engine Module \(TPE\)](#)
- [Transmit Scheduler Module \(STX\)](#)
- [Transmit CRC Generator Module \(CTX\)](#)
- [Transmit Flow Control Module \(FTX\)](#)

Transmit Bus Interface Module (TBU)

This module interfaces the transmit path of the EMAC CORE with the MAC Layer FIFO interface. This module outputs the transmit status to the application at the end of normal transmission or collision.

Transmit Frame Controller Module (TFC)

The transmit frame controller regulates frames as well as converts the 32-bit input data into an 8-bit stream.

When the number of bytes received from the application falls below 60 (DA+SA+LT+DATA), the state machine automatically appends zeros to the transmitting frame. The state machine makes the data length exactly 46 bytes to meet the requirement for minimum data field of IEEE 802.3. The EMAC module can also be programmed to not append any padding.

The frame controller receives the computed CRC and appends it as the FCS field to the data transmitting out. When the EMAC is programmed to not append the CRC value to the end of Ethernet frames, the TFC module ignores the computed CRC. However, when the EMAC is programmed to append pads for frames (DA+SA+LT+DATA) less than 60 bytes, then the CRC is always appended at the end of padded frame.

Transmit Checksum Offload Engine (TCOE)

Communication protocols such as TCP and UDP implement checksum fields, which help determine the integrity of data transmitted over a network. The most widespread use of Ethernet is to encapsulate TCP and UDP over IP datagrams. Therefore, the EMAC has a checksum offload engine (COE) to support checksum calculation and insertion in the transmit path, and error detection in the receive path.

NOTE: The checksum for TCP, UDP, or ICMP is calculated over a complete frame, and then inserted into its corresponding header field. Because of this requirement, this function is enabled only when the transmit FIFO configuration is for store-and-forward mode. (The `EMAC_DMA0_OPMODE.TSF` bit is set.) If the MAC configuration is for threshold (cut-through) mode, the transmit COE is bypassed.

NOTE: Programs must make sure that the transmit FIFO is deep enough to store a complete frame before that frame transfers to the EMAC CORE transmitter. The program must enable the checksum insertion only in the frames that are less than the following number of bytes in size (even in the store-and-forward mode):

FIFO depth – PBL – 3 FIFO locations, where PBL is the programmed burst-length in the DMA bus mode register.

IP Header Checksum

In IPv4 datagrams, the 16-bit header checksum field indicates the integrity of the header fields (bytes 11 and 12 of the IPv4 datagram). The COE detects an IPv4 datagram when the Ethernet type field of the frame has the value 0x0800 and the version field of the IP datagram has the value 0x4. The checksum field of the input frame is ignored during calculation and replaced with the calculated value.

The IP header error status bit in transmit descriptor word TDES0 indicates the result of this IP header checksum calculation. The status bit is set whenever the values of the Ethernet type field and the IP header version field are not consistent. Or, the status bit is set when the Ethernet frame does not have enough data, as indicated by the IP header length field. In other words, this bit is set when an IP header error is asserted under the following circumstances.

For IPv4 datagrams:

- The received Ethernet type is 0x0800, but the version field of the IP header is not equal to 0x4.
- The IPv4 header length field indicates a value less than 0x5 (20 bytes).
- The total frame length is less than the value given in the IPv4 header length field.

For IPv6 datagrams:

- The Ethernet type is 0x86dd but the IP header version field is not equal to 0x6.
- The frame ends before the IPv6 header (40 bytes) or extension header (as given in the corresponding header length field in an extension header) is received.

If the COE detects an IP header error, it still inserts an IPv4 header checksum if the Ethernet type field indicates an IPv4 payload.

NOTE: IPv6 headers do not have a checksum field. Therefore, the COE does not modify the IPv6 header fields.

TCP/UDP/ICMP Checksum

The TCP/UDP/ICMP checksum engine processes the IPv4 or IPv6 header (including extension headers) and determines whether the encapsulated payload is TCP, UDP, or ICMP.

NOTE: See IETF specifications RFC 791, RFC 793, RFC 768, RFC 792, RFC 2460, and RFC 4443 for IPv4, TCP, UDP, ICMP, IPv6, and ICMPv6 packet header specifications, respectively.

NOTE: For non-TCP/UDP/ICMP/ICMPv6 payloads, this checksum engine is bypassed and nothing further is modified in the frame.

NOTE: For ICMP-over-IPv4 packets, the checksum field in the ICMP packet must always be 0x0000 in both modes, because pseudo-headers are not defined for such packets. If it does not equal 0x0000, an incorrect checksum can be inserted into the packet.

NOTE: This engine does not process fragmented IP frames (IPv4 or IPv6), IP frames with security features (such as an encapsulated security payload), and IPv6 frames with routing headers. The checksum engine bypasses the checksum insertion for such frames even if the checksum insertion is enabled.

The checksum is calculated for the TCP, UDP, or ICMP payload and inserted into its corresponding field in the header. This engine can work in the following two ways.

- The TCP, UDP, or ICMPv6 pseudo-header is not included in the checksum calculation and is assumed to be present in the checksum field of the input frame. This engine includes the checksum field in the checksum calculation, and then replaces the checksum field with the final calculated checksum.
- The engine ignores the checksum field, includes the TCP, UDP, or ICMPv6 pseudo-header data into the checksum calculation, and overwrites the checksum field with the final calculated value.

The status bit for the payload checksum error in the transmit descriptor word TDES0 indicates the result of this operation. The checksum engine sets the status bit for the payload checksum error when:

- The checksum engine detects that the frame has been forwarded to the MAC transmitter engine in the store-and-forward mode, *and*
- The end of frame (EOF) has not been written to the FIFO, *or*
- The packet ends before the number of bytes indicated by the payload length field in the IP header is received.

When the packet is longer than the indicated payload length, the COE ignores them as stuff bytes, and no error is reported. When the engine detects the first type of error, it does not modify the TCP, UDP, or ICMP header. For the second error type, it still inserts the calculated checksum into the corresponding header field.

Transmit checksum offloading is enabled by setting the CIC bits [23:22] of TDES0 word in the transmit descriptor.

Transmit Protocol Engine Module (TPE)

The transmit protocol engine consists of a state-machine that controls the protocol-level operation of Ethernet frame transmission. The module performs the following functions to meet the IEEE 802.3 specifications.

- Generates preamble and SFD
- Generates carrier extension in half-duplex mode (only in RGMII mode)
- Supports frame bursting in half-duplex mode (only in RGMII mode)
- Generates jam pattern in half-duplex mode
- Contains time stamp snapshot logic for IEEE 1588 support
- Jabber timeout
- Flow control for half-duplex mode (back pressure)
- Generates transmit frame status

When a new frame transmission is requested, the protocol engine sends out the preamble and SFD, followed by the data received. The preamble is defined as 7 bytes of 10101010 pattern. The SFD is defined as 1 byte of 10101011 pattern.

The collision window is defined as 1 slot time (512-bit times for 10/100 Mbps and 4096 bit times for 1000 Mbps). The jam pattern generation is applicable only to half-duplex mode, not to full-duplex mode. A collision can occur any time from the beginning of the frame to the end of the CRC field. When a collision happens, the state machine sends a 32-bit jam pattern of 0x55555555 on the RMII/RGMII. The pattern informs all other stations that a collision has occurred. If the collision happens during the preamble transmission phase, it completes the transmission of preamble and SFD and then sends the jam pattern. If the collision occurs after the collision window and before the end of the FCS field, it sends a 32-bit jam pattern. It also sets the late collision bit in the transmit frame status.

In RGMII half-duplex mode (1,000 Mbps), the transmit state machine ensures that all valid carrier events exceed a slot time of 4,096 bit times. To accomplish this, any transmit frame shorter than 512 bytes from the TFC module is extended using a carrier extension. This is signaled to the PHY using RGMII_TXCTL pin and sending 0x00 through RGMII_TXD.

The module maintains a jabber timer to cut off the transmission of Ethernet frames when the TFC module transfers more than 2,048 (default) bytes. The timeout changes to 10,240 bytes when the jumbo frame is enabled.

The transmit state machine uses the deferral mechanism for the flow control (back pressure) in half-duplex mode. When the application asks to stop receiving frames, the module sends a jam pattern of 32 bytes. It sends the pattern whenever it senses a reception of a frame. Transmit flow control must be enabled. This activity results in a collision and the remote station backs off.

The application can request a flow control signal by setting the EMAC_FLOWCTL.FCBBPA bit. If the application requests a frame transmission, then it is scheduled and transmitted even when the back pressure is activated. If the back pressure is activated for a long time, then the remote stations abort their transmissions due to excessive collisions. (For example, a long time is when more than 16 consecutive collision events occur.)

If PTP time stamping is enabled for the transmit frame, this block takes a snapshot of the PTP system time when the SFD is put onto the transmit bus.

Transmit Scheduler Module (STX)

The transmit scheduler is responsible for scheduling the frame transmission on the RMII RGMII/MII. The two major functions of this module are:

- Maintain the inter-frame gap between two transmitted frames.
- Follow the truncated binary exponential back-off algorithm for half-duplex mode.

The scheduler maintains an idle period of the configured inter-frame gap (EMAC_MACCFG.IFG bits) between any two transmitted frames. The scheduler starts its IFG counter when the carrier signal of the reduced media-independent interface goes inactive. In half-duplex mode and when IFG is configured for 96-bit times, the scheduler follows the rule of deference specified in Section 4.2.3.2.1 of the IEEE 802.3 specification. The module resets its IFG counter when a carrier is detected during the first two-thirds (64-bit times for all IFG values) of the IFG interval. If the

carrier is detected during the final one third of the IFG interval, the scheduler continues the IFG count and enables the transmitter after the IFG interval.

Transmit CRC Generator Module (CTX)

The transmit CRC generator module generates the CRC for the FCS field of the Ethernet frame (DA + SA + LT + DATA + PAD).

This module calculates the 32-bit CRC for the FCS field of the Ethernet frame. The following polynomial defines the encoding:

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1.$$

Transmit Flow Control Module (FTX)

The transmit flow control module generates pause frames and transmits them to the frame controller as necessary, in full-duplex mode. The application can request the flow control module to send a pause frame by setting the `EMAC_FLOWCTL.FCBBPA` bit.

If the application has requested flow control, the flow control module generates and transmits a single pause frame. The value of the pause time in the generated frame contains the programmed pause time value configured using the `EMAC_FLOWCTL.PT` bit. The module can extend the pause or end the pause prior to the time specified in the previously transmitted pause frame. To change the pause, the application must request another pause frame transmission after programming the `EMAC_FLOWCTL.PT` bit with an appropriate value.

If the flow control signal goes inactive prior to the sampling time, the flow control module transmits a pause frame with zero pause time. This event indicates to the remote end that the receive buffer is ready to receive new data frames.

Source Address, VLAN, and CRC Insertion, Replacement, or Deletion

The MAC supports the following functions for transmit frames:

- Source address insertion or replacement
- VLAN insertion, replacement, or deletion
- CRC replacement

Source Address Insertion or Replacement

The software can use the SA insertion or replacement feature to instruct the MAC to do the following for transmit frames:

- Insert the content of the MAC address registers in the SA field.
- Replace the content of the SA field with the content of the MAC address registers.

The software can enable the SA insertion or replacement feature for all transmit frames or selective frames.

- To enable SA insertion or replacement feature for all frames, program bits[30:28] of the MAC Configuration register.

- To enable the SA insertion or replacement feature for selective frames, program the SA insertion control field (TDES1 Bits [31:29]) in the first transmit descriptor of the frame. When bit 31 of TDES1 is set, the SA insertion control field indicates insertion or replacement by MAC address1 registers. When bit 31 of TDES1 is reset, it indicates insertion or replacement by MAC address 0 registers. If MAC address1 registers are not enabled, then EMAC uses MAC address0 registers for insertion or replacement. The choice is not based on the value of the most significant bit of the SA insertion control field.

When SA insertion is enabled, the application ensures that the frames sent to the MAC do not have the SA field. This functionality is because the MAC does not check the presence of SA field in the transmit frame. MAC inserts the content of the MAC address registers in the SA field. Similarly, when SA replacement is enabled, the application ensures that frames sent to the MAC have the SA field. The MAC replaces the 6 bytes, following the destination address field in the transmit frame, with the content of the MAC address registers.

VLAN Insertion, Replacement, or Deletion

The software can use the VLAN insertion, replacement, or deletion feature to instruct the MAC to do the following for transmit frames:

- Insert or replace the VLAN type field (C-VLAN or S-VLAN indicated by bit 19 (CSVL) of VLAN tag inclusion or replacement register and VLAN tag field in the transmit frame with bit [15:0], VLT, of VLAN tag inclusion or replacement register.
- Delete the VLAN type and VLAN tag fields in transmit frame.

The software can enable the VLAN insertion, replacement, or deletion feature for all transmit frames or selective frames. To enable this function for all transmit frames, program Bits[17:16] of the VLAN Tag Inclusion or Replacement register. To enable this function for selective, program the VLAN insertion control field (TDES0 Bits [19:18]) in the first transmit descriptor of the frame. When VLAN replacement or deletion is enabled, the MAC checks the presence of the VLAN type field (0x8100 or 0x88a8), after the Destination Address (DA) and SA fields, in the transmit frame. The replace or delete operation does not occur when the VLAN type field is not detected in the 2 bytes following the DA and SA fields. However, when VLAN insertion is enabled, the MAC does not check the presence of VLAN type field in the transmit frame. MAC inserts the VLAN type and VLAN tag fields.

CRC Replacement

The software can use the CRC replacement feature to instruct the MAC to replace the FCS field in the transmit frame with the CRC computed by the MAC. This feature works on per-frame basis. To enable the CRC replacement feature, program the CRC replacement control field (bit 24 of transmit descriptor word 0 (TDES0)) in the first transmit descriptor of the frame.

NOTE: This feature is valid only when disable CRC control (bit 27 in TDES0) is enabled. The software provides the FCS field in the transmit frame. If SA or VLAN insertion control is enabled, the MAC appends or replaces the FCS field with the computed CRC when Disable CRC Control is enabled or disabled, respectively.

The *CRC Replacement* table shows how CRC replacement is performed based on the values of bit 27 (DC) and bit 24 (CRCR) of transmit descriptor word 0 (TDES0).

Table 24-30: CRC Replacement

DC	CRCR	Description
0	x	Append CRC. When DC = 0, the MAC appends the computed CRC irrespective of the CRCR setting.
1		Replace CRC
1	0	No operation (User has appended the CRC)

Source Address Filtering

The *Source Address Filtering* table provides filtering possibilities for the source address using the EMAC AFM module. The MAC receive frame filter register ([EMAC_MACFRMFILT](#)) contains these bits.

Table 24-31: Source Address Filtering

Frame Type	SA Filter Operation			Result
	PR	SAIF	SAF	
Unicast	1	X	X	Pass all frames
	0	0	0	Pass on perfect or group filter match but do not drop failing frames
	0	1	0	Fail on perfect or group filter match but do not drop frame
	0	0	1	Pass on perfect or group filter match and drop failing frames
	0	1	1	Fail on perfect or group filter match and drop failing frames

EMAC CORE Reception Engine

The following are the functional blocks (reception engine components) in the receive path of the EMAC core.

- [Receive Protocol Engine Module \(RPE\)](#)
- [Receive CRC Module \(CRX\)](#)
- [Receive Frame Controller Module \(RFC\)](#)
- [Receive Flow Control Module \(FRX\)](#)
- [Receive Checksum Offload Engine \(RCOE\)](#)
- [Receive Bus Interface Unit Module \(RBU\)](#)
- [Address Filtering Module \(AFM\)](#)

Receive Protocol Engine Module (RPE)

The receive protocol engine is a state-machine that strips the preamble, SFD, and carrier extension (in 1000 Mbps half-duplex mode) of the received frame. Once the receive data valid signal ([ETH0_CRS](#)) signal of RMII or RXCTL

signal of RGMII becomes active, the protocol engine begins hunting for the SFD field from the receive modifier logic. Until then, the state machine drops the receiving preambles. Once the SFD is detected, it begins sending the data of the Ethernet frame to the frame controller, beginning with the first byte following the SFD (destination address).

NOTE: According to the IEEE 802.3 Ethernet specifications, the EMAC receiver does not need to look or check for the preamble pattern. It has to wait only for the SFD pattern to identify the start of a frame. Then the EMAC receiver accepts a frame even when no preamble is received before the SFD pattern.

If PTP time stamping is enabled, the RPE takes a snapshot of the PTP system time when detecting any SFD of the frame on the reduced media-independent interface. Unless the MAC filters out and drops the frame, this time stamp passes on to the application

The protocol engine also decodes the length or type field of the receiving Ethernet frame. The state machine sends the data of the frame up to the count specified in the length or type field if these conditions are met:

- The length or type field is less than 0x600
- The MAC is programmed for the auto CRC or PAD stripping option

It then starts dropping bytes (including the FCS field).

If the length or type field is greater than or equal to 0x600, the protocol engine sends all received Ethernet frame data to the frame controller. The transfer does not depend on the value of the programmed auto-CRC strip option.

The EMAC is programmed with the watchdog timer enabled (default setting). In this configuration, frames above 2,048 (10,240 if jumbo frame is enabled) bytes (DA + SA + LT + DATA + PAD + FCS) are cut off at the protocol engine. Set the `EMAC_MACCFG.WD` bit to disable this feature. However, even when the watchdog timer is disabled, frames greater than 16 KB are cut off and a watchdog timeout status is issued.

The EMAC supports loopback of transmitted frames onto its receiver. By default, the EMAC loopback function is disabled. Set the `EMAC_MACCFG.LM` bit to enable the function.

At the end of every received frame, the protocol engine generates received frame status and sends it to the frame controller. Control, missed frame, and filter fail status are added to the receive status in the frame controller.

Receive CRC Module (CRX)

The receive CRC module checks for any CRC errors in the receiving frame.

This module calculates the 32-bit CRC for the received frame that includes the destination address field through the FCS field (DA+SA+LT+DATA+PAD+FCS). The following generating polynomial defines the encoding.

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

Irrespective of the auto pad or CRC strip, the CRC module receives the entire frame to compute the CRC check for received frame.

Receive Frame Controller Module (RFC)

The main functions of the frame controller are:

- Converting the 8-bit stream data to 32-bit data
- Frame filtering
- Attaching the calculated IP checksum
- Updating the receive status

If the `EMAC_MACFRMFILT.RA` bit is set, the RFC module initiates the data transfer as soon as possible. At the end of the data transfer, the frame controller sends out the received frame status that includes the address filtering pass or fail status.

If the `EMAC_MACFRMFILT.RA` bit is reset, the frame controller performs frame filtering based on the destination or source address. (The application still must perform another level of filtering if it decides not to receive any bad frames like runt, CRC error frames, for example.) After receiving the destination or source address bytes, the frame controller checks the filter-fail signal from the AFM module for an address match. On detecting a filter-fail from AFM, the frame is dropped and not transferred to the application.

Receive Flow Control Module (FRX)

The receive flow controller detects the receiving pause frame and pauses the frame transmission for the delay specified within the received pause frame. The flow controller is enabled only in full-duplex mode. The EMAC uses the `EMAC_FLOWCTL.RFE` bit to enable or disable the function for pause frame detection.

Once the receive flow control is enabled, the flow controller begins monitoring the received frame destination address for any match with the multicast address of the control frame (0x0180C2000001). If a match is detected, it indicates to the frame controller, that the destination address of the received frame matches the reserved control frame destination address. The RFC module then decides whether to transfer the received control frame to the application, based on the `EMAC_MACFRMFILT.PCF` bit setting.

The receive flow controller also decodes the type, opcode, and pause timer field of the receiving control frame. The flow controller requests the MAC transmitter pause the transmission of any data frame.

- If the byte count of the frame status indicates 64 bytes, *and*
- If there is no CRC error

The transmission is paused for the decoded pause time value, multiplied by the slot time (64-byte times). Meanwhile, if another pause frame is detected with a zero pause time value, the module resets the pause time and gives another pause request to the transmitter. The module does not generate a pause request to the transmitter:

- If the received control frame does not match the type field (0x8808), opcode (0x00001), or byte length (64 bytes), *or*
- If there is a CRC error

For a pause frame with a multicast destination address, the frame controller filters the frame based on the address match from the flow controller. For a pause frame with a unicast destination address, the filtering in the FRX module depends on:

- If the destination address matched the contents of the MAC address register 0 (`EMAC_ADDR0_HI` or `EMAC_ADDR0_LO`), *and*
- If the `EMAC_FLOWCTL.UP` bit is set

The module detects a pause frame even with a unicast destination address. The EMAC uses the `EMAC_MACFRMFILT.PCF` bits to control the filtering for control frames in addition to the address filter module.

Receive Checksum Offload Engine (RCOE)

When checksum offloading is enabled, both IPv4 and IPv6 frames in the received Ethernet frames are detected and processed for data integrity. Programs can enable this module by setting the `EMAC_MACCFG.IPC` bit. The EMAC receiver identifies IPv4 or IPv6 frames by checking for value 0x0800 or 0x86DD, respectively, in the received Ethernet type field of frames. This identification applies to VLAN-tagged frames as well. *Extended descriptor mode (8 x32-bit words) must be enabled to get the IPC checksum engine status in RDES4.* To check status, poll bit 0 of RDES0 word of receive descriptor. Then, if this bit is set, parse bits [7:0] of RDES4 word.

The receive checksum offload engine calculates IPv4 header checksums and checks if they match the received IPv4 header checksums. The IP header error bit is set for any mismatch between the indicated payload type (Ethernet type field) and the IP header version. The IP header error bit is also set when the received frame does not have enough bytes, as indicated by the length field of the IPv4 header. (The bit is set when fewer than 20 bytes are available in an IPv4 or IPv6 header).

This engine also identifies a TCP, UDP, or ICMP payload in the received IP datagrams (IPv4 or IPv6). The engine calculates the checksum of such payloads properly, as defined in the TCP, UDP, or ICMP specifications. This engine includes the TCP/UDP/ICMPv6 pseudo-header bytes for checksum calculation and checks whether the received checksum field matches the calculated value. The result of this operation appears as a payload checksum error bit in the receive status word. This status bit is also set if the length of the TCP, UDP, or ICMP payload does not tally to the expected payload length given in the IP header.

NOTE: The COE engine bypasses the payload of fragmented IP datagrams, IP datagrams with security features, IPv6 routing headers, and payloads other than TCP, UDP, or ICMP. This information is given in the receive status(whether the checksum engine is bypassed or not).

The *Checksum Error Status* table shows bit combination in receive descriptors (frame status with full checksum offload engine enabled and advanced timestamps not enabled).

Table 24-32: Checksum Error Status

IEEE802.3 Frame: bit 5 of RDES0	Header Checksum Error (HCE): bit 3 of RDES4	Payload Checksum Error (PCE): bit 4 of RDES4	Frame Status
0	0	0	The frame is an IEEE 802.3 frame (length field value is less than 0x0600).
1	0	0	IPv4/IPv6 type frame in which no checksum error is detected.

Table 24-32: Checksum Error Status (Continued)

IEEE802.3 Frame: bit 5 of RDES0	Header Checksum Error (HCE): bit 3 of RDES4	Payload Checksum Error (PCE): bit 4 of RDES4	Frame Status
1	0	1	IPv4/IPv6 type frame in which a payload checksum error (as described for PCE) is detected
1	1	0	IPv4/IPv6 type frame in which IP header checksum error (as described for IPC HCE) is detected.
1	1	1	IPv4/IPv6 type frame in which both PCE and IPC HCE is detected.
0	0	1	IPv4/IPv6 type frame in which there is no IP HCE and the payload check is bypassed due to unsupported payload.
0	1	1	Type frame which is neither IPv4 or IPv6 (COE bypasses the checksum check completely)
0	1	0	Reserved

Receive Bus Interface Unit Module (RBU)

The receive bus interface unit (RBU) constructs the 32-bit data received from the frame controller into a 32-bit FIFO-based protocol.

Address Filtering Module (AFM)

The address filtering (AFM) module performs the destination checking function on all received frames and reports the address filtering status to the frame controller. The address checking is based on different parameters (frame filter register, `EMAC_MACFRMFILT`) chosen by the application. These parameters are inputs to the AFM module as control signals. The AFM module reports the status of the address filtering based on the combination of these inputs. The AFM module also reports whether the receiving frame is a multicast frame or a broadcast frame, as well as the address filter status. The AFM module uses the physical (MAC) address of the station and the multicast hash table for address checking.

- **Hash or Perfect Address Filter.** The destination address filter can be configured to pass a frame when its destination address matches either the hash filter or the perfect filter. Set the `EMAC_MACFRMFILT.HPF` bit, the corresponding `EMAC_MACFRMFILT.HUC`, or `EMAC_MACFRMFILT.HMC` bits. This configuration applies to both unicast and multicast frames. If the `EMAC_MACFRMFILT.HPF` bit is reset, only one of the filters (hash or perfect) is applied to the received frame.

NOTE: Hash filtering is not perfect filtering because a 48-bit MAC address is reduced to a 6-bit hash value. So, there can be instances where more than one address has the same hash value.

- **Unicast Destination Address Filter.**
 - The AFM supports one MAC address two MAC addresses for unicast perfect filtering. If perfect filtering is selected, the AFM compares all 48 bits of the received unicast address with the programmed MAC address for any match. (The `EMAC_MACFRMFILT.HUC` bit is reset for perfect filtering).

- In hash filtering mode, the AFM performs imperfect filtering for unicast addresses using a 64-bit hash table. (The `EMAC_MACFRMFILT.HUC` bit is set in hash filtering mode.) For hash filtering, the AFM uses the upper 6-bit CRC of the received destination address to index the content of the hash table. A value of 000000 selects bit 0 of the selected register, and a value of 111111 selects bit 63 of the hash table register. If the corresponding bit (indicated by the 6-bit CRC) is set to 1, the unicast frame has passed the hash filter. Otherwise, the frame has failed the hash filter.
- **Multicast Destination Address Filter.**
 - Program the EMAC to pass all multicast frames by setting the `EMAC_MACFRMFILT.PM` bit. If the `EMAC_MACFRMFILT.PM` bit is reset, the AFM filters multicast addresses based on the `EMAC_MACFRMFILT.HMC` bit. In perfect filtering mode, the multicast address is compared with the programmed MAC destination address register. The EMAC also supports group address filtering.
 - In hash filtering mode, the AFM performs imperfect filtering using a 64-bit hash table. For hash filtering, the AFM uses the upper 6-bit CRC of the received multicast address to index the content of the hash table. A value of 000000 selects bit 0 of the selected register and a value of 111111 selects bit 63 of the hash table register. If the corresponding bit is set to 1, then the multicast frame has passed the hash filter. Otherwise, the frame has failed the hash filter.
- **Broadcast Address Filter.** The AFM does not filter any broadcast frames in the default mode. However, if the EMAC is programmed to reject all broadcast frames by setting the `EMAC_MACFRMFILT.DBF` bit, the AFM asserts the filter fail signal, whenever a broadcast frame is received.
- **Unicast Source Address Filter.** EMAC can also perform a perfect filtering, based on the source address field of the received frames. By default, the AFM compares the SA field with the values programmed in the SA registers. The MAC address register 1 can be configured to contain SA instead of DA for comparison, by setting bit 30 of the corresponding register. Group filtering with SA is also supported. User can filter a group of addresses by masking one or more bytes of the address. The frames that fail the SA filter are dropped by the MAC if the `EMAC_MACFRMFILT.SAF` bit is set. When the bit is set, the result of SA filter and DA filter is AND'ed to decide whether the frame needs to be forwarded. This means that either of the filter fail result drops the frame and both filters have to pass in order to forward the frame to the application.
- **Inverse Filtering Operation.** There is an option to invert the filter-match result at the final output. The EMAC uses the `EMAC_MACFRMFILT.DAIF` bit to control this operation. The function of this bit applies to both unicast and multicast DA frames. The result of the unicast or multicast destination address filter is inverted in this mode.
- **Inverse Filtering Operation.** For both destination and source address filtering, there is an option to invert the filter-match result at the final output. This is controlled by the `EMAC_MACFRMFILT.DAIF` or `EMAC_MACFRMFILT.SAIF` bits. The `EMAC_MACFRMFILT.DAIF` bit is applicable for both unicast and multicast DA frames. The result of the unicast /multicast destination address filter is inverted in this mode. Similarly, when the `EMAC_MACFRMFILT.SAIF` bit is set, the result of unicast source address filter is reversed.

- **Inverse Filtering Operation.** There is an option to invert the filter-match result at the final output. The EMAC uses the `EMAC_MACFRMFILT.DAIF` bit to control this operation. The function of this bit applies to both unicast and multicast DA frames. The result of the unicast or multicast destination address filter is inverted in this mode.

Destination Address Filtering

The *Destination Address Filtering* table provides filtering possibilities for the destination address using the EMAC AFM module. The MAC receive frame filter register (`EMAC_MACFRMFILT`) contains these bits.

Table 24-33: Destination Address Filtering

Frame Type	Bit Setting (0 = Cleared, 1 = Set, X = Do-not-care)							DA Filter Operation
	PR	HPF	HUC	HMC	DAIF	PM	DBF	
Broadcast	1	X	X	X	X	X	X	Pass
	0	X	X	X	X	X	0	Pass
	0	X	X	X	X	X	1	Fail
Unicast	1	X	X	X	X	X	X	Pass all frames
	0	X	0	X	0	X	X	Pass on perfect or group filter match
	0	X	0	X	1	X	X	Fail on perfect or group filter match
	0	0	1	X	0	X	X	Pass on hash filter match
	0	0	1	X	1	X	X	Fail on hash filter match
	0	1	1	X	0	X	X	Pass on hash or perfect or group filter match
	0	1	1	X	1	X	X	Fail on hash or perfect or group filter match
Multicast	1	X	X	X	X	X	X	Pass all frames
	X	X	X	X	X	1	X	Pass all frames
	0	X	X	0	0	0	X	Pass on perfect or group filter match and drop PAUSE control frames if PCF = 0x
	0	0	X	1	0	0	X	Pass on hash filter match and drop PAUSE control frames if PCF = 0x
	0	1	X	1	0	0	X	Pass on hash or perfect or group filter match and drop PAUSE control frames if PCF = 0x

Table 24-33: Destination Address Filtering (Continued)

Frame Type	Bit Setting (0 = Cleared, 1 = Set, X = Do-not-care)							DA Filter Operation
	PR	HPF	HUC	HMC	DAIF	PM	DBF	
	0	X	X	0	1	0	X	Fail on perfect or group filter match and drop PAUSE control frames if PCF = 0x
	0	0	X	1	1	0	X	Fail on hash filter match and drop PAUSE control frames if PCF = 0x
	0	1	X	1	1	0	X	Fail on hash or perfect or group filter match and drop PAUSE control frames if PCF = 0x

Source Address Filtering

The *Source Address Filtering* table provides filtering possibilities for the source address using the EMAC AFM module. The MAC receive frame filter register ([EMAC_MACFRMFILT](#)) contains these bits.

Table 24-34: Source Address Filtering

Frame Type	SA Filter Operation			Result
	PR	SAIF	SAF	
Unicast	1	X	X	Pass all frames
	0	0	0	Pass on perfect or group filter match but do not drop failing frames
	0	1	0	Fail on perfect or group filter match but do not drop frame
	0	0	1	Pass on perfect or group filter match and drop failing frames
	0	1	1	Fail on perfect or group filter match and drop failing frames

VLAN Tag Based Filtering

EMAC0 supports VLAN tag perfect filtering and VLAN tag hash filtering.

VLAN Tag Perfect Filtering

In VLAN tag perfect filtering, the MAC compares the VLAN tag of the received frame and provides the VLAN frame status to the application. Based on the programmed mode, the MAC compares the lower 12 bits or all 16 bits of the received VLAN tag to determine the perfect match. If VLAN tag perfect filtering is enabled, the MAC forwards the VLAN-tagged frames along with VLAN tag match status. It drops the VLAN frames that do not match. Inverse matching for VLAN frames can be enabled using the `EMAC_VLANTAG.VTIM` bit. In addition, matching of S-VLAN tagged frames along with the default C-VLAN tagged frames can be enabled using `EMAC_VLANTAG.ESVL` bit. The VLAN frame status bit (Bit 10 of `RDES0`) indicates the VLAN tag match status for the matched frames.

NOTE: The source or destination address (if enabled) has precedence over the VLAN tag filters. A frame which fails the source or destination address filter is dropped irrespective of the VLAN tag filter results. By default, the VLAN tag-based perfect filter is available in all configurations.

VLAN Tag Hash Filtering

The MAC provides VLAN tag hash filtering with a 16-bit hash table. The MAC performs the VLAN hash matching based on the `EMAC_VLANTAG.VTHM` bit setting. If the `EMAC_VLANTAG.VTHM` bit is set, the most significant 4 bits of VLAN tag's CRC-32 are used to index the content of the VLAN hash table register (`EMAC_VLAN_HSHTBL`). A value of 1 in the `EMAC_VLAN_HSHTBL` register, corresponding to the index, indicates that the VLAN tag of the frame matched and the packet is forwarded. A value of 0 indicates that VLAN-tagged frame is dropped.

The MAC also supports the inverse matching of the VLAN frames. In the inverse matching mode, when the VLAN tag of a frame matches the perfect or hash filter, the packet is dropped. If the VLAN perfect and VLAN hash match are enabled, a frame matches if either the VLAN hash or the VLAN perfect filter matches. When inverse match is set, a packet is forwarded only when both perfect and hash filters indicate mismatch. The *VLAN Matching and Final VLAN Match Status* table shows the possibilities for VLAN matching and the final VLAN match status. When the `EMAC_MACFRMFILT.RA` bit is set, all frames are received and the VLAN match status is indicated in bit 10 of receive descriptor word 0 (RDES0). When the `EMAC_MACFRMFILT.RA` bit is not set and the `EMAC_MACFRMFILT.VTFE` bit register is set, the frame is dropped when the final VLAN match status is fail.

When VLAN VID is programmed to 0 in the `EMAC_VLANTAG.VL` bit field, all VLAN-tagged frames are perfect matches. But the status of the VLAN hash match depends on the VLAN hash enable bit and VLAN inverse filter bit. The *VLAN Matching and Final VLAN Match Status* table shows the possibilities for VLAN matching and the final VLAN match status.

Table 24-35: VLAN Matching and Final VLAN Match Status

VID	VLAN Perfect Filter Match Status (VPF)	VLAN Hash Enable Bit	VLAN Hash filter Match Status (VTHMS)	VLAN Inverse Filter Bit (VTIM)	Final VLAN Match Status
VID=0	Pass	0	X	X	Pass
	Pass	1	X	0	Pass
	Pass	1	Fail	1	Pass
	Pass	1	Pass	1	Fail
VID!=0	Pass	X	X	0	Pass
	Fail	0	X	0	Fail
	Fail	1	Fail	0	Fail
	Fail	1	Pass	0	Pass
	Fail	0	X	1	Pass
	Pass	X	X	1	Fail

Table 24-35: VLAN Matching and Final VLAN Match Status (Continued)

VID	VLAN Perfect Filter Match Status (VPF)	VLAN Hash Enable Bit	VLAN Hash filter Match Status (VTHMS)	VLAN Inverse Filter Bit (VTIM)	Final VLAN Match Status
	Fail	1	Pass	1	Fail
	Fail	1	Fail	1	Pass

Layer 3 and Layer 4 Frame Filtering

The MAC supports Layer 3 and Layer 4 based frame filtering. The Layer 3 filtering refers to the IP source or destination address filtering in the IPv4 or IPv6 frames whereas Layer 4 filtering refers to the source or destination port number filtering in TCP or UDP. The Layer 3 and Layer 4 frame filtering feature automatically enables the IPC full checksum offload engine on the receive side.

When Layer 3 and Layer 4 filtering is enabled, the frames are filtered in the following way:

Matched Frames. The MAC forwards the frames, which match all enabled fields, to the application along with the status. The MAC gives the matched field status only if the `EMAC_MACCFG . IPC` bit is set. The MAC forwards the frames, which match all enabled fields, to the application along with the status.

NOTE: The source or destination address and VLAN tag filters (if enabled) have precedence over Layer 3 and Layer 4 filtering. This means that a frame which fails the source or destination address or VLAN tag filter is dropped irrespective of the Layer 3 and Layer 4 filter results.

Unmatched Frames. The MAC drops the frames that do not match any of the enabled fields. You can use the inverse match feature to block or drop a frame with specific TCP or UDP over IP fields and forward all other frames. If the Rx FIFO operates in the threshold mode and threshold is programmed to a small value, such that frame transfer to application starts before the failed Layer 3 and Layer 4 filter results are available, a partial frame with appropriate abort status may be received by the application.

Non-TCP or UDP IP Frames. By default, all non-TCP or UDP IP frames are bypassed from the Layer 3 and Layer 4 filters. You can optionally program the MAC to drop all non-TCP or UDP over IP frames.

Layer 3 and Layer 4 Filters Register Set

The MAC implements a set of registers for Layer 3 and Layer 4 based frame filtering. In a register set, there is a control register, such as the `EMAC_L3L4_CTL` register (Layer 3 and Layer 4 control register), to control the frame filtering. In addition, there are five address registers to program the Layer 3 and Layer 4 fields to be matched, which are:

- `EMAC_L4_ADDR` (Layer 4 Address Register)
- `EMAC_L3_ADDR0` (Layer 3 Address 0 Register)
- `EMAC_L3_ADDR1` (Layer 3 Address 1 Register)
- `EMAC_L3_ADDR2` (Layer 3 Address 2 Register)

- [EMAC_L3_ADDR3](#) (Layer 3 Address 3 Register)

Layer 3 Filtering

The MAC supports perfect matching or inverse matching for IP source and destination address. In addition, a program can match the complete IP address or mask the lower bits matching, that is, compare all bits of the address except the specified lower mask bits.

For IPv6 frames filtering, programs can enable the last four data registers of a register set to contain the 128-bit IP source or IP destination address. Program the IP source or destination address in the order defined in the IPv6 specification, that is, the first byte of the IP source or destination address in the received frame is in the higher byte of the register and the subsequent registers follow the same order.

For IPv4 frames filtering, a program can enable the second and third data registers of a register set to contain the 32-bit IP source address and IP destination address. The remaining two data registers are reserved. Program the IP source and destination address in the order defined in the IPv4 specification, that is, the first byte of IP source and destination address in the received frame in the higher byte of the respective register.

Layer 4 Filtering

The MAC supports perfect matching or inverse matching for TCP or UDP source and destination port numbers. Only one type (TCP or UDP) is programmable at a time. The first data register contains the 16-bit source and destination port numbers of TCP or UDP, that is, the lower 16 bits for source port number and higher 16 bits for destination port number. Program the TCP or UDP source and destination port numbers in the order defined in the TCP or UDP specification, that is, the first byte of TCP or UDP source and destination port number in the received frame is in the higher byte of the register.

EMAC Station Management Interface (SMI)

The IEEE 802.3 MII station management interface, also known as the MDIO management interface, allows the processor to monitor and control one or more external Ethernet physical-layer transceivers. (Physical-layer transceivers are commonly called PHYs). The management interface physically consists of a 2-wire serial connection composed of the MDC (management data clock) output signal and the MDIO (management data input/output) bidirectional data signal. The IEEE 802.3 MII station management interface also applies to RMII .

The application can address only one register in the PHY in any given time and send control data or receive status information. All the transfers are initiated by the EMAC CORE, and the PHY chip only acts as a slave device.

Standard PHY control and status registers typically provide

- Device capability status bits (for example: auto-negotiation, duplex modes, 10/100 speeds, and protocols)
- Device status bits (for example: auto-negotiation complete, link status, remote fault)
- Device control bits (for example: reset, speed selection, loopback, and auto-negotiation start)

Upon power-up, an MDIO read access (at default rates) of device capabilities in PHY status registers can determine the supported PHY features.

The MII management logical interface specifies:

- A set of 16-bit device control or status registers within the PHYs, including both required registers with standardized bit definitions as well as optional vendor-specified registers.
- A 5-bit device addressing scheme which allows the MAC to select one of up to 32 externally connected PHY devices.
- A 5-bit register addressing scheme for selecting the target register within the addressed device.
- A transfer frame protocol for 16-bit read and write accesses to PHY registers through the MDC and MDIO signals under control of the MAC.

Table 24-36: Station Management Interface pins

Station Management Interface Pins	Pin Description
MDIO – Management Data I/O	A periodic clock that runs at a maximum period of 400 ns. Always driven by the EMAC to PHY.
MDC – Management Data Clock	Data signal driven by EMAC or PHY, depending on write or read access based on EMAC; synchronous to MDC.

MDC Clock Frequency

The EMAC uses the `EMAC_SMI_ADDR.CR` bit field to determine the frequency of MDC as shown in the *MDC Clock Frequency Selection* table. The clock range selection determines the frequency of the clock relative to the SCLK0 frequency. The table shows the suggested range of SCLK0 frequency applicable for each value of the `EMAC_SMI_ADDR.CR` field. The programmability based on SCLK0 frequency range ensures that the MDC clock frequency range is within the IEEE specifications of 1.0 MHz to 2.4 MHz. However, the EMAC MDC can also support higher frequencies for PHY devices that support the frequencies.

Table 24-37: MDC Clock Frequency Selection

EMAC_SMI_ADDR.CR Selection	Programmed SCLK0 Frequency Range	Frequency of MDC	Min and Max MDC Freq (Per Specifications)
0000	60–100 MHz	SCLK0/42	MIN = 1.43 MHz and MAX = 2.39 MHz
0010	20–35 MHz	SCLK0/16	MIN = 1.25 MHz and MAX = 2.19 MHz
0011	35–60 MHz	SCLK0/26	MIN = 1.35 MHz and MAX = 2.31 MHz

The *MDIO Frame Parameters* table provides MDIO data transfer parameters. The write and read sequences provided in the tables, *MDIO Write Data Sequence* and *MDIO Read Data Sequence*, are based on these parameters.

Table 24-38: MDIO Frame Parameters

Parameter	Description
IDLE	The MDIO line is three-state (noted as Z in sequence); there is no clock on MDC.
PREAMBLE	32 continuous bits, each of value 1
START	Start of frame is 01
OPCODE	10 for read and 01 for write
PHY ADDR	5-bit address select for one of 32 PHYs (noted as AAAAA in sequence)
REG ADDR	Register address in the selected PHY (noted as RRRRR in sequence)
TA	Turnaround is Z0 for read and 10 for write (Z = high impedance)
DATA	Any 16-bit value. Driven by MAC or PHY based on direction (noted as DDD...DDD).

Table 24-39: MDIO Write Data Sequence

IDLE	PREAMBLE	START	OPCODE	PHY ADDR	REG ADDR	TA	DATA	IDLE
Z	111...111	01	01	AAAAA	RRRRR	10	DDD... DDD	Z

Table 24-40: MDIO Read Data Sequence

IDLE	PREAMBLE	START	OPCODE	PHY ADDR	REG ADDR	TA	DATA	IDLE
Z	111...111	01	10	AAAAA	RRRRR	Z0	DDD... DDD	Z

SMI Write Operation

When programs set the `EMAC_SMI_ADDR.SMIW` (write) and `EMAC_SMI_ADDR.SMIB` (busy) bits, the station management interface (SMI) initiates a write operation into the PHY registers. The write operation uses the management frame format (the PHY address, the register address in PHY, and the write data) specified in the IEEE specifications. (Section 22.2.4.5 of IEEE standard). The application must not change the `EMAC_SMI_ADDR` register contents or the `EMAC_SMI_DATA` register while the transaction is ongoing.

Write operations to the `EMAC_SMI_ADDR` register or the `EMAC_SMI_DATA` register during the transfer period are ignored (while the `EMAC_SMI_ADDR.SMIB` bit is high). The transaction completes without error. After the write operation has completed, the SMI indicates the same by resetting the `EMAC_SMI_ADDR.SMIB` bit. The EMAC drives the MDIO line for the complete duration of the frame. The *SMI Write Operation through MDIO/MDC Pins* figure shows this operation.

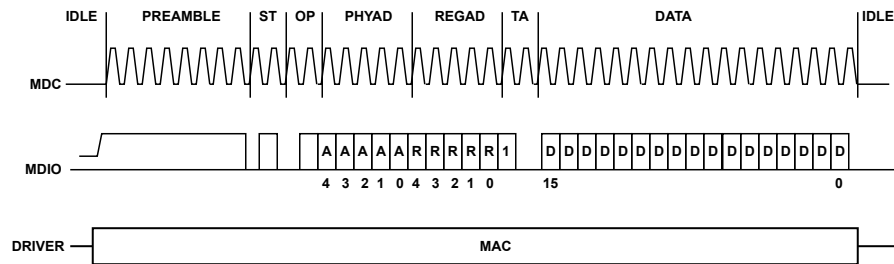


Figure 24-16: SMI Write Operation through MDIO/MDC Pins

SMI Read Operation

When programs set the `EMAC_SMI_ADDR.SMIB` bit with the `EMAC_SMI_ADDR.SMIW` bit cleared (`=0`), the station management interface (SMI) initiates a read operation in the PHY registers. It transfers the PHY address and the register address in the PHY to the SMI. The application must not change the `EMAC_SMI_ADDR` register contents or the `EMAC_SMI_DATA` register while the transaction is ongoing.

Write operations to the `EMAC_SMI_ADDR` register or the `EMAC_SMI_DATA` register during the transfer period are ignored (while the `EMAC_SMI_ADDR.SMIB` bit is high). The transaction completes without error. After the read operation has completed, the SMI resets the `EMAC_SMI_ADDR.SMIB` bit and updates the `EMAC_SMI_DATA` register with the data read from the PHY. The EMAC drives the MDIO line for the complete duration of the frame except during the data fields when the PHY drives the MDIO line. The *SMI Read Operation through MDIO/MDC Pins* figure shows this operation.

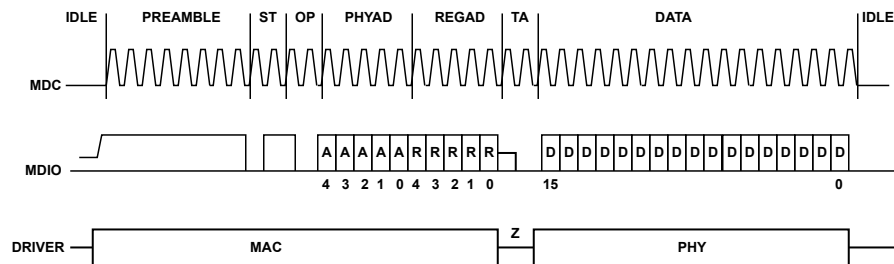


Figure 24-17: SMI Read Operation through MDIO/MDC Pins

EMAC Management Counters (MMC)

The EMAC provides a comprehensive set of 32-bit MAC management counters. It uses these counters for gathering statistics on the received and transmitted frames. The MMC subblock also includes

- A control register (`EMAC_MMC_CTL`) for managing the behavior of the counters
- Two 32-bit registers containing interrupts generated (`EMAC_MMC_RXINT` and `EMAC_MMC_TXINT`)
- Two 32-bit registers containing masks for the interrupt register (`EMAC_MMC_RXIMSK` and `EMAC_MMC_TXIMSK`)

The MMC receive counters are updated for frames passed by the address filtering subblock in the EMAC CORE. Statistics of frames dropped by the AFM module are not updated unless they are runt frames of less than 6 bytes. (Destination address bytes are not received fully.) The module is also capable of gathering statistics on encapsulated IPv4, IPv6, and TCP, UDP, or ICMP payloads in received Ethernet frames.

The MMC register naming conventions are as follows:

- Tx as a prefix or suffix indicates counters associated with transmission.
- Rx as a prefix or suffix indicates counters associated with reception.
- _G as a suffix indicates registers that count good frames only.
- _GB as a suffix indicates registers that count frames regardless of whether they are good or bad.

Transmitted frames are considered *good* when transmitted successfully. In other words, a transmitted frame is good if the frame transmission does not abort due to any of the following errors:

- Jabber timeout
- No carrier or loss of carrier
- Late collision
- Frame underflow
- Excessive deferral
- Excessive collision

Received frames are good when none of the following errors exists:

- CRC error
- Runt frame (shorter than 64 bytes)
- Alignment error
- Length error (non-type frames only)
- Out-of-range (non-type frames only, longer than maximum size)

The maximum frame size depends on the frame type, as follows:

- Untagged frame maxsize = 1518
- VLAN frame maxsize = 1522
- Jumbo frame maxsize = 9018
- Jumbo VLAN frame maxsize = 9022

The `EMAC_MMC_CTL` register also contains bits that control preset, freeze and roll-over of counters. The EMAC uses the `EMAC_MMC_CTL.RDRST` bit to enable an auto-reset feature whenever the counters are read. The EMAC uses the `EMAC_MMC_CTL.RST` bit to reset all the counters.

The MMC can trigger an interrupt when the corresponding bits are enabled in the transmit, receive, and IPC mask registers, and when the particular counter reaches half or full. The status is also updated in the corresponding interrupt register.

MMC Receive Interrupt Register

The `EMAC_MMC_RXINT` register maintains the interrupts that are generated when the receive statistic counters reach half their maximum values (0x80000000), and when they cross their maximum values (0xFFFFFFFF). When `EMAC_MMC_CTL.NOROLL` is set, then interrupts are set, but the counter remains at all ones. The `EMAC_MMC_RXINT` register is a 32-bit wide register. An interrupt bit is cleared when the respective MMC counter that caused the interrupt is read. The least significant byte lane (bits 7:0) of the respective counter must be read to clear the interrupt bit.

MMC Transmit Interrupt Register

The `EMAC_MMC_TXINT` register maintains the interrupts generated when the transmit statistic counters reach half their maximum values (0x80000000), and when they cross their maximum values (0xFFFFFFFF). When `EMAC_MMC_CTL.NOROLL` is set, then interrupts are set, but the counter remains at all ones. The `EMAC_MMC_TXINT` register is a 32-bit wide register. An interrupt bit is cleared when the respective MMC counter that caused the interrupt is read. The least significant byte lane (bits 7:0) of the respective counter must be read to clear the interrupt bit.

MMC Receive Checksum Offload Interrupt Register

The `EMAC_MMC_RXINT.CRCERR` register maintains the interrupts generated when receive IPC statistic counters reach half their maximum values (0x80000000), and when they cross their maximum values (0xFFFFFFFF). When `EMAC_MMC_CTL.NOROLL` is set, then interrupts are set, but the counter remains at all ones. The `EMAC_MMC_RXINT.CRCERR` register is 32 bits wide. When the MMC IPC counter that caused the interrupt is read, its corresponding interrupt bit is cleared. The least-significant byte lane (bits 7:0) of the counter must be read to clear the interrupt bit.

EMAC Precision Time Protocol (PTP) Engine

The following sections describe the precision time protocol (PTP) engine.

IEEE1588 and the PTP Engine

The Ethernet MAC peripheral includes a PTP engine to assist applications requiring time synchronization. The PTP module is tightly integrated with the EMAC CORE to aid hardware time stamping defined in the IEEE1588 2002/2008 standards. Applications can use accurate hardware time stamps through TCP/IP stacks (if using network layer communication) to exchange time information across devices connected over network. Applications can also use accurate hardware timestamps through Ethernet device drivers (if using MAC layer communication) to exchange time information.

PTP Engine

For calculation of drift in time between two Ethernet devices, the device records its system time whenever a timing message is sent or received (IEEE 1588 protocol). Due to the indeterministic delay of a software system for a node, the software is unable to capture an accurate time when the message is sent or received. However, the hardware can monitor the signal on the communication media and get an accurate message of arrival and departure time.

The PTP (precision time protocol) module is closely integrated with the EMAC module. It provides hardware assistance to implement both the IEEE 1588–2002 and IEEE 1588–2008 standards on Ethernet (IEEE 802.3). It takes

one input clock signal as its PTP clock and maintains the timing information (called *system time*) at the nanosecond level.

The PTP module includes hardware for clock and system time adjustment. The pulse-per-second (PPS) signals physically represent the system time. PPS can be programmed to a fixed frequency or provide flexibility to the signal in terms of pulse width, interval, start, and stop time of the signals. The PTP module can be programmed to trigger an alarm interrupt when system time reaches specified time.

The PTP module can be programmed to detect different types of received frames, capture the system time, and time stamp those frames with the captured system time. Programs can configure any frame so that the PTP module capture the system time when it is transmitted. The PTP module can also capture the system time when an event is detected on the auxiliary snapshot trigger input pins (EMAC_PTPAUXIN[n]).

IEEE1588 Standard

Many systems require two independent devices to operate in a time synchronized fashion. If each system relied solely on its oscillator, differences between the characteristics and operating conditions of each oscillator would limit the ability of the clocks to operate synchronously. To serve applications requiring synchronized clocks, the system uses a periodic correction mechanism.

A simple way to synchronize multiple systems is to choose one system (with the best clock) as a master. The system master broadcasts the clock and timing information to other systems (slaves) and then the slaves adjust their clocks and timing according to that of master. However, this method has limitations. The master cannot broadcast the time at infinitesimal intervals. Path delay (propagation delay) exists between a master and a slave and the delay varies between each slave and master.

IEEE 1588 is also known as precision time protocol or PTP. The standard specifies a protocol used to synchronize the time and clock of multiple devices, dispersed but interconnected by any communication. For example, Ethernet (IEEE 802.3). According to the protocol, timing messages are exchanged between two devices. Then, one of the devices calculates its drift from other device and corrects its system time. (Both devices must have the same representation of their system time). The protocol resolves path delay between devices. It also helps synchronize the clocks of multiple devices and all of the limitations mentioned are resolved.

IEEE 1588 was published in 2002 where four types of timing messages were defined: Sync, Follow_Up, Delay_Req, and Delay_Resp. Here the protocol synchronizes two or more devices where one is a master and others are slaves. The master device sends Sync, Follow_Up, and Delay_Resp messages to the slave device in the system. The slave sends the Delay_Req messages to the master device. A following section provides more information on IEEE 1588–2002.

In 2008 a newer version of IEEE 1588 was introduced which provides further mechanisms to measure the peer-to-peer delay. Three more timing messages (PdelayReq, PdelayResp, and PdelayRespFollowup) were added to implement peer-to-peer synchronization. A following section provides more information on IEEE 1588–2008.

IEEE 1588–2002

The IEEE 1588–2002 standard defines the precision time protocol (PTP). The protocol allows precise synchronization of clocks in measurement and control systems that use network communication, local computing, and distributed objects. The protocol applies to systems that communicate by local area networks that support multicast

messaging, including (but not limited to) Ethernet. This protocol also allows heterogeneous systems that include clocks of varying inherent precision, resolution, and stability to synchronize. The protocol supports system-wide synchronization accuracy in the submicrosecond range with minimal network and local clock computing resources.

The PTP is transported over UDP/IP. The system or network is classified into master and slave nodes for distributing the timing or clock information. The *IEEE 1588–2002 PTP Process* figure shows the process that PTP uses for synchronizing a slave node to a master node by exchanging PTP messages.

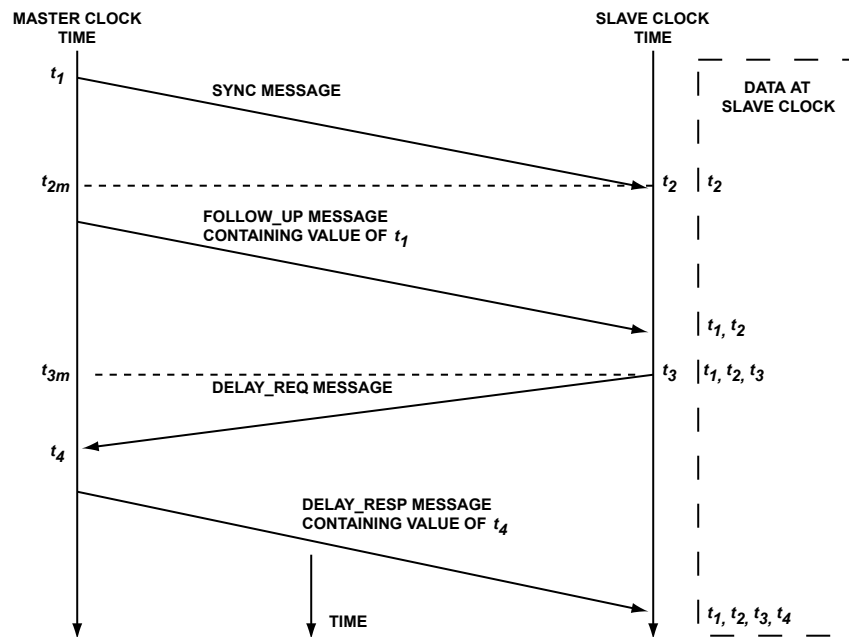


Figure 24-18: IEEE 1588–2002 PTP Process

As shown in the figure, the PTP uses the following process:

1. The master broadcasts the PTP sync messages to all its nodes. The sync message contains the reference time information of the master. The time at which this message leaves the system of the master is t_1 . The master must capture this time for Ethernet ports, at RMII.
2. The slave receives the Sync message and also captures the exact time, t_2 , using its timing reference.
3. The master sends a Follow_up message to the slave, which contains t_1 information for later use.
4. The slave sends a Delay_Req message to the master, noting the exact time, t_3 , at which this frame leaves the RMII.
5. The master receives the message, capturing the exact time, t_4 , at which it enters its system.
6. The master sends the t_4 information to the slave in the Delay_Resp message.
7. The slave uses the four values of t_1 , t_2 , t_3 , and t_4 to synchronize its local timing reference to the timing reference of the master.

Most of the PTP implementation happens in the software above the UDP layer. However, the hardware support must capture the exact time when specific PTP packets enter or leave the Ethernet port at the RMII/RGMII. Hardware must capture this timing information and return it to the software for the proper implementation of PTP with high accuracy.

IEEE 1588–2008 Advanced Time Stamps

In addition to the basic time stamp features mentioned in IEEE 1588–2002 time stamps, the EMAC supports the following advanced time stamp features defined in the IEEE 1588–2008 standard.

- Support for the IEEE 1588–2008 (Version 2) time stamp format.
- Provides an option to take snapshot of all frames or only PTP type frames.
- Provides an option to take snapshot of only event messages.
- Provides an option to select the node to be a master or slave.
- Identifies the PTP message type, version, and PTP payload in frames sent directly over Ethernet and sends the status.
- Provides an option to run nanoseconds time in digital or binary format.

Peer-to-Peer (P2P) PTP Message Support

The IEEE 1588–2008 version supports Peer-to-Peer PTP (Pdelay) message in addition to SYNC, Delay Request, Follow-up, and Delay Response messages. Refer to the *Propagation Delay Calculation between Nodes Supporting Peer-to-Peer Path Correction* figure. The figure shows the method to calculate the propagation delay between nodes supporting peer-to-peer path correction.

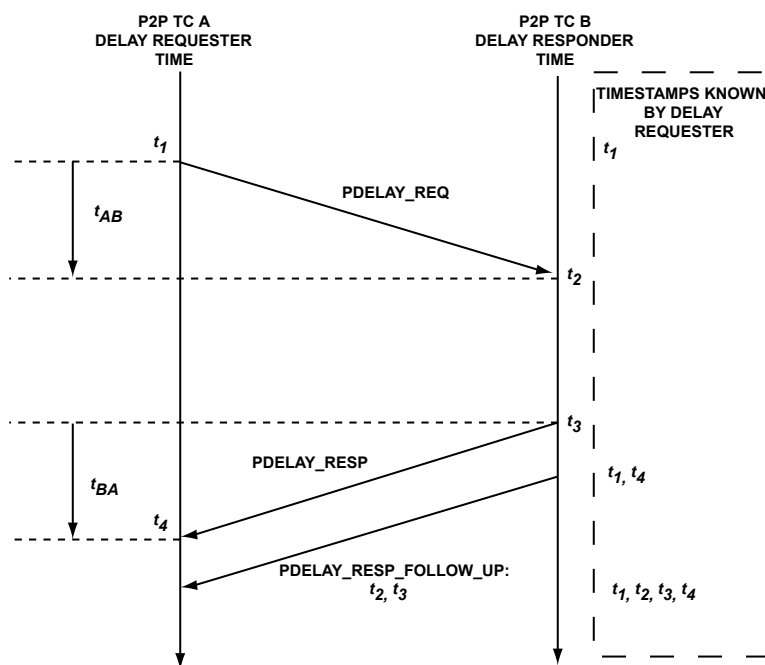


Figure 24-19: Propagation Delay Calculation between Nodes Supporting Peer-to-Peer Path Correction

As shown in the figure, the propagation delay is calculated in the following way:

1. Port-1 issues a Pdelay_Req message and generates a time stamp, t_1 , for the Pdelay_Req message.
2. Port-2 receives the Pdelay_Req message and generates a time stamp, t_2 , for this message.
3. Port-2 returns a Pdelay_Resp message and generates a time stamp, t_3 , for this message. To minimize errors due to frequency offset between the two ports, Port-2 returns the Pdelay_Resp message as quickly as possible after the receipt of the Pdelay_Req message. The Port-2 returns any one of the following:
 - The difference between the time stamps t_2 and t_3 in the Pdelay_Resp message.
 - The difference between the time stamps t_2 and t_3 in the Pdelay_Resp_Follow_Up message.
 - The time stamps t_2 and t_3 in the Pdelay_Resp and Pdelay_Resp_Follow_Up messages respectively.
4. Port-1 generates a time stamp, t_4 , on receiving the Pdelay_Resp message.

Port-1 uses all four time stamps to compute the mean link delay.

Block Diagram

The *PTP Block Diagram* figure shows the functional block diagram of PTP module.

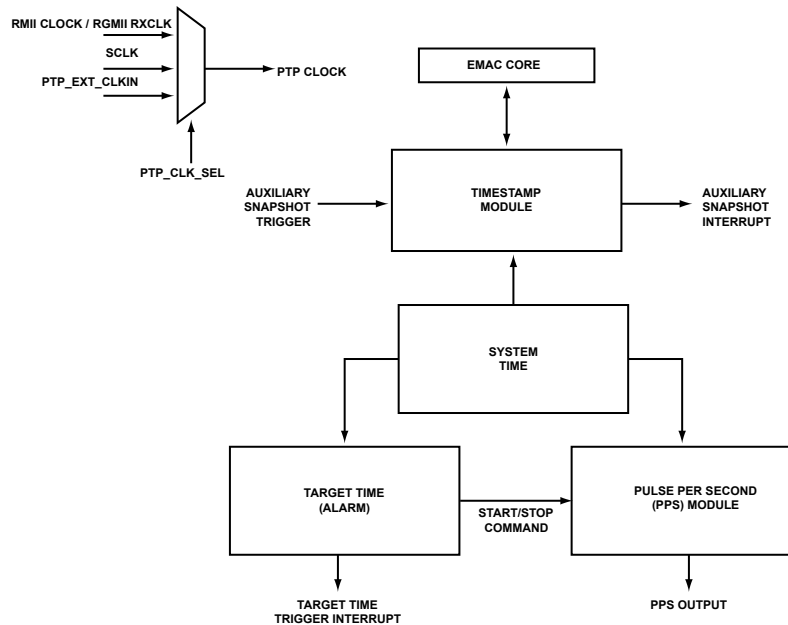


Figure 24-20: PTP Block Diagram

A system time module is present which keeps the time of PTP module. It consists of hardware which can be programmed for time initialization, time correction, and clock correction.

The time stamp module captures the time (provided by the system time module) at various conditions. For example, when the EMAC sends or receives a frame or during the rising edge of the auxiliary snapshot trigger (EMAC_PTPAUXIN[n]) pins. When system time is captured after detection of a frame, the time stamp module

automatically includes the time information in the frame descriptor. Time stamping on the detection of a frame can be programmed on a per frame basis.

The PTP clock drives the PTP module. This clock can be selected from three different clock sources.

The pulse per second (PPS) module generates a pulse or train of pulse on the PPS output pins, (EMAC_PTPPPS[n]). It is the physical representation of system time. PPS can be fixed (where only frequency varies) or flexible (where width, interval, start time, and stop time can be programmed).

The target time module acts as an alarm for the PTP module. Whenever system time reaches a value equal to programmed target time, the target time trigger interrupt is generated. By appropriate programming, The target time trigger can also start or stop flexible PPS output at specific time.

PTP Module Clock

The PTP module clock features include [Clock Source Selection](#) and [Clock Frequency Range](#).

Clock Source Selection

The PTP module can take one of these clock sources as its input clock — SCLK0, RMII reference RGMII Rx clock, or PTP external clock.

As shown in the *PTP Clock Source Selection* table, the [PADS_PCFG0](#) register selects the PTP clock source.

Table 24-41: PTP Clock Source Selection

PADS_PCFG0	PTP Clock Source	Clock Description
00	EMAC_RMII	RMII reference clock
00	RGMII_RXCLK	RGMII Rx Clock
10	PTP external clock	Clock available on EMAC_PTCLKIN[n] pin
X1	SCLK0	Processor system clock

Clock Frequency Range

The resolution, or granularity, of the reference time source determines the accuracy of the synchronization. Therefore, a higher PTP clock frequency gives better system performance. The timing constraints achievable for logic operating on the selected PTP clock source limit the maximum PTP clock frequency.

The minimum PTP clock frequency depends on the time required between two consecutive frames. The IEEE specification determines the RMII clock frequency. The minimum PTP clock frequency required for proper operation depends on the operating mode and operating speed of the MAC. See the *Minimum PTP Clock Frequency* table.

A minimum delay required between two consecutive time stamp captures is 8 clock cycles of RMII or 4 clock cycles of RGMII and 3 clock cycles of PTP clocks. If the delay between two time stamp captures is less than this delay, the EMAC does not take a time stamp snapshot for the second frame.

The following table assumes:

- Minimum Ethernet packet size = 64 bytes

- Minimum IFG = 96 bit times = 12 bytes
- Preamble = 8 bytes
- 3 PTP Clock + 4GMII/MII Clock Min gap between two SFDs (start of frame delimiters)
- 3 PTP Clock + 8 RMII Clock Min gap between two SFDs
- 3 PTP Clock + 4 RGMII Clock Min gap between two SFDs

Mode (Full Duplex in all cases)	Minimum Gap between two SFDs	PTP clock	Comments
10 Mbps RMII	64 bytes of data + 8 bytes of preamble + min IFG	3 PTP clock cycle + 8RMII clock cycle 3360 RMII clock cycles	In RMII @ 10 Mbps, 1 byte transmitted in 40 clock cycles
	(2560 + 320 + 480) RMII clock cycles	3 PTP clock cycle 3352 RMII clock cycles	RMII clock = 50 MHz
	3360 RMII clocks	PTP clock cycle 1117.33 20 ns = 22346 ns	1 RMII clock cycle = (1/50 MHz) = 20 ns
		<i>PTP frequency min = 0.045 MHz</i>	
10 Mbps RGMII	64 bytes of data + 8 bytes of preamble + min IFG	3 PTP clock cycle + 4RGMII clock cycle 168 RGMII clock cycles	In RGMII, 1 byte transmitted in 4 clock cycles
	(128 + 16 + 24) MII clock cycles	3 PTP clock cycle <= 164 RGMII clock cycles	RGMII clock at 100 Mbps = 2.5 MHz
	168 MII clock cycles	PTP clock cycle 54.67 400ns = 21860 ns	1 RGMII clock cycle = (1/2.5 MHz) = 400 ns
		<i>PTP frequency min = 0.045 MHz</i>	
100 Mbps RMII	64 bytes of data + 8 bytes of preamble + min IFG	3 PTP clock cycle + 8RMII clock cycle 336 RMII clock cycles	In RMII, 1 byte transmitted in 4 clock cycles
	(256 + 32 + 48) RMII clock cycles	3 PTP clock cycle 328 RMII clock cycles	RMII clock = 50 MHz
	336 RMII clocks	PTP clock cycle 109.33 20 ns = 2186 ns	1 RMII clock cycle = (1/50 MHz) = 20 ns
		<i>PTP frequency min = 0.45 MHz</i>	
100 Mbps RGMII	64 bytes of data + 8 bytes of preamble + min IFG	3 PTP clock cycle + 4RGMII clock cycle 168 RGMII clock cycles	In RGMII, 1 byte transmitted in 4 clock cycles
	(128 + 16 + 24) MII clock cycles	3 PTP clock cycle 164 RGMII clock cycles	RGMII clock at 100 Mbps = 25 MHz
	168 MII clock cycles	PTP clock cycle 54.67 40 ns = 2186 ns	1 RGMII clock cycle = (1/25 MHz) = 40 ns
		<i>PTP frequency min = 0.45 MHz</i>	

Mode (Full Duplex in all cases)	Minimum Gap between two SFDs	PTP clock	Comments
1000 Mbps RGMII	64 bytes of data + 8 bytes of preamble + min IFG	3 PTP clock cycle + 4 RGMII clock cycle 84 RGMII clock cycles	In RGMII @ 1000 Mbps, 1 byte transmitted in 1 clock cycles
	(64 + 8 + 12) RGMII clock cycles	3 PTP clock cycle 80 RGMII clock cycles	RGMII clock at 100 Mbps = 125 MHz
	84 RGMII clocks	PTP clock cycle 26.67 8 ns = 213 ns	1 RGMII clock cycle = (1/125 MHz) = 8 ns
		<i>PTP frequency min = 4.6 MHz</i>	

The minimum PTP clock frequency also depends on the maximum value of the `EMAC_TM_SUBSEC` register, so that even at the highest `EMAC_TM_SUBSEC.SSINC` value, the `EMAC_TM_SEC` register value can be incremented every second. Since the `EMAC_TM_SUBSEC.SSINC` is an 8-bit field, the minimum PTP clock frequency allowed is approximately 4 MHz.

Time Stamp Module

The time stamp module captures time in seconds and nanoseconds maintained as system time. The time stamp module also captures time when specific events occur. Events include detection of a frame transmitted or received over the EMAC and a rising edge on the `EMAC_PTPAUXIN[n]` pins. The time stamp module does not need to time stamp all of the transmitted or received frames over the EMAC. The PTP module can be programmed to detect specific kinds of frames for time stamping.

Frame Detection and Time Stamping

The PTP module automatically monitors all received and transmitted IEEE 1588 event messages on the Ethernet. If the module detects an event message, it takes a snapshot of the system time. The PTP module stores the value to the 64-bit fields in transmit or receive descriptor.

The time stamping occurs at the EMAC RMII/RGMII interface when the module sees the start of frame of an event message packet.

Transmit Path Time Stamping

The EMAC captures a time stamp when a frame transmits on the RMII/RGMII. Time stamp capture is controllable on a per-frame basis. In other words, each transmit frame can be marked to indicate whether a time stamp is captured for that frame or not.

Applications can extend the descriptor word length from 4 words to 8 words by setting the `EMAC_DMA0_BUSMODE.ATDS` bit. To enable the time stamp function, set the TTSE (transmit time stamp enable) bit in transmit descriptor word TDES0. When the PTP module captures a time stamp of a transmitted frame, it notifies the application by setting the TTSS (transmit time stamp status) in TDES0.

The EMAC returns the time stamp to the software inside the corresponding transmit descriptor, automatically connecting the time stamp to the specific frame. The 64-bit time stamp information is written to the TDES6 and TDES7 fields. The TDES6 field holds the 32-bit LSBs of the time stamp (system time nanoseconds), except as described in transmit time stamp field, and the TDES7 field holds the 32-bit MSBs (system time seconds). After the

PTP module time stamps the frame, the application can get the time stamp along with the transmit status from the EMAC.

NOTE: The PTP module time stamps all the transmitting frames having TTSE set in its TDES0. It does not distinguish according to the type of transmitting frame.

Receive Path Time Stamping

The PTP module automatically monitors all received and transmitted IEEE 1588 event messages on the Ethernet. If an event message is detected, the module takes a snapshot of the system time. The module stores its value to the 64-bit fields in transmit or receive descriptor. The time stamping is done at the EMAC RMII/RGMII interface when the module sees the start of frame of an event message packet.

PTP module captures the time stamp of received frames on the RMII/RGMII. Time stamp capture is controllable on a per-frame and per-type basis. In other words, each received frame is time stamped according to the frame type.

Applications can extend the descriptor word length from 4 words to 8 words by setting the `EMAC_DMA0_BUSMODE.ATDS` bit to store time stamp and received message status. The PTP notifies the application of receive time stamp availability when it sets bit 7 (time stamp available) in receive descriptor word RDES0.

When bit 0 (extended status available) is set in RDES0, it indicates that the extended status of the PTP frame is provided in the RDES4 word. Extended status includes PTP version, PTP frame type, and message type. The EMAC returns the time stamp to the software inside the corresponding receive descriptor. The 64-bit time stamp information is written back to the RDES6 and RDES7 fields in memory. The RDES6 holds the 32-bit LSBs of the time stamp (system time nanoseconds), except as mentioned in receive time stamp field, and the RDES7 field holds 32-bit MSBs (system time seconds).

The time stamp is written only to that receive descriptor for which the last descriptor status field has been set to 1. When the time stamp is not available (for example, because of an RxFIFO overflow), an all-ones pattern is written to the descriptors (RDES6 and RDES7). The write operation indicates that the time stamp is not correct. RDES0 [7] indicates whether the time stamp is updated in RDES6/7.

The PTP module processes received frames to identify valid PTP frames. Use the `EMAC_TM_CTL` register to control the snapshot of the time sent to the application.

The PTP module can be programmed to detect all received frames or only some types of PTP frames, according to bit settings in the `EMAC_TM_CTL` register. Refer to the *PTP Frame Type Selections* table.

Table 24-42: PTP Frame Type Selections

TSENALL (bit 8)	SNAPTYPSEL (bits [17:16])	TSMSTRENA (bit 15)	TSEVNTENA (bit 14)	Frames
1	X	X	X	All
0	00	X	0	Sync, Follow_Up, Delay_Req, Delay_Resp
0	00	0	1	Sync
0	00	1	1	Delay_Req

Table 24-42: PTP Frame Type Selections (Continued)

TSENALL (bit 8)	SNAPTYPSEL (bits [17:16])	TSMSTRENA (bit 15)	TSEVNTENA (bit 14)	Frames
0	01	X	0	Sync, Follow_Up, Delay_Req, Delay_Resp, Pdelay_Req, Pdelay_Resp, Pdelay_Resp_Follow_Up
0	01	0	1	Sync, Pdelay_Req, Pdelay_Resp
0	01	1	1	Delay_Req, Pdelay_Req, Pdelay_Resp
0	10	X	X	Sync, Delay_Req
0	11	X	X	Pdelay_Req, Pdelay_Resp

PTP Processing and Control

When the EMAC receives a frame, frame detection and time stamping are based on some of the PTP fields in the frame. The *PTP Message Format (IEEE 1588–2008)* table shows the common message header for the PTP messages. This format is derived from the IEEE standard 1588–2008. When the EMAC sends a PTP frame, the frame follows this format.

When a frame is received, the PTP module compares these fields with standard values and finds out the type of PTP frame and other information. For example, PTP version, IP version, and others. The module then updates the related fields in RDES4. When a frame is transmitted, programs must ensure that all the fields are appropriate. The PTP module on the other end of a communication must correctly detect and decode the frame.

Table 24-43: PTP Message Format (IEEE 1588–2008)

Bits								Octets	Offset
7	6	5	4	3	2	1	0		
transportSpecific				messageType				1	0
Reserved				versionPTP				1	1
messageLength								2	2
domainNumber								1	4
Reserved								1	5
flagField								2	6
correctionField								8	8
Reserved								4	16
sourcePortIdentity								10	20
sequenceId								2	30

Table 24-43: PTP Message Format (IEEE 1588–2008) (Continued)

Bits	Octets	Offset
controlField (used in version 1. For version 2, messageType field is used for detecting different message types.)	1	32
logMessageInterva	1	33

There are some fields in the Ethernet payload that can be used to detect the PTP packet type and control the snapshot taken. These fields are different for the following PTP frames:

- PTP Frames Over IPv4
- PTP Frames Over IPv6
- PTP Frames Over Ethernet

For these PTP frames, EMAC does not consider the PTP version 1 messages as valid when the frame consists of peer delay multicast address as destination address (DA).

PTP Frame Over IPv4

The *IPv4-UDP PTP Frame Fields Required for Control and Status* table provides information about the fields that are matched to control snapshot for the PTP packets. The packets are sent over UDP over IPv4 for IEEE 1588 version 1 and 2. The octet positions for the tagged frames are offset by 4. The positions are based on IEEE 1588–2008 standards and the message format. The format is defined in the *PTP Message Format (IEEE 1588–2008)* table in the [PTP Processing and Control](#) section.

Table 24-44: IPv4-UDP PTP Frame Fields Required for Control and Status

Field Matched	Octet Position	Matched Value	Description
MAC Frame type	12, 13	0x0800	IPv4 datagram
IP Version and Header Length	14	0x45	IP version is IPv4
Layer 4 Protocol	23	0x11	UDP
IP Multicast Address (IEEE 1588 Version 1)	30, 31, 32, 33	0xE0,0x00, 0x01,0x81 (or 0x82 or 0x83 or 0x84)	Multicast IPv4 addresses allowed. 224.0.1.129 224.0.1.130 224.0.1.131 224.0.1.132
IP Multicast Address (IEEE 1588 Version 2)	30, 31, 32, 33	0xE0, 0x00, 0x01, 0x81 (Hex) 0xE0, 0x00, 0x00, 0x6B (Hex)	PTP-Primary multicast address: 224.0.1.129 PTP-Peer delay multicast address: 224.0.0.107

Table 24-44: IPv4-UDP PTP Frame Fields Required for Control and Status (Continued)

Field Matched	Octet Position	Matched Value	Description
UDP Destination Port	36, 37	0x013F 0x0140	0x013F - PTP Event Messages. These are SYNC, Delay_Req (IEEE 1588 version 1 and 2) or Pdelay_Req, Pdelay_Resp (IEEE 1588 version 2 only). 0x0140 - PTP general messages
PTP Control Field (IEEE version 1)	74	0x00/0x01/0x02/ 0x03/0x04	0x00 - SYNC 0x01 - Delay_Req 0x02 - Follow_Up 0x03 - Delay_Resp 0x04 - Management
PTP Message Type Field (IEEE version 2)	42 (nibble)	0x0/0x1/0x2/0x3/0x8/0x9/0xA/0xB/ 0xC/0xD	0x0 - SYNC 0x1 - Delay_Req 0x2 - Pdelay_Req 0x3 - Pdelay_Resp 0x8 - Follow_Up 0x9 - Delay_Resp 0xA - Pdelay_Resp_Follow_Up 0xB - Announce 0xC - Signaling 0xD - Management
PTP Version	43 (nibble)	0x1 or 0x2	0x1 - Supports PTP version 1 0x2 - Supports PTP version 2

PTP Frame Over IPv6

The *IPv6-UDP PTP Frame Fields Required for Control And Status* table provides information about the fields that are matched to control the snapshots for the PTP packets. The packets are sent over UDP over IPv6 for IEEE 1588 version 1 and 2. The octet positions for the tagged frames are offset by 4. The positions are based on IEEE 1588–2008 standards and the message format defined in PTP Message Format (IEEE 1588–2008).

Table 24-45: IPv6-UDP PTP Frame Fields Required for Control and Status

Field Matched	Octet Position	Matched Value	Description
MAC Frame type	12, 13	0x86DD	IP datagram
IP Version	14 (bits [7:4])	0x06	IP version is IPv6
Layer 4 Protocol	20 (IPv6 extension header not defined for PTP packets)	0x11	UDP

Table 24-45: IPv6-UDP PTP Frame Fields Required for Control and Status (Continued)

Field Matched	Octet Position	Matched Value	Description
PTP Multicast Address	38–53	FF0x:0:0:0:0:0:0:0:181 (Hex) FF02:0:0:0:0:0:0:0:6B (Hex)	PTP - primary multicast address: FF0x:0:0:0:0:0:0:0:181 (Hex) PTP - Peer delay multicast address: FF02:0:0:0:0:0:0:0:6B (Hex)
UDP Destination Port	56, 57 (IPv6 extension header not defined for PTP packets)	0x013F, 0x0140	0x013F - PTP event messages 0x0140 - PTP general messages
PTP Control Field (IEEE 1588 version 1)	93 (IPv6 extension header not defined for PTP packets)	0x00/0x01/0x02/ 0x03/0x04	0x00 - SYNC 0x01 - Delay_Req 0x02 - Follow_Up 0x03 - Delay_Resp 0x04 - Management (version1)
PTP Message Type Field (IEEE version 2)	74 (nibble) (IPv6 extension header not defined for PTP packets)	0x0/0x1/0x2/0x3/0x8/0x9/0xA/0xB/ 0xC/0xD	0x0 - SYNC 0x1 - Delay_Req 0x2 - Pdelay_Req 0x3 - Pdelay_Resp 0x8 - Follow_Up 0x9 - Delay_Resp 0xA - Pdelay_Resp_Follow_Up 0xB - Announce 0xC - Signaling 0xD - Management
PTP Version	75 (nibble)	0x1 or 0x2	0x1 - Supports PTP version 1 0x2 - Supports PTP version 2

PTP Frame Over Ethernet

Refer to the *Ethernet PTP Frame Fields Required for Control and Status* table. The table provides information about the fields that are matched to control the snapshots for the PTP packets sent over Ethernet for IEEE 1588 version 1 and 2. The octet positions for the tagged frames are offset by 4. The positions are based on IEEE 1588–2008 standards and the message format defined in the table.

Table 24-46: Ethernet PTP Frame Fields Required for Control and Status

Field Matched	Octet Position	Matched value	Description
MAC Destination Multicast Address (The address match of destination address (DA) programmed in MAC address 0 is used when the <code>EMAC_TM_CTL.TSENMACADDR</code> bit is set)	0–5	01-1B-19-00-00-00 01-80-C2-00-00-0E	All PTP messages can use any of the following multicast addresses: 01-1B-19-00-00-00 01-80-C2-00-00-0E
MAC Frame Type	12, 13	0x88F7	PTP Ethernet frame
PTP control field (IEEE Version 1)	45	0x00/0x01/0x02/ 0x03/0x04	0x00 - SYNC 0x01 - Delay_Req 0x02 - Follow_Up 0x03 - Delay_Resp 0x04 - Management
PTP Message Type Field (IEEE version 2)	14 (nibble)	0x0/0x1/0x2/0x3/0x8/0x9/0xA/0xB/ 0xC/0xD	0x0 - SYNC 0x1 - Delay_Req 0x2 - Pdelay_Req 0x3 - Pdelay_Resp 0x8 - Follow_Up 0x9 - Delay_Resp 0xA - Pdelay_Resp_Follow_Up 0xB - Announce 0xC - Signaling 0xD - Management
PTP Version	15(nibble)	0x1 or 0x2	0x1 - Supports PTP version 1 0x2 - Supports PTP version 2

Auxiliary Time Stamp Snapshot

The auxiliary snapshot feature stores snapshots of the system time whenever a rising edge is detected on the `EMAC_PTPAUXIN[n]` pins.

The PTP stores 64 bits of captured time stamp in a 4-deep FIFO. When a snapshot is stored, the PTP indicates this event to the EMAC with the auxiliary snapshot interrupt. The `EMAC_TM_STMPSTAT.ATSTS` bit is set. The value of the snapshot is read through the `EMAC_TM_AUXSTMP_SEC` and `EMAC_TM_AUXSTMP_NSEC` registers. If the FIFO becomes full and an external trigger to take the snapshot is asserted, then the snapshot trigger-missed status is set in the `EMAC_TM_STMPSTAT.ATSSTM` bit. The latest snapshot is not written to the FIFO when it is full.

When a host reads the 64-bit time stamp from the FIFO through the `EMAC_TM_AUXSTMP_SEC` and `EMAC_TM_AUXSTMP_NSEC` registers, the space becomes available to store the next snapshot.

NOTE: A space in the FIFO is created whenever the `EMAC_TM_AUXSTMP_SEC` register is read. Therefore, read the `EMAC_TM_AUXSTMP_NSEC` register before reading the `EMAC_TM_AUXSTMP_SEC` register.

The program can clear the FIFO by setting the `EMAC_TM_CTL.ATSFC` bit. When multiple snapshots are present in the FIFO, the `EMAC_TM_STMPSTAT.ATSNS` bits indicate the count.

NOTE: The minimum gap between two events on the `EMAC_PTPAUXIN[n]` pin must be 4 cycles of `PTP_CLK` + 3 cycles of `SCLK0`). Otherwise, the logic misses the rising-edge of the trigger.

System Time

To get a snapshot of the time, the EMAC requires a reference time in 64-bit format as defined in the IEEE 1588 specification. The PTP module maintains 80-bit time, known as system time. The PTP clock updates system time.

The 80-bit timing reference is split into the following three registers:

- `EMAC_TM_NSEC` – 32-bit nanoseconds register which provides time in nanoseconds
- `EMAC_TM_SEC` – 32-bit seconds register which provides time in seconds
- `EMAC_TM_HISEC` – 16-bit high seconds register which provides time beyond the seconds register. The IEEE 1588 standard does not include this register. Its use is application-specific.

The 64-bit system time (seconds and nanoseconds) is the source for taking time stamps for Ethernet frames being transmitted or received at the RMII.

Since the PTP clock frequency does not correspond to a 1ns period, the `EMAC_TM_NSEC` register is incremented with a value equal to the PTP clock period for every PTP clock cycle. The function uses the `EMAC_TM_SUBSEC` register. The `EMAC_TM_NSEC` value is incremented with the value programmed in `EMAC_TM_SUBSEC` register every PTP clock cycle.

Whenever the `EMAC_TM_SEC` register overflows from `0xFFFFFFFF` to `0x00000000`, the seconds overflow interrupt is triggered. The EMAC uses the `EMAC_TM_STMPSTAT.TSSOVF` bit to indicate the event. After a seconds overflow, the `EMAC_TM_HISEC` register increments by one.

The system time module supports the following two types of rollover modes for the `EMAC_TM_NSEC` register.

- Digital rollover mode. The maximum value in the nanoseconds field is `0x3B9AC9FF`, that is, 10^9 nanoseconds. After it reaches this value, the `EMAC_TM_SEC` register increments and the `EMAC_TM_NSEC` register restarts counting from zero. Accuracy in digital rollover mode it is 1 ns per bit.
- Binary rollover mode. The nanoseconds field rolls over and increments the seconds field after the value reaches `0x7FFFFFFF`. Accuracy in binary rollover mode is ~0.466 ns per bit.

System Time Adjustment

The following sections describe the process for system time adjustment.

System Time Initialization

System time can be initialized with 64-bit time when the PTP module is enabled. The initial value is written to the `EMAC_TM_SECUPDT` and `EMAC_TM_NSECUPDT` system time update registers. The system time counter is written with the value in the registers when the `EMAC_TM_CTL.TSINIT` bit is set.

Coarse Correction Method

If slave system time has an offset based on the system time of the master, then the coarse correction method can correct it. The time offset value is written to the `EMAC_TM_SECUPDT` and `EMAC_TM_NSECUPDT` registers. The offset value is then added to or subtracted from the system time when the `EMAC_TM_CTL.TSUPDT` bit is set. Use the `EMAC_TM_NSECUPDT.ADDSUB` bit to choose addition or subtraction. System time correction occurs in one clock cycle using the coarse correction method.

NOTE: During subtraction, the `EMAC_TM_SECUPDT` register value must be less than the value of the `EMAC_TM_SEC` register. Check the value prior to subtracting using coarse correction.

Fine Correction Method

If a slave PTP clock frequency has a drift based on the master PTP clock (as defined in IEEE 1588), it can be corrected using the fine correction method. Using this method, system time is corrected over a period (unlike coarse correction where it happens in one clock cycle). This correction helps maintain linear time and does not introduce drastic changes (or a large jitter) in the reference time between PTP sync message intervals.

Using this method, an accumulator sums the contents of the `EMAC_TM_ADDEND` register. The *System Time Update, Fine Correction Method* figure shows the method. The arithmetic-carry that the accumulator generates acts as a pulse to increment the system time counter. The accumulator and the addend are 32-bit registers. Here, the accumulator acts as a high-precision frequency divider.

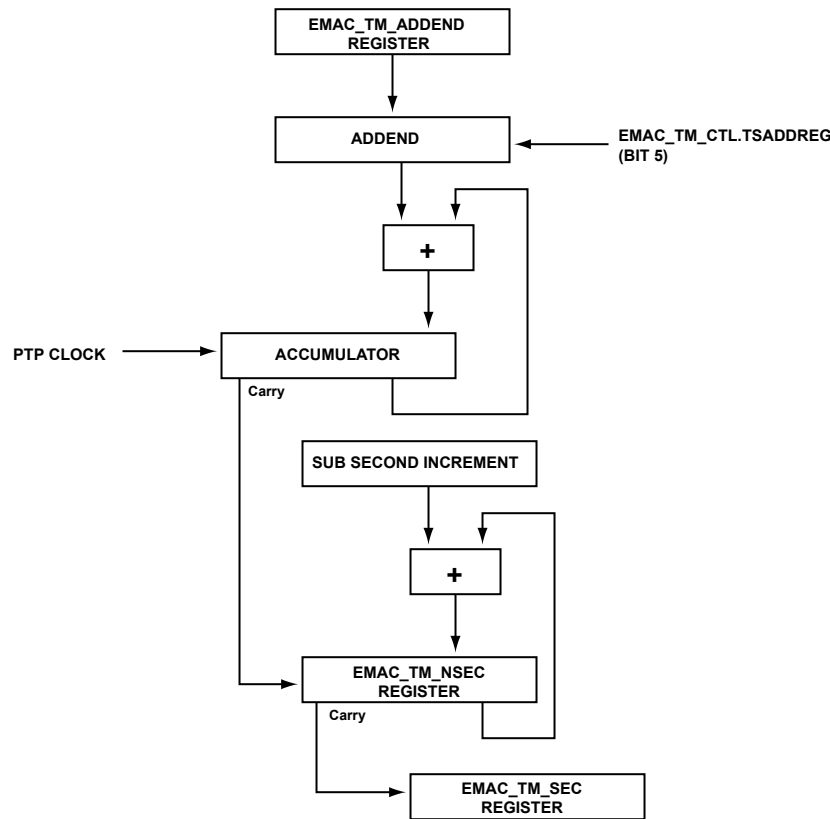


Figure 24-21: System Time Update, Fine Correction Method

Calculating Addend Value

This section describes the fine correction process for system time.

In this example, the master clock runs at 50 MHz and the slave clock has drifted to 66 MHz. The goal is to adjust the slave system time to 50 MHz, so that the slave PTP module synchronizes with the master. Using the figure in [Fine Correction Method](#), the nanoseconds increment signal runs at 50 MHz. The nanoseconds increment is the carry from accumulator register. The addend value increments the carry from accumulator at the rate of the slave clock (66 MHz).

The accumulator overflows and generates a carry every N addend value, so $N \times \text{Addend} = 2^{32}$.

The accumulator increments at 66 MHz. This addition brings the carry to 50 MHz $N = 66/50 = 1.32$.

Hence, the addend = $2^{32}/1.32 = 0xC1F07C1F$.

Therefore, if addend is programmed with $0xC1F07C1F$, the slave system time runs at 50 MHz which synchronizes with the master.

In the [Fine Correction Method](#) figure, the subsecond increment is the value programmed in the `EMAC_TM_SUBSEC` register which increments the `EMAC_TM_NSEC` register according to the frequency of the nanoseconds increment signal.

In the example, the sub second increment is 20 (for digital rollover) or 43 (for binary rollover). This addition increments the `EMAC_TM_NSEC` register by 20 ns (1/50 MHz).

The software must calculate the drift in frequency and update the `EMAC_TM_ADDEND` register accordingly.

NOTE: The PTP reference clock is the clock at which the system time is updated. When the `EMAC_TM_CTL.TSCFUPDT` bit is set to 0, this clock equals the PTP clock. Using fine correction, the PTP reference clock is generated on the nanoseconds increment signal at which the system time updates.

Target Time Trigger (Alarm)

The PTP module provides an alarm function by triggering an alarm at a preset time. It sets the `EMAC_TM_STMPSTAT.TSTARGET0` bit when the system time matches the value of the `EMAC_TM_PPS0TGTM` and `EMAC_TM_PPS0NTGTM` registers. This trigger can generate an interrupt and command the flexible PPS module to start or stop PPS output, depending on value programmed in `EMAC_TM_PPSCCTL.TRGTMODSEL0` bits.

The trigger is enabled by setting `EMAC_TM_CTL.TSTRIG` bit. Once an alarm has occurred, if the PTP needs another alarm, the software must:

- Clear the status bit
- Reprogram the `EMAC_TM_PPS0TGTM` and `EMAC_TM_PPS0NTGTM` registers to a future value, and
- Set the `EMAC_TM_CTL.TSTRIG` bit

If the time programmed in the target time registers has elapsed, then a target time programming error is indicated by setting the `EMAC_TM_STMPSTAT.TSTRGTERR0` bit.

The alarm time is represented in absolute units, not relative units. For example, if the software must generate an alarm after 10 seconds, it must read the current system time value. Then, the software must add the number corresponding to 10 seconds, and write the result back to the target time registers.

Pulse-Per-Second (PPS)

Pulse-per-second (PPS) is a physical representation of system time. It consists of a single pulse or train of pulses. The PTP uses PPS for extra synchronization or to monitor the synchronization performance between clocks. With proper configuration, the PTP module can generate PPS signals that are output on the `EMAC_PTPPPS[n]` pins. The PTP supports two kinds of PPS output, fixed and flexible.

Fixed Pulse-Per-Second Output

The EMAC supports fixed pulse-per-second (PPS) output that indicates 1-second intervals (default). Change the frequency of the PPS output by configuring the `EMAC_TM_PPSCCTL.PPSCTL0` bits. The default value for these bits is 0000, which configures a 1Hz signal with a pulse width equal to the period of the PTP clock.

The *PPS Output Frequencies* table shows various PPS output frequencies.

Table 24-47: PPS Output Frequencies

PPSCTL Bit Setting	Binary Rollover	Digital Rollover
0001	2 Hz	1 Hz

Table 24-47: PPS Output Frequencies (Continued)

PPSCTL Bit Setting	Binary Rollover	Digital Rollover
0010	4 Hz	2 Hz
0011	8 Hz	4 Hz
...	...	...
1111	32.768 kHz	16.384 kHz

In binary rollover mode, the PPS output has a duty cycle of 50% with these frequencies.

In digital rollover mode, the PPS output frequency is an average number. The actual clock is a different frequency that is synchronized every second. PPS output pulses have different periods and duty cycles and this behavior is because of the non-linear toggling of the bits in digital rollover mode. For example:

- When `EMAC_TM_PPSCTL.PPSCTL0 = 0001`, the PPS (1 Hz) has a low period of 537 ms and a high period of 463 ms.
- When `EMAC_TM_PPSCTL.PPSCTL0 = 0010`, the PPS (2 Hz) is a sequence of:
 - One clock of 50-percent duty cycle and 537-ms period
 - Second clock of 463-ms period (268 ms low and 195 ms high)
- When `EMAC_TM_PPSCTL.PPSCTL0 = 0011`, the PPS (4 Hz) is a sequence of:
 - Three clocks of 50-percent duty cycle and 268-ms period
 - Fourth clock of 195-ms period (134 ms low and 61 ms high)

Flexible Pulse-Per-Second Output

The EMAC also provides the flexibility to program the start or stop time, width, and interval of the pulse generated on the PPS output. Enable this feature, called flexible PPS, by setting the `EMAC_TM_PPSCTL.PPSEN` bit.

The flexible PPS output options are:

- Supports programming the start point of the single pulse and start and stop points of the pulse train in terms of system time. The target time registers program the start and stop time.
- Supports programming the stop time in advance. Programs can configure the stop time before the actual start time has elapsed.
- Supports programming the width, between the rising edge and corresponding falling edge of the PPS signal output, in terms of number of units of subsecond increment. This value is configured in the `EMAC_TM_SUBSEC` register.
- Supports programming the interval, between the rising edges of PPS signal, in terms of number of units of subsecond increment. This value is configured in the `EMAC_TM_SUBSEC` register.
- Provides the option to cancel the programmed PPS start or stop request.

- Indicates error if the start or stop time programmed has already elapsed.

PPS Start or Stop Time

Start time can initially be programmed in the target time registers. If necessary, the start or stop time can be programmed again but only after the earlier programmed value is synchronized to the PTP clock domain. The `EMAC_TM_PPS0NTGTM.TSTRBUSY` bit indicates the status of synchronization. Programs can configure the start or stop time in advance, even before the earlier stop or start time has elapsed.

Program the start or stop time with advanced system time to ensure proper PPS signal output. If the application programs a start or stop time that has already elapsed, then the EMAC sets the `EMAC_TM_STMPSTAT.TSTRGTERR0` bit, indicating the error. If enabled, the EMAC also sets the target time trigger (alarm) interrupt event. The application can cancel the start or stop request only if the corresponding start or stop time has not elapsed. If the time has elapsed, the cancel command has no effect.

PPS Width and Interval

The PPS width and interval are programmed in terms of granularity of system time, that is, the number of the units of subsecond increment value. For example, with the PTP reference clock of 50 MHz, a PPS pulse width of 40 ns, and an interval of 100 ns, program the width and interval to 2 and 5, respectively.

Use a faster PTP reference clock to achieve smaller granularity. Before commanding to trigger a pulse or pulse train on the PPS output, programs must configure or update the interval and width of the PPS signal output.

PPS Command

When the PPS module has a flexible PPS output configuration, the PTP can use the `EMAC_TM_PPSCTL.PPSCTL0` bits to command the PPS module to use any of the flexible PPS features.

Programming these bits with a non-zero value instructs the PPS module to initiate an event. Once the command transfers or synchronizes to the PTP clock domain, these bits clear automatically. Software must ensure that these bits are programmed only when they are all-zero.

The *Flexible PPS Output Commands* table explains the different commands and their function.

Table 24-48: Flexible PPS Output Commands

PPSCTL (Bits 3–0)	Command	Description
0000	No Command	
0001	Start Single Pulse	Generates single pulse rising at start point defined in target time registers and of duration defined in <code>EMAC_TM_PPS0WIDTH</code> register.
0010	Start Pulse Train	Generates train of pulses rising at the start time configured in the target time registers, of duration configured in the <code>EMAC_TM_PPS0WIDTH</code> register. The train of pulses repeats at the interval configured in the <code>EMAC_TM_PPS0INTVL</code> register. By default, the PPS pulse train is free-running unless stopped by stop pulse train at time or stop pulse train immediately commands.
0011	Cancel Start	Cancels the start single pulse and start pulse train commands when the system time has not crossed the programmed start time.

Table 24-48: Flexible PPS Output Commands (Continued)

PPSCTL (Bits 3–0)	Command	Description
0100	Stop Pulse Train at Time	Stops the train of pulses initiated by the command for start pulse train after the time programmed in the target time registers elapses.
0101	Stop Pulse Train Immediately	Immediately stops the train of pulses initiated by the command for start pulse train.
0110	Cancel Stop Pulse Train	Cancels the stop pulse train at time command when the programmed stop time has not elapsed. The PPS pulse train becomes free-running on the successful execution of this command.
0111-1111	Reserved	

PTP Interrupts

Set the `EMAC_IMSK.TS` bit to enable interrupts from the PTP module. The EMAC uses the `EMAC_ISTAT.TS` bit to indicate the status of the interrupt. The PTP supports the following three types of interrupts.

Auxiliary Snapshot Trigger

When an external event occurs on the `EMAC_PTPAUXIN[n]` pins and a time stamp snapshot occurs, an auxiliary snapshot interrupt is triggered. The EMAC uses the `EMAC_TM_STMPSTAT.ATSTS` bit to indicate the interrupt.

Target Time Reached

This interrupt is triggered when the system time becomes equal to the value written in the `EMAC_TM_PPSONTGTM` and `EMAC_TM_PPSONTGTM` registers. Enable or disable the interrupt using the `EMAC_TM_CTL.TSTRIG` and `EMAC_TM_PPSCTL.TRGTMODESEL0` bits. This interrupt can be used as an alarm and is indicated on the `EMAC_TM_STMPSTAT.TSTARGET0` bit.

System Time Seconds Register Overflow

This interrupt is triggered when the `EMAC_TM_SEC` register overflows from `0xFFFF FFFF` to `0x0000 0000`. This interrupt is indicated on the `EMAC_TM_STMPSTAT.TSSOVF` bit. As soon as `EMAC_TM_SEC` register overflows, the `EMAC_TM_HISEC` register increments by one.

Audio Video Data Transmission

The audio video (AV) feature enables transmission of time-sensitive traffic over bridged local area networks (LANs). The following standards define the various aspects of the AV feature implementation.

- IEEE 802.1Qav-2009: Allows the bridges to provide time-sensitive and loss-sensitive real-time audio video data transmission (AV traffic). It specifies the priority regeneration and controlled bandwidth queue draining algorithms that are used in bridges and AV traffic sources.
- IEEE 802.1Qat-2009: Allows reservation of the network resources for specific traffic streams traversing a bridged local area network.
- IEEE 802.1AS-2011: Specifies the protocol and procedures used to ensure that the synchronization requirements are met for time-sensitive applications. For example, audio and video and across bridged and virtual-bridged LANs. Virtual-bridged LANs consist of LAN media where the transmission delays are fixed and

symmetrical. For example, IEEE 802.3 full-duplex links include the maintenance of synchronized time during normal operation followed by addition, removal, or failure of network components and network reconfiguration.

As shown in the *Transmit and Receive Path Block Diagram* figure, one SCB master interface connects to three DMA channels (channel 0, channel 1, and channel 2). The DMA arbiter helps in arbitration of all the paths (transmit and receive) in channel 0, channel 1 and channel 2. Each channel has a separate control and status register (CSR) for managing the transmit and receive functions, descriptor handling, and interrupt handling.

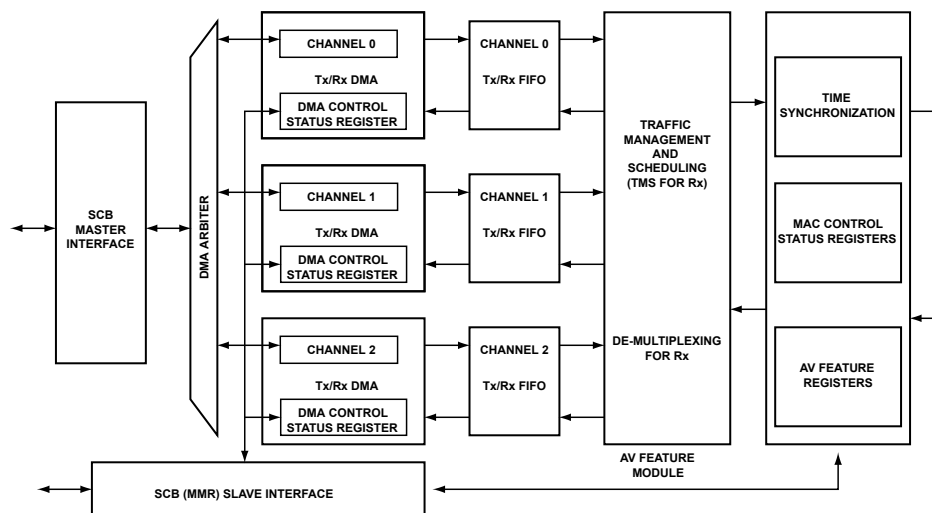


Figure 24-22: Transmit and Receive Path Block Diagram

Transmit Path Functions

The transmit path of channel 0 supports strict-priority algorithm and is used for best-effort traffic. For a channel, the strict-priority algorithm determines that a frame is available for transmission if the channel contains one or more frames. When the threshold mode for EMAC MFL Tx FIFO is enabled, the strict-priority algorithm determines that a frame is available for transmission. The algorithm determines when the channel contains a partial frame of size equal to the programmed threshold limit.

The transmit paths of channel 1 and channel 2 support traffic management by using the credit-based shaper algorithm. For a channel, the credit-based shaper algorithm determines that a frame is available for transmission if the following conditions are true:

- The channel contains one or more frames.
- The credit for the channel is positive as per the algorithm.

Programs can disable the credit-based shaper algorithm for all channels or for lower-priority channels. The credit-based shaper algorithm can be disabled for channel 1 and channel 2 or for channel 1 only. When the credit-based shaper algorithm for a channel is disabled, the channel uses the default strict-priority algorithm.

Each transmit DMA has a separate descriptor chain for fetching the transmit data. The transmit channel that receives access to the system bus depends on the DMA arbiter. For information about the DMA arbiter, see [DMA Arbiter](#).

The transmit path has separate FIFOs for each channel, as shown in the *Transmit and Receive Path Block Diagram* figure. The data fetched by the DMA is placed in the respective FIFO. The traffic management and scheduler unit (TMS) controls which FIFO data the MAC transmits. If the credit-based shaper algorithm is enabled for a channel (1 or 2), then the corresponding channel is selected for transmission when the following conditions are true:

- The frame is available in the channel and has a positive or zero credit.
- The higher priority channel has no frame waiting in the FIFO.

If the credit-based shaper algorithm is disabled for all channels, then the frame awaiting transmission from a channel is selected. The selection is made based on the priority scheme described in the *Fixed Priority Scheme for DMA Channels* table.

Receive Path Functions

To differentiate between the AV and non-AV traffic, the MAC provides a status. The status indicates if data is an AV packet and identifies its corresponding VLAN Priority tag value. This status is updated in the extended status field of the receive descriptor as explained in "Receive Descriptor". The `EMAC_MAC_AVCTL.AVTbit` field specifies a value that is compared with the `EtherType` field of the incoming Ethernet frame to detect an AV packet. The AV packets can be of the following two types:

- **AV data packets.** The AV data packets are always tagged. The tagged AV control packets are received based on the programmed priority value. The `EMAC_MAC_AVCTL.AVP` bit field specifies the channel to which an AV packet with a given priority must be sent.
- **AV control packets.** The AV control packets are either tagged or untagged. The untagged AV control packets are received on channel 0 by default. To receive these packets on channel 1 or channel 2, program the `EMAC_MAC_AVCTL.AVCH` bit field. Similar to the AV data packets, the tagged AV control packets are received based on the programmed priority value.

The following describes how tagged AV control packets and AV data packets are sent to a channel.

Table 24-49: AV Packets

Receive paths of channel 1 and channel 2 are enabled.	Channel 2 The following packets, with priority value greater than or equal to the value programmed in <code>EMAC_MAC_AVCTL.AVP</code> bit field, are received on channel 2: AV packets Non-AV tagged packets (if <code>EMAC_MAC_AVCTL.VQE</code> bit is set)
	Channel 1 The following packets, with priority value less than the value programmed in AV MAC control register, are received on channel 1: AV packets Non-AV tagged packets (if <code>EMAC_MAC_AVCTL.VQE</code> bit is set)
	Channel 0 All other packets are received on channel 0.
For example, priority value of 3 is programmed in the <code>EMAC_MAC_AVCTL.AVP</code> bit field.	Channel 2 The following packets with priority value from 3 to 7 are received on channel 2: AV packets Non-AV tagged packets (if the <code>EMAC_MAC_AVCTL.VQE</code> bit is set)
	Channel 1 The following packets with priority value 2 are received on channel 1: AV packets Non-AV tagged packets (if the <code>EMAC_MAC_AVCTL.VQE</code> bit is set)
	Channel 0 All other packets are received on channel 0.

DMA Arbiter

The DMA arbitrates between the transmit and receive paths of DMA channel 0, channel 1, and channel 2 for accessing descriptors and data buffers.

The fixed priority scheme is the default priority scheme for the DMA channels. In fixed priority scheme, the highest priority channel (channel 2) always wins the arbitration whenever it requests the bus. The *Fixed Priority Scheme for DMA Channels* table provides information about the priority levels of the DMA channels.

Table 24-50: Fixed Priority Scheme for DMA Channels

Priority Level	Channel
0 (low)	Channel 0
1	Channel 1
2 (high)	Channel 2

Slot Number Function

The slot number function schedules the data fetching by DMA from the system memory. This feature is useful when the source AV data must transmit at specific intervals. The transmit descriptor word 0 (TDES0) [6:3] bits program the slot number at which the DMA fetches the data from system memory. This 4-bit field allows the host to schedule data up to 16 slots of 125 micro-second each. This field is applicable only for the AV channels (channel 1 and channel 2).

When DMA fetches a transmit descriptor, it compares the slot number of the transmit descriptor with the internally generated reference slot interval. The slot interval is a counter that updates every 125 usec of the IEEE 1588 system time. In addition, the slot interval counter is initialized to zero when the value in the `EMAC_TM_SEC` register increments, that is, `EMAC_TM_NSEC` rolls over. The DMA fetches the data only if it matches the current slot or the next slot. The DMA remains in the descriptor fetch state until there is a match.

Programs can also set the `EMAC_DMA1_CHSFCS.ASC` bit to enable the DMA to fetch the data only if it matches the current slot or the next two slots.

NOTE: If the slot number in the descriptor is less than the reference slot number, the DMA takes it as a future slot.

Programs can enable the check for slot number by setting the `EMAC_DMA1_CHSFCS.ESC` bit. When this check is not enabled, the packets are fetched immediately after the descriptor is read. Programs can read the `EMAC_DMA1_CHSFCS.RSN` bit field to discover the value of the reference slot number in DMA.

Interrupts

Each DMA channel has its own dedicated interrupt channel, the software can take advantage of it for processing various DMA channels.

Credit-Based Shaper Algorithm Functions

The Traffic Manager and Scheduler (TMS) block (shown in the *Transmit and Receive Path Block Diagram* figure) uses the credit-based shaper algorithm to arbitrate the AV traffic in all channels and the legacy Ethernet traffic in channel 0. Channel 1 and channel 2 can be programmed to use the credit-based shaper algorithm. The following sections provide information about implementing the Credit-Based Shaper Algorithm:

Credit Value

The credit value is accumulated every transmit clock cycle, that is, 40 ns for 100 Mbps or 8 ns for 1000 Mbps. The credit to be added or subtracted per cycle can be fractional, based on the required `idleSlope` and `sendSlope` values as described in the following table.

Table 24-51: idleSlope and sendSlope Values

Mode	Values	Description
100 Mbps	portTransmitRate = 100 Mbps idleSlope = 70 Mbps (assuming 70% bandwidth reserved for a higher priority traffic class) sendSlope = 30 Mbps	Credit = 2.8 bits accumulates per cycle (40 ns) for the higher priority traffic class when besteffort frame is being transmitted. Credit = 1.2 bits drains per cycle (40 ns) when higher priority traffic class frame is being transmitted.

The DMA stores the channel traffic in the respective Tx FIFO based on the slot number in the transmit descriptor (if enabled) or depending on the bandwidth availability on the SCB.

The credit for a channel builds up only when the frame is available but it cannot be transmitted because the MAC is sending a frame from another channel. The EMAC supports another mode in which the credit can build up in advance for a channel in which no frame is available in its FIFO. This enables sending a burst of high priority traffic in a channel as soon as data is available. This can be enabled with the `EMAC_DMA1_CHCBSCTL.CC/EMAC_DMA2_CHCBSCTL.CC` bit of the channel 1 and channel 2 CBS control registers.

When the `EMAC_DMA1_CHCBSCTL.CC/EMAC_DMA2_CHCBSCTL.CC` bit is reset, the accumulated credit parameter in the Credit-Based Shaper Algorithm is set to zero if there is positive credit and there is no frame to transmit in a channel. The credit does not accumulate when there is no frame waiting in a channel and other channels are transmitting. When the `EMAC_DMA1_CHCBSCTL.CC/EMAC_DMA2_CHCBSCTL.CC` bit is set, the accumulated credit parameter in the Credit-Based Shaper Algorithm is not reset to zero if there is positive credit and no frame to transmit in a channel. The credit accumulates even when there is no frame waiting in a channel and other channels are transmitting.

idleSlopeCredit and sendSlopeCredit Values

The software must program the idleSlopeCredit and sendSlopeCredit values. The programmed values should be the credit accumulated or drained per clock cycle scaled by 1024, such as, $2.8 \times 1024 = 2867$ and $1.2 \times 1024 = 1229$ respectively. In addition, the software must program the hiCredit and loCredit values, scaled by 1024, to adjust for scaling of the idleSlopeCredit and sendSlopeCredit values.

This means that if computed hiCredit and loCredit values are 12000 bits and 3036 bits respectively, then the values to be programmed in the `EMAC_DMA1_CHHIC/EMAC_DMA2_CHHIC` and `EMAC_DMA1_CHLOC/EMAC_DMA2_CHLOC` registers are 12000×1024 bits and two's complement of 3036×1024 respectively.

Bandwidth Status

The hardware maintains the status of the actual bandwidth consumed by each higher priority channel (channel 1 and channel 2) in the CBS status registers (`EMAC_DMA1_CHCBSSTAT/EMAC_DMA2_CHCBSSTAT`). This allows the software to estimate the average bandwidth consumed by numerically higher traffic classes as compared to the reserved bandwidth.

The CBS status register gives the average number of bits transmitted during the previous programmed slot interval (1, 2, 4, 8, or 16 slots of 125 us) in a channel. The status register is updated even if the Credit-Based Shaper Algorithm is not enabled for a channel. The number of slots over which the average bits transmitted per slot are computed is programmed in the `EMAC_DMA1_CHCBSCTL.SLC/EMAC_DMA2_CHCBSCTL.SLC` bits. For example, if these bit fields are programmed for two slots, then the average bits are computed over slot numbers 0-1, 2-3, 4-5, and so on.

The value programmed in the `EMAC_DMA1_CHISC/EMAC_DMA2_CHISC` register of a channel is proportional to the bandwidth reserved for the channel. The software can allocate any bandwidth that is not used by the higher priority channel to the reserved bandwidth of the lower priority channel.

A lower priority channel, using the Credit-Based Shaper Algorithm, cannot use the unused reserved bandwidth of any higher priority channel that is using the Credit-Based Shaper Algorithm. However, a lower priority channel that is using the strict-priority algorithm can use the unused reserved bandwidth of any higher priority channel that uses the Credit-Based Shaper Algorithm. For example, channel 1 and channel 2 use the Credit-Based Shaper Algorithm (with reserved bandwidth of 50% and 25% respectively) and channel 0 uses the strict-priority algorithm. If channel 1 uses only 40% of the reserved bandwidth, then the remaining 10% is used by channel 0. The channel 2 cannot exceed the reserved bandwidth of 25%.

Energy-Efficient Ethernet

Energy-Efficient Ethernet (EEE) is an optional operational mode that enables the IEEE 802.3 Media Access Control (MAC) sublayer along with a family of Physical layers to operate in the Low-Power Idle (LPI) mode. The EEE operational mode supports the IEEE 802.3 MAC operation at 100 Mbps, 1000 Mbps, and 10 Gbps. The MAC supports the IEEE Standard 802.3az-2010 for EEE.

The LPI mode allows power saving by switching off parts of the communication device functionality when there is no data awaiting transmission and receipt. The systems on both sides of the link can disable some functionality and save power during the periods of low-link utilization. The MAC controls whether the system should enter or exit the LPI mode and communicates this to the PHY.

The EEE specifies the capabilities negotiation methods that the link partners can use to determine whether EEE is supported and then select the set of parameters that common to both devices.

NOTE: The EEE feature is not supported when the MAC is configured to use the RMII interface. You should activate the EEE mode only when the MAC is operating with RGMII interface.

NOTE: According to the Energy-Efficient Ethernet standard (IEEE 802.3az-2010), the LPI mode is supported only in the full-duplex mode. Therefore, you should not enable the LPI mode when the MAC Transmitter is configured for the half-duplex mode.

Transmit Path Functions

In the transmit path, the software must set the `EMAC_LPI_CTLSTAT.LPIEN` bit to indicate to the MAC to stop transmission and initiate the LPI protocol. The MAC completes the transmission in progress, generates its transmission status, and then starts transmitting the LPI pattern instead of the IDLE pattern during Interframe gap (IFG).

To make the PHY enter the LPI state, the MAC performs the following tasks:

1. De-asserts the `ETH0_TXCTL_TXEN` signal.
2. Asserts the `TX_ER` signal.
3. Sets `EMAC_TXD[n][3:0]` to 0x1 (for 100 Mbps) or `EMAC_TXD[n][7:0]` to 0x01 (for 1000 Mbps).

NOTE: The MAC maintains the same state of the `TX_EN`, `TX_ER`, and `EMAC_TXD[n]` signals for the entire duration during which the PHY remains in the LPI state.

4. Updates the status using the `EMAC_LPI_CTLSTAT.TLPIEN` bit and generates an interrupt.

To bring the PHY out of the LPI state, that is, when the software resets the `EMAC_LPI_CTLSTAT.LPIEN` bit, the MAC performs the following tasks:

1. Stops transmitting the LPI pattern and starts transmitting the IDLE pattern.
2. Starts the LPI TW TIMER: The MAC cannot start the transmission until the wake-up time specified for the PHY expires. The auto-negotiated wake-up interval is programmed using the `EMAC_LPI_TMRCTL.TWT` bit field.
3. Updates the LPI exit status using the `EMAC_LPI_CTLSTAT.TLPIEX` bit and generates an interrupt.

The *LPI Transitions (Transmit)* figure shows the behavior of `TX_EN`, `TX_ER`, and `EMAC_TXD[n][3:0]` signals during the LPI mode transitions.

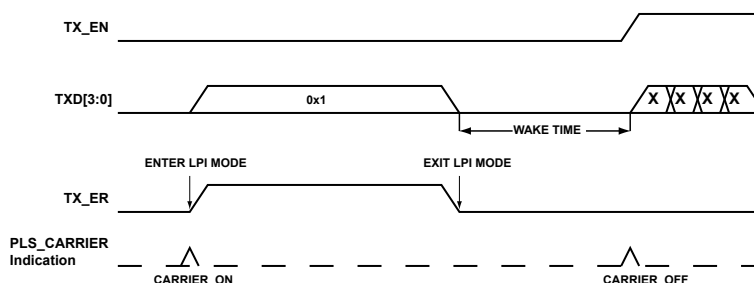


Figure 24-23: LPI Transitions (Transmit)

NOTE: The MAC does not stop the `TX_CLK` clock.

NOTE: If the MAC is in the Tx LPI mode and the Tx clock is stopped, the application should not write to CSR registers that are synchronized to Tx clock domain.

NOTE: If the MAC is in the LPI mode and the host issues a soft reset or hard reset, the MAC transmitter comes out of the LPI mode.

LPI Timers

LPI LS TIMER

The LPI LS timer counts, in milliseconds, the time expired since the link status is up. Programs can enable the monitoring of Bit 3 `EMAC_I_STAT.RGMIIIS` (link status) of Register 54 (SGMII/RGMII/SMII Control and Status Register) by setting the `EMAC_LPI_CTLSTAT.PLSEN` bit.

The link status is indicated to the MAC by Bit 3 (link status) of Register 54 (RGMII Control and Status Register) or the value programmed by the software in the `EMAC_LPI_CTLSTAT.PLS` bit. If the link status is not available in Register 54 (RGMII Control and Status Register), the software should get the PHY link status by reading the PHY register and accordingly update the `EMAC_LPI_CTLSTAT.PLS` bit.

This timer is cleared every time the link goes down. It starts to increment when the link is up again and continues to increment until the value of the timer becomes equal to the terminal count. When the terminal count is reached, the timer remains at the same value as long as the link is up. The terminal count is the value programmed in the `EMAC_LPI_TMRCTL.LST` bit field. The GMII interface does not assert the LPI pattern unless the terminal count is reached. This ensures a minimum time for which no LPI pattern is asserted after a link is established with the remote station. This period is defined as 1 second in the IEEE standard 802.3-az, version D2.0. The LPI LS timer is 10-bit wide. Therefore, the software can program up to 1023 milliseconds

LPI TW TIMER

The LPI TW timer counts, in microseconds, the time expired since the de-assertion of LPI. The terminal should be programmed using the `EMAC_LPI_TMRCTL.TWT` bit. The terminal count of the timer is the value of resolved Transmit TW that is the auto-negotiated time after which the MAC can resume the normal transmit operation. After exiting the LPI mode, the MAC resumes its normal operation after the TW timer reaches the terminal count.

The MAC supports the LPI TW timer in units of microsecond. The LPI TW timer is 16-bit wide. Therefore, the software can program up to 65535 us.

NOTE: Program the `EMAC_LPI_CTLSTAT.PLS` bit to 1'b0 before switching between the GMII and MII modes. This resets the internal timers. If the mode is changed after the LPI LS timer or LPI TW timer starts, the change in the TX clock frequency can result in incorrect timeout.

LPI Interrupt

The MAC generates the LPI interrupt when the Tx or Rx side enters or exits the LPI state. The interrupt is asserted when the LPI interrupt status is set. The LPI interrupt can be cleared by reading the [EMAC_LPI_CTLSTAT](#) register.

EMAC Event Control

The EMAC has dedicated interrupt signals registered with the system event controller (SEC) module. Various interrupt sources within the EMAC peripheral are shared through this interrupt line. Refer to the [System Event Controller \(SEC\) and Generic Interrupt Controller \(GIC\)](#) chapter for details on how interrupts work in this product and how to configure them.

EMAC Interrupt Signals

Interrupts from the EMAC are triggered from the EMAC DMA layer or the EMAC CORE layer. Interrupts are triggered from EMAC DMA when a particular status bit is set in the [EMAC_DMA0_STAT](#) register. An interrupt line is asserted only when the corresponding bits are enabled in the DMA interrupt enable register. Similarly, interrupts are triggered from the EMAC CORE when a particular MMC status bit, RGMII Link status bit, or PTP status bit is set in the interrupt status register.

An interrupt line is asserted only when the corresponding bits are enabled in the MMC mask registers for MMC counters or the interrupt mask register for PTP. DMA status register also reflects the MMC interrupt status. The following lists show the two groups of interrupts in the DMA status register.

NIS – Normal Interrupt source summary:

- Transmit Interrupt
- Transmit Buffer Unavailable
- Receive Interrupt
- Early Receive Interrupt

AIS – Abnormal Interrupt source summary:

- Transmit Process Stopped
- Transmit Jabber Timeout
- Receive FIFO Overflow
- Transmit Underflow
- Receive Buffer Unavailable
- Receive Process Stopped
- Receive Watchdog Timeout
- Early Transmit Interrupt
- Fatal Bus Error

The EMAC generates an interrupt only once for simultaneous, multiple events. The driver must read the [EMAC_DMA0_STAT](#) register for the cause of the interrupt. It can generate a new interrupt once the driver has cleared the appropriate bit in DMA status register.

For example, the controller generates a receive interrupt (`EMAC_DMA0_STAT.RI` bit) and the driver begins reading the [EMAC_DMA0_STAT](#) register. Next, a receive buffer unavailable interrupt (`EMAC_DMA0_STAT.RU` bit) occurs. The driver clears the `EMAC_DMA0_STAT.RI` bit but the internal interrupt signal is not deasserted, because of the active or pending `EMAC_DMA0_STAT.RU` interrupt. The driver must scan all of the descriptors, from the last recorded position to the first one owned by the DMA, to know which descriptor has asserted the interrupt.

Interrupts are cleared by writing a 1 to the corresponding bit position in the `EMAC_DMA0_STAT` register. When all the enabled interrupts within a group are cleared, the corresponding summary bit is cleared.

An interrupt delay timer provides (receive interrupt watchdog timer register) flexible control of the receive interrupt.

When the interrupt timer is programmed with a non-zero value, it is activated as soon as the RxDMA transfers a received frame to system memory. The transfer occurs without asserting the receive interrupt. This interrupt is not enabled in the corresponding receive descriptor (`RDES1[31]` in the receive DMA descriptors).

When this timer runs out (per the programmed value), the `EMAC_DMA0_STAT.RI` bit is set. The interrupt is asserted when the corresponding `EMAC_DMA0_STAT.RI` bit is enabled in the interrupt enable register. The timer is disabled before it runs out, when a frame is transferred to memory and when the `EMAC_DMA0_STAT.RI` bit is set because it is enabled for that descriptor.

PHYINT Interrupt Signal

A PHY device can notify the EMAC when it detects changes to the link status, such as auto-negotiation or a duplex-mode change. The external PHY chip typically includes an interrupt generation pin to aid this status change notification to the MAC. This signal is typically called PHYINT. A falling or rising edge on this signal can detect a PHY interrupt at the EMAC.

In the processor, any of the GPIO pins can be used as a PHYINT signal. Use the following procedure to configure a GPIO as a PHYINT signal.

1. Program the GPIO to detect a falling or rising edge sensitive interrupt.
2. Program the PHY to generate the interrupt on a signal status change.
3. If PHYINT is asserted, read the PHY status register through the station management interface.

NOTE: The PHYINT is not part of EMAC module. However, any GPIO pin can be configured to interrupt the processor when a rising edge generated by PHY is detected.

Refer to the PORT chapter for more info on configuring GPIO pins for input.

EMAC Programming Model

This section provides the programming model of Ethernet MAC peripheral for developers.

EMAC Programming Steps

The following sections provide some general programming information. The steps are written for EMAC DMA0, but apply to DMA1 and DMA2 as well.

DMA Initialization

Use the following procedure to initialize DMA. This procedure applies to all DMA channels.

1. Perform a software reset by setting the `EMAC_DMA0_BUSMODE.SWR` bit. This action resets all of the EMAC internal registers and logic.
2. Wait for the completion of the reset process by polling the `EMAC_DMA0_BUSMODE.SWR` bit which is only cleared (automatically) after the reset operation completes.
3. Poll the `EMAC_DMA0_BMSTAT.BUSRD` and `EMAC_DMA0_BMSTAT.BUSWR` bits to confirm that all previously initiated (before software-reset) or ongoing SCB transactions are complete.
4. Program the required fields in the `EMAC_DMA0_BMODE` register:
 - a. Address aligned bursts
 - b. Fixed burst or undefined burst
 - c. Burst length values and burst mode values
 - d. Descriptor length (only valid when using ring mode)
5. Program the SCB interface options in the `EMAC_DMA0_BMODE` register. If fixed burst-length is enabled, then select the maximum burst-length possible on the SCB bus (bits `EMAC_DMA0_BMODE.BLEN4`, `EMAC_DMA0_BMODE.BLEN8`, `EMAC_DMA0_BMODE.BLEN16`).
6. Create a proper descriptor chain for transmit and receive. In addition, ensure that the DMA owns the receive descriptors (the `OWN` bit of the descriptor is set). For OSF mode, create at least two descriptors.
7. Ensure that the software creates three or more different transmit or receive descriptors in the chain before reusing any of the descriptors.
8. Initialize the `EMAC_DMA0_RXDSC_CUR` and `EMAC_DMA0_TXDSC_CUR` registers with the base address of the receive and transmit descriptors respectively.
9. Program the required fields in the `EMAC_DMA0_OPMODE` register to initialize the mode of operation as follows:
 - a. Receive and transmit store and forward
 - b. Receive and transmit threshold control
 - c. Error Frame and undersized good frame forwarding enable
 - d. OSF mode
10. Clear the interrupt requests by writing to those bits of the `EMAC_DMA0_STAT` register (interrupt bits only) that are set. For example, by writing 1 into bit 16, the normal interrupt summary clears this bit.
11. Enable the interrupts by programming the `EMAC_DMA0_IEN` register.
12. Start the receive and transmit DMA by setting the `EMAC_DMA0_OPMODE.SR` and `EMAC_DMA0_OPMODE.ST` bits.

EMAC CORE Initialization

Use the following procedure to initialize the EMAC CORE.

1. Program the EMAC management address register ([EMAC_SMI_ADDR](#)) for controlling the management cycles for external PHY. For example, physical layer address ([EMAC_SMI_ADDR.PA](#)). In addition, set the [EMAC_SMI_ADDR.SMIB](#) bit for writing into PHY and reading from PHY.
2. Read the 16-bit data of management address register ([EMAC_SMI_DATA](#)) from the PHY for link up, speed of operation, and mode of operation. Specify the appropriate address value in the [EMAC_SMI_ADDR.PA](#) bit field.
3. Program the MAC address in the [EMAC_ADDR0_HI](#) and [EMAC_ADDR0_LO](#) registers.
4. If using hash filtering, program the hash table high and low registers register ([EMAC_HASHTBL_HI](#), [EMAC_HASHTBL_LO](#)).
5. Program the fields to set the appropriate filters for the incoming frames in the MAC frame filter register ([EMAC_MACFRMFILT](#)):
 - a. Receive all.
 - b. Promiscuous mode.
 - c. Hash or perfect filter.
 - d. Unicast, multicast, broadcast, and control frames filter settings.
6. Program the fields for proper flow control in flow control register ([EMAC_FLOWCTL](#)):
 - a. Pause time and other pause frame control bits.
 - b. Receive and transmit flow control bits.
 - c. Flow control busy or backpressure activate.
7. Program the EMAC interrupt mask register bits ([EMAC_IMSK](#)), as needed.
8. Program the appropriate fields in MAC configuration register ([EMAC_MACCFG](#)). For example, inter-frame gap while transmission and jabber disable. Based on the auto-negotiation desired, set the duplex mode ([EMAC_MACCFG.DM](#) bit) or speed select ([EMAC_MACCFG.FES](#) bit).
9. Set the transmit enable ([EMAC_MACCFG.TE](#)) and receive enable ([EMAC_MACCFG.RE](#)) bits.

Performing Normal Transmit and Receive Operations

For normal transmit and receive interrupts, the program must first read the interrupt status.

1. Poll the descriptors, reading the status of the descriptor owned by the application (either transmit or receive).
2. Set the appropriate values for the descriptors, ensuring that transmit and receive descriptors are DMA descriptors to resume the transmission and reception of data.

ADDITIONAL INFORMATION: If the descriptors are not DMA (or no descriptor is available), the DMA goes into SUSPEND state.

3. Write a 0 into the Tx/Rx poll demand registers ([EMAC_DMA0_TXPOLL](#) and [EMAC_DMA0_RXPOLL](#)).
Transmit or receive operations are resumed by freeing the descriptors and issuing a poll demand.
4. Read (for the debug process), the values of the current host transmitter or receiver descriptor address pointer ([EMAC_DMA0_TXDSC_CUR](#), [EMAC_DMA0_RXDSC_CUR](#)) registers.
5. Read (for the debug process), the values of the current host transmit buffer address pointer and receive buffer address pointer ([EMAC_DMA0_TXBUF_CUR](#), [EMAC_DMA0_RXBUF_CUR](#)) registers.

Stopping and Starting Transfers

Use the following procedure to stop and start EMAC transfers.

1. Disable the transmit DMA (if applicable), by clearing the [EMAC_DMA0_OPMODE](#).ST bit.
2. Wait for any previous frame transmissions to complete. Check for completion by reading the appropriate bits of the debug register ([EMAC_DBG](#)).
3. Disable the MAC transmitter and MAC receiver by clearing the [EMAC_MACCFG](#).TE and [EMAC_MACCFG](#).RE bits.
4. Disable the receive DMA (if applicable), after ensuring that the data in the receive FIFO transfers to the system memory by reading the [EMAC_DBG](#) register.
5. Make sure that both the transmit and receive FIFOs are empty.
6. To restart the operation, first start the DMA, and then enable the MAC transmitter and receiver.

Interrupts and Interrupt Service Routines

The following procedure describes specific steps for enabling interrupts and using their ISRs.

This procedure is typically performed with EMAC and DMA initialization and operations.

1. Receive interrupts are enabled for descriptors by default. Enable transmit interrupts for individual descriptors by setting the IC bit (bit 30) in the TDES0 word of the transmit descriptor.
2. Enable the required bits in the DMA interrupt enable register ([EMAC_DMA0_IEN](#).NIE).

ADDITIONAL INFORMATION: Setting the [EMAC_DMA0_IEN](#).NIE or [EMAC_DMA0_IEN](#).AIE bits can turn on the occurrence of all normal or abnormal interrupt conditions. Individual conditions can also be enabled on using individual bits in the [EMAC_DMA0_IEN](#) register.

3. Enable MMC overflow interrupts by setting appropriate bits in the [EMAC_MMC_RXIMSK](#) and [EMAC_MMC_TXIMSK](#) registers.
4. Enable PTP interrupts by setting the [EMAC_IMSK](#).TS bit.

5. Once an EMAC interrupt is asserted and the SEC branches execution to the EMAC ISR, perform the following software program sequence.
 - a. Read DMA status from the `EMAC_DMA0_STAT` register.
 - b. Clear the interrupt source by writing 1 (W1C) to the bits that are set in the `EMAC_DMA0_STAT` register.
 - c. Check for normal/abnormal/mmc/ptp interrupts by parsing the status bits read earlier, and call the appropriate service function.

ADDITIONAL INFORMATION: Normal interrupt assertions include the transmit and receive interrupt. Abnormal interrupt assertions include the receive underflow.

6. The MMC handler functions use the following sequence.
 - a. Read the `EMAC_ISTAT` register and parse for the `EMAC_ISTAT.MMCTX` and `EMAC_ISTAT.MMCRX` bits to determine if the interrupt is a transmit counter or receive counter-interrupt.
 - b. Read the `EMAC_MMC_RXINT` or `EMAC_MMC_TXINT` registers to determine which of the counters have triggered the interrupt.
 - c. Read the respective MMC counter that caused the interrupt to clear it.
7. PTP handler functions use the following sequence:
 - a. Read the `EMAC_ISTAT.TS` bit to determine if a PTP interrupt occurred.
 - b. Read `EMAC_TM_STMPSTAT` register to determine the interrupt source by parsing the `EMAC_TM_STMPSTAT.ATSTS`, `EMAC_TM_STMPSTAT.TSTARGET0` - `EMAC_TM_STMPSTAT.TSTARGET3`, and `EMAC_TM_STMPSTAT.TSSOVF` bits.
 - c. Clear the interrupt source by reading the `EMAC_TM_STMPSTAT` register.

Enabling Checksum for Transmit and Receive

Use the following steps to enable transmit and receive checksums.

Enabling receive and transmit checksums is typically performed with EMAC and DMA initialization and operations. Transmit and receive checksum features are independent of each other.

1. To enable transmit checksum insertion:
 - a. Enable store-and-forward mode in the FIFO by setting the `EMAC_DMA0_OPMODE.TSF` bit.
 - b. Ensure that the transmit frame can be contained within the 256 byte Tx FIFO conforming to the size rule: $\text{FIFO Depth} - \text{PBL} - 3$ FIFO locations, where PBL is burst length.
 - c. Program the following necessary parameters for transmit checksum, by programming (CIC) checksum insertion control in TDES0: IP header checksum, IP header checksum and payload checksum, IP header checksum, payload checksum, and pseudo header checksum.

A higher layer such as the IP stack sends out the packet to the EMAC which inserts the checksum as configured.

2. To enable receive checksum verification:

- a. Enable receive checksum offload engine by setting the `EMAC_MACCFG.IPC` bit.
- b. Enable 8 word descriptor (32 bytes), by setting the `EMAC_DMA0_BUSMODE.ATDS` bit.
- c. Provide a total of 8 x 32-bit word space for the receive descriptor.
- d. Wait for the receive interrupt and check for extended status availability by parsing bit 0 in the `RDES0` word.
- e. If extended status available, read `RDES4` and pass to a higher layer such as the IP stack.

The higher software layer can check for IPv4/IPv6/payload type and checksum payload or header errors.

Programming the System Time Module

Use the following procedure to configure the PTP module

1. Enable the PTP module by setting the `EMAC_TM_CTL.TSENA` bit 0.
2. System Time Initialization
 - a. The time (seconds and nanoseconds) at which system time is initialized. Write the time into the `EMAC_TM_SECUPDT` and `EMAC_TM_NSECUPDT` registers.
 - b. Set `EMAC_TM_CTL.TSINIT` bit. System time is initialized and this bit is cleared automatically.
 - c. Configure binary or digital rollover of the `EMAC_TM_NSEC` register using the `EMAC_TM_CTL.TSCTRLSSR` bit.
3. System Time Coarse Correction
 - a. Write the offset time (seconds and nanoseconds) to add to or subtract from the system time using the `EMAC_TM_SECUPDT` and `EMAC_TM_NSECUPDT` registers.
 - b. Choose between add or subtract offset time using the `EMAC_TM_NSECUPDT.ADDSUB` bit.
 - c. Set the `EMAC_TM_CTL.TSUPDT` bit to correct system time with offset time. This bit is cleared automatically.
4. System Time Fine Correction
 - a. Calculate the addend value based on the input PTP clock frequency and the frequency requirement. See [Fine Correction Method](#).
 - b. Write the calculated addend value in `EMAC_TM_ADDEND` register and set the `EMAC_TM_CTL.TSADDREG` bit to update the addend value. This bit is cleared automatically.
 - c. Configure the `EMAC_TM_SUBSEC` register based on new PTP frequency.

5. Target Time Trigger (Alarm)

- a. Set the `EMAC_IMSK.TS` bit to enable PTP interrupts.
- b. Program the `EMAC_TM_PPSCTL.TRGTMODSEL0` bit with 00 or 10 (for PPS start or stop time programming).
- c. Program the time when the interrupt must trigger using the `EMAC_TM_PPS0TGTM` and `EMAC_TM_PPS0NTGTM` registers. The programmed time must be greater than the current system time.

ADDITIONAL INFORMATION: If the programmed time is not greater than the target time, a programming error occurs. The EMAC uses the `EMAC_TM_STMPSTAT.TSTRGTERR0` bit to indicate the error.

- d. Set the `EMAC_TM_CTL.TSTRIG` bit to enable the target time trigger interrupt.

After the system time reaches the programmed target time (in step 2), the target time trigger interrupt occurs. The `EMAC_TM_STMPSTAT.TSTRGTERR0` and `EMAC_ISTAT.TS` bits indicate the error. The `EMAC_TM_CTL.TSTRIG` bit is cleared automatically.

Programming the PTP for Frame Detection and Time Stamping

Use the following procedure to configure the PTP module.

1. For time stamping a transmitting frame, set the `TTSE` bit in the `TDES0` register of the corresponding frame.
2. Extend the descriptor word length from 4 words to 8 words by setting the `EMAC_DMA0_BUSMODE.ATDS` bit.
3. Configure bits 18–10 in the `EMAC_TM_CTL` register so that the PTP module detects and time stamps only specific types of received frames. Refer to the `EMAC_TM_CTL` register description for more information.
4. Enable the PTP module by setting the `EMAC_TM_CTL.TSENA` bit.
5. Initialize the system time.
6. Verify the `RDES4` register for the status of the received frame and the `RDES6` and `RDES7` registers for time stamp nanoseconds and seconds value.

Programming for Auxiliary Time Stamps

1. Set the `EMAC_IMSK.TS` bit to enable PTP interrupts.
2. Set the `EMAC_TM_CTL.TSENA` bit to enable the PTP module.
3. Initialize system time.

ADDITIONAL INFORMATION: Whenever a rising edge on the auxiliary time stamp trigger pin is detected, system time seconds and nanoseconds are captured and stored into 4-deep auxiliary time stamp FIFO. An auxiliary time stamp trigger interrupt occurs. The EMAC uses the `EMAC_TM_STMPSTAT.ATSTS` and the `EMAC_IMSK.TS` bits to indicate the interrupt.

4. Read the contents of the FIFO one-by-one through `EMAC_TM_AUXSTMP_SEC` and `EMAC_TM_AUXSTMP_NSEC` registers. One level of the FIFO is cleared when the `EMAC_TM_AUXSTMP_SEC` register is read. Therefore, read the `EMAC_TM_AUXSTMP_NSEC` register before the `EMAC_TM_AUXSTMP_SEC` register.
5. Set the `EMAC_TM_CTL.ATSFC` bit to clear the FIFO.

Programming Fixed Pulse-Per-Second Output

Use the following procedure to program the fixed PPS output.

1. Enable the PTP module by setting the `EMAC_TM_CTL.TSENA` bit.
2. Output the PPS waveform by configuring the `EMAC_TM_PPSTCTL.PPSCTL0` bits and binary or digital roll-over using the `EMAC_TM_CTL.TSCTRLSSR` bit. See [Fixed Pulse-Per-Second Output](#).

Programming Flexible Pulse-Per-Second Output

Use the following procedure to program flexible PPS output.

1. Enable the PTP module by setting the `EMAC_TM_CTL.TSENA` bit.
2. Set the `EMAC_TM_PPSTCTL.PPSEN` bit to enable flexible PPS output.
3. Program the `EMAC_TM_PPSTCTL.TRGTMODSEL0` bits with 11 or 10 (for target time trigger interrupt).
4. Program the start time value when the PPS output starts using the `EMAC_TM_PPS0TGTM` and `EMAC_TM_PPS0NTGTM` registers. Ensure that the `EMAC_TM_PPS0NTGTM.TSTRBUSY` bit is reset before programming the target time registers again.
5. Program the period of the PPS signal output using the `EMAC_TM_PPS0INTVL` register for pulse train output. Program the width of the PPS signal output in the `EMAC_TM_PPS0WIDTH` register for single pulse or pulse train output.
6. Ensure that the `EMAC_TM_PPSTCTL.PPSCTL0` bits are cleared. Then, program the bits to 0001 to start single pulse, or to 0010 to start pulse train at programmed start time (Step 4).

ADDITIONAL INFORMATION: The PPS pulse train is free-running unless stopped by a STOP pulse train at time command (`EMAC_TM_PPSTCTL.PPSCTL0 = 0100`) or STOP pulse train immediately command (`EMAC_TM_PPSTCTL.PPSCTL0 = 0101`).

7. The start of pulse generation can be canceled by giving the cancel start command (`EMAC_TM_PPSTCTL.PPSCTL0 = 0011`) before the programmed start time (Step 4) elapses.
8. Program the stop time value when the PPS output must stop using the `EMAC_TM_PPS0TGTM` and `EMAC_TM_PPS0NTGTM` registers. Ensure that the `EMAC_TM_PPS0NTGTM.TSTRBUSY` bit is reset before programming the target time registers again.
9. Ensure that the `EMAC_TM_PPSTCTL.PPSCTL0` bits are cleared. Then, program the bits to 0100. This programming stops the train of pulses on PPS signal output after the programmed stop time (Step 8) elapses.

ADDITIONAL INFORMATION: The pulse train can be stopped immediately by giving the STOP pulse train immediately command (EMAC_TM_PPSCCTL.PPSCCTL0 = 0101). Program the EMAC_TM_PPSCCTL.PPSCCTL0 bits to 0110 before the programmed stop time (Step 8) elapses to cancel the stop pulse train command (given in Step 9).

EMAC Programming Concepts

The following sections provide basic information and guidelines to help with programming the EMAC module.

IEEE 802.3 Ethernet Packet Structure

The *IEEE 802.3 Frame Structure* table provides typical frame format of an Ethernet packet. Refer to the IEEE standards for detailed information on Ethernet packets and their format.

Table 24-52: IEEE 802.3 Frame Structure

Parameter	Description	Position in Ethernet Packet	Total Bytes
PREAMBLE	This parameter is a 56-bit (7-byte) pattern of alternating 1 and 0 bits (#10101010), which allows devices on the network to detect a new incoming frame for synchronization.	1	7
SFD	The SFD (#10101011) is a 1-byte pattern designed to break the preamble pattern, and signal the start of the actual frame.	2	1
DA	48-bit destination address. This parameter can be a unicast, multicast, or broadcast address.	3	6
SA	48-bit long source address, typically a unicast, multicast, or broadcast address.	4	6
LT	Typically this field is the length, in terms of the number of bytes, and can be anywhere between 0–1500. When the value is greater than or equal to 0x0600, this field also indicates the type of special payload carried by the frame. Examples include 0x8808 for flow control and 0x0800 for IPv4.	5	2
DATA	Actual application data payload, usually between 0–1500.	6	0–1500
PAD	This field compensates for data frames that are shorter than 64 bytes long, not including the preamble.	7	0–46
FCS	The frame check sequence is a 32-bit cyclic redundancy check that detects corrupted data within the entire frame. This parameter is generated from a CRC-32 polynomial code (CRC-32-IEEE): $G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$.	8	4

Frame Size Statistics for Application Software

Table 24-53: Ethernet Frame Size Statistics

Frame size statistics	VLAN-specific change Comments	
Information bytes/Header	4 byte 802.1Q header inserted after source address and before Type/LAN in 802.3 packets = 22 bytes.	$6 \times 2 + 2 + 4 = 18$ bytes (DA+SA+LT+FCS)
Minimum Frame Size (typical)	If DATA is NULL, 42-byte padding makes 64 bytes (42 + 22)	64 bytes. If DATA is NULL, 46-byte padding makes 64 bytes (46 + 18)
Maximum Frame Size (typical)	1522 bytes	1518 bytes (1500 bytes DATA and 18-bytes header)
Jumbo Frame Size	9022 bytes	Typical industry standard. Ethernet jumbo frame size treated as 9018 bytes.

Software Visualization of Programmable Packet Size

The *Visualization of Programmable Packet Size* table provides the byte sizes of packets with various configurations.

Table 24-54: Visualization of Programmable Packet Size

Size in Bytes	Comments
16384	Receive watchdog and transmit jabber disabled, jumbo frames enabled.
10240	Receive watchdog and transmit jabber disabled, jumbo frames disabled.
2048	Receive watchdog and transmit jabber enabled.
1518	Typical max size of Ethernet frame. Receive watchdog and transmit jabber enabled.
64	Typical minimum size of Ethernet frame.
< 64	Runt frames requiring Zero-PAD.

Ethernet Packet Structure in C

The following is an example for Ethernet packet structure in the C language.

```
typedef struct ETHER_PACKET
{
    char  dst_addr[6];           //destination address
    char  src_addr[6];           //source address
    char  length[2];             //length of actual data
    char  data[DATA_SIZE];       //application data
    char  fdlimit[DELIMIT_SIZE]; //32-bit delimit (if manual appending)
    char  fcs[4];                //crc frame checksum, used by RX buffer.
} ETHER_PACKET;
```


DMA Descriptor Implementation in C

The following code is a simple implementation of descriptors in ring and chain model in C language. Typically 4 WORDs (32-bit) are used for descriptors. Using checksum offload or the PTP engine requires 8 WORDs. Only high-level common functions across transmit and receive descriptors are considered here.

```
/* DMA Ring Descriptor */
typedef struct EMAC_DMADESC_RING
{
    unsigned int      Status;          //TDES0 OR RDES0
    unsigned int      ControlDesc;     //TDES1 OR RDES1
    unsigned int      StartAddr1;     //TDES2 OR RDES2
    unsigned int      StartAddr2;     //TDES3 OR RDES3
#ifdef CHECKSUM_OFFLOAD
    struct EMAC_EXT_STAT ExtendedStat;
#endif
} EMAC_DMADESC_RING;

/* DMA Chain Descriptor */
typedef struct EMAC_DMADESC_CHAIN
{
    unsigned int      Status;          //TDES0 OR RDES0
    unsigned int      ControlDesc;     //TDES1 OR RDES1
    unsigned int      StartAddr;      //TDES2 OR RDES2
    struct EMAC_DMADESC_CHAIN *pNextDesc; //TDES3 OR RDES3
#ifdef CHECKSUM_OFFLOAD
    struct EMAC_EXT_STAT ExtendedStat;
#endif
} EMAC_DMADESC_CHAIN;

/* Extended Status Descriptor with PTP not enabled*/
typedef struct EMAC_EXT_STAT
{
#ifdef RX_DESC
    unsigned int      CheckSumStat;    //RDES4
#endif
#ifdef TX_DESC
    unsigned int      Reserved;        //TDES4
#endif
    unsigned int      Reserved;        //RDES5 OR TDES5
    unsigned int      Reserved;        //RDES6 OR TDES6
    unsigned int      Reserved;        //RDES7 OR TDES7
} EMAC_EXT_STAT;
```

PTP Header Structure in C

The following code is an example of the PTP message format.

```
/* PTP Message Format (Refer to PTP Frame Over IPv4) */
typedef struct EMAC_PTP_HEADER
{
    unsigned char      messageType:4, //PTP Version 2 message type
```

```

transportSpecific:4;
unsigned char    versionPTP;           //PTP Version (1 or 2)
unsigned short   messageLength;
unsigned char    domainNumber;
unsigned char    RESERVED1;
unsigned short   flagField;
unsigned char    correctionField[8];
unsigned char    RESERVED2[4];
unsigned char    sourcePortIdentity[10];
unsigned short   sequenceid;
unsigned char    controlField;         //PTP Version 1 message type
unsigned char    logMessageInterval;
}EMAC_PTP_HEADER;

```

ADSP-2159x EMAC Register Descriptions

Ethernet MAC (EMAC) contains the following registers.

Table 24-55: ADSP-2159x EMAC Register List

Name	Description
EMAC_ADDR0_HI	MAC Address 0 High Register
EMAC_ADDR0_LO	MAC Address 0 Low Register
EMAC_ADDR1_HI	MAC Address 1 High Register
EMAC_ADDR1_LO	MAC Address 1 Low Register
EMAC_DBG	Debug Register
EMAC_DMA0_BMMODE	DMA SCB Bus Mode Register
EMAC_DMA0_BMSTAT	DMA SCB Status Register
EMAC_DMA0_BUSMODE	DMA Bus Mode Register
EMAC_DMA0_IEN	DMA Interrupt Enable Register
EMAC_DMA0_MISS_FRM	DMA Missed Frame Register
EMAC_DMA0_OPMODE	DMA Operation Mode Register
EMAC_DMA0_RXBUF_CUR	DMA Rx Buffer Current Register
EMAC_DMA0_RXDSC_ADDR	DMA Rx Descriptor List Address Register
EMAC_DMA0_RXDSC_CUR	DMA Rx Descriptor Current Register
EMAC_DMA0_RXIWDOG	DMA Rx Interrupt Watch Dog Register
EMAC_DMA0_RXPOLL	DMA Rx Poll Demand register
EMAC_DMA0_STAT	DMA Status Register
EMAC_DMA0_TXBUF_CUR	DMA Tx Buffer Current Register
EMAC_DMA0_TXDSC_ADDR	DMA Tx Descriptor List Address Register

Table 24-55: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_DMA0_TXDSC_CUR	DMA Tx Descriptor Current Register
EMAC_DMA0_TXPOLL	DMA Tx Poll Demand Register
EMAC_DMA1_BUSMODE	DMA Bus Mode Register
EMAC_DMA1_CHCBSCTL	Channel 1 Credit Shaping Control Register
EMAC_DMA1_CHCBSSTAT	Channel 1 Average Traffic Transmitted Register
EMAC_DMA1_CHHIC	Channel 1 High Credit Value Register
EMAC_DMA1_CHISC	Channel 1 Idle Slope Credit Value Register
EMAC_DMA1_CHLOC	Channel 1 Low Credit Value Register
EMAC_DMA1_CHSFCS	Channel 1 Control Bits for Slot Function Register
EMAC_DMA1_CHSSC	Channel 1 Send Slope Credit Value Register
EMAC_DMA1_IEN	DMA Interrupt Enable Register
EMAC_DMA1_MISS_FRM	DMA Missed Frame Register
EMAC_DMA1_OPMODE	DMA Operation Mode Register
EMAC_DMA1_RXBUF_CUR	DMA Rx Buffer Current Register
EMAC_DMA1_RXDSC_ADDR	DMA Rx Descriptor List Address Register
EMAC_DMA1_RXDSC_CUR	DMA Rx Descriptor Current Register
EMAC_DMA1_RXIWDOG	DMA Rx Interrupt Watch Dog Register
EMAC_DMA1_RXPOLL	DMA Rx Poll Demand Register
EMAC_DMA1_STAT	DMA Status Register
EMAC_DMA1_TXBUF_CUR	DMA Tx Buffer Current Register
EMAC_DMA1_TXDSC_ADDR	DMA Tx Descriptor List Address Register
EMAC_DMA1_TXDSC_CUR	DMA Tx Descriptor Current Register
EMAC_DMA1_TXPOLL	DMA Tx Poll Demand Register
EMAC_DMA2_BUSMODE	DMA Bus Mode Register
EMAC_DMA2_CHCBSCTL	Channel 2 Credit Shaping Control Register
EMAC_DMA2_CHCBSSTAT	Channel 2 Avg Traffic Transmitted Status Register
EMAC_DMA2_CHHIC	Channel 2 High Credit Value Register
EMAC_DMA2_CHISC	Channel 2 Idle Slope Credit Value Register
EMAC_DMA2_CHLOC	Channel 2 Low Credit Value Register
EMAC_DMA2_CHSFCS	Channel 2 Control Bits for Slot Function Register
EMAC_DMA2_CHSSC	Channel 2 Send Slope Credit Value Register

Table 24-55: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_DMA2_IEN	DMA Interrupt Enable Register
EMAC_DMA2_MISS_FRM	DMA Missed Frame Register
EMAC_DMA2_OPMODE	DMA Operation Mode Register
EMAC_DMA2_RXBUF_CUR	DMA Rx Buffer Current Register
EMAC_DMA2_RXDSC_ADDR	DMA Rx Descriptor List Address Register
EMAC_DMA2_RXDSC_CUR	DMA Rx Descriptor Current Register
EMAC_DMA2_RXIWDOG	DMA Rx Interrupt Watch Dog Register
EMAC_DMA2_RXPOLL	DMA Rx Poll Demand register
EMAC_DMA2_STAT	DMA Status Register
EMAC_DMA2_TXBUF_CUR	DMA Tx Buffer Current Register
EMAC_DMA2_TXDSC_ADDR	DMA Tx Descriptor List Address Register
EMAC_DMA2_TXDSC_CUR	DMA Tx Descriptor Current Register
EMAC_DMA2_TXPOLL	DMA Tx Poll Demand Register
EMAC_FLOWCTL	FLow Control Register
EMAC_GIGE_CTLSTAT	RGMII Control and Status Register
EMAC_HASHTBL_HI	Hash Table High Register
EMAC_HASHTBL_LO	Hash Table Low Register
EMAC_IMSK	Interrupt Mask Register
EMAC_IPC_RXIMSK	MMC IPC Rx Interrupt Mask Register
EMAC_IPC_RXINT	MMC IPC Rx Interrupt Register
EMAC_ISTAT	Interrupt Status Register
EMAC_L3L4_CTL	Layer3 and Layer4 Control Register
EMAC_L3_ADDR0	Layer 3 Address0 Register
EMAC_L3_ADDR1	Layer 3 Address1 Register
EMAC_L3_ADDR2	Layer 3 Address2 Register
EMAC_L3_ADDR3	Layer 3 Address3 Register
EMAC_L4_ADDR	Layer 4 Address Register
EMAC_LPI_CTLSTAT	Low Power Idle Control and Status Register
EMAC_LPI_TMRCTL	Low Power Idle Timeout Register
EMAC_MACCFG	MAC Configuration Register
EMAC_MACFRMFILT	MAC Rx Frame Filter Register

Table 24-55: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_MAC_AVCTL	AV MAC Control Register
EMAC_MMC_CTL	MMC Control Register
EMAC_MMC_RXIMSK	MMC Rx Interrupt Mask Register
EMAC_MMC_RXINT	MMC Rx Interrupt Register
EMAC_MMC_TXIMSK	MMC TX Interrupt Mask Register
EMAC_MMC_TXINT	MMC Tx Interrupt Register
EMAC_RX1024TOMAX_GB	Rx 1024- to Max-Byte Frames (Good/Bad) Register
EMAC_RX128TO255_GB	Rx 128- to 255-Byte Frames (Good/Bad) Register
EMAC_RX256TO511_GB	Rx 256- to 511-Byte Frames (Good/Bad) Register
EMAC_RX512TO1023_GB	Rx 512- to 1023-Byte Frames (Good/Bad) Register
EMAC_RX64_GB	Rx 64-Byte Frames (Good/Bad) Register
EMAC_RX65TO127_GB	Rx 65- to 127-Byte Frames (Good/Bad) Register
EMAC_RXALIGN_ERR	Rx alignment Error Register
EMAC_RXBCASTFRM_G	Rx Broadcast Frames (Good) Register
EMAC_RXCRC_ERR	Rx CRC Error Register
EMAC_RXCTLFRM_G	Rx Good Control Frames Register
EMAC_RXFIFO_OVF	Rx FIFO Overflow Register
EMAC_RXFRMCNT_GB	Rx Frame Count (Good/Bad) Register
EMAC_RXICMP_ERR_FRM	Rx ICMP Error Frames Register
EMAC_RXICMP_ERR_OCT	Rx ICMP Error Octets Register
EMAC_RXICMP_GD_FRM	Rx ICMP Good Frames Register
EMAC_RXICMP_GD_OCT	Rx ICMP Good Octets Register
EMAC_RXIPV4_FRAG_FRM	Rx IPv4 Datagrams Fragmented Frames Register
EMAC_RXIPV4_FRAG_OCT	Rx IPv4 Datagrams Fragmented Octets Register
EMAC_RXIPV4_GD_FRM	Rx IPv4 Datagrams (Good) Register
EMAC_RXIPV4_GD_OCT	Rx IPv4 Datagrams Good Octets Register
EMAC_RXIPV4_HDR_ERR_FRM	Rx IPv4 Datagrams Header Errors Register
EMAC_RXIPV4_HDR_ERR_OCT	Rx IPv4 Datagrams Header Errors Register
EMAC_RXIPV4_NOPAY_FRM	Rx IPv4 Datagrams No Payload Frame Register
EMAC_RXIPV4_NOPAY_OCT	Rx IPv4 Datagrams No Payload Octets Register
EMAC_RXIPV4_UDSBL_FRM	Rx IPv4 UDP Disabled Frames Register

Table 24-55: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_RXIPV4_UDSBL_OCT	Rx IPv4 UDP Disabled Octets Register
EMAC_RXIPV6_GD_FRM	Rx IPv6 Datagrams Good Frames Register
EMAC_RXIPV6_GD_OCT	Rx IPv6 Good Octets Register
EMAC_RXIPV6_HDR_ERR_FRM	Rx IPv6 Datagrams Header Error Frames Register
EMAC_RXIPV6_HDR_ERR_OCT	Rx IPv6 Header Errors Register
EMAC_RXIPV6_NOPAY_FRM	Rx IPv6 Datagrams No Payload Frames Register
EMAC_RXIPV6_NOPAY_OCT	Rx IPv6 No Payload Octets Register
EMAC_RXJAB_ERR	Rx Jab Error Register
EMAC_RXLEN_ERR	Rx Length Error Register
EMAC_RXMCASTFRM_G	Rx Multicast Frames (Good) Register
EMAC_RXOCTCNT_G	Rx Octet Count (Good) Register
EMAC_RXOCTCNT_GB	Rx Octet Count (Good/Bad) Register
EMAC_RXOORTYPE	Rx Out Of Range Type Register
EMAC_RXOSIZE_G	Rx Oversize (Good) Register
EMAC_RXPAUSEFRM	Rx Pause Frames Register
EMAC_RXRCV_ERR	Rx Error Frames Received Register
EMAC_RXRUNT_ERR	Rx Runt Error Register
EMAC_RXTCP_ERR_FRM	Rx TCP Error Frames Register
EMAC_RXTCP_ERR_OCT	Rx TCP Error Octets Register
EMAC_RXTCP_GD_FRM	Rx TCP Good Frames Register
EMAC_RXTCP_GD_OCT	Rx TCP Good Octets Register
EMAC_RXUCASTFRM_G	Rx Unicast Frames (Good) Register
EMAC_RXUDP_ERR_FRM	Rx UDP Error Frames Register
EMAC_RXUDP_ERR_OCT	Rx UDP Error Octets Register
EMAC_RXUDP_GD_FRM	Rx UDP Good Frames Register
EMAC_RXUDP_GD_OCT	Rx UDP Good Octets Register
EMAC_RXUSIZE_G	Rx Undersize (Good) Register
EMAC_RXVLANFRM_GB	Rx VLAN Frames (Good/Bad) Register
EMAC_RXWDOG_ERR	Rx Watch Dog Error Register
EMAC_SMI_ADDR	SMI Address Register
EMAC_SMI_DATA	SMI Data Register

Table 24-55: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_TM_ADDEND	Time Stamp Addend Register
EMAC_TM_AUXSTMP_NSEC	Time Stamp Auxiliary TS Nano Seconds Register
EMAC_TM_AUXSTMP_SEC	Time Stamp Auxiliary TM Seconds Register
EMAC_TM_CTL	Time Stamp Control Register
EMAC_TM_HISEC	Time Stamp High Second Register
EMAC_TM_NSEC	Time Stamp Nanoseconds Register
EMAC_TM_NSECUPDT	Time Stamp Nanoseconds Update Register
EMAC_TM_PPS0INTVL	Time Stamp PPS Interval Register
EMAC_TM_PPS0NTGTM	Time Stamp Target Time Nanoseconds Register
EMAC_TM_PPS0TGTM	Time Stamp Target Time Seconds Register
EMAC_TM_PPS0WIDTH	PPS Width Register
EMAC_TM_PPS1INTVL	PPS 1 Interval Register
EMAC_TM_PPS1NTGTM	PPS 1 Target Time Nanoseconds Register
EMAC_TM_PPS1TGTM	PPS 1 Target Time Seconds Register
EMAC_TM_PPS1WIDTH	PPS 1 Width Register
EMAC_TM_PPS2INTVL	PPS 2 Interval Register
EMAC_TM_PPS2NTGTM	PPS 2 Target Time Nanoseconds Register
EMAC_TM_PPS2TGTM	PPS 2 Target Time Seconds Register
EMAC_TM_PPS2WIDTH	PPS 2 Width Register
EMAC_TM_PPS3INTVL	PPS 3 Interval Register
EMAC_TM_PPS3NTGTM	PPS 3 Target Time Nanoseconds Register
EMAC_TM_PPS3TGTM	PPS 3 Target Time Seconds Register
EMAC_TM_PPS3WIDTH	PPS 3 Width Register
EMAC_TM_PPSCTL	PPS Control Register
EMAC_TM_SEC	Time Stamp Low Seconds Register
EMAC_TM_SECUPDT	Time Stamp Seconds Update Register
EMAC_TM_STMPSTAT	Time Stamp Status Register
EMAC_TM_SUBSEC	Time Stamp Sub Second Increment Register
EMAC_TX1024TOMAX_GB	Tx 1024- to Max-Byte Frames (Good/Bad) Register
EMAC_TX128TO255_GB	Tx 128- to 255-Byte Frames (Good/Bad) Register
EMAC_TX256TO511_GB	Tx 256- to 511-Byte Frames (Good/Bad) Register

Table 24-55: ADSP-2159x EMAC Register List (Continued)

Name	Description
EMAC_TX512TO1023_GB	Tx 512- to 1023-Byte Frames (Good/Bad) Register
EMAC_TX64_GB	Tx 64-Byte Frames (Good/Bad) Register
EMAC_TX65TO127_GB	Tx 65- to 127-Byte Frames (Good/Bad) Register
EMAC_TXBCASTFRM_G	Tx Broadcast Frames (Good) Register
EMAC_TXBCASTFRM_GB	Tx Broadcast Frames (Good/Bad) Register
EMAC_TXCARR_ERR	Tx Carrier Error Register
EMAC_TXDEFERRED	Tx Deferred Register
EMAC_TXEXCESSCOL	Tx Excess Collision Register
EMAC_TXEXCESSDEF	Tx Excess Deferral Register
EMAC_TXFRMCNT_G	Tx Frame Count (Good) Register
EMAC_TXFRMCNT_GB	Tx Frame Count (Good/Bad) Register
EMAC_TXLATECOL	Tx Late Collision Register
EMAC_TXMCASTFRM_G	Tx Multicast Frames (Good) Register
EMAC_TXMCASTFRM_GB	Tx Multicast Frames (Good/Bad) Register
EMAC_TXMULTCOL_G	Tx Multiple Collision (Good) Register
EMAC_TXOCTCNT_G	Tx Octet Count (Good) Register
EMAC_TXOCTCNT_GB	Tx OCT Count (Good/Bad) Register
EMAC_TXOVRSIZE_G	Number of Tx Frames (Good) greater than maxsize
EMAC_TXPAUSEFRM	Tx Pause Frame Register
EMAC_TXSNGCOL_G	Tx Single Collision (Good) Register
EMAC_TXUCASTFRM_GB	Tx Unicast Frames (Good/Bad) Register
EMAC_TXUNDR_ERR	Tx Underflow Error Register
EMAC_TXVLANFRM_G	Tx VLAN Frames (Good) Register
EMAC_VLANTAG	VLAN Tag Register
EMAC_VLAN_HSHTBL	VLAN Hash Table Register
EMAC_VLAN_INCL	VLAN Tag Inclusion or Replacement Register
EMAC_WDOG_TIMEOUT	Watchdog Timeout Register

MAC Address 0 High Register

The `EMAC_ADDR0_HI` register holds the address 0 high bits.

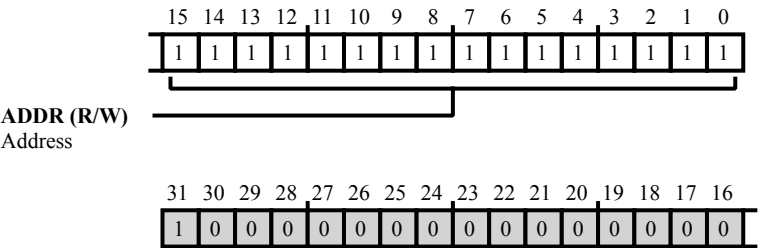


Figure 24-24: EMAC_ADDR0_HI Register Diagram

Table 24-56: EMAC_ADDR0_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	ADDR	Address. The <code>EMAC_ADDR0_HI</code> . <code>ADDR</code> bits contain the upper 16 bits (47:32) of the 6-byte first MAC address. This address is used by the MAC for filtering for received frames and for inserting the MAC address in the Transmit Flow Control (PAUSE) Frames.

MAC Address 0 Low Register

The `EMAC_ADDR0_LO` register holds the address 0 low bits.

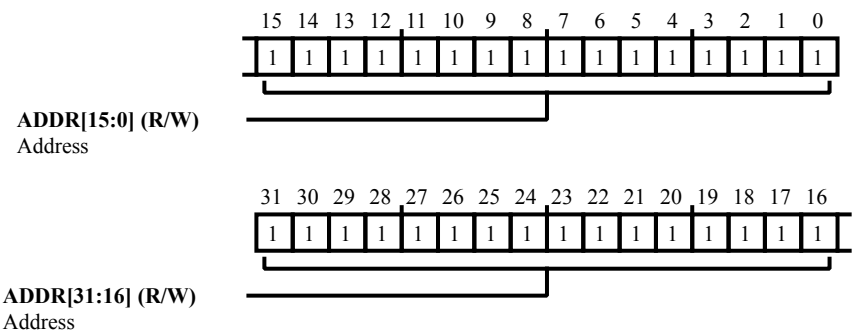


Figure 24-25: `EMAC_ADDR0_LO` Register Diagram

Table 24-57: `EMAC_ADDR0_LO` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ADDR	Address. The <code>EMAC_ADDR0_LO.ADDR</code> bits contain the lower 32 bits of the 6-byte first MAC address. This address is used by the MAC for filtering for received frames and for inserting the MAC address in the Transmit Flow Control (PAUSE) Frames.

MAC Address 1 High Register

The `EMAC_ADDR1_HI` register Contains the higher 16 bits of the second MAC address.

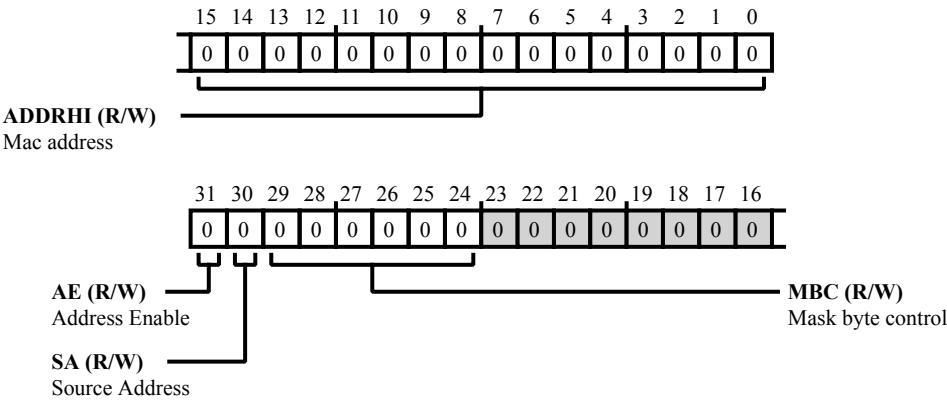


Figure 24-26: EMAC_ADDR1_HI Register Diagram

Table 24-58: EMAC_ADDR1_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	AE	Address Enable. The <code>EMAC_ADDR1_HI . AE</code> bit, When this bit is set, the address filter module uses the second MAC address for perfect filtering.
30 (R/W)	SA	Source Address. The <code>EMAC_ADDR1_HI . SA</code> bit, When this bit is set, the <code>MAC Address1[47:0]</code> is used to compare with the SA fields of the received frame.
29:24 (R/W)	MBC	Mask byte control. The <code>EMAC_ADDR1_HI . MBC</code> bit, are mask control bits for comparison of each of the MAC Address bytes.
15:0 (R/W)	ADDRHI	Mac address. The <code>EMAC_ADDR1_HI . ADDRHI</code> bit, contains the upper 16 bits (47:32) of the second 6-byte MAC address.

MAC Address 1 Low Register

The `EMAC_ADDR1_LO` register Contains the lower 32 bits of the second MAC address.

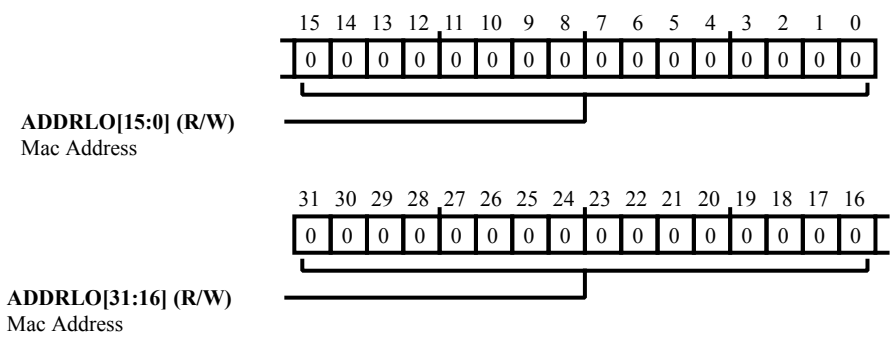


Figure 24-27: EMAC_ADDR1_LO Register Diagram

Table 24-59: EMAC_ADDR1_LO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ADDRLO	Mac Address. The <code>EMAC_ADDR1_LO.ADDRLO</code> bit, contains the lower 32 bits of the first 6-byte MAC address. This is used by the MAC for filtering the received frames and inserting the MAC address in the Transmit Flow Control (Pause) Frames.

Debug Register

The `EMAC_DBG` register contains EMAC debug status information.

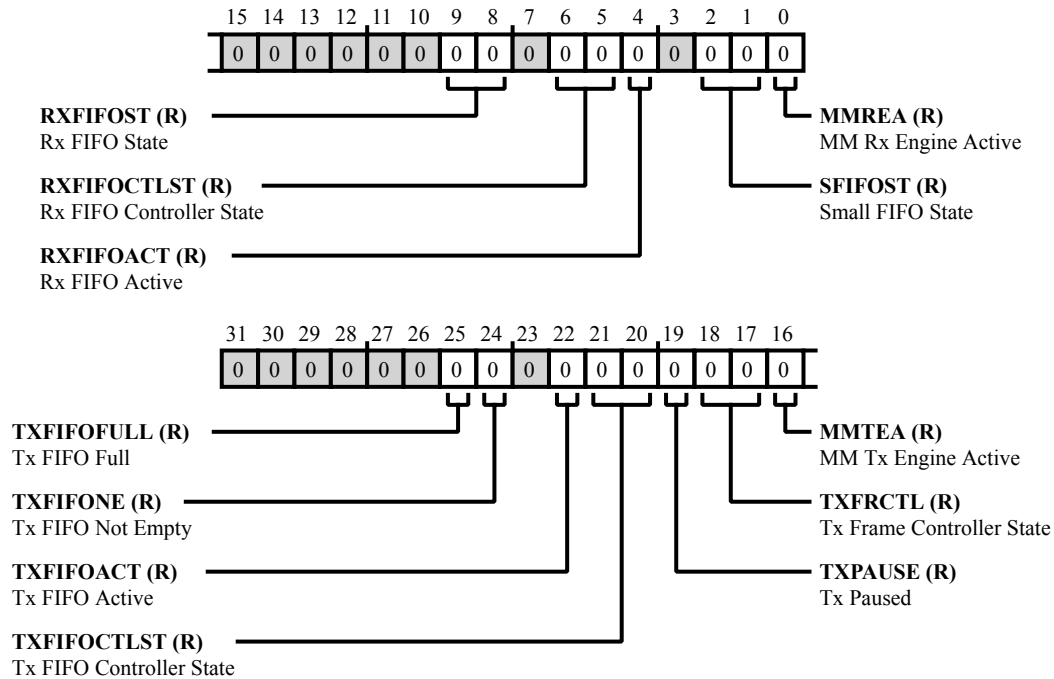


Figure 24-28: EMAC_DBG Register Diagram

Table 24-60: EMAC_DBG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/NW)	TXFIFOFULL	Tx FIFO Full. The <code>EMAC_DBG.TXFIFOFULL</code> bit, when high, indicates that the MFL TxStatus FIFO is full, and the MFL cannot accept any more frames for transmission.
24 (R/NW)	TXFIFONE	Tx FIFO Not Empty. The <code>EMAC_DBG.TXFIFONE</code> bit, when high, indicates that the MFL TxFIFO is not empty and has some data left for transmission.
22 (R/NW)	TXFIFOACT	Tx FIFO Active. The <code>EMAC_DBG.TXFIFOACT</code> bit, when high, indicates that the MFL TxFIFO write controller is active and transferring data to the TxFIFO.
21:20 (R/NW)	TXFIFOCTLST	Tx FIFO Controller State. The <code>EMAC_DBG.TXFIFOCTLST</code> bits indicate the state of the TxFIFO read controller as: 00=IDLE state, 01=READ state (transferring data to MAC transmitter), 10=Waiting for TxStatus from MAC transmitter, and 11=Writing the received TxStatus or flushing the TxFIFO

Table 24-60: EMAC_DBG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/NW)	TXPAUSE	Tx Paused. The EMAC_DBG.TXPAUSE bit, when high, indicates that the MAC transmitter is in PAUSE condition (in full-duplex only) and does not schedule any frame for transmission.
18:17 (R/NW)	TXFRCTL	Tx Frame Controller State. The EMAC_DBG.TXFRCTL bits indicate the state of the MAC transmit frame controller module.
	0	Idle Frame controller is in idle state.
	1	Wait Frame controller is waiting for status of previous frame or IFG/backoff period end.
	2	Pause Frame controller is generating and transmitting a PAUSE control frame (in full duplex mode).
	3	Transmit Frame controller is transferring input frame for transmission.
16 (R/NW)	MMTEA	MM Tx Engine Active. The EMAC_DBG.MMTEA bit, when high, indicates that the MAC core transmit protocol engine is actively transmitting data and is not in IDLE state.
9:8 (R/NW)	RXFIFOST	Rx FIFO State. The EMAC_DBG.RXFIFOST bits give the status of the RxFIFO fill level and indicate the relationship to the flow-control activation threshold.
	0	Rx FIFO Empty
	1	Rx FIFO Below De-activate FCT
	2	Rx FIFO Above De-activate FCT
	3	Rx FIFO Full
6:5 (R/NW)	RXFIFOCTLST	Rx FIFO Controller State. The EMAC_DBG.RXFIFOCTLST bits give the state of the RxFIFO read controller.
	0	Idle Read controller is in idle state.
	1	Read Data Read controller is reading frame data.
	2	Read Status Read controller is reading frame status or time-stamp.
	3	Flush Read controller is flushing the frame data and status.
4 (R/NW)	RXFIFOACT	Rx FIFO Active. The EMAC_DBG.RXFIFOACT bit, when high, indicates that the MFL RxFIFO write controller is active and is transferring a received frame to the FIFO.

Table 24-60: EMAC_DBG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2:1 (R/NW)	SFIFOST	Small FIFO State. The EMAC_DBG.SFIFOST bit, when high, indicates the active state of the small FIFO read and write controllers respectively of the MAC receive frame controller module.
0 (R/NW)	MMREA	MM Rx Engine Active. The EMAC_DBG.MMREA bit, when high, indicates that the MAC core receive protocol engine is actively receiving data and is not in IDLE state.

DMA SCB Bus Mode Register

The `EMAC_DMA0_BMMODE` register selects EMAC DMA system cross bar bus mode features.

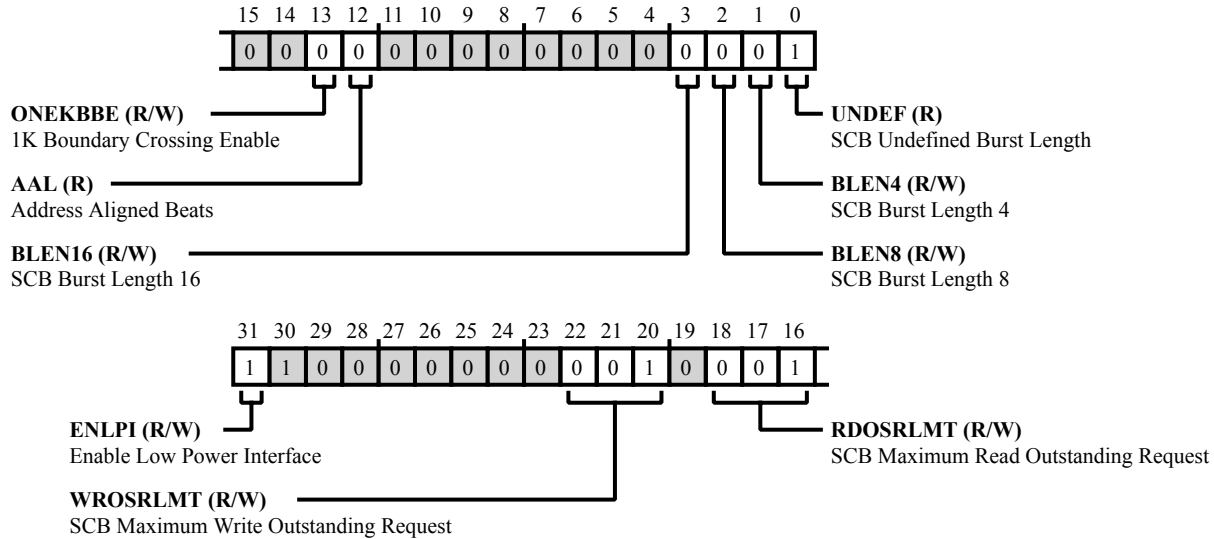


Figure 24-29: EMAC_DMA0_BMMODE Register Diagram

Table 24-61: EMAC_DMA0_BMMODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ENLPI	Enable Low Power Interface. The <code>EMAC_DMA0_BMMODE.ENLPI</code> bit field's when set to 1, it enable LPI mode supported by the GMAC configuration and accepts the LPI request from the system Clock controller.
22:20 (R/W)	WROSRLMT	SCB Maximum Write Outstanding Request. The <code>EMAC_DMA0_BMMODE.WROSRLMT</code> bit field's value limits the maximum outstanding request on the SCB write interface. Maximum outstanding requests = <code>WR_OSR_LMT+1</code> . EMAC-SCB supports up to 4 outstanding write requests.
18:16 (R/W)	RDOSRLMT	SCB Maximum Read Outstanding Request. The <code>EMAC_DMA0_BMMODE.RDOSRLMT</code> bit field's value limits the maximum outstanding request on the SCB read interface. Maximum outstanding requests = <code>RD_OSR_LMT+1</code> . EMAC-SCB supports up to 4 outstanding read requests.
13 (R/W)	ONEKBBE	1K Boundary Crossing Enable. When the <code>EMAC_DMA0_BMMODE.ONEKBBE</code> bit is set, the GMAC extensible bus master performs burst transfers that do not cross 1 KB boundary. When reset, the GMAC extensible bus master performs burst transfers that do not cross 4 KB boundary.

Table 24-61: EMAC_DMA0_BMMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/NW)	AAL	Address Aligned Beats. The EMAC_DMA0_BMMODE.AAL bit (read-only) reflects the state of the EMAC_DMA0_BUSMODE.AAL bit. When this bit is set to 1, EMAC-SCB performs address-aligned burst transfers on both read and write channels.
3 (R/W)	BLN16	SCB Burst Length 16. The EMAC_DMA0_BMMODE.BLEN16 bit, when set (or when EMAC_DMA0_BMMODE.UNDEF is set to 1), directs the EMAC-SCB to select a burst length of 16 on the SCB master interface.
2 (R/W)	BLN8	SCB Burst Length 8. The EMAC_DMA0_BMMODE.BLEN8 bit, when set (or when EMAC_DMA0_BMMODE.UNDEF is set to 1), directs the EMAC-SCB to select a burst length of 8 on the SCB master interface.
1 (R/W)	BLN4	SCB Burst Length 4. The EMAC_DMA0_BMMODE.BLEN4 bit, when set (or when EMAC_DMA0_BMMODE.UNDEF is set to 1), directs the EMAC-SCB to select a burst length of 4 on the SCB master interface.
0 (R/NW)	UNDEF	SCB Undefined Burst Length. The EMAC_DMA0_BMMODE.UNDEF bit (read-only) indicates the complement (invert) value of EMAC_DMA0_BUSMODE.FB bit. When this bit is set to 1, the EMAC-SCB is allowed to perform any burst length equal to or below the maximum allowed burst length as programmed in bits[3:1]. When this bit is set to 0, the EMAC-SCB is allowed to perform only fixed burst lengths as indicated by 16/8/4, or a burst length of 1.

DMA SCB Status Register

The `EMAC_DMA0_BMSTAT` register indicates EMAC DMA system cross bar status.

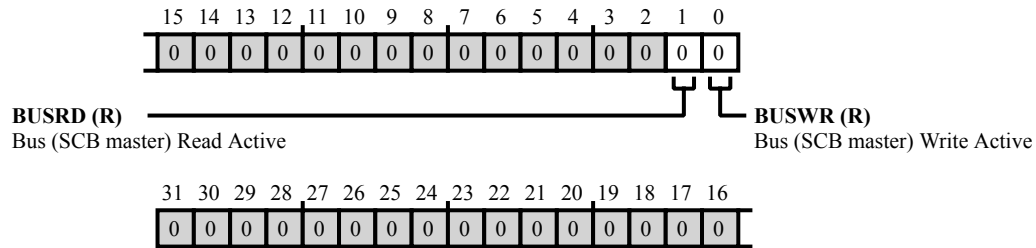


Figure 24-30: EMAC_DMA0_BMSTAT Register Diagram

Table 24-62: EMAC_DMA0_BMSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	BUSRD	Bus (SCB master) Read Active. The <code>EMAC_DMA0_BMSTAT.BUSRD</code> bit, when high, indicates that SCB Master's read channel is active and transferring data.
0 (R/NW)	BUSWR	Bus (SCB master) Write Active. The <code>EMAC_DMA0_BMSTAT.BUSWR</code> bit, when high, indicates that SCB Master's write channel is active and transferring data.

DMA Bus Mode Register

The `EMAC_DMA0_BUSMODE` register selects the DMA bus operating modes for EMAC DMA.

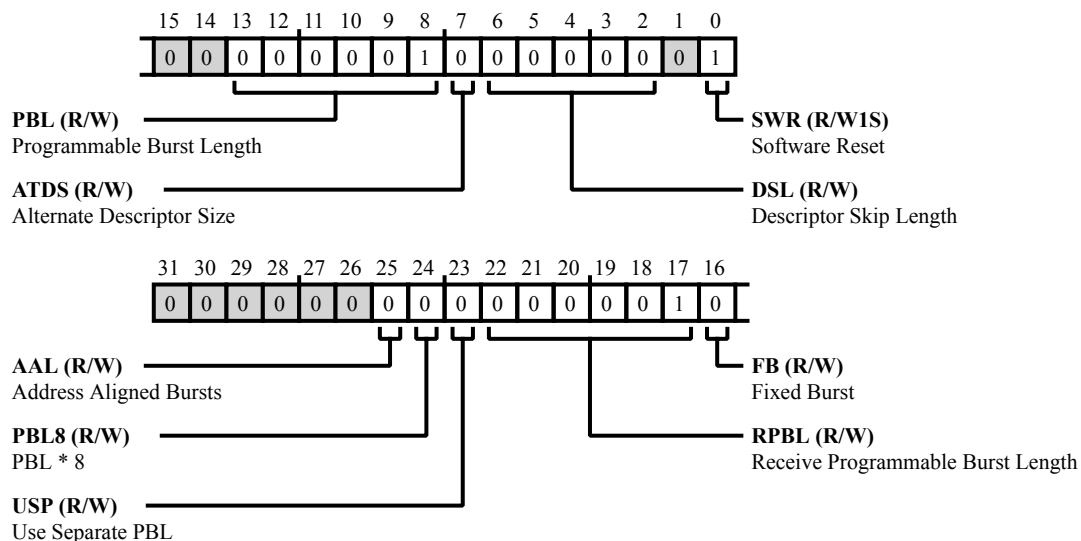


Figure 24-31: EMAC_DMA0_BUSMODE Register Diagram

Table 24-63: EMAC_DMA0_BUSMODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	AAL	Address Aligned Bursts. The <code>EMAC_DMA0_BUSMODE.AAL</code> bit, when set high and the <code>FB</code> bit equals 1, directs the SCB interface to generate all bursts aligned to the start address LS bits. If the <code>FB</code> bit is equal to 0, the first burst (accessing the data buffers start address) is not aligned, but subsequent bursts are aligned to the address.
24 (R/W)	PBL8	PBL * 8. The <code>EMAC_DMA0_BUSMODE.PBL8</code> bit, when set high, multiplies the PBL value programmed (bits [22:17] and bits [13:8]) eight times. Therefore, the DMA transfers the data in 8, 16, and 32 beats depending on the PBL value.
23 (R/W)	USP	Use Separate PBL. The <code>EMAC_DMA0_BUSMODE.USP</code> bit, when set high, configures the Rx DMA to use the value configured in bits [22:17] as PBL while the PBL value in bits [13:8] is applicable to Tx DMA operations only.

Table 24-63: EMAC_DMA0_BUSMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22:17 (R/W)	RPBL	Receive Programmable Burst Length. The <code>EMAC_DMA0_BUSMODE.RPBL</code> bits indicate the maximum number of beats to be transferred in one Rx DMA transaction. This is the maximum value that is used in a single block Read/Write. The Rx DMA always attempts to burst as specified in RPBL every time it starts a Burst transfer on the host bus. RPBL can be programmed with permissible values of 1, 2, 4, 8, 16, and 32. Any other value results in undefined behavior. These bits are valid and applicable only when USP is set high.
16 (R/W)	FB	Fixed Burst. The <code>EMAC_DMA0_BUSMODE.FB</code> bit controls whether the SCB Master interface performs fixed burst transfers or not. See the <code>EMAC_DMA0_BMODE.UNDEF</code> bit description for more information.
13:8 (R/W)	PBL	Programmable Burst Length. The <code>EMAC_DMA0_BUSMODE.PBL</code> bits indicate the maximum number of beats to be transferred in one DMA transaction. This is the maximum value that is used in a single block Read/Write. The DMA always attempts to burst as specified in PBL each time it starts a Burst transfer on the host bus. Any other value results in undefined behavior. When USP is set high, this PBL value is applicable for Tx DMA transactions only. $PBL\text{-max limit} = (FIFO\text{ size} / 2) / 4.$ $PBL\text{-max limit (transmit)} = 256\text{ bytes} / 2 / 4 = 32.$ $PBL\text{-max limit (receive)} = 128\text{ bytes} / 2 / 4 = 16.$ Note that this PBL is at the DMA end. If $PBL = 32$ and if <code>BLEN16</code> is enabled, the DMA automatically splits 32 bursts in to 2×16 bursts. If <code>EMAC_DMA0_BUSMODE.PBL = 8</code>, and if <code>EMAC_DMA0_BMODE.BLEN16</code> is enabled, the max burst is limited to <code>EMAC_DMA0_BMODE.BLEN8</code>. If <code>EMAC_DMA0_BUSMODE.PBL8</code> bit is set, the programmed PBL value is multiplied by 8 times internally. However, the result cannot be more than the above maximum limits specified above.
7 (R/W)	ATDS	Alternate Descriptor Size. The <code>EMAC_DMA0_BUSMODE.ATDS</code> bit, when set, increases the size of the alternate descriptor to 32 bytes (8 DWORDS). This is required when the Advanced Time Stamp feature or Full IPC Offload Engine is enabled in the receiver. When reset, the descriptor size reverts back to 4 DWORDS (16 bytes). The enhanced descriptor is not required if the Advanced Time Stamp and IPC Full Checksum Offload features are not enabled. In such case, you can use the 16 bytes descriptor to save 4 bytes of memory.

Table 24-63: EMAC_DMA0_BUSMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:2 (R/W)	DSL	Descriptor Skip Length. The EMAC_DMA0_BUSMODE.DSL bit specifies the number of 32-bit words to skip between two unchained descriptors. The address skipping starts from the end of current descriptor to the start of next descriptor. When DSL value is equal to zero, then the descriptor table is taken as contiguous by the DMA, in Ring mode.
0 (R/W1S)	SWR	Software Reset. The EMAC_DMA0_BUSMODE.SWR bit, when set, directs the MAC DMA Controller to reset all MAC Subsystem internal registers and logic. It is cleared automatically after the reset operation has completed in all of the core clock domains. Read a 0 value in this bit before re-programming any register of the core. Note: The reset operation is completed only when all the resets in all the active clock domains are de-asserted. Therefore, it is essential that all the PHY inputs clocks (applicable for the selected PHY interface) are present for software reset completion. This field cleared to 1b0 by the core (Self Clear). The application cannot clear this type of field, and a register write of 1b0 to this bit has no effect on this field.

DMA Interrupt Enable Register

The `EMAC_DMA0_IEN` register enables (unmasks) EMAC DMA interrupts.

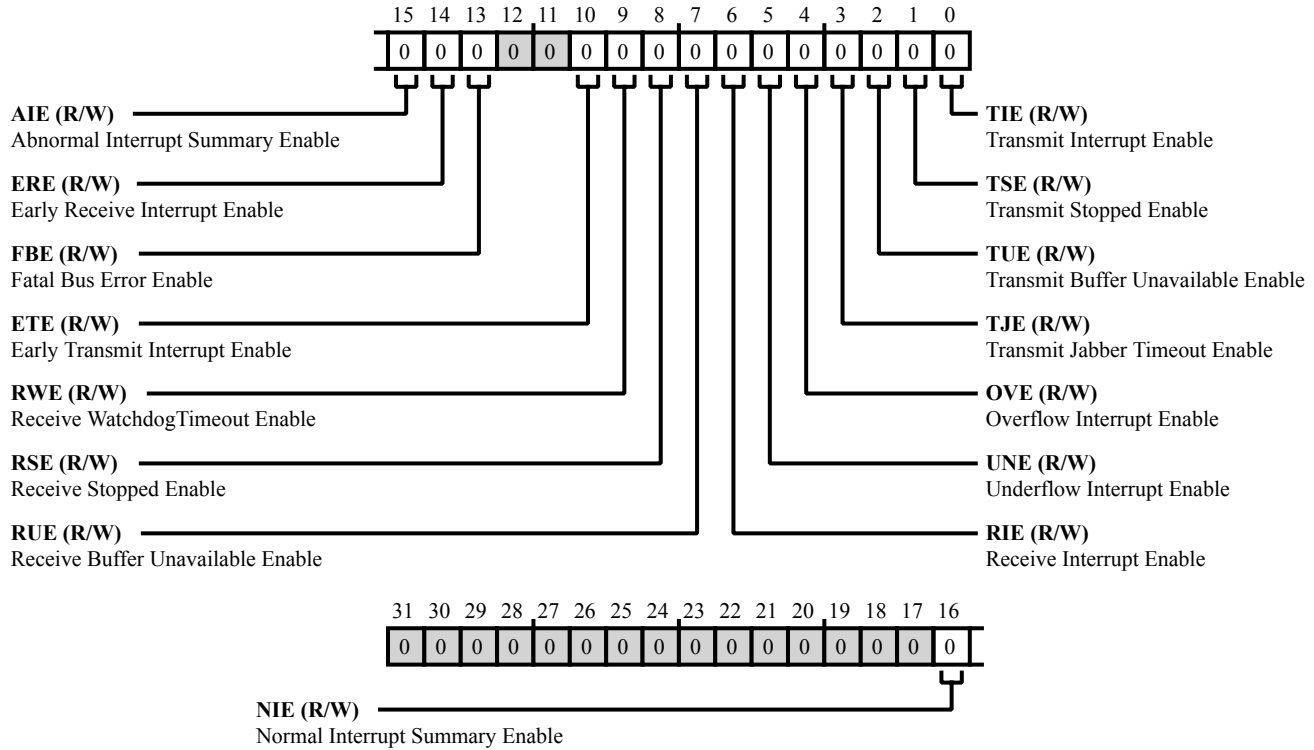


Figure 24-32: EMAC_DMA0_IEN Register Diagram

Table 24-64: EMAC_DMA0_IEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	NIE	Normal Interrupt Summary Enable. The <code>EMAC_DMA0_IEN.NIE</code> bit, when set, enables a normal interrupt. When this bit is reset, a normal interrupt is disabled. This bit enables the following bits: <code>EMAC_DMA0_STAT.TI</code> , <code>EMAC_DMA0_STAT.TU</code> , <code>EMAC_DMA0_STAT.RI</code> , and <code>EMAC_DMA0_STAT.ERI</code> .
15 (R/W)	AIE	Abnormal Interrupt Summary Enable. The <code>EMAC_DMA0_IEN.AIE</code> bit, when set, enables an abnormal interrupt. When this bit is reset, an Abnormal Interrupt is disabled. This bit enables the following bits: <code>EMAC_DMA0_STAT.TPS</code> , <code>EMAC_DMA0_STAT.TJT</code> , <code>EMAC_DMA0_STAT.OVF</code> , <code>EMAC_DMA0_STAT.RU</code> , <code>EMAC_DMA0_STAT.RPS</code> , <code>EMAC_DMA0_STAT.RWT</code> , <code>EMAC_DMA0_STAT.ETI</code> , and <code>EMAC_DMA0_STAT.FBI</code> .

Table 24-64: EMAC_DMA0_IEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	ERE	Early Receive Interrupt Enable. The EMAC_DMA0_IEN.ERE bit, when set (and with EMAC_DMA0_IEN.NIE =1), enables the Early Receive Interrupt. When this bit is reset, Early Receive Interrupt is disabled.
13 (R/W)	FBE	Fatal Bus Error Enable. The EMAC_DMA0_IEN.FBE bit, when set (and with EMAC_DMA0_IEN.AIE =1), enables the Fatal Bus Error Interrupt. When this bit is reset, Fatal Bus Error Enable Interrupt is disabled.
10 (R/W)	ETE	Early Transmit Interrupt Enable. The EMAC_DMA0_IEN.ETE bit, when this bit is set (and with EMAC_DMA0_IEN.AIE =1), enables the Early Transmit Interrupt. When this bit is reset, Early Transmit Interrupt is disabled.
9 (R/W)	RWE	Receive Watchdog Timeout Enable. The EMAC_DMA0_IEN.RWE bit, when set (and with EMAC_DMA0_IEN.AIE =1), enables the Receive Watchdog Timeout Interrupt. When this bit is reset, Receive Watchdog Timeout Interrupt is disabled.
8 (R/W)	RSE	Receive Stopped Enable. The EMAC_DMA0_IEN.RSE bit, when set (and with EMAC_DMA0_IEN.AIE =1), enables the Receive Stopped Interrupt is enabled. When this bit is reset, Receive Stopped Interrupt is disabled.
7 (R/W)	RUE	Receive Buffer Unavailable Enable. The EMAC_DMA0_IEN.RUE bit, when set (and with EMAC_DMA0_IEN.AIE =1), enables the Receive Buffer Unavailable Interrupt. When this bit is reset, the Receive Buffer Unavailable Interrupt is disabled.
6 (R/W)	RIE	Receive Interrupt Enable. The EMAC_DMA0_IEN.RIE bit, when set (and with EMAC_DMA0_IEN.NIE =1), enables the Receive Interrupt. When this bit is reset, Receive Interrupt is disabled.
5 (R/W)	UNE	Underflow Interrupt Enable. The EMAC_DMA0_IEN.UNE bit, when set (and with EMAC_DMA0_IEN.AIE =1), enables the Transmit Underflow Interrupt. When this bit is reset, Underflow Interrupt is disabled.
4 (R/W)	OVE	Overflow Interrupt Enable. The EMAC_DMA0_IEN.OVE bit, when set (and with EMAC_DMA0_IEN.AIE =1), enables the Receive Overflow Interrupt. When this bit is reset, Overflow Interrupt is disabled.

Table 24-64: EMAC_DMA0_IEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	TJE	Transmit Jabber Timeout Enable. The EMAC_DMA0_IEN.TJE bit, when set (and with EMAC_DMA0_IEN.AIE =1), enables the Transmit Jabber Timeout Interrupt. When this bit is reset, Transmit Jabber Timeout Interrupt is disabled.
2 (R/W)	TUE	Transmit Buffer Unavailable Enable. The EMAC_DMA0_IEN.TUE bit, when set (and with EMAC_DMA0_IEN.NIE =1), enables the Transmit Buffer Unavailable Interrupt. When this bit is reset, Transmit Buffer Unavailable Interrupt is disabled.
1 (R/W)	TSE	Transmit Stopped Enable. The EMAC_DMA0_IEN.TSE bit, when set (and with EMAC_DMA0_IEN.AIE =1), enables the Transmission Stopped Interrupt. When this bit is reset, Transmission Stopped Interrupt is disabled.
0 (R/W)	TIE	Transmit Interrupt Enable. The EMAC_DMA0_IEN.TIE bit, when set (and with EMAC_DMA0_IEN.NIE =1), enables the Transmit Interrupt. When this bit is reset, Transmit Interrupt is disabled.

DMA Missed Frame Register

The `EMAC_DMA0_MISS_FRM` register contains counters for EMAC DMA missed frames and buffer overflows.

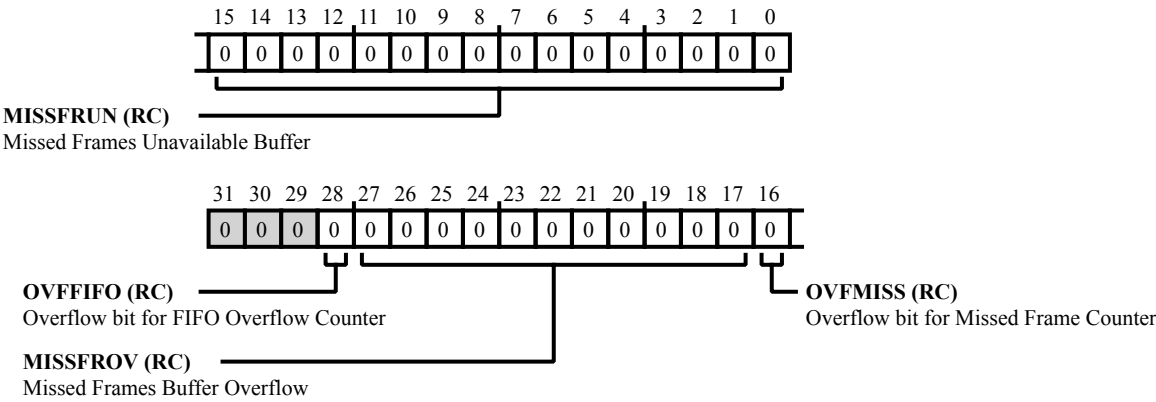


Figure 24-33: EMAC_DMA0_MISS_FRM Register Diagram

Table 24-65: EMAC_DMA0_MISS_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (RC/NW)	OVFFIFO	Overflow bit for FIFO Overflow Counter. The <code>EMAC_DMA0_MISS_FRM.OVFFIFO</code> bit holds the overflow bit for FIFO Overflow Counter.
27:17 (RC/NW)	MISSFROV	Missed Frames Buffer Overflow. The <code>EMAC_DMA0_MISS_FRM.MISSFROV</code> bits indicate the number of frames missed by the application due to buffer overflow.
16 (RC/NW)	OVFMIS	Overflow bit for Missed Frame Counter. The <code>EMAC_DMA0_MISS_FRM.OVFMIS</code> bit holds the overflow bit for the Missed Frame Counter.
15:0 (RC/NW)	MISSFRUN	Missed Frames Unavailable Buffer. The <code>EMAC_DMA0_MISS_FRM.MISSFRUN</code> bits indicate the number of frames missed by the controller because of the Application Receive Buffer being unavailable.

DMA Operation Mode Register

The `EMAC_DMA0_OPMODE` register selects receive and transmit DMA operating modes.

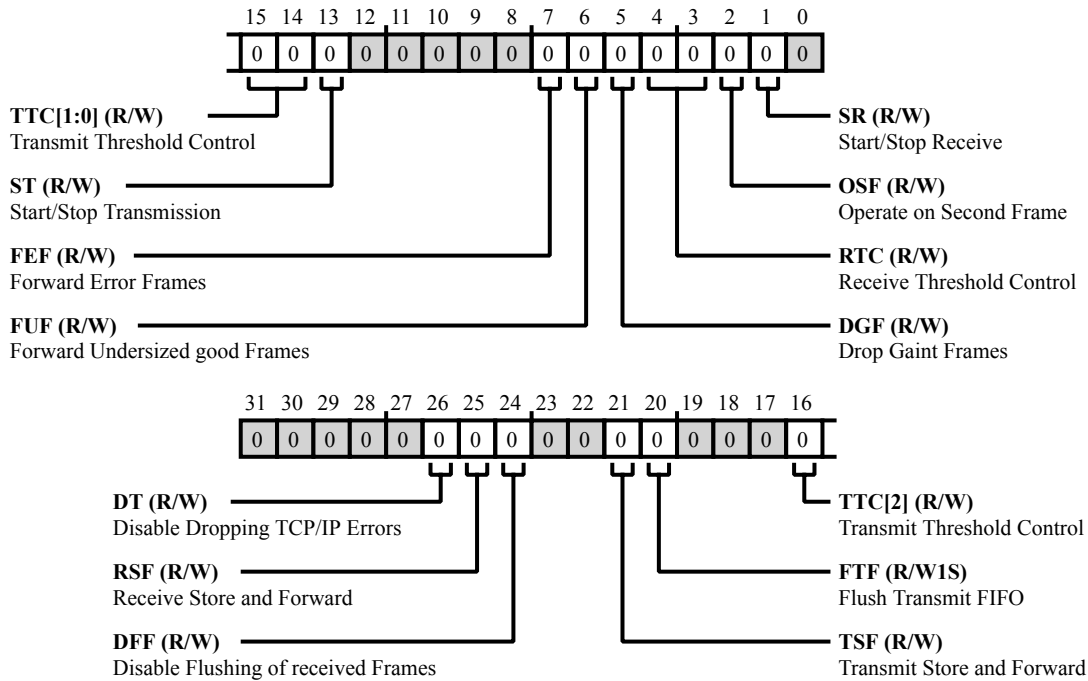


Figure 24-34: `EMAC_DMA0_OPMODE` Register Diagram

Table 24-66: `EMAC_DMA0_OPMODE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	DT	Disable Dropping TCP/IP Errors. The <code>EMAC_DMA0_OPMODE.DT</code> bit, when set, directs the core not to drop frames that only have errors detected by the Receive Checksum Offload engine. Such frames do not have any errors (including FCS error) in the Ethernet frame received by the MAC but have errors in the encapsulated payload only. When this bit is reset, all error frames are dropped if the <code>EMAC_DMA0_OPMODE.FEF</code> bit is reset.
25 (R/W)	RSF	Receive Store and Forward. The <code>EMAC_DMA0_OPMODE.RSF</code> bit, when set, directs the MFL only to read a frame from the Rx FIFO after the complete frame has been written to it, ignoring the <code>EMAC_DMA0_OPMODE.RTC</code> bits. When this bit is reset, the Rx FIFO operates in threshold mode, subject to the threshold specified by the <code>EMAC_DMA0_OPMODE.RTC</code> bits.
24 (R/W)	DFF	Disable Flushing of received Frames. The <code>EMAC_DMA0_OPMODE.DFF</code> bit, when set, directs the Rx DMA not to flush any frames because of the unavailability of receive descriptors/buffers as it does normally when this bit is reset.

Table 24-66: EMAC_DMA0_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration																
21 (R/W)	TSF	Transmit Store and Forward. The EMAC_DMA0_OPMODE . TSF bit, when set, starts transmission when a full frame resides in the MFL Transmit FIFO. When this bit is set, the TTC values specified in Register 6[16:14] are ignored. This bit should be changed only when transmission is stopped.																
20 (R/W1S)	FTF	Flush Transmit FIFO. The EMAC_DMA0_OPMODE . FTF bit, when set, directs the transmit FIFO controller logic to reset to its default values and thus all data in the Tx FIFO is lost/flushed. This bit is cleared internally when the flushing operation is completed fully. The Operation Mode register should not be written to until this bit is cleared. The data which is already accepted by the MAC transmitter is not flushed. It is scheduled for transmission and results in underflow and runt frame transmission. Note: The flush operation completes only after emptying the Tx FIFO of its contents and all the pending Transmit Status of the transmitted frames are accepted by the host. In order to complete this flush operation, the PHY transmit clock is required to be active. This field cleared to 1b0 by the core (Self Clear). The application cannot clear this type of field, and a register write of 1b0 to this bit has no effect on this field.																
16:14 (R/W)	TTC	Transmit Threshold Control. The EMAC_DMA0_OPMODE . TTC bits control the threshold level of the MFL Transmit FIFO. Transmission starts when the frame size within the MFL Transmit FIFO is larger than the threshold. In addition, full frames with a length less than the threshold are also transmitted. These bits are used only when the EMAC_DMA0_OPMODE . TSF bit is reset. The value =011 is not used. <table><tr><td>0</td><td>64</td></tr><tr><td>1</td><td>128</td></tr><tr><td>2</td><td>192</td></tr><tr><td>3</td><td>256</td></tr><tr><td>4</td><td>40</td></tr><tr><td>5</td><td>32</td></tr><tr><td>6</td><td>24</td></tr><tr><td>7</td><td>16</td></tr></table>	0	64	1	128	2	192	3	256	4	40	5	32	6	24	7	16
0	64																	
1	128																	
2	192																	
3	256																	
4	40																	
5	32																	
6	24																	
7	16																	

Table 24-66: EMAC_DMA0_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	ST	Start/Stop Transmission. The EMAC_DMA0_OPMODE.ST bit, when set, places transmission in the Running state, and the DMA checks the Transmit List at the current position for a frame to be transmitted. Descriptor acquisition is attempted either from the current position in the list, which is the Transmit List Base Address set by Transmit Descriptor List Address, or from the position retained when transmission was stopped previously. If the current descriptor is not owned by the DMA, transmission enters the Suspended state, and the EMAC_DMA0_STAT.TU bit is set. The Start Transmission command is effective only when transmission is stopped. If the command is issued before setting the EMAC_DMA0_TXDSC_CUR address register, then the DMA behavior is unpredictable. When this bit is reset, the transmission process is placed in the Stopped state after completing the transmission of the current frame. The Next Descriptor position in the Transmit List is saved, and becomes the current position when transmission is restarted. The stop transmission command is effective only when the transmission of the current frame is complete or the transmission is in the Suspended state.
7 (R/W)	FEF	Forward Error Frames. The EMAC_DMA0_OPMODE.FEF bit, when reset, directs the Rx FIFO to drop frames with error status (CRC error, collision error, giant frame, watchdog timeout, overflow). However, if the frames start byte (write) pointer is already transferred to the read controller side (in Threshold mode), then the frames are not dropped. When EMAC_DMA0_OPMODE.FEF bit is set, all frames except runt error frames are forwarded to the DMA. But when Rx FIFO overflows when a partial frame is written, then such frames are dropped even when EMAC_DMA0_OPMODE.FEF is set.
6 (R/W)	FUF	Forward Undersized good Frames. The EMAC_DMA0_OPMODE.FUF bit, when set, directs the Rx FIFO to forward Undersized frames (frames with no Error and length less than 64 bytes) including pad-bytes and CRC). When reset, the Rx FIFO drops all frames of less than 64 bytes, unless it is already transferred because of lower value of Receive Threshold (for example, EMAC_DMA0_OPMODE.RTC = 01).
5 (R/W)	DGF	Drop Gaint Frames. The EMAC_DMA0_OPMODE.DGF bit, when set, the MAC drops the received giant frames in the Rx FIFO, that is, frames that are larger than the computed giant frame limit. When reset, the MAC does not drop the giant frames in the Rx FIFO.

Table 24-66: EMAC_DMA0_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:3 (R/W)	RTC	Receive Threshold Control. The <code>EMAC_DMA0_OPMODE.RTC</code> bits control the threshold level of the MFL Receive FIFO. Transfer (request) to DMA starts when the frame size within the MFL Receive FIFO is larger than the threshold. In addition, full frames with a length less than the threshold are transferred automatically. These bits are valid only when the <code>EMAC_DMA0_OPMODE.RSF</code> bit is zero, and are ignored when the <code>EMAC_DMA0_OPMODE.RSF</code> bit is set to 1. The value =11 is not used.
		0 64
		1 32
		2 96
		3 128
2 (R/W)	OSF	Operate on Second Frame. The <code>EMAC_DMA0_OPMODE.OSF</code> bit, when set, instructs the DMA to process a second frame of Transmit data even before status for first frame is obtained.
1 (R/W)	SR	Start/Stop Receive. The <code>EMAC_DMA0_OPMODE.SR</code> bit, when set, places the Receive process in the Running state. The DMA attempts to acquire the descriptor from the Receive list and processes incoming frames. Descriptor acquisition is attempted from the current position in the list, which is the address set by DMA Receive Descriptor List Address or the position retained when the Receive process was previously stopped. If no descriptor is owned by the DMA, reception is suspended, and the <code>EMAC_DMA0_STAT.RU</code> bit is set. The Start Receive command is effective only when reception has stopped. If the command was issued before setting <code>EMAC_DMA0_RXDSC_CUR</code> address register, DMA behavior is unpredictable. When this bit is cleared, Rx DMA operation is stopped after the transfer of the current frame. The next descriptor position in the Receive list is saved and becomes the current position after the Receive process is restarted. The Stop Receive command is effective only when the Receive process is in either the Running (waiting for receive packet) or in the Suspended state.

DMA Rx Buffer Current Register

The `EMAC_DMA0_RXBUF_CUR` register holds the pointer to the current receive DMA buffer.

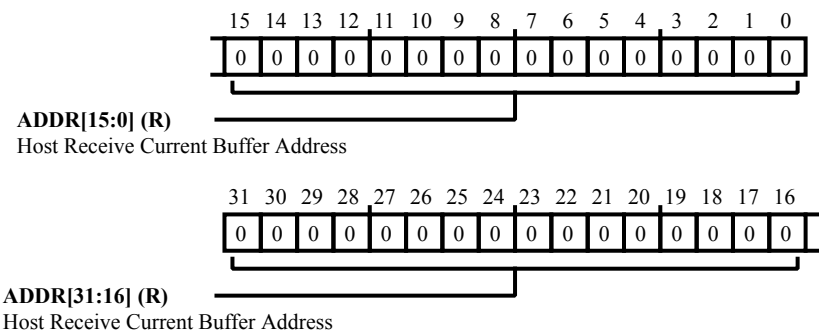


Figure 24-35: EMAC_DMA0_RXBUF_CUR Register Diagram

Table 24-67: EMAC_DMA0_RXBUF_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Receive Current Buffer Address. The <code>EMAC_DMA0_RXBUF_CUR.ADDR</code> bit field points to the current Receive Buffer address being read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Rx Descriptor List Address Register

The `EMAC_DMA0_RXDSC_ADDR` register holds the address for the DMA receive descriptor list. Writing to this Register is permitted only when reception is stopped. When stopped, this must be written to before the receive Start command is given. The processor can write to `EMAC_DMA0_RXDSC_ADDR` only when Rx DMA has stopped (`EMAC_DMA0_OPMODE.SR` bit =0). When stopped, it can be written with a new descriptor list address. When the processor sets the `EMAC_DMA0_OPMODE.SR` bit to 1, the DMA takes the newly programmed descriptor base address. If this register is not changed when the `EMAC_DMA0_OPMODE.SR` bit is cleared to 0, the DMA takes the descriptor address where it was stopped earlier.

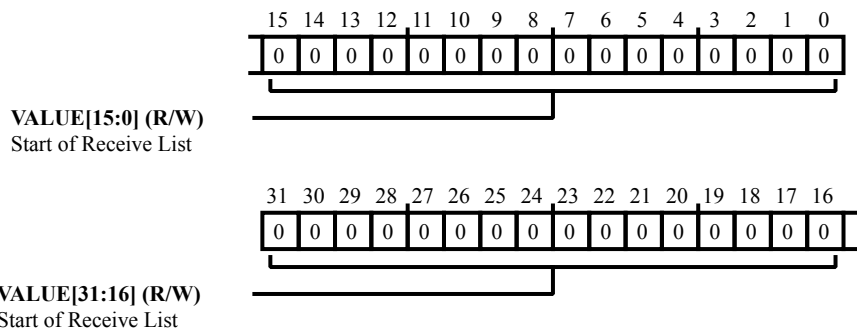


Figure 24-36: `EMAC_DMA0_RXDSC_ADDR` Register Diagram

Table 24-68: `EMAC_DMA0_RXDSC_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Start of Receive List. The <code>EMAC_DMA0_RXDSC_ADDR.VALUE</code> bit field contains the base address of the First Descriptor in the Receive Descriptor list. The LSB bits [1:0] for the 32bit bus width are ignored and are taken as all-zero by the DMA internally. Therefore, these LSB bits are Read-Only (RO).

DMA Rx Descriptor Current Register

The `EMAC_DMA0_RXDSC_CUR` register contains the current DMA receive descriptor.

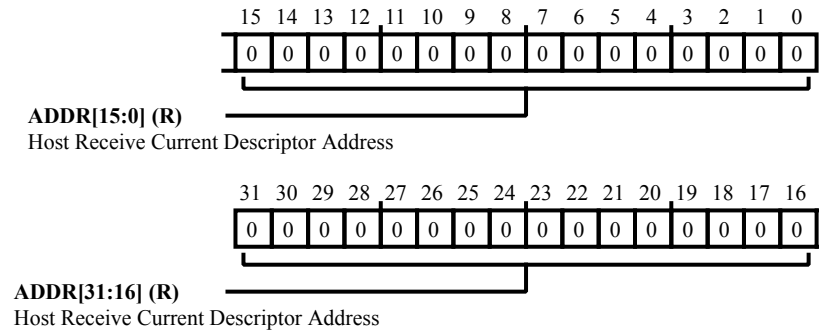


Figure 24-37: EMAC_DMA0_RXDSC_CUR Register Diagram

Table 24-69: EMAC_DMA0_RXDSC_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Receive Current Descriptor Address. The <code>EMAC_DMA0_RXDSC_CUR.ADDR</code> bit field points to the start address of the current Receive Descriptor read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Rx Interrupt Watch Dog Register

The `EMAC_DMA0_RXIWDOG` register contains the timeout value for the EMAC DMA receive interrupt watch dog timer.

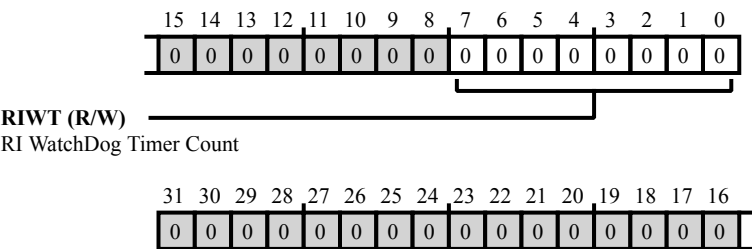


Figure 24-38: EMAC_DMA0_RXIWDOG Register Diagram

Table 24-70: EMAC_DMA0_RXIWDOG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	RIWT	RI WatchDog Timer Count. The <code>EMAC_DMA0_RXIWDOG.RIWT</code> bit field indicates the number of system clock cycles multiplied by 256 for which the watchdog timer is set. The watchdog timer gets triggered with the programmed value after the Rx DMA completes the transfer of a frame for which the RI status bit is not set because of the setting in the corresponding descriptor <code>RDES1[31]</code> . When the watch-dog timer runs out, the RI bit is set and the timer is stopped. The watchdog timer is reset when <code>EMAC_DMA0_STAT.RI</code> bit is set high because of automatic setting of <code>EMAC_DMA0_STAT.RI</code> as per <code>RDES1[31]</code> of any received frame.

DMA Rx Poll Demand register

The `EMAC_DMA0_RXPOLL` register directs the EMAC to poll the receive descriptor list.

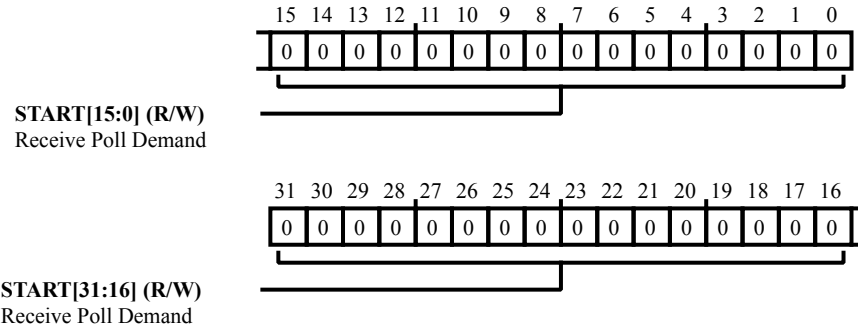


Figure 24-39: `EMAC_DMA0_RXPOLL` Register Diagram

Table 24-71: `EMAC_DMA0_RXPOLL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	START	Receive Poll Demand. The <code>EMAC_DMA0_RXPOLL.START</code> bits, when written with any value, cause the DMA to read the current descriptor pointed to by the <code>EMAC_DMA0_RXDSC_CUR</code> register. If that descriptor is not available (owned by application), reception returns to the Suspended state, and the <code>EMAC_DMA0_STAT.RU</code> bit is asserted. If the descriptor is available, the Receive DMA returns to the active state.

DMA Status Register

The `EMAC_DMA0_STAT` register indicates EMAC DMA status.

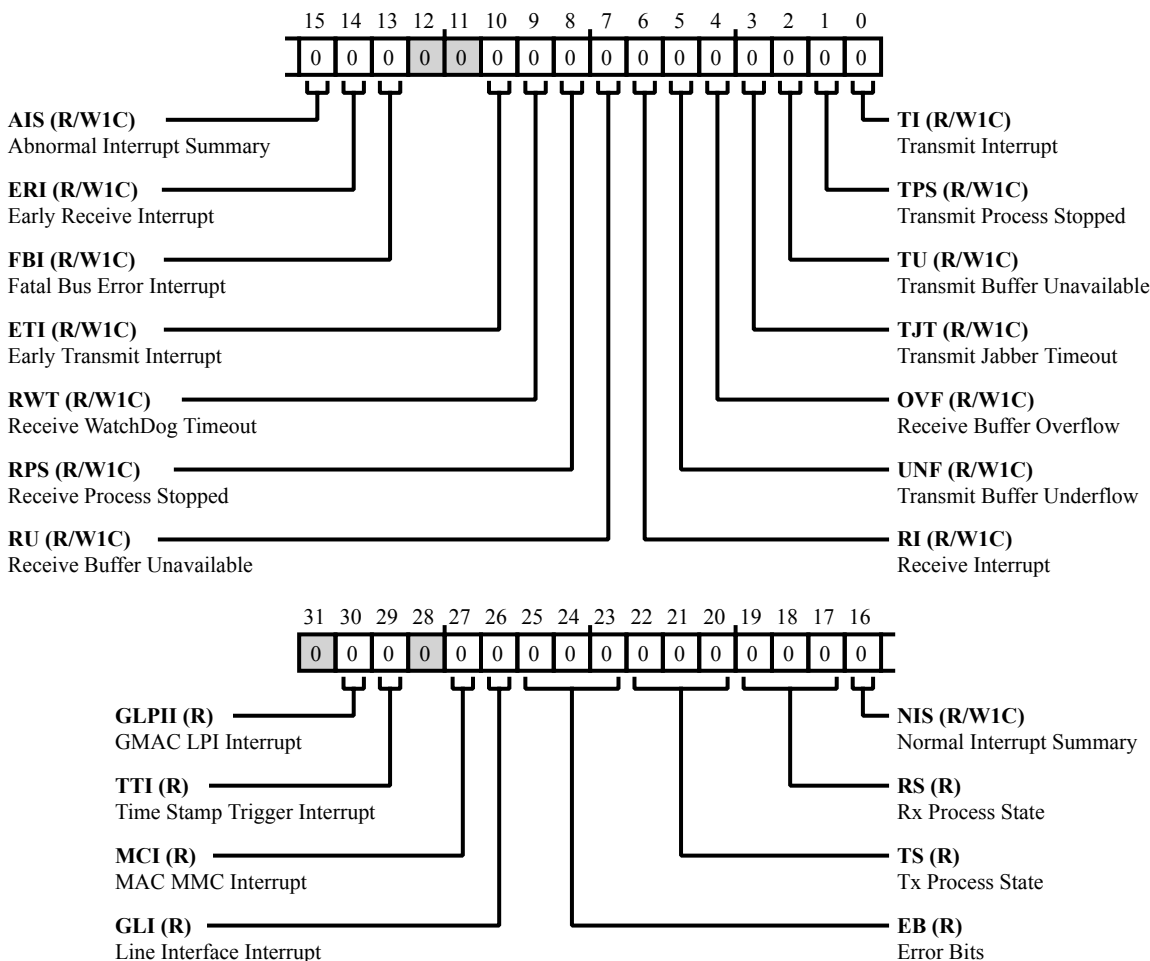


Figure 24-40: EMAC_DMA0_STAT Register Diagram

Table 24-72: EMAC_DMA0_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/NW)	GLPII	GMAC LPI Interrupt. The <code>EMAC_DMA0_STAT.GLPII</code> bit indicates an interrupt event in the LPI logic of the MAC. To reset this bit to 1'b0, the software must read the corresponding registers in the <code>DWC_gmac</code> to get the exact cause of the interrupt and clear its source.

Table 24-72: EMAC_DMA0_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/NW)	TTI	Time Stamp Trigger Interrupt. The EMAC_DMA0_STAT.TTI bit indicates an interrupt event in the MAC core's Time Stamp Generator block. The software must read the corresponding registers in the MAC core to get the exact cause of interrupt and clear its source to reset this bit to =0. When this bit is high, the interrupt signal from the MAC is high.
27 (R/NW)	MCI	MAC MMC Interrupt. The EMAC_DMA0_STAT.MCI bit reflects an interrupt event in the MMC module of the MAC core. The software must read the corresponding registers in the MAC core to get the exact cause of interrupt and clear the source of interrupt to make this bit as =0. The interrupt signal from the MAC is high when this bit is high.
26 (R/NW)	GLI	Line Interface Interrupt. The EMAC_DMA0_STAT.GLI bit When set, this bit reflects any of the following interrupt events in the DWC_gmac interfaces
25:23 (R/NW)	EB	Error Bits. The EMAC_DMA0_STAT.EB bits indicate the type of error that caused a Bus Error (for example, error response on the SCB interface). These bits are valid only when the EMAC_DMA0_STAT.FBI bit is set. This field does not generate an interrupt.
		0 Error during data buffer access, write transfer, Rx DMA
		1 Error during data buffer access, write transfer, Tx DMA
		2 Error during data buffer access, read transfer, Rx DMA
		3 Error during data buffer access, read transfer, Tx DMA
		4 Error during descriptor access, write transfer, Rx DMA
		5 Error during descriptor access, write transfer, Tx DMA
		6 Error during descriptor access, read transfer, Rx DMA
		7 Error during descriptor access, read transfer, Tx DMA
22:20 (R/NW)	TS	Tx Process State. The EMAC_DMA0_STAT.TS bits indicate the transmit DMA state. This field does not generate an interrupt.
		0 Stopped; Reset or Stop Tx Command Issued
		1 Running; Fetching Tx Transfer Descriptor
		2 Running; Waiting for Status
		3 Reading Data from Host Memory Buffer and Queuing It to Tx Buffer
		4 TIME_STAMP Write State

Table 24-72: EMAC_DMA0_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		5	Reserved
		6	Suspended; Tx Descriptor Unavailable or Tx Buffer Underflow
		7	Closing Tx Descriptor
19:17 (R/NW)	RS	Rx Process State. The EMAC_DMA0_STAT.RS bits indicate the receive DMA state. This field does not generate an interrupt.	
		0	Stopped: Reset or Stop Rx Command Issued.
		1	Running: Fetching Rx Transfer Descriptor.
		2	Reserved
		3	Running: Waiting for Rx Packet
		4	Suspended: Rx Descriptor Unavailable
		5	Running: Closing Rx Descriptor
		6	TIME_STAMP Write State
		7	Running: Transferring Rx Packet Data from Rx Buffer to Host Memory
16 (R/W1C)	NIS	Normal Interrupt Summary. The value of the EMAC_DMA0_STAT.NIS bit field is the logical OR of the following when the corresponding interrupt bits are enabled in DMA Interrupt Enable Register: EMAC_DMA0_STAT.TI, EMAC_DMA0_STAT.TU, EMAC_DMA0_STAT.RI, and EMAC_DMA0_STAT.ERI. Only unmasked bits affect the Normal Interrupt Summary bit. This is a sticky bit and must be cleared (by writing a 1 to this bit) each time a corresponding bit that causes EMAC_DMA0_STAT.NIS to be set is cleared.	
15 (R/W1C)	AIS	Abnormal Interrupt Summary. The value of the EMAC_DMA0_STAT.AIS bit field is the logical OR of the following when the corresponding interrupt bits are enabled in DMA Interrupt Enable Register: EMAC_DMA0_IEN.TSE, EMAC_DMA0_IEN.TJE, EMAC_DMA0_IEN.OVE, EMAC_DMA0_IEN.UNE, EMAC_DMA0_IEN.RUE, EMAC_DMA0_IEN.RSE, EMAC_DMA0_IEN.RWE, EMAC_DMA0_IEN.ETE, and EMAC_DMA0_IEN.FBE. Only unmasked bits affect the Abnormal Interrupt Summary bit. This is a sticky bit and must be cleared each time a corresponding bit that causes EMAC_DMA0_STAT.AIS to be set is cleared.	
14 (R/W1C)	ERI	Early Receive Interrupt. The EMAC_DMA0_STAT.ERI bit indicates that the DMA had filled the first data buffer of the packet. The EMAC_DMA0_STAT.RI bit automatically clears this bit.	

Table 24-72: EMAC_DMA0_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1C)	FBI	Fatal Bus Error Interrupt. The EMAC_DMA0_STAT.FBI bit indicates that a bus error occurred, as detailed in the EMAC_DMA0_STAT.EB field. When this bit is set, the corresponding DMA engine disables all its bus accesses.
10 (R/W1C)	ETI	Early Transmit Interrupt. The EMAC_DMA0_STAT.ETI bit indicates that the frame to be transmitted was fully transferred to the MFL Transmit FIFO.
9 (R/W1C)	RWT	Receive WatchDog Timeout. The EMAC_DMA0_STAT.RWT bit is asserted when a frame with a length greater than 2,048 bytes is received (10, 240 when Jumbo Frame mode is enabled).
8 (R/W1C)	RPS	Receive Process Stopped. The EMAC_DMA0_STAT.RPS bit is asserted when the Receive Process enters the Stopped state.
7 (R/W1C)	RU	Receive Buffer Unavailable. The EMAC_DMA0_STAT.RU bit indicates that the Next Descriptor in the Receive List is owned by the application and cannot be acquired by the DMA. Receive Process is suspended. To resume processing Receive descriptors, the application should change the ownership of the descriptor and issue a Receive Poll Demand command. If no Receive Poll Demand is issued, Receive Process resumes when the next recognized incoming frame is received. This bit is set only when the previous Receive Descriptor was owned by the DMA.
6 (R/W1C)	RI	Receive Interrupt. The EMAC_DMA0_STAT.RI bit indicates the completion of frame reception. Specific frame status information has been posted in the descriptor. Reception remains in the Running state.
5 (R/W1C)	UNF	Transmit Buffer Underflow. The EMAC_DMA0_STAT.UNF bit indicates that the Transmit Buffer had an Underflow during frame transmission. Transmission is suspended and an Underflow Error TDES0[1] is set.
4 (R/W1C)	OVF	Receive Buffer Overflow. The EMAC_DMA0_STAT.OVF bit indicates that the Receive Buffer had an Overflow during frame reception. If the partial frame is transferred to application, the overflow status is set in RDES0[11].
3 (R/W1C)	TJT	Transmit Jabber Timeout. The EMAC_DMA0_STAT.TJT bit indicates that the Transmit Jabber Timer expired, meaning that the transmitter had been excessively active. The transmission process is aborted and placed in the Stopped state. This causes the Transmit Jabber Timeout TDES0[14] flag to assert.

Table 24-72: EMAC_DMA0_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W1C)	TU	Transmit Buffer Unavailable. The EMAC_DMA0_STAT.TU bit indicates that the Next Descriptor in the Transmit List is owned by the application and cannot be acquired by the DMA. Transmission is suspended. The value in the EMAC_DMA0_STAT.TS bits explain the Transmit Process state transitions. To resume processing transmit descriptors, the application should change the ownership of the bit of the descriptor and then issue a Transmit Poll Demand command.
1 (R/W1C)	TPS	Transmit Process Stopped. The EMAC_DMA0_STAT.TPS bit is set when the transmission is stopped.
0 (R/W1C)	TI	Transmit Interrupt. The EMAC_DMA0_STAT.TI bit indicates that frame transmission is finished and TDES1[31] is set in the First Descriptor.

DMA Tx Buffer Current Register

The `EMAC_DMA0_TXBUF_CUR` register holds the pointer to the current transmit DMA buffer.

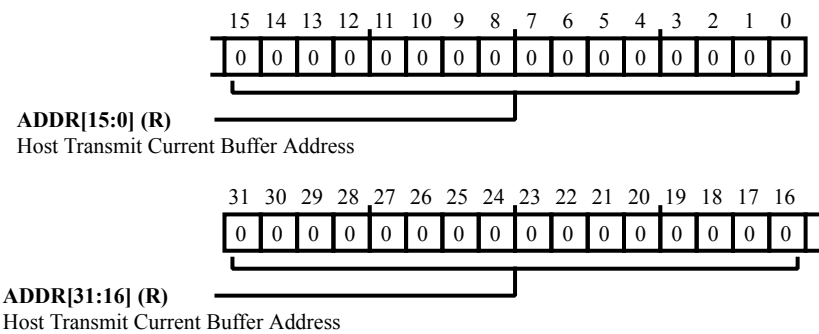


Figure 24-41: EMAC_DMA0_TXBUF_CUR Register Diagram

Table 24-73: EMAC_DMA0_TXBUF_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Transmit Current Buffer Address. The <code>EMAC_DMA0_TXBUF_CUR.ADDR</code> bit field points to the current Transmit Buffer Address being read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Tx Descriptor List Address Register

The `EMAC_DMA0_TXDSC_ADDR` register holds the address for the DMA transmit descriptor list. The processor can write to this Register only when Tx DMA has stopped (`EMAC_DMA0_OPMODE.ST` bit =0). When stopped, this can be written with a new descriptor list address. When the processor sets the `EMAC_DMA0_OPMODE.ST` bit to 1, the DMA takes the newly programmed descriptor base address. If this register is not changed when the `EMAC_DMA0_OPMODE.ST` bit is cleared to 0, then the DMA takes the descriptor address where it was stopped earlier.

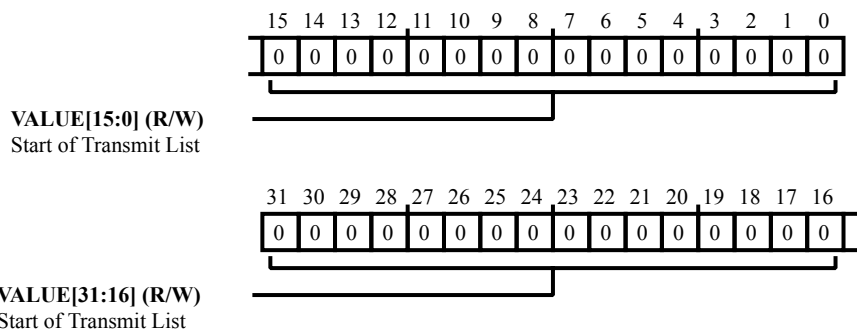


Figure 24-42: EMAC_DMA0_TXDSC_ADDR Register Diagram

Table 24-74: EMAC_DMA0_TXDSC_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Start of Transmit List. The <code>EMAC_DMA0_TXDSC_ADDR.VALUE</code> bit field contains the base address of the First Descriptor in the Transmit Descriptor list. The LSB bits [1:0] for 32bit bus width are ignored and are taken as all-zero by the DMA internally. Therefore, these LSB bits are Read-Only (RO).

DMA Tx Descriptor Current Register

The `EMAC_DMA0_TXDSC_CUR` register contains the current DMA transmit descriptor.

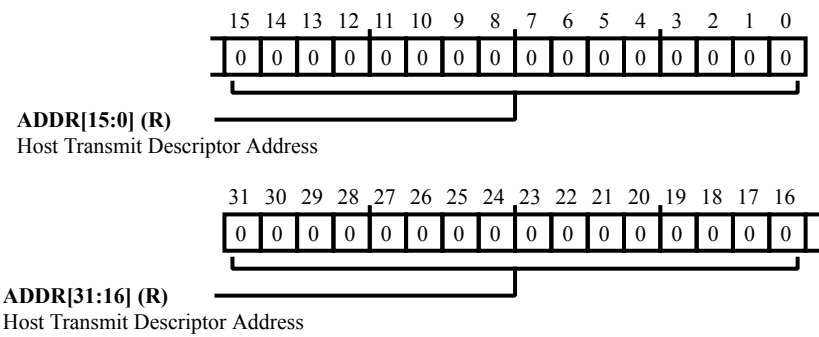


Figure 24-43: `EMAC_DMA0_TXDSC_CUR` Register Diagram

Table 24-75: `EMAC_DMA0_TXDSC_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Transmit Descriptor Address. The <code>EMAC_DMA0_TXDSC_CUR.ADDR</code> bit field points to the start address of the current Transmit Descriptor read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Tx Poll Demand Register

The `EMAC_DMA0_TXPOLL` register directs the EMAC to poll the transmit descriptor list.

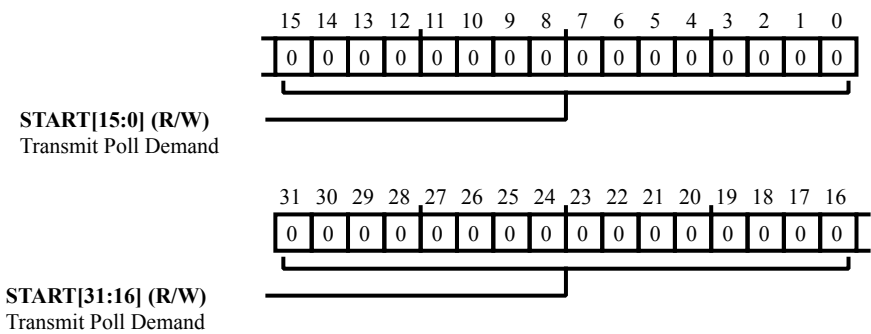


Figure 24-44: `EMAC_DMA0_TXPOLL` Register Diagram

Table 24-76: `EMAC_DMA0_TXPOLL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	START	Transmit Poll Demand. The <code>EMAC_DMA0_TXPOLL.START</code> bits, when written with any value, cause the DMA to read the current descriptor pointed to by <code>EMAC_DMA0_TXDSC_CUR</code> register. If that descriptor is not available (owned by application), transmission returns to the Suspend state, and the <code>EMAC_DMA0_STAT.TU</code> bit is asserted. If the descriptor is available, transmission resumes.

DMA Bus Mode Register

The `EMAC_DMA1_BUSMODE` register selects the DMA bus operating modes for EMAC DMA.

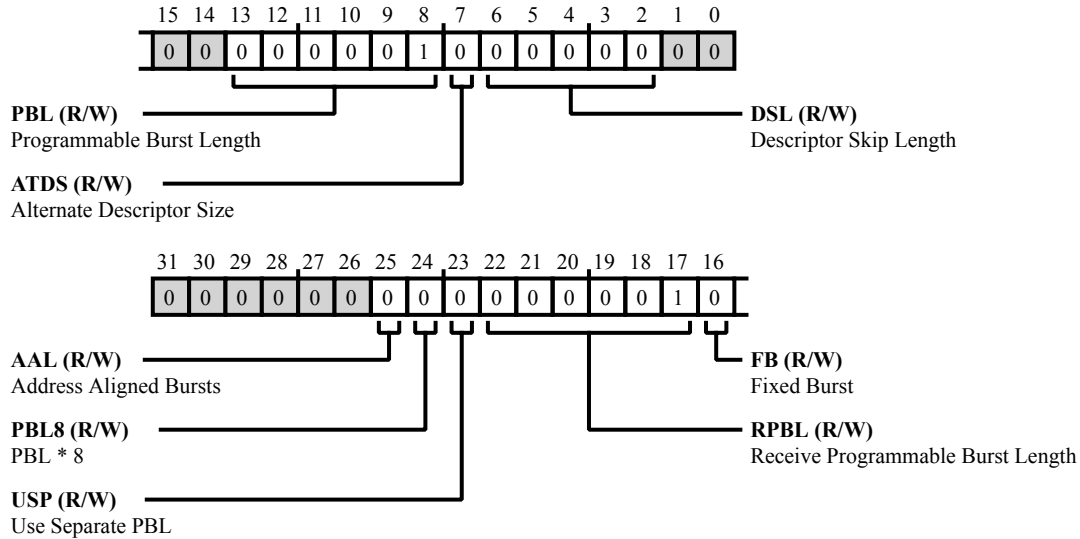


Figure 24-45: EMAC_DMA1_BUSMODE Register Diagram

Table 24-77: EMAC_DMA1_BUSMODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	AAL	Address Aligned Bursts. The <code>EMAC_DMA1_BUSMODE.AAL</code> bit, when set high and the <code>FB</code> bit equals 1, directs the SCB interface to generate all bursts aligned to the start address LS bits. If the <code>FB</code> bit is equal to 0, the first burst (accessing the data buffers start address) is not aligned, but subsequent bursts are aligned to the address.
24 (R/W)	PBL8	PBL * 8. The <code>EMAC_DMA1_BUSMODE.PBL8</code> bit, when set high, multiplies the PBL value programmed (bits [22:17] and bits [13:8]) eight times. Therefore, the DMA transfers the data in 8, 16, and 32 beats depending on the PBL value.
23 (R/W)	USP	Use Separate PBL. The <code>EMAC_DMA1_BUSMODE.USP</code> bit, when set high, configures the Rx DMA to use the value configured in bits [22:17] as PBL while the PBL value in bits [13:8] is applicable to Tx DMA operations only.

Table 24-77: EMAC_DMA1_BUSMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22:17 (R/W)	RPBL	Receive Programmable Burst Length. The <code>EMAC_DMA1_BUSMODE.RPBL</code> bits indicate the maximum number of beats to be transferred in one Rx DMA transaction. This is the maximum value that is used in a single block Read/Write. The Rx DMA always attempts to burst as specified in RPBL every time it starts a Burst transfer on the host bus. RPBL can be programmed with permissible values of 1, 2, 4, 8, 16, and 32. Any other value results in undefined behavior. These bits are valid and applicable only when USP is set high.
16 (R/W)	FB	Fixed Burst. The <code>EMAC_DMA1_BUSMODE.FB</code> bit controls whether the SCB Master interface performs fixed burst transfers or not. See the <code>EMAC_DMA0_BMODE.UNDEF</code> bit description for more information.
13:8 (R/W)	PBL	Programmable Burst Length. The <code>EMAC_DMA1_BUSMODE.PBL</code> bits indicate the maximum number of beats to be transferred in one DMA transaction. This is the maximum value that is used in a single block Read/Write. The DMA always attempts to burst as specified in PBL each time it starts a Burst transfer on the host bus. Any other value results in undefined behavior. When USP is set high, this PBL value is applicable for Tx DMA transactions only. $PBL\text{-max limit} = (FIFO\ size / 2) / 4.$ $PBL\text{-max limit (transmit)} = 256\ bytes / 2 / 4 = 32.$ $PBL\text{-max limit (receive)} = 128\ bytes / 2 / 4 = 16.$ Note that this PBL is at the DMA end. If $PBL = 32$ and if <code>BLEN16</code> is enabled, the DMA automatically splits 32 bursts in to 2×16 bursts. If <code>EMAC_DMA1_BUSMODE.PBL = 8</code>, and if <code>EMAC_DMA0_BMODE.BLEN16</code> is enabled, the max burst is limited to <code>EMAC_DMA0_BMODE.BLEN8</code>. If <code>EMAC_DMA1_BUSMODE.PBL8</code> bit is set, the programmed PBL value is multiplied by 8 times internally. However, the result cannot be more than the above maximum limits specified above.
7 (R/W)	ATDS	Alternate Descriptor Size. The <code>EMAC_DMA1_BUSMODE.ATDS</code> bit, when set, increases the size of the alternate descriptor to 32 bytes (8 DWORDS). This is required when the Advanced Time Stamp feature or Full IPC Offload Engine is enabled in the receiver. When reset, the descriptor size reverts back to 4 DWORDS (16 bytes). The enhanced descriptor is not required if the Advanced Time Stamp and IPC Full Checksum Offload features are not enabled. In such case, you can use the 16 bytes descriptor to save 4 bytes of memory.

Table 24-77: EMAC_DMA1_BUSMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:2 (R/W)	DSL	Descriptor Skip Length. The <code>EMAC_DMA1_BUSMODE.DSL</code> bit specifies the number of 32-bit words to skip between two unchained descriptors. The address skipping starts from the end of current descriptor to the start of next descriptor. When DSL value is equal to zero, then the descriptor table is taken as contiguous by the DMA, in Ring mode.

Channel 1 Credit Shaping Control Register

The `EMAC_DMA1_CHCBSCTL` register controls the credit-based shaper algorithm in the Traffic Manager for scheduling the frames for transmission. This register is present only when you select the Transmit Channel 1 in the AV mode.

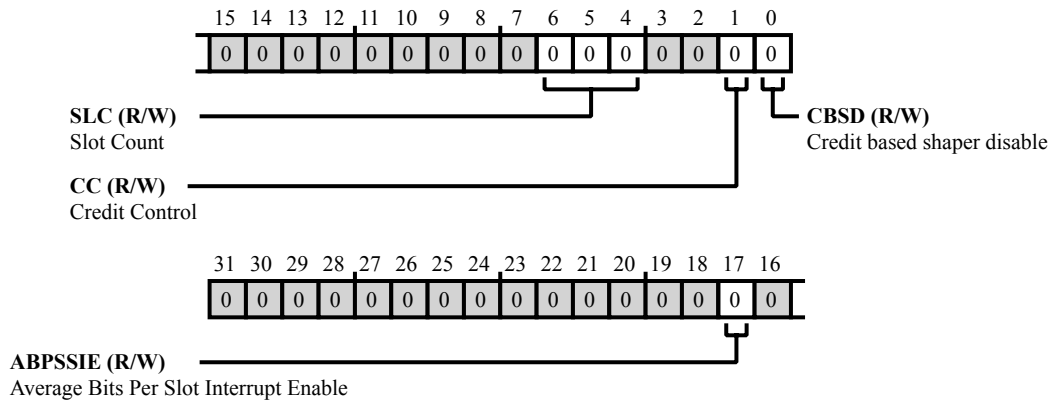


Figure 24-46: EMAC_DMA1_CHCBSCTL Register Diagram

Table 24-78: EMAC_DMA1_CHCBSCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	ABPSSIE	Average Bits Per Slot Interrupt Enable. When the <code>EMAC_DMA1_CHCBSCTL.ABPSSIE</code> bit is set, the MAC asserts an interrupt (<code>sbd_intr_o</code> or <code>mci_intr_o</code>) when the average bits per slot status is updated for Channel 1. When this bit is cleared, an interrupt is not asserted for such an event.
6:4 (R/W)	SLC	Slot Count. The <code>EMAC_DMA1_CHCBSCTL.SLC</code> bit field programs the number of slots (of duration 125 micro-sec) over which the average transmitted bits per slot (provided in the CBS Status register) are computed for Channel 1 when the credit-based shaper algorithm is enabled.
1 (R/W)	CC	Credit Control. The <code>EMAC_DMA1_CHCBSCTL.CC</code> bit, when reset, sets the accumulated credit parameter in the credit-based shaper algorithm logic to zero when there is positive credit and no frame to transmit in Channel 1.
0 (R/W)	CBSD	Credit based shaper disable. The <code>EMAC_DMA1_CHCBSCTL.CBSD</code> bit disables the credit-based shaper algorithm for Channel 1 traffic and makes the traffic management algorithm to strict priority for Channel 1 over Channel 0. When reset, the credit-based shaper algorithm schedules the traffic in Channel 1 for transmission.

Channel 1 Average Traffic Transmitted Register

The `EMAC_DMA1_CHCBSSTAT` register provides the average traffic transmitted in Channel 1. This register is present only when you select the Transmit Channel 1 in the AV mode.

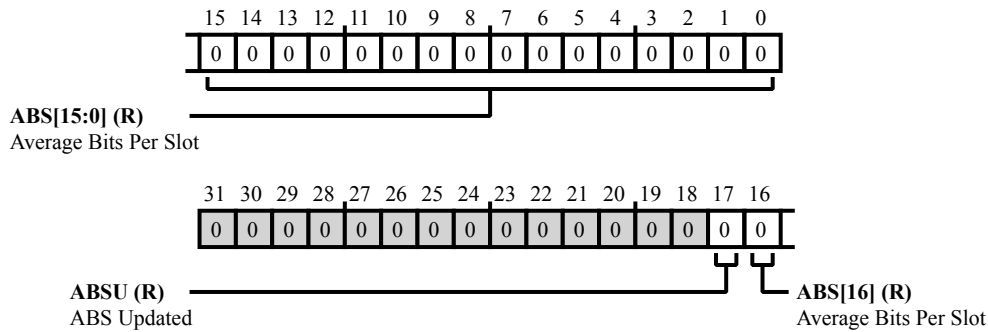


Figure 24-47: EMAC_DMA1_CHCBSSTAT Register Diagram

Table 24-79: EMAC_DMA1_CHCBSSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/NW)	ABSU	ABS Updated. The <code>EMAC_DMA1_CHCBSSTAT.ABSU</code> bit indicates that the MAC has updated the ABS value. This bit is cleared when the application reads the ABS value.
16:0 (R/NW)	ABS	Average Bits Per Slot. The <code>EMAC_DMA1_CHCBSSTAT.ABS</code> bit field contains the average transmitted bits per slot. This field is computed over programmed number of slots (<code>EMAC_DMA1_CHCBSCTL.SLC</code> bit field) for Channel 1 traffic. The maximum value is 0x30D4 for 100 Mbps and 0x1E848 for 1000 Mbps.

Channel 1 High Credit Value Register

The `EMAC_DMA1_CHHIC` register provides the maximum value that can be accumulated for Channel 1 in the credit parameter of the credit-based shaper algorithm. This register is present only when you select the Transmit Channel 1 in the AV mode.

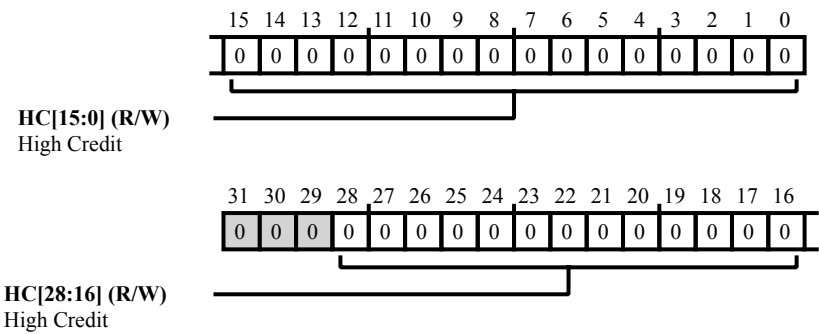


Figure 24-48: EMAC_DMA1_CHHIC Register Diagram

Table 24-80: EMAC_DMA1_CHHIC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:0 (R/W)	HC	High Credit. The <code>EMAC_DMA1_CHHIC.HC</code> bit field contains the hiCredit value required for the credit-based shaper algorithm for Channel 1.

Channel 1 Idle Slope Credit Value Register

The `EMAC_DMA1_CHISC` register provides the bandwidth allocated for the AV traffic on Channel 1. This register is present only when you select the Transmit Channel 1 in the AV mode.

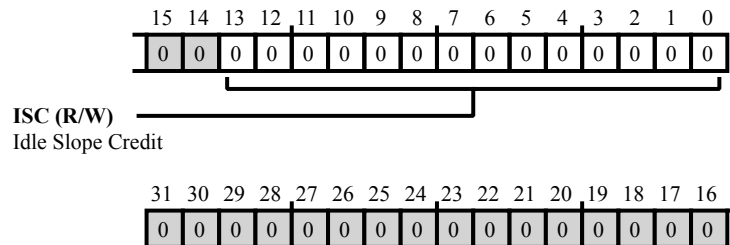


Figure 24-49: EMAC_DMA1_CHISC Register Diagram

Table 24-81: EMAC_DMA1_CHISC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:0 (R/W)	ISC	Idle Slope Credit. The <code>EMAC_DMA1_CHISC.ISC</code> bit field contains the <code>idleSlopeCredit</code> value required for the credit-based shaper algorithm for Channel 1.

Channel 1 Low Credit Value Register

The `EMAC_DMA1_CHLOC` register provides the minimum value that can be accumulated for Channel 1 in the credit parameter of the credit-based shaper algorithm. This register is present only when you select Transmit Channel 1 in the AV mode.

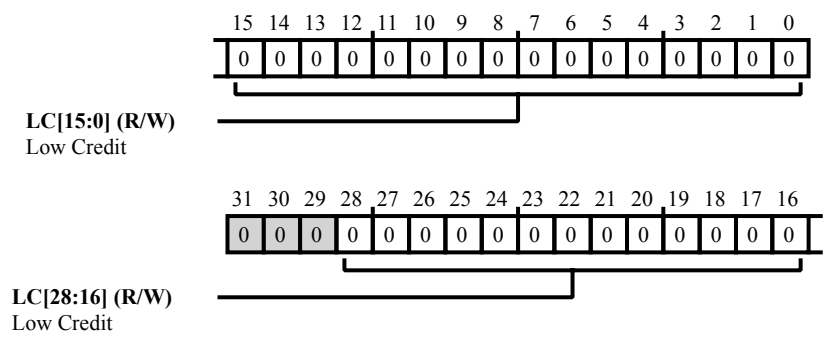


Figure 24-50: EMAC_DMA1_CHLOC Register Diagram

Table 24-82: EMAC_DMA1_CHLOC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:0 (R/W)	LC	Low Credit. The <code>EMAC_DMA1_CHLOC.LC</code> bit field contains the <code>loCredit</code> value required for the credit-based shaper algorithm for Channel 1.

Channel 1 Control Bits for Slot Function Register

The `EMAC_DMA1_CHSFCS` register controls the slot comparison feature that the Channel 1 transmit DMA uses to fetch the buffer data from system memory.

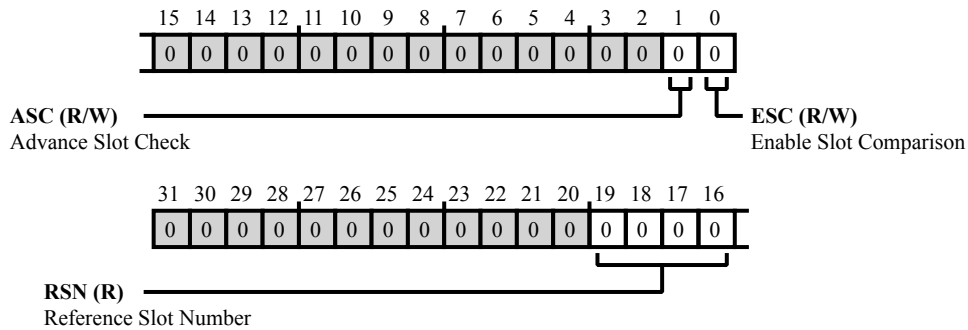


Figure 24-51: EMAC_DMA1_CHSFCS Register Diagram

Table 24-83: EMAC_DMA1_CHSFCS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:16 (R/NW)	RSN	Reference Slot Number. The <code>EMAC_DMA1_CHSFCS</code> .RSN bits, gives the current value of the reference slot number in DMA used for comparison checking.
1 (R/W)	ASC	Advance Slot Check. When set, this bit enables the DMA to fetch the data from the buffer when the slot number (SLOTNUM) programmed in the transmit descriptor is ---equal to the reference slot number given in Bits [19:16] -or- ahead of the reference slot number by up to two slots. This bit is applicable only when Bit 0 (ESC) is set.
0 (R/W)	ESC	Enable Slot Comparison. The <code>EMAC_DMA1_CHSFCS</code> .ESC bits, When set, this bit enables the checking of the slot numbers, programmed in the transmit descriptor, with the current reference given in Bits [19:16]. The DMA fetches the data from the corresponding buffer only when the slot number is equal to the reference slot number or is ahead of the reference slot number by one slot. When reset, this bit disables the checking of the slot numbers. The DMA fetches the data immediately after the descriptor is processed.

Channel 1 Send Slope Credit Value Register

The `EMAC_DMA1_CHSSC` register provides the bandwidth that is available for the AV traffic on other channels. This register is present only when you select the Transmit Channel 1 in the AV mode.

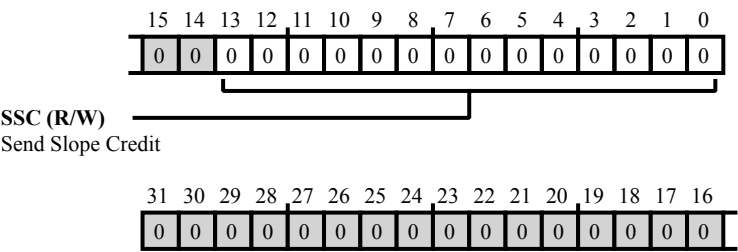


Figure 24-52: EMAC_DMA1_CHSSC Register Diagram

Table 24-84: EMAC_DMA1_CHSSC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:0 (R/W)	SSC	Send Slope Credit. The <code>EMAC_DMA1_CHSSC.SSC</code> bit field contains the <code>sendSlopeCredit</code> value required for credit-based shaper algorithm for Channel 1.

DMA Interrupt Enable Register

The `EMAC_DMA1_IEN` register enables (unmasks) EMAC DMA interrupts.

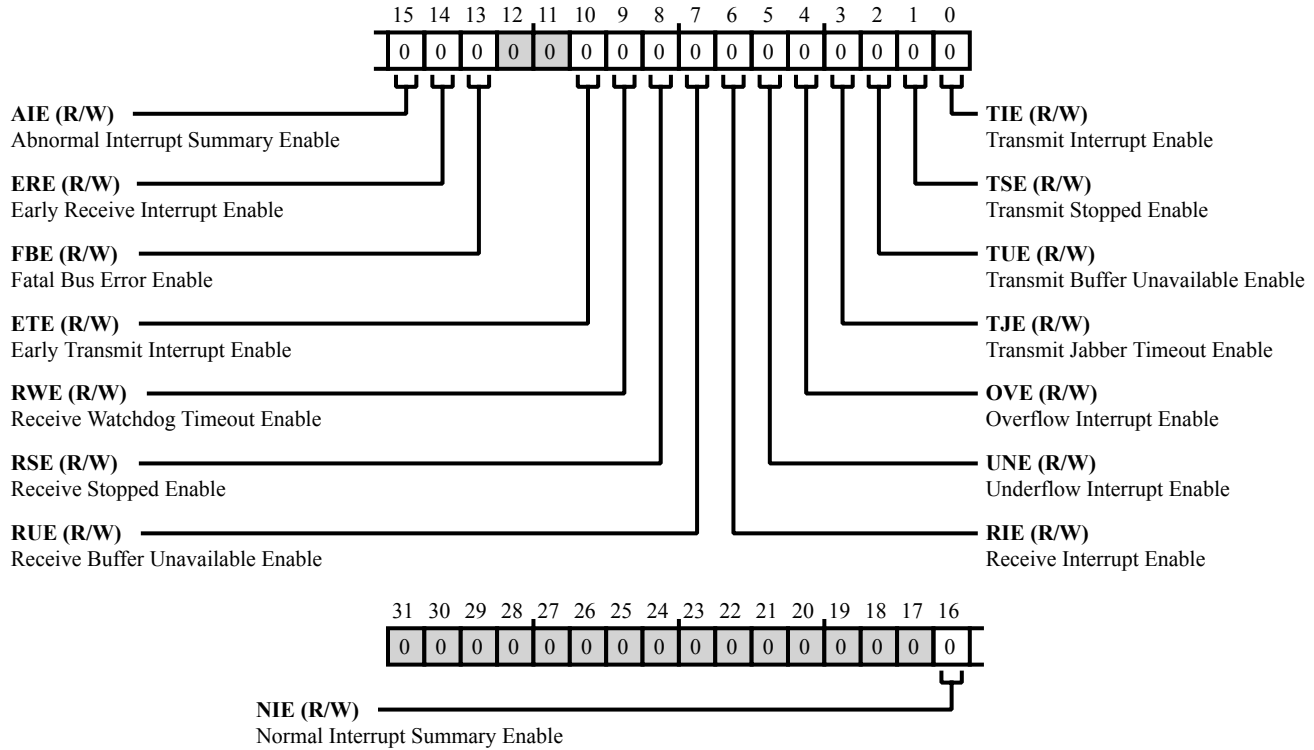


Figure 24-53: EMAC_DMA1_IEN Register Diagram

Table 24-85: EMAC_DMA1_IEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	NIE	Normal Interrupt Summary Enable. The <code>EMAC_DMA1_IEN.NIE</code> bit, when set, enables a normal interrupt. When this bit is reset, a normal interrupt is disabled. This bit enables the following bits: <code>EMAC_DMA1_STAT.TI</code> , <code>EMAC_DMA1_STAT.TU</code> , <code>EMAC_DMA1_STAT.RI</code> , and <code>EMAC_DMA1_STAT.ERI</code> .
15 (R/W)	AIE	Abnormal Interrupt Summary Enable. The <code>EMAC_DMA1_IEN.AIE</code> bit, when set, enables an abnormal interrupt. When this bit is reset, an Abnormal Interrupt is disabled. This bit enables the following bits: <code>EMAC_DMA1_STAT.TPS</code> , <code>EMAC_DMA1_STAT.TJT</code> , <code>EMAC_DMA1_STAT.OVF</code> , <code>EMAC_DMA1_STAT.RU</code> , <code>EMAC_DMA1_STAT.RPS</code> , <code>EMAC_DMA1_STAT.RWT</code> , <code>EMAC_DMA1_STAT.ETI</code> , and <code>EMAC_DMA1_STAT.FBI</code> .

Table 24-85: EMAC_DMA1_IEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	ERE	Early Receive Interrupt Enable. The EMAC_DMA1_IEN.ERE bit, when set (and with EMAC_DMA1_IEN.NIE =1), enables the Early Receive Interrupt. When this bit is reset, Early Receive Interrupt is disabled.
13 (R/W)	FBE	Fatal Bus Error Enable. The EMAC_DMA1_IEN.FBE bit, when set (and with EMAC_DMA1_IEN.AIE =1), enables the Fatal Bus Error Interrupt. When this bit is reset, Fatal Bus Error Enable Interrupt is disabled.
10 (R/W)	ETE	Early Transmit Interrupt Enable. The EMAC_DMA1_IEN.ETE bit, when this bit is set (and with EMAC_DMA1_IEN.AIE =1), enables the Early Transmit Interrupt. When this bit is reset, Early Transmit Interrupt is disabled.
9 (R/W)	RWE	Receive Watchdog Timeout Enable. The EMAC_DMA1_IEN.RWE bit, when set (and with EMAC_DMA1_IEN.AIE =1), enables the Receive Watchdog Timeout Interrupt. When this bit is reset, Receive Watchdog Timeout Interrupt is disabled.
8 (R/W)	RSE	Receive Stopped Enable. The EMAC_DMA1_IEN.RSE bit, when set (and with EMAC_DMA1_IEN.AIE =1), enables the Receive Stopped Interrupt is enabled. When this bit is reset, Receive Stopped Interrupt is disabled.
7 (R/W)	RUE	Receive Buffer Unavailable Enable. The EMAC_DMA1_IEN.RUE bit, when set (and with EMAC_DMA1_IEN.AIE =1), enables the Receive Buffer Unavailable Interrupt. When this bit is reset, the Receive Buffer Unavailable Interrupt is disabled.
6 (R/W)	RIE	Receive Interrupt Enable. The EMAC_DMA1_IEN.RIE bit, when set (and with EMAC_DMA1_IEN.NIE =1), enables the Receive Interrupt. When this bit is reset, Receive Interrupt is disabled.
5 (R/W)	UNE	Underflow Interrupt Enable. The EMAC_DMA1_IEN.UNE bit, when set (and with EMAC_DMA1_IEN.AIE =1), enables the Transmit Underflow Interrupt. When this bit is reset, Underflow Interrupt is disabled.
4 (R/W)	OVE	Overflow Interrupt Enable. The EMAC_DMA1_IEN.OVE bit, when set (and with EMAC_DMA1_IEN.AIE =1), enables the Receive Overflow Interrupt. When this bit is reset, Overflow Interrupt is disabled.

Table 24-85: EMAC_DMA1_IEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	TJE	Transmit Jabber Timeout Enable. The EMAC_DMA1_IEN.TJE bit, when set (and with EMAC_DMA1_IEN.AIE =1), enables the Transmit Jabber Timeout Interrupt. When this bit is reset, Transmit Jabber Timeout Interrupt is disabled.
2 (R/W)	TUE	Transmit Buffer Unavailable Enable. The EMAC_DMA1_IEN.TUE bit, when set (and with EMAC_DMA1_IEN.NIE =1), enables the Transmit Buffer Unavailable Interrupt. When this bit is reset, Transmit Buffer Unavailable Interrupt is disabled.
1 (R/W)	TSE	Transmit Stopped Enable. The EMAC_DMA1_IEN.TSE bit, when set (and with EMAC_DMA1_IEN.AIE =1), enables the Transmission Stopped Interrupt. When this bit is reset, Transmission Stopped Interrupt is disabled.
0 (R/W)	TIE	Transmit Interrupt Enable. The EMAC_DMA1_IEN.TIE bit, when set (and with EMAC_DMA1_IEN.NIE =1), enables the Transmit Interrupt. When this bit is reset, Transmit Interrupt is disabled.

DMA Missed Frame Register

The `EMAC_DMA1_MISS_FRM` register contains counters for EMAC DMA missed frames and buffer overflows.

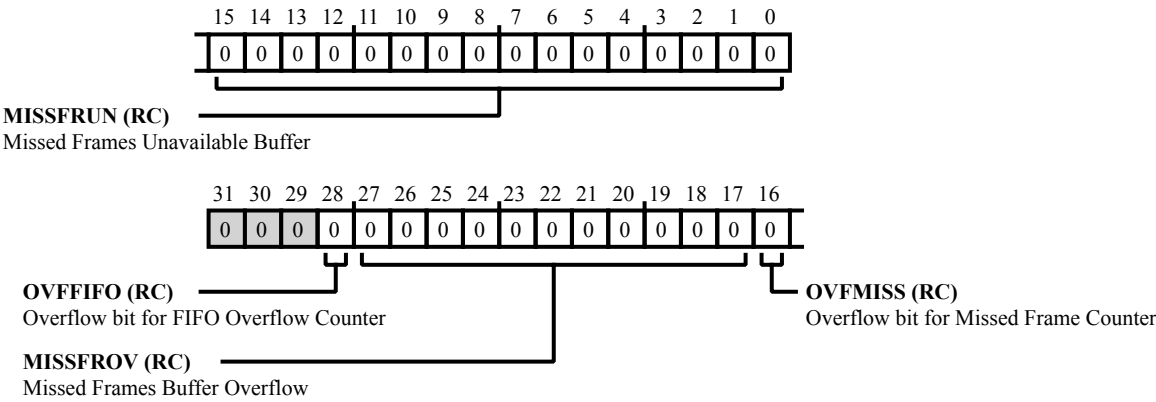


Figure 24-54: `EMAC_DMA1_MISS_FRM` Register Diagram

Table 24-86: `EMAC_DMA1_MISS_FRM` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (RC/NW)	OVFFIFO	Overflow bit for FIFO Overflow Counter. The <code>EMAC_DMA1_MISS_FRM.OVFFIFO</code> bit holds the overflow bit for FIFO Overflow Counter.
27:17 (RC/NW)	MISSFROV	Missed Frames Buffer Overflow. The <code>EMAC_DMA1_MISS_FRM.MISSFROV</code> bits indicate the number of frames missed by the application due to buffer overflow.
16 (RC/NW)	OVFMIS	Overflow bit for Missed Frame Counter. The <code>EMAC_DMA1_MISS_FRM.OVFMIS</code> bit holds the overflow bit for the Missed Frame Counter.
15:0 (RC/NW)	MISSFRUN	Missed Frames Unavailable Buffer. The <code>EMAC_DMA1_MISS_FRM.MISSFRUN</code> bits indicate the number of frames missed by the controller because of the Application Receive Buffer being unavailable.

DMA Operation Mode Register

The `EMAC_DMA1_OPMODE` register selects receive and transmit DMA operating modes.

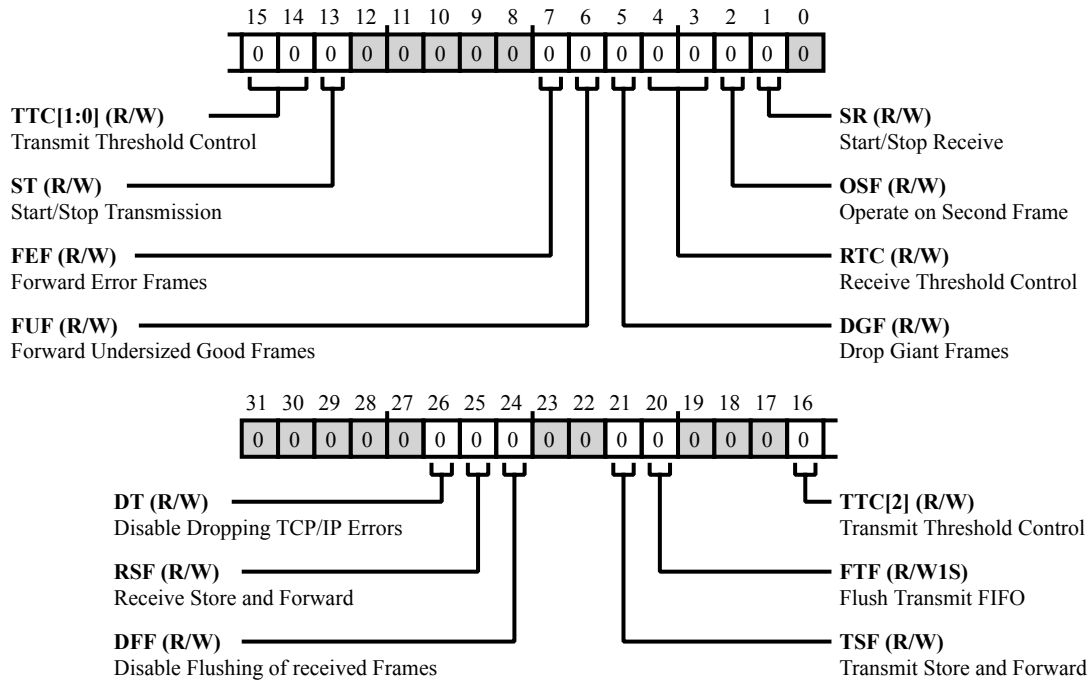


Figure 24-55: `EMAC_DMA1_OPMODE` Register Diagram

Table 24-87: `EMAC_DMA1_OPMODE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	DT	Disable Dropping TCP/IP Errors. The <code>EMAC_DMA1_OPMODE.DT</code> bit, when set, directs the core not to drop frames that only have errors detected by the Receive Checksum Offload engine. Such frames do not have any errors (including FCS error) in the Ethernet frame received by the MAC but have errors in the encapsulated payload only. When this bit is reset, all error frames are dropped if the <code>EMAC_DMA1_OPMODE.FEF</code> bit is reset.
25 (R/W)	RSF	Receive Store and Forward. The <code>EMAC_DMA1_OPMODE.RSF</code> bit, when set, directs the MFL only to read a frame from the Rx FIFO after the complete frame has been written to it, ignoring the <code>EMAC_DMA1_OPMODE.RTC</code> bits. When this bit is reset, the Rx FIFO operates in threshold mode, subject to the threshold specified by the <code>EMAC_DMA1_OPMODE.RTC</code> bits.
24 (R/W)	DFF	Disable Flushing of received Frames. The <code>EMAC_DMA1_OPMODE.DFF</code> bit, when set, directs the Rx DMA not to flush any frames because of the unavailability of receive descriptors/buffers as it does normally when this bit is reset.

Table 24-87: EMAC_DMA1_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration																
21 (R/W)	TSF	Transmit Store and Forward. The EMAC_DMA1_OPMODE . TSF bit, when set, starts transmission when a full frame resides in the MFL Transmit FIFO. When this bit is set, the TTC values specified in Register 6[16:14] are ignored. This bit should be changed only when transmission is stopped.																
20 (R/W1S)	FTF	Flush Transmit FIFO. The EMAC_DMA1_OPMODE . FTF bit, when set, directs the transmit FIFO controller logic to reset to its default values and thus all data in the Tx FIFO is lost/flushed. This bit is cleared internally when the flushing operation is completed fully. The Operation Mode register should not be written to until this bit is cleared. The data which is already accepted by the MAC transmitter is not flushed. It is scheduled for transmission and results in underflow and runt frame transmission. Note: The flush operation completes only after emptying the Tx FIFO of its contents and all the pending Transmit Status of the transmitted frames are accepted by the host. In order to complete this flush operation, the PHY transmit clock is required to be active. This field cleared to 1b0 by the core (Self Clear). The application cannot clear this type of field, and a register write of 1b0 to this bit has no effect on this field.																
16:14 (R/W)	TTC	Transmit Threshold Control. The EMAC_DMA1_OPMODE . TTC bits control the threshold level of the MFL Transmit FIFO. Transmission starts when the frame size within the MFL Transmit FIFO is larger than the threshold. In addition, full frames with a length less than the threshold are also transmitted. These bits are used only when the EMAC_DMA1_OPMODE . TSF bit is reset. The value =011 is not used. <table><tr><td>0</td><td>64</td></tr><tr><td>1</td><td>128</td></tr><tr><td>2</td><td>192</td></tr><tr><td>3</td><td>256</td></tr><tr><td>4</td><td>40</td></tr><tr><td>5</td><td>32</td></tr><tr><td>6</td><td>24</td></tr><tr><td>7</td><td>16</td></tr></table>	0	64	1	128	2	192	3	256	4	40	5	32	6	24	7	16
0	64																	
1	128																	
2	192																	
3	256																	
4	40																	
5	32																	
6	24																	
7	16																	

Table 24-87: EMAC_DMA1_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	ST	Start/Stop Transmission. The EMAC_DMA1_OPMODE.ST bit, when set, places transmission in the Running state, and the DMA checks the Transmit List at the current position for a frame to be transmitted. Descriptor acquisition is attempted either from the current position in the list, which is the Transmit List Base Address set by Transmit Descriptor List Address, or from the position retained when transmission was stopped previously. If the current descriptor is not owned by the DMA, transmission enters the Suspended state, and the EMAC_DMA1_STAT.TU bit is set. The Start Transmission command is effective only when transmission is stopped. If the command is issued before setting the EMAC_DMA1_TXDSC_CUR address register, then the DMA behavior is unpredictable. When this bit is reset, the transmission process is placed in the Stopped state after completing the transmission of the current frame. The Next Descriptor position in the Transmit List is saved, and becomes the current position when transmission is restarted. The stop transmission command is effective only when the transmission of the current frame is complete or the transmission is in the Suspended state.
7 (R/W)	FEF	Forward Error Frames. The EMAC_DMA1_OPMODE.FEF bit, when reset, directs the Rx FIFO to drop frames with error status (CRC error, collision error, giant frame, watchdog timeout, overflow). However, if the frames start byte (write) pointer is already transferred to the read controller side (in Threshold mode), then the frames are not dropped. When EMAC_DMA1_OPMODE.FEF bit is set, all frames except runt error frames are forwarded to the DMA. But when Rx FIFO overflows when a partial frame is written, then such frames are dropped even when EMAC_DMA1_OPMODE.FEF is set.
6 (R/W)	FUF	Forward Undersized Good Frames. The EMAC_DMA1_OPMODE.FUF bit, when set, directs the Rx FIFO to forward Undersized frames (frames with no Error and length less than 64 bytes) including pad-bytes and CRC). When reset, the Rx FIFO drops all frames of less than 64 bytes, unless it is already transferred because of lower value of Receive Threshold (for example, EMAC_DMA1_OPMODE.RTC = 01).
5 (R/W)	DGF	Drop Giant Frames. The EMAC_DMA1_OPMODE.DGF bit, when set, the MAC drops the received giant frames in the Rx FIFO, that is, frames that are larger than the computed giant frame limit. When reset, the MAC does not drop the giant frames in the Rx FIFO.

Table 24-87: EMAC_DMA1_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:3 (R/W)	RTC	Receive Threshold Control. The <code>EMAC_DMA1_OPMODE.RTC</code> bits control the threshold level of the MFL Receive FIFO. Transfer (request) to DMA starts when the frame size within the MFL Receive FIFO is larger than the threshold. In addition, full frames with a length less than the threshold are transferred automatically. These bits are valid only when the <code>EMAC_DMA1_OPMODE.RSF</code> bit is zero, and are ignored when the <code>EMAC_DMA1_OPMODE.RSF</code> bit is set to 1. The value =11 is not used.
		0 64
		1 32
		2 96
		3 128
2 (R/W)	OSF	Operate on Second Frame. The <code>EMAC_DMA1_OPMODE.OSF</code> bit, when set, instructs the DMA to process a second frame of Transmit data even before status for first frame is obtained.
1 (R/W)	SR	Start/Stop Receive. The <code>EMAC_DMA1_OPMODE.SR</code> bit, when set, places the Receive process in the Running state. The DMA attempts to acquire the descriptor from the Receive list and processes incoming frames. Descriptor acquisition is attempted from the current position in the list, which is the address set by DMA Receive Descriptor List Address or the position retained when the Receive process was previously stopped. If no descriptor is owned by the DMA, reception is suspended, and the <code>EMAC_DMA1_STAT.RU</code> bit is set. The Start Receive command is effective only when reception has stopped. If the command was issued before setting <code>EMAC_DMA1_RXDSC_CUR</code> address register, DMA behavior is unpredictable. When this bit is cleared, Rx DMA operation is stopped after the transfer of the current frame. The next descriptor position in the Receive list is saved and becomes the current position after the Receive process is restarted. The Stop Receive command is effective only when the Receive process is in either the Running (waiting for receive packet) or in the Suspended state.

DMA Rx Buffer Current Register

The `EMAC_DMA1_RXBUF_CUR` register holds the pointer to the current receive DMA buffer.

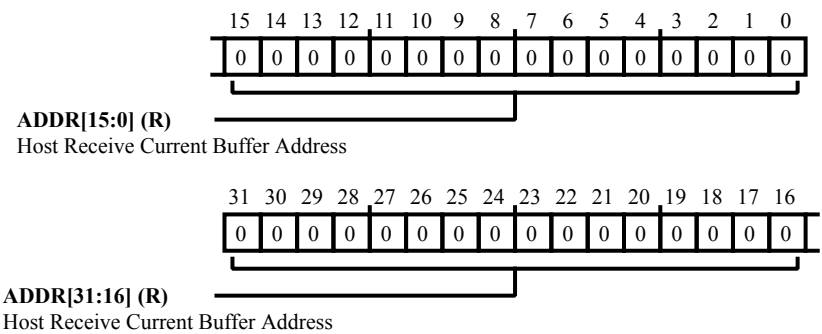


Figure 24-56: EMAC_DMA1_RXBUF_CUR Register Diagram

Table 24-88: EMAC_DMA1_RXBUF_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Receive Current Buffer Address. The <code>EMAC_DMA1_RXBUF_CUR.ADDR</code> bit field points to the current Receive Buffer address being read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Rx Descriptor List Address Register

The `EMAC_DMA1_RXDSC_ADDR` register holds the address for the DMA receive descriptor list. Writing to this Register is permitted only when reception is stopped. When stopped, this must be written to before the receive Start command is given. The processor can write to `EMAC_DMA1_RXDSC_ADDR` only when Rx DMA has stopped (`EMAC_DMA1_OPMODE.SR` bit =0). When stopped, it can be written with a new descriptor list address. When the processor sets the `EMAC_DMA1_OPMODE.SR` bit to 1, the DMA takes the newly programmed descriptor base address. If this register is not changed when the `EMAC_DMA1_OPMODE.SR` bit is cleared to 0, the DMA takes the descriptor address where it was stopped earlier.

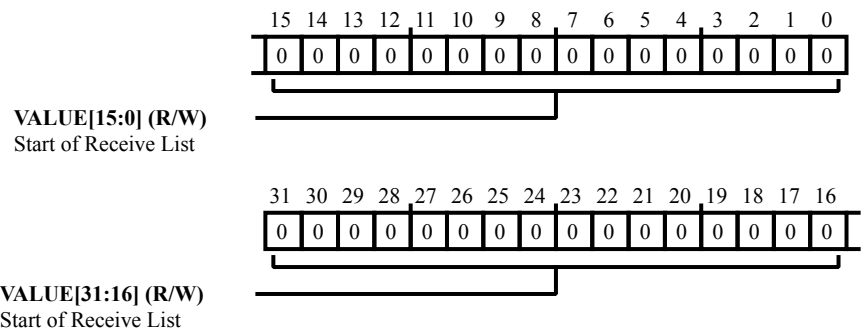


Figure 24-57: `EMAC_DMA1_RXDSC_ADDR` Register Diagram

Table 24-89: `EMAC_DMA1_RXDSC_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Start of Receive List. The <code>EMAC_DMA1_RXDSC_ADDR.VALUE</code> bit field contains the base address of the First Descriptor in the Receive Descriptor list. The LSB bits [1:0] for the 32bit bus width are ignored and are taken as all-zero by the DMA internally. Therefore, these LSB bits are Read-Only (RO).

DMA Rx Descriptor Current Register

The `EMAC_DMA1_RXDSC_CUR` register contains the current DMA receive descriptor.

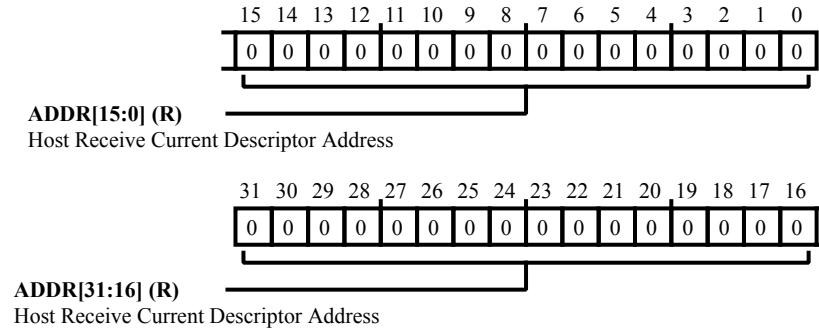


Figure 24-58: `EMAC_DMA1_RXDSC_CUR` Register Diagram

Table 24-90: `EMAC_DMA1_RXDSC_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Receive Current Descriptor Address. The <code>EMAC_DMA1_RXDSC_CUR.ADDR</code> bit field points to the start address of the current Receive Descriptor read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Rx Interrupt Watch Dog Register

The `EMAC_DMA1_RXIWDOG` register contains the timeout value for the EMAC DMA receive interrupt watch dog timer.

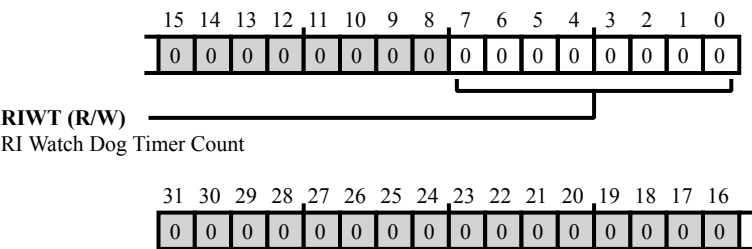


Figure 24-59: EMAC_DMA1_RXIWDOG Register Diagram

Table 24-91: EMAC_DMA1_RXIWDOG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	RIWT	RI Watch Dog Timer Count. The <code>EMAC_DMA1_RXIWDOG.RIWT</code> bit field indicates the number of system clock cycles multiplied by 256 for which the watchdog timer is set. The watchdog timer gets triggered with the programmed value after the Rx DMA completes the transfer of a frame for which the RI status bit is not set because of the setting in the corresponding descriptor <code>RDES1[31]</code> . When the watchdog timer runs out, the RI bit is set and the timer is stopped. The watchdog timer is reset when <code>EMAC_DMA1_STAT.RI</code> bit is set high because of automatic setting of <code>EMAC_DMA1_STAT.RI</code> as per <code>RDES1[31]</code> of any received frame.

DMA Rx Poll Demand Register

The `EMAC_DMA1_RXPOLL` register directs the EMAC to poll the receive descriptor list.

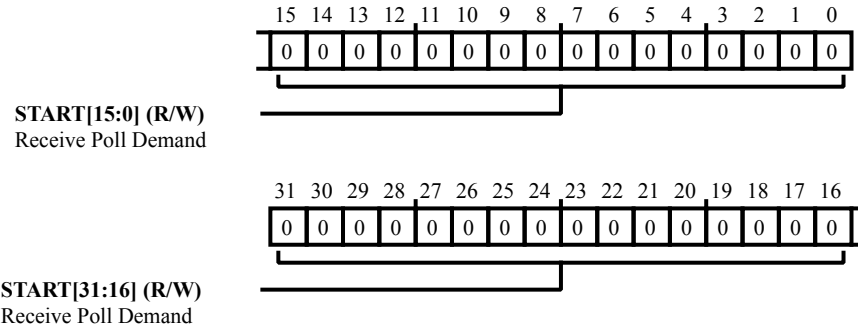


Figure 24-60: `EMAC_DMA1_RXPOLL` Register Diagram

Table 24-92: `EMAC_DMA1_RXPOLL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	START	Receive Poll Demand. The <code>EMAC_DMA1_RXPOLL.START</code> bits, when written with any value, cause the DMA to read the current descriptor pointed to by the <code>EMAC_DMA1_RXDSC_CUR</code> register. If that descriptor is not available (owned by application), reception returns to the Suspended state, and the <code>EMAC_DMA1_STAT.RU</code> bit is asserted. If the descriptor is available, the Receive DMA returns to the active state.

DMA Status Register

The `EMAC_DMA1_STAT` register indicates EMAC DMA status.

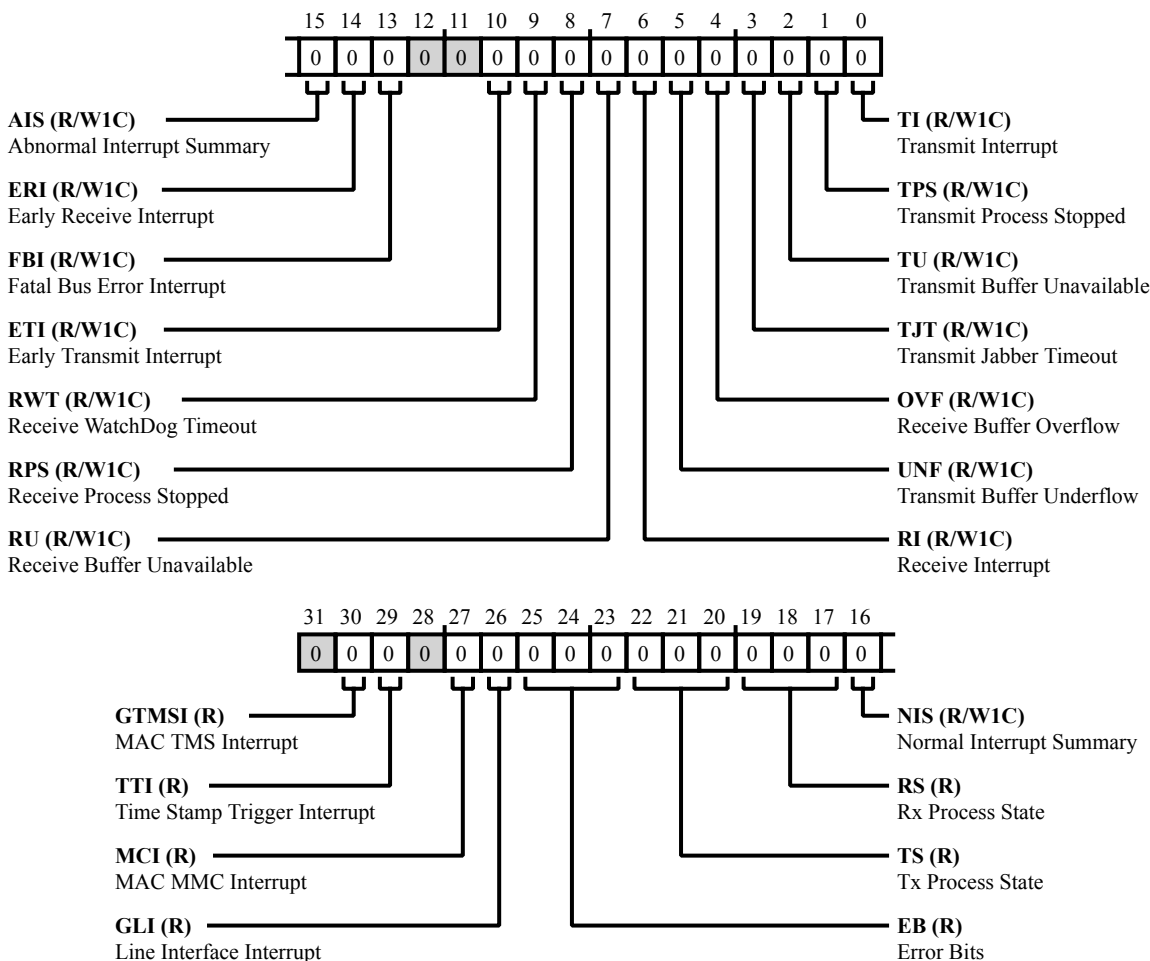


Figure 24-61: EMAC_DMA1_STAT Register Diagram

Table 24-93: EMAC_DMA1_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/NW)	GTMSI	MAC TMS Interrupt. MAC TMS Interrupt: The <code>EMAC_DMA1_STAT.GTMSI</code> bit indicates an interrupt event in the traffic manager and scheduler logic. To reset this bit, the software must read the corresponding registers (Channel Status Register) to get the exact cause of the interrupt and clear its source.

Table 24-93: EMAC_DMA1_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/NW)	TTI	Time Stamp Trigger Interrupt. The EMAC_DMA1_STAT.TTI bit indicates an interrupt event in the MAC core's Time Stamp Generator block. The software must read the corresponding registers in the MAC core to get the exact cause of interrupt and clear its source to reset this bit to =0. When this bit is high, the interrupt signal from the MAC is high.
27 (R/NW)	MCI	MAC MMC Interrupt. The EMAC_DMA1_STAT.MCI bit reflects an interrupt event in the MMC module of the MAC core. The software must read the corresponding registers in the MAC core to get the exact cause of interrupt and clear the source of interrupt to make this bit as =0. The interrupt signal from the MAC is high when this bit is high.
26 (R/NW)	GLI	Line Interface Interrupt. The EMAC_DMA1_STAT.GLI bit When set, this bit reflects any of the following interrupt events in the DWC_gmac interfaces
25:23 (R/NW)	EB	Error Bits. The EMAC_DMA1_STAT.EB bits indicate the type of error that caused a Bus Error (for example, error response on the SCB interface). These bits are valid only when the EMAC_DMA1_STAT.FBI bit is set. This field does not generate an interrupt.
		0 Error during data buffer access, write transfer, Rx DMA
		1 Error during data buffer access, write transfer, Tx DMA
		2 Error during data buffer access, read transfer, Rx DMA
		3 Error during data buffer access, read transfer, Tx DMA
		4 Error during descriptor access, write transfer, Rx DMA
		5 Error during descriptor access, write transfer, Tx DMA
		6 Error during descriptor access, read transfer, Rx DMA
		7 Error during descriptor access, read transfer, Tx DMA
22:20 (R/NW)	TS	Tx Process State. The EMAC_DMA1_STAT.TS bits indicate the transmit DMA state. This field does not generate an interrupt.
		0 Stopped; Reset or Stop Tx Command Issued
		1 Running; Fetching Tx Transfer Descriptor
		2 Running; Waiting for Status
		3 Reading Data from Host Memory Buffer and Queuing It to Tx Buffer
		4 TIME_STAMP Write State

Table 24-93: EMAC_DMA1_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		5	Reserved
		6	Suspended; Tx Descriptor Unavailable or Tx Buffer Underflow
		7	Closing Tx Descriptor
19:17 (R/NW)	RS	Rx Process State. The EMAC_DMA1_STAT.RS bits indicate the receive DMA state. This field does not generate an interrupt.	
		0	Stopped: Reset or Stop Rx Command Issued.
		1	Running: Fetching Rx Transfer Descriptor.
		2	Reserved
		3	Running: Waiting for Rx Packet
		4	Suspended: Rx Descriptor Unavailable
		5	Running: Closing Rx Descriptor
		6	TIME_STAMP Write State
		7	Running: Transferring Rx Packet Data from Rx Buffer to Host Memory
16 (R/W1C)	NIS	Normal Interrupt Summary. The value of the EMAC_DMA1_STAT.NIS bit field is the logical OR of the following when the corresponding interrupt bits are enabled in DMA Interrupt Enable Register: EMAC_DMA1_STAT.TI, EMAC_DMA1_STAT.TU, EMAC_DMA1_STAT.RI, and EMAC_DMA1_STAT.ERI. Only unmasked bits affect the Normal Interrupt Summary bit. This is a sticky bit and must be cleared (by writing a 1 to this bit) each time a corresponding bit that causes EMAC_DMA1_STAT.NIS to be set is cleared.	
15 (R/W1C)	AIS	Abnormal Interrupt Summary. The value of the EMAC_DMA1_STAT.AIS bit field is the logical OR of the following when the corresponding interrupt bits are enabled in DMA Interrupt Enable Register: EMAC_DMA1_IEN.TSE, EMAC_DMA1_IEN.TJE, EMAC_DMA1_IEN.OVE, EMAC_DMA1_IEN.UNE, EMAC_DMA1_IEN.RUE, EMAC_DMA1_IEN.RSE, EMAC_DMA1_IEN.RWE, EMAC_DMA1_IEN.ETE, and EMAC_DMA1_IEN.FBE. Only unmasked bits affect the Abnormal Interrupt Summary bit. This is a sticky bit and must be cleared each time a corresponding bit that causes EMAC_DMA1_STAT.AIS to be set is cleared.	
14 (R/W1C)	ERI	Early Receive Interrupt. The EMAC_DMA1_STAT.ERI bit indicates that the DMA had filled the first data buffer of the packet. The EMAC_DMA1_STAT.RI bit automatically clears this bit.	

Table 24-93: EMAC_DMA1_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1C)	FBI	Fatal Bus Error Interrupt. The EMAC_DMA1_STAT.FBI bit indicates that a bus error occurred, as detailed in the EMAC_DMA1_STAT.EB field. When this bit is set, the corresponding DMA engine disables all its bus accesses.
10 (R/W1C)	ETI	Early Transmit Interrupt. The EMAC_DMA1_STAT.ETI bit indicates that the frame to be transmitted was fully transferred to the MFL Transmit FIFO.
9 (R/W1C)	RWT	Receive WatchDog Timeout. The EMAC_DMA1_STAT.RWT bit is asserted when a frame with a length greater than 2,048 bytes is received (10, 240 when Jumbo Frame mode is enabled).
8 (R/W1C)	RPS	Receive Process Stopped. The EMAC_DMA1_STAT.RPS bit is asserted when the Receive Process enters the Stopped state.
7 (R/W1C)	RU	Receive Buffer Unavailable. The EMAC_DMA1_STAT.RU bit indicates that the Next Descriptor in the Receive List is owned by the application and cannot be acquired by the DMA. Receive Process is suspended. To resume processing Receive descriptors, the application should change the ownership of the descriptor and issue a Receive Poll Demand command. If no Receive Poll Demand is issued, Receive Process resumes when the next recognized incoming frame is received. This bit is set only when the previous Receive Descriptor was owned by the DMA.
6 (R/W1C)	RI	Receive Interrupt. The EMAC_DMA1_STAT.RI bit indicates the completion of frame reception. Specific frame status information has been posted in the descriptor. Reception remains in the Running state.
5 (R/W1C)	UNF	Transmit Buffer Underflow. The EMAC_DMA1_STAT.UNF bit indicates that the Transmit Buffer had an Underflow during frame transmission. Transmission is suspended and an Underflow Error TDES0[1] is set.
4 (R/W1C)	OVF	Receive Buffer Overflow. The EMAC_DMA1_STAT.OVF bit indicates that the Receive Buffer had an Overflow during frame reception. If the partial frame is transferred to application, the overflow status is set in RDES0[11].
3 (R/W1C)	TJT	Transmit Jabber Timeout. The EMAC_DMA1_STAT.TJT bit indicates that the Transmit Jabber Timer expired, meaning that the transmitter had been excessively active. The transmission process is aborted and placed in the Stopped state. This causes the Transmit Jabber Timeout TDES0[14] flag to assert.

Table 24-93: EMAC_DMA1_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W1C)	TU	Transmit Buffer Unavailable. The EMAC_DMA1_STAT.TU bit indicates that the Next Descriptor in the Transmit List is owned by the application and cannot be acquired by the DMA. Transmission is suspended. The value in the EMAC_DMA1_STAT.TS bits explain the Transmit Process state transitions. To resume processing transmit descriptors, the application should change the ownership of the bit of the descriptor and then issue a Transmit Poll Demand command.
1 (R/W1C)	TPS	Transmit Process Stopped. The EMAC_DMA1_STAT.TPS bit is set when the transmission is stopped.
0 (R/W1C)	TI	Transmit Interrupt. The EMAC_DMA1_STAT.TI bit indicates that frame transmission is finished and TDES1[31] is set in the First Descriptor.

DMA Tx Buffer Current Register

The `EMAC_DMA1_TXBUF_CUR` register holds the pointer to the current transmit DMA buffer.

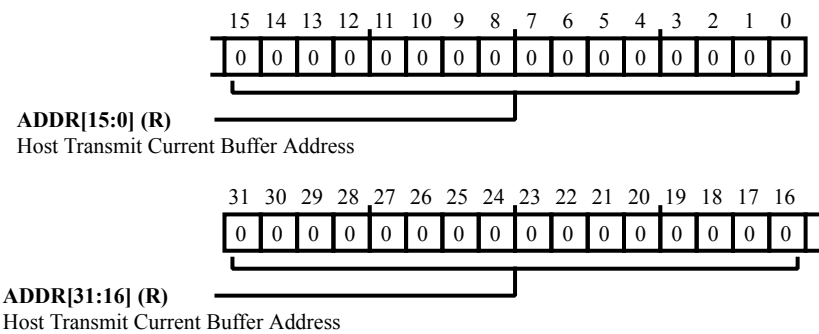


Figure 24-62: EMAC_DMA1_TXBUF_CUR Register Diagram

Table 24-94: EMAC_DMA1_TXBUF_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Transmit Current Buffer Address. The <code>EMAC_DMA1_TXBUF_CUR.ADDR</code> bit field points to the current Transmit Buffer Address being read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Tx Descriptor List Address Register

The `EMAC_DMA1_TXDSC_ADDR` register holds the address for the DMA transmit descriptor list. The processor can write to this Register only when Tx DMA has stopped (`EMAC_DMA1_OPMODE.ST` bit =0). When stopped, this can be written with a new descriptor list address. When the processor sets the `EMAC_DMA1_OPMODE.ST` bit to 1, the DMA takes the newly programmed descriptor base address. If this register is not changed when the `EMAC_DMA1_OPMODE.ST` bit is cleared to 0, then the DMA takes the descriptor address where it was stopped earlier.

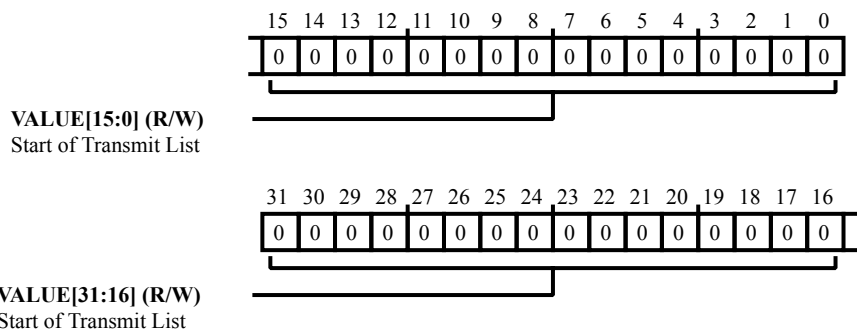


Figure 24-63: EMAC_DMA1_TXDSC_ADDR Register Diagram

Table 24-95: EMAC_DMA1_TXDSC_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Start of Transmit List. The <code>EMAC_DMA1_TXDSC_ADDR.VALUE</code> bit field contains the base address of the First Descriptor in the Transmit Descriptor list. The LSB bits [1:0] for 32bit bus width are ignored and are taken as all-zero by the DMA internally. Therefore, these LSB bits are Read-Only (RO).

DMA Tx Descriptor Current Register

The `EMAC_DMA1_TXDSC_CUR` register contains the current DMA transmit descriptor.

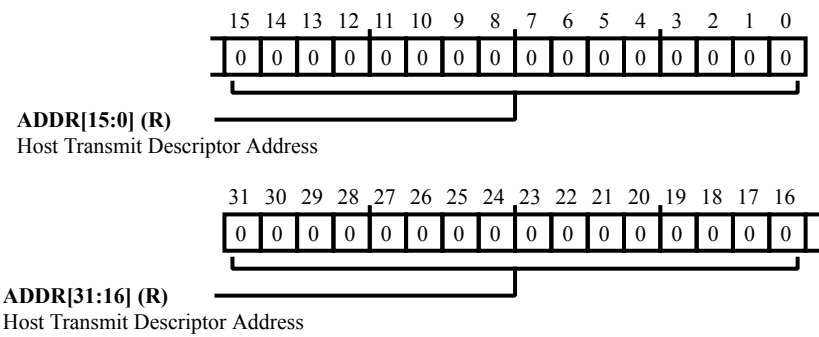


Figure 24-64: `EMAC_DMA1_TXDSC_CUR` Register Diagram

Table 24-96: `EMAC_DMA1_TXDSC_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Transmit Descriptor Address. The <code>EMAC_DMA1_TXDSC_CUR.ADDR</code> bit field points to the start address of the current Transmit Descriptor read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Tx Poll Demand Register

The `EMAC_DMA1_TXPOLL` register directs the EMAC to poll the transmit descriptor list.

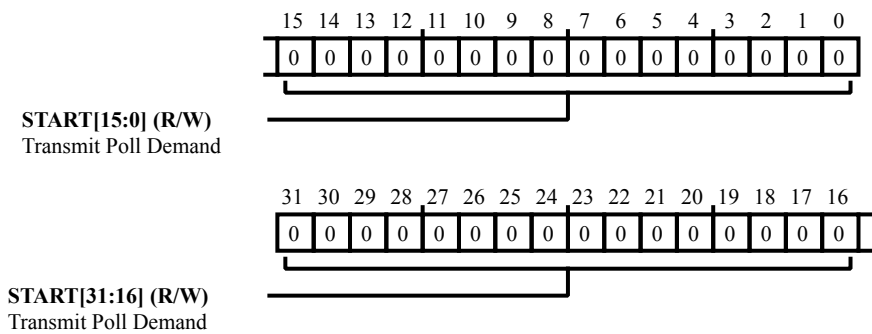


Figure 24-65: `EMAC_DMA1_TXPOLL` Register Diagram

Table 24-97: `EMAC_DMA1_TXPOLL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	START	Transmit Poll Demand. The <code>EMAC_DMA1_TXPOLL.START</code> bits, when written with any value, cause the DMA to read the current descriptor pointed to by <code>EMAC_DMA1_TXDSC_CUR</code> register. If that descriptor is not available (owned by application), transmission returns to the Suspend state, and the <code>EMAC_DMA1_STAT.TU</code> bit is asserted. If the descriptor is available, transmission resumes.

DMA Bus Mode Register

The `EMAC_DMA2_BUSMODE` register selects the DMA bus operating modes for EMAC DMA.

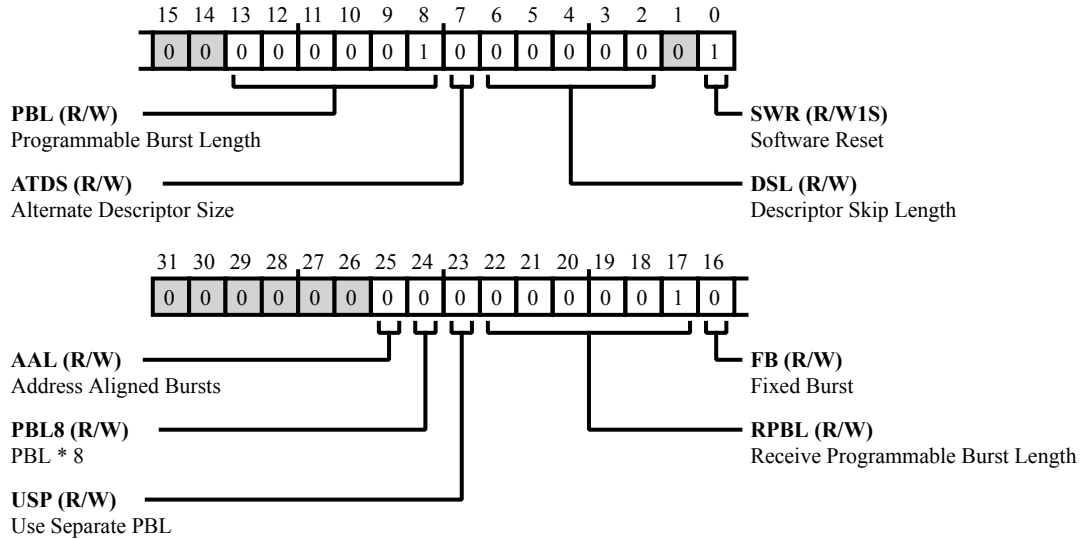


Figure 24-66: EMAC_DMA2_BUSMODE Register Diagram

Table 24-98: EMAC_DMA2_BUSMODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	AAL	Address Aligned Bursts. The <code>EMAC_DMA2_BUSMODE.AAL</code> bit, when set high and the <code>FB</code> bit equals 1, directs the SCB interface to generate all bursts aligned to the start address LS bits. If the <code>FB</code> bit is equal to 0, the first burst (accessing the data buffers start address) is not aligned, but subsequent bursts are aligned to the address.
24 (R/W)	PBL8	PBL * 8. The <code>EMAC_DMA2_BUSMODE.PBL8</code> bit, when set high, multiplies the PBL value programmed (bits [22:17] and bits [13:8]) eight times. Therefore, the DMA transfers the data in 8, 16, and 32 beats depending on the PBL value.
23 (R/W)	USP	Use Separate PBL. The <code>EMAC_DMA2_BUSMODE.USP</code> bit, when set high, configures the Rx DMA to use the value configured in bits [22:17] as PBL while the PBL value in bits [13:8] is applicable to Tx DMA operations only.

Table 24-98: EMAC_DMA2_BUSMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22:17 (R/W)	RPBL	Receive Programmable Burst Length. The <code>EMAC_DMA2_BUSMODE.RPBL</code> bits indicate the maximum number of beats to be transferred in one Rx DMA transaction. This is the maximum value that is used in a single block Read/Write. The Rx DMA always attempts to burst as specified in RPBL every time it starts a Burst transfer on the host bus. RPBL can be programmed with permissible values of 1, 2, 4, 8, 16, and 32. Any other value results in undefined behavior. These bits are valid and applicable only when USP is set high.
16 (R/W)	FB	Fixed Burst. The <code>EMAC_DMA2_BUSMODE.FB</code> bit controls whether the SCB Master interface performs fixed burst transfers or not. See the <code>EMAC_DMA0_BMODE.UNDEF</code> bit description for more information.
13:8 (R/W)	PBL	Programmable Burst Length. The <code>EMAC_DMA2_BUSMODE.PBL</code> bits indicate the maximum number of beats to be transferred in one DMA transaction. This is the maximum value that is used in a single block Read/Write. The DMA always attempts to burst as specified in PBL each time it starts a Burst transfer on the host bus. Any other value results in undefined behavior. When USP is set high, this PBL value is applicable for Tx DMA transactions only. $PBL\text{-max limit} = (FIFO\text{ size} / 2) / 4.$ $PBL\text{-max limit (transmit)} = 256\text{ bytes} / 2 / 4 = 32.$ $PBL\text{-max limit (receive)} = 128\text{ bytes} / 2 / 4 = 16.$ Note that this PBL is at the DMA end. If $PBL = 32$ and if <code>BLEN16</code> is enabled, the DMA automatically splits 32 bursts in to 2×16 bursts. If <code>EMAC_DMA2_BUSMODE.PBL = 8</code>, and if <code>EMAC_DMA0_BMODE.BLEN16</code> is enabled, the max burst is limited to <code>EMAC_DMA0_BMODE.BLEN8</code>. If <code>EMAC_DMA2_BUSMODE.PBL8</code> bit is set, the programmed PBL value is multiplied by 8 times internally. However, the result cannot be more than the above maximum limits specified above.
7 (R/W)	ATDS	Alternate Descriptor Size. The <code>EMAC_DMA2_BUSMODE.ATDS</code> bit, when set, increases the size of the alternate descriptor to 32 bytes (8 DWORDS). This is required when the Advanced Time Stamp feature or Full IPC Offload Engine is enabled in the receiver. When reset, the descriptor size reverts back to 4 DWORDS (16 bytes). The enhanced descriptor is not required if the Advanced Time Stamp and IPC Full Checksum Offload features are not enabled. In such case, you can use the 16 bytes descriptor to save 4 bytes of memory.

Table 24-98: EMAC_DMA2_BUSMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:2 (R/W)	DSL	Descriptor Skip Length. The EMAC_DMA2_BUSMODE.DSL bit specifies the number of 32-bit words to skip between two unchained descriptors. The address skipping starts from the end of current descriptor to the start of next descriptor. When DSL value is equal to zero, then the descriptor table is taken as contiguous by the DMA, in Ring mode.
0 (R/W1S)	SWR	Software Reset. The EMAC_DMA2_BUSMODE.SWR bit, when set, directs the MAC DMA Controller to reset all MAC Subsystem internal registers and logic. It is cleared automatically after the reset operation has completed in all of the core clock domains. Read a 0 value in this bit before re-programming any register of the core. Note: The reset operation is completed only when all the resets in all the active clock domains are de-asserted. Therefore, it is essential that all the PHY inputs clocks (applicable for the selected PHY interface) are present for software reset completion. This field cleared to 1b0 by the core (Self Clear). The application cannot clear this type of field, and a register write of 1b0 to this bit has no effect on this field.

Channel 2 Credit Shaping Control Register

The `EMAC_DMA2_CHCBSCTL` register controls the credit-based shaper algorithm in the Traffic Manager for scheduling the frames for transmission. This register is present only when you select the Transmit Channel 1 in the AV mode.

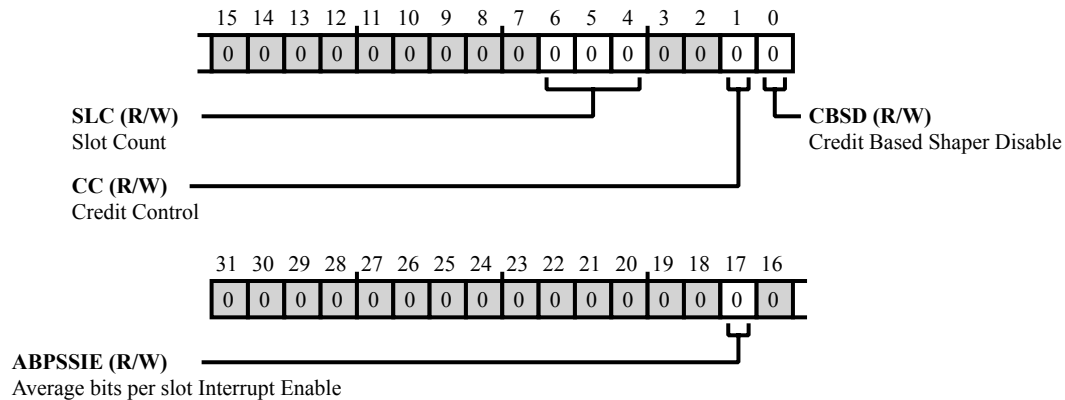


Figure 24-67: EMAC_DMA2_CHCBSCTL Register Diagram

Table 24-99: EMAC_DMA2_CHCBSCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	ABPSSIE	Average bits per slot Interrupt Enable. The <code>EMAC_DMA2_CHCBSCTL.ABPSSIE</code> bit asserts an interrupt (<code>sbd_intr_o</code> or <code>mci_intr_o</code>) when the average bits per slot status is updated for Channel 1. When this bit is cleared, interrupt is not asserted for such an event.
6:4 (R/W)	SLC	Slot Count. The <code>EMAC_DMA2_CHCBSCTL.SLC</code> bit field programs the number of slots (of duration 125 micro-sec) over which the average transmitted bits per slot (provided in the CBS Status register) need to be computed for Channel 1 when the credit-based shaper algorithm is enabled. The
1 (R/W)	CC	Credit Control. The <code>EMAC_DMA2_CHCBSCTL.CC</code> bit, when reset, sets the accumulated credit parameter in the credit-based shaper algorithm logic to zero when there is positive credit and no frame to transmit in Channel 1.
0 (R/W)	CBSD	Credit Based Shaper Disable. The <code>EMAC_DMA2_CHCBSCTL.CBSD</code> bit disables the credit-based shaper algorithm for Channel 1 traffic and makes the traffic management algorithm to strict priority for Channel 1 over Channel 0. When reset, the credit-based shaper algorithm schedules the traffic in Channel 1 for transmission.

Channel 2 Avg Traffic Transmitted Status Register

The `EMAC_DMA2_CHCBSSTAT` register provides the average traffic transmitted in Channel 1. This register is present only when you select the Transmit Channel 1 in the AV mode.

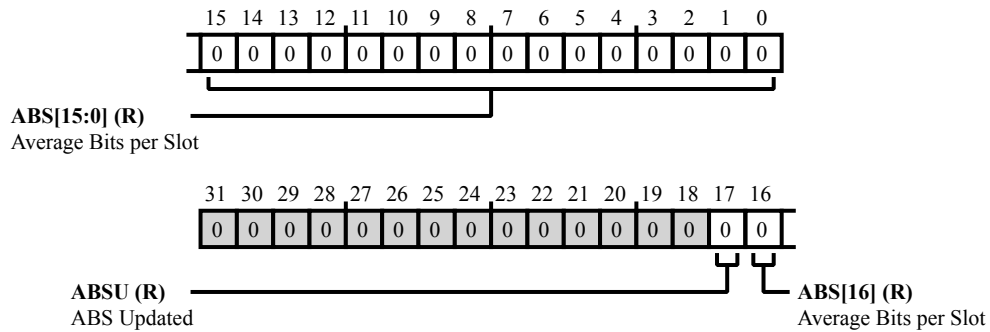


Figure 24-68: EMAC_DMA2_CHCBSSTAT Register Diagram

Table 24-100: EMAC_DMA2_CHCBSSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/NW)	ABSU	ABS Updated. The <code>EMAC_DMA2_CHCBSSTAT.ABSU</code> bits. When set, this bit indicates that the MAC has updated the ABS value. This bit is cleared when the application reads the ABS value.
16:0 (R/NW)	ABS	Average Bits per Slot. The <code>EMAC_DMA2_CHCBSSTAT.ABS</code> bit field contains the average transmitted bits per slot. This field is computed over programmed number of slots (<code>EMAC_DMA2_CHCBSCTL.SLC</code> bit field) for Channel 1 traffic. The maximum value is 0x30D4 for 100 Mbps and 0x1E848 for 1000 Mbps.

Channel 2 High Credit Value Register

The `EMAC_DMA2_CHHIC` register provides the maximum value that can be accumulated for Channel 1 in the credit parameter of the credit-based shaper algorithm. This register is present only when you select the Transmit Channel 1 in the AV mode.

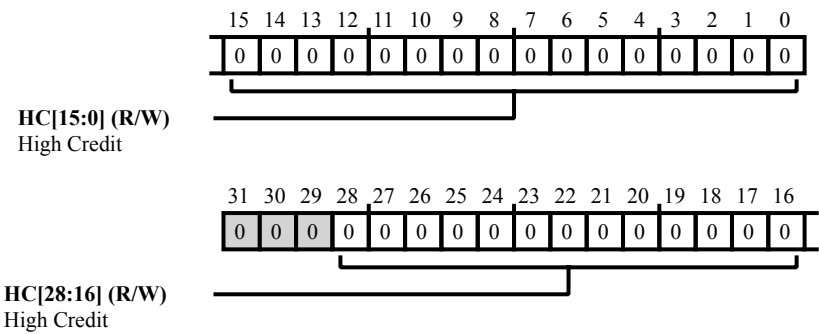


Figure 24-69: EMAC_DMA2_CHHIC Register Diagram

Table 24-101: EMAC_DMA2_CHHIC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:0 (R/W)	HC	High Credit. The <code>EMAC_DMA2_CHHIC.HC</code> bit field contains the hiCredit value required for the credit-based shaper algorithm for Channel 1.

Channel 2 Idle Slope Credit Value Register

The `EMAC_DMA2_CHISC` register provides the bandwidth allocated for the AV traffic on Channel 1. This register is present only when you select the Transmit Channel 1 in the AV mode.

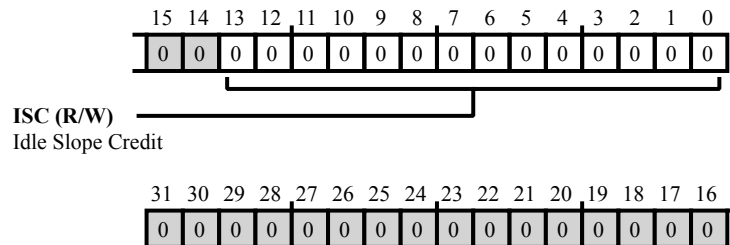


Figure 24-70: EMAC_DMA2_CHISC Register Diagram

Table 24-102: EMAC_DMA2_CHISC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:0 (R/W)	ISC	Idle Slope Credit. The <code>EMAC_DMA2_CHISC.ISC</code> bit field contains the <code>idleSlopeCredit</code> value required for the credit-based shaper algorithm for Channel 1.

Channel 2 Low Credit Value Register

The `EMAC_DMA2_CHLOC` register provides the minimum value that can be accumulated for Channel 1 in the credit parameter of the credit-based shaper algorithm. This register is present only when you select Transmit Channel 1 in the AV mode.

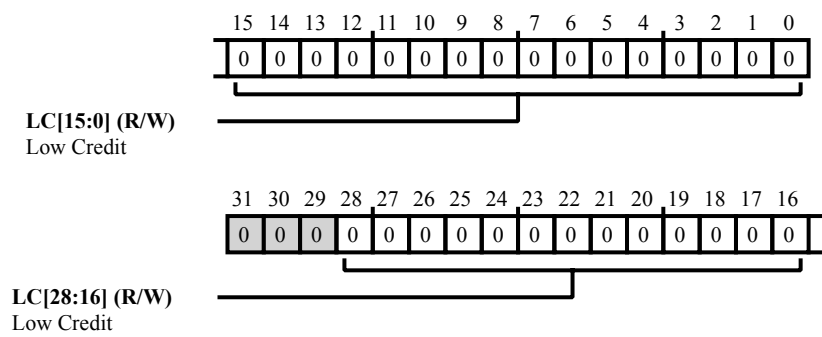


Figure 24-71: EMAC_DMA2_CHLOC Register Diagram

Table 24-103: EMAC_DMA2_CHLOC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28:0 (R/W)	LC	Low Credit. The <code>EMAC_DMA2_CHLOC.LC</code> bit field contains the <code>loCredit</code> value required for the credit-based shaper algorithm for Channel 1.

Channel 2 Control Bits for Slot Function Register

The `EMAC_DMA2_CHSFCS` register controls the slot comparison feature that the Channel 1 transmit DMA uses to fetch the buffer data from system memory.

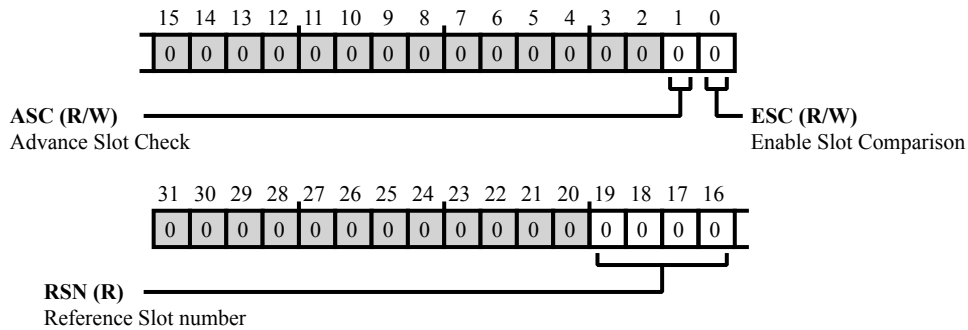


Figure 24-72: EMAC_DMA2_CHSFCS Register Diagram

Table 24-104: EMAC_DMA2_CHSFCS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:16 (R/NW)	RSN	Reference Slot number. The <code>EMAC_DMA2_CHSFCS</code> .RSN bits, gives the current value of the reference slot number in DMA used for comparison checking.
1 (R/W)	ASC	Advance Slot Check. The <code>EMAC_DMA2_CHSFCS</code> .ASC bits, When set, this bit enables the DMA to fetch the data from the buffer when the slot number (SLOTNUM) programmed in the transmit descriptor
0 (R/W)	ESC	Enable Slot Comparison. The <code>EMAC_DMA2_CHSFCS</code> .ESC bit enables the checking of the slot numbers, programmed in the transmit descriptor, with the current reference given in Bits [19:16]. The DMA fetches the data from the corresponding buffer only when the slot number is equal to the reference slot number or is ahead of the reference slot number by one slot. When reset, this bit disables the checking of the slot numbers. The DMA fetches the data immediately after the descriptor is processed.

Channel 2 Send Slope Credit Value Register

The `EMAC_DMA2_CHSSC` register provides the bandwidth that is available for the AV traffic on other channels. This register is present only when you select the Transmit Channel 1 in the AV mode.

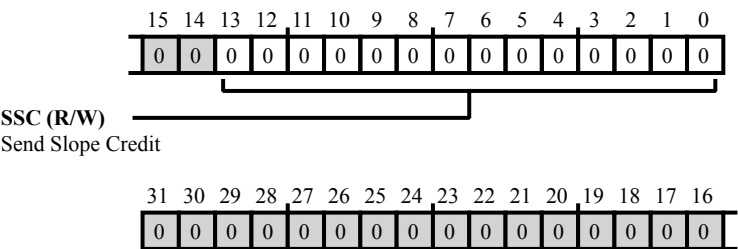


Figure 24-73: EMAC_DMA2_CHSSC Register Diagram

Table 24-105: EMAC_DMA2_CHSSC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:0 (R/W)	SSC	Send Slope Credit. The <code>EMAC_DMA2_CHSSC.SSC</code> bit field contains the <code>sendSlopeCredit</code> value required for credit-based shaper algorithm for Channel 1.

DMA Interrupt Enable Register

The `EMAC_DMA2_IEN` register enables (unmasks) EMAC DMA interrupts.

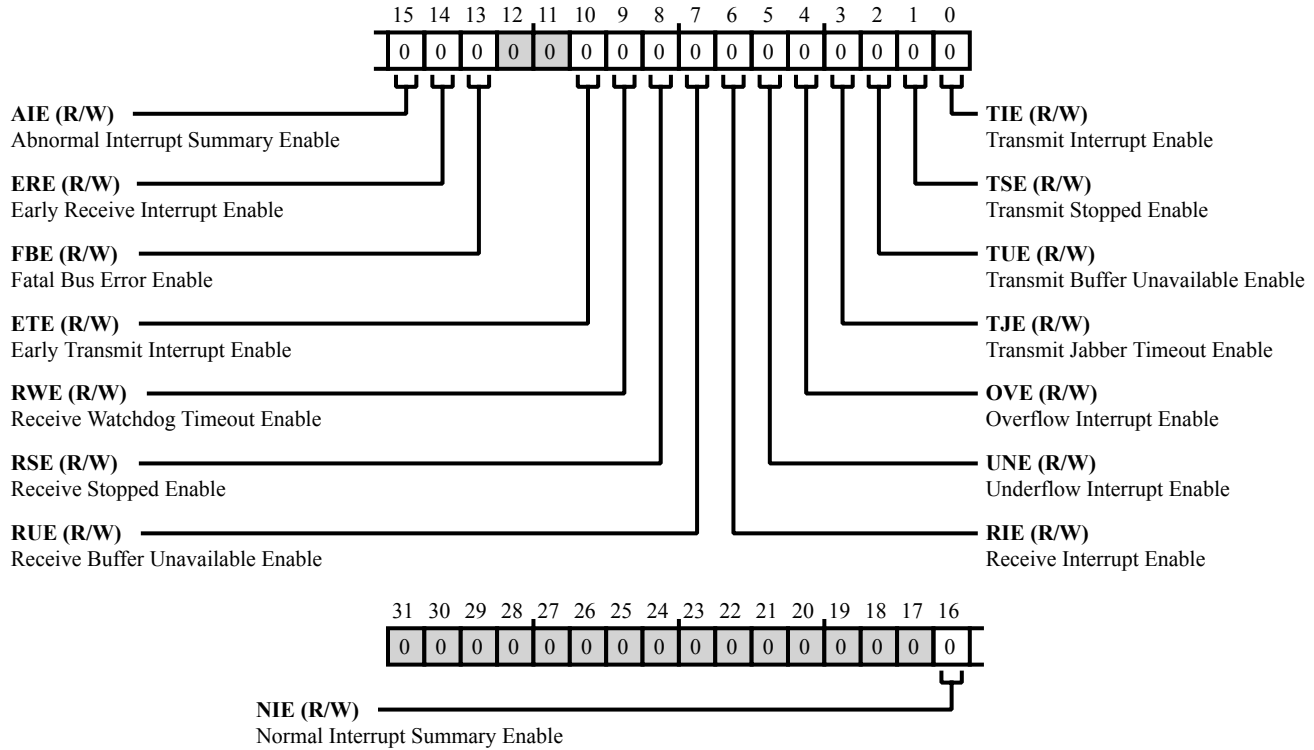


Figure 24-74: EMAC_DMA2_IEN Register Diagram

Table 24-106: EMAC_DMA2_IEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	NIE	Normal Interrupt Summary Enable. The <code>EMAC_DMA2_IEN.NIE</code> bit, when set, enables a normal interrupt. When this bit is reset, a normal interrupt is disabled. This bit enables the following bits: <code>EMAC_DMA2_STAT.TI</code> , <code>EMAC_DMA2_STAT.TU</code> , <code>EMAC_DMA2_STAT.RI</code> , and <code>EMAC_DMA2_STAT.ERI</code> .
15 (R/W)	AIE	Abnormal Interrupt Summary Enable. The <code>EMAC_DMA2_IEN.AIE</code> bit, when set, enables an abnormal interrupt. When this bit is reset, an Abnormal Interrupt is disabled. This bit enables the following bits: <code>EMAC_DMA2_STAT.TPS</code> , <code>EMAC_DMA2_STAT.TJT</code> , <code>EMAC_DMA2_STAT.OVF</code> , <code>EMAC_DMA2_STAT.RU</code> , <code>EMAC_DMA2_STAT.RPS</code> , <code>EMAC_DMA2_STAT.RWT</code> , <code>EMAC_DMA2_STAT.ETI</code> , and <code>EMAC_DMA2_STAT.FBI</code> .

Table 24-106: EMAC_DMA2_IEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	ERE	Early Receive Interrupt Enable. The EMAC_DMA2_IEN.ERE bit, when set (and with EMAC_DMA2_IEN.NIE =1), enables the Early Receive Interrupt. When this bit is reset, Early Receive Interrupt is disabled.
13 (R/W)	FBE	Fatal Bus Error Enable. The EMAC_DMA2_IEN.FBE bit, when set (and with EMAC_DMA2_IEN.AIE =1), enables the Fatal Bus Error Interrupt. When this bit is reset, Fatal Bus Error Enable Interrupt is disabled.
10 (R/W)	ETE	Early Transmit Interrupt Enable. The EMAC_DMA2_IEN.ETE bit, when this bit is set (and with EMAC_DMA2_IEN.AIE =1), enables the Early Transmit Interrupt. When this bit is reset, Early Transmit Interrupt is disabled.
9 (R/W)	RWE	Receive Watchdog Timeout Enable. The EMAC_DMA2_IEN.RWE bit, when set (and with EMAC_DMA2_IEN.AIE =1), enables the Receive Watchdog Timeout Interrupt. When this bit is reset, Receive Watchdog Timeout Interrupt is disabled.
8 (R/W)	RSE	Receive Stopped Enable. The EMAC_DMA2_IEN.RSE bit, when set (and with EMAC_DMA2_IEN.AIE =1), enables the Receive Stopped Interrupt is enabled. When this bit is reset, Receive Stopped Interrupt is disabled.
7 (R/W)	RUE	Receive Buffer Unavailable Enable. The EMAC_DMA2_IEN.RUE bit, when set (and with EMAC_DMA2_IEN.AIE =1), enables the Receive Buffer Unavailable Interrupt. When this bit is reset, the Receive Buffer Unavailable Interrupt is disabled.
6 (R/W)	RIE	Receive Interrupt Enable. The EMAC_DMA2_IEN.RIE bit, when set (and with EMAC_DMA2_IEN.NIE =1), enables the Receive Interrupt. When this bit is reset, Receive Interrupt is disabled.
5 (R/W)	UNE	Underflow Interrupt Enable. The EMAC_DMA2_IEN.UNE bit, when set (and with EMAC_DMA2_IEN.AIE =1), enables the Transmit Underflow Interrupt. When this bit is reset, Underflow Interrupt is disabled.
4 (R/W)	OVE	Overflow Interrupt Enable. The EMAC_DMA2_IEN.OVE bit, when set (and with EMAC_DMA2_IEN.AIE =1), enables the Receive Overflow Interrupt. When this bit is reset, Overflow Interrupt is disabled.

Table 24-106: EMAC_DMA2_IEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	TJE	Transmit Jabber Timeout Enable. The EMAC_DMA2_IEN.TJE bit, when set (and with EMAC_DMA2_IEN.AIE =1), enables the Transmit Jabber Timeout Interrupt. When this bit is reset, Transmit Jabber Timeout Interrupt is disabled.
2 (R/W)	TUE	Transmit Buffer Unavailable Enable. The EMAC_DMA2_IEN.TUE bit, when set (and with EMAC_DMA2_IEN.NIE =1), enables the Transmit Buffer Unavailable Interrupt. When this bit is reset, Transmit Buffer Unavailable Interrupt is disabled.
1 (R/W)	TSE	Transmit Stopped Enable. The EMAC_DMA2_IEN.TSE bit, when set (and with EMAC_DMA2_IEN.AIE =1), enables the Transmission Stopped Interrupt. When this bit is reset, Transmission Stopped Interrupt is disabled.
0 (R/W)	TIE	Transmit Interrupt Enable. The EMAC_DMA2_IEN.TIE bit, when set (and with EMAC_DMA2_IEN.NIE =1), enables the Transmit Interrupt. When this bit is reset, Transmit Interrupt is disabled.

DMA Missed Frame Register

The `EMAC_DMA2_MISS_FRM` register contains counters for EMAC DMA missed frames and buffer overflows.

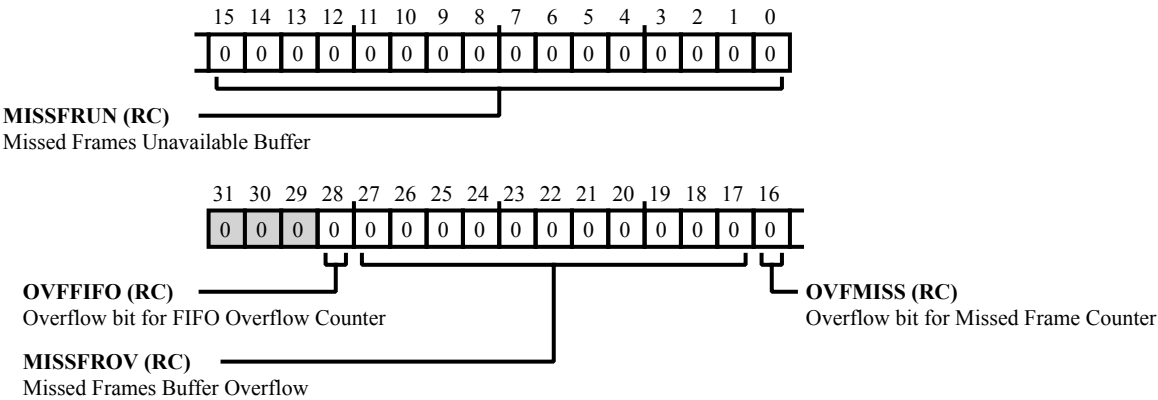


Figure 24-75: `EMAC_DMA2_MISS_FRM` Register Diagram

Table 24-107: `EMAC_DMA2_MISS_FRM` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (RC/NW)	OVFFIFO	Overflow bit for FIFO Overflow Counter. The <code>EMAC_DMA2_MISS_FRM.OVFFIFO</code> bit holds the overflow bit for FIFO Overflow Counter.
27:17 (RC/NW)	MISSFROV	Missed Frames Buffer Overflow. The <code>EMAC_DMA2_MISS_FRM.MISSFROV</code> bits indicate the number of frames missed by the application due to buffer overflow.
16 (RC/NW)	OVFMIS	Overflow bit for Missed Frame Counter. The <code>EMAC_DMA2_MISS_FRM.OVFMIS</code> bit holds the overflow bit for the Missed Frame Counter.
15:0 (RC/NW)	MISSFRUN	Missed Frames Unavailable Buffer. The <code>EMAC_DMA2_MISS_FRM.MISSFRUN</code> bits indicate the number of frames missed by the controller because of the Application Receive Buffer being unavailable.

DMA Operation Mode Register

The `EMAC_DMA2_OPMODE` register selects receive and transmit DMA operating modes.

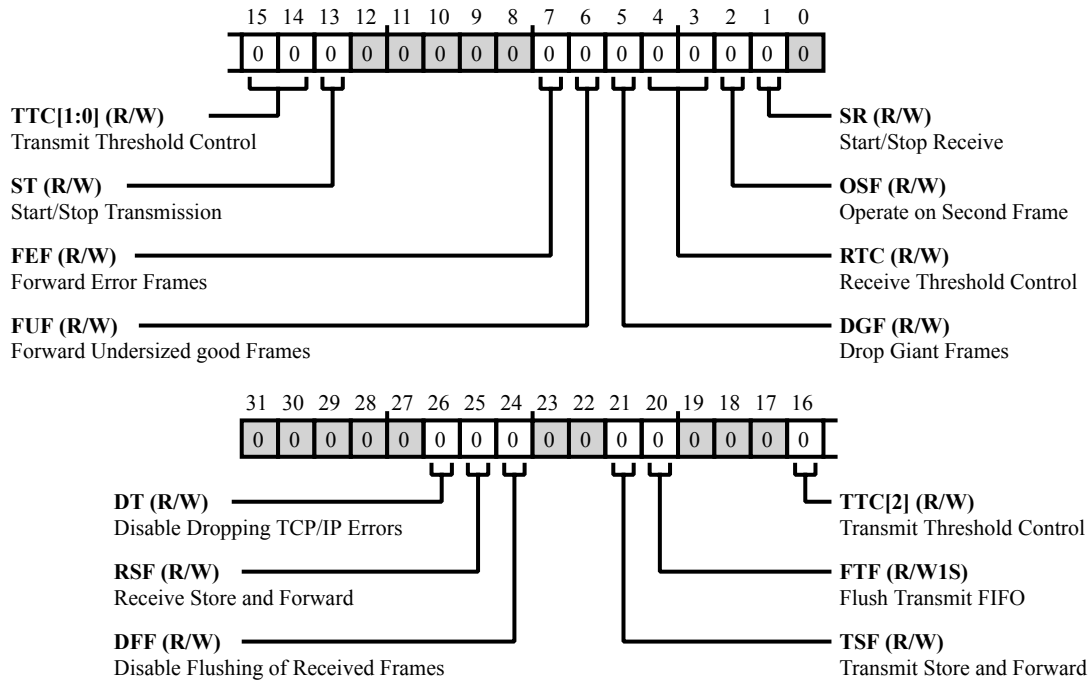


Figure 24-76: `EMAC_DMA2_OPMODE` Register Diagram

Table 24-108: `EMAC_DMA2_OPMODE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	DT	Disable Dropping TCP/IP Errors. The <code>EMAC_DMA2_OPMODE.DT</code> bit, when set, directs the core not to drop frames that only have errors detected by the Receive Checksum Offload engine. Such frames do not have any errors (including FCS error) in the Ethernet frame received by the MAC but have errors in the encapsulated payload only. When this bit is reset, all error frames are dropped if the <code>EMAC_DMA2_OPMODE.FEF</code> bit is reset.
25 (R/W)	RSF	Receive Store and Forward. The <code>EMAC_DMA2_OPMODE.RSF</code> bit, when set, directs the MFL only to read a frame from the Rx FIFO after the complete frame has been written to it, ignoring the <code>EMAC_DMA2_OPMODE.RTC</code> bits. When this bit is reset, the Rx FIFO operates in threshold mode, subject to the threshold specified by the <code>EMAC_DMA2_OPMODE.RTC</code> bits.
24 (R/W)	DFF	Disable Flushing of Received Frames. The <code>EMAC_DMA2_OPMODE.DFF</code> bit, when set, directs the Rx DMA not to flush any frames because of the unavailability of receive descriptors/buffers as it does normally when this bit is reset.

Table 24-108: EMAC_DMA2_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration																
21 (R/W)	TSF	Transmit Store and Forward. The EMAC_DMA2_OPMODE . TSF bit, when set, starts transmission when a full frame resides in the MFL Transmit FIFO. When this bit is set, the TTC values specified in Register 6[16:14] are ignored. This bit should be changed only when transmission is stopped.																
20 (R/W1S)	FTF	Flush Transmit FIFO. The EMAC_DMA2_OPMODE . FTF bit, when set, directs the transmit FIFO controller logic to reset to its default values and thus all data in the Tx FIFO is lost/flushed. This bit is cleared internally when the flushing operation is completed fully. The Operation Mode register should not be written to until this bit is cleared. The data which is already accepted by the MAC transmitter is not flushed. It is scheduled for transmission and results in underflow and runt frame transmission. Note: The flush operation completes only after emptying the Tx FIFO of its contents and all the pending Transmit Status of the transmitted frames are accepted by the host. In order to complete this flush operation, the PHY transmit clock is required to be active. This field cleared to 1b0 by the core (Self Clear). The application cannot clear this type of field, and a register write of 1b0 to this bit has no effect on this field.																
16:14 (R/W)	TTC	Transmit Threshold Control. The EMAC_DMA2_OPMODE . TTC bits control the threshold level of the MFL Transmit FIFO. Transmission starts when the frame size within the MFL Transmit FIFO is larger than the threshold. In addition, full frames with a length less than the threshold are also transmitted. These bits are used only when the EMAC_DMA2_OPMODE . TSF bit is reset. The value =011 is not used. <table><tr><td>0</td><td>64</td></tr><tr><td>1</td><td>128</td></tr><tr><td>2</td><td>192</td></tr><tr><td>3</td><td>256</td></tr><tr><td>4</td><td>40</td></tr><tr><td>5</td><td>32</td></tr><tr><td>6</td><td>24</td></tr><tr><td>7</td><td>16</td></tr></table>	0	64	1	128	2	192	3	256	4	40	5	32	6	24	7	16
0	64																	
1	128																	
2	192																	
3	256																	
4	40																	
5	32																	
6	24																	
7	16																	

Table 24-108: EMAC_DMA2_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	ST	Start/Stop Transmission. The EMAC_DMA2_OPMODE.ST bit, when set, places transmission in the Running state, and the DMA checks the Transmit List at the current position for a frame to be transmitted. Descriptor acquisition is attempted either from the current position in the list, which is the Transmit List Base Address set by Transmit Descriptor List Address, or from the position retained when transmission was stopped previously. If the current descriptor is not owned by the DMA, transmission enters the Suspended state, and the EMAC_DMA2_STAT.TU bit is set. The Start Transmission command is effective only when transmission is stopped. If the command is issued before setting the EMAC_DMA2_TXDSC_CUR address register, then the DMA behavior is unpredictable. When this bit is reset, the transmission process is placed in the Stopped state after completing the transmission of the current frame. The Next Descriptor position in the Transmit List is saved, and becomes the current position when transmission is restarted. The stop transmission command is effective only when the transmission of the current frame is complete or the transmission is in the Suspended state.
7 (R/W)	FEF	Forward Error Frames. The EMAC_DMA2_OPMODE.FEF bit, when reset, directs the Rx FIFO to drop frames with error status (CRC error, collision error, giant frame, watchdog timeout, overflow). However, if the frames start byte (write) pointer is already transferred to the read controller side (in Threshold mode), then the frames are not dropped. When EMAC_DMA2_OPMODE.FEF bit is set, all frames except runt error frames are forwarded to the DMA. But when Rx FIFO overflows when a partial frame is written, then such frames are dropped even when EMAC_DMA2_OPMODE.FEF is set.
6 (R/W)	FUF	Forward Undersized good Frames. The EMAC_DMA2_OPMODE.FUF bit, when set, directs the Rx FIFO to forward Undersized frames (frames with no Error and length less than 64 bytes) including pad-bytes and CRC). When reset, the Rx FIFO drops all frames of less than 64 bytes, unless it is already transferred because of lower value of Receive Threshold (for example, EMAC_DMA2_OPMODE.RTC = 01).
5 (R/W)	DGF	Drop Giant Frames. The EMAC_DMA2_OPMODE.DGF bit, when set, the MAC drops the received giant frames in the Rx FIFO, that is, frames that are larger than the computed giant frame limit. When reset, the MAC does not drop the giant frames in the Rx FIFO.

Table 24-108: EMAC_DMA2_OPMODE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:3 (R/W)	RTC	Receive Threshold Control. The <code>EMAC_DMA2_OPMODE.RTC</code> bits control the threshold level of the MFL Receive FIFO. Transfer (request) to DMA starts when the frame size within the MFL Receive FIFO is larger than the threshold. In addition, full frames with a length less than the threshold are transferred automatically. These bits are valid only when the <code>EMAC_DMA2_OPMODE.RSF</code> bit is zero, and are ignored when the <code>EMAC_DMA2_OPMODE.RSF</code> bit is set to 1. The value =11 is not used.
		0 64
		1 32
		2 96
		3 128
2 (R/W)	OSF	Operate on Second Frame. The <code>EMAC_DMA2_OPMODE.OSF</code> bit, when set, instructs the DMA to process a second frame of Transmit data even before status for first frame is obtained.
1 (R/W)	SR	Start/Stop Receive. The <code>EMAC_DMA2_OPMODE.SR</code> bit, when set, places the Receive process in the Running state. The DMA attempts to acquire the descriptor from the Receive list and processes incoming frames. Descriptor acquisition is attempted from the current position in the list, which is the address set by DMA Receive Descriptor List Address or the position retained when the Receive process was previously stopped. If no descriptor is owned by the DMA, reception is suspended, and the <code>EMAC_DMA2_STAT.RU</code> bit is set. The Start Receive command is effective only when reception has stopped. If the command was issued before setting <code>EMAC_DMA2_RXDSC_CUR</code> address register, DMA behavior is unpredictable. When this bit is cleared, Rx DMA operation is stopped after the transfer of the current frame. The next descriptor position in the Receive list is saved and becomes the current position after the Receive process is restarted. The Stop Receive command is effective only when the Receive process is in either the Running (waiting for receive packet) or in the Suspended state.

DMA Rx Buffer Current Register

The `EMAC_DMA2_RXBUF_CUR` register holds the pointer to the current receive DMA buffer.

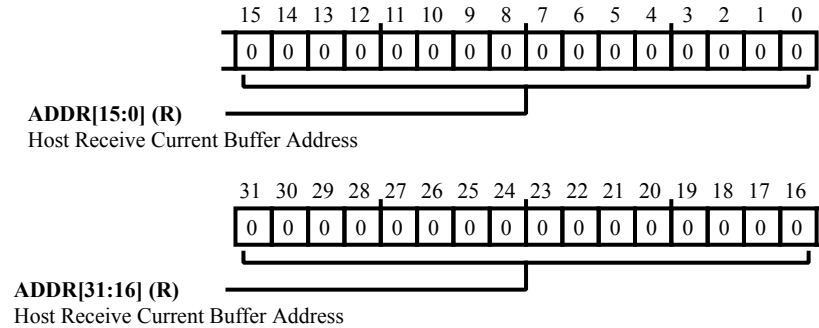


Figure 24-77: EMAC_DMA2_RXBUF_CUR Register Diagram

Table 24-109: EMAC_DMA2_RXBUF_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Receive Current Buffer Address. The <code>EMAC_DMA2_RXBUF_CUR.ADDR</code> bit field points to the current Receive Buffer address being read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Rx Descriptor List Address Register

The `EMAC_DMA2_RXDSC_ADDR` register holds the address for the DMA receive descriptor list. Writing to this Register is permitted only when reception is stopped. When stopped, this must be written to before the receive Start command is given. The processor can write to `EMAC_DMA2_RXDSC_ADDR` only when Rx DMA has stopped (`EMAC_DMA2_OPMODE.SR` bit =0). When stopped, it can be written with a new descriptor list address. When the processor sets the `EMAC_DMA2_OPMODE.SR` bit to 1, the DMA takes the newly programmed descriptor base address. If this register is not changed when the `EMAC_DMA2_OPMODE.SR` bit is cleared to 0, the DMA takes the descriptor address where it was stopped earlier.

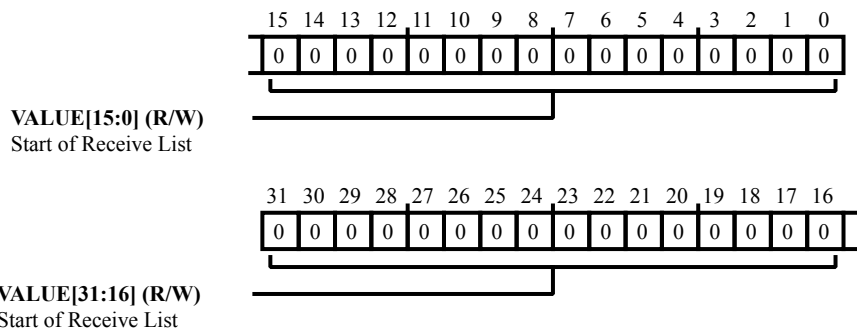


Figure 24-78: `EMAC_DMA2_RXDSC_ADDR` Register Diagram

Table 24-110: `EMAC_DMA2_RXDSC_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Start of Receive List. The <code>EMAC_DMA2_RXDSC_ADDR.VALUE</code> bit field contains the base address of the First Descriptor in the Receive Descriptor list. The LSB bits [1:0] for the 32bit bus width are ignored and are taken as all-zero by the DMA internally. Therefore, these LSB bits are Read-Only (RO).

DMA Rx Descriptor Current Register

The `EMAC_DMA2_RXDSC_CUR` register contains the current DMA receive descriptor.

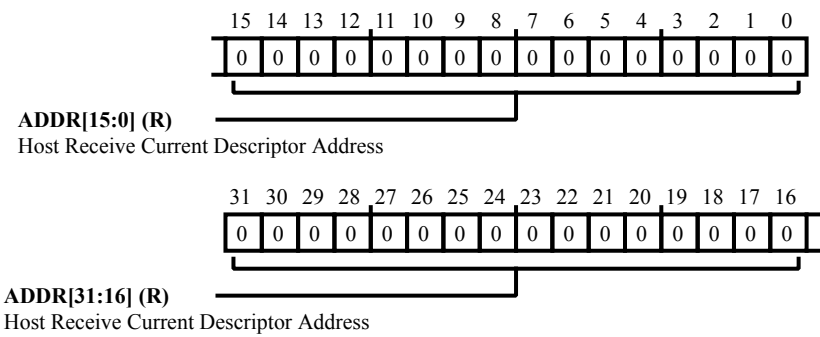


Figure 24-79: EMAC_DMA2_RXDSC_CUR Register Diagram

Table 24-111: EMAC_DMA2_RXDSC_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Receive Current Descriptor Address. The <code>EMAC_DMA2_RXDSC_CUR.ADDR</code> bit field points to the start address of the current Receive Descriptor read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Rx Interrupt Watch Dog Register

The `EMAC_DMA2_RXIWDOG` register contains the timeout value for the EMAC DMA receive interrupt watch dog timer.

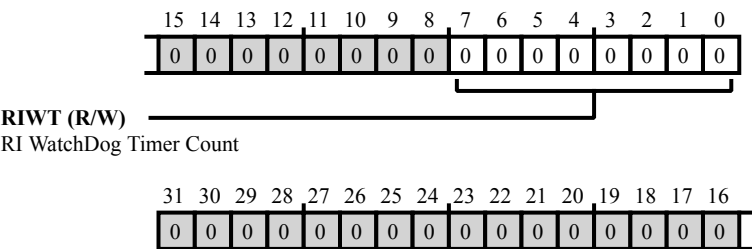


Figure 24-80: EMAC_DMA2_RXIWDOG Register Diagram

Table 24-112: EMAC_DMA2_RXIWDOG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	RIWT	RI WatchDog Timer Count. The <code>EMAC_DMA2_RXIWDOG.RIWT</code> bit field indicates the number of system clock cycles multiplied by 256 for which the watchdog timer is set. The watchdog timer gets triggered with the programmed value after the Rx DMA completes the transfer of a frame for which the RI status bit is not set because of the setting in the corresponding descriptor <code>RDES1[31]</code>. When the watch-dog timer runs out, the RI bit is set and the timer is stopped. The watchdog timer is reset when <code>EMAC_DMA2_STAT.RI</code> bit is set high because of automatic setting of <code>EMAC_DMA2_STAT.RI</code> as per <code>RDES1[31]</code> of any received frame.

DMA Rx Poll Demand register

The `EMAC_DMA2_RXPOLL` register directs the EMAC to poll the receive descriptor list.

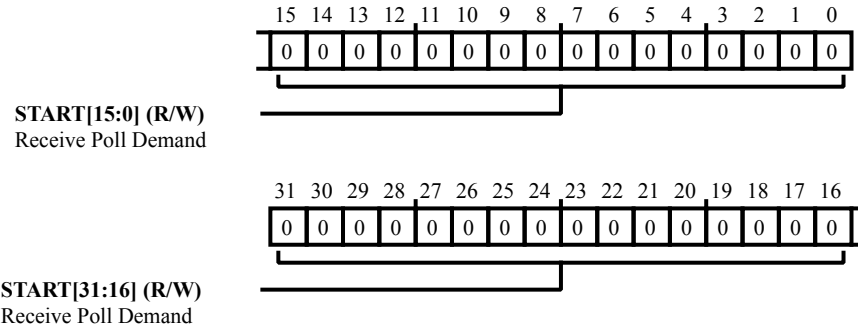


Figure 24-81: EMAC_DMA2_RXPOLL Register Diagram

Table 24-113: EMAC_DMA2_RXPOLL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	START	Receive Poll Demand. The <code>EMAC_DMA2_RXPOLL.START</code> bits, when written with any value, cause the DMA to read the current descriptor pointed to by the <code>EMAC_DMA2_RXDSC_CUR</code> register. If that descriptor is not available (owned by application), reception returns to the Suspended state, and the <code>EMAC_DMA2_STAT.RU</code> bit is asserted. If the descriptor is available, the Receive DMA returns to the active state.

DMA Status Register

The `EMAC_DMA2_STAT` register indicates EMAC DMA status.

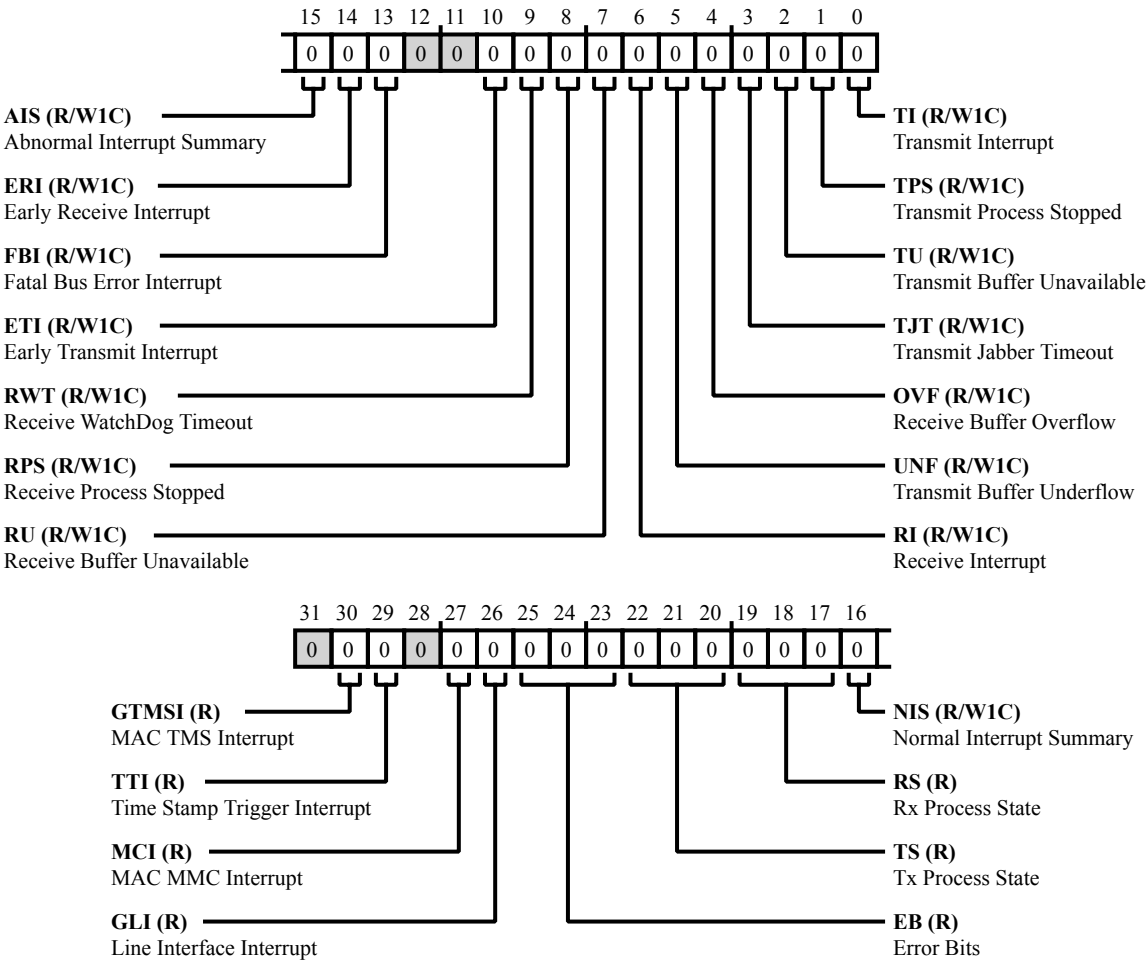


Figure 24-82: EMAC_DMA2_STAT Register Diagram

Table 24-114: EMAC_DMA2_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/NW)	GTMSI	MAC TMS Interrupt. The <code>EMAC_DMA2_STAT.GTMSI</code> bit indicates an interrupt event in the traffic manager and scheduler logic. To reset this bit, the software must read the corresponding registers (Channel Status Register) to get the exact cause of the interrupt and clear its source.

Table 24-114: EMAC_DMA2_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/NW)	TTI	Time Stamp Trigger Interrupt. The EMAC_DMA2_STAT.TTI bit indicates an interrupt event in the MAC core's Time Stamp Generator block. The software must read the corresponding registers in the MAC core to get the exact cause of interrupt and clear its source to reset this bit to =0. When this bit is high, the interrupt signal from the MAC is high.
27 (R/NW)	MCI	MAC MMC Interrupt. The EMAC_DMA2_STAT.MCI bit reflects an interrupt event in the MMC module of the MAC core. The software must read the corresponding registers in the MAC core to get the exact cause of interrupt and clear the source of interrupt to make this bit as =0. The interrupt signal from the MAC is high when this bit is high.
26 (R/NW)	GLI	Line Interface Interrupt. The EMAC_DMA2_STAT.GLI bit When set, this bit reflects any of the following interrupt events in the DWC_gmac interfaces
25:23 (R/NW)	EB	Error Bits. The EMAC_DMA2_STAT.EB bits indicate the type of error that caused a Bus Error (for example, error response on the SCB interface). These bits are valid only when the EMAC_DMA2_STAT.FBI bit is set. This field does not generate an interrupt.
		0 Error during data buffer access, write transfer, Rx DMA
		1 Error during data buffer access, write transfer, Tx DMA
		2 Error during data buffer access, read transfer, Rx DMA
		3 Error during data buffer access, read transfer, Tx DMA
		4 Error during descriptor access, write transfer, Rx DMA
		5 Error during descriptor access, write transfer, Tx DMA
		6 Error during descriptor access, read transfer, Rx DMA
		7 Error during descriptor access, read transfer, Tx DMA
22:20 (R/NW)	TS	Tx Process State. The EMAC_DMA2_STAT.TS bits indicate the transmit DMA state. This field does not generate an interrupt.
		0 Stopped; Reset or Stop Tx Command Issued
		1 Running; Fetching Tx Transfer Descriptor
		2 Running; Waiting for Status
		3 Reading Data from Host Memory Buffer and Queuing It to Tx Buffer
		4 TIME_STAMP Write State

Table 24-114: EMAC_DMA2_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		5	Reserved
		6	Suspended; Tx Descriptor Unavailable or Tx Buffer Underflow
		7	Closing Tx Descriptor
19:17 (R/NW)	RS	Rx Process State. The EMAC_DMA2_STAT.RS bits indicate the receive DMA state. This field does not generate an interrupt.	
		0	Stopped: Reset or Stop Rx Command Issued.
		1	Running: Fetching Rx Transfer Descriptor.
		2	Reserved
		3	Running: Waiting for Rx Packet
		4	Suspended: Rx Descriptor Unavailable
		5	Running: Closing Rx Descriptor
		6	TIME_STAMP Write State
		7	Running: Transferring Rx Packet Data from Rx Buffer to Host Memory
16 (R/W1C)	NIS	Normal Interrupt Summary. The value of the EMAC_DMA2_STAT.NIS bit field is the logical OR of the following when the corresponding interrupt bits are enabled in DMA Interrupt Enable Register: EMAC_DMA2_STAT.TI, EMAC_DMA2_STAT.TU, EMAC_DMA2_STAT.RI, and EMAC_DMA2_STAT.ERI. Only unmasked bits affect the Normal Interrupt Summary bit. This is a sticky bit and must be cleared (by writing a 1 to this bit) each time a corresponding bit that causes EMAC_DMA2_STAT.NIS to be set is cleared.	
15 (R/W1C)	AIS	Abnormal Interrupt Summary. The value of the EMAC_DMA2_STAT.AIS bit field is the logical OR of the following when the corresponding interrupt bits are enabled in DMA Interrupt Enable Register: EMAC_DMA2_IEN.TSE, EMAC_DMA2_IEN.TJE, EMAC_DMA2_IEN.OVE, EMAC_DMA2_IEN.UNE, EMAC_DMA2_IEN.RUE, EMAC_DMA2_IEN.RSE, EMAC_DMA2_IEN.RWE, EMAC_DMA2_IEN.ETE, and EMAC_DMA2_IEN.FBE. Only unmasked bits affect the Abnormal Interrupt Summary bit. This is a sticky bit and must be cleared each time a corresponding bit that causes EMAC_DMA2_STAT.AIS to be set is cleared.	
14 (R/W1C)	ERI	Early Receive Interrupt. The EMAC_DMA2_STAT.ERI bit indicates that the DMA had filled the first data buffer of the packet. The EMAC_DMA2_STAT.RI bit automatically clears this bit.	

Table 24-114: EMAC_DMA2_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W1C)	FBI	Fatal Bus Error Interrupt. The EMAC_DMA2_STAT.FBI bit indicates that a bus error occurred, as detailed in the EMAC_DMA2_STAT.EB field. When this bit is set, the corresponding DMA engine disables all its bus accesses.
10 (R/W1C)	ETI	Early Transmit Interrupt. The EMAC_DMA2_STAT.ETI bit indicates that the frame to be transmitted was fully transferred to the MFL Transmit FIFO.
9 (R/W1C)	RWT	Receive WatchDog Timeout. The EMAC_DMA2_STAT.RWT bit is asserted when a frame with a length greater than 2,048 bytes is received (10, 240 when Jumbo Frame mode is enabled).
8 (R/W1C)	RPS	Receive Process Stopped. The EMAC_DMA2_STAT.RPS bit is asserted when the Receive Process enters the Stopped state.
7 (R/W1C)	RU	Receive Buffer Unavailable. The EMAC_DMA2_STAT.RU bit indicates that the Next Descriptor in the Receive List is owned by the application and cannot be acquired by the DMA. Receive Process is suspended. To resume processing Receive descriptors, the application should change the ownership of the descriptor and issue a Receive Poll Demand command. If no Receive Poll Demand is issued, Receive Process resumes when the next recognized incoming frame is received. This bit is set only when the previous Receive Descriptor was owned by the DMA.
6 (R/W1C)	RI	Receive Interrupt. The EMAC_DMA2_STAT.RI bit indicates the completion of frame reception. Specific frame status information has been posted in the descriptor. Reception remains in the Running state.
5 (R/W1C)	UNF	Transmit Buffer Underflow. The EMAC_DMA2_STAT.UNF bit indicates that the Transmit Buffer had an Underflow during frame transmission. Transmission is suspended and an Underflow Error TDES0[1] is set.
4 (R/W1C)	OVF	Receive Buffer Overflow. The EMAC_DMA2_STAT.OVF bit indicates that the Receive Buffer had an Overflow during frame reception. If the partial frame is transferred to application, the overflow status is set in RDES0[11].
3 (R/W1C)	TJT	Transmit Jabber Timeout. The EMAC_DMA2_STAT.TJT bit indicates that the Transmit Jabber Timer expired, meaning that the transmitter had been excessively active. The transmission process is aborted and placed in the Stopped state. This causes the Transmit Jabber Timeout TDES0[14] flag to assert.

Table 24-114: EMAC_DMA2_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W1C)	TU	Transmit Buffer Unavailable. The EMAC_DMA2_STAT.TU bit indicates that the Next Descriptor in the Transmit List is owned by the application and cannot be acquired by the DMA. Transmission is suspended. The value in the EMAC_DMA2_STAT.TS bits explain the Transmit Process state transitions. To resume processing transmit descriptors, the application should change the ownership of the bit of the descriptor and then issue a Transmit Poll Demand command.
1 (R/W1C)	TPS	Transmit Process Stopped. The EMAC_DMA2_STAT.TPS bit is set when the transmission is stopped.
0 (R/W1C)	TI	Transmit Interrupt. The EMAC_DMA2_STAT.TI bit indicates that frame transmission is finished and TDES1[31] is set in the First Descriptor.

DMA Tx Buffer Current Register

The `EMAC_DMA2_TXBUF_CUR` register holds the pointer to the current transmit DMA buffer.

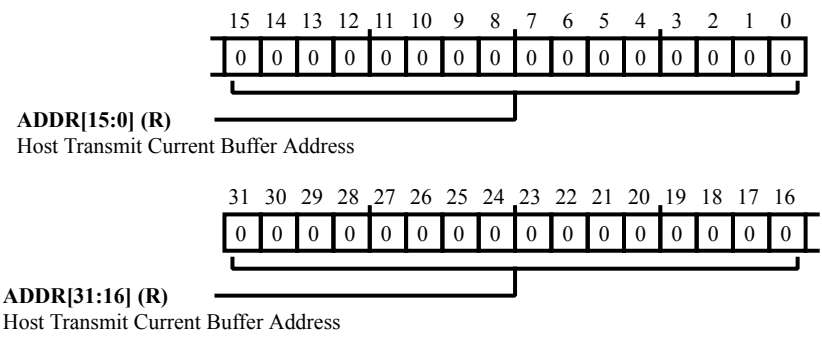


Figure 24-83: EMAC_DMA2_TXBUF_CUR Register Diagram

Table 24-115: EMAC_DMA2_TXBUF_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Transmit Current Buffer Address. The <code>EMAC_DMA2_TXBUF_CUR.ADDR</code> bit field points to the current Transmit Buffer Address being read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Tx Descriptor List Address Register

The `EMAC_DMA2_TXDSC_ADDR` register holds the address for the DMA transmit descriptor list. The processor can write to this Register only when Tx DMA has stopped (`EMAC_DMA2_OPMODE.ST` bit =0). When stopped, this can be written with a new descriptor list address. When the processor sets the `EMAC_DMA2_OPMODE.ST` bit to 1, the DMA takes the newly programmed descriptor base address. If this register is not changed when the `EMAC_DMA2_OPMODE.ST` bit is cleared to 0, then the DMA takes the descriptor address where it was stopped earlier.

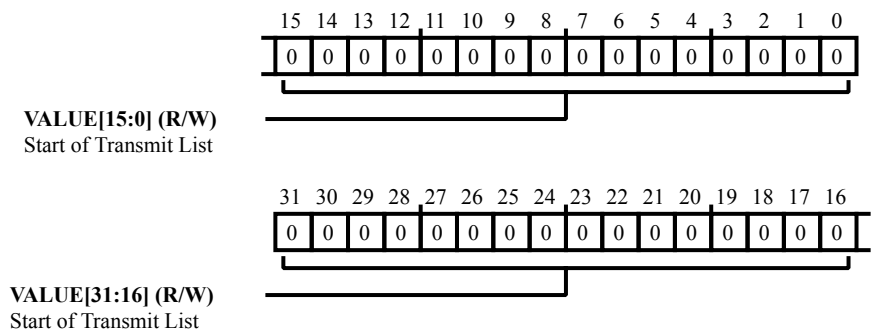


Figure 24-84: EMAC_DMA2_TXDSC_ADDR Register Diagram

Table 24-116: EMAC_DMA2_TXDSC_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Start of Transmit List. The <code>EMAC_DMA2_TXDSC_ADDR.VALUE</code> bit field contains the base address of the First Descriptor in the Transmit Descriptor list. The LSB bits [1:0] for 32bit bus width are ignored and are taken as all-zero by the DMA internally. Therefore, these LSB bits are Read-Only (RO).

DMA Tx Descriptor Current Register

The `EMAC_DMA2_TXDSC_CUR` register contains the current DMA transmit descriptor.

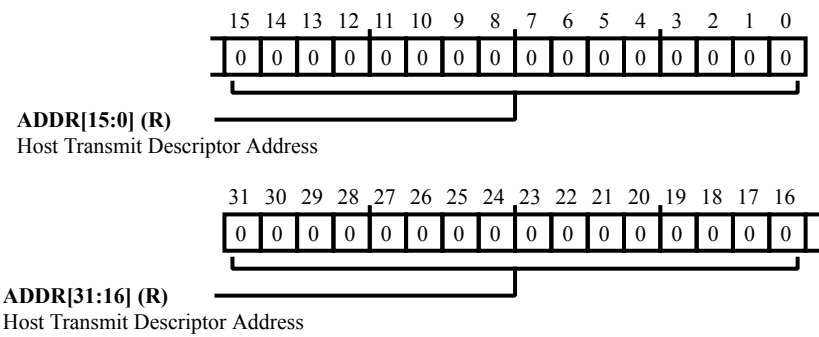


Figure 24-85: `EMAC_DMA2_TXDSC_CUR` Register Diagram

Table 24-117: `EMAC_DMA2_TXDSC_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	ADDR	Host Transmit Descriptor Address. The <code>EMAC_DMA2_TXDSC_CUR.ADDR</code> bit field points to the start address of the current Transmit Descriptor read by the DMA. Pointer updated by DMA during operation. Cleared on Reset.

DMA Tx Poll Demand Register

The `EMAC_DMA2_TXPOLL` register directs the EMAC to poll the transmit descriptor list.

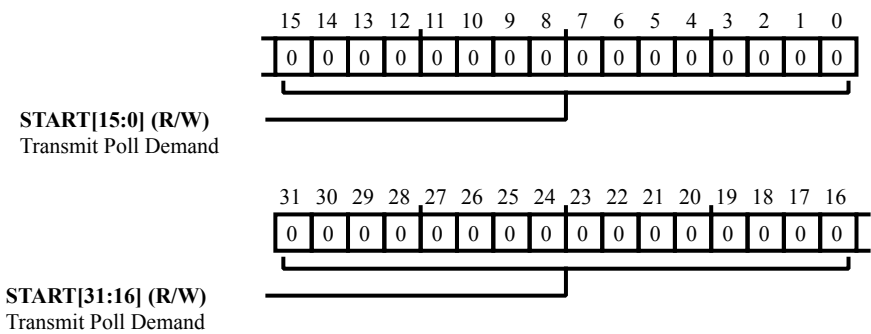


Figure 24-86: `EMAC_DMA2_TXPOLL` Register Diagram

Table 24-118: `EMAC_DMA2_TXPOLL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	START	Transmit Poll Demand. The <code>EMAC_DMA2_TXPOLL.START</code> bits, when written with any value, cause the DMA to read the current descriptor pointed to by <code>EMAC_DMA2_TXDSC_CUR</code> register. If that descriptor is not available (owned by application), transmission returns to the Suspend state, and the <code>EMAC_DMA2_STAT.TU</code> bit is asserted. If the descriptor is available, transmission resumes.

Flow Control Register

The `EMAC_FLOWCTL` register controls EMAC flow control features.

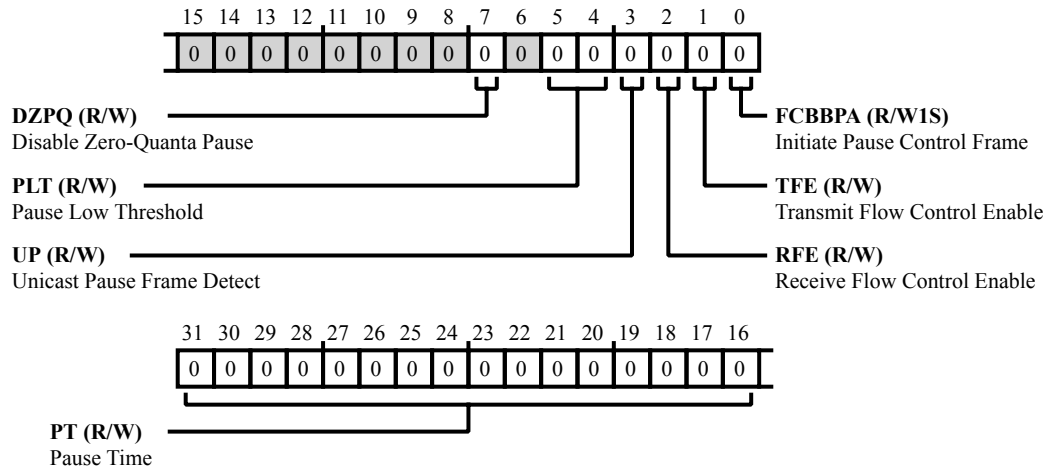


Figure 24-87: EMAC_FLOWCTL Register Diagram

Table 24-119: EMAC_FLOWCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	PT	Pause Time. The <code>EMAC_FLOWCTL.PT</code> bits hold the value to be used in the Pause Time field in the transmit control frame.
7 (R/W)	DZPQ	Disable Zero-Quanta Pause. The <code>EMAC_FLOWCTL.DZPQ</code> bit disables the automatic generation of the Zero-Quanta Pause frames on the de-assertion of the flow-control signal from the FIFO layer.
5:4 (R/W)	PLT	Pause Low Threshold. The <code>EMAC_FLOWCTL.PLT</code> bit configures the threshold of the Pause timer at which the input flow control signal <code>mti_flowctrl_i</code> (or <code>sbd_flowctrl_i</code>) is checked for automatic retransmission of the Pause frame.
3 (R/W)	UP	Unicast Pause Frame Detect. The <code>EMAC_FLOWCTL.UP</code> bit, when set, directs the MAC to detect the Pause frames with the station's unicast address specified in <code>EMAC_ADDR0_HI</code> and <code>EMAC_ADDR0_LO</code> address registers. This bit also directs the MAC to the detect Pause frames with the unique multicast address. When this bit is reset, the MAC will detect only a Pause frame with the unique multicast address specified in the 802.3x standard.

Table 24-119: EMAC_FLOWCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	RFE	Receive Flow Control Enable. The <code>EMAC_FLOWCTL.RFE</code> bit, when set, directs the MAC to decode the received Pause frame and disable its transmitter for a specified (Pause Time) time. When this bit is reset, the decode function of the Pause frame is disabled.
1 (R/W)	TFE	Transmit Flow Control Enable. In Full-Duplex mode, when the <code>EMAC_FLOWCTL.TFE</code> bit is set, the MAC enables the flow control operation to transmit Pause frames. When this bit is reset, the flow control operation in the MAC is disabled, and the MAC does not transmit any Pause frames. In Half-Duplex mode, when this bit is set, the MAC enables the back pressure operation. When this bit is reset, the back pressure feature is disabled.
0 (R/W1S)	FCBBPA	Initiate Pause Control Frame. The <code>EMAC_FLOWCTL.FCBBPA</code> bit initiates a Pause Control frame in Full-Duplex mode and activates the back pressure function in Half-Duplex mode if TFE bit is set. In Full-Duplex mode, this bit should be read as =0 before writing to the <code>EMAC_FLOWCTL</code> register. To initiate a Pause control frame, the Application must set this bit to =1. During a transfer of the Control Frame, this bit continues to be set to signify that a frame transmission is in progress. After the completion of Pause control frame transmission, the MAC resets this bit to =0. The <code>EMAC_FLOWCTL</code> register should not be written to until this bit is cleared. In Half-Duplex mode, when this bit is set (and <code>EMAC_FLOWCTL.TFE</code> is set), the back pressure is asserted by the MAC Core. During back pressure, when the MAC receives a new frame, the transmitter starts sending a JAM pattern resulting in a collision. The <code>EMAC_FLOWCTL.FCBBPA</code> bit is logically OR'ed with the flow control input signal for the back pressure function. When the MAC is configured to Full-Duplex mode, the back pressure function is automatically disabled.

RGMI Control and Status Register

The `EMAC_GIGE_CTLSTAT` register indicates the status signals received from the PHY through the SGMII, RGMII, or SMII interface.

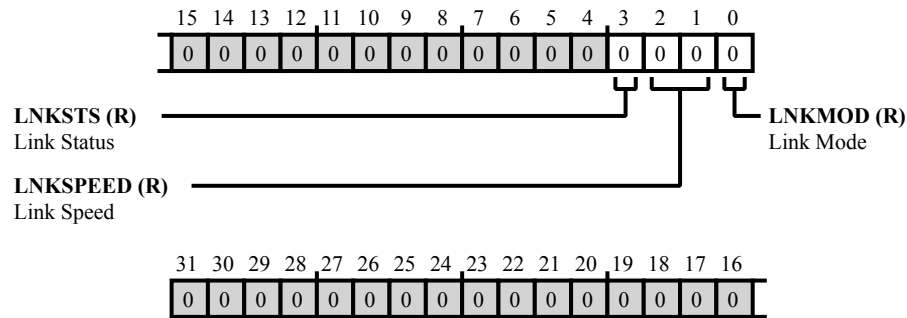


Figure 24-88: EMAC_GIGE_CTLSTAT Register Diagram

Table 24-120: EMAC_GIGE_CTLSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/NW)	LNKSTS	Link Status. The <code>EMAC_GIGE_CTLSTAT.LNKSTS</code> bit indicates that the link is up between the local PHY and the remote PHY. When cleared, this bit indicates that the link is down between the local PHY and the remote PHY.
2:1 (R/NW)	LNKSPED	Link Speed. The <code>EMAC_GIGE_CTLSTAT.LNKSPED</code> bit indicates the current speed of the link.
0 (R/NW)	LNKMOD	Link Mode. The <code>EMAC_GIGE_CTLSTAT.LNKMOD</code> bit indicates whether the current mode of link operation is half duplex or full duplex.

Hash Table High Register

The `EMAC_HASHTBL_HI` register contains the upper 32 bits of the hash table.

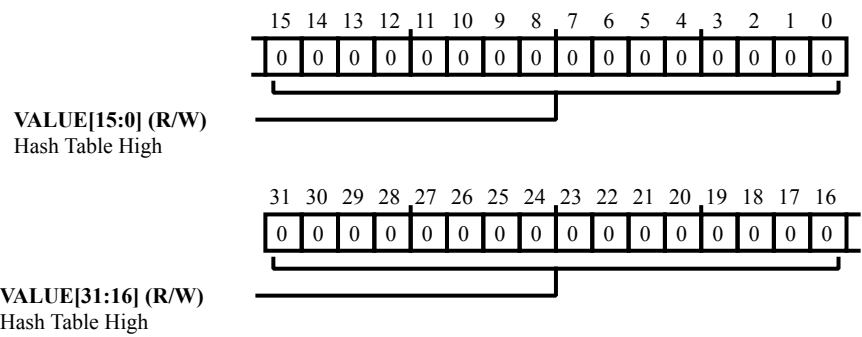


Figure 24-89: EMAC_HASHTBL_HI Register Diagram

Table 24-121: EMAC_HASHTBL_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Hash Table High. The <code>EMAC_HASHTBL_HI.VALUE</code> bits contain the upper 32 bits of Hash table.

Hash Table Low Register

The `EMAC_HASHTBL_LO` register contains the lower 32 bits of the hash table.

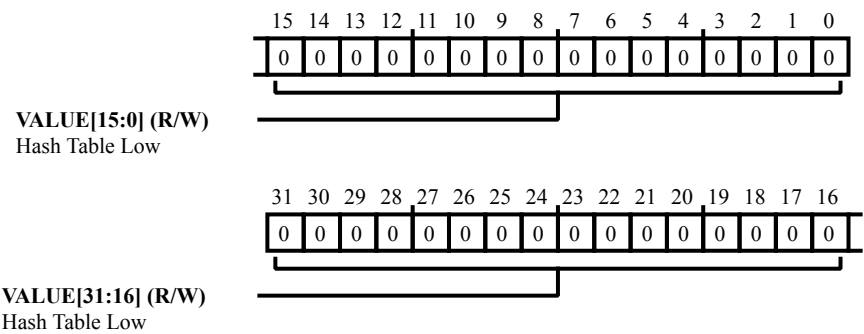


Figure 24-90: EMAC_HASHTBL_LO Register Diagram

Table 24-122: EMAC_HASHTBL_LO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Hash Table Low. The <code>EMAC_HASHTBL_LO.VALUE</code> bits contain the lower 32 bits of Hash table.

Interrupt Mask Register

The `EMAC_IMSK` register enables (unmasks) EMAC interrupts.

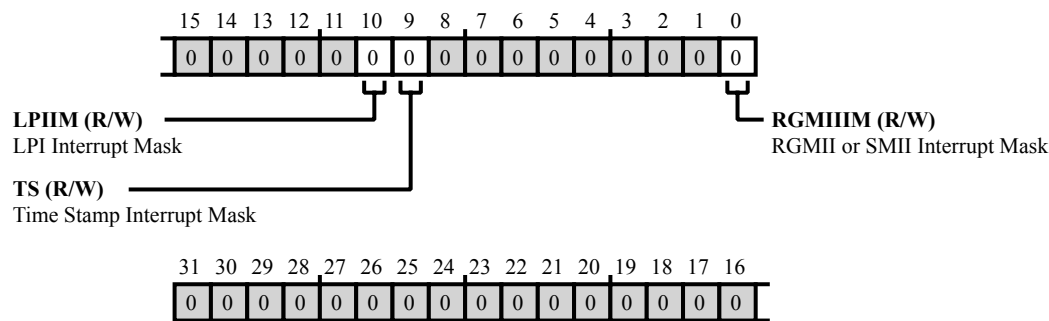


Figure 24-91: EMAC_IMSK Register Diagram

Table 24-123: EMAC_IMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	LPIIM	LPI Interrupt Mask. The <code>EMAC_IMSK.LPIIM</code> bit, When set, disables the assertion of the interrupt signal because of the setting of the LPI Interrupt Status bit in Interrupt Status Register.
9 (R/W)	TS	Time Stamp Interrupt Mask. The <code>EMAC_IMSK.TS</code> bit, when set, disables the assertion of the interrupt signal, which is generated when the <code>EMAC_ISTAT.TS</code> bit is set.
0 (R/W)	RGMIIIM	RGMII or SMII Interrupt Mask. The <code>EMAC_IMSK.RGMIIIM</code> bit, When set, disables the assertion of the interrupt signal because of the setting of the RGMII Interrupt Status bit in Interrupt Status Register.

MMC IPC Rx Interrupt Mask Register

The `EMAC_IPC_RXIMSK` register enables (unmasks) MMC IPC receive interrupts.



Figure 24-92: EMAC_IPC_RXIMSK Register Diagram

Table 24-124: EMAC_IPC_RXIMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	ICMPERROCT	Rx ICMP Error Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.ICMPERROCT bit, when set, masks the interrupt when the EMAC_RXICMP_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
28 (R/W)	ICMPGOCT	Rx ICMP (Good) Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.ICMPGOCT bit, when set, masks the interrupt when the EMAC_RXICMP_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
27 (R/W)	TCPPERROCT	Rx TCP Error Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.TCPPERROCT bit, when set, masks the interrupt when the EMAC_RXTCP_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
26 (R/W)	TCPGOCT	Rx TCP (Good) Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.TCPGOCT bit, when set, masks the interrupt when the EMAC_RXTCP_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
25 (R/W)	UDPPERROCT	Rx UDP Error Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.UDPPERROCT bit, when set, masks the interrupt when the EMAC_RXUDP_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
24 (R/W)	UDPGOCT	Rx UDP (Good) Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.UDPGOCT bit, when set, masks the interrupt when the EMAC_RXUDP_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
23 (R/W)	V6NOPAYOCT	Rx IPv6 No Payload Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.V6NOPAYOCT bit, when set, masks the interrupt when the EMAC_RXIPV6_NOPAY_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
22 (R/W)	V6HDERROCT	Rx IPv6 Header Error Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.V6HDERROCT bit, when set, masks the interrupt when the EMAC_RXIPV6_HDR_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
21 (R/W)	V6GOCT	Rx IPv6 (Good) Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.V6GOCT bit, when set, masks the interrupt when the EMAC_RXIPV6_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.

Table 24-124: EMAC_IPC_RXIMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	V4UDSBLOCT	Rx IPv4 UDS Disable Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4UDSBLOCT bit, when set, masks the interrupt when the EMAC_RXIPV4_UDSBL_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
19 (R/W)	V4FRAGOCT	Rx IPv4 Fragmented Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4FRAGOCT bit, when set, masks the interrupt when the EMAC_RXIPV4_FRAG_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
18 (R/W)	V4NOPAYOCT	Rx IPv4 No Payload Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4NOPAYOCT bit, when set, masks the interrupt when the EMAC_RXIPV4_NOPAY_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
17 (R/W)	V4HDERROCT	Rx IPv4 Header Error Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4HDERROCT bit, when set, masks the interrupt when the EMAC_RXIPV4_HDR_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
16 (R/W)	V4GOCT	Rx IPv4 (Good) Octets Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4GOCT bit, when set, masks the interrupt when the EMAC_RXIPV4_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
13 (R/W)	ICMPERRFRM	Rx ICMP Error Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.ICMPERRFRM bit, when set, masks the interrupt when the EMAC_RXICMP_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
12 (R/W)	ICMPGFRM	Rx ICMP (Good) Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.ICMPGFRM bit, when set, masks the interrupt when the EMAC_RXICMP_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
11 (R/W)	TCPERRFRM	Rx TCP Error Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.TCPERRFRM bit, when set, masks the interrupt when the EMAC_RXTCP_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
10 (R/W)	TCPGFRM	Rx TCP (Good) Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.TCPGFRM bit, when set, masks the interrupt when the EMAC_RXTCP_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.

Table 24-124: EMAC_IPC_RXIMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	UDPERRFRM	Rx UDP Error Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.UDPERRFRM bit, when set, masks the interrupt when the EMAC_RXUDP_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
8 (R/W)	UDPGFRM	Rx UDP (Good) Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.UDPGFRM bit, when set, masks the interrupt when the EMAC_RXUDP_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
7 (R/W)	V6NOPAYFRM	Rx IPv6 No Payload Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.V6NOPAYFRM bit, when set, masks the interrupt when the EMAC_RXIPV6_NOPAY_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
6 (R/W)	V6HDERRFRM	Rx IPv6 Header Error Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.V6HDERRFRM bit, when set, masks the interrupt when the EMAC_RXIPV6_HDR_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
5 (R/W)	V6GFRM	Rx IPv6 (Good) Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.V6GFRM bit, when set, masks the interrupt when the EMAC_RXIPV6_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
4 (R/W)	V4UDSBLFRM	Rx IPv4 UDS Disable Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4UDSBLFRM bit, when set, masks the interrupt when the EMAC_RXIPV4_UDSBL_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
3 (R/W)	V4FRAGFRM	Rx IPv4 Fragmented Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4FRAGFRM bit, when set, masks the interrupt when the EMAC_RXIPV4_FRAG_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
2 (R/W)	V4NOPAYFRM	Rx IPv4 No Payload Frame Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4NOPAYFRM bit, when set, masks the interrupt when the EMAC_RXIPV4_NOPAY_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
1 (R/W)	V4HDERRFRM	Rx IPv4 Header Error Frame Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4HDERRFRM bit, when set, masks the interrupt when the EMAC_RXIPV4_HDR_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.

Table 24-124: EMAC_IPC_RXIMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	V4GFRM	Rx IPv4 (Good) Frames Count Half/Full Mask. The EMAC_IPC_RXIMSK.V4GFRM bit, when set, masks the interrupt when the EMAC_RXIPV4_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.

MMC IPC Rx Interrupt Register

The `EMAC_IPC_RXINT` register indicates status of MMC IPC receive interrupts.

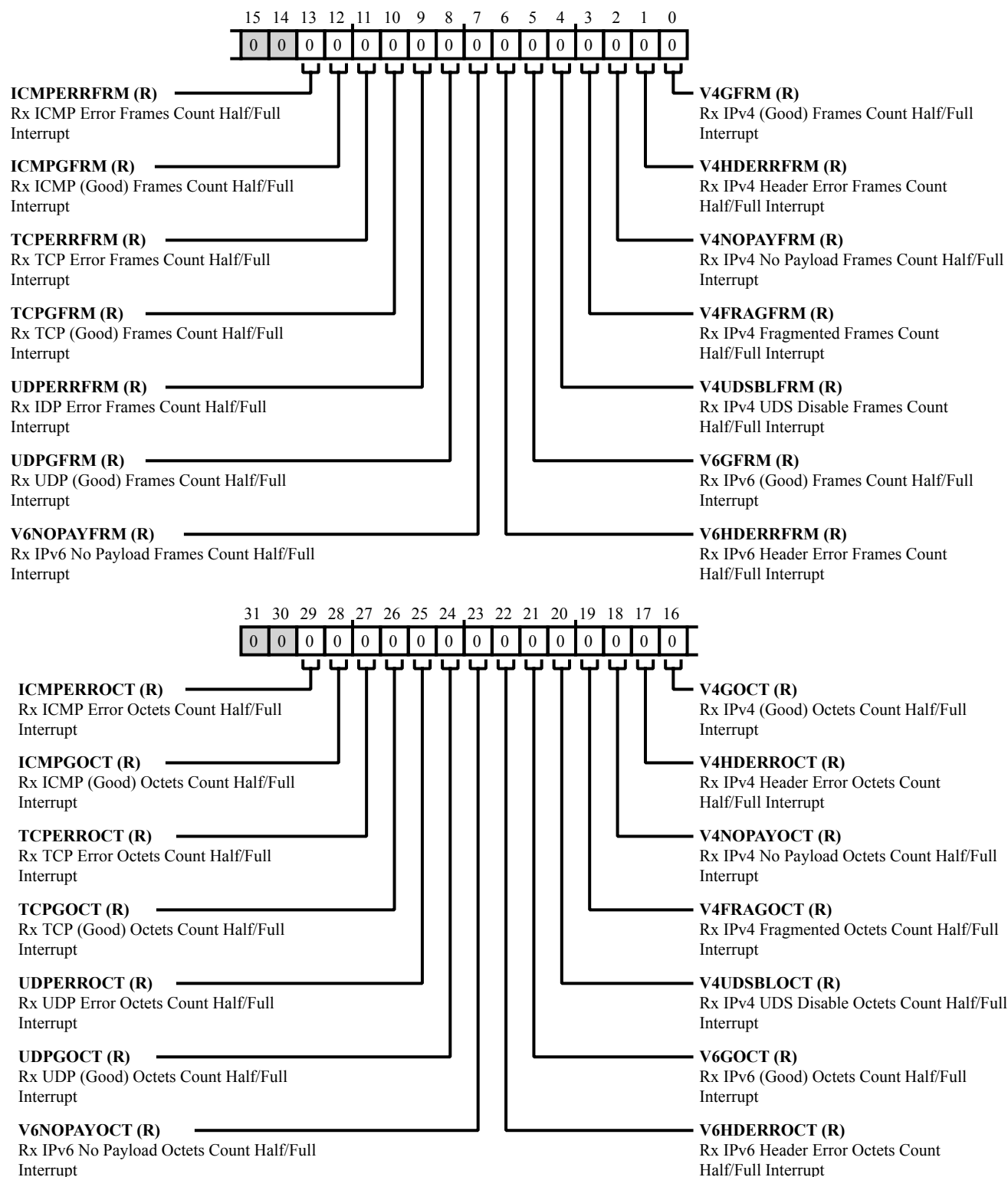


Figure 24-93: EMAC_IPC_RXINT Register Diagram

Table 24-125: EMAC_IPC_RXINT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/NW)	ICMPERROCT	Rx ICMP Error Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.ICMPERROCT bit is set when the EMAC_RXICMP_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
28 (R/NW)	ICMPGOCT	Rx ICMP (Good) Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.ICMPGOCT bit is set when the EMAC_RXICMP_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
27 (R/NW)	TCPPERROCT	Rx TCP Error Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.TCPPERROCT bit is set when the EMAC_RXTCP_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
26 (R/NW)	TCPGOCT	Rx TCP (Good) Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.TCPGOCT bit is set when the EMAC_RXTCP_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
25 (R/NW)	UDPPERROCT	Rx UDP Error Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.UDPPERROCT bit is set when the EMAC_RXUDP_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
24 (R/NW)	UDPGOCT	Rx UDP (Good) Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.UDPGOCT bit is set when the EMAC_RXUDP_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
23 (R/NW)	V6NOPAYOCT	Rx IPv6 No Payload Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.V6NOPAYOCT bit is set when the EMAC_RXIPV6_NOPAY_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
22 (R/NW)	V6HDERROCT	Rx IPv6 Header Error Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.V6HDERROCT bit is set when the EMAC_RXIPV6_HDR_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
21 (R/NW)	V6GOCT	Rx IPv6 (Good) Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.V6GOCT bit is set when the EMAC_RXIPV6_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
20 (R/NW)	V4UDSBLOCT	Rx IPv4 UDS Disable Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4UDSBLOCT bit is set when the EMAC_RXIPV4_UDSBL_OCT counter reaches half the maximum value, and also when it reaches the maximum value.

Table 24-125: EMAC_IPC_RXINT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/NW)	V4FRAGOCT	Rx IPv4 Fragmented Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4FRAGOCT bit is set when the EMAC_RXIPV4_FRAG_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
18 (R/NW)	V4NOPAYOCT	Rx IPv4 No Payload Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4NOPAYOCT bit set when the EMAC_RXIPV4_NOPAY_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
17 (R/NW)	V4HDERROCT	Rx IPv4 Header Error Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4HDERROCT bit is set when the EMAC_RXIPV4_HDR_ERR_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
16 (R/NW)	V4GOCT	Rx IPv4 (Good) Octets Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4GOCT bit is set when the EMAC_RXIPV4_GD_OCT counter reaches half the maximum value, and also when it reaches the maximum value.
13 (R/NW)	ICMPERRFRM	Rx ICMP Error Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.ICMPERRFRM bit is set when the EMAC_RXICMP_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
12 (R/NW)	ICMPGFRM	Rx ICMP (Good) Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.ICMPGFRM bit is set when the EMAC_RXICMP_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
11 (R/NW)	TCPERRFRM	Rx TCP Error Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.TCPERRFRM bit is set when the EMAC_RXTCP_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
10 (R/NW)	TCPGFRM	Rx TCP (Good) Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.TCPGFRM bit is set when the EMAC_RXTCP_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
9 (R/NW)	UDPERFRM	Rx IDP Error Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.UDPERFRM bit is set when the EMAC_RXUDP_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
8 (R/NW)	UDPGFRM	Rx UDP (Good) Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.UDPGFRM bit is set when the EMAC_RXUDP_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.

Table 24-125: EMAC_IPC_RXINT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	V6NOPAYFRM	Rx IPv6 No Payload Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.V6NOPAYFRM bit is set when the EMAC_RXIPV6_NOPAY_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
6 (R/NW)	V6HDERRFRM	Rx IPv6 Header Error Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.V6HDERRFRM bit is set when the EMAC_RXIPV6_HDR_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
5 (R/NW)	V6GFRM	Rx IPv6 (Good) Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.V6GFRM bit is set when the EMAC_RXIPV6_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
4 (R/NW)	V4UDSBLFRM	Rx IPv4 UDS Disable Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4UDSBLFRM bit is set when the EMAC_RXIPV4_UDSBL_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
3 (R/NW)	V4FRAGFRM	Rx IPv4 Fragmented Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4FRAGFRM bit is set when the EMAC_RXIPV4_FRAG_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
2 (R/NW)	V4NOPAYFRM	Rx IPv4 No Payload Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4NOPAYFRM bit is set when the EMAC_RXIPV4_NOPAY_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
1 (R/NW)	V4HDERRFRM	Rx IPv4 Header Error Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4HDERRFRM bit is set when the EMAC_RXIPV4_HDR_ERR_FRM counter reaches half the maximum value, and also when it reaches the maximum value.
0 (R/NW)	V4GFRM	Rx IPv4 (Good) Frames Count Half/Full Interrupt. The EMAC_IPC_RXINT.V4GFRM bit is set when the EMAC_RXIPV4_GD_FRM counter reaches half the maximum value, and also when it reaches the maximum value.

Interrupt Status Register

The `EMAC_ISTAT` register indicates EMAC interrupt status.

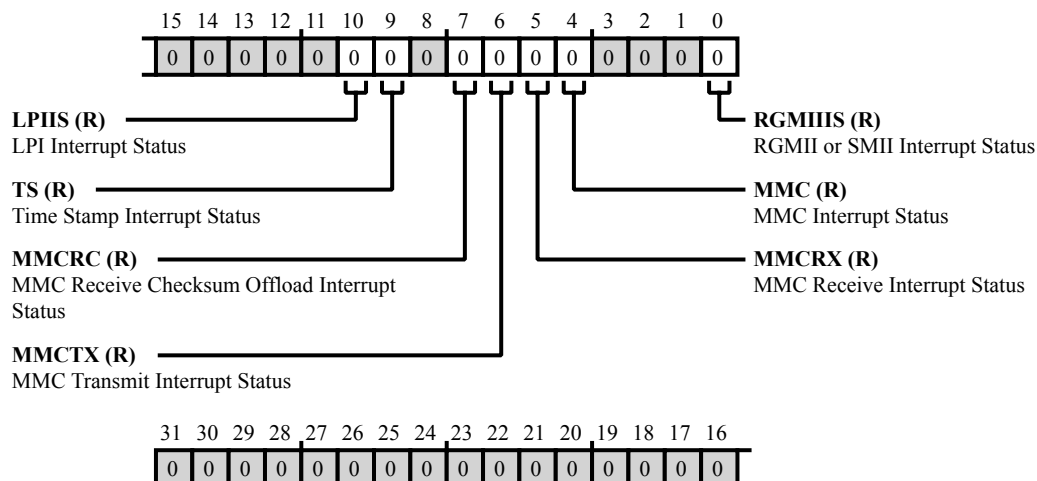


Figure 24-94: EMAC_ISTAT Register Diagram

Table 24-126: EMAC_ISTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/NW)	LPIIS	LPI Interrupt Status. The <code>EMAC_ISTAT.LPIIS</code> bit is set for any LPI state entry or exit in the MAC Transmitter or Receiver.
9 (R/NW)	TS	Time Stamp Interrupt Status. The <code>EMAC_ISTAT.TS</code> bit is set when:
7 (R/NW)	MMCRC	MMC Receive Checksum Offload Interrupt Status. The <code>EMAC_ISTAT.MMCRC</code> bit is set high whenever an interrupt is generated in the <code>EMAC_IPC_RXINT</code> . This bit is cleared when all the bits in this interrupt register are cleared.
6 (R/NW)	MMCTX	MMC Transmit Interrupt Status. The <code>EMAC_ISTAT.MMCTX</code> bit is set high whenever an interrupt is generated in the <code>EMAC_MMC_TXINT</code> register. This bit is cleared when all the bits in this interrupt register are cleared.
5 (R/NW)	MMCRX	MMC Receive Interrupt Status. The <code>EMAC_ISTAT.MMCRX</code> bit is set high whenever an interrupt is generated in the <code>EMAC_MMC_RXINT</code> register. This bit is cleared when all the bits in this interrupt register are cleared.

Table 24-126: EMAC_ISTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/NW)	MMC	MMC Interrupt Status. The EMAC_ISTAT.MMC bit is set high whenever any of EMAC_ISTAT bits [7:5] is set (=1) and is cleared only when all of these bits are cleared (=0).
0 (R/NW)	RGMIIS	RGMII or SMII Interrupt Status. The EMAC_ISTAT.RGMIIS bit is set because of any change in value of the Link Status of RGMII interface.

Layer3 and Layer4 Control Register

The `EMAC_L3L4_CTL` register controls the operations of the filter 0 of Layer 3 and Layer 4. This register is reserved if the Layer 3 and Layer 4 Filtering feature is not selected during core configuration.

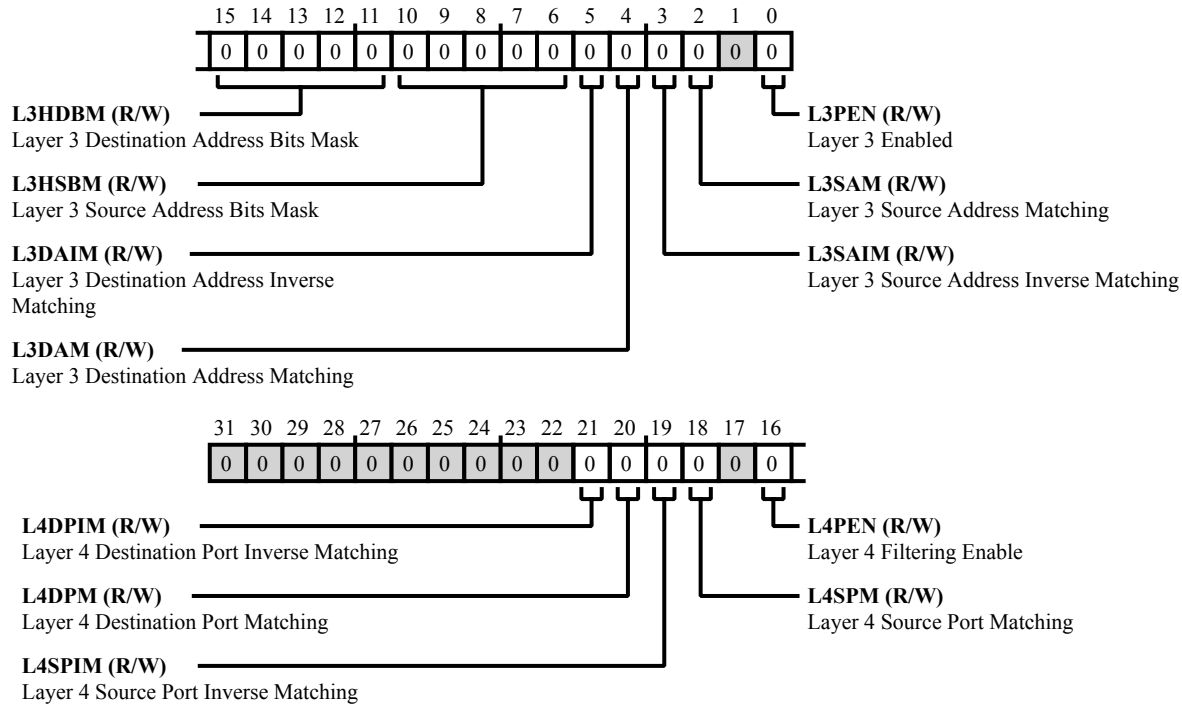


Figure 24-95: EMAC_L3L4_CTL Register Diagram

Table 24-127: EMAC_L3L4_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	L4DPIM	Layer 4 Destination Port Inverse Matching. The <code>EMAC_L3L4_CTL.L4DPIM</code> bit indicates that the Layer 4 Destination Port number field is enabled for inverse matching.
20 (R/W)	L4DPM	Layer 4 Destination Port Matching. The <code>EMAC_L3L4_CTL.L4DPM</code> bit indicates that the Layer 4 Destination Port number field is enabled for matching.
19 (R/W)	L4SPIM	Layer 4 Source Port Inverse Matching. The <code>EMAC_L3L4_CTL.L4SPIM</code> bit indicates that the Layer 4 Source Port number field is enabled for inverse matching.
18 (R/W)	L4SPM	Layer 4 Source Port Matching. The <code>EMAC_L3L4_CTL.L4SPM</code> bit indicates that the Layer 4 Source Port number field is enabled for matching.

Table 24-127: EMAC_L3L4_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	L4PEN	Layer 4 Filtering Enable. The EMAC_L3L4_CTL.L4PEN bit indicates that source and destination port number fields for UDP (TCP) frames are used for matching.
15:11 (R/W)	L3HDBM	Layer 3 Destination Address Bits Mask. The EMAC_L3L4_CTL.L3HDBM bits are the number of lower bits of IP destination Address that are masked for matching in the IPv4 frames.
10:6 (R/W)	L3HSBM	Layer 3 Source Address Bits Mask. The EMAC_L3L4_CTL.L3HSBM bit are the number of lower bits of IP source address that are masked for matching in the IPv4 frames.
5 (R/W)	L3DAIM	Layer 3 Destination Address Inverse Matching. The EMAC_L3L4_CTL.L3DAIM bit indicates that the Layer 3 IP destination address field is enabled for inverse matching.
4 (R/W)	L3DAM	Layer 3 Destination Address Matching. The EMAC_L3L4_CTL.L3DAM bit indicates that Layer 3 IP destination address field is enabled for matching.
3 (R/W)	L3SAIM	Layer 3 Source Address Inverse Matching. The EMAC_L3L4_CTL.L3SAIM bit indicates that the Layer 3 IP Source Address field is enabled for inverse matching.
2 (R/W)	L3SAM	Layer 3 Source Address Matching. The EMAC_L3L4_CTL.L3SAM bit indicates that the Layer 3 IP Source Address field is enabled for matching.
0 (R/W)	L3PEN	Layer 3 Enabled. The EMAC_L3L4_CTL.L3PEN bit indicates that layer 3 source or destination address matching is enabled for IPV6 (IPV4) frames.

Layer 3 Address0 Register

The `EMAC_L3_ADDR0` register tells For IPv4 frames, the Layer 3 Address 0 Register 0 contains the 32-bit IP Source Address field.

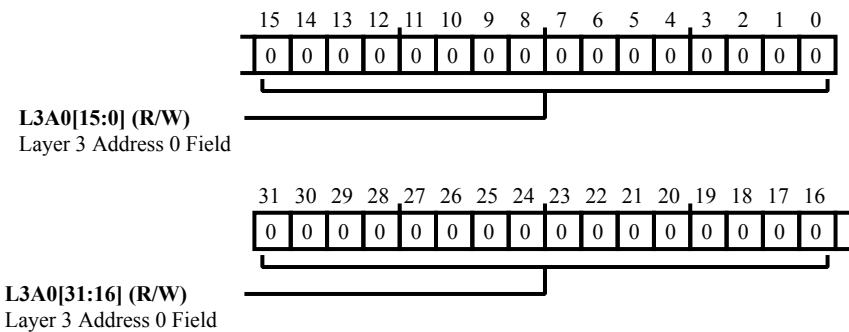


Figure 24-96: EMAC_L3_ADDR0 Register Diagram

Table 24-128: EMAC_L3_ADDR0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	L3A0	Layer 3 Address 0 Field. When Bit 0 (L3PEN0) and Bit 2 (L3SAM0) are set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with Bits [31:0] of the IP Source Address field in the IPv6 frames. When Bit 0 (L3PEN0) and Bit 4 (L3DAM0) are set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with Bits [31:0] of the IP Destination Address field in the IPv6 frames. When Bit 0 (L3PEN0) is reset and Bit 2 (L3SAM0) is set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with the IP Source Address field in the IPv4 frames.

Layer 3 Address1 Register

The `EMAC_L3_ADDR1` register tells For IPv4 frames, the Layer 3 Address 1 Register 0 contains the 32-bit IP Source Address field.

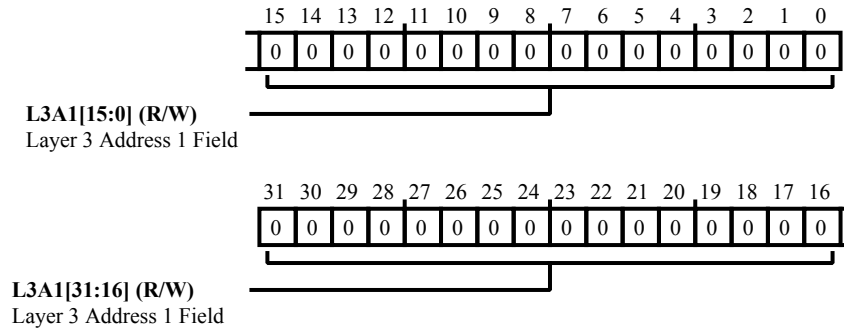


Figure 24-97: EMAC_L3_ADDR1 Register Diagram

Table 24-129: EMAC_L3_ADDR1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	L3A1	Layer 3 Address 1 Field. When Bit 0 (L3PEN0) and Bit 2 (L3SAM0) are set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with Bits [31:0] of the IP Source Address field in the IPv6 frames. When Bit 0 (L3PEN0) and Bit 4 (L3DAM0) are set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with Bits [31:0] of the IP Destination Address field in the IPv6 frames. When Bit 0 (L3PEN0) is reset and Bit 2 (L3SAM0) is set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with the IP Source Address field in the IPv4 frames.

Layer 3 Address2 Register

The `EMAC_L3_ADDR2` register tells For IPv4 frames, the Layer 3 Address 2 Register 0 contains the 32-bit IP Source Address field.

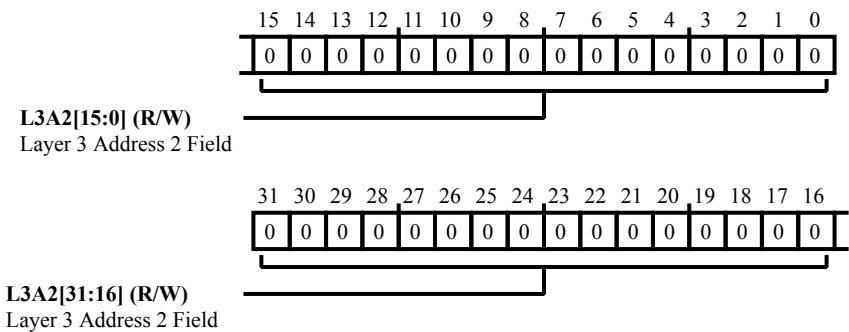


Figure 24-98: `EMAC_L3_ADDR2` Register Diagram

Table 24-130: `EMAC_L3_ADDR2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	L3A2	Layer 3 Address 2 Field. When Bit 0 (L3PEN0) and Bit 2 (L3SAM0) are set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with Bits [31:0] of the IP Source Address field in the IPv6 frames. When Bit 0 (L3PEN0) and Bit 4 (L3DAM0) are set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with Bits [31:0] of the IP Destination Address field in the IPv6 frames. When Bit 0 (L3PEN0) is reset and Bit 2 (L3SAM0) is set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with the IP Source Address field in the IPv4 frames.

Layer 3 Address3 Register

The `EMAC_L3_ADDR3` register tells For IPv4 frames, the Layer 3 Address 3 Register 0 contains the 32-bit IP Source Address field.

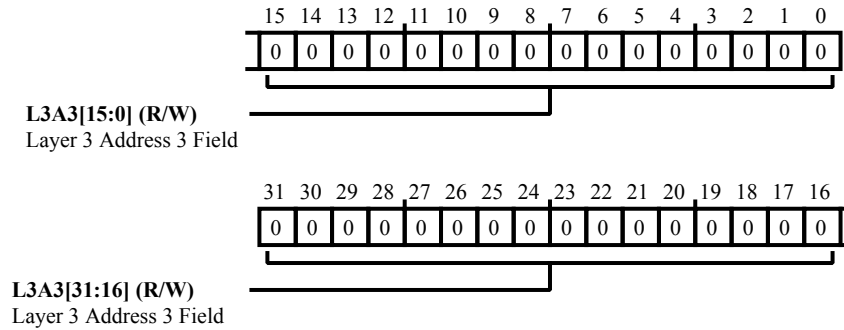


Figure 24-99: EMAC_L3_ADDR3 Register Diagram

Table 24-131: EMAC_L3_ADDR3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	L3A3	Layer 3 Address 3 Field. When Bit 0 (L3PEN0) and Bit 2 (L3SAM0) are set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with Bits [31:0] of the IP Source Address field in the IPv6 frames. When Bit 0 (L3PEN0) and Bit 4 (L3DAM0) are set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with Bits [31:0] of the IP Destination Address field in the IPv6 frames. When Bit 0 (L3PEN0) is reset and Bit 2 (L3SAM0) is set in Register 256 (Layer 3 and Layer 4 Control Register 0), this field contains the value to be matched with the IP Source Address field in the IPv4 frames.

Layer 4 Address Register

The `EMAC_L4_ADDR` register contains Layer 4 Port number field. It contains the 16-bit Source and Destination Port numbers of the TCP or UDP frame.

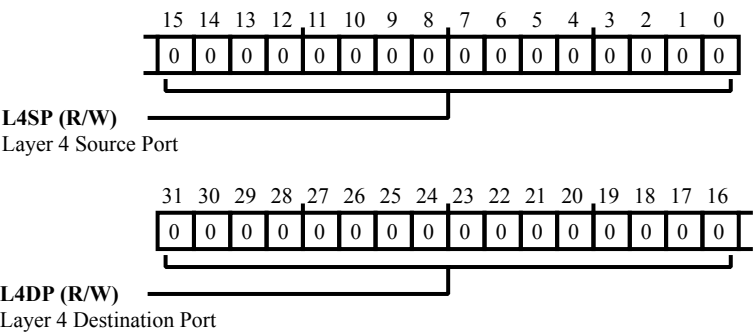


Figure 24-100: `EMAC_L4_ADDR` Register Diagram

Table 24-132: `EMAC_L4_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	L4DP	Layer 4 Destination Port. When <code>EMAC_L3L4_CTL.L4PEN</code> is reset and <code>EMAC_L3L4_CTL.L4DPM</code> is set, the <code>EMAC_L4_ADDR.L4DP</code> bit field contains the value to be matched with the TCP Destination Port Number field in the IPv4 or IPv6 frames. When <code>EMAC_L3L4_CTL.L4PEN</code> and <code>EMAC_L3L4_CTL.L4DPM</code> are set, the <code>EMAC_L4_ADDR.L4DP</code> bit field contains the value to be matched with the UDP Destination Port Number field in the IPv4 or IPv6 frames.
15:0 (R/W)	L4SP	Layer 4 Source Port. When <code>EMAC_L3L4_CTL.L4PEN</code> is reset and <code>EMAC_L3L4_CTL.L4DPM</code> is set, the <code>EMAC_L4_ADDR.L4SP</code> bit field contains the value to be matched with the TCP Source Port Number field in the IPv4 or IPv6 frames. When <code>EMAC_L3L4_CTL.L4PEN</code> and <code>EMAC_L3L4_CTL.L4DPM</code> are set, the <code>EMAC_L4_ADDR.L4SP</code> bit field contains the value to be matched with the UDP Source Port Number field in the IPv4 or IPv6 frames.

Low Power Idle Control and Status Register

The `EMAC_LPI_CTLSTAT` register controls the behavior of EMAC0 for LPI mode and reports status.

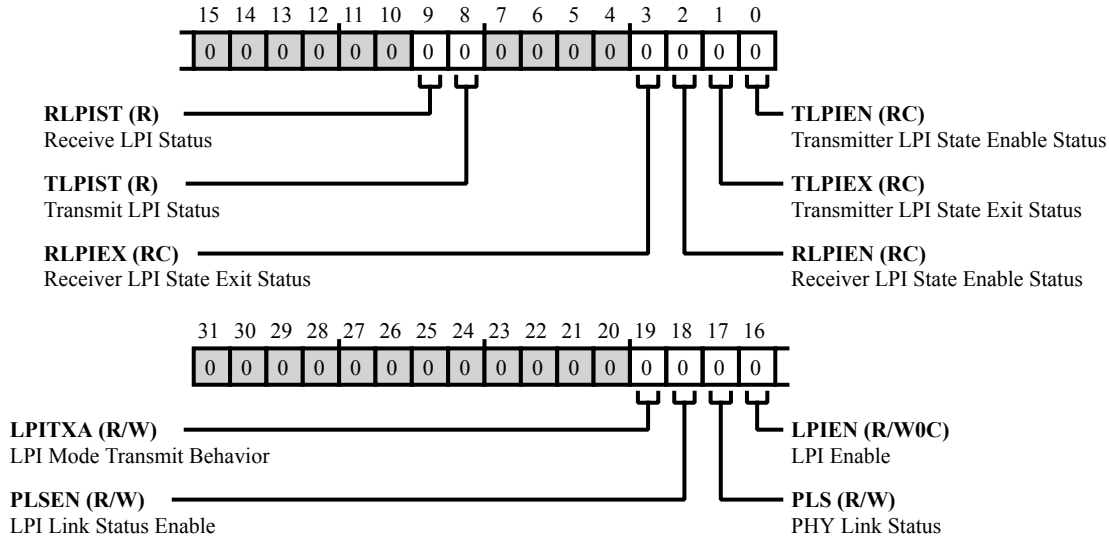


Figure 24-101: EMAC_LPI_CTLSTAT Register Diagram

Table 24-133: EMAC_LPI_CTLSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	LPITXA	LPI Mode Transmit Behavior. The <code>EMAC_LPI_CTLSTAT.LPITXA</code> bit controls the behavior of the MAC when it is entering or exiting of the LPI mode on the transmit side.
18 (R/W)	PLSEN	LPI Link Status Enable. The <code>EMAC_LPI_CTLSTAT.PLS</code> bit enables the link status received on the RGMII receive paths to be used for activating the LPI LS timer.
17 (R/W)	PLS	PHY Link Status. The <code>EMAC_LPI_CTLSTAT.PLS</code> bit indicates the link status of the PHY
16 (R/W0C)	LPIEN	LPI Enable. The <code>EMAC_LPI_CTLSTAT.LPIEN</code> bit instructs the MAC Transmitter to enter the LPI state. When reset, this bit instructs the MAC to exit the LPI state and resume normal transmission.
9 (R/NW)	RLPIST	Receive LPI Status. The <code>EMAC_LPI_CTLSTAT.RLPIST</code> bit indicates that the MAC is receiving the LPI pattern on the GMII or MII interface.

Table 24-133: EMAC_LPI_CTLSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/NW)	TLPIST	Transmit LPI Status. The EMAC_LPI_CTLSTAT.TLPIST bit indicates that the MAC is transmitting the LPI pattern on the GMII or MII interface.
3 (RC/NW)	RLPIEX	Receiver LPI State Exit Status. The EMAC_LPI_CTLSTAT.RLPIEX bit indicates that the MAC Receiver has stopped receiving the LPI pattern on the GMII or MII interface, exited the LPI state, and resumed the normal reception. This bit is cleared by a read into this register.
2 (RC/NW)	RLPIEN	Receiver LPI State Enable Status. The EMAC_LPI_CTLSTAT.RLPIEN bit indicates that the MAC Receiver has received an LPI pattern and entered the LPI state. This bit is cleared by a read into this register.
1 (RC/NW)	TLPIEX	Transmitter LPI State Exit Status. The EMAC_LPI_CTLSTAT.TLPIEX bit indicates that the MAC transmitter has exited the LPI state after the program has cleared the EMAC_LPI_CTLSTAT.LPIEN bit and the LPI TW Timer has expired. This bit is cleared by a read into this register.
0 (RC/NW)	TLPIEN	Transmitter LPI State Enable Status. The EMAC_LPI_CTLSTAT.TLPIEN bit indicates that the MAC Transmitter has entered the LPI state due to the setting of the EMAC_LPI_CTLSTAT.LPIEN bit. This bit is cleared by a read into this register.

Low Power Idle Timeout Register

The `EMAC_LPI_TMRCTL` controls the timeout values in the LPI states. It specifies the time for which the MAC transmits the LPI pattern and also the time for which the MAC waits before resuming the normal transmission.

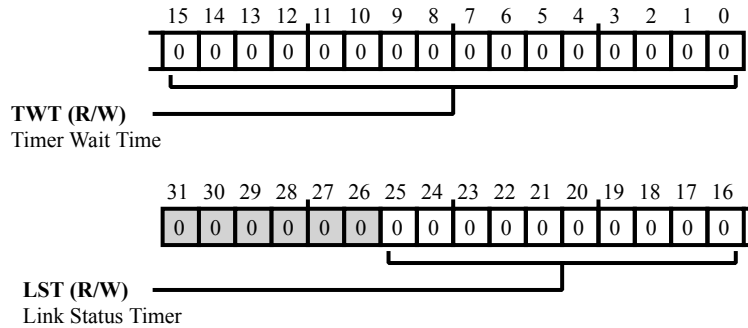


Figure 24-102: EMAC_LPI_TMRCTL Register Diagram

Table 24-134: EMAC_LPI_TMRCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:16 (R/W)	LST	Link Status Timer. The <code>EMAC_LPI_TMRCTL.LST</code> bit, specifies the minimum time (in milliseconds) for which the link status from the PHY should be up (OKAY) before the LPI pattern can be transmitted to the PHY.
15:0 (R/W)	TWT	Timer Wait Time. The <code>EMAC_LPI_TMRCTL.TWT</code> bit, specifies the minimum time (in microseconds) for which the MAC waits after it stops transmitting the LPI pattern to the PHY and before it resumes the normal transmission.

MAC Configuration Register

The `EMAC_MACCFG` register configures MAC features.

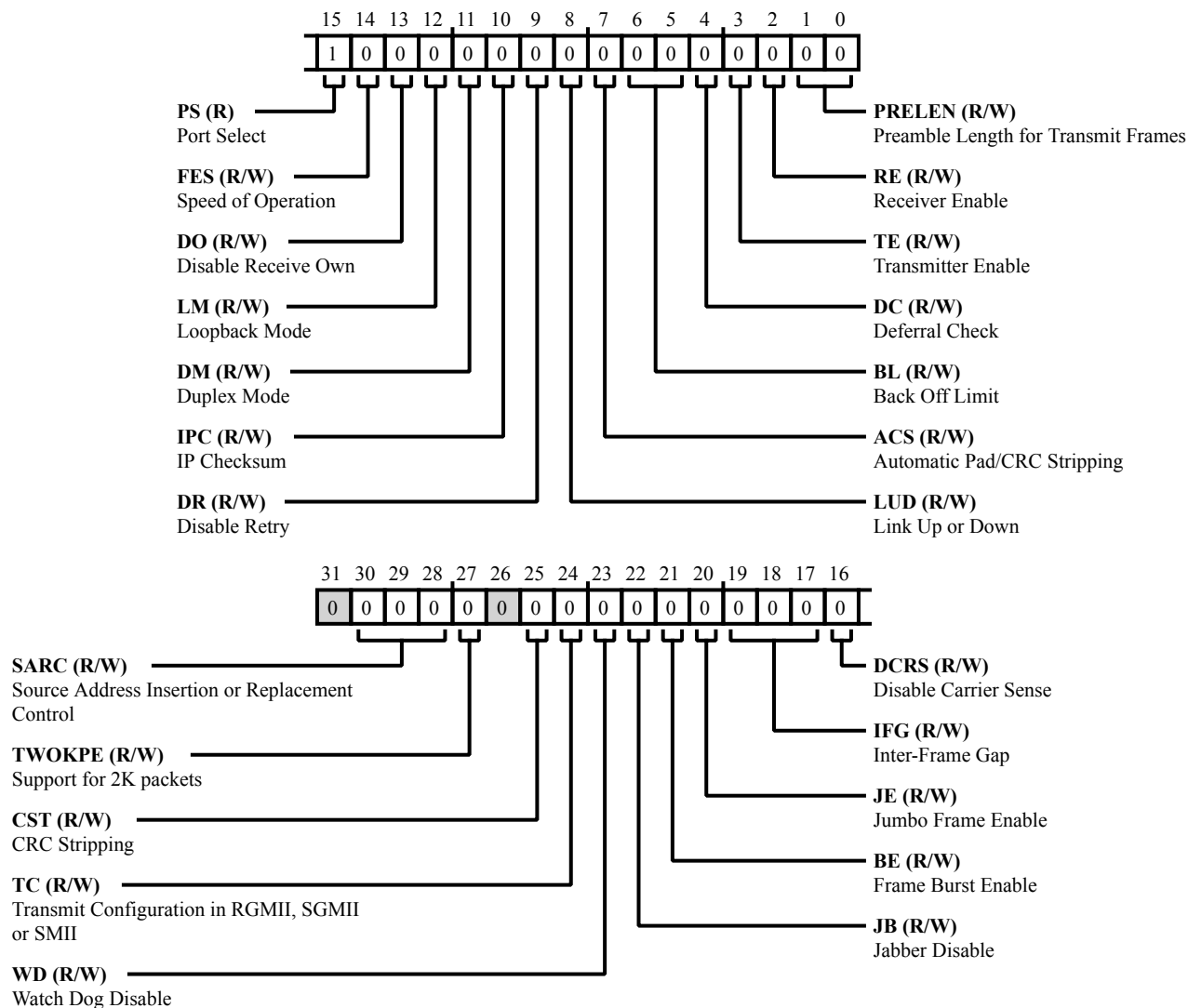


Figure 24-103: EMAC_MACCFG Register Diagram

Table 24-135: EMAC_MACCFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30:28 (R/W)	SARC	Source Address Insertion or Replacement Control. The <code>EMAC_MACCFG.SARC</code> bit, controls the source address insertion or replacement for all transmitted frames.

Table 24-135: EMAC_MACCFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
27 (R/W)	TWOKPE	Support for 2K packets. The EMAC_MACCFG.TWOKPE bit, IEEE 802.3as Support for 2K Packets When set, the MAC considers all frames, with up to 2,000 bytes length, as normal packets.
25 (R/W)	CST	CRC Stripping. The EMAC_MACCFG.CST bit, when set, directs the MAC to strip the last 4 bytes (FCS) of all frames of Ether type (Type field of frame greater than 0x0600) and drop these bytes before forwarding the frame to the application.
24 (R/W)	TC	Transmit Configuration in RGMII, SGMII or SMII. The EMAC_MACCFG.TC bit, enables the transmission of duplex mode, link speed, and link up or down information to the PHY in the RGMII port.
23 (R/W)	WD	Watch Dog Disable. The EMAC_MACCFG.WD bit, when set, disables the watchdog timer on the receiver, and can receive frames of up to 16,384 bytes. When this bit is reset, the MAC allows no more than 2,048 bytes (10,240 if EMAC_MACCFG.JE is set high) of the frame being received and cuts off any bytes received after that.
22 (R/W)	JB	Jabber Disable. The EMAC_MACCFG.JB bit, when set, disables the jabber timer on the transmitter, and can transfer frames of up to 16,384 bytes. When this bit is reset, the MAC cuts off the transmitter if the application sends out more than 2,048 bytes of data (10,240 if EMAC_MACCFG.JE is set high) during transmission.
21 (R/W)	BE	Frame Burst Enable. The EMAC_MACCFG.BE bit, allows frame bursting during transmission in the GMII half-duplex mode.
20 (R/W)	JE	Jumbo Frame Enable. The EMAC_MACCFG.JE bit, when set, directs the MAC to allow Jumbo frames of 9,018 bytes (9,022 bytes for VLAN tagged frames).
19:17 (R/W)	IFG	Inter-Frame Gap. The EMAC_MACCFG.IFG bits control the minimum inter-frame gap between frames during transmission. Note that in Half-Duplex mode, the minimum gap can be configured for 64 bit times (EMAC_MACCFG.IFG = 100) only. Lower values are not considered.
		0 96 bit times
		1 88 bit times
		2 80 bit times
		3 72 bit times
		4 64 bit times

Table 24-135: EMAC_MACCFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		5	56 bit times
		6	48 bit times
		7	40 bit times
16 (R/W)	DCRS	Disable Carrier Sense. The EMAC_MACCFG.DCRS bit, when set, makes the MAC transmitter ignore the CRS signal during frame transmission in Half-Duplex mode. This request results in no errors generated due to Loss of Carrier or No Carrier during such transmission. When this bit is low, the MAC transmitter generates such errors due to Carrier Sense and will even abort the transmissions.	
15 (R/NW)	PS	Port Select. The EMAC_MACCFG.PS bit selects between GMII and MII as: 0=GMII (1000 Mbps) and 1=MII (10/100 Mbps). This bit is read-only with the appropriate value in the 10/100 Mbps-only (always 1) or 1000 Mbps-only (always 0) configurations, and R_W in the default 10/100/1000 Mbps configuration.	
14 (R/W)	FES	Speed of Operation. The EMAC_MACCFG.FES bit indicates the Ethernet speed as 10 Mbps (bit =0) or 100 Mbps (bit =1).	
13 (R/W)	DO	Disable Receive Own. The EMAC_MACCFG.DO bit, when set, disables MAC reception of frames when MAC is transmitting in Half-Duplex mode. When this bit is reset, the MAC receives all packets that are given by the PHY while transmitting. This bit is not applicable if the MAC is operating in Full-Duplex mode.	
12 (R/W)	LM	Loopback Mode. The EMAC_MACCFG.LM bit, when set, directs the MAC to operate in internal loop back mode. (The media independent interface pins are not driven or sampled.)	
11 (R/W)	DM	Duplex Mode. The EMAC_MACCFG.DM bit, when set, directs the MAC to operate in a Full-Duplex mode where it can transmit and receive simultaneously.	
10 (R/W)	IPC	IP Checksum. The EMAC_MACCFG.IPC bit, when set, directs the MAC to calculate the 16-bit one's complement of the one's complement sum of all received Ethernet frame payloads. It also checks whether the IPv4 Header checksum (assumed to be bytes 25-26 or 29-30 (VLAN-tagged) of the received Ethernet frame) is correct for the received frame and gives the status in the receive status word. The EMAC_MACCFG.IPC bit, when set, enables IPv4 checksum checking for received frame payloads' TCP/UDP/ICMP headers. When this bit is reset, the Checksum Offload Engine function in the receiver is disabled and the corresponding PCE and IP HCE status bits are always cleared.	

Table 24-135: EMAC_MACCFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	DR	Disable Retry. The EMAC_MACCFG.DR bit, when set, directs the MAC to attempt only 1 transmission. When a collision occurs on the media independent interface, the MAC ignores the current frame transmission and reports a Frame Abort with excessive collision error in the transmit frame status. When the EMAC_MACCFG.DR bit is reset, the MAC attempts retries based on the settings of BL. This bit is applicable only to Half-Duplex mode.
		0 Retry enabled
		1 Retry disabled
8 (R/W)	LUD	Link Up or Down. The EMAC_MACCFG.LUD bit, indicates whether the link is up or down during the transmission of configuration in the RGMII, SGMII, or SMII interface: 0 means Link down and 1 means Link Up
7 (R/W)	ACS	Automatic Pad/CRC Stripping. The EMAC_MACCFG.ACS bit, when set, directs the MAC to strip the Pad/FCS field on incoming frames only if the length fields value is less than or equal to 1,500 bytes. All received frames with length field greater than or equal to 1,501 bytes are passed to the application without stripping the Pad/FCS field. When the EMAC_MACCFG.ACS bit is reset, the MAC passes all incoming frames to the Host unmodified.
6:5 (R/W)	BL	Back Off Limit. The EMAC_MACCFG.BL bit selects the back-off limit, determining the random integer number (r) of slot time delays (512 bit times for 10/100 Mbps) the MAC waits before rescheduling a transmission attempt during retries after a collision. This bit is applicable only to Half-Duplex mode. The random integer r takes the value in the range: $0 \text{ less-than-equal-to } r \text{ less-than } 2^k$ Where k is the minimum of n (number of transmission attempts) or a limit value.
		0 $k = \min(n, 10)$
		1 $k = \min(n, 8)$
		2 $k = \min(n, 4)$
		3 $k = \min(n, 1)$

Table 24-135: EMAC_MACCFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	DC	Deferral Check. The EMAC_MACCFG.DC bit, when set, enables the deferral check function in the MAC. The MAC issues a Frame Abort status, along with the excessive deferral error bit set in the transmit frame status when the transmit state machine is deferred for more than 24,288 bit times in 10/100-Mbps mode. If the Jumbo frame mode is enabled in 10/100-Mbps mode, the threshold for deferral is 155,680 bits times. Deferral begins when the transmitter is ready to transmit, but is prevented because of an active CRS (carrier sense) signal. Defer time is not cumulative. If the transmitter defers for 10,000 bit times, then transmits, collides, backs off, and then has to defer again after completion of back-off, the deferral timer resets to 0 and restarts. When the EMAC_MACCFG.DC bit is reset, the deferral check function is disabled and the MAC defers until the CRS signal goes inactive. This bit is applicable only in Half-Duplex mode.
3 (R/W)	TE	Transmitter Enable. The EMAC_MACCFG.TE bit, when set, enables the transmit state machine of the MAC for transmission. When this bit is reset, the MAC transmit state machine is disabled after the completion of the transmission of the current frame, and will not transmit any further frames.
2 (R/W)	RE	Receiver Enable. The EMAC_MACCFG.RE bit, when set, enables the receiver state machine of the MAC for receiving frames. When this bit is reset, the MAC receive state machine is disabled after the completion of the reception of the current frame, and does not receive any further frames.
1:0 (R/W)	PRELEN	Preamble Length for Transmit Frames. The EMAC_MACCFG.PRELEN bit, control the number of preamble bytes that are added to the beginning of every Transmit frame.

MAC Rx Frame Filter Register

The `EMAC_MACFRMFILT` register controls receive frame filter features.

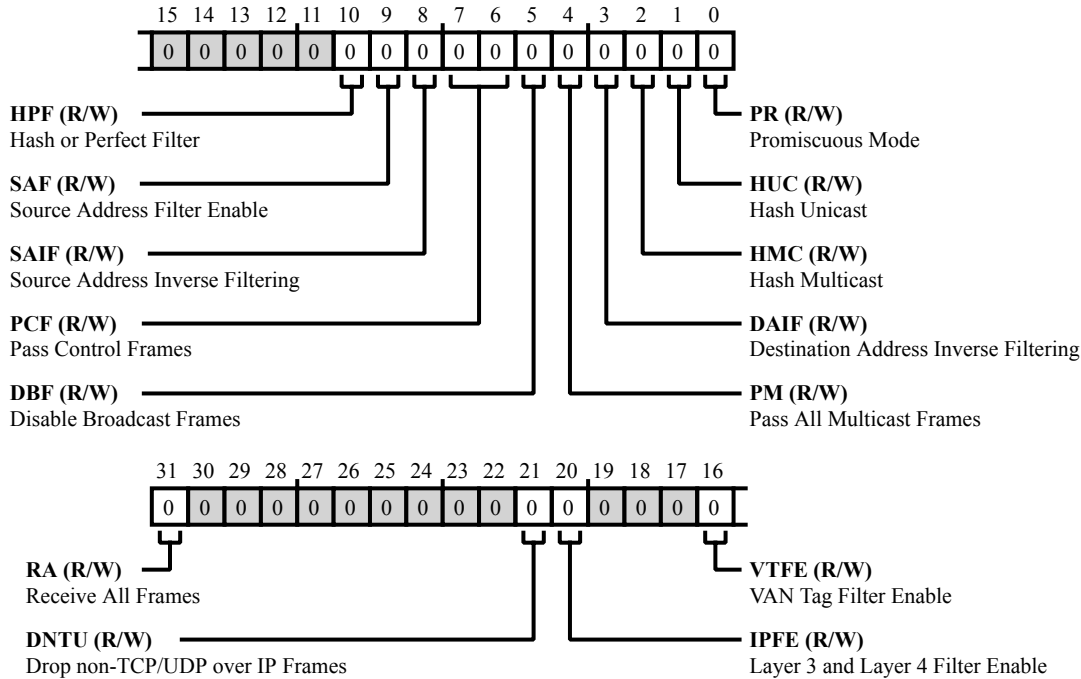


Figure 24-104: EMAC_MACFRMFILT Register Diagram

Table 24-136: EMAC_MACFRMFILT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	RA	Receive All Frames. The <code>EMAC_MACFRMFILT.RA</code> bit, when set, directs the MAC Receiver module to pass to the Application all frames received irrespective of whether they pass the address filter. The result of the DA filtering is updated (pass or fail) in the corresponding bits in the Receive Status Word. When this bit is reset, the Receiver module passes to the Application only those frames that pass the DA address filter.
21 (R/W)	DNTU	Drop non-TCP/UDP over IP Frames. The <code>EMAC_MACFRMFILT.DNTU</code> bit, enables the MAC to drop the non-TCP or UDP over IP frames. The MAC forward only those frames that are processed by the Layer 4 filter. When reset, this bit enables the MAC to forward all non-TCP or UDP over IP frames.
20 (R/W)	IPFE	Layer 3 and Layer 4 Filter Enable. The <code>EMAC_MACFRMFILT.IPFE</code> bit, enables the MAC to drop frames that do not match the enabled Layer 3 and Layer 4 filters. If Layer 3 or Layer 4 filters are not enabled for matching, this bit does not have any effect.

Table 24-136: EMAC_MACFRMFILT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	VTFE	VAN Tag Filter Enable. The EMAC_MACFRMFILT.VTFE bit, enables the MAC to drop VLAN tagged frames that do not match the VLAN Tag comparison.
10 (R/W)	HPF	Hash or Perfect Filter. The EMAC_MACFRMFILT.HPF bit, when set, configures the address filter to pass a frame if it matches either the perfect filtering or the hash filtering as set by EMAC_MACFRMFILT.HMC or EMAC_MACFRMFILT.HUC bits. When EMAC_MACFRMFILT.HPF is low and either the EMAC_MACFRMFILT.HUC bit or EMAC_MACFRMFILT.HMC bit is set, the frame is passed only if it matches the Hash filter.
9 (R/W)	SAF	Source Address Filter Enable. When the EMAC_MACFRMFILT.SAF bit, is set, the MAC compares the SA field of the received frames with the values programmed in the enabled SA registers. If the comparison fails, the MAC drops the frame.
8 (R/W)	SAIF	Source Address Inverse Filtering. When the EMAC_MACFRMFILT.SAIF bit is set, the Address Check block operates in inverse filtering mode for the SA address comparison. The frames whose SA matches the SA registers are marked as failing the SA Address filter.
7:6 (R/W)	PCF	Pass Control Frames. The EMAC_MACFRMFILT.PCF bits control the forwarding of all control frames (including unicast and multicast PAUSE frames). Note that the processing of PAUSE control frames depends only on the value of the EMAC_FLOWCTL.RFE bit.
		0 Pass no control frames All control frames are filtered from reaching the application.
		1 Pass no PAUSE frames All control frames are passed to the application (even if they fail the address filter), except for PAUSE frames.
		2 Pass all control frames All control frames are passed to the application (even if they fail the address filter).
		3 Pass address filtered control frames All control frames that pass the address filter are passed to the application.
5 (R/W)	DBF	Disable Broadcast Frames. The EMAC_MACFRMFILT.DBF bit, when set, directs the AFM module to filter all incoming broadcast frames. When this bit is reset, the AFM module passes all received broadcast frames.
		0 AFM module passes all received broadcast frames
		1 AFM module filters all incoming broadcast frames

Table 24-136: EMAC_MACFRMFILT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	PM	Pass All Multicast Frames. The EMAC_MACFRMFILT . PM bit, when set, indicates that all received frames with a multicast destination address (first bit in the destination address field is =1) are passed. When this bit is reset, filtering of multicast frame depends on EMAC_MACFRMFILT . HMC bit.
3 (R/W)	DAIF	Destination Address Inverse Filtering. The EMAC_MACFRMFILT . DAIF bit, when set, directs the Address Check block to operate in inverse filtering mode for the DA address comparison for both unicast and multicast frames. When this bit is reset, normal filtering of frames is performed.
2 (R/W)	HMC	Hash Multicast. The EMAC_MACFRMFILT . HMC bit, when set, directs the EMAC to perform destination address filtering of received multicast frames according to the hash table. When this bit is reset, the MAC performs a perfect destination address filtering for multicast frames, that is, the MAC compares the DA field with the values programmed in the EMAC_ADDR0_HI and EMAC_ADDR0_LO address registers.
1 (R/W)	HUC	Hash Unicast. The EMAC_MACFRMFILT . HUC bit, when set, directs the EMAC to perform destination address filtering of unicast frames according to the hash table. When this bit is reset, the MAC performs a perfect destination address filtering for unicast frames, that is, it compares the DA field with the values programmed in the EMAC_ADDR0_HI and EMAC_ADDR0_LO address registers.
0 (R/W)	PR	Promiscuous Mode. The EMAC_MACFRMFILT . PR bit, when set, directs the Address Filter module to pass all incoming frames regardless of its destination or source address. The DA Filter Fails status bits of the Receive Status Word is always cleared when EMAC_MACFRMFILT . PR is set.

AV MAC Control Register

The `EMAC_MAC_AVCTL` register controls the AV traffic by identifying the AV traffic and queuing it to appropriate channel. This register is present only when you select the AV feature during core configuration.

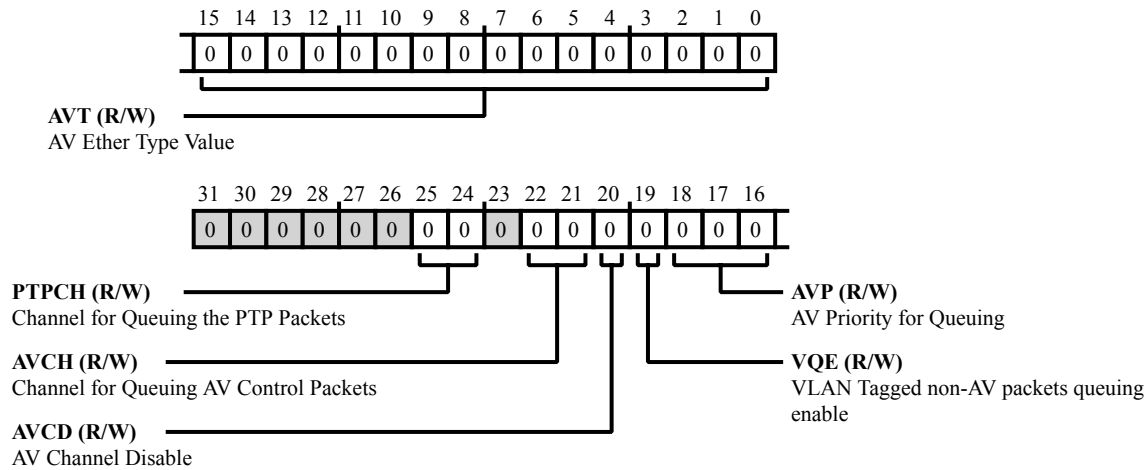


Figure 24-105: EMAC_MAC_AVCTL Register Diagram

Table 24-137: EMAC_MAC_AVCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:24 (R/W)	PTPCH	Channel for Queuing the PTP Packets. The <code>EMAC_MAC_AVCTL.PTPCH</code> bit, specifies the channel on which the untagged PTP packets, sent over the Ethernet payload and not over IPv4 or IPv6, are queued.
22:21 (R/W)	AVCH	Channel for Queuing AV Control Packets. The <code>EMAC_MAC_AVCTL.AVCH</code> bit, specifies the channel on which the received untagged AV control packets are queued.
20 (R/W)	AVCD	AV Channel Disable. The <code>EMAC_MAC_AVCTL.AVCD</code> bit, When this bit is set, the MAC forwards all packets to the default Channel 0 and the values programmed in the AVP, AVCH, and PTPCH fields are ignored. This bit is reserved and read-only if Channel 1 or Channel 2 receive paths are not selected during core configuration.
19 (R/W)	VQE	VLAN Tagged non-AV packets queuing enable. The <code>EMAC_MAC_AVCTL.VQE</code> bit, When this bit is set, the MAC also queues non-AV VLAN tagged packets into the available channels according to the value of the AVP bits. This bit is reserved and read-only if Channel 1 and Channel 2 Receive paths are not selected during core configuration.

Table 24-137: EMAC_MAC_AVCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18:16 (R/W)	AVP	AV Priority for Queuing. The EMAC_MAC_AVCTL.AVP bit, The value programmed in these bits control the receive channel (0, 1, or 2) to which an AV packet with a given priority must be queued.
15:0 (R/W)	AVT	AV Ether Type Value. The EMAC_MAC_AVCTL.AVT bit, contains the value that is compared with the EtherType field of the incoming (tagged or untagged) Ethernet frame to detect an AV packet.

MMC Control Register

The `EMAC_MMC_CTL` register selects the MMC operating mode.

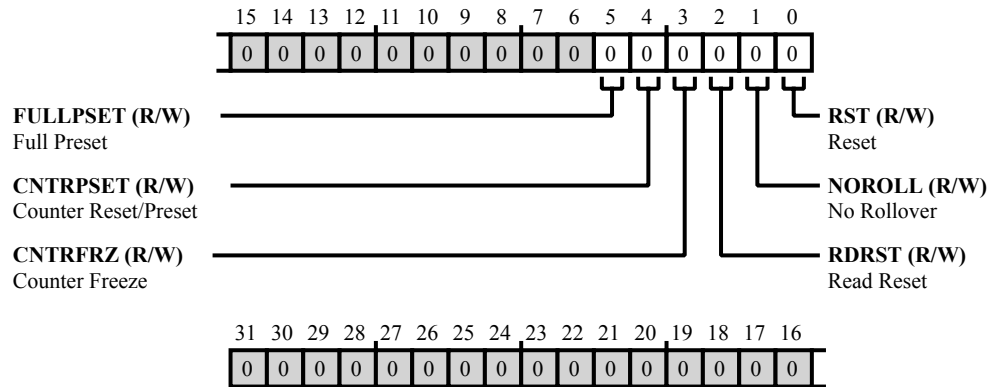


Figure 24-106: EMAC_MMC_CTL Register Diagram

Table 24-138: EMAC_MMC_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	FULLPSET	Full Preset. The <code>EMAC_MMC_CTL.FULLPSET</code> bit, when =0 (and <code>EMAC_MMC_CTL.CNTRPSET</code> =1), presets all MMC counters to almost-half value. All octet counters get preset to 0x7FFF_F800 (half - 2KBytes) and all frame-counters gets preset to 0x7FFF_FFF0 (half - 16). When <code>EMAC_MMC_CTL.FULLPSET</code> =1 (and <code>EMAC_MMC_CTL.CNTRPSET</code> =1), all MMC counters get preset to almost-full value. All octet counters get preset to 0xFFFF_F800 (full - 2KBytes) and all frame-counters gets preset to 0xFFFF_FFF0 (full - 16). For 16-bit counters, the almost-half preset values are 0x7800 and 0x7FF0 for the respective octet and frame counters. Similarly, the almost-full preset values for the 16-bit counters are 0xF800 and 0xFFFF0.
4 (R/W)	CNTRPSET	Counter Reset/Preset. The <code>EMAC_MMC_CTL.CNTRPSET</code> bit, when set, initializes all counters or presets counters to almost full or almost half as per <code>EMAC_MMC_CTL.FULLPSET</code> . The <code>EMAC_MMC_CTL.CNTRPSET</code> bit is cleared automatically after 1 clock cycle. This bit along with bit5 is useful for debugging and testing the assertion of interrupts because of MMC counter becoming half-full or full.
3 (R/W)	CNTRFRZ	Counter Freeze. The <code>EMAC_MMC_CTL.CNTRFRZ</code> bit, when set, freezes all the MMC counters to their current value. None of the MMC counters are updated due to any transmitted or received frame, until this bit is reset to 0. If any MMC counter is read with the <code>EMAC_MMC_CTL.RDRST</code> bit set, then that counter is also cleared in this mode.

Table 24-138: EMAC_MMC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	RDRST	Read Reset. The EMAC_MMC_CTL.RDRST bit, when set, resets the MMC counters to zero after Read (self-clearing after reset). The counters are cleared when the least significant byte lane (bits[7:0]) is read.
1 (R/W)	NOROLL	No Rollover. The EMAC_MMC_CTL.NOROLL bit, when set, prevents counter rolls over to 0 after reaching max.
0 (R/W)	RST	Reset. The EMAC_MMC_CTL.RST bit, when set, resets all counters. This bit is cleared automatically after 1 clock cycle.

MMC Rx Interrupt Mask Register

The `EMAC_MMC_RXIMSK` register enables (unmasks) MMC receive interrupts.

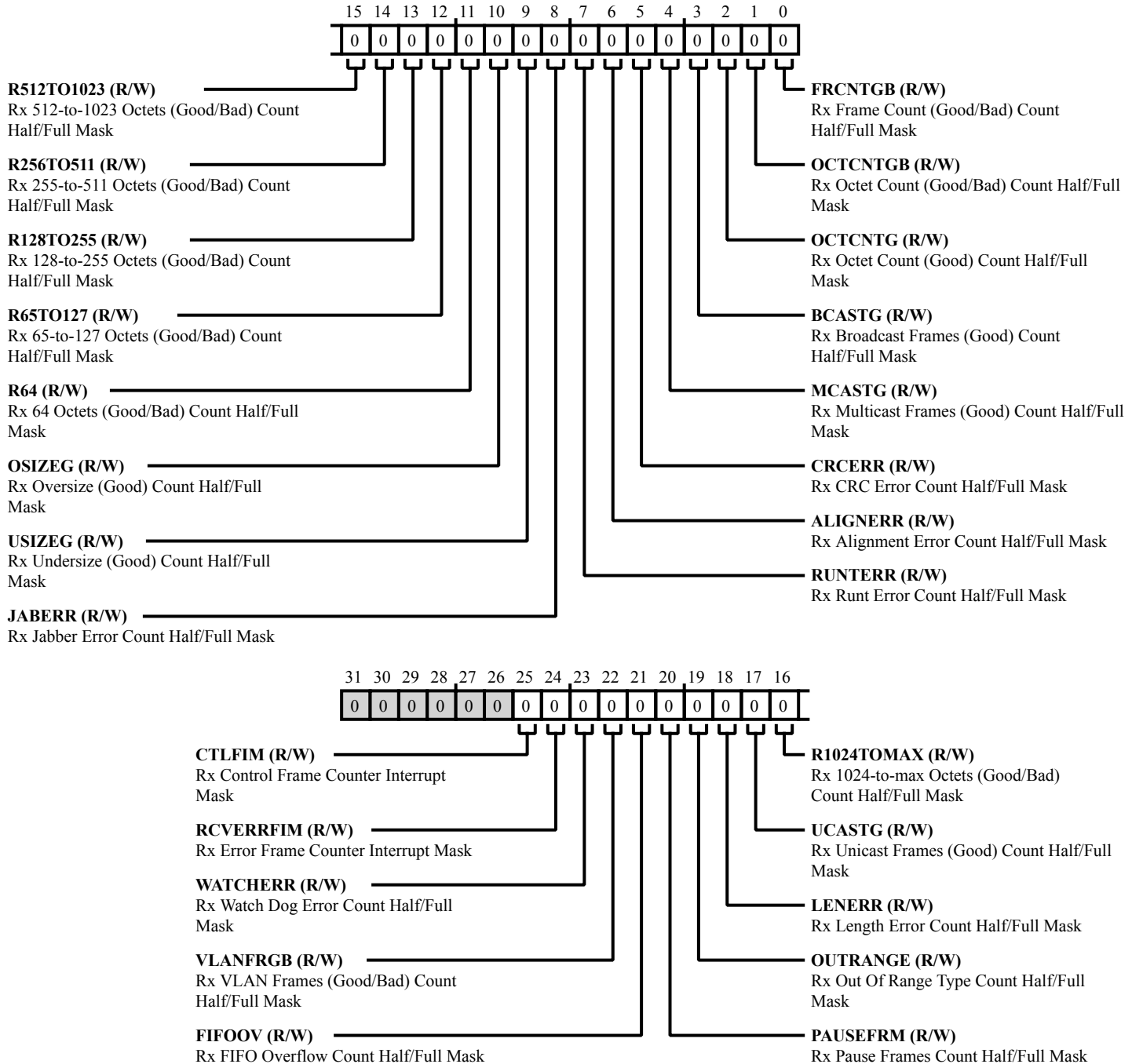


Figure 24-107: EMAC_MMC_RXIMSK Register Diagram

Table 24-139: EMAC_MMC_RXIMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	CTLFIM	Rx Control Frame Counter Interrupt Mask. The EMAC_MMC_RXIMSK.CTLFIM bit, masks the interrupt when the rxctrlframes_g counter reaches half of the maximum value or the maximum value.
24 (R/W)	RCVERRFIM	Rx Error Frame Counter Interrupt Mask. The EMAC_MMC_RXIMSK.RCVERRFIM bit, masks the interrupt when the rxrcverror counter reaches half of the maximum value or the maximum value.
23 (R/W)	WATCHERR	Rx Watch Dog Error Count Half/Full Mask. The EMAC_MMC_RXIMSK.WATCHERR bit, when set, masks the interrupt when EMAC_RXWDOG_ERR counter reaches full or half.
22 (R/W)	VLANFRGB	Rx VLAN Frames (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.VLANFRGB bit, when set, masks the interrupt when EMAC_RXVLANFRM_GB counter reaches full or half.
21 (R/W)	FIFOOV	Rx FIFO Overflow Count Half/Full Mask. The EMAC_MMC_RXIMSK.FIFOOV bit, when set, masks the interrupt when EMAC_RXFIFO_OVF counter reaches full or half.
20 (R/W)	PAUSEFRM	Rx Pause Frames Count Half/Full Mask. The EMAC_MMC_RXIMSK.PAUSEFRM bit, when set, masks the interrupt when EMAC_RXPAUSEFRM counter reaches full or half.
19 (R/W)	OUTRANGE	Rx Out Of Range Type Count Half/Full Mask. The EMAC_MMC_RXIMSK.OUTRANGE bit, when set, masks the interrupt when EMAC_RXOORTYPE counter reaches full or half.
18 (R/W)	LENERR	Rx Length Error Count Half/Full Mask. The EMAC_MMC_RXIMSK.LENERR bit, when set, masks the interrupt when EMAC_RXLEN_ERR counter reaches full or half.
17 (R/W)	UCASTG	Rx Unicast Frames (Good) Count Half/Full Mask. The EMAC_MMC_RXIMSK.UCASTG bit, when set, masks the interrupt when EMAC_RXUCASTFRM_G counter reaches full or half.
16 (R/W)	R1024TOMAX	Rx 1024-to-max Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.R1024TOMAX bit, when set, masks the interrupt when EMAC_RX1024TOMAX_GB counter reaches full or half.
15 (R/W)	R512TO1023	Rx 512-to-1023 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.R512TO1023 bit, when set, masks the interrupt when EMAC_RX512TO1023_GB counter reaches full or half.
14 (R/W)	R256TO511	Rx 255-to-511 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.R256TO511 bit, when set, masks the interrupt when EMAC_RX256TO511_GB counter reaches full or half.

Table 24-139: EMAC_MMC_RXIMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	R128TO255	Rx 128-to-255 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.R128TO255 bit, when set, masks the interrupt when EMAC_RX128TO255_GB counter reaches full or half.
12 (R/W)	R65TO127	Rx 65-to-127 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.R65TO127 bit, when set, masks the interrupt when EMAC_RX65TO127_GB counter reaches full or half.
11 (R/W)	R64	Rx 64 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.R64 bit, when set, masks the interrupt when EMAC_RX64_GB counter reaches full or half.
10 (R/W)	OSIZEG	Rx Oversize (Good) Count Half/Full Mask. The EMAC_MMC_RXIMSK.OSIZEG bit, when set, masks the interrupt when EMAC_RXOSIZE_G counter reaches full or half.
9 (R/W)	USIZEG	Rx Undersize (Good) Count Half/Full Mask. The EMAC_MMC_RXIMSK.USIZEG bit, when set, masks the interrupt when EMAC_RXUSIZE_G counter reaches full or half.
8 (R/W)	JABERR	Rx Jabber Error Count Half/Full Mask. The EMAC_MMC_RXIMSK.JABERR bit, when set, masks the interrupt when EMAC_RXJAB_ERR counter reaches full or half.
7 (R/W)	RUNTERR	Rx Runt Error Count Half/Full Mask. The EMAC_MMC_RXIMSK.RUNTERR bit, when set, masks the interrupt when EMAC_RXRUNT_ERR counter reaches full or half.
6 (R/W)	ALIGNERR	Rx Alignment Error Count Half/Full Mask. The EMAC_MMC_RXIMSK.ALIGNERR bit, when set, masks the interrupt when EMAC_RXALIGN_ERR counter reaches full or half.
5 (R/W)	CRCERR	Rx CRC Error Count Half/Full Mask. The EMAC_MMC_RXIMSK.CRCERR bit, when set, masks the interrupt when EMAC_RXCRC_ERR counter reaches full or half.
4 (R/W)	MCASTG	Rx Multicast Frames (Good) Count Half/Full Mask. The EMAC_MMC_RXIMSK.MCASTG bit, when set, masks the interrupt when EMAC_RXMCASTFRM_G counter reaches full or half.
3 (R/W)	BCASTG	Rx Broadcast Frames (Good) Count Half/Full Mask. The EMAC_MMC_RXIMSK.BCASTG bit, when set, masks the interrupt when EMAC_RXBCASTFRM_G counter reaches full or half.

Table 24-139: EMAC_MMC_RXIMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	OCTCNTG	Rx Octet Count (Good) Count Half/Full Mask. The EMAC_MMC_RXIMSK.OCTCNTG bit, when set, masks the interrupt when EMAC_RXOCTCNT_G counter reaches full or half.
1 (R/W)	OCTCNTGB	Rx Octet Count (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.OCTCNTGB bit, when set, masks the interrupt when EMAC_RXOCTCNT_GB counter reaches half or full.
0 (R/W)	FRCNTGB	Rx Frame Count (Good/Bad) Count Half/Full Mask. The EMAC_MMC_RXIMSK.FRCNTGB bit, when set, masks the interrupt when EMAC_RXFRMCNT_GB counter reaches half or full.

MMC Rx Interrupt Register

The `EMAC_MMC_RXINT` register indicates status of MMC receive interrupts.

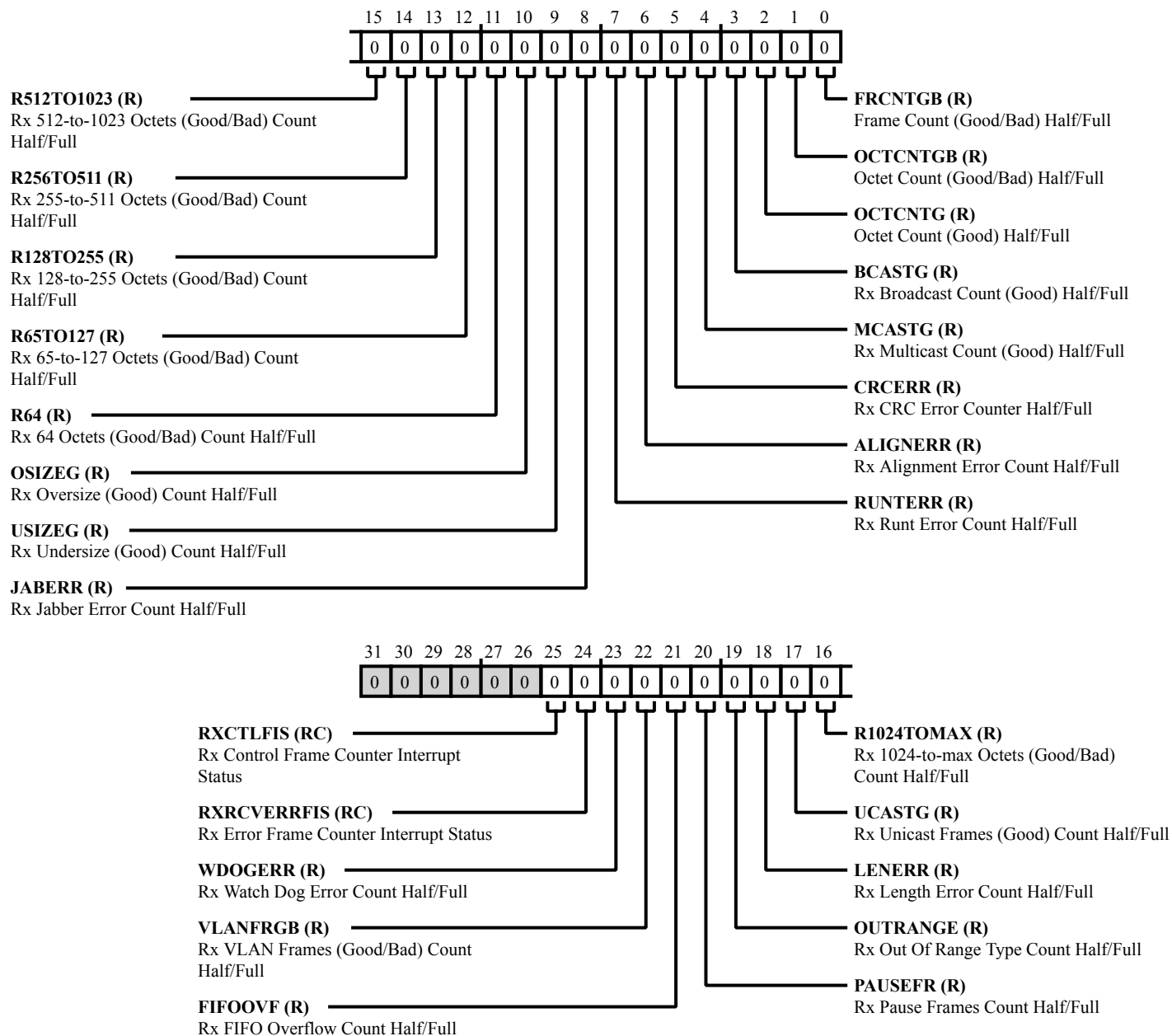


Figure 24-108: EMAC_MMC_RXINT Register Diagram

Table 24-140: EMAC_MMC_RXINT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (RC/NW)	RXCTLFIS	Rx Control Frame Counter Interrupt Status. The EMAC_MMC_RXINT.RXCTLFIS bit is set when the rxctrlframes_g counter reaches half of the maximum value or the maximum value.
24 (RC/NW)	RXRCVERRFIS	Rx Error Frame Counter Interrupt Status. The EMAC_MMC_RXINT.RXRCVERRFIS bit is set when the rxrcverror counter reaches half of the maximum value or the maximum value.
23 (R/NW)	WDOGERR	Rx Watch Dog Error Count Half/Full. The EMAC_MMC_RXINT.WDOGERR bit is set when the EMAC_RXWDG_ERR counter reaches full or half.
22 (R/NW)	VLANFRGB	Rx VLAN Frames (Good/Bad) Count Half/Full. The EMAC_MMC_RXINT.VLANFRGB bit is set when EMAC_RXVLANFRM_GB counter reaches full or half.
21 (R/NW)	FIFOOVF	Rx FIFO Overflow Count Half/Full. The EMAC_MMC_RXINT.FIFOOVF bit is set when EMAC_RXFIFO_OVF counter reaches full or half.
20 (R/NW)	PAUSEFR	Rx Pause Frames Count Half/Full. The EMAC_MMC_RXINT.PAUSEFR bit is set when EMAC_RXPAUSEFRM counter reaches full or half.
19 (R/NW)	OUTRANGE	Rx Out Of Range Type Count Half/Full. The EMAC_MMC_RXINT.OUTRANGE bit is set when EMAC_RXOORTYPE counter reaches full or half.
18 (R/NW)	LENERR	Rx Length Error Count Half/Full. The EMAC_MMC_RXINT.LENERR bit is set when EMAC_RXLEN_ERR counter reaches full or half.
17 (R/NW)	UCASTG	Rx Unicast Frames (Good) Count Half/Full. The EMAC_MMC_RXINT.UCASTG bit is set when EMAC_RXUCASTFRM_G counter reaches full or half.
16 (R/NW)	R1024TOMAX	Rx 1024-to-max Octets (Good/Bad) Count Half/Full. The EMAC_MMC_RXINT.R1024TOMAX bit is set when EMAC_RX1024TOMAX_GB counter reaches full or half.
15 (R/NW)	R512TO1023	Rx 512-to-1023 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_RXINT.R512TO1023 bit is set when EMAC_RX512TO1023_GB counter reaches full or half.
14 (R/NW)	R256TO511	Rx 255-to-511 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_RXINT.R256TO511 bit is set when EMAC_RX256TO511_GB counter reaches full or half.

Table 24-140: EMAC_MMC_RXINT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/NW)	R128TO255	Rx 128-to-255 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_RXINT.R128TO255 bit is set when EMAC_RX128TO255_GB counter reaches full or half.
12 (R/NW)	R65TO127	Rx 65-to-127 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_RXINT.R65TO127 bit is set when EMAC_RX65TO127_GB counter reaches full or half.
11 (R/NW)	R64	Rx 64 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_RXINT.R64 bit is set when EMAC_RX64_GB counter reaches full or half.
10 (R/NW)	OSIZEG	Rx Oversize (Good) Count Half/Full. The EMAC_MMC_RXINT.OSIZEG bit is set when EMAC_RXOSIZE_G counter reaches full or half.
9 (R/NW)	USIZEG	Rx Undersize (Good) Count Half/Full. The EMAC_MMC_RXINT.USIZEG bit is set when EMAC_RXUSIZE_G counter reaches full or half.
8 (R/NW)	JABERR	Rx Jabber Error Count Half/Full. The EMAC_MMC_RXINT.JABERR bit is set when EMAC_RXJAB_ERR counter reaches full or half.
7 (R/NW)	RUNTERR	Rx Runt Error Count Half/Full. The EMAC_MMC_RXINT.RUNTERR bit is set when EMAC_RXRUNT_ERR counter reaches full or half.
6 (R/NW)	ALIGNERR	Rx Alignment Error Count Half/Full. The EMAC_MMC_RXINT.ALIGNERR bit is set when EMAC_RXALIGN_ERR counter reaches full or half.
5 (R/NW)	CRCERR	Rx CRC Error Counter Half/Full. The EMAC_MMC_RXINT.CRCERR bit is set when EMAC_RXCRC_ERR counter reaches full or half.
4 (R/NW)	MCASTG	Rx Multicast Count (Good) Half/Full. The EMAC_MMC_RXINT.MCASTG bit is set when EMAC_RXMCASTFRM_G counter reaches full or half.
3 (R/NW)	BCASTG	Rx Broadcast Count (Good) Half/Full. The EMAC_MMC_RXINT.BCASTG bit is set when EMAC_RXBCASTFRM_G counter reaches full or half.

Table 24-140: EMAC_MMC_RXINT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/NW)	OCTCNTG	Octet Count (Good) Half/Full. The EMAC_MMC_RXINT.OCTCNTG bit is set when EMAC_RXOCTCNT_G counter reaches full or half.
1 (R/NW)	OCTCNTGB	Octet Count (Good/Bad) Half/Full. The EMAC_MMC_RXINT.OCTCNTGB bit is set when EMAC_RXOCTCNT_GB counter reaches half or full.
0 (R/NW)	FRCNTGB	Frame Count (Good/Bad) Half/Full. The EMAC_MMC_RXINT.FRCNTGB bit is set when EMAC_RXFRMCNT_GB counter reaches half or full.

MMC TX Interrupt Mask Register

The `EMAC_MMC_TXIMSK` register enables (unmasks) MMC transmit interrupts.

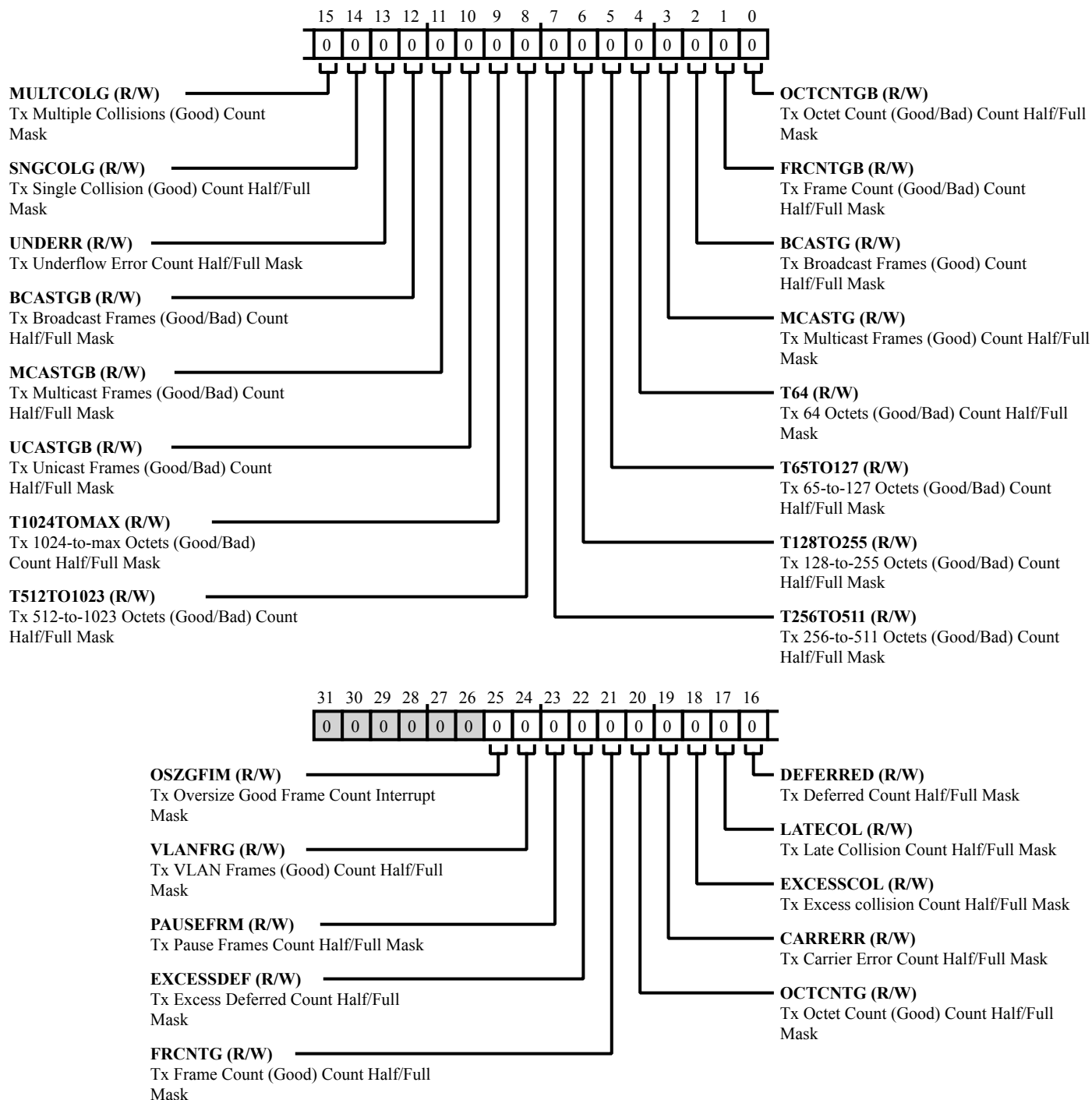


Figure 24-109: EMAC_MMC_TXIMSK Register Diagram

Table 24-141: EMAC_MMC_TXIMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	OSZGFIM	Tx Oversize Good Frame Count Interrupt Mask. The EMAC_MMC_TXIMSK.OSZGFIM bit masks the interrupt when the txoversize_g counter reaches half of the maximum value or the maximum value.
24 (R/W)	VLANFRG	Tx VLAN Frames (Good) Count Half/Full Mask. The EMAC_MMC_TXIMSK.VLANFRG bit, when set, masks the interrupt when EMAC_TXVLANFRM_G counter reaches full or half.
23 (R/W)	PAUSEFRM	Tx Pause Frames Count Half/Full Mask. The EMAC_MMC_TXIMSK.PAUSEFRM bit, when set, masks the interrupt when EMAC_TXPAUSEFRM counter reaches full or half.
22 (R/W)	EXCESSDEF	Tx Excess Deferred Count Half/Full Mask. The EMAC_MMC_TXIMSK.EXCESSDEF bit, when set, masks the interrupt when EMAC_TXEXCESSDEF counter reaches full or half.
21 (R/W)	FRCNTG	Tx Frame Count (Good) Count Half/Full Mask. The EMAC_MMC_TXIMSK.FRCNTG bit, when set, masks the interrupt when EMAC_TXFRMCNT_G counter reaches full or half.
20 (R/W)	OCTCNTG	Tx Octet Count (Good) Count Half/Full Mask. The EMAC_MMC_TXIMSK.OCTCNTG bit, when set, masks the interrupt when EMAC_TXOCTCNT_G counter reaches full or half.
19 (R/W)	CARRERR	Tx Carrier Error Count Half/Full Mask. The EMAC_MMC_TXIMSK.CARRERR bit, when set, masks the interrupt when EMAC_TXCARR_ERR counter reaches full or half.
18 (R/W)	EXCESSCOL	Tx Excess collision Count Half/Full Mask. The EMAC_MMC_TXIMSK.EXCESSCOL bit, when set, masks the interrupt when EMAC_TXEXCESSCOL counter reaches full or half.
17 (R/W)	LATECOL	Tx Late Collision Count Half/Full Mask. The EMAC_MMC_TXIMSK.LATECOL bit, when set, masks the interrupt when EMAC_TXLATECOL counter reaches full or half.
16 (R/W)	DEFERRED	Tx Deferred Count Half/Full Mask. The EMAC_MMC_TXIMSK.DEFERRED bit, when set, masks the interrupt when EMAC_TXDEFERRED counter reaches full or half.
15 (R/W)	MULTCOLG	Tx Multiple Collisions (Good) Count Mask. The EMAC_MMC_TXIMSK.MULTCOLG bit, when set, masks the interrupt when EMAC_TXMULTCOL_G counter reaches full or half.
14 (R/W)	SNGCOLG	Tx Single Collision (Good) Count Half/Full Mask. The EMAC_MMC_TXIMSK.SNGCOLG bit, when set, masks the interrupt when EMAC_TXSNGCOL_G counter reaches full or half.

Table 24-141: EMAC_MMC_TXIMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	UNDERR	Tx Underflow Error Count Half/Full Mask. The EMAC_MMC_TXIMSK.UNDERR bit, when set, masks the interrupt when EMAC_TXUNDR_ERR counter reaches full or half.
12 (R/W)	BCASTGB	Tx Broadcast Frames (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.BCASTGB bit, when set, masks the interrupt when EMAC_TXBCASTFRM_GB counter reaches full or half.
11 (R/W)	MCASTGB	Tx Multicast Frames (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.MCASTGB bit, when set, masks the interrupt when EMAC_TXMCASTFRM_GB counter reaches full or half.
10 (R/W)	UCASTGB	Tx Unicast Frames (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.UCASTGB bit, when set, masks the interrupt when EMAC_TXUCASTFRM_GB counter reaches full or half.
9 (R/W)	T1024TOMAX	Tx 1024-to-max Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.T1024TOMAX bit, when set, masks the interrupt when EMAC_TX1024TOMAX_GB counter reaches full or half.
8 (R/W)	T512TO1023	Tx 512-to-1023 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.T512TO1023 bit, when set, masks the interrupt when EMAC_TX512TO1023_GB counter reaches full or half.
7 (R/W)	T256TO511	Tx 256-to-511 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.T256TO511 bit, when set, masks the interrupt when EMAC_TX256TO511_GB counter reaches full or half.
6 (R/W)	T128TO255	Tx 128-to-255 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.T128TO255 bit, when set, masks the interrupt when EMAC_TX128TO255_GB counter reaches full or half.
5 (R/W)	T65TO127	Tx 65-to-127 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.T65TO127 bit, when set, masks the interrupt when EMAC_TX65TO127_GB counter reaches full or half.
4 (R/W)	T64	Tx 64 Octets (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.T64 bit, when set, masks the interrupt when EMAC_TX64_GB counter reaches full or half.
3 (R/W)	MCASTG	Tx Multicast Frames (Good) Count Half/Full Mask. The EMAC_MMC_TXIMSK.MCASTG bit, when set, masks the interrupt when EMAC_TXMCASTFRM_G counter reaches full or half.

Table 24-141: EMAC_MMC_TXIMSK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	BCASTG	Tx Broadcast Frames (Good) Count Half/Full Mask. The EMAC_MMC_TXIMSK.BCASTG bit, when set, masks the interrupt when EMAC_TXBCASTFRM_G counter reaches full or half.
1 (R/W)	FRCNTGB	Tx Frame Count (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.FRCNTGB bit, when set, masks the interrupt when EMAC_TXFRMCNT_GB counter reaches full or half.
0 (R/W)	OCTCNTGB	Tx Octet Count (Good/Bad) Count Half/Full Mask. The EMAC_MMC_TXIMSK.OCTCNTGB bit, when set, masks the interrupt when EMAC_TXOCTCNT_GB counter reaches full or half.

MMC Tx Interrupt Register

The `EMAC_MMC_TXINT` register indicates status of MMC transmit interrupts.

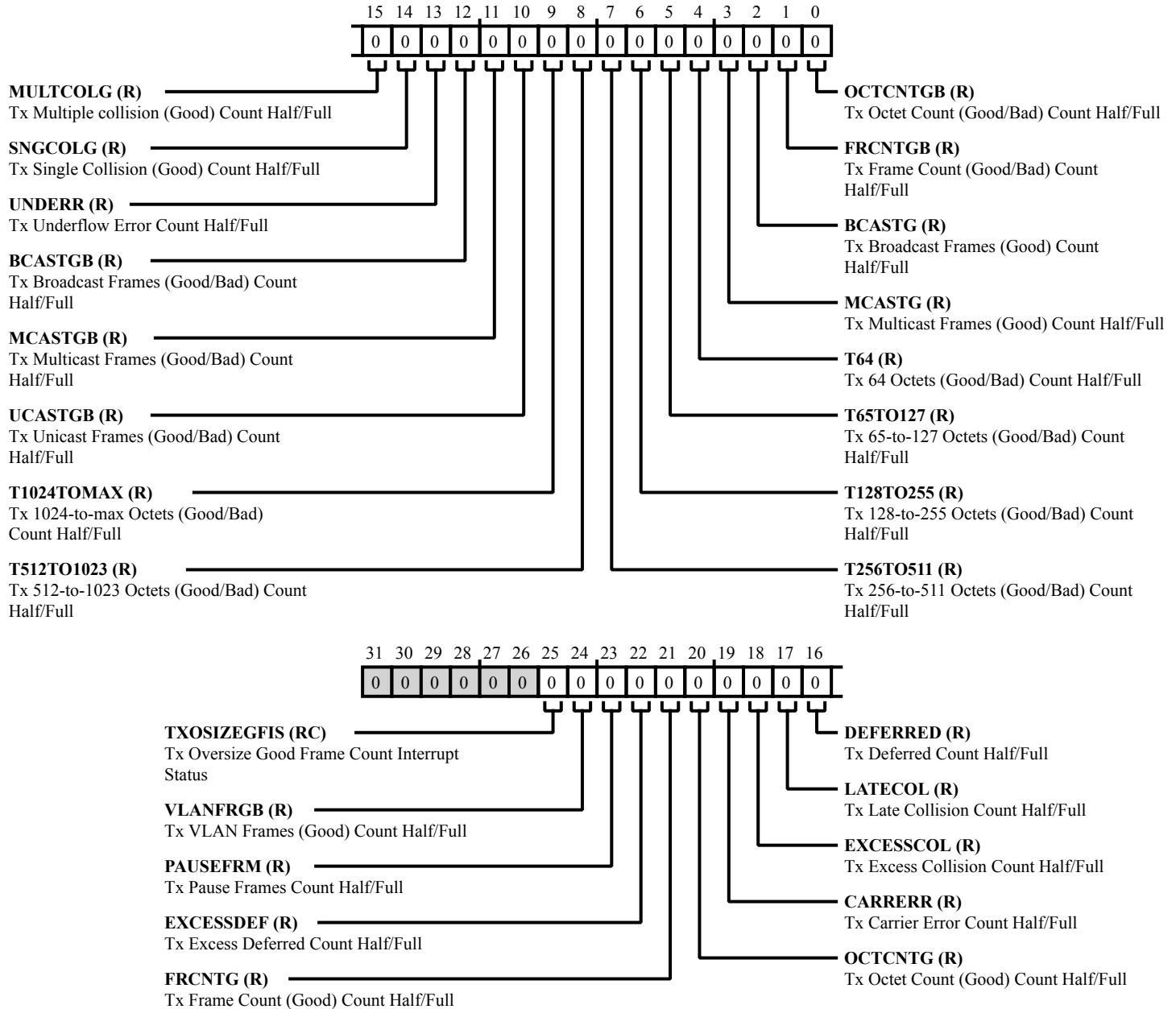


Figure 24-110: EMAC_MMC_TXINT Register Diagram

Table 24-142: EMAC_MMC_TXINT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (RC/NW)	TXOSIZEGFIS	Tx Oversize Good Frame Count Interrupt Status. The EMAC_MMC_TXINT.TXOSIZEGFIS bit is set when the txoversize_g counter reaches half of the maximum value or the maximum value.
24 (R/NW)	VLANFRGB	Tx VLAN Frames (Good) Count Half/Full. The EMAC_MMC_TXINT.VLANFRGB bit is set when EMAC_TXVLANFRM_G counter reaches full or half.
23 (R/NW)	PAUSEFRM	Tx Pause Frames Count Half/Full. The EMAC_MMC_TXINT.PAUSEFRM bit is set when EMAC_TXPAUSEFRM counter reaches full or half.
22 (R/NW)	EXCESSDEF	Tx Excess Deferred Count Half/Full. The EMAC_MMC_TXINT.EXCESSDEF bit is set when EMAC_TXEXCESSDEF counter reaches full or half.
21 (R/NW)	FRCNTG	Tx Frame Count (Good) Count Half/Full. The EMAC_MMC_TXINT.FRCNTG bit is set when EMAC_TXFRMCNT_G counter reaches full or half.
20 (R/NW)	OCTCNTG	Tx Octet Count (Good) Count Half/Full. The EMAC_MMC_TXINT.OCTCNTG bit is set when EMAC_TXOCTCNT_G counter reaches full or half.
19 (R/NW)	CARRERR	Tx Carrier Error Count Half/Full. The EMAC_MMC_TXINT.CARRERR bit is set when EMAC_TXCARR_ERR counter reaches full or half.
18 (R/NW)	EXCESSCOL	Tx Excess Collision Count Half/Full. The EMAC_MMC_TXINT.EXCESSCOL bit is set when EMAC_TXEXCESSCOL counter reaches full or half.
17 (R/NW)	LATECOL	Tx Late Collision Count Half/Full. The EMAC_MMC_TXINT.LATECOL bit is set when EMAC_TXLATECOL counter reaches full or half.
16 (R/NW)	DEFERRED	Tx Deferred Count Half/Full. The EMAC_MMC_TXINT.DEFERRED bit is set when EMAC_TXDEFERRED counter reaches full or half.
15 (R/NW)	MULTCOLG	Tx Multiple collision (Good) Count Half/Full. The EMAC_MMC_TXINT.MULTCOLG bit is set when EMAC_TXMULTCOL_G counter reaches full or half.
14 (R/NW)	SNGCOLG	Tx Single Collision (Good) Count Half/Full. The EMAC_MMC_TXINT.SNGCOLG bit is set when EMAC_TXSNGCOL_G counter reaches full or half.

Table 24-142: EMAC_MMC_TXINT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/NW)	UNDERR	Tx Underflow Error Count Half/Full. The EMAC_MMC_TXINT.UNDERR bit is set when EMAC_TXUNDR_ERR counter reaches full or half.
12 (R/NW)	BCASTGB	Tx Broadcast Frames (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.BCASTGB bit is set when EMAC_TXBCASTFRM_GB counter reaches full or half.
11 (R/NW)	MCASTGB	Tx Multicast Frames (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.MCASTGB bit is set when EMAC_TXMCASTFRM_GB counter reaches full or half.
10 (R/NW)	UCASTGB	Tx Unicast Frames (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.UCASTGB bit is set when EMAC_TXUCASTFRM_GB counter reaches full or half.
9 (R/NW)	T1024TOMAX	Tx 1024-to-max Octets (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.T1024TOMAX bit is set when EMAC_TX1024TOMAX_GB counter reaches full or half.
8 (R/NW)	T512TO1023	Tx 512-to-1023 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.T512TO1023 bit is set when EMAC_TX512TO1023_GB counter reaches full or half.
7 (R/NW)	T256TO511	Tx 256-to-511 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.T256TO511 bit is set when EMAC_TX256TO511_GB counter reaches full or half.
6 (R/NW)	T128TO255	Tx 128-to-255 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.T128TO255 bit is set when EMAC_TX128TO255_GB counter reaches full or half.
5 (R/NW)	T65TO127	Tx 65-to-127 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.T65TO127 bit is set when EMAC_TX65TO127_GB counter reaches full or half.
4 (R/NW)	T64	Tx 64 Octets (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.T64 bit is set when EMAC_TX64_GB counter reaches full or half.
3 (R/NW)	MCASTG	Tx Multicast Frames (Good) Count Half/Full. The EMAC_MMC_TXINT.MCASTG bit is set when EMAC_TXMCASTFRM_G counter reaches full or half.

Table 24-142: EMAC_MMC_TXINT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/NW)	BCASTG	Tx Broadcast Frames (Good) Count Half/Full. The EMAC_MMC_TXINT.BCASTG bit is set when EMAC_TXBCASTFRM_G counter reaches full or half.
1 (R/NW)	FRCNTGB	Tx Frame Count (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.FRCNTGB bit is set when EMAC_TXFRMCNT_GB counter reaches full or half.
0 (R/NW)	OCTCNTGB	Tx Octet Count (Good/Bad) Count Half/Full. The EMAC_MMC_TXINT.OCTCNTGB bit is set when EMAC_TXOCTCNT_GB counter reaches full or half.

Rx 1024- to Max-Byte Frames (Good/Bad) Register

The `EMAC_RX1024TOMAX_GB` register contains a count of the number of good and bad frames received with length between 1024 and maxsize (inclusive) bytes, exclusive of preamble.

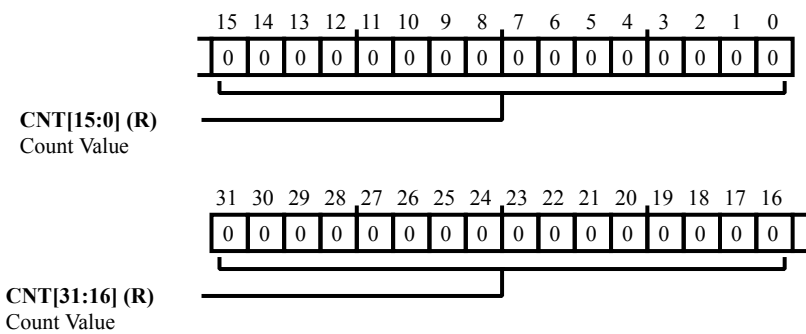


Figure 24-111: EMAC_RX1024TOMAX_GB Register Diagram

Table 24-143: EMAC_RX1024TOMAX_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx 128- to 255-Byte Frames (Good/Bad) Register

The `EMAC_RX128TO255_GB` register contains a count of the number of good and bad frames received with length between 128 and 255 (inclusive) bytes, exclusive of preamble.

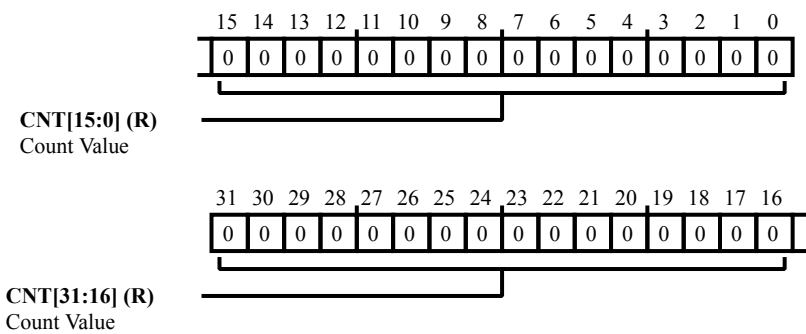


Figure 24-112: EMAC_RX128TO255_GB Register Diagram

Table 24-144: EMAC_RX128TO255_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx 256- to 511-Byte Frames (Good/Bad) Register

The `EMAC_RX256TO511_GB` register contains a count of the number of good and bad frames received with length between 256 and 511 (inclusive) bytes, exclusive of preamble.

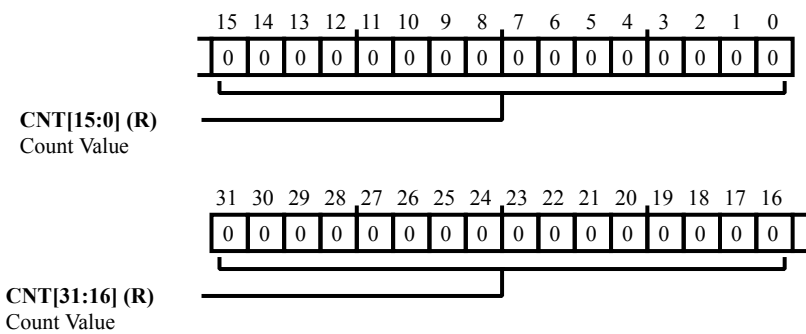


Figure 24-113: EMAC_RX256TO511_GB Register Diagram

Table 24-145: EMAC_RX256TO511_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx 512- to 1023-Byte Frames (Good/Bad) Register

The `EMAC_RX512TO1023_GB` register contains a count of the number of good and bad frames received with length between 512 and 1023 (inclusive) bytes, exclusive of preamble.

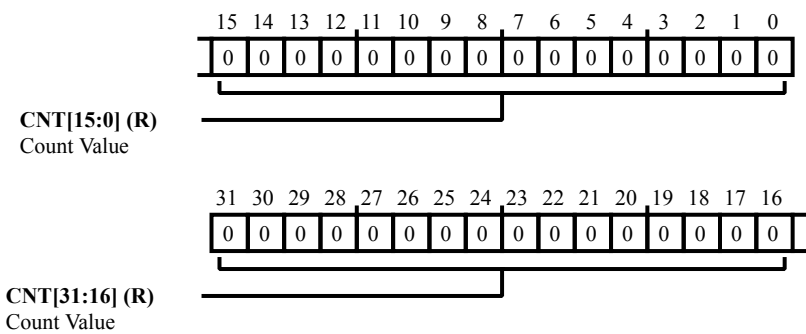


Figure 24-114: EMAC_RX512TO1023_GB Register Diagram

Table 24-146: EMAC_RX512TO1023_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx 64-Byte Frames (Good/Bad) Register

The `EMAC_RX64_GB` register contains a count of the number of good and bad frames received with length 64 bytes, exclusive of preamble.

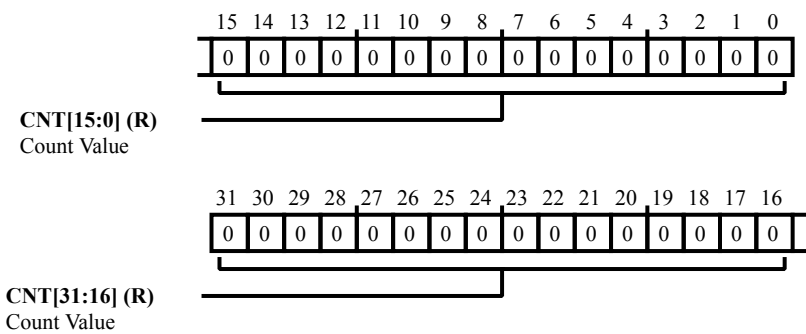


Figure 24-115: EMAC_RX64_GB Register Diagram

Table 24-147: EMAC_RX64_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx 65- to 127-Byte Frames (Good/Bad) Register

The `EMAC_RX65TO127_GB` register contains a count of the number of good and bad frames received with length between 65 and 127 (inclusive) bytes, exclusive of preamble.

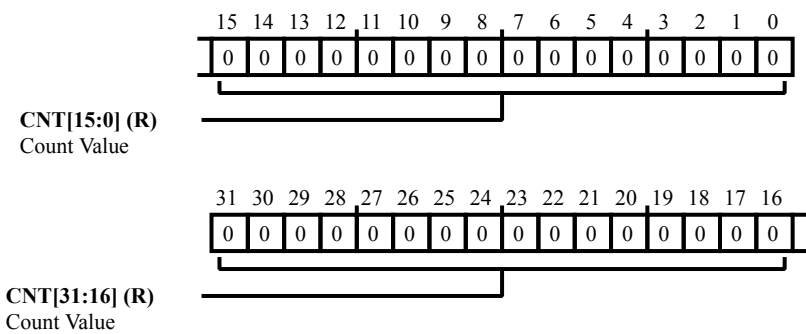


Figure 24-116: EMAC_RX65TO127_GB Register Diagram

Table 24-148: EMAC_RX65TO127_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx alignment Error Register

The `EMAC_RXALIGN_ERR` register contains a count of the number of frames received with alignment (dribble) error. Valid only in 10/100 mode.

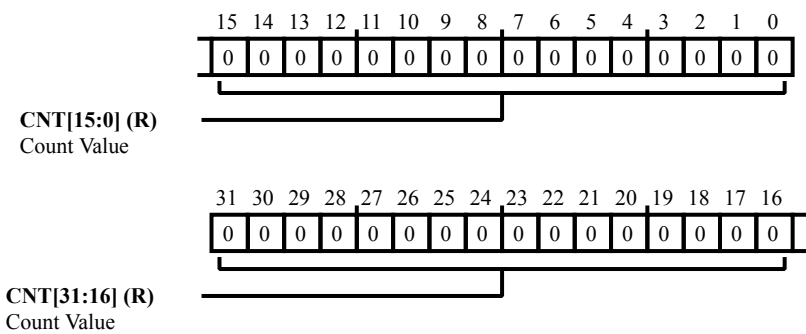


Figure 24-117: EMAC_RXALIGN_ERR Register Diagram

Table 24-149: EMAC_RXALIGN_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Broadcast Frames (Good) Register

The `EMAC_RXBCASTFRM_G` register contains a count of the number of good broadcast frames received.

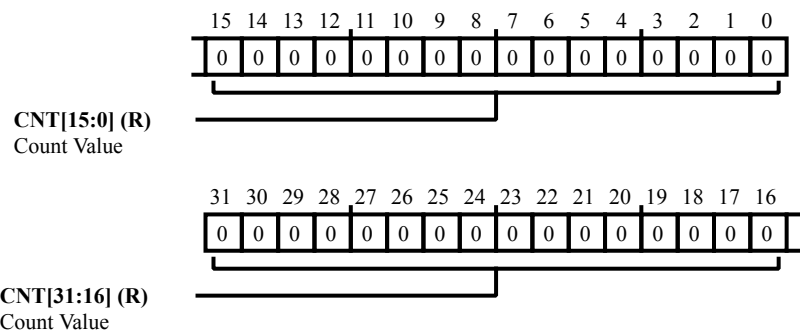


Figure 24-118: EMAC_RXBCASTFRM_G Register Diagram

Table 24-150: EMAC_RXBCASTFRM_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx CRC Error Register

The `EMAC_RXCRC_ERR` register contains a count of the number of frames received with CRC error.

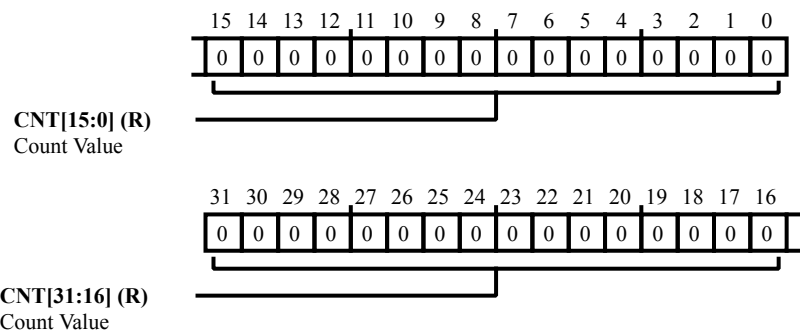


Figure 24-119: EMAC_RXCRC_ERR Register Diagram

Table 24-151: EMAC_RXCRC_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Good Control Frames Register

The `EMAC_RXCTLFRM_G` register contains a count of the number of good control frames received

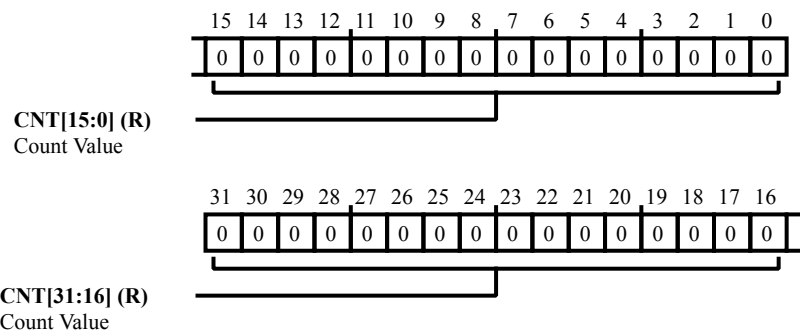


Figure 24-120: EMAC_RXCTLFRM_G Register Diagram

Table 24-152: EMAC_RXCTLFRM_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx FIFO Overflow Register

The `EMAC_RXFIFO_OVF` register contains a count of the number of missed received frames due to FIFO overflow.

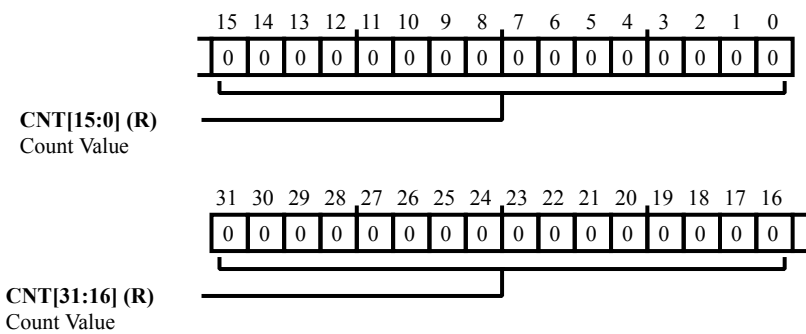


Figure 24-121: EMAC_RXFIFO_OVF Register Diagram

Table 24-153: EMAC_RXFIFO_OVF Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Frame Count (Good/Bad) Register

The `EMAC_RXFRMCNT_GB` register contains a count of the number of good and bad frames received.

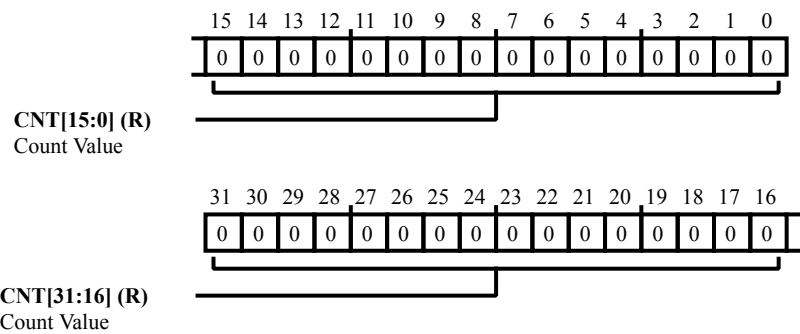


Figure 24-122: EMAC_RXFRMCNT_GB Register Diagram

Table 24-154: EMAC_RXFRMCNT_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx ICMP Error Frames Register

The `EMAC_RXICMP_ERR_FRM` register contains a count of the number of good IP datagrams whose ICMP payload has a checksum error.

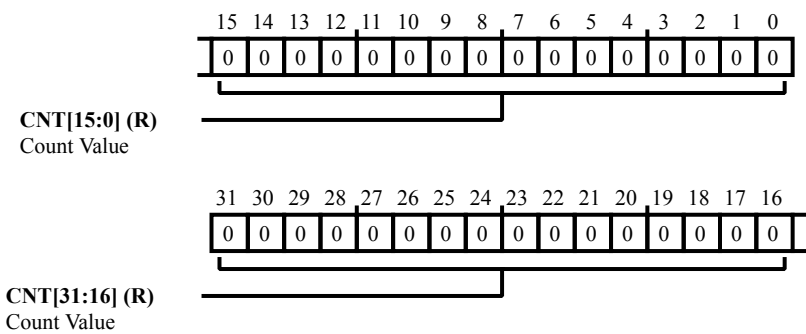


Figure 24-123: EMAC_RXICMP_ERR_FRM Register Diagram

Table 24-155: EMAC_RXICMP_ERR_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx ICMP Error Octets Register

The `EMAC_RXICMP_ERR_OCT` register contains a count of the number of bytes received in an ICMP segment with checksum errors.

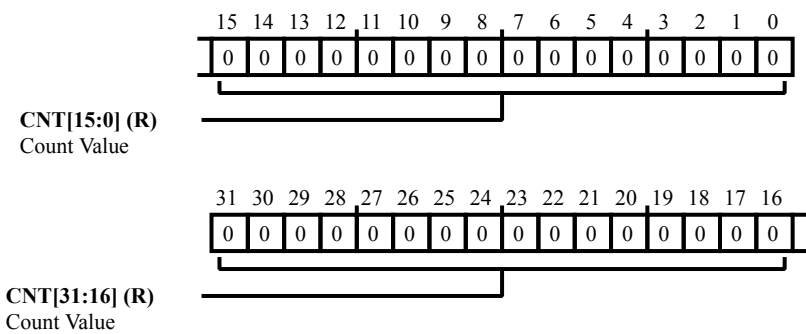


Figure 24-124: EMAC_RXICMP_ERR_OCT Register Diagram

Table 24-156: EMAC_RXICMP_ERR_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx ICMP Good Frames Register

The `EMAC_RXICMP_GD_FRM` register contains a count of the number of good IP datagrams with a good ICMP payload.

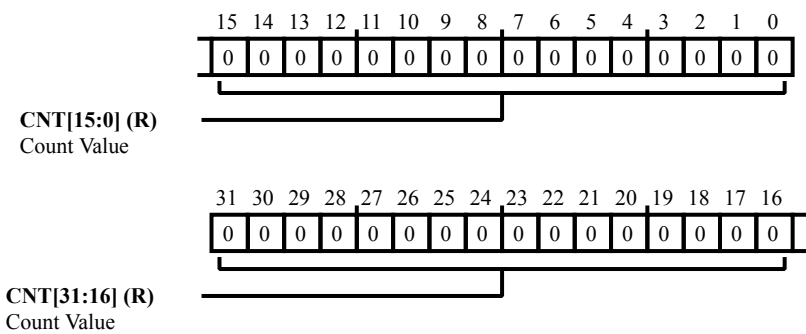


Figure 24-125: EMAC_RXICMP_GD_FRM Register Diagram

Table 24-157: EMAC_RXICMP_GD_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx ICMP Good Octets Register

The `EMAC_RXICMP_GD_OCT` register contains a count of the Number of bytes received in a good ICMP segment.

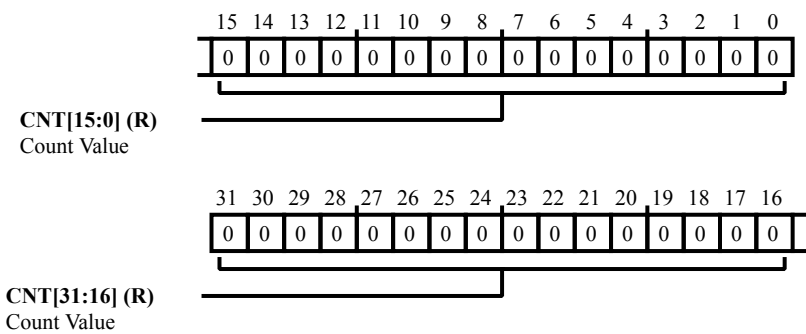


Figure 24-126: EMAC_RXICMP_GD_OCT Register Diagram

Table 24-158: EMAC_RXICMP_GD_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 Datagrams Fragmented Frames Register

The `EMAC_RXIPV4_FRAG_FRM` register contains a count of the number of good IPv4 datagrams with fragmentation.

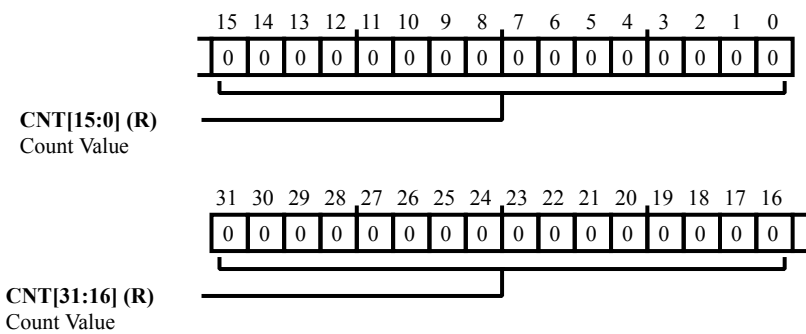


Figure 24-127: EMAC_RXIPV4_FRAG_FRM Register Diagram

Table 24-159: EMAC_RXIPV4_FRAG_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 Datagrams Fragmented Octets Register

The `EMAC_RXIPV4_FRAG_OCT` register contains a count of the number of bytes received in fragmented IPv4 datagrams. The value in the IPv4 headers Length field is used to update this counter.

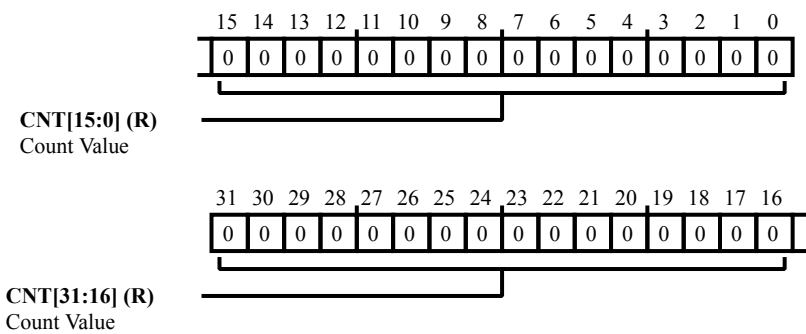


Figure 24-128: EMAC_RXIPV4_FRAG_OCT Register Diagram

Table 24-160: EMAC_RXIPV4_FRAG_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 Datagrams (Good) Register

The `EMAC_RXIPV4_GD_FRM` register contains a count of the number of good IPv4 datagrams received with the TCP, UDP, or ICMP payload.

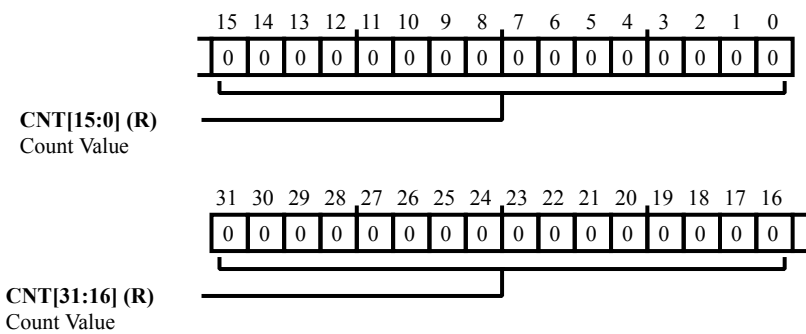


Figure 24-129: EMAC_RXIPV4_GD_FRM Register Diagram

Table 24-161: EMAC_RXIPV4_GD_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 Datagrams Good Octets Register

The `EMAC_RXIPV4_GD_OCT` register contains a count of the number of bytes received in good IPv4 datagrams encapsulating TCP, UDP, or ICMP data.

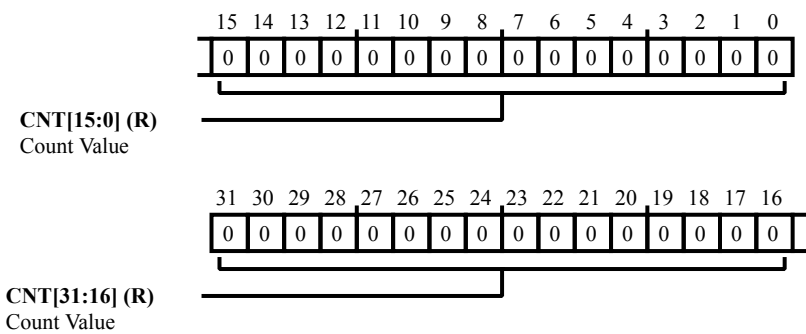


Figure 24-130: EMAC_RXIPV4_GD_OCT Register Diagram

Table 24-162: EMAC_RXIPV4_GD_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 Datagrams Header Errors Register

The `EMAC_RXIPV4_HDR_ERR_FRM` register contains a count of the number of IPv4 datagrams received with header (checksum, length, or version mismatch) errors.

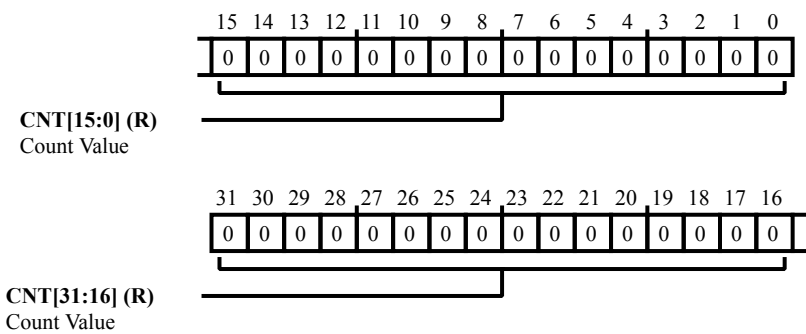


Figure 24-131: EMAC_RXIPV4_HDR_ERR_FRM Register Diagram

Table 24-163: EMAC_RXIPV4_HDR_ERR_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 Datagrams Header Errors Register

The `EMAC_RXIPV4_HDR_ERR_OCT` register contains a count of the number of bytes received in IPv4 datagrams with header errors (checksum, length, version mismatch). The value in the Length field of IPv4 header is used to update this counter.

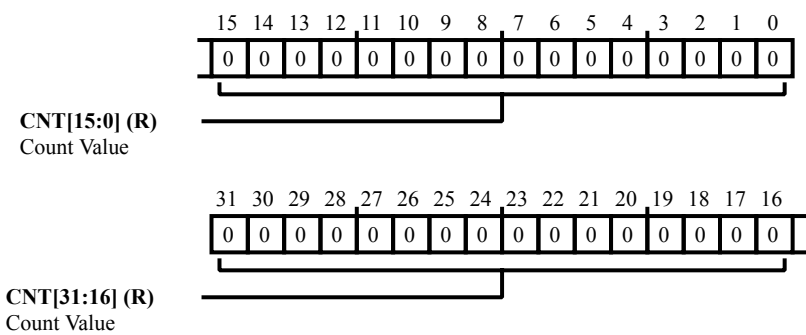


Figure 24-132: EMAC_RXIPV4_HDR_ERR_OCT Register Diagram

Table 24-164: EMAC_RXIPV4_HDR_ERR_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 Datagrams No Payload Frame Register

The `EMAC_RXIPV4_NOPAY_FRM` register contains a count of the number of IPv4 datagram frames received that did not have a TCP, UDP, or ICMP payload processed by the Checksum engine.

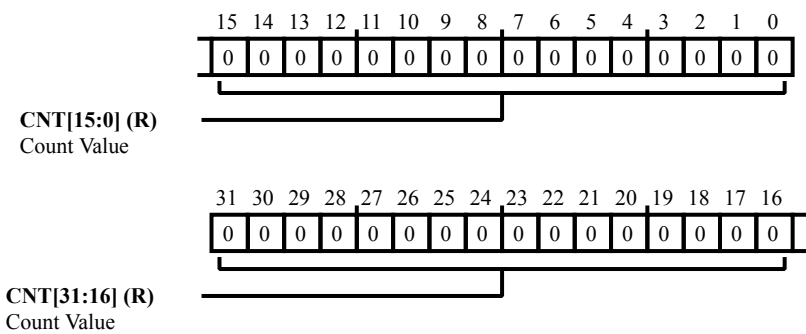


Figure 24-133: EMAC_RXIPV4_NOPAY_FRM Register Diagram

Table 24-165: EMAC_RXIPV4_NOPAY_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 Datagrams No Payload Octets Register

The `EMAC_RXIPV4_NOPAY_OCT` register contains a count of the number of bytes received in IPv4 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the IPv4 headers Length field is used to update this counter.

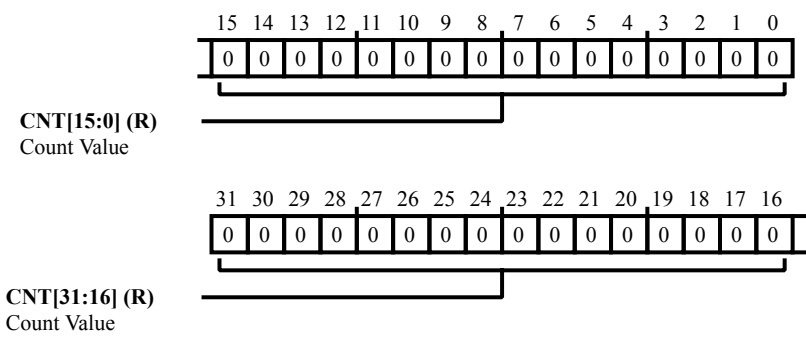


Figure 24-134: EMAC_RXIPV4_NOPAY_OCT Register Diagram

Table 24-166: EMAC_RXIPV4_NOPAY_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 UDP Disabled Frames Register

The `EMAC_RXIPV4_UDSBL_FRM` register contains a count of the number of good IPv4 datagrams received that had a UDP payload with checksum disabled.

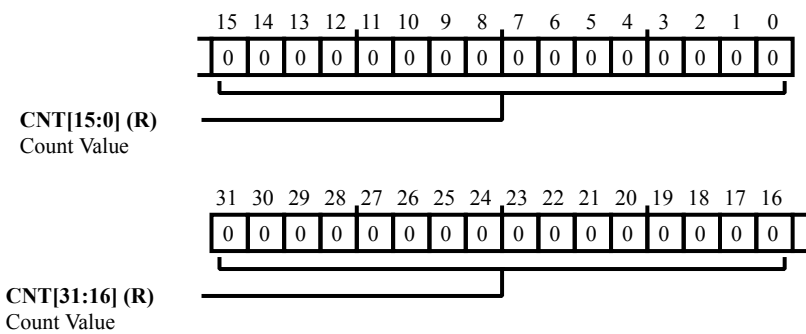


Figure 24-135: EMAC_RXIPV4_UDSBL_FRM Register Diagram

Table 24-167: EMAC_RXIPV4_UDSBL_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv4 UDP Disabled Octets Register

The `EMAC_RXIPV4_UDSBL_OCT` register contains a count of the number of bytes received in a UDP segment that had the UDP checksum disabled. This counter does not count IP Header bytes.

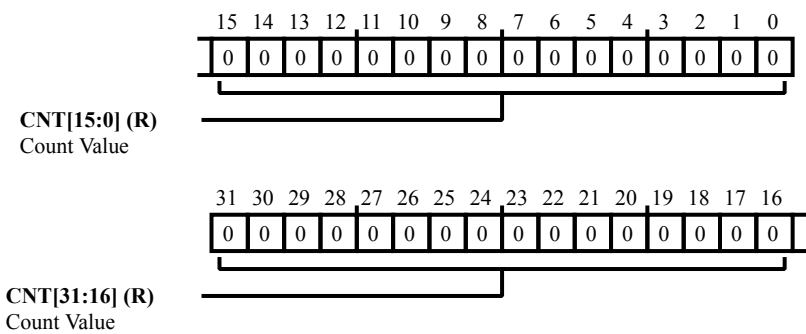


Figure 24-136: EMAC_RXIPV4_UDSBL_OCT Register Diagram

Table 24-168: EMAC_RXIPV4_UDSBL_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv6 Datagrams Good Frames Register

The `EMAC_RXIPV6_GD_FRM` register contains a count of the number of good IPv6 datagrams received with TCP, UDP, or ICMP payloads.

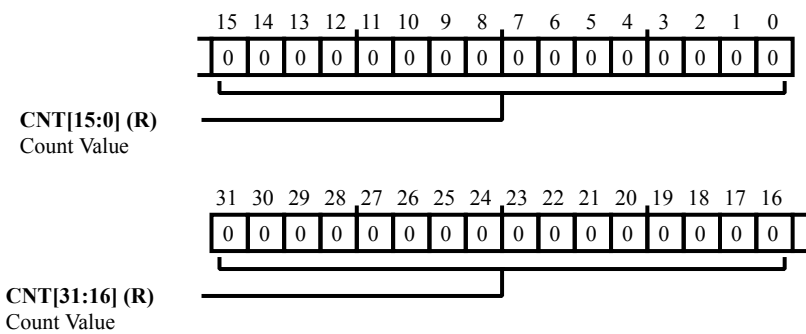


Figure 24-137: EMAC_RXIPV6_GD_FRM Register Diagram

Table 24-169: EMAC_RXIPV6_GD_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv6 Good Octets Register

The `EMAC_RXIPV6_GD_OCT` register contains a count of the number of bytes received in good IPv6 datagrams encapsulating TCP, UDP or ICMPv6 data

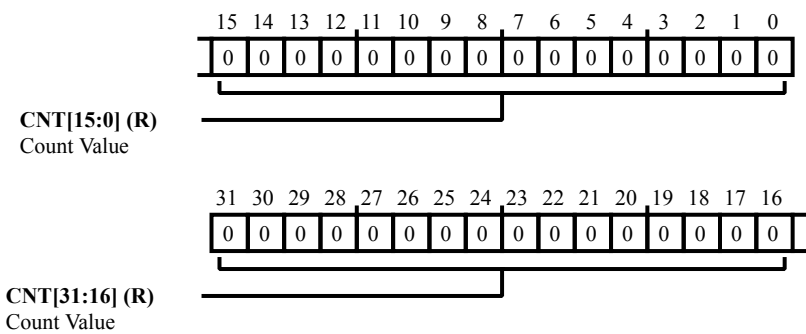


Figure 24-138: EMAC_RXIPV6_GD_OCT Register Diagram

Table 24-170: EMAC_RXIPV6_GD_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv6 Datagrams Header Error Frames Register

The `EMAC_RXIPV6_HDR_ERR_FRM` register contains a count of the number of IPv6 datagrams received with header errors (length or version mismatch).

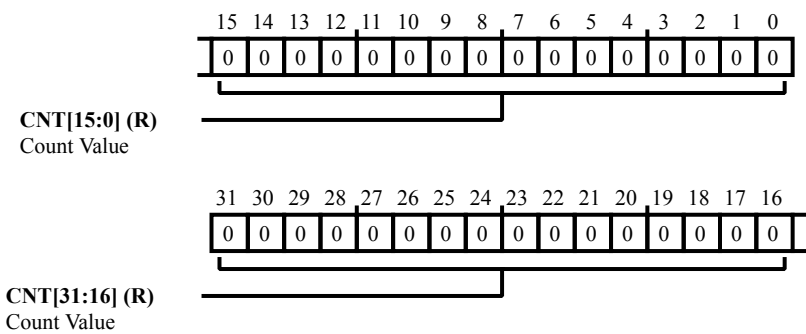


Figure 24-139: EMAC_RXIPV6_HDR_ERR_FRM Register Diagram

Table 24-171: EMAC_RXIPV6_HDR_ERR_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv6 Header Errors Register

The `EMAC_RXIPv6_HDR_ERR_OCT` register contains a count of the number of bytes received in IPv6 datagrams with header errors (length, version mismatch). The value in the IPv6 headers Length field is used to update this counter.

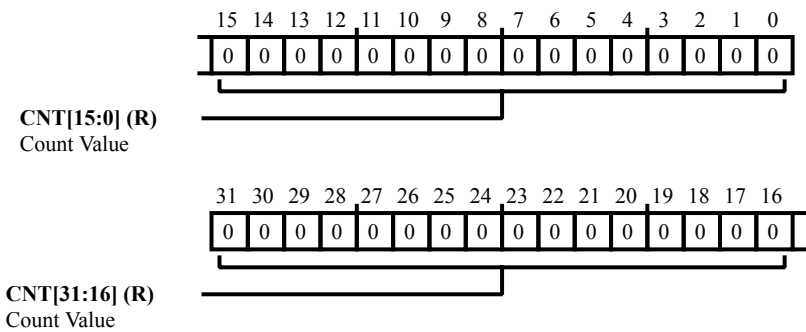


Figure 24-140: EMAC_RXIPv6_HDR_ERR_OCT Register Diagram

Table 24-172: EMAC_RXIPv6_HDR_ERR_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv6 Datagrams No Payload Frames Register

The `EMAC_RXIPV6_NOPAY_FRM` register contains a count of the number of IPv6 datagram frames received that did not have a TCP, UDP, or ICMP payload. This includes all IPv6 datagrams with fragmentation or security extension headers.

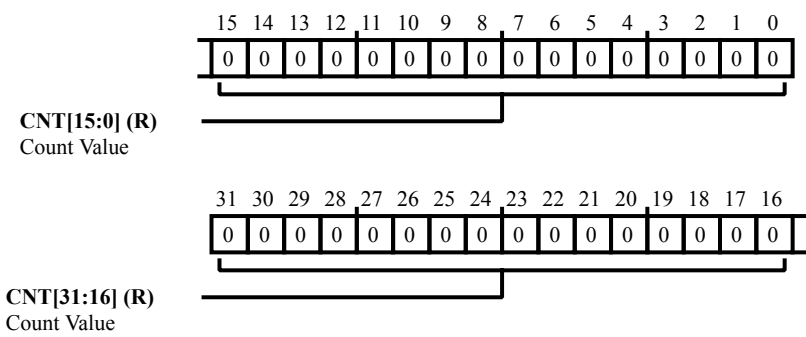


Figure 24-141: EMAC_RXIPV6_NOPAY_FRM Register Diagram

Table 24-173: EMAC_RXIPV6_NOPAY_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx IPv6 No Payload Octets Register

The `EMAC_RXIPv6_NOPAY_OCT` register contains a count of the number of bytes received in IPv6 datagrams that did not have a TCP, UDP, or ICMP payload. The value in the IPv6 headers Length field is used to update this counter.

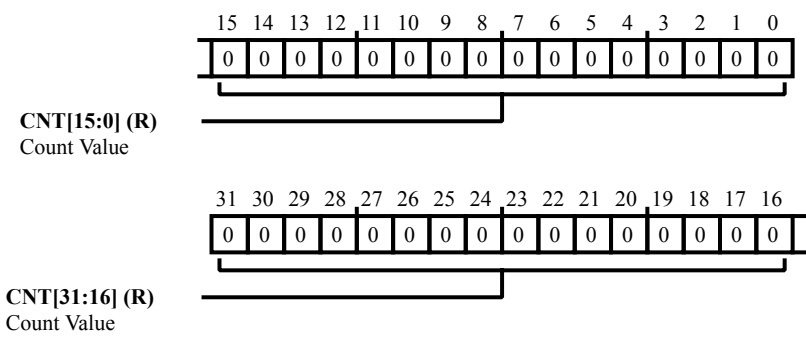


Figure 24-142: EMAC_RXIPv6_NOPAY_OCT Register Diagram

Table 24-174: EMAC_RXIPv6_NOPAY_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Jab Error Register

The `EMAC_RXJAB_ERR` register contains a count of the number of giant frames received with length greater than 1,518 bytes and with CRC error.

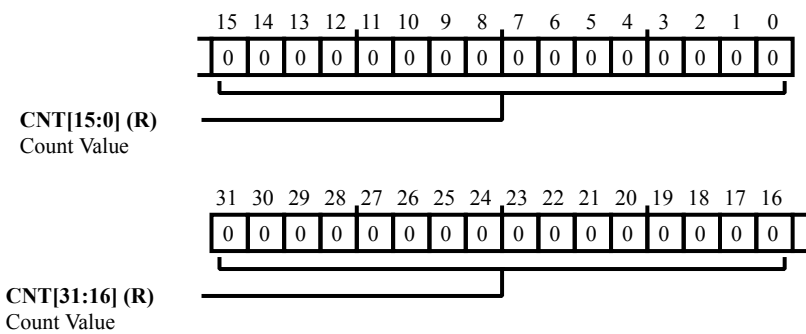


Figure 24-143: EMAC_RXJAB_ERR Register Diagram

Table 24-175: EMAC_RXJAB_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Length Error Register

The `EMAC_RXLEN_ERR` register contains a count of the number of frames received with length error (Length type field frame size), for all frames with valid length field.

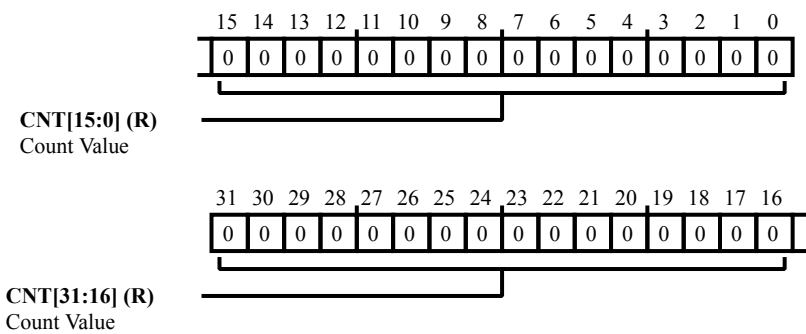


Figure 24-144: EMAC_RXLEN_ERR Register Diagram

Table 24-176: EMAC_RXLEN_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Multicast Frames (Good) Register

The `EMAC_RXMCASTFRM_G` register contains a count of the number of good multicast frames received.

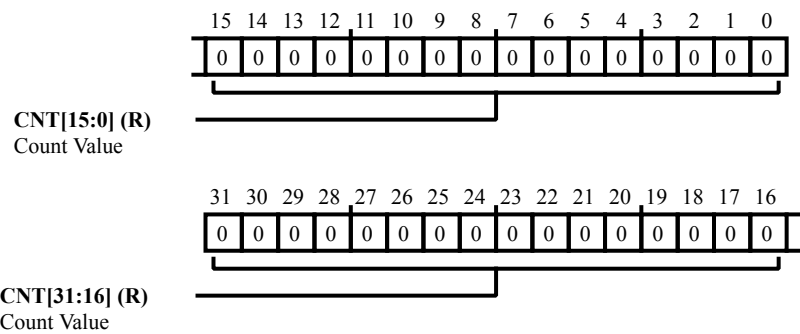


Figure 24-145: EMAC_RXMCASTFRM_G Register Diagram

Table 24-177: EMAC_RXMCASTFRM_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Octet Count (Good) Register

The `EMAC_RXOCTCNT_G` register contains a count of the number of bytes received, exclusive of preamble, only in good frames.

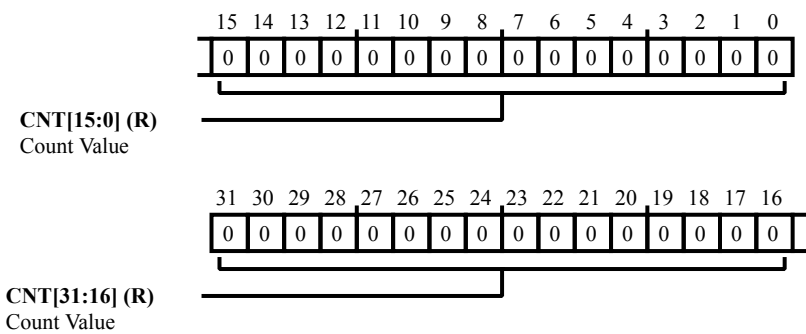


Figure 24-146: EMAC_RXOCTCNT_G Register Diagram

Table 24-178: EMAC_RXOCTCNT_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Octet Count (Good/Bad) Register

The `EMAC_RXOCTCNT_GB` register contains a count of the number of bytes received, exclusive of preamble, in good and bad frames.

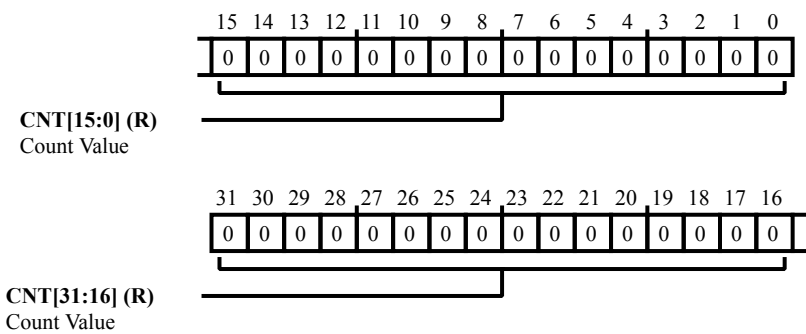


Figure 24-147: EMAC_RXOCTCNT_GB Register Diagram

Table 24-179: EMAC_RXOCTCNT_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Out Of Range Type Register

The `EMAC_RXOORTYPE` register contains a count of the number of frames received with length field not equal to the valid frame size (greater than 1,500 but less than 1,536).

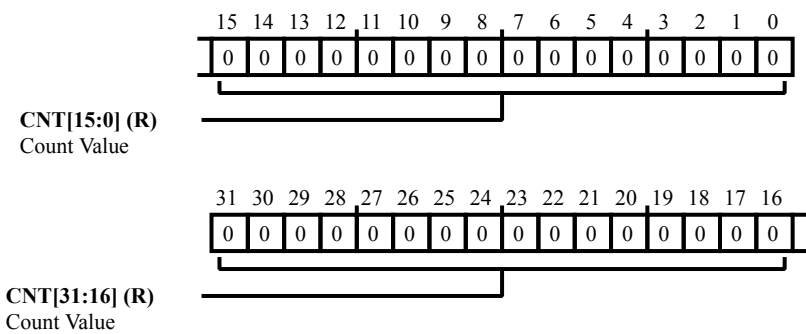


Figure 24-148: EMAC_RXOORTYPE Register Diagram

Table 24-180: EMAC_RXOORTYPE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Oversize (Good) Register

The `EMAC_RXOSIZE_G` register contains a count of the number of frames received with length greater than the maxsize, without errors.

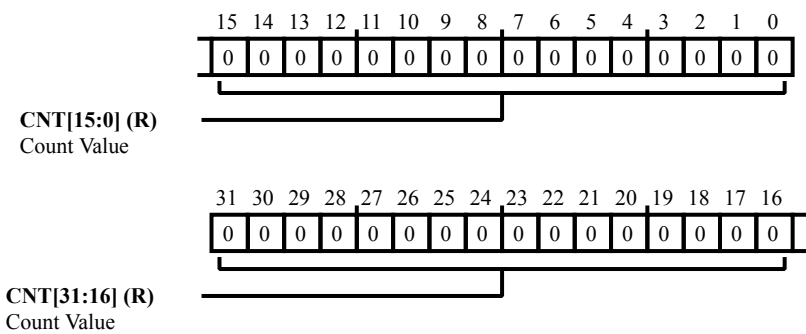


Figure 24-149: EMAC_RXOSIZE_G Register Diagram

Table 24-181: EMAC_RXOSIZE_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Pause Frames Register

The `EMAC_RXPAUSEFRM` register contains a count of the number of good and valid PAUSE frames received.

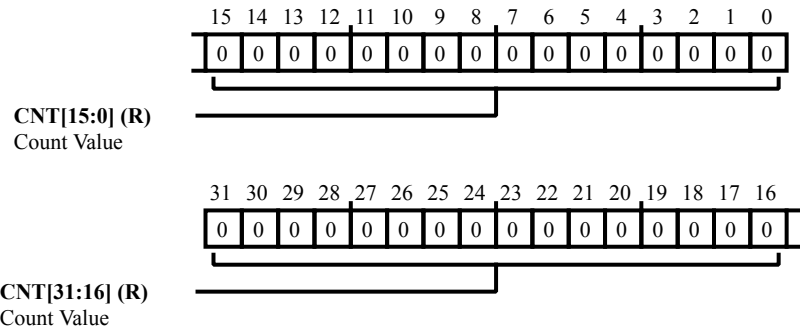


Figure 24-150: EMAC_RXPAUSEFRM Register Diagram

Table 24-182: EMAC_RXPAUSEFRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Error Frames Received Register

The `EMAC_RXRCV_ERR` register contains a count of the number of frames received with Receive error or Frame Extension error on GMII or MII interface

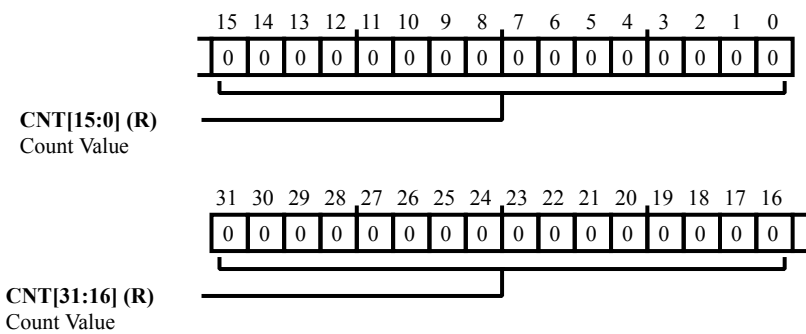


Figure 24-151: EMAC_RXRCV_ERR Register Diagram

Table 24-183: EMAC_RXRCV_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Runt Error Register

The `EMAC_RXRUNT_ERR` register contains a count of the number of frames received with runt error.

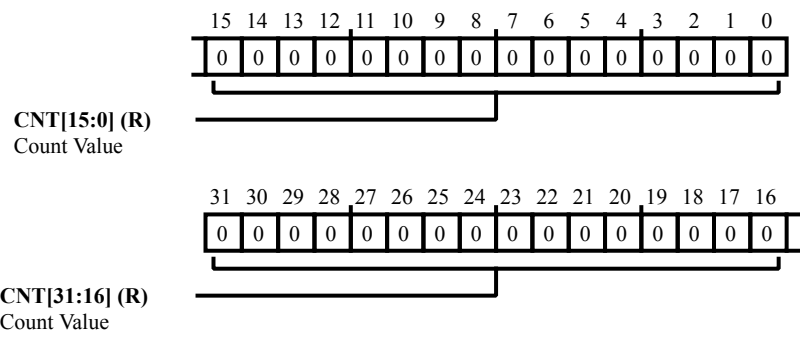


Figure 24-152: EMAC_RXRUNT_ERR Register Diagram

Table 24-184: EMAC_RXRUNT_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx TCP Error Frames Register

The `EMAC_RXTCP_ERR_FRM` register contains a count of the number of good IP datagrams whose TCP payload has a checksum error.

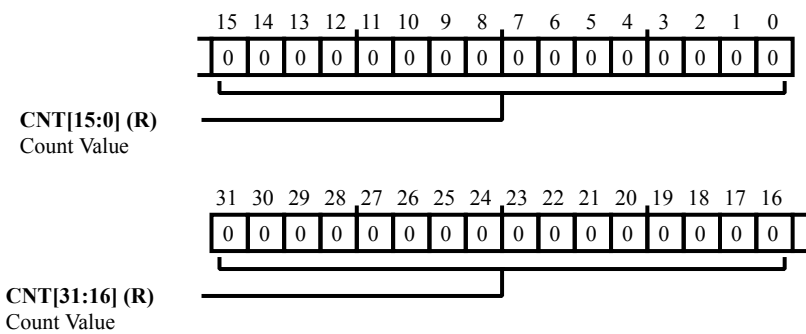


Figure 24-153: EMAC_RXTCP_ERR_FRM Register Diagram

Table 24-185: EMAC_RXTCP_ERR_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx TCP Error Octets Register

The `EMAC_RXTCP_ERR_OCT` register contains a count of the number of bytes received in a TCP segment with checksum errors.

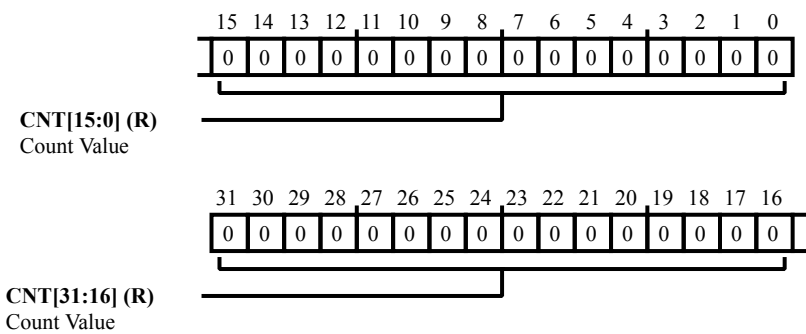


Figure 24-154: EMAC_RXTCP_ERR_OCT Register Diagram

Table 24-186: EMAC_RXTCP_ERR_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx TCP Good Frames Register

The `EMAC_RXTCP_GD_FRM` register contains a count of the number of good IP datagrams with a good TCP payload.

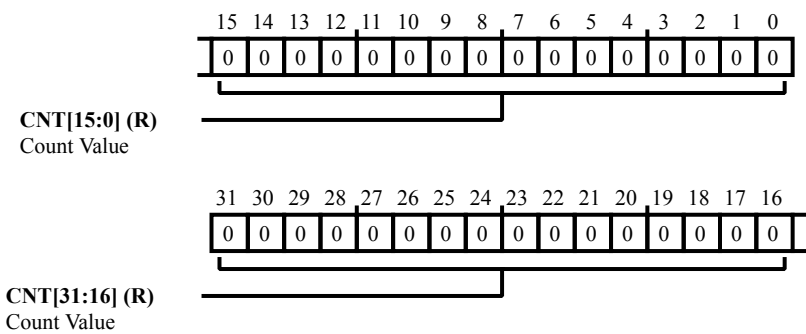


Figure 24-155: EMAC_RXTCP_GD_FRM Register Diagram

Table 24-187: EMAC_RXTCP_GD_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx TCP Good Octets Register

The `EMAC_RXTCP_GD_OCT` register contains a count of the number of bytes received in a good TCP segment.

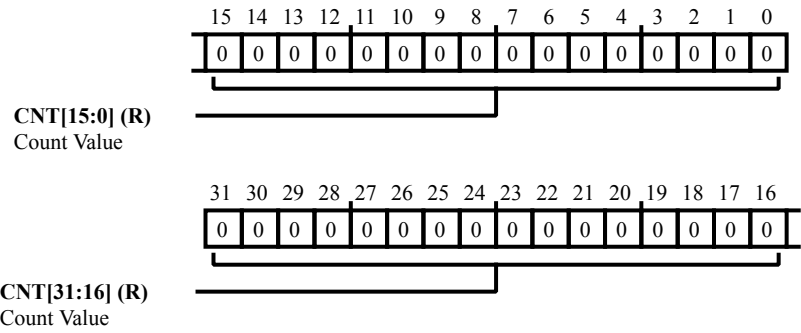


Figure 24-156: EMAC_RXTCP_GD_OCT Register Diagram

Table 24-188: EMAC_RXTCP_GD_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Unicast Frames (Good) Register

The `EMAC_RXUCASTFRM_G` register contains a count of the number of good unicast frames received.

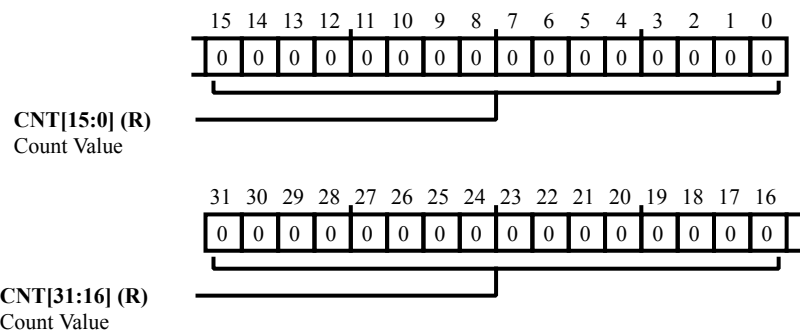


Figure 24-157: EMAC_RXUCASTFRM_G Register Diagram

Table 24-189: EMAC_RXUCASTFRM_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx UDP Error Frames Register

The `EMAC_RXUDP_ERR_FRM` register contains a count of the number of good IP datagrams whose UDP payload has a checksum error.

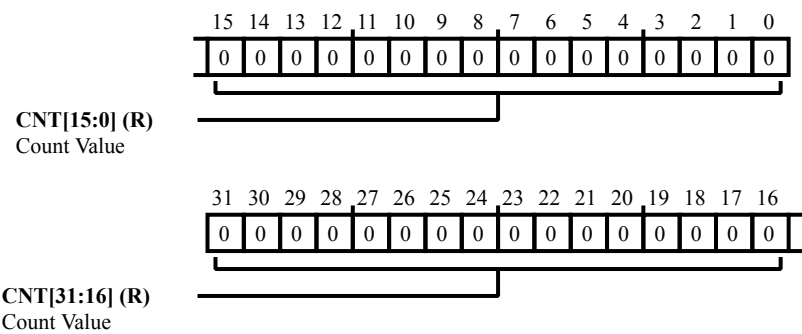


Figure 24-158: EMAC_RXUDP_ERR_FRM Register Diagram

Table 24-190: EMAC_RXUDP_ERR_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx UDP Error Octets Register

The `EMAC_RXUDP_ERR_OCT` register contains a count of the number of bytes received in a UDP segment that had checksum errors.

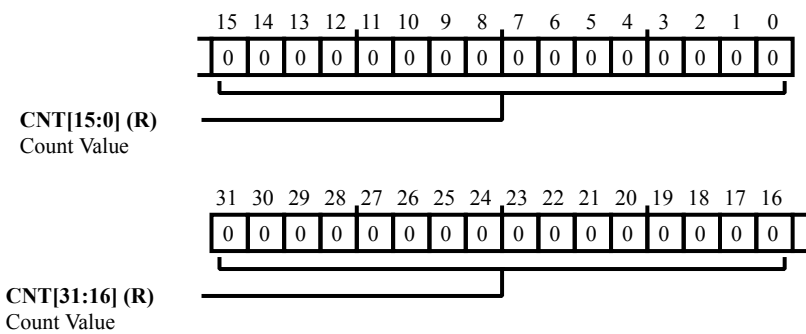


Figure 24-159: EMAC_RXUDP_ERR_OCT Register Diagram

Table 24-191: EMAC_RXUDP_ERR_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx UDP Good Frames Register

The `EMAC_RXUDP_GD_FRM` register contains a count of the number of good IP datagrams with a good UDP payload. This counter is not updated when the `rxipv4_udsbl_frms` counter is incremented.

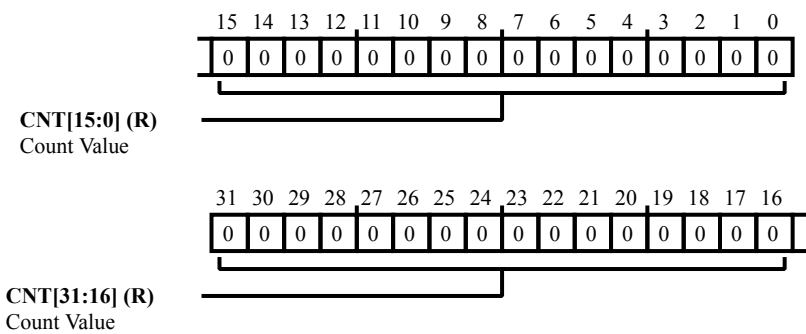


Figure 24-160: EMAC_RXUDP_GD_FRM Register Diagram

Table 24-192: EMAC_RXUDP_GD_FRM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx UDP Good Octets Register

The `EMAC_RXUDP_GD_OCT` register contains a count of the number of bytes received in a good UDP segment. This counter (and the counters below) does not count IP header bytes.

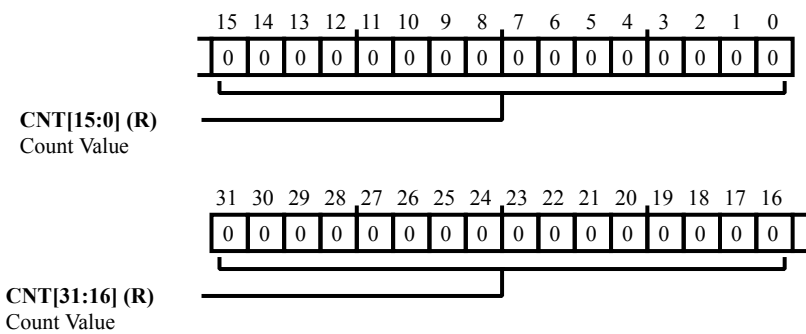


Figure 24-161: EMAC_RXUDP_GD_OCT Register Diagram

Table 24-193: EMAC_RXUDP_GD_OCT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Undersize (Good) Register

The `EMAC_RXUSIZE_G` register contains a count of the number of frames received with length less than 64 bytes, without any errors.

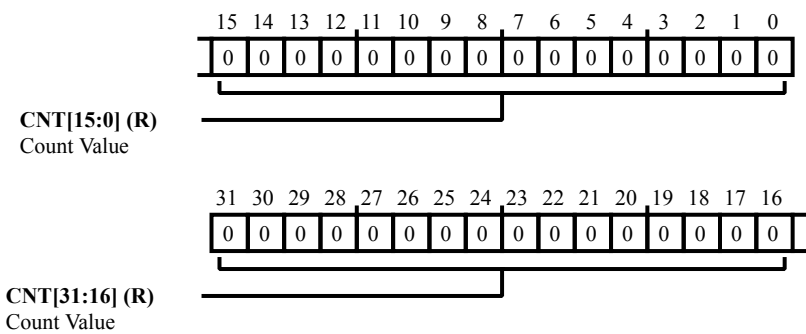


Figure 24-162: EMAC_RXUSIZE_G Register Diagram

Table 24-194: EMAC_RXUSIZE_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx VLAN Frames (Good/Bad) Register

The `EMAC_RXVLANFRM_GB` register contains a count of the number of good and bad VLAN frames received.

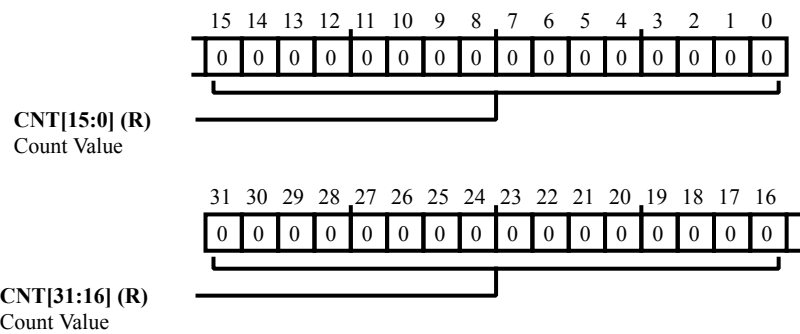


Figure 24-163: EMAC_RXVLANFRM_GB Register Diagram

Table 24-195: EMAC_RXVLANFRM_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Rx Watch Dog Error Register

The `EMAC_RXWDOG_ERR` register contains a count of the number of frames received with error due to watchdog timeout error (frames with a data load larger than 2,048 bytes).

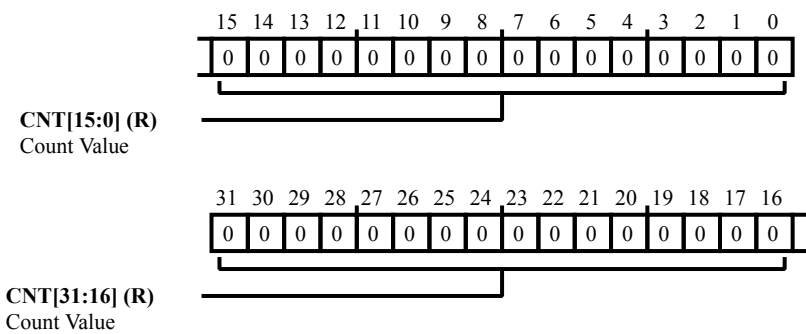


Figure 24-164: EMAC_RXWDOG_ERR Register Diagram

Table 24-196: EMAC_RXWDOG_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

SMI Address Register

The `EMAC_SMI_ADDR` register contains the station management interface address and feature settings.

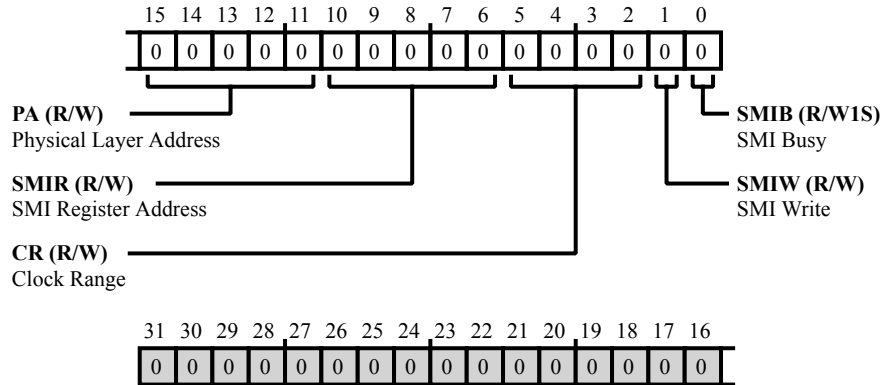


Figure 24-165: EMAC_SMI_ADDR Register Diagram

Table 24-197: EMAC_SMI_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:11 (R/W)	PA	Physical Layer Address. The <code>EMAC_SMI_ADDR</code> . PA bits select the PHY. This field tells which of the 32 possible PHY devices are being accessed.
10:6 (R/W)	SMIR	SMI Register Address. The <code>EMAC_SMI_ADDR</code> . SMIR bits select the desired Station Management Interface register in the selected PHY device.
5:2 (R/W)	CR	Clock Range. The <code>EMAC_SMI_ADDR</code> . CR bits select the Clock Range, determining the frequency of the MDC clock as per the SCLK0 frequency. The suggested range of SCLK0 frequency applicable for each value below (when Bit[5] =0) ensures that the MDC clock is approximately between the frequency range 1.0 MHz - 2.5 MHz. When the MSB of this field is set, you can achieve MDC clock of frequency higher than the IEEE 802.3 specified frequency limit of 2.5 MHz and program a clock divider of lower value. For example, when SCLK0=100 MHz and you program these bits to b#1010, the resulting MDC clock is 12.5 MHz, which is outside the limit of IEEE 802.3 specified range. Use the values shown only if the interface chips support faster MDC clocks.
	0	MDC Clock= SCLK0/42 (for SCLK0=60-100MHz)
	1	MDC Clock= SCLK0/62 (for SCLK0=100-125 MHz)
	2	MDC Clock= SCLK0/16 (for SCLK0=20-35 MHz)
	3	MDC Clock= SCLK0/26 (for SCLK0=35-60 MHz)
	8	MDC Clock= SCLK0/4

Table 24-197: EMAC_SMI_ADDR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		9	MDC Clock= SCLK0/6
		10	MDC Clock= SCLK0/8
		11	MDC Clock= SCLK0/10
		12	MDC Clock= SCLK0/12
		13	MDC Clock= SCLK0/14
		14	MDC Clock= SCLK0/16
		15	MDC Clock= SCLK0/18
1 (R/W)	SMIW	SMI Write. The EMAC_SMI_ADDR.SMIW bit, when set, tells the PHY this is a Write operation using the Station Management Interface Data register. If this bit is not set, this is a Read operation.	
0 (R/W1S)	SMIB	SMI Busy. The EMAC_SMI_ADDR.SMIB bit should read low (=0) before writing to the EMAC_SMI_ADDR and EMAC_SMI_DATA registers. This bit must also =0 during a Write to EMAC_SMI_ADDR. During a PHY register access, this bit is set (=1) by the Application to indicate that a Read or Write access is in progress. The EMAC_SMI_DATA register should be kept valid until this bit is cleared by the MAC during a PHY Write operation. EMAC_SMI_DATA is invalid until this bit is cleared by the MAC during a PHY Read operation. The EMAC_SMI_ADDR should not be written to until this bit is cleared.	

SMI Data Register

The `EMAC_SMI_DATA` register contains the station management interface data.

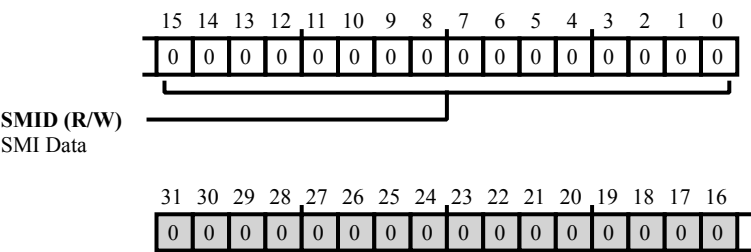


Figure 24-166: EMAC_SMI_DATA Register Diagram

Table 24-198: EMAC_SMI_DATA Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	SMID	SMI Data. The <code>EMAC_SMI_DATA</code> . <code>SMID</code> bits contain the 16-bit data value read from the PHY after a Management Read operation or the 16-bit data value to be written to the PHY before a Management Write operation.

Time Stamp Addend Register

The `EMAC_TM_ADDEND` register lets software adjust the clock frequency linearly to match the master clock frequency.

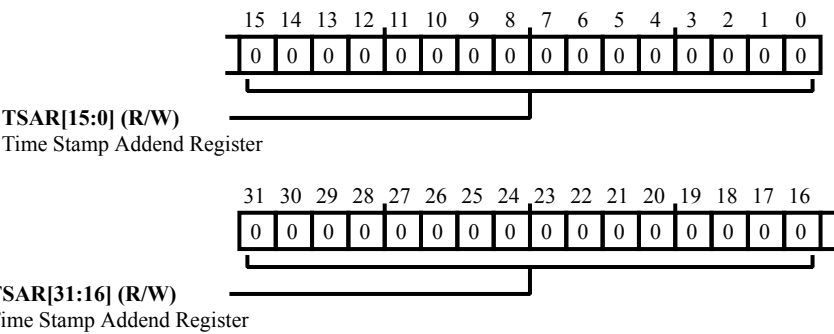


Figure 24-167: EMAC_TM_ADDEND Register Diagram

Table 24-199: EMAC_TM_ADDEND Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	TSAR	Time Stamp Addend Register. The <code>EMAC_TM_ADDEND.TSAR</code> bits indicate the 32-bit time value to be added to the Accumulator register to achieve time synchronization.

Time Stamp Auxiliary TS Nano Seconds Register

The `EMAC_TM_AUXSTMP_NSEC` register contains the low 32 bits (nanoseconds field) of the auxiliary time stamp.

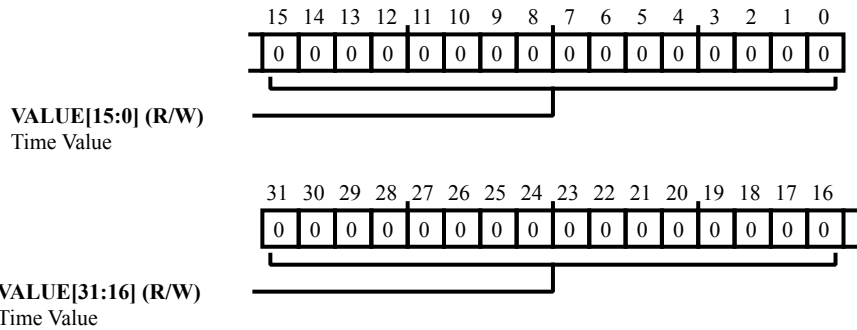


Figure 24-168: EMAC_TM_AUXSTMP_NSEC Register Diagram

Table 24-200: EMAC_TM_AUXSTMP_NSEC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Time Value.

Time Stamp Auxiliary TM Seconds Register

The `EMAC_TM_AUXSTMP_SEC` register contains the low 32 bits of the seconds field of the auxiliary time stamp.

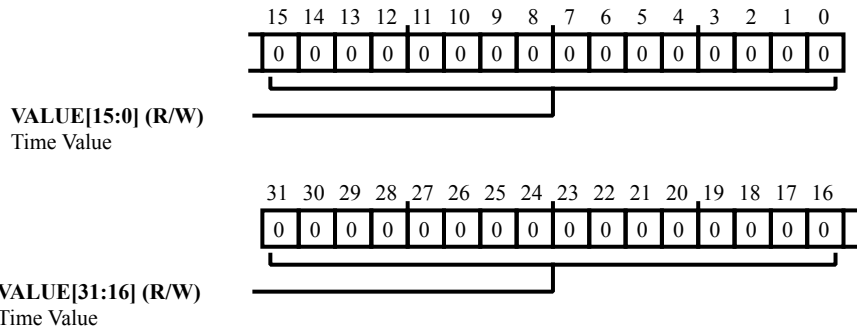


Figure 24-169: EMAC_TM_AUXSTMP_SEC Register Diagram

Table 24-201: EMAC_TM_AUXSTMP_SEC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Time Value.

Time Stamp Control Register

The `EMAC_TM_CTL` register controls time stamp generation and update. The `EMAC_TM_CTL.SNAPTYPSEL`, `EMAC_TM_CTL.TSMSTRENA`, and `EMAC_TM_CTL.TSEVNTENA` bits work together to decide the set of PTP packet types for which snapshot needs to be taken. (Encoding shown in table.)

SNAPTYPSEL()	TSMSTRENA	TSEVNTENA	Messages for which snapshot is taken
00	X	0	SYNC, Follow_Up, Delay_Req, Delay_Resp
00	0	1	SYNC
00	1	1	Delay_Req
01	X	0	SYNC, Follow_Up, Delay_Req, Delay_Resp, Pdelay_Req, Pdelay_Resp, Pdelay_Resp_Follow_Up
01	0	1	SYNC, Pdelay_Req, Pdelay_Resp
01	1	1	Delay_Req, Pdelay_Req, Pdelay_Resp
10	X	X	SYNC, Delay_Req
11	X	X	Pdelay_Req, Pdelay_Resp

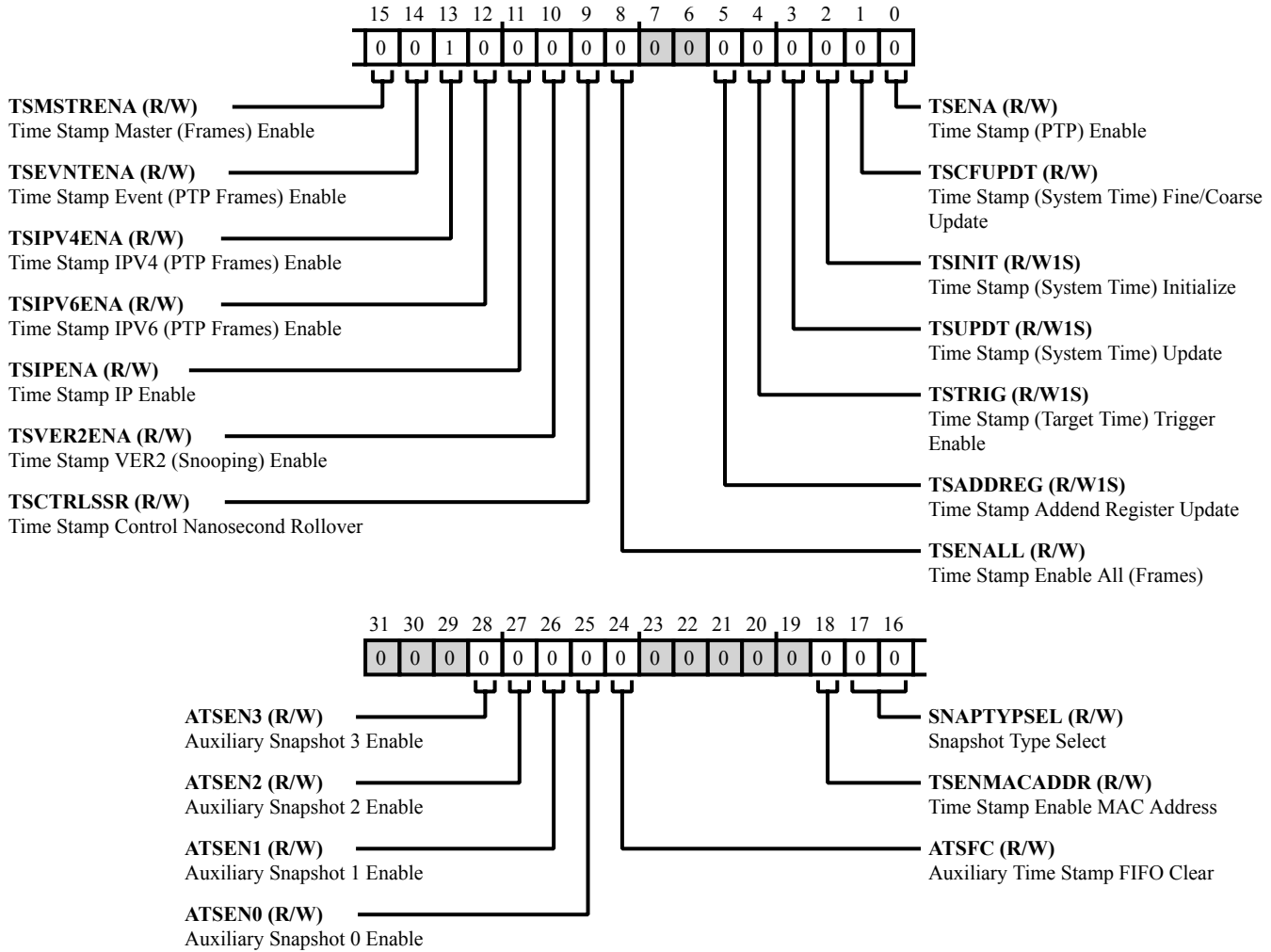


Figure 24-170: EMAC_TM_CTL Register Diagram

Table 24-202: EMAC_TM_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	ATSEN3	Auxiliary Snapshot 3 Enable. The EMAC_TM_CTL.ATSEN3 bit, controls capturing the Auxiliary Snapshot Trigger 3
27 (R/W)	ATSEN2	Auxiliary Snapshot 2 Enable. The EMAC_TM_CTL.ATSEN2 bit, controls capturing the Auxiliary Snapshot Trigger 2
26 (R/W)	ATSEN1	Auxiliary Snapshot 1 Enable. The EMAC_TM_CTL.ATSEN1 bit, controls capturing the Auxiliary Snapshot Trigger 1

Table 24-202: EMAC_TM_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	ATSEN0	Auxiliary Snapshot 0 Enable. The EMAC_TM_CTL.ATSEN0 bit, controls capturing the Auxiliary Snapshot Trigger 0
24 (R/W)	ATSFC	Auxiliary Time Stamp FIFO Clear. The EMAC_TM_CTL.ATSFC bit, when set, resets the pointers of the Auxiliary Snapshot FIFO. This bit is cleared when the pointers are reset and the FIFO is empty. When this bit is cleared, auxiliary snapshots gets stored in the FIFO.
18 (R/W)	TSENMACADDR	Time Stamp Enable MAC Address. The EMAC_TM_CTL.TSENMACADDR bit, when set, uses the DA MAC address (that matches the EMAC_ADDR0_LO and EMAC_ADDR0_HI registers) to filter the PTP frames when PTP is sent directly over Ethernet.
		0 Disable PTP MAC address filter
		1 Enable PTP MAC address filter
17:16 (R/W)	SNAPTYPSEL	Snapshot Type Select. The EMAC_TM_CTL.SNAPTYPSEL bits along with bit 15 and 14 decide the set of PTP packet types for which snapshot needs to be taken. (See the table in the EMAC_TM_CTL register description.)
15 (R/W)	TSMSTRENA	Time Stamp Master (Frames) Enable. The EMAC_TM_CTL.TSMSTRENA bit, when set, takes the snapshot for messages relevant to master node only else snapshot is taken for PTP messages relevant to slave node.
		0 Enable Snapshot for Slave Messages
		1 Enable Snapshot for Master Messages
14 (R/W)	TSEVNTENA	Time Stamp Event (PTP Frames) Enable. The EMAC_TM_CTL.TSEVNTENA bit, when set, takes the time stamp snapshot for PTP event messages only (SYNC, Delay_Req, Pdelay_Req, or Pdelay_Resp). When reset, the snapshot is taken for all PTP messages except Announce, Management, and Signaling.
		0 Enable Time Stamp for All Messages
		1 Enable Time Stamp for Event Messages Only

Table 24-202: EMAC_TM_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	TSIPV4ENA	Time Stamp IPv4 (PTP Frames) Enable. The EMAC_TM_CTL.TSIPV4ENA bit, when set, directs the EMAC receiver to process the PTP packets encapsulated in UDP over IPv4 packets. When this bit is clear, the MAC ignores the PTP transported over UDP-IPv4 packets. This bit is set by default.
		0 Disable Time Stamp for PTP Over IPv4 Frames
		1 Enable Time Stamp for PTP Over IPv4 Frames
12 (R/W)	TSIPV6ENA	Time Stamp IPv6 (PTP Frames) Enable. The EMAC_TM_CTL.TSIPV6ENA bit, when set, directs the EMAC receiver to process PTP packets encapsulated in UDP over IPv6 packets. When this bit is clear, the MAC ignores the PTP transported over UDP-IPv6 packets.
		0 Disable Time Stamp for PTP Over IPv6 frames
		1 Enable Time Stamp for PTP Over IPv6 Frames
11 (R/W)	TSIPENA	Time Stamp IP Enable. The EMAC_TM_CTL.TSIPENA bit, when set, directs the EMAC receiver to process the PTP packets encapsulated directly in the Ethernet frames. When this bit is clear, the MAC ignores PTP over Ethernet packets.
		0 Disable PTP Over Ethernet Frames
		1 Enable PTP Over Ethernet Frames
10 (R/W)	TSVER2ENA	Time Stamp VER2 (Snooping) Enable. The EMAC_TM_CTL.TSVER2ENA bit, when set, processes the PTP packets using the 1588 version 2 format (enables PTP packet snooping for VER2) else processed using the version 1 format.
		0 Disable packet snooping for V2 frames
		1 Enable packet snooping for V2 frames
9 (R/W)	TSCTRLSSR	Time Stamp Control Nanosecond Rollover. The EMAC_TM_CTL.TSCTRLSSR bit, when set, rolls over the EMAC_TM_NSEC register after 0x3B9A_C9FF value (10^9-1) and increments the EMAC_TM_SEC register. When reset, the roll over value of EMAC_TM_NSEC register is 0x7FFF_FFFF. The nanosecond increment has to be programmed correctly depending on the PTP reference clock frequency and this bit value.
		0 Roll Over Nanosecond After 0x7FFFFFFF
		1 Roll Over Nanosecond After 0x3B9AC9FF

Table 24-202: EMAC_TM_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	TSENALL	Time Stamp Enable All (Frames). The EMAC_TM_CTL.TSENALL bit, when set, enables the time stamp snapshot for all frames received by the core.
		0 Disable timestamp for all frames
		1 Enable timestamp for all frames
5 (R/W1S)	TSADDREG	Time Stamp Addend Register Update. The EMAC_TM_CTL.TSADDREG bit, when set, updates the contents of the EMAC_TM_ADDEND register for fine correction. This bit is cleared when the update is completed. This bit should be zero before setting it.
4 (R/W1S)	TSTRIG	Time Stamp (Target Time) Trigger Enable. The EMAC_TM_CTL.TSTRIG bit, when set, generates the time stamp interrupt when the System Time becomes greater than the value written in Time Stamp Target Time Seconds register. This bit is reset after the generation of the Time Stamp Trigger Interrupt.
		1 Interrupt (TS) if system time is greater than target time register
3 (R/W1S)	TSUPDT	Time Stamp (System Time) Update. The EMAC_TM_CTL.TSUPDT bit, when set, updates (adds/subtracts) the system time with the value specified in the EMAC_TM_SECUPDT and EMAC_TM_NSECUPDT registers. This bit should read =0 before updating it. This bit is reset when the update is completed in hardware. The EMAC_TM_NSEC register is not updated.
		1 System time updated with Time stamp register values
2 (R/W1S)	TSINIT	Time Stamp (System Time) Initialize. The EMAC_TM_CTL.TSINIT bit, when set, initializes (over-writes) the system time with the value specified in the EMAC_TM_SECUPDT and EMAC_TM_NSECUPDT registers. This bit should read =0 before updating it. This bit is reset when the initialization is complete. Only the EMAC_TM_NSEC register can be initialized.
		1 System time initialized with Time stamp register values
1 (R/W)	TSCFUPDT	Time Stamp (System Time) Fine/Coarse Update. The EMAC_TM_CTL.TSCFUPDT bit, when set, indicates that the system time update is done using the fine correction method. When reset, it indicates the system time correction to be done using Coarse method.
		0 Use Coarse Correction Method for System Time Update
		1 Use Fine Correction Method for System Time Update

Table 24-202: EMAC_TM_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	TSENA	Time Stamp (PTP) Enable. The EMAC_TM_CTL.TSENA bit, when set, enables PTP module for time stamping transmitted and received frames. It also enables System Time which is used for time stamping the frames. Programs should initialize the System Time after setting this bit.
		0 Disable PTP Module
		1 Enable PTP Module

Time Stamp High Second Register

The `EMAC_TM_HISEC` register contains the upper 32 bits of the seconds field of the system time.

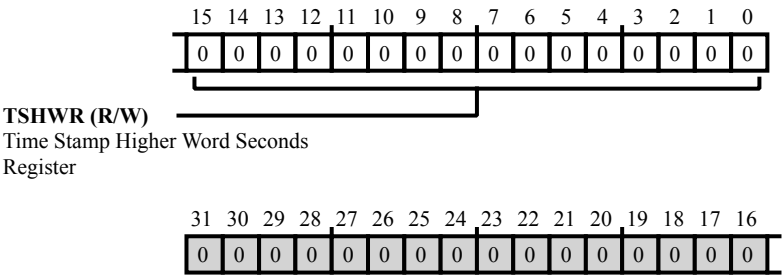


Figure 24-171: EMAC_TM_HISEC Register Diagram

Table 24-203: EMAC_TM_HISEC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	TSHWR	Time Stamp Higher Word Seconds Register. The <code>EMAC_TM_HISEC.TSHWR</code> bit field contains the most significant 16-bits of the time stamp seconds value. The register is directly written to initialize the value. This register is incremented when there is an overflow from the 32-bits of the <code>EMAC_TM_SEC</code> register.

Time Stamp Nanoseconds Register

The `EMAC_TM_NSEC` register contains the nanoseconds field of the system time.

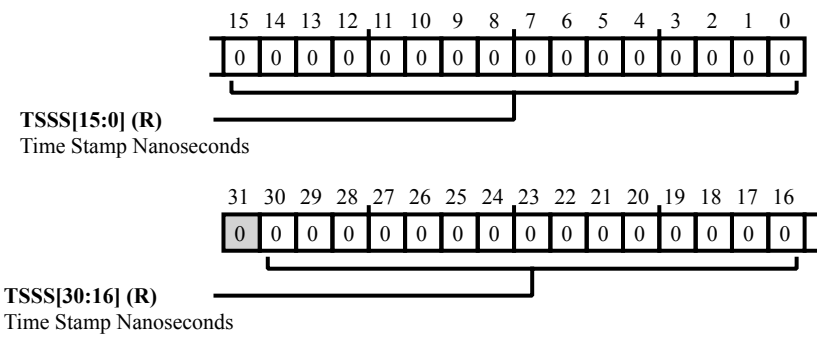


Figure 24-172: EMAC_TM_NSEC Register Diagram

Table 24-204: EMAC_TM_NSEC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30:0 (R/NW)	TSSS	Time Stamp Nanoseconds. The value in the <code>EMAC_TM_NSEC.TSSS</code> bit field has the nanosecond representation of time, with an accuracy of 0.46 nanosecond. (When <code>EMAC_TM_CTL.TSCTRLSSR</code> is set, each bit represents 1 ns and the maximum value will be <code>0x3B9A_C9FE</code> , after which it rolls-over to zero).

Time Stamp Nanoseconds Update Register

The `EMAC_TM_NSECUPDT` register contains the low 32 bits to be added to, subtracted from, or written to the nanoseconds field of the system time.

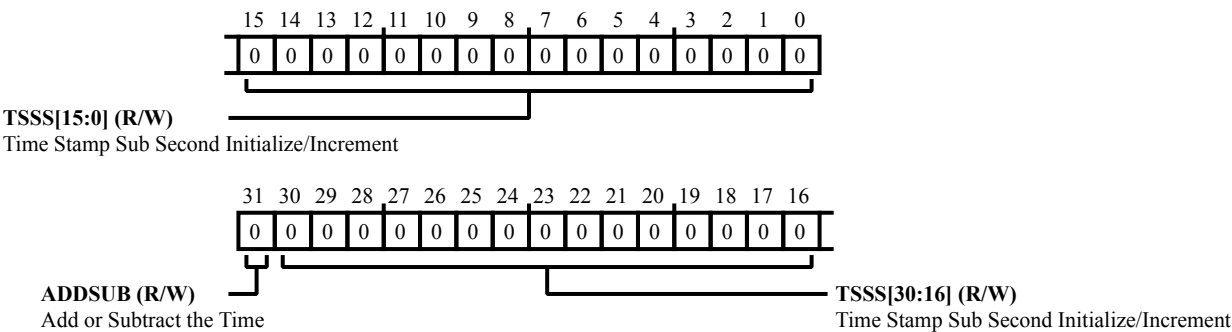


Figure 24-173: EMAC_TM_NSECUPDT Register Diagram

Table 24-205: EMAC_TM_NSECUPDT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ADDSUB	Add or Subtract the Time. The <code>EMAC_TM_NSECUPDT.ADDSUB</code> bit, when set, subtracts the time value with the contents of the update registers. When this bit is reset, the time value is added with the contents of the update registers.
30:0 (R/W)	TSSS	Time Stamp Sub Second Initialize/Increment. The value in the <code>EMAC_TM_NSECUPDT.TSSS</code> bit field indicates the time, in nano-seconds, to be initialized or added to or subtracted from the system time nanoseconds.

Time Stamp PPS Interval Register

The `EMAC_TM_PPS0INTVL` register contains the interval value for the time between rising edges (period) of PPS output.

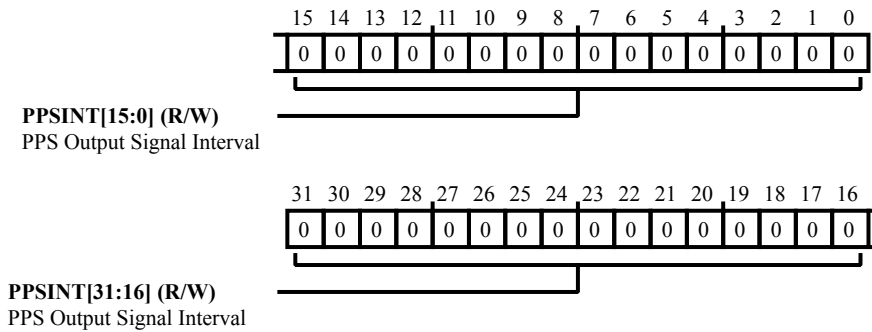


Figure 24-174: EMAC_TM_PPS0INTVL Register Diagram

Table 24-206: EMAC_TM_PPS0INTVL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PPSINT	PPS Output Signal Interval. The <code>EMAC_TM_PPS0INTVL.PPSINT</code> bits store the interval between the rising edges of PPS signal output in terms of units of sub-second increment value. You need to program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20ns), and desired interval between rising edges of PPS signal output is 100ns (that is, 5 units of sub-second increment value), then you should program value 4 (5-1) in this register.

Time Stamp Target Time Nanoseconds Register

The `EMAC_TM_PPS0NTGTM` register contains the high 32 bits of the target nanoseconds field for comparison to the corresponding system time field.

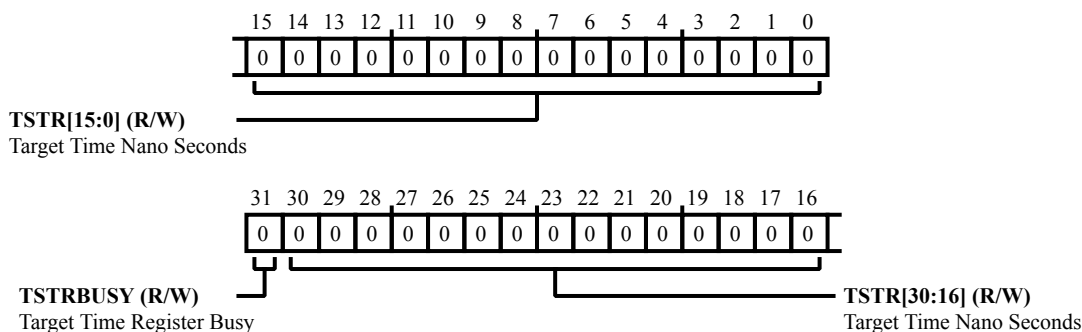


Figure 24-175: EMAC_TM_PPS0NTGTM Register Diagram

Table 24-207: EMAC_TM_PPS0NTGTM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	TSTRBUSY	Target Time Register Busy. The <code>EMAC_TM_PPS0NTGTM.TSTRBUSY</code> bit is set when Flexible PPS is enabled and the PPS Frequency Control bit field in the PPS Control register is programmed to 0001, 0010 or 0100. Programming the PPS Frequency Control bit field to 0001, 0010 or 0100, instructs the core to synchronize the <code>EMAC_TM_PPS0TGTM</code> and <code>EMAC_TM_NSEC</code> registers to the PTP clock domain. The EMAC clears this bit after synchronizing the <code>EMAC_TM_PPS0TGTM</code> and <code>EMAC_TM_NSEC</code> registers to the PTP clock domain. The application must not update the <code>EMAC_TM_PPS0TGTM</code> and <code>EMAC_TM_NSEC</code> registers when this bit is read as 1. Otherwise, the synchronization of the previous programmed time gets corrupted.
30:0 (R/W)	TSTR	Target Time Nano Seconds. The <code>EMAC_TM_PPS0NTGTM.TSTR</code> bit field stores the time in (signed) nanoseconds. When the value of the time stamp matches the both <code>EMAC_TM_PPS0TGTM</code> and <code>EMAC_TM_NSEC</code> registers, based on the Target Time Register Mode bit field in the PPS control register, the MAC starts or stops the PPS signal output and generates an interrupt (if enabled). This value should not exceed 0x3B9A_C9FF when the Target Time Register Mode bit field is set. The actual start or stop time of the PPS signal output may have an error margin up to one unit of sub-second increment value.

Time Stamp Target Time Seconds Register

The `EMAC_TM_PPS0TGTM` register contains the high 32 bits of the target seconds field for comparison to the corresponding system time field.

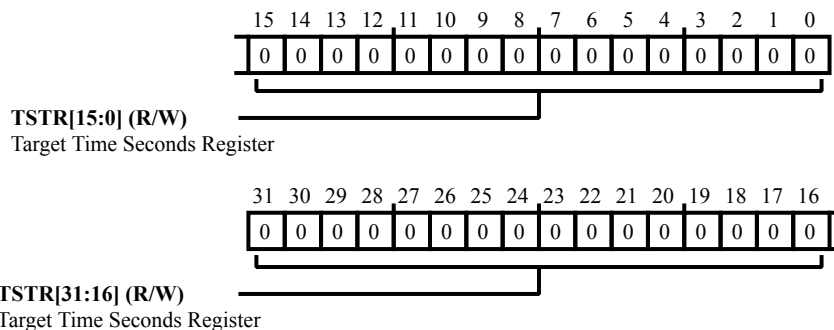


Figure 24-176: EMAC_TM_PPS0TGTM Register Diagram

Table 24-208: EMAC_TM_PPS0TGTM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	TSTR	Target Time Seconds Register. The <code>EMAC_TM_PPS0TGTM.TSTR</code> bit field stores the time in seconds. When the time stamp value matches or exceeds both the value in this field and the value in the <code>EMAC_TM_NSEC</code> register, based on the selection in the Target Time Register Mode bit field, the MAC starts or stops the PPS signal output and generates an interrupt (if enabled).

PPS Width Register

The `EMAC_TM_PPS0WIDTH` register contains the interval value for the time between a rising and the next falling edge (width) of PPS output.

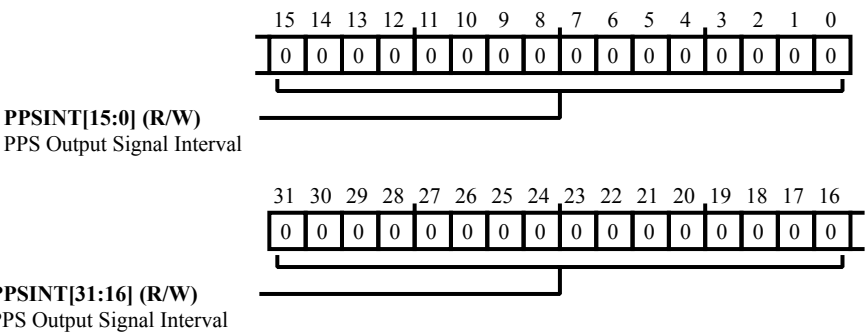


Figure 24-177: EMAC_TM_PPS0WIDTH Register Diagram

Table 24-209: EMAC_TM_PPS0WIDTH Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PPSINT	PPS Output Signal Interval. The <code>EMAC_TM_PPS0WIDTH.PPSINT</code> bits store the interval between a rising edge and the next falling edge (width) of PPS output in terms of units of sub second increment value. Program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns) and the desired width of the PPS signal output is 60 ns (3 units of sub-second increment value), program the value 2 (3-1) in this register.

PPS 1 Interval Register

The `EMAC_TM_PPS1INTVL` register contains the number of units of sub-second increment value between the rising edges of PPS0 signal output.

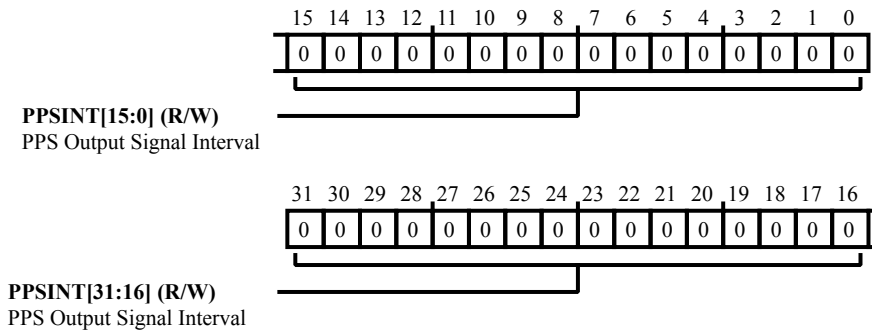


Figure 24-178: EMAC_TM_PPS1INTVL Register Diagram

Table 24-210: EMAC_TM_PPS1INTVL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PPSINT	PPS Output Signal Interval. The <code>EMAC_TM_PPS1INTVL.PPSINT</code> bit field contains the number of units of sub-second increment value between the rising edges of PPS0 signal output.

PPS 1 Target Time Nanoseconds Register

The `EMAC_TM_PPS1NTGTM` register is present only when the IEEE 1588 time-stamp feature is selected without external time-stamp input.

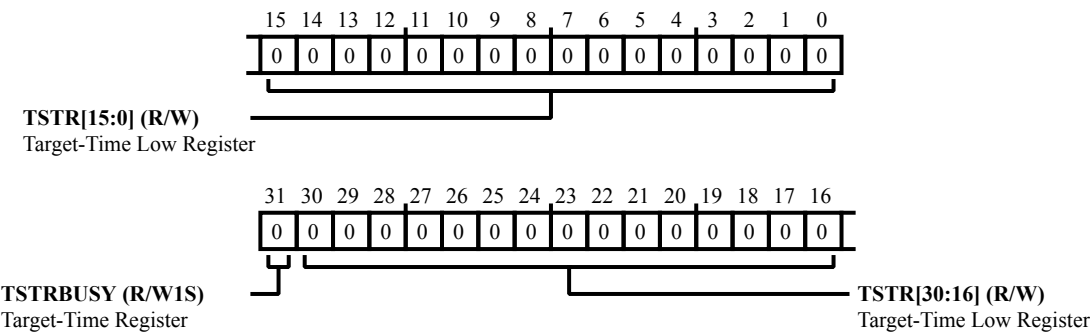


Figure 24-179: EMAC_TM_PPS1NTGTM Register Diagram

Table 24-211: EMAC_TM_PPS1NTGTM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	TSTRBUSY	Target-Time Register. The <code>EMAC_TM_PPS1NTGTM.TSTRBUSY</code> bit field is cleared to 1b0 by the core (self clear). The application cannot clear this type of field, and a register write of 1b0 to this bit has no effect on this field.
30:0 (R/W)	TSTR	Target-Time Low Register.

PPS 1 Target Time Seconds Register

The `EMAC_TM_PPS1TGTM` register schedule an interrupt event when the system time exceeds the value programmed in these registers.

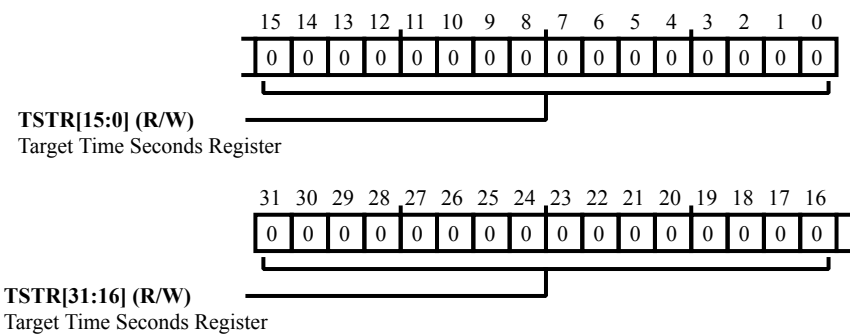


Figure 24-180: EMAC_TM_PPS1TGTM Register Diagram

Table 24-212: EMAC_TM_PPS1TGTM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	TSTR	Target Time Seconds Register. The <code>EMAC_TM_PPS1TGTM.TSTR</code> bit field stores the time in seconds. When the time-stamp value matches or exceeds both target time-stamp registers, the MAC starts or stops the PPS signal output and generates an interrupt.

PPS 1 Width Register

The `EMAC_TM_PPS1WIDTH` register contains the number of units of sub-second increment value between the rising and corresponding falling edges of the PPS0 signal output

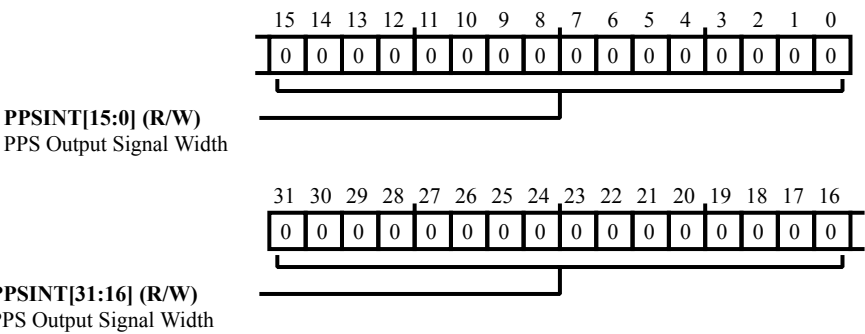


Figure 24-181: EMAC_TM_PPS1WIDTH Register Diagram

Table 24-213: EMAC_TM_PPS1WIDTH Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PPSINT	PPS Output Signal Width. The <code>EMAC_TM_PPS1WIDTH.PPSINT</code> bits store the interval between a rising edge and the next falling edge (width) of PPS output in terms of units of sub second increment value. Program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns) and the desired width of the PPS signal output is 60 ns (3 units of sub-second increment value), program the value 2 (3-1) in this register.

PPS 2 Interval Register

The `EMAC_TM_PPS2INTVL` register contains the number of units of sub-second increment value between the rising edges of PPS0 signal output

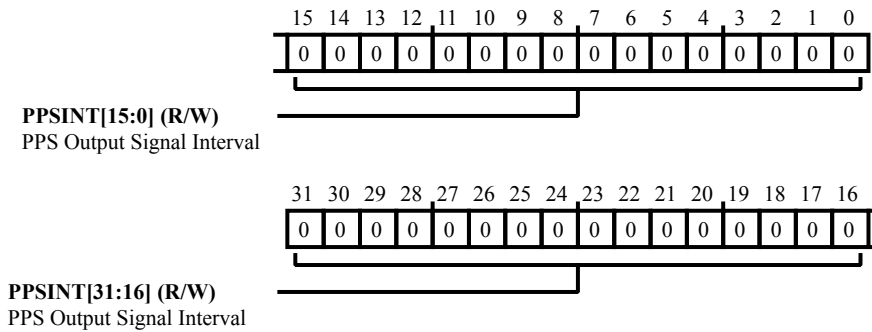


Figure 24-182: EMAC_TM_PPS2INTVL Register Diagram

Table 24-214: EMAC_TM_PPS2INTVL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PPSINT	PPS Output Signal Interval. The <code>EMAC_TM_PPS2INTVL.PPSINT</code> bit field contains the number of units of sub-second increment value between the rising edges of PPS2 signal output.

PPS 2 Target Time Nanoseconds Register

The `EMAC_TM_PPS2NTGTM` register is present only when the IEEE 1588 Time stamp feature is selected without external time stamp input.

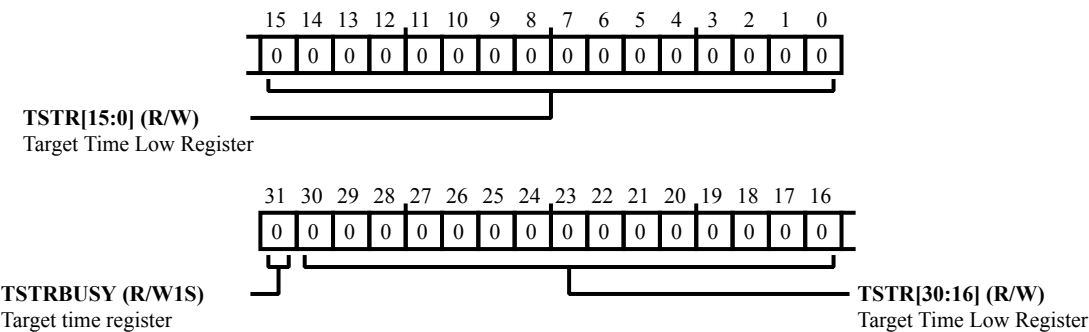


Figure 24-183: EMAC_TM_PPS2NTGTM Register Diagram

Table 24-215: EMAC_TM_PPS2NTGTM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	TSTRBUSY	Target time register. The <code>EMAC_TM_PPS2NTGTM.TSTRBUSY</code> bit field is cleared to 1b0 by the core (self clear). The application cannot clear this type of field, and a register write of 1b0 to this bit has no effect on this field.
30:0 (R/W)	TSTR	Target Time Low Register.

PPS 2 Target Time Seconds Register

The `EMAC_TM_PPS2TGTM` register schedule an interrupt event when the system time exceeds the value programmed in these registers.

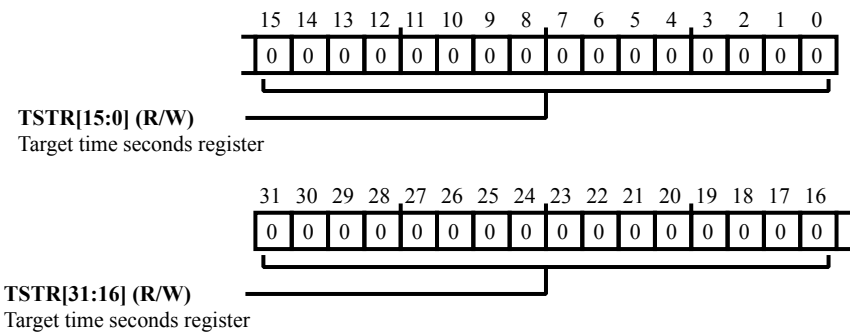


Figure 24-184: EMAC_TM_PPS2TGTM Register Diagram

Table 24-216: EMAC_TM_PPS2TGTM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	TSTR	Target time seconds register. The <code>EMAC_TM_PPS2TGTM.TSTR</code> bit field stores the time in seconds. When the time-stamp value matches or exceeds both target time-stamp registers, the MAC starts or stops the PPS signal output and generates an interrupt.

PPS 2 Width Register

The `EMAC_TM_PPS2WIDTH` register contains the number of units of sub-second increment value between the rising and corresponding falling edges of the PPS0 signal output

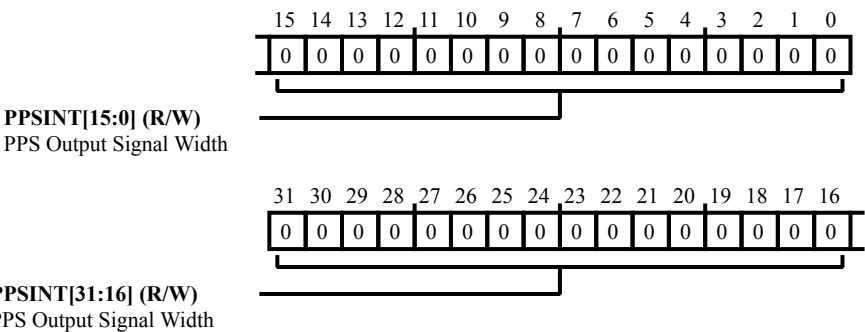


Figure 24-185: EMAC_TM_PPS2WIDTH Register Diagram

Table 24-217: EMAC_TM_PPS2WIDTH Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PPSINT	PPS Output Signal Width. The <code>EMAC_TM_PPS2WIDTH.PPSINT</code> bits store the interval between a rising edge and the next falling edge (width) of PPS output in terms of units of sub second increment value. Program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns) and the desired width of the PPS signal output is 60 ns (3 units of sub-second increment value), program the value 2 (3-1) in this register.

PPS 3 Interval Register

The `EMAC_TM_PPS3INTVL` register contains the number of units of sub-second increment value between the rising edges of PPS0 signal output

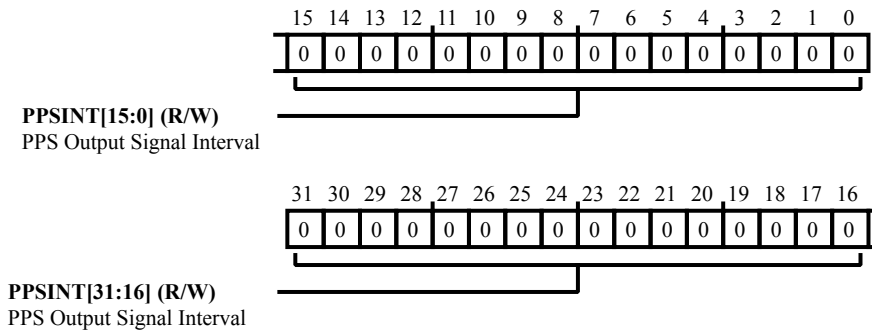


Figure 24-186: EMAC_TM_PPS3INTVL Register Diagram

Table 24-218: EMAC_TM_PPS3INTVL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PPSINT	PPS Output Signal Interval. The <code>EMAC_TM_PPS3INTVL.PPSINT</code> bit field contains the number of units of sub-second increment value between the rising edges of PPS3 signal output.

PPS 3 Target Time Nanoseconds Register

The `EMAC_TM_PPS3NTGTM` register is present only when the IEEE 1588 Time stamp feature is selected without external time stamp input.

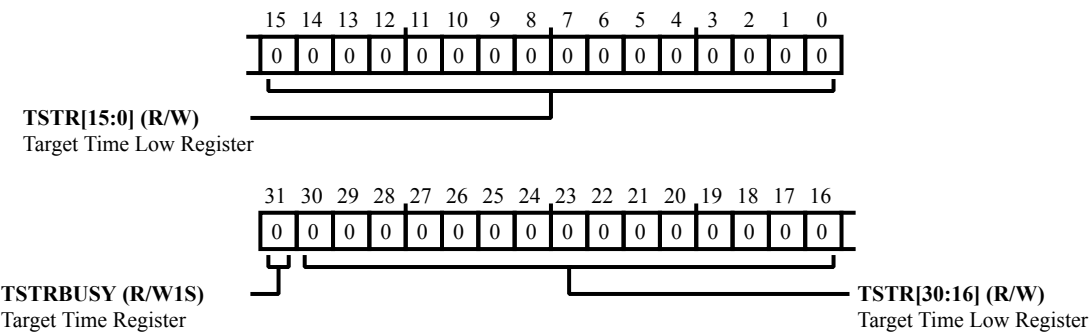


Figure 24-187: EMAC_TM_PPS3NTGTM Register Diagram

Table 24-219: EMAC_TM_PPS3NTGTM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1S)	TSTRBUSY	Target Time Register. The <code>EMAC_TM_PPS3NTGTM.TSTRBUSY</code> bit field is cleared to 1b0 by the core (self clear). The application cannot clear this type of field, and a register write of 1b0 to this bit has no effect on this field.
30:0 (R/W)	TSTR	Target Time Low Register.

PPS 3 Target Time Seconds Register

The `EMAC_TM_PPS3TGTM` register schedule an interrupt event when the system time exceeds the value programmed in these registers.

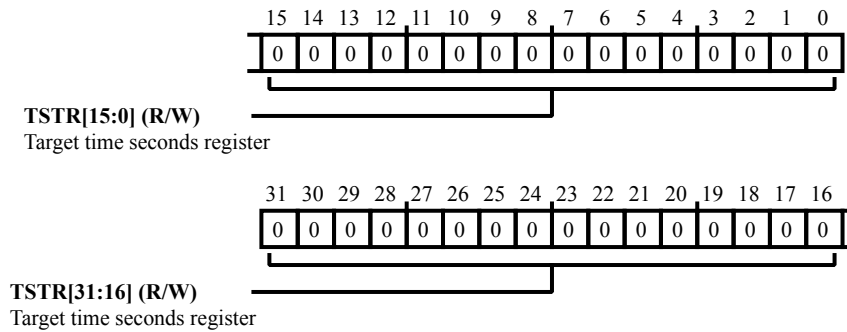


Figure 24-188: EMAC_TM_PPS3TGTM Register Diagram

Table 24-220: EMAC_TM_PPS3TGTM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	TSTR	Target time seconds register. The <code>EMAC_TM_PPS3TGTM.TSTR</code> bit field stores the time in seconds. When the time-stamp value matches or exceeds both target time-stamp registers, the MAC starts or stops the PPS signal output and generates an interrupt.

PPS 3 Width Register

The `EMAC_TM_PPS3WIDTH` register contains the number of units of sub-second increment value between the rising and corresponding falling edges of the PPS0 signal output

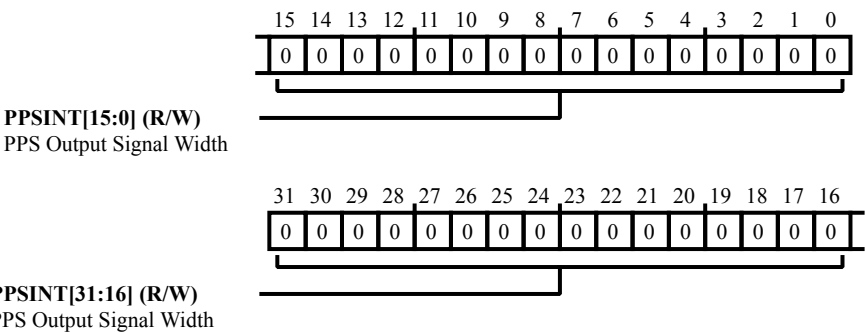


Figure 24-189: EMAC_TM_PPS3WIDTH Register Diagram

Table 24-221: EMAC_TM_PPS3WIDTH Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	PPSINT	PPS Output Signal Width. The <code>EMAC_TM_PPS3WIDTH.PPSINT</code> bits store the interval between a rising edge and the next falling edge (width) of PPS output in terms of units of sub second increment value. Program one value less than the required interval. For example, if the PTP reference clock is 50 MHz (period of 20 ns) and the desired width of the PPS signal output is 60 ns (3 units of sub-second increment value), program the value 2 (3-1) in this register.

PPS Control Register

The `EMAC_TM_PPSCCTL` register controls the interval of PPS output.

When the `EMAC_TM_PPSCCTL.PPSEN` bit is disabled (`=0`, fixed PPS output), the PPS Frequency Control (`PPSCTL0`) bits control the behavior of the PPS output signal. The default value of `PPSCTRL` is `0000` and the PPS output is 1 pulse every second. For other values of `PPSCTRL`, the PPS output becomes a generated clock. (See bit enumerations for frequencies.) In the binary rollover mode, the PPS output has a duty cycle of 50 percent with these frequencies. In the digital rollover mode, the PPS output frequency is an average number. The actual clock is of different frequency that gets synchronized every second. This behavior is because of the non-linear toggling of the bits in the digital rollover mode in System Time - Nanoseconds Register.

When the `EMAC_TM_PPSCCTL.PPSEN` bit is enabled (`=1`, flexible PPS output), the PPS Frequency Control bits function as `PPSCMD`. (See bit enumerations for commands.) Programming these bits with a non-zero value instructs the core to initiate an event. After the command is transferred or synchronized to the PTP clock domain, these bits get cleared automatically. Software should ensure that these bits are programmed only when they are all-zero.

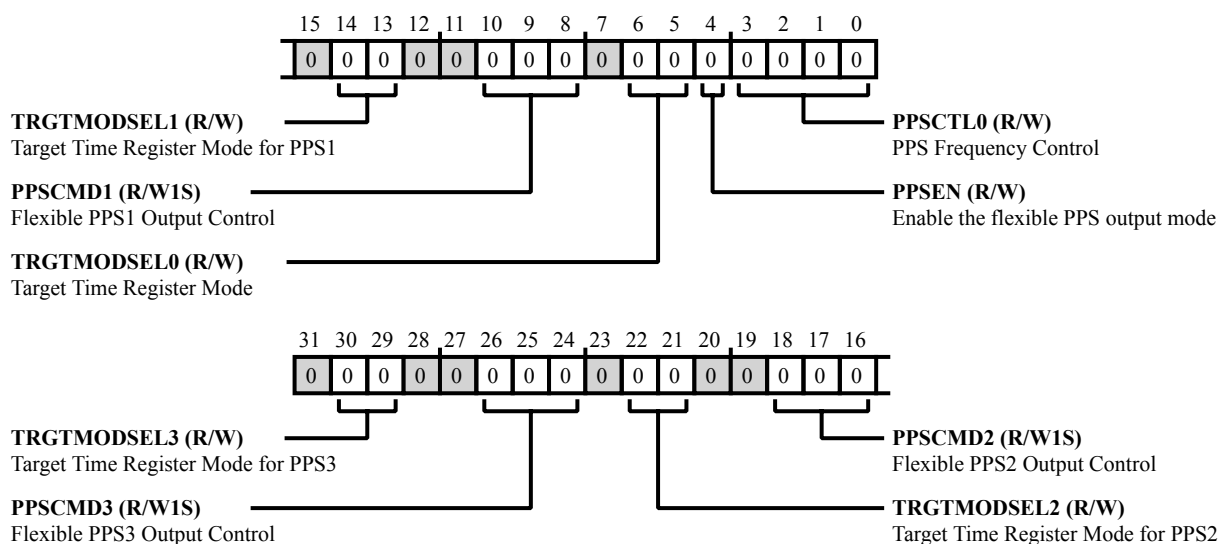


Figure 24-190: EMAC_TM_PPSCCTL Register Diagram

Table 24-222: EMAC_TM_PPSCCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30:29 (R/W)	TRGTMODESEL3	Target Time Register Mode for PPS3. The <code>EMAC_TM_PPSCCTL.TRGTMODESEL3</code> bits indicates the Target Time registers mode for PPS3 output signal.

Table 24-222: EMAC_TM_PPCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26:24 (R/W1S)	PPSCMD3	Flexible PPS3 Output Control. The EMAC_TM_PPCTL.PPSCMD3 bits controls the flexible PPS3 output signal.
22:21 (R/W)	TRGTMODSEL2	Target Time Register Mode for PPS2. The EMAC_TM_PPCTL.TRGTMODSEL2 bits indicates the Target Time registers mode for PPS2 output signal.
18:16 (R/W1S)	PPSCMD2	Flexible PPS2 Output Control. The EMAC_TM_PPCTL.PPSCMD2 bits controls the flexible PPS3 output signal.
14:13 (R/W)	TRGTMODSEL1	Target Time Register Mode for PPS1. The EMAC_TM_PPCTL.TRGTMODSEL1 bits indicates the Target Time registers mode for PPS1 output signal.
10:8 (R/W1S)	PPSCMD1	Flexible PPS1 Output Control. The EMAC_TM_PPCTL.PPSCMD1 bits controls the flexible PPS1 output signal.
6:5 (R/W)	TRGTMODSEL0	Target Time Register Mode. The EMAC_TM_PPCTL.TRGTMODSEL0 bits select the target time register mode.
		0 Interrupt Only The Target Time registers are programmed only for interrupt event generation.
		1 Reserved
		2 Interrupt and PPS Start/Stop The Target Time registers are programmed for interrupt event and for starting or stopping the PPS output signal generation.
		3 PPS Start/Stop Only The Target Time registers are programmed only for starting or stopping the PPS output signal generation. No interrupt is asserted.
4 (R/W)	PPSEN	Enable the flexible PPS output mode. The EMAC_TM_PPCTL.PPSEN bit enables PPS operation. When set low, the PPS Frequency Control field controls frequency of Fixed PPS output. When set high, PPS Frequency Control field is used to command Flexible PPS output.

Table 24-222: EMAC_TM_PPSTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	PPSTL0	PPS Frequency Control. When the EMAC_TM_PPSTL.PPSEN bit is disabled (=0, fixed PPS output), the EMAC_TM_PPSTL.PPSTL0 bits control the behavior of the PPS output signal. When the EMAC_TM_PPSTL.PPSEN bit is enabled (=1, flexible PPS output), the EMAC_TM_PPSTL.PPSTL0 bits function as PPSCMD. (See bit enumerations for PPS output frequency, rollover, and PPS commands.) Programming these bits with a non-zero value instructs the core to initiate an event. After the command is transferred or synchronized to the PTP clock domain, these bits gets cleared automatically. Software should ensure that these bits are programmed only when they are all-zero. All values not shown in the bit enumerations are reserved. For more information about the EMAC_TM_PPSTL.PPSTL0 bits, see the pulse-per-second functional description.
		0 CMD=No Command
		1 CMD=START Single; BR=2kHz; DR=1kHz For more info, see register description.
		2 CMD=START Pulse; BR=4kHz; DR=2kHz For more info, see register description.
		3 CMD=Cancel START; BR=8kHz; DR=4kHz For more info, see register description.
		4 CMD=STOP Pulse Time; BR=16kHz; DR=8kHz For more info, see register description.
		5 CMD=STOP Pulse Now For more info, see register description.
		6 CMD=Cancel STOP Pulse For more info, see register description.

Time Stamp Low Seconds Register

The `EMAC_TM_SEC` register contains the lower 32 bits of the seconds field of the system time.

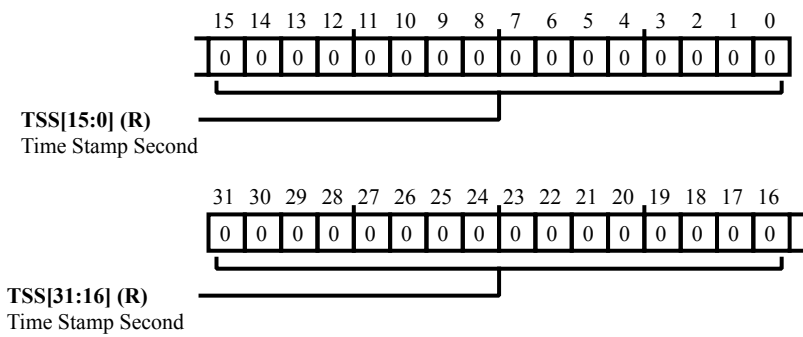


Figure 24-191: EMAC_TM_SEC Register Diagram

Table 24-223: EMAC_TM_SEC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	TSS	Time Stamp Second. The value in the <code>EMAC_TM_SEC.TSS</code> bit field indicates the current value in seconds of the System Time maintained by the core.

Time Stamp Seconds Update Register

The `EMAC_TM_SECUPDT` register contains the low 32 bits to be added to, subtracted from, or written to the seconds field of the system time.

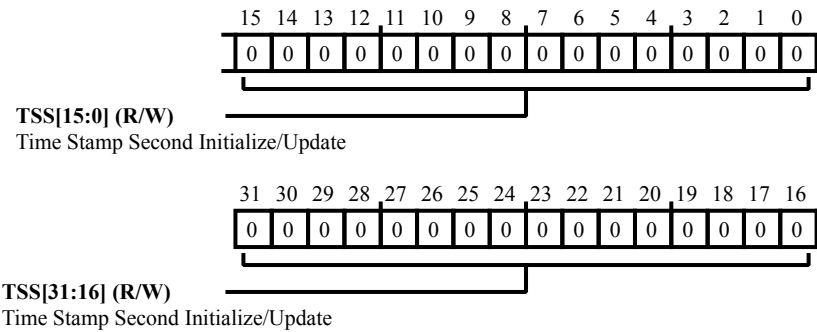


Figure 24-192: EMAC_TM_SECUPDT Register Diagram

Table 24-224: EMAC_TM_SECUPDT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	TSS	Time Stamp Second Initialize/Update. The value in the <code>EMAC_TM_SECUPDT.TSS</code> bit field indicates the time, in seconds, to be initialized or added to or subtracted from the system time seconds.

Time Stamp Status Register

The `EMAC_TM_STMPSTAT` register contains the PTP status.

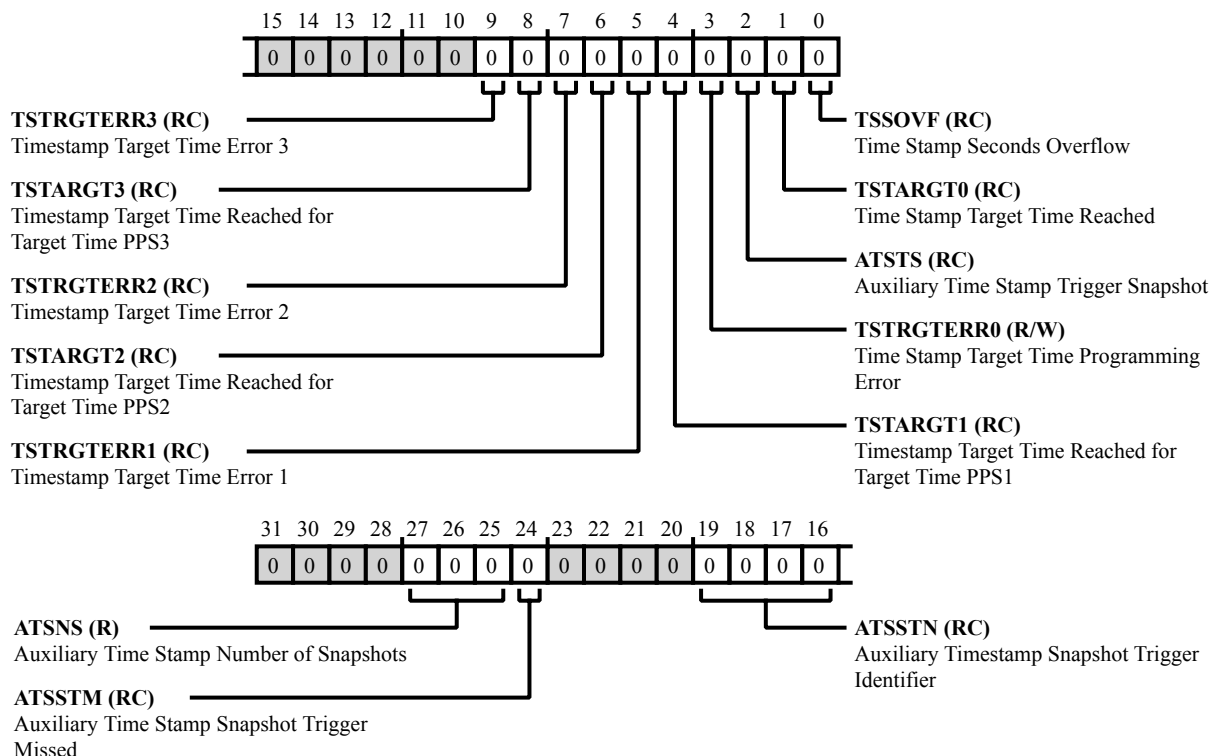


Figure 24-193: EMAC_TM_STMPSTAT Register Diagram

Table 24-225: EMAC_TM_STMPSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:25 (R/NW)	ATSNS	Auxiliary Time Stamp Number of Snapshots. The <code>EMAC_TM_STMPSTAT.ATSNS</code> bits indicate the number of Snapshots available in the FIFO. A value of 4 (100) indicates that the Auxiliary Snapshot FIFO is full. These bits are cleared (to 000) when the Auxiliary snapshot FIFO clear bit is set.
24 (RC/NW)	ATSSTM	Auxiliary Time Stamp Snapshot Trigger Missed. The <code>EMAC_TM_STMPSTAT.ATSSTM</code> bit is set when the Auxiliary time stamp snapshot FIFO is full and external trigger was set. This indicates that the latest snapshot was not stored in the FIFO.
19:16 (RC/NW)	ATSSTN	Auxiliary Timestamp Snapshot Trigger Identifier. The <code>EMAC_TM_STMPSTAT.ATSSTN</code> bit identify the Auxiliary trigger inputs for which the timestamp available in the Auxiliary Snapshot Register is applicable.

Table 24-225: EMAC_TM_STMPSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (RC/NW)	TSTRGTERR3	Timestamp Target Time Error 3. The EMAC_TM_STMPSTAT.TSTRGTERR3 bit when set the target time being programmed in PPS3 Target Time High Register and PPS3 Target Time Low Register, is already elapsed.
8 (RC/NW)	TSTARGET3	Timestamp Target Time Reached for Target Time PPS3. The EMAC_TM_STMPSTAT.TSTARGET3 bit indicates that the value of system time is greater than or equal to the value specified in PPS3 Target Time High Register and PPS3 Target Time Low Register.
7 (RC/NW)	TSTRGTERR2	Timestamp Target Time Error 2. The EMAC_TM_STMPSTAT.TSTRGTERR2 bit when set the target time being programmed in PPS2 Target Time High Register and PPS2 Target Time Low Register, is already elapsed.
6 (RC/NW)	TSTARGET2	Timestamp Target Time Reached for Target Time PPS2. The EMAC_TM_STMPSTAT.TSTARGET2 bit indicates that the value of system time is greater than or equal to the value specified in PPS2 Target Time High Register and PPS2 Target Time Low Register.
5 (RC/NW)	TSTRGTERR1	Timestamp Target Time Error 1. The EMAC_TM_STMPSTAT.TSTRGTERR1 bit when set the target time being programmed in PPS1 Target Time High Register and PPS1 Target Time Low Register, is already elapsed.
4 (RC/NW)	TSTARGET1	Timestamp Target Time Reached for Target Time PPS1. The EMAC_TM_STMPSTAT.TSTARGET1 bit indicates that the value of system time is greater than or equal to the value specified in PPS1 Target Time High Register and PPS1 Target Time Low Register.
3 (R/W)	TSTRGTERR0	Time Stamp Target Time Programming Error. The EMAC_TM_STMPSTAT.TSTRGTERR0 bit is set when the target time, which is being programmed in the EMAC_TM_SEC and EMAC_TM_NSEC registers, has already elapsed. This bit is cleared when read by the application.
2 (RC/NW)	ATSTS	Auxiliary Time Stamp Trigger Snapshot. The EMAC_TM_STMPSTAT.ATSTS bit is set high when the auxiliary snapshot is written to the FIFO.
1 (RC/NW)	TSTARGET0	Time Stamp Target Time Reached. The EMAC_TM_STMPSTAT.TSTARGET0 bit, when set, indicates the value of system time has reached or passed the value specified in the EMAC_TM_PPS0TGTM and EMAC_TM_NSEC registers.

Table 24-225: EMAC_TM_STMPSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (RC/NW)	TSSOVF	Time Stamp Seconds Overflow. The EMAC_TM_STMPSTAT.TSSOVF bit, when set, indicates that the seconds value of the time stamp (when supporting PTP version 2 format) has overflowed beyond 0xFFFF_FFFF.

Time Stamp Sub Second Increment Register

The `EMAC_TM_SUBSEC` register contains the value by which the system time nano second is incremented.

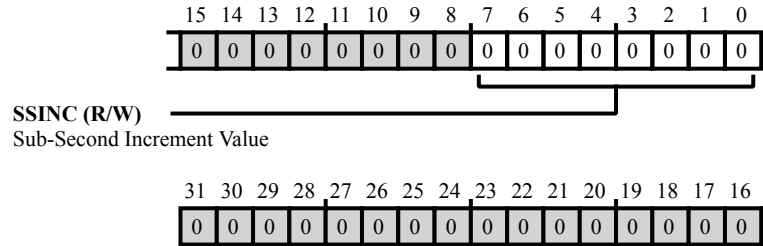


Figure 24-194: EMAC_TM_SUBSEC Register Diagram

Table 24-226: EMAC_TM_SUBSEC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	SSINC	Sub-Second Increment Value. The value in the <code>EMAC_TM_SUBSEC.SSINC</code> bits is accumulated every PTP clock cycle with the contents of the nanosecond register. For example, when PTP clock is 50 MHz (period is 20 ns), the processor should program 20 (0x14) when the <code>EMAC_TM_NSEC</code> register has an accuracy of 1 ns (<code>EMAC_TM_CTL.TSCTRLSSR</code> bit is set). When <code>EMAC_TM_CTL.TSCTRLSSR</code> is clear, the <code>EMAC_TM_NSEC</code> register has a resolution of ~0.465ns. In this case, the processor should program a value of 43 (0x2B) that is derived by $20\text{ns}/0.465$.

Tx 1024- to Max-Byte Frames (Good/Bad) Register

The `EMAC_TX1024TOMAX_GB` register contains the count of the number of good and bad frames transmitted with length between 1024 and maxsize (inclusive) bytes, exclusive of preamble and retried frames.

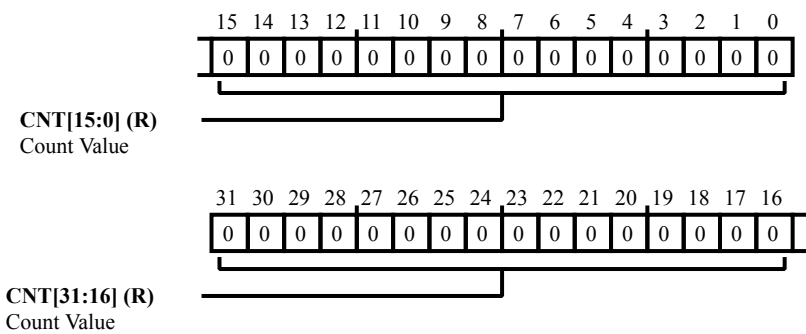


Figure 24-195: EMAC_TX1024TOMAX_GB Register Diagram

Table 24-227: EMAC_TX1024TOMAX_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx 128- to 255-Byte Frames (Good/Bad) Register

The `EMAC_TX128TO255_GB` register contains the count of the number of good and bad frames transmitted with length between 128 and 255 (inclusive) bytes, exclusive of preamble and retried frames.

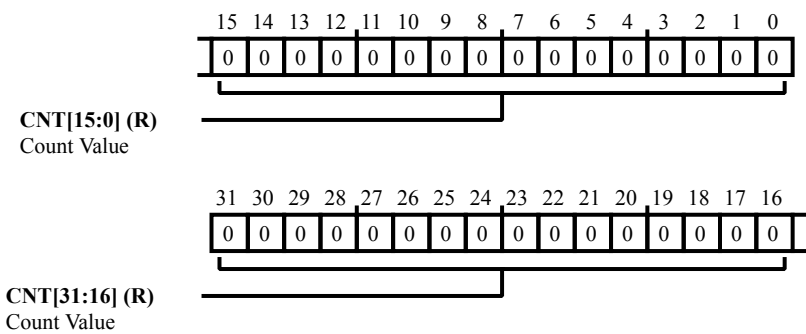


Figure 24-196: EMAC_TX128TO255_GB Register Diagram

Table 24-228: EMAC_TX128TO255_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx 256- to 511-Byte Frames (Good/Bad) Register

The `EMAC_TX256TO511_GB` register contains the count of the number of good and bad frames transmitted with length between 256 and 511 (inclusive) bytes, exclusive of preamble and retried frames.

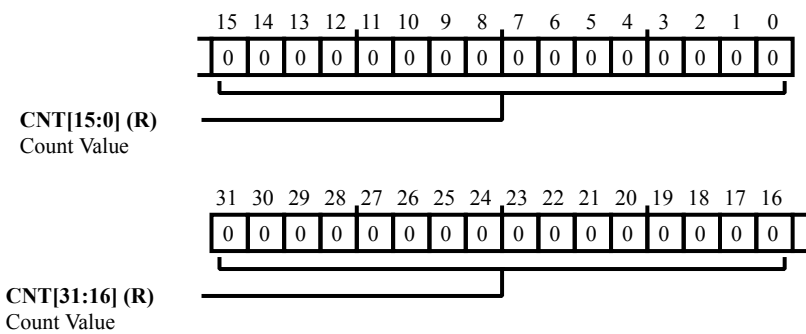


Figure 24-197: EMAC_TX256TO511_GB Register Diagram

Table 24-229: EMAC_TX256TO511_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx 512- to 1023-Byte Frames (Good/Bad) Register

The `EMAC_TX512TO1023_GB` register contains the count of the number of good and bad frames transmitted with length between 512 and 1023 (inclusive) bytes, exclusive of preamble and retried frames.

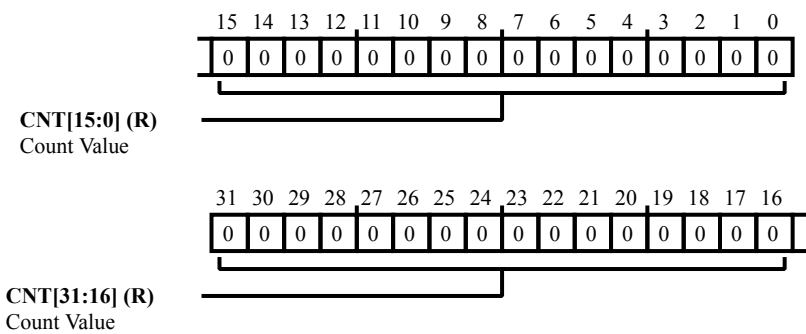


Figure 24-198: EMAC_TX512TO1023_GB Register Diagram

Table 24-230: EMAC_TX512TO1023_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx 64-Byte Frames (Good/Bad) Register

The `EMAC_TX64_GB` register contains the count of the number of good and bad frames transmitted with length 64 bytes, exclusive of preamble and retried frames.

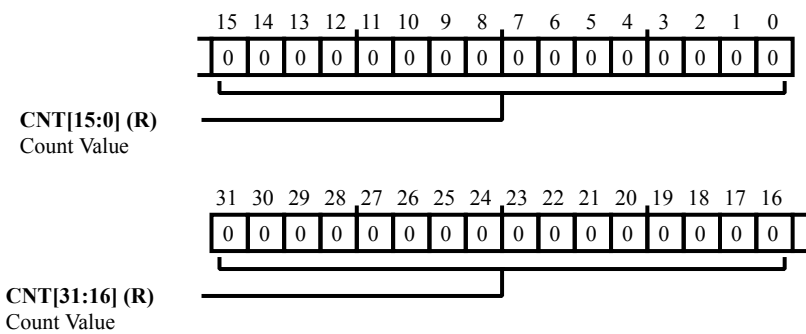


Figure 24-199: EMAC_TX64_GB Register Diagram

Table 24-231: EMAC_TX64_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx 65- to 127-Byte Frames (Good/Bad) Register

The `EMAC_TX65TO127_GB` register contains the count of the number of good and bad frames transmitted with length between 65 and 127 (inclusive) bytes, exclusive of preamble and retried frames.

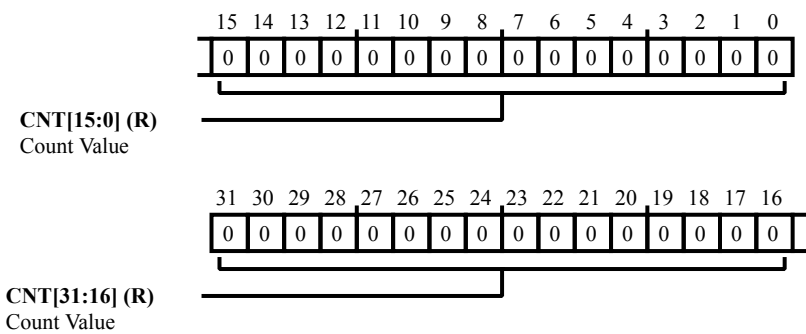


Figure 24-200: EMAC_TX65TO127_GB Register Diagram

Table 24-232: EMAC_TX65TO127_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Broadcast Frames (Good) Register

The `EMAC_TXBCASTFRM_G` register contains the count of the number of good broadcast frames transmitted.

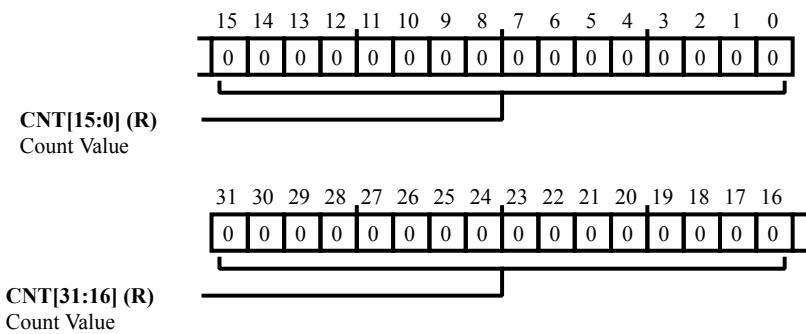


Figure 24-201: EMAC_TXBCASTFRM_G Register Diagram

Table 24-233: EMAC_TXBCASTFRM_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Broadcast Frames (Good/Bad) Register

The `EMAC_TXBCASTFRM_GB` register contains the count of the number of good and bad broadcast frames transmitted.

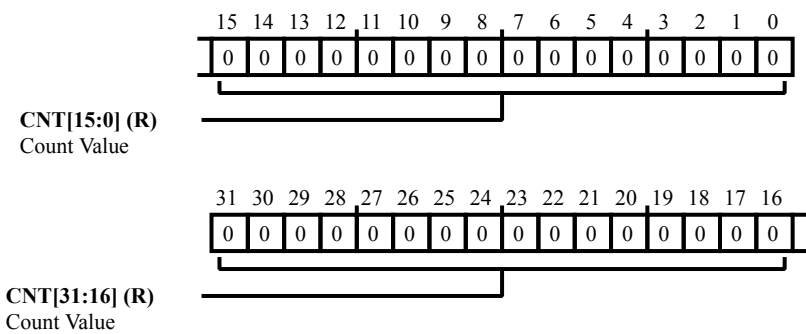


Figure 24-202: EMAC_TXBCASTFRM_GB Register Diagram

Table 24-234: EMAC_TXBCASTFRM_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Carrier Error Register

The `EMAC_TXCARR_ERR` register contains a count of the number of frames aborted due to carrier sense error (no carrier or loss of carrier).

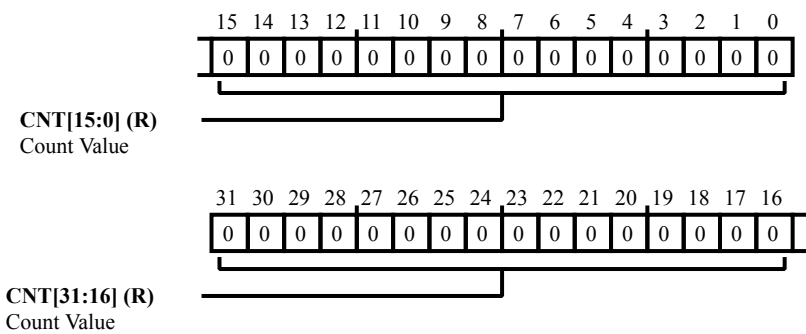


Figure 24-203: EMAC_TXCARR_ERR Register Diagram

Table 24-235: EMAC_TXCARR_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Deferred Register

The `EMAC_TXDEFERRED` register contains a count of the number of successfully transmitted frames after a deferral in Half-duplex mode.

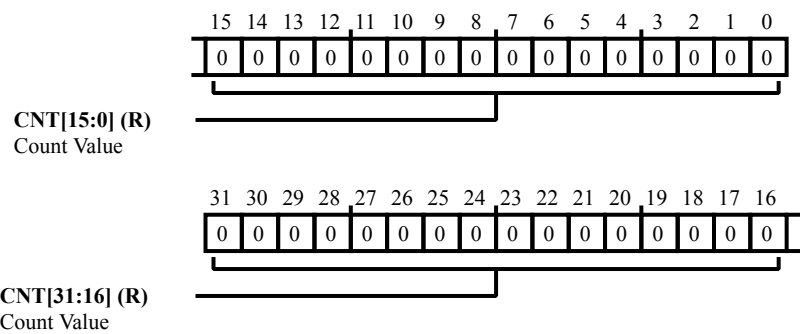


Figure 24-204: EMAC_TXDEFERRED Register Diagram

Table 24-236: EMAC_TXDEFERRED Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Excess Collision Register

The `EMAC_TXEXCESSCOL` register contains a count of the number of frames aborted due to excessive (16) collision errors.

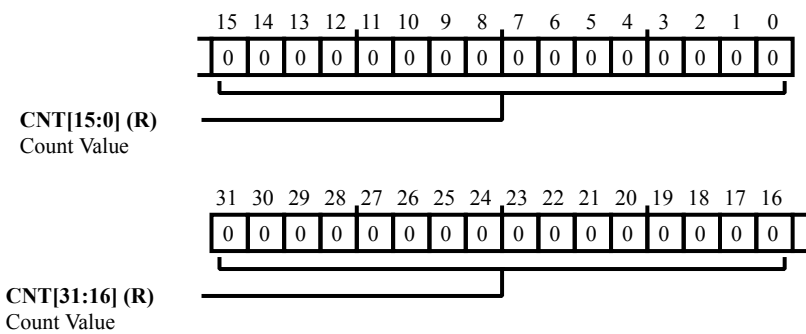


Figure 24-205: EMAC_TXEXCESSCOL Register Diagram

Table 24-237: EMAC_TXEXCESSCOL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Excess Deferral Register

The `EMAC_TXEXCESSDEF` register contains a count of the number of frames aborted due to excessive deferral error (deferred for more than two max-sized frame times).

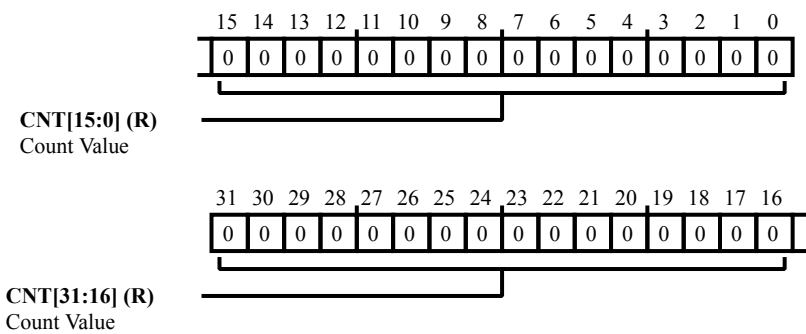


Figure 24-206: EMAC_TXEXCESSDEF Register Diagram

Table 24-238: EMAC_TXEXCESSDEF Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Frame Count (Good) Register

The `EMAC_TXFRMCNT_G` register contains a count of the number of good frames transmitted.

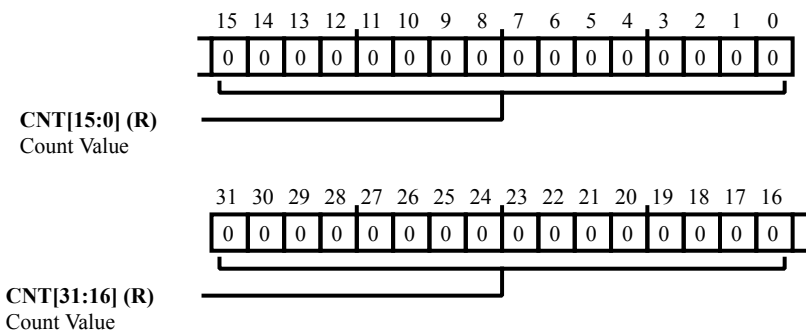


Figure 24-207: EMAC_TXFRMCNT_G Register Diagram

Table 24-239: EMAC_TXFRMCNT_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Frame Count (Good/Bad) Register

The `EMAC_TXFRMCNT_GB` register contains the count of the number of good and bad frames transmitted, exclusive of retried frames.

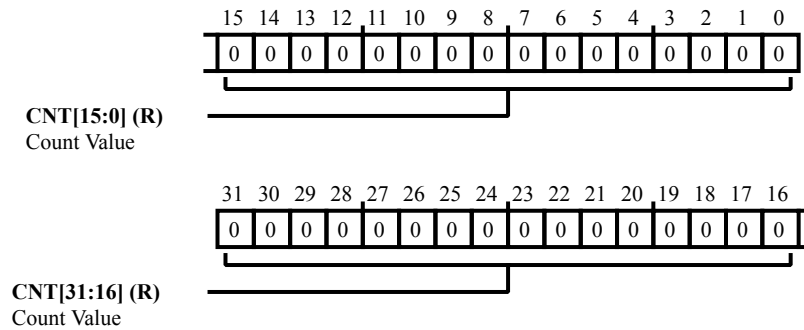


Figure 24-208: EMAC_TXFRMCNT_GB Register Diagram

Table 24-240: EMAC_TXFRMCNT_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Late Collision Register

The `EMAC_TXLATECOL` register contains a count of the number of frames aborted due to late collision error.

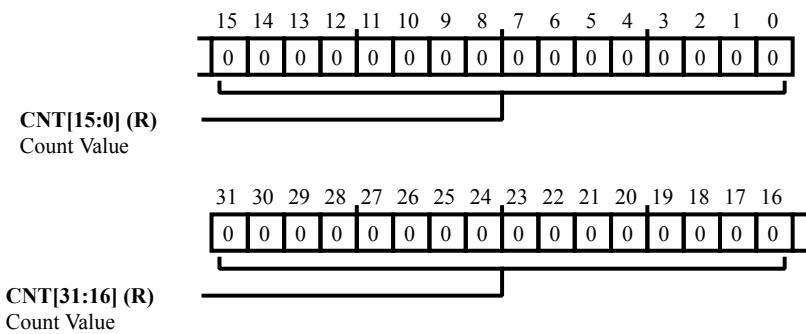


Figure 24-209: EMAC_TXLATECOL Register Diagram

Table 24-241: EMAC_TXLATECOL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Multicast Frames (Good) Register

The `EMAC_TXMCASTFRM_G` register contains the count of the number of good multicast frames transmitted.

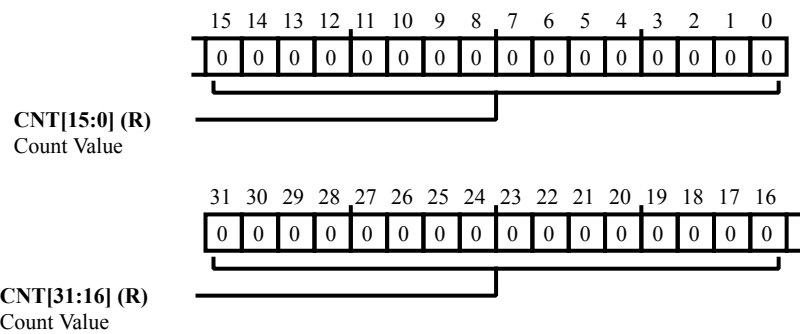


Figure 24-210: EMAC_TXMCASTFRM_G Register Diagram

Table 24-242: EMAC_TXMCASTFRM_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Multicast Frames (Good/Bad) Register

The `EMAC_TXMCASTFRM_GB` register contains the count of the number of good and bad multicast frames transmitted.

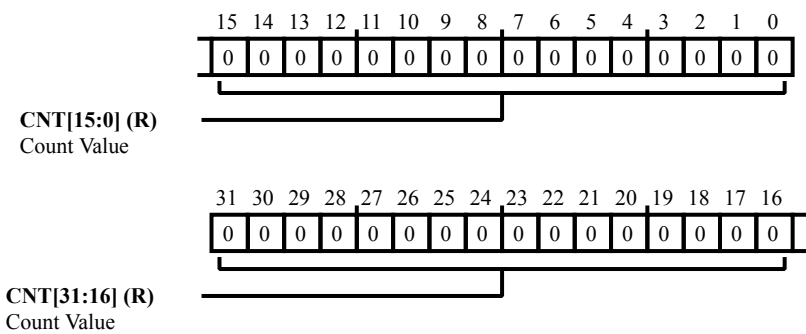


Figure 24-211: EMAC_TXMCASTFRM_GB Register Diagram

Table 24-243: EMAC_TXMCASTFRM_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Multiple Collision (Good) Register

The `EMAC_TXMULTCOL_G` register contains a count of the number of successfully transmitted frames after more than a single collision in Half-duplex mode.

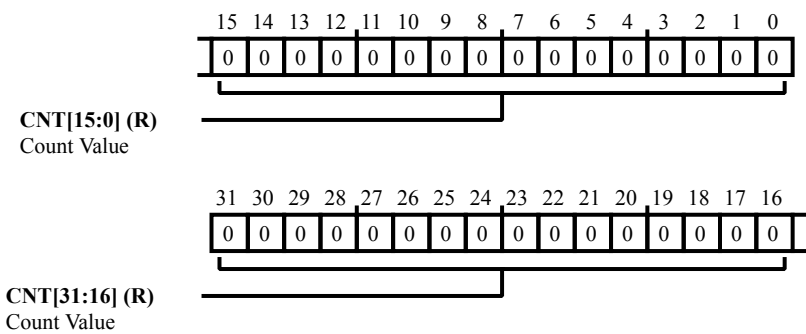


Figure 24-212: EMAC_TXMULTCOL_G Register Diagram

Table 24-244: EMAC_TXMULTCOL_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Octet Count (Good) Register

The `EMAC_TXOCTCNT_G` register contains a count of the number of bytes transmitted, exclusive of preamble, in good frames only.

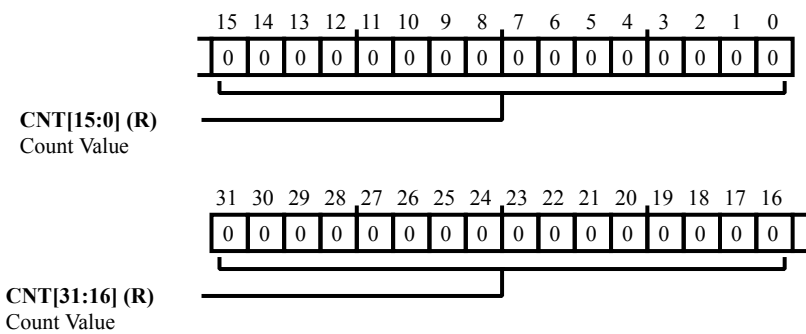


Figure 24-213: EMAC_TXOCTCNT_G Register Diagram

Table 24-245: EMAC_TXOCTCNT_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx OCT Count (Good/Bad) Register

The `EMAC_TXOCTCNT_GB` register contains the count of the number of bytes transmitted, exclusive of the preamble and retried bytes, in good and bad frames.

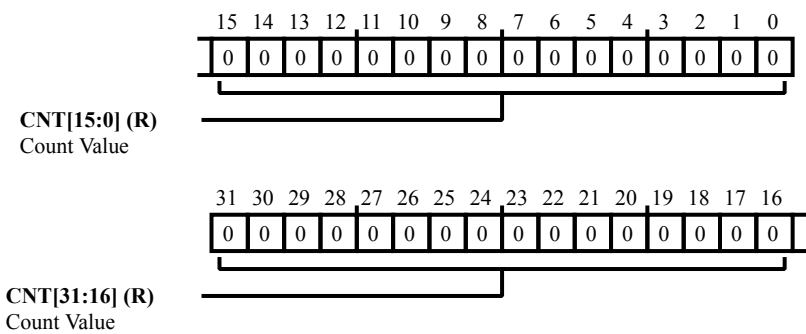


Figure 24-214: EMAC_TXOCTCNT_GB Register Diagram

Table 24-246: EMAC_TXOCTCNT_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Number of Tx Frames (Good) greater than maxsize

The `EMAC_TXOVRSIZE_G` register contains a count of the number of good frames transmitted with length greater than the maxsize.

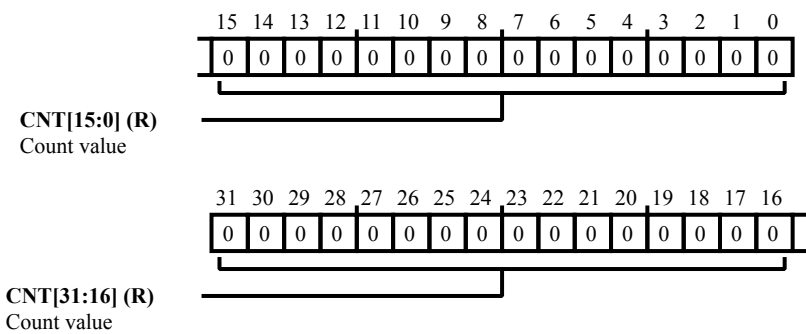


Figure 24-215: EMAC_TXOVRSIZE_G Register Diagram

Table 24-247: EMAC_TXOVRSIZE_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count value.

Tx Pause Frame Register

The `EMAC_TXPAUSEFRM` register contains a count of the number of good PAUSE frames transmitted.

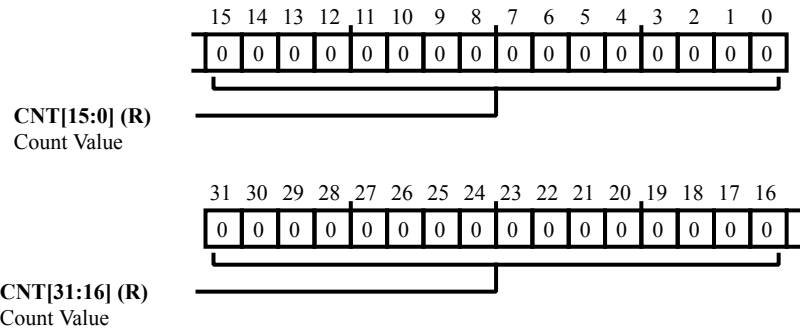


Figure 24-216: `EMAC_TXPAUSEFRM` Register Diagram

Table 24-248: `EMAC_TXPAUSEFRM` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Single Collision (Good) Register

The `EMAC_TXSNGCOL_G` register contains a count of the number of successfully transmitted frames after a single collision in Half-duplex mode.

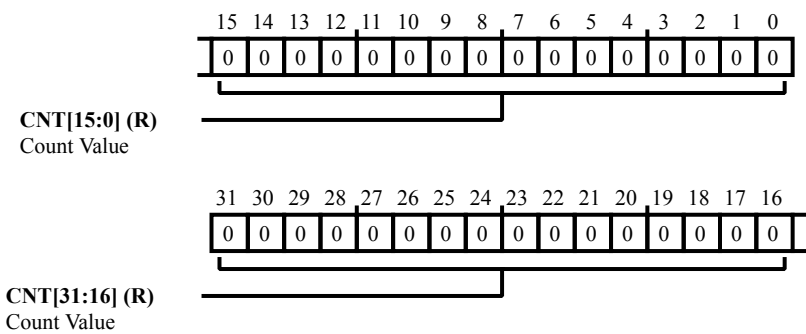


Figure 24-217: EMAC_TXSNGCOL_G Register Diagram

Table 24-249: EMAC_TXSNGCOL_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Unicast Frames (Good/Bad) Register

The `EMAC_TXUCASTFRM_GB` register contains the count of the number of good and bad unicast frames transmitted.

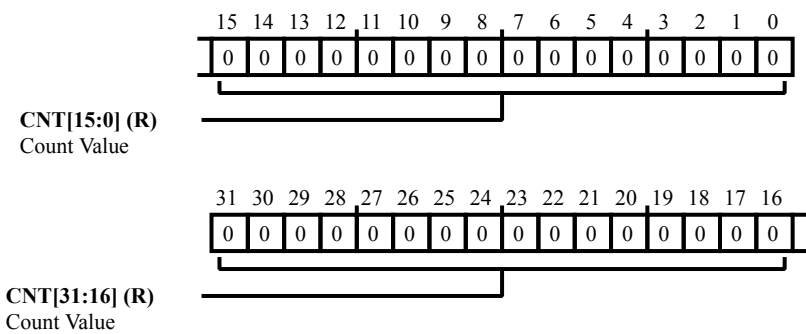


Figure 24-218: EMAC_TXUCASTFRM_GB Register Diagram

Table 24-250: EMAC_TXUCASTFRM_GB Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx Underflow Error Register

The `EMAC_TXUNDR_ERR` register contains a count of the number of frames aborted due to frame underflow error.

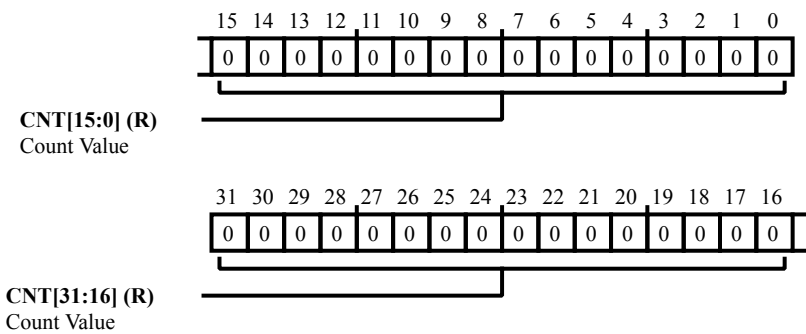


Figure 24-219: EMAC_TXUNDR_ERR Register Diagram

Table 24-251: EMAC_TXUNDR_ERR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

Tx VLAN Frames (Good) Register

The `EMAC_TXVLANFRM_G` register contains a count of the number of good VLAN frames transmitted, exclusive of retried frames.

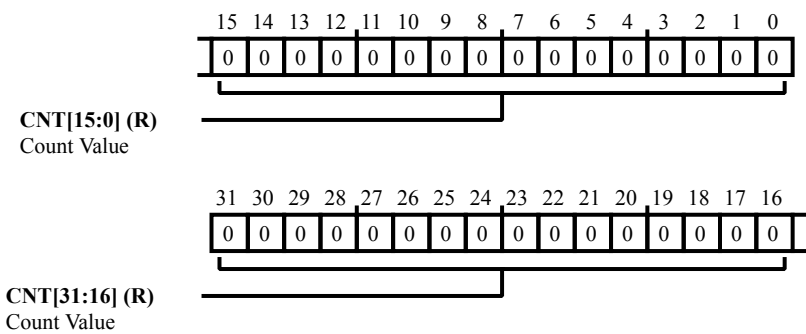


Figure 24-220: EMAC_TXVLANFRM_G Register Diagram

Table 24-252: EMAC_TXVLANFRM_G Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT	Count Value.

VLAN Tag Register

The `EMAC_VLANTAG` register contains the VLAN tag.

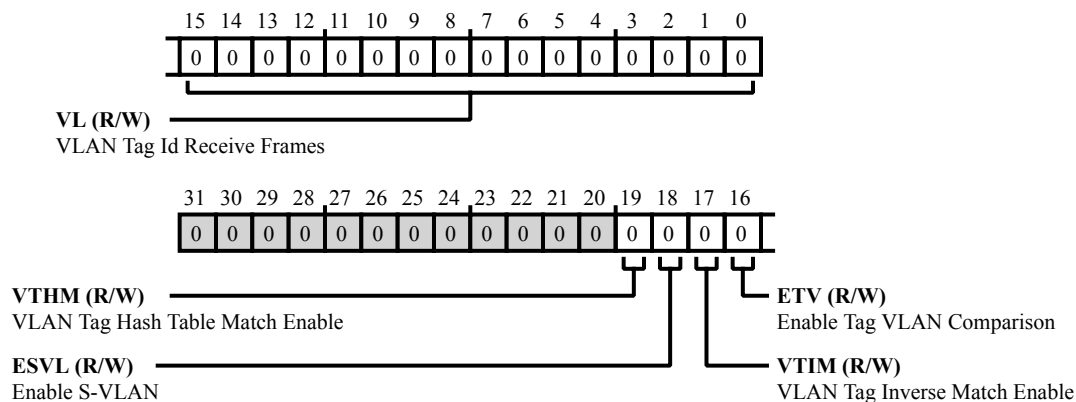


Figure 24-221: EMAC_VLANTAG Register Diagram

Table 24-253: EMAC_VLANTAG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	VTHM	VLAN Tag Hash Table Match Enable. The <code>EMAC_VLANTAG.VTHM</code> bit, When set, the most significant four bits of the VLAN tags CRC are used to index the content of VLAN Hash Table Register. A value of 1 in the VLAN Hash Table register, corresponding to the index, indicates that the frame matched the VLAN hash table.
18 (R/W)	ESVL	Enable S-VLAN. When this bit is set, the MAC transmitter and receiver also consider the S-VLAN (Type = 0x88A8) frames as valid VLAN tagged frames.
17 (R/W)	VTIM	VLAN Tag Inverse Match Enable. The <code>EMAC_VLANTAG.VTIM</code> bit, enables the VLAN Tag inverse matching. The frames that do not have matching VLAN Tag are marked as matched.
16 (R/W)	ETV	Enable Tag VLAN Comparison. The <code>EMAC_VLANTAG.ETV</code> bit, when set, directs the EMAC to use a 12-bit VLAN identifier, rather than the complete 16-bit VLAN tag, for comparison and filtering. Bits[11:0] of the VLAN tag are compared with the corresponding field in the received VLAN-tagged frame. When this bit is reset, all 16 bits of the received VLAN frame's fifteenth and sixteenth bytes are used for comparison.

Table 24-253: EMAC_VLANTAG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VL	VLAN Tag Id Receive Frames. The <code>EMAC_VLANTAG.VL</code> bits contain the 802.1Q VLAN tag to identify VLAN frames, and is compared to the fifteenth and sixteenth bytes of the frames being received for VLAN frames. Bits[15:13] are the User Priority, Bit[12] is the Canonical Format Indicator (CFI) and bits[11:0] are the VLAN tag's VLAN Identifier (VID) field. When the ETV bit is set, only the VID (Bits[11:0]) is used for comparison. If VL (VL[11:0] if ETV is set) is all zeros, the MAC does not check the fifteenth and sixteenth bytes for VLAN tag comparison, and declares all frames with a Type field value of 0x8100 to be VLAN frames.

VLAN Hash Table Register

The `EMAC_VLAN_HSHTBL` register contains the VLAN hash table

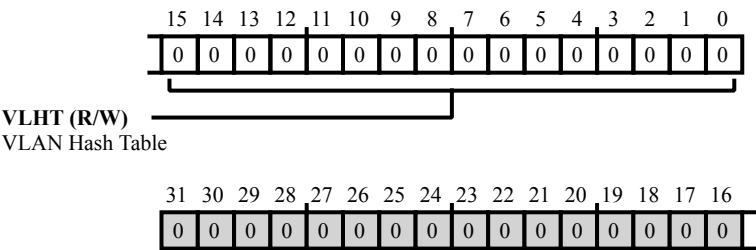


Figure 24-222: EMAC_VLAN_HSHTBL Register Diagram

Table 24-254: EMAC_VLAN_HSHTBL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VLHT	VLAN Hash Table. This field contains the 16-bit VLAN Hash Table.

VLAN Tag Inclusion or Replacement Register

The `EMAC_VLAN_INCL` register contains the VLAN tag for insertion into or replacement in the transmit frames.

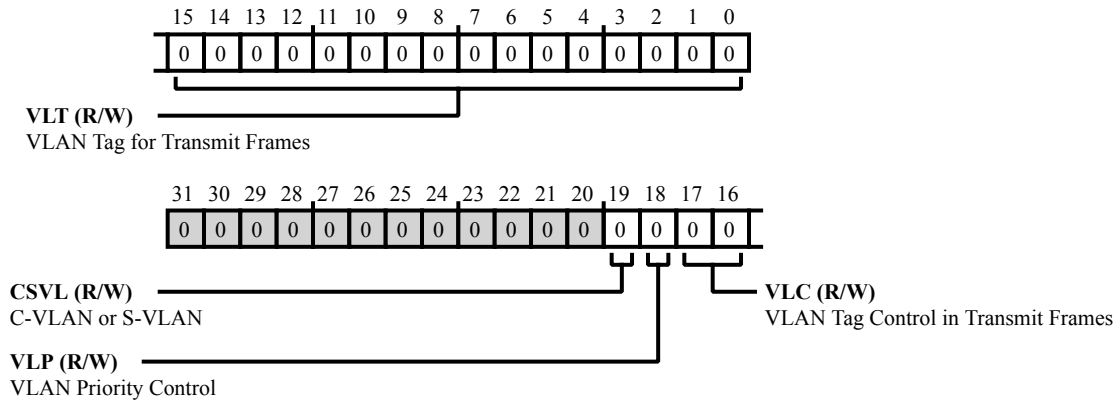


Figure 24-223: EMAC_VLAN_INCL Register Diagram

Table 24-255: EMAC_VLAN_INCL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	CSVL	C-VLAN or S-VLAN. The <code>EMAC_VLAN_INCL.CSVL</code> bit, When this bit is set, S-VLAN type (0x88A8) is inserted or replaced in the 13th and 14th bytes of transmitted frames.
18 (R/W)	VLP	VLAN Priority Control. The <code>EMAC_VLAN_INCL.VLP</code> bit, When this bit is set, the control Bits [17:16] are used for VLAN deletion, insertion, or replacement
17:16 (R/W)	VLC	VLAN Tag Control in Transmit Frames. The <code>EMAC_VLAN_INCL.VLC</code> bit, 2'b00:No VLAN tag deletion, insertion, or replacement 2'b01:VLAN tag deletion. The MAC removes the VLAN type (bytes 13 and 14) and VLAN tag (bytes 15 and 16) of all transmitted frames with VLAN tags. 2'b10:VLAN tag insertion. The MAC inserts VLT in bytes 15 and 16 of the frame after inserting the Type value (0x8100/0x88a8) in bytes 13 and 14. This operation is performed on all transmitted frames, irrespective of whether they already have a VLAN tag. 2'b11:VLAN tag replacement. The MAC replaces VLT in bytes 15 and 16 of all VLAN-type transmitted frames (Bytes 13 and 14 are 0x8100/0x88a8).
15:0 (R/W)	VLT	VLAN Tag for Transmit Frames. The <code>EMAC_VLAN_INCL.VLT</code> bit, contains the value of the VLAN tag to be inserted or replaced.

Watchdog Timeout Register

The `EMAC_WDOG_TIMEOUT` register controls the watchdog timeout for received frames.

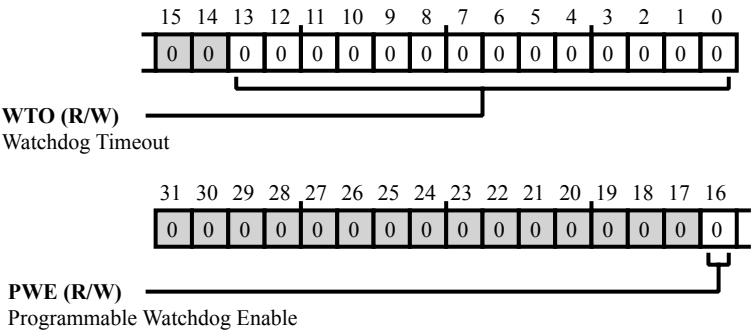


Figure 24-224: `EMAC_WDOG_TIMEOUT` Register Diagram

Table 24-256: `EMAC_WDOG_TIMEOUT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	PWE	Programmable Watchdog Enable. The <code>EMAC_WDOG_TIMEOUT.PWE</code> bit, When this bit is set and Bit 23 (WD) of Register 0 (MAC Configuration Register) is reset, the WTO field (Bits[13:0]) is used as watchdog timeout for a received frame.
13:0 (R/W)	WTO	Watchdog Timeout. The <code>EMAC_WDOG_TIMEOUT.WTO</code> bit, When Bit 16 (PWE) is set and Bit 23 (WD) of Register 0 (MAC Configuration Register) is reset, this field is used as watchdog timeout for a received frame. If the length of a received frame exceeds the value of this field, such frame is terminated and declared as an error frame.

25 Digital Audio Interface (DAI)

The Digital Audio Interfaces (DAI_n) are comprised of groups of identical peripherals and their respective Signal Routing Units (SRU_n). The SRU connects inputs and outputs of the DAI peripherals with each other and to the external pins. This configuration allows peripherals to be interconnected to accommodate a wide variety of systems without making external pin connections.

In a typical processor, static (multiplexed) pins are assigned to specific peripherals. When certain peripherals are not required for an application, these pins are unnecessary and expensive. The pins may need to be defined as high or low to prevent any illegal conditions. The signal routing units on the SHARC processors address this situation by controlling a number of general-purpose pins which can be assigned flexibly (a virtual connectivity between peripherals) depending on system requirements. This virtual connectivity includes pin buffers and routing logic (multiplexer). It also allows the SHARC processors to include an arbitrary number and variety of peripherals while retaining high levels of compatibility without increasing pin count.

SRU Features

The SRU has the following features and capabilities.

- Flexible connections that can be made through software and during run time; no hard-wiring is required.
- At reset, a default routing scheme is already programmed.
- Connectivity can be made internally between peripherals, externally between pin buffers, or a mix of both.
- Status of the pin buffers can be programmed for conditional execution or interrupts.
- Some pin buffers allow control of signal polarity changes.
- No fan-out limitation, a peripheral or pin buffer output that can be routed to multiple peripheral or pin buffer inputs.
- Two independent routing systems are available: the DAI0 and the DAI1. Signals cannot be interconnected between both routing units with the exception of the following modules under some specific scenarios.
 - The precision clock generator (PCG)
 - A pair of DAI pin buffers from each DAI unit
 - Asynchronous Sample Rate Converter (ASRC) data for daisy chaining across the DAI units

NOTE: The 400-ball CSP BGA package has 2 x 14 DAI pins bonded off-chip (DAIx_PIN01 through DAIx_PIN12, DAIx_PIN19, and DAIx_PIN20). The 120-lead LQFP-EP package has 2 x 12 DAI pins bonded off-chip (DAIx_PIN01 through DAIx_PIN10, DAIx_PIN19, and DAIx_PIN20).

Functional Description

The fundamental timing clock of the DAI modules is SCLK0.

The [DAI Block Diagram](#) shows how the DAI pin buffers are connected through the SRUn. This configuration allows for flexible signal routing.

The DAI units are comprised of four primary blocks:

- Peripherals (A/B/C) associated with DAI_n
- Signal Routing Units (SRUn)
- DAI_n I/O pin buffers
- Miscellaneous buffers

The peripherals shown in [DAI Block Diagram](#) can have up to three connections (if master or slave capable); one acts as a signal input, one as a signal output and the third as an output enable. The SRUs are based on a group of multiplexers which are controlled by registers to establish the desired interconnections. The DAI pin buffers have three signals which are used for input and output to or from off-chip and the third for output enable.

The miscellaneous buffers have an input and an output and are used for group interconnection.

The figures are a simplified representation of a DAI system. In a real representation, the SRU and DAI would show several types of data being routed from several sources including the following:

- Serial ports (SPORT)
- Precision clock generators (PCG)
- Asynchronous sample rate converters (SRC)
- S/PDIF transmitter
- S/PDIF receiver
- DAI interrupts (miscellaneous)
- PDM interface

ADSP-2159x DAI Register List

The Digital Audio Interfaces (DAI_n) contain groups of identical peripherals which can be connected internally between peripherals, externally between pin buffers, or a mix of both. This module contains the following registers.

Table 25-1: ADSP-2159x DAI Register List

Name	Description
DAI_CLK0	Clock Routing Control Register 0
DAI_CLK1	Clock Routing Control Register 1
DAI_CLK2	Clock Routing Control Register 2
DAI_CLK3	Clock Routing Control Register 3
DAI_CLK4	Clock Routing Control Register 4
DAI_CLK5	Clock Routing Control Register 5
DAI_DAT0	Serial Data Routing Control Register 0
DAI_DAT1	Serial Data Routing Control Register 1
DAI_DAT2	Serial Data Routing Control Register 2
DAI_DAT3	Serial Data Routing Control Register 3
DAI_DAT4	Serial Data Routing Control Register 4
DAI_DAT5	Serial Data Routing Control Register 5
DAI_DAT6	Serial Data Routing Control Register 6
DAI_EXTD_CLK0	Extended Clock Routing Control Register 0
DAI_EXTD_CLK1	Extended Clock Routing Control Register 1
DAI_EXTD_CLK2	Extended Clock Routing Control Register 2
DAI_EXTD_CLK3	Extended Clock Routing Control Register 3
DAI_EXTD_CLK4	Extended Clock Routing Control Register 4
DAI_EXTD_CLK5	Extended Clock Routing Control Register 5
DAI_EXTD_DAT0	Extended Serial Data Routing Control Register 0
DAI_EXTD_DAT1	Extended Serial Data Routing Control Register 1
DAI_EXTD_DAT2	Extended Serial Data Routing Control Register 2
DAI_EXTD_DAT3	Extended Serial Data Routing Control Register 3
DAI_EXTD_DAT4	Extended Serial Data Routing Control Register 4
DAI_EXTD_DAT5	Extended Serial Data Routing Control Register 5
DAI_EXTD_DAT6	Extended Serial Data Routing Control Register 6
DAI_EXTD_FS0	Extended Frame Sync Routing Control Register 0
DAI_EXTD_FS1	Extended Frame Sync Routing Control Register 1
DAI_EXTD_FS2	Extended Frame Sync Routing Control Register 2
DAI_EXTD_FS4	Extended Frame Sync Routing Control Register 4
DAI_EXTD_MISC0	Extended Miscellaneous Control Register 0

Table 25-1: ADSP-2159x DAI Register List (Continued)

Name	Description
DAI_EXTD_MISC1	Extended Miscellaneous Control Register 1
DAI_EXTD_MISC2	Extended Miscellaneous Control Register 2
DAI_EXTD_PBEN0	Extended Pin Buffer Enable Register 0
DAI_EXTD_PBEN1	Extended Pin Buffer Enable Register 1
DAI_EXTD_PBEN2	Extended Pin Buffer Enable Register 2
DAI_EXTD_PBEN3	Extended Pin Buffer Enable Register 3
DAI_EXTD_PIN0	Extended Pin Buffer Assignment Register 0
DAI_EXTD_PIN1	Extended Pin Buffer Assignment Register 1
DAI_EXTD_PIN2	Extended Pin Buffer Assignment Register 2
DAI_EXTD_PIN3	Extended Pin Buffer Assignment Register 3
DAI_EXTD_PIN4	Extended Pin Buffer Assignment Register 4
DAI_FS0	Frame Sync Routing Control Register 0
DAI_FS1	Frame Sync Routing Control Register 1
DAI_FS2	Frame Sync Routing Control Register 2
DAI_FS4	Frame Sync Routing Control Register 4
DAI_GBL_INT_EN	Global SPORT Interrupt Grouping Register
DAI_GBL_PCG_EN	Global PCG Enable Control Register
DAI_GBL_SP_EN	Global SPORT Enable Register
DAI_IMSK_FE	Falling-Edge Interrupt Mask Register
DAI_IMSK_PRI	Core Interrupt Priority Assignment Register
DAI_IMSK_RE	Rising-Edge Interrupt Mask Register
DAI_IRPTL_H	High Priority Interrupt Latch Register
DAI_IRPTL_HS	Shadow High Priority Interrupt Latch Register
DAI_IRPTL_L	Low Priority Interrupt Latch Register
DAI_IRPTL_LS	Shadow Low Priority Interrupt Latch Register
DAI_MISC0	Miscellaneous Control Register 0
DAI_MISC1	Miscellaneous Control Register 1
DAI_MISC2	Miscellaneous Control Register 1
DAI_PBEN0	Pin Buffer Enable Register 0
DAI_PBEN1	Pin Buffer Enable Register 1
DAI_PBEN2	Pin Buffer Enable Register 2

Table 25-1: ADSP-2159x DAI Register List (Continued)

Name	Description
DAI_PBEN3	Pin Buffer Enable Register 3
DAI_PIN0	Pin Buffer Assignment Register 0
DAI_PIN1	Pin Buffer Assignment Register 1
DAI_PIN2	Pin Buffer Assignment Register 2
DAI_PIN3	Pin Buffer Assignment Register 3
DAI_PIN4	Pin Buffer Assignment Register 4
DAI_PIN_STAT	Pin Status Register

ADSP-2159x DAI Interrupt List

Table 25-2: ADSP-2159x DAI Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
44	DAI0_IRQH	DAI0 High priority Interrupt	None	
45	DAI1_IRQH	DAI1 High priority Interrupt	None	
46	DAI0_IRQL	DAI0 Low Priority Interrupt	None	
47	DAI1_IRQL	DAI1 Low Priority Interrupt	None	
113	DAI0_GBL_SPORT_INT0	DAI0 Global SPORT Interrupt 0	None	
114	DAI0_GBL_SPORT_INT1	DAI0 Global SPORT Interrupt 1	None	
115	DAI1_GBL_SPORT_INT0	DAI1 Global SPORT Interrupt 0	None	
116	DAI1_GBL_SPORT_INT1	DAI1 Global SPORT Interrupt 1	None	

ADSP-2159x DAI Trigger List

Table 25-3: ADSP-2159x DAI Trigger List Masters

Trigger ID	Name	Description	Sensitivity
106	DAI0_GBL_SPORT_TRG_00	DAI0 SPORT GROUP0 Trigger Output	None
107	DAI0_GBL_SPORT_TRG_01	DAI0 SPORT GROUP1 Trigger Output	None
108	DAI1_GBL_SPORT_TRG_00	DAI1 SPORT GROUP2 Trigger Output	None
109	DAI1_GBL_SPORT_TRG_01	DAI1 SPORT Group3 Trigger Output	None

Table 25-4: ADSP-2159x DAI Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
76	DAI0_GBL_SPORT_TRG_I0	DAI0 SPORT GROUP0 Trigger Input	Pulse
77	DAI0_GBL_SPORT_TRG_I1	DAI0 SPORT GROUP1 Trigger Input	Pulse
78	DAI1_GBL_SPORT_TRG_I0	DAI1 SPORT GROUP2 Trigger Input	Pulse
79	DAI1_GBL_SPORT_TRG_I1	DAI1 SPORT GROUP3 Trigger Input	Pulse

DAI Block Diagram

The *DAI Functional Block Diagram* and the *Digital Audio Interconnect Unit* figures show the functional blocks within the DAI and the unit connections to the peripherals.

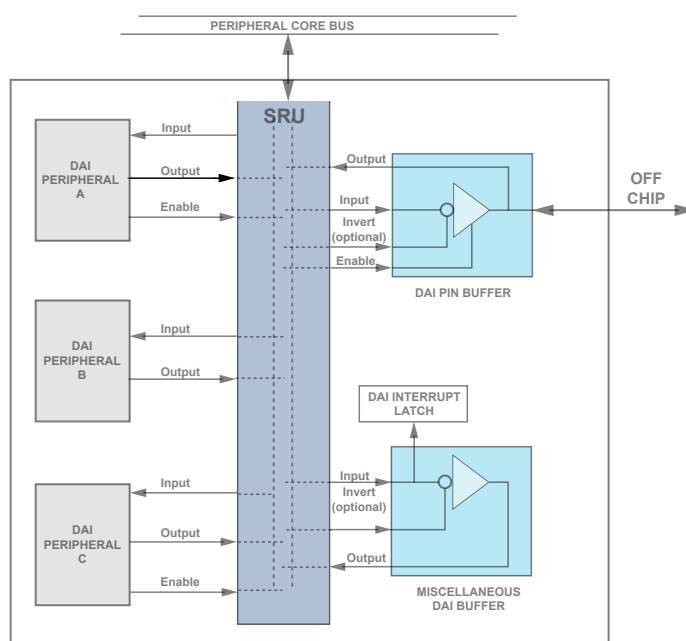


Figure 25-1: DAI Functional Block Diagram

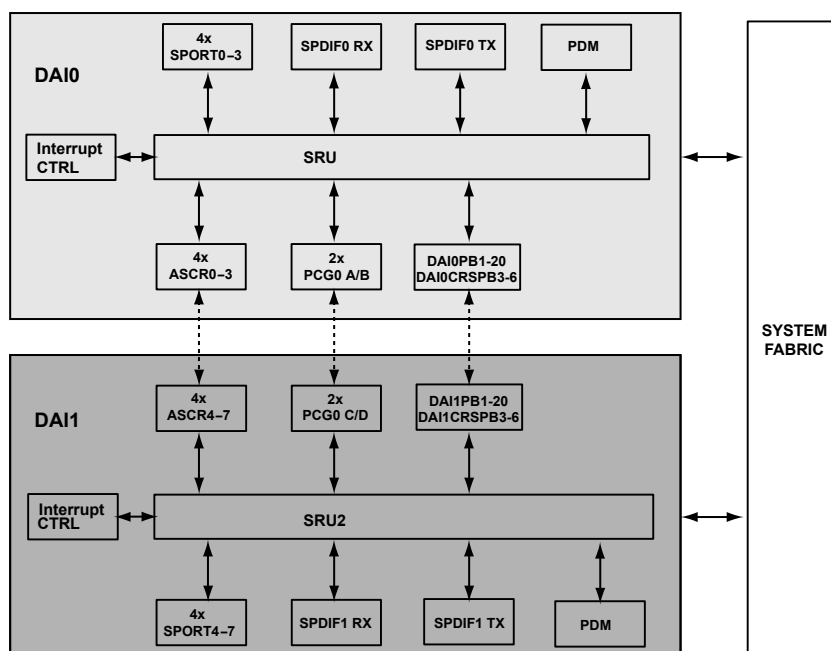


Figure 25-2: Digital Audio Interconnect Unit

DAI Signal Naming Conventions

The peripherals associated with the DAI do not have any dedicated I/O pins for off-chip communication. Instead, the I/O pin is only accessible in the chip internally and is known as an *internal node*. Every internal node of a DAI peripheral (input or output) is given a unique mnemonic. The convention is to begin the name with an identifier for the peripheral that the signal is coming to or from, followed by the function of the signal.

A number is included if the DAI contains more than one peripheral type (for example, serial ports), or if the peripheral has more than one signal that performs this function (for example, serial ports). The mnemonic always ends with *_I* if the signal is an input, or with *_O* if the signal is an output. An example is shown in the *Example DAI Mnemonics* figure.

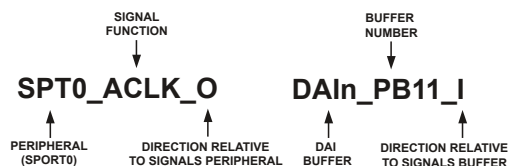


Figure 25-3: Example DAI Mnemonics

I/O Pin Buffers

Within the context of the SRU, physical connections to the DAI pins are replaced by a logical interface known as a *pin buffer*. This three terminal active device is capable of sourcing or sinking output current when its driver is enabled, and passing external input signals when disabled. Each pin has an input, an output, and an enable as shown in the *Pin Buffer Example* figure. The inputs and the outputs are defined with respect to the pin, similar to a peripheral device. This naming convention is consistent with the SRU naming convention.

Pin Buffer Signals

The pin buffer is based on three signals shown in the *Pin Buffer Example* figure and described in the following sections.

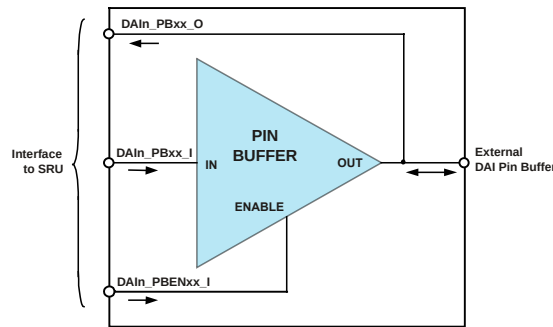


Figure 25-4: Pin Buffer Example

Pin Buffer Input Signal

A pin buffer input (DAIn_PBxx_I) is driven as an output from the processor when the pin buffer enable is set (=1). Each physical pin (connected to a bonded pad) can be connected through the SRU to any of the outputs of the DAI peripherals, based on the bit field values. The SRU can also be used to route signals that control the pins in other ways. Many signals can be configured for use as control signals.

Pin Buffer Enable Signal

When a pin buffer enable (DAIn_PBENxx_I) is set (=1), the signal present at the corresponding pin buffer input (DAIn_PBxx_I) is driven off-chip as an output. When a pin buffer enable is cleared (=0), the signal present at the corresponding pin buffer input is ignored. The pin enable control registers activate the drive buffer for each of the DAI pins. When the pins are not enabled (driven), they can be used as inputs. There are two options to control the pin buffer enable signal; setting the level high for a static solution, or connecting the dedicated peripheral's pin buffer output enable signal to its pin buffer, which automatically enables the pin buffer.

Pin Buffer Functions

Pin buffers can be configured as inputs or outputs as described in the following sections.

Pin Buffers as Signal Input

When the DAI pin is used only as an input, connect the corresponding pin buffer enable to logic low as shown in the *Pin Buffer as Input* figure. This configuration disables the buffer amplifier and allows an off-chip source to drive the value present on the DAI pin and at the pin buffer output. When the pin buffer enable (for example, DAI_PBEN0.PB01) is cleared (= 0), the pin buffer output (DAIn_PBxx_O) is the signal driven onto the DAI pin by an external source, and the pin buffer input signal (DAIn_PBxx_I) is not used.

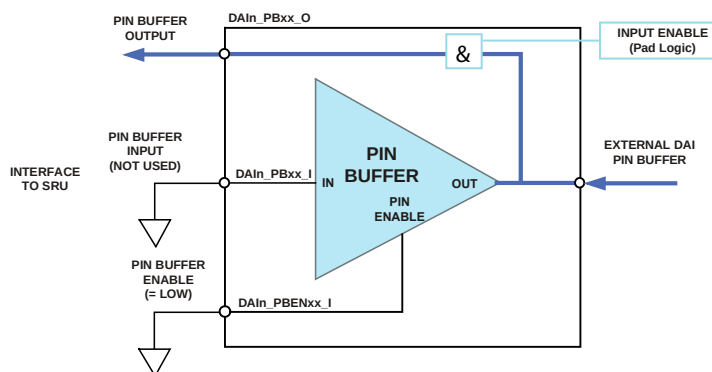


Figure 25-5: Pin Buffer as Input

NOTE: Whether programmed as input or output, a DAI buffer input always routes the same signal to an output internally. DAI pins have programmable internal pull-up resistors. For more information, see the General-Purpose Ports (PORT) chapter for details.

Pin Buffers As Signal Output

In a typical embedded system, most pins are designated as either inputs or outputs when the circuit is designed, even though they can be used in either direction. Each of the DAI pins can be used as either an output or an input. Although the direction of a DAI pin is set simply by writing to a memory-mapped register, most often the direction of the pin is dictated by the designated use of that pin.

When the DAI pin is used only as an output, connect the corresponding pin buffer enable to logic high as shown in the *Pin Buffer as Output* figure. This configuration enables the buffer amplifier to operate as a current source and to drive the value present at the pin buffer input onto the DAI pin and off-chip. When the pin buffer enable bits are set (in the `DAIn_PBxx_I` registers) ($=1$), the pin buffer output (`DAIn_PBxx_O`) is the same signal as the pin buffer input (`DAIn_PBxx_I`), and this signal is driven as an output.

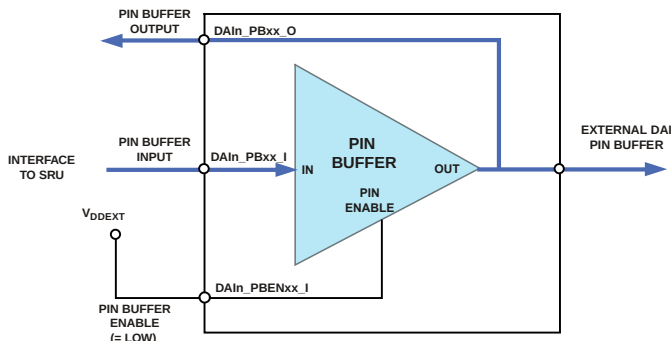


Figure 25-6: Pin Buffer as Output

DAI Pin Buffer Status

The signal levels on the DAI pins can be read with the `DAI_PIN_STAT` registers.

DAIn Peripherals

There are two categories of peripherals associated with the DAI units. These are described in the following sections.

Output Signals With Pin Buffer Enable Control

Many peripherals within the DAI that have bidirectional pins generate a corresponding pin enable signal. Typically, the settings within the control registers of a peripheral determine if a bidirectional pin is an input or an output. The pin is then driven accordingly.

Though most peripherals are capable of operating bidirectionally, it is not required that all of the `_I` and `_O` signals of a peripheral be connected to the pin buffer. If the system design only uses a signal in one direction, it is simpler to connect the pin buffer accordingly.

NOTE: All available pin buffer output enables must be routed to their pin buffer input enable signals in cases where data streaming connections are used. This arrangement guarantees timing requirements.

NOTE: In some cases, it is necessary to use a peripheral's dedicated pin buffer enable signal instead of static levels. For example, SPORT TDM mode requires the SPORT's dedicated data pin buffer enable signal to be used for the SPORT's data pin to three-state the data pins on inactive channels.

Output Signals Without Pin Buffer Enable Control

Some peripherals have signal outputs without an automated pin buffer control enable signal. The operation of these peripherals is simplified. The routing to a DAI pin buffer enable input requires a static high from the SRUn. In order to disable the pin buffer output, software must clear the pin buffer enable input accordingly.

Signal Routing Units (SRUs)

The following sections provide details specific to the SRUs.

Cross Mode Connections

The DAI allows a few signals to be interconnected across DAI units. These connections are commonly referred as *cross mode connections*. These cross mode connections are useful for the system designer to share signals across both DAI's so that synchronization can be achieved on the peripheral modules across DAI0 and DAI1.

NOTE: The 400-ball CSP BGA package has 2 x 14 DAI pins bonded off-chip (DAIx_PIN01 through DAIx_PIN12, DAIx_PIN19, and DAIx_PIN20). The 120-lead LQFP-EP package has 2 x 12 DAI pins bonded off-chip (DAIx_PIN01 through DAIx_PIN10, DAIx_PIN19, and DAIx_PIN20).

Shared Clock

This cross mode connection (see [Figure 25-7 Cross Connect Clock Example](#)) can be used to share a clock signal cross DAIs. DAI0_PIN03/DAI0_PIN05 and DAI1_PIN03/DAI1_PIN05 can be used to share clock signal across DAI. When DAI0_PIN03/DAI0_PIN05 is configured as input clock pin then this clock signal is available in Group A of both DAIs. This source signal is DAI0_PB03_O/DAI0_PB05_O in group A of DAI 0 and DAI1_CRS_PB03_O/DAI1_CRS_PB05_O in group A of DAI1.

Peripherals routed through DAI0 like PCG A and SPORT 0A can use DAI0_PB03_O/ DAI0_PB05_O as the input clock signal. Peripherals routed through DAI1 like PCG C, SPORT 4A can use DAI1_CRS_PB03/ DAI1_CRS_PB05 as the input clock signal. Similarly when DAI1_PIN03 is configured as the input clock pin, then this clock signal is available as DAI1_PB03_O/DAI1_PB05_O in group A of DAI1 and DAI0_CRS_PB03/DAI0_CRS_PB05 in group A of DAI0.

Shared Frame Sync

This cross mode connection can be used to share a frame sync signal cross DAIs. DAI0_PIN04/DAI0_PIN06 and DAI1_PIN04/DAI1_PIN06 can be used to share the frame sync signal across the DAIs. When DAI0_PIN04/DAI0_PIN06 is configured as the input frame sync pin, then this frame sync signal is available in Group C of both DAIs. This source signal is DAI0_PB04_O/DAI0_PB06_O in group C of DAI 0 and DAI1_CRS_PB04_O/DAI1_CRS_PB06_O in group C of DAI1.

Peripherals on DAI0 like SPORT 0A can use DAI0_PB04_O/DAI0_PB06_O as the input frame sync signal. The peripherals on DAI1 like SPORT 4A can use DAI1_CRS_PB04/DAI1_CRS_PB06 as the input sync signal. Similarly, when DAI1_PIN04 /DAI1_PIN06 is configured as input frame sync pin, then this frame sync signal is available as DAI1_PB04_O/DAI1_PB06_O in group A of DAI1 and DAI0_CRS_PB04/DAI0_CRS_PB06 in group C of DAI0.

Shared Pin Buffer

This cross mode connection can be used to connect input signals on DAI0_PIN03/DAI0_PIN05, DAI0_PIN04/DAI0_PIN06, DAI1_PIN03/DAI1_PIN05 and DAI1_PIN04 /DAI1_PIN06 to other DAI pins across DAI. When DAI0_PIN03/DAI0_PIN05 and DAI0_PIN04 are configured as input signals, these signals can be routed to the pin buffers of DAI1 in Group D using DAI1_CRS_PIN03/DAI1_CRS_PIN05 and DAI1_CRS_PIN04. When DAI1_PIN03/DAI1_PIN05 and DAI1_PIN04/DAI1_PIN06 are configured as input signals, they can be routed to the pin buffers of DAI0 in Group D using DAI0_CRS_PIN03/ DAI0_CRS_PIN05 and DAI0_CRS_PIN04/DAI0_CRS_PIN06 signals.

PCGs and Pin Buffers

This cross mode connection (see [Figure 25-8 Cross Connect PCG Example](#)) can be used to connect PCG output signals of one DAI to pin buffers of other DAI. For example, PCG C outputs of DAI 1 can be connected to pin buffers of DAI 0 using PCG0_CRS_CLKC_O of Group D.

ASRC TDM and Data Signal

This cross mode connection is available in Group B (Data Signals). This allows programs to connect the Data and TDM signals of SRC3 and SRC 7 across DAIs so that eight SRCs can be daisy chained for TDM 16 operation. For example SRC3_DAT_OP_O and SRC3_TDM_IP_O of DAI0 are available as SRC3_CRS_DAT_OP_O and SRC3_CRS_IP_O in DAI1 so that DAI0 SRC3 can be connected to DAI1's SRC for daisy chaining.

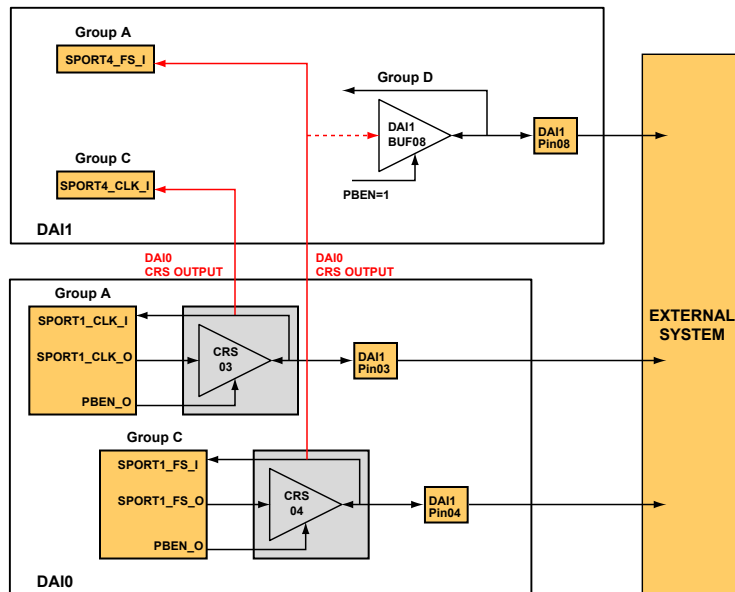


Figure 25-7: Cross Connect Clock Example

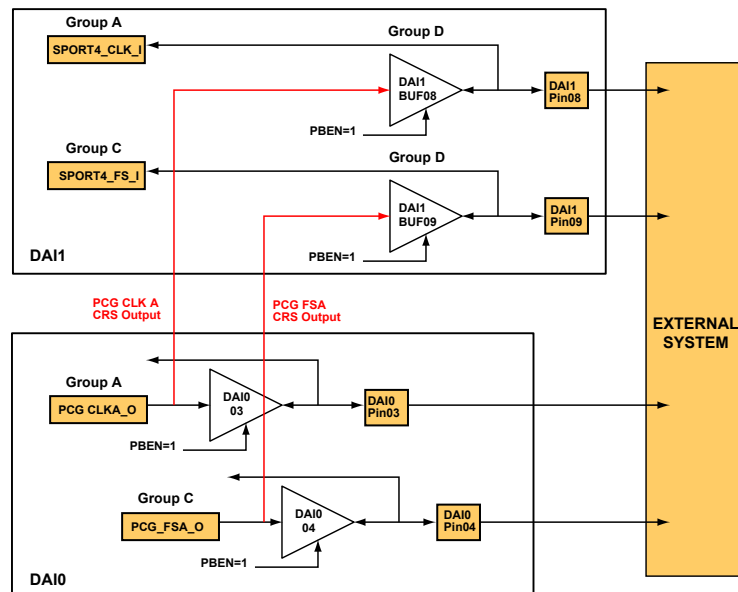


Figure 25-8: Cross Connect PCG Example

Refer to the [DAI Group Routing](#) section for more details on DAI routing.

Signal Routing Matrix by Groups

The SRU is similar to a set of patch bays, which contain a bank of inputs and a bank of outputs. For each input (destination), there is a set of permissible output (source) options. Outputs can feed to any number of inputs in parallel, but every input must be patched to exactly one valid output source. Together, the set of inputs and outputs are called a group. The signal's inputs and outputs that comprise each group all serve similar purposes. They are

compatible such that almost any output-to-input patch makes functional sense. With the grouping, the multiplexing scheme becomes highly efficient since it does not make sense (for example) to route a frame sync signal to a data signal.

The SRU for the DAI contains six groups named A through F. Each group routes a unique set of signals with a specific purpose:

- Group A routes clock signals
- Group B routes serial data signals
- Group C routes frame sync signals
- Group D routes pin signals
- Group E routes miscellaneous signals
- Group F routes pin output enable signals

Together, the six groups of the SRU include all of the inputs and outputs of the DAI peripherals, a number of additional signals from the core, and all of the connections to the DAI pins.

NOTE: It is not possible to connect a signal in one group directly to a signal in a different group (analogous to wiring from one patch bay to another). However, group D (DAI) is largely devoted to routing in this vein.

DAI Group Routing

Each group has a unique encoding for its associated output signals and a set of configuration registers. For example, DAI group A is used to route clock signals. The memory-mapped group A registers, `DAI_CLK0` through `DAI_CLK5`, contain bit fields corresponding to the clock inputs of various peripherals. The values written to these bit fields specify a signal source that is an output from another peripheral. All of the possible encodings represent sources that are clock signals (or at least could be clock signals in some systems). The *Example DAI Group A Multiplexing (DAI_CLKx)* diagrams the input signals that are controlled by the group A registers. All bit fields in the SRU configuration registers correspond to inputs. The value written to the bit field specifies the signal source. This value is also an output from some other component within the SRU.

The SRU is similar to a set of patch bays. Each bay routes a distinct set of outputs to compatible inputs. These connections are implemented as a set of memory-mapped registers with a bit field for each input. The outputs are implemented as a set of bit encodings. Conceptually, a patch cord is used to connect an output to an input. In the SRU, a bit pattern that is associated with a signal output (shown in the *Example DAI Group A Multiplexing (DAI_CLKx)* figure) is written to a bit field corresponding to a signal input.

The same encoding can be written to any number of bit fields in the same group. It is not possible to run out of patch points for an output signal.

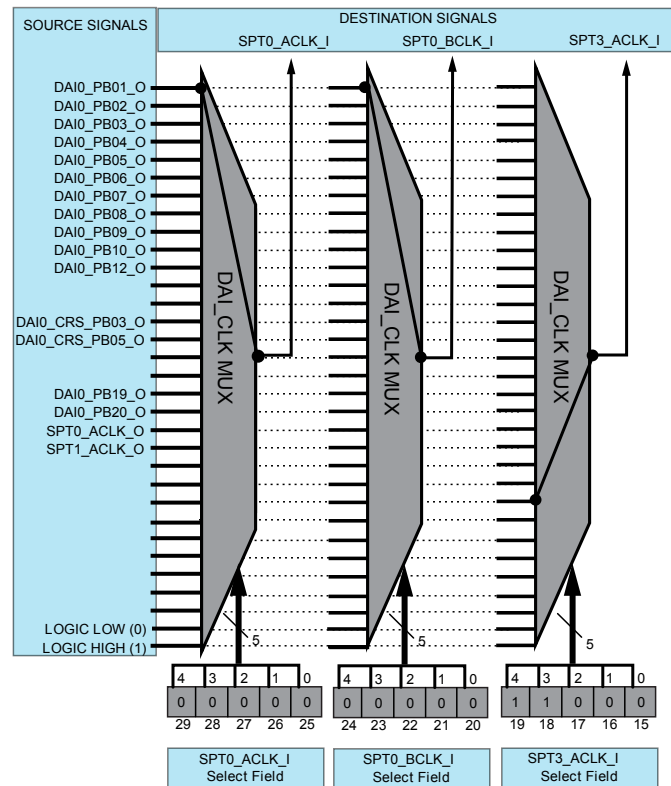


Figure 25-9: Example DAI Group A Multiplexing (DAI_CLKx)

Just as group A routes clock signals, each of the other groups route a collection of compatible signals. Group B routes serial data streams while group C routes frame sync signals. All of the groups have an encoding that allow a signal to flow from a pin output to the input being specified by the bit field.

Group D routes signals to pins so that they may be driven off-chip (required to route a signal to the pin input). Group F routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input. The input of one pin can be patched to the output of another pin, allowing board-level routing under software control.

Rules for SRU Connections

There are two rules which apply to all routing:

1. One source (output node) can drive different destinations (input nodes).
2. One destination (input node) can only be assigned to one source (output node).

As an example from the *Example DAI Group A Multiplexing (DAI_CLKx)* figure:

- DAI0_PB02_O is routed to SPT0_ACLK_I
- DAI0_PIN02_O is routed to SPT0_BCLK_I
- SPT2_ACLK_O is routed to SPT3_ACLK_I

NOTE: Inputs may only be connected to outputs.

Miscellaneous Buffers and Functions

The SRU group E provides miscellaneous buffers used for group interconnect.

DAI group E connections are slightly different from the others in that the inputs and outputs being routed vary considerably in function. This group routes control signals and provides a means of connecting signals between groups.

In the *DAI MISCAx SRU Signal Connections* table, the `DAIn_MISCAx_I` signals appear as inputs in group E (also connected to the DAI interrupt logic), but do not directly feed any peripheral. Rather, the `MISCAx_O` signals reappear as outputs in group F.

Table 25-5: DAI MISCAx SRU Signal Connections

MISCA Source	DAI Connection	MISCA Destination
	Group E	DAIn_MISCA5-0_I
DAIn_MISCA5-0_O	Group F	

Additional connections among groups provide a great amount of utility. Since the output groups F (DAI) dictate pin direction, these few signal paths enable a number of possible uses and connections for the DAI pins. Other examples include:

- A pin input can be patched to another pin's enable, allowing an off-chip signal to gate an output from the processor.
- Any of the DAI pins can be used as interrupt sources or general-purpose I/O (GPIO) signals.

In summary, the SRU enables many possible functional changes, both within the processor as well as externally. Used creatively, it allows system designers to radically change functionality at run time, and to potentially reuse circuit boards across many products.

DAI Routing Unit (DRU)

The DAI routing unit provides all possible connections across DAIs by interconnecting two DAI units through a multiplexer.

- For every input signal of an SRU, there is a control bit that selects either the original signal from the native DAI or its corresponding signal from the other DAI. For example, either `SP0_ACLK_O` from DAI0 or `SP0_ACLK_O` from DAI1 is selected and routed to the `SP0_ACLK_O` input of the SRU of DAI0 based on the programmed control bit.
- By default, all native signals are connected to their respective SRU (for example, `SP0_ACLK_O` from DAI0 is connected to `SP0_ACLK_O` input of the SRU of DAI0).

As a part of fully cross-connected DAI Routing unit, following source signals are added to the existing SRU's source group.

- All the source signals of Group A/B/C/D/E/F from the other DAI are added as source signals to the native group A/B/C/D/E/F's respectively.

Figure 1: DAI Routing Unit depicts the block level view of the DAI Routing Unit connecting DAI0 and DAI1 through a multiplexer.

Figure 25-10: DAI Routing Unit System - TBD

DAI Routing Capabilities

This section describes the routing options to aid in designing a system using the DAI units. The [DAI Default Routing](#) section provides diagrams that show how that DAIs connect at default. The DAI Group tables provide the source signals and selections codes. Finally, the xxx provides information about configuring destinations.

The *DAI Routing Capabilities* tables provide an overview of the different routing capabilities for the DAI unit. For information on an individual peripherals routing, see the “SRU Programming” section of the specific peripheral chapter.

Table 25-6: DAI0 Routing Capabilities

Source Signals - Output (xxxx_O)		DAI0 Group	Destination Signals - Input (xxxx_I)
SPORT2B-0A (clocks) PCG A, B S/PDIF0 Rx (clock, TDM clock) SPORT3B-3A, PCG E, F PCG A, B, E, F (Inverted) PDM0 All Group A source signals from other DAI	DAI Pin Buffer20-1 Logic level high Logic level low	A – clocks	SPORT3B-0A (clocks) SRC3-0 (clocks) PCG A-B ext. clock, ext. sync S/PDIF0 Tx (clock, HF clock) PCG E, F PDM0 (clk, bclk)
SPORT3B-0A (data) SRC3-0 (data, TDM data) S/PDIF0 Tx/Rx (data) PDM0 All Group B source signals from other DAI	DAI0 pin buffer 20-1 Logic level high Logic level low	B – data	SPORT3B-0A (data) SRC3-0 (data, TDM data) S/PDIF0 Tx/Rx (data) PDM0 (data0/1)
SPORT2B-0A (FS) PCG A, B (FS) S/PDIF0 Rx (FS) SPORT3A-3B, PCG E, F PCG A, B, E, F (Inverted) All Group C source signals from other DAI	DAI0 pin buffer 20-1 Logic level high Logic level low	C – frame sync	SPORT3B-0A (FS) SRC3-0 (FS) PDM0 (lrlk) TMR_ACI14

Table 25-6: DAI0 Routing Capabilities (Continued)

Source Signals - Output (xxxx_O)		DAI0 Group	Destination Signals - Input (xxxx_I)
SPORT3B-0A (clock, FS, TDV, data) PCG A,B (clock, FS) S/PDIF0 Rx (clock, TDM clock, FS, data) S/PDIF0 Tx (data, block start) PCG E, F (clock, FS) PCG A, B, E, F (inverted-Clock, FS) PDM0 (clock, FS) All Group D source signals from other DAI	DAI0 pin buffer 20-1 Logic level high Logic level low	D – pin buffer inputs	DAI0 pin buffer 12-1 and 20-19 Options: DAI0 pin buffer 20-19 Polarity change
SPORT2B-0A (FS) PCG A (clock) PCG B (clock, FS) S/PDIF0 Tx (block start) SPORT3B-3A (FS) PCG E, F (FS) PCG A, B, E, F (Slave Trigger) All Group E source signals from other DAI	DAI0 pin buffer 20-1 Logic level high Logic level low	E – miscellaneous signals	<i>DAI Interrupt 31-22</i> MISCA5-0 Options: MISCA5- Polarity change PCG A, B, E, F (Master Trigger)
SPORT3B-0A (clock OE, FS OE, data OE, TDV) MISCA5-0 PDM0 (clock OE, FS OE) All Group F source signals from other DAI	Logic level high Logic level low	F – pin buffer enable	DAI Pin Buffer Enable 20-1

Table 25-7: DAI1 Routing Capabilities

Source Signals - Output (xxxx_O)		DAI1 Group	Destination Signals - Input (xxxx_I)
SPORT6B-4A (clocks) PCG C, D (clocks) S/PDIF1 Rx (clock, TDM clock) SPORT7B-7A, PCG G, H PCG C, D, G, H (Inverted) PDM1 All Group A source signals from other DA	DAI1 Pin Buffer 20-1 Logic level high Logic level low	A – clocks	SPORT7B-4A (clocks) SRC4-7 (clocks) PCG C-D Ext. clock, Ext. sync S/PDIF1 Tx (clock, HF clock) PCG G, H PDM1 (clk, bclk)

Table 25-7: DAI1 Routing Capabilities (Continued)

Source Signals - Output (xxxx_O)		DAI1 Group	Destination Signals - Input (xxxx_I)
SPORT7B-4A (data A, B) SRC4-7 (data, TDM data) S/PDIF1 Tx/Rx (data) PDM1 All Group B source signals from other DA	DAI1 Pin Buffer 20-1 Logic level high Logic level low	B – data	SPORT7B-4A (data) SRC4-7 (data, TDM data) S/PDIF1 Tx/Rx (data) PDM1 (data0/1)
SPORT6B-4A (FS) PCG C, D (FS) S/PDIF1 Rx (FS) SPORT7B-7A, PCG G, H PCG C, D, G, H (Inverted) All Group C source signals from other DA	DAI1 Pin Buffer 20-1 Logic level high Logic level low	C – frame sync	SPORT7B-4A (FS) SRC4-7 (FS) PDM1 (lrclk) TMR_ACI14
SPORT7B-4A (clock, FS, TDV, data) PCG C, D (clock, FS) S/PDIF1 Rx (clock, TDM clock, FS, data) S/PDIF1 Tx (data, block start) PCG G, H (clock, FS) PCG C, D, G, H (inverted-Clock, FS) PDM1 (clock, FS) All Group D source signals from other DAI	DAI1 Pin Buffer 20-1 Logic level high Logic level low	D – pin buffer inputs	DAI1 Pin Buffer 20-1 Options: DAI1 Pin Buffer 20-19 Polarity Change
SPORT6B-4A (FS) PCG C (clock) PCG D (clock, FS) S/PDIF1 Tx (block start) SPORT7B-7A (FS) PCG C, D (FS) PCG C, D, G, H (Slave Trigger) All Group E source signals from other DAI	DAI1 Pin Buffer 20-1 Logic level high Logic level low	E – miscellaneous signals	<i>DAI Interrupt 31-22</i> MISCA5-0 Options: MISCA5-4 Polarity Change PCG C, D, G, H (Master Trigger)
SPORT7B-4A (clock, FS, data, TDV) PDM1 (clock OE, FS OE) All Group F source signals from other DAI.	Logic level high Logic level low	F – pin buffer enable	DAI1 Pin Buffer Enable 20-1

DAI Default Routing

When the processor comes out of reset, the SPORT junctions are bidirectional to the DAI pin buffers. This configuration allows systems to use the SPORTs as either master or slave (without changing the routing scheme). Therefore, programs only need to use the SPORT control register settings to configure master or slave operations. Note that all DAI inputs which are not routed by default are tied to signal low.

NOTE: All DAI input buffers which are not routed by default are driven low and all DAI pin buffer enable signals are driven low.

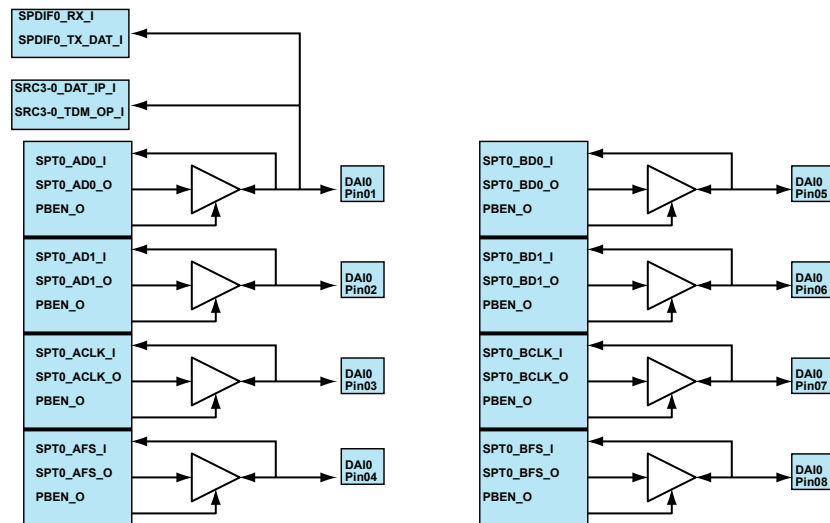


Figure 25-11: DAI0 Default Routing Pins 01-08

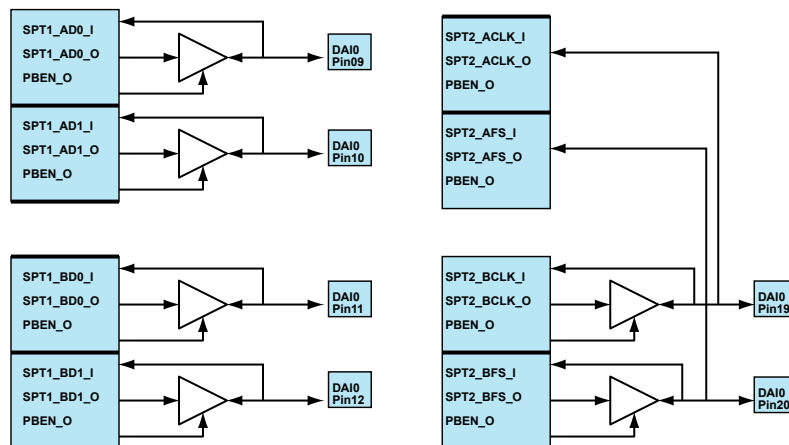


Figure 25-12: DAI0 Default Routing Pins 09-20

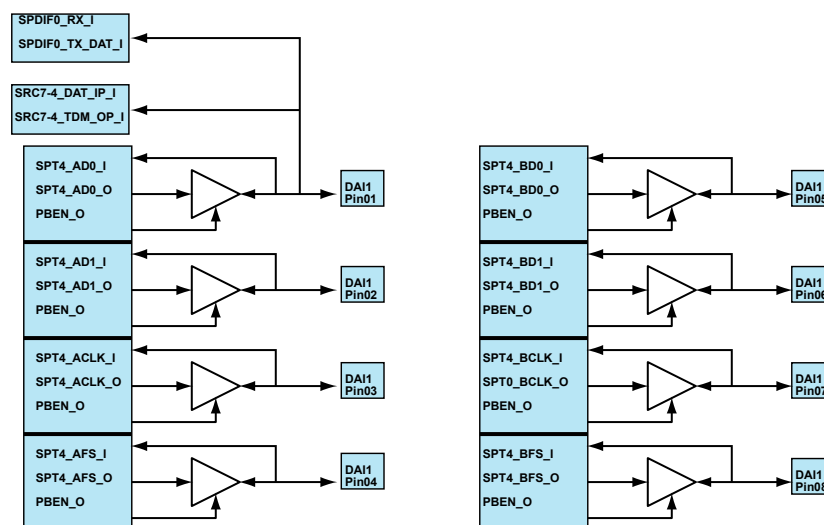


Figure 25-13: DAI1 Default Routing Pins 01-08

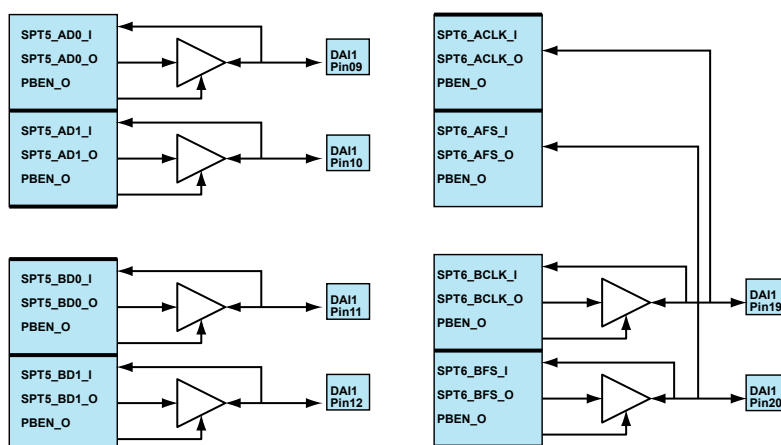


Figure 25-14: DAI1 Default Routing Pins 09-20

Unused DAI Connections

The SRUs have a default general-purpose routing scheme which can be modified to accommodate any number of different system designs. Regardless of the system design, it is good practice to tie all unused inputs to a high or low level to reduce dynamic power consumption.

DAI Operating Modes

Some buffers allow polarity changes, described as follows.

DAI Pin Buffer Polarity

As shown in the *Pin Buffer Polarity* figure, the DAI pin buffer 20-19 can change the polarity of the input signal if the corresponding control bits (DAI_PIN4 . INV20, DAI_PIN4 . INV19) are set. These bits can be set during run time and the buffer should not loopback to itself.

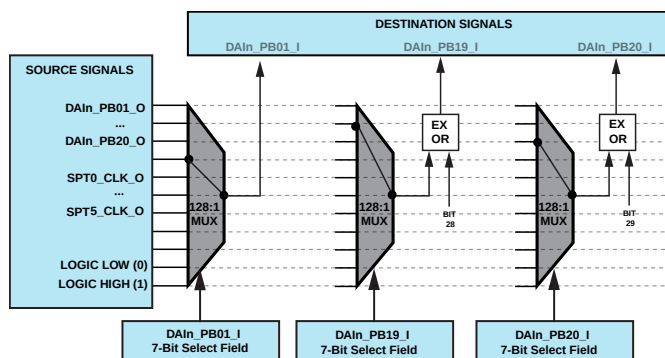


Figure 25-15: Pin Buffer Polarity

DAI Miscellaneous Buffer Polarity

As shown in the *Pin Buffer Polarity* figure, the A5-4 miscellaneous buffers can change the polarity of the input signal if the corresponding control bits (DAI_MISC1 . IN5, DAI_MISC1 . IN4) are set. Both buffers are not connected to the DAI interrupt latch register. These bits can be set during run time.

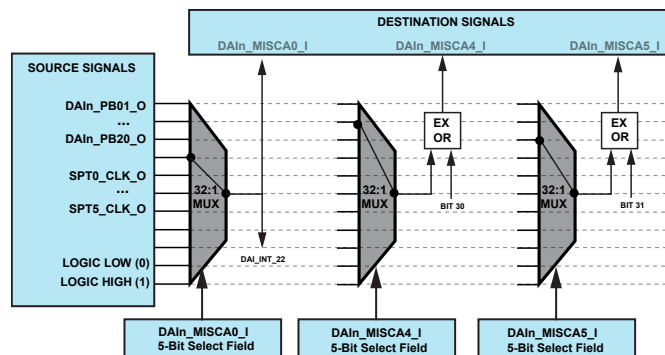


Figure 25-16: Miscellaneous Buffer Polarity

DAI System Interrupt Controller (SIC)

The DAI module incorporates a system interrupt controller (SIC) which is connected to the SEC as seen in the *DAI System Interrupt Controller* figure.

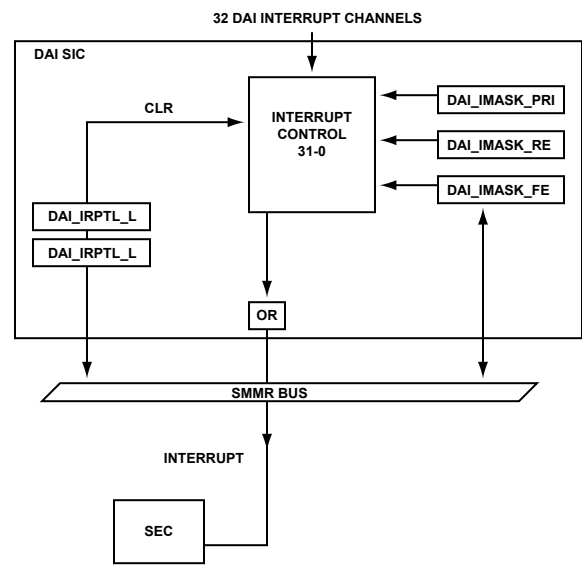


Figure 25-17: DAI System Interrupt Controller

The DAI has its own system interrupt controllers that indicate to the core when DAI audio peripheral-related events have occurred. Since audio events generally occur infrequently relative to the SHARC core, the DAI interrupt controller reduces all of its interrupts onto two interrupt signals within the core's primary interrupt systems. One interrupt is mapped with DAI low priority. The second interrupt is mapped with DAI high priority. This configuration allows programs to broadly indicate priority. In this way, the DAI SIC provides 32 independently configurable sources or channels. The output bus interrupt signals are logically OR'ed into one interrupt line and fed to the interrupt controller logic of the core.

Three registers are used to configure the DAI interrupt controller. Each of the 32 interrupt sources can be independently configured to trigger on a rising edge, falling edge, both edges, or neither edge of an incoming signal. All DAI interrupt control registers are memory-mapped registers and are accessed through the peripheral bus.

Interrupt Sources

The DAI's five peripheral sources are multiplexed into 32 interrupt sources and are labeled DAI_INT31-0 (*DAI Interrupt Sources* table).

NOTE: There are two naming conventions. The DAI interrupt controller register bits are labeled DAI_31-0_INT. Their corresponding SRU routing signals are labeled DAI_INT_31-0_I.

Table 25-8: DAI Interrupt Sources

Interrupt Source	Description	Signal Response
DAI_INT2-0, DAI_INT4	S/PDIF RX, 4 channels	Event
DAI_INT2-0, DAI_INT4	S/PDIF RX, 4 channels	Waveform
DAI_INT21-18	ASRC, 4 channels	
DAI_INT31-22	Miscellaneous, S/PDIF TX, 9 channels	

Interrupt Latch Priority Option

The `DAI_IMSK_PRI` register specifies the priority for the DAI interrupt channels. DAI system interrupt controller has a pair of interrupt latch registers, `DAI_IRPTL_H` and `DAI_IRPTL_L`. The configuration of the `DAI_IMSK_PRI` register also determines the interrupt latch mapping for a particular DAI interrupt. When a DAI interrupt is configured as low priority (`DAI_IMSK_PRI` bit cleared, default setting), interrupts are mapped to the `DAI_INTR_IRQL` signal and when an interrupt occurs, the corresponding bit of the `DAI_IRPTL_L` register is set. When a DAI interrupt is configured as high priority (`DAI_IMSK_PRI` bit set), interrupts are mapped to the `DAI_INTR_IRQH` signal and the interrupt is latched to the `DAI_IRPTL_H` register. The low priority DAI interrupt (`INTR_DAI_IRQL`) and high priority DAI interrupt (`INTR_DAI_IRQH`) are connected to the SEC.

Interrupt Mask for Waveforms

The `DAI_IMSK_RE` and `DAI_IMSK_FE` registers allow programs to mask or unmask interrupts for specific edges of a signal mapped to the channel. It can be configured for rising edges, falling edges, both rising and falling edges, or neither rising nor falling edges by masking them separately. Signals from the SRU can be used to generate interrupts. For example, when the `DAI_IMSK_FE.MISCINT9` bit is set to one, any falling edge signals from the external channel generate an interrupt and the interrupt latch is set.

Interrupt Mask for Events

The system interrupt controller needs information about a peripheral's interrupt sources that correspond to event signals (see the *DAI Interrupt Sources* table). As a result, the rising edge is used as an interrupt source only. For DAI peripherals marked as events, programs may unmask an interrupt source on the rising edge only.

Shadow Interrupt Register

The DAI interrupt controller has shadow registers to simplify debug activities since these registers do not manipulate status control. Any read of the shadow registers (`DAI_IRPTL_HS`, `DAI_IRPTL_LS`), provides the same data as a read of the `DAI_IRPTL_H` and `DAI_IRPTL_L` registers. However, reads of the DAI shadow registers do not change the interrupt acknowledge status to the core interrupt controller.

Interrupt Service

The interrupt acknowledge operates differently when multiple channels are multiplexed into one interrupt output signal. When an interrupt from the DAI must be serviced, any of the two interrupt service routines (`INTR_DAI_IRQL`, `INTR_DAI_IRQH`) must query the SIC to determine the source(s). Sources can be any one or more of the DAI channels (`DAI_INT31-0`).

- When the `DAI_IRPTL_H` register is read, the high priority latched interrupts are cleared.
- When the `DAI_IRPTL_L` register is read, the low priority latched interrupts are cleared.

If an interrupt occurs in the same cycle as a latch register is cleared, the clear mechanism has lower priority and the new interrupt is registered.

Signal Routing Unit Effect Latency

After the DAIy registers are configured the effect latency is 2 *SCLK0* cycles minimum and 3 *SCLK0* cycles maximum.

DAI Programming Model

As discussed in the previous sections, the signal routing unit is controlled by writing values that provide a plug-in tool in CCES so that configuring the SRU is done graphically. Analog Devices offers macros that are included with the CrossCore or VisualDSP++ tools, greatly easing code development in the SRU.

There is a macro that has been created to connect peripherals used in a DAI configuration. This code can be used in both assembly and C code. See the INCLUDE file SRU.H. In practice the macro is provided and forms the style `SRU(source_O, destination_I)` for DAI0. Example appears as: `SRU(DAI0_PB12_O, SPT0_ACLK_I);`

Debug Features

The following section describes the feature that can be used to help in debugging the DAI.

Loopback Routing

The SPORT serial peripheral supports an internal loopback mode. If the loopback bit for each peripheral is enabled, it connects the transmitter with the receiver block internally (does not signal off-chip). The SRU can be used for this purpose. The *Loopback Routing* table describes the different possible routings based on the peripheral.

NOTE: The peripheral's loopback mode for debug is independent from both of the signal routing units.

Table 25-9: Loopback Routing

Peripheral	Loopback Mode	SRU1-0 Internal Routing for Loopback	SRU1-0 External Routing for Loopback
SPORT	Yes	SPTx_xx_O → SPTx_xx_I	SPTx_xx_O → DAIn_PBxx_I DAIn_PBxx_O → SPTx_xx_I
S/PDIF Tx/Rx	No	SPDIFn_TX_O → SPDIFn_RX_I	SPDIFn_TX_O → DAI_PBxx_I DAI_PBxx_O → SPDIFn_RX_I
SRC	No	SRCx_DAT_OP_O → SRCx_DAT_IP_I	SRCx_DAT_OP_O → DAI_PBxx_I DAI_PBxx_O → SRCx_DAT_IP_I

DAI Sources Overview

The following tables provide information of the various DAI sources sorted by group. The shaded cells in the tables denote connection to the other DAI.

DAI0 Group A – Clock Routing Signals

The group A clock routing sources are listed in the following table.

Table 25-10: DAI0 Group A – Clock Routing Sources

Extended selection code [1:0]		Selection code [4:0]	Source Signal	Description (Source Selection)
Bit 1	Bit 0			
0	0	00000 (0x0)	DAI0_PB01_O	Pin Buffer 1
0	0	00001 (0x1)	DAI0_PB02_O	Pin Buffer 2
0	0	00010 (0x2)	DAI0_PB03_O	Pin Buffer 3
0	0	00011 (0x3)	DAI0_PB04_O	Pin Buffer 4
0	0	00100 (0x4)	DAI0_PB05_O	Pin Buffer 5
0	0	00101 (0x5)	DAI0_PB06_O	Pin Buffer 6
0	0	00110 (0x6)	DAI0_PB07_O	Pin Buffer 7
0	0	00111 (0x7)	DAI0_PB08_O	Pin Buffer 8
0	0	01000 (0x8)	DAI0_PB09_O	Pin Buffer 9
0	0	01001 (0x9)	DAI0_PB10_O	Pin Buffer 10
0	0	01010 (0xA)	DAI0_PB11_O	Pin Buffer 11
0	0	01011 (0xB)	DAI0_PB12_O	Pin Buffer 12
0	0	01100 (0xC)	DAI0_PB13_O	Pin Buffer 13
0	0	01101 (0xD)	DAI0_PB14_O	Pin Buffer 14
0	0	01110 (0xE)	DAI0_PB15_O	Pin Buffer 15
0	0	01111 (0xF)	DAI0_PB16_O	Pin Buffer 16
0	0	10000 (0x10)	DAI0_PB17_O	Pin Buffer 17
0	0	10001 (0x11)	DAI0_PB18_O	Pin Buffer 18
0	0	10010 (0x12)	DAI0_PB19_O	Pin Buffer 19
0	0	10011 (0x13)	DAI0_PB20_O	Pin Buffer 20
0	0	10100 (0x14)	SPT0_ACLK_O	SPORT 0 Clock A
0	0	10101 (0x15)	SPT0_BCLK_O	SPORT 0 Clock B
0	0	10110 (0x16)	SPT1_ACLK_O	SPORT 1 Clock A
0	0	10111 (0x17)	SPT1_BCLK_O	SPORT 1 Clock B
0	0	11000 (0x18)	SPT2_ACLK_O	SPORT 2 Clock A
0	0	11001 (0x19)	SPT2_BCLK_O	SPORT 2 Clock B
0	0	11010 (0x1A)	SPDIF0_RX_CLK_O	SPDIF 0 Receive Clock Output

Table 25-10: DAI0 Group A – Clock Routing Sources (Continued)

Extended selection code [1:0]		Selection code [4:0]	Source Signal	Description (Source Selection)
Bit 1	Bit 0			
0	0	11011 (0x1B)	SPDIF0_RX_TDMCLK_O	SPDIF 0 Receive TDM ClockOutput
0	0	11100 (0x1C)	PCG_CLKA_O	Precision Clock A Output
0	0	11101 (0x1D)	PCG_CLKB_O	Precision Clock B Output
0	0	11110 (0x1E)	LOW	Logic Level Low (0)
0	0	11111 (0x1F)	HIGH	Logic Level High (1)
0	1	00000 (0x0)	SPT3_ACLK_O	SPORT 3 clock A
0	1	00001 (0x1)	SPT3_BCLK_O	SPORT 3 clock B
0	1	00010 (0x2)	PCG_CLKE_O	Precision Clock E output
0	1	00011 (0x3)	PCG_CLKF_O	Precision Clock F output
0	1	00100 (0x4)	PCG_CLKA_INV_O	Inverted Precision Clock A output
0	1	00101 (0x5)	PCG_CLKB_INV_O	Inverted Precision Clock B output
0	1	00110 (0x6)	PCG_CLKE_INV_O	Inverted Precision Clock E output
0	1	00111 (0x7)	PCG_CLKF_INV_O	Inverted Precision Clock F output
0	1	01000 (0x8)	PDM0_CLK0_O	PDM0 Clock0 output
0	1	01001 (0x9) -11111 (0x1F)	Reserved	
1	0	00000 (0x0)	DAI1_PB01_O	Pin Buffer 1 (from other DAI)
1	0	00001(0x1)	DAI1_PB02_O	Pin Buffer 2 (from other DAI)
1	0	00010 (0x2)	DAI1_PB03_O	Pin Buffer 3 (from other DAI). This pin buffer is used for sharing clocks between DAI modules. DAI0_PIN03 and DAI1_PIN03 can be used to share clock signal across DAI. When DAI0_PIN03 is configured as input clock pin then this clock signal is available in Group A of both DAIs. This source signal is DAI0_CRS_PB03_O in group A of DAI 0 and DAI1_CRS_PB03_O in group A of DAI1.
1	0	00011 (0x3)	DAI1_PB04_O	Pin Buffer 4 (from other DAI)

Table 25-10: DAI0 Group A – Clock Routing Sources (Continued)

Extended selection code [1:0]		Selection code [4:0]	Source Signal	Description (Source Selection)
Bit 1	Bit 0			
1	0	00100 (0x4)	DAI1_PB05_O	Pin Buffer 5 (from other DAI). This pin buffer is used for sharing clocks between DAI modules. DAI0_PIN05 and DAI1_PIN05 can be used to share clock signal across DAI. When DAI0_PIN05 is configured as input clock pin then this clock signal is available in Group A of both DAIs. This source signal is DAI0_CRS_PB05_O in group A of DAI 0 and DAI1_CRS_PB05_O in group A of DAI1.
1	0	00101 (0x5)	DAI1_PB06_O	Pin Buffer 6 (from other DAI)
1	0	00110 (0x6)	DAI1_PB07_O	Pin Buffer 7 (from other DAI)
1	0	00111 (0x7)	DAI1_PB08_O	Pin Buffer 8 (from other DAI)
1	0	01000 (0x8)	DAI1_PB09_O	Pin Buffer 9 (from other DAI)
1	0	01001 (0x9)	DAI1_PB10_O	Pin Buffer 10 (from other DAI)
1	0	01010 (0xA)	DAI1_PB11_O	Pin Buffer 11 (from other DAI)
1	0	01011 (0xB)	DAI1_PB12_O	Pin Buffer 12 (from other DAI)
1	0	01100 (0xC)	DAI1_PB13_O	Pin Buffer 13 (from other DAI)
1	0	01101 (0xD)	DAI1_PB14_O	Pin Buffer 14 (from other DAI)
1	0	01110 (0xE)	DAI1_PB15_O	Pin Buffer 15 (from other DAI)
1	0	01111 (0xF)	DAI1_PB16_O	Pin Buffer 16 (from other DAI)
1	0	10000 (0x10)	DAI1_PB17_O	Pin Buffer 17 (from other DAI)
1	0	10001 (0x11)	DAI1_PB18_O	Pin Buffer 18 (from other DAI)
1	0	10010 (0x12)	DAI1_PB19_O	Pin Buffer 19 (from other DAI)
1	0	10011 (0x13)	DAI1_PB20_O	Pin Buffer 20 (from other DAI)
1	0	10100 (0x14)	SPT4_ACLK_O	SPORT 4 Clock A (from other DAI)
1	0	10101 (0x15)	SPT4_BCLK_O	SPORT 4 Clock B (from other DAI)
1	0	10110 (0x16)	SPT5_ACLK_O	SPORT 5 Clock A (from other DAI)
1	0	10111 (0x17)	SPT5_BCLK_O	SPORT 5 Clock B (from other DAI)
1	0	11000 (0x18)	SPT6_ACLK_O	SPORT 6 Clock A (from other DAI)
1	0	11001 (0x19)	SPT6_BCLK_O	SPORT 6 Clock B (from other DAI)
1	0	11010 (0x1A)	SPDIF1_RX_CLK_O	SPDIF 0 Receive Clock Output (from other DAI)

Table 25-10: DAI0 Group A – Clock Routing Sources (Continued)

Extended selection code [1:0]		Selection code [4:0]	Source Signal	Description (Source Selection)
Bit 1	Bit 0			
1	0	11011 (0x1B)	SPDIF1_RX_TDMCLK_O	SPDIF 0 Receive TDM Clock Output (from other DAI)
1	0	11100 (0x1C)	PCG_CLKC_O	Precision Clock C Output (from other DAI)
1	0	11101 (0x1D)	PCG_CLKD_O	Precision Clock D Output (from other DAI)
1	0	11110 (0x1E)	Reserved	
1	0	11111 (0x1F)	Reserved	
1	1	00000 (0x0)	SPT7_ACLK_O	SPORT 3 clock A (from other DAI)
1	1	00001 (0x1)	SPT7_BCLK_O	SPORT 3 clock B (from other DAI)
1	1	00010 (0x2)	PCG_CLKG_O	Precision Clock G output (from other DAI)
1	1	00011 (0x3)	PCG_CLKH_O	Precision Clock H output (from other DAI)
1	1	00100 (0x4)	PCG_CLKC_INV_0	Inverted Precision Clock C output (from other DAI)
1	1	00101 (0x5)	PCG_CLKD_INV_0	Inverted Precision Clock D output (from other DAI)
1	1	00110 (0x6)	PCG_CLKG_INV_0	Inverted Precision Clock G output (from other DAI)
1	1	00111 (0x7)	PCG_CLKH_INV_0	Inverted Precision Clock H output (from other DAI)
1	1	01000 (0x8)	PDM1_CLK0_O	PDM1 Clock0 Output (from other DAI)
1	1	01001 (0x9) -11111 (0x1F)	Reserved	

DAI0 Group B – Serial Data Source Signals

The group B data sources are listed in the following table. The group B data destinations are configured using [DAI_PBEN0](#) through [DAI_PBEN3](#).

Table 25-11: DAI0 Group B – Serial Data Signals

Selection Code	Source Signal	Description (Source Selection)
000000 (0x0)	DAI0_PB01_O	Pin Buffer 1
000001 (0x1)	DAI0_PB02_O	Pin Buffer 2
000010 (0x2)	DAI0_PB03_O	Pin Buffer 3
000011 (0x3)	DAI0_PB04_O	Pin Buffer 4
000100 (0x4)	DAI0_PB05_O	Pin Buffer 5
000101 (0x5)	DAI0_PB06_O	Pin Buffer 6
000110 (0x6)	DAI0_PB07_O	Pin Buffer 7
000111 (0x7)	DAI0_PB08_O	Pin Buffer 8
001000 (0x8)	DAI0_PB09_O	Pin Buffer 9
001001 (0x9)	DAI0_PB10_O	Pin Buffer 10
001010 (0xA)	DAI0_PB11_O	Pin Buffer 11
001011 (0xB)	DAI0_PB12_O	Pin Buffer 12
001100 (0xC)	Reserved	
001101 (0xD)	Reserved	
001110 (0xE)	Reserved	
001111 (0xF)	Reserved	
010000 (0x10)	Reserved	
010001 (0x11)	Reserved	
010010 (0x12)	DAI0_PB19_O	Pin Buffer 19
010011 (0x13)	DAI0_PB20_O	Pin Buffer 20
010100 (0x14)	SPT0_AD0_O	SPORT 0 Data AD0
010101 (0x15)	SPT0_AD1_O	SPORT 0 Data AD1
010110 (0x16)	SPT0_BD0_O	SPORT 0 Data BD0
010111 (0x17)	SPT0_BD1_O	SPORT 0 Data BD1
011000 (0x18)	SPT1_AD0_O	SPORT 1 Data AD0
011001 (0x19)	SPT1_AD1_O	SPORT 1 Data AD1
011010 (0x1A)	SPT1_BD0_O	SPORT 1 Data BD0
011011 (0x1B)	SPT1_BD1_O	SPORT 1 Data BD1

Table 25-11: DAI0 Group B – Serial Data Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
011100 (0x1C)	SPT2_AD0_O	SPORT 2 Data AD0
011101 (0x1D)	SPT2_AD1_O	SPORT 2 Data AD1
011110 (0x1E)	SPT2_BD0_O	SPORT 2 Data BD0
011111 (0x1F)	SPT2_BD1_O	SPORT 2 Data BD1
100000 (0x20)	SRC0_DAT_OP_O	SRC0 Data Out
100001 (0x21)	SRC1_DAT_OP_O	SRC1 Data Out
100010 (0x22)	SRC2_DAT_OP_O	SRC2 Data Out
100011 (0x23)	SRC3_DAT_OP_O	SRC3 Data Out
100100 (0x24)	SRC0_TDM_IP_O	SRC0 Data Out
100101 (0x25)	SRC1_TDM_IP_O	SRC1 Data Out
100110 (0x26)	SRC2_TDM_IP_O	SRC2 Data Out
100111 (0x27)	SRC3_TDM_IP_O	SRC3 Data Out
101000 (0x28)	SPDIF0_RX_DAT_O	SPDIF 0 RX Serial Data Out
101100 (0x2C)	SPT3_AD0_O	SPORT 3 Data AD0
101101 (0x2D)	SPT3_AD1_O	SPORT 3 Data AD1
101110 (0x2E)	SPT3_BD0_O	SPORT 3 Data BD0
101111 (0x2F)	SPT3_BD1_O	SPORT 3 Data BD1
110000 (0x30)	SPDIF0_TX_O	SPDIF 0 TX Biphase Stream
110001 (0x31)	SRC7_CRS_DAT_OP_O	SRC7 data out (DAI1)
110010 (0x32)	SRC7_CRS_TDM_IP_O	SRC7 data out (DAI1)
110011 (0x33)–111101 (0x3D)	Reserved	
111110 (0x3E)	LOW	Logic Level Low (0)
111111 (0x3F)	HIGH	Logic Level High (1)

DAI0 Group C – Frame Sync Source Signals

The group C frame sync signal sources are listed in the following table.

Table 25-12: DAI0 Group C – Frame Sync Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI0_PB01_O	Pin Buffer 1
00001 (0x1)	DAI0_PB02_O	Pin Buffer 2
00010 (0x2)	DAI0_PB03_O	Pin Buffer 3

Table 25-12: DAI0 Group C – Frame Sync Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
00011 (0x3)	DAI0_PB04_O	Pin Buffer 4
00100 (0x4)	DAI0_PB05_O	Pin Buffer 5
00101 (0x5)	DAI0_PB06_O	Pin Buffer 6
00110 (0x6)	DAI0_PB07_O	Pin Buffer 7
00111 (0x7)	DAI0_PB08_O	Pin Buffer 8
01000 (0x8)	DAI0_PB09_O	Pin Buffer 9
01001 (0x9)	DAI0_PB10_O	Pin Buffer 10
01010 (0xA)	DAI0_PB11_O	Pin Buffer 11
01011 (0xB)	DAI0_PB12_O	Pin Buffer 12
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	DAI0_CRS_PB04_O	Pin Buffer 4 (DAI1)
01111 (0xF)	DAI0_CRS_PB06_O	Pin Buffer 6 (DAI1)
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI0_PB19_O	Pin Buffer 19
10011 (0x13)	DAI0_PB20_O	Pin Buffer 20
10100 (0x14)	SPT0_AFS_O	SPORT 0 Frame Sync A
10101 (0x15)	SPT0_BFS_O	SPORT 0 Frame Sync B
10110 (0x16)	SPT1_AFS_O	SPORT 1 Frame Sync A
10111 (0x17)	SPT1_BFS_O	SPORT 1 Frame Sync B
11000 (0x18)	SPT2_AFS_O	SPORT 2 Frame Sync A
11001 (0x19)	SPT2_BFS_O	SPORT 2 Frame Sync B
11010 (0x1A)	SPDIF0_FS_O	SPDIF 0 RX Frame Sync Output
11011 (0x1B)	Reserved	
11100 (0x1C)	PCG0_FSA_O	Precision Frame Sync A Output
11101 (0x1D)	PCG0_FSB_O	Precision Frame Sync B Output
11110 (0x1E)	LOW	Logic Level Low (0)
11111 (0x1F)	HIGH	Logic Level High (1)

DAI0 Group D – Pin Signal Assignment Source Signals

The group D pin signal assignment sources are listed in the following table.

Table 25-13: DAI0 Group D – Pin Signal Assignments

Selection Code	Source Signal	Description (Source Selection)
0000000 (0x0)	DAI0_PB01_O	Pin Buffer 1
0000001 (0x1)	DAI0_PB02_O	Pin Buffer 2
0000010 (0x2)	DAI0_PB03_O	Pin Buffer 3
0000011 (0x3)	DAI0_PB04_O	Pin Buffer 4
0000100 (0x4)	DAI0_PB05_O	Pin Buffer 5
0000101 (0x5)	DAI0_PB06_O	Pin Buffer 6
0000110 (0x6)	DAI0_PB07_O	Pin Buffer 7
0000111 (0x7)	DAI0_PB08_O	Pin Buffer 8
0001000 (0x8)	DAI0_PB09_O	Pin Buffer 9
0001001 (0x9)	DAI0_PB10_O	Pin Buffer 10
0001010 (0xA)	DAI0_PB11_O	Pin Buffer 11
0001011 (0xB)	DAI0_PB12_O	Pin Buffer 12
0001100 (0xC)	Reserved	
0001101 (0xD)	Reserved	
0001110 (0xE)	Reserved	
0001111 (0xF)	Reserved	
0010000 (0x10)	Reserved	
0010001 (0x11)	Reserved	
0010010 (0x12)	DAI0_PB19_O	Pin Buffer 19
0010011 (0x13)	DAI0_PB20_O	Pin Buffer 20
0010100 (0x14)	SPT0_AD0_O	SPORT 0 Data AD0
0010101 (0x15)	SPT0_AD1_O	SPORT 0 Data AD1
0010110 (0x16)	SPT0_BD0_O	SPORT 0 Data BD0
0010111 (0x17)	SPT0_BD1_O	SPORT 0 Data BD1
0011000 (0x18)	SPT1_AD0_O	SPORT 1 Data AD0
0011001 (0x19)	SPT1_AD1_O	SPORT 1 Data AD1
0011010 (0x1A)	SPT1_BD0_O	SPORT 1 Data BD0
0011011 (0x1B)	SPT1_BD1_O	SPORT 1 Data BD1
0011100 (0x1C)	SPT2_AD0_O	SPORT 2 Data AD0
0011101 (0x1D)	SPT2_AD1_O	SPORT 2 Data AD1
0011110 (0x1E)	SPT2_BD0_O	SPORT 2 Data BD0

Table 25-13: DAI0 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
0011111 (0x1F)	SPT2_BD1_O	SPORT 2 Data BD1
0100000 (0x20)	SPT0_ACLK_O	SPORT 0 Clock A
0100001 (0x21)	SPT0_BCLK_O	SPORT 0 Clock B
0100010 (0x22)	SPT1_ACLK_O	SPORT 1 Clock A
0100011 (0x23)	SPT1_BCLK_O	SPORT 1 Clock B
0100100 (0x24)	SPT2_ACLK_O	SPORT 2 Clock A
0100101 (0x25)	SPT2_BCLK_O	SPORT 2 Clock B
0100110 (0x26)	SPT0_AFS_O	SPORT 0 Frame Sync A
0100111 (0x27)	SPT0_BFS_O	SPORT 0 Frame Sync B
0101000 (0x28)	SPT1_AFS_O	SPORT 1 Frame Sync A
0101001 (0x29)	SPT1_BFS_O	SPORT 1 Frame Sync B
0101010 (0x2A)	SPT2_AFS_O	SPORT 2 Frame Sync A
0101011 (0x2B)	SPT2_BFS_O	SPORT 2 Frame Sync B
0101100 (0x2C)	SPT3_AD0_O	SPORT 3 Data AD0
0101101 (0x2D)	SPT3_AD1_O	SPORT 3 Data AD1
0101110 (0x2E)	SPT3_BD0_O	SPORT 3 Data BD0
0101111 (0x2F)	SPT3_BD1_O	SPORT 3 Data BD1
0110000 (0x30)	MLB0_CLKOUT	MLB PLL clock output
0110001 (0x31)	SPDIF0_TX_BLKSTART_O	SPDIF 0 TX Block Start Output
0110100 (0x34)	SPT3_ACLK_O	SPORT 3 Clock A
0110101 (0x35)	SPT3_BCLK_O	SPORT 3 Clock B
0110110 (0x36)	SPT3_AFS_O	SPORT 3 Frame Sync A
0110111 (0x37)	SPT3_BFS_O	SPORT 3 Frame Sync B
0111000 (0x38)	PCG0_CLKA_O	Precision Clock A
0111001 (0x39)	PCG0_CLKB_O	Precision Clock B
0111010 (0x3A)	PCG0_FSA_O	Precision Frame Sync A
0111011 (0x3B)	PCG0_FSB_O	Precision Frame Sync B
0111100 (0x3C)	Reserved	
0111101 (0x3D)	SRC0_DAT_OP_O	SRC0 Data Output
0111110 (0x3E)	SRC1_DAT_OP_O	SRC1 Data Output
0111111 (0x3F)	SRC2_DAT_OP_O	SRC2 Data Output

Table 25-13: DAI0 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
1000000 (0x40)	SRC3_DAT_OP_O	SRC3 Data Output
1000001 (0x41)	SPDIF0_RX_DAT_O	SPDIF 0 RX Data Output
1000010 (0x42)	SPDIF0_FS_O	SPDIF 0 RX Frame Sync Output
1000011 (0x43)	SPDIF0_RXCLK_O	SPDIF 0 RX Clock Output
1000100 (0x44)	SPDIF0_RX_TDMCLK_O	SPDIF 0 RX TDM Clock Output
1000101 (0x45)	SPDIF0_TX_O	SPDIF 0 TX Biphase Encoded Data Output
1000110 (0x46)	SPT0_ATDV_O	SPORT0 Transmit A Data Valid Output
1000111 (0x47)	SPT0_BTDTV_O	SPORT0 Transmit B Data Valid Output
1001000 (0x48)	SPT1_ATDV_O	SPORT1 Transmit A Data Valid Output
1001001 (0x49)	SPT1_BTDTV_O	SPORT1 Transmit B Data Valid Output
1001010 (0x4A)	SPT2_ATDV_O	SPORT2 Transmit A Data Valid Output
1001011 (0x4B)	SPT2_BTDTV_O	SPORT2 Transmit B Data Valid Output
1001100 (0x4C)	SPT3_ATDV_O	SPORT3 Transmit A Data Valid Output
1001101 (0x4D)	SPT3_BTDTV_O	SPORT3 Transmit B Data Valid Output
1001110 (0x4E)	DAI0_CRS_PB05_O	Pin Buffer 5 (DAI1)
1001111 (0x4F)	DAI0_CRS_PB06_O	Pin Buffer 6 (DAI1)
1010000 (0x50)	PCG0_CRS_CLKC_O	Precision Clock C (DAI1)
1010001 (0x51)	PCG0_CRS_CLKD_O	Precision Clock D (DAI1)
1010010 (0x52)	PCG0_CRS_FSC_O	Precision Frame Sync C (DAI1)
1010011 (0x53)	PCG0_CRS_FSD_O	Precision Frame Sync D (DAI1)
1010100 (0x54)	DAI0_CRS_PB03_O	Pin Buffer 3 (DAI1)
1010101 (0x55)	DAI0_CRS_PB04_O	Pin Buffer 4 (DAI1)
1010110 – 1111101	Reserved	
1111110 (0x7E)	LOW	Logic Level Low (0)
1111111 (0x7F)	HIGH	Logic Level High (1)

DAI0 Group E – Miscellaneous Source Signals

The following table lists the group E miscellaneous signal sources.

Table 25-14: DAI0 Group E – Miscellaneous Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI0_PB01_O	Pin Buffer 1 Output
00001 (0x1)	DAI0_PB02_O	Pin Buffer 2 Output
00010 (0x2)	DAI0_PB03_O	Pin Buffer 3 Output
00011 (0x3)	DAI0_PB04_O	Pin Buffer 4 Output
00100 (0x4)	DAI0_PB05_O	Pin Buffer 5 Output
00101 (0x5)	DAI0_PB06_O	Pin Buffer 6 Output
00110 (0x6)	DAI0_PB07_O	Pin Buffer 7 Output
00111 (0x7)	DAI0_PB08_O	Pin Buffer 8 Output
01000 (0x8)	DAI0_PB09_O	Pin Buffer 9 Output
01001 (0x9)	DAI0_PB10_O	Pin Buffer 10 Output
01010 (0xA)	DAI0_PB11_O	Pin Buffer 11 Output
01011 (0xB)	DAI0_PB12_O	Pin Buffer 12 Output
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	Reserved	
01111 (0xF)	Reserved	
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI0_PB19_O	Pin Buffer 19 Output
10011 (0x13)	DAI0_PB20_O	Pin Buffer 20 Output
10100 (0x14)	SPT0_AFS_O	SPORT 0 Frame Sync A
10101 (0x15)	SPT0_BFS_O	SPORT 0 Frame Sync B
10110 (0x16)	SPT1_AFS_O	SPORT 1 Frame Sync A
10111 (0x17)	SPT1_BFS_O	SPORT 1 Frame Sync B
11000 (0x18)	SPT2_AFS_O	SPORT 2 Frame Sync A
11001 (0x19)	SPT2_BFS_O	SPORT 2 Frame Sync B
11010 (0x1A)	SPDIF0_TX_BLKSTART_O	SPDIF 0 TX Block Start Output
11011 (0x1B)	PCG0_FSA_O	Precision Frame Sync A
11100 (0x1C)	PCG0_CLKB_O	Precision Clock B
11101 (0x1D)	PCG0_FSB_O	Precision Frame Sync B
11110 (0x1E)	LOW	Logic Level Low (0) as a Source

Table 25-14: DAI0 Group E – Miscellaneous Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
11111 (0x1F)	HIGH	Logic Level High (1) as a Source

DAI0 Group F – Pin Output Enable Source Signals

The group F pin output enable signal sources are listed in the following table. The group F destinations are configured using [DAI_PBEN0](#) to [DAI_PBEN2](#) registers.

Table 25-15: DAI0 Group F – Pin Output Enable

Selection Code	Source Signal	Description (Source Selection)
000000 (0x0)	LOW	Logic Level Low (0)
000001 (0x1)	HIGH	Logic Level High (1)
000010 (0x2)	DAI0_MISCA0_O	DAI0 Miscellaneous Control A0 Output
000011 (0x3)	DAI0_MISCA1_O	DAI0 Miscellaneous Control A1 Output
000100 (0x4)	DAI0_MISCA2_O	DAI0 Miscellaneous Control A2 Output
000101 (0x5)	DAI0_MISCA3_O	DAI0 Miscellaneous Control A3 Output
000110 (0x6)	DAI0_MISCA4_O	DAI0 Miscellaneous Control A4 Output
000111 (0x7)	DAI0_MISCA5_O	DAI0 Miscellaneous Control A5 Output
001000 (0x8)	SPT0_ACLK_PBEN_O	SPORT 0 Clock A Output Enable
001001 (0x9)	SPT0_AFS_PBEN_O	SPORT 0 Frame Sync A Output Enable
001010 (0xA)	SPT0_AD0_PBEN_O	SPORT 0 Data AD0 Output Enable
001011 (0xB)	SPT0_AD1_PBEN_O	SPORT 0 Data AD1 Output Enable
001100 (0xC)	SPT0_BCLK_PBEN_O	SPORT 0 Clock B Output Enable
001101 (0xD)	SPT0_BFS_PBEN_O	SPORT 0 Frame Sync B Output Enable
001110 (0xE)	SPT0_BD0_PBEN_O	SPORT 0 Data BD0 Output Enable
001111 (0xF)	SPT0_BD1_PBEN_O	SPORT 0 Data BD1 Output Enable
010000 (0x10)	SPT1_ACLK_PBEN_O	SPORT 1 Clock A Output Enable
010001 (0x11)	SPT1_AFS_PBEN_O	SPORT 1 Frame Sync A Output Enable
010010 (0x12)	SPT1_AD0_PBEN_O	SPORT 1 Data AD0 Output Enable
010011 (0x13)	SPT1_AD1_PBEN_O	SPORT 1 Data AD1 Output Enable
010100 (0x14)	SPT1_BCLK_PBEN_O	SPORT 1 Clock B Output Enable
010101 (0x15)	SPT1_BFS_PBEN_O	SPORT 1 Frame Sync B Output Enable
010110 (0x16)	SPT1_BD0_PBEN_O	SPORT 1 Data BD0 Output Enable
010111 (0x17)	SPT1_BD1_PBEN_O	SPORT 1 Data BD1 Output Enable
011000 (0x18)	SPT2_ACLK_PBEN_O	SPORT 2 Clock A Output Enable
011001 (0x19)	SPT2_AFS_PBEN_O	SPORT 2 Frame Sync A Output Enable
011010 (0x1A)	SPT2_AD0_PBEN_O	SPORT 2 Data AD0 Output Enable
011011 (0x1B)	SPT2_AD1_PBEN_O	SPORT 2 Data AD1 Output Enable

Table 25-15: DAI0 Group F – Pin Output Enable (Continued)

Selection Code	Source Signal	Description (Source Selection)
011100 (0x1C)	SPT2_BCLK_PBEN_O	SPORT 2 Clock B Output Enable
011101 (0x1D)	SPT2_BFS_PBEN_O	SPORT 2 Frame Sync B Output Enable
011110 (0x1E)	SPT2_BD0_PBEN_O	SPORT 2 Data BD0 Output Enable
011111 (0x1F)	SPT2_BD1_PBEN_O	SPORT 2 Data BD1 Output Enable
100000 (0x20)	SPT3_ACLK_PBEN_O	SPORT 3 Clock A Output Enable
100001 (0x21)	SPT3_AFS_PBEN_O	SPORT 3 Frame Sync A Output Enable
100010 (0x22)	SPT3_AD0_PBEN_O	SPORT 3 Data AD0 Output Enable
100011 (0x23)	SPT3_AD1_PBEN_O	SPORT 3 Data AD1 Output Enable
100100 (0x24)	SPT3_BCLK_PBEN_O	SPORT 2 Clock B Output Enable
100101 (0x25)	SPT3_BFS_PBEN_O	SPORT 3 Frame Sync B Output Enable
100110 (0x26)	SPT3_BD0_PBEN_O	SPORT 3 Data BD0 Output Enable
100111 (0x27)	SPT3_BD1_PBEN_O	SPORT 3 Data BD1 Output Enable
101000 (0x28)	SPT0_ATDV_PBEN_O	SPORT 0 A Transmit Data Valid Output
101001 (0x29)	SPT0_BTDTV_PBEN_O	SPORT 0 B Transmit Data Valid Output
101010 (0x2A)	SPT1_ATDV_PBEN_O	SPORT 1 A Transmit Data Valid Output
101011 (0x2B)	SPT1_BTDTV_PBEN_O	SPORT 1 B Transmit Data Valid Output
101100 (0x2C)	SPT2_ATDV_PBEN_O	SPORT 2 A Transmit Data Valid Output
101101 (0x2D)	SPT2_BTDTV_PBEN_O	SPORT 2 B Transmit Data Valid Output
100111 (0x2E)	SPT3_ATDV_PBEN_O	SPORT 3 A Transmit Data Valid Output
101110 (0x2F)	SPT3_BTDTV_PBEN_O	SPORT 3 B Transmit Data Valid Output
110000 (0x30)–1111111 (0x3F)	Reserved	

DAI1 Group A – Clock Routing Source Signals

The group A clock routing signal sources are listed in the following table.

Table 25-16: DAI1 Group A – Clock Routing Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI1_PB01_O	Pin Buffer 1
00001 (0x1)	DAI1_PB02_O	Pin Buffer 2
00010 (0x2)	DAI1_PB03_O	Pin Buffer 3
00011 (0x3)	DAI1_PB04_O	Pin Buffer 4
00100 (0x4)	DAI1_PB05_O	Pin Buffer 5

Table 25-16: DAI1 Group A – Clock Routing Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
00101 (0x5)	DAI1_PB06_O	Pin Buffer 6
00110 (0x6)	DAI1_PB07_O	Pin Buffer 7
00111 (0x7)	DAI1_PB08_O	Pin Buffer 8
01000 (0x8)	DAI1_PB09_O	Pin Buffer 9
01001 (0x9)	DAI1_PB10_O	Pin Buffer 10
01010 (0xA)	DAI1_PB11_O	Pin Buffer 11
01011 (0xB)	DAI1_PB12_O	Pin Buffer 12
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	DAI1_CRS_PB03_O	Pin Buffer 3 (DAI0)
01111 (0xF)	DAI1_CRS_PB05_O	Pin Buffer 5 (DAI0)
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI1_PB19_O	Pin Buffer 19
10011 (0x13)	DAI1_PB20_O	Pin Buffer 20
10100 (0x14)	SPT4_ACLK_O	SPORT 4 Clock A
10101 (0x15)	SPT4_BCLK_O	SPORT 4 Clock B
10110 (0x16)	SPT5_ACLK_O	SPORT 5 Clock A
10111 (0x17)	SPT5_BCLK_O	SPORT 5 Clock B
11000 (0x18)	SPT6_ACLK_O	SPORT 6 Clock A
11001 (0x19)	SPT6_BCLK_O	SPORT 6 Clock B
11010 (0x1A)	SPDIF1_RX_CLK_O	SPDIF 1 Receive Clock Output
11011 (0x1B)	SPDIF1_RX_TDMCLK_O	SPDIF 1 Receive TDM Clock Output
11100 (0x1C)	PCG0_CLKC_O	Precision Clock C Output
11101 (0x1D)	PCG0_CLKD_O	Precision Clock D Output
11110 (0x1E)	LOW	Logic Level Low (0)
11111 (0x1F)	HIGH	Logic Level High (1)

DAI1 Group B – Serial Data Source Signals

The group B data sources are listed in the following table.

Table 25-17: DAI1 Group B – Serial Data Signals

Selection Code	Source Signal	Description (Source Selection)
000000 (0x0)	DAI1_PB01_O	Pin Buffer 1
000001 (0x1)	DAI1_PB02_O	Pin Buffer 2
000010 (0x2)	DAI1_PB03_O	Pin Buffer 3
000011 (0x3)	DAI1_PB04_O	Pin Buffer 4
000100 (0x4)	DAI1_PB05_O	Pin Buffer 5
000101 (0x5)	DAI1_PB06_O	Pin Buffer 6
000110 (0x6)	DAI1_PB07_O	Pin Buffer 7
000111 (0x7)	DAI1_PB08_O	Pin Buffer 8
001000 (0x8)	DAI1_PB09_O	Pin Buffer 9
001001 (0x9)	DAI1_PB10_O	Pin Buffer 10
001010 (0xA)	DAI1_PB11_O	Pin Buffer 11
001011 (0xB)	DAI1_PB12_O	Pin Buffer 12
001100 (0xC) -010001 (0x11)	Reserved	
010010 (0x12)	DAI1_PB19_O	Pin Buffer 19
010011 (0x13)	DAI1_PB20_O	Pin Buffer 20
010100 (0x14)	SPT4_AD0_O	SPORT 4 Data AD0
010101 (0x15)	SPT4_AD1_O	SPORT 4 Data AD1
010110 (0x16)	SPT4_BD0_O	SPORT 4 Data BD0
010111 (0x17)	SPT4_BD1_O	SPORT 4 Data BD1
011000 (0x18)	SPT5_AD0_O	SPORT 5 Data AD0
011001 (0x19)	SPT5_AD1_O	SPORT 5 Data AD1
011010 (0x1A)	SPT5_BD0_O	SPORT 5 Data BD0
011011 (0x1B)	SPT5_BD1_O	SPORT 5 Data BD1
011100 (0x1C)	SPT6_AD0_O	SPORT 6 Data AD0
011101 (0x1D)	SPT6_AD1_O	SPORT 6 Data AD1
011110 (0x1E)	SPT6_BD0_O	SPORT 6 Data BD0
011111 (0x1F)	SPT6_BD1_O	SPORT 6 Data BD1
100000 (0x20)	SRC4_DAT_OP_O	SRC4 Data Out
100001 (0x21)	SRC5_DAT_OP_O	SRC5 Data Out
100010 (0x22)	SRC6_DAT_OP_O	SRC6 Data Out
100011 (0x23)	SRC7_DAT_OP_O	SRC7 Data Out

Table 25-17: DAI1 Group B – Serial Data Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
100100 (0x24)	SRC4_TDM_IP_O	SRC4 Data Out
100101 (0x25)	SRC5_TDM_IP_O	SRC5 Data Out
100110 (0x26)	SRC6_TDM_IP_O	SRC6 Data Out
100111 (0x27)	SRC7_TDM_IP_O	SRC7 Data Out
101000 (0x28)	SPDIF1_RX_DAT_O	SPDIF 1 RX Serial Data Out
101100(0x2C)	SPT7_AD0_O	SPORT 7 Data AD0
101101(0x2D)	SPT7_AD1_O	SPORT 7 Data AD1
101110(0x2E)	SPT7_BD0_O	SPORT 7 Data BD0
101111(0x2F)	SPT7_BD1_O	SPORT 7 Data BD1
110000(0x30)	SPDIF1_TX_O	SPDIF 1 TX Biphase Stream
110001 (0x31)	SRC3_CRS_DAT_OP_O	SRC3 Data Out (DAI0)
110010 (0x32)	SRC3_CRS_TDM_IP_O	SRC3 Data Out (DAI0)
110001(0x33)–111101(0x3D)	Reserved	
111110 (0x3E)	LOW	Logic Level Low (0)
111111 (0x3F)	HIGH	Logic Level High (1)

DAI1 Group C – Frame Sync Source Signals

The group C frame sync signals are listed in the following table.

Table 25-18: DAI1 Group C – Frame Sync Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI0_PB01_O	Pin Buffer 1
00001 (0x1)	DAI1_PB02_O	Pin Buffer 2
00010 (0x2)	DAI1_PB03_O	Pin Buffer 3
00011 (0x3)	DAI1_PB04_O	Pin Buffer 4
00100 (0x4)	DAI1_PB05_O	Pin Buffer 5
00101 (0x5)	DAI1_PB06_O	Pin Buffer 6
00110 (0x6)	DAI1_PB07_O	Pin Buffer 7
00111 (0x7)	DAI1_PB08_O	Pin Buffer 8
01000 (0x8)	DAI1_PB09_O	Pin Buffer 9
01001 (0x9)	DAI1_PB10_O	Pin Buffer 10
01010 (0xA)	DAI1_PB11_O	Pin Buffer 11

Table 25-18: DAI1 Group C – Frame Sync Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
01011 (0xB)	DAI1_PB12_O	Pin Buffer 12
01100 (0xC)	Reserved	
01101 (0xD)	Reserved	
01110 (0xE)	DAI1_CRS_PB04_O	Pin Buffer 4 (DAI0)
01111 (0xF)	DAI1_CRS_PB06_O	Pin Buffer 6 (DAI0)
10000 (0x10)	Reserved	
10001 (0x11)	Reserved	
10010 (0x12)	DAI1_PB19_O	Pin Buffer 19
10011 (0x13)	DAI1_PB20_O	Pin Buffer 20
10100 (0x14)	SPT4_AFS_O	SPORT 4 Frame Sync A
10101 (0x15)	SPT4_BFS_O	SPORT 4 Frame Sync B
10110 (0x16)	SPT5_AFS_O	SPORT 5 Frame Sync A
10111 (0x17)	SPT5_BFS_O	SPORT 5 Frame Sync B
11000 (0x18)	SPT6_AFS_O	SPORT 6 Frame Sync A
11001 (0x19)	SPT6_BFS_O	SPORT 6 Frame Sync B
11010 (0x1A)	SPDIF1_FS_O	SPDIF 1 RX Frame Sync Output
11011 (0x1B)	Reserved	
11100 (0x1C)	PCG0_FSC_O	Precision Frame Sync C Output
11101 (0x1D)	PCG0_FSD_O	Precision Frame Sync D Output
11110 (0x1E)	LOW	Logic Level Low (0)
11111 (0x1F)	HIGH	Logic Level High (1)

DAI1 Group D – Pin Signal Assignment Source Signals

The group D pin signal assignments are listed in the following table.

Table 25-19: DAI1 Group D – Pin Signal Assignments

Selection Code	Source Signal	Description (Source Selection)
0000000 (0x0)	DAI1_PB01_O	Pin Buffer 1
0000001 (0x1)	DAI1_PB02_O	Pin Buffer 2
0000010 (0x2)	DAI1_PB03_O	Pin Buffer 3
0000011 (0x3)	DAI1_PB04_O	Pin Buffer 4
0000100 (0x4)	DAI1_PB05_O	Pin Buffer 5

Table 25-19: DAI1 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
0000101 (0x5)	DAI1_PB06_O	Pin Buffer 6
0000110 (0x6)	DAI1_PB07_O	Pin Buffer 7
0000111 (0x7)	DAI1_PB08_O	Pin Buffer 8
0001000 (0x8)	DAI1_PB09_O	Pin Buffer 9
0001001 (0x9)	DAI1_PB10_O	Pin Buffer 10
0001010 (0xA)	DAI1_PB11_O	Pin Buffer 11
0001011 (0xB)	DAI1_PB12_O	Pin Buffer 12
0001100 (0xC) - 0010001 (0x11)	Reserved	
0010010 (0x12)	DAI1_PB19_O	Pin Buffer 19
0010011 (0x13)	DAI1_PB20_O	Pin Buffer 20
0010100 (0x14)	SPT4_AD0_O	SPORT 4 Data AD0
0010101 (0x15)	SPT4_AD1_O	SPORT 4 Data AD1
0010110 (0x16)	SPT4_BD0_O	SPORT 4 Data BD0
0010111 (0x17)	SPT4_BD1_O	SPORT 4 Data BD1
0011000 (0x18)	SPT5_AD0_O	SPORT 5 Data AD0
0011001 (0x19)	SPT5_AD1_O	SPORT 5 Data AD1
0011010 (0x1A)	SPT5_BD0_O	SPORT 5 Data BD0
0011011 (0x1B)	SPT5_BD1_O	SPORT 5 Data BD1
0011100 (0x1C)	SPT6_AD0_O	SPORT 6 Data AD0
0011101 (0x1D)	SPT6_AD1_O	SPORT 6 Data AD1
0011110 (0x1E)	SPT6_BD0_O	SPORT 6 Data BD0
0011111 (0x1F)	SPT6_BD1_O	SPORT 6 Data BD1
0100000 (0x20)	SPT4_ACLK_O	SPORT 4 Clock A
0100001 (0x21)	SPT4_BCLK_O	SPORT4 Clock B
0100010 (0x22)	SPT5_ACLK_O	SPORT 5 Clock A
0100011 (0x23)	SPT5_BCLK_O	SPORT 5 Clock B
0100100 (0x24)	SPT6_ACLK_O	SPORT 6 Clock A
0100101 (0x25)	SPT6_BCLK_O	SPORT 6 Clock B
0100110 (0x26)	SPT4_AFS_O	SPORT 4 Frame Sync A
0100111 (0x27)	SPT4_BFS_O	SPORT 4 Frame Sync B
0101000 (0x28)	SPT5_AFS_O	SPORT 5 Frame Sync A

Table 25-19: DAI1 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
0101001 (0x29)	SPT5_BFS_O	SPORT 5 Frame Sync B
0101010 (0x2A)	SPT6_AFS_O	SPORT 5 Frame Sync A
0101011 (0x2B)	SPT6_BFS_O	SPORT 5 Frame Sync B
0101100 (0x2C)	SPT7_AD0_O	SPORT 7 Data AD0
0101101 (0x2D)	SPT7_AD1_O	SPORT 7 Data AD1
0101110 (0x2E)	SPT7_BD0_O	SPORT 7 Data BD0
0101111 (0x2F)	SPT7_BD1_O	SPORT 7 Data BD1
0110000 (0x30)	MLB0_CLKOUT	MLB PLL clock output
0110001 (0x31)	SPDIF1_TX_BLKSTART_O	SPDIF 1 TX Block Start Output
0110100 (0x34)	SPT7_ACLK_O	SPORT 7 Clock A
0110101 (0x35)	SPT7_BCLK_O	SPORT 7 Clock B
0110110 (0x36)	SPT7_AFS_O	SPORT 7 Frame Sync A
0110111 (0x37)	SPT7_BFS_O	SPORT 7 Frame Sync B
0111000 (0x38)	PCG0_CLKC_O	Precision Clock C
0111001 (0x39)	PCG0_CLKD_O	Precision Clock D
0111010 (0x3A)	PCG0_FSC_O	Precision Frame Sync C
0111011 (0x3B)	PCG0_FSD_O	Precision Frame Sync D
0111100 (0x3C)	Reserved	
0111101 (0x3D)	SRC4_DAT_OP_O	SRC4 Data Output
0111110 (0x3E)	SRC5_DAT_OP_O	SRC5 Data Output
0111111 (0x3F)	SRC6_DAT_OP_O	SRC6 Data Output
1000000 (0x40)	SRC7_DAT_OP_O	SRC7 Data Output
1000001 (0x41)	SPDIF1_RX_DAT_O	SPDIF 1 RX Data Output
1000010 (0x42)	SPDIF1_FS_O	SPDIF 1 RX Frame Sync Output
1000011 (0x43)	SPDIF1_RXCLK_O	SPDIF 1 RX Clock Output
1000100 (0x44)	SPDIF1_RX_TDMCLK_O	SPDIF 1 RX TDM Clock Output
1000101 (0x45)	SPDIF1_TX_O	SPDIF 1 TX Biphase Encoded Data Output
1000110 (0x46)	SPT4_ATDV_O	SPORT4 Transmit A Data Valid Output
1000111 (0x47)	SPT4_BTDTV_O	SPORT4 Transmit B Data Valid Output
1001000 (0x48)	SPT5_ATDV_O	SPORT5 Transmit A Data Valid Output
1001001 (0x49)	SPT5_BTDTV_O	SPORT5 Transmit B Data Valid Output

Table 25-19: DAI1 Group D – Pin Signal Assignments (Continued)

Selection Code	Source Signal	Description (Source Selection)
1001010 (0x4A)	SPT6_ATDV_O	SPORT6 Transmit A Data Valid Output
1001011 (0x4B)	SPT6_BTDTV_O	SPORT6 Transmit B Data Valid Output
1001100 (0x4C)	SPT7_ATDV_O	SPORT7 Transmit A Data Valid Output
1001101 (0x4D)	SPT7_BTDTV_O	SPORT7 Transmit B Data Valid Output
1001110 (0x4E)	DAI1_CRS_PB05_O	Pin Buffer 5 (DAI0)
1001111 (0x4F)	DAI1_CRS_PB06_O	Pin Buffer 6 (DAI0)
1010000 (0x50)	PCG0_CRS_CLKC_O	Precision Clock C (DAI0)
1010001 (0x51)	PCG0_CRS_CLKD_O	Precision Clock D (DAI0)
1010010 (0x52)	PCG0_CRS_FSC_O	Precision Frame Sync C (DAI0)
1010011 (0x53)	PCG0_CRS_FSD_O	Precision Frame Sync D (DAI0)
1010100 (0x54)	DAI1_CRS_PB03_O	Pin Buffer 3 (DAI0)
1010101 (0x55)	DAI1_CRS_PB04_O	Pin Buffer 4 (DAI0)
1010110 – 1111101	Reserved	
1111110 (0x7E)	LOW	Logic Level Low (0)
1111111 (0x7F)	HIGH	Logic Level High (1)

DAI1 Group E – Miscellaneous Source Signals

The group E miscellaneous signals are listed in the following table.

Table 25-20: DAI1 Group E – Miscellaneous Signals

Selection Code	Source Signal	Description (Source Selection)
00000 (0x0)	DAI1_PB01_O	Pin Buffer 1 Output
00001 (0x1)	DAI1_PB02_O	Pin Buffer 2 Output
00010 (0x2)	DAI1_PB03_O	Pin Buffer 3 Output
00011 (0x3)	DAI1_PB04_O	Pin Buffer 4 Output
00100 (0x4)	DAI1_PB05_O	Pin Buffer 5 Output
00101 (0x5)	DAI1_PB06_O	Pin Buffer 6 Output
00110 (0x6)	DAI1_PB07_O	Pin Buffer 7 Output
00111 (0x7)	DAI1_PB08_O	Pin Buffer 8 Output
01000 (0x8)	DAI1_PB09_O	Pin Buffer 9 Output
01001 (0x9)	DAI1_PB10_O	Pin Buffer 10 Output
01010 (0xA)	DAI1_PB11_O	Pin Buffer 11 Output

Table 25-20: DAI1 Group E – Miscellaneous Signals (Continued)

Selection Code	Source Signal	Description (Source Selection)
01011 (0xB)	DAI1_PB12_O	Pin Buffer 12 Output
01100 (0xC) - 10001 (0x11)	Reserved	
10010 (0x12)	DAI1_PB19_O	Pin Buffer 19 Output
10011 (0x13)	DAI1_PB20_O	Pin Buffer 20 Output
10100 (0x14)	SPT4_AFS_O	SPORT 4 Frame Sync A
10101 (0x15)	SPT4_BFS_O	SPORT 4 Frame Sync B
10110 (0x16)	SPT5_AFS_O	SPORT 5 Frame Sync A
10111 (0x17)	SPT5_BFS_O	SPORT 5 Frame Sync B
11000 (0x18)	SPT6_AFS_O	SPORT 6 Frame Sync A
11001 (0x19)	SPT6_BFS_O	SPORT 6 Frame Sync B
11010 (0x1A)	SPDIF1_TX_BLKSTART_O	SPDIF 1 TX Block Start Output
11011 (0x1B)	PCG0_FSC_O	Precision Frame Sync C
11100 (0x1C)	PCG0_CLKD_O	Precision Clock D
11101 (0x1D)	PCG0_FSD_O	Precision Frame Sync D
11110 (0x1E)	LOW	Logic Level Low (0) as a Source
11111 (0x1F)	HIGH	Logic Level High (1) as a Source

DAI1 Group F – Pin Output Enable Source Signals

The group F pin output enable signals are listed in the following table.

Table 25-21: DAI1 Group F – Pin Output Enable

Selection Code	Source Signal	Description (Source Selection)
000000 (0x0)	LOW	Logic Level Low (0)
000001 (0x1)	HIGH	Logic Level High (1)
000010 (0x2)	DAI1_MISCA0_O	DAI1 Miscellaneous Control A0 Output
000011 (0x3)	DAI1_MISCA1_O	DAI1 Miscellaneous Control A1 Output
000100 (0x4)	DAI1_MISCA2_O	DAI1 Miscellaneous Control A2 Output
000101 (0x5)	DAI1_MISCA3_O	DAI1 Miscellaneous Control A3 Output
000110 (0x6)	DAI1_MISCA4_O	DAI1 Miscellaneous Control A4 Output
000111 (0x7)	DAI1_MISCA5_O	DAI1 Miscellaneous Control A5 Output
001000 (0x8)	SPT4_ACLK_PBEN_O	SPORT 4 Clock A Output Enable
001001 (0x9)	SPT4_AFS_PBEN_O	SPORT 4 Frame Sync A Output Enable

Table 25-21: DAI1 Group F – Pin Output Enable (Continued)

Selection Code	Source Signal	Description (Source Selection)
001010 (0xA)	SPT4_AD0_PBEN_O	SPORT 4 Data AD0 Output Enable
001011 (0xB)	SPT4_AD1_PBEN_O	SPORT 4 Data AD1 Output Enable
001100 (0xC)	SPT4_BCLK_PBEN_O	SPORT 4 Clock B Output Enable
001101 (0xD)	SPT4_BFS_PBEN_O	SPORT 4 Frame Sync B Output Enable
001110 (0xE)	SPT4_BD0_PBEN_O	SPORT 4 Data BD0 Output Enable
001111 (0xF)	SPT4_BD1_PBEN_O	SPORT 4 Data BD1 Output Enable
010000 (0x10)	SPT5_ACLK_PBEN_O	SPORT 5 Clock A Output Enable
010001 (0x11)	SPT5_AFS_PBEN_O	SPORT 5 Frame Sync A Output Enable
010010 (0x12)	SPT5_AD0_PBEN_O	SPORT 5 Data AD0 Output Enable
010011 (0x13)	SPT5_AD1_PBEN_O	SPORT 5 Data AD1 Output Enable
010100 (0x14)	SPT5_BCLK_PBEN_O	SPORT 5 Clock B Output Enable
010101 (0x15)	SPT5_BFS_PBEN_O	SPORT 5 Frame Sync B Output Enable
010110 (0x16)	SPT5_BD0_PBEN_O	SPORT 5 Data BD0 Output Enable
010111 (0x17)	SPT5_BD1_PBEN_O	SPORT 5 Data BD1 Output Enable
011000 (0x18)	SPT6_ACLK_PBEN_O	SPORT 6 Clock A Output Enable
011001 (0x19)	SPT6_AFS_PBEN_O	SPORT 6 Frame Sync A Output Enable
011010 (0x1A)	SPT6_AD0_PBEN_O	SPORT 6 Data AD0 Output Enable
011011 (0x1B)	SPT6_AD1_PBEN_O	SPORT 6 Data AD1 Output Enable
011100 (0x1C)	SPT6_BCLK_PBEN_O	SPORT 6 Clock B Output Enable
011101 (0x1D)	SPT6_BFS_PBEN_O	SPORT 6 Frame Sync B Output Enable
011110 (0x1E)	SPT6_BD0_PBEN_O	SPORT 6 Data BD0 Output Enable
011111 (0x1F)	SPT6_BD1_PBEN_O	SPORT 6 Data BD1 Output Enable
100000 (0x20)	SPT7_ACLK_PBEN_O	SPORT 7 Clock A Output Enable
100001 (0x21)	SPT7_AFS_PBEN_O	SPORT 7 Frame Sync A Output Enable
100010 (0x22)	SPT7_AD0_PBEN_O	SPORT 7 Data AD0 Output Enable
100011 (0x23)	SPT7_AD1_PBEN_O	SPORT 7 Data AD1 Output Enable
100100 (0x24)	SPT7_BCLK_PBEN_O	SPORT 6 Clock B Output Enable
100101 (0x25)	SPT7_BFS_PBEN_O	SPORT 7 Frame Sync B Output Enable
100110 (0x26)	SPT7_BD0_PBEN_O	SPORT 7 Data BD0 Output Enable
100111 (0x27)	SPT7_BD1_PBEN_O	SPORT 7 Data BD1 Output Enable
101000 (0x28)	SPT4_ATDV_PBEN_O	SPORT 4 A Transmit Data Valid Output

Table 25-21: DAI1 Group F – Pin Output Enable (Continued)

Selection Code	Source Signal	Description (Source Selection)
101001 (0x29)	SPT4_BTDTV_PBEN_O	SPORT 4 B Transmit Data Valid Output
101010 (0x2A)	SPT5_ATDV_PBEN_O	SPORT 5 A Transmit Data Valid Output
101011 (0x2B)	SPT5_BTDTV_PBEN_O	SPORT 5 B Transmit Data Valid Output
101100 (0x2C)	SPT6_ATDV_PBEN_O	SPORT 6 A Transmit Data Valid Output
101101 (0x2D)	SPT6_BTDTV_PBEN_O	SPORT 6 B Transmit Data Valid Output
100111 (0x2E)	SPT7_ATDV_PBEN_O	SPORT 7 A Transmit Data Valid Output
101110 (0x2F)	SPT7_BTDTV_PBEN_O	SPORT 7 B Transmit Data Valid Output
110000 (0x30)–1111111 (0x3F)	Reserved	

DAI Destination Registers Overview

The tables in DAI Routing Capabilities provide high level descriptions that illustrate source (output) connections to destinations (inputs) depending on the routing groups A through F. This sections lists the various input fields (INx), which are described in detail in the register descriptions section.

Table 25-22: Clock Destination Registers (Group A)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_CLK0	IN0	SPT0_ACLK_I	SPT4_ACLK_I	SPORT0A or SPORT4A Clock
	IN1	SPT0_BCLK_I	SPT4_BCLK_I	SPORT0B or SPORT4B Clock
	IN2	SPT1_ACLK_I	SPT5_ACLK_I	SPORT1A or SPORT5A Clock
	IN3	SPT1_BCLK_I	SPT5_BCLK_I	SPORT1B or SPORT5B Clock
	IN4	SPT2_ACLK_I	SPT6_ACLK_I	SPORT2A or SPORT6A Clock
	IN5	SPT2_BCLK_I	SPT6_BCLK_I	SPORT2B or SPORT6B Clock
DAI_CLK1	IN0	SRC0_CLK_IP_I	SRC4_CLK_IP_I	SRC0 or SRC4 Clock Input
	IN1	SRC0_CLK_OP_I	SRC4_CLK_OP_I	SRC0 or SRC4 Clock Output
	IN2	SRC1_CLK_IP_I	SRC5_CLK_IP_I	SRC1 or SRC5 Clock Input
	IN3	SRC1_CLK_OP_I	SRC5_CLK_OP_I	SRC1 or SRC5 Clock Output
	IN4	SRC2_CLK_IP_I	SRC6_CLK_IP_I	SRC2 or SRC6 Clock Input
	IN5	SRC2_CLK_OP_I	SRC6_CLK_OP_I	SRC2 or SRC6 Clock Output

Table 25-22: Clock Destination Registers (Group A) (Continued)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_CLK2	IN0	SRC3_CLK_IP_I	SRC7_CLK_IP_I	SRC3 or SRC7 Clock Input
	IN1	SRC3_CLK_OP_I	SRC7_CLK_OP_I	SRC3 or SRC7 Clock Output
	IN2	SPDIF0_TX_CLK_I	SPDIF1_TX_CLK_I	SPDIF0 or SPDIF1 TX Clock
	IN3	PDM0_CLK0_I	PDM1_CLK0_I	PDM0 or PDM1 Clock0 Input
	IN4	PDM0_BCLK_I	PDM1_BCLK_I	PDM0 or PDM1 Bit Clock Input
DAI_CLK3	IN5	SPDIF0_TX_HFCLK_I	SPDIF1_TX_HFCLK_I	SPDIF0 or SPDIF1 TX HF Clock
DAI_CLK4	IN0	PCG0_EXTCLKA_I	PCG0_EXTCLKC_I	PCG0 External Clock A or C
	IN1	PCG0_EXTCLKB_I	PCG0_EXTCLKD_I	PCG0 External Clock B or D
	IN3	SPDIF0_TX_EXT_SYNC_I	SPDIF1_TX_EXT_SYNC_I	SPDIF0 or SPDIF1 TX External Sync
	IN4	PCG0_SYNC_CLKA_I	PCG0_SYNC_CLKC_I	PCG0 Sync Clock A or C
	IN5	PCG0_SYNC_CLKB_I	PCG0_SYNC_CLKD_I	PCG0 Sync Clock B or D
DAI_CLK5	IN0	SPT3_ACLK_I	SPT7_ACLK_I	SPORT3A or SPORT 7A Clock
	IN1	SPT3_BCLK_I	SPT7_BCLK_I	SPORT3B or SPORT 7B Clock
	IN2	PCG0_SYNC_CLK_E_I	PCG0_SYNC_CLK_G_I	PCG0 Sync Clock E or G
	IN3	PCG0_SYNC_CLK_F_I	PCG0_SYNC_CLK_H_I	PCG0 Sync Clock F or H
	IN4	PCG0_EXTCLKE_I	PCG0_EXTCLKG_I	PCG0 External Clock E or G
	IN5	PCG0_EXTCLKF_I	PCG0_EXTCLKH_I	PCG0 External Clock F or H

Table 25-23: Data Destination Registers (Group B)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_DAT0	IN0	SPT0_AD0_I	SPT4_AD0_I	SPORT0A or SPORT4A Primary Data
	IN1	SPT0_AD1_I	SPT4_AD1_I	SPORT0A or SPORT4A Secondary Data
	IN2	SPT0_BD0_I	SPT4_BD0_I	SPORT0B or SPORT4B Primary Data
	IN3	SPT0_BD1_I	SPT4_BD1_I	SPORT0B or SPORT4B Secondary Data
	IN4	SPT1_AD0_I	SPT5_AD0_I	SPORT1A or SPORT5A Primary Data

Table 25-23: Data Destination Registers (Group B) (Continued)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_DAT1	IN0	SPT1_AD1_I	SPT5_AD1_I	SPORT1A or SPORT5A Secondary Data
	IN1	SPT1_BD0_I	SPT5_BD0_I	SPORT1B or SPORT5B Primary Data
	IN2	SPT1_BD1_I	SPT5_BD1_I	SPORT1B or SPORT5B Secondary Data
	IN3	SPT2_AD0_I	SPT6_AD0_I	SPORT2A or SPORT6A Primary Data
	IN4	SPT2_AD1_I	SPT6_AD1_I	SPORT2A or SPORT6A Secondary Data
DAI_DAT2	IN0	SPT2_BD0_I	SPT6_BD0_I	SPORT2B or SPORT6B Primary Data
	IN1	SPT2_BD1_I	SPT6_BD1_I	SPORT2B or SPORT6B Secondary Data
	IN2	SRC0_DAT_IP_I	SRC4_DAT_IP_I	SRC0 or SRC4 Data
	IN3	SRC1_DAT_IP_I	SRC5_DAT_IP_I	SRC1 or SRC5 Data
	IN4	SRC2_DAT_IP_I	SRC6_DAT_IP_I	SRC2 or SRC6 Data
DAI_DAT3	IN0	SRC3_DAT_IP_I	SRC7_DAT_IP_I	SRC3 or SRC7 Data
	IN1	SRC0_DAT_TDM_OP_I	SRC4_DAT_TDM_OP_I	SRC0 or SRC4 TDM output port
	IN2	SRC1_DAT_TDM_OP_I	SRC5_DAT_TDM_OP_I	SRC1 or SRC5 TDM Output port Data
	IN3	SRC2_DAT_TDM_OP_I	SRC6_DAT_TDM_OP_I	SRC2 or SRC6 TDM Output port Data
	IN4	SRC3_DAT_TDM_OP_I	SRC7_DAT_TDM_OP_I	SRC3 or SRC7 TDM Output port Data
DAI_DAT4	IN0	SPDIF0_TX_DAT_I	SPDIF1_TX_DAT_I	SPDIF0 or SPDIF1 serial transmitter Data
	IN1	PDM0_DAT0_I	PDM1_DAT0_I	PDM0 or PDM1 Data0 Input
	IN2	PDM0_DAT1_I	PDM1_DAT1_I	PDM0 or PDM1 Data1 Input
DAI_DAT5	IN4	SPDIF0_RX_I	SPDIF1_RX_I	SPDIF0 or SPDIF1 receiver bi-phase Data

Table 25-23: Data Destination Registers (Group B) (Continued)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_DAT6	IN0	SPT3_AD0_I	SPT7_AD0_I	SPORT3A or SPORT7A Primary Data
	IN1	SPT3_AD1_I	SPT7_AD1_I	SPORT3A or SPORT7A Secondary Data
	IN2	SPT3_BD0_I	SPT7_BD0_I	SPORT3B or SPORT7B Primary Data
	IN3	SPT3_BD1_I	SPT7_BD1_I	SPORT3B or SPORT7B Secondary Data
	IN4	Reserved	Reserved	

Table 25-24: Frame Sync Destination Registers (Group C)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_FS0	IN0	SPT0_AFS_I	SPT4_AFS_I	SPORT0A or SPORT4A Frame Sync
	IN1	SPT0_BFS_I	SPT4_BFS_I	SPORT0B or SPORT4B Frame Sync
	IN2	SPT1_AFS_I	SPT5_AFS_I	SPORT1A or SPORT5A Frame Sync
	IN3	SPT1_BFS_I	SPT5_BFS_I	SPORT1B or SPORT5B Frame Sync
	IN4	SPT2_AFS_I	SPT6_AFS_I	SPORT2A or SPORT6A Frame Sync
	IN5	SPT2_BFS_I	SPT6_BFS_I	SPORT2B or SPORT6B Frame Sync
DAI_FS1	IN0	SRC0_FS_IP_I	SRC4_FS_IP_I	SRC0 or SRC4 Frame Sync Input
	IN1	SRC0_FS_OP_I	SRC4_FS_OP_I	SRC0 or SRC4 Frame Sync Output
	IN2	SRC1_FS_IP_I	SRC5_FS_IP_I	SRC1 or SRC5 Frame Sync Input
	IN3	SRC1_FS_OP_I	SRC5_FS_OP_I	SRC1 or SRC5 Frame Sync Output
	IN4	SRC2_FS_IP_I	SRC6_FS_IP_I	SRC2 or SRC6 Frame Sync Input
	IN5	SRC2_FS_OP_I	SRC6_FS_OP_I	SRC2 or SRC6 Frame Sync Output
DAI_FS2	IN0	SRC3_FS_IP_I	SRC7_FS_IP_I	SRC3 or SRC7 Frame Sync Input
	IN1	SRC3_FS_OP_I	SRC7_FS_OP_I	SRC3 or SRC7 Frame Sync Output
	IN2	SPDIF0_TX_FS_I	SPDIF1_TX_FS_I	SPDIF0 or SPDIF1 Transmit Frame Sync
DAI_FS4	IN0	SPT3_AFS_I	SPT7_AFS_I	SPORT3A or SPORT7A Frame Sync
	IN1	SPT3_BFS_I	SPT7_BFS_I	SPORT3B or SPORT7B Frame Sync
	IN2	TM0_ACI14_I	TM0_ACI15_I	ACI 14 or ACI 15 Timer Input
	IN3	PDM0_LRCLK_I	PDM1_LRCLK_I	PDM0 or PDM1 Left/Right Clock or Frame Sync Input

Table 25-25: Pin Buffer Assignment Destination Registers (Group D)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_PIN0	IN0	DAI0_PB01_I	DAI1_PB01_I	Pin Buffer 1
	IN1	DAI0_PB02_I	DAI1_PB02_I	Pin Buffer 2
	IN2	DAI0_PB03_I	DAI1_PB03_I	Pin Buffer 3
	IN3	DAI0_PB04_I	DAI1_PB04_I	Pin Buffer 4
DAI_PIN1	IN0	DAI0_PB05_I	DAI1_PB05_I	Pin Buffer 5
	IN1	DAI0_PB06_I	DAI1_PB06_I	Pin Buffer 6
	IN2	DAI0_PB07_I	DAI1_PB07_I	Pin Buffer 7
	IN3	DAI0_PB08_I	DAI1_PB08_I	Pin Buffer 8
DAI_PIN2	IN0	DAI0_PB09_I	DAI1_PB09_I	Pin Buffer 9
	IN1	DAI0_PB10_I	DAI1_PB10_I	Pin Buffer 10
	IN2	DAI0_PB11_I	DAI1_PB11_I	Pin Buffer 11
	IN3	DAI0_PB12_I	DAI1_PB12_I	Pin Buffer 12
DAI_PIN3	IN0	Reserved		
	IN1	Reserved		
	IN2	Reserved		
	IN3	Reserved		
DAI_PIN4	IN0	Reserved		
	IN1	Reserved		
	IN2	DAI0_PB19_I	DAI1_PB19_I	Pin Buffer 19
	IN3	DAI0_PB20_I	DAI1_PB20_I	Pin Buffer 20
	IN4	INV_DAI0_PB19_I	INV_DAI1_PB19_I	Inverted Pin Buffer 19
	IN5	INV_DAI0_PB20_I	INV_DAI1_PB20_I	Inverted Pin Buffer 20

Table 25-26: Pin Buffer Enable Destination Registers (Group F)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_PBEN0	IN0	DAI0_PBEN01_I	DAI1_PBEN01_I	Pin Buffer 1 Enable
	IN1	DAI0_PBEN02_I	DAI1_PBEN02_I	Pin Buffer 2 Enable
	IN2	DAI0_PBEN03_I	DAI1_PBEN03_I	Pin Buffer 3 Enable
	IN3	DAI0_PBEN04_I	DAI1_PBEN04_I	Pin Buffer 4 Enable
	IN4	DAI0_PBEN05_I	DAI1_PBEN05_I	Pin Buffer 5 Enable

Table 25-26: Pin Buffer Enable Destination Registers (Group F) (Continued)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_PBEN1	IN0	DAI0_PBEN06_I	DAI1_PBEN06_I	Pin Buffer 6 Enable
	IN1	DAI0_PBEN07_I	DAI1_PBEN07_I	Pin Buffer 7 Enable
	IN2	DAI0_PBEN08_I	DAI1_PBEN08_I	Pin Buffer 8 Enable
	IN3	DAI0_PBEN09_I	DAI1_PBEN09_I	Pin Buffer 9 Enable
	IN4	DAI0_PBEN10_I	DAI1_PBEN10_I	Pin Buffer 10 Enable
DAI_PBEN2	IN0	DAI0_PBEN11_I	DAI1_PBEN11_I	Pin Buffer 11 Enable
	IN1	DAI0_PBEN12_I	DAI1_PBEN12_I	Pin Buffer 12 Enable
	IN2	DAI0_PBEN13_I	DAI1_PBEN13_I	Pin Buffer 13 Enable
	IN3	DAI0_PBEN14_I	DAI1_PBEN14_I	Pin Buffer 14 Enable
	IN4	DAI0_PBEN15_I	DAI1_PBEN15_I	Pin Buffer 15 Enable
DAI_PBEN3	IN0	DAI0_PBEN16_I	DAI1_PBEN16_I	Pin Buffer 16 Enable
	IN1	DAI0_PBEN17_I	DAI1_PBEN17_I	Pin Buffer 17 Enable
	IN2	DAI0_PBEN18_I	DAI1_PBEN18_I	Pin Buffer 18 Enable
	IN3	DAI0_PBEN19_I	DAI1_PBEN19_I	Pin Buffer 19 Enable
	IN4	DAI0_PBEN20_I	DAI1_PBEN20_I	Pin Buffer 20 Enable

The the *Miscellaneous Control Registers* table, DAI_MISCAx_I is an alias name for DAI_INT_x_I.

Table 25-27: Miscellaneous Control Destination Registers (Group E)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
DAI_MISC0	IN6	DAI0_MISCA0_I/ DAI0_INT_6_I	DAI1_MISCA0_I/ DAI1_INT_6_I	DAIx miscellaneous A0 input, or DAIx interrupt 28
	IN7	DAI0_MISCA1_I/ DAI0_INT_7_I	DAI1_MISCA1_I/ DAI1_INT_7_I	DAIx miscellaneous A1 input, or DAIx interrupt 29
	IN8	DAI0_MISCA2_I/ DAI0_INT_8_I	DAI1_MISCA2_I/ DAI1_INT_8_I	DAIx miscellaneous A2 input, or DAIx interrupt 30
	IN9	DAI0_MISCA3_I/ DAI0_INT_9_I	DAI1_MISCA3_I/ DAI1_INT_9_I	DAIx miscellaneous A3 input, or DAIx interrupt 31
DAI_MISC1	IN0	DAI0_INT_0	DAI1_INT_0	DAIx interrupt 22
	IN1	DAI0_INT_1	DAI1_INT_1	DAIx interrupt 23
	IN2	DAI0_INT_2	DAI1_INT_2	DAIx interrupt 24

Table 25-27: Miscellaneous Control Destination Registers (Group E) (Continued)

DAI Register	Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
	IN3	DAI0_INT_3	DAI1_INT_3	DAIx interrupt 25
	IN4	DAI0_INT_4	DAI1_INT_4	DAIx interrupt 26
	IN5	DAI0_INT_5	DAI1_INT_5	DAIx interrupt 27
DAI_MISC2	IN0	PCG_HWA_TRIG_I	PCG_HWC_TRIG_I	PCG hardware trigger input A or C
	IN1	PCG_HWB_TRIG_I	PCG_HWD_TRIG_I	PCG hardware trigger input B or D
	IN2	PCG_HWE_TRIG_I	PCG_HWG_TRIG_I	PCG hardware trigger input E or G
	IN3	PCG_HWF_TRIG_I	PCG_HWH_TRIG_I	PCG hardware trigger input A or C

The *Interrupt Events* table presents the information for the following registers.

- DAI_IMSK_PRI (Core Interrupt Priority Assignment) register
- DAI_IMSK_FE (Falling-Edge Interrupt Mask) and DAI_IMSK_RE (Rising-Edge Interrupt Mask) registers
- DAI_IRPTL_H (High-Priority Interrupt Latch) and DAI_IRPTL_L (Low-Priority Interrupt Latch) registers
- DAI_IRPTL_HS (Shadow High Interrupt Latch) and DAI_IRPTL_LS (Shadow Low Interrupt Latch) registers

Table 25-28: Interrupt Events

Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
RXVALID	SPDIF0_RXVALID_INT	SPDIF1_RXVALID_INT	SPDIF0 or SPDIF1 RX Valid interrupt
RXLOCK	SPDIF0_RXLOCK_INT	SPDIF1_RXLOCK_INT	SPDIF0 or SPDIF1 RX Lock interrupt
RXLOSSOF-LOCK	SPDIF0_RXLOSSOFLOCK_INT	SPDIF1_RXLOSSOFLOCK_INT	SPDIF0 or SPDIF1 RX Loss of Lock interrupt
RXNONAUDIO	SPDIF0_RXNONAUDIO_INT	SPDIF1_RXNONAUDIO_INT	SPDIF0 or SPDIF1 RX Non-audio interrupt
SRC0MUTE	SRC0_MUTE_INT	SRC4_MUTE_INT	DAI0 or DAI1 ASRC0 Mute interrupt
SRC1MUTE	SRC1_MUTE_INT	SRC5_MUTE_INT	DAI0 or DAI1 ASRC1 Mute interrupt
SRC2MUTE	SRC2_MUTE_INT	SRC6_MUTE_INT	DAI0 or DAI1 ASRC2 Mute interrupt
SRC3MUTE	SRC3_MUTE_INT	SRC7_MUTE_INT	DAI0 or DAI1 ASRC3 Mute interrupt

Table 25-28: Interrupt Events (Continued)

Bit Field Name	DAI0 Mapping	DAI1 Mapping	Description
MISCINT0	DAI0_INT_00	DAI1_INT_00	DAI0 or DAI1 Miscellaneous Interrupt 0
MISCINT1	DAI0_INT_01	DAI1_INT_01	DAI0 or DAI1 Miscellaneous Interrupt 1
MISCINT2	DAI0_INT_02	DAI1_INT_02	DAI0 or DAI1 Miscellaneous Interrupt 2
MISCINT3	DAI0_INT_03	DAI1_INT_03	DAI0 or DAI1 Miscellaneous Interrupt 3
MISCINT4	DAI0_INT_04	DAI1_INT_04	DAI0 or DAI1 Miscellaneous Interrupt 4
MISCINT5	DAI0_INT_05	DAI1_INT_05	DAI0 or DAI1 Miscellaneous Interrupt 5
MISCINT6	DAI0_EXTMISCA0_INT/ DAI0_INT_06	DAI1_EXTMISCA0_INT/ DAI1_INT_06	DAI0 or DAI1 Miscellaneous Interrupt 6 / External Miscellaneous A0 Interrupt
MISCINT7	DAI0_EXTMISCA1_INT/ DAI0_INT_07	DAI1_EXTMISCA1_INT/ DAI1_INT_07	DAI0 or DAI1 Miscellaneous Interrupt 7 / External Miscellaneous A1 Interrupt
MISCINT8	DAI0_EXTMISCA2_INT/ DAI0_INT_08	DAI1_EXTMISCA2_INT/ DAI1_INT_08	DAI0 or DAI1 Miscellaneous Interrupt 8 / External Miscellaneous A2 Interrupt
MISCINT9	DAI0_EXTMISCA3_INT/ DAI0_INT_09	DAI1_EXTMISCA3_INT/ DAI1_INT_09	DAI0 or DAI1 Miscellaneous Interrupt 9 / External Miscellaneous A3 Interrupt

SPORT Grouping with DAI

DAI supports two registers [DAI_GBL_SP_EN](#) and [DAI_GBL_INT_EN](#) to group enabling/disabling and interrupt/trigger generation/reception for multiple SPORTs.

For more information about this, refer to the [Grouping of SPORTs](#) section.

Global PCG Enable

The DAI module includes the [DAI_GBL_PCG_EN](#) register that enables and disables multiple PCGs globally at the same time. For more information, see Global PCG Enable section in the PCG chapter.

ADSP-2159x DAI Register Descriptions

The Digital Audio Interface (DAI_n) registers are used to configure DAI destinations for the DAI sources shown in the Group A - F Routing tables. (DAI) contains the following registers.

Table 25-29: ADSP-2159x DAI Register List

Name	Description
DAI_CLK0	Clock Routing Control Register 0
DAI_CLK1	Clock Routing Control Register 1
DAI_CLK2	Clock Routing Control Register 2
DAI_CLK3	Clock Routing Control Register 3
DAI_CLK4	Clock Routing Control Register 4
DAI_CLK5	Clock Routing Control Register 5
DAI_DAT0	Serial Data Routing Control Register 0
DAI_DAT1	Serial Data Routing Control Register 1
DAI_DAT2	Serial Data Routing Control Register 2
DAI_DAT3	Serial Data Routing Control Register 3
DAI_DAT4	Serial Data Routing Control Register 4
DAI_DAT5	Serial Data Routing Control Register 5
DAI_DAT6	Serial Data Routing Control Register 6
DAI_EXTD_CLK0	Extended Clock Routing Control Register 0
DAI_EXTD_CLK1	Extended Clock Routing Control Register 1
DAI_EXTD_CLK2	Extended Clock Routing Control Register 2
DAI_EXTD_CLK3	Extended Clock Routing Control Register 3
DAI_EXTD_CLK4	Extended Clock Routing Control Register 4
DAI_EXTD_CLK5	Extended Clock Routing Control Register 5
DAI_EXTD_DAT0	Extended Serial Data Routing Control Register 0
DAI_EXTD_DAT1	Extended Serial Data Routing Control Register 1
DAI_EXTD_DAT2	Extended Serial Data Routing Control Register 2
DAI_EXTD_DAT3	Extended Serial Data Routing Control Register 3
DAI_EXTD_DAT4	Extended Serial Data Routing Control Register 4
DAI_EXTD_DAT5	Extended Serial Data Routing Control Register 5
DAI_EXTD_DAT6	Extended Serial Data Routing Control Register 6
DAI_EXTD_FS0	Extended Frame Sync Routing Control Register 0
DAI_EXTD_FS1	Extended Frame Sync Routing Control Register 1

Table 25-29: ADSP-2159x DAI Register List (Continued)

Name	Description
DAI_EXTD_FS2	Extended Frame Sync Routing Control Register 2
DAI_EXTD_FS4	Extended Frame Sync Routing Control Register 4
DAI_EXTD_MISC0	Extended Miscellaneous Control Register 0
DAI_EXTD_MISC1	Extended Miscellaneous Control Register 1
DAI_EXTD_MISC2	Extended Miscellaneous Control Register 2
DAI_EXTD_PBEN0	Extended Pin Buffer Enable Register 0
DAI_EXTD_PBEN1	Extended Pin Buffer Enable Register 1
DAI_EXTD_PBEN2	Extended Pin Buffer Enable Register 2
DAI_EXTD_PBEN3	Extended Pin Buffer Enable Register 3
DAI_EXTD_PIN0	Extended Pin Buffer Assignment Register 0
DAI_EXTD_PIN1	Extended Pin Buffer Assignment Register 1
DAI_EXTD_PIN2	Extended Pin Buffer Assignment Register 2
DAI_EXTD_PIN3	Extended Pin Buffer Assignment Register 3
DAI_EXTD_PIN4	Extended Pin Buffer Assignment Register 4
DAI_FS0	Frame Sync Routing Control Register 0
DAI_FS1	Frame Sync Routing Control Register 1
DAI_FS2	Frame Sync Routing Control Register 2
DAI_FS4	Frame Sync Routing Control Register 4
DAI_GBL_INT_EN	Global SPORT Interrupt Grouping Register
DAI_GBL_PCG_EN	Global PCG Enable Control Register
DAI_GBL_SP_EN	Global SPORT Enable Register
DAI_IMSK_FE	Falling-Edge Interrupt Mask Register
DAI_IMSK_PRI	Core Interrupt Priority Assignment Register
DAI_IMSK_RE	Rising-Edge Interrupt Mask Register
DAI_IRPTL_H	High Priority Interrupt Latch Register
DAI_IRPTL_HS	Shadow High Priority Interrupt Latch Register
DAI_IRPTL_L	Low Priority Interrupt Latch Register
DAI_IRPTL_LS	Shadow Low Priority Interrupt Latch Register
DAI_MISC0	Miscellaneous Control Register 0
DAI_MISC1	Miscellaneous Control Register 1
DAI_MISC2	Miscellaneous Control Register 1

Table 25-29: ADSP-2159x DAI Register List (Continued)

Name	Description
DAI_PBEN0	Pin Buffer Enable Register 0
DAI_PBEN1	Pin Buffer Enable Register 1
DAI_PBEN2	Pin Buffer Enable Register 2
DAI_PBEN3	Pin Buffer Enable Register 3
DAI_PIN0	Pin Buffer Assignment Register 0
DAI_PIN1	Pin Buffer Assignment Register 1
DAI_PIN2	Pin Buffer Assignment Register 2
DAI_PIN3	Pin Buffer Assignment Register 3
DAI_PIN4	Pin Buffer Assignment Register 4
DAI_PIN_STAT	Pin Status Register

Clock Routing Control Register 0

The `DAI_CLK0` register provides clock routing connections for the serial ports (SPORTs).

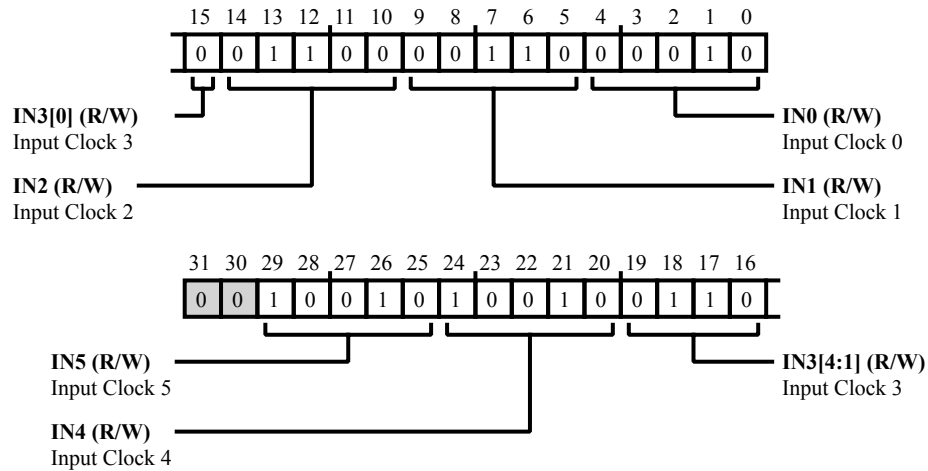


Figure 25-18: DAI_CLK0 Register Diagram

Table 25-30: DAI_CLK0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. DAI_CLK0 . IN5 holds the Source signal assignment that will be routed to the DAI_CLK0 . IN5 Destination. Refer to the Group A Signals table for Source and Destination mappings
24:20 (R/W)	IN4	Input Clock 4. DAI_CLK0 . IN4 holds the Source signal assignment that will be routed to the DAI_CLK0 . IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings
19:15 (R/W)	IN3	Input Clock 3. DAI_CLK0 . IN3 holds the Source signal assignment that will be routed to the DAI_CLK0 . IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings
14:10 (R/W)	IN2	Input Clock 2. DAI_CLK0 . IN2 holds the Source signal assignment that will be routed to the DAI_CLK0 . IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK0 . IN1 holds the Source signal assignment that will be routed to the DAI_CLK0 . IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings

Table 25-30: DAI_CLK0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK0.IN0 holds the Source signal assignment that will be routed to the DAI_CLK0.IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings

Clock Routing Control Register 1

The `DAI_CLK1` register provides clock routing connections for the asynchronous sample rate converters (ASRC).

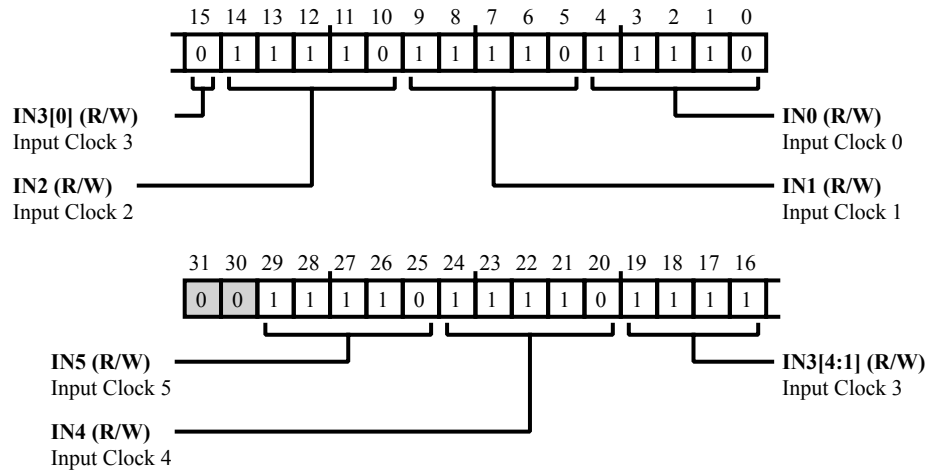


Figure 25-19: DAI_CLK1 Register Diagram

Table 25-31: DAI_CLK1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. DAI_CLK1 . IN5 holds the Source signal assignment that will be routed to the DAI_CLK1 . IN5 Destination. Refer to the Group A Signals table for Source and Destination mappings
24:20 (R/W)	IN4	Input Clock 4. DAI_CLK1 . IN4 holds the Source signal assignment that will be routed to the DAI_CLK1 . IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings
19:15 (R/W)	IN3	Input Clock 3. DAI_CLK1 . IN3 holds the Source signal assignment that will be routed to the DAI_CLK1 . IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings
14:10 (R/W)	IN2	Input Clock 2. DAI_CLK1 . IN2 holds the Source signal assignment that will be routed to the DAI_CLK1 . IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK1 . IN1 holds the Source signal assignment that will be routed to the DAI_CLK1 . IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings.

Table 25-31: DAI_CLK1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK1 . IN0 holds the Source signal assignment that will be routed to the DAI_CLK1 . IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings.

Clock Routing Control Register 2

The `DAI_CLK2` register provides clock routing connections for the S/PDIF and ASRC.

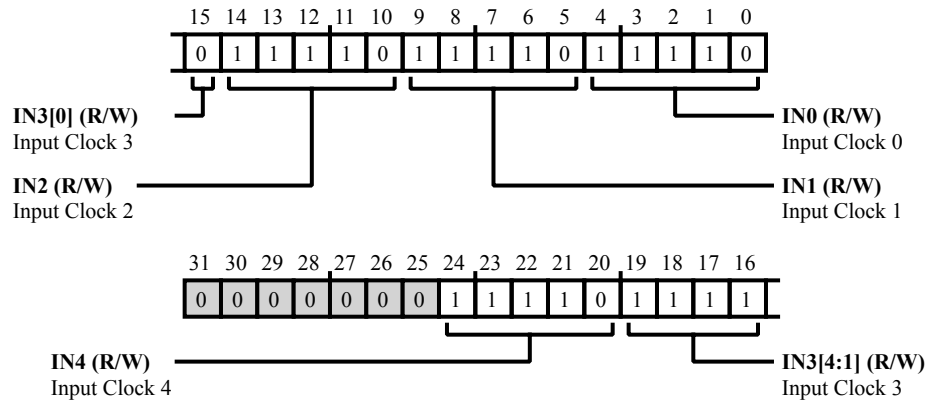


Figure 25-20: DAI_CLK2 Register Diagram

Table 25-32: DAI_CLK2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
24:20 (R/W)	IN4	Input Clock 4. DAI_CLK2 . IN4 holds the Source signal assignment that will be routed to the DAI_CLK2 . IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings.
19:15 (R/W)	IN3	Input Clock 3. DAI_CLK2 . IN3 holds the Source signal assignment that will be routed to the DAI_CLK2 . IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings.
14:10 (R/W)	IN2	Input Clock 2. DAI_CLK2 . IN2 holds the Source signal assignment that will be routed to the DAI_CLK2 . IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings.
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK2 . IN1 holds the Source signal assignment that will be routed to the DAI_CLK2 . IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings.
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK2 . IN0 holds the Source signal assignment that will be routed to the DAI_CLK2 . IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings.

Clock Routing Control Register 3

The `DAI_CLK3` register provides clock routing connections for the S/PDIF.

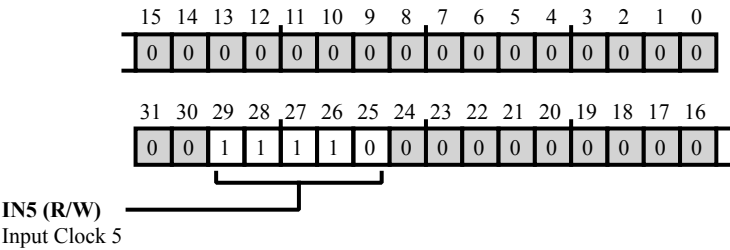


Figure 25-21: DAI_CLK3 Register Diagram

Table 25-33: DAI_CLK3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. The <code>DAI_CLK3</code> . IN5 bit field provides the S/PDIF oversampling clock.

Clock Routing Control Register 4

The `DAI_CLK4` register provides clock routing connections for the precision clock generators (PCGs).

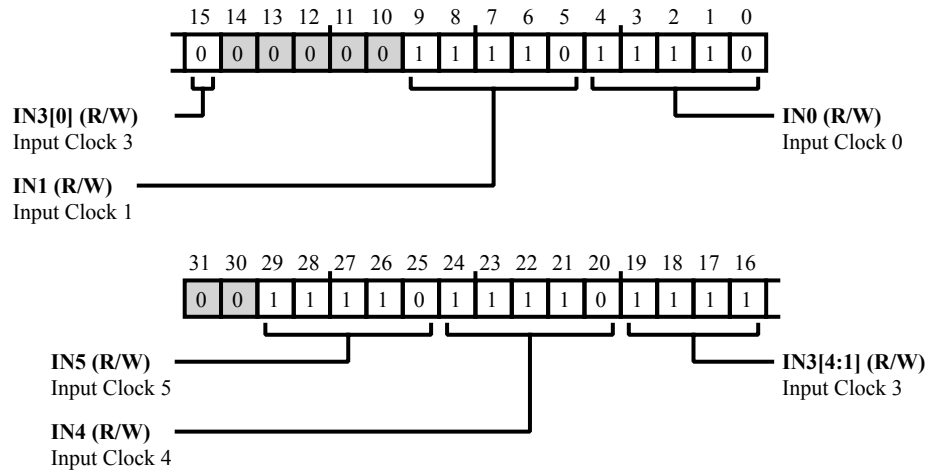


Figure 25-22: DAI_CLK4 Register Diagram

Table 25-34: DAI_CLK4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. DAI_CLK4 . IN5 holds the Source signal assignment that will be routed to the DAI_CLK4 . IN5 Destination. Refer to the Group A Signals table for Source and Destination mappings.
24:20 (R/W)	IN4	Input Clock 4. DAI_CLK4 . IN4 holds the Source signal assignment that will be routed to the DAI_CLK4 . IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings.
19:15 (R/W)	IN3	Input Clock 3. DAI_CLK4 . IN3 holds the Source signal assignment that will be routed to the DAI_CLK4 . IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings.
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK4 . IN1 holds the Source signal assignment that will be routed to the DAI_CLK4 . IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings.
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK4 . IN0 holds the Source signal assignment that will be routed to the DAI_CLK4 . IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings.

Clock Routing Control Register 5

The `DAI_CLK5` register provides clock routing connections for the serial ports.

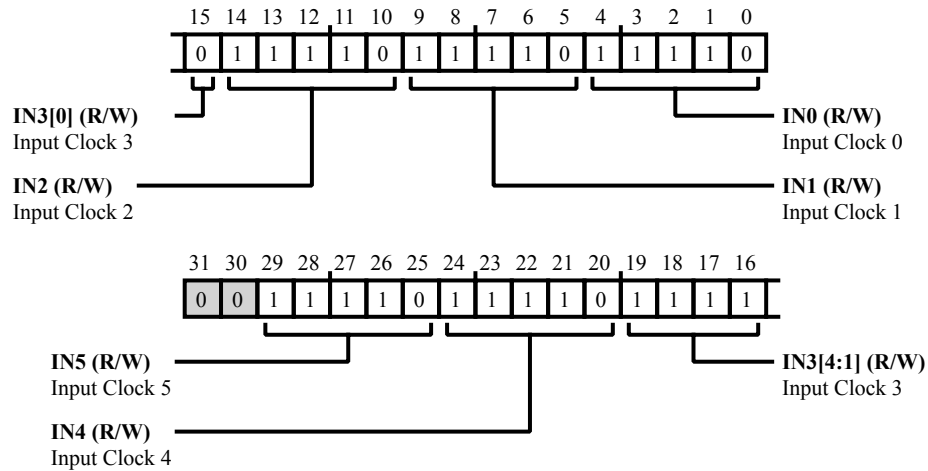


Figure 25-23: DAI_CLK5 Register Diagram

Table 25-35: DAI_CLK5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Clock 5. DAI_CLK5 . IN5 holds the Source signal assignment that will be routed to the DAI_CLK5 . IN5 Destination. Refer to the Group A Signals table for Source and Destination mappings.
24:20 (R/W)	IN4	Input Clock 4. DAI_CLK5 . IN4 holds the Source signal assignment that will be routed to the DAI_CLK5 . IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings.
19:15 (R/W)	IN3	Input Clock 3. DAI_CLK5 . IN3 holds the Source signal assignment that will be routed to the DAI_CLK5 . IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings.
14:10 (R/W)	IN2	Input Clock 2. DAI_CLK5 . IN2 holds the Source signal assignment that will be routed to the DAI_CLK5 . IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings.
9:5 (R/W)	IN1	Input Clock 1. DAI_CLK5 . IN1 holds the Source signal assignment that will be routed to the DAI_CLK5 . IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings.

Table 25-35: DAI_CLK5 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Clock 0. DAI_CLK5.IN0 holds the Source signal assignment that will be routed to the DAI_CLK5.IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings.

Serial Data Routing Control Register 0

The `DAI_DAT0` register routes serial data to the serial ports.

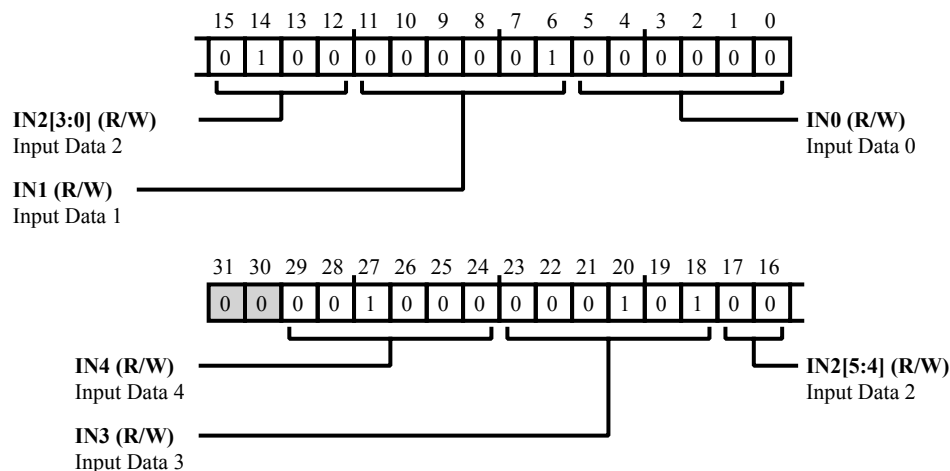


Figure 25-24: DAI_DAT0 Register Diagram

Table 25-36: DAI_DAT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT0 . IN4 holds the Source signal assignment that will be routed to the DAI_DAT0 . IN4 Destination. Refer to the Group B Signals table for Source and Destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT0 . IN3 holds the Source signal assignment that will be routed to the DAI_DAT0 . IN3 Destination. Refer to the Group B Signals table for Source and Destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT0 . IN2 holds the Source signal assignment that will be routed to the DAI_DAT0 . IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT0 . IN1 holds the Source signal assignment that will be routed to the DAI_DAT0 . IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT0 . IN0 holds the Source signal assignment that will be routed to the DAI_DAT0 . IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Serial Data Routing Control Register 1

The `DAI_DAT1` register routes serial data to the serial ports.

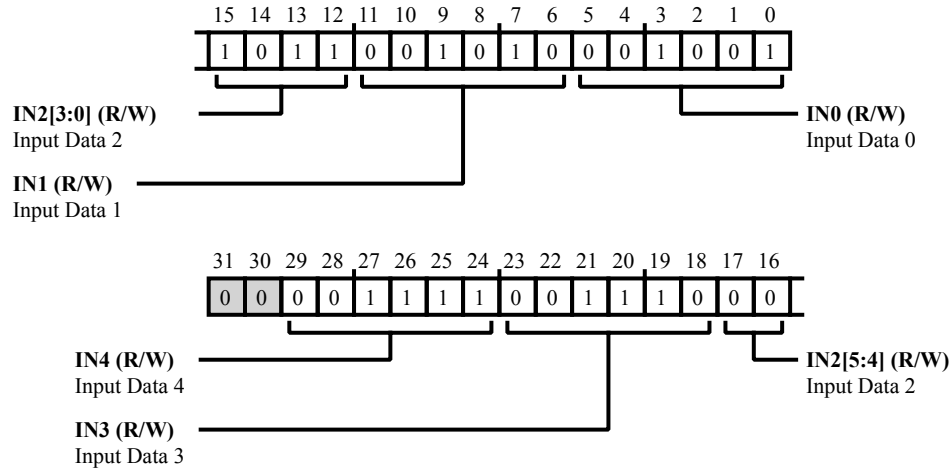


Figure 25-25: DAI_DAT1 Register Diagram

Table 25-37: DAI_DAT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT1 . IN4 holds the Source signal assignment that will be routed to the DAI_DAT1 . IN4 Destination. Refer to the Group B Signals table for Source and Destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT1 . IN3 holds the Source signal assignment that will be routed to the DAI_DAT1 . IN3 Destination. Refer to the Group B Signals table for Source and Destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT1 . IN2 holds the Source signal assignment that will be routed to the DAI_DAT1 . IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT1 . IN1 holds the Source signal assignment that will be routed to the DAI_DAT1 . IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT1 . IN0 holds the Source signal assignment that will be routed to the DAI_DAT1 . IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Serial Data Routing Control Register 2

The `DAI_DAT2` register routes serial data to the serial ports and the asynchronous sample rate converter (ASRC).

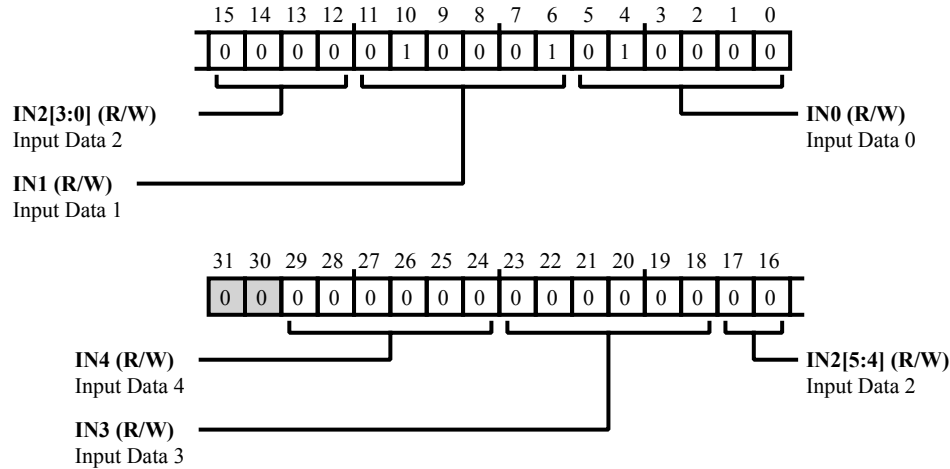


Figure 25-26: DAI_DAT2 Register Diagram

Table 25-38: DAI_DAT2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT2 . IN4 holds the Source signal assignment that will be routed to the DAI_DAT2 . IN4 Destination. Refer to the Group B Signals table for Source and Destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT2 . IN3 holds the Source signal assignment that will be routed to the DAI_DAT2 . IN3 Destination. Refer to the Group B Signals table for Source and Destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT2 . IN2 holds the Source signal assignment that will be routed to the DAI_DAT2 . IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT2 . IN1 holds the Source signal assignment that will be routed to the DAI_DAT2 . IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT2 . IN0 holds the Source signal assignment that will be routed to the DAI_DAT2 . IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Serial Data Routing Control Register 3

The `DAI_DAT3` register routes serial data to the asynchronous sample rate converter (ASRC).

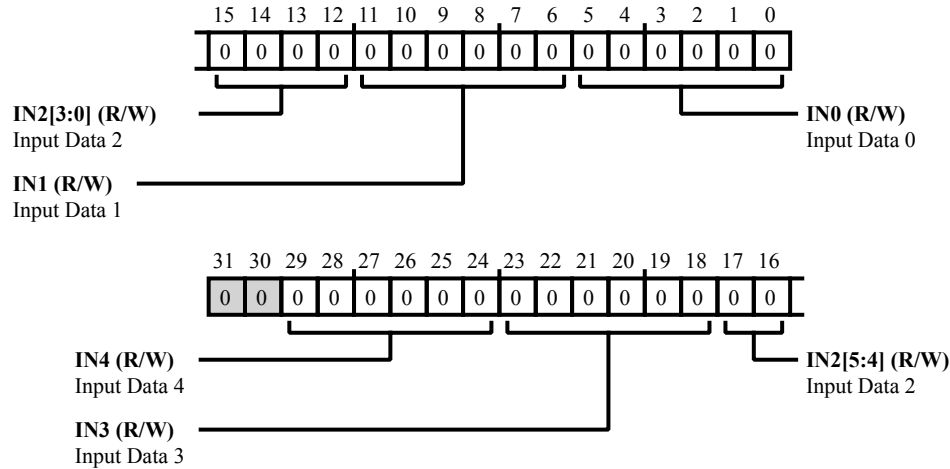


Figure 25-27: DAI_DAT3 Register Diagram

Table 25-39: DAI_DAT3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT3 . IN4 holds the Source signal assignment that will be routed to the DAI_DAT3 . IN4 Destination. Refer to the Group B Signals table for Source and Destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT3 . IN3 holds the Source signal assignment that will be routed to the DAI_DAT3 . IN3 Destination. Refer to the Group B Signals table for Source and Destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT3 . IN2 holds the Source signal assignment that will be routed to the DAI_DAT3 . IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT3 . IN1 holds the Source signal assignment that will be routed to the DAI_DAT3 . IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT3 . IN0 holds the Source signal assignment that will be routed to the DAI_DAT3 . IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Serial Data Routing Control Register 4

The `DAI_DAT4` register routes serial data to the S/PDIF.

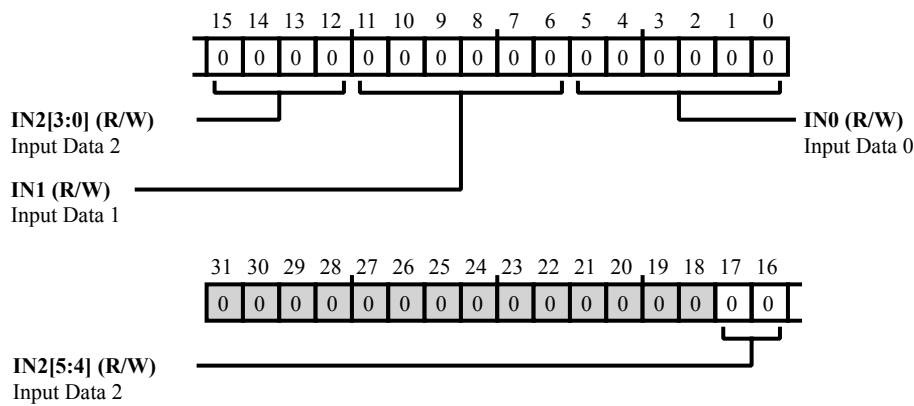


Figure 25-28: `DAI_DAT4` Register Diagram

Table 25-40: `DAI_DAT4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17:12 (R/W)	IN2	Input Data 2. <code>DAI_DAT4</code> . IN2 holds the Source signal assignment that will be routed to the <code>DAI_DAT4</code> . IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
11:6 (R/W)	IN1	Input Data 1. <code>DAI_DAT4</code> . IN1 holds the Source signal assignment that will be routed to the <code>DAI_DAT4</code> . IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5:0 (R/W)	IN0	Input Data 0. <code>DAI_DAT4</code> . IN0 holds the Source signal assignment that will be routed to the <code>DAI_DAT4</code> . IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Serial Data Routing Control Register 5

The `DAI_DAT5` register routes serial data to the S/PDIF.

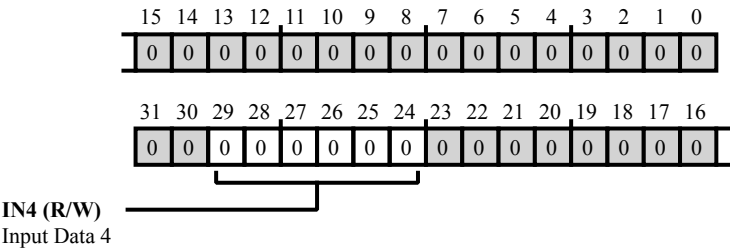


Figure 25-29: DAI_DAT5 Register Diagram

Table 25-41: DAI_DAT5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. The <code>DAI_DAT5</code> . IN4 bit field routes input data to the S/PDIF Biphase receiver stream.

Serial Data Routing Control Register 6

The `DAI_DAT6` register routes serial data to the serial ports.

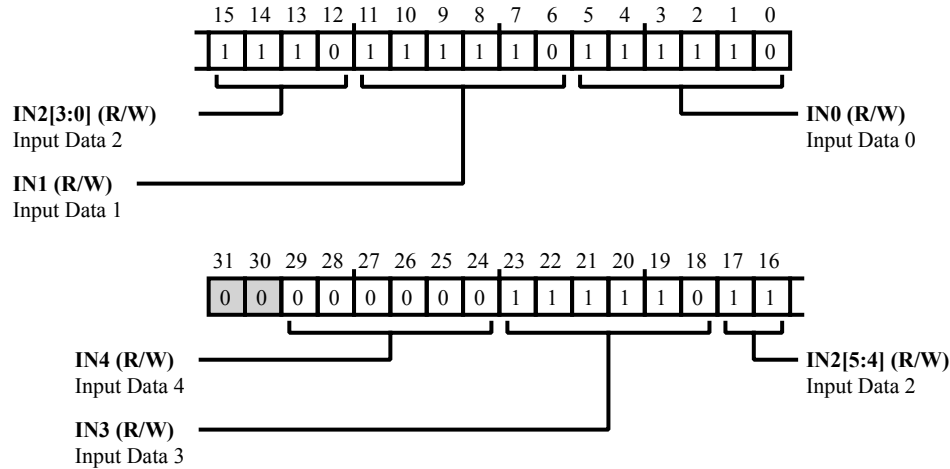


Figure 25-30: DAI_DAT6 Register Diagram

Table 25-42: DAI_DAT6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	IN4	Input Data 4. DAI_DAT6 . IN4 holds the Source signal assignment that will be routed to the DAI_DAT6 . IN4 Destination. Refer to the Group B Signals table for Source and Destination mappings.
23:18 (R/W)	IN3	Input Data 3. DAI_DAT6 . IN3 holds the Source signal assignment that will be routed to the DAI_DAT6 . IN3 Destination. Refer to the Group B Signals table for Source and Destination mappings.
17:12 (R/W)	IN2	Input Data 2. DAI_DAT6 . IN2 holds the Source signal assignment that will be routed to the DAI_DAT6 . IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
11:6 (R/W)	IN1	Input Data 1. DAI_DAT6 . IN1 holds the Source signal assignment that will be routed to the DAI_DAT6 . IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5:0 (R/W)	IN0	Input Data 0. DAI_DAT6 . IN0 holds the Source signal assignment that will be routed to the DAI_DAT6 . IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Extended Clock Routing Control Register 0

The `DAI_EXTD_CLK0` and `DAI_CLK0` registers together provides the extended clock routing connections for the serial ports (SPORTs).

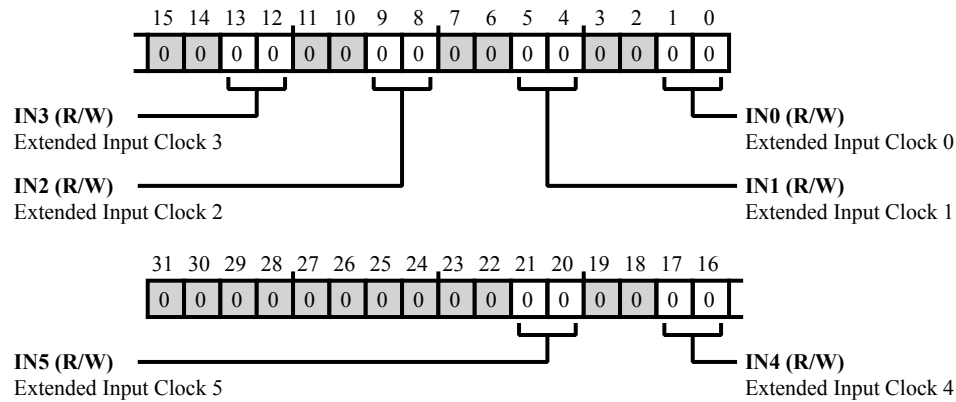


Figure 25-31: `DAI_EXTD_CLK0` Register Diagram

Table 25-43: `DAI_EXTD_CLK0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN5	Extended Input Clock 5. <code>DAI_EXTD_CLK0</code> . IN5 and <code>DAI_CLK0</code> .IN5 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK0</code> .IN5 Destination. Refer to the Group A Signals table for Source and Destination mappings
17:16 (R/W)	IN4	Extended Input Clock 4. <code>DAI_EXTD_CLK0</code> . IN4 and <code>DAI_CLK0</code> .IN4 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK0</code> .IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings
13:12 (R/W)	IN3	Extended Input Clock 3. <code>DAI_EXTD_CLK0</code> . IN3 and <code>DAI_CLK0</code> .IN3 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK0</code> .IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings
9:8 (R/W)	IN2	Extended Input Clock 2. <code>DAI_EXTD_CLK0</code> . IN2 and <code>DAI_CLK0</code> .IN2 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK0</code> .IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings
5:4 (R/W)	IN1	Extended Input Clock 1. <code>DAI_EXTD_CLK0</code> . IN1 and <code>DAI_CLK0</code> .IN1 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK0</code> .IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings

Table 25-43: DAI_EXTD_CLK0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	IN0	Extended Input Clock 0. DAI_EXTD_CLK0.IN0 and DAI_CLK0.IN0 together holds the Extended Source signal assignment that will be routed to the DAI_CLK0.IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings

Extended Clock Routing Control Register 1

The `DAI_EXTD_CLK1` and `DAI_CLK1` registers together provides the extended clock routing connections for the serial ports (SPORTs).

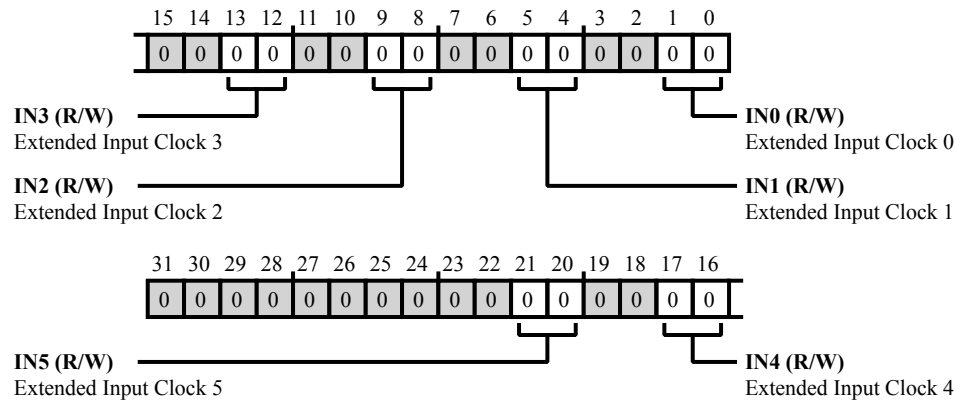


Figure 25-32: `DAI_EXTD_CLK1` Register Diagram

Table 25-44: `DAI_EXTD_CLK1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN5	Extended Input Clock 5. <code>DAI_EXTD_CLK1</code> . IN5 and <code>DAI_CLK1</code> .IN5 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK1</code> .IN5 Destination. Refer to the Group A Signals table for Source and Destination mappings
17:16 (R/W)	IN4	Extended Input Clock 4. <code>DAI_EXTD_CLK1</code> . IN4 and <code>DAI_CLK1</code> .IN4 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK1</code> .IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings
13:12 (R/W)	IN3	Extended Input Clock 3. <code>DAI_EXTD_CLK1</code> . IN3 and <code>DAI_CLK1</code> .IN3 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK1</code> .IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings
9:8 (R/W)	IN2	Extended Input Clock 2. <code>DAI_EXTD_CLK1</code> . IN2 and <code>DAI_CLK1</code> .IN2 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK1</code> .IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings
5:4 (R/W)	IN1	Extended Input Clock 1. <code>DAI_EXTD_CLK1</code> . IN1 and <code>DAI_CLK1</code> .IN1 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK1</code> .IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings

Table 25-44: DAI_EXTD_CLK1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	IN0	Extended Input Clock 0. DAI_EXTD_CLK1 . IN0 and DAI_CLK1.IN0 together holds the Extended Source signal assignment that will be routed to the DAI_CLK1.IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings

Extended Clock Routing Control Register 2

The `DAI_EXTD_CLK2` and `DAI_CLK2` registers together provides the extended clock routing connections for the serial ports (SPORTs).

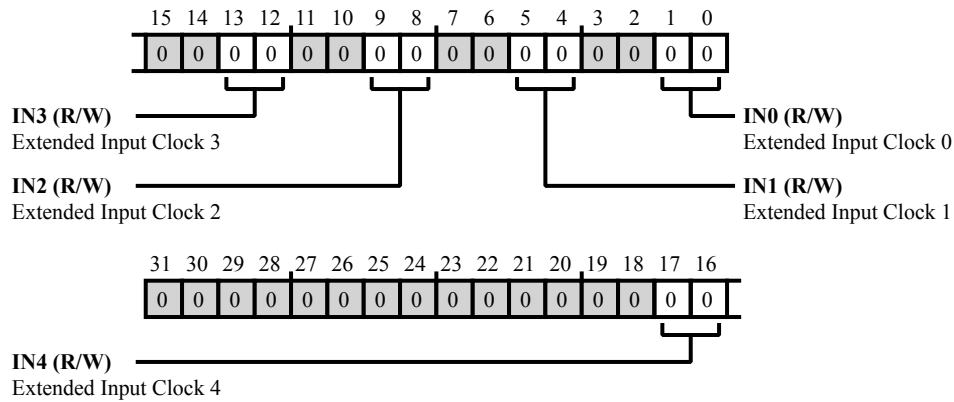


Figure 25-33: `DAI_EXTD_CLK2` Register Diagram

Table 25-45: `DAI_EXTD_CLK2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17:16 (R/W)	IN4	Extended Input Clock 4. <code>DAI_EXTD_CLK2</code> . IN4 and <code>DAI_CLK2</code> .IN4 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK2</code> .IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings
13:12 (R/W)	IN3	Extended Input Clock 3. <code>DAI_EXTD_CLK2</code> . IN3 and <code>DAI_CLK2</code> .IN3 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK2</code> .IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings
9:8 (R/W)	IN2	Extended Input Clock 2. <code>DAI_EXTD_CLK2</code> . IN2 and <code>DAI_CLK2</code> .IN2 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK2</code> .IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings
5:4 (R/W)	IN1	Extended Input Clock 1. <code>DAI_EXTD_CLK2</code> . IN1 and <code>DAI_CLK2</code> .IN1 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK2</code> .IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings
1:0 (R/W)	IN0	Extended Input Clock 0. <code>DAI_EXTD_CLK2</code> . IN0 and <code>DAI_CLK2</code> .IN0 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK2</code> .IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings

Extended Clock Routing Control Register 3

The `DAI_EXTD_CLK3` and `DAI_CLK3` registers together provides the extended clock routing connections for the serial ports (SPORTs).

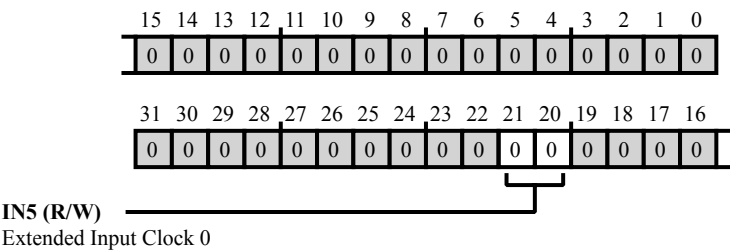


Figure 25-34: `DAI_EXTD_CLK3` Register Diagram

Table 25-46: `DAI_EXTD_CLK3` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN5	Extended Input Clock 0. <code>DAI_EXTD_CLK3</code> . IN5 and <code>DAI_CLK3</code> .IN0 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK3</code> .IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings

Extended Clock Routing Control Register 4

The `DAI_EXTD_CLK4` and `DAI_CLK4` registers together provides the extended clock routing connections for the serial ports (SPORTs).

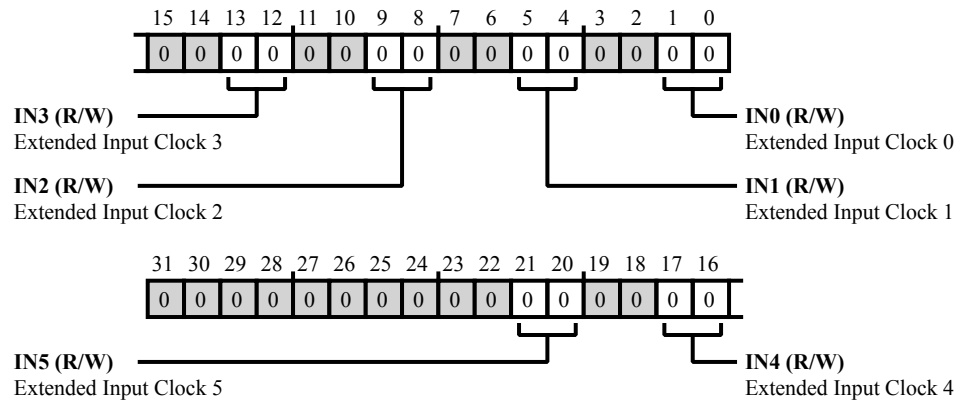


Figure 25-35: `DAI_EXTD_CLK4` Register Diagram

Table 25-47: `DAI_EXTD_CLK4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN5	Extended Input Clock 5. <code>DAI_EXTD_CLK4</code> . IN5 and <code>DAI_CLK4</code> .IN5 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK4</code> .IN5 Destination. Refer to the Group A Signals table for Source and Destination mappings
17:16 (R/W)	IN4	Extended Input Clock 4. <code>DAI_EXTD_CLK4</code> . IN4 and <code>DAI_CLK4</code> .IN4 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK4</code> .IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings
13:12 (R/W)	IN3	Extended Input Clock 3. <code>DAI_EXTD_CLK4</code> . IN3 and <code>DAI_CLK4</code> .IN3 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK4</code> .IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings
9:8 (R/W)	IN2	Extended Input Clock 2. <code>DAI_EXTD_CLK4</code> . IN2 and <code>DAI_CLK4</code> .IN2 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK4</code> .IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings
5:4 (R/W)	IN1	Extended Input Clock 1. <code>DAI_EXTD_CLK4</code> . IN1 and <code>DAI_CLK4</code> .IN1 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK4</code> .IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings

Table 25-47: DAI_EXTD_CLK4 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	IN0	Extended Input Clock 0. DAI_EXTD_CLK4 . IN0 and DAI_CLK4.IN0 together holds the Extended Source signal assignment that will be routed to the DAI_CLK4.IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings

Extended Clock Routing Control Register 5

The `DAI_EXTD_CLK5` and `DAI_CLK5` registers together provides the extended clock routing connections for the serial ports (SPORTs).

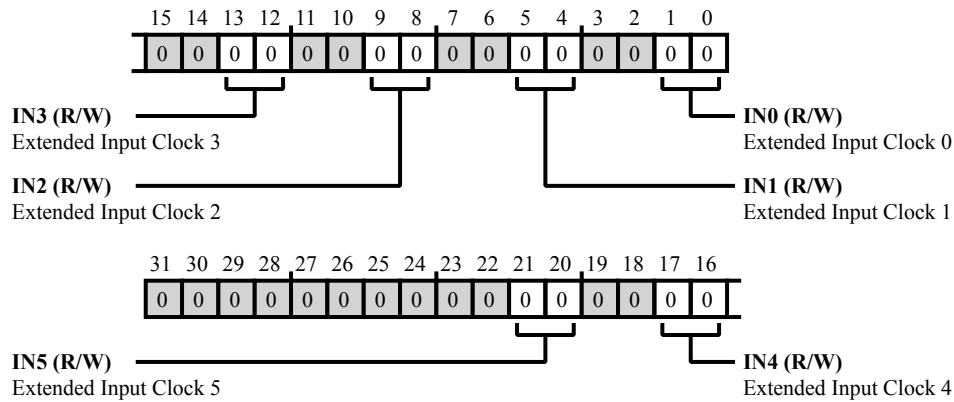


Figure 25-36: `DAI_EXTD_CLK5` Register Diagram

Table 25-48: `DAI_EXTD_CLK5` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN5	Extended Input Clock 5. <code>DAI_EXTD_CLK5</code> . IN5 and <code>DAI_CLK5</code> .IN5 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK5</code> .IN5 Destination. Refer to the Group A Signals table for Source and Destination mappings
17:16 (R/W)	IN4	Extended Input Clock 4. <code>DAI_EXTD_CLK5</code> . IN4 and <code>DAI_CLK5</code> .IN4 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK5</code> .IN4 Destination. Refer to the Group A Signals table for Source and Destination mappings
13:12 (R/W)	IN3	Extended Input Clock 3. <code>DAI_EXTD_CLK5</code> . IN3 and <code>DAI_CLK5</code> .IN3 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK5</code> .IN3 Destination. Refer to the Group A Signals table for Source and Destination mappings
9:8 (R/W)	IN2	Extended Input Clock 2. <code>DAI_EXTD_CLK5</code> . IN2 and <code>DAI_CLK5</code> .IN2 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK5</code> .IN2 Destination. Refer to the Group A Signals table for Source and Destination mappings
5:4 (R/W)	IN1	Extended Input Clock 1. <code>DAI_EXTD_CLK5</code> . IN1 and <code>DAI_CLK5</code> .IN1 together holds the Extended Source signal assignment that will be routed to the <code>DAI_CLK5</code> .IN1 Destination. Refer to the Group A Signals table for Source and Destination mappings

Table 25-48: DAI_EXTD_CLK5 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	IN0	Extended Input Clock 0. DAI_EXTD_CLK5.IN0 and DAI_CLK5.IN0 together holds the Extended Source signal assignment that will be routed to the DAI_CLK5.IN0 Destination. Refer to the Group A Signals table for Source and Destination mappings

Extended Serial Data Routing Control Register 0

The `DAI_EXTD_DAT0` and `DAI_DAT0` together provides the extended data routing connections to the serial data of serial ports.

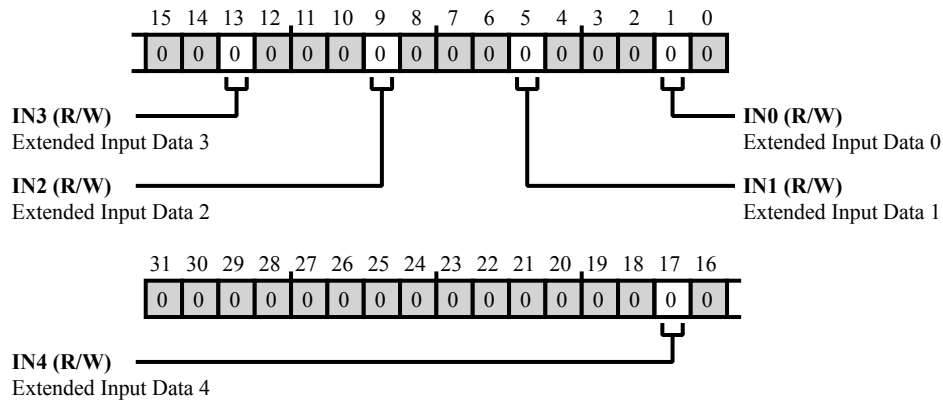


Figure 25-37: DAI_EXTD_DAT0 Register Diagram

Table 25-49: DAI_EXTD_DAT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	IN4	Extended Input Data 4. <code>DAI_EXTD_DAT0.IN4</code> and <code>DAI_DAT0.IN4</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT0.IN4</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.
13 (R/W)	IN3	Extended Input Data 3. <code>DAI_EXTD_DAT0.IN3</code> and <code>DAI_DAT0.IN3</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT0.IN3</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.
9 (R/W)	IN2	Extended Input Data 2. <code>DAI_EXTD_DAT0.IN2</code> and <code>DAI_DAT0.IN2</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT0.IN2</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.
5 (R/W)	IN1	Extended Input Data 1. <code>DAI_EXTD_DAT0.IN1</code> and <code>DAI_DAT0.IN1</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT0.IN1</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.
1 (R/W)	IN0	Extended Input Data 0. <code>DAI_EXTD_DAT0.IN0</code> and <code>DAI_DAT0.IN0</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT0.IN0</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.

Extended Serial Data Routing Control Register 1

The `DAI_EXTD_DAT1` and `DAI_DAT1` together provides the extended data routing connections to the serial data of serial ports.

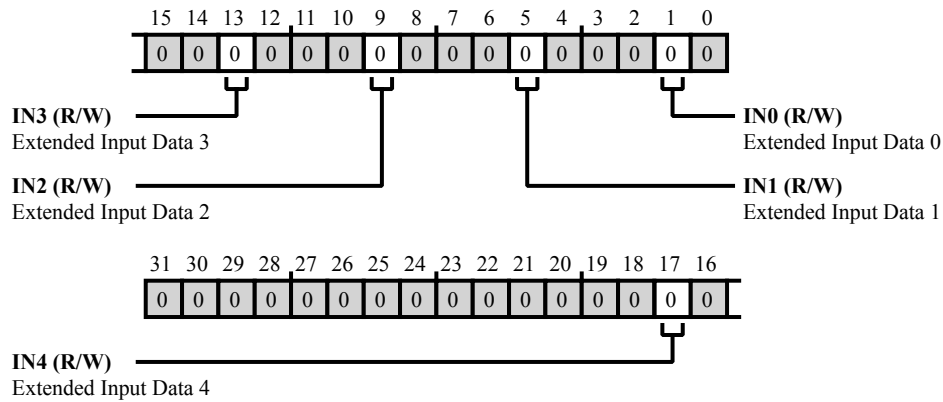


Figure 25-38: DAI_EXTD_DAT1 Register Diagram

Table 25-50: DAI_EXTD_DAT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	IN4	Extended Input Data 4. <code>DAI_EXTD_DAT1</code> . IN4 and <code>DAI_DAT1</code> .IN4 together holds the Source signal assignment that will be routed to the <code>DAI_DAT1</code> .IN4 Destination. Refer to the Group B Signals table for Source and Destination mappings.
13 (R/W)	IN3	Extended Input Data 3. <code>DAI_EXTD_DAT1</code> . IN3 and <code>DAI_DAT1</code> .IN3 together holds the Source signal assignment that will be routed to the <code>DAI_DAT1</code> .IN3 Destination. Refer to the Group B Signals table for Source and Destination mappings.
9 (R/W)	IN2	Extended Input Data 2. <code>DAI_EXTD_DAT1</code> . IN2 and <code>DAI_DAT1</code> .IN2 together holds the Source signal assignment that will be routed to the <code>DAI_DAT1</code> .IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5 (R/W)	IN1	Extended Input Data 1. <code>DAI_EXTD_DAT1</code> . IN1 and <code>DAI_DAT1</code> .IN1 together holds the Source signal assignment that will be routed to the <code>DAI_DAT1</code> .IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
1 (R/W)	IN0	Extended Input Data 0. <code>DAI_EXTD_DAT1</code> . IN0 and <code>DAI_DAT1</code> .IN0 together holds the Source signal assignment that will be routed to the <code>DAI_DAT1</code> .IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Extended Serial Data Routing Control Register 2

The `DAI_EXTD_DAT2` and `DAI_DAT2` together provides the extended data routing connections to the serial data of serial ports.

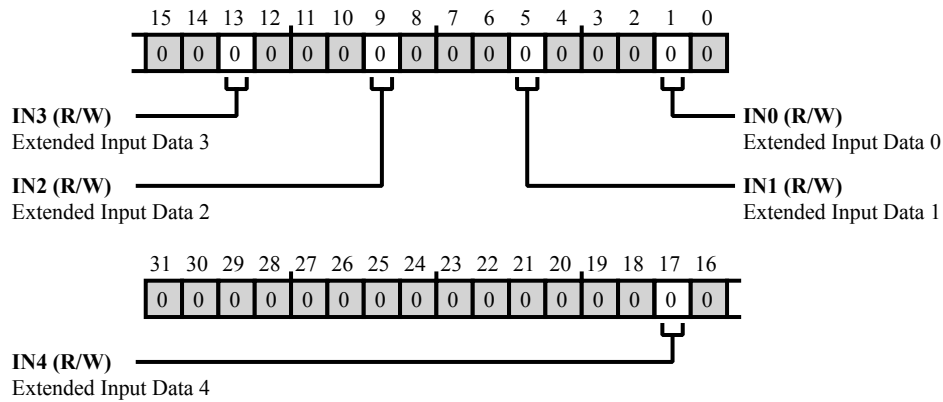


Figure 25-39: DAI_EXTD_DAT2 Register Diagram

Table 25-51: DAI_EXTD_DAT2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	IN4	Extended Input Data 4. <code>DAI_EXTD_DAT2</code> . IN4 and <code>DAI_DAT2</code> .IN4 together holds the Source signal assignment that will be routed to the <code>DAI_DAT2</code> .IN4 Destination. Refer to the Group B Signals table for Source and Destination mappings.
13 (R/W)	IN3	Extended Input Data 3. <code>DAI_EXTD_DAT2</code> . IN3 and <code>DAI_DAT2</code> .IN3 together holds the Source signal assignment that will be routed to the <code>DAI_DAT2</code> .IN3 Destination. Refer to the Group B Signals table for Source and Destination mappings.
9 (R/W)	IN2	Extended Input Data 2. <code>DAI_EXTD_DAT2</code> . IN2 and <code>DAI_DAT2</code> .IN2 together holds the Source signal assignment that will be routed to the <code>DAI_DAT2</code> .IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5 (R/W)	IN1	Extended Input Data 1. <code>DAI_EXTD_DAT2</code> . IN1 and <code>DAI_DAT2</code> .IN1 together holds the Source signal assignment that will be routed to the <code>DAI_DAT2</code> .IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
1 (R/W)	IN0	Extended Input Data 0. <code>DAI_EXTD_DAT2</code> . IN0 and <code>DAI_DAT2</code> .IN0 together holds the Source signal assignment that will be routed to the <code>DAI_DAT2</code> .IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Extended Serial Data Routing Control Register 3

The `DAI_EXTD_DAT3` and `DAI_DAT3` together provides the extended data routing connections to the serial data of serial ports.

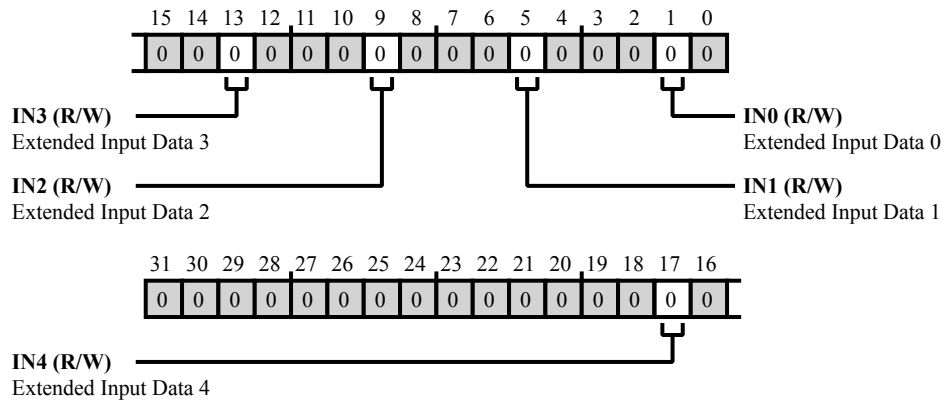


Figure 25-40: DAI_EXTD_DAT3 Register Diagram

Table 25-52: DAI_EXTD_DAT3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	IN4	Extended Input Data 4. <code>DAI_EXTD_DAT3.IN4</code> and <code>DAI_DAT3.IN4</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT3.IN4</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.
13 (R/W)	IN3	Extended Input Data 3. <code>DAI_EXTD_DAT3.IN3</code> and <code>DAI_DAT3.IN3</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT3.IN3</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.
9 (R/W)	IN2	Extended Input Data 2. <code>DAI_EXTD_DAT3.IN2</code> and <code>DAI_DAT3.IN2</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT3.IN2</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.
5 (R/W)	IN1	Extended Input Data 1. <code>DAI_EXTD_DAT3.IN1</code> and <code>DAI_DAT3.IN1</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT3.IN1</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.
1 (R/W)	IN0	Extended Input Data 0. <code>DAI_EXTD_DAT3.IN0</code> and <code>DAI_DAT3.IN0</code> together holds the Source signal assignment that will be routed to the <code>DAI_DAT3.IN0</code> Destination. Refer to the Group B Signals table for Source and Destination mappings.

Extended Serial Data Routing Control Register 4

The `DAI_EXTD_DAT4` and `DAI_DAT4` together provides the extended data routing connections to the serial data of serial ports.

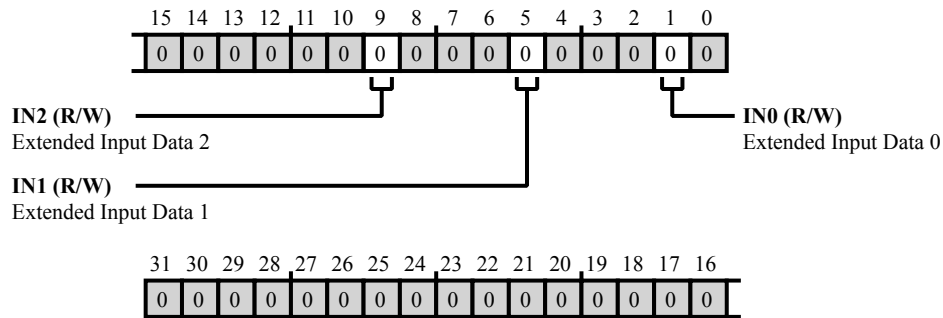


Figure 25-41: `DAI_EXTD_DAT4` Register Diagram

Table 25-53: `DAI_EXTD_DAT4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	IN2	Extended Input Data 2. DAI_EXTD_DAT4 . IN2 and DAI_DAT4.IN2 together holds the Source signal assignment that will be routed to the DAI_DAT4.IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5 (R/W)	IN1	Extended Input Data 1. DAI_EXTD_DAT4 . IN1 and DAI_DAT4.IN2 together holds the Source signal assignment that will be routed to the DAI_DAT4.IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
1 (R/W)	IN0	Extended Input Data 0. DAI_EXTD_DAT4 . IN0 and DAI_DAT4.IN0 together holds the Source signal assignment that will be routed to the DAI_DAT4.IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Extended Serial Data Routing Control Register 5

The `DAI_EXTD_DAT5` and `DAI_DAT5` together provides the extended data routing connections to the serial data of serial ports.

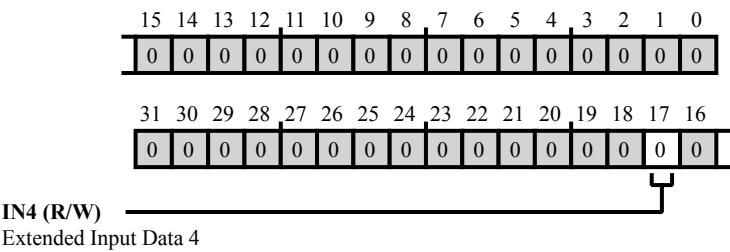


Figure 25-42: `DAI_EXTD_DAT5` Register Diagram

Table 25-54: `DAI_EXTD_DAT5` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	IN4	Extended Input Data 4. <code>DAI_EXTD_DAT5</code> . IN4 and <code>DAI_DAT5</code> .IN4 together holds the Source signal assignment that will be routed to the <code>DAI_DAT5</code> .IN4 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Extended Serial Data Routing Control Register 6

The `DAI_EXTD_DAT6` and `DAI_DAT6` together provides the extended data routing connections to the serial data of serial ports.

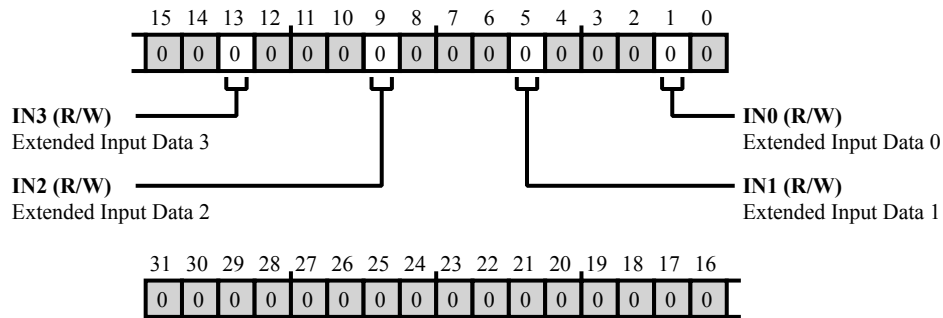


Figure 25-43: DAI_EXTD_DAT6 Register Diagram

Table 25-55: DAI_EXTD_DAT6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	IN3	Extended Input Data 3. DAI_EXTD_DAT6.IN3 and DAI_DAT6.IN3 together holds the Source signal assignment that will be routed to the DAI_DAT6.IN3 Destination. Refer to the Group B Signals table for Source and Destination mappings.
9 (R/W)	IN2	Extended Input Data 2. DAI_EXTD_DAT6.IN2 and DAI_DAT6.IN2 together holds the Source signal assignment that will be routed to the DAI_DAT6.IN2 Destination. Refer to the Group B Signals table for Source and Destination mappings.
5 (R/W)	IN1	Extended Input Data 1. DAI_EXTD_DAT6.IN1 and DAI_DAT6.IN1 together holds the Source signal assignment that will be routed to the DAI_DAT6.IN1 Destination. Refer to the Group B Signals table for Source and Destination mappings.
1 (R/W)	IN0	Extended Input Data 0. DAI_EXTD_DAT6.IN0 and DAI_DAT6.IN0 together holds the Source signal assignment that will be routed to the DAI_DAT6.IN0 Destination. Refer to the Group B Signals table for Source and Destination mappings.

Extended Frame Sync Routing Control Register 0

The `DAI_EXTD_FS0` and `DAI_FS0` together provides the extended frame sync routing connections to the frame syncs of serial ports.

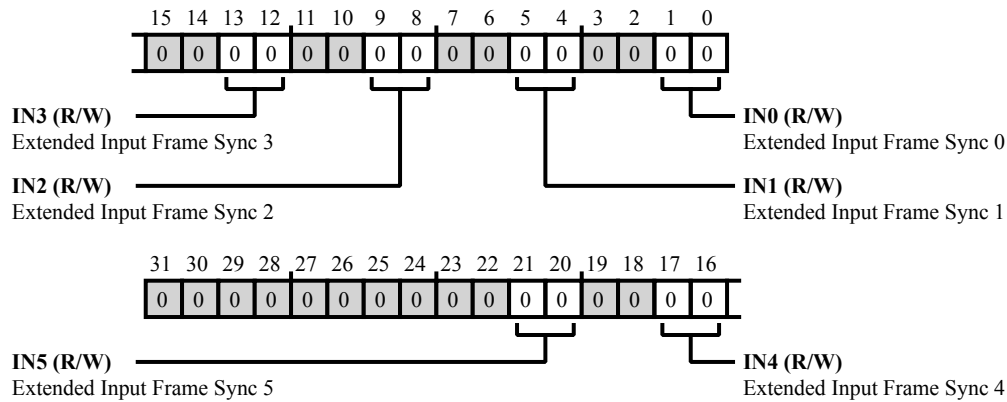


Figure 25-44: `DAI_EXTD_FS0` Register Diagram

Table 25-56: `DAI_EXTD_FS0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN5	Extended Input Frame Sync 5. <code>DAI_EXTD_FS0</code> . IN5 and <code>DAI_FS0</code> .IN5 together holds the Source signal assignment that will be routed to the <code>DAI_FS0</code> .IN5 Destination. Refer to the Group C Signals table for Source and Destination mappings.
17:16 (R/W)	IN4	Extended Input Frame Sync 4. <code>DAI_EXTD_FS0</code> . IN4 and <code>DAI_FS0</code> .IN4 together holds the Source signal assignment that will be routed to the <code>DAI_FS0</code> .IN4 Destination. Refer to the Group C Signals table for Source and Destination mappings.
13:12 (R/W)	IN3	Extended Input Frame Sync 3. <code>DAI_EXTD_FS0</code> . IN3 and <code>DAI_FS0</code> .IN3 together holds the Source signal assignment that will be routed to the <code>DAI_FS0</code> .IN3 Destination. Refer to the Group C Signals table for Source and Destination mappings.
9:8 (R/W)	IN2	Extended Input Frame Sync 2. <code>DAI_EXTD_FS0</code> . IN2 and <code>DAI_FS0</code> .IN2 together holds the Source signal assignment that will be routed to the <code>DAI_FS0</code> .IN2 Destination. Refer to the Group C Signals table for Source and Destination mappings.
5:4 (R/W)	IN1	Extended Input Frame Sync 1. <code>DAI_EXTD_FS0</code> . IN1 and <code>DAI_FS0</code> .IN1 together holds the Source signal assignment that will be routed to the <code>DAI_FS0</code> .IN1 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Table 25-56: DAI_EXTD_FS0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	IN0	Extended Input Frame Sync 0. DAI_EXTD_FS0.IN0 and DAI_FS0.IN0 together holds the Source signal assignment that will be routed to the DAI_FS0.IN0 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Extended Frame Sync Routing Control Register 1

The `DAI_EXTD_FS1` and `DAI_FS1` together provides the extended frame sync routing connections to the frame syncs of serial ports.

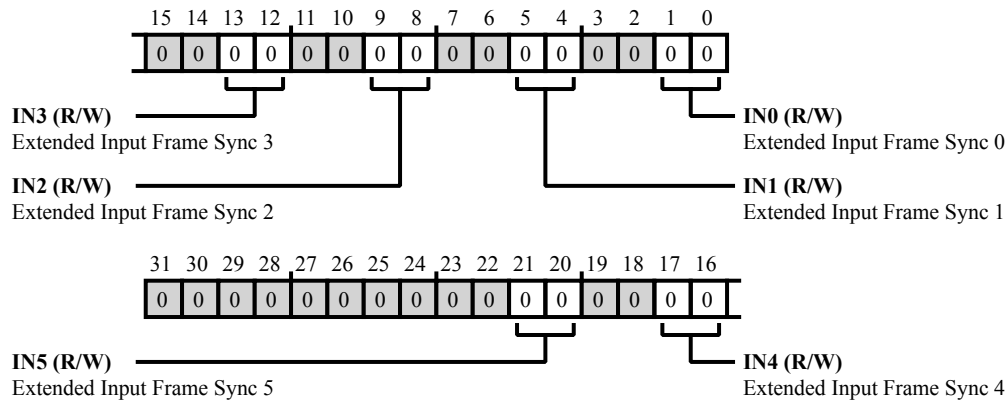


Figure 25-45: `DAI_EXTD_FS1` Register Diagram

Table 25-57: `DAI_EXTD_FS1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN5	Extended Input Frame Sync 5. <code>DAI_EXTD_FS1</code> . IN5 and <code>DAI_FS1</code> .IN5 together holds the Source signal assignment that will be routed to the <code>DAI_FS1</code> .IN5 Destination. Refer to the Group C Signals table for Source and Destination mappings.
17:16 (R/W)	IN4	Extended Input Frame Sync 4. <code>DAI_EXTD_FS1</code> . IN4 and <code>DAI_FS1</code> .IN4 together holds the Source signal assignment that will be routed to the <code>DAI_FS1</code> .IN4 Destination. Refer to the Group C Signals table for Source and Destination mappings.
13:12 (R/W)	IN3	Extended Input Frame Sync 3. <code>DAI_EXTD_FS1</code> . IN3 and <code>DAI_FS1</code> .IN3 together holds the Source signal assignment that will be routed to the <code>DAI_FS1</code> .IN3 Destination. Refer to the Group C Signals table for Source and Destination mappings.
9:8 (R/W)	IN2	Extended Input Frame Sync 2. <code>DAI_EXTD_FS1</code> . IN2 and <code>DAI_FS1</code> .IN2 together holds the Source signal assignment that will be routed to the <code>DAI_FS1</code> .IN2 Destination. Refer to the Group C Signals table for Source and Destination mappings.
5:4 (R/W)	IN1	Extended Input Frame Sync 1. <code>DAI_EXTD_FS1</code> . IN1 and <code>DAI_FS1</code> .IN1 together holds the Source signal assignment that will be routed to the <code>DAI_FS1</code> .IN1 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Table 25-57: DAI_EXTD_FS1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	IN0	Extended Input Frame Sync 0. DAI_EXTD_FS1.IN0 and DAI_FS1.IN0 together holds the Source signal assignment that will be routed to the DAI_FS1.IN0 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Extended Frame Sync Routing Control Register 2

The `DAI_EXTD_FS2` and `DAI_FS2` together provides the extended frame sync routing connections to the frame syncs of serial ports.

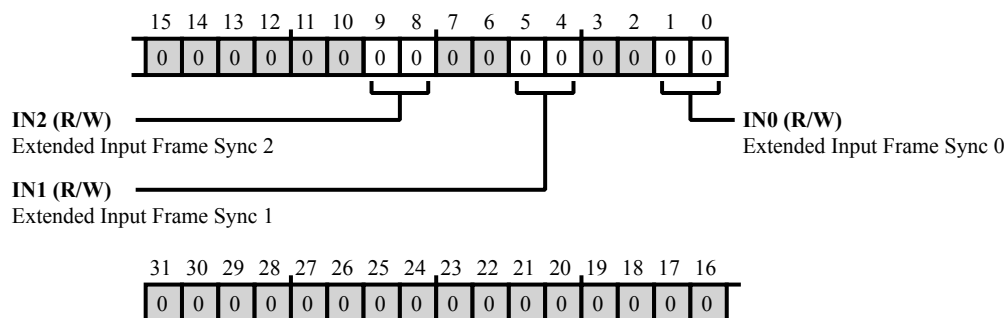


Figure 25-46: `DAI_EXTD_FS2` Register Diagram

Table 25-58: `DAI_EXTD_FS2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:8 (R/W)	IN2	Extended Input Frame Sync 2. <code>DAI_EXTD_FS2</code> . IN2 and <code>DAI_FS2</code> .IN2 together holds the Source signal assignment that will be routed to the <code>DAI_FS2</code> .IN2 Destination. Refer to the Group C Signals table for Source and Destination mappings.
5:4 (R/W)	IN1	Extended Input Frame Sync 1. <code>DAI_EXTD_FS2</code> . IN1 and <code>DAI_FS2</code> .IN1 together holds the Source signal assignment that will be routed to the <code>DAI_FS2</code> .IN1 Destination. Refer to the Group C Signals table for Source and Destination mappings.
1:0 (R/W)	IN0	Extended Input Frame Sync 0. <code>DAI_EXTD_FS2</code> . IN0 and <code>DAI_FS2</code> .IN0 together holds the Source signal assignment that will be routed to the <code>DAI_FS2</code> .IN0 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Extended Frame Sync Routing Control Register 4

The `DAI_EXTD_FS4` and `DAI_FS4` together provides the extended frame sync routing connections to the frame syncs of serial ports.

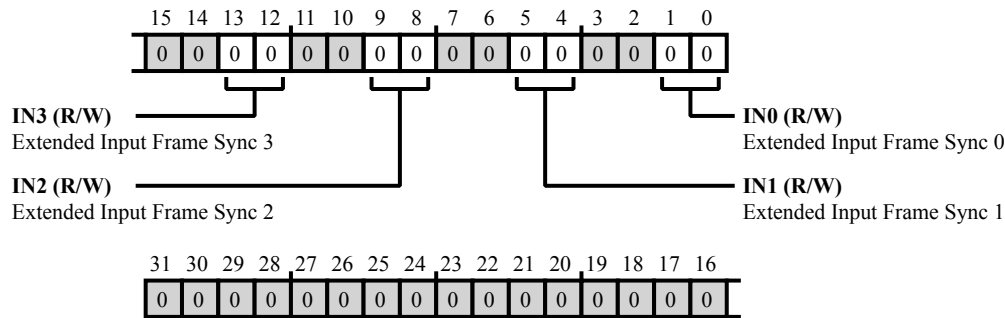


Figure 25-47: `DAI_EXTD_FS4` Register Diagram

Table 25-59: `DAI_EXTD_FS4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:12 (R/W)	IN3	Extended Input Frame Sync 3. <code>DAI_EXTD_FS4.IN3</code> and <code>DAI_FS4.IN3</code> together holds the Source signal assignment that will be routed to the <code>DAI_FS4.IN3</code> Destination. This register is used to map the frame sync to the Timer ACI inputs. Refer to the Group C Signals table for Source and Destination mappings.
9:8 (R/W)	IN2	Extended Input Frame Sync 2. <code>DAI_EXTD_FS4.IN2</code> and <code>DAI_FS4.IN2</code> together holds the Source signal assignment that will be routed to the <code>DAI_FS4.IN2</code> Destination. This register is used to map the frame sync to the Timer ACI inputs. Refer to the Group C Signals table for Source and Destination mappings.
5:4 (R/W)	IN1	Extended Input Frame Sync 1. <code>DAI_EXTD_FS4.IN1</code> and <code>DAI_FS4.IN1</code> together holds the Source signal assignment that will be routed to the <code>DAI_FS4.IN1</code> Destination. Refer to the Group C Signals table for Source and Destination mappings.
1:0 (R/W)	IN0	Extended Input Frame Sync 0. <code>DAI_EXTD_FS4.IN0</code> and <code>DAI_FS4.IN0</code> together holds the Source signal assignment that will be routed to the <code>DAI_FS4.IN0</code> Destination. Refer to the Group C Signals table for Source and Destination mappings.

Extended Miscellaneous Control Register 0

The `DAI_EXTD_MISC0` and `DAI_MISC0` register together allows programs to route to the DAI interrupt latch, PBEN input routing, or input signal inversion. This register belongs to group E which routes control signals and provides a means of connecting signals between groups.

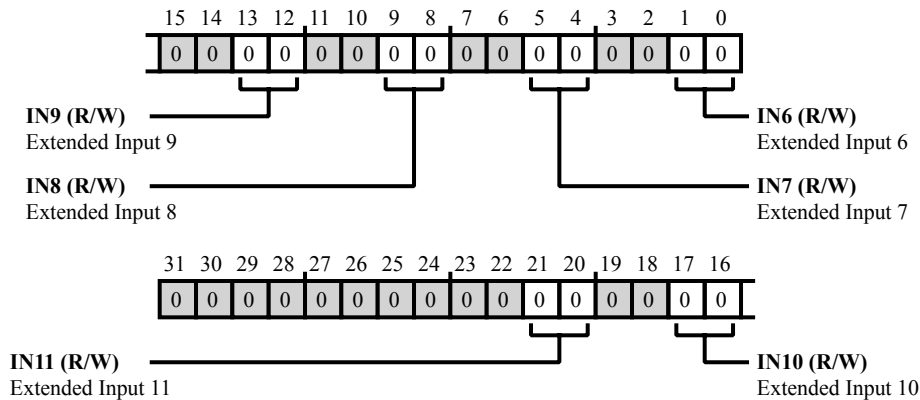


Figure 25-48: `DAI_EXTD_MISC0` Register Diagram

Table 25-60: `DAI_EXTD_MISC0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN11	Extended Input 11. The <code>DAI_EXTD_MISC0.IN11</code> bit field and <code>DAI_MISC0.IN11</code> bit field together holds the source signal assignment for miscellaneous A5 input.
17:16 (R/W)	IN10	Extended Input 10. The <code>DAI_EXTD_MISC0.IN10</code> bit field and <code>DAI_MISC0.IN10</code> bit field together holds the source signal assignment for miscellaneous A4 input.
13:12 (R/W)	IN9	Extended Input 9. The <code>DAI_EXTD_MISC0.IN9</code> bit field and <code>DAI_MISC0.IN9</code> bit field together holds the source signal assignment for miscellaneous A3 input/miscellaneous Interrupt 9(DAI interrupt 31).
9:8 (R/W)	IN8	Extended Input 8. The <code>DAI_EXTD_MISC0.IN8</code> bit field and <code>DAI_MISC0.IN8</code> bit field together holds the source signal assignment for miscellaneous A2 input/miscellaneous Interrupt 8(DAI interrupt 30).
5:4 (R/W)	IN7	Extended Input 7. The <code>DAI_EXTD_MISC0.IN7</code> bit field and <code>DAI_MISC0.IN7</code> bit field together holds the source signal assignment for miscellaneous A1 input/miscellaneous Interrupt 7(DAI interrupt 29).

Table 25-60: DAI_EXTD_MISC0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1:0 (R/W)	IN6	Extended Input 6. The DAI_EXTD_MISC0.IN6 and DAI_MISC0.IN6 bit field together holds the source signal assignment for miscellaneous A0 input/miscellaneous Interrupt 6(DAI interrupt 28).

Extended Miscellaneous Control Register 1

The `DAI_EXTD_MISC1` and `DAI_MISC1` register together allows programs to route to the DAI interrupt latch, PBEN input routing, or input signal inversion. This register belongs to group E which routes control signals and provides a means of connecting signals between groups.

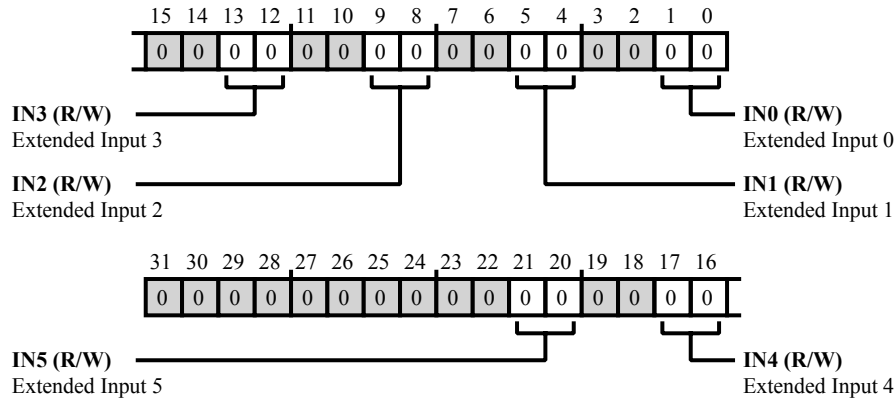


Figure 25-49: `DAI_EXTD_MISC1` Register Diagram

Table 25-61: `DAI_EXTD_MISC1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	IN5	Extended Input 5. The <code>DAI_EXTD_MISC1</code> .IN5 bit field and <code>DAI_MISC1</code> .IN5 bit field together holds the source signal assignment for miscellaneous Interrupt 5(DAI interrupt 27).
17:16 (R/W)	IN4	Extended Input 4. The <code>DAI_EXTD_MISC1</code> .IN4 bit field and <code>DAI_MISC1</code> .IN4 bit field together holds the source signal assignment for miscellaneous Interrupt 4(DAI interrupt 26).
13:12 (R/W)	IN3	Extended Input 3. The <code>DAI_EXTD_MISC1</code> .IN3 bit field and <code>DAI_MISC1</code> .IN3 bit field together holds the source signal assignment for miscellaneous Interrupt 3(DAI interrupt 25).
9:8 (R/W)	IN2	Extended Input 2. The <code>DAI_EXTD_MISC1</code> .IN2 bit field and <code>DAI_MISC1</code> .IN2 bit field together holds the source signal assignment for miscellaneous Interrupt 2(DAI interrupt 24).
5:4 (R/W)	IN1	Extended Input 1. The <code>DAI_EXTD_MISC1</code> .IN1 bit field and <code>DAI_MISC1</code> .IN1 bit field together holds the source signal assignment for miscellaneous Interrupt 1(DAI interrupt 23).
1:0 (R/W)	IN0	Extended Input 0. The <code>DAI_EXTD_MISC1</code> .IN0 bit field and <code>DAI_MISC1</code> .IN0 bit field together holds the source signal assignment for miscellaneous Interrupt 0(DAI interrupt 22).

Extended Miscellaneous Control Register 2

The `DAI_EXTD_MISC2` and `DAI_MISC2` register together allows programs to route to the DAI interrupt latch, PCGs hardware slave trigger, PBEN input routing, or input signal inversion. This register belongs to group E which routes control signals and provides a means of connecting signals between groups.

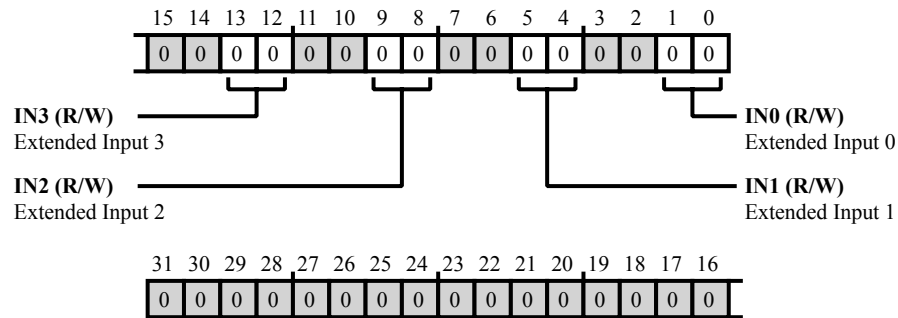


Figure 25-50: `DAI_EXTD_MISC2` Register Diagram

Table 25-62: `DAI_EXTD_MISC2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13:12 (R/W)	IN3	Extended Input 3. The <code>DAI_EXTD_MISC2</code> . IN3 bit field and <code>DAI_MISC1</code> .IN3 bit field together holds the source signal assignment for PCG-F's hardware slave trigger input.
9:8 (R/W)	IN2	Extended Input 2. The <code>DAI_EXTD_MISC2</code> . IN2 bit field and <code>DAI_MISC2</code> .IN2 bit field together holds the source signal assignment for PCG-E's hardware slave trigger input.
5:4 (R/W)	IN1	Extended Input 1. The <code>DAI_EXTD_MISC2</code> . IN1 bit field and <code>DAI_MISC2</code> .IN1 bit field together holds the source signal assignment for PCG-B's hardware slave trigger input.
1:0 (R/W)	IN0	Extended Input 0. The <code>DAI_EXTD_MISC2</code> . IN0 bit field and <code>DAI_MISC2</code> .IN0 bit field together holds the source signal assignment for PCG-A's hardware slave trigger input.

Extended Pin Buffer Enable Register 0

The `DAI_EXTD_PBEN0` and `DAI_PBEN0` register together routes extended signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

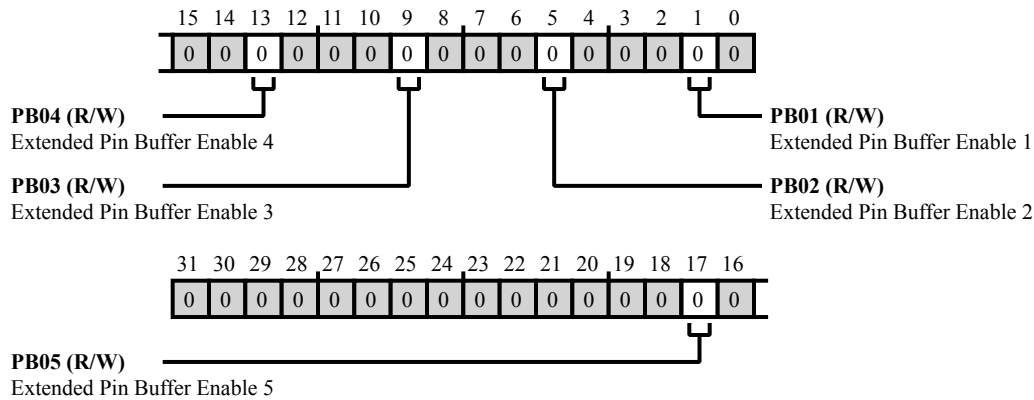


Figure 25-51: `DAI_EXTD_PBEN0` Register Diagram

Table 25-63: `DAI_EXTD_PBEN0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	PB05	Extended Pin Buffer Enable 5. The <code>DAI_EXTD_PBEN0 . PB05</code> bit field and <code>DAI_PBEN0.PB05</code> bit field together holds the extended source signal assignment that will be routed to the pin buffer enable of DAI port 5.
13 (R/W)	PB04	Extended Pin Buffer Enable 4. The <code>DAI_EXTD_PBEN0 . PB04</code> bit field and <code>DAI_PBEN0.PB04</code> bit field together holds the extended source signal assignment that will be routed to the pin buffer enable of DAI port 4.
9 (R/W)	PB03	Extended Pin Buffer Enable 3. The <code>DAI_EXTD_PBEN0 . PB03</code> bit field and <code>DAI_PBEN0.PB03</code> bit field together holds the extended source signal assignment that will be routed to the pin buffer enable of DAI port 3.
5 (R/W)	PB02	Extended Pin Buffer Enable 2. The <code>DAI_EXTD_PBEN0 . PB02</code> bit field and <code>DAI_PBEN0.PB02</code> bit field together holds the extended source signal assignment that will be routed to the pin buffer enable of DAI port 2.
1 (R/W)	PB01	Extended Pin Buffer Enable 1. The <code>DAI_EXTD_PBEN0 . PB01</code> bit field and <code>DAI_PBEN0.PB01</code> bit field together holds the extended source signal assignment that will be routed to the pin buffer enable of DAI port 1.

Extended Pin Buffer Enable Register 1

The `DAI_EXTD_PBEN1` and `DAI_PBEN1` register together routes extended signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

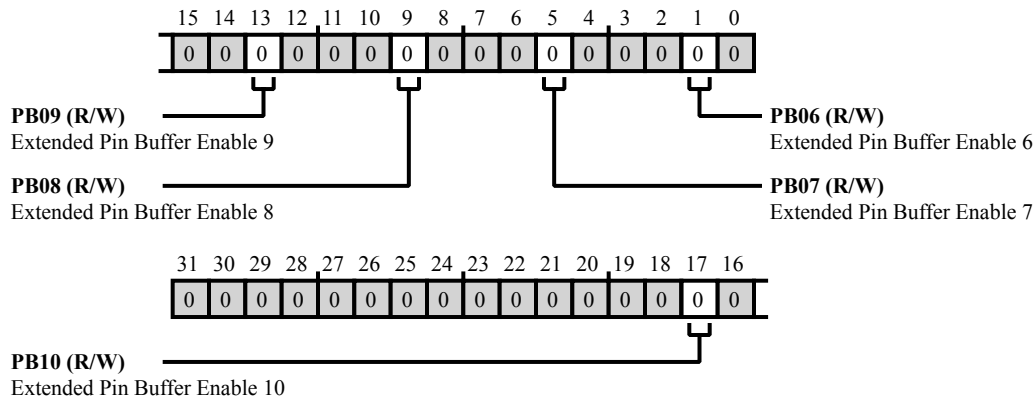


Figure 25-52: DAI_EXTD_PBEN1 Register Diagram

Table 25-64: DAI_EXTD_PBEN1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	PB10	Extended Pin Buffer Enable 10. The <code>DAI_EXTD_PBEN1 . PB10</code> bit field and <code>DAI_PBEN0.PB10</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 10.
13 (R/W)	PB09	Extended Pin Buffer Enable 9. The <code>DAI_EXTD_PBEN1 . PB09</code> bit field and <code>DAI_PBEN0.PB09</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 9.
9 (R/W)	PB08	Extended Pin Buffer Enable 8. The <code>DAI_EXTD_PBEN1 . PB08</code> bit field and <code>DAI_PBEN0.PB08</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 8.
5 (R/W)	PB07	Extended Pin Buffer Enable 7. The <code>DAI_EXTD_PBEN1 . PB07</code> bit field and <code>DAI_PBEN0.PB07</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 7.
1 (R/W)	PB06	Extended Pin Buffer Enable 6. The <code>DAI_EXTD_PBEN1 . PB06</code> bit field and <code>DAI_PBEN0.PB06</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 6.

Extended Pin Buffer Enable Register 2

The `DAI_EXTD_PBEN2` and `DAI_PBEN2` register together routes extended signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

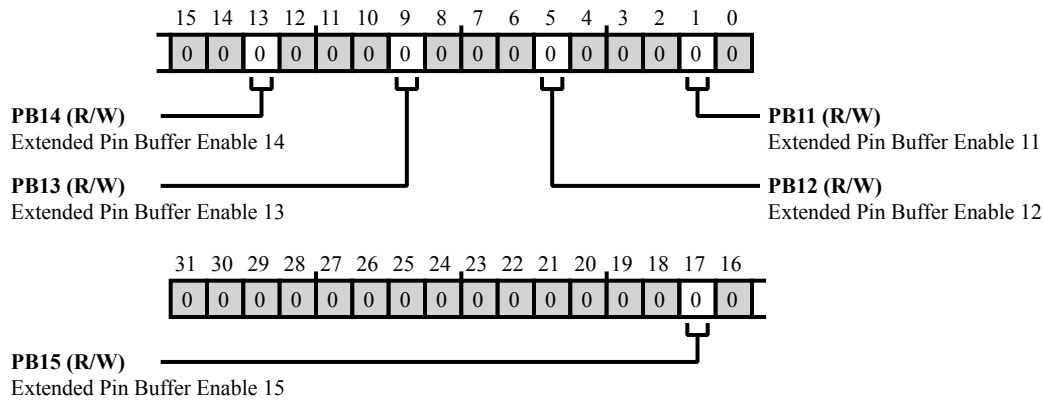


Figure 25-53: `DAI_EXTD_PBEN2` Register Diagram

Table 25-65: `DAI_EXTD_PBEN2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	PB15	Extended Pin Buffer Enable 15. The <code>DAI_EXTD_PBEN2 . PB15</code> bit field and <code>DAI_PBEN0.PB15</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 15.
13 (R/W)	PB14	Extended Pin Buffer Enable 14. The <code>DAI_EXTD_PBEN2 . PB14</code> bit field and <code>DAI_PBEN0.PB14</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 14.
9 (R/W)	PB13	Extended Pin Buffer Enable 13. The <code>DAI_EXTD_PBEN2 . PB13</code> bit field and <code>DAI_PBEN0.PB13</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 13.
5 (R/W)	PB12	Extended Pin Buffer Enable 12. The <code>DAI_EXTD_PBEN2 . PB12</code> bit field and <code>DAI_PBEN0.PB12</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 12.
1 (R/W)	PB11	Extended Pin Buffer Enable 11. The <code>DAI_EXTD_PBEN2 . PB11</code> bit field and <code>DAI_PBEN0.PB11</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 11.

Extended Pin Buffer Enable Register 3

The `DAI_EXTD_PBEN3` and `DAI_PBEN3` register together routes extended signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

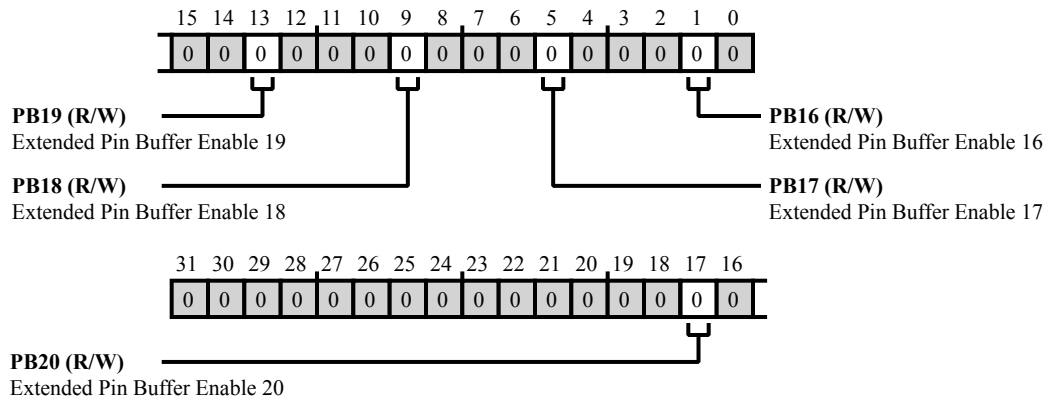


Figure 25-54: DAI_EXTD_PBEN3 Register Diagram

Table 25-66: DAI_EXTD_PBEN3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	PB20	Extended Pin Buffer Enable 20. The <code>DAI_EXTD_PBEN3 . PB20</code> bit field and <code>DAI_PBEN0.PB20</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 20.
13 (R/W)	PB19	Extended Pin Buffer Enable 19. The <code>DAI_EXTD_PBEN3 . PB19</code> bit field and <code>DAI_PBEN0.PB19</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 19.
9 (R/W)	PB18	Extended Pin Buffer Enable 18. The <code>DAI_EXTD_PBEN3 . PB18</code> bit field and <code>DAI_PBEN0.PB18</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 18.
5 (R/W)	PB17	Extended Pin Buffer Enable 17. The <code>DAI_EXTD_PBEN3 . PB17</code> bit field and <code>DAI_PBEN0.PB17</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 17.
1 (R/W)	PB16	Extended Pin Buffer Enable 16. The <code>DAI_EXTD_PBEN3 . PB16</code> bit field and <code>DAI_PBEN0.PB16</code> bit field together holds the extended source signal assignment which will be routed to the pin buffer enable of DAI port 16.

Extended Pin Buffer Assignment Register 0

The `DAI_EXTD_PIN0` and `DAI_PIN0` register routes extended physical pins that are connected to a bonded pad.

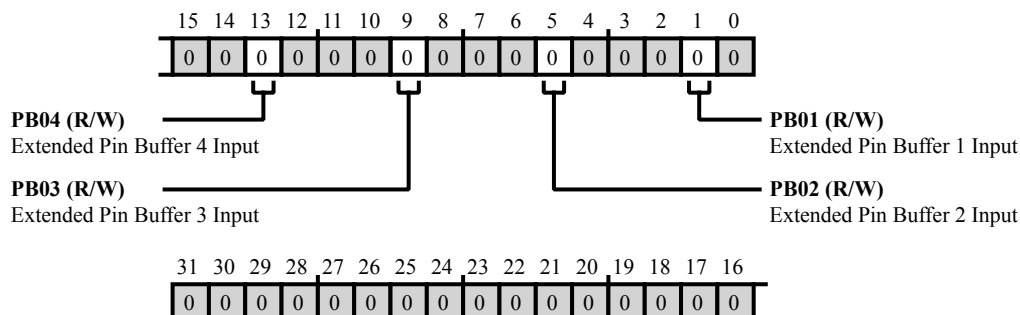


Figure 25-55: DAI_EXTD_PIN0 Register Diagram

Table 25-67: DAI_EXTD_PIN0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PB04	Extended Pin Buffer 4 Input. DAI_EXTD_PIN0.PB04 and DAI_PIN0.PB04 together holds the Source signal assignment that will be routed to the DAI_PIN0.PB04 Destination. Refer to the Group D Signals table for Source and Destination mappings.
9 (R/W)	PB03	Extended Pin Buffer 3 Input. DAI_EXTD_PIN0.PB03 and DAI_PIN.PB03 together holds the Source signal assignment that will be routed to the DAI_PIN0.PB03 Destination. Refer to the Group D Signals table for Source and Destination mappings.
5 (R/W)	PB02	Extended Pin Buffer 2 Input. DAI_EXTD_PIN0.PB02 and DAI_PIN0.PB02 together holds the Source signal assignment that will be routed to the DAI_PIN0.PB02 Destination. Refer to the Group D Signals table for Source and Destination mappings.
1 (R/W)	PB01	Extended Pin Buffer 1 Input. DAI_EXTD_PIN0.PB01 and DAI_PIN0.PB01 together holds the Source signal assignment that will be routed to the DAI_PIN0.PB01 Destination. Refer to the Group D Signals table for Source and Destination mappings.

Extended Pin Buffer Assignment Register 1

The `DAI_EXTD_PIN1` and `DAI_PIN1` register routes extended physical pins that are connected to a bonded pad.

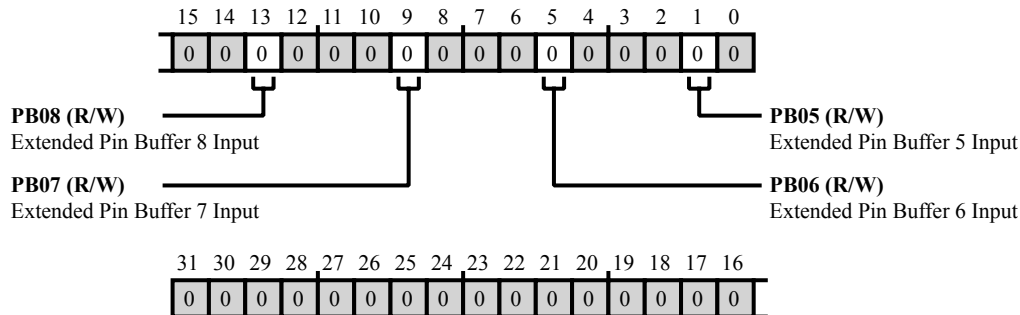


Figure 25-56: `DAI_EXTD_PIN1` Register Diagram

Table 25-68: `DAI_EXTD_PIN1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PB08	Extended Pin Buffer 8 Input. <code>DAI_EXTD_PIN1 . PB08</code> and <code>DAI_PIN1.PB08</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN1.PB08</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
9 (R/W)	PB07	Extended Pin Buffer 7 Input. <code>DAI_EXTD_PIN1 . PB07</code> and <code>DAI_PIN1.PB07</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN1.PB07</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
5 (R/W)	PB06	Extended Pin Buffer 6 Input. <code>DAI_EXTD_PIN1 . PB06</code> and <code>DAI_PIN1.PB06</code> together the Source signal assignment that will be routed to the <code>DAI_PIN1.PB06</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
1 (R/W)	PB05	Extended Pin Buffer 5 Input. <code>DAI_EXTD_PIN1 . PB05</code> and <code>DAI_PIN1.PB05</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN1.PB05</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.

Extended Pin Buffer Assignment Register 2

The `DAI_EXTD_PIN2` and `DAI_PIN2` register routes extended physical pins that are connected to a bonded pad.

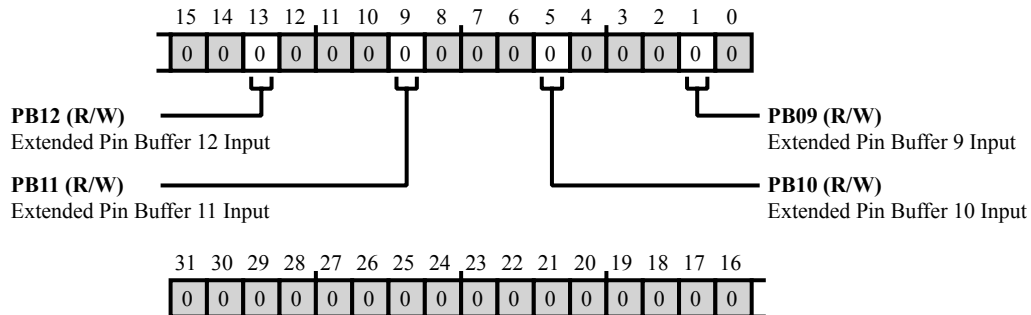


Figure 25-57: `DAI_EXTD_PIN2` Register Diagram

Table 25-69: `DAI_EXTD_PIN2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PB12	Extended Pin Buffer 12 Input. <code>DAI_EXTD_PIN2.PB12</code> and <code>DAI_PIN2.PB12</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN2.PB12</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
9 (R/W)	PB11	Extended Pin Buffer 11 Input. <code>DAI_EXTD_PIN2.PB11</code> and <code>DAI_PIN2.PB11</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN2.PB11</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
5 (R/W)	PB10	Extended Pin Buffer 10 Input. <code>DAI_EXTD_PIN2.PB10</code> and <code>DAI_PIN2.PB10</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN2.PB10</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
1 (R/W)	PB09	Extended Pin Buffer 9 Input. <code>DAI_EXTD_PIN2.PB09</code> and <code>DAI_PIN2.PB09</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN2.PB09</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.

Extended Pin Buffer Assignment Register 3

The `DAI_EXTD_PIN3` and `DAI_PIN3` register routes extended physical pins that are connected to a bonded pad.

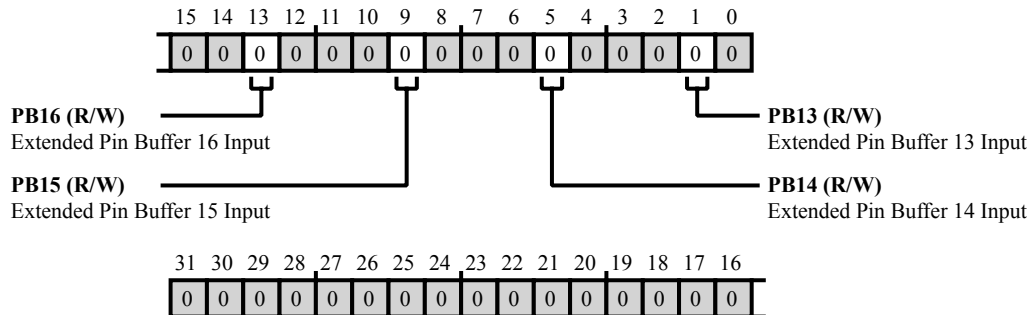


Figure 25-58: `DAI_EXTD_PIN3` Register Diagram

Table 25-70: `DAI_EXTD_PIN3` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PB16	Extended Pin Buffer 16 Input. <code>DAI_EXTD_PIN3.PB16</code> and <code>DAI_PIN3.PB16</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN3.PB16</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
9 (R/W)	PB15	Extended Pin Buffer 15 Input. <code>DAI_EXTD_PIN3.PB15</code> and <code>DAI_PIN3.PB15</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN3.PB15</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
5 (R/W)	PB14	Extended Pin Buffer 14 Input. <code>DAI_EXTD_PIN3.PB14</code> and <code>DAI_PIN3.PB14</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN3.PB14</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
1 (R/W)	PB13	Extended Pin Buffer 13 Input. <code>DAI_EXTD_PIN3.PB13</code> and <code>DAI_PIN3.PB13</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN3.PB13</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.

Extended Pin Buffer Assignment Register 4

The `DAI_EXTD_PIN4` and `DAI_PIN4` register routes extended physical pins that are connected to a bonded pad.

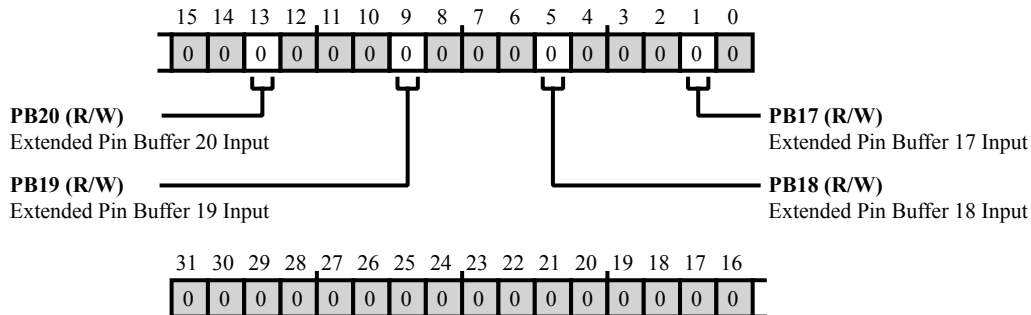


Figure 25-59: `DAI_EXTD_PIN4` Register Diagram

Table 25-71: `DAI_EXTD_PIN4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	PB20	Extended Pin Buffer 20 Input. <code>DAI_EXTD_PIN4.PB20</code> and <code>DAI_PIN4.PB20</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN4.PB20</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
9 (R/W)	PB19	Extended Pin Buffer 19 Input. <code>DAI_EXTD_PIN4.PB19</code> and <code>DAI_PIN4.PB19</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN4.PB19</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
5 (R/W)	PB18	Extended Pin Buffer 18 Input. <code>DAI_EXTD_PIN4.PB18</code> and <code>DAI_PIN4.PB18</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN4.PB18</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
1 (R/W)	PB17	Extended Pin Buffer 17 Input. <code>DAI_EXTD_PIN4.PB17</code> and <code>DAI_PIN4.PB17</code> together holds the Source signal assignment that will be routed to the <code>DAI_PIN4.PB17</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.

Frame Sync Routing Control Register 0

The `DAI_FS0` register routes frame syncs to the serial ports.

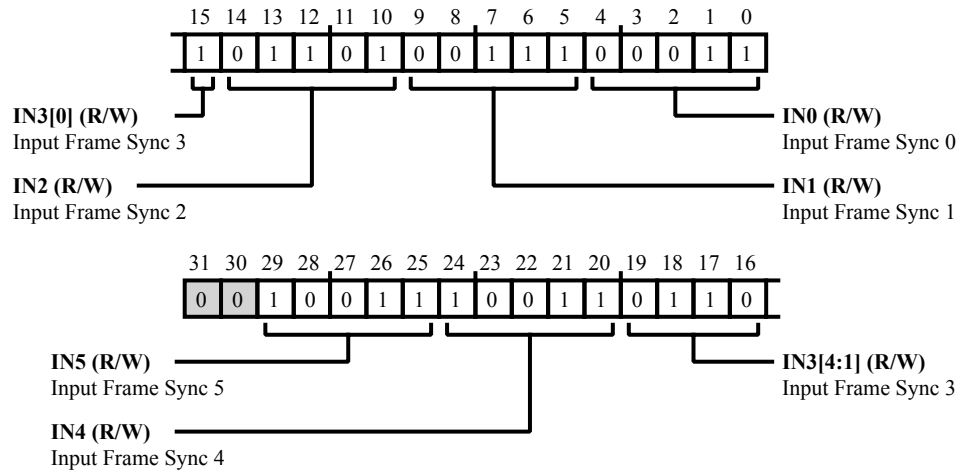


Figure 25-60: DAI_FS0 Register Diagram

Table 25-72: DAI_FS0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Frame Sync 5. DAI_FS0 . IN5 holds the Source signal assignment that will be routed to the DAI_FS0 . IN5 Destination. Refer to the Group C Signals table for Source and Destination mappings.
24:20 (R/W)	IN4	Input Frame Sync 4. DAI_FS0 . IN4 holds the Source signal assignment that will be routed to the DAI_FS0 . IN4 Destination. Refer to the Group C Signals table for Source and Destination mappings.
19:15 (R/W)	IN3	Input Frame Sync 3. DAI_FS0 . IN3 holds the Source signal assignment that will be routed to the DAI_FS0 . IN3 Destination. Refer to the Group C Signals table for Source and Destination mappings.
14:10 (R/W)	IN2	Input Frame Sync 2. DAI_FS0 . IN2 holds the Source signal assignment that will be routed to the DAI_FS0 . IN2 Destination. Refer to the Group C Signals table for Source and Destination mappings.
9:5 (R/W)	IN1	Input Frame Sync 1. DAI_FS0 . IN1 holds the Source signal assignment that will be routed to the DAI_FS0 . IN1 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Table 25-72: DAI_FS0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Frame Sync 0. DAI_FS0.IN0 holds the Source signal assignment that will be routed to the DAI_FS0.IN0 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Frame Sync Routing Control Register 1

The `DAI_FS1` register routes frame syncs to the asynchronous sample rate converter (ASRC).

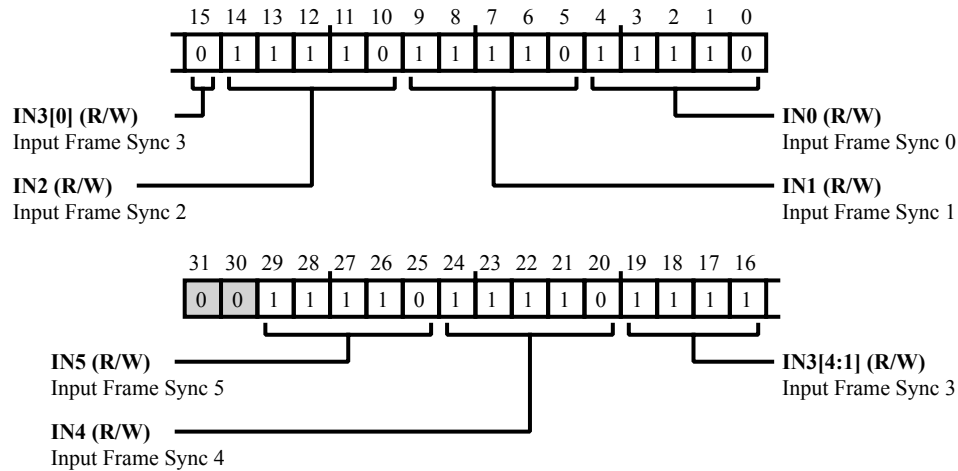


Figure 25-61: DAI_FS1 Register Diagram

Table 25-73: DAI_FS1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input Frame Sync 5. DAI_FS1 . IN5 holds the Source signal assignment that will be routed to the DAI_FS1 . IN5 Destination. Refer to the Group C Signals table for Source and Destination mappings.
24:20 (R/W)	IN4	Input Frame Sync 4. DAI_FS1 . IN4 holds the Source signal assignment that will be routed to the DAI_FS1 . IN4 Destination. Refer to the Group C Signals table for Source and Destination mappings.
19:15 (R/W)	IN3	Input Frame Sync 3. DAI_FS1 . IN3 holds the Source signal assignment that will be routed to the DAI_FS1 . IN3 Destination. Refer to the Group C Signals table for Source and Destination mappings.
14:10 (R/W)	IN2	Input Frame Sync 2. DAI_FS1 . IN2 holds the Source signal assignment that will be routed to the DAI_FS1 . IN2 Destination. Refer to the Group C Signals table for Source and Destination mappings.
9:5 (R/W)	IN1	Input Frame Sync 1. DAI_FS1 . IN1 holds the Source signal assignment that will be routed to the DAI_FS1 . IN1 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Table 25-73: DAI_FS1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input Frame Sync 0. DAI_FS1 . IN0 holds the Source signal assignment that will be routed to the DAI_FS1 . IN0 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Frame Sync Routing Control Register 2

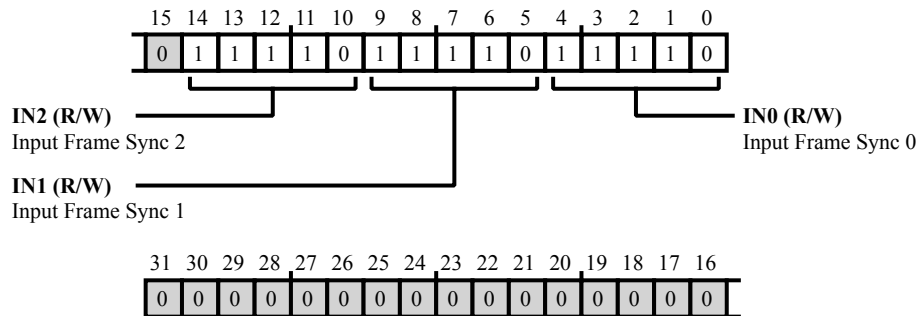


Figure 25-62: DAI_FS2 Register Diagram

Table 25-74: DAI_FS2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14:10 (R/W)	IN2	Input Frame Sync 2. DAI_FS2 . IN2 holds the Source signal assignment that will be routed to the DAI_FS2 . IN2 Destination. Refer to the Group C Signals table for Source and Destination mappings.
9:5 (R/W)	IN1	Input Frame Sync 1. The DAI_FS2 . IN1 bit field routes the frame sync output to the ASRC3 frame sync input port.
4:0 (R/W)	IN0	Input Frame Sync 0. DAI_FS2 . IN0 holds the Source signal assignment that will be routed to the DAI_FS2 . IN0 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Frame Sync Routing Control Register 4

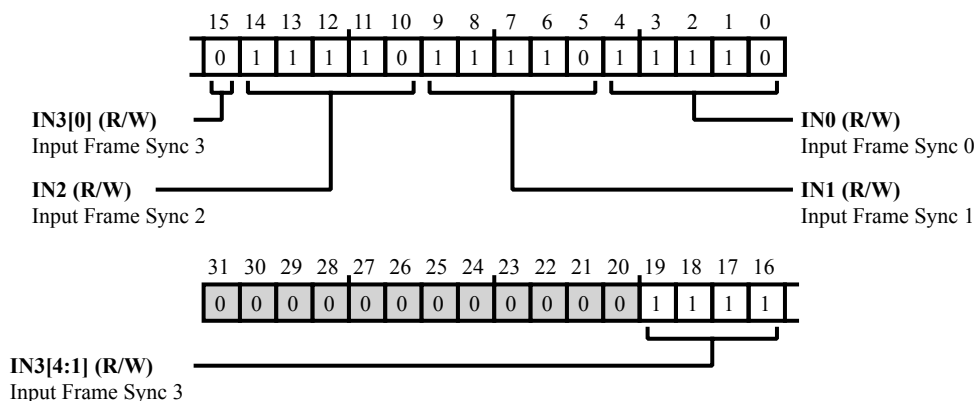


Figure 25-63: DAI_FS4 Register Diagram

Table 25-75: DAI_FS4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:15 (R/W)	IN3	Input Frame Sync 3. DAI_FS4 . IN3 holds the Source signal assignment that will be routed to the DAI_FS4 . IN3 Destination. This register is used to map the frame sync to the Timer ACI inputs. Refer to the Group C Signals table for Source and Destination mappings.
14:10 (R/W)	IN2	Input Frame Sync 2. DAI_FS4 . IN2 holds the Source signal assignment that will be routed to the DAI_FS4 . IN2 Destination. This register is used to map the frame sync to the Timer ACI inputs. Refer to the Group C Signals table for Source and Destination mappings.
9:5 (R/W)	IN1	Input Frame Sync 1. DAI_FS4 . IN1 holds the Source signal assignment that will be routed to the DAI_FS4 . IN1 Destination. Refer to the Group C Signals table for Source and Destination mappings.
4:0 (R/W)	IN0	Input Frame Sync 0. DAI_FS4 . IN0 holds the Source signal assignment that will be routed to the DAI_FS4 . IN0 Destination. Refer to the Group C Signals table for Source and Destination mappings.

Global SPORT Interrupt Grouping Register

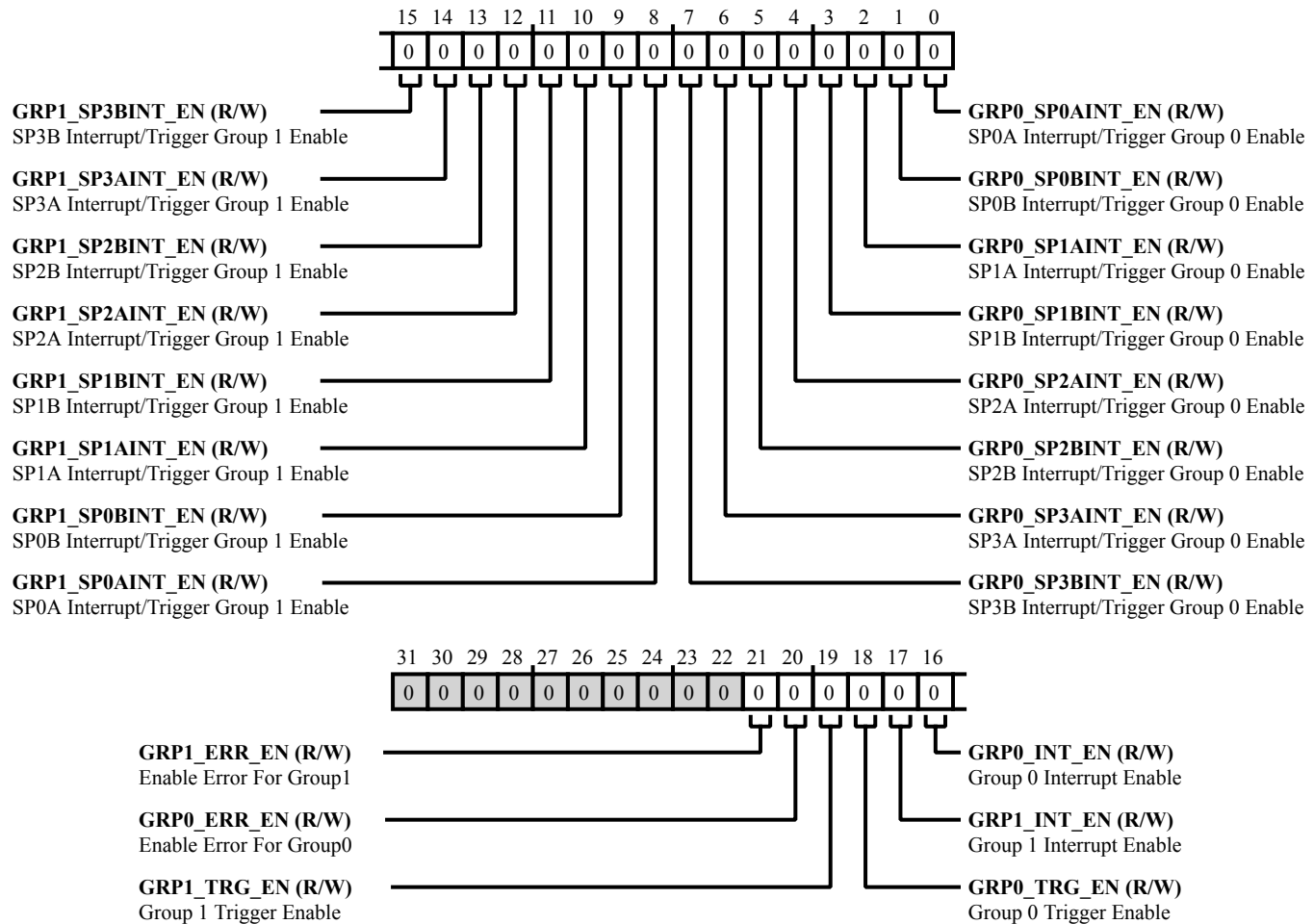


Figure 25-64: DAI_GBL_INT_EN Register Diagram

Table 25-76: DAI_GBL_INT_EN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	GRP1_ERR_EN	Enable Error For Group1.
20 (R/W)	GRP0_ERR_EN	Enable Error For Group0.
19 (R/W)	GRP1_TRG_EN	Group 1 Trigger Enable. The DAI_GBL_INT_EN.GRP1_TRG_EN bit enables the trigger-out for group 1.
18 (R/W)	GRP0_TRG_EN	Group 0 Trigger Enable. The DAI_GBL_INT_EN.GRP0_TRG_EN bit enables the trigger-out for group 0.

Table 25-76: DAI_GBL_INT_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	GRP1_INT_EN	Group 1 Interrupt Enable. The DAI_GBL_INT_EN.GRP1_INT_EN bit enables the interrupt for group 1.
16 (R/W)	GRP0_INT_EN	Group 0 Interrupt Enable. The DAI_GBL_INT_EN.GRP0_INT_EN bit enables the interrupt for group 0.
15 (R/W)	GRP1_SP3BINT_EN	SP3B Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP3BINT_EN bit enables the interrupt or trigger-out for SPORT3 B when it is part of group 1.
14 (R/W)	GRP1_SP3AINT_EN	SP3A Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP3AINT_EN bit enables the interrupt or trigger-out for SPORT3 A when it is part of group 1.
13 (R/W)	GRP1_SP2BINT_EN	SP2B Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP2BINT_EN bit enables the interrupt or trigger-out for SPORT2 B when it is part of group 1.
12 (R/W)	GRP1_SP2AINT_EN	SP2A Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP2AINT_EN bit enables the interrupt or trigger-out for SPORT2 A when it is part of group 1.
11 (R/W)	GRP1_SP1BINT_EN	SP1B Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP1BINT_EN bit enables the interrupt or trigger-out for SPORT1 B when it is part of group 1.
10 (R/W)	GRP1_SP1AINT_EN	SP1A Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP1AINT_EN bit enables the interrupt or trigger-out for SPORT1 A when it is part of group 1.
9 (R/W)	GRP1_SP0BINT_EN	SP0B Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP0BINT_EN bit enables the interrupt or trigger-out for SPORT0 B when it is part of group 1.
8 (R/W)	GRP1_SP0AINT_EN	SP0A Interrupt/Trigger Group 1 Enable. The DAI_GBL_INT_EN.GRP1_SP0AINT_EN bit enables the interrupt or trigger-out for SPORT0 A when it is part of group 1.
7 (R/W)	GRP0_SP3BINT_EN	SP3B Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP3BINT_EN bit enables the interrupt or trigger-out for SPORT3 B when it is part of group 0.
6 (R/W)	GRP0_SP3AINT_EN	SP3A Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP3AINT_EN bit enables the interrupt or trigger-out for SPORT3 A when it is part of group 0.

Table 25-76: DAI_GBL_INT_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	GRP0_SP2BINT_EN	SP2B Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP2BINT_EN bit enables the interrupt or trigger-out for SPORT2 B when it is part of group 0.
4 (R/W)	GRP0_SP2AINT_EN	SP2A Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP2AINT_EN bit enables the interrupt or trigger-out for SPORT2 A when it is part of group 0.
3 (R/W)	GRP0_SP1BINT_EN	SP1B Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP1BINT_EN bit enables the interrupt or trigger-out for SPORT1 B when it is part of group 0.
2 (R/W)	GRP0_SP1AINT_EN	SP1A Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP1AINT_EN bit enables the interrupt or trigger-out for SPORT1 A when it is part of group 0.
1 (R/W)	GRP0_SP0BINT_EN	SP0B Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP0BINT_EN bit enables the interrupt or trigger-out for SPORT0 B when it is part of group 0.
0 (R/W)	GRP0_SP0AINT_EN	SP0A Interrupt/Trigger Group 0 Enable. The DAI_GBL_INT_EN.GRP0_SP0AINT_EN bit enables the interrupt or trigger-out for SPORT0 A when it is part of group 0.

Global PCG Enable Control Register

The `DAI_GBL_PCG_EN` register bits indicate the status (signal high or low) for each pin. The individual bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

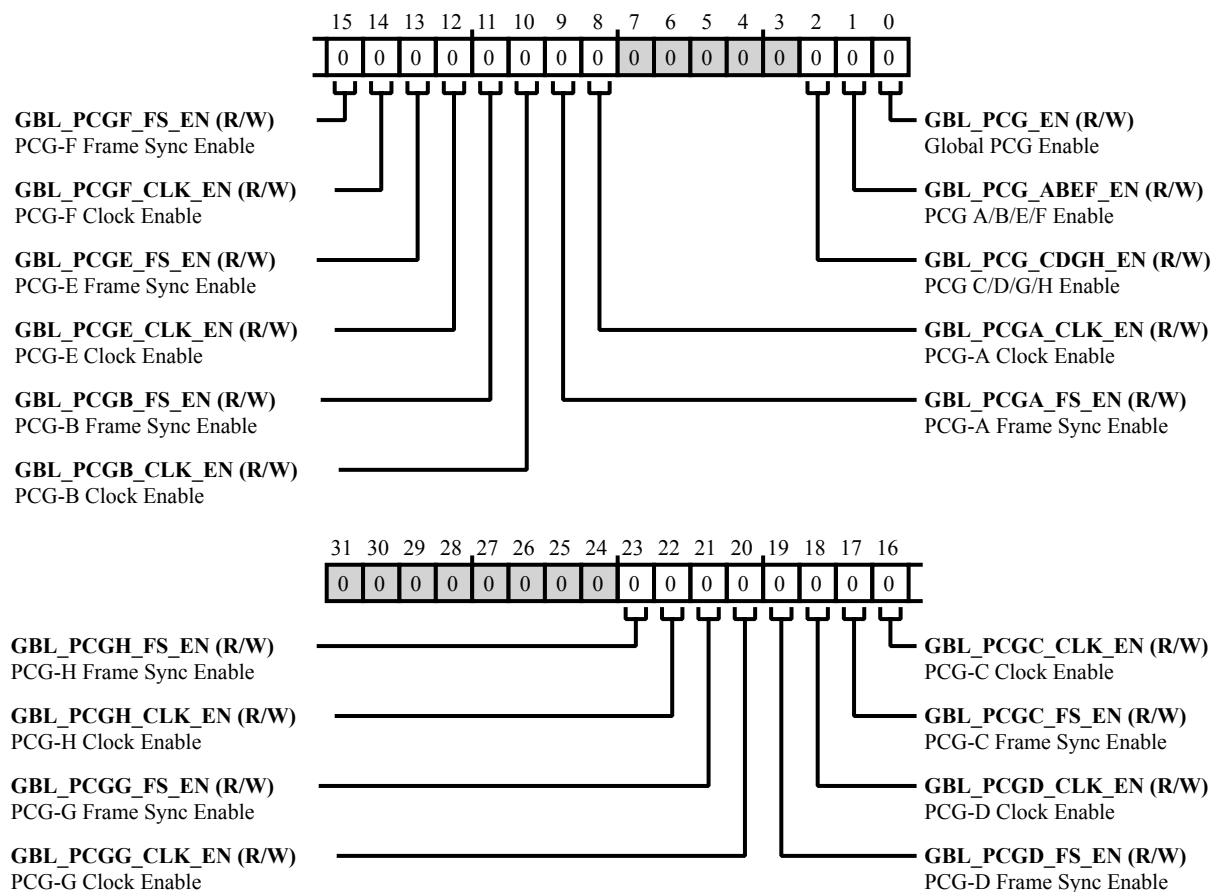


Figure 25-65: `DAI_GBL_PCG_EN` Register Diagram

Table 25-77: `DAI_GBL_PCG_EN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23 (R/W)	<code>GBL_PCGH_FS_EN</code>	PCG-H Frame Sync Enable. This bit when set to 1 enables the frame sync generation and when set to 0 disables the frame sync generation.
22 (R/W)	<code>GBL_PCGH_CLK_EN</code>	PCG-H Clock Enable. This bit when set to 1 enables the clock generation and when set to 0 disables the clock generation.

Table 25-77: DAI_GBL_PCG_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	GBL_PCGG_FS_EN	PCG-G Frame Sync Enable. This bit when set to 1 enables the frame sync generation and when set to 0 disables the frame sync generation.
20 (R/W)	GBL_PCGG_CLK_EN	PCG-G Clock Enable. This bit when set to 1 enables the clock generation and when set to 0 disables the clock generation.
19 (R/W)	GBL_PCGD_FS_EN	PCG-D Frame Sync Enable. This bit when set to 1 enables the frame sync generation and when set to 0 disables the frame sync generation.
18 (R/W)	GBL_PCGD_CLK_EN	PCG-D Clock Enable. This bit when set to 1 enables the clock generation and when set to 0 disables the clock generation.
17 (R/W)	GBL_PCGC_FS_EN	PCG-C Frame Sync Enable. This bit when set to 1 enables the frame sync generation and when set to 0 disables the frame sync generation.
16 (R/W)	GBL_PCGC_CLK_EN	PCG-C Clock Enable. This bit when set to 1 enables the clock generation and when set to 0 disables the clock generation.
15 (R/W)	GBL_PCGF_FS_EN	PCG-F Frame Sync Enable. This bit when set to 1 enables the frame sync generation and when set to 0 disables the frame sync generation.
14 (R/W)	GBL_PCGF_CLK_EN	PCG-F Clock Enable. This bit when set to 1 enables the clock generation and when set to 0 disables the clock generation.
13 (R/W)	GBL_PCGE_FS_EN	PCG-E Frame Sync Enable. This bit when set to 1 enables the frame sync generation and when set to 0 disables the frame sync generation.
12 (R/W)	GBL_PCGE_CLK_EN	PCG-E Clock Enable. This bit when set to 1 enables the clock generation and when set to 0 disables the clock generation.
11 (R/W)	GBL_PCGB_FS_EN	PCG-B Frame Sync Enable. This bit when set to 1 enables the frame sync generation and when set to 0 disables the frame sync generation.
10 (R/W)	GBL_PCGB_CLK_EN	PCG-B Clock Enable. This bit when set to 1 enables the clock generation and when set to 0 disables the clock generation.

Table 25-77: DAI_GBL_PCG_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	GBL_PCGA_FS_EN	PCG-A Frame Sync Enable. This bit when set to 1 enables the frame sync generation and when set to 0 disables the frame sync generation.
8 (R/W)	GBL_PCGA_CLK_EN	PCG-A Clock Enable. This bit when set to 1 enables the clock generation and when set to 0 disables the clock generation.
2 (R/W)	GBL_PCG_CDGH_EN	PCG C/D/G/H Enable. This bit when set to 1 along with bits [23:16] of the same register enables the respective PCGs within a DAI1. This bit when set to 0 disables the PCGs within DAI1.
1 (R/W)	GBL_PCG_ABEF_EN	PCG A/B/E/F Enable. This bit when set to 1 along with bits [15:8] of the same register enables the respective PCGs within a DAI0. This bit when set to 0 disables the PCGs within DAI0.
0 (R/W)	GBL_PCG_EN	Global PCG Enable. This bit Globally enables the PCGs across DAI0 and DAI1. This bit when set to 1 along with bit [2:1] and [23:8] enables all the PCGs across DAI0 and DAI1. This bit when set to 0 disables all the PCGs across DAI0 and DAI1.

Global SPORT Enable Register

The `DAI_GBL_SP_EN` register indicate the status (signal high or low) for each pin. The individual bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

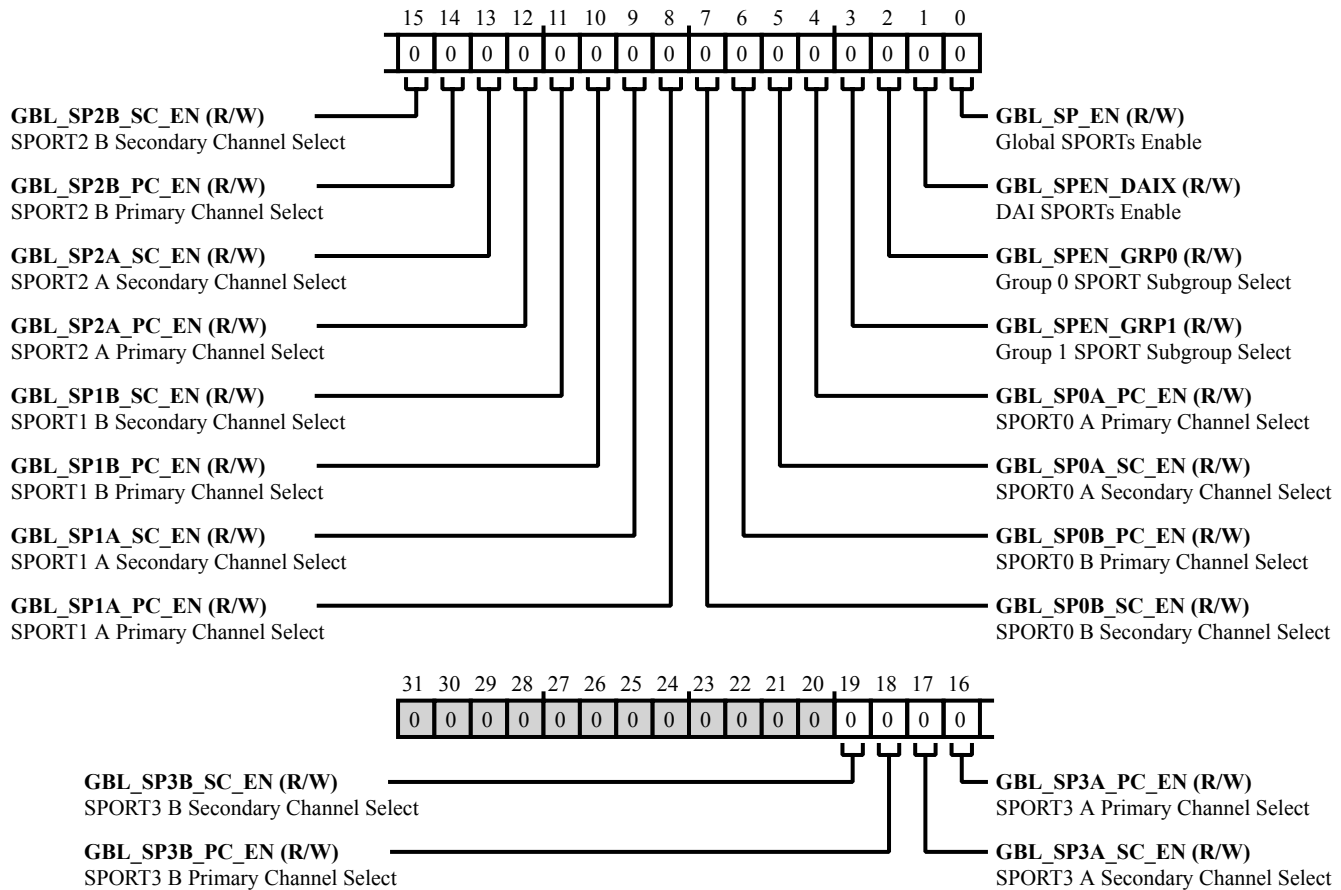


Figure 25-66: `DAI_GBL_SP_EN` Register Diagram

Table 25-78: `DAI_GBL_SP_EN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/W)	<code>GBL_SP3B_SC_EN</code>	SPORT3 B Secondary Channel Select. The <code>DAI_GBL_SP_EN.GBL_SP3B_SC_EN</code> bit selects SPORT3 B as the secondary channel for global enable.
18 (R/W)	<code>GBL_SP3B_PC_EN</code>	SPORT3 B Primary Channel Select. The <code>DAI_GBL_SP_EN.GBL_SP3B_PC_EN</code> bit selects SPORT3 B as the primary channel for global enable.

Table 25-78: DAI_GBL_SP_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	GBL_SP3A_SC_EN	SPORT3 A Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP3A_SC_EN bit selects SPORT3 A as the secondary channel for global enable.
16 (R/W)	GBL_SP3A_PC_EN	SPORT3 A Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP3A_PC_EN bit selects SPORT3 A as the primary channel for global enable.
15 (R/W)	GBL_SP2B_SC_EN	SPORT2 B Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP2B_SC_EN bit selects SPORT2 B as the secondary channel for global enable.
14 (R/W)	GBL_SP2B_PC_EN	SPORT2 B Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP2B_PC_EN bit selects SPORT2 B as the primary channel for global enable.
13 (R/W)	GBL_SP2A_SC_EN	SPORT2 A Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP2A_SC_EN bit selects SPORT2 A as the secondary channel for global enable.
12 (R/W)	GBL_SP2A_PC_EN	SPORT2 A Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP2A_PC_EN bit selects SPORT2 A as the primary channel for global enable.
11 (R/W)	GBL_SP1B_SC_EN	SPORT1 B Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP1B_SC_EN bit selects SPORT1 B as the secondary channel for global enable.
10 (R/W)	GBL_SP1B_PC_EN	SPORT1 B Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP1B_PC_EN bit selects SPORT1 B as the primary channel for global enable.
9 (R/W)	GBL_SP1A_SC_EN	SPORT1 A Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP1A_SC_EN bit selects SPORT1 A as the secondary channel for global enable.
8 (R/W)	GBL_SP1A_PC_EN	SPORT1 A Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP1A_PC_EN bit selects SPORT1 A as the primary channel for global enable.
7 (R/W)	GBL_SP0B_SC_EN	SPORT0 B Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP0B_SC_EN bit selects SPORT0 B as the secondary channel for global enable.
6 (R/W)	GBL_SP0B_PC_EN	SPORT0 B Primary Channel Select. Selects SPORT0 B primary channel for global enable.

Table 25-78: DAI_GBL_SP_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	GBL_SP0A_SC_EN	SPORT0 A Secondary Channel Select. The DAI_GBL_SP_EN.GBL_SP0A_SC_EN bit selects SPORT0 A as the secondary channel for global enable.
4 (R/W)	GBL_SP0A_PC_EN	SPORT0 A Primary Channel Select. The DAI_GBL_SP_EN.GBL_SP0A_PC_EN bit selects SPORT0 A as the primary channel for global enable.
3 (R/W)	GBL_SPEN_GRP1	Group 1 SPORT Subgroup Select. The DAI_GBL_SP_EN.GBL_SPEN_GRP1 bit creates a subgroup of selected SPORTs.
2 (R/W)	GBL_SPEN_GRP0	Group 0 SPORT Subgroup Select. The DAI_GBL_SP_EN.GBL_SPEN_GRP0 bit creates a subgroup of selected SPORTs.
1 (R/W)	GBL_SPEN_DAIX	DAI SPORTs Enable. The DAI_GBL_SP_EN.GBL_SPEN_DAIX bit enables the SPORTs selected within a DAI.
0 (R/W)	GBL_SP_EN	Global SPORTs Enable. The DAI_GBL_SP_EN.GBL_SP_EN bit enables the selected SPORTs.

Falling-Edge Interrupt Mask Register

The `DAI_IMSK_FE` register masks and unmasks interrupts generated on the falling edge of a waveform. Note that any of the Group E signals can be mapped to any of the Miscellaneous interrupts (9-0).

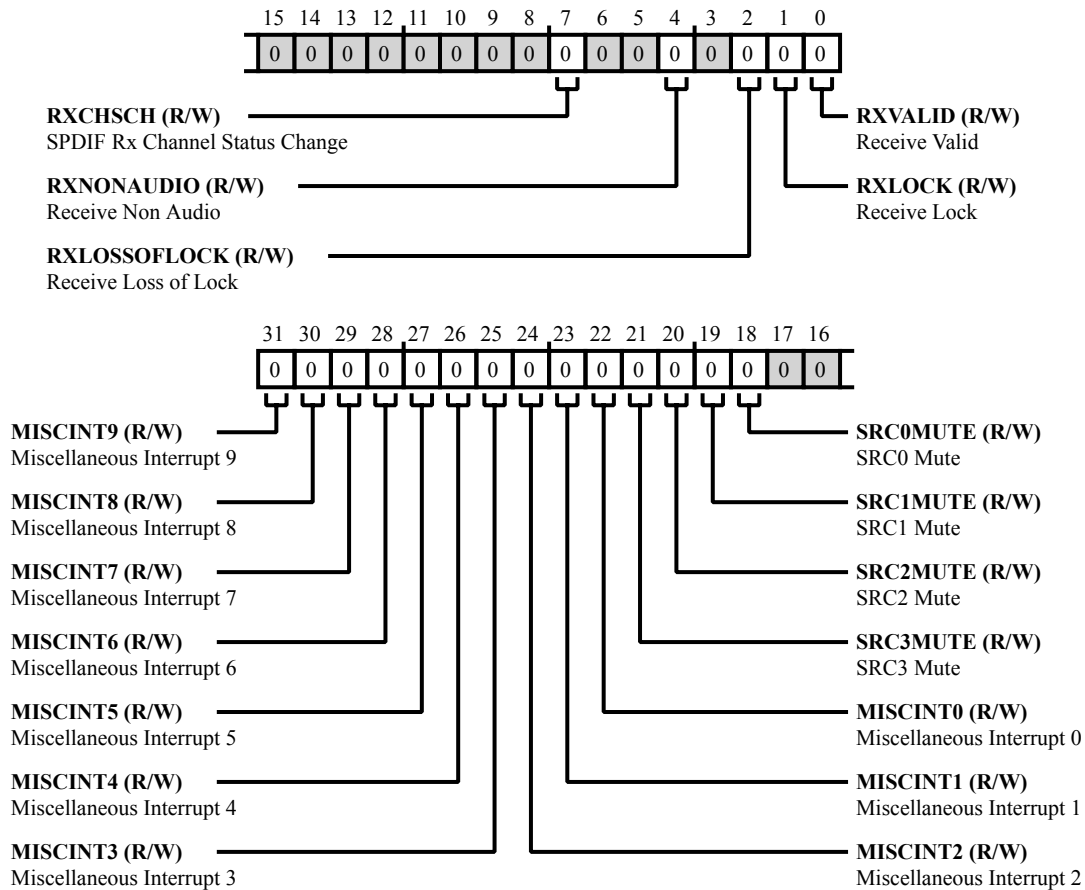


Figure 25-67: `DAI_IMSK_FE` Register Diagram

Table 25-79: `DAI_IMSK_FE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	MISCINT9	Miscellaneous Interrupt 9. Setting the <code>DAI_IMSK_FE.MISCINT9</code> bit unmasks the interrupt for the falling edge of the routed signal.
30 (R/W)	MISCINT8	Miscellaneous Interrupt 8. Setting the <code>DAI_IMSK_FE.MISCINT8</code> bit unmasks the interrupt for the falling edge of the routed signal.

Table 25-79: DAI_IMSK_FE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	MISCINT7	Miscellaneous Interrupt 7. Setting the DAI_IMSK_FE.MISCINT7 bit unmask the interrupt for the falling edge of the routed signal.
28 (R/W)	MISCINT6	Miscellaneous Interrupt 6. Setting the DAI_IMSK_FE.MISCINT6 bit unmask the interrupt for the falling edge of the routed signal.
27 (R/W)	MISCINT5	Miscellaneous Interrupt 5. Setting the DAI_IMSK_FE.MISCINT5 bit unmask the interrupt for the falling edge of the routed signal.
26 (R/W)	MISCINT4	Miscellaneous Interrupt 4. Setting the DAI_IMSK_FE.MISCINT4 bit unmask the interrupt for the falling edge of the routed signal.
25 (R/W)	MISCINT3	Miscellaneous Interrupt 3. Setting the DAI_IMSK_FE.MISCINT3 bit unmask the interrupt for the falling edge of the routed signal.
24 (R/W)	MISCINT2	Miscellaneous Interrupt 2. Setting the DAI_IMSK_FE.MISCINT2 bit unmask the interrupt for the falling edge of the routed signal.
23 (R/W)	MISCINT1	Miscellaneous Interrupt 1. Setting the DAI_IMSK_FE.MISCINT1 bit unmask the interrupt for the falling edge of the routed signal.
22 (R/W)	MISCINT0	Miscellaneous Interrupt 0. Setting the DAI_IMSK_FE.MISCINT0 bit unmask the interrupt for the falling edge of the routed signal.
21 (R/W)	SRC3MUTE	SRC3 Mute. The DAI_IMSK_FE.SRC3MUTE bit masks or unmask the corresponding SRC mute out interrupt for the falling edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
20 (R/W)	SRC2MUTE	SRC2 Mute. The DAI_IMSK_FE.SRC2MUTE bit masks or unmask the corresponding SRC mute out interrupt for the falling edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
19 (R/W)	SRC1MUTE	SRC1 Mute. The DAI_IMSK_FE.SRC1MUTE bit masks or unmask the corresponding SRC mute out interrupt for the falling edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.

Table 25-79: DAI_IMSK_FE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	SRC0MUTE	SRC0 Mute. The <code>DAI_IMSK_FE.SRC0MUTE</code> bit masks or unmasks the corresponding SRC mute out interrupt for the falling edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
7 (R/W)	RXCHSCH	SPDIF Rx Channel Status Change. The <code>DAI_IMSK_FE.RXCHSCH</code> bit masks or unmasks RXCHSCH interrupt for the falling edge of this interrupt. This interrupt is set if there is a change in the SPDIF Rx channel status bits (<code>SPDIF_RX_STAT0/1_A/B</code>).
4 (R/W)	RXNONAUDIO	Receive Non Audio. The <code>DAI_IMSK_FE.RXNONAUDIO</code> bit masks or unmasks the non audio frame mode interrupt for the falling edge of this interrupt. If the channel status indicates non-PCM audio, the NONAUDIO bit flag is set.
2 (R/W)	RXLOSSOFLOCK	Receive Loss of Lock. The <code>DAI_IMSK_FE.RXLOSSOFLOCK</code> bit masks or unmasks the emphasis loss of lock interrupt for the falling edge of this interrupt. The loss of lock status is set by the S/PDIF receiver if receiver loses the lock of biphase stream.
1 (R/W)	RXLOCK	Receive Lock. The <code>DAI_IMSK_FE.RXLOCK</code> bit masks or unmasks the S/PDIF receiver lock for the falling edge of this interrupt. This interrupt occurs when the S/PDIF receiver locks to the S/PDIF stream.
0 (R/W)	RXVALID	Receive Valid. Setting the <code>DAI_IMSK_FE.RXVALID</code> bit unmasks the interrupt for the falling edge of the routed signal. This interrupt is set based on whether data received by the S/PDIF is linear PCM or non-linear audio data.

Core Interrupt Priority Assignment Register

The `DAI_IMSK_PRI` register masks interrupts for DAI high or DAI low interrupt priority.

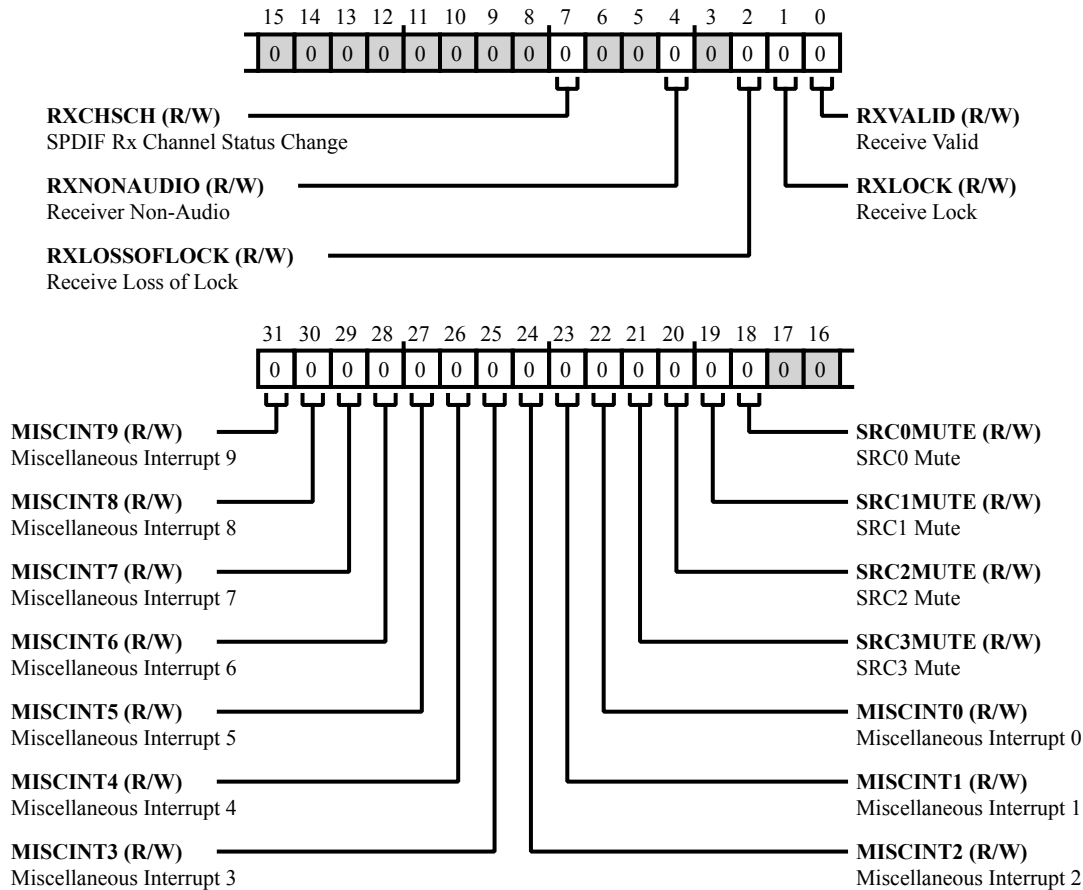


Figure 25-68: DAI_IMSK_PRI Register Diagram

Table 25-80: DAI_IMSK_PRI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	MISCINT9	Miscellaneous Interrupt 9. If the <code>DAI_IMSK_PRI.MISCINT9</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT9</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
30 (R/W)	MISCINT8	Miscellaneous Interrupt 8. If the <code>DAI_IMSK_PRI.MISCINT8</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT8</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.

Table 25-80: DAI_IMSK_PRI Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	MISCINT7	Miscellaneous Interrupt 7. If the <code>DAI_IMSK_PRI.MISCINT7</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT7</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
28 (R/W)	MISCINT6	Miscellaneous Interrupt 6. If the <code>DAI_IMSK_PRI.MISCINT6</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT6</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
27 (R/W)	MISCINT5	Miscellaneous Interrupt 5. If the <code>DAI_IMSK_PRI.MISCINT5</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT5</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
26 (R/W)	MISCINT4	Miscellaneous Interrupt 4. If the <code>DAI_IMSK_PRI.MISCINT4</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT4</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
25 (R/W)	MISCINT3	Miscellaneous Interrupt 3. If the <code>DAI_IMSK_PRI.MISCINT3</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT3</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
24 (R/W)	MISCINT2	Miscellaneous Interrupt 2. If the <code>DAI_IMSK_PRI.MISCINT2</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT2</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
23 (R/W)	MISCINT1	Miscellaneous Interrupt 1. If the <code>DAI_IMSK_PRI.MISCINT1</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT1</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
22 (R/W)	MISCINT0	Miscellaneous Interrupt 0. If the <code>DAI_IMSK_PRI.MISCINT0</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.MISCINT0</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
21 (R/W)	SRC3MUTE	SRC3 Mute. If the <code>DAI_IMSK_PRI.SRC3MUTE</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.SRC3MUTE</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.

Table 25-80: DAI_IMSK_PRI Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	SRC2MUTE	SRC2 Mute. If the <code>DAI_IMSK_PRI.SRC2MUTE</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.SRC2MUTE</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
19 (R/W)	SRC1MUTE	SRC1 Mute. If the <code>DAI_IMSK_PRI.SRC1MUTE</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.SRC1MUTE</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
18 (R/W)	SRC0MUTE	SRC0 Mute. If the <code>DAI_IMSK_PRI.SRC0MUTE</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.SRC0MUTE</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
7 (R/W)	RXCHSCH	SPDIF Rx Channel Status Change. If the <code>DAI_IMSK_PRI.RXCHSCH</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.RXCHSCH</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
4 (R/W)	RXNONAUDIO	Receiver Non-Audio. If the <code>DAI_IMSK_PRI.RXNONAUDIO</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.RXNONAUDIO</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
2 (R/W)	RXLOSSOFLOCK	Receive Loss of Lock. The <code>DAI_IMSK_PRI.RXLOSSOFLOCK</code> bit masks or unmasks the emphasis loss of lock interrupt for the falling edge of this interrupt. The loss of lock status is set by the S/PDIF receiver if receiver loses the lock of biphase stream.
1 (R/W)	RXLOCK	Receive Lock. If the <code>DAI_IMSK_PRI.RXLOCK</code> bit is cleared (=0), the interrupt is mapped to <code>INTR_DAI_IRQL</code> signal from the SEC. If the <code>DAI_IMSK_PRI.RXLOCK</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.
0 (R/W)	RXVALID	Receive Valid. If the <code>DAI_IMSK_PRI.RXVALID</code> bit is cleared (=0), the interrupt is mapped to the <code>INTR_DAI_IRQL</code> from the SEC. If the <code>DAI_IMSK_PRI.RXVALID</code> bit is set (=1) the interrupt is mapped to <code>INTR_DAI_IRQH</code> signal from the SEC.

Rising-Edge Interrupt Mask Register

The `DAI_IMSK_RE` register masks and unmasks interrupts generated on the rising edge of a waveform. Note that any of the Group E signals can be mapped to any of the miscellaneous interrupts (9-0).

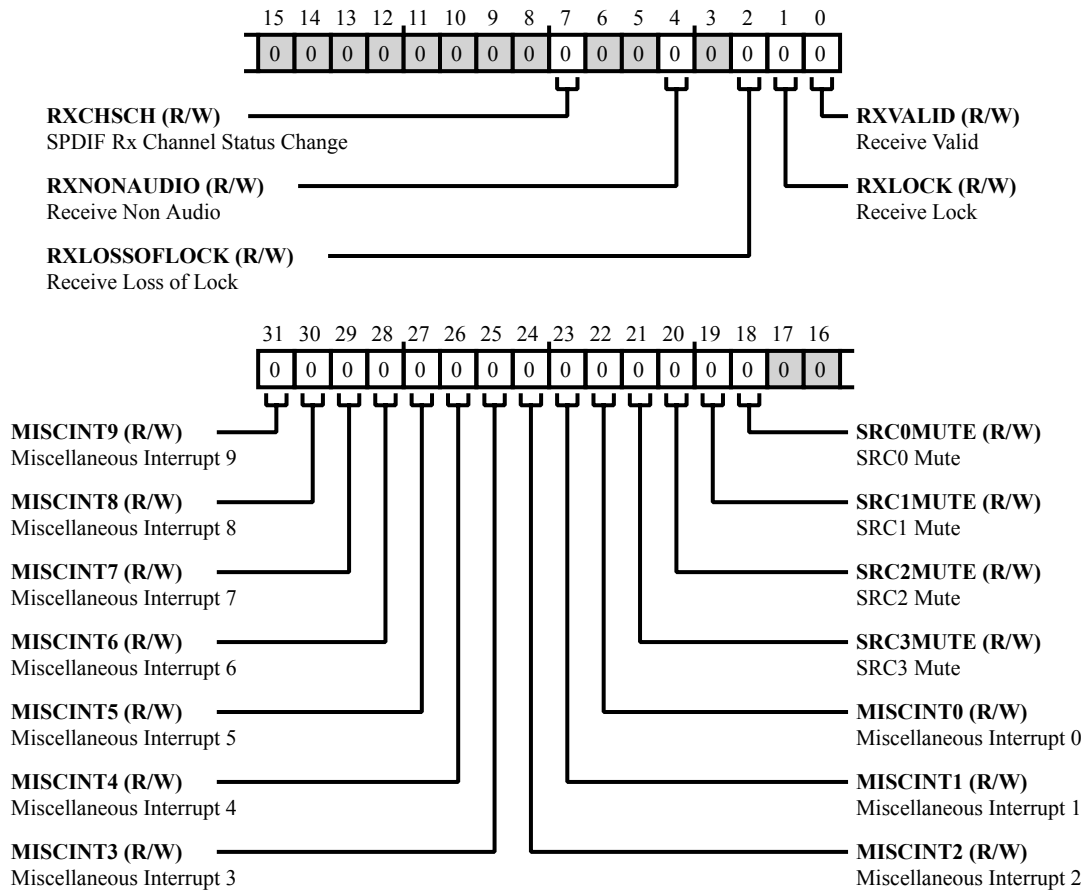


Figure 25-69: DAI_IMSK_RE Register Diagram

Table 25-81: DAI_IMSK_RE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	MISCINT9	Miscellaneous Interrupt 9. Setting the <code>DAI_IMSK_RE.MISCINT9</code> bit unmasks the interrupt for the rising edge of the routed signal.
30 (R/W)	MISCINT8	Miscellaneous Interrupt 8. Setting the <code>DAI_IMSK_RE.MISCINT8</code> bit unmasks the interrupt for the rising edge of the routed signal.

Table 25-81: DAI_IMSK_RE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	MISCINT7	Miscellaneous Interrupt 7. Setting the DAI_IMSK_RE.MISCINT7 bit unmask the interrupt for the rising edge of the routed signal.
28 (R/W)	MISCINT6	Miscellaneous Interrupt 6. Setting the DAI_IMSK_RE.MISCINT6 bit unmask the interrupt for the rising edge of the routed signal.
27 (R/W)	MISCINT5	Miscellaneous Interrupt 5. Setting the DAI_IMSK_RE.MISCINT5 bit unmask the interrupt for the rising edge of the routed signal.
26 (R/W)	MISCINT4	Miscellaneous Interrupt 4. Setting the DAI_IMSK_RE.MISCINT4 bit unmask the interrupt for the rising edge of the routed signal.
25 (R/W)	MISCINT3	Miscellaneous Interrupt 3. Setting the DAI_IMSK_RE.MISCINT3 bit unmask the interrupt for the rising edge of the routed signal.
24 (R/W)	MISCINT2	Miscellaneous Interrupt 2. Setting the DAI_IMSK_RE.MISCINT2 bit unmask the interrupt for the rising edge of the routed signal.
23 (R/W)	MISCINT1	Miscellaneous Interrupt 1. Setting the DAI_IMSK_RE.MISCINT1 bit unmask the interrupt for the rising edge of the routed signal.
22 (R/W)	MISCINT0	Miscellaneous Interrupt 0. Setting the DAI_IMSK_RE.MISCINT0 bit unmask the interrupt for the rising edge of the routed signal.
21 (R/W)	SRC3MUTE	SRC3 Mute. The DAI_IMSK_RE.SRC3MUTE bit masks or unmask the corresponding SRC mute out interrupt for the rising edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
20 (R/W)	SRC2MUTE	SRC2 Mute. The DAI_IMSK_RE.SRC2MUTE bit masks or unmask the corresponding SRC mute out interrupt for the rising edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
19 (R/W)	SRC1MUTE	SRC1 Mute. The DAI_IMSK_RE.SRC1MUTE bit masks or unmask the corresponding SRC mute out interrupt for the rising edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.

Table 25-81: DAI_IMSK_RE Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	SRC0MUTE	SRC0 Mute. The <code>DAI_IMSK_RE.SRC0MUTE</code> bit masks or unmasks the corresponding SRC mute out interrupt for the rising edge of this interrupt. This interrupt can be generated either entering mute, exiting muting or both.
7 (R/W)	RXCHSCH	SPDIF Rx Channel Status Change. The <code>DAI_IMSK_RE.RXCHSCH</code> bit masks or unmasks RXCHSCH (Status Change) interrupt for the rising edge of this interrupt. This interrupt is set if there is a change in the SPDIF Rx channel status bits (<code>SPDIF_RX_STAT0/1_A/B</code>).
4 (R/W)	RXNONAUDIO	Receive Non Audio. The <code>DAI_IMSK_RE.RXNONAUDIO</code> bit masks or unmasks the non audio frame mode interrupt for the rising edge of this interrupt. If the channel status indicates non-PCM audio, the NONAUDIO bit flag is set.
2 (R/W)	RXLOSSOFLOCK	Receive Loss of Lock. The <code>DAI_IMSK_RE.RXLOSSOFLOCK</code> bit masks or unmasks the emphasis loss of lock interrupt for the rising edge of this interrupt. The loss of lock status is set by the S/PDIF receiver if receiver loses the lock of biphas stream.
1 (R/W)	RXLOCK	Receive Lock. The <code>DAI_IMSK_RE.RXLOCK</code> bit masks or unmasks the S/PDIF receiver lock for the rising edge of this interrupt. This interrupt occurs when the S/PDIF receiver locks to the S/PDIF stream.
0 (R/W)	RXVALID	Receive Valid. Setting the <code>DAI_IMSK_RE.RXVALID</code> bit unmasks the interrupt for the rising edge of the routed signal. This interrupt is set based on whether data received by the S/PDIF is linear PCM or non-linear audio data.

High Priority Interrupt Latch Register

The `DAI_IRPTL_H` register holds the high priority latched interrupt status for interrupt requests that have been unmasked (enabled) by the `DAI_IMSK_FE`, `DAI_IMSK_RE` registers and mapped to the `INTR_DAI_IRQH` signal in the SEC by `DAI_IMSK_PRI` registers. If a bit in this register is already set and the corresponding interrupt is masked in the `DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI` registers, the latch holds its old value, leaving the interrupt asserted until its mask registers (`DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI`) are reset by software with a W1C operation.

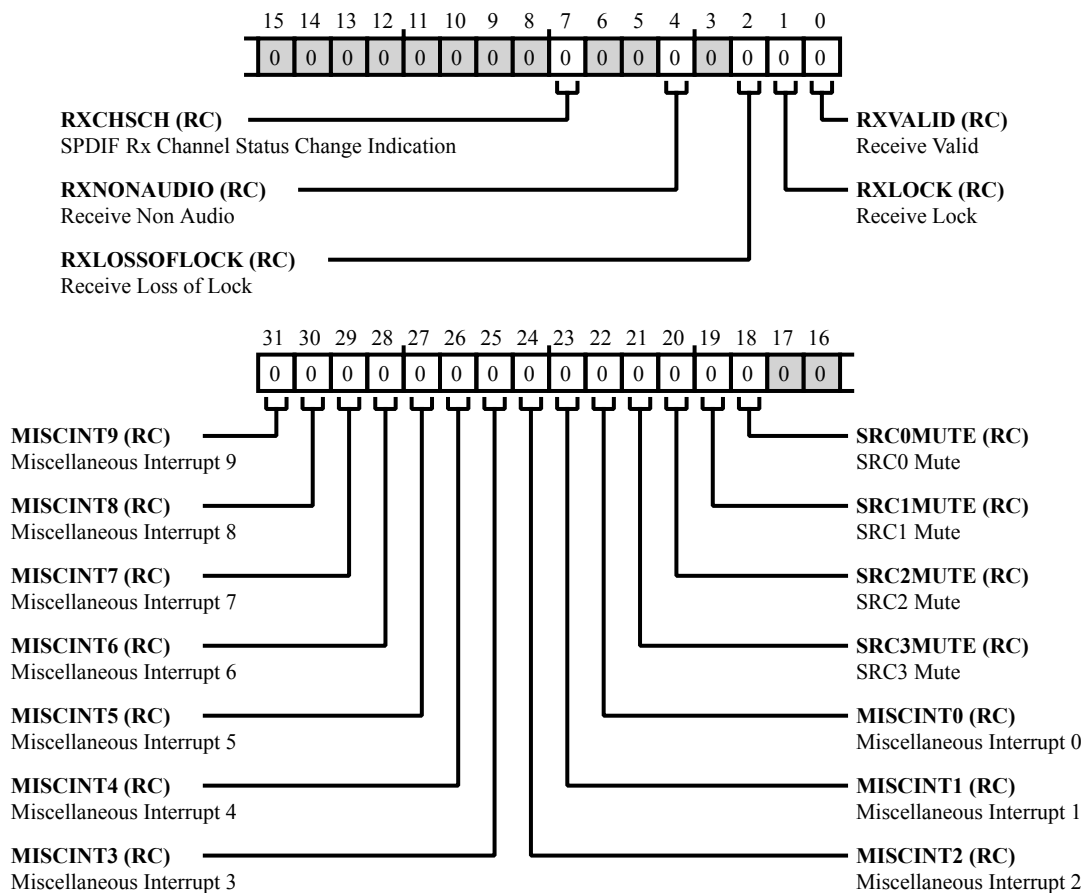


Figure 25-70: `DAI_IRPTL_H` Register Diagram

Table 25-82: `DAI_IRPTL_H` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (RC/NW)	MISCINT9	Miscellaneous Interrupt 9. The <code>DAI_IRPTL_H.MISCINT9</code> bit is set if the interrupt is mapped to the <code>INTR_DAI_IRQH</code> signal in the SEC using the <code>DAI_IMSK_PRI</code> register.

Table 25-82: DAI_IRPTL_H Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (RC/NW)	MISCINT8	Miscellaneous Interrupt 8. The DAI_IRPTL_H.MISCINT8 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
29 (RC/NW)	MISCINT7	Miscellaneous Interrupt 7. The DAI_IRPTL_H.MISCINT7 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
28 (RC/NW)	MISCINT6	Miscellaneous Interrupt 6. The DAI_IRPTL_H.MISCINT6 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
27 (RC/NW)	MISCINT5	Miscellaneous Interrupt 5. The DAI_IRPTL_H.MISCINT5 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
26 (RC/NW)	MISCINT4	Miscellaneous Interrupt 4. The DAI_IRPTL_H.MISCINT4 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
25 (RC/NW)	MISCINT3	Miscellaneous Interrupt 3. The DAI_IRPTL_H.MISCINT3 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
24 (RC/NW)	MISCINT2	Miscellaneous Interrupt 2. The DAI_IRPTL_H.MISCINT2 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
23 (RC/NW)	MISCINT1	Miscellaneous Interrupt 1. The DAI_IRPTL_H.MISCINT1 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
22 (RC/NW)	MISCINT0	Miscellaneous Interrupt 0. The DAI_IRPTL_H.MISCINT0 bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
21 (RC/NW)	SRC3MUTE	SRC3 Mute. The DAI_IRPTL_H.SRC3MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
20 (RC/NW)	SRC2MUTE	SRC2 Mute. The DAI_IRPTL_H.SRC2MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.

Table 25-82: DAI_IRPTL_H Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (RC/NW)	SRC1MUTE	SRC1 Mute. The DAI_IRPTL_H.SRC1MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
18 (RC/NW)	SRC0MUTE	SRC0 Mute. The DAI_IRPTL_H.SRC0MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
7 (RC/NW)	RXCHSCH	SPDIF Rx Channel Status Change Indication. The DAI_IRPTL_H.RXCHSCH bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
4 (RC/NW)	RXNONAUDIO	Receive Non Audio. The DAI_IRPTL_H.RXNONAUDIO bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
2 (RC/NW)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IRPTL_H.RXLOSSOFLOCK bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
1 (RC/NW)	RXLOCK	Receive Lock. The DAI_IRPTL_H.RXLOCK bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.
0 (RC/NW)	RXVALID	Receive Valid. The DAI_IRPTL_H.RXVALID bit is set if the interrupt is mapped to the INTR_DAI_IRQH signal in the SEC using the DAI_IMSK_PRI register.

Shadow High Priority Interrupt Latch Register

The `DAI_IRPTL_HS` register is the shadow register of the `DAI_IRPTL_H` register. Its content is the same as the `DAI_IRPTL_H` register. Reading the `DAI_IRPTL_HS` register does not affect its contents while reading the contents of the `DAI_IRPTL_H` registers clears it.

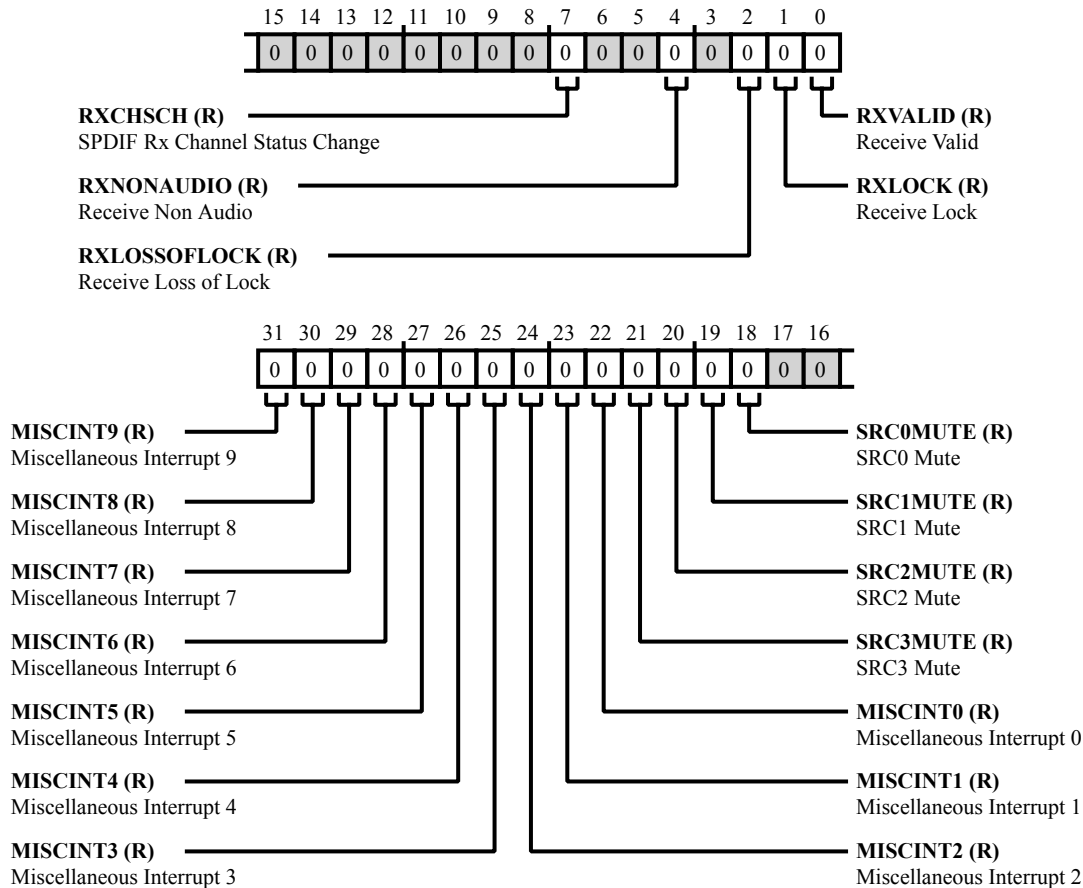


Figure 25-71: `DAI_IRPTL_HS` Register Diagram

Table 25-83: `DAI_IRPTL_HS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	MISCINT9	Miscellaneous Interrupt 9. The <code>DAI_IRPTL_HS.MISCINT9</code> bit is the shadow of the <code>DAI_IRPTL_H.MISCINT9</code> bit and contains the same content. Reading the <code>DAI_IRPTL_HS.MISCINT9</code> bit does not affect its contents while reading the <code>DAI_IRPTL_H.MISCINT9</code> bit clears it.

Table 25-83: DAI_IRPTL_HS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/NW)	MISCINT8	Miscellaneous Interrupt 8. The DAI_IRPTL_HS.MISCINT8 bit is the shadow of the DAI_IRPTL_H.MISCINT8 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT8 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT8 bit clears it.
29 (R/NW)	MISCINT7	Miscellaneous Interrupt 7. The DAI_IRPTL_HS.MISCINT7 bit is the shadow of the DAI_IRPTL_H.MISCINT7 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT7 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT7 bit clears it.
28 (R/NW)	MISCINT6	Miscellaneous Interrupt 6. The DAI_IRPTL_HS.MISCINT6 bit is the shadow of the DAI_IRPTL_H.MISCINT6 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT6 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT6 bit clears it.
27 (R/NW)	MISCINT5	Miscellaneous Interrupt 5. The DAI_IRPTL_HS.MISCINT5 bit is the shadow of the DAI_IRPTL_H.MISCINT5 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT5 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT5 bit clears it.
26 (R/NW)	MISCINT4	Miscellaneous Interrupt 4. The DAI_IRPTL_HS.MISCINT4 bit is the shadow of the DAI_IRPTL_H.MISCINT4 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT4 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT4 bit clears it.
25 (R/NW)	MISCINT3	Miscellaneous Interrupt 3. The DAI_IRPTL_HS.MISCINT3 bit is the shadow of the DAI_IRPTL_H.MISCINT3 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT3 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT3 bit clears it.
24 (R/NW)	MISCINT2	Miscellaneous Interrupt 2. The DAI_IRPTL_HS.MISCINT2 bit is the shadow of the DAI_IRPTL_H.MISCINT2 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT2 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT2 bit clears it.

Table 25-83: DAI_IRPTL_HS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
23 (R/NW)	MISCINT1	Miscellaneous Interrupt 1. The DAI_IRPTL_HS.MISCINT1 bit is the shadow of the DAI_IRPTL_H.MISCINT1 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT1 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT1 bit clears it.
22 (R/NW)	MISCINT0	Miscellaneous Interrupt 0. The DAI_IRPTL_HS.MISCINT0 bit is the shadow of the DAI_IRPTL_H.MISCINT0 bit and contains the same content. Reading the DAI_IRPTL_HS.MISCINT0 bit does not affect its contents while reading the DAI_IRPTL_H.MISCINT0 bit clears it.
21 (R/NW)	SRC3MUTE	SRC3 Mute. The DAI_IRPTL_HS.SRC3MUTE bit is the shadow of the DAI_IRPTL_H.SRC3MUTE bit and contains the same content. Reading the DAI_IRPTL_HS.SRC3MUTE bit does not affect its contents while reading the DAI_IRPTL_H.SRC3MUTE bit clears it.
20 (R/NW)	SRC2MUTE	SRC2 Mute. The DAI_IRPTL_HS.SRC2MUTE bit is the shadow of the DAI_IRPTL_H.SRC2MUTE bit and contains the same content. Reading the DAI_IRPTL_HS.SRC2MUTE bit does not affect its contents while reading the DAI_IRPTL_H.SRC2MUTE bit clears it.
19 (R/NW)	SRC1MUTE	SRC1 Mute. The DAI_IRPTL_HS.SRC1MUTE bit is the shadow of the DAI_IRPTL_H.SRC1MUTE bit and contains the same content. Reading the DAI_IRPTL_HS.SRC1MUTE bit does not affect its contents while reading the DAI_IRPTL_H.SRC1MUTE bit clears it.
18 (R/NW)	SRC0MUTE	SRC0 Mute. The DAI_IRPTL_HS.SRC0MUTE bit is the shadow of the DAI_IRPTL_H.SRC0MUTE bit and contains the same content. Reading the DAI_IRPTL_HS.SRC0MUTE bit does not affect its contents while reading the DAI_IRPTL_H.SRC0MUTE bit clears it.
7 (R/NW)	RXCHSCH	SPDIF Rx Channel Status Change. The DAI_IRPTL_HS.RXCHSCH bit is the shadow of the DAI_IRPTL_H.RXCHSCH bit and contains the same content. Reading the DAI_IRPTL_HS.RXCHSCH bit does not affect its contents while reading the DAI_IRPTL_H.RXCHSCH bit clears it.

Table 25-83: DAI_IRPTL_HS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/NW)	RXNONAUDIO	Receive Non Audio. The DAI_IRPTL_HS.RXNONAUDIO bit is the shadow of DAI_IRPTL_H.RXNONAUDIO bit and contains the same content. Reading this bit does not affect its content.
2 (R/NW)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IRPTL_HS.RXLOSSOFLOCK bit is the shadow of DAI_IRPTL_H.RXLOSSOFLOCK bit and contains the same content. Reading this bit does not affect its content.
1 (R/NW)	RXLOCK	Receive Lock. The DAI_IRPTL_HS.RXLOCK bit is the shadow of DAI_IRPTL_H.RXLOCK bit and contains the same content. Reading this bit does not affect its content.
0 (R/NW)	RXVALID	Receive Valid. Setting the DAI_IRPTL_HS.RXVALID bit is the shadow of DAI_IRPTL_H.RXVALID bit and contains the same content. Reading this bit does not affect its content.

Low Priority Interrupt Latch Register

The `DAI_IRPTL_L` register holds the low priority latched interrupt status for interrupt requests that have been unmasked (enabled) by the `DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI` registers. If a bit in `DAI_IRPTL_L` is already set and the corresponding interrupt is masked in `DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI` registers, the latch holds its old value, leaving the interrupt asserted until its masked registers (`DAI_IMSK_FE`, `DAI_IMSK_RE` or `DAI_IMSK_PRI`) are reset by software with a W1C operation.

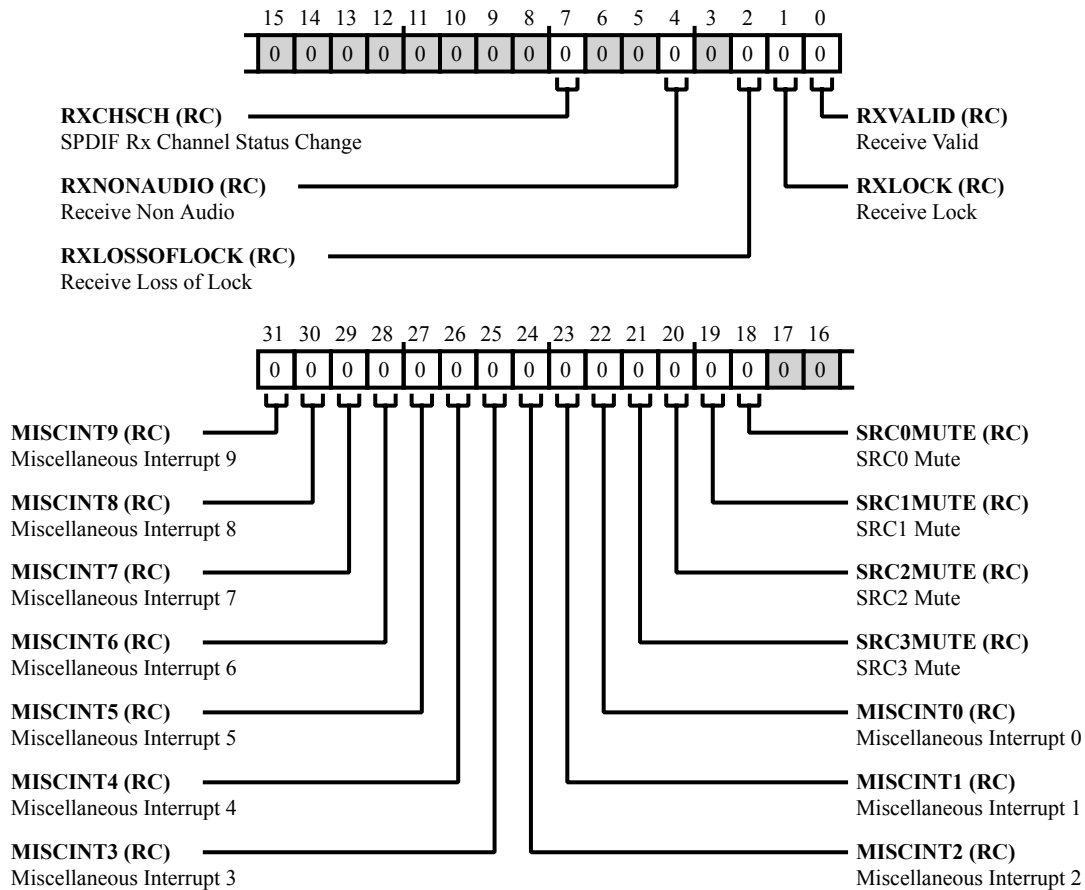


Figure 25-72: `DAI_IRPTL_L` Register Diagram

Table 25-84: `DAI_IRPTL_L` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (RC/NW)	MISCINT9	Miscellaneous Interrupt 9. The <code>DAI_IRPTL_L.MISCINT9</code> bit is set if the interrupt is mapped to the <code>INTR_DAI_IRQL</code> signal in the SEC using the <code>DAI_IMSK_PRI</code> register.

Table 25-84: DAI_IRPTL_L Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (RC/NW)	MISCINT8	Miscellaneous Interrupt 8. The DAI_IRPTL_L.MISCINT8 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
29 (RC/NW)	MISCINT7	Miscellaneous Interrupt 7. The DAI_IRPTL_L.MISCINT7 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
28 (RC/NW)	MISCINT6	Miscellaneous Interrupt 6. The DAI_IRPTL_L.MISCINT6 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
27 (RC/NW)	MISCINT5	Miscellaneous Interrupt 5. The DAI_IRPTL_L.MISCINT5 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
26 (RC/NW)	MISCINT4	Miscellaneous Interrupt 4. The DAI_IRPTL_L.MISCINT4 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
25 (RC/NW)	MISCINT3	Miscellaneous Interrupt 3. The DAI_IRPTL_L.MISCINT3 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
24 (RC/NW)	MISCINT2	Miscellaneous Interrupt 2. The DAI_IRPTL_L.MISCINT2 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
23 (RC/NW)	MISCINT1	Miscellaneous Interrupt 1. The DAI_IRPTL_L.MISCINT1 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
22 (RC/NW)	MISCINT0	Miscellaneous Interrupt 0. The DAI_IRPTL_L.MISCINT0 bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
21 (RC/NW)	SRC3MUTE	SRC3 Mute. The DAI_IRPTL_L.SRC3MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
20 (RC/NW)	SRC2MUTE	SRC2 Mute. The DAI_IRPTL_L.SRC2MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.

Table 25-84: DAI_IRPTL_L Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (RC/NW)	SRC1MUTE	SRC1 Mute. The DAI_IRPTL_L.SRC1MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
18 (RC/NW)	SRC0MUTE	SRC0 Mute. The DAI_IRPTL_L.SRC0MUTE bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
7 (RC/NW)	RXCHSCH	SPDIF Rx Channel Status Change. The DAI_IRPTL_L.RXCHSCH bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
4 (RC/NW)	RXNONAUDIO	Receive Non Audio. The DAI_IRPTL_L.RXNONAUDIO bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
2 (RC/NW)	RXLOSSOFLOCK	Receive Loss of Lock. The DAI_IRPTL_L.RXLOSSOFLOCK bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
1 (RC/NW)	RXLOCK	Receive Lock. The DAI_IRPTL_L.RXLOCK bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.
0 (RC/NW)	RXVALID	Receive Valid. The DAI_IRPTL_L.RXVALID bit is set if the interrupt is mapped to the INTR_DAI_IRQL signal in the SEC using the DAI_IMSK_PRI register.

Shadow Low Priority Interrupt Latch Register

The `DAI_IRPTL_LS` register is the shadow register of the `DAI_IRPTL_L` register. Its content is the same as the `DAI_IRPTL_L` register. Reading the `DAI_IRPTL_LS` register does not affect its contents while reading the contents of the `DAI_IRPTL_L` registers clears it.

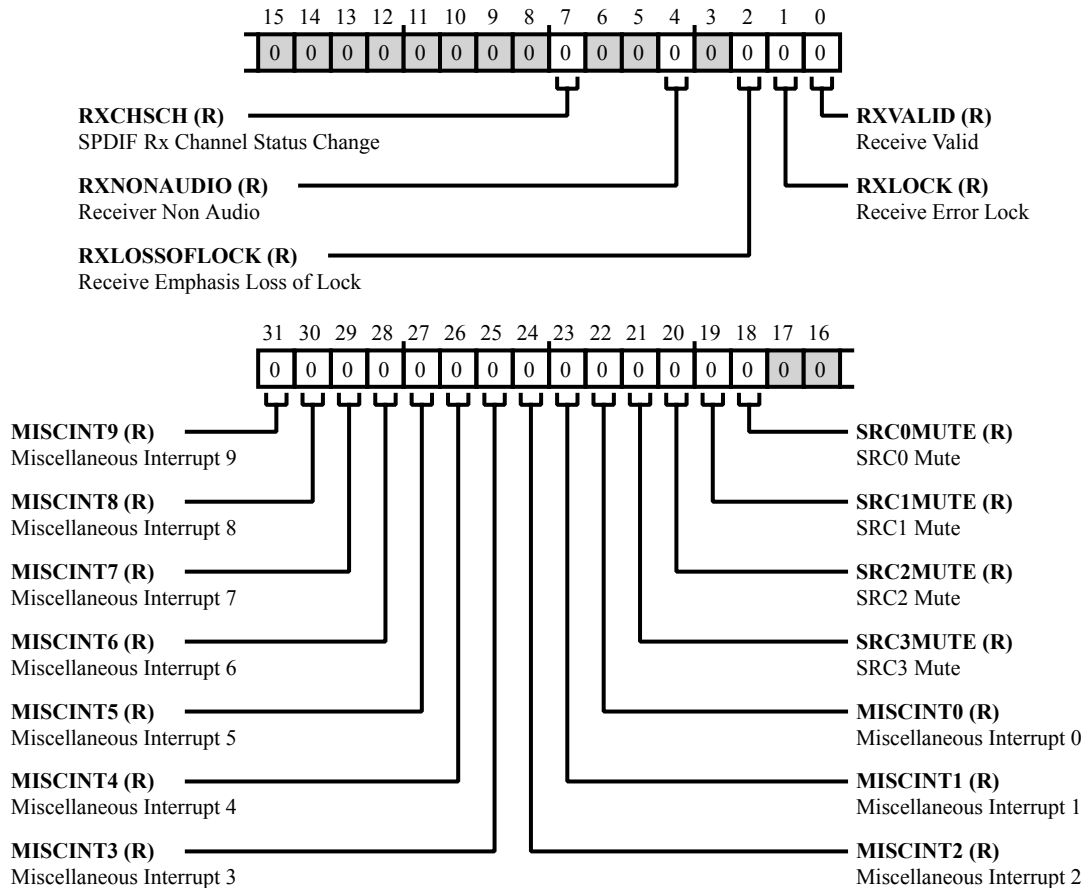


Figure 25-73: `DAI_IRPTL_LS` Register Diagram

Table 25-85: `DAI_IRPTL_LS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	MISCINT9	Miscellaneous Interrupt 9. The <code>DAI_IRPTL_LS.MISCINT9</code> bit is the shadow of the <code>DAI_IRPTL_L.MISCINT9</code> bit and contains the same content. Reading the <code>DAI_IRPTL_LS.MISCINT9</code> bit does not affect its contents while reading the <code>DAI_IRPTL_L.MISCINT9</code> bit clears it.

Table 25-85: DAI_IRPTL_LS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/NW)	MISCINT8	Miscellaneous Interrupt 8. The DAI_IRPTL_LS.MISCINT8 bit is the shadow of the DAI_IRPTL_L.MISCINT8 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT8 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT8 bit clears it.
29 (R/NW)	MISCINT7	Miscellaneous Interrupt 7. The DAI_IRPTL_LS.MISCINT7 bit is the shadow of the DAI_IRPTL_L.MISCINT7 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT7 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT7 bit clears it.
28 (R/NW)	MISCINT6	Miscellaneous Interrupt 6. The DAI_IRPTL_LS.MISCINT6 bit is the shadow of the DAI_IRPTL_L.MISCINT6 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT6 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT6 bit clears it.
27 (R/NW)	MISCINT5	Miscellaneous Interrupt 5. The DAI_IRPTL_LS.MISCINT5 bit is the shadow of the DAI_IRPTL_L.MISCINT5 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT5 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT5 bit clears it.
26 (R/NW)	MISCINT4	Miscellaneous Interrupt 4. The DAI_IRPTL_LS.MISCINT4 bit is the shadow of the DAI_IRPTL_L.MISCINT4 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT4 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT4 bit clears it.
25 (R/NW)	MISCINT3	Miscellaneous Interrupt 3. The DAI_IRPTL_LS.MISCINT3 bit is the shadow of the DAI_IRPTL_L.MISCINT3 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT3 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT3 bit clears it.
24 (R/NW)	MISCINT2	Miscellaneous Interrupt 2. The DAI_IRPTL_LS.MISCINT2 bit is the shadow of the DAI_IRPTL_L.MISCINT2 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT2 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT2 bit clears it.

Table 25-85: DAI_IRPTL_LS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
23 (R/NW)	MISCINT1	Miscellaneous Interrupt 1. The DAI_IRPTL_LS.MISCINT1 bit is the shadow of the DAI_IRPTL_L.MISCINT1 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT1 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT1 bit clears it.
22 (R/NW)	MISCINT0	Miscellaneous Interrupt 0. The DAI_IRPTL_LS.MISCINT0 bit is the shadow of the DAI_IRPTL_L.MISCINT0 bit and contains the same content. Reading the DAI_IRPTL_LS.MISCINT0 bit does not affect its contents while reading the DAI_IRPTL_L.MISCINT0 bit clears it.
21 (R/NW)	SRC3MUTE	SRC3 Mute. The DAI_IRPTL_LS.SRC3MUTE bit is the shadow of the DAI_IRPTL_L.SRC3MUTE bit and contains the same content. Reading the DAI_IRPTL_LS.SRC3MUTE bit does not affect its contents while reading the DAI_IRPTL_L.SRC3MUTE bit clears it.
20 (R/NW)	SRC2MUTE	SRC2 Mute. The DAI_IRPTL_LS.SRC2MUTE bit is the shadow of the DAI_IRPTL_L.SRC2MUTE bit and contains the same content. Reading the DAI_IRPTL_LS.SRC2MUTE bit does not affect its contents while reading the DAI_IRPTL_L.SRC2MUTE bit clears it.
19 (R/NW)	SRC1MUTE	SRC1 Mute. The DAI_IRPTL_LS.SRC1MUTE bit is the shadow of the DAI_IRPTL_L.SRC1MUTE bit and contains the same content. Reading the DAI_IRPTL_LS.SRC1MUTE bit does not affect its contents while reading the DAI_IRPTL_L.SRC1MUTE bit clears it.
18 (R/NW)	SRC0MUTE	SRC0 Mute. The DAI_IRPTL_LS.SRC0MUTE bit is the shadow of the DAI_IRPTL_L.SRC0MUTE bit and contains the same content. Reading the DAI_IRPTL_LS.SRC0MUTE bit does not affect its contents while reading the DAI_IRPTL_L.SRC0MUTE bit clears it.
7 (R/NW)	RXCHSCH	SPDIF Rx Channel Status Change. The DAI_IRPTL_LS.RXCHSCH bit is the shadow of the DAI_IRPTL_L.RXCHSCH bit and contains the same content. Reading the DAI_IRPTL_LS.RXCHSCH bit does not affect its contents while reading the DAI_IRPTL_L.RXCHSCH bit clears it.

Table 25-85: DAI_IRPTL_LS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/NW)	RXNONAUDIO	Receiver Non Audio. The DAI_IRPTL_LS.RXNONAUDIO bit is the shadow of the DAI_IRPTL_L.RXNONAUDIO bit and contains the same content. Reading the DAI_IRPTL_LS.RXNONAUDIO bit does not affect its contents while reading the DAI_IRPTL_L.RXNONAUDIO bit clears it.
2 (R/NW)	RXLOSSOFLOCK	Receive Emphasis Loss of Lock. The DAI_IRPTL_LS.RXLOSSOFLOCK bit is the shadow of DAI_IRPTL_L.RXLOSSOFLOCK bit and contains the same content. Reading this bit does not affect its content. Reading the DAI_IRPTL_L.RXLOSSOFLOCK bit clears it.
1 (R/NW)	RXLOCK	Receive Error Lock. The DAI_IRPTL_LS.RXLOCK bit is the shadow of DAI_IRPTL_L.RXLOCK bit and contains the same content. Reading this bit does not affect its content. Reading the DAI_IRPTL_L.RXLOCK bit clears it.
0 (R/NW)	RXVALID	Receive Valid. The DAI_IRPTL_LS.RXVALID bit is the shadow of DAI_IRPTL_L.RXVALID bit and contains the same content. Reading this bit does not affect its content. Reading the DAI_IRPTL_L.RXVALID bit clears it.

Miscellaneous Control Register 0

The `DAI_MISC0` register allows programs to route to the DAI interrupt latch, PBEN input routing, or input signal inversion. This register belongs to group E which routes control signals and provides a means of connecting signals between groups.

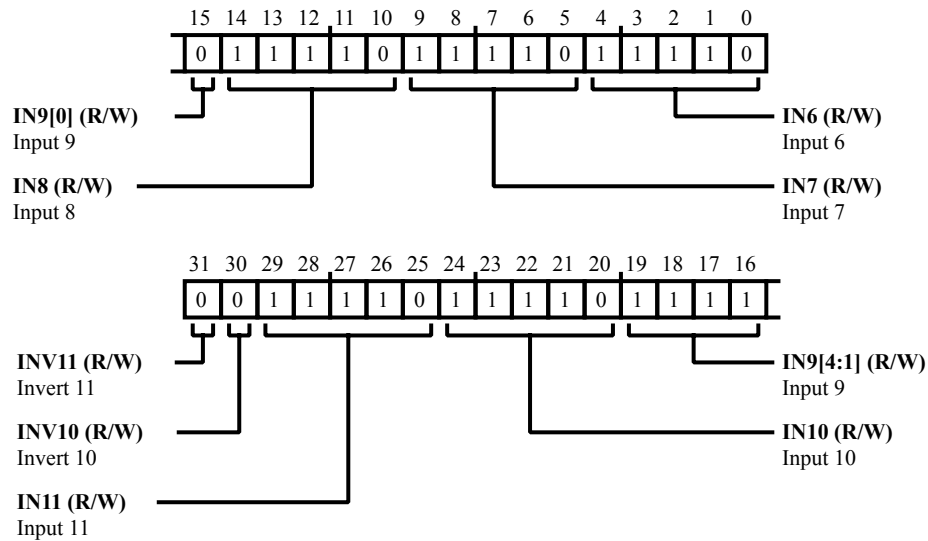


Figure 25-74: DAI_MISC0 Register Diagram

Table 25-86: DAI_MISC0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	INV11	Invert 11. The <code>DAI_MISC0.INV11</code> bit field inverts miscellaneous A5 input.
30 (R/W)	INV10	Invert 10. The <code>DAI_MISC0.INV10</code> bit field inverts miscellaneous A4 input.
29:25 (R/W)	IN11	Input 11. The <code>DAI_MISC0.IN11</code> bit field configures miscellaneous A5 input.
24:20 (R/W)	IN10	Input 10. The <code>DAI_MISC0.IN10</code> bit field configures miscellaneous A4 input.
19:15 (R/W)	IN9	Input 9. The <code>DAI_MISC0.IN9</code> bit field configures miscellaneous A3 input/miscellaneous Interrupt 9(DAI interrupt 31).
14:10 (R/W)	IN8	Input 8. The <code>DAI_MISC0.IN8</code> bit field configures miscellaneous A2 input/miscellaneous Interrupt 8(DAI interrupt 30).

Table 25-86: DAI_MISC0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9:5 (R/W)	IN7	Input 7. The <code>DAI_MISC0.IN7</code> bit field configures miscellaneous A1 input/miscellaneous Interrupt 7(DAI interrupt 29).
4:0 (R/W)	IN6	Input 6. The <code>DAI_MISC0.IN6</code> bit field configures miscellaneous A0 input/miscellaneous Interrupt 6(DAI interrupt 28).

Miscellaneous Control Register 1

The `DAI_MISC1` register allows programs to route to the DAI interrupt latch, PBEN input routing, or input signal inversion. This register belongs to group E which routes control signals and provides a means of connecting signals between groups.

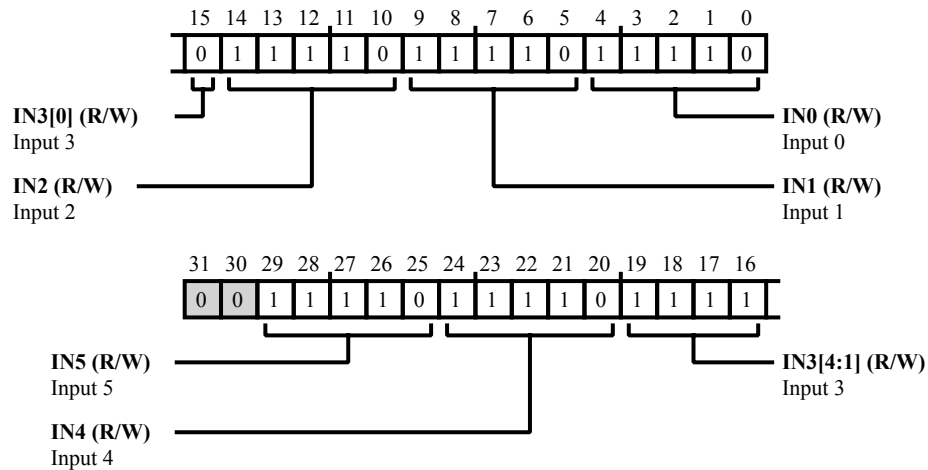


Figure 25-75: DAI_MISC1 Register Diagram

Table 25-87: DAI_MISC1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:25 (R/W)	IN5	Input 5. The <code>DAI_MISC1 . IN5</code> bit field configures miscellaneous Interrupt 5(DAI interrupt 27).
24:20 (R/W)	IN4	Input 4. The <code>DAI_MISC1 . IN4</code> bit field configures miscellaneous Interrupt 4(DAI interrupt 26).
19:15 (R/W)	IN3	Input 3. The <code>DAI_MISC1 . IN3</code> bit field configures miscellaneous Interrupt 3(DAI interrupt 25).
14:10 (R/W)	IN2	Input 2. The <code>DAI_MISC1 . IN2</code> bit field configures miscellaneous Interrupt 2(DAI interrupt 24).
9:5 (R/W)	IN1	Input 1. The <code>DAI_MISC1 . IN1</code> bit field configures miscellaneous Interrupt 1(DAI interrupt 23).

Table 25-87: DAI_MISC1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	IN0	Input 0. The <code>DAI_MISC1.IN0</code> bit field configures miscellaneous Interrupt 0(DAI interrupt 22).

Miscellaneous Control Register 1

The `DAI_MISC2` register allows programs to route to the DAI interrupt latch, PBEN input routing, or input signal inversion. This register belongs to group E which routes control signals and provides a means of connecting signals between groups.

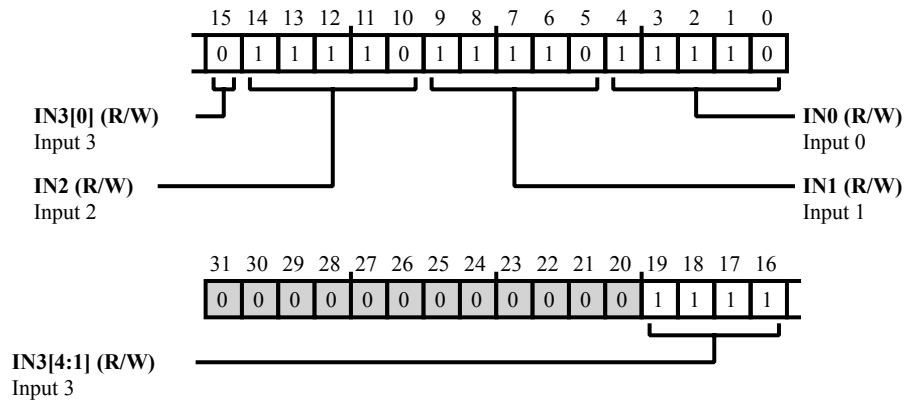


Figure 25-76: DAI_MISC2 Register Diagram

Table 25-88: DAI_MISC2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19:15 (R/W)	IN3	Input 3. The <code>DAI_MISC2.IN3</code> bit field configures PCG-F's hardware slave trigger input.
14:10 (R/W)	IN2	Input 2. The <code>DAI_MISC2.IN2</code> bit field configures PCG-E's hardware slave trigger input.
9:5 (R/W)	IN1	Input 1. The <code>DAI_MISC2.IN1</code> bit field configures the PCG-B's hardware slave trigger input.
4:0 (R/W)	IN0	Input 0. The <code>DAI_MISC2.IN0</code> bit field configures the PCG-A's hardware slave trigger input.

Pin Buffer Enable Register 0

The `DAI_PBEN0` register routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

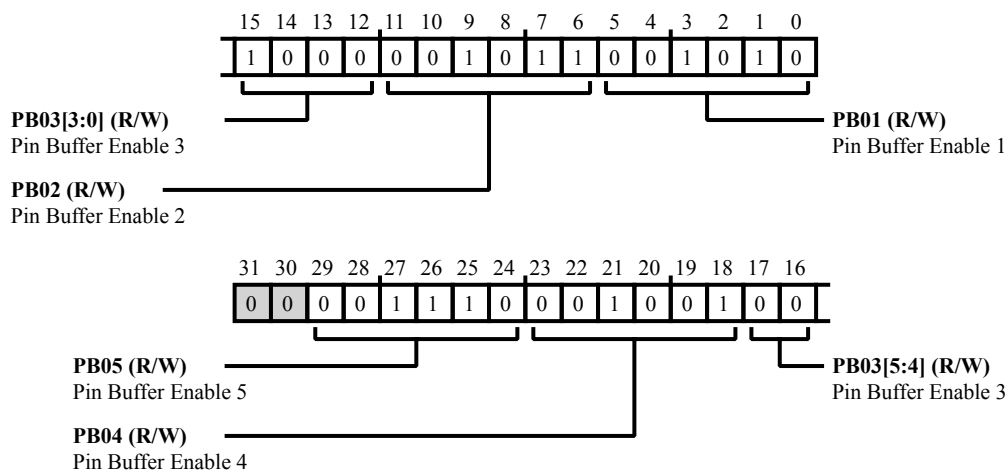


Figure 25-77: `DAI_PBEN0` Register Diagram

Table 25-89: `DAI_PBEN0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	PB05	Pin Buffer Enable 5. The <code>DAI_PBEN0 . PB05</code> bit field is the pin buffer enable for DAI port 5.
23:18 (R/W)	PB04	Pin Buffer Enable 4. The <code>DAI_PBEN0 . PB04</code> bit field is the pin buffer enable for DAI port 4.
17:12 (R/W)	PB03	Pin Buffer Enable 3. The <code>DAI_PBEN0 . PB03</code> bit field is the pin buffer enable for DAI port 3.
11:6 (R/W)	PB02	Pin Buffer Enable 2. The <code>DAI_PBEN0 . PB02</code> bit field is the pin buffer enable for DAI port 2.
5:0 (R/W)	PB01	Pin Buffer Enable 1. The <code>DAI_PBEN0 . PB01</code> bit field is the pin buffer enable for DAI port 1.

Pin Buffer Enable Register 1

The `DAI_PBEN1` register routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

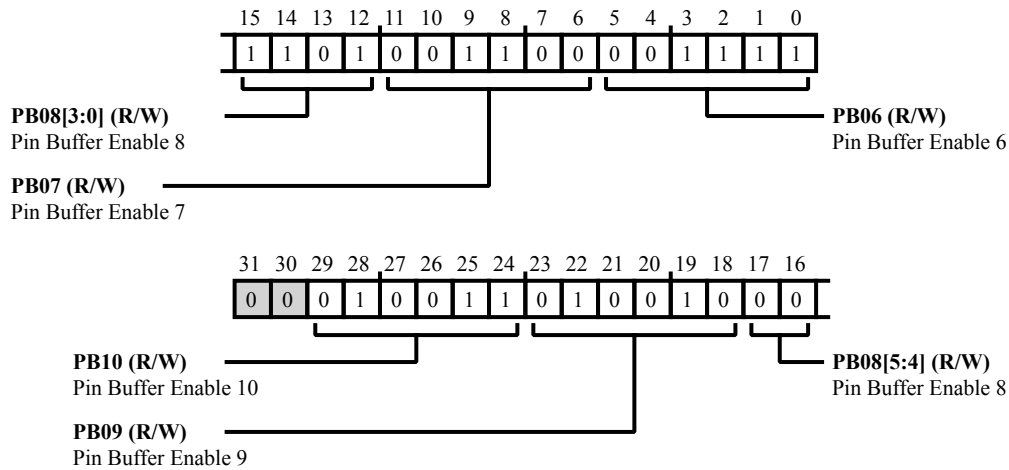


Figure 25-78: DAI_PBEN1 Register Diagram

Table 25-90: DAI_PBEN1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	PB10	Pin Buffer Enable 10. The <code>DAI_PBEN1 . PB10</code> bit field is the pin buffer enable for DAI port 10.
23:18 (R/W)	PB09	Pin Buffer Enable 9. The <code>DAI_PBEN1 . PB09</code> bit field is the pin buffer enable for DAI port 9.
17:12 (R/W)	PB08	Pin Buffer Enable 8. The <code>DAI_PBEN1 . PB08</code> bit field is the pin buffer enable for DAI port 8.
11:6 (R/W)	PB07	Pin Buffer Enable 7. The <code>DAI_PBEN1 . PB07</code> bit field is the pin buffer enable for DAI port 7.
5:0 (R/W)	PB06	Pin Buffer Enable 6. The <code>DAI_PBEN1 . PB06</code> bit field is the pin buffer enable for DAI port 6.

Pin Buffer Enable Register 2

The `DAI_PBEN2` register routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

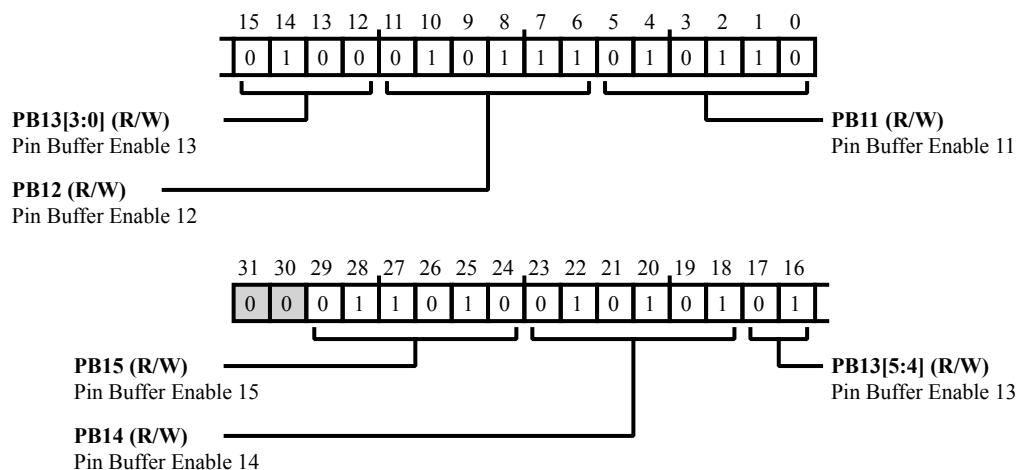


Figure 25-79: DAI_PBEN2 Register Diagram

Table 25-91: DAI_PBEN2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	PB15	Pin Buffer Enable 15. The <code>DAI_PBEN2 . PB15</code> bit field is the pin buffer enable for DAI port 15.
23:18 (R/W)	PB14	Pin Buffer Enable 14. The <code>DAI_PBEN2 . PB14</code> bit field is the pin buffer enable for DAI port 14.
17:12 (R/W)	PB13	Pin Buffer Enable 13. The <code>DAI_PBEN2 . PB13</code> bit field is the pin buffer enable for DAI port 13.
11:6 (R/W)	PB12	Pin Buffer Enable 12. The <code>DAI_PBEN2 . PB12</code> bit field is the pin buffer enable for DAI port 12.
5:0 (R/W)	PB11	Pin Buffer Enable 11. The <code>DAI_PBEN2 . PB11</code> bit field is the pin buffer enable for DAI port 11.

Pin Buffer Enable Register 3

The `DAI_PBEN3` register routes signals to the pin enables, and the value of these signals determines if a DAI pin is used as an output or an input.

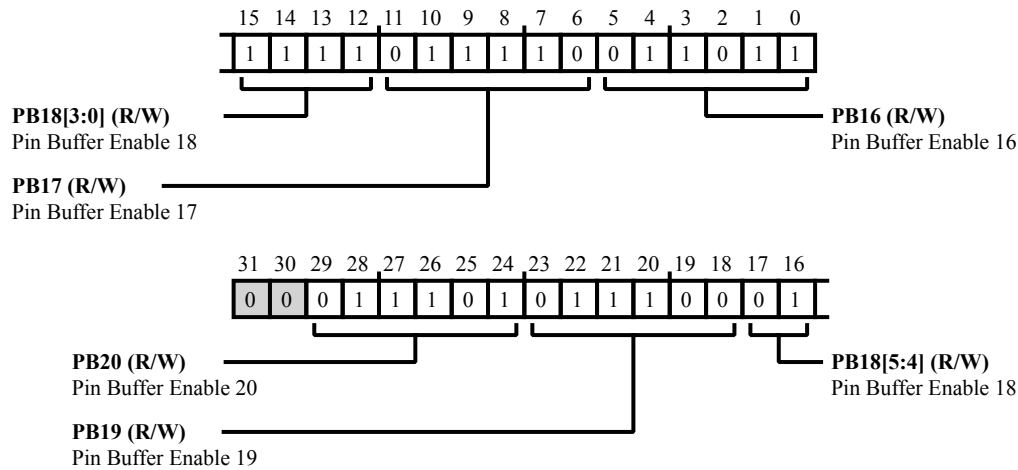


Figure 25-80: DAI_PBEN3 Register Diagram

Table 25-92: DAI_PBEN3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/W)	PB20	Pin Buffer Enable 20. The <code>DAI_PBEN3.PB20</code> bit field is the pin buffer enable for DAI port 20.
23:18 (R/W)	PB19	Pin Buffer Enable 19. The <code>DAI_PBEN3.PB19</code> bit field is the pin buffer enable for DAI port 19.
17:12 (R/W)	PB18	Pin Buffer Enable 18. The <code>DAI_PBEN3.PB18</code> bit field is the pin buffer enable for DAI port 18.
11:6 (R/W)	PB17	Pin Buffer Enable 17. The <code>DAI_PBEN3.PB17</code> bit field is the pin buffer enable for DAI port 17.
5:0 (R/W)	PB16	Pin Buffer Enable 16. The <code>DAI_PBEN3.PB16</code> bit field is the pin buffer enable for DAI port 16.

Pin Buffer Assignment Register 0

The `DAI_PIN0` register routes physical pins that are connected to a bonded pad.

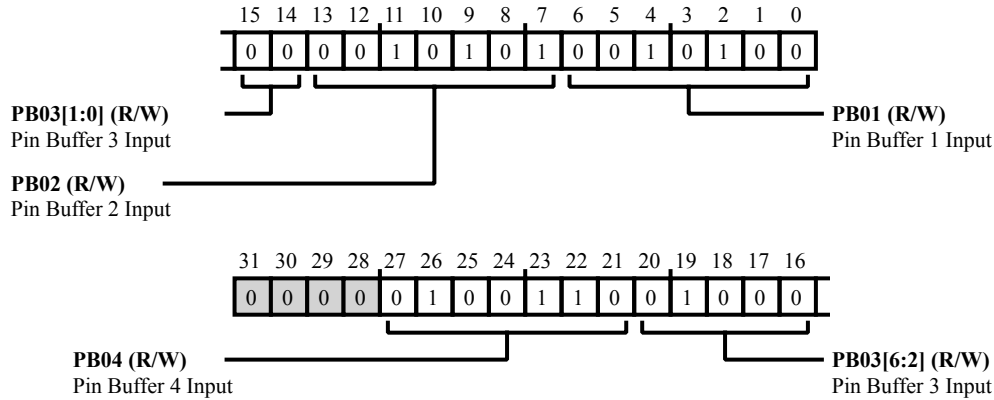


Figure 25-81: `DAI_PIN0` Register Diagram

Table 25-93: `DAI_PIN0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:21 (R/W)	PB04	Pin Buffer 4 Input. <code>DAI_PIN0.PB04</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN0.PB04</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
20:14 (R/W)	PB03	Pin Buffer 3 Input. <code>DAI_PIN0.PB03</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN0.PB03</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
13:7 (R/W)	PB02	Pin Buffer 2 Input. <code>DAI_PIN0.PB02</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN0.PB02</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
6:0 (R/W)	PB01	Pin Buffer 1 Input. <code>DAI_PIN0.PB01</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN0.PB01</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.

Pin Buffer Assignment Register 1

The `DAI_PIN1` register routes physical pins that are connected to a bonded pad.

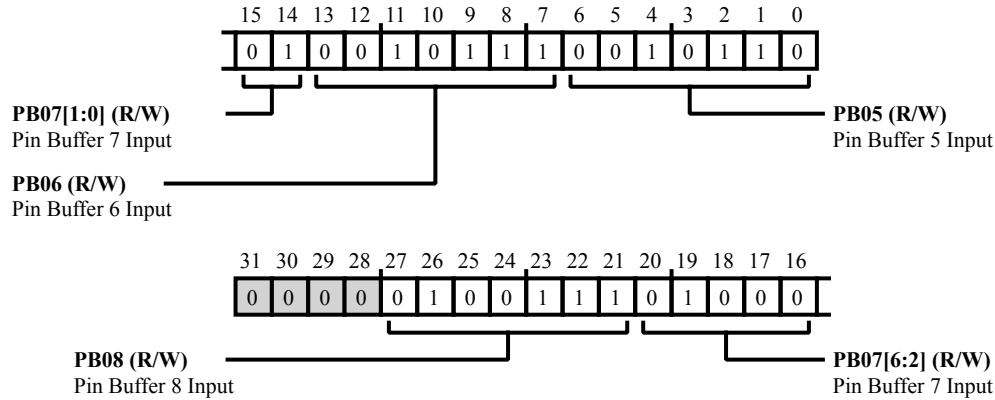


Figure 25-82: `DAI_PIN1` Register Diagram

Table 25-94: `DAI_PIN1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:21 (R/W)	PB08	Pin Buffer 8 Input. <code>DAI_PIN1 . PB08</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN1 . PB08</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
20:14 (R/W)	PB07	Pin Buffer 7 Input. <code>DAI_PIN1 . PB07</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN1 . PB07</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
13:7 (R/W)	PB06	Pin Buffer 6 Input. <code>DAI_PIN1 . PB06</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN1 . PB06</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
6:0 (R/W)	PB05	Pin Buffer 5 Input. <code>DAI_PIN1 . PB05</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN1 . PB05</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.

Pin Buffer Assignment Register 2

The `DAI_PIN2` register routes physical pins that are connected to a bonded pad.

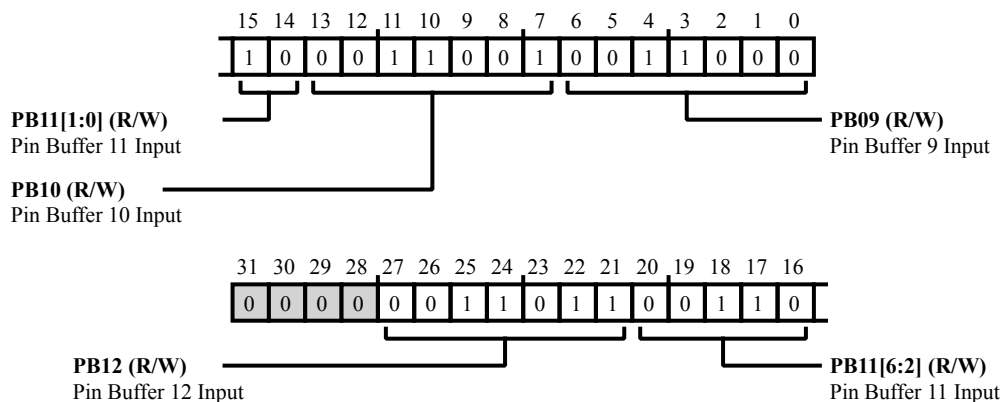


Figure 25-83: `DAI_PIN2` Register Diagram

Table 25-95: `DAI_PIN2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:21 (R/W)	PB12	Pin Buffer 12 Input. <code>DAI_PIN2</code> . PB12 holds the Source signal assignment that will be routed to the <code>DAI_PIN2</code> . PB12 Destination. Refer to the Group D Signals table for Source and Destination mappings.
20:14 (R/W)	PB11	Pin Buffer 11 Input. <code>DAI_PIN2</code> . PB11 holds the Source signal assignment that will be routed to the <code>DAI_PIN2</code> . PB11 Destination. Refer to the Group D Signals table for Source and Destination mappings.
13:7 (R/W)	PB10	Pin Buffer 10 Input. <code>DAI_PIN2</code> . PB10 holds the Source signal assignment that will be routed to the <code>DAI_PIN2</code> . PB10 Destination. Refer to the Group D Signals table for Source and Destination mappings.
6:0 (R/W)	PB09	Pin Buffer 9 Input. <code>DAI_PIN2</code> . PB09 holds the Source signal assignment that will be routed to the <code>DAI_PIN2</code> . PB09 Destination. Refer to the Group D Signals table for Source and Destination mappings.

Pin Buffer Assignment Register 3

The `DAI_PIN3` register routes physical pins that are connected to a bonded pad.

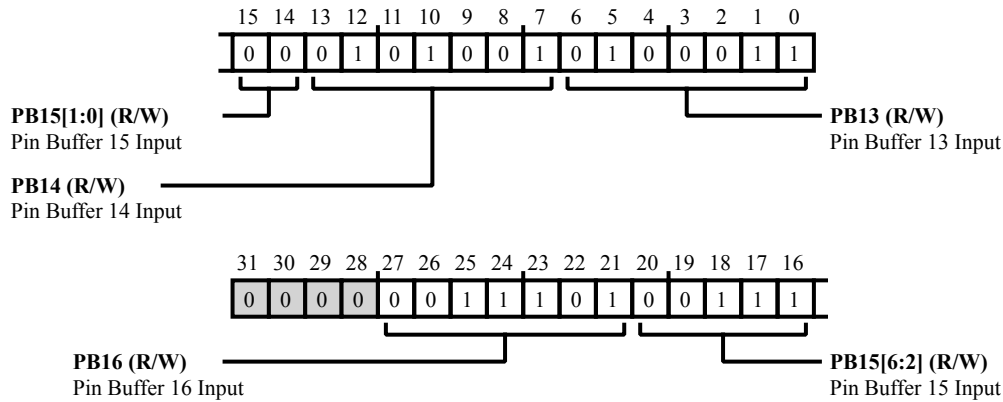


Figure 25-84: `DAI_PIN3` Register Diagram

Table 25-96: `DAI_PIN3` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:21 (R/W)	PB16	Pin Buffer 16 Input. The <code>DAI_PIN3.PB16</code> bit field is the pin buffer 16 input.
20:14 (R/W)	PB15	Pin Buffer 15 Input. <code>DAI_PIN3.PB15</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN3.PB15</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
13:7 (R/W)	PB14	Pin Buffer 14 Input. <code>DAI_PIN3.PB14</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN3.PB14</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.
6:0 (R/W)	PB13	Pin Buffer 13 Input. <code>DAI_PIN3.PB13</code> holds the Source signal assignment that will be routed to the <code>DAI_PIN3.PB13</code> Destination. Refer to the Group D Signals table for Source and Destination mappings.

Pin Buffer Assignment Register 4

The `DAI_PIN4` register routes physical pins that are connected to a bonded pad.

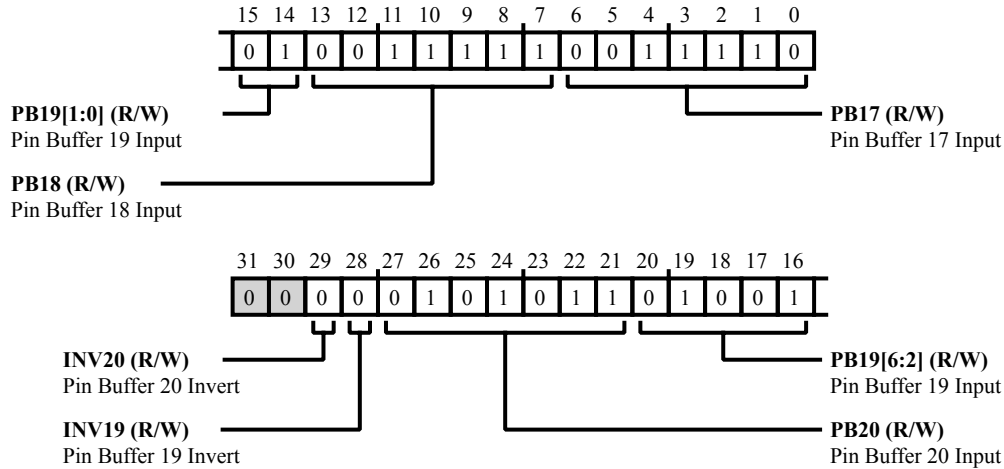


Figure 25-85: DAI_PIN4 Register Diagram

Table 25-97: DAI_PIN4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	INV20	Pin Buffer 20 Invert. DAI_PIN4 . INV20 holds the Source signal assignment that will be routed to the DAI_PIN4 . INV20 Destination. Refer to the Group D Signals table for Source and Destination mappings.
28 (R/W)	INV19	Pin Buffer 19 Invert. DAI_PIN4 . INV19 holds the Source signal assignment that will be routed to the DAI_PIN4 . INV19 Destination. Refer to the Group D Signals table for Source and Destination mappings.
27:21 (R/W)	PB20	Pin Buffer 20 Input. DAI_PIN4 . PB20 holds the Source signal assignment that will be routed to the DAI_PIN4 . PB20 Destination. Refer to the Group D Signals table for Source and Destination mappings.
20:14 (R/W)	PB19	Pin Buffer 19 Input. DAI_PIN4 . PB19 holds the Source signal assignment that will be routed to the DAI_PIN4 . PB19 Destination. Refer to the Group D Signals table for Source and Destination mappings.
13:7 (R/W)	PB18	Pin Buffer 18 Input. DAI_PIN4 . PB18 holds the Source signal assignment that will be routed to the DAI_PIN4 . PB18 Destination. Refer to the Group D Signals table for Source and Destination mappings.

Table 25-97: DAI_PIN4 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	PB17	Pin Buffer 17 Input. DAI_PIN4 . PB17 holds the Source signal assignment that will be routed to the DAI_PIN4 . PB17 Destination. Refer to the Group D Signals table for Source and Destination mappings.

Pin Status Register

The `DAI_PIN_STAT` register bits indicate the status (signal high or low) for each pin. The individual bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

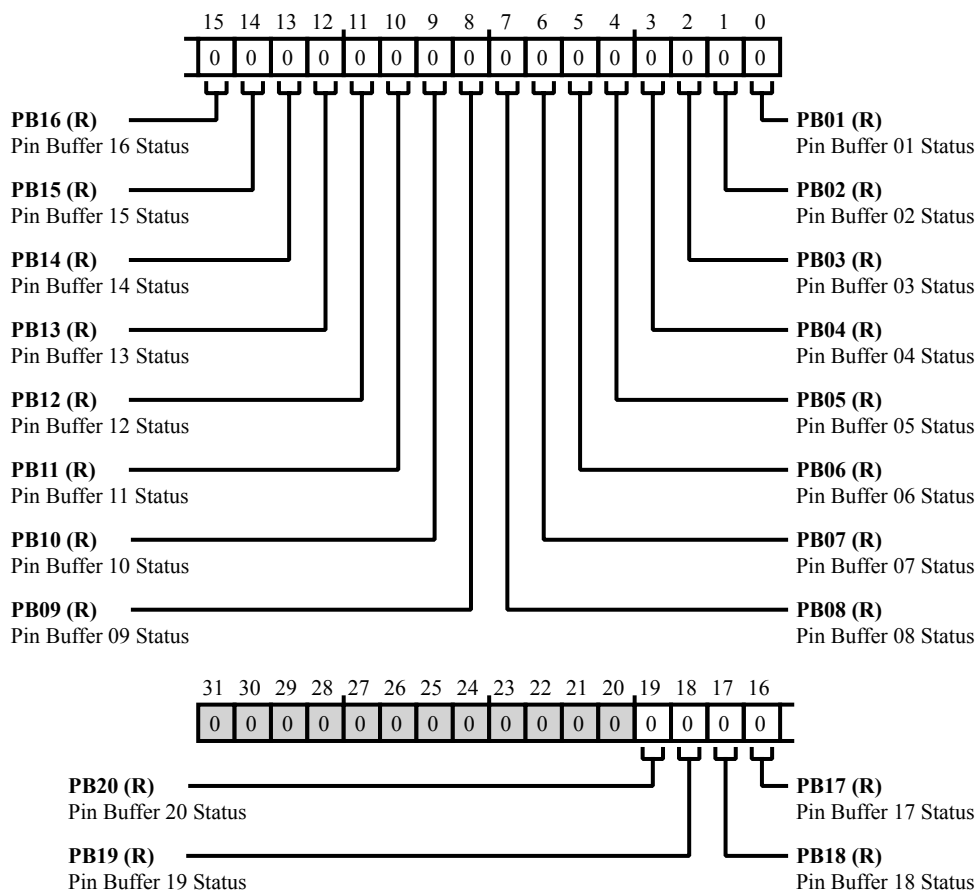


Figure 25-86: `DAI_PIN_STAT` Register Diagram

Table 25-98: `DAI_PIN_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/NW)	PB20	Pin Buffer 20 Status. The <code>DAI_PIN_STAT.PB20</code> bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
18 (R/NW)	PB19	Pin Buffer 19 Status. The <code>DAI_PIN_STAT.PB19</code> bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

Table 25-98: DAI_PIN_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/NW)	PB18	Pin Buffer 18 Status. The DAI_PIN_STAT.PB18 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
16 (R/NW)	PB17	Pin Buffer 17 Status. The DAI_PIN_STAT.PB17 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
15 (R/NW)	PB16	Pin Buffer 16 Status. The DAI_PIN_STAT.PB16 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
14 (R/NW)	PB15	Pin Buffer 15 Status. The DAI_PIN_STAT.PB15 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
13 (R/NW)	PB14	Pin Buffer 14 Status. The DAI_PIN_STAT.PB14 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
12 (R/NW)	PB13	Pin Buffer 13 Status. The DAI_PIN_STAT.PB13 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
11 (R/NW)	PB12	Pin Buffer 12 Status. The DAI_PIN_STAT.PB12 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
10 (R/NW)	PB11	Pin Buffer 11 Status. The DAI_PIN_STAT.PB11 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
9 (R/NW)	PB10	Pin Buffer 10 Status. The DAI_PIN_STAT.PB10 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
8 (R/NW)	PB09	Pin Buffer 09 Status. The DAI_PIN_STAT.PB09 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
7 (R/NW)	PB08	Pin Buffer 08 Status. The DAI_PIN_STAT.PB08 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

Table 25-98: DAI_PIN_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/NW)	PB07	Pin Buffer 07 Status. The DAI_PIN_STAT . PB07 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
5 (R/NW)	PB06	Pin Buffer 06 Status. The DAI_PIN_STAT . PB06 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
4 (R/NW)	PB05	Pin Buffer 05 Status. The DAI_PIN_STAT . PB05 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
3 (R/NW)	PB04	Pin Buffer 04 Status. The DAI_PIN_STAT . PB04 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
2 (R/NW)	PB03	Pin Buffer 03 Status. The DAI_PIN_STAT . PB03 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
1 (R/NW)	PB02	Pin Buffer 02 Status. The DAI_PIN_STAT . PB02 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.
0 (R/NW)	PB01	Pin Buffer 01 Status. The DAI_PIN_STAT . PB01 bit reads 1 if the signal to this pin is high and reads 0 if signal to this pin is low.

26 Serial Port (SPORT)

The programmable serial ports (SPORTs) support various protocols for serial data communication and provide a glueless hardware interface to many industry-standard data converters and codecs. They have high data rates and dual half-duplex datapaths and are ideal for establishing a direct serial connection among two or more processors in a multiprocessor system, as many processors provide compatible serial interfaces.

The SPORT top module consists of two half SPORTs with identical functionality and programming requirements. Each half SPORT can be independently configured as either a transmitter or receiver and can be coupled with the other half SPORT within the same SPORT top module. Further, each half SPORT provides two synchronous half-duplex data lines to double the total supported throughput. As such, a single SPORT top module can be used to provide up to four unidirectional or up to two full-duplex data streams. Further channels are possible as well, but utilization of multiple SPORT top modules is required, thus requiring external connections to provide a common time base.

Features

An individual SPORT top module consists of two independently configurable SPORT halves with identical functionality. These SPORT halves offer the following features:

- Up to two bidirectional data lines - each half SPORT supports up to two transmit or receive channels, thus allowing two unidirectional streams into or out of each half SPORT and providing greater flexibility for serial communications. If full-duplex functionality is desired, two SPORT halves can be combined to enable dual-stream bidirectional communication.
- Six operating modes:
 1. Standard DSP serial mode
 2. I²S mode
 3. Left-Justified mode
 4. Right-Justified mode
 5. Multichannel (TDM) mode
 6. Packed mode

- Supports internally or externally generated clock.
- Support for both even and odd SCLK0 to SPORT clock (SPORT_CLK) ratios. If both data lines of a half SPORT are active, the maximum throughput is 2 x SPORT_CLK bps.
- Configurable rising or falling edge of the SPORT_CLK for driving and sampling data and frame syncs.
- Gated clock mode support for internally or externally generated clocks in DSP serial mode and stereo modes (left-justified and I²S mode).
- Supports frameless operation.
- Supports internally or externally generated frame sync signals.
- Programmable frame sync polarity.
- Programmable frame sync timing (synchronous to data or 1 SPORT clock in advance of it).
- Detection of prematurely received external frame syncs (with optional interrupt request generation).
- Programmable level-/edge-sensitivity for external frame syncs.
- Programmable (4–32-bit) data length, either in most significant bit (MSB) first or in least significant bit (LSB) first format, with optional sign-extension on received data.
- Optional 16-bit to 32-bit word packing (as receiver) and 32-bit to 16-bit word unpacking (as transmitter).
- Support for A-law and μ -law compression/decompression hardware companding, according to the G.711 specification, on transmitted/received words in all operating modes.
- Transmit underrun and receive overflow detection (with optional interrupt request generation).
- TDM mode transfers data on 128 contiguous channels from a stream of up to 1024 total channels (useful for H.100/H.110 and other telephony interfaces as a network communication scheme for multiple processors).
- Performs interrupt-driven, single word core transfers to and from on-chip or off-chip memory.
- Dedicated DMA channel for each SPORT half supporting autobuffer (for a repeated, identical range of transfers) and numerous descriptor-based (individual or repeated ranges of transfers with differing DMA parameters) modes.
- Master and slave trigger functionality.
- Unique transfer finish interrupt (TFI) signaling when the last transmit word is fully out of the transmit shift register.
- Multiplexer to internally connect critical timing signals between SPORT halves.
- Support for Global Sport Enable functionality, where enabling multiple SPORTs need to be synchronized so that all of them start and/or end at the same time.
- Support for late reception of SPORT data and frame sync.

Signal Descriptions

Each half SPORT module has five dedicated signals, as described in the *SPORT Signal Descriptions* table. The actual pin name varies with different SPORT halves. Individual SPORT halves do not share any of its signals across the pair that comprises the SPORT top module; however, it is possible to connect the clock and frame sync signals between the SPORT half pair, as explained in the [Multiplexer Logic](#) section.

All of the SPORT signals are multiplexed on the PORT pins, possibly sharing functionality with other peripherals on the device. By default, the PORT pins are in GPIO mode and must be reconfigured for SPORT functionality by setting the appropriate bits in the [PORT_FER](#) and [PORT_MUX](#) registers. Consult the processor datasheet for details regarding which ports the SPORT signals are available on, and be sure to configure the [PORT_MUX](#) register before the [PORT_FER](#) register.

Table 26-1: SPORT Signal Descriptions

Internal Node	Direction	Description
SPORTx_CLK	I/O	Transmit or receive serial clock. Data and frame syncs are driven or sampled on this clock's edges. This signal can be either internally or externally generated.
SPORTx_FS	I/O	Transmit or receive frame sync. The frame sync pulse initiates shifting of serial data. This signal is either internally or externally generated.
SPORTx_D0	I/O	Primary transmit or receive data channel. This signal can be configured as an output to transmit serial data or as an input to receive serial data.
SPORTx_D1	I/O	Secondary transmit or receive data channel. This signal can be configured as an output to transmit serial data or as an input to receive serial data.
SPORTx_TDV	O	Multichannel transmit data valid. This signal is only active in multichannel transmit mode and is asserted during enabled slots, as defined by the channel selection registers (SPORT_CS0_A through SPORT_CS3_B).

The data channel signals are transmit signals when the serial port is configured in transmit mode ([SPORT_CTL_A.SPTRAN](#) = 1). They are receive signals when the serial port is configured in receive mode ([SPORT_CTL_A.SPTRAN](#) = 0). The following sections further describe the SPORT signals.

NOTE: These sections explicitly refer to the registers associated with half SPORT A, but the same concepts also apply to half SPORT B.

Serial Clock

The serial port clock ([SPORT_ACLK](#)) is either a receive serial clock or a transmit serial clock, depending on the transfer direction ([SPORT_CTL_A.SPTRAN](#)), governing when the data bits are serially shifted into or out of the SPORT and when the frame sync signal is driven (in internal frame sync mode) or sampled (in external frame sync mode). It can be internally generated from the processor's system clock (SCLK) or externally provided. If the half SPORT is configured in internal clock mode ([SPORT_CTL_A.ICLK](#) = 1), then the [SPORT_DIV_A.CLKDIV](#) field specifies the divisor applied to SCLK to generate the SPORT clock. As it is a 16-bit divisor, a wide range of serial clock rates is possible. Use the following equation to calculate the serial clock frequency:

$$\text{SPORT_ACLK} = [\text{SCLK} \div (\text{SPORT_DIV_A.CLKDIV} + 1)]$$

From this, the following equation can be used to determine the value of `SPORT_DIV_A.CLKDIV`, given the `SCLK` frequency and the desired frequency of the `SPORT` clock:

$$\text{SPORT_DIV_A.CLKDIV} = [(\text{SCLK} \div \text{SPORT_ACLK}) - 1]$$

The half `SPORT` also supports a 1:1 `SPORT_ACLK` to ratio (per the equations above, program the clock divisor field to zero). In this case, the resulting `SPORT` clock frequency is equal to `SCLK`.

NOTE: Be careful not to exceed the maximum `SPORT_ACLK` frequency specified in the processor datasheet.

In certain operating modes, the `SPORT` can be configured to generate a gated clock, which is active only during valid data. In some applications, a `SPORT` uses it to generate a general-purpose clock in the system. In this case, enable the `SPORT` with the appropriate `SPORT_DIV_A.CLKDIV` divisor field in internal clock mode.

If a `SPORT` is configured in external clock mode (`SPORT_CTL_A.ICLK = 0`), the serial clock is an input signal, thus making the `SPORT` operate in slave mode. In this mode, the `SPORT_DIV_A.CLKDIV` is irrelevant and is ignored. The optional loopback capability provided by the internal `SPORT` multiplexer (`SPMUX`) block allows the slave `SPORT` to use the serial clock from the neighboring `SPORT` half in the same `SPORT` top module.

An externally-supplied serial clock does not need to be synchronous with the processor clocks. Further, the external clock can be a gated clock, but it must comply with the requirements described in the [Gated Clock Mode](#) section.

Refer to the product datasheet for exact AC timing specifications.

Frame Sync

The `SPORT` frame sync (`SPORT_AFS`) signal is either a receive frame sync or a transmit frame sync, depending on the transfer direction (`SPORT_CTL_A.SPTRAN`), which is used to determine the start of a new word or frame. When this signal goes active, the serial port starts serially shifting data into or out of the `SPORT`. The frame sync signal can be internally generated based on its serial clock (`SPORT_ACLK`) or externally provided, as configured by the `SPORT_CTL_A.IFS` bit.

If the half `SPORT` is configured to generate frame syncs (`SPORT_CTL_A.IFS = 1`), then the `SPORT_DIV_A.FSDIV` field specifies the divisor used to generate the periodic `SPORT_AFS` signal from the `SPORT` clock. As this is a 16-bit divisor, a wide range of frame sync rates to initiate periodic transfers is possible. Whether the serial clock is internally or externally generated, this divisor is a count of `SPORT` clock cycles between frame sync pulses, the formula for which is:

$$\text{Number of serial clocks between frame syncs} = (\text{SPORT_DIV_A.FSDIV} + 1)$$

From this, the following equation can be used to determine the value of `SPORT_DIV_A.FSDIV`, given the serial clock frequency and the desired frame sync frequency:

$$\text{SPORT_DIV_A.FSDIV} = [(\text{SPORT_ACLK} \div \text{SPORT_AFS}) - 1]$$

The frame sync is continuously active when `SPORT_DIV_A.FSDIV = 0`. The value of `SPORT_DIV_A.FSDIV` cannot be less than the serial word length minus one (the value of the `SPORT_CTL_A.SLEN` bit field). Failure to

adhere to this guideline can cause an external device to abort the current operation or cause other unpredictable results.

NOTE: After enabling the SPORT, the first internal frame sync appears after a delay of $\text{SPORT_DIV_A.FSDIV} + 3$ SPORT clock cycles.

If a SPORT is configured for external frame syncs ($\text{SPORT_CTL_A.IFS} = 0$), then SPORT_AFS is an input signal and the SPORT_DIV_A.FSDIV field of the [SPORT_DIV_A](#) register is irrelevant and ignored. By default, this external signal is level-sensitive, but it can be configured as an edge-sensitive signal by setting the SPORT_CTL_A.FSED bit. The frame sync is expected to be synchronous with the serial clock. If not, it must meet the timing requirements that appear in the datasheet.

The SPORT can be used as a counter for dividing an external clock to generate periodic pulses or periodic interrupts. To do so, enable the SPORT with the appropriate SPORT_DIV_A.FSDIV divisor field with the SPORT configured for an external clock and internal data-independent frame syncs.

In some of the operating modes, the SPORT can be programmed to treat the frame sync signal as an optional signal by clearing the SPORT_CTL_A.FSR bit. Even with this bit cleared, the SPORT requires a single frame sync assertion to start the continuous transfers, after which it is ignored (for externally supplied frame syncs) or not generated (for internally-generated frame syncs). Characteristics of the frame sync depend on the settings in the SPORT control registers and the operating mode of the SPORT. For more information, refer to the [SPORT_CTL_A](#) register.

Data Signals

Each half SPORT has two bidirectional data lines known as the primary (SPORT_AD0) and secondary (SPORT_AD1) data channels. Both of the data lines can be configured as either transmitters or receivers using the $\text{SPORT_CTL_A.SPTRAN}$ bit, thus permitting dual unidirectional data streams to increase the data throughput of the SPORT.

NOTE: Configuring one transmit data channel and one receive data channel on a single half SPORT is not supported.

The primary and secondary data lines can be individually enabled or disabled using the $\text{SPORT_CTL_A.SPENPRI}$ and the $\text{SPORT_CTL_A.SPENSEC}$ bits, respectively. However, if using both, enable or disable them concurrently. These data lines operate in a synchronous manner (sharing a clock and frame sync) but have separate datapaths with unique data buffers, shift registers and optional companding logic. All of the SPORT control settings are common for both channels, but the single DMA channel per half SPORT serves both the primary and secondary data channels.

When a SPORT is configured in multichannel transmit mode, the data pins three-state during inactive channel slots, thus allowing multiple transmitters to operate on the same bus with different active channels.

See the [Architectural Concepts](#) section for more details about data transfer operation.

Transmit Data Valid Signal

The transmit data valid (`SPORT_ATDV`) signal is available only in transmit multichannel modes (including packed mode). It is driven active during enabled multichannel slots, and it is driven inactive during the disabled channels. In other words, the `SPORT_ATDV` signal is active when data is being driven to the data pins and inactive when the data pins are being three-stated. As such, the `SPORT_ATDV` signal can serve as an output-enable signal for the data transmit pin.

SRU Programming

The SPORT uses the SRU (signal routing unit) to connect the SPORT data, serial clock, frame sync, and external sync (if external synchronization is required). Inputs also must be routed through the SRU. Program the corresponding SRU registers to connect the outputs to the required destinations. For details of the routing, see DAI Routing Capabilities in the *Digital Audio Interface (DAI)* chapter.

SRU SPORT Receive Master

If the SPORT is operating as receive master, it must feed its master output clock back to its input clock. This is required to trigger the SPORT’s state machine. Using SPORT 0A as an example receive master, programs should route `SPT0_ACLK_O` to `SPT0_ACLK_I`. This is not required if the SPORT is operating as a transmitter in master mode.

Functional Description

The following sections provide general information about the functionality of the processor's serial ports:

- [Architectural Concepts](#)
- [Data Types and Companding](#)
- [Transmit Path](#)
- [Receive Path](#)

ADSP-2159x SPORT Register List

The Serial Port (SPORT) controller, with its range of clock and frame synchronization options, supports a variety of serial communication protocols and provides a glueless hardware interface to many industry-standard data converters and CODECs. Each SPORT has two independent halves (A and B), and each half contains two channels (primary and secondary). A set of registers governs SPORT operations. For more information on SPORT functionality, see the SPORT register descriptions.

Table 26-2: ADSP-2159x SPORT Register List

Name	Description
<code>SPORT_CS0_A</code>	Half SPORT 'A' Multichannel 0-31 Select Register

Table 26-2: ADSP-2159x SPORT Register List (Continued)

Name	Description
SPORT_CS0_B	Half SPORT 'B' Multichannel 0-31 Select Register
SPORT_CS1_A	Half SPORT 'A' Multichannel 32-63 Select Register
SPORT_CS1_B	Half SPORT 'B' Multichannel 32-63 Select Register
SPORT_CS2_A	Half SPORT 'A' Multichannel 64-95 Select Register
SPORT_CS2_B	Half SPORT 'B' Multichannel 64-95 Select Register
SPORT_CS3_A	Half SPORT 'A' Multichannel 96-127 Select Register
SPORT_CS3_B	Half SPORT 'B' Multichannel 96-127 Select Register
SPORT_CTL2_A	Half SPORT 'A' Control 2 Register
SPORT_CTL2_B	Half SPORT 'B' Control 2 Register
SPORT_CTL_A	Half SPORT 'A' Control Register
SPORT_CTL_B	Half SPORT 'B' Control Register
SPORT_DIV_A	Half SPORT 'A' Divisor Register
SPORT_DIV_B	Half SPORT 'B' Divisor Register
SPORT_ERR_A	Half SPORT 'A' Error Register
SPORT_ERR_B	Half SPORT 'B' Error Register
SPORT_MCTL_A	Half SPORT 'A' Multichannel Control Register
SPORT_MCTL_B	Half SPORT 'B' Multichannel Control Register
SPORT_MSTAT_A	Half SPORT 'A' Multichannel Status Register
SPORT_MSTAT_B	Half SPORT 'B' Multichannel Status Register
SPORT_RXPRI_A	Half SPORT 'A' Rx Buffer (Primary) Register
SPORT_RXPRI_B	Half SPORT 'B' Rx Buffer (Primary) Register
SPORT_RXSEC_A	Half SPORT 'A' Rx Buffer (Secondary) Register
SPORT_RXSEC_B	Half SPORT 'B' Rx Buffer (Secondary) Register
SPORT_TXPRI_A	Half SPORT 'A' Tx Buffer (Primary) Register
SPORT_TXPRI_B	Half SPORT 'B' Tx Buffer (Primary) Register
SPORT_TXSEC_A	Half SPORT 'A' Tx Buffer (Secondary) Register
SPORT_TXSEC_B	Half SPORT 'B' Tx Buffer (Secondary) Register

ADSP-2159x SPORT Interrupt List

Table 26-3: ADSP-2159x SPORT Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
81	SPORT0_A_DMA	SPORT0 Channel A DMA	Level	0
82	SPORT0_A_STAT	SPORT0 Channel A Status	Level	
83	SPORT0_B_DMA	SPORT0 Channel B DMA	Level	1
84	SPORT0_B_STAT	SPORT0 Channel B Status	Level	
85	SPORT1_A_DMA	SPORT1 Channel A DMA	Level	2
86	SPORT1_A_STAT	SPORT1 Channel A Status	Level	
87	SPORT1_B_DMA	SPORT1 Channel B DMA	Level	3
88	SPORT1_B_STAT	SPORT1 Channel B Status	Level	
89	SPORT2_A_DMA	SPORT2 Channel A DMA	Level	4
90	SPORT2_A_STAT	SPORT2 Channel A Status	Level	
91	SPORT2_B_DMA	SPORT2 Channel B DMA	Level	5
92	SPORT2_B_STAT	SPORT2 Channel B Status	Level	
93	SPORT3_A_DMA	SPORT3 Channel A DMA	Level	6
94	SPORT3_A_STAT	SPORT3 Channel A Status	Level	
95	SPORT3_B_DMA	SPORT3 Channel B DMA	Level	7
96	SPORT3_B_STAT	SPORT3 Channel B Status	Level	
97	SPORT4_A_DMA	SPORT4 Channel A DMA	Level	10
98	SPORT4_A_STAT	SPORT4 Channel A Status	Level	
99	SPORT4_B_DMA	SPORT4 Channel B DMA	Level	11
100	SPORT4_B_STAT	SPORT4 Channel B Status	Level	
101	SPORT5_A_DMA	SPORT5 Channel A DMA	Level	12
102	SPORT5_A_STAT	SPORT5 Channel A Status	Level	
103	SPORT5_B_DMA	SPORT5 Channel B DMA	Level	13
104	SPORT5_B_STAT	SPORT5 Channel B Status	Level	
105	SPORT6_A_DMA	SPORT6 Channel A DMA	Level	14
106	SPORT6_A_STAT	SPORT6 Channel A Status	Level	
107	SPORT6_B_DMA	SPORT6 Channel B DMA	Level	15
108	SPORT6_B_STAT	SPORT6 Channel B Status	Level	
109	SPORT7_A_DMA	SPORT7 Channel A DMA	Level	16
110	SPORT7_A_STAT	SPORT7 Channel A Status	Level	
111	SPORT7_B_DMA	SPORT7 Channel B DMA	Level	17

Table 26-3: ADSP-2159x SPORT Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
112	SPORT7_B_STAT	SPORT7 Channel B Status	Level	
297	SPORT0_A_DMA_ERR	SPORT0 Channel A DMA Error	Level	
298	SPORT0_B_DMA_ERR	SPORT0 Channel B DMA Error	Level	
299	SPORT1_A_DMA_ERR	SPORT1 Channel A DMA Error	Level	
300	SPORT1_B_DMA_ERR	SPORT1 Channel B DMA Error	Level	
301	SPORT2_A_DMA_ERR	SPORT2 Channel A DMA Error	Level	
302	SPORT2_B_DMA_ERR	SPORT2 Channel B DMA Error	Level	
303	SPORT3_A_DMA_ERR	SPORT3 Channel A DMA Error	Level	
304	SPORT3_B_DMA_ERR	SPORT3 Channel B DMA Error	Level	
305	SPORT4_A_DMA_ERR	SPORT4 Channel A DMA Error	Level	
306	SPORT4_B_DMA_ERR	SPORT4 Channel B DMA Error	Level	
307	SPORT5_A_DMA_ERR	SPORT5 Channel A DMA Error	Level	
308	SPORT5_B_DMA_ERR	SPORT5 Channel B DMA Error	Level	
309	SPORT6_A_DMA_ERR	SPORT6 Channel A DMA Error	Level	
310	SPORT6_B_DMA_ERR	SPORT6 Channel B DMA Error	Level	
311	SPORT7_A_DMA_ERR	SPORT7 Channel A DMA Error	Level	
312	SPORT7_B_DMA_ERR	SPORT7 Channel B DMA Error	Level	

ADSP-2159x SPORT Trigger List

Table 26-4: ADSP-2159x SPORT Trigger List Masters

Trigger ID	Name	Description	Sensitivity
90	SPORT0_A_DMA	SPORT0 Channel A DMA	Edge
91	SPORT0_B_DMA	SPORT0 Channel B DMA	Edge
92	SPORT1_A_DMA	SPORT1 Channel A DMA	Edge
93	SPORT1_B_DMA	SPORT1 Channel B DMA	Edge
94	SPORT2_A_DMA	SPORT2 Channel A DMA	Edge
95	SPORT2_B_DMA	SPORT2 Channel B DMA	Edge
96	SPORT3_A_DMA	SPORT3 Channel A DMA	Edge
97	SPORT3_B_DMA	SPORT3 Channel B DMA	Edge
98	SPORT4_A_DMA	SPORT4 Channel A DMA	Edge

Table 26-4: ADSP-2159x SPORT Trigger List Masters (Continued)

Trigger ID	Name	Description	Sensitivity
99	SPORT4_B_DMA	SPORT4 Channel B DMA	Edge
100	SPORT5_A_DMA	SPORT5 Channel A DMA	Edge
101	SPORT5_B_DMA	SPORT5 Channel B DMA	Edge
102	SPORT6_A_DMA	SPORT6 Channel A DMA	Edge
103	SPORT6_B_DMA	SPORT6 Channel B DMA	Edge
104	SPORT7_A_DMA	SPORT7 Channel A DMA	Edge
105	SPORT7_B_DMA	SPORT7 Channel B DMA	Edge

Table 26-5: ADSP-2159x SPORT Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
60	SPORT0_A_DMA	SPORT0 Channel A DMA	Pulse
61	SPORT0_B_DMA	SPORT0 Channel B DMA	Pulse
62	SPORT1_A_DMA	SPORT1 Channel A DMA	Pulse
63	SPORT1_B_DMA	SPORT1 Channel B DMA	Pulse
64	SPORT2_A_DMA	SPORT2 Channel A DMA	Pulse
65	SPORT2_B_DMA	SPORT2 Channel B DMA	Pulse
66	SPORT3_A_DMA	SPORT3 Channel A DMA	Pulse
67	SPORT3_B_DMA	SPORT3 Channel B DMA	Pulse
68	SPORT4_A_DMA	SPORT4 Channel A DMA	Pulse
69	SPORT4_B_DMA	SPORT4 Channel B DMA	Pulse
70	SPORT5_A_DMA	SPORT5 Channel A DMA	Pulse
71	SPORT5_B_DMA	SPORT5 Channel B DMA	Pulse
72	SPORT6_A_DMA	SPORT6 Channel A DMA	Pulse
73	SPORT6_B_DMA	SPORT6 Channel B DMA	Pulse
74	SPORT7_A_DMA	SPORT7 Channel A DMA	Pulse
75	SPORT7_B_DMA	SPORT7 Channel B DMA	Pulse

ADSP-2159x SPORT DMA Channel List

Table 26-6: ADSP-2159x SPORT DMA Channel List

DMA ID	DMA Channel Name	Description
DMA0	SPORT0_A_DMA	SPORT0 Channel A DMA

Table 26-6: ADSP-2159x SPORT DMA Channel List (Continued)

DMA ID	DMA Channel Name	Description
DMA1	SPORT0_B_DMA	SPORT0 Channel B DMA
DMA2	SPORT1_A_DMA	SPORT1 Channel A DMA
DMA3	SPORT1_B_DMA	SPORT1 Channel B DMA
DMA4	SPORT2_A_DMA	SPORT2 Channel A DMA
DMA5	SPORT2_B_DMA	SPORT2 Channel B DMA
DMA6	SPORT3_A_DMA	SPORT3 Channel A DMA
DMA7	SPORT3_B_DMA	SPORT3 Channel B DMA
DMA10	SPORT4_A_DMA	SPORT4 Channel A DMA
DMA11	SPORT4_B_DMA	SPORT4 Channel B DMA
DMA12	SPORT5_A_DMA	SPORT5 Channel A DMA
DMA13	SPORT5_B_DMA	SPORT5 Channel B DMA
DMA14	SPORT6_A_DMA	SPORT6 Channel A DMA
DMA15	SPORT6_B_DMA	SPORT6 Channel B DMA
DMA16	SPORT7_A_DMA	SPORT7 Channel A DMA
DMA17	SPORT7_B_DMA	SPORT7 Channel B DMA

Block Diagram

Each SPORT top module consists of two separate blocks, known as half SPORTs (HSPORT) A and B, each with identical functionality and programming models. The *Half Serial Port Block Diagram* shows a detailed block diagram of a half SPORT.

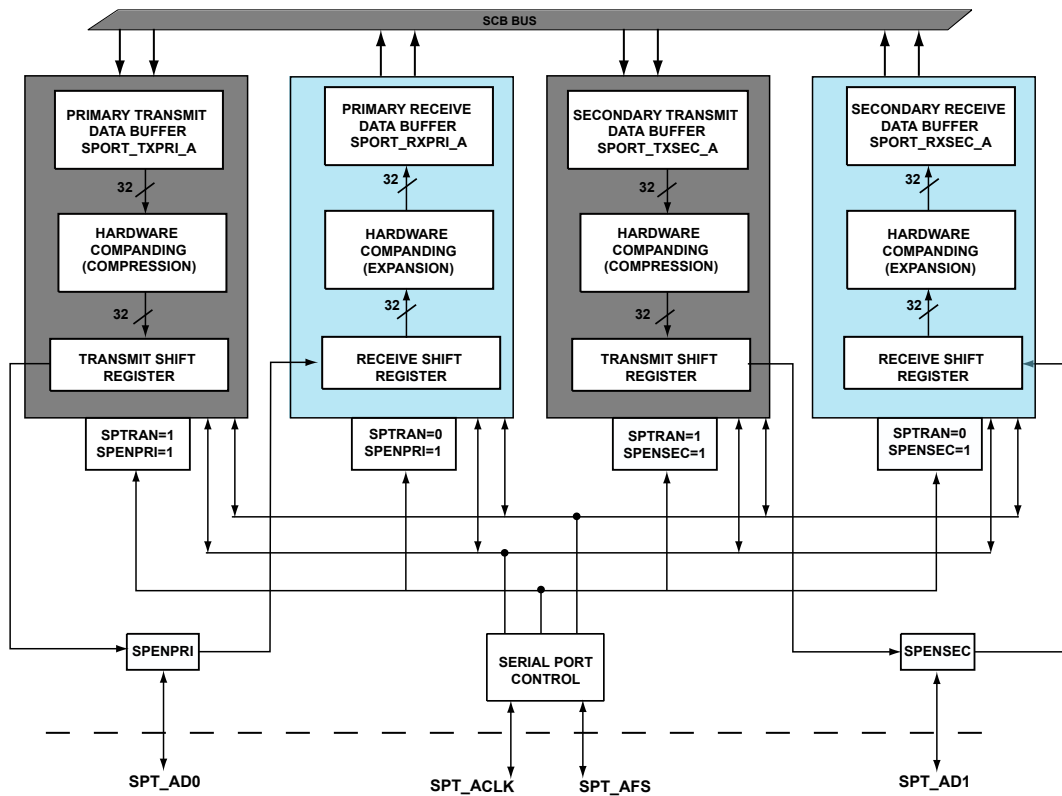


Figure 26-1: Half Serial Port Block Diagram

Architectural Concepts

Each half SPORT (HSPORT) block has its own set of control registers and data buffers, grouped per SPORT module. The HSPORT A and B blocks can be independently configured as either a transmitter or a receiver, with the option to be coupled together internally within the single SPORT top module. Each HSPORT also supports two synchronous bidirectional datapaths, referred to as the primary (D0) and secondary (D1) data lines, as shown in the *Top-Level SPORT Diagram* figure.

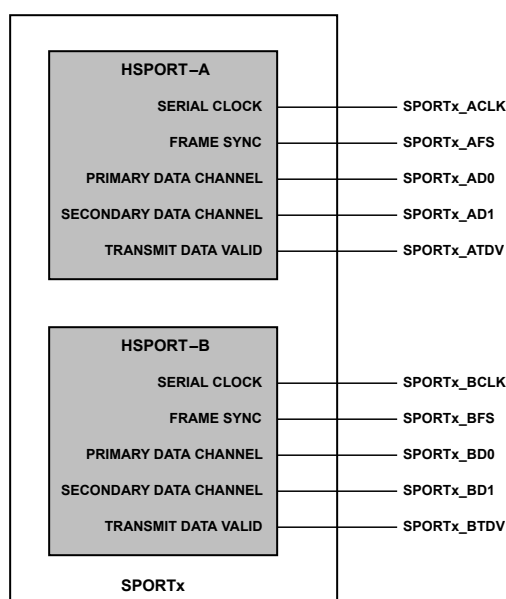


Figure 26-2: Top-Level SPORT Diagram

The `SPORT_CTL_A.SPTRAN` bit controls the direction for both datapaths of the HSPORT. Depending on whether the HSPORT is a transmitter or a receiver, the pair of data signals respectfully transmit or receive data bits synchronously. The dual data signals of each HSPORT cannot transmit and receive the data simultaneously in support of full-duplex operation, however, two HSPORTs can be combined to achieve this.

Serial communications are synchronized to the serial clock signal, where a valid clock pulse must accompany each data bit. Each HSPORT can take its clock from an external source or internally generate it from the processor's system clock using the `SPORT_DIV_A.CLKDIV` clock divisor bit field. Both primary and secondary data channels shift data based on the `SPORT_CLK` rate and the clock polarity defined by the `SPORT_CTL_A.CKRE` bit.

In addition to the serial clock signal, a frame synchronization signal is used to signify the beginning of an individual data word or a multichannel data stream (block of words). Each SPORT can take the frame sync signal from an external source or generate it (`SPORT_FS`), depending on the `SPORT_CTL_A.IFS` bit. An internally generated frame sync is derived from the SPORT clock using the `SPORT_DIV_A.FSDIV` divisor field. Both primary and secondary datapaths start shifting data either synchronous to or one serial clock in advance of detecting/generating a valid frame sync signal, as determined by the `SPORT_CTL_A.LAFS` bit. Various communication protocols for serial data can be emulated according to the frame sync format, and all frame sync options are available whether the signal is generated internally or externally.

NOTE: These SPORTs are not UARTs and cannot communicate with an RS-232 device or any other asynchronous communications protocol.

Multiplexer Logic

The SPORT multiplexing block (SPMUX) is situated between the SPORT hardware block and the processor's pin multiplexing logic. It allows the flexibility to route and share the clock and frame sync signals between the HSPORT A and B halves within each SPORT top module, which can double the data throughput (if both SPORT halves are transmitters or both are receivers) or provide full-duplex capabilities (if one HSPORT is a receiver and the other a

transmitter) without the need to allocate pins for the peripheral or make physical connections outside the processor. The `SPORT_CTL2_A` register is used to configure this loopback feature.

NOTE: Throughout this section, HSPORT A is used as a reference, but all concepts also apply to HSPORT B.

The multiplexing depends on the configuration of the `SPORT_CTL_A.IFS` and `SPORT_CTL_A.ICLK` bits, and the `SPORT_CTL2_A.CKMUXSEL` and `SPORT_CTL2_A.FSMUXSEL` bit settings control the multiplexing. The *Frame Sync Combinations* and *Clock Combinations* tables show the valid combinations for the bit settings.

NOTE: All other settings are illegal. However, hardware does not check or prevent the illegal settings. Ensure that programs use only legal combinations.

The column headers in the *Frame Sync Combinations* table are defined as follows:

- FS Combination = Frame sync combination, referenced in the notes that follow the *Clock Combinations* table.
- HSA_IFS = the setting of the `SPORT_CTL_A.IFS` configuration bit.
- HSB_IFS = the setting of the `SPORT_CTL_B.IFS` configuration bit.
- FSAMUX = the setting of the `SPORT_CTL2_A.FSMUXSEL` configuration bit.
- FSBMUX = the setting of the `SPORT_CTL2_B.FSMUXSEL` configuration bit.

The Routing column in the *Frame Sync Combinations* table defines how the signals are used between the SPORT halves and which pin is used for the frame sync (whether it is an input or an output). Within the column, the inequality characters ($\leq$ and $\geq$) are used to show the direction of the signal, and the following abbreviations are used (where x = A or B):

- HSx_FI = Frame sync input signal, provided by an external device or the complementing HSPORT.
- HSx_FO = Frame sync output signal.
- SPx_FS = HSPORT's frame sync pin, where the signal is either:
 - provided by an external source and distributed to both HSPORT frame sync signals, or
 - internally generated by one HSPORT and routed to both the pin and to the complementary HSPORT frame sync signal.

Table 26-7: Frame Sync Combinations

FS Combination	HSA_IFS	HSB_IFS	FSAMUX	FSBMUX	Routing
1	0	0	0	0	Native FS Operation
2	0	1	0	0	Native FS Operation
3	1	0	0	0	Native FS Operation
4	1	1	0	0	Native FS Operation

Table 26-7: Frame Sync Combinations (Continued)

FS Combination	HSA_IFS	HSB_IFS	FSAMUX	FSBMUX	Routing
5	0	0	1	0	$HSA_FI \leq SPB_FS$; $HSB_FI \leq SPB_FS$
6	0	1	1	0	$HSA_FI \leq HSB_FO \geq SPB_FS$
7	0	0	0	1	$HSB_FI \leq SPA_FS$; $HSA_FI \leq SPA_FS$
8	1	0	0	1	$HSB_FI \leq HSA_FO \geq SPA_FS$

The column headers in the *Clock Combinations* table are defined as follows:

- CLK Combination = Clock combination, referenced in the notes that follow the table.
- HSA_ICLK = the setting of the SPORT_CTL_A.ICLK configuration bit.
- HSB_ICLK = the setting of the SPORT_CTL_B.ICLK configuration bit.
- CKAMUX = the setting of the SPORT_CTL2_A.CKMUXSEL configuration bit.
- CKBMUX = the setting of the SPORT_CTL2_B.CKMUXSEL configuration bit.

The Routing column in the *Clock Combinations* table defines how the signals are used between the SPORT halves and which pin is used for the serial clock (whether it is an input or an output). Within the column, the inequality characters ($\leq$ and $\geq$) are used to show the direction of the signal, and the following abbreviations are used (x = A or B):

- HSx_CI = Serial clock input signal, provided by an external device or the complementing HSPORT.
- HSx_CO = Serial clock output signal.
- SPx_CLK = HSPORT's serial clock pin, where the signal is either:
 - provided by an external source and distributed to both HSPORT serial clock signals, or
 - internally generated by one HSPORT and routed to both the pin and to the complementary HSPORT serial clock signal.

Table 26-8: Clock Combinations

CLK Combination	HSA_ICLK	HSB_ICLK	CKAMUX	CKBMUX	Routing
9	0	0	0	0	Native CLK Operation
10	0	1	0	0	Native CLK Operation
11	1	0	0	0	Native CLK Operation
12	1	1	0	0	Native CLK Operation

Table 26-8: Clock Combinations (Continued)

CLK Combination	HSA_ICLK	HSB_ICLK	CKAMUX	CKBMUX	Routing
13	0	0	1	0	$HSA_CI \leq SPB_CLK$; $HSB_CI \leq SPB_CLK$
14	0	1	1	0	$HSA_CI \leq HSB_CO \geq SPB_CLK$
15	0	0	0	1	$HSB_CI \leq SPA_CLK$; $HSA_CI \leq SPA_CLK$
16	1	0	0	1	$HSB_CI \leq HSA_CO \geq SPA_CLK$

The following is a comprehensive list of the legal combinations for the above described frame sync and clock multiplexing configurations:

- FS Combinations 1–4 are compatible with all CLK Combinations (9–16)
- CLK Combinations 9–12 are compatible with all FS Combinations (1–8)
- FS Combination 5 is only compatible with CLK Combination 13 (and vice versa)
- FS Combination 6 is only compatible with CLK Combination 14 (and vice versa)
- FS Combination 7 is only compatible with CLK Combination 15 (and vice versa)
- FS Combination 8 is only compatible with CLK Combination 16 (and vice versa)

NOTE: Program only the `SPORT_CTL2` register of the HSPORT that is accepting the signal from the other HSPORT. However, be sure to set the `SPORT_CTL_A.CKRE` and `SPORT_CTL_A.LFS` polarity bits to have identical settings between the HSPORTs when making internal connections via the SPMUX block.

Data Types and Companding

The SPORT uses the data type select field `SPORT_CTL_A.DTYPE` bit to specify one of the four data formats supported by serial ports. These formats apply to any of the operating modes of serial port.

Table 26-9: Data Type Bit Field Settings

DTYPE field	SPORT Receiver	SPORT Transmitter
00	Right-justify, zero-fill unused most significant bits	Normal operation
01	Right-justify, sign-extend unused most significant bits	Reserved
10	Expand using μ -law	Compress using μ -law
11	Expand using A-law	Compress using A-law

These formats apply to data words loaded into the SPORT transmit or receive data buffers. The first two data formats (00 and 01 values of `SPORT_CTL_A.DTYPE`) are applicable only when SPORT is configured as receiver.

When SPORT is configured as transmitter, only the significant bits are transmitted (per the field defined in control register). Therefore, the transmit data buffers are not actually zero-filled or sign-extended.

The other two data formats enable the companding logic on the transmit or receive path. Companding (compressing or expanding) is the process of logarithmically encoding and decoding data to minimize the number of bits sent. The SPORTs of the processor support the two most widely used companding algorithms, A-law, and μ -law. The algorithms are performed according to the CCITT G.711 specification.

If selected, companding applies to both the enabled data channels. When enabled as SPORT transmitter, writes to transmit buffer make the content compressed to 8 bits according to algorithm selected. (The content is zero filled to the width of the transmit word.) Similarly, if configured in receive mode, the 8 bits in the receive data buffers expand in right-justified, zero fill format per the algorithm selected. If companding is enabled in multichannel mode, it applies to all the active channels.

The compression for transmit data requires a minimum word length of 8 for proper function. If `SPORT_CTL_A.SLEN` is less than 7, then expansion does not work correctly. Also, if the data value is greater than 13-bit A-law or 14-bit μ -law maximum, it automatically compresses to the maximum value.

NOTE: The processor companding logic supports in-place companding feature. So, companding can be used for debug without enabling SPORT.

Companding as a Function

Since the values in the transmit and receive buffers are companded in place, the SPORT can use the companding hardware without transmitting or receiving data, which can be useful during testing or debugging. For companding to execute properly, program the SPORT registers prior to loading data values into the SPORT buffers.

To compress data in place without transmitting, use the following procedure:

1. Set the SPORT as a transmitter (`SPORT_CTL_A.SPTRAN = 1`) with both primary and secondary data channels disabled (`SPORT_CTL_A.SPENPRI = SPORT_CTL_A.SPENSEC = 0`).
2. Enable companding in the `SPORT_CTL_A.DTYPE` field.
3. Write a 32-bit data word to the transmit buffer.
4. Wait two system clock cycles to allow the SPORT companding hardware to reload the transmit buffer with the companded value. Any instructions that do not access the transmit buffer can be used to cause this delay.
5. Read the 8-bit compressed value from the transmit buffer.

To expand data in place, use the same sequence of operations with the receive buffer instead of the transmit buffer.

Transmit Path

When the `SPORT_CTL_A.SPTRAN` control bit is set, the HSPORT is in transmit mode. Primary and secondary transmit data paths are then enabled using the `SPORT_CTL_A.SPENPRI` and `SPORT_CTL_A.SPENSEC` bits, respectively. The primary and secondary datapaths are unique and identical, each including its own transmit data buffer, optional companding logic, and transmit shift register.

The data buffer on the primary transmit data path is `SPORT_TXPRI_A`, and the data buffer on the secondary transmit data path is `SPORT_TXSEC_A`. The transmit data buffer and output shift register form a FIFO type of structure. When packing is disabled (`SPORT_CTL_A.PACK = 0`), the SPORT can hold as many as three data words. If packing is enabled (`SPORT_CTL_A.PACK = 1`), the serial port can hold two packed data words at any time.

The transmit data for primary and secondary channels is written to the `SPORT_TXPRI_A` and `SPORT_TXSEC_A` transmit data buffers, respectively. The transmit data buffers can be accessed in core mode through the peripheral bus or in DMA mode through the DMA bus. When a SPORT is configured in transmit mode, the receive paths are deactivated and do not respond to serial clock or frame sync signals. Because the receive data buffers and receive shift registers are also deactivated, reading from an empty and inactive receive data buffer can cause the core to hang indefinitely.

NOTE: Be sure to avoid accesses to inactive data buffers. Such accesses can cause unpredictable SPORT behavior or a hang condition and are not reported in any status register.

This data can optionally be compressed in hardware according to the selected algorithm (μ -law or A-law) and then automatically transferred to the transmit shift register. The shift register, clocked by the `SPORT_ACLK` signal, then serially outputs this data on the `SPORT_AD0` and/or `SPORT_AD1` pins (if both are enabled, these output data bits are transmitted synchronously). If the SPORT uses a framing signal, the `SPORT_AFS` signal indicates the start of the serial word transmission.

When using DMA mode, a single DMA feeds the data buffers of the enabled channels (primary and/or secondary). When using both channels, interleave the data of these channels starting with the primary channel in the transmit buffer.

When the SPORT is configured in non-multichannel mode as a transmitter, the enabled SPORT data pins (`SPORT_AD0` and/or `SPORT_AD1`) are always driven. When a SPORT channel is enabled, data from the transmit data buffer is loaded into the transmit shift register. The shift register then immediately latches the first bit of data (either the LSB or MSB, depending on the `SPORT_CTL_A.LSBF` configuration bit) and drives it to the respective data pin such that it is ready when the frame sync signal asserts. Similarly, if the frame sync period exceeds the serial word length, then the data pins are driven with the first bit of the next word for transmission immediately after the active word completes, and the outputs are held during the inactive serial clock cycles (clock cycles between frame sync pulses).

When the SPORT is configured in multichannel mode, the data pins are driven only during active transmit channels and are always three-stated during inactive channel slots.

The SPORT provides status of transmit data buffers and also error detection logic for transmit errors such as an underrun condition. See the [Error Detection \(Status\) Interrupt](#) section for more details.

Receive Path

When the `SPORT_CTL_A.SPTRAN` bit is cleared, the SPORT is in receive mode. Primary and/or secondary receive data paths can be enabled by setting the `SPORT_CTL_A.SPENPRI` and `SPORT_CTL_A.SPENSEC` configuration bits, respectively. These data paths are unique but identical, each with a receive shift register, optional companding logic, and a receive data buffer.

The data buffer on the primary receive path is `SPORT_RXPRI_A`, and the data buffer on the secondary receive path is `SPORT_RXSEC_A`. The receive data paths act like a three-deep (32-bit words) FIFO because they have two data registers plus an input shift register.

Upon enabling the SPORT data channels, the input shift register shifts in data bits on the `SPORT_AD0` and/or `SPORT_AD1` pins, synchronous to the SPORT clock signal. If the SPORT uses a framing signal, the `SPORT_AFS` signal indicates the beginning of the serial word (or frame) to be received. When an entire word is shifted into the primary and secondary channels, the data can be optionally expanded in hardware according to a selected algorithm (μ -law or A-law) and then automatically transferred to the `SPORT_RXPRI_A` and/or `SPORT_RXSEC_A` data buffers.

The receive data buffers can be read in core mode through the peripheral bus or in DMA mode through the DMA bus. When the SPORT uses DMA mode, a single DMA reads the data buffers of the enabled channels (primary and/or secondary) and interleaves them in memory beginning with the primary channel when both channels are enabled. When using both channels, software must de-interleave the data of these channels.

The SPORT provides the status of receive data buffers and also error detection logic for receive errors such as overflow. See the Error Detection (Status) Interrupt section for more details.

When a SPORT is configured in receive mode, the transmit paths are deactivated and do not respond to serial clock or frame sync signals. As the transmit data buffers and transmit shift registers in the data paths are also deactivated, programs must not try to access them.

NOTE: Be sure to avoid accesses to inactive data buffers. Such accesses can cause unpredictable SPORT behavior or a hang condition and are not reported in any status register.

Operating Modes and Options

The SPORT has a number of operating modes:

- Standard DSP Serial mode
- I²S mode
- Left-Justified mode
- Right-Justified mode
- Multichannel (TDM) mode
- Packed I²S mode

The SPORT halves within a SPORT top module can be independently configured in any of these operating modes unless they are coupled together using SPMUX logic, in which case they must be configured identically. Each half SPORT has its own set of control and data registers and is programmed similarly.

The *Control Bits for SPORT Operating Modes* table lists all the programmable configuration bits in the [SPORT_CTL_A](#) control register, which combine to determine the overall function and operating mode of the SPORT. The columns are arranged according to the setting of the `SPORT_CTL_A.OPMODE` bit that selects between standard DSP/multichannel modes and the various I²S modes, and the cell contents are defined as follows:

- Yes – bit is programmable for this mode of operation and may be written
- Reserved – bit is not programmable for this mode of operation and must not be written
- = value – bit must be set to this value to enable this mode of operation
- FUNCTION – indicates alternate function for this bit in this mode

NOTE: When changing operating modes, first clear the [SPORT_CTL_A](#) register before again writing the register with the new configuration settings.

Table 26-10: Control Bits for SPORT Operating Modes

Name (Bit #)	Standard DSP Serial Mode	I ² S and Left-Justified Mode	Right-Justified Mode	Multichannel (TDM) Mode	Packed I ² S Mode
SPENPRI (0)	Valid				
DTYPE (2:1)	Valid	Reserved		Valid	
LSBF (3)	Valid	Reserved		Valid	
SLEN (8:4)	Valid				
PACK (9)	Valid				
ICLK (10)	Valid				
OPMODE (11)	= 0	= 1	= 1	= 0	= 1
CKRE (12)	Valid				
FSR (13)	Valid	Reserved			
IFS (14)	Valid				
DIFS (15)	Valid			Reserved	
LFS (16)	Valid	L_FIRST/PLFS		Valid	L_FIRST/PLFS
LAFS (17)	Valid	OPMODE2	Valid	Reserved	
RJUST (18)	Reserved		= 1	Reserved	
FSED (19)	Valid	Reserved		Valid	Reserved
TFIEN (20)	Valid				
GCLKEN (21)	Valid		Reserved		

Table 26-10: Control Bits for SPORT Operating Modes (Continued)

Name (Bit #)	Standard DSP Serial Mode	I ² S and Left-Justified Mode	Right-Justified Mode	Multichannel (TDM) Mode	Packed I ² S Mode
SPENSEC (24)	Valid				
SPTRAN (25)	Valid				

Serial Word Length

The SPORT uses the `SPORT_CTL_A.SLEN` field to determine the word length of the serial data to transmit or receive. Each half SPORT can independently handle word lengths up to 32 bits, and the value that must be programmed to the `SPORT_CTL_A.SLEN` field is obtained from:

$$\text{SLEN} = \text{Desired SPORT word length} - 1$$

The minimal word length depends on the selected operating mode. Words smaller than 32 bits are right-justified in the transmit or receive buffers; however, data can be shifted in or out in MSB or LSB first format, as configured by the `SPORT_CTL_A.LSBF` bit. The received word can also be sign-extended or zero-filled when storing the data to processor memory, as governed by the `SPORT_CTL_A.DTYPE` bit.

The *Data Lengths for SPORT Operating Modes* table shows the range of valid word lengths for each of the supported SPORT operating modes.

Table 26-11: Data Lengths for SPORT Operating Modes

Mode	SPORT Word Length (SLEN+1)
Standard DSP Serial	4–32
I ² S	5–32
Left-Justified	5–32
Right-Justified	5–32
Multichannel (TDM)	5–32
Packed I ² S	5–32

NOTE: If the companding feature is enabled on the datapath, it limits the word length settings. See [Data Types and Companding](#) for more details about word lengths required for companding. If more than 32 bits per frame sync are required to transmit or receive, use the multichannel mode to spread the data across numerous continuous channels.

Clock Sample and Drive Edges

The SPORT uses two control signals to sample or drive the serial data:

1. Serial clock (`SPORT_ACLK`) - bit clock for the serial data.
2. Frame sync (`SPORT_AFS`) - divides the incoming data stream into frames.

These control signals can be internally generated or externally provided, as determined by the `SPORT_CTL_A.ICLK` and `SPORT_CTL_A.IFS` bit settings, respectively.

Data and frame syncs can be sampled on the rising or falling edges of the SPORT clock signal, as determined by the `SPORT_CTL_A.CKRE` bit. By default, the `SPORT_CTL_A.CKRE = 0` setting configures the falling edge of the `SPORT_ACLK` signal as the sampling edge for receive data and externally supplied frame syncs. The receive data and frame syncs can be sampled on the rising edges of `SPORT_ACLK` when `SPORT_CTL_A.CKRE = 1`.

NOTE: The SPORT drives transmit data and internal frame sync signals on the opposite serial clock edge of the sampling edge. Be sure to select the same value for `SPORT_CTL_A.CKRE` for transmit and receive functions for any two HSPORTs that are connected together, and always verify the correct polarity for any external device connected to the SPORT.

The *Frame Sync and Data Driven on Rising Edge* figure provides an example of the drive and sample edges when two HSPORTs are connected together, each with `SPORT_CTL_A.CKRE = 0`. In this example, the HSPORT that is configured as the transmitter drives the serial clock and frame sync signals, and both HSPORTs are configured for early, active high frame syncs and a word length of eight bits.

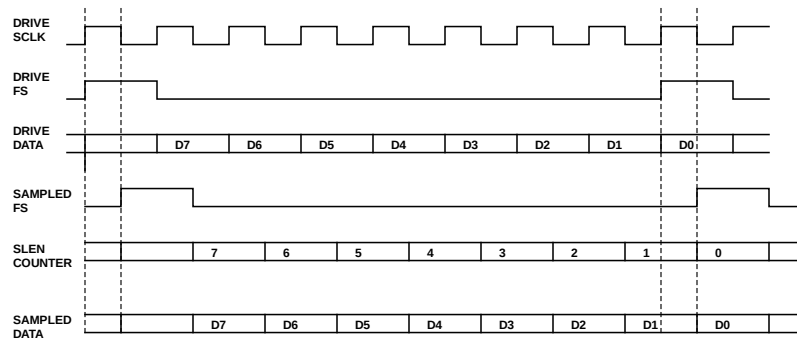


Figure 26-3: Frame Sync and Data Driven on Rising Edge

NOTE: The SCLK in the *Frame Sync and Data Driven on Rising Edge* figure is SCLK0.

As shown, the transmitting HSPORT provides the clock and generates the frame sync. Because the HSPORTs are configured for early frame mode, the first bit of data is driven one serial clock later, with subsequent bits being driven on the following rising clock edges in the signal train. When the receiving HSPORT samples the frame sync signal (as indicated in the SAMPLED FS waveform), the `SPORT_CTL_A.SLEN` bit counter is loaded with the `SPORT_CTL_A.SLEN` setting, after which each `SPORT_ACLK` decrements the `SPORT_CTL_A.SLEN` counter until the full word is received. In this figure, the DRIVE FS and SAMPLED FS waveforms show the frame sync required for the next word in a continuous data stream. Note that it is legal for this frame sync to be sampled synchronous to the last bit of the previous data being sampled, as the early frame mode means that the data lags the frame sync by one serial clock cycle. If the frame sync were sampled as asserted before the D0 bit is sampled, the frame sync error is logged in the receiver's status register.

Since the transmitter drives the internally-generated frame sync and data on the rising edge of the serial clock, the receiver must use the falling edge to sample the externally-supplied frame sync and data.

Frame Sync Options

The following sections provide details regarding the programmable aspects of the SPORT frame sync signal. See the specific operating mode sections for additional information regarding frame sync requirements and behavior for each specific operating modes.

Data-Dependent versus Data-Independent Frame Syncs

By default, the generation of a frame sync signal is data-dependent:

- When the SPORT is configured as a transmitter (`SPORT_CTL_A.SPTRAN = 1`), an internally generated transmit frame sync is output when a new data word has been loaded into the channel transmit buffer of the SPORT (by either the core or the DMA engine).
- When the SPORT is configured as a receiver (`SPORT_CTL_A.SPTRAN = 0`), an internally-generated receive frame sync is output only when the receive data buffer is not full.

The data-independent frame sync option, enabled by setting the `SPORT_CTL_A.DIFS` bit, allows for the generation of a periodic framing signal, regardless of the status of the data buffers. When this bit is set, the frame sync output will be continuous and periodic, according to the setting of the `SPORT_DIV_A.FSDIV` field.

Support for Edge-Detected and Level-Sensitive Frame Syncs

The level-sensitive nature of frame sync signals operates well in a noise-free environment. However, if noise corrupts the signals coming into the SPORT, the internal logic can lose synchronization. For example, excessive noise on the frame sync signal may cause the frame sync to be sampled as inactive on the clock edge that it is intended to be synchronous to, but then be sampled at the correct active level one cycle later. Similarly, a noisy clock signal can cause an unintended clock edge, resulting in potential premature sampling of the frame sync signal being applied to the pin.

The *Level-Sensitive Frame Sync versus Edge Sensitive Frame Sync* figure describes a scenario where an external frame sync signal is corrupted due to noise, causing the receiving SPORT module to incorrectly sample the signal. If the frame sync is driven on the rising edge of the serial clock at t_A , the SPORT would normally sample the signal on the falling edge of the serial clock at t_B . Due to the noise, however, the SPORT misses the first edge of the frame sync and instead samples it at t_C .

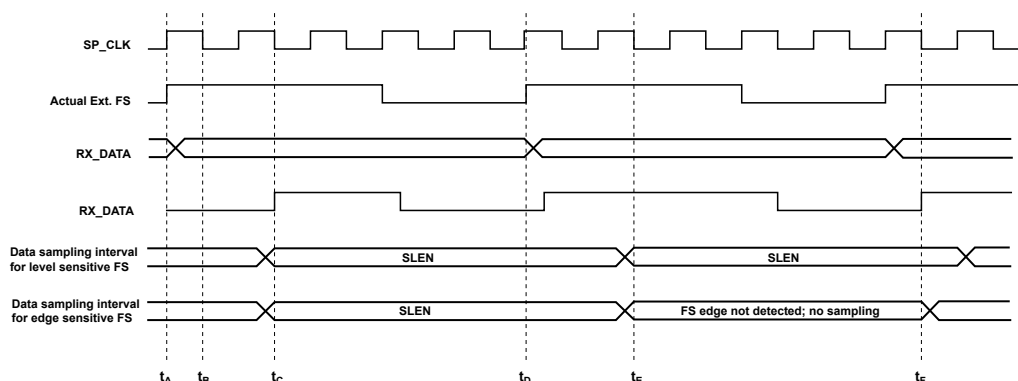


Figure 26-4: Level-Sensitive Frame Sync versus Edge Sensitive Frame Sync

NOTE: SCLK in the *Level-Sensitive Frame Sync versus Edge Sensitive Frame Sync* figure is SCLK0.

When the above occurs, the internal word length counter runs for a period equal to the `SPORT_CTL_A.SLEN` field of the control register, but it erroneously expires at t_E rather than at the appropriate point at t_D , thus receiving incorrect data. Further, if a new level-sensitive frame sync edge arrives at time t_D , the SPORT samples this framing signal again at t_E . As such, the frame sync sampling continues to be misaligned with the external data.

To help address this, the SPORT module provides an option to configure the frame sync signal to instead be edge-sensitive via the `SPORT_CTL_A.FSED` configuration bit. When this bit is set with active high frame syncs enabled (`SPORT_CTL_A.LFS = 0`), the rising edge of the frame sync is valid. Conversely, when the frame sync is active low (`SPORT_CTL_A.LFS = 1`), the falling edge is defined to be valid.

NOTE: `SPORT_CTL_A.FSED` is valid only in external frame sync mode. In internal frame sync mode, the setting of this bit is irrelevant and ignored.

In the above example, an edge-sensitive frame sync signal is not detected at t_E because the edge of the framing signal already occurred in the previous cycle (t_D) and there is no new edge to detect at t_E . As a result, the internal word length counter remains idle for this frame, thus ignoring the incorrect data, and the counter correctly resumes operation at t_F when a new frame sync edge is detected.

This activity sets the `SPORT_ERR_A.FSERRSTAT` bit and optionally generates a premature frame sync error interrupt.

Frame sync edge detection is used by default for stereo modes. MCM mode and DSP serial mode choose between edge detection and normal mode of FS detection.

NOTE: When the SPORT is first enabled, an already active externally applied frame sync will not commence operation. The SPORT will wait for a valid change in the frame sync's state from inactive to active before operation begins.

Early versus Late Frame Syncs

Frame sync signals can occur in the same serial clock cycle as the first bit of the data word (late) or one serial clock cycle before the first bit (early), as controlled by the `SPORT_CTL_A.LAFS` bit.

By default, the frame sync signal is configured to be early (`SPORT_CTL_A.LAFS = 0`). The first bit of the transmit data word will be driven one serial clock cycle after the frame sync is asserted (whether sensed externally or internally provided), and the first bit of the receive data word is expected to lag the frame sync by one serial clock cycle. The frame sync is not checked again until the entire word has been transferred.

If data transmission is continuous in early framing mode, then an internally-generated frame sync signal will be asserted (pulsed active for one serial clock cycle) synchronous to the last data bit of the current transfer, as the first bit of the next transfer will be immediately driven in the next serial clock cycle (no clocks are wasted). This event is not a premature frame sync error, so the `SPORT_ERR_A.FSERRSTAT` bit is not set.

The frame sync can alternatively be configured as late (`SPORT_CTL_A.LAFS = 1`), in which case the first bit of the transmit data word is available in the same serial clock cycle that the frame sync is asserted (whether sensed

externally or internally provided), and the first bit of the receive data word is also latched in the same cycle. Serial clock edges latch the receive data bits, but the frame sync signal is checked only during the first bit of each word. Internally generated frame syncs remain asserted for the entire length of the data word in late framing mode.

The *Normal Framing (Early Frame Sync) Versus Alternate Framing (Late Frame Sync)* figure illustrates these concepts.

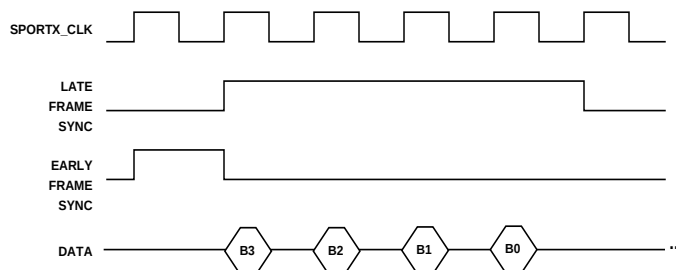


Figure 26-5: Normal Framing (Early Frame Sync) Versus Alternate Framing (Late Frame Sync)

Framed versus Unframed Frame Syncs

The use of a frame sync signal is optional for SPORT operation, as controlled by the `SPORT_CTL_A.FSR` bit. When the frame sync is configured to be required (`SPORT_CTL_A.FSR = 1`), the data is defined to be framed (a frame sync signal must accompany every data word). To allow continuous transmission from the processor, ensure that a new data word is loaded into the transmit buffer before the ongoing transfer is completed (this is automatically cared for when DMA is used to transmit blocks of data).

Data words can be transferred continuously in what is referred to as unframed data mode, which is appropriate for continuous reception, by setting `SPORT_CTL_A.FSR = 0`. In this configuration, a single frame sync is still required to initiate communication, but it is subsequently unrequired once the communication begins. From that point onward, externally provided frame syncs are ignored and internally generated frame syncs are not driven. The *Framed versus Unframed Data Stream* figure shows the differences in SPORT operation between framed and unframed data modes with the frame sync configured to be early (`SPORT_CTL_A.LAFS = 0`).

NOTE: When DMA is enabled in a mode where frame syncs are not required, chaining can delay DMA requests. DMA requests are not always serviced frequently enough to guarantee continuous unframed data flow. Monitor status bits or check for a SPORT error interrupt to detect underflow or overflow of data.

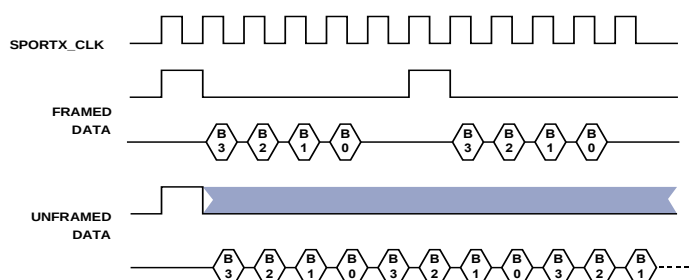


Figure 26-6: Framed versus Unframed Data Stream

Frame Sync Polarity

The framing signals can be active high or active low, as governed by the `SPORT_CTL_A.LFS` bit:

- When `SPORT_CTL_A.LFS = 0`, the corresponding frame sync signal is active high.
- When `SPORT_CTL_A.LFS = 1`, the corresponding frame sync signal is active low.

Active high is the default polarity of the frame sync signal.

Premature Frame Sync Error Detection

A SPORT framing signal is used to synchronize transmit or receive data. In external frame sync mode, any frame sync received during an active frame is premature and invalid. When this occurs, the `SPORT_ERR_A.FSERRSTAT` bit is set to indicate the framing error, and an optional error interrupt request can be generated for this event by setting the `SPORT_ERR_A.FSERRMSK` bit.

NOTE: The `SPORT_ERR_A.FSERRSTAT` bit is not set in the presence of uncleared underflow or overflow errors.

Refer to the *Frame Sync Error Detection* figure. The frame sync error bit gets set when an unexpected frame sync occurs during the ongoing data transfer (transmission or reception).

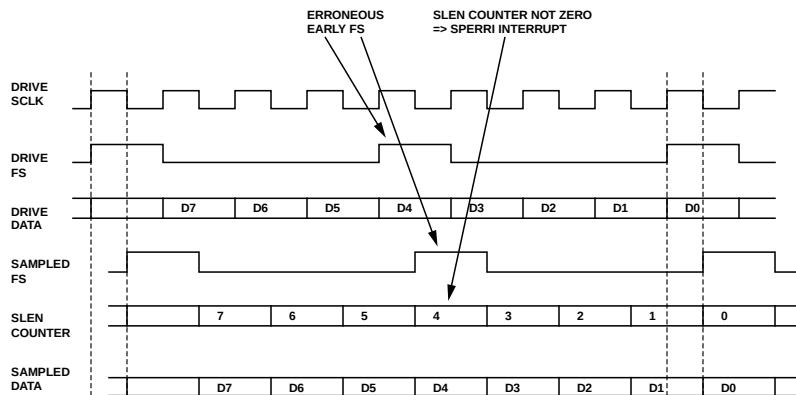


Figure 26-7: Frame Sync Error Detection

NOTE: SCLK in the *Frame Sync Error Detection* figure is SCLK0.

Whether a SPORT is receiving or transmitting, its bit count is set to the programmed serial word length when the frame sync is sampled, which is then decremented every subsequent serial clock cycle until the transfer has completed. At this point, the bit count reaches zero and will be reset to the programmed serial length when the next frame sync is sampled. As such, the bit count value is non-zero during an active transfer, and the frame sync error is asserted if a frame sync is sampled when this count is non-zero.

Mode Selection

The SPORT's operating mode is configured in the `SPORT_CTL_A` and `SPORT_MCTL_A` registers. The *SPORT Operating Modes* table provides specific guidance to properly program these SPORT control registers for the desired mode of operation.

Table 26-12: SPORT Operating Modes

Operating Modes	<code>SPORT_CTL_A.OPMODE</code>	<code>SPORT_CTL_A.LAFS</code>	<code>SPORT_CTL_A.RJUST</code>	<code>SPORT_MCTL_A.MCE</code>
Standard DSP Serial	0	Programmable	Reserved	0
I ² S	1	0	Reserved	0
Left-Justified	1	1	Reserved	0
Right-Justified	1	1	1	0
Multichannel	0	Reserved	Reserved	1
Packed I ² S mode	1	Reserved	Reserved	1

The following sections provide detailed information for each of the supported SPORT modes of operation.

Standard DSP Serial Mode

The SPORT can be configured in standard DSP serial mode by clearing the `SPORT_CTL_A.OPMODE` and `SPORT_MCTL_A.MCE` bits. This mode provides great flexibility in terms of programmable options to configure the SPORTs to communicate with various serial devices such as serial data converters and audio codecs. In order to properly connect to such devices, various clocking, framing, and data formatting options are available.

Timing Control Bits

Several bits in the `SPORT_CTL_A` control register define the configuration of the SPORT in standard DSP serial mode:

- **SLen**: serial word length (4–32 bits)
- **LSBF**: shift LSB or MSB first
- **ICLK**: internally generated or externally provided serial clock
- **CKRE**: sample on rising or falling edge of the serial clock
- **IFS**: internally generated or externally provided frame sync
- **FSR**: framed or continuous operation
- **DIFS**: data-dependent or data-independent frame sync
- **LFS**: active high or active low frame sync
- **LAFS**: frame sync synchronous to data or one clock cycle before it
- **PACK**: 16-bit to 32-bit packing option

- GCLKEN: free-running or gated clock

Clocking Options

In standard DSP serial mode, the SPORTs can either accept an external serial clock or generate one internally, as controlled by the `SPORT_CTL_A.ICLK` bit. For internally generated serial clocks (`SPORT_CTL_A.ICLK = 1`), the `SPORT_DIV_A.CLKDIV` field configures the serial clock rate from the system clock.

The SPORT clock can also be gated, where it is only valid during an active transfer, as controlled by the `SPORT_CTL_A.GCLKEN` bit.

The SPORT clock edge used for driving and sampling of serial data and frame syncs is configured using the `SPORT_CTL_A.CKRE` bit:

- If `SPORT_CTL_A.CKRE = 0`, input data and frame sync signals are sampled on the falling edge of the serial clock, and output data and frame sync signals are driven on the rising edge.
- If `SPORT_CTL_A.CKRE = 1`, input data and frame sync signals are sampled on the rising edge of the serial clock, and output data and frame sync signals are driven on the falling edge.

Stereo Modes

The SPORTs support three widely used stereo modes of operation:

- I²S mode
- Left-Justified mode
- Right-Justified mode

In these modes, the serial data stream consists of left and right channels. The following sections describe these modes in more detail.

Channel Order

The active low frame sync (`SPORT_CTL_A.LFS`) bit is used to determine the polarity of the frame sync signal in the non-stereo modes of operation. For the stereo modes of operation, it instead controls whether the right or left channel is first in the data transfer. The *Channel Order Bit Settings* table shows which word is transmitted or received first, based on the setting of the `SPORT_CTL_A.LFS` bit.

Table 26-13: Channel Order Bit Settings

Mode	<code>SPORT_CTL_A.LFS=0</code>	<code>SPORT_CTL_A.LFS=1</code>
Left-Justified or Right-Justified	Left channel first	Right channel first
I ² S or Packed I ² S	Right channel first	Left channel first

I²S Mode

I²S mode is a commonly used stereo mode, where left and right channel data words are interleaved in the serial data stream and each transition of the frame sync signal is associated with one of the channels. The left channel data is transferred during the low segment of the frame sync signal, and the right channel data is transferred during the high segment of the frame sync signal. As such, the frame sync signal is considered to be a left-right (L/R) clock in this mode.

To set the SPORT up in I²S mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 1`
- `SPORT_CTL_A.LFS = 0`
- `SPORT_MCTL_A.MCE = 0`

Protocol Configuration Options

Several bits in the `SPORT_CTL_A` control register must be configured to be compliant with the I²S standard, but they can be otherwise configured to support non-standard operation as well:

- **SLEN**: programmable (allowable word lengths are 5–32 bits)
- **LSBF**: set to 0 (MSB first)
- **ICLK**: programmable (serial bit clock can be internally generated or externally provided)
- **IFS**: programmable (serial L/R clock source must match serial bit clock source)
- **LFS**: set to 1 (left channel first)
- **CKRE**: set to 1 (sample L/R clock and data on rising edge of bit clock)

Serial Bit Clock and L/R Clock Rates

If the SPORT is configured to generate the bit clock and the L/R clock (`SPORT_CTL_A.ICLK = SPORT_CTL_A.IFS = 1`), set the serial bit clock rate using the `SPORT_DIV_A.CLKDIV` bit field and the L/R clock rate using the `SPORT_DIV_A.FSDIV` bit field.

The *Word Select Timing in I²S Mode* figure shows the SPORT timing in I²S mode. The data lags the L/R clock transition by one SCLK0 cycle, and the transfer begins with the left channel data word first.

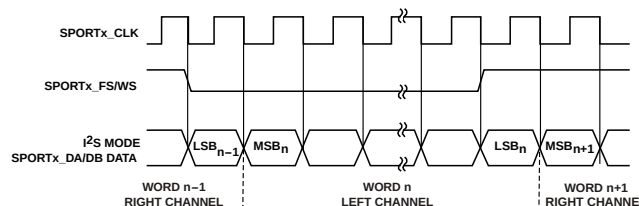


Figure 26-8: Word Select Timing in I²S Mode

Left-Justified Mode

Left-justified mode is a stereo mode subset of the I²S standard. As in I²S mode, the frame sync signal acts as a left-right clock (L/R clock), where left and right data samples are transferred each L/R clock period. The left channel is associated with the high segment of the frame sync, and the right channel aligns with the low segment of the frame sync. The difference between left-justified mode and standard I²S mode is that the channel data is driven in the same bit clock cycle as the L/R clock transition (rather than one bit clock cycle later), such that the MSB is synchronous with the leading edge of the frame sync transition.

To set the SPORT up in left-justified mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 1`
- `SPORT_CTL_A.LAFS = 1`
- `SPORT_MCTL_A.MCE = 0`

Protocol Configuration Options

Several bits in the `SPORT_CTL_A` control register must be configured to operate the SPORT in left-justified mode, but they can be otherwise configured as well:

- **SLEN**: programmable (allowable word lengths are 5–32 bits)
- **LSBF**: set to 0 (MSB first)
- **ICLK**: programmable (serial bit clock can be internally generated or externally provided)
- **IFS**: programmable (serial L/R clock source must match serial bit clock source)
- **LFS**: set to 0 (left channel first)
- **CKRE**: set to 1 (sample L/R clock and data on rising edge of bit clock)

Serial Bit Clock and L/R Clock Rates

If the SPORT is configured to generate the bit clock and the L/R clock (`SPORT_CTL_A.ICLK = SPORT_CTL_A.IFS = 1`), set the serial bit clock rate using the `SPORT_DIV_A.CLKDIV` bit field and the L/R clock rate using the `SPORT_DIV_A.FSDIV` bit field.

The *Word Select Timing in Left-Justified Mode* figure shows the SPORT timing in left-justified mode. The start of a data sample is synchronous to the L/R clock transition, and the transfer begins with the left channel data word first.

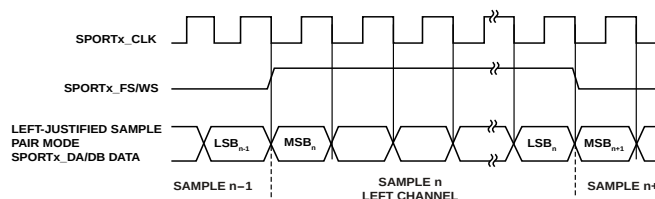


Figure 26-9: Word Select Timing in Left-Justified Mode

Right-Justified Mode

Right-justified mode is a stereo mode subset of the I²S standard. As in I²S mode and left-justified mode, the frame sync signal acts as a left-right clock (L/R clock), where left and right data samples are transferred each L/R clock period. The left channel is associated with the high segment of the frame sync, and the right channel aligns with the low segment of the frame sync. The difference between right-justified mode and standard I²S mode is that the LSB of the channel data ends at the point that the L/R clock transitions to frame the next sample (rather than one bit clock cycle after the L/R clock transition).

To set the SPORT up in right-justified mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 1`
- `SPORT_CTL_A.RJUST = 1`
- `SPORT_MCTL_A.MCE = 0`

Timing Control Bits

Several bits in the `SPORT_CTL_A` control register must be configured to operate the SPORT in right-justified mode, but they can be otherwise configured as well:

- **SLEN**: programmable (allowable word lengths are 5–32 bits)
- **LSBF**: set to 0 (MSB first)
- **ICLK**: programmable (serial bit clock can be internally generated or externally provided)
- **IFS**: programmable (serial L/R clock source must match serial bit clock source)
- **LFS**: set to 0 (left channel first)
- **CKRE**: set to 1 (sample L/R clock and data on rising edge of bit clock)

Serial Bit Clock and L/R Clock Rates

If the SPORT is configured to generate the bit clock and the L/R clock (`SPORT_CTL_A.ICLK = SPORT_CTL_A.IFS = 1`), set the serial bit clock rate using the `SPORT_DIV_A.CLKDIV` bit field and the L/R clock rate using the `SPORT_DIV_A.FSDIV` bit field.

The *Word Select Timing in Right-Justified Mode* figure shows the SPORT timing in right-justified mode. The transmitter aligns the transmit data such that the last bit of the serial word is sent in the last clock cycle of the L/R clock (frame sync) signal marking the channels.

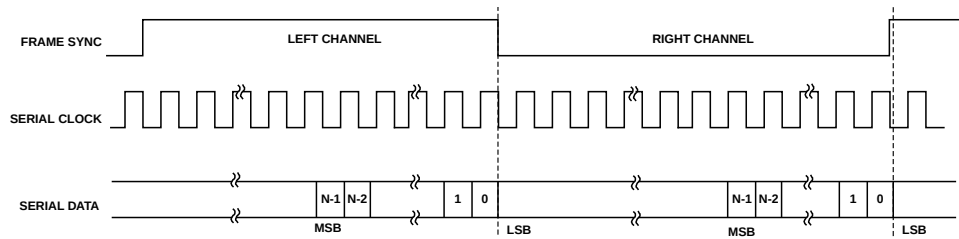


Figure 26-10: Word Select Timing in Right-Justified Mode

NOTE: For some SPORT-compatible ADCs or DACs such as the AD1871, right-justified mode is limited to commonly used ratios such as 64 FS and 128 FS. FS is the sampling frequency of ADCs and DACs, referred to as the SPORT's L/R clock (frame sync) signal.

Consider the SPORT timing for right-justified mode, as shown in the *Timing Comparison Between Different Stereo Modes* figure. The frame sync width is limited to 32 SPORT clock periods (or 32 bits per channel) if:

- the SPORT's frame sync (L/R clock) runs at the FS rate, and
- the SPORT's serial bit clock runs at the 64 FS rate

The limitation applies to the frame sync width of either channel. If the data is confined to 24 bits, the SPORT introduces a $32 - 24 = 8$ -bit clock delay before it starts to transmit or capture data.

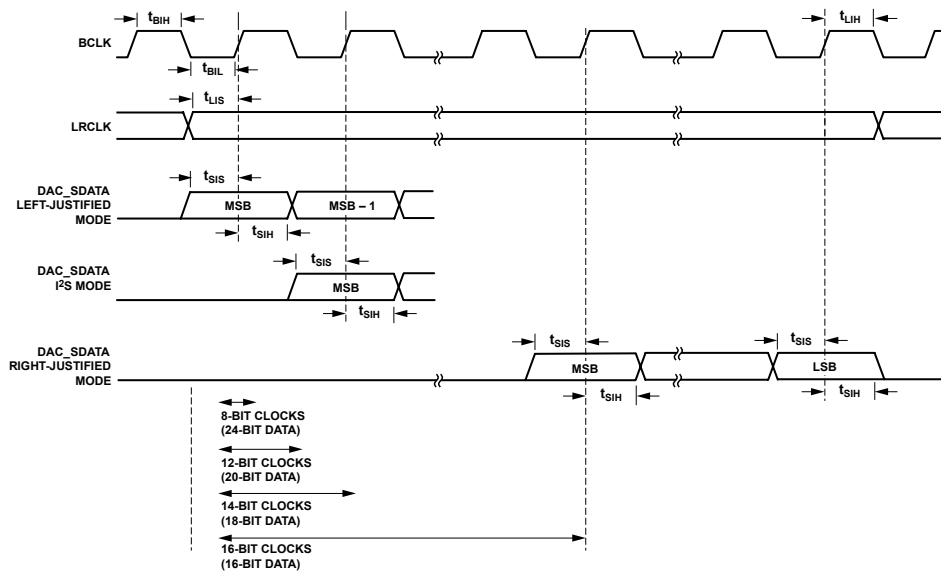


Figure 26-11: Timing Comparison Between Different Stereo Modes

Similarly, to support the 128 FS bit clock frequency, the frame sync width becomes 64 serial bit clock periods per channel. In this case, the delay can be a maximum of 59 bit clocks ($64 - 5$, which is the minimum serial data length in right-justified mode).

The starting point of the first bit is delayed so that the LSB of the serial data aligns properly with the end of the channel. A 6-bit counter is added for this purpose in the stereo mode counter, which is programmed by writing the least significant six bits of the `SPORT_MCTL_A.WOFFSET` field. Though this is a multichannel mode configuration register, the SPORT uses these bits in right-justified mode to configure the offset from the transition of the L/R

clock to where the first data bit must be driven in order to have the end of the last bit align properly with the next L/R clock transition. Software must program this register with the appropriate delay.

Multichannel (TDM) Mode

The multichannel mode of SPORT operation allows the SPORT to communicate as part of a time division multiplexed (TDM) serial system. In TDM communications, a large frame of streamed serial data words is defined to be a particular length. It consists of a specific number of channels, and each channel contains one serial data word of the defined data length. For example, a 24-word block of 24-bit data can be defined to be a window within a frame, having a duration of 576 bit clocks and comprised of 24 continuous channels. The SPORT is configured to transfer on specific channels within a defined window in this frame.

To set the SPORT up in multichannel mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 0`
- `SPORT_MCTL_A.MCE = 1`

In multichannel mode, the SPORT can selectively transfer data on up to a maximum window size of 128 continuous channels out of a maximum 1024-channel frame while ignoring all the disabled channels within the window and all the channels outside the window. The SPORT can do any of the following on each channel:

- Transmit data (`SPORT_CTL_A.SPTRAN = 1`)
- Receive data (`SPORT_CTL_A.SPTRAN = 0`)
- Do nothing (during inactive channels)

Channel selection is configured in the half SPORT multichannel select registers (`SPORT_CS0_A - SPORT_CS3_A`) before enabling SPORT operation for multichannel mode. Programming of these registers is especially important in DMA data unpacked mode, since the SPORT data buffers begin operation immediately after the SPORT data lines are enabled. Be sure to enable multichannel operation (set the `SPORT_MCTL_A.MCE` bit) prior to enabling the SPORT itself.

Clocking Options

In multichannel mode, the SPORTs can either accept an external serial clock or generate one internally, as governed by the `SPORT_CTL_A.ICLK` bit. For an internally-generated serial clock (`SPORT_CTL_A.ICLK = 1`), the `SPORT_DIV_A.CLKDIV` bit field is used to configure the serial clock rate, as derived from the system clock.

The serial clock edges used to drive and sample data and frame syncs are also configurable using the `SPORT_CTL_A.CKRE` bit:

- If `SPORT_CTL_A.CKRE = 0`, input data and frame sync signals are sampled on the falling edge of the serial clock, and output data and frame sync signals are driven on the rising edge.
- If `SPORT_CTL_A.CKRE = 1`, input data and frame sync signals are sampled on the rising edge of the serial clock, and output data and frame sync signals are driven on the falling edge.

Frame Sync Options

The frame sync signal synchronizes the channels and restarts each multichannel sequence, starting with the channel 0 data word. For internally-generated frame syncs (`SPORT_CTL_A.IFS = 1`), the frame sync period in multichannel mode is defined as:

$$\text{FS period} = [(\text{SPORT_CTL_A.SLEN} + 1) \times \text{number of channels}] - 1$$

The active level for the frame sync signal is also configurable by programming the `SPORT_CTL_A.LFS` bit. Set this bit to make the frame sync an active low signal, and clear it to make it active high.

In multichannel mode, frame sync timing resembles late framing mode (although the `SPORT_CTL_A.LAFS` bit is reserved in this mode). The first bit of the transmit data word is driven and the first bit of the receive data word is sampled in the same serial clock cycle as the frame sync, provided there is no programmed frame delay (`SPORT_MCTL_A.MFD = 0`).

Once the frame sync signal is asserted, word transfers are performed continuously for the duration of the active window, and no further frame syncs are required for different channels within the window. As such, internally-generated frame syncs are always data-independent, and the `SPORT_CTL_A.DIFS` bit is reserved.

Transmit Data Valid (TDV)

Each SPORT features a transmit data valid signal (`SPORT_ATDV`), which is driven high during enabled transmit channels. Because the SPORT output data signals are three-stated during inactive channels, the `SPORT_ATDV` signal signifies when the processor is actively driving the SPORT data outputs, thus serving as an output-enable signal for the data transmit pin(s).

Active Channel Selection Registers

In multichannel mode, the SPORT supports a window size of up to 128 channels for transmitting or receiving data, where it can selectively receive or transmit data in any of these 128 channels. Each channel can be individually enabled or disabled using the multichannel selection registers (`SPORT_CS0_A` to `SPORT_CS3_A`) to select the channels in which to transfer data during a multichannel communication stream. Data words associated with enabled channels are transmitted or received in the respective channels, while disabled channels cause a transmit SPORT to three-state the data output pins and a receive SPORT to ignore the data.

The four 32-bit multichannel selection registers combine to form up to a 128-bit meta-register to accommodate the maximum window size of 128 channels. Setting any bit within these registers enables the associated channel. The 128 channels are sequentially numbered from bit 0 in the `SPORT_CS0_A` register (corresponding to channel 0 of the window) to bit 31 of the `SPORT_CS3_A` register (corresponding to channel 127 of the window). For example, setting bit 13 of the `SPORT_CS1_A` register enables channel number 45 (add 32 for the channels in the `SPORT_CS0_A`). Likewise, setting bit 5 of the `SPORT_CS3_A` register enables channel number 101 (add 96 for the 32 channels in each of the `SPORT_CS0_A`, `SPORT_CS1_A`, and `SPORT_CS2_A` registers).

Multichannel Frame Delay (MFD)

The multichannel frame delay (`SPORT_MCTL_A.MFD`) field specifies the delay in serial bit clocks between the frame sync pulse and the first data bit in the frame. This configurability allows the processor to work with different types of telephony interface devices.

As `SPORT_MCTL_A.MFD` is a 4-bit field, the maximum value allowed for the frame delay is 15 serial clock cycles. When set to 0, the frame sync is concurrent with the first data bit. If `SPORT_MCTL_A.MFD > 0`, a new frame sync can occur during the last channel(s) of a previous frame and still be valid (does not cause a frame sync error).

NOTE: If the required frame delay exceeds 15 serial clocks, use the window offset field (`SPORT_MCTL_A.WOFFSET`) to delay the start of channel 0 in increments of the serial word length, and then adjust `SPORT_MCTL_A.MFD` accordingly. For example, if the serial word length is 12 bits and the desired frame delay is 16 serial clock cycles, set the `SPORT_MCTL_A.WOFFSET` to 1 to insert a 12-bit delay after the frame sync to where the channel 0 data begins, and then program `SPORT_MCTL_A.MFD` to 4 (i.e., $16 - 12$).

Window Size (WSIZE)

Select the number of channels used in multichannel operation by programming the 7-bit `SPORT_MCTL_A.WSIZE` field. This field must be set to the actual number of channels minus one (`SPORT_MCTL_A.WSIZE = Number of channels - 1`).

The 10-bit `SPORT_MSTAT_A.CURCHAN` field holds the channel number currently being serviced during multichannel operation.

Window Offset (WOFFSET)

The window offset (`SPORT_MCTL_A.WOFFSET`) field specifies where in the 1024-channel frame to place the start of the active window (up to 128 channels long). A value of 0 specifies no channel offset from the frame sync (channel 0 immediately follows it). Any non-zero value indicates the number of channels that come between the frame sync and the start of channel 0 of the active frame, with 896 (for example, $1024 - 128$) being the largest value that permits using all 128 channels.

As an example, a program could define an active window comprised of eight channels (`SPORT_MCTL_A.WSIZE = 7`) with a window offset of 93 (`SPORT_MCTL_A.WOFFSET = 93`). If configured in this fashion, the 8-channel window that the SPORT will transfer within resides in the channel range from 93 to 100 in the up-to-1024-channel frame.

Do not change the window offset or the number of multichannel slots (`SPORT_MCTL_A.WSIZE`) while the SPORT is enabled. If the combination of the window size and offset place any portion of the window out-of-range relative to the channel counter, none of the channels are enabled.

Companding Selection

Like the other operating modes, companding logic can optionally be applied to serial data (compression logic for transmit mode or expansion logic for receive mode). The two widely used companding algorithms, A-law and μ -law, are selectable using the `SPORT_CTL_A.DTYPE` field.

If companding is enabled, the companding algorithm is applied to both the primary and secondary datapaths. In multichannel mode, companding can be applied to either all or none of the enabled channels (companding cannot be selected on a per-channel basis).

Multichannel DMA Data Packing (MCPDE)

Multichannel DMA data packing and unpacking are enabled using the `SPORT_MCTL_A.MCPDE` bit.

When set, data is packed, and the SPORT expects the data in the DMA buffer to correspond only with enabled SPORT channels. For example, if only channels 1 and 9 are enabled in a 10-channel window (`SPORT_MCTL_A.WSIZE = 9`), the SPORT expects the buffer to be exactly two words in length, where channel 1 is associated with the first element in the buffer and channel 9 is associated with the second.

When cleared, data is unpacked, and the SPORT expects the DMA buffer to have a word for each of the channels in the active window, whether the channel is enabled or not. As such, the DMA buffer size must be exactly the size of the window. Using the same example as the packed case above, if only channels 1 and 9 are enabled in a 10-channel window (`SPORT_MCTL_A.WSIZE = 9`), then the DMA buffer size is ten words. The data at offsets 1 and 9 within the buffer are associated with the data transfers of channels 1 and 9, respectively. The rest of the words in the buffer are unused.

Packed I²S Mode

The SPORT supports a packed I²S mode, which can be used for audio codec communications using multiple channels. This mode allows applications to send more than the standard 32 bits per channel available through standard I²S mode. Packed mode is implemented using standard multichannel mode (and is therefore programmed similarly to multichannel mode).

To set the SPORT up in packed I²S mode, the following configuration is required:

- `SPORT_CTL_A.OPMODE = 1`
- `SPORT_MCTL_A.MCE = 1`

Like multichannel mode, packed I²S mode also supports a maximum of 128 channels, where up to 128 channels of data can be transferred for every transition of the frame sync signal acting as an L/R clock (for example, up to 128 left-channel words transfer during the high portion of the L/R clock, and up to 128 right-channel words transfer during the low portion).

As shown in the *Packed I²S Mode 128 Operation* figure, the packed waveforms are the same as those waveforms used in multichannel mode, except the frame sync is toggled for every frame and emulates I²S mode.

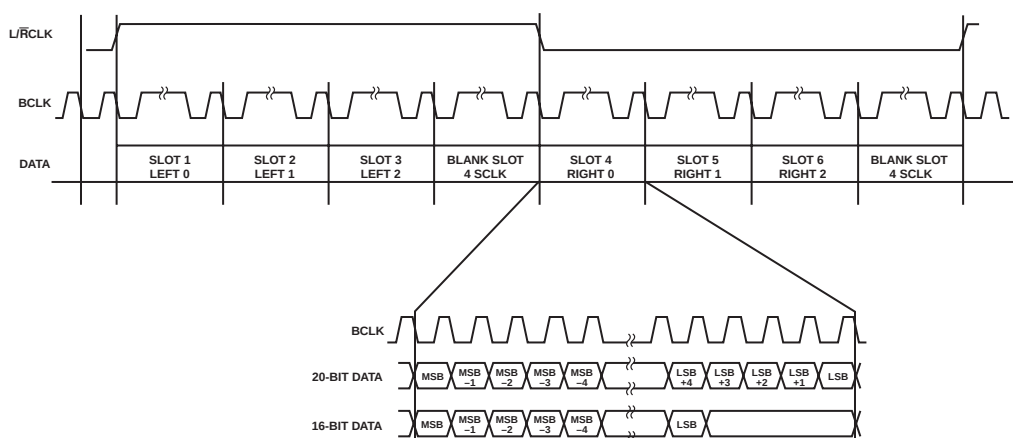


Figure 26-12: Packed I²S Mode 128 Operation

Serial Bit Clock Options

In packed I²S mode, the SPORTs can either accept an external serial bit clock or generate one internally, as governed by the `SPORT_CTL_A.ICLK` configuration bit. For an internally-generated serial bit clock (`SPORT_CTL_A.ICLK = 1`), use the `SPORT_DIV_A.CLKDIV` bit field to configure the serial bit clock rate from the system clock.

The serial bit clock edge that is used for sampling or driving data and frame syncs is programmable using the `SPORT_CTL_A.CKRE` bit.

L/R Clock (Frame Sync) Options

The frame sync period in packed I²S mode is defined as:

$$\text{FS period} = [(\text{SPORT_CTL_A.SLEN} + 1) \times \text{number of channels}] - 1.$$

The L/R clock can be supplied externally or internally generated depending on the `SPORT_CTL_A.IFS` bit setting. The logic level of the L/R clock associated with the left and right channel data can be changed using the `SPORT_CTL_A.LFS` configuration bit.

Gated Clock Mode

Some system components such as ADCs and DACs utilize a SPI-compatible protocol for the interface. To communicate with such devices, the SPORT must support a gated clock, where the data valid information is embedded in the clock (for example, the clock only toggles when data is valid). This gated clock feature is enabled using the `SPORT_CTL_A.GCLKEN` bit.

To enable the gated clock mode of operation, program the SPORT to comply with the following requirements.

- Do not enable gated clock functionality in right-justified or multichannel mode
- Gated clock mode has the following requirements for other control bits:

- The serial clock and frame sync signals must have the same source (`SPORT_CTL_A.ICLK = SPORT_CTL_A.IFS`)
- Unframed mode is not supported (`SPORT_CTL_A.FSR` must be set)
- Clear the `SPORT_CTL_A.DIFS` bit in transmit mode; set it in receive mode
- Satisfy the following necessary conditions when gated clock mode is enabled:
 - Seven serial clock cycles are required between enabling the SPORT and the first frame sync. If this requirement is not met, the SPORT can drop the first data (for subsequent data, this requirement is not applicable)
 - For externally-provided clock and frame sync, the frame sync must be inactive during clock synchronization after the SPORT has been enabled
 - For an edge-detected frame sync (`SPORT_CTL_A.FSED = 1`), the frame sync must transition back to the inactive state before the current word transfer is complete (or when the clock is still running). If this requirement is not met, the SPORT does not recognize the next valid frame sync and skips the channel. The SPORT continues to skip the frame syncs until the frame sync transitions back to an inactive state while the clock is active.

Data Transfers and Interrupts

SPORT data can be transferred to or from internal or external memory by two methods:

- Core-driven, single-word transfers
- DMA-driven, multiple-word transfers (optionally with multiple work units)

Core-driven transfers use SPORT interrupts to signal the processor core to perform MMR-based single-word transfers to or from the SPORT data buffers. DMA can be set up to automatically transfer a configurable number of serial words between the SPORT transmit/receive data buffers and memory, and then generate a data completion interrupt request when a work unit or a series of work units completes, thus signaling by the SEC to the processor core that a block of data has been transferred.

The following sections provide information on core-driven and DMA-driven data transfers.

Data Buffers

When programming the serial port data channels (primary or secondary) as a transmitter by setting `SPORT_CTL_A.SPTRAN = 1`, only the corresponding transmit data buffers (`SPORT_TXPRI_A` and `SPORT_TXSEC_A`) become active. The receive data buffers (`SPORT_RXPRI_A` and `SPORT_RXSEC_A`) remain inactive. Similarly, when the SPORT data channels are programmed for receive operation (`SPORT_CTL_A.SPTRAN = 0`), then only corresponding receive data buffers (`SPORT_RXPRI_A` and `SPORT_RXSEC_A`) are active. Do not attempt to read or write inactive data buffers. If the processor operates on the inactive transmit or receive buffers while the SPORT is enabled, unpredictable results can occur.

Each of these buffers is 32-bit wide (corresponds to maximum serial data word length). When using word lengths less than 32 bits for SPORT operation, the data in these buffers is automatically right-justified. (The LSB bit of data is at the bit 0 location of the buffer). The upper unused bits can be zero-filled or sign-extended depending on `SPORT_CTL_A.DTYPE` field.

Transmit Data Buffers (`SPORT_TXPRI_A` and `SPORT_TXSEC_A`)

When enabled as a transmitter (`SPORT_CTL_A.SPTRAN = 1`), each SPORT half has its own set of transmit data buffers. The primary (0) and secondary (1) datapaths of each SPORT half have separate data buffers, referred to as `SPORT_TXPRI_A` and `SPORT_TXSEC_A` respectively.

These transmit data buffers are 32 bits wide. Load these buffers with the data for transmission on the primary and secondary data channels. The DMA controller loads the data automatically. Or, the program running on the processor core loads the data manually.

Together with the output shift register, transmit data buffers act like a two-location FIFO. If data packing is disabled (`SPORT_CTL_A.PACK = 0`), the transmit path can hold as many as three data words. If data packing is enabled (`SPORT_CTL_A.PACK = 1`), it can hold two packed data words at any given time.

When the transmit shift register becomes empty (transfer out all the bits of previous word), data in the transmit data buffer is automatically loaded into it. An interrupt occurs when the output transmit shift register has been loaded, signifying that the transmit data buffer is empty and ready to accept the next word. This interrupt does not occur when serial port is operating in DMA mode or when the corresponding interrupt enable mask bit is set.

If only the primary datapath of a SPORT half is enabled, programs must not write to the inactive secondary transmit data buffer and conversely. If the core keeps writing to the inactive buffer, the status of that transmit buffer becomes full. This state can cause the core to hang indefinitely, since data is never transmitted to the output shift register.

Receive Data Buffers (`SPORT_RXPRI_A` and `SPORT_RXSEC_A`)

When enabled as receiver (`SPORT_CTL_A.SPTRAN = 0`), each SPORT half has its own set of receive data buffers. The primary (0) and secondary (1) datapaths of each SPORT half have separate data buffers, referred as `SPORT_RXPRI_A` and `SPORT_RXSEC_A` respectively. Together with input shift register, the receive data buffers act like a three-location FIFO, as the receive path has two data registers.

These receive data buffers are the 32 bits wide. These buffers are automatically loaded from the receive shift register when a complete word has been received into it. An interrupt occurs when the receive data buffer is loaded, signifying that new data is available in the receive data buffer and is ready to read. This interrupt does not occur when the serial port is operating in DMA mode or when the corresponding interrupt enable mask bit is set.

If only the primary datapath of a SPORT half is enabled, programs must not read from the inactive secondary receive data buffer and conversely. If the core keeps reading from the inactive buffer, the status of that receive buffer becomes empty. This state can cause the core to hang indefinitely since new data is never received through the input shift register.

Data Buffer Status

The SPORT provides status information about its primary and secondary data buffers through the `SPORT_CTL_A.DXSPRI` and `SPORT_CTL_A.DXSSEC` bits, respectively. It also provides error status information through the corresponding `SPORT_CTL_A.DERRPRI` and `SPORT_CTL_A.DERRSEC` bits, respectively. Depending on the `SPORT_CTL_A.SPTRAN` bit setting, these bits reflect the status of either the pair of transmit (`SPORT_TXPRI_A` and `SPORT_TXSEC_A`) or receive (`SPORT_RXPRI_A` and `SPORT_RXSEC_A`) buffers, indicating whether the buffer is full, partially full, or empty.

When attempting to read from an empty receive buffer or write to a full transmit buffer, the SPORT delays access until the buffer is ready, potentially resulting in excessive MMR bus response times. To avoid this when doing core-driven transfers, always check the buffer status to determine if the access can be made. The SPORT updates the status bits in the `SPORT_CTL_A` register during reads and writes by the core processor.

NOTE: These status bits are updated during reads and writes from the core processor even when the SPORT is disabled.

Two complete 32-bit words can be stored in the receive buffer while a third word shifts in. Therefore, almost three complete words can be received without the receive buffer being read before an overflow occurs. After receiving the third word completely, the shift register contents overwrite the second word, which will occur if the first word has not yet been read by the processor core or the DMA controller. This receive overflow condition is flagged through the error status bits of the `SPORT_CTL_A` register on the last bit of the third word.

Data Buffer Packing

When the SPORT is configured as a receiver with a serial data word length of 16 or less, the received data words can be packed into a 32-bit word. Similarly, if the SPORT is configured as a transmitter with a serial data word length of 16 or less, then 32-bit words being transmitted can be unpacked into 16-bit words. The `SPORT_CTL_A.PACK` bit is used to select this packing or unpacking feature.

When `SPORT_CTL_A.PACK = 1`, two consecutive received words are packed into a single 32-bit word, or each 32-bit word is unpacked and transmitted as two 16-bit words. The first 16-bit (or smaller) word is right-justified in bits 15–0 of the packed word, and the second 16-bit (or smaller) word is right-justified in bits 31–16. This packing method applies to both receive (packing) and transmit (unpacking) operations. In this case, the transmit and receive interrupt requests are generated for the 32-bit packed words, not for each 16-bit word.

NOTE: When 16-bit received data is packed into 32-bit words and stored in normal word space in the processor's internal memory, the 16-bit words can be read or written using short word space addressing.

Single-Word (Core) Transfers

The SPORTs can transmit or receive individual data words with interrupt requests occurring as each data word is transferred. When a SPORT is enabled with the corresponding DMA channel disabled, interrupt requests are generated when:

- a complete word has been received in the receive data buffer or

- the transmit data buffer is not full

When performing core transfers, be sure to access only those buffers that are associated with enabled datapaths, as governed by the transfer direction (`SPORT_CTL_A.SPTRAN`) bit and the primary/secondary (`SPORT_CTL_A.SPENPRI/SPORT_CTL_A.SPENSEC`) data enable bits. If inactive SPORT data buffers are read from or written to by the core while the SPORT is enabled, the core can hang. For example, if a half SPORT is programmed to be a transmitter and the core reads from one of the receive buffers associated with that half SPORT, the core can hang as if it were reading an empty buffer that is active and awaiting new data to arrive. Because this is a transmitting HSPORT, that data will never arrive, thus locking the core up until the SPORT is reset. To avoid such a situation, be sure to check the status of the appropriate data buffer before attempting a core access to it by interrogating the `SPORT_CTL_A.DXSPRI` or `SPORT_CTL_A.DXSSEC` status bits.

DMA Transfers

Direct memory access (DMA) provides a mechanism for transferring an entire block of serial data before an interrupt is generated. The processor's on-chip DMA controller automatically handles the DMA transfer, thus allowing the processor core to run in parallel until the entire block of data is transferred. When the interrupt request occurs, a service routine can then process the entire block of data (rather than react to single words), thus significantly reducing overhead.

Each half SPORT has a dedicated DMA channel that serves both the primary and secondary datapaths. When configured as a transmitter (`SPORT_CTL_A.SPTRAN = 1`) with both the primary and secondary datapaths enabled (`SPORT_CTL_A.SPENPRI = SPORT_CTL_A.SPENSEC = 1`), the DMA channel requires that the source DMA buffer interleave the data beginning with the primary channel, as it will alternately load to the primary and secondary transmit data buffers once it is enabled. The complementary operation is true in receive mode (`SPORT_CTL_A.SPTRAN = 0`) when both datapaths are enabled, as the DMA channel alternately reads from the primary and secondary receive data buffers and interleaves them in the destination DMA buffer. As such, software must de-interleave the data corresponding to the primary and secondary channels from the receive DMA buffer.

If the SPORT is configured in stereo mode, the same DMA channel handles both the left and right channels of both datapaths (primary and/or secondary). Therefore, for a transmit DMA with only one datapath enabled, the source buffer must be populated such that the left- and right-channel data is interleaved. If both datapaths are enabled, the DMA channel alternately loads to the primary and secondary transmit data buffers once it is enabled. As such, the interleaving requirement is for the primary left-channel data to be followed by the secondary left-channel data, then the primary right-channel data, and finally the secondary right-channel data. The complementary operation is true in receive mode, where the DMA channel alternately reads from the primary and secondary receive data buffers and interleave them in the destination DMA buffer. For the stereo modes of operation, the destination DMA buffer is interleaved as left-right data for a single data input. If both datapaths are enabled, the destination DMA buffer is written with the primary and secondary left-channel data followed by the primary and secondary right-channel data. As such, software must de-interleave the primary and secondary left- and right-channel data from the receive DMA buffer, as defined by this scheme.

Since both the primary and secondary datapaths share the single DMA channel, each half SPORT has a single interrupt request for data completion, as well as an error interrupt request. The DMA controller can generate an interrupt request at the end of a chain of DMA work units (when using multiple descriptors) or at the end of individual DMA work unit.

The SPORT DMA channels are assigned a higher priority than all the other DMA channels (for example, the SPI port). Having higher priority causes the SPORT DMA transfers to execute first when multiple DMA requests occur in the same cycle. The SPORT DMA channels are numbered and prioritized in the DMA channel list table in the DMA chapter.

Although the most efficient DMA transfers execute with 32-bit words, the SPORTs can handle word sizes from 4 to 32 bits (as defined by `SPORT_CTL_A.SLEN` field). If the serial data length is 16 bits or smaller, two pieces of data can be packed into 32-bit words for each DMA transfer, as selected by setting the `SPORT_CTL_A.PACK` bit. When this bit is set, the SPORT generates the transmit and receive interrupts for the 32-bit packed words, not for each 16-bit word. For more information, see the [Data Buffer Status](#) section.

NOTE: The SPORT DMA channel can access both internal memory and external memory of the processor without any core overhead.

Data Transfer Interrupt

Each half SPORT features a data transfer interrupt request that is shared by both the primary and secondary data channels in both transmit and receive modes. To determine the source of the data transfer interrupt request, applications can check the primary and secondary data buffer status bits (`SPORT_CTL_A.DXSPRI` and `SPORT_CTL_A.DXSSEC`, respectively).

When using core-driven transfers, this interrupt's meaning depends on the direction of the SPORT:

- As transmitter (`SPORT_CTL_A.SPTRAN = 1`) - the transmit data buffer is empty
- As receiver (`SPORT_CTL_A.SPTRAN = 0`) - new data is available in the receive data buffer

NOTE: When data packing is enabled (`SPORT_CTL_A.PACK = 1`), the core-driven transmit and receive interrupt requests are generated for 32-bit packed words, not for each 16-bit word.

In both cases, the interrupt request can be used to signal the core that an individual transfer has completed. For transmit operations, it indicates that the transmit data buffer can be safely loaded (either the buffer is already empty or the last data has moved from the data buffer to the shift register). For receive operations, it indicates that new data has arrived and can be read (or must be read before a subsequent word overwrites it).

When the SPORT is configured to use DMA to move data between memory and the peripheral (the most generic way to use dedicated DMA for sport data transfers), the same data transfer interrupt request instead indicates the completion of the transfer of a block of serial data (rather than a single word). When DMA is used, the DMA count register must be initialized to specify the number of words to transfer. This count decrements after each DMA transfer on the channel, and the data transfer interrupt request signal is asserted when the word count reaches zero (for example, a DMA work unit has finished).

For transmit DMA, the interrupt request is raised when the last word in the DMA work unit is loaded from the source memory to the HSPORT FIFO. This interrupt request can signal to the core that a new DMA work unit can be configured or that other software threads can now run. The transmit interrupt request can optionally be deferred until the last word of the work unit has fully shifted out of the shift register (see the Transfer Finish Interrupt (TFI) section for details).

For receive DMA, the interrupt request is raised when the last word is loaded to the destination memory. In addition to that described for transmit DMA, this interrupt request also serves as an indication to the core that there is a buffer of newly acquired data that is ready to be processed.

See the DMA chapter for further details regarding enabling of the DMA interrupt requests associated with the various modes of DMA operation.

NOTE: As a single DMA channel services both the primary and secondary datapaths associated with the SPORT, there is a single DMA completion interrupt request.

Transfer Finish Interrupt (TFI)

When configured for transmit DMA (`SPORT_CTL_A.SPTRAN = 1`), the data transfer interrupt request gets generated by the DMA engine itself when it decrements its count register upon loading the last element from memory to the HSPORT hardware. Alternately, the SPORT can use a Transmit Finish Interrupt (TFI) to signal the actual end of the transmission (for example, when the last bit of the last data word of the buffer has shifted out of the SPORT to the system) by setting the `SPORT_CTL_A.TFIEN` bit. When this bit is set, then DMA signal that would normally assert the data transfer interrupt request instead signals the SPORT that the DMA work unit is complete. The SPORT then waits until all the data in the FIFO is shifted out (including the transmit shift register) and asserts the TFI interrupt request upon completion.

NOTE: To enable this functionality in the DMA engine, be sure to configure the interrupt type field in the DMA configuration register for Peripheral interrupt. See the DMA chapter for further details.

Error Detection (Status) Interrupt

In addition to the dedicated data transfer interrupt request, each half SPORT also features an optional error status interrupt request that can be triggered when error conditions occur relative to data or frame syncs associated with the half SPORT.

Data-related errors depend on the direction of the SPORT and reflect overflow or underflow conditions, which are depicted in the `SPORT_CTL_A` control register as read-only sticky bits `SPORT_CTL_A.DERRPRI` and `SPORT_CTL_A.DERRSEC` (for the primary and secondary channels, respectively).

- When the SPORT is configured as a transmitter, these bits provide transmit data buffer underflow status. When the frame sync signal occurs when the transmit data buffer is empty, the underflow bit corresponding with the offending transmit data buffer is set, as the SPORT will transmit data whenever it detects a valid frame sync signal, whether new data is present or not.
- When the SPORT is configured as a receiver, these bits provide receive overflow status. When a channel receives new data while the receive buffer is already full, the new data overwrites the existing data, thus causing

an overflow. When this occurs, the overflow bit corresponding with the offending receive data buffer is set, as the SPORT receives data whenever it detects a valid frame sync signal, whether there is room in the receive buffer or not.

Each half SPORT also features an error register ([SPORT_ERR_A](#)), which is the source for the assertion of the described data-related error status bits. When a data-related error occurs on the primary or secondary datapaths, the error is logged in the `SPORT_ERR_A.DERRPSTAT` or `SPORT_ERR_A.DERRSSTAT` bits, respectively. To enable these status bits to generate the HSPORT status interrupt request in the SEC, the corresponding `SPORT_ERR_A.DERRPMSK` and `SPORT_ERR_A.DERRSMSK` bits must be set (for the primary and secondary datapaths, respectively).

The `SPORT_CTL_A.DERRPRI` and `SPORT_CTL_A.DERRSEC` channel error status bits are sticky read-only bits that can be cleared in two ways:

- Reset the error detection logic by disabling the channel associated with the error condition (clear the `SPORT_CTL_A.SPENPRI` or `SPORT_CTL_A.SPENSEC` control bit).
- Clear the source of the interrupt by writing-1-to-clear the `SPORT_ERR_A.FSERRSTAT`, `SPORT_ERR_A.DERRPSTAT`, or `SPORT_ERR_A.DERRSSTAT` status bits.

In addition to data-related errors, [SPORT_ERR_A](#) also tracks frame sync errors in the `SPORT_ERR_A.FSERRSTAT` status bit. Similar to the data-related errors, the frame sync error can be enabled as a source for raising the error status interrupt request via the SEC by setting the `SPORT_ERR_A.FSERRMSK` bit. A frame sync error occurs when the frame sync is detected prematurely, as explained in the [Premature Frame Sync Error Detection](#) section.

A frame sync error is not detected in the following cases:

- When there is no active transmit or receive data, and the frame sync pulse occurs due to noise on the input signal – if there is no active transfer, a noise-induced frame sync pulse is valid.
- If there is an active underflow or overflow error – frame sync errors cannot be detected because the SPORT error logic does not run after one of the data errors has occurred and remains unserved.
- When the frame sync pulse does not meet minimum timing requirements – if the frame sync pulse is shorter than a SPORT clock period, there is no guarantee that it gets sampled at all and may go unnoticed.

Grouping of SPORTs

For some applications, enabling multiple SPORTs need to be synchronized so that all of them start and/or end at the same time. It may also be required to generate/receive a single interrupt/trigger for the DMA transfer of all the SPORTs in the same group.

The [DAI_GBL_SP_EN](#) register is used to control enabling/disabling. The [DAI_GBL_INT_EN](#) register can be used to control interrupt/trigger generation/reception by multiple SPORTs in a group.

NOTE: All references to SPORT 0-3 in the context of DAI1 must be read as SPORT4-7.

SPORT Enable Grouping

The figure shows the block diagram of possible group enabling options for the SPORTs in DAI0 and DAI1.

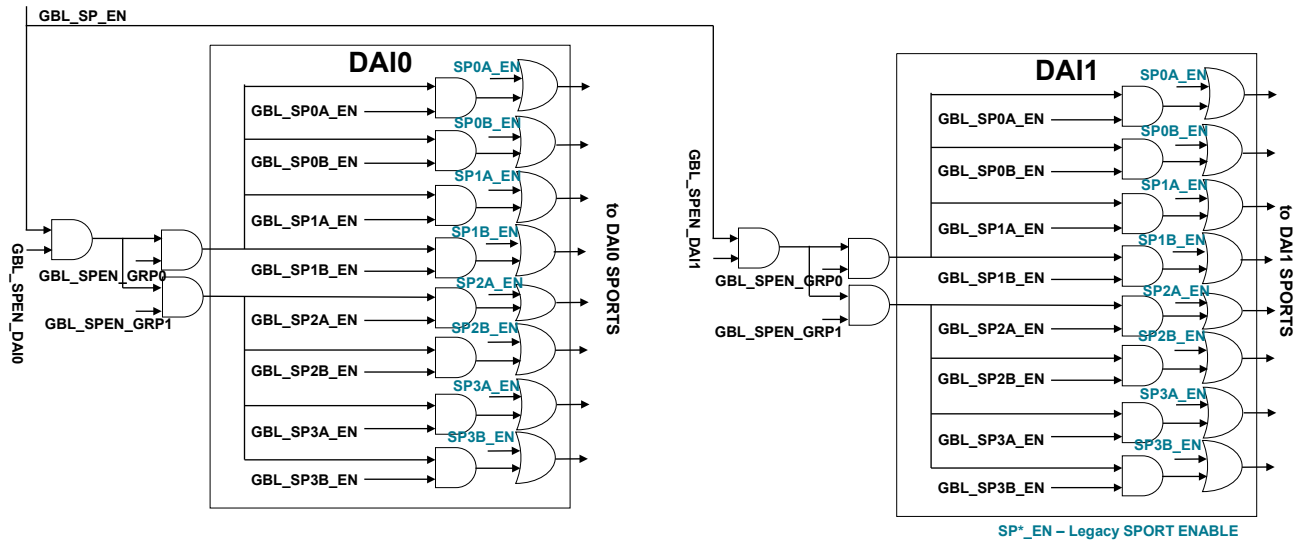


Figure 26-13: Group Enabling Options in DAI0 and DAI1

It shows one of the channels in each half sport, same is applicable for other channel as well. For example, GBL_SP0A_EN means GBL_SP0A_PC_EN or GBL_SP0A_SC_EN.

Each DAI has a `DAI_GBL_SP_EN` register. To enable the SPORTs grouped within the same DAI at the same time, write to the corresponding `DAI_GBL_SP_EN` register is required. If a group of SPORTs in different DAIs has to be enabled at the same time, write to the `DAI_GBL_SP_EN.GBL_SP_EN` field is required. This bit is reserved for the DAI1.

The following group enable options are possible:

- Up to 16 channels in 8 half SPORTs in DAI0
- Up to 16 channels in 8 half SPORTs in DAI1
- Up to 8 channels in 4 half SPORTs in each DAI
- Up to 32 channels in 16 half SPORTs (8 half SPORTs in DAI0 and 8 half SPORTs in DAI1)

NOTE: While grouping SPORTs across DAIs, two interrupts (one per DAI) are generated and managed by software. Grouping can be done only once as the `DAI_GBL_SP_EN.GBL_SP_EN` field is used.

NOTE: The grouping of SPORTs and its interrupts/triggers are two separate entities. If the interrupts and triggers must be grouped similar to the SPORT grouping, use the group interrupt/trigger enable register.

SPORT Interrupt/Trigger Grouping

Each DAI has a `DAI_GBL_INT_EN` register. Each of these registers has two group interrupt enable bits `DAI_GBL_INT_EN.GRP0_INT_EN` and `DAI_GBL_INT_EN.GRP1_INT_EN`, two group trigger (master)

enable bits `DAI_GBL_INT_EN.GRP0_TRG_EN` and `DAI_GBL_INT_EN.GRP1_TRG_EN`. There are individual `GRPx_SPyINT_EN` bits for each group to enable both interrupt/trigger for up to 8 channels (4 half SPORTs). Enable these bits as per the grouping in the `DAI_GBL_SP_EN` register.

NOTE: To enable or to disable the SPORT interrupt/trigger grouping feature, the individual sport interrupt/trigger enable bits must be programmed along with the respective group interrupt/trigger enable bits in the `DAI_GBL_INT_EN` register.

Regarding grouping of SPORT interrupts/triggers:

- All the interrupts/triggers in a single group are ANDed and a single interrupt/trigger goes to SEC/TRU.
- To generate group interrupt/trigger, all the SPORTs in the corresponding group should be configured to generate interrupt (`DMA_CFG.INT`)/trigger (`DMA_CFG.TRIG`).
- The interrupts/triggers generated by the SPORTs in a group can either be used as a single group interrupt/trigger (enabled by the `DAI_GBL_INT_EN` register) or the individual SPORT DMA done interrupts/triggers.
- As there are no separate status registers for global interrupts, to clear the global interrupts, clear (W1C) the individual SPORT `DMA_STAT.IRQDONE` bits. Theoretically, the global interrupt gets cleared by clearing the `DMA_STAT.IRQDONE` bit of at least one SPORT in the group. However, it is recommended to clear the `DMA_STAT.IRQDONE` bits of all the SPORTs in the group to achieve a clean state for the next global interrupt generation.
- The `DAI_GBL_INT_EN.GRP1_TRG_EN/DAI_GBL_INT_EN.GRP0_TRG_EN` bits enables group master trigger functionality for SPORTs in a group.

SPORT Programming Model

The following sections provide programming guidance for setting up the SPORTs for use in an application:

- [Initializing Core-Driven \(Non-MCM\) Transfers](#)
- [Initializing Multichannel Transfers](#)
- [Using DMA for SPORT Transfers](#)
- [Using Companding as a Function](#)

Initializing Core-Driven (Non-MCM) Transfers

The following programming model applies to all of [Standard DSP Serial Mode](#), [I²S Mode](#), [Left-Justified Mode](#), and [Right-Justified Mode](#) for core-driven transfers. More steps are required to properly initialize the SEC to service the SPORT interrupts (see the SEC chapter for details).

NOTE: This example uses half SPORT A registers. With appropriate changes to register names, this example also applies to half SPORT B.

1. Clear the `SPORT_CTL_A` and `SPORT_MCTL_A` configuration registers.

ADDITIONAL INFORMATION: Clearing these registers ensures that the SPORT logic (including the multi-channel logic) is fully reset before attempting to reprogram it.

2. Optionally program the `SPORT_DIV_A` clock divisor register.

ADDITIONAL INFORMATION: This step is only required for internally-generated timing signals. Configure the serial bit clock and/or frame sync (or L/R clock, for stereo modes) rates according to the guidance in the [Serial Clock](#) and [Frame Sync](#) sections.

3. Program the `SPORT_CTL_A` primary configuration register.

ADDITIONAL INFORMATION: Set the SPORT operating mode along with the configurable clock, frame sync, word length, direction, and data format options (see the [Operating Modes and Options](#) section for details). Do not set the `SPORT_CTL_A.SPENPRI` and/or `SPORT_CTL_A.SPENSEC` buffer enable bits in this step.

4. Optionally program the `SPORT_CTL2_A` secondary configuration register.

ADDITIONAL INFORMATION: This step is required only if internal multiplexing logic must be enabled to share clock and frame sync signals between a top SPORT module's A and B halves (see the [Multiplexer Logic](#) section for details).

5. Optionally program the `SPORT_ERR_A` error register.

ADDITIONAL INFORMATION: This step is required only if a separate SPORT error interrupt is desired (see the [Error Detection \(Status\) Interrupt](#) section for details).

6. For Right-Justified mode only, program the `SPORT_MCTL_A.WOFFSET` field.

ADDITIONAL INFORMATION: In Right-Justified mode, this field serves as the delay count (DCNT) required to align the LSB of each stereo channel with the L/R clock transition and must be programmed manually (see the [Right-Justified Mode](#) section for details).

7. Enable the primary/secondary datapath(s) in the `SPORT_CTL_A` register.

ADDITIONAL INFORMATION: This should be performed in a read-modify-write operation setting the `SPORT_CTL_A.SPENPRI` and/or `SPORT_CTL_A.SPENSEC` bits, as appropriate.

8. Write data to be transmitted to the transmit buffer (`SPORT_TXPRI_A` and/or `SPORT_TXSEC_A`) or read data that has been received from the receive buffer (`SPORT_RXPRI_A` and/or `SPORT_RXSEC_A`).

ADDITIONAL INFORMATION: These accesses are typically performed in the context of an interrupt service routine. See the SEC chapter for further information. Do not attempt to read or write inactive data buffers. If the core attempts to access inactive transmit or receive buffers while the SPORT is enabled, unpredictable results may occur.

Initializing Multichannel Transfers

When in [Multichannel \(TDM\) Mode](#) or [Packed I²S Mode](#), the SPORT is in a multichannel operational mode. Follow the steps below to properly initialize the SPORT for multichannel modes of operation. More steps are required to properly (see the SEC chapter for details).

NOTE: This example uses half SPORT A registers. With appropriate register changes, this example also applies to half SPORT B.

1. Clear the [SPORT_CTL_A](#) and [SPORT_MCTL_A](#) registers.

ADDITIONAL INFORMATION: Clearing these registers ensures that the SPORT logic (including the multichannel logic) is fully reset before attempting to reprogram it.

2. Optionally program the [SPORT_DIV_A](#) clock divisor register.

ADDITIONAL INFORMATION: This step is only required for internally-generated timing signals. Configure the serial bit clock and/or frame sync (or L/R clock, for stereo modes) rates according to the guidance in the [Serial Clock](#) and [SPORT_CTL2_A](#) sections.

3. Program the [SPORT_CS0_A](#) - [SPORT_CS3_A](#) channel select registers.

4. Program the [SPORT_MCTL_A](#) multichannel configuration register.

ADDITIONAL INFORMATION: The SPORT supports many multichannel options. For more information, see the [Multichannel \(TDM\) Mode](#) section. Do not set the [SPORT_MCTL_A.MCE](#) enable bit in this step.

5. Program the [SPORT_CTL_A](#) primary configuration register.

ADDITIONAL INFORMATION: Set the SPORT operating mode along with the configurable clock, frame sync, word length, direction, and data format options (see the [Operating Modes and Options](#) section for details). Do not set the [SPORT_CTL_A.SPENPRI](#) and/or [SPORT_CTL_A.SPENSEC](#) buffer enable bits in this step.

6. Optionally program the [SPORT_CTL2_A](#) secondary configuration register.

ADDITIONAL INFORMATION: This step is required only if internal multiplexing logic must be enabled to share clock and frame sync signals between a top SPORT module's A and B halves (see the [Multiplexer Logic](#) section for details).

7. Optionally program the [SPORT_ERR_A](#) error register.

ADDITIONAL INFORMATION: This step is required only if a separate SPORT error interrupt request is desired (see the [Error Detection \(Status\) Interrupt](#) section for details).

8. Set the [SPORT_MCTL_A.MCE](#) bit to enable multichannel mode.

9. Enable the primary/secondary datapath(s) in the [SPORT_CTL_A](#) register.

ADDITIONAL INFORMATION: This should be performed in a read-modify-write operation setting the `SPORT_CTL_A.SPENPRI` and/or `SPORT_CTL_A.SPENSEC` bits, as appropriate. DMA mode is recommended for multichannel modes of operation. For more information, see the [Using DMA for SPORT Transfers](#) programming model.

Using DMA for SPORT Transfers

DMA is supported in all SPORT operating modes ([Standard DSP Serial Mode](#), [I²S Mode](#), [Left-Justified Mode](#), [Right-Justified Mode](#), [Multichannel \(TDM\) Mode](#) or [Packed I²S Mode](#)). To enable DMA operation with the SPORT, execute the steps described in this section after initializing and enabling the SPORT. Instead of using the single word read or write operations described in the referenced programming models, the DMA engine automates accesses to the enabled SPORT data buffers.

NOTE: This example uses half SPORT A registers. With appropriate changes to register names, it also applies to half SPORT B.

1. Follow the guidance in the multichannel ([Initializing Multichannel Transfers](#)) or non-multichannel ([Initializing Core-Driven \(Non-MCM\) Transfers](#)) programming models to properly initialize and enable the SPORT hardware.
2. Prepare the data buffers in memory.

ADDITIONAL INFORMATION: Ensure that the DMA buffer is defined according to the [DMA Transfers](#) section. For the multichannel modes of operation, be sure to also consider the setting of the `SPORT_MCTL_A.MCPDE` bit, as described in the [Multichannel DMA Data Packing \(MCPDE\)](#) section.

3. Initialize and enable the DMA channel allocated for the SPORT, as described in the Direct Memory Access (DMA) chapter.

Using Companding as a Function

The data in the transmit and receive buffers are actually companded in place. As such, the following programming model can be used to exercise the companding hardware without transferring data, which is useful for test/debug purposes.

NOTE: This example uses half SPORT A registers. With appropriate changes to register names, this example also applies to half SPORT B.

1. Configure the SPORT as a transmitter (`SPORT_CTL_A.SPTRAN=1`) with both the primary and secondary data channels disabled (`SPORT_CTL_A.SPENPRI=0` and `SPORT_CTL_A.SPENSEC=0`).
2. Enable the desired companding scheme in the `SPORT_CTL_A.DTYPE` field.
3. Write a 32-bit word to one of the transmit buffers.
4. Wait two system clock cycles.

ADDITIONAL INFORMATION: This delay is required to allow the SPORT companding hardware to reload the transmit buffer with the companded result. Any instructions that do not access the transmit buffer can be used to cause this delay.

5. Read the 8-bit compressed value from the transmit buffer written above.

To expand data in place, use the same sequence of operations with the receive buffer instead of the transmit buffer. When expanding data in this way, set the appropriate serial word length (`SPORT_CTL_A.SLEN`).

Programming Global SPORT Groups

Complete the following steps to program global SPORT groups.

NOTE: To complete all of the SPORT group transfers simultaneously, all of the SPORTs in the group must have a similar data/clock/frame sync configuration.

1. For each SPORT, program the following registers for the desired SPORT mode of operation.

- `SPORT_DIV_A/SPORT_DIV_B`
- `SPORT_CTL2_A/SPORT_CTL2_B`
- `SPORT_CTL_A/SPORT_CTL_B`
- `SPORT_MCTL_A/SPORT_MCTL_B`
- `SPORT_CS0_A/SPORT_CS0_B`
- `SPORT_CS1_A/SPORT_CS1_B`
- `SPORT_CS2_A/SPORT_CS2_B`
- `SPORT_CS3_A/SPORT_CS3_B`

ADDITIONAL INFORMATION: The sport enable control bits (in `SPORT_CTL2_A/SPORT_CTL2_B` registers) must be cleared (disabled).

2. For a global group trigger as the master, program each SPORT DMA in the group as the master trigger.
3. Program the sport enable and group enable bits in the `DAI_GBL_SP_EN` register for the desired global sport grouping.

ADDITIONAL INFORMATION: The grouping of SPORTs (enable and disable) and the associated interrupts/triggers are separate activities. If the interrupts and triggers must be grouped similar to the SPORTs (enable and disable) grouping, use the `DAI_GBL_INT_EN` register.

4. Program the `DAI_GBL_INT_EN` register to enable the grouped interrupt or trigger generation for the SPORT group.

ADDITIONAL INFORMATION: To enable the SPORT grouped interrupt and trigger feature, each sport interrupt or trigger enable bit must be programmed with the respective group interrupt and trigger enable bits in the `DAI_GBL_INT_EN` register.

5. Program the `DAI_GBL_SP_EN.GBL_SP_EN` bit to enable the global sport groups at the same instance.

Disabling Global SPORT Groups

Complete the following steps to disable global SPORT groups.

1. Use the individual SPORT DMA status register to clear the group interrupt. There is no specific status register for global interrupts.
2. Disable the global SPORT group interrupt and trigger bits for the group in the `DAI_GBL_INT_EN` register.

ADDITIONAL INFORMATION: To disable the SPORT grouped interrupt and trigger feature, each sport interrupt and trigger enable bit must be programmed with the respective group interrupt and trigger enable bits in the `DAI_GBL_INT_EN` register.

3. Disable the global individual SPORT enable and the global SPORT group enable bits in the `DAI_GBL_SP_EN` register.
4. Disable the DMAs for the SPORTs when the transfer is complete.

ADSP-2159x SPORT Register Descriptions

Serial Port (SPORT) contains the following registers.

Table 26-14: ADSP-2159x SPORT Register List

Name	Description
<code>SPORT_CS0_A</code>	Half SPORT 'A' Multichannel 0-31 Select Register
<code>SPORT_CS0_B</code>	Half SPORT 'B' Multichannel 0-31 Select Register
<code>SPORT_CS1_A</code>	Half SPORT 'A' Multichannel 32-63 Select Register
<code>SPORT_CS1_B</code>	Half SPORT 'B' Multichannel 32-63 Select Register
<code>SPORT_CS2_A</code>	Half SPORT 'A' Multichannel 64-95 Select Register
<code>SPORT_CS2_B</code>	Half SPORT 'B' Multichannel 64-95 Select Register
<code>SPORT_CS3_A</code>	Half SPORT 'A' Multichannel 96-127 Select Register
<code>SPORT_CS3_B</code>	Half SPORT 'B' Multichannel 96-127 Select Register
<code>SPORT_CTL2_A</code>	Half SPORT 'A' Control 2 Register
<code>SPORT_CTL2_B</code>	Half SPORT 'B' Control 2 Register
<code>SPORT_CTL_A</code>	Half SPORT 'A' Control Register
<code>SPORT_CTL_B</code>	Half SPORT 'B' Control Register

Table 26-14: ADSP-2159x SPORT Register List (Continued)

Name	Description
SPORT_DIV_A	Half SPORT 'A' Divisor Register
SPORT_DIV_B	Half SPORT 'B' Divisor Register
SPORT_ERR_A	Half SPORT 'A' Error Register
SPORT_ERR_B	Half SPORT 'B' Error Register
SPORT_MCTL_A	Half SPORT 'A' Multichannel Control Register
SPORT_MCTL_B	Half SPORT 'B' Multichannel Control Register
SPORT_MSTAT_A	Half SPORT 'A' Multichannel Status Register
SPORT_MSTAT_B	Half SPORT 'B' Multichannel Status Register
SPORT_RXPRI_A	Half SPORT 'A' Rx Buffer (Primary) Register
SPORT_RXPRI_B	Half SPORT 'B' Rx Buffer (Primary) Register
SPORT_RXSEC_A	Half SPORT 'A' Rx Buffer (Secondary) Register
SPORT_RXSEC_B	Half SPORT 'B' Rx Buffer (Secondary) Register
SPORT_TXPRI_A	Half SPORT 'A' Tx Buffer (Primary) Register
SPORT_TXPRI_B	Half SPORT 'B' Tx Buffer (Primary) Register
SPORT_TXSEC_A	Half SPORT 'A' Tx Buffer (Secondary) Register
SPORT_TXSEC_B	Half SPORT 'B' Tx Buffer (Secondary) Register

Half SPORT 'A' Multichannel 0-31 Select Register

Each of the bits (when set, =1) of the `SPORT_CS0_A` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

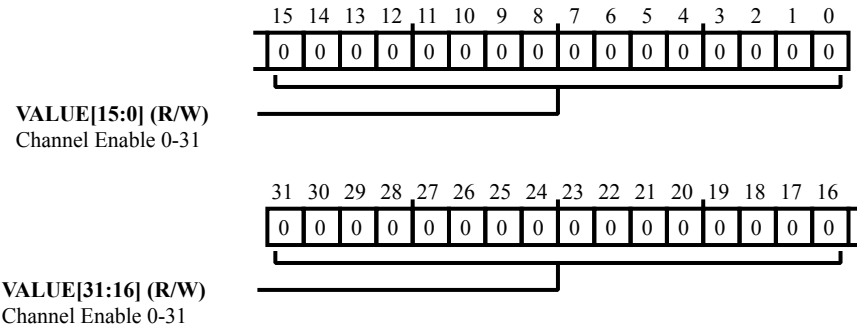


Figure 26-14: `SPORT_CS0_A` Register Diagram

Table 26-15: `SPORT_CS0_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 0-31.

Half SPORT 'B' Multichannel 0-31 Select Register

Each of the bits (when set, =1) of the `SPORT_CS0_B` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

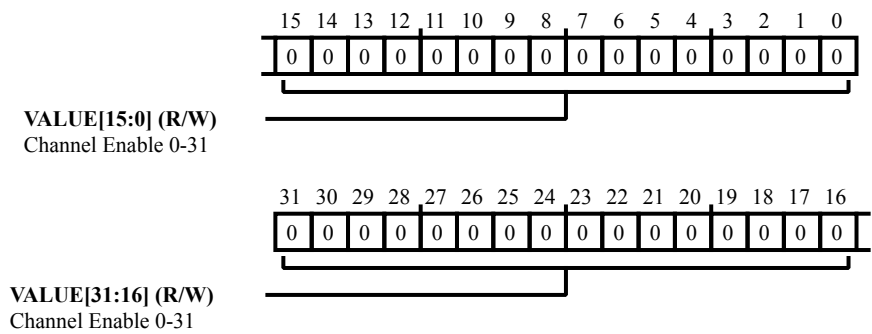


Figure 26-15: `SPORT_CS0_B` Register Diagram

Table 26-16: `SPORT_CS0_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 0-31.

Half SPORT 'A' Multichannel 32-63 Select Register

Each of the bits (when set, =1) of the `SPORT_CS1_A` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

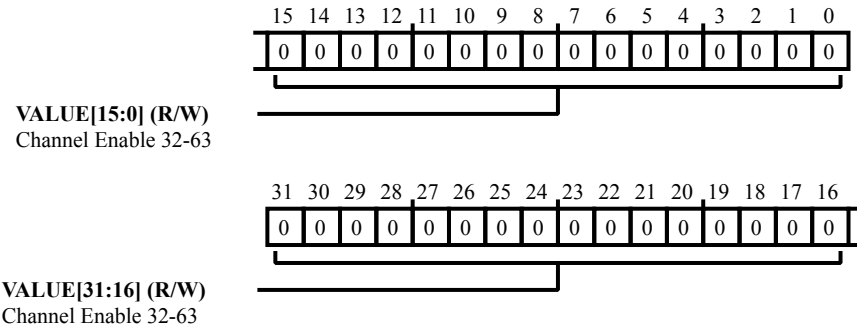


Figure 26-16: `SPORT_CS1_A` Register Diagram

Table 26-17: `SPORT_CS1_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 32-63.

Half SPORT 'B' Multichannel 32-63 Select Register

Each of the bits (when set, =1) of the `SPORT_CS1_B` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

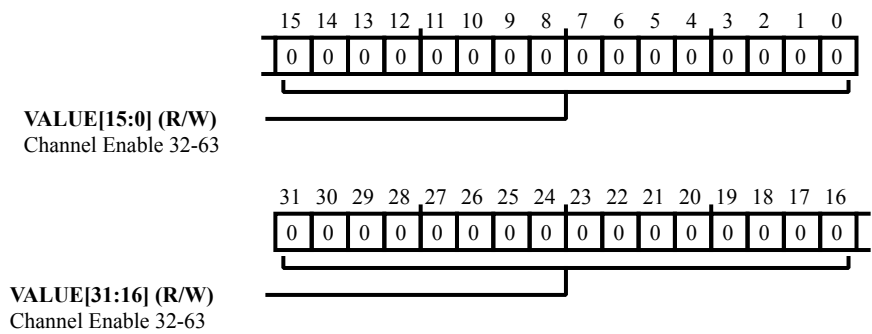


Figure 26-17: `SPORT_CS1_B` Register Diagram

Table 26-18: `SPORT_CS1_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 32-63.

Half SPORT 'A' Multichannel 64-95 Select Register

Each of the bits (when set, =1) of the `SPORT_CS2_A` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

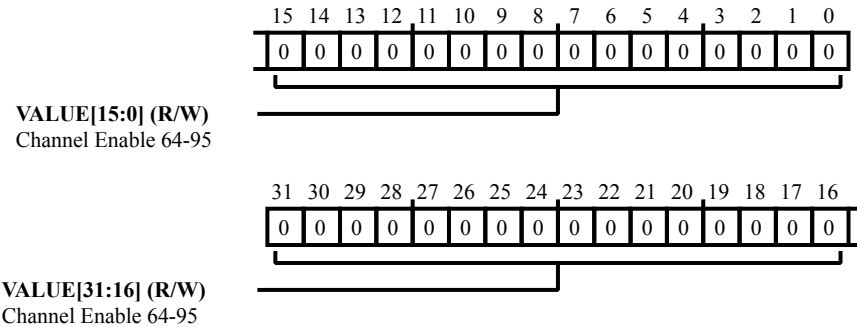


Figure 26-18: `SPORT_CS2_A` Register Diagram

Table 26-19: `SPORT_CS2_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 64-95.

Half SPORT 'B' Multichannel 64-95 Select Register

Each of the bits (when set, =1) of the `SPORT_CS2_B` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

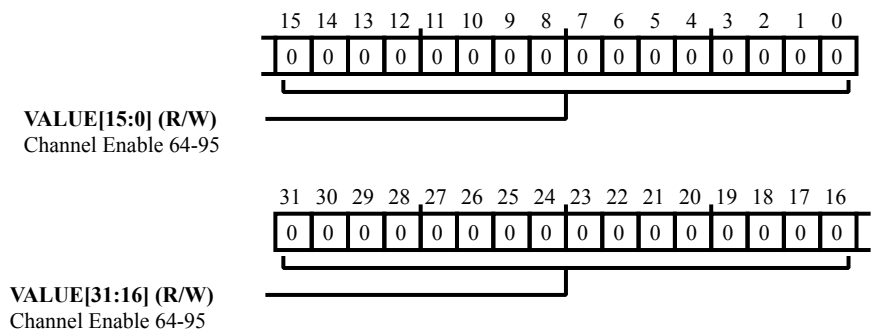


Figure 26-19: `SPORT_CS2_B` Register Diagram

Table 26-20: `SPORT_CS2_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 64-95.

Half SPORT 'A' Multichannel 96-127 Select Register

Each of the bits (when set, =1) of the `SPORT_CS3_A` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

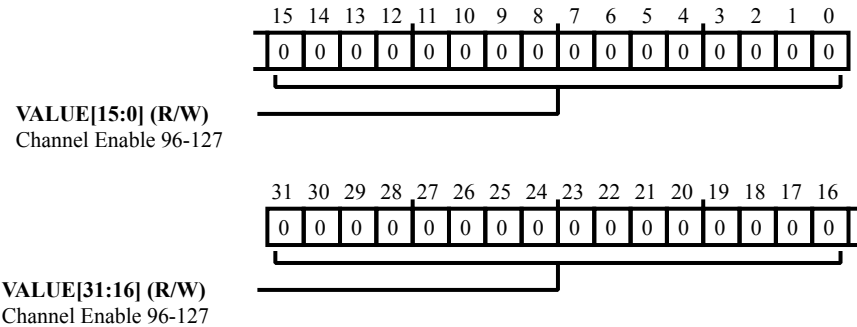


Figure 26-20: `SPORT_CS3_A` Register Diagram

Table 26-21: `SPORT_CS3_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 96-127.

Half SPORT 'B' Multichannel 96-127 Select Register

Each of the bits (when set, =1) of the `SPORT_CS3_B` register correspond to an active channel for the half SPORT in multichannel mode. When the register activates a channel (corresponding bit =1), the half SPORT transmits or receives the word in that channel's position of the data stream. When the register deactivates a channel (corresponding bit =0), the half SPORT either three-states its data transmit pin (during the channel's transmit time slot) or ignores incoming data (during the channel's receive time slot).

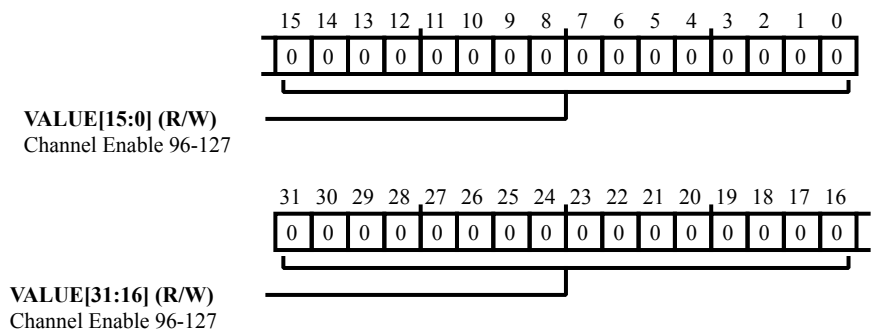


Figure 26-21: `SPORT_CS3_B` Register Diagram

Table 26-22: `SPORT_CS3_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Channel Enable 96-127.

Half SPORT 'A' Control 2 Register

The `SPORT_CTL2_A` register controls multiplexing options for sharing serial clock and frame sync signals across the related half SPORTs.

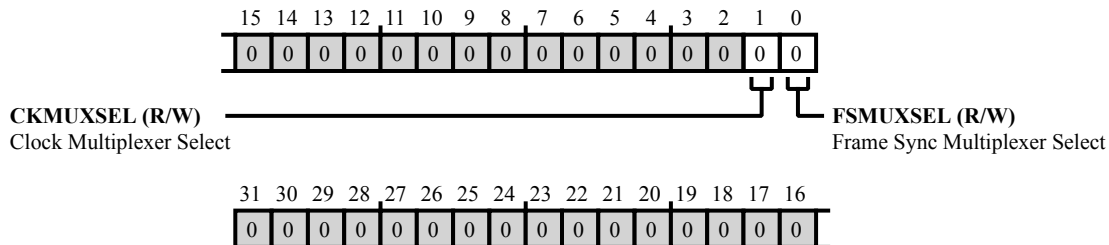


Figure 26-22: `SPORT_CTL2_A` Register Diagram

Table 26-23: `SPORT_CTL2_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	CKMUXSEL	Clock Multiplexer Select. The <code>SPORT_CTL2_A.CKMUXSEL</code> bit enables multiplexing of the half SPORT's serial clock. In this mode, the serial clock of the related half SPORT is used instead of the half SPORT's own serial clock. For example, if the <code>SPORT_CTL2_A.CKMUXSEL</code> bit is enabled, half SPORT 'A' uses <code>SPORT_BCLK</code> instead of <code>SPORT_ACLK</code> .
		0 Disable serial clock multiplexing
		1 Enable serial clock multiplexing
0 (R/W)	FSMUXSEL	Frame Sync Multiplexer Select. The <code>SPORT_CTL2_A.FSMUXSEL</code> bit enables multiplexing of the half SPORT's frame sync. In this mode, the frame sync of the related half SPORT is used instead of the half SPORT's own frame sync. For example, if the <code>SPORT_CTL2_A.FSMUXSEL</code> bit is enabled, half SPORT 'A' uses <code>SPORT_BFS</code> instead of <code>SPORT_AFS</code> .
		0 Disable frame sync multiplexing
		1 Enable frame sync multiplexing

Half SPORT 'B' Control 2 Register

The `SPORT_CTL2_B` register controls multiplexing options for sharing serial clock and frame sync signals across the related half SPORTs.

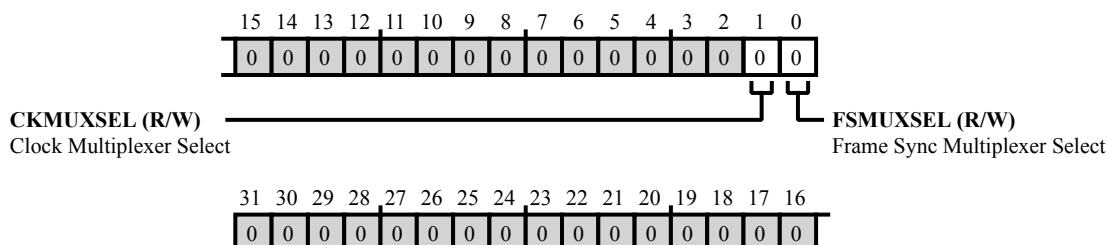


Figure 26-23: `SPORT_CTL2_B` Register Diagram

Table 26-24: `SPORT_CTL2_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	CKMUXSEL	Clock Multiplexer Select. The <code>SPORT_CTL2_B.CKMUXSEL</code> bit enables multiplexing of the half SPORT's serial clock. In this mode, the serial clock of the related half SPORT is used instead of the half SPORT's own serial clock. For example, if the <code>SPORT_CTL2_B.CKMUXSEL</code> bit is enabled, half SPORT 'B' uses <code>SPORT_ACLK</code> instead of <code>SPORT_BCLK</code> .
		0 Disable serial clock multiplexing
		1 Enable serial clock multiplexing
0 (R/W)	FSMUXSEL	Frame Sync Multiplexer Select. The <code>SPORT_CTL2_B.FSMUXSEL</code> bit enables multiplexing of the half SPORT's frame sync. In this mode, the frame sync of the related half SPORT is used instead of the half SPORT's own frame sync. For example, if the <code>SPORT_CTL2_B.FSMUXSEL</code> bit is enabled, half SPORT 'B' uses <code>SPORT_AFS</code> instead of <code>SPORT_BFS</code> .
		0 Disable frame sync multiplexing
		1 Enable frame sync multiplexing

Half SPORT 'A' Control Register

The `SPORT_CTL_A` register contains transmit and receive control bits for SPORT half 'A', including serial port mode selection for the half SPORT's primary and secondary channels. The function of some bits in the `SPORT_CTL_A` register vary depending on the SPORT's operating mode. For more information, see the SPORT operating modes description. If reading reserved bits, the read value is the last written value to these bits or is the reset value of these bits.

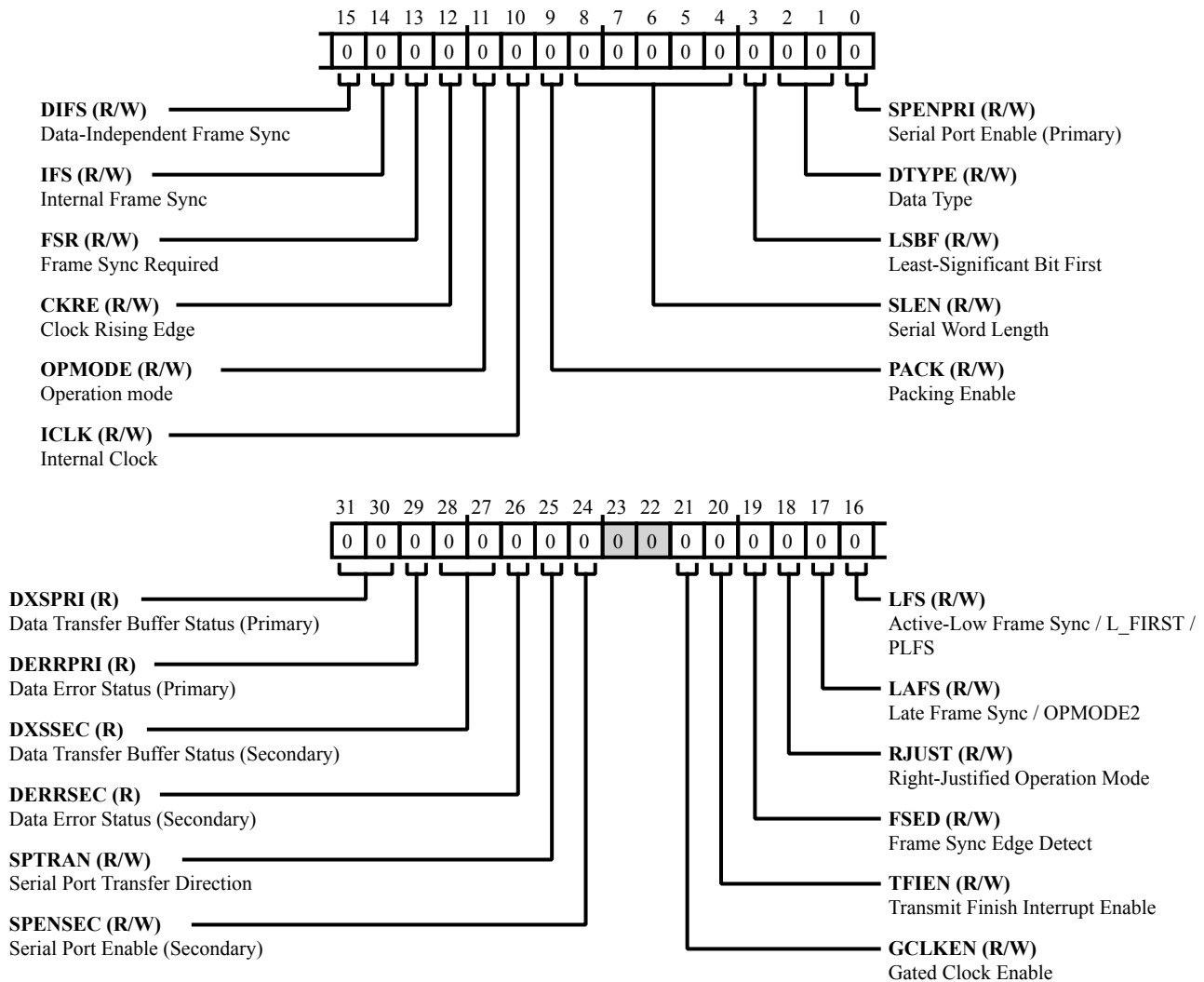


Figure 26-24: `SPORT_CTL_A` Register Diagram

Table 26-25: SPORT_CTL_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:30 (R/NW)	DXSPRI	Data Transfer Buffer Status (Primary). The <code>SPORT_CTL_A.DXSPRI</code> bit field indicates the status of the half SPORT's primary channel data buffer.
		0 Empty
		1 Reserved
		2 Partially full
		3 Full
29 (R/NW)	DERRPRI	Data Error Status (Primary). The <code>SPORT_CTL_A.DERRPRI</code> bit reports the half SPORT's primary channel transmit underflow status or receive overflow status, depending on the SPORT transfer direction. If the <code>SPORT_CTL_A.FSR</code> bit =1, the <code>SPORT_CTL_A.DERRPRI</code> bit indicates whether the <code>SPORT_AFS</code> signal (from an internal or external source) occurred while the <code>SPORT_TXPRI_A</code> data buffer was empty (during transmit) or the <code>SPORT_RXPRI_A</code> data buffer was full (during receive). The SPORT transmits or receives data whenever it detects the <code>SPORT_AFS</code> signal. It is important to note that, as a receiver, the <code>SPORT_CTL_A.DERRPRI</code> bit indicates when the channel has received new data while the <code>SPORT_RXPRI_A</code> receive buffer is full. This new data overwrites existing data. If the <code>SPORT_CTL_A.FSR</code> bit =0, the <code>SPORT_CTL_A.DERRPRI</code> bit is set whenever the SPORT is required to transmit while the <code>SPORT_TXPRI_A</code> transmit buffer is empty. It is also set whenever the SPORT is required to receive while the <code>SPORT_RXPRI_A</code> receive buffer is full. The SPORT clears the <code>SPORT_CTL_A.DERRPRI</code> bit if the <code>SPORT_ERR_A.DERRPSTAT</code> bit is cleared.
		0 No error
		1 Error (Tx underflow or Rx overflow)
28:27 (R/NW)	DXSSEC	Data Transfer Buffer Status (Secondary). The <code>SPORT_CTL_A.DXSSEC</code> bit field indicates the status of the half SPORT's secondary channel data buffer.
		0 Empty
		1 Reserved
		2 Partially full
		3 Full

Table 26-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/NW)	DERRSEC	Data Error Status (Secondary). The <code>SPORT_CTL_A.DERRSEC</code> bit reports the half SPORT's secondary channel transmit underflow status or receive overflow status, depending on the SPORT transfer direction. If the <code>SPORT_CTL_A.FSR</code> bit =1, the <code>SPORT_CTL_A.DERRSEC</code> bit indicates whether the <code>SPORT_AFS</code> signal (from an internal or external source) occurred while the <code>SPORT_TXSEC_A</code> data buffer was empty (during transmit) or the <code>SPORT_RXSEC_A</code> data buffer was full (during receive). The SPORT transmits or receives data whenever it detects the <code>SPORT_AFS</code> signal. It is important to note that, as a receiver, the <code>SPORT_CTL_A.DERRSEC</code> bit indicates when the channel has received new data while the <code>SPORT_RXSEC_A</code> receive buffer is full. This new data overwrites existing data. If the <code>SPORT_CTL_A.FSR</code> bit =0, the <code>SPORT_CTL_A.DERRSEC</code> bit is set whenever the SPORT is required to transmit while the <code>SPORT_TXSEC_A</code> transmit buffer is empty. It is also set whenever the SPORT is required to receive while the <code>SPORT_RXSEC_A</code> receive buffer is full. The SPORT clears the <code>SPORT_CTL_A.DERRSEC</code> bit if the <code>SPORT_ERR_A.DERRSSTAT</code> bit is cleared.
		0 No error
		1 Error (Tx underflow or Rx overflow)
25 (R/W)	SPTRAN	Serial Port Transfer Direction. The <code>SPORT_CTL_A.SPTRAN</code> bit selects the transfer direction (receive or transmit) for the half SPORT's primary and secondary channels. When the direction is receive, the half SPORT activates the receive buffers, and the <code>SPORT_ACLK</code> and <code>SPORT_AFS</code> pins control the receive buffers. The transmit buffers are inactive when the half SPORT's transfer direction is receive. When the direction is transmit, the half SPORT activates the transmit buffers, and the <code>SPORT_ACLK</code> and <code>SPORT_AFS</code> pins control the transmit shift registers. The receive buffers are inactive when the half SPORT's transfer direction is transmit.
		0 Receive
		1 Transmit
24 (R/W)	SPENSEC	Serial Port Enable (Secondary). The <code>SPORT_CTL_A.SPENSEC</code> bit enables the half SPORT's secondary channel. When this bit is cleared (changes from =1 to =0), the half SPORT automatically flushes the channel's data buffers.
		0 Disable
		1 Enable

Table 26-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	GCLKEN	Gated Clock Enable. The <code>SPORT_CTL_A.GCLKEN</code> bit enables gated clock operation for the half SPORT when in DSP serial mode or left-justified stereo modes (<code>SPORT_CTL_A.OPMODE = 0</code> or <code>1</code>). This bit is ignored when the half SPORT is in right-justified mode (<code>SPORT_CTL_A.RJUST = 1</code>) or multichannel mode (<code>SPORT_MCTL_A.MCE = 1</code>). When the <code>SPORT_CTL_A.GCLKEN</code> bit is enabled, the SPORT clock is active when the SPORT is transferring data or when the frame sync changes (transitions to active state).
		0 Disable
		1 Enable
20 (R/W)	TFIEN	Transmit Finish Interrupt Enable. The <code>SPORT_CTL_A.TFIEN</code> bit selects when the half SPORT issues its transmission complete interrupt request, if a DMA complete interrupt request is enabled by the <code>DMA_CFG.INT</code> configuration. When enabled (<code>SPORT_CTL_A.TFIEN = 1</code>), the DMA complete peripheral interrupt request is generated when the last bit of last word in the DMA is shifted out. When disabled (<code>SPORT_CTL_A.TFIEN = 0</code>), the DMA interrupt request is generated when the DMA counter expires (the last word goes to the transmit buffer).
		0 Last word sent (DMA count done) interrupt
		1 Last bit sent (Tx buffer done) interrupt
19 (R/W)	FSED	Frame Sync Edge Detect. The <code>SPORT_CTL_A.FSED</code> bit enables the half SPORT to start transmitting or receiving after detecting an active edge of an external frame sync. The <code>SPORT_CTL_A.FSED</code> bit may be enabled even during an active frame sync, and the half SPORT starts the transfer on the next valid rising or falling edge of external frame sync. If disabled (<code>SPORT_CTL_A.FSED = 0</code>), the half SPORT operates in the standard level-sensitive detection mode for external frame sync.
		0 Level detect frame sync
		1 Edge detect frame sync

Table 26-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	RJUST	Right-Justified Operation Mode. The SPORT_CTL_A.RJUST bit enables the half SPORT (if SPORT_CTL_A.OPMODE =1) to transfer data in right-justified operation mode. In this mode, the half SPORT aligns data to the end of the frame sync, rather than the start of the frame sync. When using right-justified mode, systems should program an appropriate delay count to introduce a clock delay before the half SPORT state machine starts to capture data. This value is set in the DCNT field (right-justified mode usage of the SPORT_MCTL_A.WOFFSET field). For information about appropriate delay selections, see the SPORT operating modes section.
		0 Disable
		1 Enable
17 (R/W)	LAFS	Late Frame Sync / OPMODE2. When the half SPORT is in DSP standard mode (SPORT_CTL_A.OPMODE =0) or in right-justified mode (SPORT_CTL_A.RJUST =1), the SPORT_CTL_A.LAFS bit selects whether the half SPORT generates a late frame sync (SPORT_AFS during first data bit) or generates an early frame sync signal (SPORT_AFS during serial clock cycle before first data bit). When the half SPORT is in I ² S / left-justified mode (SPORT_CTL_A.OPMODE =1), the SPORT_CTL_A.LAFS bit acts as OP-MODE2, selecting whether the half SPORT is in left-justified mode or I ² S mode. When the half SPORT is in multichannel mode (SPORT_MCTL_A.MCE =1), the SPORT_CTL_A.LAFS bit is reserved.
		0 Early frame sync (or I ² S mode)
		1 Late frame sync (or left-justified mode)
16 (R/W)	LFS	Active-Low Frame Sync / L_FIRST / PLFS. When the half SPORT is in DSP standard mode and multichannel mode (SPORT_CTL_A.OPMODE =0), the SPORT_CTL_A.LFS bit selects whether the half SPORT uses active low or active high frame sync. When the half SPORT is in I ² S / packed / left-justified mode (SPORT_CTL_A.OPMODE =1), the SPORT_CTL_A.LFS bit acts as L_FIRST, selecting whether the half SPORT transfers data first for the left or right channel.
		0 Active high frame sync (DSP standard mode) or rising edge frame sync (multichannel mode) or right channel first (I ² S/packed mode) or left channel first (left-justified mode)
		1 Active low frame sync (DSP standard mode) or falling edge frame sync (multichannel mode) or left channel first (I ² S/packed mode) or right channel first (left-justified mode)

Table 26-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	DIFS	Data-Independent Frame Sync. The <code>SPORT_CTL_A.DIFS</code> bit selects whether the half SPORT uses a data-independent or data-dependent frame sync. When using a data-independent frame sync, the half SPORT generates the sync at the interval selected by the <code>SPORT_DIV_A.FSDIV</code> bit. When using a data-dependent frame sync, the half SPORT generates the sync on the selected interval when the transmit buffer is not empty or when the receive buffer is not full. Note that the <code>SPORT_CTL_A.DIFS</code> bit is automatically set when the half SPORT is in packed or multichannel modes.
		0 Data-dependent frame sync
		1 Data-independent frame sync
14 (R/W)	IFS	Internal Frame Sync. The <code>SPORT_CTL_A.IFS</code> bit selects whether the half SPORT uses an internal frame sync or uses an external frame sync. Note that the externally-generated frame sync does not need to be synchronous with the processor's system clock.
		0 External frame sync
		1 Internal frame sync
13 (R/W)	FSR	Frame Sync Required. The <code>SPORT_CTL_A.FSR</code> bit selects whether or not the half SPORT requires frame sync for data transfer. This bit is automatically set when the half SPORT is in I ² S / packed / left-justified mode (<code>SPORT_CTL_A.OPMODE</code> = 1) or is in multichannel mode (<code>SPORT_MCTL_A.MCE</code> = 1).
		0 No frame sync required
		1 Frame sync required
12 (R/W)	CKRE	Clock Rising Edge. The <code>SPORT_CTL_A.CKRE</code> bit selects the rising or falling edge of the <code>SPORT_ACLK</code> clock for the half SPORT to sample receive data and frame sync. Note that the half SPORT changes the state of transmit data and frame sync signals on the non-selected edge of the <code>SPORT_ACLK</code> . Also, note that the transmit and receive related SPORT halves (A and B) should be programmed with the same value for the <code>SPORT_CTL_A.CKRE</code> bit. This programming drives the internally-generated signals on one edge of <code>SPORT_ACLK</code> and samples the received signals on the opposite edge.
		0 Clock falling edge
		1 Clock rising edge

Table 26-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	OPMODE	Operation mode. The SPORT_CTL_A.OPMODE bit selects whether the half SPORT operates in DSP standard/multichannel mode or operates in I ² S/packed/left-justified mode. The mode selection affects the operation of the SPORT_CTL_A.LAFS and SPORT_CTL_A.LFS bits. Also, the SPORT_CTL_A.OPMODE bit enables or disables operation of the SPORT_CTL_A.GCLKEN, SPORT_CTL_A.FSED, SPORT_CTL_A.RJUST, SPORT_CTL_A.DIFS, SPORT_CTL_A.FSR, and SPORT_CTL_A.CKRE bits.
		0 DSP standard/multichannel mode
		1 I ² S/packed/left-justified mode
10 (R/W)	ICLK	Internal Clock. When the half SPORT is in DSP standard mode (SPORT_CTL_A.OPMODE = 0), the SPORT_CTL_A.ICLK bit selects whether the half SPORT uses an internal or external clock. For internal clock enabled, the half SPORT generates the SPORT_ACLK clock signal, and SPORT_ACLK is an output. The SPORT_DIV_A.CLKDIV serial clock divisor value determines the clock frequency. For internal clock disabled, the SPORT_ACLK clock signal is an input, and the serial clock divisor is ignored. Note that the externally-generated serial clock does not need to be synchronous with the processor's system clock.
		0 External clock
		1 Internal clock
9 (R/W)	PACK	Packing Enable. The SPORT_CTL_A.PACK bit enables the half SPORT to perform 16- to 32-bit packing on received data and to perform 32- to 16-bit unpacking on transmitted data. The receive packing operation packs two successive received words into a single 32-bit word. The transmit unpacking operation unpacks each 32-bit word and transmits it as two 16-bit words. The first 16-bit (or smaller) word is right-justified in bits 15:0 of the packed word, and the second 16-bit (or smaller) word is right-justified in bits 31:16. This format applies to both receive (packing) and transmit (unpacking) operations. Companding may be used with word packing or unpacking. The half SPORT generates data transfer related interrupts when packing is enabled. The transmit and receive interrupts are generated for the 32-bit packed words, not for each 16-bit word.
		0 Disable
		1 Enable

Table 26-25: SPORT_CTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
8:4 (R/W)	SLEN	Serial Word Length. The SPORT_CTL_A.SLEN bits selects word length in bits for the half SPORT's data transfers. Word may be from 4- to 32-bits in length. The formula for selecting the word length in bits is: SPORT_CTL_A.SLEN = (serial word length in bits) - 1 For DSP standard mode (SPORT_CTL_A.OPMODE =0), use SPORT_CTL_A.SLEN of 3 to 31 bits. For I ² S / packed / left-justified mode (SPORT_CTL_A.OPMODE =1), use SPORT_CTL_A.SLEN of 4 to 31 bits.	
3 (R/W)	LSBF	Least-Significant Bit First. The SPORT_CTL_A.LSBF bit selects whether the half SPORT transmits or receives data LSB first or MSB first.	
		0	MSB first sent/received (big endian)
		1	LSB first sent/received (little endian)
2:1 (R/W)	DTYPE	Data Type. The SPORT_CTL_A.DTYPE bits selects the data type formatting for the half SPORT's data transfers in DSP standard mode (SPORT_CTL_A.OPMODE =0).	
		0	Right-justify data, zero-fill unused MSBs
		1	Right-justify data, sign-extend unused MSBs
		2	u-law compand data
		3	A-law compand data
0 (R/W)	SPENPRI	Serial Port Enable (Primary). The SPORT_CTL_A.SPENPRI bit enables the half SPORT's primary channel. When this bit is cleared (changes from =1 to =0), the half SPORT automatically flushes the channel's data buffers.	
		0	Disable
		1	Enable

Half SPORT 'B' Control Register

The `SPORT_CTL_B` register contains transmit and receive control bits for SPORT half 'B', including serial port mode selection for the half SPORT's primary and secondary channels. The function of some bits in the `SPORT_CTL_B` register vary, depending on the SPORT's operating mode. For more information, see the SPORT operating modes description. If reading reserved bits, the read value is the last written value to these bits or is the reset value of these bits.

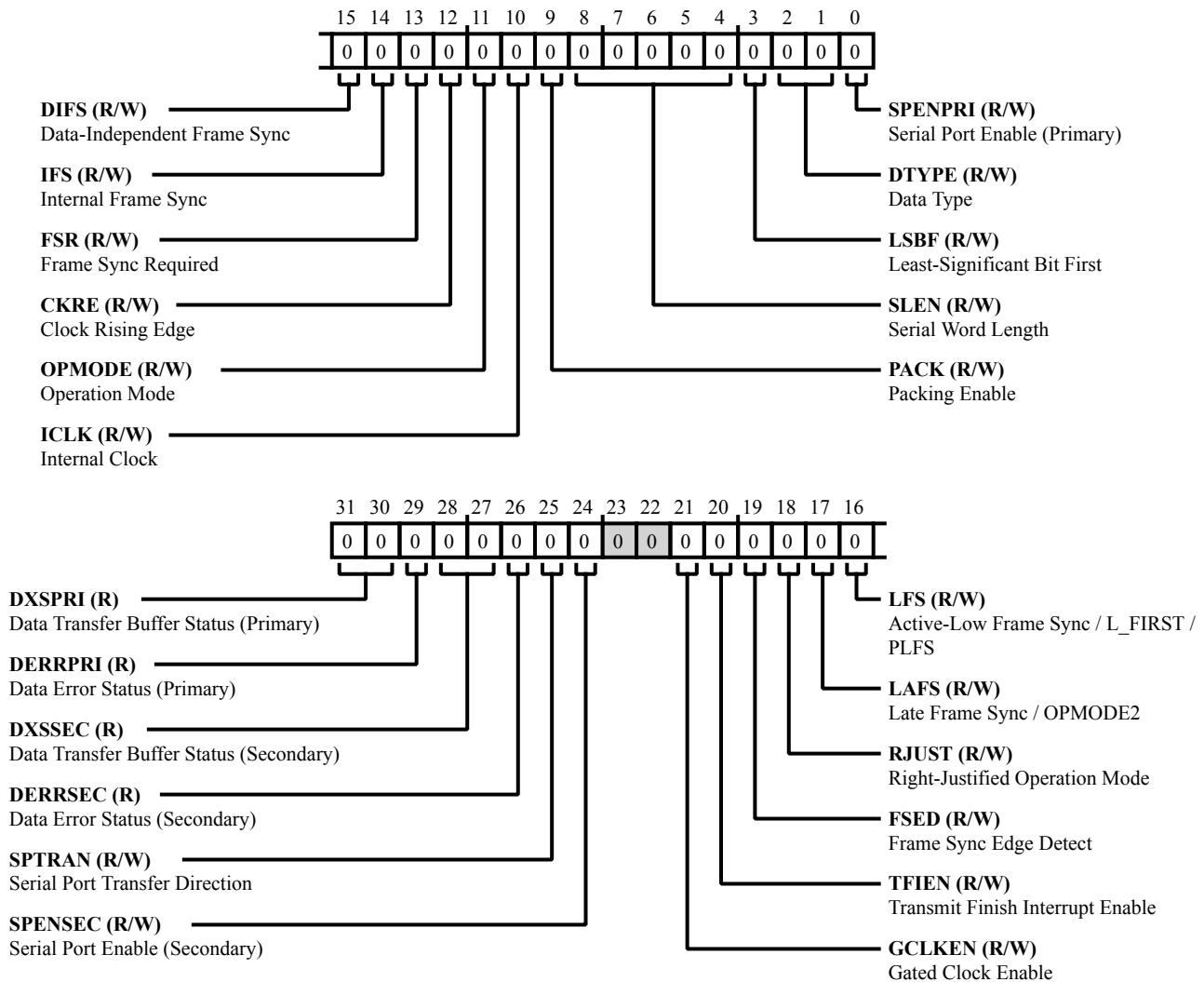


Figure 26-25: `SPORT_CTL_B` Register Diagram

Table 26-26: SPORT_CTL_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:30 (R/NW)	DXSPRI	Data Transfer Buffer Status (Primary). The <code>SPORT_CTL_B.DXSPRI</code> bit field indicates the status of the half SPORT's primary channel data buffer.
		0 Empty
		1 Reserved
		2 Partially full
		3 Full
29 (R/NW)	DERRPRI	Data Error Status (Primary). The <code>SPORT_CTL_B.DERRPRI</code> bit reports the half SPORT's primary channel transmit underflow status or receive overflow status, depending on the SPORT transfer direction. If the <code>SPORT_CTL_B.FSR</code> bit =1, the <code>SPORT_CTL_B.DERRPRI</code> bit indicates whether the <code>SPORT_BFS</code> signal (from an internal or external source) occurred while the <code>SPORT_TXPRI_B</code> data buffer was empty (during transmit) or the <code>SPORT_RXPRI_B</code> data buffer was full (during receive). The SPORT transmits or receives data whenever it detects the <code>SPORT_BFS</code> signal. It is important to note that, as a receiver, the <code>SPORT_CTL_B.DERRPRI</code> bit indicates when the channel has received new data while the <code>SPORT_RXPRI_B</code> receive buffer is full. This new data overwrites existing data. If the <code>SPORT_CTL_B.FSR</code> bit =0, the <code>SPORT_CTL_B.DERRPRI</code> bit is set whenever the SPORT is required to transmit while the <code>SPORT_TXPRI_B</code> transmit buffer is empty and is set whenever the SPORT is required to receive while the <code>SPORT_RXPRI_B</code> receive buffer is full. The SPORT clears the <code>SPORT_CTL_B.DERRPRI</code> bit if the <code>SPORT_ERR_B.DERRPSTAT</code> bit is cleared.
		0 No error
		1 Error (Tx underflow or Rx overflow)
28:27 (R/NW)	DXSSEC	Data Transfer Buffer Status (Secondary). The <code>SPORT_CTL_B.DXSSEC</code> bit field indicates the status of the half SPORT's secondary channel data buffer.
		0 Empty
		1 Reserved
		2 Partially full
		3 Full

Table 26-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/NW)	DERRSEC	Data Error Status (Secondary). The SPORT_CTL_B.DERRSEC bit reports the half SPORT's secondary channel transmit underflow status or receive overflow status, depending on the SPORT transfer direction. If the SPORT_CTL_B.FSR bit =1, the SPORT_CTL_B.DERRSEC bit indicates whether the SPORT_BFS signal (from an internal or external source) occurred while the SPORT_TXSEC_B data buffer was empty (during transmit) or the SPORT_RXSEC_B data buffer was full (during receive). The SPORT transmits or receives data whenever it detects the SPORT_BFS signal. It is important to note that, as a receiver, the SPORT_CTL_B.DERRSEC bit indicates when the channel has received new data while the SPORT_RXSEC_B receive buffer is full. This new data overwrites existing data. If the SPORT_CTL_B.FSR bit =0, the SPORT_CTL_B.DERRSEC bit is set whenever the SPORT is required to transmit while the SPORT_TXSEC_B transmit buffer is empty. It is also set whenever the SPORT is required to receive while the SPORT_RXSEC_B receive buffer is full. The SPORT clears the SPORT_CTL_B.DERRSEC bit if the SPORT_ERR_B.DERRSSTAT bit is cleared.
		0 No error
		1 Error (Tx underflow or Rx overflow)
25 (R/W)	SPTRAN	Serial Port Transfer Direction. The SPORT_CTL_B.SPTRAN bit selects the transfer direction (receive or transmit) for the half SPORT's primary and secondary channels. When the direction is receive, the half SPORT activates the receive buffers, and the SPORT_BCLK and SPORT_BFS pins control the receive buffers. The transmit buffers are inactive when the half SPORT's transfer direction is receive. When the direction is transmit, the half SPORT activates the transmit buffers, and the SPORT_BCLK and SPORT_BFS pins control the transmit shift registers. The receive buffers are inactive when the half SPORT's transfer direction is transmit.
		0 Receive
		1 Transmit
24 (R/W)	SPENSEC	Serial Port Enable (Secondary). The SPORT_CTL_B.SPENSEC bit enables the half SPORT's secondary channel. When this bit is cleared (changes from =1 to =0), the half SPORT automatically flushes the channel's data buffers.
		0 Disable
		1 Enable

Table 26-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21 (R/W)	GCLKEN	Gated Clock Enable. The <code>SPORT_CTL_B.GCLKEN</code> bit enables gated clock operation for the half SPORT when in DSP serial mode or left-justified stereo modes (<code>SPORT_CTL_B.OPMODE = 0</code> or <code>1</code>). This bit is ignored when the half SPORT is in right-justified mode (<code>SPORT_CTL_B.RJUST = 1</code>) or multichannel mode (<code>SPORT_MCTL_B.MCE = 1</code>). When <code>SPORT_CTL_B.GCLKEN</code> is enabled, the SPORT clock is active when the SPORT is transferring data or when the frame sync changes (transitions to active state).
		0 Disable
		1 Enable
20 (R/W)	TFIEN	Transmit Finish Interrupt Enable. The <code>SPORT_CTL_B.TFIEN</code> bit selects when the half SPORT issues its transmission complete interrupt request, if a DMA complete interrupt request is enabled by the <code>DMA_CFG.INT</code> configuration. When enabled (<code>SPORT_CTL_B.TFIEN = 1</code>), the DMA complete peripheral interrupt request is generated when the last bit of last word in the DMA is shifted out. When disabled (<code>SPORT_CTL_B.TFIEN = 0</code>), the DMA interrupt request is generated when the DMA counter expires (the last word goes to the transmit buffer).
		0 Last word sent (DMA count done) interrupt
		1 Last bit sent (Tx buffer done) interrupt
19 (R/W)	FSED	Frame Sync Edge Detect. The <code>SPORT_CTL_B.FSED</code> bit enables the half SPORT to start transmitting or receiving after detecting an active edge of an external frame sync. The <code>SPORT_CTL_B.FSED</code> may be enabled even during an active frame sync, and the half SPORT starts the transfer on the next valid rising or falling edge of external frame sync. If disabled (<code>SPORT_CTL_B.FSED = 0</code>), the half SPORT operates in the standard level-sensitive detection mode for external frame sync.
		0 Level detect frame sync
		1 Edge detect frame sync

Table 26-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	RJUST	Right-Justified Operation Mode. The SPORT_CTL_B.RJUST bit enables the half SPORT (if SPORT_CTL_B.OPMODE =1) to transfer data in right-justified operation mode. In this mode, the half SPORT aligns data to the end of the frame sync, rather than the start of the frame sync. When using right-justified mode, systems should program an appropriate delay count to introduce a clock delay before the half SPORT state machine starts to capture data. This value is set in the DCNT field (right-justified mode usage of the SPORT_MCTL_B.WOFFSET field). For information about appropriate delay selections, see the SPORT operating modes section.
		0 Disable
		1 Enable
17 (R/W)	LAFS	Late Frame Sync / OPMODE2. When the half SPORT is in DSP standard mode (SPORT_CTL_B.OPMODE =0) or in right-justified mode (SPORT_CTL_B.RJUST =1), the SPORT_CTL_B.LAFS bit selects whether the half SPORT generates a late frame sync (SPORT_BFS during first data bit) or generates an early frame sync signal (SPORT_BFS during serial clock cycle before first data bit). When the half SPORT is in I ² S / left-justified mode (SPORT_CTL_B.OPMODE =1), the SPORT_CTL_B.LAFS bit acts as OPMODE2, selecting whether the half SPORT is in left-justified mode or I ² S mode. When the half SPORT is in multichannel mode (SPORT_MCTL_B.MCE =1), the SPORT_CTL_B.LAFS bit is reserved.
		0 Early frame sync (or I ² S mode)
		1 Late frame sync (or left-justified mode)

Table 26-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	LFS	Active-Low Frame Sync / L_FIRST / PLFS. When the half SPORT is in DSP standard mode and multichannel mode (SPORT_CTL_B.OPMODE = 0), the SPORT_CTL_B.LFS bit selects whether the half SPORT uses active low or active high frame sync. When the half SPORT is in I ² S / packed / left-justified mode (SPORT_CTL_B.OPMODE = 1), the SPORT_CTL_B.LFS bit acts as L_FIRST, selecting whether the half SPORT transfers data first for the left or right channel.
		0 Active high frame sync (DSP standard mode) or rising edge frame sync (multichannel mode) or right channel first (I ² S/packed mode) or left channel first (left-justified mode)
		1 Active low frame sync (DSP standard mode) or falling edge frame sync (multichannel mode) or left channel first (I ² S/packed mode) or right channel first (left-justified mode)
15 (R/W)	DIFS	Data-Independent Frame Sync. The SPORT_CTL_B.DIFS bit selects whether the half SPORT uses a data-independent or data-dependent frame sync. When using a data-independent frame sync, the half SPORT generates the sync at the interval selected by SPORT_DIV_B.FSDIV. When using a data-dependent frame sync, the half SPORT generates the sync on the selected interval when the transmit buffer is not empty or when the receive buffer is not full. Note that the SPORT_CTL_B.DIFS bit is automatically set when the half SPORT is in packed or multichannel modes.
		0 Data-dependent frame sync
		1 Data-independent frame sync
14 (R/W)	IFS	Internal Frame Sync. The SPORT_CTL_B.IFS bit selects whether the half SPORT uses an internal frame sync or uses an external frame sync. Note that the externally-generated frame sync does not need to be synchronous with the processor's system clock.
		0 External frame sync
		1 Internal frame sync

Table 26-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	FSR	Frame Sync Required. The SPORT_CTL_B.FSR selects whether or not the half SPORT requires frame sync for data transfer. This bit is automatically set when the half SPORT is in I ² S / packed / left-justified mode (SPORT_CTL_B.OPMODE =1) or is in multichannel mode (SPORT_MCTL_B.MCE =1).
		0 No frame sync required
		1 Frame sync required
12 (R/W)	CKRE	Clock Rising Edge. The SPORT_CTL_B.CKRE selects the rising or falling edge of the SPORT_BCLK clock for the half SPORT to sample receive data and frame sync. Note that the half SPORT changes the state of transmit data and frame sync signals on the non-selected edge of the SPORT_BCLK. Also note that the transmit and receive related SPORT halves (A and B) should be programmed with the same value for SPORT_CTL_B.CKRE. This programming drives the internally-generated signals on one edge of SPORT_BCLK and samples the received signals on the opposite edge.
		0 Clock falling edge
		1 Clock rising edge
11 (R/W)	OPMODE	Operation Mode. The SPORT_CTL_B.OPMODE bit selects whether the half SPORT operates in DSP standard / multichannel mode or operates in I ² S / packed / left-justified mode. The mode selection affects the operation of the SPORT_CTL_B.LAFS and SPORT_CTL_B.LFS bits. Also, the SPORT_CTL_B.OPMODE bit enables or disables operation of the SPORT_CTL_B.GCLKEN, SPORT_CTL_B.FSED, SPORT_CTL_B.RJUST, SPORT_CTL_B.DIFS, SPORT_CTL_B.FSR, and SPORT_CTL_B.CKRE bits.
		0 DSP standard/multichannel mode
		1 I ² S/packed/left-justified mode
10 (R/W)	ICLK	Internal Clock. When the half SPORT is in DSP standard mode (SPORT_CTL_B.OPMODE =0), the SPORT_CTL_B.ICLK bit selects whether the half SPORT uses an internal or external clock. For internal clock enabled, the half SPORT generates the SPORT_BCLK clock signal, and the SPORT_BCLK is an output. The SPORT_DIV_B.CLKDIV serial clock divisor value determines the clock frequency. For internal clock disabled, the SPORT_BCLK clock signal is an input, and the serial clock divisor is ignored. Note that the externally-generated serial clock does not need to be synchronous with the processor's system clock.
		0 External clock
		1 Internal clock

Table 26-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	PACK	Packing Enable. The SPORT_CTL_B.PACK bit enables the half SPORT to perform 16- to 32-bit packing on received data and to perform 32- to 16-bit unpacking on transmitted data. The receive packing operation packs two successive received words into a single 32-bit word. The transmit unpacking operation unpacks each 32-bit word and transmits it as two 16-bit words. The first 16-bit (or smaller) word is right-justified in bits 15:0 of the packed word, and the second 16-bit (or smaller) word is right-justified in bits 31:16. This format applies to both receive (packing) and transmit (unpacking) operations. Companding may be used with word packing or unpacking. The half SPORT generates data transfer related interrupts when packing is enabled. The transmit and receive interrupts are generated for the 32-bit packed words, not for each 16-bit word.
		0 Disable
		1 Enable
8:4 (R/W)	SLEN	Serial Word Length. The SPORT_CTL_B.SLEN bits selects word length in bits for the half SPORT's data transfers. Word may be from 4- to 32-bits in length. The formula for selecting the word length in bits is: $\text{SPORT_CTL_B.SLEN} = (\text{serial word length in bits}) - 1$ For DSP standard mode (SPORT_CTL_B.OPMODE =0), use SPORT_CTL_B.SLEN of 3 to 31 bits. For I ² S / packed / left-justified mode (SPORT_CTL_B.OPMODE =1), use SPORT_CTL_B.SLEN of 4 to 31 bits.
3 (R/W)	LSBF	Least-Significant Bit First. The SPORT_CTL_B.LSBF bit selects whether the half SPORT transmits or receives data LSB first or MSB first.
		0 MSB first sent/received (big endian)
		1 LSB first sent/received (little endian)
2:1 (R/W)	DTYPE	Data Type. The SPORT_CTL_B.DTYPE bits selects the data type formatting for the half SPORT's data transfers in DSP standard mode (SPORT_CTL_B.OPMODE =0).
		0 Right-justify data, zero-fill unused MSBs
		1 Right-justify data, sign-extend unused MSBs
		2 u-law compand data
		3 A-law compand data

Table 26-26: SPORT_CTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	SPENPRI	Serial Port Enable (Primary). The <code>SPORT_CTL_B.SPENPRI</code> bit enables the half SPORT's primary channel. When this bit is cleared (changes from =1 to =0), the half SPORT automatically flushes the channel's data buffers.
		0 Disable
		1 Enable

Half SPORT 'A' Divisor Register

The `SPORT_DIV_A` register contains divisor values that determine frequencies of internally-generated clocks and frame syncs for half SPORT 'A'.

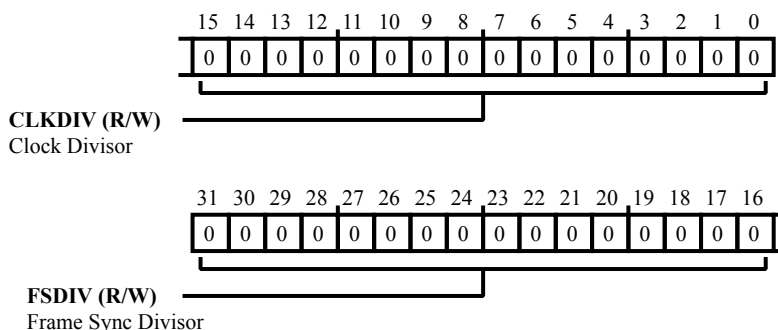


Figure 26-26: `SPORT_DIV_A` Register Diagram

Table 26-27: `SPORT_DIV_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	FSDIV	Frame Sync Divisor. The <code>SPORT_DIV_A.FSDIV</code> bits select the number of transmit or receive clock cycles that the half SPORT counts before generating a frame sync pulse. The half SPORT counts serial clock cycles whether these are from an internally- or an externally-generated serial clock. The formula relating <code>SPORT_DIV_A.FSDIV</code> to the number of cycles between frame sync pulses is: $\text{SPORT_DIV_A.FSDIV} = (\text{number of serial clocks between frame syncs}) - 1$ Use the following equation to determine the value of <code>SPORT_DIV_A.FSDIV</code>, given the serial clock frequency and desired frame sync frequency: $\text{FSDIV} = (\text{SPORT_ACLK} / \text{SPORT_AFS}) - 1$ Note that the frame sync is continuously active when <code>SPORT_DIV_A.FSDIV = 0</code>. The value of <code>SPORT_DIV_A.FSDIV</code> should not be less than the serial word length (<code>SPORT_CTL_A.SLEN</code>), as this may cause an external device to abort the current operation or cause other unpredictable results.
15:0 (R/W)	CLKDIV	Clock Divisor. The <code>SPORT_DIV_A.CLKDIV</code> bits select the divisor that the half SPORT uses to calculate the serial clock (<code>SPORT_ACLK</code>) from the processor system clock (<code>SCLK0</code>). The divisor is a 16-bit value, allowing a wide range of serial clock rates. When configured for internal clock (<code>SPORT_CTL_A.ICLK = 1</code>), legal <code>SPORT_DIV_A.CLKDIV</code> values are 0 to 65535. Given the processor system clock frequency and desired serial clock frequency, use the following formula to calculate the value of <code>SPORT_DIV_A.CLKDIV</code>: $\text{CLKDIV} = (\text{SCLK0} / \text{SPORT_ACLK}) - 1$ For the maximum serial clock frequency, see the processor data sheet.

Half SPORT 'B' Divisor Register

The `SPORT_DIV_B` contains divisor values that determine frequencies of internally-generated clocks and frame syncs for SPORT half 'B'.

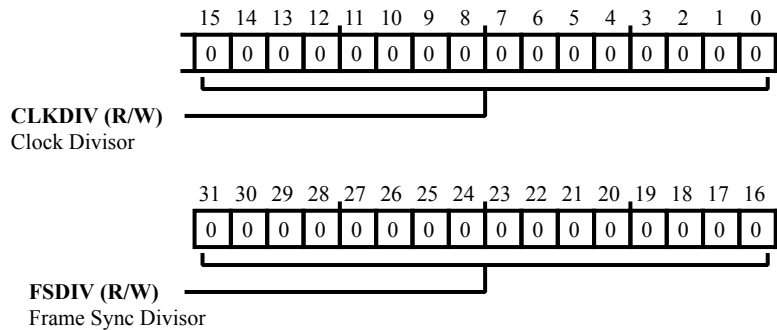


Figure 26-27: SPORT_DIV_B Register Diagram

Table 26-28: SPORT_DIV_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	FSDIV	Frame Sync Divisor. The <code>SPORT_DIV_B.FSDIV</code> bits select the number of transmit or receive clock cycles that the half SPORT counts before generating a frame sync pulse. The half SPORT counts serial clock cycles whether these are from an internally- or an externally-generated serial clock. The formula relating <code>SPORT_DIV_B.FSDIV</code> to the number of cycles between frame sync pulses is: $\text{SPORT_DIV_B.FSDIV} = (\text{number of serial clocks between frame syncs}) - 1$ Use the following equation to determine the value of <code>SPORT_DIV_B.FSDIV</code>, given the serial clock frequency and desired frame sync frequency: $\text{FSDIV} = (\text{SPORT_BCLK} / \text{SPORT_BFS}) - 1$ Note that the frame sync is continuously active when <code>SPORT_DIV_B.FSDIV = 0</code>. The value of <code>SPORT_DIV_B.FSDIV</code> should not be less than the serial word length (<code>SPORT_CTL_B.SLEN</code>), as this may cause an external device to abort the current operation or cause other unpredictable results.
15:0 (R/W)	CLKDIV	Clock Divisor. The <code>SPORT_DIV_B.CLKDIV</code> bits select the divisor that the half SPORT uses to calculate the serial clock (<code>SPORT_BCLK</code>) from the processor system clock (<code>SCLK0</code>). The divisor is a 16-bit value, allowing a wide range of serial clock rates. When configured for internal clock (<code>SPORT_CTL_B.ICLK = 1</code>), legal <code>SPORT_DIV_B.CLKDIV</code> values are 0 to 65535. Given the processor system clock frequency and desired serial clock frequency, use the following formula to calculate the value of <code>SPORT_DIV_B.CLKDIV</code>: $\text{CLKDIV} = (\text{SCLK0} / \text{SPORT_BCLK}) - 1$ For the maximum serial clock frequency, see the processor data sheet.

Half SPORT 'A' Error Register

The `SPORT_ERR_A` register contains error status and error interrupt mask bits for SPORT half 'A', including error handling bits for the half SPORT's primary and secondary channels and frame sync. Detected errors are frame sync violations or buffer over/underflow conditions.

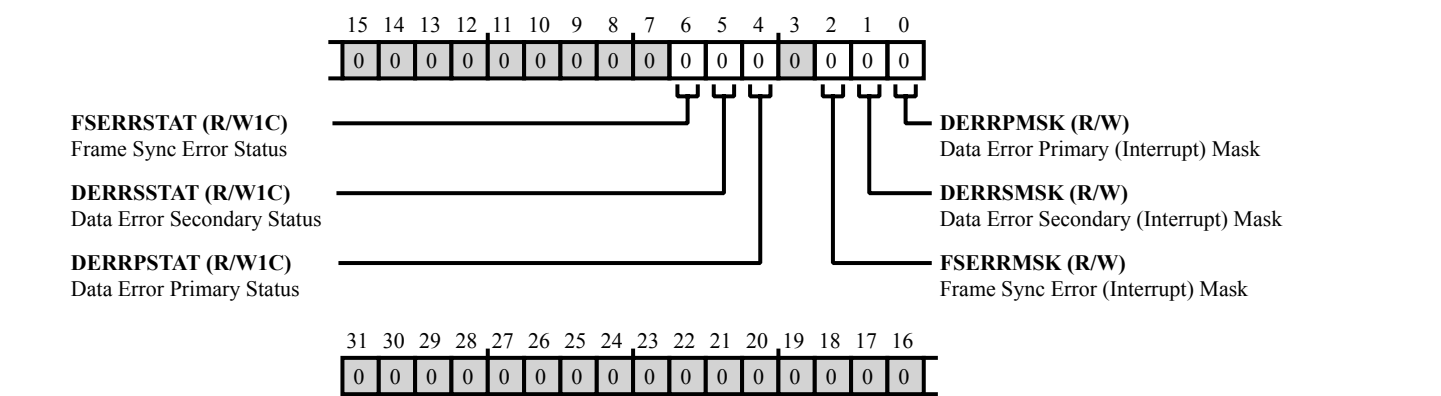


Figure 26-28: SPORT_ERR_A Register Diagram

Table 26-29: SPORT_ERR_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1C)	FSERRSTAT	Frame Sync Error Status. The <code>SPORT_ERR_A.FSERRSTAT</code> bit indicates that the half SPORT has detected a frame sync when the bit count (bits remaining in the frame) is non-zero. When a half SPORT is receiving or transmitting, its bit count is set to a word length (for example, <code>SPORT_CTL_A.SLEN = 31</code>). After each serial clock edge, the half SPORT decrements the transfer's bit count. After the word is received or transmitted, the transfer's bit count reaches zero, and the half SPORT resets it (for example, to 32) on next frame sync. Normal SPORT data transfers always have a non-zero bit count value when active transmission or reception is occurring. Normal SPORT frame syncs occur after the bit count becomes zero.
		0 No error
		1 Error (non-zero bit count at frame sync)

Table 26-29: SPORT_ERR_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W1C)	DERRSSTAT	Data Error Secondary Status. The SPORT_ERR_A.DERRSSTAT bit indicates the error status for the half SPORT's secondary channel data buffers. During transmit (SPORT_CTL_A.SPTRAN =1), the SPORT_ERR_A.DERRSSTAT bit indicates the transmit underflow status. During receive (SPORT_CTL_A.SPTRAN =0), the SPORT_ERR_A.DERRSSTAT bit indicates the receive overflow status. This bit is used to clear the latch of SPORT status interrupt request when triggered by a secondary data error. This bit can also be used to clear the read-only SPORT_CTL_A.DERRSEC status bit.
		0 No error
		1 Error (transmit underflow or receive overflow)
4 (R/W1C)	DERRPSTAT	Data Error Primary Status. The SPORT_ERR_A.DERRPSTAT bit indicates the error status for the half SPORT's primary channel data buffers. During transmit (SPORT_CTL_A.SPTRAN =1), the SPORT_ERR_A.DERRPSTAT bit indicates the transmit underflow status. During receive (SPORT_CTL_A.SPTRAN =0), the SPORT_ERR_A.DERRPSTAT bit indicates the receive overflow status. This bit is used to clear the latch of SPORT status interrupt request when triggered by a primary data error. This bit can also be used to clear the read-only SPORT_CTL_A.DERRPRI status bit.
		0 No error
		1 Error (transmit underflow or receive overflow)
2 (R/W)	FSERRMSK	Frame Sync Error (Interrupt) Mask. The SPORT_ERR_A.FSERRMSK unmask (enables) the half SPORT to generate the frame sync error interrupt request.
		0 Mask (disable)
		1 Unmask (enable)
1 (R/W)	DERRSMSK	Data Error Secondary (Interrupt) Mask. The SPORT_ERR_A.DERRSMSK unmask (enables) the half SPORT to generate the data error interrupt request for the secondary channel.
		0 Mask (disable)
		1 Unmask (enable)
0 (R/W)	DERRPMSK	Data Error Primary (Interrupt) Mask. The SPORT_ERR_A.DERRPMSK unmask (enables) the half SPORT to generate the data error interrupt request for the primary channel.
		0 Mask (disable)
		1 Unmask (enable)

Half SPORT 'B' Error Register

The `SPORT_ERR_B` register contains error status and error interrupt mask bits for SPORT half 'B', including error handling bits for the half SPORT's primary and secondary channels and frame sync. Detected errors are frame sync violations or buffer over/underflow conditions.

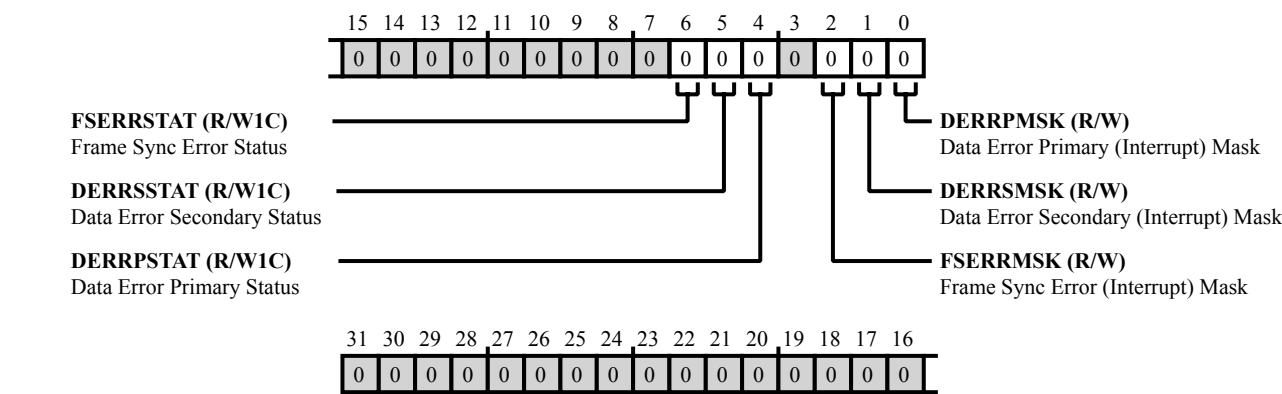


Figure 26-29: SPORT_ERR_B Register Diagram

Table 26-30: SPORT_ERR_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6 (R/W1C)	FSERRSTAT	Frame Sync Error Status. The <code>SPORT_ERR_B.FSERRSTAT</code> bit indicates that the half SPORT has detected a frame sync when the bit count (bits remaining in the frame) is non-zero. When a half SPORT is receiving or transmitting, its bit count is set to a word length (for example, <code>SPORT_CTL_B.SLEN = 31</code>). After each serial clock edge, the half SPORT decrements the transfer's bit count. After the word is received or transmitted, the transfer's bit count reaches zero, and the half SPORT resets it (for example, to 32) on next frame sync. Normal SPORT data transfers always have a non-zero bit count value when active transmission or reception is occurring. Normal SPORT frame syncs occur after the bit count becomes zero.
		0 No error
		1 Error (non-zero bit count at frame sync)

Table 26-30: SPORT_ERR_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W1C)	DERRSSTAT	Data Error Secondary Status. The SPORT_ERR_B.DERRSSTAT bit indicates the error status for the half SPORT's secondary channel data buffers. During transmit (SPORT_CTL_B.SPTRAN =1), SPORT_ERR_B.DERRSSTAT indicates the transmit underflow status. During receive (SPORT_CTL_B.SPTRAN =0), SPORT_ERR_B.DERRSSTAT indicates the receive overflow status. This bit is used to clear the latch of SPORT status interrupt request when triggered by a secondary data error. This bit can also be used to clear the read-only SPORT_CTL_B.DERRSEC status bit.
		0 No error
		1 Error (transmit underflow or receive overflow)
4 (R/W1C)	DERRPSTAT	Data Error Primary Status. The SPORT_ERR_B.DERRPSTAT bit indicates the error status for the half SPORT's primary channel data buffers. During transmit (SPORT_CTL_B.SPTRAN =1), the SPORT_ERR_B.DERRPSTAT bit indicates the transmit underflow status. During receive (SPORT_CTL_B.SPTRAN =0), the SPORT_ERR_B.DERRPSTAT bit indicates the receive overflow status. This bit is used to clear the latch of SPORT status interrupt request when triggered by a primary data error. This bit can also be used to clear the read-only SPORT_CTL_B.DERRPRI status bit.
		0 No error
		1 Error (transmit underflow or receive overflow)
2 (R/W)	FSERRMSK	Frame Sync Error (Interrupt) Mask. The SPORT_ERR_B.FSERRMSK unmask (enables) the half SPORT to generate the frame sync error interrupt request.
		0 Mask (disable)
		1 Unmask (enable)
1 (R/W)	DERRSMSK	Data Error Secondary (Interrupt) Mask. The SPORT_ERR_B.DERRSMSK unmask (enables) the half SPORT to generate the data error interrupt request for the secondary channel.
		0 Mask (disable)
		1 Unmask (enable)
0 (R/W)	DERRPMSK	Data Error Primary (Interrupt) Mask. The SPORT_ERR_B.DERRPMSK unmask (enables) the half SPORT to generate the data error interrupt request for the primary channel.
		0 Mask (disable)
		1 Unmask (enable)

Half SPORT 'A' Multichannel Control Register

The `SPORT_MCTL_A` register controls the half SPORT's multichannel operations. This register enables multichannel operation, enables multichannel data packing, selects the multichannel frame delay, selects the number of multichannel slots, and selects the multichannel window offset size.

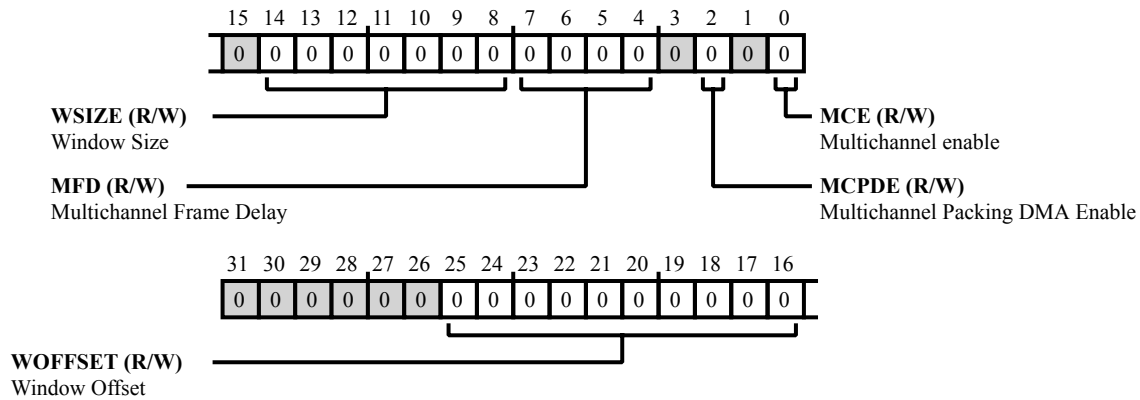


Figure 26-30: `SPORT_MCTL_A` Register Diagram

Table 26-31: `SPORT_MCTL_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:16 (R/W)	WOFFSET	Window Offset. The <code>SPORT_MCTL_A.WOFFSET</code> bits select the start location for the half SPORT's active window of channels within the 1024-channel range. A value of 0 specifies no offset and 896 is the largest value that permits using all 128 channels. When multichannel mode is disabled (<code>SPORT_MCTL_A.MCE = 0</code>) and the right-justified mode is enabled (<code>SPORT_CTL_A.RJUST = 1</code>), the least significant 6 bits of <code>SPORT_MCTL_A.WOFFSET</code> serve as the delay count (DCNT) field. These bits introduce a clock delay before the half SPORT state machine starts to capture data. For information about appropriate delay selections, see the SPORT operating modes section.
14:8 (R/W)	WSIZE	Window Size. The <code>SPORT_MCTL_A.WSIZE</code> bits select the window size for the half SPORT's active window of channels. Use the following formula to calculate the window size value: $\text{SPORT_MCTL_A.WSIZE} = (\text{number of channel slots}) - 1$
7:4 (R/W)	MFD	Multichannel Frame Delay. The <code>SPORT_MCTL_A.MFD</code> bits select the delay (in serial clock cycles) between the half SPORT's multichannel frame sync pulse and channel 0. The 4-bit field allows selecting multichannel frame delay of 0-15 serial clocks.

Table 26-31: SPORT_MCTL_A Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	MCPDE	Multichannel Packing DMA Enable. The <code>SPORT_MCTL_A.MCPDE</code> bit enables DMA data packing for transmit and enables DMA data unpacking for the half SPORT's multichannel data transfers.
		0 Disable
		1 Enable
0 (R/W)	MCE	Multichannel enable. The <code>SPORT_MCTL_A.MCE</code> bit enables multichannel operations for the half SPORT. The half SPORT is configured in normal multichannel mode if <code>SPORT_CTL_A.OPMODE=0</code> ; while it is configured in packed mode if <code>SPORT_CTL_A.OPMODE=1</code> . When configuring in these modes, the multichannel enable bit (<code>SPORT_MCTL_A.MCE</code>) should be set before enabling the SPORT data channel enable bits (<code>SPORT_CTL_A.SPENPRI</code> and/or <code>SPORT_CTL_A.SPENSEC</code>). When these channel bits transition from 1 to 0, note that the half SPORT's data transfer buffers are cleared, and the <code>SPORT_CTL_A.DERRPRI</code> and <code>SPORT_CTL_A.DERRSEC</code> bits are cleared.
		0 Disable
		1 Enable

Half SPORT 'B' Multichannel Control Register

The `SPORT_MCTL_B` register controls the half SPORT's multichannel operations. This register enables multichannel operation, enables multichannel data packing, selects the multichannel frame delay, selects the number of multichannel slots, and selects the multichannel window offset size.

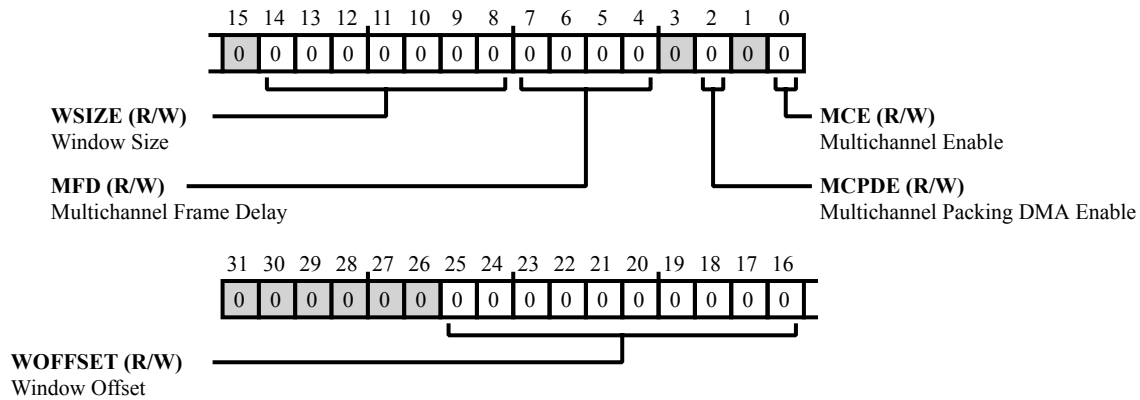


Figure 26-31: `SPORT_MCTL_B` Register Diagram

Table 26-32: `SPORT_MCTL_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:16 (R/W)	<code>WOFFSET</code>	Window Offset. The <code>SPORT_MCTL_B.WOFFSET</code> bits select the start location for the half SPORT's active window of channels within the 1024-channel range. A value of 0 specifies no offset and 896 is the largest value that permits using all 128 channels. When multichannel mode is disabled (<code>SPORT_MCTL_B.MCE = 0</code>) and right-justified mode is enabled (<code>SPORT_CTL_B.RJUST = 1</code>), the least significant 6 bits of <code>SPORT_MCTL_B.WOFFSET</code> serve as the delay count (DCNT) field. These bits introduce a clock delay before the half SPORT state machine starts to capture data. For information about appropriate delay selections, see the SPORT operating modes section.
14:8 (R/W)	<code>WSIZE</code>	Window Size. The <code>SPORT_MCTL_B.WSIZE</code> bits select the window size for the half SPORT's active window of channels. Use the following formula to calculate the window size value: $\text{SPORT_MCTL_B.WSIZE} = (\text{number of channel slots}) - 1$
7:4 (R/W)	<code>MFD</code>	Multichannel Frame Delay. The <code>SPORT_MCTL_B.MFD</code> bits select the delay (in serial clock cycles) between the half SPORT's multichannel frame sync pulse and channel 0. The 4-bit field allows selecting a multichannel frame delay of 0-15 serial clocks.

Table 26-32: SPORT_MCTL_B Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	MCPDE	Multichannel Packing DMA Enable. The SPORT_MCTL_B.MCPDE bit enables DMA data packing for transmit and enables DMA data unpacking for the half SPORT's multichannel data transfers.
		0 Disable
		1 Enable
0 (R/W)	MCE	Multichannel Enable. The SPORT_MCTL_B.MCE bit enables multichannel operations for the half SPORT. The half SPORT is configured in normal multichannel mode if SPORT_CTL_B.OPMODE=0; while it is configured in packed mode if SPORT_CTL_B.OPMODE=1. When configuring in these modes, the multichannel enable bit (SPORT_MCTL_B.MCE) should be set before enabling SPORT data channel enable bits (SPORT_CTL_B.SPENPRI and/or SPORT_CTL_B.SPENSEC). When these channel bits transition from 1 to 0, note that the half SPORT's data transfer buffers are cleared, and the SPORT_CTL_B.DERRPRI and SPORT_CTL_B.DERRSEC bits are cleared.
		0 Disable
		1 Enable

Half SPORT 'A' Multichannel Status Register

The `SPORT_MSTAT_A` register indicates the current multichannel being serviced among the half SPORT's active channels in multichannel mode. The half SPORT increments the value by one in this register as each channel is serviced. The value in the `SPORT_MSTAT_A` register restarts at 0 at each frame sync.

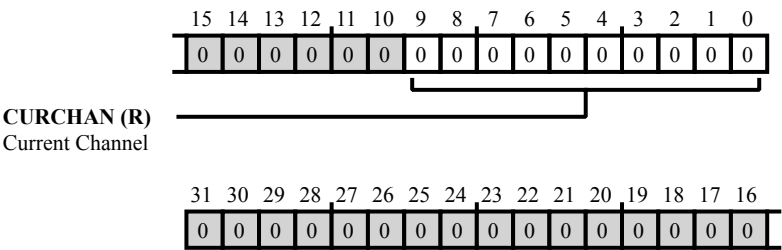


Figure 26-32: `SPORT_MSTAT_A` Register Diagram

Table 26-33: `SPORT_MSTAT_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/NW)	CURCHAN	Current Channel. The <code>SPORT_MSTAT_A.CURCHAN</code> bits indicate the half SPORT's current channel being serviced in multichannel mode.

Half SPORT 'B' Multichannel Status Register

The `SPORT_MSTAT_B` register indicates the current multichannel being serviced among the half SPORT's active channels in multichannel mode. The half SPORT increments the value by one in this register as each channel is serviced. The value in the `SPORT_MSTAT_B` register restarts at 0 at each frame sync.

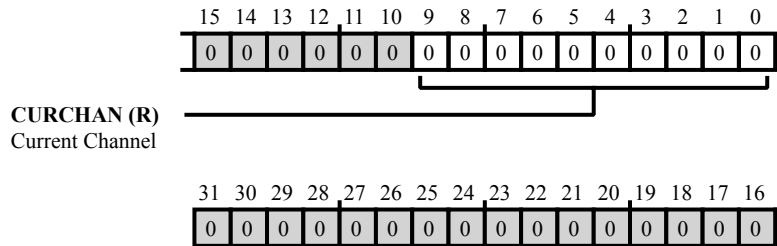


Figure 26-33: `SPORT_MSTAT_B` Register Diagram

Table 26-34: `SPORT_MSTAT_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:0 (R/NW)	CURCHAN	Current Channel. The <code>SPORT_MSTAT_B.CURCHAN</code> bits indicate the half SPORT's current channel being serviced in multichannel mode.

Half SPORT 'A' Rx Buffer (Primary) Register

The `SPORT_RXPRI_A` register buffers the half SPORT's primary channel receive data. This buffer becomes active when the half SPORT is configured to receive data on the primary channel. After a complete word has been received in the receive shifter, it is placed into the `SPORT_RXPRI_A` register. This data can be read in core mode (in interrupt-based or polling-based mechanism) or directly transferred into processor memory using the DMA controller. With a data buffer and an input shift register, the `SPORT_RXPRI_A` register acts as a two-location buffer. So, the SPORT can keep a maximum of two 32-bit received words at any given time (independent of the `SPORT_CTL_A.PACK` bit setting).

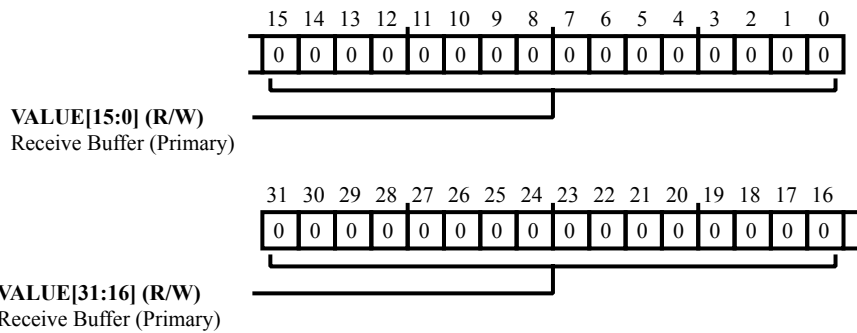


Figure 26-34: SPORT_RXPRI_A Register Diagram

Table 26-35: SPORT_RXPRI_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Receive Buffer (Primary). The <code>SPORT_RXPRI_A.VALUE</code> bits hold the half SPORT's primary channel receive data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_A.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_A</code> and <code>SPORT_MCTL_A</code> register descriptions.

Half SPORT 'B' Rx Buffer (Primary) Register

The `SPORT_RXPRI_B` register buffers the half SPORT's primary channel receive data. This buffer becomes active when the half SPORT is configured to receive data on the primary channel. After a complete word has been received in the receive shifter, it is placed into the `SPORT_RXPRI_B` register. This data can be read in core mode (in interrupt-based or polling-based mechanism) or directly transferred into processor memory using the DMA controller. With a data buffer and an input shift register, the `SPORT_RXPRI_B` register acts as a two-location buffer. So, the SPORT can keep a maximum of two 32-bit received words at any given time (independent of the `SPORT_CTL_A.PACK` bit setting).

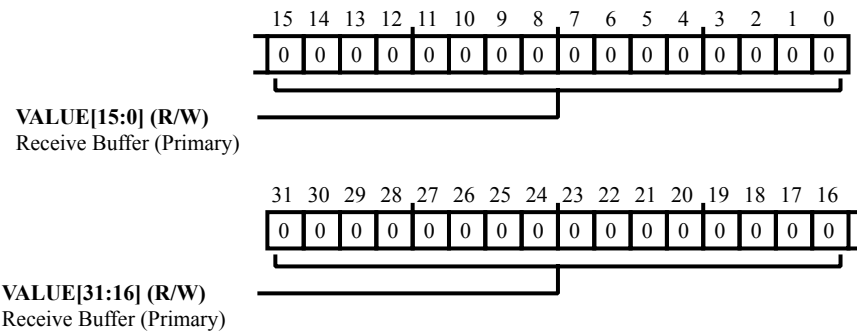


Figure 26-35: `SPORT_RXPRI_B` Register Diagram

Table 26-36: `SPORT_RXPRI_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Receive Buffer (Primary). The <code>SPORT_RXPRI_B.VALUE</code> bits hold the half SPORT's primary channel receive data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_B.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_B</code> and <code>SPORT_MCTL_B</code> register descriptions.

Half SPORT 'A' Rx Buffer (Secondary) Register

The `SPORT_RXSEC_A` register buffers the half SPORT's secondary channel receive data. This buffer becomes active when the half SPORT is configured to receive data on the secondary channel. After a complete word has been received in the receive shifter, it is placed into the `SPORT_RXSEC_A` register. This data can be read in core mode (in interrupt-based or polling-based mechanism) or directly transferred into processor memory using the DMA controller. With a data buffer and an input shift register, the `SPORT_RXSEC_A` register acts as a two-location buffer. So, the SPORT can keep a maximum of two 32-bit received words at any given time (independent of the `SPORT_CTL_A.PACK` bit setting).

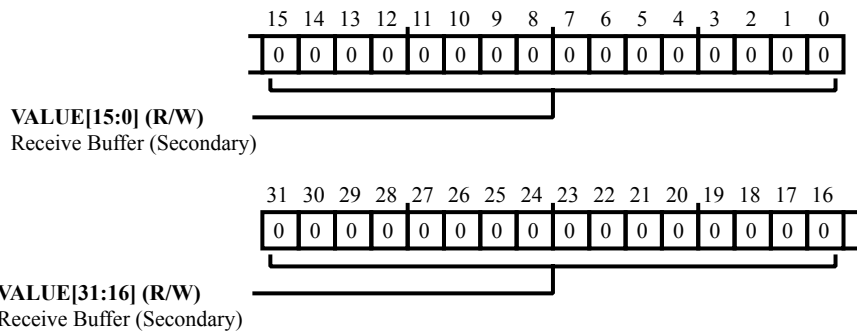


Figure 26-36: `SPORT_RXSEC_A` Register Diagram

Table 26-37: `SPORT_RXSEC_A` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Receive Buffer (Secondary). The <code>SPORT_RXSEC_A.VALUE</code> bits hold the half SPORT's secondary channel receive data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_A.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_A</code> and <code>SPORT_MCTL_A</code> register descriptions.

Half SPORT 'B' Rx Buffer (Secondary) Register

The `SPORT_RXSEC_B` register buffers the half SPORT's secondary channel receive data. This buffer becomes active when the half SPORT is configured to receive data on the secondary channel. After a complete word has been received in the receive shifter, it is placed into the `SPORT_RXSEC_B` register. This data can be read in core mode (in interrupt-based or polling-based mechanism) or directly transferred into processor memory using the DMA controller. With a data buffer and an input shift register, the `SPORT_RXSEC_B` register acts as a two-location buffer. So, the SPORT can keep a maximum of two 32-bit received words at any given time (independent of the `SPORT_CTL_A.PACK` bit setting).

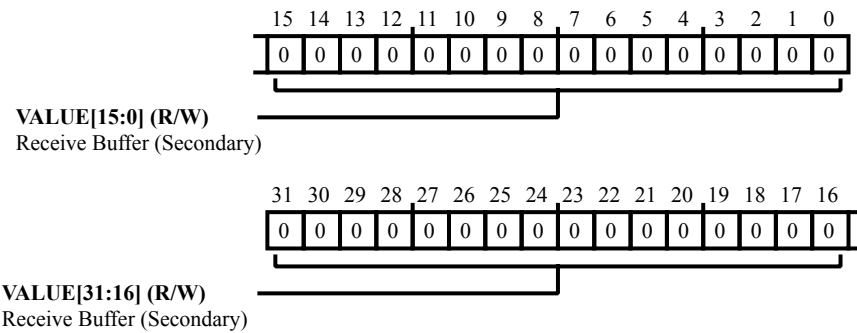


Figure 26-37: `SPORT_RXSEC_B` Register Diagram

Table 26-38: `SPORT_RXSEC_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Receive Buffer (Secondary). The <code>SPORT_RXSEC_B.VALUE</code> bits hold the half SPORT's secondary channel receive data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_B.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_B</code> and <code>SPORT_MCTL_B</code> register descriptions.

Half SPORT 'A' Tx Buffer (Primary) Register

The `SPORT_TXPRI_A` register buffers the half SPORT's primary channel transmit data. This register must be loaded with the data to be transmitted if the half SPORT is configured to transmit on the primary channel. Either a program running on the processor core loads the data into the buffer (word-by-word process) or the DMA controller automatically loads the data into the buffer (DMA process).

The `SPORT_TXPRI_A` register acts as a three-location buffer if SPORT data packing is disabled (`SPORT_CTL_A.PACK = 0`); while it acts as a two-location buffer when packing is enabled (`SPORT_CTL_A.PACK = 1`). So, depending on the `PACK` bit setting, two 32-bit words or three 32-bit words can be stored in the transmit queue at any time. When the transmit register is loaded and any previous word has been transmitted, the `SPORT_TXPRI_A` register contents are automatically loaded into the output shifter. The half SPORT can issue an interrupt request (transmit buffer is not full) when it has loaded the output transmit shifter, signifying that the transmit buffer is ready to accept the next word. This interrupt request does not occur when the half SPORT is executing a DMA-based transfer.

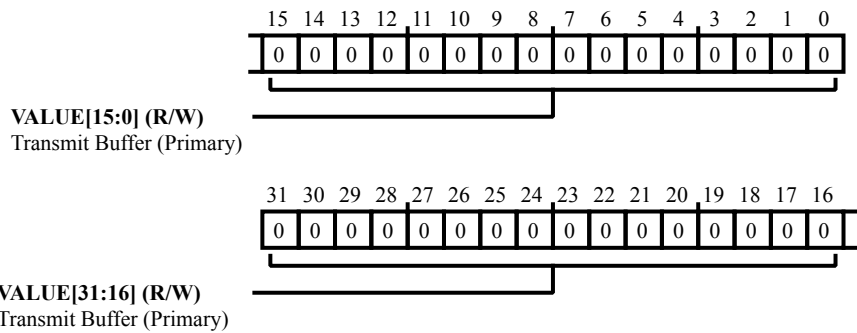


Figure 26-38: SPORT_TXPRI_A Register Diagram

Table 26-39: SPORT_TXPRI_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Transmit Buffer (Primary). The <code>SPORT_TXPRI_A.VALUE</code> bits hold the half SPORT's primary channel transmit data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_A.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_A</code> and <code>SPORT_MCTL_A</code> register descriptions.

Half SPORT 'B' Tx Buffer (Primary) Register

The `SPORT_TXPRI_B` register buffers the half SPORT's primary channel transmit data. This register must be loaded with the data to be transmitted if the half SPORT is configured to transmit on the primary channel. Either a program running on the processor core loads the data into the buffer (word-by-word process) or the DMA controller automatically loads the data into the buffer (DMA process).

The `SPORT_TXPRI_B` register acts as a three-location buffer if SPORT data packing is disabled (`SPORT_CTL_B.PACK = 0`); while it acts as a two-location buffer when packing is enabled (`SPORT_CTL_B.PACK = 1`). So, depending on the `PACK` bit setting, two 32-bit words or three 32-bit words can be stored in the transmit queue at any time. When the transmit register is loaded and any previous word has been transmitted, the `SPORT_TXPRI_B` register contents are automatically loaded into the output shifter. The half SPORT can issue an interrupt request (transmit buffer is not full) when it has loaded the output transmit shifter, signifying that the transmit buffer is ready to accept the next word. This interrupt request does not occur when the half SPORT is executing a DMA-based transfer.

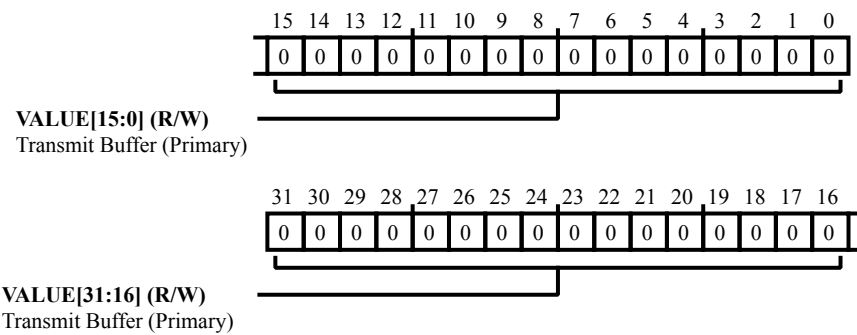


Figure 26-39: `SPORT_TXPRI_B` Register Diagram

Table 26-40: `SPORT_TXPRI_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Transmit Buffer (Primary). The <code>SPORT_TXPRI_B.VALUE</code> bits hold the half SPORT's primary channel transmit data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_B.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_B</code> and <code>SPORT_MCTL_B</code> register descriptions.

Half SPORT 'A' Tx Buffer (Secondary) Register

The `SPORT_TXSEC_A` register buffers the half SPORT's secondary channel transmit data. This register must be loaded with the data to be transmitted if the half SPORT is configured to transmit on the secondary channel. Either a program running on the processor core loads the data into the buffer (word-by-word process) or the DMA controller automatically loads the data into the buffer (DMA process).

The `SPORT_TXSEC_A` register acts as a three-location buffer if SPORT data packing is disabled (`SPORT_CTL_A.PACK = 0`); while it acts as a two-location buffer when packing is enabled (`SPORT_CTL_A.PACK = 1`). So, depending on the `PACK` bit setting, two 32-bit words or three 32-bit words can be stored in the transmit queue at any time. When the transmit register is loaded and any previous word has been transmitted, the `SPORT_TXSEC_A` register contents are automatically loaded into the output shifter. The half SPORT can issue an interrupt request (transmit buffer is not full) when it has loaded the output transmit shifter, signifying that the transmit buffer is ready to accept the next word. This interrupt request does not occur when the half SPORT is executing a DMA-based transfer.

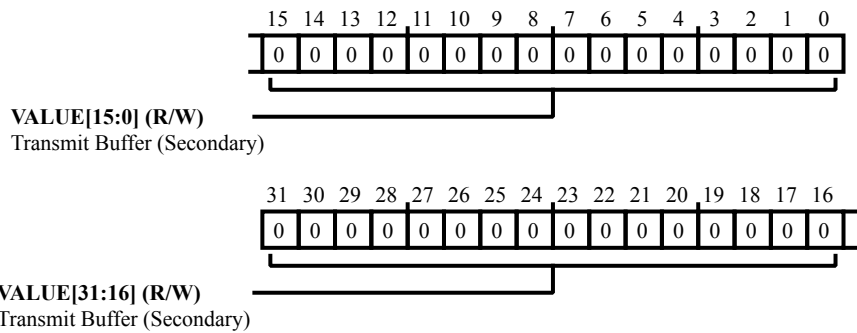


Figure 26-40: SPORT_TXSEC_A Register Diagram

Table 26-41: SPORT_TXSEC_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Transmit Buffer (Secondary). The <code>SPORT_TXSEC_A.VALUE</code> bits hold the half SPORT's secondary channel transmit data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_A.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_A</code> and <code>SPORT_MCTL_A</code> register descriptions.

Half SPORT 'B' Tx Buffer (Secondary) Register

The `SPORT_TXSEC_B` register buffers the half SPORT's secondary channel transmit data. This register must be loaded with the data to be transmitted if the half SPORT is configured to transmit on the secondary channel. Either a program running on the processor core loads the data into the buffer (word-by-word process) or the DMA controller automatically loads the data into the buffer (DMA process).

The `SPORT_TXSEC_B` register acts as a three-location buffer if SPORT data packing is disabled (`SPORT_CTL_B.PACK = 0`); while it acts as two-location buffer when packing is enabled (`SPORT_CTL_B.PACK = 1`). So, depending on the `PACK` bit setting, two 32-bit words or three 32-bit words can be stored in the transmit queue at any time. When the transmit register is loaded and any previous word has been transmitted, the `SPORT_TXSEC_B` register contents are automatically loaded into the output shifter. The half SPORT can issue an interrupt request (transmit buffer is not full) when it has loaded the output transmit shifter, signifying that the transmit buffer is ready to accept the next word. This interrupt request does not occur when the half SPORT is executing a DMA-based transfer.

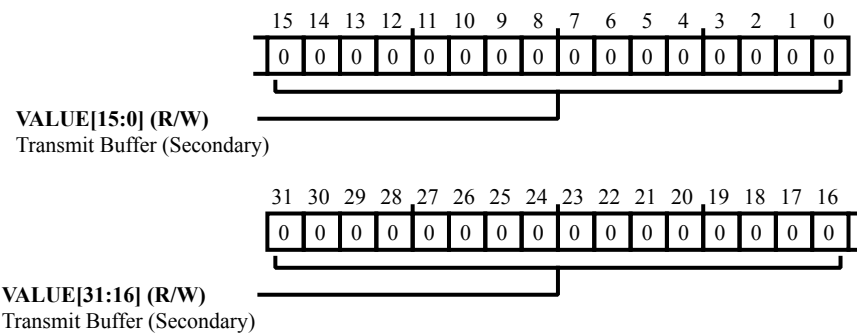


Figure 26-41: `SPORT_TXSEC_B` Register Diagram

Table 26-42: `SPORT_TXSEC_B` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Transmit Buffer (Secondary). The <code>SPORT_TXSEC_B.VALUE</code> bits hold the half SPORT's secondary channel transmit data. Note that changes to the half SPORT operation mode (for example, toggling the <code>SPORT_MCTL_B.MCE</code>) empty the contents of this data buffer. For more information, see the <code>SPORT_CTL_B</code> and <code>SPORT_MCTL_B</code> register descriptions.

27 Precision Clock Generators (PCG)

The precision clock generators are used to produce a pair of signals from a clock input signal. The two signals generated are normally used as a serial bit clock and frame sync pair. The PCG is part of the DAI. There are four PCG units per DAI. *PCGA*, *PCGB*, *PCGE*, and *PCGF* belong to DAI0; *PCGC*, *PCGD*, *PCGG*, and *PCGH* belong to DAI1.

Each of these four units (A, B, E, and F) generates one clock (CLKA_O, CLKB_O, CLKE_O, and CLKF_O) and one frame sync (FSA_O, FSB_O, FSE_O, and FSF_O) output. In addition to clock and frame sync outputs, each of these four units generates one inverted version of clock output (INV_CLKA_O, INV_CLKB_O, INV_CLKE_O, and INV_CLKF_O) and one inverted version of frame sync output (INV_FSA_O, INV_FSB_O, INV_FSE_O, and INV_FSF_O).

Features

The following list describes the features of the precision clock generators.

- SRU allows the routing of all of the PCG signals in one DAI (two PCG units in DAI)
- Input clock selection: SYS_CLKIN0 and SYS_CLKIN1 can be used as the input clock for PCG, when configured to use CLKIN as clock source
- Provides four different clock dividers for serial clock, frame sync, phase (20-bit), and pulse width (16-bit)
- Phase shift allows adjustment of the frame sync relative to the serial clock and can be shifted the full period and wrapped around
- Provides pulse width control for arbitrary frame sync signal generation
- Bypass mode for external frame sync manipulation
- External trigger mode starts PCG operation
- No additional jitter is introduced when using off-chip clocks
- Global enable. (See the DAI chapter for details)

Functional Description

The *PCG Block Diagram* shows the blocks within the module and its connection to the DAI. The following sections provide information on the function of these blocks.

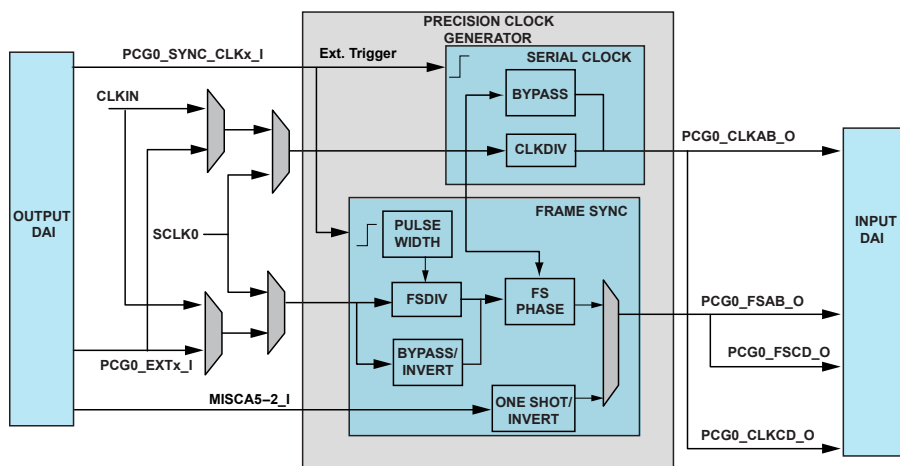


Figure 27-1: PCG Block Diagram

NOTE: In the *PCG Block Diagram*, the CLKINSEL bit field determines whether SYS_CLKIN0 or SYS_CLKIN1 is input to DAI.

ADSP-2159x PCG Register List

Precision Clock Generator

Table 27-1: ADSP-2159x PCG Register List

Name	Description
PCG_CTLA0	Precision Clock A Control 0 Register
PCG_CTLA1	Precision Clock A Control 1 Register
PCG_CTLB0	Precision Clock B Control 0 Register
PCG_CTLB1	Precision Clock B Control 1 Register
PCG_CTLC0	Precision Clock C Control 0 Register
PCG_CTLC1	Precision Clock C Control 1 Register
PCG_CTLD0	Precision Clock D Control 0 Register
PCG_CTLD1	Precision Clock D Control 1 Register
PCG_CTLF0	Precision Clock F Control 0 Register
PCG_CTLF1	Precision Clock F Control 1 Register
PCG_CTLG0	Precision Clock G Control 0 Register
PCG_CTLG1	Precision Clock G Control 1 Register
PCG_CTLH0	Precision Clock H Control 0 Register
PCG_CTLH1	Precision Clock H Control 1 Register
PCG_CTLI0	Precision Clock I Control 0 Register
PCG_CTLI1	Precision Clock I Control 1 Register
PCG_CTLJ0	Precision Clock J Control 0 Register
PCG_CTLJ1	Precision Clock J Control 1 Register
PCG_CTLK0	Precision Clock K Control 0 Register
PCG_CTLK1	Precision Clock K Control 1 Register
PCG_CTLL0	Precision Clock L Control 0 Register
PCG_CTLL1	Precision Clock L Control 1 Register
PCG_CTLM0	Precision Clock M Control 0 Register
PCG_CTLM1	Precision Clock M Control 1 Register
PCG_CTLN0	Precision Clock N Control 0 Register
PCG_CTLN1	Precision Clock N Control 1 Register
PCG_CTLN2	Precision Clock N Control 2 Register
PCG_CTLN3	Precision Clock N Control 3 Register
PCG_CTLN4	Precision Clock N Control 4 Register
PCG_CTLN5	Precision Clock N Control 5 Register
PCG_CTLN6	Precision Clock N Control 6 Register
PCG_CTLN7	Precision Clock N Control 7 Register
PCG_CTLN8	Precision Clock N Control 8 Register
PCG_CTLN9	Precision Clock N Control 9 Register
PCG_CTLN10	Precision Clock N Control 10 Register
PCG_CTLN11	Precision Clock N Control 11 Register
PCG_CTLN12	Precision Clock N Control 12 Register
PCG_CTLN13	Precision Clock N Control 13 Register
PCG_CTLN14	Precision Clock N Control 14 Register
PCG_CTLN15	Precision Clock N Control 15 Register
PCG_CTLN16	Precision Clock N Control 16 Register
PCG_CTLN17	Precision Clock N Control 17 Register
PCG_CTLN18	Precision Clock N Control 18 Register
PCG_CTLN19	Precision Clock N Control 19 Register
PCG_CTLN20	Precision Clock N Control 20 Register
PCG_CTLN21	Precision Clock N Control 21 Register
PCG_CTLN22	Precision Clock N Control 22 Register
PCG_CTLN23	Precision Clock N Control 23 Register
PCG_CTLN24	Precision Clock N Control 24 Register
PCG_CTLN25	Precision Clock N Control 25 Register
PCG_CTLN26	Precision Clock N Control 26 Register
PCG_CTLN27	Precision Clock N Control 27 Register
PCG_CTLN28	Precision Clock N Control 28 Register
PCG_CTLN29	Precision Clock N Control 29 Register
PCG_CTLN30	Precision Clock N Control 30 Register
PCG_CTLN31	Precision Clock N Control 31 Register

Table 27-1: ADSP-2159x PCG Register List (Continued)

Name	Description
PCG_CTLF1	Precision Clock F Control 1 Register
PCG_CTLG0	Precision Clock G Control 0 Register
PCG_CTLG1	Precision Clock G Control 1 Register
PCG_CTLH0	Precision Clock H Control 0 Register
PCG_CTLH1	Precision Clock H Control 1 Register
PCG_PW1	Precision Clock Pulse Width Control 1 Register
PCG_PW2	Precision Clock Pulse Width Control 2 Register
PCG_PW3	Precision Clock Pulse Width Control 3 Register
PCG_PW4	Precision Clock Pulse Width Control 4 Register
PCG_SYNC1	Precision Clock Frame Sync Synchronization 1 Register
PCG_SYNC2	Precision Clock Frame Sync Synchronization 2 Register
PCG_SYNC3	Precision Clock Frame Sync Synchronization 3 Register
PCG_SYNC4	Precision Clock Frame Sync Synchronization 4 Register

Internal Interface

The fundamental clock of the PCG is *SCLK0*. The clock to this module can be shut off for power savings.

Serial Clock

Each of the four units (A, B, C, and D) produces a clock output. Serial clock generation from a unit is independently enabled and controlled. Sources for the serial clock generation can be either from the CLKIN, SCLK0, or a DAI pin source.

When CLKIN is chosen as input clock in PCG, the clock source bits (PCG_SYNC1.CLKA_CLKINSEL, PCG_SYNC1.CLKB_CLKINSEL) determine whether the clock source is SYS_CLKIN0 or SYS_CLKIN1.

Note that the divider is working in normal mode for PCG0_CTLx1.CLKDIV > 1. For PCG_CTLA1.CLKDIV/PCG_CTLB1.CLKDIV = 0 or 1, the divider operates in bypass mode (input clock is fed directly to its output). In bypass mode, the clock at the output can theoretically run at up to the SCLK0 frequency. Check the data sheet for the specified maximum operation speed of the DAI pin buffers.

Note that the clock output is always set (as closely as possible) to a 50% duty cycle. If the clock divisor is even, the duty cycle of the clock output is exactly 50%. If the clock divisor is odd, then the duty cycle is slightly less than 50%. The low period of the output clock is one input clock period more than the high period of the output clock. For higher values of an odd divisor, the duty cycle is close to 50%.

NOTE: A PCG clock output cannot be fed to its own input.

Frame Sync

The following sections describe the use of frame syncs in the PCGs.

Frame Sync Output

Each of the four units (A through D) produces a synchronization signal for framing serial data. The frame sync outputs are much more flexible since they must accommodate the wide variety of serial protocols used by peripherals.

Frame sync generation from a unit is independently enabled and controlled. Sources for the frame sync generation can be either from the crystal buffer output, SCLK0, or an external pin source. There is only one external source pin for both frame sync and clock output for a unit.

If an external source is selected for both frame sync and clock output for a unit, then they operate on the same input signal. Apart from enable and source select control bits, a 20-bit divisor controls frame sync generation.

Divider Mode Selection

If a frame sync divisor is greater than 1, the PCG frame sync output frequency is equal to the input clock frequency, divided by a 20-bit integer. This integer is specified in the `PCG_CTLA0.FSDIV/PCG_CTLB0.FSDIV` bit field (bits 19:0).

However, if the frame sync divisor is 0 or 1, the PCG's frame sync clock generation unit is bypassed, and the frame sync input is connected directly to the frame sync output. For `PCG_CTLB0.FSDIV = 0, 1` the differently than in normal mode.

Phase Shift

Phase shift is a frame sync parameter that defines the phase shift of the frame sync relative to the input clock of the same unit. This feature allows the shifting of the frame sync signal in time relative to the clock input signal. Frame sync phase shifting is often required by peripherals that need a frame sync signal to lead or lag a clock signal.

For example, the I²S protocol specifies that the frame sync transition from high-to-low occurs one clock cycle before the beginning of a frame. Since an I²S frame is 64 clock cycles long, delaying the frame sync by 63 cycles produces the required framing.

Phase shifting is represented as a full 20-bit value. Even when the frame sync is divided by the maximum amount, the phase can be shifted to the full range, from zero to one input clock short of the period.

NOTE: Phase shifting is specified as a 2 x 10-bit divider value in the `PCG_CTLA0.FSPHASEHI` bit field (bits 29:20) and in the `PCG_CTLA1.FSPHASELO` bit field (bits 29:20).

A single 20-bit value spans these two-bit fields. The upper half of the word (bits 19:10) is in the `PCG_CTLA0` register, and the lower half (bits 9:0) is in the `PCG_CTLA1` register.

The phase shift between clock and frame sync outputs can be programmed using the and all of the control registers under these conditions:

- The input clock source for the clock generator output and the frame sync generator output is the same.

- The clock and frame sync are enabled at the same time using a single atomic instruction.
- The frame sync divisor is an integral multiple of the clock divisor.

NOTE: When using a clock and frame sync as a synchronous pair, the units must be enabled in a single atomic instruction before their parameters are modified. Both units must also be disabled in a single atomic instruction.

NOTE: If the phase shift is 0 (see the *Phase and Pulse Width Settings* figure), the clock and frame sync outputs rise at the same time.

If the phase shift is 1, the frame sync output transitions one input clock period ahead of the clock transition.

If the phase shift is $divisor - 1$, the frame sync transitions $divisor - 1$ input clock periods ahead of the clock transitions.

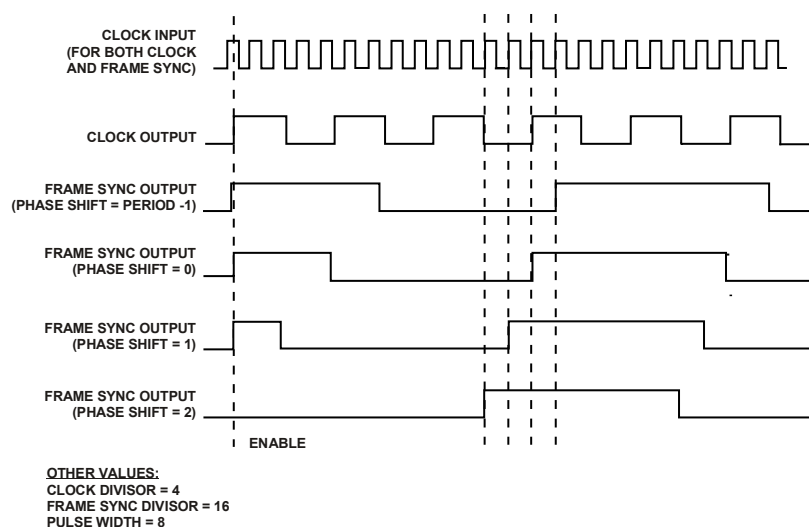


Figure 27-2: Phase and Pulse Width Settings

NOTE: When generating single frame sync pulses (the length of one SPORT clock cycle), take care with respect to the drive and sampling edges. If the rules are violated, for example if the SPORT is not driving data, the module cannot detect a valid sample edge.

Pulse Width

Pulse width is the number of input clock periods for which the frame sync output is high.

A 16-bit value determines the width of the framing pulse. Settings for pulse width can range from zero to $DIV - 1$. The pulse width should be less than the divisor of the frame sync.

Default Pulse Width

If the pulse width count is equal to 0 and if the `PCG_CTLA0.FSDIV/PCG_CTLB0.FSDIV` bit field is even, then the actual pulse width of the frame sync output is equal to:

For even divisors: frame sync divisor/2

If the pulse width count is equal to 0 and if the `PCG_CTLA0.FSDIV/PCG_CTLB0.FSDIV` bit field is odd, then the actual pulse width of the frame sync output is equal to:

For odd divisors: frame sync divisor – 1/2

Input Clock Source Considerations

The core Phase-Locked Loop (PLL) has been designed to provide clocking for the processor core. The performance specifications of this PLL are appropriate for the core. But, they have not been optimized or specified for precision data converters where jitter directly translates into time quantization errors and distortion.

Therefore, the PCG allows the routing of external clock sources which are independent of the core PLL.

Timing Example for I²S Mode

For I²S mode, the frame sync should be driven at the falling edge of SPORT clock. In other words, the frame sync edge must coincide with the falling edge of the SPORT clock. To satisfy this requirement, program the phase of the frame sync accordingly in the PCG control registers.

For example, assume that the input clock source for both clock and frame sync are the same and both the clock and frame sync are enabled at the same time. Also, assume that the clock divisor value for generating the required SPORT clock is `PCG_CTLA1.CLKDIV = 4`. Then, for a 32-bit word length, the frame sync divisor value is:

`PCG_CTLA0.FSDIV = 64`, `PCG_CTLA1.CLKDIV = 256`.

By default, for phase = 0, the rising edge of both SPORT clock and frame sync coincide. To make sure that the frame sync edges coincide with the falling edge of the SPORT clock, program the phase value as:

`PCG_CTLA1.CLKDIV/2 = 2`.

Cross Mode Connections

The symmetric dual DAI architecture allows cross connections between both PCGs (A,B) and (C,D) to the other DAI. Each PCG (A through D) supports an alternative input clock (`PCG0_EXTx_I`) (see [Figure 27-1 PCG Block Diagram](#)) which can be sourced via a DAI pin buffer from the other DAI. Note however if routing a source (clock or FS) only DAI pin buffer 2 to 20 can be used (DAI pin buffer 1 is no longer available and is replaced by the DAI CRS buffer for the other DAI). See DAI Routing Capabilities for more information.

Operating Modes

The following sections provide information on the operating modes of the precision clock generator.

Normal Mode

When the frame sync divisor is set to any value other than zero or one, the PCGs operate in normal mode. In normal mode, the divisor determines the frequency of the frame sync output where:

Frequency of Frame Sync Output = Input Frequency/Divisor

The value of the pulse width control determines the high period of the frame sync output. The value of the pulse width control must be less than the value of the divisor.

The value of the phase control determines the phase of the frame sync output. If the phase is zero, then the positive edges of the clock and frame sync coincide when:

- the clock and frame sync dividers are enabled at the same time using an atomic instruction
- the divisors of the clock and frame sync are the same
- the source for the clock and frame sync is the same

The number of input clock cycles that have already elapsed before the frame sync is enabled is equal to the difference between the divisor and the phase values. If the phase is a small fraction of the divisor, then the frame sync appears to lead the clock. If the phase is only slightly less than the frame sync divisor, then the frame sync appears to lag the clock. The frame sync phase must not be greater than the divisor.

Bypass Mode

When the frame sync divisor for the frame sync has a value of zero or one, the frame sync is in bypass mode, and the different functionality than in normal mode.

NOTE: In normal mode, bits 15:0 and 31:18 of the used to program the pulse width count. In bypass mode, bits 15:2 and 31:18 are ignored. Bits 1:0 and 17:16 are renamed to `PCG_PW1 . STROBEA` and `PCG_PW1 . INVFSA`, respectively. This functionality is described in more detail as follows.

If the `PCG_PW1 . STROBEA` register is cleared, then the input is directly passed (see the *Bypass and Inverted Bypass* figure) to the frame sync output either inverted or not inverted, depending on the `PCG_PW1 . INVFSA` bits.

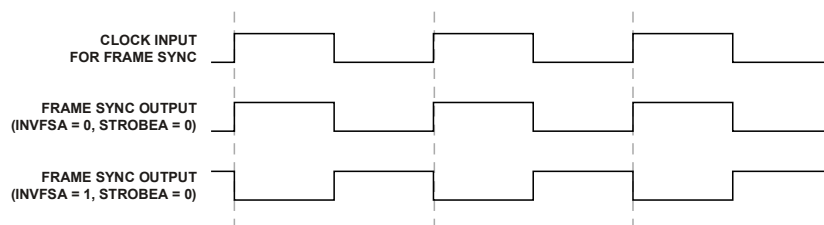


Figure 27-3: Bypass and Inverted Bypass

One-Shot Mode

In the one-shot mode operation shown in the *One Shot Mode PCG A (MISCA2_I Input)* figure, the PCG produces a series of periods but does not run continuously.

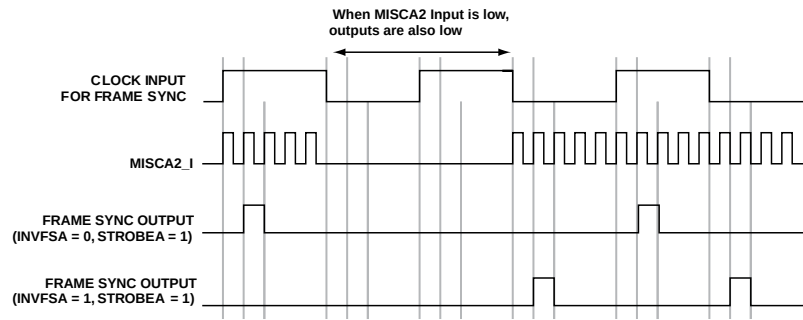


Figure 27-4: One Shot Mode PCG A (MISCA2_I Input)

Bypass mode also enables the generation of a strobe pulse (one-shot frame sync). Strobe usage ignores the divider counters and looks to the SRU to provide the input signal.

In the bypass mode, if the `PCG_PW1.STROBEA` bit = 1, then a one-shot pulse is generated. This one-shot pulse has the duration equal to the period of `DAI_MISCAx_I` for the `PCGx` unit. This pulse is generated either at the falling or rising edge of the input clock, depending on the value of the `PCG_PW1.INVFSA` bit. The output pulse width is equal to the period of the SRU source signal `DAI_MISCAx_I`. The pulse begins at the second rising edge of `MISCAx_I` following a rising edge of the clock input. When the `PCG_PW1.INVFSA` bit is set, the pulse begins at the second rising edge of `DAI_MISCAx_I` coinciding with or following a falling edge of the clock input.

NOTE: A strobe period is defined to be the period of the FS input clock signal as specified by the `PCG_CTLA1.FSSRC` bit.

Audio System Example

The *PCG Setup for I2S or Left-Justified DAI* figure shows an example of the interconnections between the S/PDIF receiver, ASRC, and the PCGs. The interconnections are made by programming the signal routing unit. It shows how to set up two precision clock generators using the S/PDIF receiver and an asynchronous sample rate converter (ASRC) to interface to an external audio DAC. The PCG is configured to provide a fixed ASRC/DAC output sample rate of 65.098 kHz. The input to the S/PDIF receiver is typically 44.1 kHz if supplied by a CD player, but can also be from other source at any nominal sample rates.

Similarly, the phase shift for frame syncs B, C, and D is specified in the corresponding control registers (`PCG_CTLA0` through `PCG_CTLB1`).

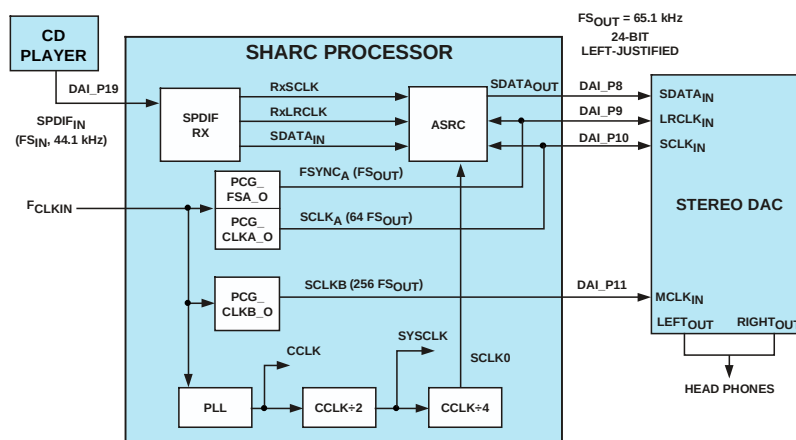


Figure 27-5: PCG Setup for I²S or Left-Justified DAI

Three synchronous clocks are required in audio systems:

1. Frame sync (FS)
2. Serial bit clock (64 FS)
3. Master DAC clock (256 FS)

Since each PCG has only two outputs, this example requires two PCGs. Furthermore, because the digital audio interface requires a fixed-phase relation between serial clock and FS, these two outputs should come from one PCG (PCG A). The master clock comes from the second (PCG B).

The combined PCGs can provide a selection of synchronous clock frequencies to support alternate sample rates for the ASRCs and external DACs. However, the range of choices is limited by CLKIN and the ratio of PCG_CLKx_O: serial clock:FS. The ratio is normally fixed at 256:64:1 to support digital audio left-justified, I²S and right-justified interface modes.

Many DACs also support 384, 512, and 786x FS for PCG_CLKx_O, which allows some additional flexibility in choosing serial clock.

Note that the falling edge of serial clock must always be synchronous with both edges of FS. This condition requires that the phase of the serial clock and FS signals for a common PCG (PCG A) be adjustable.

While the frequency of the master DAC clock (PCG_CLKx_O) must be synchronous with the sample rate supplied to the external DAC, there is no fixed-phase requirement.

Set the clock divisor and source and low-phase word first, followed by the control register enable bits, which are set together. When the set to zero (default), the FS pulse width is (divisor 2) for even divisors and (divisor - 1) 2 for odd divisors. Alternatively, the can be set high for exactly one-half the period of CLKIN cycles for a 50% duty cycle, provided the FS divisor is an even number.

Hardware Trigger Control

In addition to the existing enable control to generate the clock and frame sync, this feature provides a flexibility to start the clock and frame sync generation upon receiving a hardware trigger (internal slave trigger). This feature applies to all modes of operation.

For example, when the `PCG_SYNC1.HWB_TRIGEN` bit is set, it enables the synchronization of clock A and frame sync A to the external LRCLK sync (in external LRCLK synchronization mode) upon receiving a hardware trigger. Refer to the `PCG_SYNC1.HWA_TRIGEN/PCG_SYNC1.HWB_TRIGEN` and `PCG_SYNC2.HWC_TRIGEN/PCG_SYNC2.HWD_TRIGEN` bit descriptions.

NOTE: To exercise the hardware trigger based synchronization, program the destination registers (`DAI_MISC2.IN0`, `DAI_MISC2.IN1`, `DAI_MISC2.IN2`, and `DAI_MISC2.IN3`) to route the trigger signals to the PCGs. For example, if PCGA must be enabled for the hardware trigger based synchronization, program the `DAI_MISC2.IN0` and `DAI_EXTD_MISC2.IN0` bit fields to `5'b00000` and `2'b01`, respectively. Each PCG has a dedicated trigger control from the TRU unit, there are four trigger controls for DAI0 and four trigger controls for DAI1.

Refer to TRU Slave Trigger list table for the slave trigger information.

Clock Configuration Examples

For a $CLKIN = 33.330$ MHz, the two PCGs provide the three synchronous clocks `PCGx_CLK`, serial clock and *FS* for the SRCs and external DAC. These divisors are stored in the `PCG_CTLA1.CLKDIV/PCG_CTLB1.CLKDIV` bit fields.

The integer divisors for several possible sample rates based on 33.330 MHz *CLKIN* are shown in the *Precision Clock Generator Division Ratios* table.

Table 27-2: Precision Clock Generator Division Ratios (33.330 CLKIN)

Sample Rate kHz)	PCG Divisors		FSDIV A ^{*1}
	CLKDIV B	CLKDIV A	
130.195	1	4	256
<i>65.098</i>	<i>2</i>	<i>8</i>	<i>512</i>
43.398	3	12	768
32.549	4	16	1024
26.039	5	20	1280
21.699	6	24	1536
18.599	7	28	1792

*1 The frame sync divisor must be an even integer to produce a 50% duty cycle waveform. See [Frame Sync](#).

Global PCG Enable

For some applications, enabling multiple PCGs (CLK and FS) must be synchronized so that they start at the same time. This feature provides the flexibility to enable multiple PCGs (CLK and FS) globally. The global PCG enable feature works independently, while the legacy PCG enable (CLK and FS) remains the same. The legacy PCG enable can be used when the PCGs need to be enabled individually.

The `DAI_GBL_PCG_EN` register controls the enabling and disabling of PCGs globally. The global enable feature can be used in the following scenarios:

- to enable all the eight PCGs (includes both DAI0 and DAI1) at the same time
- to enable all four PCGs within each DAIx at the same time
- to enable multiple PCGs (across DAIs) at the same time

NOTE: See the `DAI_GBL_PCG_EN` register description for details.

Programming Guidelines

TBD

PCG Event Control

The following sections describe the generation and control of PCG events.

External Event Trigger

The trigger with the external clock is enabled by setting bits 0 and 16 of the `PCG_SYNC1/PCG_SYNC2/PCG_SYNC3/PCG_SYNC4` registers .

Refer to the *FS Output Synchronization With External Trigger Input* figure. Since the rising edge of the external clock is used to synchronize with the frame sync, the frame sync output is not generated until a rising edge of the external clock is sensed.

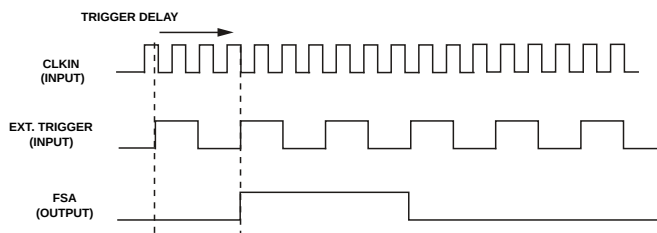


Figure 27-6: FS Output Synchronization With External Trigger Input

External Event Trigger Delay

The time delay between the rising trigger edge and the start of serial clock and frame sync varies between 2.5 to 3.5 input clock periods. If the input clock and the trigger signal are synchronous, the delay is 3 input clock periods. Consider the following cases:

- SCLK0 is the input source. In this case, if the given trigger event is synchronous to SCLK0, the delay is 3 SCLK0 periods. If the trigger signal is asynchronous with SCLK0, the delay varies from 2.5 SCLK0 periods to 3.5 SCLK0 periods. (It depends on whether the trigger edge occurs in the positive half cycle or negative half cycle of SCLK0.)
- The CLKIN source bits of the `PCG_SYNC1` register select the input source for PCG. In this case, if the given trigger signal is synchronous to CLKIN, the delay is 3 CLKIN periods. But if they are asynchronous to CLKIN, the delay can vary between 2.5 CLKIN periods to 3.5 CLKIN periods.
- SRU DAI0 is the input source for PCG A, B. If the input clock and trigger signal are synchronous, the delay is exactly 3 input clock periods. If asynchronous, it varies between 2.5 to 3.5 input clock periods depending on the phase difference between the input clock and trigger signal.

Programming Model

This section describes the sequence of software steps required for successful PCG operation.

If the PCG is disabled to reprogram a parameter, use a delay after writing to the disable bit. This delay in core clock (*CCLK*) cycles = (PCG source clock period/*CCLK* period). In summary, use the following general procedure:

1. Clear the PCG enable bits without modifying any other settings.
2. Wait for N *CCLK* cycles (N = PCG source clock period/processor clock period).
3. Program all new parameters without setting the PCG enable bit.
4. Enable the PCG.

Frame Sync Phase Setting

The phase unit requires that the clock and FS are enabled simultaneously in an atomic instruction.

1. Write the clock divider/low 10-bit phase divider to the `PCG_CTLA1/PCG_CTLB1` registers.
2. Program the FS divider/high 10-bit phase divider, enabling both the `PCG_CTLA0.CLKEN/PCG_CTLA0.FSEN` and the `PCG_CTLB0.CLKEN/PCG_CTLB0.FSEN` bits.

Note that both units must be disabled in the same way.

External Event Trigger

The trigger with the external clock is enabled by setting bits 0 and 16 of the `PCG_SYNC` register. Program the phase to 3, so that the rising edge of the external clock is in-sync with the frame sync (*FS Output Synchronization With External Trigger Input*).

Use the following steps.

1. Program the and the `PCG_CTLA0` through `PCG_CTLB1` registers appropriately.
2. Enable the clock or frame sync, or both.

Since the rising edge of the external clock is used to synchronize with the frame sync, the frame sync output is not generated until a rising edge of the external clock is sensed.

Debug Features

Take care in cases where any input to the phase unit is modified. Any individual change of the `PCG_CTLA1.CLKDIV` or `PCG_CTLA0.FSDIV` dividers can cause a failure in the PCG sync operation between the serial clock and the frame sync. Only the programming model ensures a correct setup for phase settings.

ADSP-2159x PCG Register Descriptions

Precision Clock Generator (PCG) contains the following registers.

Table 27-3: ADSP-2159x PCG Register List

Name	Description
<code>PCG_CTLA0</code>	Precision Clock A Control 0 Register
<code>PCG_CTLA1</code>	Precision Clock A Control 1 Register
<code>PCG_CTLB0</code>	Precision Clock B Control 0 Register
<code>PCG_CTLB1</code>	Precision Clock B Control 1 Register
<code>PCG_CTLC0</code>	Precision Clock C Control 0 Register
<code>PCG_CTLC1</code>	Precision Clock C Control 1 Register
<code>PCG_CTLD0</code>	Precision Clock D Control 0 Register
<code>PCG_CTLD1</code>	Precision Clock D Control 1 Register
<code>PCG_CTLF0</code>	Precision Clock E Control 0 Register
<code>PCG_CTLF1</code>	Precision Clock E Control 1 Register
<code>PCG_CTLF0</code>	Precision Clock F Control 0 Register
<code>PCG_CTLF1</code>	Precision Clock F Control 1 Register
<code>PCG_CTLG0</code>	Precision Clock G Control 0 Register
<code>PCG_CTLG1</code>	Precision Clock G Control 1 Register
<code>PCG_CTLH0</code>	Precision Clock H Control 0 Register
<code>PCG_CTLH1</code>	Precision Clock H Control 1 Register
<code>PCG_PW1</code>	Precision Clock Pulse Width Control 1 Register
<code>PCG_PW2</code>	Precision Clock Pulse Width Control 2 Register
<code>PCG_PW3</code>	Precision Clock Pulse Width Control 3 Register
<code>PCG_PW4</code>	Precision Clock Pulse Width Control 4 Register
<code>PCG_SYNC1</code>	Precision Clock Frame Sync Synchronization 1 Register

Table 27-3: ADSP-2159x PCG Register List (Continued)

Name	Description
PCG_SYNC2	Precision Clock Frame Sync Synchronization 2 Register
PCG_SYNC3	Precision Clock Frame Sync Synchronization 3 Register
PCG_SYNC4	Precision Clock Frame Sync Synchronization 4 Register

Precision Clock A Control 0 Register

The `PCG_CTLA0` register enables the clock, frame sync, and select divisor for the PCG0 clock A signal.

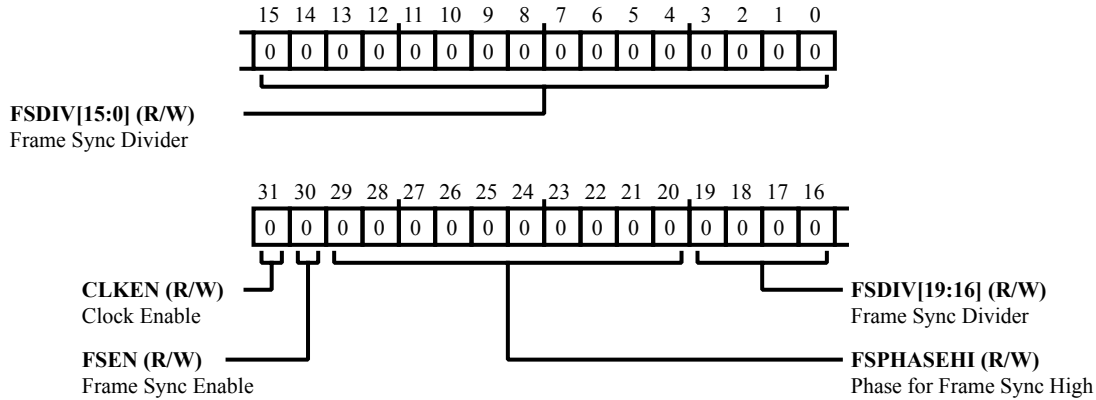


Figure 27-7: PCG_CTLA0 Register Diagram

Table 27-4: PCG_CTLA0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLA0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLA0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLA0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLA0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLA0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLA0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 PCG is in normal mode

Precision Clock A Control 1 Register

The `PCG_CTLA1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock A signal.

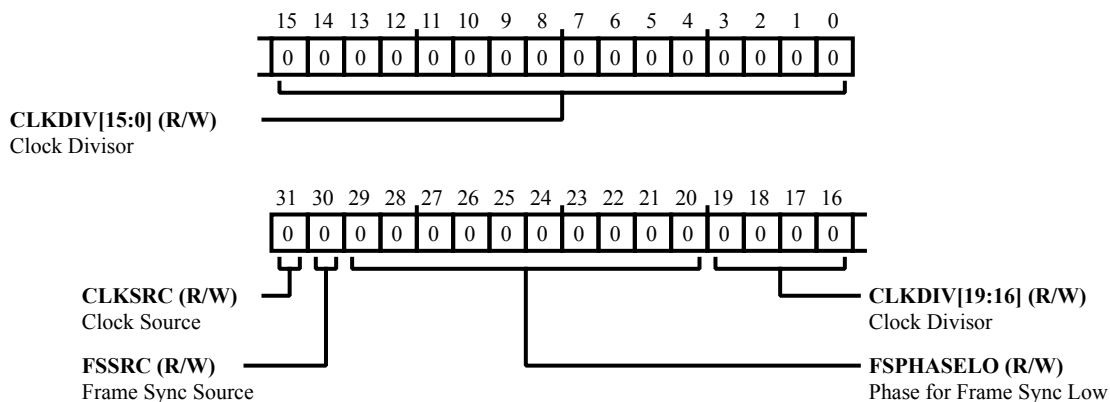


Figure 27-8: PCG_CTLA1 Register Diagram

Table 27-5: PCG_CTLA1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLA1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN0 pin selected for clock
		1 PCG_EXT_DAI0 selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLA1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN0 pin selected for frame sync
		1 PCG_EXTX_DAI0 selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLA1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLA1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock B Control 0 Register

The `PCG_CTLB0` register enables the clock, frame sync, and select divisor for the PCG0 clock B signal.

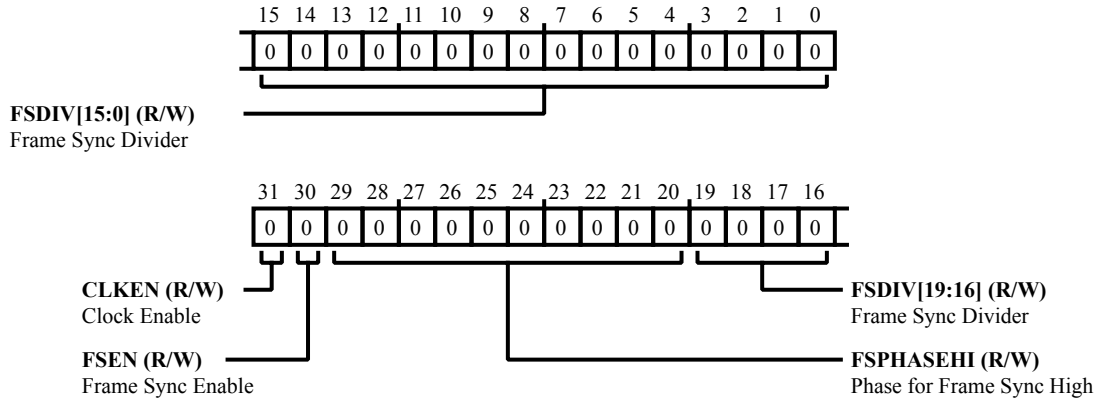


Figure 27-9: PCG_CTLB0 Register Diagram

Table 27-6: PCG_CTLB0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLB0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLB0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLB0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLB0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLB0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLB0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock B Control 1 Register

The `PCG_CTLB1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock B signal.

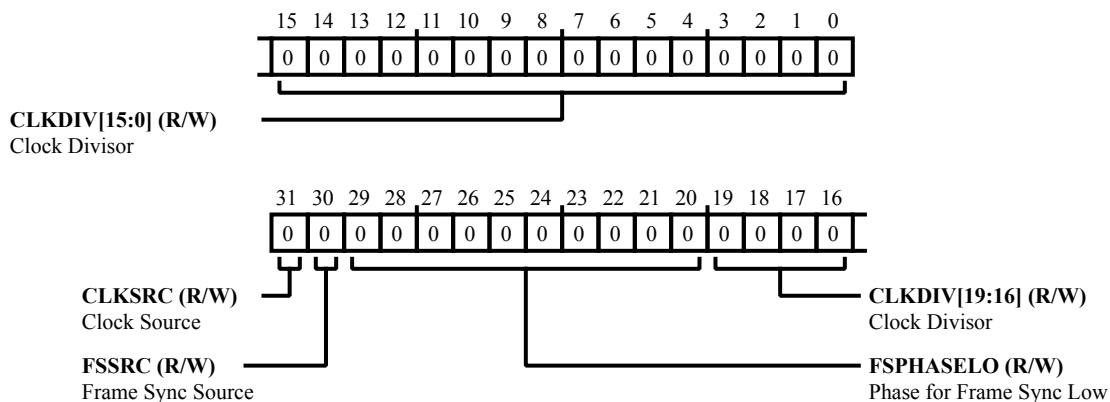


Figure 27-10: PCG_CTLB1 Register Diagram

Table 27-7: PCG_CTLB1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLB1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN0 pin selected for clock
		1 PCG_EXT_DAI0 selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLB1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN0 pin selected for frame sync
		1 PCG_EXT_DAI0 selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLB1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLB1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock C Control 0 Register

The `PCG_CTLC0` register enables the clock, frame sync, and select divisor for the PCG0 clock C signal.

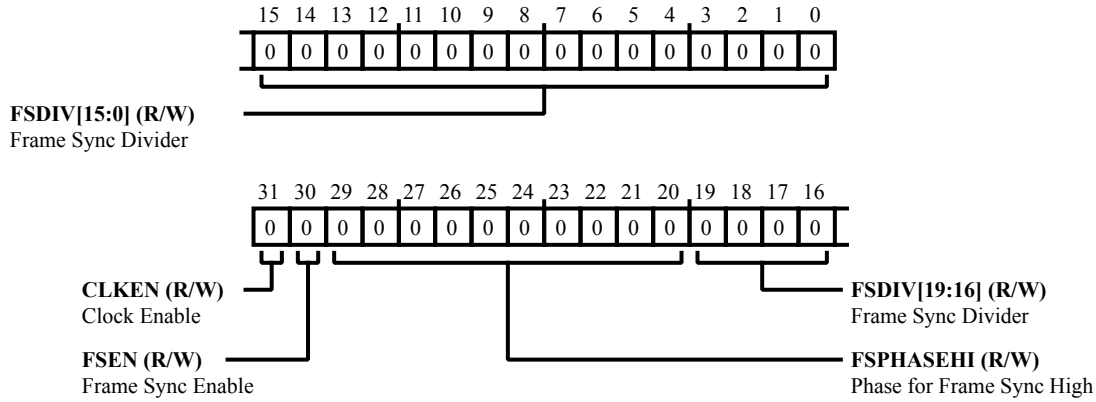


Figure 27-11: `PCG_CTLC0` Register Diagram

Table 27-8: `PCG_CTLC0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLC0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLC0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLC0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLC0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLC0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLC0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock C Control 1 Register

The `PCG_CTLC1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock C signal.

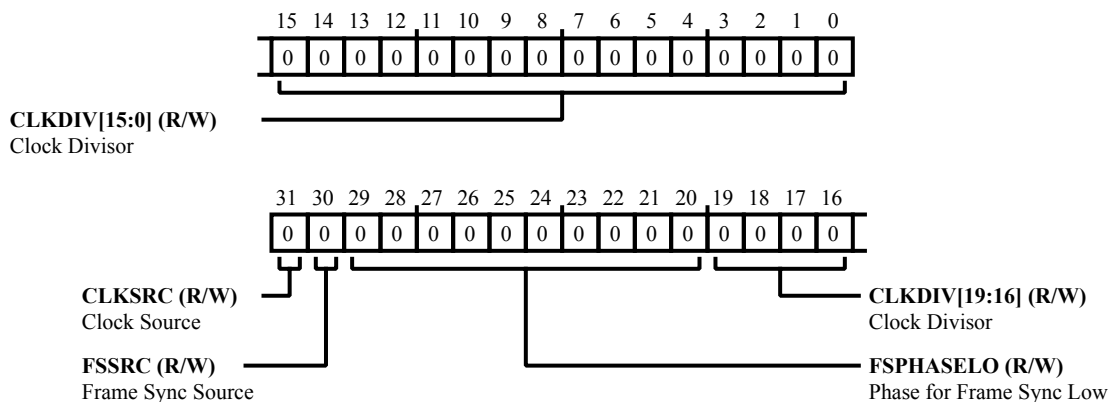


Figure 27-12: `PCG_CTLC1` Register Diagram

Table 27-9: `PCG_CTLC1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLC1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN1 pin selected for clock
		1 PCG_EXT_DAI1 selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLC1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN1 pin selected for frame sync
		1 PCG_EXT_DAI1 selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLC1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLC1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock D Control 0 Register

The `PCG_CTLD0` register enables the clock, frame sync, and select divisor for the PCG0 clock D signal.

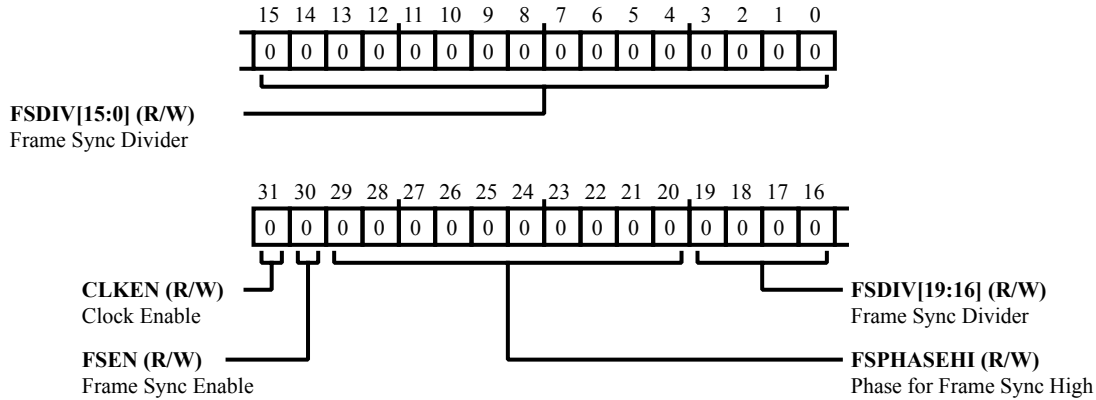


Figure 27-13: PCG_CTLD0 Register Diagram

Table 27-10: PCG_CTLD0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLD0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLD0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLD0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLD0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLD0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLD0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock D Control 1 Register

The `PCG_CTLD1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock D signal.

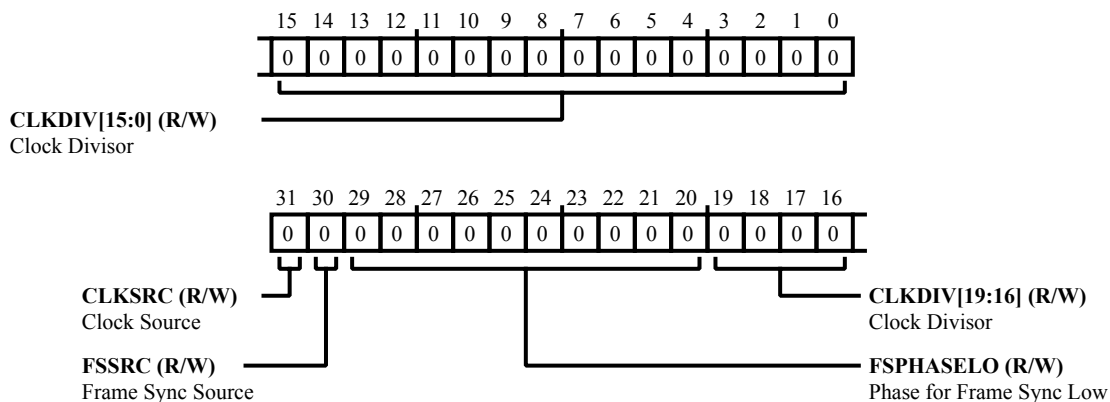


Figure 27-14: PCG_CTLD1 Register Diagram

Table 27-11: PCG_CTLD1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLD1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN1 pin selected for clock
		1 PCG_EXT_DAI1 selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLD1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN1 pin selected for frame sync
		1 PCG_EXT_DAI1 selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLD1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLD1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock E Control 0 Register

The `PCG_CTLE0` register enables the clock, frame sync, and select divisor for the PCG0 clock E signal.

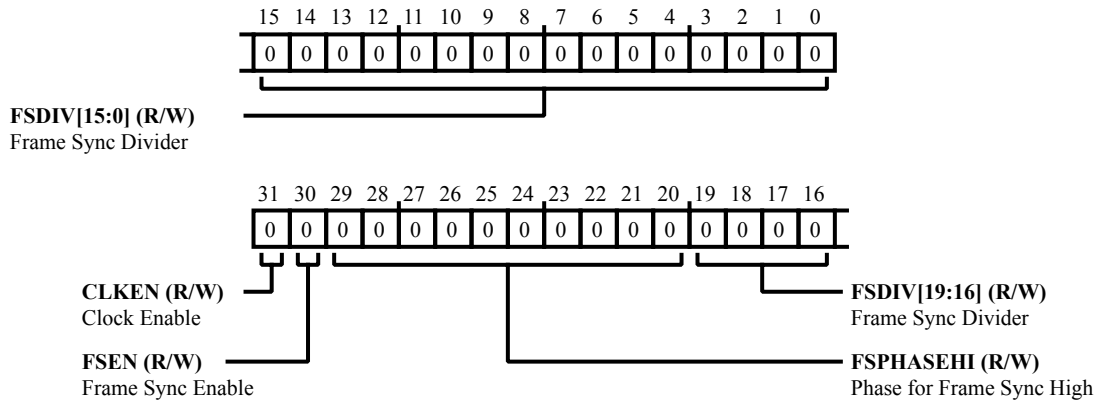


Figure 27-15: PCG_CTLE0 Register Diagram

Table 27-12: PCG_CTLE0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLE0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLE0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLE0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLE0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLE0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLE0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock E Control 1 Register

The `PCG_CTLE1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock E signal.

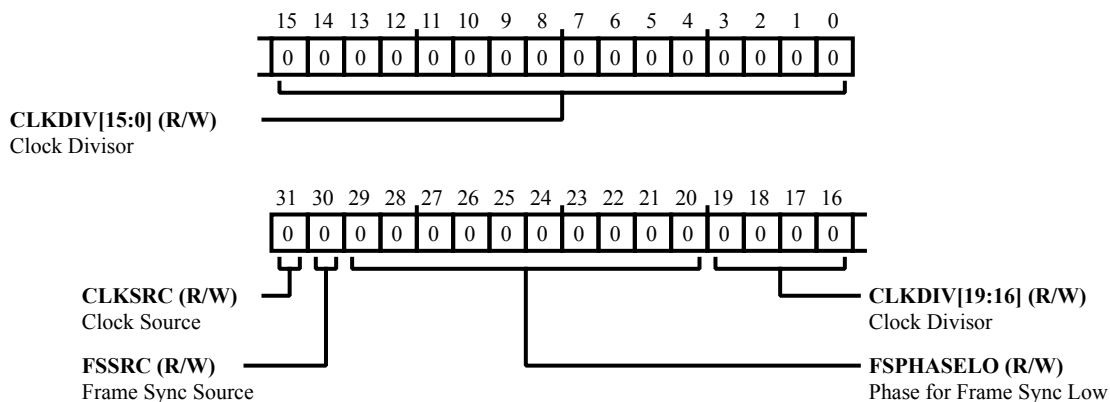


Figure 27-16: `PCG_CTLE1` Register Diagram

Table 27-13: `PCG_CTLE1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLE1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN0 pin selected for clock
		1 PCG_EXT_DAI selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLE1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN0 pin selected for frame sync
		1 PCG_EXT_DAI selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLE1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLE1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock F Control 0 Register

The `PCG_CTLF0` register enables the clock, frame sync, and select divisor for the PCG0 clock F signal.

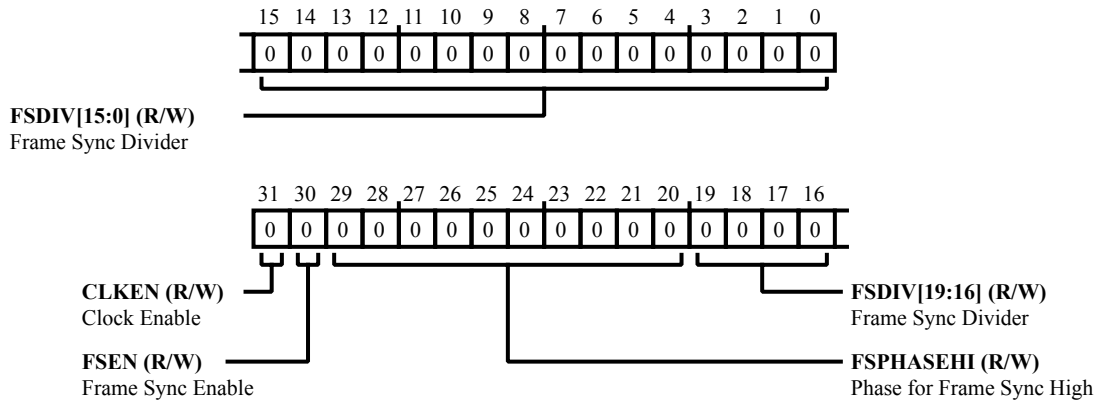


Figure 27-17: PCG_CTLF0 Register Diagram

Table 27-14: PCG_CTLF0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLF0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLF0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLF0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLF0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLF0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLF0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock F Control 1 Register

The `PCG_CTLF1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock F signal.

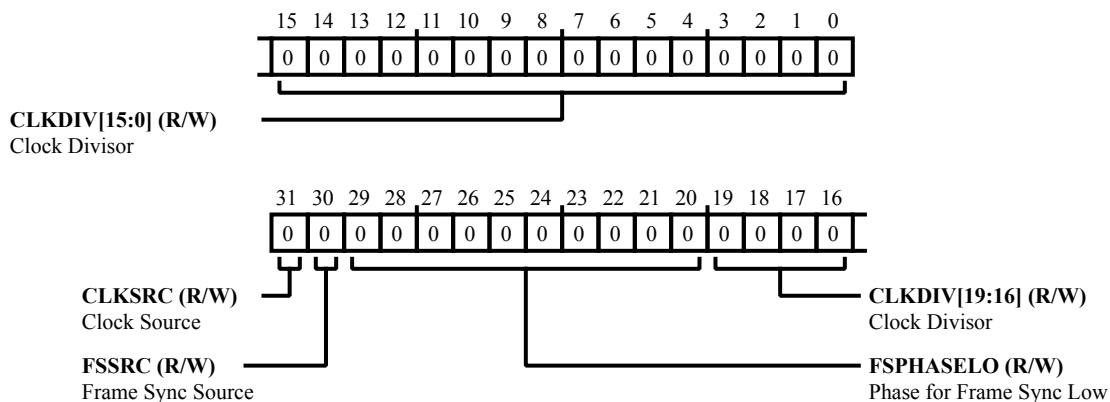


Figure 27-18: `PCG_CTLF1` Register Diagram

Table 27-15: `PCG_CTLF1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLF1 . CLKSRC</code> bit specifies the clock source.
		0 CLKIN0 pin selected for clock
		1 PCG_EXT_DAI0 selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLF1 . FSSRC</code> bit specifies the frame sync source.
		0 CLKIN0 pin selected for frame sync
		1 PCG_EXT_DAI0 selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLF1 . FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLF1 . CLKDIV</code> bit field contains the clock divisor value.

Precision Clock G Control 0 Register

The `PCG_CTLG0` register enables the clock, frame sync, and select divisor for the PCG0 clock G signal.

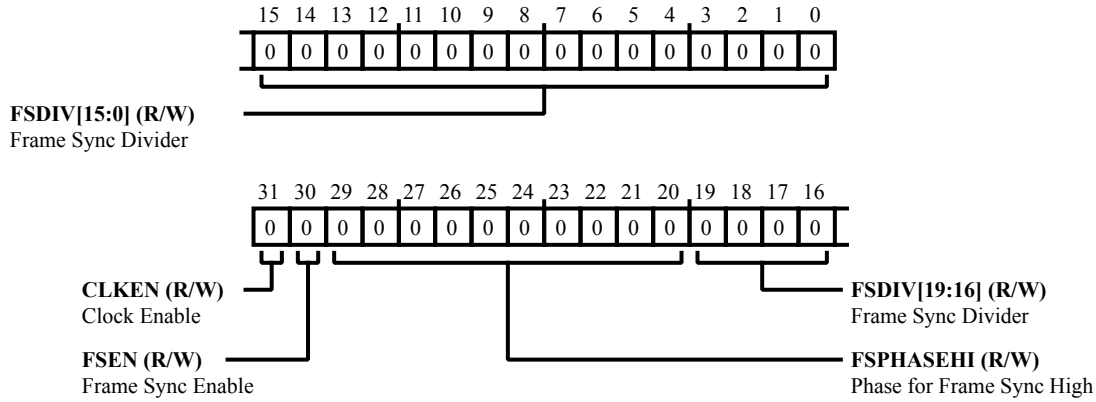


Figure 27-19: `PCG_CTLG0` Register Diagram

Table 27-16: `PCG_CTLG0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLG0 . CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLG0 . FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLG0 . FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLG0 . FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLG0 . FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLG0 . FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock G Control 1 Register

The `PCG_CTLG1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock G signal.

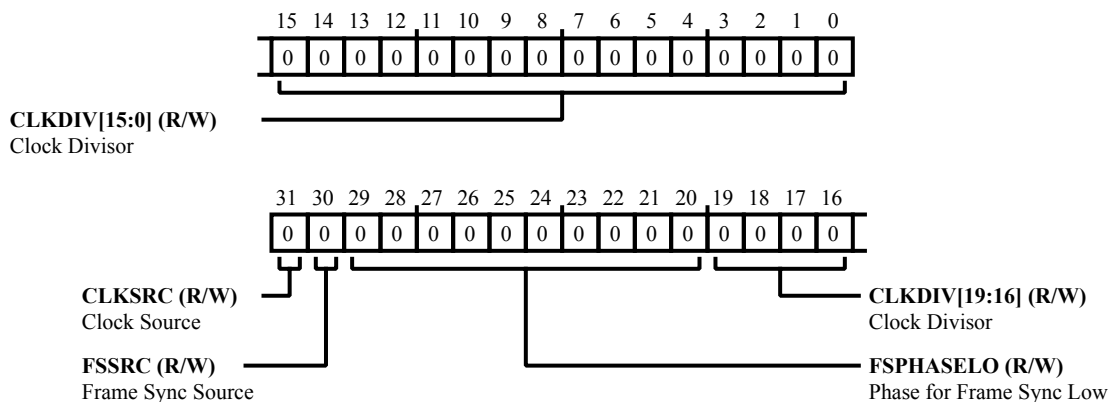


Figure 27-20: `PCG_CTLG1` Register Diagram

Table 27-17: `PCG_CTLG1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLG1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN0 pin selected for clock
		1 PCG_EXT_DAI selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLG1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN0 pin selected for frame sync
		1 PCG_EXT_DAI selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLG1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLG1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock H Control 0 Register

The `PCG_CTLH0` register enables the clock, frame sync, and select divisor for the PCG0 clock H signal.

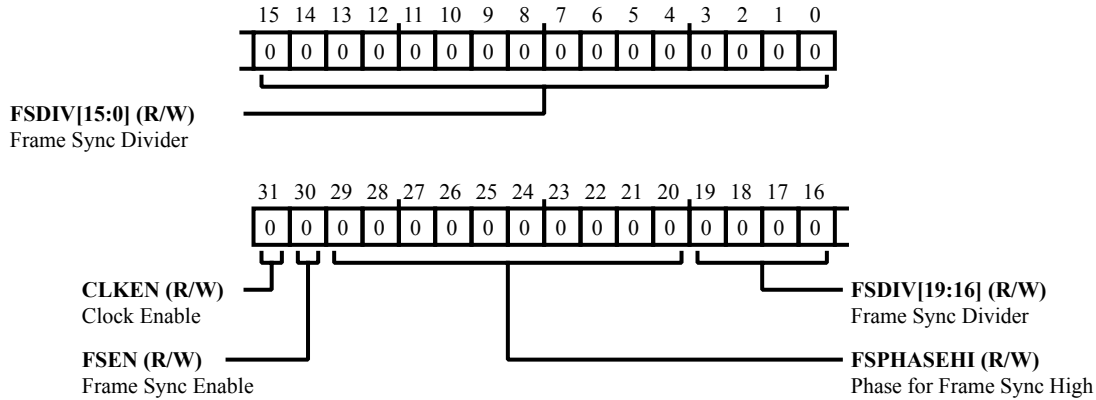


Figure 27-21: PCG_CTLH0 Register Diagram

Table 27-18: PCG_CTLH0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKEN	Clock Enable. The <code>PCG_CTLH0.CLKEN</code> bit enables the clock.
		0 Clock generation disabled
		1 Clock generation enabled
30 (R/W)	FSEN	Frame Sync Enable. The <code>PCG_CTLH0.FSEN</code> bit enables the frame sync.
		0 Frame sync generation disabled
		1 Frame sync generation enabled
29:20 (R/W)	FSPHASEHI	Phase for Frame Sync High. The <code>PCG_CTLH0.FSPHASEHI</code> bit field represents the upper half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	FSDIV	Frame Sync Divider. The <code>PCG_CTLH0.FSDIV</code> bit field provides the frame sync divider value. This 20-bit field frame sync divider is multiplexed where: <code>PCG_CTLH0.FSDIV > 1</code> PCGx is in normal mode, <code>PCG_CTLH0.FSDIV = 0, 1</code> PCGx is in bypass mode.
		0 PCG is in bypass mode
		1 PCG is in bypass mode
		2-1048575 FSDIV > 1 PCG is in normal mode

Precision Clock H Control 1 Register

The `PCG_CTLH1` register sets the clock divisor, frame sync source, and clock source for the PCG1 clock H signal.

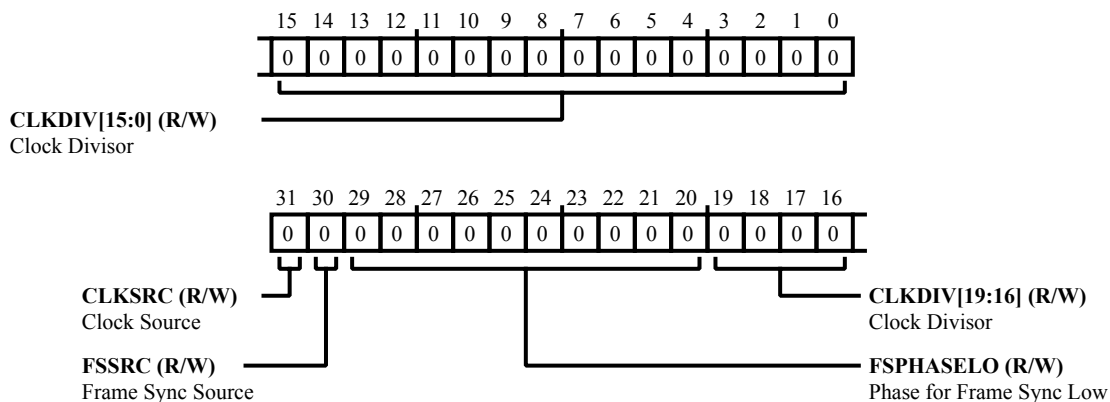


Figure 27-22: PCG_CTLH1 Register Diagram

Table 27-19: PCG_CTLH1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CLKSRC	Clock Source. The <code>PCG_CTLH1</code> . <code>CLKSRC</code> bit specifies the clock source.
		0 CLKIN0 pin selected for clock
		1 PCG_EXT_DAI selected for clock
30 (R/W)	FSSRC	Frame Sync Source. The <code>PCG_CTLH1</code> . <code>FSSRC</code> bit specifies the frame sync source.
		0 CLKIN0 pin selected for frame sync
		1 PCG_EXT_DAI selected for frame sync
29:20 (R/W)	FSPHASELO	Phase for Frame Sync Low. The <code>PCG_CTLH1</code> . <code>FSPHASELO</code> bit field represents the lower half of the 20-bit value for the channel A/B/C/D frame sync phase.
19:0 (R/W)	CLKDIV	Clock Divisor. The <code>PCG_CTLH1</code> . <code>CLKDIV</code> bit field contains the clock divisor value.

Precision Clock Pulse Width Control 1 Register

The `PCG_PW1` register sets the one shot frame sync and the active low frame sync select for PCG A and PCG B.

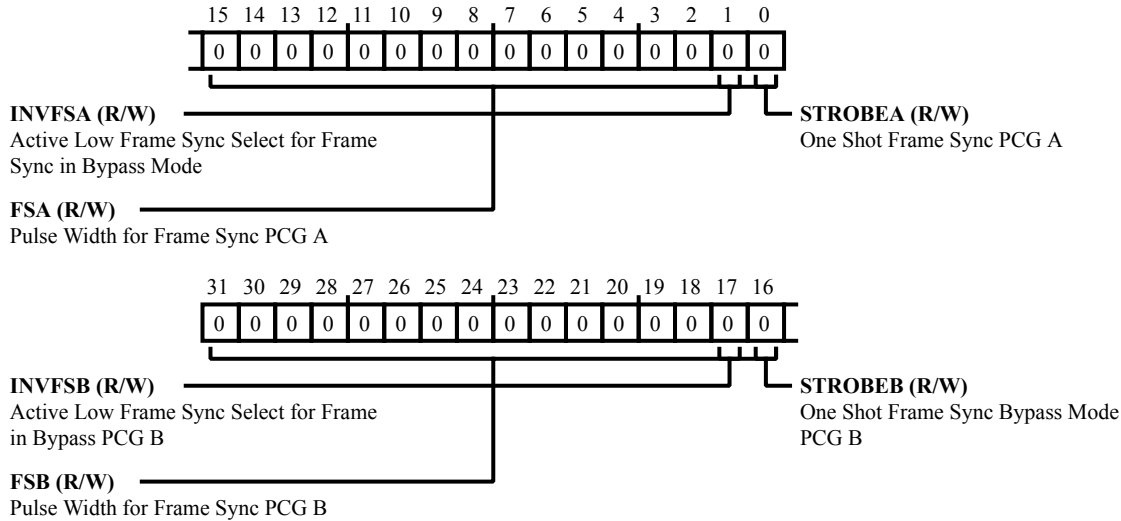


Figure 27-23: `PCG_PW1` Register Diagram

Table 27-20: `PCG_PW1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	INVFSB	Active Low Frame Sync Select for Frame in Bypass PCG B. The <code>PCG_PW1</code> . INVFSB bit selects active low or active high frame sync in bypass mode for PCG B.
16 (R/W)	STROBEB	One Shot Frame Sync Bypass Mode PCG B. The <code>PCG_PW1</code> . STROBEB bit sets the frame sync pulse in bypass mode for PCG B. This is the duration equal to one period of the DAI_MISCA2_I signal (PCG B) repeating at the beginning of every frame.
31:16 (R/W)	FSB	Pulse Width for Frame Sync PCG B. The <code>PCG_PW1</code> . FSB bit field sets the number of input clock periods for which the frame sync output is high. Pulse width should be less than the divisor of the frame sync.
1 (R/W)	INVFSB	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW1</code> . INVFSB bit selects active low or active high frame sync for PCG A in bypass mode.
15:0 (R/W)	FSA	Pulse Width for Frame Sync PCG A. The <code>PCG_PW1</code> . FSA bit field sets the number of input clock periods for which the frame sync output is high. Pulse width should be less than the divisor of the frame sync.

Table 27-20: PCG_PW1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	STROBEA	One Shot Frame Sync PCG A. The PCG_PW1 . STROBEA bit sets the frame sync pulse for PCG A in bypass mode. This is the duration equal to one period of the DAI_MISCA2_I signal (PCG A) repeating at the beginning of every frame.

Precision Clock Pulse Width Control 2 Register

The `PCG_PW2` register sets the one shot frame sync and the active low frame sync select for PCG C and PCG D.

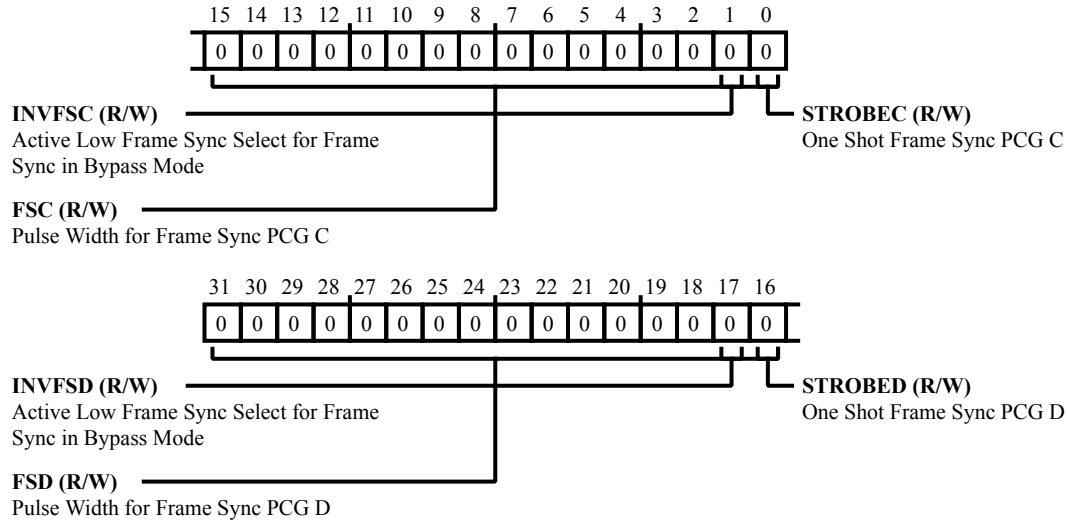


Figure 27-24: `PCG_PW2` Register Diagram

Table 27-21: `PCG_PW2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	INVFS	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW2</code> . <code>INVFS</code> bit selects active low or active high frame sync for PCG D in bypass mode.
16 (R/W)	STROBED	One Shot Frame Sync PCG D. The <code>PCG_PW2</code> . <code>STROBED</code> bit sets the frame sync pulse for PCG D in bypass mode. This is the duration equal to one period of the <code>DAI_MISCA4_I</code> signal (PCG D) repeating at the beginning of every frame.
31:16 (R/W)	FSD	Pulse Width for Frame Sync PCG D. The <code>PCG_PW2</code> . <code>FSD</code> bit field sets the number of input clock periods for which the frame sync output is high for PCG D. Pulse width should be less than the divisor of the frame sync.
1 (R/W)	INVFS	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW2</code> . <code>INVFS</code> bit selects active low or active high frame sync for PCG C in bypass mode.
15:0 (R/W)	FSC	Pulse Width for Frame Sync PCG C. The <code>PCG_PW2</code> . <code>FSC</code> bit field sets the number of input clock periods for which the frame sync output is high for PCG C. Pulse width should be less than the divisor of the frame sync.

Table 27-21: PCG_PW2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	STROBEC	One Shot Frame Sync PCG C. The PCG_PW2 . STROBEC bit sets the frame sync pulse for PCG C in bypass mode. This is the duration equal to one period of the DAI_MISCA3_I signal (PCG C) repeating at the beginning of every frame.

Precision Clock Pulse Width Control 3 Register

The `PCG_PW3` register sets the one shot frame sync and the active low frame sync select for PCG E and PCG F.

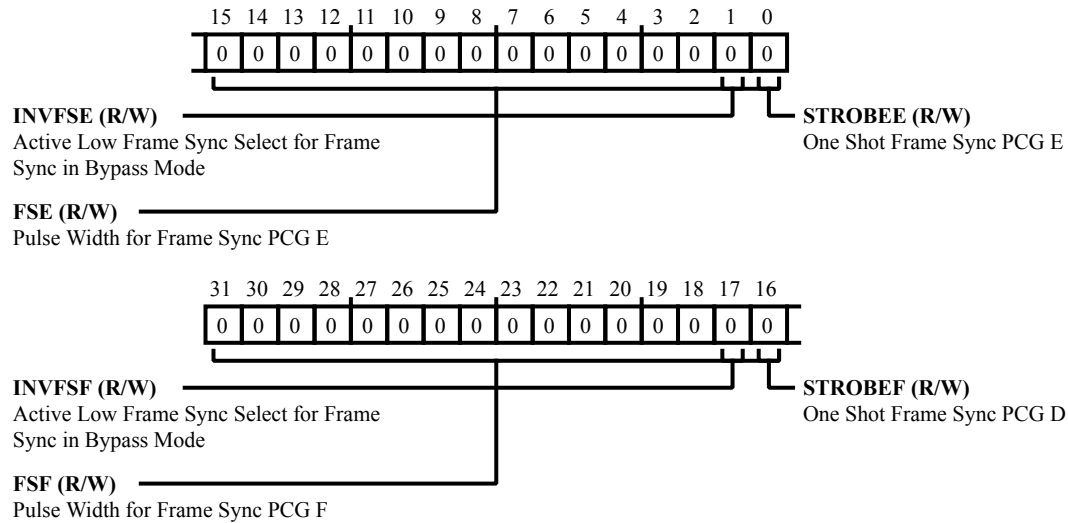


Figure 27-25: PCG_PW3 Register Diagram

Table 27-22: PCG_PW3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	INVFSF	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW3</code> . <code>INVFSF</code> bit selects active low or active high frame sync for PCG F in bypass mode.
16 (R/W)	STROBEF	One Shot Frame Sync PCG D. The <code>PCG_PW3</code> . <code>STROBEF</code> bit sets the frame sync pulse for PCG F in bypass mode. This is the duration equal to one period of the <code>DAI_MISCA4_I</code> signal (PCG F) repeating at the beginning of every frame.
31:16 (R/W)	FSF	Pulse Width for Frame Sync PCG F. The <code>PCG_PW3</code> . <code>FSF</code> bit field sets the number of input clock periods for which the frame sync output is high for PCG F. Pulse width should be less than the divisor of the frame sync.
1 (R/W)	INVSE	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW3</code> . <code>INVSE</code> bit selects active low or active high frame sync for PCG E in bypass mode.
0 (R/W)	STROBEE	One Shot Frame Sync PCG E. The <code>PCG_PW3</code> . <code>STROBEE</code> bit sets the frame sync pulse for PCG E in bypass mode. This is the duration equal to one period of the <code>DAI_MISCA3_I</code> signal (PCG E) repeating at the beginning of every frame.

Table 27-22: PCG_PW3 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	FSE	Pulse Width for Frame Sync PCG E. The <code>PCG_PW3.FSE</code> bit field sets the number of input clock periods for which the frame sync output is high for PCG E. Pulse width should be less than the divisor of the frame sync.

Precision Clock Pulse Width Control 4 Register

The `PCG_PW4` register sets the one shot frame sync and the active low frame sync select for PCG G and PCG H.

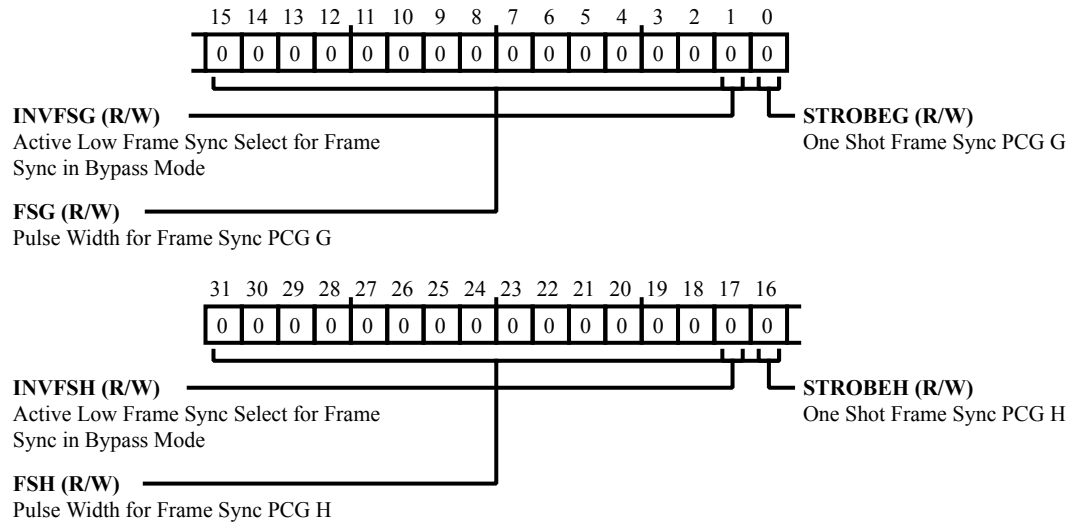


Figure 27-26: `PCG_PW4` Register Diagram

Table 27-23: `PCG_PW4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	INVFSH	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW4</code> . <code>INVFSH</code> bit selects active low or active high frame sync for PCG H in bypass mode.
16 (R/W)	STROBEH	One Shot Frame Sync PCG H. The <code>PCG_PW4</code> . <code>STROBEH</code> bit sets the frame sync pulse for PCG H in bypass mode. This is the duration equal to one period of the <code>DAI_MISCA4_I</code> signal (PCG H) repeating at the beginning of every frame.
31:16 (R/W)	FSH	Pulse Width for Frame Sync PCG H. The <code>PCG_PW4</code> . <code>FSH</code> bit field sets the number of input clock periods for which the frame sync output is high for PCG H. Pulse width should be less than the divisor of the frame sync.
1 (R/W)	INVFSG	Active Low Frame Sync Select for Frame Sync in Bypass Mode. The <code>PCG_PW4</code> . <code>INVFSG</code> bit selects active low or active high frame sync for PCG G in bypass mode.
15:0 (R/W)	FSG	Pulse Width for Frame Sync PCG G. The <code>PCG_PW4</code> . <code>FSG</code> bit field sets the number of input clock periods for which the frame sync output is high for PCG G. Pulse width should be less than the divisor of the frame sync.

Table 27-23: PCG_PW4 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	STROBEG	One Shot Frame Sync PCG G. The PCG_PW4 . STROBEG bit sets the frame sync pulse for PCG G in bypass mode. This is the duration equal to one period of the DAI_MISCA3_I signal (PCG G) repeating at the beginning of every frame.

Precision Clock Frame Sync Synchronization 1 Register

The `PCG_SYNC1` register allows programs to synchronize the clock frame sync units with external frame syncs. Note that the `PCG_CTLA1.CLKSRC` bit is overridden if `PCG_SYNC1.CLKASRC` bit in the `PCG_SYNC1` register is set.

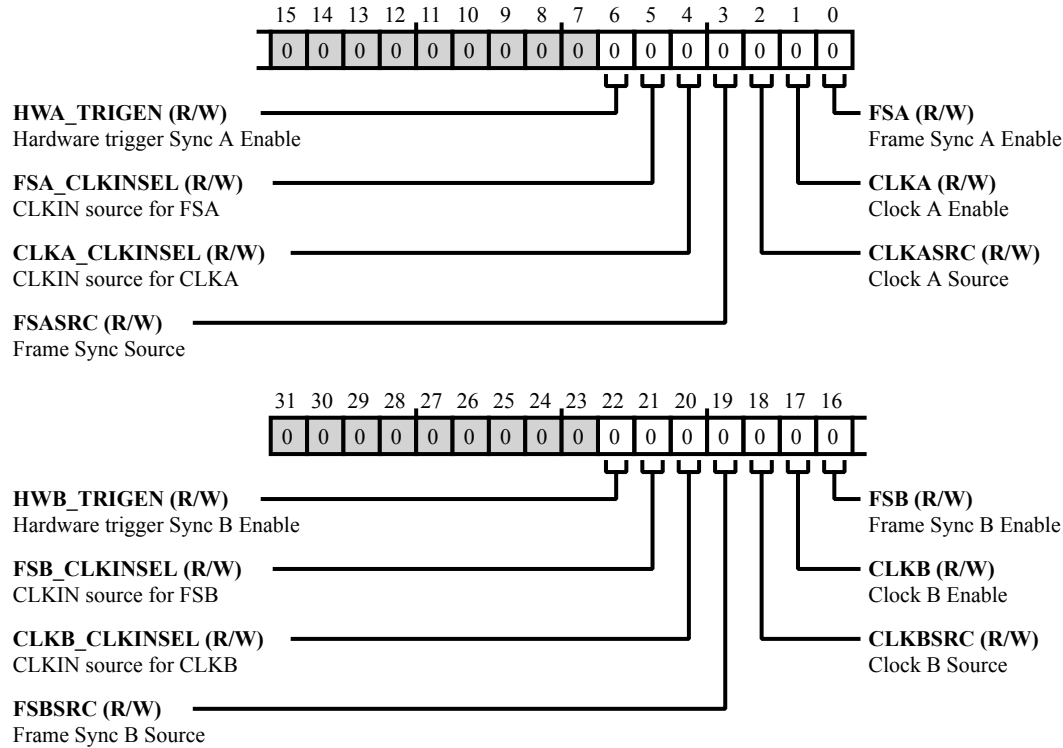


Figure 27-27: PCG_SYNC1 Register Diagram

Table 27-24: PCG_SYNC1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	HWB_TRIGEN	Hardware trigger Sync B Enable. The <code>PCG_SYNC1.HWB_TRIGEN</code> bit enables synchronization of clock B and frame sync B with the external frame sync only after an internal slave trigger for PCG B is received.
21 (R/W)	FSB_CLKINSEL	CLKIN source for FSB. The <code>PCG_SYNC1.FSB_CLKINSEL</code> bit enables the CLKIN1 input source for FSB
		0 Selects CLKIN0 source for FSB CLKIN
		1 Selects CLKIN1 source for FSB CLKIN

Table 27-24: PCG_SYNC1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	CLKB_CLKINSEL	CLKIN source for CLKB. The PCG_SYNC1 . CLKB_CLKINSEL bit enables the CLKIN1 input source for CLKB
		0 Select CLKIN0 source for CLKB CLKIN
		1 Select CLKIN1 source for CLKB CLKIN
19 (R/W)	FSBSRC	Frame Sync B Source. The PCG_SYNC1 . FSBSRC bit enables the frame sync B input source.
		0 Output selected by FSBSOURCE bit
		1 Clock derived from core PLL selected for frame sync B
18 (R/W)	CLKBSRC	Clock B Source. The PCG_SYNC1 . CLKBSRC bit enables the clock B input source.
		0 Output selected by CLKBSOURCE bit
		1 Clock derived from core PLL selected for clock B
17 (R/W)	CLKB	Clock B Enable. The PCG_SYNC1 . CLKB bit enables synchronization of clock B with the external frame sync.
		0 Clock disabled
		1 Clock enabled
16 (R/W)	FSB	Frame Sync B Enable. The PCG_SYNC1 . FSB bit enables synchronization of frame sync B with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled
6 (R/W)	HWA_TRIGEN	Hardware trigger Sync A Enable. The PCG_SYNC1 . HWA_TRIGEN bit enables synchronization of clock A and frame sync A with the external frame sync only after an internal slave trigger for PCG A is received.
5 (R/W)	FSA_CLKINSEL	CLKIN source for FSA. The PCG_SYNC1 . FSA_CLKINSEL bit enables the CLKIN1 input source for FSA
		0 Selects CLKIN0 as source for FSA CLKIN
		1 Selects CLKIN1 as source for FSA CLKIN

Table 27-24: PCG_SYNC1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	CLKA_CLKINSEL	CLKIN source for CLKA. The PCG_SYNC1 . CLKA_CLKINSEL bit enables the CLKIN1 input source for CLKA
		0 Select CLKIN0 source for CLKA CLKIN
		1 Select CLKIN1 source for CLKA CLKIN
3 (R/W)	FSASRC	Frame Sync Source. The PCG_SYNC1 . FSASRC bit enables the frame sync A input source.
		0 Output selected by FSASOURCE bit
		1 Clock derived from core PLL selected for frame sync A
2 (R/W)	CLKASRC	Clock A Source. The PCG_SYNC1 . CLKASRC bit enables the clock A input source.
		0 Output selected by CLKASOURCE bit
		1 Clock derived from core PLL selected for clock A
1 (R/W)	CLKA	Clock A Enable. The PCG_SYNC1 . CLKA bit enables synchronization of clock A with the external frame sync.
		0 Clock disabled
		1 Clock enabled
0 (R/W)	FSA	Frame Sync A Enable. The PCG_SYNC1 . FSA bit enables synchronization of frame sync A with the external frame sync.
		0 Frame Sync Disabled
		1 Frame Sync Enabled

Precision Clock Frame Sync Synchronization 2 Register

The `PCG_SYNC2` register allows programs to synchronize the clock frame sync units with external frame syncs. Note that the `PCG_CTLD1.CLKSRC` bit is overridden if `PCG_SYNC2.CLKDSRC` bit in the `PCG_SYNC2` register is set.

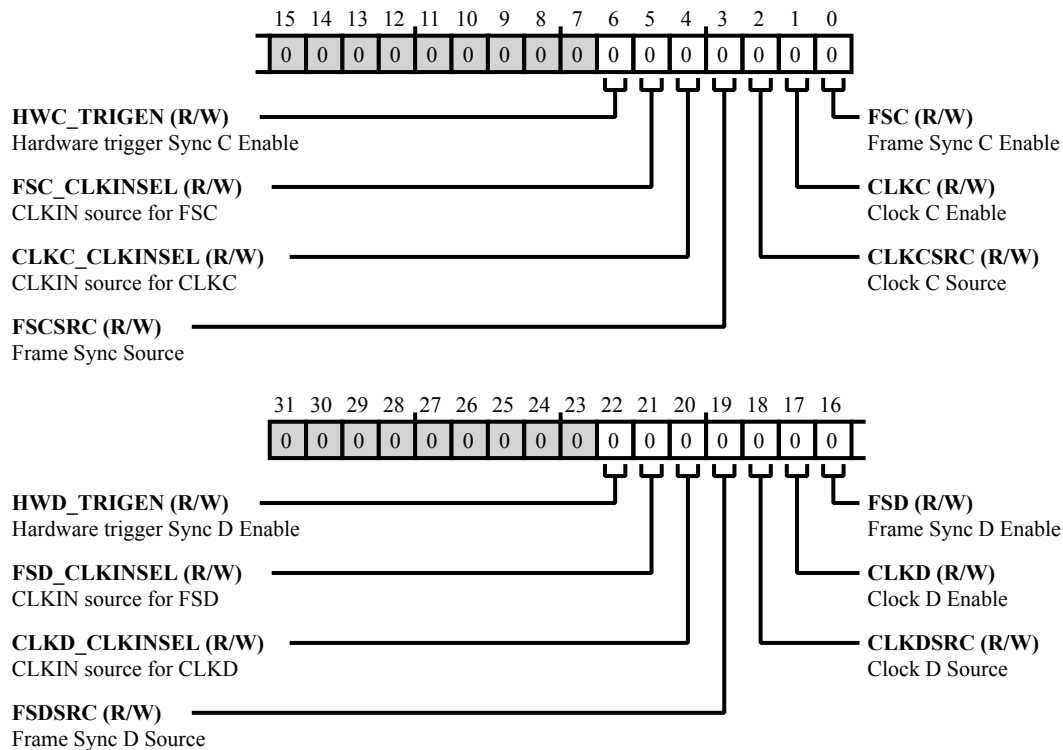


Figure 27-28: `PCG_SYNC2` Register Diagram

Table 27-25: `PCG_SYNC2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	<code>HWD_TRIGEN</code>	Hardware trigger Sync D Enable. The <code>PCG_SYNC2.HWD_TRIGEN</code> bit enables synchronization of clock D and frame sync D with the external frame sync only after an internal slave trigger for PCG D is received.
21 (R/W)	<code>FSD_CLKINSEL</code>	CLKIN source for FSD. The <code>PCG_SYNC2.FSD_CLKINSEL</code> bit enables the CLKIN1 input source for FSD
		0 Selects CLKIN0 source for FSB CLKIN
		1 Selects CLKIN1 source for FSB CLKIN

Table 27-25: PCG_SYNC2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	CLKD_CLKINSEL	CLKIN source for CLKD. The PCG_SYNC2.CLKD_CLKINSEL bit enables the CLKIN1 input source for CLKD
		0 Select CLKIN0 source for CLKB CLKIN
		1 Select CLKIN1 source for CLKB CLKIN
19 (R/W)	FSDSRC	Frame Sync D Source. The PCG_SYNC2.FSDSRC bit enables the frame sync D input source.
		0 Output selected by FSDSOURCE bit
		1 Clock derived from core PLL selected for frame sync D
18 (R/W)	CLKDSRC	Clock D Source. The PCG_SYNC2.CLKDSRC bit enables the clock D input source.
		0 Output selected by CLKDSOURCE bit
		1 Clock derived from core PLL selected for clock D
17 (R/W)	CLKD	Clock D Enable. The PCG_SYNC2.CLKD bit enables synchronization of clock D with the external frame sync.
		0 Clock disabled
		1 Clock enabled
16 (R/W)	FSD	Frame Sync D Enable. The PCG_SYNC2.FSD bit enables synchronization of frame sync D with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled
6 (R/W)	HWC_TRIGEN	Hardware trigger Sync C Enable. The PCG_SYNC2.HWC_TRIGEN bit enables synchronization of clock C and frame sync C with the external frame sync only after an internal slave trigger for PCG C is received.
5 (R/W)	FSC_CLKINSEL	CLKIN source for FSC. The PCG_SYNC2.FSC_CLKINSEL bit enables the CLKIN1 input source for FSC
		0 Selects CLKIN0 as source for FSA CLKIN
		1 Selects CLKIN1 as source for FSA CLKIN

Table 27-25: PCG_SYNC2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	CLKC_CLKINSEL	CLKIN source for CLKC. The PCG_SYNC2.CLKC_CLKINSEL bit enables the CLKIN1 input source for CLKC
		0 Select CLKIN0 source for CLKA CLKIN
		1 Select CLKIN1 source for CLKA CLKIN
3 (R/W)	FSCSRC	Frame Sync Source. The PCG_SYNC2.FSCSRC bit enables the frame sync C input source.
		0 Output selected by FSCSOURCE bit
		1 Clock derived from core PLL selected for frame sync C
2 (R/W)	CLKCSRC	Clock C Source. The PCG_SYNC2.CLKCSRC bit enables the clock C input source.
		0 Output selected by CLKCSOURCE bit
		1 Clock derived from core PLL selected for clock C
1 (R/W)	CLKC	Clock C Enable. The PCG_SYNC2.CLKC bit enables synchronization of clock C with the external frame sync.
		0 Clock disabled
		1 Clock enabled
0 (R/W)	FSC	Frame Sync C Enable. The PCG_SYNC2.FSC bit enables synchronization of frame sync C with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled

Precision Clock Frame Sync Synchronization 3 Register

The `PCG_SYNC3` register allows programs to synchronize the clock frame sync units with external frame syncs. Note that the `PCG_CTLD1.CLKSRC` bit is overridden if `CLKDSRC` bit in the `PCG_SYNC3` register is set.

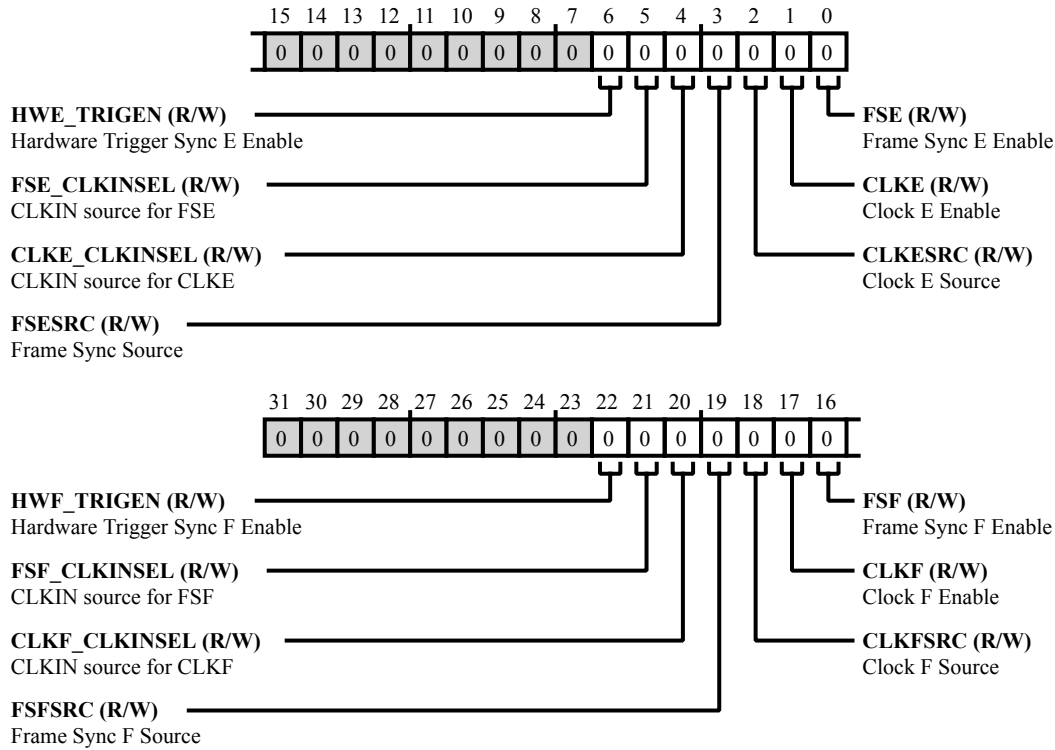


Figure 27-29: `PCG_SYNC3` Register Diagram

Table 27-26: `PCG_SYNC3` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	<code>HWF_TRIGEN</code>	Hardware Trigger Sync F Enable. The <code>PCG_SYNC3.HWF_TRIGEN</code> bit enables synchronization of clock F and frame sync F with the external frame sync only after an internal slave trigger for PCG-F is received.
21 (R/W)	<code>FSF_CLKINSEL</code>	CLKIN source for FSF. The <code>PCG_SYNC3.FSF_CLKINSEL</code> bit enables the CLKIN1 input source for FSF
	0	Selects CLKIN0 as source for FSA CLKIN
	1	Selects CLKIN1 as source for FSA CLKIN

Table 27-26: PCG_SYNC3 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	CLKF_CLKINSEL	CLKIN source for CLKF. The PCG_SYNC3.CLKF_CLKINSEL bit enables the CLKIN1 input source for CLKF
		0 Select CLKIN0 source for CLKA CLKIN
		1 Select CLKIN1 source for CLKA CLKIN
19 (R/W)	FSFSRC	Frame Sync F Source. The PCG_SYNC3.FSFSRC bit enables the frame sync F input source.
		0 Output selected by FSDSOURCE bit
		1 Clock derived from core PLL selected for frame sync D
18 (R/W)	CLKFSRC	Clock F Source. The PCG_SYNC3.CLKFSRC bit enables the clock F input source.
		0 Output selected by CLKDSOURCE bit
		1 Clock derived from core PLL selected for clock D
17 (R/W)	CLKF	Clock F Enable. The PCG_SYNC3.CLKF bit enables synchronization of clock F with the external frame sync.
		0 Clock disabled
		1 Clock enabled
16 (R/W)	FSF	Frame Sync F Enable. The PCG_SYNC3.FSF bit enables synchronization of frame sync F with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled
6 (R/W)	HWE_TRIGEN	Hardware Trigger Sync E Enable. The PCG_SYNC3.HWE_TRIGEN bit enables synchronization of clock E and frame sync E with the external frame sync only after an internal slave trigger for PCG-E is received.
5 (R/W)	FSE_CLKINSEL	CLKIN source for FSE. The PCG_SYNC3.FSE_CLKINSEL bit enables the CLKIN1 input source for FSE
		0 Selects CLKIN0 as source for FSA CLKIN
		1 Selects CLKIN1 as source for FSA CLKIN

Table 27-26: PCG_SYNC3 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	CLKE_CLKINSEL	CLKIN source for CLKE. The PCG_SYNC3.CLKE_CLKINSEL bit enables the CLKIN1 input source for CLKE
		0 Select CLKIN0 source for CLKA CLKIN
		1 Select CLKIN1 source for CLKA CLKIN
3 (R/W)	FSESRC	Frame Sync Source. The PCG_SYNC3.FSESRC bit enables the frame sync E input source.
		0 Output selected by FSCSOURCE bit
		1 Clock derived from core PLL selected for frame sync C
2 (R/W)	CLKESRC	Clock E Source. The PCG_SYNC3.CLKESRC bit enables the clock E input source.
		0 Output selected by CLKCSOURCE bit
		1 Clock derived from core PLL selected for clock C
1 (R/W)	CLKE	Clock E Enable. The PCG_SYNC3.CLKE bit enables synchronization of clock E with the external frame sync.
		0 Clock disabled
		1 Clock enabled
0 (R/W)	FSE	Frame Sync E Enable. The PCG_SYNC3.FSE bit enables synchronization of frame sync E with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled

Precision Clock Frame Sync Synchronization 4 Register

The `PCG_SYNC4` register allows programs to synchronize the clock frame sync units with external frame syncs. Note that the `PCG_CTLD1.CLKSRC` bit is overridden if `CLKDSRC` bit in the `PCG_SYNC4` register is set.

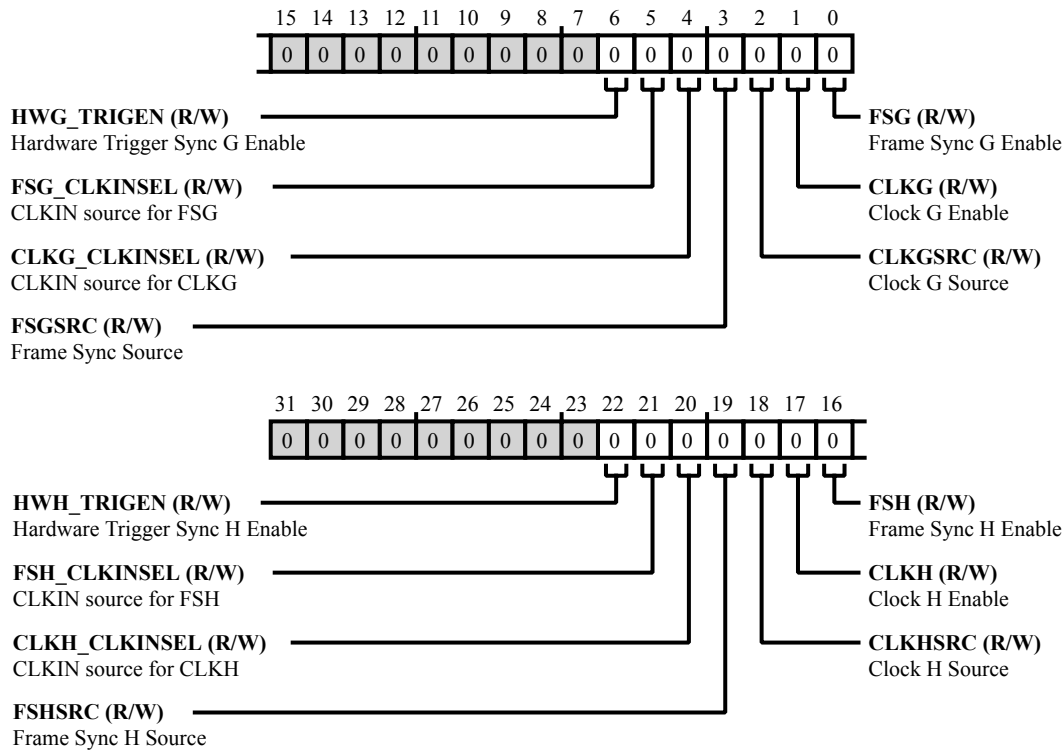


Figure 27-30: `PCG_SYNC4` Register Diagram

Table 27-27: `PCG_SYNC4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	<code>HWH_TRIGEN</code>	Hardware Trigger Sync H Enable. The <code>PCG_SYNC4.HWH_TRIGEN</code> bit enables synchronization of clock H and frame sync H with the external frame sync only after an internal slave trigger for PCG-H is received.
21 (R/W)	<code>FSH_CLKINSEL</code>	CLKIN source for FSH. The <code>PCG_SYNC4.FSH_CLKINSEL</code> bit enables the CLKIN1 input source for FSH
	0	Selects CLKIN0 as source for FSA CLKIN
	1	Selects CLKIN1 as source for FSA CLKIN

Table 27-27: PCG_SYNC4 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	CLKH_CLKINSEL	CLKIN source for CLKH. The PCG_SYNC4.CLKH_CLKINSEL bit enables the CLKIN1 input source for CLKH
		0 Select CLKIN0 source for CLKA CLKIN
		1 Select CLKIN1 source for CLKA CLKIN
19 (R/W)	FSHSRC	Frame Sync H Source. The PCG_SYNC4.FSHSRC bit enables the frame sync H input source.
		0 Output selected by FSDSOURCE bit
		1 Clock derived from core PLL selected for frame sync D
18 (R/W)	CLKHSRC	Clock H Source. The PCG_SYNC4.CLKHSRC bit enables the clock H input source.
		0 Output selected by CLKDSOURCE bit
		1 Clock derived from core PLL selected for clock D
17 (R/W)	CLKH	Clock H Enable. The PCG_SYNC4.CLKH bit enables synchronization of clock H with the external frame sync.
		0 Clock disabled
		1 Clock enabled
16 (R/W)	FSH	Frame Sync H Enable. The PCG_SYNC4.FSH bit enables synchronization of frame sync H with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled
6 (R/W)	HWG_TRIGEN	Hardware Trigger Sync G Enable. The PCG_SYNC4.HWG_TRIGEN bit enables synchronization of clock G and frame sync G with the external frame sync only after an internal slave trigger for PCG-G is received.
5 (R/W)	FSG_CLKINSEL	CLKIN source for FSG. The PCG_SYNC4.FSG_CLKINSEL bit enables the CLKIN1 input source for FSG
		0 Selects CLKIN0 as source for FSA CLKIN
		1 Selects CLKIN1 as source for FSA CLKIN

Table 27-27: PCG_SYNC4 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	CLKG_CLKINSEL	CLKIN source for CLKG. The PCG_SYNC4 . CLKG_CLKINSEL bit enables the CLKIN1 input source for CLKG
		0 Select CLKIN0 source for CLKA CLKIN
		1 Select CLKIN1 source for CLKA CLKIN
3 (R/W)	FSGSRC	Frame Sync Source. The PCG_SYNC4 . FSGSRC bit enables the frame sync G input source.
		0 Output selected by FSCSOURCE bit
		1 Clock derived from core PLL selected for frame sync C
2 (R/W)	CLKGSRC	Clock G Source. The PCG_SYNC4 . CLKGSRC bit enables the clock G input source.
		0 Output selected by CLKCSOURCE bit
		1 Clock derived from core PLL selected for clock C
1 (R/W)	CLKG	Clock G Enable. The PCG_SYNC4 . CLKG bit enables synchronization of clock G with the external frame sync.
		0 Clock disabled
		1 Clock enabled
0 (R/W)	FSG	Frame Sync G Enable. The PCG_SYNC4 . FSG bit enables synchronization of frame sync G with the external frame sync.
		0 Frame sync disabled
		1 Frame sync enabled

28 Asynchronous Sample Rate Converter (ASRC)

Sample rate converters (SRC) are frequently used in digital signal processing audio applications. The most frequently used sample rate conversions are off-loaded into hardware modules that are dedicated for filter processing and reduce the instruction processing load on the core, freeing it up for other tasks.

Features

The ASRC has these features and capabilities.

- 4 asynchronous stereo SRCs operating in slave mode are available in each DAI
- Simple programming model
- Controllable muting options (hardware, software and automatic)
- Automatically senses input and output sample frequencies
- Supports left-justified, I²S, right-justified (16-, 18-, 20-, 24-bits), and TDM serial port modes
- Daisy-chain configuration in TDM modes (including between DAI0 and DAI1) for input and output ports to create a serial frame
- Different protocols on input/output port allow format conversions
- De-emphasis filter for 32, 44.1 and 48 kHz sampling frequencies
- Up to 192 kHz sample rate input/output continuous sample ratios from 7.5:1 to 1:8
- Group delay (latency of interpolation filter) is 16 samples
- SNR from 128 to 140 dB (depending on processor model)
- Matched phase mode available to compensate for group delays
- Can be used to de-jitter clocks in systems

Functional Description

Conceptually, the sample rate converter interpolates the serial input data at a rate of 220 and samples the interpolated data stream by the output sample rate. In practice, a 64-tap FIR filter with 220 polyphases, a FIFO, a digital servo loop that measures the time difference between the input and output samples within 5 ps, and a digital circuit to track the sample rate ratio are used to perform the interpolation and output sampling.

ADSP-2159x ASRC Register List

Sample Rate Converter Module

Table 28-1: ADSP-2159x ASRC Register List

Name	Description
ASRC_CTL01	Control Register for ASRC 0 and 1
ASRC_CTL23	Control Register for ASRC 2 and 3
ASRC_MUTE	Mute Register
ASRC_RAT01	Ratio Register for ASRC 0 and 1
ASRC_RAT23	Ratio Register for ASRC 2 and 3

ASRC Interrupt List

The ASRC interrupts are controlled through the DAI.

Table 28-2: ASRC Interrupt List

Interrupt ID	Interrupt Name	Interrupt Condition
20	DAI0_IRQH	ASRC initialization
21	DAI1_IRQH	ASRC sample rate change
116	DAI0_IRQL	
117	DAI1_IRQL	

ASRC Block Diagram

The *ASRC Block Diagram* figure shows a top level block diagram of the ASRC module and the *Core Architecture* figure shows architecture details.

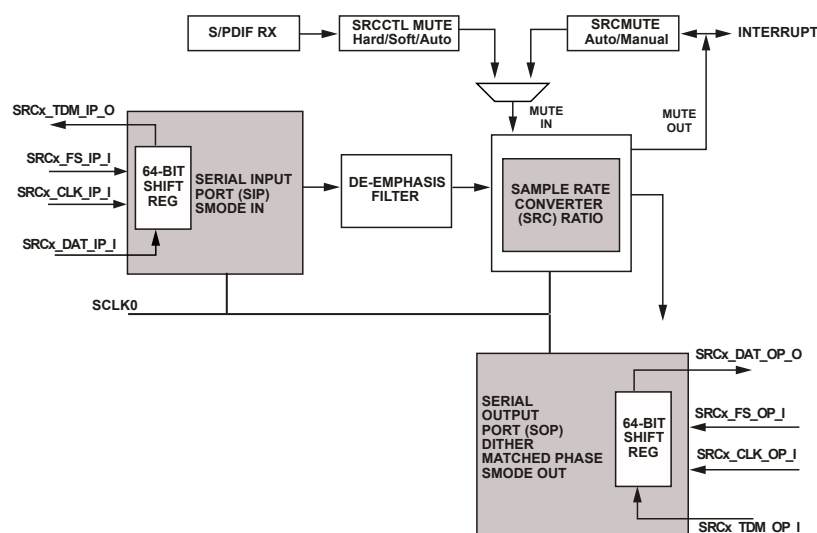


Figure 28-1: ASRC Block Diagram

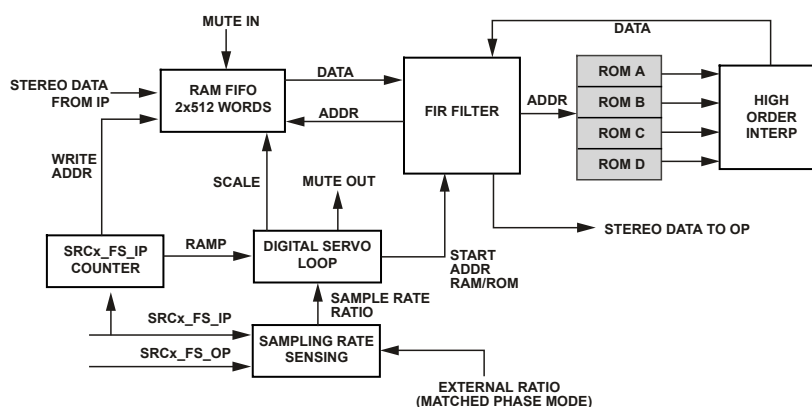


Figure 28-2: ASRC Core Architecture

SRU Programming

The SRU (signal routing unit) needs to be programmed in order to connect the ASRCs to the output pins or any other peripherals. For more information, see the [Digital Audio Interface \(DAI\)](#) chapter.

Clocking

The ASRC module is in the SCLK0 clock domain. An internal divided version of the SCLK0 clock is generated and used as the fundamental clock for the ASRC module.

I/O Ports

The I/O ports provide the interface through which data is transferred asynchronously into and out of the SRC modules. The SRC has a 3-wire interface for the serial input and output ports that supports left-justified, I²S, and right-justified (16-, 18-, 20-, 24-bit) modes. Additionally, the serial interfaces support TDM mode for daisy-chaining

multiple SRCs to form a frame. The serial output data is dithered down to 20, 18, or 16 bits when 20-, 18-, or 16-bit output data is selected.

NOTE: The SRC converts the data from the serial input port to the sample rate of the serial output port. The sample rate at the serial input port can be asynchronous with respect to the output sample rate of the output serial port.

De-Emphasis Filter

The de-emphasis filter is used to de-emphasize audio data that has been emphasized.

Mute Control

When either the SRC starts up (or there is a change in sample ratio), the mute out signal (`SRCx_MUTEOUT`) is asserted (=1). The mute out signal stays high until the SRC settles on the new sample rates. While mute out is asserted high, the mute in signal should be asserted high as well. The mute in signal performs a soft mute of the audio input data when asserted and un-mutes the input audio data softly when deasserted.

Note that it takes 4096 input port FS samples until the audio input data is completely muted and 4096 FS samples until the audio input data is completely unmuted.

SRC Core

As shown in the *ASRC Core Architecture* figure, the sample rate converter's RAM FIFO block adjusts the left and right input samples and stores them for the FIR filter's convolution cycle. The `ASRCx_FS_IP` counter provides the write address (for scaling) to the FIFO block and the ramp input to the digital-servo loop. The ROM stores the coefficients for the FIR filter convolution and performs a high-order interpolation between the stored coefficients. The sample rate ratio block measures the sample rate by dynamically altering the ROM coefficients and scaling the FIR filter length and input data. The digital-servo loop automatically tracks the `SRCx_FS_IP` and `SRCx_FS_OP` sample rates and provides the RAM and ROM start addresses for the start of the FIR filter convolution.

NOTE: Unlike other peripherals, the sample rate converters own local memories (RAM and ROM) which are dedicated for the purpose of sample rate conversion only.

The sample rate converter only operates asynchronously and is always a slave to the input and output ports.

RAM FIFO

The RAM FIFO receives the left and right input data and adjusts the amplitude of the data for both the soft muting of the SRC and the scaling of the input data by the sample rate ratio before storing the samples in RAM. The input data is scaled by the sample rate ratio because as the FIR filter length of the convolution increases, so does the amplitude of the convolution output. To keep the output of the FIR filter from saturating, the input data is scaled down by multiplying it by $(\text{SRCx_FS_OP})/(\text{SRCx_FS_IP})$ when $\text{SRCx_FS_OP} < \text{SRCx_FS_IP}$. The FIFO also scales the input data to mute and stop muting the SRC.

Digital Servo Loop

The digital-servo loop is essentially a ramp filter that provides the initial pointer to the address in RAM and ROM for the start of the FIR convolution. The RAM pointer is the integer output of the ramp filter while the ROM pointer is the fractional part. The digital-servo loop must be able to provide excellent rejection of jitter on the SRCx_FS_IP and SRCx_FS_OP clocks as well as measure the arrival of the SRCx_FS_OP clock within 5 ps. The digital-servo loop also divides the fractional part of the ramp output by the ratio of (SRCx_FS_IP)/(SRCx_FS_OP) for the case when SRCx_FS_IP > SRCx_FS_OP, to dynamically alter the ROM coefficients.

The digital-servo loop is implemented with a multi-rate filter. To settle the digital-servo loop filter quickly at startup or at a change in the sample rate, a fast mode has been added to the filter. When the digital-servo loop starts up or the sample rate is changed, the digital-servo loop kicks into fast mode to adjust and settle on the new sample rate. Upon sensing the digital-servo loop settling down to some reasonable value, the digital-servo loop kicks into normal or slow mode. During fast mode, the SRCx_MUTE_OUT bit of the ASRC is asserted to mute the ASRC input which avoids clicks and pops.

FIR Filter

The FIR filter is a 64-tap filter in the case of SRCx_FS_OP < SRCx_FS_IP and is (SRCx_FS_IP)/(SRCx_FS_OP) × 64 taps for the case when SRCx_FS_IP > SRCx_FS_OP. The FIR filter performs its convolution by loading in the starting address of the RAM address pointer and the ROM address pointer from the digital-servo loop at the start of the SRCx_FS_OP period. The FIR filter then steps through the RAM by decrementing its address by 1 for each tap, and the ROM pointer increments its address by the (SRCx_FS_OP/SRCx_FS_IP) × 2²⁰ ratio for SRCx_FS_IP > SRCx_FS_OP or 2²⁰ for SRCx_FS_OP < SRCx_FS_IP. Once the ROM address rolls over, the convolution is complete. The convolution is performed for both the left and right channels, and the multiply/accumulate circuit used for the convolution is shared between the channels.

Sample Rate Sensing

The (SRCx_FS_IP)/(SRCx_FS_OP) sample rate ratio circuit is used to dynamically alter the coefficients in the ROM for the case when SRCx_FS_IP > SRCx_FS_OP. The ratio is calculated by comparing the output of an SRCx_FS_OP counter to the output of an SRCx_FS_IP counter. If SRCx_FS_OP > SRCx_FS_IP, the ratio is held at one. If SRCx_FS_IP > SRCx_FS_OP, the sample rate ratio is updated if it is different by more than two SRCx_FS_OP periods from the previous SRCx_FS_OP to SRCx_FS_IP comparison. This is done to provide some hysteresis to prevent the filter length from oscillating and causing distortion.

Digital Filter Group Delay

The RAM in the FIFO is 512 words deep for both left and right channels. An offset of 16 samples to the write address, provided by the SRCx_FS_IP counter, is added to prevent the RAM read pointer from overlapping the write address. The maximum decimation rate can be calculated from the RAM word: depth = (512 – 16) ÷ 64 taps = 7.75:1.

The 64 samples effect latency in the interpolation filter. This latency (group delay) depends on interpolation or decimation ratio and is determined as follows:

Interpolation or Decimation Ratio (1): $GDL = 16/f_{S_IN} + 32/f_{S_IN}$ seconds for SRC_FS_OP > SRC_FS_IP

Interpolation or Decimation Ratio (2): $GDL = 16/f_{S_IN} + 32/f_{S_IN} \times f_{S_IN}/f_{S_OUT}$ seconds for $SRC_FS_OP < SRC_FS_IP$

Data Format

The *ASRC Data Frame Format by Protocol* figure shows the data input format for a frame (stereo data). The frame format is valid for all protocols. For models which do not support matched phase mode the 8-bit data field is ignored.

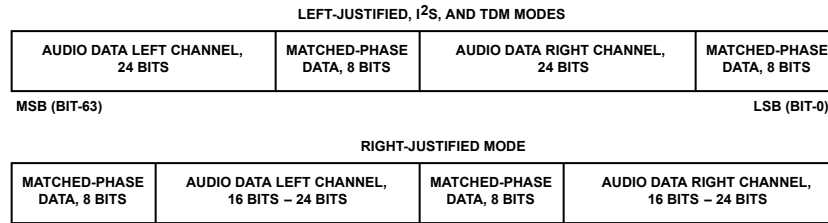


Figure 28-3: ASRC Data Frame Format by Protocol

Operating Modes

The ASRC can operate in TDM, I²S, left-justified, right-justified, and bypass modes. The serial ports of the processor can be used for moving the ASRC data to/from the internal memory.

In I²S, left-justified and right-justified modes, the ASRCs operate individually. The serial data provided in the input port is converted to the sample rate of the output port.

TDM Input Mode

In TDM input port, several ASRCs can be daisy-chained together and connected to the serial input port of a SHARC processor or other processor (see the *TDM Input/Output Modes* figure). The ASRC IP contains a 64-bit parallel load shift register. When the $SRCx_FS_IP_I$ pulse arrives, each ASRC parallel loads its left and right data into the 64-bit shift register. The input to the shift register is connected to $SRCx_DATA_IP_I$, while the output is connected to $SRCx_TDM_IP_O$. By connecting the $SRCx_TDM_IP_O$ to the $SRCx_DATA_IP_I$ of the next ASRC, a large shift register is created, which is clocked by $SRCx_CLK_IP_I$.

NOTE: In TDM mode, the ASRC drives at the rising edge and samples at the falling edge of the serial clock. In all other modes, the serial clock rising edge is the sampling edge, and the falling edge is the driving edge.

NOTE: The number of ASRCs that can be daisy-chained together is limited by the maximum frequency of $SRCx_CLK_xx_I$, refer to the data sheet for exact values. For example, if the maximum frequency of $SRCx_CLK_xx_I$ is x MHz, and the output sample rate is f_s , then number of ASRCs (n) that can be connected in daisy chained fashion is: $n \cdot 64 \cdot f_s \leq x \text{ MHz}$.

TDM Output Mode

As shown in the *TDM Input/Output Modes* figure, using the TDM output port several ASRCs can be daisy-chained together and connected to the SPORT of this or another processor. The ASRC OP contains a 64-bit parallel load shift register. When the $SRCx_FS_OP_I$ pulse arrives, each ASRC loads its left and right data into the 64-bit shift register. The input to the shift register is connected to $SRCx_TDM_OP_I$, and the output is connected to $SRCx_DAT_OP_O$. By connecting the $SRCx_DAT_OP_O$ to the $SRCx_TDM_OP_I$ of the next ASRC, a large shift register is created, which is clocked by $SRCx_CLK_OP_I$.

As shown in *TDM Input/Output Modes*, with three ASRCs in a daisy-chain connection, the serial clock for input/output port is defined as: $SCLK = 3 \times 64 \text{ FS} = 192 \text{ FS}$.

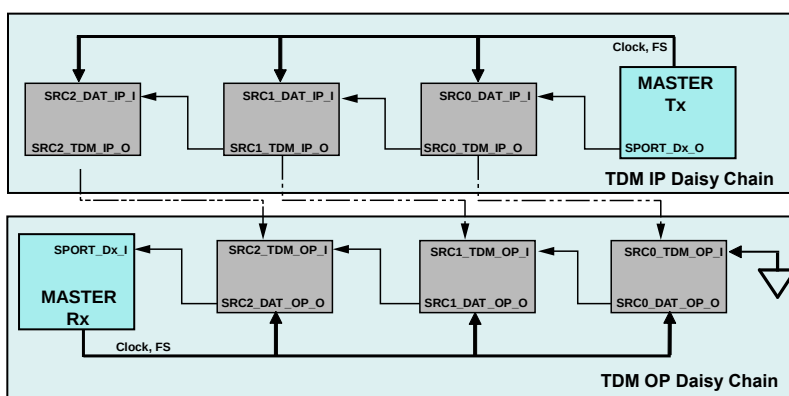


Figure 28-4: TDM Input/Output Modes

Matched-Phase Mode

The matched-phase mode of the sample rate converter, shown in *Typical Configuration for Matched-Phase Mode Operation*, is enabled by the $ASRC_CTL01.MPHASE0$, $ASRC_CTL01.MPHASE1$, $ASRC_CTL23.MPHASE2$ and $ASRC_CTL23.MPHASE3$ bits. This mode is used to match the phase (group delay) between two or more adjacent sample rate converters that are operating with the same input and output clocks.

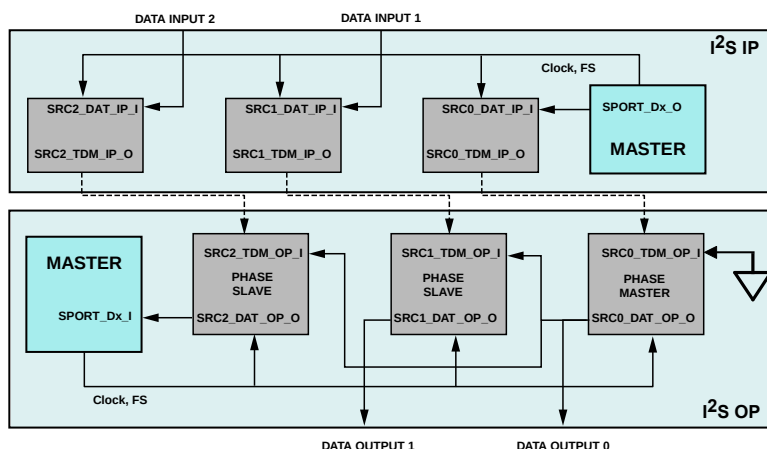


Figure 28-5: Typical Configuration for Matched-Phase Mode Operation

Hysteresis of the $(\text{SRCx_FS_OP})/(\text{SRCx_FS_IP})$ ratio circuit can cause phase mismatching between two ASRCs operating with the same input and output clocks. Since the hysteresis requires a difference of more than two SRCx_FS_OP periods to update the SRCx_FS_OP and SRCx_FS_IP ratios, two ASRCs may have differences in their ratios from 0 to 4 SRCx_FS_OP period counts. The $(\text{SRCx_FS_OP})/(\text{SRCx_FS_IP})$ ratio adjusts the filter length of the ASRC, which corresponds directly with the group delay. Thus, the magnitude in the phase difference depends upon the resolution of the SRCx_FS_OP and SRCx_FS_IP counters. The greater the resolution of the counters, the smaller the phase difference error.

When the slave SRC matched-phase mode bit is set (=1), it accepts the sample rate ratio transmitted by another SRC, (the matched-phase master) which has its matched-phase mode bit cleared (=0), through its serial output.

The phase master ASRC device transmits its $\text{SRCx_FS_OP}/\text{SRCx_FS_IP}$ ratio through the data output pin (SRCx_DAT_OP_O) to the slave's ASRC's data input pins (SRCx_TDM_OP_I). The transmitted data (32-bit subframe) contains 24-bit data and 8-bits matched phase (see the *ASRC Data Frame Format by Protocol* figure).

The slave SRCs receive the 8-bit matched phase bits (instead of their own internally-derived ratio) if their SRCx_MPHASE bits are set to 1, respectively. The SRCx_FS_IP and SRCx_FS_OP signals may be asynchronous with respect to each other in this mode. Note that there must be 64 SRCx_CLK_OP cycles per frame in matched-phase mode (two 24-bits data and two 8-bits phase match).

NOTE: By default, matched phased data is sent on the SRCx_DAT_OP_O pin, but only if the SRCx_TDM_OP_I pin is tied low. The slaves simply ignore the matched phased data if their $\text{ASRC_CTL01.MPHASE1}$ through $\text{ASRC_CTL23.MPHASE3}$ bits are cleared (= 0).

Bypass Mode

When the ASRC_CTL01.BYP0 , ASRC_CTL01.BYP1 , ASRC_CTL23.BYP2 and ASRC_CTL23.BYP3 bits are set (=1), the input data bypasses the sample rate converter and is sent directly to the serial output port. Dithering is disabled. This mode is ideal when the input and output sample rates are the same and the SRCx_FS_IP_I and SRCx_FS_OP_I signals are synchronous with respect to each other. In matched phase bypass mode, the SRCx_FS_OP_I signal should come at least one SRCx_CLK_xx_I period before SRCx_FS_IP_I . Cases where this is not met could result in data loss. For example, if internal SPORTs are used then the SRCx_FS_OP_I and SRCx_FS_IP_I signals could be driven by different SPORTs so that the timing of these signals can be controlled by enabling them at different times. This mode can also be used for passing through non-audio data since no processing is performed on the input data.

De-Emphasis Mode

The $\text{ASRC_CTL01.DEEMPHASIS0}$, $\text{ASRC_CTL01.DEEMPHASIS1}$, $\text{ASRC_CTL23.DEEMPHASIS2}$ and $\text{ASRC_CTL23.DEEMPHASIS3}$ bits choose the type of de-emphasis filter based on the input sample rate for 32, 44.1 or 48 kHz sampling rates.

Dithering Mode

The $\text{ASRC_CTL01.DITHER0}$, $\text{ASRC_CTL01.DITHER1}$, $\text{ASRC_CTL23.DITHER2}$, and $\text{ASRC_CTL23.DITHER3}$ control this mode of operation. Serial output data is dithered down to 20, 18, or 16 bits

when 20-, 18-, or 16-bit output data is selected. In the case of 20-, 18- and 16-bit word lengths, the least significant bits of the 24-bit word coming from the SRC into the serial output port are truncated. The `DITHER_EN` signal (not user configurable) automatically adds dithering to the 24-bit word before truncating to the appropriate output word length. The `21BIT_DITHER` signal is used for the consumer version of the SRC to reduce the dynamic range performance to approximately 128 dB.

NOTE: The ASRC can be programmed to add the triangular Probability Distribution Function (PDF) dither to the digital audio samples. It is advisable to add dither when the input word width exceeds the output word width, for example the input word is 20 bits and the output word is 16 bits. Triangular PDF is generally considered to create the most favorable noise shaping of the residual quantization noise.

Muting Modes

The mute feature of the ASRC can be controlled automatically in hardware using the `MUTE_IN` signal by connecting it to the `MUTE_OUT` signal. Automatic muting can be disabled by setting (=1) the `ASRC_MUTE.MUTE0` through `ASRC_MUTE.MUTE3` bits.

NOTE: Note that by default, the `ASRC_MUTE` register connects the `MUTE_IN` signal to the `MUTE_OUT` signal, but not conversely.

Soft Mute

When the `ASRC_CTL01.SOFTMUTE0`, `ASRC_CTL01.SOFTMUTE1`, `ASRC_CTL23.SOFTMUTE2` and `ASRC_CTL23.SOFTMUTE3` bits are set, the `MUTE_IN` signal is asserted, and the ASRC performs a soft mute by linearly decreasing the input data to the ASRC FIFO to zero, (-144 dB) attenuation as described for automatic hardware muting.

A 12-bit counter, clocked by `SRCx_FS_IP_I`, is used to control the mute attenuation. Therefore, the time it takes from the assertion of the `MUTE_IN` signal to -144 dB, full mute attenuation is 4096 FS cycles. Likewise, the time it takes to reach 0 dB mute attenuation from the deassertion of the `MUTE_IN` signal is 4096 FS cycles.

Hard Mute

When the `ASRC_CTL01.HARDMUTE0`, `ASRC_CTL01.HARDMUTE1`, `ASRC_CTL23.HARDMUTE2` and `ASRC_CTL23.HARDMUTE3` bits are set, the ASRC immediately mutes the input data to the ASRC FIFO to zero, (-144 dB) attenuation.

Auto Mute

When the `ASRC_CTL01.AUTOMUTE0`, `ASRC_CTL01.AUTOMUTE1`, `ASRC_CTL23.AUTOMUTE2` and `ASRC_CTL23.AUTOMUTE3` bits are set, the ASRC communicates with the S/PDIF receiver peripheral to determine when the input should mute.

This mode is useful for automatic detection of non-PCM audio data received from the S/PDIF receiver.

Interrupts

The following sections provide information about interrupt sources, masking and servicing.

Sources

Each ASRC module drives one interrupt signal (mute out asserted). All these signals are connected into the `DAI_IRPTL_H` or `DAI_IRPTL_L` latch registers. The ASRC ports generate interrupts as described below.

SRC Mute Out

The SRC mute out signal can be used to generate interrupts on their rising edge, falling edge, or both, depending on how the DAI interrupt mask registers (`DAI_IMSK_RE/DAI_IMSK_FE`) are programmed. This programming allows the generation of `DAI_IRPTL_H/DAI_IRPTL_L` interrupts either entering mute, exiting muting or both. The `SRCx_MUTE_OUT` interrupt is generated only once when the SRC is locked (after 4096 FS input samples) and after changes to the sample ratio. Hard mute, soft mute, and auto mute only control the muting of the input data to the SRC.

Masking

The `DAI_IMSK_FE`, `DAI_IMSK_RE`, and `DAI_IMSK_PRI` registers must be unmasked accordingly. The `DAI_IRQH` and `DAI_IRQL` signals are routed to the system event controller (SEC) and general interrupt controller (GIC).

Service

The ISR reads the `DAI_IRPTL_H` and `DAI_IRPTL_L` registers to clear the interrupt request.

Programming Model

The following is basic information on programming the ASRC module.

1. Program the `ASRC_CTL01` and `ASRC_CTL23` registers and keep the `ASRC_CTL01.EN0` through `ASRC_CTL23.EN3` bits cleared.
2. Set the `ASRC_CTL01.EN0` through `ASRC_CTL23.EN3` bits. After 4096 input port FS cycles the ASRC has un-muted.

Debug Features

The asynchronous sample rate converter allow the bypass mode. When the `ASRC_CTL01.BYP0` through `ASRC_CTL23.BYP3` bits are set (=1), the input data bypasses the sample rate converter and is sent directly to the serial output port. This mode can be used for testing both ports when the input and output sample rates are at the same frequency, therefore both input and output ports can be routed to the same serial clock and frame sync.

ADSP-2159x ASRC Register Descriptions

Sample Rate Converter Module (ASRC) contains the following registers.

Table 28-3: ADSP-2159x ASRC Register List

Name	Description
ASRC_CTL01	Control Register for ASRC 0 and 1
ASRC_CTL23	Control Register for ASRC 2 and 3
ASRC_MUTE	Mute Register
ASRC_RAT01	Ratio Register for ASRC 0 and 1
ASRC_RAT23	Ratio Register for ASRC 2 and 3

Control Register for ASRC 0 and 1

The `ASRC_CTL01` register (read/write) controls the operating modes, filters, and data formats used in the ASRC modules 0 and 1.

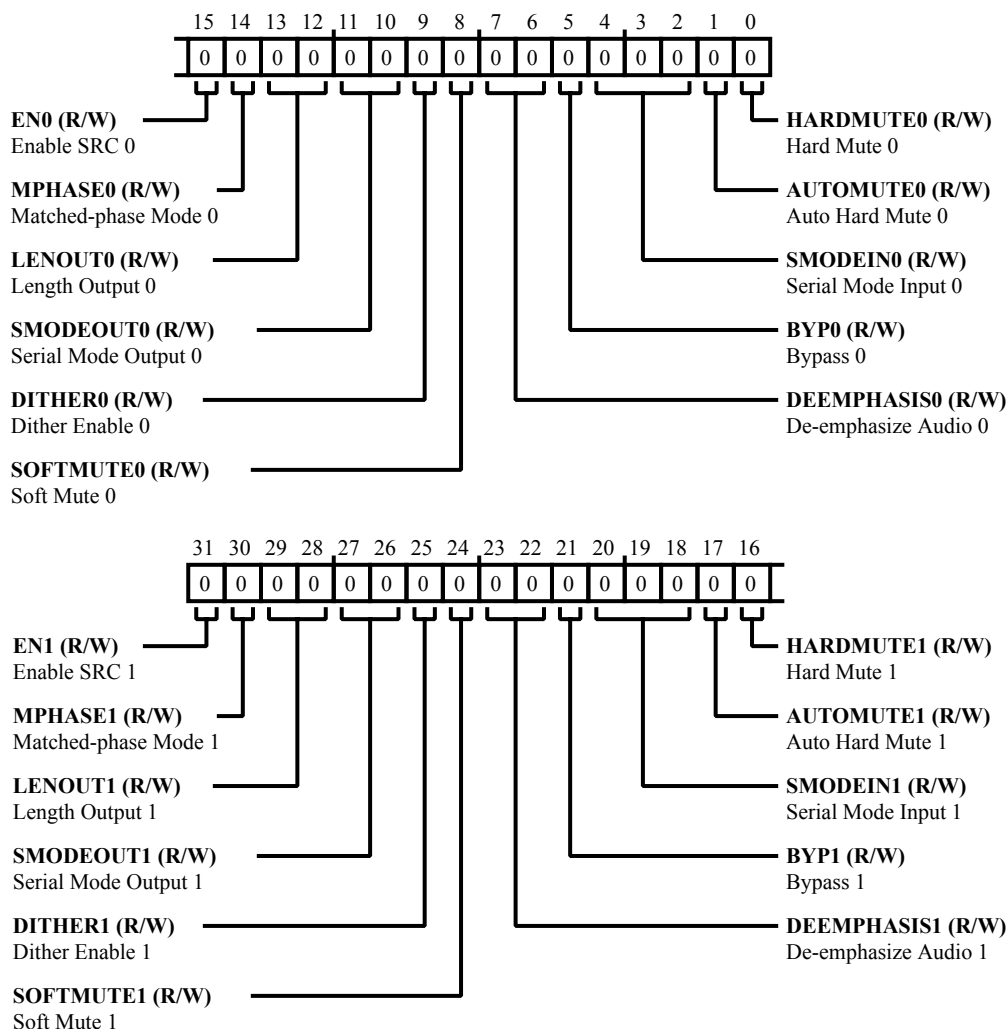


Figure 28-6: ASRC_CTL01 Register Diagram

Table 28-4: ASRC_CTL01 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
31 (R/W)	EN1	Enable SRC 1. The ASRC_CTL01 . EN1 bit enables SRC 1. When (set = 1), or when the sample rate (frame sync) between the input and output changes, the SRC begins its initialization routine where; 1) MUTE_OUT is asserted, 2) soft mute control counter for input samples is set to maximum attenuation (144 dB). Note that SRC power-up completion is finished by clearing the ASRC_RAT01 . MUTEOUT1 bit. Writes to the ASRC_CTL01 register should be at least one cycle before setting the ASRC_CTL01 . EN1 bit. When setting and clearing this bit, it should be held low for a minimum of 5 SYSCLK cycles.	
30 (R/W)	MPHASE1	Matched-phase Mode 1. The ASRC_CTL01 . MPHASE1 bit configures SRC1 to not use its own internally-generated sample rate ratio but use an externally-generated ratio. Used with TDM data.	
		0	Matched phase slave disabled
		1	Matched phase slave enabled
29:28 (R/W)	LENOUT1	Length Output 1. The ASRC_CTL01 . LENOUT1 bit field selects the serial output word length on SRC1.	
		0	24 bits
		1	20 bits
		2	18 bits
		3	16 bits
27:26 (R/W)	SMODEOUT1	Serial Mode Output 1. The ASRC_CTL01 . SMODEOUT1 bit field selects the serial output format on SRC1.	
		0	Left-justified
		1	I2S
		2	TDM
		3	Right-justified
25 (R/W)	DITHER1	Dither Enable 1. The ASRC_CTL01 . DITHER1 bit enables dithering before truncation on SRC1 when a word length less than 24 bits is selected.	
		0	Truncation only
		1	Dithering before truncation

Table 28-4: ASRC_CTL01 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/W)	SOFTMUTE1	Soft Mute 1. The ASRC_CTL01 . SOFTMUTE1 bit enables soft mute on SRC1.
		0 Unmute
		1 Mute
23:22 (R/W)	DEEMPHASIS1	De-emphasize Audio 1. The ASRC_CTL01 . DEEMPHASIS1 bits are used to de-emphasize audio data that has been emphasized. The type of de-emphasis filter is based on the input sample rate (SRCx_FS_IP_I signal).
		0 No de-emphasis
		1 32 kHz
		2 44.1 kHz
		3 48 kHz
21 (R/W)	BYP1	Bypass 1. The ASRC_CTL01 . BYP1 bit makes the output of SRC1 the same as the input.
20:18 (R/W)	SMODEIN1	Serial Mode Input 1. The ASRC_CTL01 . SMODEIN1 bit field selects the serial input format for SRC1.
		0 left-justified
		1 I2S
		2 TDM
		4 24-bit right-justified
		5 20-bit right-justified
		6 18-bit right-justified
		7 16-bit right-justified
17 (R/W)	AUTOMUTE1	Auto Hard Mute 1. The ASRC_CTL01 . AUTOMUTE1 bit auto hard mutes SRC1 when non audio is asserted by the SPDIF receiver.
		0 Unmute
		1 Mute
16 (R/W)	HARDMUTE1	Hard Mute 1. The ASRC_CTL01 . HARDMUTE1 bit hard mutes SRC1.
		0 Unmute
		1 Mute

Table 28-4: ASRC_CTL01 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
15 (R/W)	EN0	Enable SRC 0. The ASRC_CTL01.EN0 bit enables SRC 0. When (set =1), or when the sample rate (frame sync) between the input and output changes, the SRC begins its initialization routine where; 1) MUTE_OUT is asserted, 2) soft mute control counter for input samples is set to maximum attenuation (144 dB). Note that SRC power-up completion is finished by clearing the ASRC_RATE01.MUTEOUT0 bit. Writes to the ASRC_CTL01 register should be at least one cycle before setting the ASRC_CTL01.EN0 bit. When setting and clearing this bit, it should be held low for a minimum of 5 CLK cycles.	
14 (R/W)	MPHASE0	Matched-phase Mode 0. The ASRC_CTL01.MPHASE0 bit configures SRC0 to not use its own internally-generated sample rate ratio but use an externally-generated ratio. Used with TDM data.	
		0	Matched phase slave disabled
		1	Matched phase slave enabled
13:12 (R/W)	LENOUT0	Length Output 0. The ASRC_CTL01.LENOUT0 bit field selects the serial output word length on SRC0.	
		0	24 bits
		1	20 bits
		2	18 bits
		3	16 bits
11:10 (R/W)	SMODEOUT0	Serial Mode Output 0. The ASRC_CTL01.SMODEOUT0 bit field selects the serial output format on SRC0.	
		0	Left-justified
		1	I2S
		2	TDM
		3	Right-justified
9 (R/W)	DITHER0	Dither Enable 0. The ASRC_CTL01.DITHER0 bit enables dithering before truncation on SRC0 when a word length less than 24 bits is selected.	
		0	Truncation only
		1	Dithering before truncation

Table 28-4: ASRC_CTL01 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	SOFTMUTE0	Soft Mute 0. The ASRC_CTL01 . SOFTMUTE0 bit enables soft mute on SRC0.
		0 Unmute
		1 Mute
7:6 (R/W)	DEEMPHASIS0	De-emphasize Audio 0. The ASRC_CTL01 . DEEMPHASIS0 bits are used to de-emphasize audio data that has been emphasized. The type of de-emphasis filter is based on the input sample rate (SRCx_FS_IP_I signal).
		0 No de-emphasis
		1 32 kHz
		2 44.1 kHz
		3 48 kHz
5 (R/W)	BYP0	Bypass 0. The ASRC_CTL01 . BYP0 bit makes the output of SRC0 the same as the input.
4:2 (R/W)	SMODEIN0	Serial Mode Input 0. The ASRC_CTL01 . SMODEIN0 bit field selects the serial input format for SRC0.
		0 left-justified
		1 I2S
		2 TDM
		4 24-bit right-justified
		5 20-bit right-justified
		6 18-bit right-justified
		7 16-bit right-justified
1 (R/W)	AUTOMUTE0	Auto Hard Mute 0. The ASRC_CTL01 . AUTOMUTE0 bit auto hard mutes SRC0 when non audio is asserted by the SPDIF receiver.
		0 Unmute
		1 Mute
0 (R/W)	HARDMUTE0	Hard Mute 0. The ASRC_CTL01 . HARDMUTE0 bit hard mutes SRC0.
		0 Unmute
		1 Mute (default)

Control Register for ASRC 2 and 3

The `ASRC_CTL23` register (read/write) controls the operating modes, filters, and data formats used in the sample rate converter modules 2 and 3.

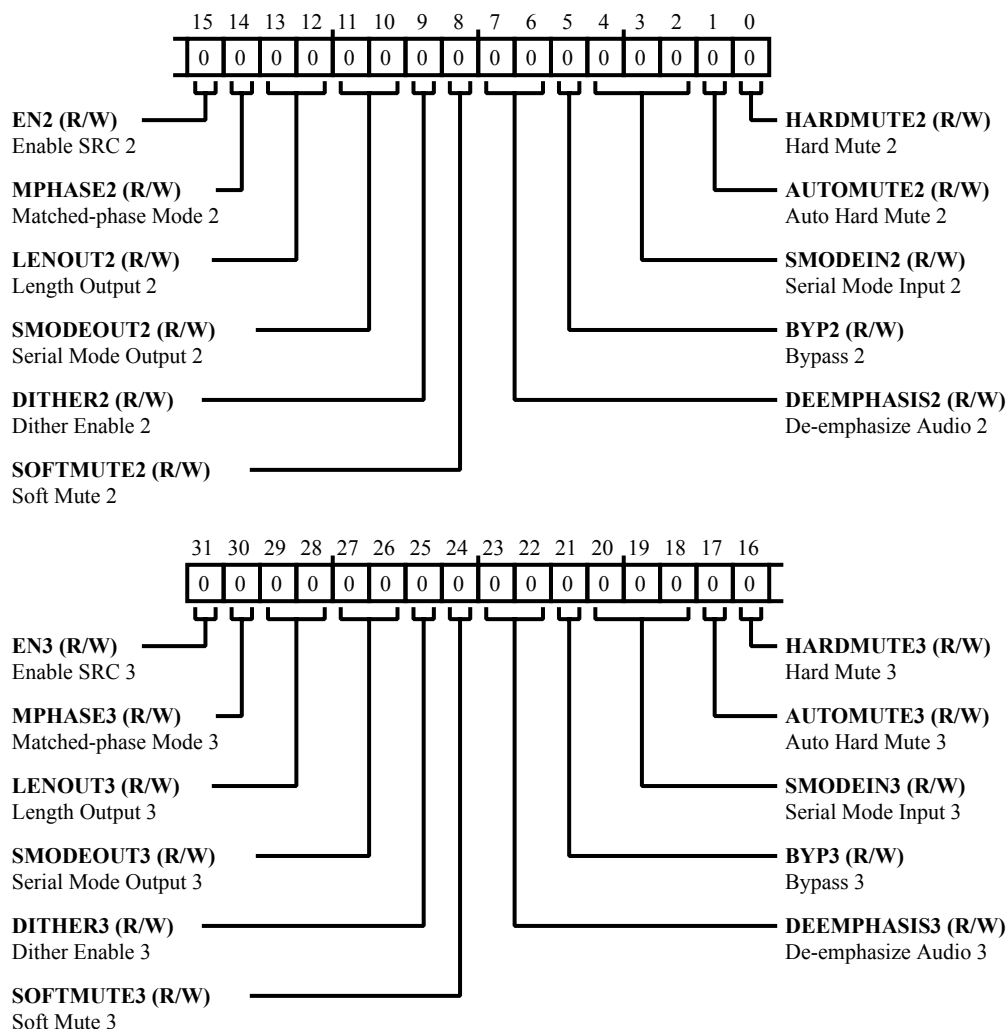


Figure 28-7: ASRC_CTL23 Register Diagram

Table 28-5: ASRC_CTL23 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration								
31 (R/W)	EN3	Enable SRC 3. The ASRC_CTL23.EN3 bit enables SRC 3. When (set =1), or when the sample rate (frame sync) between the input and output changes, the SRC begins its initialization routine where; 1) MUTE_OUT is asserted, 2) soft mute control counter for input samples is set to maximum attenuation (144 dB). Note that SRC power-up completion is finished by clearing the ASRC_RAT23.MUTEOUT3 bit. Writes to the ASRC_CTL23 register should be at least one cycle before setting the ASRC_CTL23.EN3 bit. When setting and clearing this bit, it should be held low for a minimum of 5 CLK cycles.								
30 (R/W)	MPHASE3	Matched-phase Mode 3. The ASRC_CTL23.MPHASE3 bit configures SRC3 to not use its own internally-generated sample rate ratio but use an externally-generated ratio. Used with TDM data. <table><tr><td>0</td><td>Matched phase slave disabled</td></tr><tr><td>1</td><td>Matched phase slave enabled</td></tr></table>	0	Matched phase slave disabled	1	Matched phase slave enabled				
0	Matched phase slave disabled									
1	Matched phase slave enabled									
29:28 (R/W)	LENOUT3	Length Output 3. The ASRC_CTL23.LENOUT3 bit field selects the serial output word length on SRC3. <table><tr><td>0</td><td>24 bits</td></tr><tr><td>1</td><td>20 bits</td></tr><tr><td>2</td><td>18 bits</td></tr><tr><td>3</td><td>16 bits</td></tr></table>	0	24 bits	1	20 bits	2	18 bits	3	16 bits
0	24 bits									
1	20 bits									
2	18 bits									
3	16 bits									
27:26 (R/W)	SMODEOUT3	Serial Mode Output 3. The ASRC_CTL23.SMODEOUT3 bit field selects the serial output format on SRC3. <table><tr><td>0</td><td>Left-justified</td></tr><tr><td>1</td><td>I2S</td></tr><tr><td>2</td><td>TDM</td></tr><tr><td>3</td><td>Right-justified</td></tr></table>	0	Left-justified	1	I2S	2	TDM	3	Right-justified
0	Left-justified									
1	I2S									
2	TDM									
3	Right-justified									
25 (R/W)	DITHER3	Dither Enable 3. The ASRC_CTL23.DITHER3 bit enables dithering before truncation on SRC3 when a word length less than 24 bits is selected. <table><tr><td>0</td><td>Truncation only</td></tr><tr><td>1</td><td>Dithering before truncation</td></tr></table>	0	Truncation only	1	Dithering before truncation				
0	Truncation only									
1	Dithering before truncation									

Table 28-5: ASRC_CTL23 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/W)	SOFTMUTE3	Soft Mute 3. The ASRC_CTL23.SOFTMUTE3 bit enables soft mute on SRC3.
		0 Unmute
		1 Mute
23:22 (R/W)	DEEMPHASIS3	De-emphasize Audio 3. The ASRC_CTL23.DEEMPHASIS3 bits are used to de-emphasize audio data that has been emphasized. The type of de-emphasis filter is based on the input sample rate (SRCx_FS_IP_I signal).
		0 No de-emphasis
		1 32 kHz
		2 44.1 kHz
		3 48 kHz
21 (R/W)	BYP3	Bypass 3. The ASRC_CTL23.BYP3 bit makes the output of SRC3 the same as the input.
20:18 (R/W)	SMODEIN3	Serial Mode Input 3. The ASRC_CTL23.SMODEIN3 bit field selects the serial input format for SRC3.
		0 left-justified
		1 I2S
		2 TDM
		4 24-bit right-justified
		5 20-bit right-justified
		6 18-bit right-justified
		7 16-bit right-justified
17 (R/W)	AUTOMUTE3	Auto Hard Mute 3. The ASRC_CTL23.AUTOMUTE3 bit auto hard mutes SRC3 when non audio is asserted by the SPDIF receiver.
		0 Unmute
		1 Mute
16 (R/W)	HARDMUTE3	Hard Mute 3. The ASRC_CTL23.HARDMUTE3 bit hard mutes SRC3.
		0 Unmute
		1 Mute

Table 28-5: ASRC_CTL23 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
15 (R/W)	EN2	Enable SRC 2. The ASRC_CTL23.EN2 bit enables SRC 2. When (set =1), or when the sample rate (frame sync) between the input and output changes, the SRC begins its initialization routine where; 1) MUTE_OUT is asserted, 2) soft mute control counter for input samples is set to maximum attenuation (144 dB). Note that SRC power-up completion is finished by clearing the ASRC_RATE23.MUTEOUT2 bit. Writes to the ASRC_CTL23 register should be at least one cycle before setting the ASRC_CTL23.EN2 bit. When setting and clearing this bit, it should be held low for a minimum of 5 CLK cycles.	
14 (R/W)	MPHASE2	Matched-phase Mode 2. The ASRC_CTL23.MPHASE2 bit configures SRC2 to not use its own internally-generated sample rate ratio but use an externally-generated ratio. Used with TDM data.	
		0	Matched phase slave disabled
		1	Matched phase slave enabled
13:12 (R/W)	LENOUT2	Length Output 2. The ASRC_CTL23.LENOUT2 bit field selects the serial output word length on SRC2.	
		0	24 bits
		1	20 bits
		2	18 bits
		3	16 bits
11:10 (R/W)	SMODEOUT2	Serial Mode Output 2. The ASRC_CTL23.SMODEOUT2 bit field selects the serial output format on SRC2.	
		0	Left-justified
		1	I2S
		2	TDM
		3	Right-justified
9 (R/W)	DITHER2	Dither Enable 2. The ASRC_CTL23.DITHER2 bit enables dithering before truncation on SRC2 when a word length less than 24 bits is selected.	
		0	Truncation only
		1	Dithering before truncation

Table 28-5: ASRC_CTL23 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W)	SOFTMUTE2	Soft Mute 2. The ASRC_CTL23.SOFTMUTE2 bit enables soft mute on SRC2.
		0 Unmute
		1 Mute
7:6 (R/W)	DEEMPHASIS2	De-emphasize Audio 2. The ASRC_CTL23.DEEMPHASIS2 bits are used to de-emphasize audio data that has been emphasized. The type of de-emphasis filter is based on the input sample rate (SRCx_FS_IP_I signal).
		0 No de-emphasis
		1 32 kHz
		2 44.1 kHz
		3 48 kHz
5 (R/W)	BYP2	Bypass 2. The ASRC_CTL23.BYP2 bit makes the output of SRC2 the same as the input.
4:2 (R/W)	SMODEIN2	Serial Mode Input 2. The ASRC_CTL23.SMODEIN2 bit field selects the serial input format for SRC2.
		0 left-justified
		1 I2S
		2 TDM
		4 24-bit right-justified
		5 20-bit right-justified
		6 18-bit right-justified
		7 16-bit right-justified
1 (R/W)	AUTOMUTE2	Auto Hard Mute 2. The ASRC_CTL23.AUTOMUTE2 bit auto hard mutes SRC2 when non audio is asserted by the SPDIF receiver.
		0 Unmute
		1 Mute
0 (R/W)	HARDMUTE2	Hard Mute 2. The ASRC_CTL23.HARDMUTE2 bit hard mutes SRC2.
		0 Unmute
		1 Mute

Mute Register

This register connects an ASRCx mute input and output when the mute bit is cleared (=0). This allows ASRCx to automatically mute input while the ASRC is initializing (0 = automatic muting and 1 = manual muting). Bit 0 controls ASRC0, bit 1 controls ASRC1, bit 2 controls ASRC2, and bit 3 controls ASRC3.

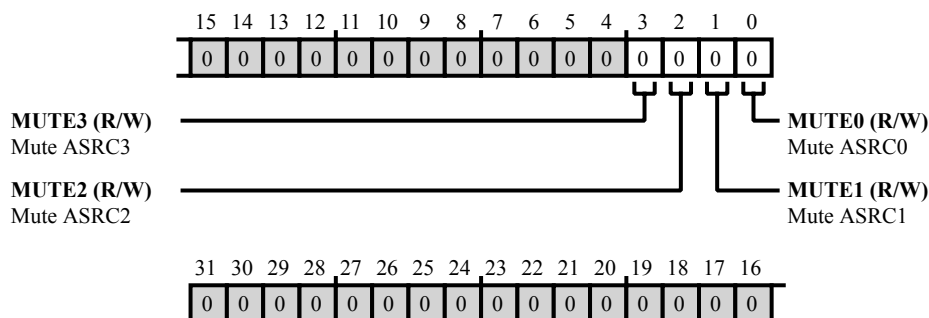


Figure 28-8: ASRC_MUTE Register Diagram

Table 28-6: ASRC_MUTE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	MUTE3	Mute ASRC3. The ASRC_MUTE.MUTE3 bit automatically mutes ASRC3 output when cleared (=0).
2 (R/W)	MUTE2	Mute ASRC2. The ASRC_MUTE.MUTE2 bit automatically mutes ASRC2 output when cleared (=0).
1 (R/W)	MUTE1	Mute ASRC1. The ASRC_MUTE.MUTE1 bit automatically mutes ASRC1 output when cleared (=0).
0 (R/W)	MUTE0	Mute ASRC0. The ASRC_MUTE.MUTE0 bit automatically mutes ASRC0 output when cleared (=0).

Ratio Register for ASRC 0 and 1

The `ASRC_RAT01` register reports the mute and I/O sample ratio for ASRC0 and ASRC1.

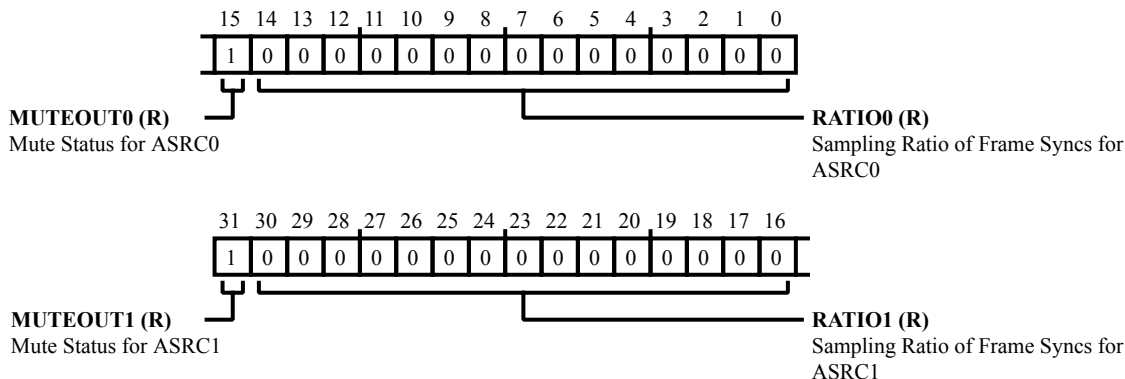


Figure 28-9: ASRC_RAT01 Register Diagram

Table 28-7: ASRC_RAT01 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	MUTEOUT1	Mute Status for ASRC1. The <code>ASRC_RAT01.MUTEOUT1</code> bit field reports the status of the <code>MUTE_OUT</code> signal. Once the <code>SRCx_MUTEOUT</code> signal is cleared, the ratio can be read. When ASRC1 is enabled or there is a change in the sample ratio, the <code>MUTE_OUT</code> signal is asserted. The <code>MUTE_OUT</code> signal remains asserted until the digital servo loops internal fast settling mode is complete. When the digital servo loop has switched to slow settling mode, the <code>MUTE_OUT</code> signal is deasserted.
30:16 (R/NW)	RATIO1	Sampling Ratio of Frame Syncs for ASRC1. The <code>ASRC_RAT01.RATIO1</code> bit field is read to find the ratio of output to input sampling frequency for ASRC1 (<code>SRCx_FS_OP_I/SRCx_FS_IP_I</code>). This ratio is reported in 4.11 (integer.fraction) format where the 15-bit value of the normal binary number is comprised of 4 bits for the integer and 11 bits for the fraction.
15 (R/NW)	MUTEOUT0	Mute Status for ASRC0. The <code>ASRC_RAT01.MUTEOUT0</code> bit field reports the status of the <code>MUTE_OUT</code> signal. Once the <code>SRCx_MUTEOUT</code> signal is cleared, the ratio can be read. When ASRC0 is enabled or there is a change in the sample ratio, the <code>MUTE_OUT</code> signal is asserted. The <code>MUTE_OUT</code> signal remains asserted until the digital servo loops internal fast settling mode is complete. When the digital servo loop has switched to slow settling mode, the <code>MUTE_OUT</code> signal is deasserted.

Table 28-7: ASRC_RAT01 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14:0 (R/NW)	RATIO0	Sampling Ratio of Frame Syncs for ASRC0. The <code>ASRC_RAT01.RATIO0</code> bit field is read to find the ratio of output to input sampling frequency for ASRC0 (<code>SRCx_FS_OP_I/SRCx_FS_IP_I</code>). This ratio is reported in 4.11 (integer.fraction) format where the 15-bit value of the normal binary number is comprised of 4 bits for the integer and 11 bits for the fraction.

Ratio Register for ASRC 2 and 3

The `ASRC_RAT23` register reports the mute and I/O sample ratio for ASRC0 and ASRC1.

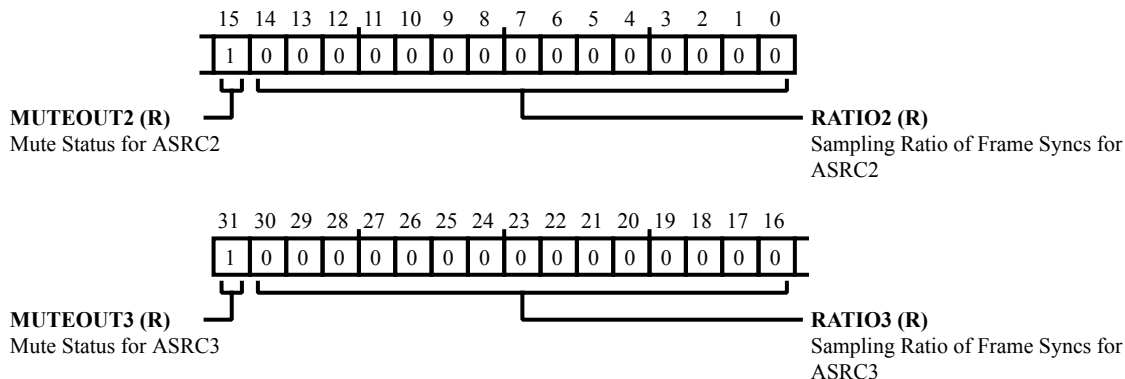


Figure 28-10: ASRC_RAT23 Register Diagram

Table 28-8: ASRC_RAT23 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/NW)	MUTEOUT3	Mute Status for ASRC3. The <code>ASRC_RAT23.MUTEOUT3</code> bit field reports the status of the <code>MUTE_OUT</code> signal. Once the <code>SRCx_MUTEOUT</code> signal is cleared, the ratio can be read. When ASRC3 is enabled or there is a change in the sample ratio, the <code>MUTE_OUT</code> signal is asserted. The <code>MUTE_OUT</code> signal remains asserted until the digital servo loops internal fast settling mode is complete. When the digital servo loop has switched to slow settling mode, the <code>MUTE_OUT</code> signal is deasserted.
30:16 (R/NW)	RATIO3	Sampling Ratio of Frame Syncs for ASRC3. The <code>ASRC_RAT23.RATIO3</code> bit field is read to find the ratio of output to input sampling frequency for ASRC3 (<code>SRCx_FS_OP_I/SRCx_FS_IP_I</code>). This ratio is reported in 4.11 (integer.fraction) format where the 15-bit value of the normal binary number is comprised of 4 bits for the integer and 11 bits for the fraction.
15 (R/NW)	MUTEOUT2	Mute Status for ASRC2. The <code>ASRC_RAT23.MUTEOUT2</code> bit field reports the status of the <code>MUTE_OUT</code> signal. Once the <code>SRCx_MUTEOUT</code> signal is cleared, the ratio can be read. When ASRC2 is enabled or there is a change in the sample ratio, the <code>MUTE_OUT</code> signal is asserted. The <code>MUTE_OUT</code> signal remains asserted until the digital servo loops internal fast settling mode is complete. When the digital servo loop has switched to slow settling mode, the <code>MUTE_OUT</code> signal is deasserted.

Table 28-8: ASRC_RAT23 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
14:0 (R/NW)	RATIO2	Sampling Ratio of Frame Syncs for ASRC2. The <code>ASRC_RAT23.RATIO2</code> bit field is read to find the ratio of output to input sampling frequency for ASRC2 (<code>SRCx_FS_OP_I/SRCx_FS_IP_I</code>). This ratio is reported in 4.11 (integer.fraction) format where the 15-bit value of the normal binary number is comprised of 4 bits for the integer and 11 bits for the fraction.

29 Sony/Philips Digital Interface (S/PDIF)

The Sony/Philips Digital Interface (S/PDIF) is a standard audio data transfer format that allows the transfer of digital audio signals from one device to another without having to convert them to an analog signal. The digital audio interface carries three types of information; audio data, non audio data (compressed data) and timing information.

Features

The S/PDIF interface has the following features.

- Supports one stereo channel or compressed audio streams.
- AES3-compatible S/PDIF transmitter and receiver.
- Transmitting a biphase mark encoded signal that may contain any number of audio channels (compressed or linear pulse code modulation) or non-audio data.
- S/PDIF receiver managing clock recovery with separate S/PDIF on-chip PLL.
- S/PDIF receiver supports the detection of DTS frames of 256, 512, 1024, 2048, and 4096.
- Manage user status information and provide error-handling capabilities in both the transmitter and receiver.
- DAI allows interactions over DAI by serial ports and the external DAI pins to interface to other S/PDIF devices. This includes using the receiver to decode incoming biphase encoded audio streams and passing them via the SPORTs to internal memory for processing-or using the transmitter to encode audio or digital data and transfer it to another S/PDIF receiver in the audio system.

It is important to be familiar with serial digital audio interface standards IEC-60958, EIAJ CP-340, AES3 and AES11.

ADSP-2159x SPDIF Register List

The S/PDIF module is a standard audio data transfer format that allows the transfer of digital audio signals from one device to another without having to convert them to an analog signal. A set of registers govern S/PDIF operations. For more information on S/PDIF functionality, see the S/PDIF register descriptions.

Table 29-1: ADSP-2159x SPDIF Register List

Name	Description
SPDIF_RX_CTL	Receive Control
SPDIF_RX_STAT	Receive Status Register
SPDIF_RX_STAT0_A	Receive Status A0 Register
SPDIF_RX_STAT0_B	Receive Status B0 Register
SPDIF_RX_STAT1_A	Receive Status A1 Register
SPDIF_RX_STAT1_B	Receive Status B1 Register
SPDIF_TX_CTL	Transmit Control Register
SPDIF_TX_STAT_A0	Transmit Status A0 Register
SPDIF_TX_STAT_A1	Transmit Status A1 Register
SPDIF_TX_STAT_A2	Transmit Status A2 Register
SPDIF_TX_STAT_A3	Transmit Status A3 Register
SPDIF_TX_STAT_A4	Transmit Status A4 Register
SPDIF_TX_STAT_A5	Transmit Status A5 Register
SPDIF_TX_STAT_B0	Transmit Status B0 Register
SPDIF_TX_STAT_B1	Transmit Status B1 Register
SPDIF_TX_STAT_B2	Transmit Status B2 Register
SPDIF_TX_STAT_B3	Transmit Status B3 Register
SPDIF_TX_STAT_B4	Transmit Status B4 Register
SPDIF_TX_STAT_B5	Transmit Status B5 Register
SPDIF_TX_UBUFF_A0	Transmit User Buffer A0 Register
SPDIF_TX_UBUFF_A1	Transmit User Buffer A1 Register
SPDIF_TX_UBUFF_A2	Transmit User Buffer A2 Register
SPDIF_TX_UBUFF_A3	Transmit User Buffer A3 Register
SPDIF_TX_UBUFF_A4	Transmit User Buffer A4 Register
SPDIF_TX_UBUFF_A5	Transmit User Buffer A5 Register
SPDIF_TX_UBUFF_B0	Transmit User Buffer B0 Register
SPDIF_TX_UBUFF_B1	Transmit User Buffer B1 Register
SPDIF_TX_UBUFF_B2	Transmit User Buffer B2 Register
SPDIF_TX_UBUFF_B3	Transmit User Buffer B3 Register
SPDIF_TX_UBUFF_B4	Transmit User Buffer B4 Register
SPDIF_TX_UBUFF_B5	Transmit User Buffer B5 Register

Table 29-1: ADSP-2159x SPDIF Register List (Continued)

Name	Description
SPDIF_TX_USRUPDT	User Bit Update Register

SRU Programming

The SRU (signal routing unit) is used to connect the S/PDIF transmitter biphasic data out to the output pins or to the S/PDIF receiver. The serial clock, frame sync, data, and external sync (if external synchronization is required) inputs also need to be routed through the SRU. For details of the routing, see the DAI Routing Capabilities section in the *Digital Audio Interface (DAI)* chapter.

The SRU needs to be programmed in order to connect the S/PDIF receiver to the output pins or any other peripherals and also for the connection to the input biphasic stream.

Program the corresponding SRU registers to connect the outputs to the required destinations (see the DAI Routing Capabilities section). The biphasic encoded data and the external PLL clock inputs to the receiver are routed through the SRU. The extracted clock, frame sync, and data are also routed through the SRU.

S/PDIF Interrupt List

Table 29-2: S/PDIF Interrupt List

Interrupt Name	Interrupt Condition	Return DAI Register	Return SEC Register	SEC ID
DAI0_IRQH	Block Start	DAIx_IRPTL	SEC_ID	24
DAI0_IRQL	Non Audio			145

Clocking

The fundamental timing clock of the S/PDIF receiver is CLK05 from the CDU. When CLK05 is configured, it supports sampling frequencies of 24 kHz to 192 kHz. The fundamental timing clock of the S/PDIF transmitter is *SCLK0/4*.

Sample rates of 24 kHz to 96 kHz are supported using a 170 MHz to 180 MHz setting on CLK05.

Sample rates of 32 kHz to 192 kHz are supported using a 225.0 MHz setting on CLK05.

For information on clock programming, see [CDU Programming Model](#).

S/PDIF Transmitter

The following sections provide information on the S/PDIF transmitter.

Functional Description

The S/PDIF transmitter, shown in the *S/PDIF Transmitter Block Diagram*, resides within the DAI, and its inputs and outputs can be routed via the SRU. It receives audio data in serial format, encloses the specified user status information, and converts it into the biphas encoded signal. The serial data input to the transmitter can be formatted as left-justified, I²S, or right-justified with word widths of 16, 18, 20 or 24 bits. *AES3 Output Block* shows the detail of the AES block.

The serial data, clock, and frame sync inputs to the S/PDIF transmitter are routed through the signal routing unit (SRU).

The S/PDIF transmitter output may be routed to an output pin via the SRU and then routed to another S/PDIF receiver or to components for off-board connections to other S/PDIF receivers. The output is also available to the S/PDIF receiver for loop-back testing through SRU.

In addition to encoding the audio data in the bi-phase format, the transmitter also provides a way to easily add the channel status information to the outgoing bi-phase stream. There are status/user registers for a frame (192-bits/24 bytes) in the transmitter that correspond to each channel or subframe.

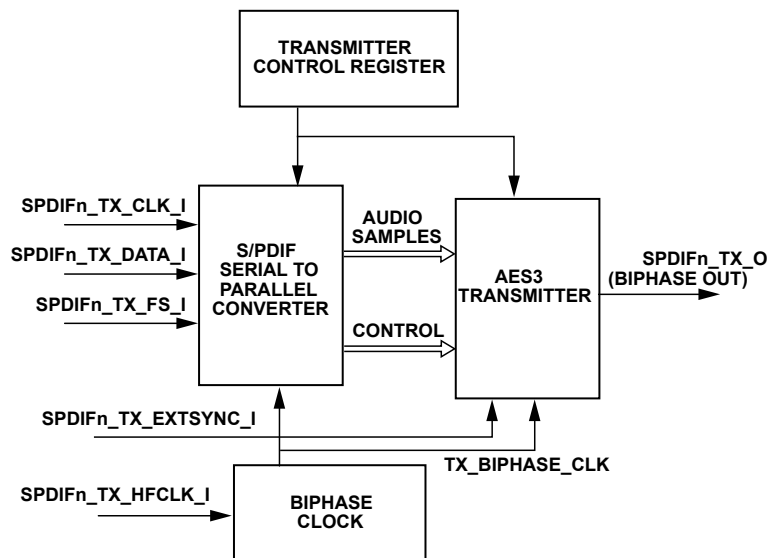


Figure 29-1: S/PDIF Transmitter Block Diagram

Validity bits for both channels may also be controlled by the transmitter control register. Optionally, the user bit, validity bit, and channel status bit are sent to the transmitter with each left/right sample. For each subframe the parity bit is automatically generated and inserted into the bi-phase encoded data.

A mute control and support for double-frequency single-channel mode are also provided. The serial data input format may be selected as left-justified, I²S, or right-justified with 16-, 18-, 20- or 24-bit word widths. The over sampling clock is also selected by the transmitter control register.

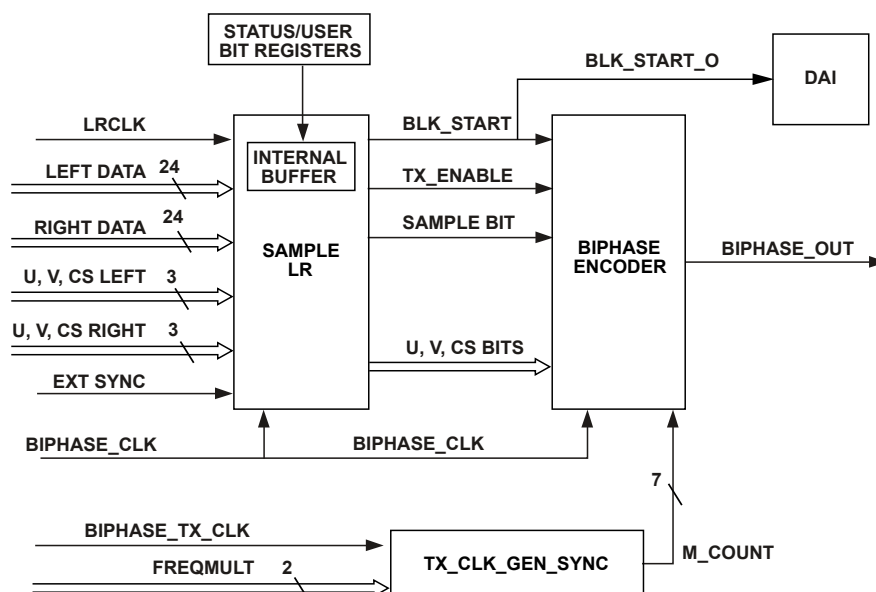
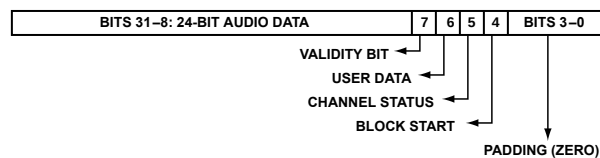


Figure 29-2: AES3 Output Block

Input Data Formats

The *I²S and Left-Justified Formats* and *Right-Justified Formats* figures show the format of data that is sent to the S/PDIF transmitter using a variety of protocol standards.

Figure 29-3: I²S and Left-Justified Formats

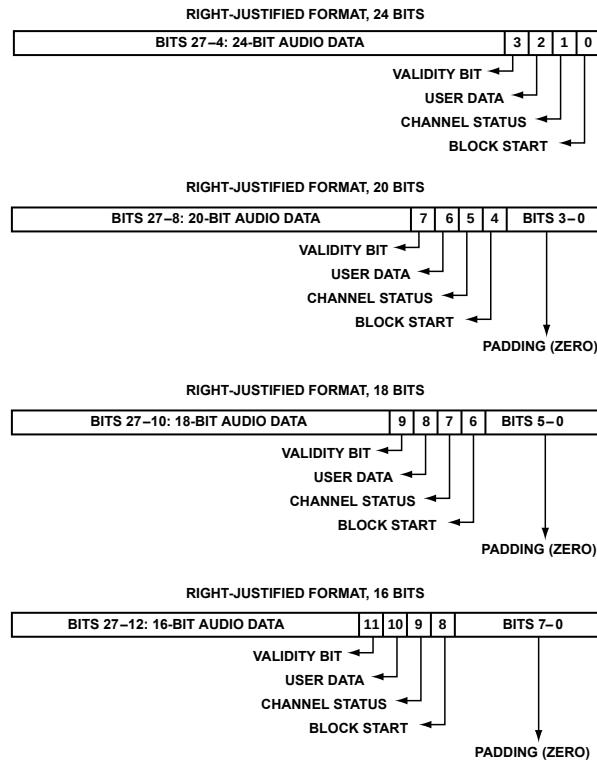


Figure 29-4: Right-Justified Formats

Operating Modes

The S/PDIF transmitter can operate in standalone and full serial modes. The following sections describe these modes in detail.

Full Serial Mode

This mode is selected by clearing the `SPDIF_TX_CTL.AUTO` bit. In this mode all the status bits, audio data and the block start bit (indicating start of a frame), come through the serial data stream (`SPDIF_TX_DATA_I`) pin. The transmitter should be enabled after or at the same time as all of the other control bits.

Standalone Mode

This mode is selected by setting the `SPDIF_TX_CTL.AUTO` bit. In this mode, the block start bit (indicating the start of a frame) is generated internally. The channel status bits come from the channel status buffer registers. The user status bits come from the user bits buffers as shown in the *AES3 Output Block* figure.

The validity bits are the `SPDIF_TX_CTL.VALIDR` and `SPDIF_TX_CTL.VALIDL`. In this mode only audio data comes from the `SPDIF_TX_DATA_I` pin. All other data, including the status bit and block start bit is either generated internally or taken from the internal register.

Once the user bits buffer registers (`SPDIF_TX_UBUFF_A0` - `SPDIF_TX_UBUFF_B5`) are programmed, they are used only for the next block of data. This allows programs to change the user bit information in every block of data.

To allow user bit updates, write a 0x1 to the `SPDIF_TX_USRUPDT` register that is used for further processing. If the `SPDIF_TX_CTL.AUTO` bit is set:

- and if `SPDIF_TX_USRUPDT` =1, at every 192nd frame end the user status bits are taken from user bits buffers and transmitted. Simultaneously, the `SPDIF_TX_USRUPDT` register is cleared automatically by hardware.
- and if `SPDIF_TX_USRUPDT` =0, at every 192nd frame end the user status bits are updated as zeros and transmitted. The `SPDIF_TX_USRUPDT` register remains low.

In general, for the next block, programs can update user bits buffers at any time during the transfer of the current block (1 block = 192 frames). There are internal buffers to store the user status bits of the current block of transfer. In other words, at the beginning of every new block, the user status bit (`SPDIF_TX_CTL.USRPEND` bit) from user bits buffers are copied to internal buffers and transmitted in each frame during the transfer.

Note that since a frame contains $192 \text{ bits} / 8 = 24 \text{ bytes}$, six status/user registers are required to store each four bytes.

Data Output Mode

Two output data formats are supported by the transmitter; *two channel mode* and *single-channel double-frequency* (SCDF) mode. The output format is determined by the transmitter control register (`SPDIF_TX_CTL`).

In two channel mode, the left channel (channel A) is transmitted when the `SPDIF_TX_FS_I` is high and the right channel (channel B) is transmitted when the `SPDIF_TX_FS_I` is low.

In SCDF mode, the transmitter sends successive audio samples of the same signal across both sub frames, instead of channel A and B. The transmitter will transmit at half the sample rate of the input bit stream. The `SPDIF_TX_CTL.SCDF` bit selects SCDF mode. When in SCDF mode, the `SPDIF_TX_CTL.SCDFLR` bit determines whether left or right channel data is transmitted.

S/PDIF Receiver

The S/PDIF receiver (*S/PDIF Receiver Block Diagram*) is compliant with all common serial digital audio interface standards including IEC-60958, IEC-61937, AES3, and AES11. For the IEC-60958 standard, all the user-data and channel-status bits (as outlined in this document) are not decoded by the S/PDIF receiver. The interface does make all 192 user-data and channel-status pairs available as an output of the block, for post-decoding.

For the IEC-61937 standard, the S/PDIF only detects compressed AC-3 and DTS formats. No decompression is performed.

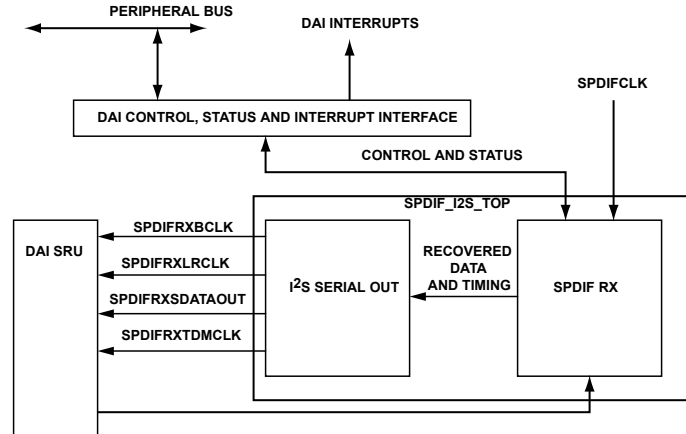


Figure 29-5: S/PDIF Receiver Block Diagram

Functional Description

If the receiver is used, programs need to enable it using the `SPDIF_RX_CTL` register. After the SRU programming is complete, write to the register with control values. At this point, the receiver attempts to lock.

NOTE: The S/PDIF receiver is disabled at default. If the receiver is used in an application, programs should enable the receiver.

The input to the receiver (`SPDIFn_RX_I`) is a biphas encoded signal that may contain two audio channels (compressed or linear PCM) or non-audio data. The receiver decodes the single biphas encoded stream, producing an I²S compatible serial data output that consists of a serial clock, a left-right frame sync, and data (channel A/B). It provides the programmer with several methods of managing the incoming status bit information.

The S/PDIF receiver receives any S/PDIF stream with a sampling frequency range of 24 kHz to 192 kHz. Refer to [Clocking](#) for more details.

The channel status bits are collected into memory-mapped registers, while other channel status and user bytes must be handled manually. The block start bit, which replaces the parity bit in the serial I²S stream, indicates the reception of the Z preamble and the start of a new block of channel status and data bits.

Clock Recovery

The S/PDIF receiver recovers the clock that generated the AES3/SPDIF biphas encoded stream from the incoming S/PDIF stream.

This clock is used by the receiver to clock in the biphas encoded data stream and also to provide clocks for either the SPORTs, sample rate converter, or the AES3 and S/PDIF transmitter. The recovered clock may also be used externally to the chip for clocking D/A and A/D converters.

In order to maintain performance, jitter on the clock is sourced to several peripherals.

To comply with the AES11 standard, the recovered left or right clock must be aligned with the preambles within a + or - 5% of the frame period. Since the PLL clock generates a clock 512 times the frame rate clock ($512 f_{\text{CLK}}$), this clock can be used and divided down to create the phase aligned jitter-free left or right clock.

The SPDIF recovered TDM master clock can be jittery depending on the biphasic input source. Do not use this as a clock source in audio applications.

TDM Clock Output

The SPDIF receiver generates the TDMCLK clock signal which is 256 times the sampling rate. For proper generation of TDMCLK, program the SPDIF receiver as shown in the *TDMCLK Configuration* table.

IMPORTANT: The SPDIF receiver guarantees 256 clocks in one frame sync. It does not guarantee the duty cycle of the generated clocks. Therefore, do not use this clock in the applications that have duty cycle requirements.

Table 29-3: TDMCLK Configuration

SPDIF_RX_CTL.TDMSEL	Recommended Sampling Rate
0000	$\leq 96\text{K}$
0001	192K
0011	96K - 192K
0101	96K

Output Data Format

The extracted 24-bit audio data, V, U, C and block start bits are sent on the SPDIF_RX_DAT pin in 32-bit I²S format as shown in *I2S and Left-Justified Formats*. The frame sync is transmitted on the SPDIF_RX_FS pin and serial clock is transmitted on the SPDIF_RX_CLK pin. All three pins are routed through the SRU.

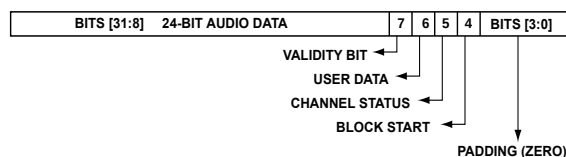


Figure 29-6: I2S Format

Channel Status

The channel status for the first bytes 4-0 (consumer mode) are collected into memory-mapped registers (`SPDIF_RX_STAT0_A`, `SPDIF_RX_STAT0_B`, `SPDIF_RX_STAT1_A` and `SPDIF_RX_STAT1_B`). All other channel status bytes 23-5 (professional mode) must be manually extracted from the receiver data stream.

NOTE: Only the first 5 channel status bytes (40-bit) for consumer mode of a frame are stored into the S/PDIF receiver status registers.

Operating Modes

This section describes the receiver channel status for the different modes.

Compressed or Non-linear Audio Data

The S/PDIF receiver processes compressed as well as non-linear audio data according to the supported standards. The following sections describe how this peripheral handles different data.

MPEG-2, AC-3, DTS, and AAC compressed data may be transmitted without setting either the `SPDIF_RX_STAT.VALID` bit or bit 1 of byte 0. To detect this data, the IEC61937 and SPMTE 337M standards dictate that there be a 96-bit sync code in the 16-, 20- or 24-bit audio data stream. This sync code consists of four words of zeros followed by a word consisting of 0xF872 and another word consisting of 0x4E1F. When this sync code is detected, the `SPDIF_RX_STAT.COMPMODE` bits hold the information regarding type of compression.

The last two words of the sync code, 0xF872 and 0x4E1F, are called the preamble-A and preamble-B of the burst preamble. Preamble-C of the burst preamble contains burst information and is captured and stored by the receiver. Preamble-D of the burst preamble contains the length code and is captured by the receiver. Even if the validity bit or bit 1 of byte 0 has been set, the receiver still looks for the sync code in order to record the preamble-C and D values. Once the sync code has not been detected in 4096 frames, the preamble-C and D registers are set to zero.

Emphasized Audio Data

Determination as to whether the received audio data is emphasized or not is done in software using the channel status bits as detailed below.

- In professional mode, (bit 0 of byte 0 = 1), channel status bits 2-4 of byte 0 indicate the audio data is emphasized if they are equal to 110 or 111.
- In consumer mode, (bit 0 of byte 0 = 0), channel status bits 3-5 indicate the audio data is emphasized if they are equal to 100, 010 or 110.

Single-Channel Double-Frequency Mode

Unlike previous processors, support for single-channel, double-frequency mode (SCDF) is not supported through specific bits within the `SPDIF_RX_CTL` register, but rather have to be implemented in software using the information provided by the CS (channel status) bits.

- 0111 = single channel double frequency mode
- 1000 = single channel double frequency mode-stereo left
- 1001 = single channel double frequency mode-stereo right

Clock Recovery Modes

The S/PDIF receiver extracts audio data, channel status, and user bits from the biphase encoded AES3 and S/PDIF stream. In addition, a 50% duty cycle reference clock running at the sampling rate of the audio input data is generated for the receiver to recover the oversampling clock.

Number Controlled Oscillator

The receiver can recover the clock from the biphasic encoded stream using an on-chip NCO shown in the following figure. Note the dedicated NCO is separate from the PLL that supplies the clock to the processor core and which is the default operation of the receiver.

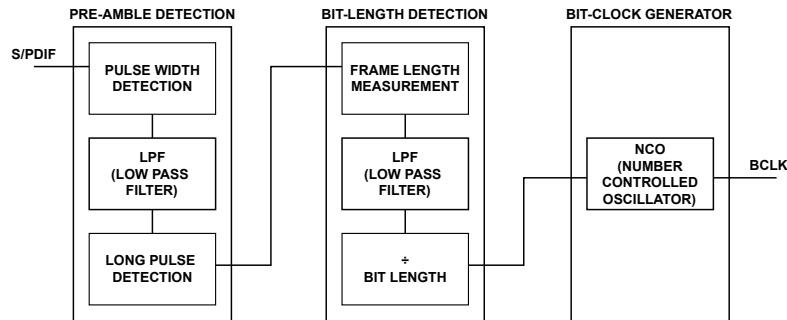


Figure 29-7: S/PDIF Clock Recovery Mechanism

The left/right frame reference clock for the NCO is generated using the preambles. The recovered low jitter left/right frame clock from the NCO attempts to align with the reference clock. However, this recovered left/right clock, like the reference clock, is not phase aligned with the preambles.

Interrupts

The following sections provide information about interrupt sources, masking and servicing.

Sources

The S/PDIF module of each DAI drives five interrupt signals. Four are status signals driven from SPDIFn_RX and one signal is driven from SPDIFn_TX (block start). These signals are connected into the [DAI_IRPTL_L/DAI_IRPTL_H](#) latch register.

Transmit Block Start

The SPDIFn_TX_BLKSTART output signal, if routed to any miscellaneous interrupt bits (DAIn_INT_31-22 in the [DAI_MISC0/DAI_MISC1](#) registers), triggers a block start interrupt during the last frame of current block.

Receiver Status

The following receiver status bits generate an interrupt.

- Validity (`SPDIF_RX_STAT.VALID`)
- Receiver locked (`SPDIF_RX_STAT.LOCK`)
- No audio (`SPDIF_RX_STAT.AUDIOTYPE`)

Receiver Error

The loss of lock (`SPDIF_RX_STAT.LOCKLOSS`) bit generates an interrupt.

Masking

For the S/PDIF receiver the `DAI_IMSK_RE` register must be unmasked accordingly. For the S/PDIF transmit the `DAIn_IMASK_x` register must be unmasked accordingly.

The `INTR_DAI_IRQH` and `INTR_DAI_IRQL` signals are routed to the SEC and GIC.

Service

The ISR reads the `DAI_IRPTL_H` and `DAI_IRPTL_L` registers to clear the interrupt request.

Programming Model

The following sections provide information on programming the transmitter and receiver.

Programming the Transmitter

Since the S/PDIF transmitter data input is not available to the core, programming the transmitter is as simple as: 1) connecting the SRU to the on-chip (serial ports or input data port) or off-chip (DAI pins) serial devices that provide the clock and data to be encoded, and 2) selecting the desired mode in the transmitter control register. This setup can be accomplished in three steps.

1. Connect the transmitter's four required input signals and one biphas encoded output in the SRU. The four input signals are the serial clock (`SPDIF_TX_CLK_I`), the serial frame sync (`SPDIF_TX_FS_I`), the serial data (`SPDIF_TX_DAT_I`), and the high frequency clock (`SPDIF_TX_HFCLK_I`) used for the encoding. The only output of the transmitter is `SPDIF_TX_O`.
2. If user bits are required, write 0x1 to the `SPDIF_TX_USRUPDT` register for the first block of transfer. Also route the `SPDIF_TX_BLK_START_O` signal to the `DAI_INT_31-22` (`DAI_IRPTLx` register). This generates interrupts during the last frame of the block (192), allowing changes of user bits for the next block.
3. Initialize the `SPDIF_TX_CTL` register to enable the data encoding.
4. Manually set the block start bit in the data stream once per block (every 384 words). This is necessary if automatic generation of block start information is not enabled using the `SPDIF_TX_CTL.AUTO` bit = 0.

NOTE: For more information, see the "DAI Routing Capabilities" section of the *Digital Audio Interface (DAI)* chapter.

Programming the Receiver

Since the S/PDIF receiver data output is not available to the core, programming the peripheral is as simple as connecting the SRU to the on-chip (serial ports) or off-chip (DAI pins) serial devices that provide the clock and data to be decoded, and selecting the desired mode in the receiver control register. This setup can be accomplished in two steps.

1. Connect the input signal and three output signals in the SRU. The only input of the receiver is the biphas encoded stream, `SPDIFn_RX_I`. The three required output signals are the serial clock (`SPDIFn_RX_CLK`),

the serial frame sync (SPDIFn_RX_FS), and the serial data (SPDIFn_RX_DAT). The high frequency clock (SPDIFn_RX_TDMCLK) derived from the encoded stream is also available if the system requires it.

2. Initialize the `SPDIF_RX_CTL` register to enable the data decoding. Note that this peripheral is disabled by default.

NOTE: For more information, see the "DAI Routing Capabilities" section of the *Digital Audio Interface (DAI)* chapter.

Interrupted Data Streams on the Receiver

When using the S/PDIF receiver with data streams that are likely to be interrupted, (in other words unplugged and reconnected), it is necessary to take some extra steps to ensure that the S/PDIF receiver's digital PLL will relock to the stream. The steps to accomplish this are described below.

1. Set up interrupts within the DAI so that the S/PDIF receiver can generate an interrupt when the stream is reconnected.
2. Within the interrupt service routine (ISR), stop and restart the NCO. This is accomplished by setting and then clearing the `SPDIF_RX_CTL.RST` bit.
3. Return from the ISR and continue normal operation.

This method of resetting the NCO has been shown to provide extremely reliable performance when the S/PDIF inputs are interrupted or unplugged momentarily.

Debug Features

The following feature supports S/PDIF debugging.

Loopback Routing

The S/PDIF supports an internal loopback mode by using the SRU. For more information about loopback, see "Loopback Routing" in the the *Digital Audio Interface (DAI)* chapter.

ADSP-2159x SPDIF Register Descriptions

The S/PDIF module (SPDIF) contains the following registers.

Table 29-4: ADSP-2159x SPDIF Register List

Name	Description
<code>SPDIF_RX_CTL</code>	Receive Control
<code>SPDIF_RX_STAT</code>	Receive Status Register
<code>SPDIF_RX_STAT0_A</code>	Receive Status A0 Register
<code>SPDIF_RX_STAT0_B</code>	Receive Status B0 Register

Table 29-4: ADSP-2159x SPDIF Register List (Continued)

Name	Description
SPDIF_RX_STAT1_A	Receive Status A1 Register
SPDIF_RX_STAT1_B	Receive Status B1 Register
SPDIF_TX_CTL	Transmit Control Register
SPDIF_TX_STAT_A0	Transmit Status A0 Register
SPDIF_TX_STAT_A1	Transmit Status A1 Register
SPDIF_TX_STAT_A2	Transmit Status A2 Register
SPDIF_TX_STAT_A3	Transmit Status A3 Register
SPDIF_TX_STAT_A4	Transmit Status A4 Register
SPDIF_TX_STAT_A5	Transmit Status A5 Register
SPDIF_TX_STAT_B0	Transmit Status B0 Register
SPDIF_TX_STAT_B1	Transmit Status B1 Register
SPDIF_TX_STAT_B2	Transmit Status B2 Register
SPDIF_TX_STAT_B3	Transmit Status B3 Register
SPDIF_TX_STAT_B4	Transmit Status B4 Register
SPDIF_TX_STAT_B5	Transmit Status B5 Register
SPDIF_TX_UBUFF_A0	Transmit User Buffer A0 Register
SPDIF_TX_UBUFF_A1	Transmit User Buffer A1 Register
SPDIF_TX_UBUFF_A2	Transmit User Buffer A2 Register
SPDIF_TX_UBUFF_A3	Transmit User Buffer A3 Register
SPDIF_TX_UBUFF_A4	Transmit User Buffer A4 Register
SPDIF_TX_UBUFF_A5	Transmit User Buffer A5 Register
SPDIF_TX_UBUFF_B0	Transmit User Buffer B0 Register
SPDIF_TX_UBUFF_B1	Transmit User Buffer B1 Register
SPDIF_TX_UBUFF_B2	Transmit User Buffer B2 Register
SPDIF_TX_UBUFF_B3	Transmit User Buffer B3 Register
SPDIF_TX_UBUFF_B4	Transmit User Buffer B4 Register
SPDIF_TX_UBUFF_B5	Transmit User Buffer B5 Register
SPDIF_TX_USRUPDT	User Bit Update Register

Receive Control

The `SPDIF_RX_CTL` register is used to enable and control the S/PDIF receiver.

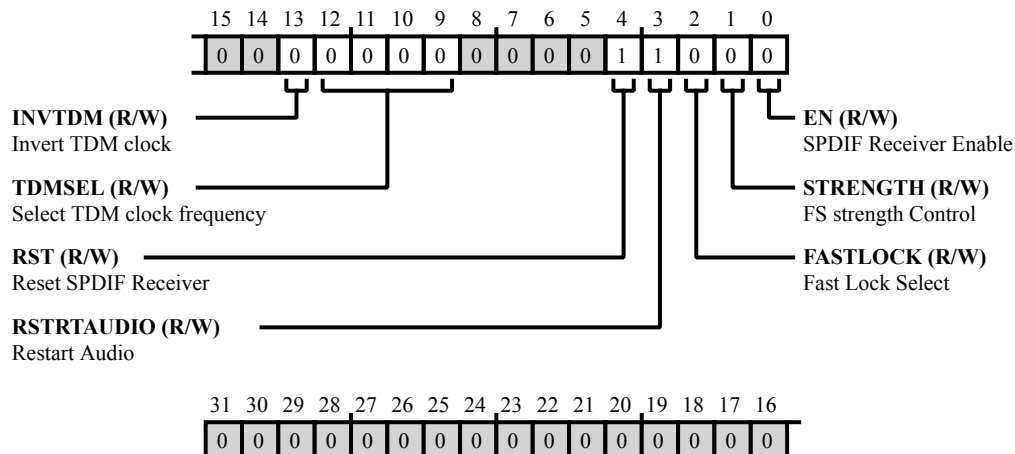


Figure 29-8: SPDIF_RX_CTL Register Diagram

Table 29-5: SPDIF_RX_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/W)	INVTDM	Invert TDM clock. When set to 0, inverted TDM clock or non-inverted Bit clock is selected. When set to 1, non-inverted TDM clock or inverted Blit clock is selected.
12:9 (R/W)	TDMSEL	Select TDM clock frequency. Select TDM clock frequency
4 (R/W)	RST	Reset SPDIF Receiver. The <code>SPDIF_RX_CTL.RST</code> bit resets the receiver.
		0 Takes the receiver out of reset
		1 Resets the SPDIF receiver
3 (R/W)	RSTRTAUDIO	Restart Audio. The <code>SPDIF_RX_CTL.RSTRTAUDIO</code> bit restarts the audio once a re-lock has occurred. When the S/PDIF receiver loses lock the audio output is set to 0. This bit determines the behavior of the audio once lock is re-established. Audio can be manually restarted by toggling this bit high and then low.
		0 Manually restart audio
		1 Automatically restart audio

Table 29-5: SPDIF_RX_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	FASTLOCK	Fast Lock Select. The SPDIF_RX_CTL.FASTLOCK bit allows the lock mechanism to lock at normal speed or at faster speed. This has the advantage of recovering very quickly whenever the S/PDIF receiver loses lock due to glitches in the signal. In normal mode the S/PDIF receiver locks after 64 consecutive valid samples, in fast mode the S/PDIF receiver locks after 8 consecutive valid samples
		0 Enable normal mode
		1 Enable fast mode
1 (R/W)	STRENGTH	FS strength Control. The SPDIF_RX_CTL.STRENGTH bit controls the strength of the bit clock and Frame sync outputs from the SPDIF receiver. In strong mode these output signals are continued (as best possible) when the receiver notices a loss-of-lock condition. Note that 'as best possible' refers to the fact that this recovered signal may not be accurate, given the loss-of-lock condition. In weak mode these output signals are interrupted as soon as the receiver notices a loss-of-lock condition.
		0 Enable strong mode
		1 Enable weak mode
0 (R/W)	EN	SPDIF Receiver Enable. When the SPDIF_RX_CTL.EN bit =0 the clock to SPDIF is switched off for power savings.
		0 Disable receiver
		1 Enable receiver

Receive Status Register

The `SPDIF_RX_STAT` register consists of bits that indicate the status of various functions supported by S/PDIF receiver.

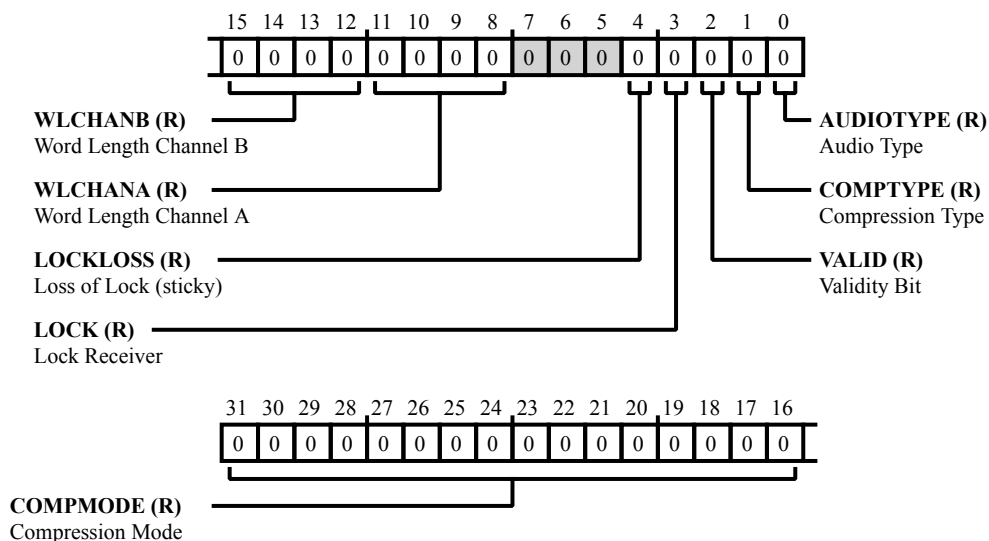


Figure 29-9: SPDIF_RX_STAT Register Diagram

Table 29-6: SPDIF_RX_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/NW)	COMPMODE	Compression Mode. The <code>SPDIF_RX_STAT.COMPMODE</code> bit field indicates the type of compression mode used in the Digital audio stream. The value in this field indicates the 16-bit burst information as specified by the IEC 62937-2 standard. Use this document to decode the value in this bit field.
15:12 (R/NW)	WLCHANB	Word Length Channel B. The <code>SPDIF_RX_STAT.WLCHANB</code> bit field indicates the S/PDIF word length for channel B. May be decoded as follows (from the S/PDIF standard).
	0	Reserved
	1	Reserved
	2	SPDIF_LENGTH_16
	3	Reserved
	4	SPDIF_LENGTH_18
	5	SPDIF_LENGTH_22
	6	Reserved

Table 29-6: SPDIF_RX_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		7	Reserved
		8	SPDIF_LENGTH_19
		9	SPDIF_LENGTH_23
		10	SPDIF_LENGTH_20
		11	SPDIF_LENGTH_24
		12	SPDIF_LENGTH_17
		13	SPDIF_LENGTH_21
		14	Reserved
		15	Reserved
11:8 (R/NW)	WLCHANA	Word Length Channel A. The SPDIF_RX_STAT.WLCHANA bit indicates the S/PDIF word length for channel A. May be decoded as follows (from the S/PDIF standard).	
		0	Reserved
		1	Reserved
		2	SPDIF_LENGTH_16
		3	Reserved
		4	SPDIF_LENGTH_18
		5	SPDIF_LENGTH_22
		6	Reserved
		7	Reserved
		8	SPDIF_LENGTH_19
		9	SPDIF_LENGTH_23
		10	SPDIF_LENGTH_20
		11	SPDIF_LENGTH_24
		12	SPDIF_LENGTH_17
		13	SPDIF_LENGTH_21
		14	Reserved
		15	Reserved

Table 29-6: SPDIF_RX_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/NW)	LOCKLOSS	Loss of Lock (sticky). The <code>SPDIF_RX_STAT.LOCKLOSS</code> bit indicates that the system has lost lock. This is different to the lock register, as it is sticky it goes high when system loses lock, but returns to low once <code>SPDIF_RX_CTL.RSTRTAUDIO</code> bit is toggled. This is to allow the programs to poll the lock status.
		0 SPDIF receiver locked
		1 SPDIF Receiver lost lock
3 (R/NW)	LOCK	Lock Receiver. The <code>SPDIF_RX_STAT.LOCK</code> bit indicates the S/PDIF receiver has successfully locked to the S/PDIF stream and is outputting valid data.
		0 Receiver not locked
		1 Receiver locked
2 (R/NW)	VALID	Validity Bit. The <code>SPDIF_RX_STAT.VALID</code> bit indicates the ORed bits of channel A and B.
		0 Linear PCM data
		1 Non-linear audio data
1 (R/NW)	COMPTYPE	Compression Type. The <code>SPDIF_RX_STAT.COMPTYPE</code> bit indicates AC3 or DTS compression. Valid only if <code>SPDIF_RX_STAT.AUDIOTYPE</code> indicates compressed data.
		0 AC3 compressed data
		1 DTS compressed data
0 (R/NW)	AUDIOTYPE	Audio Type. The <code>SPDIF_RX_STAT.AUDIOTYPE</code> bit indicates PCM or compressed data.
		0 PCM data
		1 Compressed Data

Receive Status A0 Register

The `SPDIF_RX_STAT0_A` register holds the receive channel 0 status for bytes 0-3 for sub frame A.

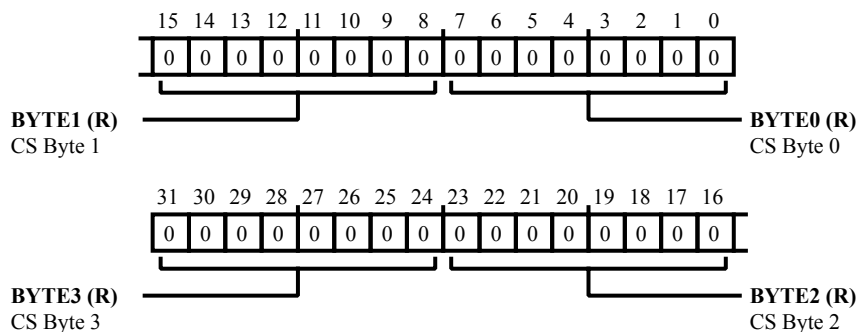


Figure 29-10: SPDIF_RX_STAT0_A Register Diagram

Table 29-7: SPDIF_RX_STAT0_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	BYTE3	CS Byte 3. The <code>SPDIF_RX_STAT0_A.BYTE3</code> bit field contains byte 3 of received channel A status.
23:16 (R/NW)	BYTE2	CS Byte 2. The <code>SPDIF_RX_STAT0_A.BYTE2</code> bit field contains byte 2 of received channel A status.
15:8 (R/NW)	BYTE1	CS Byte 1. The <code>SPDIF_RX_STAT0_A.BYTE1</code> bit field contains byte 1 of received channel A status.
7:0 (R/NW)	BYTE0	CS Byte 0. The <code>SPDIF_RX_STAT0_A.BYTE0</code> bit field contains the status of byte 0-3 of received channel A.

Receive Status B0 Register

The `SPDIF_RX_STAT0_B` register holds the receive channel 0 status for bytes 0-3 for sub frame B.

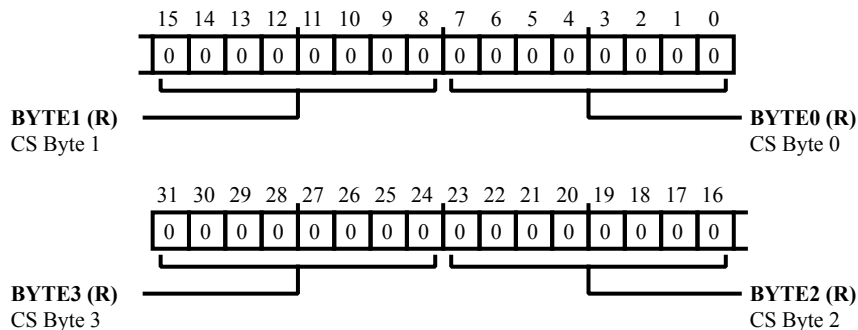


Figure 29-11: SPDIF_RX_STAT0_B Register Diagram

Table 29-8: SPDIF_RX_STAT0_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	BYTE3	CS Byte 3. The <code>SPDIF_RX_STAT0_B.BYTE3</code> bit field contains byte 3 of received channel B status.
23:16 (R/NW)	BYTE2	CS Byte 2. The <code>SPDIF_RX_STAT0_B.BYTE2</code> bit field contains byte 2 of received channel B status.
15:8 (R/NW)	BYTE1	CS Byte 1. The <code>SPDIF_RX_STAT0_B.BYTE1</code> bit field contains byte 1 of received channel B status.
7:0 (R/NW)	BYTE0	CS Byte 0. The <code>SPDIF_RX_STAT0_B.BYTE0</code> bit field contains byte 0 of received channel B status.

Receive Status A1 Register

The `SPDIF_RX_STAT1_A` register holds the receive channel 1 status for byte 4 for sub frame A.

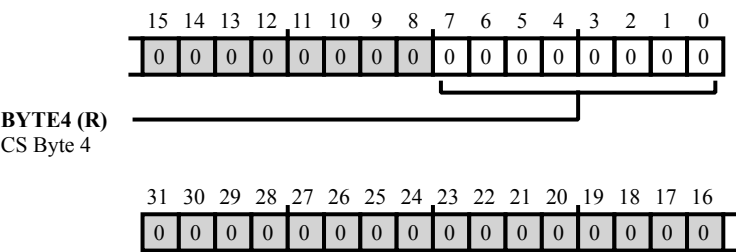


Figure 29-12: SPDIF_RX_STAT1_A Register Diagram

Table 29-9: SPDIF_RX_STAT1_A Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	BYTE4	CS Byte 4. The <code>SPDIF_RX_STAT1_A.BYTE4</code> bit field contains byte 4 of received channel A status.

Receive Status B1 Register

The `SPDIF_RX_STAT1_B` register holds the receive channel 1 status for byte 4 for sub frame B.

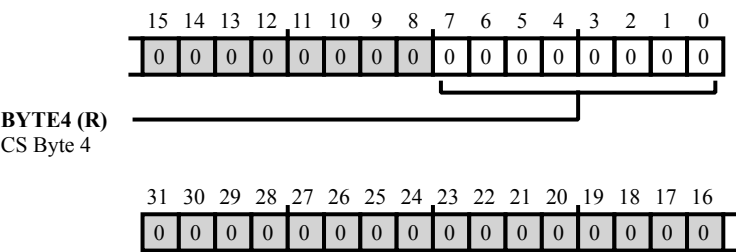


Figure 29-13: SPDIF_RX_STAT1_B Register Diagram

Table 29-10: SPDIF_RX_STAT1_B Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	BYTE4	CS Byte 4. The <code>SPDIF_RX_STAT1_B.BYTE4</code> bit field contains byte 4 of received channel B status.

Transmit Control Register

The `SPDIF_TX_CTL` register provides bits to enable/disable the transmitter and configure several options related to data transmission.

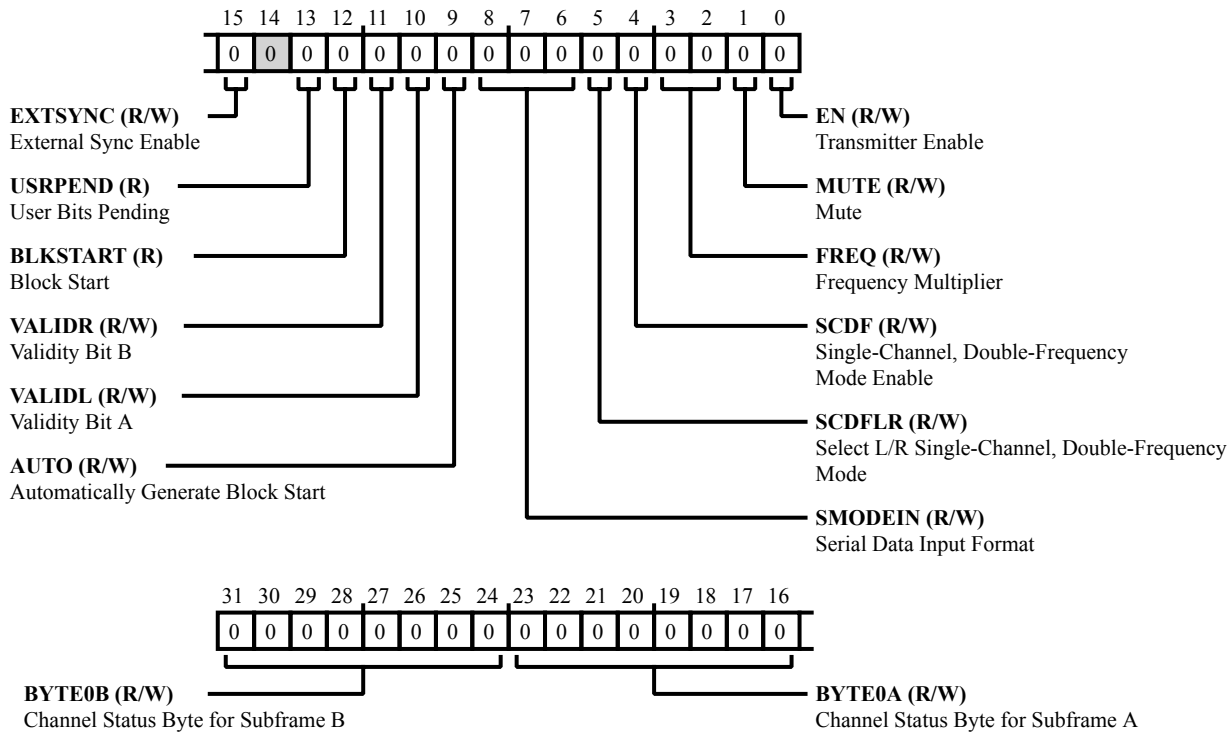


Figure 29-14: SPDIF_TX_CTL Register Diagram

Table 29-11: SPDIF_TX_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE0B	Channel Status Byte for Subframe B. The <code>SPDIF_TX_CTL.BYTE0B</code> bit field contains the channel status for the second bytes 95.
23:16 (R/W)	BYTE0A	Channel Status Byte for Subframe A. The <code>SPDIF_TX_CTL.BYTE0A</code> bit field contains the channel status for the first bytes 40.
15 (R/W)	EXTSYNC	External Sync Enable. When the <code>SPDIF_TX_CTL.EXTSYNC</code> bit is set (regardless of the <code>SPDIF_TX_CTL.AUTO</code> bit setting) the internal frame counter is set to zero at an internal LRCLK rising edge followed by an <code>EXTSYNC_I</code> rising edge.

Table 29-11: SPDIF_TX_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
13 (R/NW)	USRPEND	User Bits Pending. The SPDIF_TX_CTL.USRPEND bit is set if the update of the internal buffer from the Transmit User Bits Buffer registers has not yet completed.
12 (R/NW)	BLKSTART	Block Start. The SPDIF_TX_CTL.BLKSTART bit is a status bit that indicates block start (when the SPDIF_TX_CTL.AUTO bit = 1).
		0 Current word is not block start
		1 Current word is block start
11 (R/W)	VALIDR	Validity Bit B. Use the SPDIF_TX_CTL.VALIDR bit with the channel status buffer.
10 (R/W)	VALIDL	Validity Bit A. The SPDIF_TX_CTL.VALIDL bit either manually start block transfer according to input stream status bits or automatically start block transfer. Use the SPDIF_TX_CTL.VALIDL bit with the channel status buffer.
		0 Manually start block transfer
		1 Automatically start block transfer
9 (R/W)	AUTO	Automatically Generate Block Start. When enabled, the transmitter is in standalone mode where it inserts block start, channel status, and validity bits on its own. If the channel status or validity buffer needs to be enabled (after the DAI programming is complete), first write to the buffers with the required data and then enable the buffers by setting the SPDIF_TX_CTL.AUTO bit.
8:6 (R/W)	SMODEIN	Serial Data Input Format. The SPDIF_TX_CTL.SMODEIN bit selects the data input format.
		0 Left-justified
		1 I2S
		2 Reserved
		3 Reserved
		4 Right-justified, 24 bits
		5 Right-justified, 20 bits
		6 Right-justified, 18 bits
		7 Right-justified, 16 bits

Table 29-11: SPDIF_TX_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SCDFLR	Select L/R Single-Channel, Double-Frequency Mode. The <code>SPDIF_TX_CTL.SCDFLR</code> bit selects the left or right channel in SCDF mode.
		0 Left channel
		1 Right channel
4 (R/W)	SCDF	Single-Channel, Double-Frequency Mode Enable. The <code>SPDIF_TX_CTL.SCDF</code> bit enables single-channel, double-frequency mode.
		0 Two-channel mode
		1 SCDF mode
3:2 (R/W)	FREQ	Frequency Multiplier. The <code>SPDIF_TX_CTL.FREQ</code> bit field sets the oversampling ratio.
		0 256 x frame sync oversampling
		1 384 x frame sync oversampling
		2-3 Reserved
1 (R/W)	MUTE	Mute. The <code>SPDIF_TX_CTL.MUTE</code> bit mutes the serial data output.
		0 Disable Mute
		1 Enable Mute
0 (R/W)	EN	Transmitter Enable. The <code>SPDIF_TX_CTL.EN</code> bit enables the transmitter and resets the control registers to their defaults.
		0 Transmitter disabled
		1 Transmitter enabled

Transmit Status A0 Register

The `SPDIF_TX_STAT_A0` register holds the transmit channel 0 status for bytes 1-4 for sub frame A.

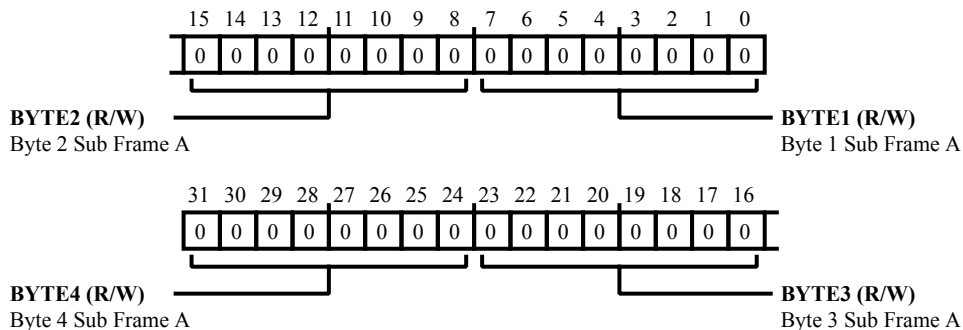


Figure 29-15: SPDIF_TX_STAT_A0 Register Diagram

Table 29-12: SPDIF_TX_STAT_A0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE4	Byte 4 Sub Frame A. The <code>SPDIF_TX_STAT_A0.BYTE4</code> bit field holds the transmit channel 0 status for byte 4 for sub frame A.
23:16 (R/W)	BYTE3	Byte 3 Sub Frame A. The <code>SPDIF_TX_STAT_A0.BYTE3</code> bit field holds the transmit channel 0 status for byte 3 for sub frame A.
15:8 (R/W)	BYTE2	Byte 2 Sub Frame A. The <code>SPDIF_TX_STAT_A0.BYTE2</code> bit field holds the transmit channel 0 status for byte 2 for sub frame A.
7:0 (R/W)	BYTE1	Byte 1 Sub Frame A. The <code>SPDIF_TX_STAT_A0.BYTE1</code> bit field holds the transmit channel 0 status for byte 1 for sub frame A.

Transmit Status A1 Register

The `SPDIF_TX_STAT_A1` register holds the transmit status for bytes 5-8 for sub frame A.

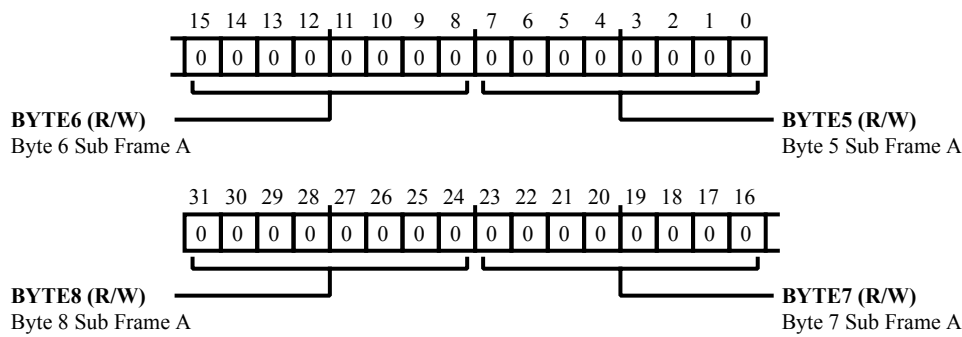


Figure 29-16: SPDIF_TX_STAT_A1 Register Diagram

Table 29-13: SPDIF_TX_STAT_A1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE8	Byte 8 Sub Frame A. The <code>SPDIF_TX_STAT_A1</code> . <code>BYTE8</code> bit field contains transmit status for byte 8 of sub frame A.
23:16 (R/W)	BYTE7	Byte 7 Sub Frame A. The <code>SPDIF_TX_STAT_A1</code> . <code>BYTE7</code> bit field contains transmit status for byte 7 of sub frame A.
15:8 (R/W)	BYTE6	Byte 6 Sub Frame A. The <code>SPDIF_TX_STAT_A1</code> . <code>BYTE6</code> bit field contains transmit status for byte 6 of sub frame A.
7:0 (R/W)	BYTE5	Byte 5 Sub Frame A. The <code>SPDIF_TX_STAT_A1</code> . <code>BYTE5</code> bit field contains transmit status for byte 5 of sub frame A.

Transmit Status A2 Register

The `SPDIF_TX_STAT_A2` register holds the transmit status for bytes 9-12 for sub frame A.

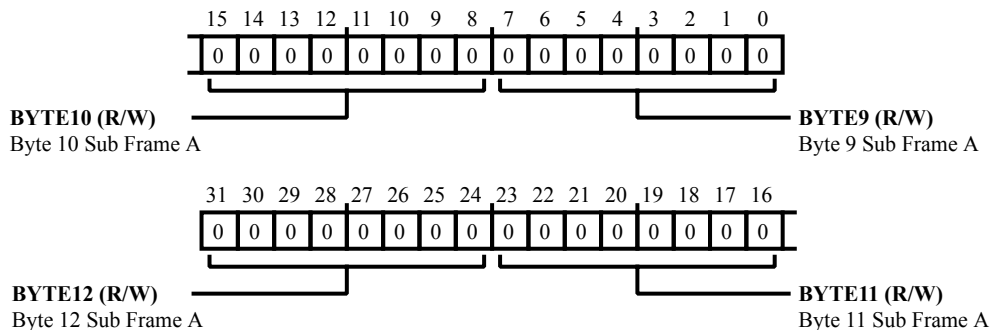


Figure 29-17: SPDIF_TX_STAT_A2 Register Diagram

Table 29-14: SPDIF_TX_STAT_A2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE12	Byte 12 Sub Frame A. The <code>SPDIF_TX_STAT_A2.BYTE12</code> bit field contains transmit status for byte 12 of sub frame A.
23:16 (R/W)	BYTE11	Byte 11 Sub Frame A. The <code>SPDIF_TX_STAT_A2.BYTE11</code> bit field contains transmit status for byte 11 of sub frame A.
15:8 (R/W)	BYTE10	Byte 10 Sub Frame A. The <code>SPDIF_TX_STAT_A2.BYTE10</code> bit field contains transmit status for byte 10 of sub frame A.
7:0 (R/W)	BYTE9	Byte 9 Sub Frame A. The <code>SPDIF_TX_STAT_A2.BYTE9</code> bit field contains transmit status for byte 9 of sub frame A.

Transmit Status A3 Register

The `SPDIF_TX_STAT_A3` register holds the transmit status for bytes 13-16 for sub frame A.

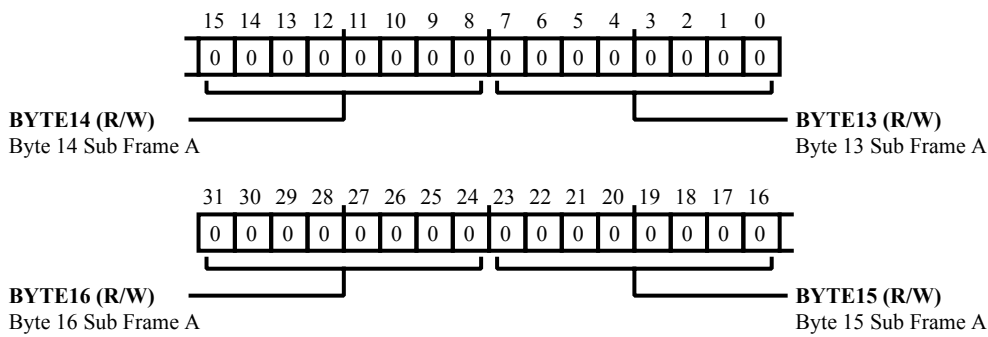


Figure 29-18: SPDIF_TX_STAT_A3 Register Diagram

Table 29-15: SPDIF_TX_STAT_A3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE16	Byte 16 Sub Frame A. The <code>SPDIF_TX_STAT_A3.BYTE16</code> bit field contains transmit status for byte 16 of sub frame A.
23:16 (R/W)	BYTE15	Byte 15 Sub Frame A. The <code>SPDIF_TX_STAT_A3.BYTE15</code> bit field contains transmit status for byte 15 of sub frame A.
15:8 (R/W)	BYTE14	Byte 14 Sub Frame A. The <code>SPDIF_TX_STAT_A3.BYTE14</code> bit field contains transmit status for byte 14 of sub frame A.
7:0 (R/W)	BYTE13	Byte 13 Sub Frame A. The <code>SPDIF_TX_STAT_A3.BYTE13</code> bit field contains transmit status for byte 13 of sub frame A.

Transmit Status A4 Register

The `SPDIF_TX_STAT_A4` register holds the transmit status for bytes 17-20 for sub frame A.

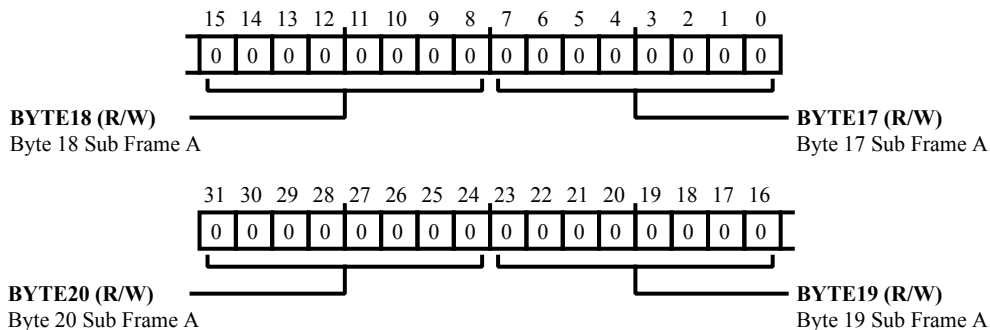


Figure 29-19: SPDIF_TX_STAT_A4 Register Diagram

Table 29-16: SPDIF_TX_STAT_A4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE20	Byte 20 Sub Frame A. The <code>SPDIF_TX_STAT_A4.BYTE20</code> bit field contains transmit status for byte 20 of sub frame A.
23:16 (R/W)	BYTE19	Byte 19 Sub Frame A. The <code>SPDIF_TX_STAT_A4.BYTE19</code> bit field contains transmit status for byte 19 of sub frame A.
15:8 (R/W)	BYTE18	Byte 18 Sub Frame A. The <code>SPDIF_TX_STAT_A4.BYTE18</code> bit field contains transmit status for byte 18 of sub frame A.
7:0 (R/W)	BYTE17	Byte 17 Sub Frame A. The <code>SPDIF_TX_STAT_A4.BYTE17</code> bit field contains transmit status for byte 17 of sub frame A.

Transmit Status A5 Register

The `SPDIF_TX_STAT_A5` register holds the transmit status for bytes 21-23 for sub frame A.

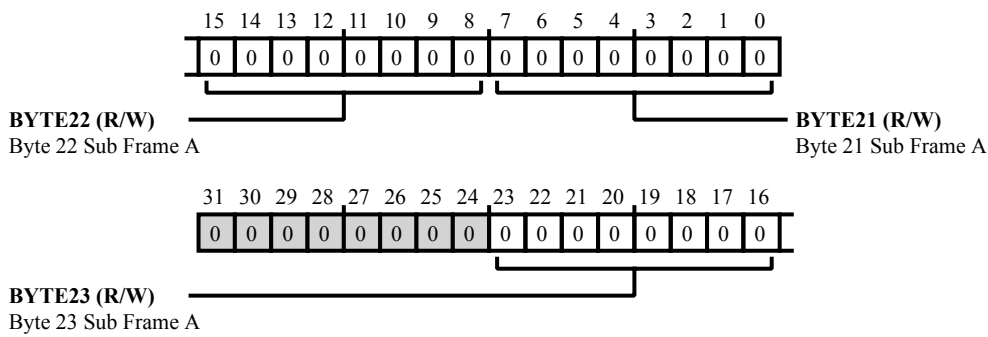


Figure 29-20: `SPDIF_TX_STAT_A5` Register Diagram

Table 29-17: `SPDIF_TX_STAT_A5` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	BYTE23	Byte 23 Sub Frame A. The <code>SPDIF_TX_STAT_A5.BYTE23</code> bit field contains transmit status for byte 23 of sub frame A.
15:8 (R/W)	BYTE22	Byte 22 Sub Frame A. The <code>SPDIF_TX_STAT_A5.BYTE22</code> bit field contains transmit status for byte 22 of sub frame A.
7:0 (R/W)	BYTE21	Byte 21 Sub Frame A. The <code>SPDIF_TX_STAT_A5.BYTE21</code> bit field contains transmit status for byte 21 of sub frame A.

Transmit Status B0 Register

The `SPDIF_TX_STAT_B0` register holds the transmit channel 0 status for bytes 1-4 for sub frame B.

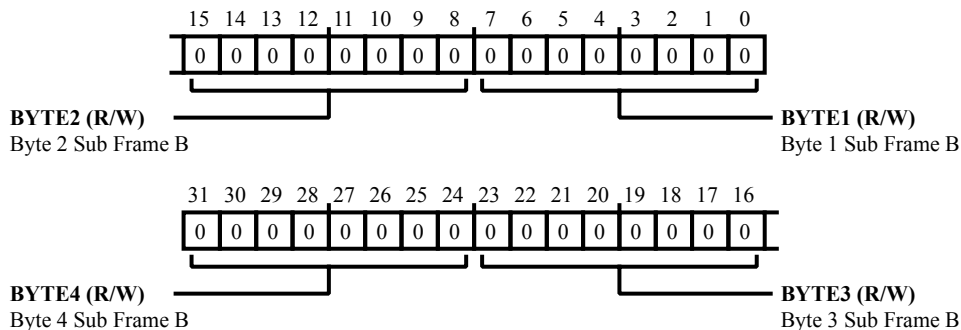


Figure 29-21: `SPDIF_TX_STAT_B0` Register Diagram

Table 29-18: `SPDIF_TX_STAT_B0` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE4	Byte 4 Sub Frame B. The <code>SPDIF_TX_STAT_B0.BYTE4</code> bit field holds the transmit channel 0 status for byte 4 for sub frame B.
23:16 (R/W)	BYTE3	Byte 3 Sub Frame B. The <code>SPDIF_TX_STAT_B0.BYTE3</code> bit field holds the transmit channel 0 status for byte 3 for sub frame B.
15:8 (R/W)	BYTE2	Byte 2 Sub Frame B. The <code>SPDIF_TX_STAT_B0.BYTE2</code> bit field holds the transmit channel 0 status for byte 2 for sub frame B.
7:0 (R/W)	BYTE1	Byte 1 Sub Frame B. The <code>SPDIF_TX_STAT_B0.BYTE1</code> bit field holds the transmit channel 0 status for byte 1 for sub frame B.

Transmit Status B1 Register

The `SPDIF_TX_STAT_B1` register holds the transmit status for bytes 5-8 for sub frame B.

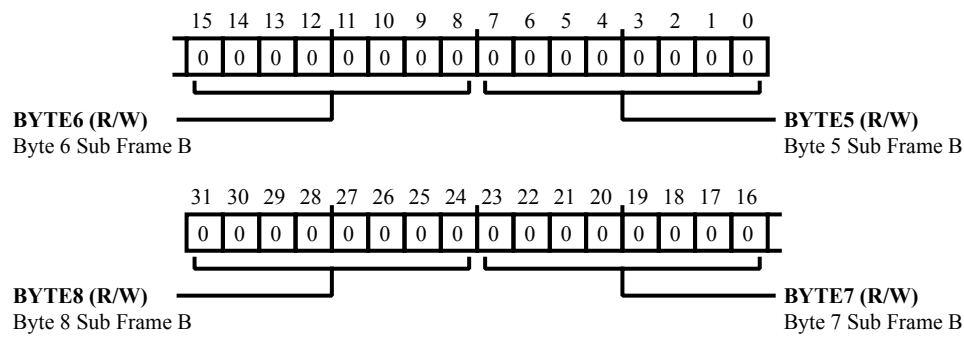


Figure 29-22: `SPDIF_TX_STAT_B1` Register Diagram

Table 29-19: `SPDIF_TX_STAT_B1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE8	Byte 8 Sub Frame B. The <code>SPDIF_TX_STAT_B1</code> . <code>BYTE8</code> bit field contains transmit status for byte 8 of sub frame B.
23:16 (R/W)	BYTE7	Byte 7 Sub Frame B. The <code>SPDIF_TX_STAT_B1</code> . <code>BYTE7</code> bit field contains transmit status for byte 7 of sub frame B.
15:8 (R/W)	BYTE6	Byte 6 Sub Frame B. The <code>SPDIF_TX_STAT_B1</code> . <code>BYTE6</code> bit field contains transmit status for byte 6 of sub frame B.
7:0 (R/W)	BYTE5	Byte 5 Sub Frame B. The <code>SPDIF_TX_STAT_B1</code> . <code>BYTE5</code> bit field contains transmit status for byte 5 of sub frame B.

Transmit Status B2 Register

The `SPDIF_TX_STAT_B2` register holds the transmit status for bytes 9-12 for sub frame B.

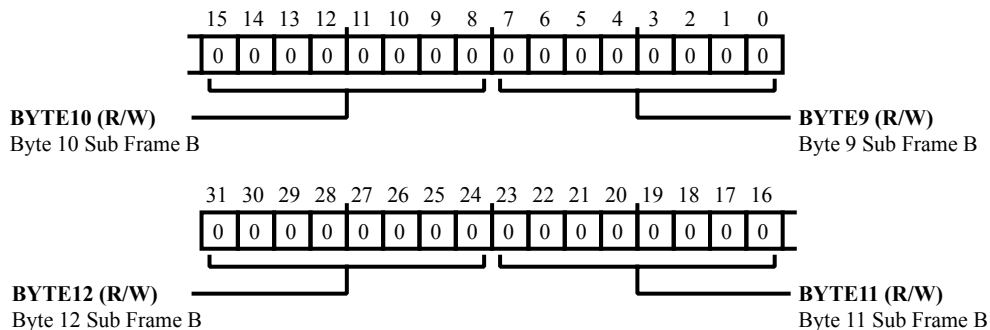


Figure 29-23: `SPDIF_TX_STAT_B2` Register Diagram

Table 29-20: `SPDIF_TX_STAT_B2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE12	Byte 12 Sub Frame B. The <code>SPDIF_TX_STAT_B2.BYTE12</code> bit field contains transmit status for byte 12 of sub frame B.
23:16 (R/W)	BYTE11	Byte 11 Sub Frame B. The <code>SPDIF_TX_STAT_B2.BYTE11</code> bit field contains transmit status for byte 11 of sub frame B.
15:8 (R/W)	BYTE10	Byte 10 Sub Frame B. The <code>SPDIF_TX_STAT_B2.BYTE10</code> bit field contains transmit status for byte 10 of sub frame B.
7:0 (R/W)	BYTE9	Byte 9 Sub Frame B. The <code>SPDIF_TX_STAT_B2.BYTE9</code> bit field contains transmit status for byte 9 of sub frame B.

Transmit Status B3 Register

The `SPDIF_TX_STAT_B3` register holds the transmit status for bytes 13-16 for sub frame B.

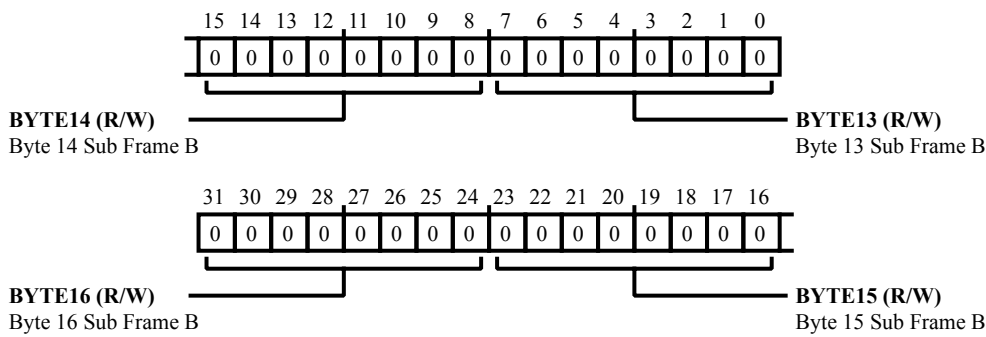


Figure 29-24: SPDIF_TX_STAT_B3 Register Diagram

Table 29-21: SPDIF_TX_STAT_B3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE16	Byte 16 Sub Frame B. The <code>SPDIF_TX_STAT_B3.BYTE16</code> bit field contains transmit status for byte 16 of sub frame B.
23:16 (R/W)	BYTE15	Byte 15 Sub Frame B. The <code>SPDIF_TX_STAT_B3.BYTE15</code> bit field contains transmit status for byte 15 of sub frame B.
15:8 (R/W)	BYTE14	Byte 14 Sub Frame B. The <code>SPDIF_TX_STAT_B3.BYTE14</code> bit field contains transmit status for byte 14 of sub frame B.
7:0 (R/W)	BYTE13	Byte 13 Sub Frame B. The <code>SPDIF_TX_STAT_B3.BYTE13</code> bit field contains transmit status for byte 13 of sub frame B.

Transmit Status B4 Register

The `SPDIF_TX_STAT_B4` register holds the transmit status for bytes 17-20 for sub frame B.

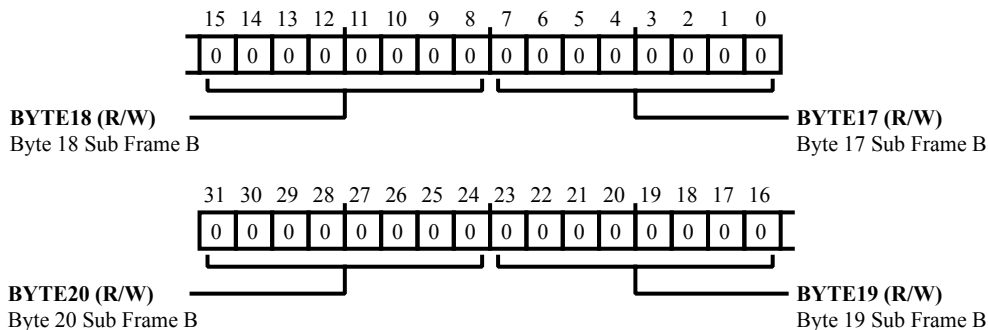


Figure 29-25: `SPDIF_TX_STAT_B4` Register Diagram

Table 29-22: `SPDIF_TX_STAT_B4` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE20	Byte 20 Sub Frame B. The <code>SPDIF_TX_STAT_B4.BYTE20</code> bit field contains transmit status for byte 20 of sub frame B.
23:16 (R/W)	BYTE19	Byte 19 Sub Frame B. The <code>SPDIF_TX_STAT_B4.BYTE19</code> bit field contains transmit status for byte 19 of sub frame B.
15:8 (R/W)	BYTE18	Byte 18 Sub Frame B. The <code>SPDIF_TX_STAT_B4.BYTE18</code> bit field contains transmit status for byte 18 of sub frame B.
7:0 (R/W)	BYTE17	Byte 17 Sub Frame B. The <code>SPDIF_TX_STAT_B4.BYTE17</code> bit field contains transmit status for byte 17 of sub frame B.

Transmit Status B5 Register

The `SPDIF_TX_STAT_B5` register holds the transmit status for bytes 21-23 for sub frame B.

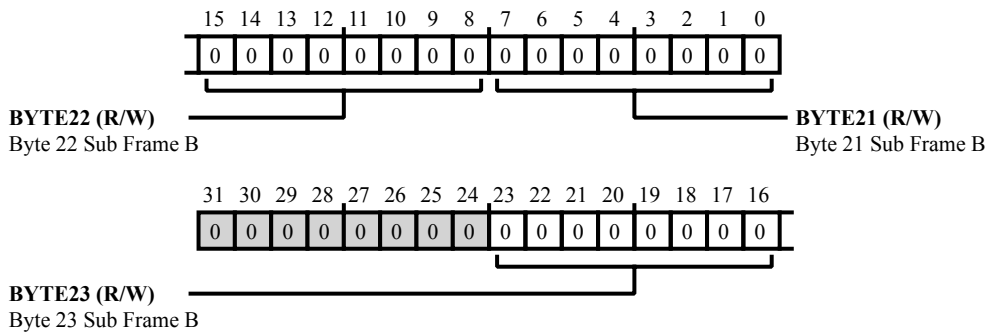


Figure 29-26: `SPDIF_TX_STAT_B5` Register Diagram

Table 29-23: `SPDIF_TX_STAT_B5` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	BYTE23	Byte 23 Sub Frame B. The <code>SPDIF_TX_STAT_B5.BYTE23</code> bit field contains transmit status for byte 23 of sub frame B.
15:8 (R/W)	BYTE22	Byte 22 Sub Frame B. The <code>SPDIF_TX_STAT_B5.BYTE22</code> bit field contains transmit status for byte 22 of sub frame B.
7:0 (R/W)	BYTE21	Byte 21 Sub Frame B. The <code>SPDIF_TX_STAT_B5.BYTE21</code> bit field contains transmit status for byte 21 of sub frame B.

Transmit User Buffer A0 Register

The `SPDIF_TX_UBUFF_A0` register holds the transmit user buffer data for bytes 0-3 for sub frame A.

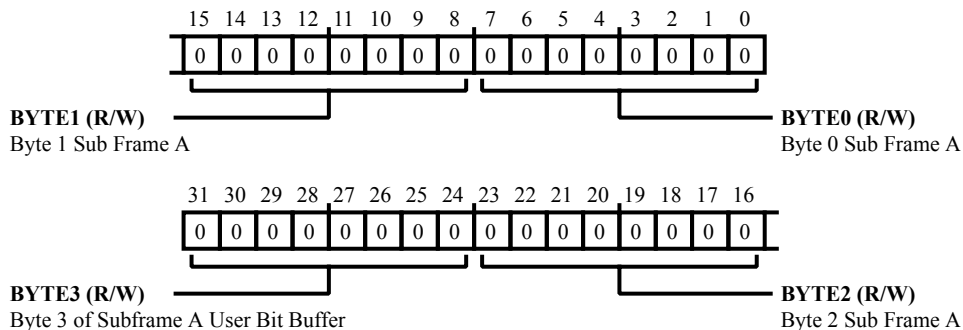


Figure 29-27: SPDIF_TX_UBUFF_A0 Register Diagram

Table 29-24: SPDIF_TX_UBUFF_A0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE3	Byte 3 of Subframe A User Bit Buffer. The <code>SPDIF_TX_UBUFF_A0.BYTE3</code> bit field contains user bit data for byte 0 of sub frame A.
23:16 (R/W)	BYTE2	Byte 2 Sub Frame A. The <code>SPDIF_TX_UBUFF_A0.BYTE2</code> bit field contains user bit data for byte 2 of sub frame A.
15:8 (R/W)	BYTE1	Byte 1 Sub Frame A. The <code>SPDIF_TX_UBUFF_A0.BYTE1</code> bit field contains user bit data for byte 1 of sub frame A.
7:0 (R/W)	BYTE0	Byte 0 Sub Frame A. The <code>SPDIF_TX_UBUFF_A0.BYTE0</code> bit field contains user bit data for byte 0 of sub frame A.

Transmit User Buffer A1 Register

The `SPDIF_TX_UBUFF_A1` register holds the transmit user buffer data for bytes 4-7 for sub frame A.

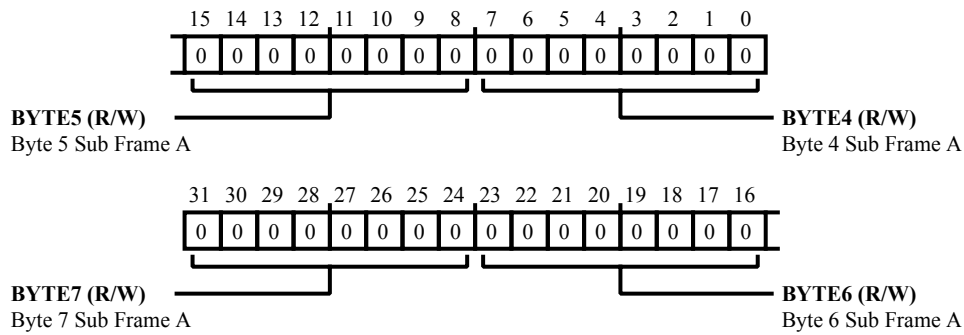


Figure 29-28: SPDIF_TX_UBUFF_A1 Register Diagram

Table 29-25: SPDIF_TX_UBUFF_A1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE7	Byte 7 Sub Frame A. The <code>SPDIF_TX_UBUFF_A1</code> . <code>BYTE7</code> bit field contains user bit data for byte 7 of sub frame A.
23:16 (R/W)	BYTE6	Byte 6 Sub Frame A. The <code>SPDIF_TX_UBUFF_A1</code> . <code>BYTE6</code> bit field contains user bit data for byte 6 of sub frame A.
15:8 (R/W)	BYTE5	Byte 5 Sub Frame A. The <code>SPDIF_TX_UBUFF_A1</code> . <code>BYTE5</code> bit field contains user bit data for byte 5 of sub frame A.
7:0 (R/W)	BYTE4	Byte 4 Sub Frame A. The <code>SPDIF_TX_UBUFF_A1</code> . <code>BYTE4</code> bit field contains user bit data for byte 0 of sub frame A.

Transmit User Buffer A2 Register

The `SPDIF_TX_UBUFF_A2` register holds the transmit user buffer data for bytes 8-11 for sub frame A.

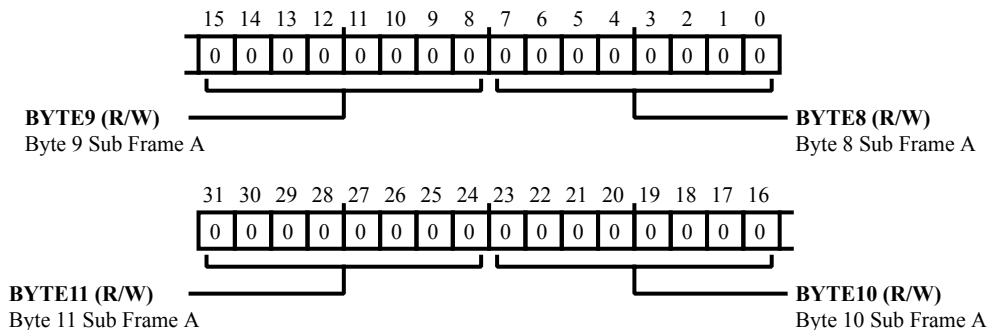


Figure 29-29: SPDIF_TX_UBUFF_A2 Register Diagram

Table 29-26: SPDIF_TX_UBUFF_A2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE11	Byte 11 Sub Frame A. The <code>SPDIF_TX_UBUFF_A2.BYTE11</code> bit field contains user bit data for byte 11 of sub frame A.
23:16 (R/W)	BYTE10	Byte 10 Sub Frame A. The <code>SPDIF_TX_UBUFF_A2.BYTE10</code> bit field contains user bit data for byte 10 of sub frame A.
15:8 (R/W)	BYTE9	Byte 9 Sub Frame A. The <code>SPDIF_TX_UBUFF_A2.BYTE9</code> bit field contains user bit data for byte 9 of sub frame A.
7:0 (R/W)	BYTE8	Byte 8 Sub Frame A. The <code>SPDIF_TX_UBUFF_A2.BYTE8</code> bit field contains user bit data for byte 8 of sub frame A.

Transmit User Buffer A3 Register

The `SPDIF_TX_UBUFF_A3` register holds the transmit user buffer data for bytes 12-15 for sub frame A.

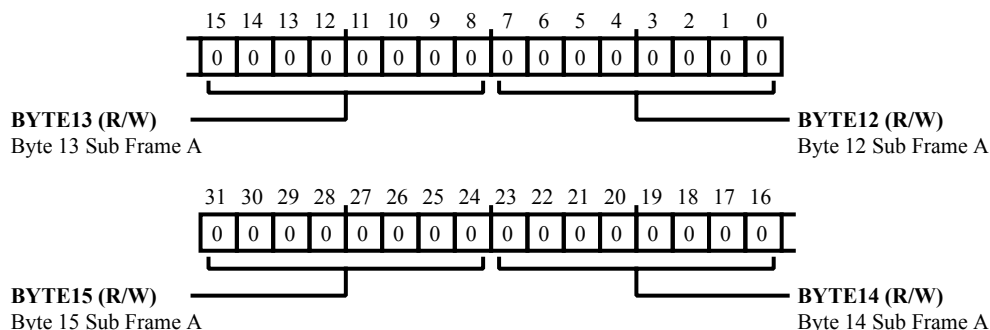


Figure 29-30: SPDIF_TX_UBUFF_A3 Register Diagram

Table 29-27: SPDIF_TX_UBUFF_A3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE15	Byte 15 Sub Frame A. The <code>SPDIF_TX_UBUFF_A3.BYTE15</code> bit field contains user bit data for byte 15 of sub frame A.
23:16 (R/W)	BYTE14	Byte 14 Sub Frame A. The <code>SPDIF_TX_UBUFF_A3.BYTE14</code> bit field contains user bit data for byte 14 of sub frame A.
15:8 (R/W)	BYTE13	Byte 13 Sub Frame A. The <code>SPDIF_TX_UBUFF_A3.BYTE13</code> bit field contains user bit data for byte 13 of sub frame A.
7:0 (R/W)	BYTE12	Byte 12 Sub Frame A. The <code>SPDIF_TX_UBUFF_A3.BYTE12</code> bit field contains user bit data for byte 12 of sub frame A.

Transmit User Buffer A4 Register

The `SPDIF_TX_UBUFF_A4` register holds the transmit user buffer data for bytes 16-19 for sub frame A.

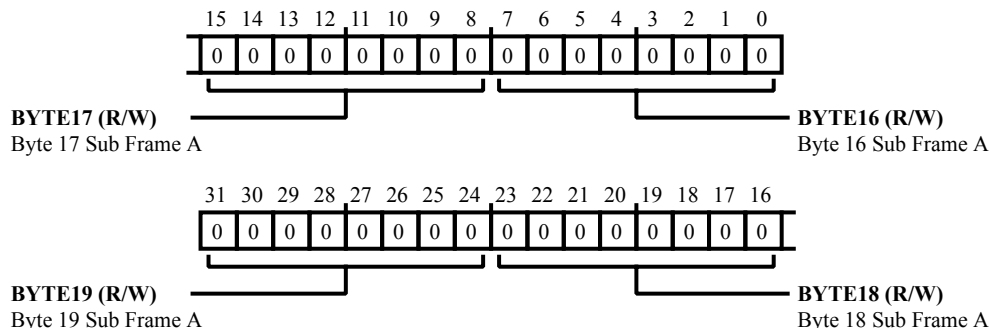


Figure 29-31: SPDIF_TX_UBUFF_A4 Register Diagram

Table 29-28: SPDIF_TX_UBUFF_A4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE19	Byte 19 Sub Frame A. The <code>SPDIF_TX_UBUFF_A4.BYTE19</code> bit field contains user bit data for byte 19 of sub frame A.
23:16 (R/W)	BYTE18	Byte 18 Sub Frame A. The <code>SPDIF_TX_UBUFF_A4.BYTE18</code> bit field contains user bit data for byte 18 of sub frame A.
15:8 (R/W)	BYTE17	Byte 17 Sub Frame A. The <code>SPDIF_TX_UBUFF_A4.BYTE17</code> bit field contains user bit data for byte 17 of sub frame A.
7:0 (R/W)	BYTE16	Byte 16 Sub Frame A. The <code>SPDIF_TX_UBUFF_A4.BYTE16</code> bit field contains user bit data for byte 16 of sub frame A.

Transmit User Buffer A5 Register

The `SPDIF_TX_UBUFF_A5` register holds the transmit user buffer data for bytes 20-23 for sub frame A.

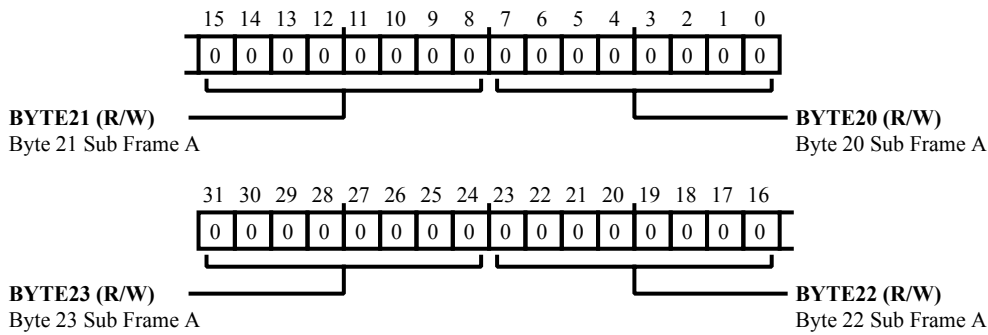


Figure 29-32: SPDIF_TX_UBUFF_A5 Register Diagram

Table 29-29: SPDIF_TX_UBUFF_A5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE23	Byte 23 Sub Frame A. The <code>SPDIF_TX_UBUFF_A5.BYTE23</code> bit field contains user bit data for byte 23 of sub frame A.
23:16 (R/W)	BYTE22	Byte 22 Sub Frame A. The <code>SPDIF_TX_UBUFF_A5.BYTE22</code> bit field contains user bit data for byte 22 of sub frame A.
15:8 (R/W)	BYTE21	Byte 21 Sub Frame A. The <code>SPDIF_TX_UBUFF_A5.BYTE21</code> bit field contains user bit data for byte 21 of sub frame A.
7:0 (R/W)	BYTE20	Byte 20 Sub Frame A. The <code>SPDIF_TX_UBUFF_A5.BYTE20</code> bit field contains user bit data for byte 20 of sub frame A.

Transmit User Buffer B0 Register

The `SPDIF_TX_UBUFF_B0` register holds the transmit user buffer data for bytes 0-3 for sub frame B.

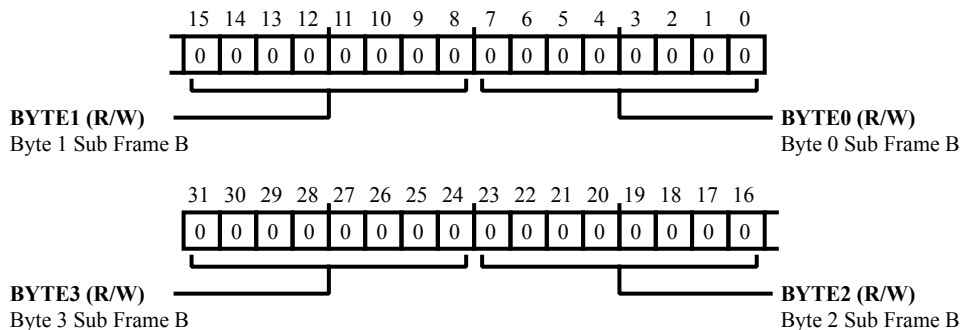


Figure 29-33: SPDIF_TX_UBUFF_B0 Register Diagram

Table 29-30: SPDIF_TX_UBUFF_B0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE3	Byte 3 Sub Frame B. The <code>SPDIF_TX_UBUFF_B0.BYTE3</code> bit field contains user bit data for byte 3 of sub frame B.
23:16 (R/W)	BYTE2	Byte 2 Sub Frame B. The <code>SPDIF_TX_UBUFF_B0.BYTE2</code> bit field contains user bit data for byte 2 of sub frame B.
15:8 (R/W)	BYTE1	Byte 1 Sub Frame B. The <code>SPDIF_TX_UBUFF_B0.BYTE1</code> bit field contains user bit data for byte 1 of sub frame B.
7:0 (R/W)	BYTE0	Byte 0 Sub Frame B. The <code>SPDIF_TX_UBUFF_B0.BYTE0</code> bit field contains user bit data for byte 0 of sub frame B.

Transmit User Buffer B1 Register

The `SPDIF_TX_UBUFF_B1` register holds the transmit user buffer data for bytes 4-7 for sub frame B.

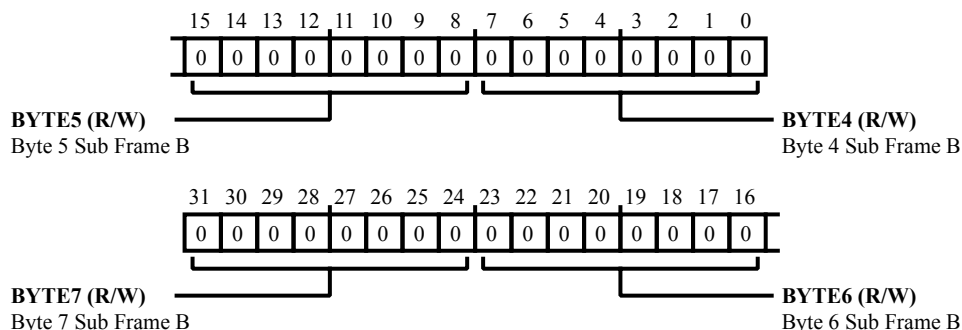


Figure 29-34: `SPDIF_TX_UBUFF_B1` Register Diagram

Table 29-31: `SPDIF_TX_UBUFF_B1` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE7	Byte 7 Sub Frame B. The <code>SPDIF_TX_UBUFF_B1</code> . <code>BYTE7</code> bit field contains user bit data for byte 7 of sub frame B.
23:16 (R/W)	BYTE6	Byte 6 Sub Frame B. The <code>SPDIF_TX_UBUFF_B1</code> . <code>BYTE6</code> bit field contains user bit data for byte 6 of sub frame B.
15:8 (R/W)	BYTE5	Byte 5 Sub Frame B. The <code>SPDIF_TX_UBUFF_B1</code> . <code>BYTE5</code> bit field contains user bit data for byte 5 of sub frame B.
7:0 (R/W)	BYTE4	Byte 4 Sub Frame B. The <code>SPDIF_TX_UBUFF_B1</code> . <code>BYTE4</code> bit field contains user bit data for byte 4 of sub frame B.

Transmit User Buffer B2 Register

The `SPDIF_TX_UBUFF_B2` register holds the transmit user buffer data for bytes 8-11 for sub frame B.

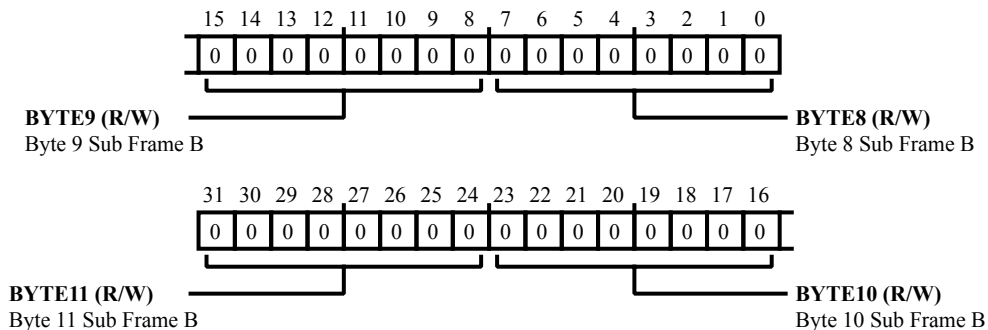


Figure 29-35: SPDIF_TX_UBUFF_B2 Register Diagram

Table 29-32: SPDIF_TX_UBUFF_B2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE11	Byte 11 Sub Frame B. The <code>SPDIF_TX_UBUFF_B2.BYTE11</code> bit field contains user bit data for byte 11 of sub frame B.
23:16 (R/W)	BYTE10	Byte 10 Sub Frame B. The <code>SPDIF_TX_UBUFF_B2.BYTE10</code> bit field contains user bit data for byte 10 of sub frame B.
15:8 (R/W)	BYTE9	Byte 9 Sub Frame B. The <code>SPDIF_TX_UBUFF_B2.BYTE9</code> bit field contains user bit data for byte 9 of sub frame B.
7:0 (R/W)	BYTE8	Byte 8 Sub Frame B. The <code>SPDIF_TX_UBUFF_B2.BYTE8</code> bit field contains user bit data for byte 8 of sub frame B.

Transmit User Buffer B3 Register

The `SPDIF_TX_UBUFF_B3` register holds the transmit user buffer data for bytes 12-15 for sub frame B.

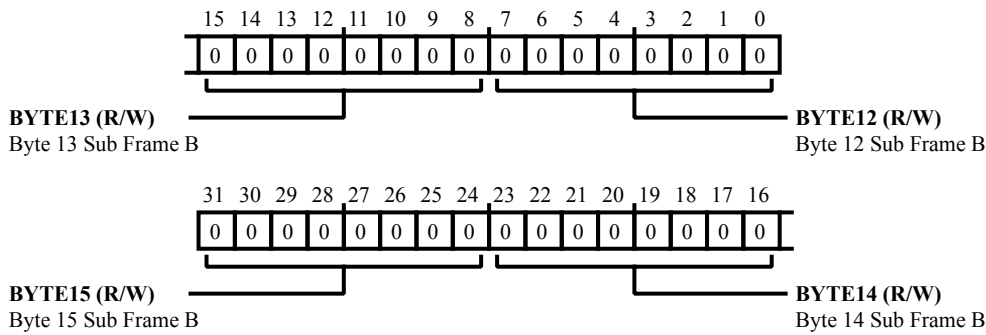


Figure 29-36: SPDIF_TX_UBUFF_B3 Register Diagram

Table 29-33: SPDIF_TX_UBUFF_B3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE15	Byte 15 Sub Frame B. The <code>SPDIF_TX_UBUFF_B3.BYTE15</code> bit field contains user bit data for byte 15 of sub frame B.
23:16 (R/W)	BYTE14	Byte 14 Sub Frame B. The <code>SPDIF_TX_UBUFF_B3.BYTE14</code> bit field contains user bit data for byte 14 of sub frame B.
15:8 (R/W)	BYTE13	Byte 13 Sub Frame B. The <code>SPDIF_TX_UBUFF_B3.BYTE13</code> bit field contains user bit data for byte 13 of sub frame B.
7:0 (R/W)	BYTE12	Byte 12 Sub Frame B. The <code>SPDIF_TX_UBUFF_B3.BYTE12</code> bit field contains user bit data for byte 12 of sub frame B.

Transmit User Buffer B4 Register

The `SPDIF_TX_UBUFF_B4` register holds the transmit user buffer data for bytes 16-19 for sub frame B.

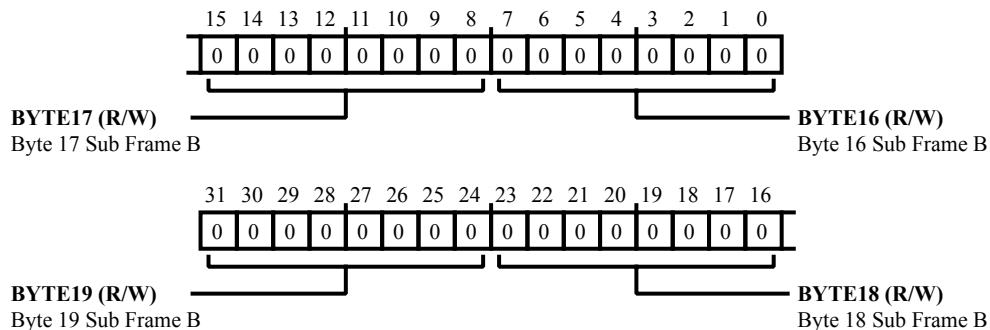


Figure 29-37: SPDIF_TX_UBUFF_B4 Register Diagram

Table 29-34: SPDIF_TX_UBUFF_B4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE19	Byte 19 Sub Frame B. The <code>SPDIF_TX_UBUFF_B4.BYTE19</code> bit field contains user bit data for byte 19 of sub frame B.
23:16 (R/W)	BYTE18	Byte 18 Sub Frame B. The <code>SPDIF_TX_UBUFF_B4.BYTE18</code> bit field contains user bit data for byte 18 of sub frame B.
15:8 (R/W)	BYTE17	Byte 17 Sub Frame B. The <code>SPDIF_TX_UBUFF_B4.BYTE17</code> bit field contains user bit data for byte 17 of sub frame B.
7:0 (R/W)	BYTE16	Byte 16 Sub Frame B. The <code>SPDIF_TX_UBUFF_B4.BYTE16</code> bit field contains user bit data for byte 16 of sub frame B.

Transmit User Buffer B5 Register

The `SPDIF_TX_UBUFF_B5` register holds the transmit user buffer data for bytes 20-23 for sub frame B.

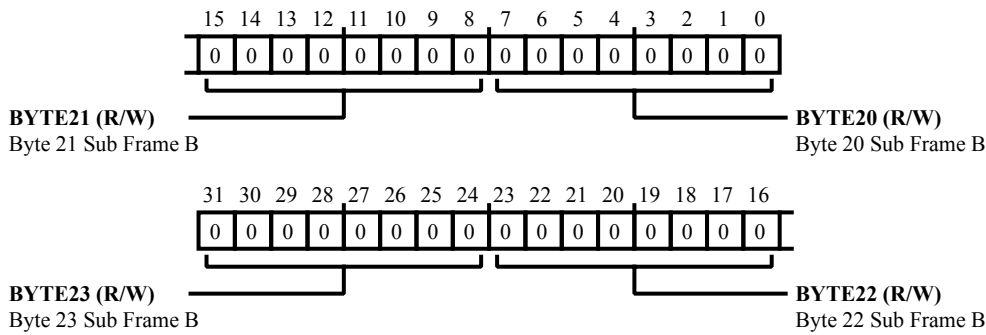


Figure 29-38: SPDIF_TX_UBUFF_B5 Register Diagram

Table 29-35: SPDIF_TX_UBUFF_B5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYTE23	Byte 23 Sub Frame B. The <code>SPDIF_TX_UBUFF_B5.BYTE23</code> bit field contains user bit data for byte 23 of sub frame B.
23:16 (R/W)	BYTE22	Byte 22 Sub Frame B. The <code>SPDIF_TX_UBUFF_B5.BYTE22</code> bit field contains user bit data for byte 22 of sub frame B.
15:8 (R/W)	BYTE21	Byte 21 Sub Frame B. The <code>SPDIF_TX_UBUFF_B5.BYTE21</code> bit field contains user bit data for byte 21 of sub frame B.
7:0 (R/W)	BYTE20	Byte 20 Sub Frame B. The <code>SPDIF_TX_UBUFF_B5.BYTE20</code> bit field contains user bit data for byte 20 of sub frame B.

User Bit Update Register

After writing to the transmit user buffer registers, a value of 0x1 must be written into the `SPDIF_TX_USRUPDT` register to enable the use of these user buffer bits in the next transfer block.

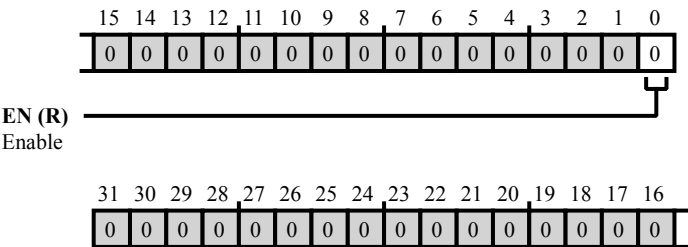


Figure 29-39: SPDIF_TX_USRUPDT Register Diagram

Table 29-36: SPDIF_TX_USRUPDT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/NW)	EN	Enable. After writing to the transmit user buffer registers, a value of 0x1 must be written into the <code>SPDIF_TX_USRUPDT</code> register to enable the use of these <code>SPDIF_TX_USRUPDT</code> . <code>EN</code> bits in the next transfer block.

30 Direct Memory Access (DMA)

The processor architecture distributes the DMA channels throughout the infrastructure. Often, the channels cluster together through system crossbars (SCB), sharing a single interface with the main system crossbar.

The DMA channels can perform transfers between memory and a peripheral or between one memory and another memory. Memory-to-memory DMA transfers (MDMA) require two DMA channels. One channel is the source channel, and the second, the destination channel.

All DMA channels can transport data to and from virtually all on-chip and off-chip memories.

DMA transfers on the processor use either a descriptor-based method or register-based method. Register-based DMA allows the processor directly to program DMA controller registers to initiate a DMA transfer. On completion, the controller registers can automatically update with their original setup values for continuous transfer, if needed. Descriptor-based DMA transfers require a set of parameters stored within memory to initiate a DMA sequence. Descriptor-based transfers allow the chaining together of multiple DMA sequences. In descriptor-based DMA operations, DMA channel programming can automatically set up and start another DMA transfer after the current sequence completes.

The DMA channel does not connect external memories and devices directly. Rather, data passes through an external-memory interface port. DMA operations can access any device the external memory interface supports. These interfaces typically include:

- Flash memory
- SRAM
- FIFOs
- Memory-mapped peripheral devices
- Dynamic Memory (if present)

DMA Channel Features

The processor uses Direct Memory Access (DMA) to transfer data within memory spaces or between a memory space and a peripheral. The processor can specify data transfer operations and return to normal processing while the fully integrated DMA channel carries out the data transfers independent of processor activity. The DMA channels are dispersed throughout the infrastructure and interface with the system crossbar unit (SCB).

The following is a list of DMA interface features.

- Supports integer byte strides including byte strides of 0 and negative byte strides
- Register-based configuration
 - Core writes DMA configuration
 - Supports automatic reloading for continuous operation
- Flexible descriptor-based configuration
 - DMA descriptors are fetched from memory
 - Support for variable descriptor sizes
- Flexible flow control – Transitions between the various descriptor-based modes and for DMA termination
- Orthogonal transfers
 - Support for three transfer dimensions
 - One and two dimensional (1D and 2D) transfers supported per descriptor set
 - Three dimensional (3D) support provided by chained descriptor sets
- Configurable memory and peripheral-transfer word sizes
 - Memory interface supports 8-bit, 16-bit, 32-bit, 64-bit, 128-bit, and 256-bit transfers
 - Peripheral interface supports for 8-bit, 16-bit, and 32-bit transfers
- Interrupt notification
 - Row or work unit completion
 - Error conditions
- Incoming and outgoing trigger support
 - Trigger generation for row or work unit completion
 - Work unit can wait for incoming trigger
- MMR access bus – Provides access to memory-mapped registers for configuration, monitoring, and debug
- SCB crossbar interface connects the DMA channel to the system crossbar
- Peripheral DMA bus – Interfaces the DMA channel to a peripheral or another DMA channel
- Peripheral data-request interrupt support
- Bandwidth monitoring and limiting for MDMA channels

DMA Channel Functional Description

This section provides a functional description of the DMA channel interface.

NOTE: There are two types of peripherals that use DMA. The first have dedicated DMA channels controlled by the Dedicated DMA Engine (DDE) and are described in this chapter. The second type is not controlled by the DDE. These peripherals have their own operating modes and programming models (see the peripheral chapter for this information). The complete list of DMA supported peripherals are shown in the Table 1 .

ADSP-2159x DMA Register List

The DMA channel controller (DMA) supports data transfers within memory spaces or between a memory space and a peripheral. The processor can specify data transfer operations and return to normal processing while the fully integrated DMA channel carries out the data transfers independent of processor activity. The DMA channels are dispersed throughout the infrastructure, as DMA's. A set of registers governs DMA operations. For more information on DMA functionality, see the DMA register descriptions.

Table 30-1: ADSP-2159x DMA Register List

Name	Description
DMA_ADDRSTART	Start Address of Current Buffer Register
DMA_ADDR_CUR	Current Address Register
DMA_BWLCNT	Bandwidth Limit Count Register
DMA_BWLCNT_CUR	Bandwidth Limit Count Current Register
DMA_BWMCNT	Bandwidth Monitor Count Register
DMA_BWMCNT_CUR	Bandwidth Monitor Count Current Register
DMA_CFG	Configuration Register
DMA_DSCPTR_CUR	Current Descriptor Pointer Register
DMA_DSCPTR_NXT	Pointer to Next Initial Descriptor Register
DMA_DSCPTR_PRV	Previous Initial Descriptor Pointer Register
DMA_STAT	Status Register
DMA_XCNT	Inner Loop Count Start Value Register
DMA_XCNT_CUR	Current Count (1D) or Intra-row XCNT (2D) Register
DMA_XMOD	Inner Loop Address Increment Register
DMA_YCNT	Outer Loop Count Start Value (2D only) Register
DMA_YCNT_CUR	Current Row Count (2D only) Register
DMA_YMOD	Outer Loop Address Increment (2D only) Register

ADSP-2159x DMA Channel List

Table 30-2: ADSP-2159x DMA Channel List

DMA ID	DMA Channel Name	Description
DMA0	SPORT0_A_DMA	SPORT0 Channel A DMA
DMA1	SPORT0_B_DMA	SPORT0 Channel B DMA
DMA2	SPORT1_A_DMA	SPORT1 Channel A DMA
DMA3	SPORT1_B_DMA	SPORT1 Channel B DMA
DMA4	SPORT2_A_DMA	SPORT2 Channel A DMA
DMA5	SPORT2_B_DMA	SPORT2 Channel B DMA
DMA6	SPORT3_A_DMA	SPORT3 Channel A DMA
DMA7	SPORT3_B_DMA	SPORT3 Channel B DMA
DMA8	MDMA0_SRC	Memory DMA Stream 0 Source Channel
DMA9	MDMA0_DST	Memory DMA Stream 0 Destination Channel
DMA10	SPORT4_A_DMA	SPORT4 Channel A DMA
DMA11	SPORT4_B_DMA	SPORT4 Channel B DMA
DMA12	SPORT5_A_DMA	SPORT5 Channel A DMA
DMA13	SPORT5_B_DMA	SPORT5 Channel B DMA
DMA14	SPORT6_A_DMA	SPORT6 Channel A DMA
DMA15	SPORT6_B_DMA	SPORT6 Channel B DMA
DMA16	SPORT7_A_DMA	SPORT7 Channel A DMA
DMA17	SPORT7_B_DMA	SPORT7 Channel B DMA
DMA18	MDMA1_SRC	Memory DMA Stream 1 Source Channel
DMA19	MDMA1_DST	Memory DMA Stream 1 Destination Channel
DMA20	UART0_TXDMA	UART0 Transmit DMA
DMA21	UART0_RXDMA	UART0 Receive DMA
DMA22	SPI0_TXDMA	SPI0 TX DMA Channel
DMA23	SPI0_RXDMA	SPI0 RX DMA Channel
DMA24	SPI1_TXDMA	SPI1 TX DMA Channel
DMA25	SPI1_RXDMA	SPI1 RX DMA Channel
DMA26	SPI2_TXDMA	SPI2 TX DMA Channel

Table 30-2: ADSP-2159x DMA Channel List (Continued)

DMA ID	DMA Channel Name	Description
DMA27	SPI2_RXDMA	SPI2 RX DMA Channel
DMA28	EPPI0_CH0_DMA	EPPI0 Channel 0 DMA
DMA29	EPPI0_CH1_DMA	EPPI0 Channel 1 DMA
DMA30	LP0_DMA	LP0 DMA Channel
DMA34	UART1_TXDMA	UART1 Transmit DMA
DMA35	UART1_RXDMA	UART1 Receive DMA
DMA36	LP1_DMA	LP1 DMA Channel
DMA37	UART2_TXDMA	UART2 Transmit DMA
DMA38	UART2_RXDMA	UART2 Receive DMA
DMA39	MDMA2_SRC	Memory DMA Stream 2 Source Channel
DMA40	MDMA2_DST	Memory DMA Stream 2 Destination Channel
DMA43	MDMA3_SRC	Memory DMA Stream 3 Source Channel
DMA44	MDMA3_DST	Memory DMA Stream 3 Destination Channel
DMA45	MDMA4_SRC	Memory DMA Stream 4 Source Channel
DMA46	MDMA4_DST	Memory DMA Stream 4 Destination Channel
DMA47	MDMA5_SRC	Memory DMA Stream 5 Source Channel
DMA48	MDMA5_DST	Memory DMA Stream 5 Destination Channel
DMA49	MDMA6_SRC	Memory DMA Stream 6 Source Channel
DMA50	MDMA6_DST	Memory DMA Stream 6 Destination Channel
DMA51	MDMA7_SRC	Memory DMA Stream 7 Source Channel
DMA52	MDMA7_DST	Memory DMA Stream 7 Destination Channel
DMA53	UART3_TXDMA	UART3 Transmit DMA
DMA54	UART3_RXDMA	UART3 Receive DMA

Table 30-2: ADSP-2159x DMA Channel List (Continued)

DMA ID	DMA Channel Name	Description
DMA55	SPI3_TXDMA	SPI3 TX DMA Channel
DMA56	SPI3_RXDMA	SPI3 RX DMA Channel

DMA Definitions

To make the best use of the DMA controller, it is useful to understand the following terms.

Descriptor

A single element of a descriptor set that maps to a specific register of a DMA channel.

Descriptor Fetch

The action of retrieving descriptors from memory through memory read operations and loading them into the DMA channel registers upon their read return.

Descriptor Set

A array of descriptors associated with a single work unit. The user can configure the size of the descriptor set. See [Descriptor-Based Flow Modes](#).

Disabled State

The channel is disabled because the enable bit = 0 or as a result of an error.

DMAC

An acronym used for a DMA cluster.

DMA Channel

A single DMA engine that has all the capabilities and registers as defined for a given processor. A DMA channel or engine is connected to a single peripheral.

DMA Cluster

A grouping of multiple DMA channels with a shared SCB crossbar interface, controller, and arbiter. Also known as a DMAC.

Initial Descriptor

The first descriptor in the descriptor set.

MDMA

Memory-to-Memory DMA data transfer. Two DMA channels are paired to perform a memory read from one address location and a memory write of that data to another address location.

Stop State

A time where the channel is enabled but not currently programmed to perform a data transfer. Programming the flow to STOP causes the channel to enter the stop state at the end of the work unit.

User

Any person, debug, emulator, software routine, or action taken by the core that accesses the MMR registers of the DMA channel or peripherals, or sets up data and descriptors in memory.

Wait State

If instructed to wait for a trigger, the channel enters this state once it has completed a work unit. The channel remains in this state until a trigger occurs. If a trigger came in before reaching the wait state, the channel skips over the wait state upon completion of the work unit.

Work Unit

A single data transaction or series of data transactions performed based on the configuration of the DMA channel. For autobuffer mode, a new work unit is defined at the time all current count registers are initialized to start values. Once all the current count registers count down to zero, the work unit has completed.

Work Unit Chain

A single work unit or a series of work units separated by a STOP or disabled state. The work units in the chain (except the last one) are programmed to the required descriptor flow. The last work unit in the chain is programmed to a flow of STOP or AUTO. STOP terminates the state at the end of that work unit. AUTO must be terminated by disabling the DMA channel. A work unit chain is also known as a descriptor chain.

Block Diagram

The *DMA Channel Block Diagram* shows the functional blocks within the DMA interface.

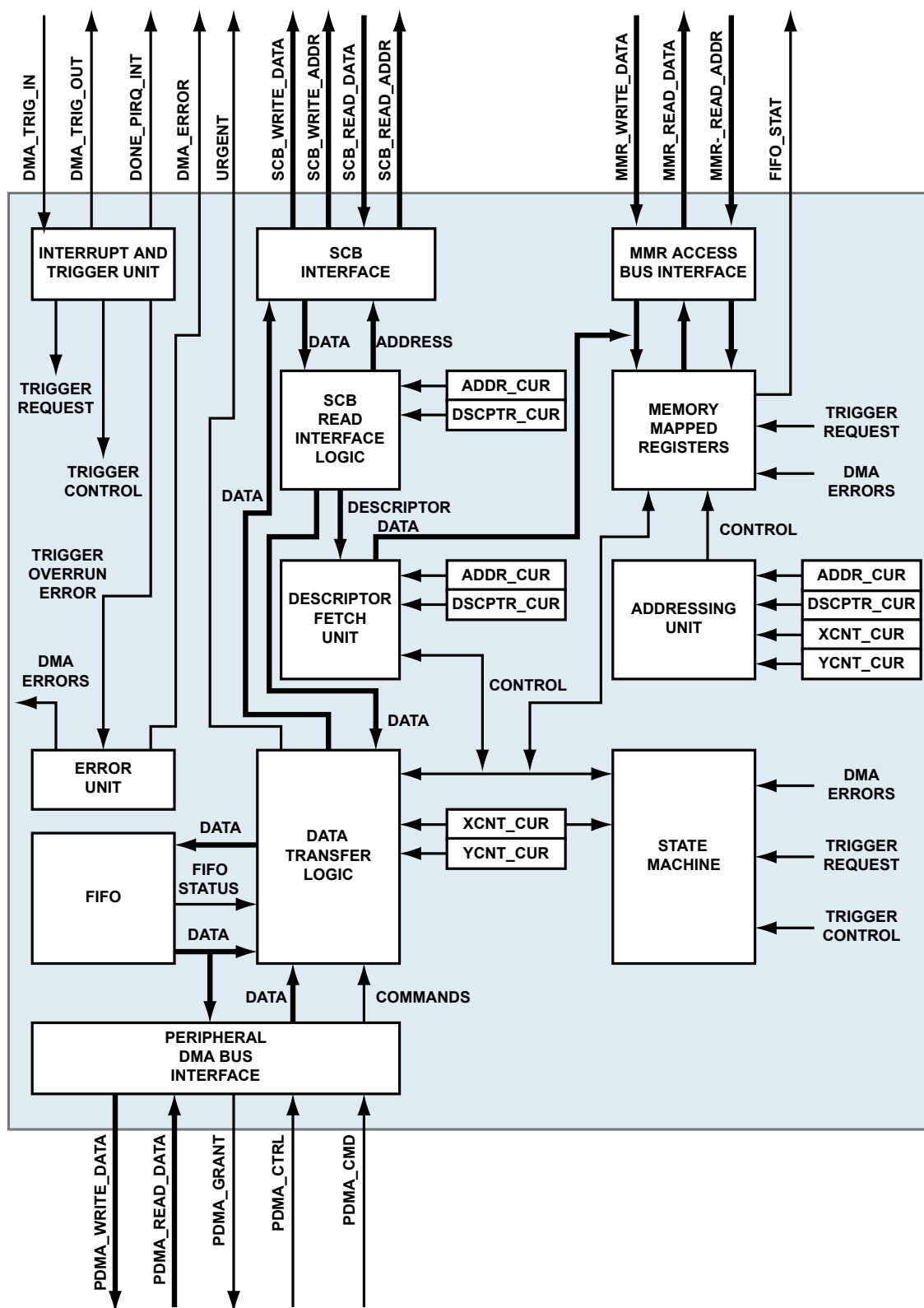


Figure 30-1: DMA Channel Block Diagram

For more information on the interfaces, see:

- [DMA Channel Peripheral DMA Bus](#)
- [Medium Band Width DMA Channel MMR Access Bus](#)
- [DMA Channel Event Control](#)
- [DMA Channel SCB Interface](#)

Architectural Concepts

The DMA channel provides a method to transfer data between memory spaces or between memory and a peripheral using a number of system interfaces. The DMA channel provides an efficient method of distributing data throughout the system, freeing up the processor core for other operations. Each peripheral that supports DMA transfers has its own dedicated DMA channel or channels with its own register set. The register set configures and controls the operating modes of the DMA transfers.

DMA Channel SCB Interface

The SCB interface connects the DMA channel to the SCB crossbar allowing for transfers to and from the processors internal memory and other suitable system resources.

The DMA channel connects to the system interconnect through the SCB interface. This connection lets the DMA channel perform work-unit data transfers with memories such as L1, L2 (internal), and L3 (external). In addition to work unit data transfers, the SCB interface also is used for fetching descriptor sets for all the descriptor-based transfer modes.

The DMA channel can support data bus widths of 16, 32, 64, or 128 bits. The data bus widths for a given DMA channel on a specific processor can vary and are not configurable. Read the `DMA_STAT.MBWID` field to determine the assigned bus widths.

SCB Interface Signals

The DMA channel operates at one of the *SCLK_n* frequencies, as does the SCB interface. SYSCLK clocks all eight DMA channels. The SCB crossbar handles the internal arbitration of the transfer requests of all the masters interfaced to the SCB crossbar instance as shown in the *SCB Interface Signals* table.

Table 30-3: SCB Interface Signals

Signal	Width (bits)	Description
SCB_WRITE_DATA	16, 32, 64, or 128	Data bus used for write operations. The width of the bus can be determined from <code>DMA_STAT.MBWID</code> .
SCB_WRITE_ADDRESS	32	Write address bus. Provides the address of the first transfer in a burst transaction.
SCB_READ_DATA	16, 32, 64, or 128	Data bus used for read operations. The width of the bus can be determined from <code>DMA_STAT.MBWID</code> .
SCB_READ_ADDRESS	32	Read address bus. Provides the address of the first transfer in a burst transaction.

SCB Burst Transfers

The SCB interface supports burst transfers for memory read and write operations. The burst length is a function of the configurable memory size of the DMA channel for the work unit and the fixed bus width of the SCB data bus of the DMA channel.

- If the DMA channel configuration selects a memory transfer size less than or equal to the DMA channels bus width, the burst length is always 1.
- If the configured memory size is greater than the SCB interface bus width, the burst length is sufficient to transfer a transaction as specified by the configured memory size.

Table 30-4: DMA Channel SCB Burst Lengths

Configured Memory Size	Burst Length			
	<i>16-bit Bus</i>	<i>32-bit Bus</i>	<i>64-Bit Bus</i>	<i>128-bit Bus</i>
1 Byte	1	1	1	1
2 Bytes	1	1	1	1
4 Bytes	2	1	1	1
8 Bytes	4	2	1	1
16 Bytes	8	4	2	1
32 Bytes	16	8	4	2

Data Address Alignment

To prevent addressing errors and to maximize bandwidth of the SCB interface to the DMA channel, data addresses align with a multiple of the programmable memory size of the DMA channels configuration. These configuration options appear in the [Descriptor Set Address Alignment](#) table.

There are situations in which entire work units may not transfer at the maximum configurable memory size. In this case, the entire work unit can transfer by reducing the configured memory size at the expense of bus bandwidth using descriptor sets as follows:

- The first descriptor set can be configured to transfer data until the larger memory size alignments are met.
- A second descriptor set with a larger memory size configuration then can be used to transfer the bulk of the data in the work unit.
- Finally, a third descriptor set can be used with a smaller memory size to complete any final data transfers that cannot meet the alignment requirements of the previous descriptor set configuration.

Table 30-5: DMA Channel Address Alignment Requirements

Configured Memory Size	Address Restriction
1 Byte	No restriction
2 Bytes	ADDR[0] == 0

Table 30-5: DMA Channel Address Alignment Requirements (Continued)

Configured Memory Size	Address Restriction
4 Bytes	ADDR[1:0] == 0
8 Bytes	ADDR[2:0] == 0
16 Bytes	ADDR[3:0] == 0
32 Bytes	ADDR[4:0] == 0

Descriptor Set Address Alignment

All descriptor set addresses and descriptors within a descriptor set must align to a 32-bit address. For descriptor set fetches, the DMA engine ignores the memory-size configuration of the DMA channel. This feature avoids the need to align descriptor sets based on the memory width configuration of the previous descriptor set.

For descriptor sets containing only a single descriptor, the transfer takes place as a single 32-bit transfer. For descriptor sets containing multiple descriptors, the DMA engine fetches each 32-bit descriptor individually and treats it as multiple 32-bit transfers.

Peripheral Control Commands

The peripheral DMA bus of the DMA channel provides a means for peripherals on the processor to issue commands to the DMA channel. These commands provide greater control over the DMA channel operation. This control improves real-time performance and relieves control and interrupt demands on the core. Peripherals can send commands to the DMA controller over the 3-bit PERI_CMD bus. The DMA control commands extend the set of operations available to the peripheral beyond the simple “request data” command used by peripherals in general. Refer to the appropriate peripheral chapter for a description on how that peripheral uses DMA control commands.

These DMA control commands (see the *PDMA_CMD Peripheral DMA Control Commands* table) are not visible to or controlled by the program. But, their use by a peripheral has implications for the structure of the DMA transfers that the peripheral can support. It is important to write application software such that it complies with certain restrictions, regarding work units and descriptor chains. Complying with this guideline makes the peripheral operate properly whenever it issues DMA control commands.

The *PDMA_CMD Peripheral DMA Control Commands* table describes the commands the DMA controller issues. The following sections describe these commands in more detail.

Table 30-6: PDMA_CMD Peripheral DMA Control Commands

Command	Name	Description
b#000	NOP	No operation
b#001	Restart	Restarts the current work unit from the beginning
b#010	Finish	Finishes the current work unit and starts the next
b#011	Interrupt	Immediately sets the DMA completion interrupt in the DMA channel

Table 30-6: PDMA_CMD Peripheral DMA Control Commands (Continued)

Command	Name	Description
b#100	Request Data	Typical DMA data request
b#101	Request Data Urgent	Urgent DMA data request
b#110	Reserved	Reserved
b#111	Reserved	Reserved

Idle Command

The DMA channel drives this command when the enabled peripheral has no data requests required.

Request-Data Command

The request data command is a request for data transfers between the DMA channel and the peripheral. The request is held by the peripheral until granted or acknowledged by the DMA channel.

Request-Data Urgent Command

The request-data urgent command behaves identically to the request data command, except that---during the commands assertion---the DMA channel performs its memory accesses with urgent priority. This priority includes both data and descriptor fetch memory accesses. For example, a DMA management capable peripheral can use this control command if an internal FIFO approaches a critical condition.

The request is held by the peripheral until granted or acknowledged by the DMA channel.

Peripheral-Control Command Restrictions

The proper operation of the DMA channel FIFO leads to certain restrictions in the sequence of DMA peripheral control commands issued by a peripheral. The following sections describe these restrictions.

DMA Channel Peripheral DMA Bus

The DMA channel connects to peripherals or other DMA channels through the peripheral DMA bus. This bus is a dedicated point-to-point interface supporting data bus widths of 8, 16, 32, or 64 bits. The data bus widths for a given DMA channel on a particular processor can vary and are not configurable. Reading the `DMA_STAT.PBWID` field permits determining the assigned bus width.

The DMA channel operates at one of the *SCLK_n* frequencies, as does the peripheral DMA bus. The *Peripheral DMA Bus Signals* table provides descriptions of the peripheral DMA bus signals.

Table 30-7: Peripheral DMA Bus Signals

Signal	Width (bits)	Description
PDMA_WRITE_DATA	8, 16, 32, or 64	Data bus used for write operations. The width of the bus can be determined from <code>DMA_STAT.PBWID</code> .
PDMA_READ_DATA	8, 16, 32, or 64	Data bus used for read operations. The width of the bus can be determined from <code>DMA_STAT.PBWID</code> .

Table 30-7: Peripheral DMA Bus Signals (Continued)

Signal	Width (bits)	Description
PDMA_DMA_GRANT		Control signals to indicate that data is valid for DMA channel read operations (peripheral transmit). These signals indicate that the DMA channel is ready to receive data for write operations (peripheral receive).
PDMA_CMD	3	The peripheral uses the signal for issuing DMA channel control commands.
PDMA_CTRL		The peripheral uses the control signals to send various commands to the DMA channel and control the direction of flow.

Memory DMA and Triggering

A memory DMA (MDMA) channel provides a means of doing memory-to-memory DMA transfers among the various memory spaces that have DMA support.

The DMA controller implements memory DMA (MDMA) channels by interfacing two DMA channels through the peripheral DMA bus interface. One DMA channel serves for memory read operations, and the second channel serves for memory writes. Depending on the processor, a memory DMA channel can have an additional peripheral, such as a CRC peripheral. The additional peripheral is inserted into the peripheral DMA bus that optionally can be enabled.

MDMA channel configurations that do not involve an additional peripheral impose no restrictions on which of the DMA channels is used for the read operation or the write operation. But, the configuration of both channels cannot have the same transfer direction. For MDMA channel configurations that enable a peripheral between the read and write channels, be aware of possible restrictions imposed on which channel can be used for a given transfer direction.

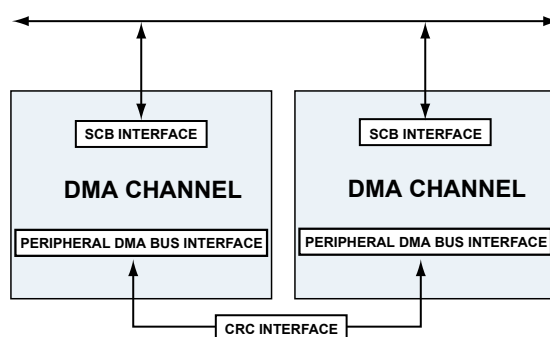


Figure 30-2: MDMA Channel Dedicated Pair

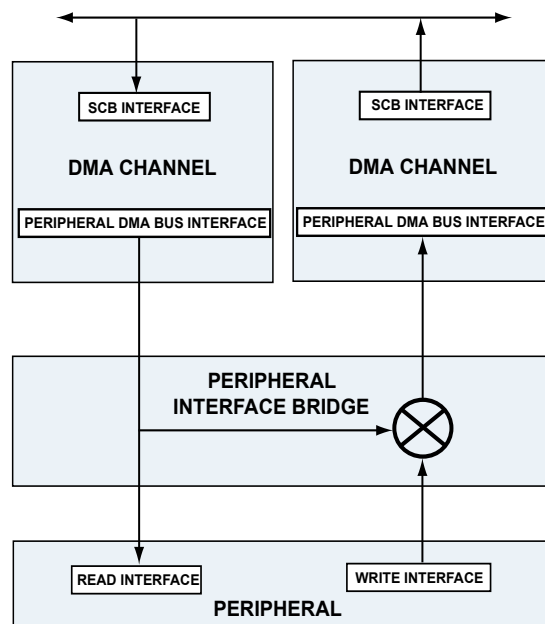


Figure 30-3: MDMA Channel Pair with Peripheral

A memory-to-memory transfer always requires enabled source and destination channels. Because the channels interface through the peripheral DMA bus and can have an additional peripheral inserted into the peripheral DMA bus, programs must make sure to set the same values in the `DMA_CFG.PSIZE` of both the source and destination channels.

The memory DMA channels support the full range of the `DMA_CFG.MSIZE` options for the DMA transfers to and from the memories.

Because the MDMA channel consists of two DMA channels, the entire MDMA channel has two sets of FIFOs, one in the read channel and one in the write channel. This FIFO usage allows for more efficient bursting of both read and write transactions using the available bandwidth. While the `DMA_CFG.PSIZE` configuration must be identical for both source and destination DMA channels, this restriction does not apply for the `DMA_CFG.MSIZE` configuration.

Configure the `DMA_CFG.PSIZE` bits to a value no larger than the supported bus width of the peripheral DMA bus. From a performance perspective, use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `PSIZE` value used.

NOTE: This is applicable for all of the DMA channels, except the enhanced and high-speed MDMA channels. For the enhanced and high-speed MDMAs, the minimum *PSIZE* and *MSIZE* is 4 bytes (32 bits).

The independent source and destination DMA channels also have their own dedicated interrupt and trigger events. While it is normal practice to have only event generation performed at destination DMA completion, programs also can use other means of interrupt generation.

Configuration of an MDMA transfer is done in a similar manner to peripheral DMA transfers, except for writing two DMA channel registers instead of one.

To control the pace of data transfers, use triggers on either the memory read or the memory write channel pair used in an MDMA operation. Setting the `DMA_CFG.TWAIT` bit in the memory read channel prevents both channels from transferring data before the system is ready. However, only configuring the memory write channel to wait for a trigger allows for data fetch from the memory in anticipation of the memory write operation.

The processor supports these categories of MDMA channels:

Standard Bandwidth MDMA. The channels run in the SYSCLK domain and support 32-bit memory and peripheral bus width. The maximum theoretical bandwidth of these MDMA streams is up to $4 \times 500 = 2000$ MB/s. This MDMA optionally supports CRC.

Enhanced Bandwidth MDMA. The channels run in SYSCLK domain and support 32-bit memory and peripheral bus width. The maximum theoretical bandwidth of this MDMA stream is up to $4 \times 500 = 2000$ MB/s.

Maximum Bandwidth MDMA. The channels run in SYSCLK domain and support 64-bit memory and peripheral bus width. The maximum theoretical bandwidth of these MDMA streams is up to $8 \times 500 = 4000$ MB/s.

Medium Band Width DMA Channel MMR Access Bus

The MMR access bus provides access to all the DMA channels memory-mapped registers for DMA channel configuration, monitoring, and debug. The interface has a fixed 32-bit data bus for read and write accesses.

The *MMR Access Bus Signals* table provides descriptions of the MMR access bus signals.

Table 30-8: MMR Access Bus Signals

Signal	Width (bits)	Description
MMR_WRITE_DATA	32	Data bus used for write operations to the MMRs from the core
MMR_READ_DATA	32	Data bus used to return read data from the MMRs
MMR_READ_ADDR	7	Address used to select the MMR to access

DMA Channel Operation Flow

A detailed description of the flow of operation of the DMA channel appears in the following topics:

- [Startup Flow](#)
- [Refresh Flow](#)
- [DMA Operating Modes](#)
- [Stop Mode](#)
- [DMA Channel Errors](#)

Startup Flow

Enabling a DMA operation on a given channel first requires directly writing some or all of the DMA parameter registers. The minimum set of register required to be initialized depends on the desired mode of operation as described in the following sections.

Startup Minimum-Enable Requirements

To start a DMA operation on a given channel, some or all of the DMA parameter registers must first be initialized and configured to the desired DMA channels operating mode.

- For descriptor-array-based flow modes, at minimum, write the `DMA_DSCPTR_CUR` register prior to writing to the `DMA_CFG` register, which is the special action required to start the DMA channel.
- For descriptor-list-based flow modes, at minimum, write the `DMA_DSCPTR_NXT` register prior to writing to the `DMA_CFG` register, which is the special action required to start the DMA channel.
- For non-descriptor-based flow modes, write the `DMA_ADDRSTART`, `DMA_XCNT`, and `DMA_XMOD` registers prior to writing the `DMA_CFG` register.

Programs can write other registers that can remain static throughout the course of the DMA activity. The write to the `DMA_CFG` register begins the DMA operation.

ATTENTION: When software directly writes the `DMA_CFG` register, the DMA controller recognizes this action as the special startup condition. This condition occurs when starting the DMA controller for the first time on this channel or occurs after the DMA channel stops. It is possible for the channel to flag a DMA error condition regardless of the `DMA_CFG.EN` bit setting.

Startup Operation

The startup operation is initiated by software directly writing the `DMA_CFG` register when starting DMA for the first time on a channel or after the channel has entered to the stop state.

When the descriptor fetch is complete and the DMA channel is enabled, the `DMA_CFG` descriptor element replaces the `DMA_CFG` register content and assumes control. Before this point, the direct write to the `DMA_CFG` register had control.

At startup, the selected flow mode and the descriptor size determine the course of the DMA initialization process. The `DMA_CFG.FLOW` field determines whether to load more current registers from descriptor sets in memory. The `DMA_CFG.NDSIZE` field details how many descriptor elements to fetch before starting the DMA operation. This process does not affect DMA registers that are not in the descriptor; no modifications are made to their prior values.

For descriptor-list flow modes, the channel copies the `DMA_DSCPTR_NXT` register value into the `DMA_DSCPTR_CUR` register. Then, the channel fetches new descriptor elements from memory. The `DMA_DSCPTR_CUR` register indexes each fetch, and the channel increments the index after each fetch. After completion of the descriptor fetch, the `DMA_DSCPTR_CUR` register points to the next 32-bit word in memory past the end of the descriptor.

If the descriptor fetch is for a descriptor-array mode transfer, the channel does *not* copy the `DMA_DSCPTR_NXT` register into the `DMA_DSCPTR_CUR` register. *Instead*, the descriptor fetch indexing begins with the value in the `DMA_DSCPTR_CUR` register.

If `DMA_CFG` is not part of the fetched descriptor set, the previous value (originally as written on startup) controls the work unit operation. If the `DMA_CFG` register is part of the fetched descriptor set, the value programmed by the MMR access controls only the loading of the first descriptor set fetched from memory. The configuration of the `DMA_CFG` register controls the subsequent DMA work units of the fetched descriptor set.

After the descriptor fetch is complete or if the flow configuration was originally for one of the register-based flow modes, the DMA operation begins. The DMA channel immediately fills its FIFO. For a memory-write operation, the DMA channel begins accepting data from the peripheral. For a memory-read operation, the DMA channel begins memory reads when the SCB bus grants access to the DMA channel.

When the DMA channel performs its first data-memory access, its address and count computations take their input operands from the start registers. These registers can include `DMA_ADDRSTART`, `DMA_XCNT`, and `DMA_YCNT`, if necessary. The channel writes results back to the current registers. These registers include `DMA_ADDR_CUR`, `DMA_XCNT_CUR`, and `DMA_YCNT_CUR`. Note that the current registers are not valid until the channel performs the first memory access, which can be some time after the write to the `DMA_CFG` register starts the channel. Once started, the channel automatically loads the current registers from the appropriate descriptor elements, overwriting their previous contents. These automatic-load operations include:

- The channel copies the `DMA_ADDRSTART` value to `DMA_ADDR_CUR`.
- The channel copies the `DMA_XCNT` value to `DMA_XCNT_CUR`.
- The channel copies the `DMA_YCNT` to `DMA_YCNT_CUR`.

Refresh Flow

When the channel completes processing of a work unit, the DMA channel performs the following operations:

- Completes the transfer of all data between memory and the DMA channel.
- Performs a synchronized transition (if the DMA channel configuration is a memory read operation with the `DMA_CFG.SYNC` bit enabled) *and* transfers all data to the peripheral before continuing.
- Forwards the signals from the DMA channel (if interrupts or triggers are enabled) *and* updates the `DMA_STAT` register to indicate the interrupt request or trigger events.
- Clears the `DMA_STAT.RUN` bit field to stop DMA operation (if the flow was set to stop mode) *and* transfers any remaining data in the FIFO of the DMA channel to the peripheral.
- Loads a new descriptor from memory into the DMA registers by way of the contents of the `DMA_DSCPTR_CUR` register (for descriptor-array mode) *and* increments the `DMA_DSCPTR_CUR` register

The channel takes the descriptor size from the `DMA_CFG.NDSIZE` value before the fetch.

- Copies the `DMA_DSCPTR_NXT` register into the `DMA_DSCPTR_CUR` register (for descriptor-list mode), fetches the descriptor from the new contents of the `DMA_DSCPTR_CUR` register, *and* places these contents into the DMA registers while incrementing the `DMA_DSCPTR_CUR` register.
- Checks for detection of an incoming trigger event (for descriptor-on-demand array mode):
 - If the channel detects a trigger event, the DMA channel loads a new descriptor from memory into the DMA registers from the contents of the `DMA_DSCPTR_CUR` register, while incrementing the `DMA_DSCPTR_CUR` register. The channel takes the descriptor size from the `DMA_CFG.NDSIZE` value before the fetch.
 - If the channel detects no trigger event, the DMA channel begins the next work unit by reloading the current registers.
- Checks for detection of an incoming trigger event (for descriptor-on-demand list mode):
 - If the channel detects a trigger event, the DMA channel copies the `DMA_DSCPTR_NXT` register value to the `DMA_DSCPTR_CUR` register, fetches the descriptor memory from the `DMA_DSCPTR_CUR` register, *and* places the contents into the DMA registers while incrementing the `DMA_DSCPTR_CUR` register.
 - If the channel detects no trigger event, the DMA channel begins the next work unit by reloading the current registers as described in the next step.
- Begins the next work unit (if flow configuration is anything other than stop mode) by reloading the current registers (`DMA_ADDR_CUR`, `DMA_XCNT_CUR`, and `DMA_YCNT_CUR`) from their descriptor registers (`DMA_ADDRSTART`, `DMA_XCNT`, and `DMA_YCNT`)

Work Unit Transition Flow

The `DMA_CFG.SYNC` bit controls transitions from one work unit to the next work unit. In general, continuous transitions have lower latency at the cost of restrictions on changes of data format or addressed memory space in the two work units. These latency gains and data restrictions arise from the way the channel handles the DMA FIFO while fetching the next descriptor.

In continuous transitions, with disabled synchronization, the DMA FIFO pipeline continues to transfer data to and from the peripheral or destination memory. These transfers continue during the descriptor fetch and during the DMA channel pause between descriptor chains. By comparison, synchronized transitions provide better real-time synchronization of interrupts and triggers with a given peripheral state. Synchronized transitions also provide greater flexibility in the data formats and memory spaces of the two work units. This flexibility comes at the cost of higher latency in the transition. In synchronized transitions, the DMA FIFO pipeline drains to the destination or flushes (received data discarded) between work units.

NOTE: The `DMA_CFG.SYNC` bit of the MDMA source channel controls work unit transitions for MDMA streams. Clear this reserved bit of the MDMA destination channel, placing it in the disabled state. In transmit (memory read) channels, the `DMA_CFG.SYNC` bit of the last descriptor before the transition controls the transition behavior. In contrast, in receive channels, the `DMA_CFG.SYNC` bit of the first descriptor of the next descriptor chain controls the transition.

Work Unit Transmit and MDMA Source Transitions

In DMA transmit (memory read) and MDMA source channels, the `DMA_CFG.SYNC` bit controls the interrupt timing at the end of the work unit. This bit also controls the handling of the DMA FIFO between the current and the next work unit.

If the `DMA_CFG.SYNC` bit configuration disables synchronization, the DMA channel operates in continuous transition. In a continuous transition, just after reading the last data item from memory, the DMA channel starts all of the following operations parallel:

- Signals the interrupt request or trigger
- Updates the `DMA_STAT` register to indicate DMA completion status
- Begins fetching the next descriptor
- Delivers the final data items from the DMA FIFO to the destination memory or peripheral

This process lets the DMA channel provide data from the FIFO to the peripheral continuously during the descriptor fetch latency period.

If the configuration disables synchronization, the final interrupt request or trigger (if enabled) occurs when the channel reads the last data from memory. This event occurs at the earliest time that the channel safely can modify the output memory buffer without affecting the previous data transmission. There can be a number of data items remaining in the FIFO and not yet at the peripheral. This number depends on the FIFO depth of the DMA channel. In this configuration, do not use the DMA interrupt request as the sole means of synchronizing the shutdown or reconfiguration of the peripheral following a transmission.

NOTE: If the configuration selects continuous transition on a transmit (memory read) descriptor, the next descriptor must have the same:

- Peripheral transfer size (`DMA_CFG.PSIZE`)
- Read or write direction
- Source memory (internal versus external) as the current descriptor

It is possible to disable synchronization by selecting continuous transition on a work unit with configuration for stop-flow mode and with enabled interrupts or triggers. This approach can result in the execution of the event service routine while draining of the final data is ongoing from the FIFO to the peripheral. If data transfers are in-progress, the FIFO is not yet empty. The `DMA_STAT.RUN` bits of the DMA channels indicate this status. Do not start a new work unit with a different peripheral transfer size or direction while data transfers are in-progress.

CAUTION: Disabling the channel with the `DMA_CFG.EN` bit while data transfers are in-progress causes the loss of the data in the FIFO.

A synchronized transition configuration directs the channel to drain the DMA FIFO to the destination memory or peripheral. This FIFO operation occurs before the channel signals any interrupt and before the channel fetches any subsequent descriptor or data. This operation incurs greater latency, but provides direct synchronization between the DMA interrupt and the state of the data at the peripheral.

If the configuration enables synchronization and enables interrupts, on the last descriptor in a work unit, the interrupt occurs when the channel transfers the final data to the peripheral. This event allows the service routine to switch properly to non-DMA transmit operation. When the event vectors to the interrupt service routine, the DMA channel FIFO is empty, and the DMA channel is no longer running (indicated by the `DMA_STAT.RUN` bits).

A synchronized transition also allows greater flexibility in the format of the DMA descriptor chain. When enabled, the next descriptor can have any `DMA_CFG.PSIZE` configuration or read/write direction supported by the peripheral and can come from either memory space (internal or external). This feature can be useful in managing MDMA work unit queues, since it is no longer necessary to interrupt the queue between dissimilar work units.

Work Unit Receive and MDMA Destination Transitions

In DMA receive channels (memory write operations), the `DMA_CFG.SYNC` bit controls the handling of the DMA FIFO between descriptor chains (not individual descriptor sets), during the DMA channel pause. The DMA channel pauses after the descriptor sets configured with stop flow mode are complete. Restart the channel (for example, after an interrupt) by writing the `DMA_CFG` register of the channel with a value that enables the DMA channel. If the configuration disables synchronization in the `DMA_CFG` value of the new work unit, the configuration selects a continuous transition. In this mode, the DMA FIFO retains any data items received during the channel pause, and they are the first items written to memory in the new work unit. This mode of operation provides lower latency at work unit transitions and ensures no dropping of data items during a DMA pause. The channel provides this operation at the cost of certain restrictions on the DMA descriptors.

NOTE: If the `DMA_CFG.SYNC` bit disables synchronization on the first descriptor of a chain after a DMA pause, do not change the configuration of the `DMA_CFG.PSIZE` field of the new chain from the previous descriptor chain (active before the pause). This restriction applies unless the DMA channel is reset between chains by disabling and then re-enabling the DMA channel.

If the `DMA_CFG.SYNC` bit configuration enables synchronization, the channel uses a synchronized transition. In this mode, only the data that the DMA channel receives from the peripheral after the write to the `DMA_CFG` register gets to memory. The channel discards any prior data items transferred from the peripheral to the DMA FIFO before this register write occurs. This operation provides direct synchronization between the data stream received from the peripheral and the timing of the channel restart, which occurs on the write to the `DMA_CFG` register.

For receive DMA operations, the synchronization has no effect in transitions between work units in the same descriptor chain. When the flow mode of previous descriptor was not stopped, the DMA channel did not pause.

If a descriptor chain begins with synchronization enabled, there is no restriction on the `DMA_CFG.PSIZE` of the new chain in comparison with the previous chain.

NOTE: The peripheral transfer size (`DMA_CFG.PSIZE`) must not change between one descriptor and the next in any DMA receive (memory write) channel within a single descriptor chain, regardless of the `DMA_CFG.SYNC` bit setting. In other words, all memory write descriptor sets in a descriptor chain must have the same `DMA_CFG.PSIZE` value. For any DMA receive channel (memory write operation), there is no restriction on changes of peripheral transfer size (internal versus external) between descriptors or descriptor chains.

Transfer Termination and Shutdown Flow

This section describes channel transfer termination and shutdown in stop flow mode and in autobuffer flow mode.

Stop Flow Mode

In stop flow mode, the DMA channel stops automatically after the work unit is complete. If using a list or array of descriptors to control DMA transfers and if every descriptor contains a `DMA_CFG` descriptor element, configure the flow of the final `DMA_CFG` descriptor element to stop mode, stopping the channel gracefully. After completion, the DMA channel remains in the stop state. Do not confuse this state with the disabled state, which either occurs due to a DMA error or occurs through disabling the DMA channel by configuring the `DMA_CFG.EN` bit.

The intention of disabling the DMA channel through a write to the `DMA_CFG.EN` bit is to shut down the DMA channel and to enter the disabled state. All memory and peripheral data transfers cease, and only peripheral interrupts pass through the DMA channels interrupt signals. However, the DMA channel maintains the `DMA_STAT.RUN` bits. For a write to memory, the outstanding memory-transaction counter tracks returning memory write acknowledgments and updates as required.

For memory reads, the outstanding memory-transaction count also tracks returning memory reads. The channel does not write the memory reads into the FIFO. The channel updates the counter to reflect the completion of the transaction, but the channel ignores the data. The `DMA_STAT.RUN` bits remain in the *waiting for write ACK or FIFO drain to peripheral* state and do not change to *stop or idle state* until the return of all outstanding transactions.

When the `DMA_CFG.EN` bit again enables the DMA channel, the channel performs a full reset and clears all counters. If an outstanding memory transaction returns an acknowledgment or read data after this event, a memory transaction error occurred, which generates an error event. Programs must ensure that all outstanding memory transactions complete before reconfiguring the DMA channel. For example, programs can poll the `DMA_STAT.RUN` bits to return to the stop or idle state before proceeding.

Autobuffer Flow Mode

In this mode, the flow does not use any descriptors in stored memory. Instead, the channel performs DMA in a continuous circular buffer fashion, based on user-programmed DMA register settings. On completion of the work unit, the channel reloads the parameter registers into the current registers, and the DMA controller resumes immediately with zero overhead. Consider this mode as a succession of automatically restarted work units.

For autobuffer-flow modes, the only way to cease operations is to disable the DMA channel through the `DMA_CFG.EN` bit. One method of changing to a new work unit is:

- Disable the DMA channel
- Set up all the registers (and descriptors in memory, if used) except for `DMA_CFG`
- Poll `DMA_STAT.RUN` to wait for the status to reflect stop or idle state, and
- Write `DMA_CFG` to the new configuration to begin the next work unit

In autobuffer-flow mode or for a list or array of descriptor sets without `DMA_CFG` descriptors, use an MMR write to the `DMA_CFG` register to terminate the DMA transfer process. Configure the value of the `DMA_CFG.EN` bit in this register to disable the DMA channel.

CAUTION: When the configuration disables a DMA channel, the DMA controller disables interrupt logic that is based on work unit transitions. Be aware of the system environment and current actions, so that additional interrupts are not required from the DMA channel.

CAUTION: If disabled through `DMA_CFG.EN` in the middle of a transaction, the DMA channel completes any transactions that have begun and avoids generating bus errors. However, the channel considers the action of re-enabling the DMA as a hard reset for all internal DMA channel components. Therefore, pay attention to that particular action to avoid unexpected results.

DMA Channel Errors

When an error occurs, the DMA channel maintains all the state and register values that allow programs to diagnose error causes more thoroughly. The greatest benefit to the programmer is to know exactly what operational state the DMA channel was in at the exact moment the error occurred.

Take care to address the root cause of the error, whether or not the problem originated in the DMA channel. If not properly resolved, the error can result in an additional error shortly after operations resume. The problem can cause other errors elsewhere in the DMA channel or associated modules and circuitry. So, take care also to address those potential problems. Ensure that all outstanding memory reads and writes are complete or cleared before resuming DMA channel operation.

After addressing all issues and neutralizing all side effects of any errors, clear the `DMA_STAT.ERRC` status field and restart the DMA channel by disabling then re-enabling the DMA channel through the `DMA_CFG.EN` bit.

The following sections describe the error types.

Status and Debug Errors

DMA channel error conditions can cause the DMA process to end abnormally. The DMA channel provides error detection as a tool for system development and debug, helping to identify DMA-related programming errors. When the DMA channel detects an error, the channel immediately stops and discards any returned memory-read transactions. The `DMA_STAT.RUN` field of the DMA channel indicates the idle state after acknowledging all outstanding memory transactions. In addition, the channel asserts an error interrupt request and updates the `DMA_STAT.IRQERR` field. Also, the channel updates the `DMA_STAT.ERRC` field, indicating the error cause of the first detected error. Unless the error occurs at the exact moment that modification of register values occurs, the registers contain the error values.

All the DMA error interrupt requests are combined into a single shared interrupt request output. Combined error signals require reading the `DMA_STAT` register of each DMA channel associated with a combined error interrupt request to determine the DMA channel responsible for the generation of the interrupt.

The DMA channel error interrupt handler performs the following actions:

- Read the `DMA_STAT` register of each DMA channel, seeking a channel with the `DMA_STAT.IRQERR` set to indicate an error.
- Read the `DMA_STAT.ERRC` field of each DMA channel, determining the cause of the error.
- Clear the problem with the DMA channel. For example, fix the register values.

- Clear the error in the DMA channel through a write-1-to-clear operation to the `DMA_STAT.IRQERR` bit.

If the channel flags any uncleared error other than a bandwidth monitor error, the channel reports no other error. If the channel reports an uncleared bandwidth monitor error, the channel reports any newly detected error through updating the `DMA_STAT.ERRC` field.

DMA Configuration Register Errors

The channel only flags these configuration errors when the `DMA_CFG.EN` bit enables the DMA channel. Error flagging occurs when the configuration:

- Uses a reserved setting
- Enables `DMA_CFG.TWAIT` in descriptor on-demand flow mode
- Uses an illegal `DMA_CFG.NDSIZE`
- Uses an illegal `DMA_CFG.MSIZE`
- Configures `DMA_XCNT = 0` or, when `DMA_YCNT = 0` in 2D DMA mode
- Uses non-zero value in `DMA_CFG.NDSIZE` when DMA is configured in stop mode or auto mode
- Enables interrupt or outgoing triggers on `DMA_YCNT` when DMA is configured in 1D mode
- Use a `DMA_CFG.MSIZE` that exceeds the FIFO size of the DMA channel
- Uses an illegal `DMA_CFG.PSIZE`
- Uses a `DMA_CFG.PSIZE` that exceeds the FIFO size
- Uses a `DMA_CFG.PSIZE` that exceeds the bus width
- Attempts to change from a transmit operation (memory read) to a receive operation without properly synching in the previous work unit or when it is the first work unit in a new chain
- Attempts to change `DMA_CFG.PSIZE` of a transmit operation (memory read) without properly synching in previous work unit or when it is the first work unit in a new chain
- Attempts to change from receive operation (memory write) to a transmit operation during a descriptor chain.
The channel only can change from receive to transmit if the new transmit is synchronized and is the first work unit.
- Attempts to change `DMA_CFG.PSIZE` of a receive operation (memory write) when the operation was not the first work unit (with `DMA_CFG.SYNC` enabled)

Illegal Register Write During Run

The channel generates an error when a write occurs to writable registers of an enabled, running DMA channel. The channel blocks the write. The `DMA_STAT`, `DMA_BWLCNT`, and `DMA_BWMCNT` registers are exempt from this behavior.

Address Alignment Error

The channel generates an address alignment error when any of the following apply:

- Alignment of a descriptor address is not on a 32-bit boundary.
- The current `DMA_CFG.MSIZE` configuration contains an unaligned transfer address. The `DMA_ADDRSTART` register is not aligned according to the `DMA_CFG.MSIZE` field.

Memory Access Error

The channel generates a memory access error when the DMA process:

- attempts to access an unpopulated address,
- attempts to access a location that provokes a security violation

The error returned from the memory triggers the memory access error.

Trigger Overrun Error

A trigger overrun error is generated when a new trigger input occurred while an outstanding trigger is waiting. This error is only generated if `DMA_CFG.TOVEN` is enabled.

Bandwidth-Monitor Error

The channel generates this error when the bandwidth-monitor count expires. This error is not fatal, and the DMA channel continues operation.

Control Interface Error

The channel reports control-interface errors as bus errors to the bus master. This error can result from:

- An address error
- A register write error (write to a read-only register)

DMA Operating Modes

The DMA channel supports a number of different flow modes that control how the DMA channel progresses from one work unit to the next.

The flow mode of a DMA channel is not a global setting. A DMA descriptor set can include the descriptor responsible for configuring the flow of the work unit. There is no restriction, limiting the flow configuration to be the same for the entire descriptor chain. If the descriptor chain is not endless, the last descriptor set configures the flow to stop mode, which results in termination of the descriptor chain after the work unit completes. Another example for mixing flow modes is to create an endless descriptor-array. The configuration of the last descriptor set in the array selects the descriptor-list mode. The next descriptor pointer in this set of descriptors points to the first descriptor in the array.

Register-Based Flow Modes

Register-based DMA operations require configuration by directly writing to the memory-mapped registers of the DMA channel.

Register-based DMA is the traditional method of DMA operation. Software writes all of the configuration of the DMA channel into the memory-mapped registers. This configuration includes information such as the source or destination address and length of the data in the transfer. The DMA controller then starts channel operation. The DMA channel supports the following register-based flow modes.

- [Stop Mode](#)
- [Autobuffer Mode](#)

The DMA channel supports variable descriptor set sizes within the configuration. The size of a descriptor set can contain as little as a single descriptor. The supported descriptor set sizes can differ between the various descriptor-based flow modes. In addition to the descriptor set size being configurable, descriptor-based DMA also allows altering the flow mode of the next descriptor set. This feature allows for the transition from descriptor-array mode to descriptor-list mode and permits configuring the flow to stop or autobuffer mode.

Stop Mode

In stop mode, the DMA operation executes only once. If started, the DMA channel transfers the desired number of data words and stops itself again when finished. If the DMA channel is no longer used, software configures the enable bit to disable a paused channel. The channel also can generate interrupts and triggers for each row or work unit completion, depending on the desired operation.

Autobuffer Mode

In autobuffer mode, the DMA operates repeatedly in a circular manner. If the transfer of all data words completes, the channel reloads the address pointer ([DMA_ADDR_CUR](#)) automatically with the [DMA_ADDRSTART](#) value. The channel also can generate an interrupt.

The `DMA_CFG.FLOW` field enables autobuffer mode. The configuration must load the `DMA_CFG.NDSIZE` field value, such that the next descriptor size is zero.

Descriptor-Based Flow Modes

Descriptor-based DMA operations fetch descriptor sets from memory allowing for autonomous loading of work units on other work units. Software does not need to set up the DMA sequences directly by writing into the DMA controller registers. Rather, software keeps DMA descriptor sets in memory.

Descriptor-based DMA operations have the following additional attributes.

- The DMA controller autonomously loads the descriptor set from memory to the affected DMA controller registers on demand.
- The channel can fetch descriptor sets from any memory space that supports DMA read operations.
- The descriptor set describes the next operation that the DMA controller performs.

- The descriptor set can include information such as the DMA configuration word as well as data source or destination address, transfer count, and address modify values.

A descriptor set describes a single work unit. The next work unit can reuse some values from the previous one descriptor set. But, this reuse is possible only if they are not overwritten in the subsequent descriptor set fetches and only if the work unit requires the use of this descriptor.

The DMA channel supports the following flow modes with descriptor-based operations.

- [Descriptor-Array Mode](#)
- [Descriptor-List Mode](#)
- [Descriptor-On-Demand Modes](#)

The DMA channel supports variable descriptor set sizes within the configuration. The size of a descriptor set can contain as little as a single descriptor and the supported descriptor set sizes can differ between the various descriptor-based flow modes. In addition to configurable descriptor set size, descriptor-based DMA also allows for altering of the flow mode of the next descriptor set. Programs can transition from one descriptor-based mode to another descriptor-based mode and can also transition to any of the register-based flow modes.

Descriptor-Array Mode

When configured in this mode, the descriptor sets do not contain further descriptor pointers. Software writes the initial descriptor-pointer value, which points to an array of descriptors. This operation assumes that the individual descriptors reside next to each other and assumes that their addresses are known.

The *Offsets for Descriptor-Array Mode Parameters and Descriptors* table illustrates how to structure a descriptor set in memory. The descriptor sets must reside in a contiguous block of memory in the format shown in the table. Locate the first descriptor of the next descriptor set in the memory location immediately following the last descriptor of the current descriptor set. The values have the same order as the corresponding offset addresses of the memory-mapped register.

Table 30-9: Offsets for Descriptor -Array Mode Parameters and Descriptors

Descriptor Offset	Parameter Register
0x00	DMA_ADDRSTART
0x04	DMA_CFG
0x08	DMA_XCNT
0x0C	DMA_XMOD
0x10	DMA_YCNT
0x14	DMA_YMOD

All other DMA channel registers not loaded as a result of the descriptor set fetch retain their previous values. The channel reloads all of the current registers between the descriptor set fetch and the start of the DMA operation for the work unit.

NOTE: At a minimum, write the [DMA_DSCPTR_CUR](#) register prior to writing to the [DMA_CFG](#) register, which is the special action required to start the DMA channel.

Descriptor-List Mode

In this flow mode, multiple descriptors form a chained list in which each descriptor set contains a pointer to the next descriptor set, allowing greater flexibility in memory layout options. When the channel fetches the descriptor set, the operation loads this pointer value into the next descriptor pointer register of the DMA channel.

Descriptor Sets

The *Offsets for Descriptor-List Mode Parameters and Descriptors* table shows how to structure a descriptor set in memory. Disperse the placement of the descriptor sets throughout memory, having sets reside in different memory blocks. But, each descriptor of the descriptor set must reside in a contiguous section of memory in the format shown in the table. The values have the same order as the corresponding offset addresses of the memory-mapped registers.

Table 30-10: Offsets for Descriptor-List Mode Parameters and Descriptors

Descriptor Offset	Parameter Register
0x00	DMA_DSCPTR_NXT
0x04	DMA_ADDRSTART
0x08	DMA_CFG
0x0C	DMA_XCNT
0x10	DMA_XMOD
0x14	DMA_YCNT
0x18	DMA_YMOD

All other DMA channel registers not loaded as a result of the descriptor set fetch retain their previous values. The channel reloads all of the current values of the registers between the descriptor set fetch and the start of the DMA operation for the work unit.

Minimum Startup Requirements

At a minimum, write the [DMA_DSCPTR_NXT](#) register prior to write to the [DMA_CFG](#) register, which is the special action required to start the DMA channel.

Descriptor-On-Demand Modes

The [Descriptor-Array Mode](#) and [Descriptor-List Mode](#) each have an on-demand mode of operation.

In on-demand mode, at the end of the work unit, if the DMA channel has not detected an incoming trigger event, the channel repeats the current work unit. If the DMA channel receives an incoming trigger before completion of the work unit, the channel fetches a new descriptor set.

The *Offsets for Descriptor-Array Mode Parameters and Descriptors* and *Offsets for Descriptor-List Mode Parameters and Descriptors* tables illustrate how to structure each descriptor set in memory.

Table 30-11: Offsets for Descriptor-Array Mode Parameters and Descriptors

Descriptor Offset	Parameter Register
0x00	DMA_ADDRSTART
0x04	DMA_CFG
0x08	DMA_XCNT
0x0C	DMA_XMOD
0x10	DMA_YCNT
0x14	DMA_YMOD

NOTE: For descriptor-array mode, at a minimum, write the [DMA_DSCPTR_CUR](#) register prior to writing to the [DMA_CFG](#) register, which is the special action required to start the DMA channel.

Table 30-12: Offsets for Descriptor-List Mode Parameters and Descriptors

Descriptor Offset	Parameter Register
0x00	DMA_DSCPTR_NXT
0x04	DMA_ADDRSTART
0x08	DMA_CFG
0x0C	DMA_XCNT
0x10	DMA_XMOD
0x14	DMA_YCNT
0x18	DMA_YMOD

NOTE: For descriptor-list mode, at a minimum, write the [DMA_DSCPTR_NXT](#) register prior to write to the [DMA_CFG](#) register, which is the special action required to start the DMA channel.

Data Transfer Modes

In addition to supporting basic one-dimensional DMA transfers, the DMA channel also supports two-dimensional functionality.

Two-Dimensional DMA

Register-based flow modes and descriptor-based flow modes support two-dimensional data transfers.

In two-dimensional (2D) mode, the X-direction count ([DMA_XCNT](#)), the X-direction modifier ([DMA_XMOD](#)), the Y-direction count ([DMA_YCNT](#)), and the Y-direction modifier ([DMA_YMOD](#)) support arbitrary row and column sizes. Also, the modify values can be negative, allowing implementation of interleaved data streams. The [DMA_XCNT](#) value specifies the row size, and the [DMA_YCNT](#) value specifies the column size; where the [DMA_XCNT](#) value must be 2 or greater.

The DMA start address (`DMA_ADDRSTART`), the X-direction modifier (`DMA_XMOD`), and the Y-direction modifier (`DMA_YMOD`) specifications all are in bytes. The alignment must be a multiple of the DMA transfer word size; configured using the `DMA_CFG.MSIZE` bit. Misalignment results in a DMA channel error.

The `DMA_XMOD` register value is the byte-address increment that the channel applies after each transfer, decrementing the `DMA_XCNT` register. The channel does not apply the `DMA_XCNT` when the inner loop count ends with the `DMA_XCNT_CUR` register decrementing to 0 from 1. Except, the channel does apply the `DMA_XCNT` on the final transfer, when the `DMA_YCNT` register is 1 and the `DMA_XCNT` register decrements from 1 to 0.

The `DMA_YMOD` register value is the byte-address increment that the channel applies after each decrement of the value in `DMA_YCNT_CUR`. However, the channel does not apply the `DMA_YMOD` value to the last item in the array on which the outer loop count (`DMA_YCNT_CUR`) also expires by decrementing from 1 to 0.

After the last transfer completes, `DMA_YCNT_CUR` is 1 and the `DMA_XCNT_CUR` register is 0. The DMA channels current address points to the last items address plus the `DMA_XMOD` register value. If the DMA channel programming selects automatic refresh (such as in autobuffer mode), the channel reloads the `DMA_XCNT_CUR`, `DMA_YCNT_CUR`, and `DMA_ADDR_CUR` for the first data transfer of the next work unit.

Interrupt notification is configurable for end-of-row or end-of-work unit completion.

For example, two-dimensional DMA can be used to extract interleaved data (such as RGB values for a video frame) by modifying both of the `DMA_XMOD` and `DMA_YMOD` values. The *Capturing a Video Data Stream 2D DMA Example* depicts the process of receiving a stream of the R, G, B values from an N*M frame. The inner loop of the 2D DMA configuration has three values (`DMA_XCNT` = 3) and a stride (`DMA_XMOD`) of N*M, chosen such that successive elements in each row are 1-2-3, 4-5-6 and so forth. The outer loop of the 2D DMA configuration has N*M values (`DMA_YCNT` = N*M) and a negative stride (`DMA_YMOD`) of $1-2*N*M$ chosen to instruct the DMA controller to jump from element 3 to 4, 6 to 7 and so forth at the end of each inner loop.

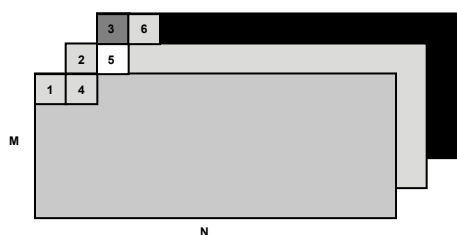


Figure 30-4: Capturing a Video Data Stream 2D DMA Example

DMA Channel Event Control

The DMA channel supports a number of events that provide notification of work unit state, peripheral data request, peripheral interrupt request and completion events, and DMA channel error conditions. In addition to flexible interrupt configuration, the DMA channel also supports incoming and outgoing triggers which are useful in synchronizing the DMA channel with other system resources.

The DMA channel has two interrupt signals for support of a number of events such as work-unit state events, peripheral interrupt request (PIRQ) events, peripheral data request (PDR) events, and DMA channel errors. The

channel reports DMA channel errors on a dedicated interrupt signal. All other interrupt sources share an interrupt signal. In addition to flexible interrupt configuration, the DMA channel also supports incoming and outgoing triggers which are useful in synchronizing the DMA channel with other system resources.

The channel can signal the processor on DMA channel events using status information and optional interrupt requests. Programs can use these events to update the progress of data transfers and to request intervention from the processor core. Configure most DMA channel interrupts using bits in the `DMA_CFG` register. Dedicated bits in the `DMA_STAT` register report the occurrence of various events. Use write-one-to-clear (W1C) operations to clear interrupt requests from the status register.

NOTE: Hardware does not clear the interrupt status bits automatically, even when programs disable then reenable the DMA channel. In this situation, the channel deasserts the interrupt signal, after the program disables the DMA channel. But, the status bit remains set until software either re-enables the DMA channel or clears the status bit.

The DMA channel supports the following categories of events on the interrupt signals:

- Work-unit state events generate interrupts on row or on work unit DMA completion.
- A peripheral uses peripheral interrupt request (PIRQ) events to signal when it has completed the transfer of all data.
- A peripheral uses peripheral data request (PDR) events to request data from a disabled or idle DMA channel.
- Error events signal a failure in the work unit.

ATTENTION: While in an error state, the DMA channel does not generate an interrupt to the processor for a work-unit state event or a PIRQ event, nor does the channel forward a PDR event.

Event Signals

The *Event Signals* table provides descriptions of DMA channel events.

Table 30-13: Event Signals

Signal	Width (bits)	Description
DMA_ERROR	1	Used to signal an error condition in the DMA channel. The source of the error can be determined by reading the <code>DMA_STAT.ERRC</code> bit.
DONE_PIRQ_INT	1	Signal used to indicate DMA completions events, PIRQ events and also for forwarding PDR events based on configuration. Read the corresponding fields in <code>DMA_STAT</code> to determine the source of the event.
DMA_TRIG_OUT	1	Trigger output that gets routed to the TRU and can be configured to provide notification on row or work unit completion.
DMA_TRIG_IN	1	Trigger input from the TRU that can be used to control the start of a work unit.

Work Unit State Events

Completing a row or a work unit generates a work-unit state event. For either of these events to generate an interrupt request, the configuration of the interrupt of the DMA channel must select one of the available work-unit completion modes.

- Current X count reaching 0 for row completion or 1D DMA work unit completion
- Current Y count reaching 0 for work unit completion of 2D DMA

NOTE: For 1D DMA, a DMA channel configuration error results if the configuration generates the interrupt request when the current Y counter reaches 0.

The DMA channel issues the last memory read or write transaction for the row or work unit, then pauses until the return of the read or write acknowledge. After successful acknowledge of the transfer, the DMA channel issues the interrupt request and continues to process the next row or work unit.

Waiting for acknowledgement of the memory access results in a delay. However, programs can read or modify data in the memory without adversely affecting or being affected by the DMA transfer.

NOTE: While the DMA channel pauses waiting for acknowledgement of the memory transfer, the DMA channel is still capable of fetching the next descriptor set. This fetch gets the channel ready to process the next work unit as soon as the memory access completes.

The channel configuration of the synchronization feature also affects interrupt timing. For memory-read operations with synchronization enabled, the channel delays the interrupt request until the completion of the last transfer from the DMA channel FIFO to the peripheral. The synchronization feature does not affect interrupt timing for memory write operations.

Peripheral Interrupt Request Events

For peripheral-transmit operations, a peripheral connected to the DMA channel can use peripheral interrupt request (PIRQ) events to indicate that data has left the channel FIFO and to indicate transfer completion.

In order to support PIRQ interrupts, correctly configure the interrupt of the DMA channel. This configuration disables the generation of interrupt requests based on the work unit state and, instead, results in generating an interrupt request when the DMA channel receives the command from the peripheral.

The channel only generates the interrupt request under the following conditions:

- The configuration enables the DMA channel
- The DMA channel is in the stop state
- The configuration of the DMA channel interrupt selects PIRQ operation

Peripheral Data Request Events

Peripheral data request (PDR) events occur when an interfaced peripheral requests data from the DMA channel and the DMA channel (either disabled or enabled) is in the stop state.

When a peripheral sends a data request command to a disabled DMA channel, the DMA channel generates an interrupt to the System Event Controller (SEC). There is no status information reported about this event in the status register of the DMA channel. Instead, the channel identifies the PDR event from the fact that the DMA channel generated an interrupt while disabled. It is possible to further confirm event status by verifying the status of the peripheral interfaced to the DMA channel.

This operation forwards data requests as interrupts when the DMA channel is in the disabled state. Also, the DMA channel is able to forward PDR events as an interrupt request when the DMA channel is in the stop state after the completion of a work unit. The forwarding of this interrupt when the DMA channel is in the stop state is optional and configured by the program during DMA channel configuration.

DMA Channel Triggers

DMA channel triggers are useful for synchronizing the DMA channel with other events in the system. One usage is to combine channel triggers with each other to create ping-pong buffers. Another usage is to combine the triggers with interrupt requests to notify the processor on reaching a particular milestone that requires service. The channel also can use triggers to enforce a handshake DMA operation in which the trigger acts as a signal for a DMA request.

NOTE: Using the trigger to control the pace of data transfers, such as for handshake DMA, requires that all the data for the entire work unit is ready for transfer.

The DMA channel has a single incoming trigger that can control the pace of the data transfers performed by the DMA channel. The configuration can direct the DMA channel to wait for the incoming trigger before starting the work unit transfer or fetching a descriptor set from memory.

The DMA channel also has a single outgoing trigger signal. This configuration can direct this trigger to signal the end of row or an entire work unit. The DMA channel issues the last memory read or memory write transaction for the row or work unit, then pauses until return of the transfer acknowledge. After acknowledgement of the transfer, the DMA channel issues the trigger before processing the next row or work unit.

Issuing Triggers

The DMA channel configuration can direct the channel to generate an outgoing trigger signal at the end of row or the end of a work unit. The DMA channel issues the last memory read or memory write transaction for the row or work unit, then pauses until the return of the transfer acknowledge. After acknowledgement of the transfer, the DMA channel issues the trigger before processing the next row or work unit.

NOTE: While the DMA channel pauses waiting for acknowledgement of the memory transfer, the DMA channel is still capable of fetching the next descriptor set. This fetch gets the channel ready to process the next work unit as soon as the memory access completes.

Waiting For Triggers

Programs can use triggering to control the pace of data transfers performed by the DMA channel. The DMA channel enters a wait state before beginning the next work unit if the configuration enables `DMA_CFG.TWAIT` and either of the following apply:

- The channel receives a trigger since the last time the DMA channel left the wait state.
- The channel receives a trigger since its transition from disable to enable.

In the wait state, the DMA channel also does not perform a descriptor fetch. After receiving a trigger, the DMA channel leaves the wait state and begins the next work unit or fetches the next descriptor if configured for a descriptor-based mode of operation.

If a memory-mapped register write operation programs the channel with stop flow mode enabled (`DMA_CFG.TWAIT` bit) and the channel has not already received a trigger, the DMA channel enters a wait state before performing the data transfer. On receiving the trigger, the DMA channel begins the data transfer portion of the work unit. Once the data transfer is complete, the DMA channel enters the stop state.

If a memory-mapped register write operation programs the DMA channel with the flow mode configured to one of the descriptor-based modes, the DMA channel enters the wait state before performing the descriptor fetch. After completing the descriptor fetch, the DMA channel immediately proceeds to the data transfer, regardless of the value of the `DMA_CFG.TWAIT` bit. If another (next) descriptor fetch follows the descriptor fetch, the DMA channel enters a wait state before fetching the next descriptor.

If the descriptor fetch returns a descriptor with stop flow mode, the `DMA_CFG.TWAIT` value for that descriptor does not affect the DMA as the channel enters the stop state after completing the data transfer. The DMA channel only enters the wait state based on `DMA_CFG.TWAIT` before the next work unit or descriptor fetch.

If the descriptor fetch returns a descriptor configured for autobuffer flow mode, the `DMA_CFG.TWAIT` for that descriptor does not affect the DMA for the first work unit of the autobuffer transfer. After completing the first work unit and not receiving another trigger, the DMA channel enters the wait state before reinitializing its counters and address registers (if not configured for current addressing). The channel performs the next work unit after receiving the trigger.

The incoming trigger can occur when the DMA channel has not entered the wait state. The trigger can occur while the DMA channel is executing a work unit, is performing descriptor fetch, or is in the stop state. The trigger is held internally. After the work unit is complete, the DMA channel skips the wait state and proceeds directly to executing the following work unit. If the `DMA_CFG.TWAIT` bit is not enabled, the DMA channel also skips the wait state. However, the trigger is held internally and is used the next time the configuration enables `DMA_CFG.TWAIT`. This trigger retention allows programs to enable the `DMA_CFG.TWAIT` functionality several work units apart without concern for losing a trigger. The DMA channels trigger-overflow enable functionality can be enabled in all work units to ensure that multiple triggers do not occur between the work units with the `DMA_CFG.TWAIT` bit enabled.

DMA Channel Programming Model

Several synchronization and control methods are available for use in development of software tasks which manage peripheral DMA and memory DMA. Software must accept requests for new DMA transfers from other software tasks, integrate these transfers into existing transfer queues, and reliably notify other tasks when the transfers are complete.

In the processor, it is possible to manage each peripheral DMA and memory DMA stream with a separate task or to manage them together with any other stream. Each DMA channel has independent, orthogonal control registers, resources, and interrupts. So, the selection of the control scheme for one channel does not affect the choice of control scheme on other channels. For example, one peripheral can use a linked-descriptor-list, interrupt-driven scheme while another peripheral can simultaneously use a demand-driven, buffer-at-a-time scheme synchronized by polling DMA events.

The topics that follow describe the steps required to configure the DMA channel for the various modes in addition to the programming concepts required for software synchronization.

Mode Configuration

Use the step-by-step directions that follow to set up the DMA channel for operating modes.

Register-Based Linear-Buffer Stop Flow Mode

This procedure configures the DMA channel of a peripheral to read data from internal memory and to send it to the peripheral for transmission. Upon DMA completion, the DMA channel enters the idle state until either disabled or reconfigured for a new transfer.

Assume that the peripheral is in a state where it is ready to transmit data received from the DMA channel.

The task involves writing to a number of DMA channel MMR registers to configure a DMA channel to:

- Read data from internal memory, and
- Send it to a peripheral connected to the peripheral DMA bus.

1. Write the `DMA_ADDRSTART` register.

ADDITIONAL INFORMATION: Software can use the address to calculate the most optimum possible `DMA_CFG.MSIZE`.

2. Calculate the optimum `DMA_CFG.MSIZE` based on the `DMA_ADDRSTART` register and number of bytes in work unit.

ADDITIONAL INFORMATION: The number of bytes in the work unit must be a multiple of the selected `DMA_CFG.MSIZE`, and the calculation also must consider the start address alignment.

3. Write the `DMA_XCNT` register based on the calculated `DMA_CFG.MSIZE`.

ADDITIONAL INFORMATION: The `DMA_XCNT` value is the number of `DMA_CFG.MSIZE` transfers to make up the entire work unit.

4. Write the `DMA_XMOD` register.

ADDITIONAL INFORMATION: For a linear buffer transfer, determine the value in `DMA_XMOD` from the selected `DMA_CFG.MSIZE`. Always specify this register as a number of bytes.

5. Write the `DMA_CFG` register with `DMA_CFG.EN` configured to enable the DMA channel.

ADDITIONAL INFORMATION: Set the `DMA_CFG.FLOW` bit for STOP mode. Configure the `DMA_CFG.WNR` bit for memory read operation. Configure the `DMA_CFG.PSIZE` bits to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used.

- The `DMA_CFG.SYNC` bit can be configured to control DMA completion notification timing.
- Interrupts and triggers also can be configured at this step depending on requirements.

Now, the DMA channel is enabled, and the buffer is transferred. The DMA channel enters the IDLE state upon completion of the work unit.

Register-Based Autobuffer Flow Mode

This procedure configures the DMA channel of a peripheral to read data from internal memory and send it to the peripheral for transmission. The transmission of the buffer repeats endlessly. Upon DMA completion, the DMA channel restarts the DMA operation, creating an endless circular buffer transfer.

Assume the peripheral is in a state where it is ready to transmit data received from the DMA channel.

The task involves writing to a number of DMA channel MMR registers to configure a DMA channel to:

- Read data from internal memory, and
- Send it to a peripheral connected to the peripheral DMA bus.

1. Write the `DMA_ADDRSTART` register.

ADDITIONAL INFORMATION: Use the address to calculate the optimum possible `DMA_CFG.MSIZE`.

2. Calculate the optimum `DMA_CFG.MSIZE` based on the `DMA_ADDRSTART` register and number of bytes in work unit.

ADDITIONAL INFORMATION: The number of bytes in the work unit must be a multiple of the selected `DMA_CFG.MSIZE`, and the calculation must consider the start address alignment.

3. Write the `DMA_XCNT` register based on calculated `DMA_CFG.MSIZE`.

ADDITIONAL INFORMATION: The `DMA_XCNT` register value is the number of `DMA_CFG.MSIZE` transfers to make up the entire work unit.

4. Write the `DMA_XMOD` register.

ADDITIONAL INFORMATION: For a linear buffer transfer, determine the value in `DMA_XMOD` from the selected `DMA_CFG.MSIZE`. Always specify this register as a number of bytes.

5. Write the `DMA_CFG` register with the `DMA_CFG.EN` bit configured to enable the DMA channel.

ADDITIONAL INFORMATION: Set the `DMA_CFG.FLOW` bit for autobuffer mode. Configure the `DMA_CFG.WNR` bit for memory read operation. Configure the `DMA_CFG.PSIZE` bit to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used.

- The `DMA_CFG.SYNC` bit can be configured to control DMA completion notification timing.
- Interrupts and triggers also can be configured at this step depending on requirements.

Now, the DMA channel is enabled, and the buffer transfers until the DMA channel is disabled.

Descriptor-Array Flow Mode

This procedure configures the DMA channel of a peripheral to:

- Read data from memory as described by the descriptor sets in the array, and
- Send the data to the peripheral for transmission.

Descriptor sets are read from an array in memory to configure the individual work units.

Assume the peripheral is in a state where it is ready to transmit data received from the DMA channel. Assume that the array of descriptors is to be initialized with the last descriptor set configured for STOP flow mode.

The task involves writing to a number of DMA channel MMR registers to:

- Configure a DMA channel to read the array in memory, containing the first descriptor set that configured the DMA channel to retrieve, and
- Send the data to a peripheral connected to the peripheral DMA bus.

On DMA completion, the DMA channel enters the idle state until either disabled or reconfigured for a new transfer.

1. Write the `DMA_DSCPTR_CUR` register with the address of the array in which the descriptor sets are stored.

ADDITIONAL INFORMATION: The array address must meet any processor alignments restrictions imposed by descriptor fetches.

2. Write the `DMA_CFG` register with the `DMA_CFG.EN` bit configured to enable the DMA channel.

ADDITIONAL INFORMATION: Configure the `DMA_CFG.PSIZE` bits to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used. Set the `DMA_CFG.FLOW` bit for descriptor-array mode. Configure the

`DMA_CFG.NDSIZE` bits to describe the number of descriptor elements contained within the first descriptor set. Configure the `DMA_CFG.WNR` bit for memory read operation.

- The descriptor set that is fetched controls the `DMA_CFG.SYNC` configuration and the interrupt or trigger configurations.

The first descriptor set is fetched from memory location provided by the `DMA_DSCPTR_CUR` register and loaded to the MMR registers of the DMA channel.

Now, the DMA channel is processing all the work units provided in the descriptor array. The DMA channel enters the IDLE state on completion of the final work unit that was configured for STOP flow mode.

Descriptor-List Flow Mode

This procedure configures the DMA channel of a peripheral to:

- Read data from memory as described by the descriptor sets in the list, and
- Send it to the peripheral for transmission.

The DMA controller reads the descriptor sets from a list of descriptors. With the list, each descriptor set has a descriptor that points to the next descriptor set location in memory.

Assume the peripheral must be in a state where it is ready to transmit data received from the DMA channel. Assume that the list of descriptors must be initialized with the last descriptor set in the list configured for Stop flow mode.

The task involves writing to a number of DMA channel MMR registers to:

- Configure a DMA channel to read the list in memory, containing the first descriptor set that configured the DMA channel to retrieve, and
- Send the data to a peripheral connected to the peripheral DMA bus.

On DMA completion, the DMA channel enters the idle state until either disabled or reconfigured for a new transfer.

1. Write the `DMA_DSCPTR_NXT` register with the address of the first descriptor in the list to be processed.

ADDITIONAL INFORMATION: The array address must meet any processor alignments restrictions imposed by descriptor fetches.

2. Write the `DMA_CFG` register with the `DMA_CFG.EN` configured to enable the DMA channel.

ADDITIONAL INFORMATION: Set the `DMA_CFG.FLOW` for descriptor-list mode. Configure the `DMA_CFG.NDSIZE` bit to describe the number of descriptor elements contained within the first descriptor set. Configure the `DMA_CFG.WNR` bit for memory read operation. Configure the `DMA_CFG.PSIZE` bit to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is

recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used.

- The descriptor set that is fetched controls the `DMA_CFG.SYNC` configuration and controls the interrupt or trigger configurations.

The first descriptor set is fetched from the memory location provided by `DMA_DSCPTR_NXT` and is loaded to the MMR registers of the DMA channel.

Now, the DMA channel is processing all the work units provided in the descriptor list. The DMA channel enters the idle state when the final work unit that was configured for stop-flow mode is complete.

Register-Based Memory-to-Memory Transfer in Stop Flow Mode

This procedure configures a memory DMA channel pair in stop flow mode. One DMA channel is configured for memory read operations, while the other DMA channel is configured for memory write.

The task involves writing to a number of DMA channels on two DMA channels that create a memory DMA pair. On DMA completion, the DMA channel enters the idle state, until either the DMA channel is disabled or is reconfigured for a new transfer.

1. Write the `DMA_ADDRSTART` register of the source DMA channel.

ADDITIONAL INFORMATION: The address can be used to calculate the optimum `DMA_CFG.MSIZE` possible.

2. Calculate the optimum `DMA_CFG.MSIZE` based on the `DMA_ADDRSTART` register and number of bytes in work unit.

ADDITIONAL INFORMATION: The number of bytes in the work unit must be a multiple of the selected `DMA_CFG.MSIZE` and the start address alignment must also be considered.

3. Write the `DMA_XCNT` register of the source DMA channel based on calculated `DMA_CFG.MSIZE`.

ADDITIONAL INFORMATION: `DMA_XCNT` is the number of `DMA_CFG.MSIZE` transfers to make up the entire work unit.

4. Write the `DMA_XMOD` register of the source DMA channel.

ADDITIONAL INFORMATION: For a linear buffer transfer, determine the value in `DMA_XMOD` from the selected `DMA_CFG.MSIZE`. This register is always specified in the number of bytes.

5. Write the `DMA_ADDRSTART` register of the destination DMA channel.

ADDITIONAL INFORMATION: The address can be used to calculate the most optimum `DMA_CFG.MSIZE` possible.

6. Calculate the optimum `DMA_CFG.MSIZE` based on the `DMA_ADDRSTART` register and number of bytes in work unit.

ADDITIONAL INFORMATION: The number of bytes in the work unit must be a multiple of the selected `DMA_CFG.MSIZE` and the start address alignment must also be considered.

7. Write the `DMA_XCNT` register of the destination DMA channel based on the calculated `DMA_CFG.MSIZE`.

ADDITIONAL INFORMATION: `DMA_XCNT` is the number of `DMA_CFG.MSIZE` transfers to make up the entire work unit.

8. Write the `DMA_XMOD` register of the destination DMA channel.

ADDITIONAL INFORMATION: For a linear buffer transfer, determine the value in `DMA_XMOD` from the selected `DMA_CFG.MSIZE`. This register is always specified in the number of bytes.

9. Write the `DMA_CFG` register of the source DMA channel with `DMA_CFG.EN` configured to enable the DMA channel.

ADDITIONAL INFORMATION: The `DMA_CFG.FLOW` bit field must be configured for stop mode. The `DMA_CFG.WNR` bit must be cleared for memory read operation. The `DMA_CFG.PSIZE` bits must be configured to a value no larger than the supported bus width of the peripheral DMA bus. From performance perspective, it is recommended to use the largest possible `DMA_CFG.PSIZE` value (for example, equal to the supported peripheral bus width (`DMA_STAT.PBWID`)). However, ensure that the number of bytes in the work unit is a multiple of the `DMA_CFG.PSIZE` value used.

- The `DMA_CFG.SYNC` bit can be configured to control DMA completion notification timing.
- Interrupts and triggers also can be configured at this step, depending on requirements. The interrupts and triggers are enabled within the destination DMA channel configuration.

The memory read DMA transfer begins.

10. Write the `DMA_CFG` register of the destination DMA channel with `DMA_CFG.EN` configured to enable the DMA channel.

ADDITIONAL INFORMATION: The `DMA_CFG.FLOW` bit field must be configured for stop mode. The `DMA_CFG.WNR` bit must be set for memory write operation. The `DMA_CFG.PSIZE` bits must be configured to a value no larger than the supported bus width of the peripheral DMA bus. This value must also match the value written for the source DMA channel configuration.

- Interrupts and triggers also can be configured at this step depending on requirements.

The memory write DMA transfer begins.

Both memory DMA channels are now running and the data is transferred from the source address to the destination address. The DMA channel enters the IDLE state upon completion of the work unit.

Programming Concepts

Using the features, operating modes, and event control for the DMA channel to their greatest potential requires an understanding of some DMA channel-related concepts.

Synchronization of Software and DMA

A critical element of software DMA management is the synchronization of DMA work unit completion with software. This synchronization can be achieved using DMA channel interrupt request and trigger events and using a poll of the status bits of these events within the DMA channel registers, or combining these techniques. Processor polling of DMA address/count/status for completion is not a recommended programming practice. The requirements and limitations of processor polling place significant responsibility onto the code developer to be deeply aware of the underlying hardware. The interrupt requests and triggers are designed for efficient code development and reuse.

Interrupt and Trigger Event-Based Synchronization

Interrupt and trigger based synchronization methods must avoid overrun. An overrun occurs when some events fail to invoke the event handler of a DMA channel for every event due to excessive latency in processing of events. The system design must ensure to either:

- Schedule only one event per channel (for example, at the end of a descriptor list), or
- Space the generated events sufficiently far apart in time that system processing budgets can guarantee service of every event.

The DMA channel issues status information through an interrupt request or trigger event or changes event status bits in the `DMA_STAT` register. This status guarantees that the last memory operation of the work unit is complete. For memory read DMA transactions, this status means that the FIFO of the DMA channel safely receives the final memory read data. For DMA transactions writing to memory, this status indicates that the DMA channel received an acknowledge of completion of the last write transfer of the work unit.

Register Polling Based Synchronization

Do not poll the DMA channel registers (`DMA_ADDR_CUR`, `DMA_DSCPTR_CUR`, `DMA_XCNT_CUR`, or `DMA_YCNT_CUR`) as a method of precisely synchronizing DMA with data processing. This approach is inaccurate due to the operation of the DMA channel FIFOs and DMA or memory pipelining. The current address, pointer, and count registers change several cycles in advance of the completion of the corresponding memory operation. This timing is measurable from the time at which the results of the operation are first visible to the core by memory read or write instructions.

For example, in a DMA channel memory write operation to external memory, assume DMA channel *A* initiates a DMA channel write operation. For memories with access latency, this operation requires many system-clock cycles. Meanwhile, DMA channel *B* (which does not in itself incur latency) initiates a transfer, which stalls behind the slow operation of channel *A*. Software monitoring channel *B* could not safely conclude whether the memory location pointed to by the `DMA_ADDR_CUR` of channel *B*. Also, the software cannot conclude whether the register has been written based solely on the contents of this register.

Polling of the current address, pointer, and count registers can permit loose synchronization of DMA with software. But, the software must allow for the lengths of the DMA or memory pipeline. Also, software must consider the length of the DMA FIFO for a particular peripheral. If the FIFOs are filled with incomplete work, the DMA channel does not advance current address, pointer, or count registers. The incomplete work includes reads that have been started but have not yet finished.

Additionally, software must consider the length of the pipelines to the destination memory. If the DMA FIFO length and channel memory-pipeline length are added, software can estimate the maximum number of incomplete memory operations in progress.

NOTE: The estimate would be a maximum, as the DMA or memory pipeline can include traffic from other DMA channels.

Descriptor Queues

A system designer may want to write a DMA manager facility which accepts DMA requests from other software. The DMA manager software does not know in advance when new work requests are received or what these requests contain. The software could manage these transfers using a circular linked list of DMA descriptors. In such a list, each descriptor sets the `DMA_DSCPTR_NXT` descriptor, which points to the next descriptor set. And, the last descriptor set in the list points to the first descriptor set.

The code that writes into this descriptor list could use the circular addressing modes of the processor. This approach does not need to use comparison and conditional instructions to manage the circular structure. In this case, the `DMA_DSCPTR_NXT` descriptor of each descriptor set can be written once at startup, and skipped over as new contents are written for each descriptor.

The recommended method for synchronization of a descriptor queue is to use an interrupt or trigger. The descriptor queue is structured, such that (at least) the final valid descriptor set is always programmed to generate an interrupt or trigger event upon completion. More detail is provided in the following sections.

- [Queues Using Event Generation for Every Descriptor Set](#)
- [Queues Using Minimal Events](#)

Queues Using Event Generation for Every Descriptor Set

In this system, the DMA manager software synchronizes with the DMA channel by enabling an interrupt request or trigger on every descriptor set. Only use this method if the system design can guarantee that each work unit completion event is serviced separately (no interrupt or trigger overrun).

To maintain synchronization of the descriptor set queue, the non-interrupt software maintains a count of descriptor sets added to the queue. The event handler (either interrupt or trigger) maintains a count of completed descriptor sets removed from the queue. The counts are equal only when the DMA channel is paused after having processed all the descriptor sets.

When each new work unit event is received, the DMA manager software initializes a new descriptor set, taking care to set the flow to stop mode. Next, the software compares the descriptor set counts to determine whether the DMA channel is running. If the DMA channel is paused (counts equal), the software increments its count. Then, the software starts the DMA channel by writing the `DMA_CFG` of the new descriptor set.

If the counts are unequal, the software instead modifies the `DMA_CFG` of the next-to-last descriptor set, such that it now describes the newly queued descriptor set. This operation does not disrupt the DMA channel provided the rest of the descriptors of the set are initialized in advance. It is necessary to synchronize the software to the DMA to determine whether the DMA channel read the new or the old `DMA_CFG` value.

The event handler performs the synchronization operation. When an event is detected, the handler reads the `DMA_STAT` register of the DMA channel. If the `DMA_STAT.RUN` bit indicates that the DMA channel is running, the channel has moved on to processing another descriptor. The event handler can increment its count and exit. If the `DMA_STAT.RUN` bit indicates that the channel is not running, the channel is paused because either:

- There are no more descriptor sets to process, or
- The last descriptor set was queued too late

Where *too late* means that the modification of the `DMA_CFG` of the next-to-last descriptor set occurred *after* that descriptor was read into the DMA channel. In this case, the event handler does the following:

- Writes the `DMA_CFG` value appropriate for the last descriptor set to `DMA_CFG` register of the DMA channel,
- Increments the completed descriptor count, and
- Exits

If the event latencies of the system are large enough to cause any of the events to be dropped, this system can fail. An event handler capable of safely synchronizing multiple descriptor set interrupt requests is complex, performing several MMR accesses to ensure robust operation. In such a system environment, a minimal event synchronization method is preferred.

Queues Using Minimal Events

In this system, only one DMA interrupt request or trigger event is generated in the queue at any time. The DMA event handler for this system can also be extremely short. Here, the descriptor queue is organized into an *active* and a *waiting* portion, where events are enabled only on the last descriptor set in each portion.

When each new DMA request is processed, the software fills in the content of a new descriptor set and adds it to the waiting portion of the queue. The `DMA_CFG` descriptor of the descriptor set must have the flow set to stop mode. If more than one request is received before the DMA queue completion event occurs, the non-interrupt code queues later descriptor sets. It forms a waiting portion of the queue separate from the active portion of the queue that the DMA channel is processing. In other words, all but the last active descriptor sets contain flow values for a descriptor-based mode and have no event enable set.

The last active descriptor set has the stop flow mode and an event generation enabled. Also, all but the last waiting descriptor sets are configured for descriptor-based flow modes with no event generation. Only the last waiting descriptor set is configured for stop flow mode and event generation enabled. This configuration ensures that the DMA channel can automatically process the whole active queue before then issuing one event. Also, this arrangement makes it easy to start the waiting queue within the event handler by a single `DMA_CFG` register write.

After queuing a new waiting descriptor, the non-interrupt software leaves a message for its interrupt handler in a memory mailbox location. The location contains the desired `DMA_CFG` value for starting the first waiting descriptor set in the waiting queue (or 0, indicating no waiting descriptors).

The software must not modify the contents of the active descriptor set queue directly once processing by the DMA channel has started, unless careful synchronization measures are taken. In the most straightforward implementation of a descriptor set queue, the DMA manager software never modifies descriptors on the active queue. Instead, the

DMA manager waits until the DMA queue completion event indicates that the processing of the entire active queue is complete.

When a DMA queue completion event is received, the event handler reads the mailbox from the non-interrupt software and writes the value to the `DMA_CFG` register of the DMA channel. This write to a register restarts the queue, effectively transforming the waiting queue to an active queue. The event handler then passes a message back to the non-interrupt software indicating the location of the last descriptor set accepted into the active queue.

However, the event handler can read its mailbox and find a `DMA_CFG` value of zero, indicating there is no more work to perform. It then passes an appropriate message back to the non-interrupt software indicating that the queue has stopped.

The non-interrupt software which accepts new DMA work unit requests must synchronize the activation of a new work unit with the interrupt handler. If the queue has stopped (the mailbox from the event handler is zero), the non-interrupt software must start the queue. (The queue starts by writing the first descriptor sets `DMA_CFG` value to the `DMA_CFG` register of the channel). If the queue is not stopped, the non-interrupt software must not write the `DMA_CFG` register. (This write causes a DMA error). Instead, it must queue the descriptor onto the waiting queue and update its mailbox directed to the event handler.

ADSP-2159x DMA Register Descriptions

The Direct Memory Access module (DMA) contains the following registers.

Table 30-14: ADSP-2159x DMA Register List

Name	Description
<code>DMA_ADDRSTART</code>	Start Address of Current Buffer Register
<code>DMA_ADDR_CUR</code>	Current Address Register
<code>DMA_BWLCNT</code>	Bandwidth Limit Count Register
<code>DMA_BWLCNT_CUR</code>	Bandwidth Limit Count Current Register
<code>DMA_BWMCNT</code>	Bandwidth Monitor Count Register
<code>DMA_BWMCNT_CUR</code>	Bandwidth Monitor Count Current Register
<code>DMA_CFG</code>	Configuration Register
<code>DMA_DSCPTR_CUR</code>	Current Descriptor Pointer Register
<code>DMA_DSCPTR_NXT</code>	Pointer to Next Initial Descriptor Register
<code>DMA_DSCPTR_PRV</code>	Previous Initial Descriptor Pointer Register
<code>DMA_STAT</code>	Status Register
<code>DMA_XCNT</code>	Inner Loop Count Start Value Register
<code>DMA_XCNT_CUR</code>	Current Count (1D) or Intra-row XCNT (2D) Register
<code>DMA_XMOD</code>	Inner Loop Address Increment Register

Table 30-14: ADSP-2159x DMA Register List (Continued)

Name	Description
DMA_YCNT	Outer Loop Count Start Value (2D only) Register
DMA_YCNT_CUR	Current Row Count (2D only) Register
DMA_YMOD	Outer Loop Address Increment (2D only) Register

Start Address of Current Buffer Register

The `DMA_ADDRSTART` register contains the start address of the work unit currently targeted for DMA. This register is read/write prior to enabling the channel, but is read-only after enabling channel.

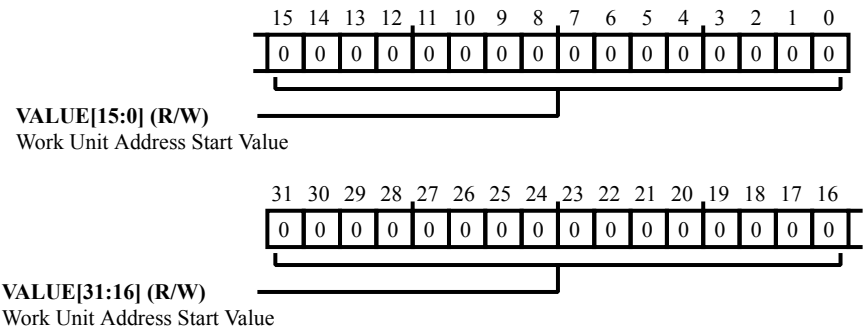


Figure 30-5: DMA_ADDRSTART Register Diagram

Table 30-15: DMA_ADDRSTART Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Work Unit Address Start Value. The <code>DMA_ADDRSTART.VALUE</code> bit field contains the start address of the work unit currently targeted for DMA.

Current Address Register

The `DMA_ADDR_CUR` register contains the present memory transfer address for a given work unit. At the start of a work unit, the `DMA_ADDR_CUR` register is loaded from the `DMA_ADDRSTART` register, and the `DMA_ADDR_CUR` register is incremented as each transfer occurs. The `DMA_ADDR_CUR` register is read/write prior to enabling the channel, but is read-only after enabling the channel.

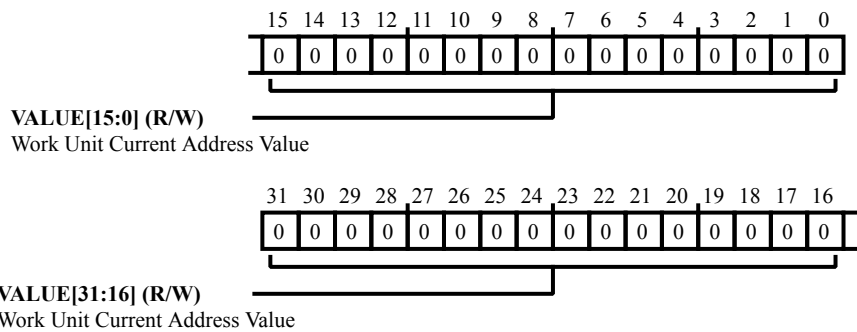


Figure 30-6: DMA_ADDR_CUR Register Diagram

Table 30-16: DMA_ADDR_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Work Unit Current Address Value. The <code>DMA_ADDR_CUR.VALUE</code> bit field contains the present memory transfer address for a given work unit.

Bandwidth Limit Count Register

The `DMA_BWLCNT` register contains a count that determines how often the DMA issues memory transactions. The DMA loads the value from the `DMA_BWLCNT` register into the `DMA_BWLCNT_CUR` register and decrements the current value each CLK06 cycle. When `DMA_BWLCNT_CUR` reaches 0x0000, the next request is issued, and the DMA reloads `DMA_BWLCNT_CUR`. This bandwidth limit functionality is not applied to descriptor fetch requests. Programming 0x0000 allows the DMA to request as often as possible. 0xFFFF is a special case and causes requests to stop.

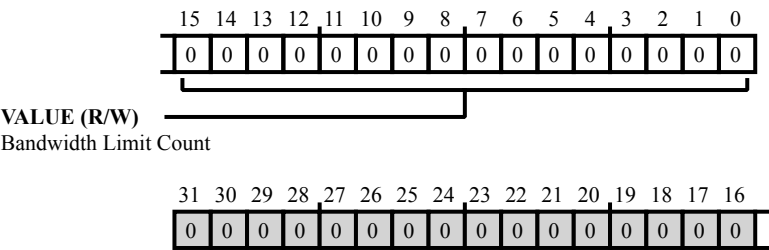


Figure 30-7: DMA_BWLCNT Register Diagram

Table 30-17: DMA_BWLCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Bandwidth Limit Count. The <code>DMA_BWLCNT.VALUE</code> bit field contains a count that determines how often the DMA issues memory transactions.

Bandwidth Limit Count Current Register

The `DMA_BWLCNT_CUR` register contains the number of CLK06 count cycles remaining before the next request is issued.

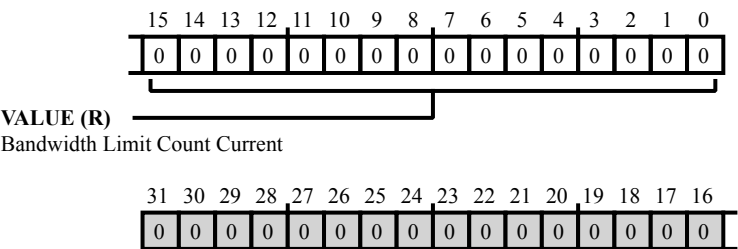


Figure 30-8: DMA_BWLCNT_CUR Register Diagram

Table 30-18: DMA_BWLCNT_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/NW)	VALUE	Bandwidth Limit Count Current. The <code>DMA_BWLCNT_CUR.VALUE</code> bit field contains the number of CLK06 count cycles remaining before the next request is issued.

Bandwidth Monitor Count Register

The `DMA_BWMCNT` register contains the maximum number of CLK06 cycles allowed for a work unit to complete. Each time the `DMA_CFG` register is written (MMR access only), a work unit ends or an autobuffer wraps. The DMA loads the value in this register into the `DMA_BWMCNT_CUR` register.

The DMA decrements `DMA_BWMCNT_CUR` every CLK06 a work unit is active. If the `DMA_BWMCNT_CUR` register reaches 0x0000_0000, the `DMA_STAT . IRQERR` bit is set, and the `DMA_STAT . ERRC` bit field is set to 0x6. The `DMA_BWMCNT_CUR` remains at 0x0000_0000 until it is reloaded when the work unit completes.

Unlike other errors, a bandwidth monitor error does not stop work unit processing. Programming 0x0000_0000 disables bandwidth monitor functionality.

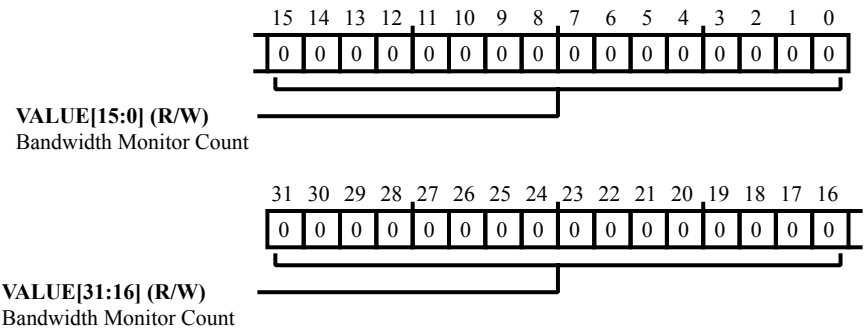


Figure 30-9: DMA_BWMCNT Register Diagram

Table 30-19: DMA_BWMCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Bandwidth Monitor Count. The <code>DMA_BWMCNT . VALUE</code> bit field contains the maximum number of CLK06 cycles allowed for a work unit to complete.

Bandwidth Monitor Count Current Register

The `DMA_BWMCNT_CUR` register contains the number of cycles remaining for the current descriptor to complete.

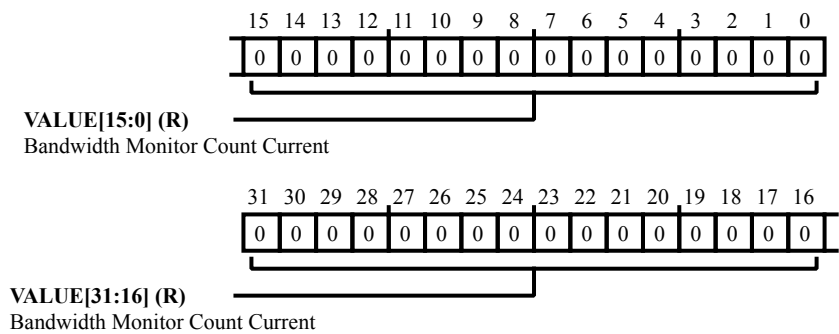


Figure 30-10: `DMA_BWMCNT_CUR` Register Diagram

Table 30-20: `DMA_BWMCNT_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Bandwidth Monitor Count Current. The <code>DMA_BWMCNT_CUR.VALUE</code> bit field contains the number of cycles remaining for the current descriptor to complete.

Configuration Register

The `DMA_CFG` register sets up DMA parameters and operation modes. Writing to the `DMA_CFG` register while a DMA process is already running causes a DMA error (except when clearing the `DMA_CFG.EN` bit).

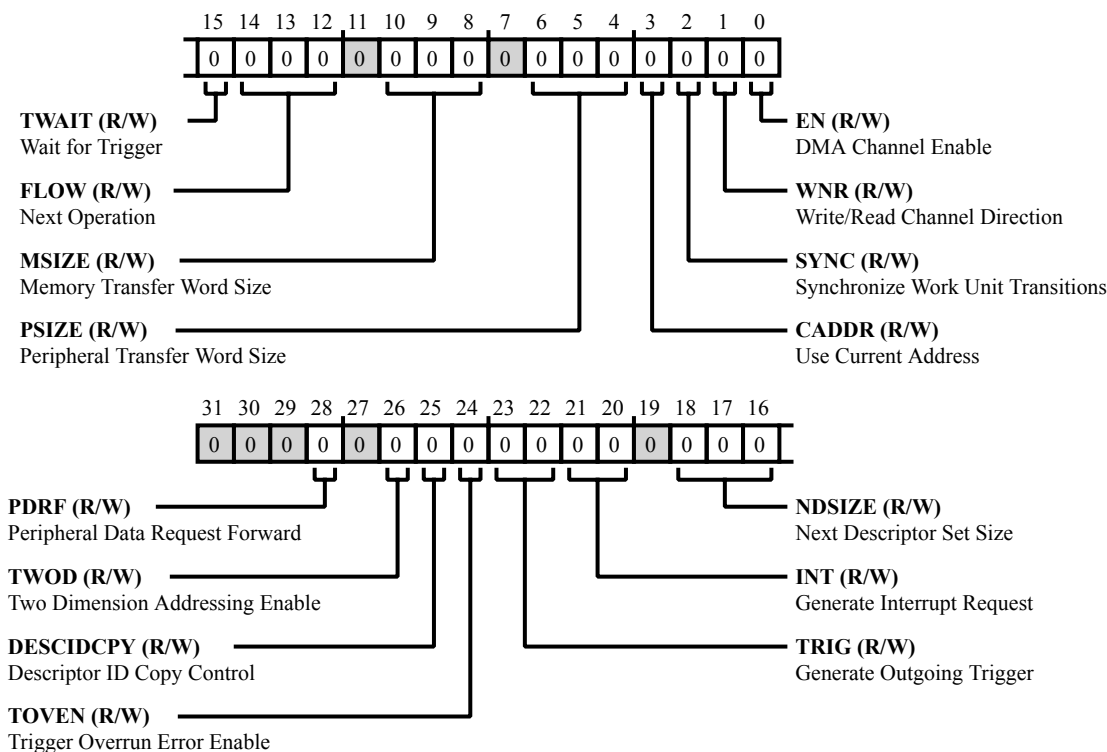


Figure 30-11: `DMA_CFG` Register Diagram

Table 30-21: `DMA_CFG` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	PDRF	Peripheral Data Request Forward.
		The <code>DMA_CFG.PDRF</code> bit defines how the DMA handles data requests from the peripheral while in idle state after a stop mode or memory read work unit. If set, the DMA forwards the peripheral data request as an interrupt.
		This bit applies only to the <code>DMA_CFG.FLOW</code> bits configured for stop and <code>DMA_CFG.WNR</code> bits configured for memory read.
		0 Peripheral Data Request Not Forwarded
		1 Peripheral Data Request Forwarded

Table 30-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/W)	TWOD	Two Dimension Addressing Enable. The DMA_CFG.TWOD bit selects whether the DMA addressing involves only DMA_XCNT and DMA_XMOD (one-dimensional DMA) or also involves DMA_YCNT and DMA_YMOD (two-dimensional DMA).
		0 One-Dimensional Addressing
		1 Two-Dimensional Addressing
25 (R/W)	DESCIDCPY	Descriptor ID Copy Control. The DMA_CFG.DESCIDCPY bit specifies when to copy the initial descriptor pointer to the DMA_DSCPTR_PRV register. A bus write to the DMA_CFG register to clear the DMA_CFG.EN bit causes the DMA to immediately use the new value of the DMA_CFG.DESCIDCPY bit. To preserve consistency (if required by application), match the new value of this bit to the previous value.
		0 Never Copy
		1 Copy on Work Unit Complete
24 (R/W)	TOVEN	Trigger Overrun Error Enable. A trigger overrun occurs if more than one trigger was received before the DMA reached the trigger wait state. If DMA_CFG.TOVEN is set, a trigger overrun causes the DMA to flag an error. In cases where a trigger overrun is not a problem, clearing DMA_CFG.TOVEN prevents the overrun from causing an error and halting the DMA. The DMA_CFG.TOVEN operates independently of the DMA_CFG.TWAIT bit selection.
		0 Ignore Trigger Overrun
		1 Error on Trigger Overrun

Table 30-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
23:22 (R/W)	TRIG	Generate Outgoing Trigger. The DMA_CFG.TRIG selects whether the DMA issues an outgoing trigger, based on the work unit counter values. In one-dimensional mode, the only options are to trigger at the end of the whole work unit (trigger when DMA_XCNT_CUR reaches 0) or not to trigger at all. If in one-dimensional addressing mode, programming the DMA_CFG.TRIG bit field to trigger when DMA_YCNT_CUR reaches 0 (or to reserved) causes the DMA to flag a configuration error. In two-dimensional addressing mode, the trigger options are: at the end of each row of the inner loop (when DMA_XCNT_CUR reaches 0), only after completing the whole work unit (when DMA_YCNT_CUR reaches 0), or no trigger. If in two-dimensional mode and set to trigger when DMA_XCNT_CUR reaches 0, the DMA also issues a trigger at the end of the work unit. If in two-dimensional addressing mode, programming DMA_CFG.TRIG to reserved causes the DMA to flag a configuration error. If DMA_CFG.TRIG is non-zero and the peripheral issues a finish command, the DMA issues a trigger after the finish procedure is complete. For more information about trigger generation timing, see the trigger section of the DMA functional description.
		0 Never Assert Trigger
		1 Trigger When XCNTCUR Reaches 0
		2 Trigger When YCNTCUR Reaches 0
		3 Reserved

Table 30-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21:20 (R/W)	INT	Generate Interrupt Request. The DMA_CFG . INT bit field selects whether an interrupt request goes to the core based on work unit status or a peripheral interrupt request. For one-dimensional mode, setting the DMA_CFG . INT bits to generate an interrupt request when the DMA_YCNT_CUR register reaches 0 causes the DMA to flag a configuration error. The peripheral interrupt setting lets the DMA generate the last grant indication and to accept and or forward the peripheral interrupt command. The peripheral interrupt selection applies only to the DMA_CFG . FLOW bits set for stop and the DMA_CFG . WNR bits set for memory read. If the DMA_CFG . INT is set for interrupt request on count completion (DMA_XCNT_CUR or DMA_YCNT_CUR reach 0) and the peripheral issues a finish command, the DMA issues an interrupt request after the finish procedure is complete. For more information, see the sections on interrupt generation and peripheral control in the DMA functional description.
		0 Never Assert Interrupt
		1 Interrupt When X Count Expires
		2 Interrupt When Y Count Expires
		3 Peripheral Interrupt request
18:16 (R/W)	NDSIZE	Next Descriptor Set Size. The DMA_CFG . NDSIZE bit field specifies the number of descriptor elements in memory to load during the next descriptor fetch. The DMA loads the descriptors in a specific order. The descriptor set contains the next descriptor pointer when it is a descriptor list. The descriptor set does not contain the next descriptor pointer when it is a descriptor array.
		0 Fetch One Descriptor Element
		1 Fetch Two Descriptor Elements
		2 Fetch Three Descriptor Elements
		3 Fetch Four Descriptor Elements
		4 Fetch Five Descriptor Elements
		5 Fetch Six Descriptor Elements
		6 Fetch Seven Descriptor Elements
		7 Reserved

Table 30-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	TWAIT	Wait for Trigger. The DMA_CFG.TWAIT bit controls whether the DMA waits for an incoming trigger from another channel or user. If the DMA_CFG.TWAIT bit is set, the DMA enters the wait state before starting the next work unit, including descriptor fetch when in descriptor mode. Do not use the wait for trigger control for descriptor-on-demand mode which causes an error. For more information, see the trigger section of the DMA functional description.
		0 Begin Work Unit Automatically (No Wait)
		1 Wait for Trigger (Halt before Work Unit)
14:12 (R/W)	FLOW	Next Operation. The DMA_CFG.FLOW bit field selects the descriptor handling options.
		0 STOP. See the Stop Flow Mode section.
		1 AUTO. See the Autobuffer Flow Mode section.
		2 Reserved
		3 Reserved
		4 DSCL. See the Descriptor List Mode section.
		5 DSCA. See the Descriptor Array Mode section.
		6 Descriptor On-Demand List. See the Descriptor List Mode section.
		7 Descriptor On Demand Array. See the Descriptor On Demand section.
10:8 (R/W)	MSIZE	Memory Transfer Word Size. The DMA_CFG.MSIZE bits select memory transfer sizes of 8-, 16-, 32-, 64-, 128-, or 256-bit words. The transfer start address (DMA_ADDRSTART) and transfer increment values (DMA_XMOD , and, if needed, DMA_YMOD) must be a multiple of the memory transfer unit size.
		0 1 Byte
		1 2 Bytes
		2 4 Bytes
		3 8 Bytes
		4 16 Bytes
		5 32 Bytes

Table 30-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:4 (R/W)	PSIZE	Peripheral Transfer Word Size. The DMA_CFG.PSIZE bits select peripheral transfer sizes as 8, 16, 32, or 64 bits wide. Each request and grant results in a single peripheral access. There are no bursts on the peripheral bus, so the DMA_CFG.PSIZE selection must be less than, or equal to, the width of the bus. If the selection is greater than the bus width, a configuration error occurs. The peripheral bus of the processor is dedicated to DMA and peripheral accesses.
		0 1 Byte
		1 2 Bytes
		2 4 Bytes
		3 8 Bytes
3 (R/W)	CADDR	Use Current Address. When the DMA_CFG.CADDR bit is cleared, the DMA loads the DMA_ADDRSTART register on the first access of the work unit. When the DMA_CFG.CADDR bit is set, the DMA uses the DMA_ADDR_CUR register value for the starting address for the work unit and writes the same value to the DMA_ADDRSTART register. This operation permits continuation of a previous work unit. When DMA uses this mode, the fetched start address value (as part of the descriptor set at the end of a descriptor list or array) is ignored.
		0 Load Starting Address
		1 Use Current Address

Table 30-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W)	SYNC	Synchronize Work Unit Transitions. Setting the DMA_CFG . SYNC bit clears the DMA FIFO and pointers before starting the first work unit of a work unit chain. When the transfer direction is memory read/transmit (DMA_CFG . WNR =0), the DMA waits until all data transmits to a peripheral before moving on to the next work unit, clearing the FIFO and pointers. When the transfer direction is memory write/receive (DMA_CFG . WNR =1), the DMA ignores the DMA_CFG . SYNC bit value after processing the first work unit of a work unit chain. As a channel can receive data when turned on but idle, data from the peripheral can still be in the FIFO even though the channel was not programmed. When the DMA_CFG . SYNC bit field is set at the beginning of a work unit chain (during the first work unit), the DMA clears the FIFO, erasing the data put into the FIFO while the channel was idle. Syncing lets programs change the DMA_CFG . PSIZE between individual work units and (in some cases) work unit chains. The sync resets the pointers in the FIFO, preventing misaligned FIFO access. Programs can change the DMA_CFG . MSIZE field between consecutive work units, independent of the DMA_CFG . SYNC bit setting. Syncing also permits changes to transfer direction. Because the data in the FIFO is eliminated, the data that went into the FIFO from one direction (transmit or receive) is not sent back in the other direction after the direction change.
		0 No Synchronization
		1 Synchronize Channel
1 (R/W)	WNR	Write/Read Channel Direction. The DMA_CFG . WNR bit selects receive (write to memory) or transmit (read from memory) channel direction.
		0 Transmit (Read from memory)
		1 Receive (Write to memory)

Table 30-21: DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	EN	DMA Channel Enable. The <code>DMA_CFG.EN</code> bit enables the selected DMA channel. When a peripheral DMA channel is enabled, data requests from the peripheral denote DMA requests. When a channel is disabled, the DMA unit ignores the peripheral data request and passes it directly to the system event controller. To avoid unexpected results, enable the DMA channel before enabling the peripheral, and disable the peripheral before disabling the DMA channel. A transition of the <code>DMA_CFG.EN</code> bit from 0 to 1 creates a hard reset of all internal counters and states, including the <code>DMA_STAT</code> register. (All other register values remain unaffected.) A transition from 1 to 0 maintains all counters and registers for reading and analysis. Note that if a descriptor loads when this bit is cleared (see the <code>DMA_CFG.FLOW</code> field), the DMA transitions to the off or idle state after the descriptor load is complete.
		0 Disable
		1 Enable

Current Descriptor Pointer Register

The `DMA_DSCPTR_CUR` register contains the memory address for the next descriptor to be loaded. The `DMA_DSCPTR_CUR` register is read/write prior to enabling the channel, but is read-only after enabling the channel. For `DMA_CFG.FLOW` mode settings that involve descriptor fetches, this register is used to read descriptors into appropriate MMRs before a work unit begins. For descriptor list mode, the `DMA_DSCPTR_CUR` register is initialized from the `DMA_DSCPTR_NXT` register before fetching each descriptor set. Then, the address in the `DMA_DSCPTR_CUR` register increments as each descriptor is read.

When the entire descriptor set has been read, the `DMA_DSCPTR_CUR` register contains this value:

$$\text{DMA_DSCPTR_CUR} = \text{Descriptor Start Address} + \text{Descriptor Size (\# of elements)}$$

For descriptor array mode, the `DMA_DSCPTR_CUR` register, and not the `DMA_DSCPTR_NXT` register, must be programmed by a MMR access before starting DMA operation.

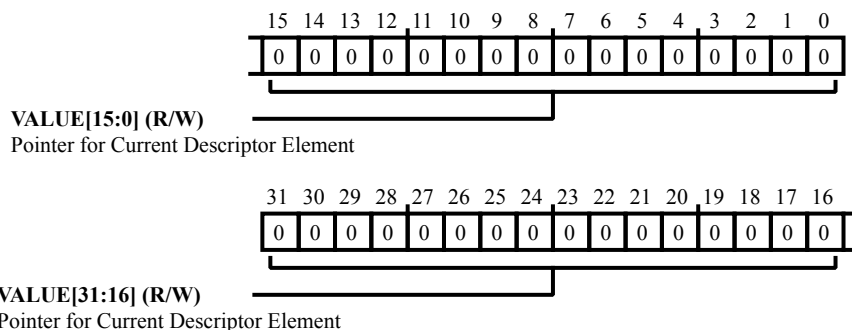


Figure 30-12: `DMA_DSCPTR_CUR` Register Diagram

Table 30-22: `DMA_DSCPTR_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Pointer for Current Descriptor Element. The <code>DMA_DSCPTR_CUR.VALUE</code> bit field contains the memory address for the next descriptor to be loaded.

Pointer to Next Initial Descriptor Register

The `DMA_DSCPTR_NXT` register specifies the start location of the next descriptor set, which begins when the DMA activity specified by the current descriptor set completes. This register is read/write prior to enabling the channel, but is read-only after enabling channel.

The `DMA_DSCPTR_NXT` register is only used in descriptor list mode. At the start of a descriptor fetch in this mode, the `DMA_DSCPTR_NXT` register is copied into the `DMA_DSCPTR_CUR` register. During descriptor fetch, the DMA increments the `DMA_DSCPTR_CUR` register value after reading each element of the descriptor set.

In descriptor list mode, the `DMA_DSCPTR_NXT` register (not the `DMA_DSCPTR_CUR` register) must be programmed directly through MMR access, before the DMA operation is started. In descriptor array mode, the DMA disregards the `DMA_DSCPTR_NXT` register and uses the `DMA_DSCPTR_CUR` register to control descriptor fetch.

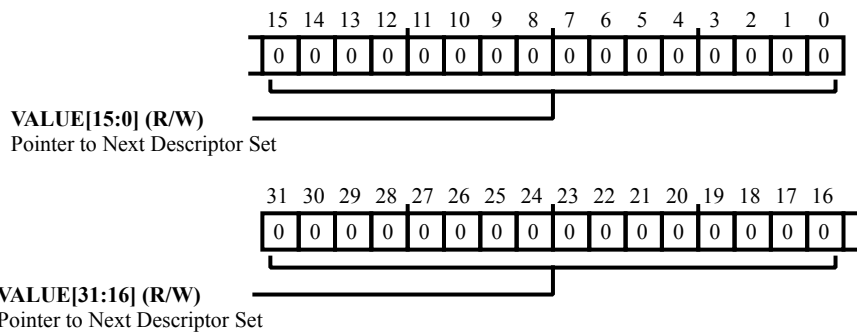


Figure 30-13: `DMA_DSCPTR_NXT` Register Diagram

Table 30-23: `DMA_DSCPTR_NXT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Pointer to Next Descriptor Set. The <code>DMA_DSCPTR_NXT.VALUE</code> bit field specifies the start location of the next descriptor set.

Previous Initial Descriptor Pointer Register

The `DMA_DSCPTR_PRV` register contains the initial descriptor pointer for the previous work unit. If `DMA_CFG.DESCIDCPY` is set, the DMA copies the initial descriptor pointer to `DMA_DSCPTR_PRV` after the work unit completes. Otherwise, the value is not updated.

To indicate an overrun, bit 0 of the `DMA_DSCPTR_PRV` register is used as a previous descriptor pointer overrun (PDPO) status bit. Due to aligned addressing combined with all descriptors being 32 bits in width, bits 0 and 1 of all descriptor pointers must be zero. Otherwise, an alignment error occurs when used for descriptor fetches. As a result, bit 1 and 0 of the `DMA_DSCPTR_PRV` register can be used for status. For more information, see the section on descriptor pointer capture in the DMA functional description.

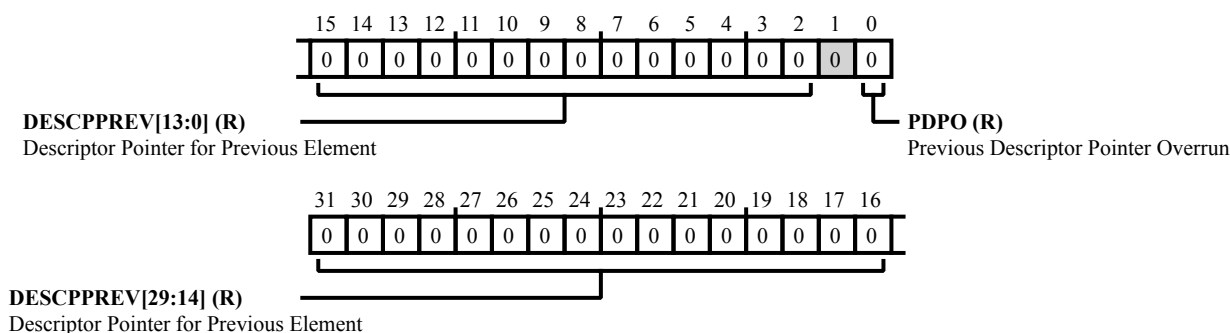


Figure 30-14: `DMA_DSCPTR_PRV` Register Diagram

Table 30-24: `DMA_DSCPTR_PRV` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:2 (R/NW)	DESCPPREV	Descriptor Pointer for Previous Element. The <code>DMA_DSCPTR_PRV.DESCPPREV</code> bit field contains the initial descriptor pointer for the previous work unit.
0 (R/NW)	PDPO	Previous Descriptor Pointer Overrun. The <code>DMA_DSCPTR_PRV.PDPO</code> bit signifies an alignment error. Due to aligned addressing combined with all descriptors being 32 bits in width, bits 0 and 1 of all descriptor pointers must be zero. Otherwise, an alignment error would occur when used for descriptor fetches.
		0 No Error Occurred
		1 Error Occurred

Status Register

The `DMA_STAT` register indicates the status of DMA work units, the FIFO, errors, interrupts, and triggers.

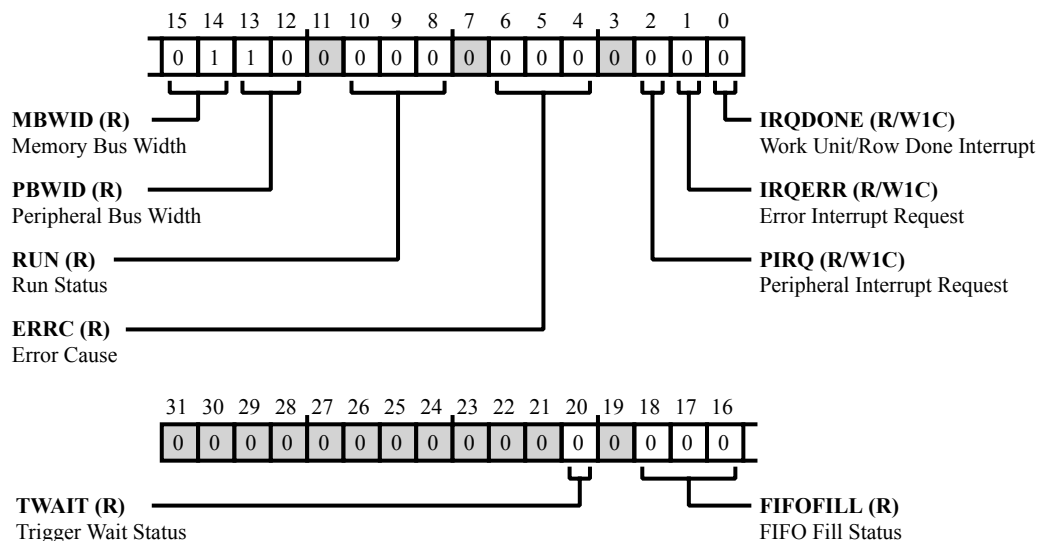


Figure 30-15: `DMA_STAT` Register Diagram

Table 30-25: `DMA_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/NW)	TWAIT	Trigger Wait Status. The <code>DMA_STAT.TWAIT</code> bit indicates whether the DMA has or has not received a trigger. This bit is set until it reaches the next wait state. At that point, the bit is cleared, the DMA stops processing that work unit, and the following work unit processes. The DMA does not distinguish between one or more triggers received.
		0 No Trigger Received
		1 Trigger Received
18:16 (R/NW)	FIFOFILL	FIFO Fill Status. The <code>DMA_STAT.FIFOFILL</code> bit field reports the quantity of data in the FIFO relative to available space.
		0 Empty
		1 Empty < FIFO = 1/4 Full
		2 1/4 Full < FIFO = 1/2 Full
		3 1/2 Full < FIFO = 3/4 Full
		4 3/4 Full < FIFO = Full
		5 Reserved

Table 30-25: DMA_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		6	Reserved
		7	Full
15:14 (R/NW)	MBWID	Memory Bus Width. The DMA_STAT.MBWID bit field indicates the width of the memory bus connected to this DMA.	
		0	2 Bytes
		1	4 Bytes
		2	8 Bytes
		3	16 Bytes
13:12 (R/NW)	PBWID	Peripheral Bus Width. The DMA_STAT.PBWID bit field indicates the width of the peripheral bus connected to this DMA.	
		0	1 Byte
		1	2 Bytes
		2	4 Bytes
		3	8 Bytes
10:8 (R/NW)	RUN	Run Status. The DMA_STAT.RUN bit field reports the DMA's current operational state. If the DMA is in idle or stop state, the DMA_CFG.EN bit is either 0 or 1. This bit field does not clear when a transition of the DMA_CFG.EN bit from 0 to 1 occurs. The DMA_STAT.RUN clears automatically when the DMA completes.	
		0	Idle/Stop State
		1	Descriptor Fetch
		2	Data Transfer
		3	Waiting for Trigger
		4	Waiting for Write ACK/FIFO Drain to Peripheral
		5	Reserved
		6	Reserved
		7	Reserved
6:4 (R/NW)	ERRC	Error Cause. When an interrupt request error occurs (DMA_STAT.IRQERR), the DMA updates the DMA_STAT.ERRC bit field to identify the type of error. For more information, see the errors section of the DMA functional description.	

Table 30-25: DMA_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		0	Configuration Error
		1	Illegal Write Occurred While Channel Running
		2	Address Alignment Error
		3	Memory Access or Fabric Error
		4	Reserved
		5	Trigger Overrun
		6	Bandwidth Monitor Error
		7	Reserved
2 (R/W1C)	PIRQ	Peripheral Interrupt Request. The DMA_STAT . PIRQ bit indicates a peripheral interrupt request. Programs can use the DMA_STAT . PIRQ bit status to help determine which DMA asserted the interrupt request. This bit also helps to distinguish between an interrupt request caused by the state of the work unit and an interrupt request caused by the peripheral.	
		0	No Interrupt request
		1	Interrupt Request signaled by peripheral
1 (R/W1C)	IRQERR	Error Interrupt Request. The DMA_STAT . IRQERR bit indicates that the DMA has detected a documented rule violation during DMA programming or operation. The DMA cannot, however, flag all possible programming or operation issues to indicate errors. Programmers can use the DMA_STAT . IRQERR bit to help determine which DMA issued the error interrupt request. The DMA_STAT . IRQERR does not clear a transition of the DMA_CFG . EN bit from 0 to 1. Clear the DMA_STAT . IRQERR bit with a write-1-to-clear operation prior to the DMA_CFG . EN transition for the fields to be reset.	
		0	No Error
		1	Error Occurred
0 (R/W1C)	IRQDONE	Work Unit/Row Done Interrupt. The DMA_STAT . IRQDONE bit indicates that the DMA has detected the completion of a work unit or row (inner loop count) and has issued an interrupt request. Programs can use the DMA_STAT . IRQDONE status to help determine which DMA asserted the interrupt request. Programs can also use these bits to help distinguish between an interrupt request based on the state of the work unit and an interrupt request made by the peripheral. For more information, see the interrupts section of the DMA functional description.	
		0	Inactive
		1	Active

Inner Loop Count Start Value Register

For 2D DMA, the `DMA_XCNT` register contains the inner loop count. This value selects the number of `DMA_CFG.MSIZE` size data transfers that make up the length of a row. For 1D DMA, the `DMA_XCNT` register specifies the number of `DMA_CFG.MSIZE` size data transfers for the entire work unit. The `DMA_XCNT` register is read/write prior to enabling the channel, but is read-only after enabling channel. Note that the DMA generates a configuration error if this register is 0x0 when a work unit begins.

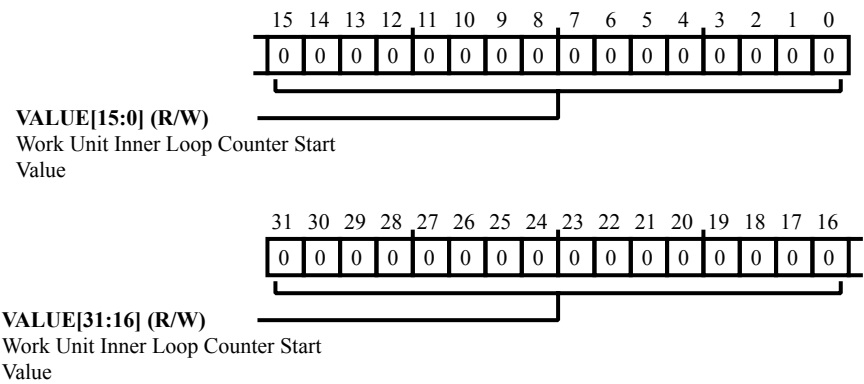


Figure 30-16: DMA_XCNT Register Diagram

Table 30-26: DMA_XCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Work Unit Inner Loop Counter Start Value. For 2D DMA, the <code>DMA_XCNT.VALUE</code> bit field contains the inner loop count. For 1D DMA, <code>DMA_XCNT.VALUE</code> specifies the number of <code>DMA_CFG.MSIZE</code> size data transfers for the entire work unit.

Current Count (1D) or Intra-row XCNT (2D) Register

For 1D DMA, the DMA loads the `DMA_XCNT_CUR` register from the `DMA_XCNT` register at the beginning of each work unit. For 2D DMA, the DMA loads the `DMA_XCNT_CUR` register from the `DMA_XCNT` register after the end of each row. The DMA decrements the value in `DMA_XCNT_CUR` register each time a `DMA_CFG.MSIZE` size data transfer occurs. When the count in `DMA_XCNT_CUR` register expires, the work unit is complete. In 2D DMA, the `DMA_XCNT_CUR` value is 0 only when the entire transfer is complete.

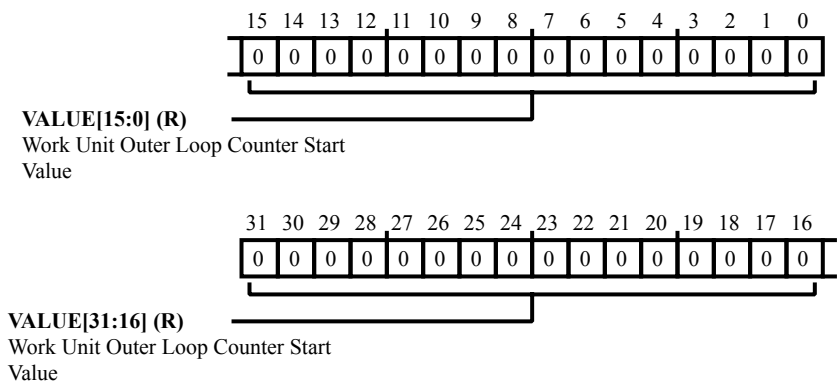


Figure 30-17: DMA_XCNT_CUR Register Diagram

Table 30-27: DMA_XCNT_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Work Unit Outer Loop Counter Start Value. For 1D DMA, the <code>DMA_XCNT_CUR.VALUE</code> bit field holds the <code>DMA_XCNT</code> value at the beginning of each work unit. For 2D DMA, the <code>DMA_XCNT_CUR.VALUE</code> bit field holds the <code>DMA_XCNT</code> value after the end of each row.

Inner Loop Address Increment Register

The `DMA_XMOD` register contains a signed, two's-complement byte address increment. In 1D DMA, this increment is the stride that is applied after each `DMA_CFG.MSIZE` size data transfer. The `DMA_XMOD` register is read/write prior to enabling the channel, but is read-only after enabling the channel.

The `DMA_XMOD` register value is specified in bytes, regardless of the work unit size. In 2D DMA, this increment is applied after each `DMA_CFG.MSIZE` size data transfer in the inner loop, up to but not including the last `DMA_CFG.MSIZE` size data transfer in each inner loop. After the last `DMA_CFG.MSIZE` size data transfer in each inner loop, the `DMA_YMOD` register is applied instead, including the last `DMA_CFG.MSIZE` size data transfer of a work unit.

The `DMA_XMOD` field can be set to 0. In this case, DMA is performed repeatedly to or from the same address. This approach can be useful for transferring data between a data register and an external memory-mapped peripheral.

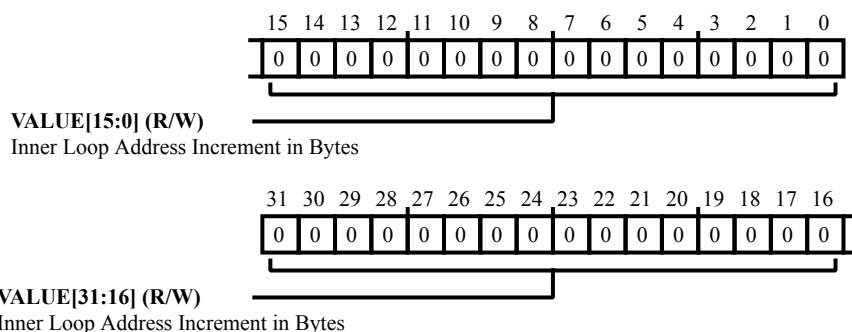


Figure 30-18: `DMA_XMOD` Register Diagram

Table 30-28: `DMA_XMOD` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Inner Loop Address Increment in Bytes. The <code>DMA_XMOD.VALUE</code> bit field contains a signed, two's-complement byte address increment.

Outer Loop Count Start Value (2D only) Register

For 2D DMA, the `DMA_YCNT` register contains the outer loop count. This register is not used in 1D DMA mode. The `DMA_YCNT` register specifies the number of rows in the outer loop of a 2D DMA sequence. The `DMA_YCNT` register is read/write prior to enabling the channel, but is read-only after enabling channel. Note that the DMA generates a configuration error if this register is 0x0 when a work unit begins.

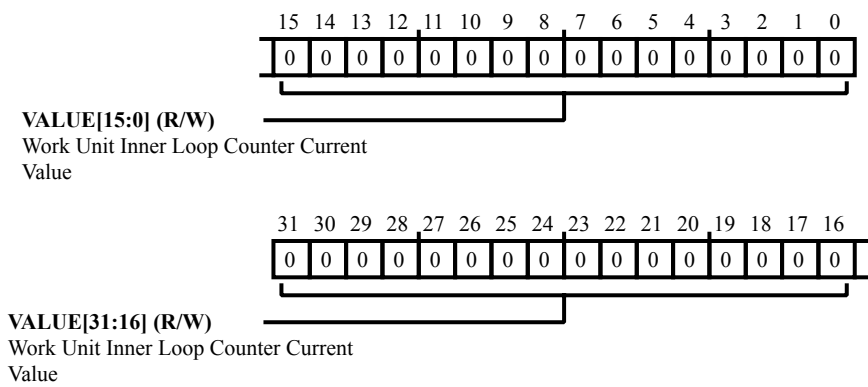


Figure 30-19: DMA_YCNT Register Diagram

Table 30-29: DMA_YCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Work Unit Inner Loop Counter Current Value. For 2D DMA, the <code>DMA_YCNT</code> .VALUE bit field contains the outer loop count.

Current Row Count (2D only) Register

For 2D DMA, the DMA loads the DMA_YCNT_CUR register from the DMA_YCNT register at the beginning of each 2D DMA session. The DMA_YCNT_CUR register is not used for 1D DMA. The DMA decrements the DMA_YCNT_CUR register each time the DMA_XCNT_CUR register expires during 2D DMA operation, signifying the completion of an entire row transfer.

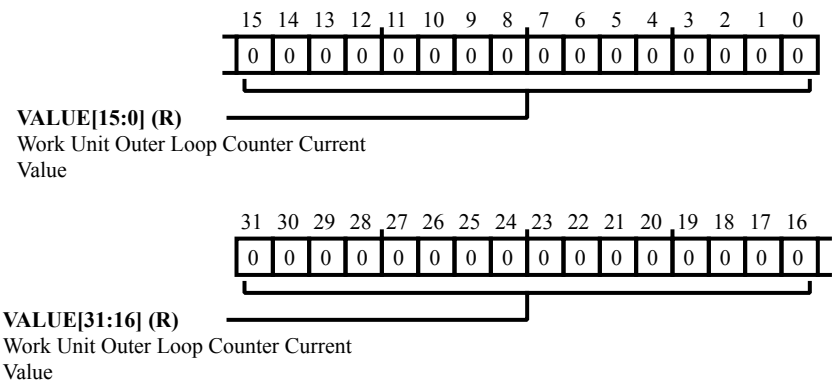


Figure 30-20: DMA_YCNT_CUR Register Diagram

Table 30-30: DMA_YCNT_CUR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Work Unit Outer Loop Counter Current Value. For 2D DMA, the DMA_YCNT_CUR.VALUE bit field holds the value from the DMA_YCNT register at the beginning of each 2D DMA session.

Outer Loop Address Increment (2D only) Register

The `DMA_YMOD` register contains a signed, two's-complement value. This byte address increment is applied after each decrement of the `DMA_YCNT_CUR` register. The value is the offset between the last word of one row and the first word of the next row. Note that `DMA_YMOD` is specified in bytes, regardless of the work unit size. The `DMA_YMOD` register is read/write prior to enabling the channel, but is read-only after enabling the channel.

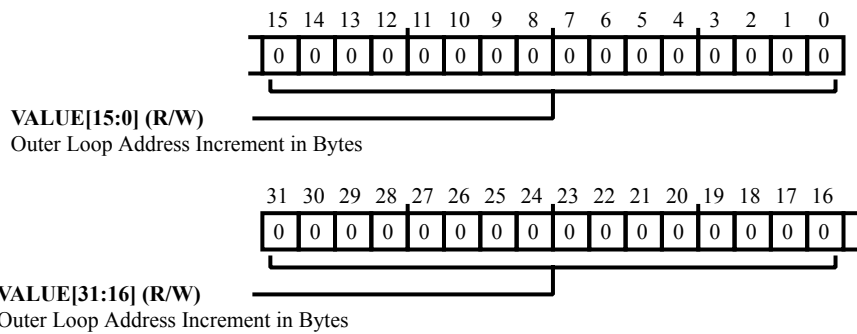


Figure 30-21: DMA_YMOD Register Diagram

Table 30-31: DMA_YMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Outer Loop Address Increment in Bytes. The <code>DMA_YMOD.VALUE</code> bit field contains a signed, two's-complement value.

31 Extended Memory DMA (EMDMA)

The Extended Memory DMA engine can be used in applications that copy data in a non-sequential manner. This includes delay lines, scatter and gather, and circular access types.

NOTE: Previous SHARC processors featured external port DMA. Current SHARC products use EMDMA that can access all memory locations (L1/L2/L3) for source and destination DMA operations.

EMDMA Features

EMDMA frees the processor core, allowing it to perform other operations while the data transfers between memories occurs as a background task.

EMDMA has the following features and capabilities.

- Standard mode DMA Transfer
- Chained mode with direction on-the-fly
- Tap list mode (scatter/gather)
- Delay line mode (write to read)
- All the DMA modes can operate in circular fashion
- In circular operation, some modes allow a write back of the index pointer for correct addressing of the next Transfer Control Block (TCB)

EMDMA Functional Description

The following sections provide information on the functional operations and operating modes of EMDMA.

Internal-to-internal DMA

This is accomplished by indexing all external parameter registers with internal addresses.

DMA bursting

DMA supports burst transfers only when the modifier is 1, for any other modifier values, single accesses are performed. The burst size is not user configurable and is chosen by the processor for optimal performance. The maximum burst size is 8 and the minimum is 1.

Transfer control blocks

The structure of a TCB is conceptually the same as that of a traditional linked-list. Each TCB has several data values and a pointer to the next TCB. Further, the chain pointer of a TCB may point to itself to continuously re-run the same DMA.

Chain pointer DMA

In chained DMA operations, the processor automatically initializes and then begins another DMA transfer when the current DMA transfer is complete.

ADSP-2159x EMDMA Register List

The EMDMA controllers support a variety of direct memory access operations which can access any system memory and transfer the entire block of data. A set of registers govern EMDMA operations. For more information on EMDMA functionality, see the EMDMA register descriptions.

Table 31-1: ADSP-2159x EMDMA Register List

Name	Description
EMDMA_BASE	External Base Address Register
EMDMA_BUFLEN	Circular Buffer Length Register
EMDMA_CHNPTR	Chain Pointer Register
EMDMA_CNT0	Internal Count Register
EMDMA_CNT1	External Count Register
EMDMA_CTL	External Memory DMA Control Register
EMDMA_INDX0	Internal Index Register
EMDMA_INDX1	External Index Register
EMDMA_MOD0	Internal Modifier Register
EMDMA_MOD1	External Modifier Register
EMDMA_TCNT	Delay Line Tap Count Register
EMDMA_TPTR	Tap List Pointer Register

ADSP-2159x EMDMA Interrupt List

Table 31-2: ADSP-2159x EMDMA Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
210	EMDMA0_DONE	EMDMA0 Transfer Done	Edge	
211	EMDMA1_DONE	EMDMA1 Transfer Done	Edge	

ADSP-2159x EMDMA Trigger List

Table 31-3: ADSP-2159x EMDMA Trigger List Masters

Trigger ID	Name	Description	Sensitivity
32	EMDMA0_DONE	EMDMA0 DMA Done	Edge
33	EMDMA1_DONE	EMDMA1 DMA Done	Edge

Table 31-4: ADSP-2159x EMDMA Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

DMA Addressing

Besides the traditional internal-to-external addressing type, the DMA module also supports internal-to-internal transfers. This is accomplished by indexing all external parameter registers with internal addresses. The DMA controller recognizes the transfer by addresses and not by an additional control bit setting.

All the DMA addresses given by EMDMA are word-aligned byte addresses. The programming for the index registers is provided in the *Register Descriptions* section.

DMA Burst Transfers

DMA supports burst transfers only when the modifier is 1. For any other modifier values, single accesses are performed.

The burst size is not user-configurable and is chosen by the processor for optimal performance. The maximum burst size is 8 and the minimum is 1.

The EMDMA uses appropriate burst transfer sizes for optimal throughput. For example, if the word count is 15, then 5 bursts are performed with burst sizes of 8+4+1+1+1 transfers.

Transfer Control Block (TCB) Memory Storage

The location of the DMA parameters for the next sequence comes from the chain pointer register that points to the next set of DMA parameters stored in the processor's internal memory. In chained DMA operations, the processor automatically initializes and then begins another DMA transfer when the current DMA transfer is complete. Each new set of parameters is stored in a user-initialized memory buffer or TCB for a chosen peripheral.

Chain Assignment

The structure of a TCB is conceptually the same as that of a traditional linked-list. Each TCB has several data values and a pointer to the next TCB. Further, the chain pointer of a TCB may point to itself to continuously rerun the same DMA. The EMDMA reads each word of the TCB and loads it into the corresponding register. The end of the chain (no further TCBs are loaded) is indicated by a TCB with a chain pointer register value of zero.

The address field of the chain pointer registers is only 30 bits wide. If a program writes a symbolic address to bit 30 of the chain pointer there may be a conflict with the `EMDMA_CHNPTR.PCI` bit. Programs should clear the upper bits of the address then AND the `EMDMA_CHNPTR.PCI` bit separately, if needed.

Starting Chain Loading

A DMA sequence is defined as the sum of the DMA transfer from when the parameter registers initialize to when the count register decrements to zero. The EMDMA module has a chaining enable bit (`EMDMA_CTL.CHEN`).

To start the chain, write the internal (channel 0) index address of the first TCB to the chain pointer register (`EMDMA_CHNPTR`). When chaining is enabled, DMA transfers are initiated by writing a memory address to the chain pointer register. This is also an easy way to start a single DMA sequence, with no subsequent chained DMAs.

During TCB chain loading, the EMDMA loads the DMA channel parameter registers with values retrieved from system memory.

The address in the chain pointer register points to the highest address of the TCB. This contains the internal (channel 0) index parameter. This means that if a program declares an array to hold the TCB, the chain pointer register should point to the last location of the array and not to the first TCB location.

Buffered Chain Loading Register

The chain pointer register (`EMDMA_CHNPTR`) is buffered. Before the chain loading starts, the buffer is copied into the chain pointer register and is decremented after each register is loaded.

The chain pointer register can be loaded with a new address at any time during the DMA sequence (`EMDMA_CTL.CHEN = 1`). This allows a DMA channel to have chaining status deactivated (chain pointer register = 0x0) until some event occurs that loads the chain pointer register with a non-zero value. Writing all zeros to the address field of the chain pointer register also deactivates chaining for the next TCB.

TCB Storage

The EMDMA supports several types of DMA, resulting in different lengths of TCBs. The TCB size varies from six locations (chained DMA) to 13 locations (delay line DMA). The *EMDMA TCBs* table shows the required TCBs for chained DMA.

In the following tables, TCB refers to the start address of the TCB array.

Table 31-5: EMDMA TCBs for Standard DMA

Address	Register
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_MOD1
TCB + 0x2	EMDMA_INDX1
TCB + 0x3	EMDMA_CNT0
TCB + 0x4	EMDMA_MOD0
TCB + 0x5	EMDMA_INDX0

The order the descriptors are fetched with circular buffering enabled is shown in the *EMDMA TCBs for Standard Circular DMA* table.

Table 31-6: EMDMA TCBs for Standard Circular DMA

Address	Register
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_BUFLen
TCB + 0x2	EMDMA_BASE
TCB + 0x3	EMDMA_MOD1
TCB + 0x4	EMDMA_INDX1
TCB + 0x5	EMDMA_CNT0
TCB + 0x6	EMDMA_MOD0
TCB + 0x7	EMDMA_INDX0

For delay line DMA, TCB loading is split into two sequences to improve overall priority. The first TCB loads the write parameters (EMDMA_INDX0 – EMDMA_BUFLen) and the second loads the read parameters (EMDMA_INDX0 – EMDMA_CHNPTR). This two stage loading is transparent to the application. The order the descriptors are fetched for delay line DMA, as shown in the *EMDMA TCBs for Delay Line DMA* table.

Table 31-7: EMDMA TCBs for Delay Line DMA

Address	Register
Delay Line Read	
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_TPTR
TCB + 0x2	EMDMA_TCNT
TCB + 0x3	EMDMA_MOD1
TCB + 0x4	EMDMA_CNT0

Table 31-7: EMDMA TCBs for Delay Line DMA (Continued)

Address	Register
TCB + 0x5	EMDMA_INDX0
Delay Line Write	
TCB + 0x6	EMDMA_BUFLN
TCB + 0x7	EMDMA_BASE
TCB + 0x8	EMDMA_MOD1
TCB + 0x9	EMDMA_INDX1
TCB + 0xA	EMDMA_CNT0
TCB + 0xB	EMDMA_MOD0
TCB + 0xC	EMDMA_INDX0

The order the descriptors are fetched for scatter/gather DMA with circular buffering disabled is shown in the *EMDMA TCBs for Scatter/Gather DMA* table.

Table 31-8: EMDMA TCBs for Scatter/Gather DMA

Address	Register
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_TPTR
TCB + 0x2	EMDMA_TCNT
TCB + 0x3	EMDMA_MOD1
TCB + 0x4	EMDMA_INDX1
TCB + 0x5	EMDMA_CNT0
TCB + 0x6	EMDMA_MOD0
TCB + 0x7	EMDMA_INDX0

The order the descriptors are fetched for scatter/gather DMA with circular buffering enabled is shown in the *EMDMA TCBs for Circular Scatter/Gather DMA* table.

Table 31-9: EMDMA TCBs for Circular Scatter/Gather DMA

Address	Register
TCB	EMDMA_CHNPTR
TCB + 0x1	EMDMA_BUFLN
TCB + 0x2	EMDMA_BASE
TCB + 0x3	EMDMA_TPTR
TCB + 0x4	EMDMA_TCNT

Table 31-9: EMDMA TCBs for Circular Scatter/Gather DMA (Continued)

Address	Register
TCB + 0x5	EMDMA_MOD1
TCB + 0x6	EMDMA_INDX1
TCB + 0x7	EMDMA_CNT0
TCB + 0x8	EMDMA_MOD0
TCB + 0x9	EMDMA_INDX0

EMDMA Operating Modes

This section and the *EMDMA_CTL Register Bit to Operating Modes* table show the different DMA modes which can be used. The complete register bit descriptions are in the *External Memory DMA Control Registers (EMDMA_CTL)*.

Table 31-10: EMDMA_CTL Register Bit to Operating Modes

Bit (Name)	Standard	Chained	Scatter/Gather	Delay Line
Control Bits				
0 (EN)	Valid			
1 (TRAN)	Valid			N/A
2 (CHEN)	Valid			
3 (DLEN)	N/A			Valid
4 (CBEN)	Valid			
5 (DFLSH)	Valid			
6	N/A			
7 (WRBEN)	N/A	Valid	(=0)	(=1)
8 (OFCEN)	Valid			N/A
9 (TLEN)	N/A		Valid	N/A
11–10	N/A			
12 (INTDONE0)	Valid			
15–13	N/A			
Status Bits				
17–16 (DFS)	Valid			
19–18	N/A			
20 (DMAS0)	Valid			
21 (CHS)	N/A	Valid	N/A	Valid

Table 31-10: EMDMA_CTL Register Bit to Operating Modes (Continued)

Bit (Name)	Standard	Chained	Scatter/Gather	Delay Line
22 (TLS)	N/A		Valid	N/A
23 (WBS)	N/A			Valid
24 (DMAS1)	Valid			
25 (DIRS)	Valid			
31–26	N/A			

NOTE: Reading additional bit-field information from N/A (Not applicable) bits does not generate a meaningful result.

A program sets up a DMA channel by writing the transfer's parameters to the DMA parameter registers.

NOTE: The `EMDMA_CNT1` parameter register (read only) is a copy of the `EMDMA_CNT0` register. If `EMDMA_CNT0` is written, the `EMDMA_CNT1` register is updated automatically.

Standard DMA

A standard DMA (once it is configured) transfers data from location A to location B. An interrupt can be used to indicate the end of the transfer. To start a new DMA sequence after the current one is finished, a program must first clear the DMA enable bit (`EMDMA_CTL.EN`), write new parameters to the index, modify, and count registers (parameter registers), then set the DMA enable bit to re-enable DMA.

This DMA type resembles the traditional DMA type to initialize the different internal and external parameters (channel0 and channel1) (index, modify and count) registers and configuration of the DMA control registers.

Circular Buffered DMA

Circular buffered DMA resembles the traditional core DAG circular buffered mode by using registers for circular buffering. In this mode, the DMA needs two additional registers (base and length) to support reads and writes to a circular buffer. The *Circular Buffering Write DMA* and *Circular Buffering Read DMA* figures illustrate circular buffered DMA, in contrast with the *Standard Write DMA* figure.

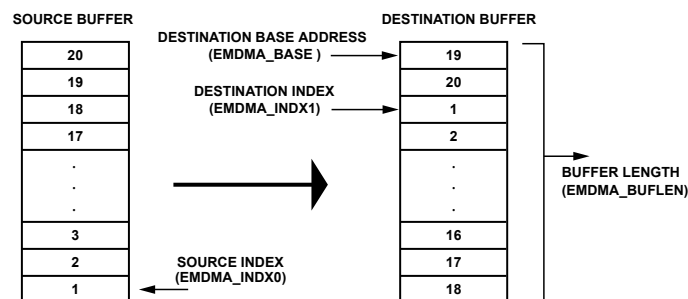


Figure 31-1: Circular Buffering Write DMA

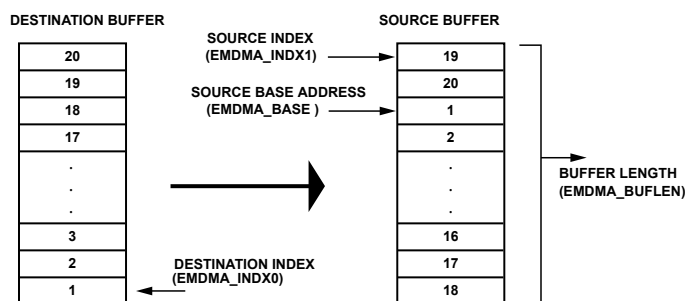


Figure 31-2: Circular Buffering Read DMA

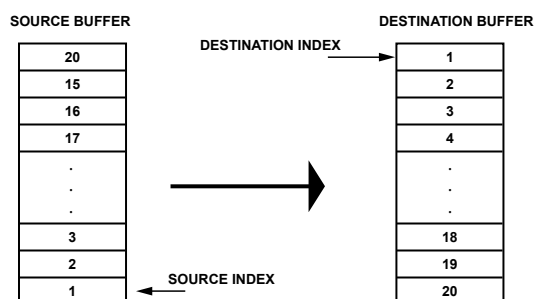


Figure 31-3: Standard Write DMA

NOTE: Circular buffering is available for all operating modes (standard, chained, tap list and delay line DMA).

Chained DMA Mode

Chained DMA sequences are a set of multiple DMA operations, each auto-initializing the next in line. It is used to support automated access by a linked list (repetitive reads and writes to a defined location defined by the individual TCBs). To start a new DMA sequence after the current one is finished, the EMDMA automatically loads new index, modify, and count values pointed to by that channel's `EMDMA_CHNPTR` register. Using chaining, programs can set up consecutive DMA operations and each operation can have different attributes.

DMA data transfers can be set up as continuous or periodic. With chained DMA, the attributes of a specific DMA are stored in internal memory and are referred to as a Transfer Control Block or TCB. Extended Memory DMA loads these attributes in chains for execution. This allows for multiple chains that are finite or infinite.

NOTE: When chaining is enabled, polling should not be used to determine DMA status only because the DMA appears inactive if it is sampled while the next TCB is loading. In such cases where chaining is enabled, along with the polling of DMA status bit, polling of the chaining status bit should also occur so that the correct status of the DMA is known. For example, the `EMDMA_CTL.CHS` bit should be polled as well as the `EMDMA_CTL.DMAS0` and `EMDMA_CTL.DMAS1` bits when EMDMA is configured in DMA chaining mode.

Data Direction On-the-Fly

A change of external memory data direction for each individual TCB in a chain sequence is allowed.

The `EMDMA_CHNPTR.CPDR` bit changes the data flow direction. If the `EMDMA_CHNPTR.CPDR = 0`, writes through channel 0 are performed; if `EMDMA_CHNPTR.CPDR = 1`, channel 0 reads are performed. This works similarly to the `EMDMA_CHNPTR.PCI` bit. The `EMDMA_CTL.OFCEN` and `EMDMA_CTL.CHEN` bits must be set (=1) to enable this functionality.

NOTE: If chaining is enabled with the `EMDMA_CTL.OFCEN = 1`, then the `EMDMA_CTL.TRAN` bit has no effect, and the data direction is determined by the `EMDMA_CHNPTR.CPDR` bit.

Write Back Circular Index Pointer

Operating the DMA in circular mode requires some special considerations. The index pointer of start address within the buffer may wrap around for the case when $IC \times IM > EL$ or it does not finish if $IC \times IM < EL$ where:

IC is the value of the `EMDMA_INDX0` register

IM is the value of the `EMDMA_MOD0` register

EL is the value of the `EMDMA_BUFLLEN` register

In both cases, the TCB start address is no longer valid.

Setting the `EMDMA_CTL.WRBEN` bit writes (at the end of current TCB block) the current index address + 1 into the TCB memory which is the start address for the next TCB. This bit is only selectable for chained DMA mode. For tap list and delay line modes, this bit is hardwired to 0 or 1.

Scatter/Gather DMA

The purpose of scatter/gather DMA is the transfer of data from/to non-contiguous memory blocks.

The scatter/gather DMA type is a fixed block size scatter/gather DMA that relies on tap list entries in system memory to calculate the (Channel 1 DMA Address) to scatter/gather the DMA. If the DMA direction is Channel 1 write (`EMDMA_CTL.TRAN = 1`) then it is a scatter DMA. If `EMDMA_CTL.TRAN = 0` then it is a gather DMA. This mode also supports chained and circular buffer chained DMAs.

See the *Scatter DMA (Writes)*, *Gather DMA (Reads)*, *Circular Buffering Scatter DMA (Writes)*, and *Circular Gather DMA (Reads)* figures.

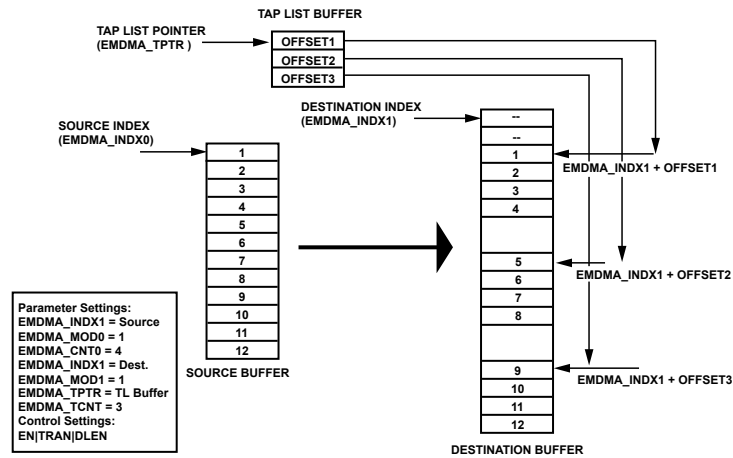


Figure 31-4: Scatter DMA (Writes)

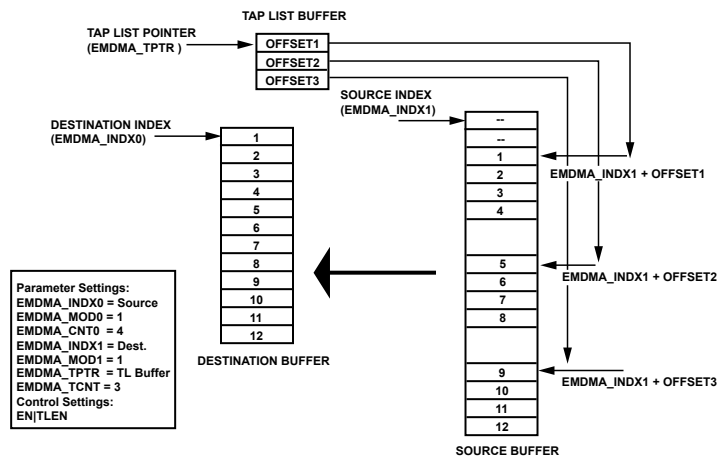


Figure 31-5: Gather DMA (Reads)

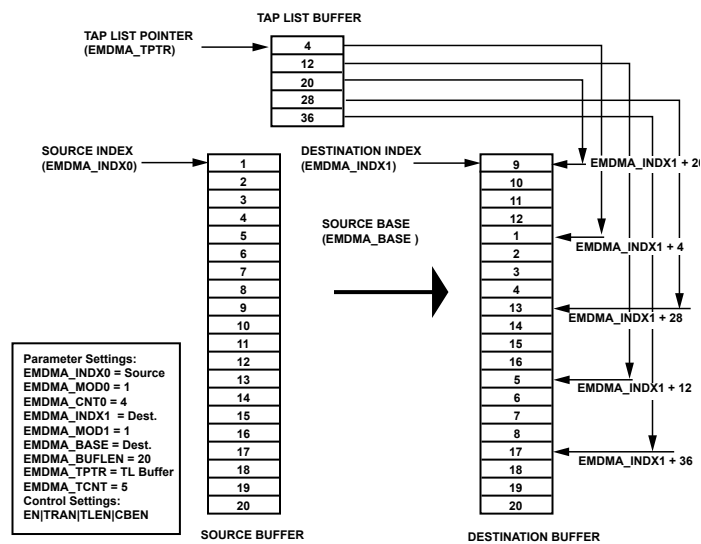


Figure 31-6: Circular Buffering Scatter DMA (Writes)

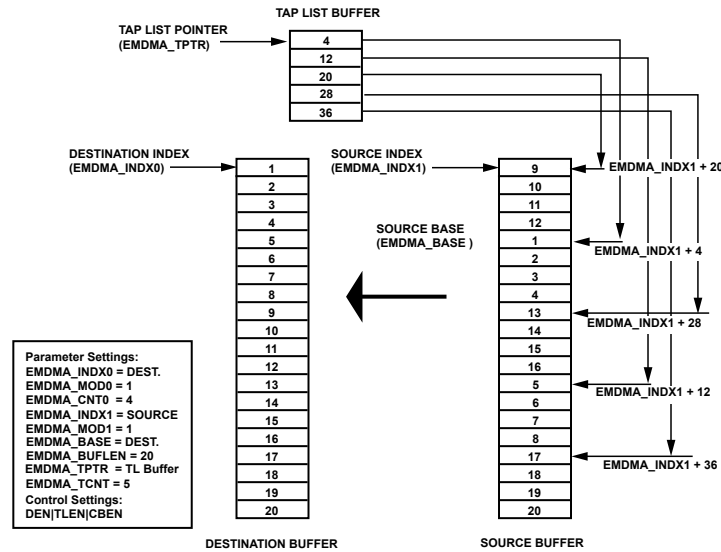


Figure 31-7: Circular Gather DMA (Reads)

For each 32-bit tap read, the Channel 1 read index is shown in the *Read/Write Index Pre-Modify (Scatter/Gather DMA)* table. Note that one tap list entry starts multiple reads.

Table 31-11: Read/Write Index Pre-Modify (Scatter/Gather DMA)

Pre-Modify Address Equation	Result	
	Block Size	Tap
$EMDMA_INDX1 + EMDMA_TPTR[EMDMA_TCNT] + (EMDMA_MOD1 \times EMDMA_CNT0)$	N	0
$EMDMA_INDX1 + EMDMA_TPTR[0] + EMDMA_MOD1 \times 1$		
$EMDMA_INDX1 + EMDMA_TPTR[0] + EMDMA_MOD1 \times 2$		
$EMDMA_INDX1 + EMDMA_TPTR[0] + EMDMA_MOD1 \times 3$		
$EMDMA_INDX1 + EMDMA_TPTR[0] + EMDMA_MOD1 \times N$	N	1
$EMDMA_INDX1 + EMDMA_TPTR[1] + EMDMA_MOD1 \times 1$		
$EMDMA_INDX1 + EMDMA_TPTR[1] + EMDMA_MOD1 \times 2$		
$EMDMA_INDX1 + EMDMA_TPTR[1] + EMDMA_MOD1 \times 3$		
$EMDMA_INDX1 + EMDMA_TPTR[1] + EMDMA_MOD1 \times N$	N	M
$EMDMA_INDX1 + EMDMA_TPTR[M] + EMDMA_MOD1 \times 1$		
$EMDMA_INDX1 + EMDMA_TPTR[M] + EMDMA_MOD1 \times 2$		
$EMDMA_INDX1 + EMDMA_TPTR[M] + EMDMA_MOD1 \times 3$		
$EMDMA_INDX1 + EMDMA_TPTR[M] + EMDMA_MOD1 \times N$		

Pre Modified Read/Write Index

For scatter/gather DMA, the tap list modifiers are employed and the number of taps is determined by the tap list count register ($EMDMA_TCNT$). The number of sequential reads (block size) from every tap is determined by the

internal count register (`EMDMA_CNT0`), and is the same for every tap. The read/write pointer in external index register (`EMDMA_INDX1`) serves as the index address for these read/writes.

`TL[N]` is the first tap list entry in the memory as pointed to by the tap list pointer register (`EMDMA_TPTR`). The tap list entries are 27-bit signed integers. For each read/write block, the DMA state machine fetches the offset from the tap list. The offset is added to the `EMDMA_INDX1` register value to get the start address of the next block. The Channel 1 addresses are circular buffered if circular buffering is enabled (see the *Circular Buffering Scatter DMA (Writes)* and *Circular Buffering Gather DMA (Reads)* figures in [Scatter/Gather DMA](#)).

Once the `EMDMA_CNT0` register for the final tap decrements to zero (both `EMDMA_TCNT` and `EMDMA_CNT0` are zero), then the tap list DMA access is complete and the DMA completion interrupt is generated (if chaining is enabled the interrupt depends on the `EMDMA_CHNPTR.PCI` bit setting).

The write back mode (`EMDMA_CTL.WRBEN` bit) is hardwired to zero for tap list based DMA (as the addressing is pre-modify, and therefore the `EMDMA_INDX1` value coincides with the TCB value even at the end of the DMA).

Delay Line DMA

Delay line DMA is used to support reads and writes to delay line buffers with limited core interaction. In this sense, delay line DMA is a quantity of integrated writes followed by reads from channel 1 (a delay line DMA access). Delay line DMA is described in the following sections.

NOTE: Delay line DMA can only operate by using chained DMA mode (`EMDMA_CTL.CHEN` bit set). In order to use delay line DMA for a single DMA sequence, initialize the `EMDMA_CHNPTR` register to zero in the TCB.

NOTE: Delay line DMA can be used in any system memory.

The delay line DMA operation follows these steps:

1. Load the first half of TCB for writing (seven parameters).
2. DMA writes to the delay line buffer until `IC = 0`.
3. Update the EI index pointer if circular mode is enabled.
4. Load the second half of TCB for reading (six parameters).
5. DMA tap based reads from the delay line buffer until `RC = 0`.

Jump to step 1.

Writes to delay line. The number of writes is determined by the `EMDMA_CNT0` register. The data is fetched from the `EMDMA_INDX0` register and the `EMDMA_MOD0` register is used as the internal modifier. The `EMDMA_INDX1` register serves as the external index and is incremented by the `EMDMA_MOD1` register after each write. These writes are circular buffered if circular buffering is enabled. See the *Write to Delay Line Buffer* figure.

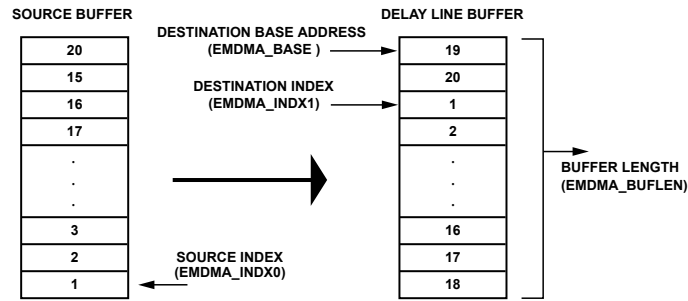


Figure 31-8: Write to Delay Line Buffer

When the writes are complete, ($\text{EMDMA_CNT0} = 0$) the EMDMA_INDX1 register, which serves as the write pointer of the delay line, is written back (EMDMA_CTL.WRBEN is hardwired to 1) to the TCB location from where it was fetched.

Reads from the delay line. For reads, the tap list (TL) modifiers are used and the number of reads is determined by the EMDMA_CNT0 register. The write pointer in the EMDMA_INDX0 register serves as the index address for these reads (reads start from where writes end). The EMDMA_INDX0 register, along with tap list modifiers, are used in a pre-modify addressing mode to create the external address for the reads. Therefore, for each read, the DMA controller fetches the external modifier (EMDMA_TCNT register) from the tap list and the reads are circular buffered (if enabled). See the *Read From Delay Line Buffer* figure.

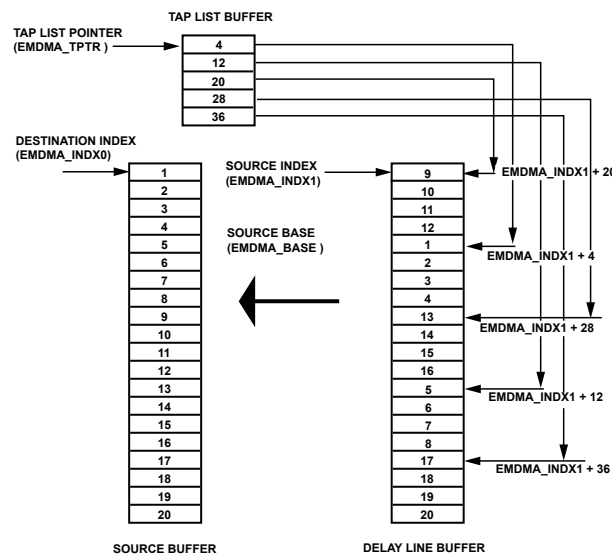


Figure 31-9: Read From Delay Line Buffer

Pre-Modified Read Index

Note that $\text{TL}[N]$ is the first tap list entry in memory pointed to by the tap list pointer register (EMDMA_TPTR). Tap list entries are 27-bit signed integers. Therefore, for each read-block, the DMA state machine fetches the offset external modifier from the tap list. The reads are circular buffered if circular buffering is enabled.

NOTE: The channel 1 DMA address generation follows pre-modify addressing for reads in delay line DMA and therefore the EMDMA_INDX1 register values are not updated. Also the EMDMA_MOD1 register does not

have any effect during these delay line reads. Once the read count completes, the `EMDMA_CNT0` register decrements to zero (both `EMDMA_CNT0` and `EMDMA_TCNT` are zero) for the final tap. Finally, the delay line DMA access completes and the DMA completion interrupt is generated. If chaining is enabled, the interrupt is dependent on the `EMDMA_CHNPTR.PCI` bit setting. The delay line DMA can only be initialized using the TCB. In order to use the delay line DMA for a single DMA sequence, initialize the `EMDMA_CHNPTR` register to zero in the TCB.

For each 32-bit tap read, the channel 1 read index is shown in the *Read/Write Index Pre-Modify (Scatter/Gather DMA)* table. Note that one tap list entry starts multiple reads.

Table 31-12: Read/Write Index Pre-Modify (Scatter/Gather DMA)

Pre-Modify Address Equation $\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[\text{EMDMA_TCNT}] + (\text{EMDMA_MOD1} \times \text{EMDMA_CNT0})$	Result	
	Block Size	Tap
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[0] + \text{EMDMA_MOD1} \times 1$	N	0
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[0] + \text{EMDMA_MOD1} \times 2$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[0] + \text{EMDMA_MOD1} \times 3$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[0] + \text{EMDMA_MOD1} \times N$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 1$	N	1
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 2$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times 3$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[1] + \text{EMDMA_MOD1} \times N$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[M] + \text{EMDMA_MOD1} \times 1$	N	M
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[M] + \text{EMDMA_MOD1} \times 2$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[M] + \text{EMDMA_MOD1} \times 3$		
$\text{EMDMA_INDX0} + \text{EMDMA_TPTR}[M] + \text{EMDMA_MOD1} \times N$		

ADSP-2159x EMDMA Register Descriptions

Extended Memory DMA (EMDMA) contains the following registers.

Table 31-13: ADSP-2159x EMDMA Register List

Name	Description
<code>EMDMA_BASE</code>	External Base Address Register
<code>EMDMA_BUFLen</code>	Circular Buffer Length Register
<code>EMDMA_CHNPTR</code>	Chain Pointer Register
<code>EMDMA_CNT0</code>	Internal Count Register
<code>EMDMA_CNT1</code>	External Count Register

Table 31-13: ADSP-2159x EMDMA Register List (Continued)

Name	Description
EMDMA_CTL	External Memory DMA Control Register
EMDMA_INDX0	Internal Index Register
EMDMA_INDX1	External Index Register
EMDMA_MOD0	Internal Modifier Register
EMDMA_MOD1	External Modifier Register
EMDMA_TCNT	Delay Line Tap Count Register
EMDMA_TPTR	Tap List Pointer Register

External Base Address Register

The `EMDMA_BASE` register contains the external base address of the Delay Line buffer. This is used for maintaining circular buffered read/writes to the delay line.

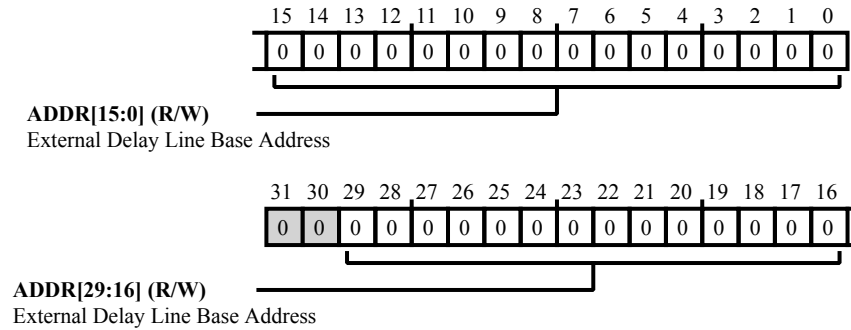


Figure 31-10: EMDMA_BASE Register Diagram

Table 31-14: EMDMA_BASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	ADDR	External Delay Line Base Address. The <code>EMDMA_BASE.ADDR</code> bit field contains the external base address of the Delay Line buffer.

Circular Buffer Length Register

The `EMDMA_BUFLEN` register holds the circular buffer length for the Delay-line DMA.

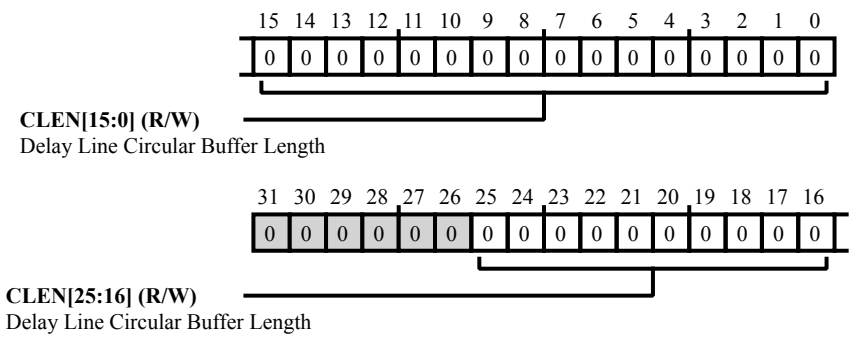


Figure 31-11: EMDMA_BUFLEN Register Diagram

Table 31-15: EMDMA_BUFLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:0 (R/W)	CLEN	Delay Line Circular Buffer Length. The <code>EMDMA_BUFLEN.CLEN</code> bit field holds the circular buffer length for the Delay-line DMA.

Chain Pointer Register

The `EMDMA_CHNPTR` register contains the address of the next descriptor in memory when the `EMDMA_CTL.CHEN` bit =1. This register also has bits to change DMA directions for each descriptor and the PCI bit. Note that the lower 30-bits of this register are to be written with 30 MSB's of the Word Aligned Byte addresses of the corresponding next descriptor address.

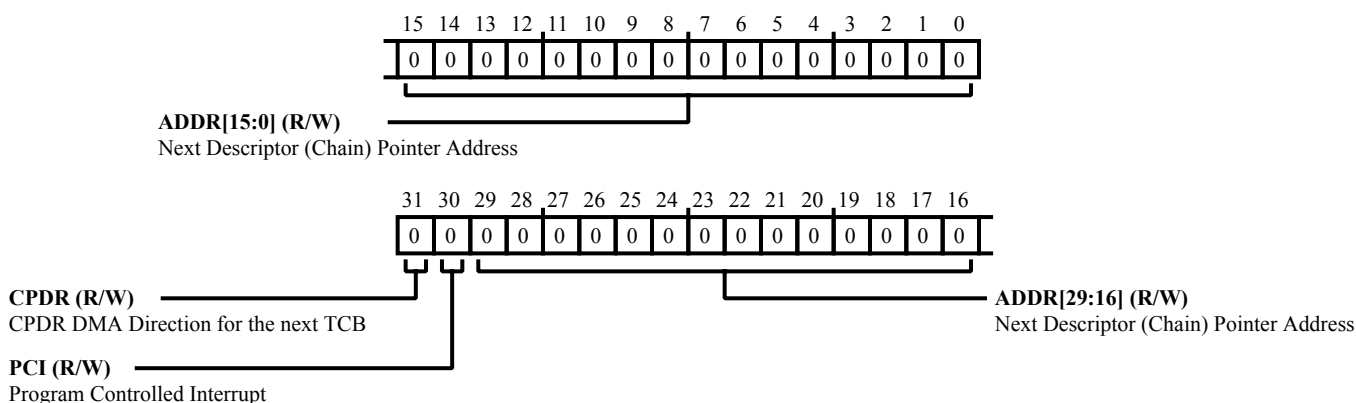


Figure 31-12: EMDMA_CHNPTR Register Diagram

Table 31-16: EMDMA_CHNPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	CPDR	CPDR DMA Direction for the next TCB. The <code>EMDMA_CHNPTR.CPDR</code> bit configures whether the DMA is a write to internal memory or a read from internal memory. Note that this setting is applicable only if <code>EMDMA_CTL.OFCEN</code> =1 and is not applicable for Delay Line DMA.
		0 Write to Channel 0 (Channel 1 reads)
		1 Read from Channel 0 (Channel 1 Writes)
30 (R/W)	PCI	Program Controlled Interrupt. The <code>EMDMA_CHNPTR.PCI</code> bit PCI sets whether an interrupt is generated after the current TCB or if no interrupt is generated. (Only affects DMA if chaining is enabled)
		0 Enable DMA Channel interrupt to occur at the completion of the entire DMA chained transfer
		1 Enable DMA Channel interrupt to occur at the completion of current DMA sequence
29:0 (R/W)	ADDR	Next Descriptor (Chain) Pointer Address. The <code>EMDMA_CHNPTR.ADDR</code> bit field provides the address of the next descriptor in memory.

Internal Count Register

The `EMDMA_CNT0` register contains the number of words to be transferred for channel 0 DMA. Note: If Delay Line DMA is enabled then the `EMDMA_CNT0` register serves as the count register for the Delay Line writes.

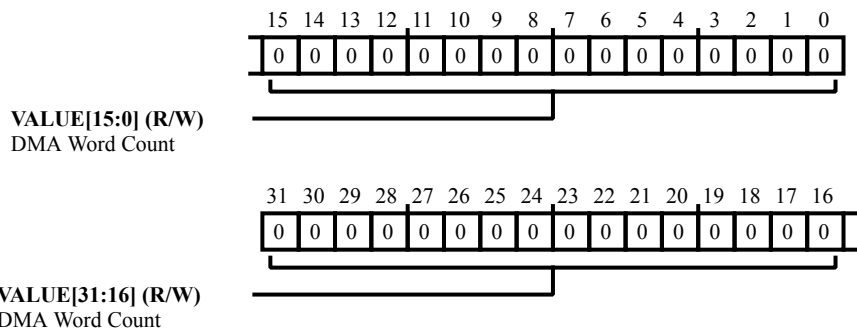


Figure 31-13: EMDMA_CNT0 Register Diagram

Table 31-17: EMDMA_CNT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	DMA Word Count. The <code>EMDMA_CNT0.VALUE</code> bit field is the DMA word count.

External Count Register

The `EMDMA_CNT1` register contains the number of words to be transferred for channel 1 DMA. Note: If Delay Line DMA is enabled then the `EMDMA_CNT1` register serves as the count register for the Delay Line writes.

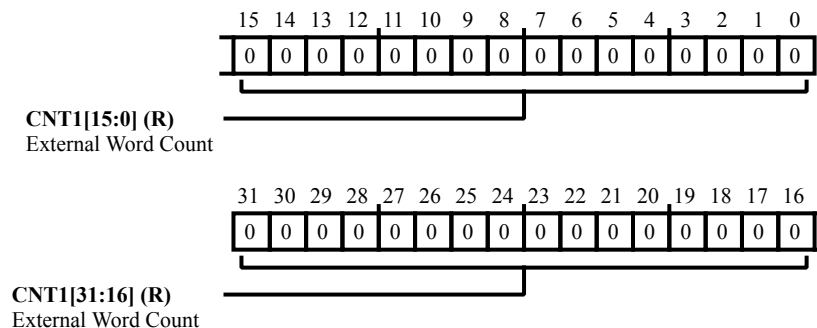


Figure 31-14: EMDMA_CNT1 Register Diagram

Table 31-18: EMDMA_CNT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	CNT1	External Word Count. The <code>EMDMA_CNT1 . CNT1</code> bit field is the external word count.

External Memory DMA Control Register

The `EMDMA_CTL` register contains bits that enable and configure EMDMA and indicate DMA transfer status.

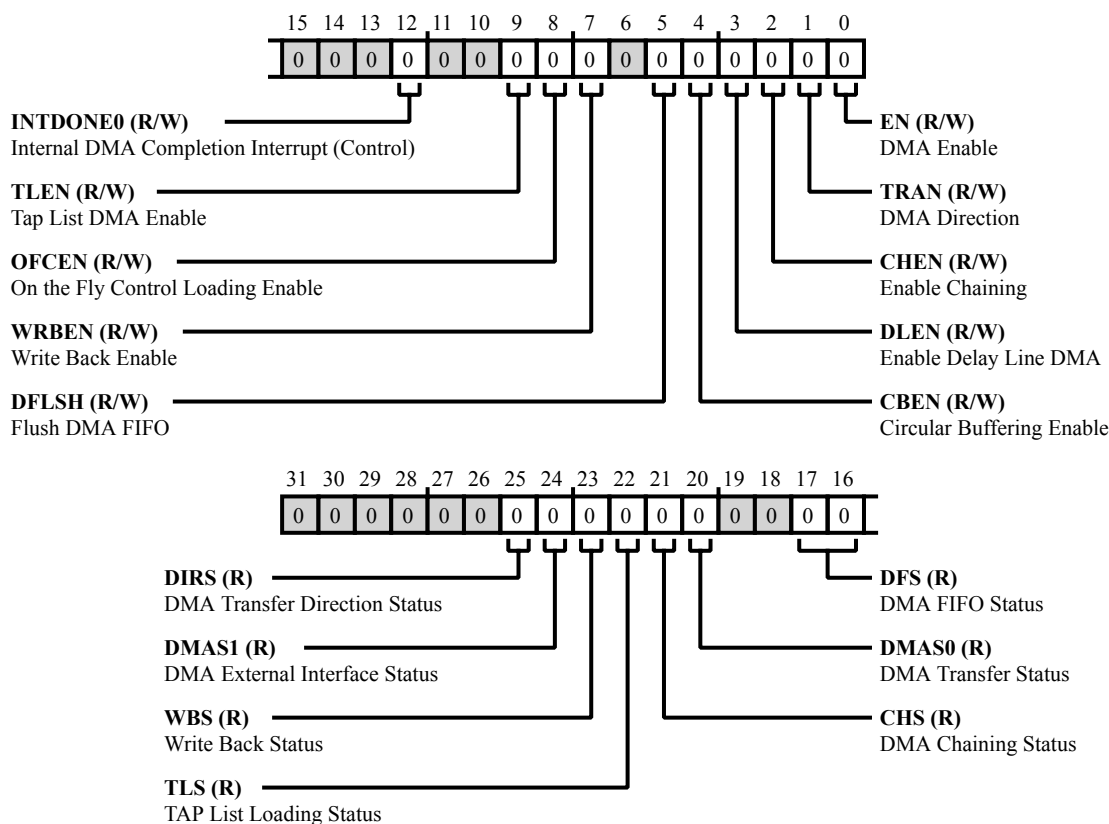


Figure 31-15: EMDMA_CTL Register Diagram

Table 31-19: EMDMA_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/NW)	DIRS	DMA Transfer Direction Status.
		The <code>EMDMA_CTL.DIRS</code> bit provides the DMA transfer status direction. This is useful for delay line DMA where the transfer direction changes with the state of the DMA state machine. For standard DMA, the <code>EMDMA_CTL.DIRS</code> bit reflects the state of the <code>EMDMA_CTL.TRAN</code> bit.
		0 DMA direction is Channel 1 Reads
		1 DMA direction is Channel 1 Writes

Table 31-19: EMDMA_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/NW)	DMAS1	DMA External Interface Status. The EMDMA_CTL.DMAS1 bit provides the DMA channel 1 transfer status.
		0 Channel 1 DMA does not have any access pending
		1 Channel 1 DMA access are pending
23 (R/NW)	WBS	Write Back Status. The EMDMA_CTL.WBS bit provides the delay line write pointer write back status.
		0 Write pointer write back is not active
		1 Write pointer write back is active
22 (R/NW)	TLS	TAP List Loading Status. The EMDMA_CTL.TLS bit provides the DMA tap list loading status.
		0 TAP list loading is not active
		1 TAP list loading is active
21 (R/NW)	CHS	DMA Chaining Status. The EMDMA_CTL.CHS bit provides the DMA chaining status.
		0 DMA chain loading is not active
		1 DMA chain loading is active
20 (R/NW)	DMAS0	DMA Transfer Status. The EMDMA_CTL.DMAS0 bit provides the DMA channel 0 transfer status.
		0 DMA idle
		1 DMA in progress
17:16 (R/NW)	DFS	DMA FIFO Status. The EMDMA_CTL.DFS bit field provides the DMA FIFO status.
		0 FIFO EMPTY
		1 FIFO Partially Full
		2 Reserved
		3 FIFO Full

Table 31-19: EMDMA_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	INTDONE0	Internal DMA Completion Interrupt (Control). The EMDMA_CTL.INTDONE0 bit configures when the DMA complete interrupt is generated. The EMDMA_CTL.INTDONE0 =1 setting is provided for backward compatibility with older SHARC processors.
		0 Interrupt on access completion (Channel 0 or Channel 1 DMA completion)
		1 Interrupt on Channel 0 DMA completion.
9 (R/W)	TLEN	Tap List DMA Enable. The EMDMA_CTL.TLEN bit enables scatter/gather tap list DMA.
		0 Disables the tap list based scatter/gather DMA
		1 Enables the tap list based scatter/gather DMA
8 (R/W)	OFCEN	On the Fly Control Loading Enable. The control bits in EMDMA_CHNPTR register are used to describe the next TCB behavior if the EMDMA_CTL.OFCEN bit is set and therefore the DMA controls can be changed from TCB to TCB. 0 = disables the control bits in the EMDMA_CHNPTR register 1 = Enables the control bits in the EMDMA_CHNPTR register. If chaining is enabled with EMDMA_CTL.OFCEN bit set then the EMDMA_CTL.TRAN bit has no effect, and direction is determined by EMDMA_CHNPTR.CPDR bit.
7 (R/W)	WRBEN	Write Back Enable. The EMDMA_CTL.WRBEN bit enables write back of the EIEP register after reads and or writes. Write back is automatically enabled for delay line DMA. WRBEN is applicable only if chaining is enabled (EMDMA_CTL.CHEN =1).
5 (R/W)	DFLSH	Flush DMA FIFO. The EMDMA_CTL.DFLSH bit flushes the DMA FIFO. The buffer is only flushed if this bit is set. It can be set with the enable bit. Setting this bit also clears the EMDMA_CTL.DFS bit.
4 (R/W)	CBEN	Circular Buffering Enable. The EMDMA_CTL.CBEN bit enables circular buffering. Circular buffering can be used with normal DMA as well, if circular buffering is enabled with chaining for normal DMA then ELEP and EBEP should be part of the TCB.
		0 Disables circular buffering with delay line DMA
		1 Enables circular buffering with delay line DMA

Table 31-19: EMDMA_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	DLEN	Enable Delay Line DMA. The EMDMA_CTL.DLEN bit enables delay line DMA. This bit is applicable only if the EMDMA_CTL.CHEN bit =1.
		0 Delay-line DMA disabled
		1 Delay-line DMA enabled
2 (R/W)	CHEN	Enable Chaining. The EMDMA_CTL.CHEN bit enables DMA chaining.
		0 Chaining disabled
		1 Chaining enabled
1 (R/W)	TRAN	DMA Direction. The EMDMA_CTL.TRAN bit determines the DMA data direction. Note: If delay line DMA is enabled then this bit does not have any effect. For delay line DMA, transfer direction depends on the state of delay line transfers. For Internal-Internal or External-External DMA this bit has to be set.
		0 Write through Channel 0 (Channel 1 reads)
		1 Read from Channel 0(Channel 1 writes)
0 (R/W)	EN	DMA Enable. The EMDMA_CTL.EN bit enables DMA.
		0 Disable DMA
		1 Enable DMA

Internal Index Register

The `EMDMA_INDX0` register contains the start address of the buffer for Channel 0 DMA. Note: For delay line DMA the `EMDMA_INDX0` register serves as the delay line write index which is the start address of the channel 0 DMA buffer for the channel 1 write data.

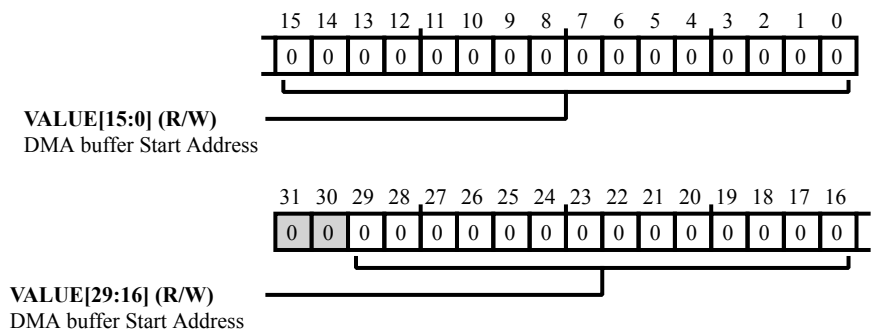


Figure 31-16: EMDMA_INDX0 Register Diagram

Table 31-20: EMDMA_INDX0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	DMA buffer Start Address. The <code>EMDMA_INDX0.VALUE</code> bit field is written with the 30 MSBs of the Word Aligned Byte addresses.

External Index Register

The `EMDMA_IND1` register contains the start address of the buffer for channel 1 DMA.

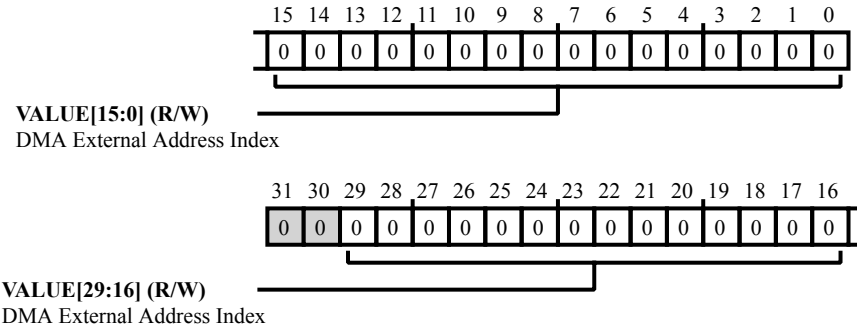


Figure 31-17: EMDMA_IND1 Register Diagram

Table 31-21: EMDMA_IND1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	DMA External Address Index. The <code>EMDMA_IND1.VALUE</code> bit field is the DMA external address index.

Internal Modifier Register

The `EMDMA_MOD0` register contains the channel 0 DMA address modifier.

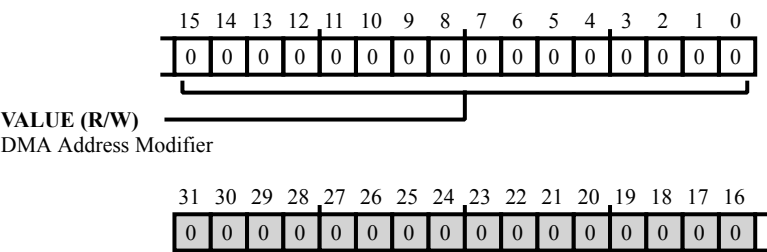


Figure 31-18: EMDMA_MOD0 Register Diagram

Table 31-22: EMDMA_MOD0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	DMA Address Modifier. The <code>EMDMA_MOD0.VALUE</code> bit field is the DMA address modifier.

External Modifier Register

The `EMDMA_MOD1` register contains the external (channel 1 DMA) address modifier.

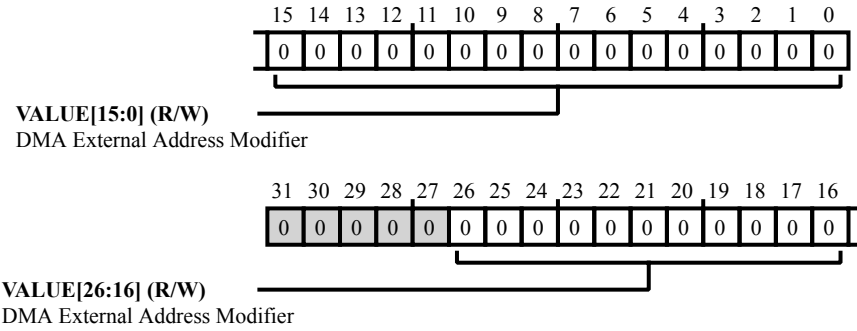


Figure 31-19: EMDMA_MOD1 Register Diagram

Table 31-23: EMDMA_MOD1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
26:0 (R/W)	VALUE	DMA External Address Modifier. The <code>EMDMA_MOD1.VALUE</code> bit field is the DMA external address modifier.

Delay Line Tap Count Register

The `EMDMA_TCNT` register is the tap count register for Delay Line DMA. This register holds the length of the tap list (the number of taps). The total number of words read from the delay line is equal to the `EMDMA_TCNT` (tap count) multiplied by the `EMDMA_CNT1` (read block size).

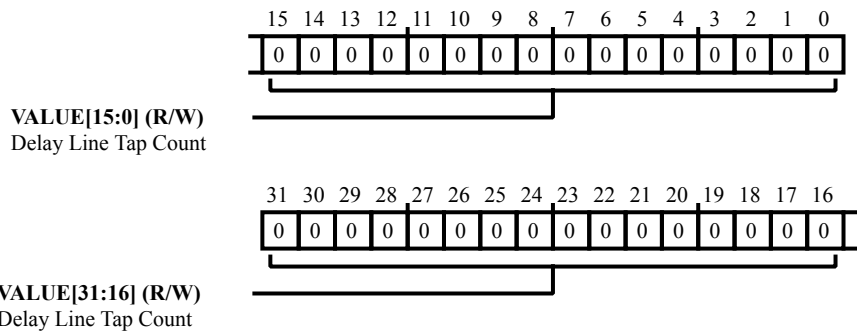


Figure 31-20: EMDMA_TCNT Register Diagram

Table 31-24: EMDMA_TCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Delay Line Tap Count. The <code>EMDMA_TCNT.VALUE</code> bit field is the tap count register for Delay Line DMA.

Tap List Pointer Register

The `EMDMA_TPTR` register holds the address of an array in memory which holds offsets to be used when accessing a Delay-line in system memory. The offset represents the first address of each read block. Note that the lower 30-bits of this register are to be written with 30 MSBs of the Word Aligned Byte address of the Array.

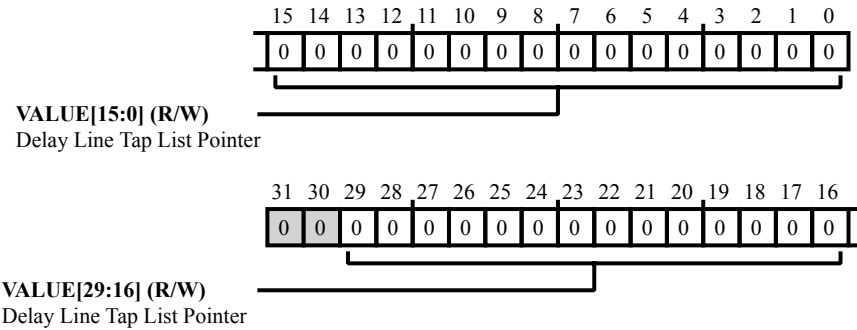


Figure 31-21: EMDMA_TPTR Register Diagram

Table 31-25: EMDMA_TPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Delay Line Tap List Pointer. The <code>EMDMA_TPTR.VALUE</code> bit field contains the offsets to be used when accessing a Delay-line in system memory.

32 Cyclic Redundancy Check (CRC)

The CRC peripheral performs the cyclic redundancy check (CRC) of the block of data that is presented to the peripheral. The peripheral provides a means to verify periodically the integrity of the system memory, the contents of memory-mapped registers (MMRs), or communication message objects. It is based on a CRC32 engine that computes the signature of 32-bit data presented to the peripheral.

The dedicated hardware compares the calculated signature of the operation to a pre-loaded expected signature. If the two signatures fail to match, the peripheral generates an error.

The source channel of the memory-to-memory DMA channels can provide data. The CRC optionally forwards data to memory through the destination DMA channel. Alternatively, the peripheral supports data presented by any qualified master of the CRC peripheral bus.

The CRC peripheral implements a reduced table-look-up algorithm to compute the signature of the data. The CRC uses a programmable 32-bit CRC polynomial to generate the look-up table (LUT) contents automatically.

More CRC peripheral modes allow for initializing large memory sections with a constant value, or for verifying that sections of memory are equal to a constant value.

NOTE: CRC is supported by MDMA0 and MDMA1 channels only.

CRC Features

The CRC peripheral supports a number of key features.

- Memory scan modes for memory verification
- Memory transfer modes for on-the-fly CRC calculations while transferring data from one memory to another
- A programmable 32-bit CRC polynomial with automatic LUT generation
- Data mirroring options

The CRC module also includes the following features.

- CRC checksum computation and comparison modes
- 32-bit programmable CRC polynomial with bit reverse option
- Automatic look-up table (LUT) generation

- Data mirroring options for endian and reflected polynomial cases
- Automatic clear and preset of results
- Fault and error interrupt reporting
- DMA and MMR based operation

Because the CRC module is closely tied to memory-to-memory DMA (MDMA) channel pairs, the use cases include the following features.

- Memory scan mode with CRC compute or compare
- Memory transfer mode with CRC compute or compare
- Memory fill operation with 32-bit data patterns
- Memory verify operation
- MMR write access to FIFO of destination DMA
- MMR read access to FIFO of source DMA
- Profiting from advanced DMA features, like descriptor mode and bandwidth control or monitor

CRC Functional Description

The CRC peripheral supports a number of modes of operation that allow for the initialization and verification of regions of memory. The peripheral supports efficient memory-fill and verification operations on regions of memory with or against a constant value. These modes of operation do not require the CRC engine to calculate a signature. Other modes of operation allow for the calculation of CRC signature and verification for a memory region. The modes allow for on-the-fly CRC calculation when performing memory-to-memory DMA transfers from one memory region to another.

To minimize the need for core accesses, the peripheral interfaces with one or more (depending on processor features) memory-to-memory DMA (MDMA) channels. This connectivity permits flexible configuration, in which data can be written-to or read-from the peripheral using DMA transactions, core transactions, or a combination of both.

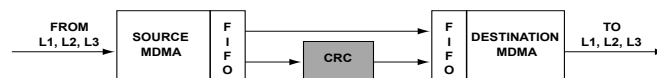


Figure 32-1: Memory Flow

ADSP-2159x CRC Register List

The Cyclic Redundancy Check (CRC) unit includes the data comparison, polynomial operation, and look up table generation features needed for CRC operation. The CRC provides CRC protection as specified by many functional safety requirements. This unit facilitates the system software's ability to periodically check the correctness of the code/data available in memory. A set of registers govern CRC operations. For more information on CRC functionality, see the CRC register descriptions.

Table 32-1: ADSP-2159x CRC Register List

Name	Description
CRC_COMP	Data Compare Register
CRC_CTL	Control Register
CRC_DCNT	Data Word Count Register
CRC_DCNTCAP	Data Count Capture Register
CRC_DCNTRL	Data Word Count Reload Register
CRC_DFIFO	Data FIFO Register
CRC_FILLVAL	Fill Value Register
CRC_INEN	Interrupt Enable Register
CRC_INEN_CLR	Interrupt Enable Clear Register
CRC_INEN_SET	Interrupt Enable Set Register
CRC_POLY	Polynomial Register
CRC_RESULT_CUR	CRC Current Result Register
CRC_RESULT_FIN	CRC Final Result Register
CRC_STAT	Status Register

ADSP-2159x CRC Interrupt List

Table 32-2: ADSP-2159x CRC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
190	CRC0_ERR	CRC0 Error	Level	
191	CRC1_ERR	CRC1 Error	Level	
196	CRC0_DCNTEXP	CRC0 Data Count Expiration	Level	
197	CRC1_DCNTEXP	CRC1 Data Count Expiration	Level	
198	CRC2_ERR	CRC2 Error	Level	
199	CRC3_ERR	CRC3 Error	Level	
204	CRC2_DCNTEXP	CRC2 Data Count Expiration	Level	
205	CRC3_DCNTEXP	CRC3 Data Count Expiration	Level	

CRC Definitions

To make the best use of the CRC, it is useful to understand the following terms.

CRC

Acronym for Cyclic Redundancy Check. An error detection code that can detect changes within a block of data.

CRC Polynomial

The 32-bit polynomial used by the CRC engine to generate the look-up table required for the CRC implementation

LUT

Acronym for the Look-up Table. The look-up table is automatically generated from the supplied 32-bit CRC polynomial.

DMA

Acronym for Direct Memory Access. Used to describe a data transfer that takes place through a DMA channel allowing data distribution around a system without intervention from the core.

MDMA

Acronym for Memory-To-Memory DMA transfer that often requires the use of two DMA channels to transfer data from one memory region to another memory region. One DMA channel is configured as a source channel and the second as a destination channel.

CRC Block Diagram

The *CRC Block Diagram* shows the functional block diagram of the CRC. The following sections describe the blocks.

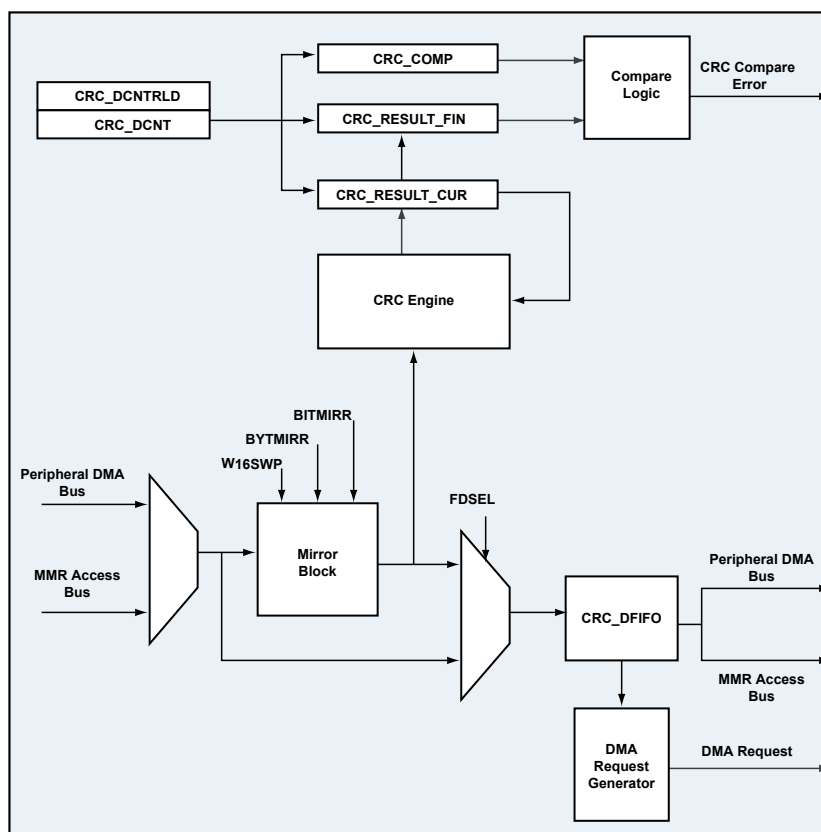


Figure 32-2: CRC Block Diagram

Peripheral DMA Bus

The CRC peripheral provides both an incoming and outgoing datapath to the peripheral DMA bus. The MDMA source channel is interfaced to the incoming datapath providing data to the CRC peripheral. For memory transfer and data fill modes, the CRC uses the MDMA destination channel to either output the data from the CRC FIFO or use the data for the fill operation.

MMR Access Bus

The core uses the MMR access bus to access all the memory-mapped registers of the peripheral for configuration, status, and debug purposes. The core can also use the MMR access bus to feed data to the CRC peripheral or read data from the FIFO of the CRC peripheral. The CRC operation is an alternative to the DMA channel operation to read data from the FIFO.

Data received by MMR writes can transfer to destination DMA. Similarly, data received by source DMA can be output through the MMR interface. Optionally, intermediate results can be made available to the MMR interface.

Mirror Block

The mirror block individually controls bit-reversing of the polynomial, the computation results, and the expected result. Bit mirroring, byte mirroring, word swapping, and any combination of these operations can control endian and the reflection of processed data.

Data FIFO

The CRC data FIFO is a 32-bit-wide 4-entry FIFO. The FIFO is accessible to both the peripheral DMA bus and the MMR access bus. The FIFO status is accessible from the [CRC_STAT](#) register.

DMA Request Generator

The DMA request generator is responsible for granting incoming DMA requests from the source DMA channel and issuing outgoing DMA requests to the destination DMA channel.

CRC Engine

The CRC engine is a 32-bit CRC engine that implements the reduced table look-up scheme. The CRC engine provides support for a user-programmable 32-bit polynomial that the CRC uses to load the look-up table parameters required for the CRC calculation. The CRC engine is a single cycle implementation operating on 32 bits of data per cycle.

Compare Logic

The compare logic takes the final CRC signature and compares it to the expected CRC signature, generating a CRC compare error when the signatures do not match. A compare error can flag a system fault.

CRC Architectural Concepts

A 32-bit polynomial is required before calculation of the CRC signature can occur. The CRC uses the polynomial to generate the contents of an internal look-up table that the reduced table look-up implementation requires. The look-up table is automatically generated when the polynomial is written. It must be initialized prior to any operation that requires the use of the CRC engine.

The mirror block logic can manipulate the data presented to the CRC engine before the CRC uses the data in the calculation of the CRC signature. The data mirror operation is configurable to allow for bit reversing, byte reversing, and 16-bit word swapping operations on the incoming data. For memory transfer compute-and-compare operations, programs can configure the peripheral to output the data in the same form in which it was received. Or, the operation can output the mirrored data in the same manner that it is presented to the CRC engine.

While the CRC peripheral is in operation, the status of the FIFO is continually updated and reflected in the [CRC_STAT](#) register. The FIFO status is required for core-based accesses to the CRC peripheral. The status indicates when:

- The CRC peripheral can receive data

- Data is available for reading from the FIFO
- The result of the `CRC_RESULT_CUR` register has been updated

The status of the `CRC_RESULT_CUR` register indicates that the current CRC calculation has completed and the result is available.

Look-up Table

The look-up table consists of a set of sixteen 32-bit registers that hardware populates automatically when a write access takes place to the `CRC_POLY` register. 16 clock cycles are required to generate all 16 look-up table entries. The status of the process for generating the look-up table is reflected in `CRC_STAT.LUTDONE` allowing for software to poll on the completion of the event or for generation of an interrupt.

NOTE: Hardware must populate the look-up table before any operation using the CRC peripheral can take place, even if the operation does not use the CRC engine. The peripheral does not issue any data requests until the table generation process is complete. In addition, the `CRC_STAT.IBR` field, that indicates the input buffer status as required for core-based transfers, is only valid upon completion of the process for generating the look-up table.

Data Mirroring

The data mirror block can be configured to manipulate the incoming data before the data passes to the CRC engine and, optionally, to the FIFO. This configuration allows the peripheral to handle various forms of endianness and to function with reflected polynomials.

There are three configuration bits that control the data mirroring process: `CRC_CTL.BITMIRR`, `CRC_CTL.BYTMIRR`, and `CRC_CTL.W16SWP`. The *Data Mirroring Options* table details how these options affect the incoming data and the output generated by the mirror block.

Table 32-3: Data Mirroring Options

W16SWP	BYTMIRR	BITMIRR	Output Data
0	0	0	Dout[31:0] = Din[31:0]
0	0	1	Dout[31:0] = Din[24:31],Din[16:23],Din[8:15],Din[0:7]
0	1	0	Dout[31:0] = Din[7:0],Din[15:8],Din[23:16],Din[31:24]
0	1	1	Dout[31:0] = Din[0:7],Din[8:15],Din[16:23],Din[24:31]
1	0	0	Dout[31:0] = Din[15:0], D[31:16]
1	0	1	Dout[31:0] = Din[8:15],Din[0:7], Din[24:31],Din[16:23]
1	1	0	Dout[31:0] = Din[23:16],Din[31:24], Din[7:0],Din[15:8]
1	1	1	Dout[31:0] = Din[16:23],Din[24:31], Din[0:7],Din[8:15]

When the CRC is configured to operate in the memory transfer compute-and-compare mode, the bit-reversed output data can be written to the FIFO. This feature is controlled through the `CRC_CTL.FDSEL` field.

In addition to providing bit swapping and mirror options to the incoming data, the CRC peripheral also supports bit mirroring on the following registers.

- `CRC_RESULT_CUR` and `CRC_RESULT_FIN`, controlled through the `CRC_CTL.RSLTMIRR` field. When mirroring is enabled, the values to be written to these registers are fully bit-reversed before the write operation occurs.
- `CRC_POLY`, controlled through the `CRC_CTL.POLYMIRR` field. When mirroring is enabled, the 32-bit polynomial is fully bit-reversed before the write operation to the register occurs.
- `CRC_COMP`, controlled through the `CRC_CTL.CMPMIRR` field. When mirroring is enabled, the contents to be loaded to this register are fully bit-reversed before the write operation occurs.

FIFO Status and Data Requests

The CRC peripheral provides indication of the input and output buffer status through `CRC_STAT.IBR` and `CRC_STAT.OBR` respectively. For core-based operations, software must monitor these status fields prior to writing to or reading from the CRC FIFO. No write to the CRC FIFO can occur while `CRC_STAT.IBR` indicates that the buffer is not ready to accept data. Similarly, the CRC FIFO cannot be read until `CRC_STAT.OBR` indicates that data is available.

The memory scan modes of operation only require the monitoring of the input buffer status. The memory transfer, compute-and-compare mode uses both input and output buffer status. If the current result of the CRC computation is required, then software must verify that the current operation has completed and that the intermediate result is ready. The `CRC_STAT.IRR` indicates the status.

NOTE: The memory transfer fill mode of operation requires the use of a DMA channel. The CRC does not support core reads from the CRC FIFO for this mode of operation.

Memory transfer, compute-and-compare mode uses burst transactions to make the most efficient use of the available resources. In this mode, when the FIFO is initially empty and the peripheral is enabled, the `CRC_STAT.IBR` bit indicates that the CRC is ready to accept data. The peripheral generates data requests to the source DMA channel (if the CRC uses DMA). While the number of words remaining in the `CRC_DCNT` register is greater than the FIFO depth, the peripheral issues data requests or accepts incoming data in bursts. The peripheral continues until the CRC FIFO becomes full.

Once full, the `CRC_STAT.IBR` and `CRC_STAT.OBR` bits are updated, and then the CRC issues outgoing data requests. Only when the FIFO is empty can the peripheral accept further incoming data, and the `CRC_STAT.IBR` and `CRC_STAT.OBR` bits are updated once again.

Once `CRC_DCNT` is decremented such that the number of words waiting for processing is less than the number required to fill the FIFO, the burst mode of operation is disabled. Incoming data is accepted as long as the FIFO is not full. Outgoing data is available as long the FIFO is not empty. Therefore, there are no restrictions requiring the word count to be a multiple of the FIFO depth.

All other CRC modes of operation indicate that incoming data can be accepted as long as the FIFO is not full. Outgoing data is available as long as the FIFO is not empty.

The `CRC_CTL.OBRSTALL` and `CRC_CTL.IRRSTALL` bit configurations also influence how the CRC generates data requests and status bits. The following list describes the bits.

- The `CRC_CTL.OBRSTALL` bit can be configured such that the CRC peripheral stalls as soon as there is output data available in the FIFO. Use this mode of operation only in memory transfer, compute-and-compare mode. This mode results in the processing of one 32-bit word at a time. The peripheral does not request or accept incoming data until the current value being processed is read from the peripheral.
- The `CRC_CTL.IRRSTALL` bit can be configured so that the CRC peripheral stalls all further incoming data requests until the `CRC_RESULT_CUR` register is read after being updated. Use this mode of operation for CRC signature generation. It is not applicable to memory transfer data-fill mode or memory scan data-verify mode of operation.

CRC Operating Modes

The following sections describe the various operating modes of the CRC interface.

Data Transfer Modes

The CRC peripheral supports two main categories of operation involving data transfers:

- Memory scan mode
- Memory transfer mode

Memory scan modes are read-only operations that allow the contents of memory to be read into the peripheral and verified for correctness. There are two forms of memory scan mode:

- CRC compute-and-compare performs a CRC calculation on data presented to the peripheral and compares the CRC result with a pre-determined and pre-loaded result. An error is generated when the results differ.
- Data verify compares each 32-bit data word presented to the CRC peripheral to a pre-loaded 32-bit value and generates an error when the data differs.

Both of these modes of operation require, at the most, a single DMA channel to read the data from memory into the peripheral. No data is forwarded to the data output or destination DMA. The CRC can also use core-driven transfers for either of these modes of operation.

The memory transfer modes involve memory write or memory read-and-write operations allowing for memory to be initialized or transferred from one region of memory to another. There are two forms of memory transfer mode:

- CRC compute-and-compare performs a full data transfer from one memory region to another memory region. The CRC generates a signature on the data presented and compares it with a pre-determined and pre-loaded result. An error is generated when the results differ.
- Data fill initializes a region of memory with a pre-loaded 32-bit constant value.

The CRC compute-and-compare mode of operation requires both incoming and outgoing data channels. The operation occurs either using DMA channels, using core-driven write or read operations to and from the FIFO or using

a combination of both. The data fill mode of operation requires only a memory write DMA destination channel—this mode does not support core driven operations.

Memory Scan Compute-and-Compare Mode

In this mode of operation, the CRC engine of the peripheral is enabled. The mode is configured through the `CRC_CTL.OPMODE` field and the CRC engine performs a 32-bit CRC operation on the incoming data stream.

The length of the data stream is configured through the `CRC_DCNT` register. The accumulated result of the CRC operation is contained in the `CRC_RESULT_CUR` register. As the CRC engine processes each 32-bit word, the `CRC_DCNT` register is decremented and `CRC_RESULT_CUR` is updated.

Once `CRC_DCNT` decrements to zero, the contents of the `CRC_RESULT_CUR` register are copied to `CRC_RESULT_FIN` and `CRC_STAT.DCNTEXP` is updated accordingly. The CRC uses the `CRC_COMP` register to store the expected result of the operation. After the CRC calculation, `CRC_COMP` is compared with `CRC_RESULT_FIN` and `CRC_STAT.CMPERR` is updated to reflect the status of the compare operation. `CRC_STAT.CMPERR` must be cleared before the next CRC operation is performed.

The CRC peripheral also contains the `CRC_DCNTRLD` register. The CRC uses this register to reload `CRC_DCNT` upon completion of the CRC operation in preparation for the next transfer.

The initial seed of the CRC computation can be configured through `CRC_CTL.AUTOCLRZ` and `CRC_CTL.AUTOCLRF`. This configuration provides a way to reset `CRC_RESULT_CUR` to 0x00000000, 0xFFFFFFFF or to leave the current register contents untouched for the next operation.

The peripheral can be configured to allow for the compare error and data expiration events to generate an interrupt.

Memory Scan Data Verify

In this mode of operation, the CRC engine of the peripheral is not required. The mode is enabled through the `CRC_CTL.OPMODE` field. Each 32-bit word of the data stream is compared with a constant value that is stored in the `CRC_COMP` register. The `CRC_DCNT` register contains the number of words for comparison. The `CRC_DCNT` register is decremented upon receiving a new 32-bit word from the data stream. If the compare operation fails, the `CRC_STAT.CMPERR` bit is updated and the contents of `CRC_DCNT` are captured in the `CRC_DCNTPCAP` register. This information can be used to identify the location in the data stream where the error occurred. Clear the `CRC_STAT.CMPERR` field to reenables capturing of further errors.

Once `CRC_DCNT` decrements to zero, `CRC_STAT.DCNTEXP` is updated accordingly to signal the end of the operation. The peripheral can be configured to allow for the compare error and data expiration events to generate an interrupt.

Memory Transfer Compute-and-Compare Mode

In this mode of operation, the CRC engine of the peripheral is enabled. The mode is configured through the `CRC_CTL.OPMODE` field and the CRC engine performs a 32-bit CRC operation on the incoming data stream.

The length of the data stream is configured through the `CRC_DCNT` register. The accumulated result of the CRC operation is contained in the `CRC_RESULT_CUR` register. As the CRC engine processes each 32-bit word, the `CRC_DCNT` register is decremented and `CRC_RESULT_CUR` is updated.

Once `CRC_DCNT` decrements to zero, the contents of the `CRC_RESULT_CUR` register are copied to `CRC_RESULT_FIN` and `CRC_STAT.DCNTEXP` is updated accordingly. The CRC uses the `CRC_COMP` register to store the expected result of the operation. Upon completion of the CRC calculation, `CRC_COMP` is compared with `CRC_RESULT_FIN` and `CRC_STAT.CMPERR` is updated to reflect the status of the compare operation. Clear `CRC_STAT.CMPERR` before the next CRC operation is performed.

The CRC peripheral also contains `CRC_DCNTRLD` register. The CRC uses this register to reload `CRC_DCNT` upon completion of the CRC operation in preparation for the next transfer.

The initial seed of the CRC computation can be configured through `CRC_CTL.AUTOCLRZ` and `CRC_CTL.AUTOCLRF`. This configuration provides a means to reset `CRC_RESULT_CUR` to 0x00000000, 0xFFFFFFFF or to leave the current register contents untouched for the next operation.

The peripheral can be configured to allow for the compare error and data expiration events to generate an interrupt.

Memory Transfer Data Fill Mode

In this mode of operation, the CRC engine of the peripheral is not required. The mode is enabled through the `CRC_CTL.OPMODE` field. The `CRC_FILLVAL` register is written with a 32-bit value. The CRC uses this value to initialize a block memory through the memory-to-memory DMA destination channel. When the CRC peripheral and the DMA destination channel are enabled, the contents of the `CRC_FILLVAL` register is written to the DMA channel to initialize the memory region. The `CRC_DCNT` register contains the number of words for the write operation.

Once `CRC_DCNT` decrements to zero, `CRC_STAT.DCNTEXP` is updated accordingly to signal the end of the operation. The peripheral can be configured to allow for the data expiration event to generate an interrupt.

CRC Event Control

The CRC peripheral can enable certain CRC status operations to generate an interrupt event to the system event controller. There, a CRC error can be qualified as a system fault.

Interrupt Signals

The CRC peripheral can generate two interrupt requests that are optionally enabled as interrupts with the Arm core, or as events to SEC1 for fault operations. One is a CRC status interrupt and the other is a CRC error interrupt.

The `CRC_STAT.CMPERR` status bit can be configured as an interrupt and is signaled through the CRC error interrupt signal. The `CRC_STAT.CMPERR` status field is set whenever the CRC peripheral performs a compare operation that fails. This status can be the result of a failed memory scan data-verify operation that compares the contents of a memory range with a constant 32-bit value. Or, it can be the result of a CRC signature calculated for a memory region that does not match the expected pre-programmed result for a memory-compare operation.

The `CRC_STAT.DCNTEXP` status bit is set when the `CRC_DCNT` register has decremented to zero. The status indicates that the CRC peripheral has now processed all the data requested for the current CRC operation. The CRC can also use this signal to generate an interrupt. The interrupt is signaled on the CRC status interrupt signal.

Both these status bits can be configured to generate and interrupt through the [CRC_INEN](#) register. The [CRC_INEN](#) register also has bit set, [CRC_INEN_SET](#), and bit clear [CRC_INEN_CLR](#) equivalent registers that the CRC uses for the enabling and disabling of these interrupt sources.

The [CRC_STAT](#) register has two write one to clear (W1C) fields for clearing the two interrupt sources.

NOTE: Disabling the CRC peripheral through the [CRC_CTL.BLKEN](#) bit does not result in the clearing of interrupt sources. Clear the interrupt sources using a W1C operation to the [CRC_STAT](#) register.

CRC Programming Model

It is important to note the following restrictions when using the CRC peripheral with the DMA channels:

1. When enabling the CRC peripheral and the DMA channels, enable the CRC peripheral prior to enabling the DMA channels.
2. When disabling the CRC peripheral and the DMA channels, disable the DMA channels prior to disabling the CRC peripheral.

CRC Mode Configuration

Describes a number of tasks showing the various operation modes of the CRC peripheral.

- [Look-up Table Generation](#)
- [Core Driven Memory Scan Compute-and-Compare Mode](#)
- [DMA Driven Memory Scan Compute-and-Compare Mode](#)
- [Core Driven Memory Scan Data Verify Mode](#)
- [DMA Driven Memory Scan Data Verify Mode](#)
- [Core Driven Memory Transfer Compute-and-Compare Mode](#)
- [DMA Driven Memory Transfer Compute-and-Compare Mode](#)
- [DMA Driven Memory Transfer Data Fill Mode](#)

Look-up Table Generation

Describes the steps required to initialize the CRC peripheral LUT.

1. Write the 32-bit CRC polynomial of choice to the [CRC_POLY](#) register.

ADDITIONAL INFORMATION: This operation results in the CRC peripheral starting the LUT initialization process. The [CRC_STAT.LUTDONE](#) bit is updated to reflect the operation is in progress.

2. Poll the [CRC_STAT.LUTDONE](#) bit until the status bit indicates that the operation is completed.

The CRC peripheral has completed initialization of all the LUT registers and is now ready for data operations. The `CRC_STAT.LUTDONE` bit remains in the current state until the `CRC_POLY` register is written again, or the peripheral or processor are reset.

Core Driven Memory Scan Compute-and-Compare Mode

Performs CRC signature calculation and verification for a region of memory using core transactions. The CRC peripheral is configured such that it operates in burst mode due to the stalling options configured through disabling the `CRC_CTL` register.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per `CRC_CTL.BLKEN`

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_RESULT_CUR` register.

ADDITIONAL INFORMATION: This register can be initialized to provide an initial seed for the CRC operation that is about to take place.

4. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the pre-calculated final CRC signature result for the memory region that the software uses in the final compare operation.

5. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts. If enabled, ensure that the corresponding interrupt handlers are also configured.

6. Initialize `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan compute-and-compare mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.

- Disable the `CRC_CTL.OBRSTALL` and `CRC_CTL.IRRSTALL` bit options for this task example.
- Configure all mirroring and bit reversal options.

- Configure CRC auto-clear options.

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

7. Write memory region data to the CRC peripheral.

- While `CRC_STAT.IBR` bit indicates that the input buffer is ready, write the `CRC_DFIFO` register with 32-bit data.

ADDITIONAL INFORMATION: Repeat this step until all data has been written.

8. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures that all the data has been processed.

9. Poll the `CRC_STAT.CMPERR` bit if the interrupt was disabled to check for a compare error.

ADDITIONAL INFORMATION: Perform this step only if the compare error interrupt is not enabled.

10. Write to the `CRC_STAT` register to clear both the `CRC_STAT.DCNTEXP` and `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear of these status bits within the interrupt handlers for the respective interrupts.

The CRC compute-and-compare operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation.

The integrity check of the memory through the expected CRC signature has completed. The final result is indicated through the `CRC_STAT.CMPERR` bit and the corresponding interrupt when enabled.

Clear any WIC CRC status bits before performing more CRC operations.

DMA Driven Memory Scan Compute-and-Compare Mode

Performs CRC signature calculation and verification for a region of memory using DMA transactions. The CRC peripheral is configured such that it operates in the burst mode of operation due to the stalling options configured through disabling `CRC_CTL`.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` bit.

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_RESULT_CUR` register.

ADDITIONAL INFORMATION: This register can be initialized to provide an initial seed for the CRC operation that is about to take place.

4. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the pre-calculated final CRC signature result for the memory region that the software uses in the final operation.

5. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts, as needed. If enabled, ensure that the corresponding interrupt handlers are also configured.

6. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan compute compare mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.
 - Disable the `CRC_CTL.OBRSTALL` and `CRC_CTL.IRRSTALL` bit options for this task example.
 - Configure all mirroring and bit reversal options.
 - Configure all CRC auto clear options.

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

7. Configure and enable the memory-to-memory source DMA channel for memory read STOP mode.

ADDITIONAL INFORMATION: This step starts the data transfer from the memory region and writes the data to the CRC peripheral.

8. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if the counter expired interrupt is disabled. Polling ensures all the data has been processed.

9. Poll the `CRC_STAT.CMPERR` bit if the interrupt was disabled to check for a compare error.

ADDITIONAL INFORMATION: Perform this step only if the compare error interrupt is not enabled.

10. Write the `CRC_STAT` register to clear both the `CRC_STAT.DCNTEXP` and `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear these status bits within the interrupt handlers for the respective interrupts.

The CRC compute-and-compare operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation.

The integrity check of the memory through the expected CRC signature has completed and the final result indicated is through `CRC_STAT.CMPERR` and the corresponding interrupt, when enabled.

Clear any W1C CRC status bits before performing a further CRC operation. Clear any W1C status bits of the memory-to-memory source DMA channel before the next CRC operation.

Core Driven Memory Scan Data Verify Mode

Reads a region of memory using core transactions and performs a compare operation on each 32-bit word against a single pre-loaded 32-bit constant. The compare error interrupt is enabled to capture and log the location of any compare errors.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per `CRC_CTL.BLKEN`

The interrupt service routine for the compare error interrupt reads and stores the contents of `CRC_DCNTCAP` register to a buffer before clearing the compare error interrupt.

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the 32-bit constant that the memory region is expected to be filled with. Each 32 bit of data presented to the peripheral is compared with this value.

4. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts. If enabled, ensure that the corresponding interrupt handlers are also configured.

5. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan data verify mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

6. Write memory region data to the CRC peripheral.

- a. Poll the `CRC_STAT.IBR` bit until input buffer is ready.
- b. Write the `CRC_DFIFO` register with 32-bit data.

ADDITIONAL INFORMATION: Repeat these two steps until the entire memory region has been written to the CRC peripheral.

7. Poll the `CRC_INEN_SET.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures all the data has been processed.

8. Check if the buffer used to capture the `CRC_DCNTCAP` register upon a compare error has any new entries.

ADDITIONAL INFORMATION: The values captures in the buffer provide a means to locate where in the memory region the failures occurred.

9. Write to the `CRC_STAT` to clear both the `CRC_INEN_SET.DCNTEXP` and `CRC_INEN.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, the clear these status bits within the interrupt handlers for the respective interrupts.

The CRC memory scan-verify operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation.

The result of the integrity check of the memory with the 32-bit constant is indicated through the `CRC_INEN.CMPERR` bit and the corresponding interrupt, when enabled. Each comparison error is traceable due to the logging of `CRC_DCNTCAP` from within the compare error interrupt handler.

Clear any W1C CRC status bits before performing a further CRC operation.

DMA Driven Memory Scan Data Verify Mode

The memory scan data verify mode reads a region of memory using DMA transactions and performs a compare operation on each 32-bit word against a single pre-loaded 32-bit constant. The compare error interrupt is enabled to capture and log the location of any compare errors.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` bit

The interrupt service routine for the compare error interrupt reads and stores the contents of the `CRC_DCNTCAP` register to a buffer before clearing the compare error interrupt.

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: The CRC module uses this register to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the 32-bit constant that the memory region is expected to be filled with. Each 32 bit of data presented to the peripheral is compared with this value.

4. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts, as needed. If enabled, ensure that the corresponding interrupt handlers are also configured.

5. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan data verify mode and `CRC_CTL.BLKEN` configured to enable the CRC peripheral.

The CRC peripheral is now enabled and ready for the core or DMA channel to write the data.

6. Configure and enable the memory-to-memory source DMA channel for memory read STOP mode.

ADDITIONAL INFORMATION: This step starts the data transfer from the memory region and writes the data to the CRC peripheral.

7. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures all the data has been processed.

8. Check if the buffer used to capture the `CRC_DCNTCAP` register upon a compare error has any new entries.

ADDITIONAL INFORMATION: The values captures in the buffer provide a means to locate where in the memory region the failures occurred.

9. Write the `CRC_STAT` register to clear both the `CRC_STAT.DCNTEXP` and `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear these status bits within the interrupt handlers for the respective interrupts.

The CRC memory scan-verify operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation.

The result of the integrity check of the memory with the 32-bit constant is indicated through the `CRC_STAT.CMPERR` bit and the corresponding interrupt when enabled. Each comparison error is traceable due to the logging of the `CRC_DCNTCAP` register from within the compare error interrupt handler.

Clear any W1C CRC status bits and DMA status bits before performing a further CRC operation.

Core Driven Memory Transfer Compute-and-Compare Mode

The memory transfer compute-and-compare mode performs CRC signature calculation and verification for a region of memory using core transactions while copying the contents to another memory region. The CRC peripheral is configured such that it operates in the burst mode of operation due to the stalling options configured through disabling the `CRC_CTL` register.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` bit

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of the current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_RESULT_CUR` register.

ADDITIONAL INFORMATION: This register can be initialized to provide an initial seed for the CRC operation that is about to take place.

4. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the pre-calculated final CRC signature result for the memory region that the software uses in the final compare operation.

5. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts, as needed. If enabled, ensure that the corresponding interrupt handlers are also configured.

6. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan compute-and-compare mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.
 - a. Disable the `CRC_CTL.OBRSTALL` bit and the `CRC_CTL.IRRSTALL` bit options for this task example.
 - b. Configure all mirroring and bit reversal options.

- c. Configure CRC auto clear options

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

7. Write memory region data to the CRC peripheral and read it back to the new destination.
 - a. While the `CRC_STAT.IBR` bit indicates that the input buffer is ready, write the `CRC_DFIFO` register with 32-bit data.
 - b. While the `CRC_STAT.OBR` bit indicates that the output buffer is ready, read the `CRC_DFIFO` register and store data to new destination.

ADDITIONAL INFORMATION: Repeat these two steps until all required data has been processed through the CRC peripheral and copied to the new destination.

8. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if the counter expired interrupt is disabled. Polling ensures all the data has been processed.

9. Poll the `CRC_STAT.CMPERR` bit if the interrupt was disabled to check for a compare error.

ADDITIONAL INFORMATION: Perform this step only if the compare error interrupt is not enabled.

10. Write the `CRC_STAT` register to clear both `CRC_STAT.DCNTEXP` and `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear these status bits within the interrupt handlers for the respective interrupts.

The CRC compute-and-compare operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation. The memory region has also been copied to its new destination.

The memory region has been copied to a new location and an integrity check of the memory through the expected CRC signature has also completed. The final result is indicated through the `CRC_STAT.CMPERR` bit and the corresponding interrupt when enabled.

Clear any W1C CRC status bits before performing a further CRC operation.

DMA Driven Memory Transfer Compute-and-Compare Mode

The memory transfer compute-and-compare mode performs CRC signature calculation and verification for a region of memory using DMA transactions. The memory region is also copied to another memory region using memory-to-memory DMA transfers. The CRC peripheral is configured such that it operates in burst mode due to the stalling options configured through disabling `CRC_CTL`.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)

- The CRC block is disabled per the `CRC_CTL.BLKEN` register.

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_RESULT_CUR` register.

ADDITIONAL INFORMATION: This register can be initialized to provide an initial seed for the CRC operation that is about to take place.

4. Initialize the `CRC_COMP` register.

ADDITIONAL INFORMATION: This register contains the pre-calculated final CRC signature result for the memory region that the software uses in the final compare operation.

5. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of compare errors and block completion. Configure these interrupts, as needed. If enabled, ensure that the corresponding interrupt handlers are also configured.

6. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory scan compute compare mode and `CRC_CTL.BLKEN` configured to enable the CRC peripheral.
 - a. Disable the `CRC_CTL.OBRSTALL` and the `CRC_CTL.IRRSTALL` bit options for this task example.
 - b. Configure all mirroring and bit reversal options
 - c. Configure CRC auto clear options

The CRC peripheral is now enabled and ready for the core or DMA channel to write data.

7. Configure and enable the memory-to-memory source DMA channel for memory read STOP mode and destination DMA channel for memory write STOP mode.

ADDITIONAL INFORMATION: This step starts the data transfer from one memory region to another through the memory-to-memory DMA channels and the CRC peripheral.

8. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures all the data has been processed.

9. Poll the `CRC_STAT.CMPERR` bit if the interrupt was disabled to check for a compare error.

ADDITIONAL INFORMATION: Perform this step only if the compare error interrupt is not enabled.

10. Write the `CRC_STAT` register to clear both the `CRC_STAT.DCNTEXP` and the `CRC_STAT.CMPERR` bits.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear these status bits within the interrupt handlers for the respective interrupts.

The CRC compute-and-compare operation is now complete. The CRC peripheral is ready to be configured for the next CRC operation. The memory region has also been copied to its new destination.

The integrity check of the memory through the expected CRC signature has completed and the final result is indicated through the `CRC_STAT.CMPERR` bit and the corresponding interrupt when enabled. The memory region has also been copied to its final destination.

Clear any W1C CRC status bits before performing a further CRC operation. Also, clear any W1C status bits of the memory-to-memory source and destination DMA channels before the next CRC operation.

DMA Driven Memory Transfer Data Fill Mode

This mode initializes a region of memory to a constant 32-bit value using DMA transactions.

The task assumes the following:

- The polynomial has been loaded and the look-up table is fully initialized
- All CRC interrupts have been serviced (none pending)
- The CRC block is disabled per the `CRC_CTL.BLKEN` bit

1. Initialize the `CRC_DCNT` register.

ADDITIONAL INFORMATION: The value loaded must represent the number of 32-bit words in the memory region for which the software calculates and verifies the signature.

2. Initialize the `CRC_DCNTRLD` register.

ADDITIONAL INFORMATION: This value is used to reload the `CRC_DCNT` register upon completion of current CRC operation. If no further operation is needed, then this register can be initialized to zero.

3. Initialize the `CRC_FILLVAL` register.

ADDITIONAL INFORMATION: This register contains the 32-bit constant that the CRC module uses to fill the memory region.

4. Initialize the `CRC_INEN` register.

ADDITIONAL INFORMATION: The CRC module uses this register to enable the generation of the CRC interrupts for notification of block completion. Configure these interrupts as required. If enabled, ensure that the corresponding interrupt handlers are also configured.

5. Initialize the `CRC_CTL` register with the `CRC_CTL.OPMODE` bit set to memory transfer fill mode and the `CRC_CTL.BLKEN` bit configured to enable the CRC peripheral.

The CRC peripheral is now enabled and is ready for the DMA channel to write data.

6. Configure and enable the memory-to-memory destination DMA channel for memory write STOP mode.

ADDITIONAL INFORMATION: This step starts the data transfer taking the constant 32-bit value from the CRC peripheral and writing the data to the DMA channel.

7. Poll the `CRC_STAT.DCNTEXP` bit if the interrupt was disabled.

ADDITIONAL INFORMATION: Perform this step only if counter expired interrupt is disabled. Polling ensures that all the data has been processed.

8. Write the `CRC_STAT` register to clear the `CRC_STAT.DCNTEXP` bit.

ADDITIONAL INFORMATION: If interrupts were enabled, then clear this status bit within the interrupt handlers for the respective interrupts.

The CRC memory transfer fill operation is now complete and the CRC peripheral is ready to be configured for the next CRC operation.

The memory region is now filled with the constant data and the CRC peripheral is ready to be configured for a new operation.

Clear any W1C CRC status bits and DMA status bits before performing a further CRC operation.

ADSP-2159x CRC Register Descriptions

Cyclic Redundancy Check Unit (CRC) contains the following registers.

Table 32-4: ADSP-2159x CRC Register List

Name	Description
<code>CRC_COMP</code>	Data Compare Register
<code>CRC_CTL</code>	Control Register
<code>CRC_DCNT</code>	Data Word Count Register
<code>CRC_DCNTCAP</code>	Data Count Capture Register
<code>CRC_DCNTRL</code>	Data Word Count Reload Register
<code>CRC_DFIFO</code>	Data FIFO Register
<code>CRC_FILLVAL</code>	Fill Value Register
<code>CRC_INEN</code>	Interrupt Enable Register
<code>CRC_INEN_CLR</code>	Interrupt Enable Clear Register
<code>CRC_INEN_SET</code>	Interrupt Enable Set Register

Table 32-4: ADSP-2159x CRC Register List (Continued)

Name	Description
CRC_POLY	Polynomial Register
CRC_RESULT_CUR	CRC Current Result Register
CRC_RESULT_FIN	CRC Final Result Register
CRC_STAT	Status Register

Data Compare Register

The `CRC_COMP` register contains the value corresponding to the expected CRC result or signature for the current data stream. At the end of the operation, the content of this register is used to compare against the result produced by the CRC operation. In data verify mode, each incoming data value is compared with the content of this register.

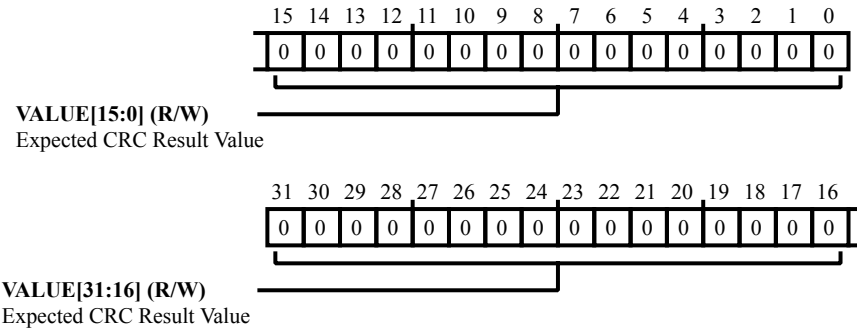


Figure 32-3: CRC_COMP Register Diagram

Table 32-5: CRC_COMP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Expected CRC Result Value. The <code>CRC_COMP.VALUE</code> bit field contains the value corresponding to the expected CRC result or signature for the current data stream.

Control Register

The `CRC_CTL` register configures the operation modes and settings for the CRC.

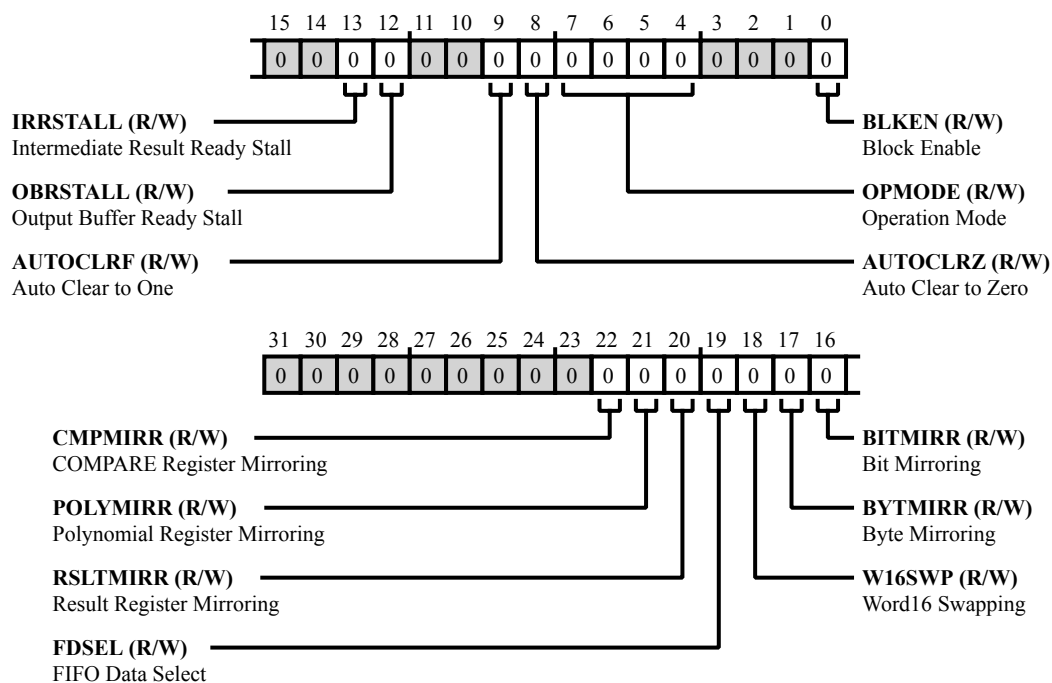


Figure 32-4: CRC_CTL Register Diagram

Table 32-6: CRC_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22 (R/W)	CMPMIRR	COMPARE Register Mirroring. The <code>CRC_CTL.CMPMIRR</code> bit enables data mirroring for the <code>CRC_COMP</code> compare register. When enabled, the 32-bit value in this register is fully bit mirrored (reversed). The bit-reversed value is used for comparison with the <code>CRC_RESULT_FIN</code> register.
		0 Disable compare mirroring
		1 Enable compare mirroring
21 (R/W)	POLYMIRR	Polynomial Register Mirroring. The <code>CRC_CTL.POLYMIRR</code> bit enables data mirroring for the <code>CRC_POLY</code> polynomial register. When enabled, the 32-bit value in this register is fully bit mirrored (reversed). The bit-reversed value is used for CRC computations.
		0 Disable polynomial mirroring
		1 Enable polynomial mirroring

Table 32-6: CRC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/W)	RSLTMIRR	Result Register Mirroring. The <code>CRC_CTL.RSLTMIRR</code> bit enables data mirroring for the <code>CRC_RESULT_CUR</code> and <code>CRC_RESULT_FIN</code> result registers. When enabled, the 32-bit values in these registers are fully bit mirrored (reversed).
		0 Disable result mirroring
		1 Enable result mirroring
19 (R/W)	FDSEL	FIFO Data Select. The <code>CRC_CTL.FDSEL</code> bit selects whether the CRC writes modified or unmodified data to the FIFO in memory transfer mode. If enabled, the data written is affected by the state of the data mirroring selections (<code>CRC_CTL.BITMIRR</code> , <code>CRC_CTL.BYTMIRR</code> , and <code>CRC_CTL.W16SWP</code>) before being written to the FIFO.
		0 Write unmodified data to FIFO
		1 Write modified data to FIFO
18 (R/W)	W16SWP	Word16 Swapping. The <code>CRC_CTL.W16SWP</code> bit enables the CRC's data mirror block to swap the upper and lower 16-bit words within the 32-bit input data, before further processing.
		0 Disable word16 swapping
		1 Enable word16 swapping
17 (R/W)	BYTMIRR	Byte Mirroring. The <code>CRC_CTL.BYTMIRR</code> bit enables the CRC's data mirror block to mirror the bytes within the 32-bit input data, before further processing.
		0 Disable byte mirroring
		1 Enable byte mirroring
16 (R/W)	BITMIRR	Bit Mirroring. The <code>CRC_CTL.BITMIRR</code> bit enables the CRC's data mirror block to mirror the bits within each byte of the 32-bit input data, before further processing.
		0 Disable bit mirroring
		1 Enable bit mirroring
13 (R/W)	IRRSTALL	Intermediate Result Ready Stall. The <code>CRC_CTL.IRRSTALL</code> bit enables stalling the state machine for input data when there is a valid intermediate result to be read in the <code>CRC_RESULT_CUR</code> register. This feature should be used only in CRC computation modes (for example, <code>CRC_CTL.OPMODE = 1</code> or <code>=3</code>).
		0 Do not stall
		1 Stall on IRR

Table 32-6: CRC_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	OBRSTALL	Output Buffer Ready Stall. The CRC_CTL.OBRSTALL bit enables stalling the state machine for input data when there is valid data in the output buffer. This feature should be used only in memory-to-memory transfer modes (for example, CRC_CTL.OPMODE =1).
		0 Do not stall
		1 Stall on OBR
9 (R/W)	AUTOCLRF	Auto Clear to One. The CRC_CTL.AUTOCLRF bit enables auto clear to one when the CRC is in intermediate results ready stall mode (CRC_CTL.IRRSTALL=1) and the CRC data count expires (CRC_DCNT=0). Note that the CRC_CTL.AUTOCLRZ bit must be disabled, or the CRC_CTL.AUTOCLRF bit has no effect.
		0 No auto clear
		1 Auto clear
8 (R/W)	AUTOCLRZ	Auto Clear to Zero. The CRC_CTL.AUTOCLRZ bit enables auto clear to zero when the CRC is in intermediate results ready stall mode (CRC_CTL.IRRSTALL=1) and the CRC data count expires (CRC_DCNT=0). Note that CRC_CTL.AUTOCLRF must be disabled, or the CRC_CTL.AUTOCLRZ has no effect.
		0 No auto clear
		1 Auto clear
7:4 (R/W)	OPMODE	Operation Mode. The CRC_CTL.OPMODE bit field selects the memory transfer or scan mode.
		0 Reserved
		1 CRC compute/compare memory transfer
		2 Data fill memory transfer
		3 CRC compute/compare memory scan
		4 Data verify memory scan
0 (R/W)	BLKEN	Block Enable. The CRC_CTL.BLKEN bit enables and disables the CRC operation.
		0 Disable
		1 Enable

Data Word Count Register

The `CRC_DCNT` register holds the word count that is used for the CRC operation. On transfer of every 32-bit word, the CRC decrements by 1 the content of this register. When the count decrements to zero, this event triggers a CRC compare action, and the `CRC_DCNT` register is automatically loaded from the `CRC_DCNTRLD` register for the next CRC operation.

Note that the initial value programmed into the `CRC_DCNT` register may be different from what is programmed in the `CRC_DCNTRLD` register.

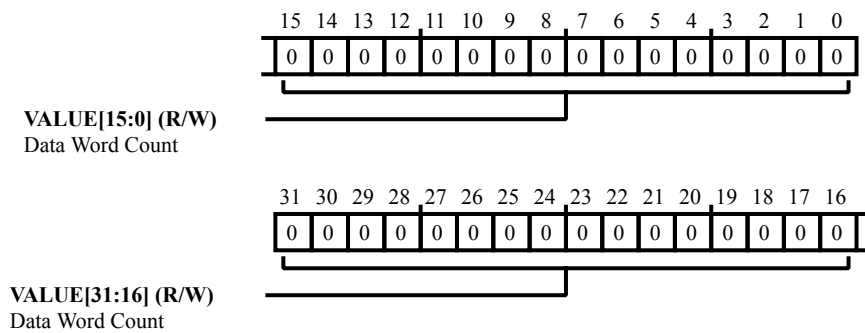


Figure 32-5: CRC_DCNT Register Diagram

Table 32-7: CRC_DCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Data Word Count. The <code>CRC_DCNT.VALUE</code> bit field holds the word count that is used for the CRC operation.

Data Count Capture Register

The `CRC_DCNTCAP` register captures the `CRC_DCNT` value when a compare operation fails in data verify mode. This capture can be used to track the position of an error in the data stream. The capture operation is enabled only if the `CRC_STAT.CMPERR` bit indicates no compare error. After an error occurs and the data count is captured, no further errors are logged until the `CRC_STAT.CMPERR` bit is cleared. To obtain the position of an error in the data stream, subtract the `CRC_DCNTCAP` register value from the initial `CRC_DCNT`.

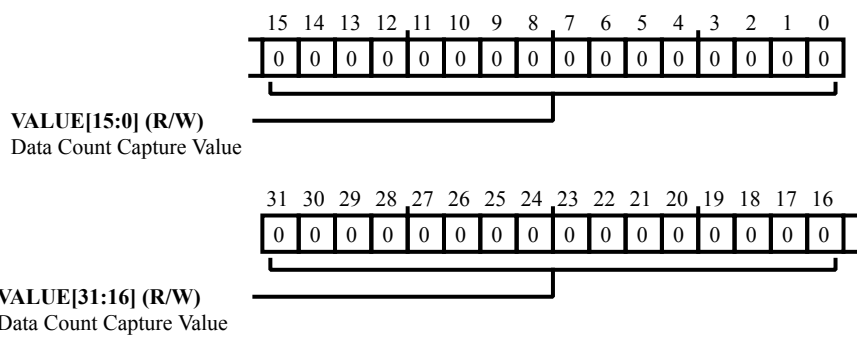


Figure 32-6: `CRC_DCNTCAP` Register Diagram

Table 32-8: `CRC_DCNTCAP` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Data Count Capture Value. The <code>CRC_DCNTCAP.VALUE</code> bit field contains the <code>CRC_DCNT</code> value when a compare operation fails in data verify mode.

Data Word Count Reload Register

The `CRC_DCNTRLD` register holds the value that the CRC automatically loads into `CRC_DCNT` when the `CRC_DCNT` decrements to 0. At startup, the value programmed in `CRC_DCNT` and the `CRC_DCNTRLD` register could be different. So, for the first iteration, the CRC operation happens for the count initially programmed in the `CRC_DCNT` register. While for subsequent CRC operations, the count is taken from the `CRC_DCNTRLD` register.

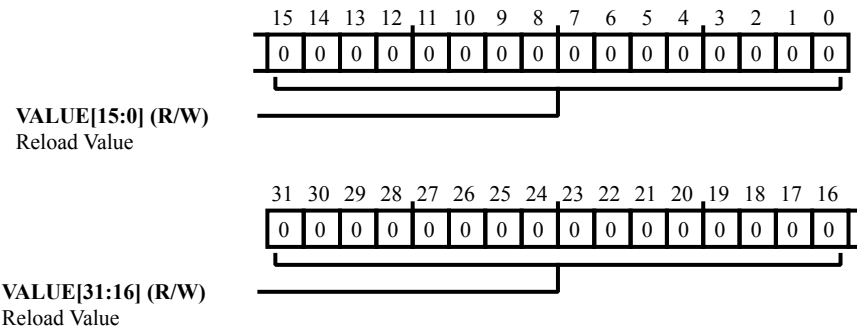


Figure 32-7: `CRC_DCNTRLD` Register Diagram

Table 32-9: `CRC_DCNTRLD` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Reload Value. The <code>CRC_DCNTRLD.VALUE</code> bit field holds the value that automatically loads into <code>CRC_DCNT</code> when the <code>CRC_DCNT</code> decrements to 0.

Data FIFO Register

In memory transfer mode (non-data fill mode), the data from the DMA or processor core buses is written into the `CRC_DFIFO` on each input data grant (DMA grant or core write). Data is read from this FIFO on each output data grant (DMA grant or core read). FIFO status information is available in the `CRC_STAT` register. Whenever, the FIFO has valid data, output data requests are generated.

Note that in non-memory transfer mode and in data fill mode, the input data does not get written into this FIFO. So, this register should not be read in these modes.

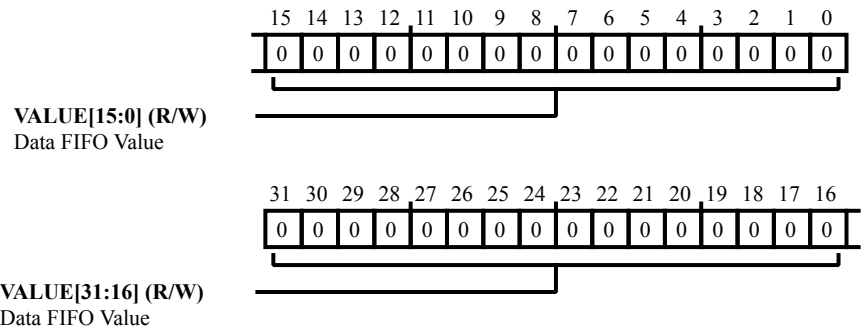


Figure 32-8: CRC_DFIFO Register Diagram

Table 32-10: CRC_DFIFO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Data FIFO Value. The <code>CRC_DFIFO.VALUE</code> bit field is the data from the DMA or processor core buses.

Fill Value Register

The `CRC_FILLVAL` register holds the value that the CRC uses for the memory fill operation. In data fill mode, the value programmed in this register is used for the memory fill operation.

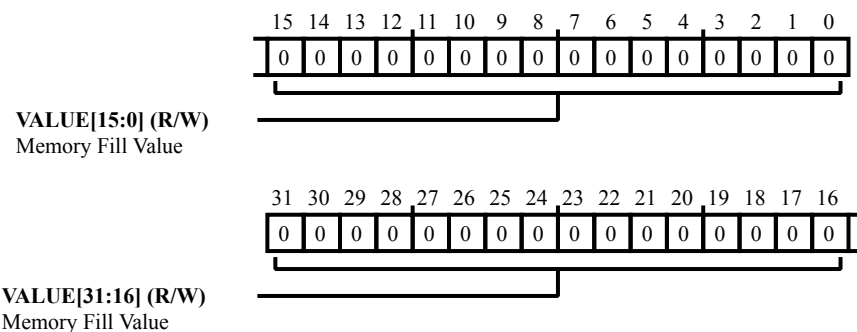


Figure 32-9: CRC_FILLVAL Register Diagram

Table 32-11: CRC_FILLVAL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Memory Fill Value. The <code>CRC_FILLVAL.VALUE</code> bit field holds the value that the CRC uses for the memory fill operation.

Interrupt Enable Register

The `CRC_INEN` register unmask (enables) or mask (disables) interrupt requests generated in the CRC from going to the processor core.

Note that CRC interrupts are not disabled when the CRC is disabled (`CRC_CTL.BLKEN = 0`).

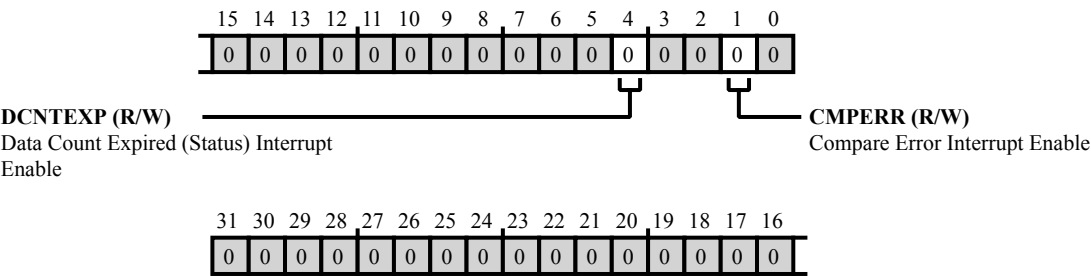


Figure 32-10: CRC_INEN Register Diagram

Table 32-12: CRC_INEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	DCNTEXP	Data Count Expired (Status) Interrupt Enable. The <code>CRC_INEN.DCNTEXP</code> enables (unmasks) the data count expired (CRC status) interrupt.
		0 Disable (mask) interrupt
		1 Enable (unmask) interrupt
1 (R/W)	CMPERR	Compare Error Interrupt Enable. The <code>CRC_INEN.CMPERR</code> enables (unmasks) the data compare interrupt, which is generated when CRC data comparison fails.
		0 Disable (mask) interrupt
		1 Enable (unmask) interrupt

Interrupt Enable Clear Register

The `CRC_INEN_CLR` register permits clearing individual bits in the `CRC_INEN` register without affecting other bits in the register.

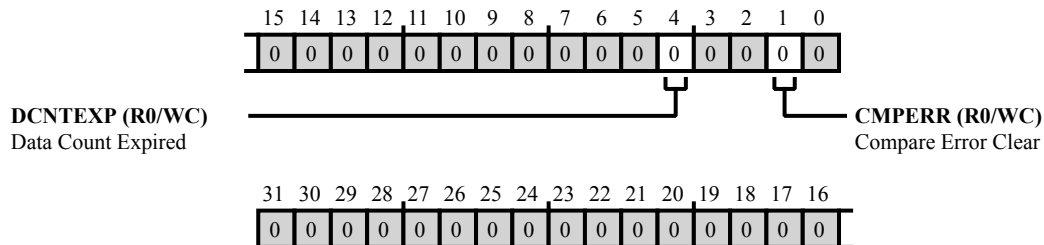


Figure 32-11: `CRC_INEN_CLR` Register Diagram

Table 32-13: `CRC_INEN_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R0/WC)	DCNTEXP	Data Count Expired. The <code>CRC_INEN_CLR.DCNTEXP</code> bit clears the data count expired (status) interrupt.
		0 No effect
		1 Clear bit
1 (R0/WC)	CMPERR	Compare Error Clear. The <code>CRC_INEN_CLR.CMPERR</code> bit clears the compare error interrupt.
		0 No effect
		1 Clear bit

Interrupt Enable Set Register

The `CRC_INEN_SET` register permits setting individual bits in the `CRC_INEN` register without affecting other bits in the register.

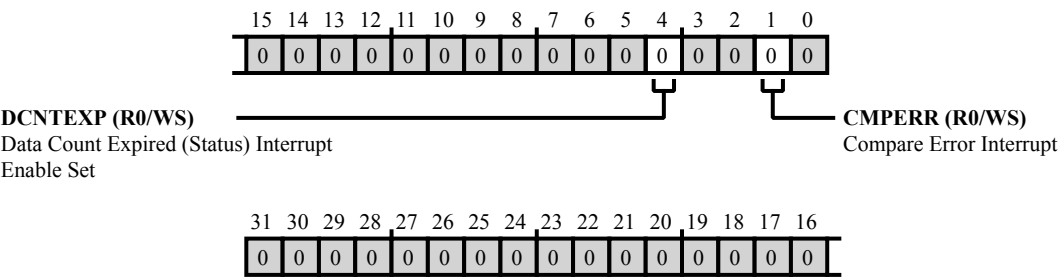


Figure 32-12: `CRC_INEN_SET` Register Diagram

Table 32-14: `CRC_INEN_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R0/WS)	DCNTEXP	Data Count Expired (Status) Interrupt Enable Set. The <code>CRC_INEN_SET.DCNTEXP</code> bit sets the data count expired (status) interrupt.
		0 No effect
		1 Set bit
1 (R0/WS)	CMPERR	Compare Error Interrupt. The <code>CRC_INEN_SET.CMPERR</code> bit sets the compare error interrupt.
		0 No effect
		1 Set bit

Polynomial Register

The `CRC_POLY` register holds a 32-bit polynomial for CRC operations. Bit 31 corresponds to the coefficient of x^{31} of the CRC polynomial, bit 30 corresponds to the coefficient of x^{30} , and so on through bit 0. A coefficient of x^{32} is assumed to be "1" for any polynomial that is selected. Based on the polynomial in the `CRC_POLY` register, the CRC generates a look-up table (LUT), which is used to compute the CRC of the incoming data stream.

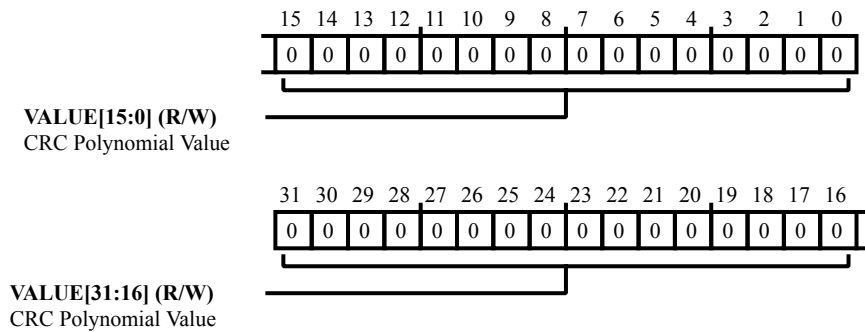


Figure 32-13: CRC_POLY Register Diagram

Table 32-15: CRC_POLY Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	CRC Polynomial Value. The <code>CRC_POLY.VALUE</code> bit field holds the 32-bit polynomial for CRC operations.

CRC Current Result Register

The `CRC_RESULT_CUR` register holds the current or intermediate CRC result. It is updated when new data is written into the CRC. Each time the `CRC_DCNT` expires, the CRC loads the value from this register into the `CRC_RESULT_FIN` register. The `CRC_RESULT_CUR` register may be set to auto clear to zero or auto clear to ones when `CRC_DCNT` expires by configuring the `CRC_CTL.AUTOCLRZ` and `CRC_CTL.AUTOCLR1` bits. Before starting a CRC operation, the `CRC_RESULT_CUR` register should be programmed to the desired value.

Note that this register can be read by the processor core at any time.

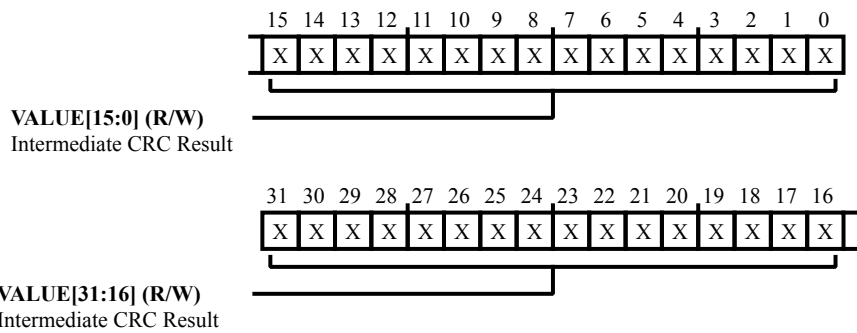


Figure 32-14: `CRC_RESULT_CUR` Register Diagram

Table 32-16: `CRC_RESULT_CUR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Intermediate CRC Result. The <code>CRC_RESULT_CUR.VALUE</code> bit field holds the current or intermediate CRC result.

CRC Final Result Register

The `CRC_RESULT_FIN` register holds the final CRC computed for a data stream. A data stream is a DMA of `CRC_DCNT` number of words into the CRC. When `CRC_DCNT` decrements to zero for each data stream, the CRC loads the `CRC_RESULT_FIN` register with the value from the `CRC_RESULT_CUR` register.

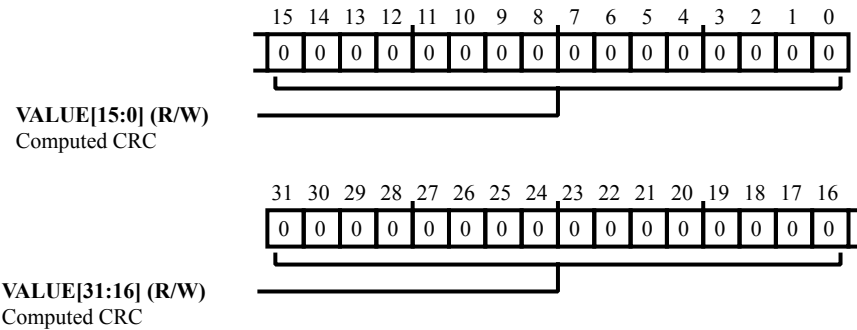


Figure 32-15: `CRC_RESULT_FIN` Register Diagram

Table 32-17: `CRC_RESULT_FIN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Computed CRC. The <code>CRC_RESULT_FIN.VALUE</code> bit field holds the final CRC computed for a data stream.

Status Register

The `CRC_STAT` register indicates the status for CRC operations and interrupt generation.

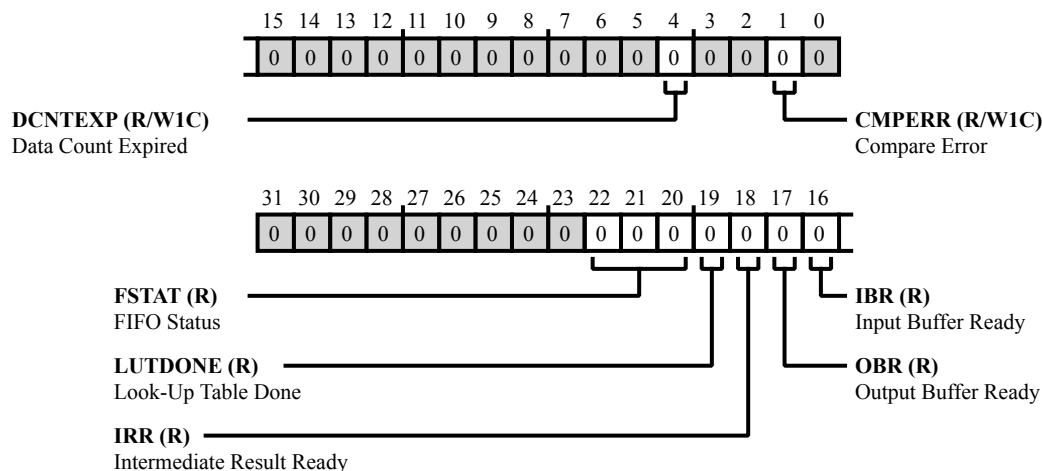


Figure 32-16: CRC_STAT Register Diagram

Table 32-18: CRC_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
22:20 (R/NW)	FSTAT	FIFO Status. The <code>CRC_STAT.FSTAT</code> indicates the current FIFO status. This field is read-only.
		0 FIFO empty
		1 FIFO has 1 data
		2 FIFO has 2 data
		3 FIFO has 3 data
		4 FIFO has 4 data (full)
19 (R/NW)	LUTDONE	Look-Up Table Done. The <code>CRC_STAT.LUTDONE</code> bit indicates that the CRC has generated the look-up table for the current polynomial. This read-only bit is cleared at reset and cleared when the <code>CRC_POLY</code> is written.
		0 No status
		1 LUT generation done

Table 32-18: CRC_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/NW)	IRR	Intermediate Result Ready. The CRC_STAT . IRR bit indicates that the CRC has updated the CRC_RESULT_CUR register with intermediate CRC results for the new data written to the CRC. The processor core should read from the CRC_RESULT_CUR register only after detecting CRC_STAT . IRR =1. This read-only bit is cleared by CRC hardware and is valid when the CRC_CTL . IRRSTALL bit is enabled.
		0 No status
		1 Intermediate results ready
17 (R/NW)	OBR	Output Buffer Ready. The CRC_STAT . OBR bit indicates that the CRC has data ready for the processor core to read. The processor core should read from the CRC only after detecting CRC_STAT . OBR =1. This read-only bit is cleared by CRC hardware.
		0 No status
		1 Output buffer ready
16 (R/NW)	IBR	Input Buffer Ready. The CRC_STAT . IBR bit indicates that the CRC is ready to accept a processor core write. The processor core should write to the input register only after detecting that CRC_STAT . IBR =1. This read-only bit is cleared by CRC hardware.
		0 No status
		1 Input buffer ready
4 (R/W1C)	DCNTEXP	Data Count Expired. The CRC_STAT . DCNTEXP bit indicates that the CRC_DCNT has expired. This W1C bit is not automatically cleared when the CRC is disabled (CRC_CTL . BLKEN =0). When the CRC sets this bit on CRC_DCNT expiry, the CRC generates the CRC_INEN . DCNTEXP interrupt.
		0 No status
		1 Data counter expired
1 (R/W1C)	CMPERR	Compare Error. The CRC_STAT . CMPERR bit indicates that a CRC mismatch or data mismatch has been detected. This W1C bit is not automatically cleared when the CRC is disabled (CRC_CTL . BLKEN =0). When the CRC sets this bit on detecting a mismatch, the CRC generates the CRC_INEN . CMPERR interrupt. While this bit is set, the CRC_DCNTCAP register is disabled from capturing the data count values.
		0 No status
		1 Compare error

33 System Security

The requirement to protect content, keys, IP and other sensitive information have become increasingly prevalent. The processor contains several modules and system elements that contribute to creating a secure operating environment for trusted code to execute.

The modules and system elements that can be used are:

- Boot Kernel
- Secure Core
- System Protection Unit (SPU)
- System Memory Protection Unit (SMPU)
- DEBUG through the Test Access Port Controller (TAPC)
- One-Time Programmable (OTP) memory
- Cryptographic Accelerators (optionally)

NOTE: This product includes security features that can be used to protect embedded non-volatile memory contents and prevent execution of non-authorized code. When security is enabled on this device (either by the ordering party or the subsequent receiving parties), Analog Devices' ability to conduct Failure Analysis on returned devices will be limited. Contact Analog Devices, Inc. for details on the Failure Analysis limitations for this device.

Security Features

The security infrastructure in the system provides the following features:

- Secure operating environment for secure code execution
- Protect sensitive IP from theft by malicious users or competitors
- Protect sensitive data (for example cipher keys)
- Allow debugging while still maintaining security

Security Functional Description

In order to provide secure operating environment in a system, it requires the involvement of multiple elements.

Boot Kernel

The Boot Kernel is the root of trust for a secure system. Since the boot kernel is developed by Analog Devices, Inc and is stored in ROM and can't be changed, it can be trusted. The boot kernel validates the authenticity of the application binary image that needs to be booted in. It also handles decrypting the binary image if it's encrypted.

Verifying the authenticity of the application binary asserts that

1. The image is not tampered with or altered
2. The image came from a trusted developer

If the image is encrypted, it ensures confidentiality since the boot image is stored on an external storage device and can be more easily read or stolen than in the part.

Once the boot kernel can verify and optionally decrypt the boot image, it can be loaded into the processor for execution. At this point, the *chain of trust* is continued with the verified application.

Secure Booting

Secure booting is when the boot kernel uses cryptographic algorithms to perform checks on the application binary and to decrypt it. When security is enabled, signature verification and optional decryption on the application binary are both performed by secure boot. See [Security Mode Configuration](#).

Secure Core

In a system with multiple master and slave resources, not everything is considered secure. There are secure and non-secure peripherals and secure and non-secure segments of memory.

For a system to be considered secure, a secure core that can access and execute instructions (verified application) from secure memory must be configured. Typically, in a single core processor, either the core is hardwired to be secure or the core can switch between secure and non-secure modes.

System Protection Unit (SPU)

The SPU in the system serves two functions. First, it acts as a gatekeeper, guarding against non-secure accesses to secure resources (peripherals). Second, it is used to define which resources in the system are secure or non-secure masters and which resources in the system are secure or non-secure slaves.

NOTE: Though the SPU can be configured by secure or non-secure master, the first steps that the verified secure application must perform are:

1. Configure the SPU as a secure slave itself so only other secure masters can configure it
2. Define and configure the secure masters and slaves using the SPU

This way, once the SPU is secure and other secure masters and slaves are configured, non-secure masters cannot tamper with the security privileges of secure masters and slaves nor can non-secure resources be changed to a secure resource.

System Memory Protection Unit (SMPU)

Similar to the SPU protecting the MMR address range for peripherals, the SMPU can guard memory ranges or pages. Memory pages can be configured as secure or non-secure. Again, like in the case of the SPU, the SMPU can guard against non-secure transaction attempts to secure memory.

NOTE: A verified application should reside in memory configured as secure. By default the SMPU has all memory configured as secure. If the program needs to update the memory protections via the SMPU, the application and its data should remain in secure memory.

Debug

The typical way of accessing a system is through the debug port via a JTAG or serial wire interface. If these access points are not secure sensitive IP such as cipher keys can be exposed and code can be changed to disable other security settings. To guard against this type of attack or security hole and still provide debugging capabilities for a developer, a debug unit with security features is used.

When the developer first receives the part, they can define a JTAG/DEBUG key that is programmed into OTP memory. Once security is enabled, the debug unit compares the key sent from the host debugger with the key inside the system. If a match occurs, debug access to secure resources are allowed. There is also a provision to bypass the loading of the JTAG/DEBUG key from the OTP memory. See [Boot ROM and Booting the Processor](#)

One-Time-Programmable (OTP) Memory

Customer programmable OTP memory is used to safely and securely store sensitive information such as cipher keys.

In a public key algorithm (for example, ECDSA which is used in secure boot), the public key is used to verify the digital signature that accompanies the application binary. The private key is used by the developer on the host development machine to create the digital signature. The public and private key pair is unique and the public key needs to be stored in non-volatile, one-time-programmable memory. If the public key is allowed to be changed then users can generate their own public/private key pairs and successfully boot in malicious code. This can either re-purpose the part or change security configurations to allow easier access to sensitive information stored elsewhere in OTP memory.

When security is not enabled, OTP can be accessed freely. When a user decides to set the LOCK bit in OTP memory to enable security, portions of the OTP, specifically the first 6K bits and another 1K bits for the customer boot information are will also be locked. From then on, only secure masters will be allowed to access those memory regions.

Cryptographic Accelerators

Cryptographic algorithms are mathematical tools to help provide security. Hardware engines provide some advantages but are not necessarily required. For computationally expensive operations like those used in Elliptic Curve Cryptography, like ECDSA used in secure boot, the operations can be accelerated while the core performs other tasks. Also, it's less likely that the hardware engine can be hacked to change the results.

Security Mode Configuration

Security as a feature does not necessarily be employed by the user. There are no steps to disable security, if security protection is not required. The part does not have security enabled.

To use the security features, perform the following steps:

- Generate the public/private key pair on the host development machine.¹
- Program OTP memory with the public key.²
- Program OTP memory with the decryption key if the application binary needs to be encrypted.²
- Program OTP memory with the debug/JTAG key.²
- Develop the application and sign it, creating the digital signature with the private key.¹
- If confidentiality is required, encrypt the application binary before signing it.
- Set the LOCK bit in OTP memory to enable security. After this, subsequent boots are secure boots.^{2, 3}

¹ Software tools are provided with developments tools to generate keys, sign boot streams and also perform encryption. Refer to the development tools manuals for information on usage.

²Refer to the OTP Chapter for programming the OTP and other related information.

³Refer to the Booting Chapter for more information on Secure Booting and other related information.

Status and Error Signals

In a fully functional secure system, non-secure resources should not even attempt to access a secure resource. If this does occur, then either the code has been altered or replaced with malicious code or the system contains a bug.

Errors or error events are dependant on the configuration of the SPU and SMPU, the protection units which guard against security violations. In the case of the SMPU, the error can simply be captured, captured and interrupt generated, or the access prevented without capturing any error. It is the developer's responsibility to:

1. Determine if there was an error due to a blocked access.
2. Determine how to handle a blocked access (for example fix the bug (offline), or try to use a different resource (run time)).

34 System Protection Unit (SPU)

In a system with multiple system MMR masters, configurations of peripherals can be changed unintentionally leading to bad data or even system malfunctions. The peripherals are shared resources in the system. The SPU restricts access to certain MMRs, similar to the functionality of a semaphore.

The SPU also protects peripherals based on security settings. It is part of the overall security infrastructure of the processor.

SPU Features

The SPU has the following features:

- Write-protect system MMR from certain system masters and core masters.
- Simultaneously lock multiple peripheral configuration registers through a global lock mechanism.
- Write-protect and block access to its own write-protection registers from other system masters.
- Defined security privileges to peripherals and system resources.
- Security protection to guard secure peripheral MMRs against non-secure accesses.

SPU Functional Description

The following sections provide information on the function of the SPU.

ADSP-2159x SPU Register List

The System Protection Unit (SPU) provides a set of registers that can protect system resources from errant writes. The protection categories are global lock (protects configuration registers) and write protect register lock (protects the write protect register). For more information on SPU functionality, see the SPU register descriptions.

Table 34-1: ADSP-2159x SPU Register List

Name	Description
SPU_CTL	Control Register
SPU_SECURECHK	Secure Check Register

Table 34-1: ADSP-2159x SPU Register List (Continued)

Name	Description
SPU_SECURECTL	Secure Control Register
SPU_SECUREC[n]	Secure Core Registers
SPU_SECUREP[n]	Secure Peripheral Register
SPU_STAT	Status Register
SPU_WP[n]	Write Protect Register n

ADSP-2159x SPU Interrupt List

Table 34-2: ADSP-2159x SPU Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
216	SPU0_INT	SPU0 Interrupt	Level	

Peripheral Register Write Protection

The SPU has a write-protection register ([SPU_WP\[n\]](#)) associated with each peripheral. Each of these write-protection registers has the exact same bits that correspond to a particular SMMR master (for example, Core 0, MDMA). When the bits are set, the SPU locks the corresponding SMMR masters from accessing the register address space of the associated peripheral. The bits in the register can be cleared to allow access to the registers of the peripheral again. When the SPU initiates the write-protection register, any writes that are in-progress complete before the SPU blocks subsequent writes.

In the *SPU Write Protect Registers* figure, each write-protect register in the SPU is associated with a particular peripheral.

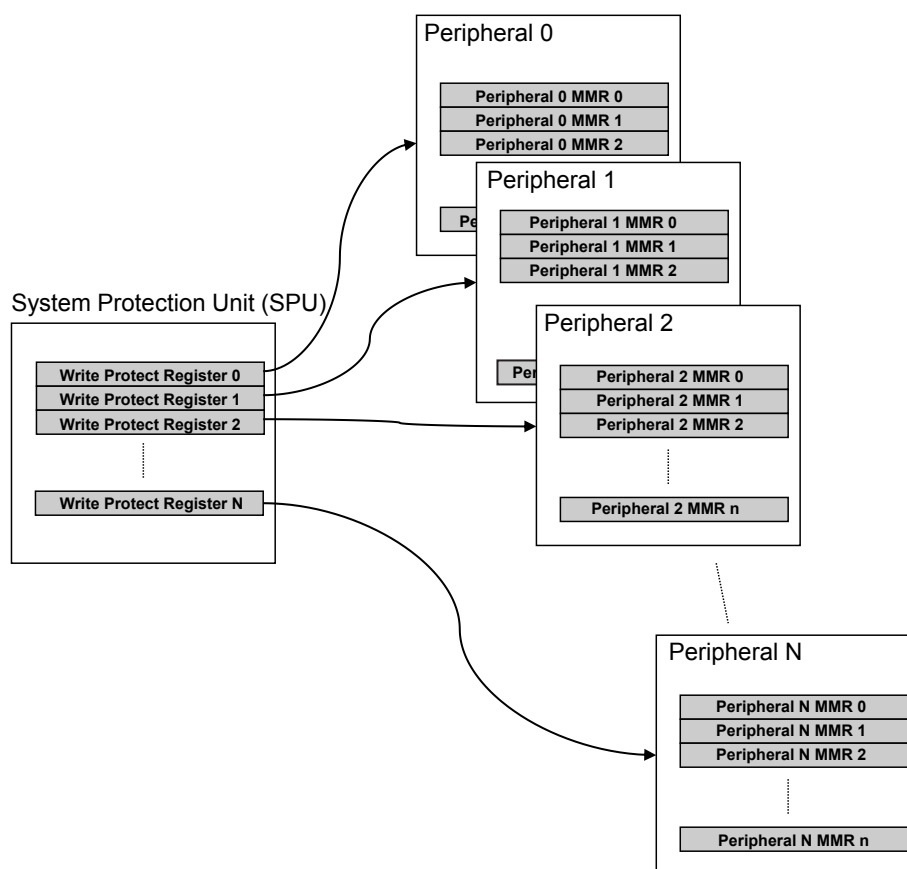


Figure 34-1: SPU Write Protect Registers

In the figure, a write-protect register in the SPU module blocks write-attempts to the MMR space of the associated peripheral. The bits in the write-protect register specify from which masters to block write-access.

NOTE: A SPU write protection register (`SPU_WP[n]`) exists for the SPU alone. If all defined bits are set in this register for the SPU, any configurations in the SPU are locked and cannot be changed. Only a system reset can restore access to the SPU.

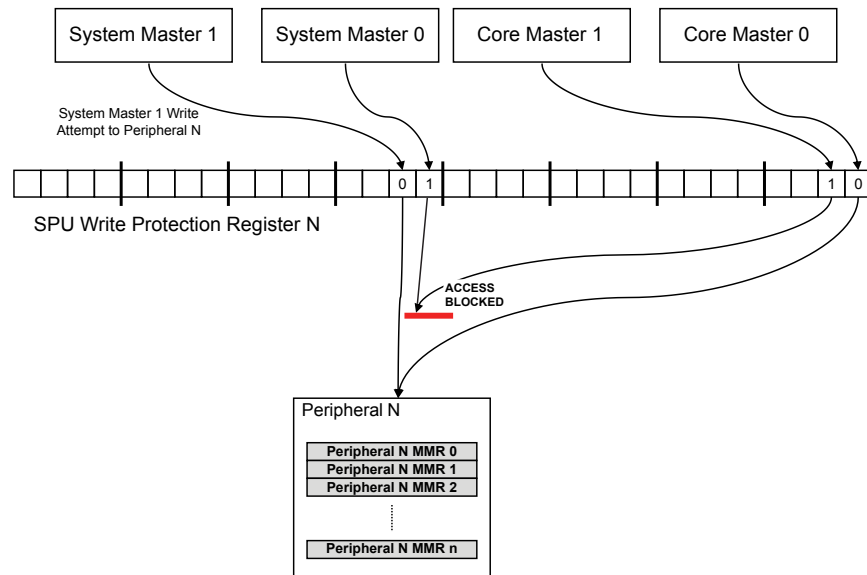


Figure 34-2: SPU Write-Protect Register Blocking Access from System Master 0 and Core Master 1

Global Locking

The SPU also has global locking capability. When enabled by setting `SPU_CTL.GLCK` bit field to a value other than `0xAD`, a system-wide global lock signal is active. Some peripherals have a lock enable bit in their control register. When this bit is set, the peripheral recognizes the global lock signal and blocks further write-accesses to its own control register. Access to the configuration register of the peripheral is enabled when the global lock is turned off in the SPU.

The *Global Locking* figure is a conceptual diagram. The diagram shows how the SPU module (or any peripheral) blocks any write attempts to its control register when:

- The global lock signal from the SPU is active, and
- The global lock enable bit is set in the control register of the peripheral

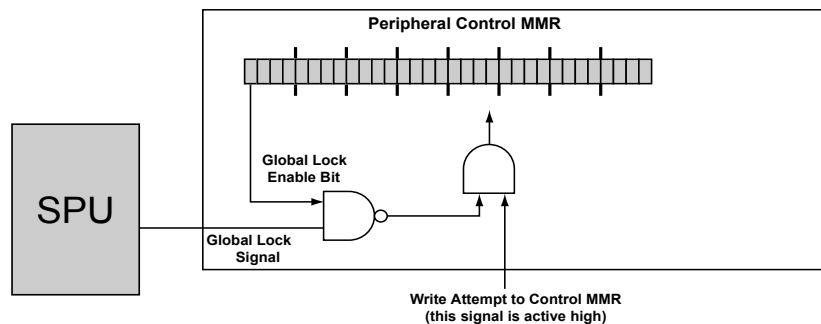


Figure 34-3: Global Locking

The SPU can write-protect its own registers. When the `SPU_CTL.WPLCK` bit is set and global locking is enabled, the SPU blocks accesses to the SPU write-protection registers. To enable write access to the write-protection registers in the SPU, disable the global locking.

SPU Block Diagram

The *SPU System-Level Block Diagram* shows a system-level block diagram of where the SPU is located in the system. It resides between the SMMR interface and the system crossbar. Depending on the configuration of the SPU write-protect registers, it can block access to some peripherals from certain SMMR masters.

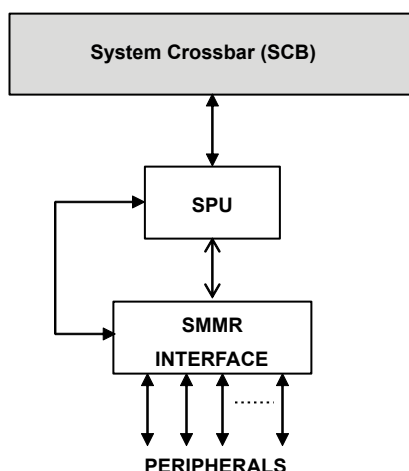


Figure 34-4: SPU System-Level Block Diagram

SPU Architectural Concepts

As shown in the block diagram, the SPU sits between the system crossbar (SCB) and the SMMR interface to the peripherals. The SPU gates any MMR access to any peripheral from any master that comes through the SCB. Depending on the configuration of the write-protection registers in the SPU, the SPU does or does not allow the MMR write to go through.

The SPU also checks whether the transaction is a secure or non-secure transaction and blocks it according to the configured security setting for the target destination. A secure master can generate secure read or secure write transactions which can access secure or non-secure slaves. A non-secure master can generate non-secure read or non-secure write transactions and can only access non-secure slaves.

SPU Event Control

The system protection unit provides write-protection against MMRs peripherals and its own write-protect registers. If a write attempt is made to any locked MMR peripheral the SPU has write-protected, it blocks the write. The SPU generates a bus error to the master that attempted the write. That master does or does not generate an event, based on the returned error.

The SPU can be configured to generate an interrupt for the write-protection violation by setting the `SPU_CTL.PINTEN` bit. The SPU can also be configured to generate an interrupt for a security violation by setting the `SPU_SECURECTL.SINTEN` bit. If either one or both bits is triggered, the `SPU_STAT.VIRQ` bit is set.

The SPU can also lock its own registers from write attempts. If a write-attempt is made to a locked register in the SPU, the SPU blocks it and records it as an error in the `SPU_STAT.LWERR` bit. Again, the SPU generates a bus error to the master that attempted the write.

The master does or does not generate an event, based on the returned error.

The SPU does not generate an event for a blocked write access to an SPU register. If the `SPU_CTL.PINTEN` bit is set, the SPU triggers an interrupt for this blocked access attempt.

The global lock is enabled by setting the `SPU_CTL.GLCK` bit to something other than 0xAD. If the lock bit is set in that same configuration register, a peripheral can block write access to its configuration register. When the SPU blocks a write attempt, the peripheral logs and reports the failed attempt. The SPU is unaware and therefore does not provide any indication of a failed write attempt to the configuration register of the peripheral.

SPU Programming Model

The system protection unit (SPU) consists of write-protect and access-protect registers. Each one corresponds to a different peripheral instance. Bits in the write-protect registers correspond to system masters that can modify the MMR contents of the peripherals. By writing to these write-protect registers, the corresponding memory-mapped registers of the peripheral are write-protected against masters whose bits in the write-protect register are set.

The SPU globally locks the control register of the peripheral. Peripherals that support this feature have a lock enable bit in their control register. The peripheral blocks any additional write attempts to its control register from any master when:

- The global lock signal is active from the SPU, and
- The lock enable bit of the peripheral is set

If the lock enable bit of a peripheral is not set and the global lock signal is active, access to that control register of the peripheral is still allowed. To grant access again, disable the global lock signal from the SPU by writing the value 0xAD into the `SPU_CTL.GLCK` bit field.

Another protection mechanism that the SPU offers is write-protection against the write-protection registers. If the write protect register lock bit (`SPU_CTL.WPLCK`) is set and the global lock signal is active, writes to the write-protect registers of the SPU are blocked. To reenabling access to the write-protect registers in the SPU, deactivate the global lock signal by writing 0xAD into the `SPU_CTL.GLCK` bit field.

For security, the SPU provides a set of `SPU_SECUREC[n]` registers (one for each processor core from Analog Devices) to configure their security settings. The SPU also provides a set of `SPU_SECUREP[n]` registers (one for each peripheral instance) to configure their security settings.

Enabling and Disabling the SPU

The SPU is always operating. There are no bits to enable or disable the SPU. The SPU configuration can be updated at any time. Any ongoing transactions finish before a new configuration is in effect. By default, the SPU does not write-protect any of the MMRs.

Write-Protecting the SPU

The SPU is treated like any other peripheral in the system. As such, the SPU also has an associated write-protection register. If this write-protection register is configured to block all writes from all masters, any SPU configuration remains the same until the next system reset.

Checking the Security State

In some cases while running a peripheral, an application system master does not know whether they are a secure master generating secure transactions or not. The SPU provides a means for checking the security state of the master through the `SPU_SECURECHK` register. When read by a secure master, the register reads 0xFFFFFFFF and when read by a non-secure master, the value is 0x00000000.

SPU Mode Configuration

The SPU can provide address range-wide protection by write-protecting the peripherals MMR address range from system MMR masters. It can also provide register wide protection using global locking. Peripherals that support this feature can enable it in their respective configuration register. When the SPU enables the global lock signal, all subsequent writes to the configuration register of the peripheral are blocked until the global lock signal is deasserted. Similarly, the write-protection registers of the SPU can be write-protected using the global lock signal as well. The SPU uses all these modes of operation together.

Locking Write-Protect Registers

Use the following steps to lock (write-protect) a register.

1. Set the `SPU_CTL.WPLCK` bit and configure the `SPU_CTL.GLCK` field to something other than 0xAD.

The SPU write-protect registers are blocked from further write accesses.

Protecting a Peripheral

Use the following procedure to protect a peripheral.

1. Determine which peripheral needs protection and locate the corresponding write-protect register (`SPU_WP[n]`) in the SPU. See the "Write-Protect and Secure Peripheral Registers" section.
2. Determine the SMMR masters from which the peripheral needs protection. Then, set the corresponding bit or bits in the write-protect register (`SPU_WP[n]`) for the peripheral. See the "Write-Protect and Secure Peripheral Registers" section.

After setting the write-protect register for the particular peripheral, the identified SMMR masters are blocked from writing to any MMR in the address space of the peripheral. This block remains in place until the bits in the write-protect register are cleared.

Configuring Security Privileges of a Peripheral

Use the following procedure to configure the security privileges of a peripheral.

- 1. Determine the peripheral and its corresponding secure peripheral register (`SPU_SECUREP[n]`) in the SPU. See the "Write-Protect and Secure Peripheral Registers" section.
- 2. If the peripheral is to be a secure slave only accepting secure transactions, set bit 0 (`SPU_SECUREP[n].SSEC`).
- 3. If the peripheral is to be a secure master that generates secure transactions (keeping in mind not all peripherals can be masters), set bit 1 (`SPU_SECUREP[n].MSEC`).

This procedure sets the security privileges of a peripheral.

NOTE: Only a secure master can set security privileges, keeping the chain of trust intact. If a non-secure master configures the security privileges, it can undermine security protection.

ADSP-2159x SPU Register Descriptions

System Protection Unit (SPU) contains the following registers.

Table 34-3: ADSP-2159x SPU Register List

Name	Description
<code>SPU_CTL</code>	Control Register
<code>SPU_SECURECHK</code>	Secure Check Register
<code>SPU_SECURECTL</code>	Secure Control Register
<code>SPU_SECUREC[n]</code>	Secure Core Registers
<code>SPU_SECUREP[n]</code>	Secure Peripheral Register
<code>SPU_STAT</code>	Status Register
<code>SPU_WP[n]</code>	Write Protect Register n

Control Register

The SPU control register (`SPU_CTL`) provides a global lock for configuration registers as well as control for locking the write protect (`SPU_WP[n]`) registers. It also controls the generation of an interrupt to report blocked accesses.

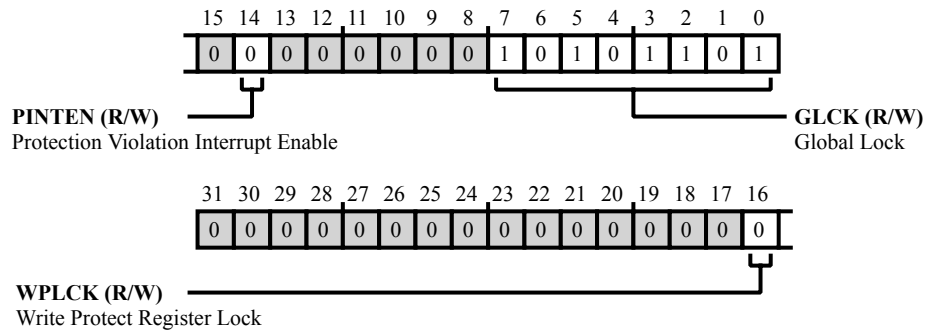


Figure 34-5: SPU_CTL Register Diagram

Table 34-4: SPU_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	WPLCK	Write Protect Register Lock.
		When the <code>SPU_CTL.WPLCK</code> bit is set in combination with the <code>SPU_CTL.GLCK</code> bit, writes to the SPU's write protect registers are blocked and return an error.
		0 Disable 1 Enable
14 (R/W)	PINTEN	Protection Violation Interrupt Enable.
		When the <code>SPU_CTL.PINTEN</code> bit is set (=1), a block of any transaction according to the configured settings produces an interrupt.
		0 Disable 1 Enable
7:0 (R/W)	GLCK	Global Lock. The <code>SPU_CTL.GLCK</code> controls the global lock signal. The global lock signal provides register-based write protection. Writing 0xAD to this field disables the lock, and writing any other value enables the lock.

Secure Check Register

The `SPU_SECURECHK` register reads by secure masters return 0xFFFFFFFF. Reads by non-secure masters return 0x00000000.

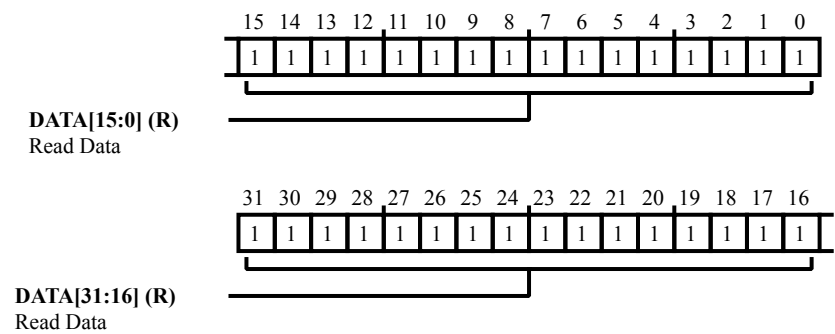


Figure 34-6: SPU_SECURECHK Register Diagram

Table 34-5: SPU_SECURECHK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	DATA	Read Data. The <code>SPU_SECURECHK</code> . <code>DATA</code> bit field performs reads. Reads by secure masters return 0xFFFFFFFF. Reads by non-secure masters return 0x00000000.

Secure Control Register

The SPU Secure Control Register (`SPU_SECURECTL`) allows the user to lock write access to all the `SPU_SECUREC[n]` and `SPU_SECUREP[n]` registers as well as configure the interrupt generation in an event of a security error. It also allows bulk clear of the SSEC bits and/or MSEC bits in the `SPU_SECUREP[n]` registers.

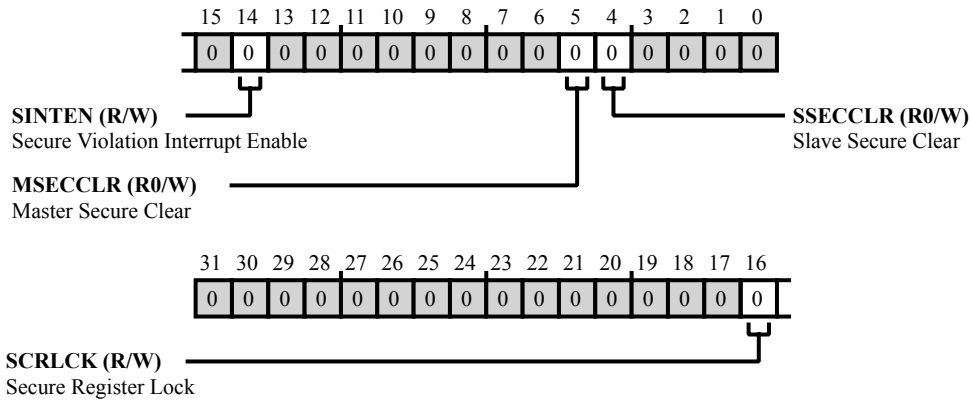


Figure 34-7: SPU_SECURECTL Register Diagram

Table 34-6: SPU_SECURECTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	SCRLCK	Secure Register Lock. When the <code>SPU_SECURECTL.SCRLCK</code> bit is set in combination with the <code>SPU_CTL.GLCK</code> bit, writes to the Security Configuration registers (<code>SPU_SECUREC[n]</code> and <code>SPU_SECUREP[n]</code>) are blocked and return an error which is captured in the <code>SPU_STAT.LWERR</code> bit.
		0 Disable
		1 Enable
14 (R/W)	SINTEN	Secure Violation Interrupt Enable. The <code>SPU_SECURECTL.SINTEN</code> bit generates an interrupt if a security violation was captured. Interrupt status is provided in the <code>SPU_STAT.VIRQ</code> bit.
		0 Disable
		1 Enable
5 (R0/W)	MSECCLR	Master Secure Clear. When the <code>SPU_SECURECTL.MSECCLR</code> bit is set, the <code>SPU_SECUREP[n].MSEC</code> bits in all <code>SPU_SECUREP[n]</code> registers are cleared. The <code>SPU_SECURECTL.MSECCLR</code> bit always reads back as a 0.
		0 No Action
		1 Clear All Master Secure Control Bits

Table 34-6: SPU_SECURECTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R0/W)	SSECCLR	Slave Secure Clear. When the SPU_SECURECTL.SSECCLR bit is set, the SPU_SECUREP[n].SSEC bits in all SPU_SECUREP[n] registers are cleared. The SPU_SECURECTL.SSECCLR bit always reads back as a 0.
		0 No Action
		1 Clear All Slave Secure Control Bits

Secure Core Registers

A SPU register exists for every DSP core in the system. The bits enable or disable security for features in the core.

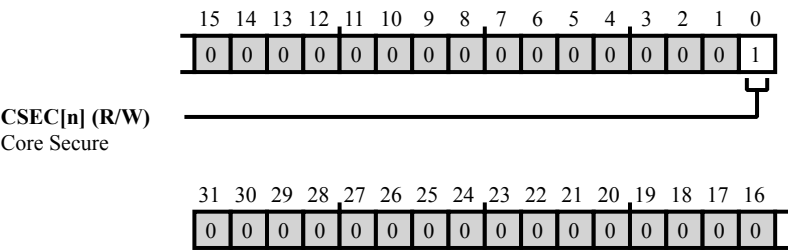


Figure 34-8: SPU_SECUREC[n] Register Diagram

Table 34-7: SPU_SECUREC[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	CSEC[n]	Core Secure. The SPU_SECUREC [n] .CSEC [n] bit controls whether non-secure accesses are allowed to L1 memory of the processor core. When =1, the core (as a slave) is set as secure meaning only secure transactions are allowed to L1.
		0 Disable
		1 Enable

Secure Peripheral Register

In the system, each `SPU_SECUREP[n]` register is assigned to a specific MMR address range associated with one peripheral. Each `SPU_SECUREP[n]` has a Slave Secure (SSEC) bit and a Master Secure (MSEC) bit. When the Slave Secure (SSEC) bit is set, the SPU will only allow Secure Masters generating secure transactions to access the peripheral's MMR address space. When the Master Secure (MSEC) bit is set, the associated peripheral will be secure and will generate secure transactions.

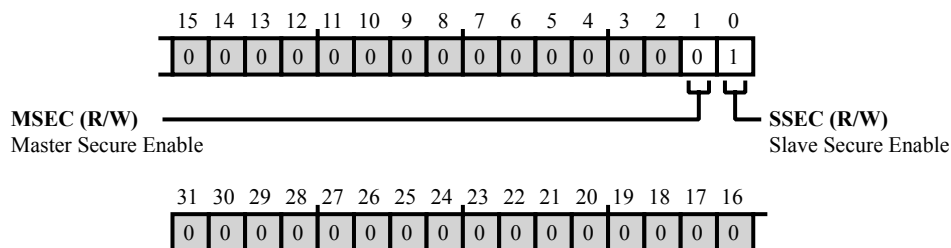


Figure 34-9: SPU_SECUREP[n] Register Diagram

Table 34-8: SPU_SECUREP[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	MSEC	Master Secure Enable. The <code>SPU_SECUREP[n]</code> .MSEC bit controls whether the peripheral generates secure transactions as a master. When clear (=0), the peripheral generates non-secure transactions as a master (if applicable). When set (=1), the peripheral generates secure transactions as a master.
		0 Disable
		1 Enable
0 (R/W)	SSEC	Slave Secure Enable. The <code>SPU_SECUREP[n]</code> .SSEC bit controls whether the peripheral is protected from non-secure transactions. When clear (=0), the security status of the transaction is ignored. When set (=1), only secure transactions are allowed to access the address space of the peripheral and non-secure transactions are blocked.
		0 Disable
		1 Enable

Status Register

The `SPU_STAT` register indicates if there have been any errors, active interrupts and global lock status.

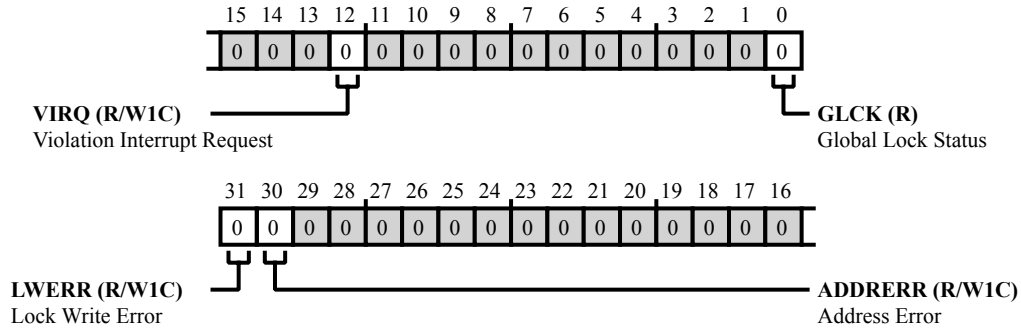


Figure 34-10: `SPU_STAT` Register Diagram

Table 34-9: `SPU_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Lock Write Error. The <code>SPU_STAT.LWERR</code> indicates whether there has been an attempted write to a register in the SPU with its lock bit (<code>SPU_CTL.WPLCK</code> or <code>SCRLCK</code>) set while <code>SPU_CTL.GLCK</code> was asserted. This bit is W1C.
		0 Inactive
		1 Active
30 (R/W1C)	ADDRERR	Address Error. The <code>SPU_STAT.ADDRERR</code> indicates whether there has been an attempted write to a read-only register or an access an invalid address in the SPU MMR address range. This bit is W1C.
		0 Inactive
		1 Active
12 (R/W1C)	VIRQ	Violation Interrupt Request. The <code>SPU_STAT.VIRQ</code> bit indicates that a security and/or protection violation has been detected and interrupt asserted. This is a W1C bit.
		0 Inactive
		1 Active
0 (R/NW)	GLCK	Global Lock Status. The <code>SPU_STAT.GLCK</code> indicates whether the global lock is enabled or disabled.
		0 Disabled (<code>global_lock=0</code>)
		1 Enabled (<code>global_lock=1</code>)

Write Protect Register n

In the system, each `SPU_WP[n]` register is assigned to a specific MMR address range associated with one peripheral. When the appropriate bits are set, writes to the peripheral from a specific master are blocked and an error is returned to the master. For more information, see the processor specific additional information for the `SPU_WP[n]` register.

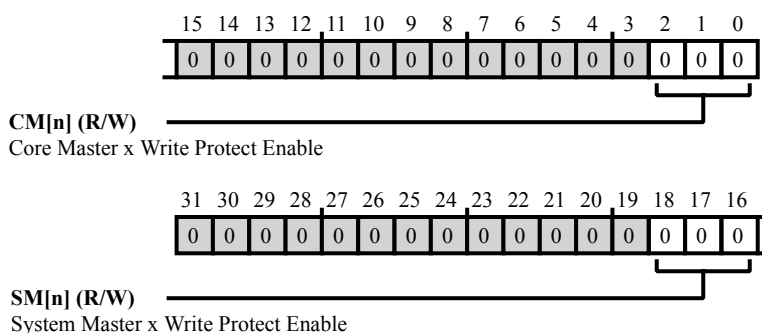


Figure 34-11: SPU_WP[n] Register Diagram

Table 34-10: SPU_WP[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18:16 (R/W)	SM[n]	System Master x Write Protect Enable. The <code>SPU_WP[n] . SM[n]</code> bits correspond to different system masters in the system. When a particular bit is set in this field, the corresponding system master cannot write to the corresponding peripheral's MMR address space. The write attempt is blocked by the SPU.
2:0 (R/W)	CM[n]	Core Master x Write Protect Enable. The <code>SPU_WP[n] . CM[n]</code> bits correspond to different cores in the system. When a particular bit is set in this field, the corresponding core cannot write to the corresponding peripheral's MMR address space. The write attempt is blocked by the SPU.

Write-Protect, Secure Peripheral, and Secure Core Registers

The SPU consists of a collection of write-protect registers each of which are associated with a specific peripheral or slave. The SPU also has a collection of secure peripheral registers which are also associated with specific peripherals. The table gives the write-protect register and secure peripheral number for each of the peripherals that are provided with write protection and security through the SPU. The SPU for the processor is configured with 136 write-protect registers and also 136 secure peripheral registers. The number corresponding to a peripheral correlates to both the Write-Protect register and the Secure Peripheral register.

For each processor, there are different numbers of masters that are able to access the SMMR space.

The `SPU_WP[n]` register shows which bits enable the protection against which master.

For each peripheral, there is a corresponding write-protect register, `SPU_WP[n]`, and secure peripheral register, `SPU_SECUREP[n]`. The table shows the write-protect register and secure peripheral number for each peripheral.

Table 34-11: Write-Protect Register and Secure Peripheral Number(n)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Master Capable
0	MMRG GPV SPACE	WP0	SECUREP0	N/A
1	DMC0_PRG_CDC_BRI DGE	WP1	SECUREP1	N/A
2	SPIF GPV SPACE	WP2	SECUREP2	N/A
3	LP0	WP3	SECUREP3	N/A
4	LP1	WP4	SECUREP4	N/A
5	LP0 DDE	WP5	SECUREP5	Yes
6	LP1 DDE	WP6	SECUREP6	Yes
7	TWI3	WP7	SECUREP7	N/A
8	TWI4	WP8	SECUREP8	N/A
9	TWI5	WP9	SECUREP9	N/A
10	TWI0	WP10	SECUREP10	N/A
11	TWI1	WP11	SECUREP11	N/A
12	TWI2	WP12	SECUREP12	N/A
13	SPORT0A	WP13	SECUREP13	N/A
14	SPORT0B	WP14	SECUREP14	N/A
15	SPORT1A	WP15	SECUREP15	N/A
16	SPORT1B	WP16	SECUREP16	N/A
17	SPORT2A	WP17	SECUREP17	N/A
18	SPORT2B	WP18	SECUREP18	N/A
19	SPORT3A	WP19	SECUREP19	N/A
20	SPORT3B	WP20	SECUREP20	N/A
21	SPORT4A	WP21	SECUREP21	N/A
22	SPORT4B	WP22	SECUREP22	N/A
23	SPORT5A	WP23	SECUREP23	N/A
24	SPORT5B	WP24	SECUREP24	N/A
25	SPORT6A	WP25	SECUREP25	N/A
26	SPORT6B	WP26	SECUREP26	N/A
27	SPORT7A	WP27	SECUREP27	N/A

Table 34-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Master Capable
28	SPORT7B	WP28	SECUREP28	N/A
29	UART0	WP29	SECUREP29	N/A
30	UART1	WP30	SECUREP30	N/A
31	UART2	WP31	SECUREP31	N/A
32	UART3	WP32	SECUREP32	N/A
33	PORTA	WP33	SECUREP33	N/A
34	PORTB	WP34	SECUREP34	N/A
35	PORTC	WP35	SECUREP35	N/A
36	PORTD	WP36	SECUREP36	N/A
37	PORTE	WP37	SECUREP37	N/A
38	PORTF	WP38	SECUREP38	N/A
39	PORTG	WP39	SECUREP39	N/A
40	PORTH	WP40	SECUREP40	N/A
41	PORTI	WP41	SECUREP41	N/A
42	PADS	WP42	SECUREP42	N/A
43	PINT0	WP43	SECUREP43	N/A
44	PINT1	WP44	SECUREP44	N/A
45	PINT2	WP45	SECUREP45	N/A
46	PINT3	WP46	SECUREP46	N/A
47	PINT4	WP47	SECUREP47	N/A
48	PINT5	WP48	SECUREP48	N/A
49	PINT6	WP49	SECUREP49	N/A
50	PINT7	WP50	SECUREP50	N/A
51	SMC0	WP51	SECUREP51	N/A
52	SMPU SMC	WP52	SECUREP52	N/A
53	WDT0	WP53	SECUREP53	N/A
54	WDT1	WP54	SECUREP54	N/A
55	WDT2	WP55	SECUREP55	N/A
56	CNT0	WP56	SECUREP56	N/A
57	OTP MMR	WP57	SECUREP57	N/A
58	SMPU-OTP	WP58	SECUREP58	N/A

Table 34-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Master Capable
59	SWU SMC	WP59	SECUREP59	N/A
60	HADC0	WP60	SECUREP60	N/A
61	TMU0	WP61	SECUREP61	N/A
62	TMR	WP62	SECUREP62	N/A
63	SPORT0A DDE	WP63	SECUREP63	Yes
64	SPORT0B DDE	WP64	SECUREP64	Yes
65	SPORT1A DDE	WP65	SECUREP65	Yes
66	SPORT1B DDE	WP66	SECUREP66	Yes
67	SPORT2A DDE	WP67	SECUREP67	Yes
68	SPORT2B DDE	WP68	SECUREP68	Yes
69	SPORT3A DDE	WP69	SECUREP69	Yes
70	SPORT3B DDE	WP70	SECUREP70	Yes
71	SPORT4A DDE	WP71	SECUREP71	Yes
72	SPORT4B DDE	WP72	SECUREP72	Yes
73	SPORT5A DDE	WP73	SECUREP73	Yes
74	SPORT5B DDE	WP74	SECUREP74	Yes
75	SPORT6A DDE	WP75	SECUREP75	Yes
76	SPORT6B DDE	WP76	SECUREP76	Yes
77	SPORT7A DDE	WP77	SECUREP77	Yes
78	SPORT7B DDE	WP78	SECUREP78	Yes
79	UART0RX DDE	WP79	SECUREP79	Yes
80	UART0TX DDE	WP80	SECUREP80	Yes
81	UART1RX DDE	WP81	SECUREP81	Yes
82	UART1TX DDE	WP82	SECUREP82	Yes
83	UART2RX DDE	WP83	SECUREP83	Yes
84	UART2TX DDE	WP84	SECUREP84	Yes
85	UART3RX DDE	WP85	SECUREP85	Yes
86	UART3TX DDE	WP86	SECUREP86	Yes
87	PPI0 DDE0	WP87	SECUREP87	Yes
88	PPI0 DDE1	WP88	SECUREP88	Yes
89	OSPI	WP89	SECUREP89	N/A

Table 34-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Master Capable
90	CORE MBIST	WP90	SECUREP90	N/A
91	EPPI0	WP91	SECUREP91	N/A
92	MEMREPAIR	WP92	SECUREP92	N/A
93	SPI0TX DDE	WP93	SECUREP93	Yes
94	SPI0RX DDE	WP94	SECUREP94	Yes
95	SPI1TX DDE	WP95	SECUREP95	Yes
96	SPI1RX DDE	WP96	SECUREP96	Yes
97	SPI2TX DDE	WP97	SECUREP97	Yes
98	SPI2RX DDE	WP98	SECUREP98	Yes
99	SPI3TX DDE	WP99	SECUREP99	Yes
100	SPI3RX DDE	WP100	SECUREP100	Yes
101	SPI0	WP101	SECUREP101	N/A
102	SPI1	WP102	SECUREP102	N/A
103	SPI2	WP103	SECUREP103	N/A
104	SPI3	WP104	SECUREP104	N/A
105	EMAC0	WP105	SECUREP105	Yes
106	EMAC1	WP106	SECUREP106	Yes
107	CAN0	WP107	SECUREP107	N/A
108	CAN1	WP108	SECUREP108	N/A
109	DMC0	WP109	SECUREP109	N/A
110	DMC0 PHY	WP110	SECUREP110	N/A
111	DMC0 DFT	WP111	SECUREP111	N/A
112	L2CTL0	WP112	SECUREP112	N/A
113	SMPU L2CTL0 CL2 0	WP113	SECUREP113	N/A
114	SMPU L2CTL0 DL2 0	WP114	SECUREP114	N/A
115	SMPU L2CTL0 CL2 1	WP115	SECUREP115	N/A
116	SMPU L2CTL0 DL2 1	WP116	SECUREP116	N/A
117	SMPU L2CTL0 CL2 2	WP117	SECUREP117	N/A
118	SEC0	WP118	SECUREP118	N/A
119	TRU0	WP119	SECUREP119	N/A
120	SPU0	WP120	SECUREP120	N/A

Table 34-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Master Capable
121	RCU0	WP121	SECUREP121	N/A
122	CGU0	WP122	SECUREP122	N/A
123	CGU1	WP123	SECUREP123	N/A
124	CDU0	WP124	SECUREP124	N/A
125	DPM0	WP125	SECUREP125	N/A
126	PLL0	WP126	SECUREP126	N/A
127	PLL1	WP127	SECUREP127	N/A
128	SWU_L2CTL0_CL2_0	WP128	SECUREP128	N/A
129	SWU_L2CTL0_DL2_0	WP129	SECUREP129	N/A
130	SWU_L2CTL0_CL2_1	WP130	SECUREP130	N/A
131	SWU_L2CTL0_DL2_1	WP131	SECUREP131	N/A
132	SWU SMMR	WP132	SECUREP132	N/A
133	SWU_L2CTL0_CL2_2	WP133	SECUREP133	N/A
134	Enhanced BW MDMA	WP134	SECUREP134	Yes
135	Max BW MDMA	WP135	SECUREP135	Yes
136	Max BW MDMA 1	WP136	SECUREP136	Yes
137	MLB0	WP137	SECUREP137	Yes
138	SWU DMC0	WP138	SECUREP138	N/A
139	SMPU DMC0	WP139	SECUREP139	N/A
140	SMPU SPI2	WP140	SECUREP140	N/A
141	MEC0	WP141	SECUREP141	N/A
142	MEC1	WP142	SECUREP142	N/A
143	MEC2	WP143	SECUREP143	N/A
144	CRC0	WP144	SECUREP144	N/A
145	CRC1	WP145	SECUREP145	N/A
146	MDMA0 DDE0 CRC0	WP146	SECUREP146	Yes
147	MDMA0 DDE1 CRC0	WP147	SECUREP147	Yes
148	MDMA1 DDE0 CRC1	WP148	SECUREP148	Yes
149	MDMA1 DDE1 CRC1	WP149	SECUREP149	Yes
150	MDMA4 DDE0 CRC2	WP150	SECUREP150	Yes
151	MDMA4 DDE1 CRC2	WP151	SECUREP151	Yes

Table 34-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Master Capable
152	MDMA5 DDE0 CRC3	WP152	SECUREP152	Yes
153	MDMA5 DDE1 CRC3	WP153	SECUREP153	Yes
154	SWU SPI2	WP154	SECUREP154	N/A
155	MISC REG	WP155	SECUREP155	N/A
156	CRC2	WP156	SECUREP156	N/A
157	CRC3	WP157	SECUREP157	N/A
158	Enhanced BW MDMA 1	WP158	SECUREP158	Yes
159	STM0	WP159	SECUREP159	N/A
160	GIC_Port0	WP160	SECUREP160	N/A
161	GIC_Port1	WP161	SECUREP161	N/A
162	SH1_IIR1	WP162	SECUREP162	N/A
163	SH1_IIR2	WP163	SECUREP163	N/A
164	SH1_IIR3	WP164	SECUREP164	N/A
165	SH1_FIR	WP165	SECUREP165	N/A
166	SH1_IIR0	WP166	SECUREP166	N/A
167	SH0_IIR1	WP167	SECUREP167	N/A
168	SH0_IIR2	WP168	SECUREP168	N/A
169	SH0_IIR3	WP169	SECUREP169	N/A
170	SH0_FIR	WP170	SECUREP170	N/A
171	SH0_IIR0	WP171	SECUREP171	N/A
172	USB0	WP172	SECUREP172	Yes
173	DAI0	WP173	SECUREP173	N/A
174	DAI1	WP174	SECUREP174	N/A
175	CRYPTO SPE	WP175	SECUREP175	Yes
176	CRYPTO PKP	WP176	SECUREP176	N/A
177	DLMDMA0	WP177	SECUREP177	Yes
178	DAP ROM	WP178	SECUREP178	N/A
179	SHARC0 DBG	WP179	SECUREP179	N/A
180	SHARC0 CTI	WP180	SECUREP180	N/A
181	SHARC0 PTM	WP181	SECUREP181	N/A
182	STM	WP182	SECUREP182	N/A

Table 34-11: Write-Protect Register and Secure Peripheral Number(n) (Continued)

No.	Peripheral/Block Name	SPU WPn Mapping	SECUREPn Mapping	Master Capable
183	SHARC1 DBG	WP183	SECUREP183	N/A
184	SHARC1 CTI	WP184	SECUREP184	N/A
185	SHARC1 PTM	WP185	SECUREP185	N/A
186	CSTF	WP186	SECUREP186	N/A
187	ETF	WP187	SECUREP187	N/A
188	ETR	WP188	SECUREP188	N/A
189	TPIU	WP189	SECUREP189	N/A
190	CTI Trace	WP190	SECUREP190	N/A
191	CTI System	WP191	SECUREP191	N/A
192	A5 Integration ROM	WP192	SECUREP192	N/A
193	A5 DBG	WP193	SECUREP193	N/A
194	A5 PMU	WP194	SECUREP194	N/A
195	A5 CTI	WP195	SECUREP195	N/A
196	A5 ETM	WP196	SECUREP196	N/A
197	TAPC MMR	WP197	SECUREP197	N/A
198	Debug Control	WP198	SECUREP198	N/A
199	SWU C0 S1	WP199	SECUREP199	N/A
200	SWU C0 S2	WP200	SECUREP200	N/A
201	SWU C1 S1	WP201	SECUREP201	N/A
202	SWU C1 S2	WP202	SECUREP202	N/A

ADSP-2159x Specific Information

Global Locking

The global lock signal from the SPU along with the peripheral lock bit can be used to provide lock functionality for the control MMR of the peripheral. The Global Lock (SPU_CTL.GLCK) field determines whether global lock is enabled or not. Global Lock is disabled if the SPU_CTL.GLCK field is 0xAD (default value), otherwise it is enabled. The following is a list of peripherals that have the global lock bit in their control MMR.

- General-Purpose IO (GPIO)
- System Event Controller (SEC0)
- Trigger Routing Unit (TRU0)
- Clock Generation Unit (CGU0)

- Clock Generation Unit1 (CGU1)
- Clock Distribution Unit (CDU0)
- Dynamic Power Management (DPM)
- Reset Control Unit (RCU0)
- System Protection Unit (SPU0)
- L2 Memory Controller (L2CTL0)

35 Security Packet Engine (PKTE)

The PKTE is a security packet engine designed to off-load the host processor to improve the speed of applications requiring cryptographic processing. The packet engine contains a set of modules for encryption and decryption, hashing, and pseudo-random number generation.

PKTE Features

The PKTE has the following features.

- Hardware assisted processing for the cryptographic ciphers, hashes, and pseudo-random number generation
- Header and trailer processing for Internet security protocols
- DMA capability to move data in and out of the engine efficiently and to allow the engine to run autonomously while moving data
- Interrupt controller to signal module status and errors
- Clock manager for enabling or disabling different features to save power

NOTE: Not all algorithms, decrypt, and hash functions and extra features are available on all product models. For complete information on included features, see the product-specific data sheet.

NOTE: This packet engine provides support for various network security protocols by processing headers and trailers as well as accelerating cryptographic functions. Not all processors have direct support for Ethernet. As such, the packet engine can still be used if Ethernet is indirectly used.

PKTE Functional Description

The packet engine contains a set of modules for encryption and decryption, hashing, and pseudo-random number generation. The following sections describe these functional blocks.

ADSP-2159x PKTE Register List

The Security Packet Engine (PKTE) provides security-related features. A set of registers governs PKTE operations. For more information on PKTE functionality, see the PKTE register descriptions.

Table 35-1: ADSP-2159x PKTE Register List

Name	Description
PKTE_ARC4STATE_ADDR	Packet Engine ARC4 State Record Address
PKTE_ARC4STATE_BUF	Starting Entry of 256-byte ARC4 State Buffer
PKTE_BUF_PTR	Packet Engine Buffer Pointer Register
PKTE_BUF_THRESH	Packet Engine Buffer Threshold Register
PKTE_CDRBASE_ADDR	Packet Engine Command Descriptor Ring Base Address
PKTE_CDSC_CNT	Packet Engine Command Descriptor Count Register
PKTE_CDSC_INCR	Packet Engine Command Descriptor Count Increment Register
PKTE_CFG	Packet Engine Configuration Register
PKTE_CLK_CTL	PE Clock Control Register
PKTE_CONT	PKTE Continue Register
PKTE_CTL_STAT	Packet Engine Control Register
PKTE_DATAIO_BUF	Starting Entry of 256-byte Data Input/Output Buffer
PKTE_DEST_ADDR	Packet Engine Destination Address
PKTE_DMA_CFG	Packet Engine DMA Configuration Register
PKTE_ENDIAN_CFG	Packet Engine Endian Configuration Register
PKTE_HLT_CTL	Packet Engine Halt Control Register
PKTE_HLT_STAT	Packet Engine Halt Status Register
PKTE_IMSK_DIS	Interrupt Mask Disable Register
PKTE_IMSK_EN	Interrupt Mask Enable Register
PKTE_IMSK_STAT	Interrupt Masked Status Register
PKTE_INBUF_CNT	Packet Engine Input Buffer Count Register
PKTE_INBUF_INCR	Packet Engine Input Buffer Count Increment Register
PKTE_INT_CFG	Interrupt Configuration Register
PKTE_INT_CLR	Interrupt Clear Register
PKTE_INT_EN	Interrupt Enable Register
PKTE_IUMSK_STAT	Interrupt Unmasked Status Register
PKTE_LEN	Packet Engine Length Register
PKTE_OUTBUF_CNT	Packet Engine Output Buffer Count Register
PKTE_OUTBUF_DECR	Packet Engine Output Buffer Count Decrement Register
PKTE_RDRBASE_ADDR	Packet Engine Result Descriptor Ring Base Address
PKTE_RDSC_CNT	Packet Engine Result Descriptor Count Registers

Table 35-1: ADSP-2159x PKTE Register List (Continued)

Name	Description
PKTE_RDSC_DECR	Packet Engine Result Descriptor Count Decrement Registers
PKTE_RING_CFG	Packet Engine Ring Configuration
PKTE_RING_PTR	Packet Engine Ring Pointer Status
PKTE_RING_STAT	Packet Engine Ring Status
PKTE_RING_THRESH	Packet Engine Ring Threshold Registers
PKTE_SA_ADDR	Packet Engine SA Address
PKTE_SA_ARC4IJPTR	ARC4 i and j Pointer Register
PKTE_SA_CMD0	SA Command 0
PKTE_SA_CMD1	SA Command 1
PKTE_SA_IDIGEST[n]	SA Inner Hash Digest Registers
PKTE_SA_KEY[n]	SA Key Registers
PKTE_SA_NONCE	SA Initialization Vector Register
PKTE_SA_ODIGEST[n]	SA Outer Hash Digest Registers
PKTE_SA_RDY	SA Ready Indicator
PKTE_SA_SEQNUM[n]	SA Sequence Number Register
PKTE_SA_SEQNUM_MSK[n]	SA Sequence Number Mask Registers
PKTE_SA_SPI	SA SPI Register
PKTE_SRC_ADDR	Packet Engine Source Address
PKTE_STAT	Packet Engine Status Register
PKTE_STATE_ADDR	Packet Engine State Record Address
PKTE_STATE_BYTE_CNT[n]	State Hash Byte Count Registers
PKTE_STATE_IDIGEST[n]	State Inner Digest Registers
PKTE_STATE_IV[n]	State Initialization Vector Registers
PKTE_USERID	Packet Engine User ID

ADSP-2159x PKTE Interrupt List

Table 35-2: ADSP-2159x PKTE Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
161	PKTE0_IRQ	PKTE0 Interrupt	Level	

PKTE Definitions

Command Descriptor

An 8-word structure that is either written directly into the packet command MMR set or is placed in a Command Descriptor Ring (CDR) in the processor memory. The packet engine sequentially processes the structure. The command descriptor contains the information that varies for every packet. This information includes pointers to the SA record, the state information, the source packet, and the destination packet.

Command Descriptor Ring (CDR)

A circular contiguous portion of memory which is used to manage one or more command descriptions for the packet engine.

Result Descriptor

When the packet engine completes the processing of a packet, it writes a result descriptor with the state information. The result descriptor can be read directly from the result register set or from the Result Descriptor Ring (RDR) in the processor memory.

Result Descriptor Ring (RDR)

A circular contiguous portion of memory which holds the mirror or copy of the CDR but contains the result descriptors. The RDR and CDR can be overlaid on top of each other.

Security Association (SA) Record

A structure that contains the remainder of the information the packet engine requires to process a packet. Most of the information fields in the SA record such as the key and encryption mode are static for the lifetime of the association. The fields do not require frequent manipulation by the processor core. The SA record non-static fields are the sequence number and sequence number mask. The SA record can have a corresponding state record for saving results from the current operations that are useful for future operations. The state record can hold the IV, the hash byte count, and the intermediate hash digest.

Cipher

A method or algorithm to encrypt or decrypt information

Hash

A cryptographic hash is a function that takes an arbitrary block of data and returns a fixed-size bit string. Four main properties define the function:

- It is easy to compute a hash value for any given input

- It is infeasible to generate the original input from a given hash
- It is infeasible to modify the input without changing the resulting hash
- It is infeasible to find two different inputs that result in the same hash

Autonomous Ring Mode (ARM)

Mode of operation in which most of the parameters as well as the data are set up in memory and moved to the engine for configuration and processing through DMA.

Target Command Mode (TCM)

Mode of operation where some parameters are set up in memory and moved into the packet engine through DMA while the other parameters are directly written to the registers. DMA moves the input and output data in and out of the engine.

Direct Host Mode (DHM)

Mode of operation that does not use DMA. All parameters are directly written to and read from the MMRs. The input and output are written to and read from the FIFO buffers.

Cipher Module

The cipher module does the symmetric encrypt or decrypt operations for:

- Data Encryption Standard (DES)
- Triple-DES
- ARC4
- Advanced Encryption Standard (AES) algorithms

The cipher module supports standard modes for DES and AES that include Electronic Code Book (ECB) and Cipher Block Chaining (CBC). The key size for DES is 56 bits, for Triple-DES is 168 bits. The AES module also provides support for AES counter-modes for IPsec and SRTP. All AES modes can use key sizes of 128 bits and 192/256 bits. Key scheduling is automatic and done in parallel with the encrypt or decrypt operation.

Hash Module

The hash module is tightly coupled with the encrypt or decrypt module and provides hardware accelerated one-way hash functions. Operations that combine both hash and encrypt or decrypt functions are provided to reduce processing time for data that needs both applied. For hash-then-decrypt operations, the packet engine performs parallel execution of both functions from the input buffer. For encrypt-then-hash operations, the processing is pipelined from the input buffer to provide minimum latency. An offset can be specified between the start of hashing and the start of encryption to support protocols such as IPsec or SRTP. The HMAC keyed hashing mechanism is supported

for MD5, SHA-1, SHA-2-224 and SHA-2-256. The SSL-MAC is supported for MD5 and SHA-1. A second AES-CBC module for the hash function enables parallel processing for AES-CCM, a combined hash and encrypt algorithm.

Pseudo-Random Number Generator

Cipher algorithms that operate in CBC mode or counter-mode, require an IV. This IV must not be secret; however the IV must be unpredictable and unique for each execution of the encryption process. Pseudo-random number generators are deterministic algorithms that output statistically independent and unbiased numbers. True random number generators are non-deterministic and use the randomness that occurs in a physical process. The packet engine incorporates an ANSI X9.31 compliant Pseudo Random Number Generator (PRNG) that it can use to generate unique IVs using strong encryption. The ANSI X9.31 PRNG is defined as part of the ANSI X9 standards that are used to secure financial transactions. The function can also be used for pseudo-random number generation as part of an implementation of the digital signature standard described in NIST FIPS PUB 186-2.

The PRNG function, as defined by ANSI X9.31, is based on the AES cipher. This section describes the function to promote understanding of the different inputs and outputs of the PRNG function itself.

NOTE: The PRNG in the packet engine is only based on the AES cipher with 128-bit keys. Other ciphers and key lengths are not supported for the PRNG based on ANSI X9.31.

Let $e \times K(Y)$ represent the AES encryption of Y under the key K.

The PRNG function uses three inputs:

- K, a 128-bit key
- V, a 128-bit seed value
- DT, a 128-bit date/ time vector which is updated on each iteration

The intermediate value I is the result of an AES encryption of the data and time vector under key K.

$$I = e \times K(DT)$$

That value I is then XOR-ed with the seed V and AES encrypted under key K. The result R is the output of the PRNG function.

$$R = e \times K(I \text{ XOR } V)$$

A new seed value V is generated from the AES encryption of the result R XOR'ed with the intermediate value I under the key K.

$$V = e \times K(R \text{ XOR } I)$$

The PRNG function is deeply integrated inside the datapath of the packet engine. The function is controlled indirectly through the PRNG mode bits in the `PKTE_CTL_STAT.PRNGMD` bit field of the command descriptor and the IV source selection bits in the `PKTE_SA_CMD0.IVSRC` bit field of the SA record.

The PKTE module supports four different modes.

1. Load IV from PRNG for the current operation: `PKTE_CTL_STAT.PRNGMD = 0b00` and `PKTE_SA_CMD0.IVSRC = 0b11`.
2. PRNG init mode initializes the PRNG with a key, seed, and date/time value: `PKTE_CTL_STAT.PRNGMD = 0b01`.

Before the PRNG function can be used, it must be initialized with a key, seed, and date/time value. At initialization, the key, seed, and date/time values are programmed. Other PRNG operations do not change the key, however the seed, and date/time values are updated (not re-programmed). The date/time is updated every 128 system clock cycles.

The date/time is a 128-bit value with randomly distributed number of ones and zeros. It must not be all zeros.

The *SA Record for PRNG Init Operation* table shows how the key, seed and date/time values are loaded into the PKTE registers for initialization.

Table 35-3: SA Record for PRNG Init Operation

Parameter	SA Field	Description
K	SA_KEY0	PRNG key [127:96]
	SA_KEY1	PRNG key [95:64]
	SA_KEY2	PRNG key [63:32]
	SA_KEY3	PRNG key [31:0]
V	SA_IDIGEST0	PRNG seed [127:96]
	SA_IDIGEST1	PRNG seed [95:64]
	SA_IDIGEST2	PRNG seed [63:32]
	SA_IDIGEST3	PRNG seed [31:0]
DT	SA_ODIGEST0	PRNG date/time [127:96]
	SA_ODIGEST1	PRNG date/time [95:64]
	SA_ODIGEST2	PRNG date/time [63:32]
	SA_ODIGEST3	PRNG date/time [31:0]

3. PRNG generate mode generates pseudo-random data on initialized key, seed, and date/time value: `PKTE_CTL_STAT.PRNGMD = 0b10`.

The PRNG function can be used to generated pseudo-random data for other purposes than IVs. For this mode, the PRNG must have been initialized once with the PRNG init mode.

The PRNG generate mode uses the initialized key and unique changing date/time value as inputs. The pseudo-random data is output to the output buffer of the packet engine.

The `LEN` field in the command descriptor indicates the amount of pseudo random data that is generated in multiples of 16 bytes. The maximum is $255 \times 16 = 4080$ bytes.

In autonomous ring mode, the output data is copied to the host memory at the destination address in the command descriptor. In direct host mode, the host must read the data directly from the output buffer. No SA record is used for this function.

Directly after the PRNG generate mode, a new pseudo-random number is generated and available for the next operation that uses the option `PKTE_SA_CMD0.IVSRC = PRNG`.

4. PRNG test mode generates pseudo-random data on initialized key, seed, and input (test) data:

`PKTE_CTL_STAT.PRNGMD = 0b11`.

The PRNG test mode can be used to test the correctness of the PRNG function. This mode is similar to the PRNG generate mode, except that the data is read from the input buffer of the packet engine, instead of the date/time value.

For this mode, the PRNG must have been initialized once with the PRNG Init mode.

The `LEN` field in the command descriptor indicates the amount of pseudo-random data to be generated in multiples of 16 bytes. The maximum is limited to the `LEN` field in bytes.

In autonomous ring mode, the output data is copied to the host memory at the destination address in the command descriptor. In the direct host mode, the host must read the data directly from the output buffer. No SA record is used for this function.

Directly after the PRNG test mode, a new pseudo-random number is generated and available for the next operation that uses the option `PKTE_SA_CMD0.IVSRC = PRNG`.

Packet Engine Processing Details

This section describes data processing through the packet engine. It describes padding and supported algorithms for each protocol.

A valid Security Association (SA) must be created before packet processing can start. A formatted SA record must reside in memory and be accessible to the packet engine. The host processor application is responsible for these tasks.

Crypto Padding

Padding is the process of adding data to fill-out a fixed-size plain text data structure. Three factors determine when to use a pad field:

1. If a block cipher encryption algorithm is used, a pad field is used to expand the plain text to a multiple of the block size.
2. Padding can be used to ensure that the cipher text terminates on an n-byte boundary.
3. Padding can conceal the actual length of the payload.

To facilitate peak encrypt or decrypt performance, the packet engine supports the following most commonly used padding functions in hardware:

1. Pad generation and insertion of pad bytes to the end of plain text prior to encryption, for outbound operations.

2. Pad verification to check for correct padding after decrypting a packet for inbound operations.
3. Pad consumption to strip the pad bytes from the plain text data after decrypting a packet, for inbound operations.

Pad Generation and Insertion

The pad type and quantity of bytes the packet engine inserts depends on the plain text length and the value of the following fields:

- `PKTE_SA_CMD0.PADTYPE` and `PKTE_SA_CMD0.ETPAD` defines the type of padding
- `PKTE_CTL_STAT.PADVAL` defines a value that is inserted in the pad
- `PKTE_SA_CMD0.CIPHER` enforces a certain pad alignment
- `PKTE_SA_CMD1.CIPHERMD` enforces a certain pad alignment
- `PKTE_SA_CMD0.SCPAD` allows stream ciphers to be padded
- `PKTE_CTL_STAT.PADCTLSTAT` controls the pad alignment

The `PKTE_CTL_STAT.PADCTLSTAT` bit field of the result descriptor returns the total number of inserted pad bytes.

Pad Types

The pad type bit field (`PKTE_SA_CMD0.PADTYPE`) and the extended pad bit (`PKTE_SA_CMD0.ETPAD`) select the pad type for the extended protocol group. The packet engine can generate different pad types in hardware as described in the *Pad Examples* table.

The `PKTE_CTL_STAT.PADVAL` bit field, together with the number of pad bytes, defines the value that is inserted in the pad. The format of the pad and the use of this field is best explained in an example (see the *Pad Examples* table).

For the IPsec pad type, this field holds the value that is inserted into the next header field (in the ESP trailer) of the innermost operation's header. For the Constant pad type or the Constant SSL pad type, this field holds the inserted fixed constant pad value. For all other pad types, this field is not used and must be zero.

Table 35-4: Pad Types

Pad Type	Value	Description
IPSec	0b000	Append 0 to 255 pad bytes, followed by a pad length byte and a next header byte. The first pad byte appended to the plain text is numbered 1, with subsequent pad bytes making up a monotonically increasing sequence: 1, 2, 3 and up. Append the pad length field that indicates the number of pad bytes (0–255), where a value of zero indicates no pad bytes present. Append the next header byte as specified in the <code>PKTE_CTL_STAT.PADVAL</code> field of the command descriptor. A minimum of 2 bytes are appended; zero pad bytes plus the pad length byte plus the next header byte, in which case the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the result descriptor returns 0x02. A maximum of 257 bytes can be appended, in which case the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the descriptor returns 0x01.
PKCS#7	0b001	Appends 1–128 pad bytes with a pad byte value equal to the pad length (1–128).
Constant	0b010	Appends 0–255 pad bytes of a user-specified character to the plain text data. This character is specified in the <code>PKTE_CTL_STAT.PADVAL</code> field of the command descriptor.
Zero	0b011	Appends 0–255 pad bytes of 0x00 to the plain text data.
Constant SSL	0b110	Appends 0–255 pad bytes of a user-specified character to the plain text data, followed by a ‘pad length’ byte (0–255). This character is specified in the <code>PKTE_CTL_STAT.PADVAL</code> field of the command descriptor. A total of 256 bytes can be appended, in which case the pad field returns 0x00.

For example, the *Pad Examples* table shows the appended pad for any of the pad types for an outbound (encrypt) operation. The table shows a plain text input of 2 bytes using the 8-byte block cipher crypto-algorithm DES-ECB and a `PKTE_CTL_STAT.PADVAL` field value of 0xAA.

Table 35-5: Pad Examples

Pad Type	Pad field (extended to Plain Text)	<code>PKTE_CTL_STAT</code>
IPSec	0x01, 0x02, 0x03, 0x04, 0x04, 0xAA	0x06
PKCS#7	0x06, 0x06, 0x06, 0x06, 0x06, 0x06	0x06
Constant	0xAA, 0xAA, 0xAA, 0xAA, 0xAA, 0xAA	0x06
Zero	0x00, 0x00, 0x00, 0x00, 0x00, 0x00	0x06
Constant SSL	0xAA, 0xAA, 0xAA, 0xAA, 0xAA, 0x05	0x06

Pad Length

The *Pad Alignment* table lists the alignment (boundaries) to which the packet engine will pad, based on:

- The selected crypto-algorithm
- Crypto mode
- The value of the pad stream cipher bit
- The value of pad control

The minimum number of inserted pad bytes depends on the cipher algorithm, selected using the `PKTE_SA_CMD0.CIPHER` bit field, and the cipher mode, selected using the `PKTE_SA_CMD1.CIPHERMD` bit field.

For block ciphers, the plain text data is always (as a minimum) padded to the next block boundary. More pad bytes beyond the algorithm or protocol alignment requirements can be inserted using the pad control (`PKTE_CTL_STAT.PADCTLSTAT`) in the command descriptor. This feature can be used for *traffic flow security* to conceal the number of plain text bytes in an encrypted packet.

Encrypt operations that use block ciphers have minimum pad requirements based on their block size. The packet engine enforces a minimum pad alignment for block ciphers according to the *Pad Alignment* table. For ESP out-bound operations, the minimum pad alignment is forced to 4 bytes.

For stream ciphers and null crypto the data is never padded when the stream cipher pad bit (`PKTE_SA_CMD0.SCPAD`) = 0. When `PKTE_SA_CMD0.SCPAD` = 1 the plain text data is padded to the length as defined by the `PKTE_CTL_STAT.PADCTLSTAT` bits in the command descriptor.

NOTE: The SSL protocol does not allow padding to exceed the ciphers block length. This length is 8 bytes for DES/Triple-DES and 16 bytes for AES. For SSL, the packet engine does not enforce this pad alignment value. The host processor must ensure that the `PKTE_CTL_STAT.PADCTLSTAT` bit field is configured correctly.

Table 35-6: Pad Alignment

Pad Control PADCTLSTAT = PKTE_CTL_STAT[31:24]			0x00	0x01* ¹	0x02	0x04	0x08	0x10	0x20	0x40	0x80* ²
Crypto Algo- rithm	Crypto Mode	Pad Stream Ciphers	%8	%1	%4	%8	%16	%32	%64	%128	%256
DES	ECB, CBC	N/A	8	8	8	8	16	32	64	128	256
AES	ECB, CBC	N/A	16	16	16	16	16	32	64	128	256
	CTR, ICM with ESP	N/A	8	4	4	8	16	32	64	128	256
	CTR, ICM no ESP	no	0	0	0	0	0	0	0	0	0
		yes	8	0	4	8	16	32	64	128	256
NULL	with ESP	N/A	8	4	4	8	16	32	64	128	256
	no ESP	no	8	0	0	0	0	0	0	0	0
		yes	8	0	4	8	16	32	64	128	256
ARC4	with ESP	N/A	8	4	4	8	16	32	64	128	256
	no ESP	no	0	0	0	0	0	0	0	0	0
		yes	8	0	4	8	16	32	64	128	256

*1 If `PKTE_CTL_STAT.PADCTLSTAT` is configured for no padding (0x01), it does not mean that no padding bytes are inserted. When PKCS#7 padding is selected, a pad length field with a value =1 is inserted. When SSL or TLS padding is selected, a pad length field with a value =0 is inserted. When IPsec padding is selected, a pad length field is forced to (`PKTE_CTL_STAT.PADCTLSTAT=0x20`). When zero pad and constant pad are selected, no pad bytes are inserted.

*2 Pad type PKCS#7 supports a maximum length of 128 pad bytes, so the packet engine overrules a 256-byte alignment (`PKTE_CTL_STAT.PADCTLSTAT=0x80`) to a 128-byte boundary (`PKTE_CTL_STAT.PADCTLSTAT=0x40`).

The packet engine does not constrain the pad type that is used for an operation; any pad type can be used for each operation. The user must be aware that some protocol specifications only allow specific pad types. The SRTP specification does not have padding defined as padding performed by RTP. The host must pad the RTP packet.

The host software can implement padding that is not supported in hardware. In this case, the host must select the *zero pad* type and set the `PKTE_CTL_STAT.PADCTLSTAT` bit field in the packet descriptor to zero (no padding). The hardware padding engine does not add any bytes. When using a block cipher, the host must insert pad bytes. Then, the data to be encrypted (plain text and pad bytes) are a multiple of the block ciphers boundary. For stream ciphers, any number of pad bytes can be added.

Pad Verification and Consumption

The packet engine can validate a pad type against the expected values. The value of the following bits controls the pad verify function:

- `PKTE_SA_CMD0.PADTYPE` and `PKTE_SA_CMD0.EXTPAD` define the type of padding
- `PKTE_SA_CMD0.CIPHER` enforces a certain pad alignment
- `PKTE_SA_CMD1.CIPHERMD` enforces a certain pad alignment
- `PKTE_SA_CMD0.SCPAD` allows stream ciphers to be padded

When packet processing is complete, the status byte in the first word of the result descriptor reports the pad verification status. Refer to the [Table 35-28 Extended Error Codes - Status Encoding](#) table.

The `PKTE_CTL_STAT.PADCTLSTAT` bits in the first word of the result descriptor return the total number of detected pad bytes and returns zero for a pad verify error.

When the IPsec pad type is selected, the `PKTE_CTL_STAT.PADVAL` bit field of the result descriptor returns the next header field. For IPsec ESP outbound operations, this field returns the decimal value 50. For IPsec inbound operations and basic inbound operations that use the IPsec pad mode, the packet engine returns the next header field it detects on the header of the innermost operation. This value is typical for the payload protocol, such as TCP or UDP. However, in bundling scenarios or in IPv6 with destination option headers, another header value could be seen. For all other inbound operations, the returned pad value is zero.

Pad verification is performed for inbound (decrypt) operations that use IPsec, TLS/DTLS or PKCS#7 pad type:

- In combination with a block cipher algorithm: DES-ECB, DES-CBC, AES-ECB, AES-CBC,
- In combination with a stream cipher and stream cipher padding `PKTE_SA_CMD0.SCPAD` enabled: AES-CTR, AES-ICM and ARC4
- In combination with null-crypto (no encryption).

Pad Types

The `PKTE_SA_CMD0.PADTYPE` bit field and the extended pad `PKTE_SA_CMD0.ETPAD` bit selects the pad type for the extended protocol group pad type. The packet engine can verify the different pad types in hardware as described in the *Pad Types* table.

The constant and zero pad types are not verified since they do not include a pad length field. The SSL pad type is not verified since it does not have a defined pattern.

Table 35-7: Pad Types

Pad Type	SA_CMD0[18, 7:6]	Description
IPSec	0b000	Verify that the pad field includes 0–255 pad bytes, followed by a correct pad length and a next header byte. Verify that pad bytes appended to the plain text are an incremental count, starting at one. Verify that the pad length field is the number of pad bytes (0–255), where a value of zero indicates no pad bytes present. Verify that a next header byte is present as the last byte of the packet, the value is not verified. This is after removal of the ICV. The total number of detected pad bytes is returned in the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the result descriptor. NOTE: A minimum of 2 bytes must be present, zero pad bytes plus the pad length byte plus the next header byte. In this case the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the descriptor returns 0x02. A maximum of 257 bytes can be present, in which case the <code>PKTE_CTL_STAT.PADCTLSTAT</code> field in the result descriptor returns 0x01. The value of the next header byte is returned in the <code>PKTE_CTL_STAT.PADVAL</code> field in the result descriptor.
PKCS#7	0b001	Verify that the pad field includes 1–128 pad bytes, with a pad byte value equal to the pad length (1–128).

When a block cipher is used and the payload is not padded to the nearest block size boundary, as required by the protocol, a *block size error* is generated for all pad types.

Pad Consumption

The packet engine can optionally consume the decrypted pad bytes for an inbound operation that uses the IPsec, SSL, TLS/DTLS, or PKCS#7 pad types. The pad types constant and zero are not consumed since they do not include a pad length field.

Pad consumption (or stripping) is selected on a flow-by-flow basis in the SA-record with the `PKTE_SA_CMD1.CPYPAD` bit. When this bit is set, the length returned in the `LEN` field of the result descriptor is the total length of the plain text including the pad. When the `PKTE_SA_CMD1.CPYPAD` is disabled, the detected pad length, as returned in the `PKTE_CTL_STAT.PADCTLSTAT` bits, is subtracted from the total length and then returned in the `LEN` field of the result descriptor.

The pad is always written to the result packet buffer in memory. When the `PKTE_SA_CMD1.CPYPAD` bit is disabled, only the result length is corrected.

Crypto and Hash Algorithms

The packet engine supports a wide range of crypto and hash algorithms to accelerate basic operations and protocol operations. These algorithms are:

- Basic Encrypt and Basic Decrypt Operations
- Basic Hash Operations
- Basic Encrypt-Hash and Basic Hash-Decrypt Operations
- IPSec ESP Operations
- SRTP Operations

The following tables provide allowed algorithm combinations. Those algorithms not listed in the tables are invalid and can give unexpected results.

NOTE: Not all crypto and hash algorithms are available on all product models. For information on algorithm availability, see the product-specific data sheet.

Table 35-8: Algorithms for Basic Encrypt and Basic Decrypt Operations

Crypto Algorithm	Crypto Mode
DES, Triple-DES	ECB, CBC
AES	ECB, CBC, CRT, ICM
NULL	—

Table 35-9: Algorithms for Basic Encrypt and Basic Decrypt Operations

Crypto Algorithm	Crypto Mode
SHA-1	Basic hash, HMAC, SSL-MAC
SHA-224	Basic hash, HMAC
SHA-256	Basic hash, HMAC
NULL	—

Table 35-10: Algorithms for Basic Encrypt-Hash and Basic Hash-Decrypt Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
DES, Triple-DES	ECB, CBC	SHA-1	Basic hash, HMAC, SSL-MAC
		SHA-224	Basic hash, HMAC
		SHA-256	Basic hash, HMAC
		MD5	Basic hash, HMAC, SSL-MAC
		NULL	—

Table 35-10: Algorithms for Basic Encrypt-Hash and Basic Hash-Decrypt Operations (Continued)

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
AES	ECB, CBC, CRT, ICM	SHA-1	Basic hash, HMAC, SSL-MAC
		SHA-224	Basic hash, HMAC
		SHA-256	Basic hash, HMAC
		MD5	Basic hash, HMAC, SSL-MAC
		NULL	—
NULL	—	SHA-1	Basic hash, HMAC, SSL-MAC
		SHA-224	Basic hash, HMAC
		SHA-256	Basic hash, HMAC
		MD5	Basic hash, HMAC, SSL-MAC

Table 35-11: Algorithms for IPsec ESP Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
DES, Triple-DES	CBC	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
AES	CBC, CTR	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
NULL	CBC	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	—

Table 35-12: Algorithms for Basic SSL and Extended SSL Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
DES, Triple-DES	CBC	SHA-1	SSL-MAC
		MD5	SSL-MAC
		NULL	—

Table 35-12: Algorithms for Basic SSL and Extended SSL Operations (Continued)

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
AES	CBC	SHA-1	SSL-MAC
		MD5	SSL-MAC
		NULL	—
ARC4	Stateful	SHA-1	SSL-MAC
		MD5	SSL-MAC
		NULL	—
NULL	—	SHA-1	SSL-MAC
	—	MD5	SSL-MAC
	—	NULL	—

Table 35-13: Algorithms for Basic TLS, Extended TLS and DTLS Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
DES, Triple-DES	CBC	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
AES	CBC, CTR	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
ARC4 ¹	Stateful	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—
NULL	—	SHA-1	HMAC
		SHA-224	HMAC
		SHA-256	HMAC
		MD5	HMAC
		NULL	—

1 Only for Basic TLS and Extended TLS

Table 35-14: Algorithms for SRTP Operations

Crypto Algorithm	Crypto Mode	Hash Algorithm	Hash Mode
AES	ICM	SHA-1	HMAC
NULL	—	SHA-1	HMAC

IV Processing

An initialization vector (IV) is necessary to start a cipher stream or a block cipher in any of the streaming modes of operation. The IV must be unique for each packet. The IV ensures that all cipher texts are unique even if produced by the same encryption key. This functionality prevents every packet from needing a unique encryption key.

Depending on the packet engine operation, the IV can be loaded from different sources. The IV format depends on the algorithm and the source of the IV. The *Format of the IV* table provides an overview of all IV formats.

Table 35-15: Format of the IV

Algorithm	IV Source (PKTE_SA_CMD0.IVSR)	Format	IV Offset (PKTE_SA_CMD1.HSHCOFFST)
DES/Triple-DES (CBC)	Previous Result of IV	Internal IV register [63:0]	0x00
	Input Buffer	Input buffer [63:0]	0x02
	Saved IV	State Record Saved IV [63:0]	0x00
	Automatic	PRNG output [63:0]	0x00
AES (CBC)	Previous Result of IV	Internal IV register [127:0]	0x00
	Input Buffer	Input Buffer [127:0]	0x04
	Saved IV	State Record Saved IV [127:0]	0x00
	Automatic	PRNG output [127:0]	0x00
AES (ICM) for Basic and SRTP	Input Buffer	Input Buffer [127:0]	0x04
	Saved IV	State Record Saved IV [127:0]	0x00
AES (CTR) for Basic and IP-sec	Input Buffer	SA_NONCE Input Buffer [63:0] 0x00000001	0x02
	Saved IV	State Record Saved IV [127:0]	0x00
	Automatic	SA_NONCE PRNG output [95:32] 0x00000001	0x00

Notes:

1. The PKTE_SA_CMD1.HSHCOFFST bit field provides the IV offset. The offset is only applicable for basic hash or encrypt operations. For protocol operations, the offset is automatically enforced.

2. AES-CTR: The Nonce value as described in RFC 3686, is mapped to the `PKTE_SA_NONCE` register. This Nonce value remains constant for the lifetime of the security association.
3. The host processor controls the IV update using the save-IV bit (`PKTE_SA_CMD0.SVIV`). When part of a packet is processed using a stream cipher and the encrypt or decrypt data is not an integer multiple of the block size, the saved IV is invalid. It must not be used to resume processing the packet.
4. The packet engine supports automatic IV generation for outbound operations. A new IV is generated for every packet with the internal PRNG. This automatic IV generation can be used for all DES, Triple-DES, and AES modes that use an IV, except for AES-ICM mode.
5. For outbound operations, automatic IV generation is the most efficient. No additional I/O is required, and the host processor does not need to provide the IV. When the saved IV option supplies the IV, the IV must be changed for each packet sent. This activity happens when the packet engine writes back the IV to the state record after processing.
6. Outbound IPsec operations put the IV explicitly at the front of the packet. For an inbound IPsec operation, loading from the input buffer is most efficient and always used.
7. CBC processing must not use a predictable IV. Do not use the saved IV and previous result IV options for CBC processing. Refer to RFC 3602 for more details.

ARC4 Processing

The ARC4 algorithm supports two modes of operation: stateless and stateful. For SSL, TLS and DTLS operations that use the ARC4 algorithm, the mode must be set to stateful.

Stateless Mode. Each packet is processed with a newly initialized ARC4 key taken from the key field of the SA record. In this mode, the state information from the SA is never to be read.

CAUTION: If an ARC4 operation in stateless mode is interrupted by an *Interface Error* and the ARC4 state building process is aborted, the next packet fails. The ARC4 is not reset between two packets. For the next packet the ARC4 proceeds from the internal state that it was aborted. Even a soft-reset does not reset the ARC4 internal state, a hardware reset is required.

Stateful Mode. When the `PKTE_CTL_STAT.INITARC4` bit =1, the ARC4 algorithm initializes using the key specified in the SA record. When the `PKTE_CTL_STAT.INITARC4` bit =0, the ARC4 context is read from the ARC4 state field of the SA record and the *i* and *j* pointer field of the SA record. The encrypt and decrypt processing continues from this algorithm state.

The packet engine supports ARC4 key sizes of 40 bits to 128 bits. Longer keys can be used, but they cannot be made inside the packet engine.

The host processor applies ARC4 key scheduling function to the s-box and puts the 256-byte result into the ARC4 state record. It writes the *i* and *j* pointers in the SA (initial *i* = 1, *j* = 0). The host programs the packet engine and specifies stateful operation to continue the ARC4 algorithm.

Hash State Loading

The hash state can be loaded from various sources, depending on the selected protocol and hash algorithm. The *Different Sources for Loading the Hash State* table provides a list of all the options.

Table 35-16: Different Sources for Loading the Hash State

Hash Algorithm PKTE_SA_CMD0.HASHSRC =	From SA 0b00	RESERVED 0b01	From State 0b10	No Load 0b11
SHA-1	yes	-	yes	Yes
SHA-224	Yes	-	Yes	Yes
SHA-256	Yes	-	Yes	Yes
Null hash	x	x	x	x

Sequence Number Processing

The packet engine supports sequence number generation and verification for IPsec and extended SSL, TLS, and DTLS protocol operations.

A Sequence Number (SN) is an unsigned number that a sender must implement and a receiver can use to support anti-replay service (replay attacks) for a specific SA. This processing includes detection of the same packet that arrives more than once and detection of packets that arrive in an incorrect sequence and is outside an accepted level of order relative to the last received packet.

The packet engine supports the following options.

- Sequence number loaded from SA for outbound operations and is retrieved from the input packet for inbound operations.

The sequence number and sequence number mask fields are part of the SA record.

Sequence Number Processing in Extended SSL/TLS

SSL and TLS use an implicit sequence number of 64 bits that is not send in the packet but part of the hash.

For inbound operations, no sequence number verification is performed; instead an incorrect sequence number results in an authentication error (PKTE_CTL_STAT.AUTHERR).

For outbound operations, with PKTE_SA_CMD0.HDRPROC enabled and PKTE_SA_CMD1.ENSQNCHK enabled, the packet engine generates a sequence number error when the 64-bit sequence number counter overflows (counter is $2^{64}-1$ and increments to 0). The host processor must not send the packet.

For both outbound and inbound operations, with PKTE_SA_CMD0.HDRPROC enabled and PKTE_SA_CMD1.ENSQNCHK enabled, the packet engine reads the sequence number from the PKTE_SA_SEQNUM[n] registers in the SA. When the operation is finished, the packet engine stores the incremented sequence number in the same SA fields. The sequence number mask PKTE_SA_SEQNUM_MSK[n] registers in the SA are not used.

For both outbound and inbound operations, with `PKTE_SA_CMD0.HDRPROC` disabled, the packet engine does not increment the sequence number. It expects the host to provide the sequence number through the input buffer as part of the header.

Sequence Number Processing in DTLS

DTLS uses an explicit sequence number of 64 bits that is sent in the packet. The DTLS sequence number is composed of the epoch (16 bits) and packet sequence number (48 bits) that together form the 64-bit number, like the TLS sequence number. The epoch is incremented after each *Change Cipher Spec* message. The packet sequence number is incremented per packet starting from zero after each change cipher spec message.

For outbound operations, with `PKTE_SA_CMD0.HDRPROC` enabled, the packet engine reads the sequence number from the `PKTE_SA_SEQNUM[n]` fields in the SA, then inserts the sequence number in the packet. Then the packet engine stores the incremented sequence number in the `PKTE_SA_SEQNUM[n]` fields in the SA. The sequence number mask `PKTE_SA_SEQNUM_MSK[n]` fields in the SA are not used.

For outbound operations, with `PKTE_SA_CMD0.HDRPROC` enabled and `PKTE_SA_CMD1.ENSQNCHK` enabled, the packet engine generates a sequence number error when the 48-bit sequence number counter overflows (counter is $2^{48} - 1$ and increments to 0). The host must not send the packet.

For outbound operations, with `PKTE_SA_CMD0.HDRPROC` disabled or `PKTE_SA_CMD1.ENSQNCHK` disabled, the packet engine does not increment and verify a sequence number counter overflow, and therefore never generates a sequence number error. With `PKTE_SA_CMD0.HDRPROC` disabled the Packet Engine does not update the sequence number and sequence number mask fields in the SA and expects the host to provide the sequence number through the input buffer as part of the header.

For inbound operations, with `PKTE_SA_CMD0.HDRPROC` enabled and `PKTE_SA_CMD1.ENSQNCHK` enabled, the packet engine verifies the `PKTE_SA_SEQNUM[n]` fields against the sequence number in the packet using the `PKTE_SA_SEQNUM_MSK[n]` from the SA. Three situations can occur:

1. If the received sequence number falls outside and above the 64-bit sequence number mask, the mask is shifted. The packet engine updates the SA with the received sequence number and the shifted sequence number mask.
2. If the received sequence number falls inside the 64-bit sequence number mask and is not a duplicate sequence number (the same as received before), the corresponding bit in the mask is set. The packet engine updates the SA with the received sequence number and the updated sequence number mask.
3. If the received sequence number falls outside the 64-bit sequence number mask or matches an earlier received number a sequence number error is generated. The packet engine does not update the sequence number and sequence number mask fields in the SA. The host must discard the packet.

For inbound operations, with `PKTE_SA_CMD0.HDRPROC` disabled or `PKTE_SA_CMD1.ENSQNCHK` disabled, the packet engine does not verify the sequence number against the sequence number in the packet and therefore never generates a sequence number error.

With `PKTE_SA_CMD0.HDRPROC` disabled the packet engine does not update the sequence number and sequence number mask fields in the SA and expects the host to provide the sequence number through the input buffer as part of the header.

The following tables provide details of sequence number processing.

Table 35-17: Sequence Number Generation and Verification Control (Outbound)

Header Processing SA_CMD0[19]	Anti-Replay Service SA_CMD1[29]	Description
Outbound		
1	1	Sequence number generation (increment)
1	1	Sequence number overflow check ($2^{32}-1$) to zero for IPsec, ($2^{48}-1$) to zero for DTLS and ($2^{64}-1$) to zero for Extended SSL/TLS.
1	0/1	Sequence number update in SA
0/1	0/1	Extended sequence number update in SA

Table 35-18: Sequence Number Generation and Verification Control (Inbound)

Header Processing SA_CMD0[19]	Anti-Replay Service SA_CMD1[29]	Description
Inbound		
1	1	Sequence number verification (check against sequence number and sequence number mask in SA). Applicable only for IPsec and DTLS.
1	0/1	Sequence number update in SA, except on authentication error
1	1	Sequence number mask update in SA. Applicable only for IPsec and DTLS.
0/1	0/1	Extended sequence number update in SA

Table 35-19: Header Processing Enabled, Anti-replay Service Enabled

	IPsec ESP	Ext. SSL	Ext. TLS	DTLS
Header processing enabled, anti-replay service enabled				
Inbound				
Initial value in the SA	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM0[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MSK	not used	not used	not used	not used
Value in the first packet or hashbyte stream, highest byte is B0	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0x00000000 B4 B7 0x00000000	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0xAA550000 B4 B7 0x00000001

Table 35-19: Header Processing Enabled, Anti-replay Service Enabled (Continued)

	IPsec ESP	Ext. SSL	Ext. TLS	DTLS
Header processing enabled, anti-replay service enabled				
Inbound				
Initial value in the SA after first packet	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM0[31:0]	0x00000001	0x00000001	0x00000001	0x00000001
SA_SEQNUM_MSK	not used	not used	not used	not used
Initial value in the SA	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0xFEDCBA98	0xFEDCBA98	0xFEDCBA98	0xAA55BA98
SA_SEQNUM0[31:0]	0x76543210	0x76543210	0x76543210	0x76543210
SA_SEQNUM_MSK	not used	not used	not used	not used
Value in the first packet or hashbyte stream, highest byte is B0	B0 B3 0xFEDCBA98 B4 B7 0x76543211	B0 B3 0xFEDCBA98 B4 B7 0x76543211	B0 B3 0xFEDCBA98 B4 B7 0x76543211	B0 B3 0xAA55BA98 B4 B7 0x76543211
Initial value in the SA after the packet	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM_1[31:0]	0xFEDCBA98	0xFEDCBA98	0xFEDCBA98	0xAA55BA98
SA_SEQNUM_0[31:0]	0x76543211	0x76543211	0x76543211	0x76543211
SA_SEQNUM_MASK	not used	not used	not used	not used
Highest value before overflow	B0 B3	B0 B3	B0 B3	B0 B3
SA_SEQNUM_1[31:0]	0xFFFFFFFF	0xFFFFFFFF	0xFFFFFFFF	0xAA55FFFF
SA_SEQNUM_0[31:0]	0xFFFFFFFF	0xFFFFFFFF	0xFFFFFFFF	0xFFFFFFFF
SA_SEQNUM_MASK	not used	not used	not used	not used
Value in the packet or hashbyte stream, highest byte is B0	B0 B3 0x00000000B4 B7 0x00000000	B0 B3 0xFFFFFFFF B4 B7 0xFFFFFFFF	B0 B3 0xFFFFFFFF B4 B7 0xFFFFFFFF	B0 B3 0xAA55FFFF B4 B7 0xFFFFFFFF
Value after overflow in SA	B0 B3	B0 B3	B0 B3	B0 B3
SA_SEQNUM_1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM_0[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MASK	not used	not used	not used	not used

Table 35-19: Header Processing Enabled, Anti-replay Service Enabled (Continued)

	IPsec ESP	Ext. SSL	Ext. TLS	DTLS
Header processing enabled, anti-replay service enabled				
Inbound				
Initial value in SA	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM0[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MSK1[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MSK0[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
Expected value in the first packet or hash-byte stream, highest byte is B0	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0x00000000 B4 B7 0x00000001	B0 B3 0xAA550000 B4 B7 0x00000001
Value in SA after first packet	B3 B0	B3 B0	B3 B0	B3 B0
SA_SEQNUM1[31:0]	0x00000000	0x00000000	0x00000000	0xAA550000
SA_SEQNUM0[31:0]	0x00000001	0x00000001	0x00000001	0x00000001
SA_SEQNUM_MSK1[31:0]	0x00000000	0x00000000	0x00000000	0x00000000
SA_SEQNUM_MSK0[31:0]	0x00000001	0x00000001	0x00000001	0x00000001

PKTE Block Diagram

The *PKTE Block Diagram* shows the functional blocks within the PKTE.

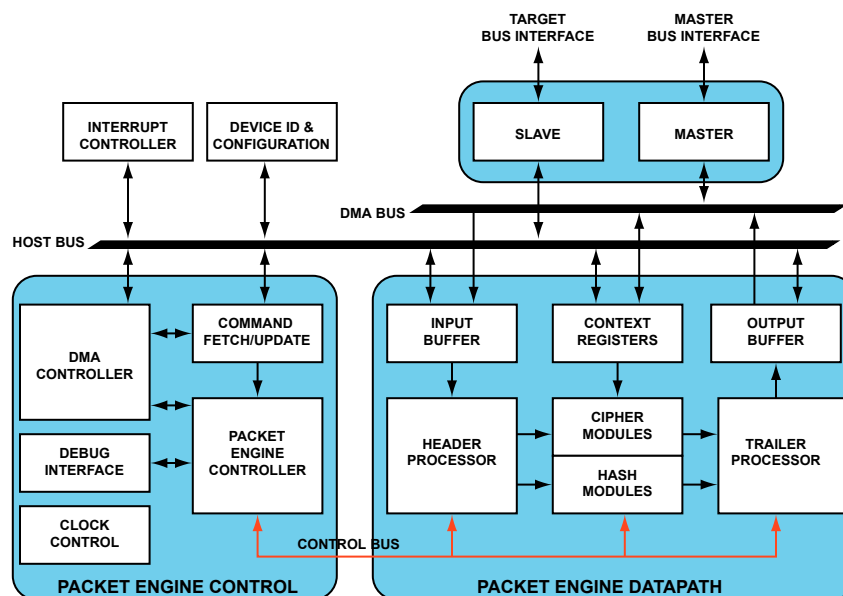


Figure 35-1: PKTE Block Diagram

PKTE Architectural Concepts

The following descriptions provide details on the functional blocks within the security packet engine.

Packet Engine

The packet engine contains symmetric cipher and hash engines. It is optimized to off-load intensive cryptographic operations from the host processor. It can perform parallel and pipelined encryption and hashing operations, reducing the latency, and processing time for packets that need both operations applied. The processor core provides the command information and packet data for the packet engine. The packet engine can run autonomously, using its local DMA controller to perform DMA transfers across the main bus to access the memory of the processor core. The DMA process incorporates flow-control to guarantee proper data flow. Two elements provide the command information that defines the processing for each packet:

- Command descriptor
- SA record

When the packet engine finishes the operation, it updates the SA record, when needed, and provides a result descriptor.

The packet engine has four different modes of operation. The modes give the processor core various levels of control over the command information and packet data transfers to and from the packet engine.

- Autonomous Ring Mode (ARM)
- Target Command Mode (with and without result descriptor ring) (TCM)
- Direct Host Mode (DHM)

Input/Output FIFO Buffers

The data for the packet engine is buffered at both input and output. These buffers decouple the DMA I/O process from the cipher and hash modules inside the packet engine. This functionality enables large DMA burst sizes and allows the crypto-engines to process data during I/O latency periods. Data moves automatically from the input buffer through the encryption and hash engines to the output buffer. If the output buffer is full, the process stops until the data is read and space is available in the output buffer. Each buffer is a 256-byte dual-port RAM.

Parallel Operations

The hash functionality and encrypt or decrypt functionality are tightly coupled. Operations that combine both hash and encrypt or decrypt functions are available to reduce processing time for data that must apply both. For hash-then-decrypt operations, the packet engine performs parallel execution of both functions from the input buffer. For encrypt-then-hash operations, the processing is pipelined from the input buffer to provide minimum latency. An offset can be specified between the start of the hashing and the start of the encryption to support protocols such as IPSec and SRTP.

DMA Controller

The packet engine uses a high-performance DMA controller for autonomous data transfers for:

- Command descriptor reads
- SA record and state record reads
- Packet data read
- Result packet writes
- SA record and state record writes
- Result descriptor writes

Interrupt Controller

The packet engine includes an interrupt controller that, under programmable configuration control, can generate an interrupt on completion of certain operations. Individual interrupts can be masked and cleared. The interrupt registers show both the raw and masked interrupt status of the internal interrupts. The processor core can use interrupts, together with their associated threshold settings, to optimize the overall packet processing in the system. One interrupt can inform the processor core that the input side of the packet engine is almost empty to avoid a stall-on-empty condition. One interrupt can inform the host that the output side is almost full to avoid a stall-on-full condition. The controller uses several interrupts to inform the processor core about errors inside the packet engine. All available interrupts are combined into a single output port as either a level- or edge-active programmable interrupt output.

Clock Controller

The packet engine includes a clock controller that generates clock enable signals. A clock manager external to the packet engine uses the clock enable signals to switch the clocks to modules in the packet engine, reducing power consumption. The power saving can be significant, depending on the crypto-operation and the idle time of the packet engine. The clock controller generates the clock enable signals dynamically depending on the current crypto-operation. A clock control register provides the processor core the possibility to override this dynamic process.

PKTE Operating Modes

The packet engine can be configured in one of three command modes. For all modes the packet engine can generate an interrupt at completion of packet processing:

- **Autonomous Ring Mode (ARM)**. The core prepares descriptors in the CDR and then initiates a descriptor fetch by triggering the packet engine. When a packet operation is complete, the packet engine writes the result descriptor out into a ring in host memory using the system master bus interface.
- **Target Command Mode (TCM)**. The core directly writes the command descriptors to the packet command register set to initiate a packet operation. This process eliminates an extra DMA transfer to fetch a descriptor, but requires the core to synchronously initiate packet processing. This mode can be configured both without RDR or with RDR. In the latter case, the packet engine writes the result descriptor out into the RDR in host memory using the system master bus interface.
- **Direct Host Mode (DHM)**. The core has full control over the packet engine and uses the system slave bus interface. The core provides all the command descriptors, SA record and state record, and packet input data.

When the packet engine completes processing, the core must read the packet output data, the SA record, the state record, and the result descriptors.

Autonomous Ring Mode (ARM)

The *Autonomous Ring Mode* allows the packet engine and the host processor to operate asynchronously. A queue of multiple packets in the host processor memory can be processed continuously to provide the highest possible throughput. The packet engine autonomously fetches the command descriptor, the SA record, and optionally the state record and the input data from host processor memory. After the packet engine finishes processing, it autonomously writes the output data, updates the SA record and state record and writes the result descriptor in the host processor memory. It accesses the host processor memory through DMA read transfers across the system bus master.

This mode uses both command descriptor ring (CDR) and result descriptor ring (RDR).

Physically the CDR, RDR, SA record, source packet, and result packet can all be in different memories depending on the system memory architecture. The host processor writes command descriptors to the CDR in host processor memory. Then, it writes to the `PKTE_CDSC_INCR` register with the number of command descriptors that it prepared in the CDR. This write to the `PKTE_CDSC_INCR` register is the trigger for the packet engine to fetch the command descriptors sequentially from the CDR. When a command descriptor is fetched and written to the internal packet command register set, the descriptor is validated. If the ownership bits are set for the packet engine and the command is valid, processing starts. If not, that command is discarded and a result descriptor is written to the RDR with the error code *invalid command descriptor*. In this mode, the host processor can set a threshold on the CDR and enable an associated interrupt. The packet engine generates an interrupt when the number of command descriptors in the CDR is equal or below the threshold value.

The SA record and state record that contains the crypto context information are stored in a memory area. The packet engine autonomously accesses the memory area through DMA transfers across the system bus master. Also, the source packet and result packet are stored in a memory area that the packet engine autonomously accesses through the same bus master interface.

After decoding the command descriptor, the packet engine fetches the SA record and then, optionally, the state record.

Then, the source packet is fetched and stored in the input buffer. Packets less than the size of the input buffers are fetched entirely at once. Larger packets are fetched in parts that completely fill the input buffer. The packet engine initiates a new fetch each time the number of empty spaces in the input buffer reaches its threshold value. When the first packet data is available in the input buffer, the crypto engines start processing the data. After processing, the crypto engines write the result packet to the output buffer.

The packet engine writes the result packet from the output buffer to host processor memory when the number of bytes in the output buffer reaches its threshold value. Packets less than the threshold value are written entirely at once. Larger packets are written in parts that completely empty the output buffer.

The source packet data fetching, data processing, and result packet data writing are parallel processes that continue until the last result packet is written to host processor memory. Then, the packet engine optionally writes the SA record and the state record to update the crypto context information. As a final step, the packet engine writes the

result descriptor to the RDR. The host processor must either poll the RDR or wait for an interrupt from the packet engine to determine when packet processing is complete.

Target Command Mode (TCM)

This mode provides a synchronous interface between the processor core and the packet engine. The Command Descriptor Ring (CDR) is disabled and the processor core initiates packet processing by writing the command descriptor directly to the internal command descriptor MMRs of the packet engine. The Result Descriptor Ring (RDR) is optional.

- For `PKTE_CFG.MODE = 01`, the RDR is disabled and the processor core reads the result descriptor directly from the internal result descriptor register set for the packet engine.
- For `PKTE_CFG.MODE = 10`, the RDR is enabled and stored in a memory area that the packet engine can access through its master bus interface as in autonomous ring mode.

In target command mode, the packet engine autonomously fetches the SA record, state record, source packet data as in autonomous ring mode. Also, as in ARM, after processing, the packet engine updates state fields of the SA record and state record in the host processor memory.

Direct Host Mode (DHM)

This mode provides a synchronous interface between the processor core and the packet engine. The packet engine is under full control of the processor core. The host processor writes the command descriptors, SA record, and state record directly to the packet engine registers. Then, the processor core writes the source packet data into the input buffer. When processing is complete, the processor core reads back the result packet data from the output buffer. Finally, it reads the result descriptor, the updated SA record, and state record directly from the packet engine registers.

PKTE Event Control

The following section provides information about interrupts in the PKTE module.

PKTE Interrupt Signals

The Packet Engine has an internal Interrupt Controller with 9 interrupt sources. There are 7 registers associated with the interrupt controller:

1. Interrupt Unmasked Status - [PKTE_IUMSK_STAT](#)
2. Interrupt Mask Status - [PKTE_IMSK_STAT](#)
3. Interrupt Clear Register - [PKTE_INT_CLR](#)
4. Interrupt Enable Register - [PKTE_INT_EN](#)
5. Interrupt Mask Disable - [PKTE_IMSK_DIS](#)
6. Interrupt Mask Enable - [PKTE_IMSK_EN](#)

7. Interrupt Configuration - `PKTE_INT_CFG`

The *Packet Engine Interrupt Controller Block Diagram* shows the blocks of the interrupt controller.

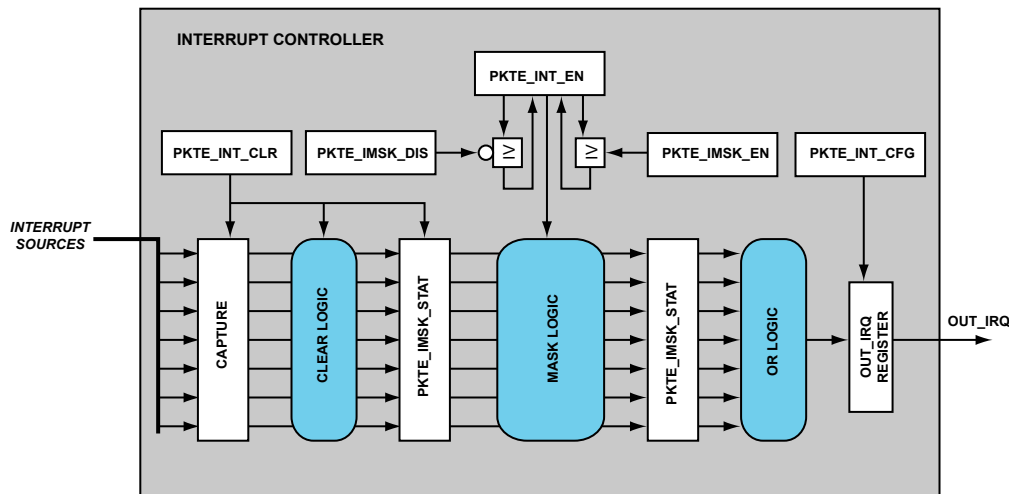


Figure 35-2: Packet Engine Interrupt Controller Block Diagram

All of the interrupt sources are pulse or level events in their native form.

These interrupts are captured and stored at their unmasked and masked status in their respective `PKTE_IUMSK_STAT` and `PKTE_IMSK_STAT` registers. This allows the host processor to read the status of any interrupt source either before or after the mask is applied.

The `PKTE_INT_EN` register provides a mask to select what interrupt source are enabled to the output interrupt request. Writing a one to the `PKTE_INT_CLR` register resets both the masked and unmasked interrupt.

The `PKTE_IMSK_EN` and `PKTE_IMSK_DIS` registers can be set to enable and disable individual interrupts respectively in the `PKTE_INT_EN` register. This avoids the need for read-modify-write operations from the host processor.

Ring Interrupts

Two interrupts are provided for efficient ring management: The CDR threshold interrupt (*cdrthrsh*) and the RDR threshold interrupt (*rdrthrsh*).

Command Descriptor Ring

The CDR threshold interrupt (*cdrthrsh*) is a level-based interrupt and connects to the threshold value in the `PKTE_RING_THRESH.CDRTHRSH` bit field. It enables the host processor to efficiently fill the CDR. The host processor writes command descriptors to the CDR with this interrupt masked until the CDR is full. Then the host processor enables the CDR threshold interrupt. When the interrupt is activated, the host processor clears the interrupt and it is guaranteed that it can put CDR threshold number of descriptors in the CDR.

Example Configuration:

```

PKTE_RING_CFG.RINGSZ=256,
PKTE_RING_THRESH.CDRTHRSH=224,
PKTE_INT_EN.CDRTHRSH=0 /*(IRQ disabled)*/

```

1. The host writes 8 Command Descriptors at once, then writes the [PKTE_CDSC_INCR](#) register to 8.
2. This is repeated until there are less than 8 empty entries in the CDR.
3. The fill level is now equal to the threshold (224)
4. The host enables the CDR threshold IRQ ([PKTE_INT_EN.CDRTHRSH=1](#))
5. The packet engine processes packets and the fill level ([PKTE_CDSC_CNT](#) register) decreases.
6. Then the CDR threshold IRQ is activated as the fill level equals 224.
7. The host handles the interrupt, clears it and continues with step 1.

Result Descriptor Ring

The RDR threshold interrupt ([rdrthrsh](#)) is a level-based interrupt and connects to the threshold value in the [PKTE_RING_THRESH.RDRTHRSH](#) bit field and the timeout value in the RD timeout ([PKTE_RING_THRESH.RDTO](#)) bit field. It enables the host processor to efficiently empty the RDR. The timeout reminds the host processor that when the threshold kicks in result descriptors stay long in the RDR and must be processed to reduce latency. The timeout counts when the ring is not empty, regardless of the fill level and restarts when the host processor writes the [PKTE_RDSC_CNT](#) register. Initially the host processor enables the RDR threshold interrupt. When the interrupt is activated the host processor reads result descriptors until the RDR is empty or contains less than the [PKTE_RING_THRESH.RDRTHRSH](#) number of descriptors.

Example Configuration:

```

PKTE_RING_CFG.RINGSZ=256,
PKTE_RING_THRESH.RDRTHRSH=32, timeout=1ms
PKTE_INT_EN.RDRTHRSH=1 /*(IRQ enabled)*/

```

1. The Packet Engine writes the Result Descriptor, timeout counter starts.
2. The Packet Engine writes 32 more Result Descriptors, fill level ([PKTE_RDSC_CNT](#) register) increases to 33.
3. The fill level exceeds threshold within 1ms, the RDR threshold IRQ is activated.
4. The host handles the interrupt, reads 8 Result Descriptors at once, then writes the [PKTE_RDSC_DECR](#) register with 8. The write to the [PKTE_RDSC_DECR](#) register restarts the timeout counter. The fill level is now under the threshold but there are still 25 descriptors left.
5. The Packet Engine writes 8 more Result Descriptors, fill level increases to 33.
6. The fill level exceeds threshold within 1 ms, the RDR threshold IRQ is activated.
7. The host handles the interrupt, reads 8 Result Descriptors at once, then writes the [PKTE_RDSC_DECR](#) register with 8. The write to the [PKTE_RDSC_DECR](#) register restarts the timeout counter. The fill level is now under the threshold but there are still 25 descriptors left.

8. After 1 ms, the timeout counter interrupt is activated.
9. The host handles the interrupt, reads 8 Result Descriptors at once, then writes the `PKTE_RDSC_DECR` register with 8. This is repeated until there are less than 8 full entries in the RDR. The fill level is now under the threshold and the RDR threshold IRQ interrupt is inactive. Each write to the `PKTE_RDSC_DECR` register restarts the timeout counter.

PKTE Programming Model

The host processor must always follow a pre-defined sequence of five phases required by the packet engine on a per packet basis when using direct host mode. The following sections describe the five phases.

Phase 1. Write the Command Descriptor

1. Write the first command descriptor word with status and control information to the `PKTE_CTL_STAT` register.
2. Optionally, write the user ID to the `PKTE_USERID` register.
3. Write the last descriptor word to the `PKTE_LEN` register.
4. Write the value 0x1 to the `PKTE_CDSC_CNT` register. This operation triggers the packet engine to validate the command descriptor. If the command descriptor is invalid, an error is generated. (See the `PKTE_CTL_STAT` section in the Register Descriptions). If the command descriptor is valid, the packet engine waits for an `PKTE_SA_RDY` register write.

Phase 2. Write the State Registers, (ARC4 Buffer) and SA Registers

All required fields of the SA record and state record must be written. The fields required depend on the operation. The last field to be written is the `PKTE_SA_RDY` register. This register triggers the packet engine to start processing.

1. Write the required state record fields.
 - ARC4 state
 - IV
 - Digest count
 - State digest
2. Write the required SA record data.
3. To complete the SA record and state record, write the `PKTE_SA_RDY` register.

Phase 3. Write the Source Packet Data and Read Result Packet Data

The packet engine has input and output buffers. If a source packet is smaller than the size of the input buffer, then the packet can be written in one part. Otherwise, it must be written in multiple parts. The same applies to the

output data. If the result packet size is smaller than the size of the output buffer, then the packet can be read in one part. Otherwise, it must be read in multiple parts.

NOTE: An outbound packet that is smaller than the size of the input buffer can increase in size due to padding and does not always fit in the output buffer. Conversely, an inbound packet that is larger than the size of the input buffer can decrease in size, and due to de-padding, can fit in the output buffer. If the input buffer becomes empty or the output buffer becomes full, the engine stalls.

Two following steps describe different situations:

- Source packet smaller than the size of the input buffer, start at step 1.
- Source packet larger than the size of the input buffer, start at step 3.

The host processor must follow these steps:

1. Write the source packet data. Write the full source packet to the input buffer. Go to step 4.
2. Write the input buffer count register (`PKTE_INBUF_CNT`) with the number of valid bytes that are written to the input buffer. This value must correspond to the value in the `PKTE_LEN.TOTLEN` field of the command descriptor rounded up to the next multiple of 4 bytes. Go to step 5 to check the packet engine status.
3. Write part of the source packet data. The `PKTE_STAT.IBUFEMPTYCNT` field indicates the amount of free space in the input buffer. Programs write the number of bytes determined by the setting in the `PKTE_BUF_THRESH` register. Write the (partial) source packet to the input buffer. The host processor must resume where it ended the previous write operation. Do not write more than the buffer size at once. Go to step 4.
4. Write the `PKTE_INBUF_CNT` register with the number of valid bytes written to the input buffer. Go to step 5 to check the packet engine status.
5. Check packet engine status. Wait for an interrupt or poll the `PKTE_STAT` register for any of the following conditions:
 - Condition 1 - An error interrupt or any of the bits [7:5] in the `PKTE_STAT` register becomes active to indicate a packet processing error. Depending on the type of error, the host processor must take appropriate action. Usually, the result packet is not valid after a processing error has occurred. Go to phase 5 to read the result descriptor.
 - Condition 2 - An operation done interrupt or the `PKTE_STAT.OPDN` bit becomes active (the packet engine completed processing). Go to step 8 to read the remaining output data.
 - Condition 3 - An output buffer threshold interrupt or the `PKTE_STAT.OBUFREQ` bit becomes active. Go to step 6 to read a block of output data.
 - Condition 4 - An input buffer threshold interrupt or the `PKTE_STAT.IBUFREQ` bit becomes active. Go to step 3 to write a block of input data.
6. Read part of the output data. The `PKTE_STAT.OBUFFULLCNT` bit field indicates the number of bytes in the output buffer. This value is rounded up to full words. Programs read the number of bytes indicated in the

`PKTE_BUF_THRESH` register. Read the (partial) output packet from the output buffer. The host processor must resume where it ended the previous read operation. Do not read more than the input buffer size at once. Go to step 7.

7. Write the `PKTE_OUTBUF_CNT` register with the number of valid bytes read from the output buffer. Go to step 5 to check the packet engine status.
8. Read the remaining output data. The `PKTE_STAT.OBUFFULLCNT` bit field indicates the number of bytes in the output buffer. This value is rounded up to words. Read the (partial) packet output data from the output buffer. The host processor must resume where it ended the previous read operation. Go to step 9.
9. Write the `PKTE_OUTBUF_CNT` register with the number of valid bytes read from the output buffer. Go to phase 4.

Phase 4. Read the Result Descriptor

1. Read the first result descriptor word from the `PKTE_CTL_STAT`.
2. Optionally read the user ID from the `PKTE_USERID` register.
3. Read the last result descriptor word from the `PKTE_LEN` register.
4. Write the value 0x1 to the `PKTE_RDSC_DECR` register. This operation allows the packet engine to accept new command descriptors. Go to phase 5.

Phase 5. Read the SA Record and State Record

Depending on the operation, the SA record or state record is updated. Check the bit fields [23:16] in the `PKTE_CTL_STAT` register for the following conditions:

- Condition 1 - At least one error bit in the bit fields [23:16] of the `PKTE_CTL_STAT` register is set. Do not update the local host processor maintained version of the SA record and state record but take any required action.
- Condition 2 - None of the error bits in the bit fields [23:16] of the `PKTE_CTL_STAT` register is set, the packet is processed normally (without errors). Update the host processor maintained version of the SA record and state record with the result read from the packet engine registers:
 - ARC4 state
 - Sequence number
 - Sequence number mask
 - Result IV
 - Result digest count
 - Result digest

PKTE Mode Configuration

Before using the packet engine, it must be configured. The mode of the packet engine must be defined and the PRNG (if used) must be initialized.

Configure the packet engine in one of three command modes:

- Autonomous Ring Mode: `PKTE_CFG.MODE = b'11`
- Target Command Mode: `PKTE_CFG.MODE = b'10`
- Direct Host Mode: `PKTE_CFG.MODE = b'00`

PKTE Programming Concepts

The following sections provide conceptual information for programming the PKTE.

Packet Engine Descriptor

IMPORTANT: Depending on the mode, ARM, TCM, or DHM, the descriptor is either:

- in the memory of the host processor in the command descriptor ring, or
- written directly to the descriptor registers in the packet engine

References to descriptor registers are for either the register that is mirrored in the descriptor structure in memory or for the actual register itself.

Command descriptors are host-supplied commands that control the real-time operation of the packet engine. The packet engine returns result descriptors at the end of an operation that provide the status information to the host. The *Command Descriptor Structure* and the *Result Descriptor Structure* tables show these descriptors.

Table 35-20: Command Descriptor Structure

Word Off- set	31:24	23:20		19:16	15:8	7:0	Address Offset
0	Pad Control	—			Next Header/ Pad Value	Control	0x000
1	Source Address						0x004
2	Destination Address						0x008
3	SA Address						0x00C
4	SA State Address						0x010
5	Reserved/ARC4 State Address						0x014
6	User ID						0x018
7	Bypass (words)	Control	Reserved	Input Packet Length (bytes)			0x01C

Table 35-21: Result Descriptor Structure

Word Off- set	31:24	23:20		19:16	15:8	7:0	Address Offset
0	Pad Status	Status			Next Header/ Pad Value	Control	0x000
1	Source Address						0x004
2	Destination Address						0x008
3	SA Address						0x00C
4	SA State Address						0x010
5	Reserved/ARC4 State Address						0x014
6	User ID						0x018
7	Bypass (words)	Control	Reserved	Input Packet Length (bytes)			0x01C

When the packet engine is configured for autonomous ring mode, command descriptors and result descriptors reside in a ring in host memory. Command descriptors are automatically fetched from the Command Descriptor Ring (CDR) through DMA into the command descriptor registers. When an operation is complete, the result descriptors are automatically read from the packet engine and through DMA to the Result Descriptor Ring (RDR).

When the packet engine is configured for direct host mode, the host processor manually writes the command descriptor directly to the internal command descriptor MMR set. When an operation is complete, the host processor manually reads the result descriptor directly from the result descriptor MMR set.

The target command mode is a combination of the direct host mode and the autonomous ring mode. The host processor writes the command descriptor directly to the internal command descriptor register set. When an operation is complete there are two options.

1. The host processor can read the result descriptor directly from the result descriptor registers.
2. The result descriptors reside in a ring in host memory and the result descriptors are automatically DMA'd from the packet engine to the RDR.

When the host processor writes a command descriptor to the command descriptor registers, the packet engine is triggered when the host processor updates the [PKTE_CDSC_CNT](#) register. This functionality guarantees that all fields in the command descriptor are valid before the command is executed.

Descriptor Processing

This section describes the functional steps of the packet engine while processing the command descriptors.

Descriptor Ring Configuration

At initialization, the host processor specifies the size of the Command Descriptor Ring (CDR). The Result Descriptor Ring (RDR) has the same size.

NOTE: In some configurations, these two rings overlay each other with the results written on top of the command descriptors. This configuration is called overlaid ring mode.

When the packet engine is configured and enabled, it fetches the descriptors from the CDR using system bus master reads.

Descriptor Ring Processing

To validate the descriptor exchange between the host processor and the packet engine, the ownership bits `PKTE_CTL_STAT.PERDY` and `PKTE_CTL_STAT.HOSTRDY` are used. One pair of ownership bits is in the first word of the descriptor (`PKTE_CTL_STAT`), and one pair is in the last word (`PKTE_LEN`). The ‘consumer’ of a descriptor must verify that both ownership pairs match to ensure that a race condition did not occur between one party writing and the other party reading the descriptor. A race condition can occur when a memory locking scheme is not used.

Each pair [`PKTE_CTL_STAT.PERDY`, `PKTE_CTL_STAT.HOSTRDY`] of ownership bits provide 3 states:

- b'00 = idle or null descriptor
- b'01 = host processor has written a descriptor in the CDR and passed ownership to the packet engine
- b'10 = packet engine processing complete: packet engine has written the descriptor in the RDR and passed ownership back to the host.
- b'11 = Reserved

At initialization, the host sets the entire CDR memory area to zero, when the CDR is used.

1. The host processor writes one or more command descriptors to the CDR. The host processor must set the `PKTE_CTL_STAT.HOSTRDY` bit to 1 and the `PKTE_CTL_STAT.PERDY` bit to 0 to indicate that ownership has passed to the packet engine. These bits are mirrored in the `PKTE_LEN` descriptor word.
2. The host processor must write the `PKTE_CDSC_INCR` register with the number of new valid command descriptors in the CDR.
3. The packet engine reads and validates one command descriptor.
4. The packet engine reads the SA record and state record, processes the packet and updates the SA record and state record.
5. If the rings are not overlaid and the `PKTE_CFG.ENCDRUPDT` bit is 1, the packet engine writes the result descriptor to the CDR with the `PKTE_CTL_STAT.HOSTRDY` bit set to 0 and `PKTE_CTL_STAT.PERDY` bit set to 1. These bits are mirrored in the `PKTE_LEN` descriptor word.
6. The packet engine writes the result descriptor to the RDR. The packet engine sets the `PKTE_CTL_STAT.PERDY` bit to 1 and the `PKTE_CTL_STAT.HOSTRDY` bit to 0 to indicate that the ownership has passed to the host. These bits are mirrored in the `PKTE_LEN` descriptor word.
7. The packet engine decrements the value in the `PKTE_CDSC_CNT` register. If the value is not zero, the packet engine reads with the next command descriptor (step 3).

8. The packet engine increments the value in the `PKTE_RDSC_CNT` register.
9. The host processor reads one or more result descriptors from the RDR and processes the results.
10. The host processor must write the `PKTE_RDSC_DECR` register with the number of processed result descriptors in the RDR.

Descriptor Ownership

The ownership of the command descriptor and result descriptor is set by ownership bits in the first and last word of the respective descriptor, in the `PKTE_CTL_STAT` register and the `PKTE_LEN` register. For the command descriptor, it is the processor core that sets the ownership to the packet engine. For the result descriptor, it is the packet engine that sets the ownership to the host processor.

The packet engine reads the ownership bits before processing, irrespective of the mode of the packet engine. The ownership bits are used to validate and identify the descriptor. When two separate rings are used, the packet engine can be programmed to clear the ownership bits of the command descriptors in the CDR so the host processor knows which descriptors are processed.

NOTE: This update of the ownership bits can be disabled in the `PKTE_CFG` register when the host processor actively counts the number of descriptors in the CDR to prevent ring wrapping.

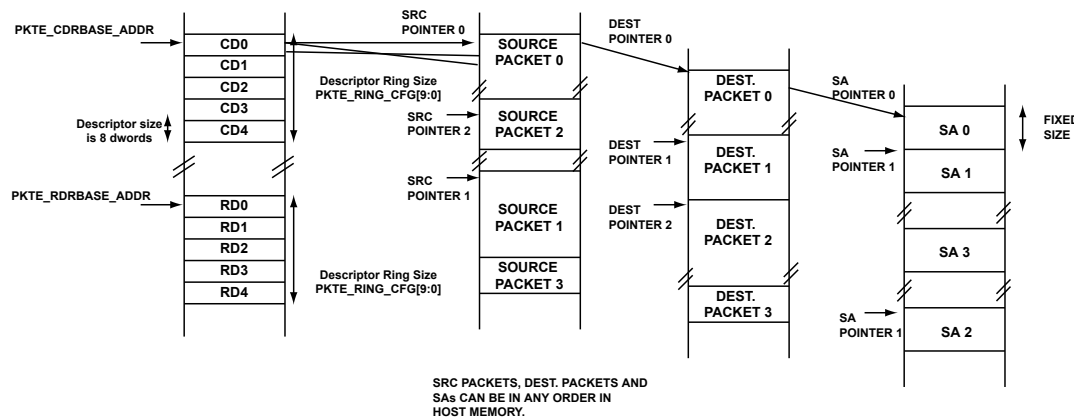


Figure 35-3: Descriptor Rings in Autonomous Ring Mode

SA Record and State Record Structure

The SA record is a packed structure that contains the remainder of the information needed by the packet engine to process a packet. Most of the information fields in the SA record, such as the key and encryption mode, are static for the lifetime of the association. The fields do not require frequent manipulation by the host processor. The SA record non-static fields are the sequence number and sequence number mask.

The SA record can have a corresponding state record that is used to save results from the current operations that can be used for future operations. The state record can hold the IV, the hash byte count, and the intermediate hash digest.

If an SA record is used for operations that use ARC4 processing in stateful mode, it has a corresponding ARC4 state record that holds the ARC4 State.

In this manual, the state record and the ARC4 state record are referred to as state record.

There is no practical limit to how many SA records and corresponding state records the packet engine can support.

In the autonomous ring mode and target command mode, once the packet engine has validated a command descriptor, it automatically fetches the SA record and optional state record. After processing, the packet engine updates the stateful fields in the SA record and state record in the host processor memory.

In direct host mode, after the descriptor is validated, the host must write the SA record directly into the internal registers of the packet engine. After processing, the host reads the stateful fields from the SA registers of the packet engine and saves them back to the SA record in the host processor memory.

SA Record Structure

The *SA Record Structure* table shows the structure for an SA Record. When using direct host mode, the corresponding elements are accessed directly with the registers. When using autonomous ring mode or target command mode, the SA Record is defined, configured and accessed in host memory.

Table 35-22: SA Record Structure

Word Offset	Description (name)	Use
0	PKTE_SA_CMD0[31:0]	SA Control word 0 (all operations)
1	PKTE_SA_CMD1[31:0]	SA Control word 1 (all operations)
2	PKTE_SA_KEY0[31:0]	Key word (DES, Triple-DES, AES-128/192/256, ARC4,)
3	PKTE_SA_KEY1[63:32]	Key word (DES, Triple-DES, AES-128/192/256, ARC4,)
4	PKTE_SA_KEY2[95:64]	Key word (Triple-DES, AES-128/192/256, ARC4,)
5	PKTE_SA_KEY3[127:96]	Key word (Triple-DES, AES-128/192/256, ARC4,)
6	PKTE_SA_KEY4[159:128]	Key word (Triple-DES, AES-192/256)
7	PKTE_SA_KEY5[191:160]	Key word (Triple-DES, AES-192/256)
8	PKTE_SA_KEY6[223:192]	Key word (AES-256)
9	PKTE_SA_KEY7[255:224]	Key word (AES-256)
10	PKTE_SA_IDIGEST0[31:0]	Inner Hash digest (Basic Hash and HMAC with MD5,SHA-1, SHA-224, SHA-256)
11	PKTE_SA_IDIGEST1[63:32]	Inner Hash digest (Basic Hash and HMAC with MD5,SHA-1, SHA-224, SHA-256)
12	PKTE_SA_IDIGEST2[95:64]	Inner Hash digest (Basic Hash and HMAC with MD5,SHA-1, SHA-224, SHA-256)
13	PKTE_SA_IDIGEST3[127:96]	Inner Hash digest (Basic Hash and HMAC with MD5,SHA-1, SHA-224, SHA-256)

Table 35-22: SA Record Structure (Continued)

Word Offset	Description (name)	Use
14	PKTE_SA_IDIGEST4[159:128]	Inner Hash digest (Basic Hash and HMAC with SHA-1, SHA-224, SHA-256)
15	PKTE_SA_IDIGEST5[191:160]	Inner Hash digest (Basic Hash and HMAC with SHA-224, SHA-256)
16	PKTE_SA_IDIGEST6[223:192]	Inner Hash digest (Basic Hash and HMAC with SHA-224, SHA-256)
17	PKTE_SA_IDIGEST7[255:224]	Inner Hash digest (Basic Hash and HMAC with SHA-256)
18	PKTE_SA_ODIGEST0[31:0]	Outer Hash digest (HMAC with MD5, SHA-1, SHA-224, SHA-256)
19	PKTE_SA_ODIGEST1[63:32]	Outer Hash digest (HMAC with MD5, SHA-1, SHA-224, SHA-256)
20	PKTE_SA_ODIGEST2[95:64]	Outer Hash digest (HMAC with MD5, SHA-1, SHA-224, SHA-256)
21	PKTE_SA_ODIGEST3[127:96]	Outer Hash digest (HMAC with MD5, SHA-1, SHA-224, SHA-256)
22	PKTE_SA_ODIGEST4[159:128]	Outer Hash digest (HMAC with SHA-1, SHA-224, SHA-256)
23	PKTE_SA_ODIGEST5[191:160]	Outer Hash digest (HMAC with SHA-224, SHA-256)
24	PKTE_SA_ODIGEST6[223:192]	Outer Hash digest (HMAC with SHA-224, SHA-256)
25	PKTE_SA_ODIGEST7[255:224]	Outer Hash digest (HMAC with SHA-256)
26	PKTE_SA_SPI[31:0]	SPI (IPsec), Type[23:16] / Version [15:0] (SSL, TLS, DTLS)
27	PKTE_SA_SEQNUM0[31:0]	Sequence Number (IPsec, SSL, TLS, DTLS with Header Processing)
28	PKTE_SA_SEQNUM1[63:32]	
29	PKTE_SA_SEQNUM_MSK0[31:0]	Sequence Number Mask (IPsec, DTLS inbound with Header Processing)
30	PKTE_SA_SEQNUM_MSK1[63:32]	
31	PKTE_SA_NONCE[31:0] / PKTE_SA_READY	Nonce value (AES-CTR, AES-ICM)/ARC4, i and j pointers (ARC4,)/SA ready indicator (Direct Host Mode)

Some of these fields may be updated by the packet engine. These include:

- PKTE_SA_SEQNUM0
- PKTE_SA_SEQNUM1
- PKTE_SA_SEQNUM_MSK0

- PKTE_SA_SEQNUM_MSK1

All the other fields remain unchanged.

SA State Structure

The security association state structure contains information that may be updated after each packet, such as the IV and the intermediate hash result. The *SA State Structure* table shows the SA state structure and usage. In direct host mode, the elements are accessed directly using the PKTE registers. In target command mode and autonomous ring mode, this structure is defined and updated in host memory.

Table 35-23: SA State Structure

Word Offset	Description (name)	Use
0	PKTE_STATE_IV0[31:0]	Initialization Vector (DES, Triple DES, AES)
1	PKTE_STATE_IV1[63:32]	Initialization Vector (DES, Triple DES, AES)
2	PKTE_STATE_IV2[95:64]	Initialization Vector (AES)
3	PKTE_STATE_IV3[127:96]	Initialization Vector (AES)
4	PKTE_STATE_BYTE_CNT0[31:0]	Current hash byte count (MD5, SHA-1, SHA-224, SHA-256)
5	PKTE_STATE_BYTE_CNT1[63:32]	Current hash byte count (MD5, SHA-1, SHA-224, SHA-256)
6	PKTE_STATE_IDIGEST0[31:0]	Inner Hash digest (mirror of PKTE_SA_IDIGEST0)
7	PKTE_STATE_IDIGEST1[63:32]	Inner Hash digest (mirror of PKTE_SA_IDIGEST1)
8	PKTE_STATE_IDIGEST2[95:64]	Inner Hash digest (mirror of PKTE_SA_IDIGEST2)
9	PKTE_STATE_IDIGEST3[127:96]	Inner Hash digest (mirror of PKTE_SA_IDIGEST3)
10	PKTE_STATE_IDIGEST4[159:128]	Inner Hash digest (mirror of PKTE_SA_IDIGEST4)
11	PKTE_STATE_IDIGEST5[191:160]	Inner Hash digest (mirror of PKTE_SA_IDIGEST5)
12	PKTE_STATE_IDIGEST6[223:192]	Inner Hash digest (mirror of PKTE_SA_IDIGEST6)
13	PKTE_STATE_IDIGEST7[255:224]	Inner Hash digest (mirror of PKTE_SA_IDIGEST7)

ARC4 State Structure

The *ARC4 State Structure* table describes the state structure used with ARC4. When using the PKTE in direct host mode, these fields are accessed with the registers, starting at the value in the [PKTE_ARC4STATE_BUF](#) register. When using the PKTE in autonomous ring mode or target command mode, these fields are defined and accessed in a structure in host memory.

Table 35-24: ARC4 State Structure

Word Offset	Description (name) ^{*1}	Use
0	PKTE_ARC4_STATE0[3:0] ^{*2}	ARC4 (Basic, SSL and TLS)
1	PKTE_ARC4_STATE1[7:4]	
...	...	
62	PKTE_ARC4_STATE2[251:248]	
63	PKTE_ARC4_STATE3[255:252]	

^{*1} The indices in these fields indicate bytes.

^{*2} There are no corresponding named registers for these fields. PKTE_ARC4_STATE0 corresponds to [PKTE_ARC4STATE_BUF](#) and PKTE_ARC4_STATE1 corresponds to the 32-bit register following [PKTE_ARC4STATE_BUF](#) and so on.

Configuring Operations in the PKTE

The operation (cipher, hash function, and others) that the PKTE performs is configured primarily in the [PKTE_SA_CMD0](#) register. The following sections include a series of tables to help configure the least significant 16 bits of the [PKTE_SA_CMD0](#) register. These fields include:

- The operation code field ([PKTE_SA_CMD0.OPCD](#))
- The direction field ([PKTE_SA_CMD0.DIR](#))
- The operation group field ([PKTE_SA_CMD0.OPGRP](#))
- The padding type ([PKTE_SA_CMD0.PADTYPE](#))
- The cipher selection ([PKTE_SA_CMD0.CIPHER](#))
- The hash selection ([PKTE_SA_CMD0.HASH](#))

Basic Operations and Decoding

Table 35-25: Basic Operation Decoding

Outbound				Inbound			
OpGroup	Dir	OpCode	Operation	OpGroup	Dir	OpCode	Operation
0b00	0	0b000	Encrypt	0b00	1	0b000	Decrypt
0b00	0	0b001	Encrypt - Hash	0b00	1	0b001	Hash - Decrypt
0b00	0	0b010	Reserved	0b00	1	0b010	Reserved
0b00	0	0b011	Hash	0b00	1	0b011	Hash
0b00	0	0b100... 0b110	Reserved	0b00	1	0b100... 0b110	Reserved
0b00	0	0b111	PRNG	0b00	1	0b111	Reserved

Table 35-26: Protocol Operation Decoding

Outbound				Inbound			
OpGroup	Dir	OpCode	Operation	OpGroup	Dir	OpCode	Operation
0b01	0	0b000	ESP Outbound	0b01	1	0b000	ESP Inbound
0b01	0	0b001... 0b011	Reserved	0b01	1	0b001	Reserved
0b01	0	0b100	Basic SSL Out-bound	0b01	1	0b010	Basic SSL In-bound
0b01	0	0b101	Basic TLS Out-bound	0b01	1	0b011	Basic TLS In-bound
0b01	0	0b110	Reserved	0b01	1	0b100... 0b110	Reserved
0b01	0	0b111	SRTP Out-bound	0b01	1	0b111	SRTP Inbound

NOTE: For SSL/TLS and SRTP, no header processing is performed in hardware.

Table 35-27: Extended Protocol Operation Decoding

Outbound				Inbound			
OpGroup	Dir	OpCode	Operation	OpGroup	Dir	OpCode	Operation
0b11 0	0	0b000	Reserved	0b11	1	0b000	Reserved
0b11 0	0	0b001	DTLS Out-bound	0b11	1	0b001	DTLS Inbound
0b11 0	0	0b010... 0b011	Reserved	0b11	1	0b010... 0b011	Reserved
0b11 0	0	0b100	Ext. SSL Out-bound	0b11	1	0b100	Ext. SSL In-bound
0b11 0	0	0b101	Ext. TLS v1.0 Outbound	0b11	1	0b101	Ext. TLS v1.0 Inbound
0b11 0	0	0b110	Ext. TLS v1.1 Outbound	0b11	1	0b110	Ext. TLS v1.1 Inbound
0b11 0	0	0b111	Reserved	0b11	1	0b111	Reserved

Error Code Description

The `PKTE_CTL_STAT` register is used to configure the packet engine for processing in Direct Host Mode (DHM) or Target Command Mode (TCM). The `PKTE_CTL_STAT` structure element in memory is used when the packet

engine is configured for Autonomous Ring Mode (ARM). In both cases, when an operation is started, errors are reported in the status field (bits [23:16]) of this register or structure element. The *Extended Error Codes - Status Encoding* table provides a guide on how to decipher the meaning of the bits that are set when an error occurs.

Extended Error Codes

The following table provides information about the extended errors associated with the PKTE module.

Table 35-28: Extended Error Codes - Status Encoding

STATUS bits [23:16]	Hex Value	Priority	Description	Processing Result
0b0000_0000	0x00	NA	Successful completion. No errors occurred during processing of the packet.	Packet fully processed
0b----_---1	0x-1	NA	Authentication Error. For an inbound IPsec ESP operation, the Integrity Check Value (ICV) does not match the computed value. For an inbound SRTP operation, the authentication tag does not match the computed value. For a basic SSL/TLS, Extended SSL/TLS or DTLS operation the Message Authentication Code (MAC) does not match the computed value.	Packet fully processed
0b----_--1-	0x-2	NA	Pad Verify Error. For inbound operations that use pad type Constant TLS, IPsec or PKCS#7, the decrypted pad does not match the expected values for the selected pad type.	Packet fully processed
0b----_-1--	0x-4	NA	Sequence Number Error. For an inbound IPsec or DTLS operation, there was a fault in the Anti-Replay Sequence Number. For an outbound IPsec packet, the sequence number overflows; count is $2^{32}-1$ and increments to 0. For an outbound DTLS operation, the sequence number overflows; count is $2^{48}-1$ and increments to 0. For an outbound SSL or TLS operation, the sequence number overflows; count is $2^{64}-1$ and increments to 0.	Packet fully processed
0b0000_1---	0x08	1	System Bus error. The master bus interface generates an error due to ERROR response from system slave. The slave bus interface generates an error due to request for non-word (32-bit) access.	Packet is aborted. The host must reject the packet and apply a hardware reset to the system.

Table 35-28: Extended Error Codes - Status Encoding (Continued)

STATUS bits [23:16]	Hex Value	Priority	Description	Processing Result
0b0001_1---	0x18	2	Invalid Command Descriptor Error. The ownership bits in the command descriptor are not set to the packet engine, after the <code>PKTE_CDSC_CNT</code> register is incremented.	Command descriptor is ignored, no packet is processed. The packet must be re-queued or discarded.
0b0010_1---	0x28	3	Invalid Crypto Operation Error. A reserved operation is selected.	The SA record is ignored, no packet is processed. The packet must be re-queued or discarded.
0b0011_1---	0x38	4	Invalid Crypto Algorithm Error. A reserved cipher is selected, refer to <code>PKTE_SA_CMD0.CIPHER</code> . A reserved hash is selected, refer to <code>PKTE_SA_CMD0.HASH</code> .	The SA record is ignored, no packet is processed. The packet must be re-queued or discarded.
0b0100_1---	0x48	5	SPI Error. On an inbound packet, the 32-bit SPI value in the packet does not match the value in the SA while header processing is enabled. Note: A failure caused by an SPI mismatch, in general should not occur because the host checks the SPI and does not send an incorrect SPI to the packet engine.	Packet is fully processed. The host must reject the packet.
0b0101_1---	0x58	3	Zero Length Error. The packet length defined in the command descriptor <code>PKTE_LEN.TOTLEN</code> is zero, which is illegal.	Packet command is ignored, no packet processed. The host must reject the packet.

Table 35-28: Extended Error Codes - Status Encoding (Continued)

STATUS bits [23:16]	Hex Value	Priority	Description	Processing Result
0b0110_1xxx	0x68	6	Invalid Packet Length Error. For Basic Encrypt-Hash and Hash-Decrypt operations: $PKTE_LEN.TOTLEN < \text{Hash/Encrypt Offset}$ For IPsec ESP inbound operations: $PKTE_LEN.TOTLEN < \text{ICV length}$ or $PKTE_LEN.TOTLEN$ is non-4 byte aligned For SRTP inbound operations: $PKTE_LEN.TOTLEN \leq \text{IV (opt.)} + \text{Bypass Offset} + \text{ROC}$ For SSL inbound operations: $PKTE_LEN.TOTLEN \leq 1$ or packet length > 65535 bytes (SSL packet-bypass length) For TLS and DTLS inbound operations: $PKTE_LEN.TOTLEN \leq 13$ or payload length > 65535 bytes (data to be hashed) Note: For IPsec ESP the ICV is stripped before the length is checked.	Packet processing is aborted. Result packet length is zero. The host must reject the packet and apply a software reset of the packet engine.
0b0111_1xxx	0x78	7	Block Size Error. The length of the inbound packet defined in the Command Descriptor $PKTE_LEN.TOTLEN$ is not a multiple of the DES or AES block cipher length. For outbound packets the size is always automatically aligned (padded) to the correct block size. The hashed packet length is not a multiple of the hash block size for intermediate hash operation. For a final hash operation no error is generated. Note: For IPsec ESP operations the ICV is stripped before the block size is checked.	Packet is fully processed. The host must reject the packet.
0b1000_1xxx	0x88	8	Processing Error. The number of bytes in the input buffer is more than defined in the $PKTE_LEN.TOTLEN$ field. The number of bytes written to the output buffer is less than processed in the datapath.	Packet processing aborted. Result packet length is zero. The host must reject the packet and apply a software reset.
0b1010_1xxx 0b1111_1xxx	Reserved			

Number Format

When dealing with cryptographic functions, data and keys are large vectors. For instance, AES supports keys of sizes 128, 192, and 256 bits. When a key needs to be loaded or read, multiple 32-bit key registers are used, namely

`PKTE_SA_KEY[n]` registers. The first key register `PKTE_SA_KEY[n] 0` holds bits 31:0, and `PKTE_SA_KEY[n] 1` holds the next thirty two bits 63:32, and so on.

Generally, for large vectors defined as Byte0, Byte1, Byte2, Byte3 and so on, the values are stored in the PKTE registers as `PKTE_REG0 = Byte3Byte2Byte1Byte0`, followed by `PKTE_REG1 = Byte7Byte6Byte5Byte4` and so on.

PKTE Programming Examples

Use these examples to extend your understanding of PKTE features, operating modes, event control, and programming modes.

Calculating SHA in Direct Host Mode

This section describes how to configure the packet engine to calculate a hash digest using one of supported SHA algorithms in direct host mode. This configuration follows the procedure outlined in the *PKTE Programming Model* section.

1. Configure the packet engine for direct host mode by setting the `PKTE_CFG.MODE` bit =0
2. Set the ownership back to the packet engine to process a command descriptor by setting the `PKTE_CTL_STAT.PERDY` bit =0 and the `PKTE_CTL_STAT.HOSTRDY` bit =1.
3. Set the `PKTE_CTL_STAT.HASHFINAL` bit to indicate this command descriptor handles all the input data for the hash calculation. This configuration is needed for the packet engine because the last block requires special handling (see FIPS 180-4 for details).
4. Set size of the input data in bytes in the `PKTE_LEN.TOTLEN` bit field.
5. Also set the `PKTE_LEN.PEDONE` bit =0 and the `PKTE_LEN.HSTRDY` bit =1. These bits must be the same as the `PKTE_CTL_STAT.PERDY` and `PKTE_CTL_STAT.HOSTRDY` bits to guarantee ownership.
6. Set the `PKTE_CDSC_CNT` register =1 to trigger the packet engine to start validating the command descriptor. In this case, the `PKTE_CTL_STAT`, `PKTE_LEN` and `PKTE_CDSC_CNT` registers are the only command descriptor registers modified.
7. Configure the `PKTE_SA_CMD0` and `PKTE_SA_CMD1` registers to define the operation. For an SHA, set the `PKTE_SA_CMD0.OPCD` bit field =0b011 for hash operation and the `PKTE_SA_CMD0.OPGRP` bit field =0b00 for basic operation.
8. Select the specific SHA function using the `PKTE_SA_CMD0.HASH` bit field as follows.
 - For SHA-1, `PKTE_SA_CMD0.HASH` =0b0001
 - SHA-224, `PKTE_SA_CMD0.HASH` =0b0010
 - for SHA-256, `PKTE_SA_CMD0.HASH` =0b0011
9. Depending on the SHA selected, the appropriate digest length must be chosen for the `PKTE_SA_CMD0.DIGESTLEN` bit field as follows.

- For SHA-1, `PKTE_SA_CMD0.DIGESTLEN = 0b0101` (5 words)
 - For SHA-224, `PKTE_SA_CMD0.DIGESTLEN = 0b0111` (7 words)
 - For SHA-256, `PKTE_SA_CMD0.DIGESTLEN = 0b1000` (8 words)
10. The SHA specifies initial constants. These constants can be pre-loaded or read from memory. In this example, by setting the `PKTE_SA_CMD0.HASHSRC` bit field = 0b11, the packet engine provides the correct initial constants depending on the SHA chosen.
 11. Next, set the `PKTE_SA_CMD1.CPYDGST` bit = 1 and `PKTE_SA_CMD1.CPYPAD` bit = 1 to move the result to the output buffer of the packet engine at the `PKTE_DATAIO_BUF` location.
 12. At this point, write to the `PKTE_SA_RDY` register with any value to trigger the operation.
 13. Start writing the input to the data buffer of the packet engine starting at the `PKTE_DATAIO_BUF` location.
 14. Write the `PKTE_INBUF_CNT` register with the length of the input rounded up to the next multiple of 4. For example, if the input length is 30 bytes, set this register to 32.
 15. Poll the `PKTE_STAT` register to see if any errors occurred or if the operation completed without errors.

Once the operation is done, the digest is available in the packet engine data I/O buffer.

NOTE: The input data or message is input into the packet engine data buffer in big endian format while the result or digest is little endian format.

Performing AES Decryption in Direct Host Mode

This section describes how to configure the packet engine to decrypt using AES-128 in direct host mode. This configuration follows the procedure outlined in the *PKTE Programming Model* section.

1. Configure the packet engine for direct host mode by setting the `PKTE_CFG.MODE` bit = 0
2. Start configuring the command descriptor registers. Set the ownership back to the packet engine to process a command descriptor by setting the `PKTE_CTL_STAT.PERDY` bit = 0 and the `PKTE_CTL_STAT.HOSTRDY` bit = 1.
3. Next, configure the `PKTE_LEN.TOTLEN` bit field with the size of the packet or message to decrypt. If the entire input message (cipher text) fits into the 256-byte data I/O buffer of the packet engine, the process can be done in one shot.
4. Set the `PKTE_LEN.PEDONE` bit = 0 and the `PKTE_LEN.HSTRDY` bit = 1. These bits must have the same setting as the `PKTE_CTL_STAT.PERDY` and `PKTE_CTL_STAT.HOSTRDY` bits to guarantee ownership.
5. Set the `PKTE_CDSC_CNT` register = 1 to trigger the packet engine to start validating the command descriptor. In this case, the `PKTE_CTL_STAT`, `PKTE_LEN`, and `PKTE_CDSC_CNT` registers are the only command descriptor registers modified.
6. Next, configure the `PKTE_SA_CMD0` and `PKTE_SA_CMD1` registers to define the operation.

- For a AES decrypt inbound cipher operation, set the `PKTE_SA_CMD0.OPCD` bit field =0b000 and the `PKTE_SA_CMD0.DIR` bit field =0b1.
 - Set the `PKTE_SA_CMD0.OPGRP` bit field =0b00 for basic operation.
 - To choose the AES cipher, set the `PKTE_SA_CMD0.CIPHER` bit field =0b0011. Set the `PKTE_SA_CMD0.HASH` bit field to 0b1111 to choose the NULL function.
7. Next, set the `PKTE_SA_CMD1.AESKEYLEN` bit field to select the appropriate key length. In this case, setting it to 0b10 select 128 bits. Also, set `PKTE_SA_CMD1.CIPHERMD` bit field to select the mode. In this case, setting it to 0b01 select CBC mode.
 8. Continue configuring the Security Association (SA) record by loading the key in the `PKTE_SA_KEY[n]` registers.
 9. Next load the initialization vector in the SA state registers (`PKTE_STATE_IV[n]`).
 10. Finally, write anything in to the `PKTE_SA_RDY` register to trigger the operation.
 11. The input data can now be written into the data I/O buffer starting at `PKTE_DATAIO_BUF`.
 12. After the data is written, write the length (or next multiple of 4) into `PKTE_INBUF_CNT` register.
 13. Poll `PKTE_STAT` to see if any errors occurred or if the operation completed without errors.

Once the operation is done, the result can be found in the same data I/O buffer.

ADSP-2159x PKTE Register Descriptions

Security Packet Engine (PKTE) contains the following registers.

Table 35-29: ADSP-2159x PKTE Register List

Name	Description
<code>PKTE_ARC4STATE_ADDR</code>	Packet Engine ARC4 State Record Address
<code>PKTE_ARC4STATE_BUF</code>	Starting Entry of 256-byte ARC4 State Buffer
<code>PKTE_BUF_PTR</code>	Packet Engine Buffer Pointer Register
<code>PKTE_BUF_THRESH</code>	Packet Engine Buffer Threshold Register
<code>PKTE_CDRBASE_ADDR</code>	Packet Engine Command Descriptor Ring Base Address
<code>PKTE_CDSC_CNT</code>	Packet Engine Command Descriptor Count Register
<code>PKTE_CDSC_INCR</code>	Packet Engine Command Descriptor Count Increment Register
<code>PKTE_CFG</code>	Packet Engine Configuration Register
<code>PKTE_CLK_CTL</code>	PE Clock Control Register
<code>PKTE_CONT</code>	PKTE Continue Register
<code>PKTE_CTL_STAT</code>	Packet Engine Control Register

Table 35-29: ADSP-2159x PKTE Register List (Continued)

Name	Description
PKTE_DATAIO_BUF	Starting Entry of 256-byte Data Input/Output Buffer
PKTE_DEST_ADDR	Packet Engine Destination Address
PKTE_DMA_CFG	Packet Engine DMA Configuration Register
PKTE_ENDIAN_CFG	Packet Engine Endian Configuration Register
PKTE_HLT_CTL	Packet Engine Halt Control Register
PKTE_HLT_STAT	Packet Engine Halt Status Register
PKTE_IMSK_DIS	Interrupt Mask Disable Register
PKTE_IMSK_EN	Interrupt Mask Enable Register
PKTE_IMSK_STAT	Interrupt Masked Status Register
PKTE_INBUF_CNT	Packet Engine Input Buffer Count Register
PKTE_INBUF_INCR	Packet Engine Input Buffer Count Increment Register
PKTE_INT_CFG	Interrupt Configuration Register
PKTE_INT_CLR	Interrupt Clear Register
PKTE_INT_EN	Interrupt Enable Register
PKTE_IUMSK_STAT	Interrupt Unmasked Status Register
PKTE_LEN	Packet Engine Length Register
PKTE_OUTBUF_CNT	Packet Engine Output Buffer Count Register
PKTE_OUTBUF_DECR	Packet Engine Output Buffer Count Decrement Register
PKTE_RDRBASE_ADDR	Packet Engine Result Descriptor Ring Base Address
PKTE_RDSC_CNT	Packet Engine Result Descriptor Count Registers
PKTE_RDSC_DECR	Packet Engine Result Descriptor Count Decrement Registers
PKTE_RING_CFG	Packet Engine Ring Configuration
PKTE_RING_PTR	Packet Engine Ring Pointer Status
PKTE_RING_STAT	Packet Engine Ring Status
PKTE_RING_THRESH	Packet Engine Ring Threshold Registers
PKTE_SA_ADDR	Packet Engine SA Address
PKTE_SA_ARC4IJPTR	ARC4 i and j Pointer Register
PKTE_SA_CMD0	SA Command 0
PKTE_SA_CMD1	SA Command 1
PKTE_SA_IDIGEST[n]	SA Inner Hash Digest Registers
PKTE_SA_KEY[n]	SA Key Registers

Table 35-29: ADSP-2159x PKTE Register List (Continued)

Name	Description
PKTE_SA_NONCE	SA Initialization Vector Register
PKTE_SA_ODIGEST[n]	SA Outer Hash Digest Registers
PKTE_SA_RDY	SA Ready Indicator
PKTE_SA_SEQNUM[n]	SA Sequence Number Register
PKTE_SA_SEQNUM_MSK[n]	SA Sequence Number Mask Registers
PKTE_SA_SPI	SA SPI Register
PKTE_SRC_ADDR	Packet Engine Source Address
PKTE_STAT	Packet Engine Status Register
PKTE_STATE_ADDR	Packet Engine State Record Address
PKTE_STATE_BYTE_CNT[n]	State Hash Byte Count Registers
PKTE_STATE_IDIGEST[n]	State Inner Digest Registers
PKTE_STATE_IV[n]	State Initialization Vector Registers
PKTE_USERID	Packet Engine User ID

Packet Engine ARC4 State Record Address

The `PKTE_ARC4STATE_ADDR` register holds the start address of the SA ARC4 state record.

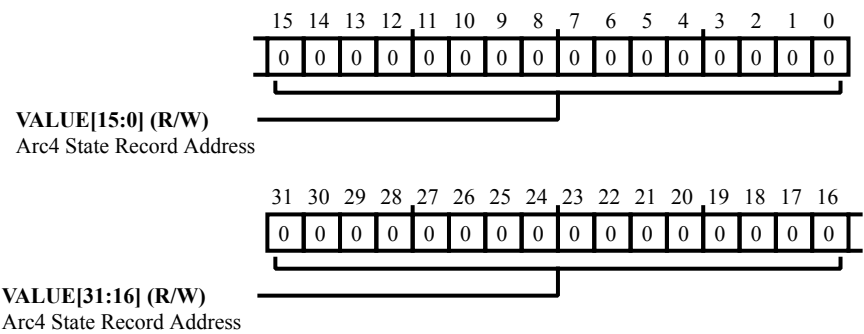


Figure 35-4: `PKTE_ARC4STATE_ADDR` Register Diagram

Table 35-30: `PKTE_ARC4STATE_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Arc4 State Record Address.

Starting Entry of 256-byte ARC4 State Buffer

The `PKTE_ARC4STATE_BUF` register is used to store the pre-processed key that initializes the ARC4 module. In direct host mode, before processing starts, the Host must write the ARC4 state, starting from the base address and increment the address pointer for each write. When processing completes the Host must read the ARC4 state and copy it to the local Host maintained state record. After a reset, a read from any address in the address range of the ARC4 buffer returns an undefined value.

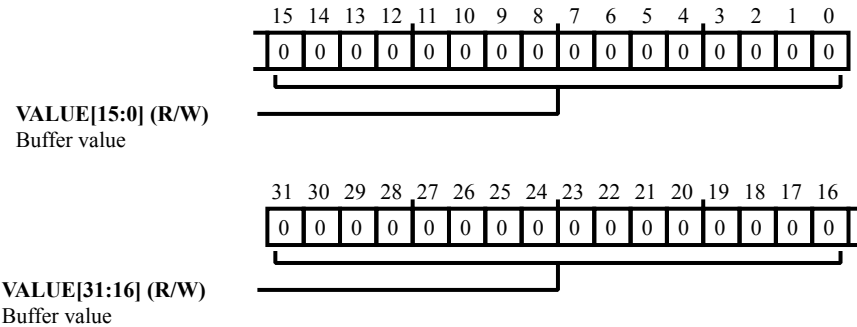


Figure 35-5: `PKTE_ARC4STATE_BUF` Register Diagram

Table 35-31: `PKTE_ARC4STATE_BUF` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Buffer value. The <code>PKTE_ARC4STATE_BUF.VALUE</code> bit field stores the pre-processed key that initializes the ARC4 module.

Packet Engine Buffer Pointer Register

The `PKTE_BUF_PTR` register contains the offset of the next buffer address (entry) to be read or written by the packet engine. This register is used in direct host mode only.

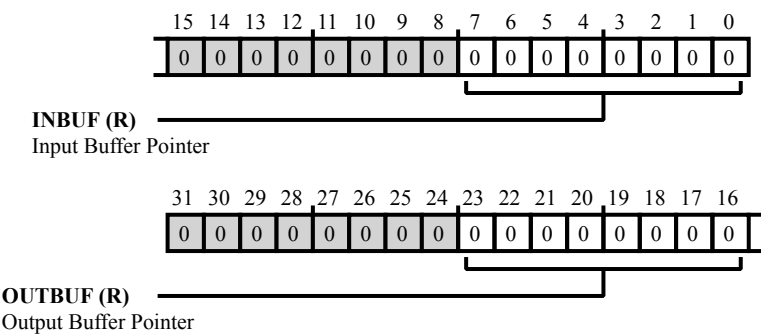


Figure 35-6: `PKTE_BUF_PTR` Register Diagram

Table 35-32: `PKTE_BUF_PTR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/NW)	OUTBUF	Output Buffer Pointer. The <code>PKTE_BUF_PTR.OUTBUF</code> bit field indicates the offset of the next address (entry) in the output buffer that will be written next by the packet engine. This bit field is reset to zero after starting up and decremented by 4 at every output buffer write operation. Pointers wrap around; the maximum value this field can have equals the output buffer size minus 4.
7:0 (R/NW)	INBUF	Input Buffer Pointer. The <code>PKTE_BUF_PTR.INBUF</code> bit field indicates the offset of the next address (entry) in the input buffer that will be read next by the packet engine. The bit field is reset to zero after starting up and incremented by 4 at every input buffer read operation. Pointers wrap around; the maximum value this field can have equals the input buffer size minus 4.

Packet Engine Buffer Threshold Register

When in autonomous ring mode or target command mode, the `PKTE_BUF_THRESH` register defines the high- and low-level value at which the packet engine starts to transfer packet data in or out of the internal packet buffers. These parameters can be used to control the DMA burst size for packet data input and output from the packet engine. In direct host mode, this register contains both threshold values to reduce the amount of packet engine interrupts.

The input buffer threshold (`ibufthrsh`) interrupt indicates that the input buffer counter is less than or equal to the input buffer threshold value set in this register - this interrupt can be used to wake up a process that stalled on a full input buffer.

The output buffer threshold (`obufthrsh`) interrupt indicates that the output buffer counter exceeds the output buffer threshold setting. The output buffer interrupt remains active until the output buffer counter is decremented to zero again.

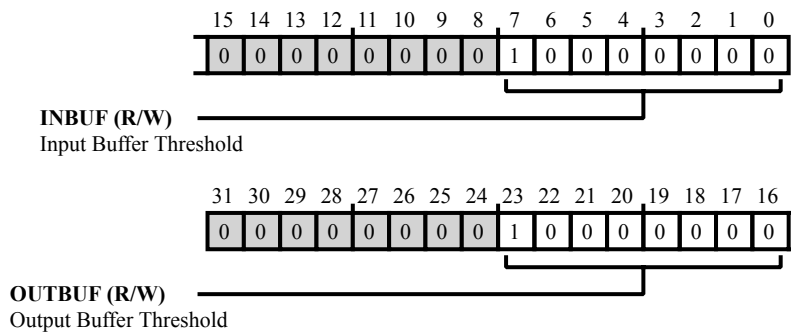


Figure 35-7: `PKTE_BUF_THRESH` Register Diagram

Table 35-33: `PKTE_BUF_THRESH` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	OUTBUF	Output Buffer Threshold. The <code>PKTE_BUF_THRESH.OUTBUF</code> bit field specifies how many bytes must be available in the packet engine output buffer before an output transfer starts. Valid values range from 0 to 252, in multiples of 4. In autonomous ring mode, a value of 128 generally gives a good performance, but the optimal value depends on the system and application. In direct host mode, the output buffer threshold (<code>obufthrsh</code>) interrupt activates when the output buffer counter for the output buffer exceeds the value set in this field. A value of 128 generally gives a good performance, but the optimal value depends on the system and application.

Table 35-33: PKTE_BUF_THRESH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	INBUF	Input Buffer Threshold. The <code>PKTE_BUF_THRESH.INBUF</code> bit field specifies how many bytes must be free in the packet engine input buffer before an input transfer starts. Valid values range from 0 to 252, in multiples of 4. In autonomous ring mode, a value of 128 generally gives a good performance, but the optimal value depends on the system and application. In direct host mode, the input buffer threshold (<code>ibufthrsh</code>) interrupt activates when the input buffer counter for the input buffer is below or equal the value set in this field. A value of 128 generally gives a good performance, but the optimal value depends on the system and application.

Packet Engine Command Descriptor Ring Base Address

The `PKTE_CDRBASE_ADDR` register holds the command descriptor ring base address in host memory. It is only applicable in autonomous ring mode. The `PKTE_CDRBASE_ADDR` register is ignored for all other modes when command descriptors are directly written into the descriptor registers.

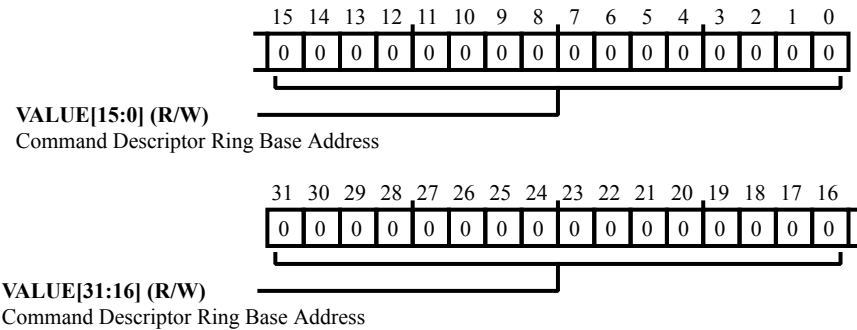


Figure 35-8: `PKTE_CDRBASE_ADDR` Register Diagram

Table 35-34: `PKTE_CDRBASE_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command Descriptor Ring Base Address. The <code>PKTE_CDRBASE_ADDR.VALUE</code> bit field specifies the base location of the command descriptor ring in the host memory space.

Packet Engine Command Descriptor Count Register

The `PKTE_CDSC_CNT` register holds the counter for the number of descriptors in the Command Descriptor Ring (CDR). It is decremented by the packet engine each time a valid descriptor is read from the CDR and processed.

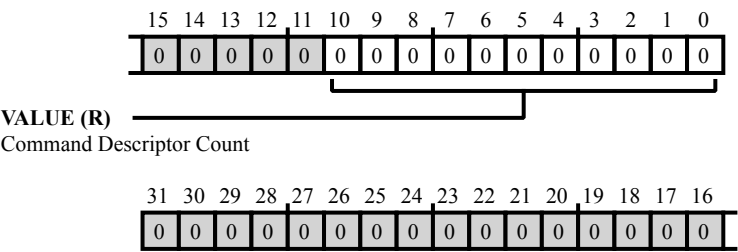


Figure 35-9: PKTE_CDSC_CNT Register Diagram

Table 35-35: PKTE_CDSC_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/NW)	VALUE	Command Descriptor Count. The <code>PKTE_CDSC_CNT.VALUE</code> bit field provides the number of command descriptors in the command descriptor ring. The packet engine decrements the counter when a valid command descriptor is read from the CDR and processed.

Packet Engine Command Descriptor Count Increment Register

The `PKTE_CDSC_INCR` register is accessible by the host connected through the system slave bus. The host can increment the command descriptor counter by writing a value between 1 and 255 to the lowest byte of this register.

In autonomous ring mode, the host must prepare 1 to 255 valid command descriptors in the CDR and then write this register with a value between 1 and 255. The write triggers the packet engine to fetch the command descriptors from the CDR. In direct host mode or target command mode, the host must write one valid command descriptor to the internal descriptor registers and then write this register with the value 1, to indicate that one valid descriptor is available.

A CDR threshold interrupt is activated when the command descriptor counter is less than or equal to the threshold value set in the `PKTE_RING_THRESH` register. This interrupt can be used to wake up a process that stalled on a full CDR.

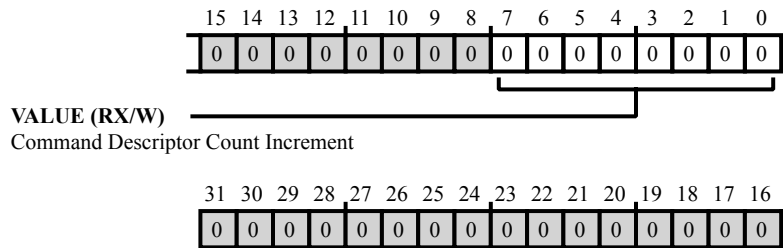


Figure 35-10: `PKTE_CDSC_INCR` Register Diagram

Table 35-36: `PKTE_CDSC_INCR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (RX/W)	VALUE	Command Descriptor Count Increment. The value written to the <code>PKTE_CDSC_INCR.VALUE</code> bit field is added to the command descriptor counter. The counter is protected against overflow (see the <code>PKTE_RING_STAT</code> register description). Note that bits[10:8] should be written with zeros.

Packet Engine Configuration Register

The `PKTE_CFG` register is used to select static settings that control the packet-processing path. This register is typically the last one to be written during the initialization sequence. These settings are typically set at initialization and not changed again.

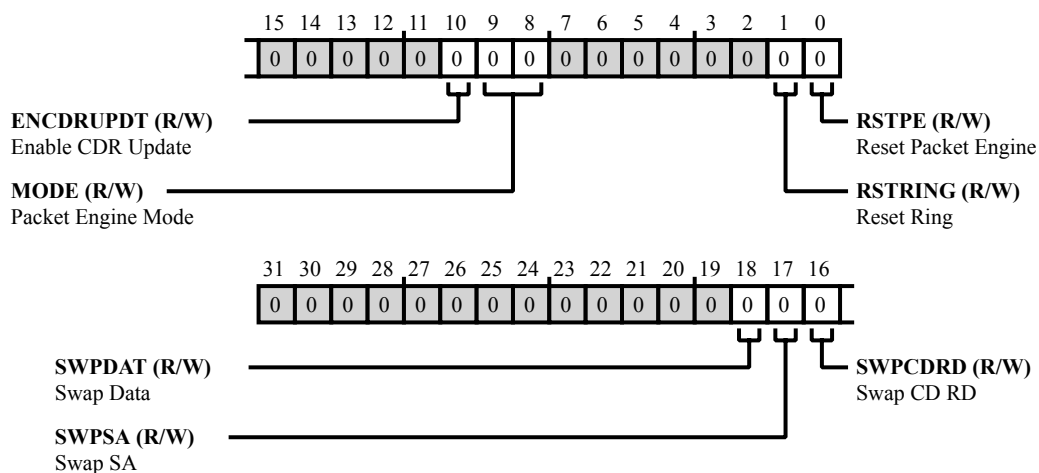


Figure 35-11: PKTE_CFG Register Diagram

Table 35-37: PKTE_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	SWPDAT	Swap Data. The <code>PKTE_CFG.SWPDAT</code> bit enables endian swap for packet data as configured in the <code>PKTE_ENDIAN_CFG.MSTRBSWP</code> bits for the packet data DMA read and write.
		0 No Endian Swap
		1 Apply Endian Swap
17 (R/W)	SWPSA	Swap SA. The <code>PKTE_CFG.SWPSA</code> bit enables endian swap for a SA record as configured in the <code>PKTE_ENDIAN_CFG.MSTRBSWP</code> bits for the SA record and state record DMA read and write. If the <code>PKTE_ENDIAN_CFG.MSTRBSWP</code> bits specify no endian swap, this bit is ignored.
		0 No Endian Swap
		1 Apply Endian Swap

Table 35-37: PKTE_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
16 (R/W)	SWPCDRD	Swap CD RD. The PKTE_CFG.SWPCDRD bit enables endian swap for descriptors as configured in the PKTE_ENDIAN_CFG.MSTRBSWP bits for the command descriptor DMA read and result descriptor DMA write. If the PKTE_ENDIAN_CFG.MSTRBSWP bits specify no endian swap, this bit is ignored.
		0 No Endian Swap
		1 Apply Endian Swap
10 (R/W)	ENCDRUPDT	Enable CDR Update. The PKTE_CFG.ENCDRUPDT bit enables the packet engine to update, (clear the ownership bits) in the command descriptor in the CDR.
		0 Do Not Clear Ownership Bits. The packet engine does not clear the ownership bits in the command descriptor when it completes an operation. The host application must clear the ownership bits in "old descriptors" before the packet engine is allowed to wrap around the CDR to re-encounter these "old descriptors". This setting has the advantage of eliminating a separate DMA write to the CDR.
		1 Clear Ownership Bits. The packet engine clears (set to zero) the ownership bits in the current command descriptor in the CDR. This prevents the packet engine from re-processing an "old descriptor" when it wraps around the CDR.
9:8 (R/W)	MODE	Packet Engine Mode. The PKTE_CFG.MODE bit field selects how the packet engine receives commands.
		0 Direct Host Mode.
		1 Target Command Mode with Result Descriptor Ring Disabled.
		2 Target Command Mode with Result Descriptor Ring Enabled.
		3 Autonomous Ring Mode

Table 35-37: PKTE_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	RSTRING	Reset Ring. The PKTE_CFG.RSTRING bit resets the internal counters for the CDR and RDR, PKTE_CDSC_CNT and PKTE_RDSC_CNT registers) to zero. Resets the PKTE_RING_PTR register to the base address. After the reset the rings are empty. This bit must be written with a '1' to reset the descriptor ring manager and then re-written with a '0' to release the reset. Note that this bit can remain in the reset state if the CDR ring is disabled (PKTE_CFG.MODE is not 0b11). Note that this reset must be coordinated with the 'owner' of the descriptor ring to ensure that the pointers are in sync after the reset.
		0 Release the Descriptor Ring Manager Reset
		1 Reset the Descriptor Ring Manager
0 (R/W)	RSTPE	Reset Packet Engine. The PKTE_CFG.RSTPE bit resets the packet engine and the state machine logic that drives header processing, DMA, and context management. The PKTE_CFG.RSTPE bit resets the PKTE_CTL_STAT and PKTE_LEN internal registers. This bit must be written with a 1 to reset the packet engine and then re-written with a 0 to release the reset. Note that this bit should not be used by a typical application. It is provided to use during development testing or to recover from critical errors. Note that the PKTE_CTL_STAT.PADVAL and PKTE_CTL_STAT.PADCTLSTAT bit fields are only reset when in autonomous ring mode, but the PKTE_CTL_STAT.PRNGMD bit is not reset. Halt mode is not affected by this reset as well. When exiting out of halt mode, a HW reset is required or a write to the PKTE_CONT register.
		0 Release the Packet Engine Reset
		1 Reset the Packet Engine

PE Clock Control Register

The `PKTE_CLK_CTL` register controls the clock enable signals. This register can be used to enable the clock for read and write access to SA registers or to enable the required clock signals for certain crypto functions. The setting of this register overrides the packet engine dynamic clock enable.

In autonomous ring mode and target command modes, this register can be all zeros; the packet engine dynamically requests the external clock manager to activate the module clocks. This register can be used in combination with the debugging interface for internal register access.

In direct host mode, the clock enable bits for the packet engine (`PKTE_CLK_CTL.ENPECLK`) and for ARC4 (`PKTE_CLK_CTL.ENARC4CLK`) must be enabled to write and read the SA record and state record registers. All module clocks that are required for the current operation must be enabled during processing.

Note that all the clocks are enabled by default to reset all the registers within the packet engine. After a system reset the host can program this register to disable clocks for power reduction.

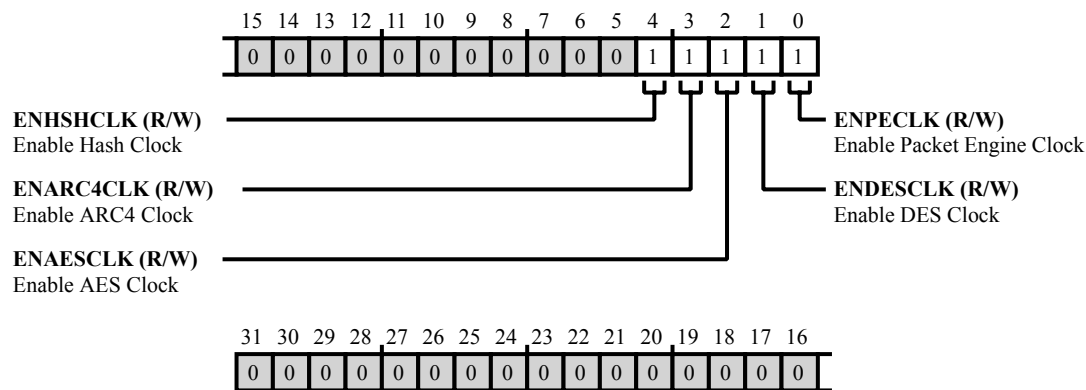


Figure 35-12: PKTE_CLK_CTL Register Diagram

Table 35-38: PKTE_CLK_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	ENHSHCLK	Enable Hash Clock. The <code>PKTE_CLK_CTL.ENHSHCLK</code> bit enables the clock to the hash functions.
		0 Do not enable the hash clock
		1 Enable the hash clock
3 (R/W)	ENARC4CLK	Enable ARC4 Clock. The <code>PKTE_CLK_CTL.ENARC4CLK</code> bit enables the clock to the ARC4 function.
		0 Do not enable the ARC4 clock
		1 Enable the ARC4 clock

Table 35-38: PKTE_CLK_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
2 (R/W)	ENAESCLK	Enable AES Clock. The PKTE_CLK_CTL.ENAESCLK bit enables the clock to the AES encrypt/decrypt function.	
		0	Do not enable the AES clock
		1	Enable the AES clock
1 (R/W)	ENDESCLK	Enable DES Clock. The PKTE_CLK_CTL.ENDESCLK bit enables the clock to the DES function.	
		0	Do not enable the DES clock
		1	Enable the DES clock
0 (R/W)	ENPECLK	Enable Packet Engine Clock. The PKTE_CLK_CTL.ENPECLK bit enables the clock in the PKTE data path.	
		0	Do not enable the PKTE data path clock
		1	Enable the PKTE data path clock

PKTE Continue Register

A write to the `PKTE_CONT` register (with any value) releases the packet engine from a halt state when in halt mode.

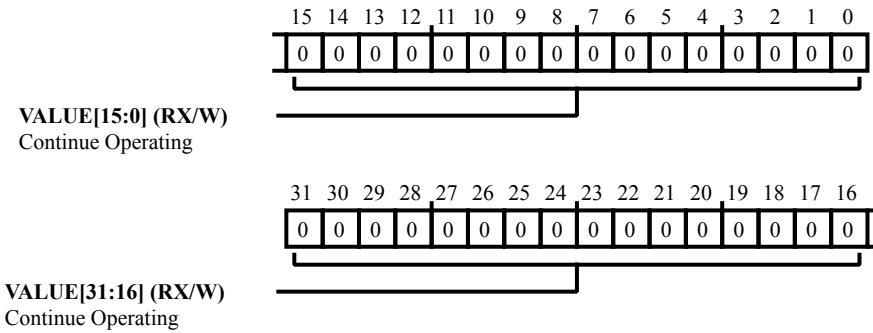


Figure 35-13: PKTE_CONT Register Diagram

Table 35-39: PKTE_CONT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Continue Operating. The <code>PKTE_CONT.VALUE</code> bit field releases the packet engine from a halt state when written with any value in halt mode.

Packet Engine Control Register

The `PKTE_CTL_STAT` register has a dual function. Together with the data in the SA, this register provides the basic command information for the packet engine to process a packet. When the packet engine successfully or unsuccessfully completes an operation, the packet engine control/status register provides the result status for the host.

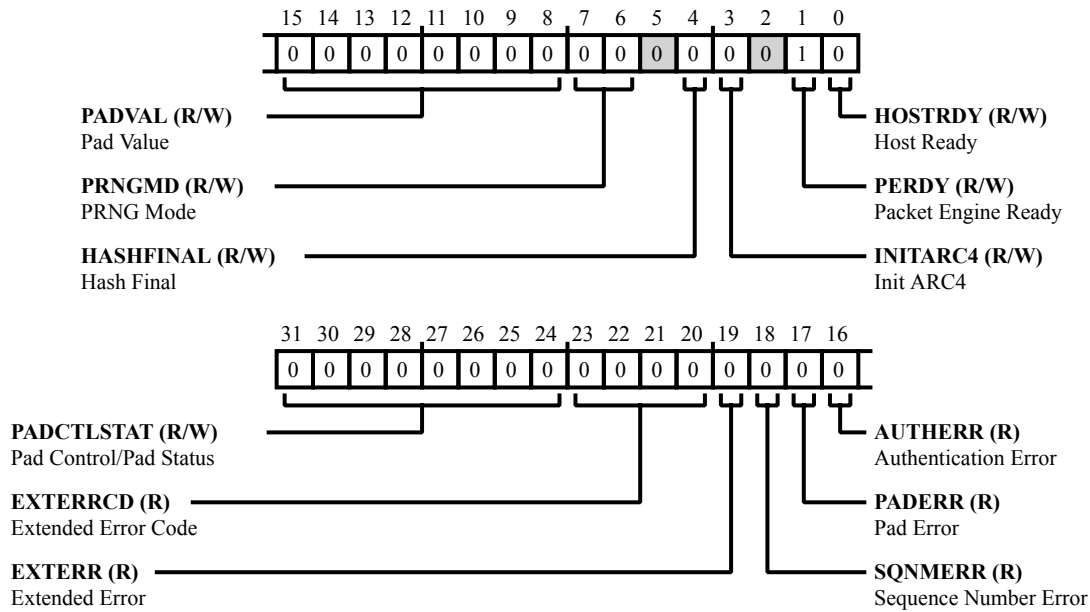


Figure 35-14: PKTE_CTL_STAT Register Diagram

Table 35-40: PKTE_CTL_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	PADCTLSTAT	Pad Control/Pad Status. The <code>PKTE_CTL_STAT.PADCTLSTAT</code> bit field is used to control the pad boundary for pad insertion (outbound) and after processing returns the number of inserted (outbound) or detected (inbound) pad bytes. For the command descriptor, the enumerations below provide the codes for the pad boundary for the outbound operations. This can be used for traffic flow security to conceal the number of payload bytes in an encrypted packet. For the result descriptor inbound operations that use pad types SSL, TLS, IPsec or PKCS#7, it returns the number of detected pad bytes. For all other inbound operations, it returns zero since the other pad modes do not allow implicit determination of pad count. If a pad verify failure occurs, it returns zero. For an outbound operation, it returns the number of inserted pad bytes for all pad types. The pad value includes added bytes such as the pad length and the next header field in an IPsec ESP pad type.
		0 Align packet end to modulo 8-byte boundary
		1 Align packet end to modulo 1-byte boundary

Table 35-40: PKTE_CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		2	Align packet end to modulo 4-byte boundary
		4	Align packet end to modulo 8-byte boundary
		8	Align packet end to modulo 16-byte boundary
		16	Align packet end to modulo 32-byte boundary
		32	Align packet end to modulo 64-byte boundary
		64	Align packet end to modulo 128-byte boundary
		128	Align packet end to modulo 256-byte boundary
23:20 (R/NW)	EXTERRCD	Extended Error Code. The PKTE_CTL_STAT.EXTERRCD bit field represents an encoded error condition.	
19 (R/NW)	EXTERR	Extended Error. The PKTE_CTL_STAT.EXTERR bit field provides an extended error code.	
		0	No Extended Error
		1	Extended Error
18 (R/NW)	SQNMERR	Sequence Number Error. The PKTE_CTL_STAT.SQNMERR bit indicates that for an inbound operation, there was a fault in the anti-replay sequence number. For an outbound operation, there was a sequence number overflow condition.	
		0	No Sequence Number Error
		1	Sequence Number Error
17 (R/NW)	PADERR	Pad Error. The PKTE_CTL_STAT.PADERR bit indicates that for an inbound operation the decrypted pad does not match the expected values.	
		0	No Pad Error
		1	Pad Error
16 (R/NW)	AUTHERR	Authentication Error. The PKTE_CTL_STAT.AUTHERR bit indicates that for an inbound operation the authentication value in the packet does not match the computed value.	
		0	No Authentication Error
		1	Authentication Error

Table 35-40: PKTE_CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration								
15:8 (R/W)	PADVAL	Pad Value. The <code>PKTE_CTL_STAT . PADVAL</code> bit field is used to pass the pad value between the host and the packet engine. Command Descriptor: (write-only) For outbound operations that use pad type IPsec, the host must populate this field with the value that is to be inserted into the next header field. For the IPsec ESP operation, this next header is part of the ESP trailer of the innermost operation's header and the value must be 50 decimal. For outbound encrypt operations that use the pad type constant or constant SSL, the host must specify the fixed constant value in this field. For all other outbound and inbound operations, this field is not used. Result Descriptor: (read only) For inbound operations that use pad type IPsec, the packet engine returns the next header field that it detects. For IPsec ESP inbound operations, this is the next header field in the innermost operation's header, which will typically be the value for the payload protocol, such as TCP or UDP. However, in bundling scenarios or in IPv6 with destination option headers, another header value could be seen. For all other outbound operations, the packet engine will not update this field. For all other inbound operations, the returned pad value is zero.								
7:6 (R/W)	PRNGMD	PRNG Mode. The <code>PKTE_CTL_STAT . PRNGMD</code> bits select the pseudo-random number generator mode. <table><tr><td>0</td><td>Operation does not use the PRNG function. Operation does not use the PRNG function.</td></tr><tr><td>1</td><td>PRNG Init. PRNG is initialized with a SEED, KEY and an LFSR value as defined in the SA.</td></tr><tr><td>2</td><td>PRNG Generate. Pseudo-random data is generated with the LFSR as input value. Before this mode can be used, the PRNG must be initialized with a valid SEED, KEY and LFSR using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).</td></tr><tr><td>3</td><td>PRNG Test. It can be used to test the PRNG function with custom input data. Before this mode can be used, the PRNG must be initialized once with a valid SEED using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).</td></tr></table>	0	Operation does not use the PRNG function. Operation does not use the PRNG function.	1	PRNG Init. PRNG is initialized with a SEED, KEY and an LFSR value as defined in the SA.	2	PRNG Generate. Pseudo-random data is generated with the LFSR as input value. Before this mode can be used, the PRNG must be initialized with a valid SEED, KEY and LFSR using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).	3	PRNG Test. It can be used to test the PRNG function with custom input data. Before this mode can be used, the PRNG must be initialized once with a valid SEED using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).
0	Operation does not use the PRNG function. Operation does not use the PRNG function.									
1	PRNG Init. PRNG is initialized with a SEED, KEY and an LFSR value as defined in the SA.									
2	PRNG Generate. Pseudo-random data is generated with the LFSR as input value. Before this mode can be used, the PRNG must be initialized with a valid SEED, KEY and LFSR using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).									
3	PRNG Test. It can be used to test the PRNG function with custom input data. Before this mode can be used, the PRNG must be initialized once with a valid SEED using PRNG Init (<code>PKTE_CTL_STAT . PRNGMD=b'01</code>).									

Table 35-40: PKTE_CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/W)	HASHFINAL	Hash Final. When the <code>PKTE_CTL_STAT.HASHFINAL</code> bit is zero, the data to be hashed must be a multiple of the hash block size, 64 bytes for SHA-1, MD5, SHA-224, SHA-256. This bit is only applicable for Basic Hash, Basic Encrypt-Hash and Basic Hash-De-crypt operations that use the SHA-1, MD5, SHA-224, SHA-256 hash algorithm. The <code>PKTE_CTL_STAT.HASHFINAL</code> bit is overruled for HMAC operations that always completes the hash and always returns the last written value on a read by the host.
		0 Perform Intermediate Hash Operation. The packet engine performs an intermediate hash operation by generating an intermediate hash digest on the data presented on the input. No hash pad is applied.
		1 Perform Final Hash Operation. The packet engine appends the required final hash pad and generates the final hash digest on the data presented on the input. This completes the hash operation.
3 (R/W)	INITARC4	Init ARC4. The <code>PKTE_CTL_STAT.INITARC4</code> bit initializes the ARC4 crypto algorithm with a new key. This bit always returns the last written value on a read by the Host. This bit is only applicable for operations that use the ARC4 algorithm and must be zero for all other operations.
		0 Load ARC4 State and ARC4 i/j pointer from the SA The ARC4 State and ARC4 i/j pointer are loaded from the SA to continue the encrypt/decrypt processing from the previous algorithm state.
		1 Read ARC4 key from the SA-record and initialize ARC4 S-boxes using this key The ARC4 key is read from the SA-record, the ARC4 S-boxes are initialized using this key, prior to the encryption/decryption of data. This bit overrules Stateful mode as defined in bits [9:8] of <code>PKTE_SA_CMD1</code> .
1 (R/W)	PERDY	Packet Engine Ready. The <code>PKTE_CTL_STAT.PERDY</code> bit indicates that the packet engine has completed processing the command descriptor and returns the result descriptor with ownership set to the host. This bit can be reset to 0 by the host and the packet engine, but only the packet engine can set this bit. When the packet engine is idle (not processing), this bit always returns '1' on a read by the host.

Table 35-40: PKTE_CTL_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	HOSTRDY	Host Ready. The <code>PKTE_CTL_STAT.HOSTRDY</code> bit indicates that the host has populated the command descriptor. This bit can be reset to 0 by the host and the packet engine, but only the host can set this bit. When the packet engine is idle (not processing), this bit always returns '0' on a read by the host.

Starting Entry of 256-byte Data Input/Output Buffer

When in direct host mode, the source packet data is written here to be transferred to the packet engine. The host can monitor the available space in the input buffer through the [PKTE_STAT](#) register. This is also the location in the packet engine from where output data is read when in direct host mode. The host can monitor the available bytes in the output buffer through the [PKTE_STAT](#) register.

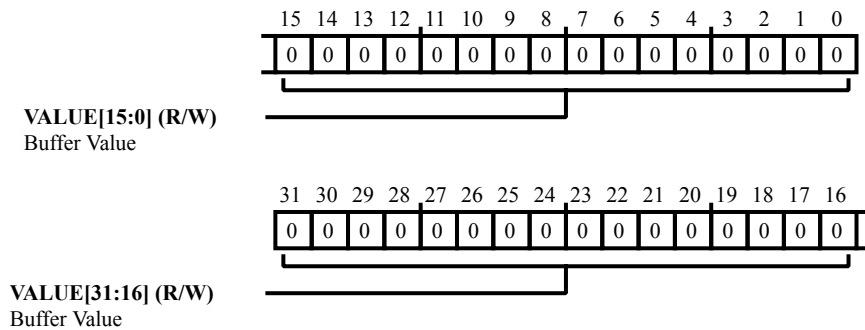


Figure 35-15: PKTE_DATAIO_BUF Register Diagram

Table 35-41: PKTE_DATAIO_BUF Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Buffer Value.

Packet Engine Destination Address

The `PKTE_DEST_ADDR` register holds the starting (byte) address to write the result packet from the requested operation.

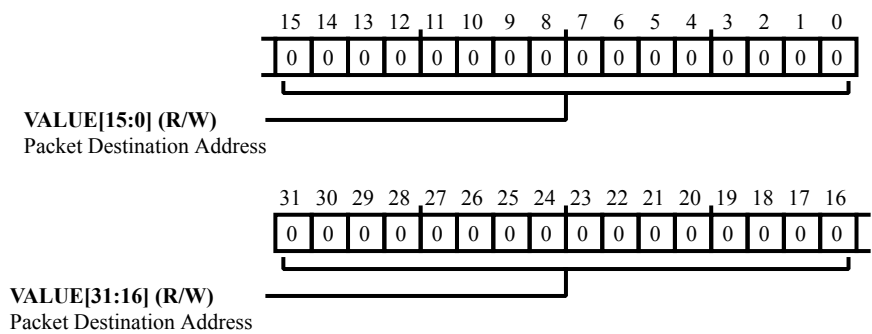


Figure 35-16: `PKTE_DEST_ADDR` Register Diagram

Table 35-42: `PKTE_DEST_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Packet Destination Address.

Packet Engine DMA Configuration Register

The `PKTE_DMA_CFG` register configures the maximum burst transfer size, enables incremental transfers, and inserts IDLE cycles between two bus transfers.

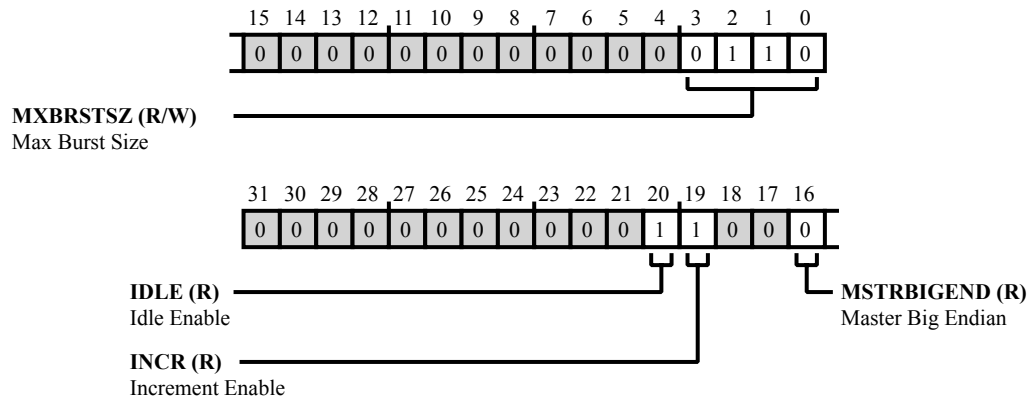


Figure 35-17: PKTE_DMA_CFG Register Diagram

Table 35-43: PKTE_DMA_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
20 (R/NW)	IDLE	Idle Enable. The <code>PKTE_DMA_CFG.IDLE</code> bit allows the peripheral bus master to insert one additional IDLE transfer between two successive peripheral bus master burst operations. This provides the arbiter one additional cycle to hand over the grant to another peripheral bus master.
		0 The peripheral bus master inserts no IDLE cycle between two successive burst operations
		1 The peripheral bus master inserts one additional IDLE transfer between two successive burst operations

Table 35-43: PKTE_DMA_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19 (R/NW)	INCR	Increment Enable. The <code>PKTE_DMA_CFG.INCR</code> bit lets the peripheral bus master generate INC4, INC8 and INC16 type of burst transfers. By default, the peripheral bus master generates the largest possible incremental burst of unspecified length (INCR) with a maximum length (in bytes) as configured by the <code>PKTE_DMA_CFG.MXBRSTSZ</code> bit field. In case there are less than 4 bytes of data available or the 1kB boundary will be crossed using a burst operation, then a single transfer of size byte is generated. When the <code>PKTE_DMA_CFG.INCR</code> bit is set, the peripheral bus master generates one or more incremental burst of specified length (INC4, INC8, INC16). In case there is less data available then the smallest possible burst (INC4) or the 1kB boundary will be crossed using a burst operation, then an unspecified length burst or a single transfer of size byte is generated.
		0 The bus master will generate only INCR burst types
		1 The bus master will generate INC4, INC8 and INC16 burst types
16 (R/NW)	MSTRBIGEND	Master Big Endian. The <code>PKTE_DMA_CFG.MSTRBIGEND</code> bit determines whether the engine is used in a little or big endian system.
		0 Little endian
		1 Big endian
3:0 (R/W)	MXBRSTSZ	Max Burst Size. The <code>PKTE_DMA_CFG.MXBRSTSZ</code> bit field configures the maximum size of an unspecified length burst (INC) at the bus in bytes. When there is less data available than the <code>PKTE_DMA_CFG.MXBRSTSZ</code> bit field setting or the 1kB boundary will be crossed using a burst operation, then the length of the burst can be less than <code>PKTE_DMA_CFG.MXBRSTSZ</code> . Any requested transfers larger than this size are broken up in to multiple burst transfers of this size or less.

Packet Engine Endian Configuration Register

The packet engine incorporates a powerful interface specific endian handler. This endian handler allows byte lane swapping in each direction for data passing through the host interface.

The `PKTE_ENDIAN_CFG` register configures the byte order function for the peripheral bus master and peripheral bus slave interface. The bits for the peripheral bus master are combined in four sets of two bits; each group configures a byte swap function for a particular DMA transfer. The same applies for the peripheral bus slave interface.

The `PKTE_ENDIAN_CFG` register also defines the endian swapping that occurs for host-initiated target transfers and for packet engine master DMA read and write transfers. Individual endian swap enable bits in the configuration (`PKTE_CFG`) register can enable the endian swap for various transaction types: command descriptors and result descriptors, SA records and state records, packet data.

In direct host mode, only target operations are supported. Only the target endian configuration of this register is applicable.

Note: This register is typically programmed once during the initialization phase, although software is allowed to dynamically change the setting in this register just before initiating a data transfer. The developer will have to analyze the benefit of the cycles needed to write the endian register dynamically versus handling endian swapping for some data structures in the host system (most modern processors support a byte swap in zero cycles). Certainly the endian swap should be set correctly for the packet data, since this represents the majority of the data transferred.

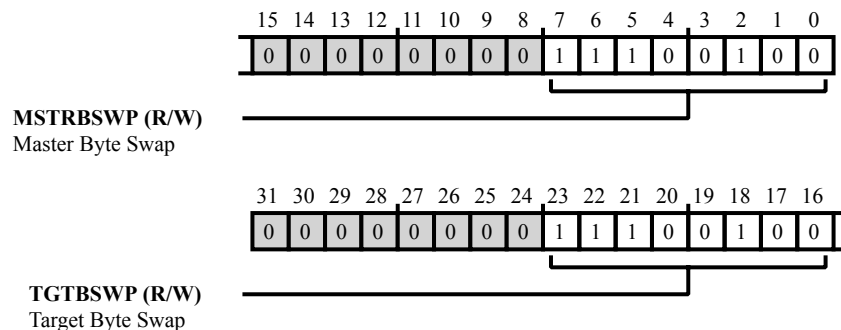


Figure 35-18: PKTE_ENDIAN_CFG Register Diagram

Table 35-44: PKTE_ENDIAN_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:16 (R/W)	TGTBSWP	Target Byte Swap. The <code>PKTE_ENDIAN_CFG.TGTBSWP</code> bit field configures the byte swap for peripheral bus target transfers. Note that only target word transfers are supported. Each double-bit field in this register specifies the source of the indicated byte lane. The field values are interpreted as follows: 00 = byte 0, 01 = byte 1, 10 = byte 2, 11 = byte 3. Note: Setting the value 0xE4 defines no swap (little endian) and setting value 0x1B defines a full byte swap within a 32-bit word (big endian).

Table 35-44: PKTE_ENDIAN_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	MSTRBSWP	Master Byte Swap. The <code>PKTE_ENDIAN_CFG.MSTRBSWP</code> bit field configures the byte swap for peripheral bus master multi-byte transfers, including command descriptors, result descriptors, SA records, state records and packet data. Separate controls in the PKTE_CFG register can enable this swap individually for each of the 4 types of data. Each double-bit field in this register specifies the source of the indicated peripheral bus byte lane. The field values are interpreted as follows: 00=byte 0, 01=byte 1, 10=byte 2, 11=byte 3. Note: Setting the value 0xE4 defines no swap (little endian) and setting value 0x1B defines a full byte swap within a 32-bit word (big endian).

Packet Engine Halt Control Register

The `PKTE_HLT_CTL` register controls the packet engine halt mode. This register can be used for debugging purposes while processing in autonomous ring mode or target command mode. During the halt mode, the host can read all internal registers for examination without side-effects. When halted, the host should not write to any registers. To continue packet engine operation, the host must write to the `PKTE_CONT` (PKTE continue) register.

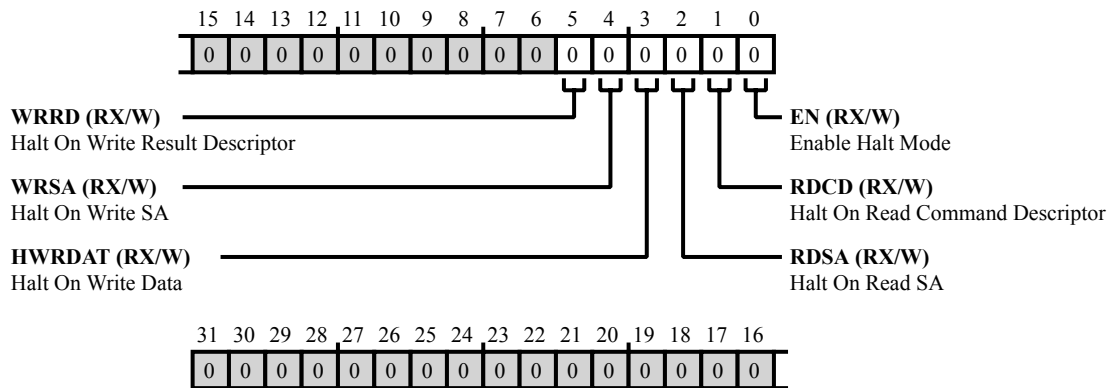


Figure 35-19: `PKTE_HLT_CTL` Register Diagram

Table 35-45: `PKTE_HLT_CTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W)	WRRD	Halt On Write Result Descriptor. The <code>PKTE_HLT_CTL.WRRD</code> bit halts the packet engine in the <code>HALT_WRITE_STATUS</code> state after it completes a result descriptor write operation to the result descriptor ring. The host can use this bit to examine the result descriptor that is currently in the host memory.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation
4 (RX/W)	WRSA	Halt On Write SA. The <code>PKTE_HLT_CTL.WRSA</code> bit halts the packet engine in the <code>HALT_WRITE_SA</code> state after it completes an SA write operation to the host memory. The host can use this bit to examine the security context that is currently in the host memory.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation

Table 35-45: PKTE_HLT_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (RX/W)	HWRDAT	Halt On Write Data. The <code>PKTE_HLT_CTL.HWRDAT</code> bit halts the packet engine in the <code>HALT_DATA</code> state after it completes writing the result packet data to the host memory. The host can use this bit to examine the result packet that is currently in the host memory.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation
2 (RX/W)	RDSA	Halt On Read SA. The <code>PKTE_HLT_CTL.RDSA</code> bit halts the packet engine in the <code>HALT_READ_SA</code> state after it completes an SA read operation from the host memory. The host can use this bit to examine the security context that is currently in the SA registers.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation
1 (RX/W)	RD CD	Halt On Read Command Descriptor. The <code>PKTE_HLT_CTL.RD CD</code> bit halts the packet engine in the <code>HALT_READ_DESCR</code> state after it completes a command descriptor read operation from the command descriptor ring. It will halt whether the descriptor is valid or invalid. The host can use this bit to examine the command descriptor that is currently in the internal command descriptor registers.
		0 Do not halt the Packet Engine operation
		1 Halt the Packet Engine operation
0 (RX/W)	EN	Enable Halt Mode. The <code>PKTE_HLT_CTL.EN</code> bit enables halt mode where the packet engine can halt processing at any processing state as indicated by bits [5:1]. When halted, the packet engine continues operation on a write to the <code>PKTE_CONT</code> register.
		0 Do not enable halt mode
		1 Enable halt mode

Packet Engine Halt Status Register

The `PKTE_HLT_STAT` register reflects the status of the packet engine in halt mode. This register can be used for debugging purposes while processing in autonomous ring mode or target command mode. When the packet engine is halted, the host can read all internal registers for examination without side effects. The host should not write to any registers.

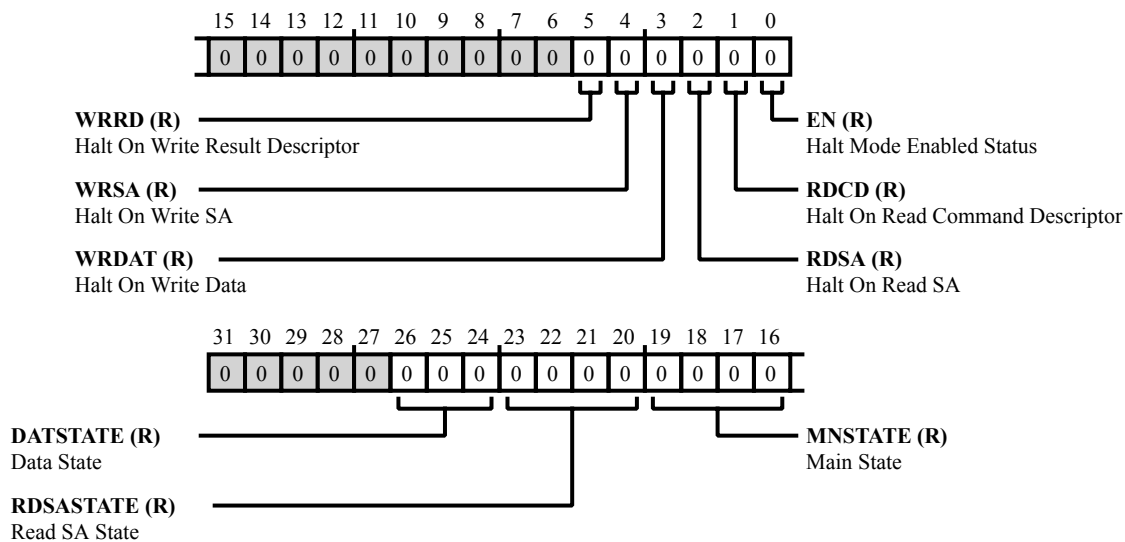


Figure 35-20: PKTE_HLT_STAT Register Diagram

Table 35-46: PKTE_HLT_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
26:24 (R/NW)	DATSTATE	Data State. The <code>PKTE_HLT_STAT.DATSTATE</code> bit field indicates the state of the packet engine read data FSM.
		0 DATA_IDLE, no operation
		1 DATA_READ
		2 DATA_WRITE
		3 DATA_WAIT
		5 DATA_PAD_READ
		6 DATA_BYP_READ
		7 RESERVED
23:20 (R/NW)	RDSASTATE	Read SA State. The <code>PKTE_HLT_STAT.RDSASTATE</code> bit field indicates the state of the packet engine read SA FSM.

Table 35-46: PKTE_HLT_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		0	SA_IDLE, no operation
		1	SA_READ_CMD
		2	SA_READ_STATE_IV
		3-5	RESERVED
		6	SA_READ_ARC4_STATE
		7	SA_READ_WAIT
		8	RESERVED
		9	SA_WRITE_PROT_HDR
		10	RESERVED
		11	SA_WRITE_IV
		12	SA_WRITE_DIGEST
		13	SA_WRITE_ARC4_IJ_PNTR
		14	SA_WRITE_ARC4_STATE
		15	SA_WRITE_WAIT
19:16 (R/NW)	MNSTATE	Main State. The PKTE_HLT_STAT.MNSTATE bit field indicates the state of the packet engine main FSM.	
		0	MAIN_IDLE, no operation
		1	MAIN_READ_CD, reading command descriptor
		2	MAIN_READ_SA, reading SA
		3	MAIN_DATA, processing data
		4	MAIN_WRITE_SA, writing SA
		5	MAIN_WRITE_STATUS, writing status
		6	MAIN_WRITE_CD, updating command descriptor
		7	MAIN_WRITE_RD, updating result descriptor
		8	MAIN_INIT_WAIT, wait single clock
		9	MAIN_HALT_READ_CD, halt after read command descriptor
		10	MAIN_HALT_READ_SA, halt after read SA
		11	MAIN_HALT_DATA, halt after processing data
		12	MAIN_HALT_WRITE_SA, halt after write SA

Table 35-46: PKTE_HLT_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		13	MAIN_WAIT_FOR_CLOCK, wait for clocks to be active
		15	MAIN_HALT_WRITE_RD, halt after write result descriptor
5 (R/NW)	WRRD	Halt On Write Result Descriptor. The PKTE_HLT_STAT.WRRD bit reflects the value in the PKTE_HLT_CTL.WRRD bit.	
4 (R/NW)	WRSA	Halt On Write SA. The PKTE_HLT_STAT.WRSA bit reflects the value in the PKTE_HLT_CTL.WRSA bit.	
3 (R/NW)	WRDAT	Halt On Write Data. The PKTE_HLT_STAT.WRDAT bit reflects the value in the PKTE_HLT_CTL.HWRDAT bit.	
2 (R/NW)	RDSA	Halt On Read SA. The PKTE_HLT_STAT.RDSA bit reflects the value in the PKTE_HLT_CTL.RDSA bit.	
1 (R/NW)	RDCD	Halt On Read Command Descriptor. The PKTE_HLT_STAT.RDCD bit reflects the value in the PKTE_HLT_CTL.RDCD bit.	
0 (R/NW)	EN	Halt Mode Enabled Status.	
		0	Halt mode not enabled
		1	Halt mode enabled

Interrupt Mask Disable Register

The host can use the `PKTE_IMSK_DIS` register to clear individual bits in the `PKTE_INT_EN` register for the host interrupt. This register is a bitmap for each of the possible interrupt sources: A 1 clears the interrupt enable bit, a 0 does not affect the interrupt enable bit in the `PKTE_INT_EN` register. Clearing the enable bits through this register avoids the time-consuming read-modify-write operation on the host.

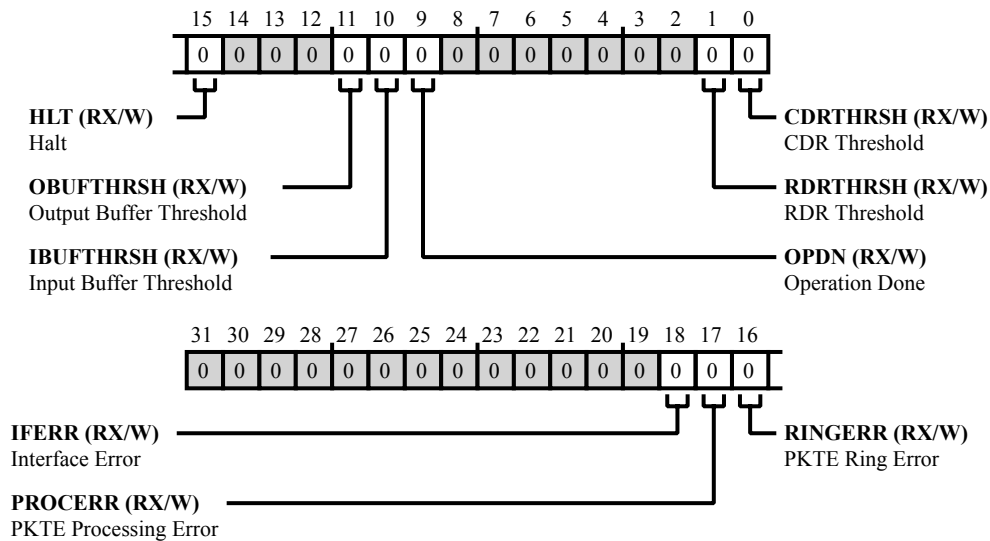


Figure 35-21: `PKTE_IMSK_DIS` Register Diagram

Table 35-47: `PKTE_IMSK_DIS` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (RX/W)	IFERR	Interface Error. Write the <code>PKTE_IMSK_DIS</code> .IFERR bit to clear when the host requests a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (RX/W)	PROCERR	PKTE Processing Error. Write the <code>PKTE_IMSK_DIS</code> .PROCERR bit to clear an extended error that occurred before, during or after processing the current packet in the packet engine.
16 (RX/W)	RINGERR	PKTE Ring Error. Write the <code>PKTE_IMSK_DIS</code> .RINGERR bit to clear a CDR overflow or an RDR underflow.
15 (RX/W)	HLT	Halt. Write the <code>PKTE_IMSK_DIS</code> .HLT bit to clear when the packet engine is in the halt state.

Table 35-47: PKTE_IMSK_DIS Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (RX/W)	OBUFTHRSH	Output Buffer Threshold. Write the <code>PKTE_IMSK_DIS.OBUFTHRSH</code> bit to clear the output buffer counter exceeds the output buffer threshold value defined in <code>PKTE_BUF_THRESH.OUTBUF</code> bit.
10 (RX/W)	IBUFTHRSH	Input Buffer Threshold. Write the <code>PKTE_IMSK_DIS.IBUFTHRSH</code> bit to clear when the input buffer counter is less than or equal to the input buffer threshold value defined in <code>PKTE_BUF_THRESH.INBUF</code> bit.
9 (RX/W)	OPDN	Operation Done.
1 (RX/W)	RDRTHRSH	RDR Threshold. Write the <code>PKTE_IMSK_DIS.RDRTHRSH</code> bit to clear when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the <code>PKTE_RING_THRESH.RDRTHRSH</code> bit, or the RD counter for the RDR in the <code>PKTE_RDSC_CNT</code> register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (RX/W)	CDRTHRSH	CDR Threshold. Write the <code>PKTE_IMSK_DIS.CDRTHRSH</code> bit to clear when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the <code>PKTE_RING_THRESH.CDRTHRSH</code> bit.

Interrupt Mask Enable Register

The host can use the `PKTE_IMSK_EN` register to set individual bits in the `PKTE_INT_EN` register for the host interrupt. This register is a bitmap for each of the possible interrupt sources: A 1 sets the interrupt enable bit, a 0 does not affect the interrupt enable bit in the `PKTE_INT_EN` register. Setting the enable bits through this register avoids the time-consuming read-modify-write operation on the host.

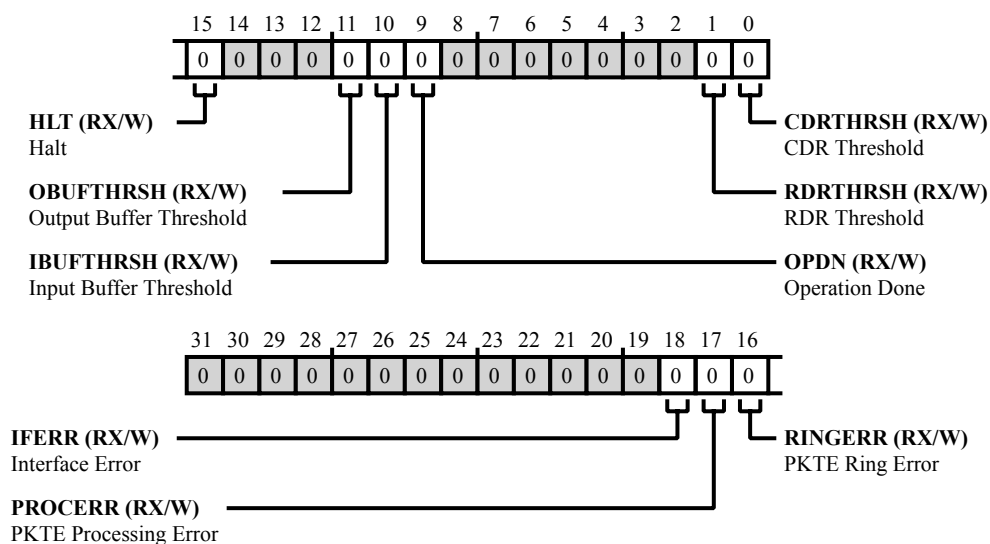


Figure 35-22: PKTE_IMSK_EN Register Diagram

Table 35-48: PKTE_IMSK_EN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (RX/W)	IFERR	Interface Error. Set the <code>PKTE_IMSK_EN</code> . <code>IFERR</code> bit to indicate a host request for a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (RX/W)	PROCERR	PKTE Processing Error. Set the <code>PKTE_IMSK_EN</code> . <code>PROCERR</code> bit to indicate an extended error occurred before, during or after processing the current packet in the packet engine.
16 (RX/W)	RINGERR	PKTE Ring Error.
15 (RX/W)	HLT	Halt. Set the <code>PKTE_IMSK_EN</code> . <code>HLT</code> bit to indicate when the packet engine is in the halt state.

Table 35-48: PKTE_IMSK_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (RX/W)	OBUFTHRSH	Output Buffer Threshold. Set the PKTE_IMSK_EN.OBUFTHRSH bit to indicate that the output buffer counter exceeds the output buffer threshold value defined in the PKTE_BUF_THRESH.OUTBUF bit.
10 (RX/W)	IBUFTHRSH	Input Buffer Threshold. Set the PKTE_IMSK_EN.IBUFTHRSH bit to indicate the input buffer counter is less than or equal to the input buffer threshold value defined in PKTE_BUF_THRESH.INBUF bit.
9 (RX/W)	OPDN	Operation Done.
1 (RX/W)	RDRTHRSH	RDR Threshold. Set the PKTE_IMSK_EN.RDRTHRSH bit to indicate when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the PKTE_RING_THRESH.RDRTHRSH bit, or the RD counter for the RDR in PKTE_RDSC_CNT register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (RX/W)	CDRTHRSH	CDR Threshold. Set the PKTE_IMSK_EN.CDRTHRSH bit to indicate when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the PKTE_RING_THRESH.CDRTHRSH bit.

Interrupt Masked Status Register

The `PKTE_IMSK_STAT` register provides interrupt status visibility to the host, after the interrupt mask is applied. This lets the host view the selected sources of interrupts that are directed to the interrupt output signal, that is connected to the system interrupt controller. As with the unmasked status register, all interrupt bits are latched and must be cleared using the `PKTE_INT_CLR` register in order to capture a subsequent event. A 1 indicates that the associated interrupt is present.

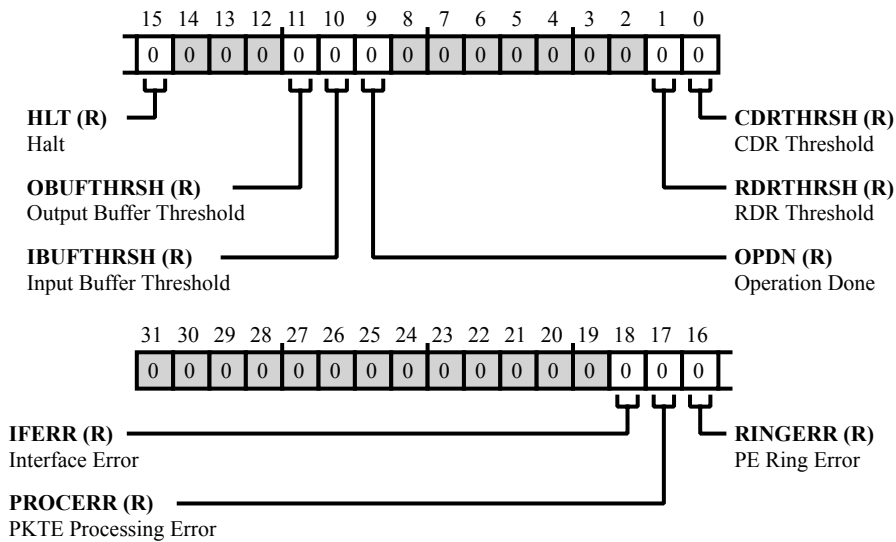


Figure 35-23: PKTE_IMSK_STAT Register Diagram

Table 35-49: PKTE_IMSK_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/NW)	IFERR	Interface Error. The <code>PKTE_IMSK_STAT</code> . <code>IFERR</code> bit is set when the host requests a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (R/NW)	PROCERR	PKTE Processing Error. The <code>PKTE_IMSK_STAT</code> . <code>PROCERR</code> bit is when an extended error occurred before, during or after processing the current packet in the packet engine.
16 (R/NW)	RINGERR	PE Ring Error. The <code>PKTE_IMSK_STAT</code> . <code>RINGERR</code> bit is set on a CDR overflow or an RDR underflow.
15 (R/NW)	HLT	Halt. The <code>PKTE_IMSK_STAT</code> . <code>HLT</code> bit is set when the packet engine is in the HALT state.

Table 35-49: PKTE_IMSK_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	OBUFTHRSH	Output Buffer Threshold. The <code>PKTE_IMSK_STAT.OBUFTHRSH</code> bit is set when the output buffer counter exceeds the output buffer threshold value defined in <code>PKTE_BUF_THRESH.OUTPUTBUF</code> bit.
10 (R/NW)	IBUFTHRSH	Input Buffer Threshold. The <code>PKTE_IMSK_STAT.IBUFTHRSH</code> bit is set when the input buffer counter is less than or equal to the input buffer threshold value defined in <code>PKTE_BUF_THRESH.INBUF</code> bit.
9 (R/NW)	OPDN	Operation Done.
1 (R/NW)	RDRTHRSH	RDR Threshold. The <code>PKTE_IMSK_STAT.RDRTHRSH</code> bit is set when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the <code>PKTE_RING_THRESH.RDRTHRSH</code> bit, or the RD counter for the RDR in <code>PKTE_RDSC_CNT</code> register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (R/NW)	CDRTHRSH	CDR Threshold. The <code>PKTE_IMSK_STAT.CDRTHRSH</code> bit is set when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the <code>PKTE_RING_THRESH.CDRTHRSH</code> bit.

Packet Engine Input Buffer Count Register

The `PKTE_INBUF_CNT` register provides the number of bytes available in the input buffer. The `PKTE_INBUF_CNT` register is used in direct host mode only.

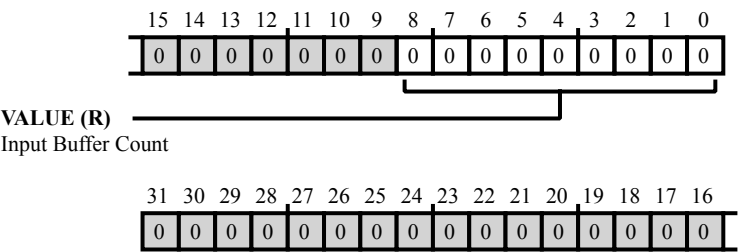


Figure 35-24: PKTE_INBUF_CNT Register Diagram

Table 35-50: PKTE_INBUF_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/NW)	VALUE	Input Buffer Count. The <code>PKTE_INBUF_CNT.VALUE</code> bit field provides the number of bytes in the input buffer. The packet engine decrements the counter by 4 when a 32-bit word is read from the input buffer.

Packet Engine Input Buffer Count Increment Register

A host connected through the system slave bus can increment the input buffer counter by writing a value between 4 and 256, in multiples of 4, to the lowest bits of this register. The `PKTE_INBUF_INCR` register is used in direct host mode only.

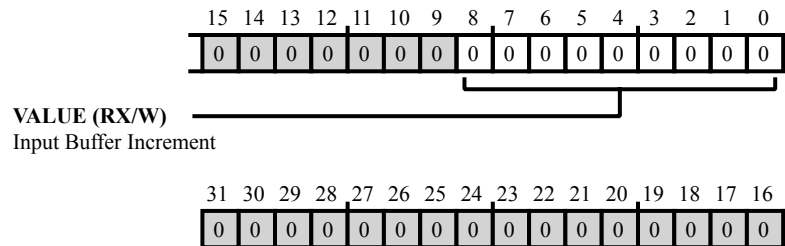


Figure 35-25: `PKTE_INBUF_INCR` Register Diagram

Table 35-51: `PKTE_INBUF_INCR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (RX/W)	VALUE	Input Buffer Increment. The value written is added to the input buffer counter. Valid values range from 4 to 256, in multiples of 4.

Interrupt Configuration Register

The `PKTE_INT_CFG` register configures the interrupt type that is sent to the interrupt line connected to the system interrupt controller. (Note that this only effects the final output of the interrupt subsystem).

Configuring the interrupt output type for pulse causes the interrupt signal to pulse low for two clock cycles when activated. When set for level, the interrupt signal is set low until cleared by the host (it follows the bit in the masked status register). For the host, this is typically set to level.

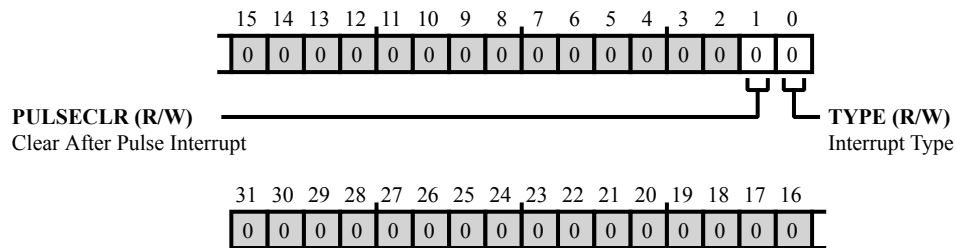


Figure 35-26: `PKTE_INT_CFG` Register Diagram

Table 35-52: `PKTE_INT_CFG` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/W)	PULSECLR	Clear After Pulse Interrupt. The <code>PKTE_INT_CFG.PULSECLR</code> bit clears the latched interrupt source after the pulse interrupt.
		0 Manually clear pulse interrupt source. Do not automatically clear the interrupt sources after pulsing the interrupt output. Clear the source by writing to the <code>PKTE_INT_CLR</code> register.
		1 Automatically clear pulse interrupt source. After pulsing the interrupt output, automatically clear the sources.
0 (R/W)	TYPE	Interrupt Type. The <code>PKTE_INT_CFG.TYPE</code> bit selects the type, pulse or level, for the interrupt output to the system.
		0 Level. The interrupt output is a level signal that is set low when an enabled interrupt is active until the interrupt is cleared.
		1 Pulse. The interrupt output is a two clock cycle low-active pulse, activated when an enabled interrupt is active.

Interrupt Clear Register

The `PKTE_INT_CLR` register allows the host processor to clear pending interrupts. A 1 written to a given bit in this register clears the corresponding interrupt. A 0 leaves the interrupt latch unchanged for that position.

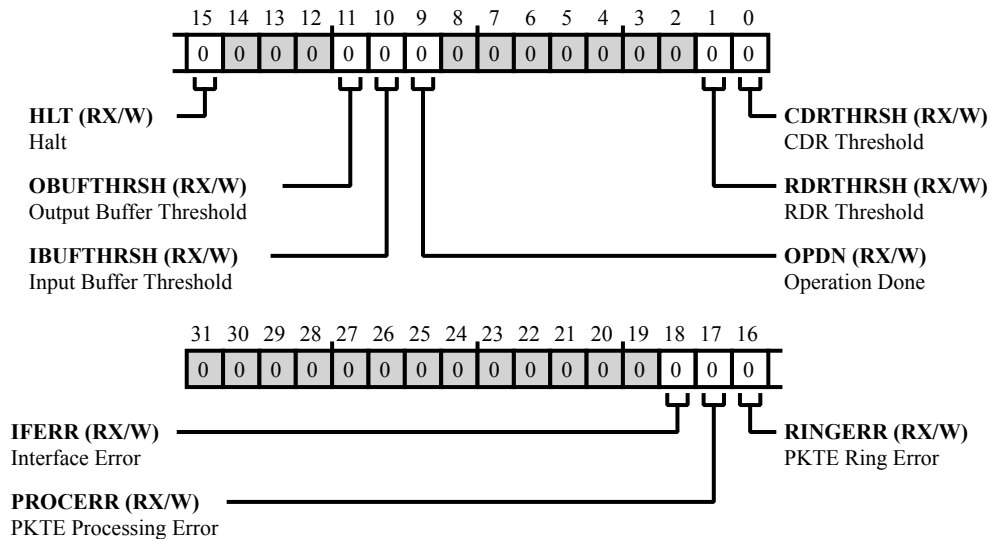


Figure 35-27: `PKTE_INT_CLR` Register Diagram

Table 35-53: `PKTE_INT_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (RX/W)	IFERR	Interface Error. The <code>PKTE_INT_CLR</code> . <code>IFERR</code> bit is set when the host requests a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (RX/W)	PROCERR	PKTE Processing Error. The <code>PKTE_INT_CLR</code> . <code>PROCERR</code> bit is set when an extended error occurred before, during or after processing the current packet in the packet engine.
16 (RX/W)	RINGERR	PKTE Ring Error. The <code>PKTE_INT_CLR</code> . <code>RINGERR</code> bit is set on a CDR overflow or an RDR under-flow.
15 (RX/W)	HLT	Halt. The <code>PKTE_INT_CLR</code> . <code>HLT</code> bit is set when the packet engine is in the HALT state.
11 (RX/W)	OBUFTHRSH	Output Buffer Threshold. The <code>PKTE_INT_CLR</code> . <code>OBUFTHRSH</code> bit is set when the output buffer counter exceeds the output buffer threshold value defined in <code>PKTE_BUF_THRESH</code> . <code>OUTBUF</code> bit.

Table 35-53: PKTE_INT_CLR Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (RX/W)	IBUFTHRSH	Input Buffer Threshold. The <code>PKTE_INT_CLR.IBUFTHRSH</code> bit is set when the input buffer counter is less than or equal to the input buffer threshold value defined in <code>PKTE_BUF_THRESH.INBUF</code> bit.
9 (RX/W)	OPDN	Operation Done.
1 (RX/W)	RDRTHRSH	RDR Threshold. The <code>PKTE_INT_CLR.RDRTHRSH</code> bit is set when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the <code>PKTE_RING_THRESH.RDRTHRSH</code> bit, or the RD counter for the RDR in <code>PKTE_RDSC_CNT</code> register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (RX/W)	CDRTHRSH	CDR Threshold. The <code>PKTE_INT_CLR.CDRTHRSH</code> bit is set when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the <code>PKTE_RING_THRESH.CDRTHRSH</code> bit.

Interrupt Enable Register

The `PKTE_INT_EN` register configures the interrupt mask for the host interrupt. This register is a bitmap for each of the possible interrupt sources. A 1 enables the interrupt source and a 0 disables the source. If an interrupt source is disabled, a cleared bit also clears the matching interrupt in the `PKTE_IMSK_STAT` register.

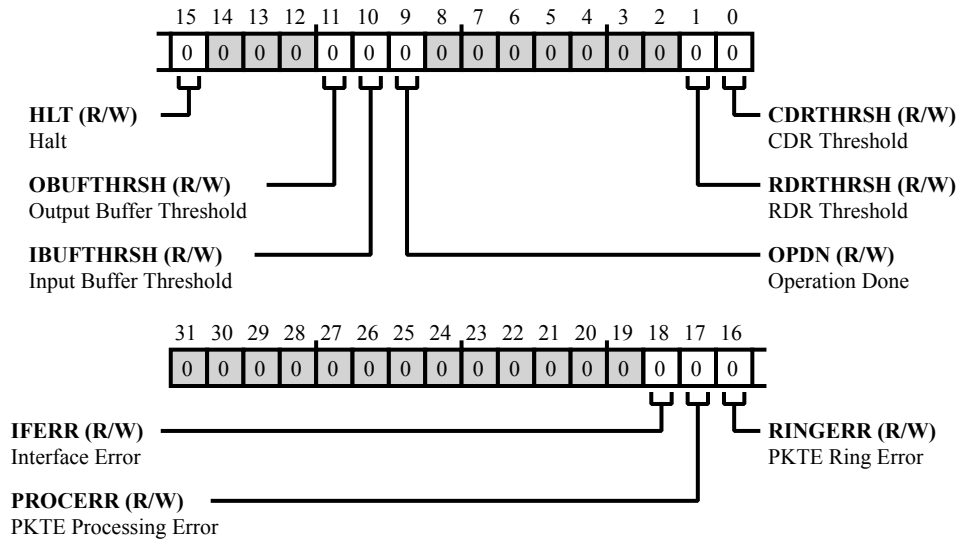


Figure 35-28: `PKTE_INT_EN` Register Diagram

Table 35-54: `PKTE_INT_EN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/W)	IFERR	Interface Error. Set the <code>PKTE_INT_EN</code> .IFERR bit for host requests for a non 32-bit access to the packet engine interrupt or when the packet engine receives an error writing data back out to the host memory system.
17 (R/W)	PROCERR	PKTE Processing Error. Set the <code>PKTE_INT_EN</code> .PROCERR bit to enable the extended error occurred before, during or after processing the current packet in the packet engine interrupt.
16 (R/W)	RINGERR	PKTE Ring Error. Set the <code>PKTE_INT_EN</code> .RINGERR bit to enable the CDR overflow or RDR under-flow interrupt.
15 (R/W)	HLT	Halt. Set the <code>PKTE_INT_EN</code> .HLT bit for when the packet engine is in the HALT state.
11 (R/W)	OBUFTHRS	Output Buffer Threshold. Set the <code>PKTE_INT_EN</code> .OBUFTHRS bit for to trigger an interrupt when the output buffer counter exceeds the output buffer threshold value defined in the <code>PKTE_BUF_THRESH</code> .OUTBUF bit.

Table 35-54: PKTE_INT_EN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	IBUFTHRSH	Input Buffer Threshold. Set the PKTE_INT_EN.IBUFTHRSH bit for to trigger an interrupt when the input buffer counter is less than or equal to the input buffer threshold value defined in the PKTE_BUF_THRESH.INBUF bit.
9 (R/W)	OPDN	Operation Done.
1 (R/W)	RDRTHRSH	RDR Threshold. Set the PKTE_INT_EN.RDRTHRSH bit for to trigger an interrupt when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the PKTE_RING_THRESH.RDRTHRSH bit, or the RD counter for the RDR in PKTE_RDSC_CNT register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (R/W)	CDRTHRSH	CDR Threshold. Set the PKTE_INT_EN.CDRTHRSH bit for to trigger an interrupt when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the PKTE_RING_THRESH.CDRTHRSH bit.

Interrupt Unmasked Status Register

The `PKTE_IUMSK_STAT` register provides interrupt status visibility to the host, prior to the interrupt mask being applied. Using this register, the host can view all potential sources of incoming interrupts. All of these sources, whether masked in or out, are latched in this register and must be cleared using the `PKTE_INT_CLR` register in order to capture a subsequent event. A 1 indicates that the associated interrupt is present.

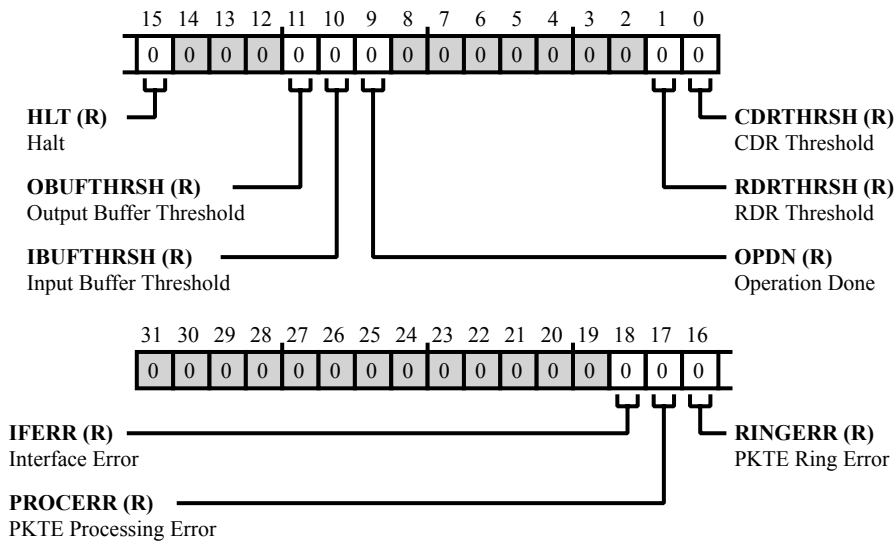


Figure 35-29: `PKTE_IUMSK_STAT` Register Diagram

Table 35-55: `PKTE_IUMSK_STAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
18 (R/NW)	IFERR	Interface Error. The <code>PKTE_IUMSK_STAT</code> . <code>IFERR</code> bit is set when the host requests a non 32-bit access to the packet engine or when the packet engine receives an error writing data back out to the host memory system.
17 (R/NW)	PROCERR	PKTE Processing Error. The <code>PKTE_IUMSK_STAT</code> . <code>PROCERR</code> bit is set when an extended error occurred before, during or after processing the current packet in the packet engine.
16 (R/NW)	RINGERR	PKTE Ring Error. The <code>PKTE_IUMSK_STAT</code> . <code>RINGERR</code> bit is set on a CDR overflow or an RDR underflow.
15 (R/NW)	HLT	Halt. The <code>PKTE_IUMSK_STAT</code> . <code>HLT</code> bit is set when the packet engine is in the HALT state.

Table 35-55: PKTE_IUMSK_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/NW)	OBUFTHRSH	Output Buffer Threshold. The <code>PKTE_IUMSK_STAT.OBUFTHRSH</code> interrupt is triggered when the output buffer counter exceeds the output buffer threshold value defined in <code>PKTE_BUF_THRESH.OUTBUF</code> bit.
10 (R/NW)	IBUFTHRSH	Input Buffer Threshold. The <code>PKTE_IUMSK_STAT.IBUFTHRSH</code> interrupt is triggered when the input buffer counter is less than or equal to the input buffer threshold value defined in <code>PKTE_BUF_THRESH.INBUF</code> bit.
9 (R/NW)	OPDN	Operation Done.
1 (R/NW)	RDRTHRSH	RDR Threshold. The <code>PKTE_IUMSK_STAT.RDRTHRSH</code> bit is set when the number of result descriptors for the host in the RDR exceeds the RD threshold value in the <code>PKTE_RING_THRESH.RDRTHRSH</code> , or the RD counter for the RDR in <code>PKTE_RDSC_CNT</code> register is non-zero for more than $2^{(N+10)}$ internal system clock cycles.
0 (R/NW)	CDRTHRSH	CDR Threshold. The <code>PKTE_IUMSK_STAT.CDRTHRSH</code> bit is set when the number of command descriptors for the packet engine in the CDR is less than or equal to the CD threshold value in the <code>PKTE_RING_THRESH.CDRTHRSH</code> bit.

Packet Engine Length Register

The `PKTE_LEN` register gives the length of the packet, the bypass data and a second set of ownership bits.

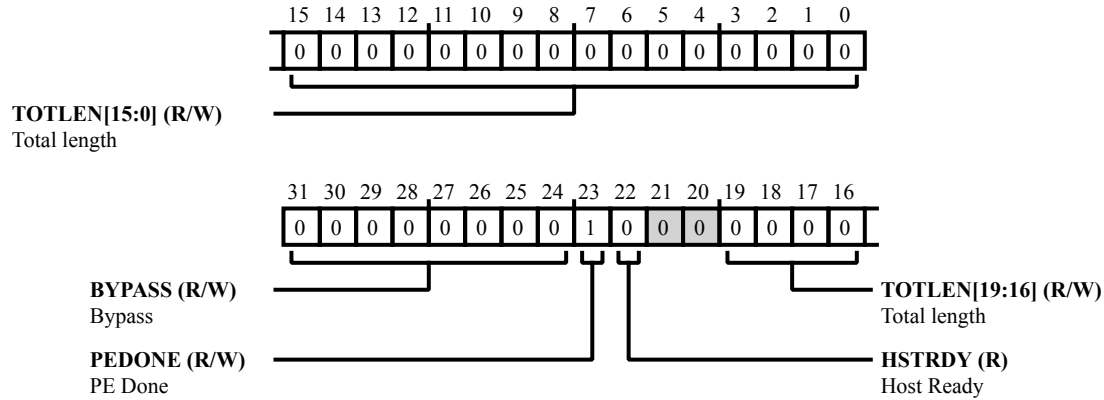


Figure 35-30: `PKTE_LEN` Register Diagram

Table 35-56: `PKTE_LEN` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/W)	BYPASS	Bypass. The <code>PKTE_LEN.BYPASS</code> bit field indicates the length of data in words that must bypass the packet engine and are directly copied from the source buffer to the destination buffer. The packet engine does not process this data. Valid bypass offsets range from 0 (0x00) to 255 (0xFF) words. For SRTP operations, this field specifies the offset in words between the hash and encrypt/decrypt data.
23 (R/W)	PEDONE	PE Done. The <code>PKTE_LEN.PEDONE</code> bit is a mirrored bit from the <code>PKTE_CTL_STAT.PERDY</code> bit. The bit is repeated here to guarantee ownership consistency between the first and last word. When the packet engine fetches a descriptor, these bits must match or the descriptor is discarded and fetched again.
22 (R/NW)	HSTRDY	Host Ready. The <code>PKTE_LEN.HSTRDY</code> bit is a mirrored bit of the <code>PKTE_CTL_STAT.HOSTRDY</code> bit. The bit is repeated here to guarantee ownership consistency between the first and last word. It should also be set along with the <code>PKTE_CTL_STAT.HOSTRDY</code> bit when the command descriptor is finished being populated. When the packet engine fetches a descriptor, these bits must match or the descriptor is discarded and fetched again.

Table 35-56: PKTE_LEN Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
19:0 (R/W)	TOTLEN	Total length. Command Descriptor: The <code>PKTE_LEN.TOTLEN</code> bit field indicates the total length (in bytes) of all data to be passed to the packet engines input buffer for an operation. Exceptions are the PRNG init and PRNG generate operations. The PRNG init operation does not require any input data; this field must be zero. For the PRNG generate operation, this field indicates the number of pseudo-random bytes to be generated. Valid lengths range from 16 (0x00010) to $255 \times 16 = 4080$ (0x00FF0) bytes in multiples of 16 bytes. Valid lengths for the basic operation range from 1 (0x00001) to 1,048,575 (0xFFFFF) bytes. This is the length of the data to be encrypted or hashed and includes the bypass data and padding bytes. Valid lengths for IPsec ESP range from 1 (0x00001) to 65535 (0xFFFFF) bytes. This is the length of the IP payload. Valid lengths for SSL v3.0, TLS v1.x and DTLS range from 1 (0x00001) to 16383 (0x03FFF). This is the length of the payload. Valid lengths for SRTP range from 1 (0x00001) to 65535 (0xFFFFF). This is the length of the payload. Note: A length of zero bytes is illegal and will result in an error status code in the result descriptor. Result Descriptor: Upon completion of an operation, the <code>PKTE_LEN.TOTLEN</code> field indicates the result length of the result packet. Valid lengths range from 1 (0x001) to 1,048,575 (0xFFFFF) bytes. This includes the bypass data and padding bytes. Note: When an extended error (<code>PKTE_CTL_STAT[18]=1</code>) is reported in the result descriptor and no packet data is processed, this field returns zero.

Packet Engine Output Buffer Count Register

The `PKTE_OUTBUF_CNT` register provides the number of data bytes there are in the output buffer. The `PKTE_OUTBUF_CNT` register is used in direct host mode only.

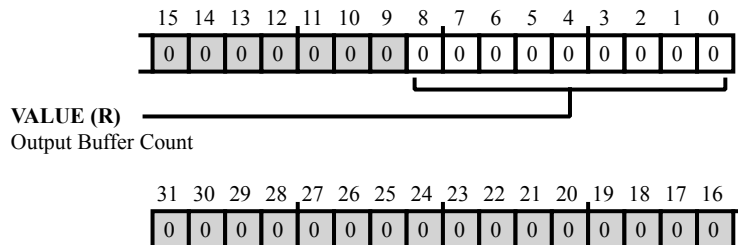


Figure 35-31: `PKTE_OUTBUF_CNT` Register Diagram

Table 35-57: `PKTE_OUTBUF_CNT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/NW)	VALUE	Output Buffer Count. The <code>PKTE_OUTBUF_CNT.VALUE</code> bit field provides the number of bytes in the output buffer. The packet engine increments the counter by 4 when a 32-bit word is written to the output buffer.

Packet Engine Output Buffer Count Decrement Register

A host connected via the system slave bus can decrement the output buffer counter by writing a value between 4 and 256, in multiples of 4, to the lowest bits of this register. The `PKTE_OUTBUF_DECR` register is used in direct host mode only.

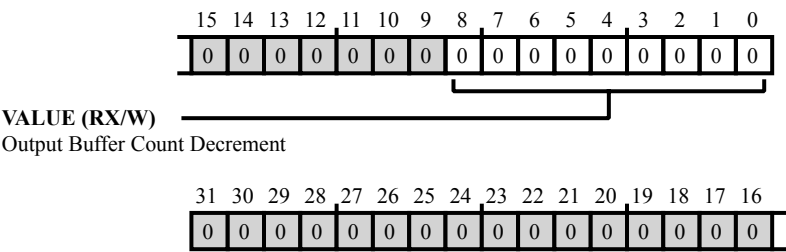


Figure 35-32: `PKTE_OUTBUF_DECR` Register Diagram

Table 35-58: `PKTE_OUTBUF_DECR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (RX/W)	VALUE	Output Buffer Count Decrement. The <code>PKTE_OUTBUF_DECR.VALUE</code> bit field is the value written is subtracted to the output buffer counter. Valid values range from 4 to 256, in multiples of 4.

Packet Engine Result Descriptor Ring Base Address

The `PKTE_RDRBASE_ADDR` register holds the result descriptor ring base address in host memory. It is only applicable in autonomous ring mode and target command mode with RDR enabled. Note that in target command mode, the CDR is not used, but the RDR must be configured when enabled so that the packet engine knows where to write the result descriptors.

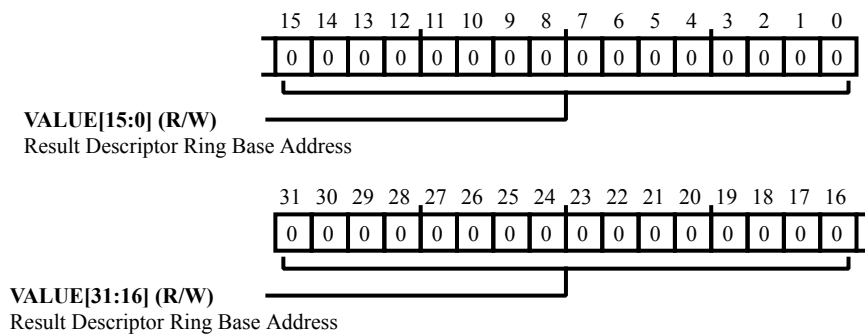


Figure 35-33: `PKTE_RDRBASE_ADDR` Register Diagram

Table 35-59: `PKTE_RDRBASE_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Result Descriptor Ring Base Address. The <code>PKTE_RDRBASE_ADDR.VALUE</code> bit field specifies the base location of the result descriptor ring in the host memory space.

Packet Engine Result Descriptor Count Registers

The `PKTE_RDSC_CNT` register holds the counter for the number of descriptors in the Result Descriptor Ring (RDR). It is incremented by the packet engine each time a valid result descriptor is written to the RDR.

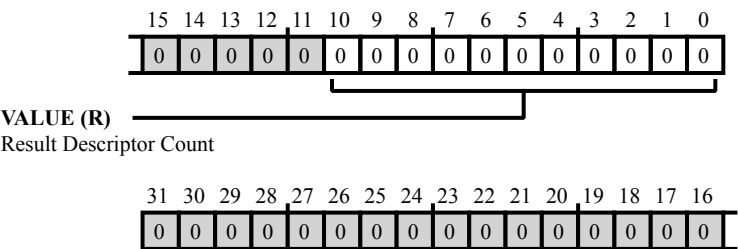


Figure 35-34: PKTE_RDSC_CNT Register Diagram

Table 35-60: PKTE_RDSC_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/NW)	VALUE	Result Descriptor Count. The <code>PKTE_RDSC_CNT.VALUE</code> bit field provides the number of result descriptors in the result descriptor ring. The packet engine increments the counter when a valid result descriptor is written to the RDR.

Packet Engine Result Descriptor Count Decrement Registers

The `PKTE_RDSC_DECR` register is accessible by the host connected through the system slave bus can decrement the result descriptor counter by writing a value between 1 and 255 to the lowest byte of this register.

With an RDR enabled, this is the number of result descriptors that have been read by the host. With an RDR disabled, this indicates that the host has read one valid result descriptor.

In autonomous ring mode or target command mode with the RDR enabled, the host must process 1 to 255 result descriptors from the RDR and then write this register with the number of result descriptors that have been processed by the host.

In direct host mode or target command mode with the RDR disabled, the host must read one result descriptor from the internal descriptor registers and then write this register with the value 1, to indicate that one valid descriptor is read. An RDR threshold interrupt is activated when the result descriptor counter exceeds the threshold value set in the `PKTE_RING_THRESH` register. This interrupt can be used to wake up a process that stalled on an empty RDR.

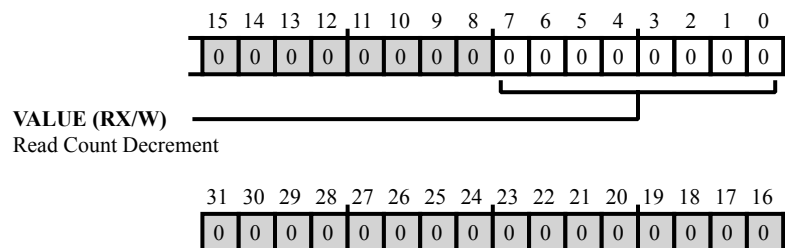


Figure 35-35: `PKTE_RDSC_DECR` Register Diagram

Table 35-61: `PKTE_RDSC_DECR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (RX/W)	VALUE	Read Count Decrement. The value written to the <code>PKTE_RDSC_DECR.VALUE</code> bit field is subtracted from the result descriptor counter. The counter is protected against underflow (See the <code>PKTE_RING_STAT</code> register). Note that bits [10:8] should be written with zeros.

Packet Engine Ring Configuration

The `PKTE_RING_CFG` register configures the size (in number of descriptor ring entries minus 1) for both the command descriptor ring and result descriptor ring in host memory. This register is only applicable for autonomous ring mode and target command mode with RDR enabled.

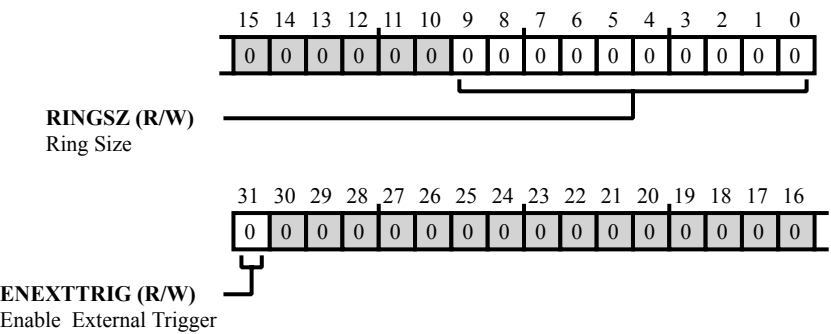


Figure 35-36: `PKTE_RING_CFG` Register Diagram

Table 35-62: `PKTE_RING_CFG` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ENEXTTRIG	Enable External Trigger. The <code>PKTE_RING_CFG.ENEXTTRIG</code> signal enables the increment of the <code>PKTE_CDSC_CNT</code> register through the external input pin <code>ext_cd_cnt_incr</code> and enables the decrement of the <code>PKTE_RDSC_CNT</code> fields through the external input pin <code>ext_rd_cnt_decr</code> .
9:0 (R/W)	RINGSZ	Ring Size. The <code>PKTE_RING_CFG.RINGSZ</code> bit field specifies the size of the command ring in number of descriptors, minus 1. Valid sizes range from 1 (for 2 descriptors) to 1023 (for 1024 descriptors). The accompanying result ring will have the same size.

Packet Engine Ring Pointer Status

The `PKTE_RING_PTR` register holds the pointers to the current entry of the Command Descriptor Ring (CDR) and Result Descriptor Ring (RDR).

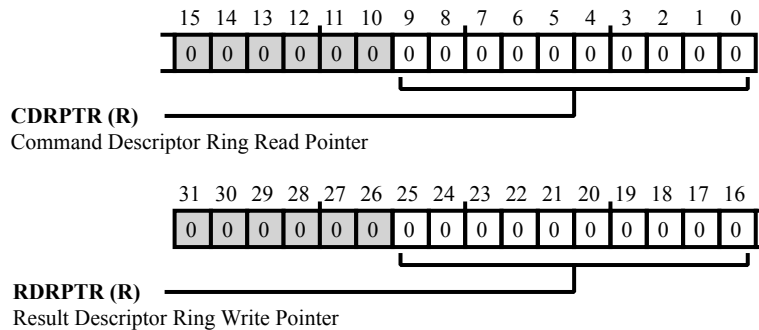


Figure 35-37: `PKTE_RING_PTR` Register Diagram

Table 35-63: `PKTE_RING_PTR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:16 (R/NW)	RDRPTR	Result Descriptor Ring Write Pointer. The <code>PKTE_RING_PTR.RDRPTR</code> bit field indicates the entry number in the RDR that will be written next by the packet engine. The <code>PKTE_RING_PTR.RDRPTR</code> bit field is reset to zero after starting up and updated after every result descriptor write DMA operation. Pointers wrap around; the maximum value this field can have equals the contents of the ring size (<code>PKTE_RING_CFG.RINGSZ</code>) bit field.
9:0 (R/NW)	CDRPTR	Command Descriptor Ring Read Pointer. The <code>PKTE_RING_PTR.CDRPTR</code> bit field indicates the entry number in the CDR that will be read next by the packet engine. The <code>PKTE_RING_PTR.CDRPTR</code> bit field is reset to zero after starting up and updated after every command descriptor read DMA operation. Pointers wrap around; the maximum value this field can have equals the contents of the ring size (<code>PKTE_RING_CFG.RINGSZ</code>) field.

Packet Engine Ring Status

The `PKTE_RING_STAT` register gives indication of either a Command Descriptor Ring (CDR) overflow or a Result Descriptor Ring (RDR) underflow. A ring error (ringerr) interrupt in the interrupt controller is activated on a command descriptor ring overflow or a result descriptor ring underflow. This type of error can occur when the host and the packet engine get out-of-sync. The host can read this register to retrieve information on which ring is corrupted. The corrupted ring must be reset and reinitialized. See the `PKTE_CFG.RSTRING` bit.

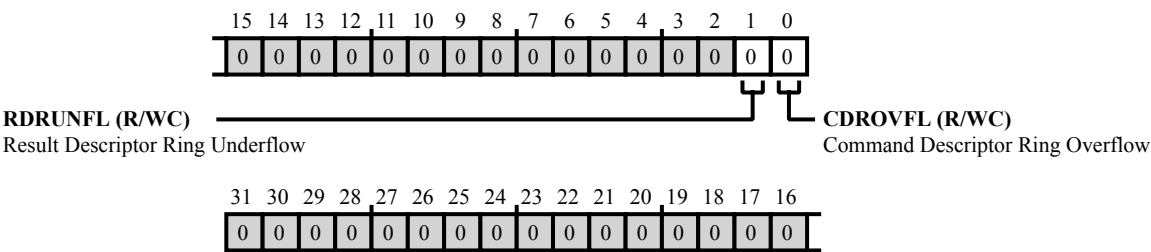


Figure 35-38: PKTE_RING_STAT Register Diagram

Table 35-64: PKTE_RING_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/WC)	RDRUNFL	Result Descriptor Ring Underflow. The <code>PKTE_RING_STAT.RDRUNFL</code> bit is set when the command descriptor count (<code>PKTE_RDSC_CNT</code>) register is decremented below zero. This bit is reset with a write of any value.
0 (R/WC)	CDROVFL	Command Descriptor Ring Overflow. The <code>PKTE_RING_STAT.CDROVFL</code> bit is set when the command descriptor count (<code>PKTE_CDSC_CNT</code>) register is incremented above the ring size (<code>PKTE_RING_CFG.RINGSZ</code>) bits. This bit is reset with a write of any value.

Packet Engine Ring Threshold Registers

To reduce the amount of packet engine result interrupts, the `PKTE_RING_THRESH` register contains threshold and time-out values.

The CDR threshold (`cdrthrsh`) interrupt indicates that the command descriptor counter is less than or equal to the CDR threshold (`cdrthrsh`) value set in this register. This interrupt can be used to wake up a process that stalled on a full CDR.

The RDR threshold (`rdrthrsh`) interrupt indicates that the result descriptor counter exceeds the result descriptor threshold set here, or that the result descriptor counter is non-zero for a time longer than the result descriptor time-out setting. The RDR result interrupt remains active until the result descriptor counter is decremented below the RDR threshold (`rdrthrsh`) value. In case the interrupt is the result of a time-out and the result descriptor counter is below the threshold value, the result descriptor counter must be decremented once before the interrupt can be cleared in the interrupt controller.

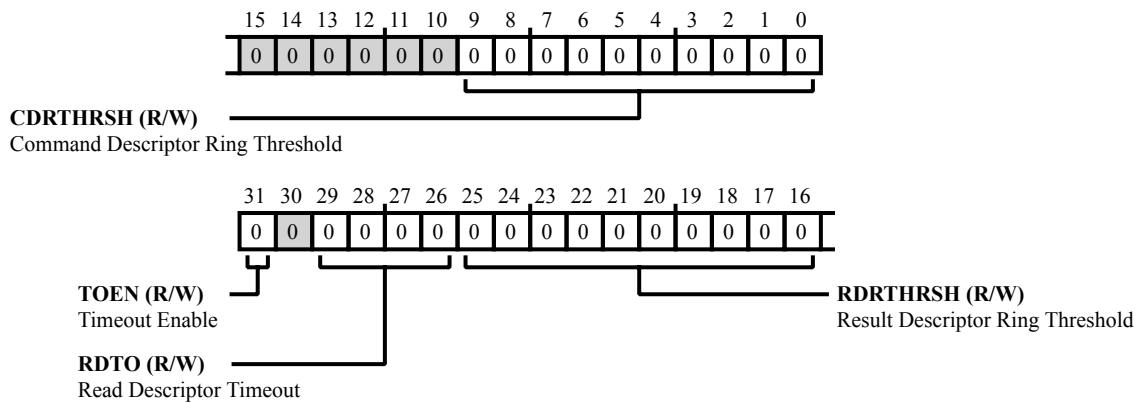


Figure 35-39: `PKTE_RING_THRESH` Register Diagram

Table 35-65: `PKTE_RING_THRESH` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	TOEN	Timeout Enable. A 1 in the <code>PKTE_RING_THRESH</code> . <code>TOEN</code> bit indicates the result descriptor timeout counter is enabled. This bit can be used to de-activate the timeout counter to save power.

Table 35-65: PKTE_RING_THRESH Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29:26 (R/W)	RDTO	Read Descriptor Timeout. The timeout enable (PKTE_RING_THRESH.TOEN) bit in this register must be set to activate this PKTE_RING_THRESH.RDTO result descriptor timeout counter. The rdrthrsh interrupt activates when the RD counter for the RDR is non-zero for more than $2^{(N+10)}$ internal system clock cycles, where 'N' is the value set in this field. Valid settings range from 0 to 15. The minimum time-out value for N=0 is 1024 clock cycles and the maximum time-out value for N=15 is 33554432 clock cycles. At 100 MHz, this is 5.12 us for N=0 and ~335.55 ms for N=15. Note: The time-out delay may not be exact - expect a variation on the order of 1024 system clock cycles (just more than one microsecond at 100 MHz system clock frequency).
25:16 (R/W)	RDRTHRSH	Result Descriptor Ring Threshold. The rdrthrsh interrupt activates when the RD counter for the RDR exceeds the value set in the PKTE_RING_THRESH.RDRTHRSH field. Valid settings range from 0 to 1023.
9:0 (R/W)	CDRTHRSH	Command Descriptor Ring Threshold. The cdrthrsh interrupt activates when CD counter for the CDR is below or equal the value set in the PKTE_RING_THRESH.CDRTHRSH field. Valid settings range from 0 to 1023.

Packet Engine SA Address

The `PKTE_SA_ADDR` register holds the start address of the SA record.

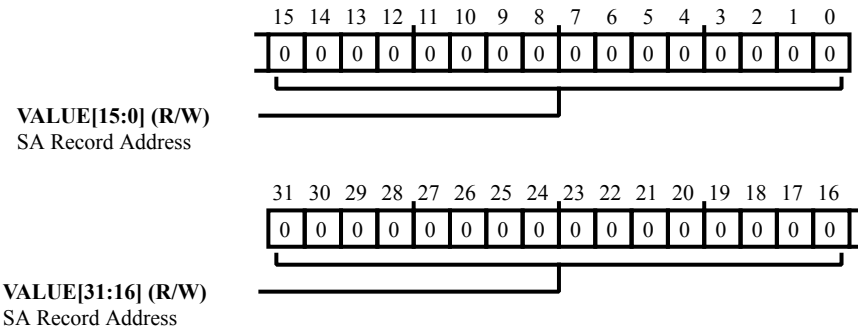


Figure 35-40: `PKTE_SA_ADDR` Register Diagram

Table 35-66: `PKTE_SA_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SA Record Address. The <code>PKTE_SA_ADDR.VALUE</code> bit field holds the start address of the SA record.

ARC4 i and j Pointer Register

When starting a new ARC4 operation the `PKTE_SA_ARC4IJPTR` register contains the initialization value, which is zeros. After processing the ARC4 algorithm it contains the latest status of the ARC4_IJ_PNTR.

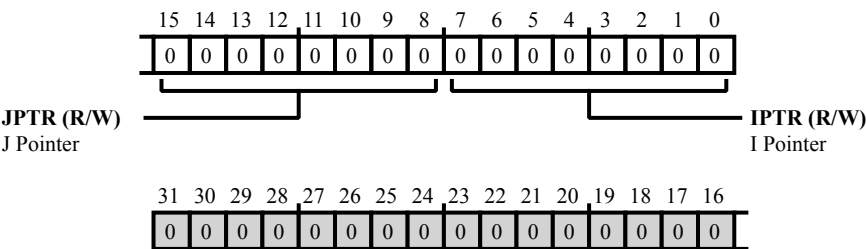


Figure 35-41: PKTE_SA_ARC4IJPTR Register Diagram

Table 35-67: PKTE_SA_ARC4IJPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:8 (R/W)	JPTR	J Pointer. The <code>PKTE_SA_ARC4IJPTR</code> . JPTR bit field contains the j pointer into s-box array for swapping bytes with i pointer.
7:0 (R/W)	IPTR	I Pointer. The <code>PKTE_SA_ARC4IJPTR</code> . IPTR bit field contains the i pointer into s-box array for swapping bytes with j pointer.

SA Command 0

The two SA command registers, `PKTE_SA_CMD0` and `PKTE_SA_CMD1`, are used to control the cryptographic operation of the packet engine. The `PKTE_SA_CMD0` register contains the major control bits to define an operation while the `PKTE_SA_CMD1` register contains the minor control bits. In direct host mode, this is a write-only register.

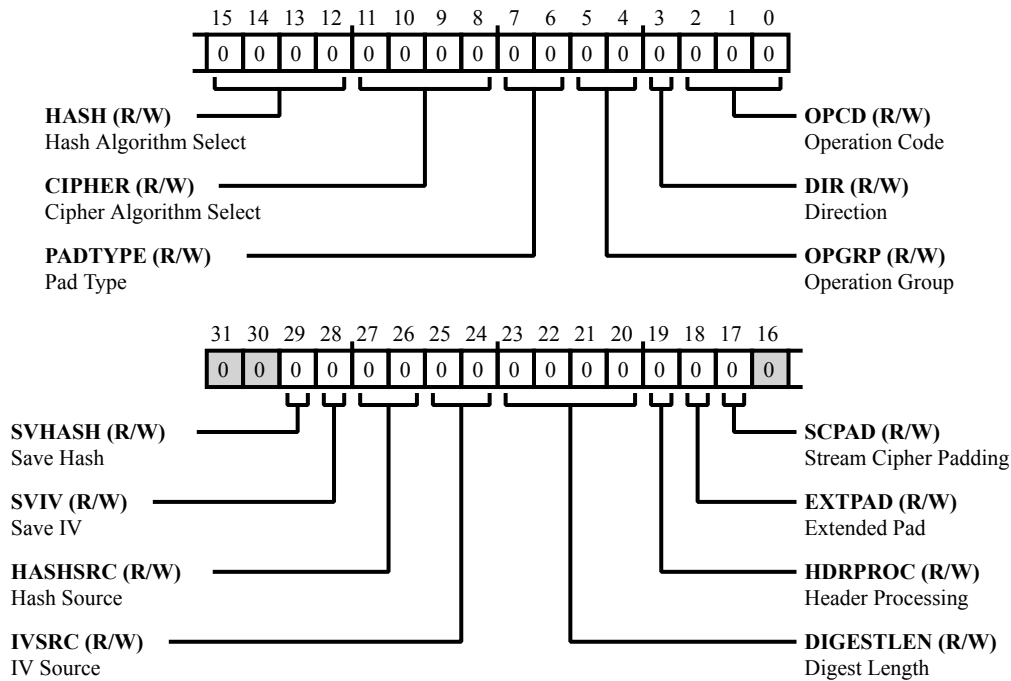


Figure 35-42: PKTE_SA_CMD0 Register Diagram

Table 35-68: PKTE_SA_CMD0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	SVHASH	Save Hash. The <code>PKTE_SA_CMD0.SVHASH</code> bit indicates that the Hash State is saved to the <code>STATE_BYTE_CNT_X</code> and <code>STATE_IDIGEST_X</code> fields in the SA record in memory after completion of a crypto operation.
		0 Hash state is not saved
		1 Hash state is saved

Table 35-68: PKTE_SA_CMD0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
28 (R/W)	SVIV	Save IV. The PKTE_SA_CMD0 . SVIV bit field indicates that for DES or the AES the Initialization Vector (IV) is saved to the STATE_IV_X fields in the state record , or the ARC4 state is saved to the ARC4 state record, after completion of the crypto operation.
		0 ARC4 State is not saved.
		1 ARC4 State is not saved.
27:26 (R/W)	HASHSRC	Hash Source. The PKTE_SA_CMD0 . HASHSRC bit field selects the source of the hash digest used by the algorithm.
		0 From SA. Digest only hash byte count is forced to 0x40.
		1 Reserved
		2 From State. Read saved inner hash digest and saved hash byte count.
		3 No Load. Use the hash algorithm defined constants for the initial hash. Hash byte count is 0x00.
25:24 (R/W)	IVSRC	IV Source. The PKTE_SA_CMD0 . IVSRC bit field selects the source of the initialization vector used by the crypto algorithm.
		0 No load. Use previous result IV, not applicable for inbound data. This option should never be used for operations with DES-CBC or AES-CBC, (see RFC3602) or any AES counter modes
		1 From input buffer. The IV is provided as part of the input data stream.
		2 From State. Read STATE_IV_X, from the SA structure. Refer to inner hash digest register structure. Useful for resume operations.
		3 From internal PRNG. Not applicable for inbound operations.
23:20 (R/W)	DIGESTLEN	Digest Length. The PKTE_SA_CMD0 . DIGESTLEN bit field defines the length of the hash digest in words as put in the output buffer.
		0 3 Words (96-bit output)
		1 1 Word
		2 2 Words

Table 35-68: PKTE_SA_CMD0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		3	3 Words (IPsec)
		4	4 Words (MD5 and AES-based hash)
		5	5 Words (SHA-1)
		6	6 Words
		7	7 Words (SHA-224)
		8	8 Words (SHA-256)
		9	Reserved
		10	10 bytes (SRTP and TLS)
		11-15	Reserved
19 (R/W)	HDRPROC	Header Processing. The <code>PKTE_SA_CMD0.HDRPROC</code> bit enables header processing for protocol operations. There is no header-processing support for basic SSL, basic TLS and SRTP protocol operations as defined in the protocol group (see the Crypto and Hash Algorithms section). This bit must be zero for these operations; however, the protocol header must be supplied to the packet engine since it is part of the hash calculation. Refer to the protocol specifications for more information about header-processing support for a protocol.	
		0	No header processing
		1	Header processing; insert the protocol header for out-bound operations, verify the protocol header for in-bound operations.
18 (R/W)	EXTPAD	Extended Pad. The <code>PKTE_SA_CMD0.EXTPAD</code> bit extends the number of padding types. Used in combination with <code>PKTE_SA_CMD0.PADTYPE</code> .	
17 (R/W)	SCPAD	Stream Cipher Padding. The <code>PKTE_SA_CMD0.SCPAD</code> bit enables padding for stream ciphers algorithms.	
15:12 (R/W)	HASH	Hash Algorithm Select. The <code>PKTE_SA_CMD0.HASH</code> bit field selects the hash algorithm.	
		0	MD5
		1	SHA-1
		2	SHA-224
		3	SHA-256
		4-14	Reserved
		15	Null

Table 35-68: PKTE_SA_CMD0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11:8 (R/W)	CIPHER	Cipher Algorithm Select. The PKTE_SA_CMD0.CIPHER bit field selects the cipher algorithm to be used for encryption and decryption. Note: Each type of protocol operation supports different sets of crypto algorithms. Refer to the Crypto and Hash Algorithms general processing section for details of the supported algorithms.
		0 DES
		1 Triple-DES
		2 ARC4
		3 AES
		4-14 Reserved
		15 Null
7:6 (R/W)	PADTYPE	Pad Type. The PKTE_SA_CMD0.PADTYPE bit field indicates the type of crypto that must be generated for outbound packets or checked for inbound packets.
		0 Select IPsec operation (if Bit 18=0); Reserved (if Bit 18=1)
		1 PKCS#7 (if Bit 18=0); Select TLS/DTLS Pad, required for TLS/DTLS operation (if Bit 18=1)
		2 Constant pad (if Bit 18=0); Select Constant SSL Pad, required for SSL operation (if Bit 18=1)
		3 Zero pad (if Bit 18=0), Reserved (if Bit 18=1)
5:4 (R/W)	OPGRP	Operation Group. The PKTE_SA_CMD0.OPGRP bit field defines the operation groups. Refer to the Basic Operations and Decoding section for more information.
		0 Basic operation group
		1 Protocol operation group
		2 Extended protocol operations group
		3 Reserved
3 (R/W)	DIR	Direction. The PKTE_SA_CMD0.DIR bit field selects the direction of operation.
		0 Outbound operations
		1 Inbound operations

Table 35-68: PKTE_SA_CMD0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	OPCD	Operation Code. The PKTE_SA_CMD0.OPCD bit field selects the operation within the operation group.

SA Command 1

The `PKTE_SA_CMD1` register contains the minor control bits that define an operation. In direct host mode, this is a write-only register.

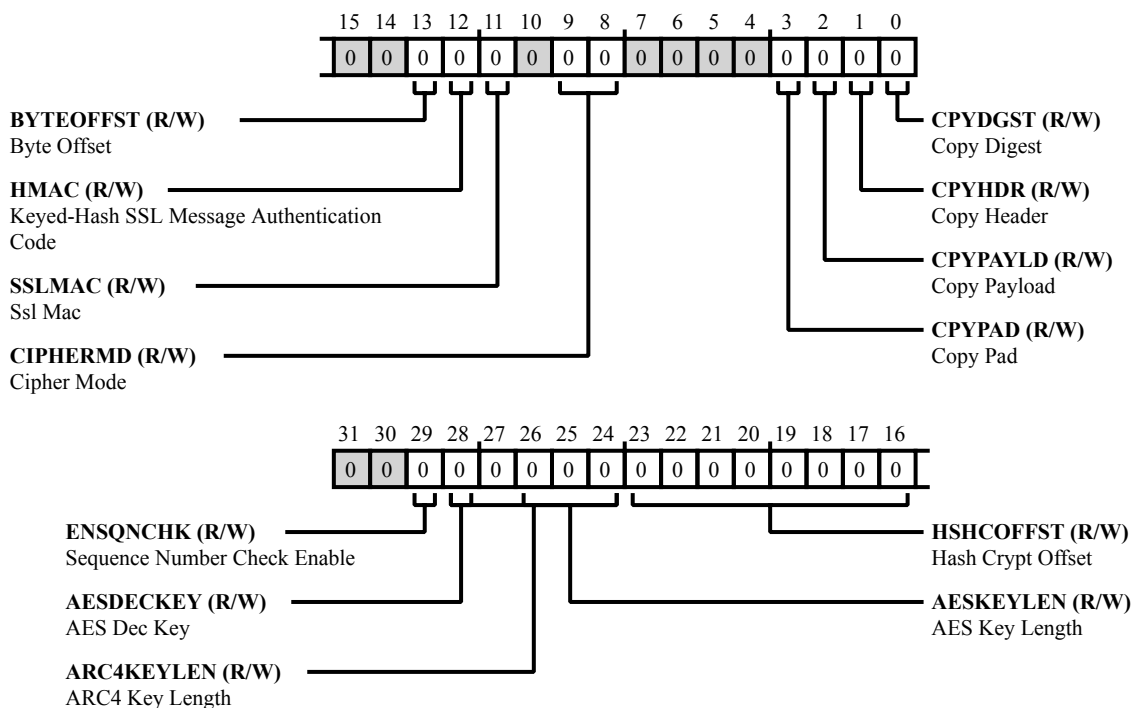


Figure 35-43: PKTE_SA_CMD1 Register Diagram

Table 35-69: PKTE_SA_CMD1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	ENSQNCHK	Sequence Number Check Enable. The <code>PKTE_SA_CMD1.ENSQNCHK</code> bit defines that the key in the SA key field is an AES encrypt key or an AES decrypt key.
		0 Disable sequence number check
		1 Enable sequence number check
28 (R/W)	AESDECKEY	AES Dec Key. If the <code>PKTE_SA_CMD1.AESDECKEY</code> bit is set, the key in loaded in the <code>PKTE_SA_KEY[n]</code> registers are expected to be the key from the last round from key expansion. If not set, the key loaded in the <code>PKTE_SA_KEY[n]</code> registers are expected to be the same key used during the encryption process.
		0 AES key is an encrypt key.
		1 AES key is a decrypt key.

Table 35-69: PKTE_SA_CMD1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
26:24 (R/W)	AESKEYLEN	AES Key Length. The PKTE_SA_CMD1.AESKEYLEN bit field select the size of the key used for the AES algorithm in increments of 64 bits.
		0-1 Reserved
		2 128 Bits
		3 192 Bits
		4 256 Bits
		5-7 Reserved
28:24 (R/W)	ARC4KEYLEN	ARC4 Key Length.
23:16 (R/W)	HSHCOFFST	Hash Crypt Offset. For Basic Encrypt-Hash and Basic Hash-Decrypt operations, the PKTE_SA_CMD1.HSHCOFFST bit field specifies the offset between the hash data and the encrypt/decrypt data. The data to be hashed is assumed to come first, with an offset to the beginning of encrypt/decrypt data. When PKTE_SA_CMD1.BYTEOFFST, bit 13, is zero, then the offset is defined in 32-bit words. When an initialization vector is loaded through the input buffer, valid values range from IV size to 255. In all other cases, valid values range from 0 to 255. When PKTE_SA_CMD1.BYTEOFFST, bit 13, is one, then the offset is defined in 8-bit bytes. When an initialization vector is loaded through the input buffer, valid values range from IV size to 255. In all other cases, valid values range from 4 to 255. (The IV size is two words for DES, Triple-DES and AES-CTR and four words for AES-CBC and AES-ICM operations). Other operations do not use these bits (a default value is applied by the packet engine).
13 (R/W)	BYTEOFFST	Byte Offset. The PKTE_SA_CMD1.BYTEOFFST bit defines how the PKTE_SA_CMD1.HSHCOFFST, bits of this register are used.
		0 HASH_CRYPT_OFFSET is defined in 32-bit words
		1 HASH_CRYPT_OFFSET is defined in 8-bit bytes
12 (R/W)	HMAC	Keyed-Hash SSL Message Authentication Code. For basic operations that include hashing, the PKTE_SA_CMD1.HMAC bit enables the HMAC processing, which calls for an extra outer hash operation.
		0 Standard Hash
		1 HMAC Processing

Table 35-69: PKTE_SA_CMD1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11 (R/W)	SSLMAC	Ssl Mac.
		0 Standard Hash
		1 SSL-MAC processing
9:8 (R/W)	CIPHERMD	Cipher Mode. The PKTE_SA_CMD1.CIPHERMD bit field selects the crypto mode to be used for the cipher algorithm.
		0 Electronic Code Book (ECB) used for DES and AES
		1 Cipher Block Chaining (CBC) used for DES and AES
		2 AES Counter Mode (CTR) for IPsec using a 32-bit counter
		3 AES Integer Counter Mode (ICM) for SRTP using a 16-bit counter. Note: This is implemented as a 32-bit counter, the Host must check for an overflow from the value 0xFF to zero and then refresh the IV.
3 (R/W)	CPYPAD	Copy Pad. The PKTE_SA_CMD1.CPYPAD bit indicates that the padding data for an inbound operation is copied to the output buffer and saved in memory.
		0 Do not copy the padding to output
		1 Copy padding to output
2 (R/W)	CPYPAYLD	Copy Payload. The PKTE_SA_CMD1.CPYPAYLD bit indicates that the payload data is copied to the output buffer and saved in memory.
		0 Do not copy the payload to output
		1 Copy payload to output
1 (R/W)	CPYHDR	Copy Header. The PKTE_SA_CMD1.CPYHDR bit indicates that the protocol header is copied to the output buffer and saved in memory. For Basic Encrypt-Hash and Basic Hash-Decrypt operations, the header is defined as the Hash/Crypt Offset data (authenticated only).
		0 Do not copy the header to output
		1 Copy header to output

Table 35-69: PKTE_SA_CMD1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	CPYDGST	Copy Digest. The PKTE_SA_CMD1.CPYDGST bit copies the hash result is to the output buffer and saves in memory. The length of the hash result is defined by the PKTE_SA_CMD0.DIGESTLEN field.
		0 Do not copy hash result to output
		1 Copy hash result to output, when the command descriptor PKTE_CTL_STAT.HASHFINAL bit is set.

SA Inner Hash Digest Registers

The `PKTE_SA_IDIGEST[n]` registers are a set of eight 32-bit read/write registers.

For MD5, SHA-1, SHA-224 and SHA-256, these read/write registers are used to enter a start hash state, and to read the interim or final hash digest.

For IPsec, TLS and DTLS operations that make use of MD5, SHA-1, SHA-224 or SHA-256 with basic hash or HMAC authentication with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the pre-computed inner hash digest. This is the hash of the hash-key padded with 0x36 hex. The starting hash byte count is automatically set to 64 decimal / 0x40 hex (to indicate that 64 bytes have already been processed through the hash).

For SSL operations that make use of SSL-MAC-MD5 with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the inner hash pre-compute; this is the hash of the `MAC_WRITE_SECRET` padded with 0x36 hex. The starting hash byte count is automatically set to 64 decimal / 0x40 hex (to indicate that 64 bytes have already been processed through the hash).

For SSL operations that make use of SSL-MAC-SHA-1 with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the `MAC_WRITE_SECRET`. Note that it is not possible to calculate a hash pre-compute for SHA-1 in combination with SSL-MAC (specification flaw). The packet engine appends the hash-key pad (0x36 hex) and sets the starting hash byte count automatically to 60 decimal / 0x3C hex (to indicate that 60 bytes have already been prepared for the hash).

The reset value for these registers is zero.

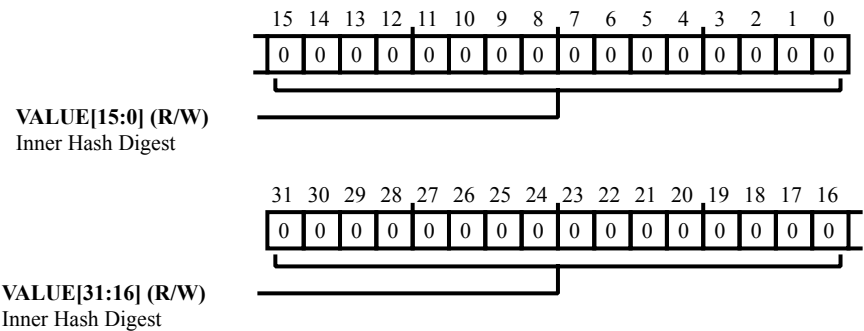


Figure 35-44: `PKTE_SA_IDIGEST[n]` Register Diagram

Table 35-70: `PKTE_SA_IDIGEST[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Inner Hash Digest.

SA Key Registers

These are the `PKTE_SA_KEY[n]` registers for DES, Triple-DES, ARC4 and AES: A set of eight 32-bit write only registers. The reset value of these registers is zero.

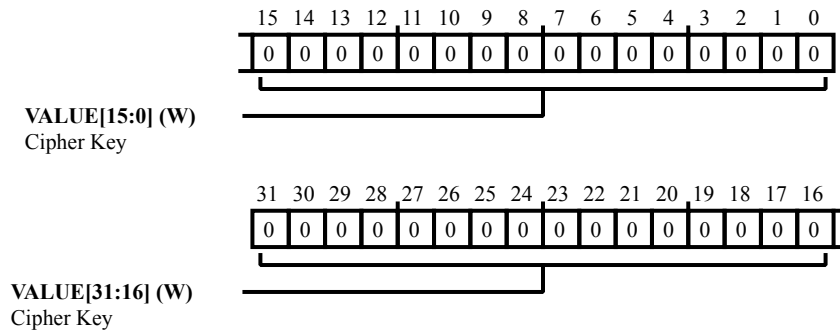


Figure 35-45: `PKTE_SA_KEY[n]` Register Diagram

Table 35-71: `PKTE_SA_KEY[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Cipher Key.

SA Initialization Vector Register

The `PKTE_SA_NONCE` register is used for operations that make use of the IV value loaded from the SA record. This register is used both to enter a starting IV state, as well as for reading the interim or final IV. For IPsec out-bound operations, it is recommended that the automatic IV insertion mode be used, this register is not needed. For IPsec inbound operations, the IV is extracted from the header of the packet.

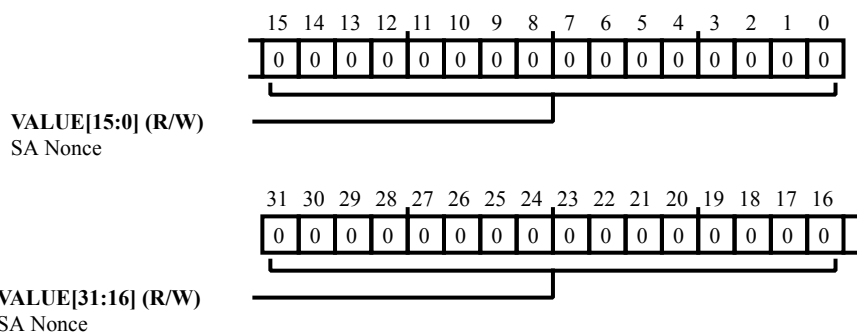


Figure 35-46: `PKTE_SA_NONCE` Register Diagram

Table 35-72: `PKTE_SA_NONCE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SA Nonce.

SA Outer Hash Digest Registers

The `PKTE_SA_ODIGEST[n]` registers are a set of five eight 32-bit write-only registers.

For write operations, these registers contain the pre-computed outer hash digest for IPsec operations with basic HMAC operations with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA).

For MD5, SHA-1, SHA-224 and SHA-256, these read/write registers hold a start hash state, or the interim outer hash digest. They are only used for HMAC processing.

For IPsec, SSL, TLS, DTLS and SRTP operations that make use of MD5, SHA-1, SHA-224 or SHA-256 with HMAC authentication with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the pre-computed outer hash digest. This is the hash of the hash-key padded with 0x5C hex. The starting hash byte count is automatically set to 64 decimal / 0x40 hex (to indicate that 64 bytes have already been processed through the hash).

For SSL operations that make use of SSL-MAC-MD5 with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the outer hash pre-compute; this is the hash of the `MAC_WRITE_SECRET` padded with 0x5C hex. The starting hash byte count is automatically set to 64 decimal / 0x40 hex (to indicate that 64 bytes have already been processed through the hash).

For SSL operations that make use of SSL-MAC-SHA-1 with the `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA), these registers hold the `MAC_WRITE_SECRET`. Note that it is not possible to calculate a hash pre-compute for SHA-1 in combination with SSL-MAC (specification flaw). The packet engine appends the required hash-key pad (0x5C hex) and sets the starting hash byte count automatically to 60 decimal / 0x3C hex (to indicate that 60 bytes have already been prepared for the hash).

The reset value for these registers is zero.

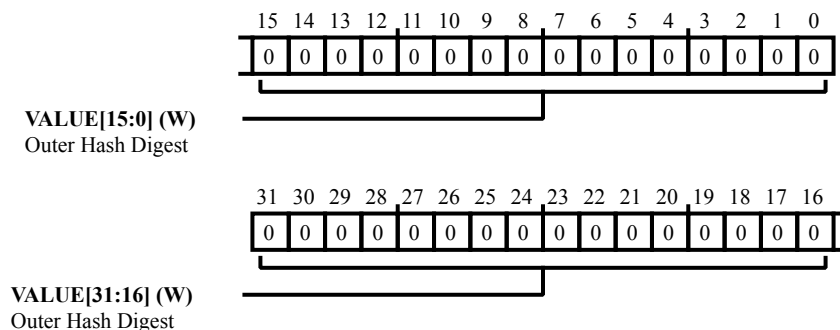


Figure 35-47: `PKTE_SA_ODIGEST[n]` Register Diagram

Table 35-73: `PKTE_SA_ODIGEST[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Outer Hash Digest.

SA Ready Indicator

In direct host mode, a write to the `PKTE_SA_RDY` register triggers the packet engine to start processing using the command descriptor, SA record and state record in the packet engine registers. This register **MUST** be written for all direct host mode packet operations. It is intended that this register is written in sequence; as the entire SA record is written.

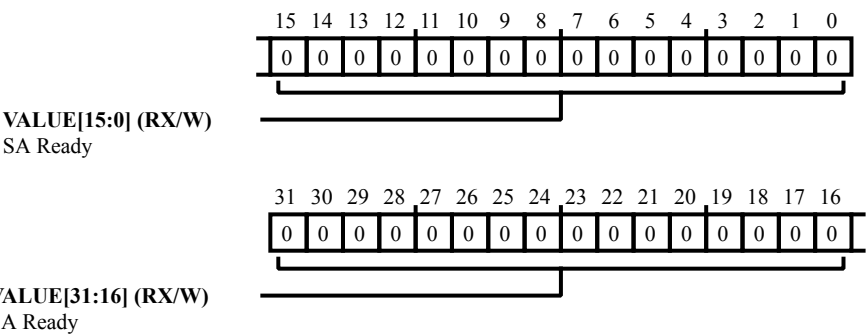


Figure 35-48: `PKTE_SA_RDY` Register Diagram

Table 35-74: `PKTE_SA_RDY` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	SA Ready.

SA Sequence Number Register

The `PKTE_SA_SEQNUM[n]` registers are a set of two read/write registers and are used for IPsec ESP, SSL, TLS, DTLS operations to specify the anti-replay sequence number value that is to be placed in the ESP header (out-bound), or to be checked against for inbound packets. The packet engine manages this counter value for both in-bound and outbound operations.

Outbound: The host writes the counter value stored in the SA record to this register to start an IPsec, SSL, TLS, DTLS operation. The packet engine automatically increments the count if header processing is selected. Upon successful completion, the host reads back this value and writes it to the SA record.

Inbound: The host writes the counter value stored in the SA record to this register to start an IPsec or DTLS operation. The packet engine automatically performs the specified inbound processing (per RFC 4303) as it processes the packet. As a result, the expected count value may or may not be updated during processing. Upon successful completion, the host should read back this value and write it to the SA record.

Note: The description is only for the direct host mode. The sequence number for autonomous ring mode and target command mode are updated by the packet engine.

The reset value of this register is zero.

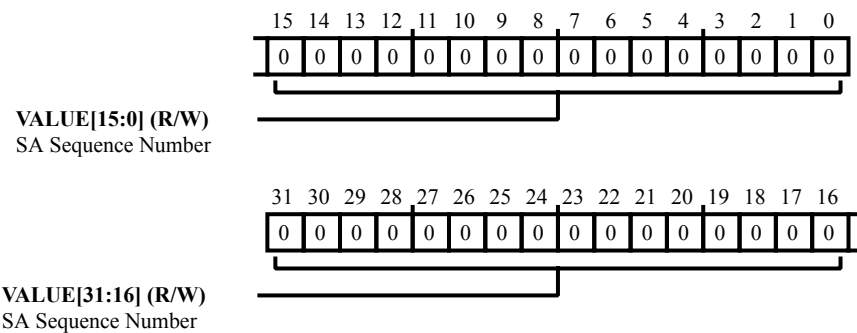


Figure 35-49: `PKTE_SA_SEQNUM[n]` Register Diagram

Table 35-75: `PKTE_SA_SEQNUM[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SA Sequence Number.

SA Sequence Number Mask Registers

The `PKTE_SA_SEQNUM_MSK[n]` registers are a set of two read/write registers and are used for IPsec ESP and DTLS operations to specify the anti-replay sequence number mask value for inbound operations. The packet engine manages this counter value automatically.

Inbound: The host writes the counter value stored in the SA record into this register upon starting an IPsec, DTLS operation. The packet engine automatically performs the specified inbound processing (per RFC 4303) as it processes the packet. As a result, the new mask value may or may not be updated during processing. Upon successful completion, the host should read back this value and write it to the SA record.

Outbound: not used.

Note that the above description only applies to the direct host mode, for autonomous ring mode and target command mode the packet engine extracts the sequence number mask from the SA record.

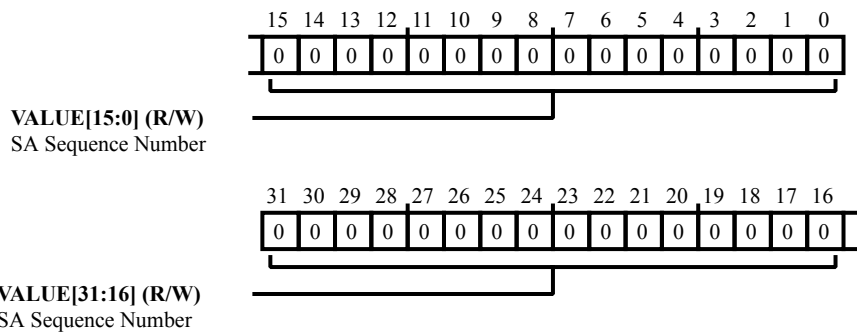


Figure 35-50: `PKTE_SA_SEQNUM_MSK[n]` Register Diagram

Table 35-76: `PKTE_SA_SEQNUM_MSK[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SA Sequence Number.

SA SPI Register

For IPsec operations, the `PKTE_SA_SPI` register is written with the SPI (Security Parameters Index) associated with the inbound or outbound flow.

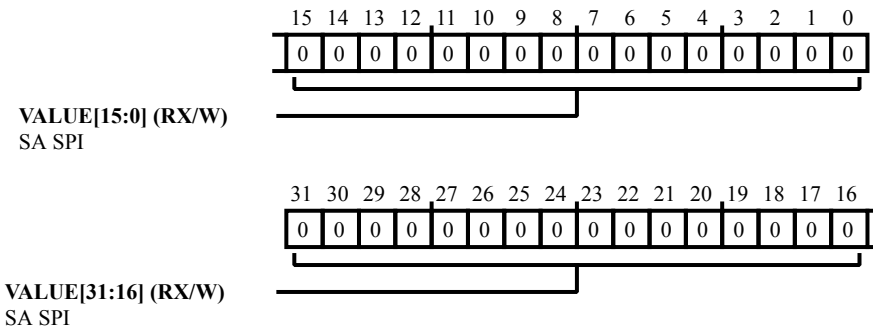


Figure 35-51: PKTE_SA_SPI Register Diagram

Table 35-77: PKTE_SA_SPI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	SA SPI. The <code>PKTE_SA_SPI.VALUE</code> bit field is used for IPsec ESP operations to specify the Security Parameters Index (SPI) value that is to be placed in the ESP header. There is no need to read back this value at the end of an operation, since the Packet Engine does not change it. For SSL, TLS and DTLS this register stores the 8-bit TYPE field in bits [23:16], and the 16-bit Version field in bits [15:0] that are part of the protocol header.

Packet Engine Source Address

The `PKTE_SRC_ADDR` register holds the starting (byte) address for the packet to be processed.

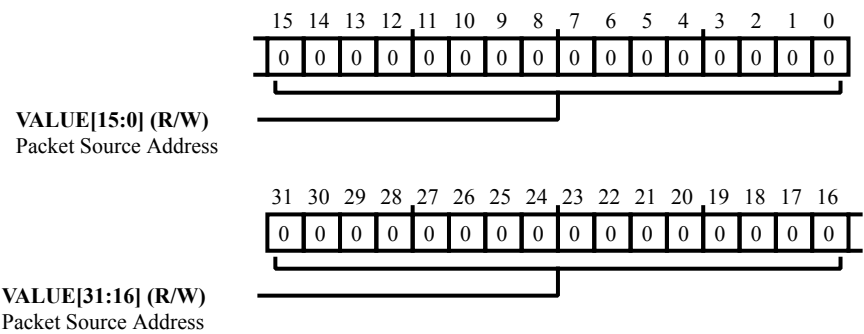


Figure 35-52: `PKTE_SRC_ADDR` Register Diagram

Table 35-78: `PKTE_SRC_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Packet Source Address. The <code>PKTE_SRC_ADDR.VALUE</code> bit field holds the starting (byte) address for the packet to be processed.

Packet Engine Status Register

The `PKTE_STAT` register is used to provide the status of the packet engine. This register is useful in the direct host mode to determine when data must be written to or read from the packet engine, or for debugging the software when errors occur. This register can be ignored in autonomous ring mode and target command mode where the DMA engine controls the packet data I/O. This is a read-only register. A write to any of the bits has no effect.

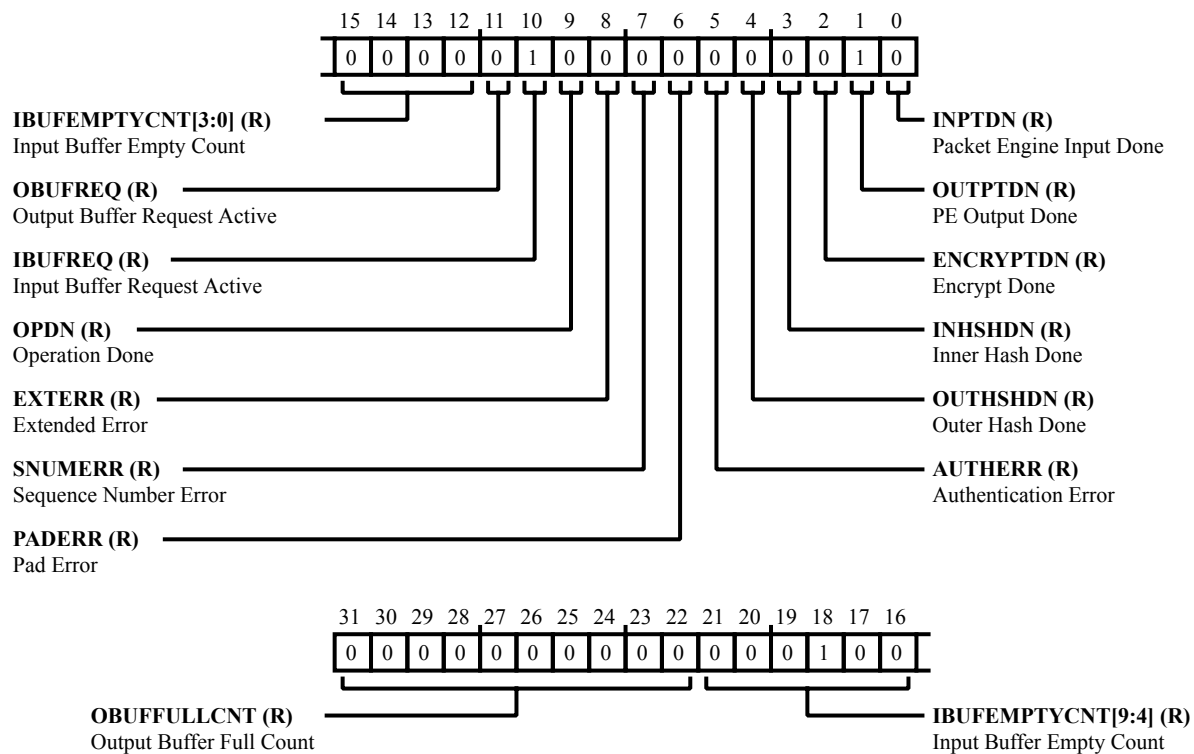


Figure 35-53: PKTE_STAT Register Diagram

Table 35-79: PKTE_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:22 (R/NW)	OBUFFULLCNT	Output Buffer Full Count. The <code>PKTE_STAT.OBUFFULLCNT</code> bit field indicates the number of 32-bit words that are available in the packet engine output buffer. It works in conjunction with bit 11 from this register. When bit 11 is asserted, to indicate a request for output, the word count matches the specified output buffer threshold setting in the <code>PKTE_BUF_THRESH</code> register. For the last output for a given packet, any value from 1 dword to the full output buffer threshold can be seen. Transfers must be a multiple of full dwords. The application must read the <code>PKTE_LEN</code> field in the result descriptor to determine the exact byte-length of the result.

Table 35-79: PKTE_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
21:12 (R/NW)	IBUFEMPTYCNT	Input Buffer Empty Count. The <code>PKTE_STAT . IBUFEMPTYCNT</code> bit field indicates the number of 32-bit empty spaces that are available in the packet engine input buffer. It works in conjunction with the <code>PKTE_STAT . IBUFREQ</code> bit (10) from this register. The value in the register is deducted from the specified packet length, so will never exceed the number of dwords that remain in the packet. For packets smaller than the buffer size, this register typically indicates that buffer space is available for the entire packet (rounded up to the nearest dword). For very large packets, these bits usually have a value around the maximum buffer size, indicating that the full input buffer is available.
11 (R/NW)	OBUFREQ	Output Buffer Request Active. The <code>PKTE_STAT . OBUFREQ</code> bit indicates that the packet engine requests output data to be read from the output buffer.
		0 No request for output data
		1 Request for output data
10 (R/NW)	IBUFREQ	Input Buffer Request Active. The <code>PKTE_STAT . IBUFREQ</code> bit indicates that the packet engine requests input data to be written to the input buffer.
		0 No request for input data
		1 Request for input data
9 (R/NW)	OPDN	Operation Done. The <code>PKTE_STAT . OPDN</code> bit indicates that the packet engine has finished processing a packet when in direct host mode. This bit is zero in autonomous ring mode and target command mode.
		0 Packet engine is idle
		1 Packet engine has finished processing a packet
8 (R/NW)	EXTERR	Extended Error. The <code>PKTE_STAT . EXTERR</code> bit indicates that an extended error occurred for this packet. For more information, refer to table Extended Error Codes - Status Encoding.
		0 No extended error
		1 Extended error

Table 35-79: PKTE_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	SNUMERR	Sequence Number Error. For an inbound operation, the <code>PKTE_STAT.SNUMERR</code> bit indicates that there was a fault in the anti-replay sequence number. For an outbound operation, there was a sequence number overflow condition. For more information, refer to table Extended Error Codes - Status Encoding.
		0 No sequence number error
		1 Input done, all bytes written to input buffer
6 (R/NW)	PADERR	Pad Error. The <code>PKTE_STAT.PADERR</code> bit indicates that an inbound crypto pad fault is detected. For more information about pad verification, refer to the Pad Verification and Consumption section.
		0 No pad error
		1 Pad error
5 (R/NW)	AUTHERR	Authentication Error. The <code>PKTE_STAT.AUTHERR</code> bit indicates that an inbound ICV (for IPsec) or TAG (for SRTP) or MAC (for SSL/TLS/DTLS) fault is detected; the value carried within the packet did not match the value just computed.
		0 No authentication error
		1 Authentication error
4 (R/NW)	OUTHSHDN	Outer Hash Done. The <code>PKTE_STAT.OUTHSHDN</code> bit indicates that the outer hash processing for this packet is finished.
		0 Outer hash busy
		1 Outer hash done
3 (R/NW)	INHSHDN	Inner Hash Done. The <code>PKTE_STAT.INHSHDN</code> bit indicates that the inner hash processing for this packet is finished.
		0 Inner hash busy
		1 Inner hash done
2 (R/NW)	ENCRYPTDN	Encrypt Done. The <code>PKTE_STAT.ENCRYPTDN</code> bit indicates that the encryption or decryption for this packet is finished.
		0 Encryption or decryption busy
		1 Encryption or decryption done

Table 35-79: PKTE_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	OUTPTDN	PE Output Done. The <code>PKTE_STAT.OUTPTDN</code> bit indicates that the output data for the current packet is read from the packet engine output buffer.
		0 Output not done, more output bytes available
		1 Output done, all bytes read from the output buffer
0 (R/NW)	INPTDN	Packet Engine Input Done. The <code>PKTE_STAT.INPTDN</code> bit indicates that the number of bytes specified in the command descriptor <code>PKTE_LEN</code> field is written into the packet engine input buffer.
		0 Input not done, more input bytes expected
		1 Input done, all bytes written to input buffer

Packet Engine State Record Address

The `PKTE_STATE_ADDR` register holds the start address of the SA state record.

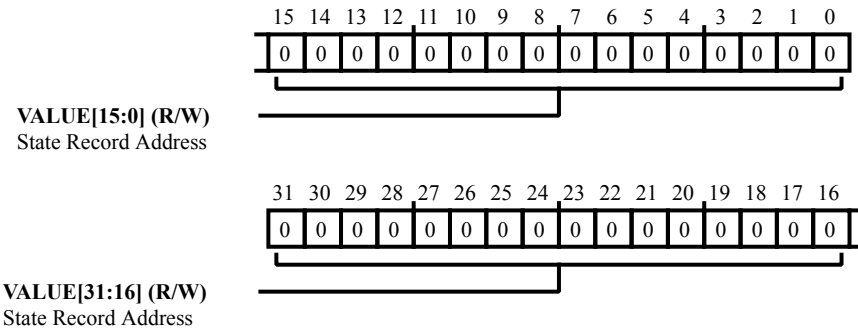


Figure 35-54: `PKTE_STATE_ADDR` Register Diagram

Table 35-80: `PKTE_STATE_ADDR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	State Record Address. The <code>PKTE_STATE_ADDR.VALUE</code> bit field holds the start address of the SA state record.

State Hash Byte Count Registers

The `PKTE_STATE_BYTE_CNT[n]` registers are used to enter a starting hash byte count, as well as to read the interim or final byte count.

For some hash operations, these registers are ignored and the byte count is internally set to 64 (0x40 hex) to indicate that the first 64 bytes (512 bits) hash block has been processed using a pre-computed hash state. These operations are:

All IPsec, SSL, TLS, DTLS and SRTP operations that use authentication; the "pre-computed" inner and outer hash digests are loaded from SA words 10 - 19.

Basic operations with `PKTE_SA_CMD0.HASHSRC` bits = 00 (from SA) specified. For Basic Hash with no HMAC, a pre-computed digest is loaded from SA words 10 - 14. For Basic Hash with HMAC, the inner and outer digests are loaded from SA words 10 - 19.

Note: Protocol operations can not be suspended in mid-packet and resumed later, therefore protocol operations do not use these registers.

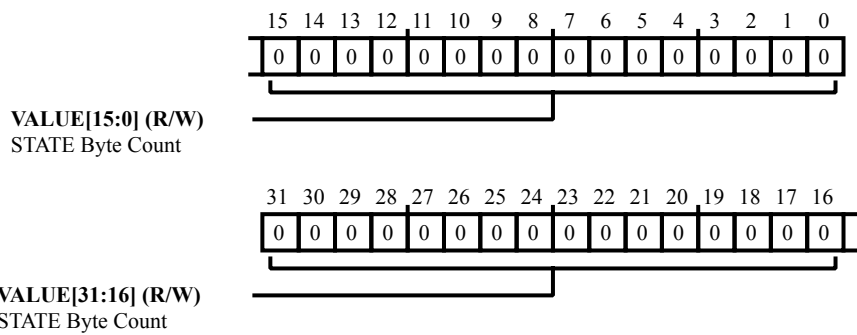


Figure 35-55: `PKTE_STATE_BYTE_CNT[n]` Register Diagram

Table 35-81: `PKTE_STATE_BYTE_CNT[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	STATE Byte Count.

State Inner Digest Registers

The `PKTE_STATE_IDIGEST[n]` registers consist of eight 32-bit registers. These read/write registers are used to read the interim or final hash digest. The `PKTE_STATE_IDIGEST[n]` registers are only used with basic operations involving basic hash, and are typically used for operations that must be suspended and resumed in the middle of a hash. The interim hash state can be read from these registers along with the hash byte-count from the previous register. Both can be restored when resuming the hash. The appropriate save hash state (`PKTE_SA_CMD0.SVHASH=1`) and load hash from state (`PKTE_SA_CMD0.HASHSRC=0b10`) settings must be used. These registers are a mirror of the SA record inner hash digest register.

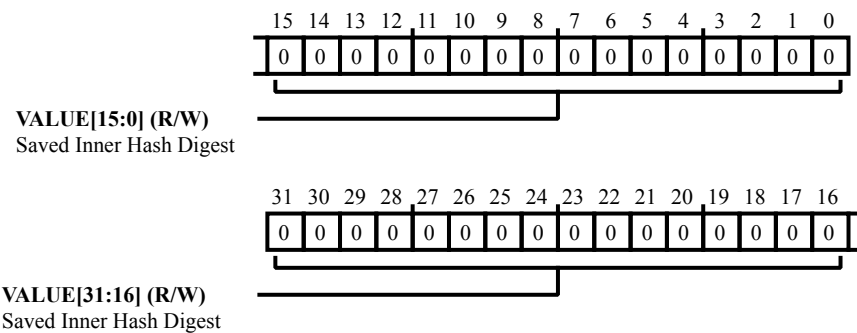


Figure 35-56: `PKTE_STATE_IDIGEST[n]` Register Diagram

Table 35-82: `PKTE_STATE_IDIGEST[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Saved Inner Hash Digest.

State Initialization Vector Registers

The `PKTE_STATE_IV[n]` consists of four 32-bit registers. These registers are used to enter a starting IV state and to read the interim or final IV. `PKTE_STATE_IV0` and `PKTE_STATE_IV1` are used with DES/3DES cipher while `PKTE_STATE_IV0` to `PKTE_STATE_IV3` are used with AES cipher. The reset value of these registers is zero.

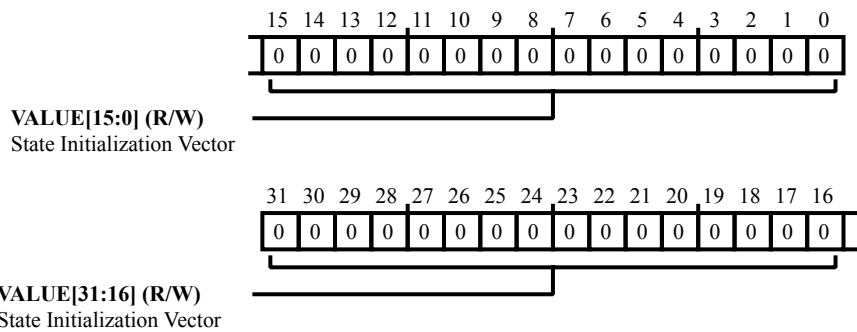


Figure 35-57: `PKTE_STATE_IV[n]` Register Diagram

Table 35-83: `PKTE_STATE_IV[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	State Initialization Vector. The <code>PKTE_STATE_IV[n].VALUE</code> bit field is used to enter a starting IV state and to read the interim or final IV.

Packet Engine User ID

The `PKTE_USERID` register is a read/write register that gives identification to a command descriptor and the resultant result descriptor. The host is free to use this field for its own purpose. The host can write a unique identifier to the register in direct host mode or includes it as part of the command descriptor in autonomous ring mode. The `PKTE_USERID` register value passes through the packet engine without alteration to the result descriptor to be read back by the host.

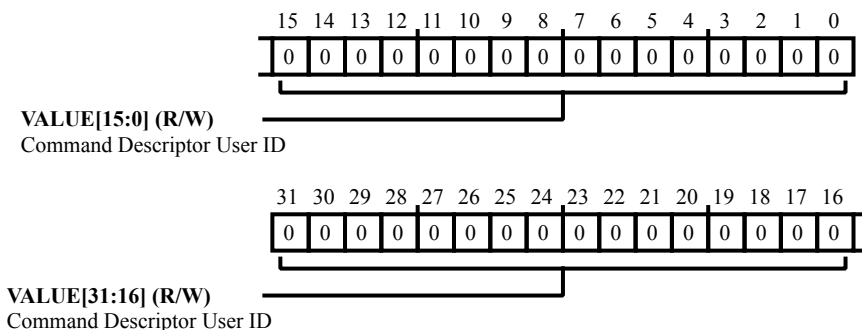


Figure 35-58: `PKTE_USERID` Register Diagram

Table 35-84: `PKTE_USERID` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Command Descriptor User ID. The <code>PKTE_USERID.VALUE</code> bit field gives identification to a command descriptor and the resultant result descriptor.

36 Public Key Accelerator (PKA)

The PKA helps offload computationally-intensive operations commonly found in public key cryptography algorithms.

PKA Features

The PKA engine provides the following basic operations:

- Large vector addition, subtraction, and combined addition/subtraction
- Large vector shift right or left
- Large vector multiplication, division (with and without quotient)
- Large vector compare and copy

The PKA engine provides the following complex operations:

- Large vector unsigned value modular exponentiation
- Large vector unsigned value modular exponentiation using the ‘Chinese Remainders Theorem’ (CRT) method with pre-calculated Q inverse vector
- Modular inversion: Given A and M, calculate B such that $((A \times B) \text{ MOD } M) = 1$
- ECC point addition/doubling on elliptic curve $y^2 = x^3 + ax + b \pmod{p}$ with prime number p and input values a and b to the operation. Adding two identical points automatically performs point doubling.
- ECC point multiplication on elliptic curve $y^2 = x^3 + ax + b \pmod{p}$ with prime number p and input values a and b to the operation. A version of the ‘Montgomery ladder’ algorithm is used to provide side channel attack resistance.

The PKA also contains hardware logic to automatically zero out the PKA RAM buffer to clear out any information that is considered sensitive or secure.

PKA Functional Description

The following sections provide details on the function of the PKA module.

ADSP-2159x PKA Register List

The Public Key Accelerator module (PKA) provides security-related features. A set of registers governs PKA operations. For more information on PKA functionality, see the PKA register descriptions.

Table 36-1: ADSP-2159x PKA Register List

Name	Description
PKA_ALEN	PKA Vector_A Length
PKA_APTR	PKA Vector_A Address
PKA_BLEN	PKA Vector_B Length
PKA_BPTR	PKA Vector_B Address
PKA_COMPARE	PKA Compare Result
PKA_CPTR	PKA Vector_C Address
PKA_DIVMSW	PKA Most-Significant-Word of Divide Remainder
PKA_DPTR	PKA Vector_D Address
PKA_FUNC	PKA Function
PKA_RAM	Start of PKA RAM space
PKA_RESULTMSW	PKA Most-Significant-Word of Result Vector
PKA_SHIFT	PKA Bit Shift Value

PKA Definitions

The following definitions are helpful when using the PKA module.

Elliptic Curve Cryptography (ECC)

A form of public key cryptography based on elliptic curves over finite fields.

RSA

An acronym for Ron Rivest, Adi Shamir, and Leonard Adleman. It is another form of a public key cryptosystem.

Chinese Remainder Theorem (CRT)

A mathematical theorem used for simplifying time-consuming arithmetic used in public key algorithm computations.

Addition Chaining Table (ACT)

A method of speeding up exponentiation by repeatedly squaring the input and storing the result and reusing the result as input. ACT2 uses a table with 2 address bits (4 entries) and ACT4 uses a table with 4 address bits (16 entries).

PKA Architectural Concepts

The following sections describe the PKA architecture.

Public Key Co-Processor (PKCP)

The Public Key Co-Processor (PKCP) handles the basic large vector processing such as addition, subtraction, multiplication, etc.

Sequencer

The sequencer is small processor that is part of the PKA which handles the more complicated vector processing for public key algorithms. Algorithms include modular exponentiation and the ECC addition and ECC multiply used in Elliptic Curve Cipher algorithms. It executes instructions stored from an internal pre-programmed ROM that handles these operations.

RAM

Input and output vectors are stored in a 4 kB RAM buffer that is part of the MMR space. The address of PKA_RAM is the beginning of the RAM space. This memory is also used as a scratchpad or workspace for the sequencer and PKCP. Programs must place the vectors appropriately following the constraints described in the *Functional Description* section of this chapter.

PKA Block Diagram

The *PKA Block Diagram* shows the top-level block diagram of the PKA engine. The PKA engine is comprised of five parts:

1. Registers for input, output, status, and control
2. Public Key Co-processor (PKCP) module which performs the basic suite of big number (vector) operations typically found in public key cryptography applications
3. Sequencer which controls modular exponentiation, elliptic curve cryptography, and modular inversion operations.
4. Program ROM associated with the PKA engine exclusively for the sequencer
5. PKA RAM holds the large input and output values as well as the workspace/scratchpad required from the sequencer and PKCP for operations.

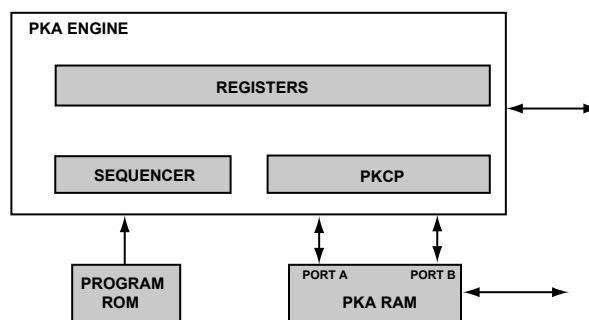


Figure 36-1: PKA Block Diagram

PKCP Vector Operations

The *Summary of PKCP Vector Operations* table lists the arguments and results for each PKCP vector operation.

Table 36-2: Summary of PKCP Vector Operations

Function	Mathematical Operation	Vector A	Vector B	Vector C	Vector D
Multiply	$A \times B \rightarrow C$	Multiplicand	Multiplier	Product	N/A
Add	$A + B \rightarrow C$	Addend	Addend	Sum	N/A
Subtract	$A - B \rightarrow C$	Minuend	Subtracthend	Difference	N/A
AddSub	$A + C - B \rightarrow D$	Addend	Subtracthend	Addend	Result
Right Shift	$A \gg \text{Shift} \rightarrow C$	Input	N/A	Result	N/A
Left Shift	$A \ll \text{Shift} \rightarrow C$	Input	N/A	Result	N/A
Divide	$A \bmod B \rightarrow C$, $A \div B \rightarrow D$	Dividend	Divisor	Remainder	Quotient
Modulo	$A \bmod B \rightarrow C$	Dividend	Divisor	Remainder	N/A
Compare	$A = B$, $A < B$, $A > B$	Input 1	Input 2	N/A	N/A
Copy	$A \rightarrow C$	Input	N/A	Result	N/A

To obtain correct result, the input vectors must meet the requirements presented in the *Operational Restrictions on Input Vectors for PKCP Operations* table.

Note the following:

- The PKCP does not check input restrictions
- A_Len and B_Len indicate the size of vectors A and B in (32-bit) words
- Max_Len equals 128 (32-bit) words, for example, the standard maximum vector size is 4096 bit

Table 36-3: Operational Restrictions on Input Vectors for PKCP Operations

Function	Requirement
Multiply	$0 < A_Len, B_Len \leq Max_Len$
Add	$0 < A_Len, B_Len \leq Max_Len$
Subtract	$0 < A_Len, B_Len \leq Max_Len$ Result must be positive ($A > B$)
AddSub	$0 < A_Len \leq Max_Len$ (B and C operands have A_Len as length, B_Len ignored) Result must be positive ($(A + C) > B$)
Right Shift	$0 < A_Len \leq Max_Len$
Left Shift	$0 < A_Len \leq Max_Len$
Divide, Modulo	$1 < B_Len \leq A_Len \leq Max_Len$ Most significant 32-bit word of B operand cannot be zero
Compare	$0 < A_Len \leq Max_Len$ (B operand has A_Len as length, B_Len ignored)
Copy	$0 < A_Len \leq Max_Len$

The host processor is responsible for allocating a block of contiguous memory in PKA RAM for the result vectors. The *PKCP Result Vector Memory Allocation* table indicates how much memory is allocated for the result vectors.

Table 36-4: PKCP Result Vector Memory Allocation

Function	Result Vector	Result Vector Length (in 32-bit words)
Multiply	C	$A_Len + B_Len + 6$ (the 6 'scratchpad' words should be discarded)
Add	C	$Max(A_Len, B_Len) + 1$
Subtract	C	$Max(A_Len, B_Len)$
AddSub	D	$A_Len + 1$
Right Shift	C	A_Len
Left Shift	C	$A_Len + 1$ (when Shift Value is non-zero) A_Len (when Shift Value is zero)
Divide	C	Remainder $\rightarrow B_Len + 1$ (one 'scratchpad' word should be discarded)
	D	Quotient $\rightarrow A_Len - B_Len + 1$
Modulo	C	Remainder $\rightarrow B_Len + 1$ (one 'scratchpad' word should be discarded)
Compare	None	Compare updates the PKA_COMPARE register
Copy	C	A_Len

Input vectors for an operation are always allowed to overlap in memory (partially or completely). The *PKCP Result Vector/Input Overlap Restrictions* table gives restrictions for the overlap of output and input vectors of the operations.

Table 36-5: PKCP Result Vector/Input Overlap Restrictions

Function	Result Vector	Restrictions
Multiply	C	No overlap with A or B vectors allowed
Add, Subtract	C	May overlap with A and/or B vector, provided the start address of the C vector does not lie above the start address of the vectors with which it overlaps
AddSub	D	May overlap with A, B and/or C vector, provided the start address of the D vector does not lie above the start address of the vectors with which it overlaps
Right Shift, Left Shift	C	May overlap with A vector, provided the start address of the C vector does not lie above the start address of the A vector
Divide	C	No overlap with A, B, or D vectors allowed
	D	No overlap with A, B, or C vectors allowed
Modulo	C	No overlap with A or B vectors allowed
Compare	None	Compare does not write a result vector
Copy	None	Same restrictions as for right or left shift, copy of a vector to a lower address is always allowed even if source and destination overlap†

†The copy operation can be used to fill memory by breaking the overlap restrictions, but it requires setting up TWO initial (32-bit) words. To zero a block of memory, set the A vector pointer to the block start, set the C vector pointer two words higher and the A vector length to the block length minus two (words). Fill the first two words of the block with constant zero and perform a PKCP copy operation to zero the remainder of the block.

Modular Exponentiation Operations

The *Summary of ExpMod Operations* table summarizes the modular exponentiation operations that the PKA supports.

Table 36-6: Summary of ExpMod Operations

Function	Mathematical Operation	Vector A	Vector B	Vector C	Vector D
ExpMod-ACT2 ExpMod-ACT4 ExpMod-variable	$C^A \bmod B \rightarrow D$	Exponent, length = A_Len	Modulus, length = B_Len	Base, length = B_Len	Result and Workspace
ExpMod-CRT	See below	Exp P followed by Exp Q at next higher even word address†, both A_Len long	Mod P + buffer word followed by Mod Q at next higher even word address†, both B_Len long	Q inverse, length = B_Len	Input, Result (both 2xB_Len long) and Workspace

† If A_Len is even, Exp Q follows Exp P immediately – if A_Len is odd, there is one empty word between Exp Q and Exp P.

‡ If B_Len is even, there are two empty words between Mod P and Mod Q – if B_Len is odd, there is one empty (buffer) word between Mod Q and Mod P. Note that the engine may zero the words following Mod P and Mod Q.

The ExpMod-CRT operation performs the following computation steps. (These steps implement Garner's recombination algorithm after the basic exponentiations.)

- $X \leftarrow (\text{Input mod Mod P}) \text{Exp P mod Mod P}$
- $Y \leftarrow (\text{Input mod Mod Q}) \text{Exp Q mod Mod Q}$
- $Z \leftarrow (((X - Y) \text{ mod Mod P}) \cdot Q \text{ inverse}) \text{ mod Mod P} \cdot \text{Mod Q}$
- $\text{Result} \leftarrow Y + Z$

The ExpMod-ACT2, -ACT4, and -variable functions implement the same mathematical operation but with a differently sized table with pre-calculated *odd powers*. The ExpMod-ACT2 function uses a table with two entries whereas ExpMod-ACT4 uses a table with eight entries. The ACT4 version gives better performance but needs more memory. ExpMod-variable and ExpMod-CRT operations allow the selection of a variable number (from 1 up to and including 16) of odd powers through the register normally used to specify the number of bits to shift for shift operations.

The exponentiation functions are extensions of the set of PKA functions. Input and result vectors are passed the same way as basic PKCP operations. The *Restrictions on Input Vectors for ExpMod Operations* table shows the restrictions on the input and result vectors for the exponentiation operations.

Table 36-7: Restrictions on Input Vectors for ExpMod Operations

Function	Requirements
ExpMod-ACT2 ExpMod-ACT4 ExpMod-variable	$0 < A_Len \leq \text{Max_Len}$ $1 < B_Len \leq \text{Max_Len}$ Modulus B must be odd (for example, the least significant bit must be ONE) Modulus $B > 2^{32}$ Base $C < \text{Modulus B}$ Vectors B and C must be followed by an empty 32-bit <i>buffer</i> word
ExpMod-CRT	$0 < A_Len \leq \text{Max_Len}$ $1 < B_Len \leq \text{Max_Len}$ Mod P and Mod Q must be odd (for example, the least significant bits must be ONE) $\text{Mod P} > \text{Mod Q} > 2^{32}$ (note that Mod P must be larger than Mod Q) Mod P and Mod Q must be co-prime (their GCD must be 1) $0 < \text{Exp P} < (\text{Mod P} - 1)$ $0 < \text{Exp Q} < (\text{Mod Q} - 1)$ $(Q \text{ inverse} \cdot \text{Mod Q}) \cdot 1 \pmod{\text{Mod P}}$ Input $< (\text{Mod P} \cdot \text{Mod Q})$ Mod P and Mod Q must be followed by an empty 32-bit <i>buffer</i> word

The *ExpMod Result Vector/Scratchpad Area Memory Allocation Starting at PKA_DPTR* table shows the required scratchpad sizes for the exponentiation operations. These sizes depend on the PKA type. The ‘M_Len’ used in the table is the ‘real’ Modulus length in 32-bit words, for example, without trailing zero words at the end. (This description also applies to Mod P in an ExpMod-CRT operation and Modulus B in the other operations.) If the last word of the modulus vector as given is non-zero, ‘M_Len’ equals B_Len.

Table 36-8: ExpMod Result Vector/Scratchpad Area Memory Allocation Starting at PKA_DPTR

Function	Scratchpad Area Size (in 32-bit words), Result Vector is either M_Len or 2xM_Len 32-bit words long
ExpMod-ACT2	5 x (M_Len + 2)
ExpMod-ACT4	11 x (M_Len + 2)
ExpMod-variable	(# odd powers + 3) x (M_Len + 2)
ExpMod-CRT	(# odd powers + 3) x (M_Len + 2) + (M_Len + 2 – (M_Len MOD 2))

NOTE: During execution of an ExpMod-ACT2, -ACT4 or -variable operation, the last 34 bytes of the PKA RAM are used as the general scratchpad for the sequencer program execution. The ExpMod-CRT operation requires the last 72 bytes of the PKA RAM as the scratchpad. These (fixed location) areas may not overlap with any of the input vectors and/or the D vector scratchpad area. They can be used freely when executing basic PKCP operations.

Table 36-9: ExpMod Scratchpad Area / Input Vector Overlap Restrictions

Function	Result Vector	Restrictions
ExpMod-ACT2 ExpMod-ACT4 ExpMod-variable	D	Scratchpad area starting at D may not overlap with any of the other vectors, except that base C may be co-located with result vector D to save space (for example, <code>PKA_CPTR = PKA_DPTR</code> is allowed).
ExpMod-CRT	D	Scratchpad area starting at D may not overlap with any of the other vectors. This area is also the location of the main input vector (with length 2 x B_Len)

The *Maximum Number of Odd Powers* table indicates the maximum number of odd powers that can be used for different standard PKA RAM sizes and PKA types (non-CRT operations using `PKA_CPTR = PKA_DPTR`). As a rule of thumb, for optimal performance, use one odd power for *Verify* operations and 4 (or as many as the implemented PKA RAM size allows) for *Sign* operations. Note the following points about odd powers:

- Using more than eight odd powers is not advisable as the speed advantage for each extra odd power decreases rapidly (and can even become negative for short exponent vector lengths due to the extra pre-processing required).
- The maximum number of odd powers is 16 (limited by the firmware). All ‘16 odd powers’ entries in the table above hit this limit – they are not limited by the PKA RAM size.

Table 36-10: Maximum Number of Odd Powers

Operation	Modulus and Exponent Sizes	Maximum Number of Odd Powers
Non-CRT	1024 bits	16
	2048 bits	10
	4096 bits	2
CRT	2 × 512 bits	16
	2 × 1024 bits	16
	2 × 2048 bits	6

The *2K-bit Modular Exponentiation PKA RAM Allocation Examples* table shows example PKA RAM vector allocations for modular exponentiation operations with and without using CRT. The *free space* start address is the first free byte following the vector workspace. The sequencer execution scratchpad of 34 bytes (non-CRT) or 72 bytes (using CRT) must fit between this address and the end of the PKA RAM. Note that the non-CRT operations use `PKA_CPTR = PKA_DPTR` to save space.

Table 36-11: 2K-bit Modular Exponentiation PKA RAM Allocation Examples

Operation	(sub-)vector	Start address (Byte Offset)	Size (words)	Buffer (words)
non-CRT (<code>PKA_ALEN</code> = 0x040, <code>PKA_BLEN</code> = 0x040, 4 odd-powers)	Exponent	0x000 (<code>PKA_APTR</code> = 0x000)	64	0
	Modulus	0x100 (<code>PKA_BPTR</code> = 0x040)	64	2
	Base	0x208 (<code>PKA_CPTR</code> = 0x082)	64	2
	Result	0x208 (<code>PKA_DPTR</code> = 0x082)	64	2
	Vector Workspace	0x208 (= Result)	7 × (64+2)=462	0
	Free space	0x940 (2368 bytes used)	-	-
using CRT (<code>PKA_ALEN</code> = 0x020, <code>PKA_BLEN</code> = 0x020, 4 odd-powers)	Exp P	0x000 (<code>PKA_APTR</code> = 0x000)	32	0
	Exp Q	0x080	32	0
	Mod P	0x100 (<code>PKA_BPTR</code> = 0x040)	32	2
	Mod Q	0x188	32	2
	Q inverse	0x210 (<code>PKA_CPTR</code> = 0x084)	32	0
	Input, Result	0x290 (<code>PKA_DPTR</code> = 0x0A4)	64	0
	Vector workspace	0x290 (= Result)	7 × (32+2)+32+2-0 = 272	0
	Free space	0x6D0 (1744 bytes used)	-	-

The following example in pseudo-code describes the execution of a non-CRT modular exponentiation operation using a 512 bits modulus and a 160 bits exponent, using actual test vectors:

```
// Perform a 512/160 bits
modular exponentiation without CRT (using 4 'odd-powers')
// Exponent equals value 0x8FD84098_8A0930CC_9CDC1E8A_B246EB46_2D39F064
// write as vector A to PKA
```

```

RAM Byte offset 0x000:
Write PKA_RAM_BASE+0x000+0x00
0x2D39F064
Write PKA_RAM_BASE+0x000+0x04
0xB246EB46
Write PKA_RAM_BASE+0x000+0x08
0x9CDC1E8A
Write PKA_RAM_BASE+0x000+0x0C
0x8A0930CC
Write PKA_RAM_BASE+0x000+0x10
0x8FD84098
// Modulus equals value 0xF42F559D
1877CA5F_449492B9_42DC7C01_...
// A3C9085B_7236A085_2102B000_A093C6B4_...
// 9D0EDA0C_292DE841_29C23723_4048BDA3_...
// 373C4C9F_45CF15A7_5F049ABF_D8A01B9B
// write as vector B to PKA
RAM Byte offset 0x018 (following exp at next aligned 64-bit word):
Write PKA_RAM_BASE+0x018+0x00
0xD8A01B9B
Write PKA_RAM_BASE+0x018+0x04
0x5F049ABF
Write PKA_RAM_BASE+0x018+0x08
0x45CF15A7
Write PKA_RAM_BASE+0x018+0x0C
0x373C4C9F
Write PKA_RAM_BASE+0x018+0x10
0x4048BDA3
Write PKA_RAM_BASE+0x018+0x14
0x29C23723
Write PKA_RAM_BASE+0x018+0x18
0x292DE841
Write PKA_RAM_BASE+0x018+0x1C
0x9D0EDA0C
Write PKA_RAM_BASE+0x018+0x20
0xA093C6B4
Write PKA_RAM_BASE+0x018+0x24
0x2102B000
Write PKA_RAM_BASE+0x018+0x28
0x7236A085
Write PKA_RAM_BASE+0x018+0x2C
0xA3C9085B
Write PKA_RAM_BASE+0x018+0x30
0x42DC7C01
Write PKA_RAM_BASE+0x018+0x34
0x449492B9
Write PKA_RAM_BASE+0x018+0x38
0x1877CA5F
Write PKA_RAM_BASE+0x018+0x3C
0xF42F559D

```

```

// Base equals value 0x3D291F48_49064887_1149594B_67935110_...
// 14EB8FF0_AB291F3A_54A1B4D1_5E611E44_...
// C989251B_44904B45_0B060482_317F8352_...
// 18CE440E_9BF509F1_6EAF26F2_95F19F12
// write as vector C to PKA
RAM Byte offset 0x060 (following mod after buffer + align words):
Write PKA_RAM_BASE+0x060+0x00
0x95F19F12
Write PKA_RAM_BASE+0x060+0x04
0x6EAF26F2
Write PKA_RAM_BASE+0x060+0x08
0x9BF509F1
Write PKA_RAM_BASE+0x060+0x0C
0x18CE440E
Write PKA_RAM_BASE+0x060+0x10
0x317F8352
Write PKA_RAM_BASE+0x060+0x14
0x0B060482
Write PKA_RAM_BASE+0x060+0x18
0x44904B45
Write PKA_RAM_BASE+0x060+0x1C
0xC989251B
Write PKA_RAM_BASE+0x060+0x20
0x5E611E44
Write PKA_RAM_BASE+0x060+0x24
0x54A1B4D1
Write PKA_RAM_BASE+0x060+0x28
0xAB291F3A
Write PKA_RAM_BASE+0x060+0x2C
0x14EB8FF0
Write PKA_RAM_BASE+0x060+0x30
0x67935110
Write PKA_RAM_BASE+0x060+0x34
0x1149594B
Write PKA_RAM_BASE+0x060+0x38
0x49064887
Write PKA_RAM_BASE+0x060+0x3C
0x3D291F48
// The result value and scratchpad
(vector D) may be co-located with the base vector C for
// a normal modular exponentiation,
so these are located at PKA RAM Byte offset 0x060 too.
// Load pointer and length
registers:
Write PKA_APTR 0x000>>2 //
Exponent pointer
Write PKA_BPTR 0x018>>2 //
Modulus pointer
Write PKA_CPTR 0x060>>2 //
Base pointer

```

```

Write PKA_DPTR 0x060>>2 //
Result/scratchpad pointer
Write PKA_ALENGTH 0x00000005
// Exponent length in 32-bit words
Write PKA_BLENGTH 0x00000010
// Mod/base/result length in 32-bit words
// Start modular exponentiation
and wait until it's done:
Write PKA_SHIFT 0x00000004
// Number of 'odd powers'
Write PKA_FUNCTION 0x0000E000
// 'Run' bit set, 'Sequencer Operations' = 0b110
Wait PKA_FUNCTION[15] == '0'
// 'Run' bit clears itself - Host can also use interrupt!
// Result value equals 0xA497BF8B_DB729088_954005B0_B5CA6691_...
// A3EC491B_091A3D62_03C24214_0863A389_...
// 0C7C03CD_2333E231_35EC10ED_8F91281C_...
// 30F4253B_FE38FAFB_BB4A39DB_C14F2661
// written as vector D at
PKA RAM Byte offset 0x060:
Check PKA_RAM_BASE+0x060+0x00
== 0xC14F2661
Check PKA_RAM_BASE+0x060+0x04
== 0xBB4A39DB
Check PKA_RAM_BASE+0x060+0x08
== 0xFE38FAFB
Check PKA_RAM_BASE+0x060+0x0C
== 0x30F4253B
Check PKA_RAM_BASE+0x060+0x10
== 0x8F91281C
Check PKA_RAM_BASE+0x060+0x14
== 0x35EC10ED
Check PKA_RAM_BASE+0x060+0x18
== 0x2333E231
Check PKA_RAM_BASE+0x060+0x1C
== 0x0C7C03CD
Check PKA_RAM_BASE+0x060+0x20
== 0x0863A389
Check PKA_RAM_BASE+0x060+0x24
== 0x03C24214
Check PKA_RAM_BASE+0x060+0x28
== 0x091A3D62
Check PKA_RAM_BASE+0x060+0x2C
== 0xA3EC491B
Check PKA_RAM_BASE+0x060+0x30
== 0xB5CA6691
Check PKA_RAM_BASE+0x060+0x34
== 0x954005B0
Check PKA_RAM_BASE+0x060+0x38
== 0xDB729088

```

```
Check PKA_RAM_BASE+0x060+0x3C
== 0xA497BF8B
```

Modular Inversion

Besides modular exponentiation, the sequencer also controls modular inversion operations.

Table 36-12: Summary of ModInv Operation

Function	Mathematical Operation	Vector A	Vector B	Vector C	Vector D
ModInv	$A^{-1} \bmod B \rightarrow D$	NumToInvert, length = A_Len	Modulus, length = B_Len	Not Used	Result and Workspace

The above function appears to be an extension of the set of basic PKCP functions with the following exceptions:

- Vector D not only addresses the result but also a workspace
- The `PKA_SHIFT` register field is used to return info on the operation's result.

Table 36-13: PKA_SHIFT Result Values for ModInv Operation

Function	PKA_SHIFT Register Field Value At Conclusion
ModInv	0 → success; VectorD holds result 7 → no inverse exists ($\text{GCD}(A, B) \neq 1$, for example, A and B have common factors); result undefined 31 → error, modulus even; result undefined other values are reserved

The following tables list the restrictions on the input and result vectors for the ModInv operation:

Table 36-14: Operational Restrictions on Input Vectors for the ModInv Operation

Function	Requirements
ModInv	$0 < A_Len \leq \text{Max_Len}$ $0 < B_Len \leq \text{Max_Len}$ Modulus B must be odd (for example, the least significant bit must be ONE) Modulus B may not have value 1 (result is undefined, no error indicated) The highest word of the modulus vector, as indicated by B_Len, may not be zero.

Table 36-15: ModInv Scratchpad Area/Input Vector Overlap Restrictions

Function	Result Vector	Restrictions
ModInv	D	Scratchpad area starting at D may not overlap with any of the other vectors

The following table shows the required scratchpad sizes for the ModInv Operation:

Table 36-16: ModInv Result Vector/Scratchpad Area Memory Allocation (Both Starting at PKA_DPTR)

Function	Scratchpad area size (in 32-bit words), Result Vector is B_Length 32-bit words long
ModInv	$5 \times (M + \varepsilon(M))$, with $M = \text{Max}(A_Length, B_Length)$ $\varepsilon(n) = 2 + (n \text{ MOD } 2)$, for example, 2 (for n even) or 3 (for n odd)

NOTE: During execution of a ModInv operation, the last 34 bytes of the PKA RAM are used as general scratchpad for the sequencer program execution. This (fixed location) area may not overlap with any of the input vectors and/or the D vector scratchpad area during execution.

Modular Inversion with an Even Modulus

The ModInv operation requires the modulus to be odd. At first, this requirement appears to make the operation useless in the case of RSA key generation where the private key exponent d is derived from a chosen public exponent e as follows:

$$d = \text{ModInv}(e, \phi); \text{ where } \phi = (p-1) \times (q-1) \text{ and } p \text{ and } q \text{ both prime}$$

Note that ϕ is even. However, since e must be odd (otherwise no inverse exists), d can be calculated as:

$$d = (1 + (\phi \times (e - \text{ModInv}(\phi, e)))) / e$$

With four more basic PKCP operations, ModInv can also be used to find inverse values in case the modulus is even.

Modular Inversion with a Prime Modulus

Modular inversion can be performed with a modular exponentiation using the modulus value minus two as exponent, provided that the modulus value is a prime. This is due to the following:

$$(A^M) \bmod M = A \Rightarrow$$

$$(A^{M-1}) \bmod M = 1 \Rightarrow$$

$$(A^{M-2}) \bmod M = A^{-1} \bmod M$$

Under the constraint that M is a prime value.

Especially with the large PKA engines containing an LNME, it is worthwhile to check whether this method is faster than using the ModInv operation directly. The modulus values for the ECC curves supported by this PKA engine must be prime, so this method can be used in ECDSA operations.

ECC Operations

Besides modular exponentiation and modular inversion, the sequencer also controls ECC operations.

Table 36-17: Summary of ECC Operations

Function	Mathematical Operation	Vector A	Vector B	Vector C	Vector D
ECC-ADD	Point addition/ doubling [†] on elliptic curve: $y^2 = x^3 + ax + b \pmod{p}$ $\text{pntA} + \text{pntC} \rightarrow \text{pntD}$	pntA.x followed [‡] by pntA.y both B_Len long (A_Len <i>not</i> used)	Curve parameter p fol- lowed [‡] by a (b is not needed) all B_Len long	pntC.x followed by pntC.y both B_Len long	Result, for example, pntD.x followed [‡] by pntD.y and work- space
ECC-MUL	Point multiplication on elliptic curve: $y^2 = x^3 + ax + b \pmod{p}$ $k \times \text{pntC} \rightarrow \text{pntD}$	Scalar k A_Len long	Curve parameter p fol- lowed [‡] by a and b all B_Len long.	pntC.x followed [‡] by pntC.y both B_Len long	Result, for example pntD.x followed [‡] by pntD.y and work- space

[†] If $\text{pntA} = \text{pntC}$, a point doubling operation is performed automatically.

[‡] All input components must be located on a 64-bit boundary and must have extra 'buffer' words (of 32 bits each) after their most significant word. ϵ must be 3 (B_Len odd) or 2 (B_Len even). Each result component (for example, pntD.x , pntD.y) is followed by ϵ buffer (zero) words.

The above functions appear to be extensions of the set of PKCP basic functions with the following exceptions:

- Input and result vectors can now be composite (for example, consist of two or three equal-sized subvectors)
- Vector D not only addresses the result but also a workspace
- The [PKA_SHIFT](#) register is used to return info on the result of the operation

Table 36-18: PKA_SHIFT Result Values for ECC Operations

Function	PKA_SHIFT Register Field Value at Conclusion
ECC-ADD	0 → success; VectorD holds result point
ECC-MUL	7 → result is point-at-infinity; VectorD result point undefined
	31 → error, (p not odd, p too short, etc); VectorD result point undefined
	Other values are reserved

The following tables below list the restrictions on the input and result vectors for the ECC operations.

Table 36-19: Operational Restrictions on Input Vectors for ECC Operations

Function	Requirements
ECC-ADD	$1 < B_Len \leq 24$ (maximum vector length is 768 bits) Modulus p must be a prime $> 2^{63}$ Effective modulus size (in bits) must be a multiple of 32† The highest word of the modulus vector, as indicated by B_Len , may not be zero. $a < p$ and $b < p$ $pntA$ and $pntC$ must be on the curve (this condition is not checked) Neither $pntA$ nor $pntC$ can be the point-at-infinity, although ECC-ADD can return this point as a result
ECC-MUL	$0 < A_Len \leq 24$ (maximum vector length is 768 bits) $1 < B_Len \leq 24$ (maximum vector length is 768 bits) Modulus p must be a prime $> 2^{63}$ Effective modulus size (in bits) must be a multiple of 32† The highest word of the modulus vector, as indicated by B_Len , may not be zero. $a < p$ and $b < p$ $pntC$ must be on the curve (this condition is not checked) $pntC$ cannot be the point-at-infinity, although ECC-MUL can return this point as a result

† Modulus lengths of 112 and 521 bits are exceptions to this rule.

Table 36-20: ECC Scratchpad Area/Input Vector Overlap Restrictions

Function	Result Vector	Restrictions
ECC-ADD ECC-MUL	D	Scratchpad area starting at D may not overlap with any of the other vectors

The *>ECC Result Vector/Scratchpad Area Memory Allocation* table shows the required scratchpad sizes for the ECC operations:

Table 36-21: ECC Result Vector/Scratchpad Area Memory Allocation (Both Starting at PKA_DPTR)

Function	Scratchpad area size (in 32-bit words), Result Vector is $2 \times (B_Length + \epsilon(B_Length))$ 32-bit words long
ECC-ADD	$2 \times L + 5 \times M$, where $L = B_Length + \epsilon(B_Length)$ $M = B_Length + 1 + \epsilon(B_Length + 1)$
ECC-MUL	$18 \times L + \text{Max}(8, L)$, where $L = (B_Length + \epsilon(B_Length))$

† $\epsilon(n) = 2 + (n \text{ MOD } 2)$, for example, 2 (for n even) or 3 (for n odd)

NOTE: During execution of an ECC-ADD or ECC-MUL operation, the last 72 bytes of the PKA RAM are used as a general scratchpad for the sequencer program execution. These (fixed location) areas must not overlap with any of the input vectors and the D vector scratchpad area during execution.

The *ECC Point Multiplication PKA RAM Allocation Examples* table shows example PKA RAM vector allocations for ECC point multiplication operations. The *free space* start address is the first free byte following the vector scratchpad. The sequencer execution scratchpad of 72 bytes must fit between this address and the end of the PKA RAM. Because of this requirement, a 521-bit ECC point multiplication cannot be performed with 2K byte PKA RAM.

Table 36-22: ECC Point Multiplication PKA RAM Allocation Examples

Modulus Length	(sub-)vector	Start Address (byte offset)	Size (words)	Buffer (words)
192 bits (=6 words, <code>PKA_ALEN=0x006</code> , <code>PKA_BLEN=0x006</code>)	Scalar k	0x000 (<code>PKA_APTR = 0x000</code>)	6	0
	p	0x018 (<code>PKA_BPTR = 0x006</code>)	6	2
	a	0x038	6	2
	b	0x058	6	2
	PntC.x (base)	0x078 (<code>PKA_CPTR = 0x01E</code>)	6	2
	PntC.y (base)	0x098	6	2
	PntD.x (result)	0x0B8 (<code>PKA_DPTR = 0x02E</code>)	6	2
	PntD.y (result)	0x0D8	6	0
	Vector scratchpad	0x0B8 (= PntD.x)	$(18 \times 8) + 8 = 152$	0
	Free space	0x318 (792 bytes used)	-	-
384 bits (=12 words, <code>ALENGTH=0x00C</code> , <code>BLENGTH=0x00C</code>)	Scalar k	0x000 (<code>PKA_APTR = 0x000</code>)	12	0
	p	0x030 (<code>PKA_BPTR = 0x00C</code>)	12	2
	a	0x068	12	2
	b	0x0A0	12	2
	PntC.x (base)	0x0D8 (<code>PKA_CPTR = 0x036</code>)	12	2
	PntC.y (base)	0x110	12	2
	PntD.x (result)	0x148 (<code>PKA_DPTR = 0x052</code>)	12	2
	PntD.y (result)	0x180	12	0
	Vector scratchpad	0x148 (= PntD.x)	$(18 \times 14) + 14 = 266$	0
	Free space	0x570 (1392 bytes used)	-	-
521 bits (=17 words, <code>ALENGTH=0x011</code> , <code>BLENGTH=0x011</code>)	Scalar k	0x000 (<code>PKA_APTR = 0x000</code>)	17	1 (to align p)
	p	0x048 (<code>PKA_BPTR = 0x012</code>)	17	3
	a	0x098	17	3
	b	0x0E8	17	3
	PntC.x (base)	0x138 (<code>PKA_CPTR = 0x04E</code>)	17	3
	PntC.y (base)	0x188	17	3
	PntD.x (result)	0x1D8 (<code>PKA_DPTR = 0x076</code>)	17	3

Table 36-22: ECC Point Multiplication PKA RAM Allocation Examples (Continued)

Modulus Length	(sub-)vector	Start Address (byte offset)	Size (words)	Buffer (words)
	PntD.y (result)	0x228	17	0
	Vector scratchpad	0x1D8 (= PntD.x)	$(18 \times 20) + 20 = 380$	0
	Free space	0x7C8 (1992 bytes used)	-	-

The following example in pseudo-code describes the execution of a 192 bits ECC point multiplication, using actual test vectors (the curve parameters and generator point are from standard curve 'secp192r1').

```
// Perform a 192 bits ECC point multiplication using PKA RAM layout from table
// above.
// Scalar 'k' equals value 0x8D98D058_9EFD018A_C9BCF3CF_2C33AEC0_24867D7F_6ADACBFF
// write as vector A to PKA RAM Byte offset 0x000:

    Write PKA_RAM_BASE+0x000+0x00 0x6ADACBFF
    Write PKA_RAM_BASE+0x000+0x04 0x24867D7F
    Write PKA_RAM_BASE+0x000+0x08 0x2C33AEC0
    Write PKA_RAM_BASE+0x000+0x0C 0xC9BCF3CF
    Write PKA_RAM_BASE+0x000+0x10 0x9EFD018A
    Write PKA_RAM_BASE+0x000+0x14 0x8D98D058
// Curve parameter 'p' equals value
0xFFFFFFFF_FFFFFFFF_FFFFFFFF_FFFFFFFFE_FFFFFFFF_FFFFFFFF
// write as 1st part of vector B immediately following vector A at PKA RAM Byte
// offset 0x018
// (no buffer word needed after 'k' vector, 64-bit alignment is OK):
    Write PKA_RAM_BASE+0x018+0x00 0xFFFFFFFF
    Write PKA_RAM_BASE+0x018+0x04 0xFFFFFFFF
    Write PKA_RAM_BASE+0x018+0x08 0xFFFFFFFFFE
    Write PKA_RAM_BASE+0x018+0x0C 0xFFFFFFFF
    Write PKA_RAM_BASE+0x018+0x10 0xFFFFFFFF
    Write PKA_RAM_BASE+0x018+0x14 0xFFFFFFFF
// Curve parameter 'a' equals value
0xFFFFFFFF_FFFFFFFF_FFFFFFFF_FFFFFFFFE_FFFFFFFF_FFFFFFFC
// write as 2nd part of vector B after one buffer word and one re-alignment word
// at 0x038:
    Write PKA_RAM_BASE+0x038+0x00 0xFFFFFFFFC
    Write PKA_RAM_BASE+0x038+0x04 0xFFFFFFFF
    Write PKA_RAM_BASE+0x038+0x08 0xFFFFFFFFFE
    Write PKA_RAM_BASE+0x038+0x0C 0xFFFFFFFF
    Write PKA_RAM_BASE+0x038+0x10 0xFFFFFFFF
    Write PKA_RAM_BASE+0x038+0x14 0xFFFFFFFF
// Curve parameter 'b' equals value
0x64210519_E59C80E7_0FA7E9AB_72243049_FEB8DEEC_C146B9B1
// write as 3rd part of vector B after one buffer word and one re-alignment word
// at 0x058:
    Write PKA_RAM_BASE+0x058+0x00
0xC146B9B1
```

```

Write PKA_RAM_BASE+0x058+0x04 0xFEB8DEEC
Write PKA_RAM_BASE+0x058+0x08 0x72243049
Write PKA_RAM_BASE+0x058+0x0C 0x0FA7E9AB
Write PKA_RAM_BASE+0x058+0x10 0xE59C80E7
Write PKA_RAM_BASE+0x058+0x14 0x64210519
// X-coord of generator point is value
0x188DA80E_B03090F6_7CBF20EB_43A18800_F4FF0AFD_82FF1012
// write as 1st part of vector C following vector B after buffer + alignment
words at 0x078:
Write PKA_RAM_BASE+0x078+0x00 0x82FF1012
Write PKA_RAM_BASE+0x078+0x04 0xF4FF0AFD
Write PKA_RAM_BASE+0x078+0x08 0x43A18800
Write PKA_RAM_BASE+0x078+0x0C 0x7CBF20EB
Write PKA_RAM_BASE+0x078+0x10 0xB03090F6
Write PKA_RAM_BASE+0x078+0x14 0x188DA80E
// Y-coord of generator point is value
0x07192B95_FFC8DA78_631011ED_6B24CDD5_73F977A1_1E794811
// write as 2nd part of vector C after one buffer word and one re-alignment word
at 0x098:
Write PKA_RAM_BASE+0x098+0x00 0x1E794811
Write PKA_RAM_BASE+0x098+0x04 0x73F977A1
Write PKA_RAM_BASE+0x098+0x08 0x6B24CDD5
Write PKA_RAM_BASE+0x098+0x0C 0x631011ED
Write PKA_RAM_BASE+0x098+0x10 0xFFC8DA78
Write PKA_RAM_BASE+0x098+0x14 0x07192B95
// The result point and scratchpad (vector D) follow vector C after one buffer
word and one
// re-alignment word, so these are located at PKA RAM Byte offset 0x0B8.
// Load pointer and length registers:
Write PKA_APTR 0x000>>2 // Scalar 'k' pointer
Write PKA_BPTR 0x018>>2 // Curve parameters 'p', 'a' & 'b' pointer
Write PKA_CPTR 0x078>>2 // Generator point X & Y coordinates pointer
Write PKA_DPTR 0x0B8>>2 // Result point X & Y coordinates/scratchpad pointer
Write PKA_ALENGTH 0x00000006 // Scalar 'k' length in 32-bit words
Write PKA_BLENGTH 0x00000006 // Curve parameters and coordinate lengths in 32-
bit words
// Start ECC point multiplication and wait until it's done:
Write PKA_FUNCTION 0x0000D000
// 'Run' bit set, 'Sequencer Operations' = 0b101
Wait PKA_FUNCTION[15] == '0'
// 'Run' bit clears itself - Host can also use interrupt!
Check PKA_SHIFT == 0x00000000
// Shift field value 0 indicates success - check this
// X-coord of result point is value
0x759B9F39_0E81D268_18C82BB9_CB42BCF5_0E0AE958_85BA3097
// written as 1st part of vector D at PKA RAM Byte offset 0x0B8:
Check PKA_RAM_BASE+0x0B8+0x00== 0x85BA3097
Check PKA_RAM_BASE+0x0B8+0x04== 0x0E0AE958
Check PKA_RAM_BASE+0x0B8+0x08== 0xCB42BCF5
Check PKA_RAM_BASE+0x0B8+0x0C== 0x18C82BB9

```

```

Check PKA_RAM_BASE+0x0B8+0x10== 0x0E81D268
Check PKA_RAM_BASE+0x0B8+0x14== 0x759B9F39
// Y-coord of result point is value
0xECA14640_F92EFF07_CAF2BD55_3FBE28EF_D043F28E_1CC3D238
// written as 2nd part of vector D at PKA RAM Byte offset 0x0D8:
Check PKA_RAM_BASE+0x0D8+0x00== 0x1CC3D238
Check PKA_RAM_BASE+0x0D8+0x04== 0xD043F28E
Check PKA_RAM_BASE+0x0D8+0x08== 0x3FBE28EF
Check PKA_RAM_BASE+0x0D8+0x0C== 0xCAF2BD55
Check PKA_RAM_BASE+0x0D8+0x10== 0xF92EFF07
Check PKA_RAM_BASE+0x0D8+0x14== 0xECA14640

```

ADSP-2159x PKA Register Descriptions

Public Key Accelerator (PKA) contains the following registers.

Table 36-23: ADSP-2159x PKA Register List

Name	Description
PKA_ALEN	PKA Vector_A Length
PKA_APTR	PKA Vector_A Address
PKA_BLEN	PKA Vector_B Length
PKA_BPTR	PKA Vector_B Address
PKA_COMPARE	PKA Compare Result
PKA_CPTR	PKA Vector_C Address
PKA_DIVMSW	PKA Most-Significant-Word of Divide Remainder
PKA_DPTR	PKA Vector_D Address
PKA_FUNC	PKA Function
PKA_RAM	Start of PKA RAM space
PKA_RESULTMSW	PKA Most-Significant-Word of Result Vector
PKA_SHIFT	PKA Bit Shift Value

PKA Vector_A Length

During execution of basic PKCP operations, the `PKA_ALEN` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_ALEN` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

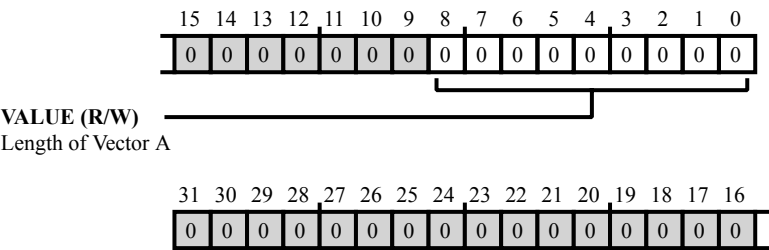


Figure 36-2: PKA_ALEN Register Diagram

Table 36-24: PKA_ALEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/W)	VALUE	Length of Vector A. Length (in 32-bit words) of Vector A.

PKA Vector_A Address

During execution of basic PKCP operations, the `PKA_APTR` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_APTR` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

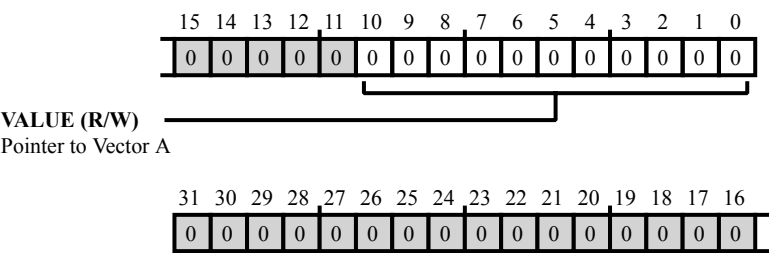


Figure 36-3: PKA_APTR Register Diagram

Table 36-25: PKA_APTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Pointer to Vector A. The <code>PKA_APTR.VALUE</code> bit field is the location of Vector A within the PKA RAM. Vectors are identified through the location of their least-significant 32-bit word. Note that bit [0] must be zero to ensure that the vector starts at an 8-byte boundary.

PKA Vector_B Length

During execution of basic PKCP operations, the `PKA_BLEN` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_BLEN` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

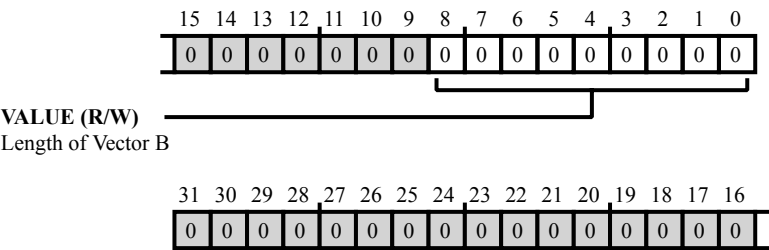


Figure 36-4: PKA_BLEN Register Diagram

Table 36-26: PKA_BLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
8:0 (R/W)	VALUE	Length of Vector B. Length (in 32-bit words) of Vector B.

PKA Vector_B Address

During execution of basic PKCP operations, the `PKA_BPTR` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_BPTR` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

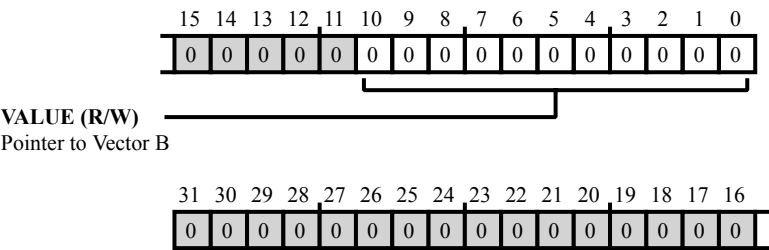


Figure 36-5: PKA_BPTR Register Diagram

Table 36-27: PKA_BPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Pointer to Vector B. The <code>PKA_BPTR.VALUE</code> bit field is the location of Vector B within the PKA RAM. Vectors are identified through the location of their least-significant 32-bit word. Note that bit [0] must be zero to ensure that the vector starts at an 8-byte boundary.

PKA Compare Result

The `PKA_COMPARE` register provides the result of a basic PKCP Compare operation. It is updated when the `PKA_FUNC.RUN` bit is reset at the end of that operation. The status after a complex sequencer operation is unknown.

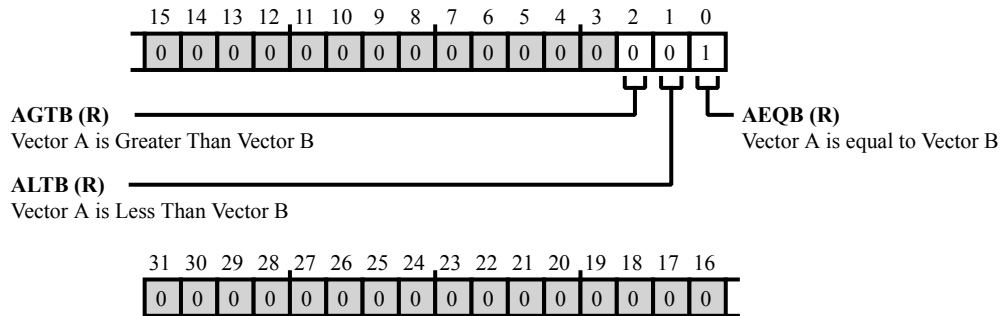


Figure 36-6: `PKA_COMPARE` Register Diagram

Table 36-28: `PKA_COMPARE` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/NW)	AGTB	Vector A is Greater Than Vector B. The <code>PKA_COMPARE.AGTB</code> bit shows the result of the basic compare operation is <code>PKCP.Vector_A</code> is greater than <code>Vector_B</code> .
1 (R/NW)	ALTB	Vector A is Less Than Vector B. The <code>PKA_COMPARE.ALTB</code> bit shows the result of the basic compare operation is <code>Vector_A</code> is less than <code>Vector_B</code> .
0 (R/NW)	AEQB	Vector A is equal to Vector B. The <code>PKA_COMPARE.AEQB</code> bit shows the result of the basic compare operation is <code>Vector_A</code> is equal to <code>Vector_B</code> .

PKA Vector_C Address

During execution of basic PKCP operations, the `PKA_CPTR` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_CPTR` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

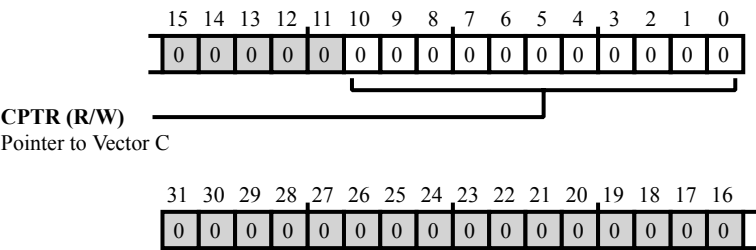


Figure 36-7: PKA_CPTR Register Diagram

Table 36-29: PKA_CPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	CPTR	Pointer to Vector C. The <code>PKA_CPTR.CPTR</code> bit field is the location of Vector C within the PKA RAM. Vectors are identified through the location of their least-significant 32-bit word. Note that bit [0] must be zero to ensure that the vector starts at an 8-byte boundary.

PKA Most-Significant-Word of Divide Remainder

The `PKA_DIVMSW` register indicates the (32-bit word) address in the PKA RAM where the most significant non-zero 32-bit word of the Remainder result for the basic Divide and Modulo operations is stored. Bits [4:0] are loaded with the bit number of the most significant non-zero bit in the most significant non-zero word when MS one control bit is set. For Divide, Modulo and MS one reporting, this register is updated when the `PKA_FUNC.RUN` bit is reset at the end of the operation.

For the complex sequencer controlled operations, updating bits [4:0] of this register with the actual result's most significant bit location is done near the end of the operation. Note that the result is only meaningful if no errors were detected and that for ECC operations, the `PKA_DIVMSW` register provides information for the x-coordinate of the result point only.

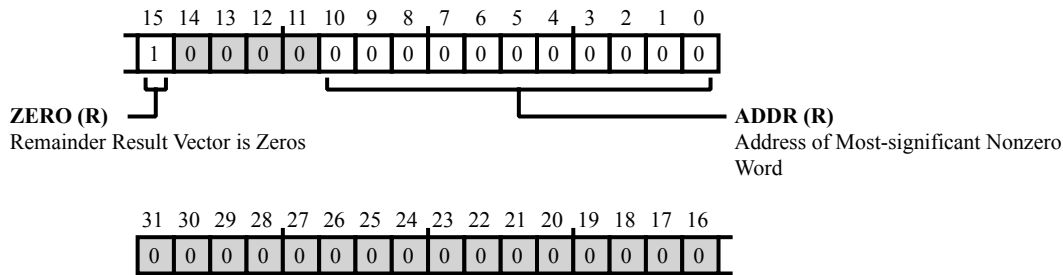


Figure 36-8: `PKA_DIVMSW` Register Diagram

Table 36-30: `PKA_DIVMSW` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/NW)	ZERO	Remainder Result Vector is Zeros. The <code>PKA_DIVMSW.ZERO</code> bit shows the remainder result vector is all zeros, ignore the address returned in bits [10:0].
10:0 (R/NW)	ADDR	Address of Most-significant Nonzero Word. The <code>PKA_DIVMSW.ADDR</code> bit shows the address of the most significant non-zero 32-bit word of the remainder result vector in PKA RAM.

PKA Vector_D Address

During execution of basic PKCP operations, the `PKA_DPTR` register is double buffered and can be written with a new value for the next operation. When not written, the value remains intact. During the execution of sequencer controlled complex operations, the `PKA_DPTR` register may not be written and its value is undefined at the conclusion of the operation. The driver software cannot rely on the written value to remain intact.

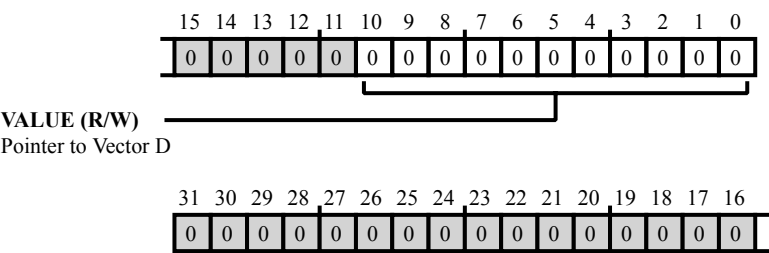


Figure 36-9: PKA_DPTR Register Diagram

Table 36-31: PKA_DPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Pointer to Vector D. The <code>PKA_DPTR.VALUE</code> bit field is the location of Vector D within the PKA RAM. Vectors are identified through the location of their least-significant 32-bit word. Note that bit [0] must be zero to ensure that the vector starts at an 8-byte boundary.

PKA Function

The `PKA_FUNC` register contains the control bits to start basic PKCP as well as complex sequencer operations. The `PKA_FUNC.RUN` bit can be used to poll for the completion of the operation. Modifying bits [11:0] is made impossible during the execution of a basic PKCP operation.

During the execution of Sequencer controlled complex operations, this register is modified - the `PKA_FUNC.RUN` and `PKA_FUNC.STALLRSLT` bits are set to zero at the conclusion, but other bits are undefined.

Continuously reading this register to poll the `PKA_FUNC.RUN` bit is NOT allowed when executing complex sequencer operations (the sequencer cannot access the PKCP when this is done).

Leave at least one SCLK cycle between poll operations.

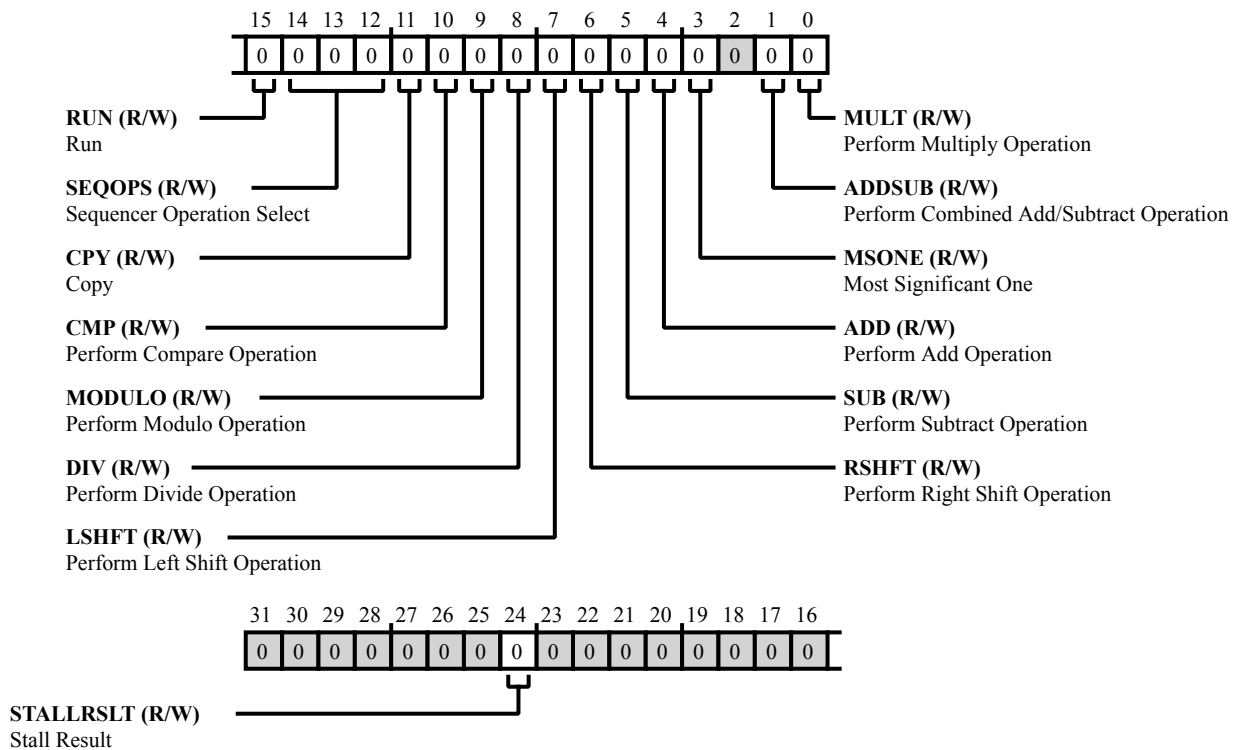


Figure 36-10: `PKA_FUNC` Register Diagram

Table 36-32: PKA_FUNC Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration																
24 (R/W)	STALLRSLT	Stall Result. When the <code>PKA_FUNC . STALLRSLT</code> bit is set, updating the <code>PKA_COMPARE</code>, <code>PKA_RESULTMSW</code> and <code>PKA_DIVMSW</code> registers, as well as resetting the Run bit, is stalled beyond the point that a running operation is actually finished. Use this to allow software enough time to read results from a previous operation when the newly started operation is known to take only a short amount of time. If a result is waiting, the result registers is updated and the Run bit is reset in the clock cycle following writing the Stall Result bit back to 0. The Stall result function may only be used for basic PKCP operations.																
15 (R/W)	RUN	Run. Set the <code>PKA_FUNC . RUN</code> bit to instruct the PKA module to begin processing the basic PKCP or complex Sequencer operation. This bit is reset low automatically when the operation is complete. The complement of this bit is output as the <code>pkaint1</code> interrupt. After a reset, the Run bit is always set to 1b but the first Sequencer firmware instruction sets this bit to 0 immediately after the hardware reset is released. A few clock cycles are needed before the first instruction is executed and the Run bit state has been propagated.																
14:12 (R/W)	SEQOPS	Sequencer Operation Select. The <code>PKA_FUNC . SEQOPS</code> bit field select the complex Sequencer operation to perform. <table><tr><td>0</td><td>None</td></tr><tr><td>1</td><td>ExpMod-CRT</td></tr><tr><td>2</td><td>ExpMod-ACT4</td></tr><tr><td>3</td><td>ECC-ADD</td></tr><tr><td>4</td><td>ExpMod-ACT2</td></tr><tr><td>5</td><td>ECC-MUL</td></tr><tr><td>6</td><td>ExpMod-variable</td></tr><tr><td>7</td><td>ModInv</td></tr></table>	0	None	1	ExpMod-CRT	2	ExpMod-ACT4	3	ECC-ADD	4	ExpMod-ACT2	5	ECC-MUL	6	ExpMod-variable	7	ModInv
0	None																	
1	ExpMod-CRT																	
2	ExpMod-ACT4																	
3	ECC-ADD																	
4	ExpMod-ACT2																	
5	ECC-MUL																	
6	ExpMod-variable																	
7	ModInv																	
11 (R/W)	CPY	Copy. Setting the <code>PKA_FUNC . CPY</code> bit performs the copy operation. For more information, see the "PKCP Vector Operations" section.																
10 (R/W)	CMP	Perform Compare Operation. Setting the <code>PKA_FUNC . CMP</code> bit performs the compare operation. For more information, see the "PKCP Vector Operations" section.																

Table 36-32: PKA_FUNC Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	MODULO	Perform Modulo Operation. Setting the PKA_FUNC . MODULO bit performs the modulo operation. For more information, see the "PKCP Vector Operations" section.
8 (R/W)	DIV	Perform Divide Operation. Setting the PKA_FUNC . DIV bit performs the divide operation. For more information, see the "PKCP Vector Operations" section.
7 (R/W)	LSHFT	Perform Left Shift Operation. Setting the PKA_FUNC . LSHFT bit performs the Left shift operation. For more information, see the "PKCP Vector Operations" section.
6 (R/W)	RSHFT	Perform Right Shift Operation. Setting the PKA_FUNC . RSHFT bit performs the right shift operation. For more information, see the "PKCP Vector Operations" section.
5 (R/W)	SUB	Perform Subtract Operation. Setting the PKA_FUNC . SUB bit performs the subtract operation. For more information, see the "PKCP Vector Operations" section.
4 (R/W)	ADD	Perform Add Operation. Setting the PKA_FUNC . ADD bit performs the add operation. For more information, see the "PKCP Vector Operations" section.
3 (R/W)	MSONE	Most Significant One. Setting the PKA_FUNC . MSONE bit loads the location of the Most Significant one bit within the result word indicated in the PKA_RESULTMSW register into bits [4:0] of the PKA_DIVMSW register can only be used with basic PKCP operations, except for Divide, Modulo and Compare.
1 (R/W)	ADDSUB	Perform Combined Add/Subtract Operation. Setting the PKA_FUNC . ADDSUB bit performs the combined Add/Subtract operation. For more information, see the "PKCP Vector Operations" section.
0 (R/W)	MULT	Perform Multiply Operation. Setting the PKA_FUNC . MULT bit performs the multiply operation. For more information, see the "PKCP Vector Operations" section.

Start of PKA RAM space

The `PKA_RAM` register provides the the starting location of the RAM space to hold the input, output and other vectors.

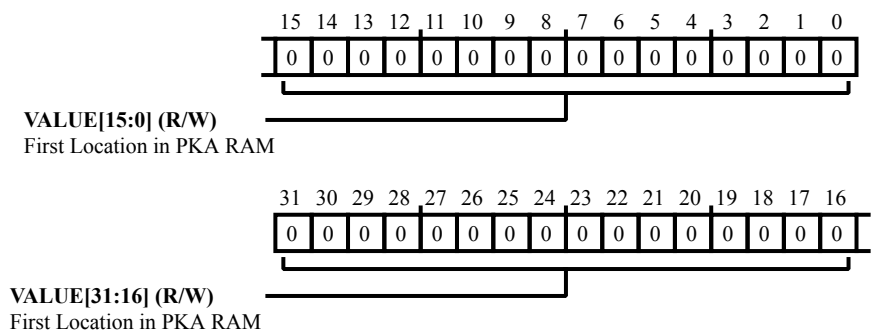


Figure 36-11: PKA_RAM Register Diagram

Table 36-33: PKA_RAM Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	First Location in PKA RAM. The <code>PKA_RAM.VALUE</code> bit field provides the the starting location of the RAM space to hold the input, output and other vectors.

PKA Most-Significant-Word of Result Vector

The `PKA_RESULTMSW` register indicates the (word) address in the PKA RAM where the most significant non-zero 32-bit word of the result is stored and should be ignored for modulo operations. For basic PKCP operations, the `PKA_RESULTMSW` register is updated when the `PKA_FUNC.RUN` bit is reset at the end of the operation.

For the complex sequencer controlled operations, updating the final value matching the actual result is done near the end of the operation. Note that the result is only meaningful if no errors are detected and that for ECC operations, the `PKA_DIVMSW` register provides information for the x-coordinate of the result point only.

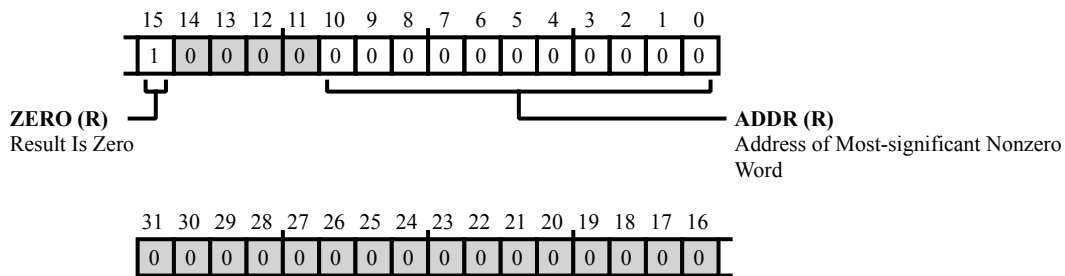


Figure 36-12: `PKA_RESULTMSW` Register Diagram

Table 36-34: `PKA_RESULTMSW` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/NW)	ZERO	Result Is Zero. The <code>PKA_RESULTMSW.ZERO</code> bit indicates the result vector is all zeros, ignore the address returned in bits [10:0].
10:0 (R/NW)	ADDR	Address of Most-significant Nonzero Word. The <code>PKA_RESULTMSW.ADDR</code> bit is the address of the most significant non-zero 32-bit word of the result vector in PKA RAM.

PKA Bit Shift Value

For basic PKCP operations, modifying the contents of the `PKA_SHIFT` register is made impossible while the operation is being performed. For the ExpMod-variable and ExpMod-CRT operations, the `PKA_SHIFT` register is used to indicate the number of odd powers to use (directly as a value in the range 1-16). For the ModInv and ECC operations, this register is used to hold a completion code.

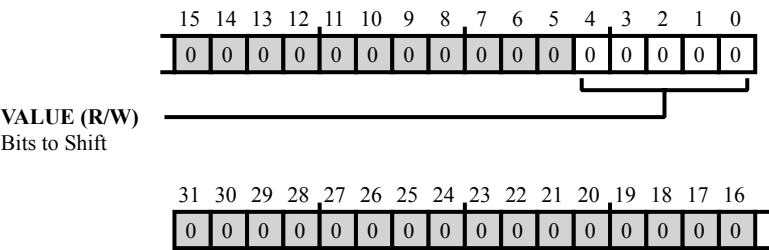


Figure 36-13: PKA_SHIFT Register Diagram

Table 36-35: PKA_SHIFT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
4:0 (R/W)	VALUE	Bits to Shift. The <code>PKA_SHIFT.VALUE</code> bit field is the number of bits to shift the input vector (in the range 0-31) during a Rshift or Lshift operation.

37 Public Key Interrupt Controller (PKIC)

The Public Key Accelerator (PKA) and the True Random Number Generator (TRNG) share a common interrupt controller, the Public Key Interrupt Controller (PKIC). The host processor configures the PKIC to generate interrupts when certain operations are complete or interrupts are caused by errors.

PKIC Functional Description

The main purpose and function of the PKIC is to capture the interrupts from different sources, either from the PKA or the TRNG and combine them to one interrupt output. The interrupt controller is managed using the following register groups:

- Control for polarity, edge, and level detection and enabling of individual interrupts
- Acknowledgment (to clear edge detected interrupts)
- Status:
 - A raw source status register after edge detection, if edge selected.
 - A status register after masking with the interrupt enable control bits.

ADSP-2159x PKIC Register List

The Public Key Processor Interrupt Controller (PKIC) provides security-related features. A set of registers governs PKIC operations. For more information on PKIC functionality, see the PKIC register descriptions.

Table 37-1: ADSP-2159x PKIC Register List

Name	Description
PKIC_ACK	Acknowledge Register
PKIC_EN_CLR	Enable Clear Register
PKIC_EN_CTL	Enable Control Register
PKIC_EN_SET	Enable Set Register
PKIC_EN_STAT	Enabled Status Register
PKIC_POL_CTL	Polarity Control Register

Table 37-1: ADSP-2159x PKIC Register List (Continued)

Name	Description
PKIC_RAW_STAT	Raw Status Register
PKIC_TYPE_CTL	Type Control Register

ADSP-2159x PKIC Interrupt List

Table 37-2: ADSP-2159x PKIC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
160	PKIC0_IRQ	PKIC0 Interrupt	Level	

PKIC Programming Model

The following sections provide information on how to program the PKIC.

Enabling/Disabling and Status

The [PKIC_EN_STAT](#) register provides the mask to which interrupt source is enabled. There are two status registers, [PKIC_RAW_STAT](#) and [PKIC_EN_STAT](#). They allow the host processor to read the status of the interrupt source before and after the mask is applied.

Level or Edge

All of the interrupt sources are level or edge events. The [PKIC_TYPE_CTL](#) register configures each interrupt to either level or edge. The [PKIC_POL_CTL](#) register controls the polarity of the signal.

These interrupts are latched at both status registers in case edge detection is selected. The edge detectors are reset by clearing the interrupts using the [PKIC_ACK](#) registers.

PKIC Programming Concepts

The following concepts help with proper programming for the PKIC module.

Interrupt Handling

When an interrupt is triggered, the handler must first examine this module, the PKIC to determine what triggered the interrupt. By reading the [PKIC_EN_STAT](#), the bits that are set are the pending interrupts of the ones that were not masked. After determining the source of the interrupt, the appropriate action must be taken to service the interrupt from the corresponding module, the PKA, or the TRNG. After handling the interrupt in a particular module, the corresponding interrupt must be acknowledged and cleared in the PKIC to allow further interrupts.

While handling an interrupt, any events that would cause another interrupt would happen without triggering another interrupt.

Overlapping Registers

There are two sets of overlapping registers in the PKIC. The `PKIC_EN_STAT` and `PKIC_ACK` registers share one address. If read, the register tells which enabled interrupts are pending. If written to (W1C), the interrupt is acknowledged and cleared. The `PKIC_RAW_STAT` and `PKIC_EN_SET` registers are another pair that share the address. When read, the register tells which interrupts are pending and if written to will enable certain interrupts. This register cannot be used to disable any interrupts.

ADSP-2159x PKIC Register Descriptions

Public Key Processor Interrupt Controller (PKIC) contains the following registers.

Table 37-3: ADSP-2159x PKIC Register List

Name	Description
<code>PKIC_ACK</code>	Acknowledge Register
<code>PKIC_EN_CLR</code>	Enable Clear Register
<code>PKIC_EN_CTL</code>	Enable Control Register
<code>PKIC_EN_SET</code>	Enable Set Register
<code>PKIC_EN_STAT</code>	Enabled Status Register
<code>PKIC_POL_CTL</code>	Polarity Control Register
<code>PKIC_RAW_STAT</code>	Raw Status Register
<code>PKIC_TYPE_CTL</code>	Type Control Register

Acknowledge Register

The `PKIC_ACK` register is used to acknowledge the interrupt and clear the corresponding interrupt bit in the status register.

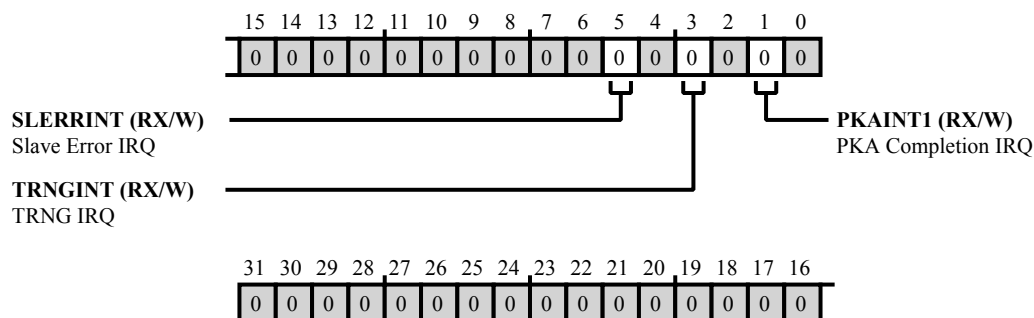


Figure 37-1: PKIC_ACK Register Diagram

Table 37-4: PKIC_ACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W)	SLERRINT	Slave Error IRQ. The <code>PKIC_ACK.SLERRINT</code> bit is the acknowledge bit for the Slave Error interrupt. When set =1 the <code>PKIC_ACK.SLERRINT</code> bit acknowledges the interrupt signal and clears the status bit (and is cleared automatically).
		0 Do not acknowledge interrupt and clear status bit
		1 Acknowledge interrupt and clear status bit
3 (RX/W)	TRNGINT	TRNG IRQ. The <code>PKIC_ACK.TRNGINT</code> bit is the acknowledge bit for the TRNG interrupt. When set =1 the <code>PKIC_ACK.TRNGINT</code> bit acknowledges the interrupt signal and clears the status bit (and is cleared automatically).
		0 Do not acknowledge interrupt and clear status bit
		1 Acknowledge interrupt and clear status bit
1 (RX/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_ACK.PKAINT1</code> bit is the acknowledge bit for the PKA completion interrupt. When set =1 the <code>PKIC_ACK.PKAINT1</code> bit acknowledges the interrupt signal and clears the status bit (and is cleared automatically).

Enable Clear Register

The `PKIC_EN_CLR` register allows the user to disable certain interrupts without enabling others. The disabled interrupts are also reflected in `PKIC_EN_CTL` register.

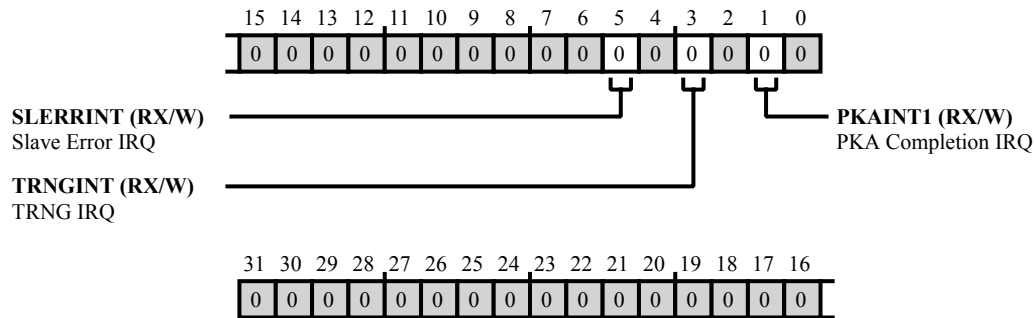


Figure 37-2: `PKIC_EN_CLR` Register Diagram

Table 37-5: `PKIC_EN_CLR` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W)	SLERRINT	Slave Error IRQ. The <code>PKIC_EN_CLR.SLERRINT</code> bit is the individual disable for the Slave Error interrupt. When set =1 this bit clears/resets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 0 (disables the interrupt). This bit is cleared automatically.
		0 No action
		1 Clear/reset corresponding CTL bit
3 (RX/W)	TRNGINT	TRNG IRQ. The <code>PKIC_EN_CLR.TRNGINT</code> bit is the individual disable for the TRNG interrupt. When set =1 this bit clears/resets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 0 (disables the interrupt). This bit is cleared automatically. 0= no effect.
		0 No action
		1 Clear/reset corresponding CTL bit
1 (RX/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_EN_CLR.PKAINT1</code> bit is the individual disable for the PKA Completion interrupt. When set =1 this bit clears/resets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 0 (disables the interrupt). This bit is cleared automatically. 0= no effect.
		0 No action
		1 Clear/reset corresponding CTL bit

Enable Control Register

The `PKIC_EN_CTL` register provides individual enable bits for the interrupt sources.

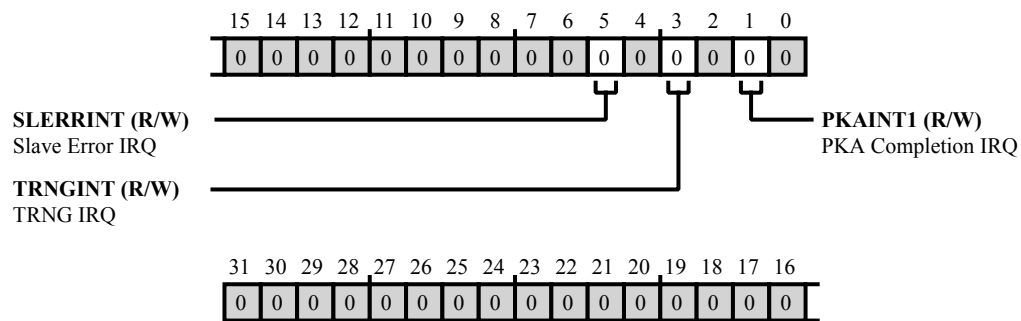


Figure 37-3: PKIC_EN_CTL Register Diagram

Table 37-6: PKIC_EN_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SLERRINT	Slave Error IRQ. The <code>PKIC_EN_CTL.SLERRINT</code> bit enables control for the Slave Error interrupt.
		0 Disable interrupt
		1 Enable interrupt
3 (R/W)	TRNGINT	TRNG IRQ. The <code>PKIC_EN_CTL.TRNGINT</code> bit enables control for the TRNG interrupt.
		0 Disable interrupt
		1 Enable interrupt
1 (R/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_EN_CTL.PKAINT1</code> bit enables control for the PKA completion interrupt.
		0 Disable interrupt
		1 Enable interrupt

Enable Set Register

The `PKIC_EN_SET` register allows the user to only set certain interrupt sources without disabling any others. The enabled interrupts are reflected in `PKIC_EN_CTL` register.

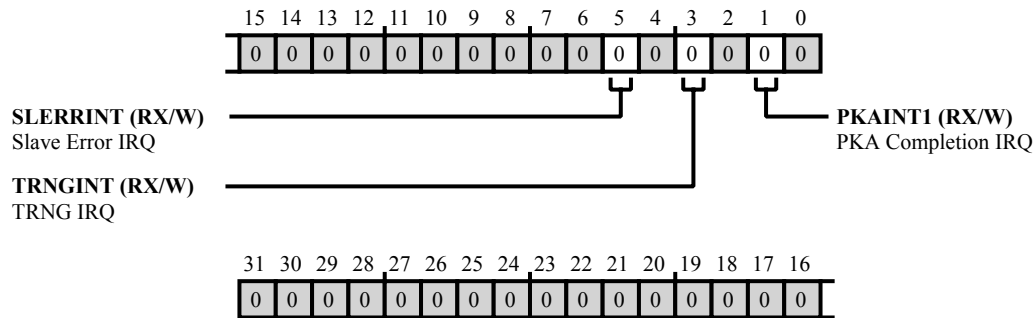


Figure 37-4: `PKIC_EN_SET` Register Diagram

Table 37-7: `PKIC_EN_SET` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W)	SLERRINT	Slave Error IRQ. The <code>PKIC_EN_SET.SLERRINT</code> bit is the individual enable for the Slave Error interrupt. If =1, sets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 1 (enables interrupt). This bit is cleared automatically. 0=no effect
		0 No effect
		1 Enable interrupt
3 (RX/W)	TRNGINT	TRNG IRQ. The <code>PKIC_EN_SET.TRNGINT</code> bit is the individual enable for the TRNG interrupt. If =1, sets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 1 (enables interrupt). This bit is cleared automatically.
		0 No effect
		1 Enable interrupt
1 (RX/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_EN_SET.PKAINT1</code> bit is the individual enable for the PKA Completion interrupt. If =1, sets the corresponding bit in the <code>PKIC_EN_CTL</code> register to 1 (enables interrupt). This bit is cleared automatically.
		0 No effect
		1 Enable interrupt

Enabled Status Register

The `PKIC_EN_STAT` register is used to tell the status of the interrupts after the gating with the `PKIC_EN_CTL` register.

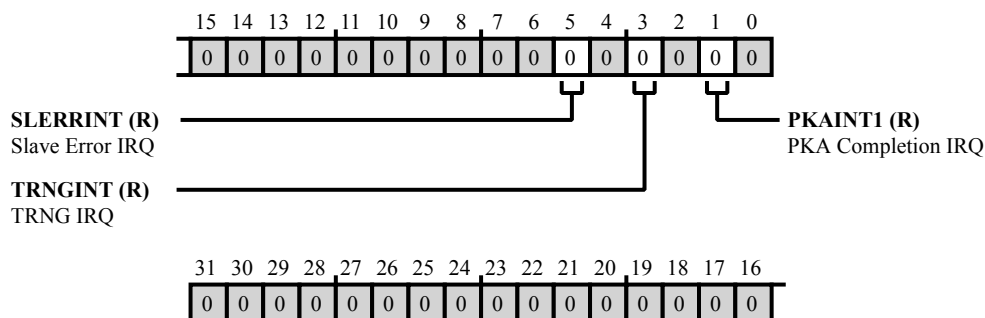


Figure 37-5: PKIC_EN_STAT Register Diagram

Table 37-8: PKIC_EN_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/NW)	SLERRINT	Slave Error IRQ. The <code>PKIC_EN_STAT.SLERRINT</code> bit provides the status of the Slave Error interrupt (after masking from the <code>PKIC_EN_CTL</code> register).
		0 Interrupt is inactive
		1 Interrupt is pending
3 (R/NW)	TRNGINT	TRNG IRQ. The <code>PKIC_EN_STAT.TRNGINT</code> bit provides the status of the TRNG interrupt (after masking from the <code>PKIC_EN_CTL</code> register).
		0 Interrupt is inactive
		1 Interrupt is pending
1 (R/NW)	PKAINT1	PKA Completion IRQ. The <code>PKIC_EN_STAT.PKAINT1</code> bit provides the status of the PKA Completion interrupt (after masking from the <code>PKIC_EN_CTL</code> register).
		0 Interrupt is inactive
		1 Interrupt is pending

Polarity Control Register

The `PKIC_POL_CTL` register is used to configure the signal polarity for each individual interrupt. During the initialization phase of the PKA the host processor must set each interrupt in this register to (high level/rising edge or low level/falling edge).

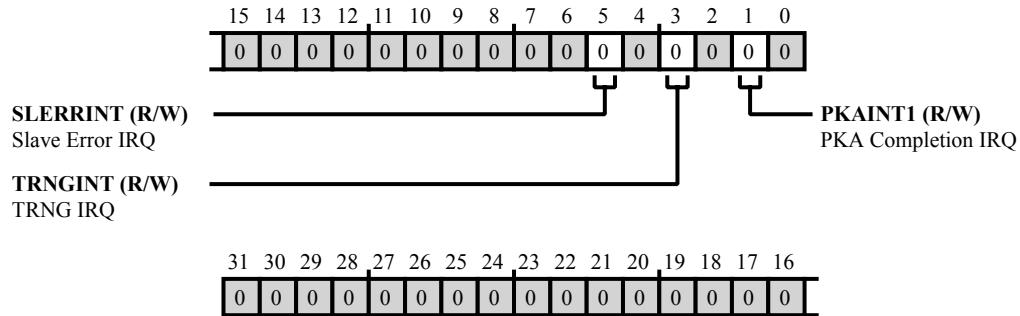


Figure 37-6: PKIC_POL_CTL Register Diagram

Table 37-9: PKIC_POL_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SLERRINT	Slave Error IRQ. The <code>PKIC_POL_CTL.SLERRINT</code> bit provides polarity control for the Slave Error interrupt.
		0 Low level/falling edge
		1 High level/rising edge
3 (R/W)	TRNGINT	TRNG IRQ. The <code>PKIC_POL_CTL.TRNGINT</code> bit provides polarity control for the TRNG interrupt.
		0 Low level/falling edge
		1 High level/rising edge
1 (R/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_POL_CTL.PKAINT1</code> bit provides polarity control for PKA completion interrupt.
		0 Low level/falling edge
		1 High level/rising edge

Raw Status Register

The `PKIC_RAW_STAT` register reflects the status of the individual interrupts before masking with the `PKIC_EN_CTL` register.

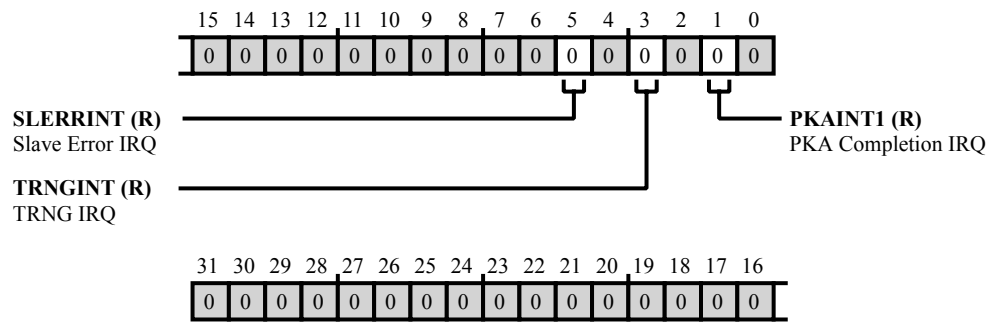


Figure 37-7: PKIC_RAW_STAT Register Diagram

Table 37-10: PKIC_RAW_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/NW)	SLERRINT	Slave Error IRQ. The <code>PKIC_RAW_STAT.SLERRINT</code> bit provides the raw status of the Slave Error interrupt.
		0 Inactive interrupt
		1 Pending interrupt
3 (R/NW)	TRNGINT	TRNG IRQ. The <code>PKIC_RAW_STAT.TRNGINT</code> bit provides the raw status of the TRNG interrupt where 1=pending and 0=inactive.
		0 Inactive interrupt
		1 Pending interrupt
1 (R/NW)	PKAINT1	PKA Completion IRQ. The <code>PKIC_RAW_STAT.PKAINT1</code> bit provides raw status of the PKA completion interrupt where 1=pending and 0=inactive.
		0 Inactive interrupt
		1 Pending interrupt

Type Control Register

The `PKIC_TYPE_CTL` register is used to configure the signal type for each individual interrupt. During the initialization phase of the PKA the host processor must set each interrupt in this register to level or edge.

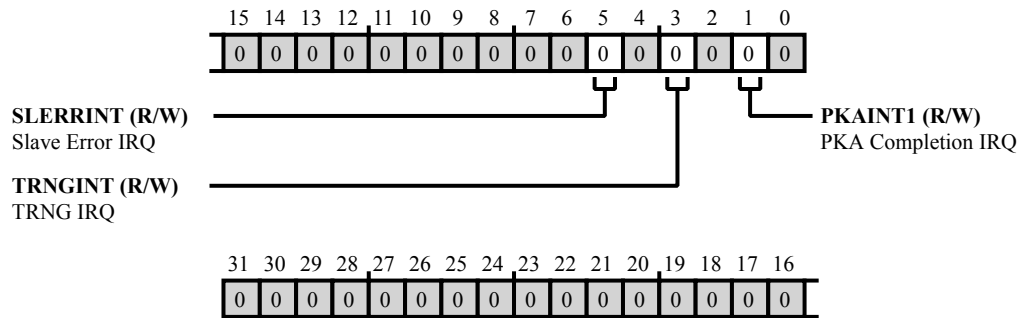


Figure 37-8: PKIC_TYPE_CTL Register Diagram

Table 37-11: PKIC_TYPE_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	SLERRINT	Slave Error IRQ. The <code>PKIC_TYPE_CTL.SLERRINT</code> bit provides signal type control for the Slave Error interrupt.
		0 Level
		1 Edge
3 (R/W)	TRNGINT	TRNG IRQ. The <code>PKIC_TYPE_CTL.TRNGINT</code> bit provides signal type control for the TRNG interrupt.
		0 Level
		1 Edge
1 (R/W)	PKAINT1	PKA Completion IRQ. The <code>PKIC_TYPE_CTL.PKAINT1</code> bit provides signal type control for the PKA completion interrupt.
		0 Level
		1 Edge

38 True Random Number Generator (TRNG)

The TRNG engine provides a true, non-deterministic, noise source for generating keys, Initialization Vectors (IVs), and other random number requirements. Other non-cryptographic purposes include statistical sampling, retry timers for communications protocols and noise generation.

TRNG Features

The TRNG features include:

- Hardware-based non-deterministic random number generator
- ANSI X9.31 postprocessing (depending on processor)
- Redundant 'fail-safe' design with self-test circuits
- Reliable shot noise oscillator implementation with auto-tuning
- Debug output to allow monitoring of internal operation
- Alarm count overflow and auto-tuning error interrupts
- Buffer to allow generation of large blocks of random data in the background

TRNG Functional Description

The following sections provide details on the function of the TRNG module.

ADSP-2159x TRNG Register List

The True Random Number Generator (TRNG) provides random number generation, intended mainly for security-related applications. A set of registers governs TRNG operations. For more information on TRNG functionality, see the TRNG register descriptions.

Table 38-1: ADSP-2159x TRNG Register List

Name	Description
TRNG_ALMCNT	TRNG Alarm Counter Register

Table 38-1: ADSP-2159x TRNG Register List (Continued)

Name	Description
TRNG_ALMMSK	TRNG Alarm Mask Register
TRNG_ALMSTP	TRNG Alarm Stop Register
TRNG_BLKCNT	TRNG Block Count Register
TRNG_CFG	TRNG Configuration Register
TRNG_CNT	Counter Register
TRNG_CTL	TRNG Control Register
TRNG_FRODETUNE	TRNG FRO De-tune Register
TRNG_FROEN	TRNG FRO Enable Register
TRNG_INPUT[n]	TRNG Input Registers
TRNG_INTACK	TRNG Interrupt Acknowledge Register
TRNG_KEY[n]	Post-Process Key Registers
TRNG_LFSR_H	TRNG LFSR Access Register
TRNG_LFSR_L	TRNG LFSR Access Register
TRNG_LFSR_M	TRNG LFSR Access Register
TRNG_MONOBITCNT	TRNG Monobit Test Result Register
TRNG_OUTPUT[n]	TRNG Output Registers
TRNG_POKER[n]	TRNG Poker Test Result Registers
TRNG_RUNCNT	TRNG Run Count Registers
TRNG_RUN[n]	TRNG Run Test State and Result Registers
TRNG_STAT	TRNG Status Register
TRNG_TEST	TRNG Test Register
TRNG_V[n]	TRNG Post-Process "V" Value Registers

Random Number Generation

The random numbers that the TRNG generates are produced by sampling Free Running Oscillators (FRO). The (TRNG_CTL.STARTUPCYC) bit field along with TRNG_CFG.MINREFCYC (minimum refill cycles) and TRNG_CFG.MAXREFCYC (maximum refill cycles) bit fields determine the number of samples taken to generate the first random value and subsequent random values.

1. After enabling the TRNG (TRNG_CTL.TRNGEN bit =1), a number of FRO output samples defined by the TRNG_CTL.STARTUPCYC bit field are gathered in the main linear-feedback shift register (LFSR) before taking a snapshot of the LFSR and storing that snapshot in the random data buffer (after optional post-processing).

2. After taking a snapshot of the LFSR, a number of FRO output samples defined by the `TRNG_CFG.MINREFCYC` bit field, are gathered in the main LFSR. If the random data buffer is full, sample-taking continues until the number of samples (counting from the snapshot) matches the number of samples defined by the `TRNG_CFG.MAXREFCYC` bit field. At that point, the TRNG switches off the FROs and powers down.
3. If, after the `TRNG_CFG.MINREFCYC` sampling period, the random data buffer is not completely filled, a new snapshot of the LFSR is taken and stored in the random data buffer (after optional postprocessing). Control branches back to point step 2 above and a new `TRNG_CFG.MINREFCYC` sample period starts.

Locking Detection and Prevention

Lock detection in functional mode uses the sampled outputs of the individual FROs. A FRO alarm event is declared when a repeating pattern (of up to four sample lengths) is detected continuously for the number of samples defined in the alarm threshold `TRNG_ALMCNT.ALMTHRESH` bit field. The alarm event is logged by setting the bit that corresponds to the FRO that caused the alarm in the `TRNG_ALMMSK` register. If that bit was already set, the corresponding bit in the `TRNG_ALMSTP` register is set. The FRO is switched off to prevent further alarm events from that FRO. If the `TRNG_ALMMSK` register bit was not yet set, the FRO is restarted automatically in an attempt to break locking.

The shutdown count field in the alarm count `TRNG_ALMCNT.SHDNCNT` register monitors the number of FROs switched off. (It counts the number of 1 bits in the `TRNG_ALMSTP` register.) The shutdown threshold field (`TRNG_ALMCNT.SHDNTHRESH`) can be configured to generate the shutdown overflow interrupt (`TRNG_STAT.SHDNOVR`). When the shutdown count in the `TRNG_ALMCNT.SHDNCNT` bit field exceeds the shutdown threshold in the `TRNG_ALMCNT.SHDNTHRESH` bit field, the shutdown overflow bit (`TRNG_STAT.SHDNOVR`) is set to 1 (which can be used to generate an interrupt).

Software can use two strategies for the TRNG operation:

- **Monitored Operation.** Software checks the `TRNG_ALMMSK` register at regular intervals (on the order of seconds). If a bit is set in that register, then the program must also check the `TRNG_ALMSTP` register to determine if a FRO was shut down due to multiple alarm events. If no FROs are shut down, the program clears the `TRNG_ALMMSK` register to remove the incidental alarm events. If one or more FROs are shut down, the host processor can modify the delay selection of those FROs using the `TRNG_FRODETUNE` register to prevent further locking. For this type of operation, the shutdown threshold is normally set to a low value (two, for example). The shutdown overflow interrupt can then be used to signal abnormal operation conditions or the breakdown of FROs.
- **Unmonitored Operation.** Software sets the shutdown threshold to the acceptable number of FROs to be shut down before taking corrective actions. It then uses the shutdown overflow interrupt to initiate corrective actions (clearing the `TRNG_ALMMSK` and `TRNG_ALMSTP` registers, toggling bits in the `TRNG_FRODETUNE` register). The software must monitor the time interval between these interrupts. If they occur too often (for example, within a minute after each other), this frequency indicates abnormal operating conditions or the breakdown of FROs.

Run Testing

Run Test

The TRNG block counts the number of consecutive zeros and ones (runs) in the data stream shifted into the main LFSR. The run length and bit value is then used to increment a specific bucket counter for these values. After 20,000 bits, the bucket counters must be within specified limits for this test to pass. If not, a run fail interrupt (RUNFAIL) is generated.

Table 38-2: Allowable Limits on Runs of 0's and 1's

Run Count	Bit Value	Min (inclusive)	Max (inclusive)
1	0	2267	2733
	1	2267	2733
2	0	1079	1421
	1	1079	1421
3	0	502	748
	1	502	748
4	0	233	402
	1	233	402
5	0	90	223
	1	90	223
6 and up	0	90	233
	1	90	233

Long Run Test

The long run test fails immediately when a run longer than 33 bits is found and a long run fail (TRNG_STAT.LRUNFAIL) interrupt is generated.

Noise Source Test

A noise source failure is declared when a run of 48 or more identical bits is found and a noise fail interrupt (TRNG_STAT.NOISEFAIL) is generated.

The status and counts are stored in the [TRNG_RUNCNT](#) and [TRNG_RUN\[n\]](#) registers. Unless otherwise indicated, all counters and state bits in these registers are reset when writing a 1 to either the monobit fail acknowledge (TRNG_INTACK.MBITFAIL), the run fail acknowledge (TRNG_INTACK.RUNFAIL), or the poker fail acknowledge (TRNG_INTACK.PKRFail) bits.

Monobit Testing

The TRNG block performs the monobit test on blocks of 20,000 bits (in parallel with the run test and poker test). It monitors the number of zeros and ones in the data stream shifted into the main LFSR. At the start of the block,

the counter is initialized to 10,000. Each 1 value increments the counter, each 0 value decrements the counter. After 20,000 bits, the counter value must be within 9310–10690 (inclusive) for this test to pass. If not, a monobit fail interrupt (`TRNG_STAT.MBITFAIL`) is generated. The AIS-31 standard (test T1, ref 0) specifies this run time testing of the TRNG and the parameters.

NOTE: The actual limits stated here are different than the limits stated in the AIS-31 standard due to the implementation. The circuitry in the TRNG uses an up-down counter while the standard just evaluates the sum of the 20,000 bits.

When the continue poker (`TRNG_TEST.CONTPKR`) bit is set to 1, the test is not stopped after 20,000 bits. The counter keeps incrementing and decrementing (protected against overflow and underflow). But, no actual limit checking happens. The offset from starting value 10,000 indicates the balance of 0 and 1 bits that were checked since the start of the continuous test. The offset is twice the number of missing or extra 1 bits. (An extra 1 bit adds an increment operation and removes one decrement operation. So, having 10,001 1 bits in the block gives a counter value of 10,002.)

Poker Testing

The poker test is run in parallel with the monobit test and run test. Counters in the `TRNG_POKER[n]` registers are used to count the occurrences of one specific 4-bit value in the data stream fed into the main LFSR. All of the counters are decremented by one every 64 data bits and reset to their start value every 20,000 bits. All counters start at a value of –1 and are decremented 312 times during the 20,000-bit test run.

Each 8-bit counter holds a two's complement value and does not overflow past the range of –128 to +127. At the end of the 20,000-bits block, the values of the counters that contain a single 1 bit appended at the least significant bit side are individually squared and then added. The poker test fails with a poker fail (`PKRFAIL`) interrupt when:

- the resulting sum (accumulated in the `TRNG_MONOBITCNT` register) is outside the range 1288 – 71750 (inclusive), or
- one of the counters tries to increment or decrement outside its limit range

NOTE: The poker test fails when the 4-bit values of the data stream are distributed too evenly (with eight counters having incremented 312 times and the others incremented 313 times). This failure is intentional. The minimum mean deviation from the expected value of 312.5 is 4.5. Failure at counter overflow is not an official part of the poker test as specified in the AIS-31 standard (ref 0). It can be shown that the maximum deviation for one counter's value from the mean value of 312.5 (without the poker test failing) is 129.5. Since this deviation is more than 40% of the mean value, it indicates that something is wrong. Here, the counter overflow failure is combined with the official poker test failure.

When the continue poker (`TRNG_TEST.CONTPKR`) bit is set to 1, the test does not stop after 20,000 bits. The counters keep incrementing and decrementing (the latter every 64 bits). A `PKRFAIL` interrupt is generated when one of the counters tries to increment or decrement outside its limit range.

Data for Tests

For the monobit test, run test and poker test circuits self-test, the test data written to the `TRNG_INPUT[n]0` register is used for the monobit test and run test bit-by-bit. It executes from bit 31 down to bit 0. For the poker test, the written test data is used in eight blocks of 4 bits, starting from bits 31:28 down to bits 3:0.

When the `TRNG_TEST.CONTPKR` bit =0 during run test and poker test circuits self-tests, the state of the poker and runs test status registers is frozen after all calculations are made and after inputting 20,000 test bits. This state allows time to read the test results. These calculations take around 20 clock cycles to complete. The status registers are reset to their starting states when the first word for the next test block of test bits is written to the `TRNG_INPUT[n]0` register. Then, the contents of this word are processed.

X9.31 Postprocessing

Postprocessing is available on some parts using the TRNG. If available, the postprocessing block is situated after the main LFSRs and before the output buffer that stores the random numbers for consumption. The online test logic for monobit, run, and poker testing is before the postprocessor block. The bits used for testing are from the main LFSR.

The postprocessor is based on the ANSI X9.31 specification using 3-Key 3DES cipher algorithm. It does not provide any more entropy than is what is already achieved from sampling the Free Running Oscillators (FROs). The use of the postprocessor only helps with applications requiring the use of an X9.31 compliant RNG.

The following section is an example of postprocessing.

The variables include:

- DT is a 64-bit date/time vector. This value is the input coming from random bits from the main LFSR.
- I is an intermediate value, a 64-bit value
- R is the final result, a 64-bit value
- V is a 64-bit seed value that is to be kept secret
- K is the 3-key for 3DES, each being 64-bits

The postprocessing uses the steps:

1. The intermediate value I is calculated: $I = 3DES_{EDE}(K, DT)$ with 3DES.
2. The result R is calculated: $R = 3DES_{EDE}(K, I \text{ XOR } V)$.
3. Finally, V is updated: $V = 3DES_{EDE}(K, R \text{ XOR } I)$.

TRNG Block Diagram

The *System Block Diagram of the TRNG* diagram shows the system block for the TRNG. The system includes:

- Free Running Oscillators (FROs) that are the source of sampled bits
- A post-processing unit (processor dependent) for standards compliance

- Test circuitry to detect non-randomness due to failures in the system

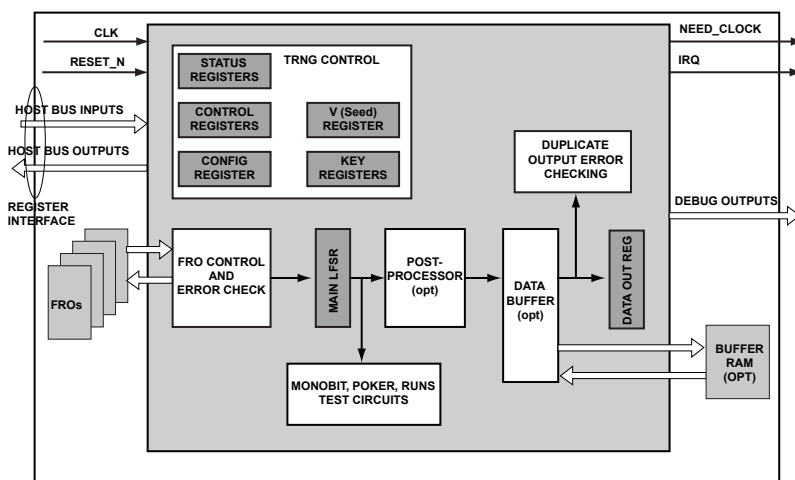


Figure 38-1: System Block Diagram of the TRNG

TRNG Architectural Concepts

The random numbers are accessible to the host processor in four 32-bit registers allowing a single burst read of a 128-bit random number. Acknowledging the data ready (interrupt) state causes the TRNG to move a new value (when available in the data buffer) to the TRNG output register. The TRNG always tries to keep the data buffer completely full. Pulling out data starts the regeneration of a new number by:

- Enabling the FROs
- Capturing their outputs in the LFSR, and
- Cryptographically postprocessing the values from the LFSR

The process produces new random values to replenish the buffer.

The major functional blocks of the TRNG module are:

- The actual TRNG core with control and test circuits, optional post-processor, and optional data buffer control logic
- Free Running Oscillators (FROs) instantiated outside the TRNG core
- A 128-byte buffer RAM instantiated outside the TRNG core

In the TRNG core, the true entropy source uses FROs as the basic building block. The accumulation of timing jitter, caused (for the largest part) by shot noise, creates uncertainty intervals for the output transitions of each FRO. Sampling within an uncertainty interval generates a single bit of entropy, which is ‘accumulated’ in a LFSR. As the uncertainty interval is very narrow compared to the cycle time of a FRO, the mean amount of entropy generated per sample is small (less than 1/100 bit per sample). To increase the entropy generation rate, multiple FROs are used in parallel.

The FROs are asynchronous to one another and asynchronous to the sampling clock to make their behavior truly non-deterministic.

The output signals of the FROs are sampled at regular intervals (in general, at the TRNG core module clock frequency). The samples feed into an error detection circuit that checks for repeating patterns coming out of a FRO. If a repeating pattern persists for a configurable number of samples, the FRO is suspect of having synchronized to (a harmonic of) the sampling interval. This activity drastically reduces the amount of entropy generated by that FRO. The error detection circuit signals this activity as a FRO error event.

Error events can occur during normal operation. The FRO control circuit attempts to restart a FRO that on a first error event. A second error event causes an automatic shutdown of the FRO. Because there are multiple FROs, shutting down a FRO reduces the amount of entropy generated, but it does not immediately jeopardize the TRNG operation. A limit can be configured below which the number of operational FROs does not drop. If this limit is crossed, an interrupt can be generated on the host processor. Software on the host processor can then attempt to prevent frequent locking of a FRO by *de-tuning* it to a slightly different frequency.

An XOR tree combines the sampled outputs and feeds them into an 81-bit LFSR to accumulate entropy and whiten the random bits stream.

NOTE: Here, *whitening* means balancing the number of one and zero bits in the stream.

TRNG Operating Modes

The TRNG has the following operating modes.

Normal Reading Mode

In normal reading mode, random data can only be read out of the output registers of the TRNG module when the ready bit (`TRNG_STAT.RDY`) = 1. Acknowledging the data by writing a 1 to the ready acknowledge bit in the `TRNG_INTACK.RDY` register clears the ready bit from the status register and clears the output registers. The registers remain at zero until the next 128-bit data block is available.

Secure Reading Mode

An attacker can try to read the output registers (without acknowledging the data) to obtain a copy of data to be read later by an application. To block this attack, secure reading mode is used. In this mode, enable reading from the output registers (by writing 0x00 to the open read gate bits (`TRNG_INTACK.OPENRDGATE`)) before it is possible to access the output registers. Enabling the reading starts a timeout (controlled by the `TRNG_CFG.RDTIMEOUT` bit field). When this timeout expires, reading is disabled and the offered data is acknowledged so that it is not offered again. The host processor must set this timeout such that there is enough time to read the output registers and perform a normal data acknowledge (which aborts the timeout).

Test Mode

In addition to the test circuitry that operates during normal operation, the TRNG has a test mode that allows further diagnosis when errors occur. In test mode, programs have access to the main LFSR through the

`TRNG_LFSR_L`, `TRNG_LFSR_M`, and `TRNG_LFSR_H` registers. Programs can also control the finite state machine sample counter using the `TRNG_CNT` register.

In test mode, the TRNG can be configured to test individual or a chosen set of FROs. It can also be configured to feed test patterns to the delay chain.

TRNG Data Transfer Modes

The host processor reads four 32-bit registers to access the random numbers. Once the registers are read and the data ready interrupt has been acknowledged, the TRNG moves more data from the internal buffer to the output registers (`TRNG_OUTPUT[n]`).

TRNG Event Control

There are eight events that the TRNG generates. The events are common error events from the run-time testing of the TRNG. While the TRNG is operating and generating random bits, test circuitry is also running statistical tests. The tests determine if the sources of the random bits have started to fail and are not truly producing random bits. There is also a single event to signal when data is ready in the output registers.

These events are captured in the `TRNG_STAT` register and can generate a single interrupt in the Public Key Interrupt Controller (PKIC). The individual events can be masked so the interrupt is not triggered. This configuration uses the `TRNG_CTL` register. The event and associated interrupt can be acknowledged in the `TRNG_INTACK` register.

The *TRNG Interrupt Signals* table lists all events or interrupts that the TRNG can generate.

TRNG Interrupt Signals

The TRNG provides a total of eight interrupts multiplexed into one output.

Table 38-3: TRNG Interrupt Signals

Interrupt	Name	Description
0	RDY	Ready. Random number is ready to be read from registers
1	SHDNOVR	Shutdown Overflow. The number of FRO's automatically shut down due to failures or errors have gone above the threshold specified in <code>TRNG_ALMCNT.SHDNTHRESH</code> .
2	STUCKOUT	Stuck Out. Logic circuitry around the output registers has detected the same output has been provided twice.
3	NOISEFAIL	Noise Fail. Logic circuitry monitoring the data shifted into the main LFSR detected 48 identical bits, which is considered a noise source failure.
4	RUNFAIL	Run Fail. Logic circuitry monitoring the data shifted into the main LFSR detected an out-of-bounds value for at least one of the <code>TRNG_RUN[n]</code> counters after checking 20,000 bits.

Table 38-3: TRNG Interrupt Signals (Continued)

Interrupt	Name	Description
5	LRUNFAIL	Long Run Fail. Logic circuitry monitoring the data shifted into the main LFSR detected 34 identical bits.
6	PKRFAIL	Poker Fail. Logic circuitry monitoring the data shifted into the main LFSR detected an out-of-bounds value in at least one of the 16 <code>TRNG_POKER[n]</code> counters or an out-of-bounds sum of squares values after checking 20,000 bits.
7	MBITFAIL	Monobit Fail. Logic circuitry monitoring the data shifted into the main LFSR detected an out-of-bounds number of 1's after checking 20,000 bits.

ADSP-2159x TRNG Register Descriptions

True Random Number Generator (TRNG) contains the following registers.

Table 38-4: ADSP-2159x TRNG Register List

Name	Description
<code>TRNG_ALMCNT</code>	TRNG Alarm Counter Register
<code>TRNG_ALMMSK</code>	TRNG Alarm Mask Register
<code>TRNG_ALMSTP</code>	TRNG Alarm Stop Register
<code>TRNG_BLKCNT</code>	TRNG Block Count Register
<code>TRNG_CFG</code>	TRNG Configuration Register
<code>TRNG_CNT</code>	Counter Register
<code>TRNG_CTL</code>	TRNG Control Register
<code>TRNG_FRODETUNE</code>	TRNG FRO De-tune Register
<code>TRNG_FROEN</code>	TRNG FRO Enable Register
<code>TRNG_INPUT[n]</code>	TRNG Input Registers
<code>TRNG_INTACK</code>	TRNG Interrupt Acknowledge Register
<code>TRNG_KEY[n]</code>	Post-Process Key Registers
<code>TRNG_LFSR_H</code>	TRNG LFSR Access Register
<code>TRNG_LFSR_L</code>	TRNG LFSR Access Register
<code>TRNG_LFSR_M</code>	TRNG LFSR Access Register
<code>TRNG_MONOBITCNT</code>	TRNG Monobit Test Result Register
<code>TRNG_OUTPUT[n]</code>	TRNG Output Registers
<code>TRNG_POKER[n]</code>	TRNG Poker Test Result Registers
<code>TRNG_RUNCNT</code>	TRNG Run Count Registers
<code>TRNG_RUN[n]</code>	TRNG Run Test State and Result Registers

Table 38-4: ADSP-2159x TRNG Register List (Continued)

Name	Description
TRNG_STAT	TRNG Status Register
TRNG_TEST	TRNG Test Register
TRNG_V[n]	TRNG Post-Process "V" Value Registers

TRNG Alarm Counter Register

The `TRNG_ALMCNT` register, together with the `TRNG_ALMMSK` and `TRNG_ALMSTP` registers, can be used by the host processor to determine if the FRO/sample cycle locking is a problem. Note that incidental alarm events are expected to occur during normal operation. This register also controls the way the monobit test and poker test circuits operate (using the standard 20,000 bit blocks or running continuously).

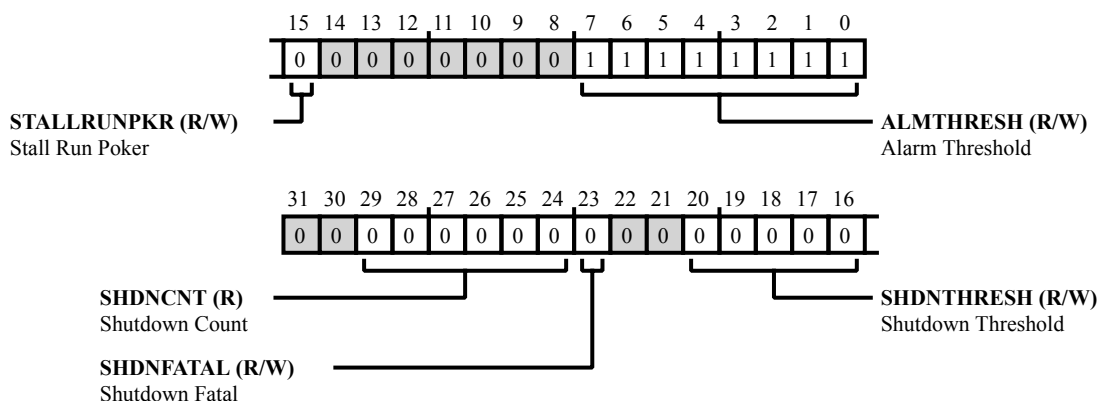


Figure 38-2: TRNG_ALMCNT Register Diagram

Table 38-5: TRNG_ALMCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/NW)	SHDNCNT	Shutdown Count. This read-only field indicates the number of 1 bits in the <code>TRNG_ALMSTP</code> register, the number of FRO's that's been turned off.
23 (R/W)	SHDNFATAL	Shutdown Fatal. When the <code>TRNG_ALMCNT.SHDNFATAL</code> bit field is set, the shutdown overflow (SHDNOVR) interrupt is considered a fatal error requiring taking the complete TRNG engine off-line.
20:16 (R/W)	SHDNTHRESH	Shutdown Threshold. The <code>TRNG_ALMCNT.SHDNTHRESH</code> bit field provides the threshold setting for generating the shutdown overflow (SHDNOVR) interrupt, which is activated when the shutdown count (<code>TRNG_ALMCNT.SHDNCNT</code>) value in this register exceeds the threshold value set here.
15 (R/W)	STALLRUNPKR	Stall Run Poker. When the <code>TRNG_ALMCNT.STALLRUNPKR</code> bit is set, stalls the Monobit Test, Run Test and Poker Test circuits when either the <code>TRNG_STAT.MBITFAIL</code> , <code>TRNG_STAT.RUNFAIL</code> or <code>TRNG_STAT.PKRFAIL</code> bits =1. This allows inspection of the state of the result counters (which would otherwise be reset immediately for the next 20,000 bits block to test).

Table 38-5: TRNG_ALMCNT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	ALMTHRESH	Alarm Threshold. The <code>TRNG_ALMCNT.ALMTHRESH</code> bit field sets the alarm detection threshold for the repeating pattern detectors on each FRO. A FRO alarm event is declared when a repeating pattern (of up to four samples length) is detected continuously for the number of samples defined by this fields value. Reset value 255 (decimal) should keep the number of alarm events to a manageable level.

TRNG Alarm Mask Register

A set bit (=1) in the `TRNG_ALMMSK` register signifies an alarm event and is used by the host processor to determine which of the individual FROs generated the alarm. If a bit in this register is set, the corresponding bit in the `TRNG_ALMSTP` register is set and the FRO is turned off by clearing the corresponding bit in the `TRNG_FROEN` register. If a bit is not set, the FRO restarts automatically to try to break sample cycle locking that could have caused the alarm event.

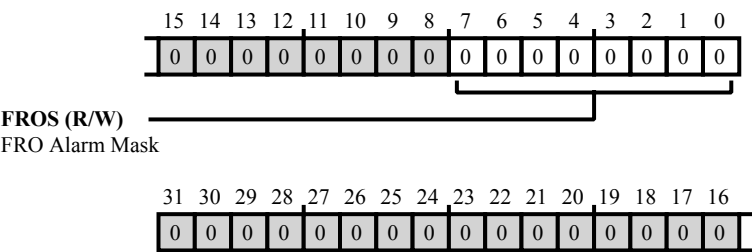


Figure 38-3: TRNG_ALMMSK Register Diagram

Table 38-6: TRNG_ALMMSK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	FROS	FRO Alarm Mask. The <code>TRNG_ALMMSK.FROS</code> bit field provides logging for the alarm events of individual FROs. A 1 in bit [n] indicates FRO n experienced an alarm event.

TRNG Alarm Stop Register

The `TRNG_ALMSTP` register is used by the host processor to determine which of the individual FROs generated more than one alarm event in quick succession. If a FRO generates an alarm event while a previous event is still logged in the `TRNG_ALMMSK` register, the corresponding bit in this register is set (=1) and the FRO is turned off by clearing (=0) the corresponding bit in the `TRNG_FROEN` register. The `TRNG_ALMCNT.SHDCNT` bit field keeps track of the number of bits that are set in this register.

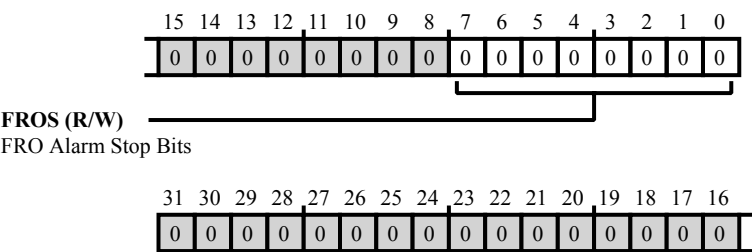


Figure 38-4: TRNG_ALMSTP Register Diagram

Table 38-7: TRNG_ALMSTP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	FROS	FRO Alarm Stop Bits. The <code>TRNG_ALMSTP.FROS</code> bit field provides logging for the alarm events of individual FROs. A 1 in bit [n] indicates FRO n experienced more than one alarm event in quick succession and has been turned off. A 1 in this field forces the corresponding bit in the <code>TRNG_FROEN</code> register to 0.

TRNG Block Count Register

The `TRNG_BLKCNT` register is the counter for the 128-bit blocks generated by the post-processor. These bits are forced to zero when the post-processor is disabled and are cleared to zero when an internal re-seed operation has finished. This register can be used by driver software to determine when to re-seed the post-processor.

The whole 32 bits of this register represent the amount of data (in bytes) generated since a re-seed. The `TRNG_BLKCNT` register is only present when a post-processor is available.

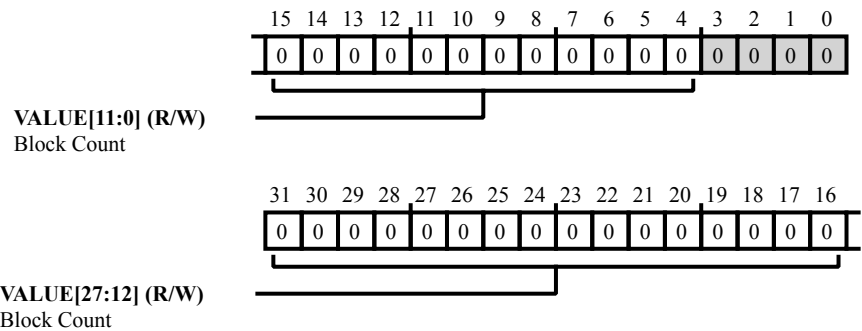


Figure 38-5: TRNG_BLKCNT Register Diagram

Table 38-8: TRNG_BLKCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:4 (R/W)	VALUE	Block Count. The <code>TRNG_BLKCNT.VALUE</code> bit field is the counter for the 128-bit blocks generated by the post-processor. These bits are forced to zero when the post-processor is disabled and are cleared to zero when an internal re-seed operation has finished.

TRNG Configuration Register

The `TRNG_CFG` register holds the lower and upper limits of the samples taken from the FROs in order to refill the random data buffer. This register also holds the time out value used for secure reading mode.

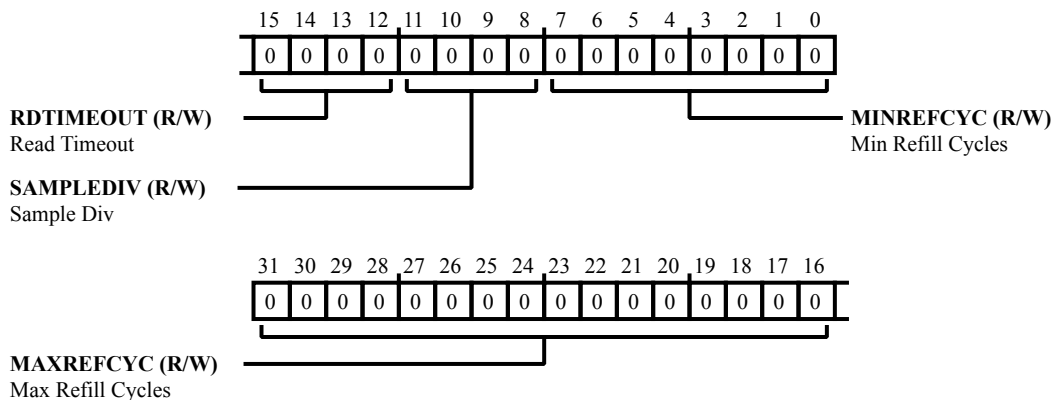


Figure 38-6: TRNG_CFG Register Diagram

Table 38-9: TRNG_CFG Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	MAXREFCYC	Max Refill Cycles. The <code>TRNG_CFG.MAXREFCYC</code> bit field determines the maximum number of samples (between 2^8 and 2^{24}) taken to re-generate entropy from the FROs after reading out a 64 bit random number. If the written value of this field is zero, the number of samples is 2^{24}, otherwise the number of samples equals the written value times 2^8. This field can only be modified while the <code>TRNG_CTL.TRNGEN</code> bit =0.
15:12 (R/W)	RDTIMEOUT	Read Timeout. The <code>TRNG_CFG.RDTIMEOUT</code> bit field controls the Secure Reading Mode. When this field is 0, Secure Reading Mode is disabled. Values in the range 115 enable Secure Reading and set a read gate closure timeout of approximately $(\text{read_timeout} + 1) \times 16$ clock input cycles. This field can only be modified while the <code>TRNG_CTL.TRNGEN</code> bit =0.
11:8 (R/W)	SAMPLEDIV	Sample Div. The <code>TRNG_CFG.SAMPLEDIV</code> bit field directly controls the number of input cycles between samples taken from the FROs. The default value 0 indicates that samples are taken every cycle, maximum value 15 (decimal) takes one sample every 16 cycles. This field must be set to a value such that the slowest FRO (even under worst-case conditions) has a cycle time less than twice the sample period. The default configuration of the FROs allows this field to remain 0. This field can only be modified while <code>TRNG_CTL.TRNGEN</code>=0.

Table 38-9: TRNG_CFG Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	MINREFCYC	Min Refill Cycles. The <code>TRNG_CFG.MINREFCYC</code> bit field determines the minimum number of samples (between 2^6 and 2^{24}) taken to re-generate entropy from the FROs after reading out a 64 bit random number. If the value of this field is zero, the number of samples is fixed to the value determined by the maximum refill cycles (<code>TRNG_CFG.MAXREFCYC</code>) field, otherwise the minimum number of samples equals the written value times 64 (which can be up to 2^{14}). The number of samples defined here cannot be higher than the number defined by the <code>TRNG_CFG.MAXREFCYC</code> field (i.e. that field takes precedence). This field can only be modified while the <code>TRNG_CTL.TRNGEN</code> bit =0.

Counter Register

The `TRNG_CNT` register is used to access the main control Finite State Machine's (FSM) sample counter while the `TRNG_CTL.TSTMODE` bit =1.

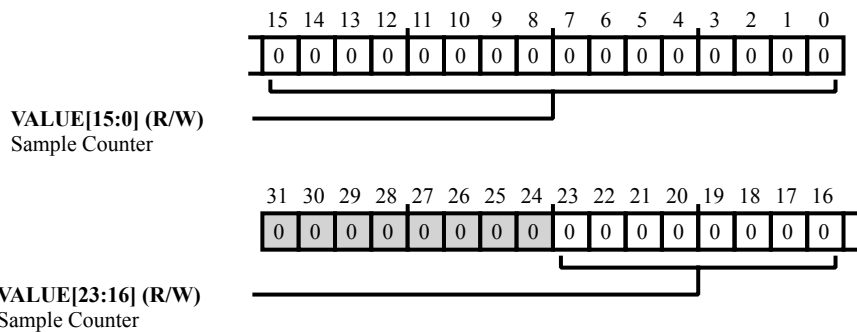


Figure 38-7: TRNG_CNT Register Diagram

Table 38-10: TRNG_CNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23:0 (R/W)	VALUE	Sample Counter. The <code>TRNG_CNT.VALUE</code> bit field is the sample counter used by control finite state machine. This counter can only be accessed when the <code>TRNG_CTL.TSTMODE</code> bit =1.

TRNG Control Register

The `TRNG_CTL` register must be written to start accumulating entropy before random numbers can be generated. In most cases, the `TRNG_CFG` register must also be written prior to writing the `TRNG_CTL` register. To enable the TRNG, set the `TRNG_CTL.TRNGEN` bit. This register also controls post-processing (if available). Note that when the `TRNG_CTL.TRNGEN` bit = 1, the start up cycles field (`TRNG_CTL.STARTUPCYC`) and the post-processing enable bit (if available) are locked. Any writes to these fields are ignored.

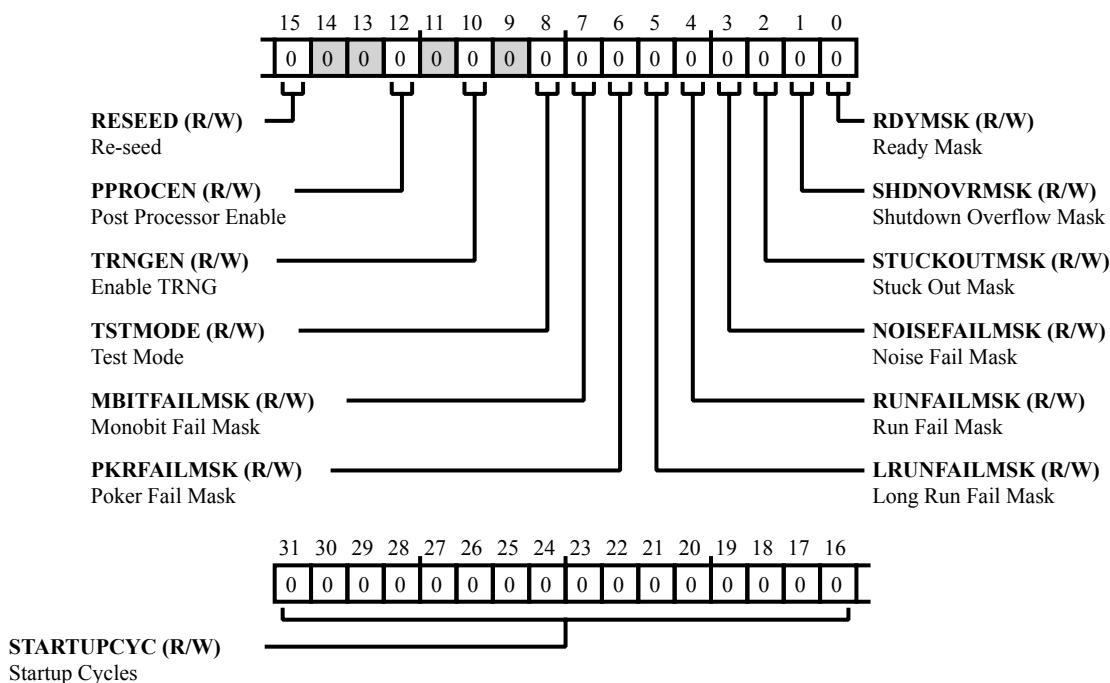


Figure 38-8: TRNG_CTL Register Diagram

Table 38-11: TRNG_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	STARTUPCYC	Startup Cycles. The <code>TRNG_CTL.STARTUPCYC</code> bit field determines the number of samples (between 2^8 and 2^{24}) taken to gather entropy from the FROs during startup. If the written value of this field is zero, the number of samples is 2^{24} , otherwise the number of samples equals the written value times 2^8 . This field can only be written when <code>TRNG_CTL.TRNGEN=0</code> before the write.

Table 38-11: TRNG_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	RESEED	Re-seed. The TRNG_CTL.RESEED bit is set-only, writing a 1 starts a re-seed cycle and writing a 0 has no effect. A re-seed cycle entails loading the TRNG_KEY[n] and TRNG_V[n] registers with random values generated internally these values are not visible outside the TRNG core. This bit falls back to 0 automatically after the re-seed operation is complete at that time the TRNG_BLKCNT register is reset to zero and the random data buffer is zero-ized so that any new data read from the TRNG will use the new seed values. This bit is only present when post-processing is available and can only be set to 1 when TRNG_CTL.TRNGEN=1 before the write. Note that re-seeding can be done with the post-processor disabled (normally used to seed the post-processor before enabling it). When writing a 1 to this bit, all other bits in this register remain unchanged.
12 (R/W)	PPROCEN	Post Processor Enable. Setting the TRNG_CTL.PPROCEN bit enables the FIPS post-processor. If this bit is reset to 0, the post-processor is forced back into the idle state immediately. This bit is only present when post-processing is available and can only be changed when the TRNG_CTL.TRNGEN bit (enable TRNG) was 0 before the write. To change the TRNG_CTL.PPROCEN bit during operation, first put the TRNG into reset (TRNG_CTL.TRNGEN =0). If the post-processor is enabled, it can be disabled by a subsequent write of 0 to this bit (writing this bit when the TRNG_CTL.TRNGEN bit is still 1 has no effect). The post-processor then stops immediately. To enable it, this bit must be written with 1. Changing the enabled/disabled state does not affect the contents of the the TRNG_KEY[n] and TRNG_V[n] registers. After changing the state, the TRNG must be started again by setting the TRNG_CTL.TRNGEN bit to 1. Note that it is required to re-gather entropy, so the same number of start-up cycles must be used as when starting the TRNG out of a system reset state.
10 (R/W)	TRNGEN	Enable TRNG. Setting the TRNG_CTL.TRNGEN bit to 1 starts the TRNG, gathering entropy from the FROs for the number of samples determined by the value in the TRNG_CTL.STARTUPCYC (Startup Cycles) field. Resetting this bit to 0 forces all TRNG logic back into the idle state immediately. Resetting this bit to 0 also performs the Un-instantiate operation, clearing all internal post-processor registers.
8 (R/W)	TSTMODE	Test Mode. When the TRNG_CTL.TSTMODE bit is set, access is enabled to the TRNG_CNT and TRNG_LFSR_L, TRNG_LFSR_M and TRNG_LFSR_H registers (the latter are cleared before enabling access) and sets the TRNG_STAT.NEEDCLK bit for testing purposes. This bit must be set to 1 before various test modes in the TRNG_TEST register can be enabled.

Table 38-11: TRNG_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/W)	MBITFAILMSK	Monobit Fail Mask. When the TRNG_CTL.MBITFAILMSK bit is set, this mask allows the TRNG_STAT.MBITFAIL bit to activate the (active HIGH) interrupt output.
6 (R/W)	PKRFAILMSK	Poker Fail Mask. When the TRNG_CTL.PKRFAILMSK bit is set, this mask allows the TRNG_STAT.PKRFAIL bit to activate the (active HIGH) interrupt output.
5 (R/W)	LRUNFAILMSK	Long Run Fail Mask. When the TRNG_CTL.LRUNFAILMSK bit is set, this mask allows the TRNG_STAT.LRUNFAIL bit to activate the (active HIGH) interrupt output.
4 (R/W)	RUNFAILMSK	Run Fail Mask. When the TRNG_CTL.RUNFAILMSK bit is set, this mask allows the TRNG_STAT.RUNFAIL bit to activate the (active HIGH) interrupt output.
3 (R/W)	NOISEFAILMSK	Noise Fail Mask. When the TRNG_CTL.NOISEFAILMSK bit is set, this mask allows the TRNG_STAT.NOISEFAIL bit to activate the (active HIGH) interrupt output.
2 (R/W)	STUCKOUTMSK	Stuck Out Mask. When the TRNG_CTL.STUCKOUTMSK bit is set, this mask allows the TRNG_STAT.STUCKOUT bit to activate the (active HIGH) interrupt output.
1 (R/W)	SHDNOVRMSK	Shutdown Overflow Mask. When the TRNG_CTL.SHDNOVRMSK bit is set, this mask allows the TRNG_STAT.SHDNOVR bit to activate the (active HIGH) interrupt output.
0 (R/W)	RDYMSK	Ready Mask. When the TRNG_CTL.RDYMSK bit is set, this mask allows the TRNG_STAT.RDY bit to activate the (active HIGH) interrupt output.

TRNG FRO De-tune Register

The `TRNG_FRODETUNE` register is used by the host processor to change the frequencies of individual FROs. This can reduce the number of alarm events generated by a specific FRO.

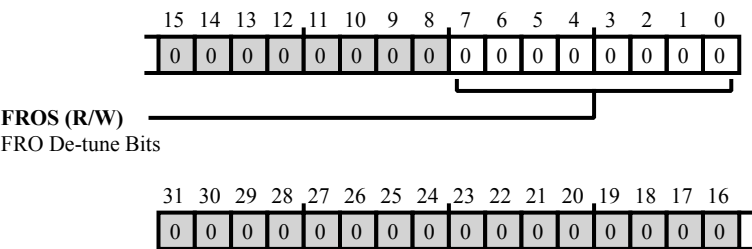


Figure 38-9: TRNG_FRODETUNE Register Diagram

Table 38-12: TRNG_FRODETUNE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	FROS	FRO De-tune Bits. The <code>TRNG_FRODETUNE.FROS</code> bits De-tune the FROs. A 1 in bit [n] lets FRO n run approximately 5% faster. The value of one of these bits may only be changed while the corresponding FRO is turned off (by temporarily writing a 0 in the corresponding bit of the <code>TRNG_FROEN</code> register).

TRNG FRO Enable Register

The `TRNG_FROEN` register can be used by the host processor to enable and disable FROs individually. Only enabled FROs contribute to entropy generation, but require power to do so. Disabled FROs cannot generate alarm events.

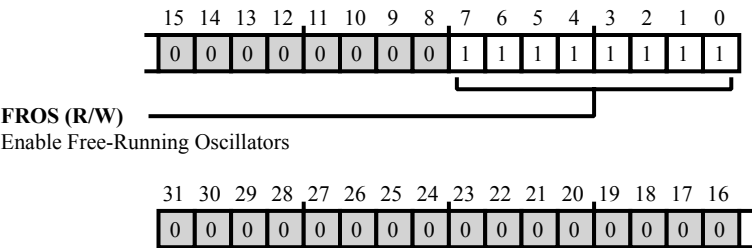


Figure 38-10: TRNG_FROEN Register Diagram

Table 38-13: TRNG_FROEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	FROS	Enable Free-Running Oscillators. The <code>TRNG_FROEN.FROS</code> bits are the enables for the individual FROs. A 1 in bit [n] enables FRO n. The default state is all ones to enable all FROs after power-up. Note that the FROs are not actually started up before the <code>TRNG_CTL.TRNGEN</code> bit is set to 1. These bits are automatically forced to 0 (and cannot be written to 1) when the corresponding bit in the <code>TRNG_ALMSTP</code> register has value 1.

TRNG Input Registers

The `TRNG_INPUT[n]` registers are used as input for post-processor testing (if post processing is available) and as input for Monobit Test, Run Test and Poker Test functionality tests (`TRNG_INPUT0` only). They share their addresses with the corresponding `TRNG_OUTPUT[n]` registers. The least significant word is contained in the `TRNG_INPUT0` register.

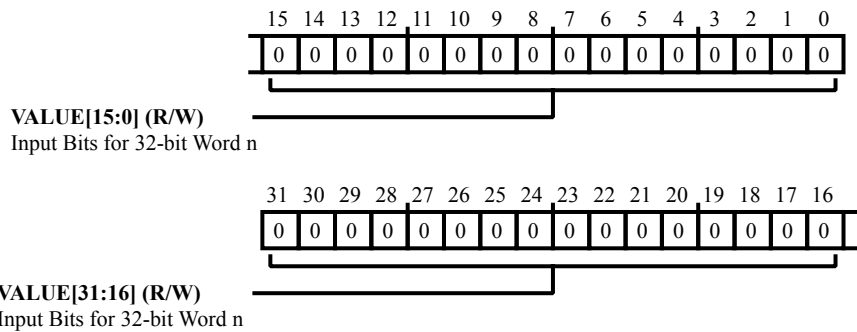


Figure 38-11: TRNG_INPUT[n] Register Diagram

Table 38-14: TRNG_INPUT[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Input Bits for 32-bit Word n. The <code>TRNG_INPUT[n].VALUE</code> bit field is used to hold 32-bits of the 64-bit word of test data for 3-DES post-processor (if available). Or, the <code>TRNG_INPUT[n].VALUE</code> bit field is used to hold 32-bit data word for Run Test and Poker Test circuits self test. Can only be written to when the <code>TRNG_STAT.TSTRDY</code> bit =1.

TRNG Interrupt Acknowledge Register

The `TRNG_INTACK` register is written to acknowledge interrupts indicated in bits [7:0] of the `TRNG_STAT` register. Writing a 1 to any of the bits [7:2] has side effects in resetting various parts of the TRNG core logic which can also be used even if no interrupts are actually active.

When acknowledging the interrupts, these bits are write '1' to clear the associated bit in `TRNG_STAT` register. The bit in this register will also automatically be reset to zero.

When secure reading mode is enabled, write bits [7:0] of this register with zeros to enable TRNG data reads from the `TRNG_OUTPUT[n]` registers. Writing bits [7:0] also starts the (configurable) timeout counter that automatically acknowledges the TRNG data (and disables reads) if the `TRNG_INTACK.RDY` bit is not written with a 1 within that timeout period.

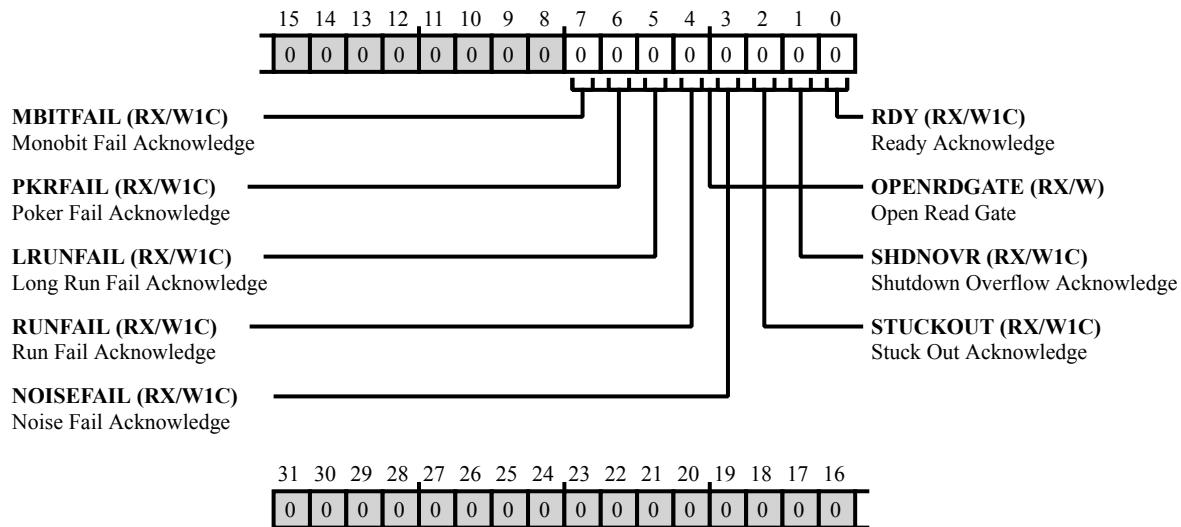


Figure 38-12: TRNG_INTACK Register Diagram

Table 38-15: TRNG_INTACK Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (RX/W1C)	MBITFAIL	Monobit Fail Acknowledge. Set the <code>TRNG_INTACK.MBITFAIL</code> bit to acknowledge the Monobit Fail Interrupt. This also resets all counter and state bits in the <code>TRNG_RUN[n]</code> , <code>TRNG_MONOBITCNT</code> and the <code>TRNG_POKER[n]</code> registers (except for the <code>TRNG_RUNCNT.LENMAX</code> field).
6 (RX/W1C)	PKRFAIL	Poker Fail Acknowledge. Set the <code>TRNG_INTACK.PKRFAIL</code> bit to acknowledge the Poker Fail Interrupt. This also resets all counter and state bits in the <code>TRNG_RUN[n]</code> , <code>TRNG_MONOBITCNT</code> and the <code>TRNG_POKER[n]</code> registers (except for the <code>TRNG_RUNCNT.LENMAX</code> field).

Table 38-15: TRNG_INTACK Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5 (RX/W1C)	LRUNFAIL	Long Run Fail Acknowledge. Set the TRNG_INTACK.LRUNFAIL bit to acknowledge the Long Run Fail Interrupt. Also clears the TRNG_RUNCNT.LENMAX field.
4 (RX/W1C)	RUNFAIL	Run Fail Acknowledge. Set the TRNG_INTACK.RUNFAIL bit to acknowledge the Run Fail Interrupt. Also resets all counter and state bits in the TRNG_RUN[n], TRNG_MONOBITCNT and the TRNG_POKER[n] registers (except for the TRNG_RUNCNT.LENMAX field).
3 (RX/W1C)	NOISEFAIL	Noise Fail Acknowledge. Set the TRNG_INTACK.NOISEFAIL bit to acknowledge the Noise Fail Interrupt. Setting this bit also clears the TRNG_RUNCNT.LENMAX (Run Length Max) field, the random data buffer, the TRNG_OUTPUT[n] registers and the TRNG_STAT.RDY bit.
2 (RX/W1C)	STUCKOUT	Stuck Out Acknowledge. Set the TRNG_INTACK.STUCKOUT bit to acknowledge the Stuck Out Interrupt. Setting this bit also clears the random data buffer, the TRNG_OUTPUT[n] registers and the TRNG_STAT.RDY bit.
1 (RX/W1C)	SHDNOVR	Shutdown Overflow Acknowledge. Set the TRNG_INTACK.SHDNOVR bit to acknowledge the Shutdown Overflow Interrupt.
0 (RX/W1C)	RDY	Ready Acknowledge. The TRNG_INTACK.RDY bit allows a new number (if it is ready in the random data buffer), to directly move into the result register. Once done, the TRNG_STAT.RDY bit is reset, after at most size clock cycles.
7:0 (RX/W)	OPENRDGATE	Open Read Gate. In Secure Reading Mode, the TRNG_INTACK.OPENRDGATE bit writes an all zeros value to bits [7:0] to enable reading of TRNG data from the TRNG_OUTPUT[n] registers. This starts the timeout counter that automatically acknowledges the TRNG data (and disables reading) if the TRNG_INTACK.RDY bit is not written with a 1 within that timeout period.

Post-Process Key Registers

The `TRNG_KEY[n]` registers are used to load the key used for post-processing (if available). These registers are write-only. Reads return the values of the other registers mapped at the same addresses.

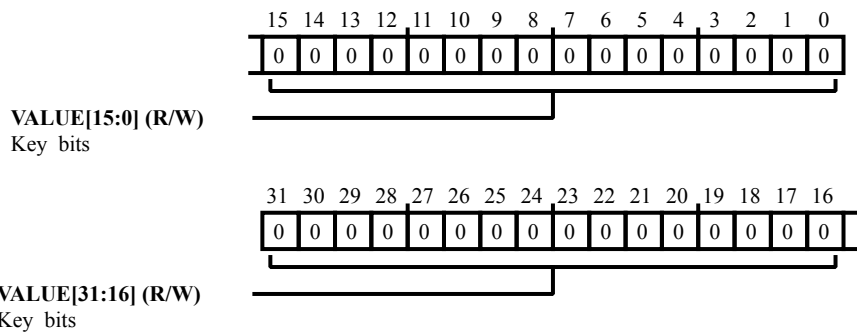


Figure 38-13: TRNG_KEY[n] Register Diagram

Table 38-16: TRNG_KEY[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Key bits. Bits for cipher key used in the post-processor.

TRNG LFSR Access Register

The `TRNG_LFSR_H` register is used to access bits [80:64] of the main entropy accumulation LFSR while in test mode (`TRNG_CTL.TSTMODE = 1`).

For security reasons, the LFSR contents are zeroed before enabling access.

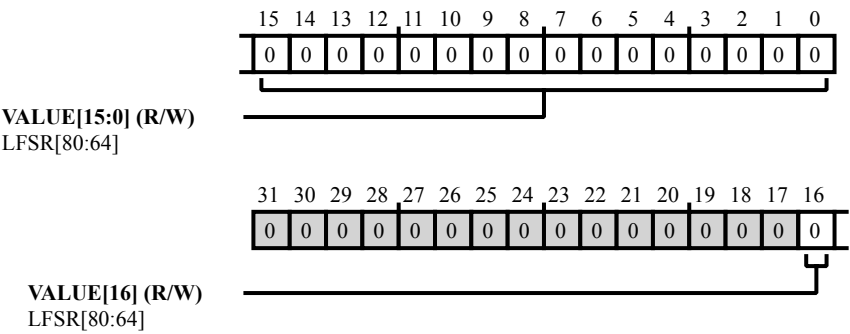


Figure 38-14: TRNG_LFSR_H Register Diagram

Table 38-17: TRNG_LFSR_H Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:0 (R/W)	VALUE	LFSR[80:64]. The <code>TRNG_LFSR_H.VALUE</code> bit field contains bits [80:64] of the main entropy accumulation LFSR. This field can only be accessed when the <code>TRNG_CTL.TSTMODE</code> bit =1. Contents are cleared (=0) before access is enabled.

TRNG LFSR Access Register

The `TRNG_LFSR_L` register is used to access bits [31:0] of the main entropy accumulation LFSR while in test mode (`TRNG_CTL.TSTMODE` =1).

For security reasons, the LFSR contents are zeroed before enabling access.

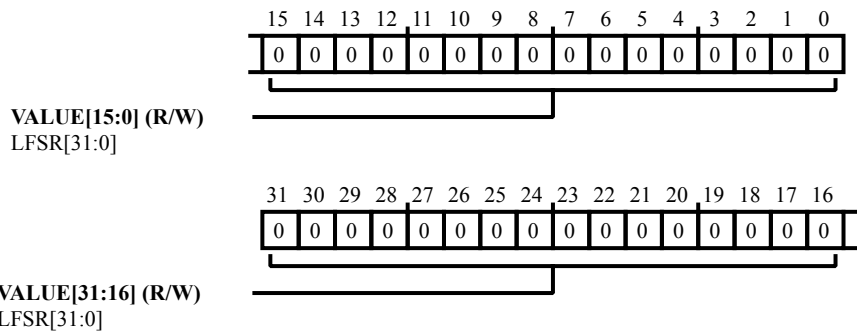


Figure 38-15: TRNG_LFSR_L Register Diagram

Table 38-18: TRNG_LFSR_L Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	LFSR[31:0]. The <code>TRNG_LFSR_L.VALUE</code> bit field contains bits [31:0] of the main entropy accumulation LFSR. This field can only be accessed when the <code>TRNG_CTL.TSTMODE</code> bit =1. Contents are cleared (=0) before access is enabled.

TRNG LFSR Access Register

The `TRNG_LFSR_M` register is used to access bits [63:32] of the main entropy accumulation LFSR while in test mode (`TRNG_CTL.TSTMODE` =1).

For security reasons, the LFSR contents are zeroed before enabling access.

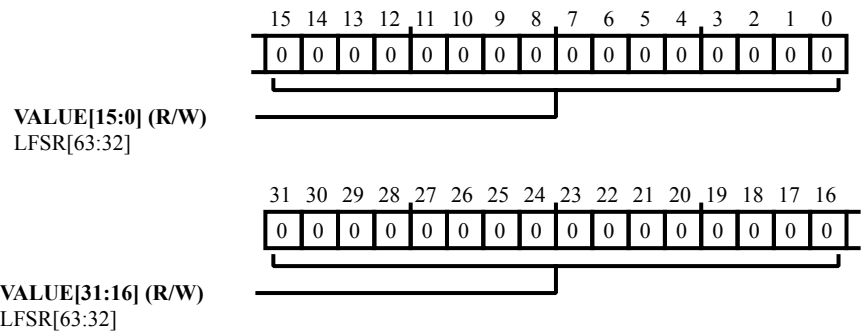


Figure 38-16: TRNG_LFSR_M Register Diagram

Table 38-19: TRNG_LFSR_M Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	LFSR[63:32]. The <code>TRNG_LFSR_M.VALUE</code> bit field contains bits [63:32] of the main entropy accumulation LFSR. This field can only be accessed when the <code>TRNG_CTL.TSTMODE</code> bit =1. Contents are cleared (=0) before access is enabled.

TRNG Monobit Test Result Register

The `TRNG_MONOBITCNT` register accesses the counter used perform a Monobit Test as specified by the AIS-31 standard (test T1, ref 4). This test is performed on blocks of 20,000 bits (in parallel to the run test and Poker Test).

Note: Immediately after performing the actual Monobit Test at the end of the 20,000 bits block, the counter is used to accumulate the Poker Test results. As a result, the actual Monobit Test count result value can only be read in the `TRNG_MONOBITCNT` register if the test fails and the stall run Poker (`TRNG_ALMCNT.STALLRUNPKR`) bit =1.

The monobit test result register is read-only; writing it accesses the registers mapped at the same address. The counter in this register is reset when writing a 1 to either the monobit fail acknowledge (`TRNG_INTACK.MBITFAIL`), run fail acknowledge (`TRNG_INTACK.RUNFAIL`) or the poker fail acknowledge (`TRNG_INTACK.PKRFAIL`) bits.

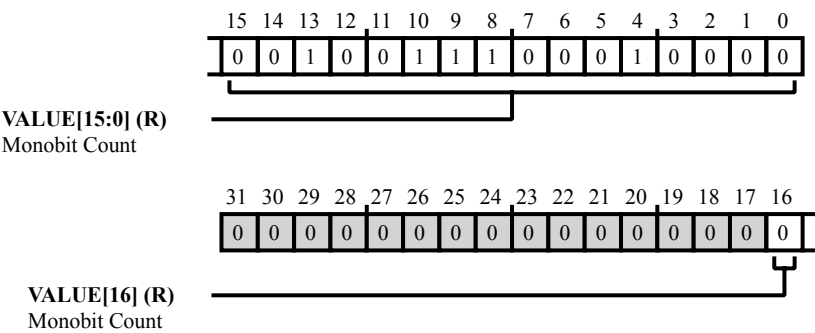


Figure 38-17: `TRNG_MONOBITCNT` Register Diagram

Table 38-20: `TRNG_MONOBITCNT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:0 (R/NW)	VALUE	Monobit Count. The <code>TRNG_MONOBITCNT.VALUE</code> bit field is the up/down counter which monitors 1 and 0 bits. After 20,000 bits, this counter should have a value in the range 9310 through 10690 (inclusive) to pass the Monobit Test. This counter is protected against overflow and underflow.

TRNG Output Registers

The `TRNG_OUTPUT[n]` registers provide read access to the 128-bit random number output. A subset of these registers are also used as output for post-processor testing (if available). They share their addresses with the `TRNG_INPUT0` through `TRNG_INPUT3` registers. The least significant word is contained in the `TRNG_OUTPUT0` register.

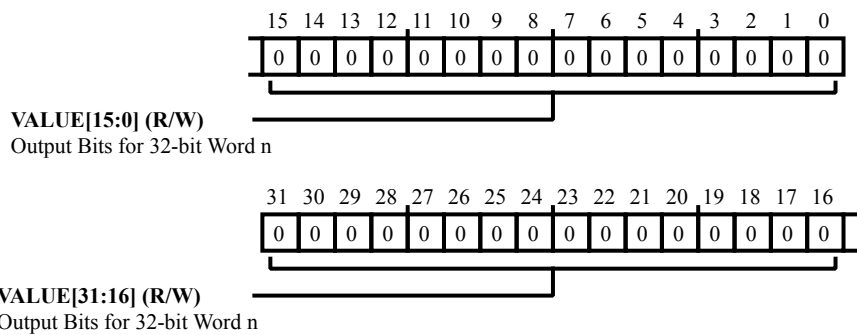


Figure 38-18: `TRNG_OUTPUT[n]` Register Diagram

Table 38-21: `TRNG_OUTPUT[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Output Bits for 32-bit Word n. The <code>TRNG_OUTPUT[n].VALUE</code> bit field is used to holds 32 bits of the 128-bit word of random data. Only valid when the <code>TRNG_STAT.RDY</code> bit =1. Alternatively, this register holds the 32-bits of the 42-bit word of result data for 3-DES post-processing testing. Only valid when the <code>TRNG_STAT.TSTRDY</code> bit =1.

TRNG Poker Test Result Registers

The `TRNG_POKER[n]` registers are used to access the 16 counters used perform a poker test on blocks of 20,000 bits (in parallel to the monobit and run tests).

Poker test result registers are read-only; writing them accesses the registers mapped at these same addresses. All counters in these registers are reset when writing a 1 to either the monobit fail acknowledge (`TRNG_INTACK.MBITFAIL`), run fail acknowledge (`TRNG_INTACK.RUNFAIL`) or the poker fail acknowledge (`TRNG_INTACK.PKRFALL`) bits.

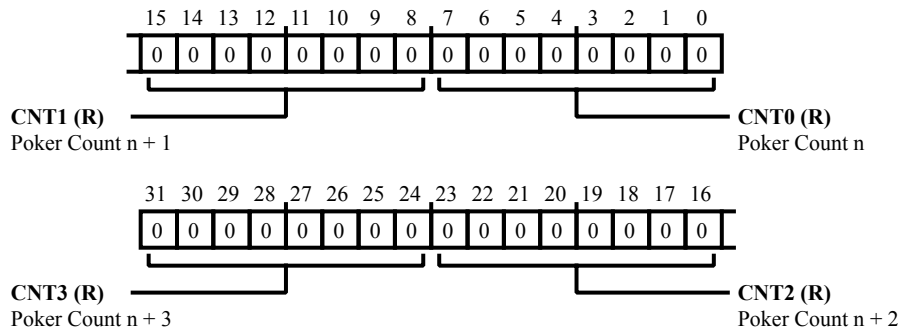


Figure 38-19: `TRNG_POKER[n]` Register Diagram

Table 38-22: `TRNG_POKER[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:24 (R/NW)	CNT3	Poker Count $n + 3$. The <code>TRNG_POKER[n].CNT3</code> bit field provides the counter for 4-bit value 0x3, 0x7, 0xB, 0xF in <code>TRNG_POKER0</code> , <code>TRNG_POKER1</code> , <code>TRNG_POKER2</code> , <code>TRNG_POKER3</code> , respectively.
23:16 (R/NW)	CNT2	Poker Count $n + 2$. The <code>TRNG_POKER[n].CNT2</code> bit field provides the counter for 4-bit value 0x2, 0x6, 0xA, 0xE in <code>TRNG_POKER0</code> , <code>TRNG_POKER1</code> , <code>TRNG_POKER2</code> , <code>TRNG_POKER3</code> , respectively.
15:8 (R/NW)	CNT1	Poker Count $n + 1$. The <code>TRNG_POKER[n].CNT1</code> bit field provides the counter for 4-bit value 0x1, 0x5, 0x9, 0xD in <code>TRNG_POKER0</code> , <code>TRNG_POKER1</code> , <code>TRNG_POKER2</code> , <code>TRNG_POKER3</code> , respectively.
7:0 (R/NW)	CNT0	Poker Count n . The <code>TRNG_POKER[n].CNT0</code> bit field provides the counter for 4-bit value 0x0, 0x4, 0x8, 0xC in the <code>TRNG_POKER0</code> , <code>TRNG_POKER1</code> , <code>TRNG_POKER2</code> , <code>TRNG_POKER3</code> , respectively.

TRNG Run Count Registers

The `TRNG_RUNCNT` registers are used to access the 10 counters that perform a run test and long run test as specified by the AIS-31 standard (tests T3 and T4, ref 4). They are also used to perform the noise source failure test proposed in section E.5 of that same standard.

The `TRNG_RUNCNT` registers are read-only; writing them accesses the other registers which are mapped at the same addresses. Unless otherwise indicated, all counters and state bits in these registers are reset when writing a 1 to either the Monobit Fail acknowledge (`TRNG_INTACK.MBITFAIL`), Run Fail acknowledge (`TRNG_INTACK.RUNFAIL`) or the Poker Fail acknowledge (`TRNG_INTACK.PKRFAIL`) bits.

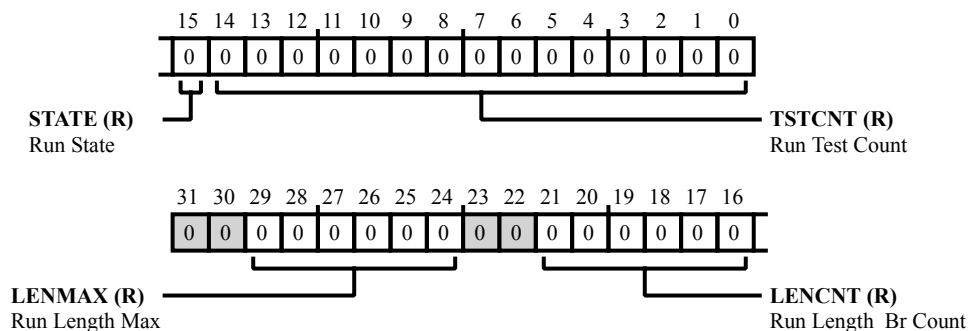


Figure 38-20: `TRNG_RUNCNT` Register Diagram

Table 38-23: `TRNG_RUNCNT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:24 (R/NW)	LENMAX	Run Length Max. The <code>TRNG_RUNCNT.LENMAX</code> bit field configures the maximum run length count value encountered since start of test. This value is reset back to zero when writing a 1 to either the Noise Fail acknowledge (<code>TRNG_INTACK.NOISEFAIL</code>) or the Long Run Fail acknowledge (<code>TRNG_INTACK.LRUNFAIL</code>) bits.
21:16 (R/NW)	LENCNT	Run Length Br Count. The <code>TRNG_RUNCNT.LENCNT</code> bit field configures the counter for the current run of consecutive 0/1 bits; cannot increment past its maximum value of 63.
15 (R/NW)	STATE	Run State. The <code>TRNG_RUNCNT.STATE</code> bit field provides the state of bits in the current run.
14:0 (R/NW)	TSTCNT	Run Test Count. The <code>TRNG_RUNCNT.TSTCNT</code> bit field configures the block length counter for the run and poker tests - counts up for 20,000 tested bits and then controls testing of the <code>run_X_count...</code> and <code>poker_count_X</code> counters to contain expected values, after which they - and this counter - are reset for the next block.

TRNG Run Test State and Result Registers

The `TRNG_RUN[n]` registers holds the counts for the associated run bucket for 1's and 0's.

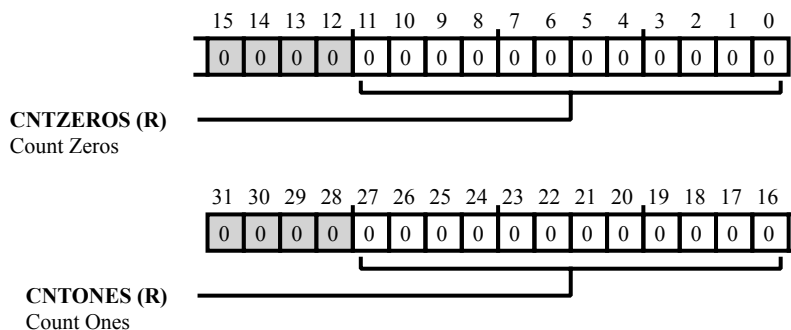


Figure 38-21: TRNG_RUN[n] Register Diagram

Table 38-24: TRNG_RUN[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
27:16 (R/NW)	CNTONES	Count Ones. In TRNG_RUN1, this counter is for single bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 2267 to 2733 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 4095. In TRNG_RUN2, this counter is for two bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 1079 to 1421 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 2047. In TRNG_RUN3, this counter is for three bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 502 to 748 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 1023. In TRNG_RUN4, this counter for four bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 233 to 402 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 511. In TRNG_RUN5, this counter is for five bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 90 to 223 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 255. In TRNG_RUN6, this counter for six and higher bit runs of value one bits. After 20,000 bits, this counter should have a value in the range 90 to 233 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 255.

Table 38-24: TRNG_RUN[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
11:0 (R/NW)	CNTZEROS	Count Zeros. In TRNG_RUN1, this counter is for single bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 2267 to 2733 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 4095. In TRNG_RUN2, this counter is for two bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 1079 to 1421 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 2047. In TRNG_RUN3, this counter is for three bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 502 to 748 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 1023. In TRNG_RUN4, this counter is for four bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 233 to 402 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 511. In TRNG_RUN5, this counter is for five bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 90 to 223 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 255. In TRNG_RUN6, this counter is for six and higher bit runs of value zero bits. After 20,000 bits, this counter should have a value in the range 90 to 233 (inclusive) to pass the run test. This counter cannot increment past its maximum value of 255.

TRNG Status Register

The `TRNG_STAT` register provides status results. This register shares the same address as the Interrupt Acknowledge (`TRNG_INTACK`) register.

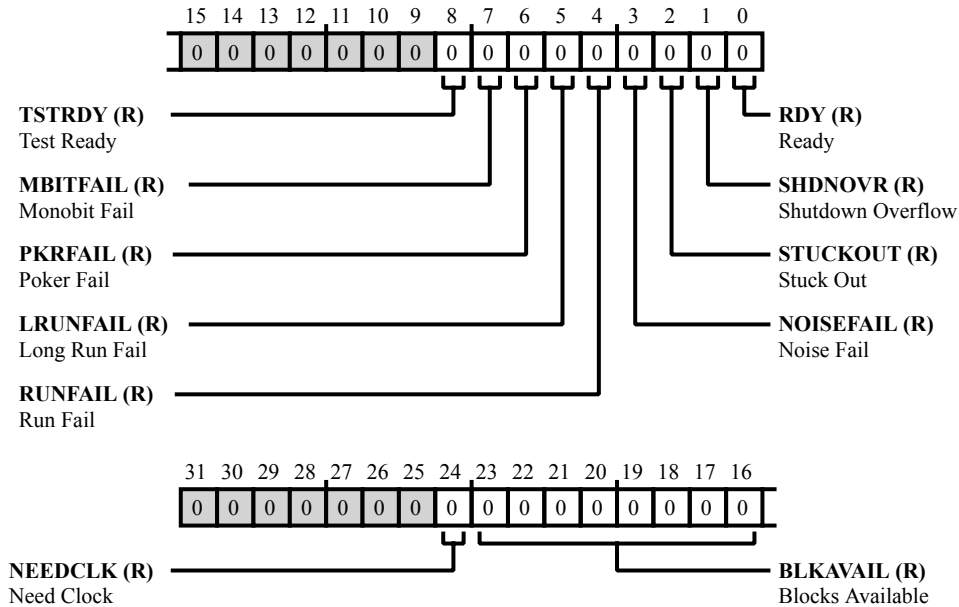


Figure 38-22: TRNG_STAT Register Diagram

Table 38-25: TRNG_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/NW)	NEEDCLK	Need Clock. When the <code>TRNG_STAT.NEEDCLK</code> bit is set, it indicates that the TRNG is busy generating entropy or is in one of its test modes the module clock may not be turned off.
23:16 (R/NW)	BLKAVAIL	Blocks Available. This field indicates the number of 128 bits blocks of random data that are available in the random data buffer. If this value is non-zero, the output registers will be re-filled from the random data buffer immediately after acknowledging the <code>TRNG_STAT.RDY</code> by writing a '1' to <code>TRNG_INTACK.RDY</code> .
8 (R/NW)	TSTRDY	Test Ready. When the <code>TRNG_STAT.TSTRDY</code> bit is set, it indicates that data for known-answer tests on the Monobit Test, Run Test, Poker Test and post-processor functions can be written to the <code>TRNG_INPUT[n]</code> registers. When testing the post-processor, result data can be read from those same registers when this bit has become 1 again (after dropping to 0).

Table 38-25: TRNG_STAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	MBITFAIL	Monobit Fail. When the <code>TRNG_STAT.MBITFAIL</code> bit is set, the Monobit Test logic monitoring data, shifted into the main LFSR, detected an out-of-bounds number of 1s after checking 20,000 bits (test T1 as specified in the AIS-31 standard).
6 (R/NW)	PKRFAIL	Poker Fail. When the <code>TRNG_STAT.PKRFAIL</code> bit is set, the Poker Test logic monitoring data shifted into the main LFSR detected an out-of-bounds value in at least one of the 16 Poker Counters or an out of bounds sum of squares value after checking 20,000 bits (test T2 as specified in the AIS-31 standard).
5 (R/NW)	LRUNFAIL	Long Run Fail. When the <code>TRNG_STAT.LRUNFAIL</code> bit is set, the Run Test logic monitoring data shifted into the main LFSR detected a sequence of 34 identical bits (test T4 as specified in the AIS-31 standard).
4 (R/NW)	RUNFAIL	Run Fail. When the <code>TRNG_STAT.RUNFAIL</code> bit is set, the Run Test logic monitoring data shifted into the main LFSR detected an out-of-bounds value for at least one of the <code>TRNG_RUN[n].CNTZEROS</code> or <code>TRNG_RUN[n].CNTONES</code> counters after checking 20,000 bits (test T3 as specified in the AIS-31 standard).
3 (R/NW)	NOISEFAIL	Noise Fail. When the <code>TRNG_STAT.NOISEFAIL</code> bit is set, the Run Test logic monitoring data shifted into the main LFSR detected a sequence of 48 identical bits, which is considered a noise source failure as proposed in section E.5 of the AIS-31 standard.
2 (R/NW)	STUCKOUT	Stuck Out. When the <code>TRNG_STAT.STUCKOUT</code> bit is set, the logic around the output data registers detected that the TRNG generates the same value twice in a row.
1 (R/NW)	SHDNOVR	Shutdown Overflow. When the <code>TRNG_STAT.SHDNOVR</code> bit is set, the number of FROs shut down after a second error event (the number of 1 bits in the <code>TRNG_ALMSTP</code> register) has exceeded the threshold set by the <code>TRNG_ALMCNT.SHDNTHRESH</code> bit field.
0 (R/NW)	RDY	Ready. When the <code>TRNG_STAT.RDY</code> bit is set, data is available in the <code>TRNG_OUTPUT0</code> to <code>TRNG_OUTPUT3</code> registers. If a new number is already available in the random data buffer, that number is directly moved into the result register. In this case the ready status bit is asserted again, after at most six module clock cycles.

TRNG Test Register

The `TRNG_TEST` register can be used by the host processor to perform a number of tests on the TRNG logic including:

- Register controlled characterization by connecting the `tst_fro_clk_out` output to a selected FRO clock output
- FRO logic connectivity and error event detection checking by feeding known patterns through the FRO delay line and error event detection circuits
- Direct XOR-ed FRO outputs capture by disabling the main LFSR feedback logic
- Extend the Monobit Test and Poker Test by not resetting the Monobit count and Poker Test X counters after each 20,000 bits block
- Perform known answer tests on the Run Test, Poker Test and post-processor functions.

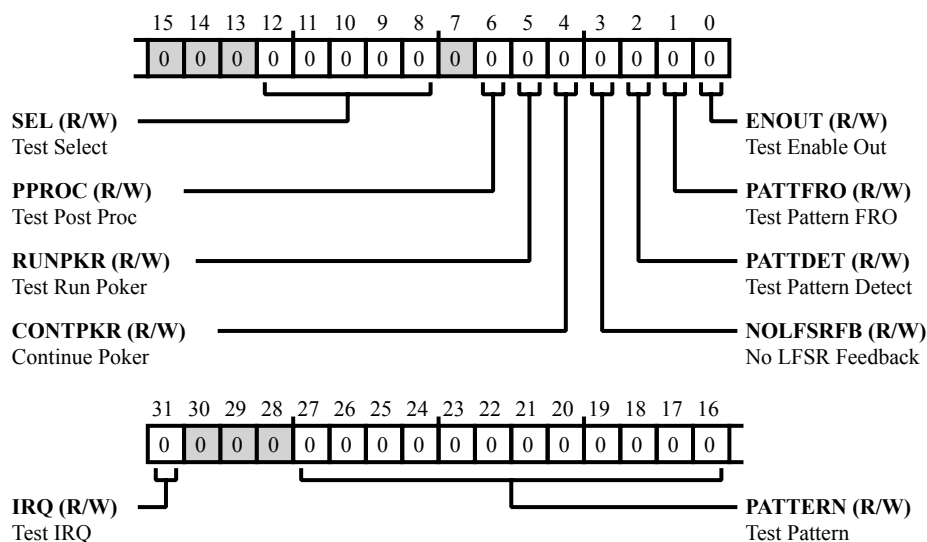


Figure 38-23: TRNG_TEST Register Diagram

Table 38-26: TRNG_TEST Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	IRQ	Test IRQ. When the <code>TRNG_TEST . IRQ</code> bit is set force irq output HIGH for interrupt signal connectivity testing.
		0 Do not force IRQ high
		1 Force IRQ output high

Table 38-26: TRNG_TEST Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
27:16 (R/W)	PATTERN	Test Pattern. The TRNG_TEST . PATTERN bit field sets up a repeating sequence of bits to be fed into the selected FRO delay chain TRNG_TEST . PATTFRO =1 and/or the selected FRO error detection circuit TRNG_TEST . PATTDET =1. This field is rotated right over one bit, once every sample period, when either of these control bits is 1. Therefore, bit [16] is the actual pattern bit fed into the test target.
12:8 (R/W)	SEL	Test Select. The TRNG_TEST . SEL bit field configures the number of the FRO to be tested, the value should be in the range of 0 to 7.
6 (R/W)	PPROC	Test Post Proc. When the TRNG_TEST . PPROC bit is set, it provides direct access to the post-processor for known-answer tests (writing input data to the TRNG_INPUT[n] registers). While this bit is set, the TRNG can continue to generate entropy in the main LFSR and any buffered random data is preserved, to be loaded into the output registers as soon as this bit is reset to 0 again. The need clock output is forced active while this bit is set. For X9.31 post-processors, it is advisable to re-seed the post-processor after running known-answer tests as the original key and V values are modified to known values. This bit is only present when post-processing is available and can only be set to 1 when the TRNG_CTL . PPROCEN bit =1 and the test run poker (TRNG_TEST . RUNPKR) bit in this register is 0.
5 (R/W)	RUNPKR	Test Run Poker. When the TRNG_TEST . RUNPKR bit is set, it provides direct access to the inputs of the Monobit, Run and Poker Test circuits (writing input data in chunks of 32 bits to the TRNG_INPUT0 register). While this bit is 1, the TRNG is not allowed to generate entropy but any buffered random data is preserved, to be loaded into the output registers as soon as this bit is reset to 0 again. The TRNG_STAT . NEEDCLK bit is forced active while this bit is 1. The Monobit, Run and Poker Test circuits are reset to their initial states on any change of this bit.
4 (R/W)	CONTPKR	Continue Poker. When the TRNG_TEST . CONTPKR bit is set, Monobit Test and Poker Test keep running continuously by not resetting the Monobit count (TRNG_MONOBITCNT) and poker counters (TRNG_POKER[n] register) at the end of each 20,000 bits test block. This bit can only be set to 1 when TRNG_CTL . TSTMODE =1.
		0 Do not continue poker test
		1 Continue poker test

Table 38-26: TRNG_TEST Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	NOLFSRFB	No LFSR Feedback. When the TRNG_TEST.NOLFSRFB bit is set, it removes XNOR feedback from the main LFSR, converting it into a normal shift register for the XOR-ed outputs of the FROs (shifting data in on the LSB side). A 1 also forces the LFSR to sample continuously. This bit can only be set to 1 when TRNG_CTL.TSTMODE = 1.
		0 Keep XNOR feedback
		1 Remove XNOR feedback
2 (R/W)	PATTDDET	Test Pattern Detect. When the TRNG_TEST.PATTDDET bit is set, it repeatedly feeds test pattern (PATTERN) into the error detection circuit of the FRO selected by the test select (TRNG_TEST.SEL) field. This bit can only be set to 1 when TRNG_CTL.TSTMODE = 1.
		0 Do not repeat feed test pattern
		1 Repeat feed test pattern
1 (R/W)	PATTFRO	Test Pattern FRO. When the TRNG_TEST.PATTFRO bit is set, it repeatedly feeds test pattern (PATTERN) into the delay chain of the FRO selected by the test select (TRNG_TEST.SEL) field by forcing the corresponding FRO enable (FROEN) output LOW. This bit can only be set to 1 when TRNG_CTL.TSTMODE = 1.
		0 Do not repeat feed test pattern
		1 Repeat feed test pattern
0 (R/W)	ENOUT	Test Enable Out. When the TRNG_TEST.ENOUT bit is set, it enables the tst_fro_clk_out output, connecting to the FRO selected by the test select (TRNG_TEST.SEL) field. This bit can only be set to 1 when TRNG_CTL.TSTMODE = 1.
		0 Disable tst_fro_clk_out
		1 Enable tst_fro_clk_out

TRNG Post-Process "V" Value Registers

The `TRNG_V[n]` registers are used to load the V value used for post-processing (if available). These registers are write-only. Reads return the values of the other registers mapped at the same addresses.

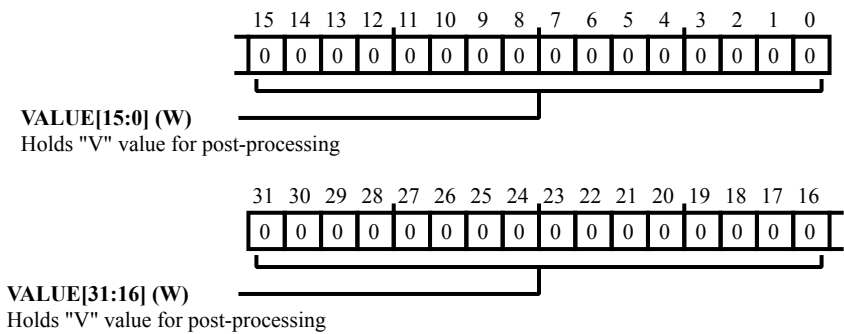


Figure 38-24: TRNG_V[n] Register Diagram

Table 38-27: TRNG_V[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Holds "V" value for post-processing. Bits of the post-processing 'V' value.

39 FIR Accelerator (FIR)

Finite Impulse Response (FIR) filters are frequently used in digital signal processing applications. The FIR accelerator is a dedicated hardware interface used to perform filter processing to reduce the instruction processing load on the core. FIR filters are used in a wide array of applications including multi-rate processing with an interpolator or decimator.

Features

This hardware module can perform FIR filters without core intervention. This feature gives programs freedom to use the core to implement complex algorithms, effectively adding more bandwidth to the processor.

The FIR supports the following features:

- Fixed-point and 32-bit IEEE floating-point format
- Four SHARC+ core compatible MAC units that operate in parallel
- Rounding modes compatible with SHARC+ core MACs
- Single rate or multi-rate window processing
- Programmable rates with decimation or interpolation mode
- Up to 32 filter channels available in TDM in legacy mode
- Burst transfers on data or coefficient loads

NOTE: The FIR accelerator module has a local memory that the core cannot access during regular operation. Unlike previous SHARC processors, the FIR accelerator modules each have access to the system memory (on-chip or off-chip).

Also, unlike the previous SHARC processors, where only one of the FIR or IIR accelerators can be used at a time, the SHARC+ processor can use both accelerators simultaneously.

Clocking

The FIR accelerator runs at the speed of the core clock frequency (CCLK).

Functional Description

The *FIR Block Diagram* shows the 1024-TAP FIR hardware accelerator. The accelerator consists of a 1024 word coefficient memory, a 1024 deep delay line for data, and four MAC units. The accelerator runs at CCLK frequency.

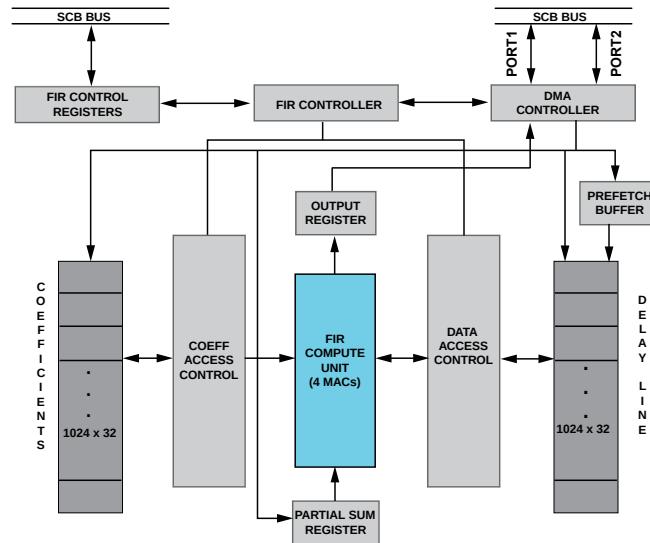


Figure 39-1: FIR Block Diagram

The FIR accelerator has the following logical sub blocks:

1. A datapath unit that consists of:
 - A 1024 deep coefficient memory
 - A 1024 deep delay line for the data
 - Four 32-bit floating-point and fixed-point multiplier and adder units
 - One 32-bit prefetch buffer to operate in a pipelined fashion
 - One 32-bit buffer to hold the previous partial sum
 - One 32-bit buffer to hold the output
2. Configuration registers for the number of TAPs, number of channels, filter enable, interrupt control, DMA enable, up sample or down sample control, and ratios.
3. Core access interface for writing to the DMA and filter configuration registers and reading the status register.
4. DMA bus interface for transferring data and coefficients to and from the accelerator.
5. DMA configuration registers including chain pointer, input, output, and coefficient registers.

The accelerator block is integrated with the SHARC core. For more information about this, see SHARC SCB with FIR/IIR Accelerator.

ADSP-2159x FIR Register List

The FIR accelerator is a dedicated hardware interface used to perform filter processing to reduce the instruction processing load on the core.

Table 39-1: ADSP-2159x FIR Register List

Name	Description
FIR_CHNPTR	FIR Chain Pointer Register
FIR_COEFCNT	FIR Coefficient Count Register
FIR_COEFIDX	FIR Coefficient Index Register
FIR_COEFMOD	FIR Coefficient Modifier Register
FIR_CTL1	FIR Global Control Register
FIR_CTL2	FIR Channel Control Register
FIR_DBG_ADDR	Debug Address Register
FIR_DBG_CTL	FIR Debug Control Register
FIR_DBG_RDDAT	FIR Debug Data Read Register
FIR_DBG_WRDAT	FIR Debug Data Write Register
FIR_DMASTAT	FIR DMA Status Register
FIR_INBASE	FIR Input Data Base Register
FIR_INCNT	FIR Input Data Count Register
FIR_INIDX	FIR Input Data Index Register
FIR_INMOD	FIR Input Data Modifier Register
FIR_MACSTAT	FIR MAC Status Register
FIR_OUTBASE	FIR Output Data Base Register
FIR_OUTCNT	FIR Output Data Count Register
FIR_OUTIDX	FIR Output Data Index Register
FIR_OUTMOD	FIR Output Data Modifier Register
FIR_SCTL1	software control register 1
FIR_SCTL2	software control register 2
FIR_SGCTL	Secondary global control register

ADSP-2159x FIR Interrupt List

Table 39-2: ADSP-2159x FIR Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
166	FIR0_DMA	FIR0 Core 1 DMA	Edge	
167	FIR0_STAT	FIR0 Core 1 Status	Edge	
176	FIR1_DMA	FIR1 Core 2 DMA	Edge	
177	FIR1_STAT	FIR1 Core 2 Status	Edge	
251	FIR0_BUS_ERR	FIR0 Core 1 FIR Bus Error	Edge	
256	FIR1_BUS_ERR	FIR1 Core 2 FIR Bus Error	Edge	

ADSP-2159x FIR Trigger List

Table 39-3: ADSP-2159x FIR Trigger List Masters

Trigger ID	Name	Description	Sensitivity
36	C1_FIR0_DMA	FIR0 Core 1 DMA	Edge
37	C2_FIR0_DMA	FIR1 Core 2 DMA	Edge

Table 39-4: ADSP-2159x FIR Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
21	C1_FIR0_TRGI	FIR0 Core 1 FIR Wait on Trigger Input	Pulse
22	C2_FIR0_TRGI	FIR1 Core 2 FIR Wait on Trigger Input	Pulse

Compute Block

The MAC unit, shown in the *FIR MAC Unit* figure, has four multiply accumulators. The accumulators operate simultaneously on a single filter as described below.

- The MAC unit operates on the data and coefficient fetched from the data and coefficient RAMs
- Each MAC can perform 32-bit floating-point or 32-bit fixed-point MAC operations
- Floating-point format is IEEE-compliant
- Multiply and accumulation operation (addition) are pipelined
- A 32-bit floating-point MAC operation generates 32-bit multiply results
- A 32-bit fixed-point operation generates 80-bit results (64-bit result + 16 guard bits)

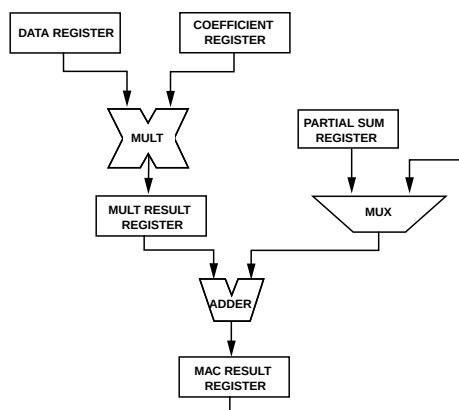


Figure 39-2: FIR MAC Unit

Partial Sum Register

The partial sum register is useful for [Floating-point Multi-Iteration](#) mode. For a particular channel, the intermediate MAC result is written to the output buffer of the system (on-chip or off-chip) memory. If the same channel is requested again, the partial result register is updated with the intermediate MAC result through DMA from output buffer of the system memory. The result is added to the current MAC result after each iteration. This process repeats until all iterations complete (the entire soft filter length is processed).

Delay Line Memory

The accelerator has a 1024 TAP delay line to hold the data locally. The DMA controller fetches the data from system memory and loads it into the delay line. Four read accesses can be made to the delay line simultaneously.

Coefficient Memory

The accelerator has a 1024 deep coefficient memory to store the coefficients. The DMA controller loads the coefficients from system memory into coefficient memory. Four coefficients can be fetched from the coefficient memory simultaneously. If the soft filter length is more than 1024, processing happens in multi-iteration mode.

Prefetch Data Buffer

The prefetch data buffer enables pipeline operation. One data sample is prefetched when the compute unit is operating on the delay line corresponding to the current sample. The data prefetched in this buffer is later used to update the delay line for the next sample. This operation happens in parallel again when the compute unit is not accessing the delay line. In other words, it happens when the compute unit is adding the output from the four MACs and the partial sum register.

Table 39-5: Pipeline Operation for Window Size = 1

Cycles	1	2	3	4	5	6
<i>Output DMA</i>			N	N1	N2	N3
<i>Compute</i>		N	N1	N2	N3	

Table 39-5: Pipeline Operation for Window Size = 1 (Continued)

Cycles	1	2	3	4	5	6
<i>Input DMA</i>	N	prefetch N1	prefetch N2	prefetch N3		

Processing Output

The accelerator uses all four MACs simultaneously to calculate one output sample as shown in the *Multi-Iteration Filtering Flow* figure and the following procedure.

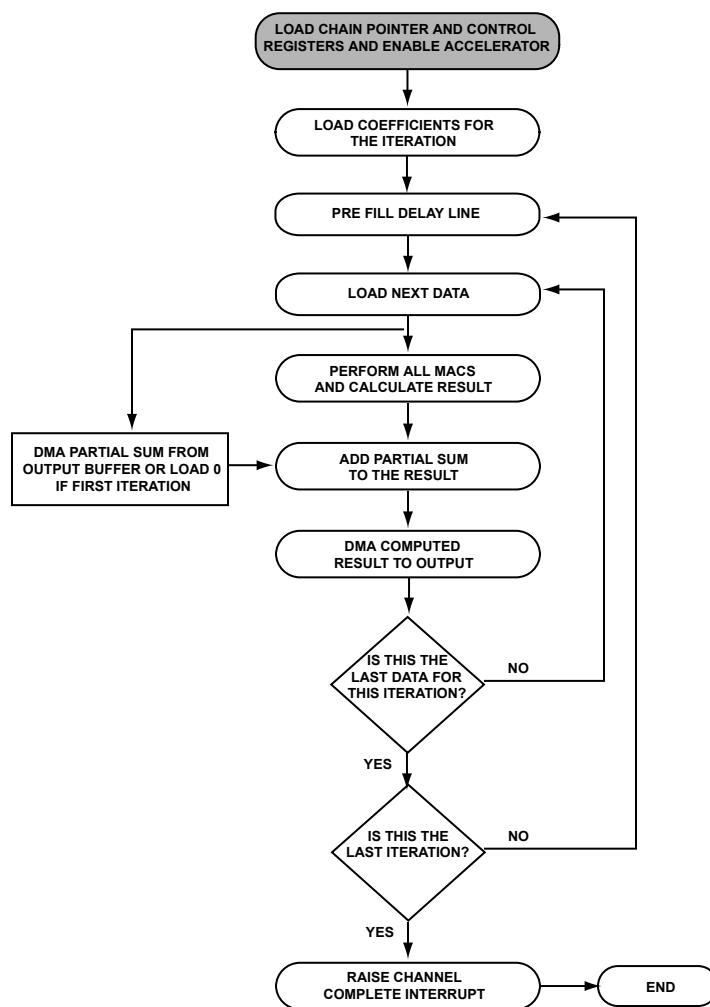


Figure 39-3: Multi-Iteration Filtering Flow

1. The accelerator fetches four input data from the delay line and four corresponding coefficients from the coefficient memory and feeds them to the MAC units for multiply and accumulation operations.
2. The accelerator repeats the procedure with the next four input data and coefficients until all the TAPs complete. For example, this procedure happens $N/4$ times for an N TAP filter.

3. When all the TAPs are complete, the accelerator adds the four MAC outputs together to the previous partial sum (if any) to calculate the final result.
4. Finally, that output sample is stored back in system memory.

System Memory Storage

The following sections describe the storage format for the accelerator.

CAUTION: Store all data in memory aligned to the word address boundaries. Any other programmed addresses do not flag an error.

Coefficients and Input Buffer Storage

For any N TAP filter with coefficients:

```
C[i] i = 0,1,
...
N - 1
```

store the coefficients in system memory buffer in the order:

```
C[N - 1], C[N - 2]
...
C[1], C[0]
```

and CI should point to $C(N - 1)$.

Single Rate Input Filtering

The total size of the input buffer must at least be equal to $N - 1 + W$. If the input buffer that needs to be processed is:

```
x[n], x[n+1], x[n+2]
...
x[n+W-1]
```

store it in the memory as

```
x[n - (N-1)], x[n - (N-2)]
...
x[n-1], x[n], x[n+1]
...
x[n+W-1]
```

and the [FIR_INIDX](#) register value should point to $x[n - (N - 1)]$

Decimation

Assuming M = decimation ratio, the total size of the input buffer should at least be equal to $N - 1 + W \times M$. If the input buffer that needs to be processed is:

```
x[n], x[n+1], x[n+2] ... x[n+W×M-1],
```

store it in the memory as:

```
x[n-(N-1)], x[n-(N-2)]...x[n-1],
x[n], x[n+1]...x[n+WxM-1]
```

and the `FIR_INIDX` register value should point to `x[n-(N-1)]`.

Interpolation

Assuming L = interpolation ratio, the total size of the input buffer should be at least equal to:

$\text{Ceil}((N-1)/L) + W/L$.

If the input buffer that needs to be processed is:

```
x[n], x[n+1], x[n+2]...x[n+W/L-1],
and K = Ceil((N-1)/L)
```

store it in the memory as:

```
x[n-k], x[n-(K-1)], x[n-(K-2)]...x[n-1],
x[n], x[n+1]...x[n+W/L-1]
```

and the `FIR_INIDX` register value should point to `x[n-K]`.

Operating Modes

The FIR core performs a sum-of-products operation to compute the convolution sum. It supports single-rate, decimation, and interpolation functions.

Single Rate Processing

In a single-rate filter, the output result rate is equal to the input sample rate. The filter output $Y(n)$ is computed according to following equation where N is the number of filter coefficients: $c(k)$ $k = 0$ to $N - 1$ are the filter coefficients and $x(n)$ represents the input time-series.

$$Y(n) = \sum_{k=0}^{N-1} c(k) \cdot x(n-k)$$

Figure 39-4: Filter Output Calculation

Single Iteration

Results are computed in a single iteration when the soft filter length is less than or equal to 1024.

Floating-point Multi-Iteration

Results are computed in multiple iterations when the soft filter length is greater than 1024 (for example, 2048 TAPs on a 1024 hard filter length). In this mode, the accelerator implements two iterations of 1024 TAPs.

NOTE: If the soft filter length is not a multiple of the hard filter length, the accelerator iterates until the soft filter length is satisfied.

Example: 550 taps on a 256 tap filter. In this example, the FIR accelerator implements two iterations of 256 taps and one iteration of 38 taps.

NOTE: Multi-iteration mode is not supported in fixed-point format.

Window Processing

In window-based mode, multiple output samples (up to 1024) equal to the window size of that channel are calculated. After these calculations are complete, the accelerator begins processing the next channel. A configurable window size parameter is provided to specify the length of the window.

To select sample-based processing mode, configure the window size to 1. In this mode, one sample from a particular channel is processed through all the taps of that channel and the final output sample is calculated.

Multi-Rate Processing

Multi-rate filters change the sampling rate of a signal—they convert the input samples of a signal to a different set of data that represents the same signal sampled at a different rate.

Decimation

A decimation filter provides a single output result for every M input samples, where M is the decimation ratio. The output rate is $1/M$ 'th of the input rate. The filter implementation exploits the low output sample rate by not starting a computation until a new set of M input samples is available.

In this mode, after low-pass filtering (for anti-aliasing), FIR logic discards the ratio – 1 samples of output data. For performance optimization, FIR logic skips the computation of output samples, which are discarded.

The input buffer size for decimation filters is $N - 1 + (W \times M)$ where:

- N is the number of taps
- W is the window size
- M is the decimation ratio

The window size (`FIR_CTL2.WINDOW` bits) must be programmed with the number of output samples.

To start this mode, programs set the `FIR_CTL2.RATIO` and `FIR_CTL2.UPSAMP` bits (along with normal filter setting). Also, the `FIR_CTL2.TAPLEN` bit field value should be greater than or equal to the `FIR_CTL2.RATIO` bit field value for the decimation filter.

Interpolation

An interpolation filter provides L output results for each new input sample, where L is the interpolation ratio. The output rate is L times the input rate.

In this mode, according to the ratio specified in configuration register, FIR logic inserts $L - 1$ zeros between any two input samples (upsampling). It then performs the interpolation (through the FIR filter).

Both upsampling and downsampling do not support multi-iteration mode. Therefore, the filtering operation only happens on up to 1024 TAPs and the ratio of up and downsampling can only be an integer value.

In an interpolation filter, FIR logic inserts $L - 1$ zeros between each sample. The program has to make sure that these zeros fully shift out of the delay line before moving on to the next channel. This operation puts a restriction on window size in terms of L – *the sample ratio* as showing in the expression:

$WINDOWSIZE = n \times SAMPLERATIO$, where n is the number of input samples.

The input buffer size is smallest integer greater than or equal to $(N - 1 + W)/L$ for interpolation filters where:

- N is the number of taps
- W is the window size
- L is the interpolation ratio

To start the mode, programs configure the `FIR_CTL2.RATIO` and `FIR_CTL2.UPSAMP` bits (along with filter settings).

Floating-Point Data Format

The FIR accelerator treats data and coefficients in 32-bit floating-point format as the default functional mode.

Fixed-Point Data Format

In fixed-point mode, the 32-bit input data or coefficient is treated as fixed point. A 32-bit fixed-point MAC operation generates an 80-bit result. Fixed-point data or coefficients can be unsigned integer, unsigned fractional and signed integer.

NOTE: In fixed-point mode, the entire 80-bit result register is always written back in bursts of 3×32 bits. The first word is the LSW, the second word is the MSW, and the third word is a 16-bit overflow. The remaining 16 bits are padded with zeros. Therefore, for fixed-point mode: $WINDOWSIZE = WINDOWSIZE \times 3$.

If the signed fractional format is used, the output must be scaled by 2. The MAC does not right shift to remove the redundant sign bit. A final routine must decimate the output buffer to the desired samples.

Multi-iteration mode is not supported in this format. Therefore, the maximum TAP length is 1024.

Auto Configuration Mode (ACM)

The accelerator can be operated in legacy mode or Auto Configuration Mode (ACM), controlled by the `FIR_CTL1.ACM` bit. The default functional mode is legacy mode. The accelerator mode can only be changed when the accelerator is disabled.

The ACM provides the following additional features:

- Halt and Queuing

Core may pause the current TCB chain being processed by setting the `FIR_CTL1.HALT` bit. The accelerator acknowledges the core by setting the `FIR_DMASTAT.HALT_STAT` bit. The core can appropriately take action to submit/insert new TCB's. After the core takes action, the accelerator processing can be resumed by clearing the `FIR_CTL1.HALT` bit. Before halting the accelerator, if the initial TCB chain is processed, the accelerator comes to idle state. In this case, the accelerator has to be disabled and enabled by toggling the `FIR_CTL1.EN` bit and the `FIR_CTL1.HALT` bit has to be cleared to resume processing.

- No Channel Number Limitation

Unlike in legacy mode, there is no fixed channel number limitation and, therefore, the accelerator ignores the value programmed in the `FIR_CTL1.CH` field. The application can queue unlimited number of channels/TCBs dynamically and the accelerator keeps processing the TCBs until the chain pointer becomes null.

- Selective Interrupt

Core can enable/mask interrupt generation for each channel using the `FIR_CTL2.IMASK` bit. If the bit is cleared, an interrupt is generated after completion of the channel.

- Selective Master/Slave Trigger

Core can enable/mask trigger generation by the accelerator after the end of processing of each channel using the `FIR_CTL2.TMASK` bit. The accelerator can also wait for a trigger after loading the TCB and coefficients and before processing a channel for which the `FIR_CTL2.TWAIT` bit is set.

In addition to the above features, there are three additional fields as part of the TCB `FIR_SCTL1`, `FIR_SCTL2`, and `FIR_SGCTL`. The `FIR_SGCTL` can be used to change the `FIR_CTL1` parameters such as rounding mode and fixed point mode for each channel. The `FIR_SCTL1` and `FIR_SCTL2` can be used as general-purpose registers.

Data Transfer

The FIR filter works exclusively through DMA.

Chain Assignment

The structure of a TCB is conceptually the same as a traditional linked-list. Each TCB has several data values and a pointer to the next TCB. Further, the chain pointer of a TCB can point to itself to continuously re-run the same DMA. The FIR accelerator reads each word of the TCB and loads it into the corresponding register. The end of the chain (no further TCBs are loaded) is indicated by a TCB with a chain pointer register value of zero.

The FIR accelerator DMA supports circular buffer chained DMA. The FIR accelerator does not support circular buffering for the coefficient buffer.

Table 39-6: TCBs for Chained DMA in Legacy Mode

Address	Register
TCB	<code>FIR_CHNPTR</code>

Table 39-6: TCBs for Chained DMA in Legacy Mode (Continued)

Address	Register
TCB + 0x1	FIR_COEFCNT
TCB + 0x2	FIR_COEFMOD
TCB + 0x3	FIR_COEFIDX
TCB + 0x4	FIR_OUTBASE
TCB + 0x5	FIR_OUTCNT
TCB + 0x6	FIR_OUTMOD
TCB + 0x7	FIR_OUTIDX
TCB + 0x8	FIR_INBASE
TCB + 0x9	FIR_INCNT
TCB + 0xA	FIR_INMOD
TCB + 0xB	FIR_INIDX
TCB + 0xC	FIR_CTL2

Table 39-7: TCBs for Chained DMA in Auto Configuration Mode

Address	Register
TCB	FIR_CHNPTR
TCB + 0x1	FIR_SCTL1
TCB + 0x2	FIR_SCTL2
TCB + 0x3	FIR_SGCTL
TCB + 0x4	FIR_COEFCNT
TCB + 0x5	FIR_COEFMOD
TCB + 0x6	FIR_COEFIDX
TCB + 0x7	FIR_OUTBASE
TCB + 0x8	FIR_OUTCNT
TCB + 0x9	FIR_OUTMOD
TCB + 0xA	FIR_OUTIDX
TCB + 0xB	FIR_INBASE
TCB + 0xC	FIR_INCNT
TCB + 0xD	FIR_INMOD
TCB + 0xE	FIR_INIDX
TCB + 0xF	FIR_CTL2

The `FIR_COEFCNT` register is loaded with the values in the `FIR_COEFCNT` TCB field and is decremented from that value onwards. However, coefficient loading continues until the number of coefficients, equal to the tap length, are read. This condition is true even if the `FIR_COEFCNT` register reaches zero as in the case of a tap length = 10, and the `FIR_COEFCNT` field in the TCB is initialized to 0. The value in the `FIR_COEFCNT` register is -10 after all coefficients are loaded.

NOTE: Initialize `FIR_CHNPTR` to TCB + 12 in legacy mode and TCB + 15 in ACM.

DMA Access

The FIR accelerator has two DMA channels (accelerator input and output) to connect to the system memory. The DMA controller fetches the data and coefficients from memory and stores the result.

Burst Access Support

Burst access enhances the throughput of the DMA channel and reduces the overall load on the system fabric. Burst support is provided for TCB, data, and coefficient loads on the DMA channel. The FIR module supports burst transfers of size SINGLE, INCR4 INCR8 and INCR16. For TCB load, a burst access of length 16 is issued thereby reducing the TCB load time.

An additional bit is provided in the `FIR_CTL1` register for enabling the burst feature. Also, burst transfers are always of size SINGLE when the modifier is other than 1. In case of burst transfers around the circular buffer boundary, the design ensures that the burst does not cross the buffer boundary.

Data and Coefficient Load in Parallel

To reduce the overall loading time of data and coefficients, the operation occurs over separate channels. After the TCB load, both data and coefficients are loaded in parallel over two DMA channels (CH0 and CH1). If there is no conflict in the system when accessing the data and coefficients, the loading time is significantly improved over when loaded sequentially using a single channel. This mode is enabled by default and can be disabled by setting the `FIR_CTL1.DCP_DIS` bit. When disabled, coefficient load and data load occur sequentially over a single channel (CH0).

Accelerator TCB

The location of the DMA parameters for the next sequence comes from the chain pointer register that points to the next set of DMA parameters stored in the internal memory of the processor. In chained DMA operations, the accelerator automatically initializes and starts another DMA transfer when the current DMA transfer is complete. Each new set of parameters is stored in a user-initialized memory buffer or TCB for a chosen peripheral.

Chain Pointer DMA

The DMA controller supports circular buffer chain pointer DMA. One TCB must be configured for each channel. It contains the following:

- A control register value to configure the filter parameters (such as filter tap length, window size, sample rate conversion settings) for each channel. In ACM, additional parameter such as interrupt mask, trigger mask, and trigger wait are also available.
- Software control register values in ACM for each channel.
- Secondary control register value to configure rounding mode, fixed point mode, and two's complement for each channel.
- DMA parameter register values for the input data (delay line).
- DMA parameter register values for coefficient load.
- DMA parameter register values for output data.

Intermediate results in multi-iteration mode are saved in the output buffer.

As shown in the *Circular Buffer Addressing* figure, the accelerator loads the TCB into its internal registers and uses these values to fetch coefficients and data and to store results. After processing a window of data for any channel, the accelerator writes back the appropriate values to the `FIR_INIDX` and `FIR_OUTIDX` bit fields of the TCB in memory. Then, data processing can begin from where it left off during the next time slot of that channel.

The write-back value for input buffer is:

- `FIR_INIDX + W` for single rate filtering
- `FIR_INIDX + W × M` for decimation (M = decimation ratio)
- `FIR_INIDX + W/L` for interpolation (L = interpolation ratio)
- The write-back value for output buffer in floating point mode is: `FIR_OUTIDX + W`
- The write-back value for output buffer in fixed-point mode is: `FIR_OUTIDX + 3 × W`

NOTE: The `FIR_CTL2` register is part of the FIR TCB. This configuration allows programming individual FIR channels with different control attributes.

In ACM, when the `FIR_CTL1.SMQ_LIUPS_EN` bit is set, the accelerator updates the `FIR_INIDX` and `FIR_OUTIDX` bit fields of the TCB in memory after processing a window of data and according to the circular buffer scheme. When the `FIR_CTL1.SMQ_LIUPS_EN` bit is cleared, the accelerator updates the `FIR_INIDX` and `FIR_OUTIDX` bit fields of the TCB in memory to 0x00000000 and 0xFFFFFFFF after processing a window of data. The `FIR_CTL1.SMQ_LIUPS_EN` bit is only valid in ACM.

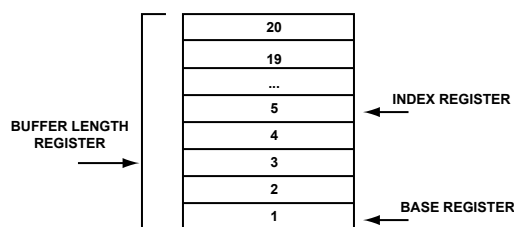


Figure 39-5: Circular Buffer Addressing

Programming Model

The following sections provide general programming information for the FIR accelerator.

Legacy Mode

The *Wait for Core Intervention $\geq$ Idle (if auto channel iterate bit = 0)* figure shows the diagram for multichannel filtering. Multiple channels are processed in a time division multiplexed (TDM) format. After completing all the channels, the accelerator can either repeat the slots or wait for core intervention.

For multichannel filtering, use the following steps.

1. Program the number of channels using the `FIR_CTL1.CH` bits.
2. Configure the TCBs in system memory with one channel's TCB pointing to the next channel's TCB.
3. Write the first TCB value into the `FIR_CHNPTR` register and enable the accelerator.

The accelerator fetches the first channel's TCB and, using it as pointer, prefills the delay line and coefficient memory and loads the `FIR_CTL2` register to configure the filter parameters corresponding to that channel.

The accelerator then calculates output samples corresponding to one window and stores the data back in internal memory.

At the end of the window the accelerator updates the `FIR_INIDX` and `FIR_OUTIDX` registers in the TCB of system memory and moves to the next channel.

When all the channels are finished and the auto channel iterate bit (`FIR_CTL1.CAI`) = 1, the accelerator processes the first channel again and iterates through the channels. If the `FIR_CTL1.CAI` bit = 0, the accelerator waits for core intervention.

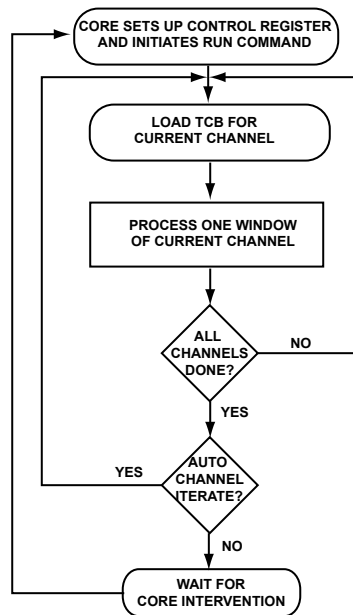


Figure 39-6: Wait for Core Intervention $\geq$ Idle (if auto channel iterate bit = 0)

NOTE: All the addresses programmed in the TCB correspond to 32-bit address boundaries and should not contain the lower 2 bits (assumed as zeros).

Auto Configuration Mode (ACM)

The figure shows multi-channel filtering in ACM. Multiple channels are processed in a time multiplexed format (TDM).

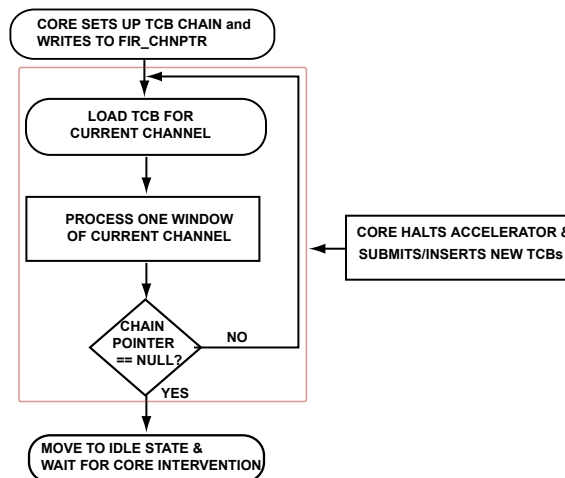


Figure 39-7: Multi-Channel Filtering in Auto Configuration Mode

For multi-channel filtering:

1. Configure the TCBs in system memory with one channel's TCB pointing to the next channel's TCB. There is no limit on the number of channels to be configured.

2. Write the first TCB value to the `FIR_CHNPTR` register and enable the accelerator.

The accelerator fetches the first channel's TCB. Using it as pointer, it prefills the delay line and coefficient memory and loads the `FIR_CTL2` and `FIR_SGCTL` registers to configure the filter parameters corresponding to that channel.

If the `FIR_CTL2.TWAIT` bit is set, the accelerator waits for a input trigger to start the window processing for the channel.

The accelerator then calculates output samples corresponding to one window and stores the data back in internal memory. If the `FIR_CTL2.TMASK` bit is cleared the accelerator sends an output trigger after completion of processing of the channel.

If the `FIR_CTL2.IMASK` bit is cleared, the accelerator interrupts the core after completion of processing of a particular channel.

At the end of the window, the accelerator updates the `FIR_INIDX` and `FIR_OUTIDX` registers to 0x00000000 and 0xFFFFFFFF in the TCB of system memory and moves to the next channel.

3. At any instant, as required, the core halts the accelerator. It sets the `FIR_CTL1.HALT` bit and appropriately takes action to submit or insert new TCBs and clears the `FIR_CTL1.HALT` bit to resume channel processing.

If the `FIR_CHNPTR` register is zero (last channel is being processed or channel processing is complete) after halting the accelerator, the `FIR_CHNPTR` register is appropriately written before resuming the accelerator channel processing.

If the accelerator is idle after halt, `FIR_CTL1.EN` bit is toggled to disable and re-enable the accelerator and the `FIR_CTL1.HALT` bit is cleared and the accelerator resumes channel processing.

4. The accelerator continues processing until all the channels are complete. Repeat *Step 3* (if required) to submit/insert new channels.

NOTE: Channel Auto Iterate (`FIR_CTL1.CAI`) is not supported in ACM.

Debug Mode

The next sections show the steps required for reading and writing local memory in debug mode.

Write to Local Memory

1. Clear the `FIR_CTL1.DMAEN` bit.
2. Set the `FIR_DBG_CTL.EN`, `FIR_DBG_CTL.MEM`, and `FIR_DBG_CTL.HLD` bits.
3. Set the `FIR_DBG_CTL.ADRINC` bit for address auto increment.
4. Write the start address to the `FIR_DBG_ADDR` register.

NOTE: If the bit 11 in the `FIR_DBG_ADDR` register is set, coefficient memory is selected.

5. Write data to the `FIR_DBG_WRDAT` register.

Read from Local Memory

1. Clear the `FIR_CTL1.DMAEN` bit.
2. Set the `FIR_DBG_CTL.EN`, `FIR_DBG_CTL.MEM`, and `FIR_DBG_CTL.HLD` bits.
3. Set the `FIR_DBG_CTL.ADRINC` bit for address auto increment.
4. Write the start address to the `FIR_DBG_ADDR` register.

NOTE: If bit 11 in the `FIR_DBG_ADDR` register is set, coefficient memory is selected.

5. Read data from the `FIR_DBG_RDDAT` register.

Single-Step Mode

Single-step mode can be used for debug purposes. An extra debug register is used in this mode.

1. Enable stop DMA during breakpoint hit in the emulator settings.
2. Clear the `FIR_DBG_CTL.HLD` bit and enable `FIR_DBG_CTL.EN` and `FIR_DBG_CTL.RUN` bits.
3. Program the FIR module according to the application.
4. In single-step mode, each iteration is updated in the emulator session.

Computing FIR Output, Tap Length Greater than 4096

With little core intervention, the FIR accelerator can also be used to calculate output for a tap length greater than 4096 taps. The section demonstrates the calculation with an example of 8192 taps.

1. Divide the transfer function of an 8192 FIR filter into two 4096 FIR filters:

$$\begin{aligned}
 H(Z) &= b_0 + b_1Z^{-1} + b_2Z^{-2} + \dots b_{4095}Z^{-4095} + b_{4096}Z^{-4096}b_{4097}Z^{-4097} + \dots b_{8191}Z^{-8191} \\
 &= b_0 + b_1Z^{-1} + b_2Z^{-2} + \dots b_{4095}Z^{-4095} + Z^{-4096}(b_{4096} + b_{4097} + \dots b_{8191}Z^{-4096})
 \end{aligned}$$

2. Divide the filter coefficients of an 8192 tap filter among two 4096 tap FIR filters:

Filter 1

Coefficients = $b_0, b_1, b_2, \dots, b_{4095}$

Input data = $x[n], x[n-1], \dots, x[n-4095]$

Filter 2

Coefficients = $b_{4096}, b_{4097}, \dots, b_{8191}$

Input data = $x[n-4096], x[n-4097], \dots, x[n-8191]$

The accelerator can be used in two-channel mode where channel 1 operates on $x[n] \dots x[n-4095]$ input data with the filter coefficients of filter 1 and channel 2 operates on $x[n-4096] \dots x[n-8191]$ with the filter coefficients of filter 2.

Once both the channels are processed, add the partial sum output of both the channels can to get the final output. Implement this approach (tap length = TAPS = 8192, window size = WINDOW) using the following programming steps.

1. Create a circular input data buffer in system memory (IBUF). The buffer must be large enough to avoid over-writing data before the accelerator processes it. Ideally, the input buffer size for a channel is $TAPS + WINDOW - 1$.
2. Create a coefficient buffer of size TAPS (8192) (CBUF).
3. Create one output buffer of size WINDOW (OBUF) and another temporary output buffer (OBUF1) to store the partial sum.
4. Create two TCBs in system memory with first TCB chained to the second and second one chained to the first in circular manner.
 - a. The `FIR_COEFIDX` bit field of the first TCB should point to the start address of the coefficient buffer (CBUF) and that of the second TCB should point to 4096 offset from the start of the coefficient buffer (CBUF + 4096).
 - b. The `FIR_OUTBASE` and `FIR_OUTIDX` bit field of the first TCB should point to the start address of OBUF and that of the second TCB should point to the start address of OBUF1.
 - c. The `FIR_INIDX` bit field of the first TCB should point to the start address of IBUF and that of the second TCB should point to 4096 offset from the start address of IBUF.
 - d. Configure the `FIR_CTL2` bit field of both the TCB for tap length = $TAP/2 = 4096$ and window size = WINDOW.
5. Initialize the `FIR_CHNPTR` register pointing to the first TCB.
6. Program the `FIR_CTL1` register to initiate the accelerator processing now by setting the `FIR_CTL1.EN` and `FIR_CTL1.DMAEN` bits and the number of channels configured as 2.
7. Wait for the FIR all channel done interrupt (`FIR_DMASTAT.ACDONE`) to occur. Inside the ISR, add the partial sum results using the core from both the output buffers (OBUF and OBUF1) to get the final output. To save memory, replace the contents of the buffer OBUF with the final output result.

Debug Features

The following sections provide information for debugging the FIR accelerator.

Local Memory Access

The contents of FIR delay line and coefficient memories are made observable for debug by setting the `FIR_DBG_CTL.EN`/`FIR_DBG_CTL.MEM` and `FIR_DBG_CTL.HLD` bits. The debug address register (`FIR_DBG_ADDR`) and two data registers are provided for debug operations. Bit 11 of the `FIR_DBG_ADDR` register selects coefficient memory when set (=1) and selects delay line memory when cleared (=0).

In the debug mode, the read data register ([FIR_DBG_RDDAT](#)) returns the contents of the memory location pointed to by the address register. Data can be written into any memory location using [FIR_DBG_WRDAT](#) register writes. If the address auto-increment bit ([FIR_DBG_CTL.ADRINC](#)) is set, the address register auto-increments on [FIR_DBG_WRDAT](#) writes and [FIR_DBG_RDDAT](#) reads. During auto-increment the [FIR_DBG_ADDR](#) register cannot cross the data memory or coefficient memory boundary.

Single-Step Mode

Programs can single step through the MAC operations and observe the memory contents after each step. The [FIR_DBG_CTL.EN](#), [FIR_DBG_CTL.HLD](#), and [FIR_DBG_CTL.MEM](#) bits control the FIR MAC units.

Emulation Considerations

In FIR debug mode, the DMA operations are not observable.

Interrupts

The *FIR Interrupt Overview* table provides the source of interrupt and service instructions for the FIR interrupts.

Accelerator Mode	Default Programmable Interrupt	Sources	Masking	Service
Legacy Mode	FIR_DMA	Window Complete	N/A	ROC from FIR_DMASTAT + RTI instruction
		All channels complete		
	FIR_STAT	MAC IEEE floating point exceptions	N/A	ROC from FIR_MACSTAT + RTI instruction
		MAC fixed point overflow		
Auto Configuration Mode	FIR_DMA	Window Complete	FIR_CTL2.IMASK	ROC from FIR_DMASTAT + RTI instruction
	FIR_STAT	MAC IEEE floating point exceptions	N/A	ROC from FIR_MACSTAT + RTI instruction
		MAC fixed point overflow	N/A	

Sources

The FIR module drives two interrupt signals: [FIR_DMA](#) for the DMA status and [FIR_STAT](#) for the MAC status. The FIR module generates interrupts as described in the following sections.

Window Complete

This interrupt is generated at the end of each channel when all the output samples are calculated corresponding to a window and updated index values are written back.

In legacy mode, if the `FIR_CTL1.CCINTR` bit is set, an interrupt is generated after completion of window processing of each channel. In ACM, the interrupt generation can be selectively masked using the `FIR_CTL2.IMASK` bit for each channel.

All Channels Complete

This interrupt is generated when all the channels are complete or when one iteration of time slots completes. Note that the interrupt follows the access completion rule, where the interrupt is generated when all data are written back to system memory.

The All Channels Complete interrupt source is only applicable in legacy mode. If the `FIR_CTL1.CCINTR` bit is not set in legacy mode, all channel complete interrupt is generated. In ACM, interrupt generation for each channel is controlled using the `FIR_CTL2.IMASK` bit.

NOTE: The `FIR_CTL1.CCINTR` bit is valid only in legacy mode. In ACM, interrupt generation for each channel is controlled using the `FIR_CTL2.IMASK` bit.

MAC Status

A MAC status interrupt is generated under the following conditions and is reflected in the `FIR_MACSTAT` register.

- Multiplier result zero – Set if multiplier result is zero
- Multiplier result infinity – Set if multiplier result is infinity
- Multiply invalid – Set if multiply operation is invalid
- Adder result zero – Set if adder result is zero
- Adder result infinity – Set if adder result is infinity
- Adder invalid – Set if addition is invalid
- Adder overflow – for fixed-point operation

Service

The DMA interrupt status bits are sticky and are cleared when the DMA status register is read. When a MAC status interrupt occurs, programs can find this state by reading the MAC status register (`FIR_MACSTAT`). The read of the register clears the (sticky) MAC interrupt status bits.

The status interrupt sources are derived from the `FIR_MACSTAT` register. A status interrupt can occur due to the last set of MAC operations of a processing iteration that correspond to a particular channel. The interrupt is generated continuously and cannot be stopped, even after disabling the accelerator. The interrupt can only be stopped when another processing iteration results in a non-zero or valid multiply or add result.

ADSP-2159x FIR Register Descriptions

The FIR accelerator (FIR) contains the following registers.

Table 39-8: ADSP-2159x FIR Register List

Name	Description
FIR_CHNPTR	FIR Chain Pointer Register
FIR_COEFCNT	FIR Coefficient Count Register
FIR_COEFIDX	FIR Coefficient Index Register
FIR_COEFMOD	FIR Coefficient Modifier Register
FIR_CTL1	FIR Global Control Register
FIR_CTL2	FIR Channel Control Register
FIR_DBG_ADDR	Debug Address Register
FIR_DBG_CTL	FIR Debug Control Register
FIR_DBG_RDDAT	FIR Debug Data Read Register
FIR_DBG_WRDAT	FIR Debug Data Write Register
FIR_DMASTAT	FIR DMA Status Register
FIR_INBASE	FIR Input Data Base Register
FIR_INCNT	FIR Input Data Count Register
FIR_INIDX	FIR Input Data Index Register
FIR_INMOD	FIR Input Data Modifier Register
FIR_MACSTAT	FIR MAC Status Register
FIR_OUTBASE	FIR Output Data Base Register
FIR_OUTCNT	FIR Output Data Count Register
FIR_OUTIDX	FIR Output Data Index Register
FIR_OUTMOD	FIR Output Data Modifier Register
FIR_SCTL1	software control register 1
FIR_SCTL2	software control register 2
FIR_SGCTL	Secondary global control register

FIR Chain Pointer Register

The `FIR_CHNPTR` register contains the chain pointer address.

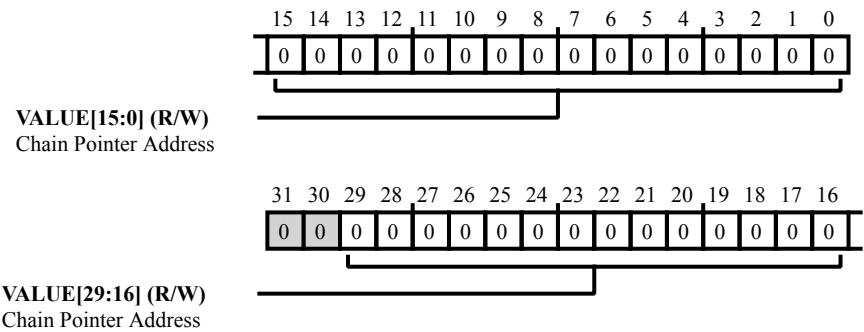


Figure 39-8: FIR_CHNPTR Register Diagram

Table 39-9: FIR_CHNPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Chain Pointer Address. The <code>FIR_CHNPTR.VALUE</code> bit field contains the chain pointer address.

FIR Coefficient Count Register

The `FIR_COEFCNT` register contains the 16-bit coefficient buffer count.

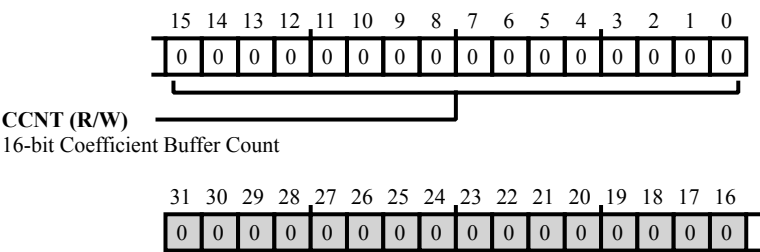


Figure 39-9: FIR_COEFCNT Register Diagram

Table 39-10: FIR_COEFCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	CCNT	16-bit Coefficient Buffer Count. The <code>FIR_COEFCNT.CCNT</code> bit field contains the 16-bit coefficient buffer count.

FIR Coefficient Index Register

The `FIR_COEFIDX` register contains the coefficient index word address with the lower two bits removed.

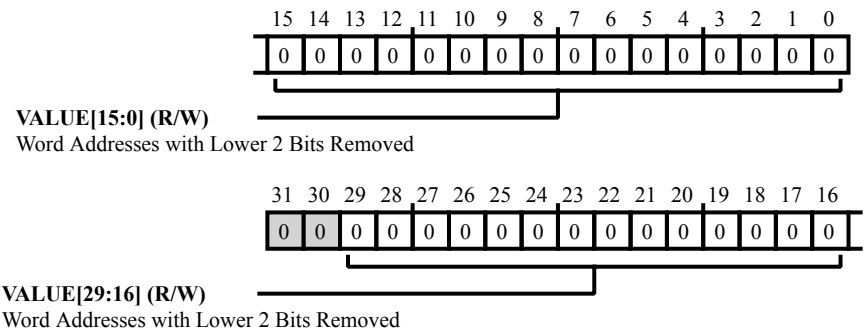


Figure 39-10: FIR_COEFIDX Register Diagram

Table 39-11: FIR_COEFIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Addresses with Lower 2 Bits Removed. The <code>FIR_COEFIDX.VALUE</code> bit field contains the word addresses with the lower 2 bits removed.

FIR Coefficient Modifier Register

The `FIR_COEFMOD` register contains the 16-bit coefficient index modifier.

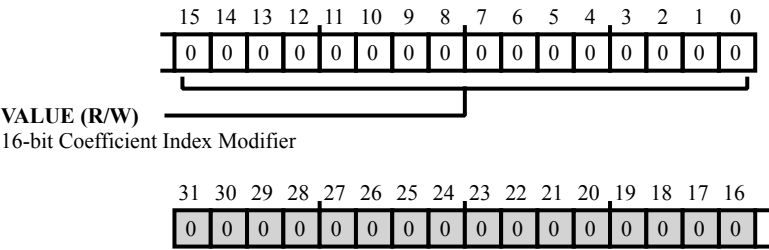


Figure 39-11: FIR_COEFMOD Register Diagram

Table 39-12: FIR_COEFMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Coefficient Index Modifier. The <code>FIR_COEFMOD.VALUE</code> bit field contains the 16-bit coefficient index modifier.

FIR Global Control Register

The `FIR_CTL1` register is used to configure the global parameters for the accelerator. These parameters include the number of channels, channel auto iterate, DMA enable, and accelerator enable.

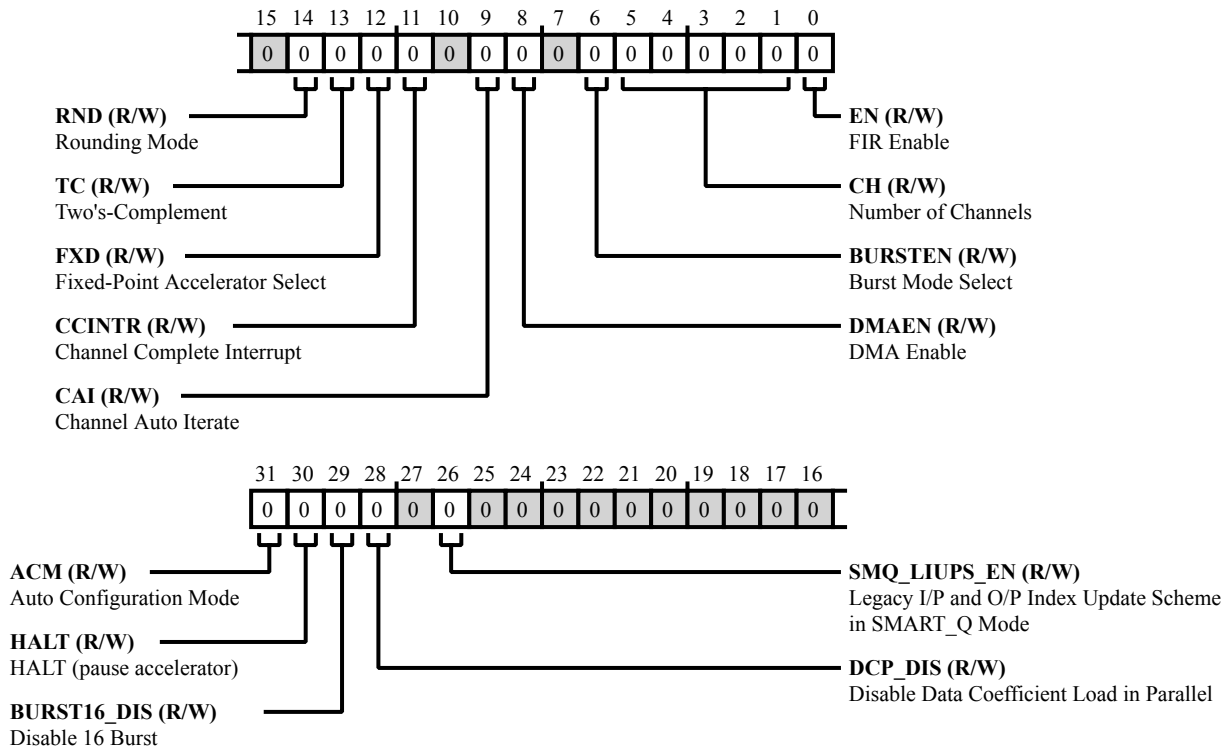


Figure 39-12: FIR_CTL1 Register Diagram

Table 39-13: FIR_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ACM	Auto Configuration Mode. The <code>FIR_CTL1.ACM</code> bit configures the mode for loading the TCB.
		0 Legacy Mode. Load TCB without the <code>FIR_CTL1</code> register. Core intervention may be required.
		1 Auto Configuration Mode. Load TCB with the <code>FIR_CTL1</code> register. Core intervention is not required.

Table 39-13: FIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W)	HALT	HALT (pause accelerator). This bit is only valid in Auto Configuration mode (ACM)
		0 Release. Release accelerator for further processing of data.
		1 Pause. Pause accelerator for core to check status and modify or submit jobs.
29 (R/W)	BURST16_DIS	Disable 16 Burst. The <code>FIR_CTL1.BURST16_DIS</code> bit configures the accelerator to disable 16 burst when in burst mode.
		0 Maximum burst of INCR16 in burst mode for DMA and TCB
		1 Maximum burst of INCR8 in burst mode for DMA and non-burst for TCB
28 (R/W)	DCP_DIS	Disable Data Coefficient Load in Parallel. The <code>FIR_CTL1.DCP_DIS</code> bit enables data and coefficient loading to occur in parallel.
		0 Enable
		1 Disable
26 (R/W)	SMQ_LIUPS_EN	Legacy I/P and O/P Index Update Scheme in SMART_Q Mode. The <code>FIR_CTL1.SMQ_LIUPS_EN</code> bit configures the scheme the accelerator uses to update the input index (II) and output index (OI). This bit is only valid in ACM.
		0 Update the II with all '0' and the OI with all 'F', respectively
		1 Update II and OI after circular buffer scheme
14 (R/W)	RND	Rounding Mode. The <code>FIR_CTL1.RND</code> bit configures the accelerator to use a rounding mode.
		0 Round to Nearest
		1 Truncate (round towards zero)
13 (R/W)	TC	Two's-Complement. The <code>FIR_CTL1.TC</code> bit configures the accelerator to use either unsigned integer or signed integer
		0 Unsigned integer
		1 Signed integer

Table 39-13: FIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	FXD	Fixed-Point Accelerator Select. The <code>FIR_CTL1.FXD</code> bit configures the accelerator to use either 32-bit IEEE floating-point or 32-bit fixed-point.
		0 32-bit IEEE floating-point
		1 32-bit fixed point
11 (R/W)	CCINTR	Channel Complete Interrupt. The <code>FIR_CTL1.CCINTR</code> bit configures the accelerator to generate an interrupt when each or all channels are done. This bit is only valid in Legacy Mode. In Auto Configuration Mode (ACM), interrupt generation for each channel is controlled using the <code>FIR_CTL2.IMASK</code> bit.
		0 Interrupt is generated only when all channels are done
		1 Interrupt is generated after each channel is done
9 (R/W)	CAI	Channel Auto Iterate. The <code>FIR_CTL1.CAI</code> bit, if cleared, causes the TDM processing to stop (idle) once all channels are done. If set, processing moves to the first channel and continues TDM processing in a loop when all channels are done. Channel Auto Iterate is not available in Auto Configuration Mode (ACM).
		0 TDM processing stops (idle) once all channels are done
		1 Moves to first channel and continues TDM processing in a loop when all channels are done
8 (R/W)	DMAEN	DMA Enable. The <code>FIR_CTL1.DMAEN</code> bit enables and disables DMA on the FIR accelerator.
		0 DMA disabled
		1 DMA enabled
6 (R/W)	BURSTEN	Burst Mode Select. When the <code>FIR_CTL1.BURSTEN</code> bit is set, burst mode is enabled for coefficient loads and data loads. When cleared, burst mode is disabled. By default, burst mode is disabled.

Table 39-13: FIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
5:1 (R/W)	CH	Number of Channels. The <code>FIR_CTL1.CH</code> bit field configures the number of channels and is programmable from 0 to 31. This bit field is only valid in Legacy Mode. There is no channel number limitation in Auto Configuration Mode (ACM) as the accelerator keeps processing the TCBs until the chain pointer becomes null.
		0 Channel 1
		1-30 Channel 2-31
		31 Channel 32
0 (R/W)	EN	FIR Enable. The <code>FIR_CTL1.EN</code> bit enables and disables the FIR accelerator.
		0 Disable FIR
		1 Enable FIR

FIR Channel Control Register

The `FIR_CTL2` register is used to configure the channel specific parameters such as filter TAP length, window size, sample rate conversion, up/down sampling and ratio. In Auto Configuration Mode(ACM), this register is also used to configure additional channel specific parameters like interrupts and triggers.

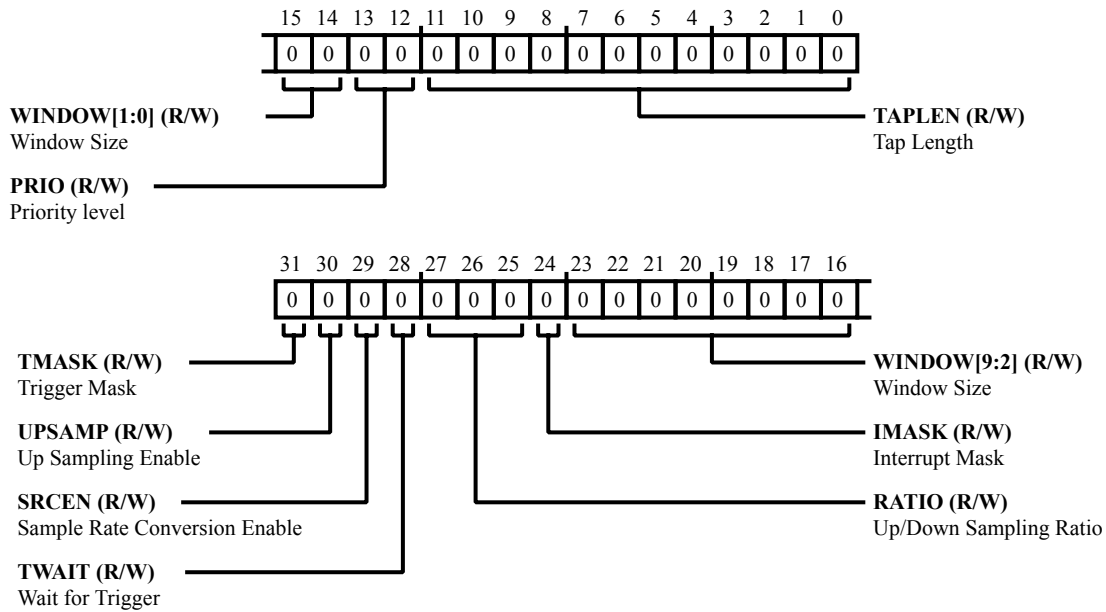


Figure 39-13: FIR_CTL2 Register Diagram

Table 39-14: FIR_CTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	TMASK	Trigger Mask. The <code>FIR_CTL2</code> . <code>TMASK</code> bit enables trigger generation for the channel. It is only valid in Auto Configuration Mode (ACM).
		0 Enable
		1 Mask
30 (R/W)	UPSAMP	Up Sampling Enable. The <code>FIR_CTL2</code> . <code>UPSAMP</code> bit enables up sampling.
		0 Down sampling
		1 Up sampling

Table 39-14: FIR_CTL2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	SRCEN	Sample Rate Conversion Enable. The <code>FIR_CTL2 . SRCEN</code> bit field enables sample rate conversion.
		0 Disabled
		1 Enabled
28 (R/W)	TWAIT	Wait for Trigger. The <code>FIR_CTL2 . TWAIT</code> bit disables the wait for the trigger. It is only valid in Auto Configuration Mode (ACM).
		0 Disable wait for trigger for the channel
		1 Enable wait for the external trigger assertion
27:25 (R/W)	RATIO	Up/Down Sampling Ratio. The <code>FIR_CTL2 . RATIO</code> bit field sets the sampling ratio (<code>FIR_CTL2 . RATIO + 1</code>).
24 (R/W)	IMASK	Interrupt Mask. The <code>FIR_CTL2 . IMASK</code> bit enables interrupt generation for the channel. This bit is only valid in Auto Configuration Mode (ACM).
		0 Enable
		1 Mask
23:14 (R/W)	WINDOW	Window Size. The <code>FIR_CTL2 . WINDOW</code> bit field sets the window size which specifies the number of sample/block to process (sample based processing = window size of 0).
13:12 (R/W)	PRIO	Priority level. The <code>FIR_CTL2 . PRIO</code> bit field indicates the priority.
		0 Level 0 (lowest)
		1 Level 1
		2 Level 2
		3 Level 3 (highest)
11:0 (R/W)	TAPLEN	Tap Length. The <code>FIR_CTL2 . TAPLEN</code> bit field sets the tap length which is programmable between 0-4095 (Tap Length = <code>FIR_CTL2 . TAPLEN + 1</code>).

Debug Address Register

The `FIR_DBG_ADDR` register holds the debug address.

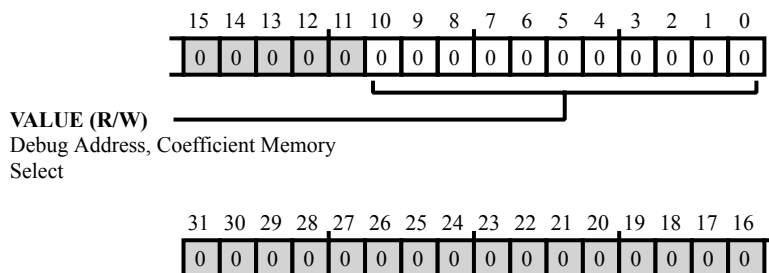


Figure 39-14: FIR_DBG_ADDR Register Diagram

Table 39-15: FIR_DBG_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Debug Address, Coefficient Memory Select. The <code>FIR_DBG_ADDR.VALUE</code> bit field holds the debug address (bits 0-10). Bit 11 configures whether the memory access is to coefficient memory (=0) or to delay line memory (=1).

FIR Debug Control Register

The `FIR_DBG_CTL` register controls debugging operations such as enabling debug mode running, hold or single stepping.

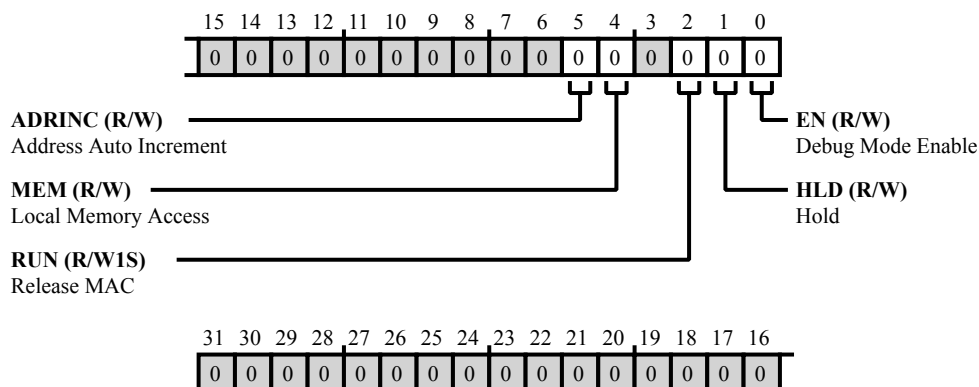


Figure 39-15: FIR_DBG_CTL Register Diagram

Table 39-16: FIR_DBG_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W)	ADRINC	Address Auto Increment. The <code>FIR_DBG_CTL.ADRINC</code> bit allows the address register to auto-increment on <code>FIR_DBG_WRDAT</code> writes and <code>FIR_DBG_RDDAT</code> reads.
4 (R/W)	MEM	Local Memory Access. When the <code>FIR_DBG_CTL.MEM</code> bit is set, the data and coefficients memory can be indirectly accessed.
2 (R/W1S)	RUN	Release MAC. The <code>FIR_DBG_CTL.RUN</code> bit releases the MAC. This bit is self-clearing after one FIR clock cycle.
1 (R/W)	HLD	Hold. The <code>FIR_DBG_CTL.HLD</code> bit holds or single-steps through the FIR.
		0 Hold
		1 Single-step
0 (R/W)	EN	Debug Mode Enable. The <code>FIR_DBG_CTL.EN</code> bit enables debug mode. For local memory access, the <code>FIR_CTL1</code> register can be cleared.
		0 Disable debug mode
		1 Enable debug mode

FIR Debug Data Read Register

The `FIR_DBG_RDDAT` register hold the debug read data.

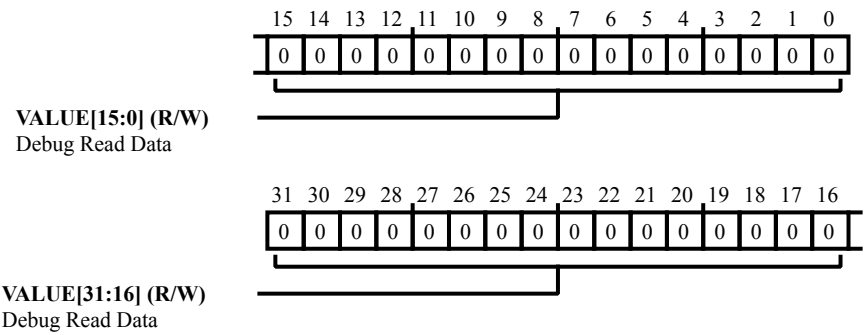


Figure 39-16: `FIR_DBG_RDDAT` Register Diagram

Table 39-17: `FIR_DBG_RDDAT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Debug Read Data. The <code>FIR_DBG_RDDAT.VALUE</code> bit field holds the debug read data.

FIR Debug Data Write Register

The `FIR_DBG_WRDAT` register holds the debug write data.

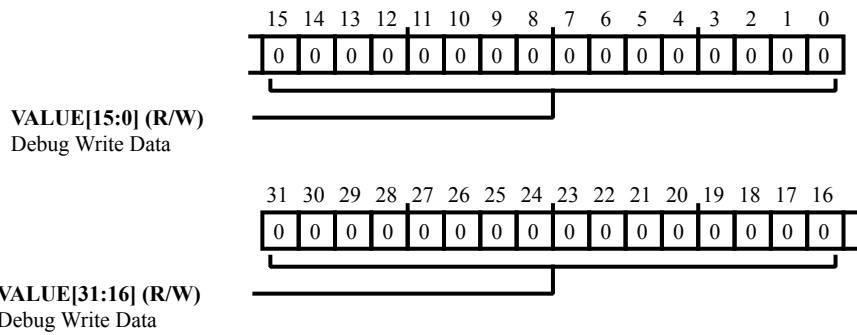


Figure 39-17: FIR_DBG_WRDAT Register Diagram

Table 39-18: FIR_DBG_WRDAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Debug Write Data. The <code>FIR_DBG_WRDAT.VALUE</code> bit field holds the debug write data.

FIR DMA Status Register

The `FIR_DMASTAT` register provides information about chain pointer loading, coefficient DMA, data preload DMA, processing in progress, window complete, all channels complete.

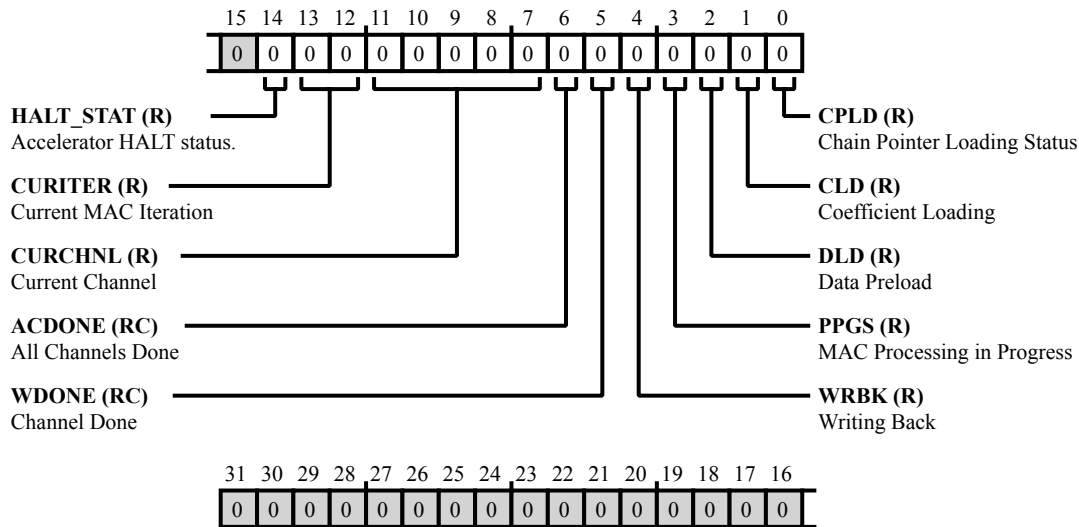


Figure 39-18: FIR_DMASTAT Register Diagram

Table 39-19: FIR_DMASTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/NW)	HALT_STAT	Accelerator HALT status.. Accelerator HALT status. 1 - Acknowledge to core that acceleration is in halt state. 0 - Not in halt state
13:12 (R/NW)	CURITER	Current MAC Iteration. The <code>FIR_DMASTAT.CURITER</code> bit indicates the current MAC iteration in multi-iteration mode. Zero indicates the final iteration.
11:7 (R/NW)	CURCHNL	Current Channel. The <code>FIR_DMASTAT.CURCHNL</code> bit indicates the channel that is being processed in the TDM slot. Zero indicates the last slot.
6 (RC/NW)	ACDONE	All Channels Done. The <code>FIR_DMASTAT.ACDONE</code> bit indicates the accelerator that processing all channels is complete. This is a sticky bit and is cleared on a register read. The <code>FIR_CTL1.CCINTR</code> bit does not affect the <code>FIR_DMASTAT.ACDONE</code> bit.
5 (RC/NW)	WDONE	Channel Done. The <code>FIR_DMASTAT.WDONE</code> bit indicates the accelerator that processing the current channel is complete. This is a sticky bit and is cleared on a register read. The <code>FIR_CTL1.CCINTR</code> bit does not affect the <code>FIR_DMASTAT.WDONE</code> bit.

Table 39-19: FIR_DMASTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4 (R/NW)	WRBK	Writing Back. The FIR_DMASTAT.WRBK bit indicates the accelerator is writing back the updated index registers.
3 (R/NW)	PPGS	MAC Processing in Progress. The FIR_DMASTAT.PPGS bit indicates MAC processing in progress.
2 (R/NW)	DLD	Data Preload. The FIR_DMASTAT.DLD bit indicates data preloading.
1 (R/NW)	CLD	Coefficient Loading. The FIR_DMASTAT.CLD bit indicates coefficient loading.
0 (R/NW)	CPLD	Chain Pointer Loading Status. The FIR_DMASTAT.CPLD bit indicates the state machine is in chain pointer load state.

FIR Input Data Base Register

The `FIR_INBASE` register contains the input word base address with the lower two bits removed.

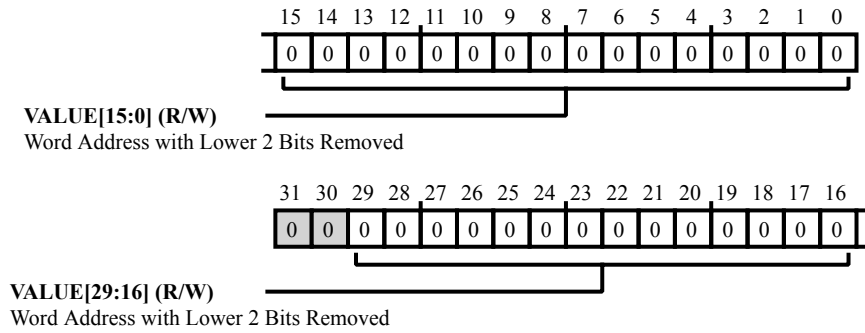


Figure 39-19: FIR_INBASE Register Diagram

Table 39-20: FIR_INBASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Address with Lower 2 Bits Removed. The <code>FIR_INBASE.VALUE</code> bit field contains the the word address with the lower 2 bits removed.

FIR Input Data Count Register

The `FIR_INCNT` register contains the 16-bit input data count.

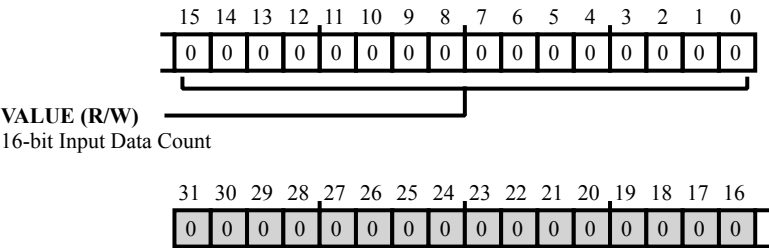


Figure 39-20: FIR_INCNT Register Diagram

Table 39-21: FIR_INCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Input Data Count. The <code>FIR_INCNT.VALUE</code> bit field contains the 16-bit input data count.

FIR Input Data Index Register

The `FIR_INIDX` register contains the input word address with the lower two bits removed.

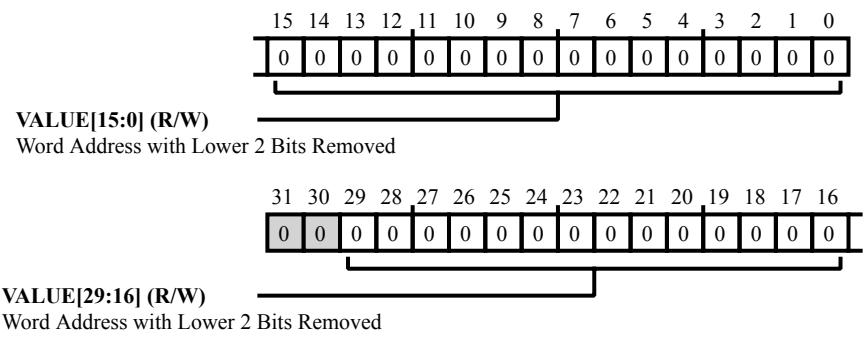


Figure 39-21: FIR_INIDX Register Diagram

Table 39-22: FIR_INIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Address with Lower 2 Bits Removed. The <code>FIR_INIDX.VALUE</code> bit field contains the input word address with the lower two bits removed.

FIR Input Data Modifier Register

The `FIR_INMOD` register contains the 16-bit input data buffer modifier.

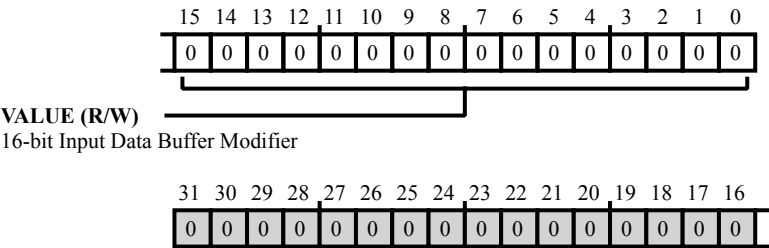


Figure 39-22: FIR_INMOD Register Diagram

Table 39-23: FIR_INMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Input Data Buffer Modifier. The <code>FIR_INMOD.VALUE</code> bit field contains the 16-bit input data buffer modifier.

FIR MAC Status Register

The `FIR_MACSTAT` register provides the status of MAC operations. The status of all four multipliers/adders are available separately for programs to poll. In fixed-point mode only, the `ARi` bits are used (all other bits are reserved).

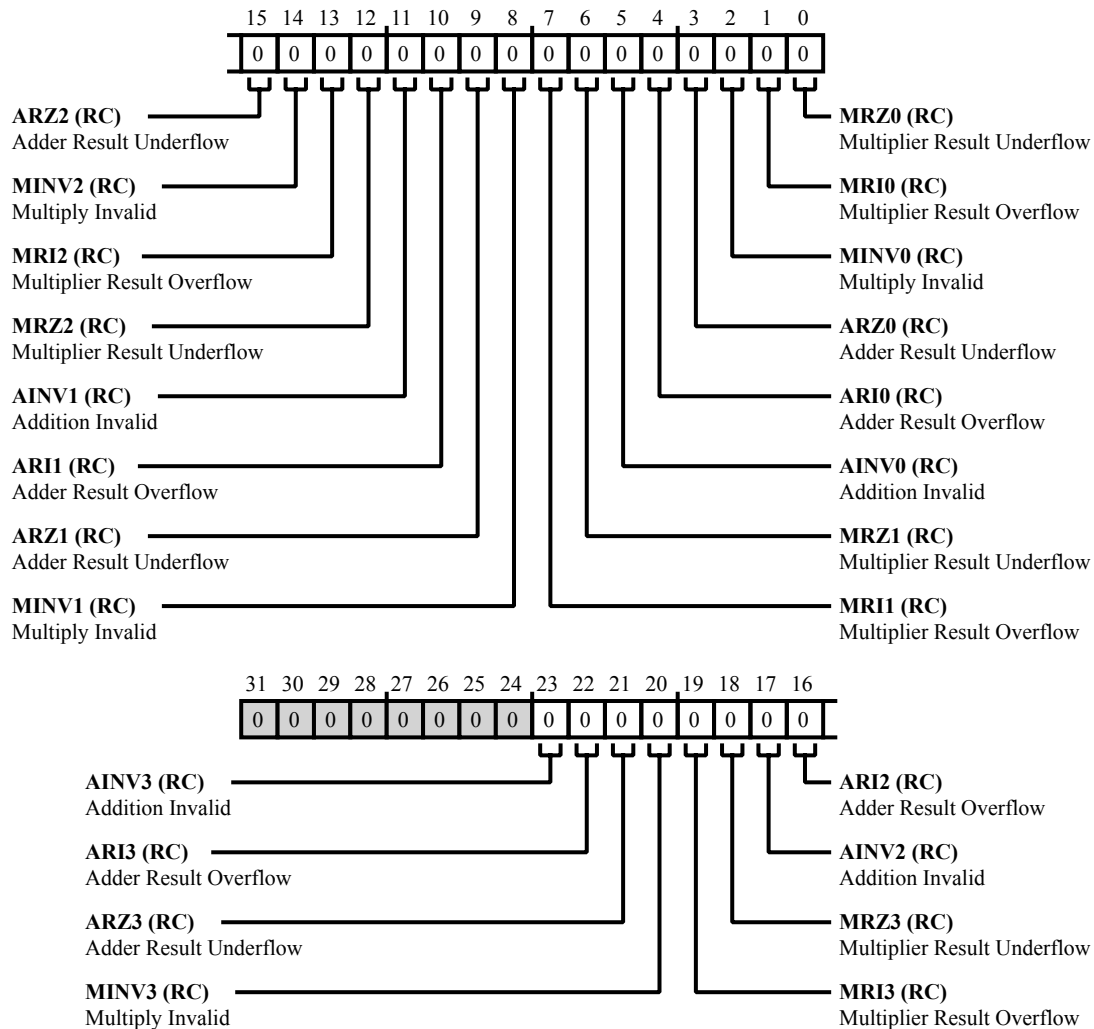


Figure 39-23: FIR_MACSTAT Register Diagram

Table 39-24: FIR_MACSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
23 (RC/NW)	AINV3	Addition Invalid. The <code>FIR_MACSTAT.AINV3</code> bit is set if the adder 3 addition is invalid.

Table 39-24: FIR_MACSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
22 (RC/NW)	ARI3	Adder Result Overflow. The <code>FIR_MACSTAT.ARI3</code> bit is set if the adder 3 result is infinity. Indicates overflow in fixed-point mode.
21 (RC/NW)	ARZ3	Adder Result Underflow. The <code>FIR_MACSTAT.ARZ3</code> bit is set if the adder 3 result is zero.
20 (RC/NW)	MINV3	Multiply Invalid. The <code>FIR_MACSTAT.MINV3</code> bit is set if the multiplier 3 multiply operation is invalid.
19 (RC/NW)	MRI3	Multiplier Result Overflow. The <code>FIR_MACSTAT.MRI3</code> bit is set if the multiplier 3 result is infinity.
18 (RC/NW)	MRZ3	Multiplier Result Underflow. The <code>FIR_MACSTAT.MRZ3</code> bit is set if the multiplier 3 result is zero.
17 (RC/NW)	AINV2	Addition Invalid. The <code>FIR_MACSTAT.AINV2</code> bit is set if the adder 2 addition is invalid.
16 (RC/NW)	ARI2	Adder Result Overflow. The <code>FIR_MACSTAT.ARI2</code> bit is set if the adder 2 result is infinity. Indicates overflow in fixed-point mode.
15 (RC/NW)	ARZ2	Adder Result Underflow. The <code>FIR_MACSTAT.ARZ2</code> bit is set if the adder 2 result is zero.
14 (RC/NW)	MINV2	Multiply Invalid. The <code>FIR_MACSTAT.MINV2</code> bit is set if the multiplier 2 multiply operation is invalid.
13 (RC/NW)	MRI2	Multiplier Result Overflow. The <code>FIR_MACSTAT.MRI2</code> bit is set if the multiplier 2 result is infinity.
12 (RC/NW)	MRZ2	Multiplier Result Underflow. The <code>FIR_MACSTAT.MRZ2</code> bit is set if the multiplier 2 result is zero.
11 (RC/NW)	AINV1	Addition Invalid. The <code>FIR_MACSTAT.AINV1</code> bit is set if the adder 1 addition is invalid.
10 (RC/NW)	ARI1	Adder Result Overflow. The <code>FIR_MACSTAT.ARI1</code> bit is set if the adder 1 result is infinity. Indicates overflow in fixed-point mode.
9 (RC/NW)	ARZ1	Adder Result Underflow. The <code>FIR_MACSTAT.ARZ1</code> bit is set if the adder 1 result is zero.

Table 39-24: FIR_MACSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (RC/NW)	MINV1	Multiply Invalid. The <code>FIR_MACSTAT.MINV1</code> bit is set if the multiplier 1 multiply operation is invalid.
7 (RC/NW)	MRI1	Multiplier Result Overflow. The <code>FIR_MACSTAT.MRI1</code> bit is set if the multiplier 1 result is infinity.
6 (RC/NW)	MRZ1	Multiplier Result Underflow. The <code>FIR_MACSTAT.MRZ1</code> bit is set if the multiplier 1 result is zero.
5 (RC/NW)	AINV0	Addition Invalid. The <code>FIR_MACSTAT.AINV0</code> bit is set if the adder 0 addition is invalid.
4 (RC/NW)	ARI0	Adder Result Overflow. The <code>FIR_MACSTAT.ARI0</code> bit is set if the adder 0 result is infinity. Indicates overflow in fixed-point mode.
3 (RC/NW)	ARZ0	Adder Result Underflow. The <code>FIR_MACSTAT.ARZ0</code> bit is set if the adder 0 result is zero.
2 (RC/NW)	MINV0	Multiply Invalid. The <code>FIR_MACSTAT.MINV0</code> bit is set if the multiplier 0 multiply operation is invalid.
1 (RC/NW)	MRI0	Multiplier Result Overflow. The <code>FIR_MACSTAT.MRI0</code> bit is set if the multiplier 0 result is infinity.
0 (RC/NW)	MRZ0	Multiplier Result Underflow. The <code>FIR_MACSTAT.MRZ0</code> bit is set if multiplier 0 result is zero.

FIR Output Data Base Register

The `FIR_OUTBASE` register contains the output word base address with the lower two bits removed

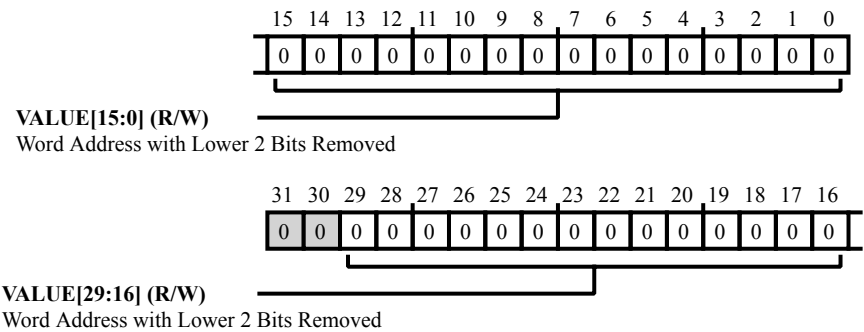


Figure 39-24: FIR_OUTBASE Register Diagram

Table 39-25: FIR_OUTBASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Address with Lower 2 Bits Removed. The <code>FIR_OUTBASE.VALUE</code> bit field contains the word address with the lower 2 bits removed.

FIR Output Data Count Register

The `FIR_OUTCNT` register contains the 16-bit output buffer count.

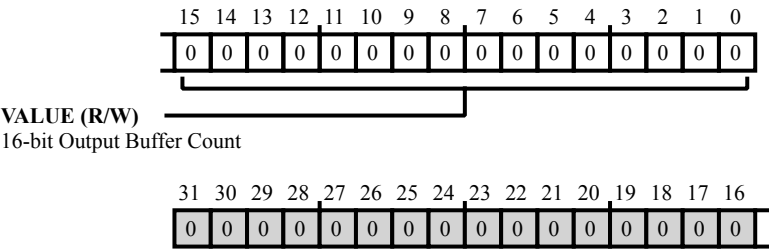


Figure 39-25: FIR_OUTCNT Register Diagram

Table 39-26: FIR_OUTCNT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Output Buffer Count. The <code>FIR_OUTCNT.VALUE</code> bit field contains the 16-bit output buffer count.

FIR Output Data Index Register

The `FIR_OUTIDX` register contains the output word address with the lower two bits removed.

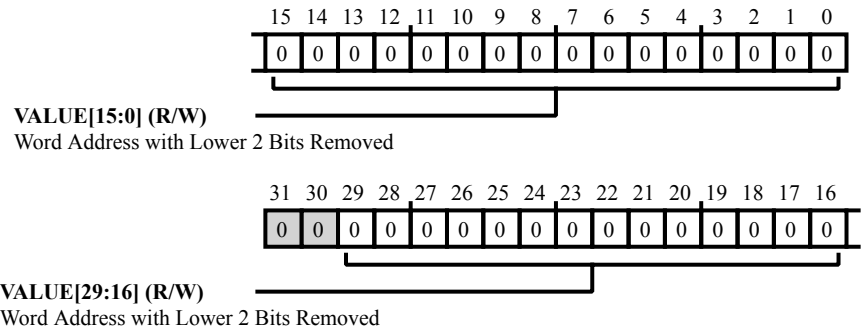


Figure 39-26: FIR_OUTIDX Register Diagram

Table 39-27: FIR_OUTIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Word Address with Lower 2 Bits Removed. The <code>FIR_OUTIDX.VALUE</code> bit field contains the the word address with the lower 2 bits removed.

FIR Output Data Modifier Register

The `FIR_OUTMOD` register contains the 16-bit output data buffer modifier.

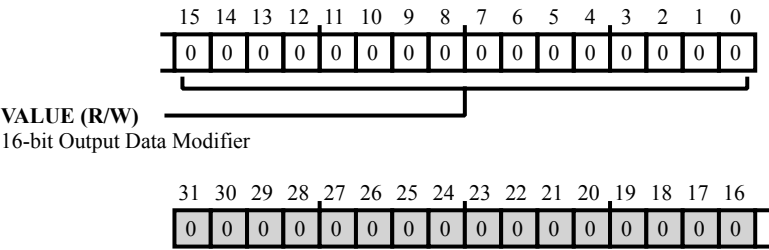


Figure 39-27: FIR_OUTMOD Register Diagram

Table 39-28: FIR_OUTMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Output Data Modifier. The <code>FIR_OUTMOD.VALUE</code> bit field contains the 16-bit output data modifier.

software control register 1

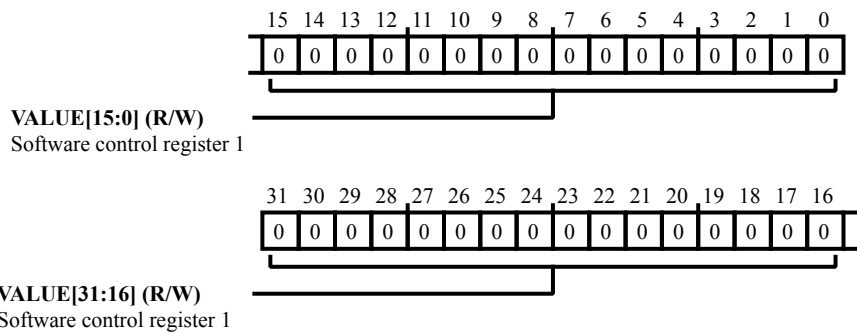


Figure 39-28: FIR_SCTL1 Register Diagram

Table 39-29: FIR_SCTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Software control register 1.

software control register 2

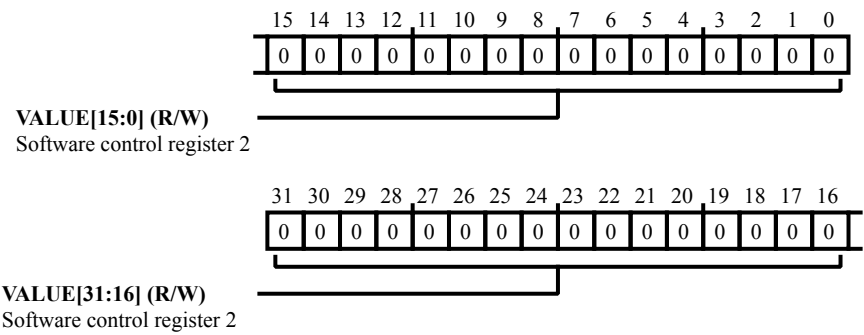


Figure 39-29: FIR_SCTL2 Register Diagram

Table 39-30: FIR_SCTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Software control register 2.

Secondary global control register

The FIRSGCTL register is used to configure the global parameters for the accelerator in ACM mode for loading CTL1 register bits as part of TCB.

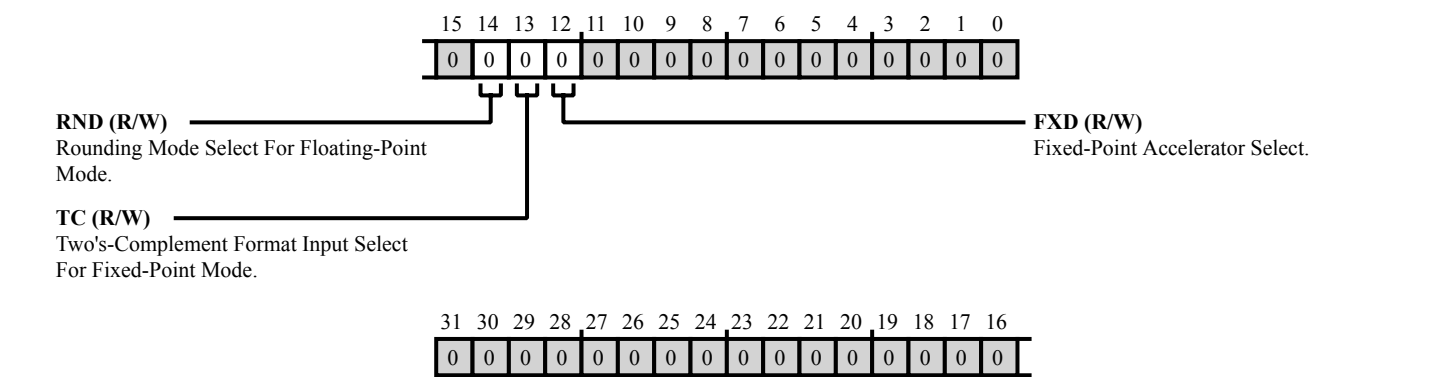


Figure 39-30: FIR_SGCTL Register Diagram

Table 39-31: FIR_SGCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
14 (R/W)	RND	Rounding Mode Select For Floating-Point Mode.. 0- Round to nearest 1 -Truncate (Round towards zero)
13 (R/W)	TC	Two's-Complement Format Input Select For Fixed-Point Mode.. 0 - Unsigned integer 1 - Signed integer
12 (R/W)	FXD	Fixed-Point Accelerator Select.. 0 - 32-bit floating-point compatible to SHARCXI 1 - 32-bit fixed point

40 IIR Accelerator (IIR)

The processor includes an Infinite Impulse Response (IIR) filter accelerator implemented in hardware that reduces the processing load on the core, freeing it up for other tasks.

Features

The accelerator supports a maximum of 32 channels in legacy mode. In Auto Configuration Mode (ACM), there is no limit on number of channels. The accelerator locally stores all the biquad coefficients of 32 channels in legacy mode. Window size can be configured from 1 (sample based) to 1024.

The IIR has the following features:

- Supports IEEE floating point format 32/40-bit
- Rounding modes compatible with SHARC+ core MACs
- Sample-based or window-based processing
- Up to 64 cascaded biquads per channel
- Up to 32 filter channels available in TDM
- Allows biquad save state storage

NOTE: The IIR accelerator module has local memory which is not accessible by the core during regular operation mode. Unlike in the previous SHARC processors, the IIR accelerator modules each have access to the system memory (on-chip/off-chip).

Unlike in the previous SHARC processors, where only one of the IIR or FIR accelerator can be enabled at a time, the processor can use both the IIR and the FIR accelerators at the same time.

Clocking

The IIR accelerator runs at the speed of core clock.

Functional Description

The *IIR Accelerator Block Diagram* shows the various blocks of the IIR hardware accelerator.

The accelerator has:

- a coefficient memory size of 1440×40 bits (288 Biquads $\times$ 5 coefficients).
- a data memory size of 576×40 bits (288 Biquads $\times$ 2 states).
- one SHARC+ core compatible MAC unit with an input data buffer to supply data to the MAC.

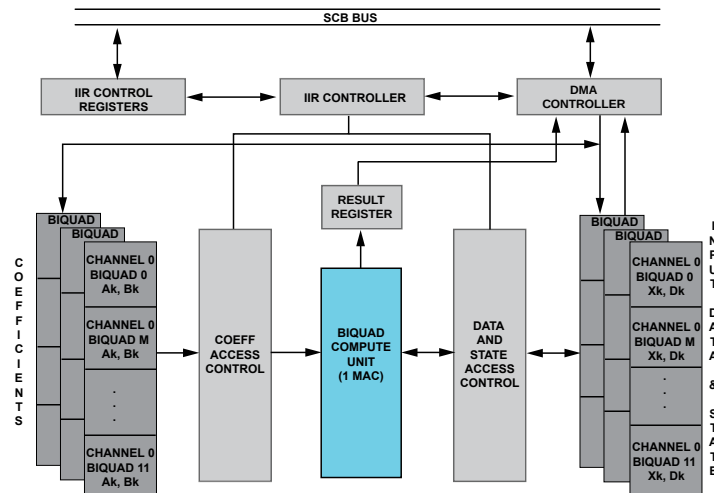


Figure 40-1: IIR Accelerator Block Diagram

The IIR accelerator is implemented using Transposed Direct Form II biquad which has less coefficient sensitivity. The *Transposed Direct Form II Biquad* figure shows the signal flow graph for the biquad structure.

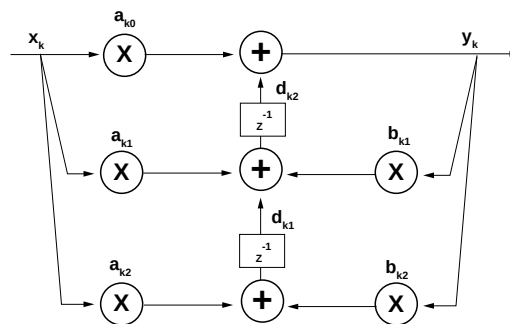


Figure 40-2: Transposed Direct Form II Biquad

The accelerator has the following logical subblocks.

- A datapath unit with the following elements:
 - 32/40-bit coefficient memory (A_k , B_k) for storing biquad coefficients
 - 32/40-bit input data (X_k) and state (D_k)

- One 40/32-bit floating-point multiplier and adder (MAC) unit
- An input data buffer to efficiently supply data to MAC
- One 40-bit result register to hold result of biquad
- Configuration registers for controlling various parameters such as the number of biquads, the number of channels, interrupt control, and DMA control
- A core access interface for writing the DMA/filter configuration registers and for reading the status registers
- A DMA bus interface for transferring data to and from the accelerator. This interface is also used to preload the coefficients (Ak, Bk) and state (Dk) at startup.
- DMA configuration registers for the transfer of input data, output data, and coefficients

The accelerator block is integrated with the SHARC core. For more information about this, see SHARC SCB with FIR/IIR Accelerator.

ADSP-2159x IIR Register List

The IIR module reduces the processing load on the core. For more information on IIR functionality, see the IIR register descriptions.

Table 40-1: ADSP-2159x IIR Register List

Name	Description
IIR_CHNPTR	Chain Pointer Register
IIR_COEFIDX	Coefficient Buffer Index Register
IIR_COEFLEN	Coefficient Buffer Length Register
IIR_COEFMOD	Coefficient Index Modifier Register
IIR_CTL1	Global Control Register
IIR_CTL2	Channel Control Register
IIR_DBG_ADDR	IIR Debug Address Register
IIR_DBG_CTL	IIR Debug Control Register
IIR_DBG_RDDAT_HI	IIR Debug Read Data High Register
IIR_DBG_RDDAT_LO	IIR Debug Read Data Low Register
IIR_DBG_WRDAT_HI	IIR Debug Write Data High Register
IIR_DBG_WRDAT_LO	IIR Debug Write Data Low Register
IIR_DMASTAT	DMA Status Register
IIR_INBASE	Input Buffer Base Register
IIR_INIDX	Input Data Index Register
IIR_INLEN	Input Data Buffer Length Register

Table 40-1: ADSP-2159x IIR Register List (Continued)

Name	Description
IIR_INMOD	Input Data Index Modifier Register
IIR_MACSTAT	MAC Status Register
IIR_OUTBASE	Output Buffer Base Register
IIR_OUTIDX	Output Data Buffer Index Register
IIR_OUTLEN	IIR Output Data Buffer Length Register
IIR_OUTMOD	IIR Output Data Index Modifier Register
IIR_SCTL1	Software Control Register1
IIR_SCTL2	Software Control Register2
IIR_SGCTL	Secondary Global Control Register

ADSP-2159x IIR Interrupt List

Table 40-2: ADSP-2159x IIR Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
168	IIR0_DMA	IIR0 Core 1 DMA	Edge	
169	IIR0_STAT	IIR0 Core 1 Status	Edge	
170	IIR1_DMA	IIR1 Core 1 DMA	Edge	
171	IIR1_STAT	IIR1 Core 1 Status	Edge	
172	IIR2_DMA	IIR2 Core 1 DMA	Edge	
173	IIR2_STAT	IIR2 Core 1 Status	Edge	
174	IIR3_DMA	IIR3 Core 1 DMA	Edge	
175	IIR3_STAT	IIR3 Core 1 Status	Edge	
178	IIR4_DMA	IIR4 Core 2 DMA	Edge	
179	IIR4_STAT	IIR4 Core 2 Status	Edge	
180	IIR5_DMA	IIR5 Core 2 DMA	Edge	
181	IIR5_STAT	IIR5 Core 2 Status	Edge	
182	IIR6_DMA	IIR6 Core 2 DMA	Edge	
183	IIR6_STAT	IIR6 Core 2 Status	Edge	
184	IIR7_DMA	IIR7 Core 2 DMA	Edge	
185	IIR7_STAT	IIR7 Core 2 Status	Edge	
252	IIR0_BUS_ERR	IIR0 Core 1 IIR Bus Error	Edge	

Table 40-2: ADSP-2159x IIR Interrupt List (Continued)

Interrupt ID	Name	Description	Sensitivity	DMA Channel
253	IIR1_BUS_ERR	IIR1 Core 1 IIR Bus Error	Edge	
254	IIR2_BUS_ERR	IIR2 Core 1 IIR Bus Error	Edge	
255	IIR3_BUS_ERR	IIR3 Core 1 IIR Bus Error	Edge	
257	IIR4_BUS_ERR	IIR4 Core 2 IIR Bus Error	Edge	
258	IIR5_BUS_ERR	IIR5 Core 2 IIR Bus Error	Edge	
259	IIR6_BUS_ERR	IIR6 Core 2 IIR Bus Error	Edge	
260	IIR7_BUS_ERR	IIR7 Core 2 IIR Bus Error	Edge	

ADSP-2159x IIR Trigger List

Table 40-3: ADSP-2159x IIR Trigger List Masters

Trigger ID	Name	Description	Sensitivity
39	C1_IIR0_DMA	IIR0 Core 1 DMA	Edge
40	C1_IIR1_DMA	IIR1 Core 1 DMA	Edge
41	C1_IIR2_DMA	IIR2 Core 1 DMA	Edge
42	C1_IIR3_DMA	IIR3 Core 1 DMA	Edge
43	C2_IIR0_DMA	IIR4 Core 2 DMA	Edge
44	C2_IIR1_DMA	IIR5 Core 2 DMA	Edge
45	C2_IIR2_DMA	IIR6 Core 2 DMA	Edge
46	C2_IIR3_DMA	IIR7 Core 2 DMA	Edge

Table 40-4: ADSP-2159x IIR Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
23	C1_IIR0_TRGI	IIR0 Core 1 IIR Wait on Trigger Input	Pulse
24	C1_IIR1_TRGI	IIR1 Core 1 IIR Wait on Trigger Input	Pulse
25	C1_IIR2_TRGI	IIR2 Core 1 IIR Wait on Trigger Input	Pulse
26	C1_IIR3_TRGI	IIR3 Core 1 IIR Wait on Trigger Input	Pulse
27	C2_IIR0_TRGI	IIR4 Core 2 IIR Wait on Trigger Input	Pulse
28	C2_IIR1_TRGI	IIR5 Core 2 IIR Wait on Trigger Input	Pulse
29	C2_IIR2_TRGI	IIR6 Core 2 IIR Wait on Trigger Input	Pulse
30	C2_IIR3_TRGI	IIR7 Core 2 IIR Wait on Trigger Input	Pulse

Multiply and Accumulate (MAC) Unit

The *IIR MAC Unit* figure shows a pipelined multiplier and accumulator unit that operates on the data and coefficient fetched from the data and coefficient memory. The MAC can perform either 32-bit floating-point or 40-bit floating-point MAC operations. 32-bit floating-point operations generate 32-bit results and 40-bit floating-point operations generate 40-bit results.

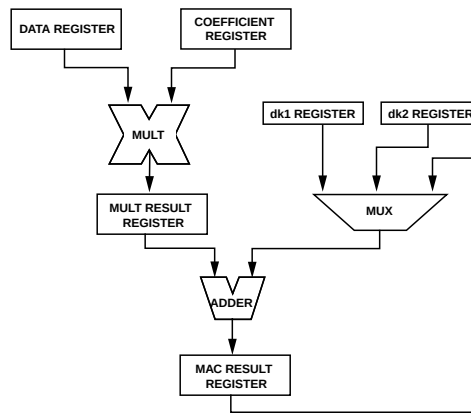


Figure 40-3: IIR MAC Unit

Input Data and Biquad State

The size of data memory is 576×40 bits and is used to hold the dk1 and dk2 state of all the biquads locally. The DMA controller fetches the sample data from internal memory and calculates the output as well as the dk1 and dk2 values for each biquad and stores them in local data memory.

Coefficient Memory

The size of coefficient memory is 1440×40 bits and is used to store all the coefficients of all the biquads. At start-up, DMA loads the coefficients from system memory into local coefficient memory.

Internal Memory Storage

This section describes the required storage model for the IIR accelerator.

Coefficient Memory Storage

Coefficients and Dk values for a particular biquad BQD[k] should be stored in internal memory in the order: Ak0, Ak1, Bk1, Ak2, Bk2, Dk2, Dk1.

NOTE: The naming convention for the filter coefficients used here is different from the one used in MATLAB. The following conversion should be used when using MATLAB generated coefficients:

$$(Ak_x = b_x \text{ and } Bk_x = -a_x).$$

Store the coefficients for each biquad in the order:

b0, b1, -a1, b2, -a2, dk2, dk1

For N biquad stages, store the coefficients in the order:

```
b01, b11, -a11, b21, -a21, dk21, dk11,
b02, b12, -a12, b22, -a22, dk22, dk12,
, .....
b0N, b1N, -a1N, b2N, -a2N, dk2N, dk1N.
```

where b_{xN} and a_{xN} are the coefficients ($[b, a]$) for the Nth biquad stage.

Operating Modes

The accelerator operates in [Window Processing Mode](#), [40-Bit Floating-Point Mode](#) and [Save Biquad State Mode](#).

Window Processing Mode

Sample-based processing mode is selected by configuring window size to 1. In this mode, one sample from a particular channel is processed through all the biquads of that channel and the final output sample is calculated.

In window-based mode, multiple output samples (up to 1024) equal to the window size of that channel are calculated. After these calculations are complete, the accelerator begins processing the next channel. A configurable window size parameter is provided to specify the length of the window.

40-Bit Floating-Point Mode

In 40-bit floating-point mode, the input data/coefficient is treated as a 40-bit floating-point number. 40-bit floating-point MAC operations generate 40-bit results. This mode can be selected by setting the `IIR_CTL1.FORTYBIT` bit.

In ACM, 40-bit floating-point mode can be selected by setting the `IIR_SGCTL.FORTYBIT` field for each channel.

As the DMA bus width is 32 bits, in 40-bit mode the IIR accelerator performs two packed 32-bit accesses to the memory to:

- fetch one 40-bit input or coefficient data, or
- to store one 40-bit output word

The first 32-bit word provides the lower 32 bits and the 8 LSBs of the second 32-bit word provides rest of the upper 8 bits of the complete 40-bit word. The *32-Bit to 40-Bit Packing* figure shows the 32–40 bit packing used by accelerator.

NOTE: Overheads could be required to pack the input 40-bit data into the format acceptable by the IIR accelerator and for unpacking the output of accelerator to the format acceptable by the rest of the application.

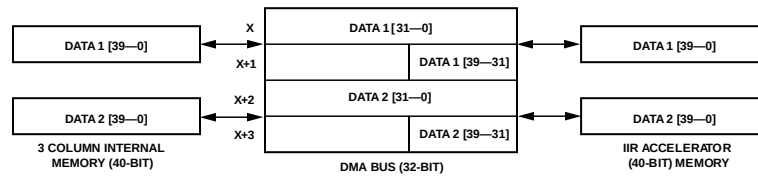


Figure 40-4: 32-Bit to 40-Bit Packing

Save Biquad State Mode

The `IIR_CTL1.SS` bit completely stores the current biquad states in local memory (writes all the DK1 and DK2 states back into the system memory states). This functionality is useful in applications that require fast switching to another high-priority accelerator task—a required IIR to FIR processing transition for example. After resuming, these states can be reloaded and IIR processing can be continued. DMA status is automatically stored after each iteration.

NOTE: In legacy mode, the accelerator loads the biquad coefficients for all the channels before processing the first channel. After processing the last channel, if `IIR_CTL1.SS` bit is set, the accelerator stores the biquad states of all the channels in local memory (writes all the DK1 and DK2 states back into the system memory states). In ACM, the accelerator loads the biquad coefficients for only the current channel before starting processing of the current channel. If the `IIR_SGCTL.SS` bit is set, the accelerator stores the biquad states of the current channel in local memory before moving on to the next channel.

NOTE: Write access to any of the IIR accelerator registers loaded by chaining is not allowed while the save state operation is in progress. Attempted writes to these registers could result in the blocking of IOP core reads until the save state operation completes.

Auto Configuration Mode (ACM)

The accelerator can be operated in the legacy mode or Auto Configuration Mode (ACM), controlled by the `IIR_CTL1.ACM` bit. The default functional mode is legacy mode. The accelerator mode can only be changed when the accelerator is disabled.

The ACM provides the following additional features:

- Halt and Queuing

Core may pause the current TCB chain being processed by setting the `IIR_CTL1.HALT` bit. The accelerator acknowledges the core by setting the `IIR_DMASTAT.HALT_STAT` bit. The core can appropriately take action to submit/insert new TCB's. After the core takes action, the accelerator processing can be resumed by clearing the `IIR_CTL1.HALT` bit. Before halting the accelerator, if the initial TCB chain is processed, the accelerator comes to idle state. In this case, the accelerator has to be disabled and enabled by toggling the `IIR_CTL1.EN` bit and the `IIR_CTL1.HALT` bit has to be cleared to resume processing.

- No Channel Number Limitation

Unlike in legacy mode, there is no fixed channel number limitation and, therefore, the accelerator ignores the value programmed in the `IIR_CTL1.CH` field. The application can queue unlimited number of channels/TCBs dynamically and the accelerator keeps processing the TCBs until the chain pointer becomes null.

- Selective Interrupt

Core can enable/mask interrupt generation for each channel using the `IIR_CTL2.IMASK` bit. If the bit is cleared, an interrupt is generated after completion of the channel.

- Selective Master/Slave Trigger

Core can enable/mask trigger generation by the accelerator after the end of processing of each channel using the `IIR_CTL2.TMASK` bit. The accelerator can also wait for a trigger after loading the TCB and coefficients and before processing a channel for which the `IIR_CTL2.TWAIT` bit is set.

This feature can also be enabled in legacy mode by setting `IIR_CTL1.L_TIMASK_EN` and `IIR_CTL1.L_TWAIT_EN` bits. In legacy mode, when the `IIR_CTL1.L_TIMASK_EN` and `IIR_CTL1.L_TWAIT_EN` bits are set, then `IIR_CTL2.TMASK` and `IIR_CTL2.TWAIT` bits can be used to enable/mask trigger generation and wait for trigger for each channel respectively.

In addition to the above features, there are three additional fields as part of the TCB `IIR_SCTL1`, `IIR_SCTL2`, and `IIR_SGCTL`. The `IIR_SGCTL` can be used to change the `IIR_CTL1` parameters such as rounding mode, forty bit mode, and save state for each channel. The `IIR_SCTL1` and `IIR_SCTL2` can be used as general-purpose registers.

Data Transfers

The IIR filter works exclusively through DMA.

IIR Accelerator TCB

The location of the DMA parameters for the next sequence comes from the chain pointer register. This register points to the next set of DMA parameters stored in the system memory of the processor known as TCB. In chained DMA operations, the processor automatically initializes and then begins another DMA transfer when the current DMA transfer is complete. Each new set of parameters is stored in a user-initialized memory buffer or TCB for a chosen peripheral.

Chain Assignment

The structure of a TCB is conceptually the same as the structure of a traditional linked list. Each TCB has several data values and a pointer to the next TCB. The chain pointer of a TCB can point to itself to re-run the same DMA continuously. The IIR accelerator reads each word of the TCB and loads it into the corresponding register. A TCB with a chain pointer register value of zero indicates the end of the chain (no further TCBs are loaded). The IIR accelerator supports circular buffer chained DMA. The following tables show the required TCBs for chained DMA in legacy mode and ACM. TCB refers to the start address of the TCB array.

NOTE: In the IIR accelerator DMA, two different TCB loading sequences are available: one TCB loads five parameters for the coefficients (`IIR_CTL2`, `IIR_COEFIDX`, `IIR_COEFMOD`, `IIR_COEFLEN`, and `IIR_CHNPTR`). The second loads 10 parameters for the data (`IIR_CTL2`, `IIR_INIDX`, `IIR_INMOD`, `IIR_INLEN`, `IIR_INBASE`, `IIR_OUTIDX`, `IIR_OUTMOD`, `IIR_OUTLEN`, `IIR_OUTBASE`, and `IIR_CHNPTR`).

Initialize `IIR_CHNPTR` to TCB+12 in legacy mode and TCB + 15 in ACM.

Table 40-5: IIR TCBs for Chained DMA in Legacy Mode

Address	Register
TCB	<code>IIR_CHNPTR</code>
TCB + 0x1	<code>IIR_COEFLEN</code>
TCB + 0x2	<code>IIR_COEFMOD</code>
TCB + 0x3	<code>IIR_COEFIDX</code>
TCB + 0x4	<code>IIR_OUTBASE</code>
TCB + 0x5	<code>IIR_OUTLEN</code>
TCB + 0x6	<code>IIR_OUTMOD</code>
TCB + 0x7	<code>IIR_OUTIDX</code>
TCB + 0x8	<code>IIR_INBASE</code>
TCB + 0x9	<code>IIR_INLEN</code>
TCB + 0xA	<code>IIR_INMOD</code>
TCB + 0xB	<code>IIR_INIDX</code>
TCB + 0xC	<code>IIR_CTL2</code>

Table 40-6: IIR TCBs for Chained DMA in ACM

Address	Register
TCB	<code>IIR_CHNPTR</code>
TCB + 0x1	<code>IIR_SCTL1</code>
TCB + 0x2	<code>IIR_SCTL2</code>
TCB + 0x3	<code>IIR_SGCTL</code>
TCB + 0x4	<code>IIR_COEFLEN</code>
TCB + 0x5	<code>IIR_COEFMOD</code>
TCB + 0x6	<code>IIR_COEFIDX</code>
TCB + 0x7	<code>IIR_OUTBASE</code>
TCB + 0x8	<code>IIR_OUTLEN</code>
TCB + 0x9	<code>IIR_OUTMOD</code>

Table 40-6: IIR TCBs for Chained DMA in ACM (Continued)

Address	Register
TCB + 0xA	IIR_OUTIDX
TCB + 0xB	IIR_INBASE
TCB + 0xC	IIR_INLEN
TCB + 0xD	IIR_INMOD
TCB + 0xE	IIR_INIDX
TCB + 0xF	IIR_CTL2

DMA Access

The IIR accelerator has two DMA channels (accelerator input and output) to connect to the system memory. The DMA controller fetches the data and coefficients from memory and stores the result.

Burst Mode Access

Burst mode enhances the throughput of the DMA channel and reduce the overall load on the system fabric. If `IIR_CTL1.BURSTEN` is set, all the biquads of all the channels are loaded with burst enabled. The IIR module supports only INCR8 burst.

Chain Pointer DMA

The DMA controller supports circular buffer chain pointer DMA. One transfer control block (TCB) must be configured for each channel. The TCB contains:

- A control register value to configure the filter parameters (such as number of biquads, window size) for each channel. In ACM, additional parameter such as interrupt mask, trigger mask, and trigger wait are also available.
- Software control register values in ACM for each channel.
- Secondary control register value in ACM to configure rounding mode, forty bit mode, and save state for each channel.
- DMA parameter register values for the input data
- DMA parameter register values for coefficient load
- DMA parameter register values for output data

In legacy mode, The chain pointer (`IIR_CHNPTR`) field of the last channel's TCB should point to the first channel's TCB. This configuration exists so that when the IIR accelerator is enabled, the module:

1. Loads the coefficients (A_k , B_k) and state variables (D_k) for all the channels into the local coefficient memory of the IIR, and
2. Loops back to the first channel again to start fetching the input data for processing.

In ACM, the chain pointer field of the last channel's TCB need not be pointing to the first channel's TCB.

When the accelerator is enabled, the module:

1. Loads the coefficients (A_k, B_k) and state variables (D_k) only for the current channel being processed.
2. Starts fetching input data for processing.
3. After processing the current channel, if the `IIR_SGCTL.SS` bit is set, it saves the biquad states of the current channel in local memory and moves to the next channel.

In ACM, the accelerator keeps processing the TCB's until the chain pointer becomes null.

The accelerator loads the TCB into its internal registers and uses these values to fetch coefficients and data and to store results. After processing a window of data for any channel, the accelerator writes back the `IIR_INIDX` (input index register) and `IIR_OUTIDX` (output index register) values to the TCB in memory. Then, data processing can begin from where it left off during the next time slot of that channel.

For 32-bit mode, the write-back values for the index registers are equal to $IIR_{II} + W$ and $IIR_{OI} + W$.

For 40-bit mode, the write-back values are: $IIR_INIDX + 2 \times W$ and $IIR_OUTIDX + 2 \times W$.

Accelerator input and output channels connect to system memory.

NOTE: The `IIR_CTL2` register is part of the IIR TCB. This configuration allows software to program individual IIR channels having different control attributes.

In ACM, when the `IIR_CTL1.SMQ_LIUPS_EN` bit is set, the accelerator updates the `IIR_INIDX` and `IIR_OUTIDX` bit fields of the TCB in memory after processing a window of data and according to the circular buffer scheme. When the `IIR_CTL1.SMQ_LIUPS_EN` bit is cleared, the accelerator updates the `IIR_INIDX` and `IIR_OUTIDX` bit fields of the TCB in memory to `0x00000000` and `0xFFFFFFFF` after processing a window of data. The `IIR_CTL1.SMQ_LIUPS_EN` bit is only valid in ACM.

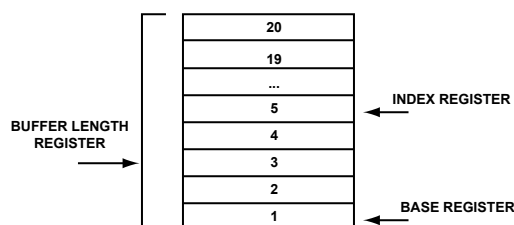


Figure 40-5: Circular Buffer Addressing

Interrupts

The *IIR Interrupt Overview* table provides the source of interrupt and service instructions for the IIR interrupts.

Accelerator Mode	Default Programmable Interrupt	Sources	Masking	Service
Legacy Mode	IIR_DMA	Window Complete	N/A	ROC from IIR_DMASTAT + RTI instruction
		All channels complete		
	IIR_STAT	MAC IEEE floating point exceptions	N/A	ROC from IIR_MACSTAT + RTI instruction
		MAC fixed point over-flow		
Auto Configuration Mode	IIR_DMA	Window Complete	IIR_CTL2 . IMASK	ROC from IIR_DMASTAT + RTI instruction
	IIR_STAT	MAC IEEE floating point exceptions	N/A	ROC from IIR_MACSTAT + RTI instruction
		MAC fixed point over-flow	N/A	

Sources

The IIR module drives two interrupt signals, IIR_DMA for the DMA status and IIR_STAT for the MAC status. The IIR module generates interrupts as described in the following sections.

Window Complete

This interrupt is generated at the end of each channel when all the output samples are calculated corresponding to a window and updated index values are written back.

In legacy mode, if the IIR_CTL1 . CCINTR bit is set, an interrupt is generated after completion of window processing of each channel. In ACM, the interrupt generation can be selectively masked using the IIR_CTL2 . IMASK bit for each channel.

Interrupt masking for each channel can be enabled in legacy mode by setting the IIR_CTL1 . L_TIMASK_EN bit. In legacy mode, when the IIR_CTL1 . L_TIMASK_EN bit is set, interrupt generation for each channel is controlled using the IIR_CTL2 . IMASK bit.

All Channels Complete

This interrupt is generated when all the channels are complete or when one iteration of time slots completes. The interrupt follows the access completion rule, where the interrupt is generated when all data are written back to system memory.

The All Channels Complete interrupt source is only applicable in legacy mode. If the IIR_CTL1 . CCINTR and IIR_CTL1 . L_TIMASK_EN bits are not set in legacy mode, then the all channel complete interrupt is generated. In ACM, interrupt generation for each channel is controlled using the IIR_CTL2 . IMASK bit.

NOTE: In ACM, the Output DMA complete interrupt can be generated after Save State completes for the channel by clearing the `IIR_SGCTL.SSESEL` bit. If this bit is set, the Output DMA interrupt is generated before the Save State completes for the channel.

In Legacy Mode, if the `IIR_CTL1.CCINTR` and `IIR_CTL1.SSESEL` bits are cleared, the All Channel Done Interrupt is generated after the Save State operation is completed for all the channels.

MAC Status

A MAC status interrupt is generated under these conditions:

- Multiplier result zero - Set if multiplier result is zero
- Multiplier result infinity - Set if multiplier result is infinity
- Multiply invalid - Set if multiply operation is invalid
- Adder result zero - Set if adder result is zero
- Adder result infinity - Set if adder result is infinity
- Adder invalid - Set if addition is invalid

Programming Model

The following sections provide programming examples in legacy mode and ACM.

Legacy Mode

The IIR accelerator supports up to 32 channels which are time division multiplexed (TDM). Each channel can have a maximum of 64 biquads. The window size for each channel is configurable using the control registers. The maximum configurable window size is 1024.

For multi-channel filtering:

1. Program the number of channels using the `IIR_CTL1.CH` bits.
2. Configure the TCBs in system memory with one channel's TCB pointing to the next channels TCB. The TCB of the last channel should point to the TCB of the first channel.
3. Write the first TCB value into the `IIR_CHNPTR` register and enable the accelerator.
4. If the `IIR_CTL1.BURSTEN` bit is set, ensure that the coefficient buffer end address plus 7 words do not fall outside the valid address range and that the buffer is initialized. When the burst feature is enabled, the IIR module issues an INCR8 burst. When the data count % 8 == 7, the extra 7 words are always read, but ignored by the IIR module.

NOTE: Data count = $\sum(B_n * 7)$ or $\sum(B_n * 14)$; where B_n is number of biquads for channel N. Each biquads contains 7 words for 32-bit mode or 14 words in 40-bit mode.

The *Biquad Processing Program Flow* figure shows the biquad processing program flow.

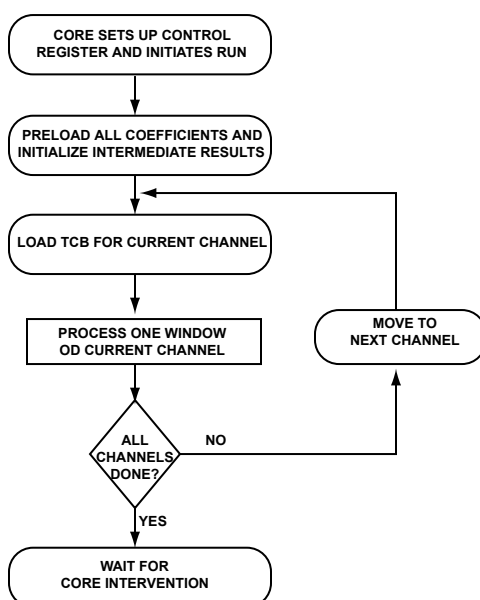


Figure 40-6: Biquad Processing Program Flow

The accelerator uses the following procedure to process biquads:

1. The controller loads all coefficients of all the channels into local storage.
2. Once all the coefficients are loaded, the controller goes to the first biquad of the first channel and calculates the output of the first biquad and updates the intermediate results for that biquad. If the `IIR_CTL1.L_TWAIT_EN` bit is set and the `IIR_CTL2.TWAIT` bit is set for the channel, the accelerator waits for the input trigger to start the computation.
3. The accelerator then moves to the next biquad of that channel and repeats the process until all the biquads for that channel are completed and the results are stored to memory.
4. The process is repeated for next sample until one window of the corresponding channel is processed.
5. After one window of the channel is processed, the accelerator moves to the next channel and computes the results.

NOTE: All the addresses programmed in the 32-bit address boundaries and must not contain the lower 2 bits (which are assumed as zeros).

Auto Configuration Mode

In ACM, there is no limit on the number of channels the accelerator can process. The accelerator keeps processing the TCBs until the chain pointer becomes null. Each channel can have a maximum of 64 biquads. Maximum window size configurable is 1024.

The figure shows multi-channel filtering in ACM. Multiple channels are processed in a time multiplexed format (TDM).

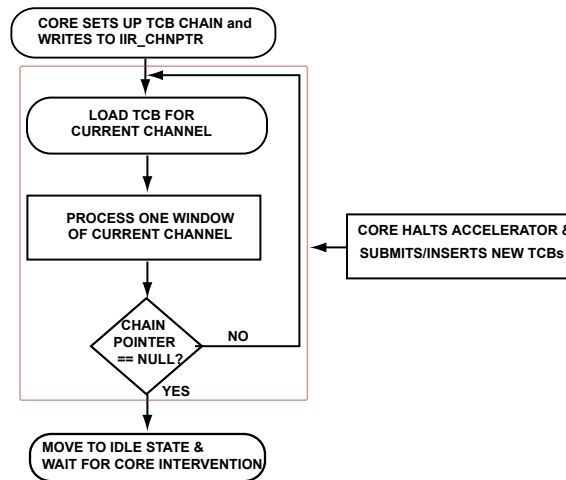


Figure 40-7: Multi-Channel Filtering in Auto Configuration Mode

For multi-channel filtering:

1. Configure the TCBs in system memory with one channel's TCB pointing to the next channel's TCB. There is no limit on the number of channels to be configured. The TCB of the last channel need not be pointing to the TCB of the first channel.
2. Write the first TCB value to the `IIR_CHNPTR` register and enable the accelerator.

The accelerator fetches the first channel's TCB. Using it as pointer, loads the biquad coefficients and biquad states for that channel into its local memory and loads the `IIR_CTL2` and `IIR_SGCTL` registers to configure the filter parameters corresponding to that channel.

If the `IIR_CTL2.TWAIT` bit is set, the accelerator waits for an input trigger to start the window processing for the channel.

The accelerator computes the output of the first biquad and updates the intermediate results for that biquad. Then, the accelerator moves to the next biquad of that channel and repeats the process until all the biquads for that channel are completed and the results are stored to memory. The process is repeated with the next sample until one window of the corresponding channel is processed. If the `IIR_CTL1.SS` bit is set, the accelerator saves the states of the biquads of the channel into system memory. If the `IIR_CTL2.TMASK` bit is cleared the accelerator sends an output trigger after completion of processing the channel.

If the `IIR_CTL2.IMASK` bit is cleared, the accelerator interrupts the core after completion of processing a particular channel.

At the end of the window, the accelerator updates the `IIR_INIDX` and `IIR_OUTIDX` registers to `0x00000000` and `0xFFFFFFFF` in the TCB of system memory and moves to the next channel.

3. At any instant, as required, the core halts the accelerator. It sets the `IIR_CTL1.HALT` bit and appropriately takes action to submit/insert new TCBs and clears the `IIR_CTL1.HALT` bit to resume channel processing.

If the `IIR_CHNPTR` register is zero (last channel is being processed or channel processing is complete) after halting the accelerator, the `IIR_CHNPTR` register is appropriately written before resuming the accelerator channel processing.

If the accelerator is idle after halt, `IIR_CTL1.EN` bit is toggled to disable and re-enable the accelerator and the `IIR_CTL1.HALT` bit is cleared and the accelerator resumes channel processing.

4. The accelerator continues processing until all the channels are complete. Repeat *Step 3* (if required) to submit/insert new channels.

ADSP-2159x IIR Register Descriptions

The IIR filter accelerator (IIR) contains the following registers.

Table 40-7: ADSP-2159x IIR Register List

Name	Description
<code>IIR_CHNPTR</code>	Chain Pointer Register
<code>IIR_COEFIDX</code>	Coefficient Buffer Index Register
<code>IIR_COEFLEN</code>	Coefficient Buffer Length Register
<code>IIR_COEFMOD</code>	Coefficient Index Modifier Register
<code>IIR_CTL1</code>	Global Control Register
<code>IIR_CTL2</code>	Channel Control Register
<code>IIR_DBG_ADDR</code>	IIR Debug Address Register
<code>IIR_DBG_CTL</code>	IIR Debug Control Register
<code>IIR_DBG_RDDAT_HI</code>	IIR Debug Read Data High Register
<code>IIR_DBG_RDDAT_LO</code>	IIR Debug Read Data Low Register
<code>IIR_DBG_WRDAT_HI</code>	IIR Debug Write Data High Register
<code>IIR_DBG_WRDAT_LO</code>	IIR Debug Write Data Low Register
<code>IIR_DMASTAT</code>	DMA Status Register
<code>IIR_INBASE</code>	Input Buffer Base Register
<code>IIR_INIDX</code>	Input Data Index Register
<code>IIR_INLEN</code>	Input Data Buffer Length Register
<code>IIR_INMOD</code>	Input Data Index Modifier Register
<code>IIR_MACSTAT</code>	MAC Status Register
<code>IIR_OUTBASE</code>	Output Buffer Base Register
<code>IIR_OUTIDX</code>	Output Data Buffer Index Register
<code>IIR_OUTLEN</code>	IIR Output Data Buffer Length Register

Table 40-7: ADSP-2159x IIR Register List (Continued)

Name	Description
IIR_OUTMOD	IIR Output Data Index Modifier Register
IIR_SCTL1	Software Control Register1
IIR_SCTL2	Software Control Register2
IIR_SGCTL	Secondary Global Control Register

Chain Pointer Register

The `IIR_CHNPTR` register should be written with word address without the lower 2 bits.

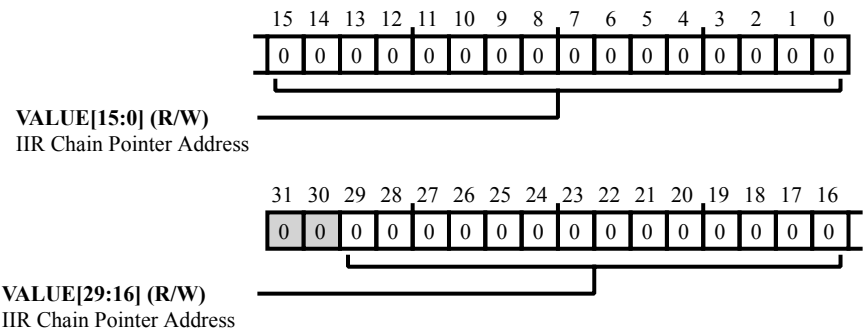


Figure 40-8: IIR_CHNPTR Register Diagram

Table 40-8: IIR_CHNPTR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	IIR Chain Pointer Address. The <code>IIR_CHNPTR.VALUE</code> bit field contains the chain pointer address.

Coefficient Buffer Index Register

The `IIR_COEFIDX` register contains the word address with the lower 2 bits removed.

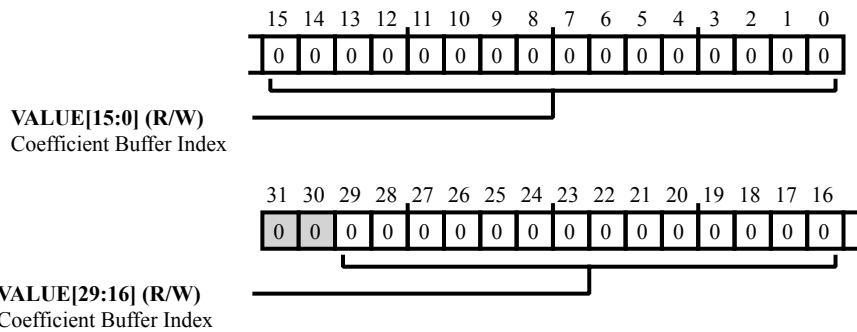


Figure 40-9: IIR_COEFIDX Register Diagram

Table 40-9: IIR_COEFIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Coefficient Buffer Index. The <code>IIR_COEFIDX.VALUE</code> bit field provides the coefficient buffer index.

Coefficient Buffer Length Register

The `IIR_COEFLEN` register provides the coefficient buffer length.

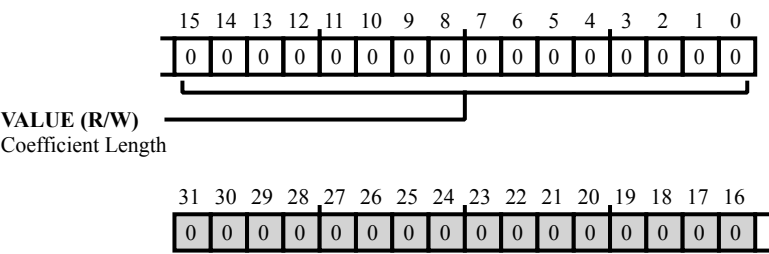


Figure 40-10: IIR_COEFLEN Register Diagram

Table 40-10: IIR_COEFLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Coefficient Length. The <code>IIR_COEFLEN.VALUE</code> bit field provides the coefficient buffer length.

Coefficient Index Modifier Register

The `IIR_COEFMOD` register provides the coefficient index modifier.

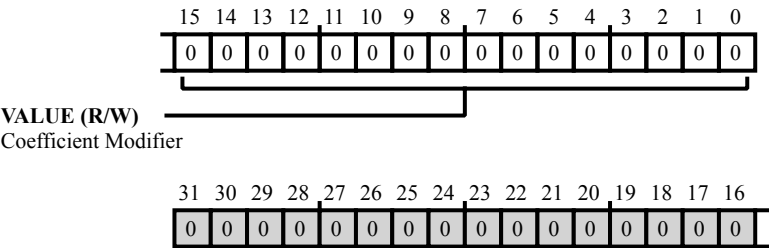


Figure 40-11: IIR_COEFMOD Register Diagram

Table 40-11: IIR_COEFMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Coefficient Modifier. The <code>IIR_COEFMOD.VALUE</code> bit field provides the coefficient modifier.

Global Control Register

The `IIR_CTL1` register is used to configure the global parameters for the accelerator. These parameters include the number of channels, channel auto iterate, DMA enable, and accelerator enable.

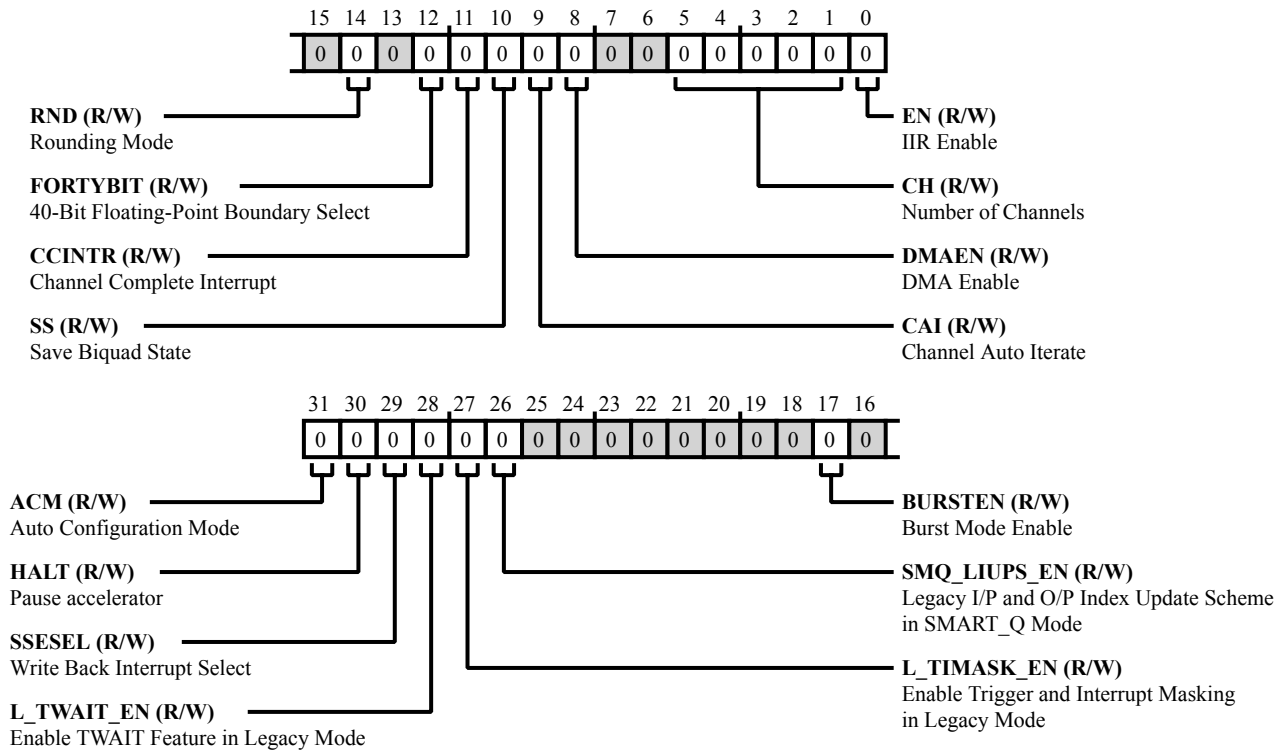


Figure 40-12: IIR_CTL1 Register Diagram

Table 40-12: IIR_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	ACM	Auto Configuration Mode. The <code>IIR_CTL1.ACM</code> bit configures the mode for loading the TCB.
		0 Legacy Mode
		1 Auto Configuration Mode
30 (R/W)	HALT	Pause accelerator. The <code>IIR_CTL1.HALT</code> bit determines whether the accelerator pauses so that the core can check the status and modify or submit a job or the accelerator is released for further processing of data. This bit is only valid in Auto Configuration Mode (ACM).
		0 Release accelerator
		1 Pause accelerator

Table 40-12: IIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	SSESEL	Write Back Interrupt Select. The IIR_CTL1 . SSESEL bit selects whether to generate an interrupt after the write back or after save state completion.
		0 Generate Interrupt after save state completion
		1 Generate interrupt after write back completion
28 (R/W)	L_TWAIT_EN	Enable TWAIT Feature in Legacy Mode. The IIR_CTL1 . L_TWAIT_EN bit indicates whether the TWAIT feature is available in legacy mode. This bit is only valid in legacy mode.
		0 Not Available
		1 Available
27 (R/W)	L_TIMASK_EN	Enable Trigger and Interrupt Masking in Legacy Mode. The IIR_CTL1 . L_TIMASK_EN bit enables trigger and interrupt masking in legacy mode. This bit is only valid in legacy mode.
		0 Disable
		1 Enable
26 (R/W)	SMQ_LIUPS_EN	Legacy I/P and O/P Index Update Scheme in SMART_Q Mode. The IIR_CTL1 . SMQ_LIUPS_EN bit configures the scheme the accelerator uses to update the input index (II) and output index (OI). This bit is only valid in ACM.
		0 Update the II with all '0' and the OI with all 'F', respectively
		1 Update II and OI after circular buffer scheme
17 (R/W)	BURSTEN	Burst Mode Enable. The IIR_CTL1 . BURSTEN bit field is set, burst access is enabled.
		0 Disable burst mode
		1 Enable burst mode
14 (R/W)	RND	Rounding Mode. The IIR_CTL1 . RND bit field selects the rounding mode for floating-point format.
		0 Round to nearest (even)
		1 Truncate (Round towards zero)

Table 40-12: IIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	FORTYBIT	40-Bit Floating-Point Boundary Select. The IIR_CTL1.FORTYBIT bit selects between 32-bit IEEE floating-point format or 40-bit IEEE floating-point format.
		0 32-bit IEEE floating-point
		1 40-bit IEEE floating-point
11 (R/W)	CCINTR	Channel Complete Interrupt. The IIR_CTL1.CCINTR bit configures the channel complete interrupt to generate when all channels are done or after each channel is done. This bit is only valid in legacy mode when the IIR_CTL1.L_TIMASK_EN bit is cleared (=0). In Auto Configuration Mode (ACM) or when the IIR_CTL1.L_TIMASK_EN is set (=1), interrupt generation for each channel is controlled using IIR_CTL2.IMASK bit.
		0 Interrupt is generated only when all channels are done (default)
		1 Interrupt is generated after each channels is done (default)
10 (R/W)	SS	Save Biquad State. The IIR_CTL1.SS bit configures the accelerator to store the Dk register settings into the internal memory. This can be used to save the biquad states before switching to another high priority accelerator task. This bit is only valid in legacy mode. In Auto Configuration Mode(ACM), Save State for each channel is controlled using IIR_SGCTL.SS bit
9 (R/W)	CAI	Channel Auto Iterate. The IIR_CTL1.CAI bit sets whether TDM processing stops (idle) once all channels complete processing or moves to first channel and continues TDM processing in a loop when all channels complete processing. Channel Auto Iterate is not available in Auto Configuration Mode(ACM). The accelerator keeps processing the TCBs until the chain pointer becomes null in Auto Configuration Mode(ACM).
		0 TDM processing stops (idle) once all channels complete processing
		1 Moves to first channel and continues TDM processing in a loop when all channels complete processing
8 (R/W)	DMAEN	DMA Enable. The IIR_CTL1.DMAEN bit enables DMA on the accelerator.
		0 Disable
		1 Enable

Table 40-12: IIR_CTL1 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
5:1 (R/W)	CH	Number of Channels. The IIR_CTL1 .CH bit field configures the number of channels and is programmable between 0-31 (channels = NCH + 1). This bit field is only valid in Legacy Mode. In Auto Configuration Mode(ACM), there is no limit on the number of channels. the accelerator keeps processing the TCBs until the chain pointer becomes null.	
0 (R/W)	EN	IIR Enable. The IIR_CTL1 .EN bit enables or disables the IIR accelerator.	
		0	IIR disabled
		1	IIR enabled

Channel Control Register

The `IIR_CTL2` register is used to configure the channel specific parameters. These parameters include the number of biquads and window size. In Auto Configuration Mode(ACM), this register is also used to configure additional channel specific parameters like interrupts and triggers.

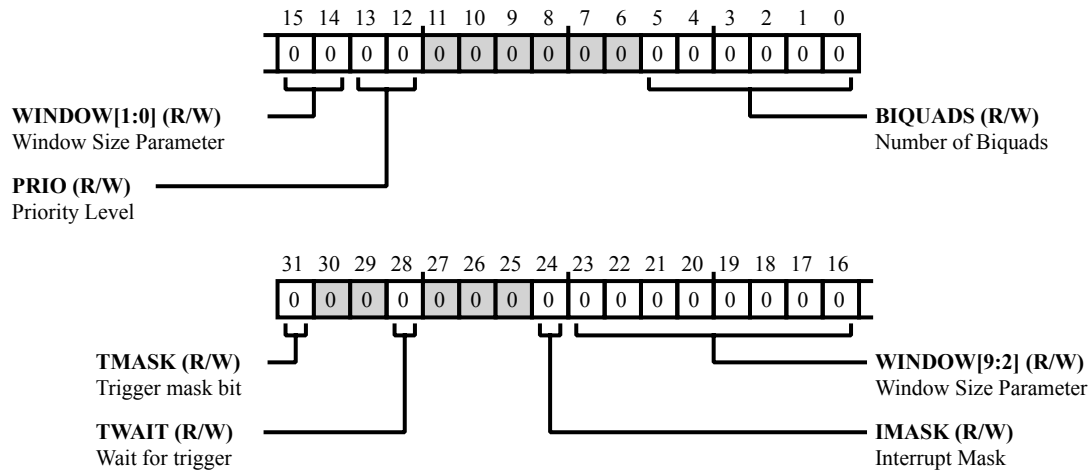


Figure 40-13: IIR_CTL2 Register Diagram

Table 40-13: IIR_CTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	TMASK	Trigger mask bit. The <code>IIR_CTL2</code> . <code>TMASK</code> bit enables trigger generation for the channel. It is valid in Auto Configuration Mode (ACM) or if <code>CTL1.L_TIMASK_EN</code> is set.
		0 Enable
		1 Mask
28 (R/W)	TWAIT	Wait for trigger. The <code>IIR_CTL2</code> . <code>TWAIT</code> bit disables the wait for the trigger. It is valid in Auto Configuration Mode (ACM) or if <code>CTL1.L_TIMASK_EN</code> is set.
		0 Disable wait for the trigger for the channel
		1 Enable wait for external trigger input assertion for the channel
24 (R/W)	IMASK	Interrupt Mask. The <code>IIR_CTL2</code> . <code>IMASK</code> bit enables interrupt generation for the channel. This bit is valid in Auto Configuration Mode (ACM) or if <code>CTL1.L_TIMASK_EN</code> is set.
		0 Enable
		1 Mask

Table 40-13: IIR_CTL2 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
23:14 (R/W)	WINDOW	Window Size Parameter. The IIR_CTL2.WINDOW bit field configures the window size which specifies the number of sample/block to process (sample based processing = window size of 1). This bit field should be programmed to "actual window size required -1". For example, for sample based processing this bit field should be programmed to 0.
13:12 (R/W)	PRIO	Priority Level. The IIR_CTL2.PRIO bit field indicates the priority.
		0 Level 0 (lowest)
		1 Level 1
		2 Level 2
		3 Level 3 (highest)
5:0 (R/W)	BIQUADS	Number of Biquads. The IIR_CTL2.BIQUADS bit field configures the number of biquads and is programmable between 0-63 (number of Biquads = BIQUADS + 1).

IIR Debug Address Register

The `IIR_DBG_ADDR` register holds the debug address. If bit 11 is set, coefficient memory is selected.

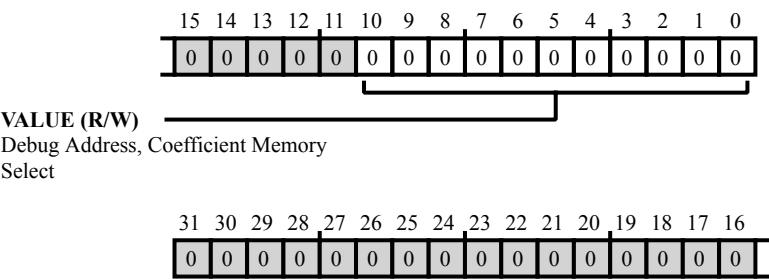


Figure 40-14: IIR_DBG_ADDR Register Diagram

Table 40-14: IIR_DBG_ADDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
10:0 (R/W)	VALUE	Debug Address, Coefficient Memory Select. The <code>IIR_DBG_ADDR.VALUE</code> bit field holds the debug address (bits 0-10). Bit 11 configures whether the memory access is to coefficient memory (=0) or to delay line memory (=1).

IIR Debug Control Register

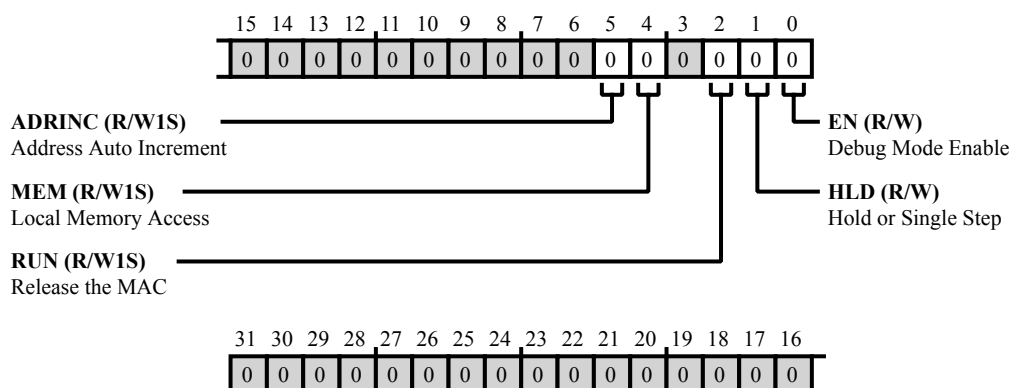


Figure 40-15: IIR_DBG_CTL Register Diagram

Table 40-15: IIR_DBG_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/W1S)	ADRINC	Address Auto Increment. The <code>IIR_DBG_CTL.ADRINC</code> bit allows the address register to auto increment on <code>IIR_DBG_WRDAT_HI/IIR_DBG_WRDAT_LO</code> writes and <code>IIR_DBG_RDDAT_HI/IIR_DBG_RDDAT_LO</code> reads.
4 (R/W1S)	MEM	Local Memory Access. The <code>IIR_DBG_CTL.MEM</code> bit allows the data and coefficients memory to be indirectly accessed.
2 (R/W1S)	RUN	Release the MAC. The <code>IIR_DBG_CTL.RUN</code> bit releases the MAC and is self clearing after one IIR clock cycle.
1 (R/W)	HLD	Hold or Single Step. The <code>IIR_DBG_CTL.HLD</code> bit function is based on the <code>IIR_DBG_CTL.MEM</code> bit setting. For <code>IIR_DBG_CTL.MEM = 0</code> this bit sets single step. For <code>IIR_DBG_CTL.MEM = 1</code> this bit sets hold data.
		0 No effect
		1 Single step (<code>IIR_DBGMEM=0</code>) or Hold data (<code>IIR_DBGMEM=1</code>)
0 (R/W)	EN	Debug Mode Enable. The <code>IIR_DBG_CTL.EN</code> bit enables debug mode. For local memory access, the <code>IIR_CTL1</code> register can be cleared.
		0 Disable
		1 Enable

IIR Debug Read Data High Register

The `IIR_DBG_RDDAT_HI` register is part of the 40-bit wide debug mode read data register and holds the upper 8 bits.

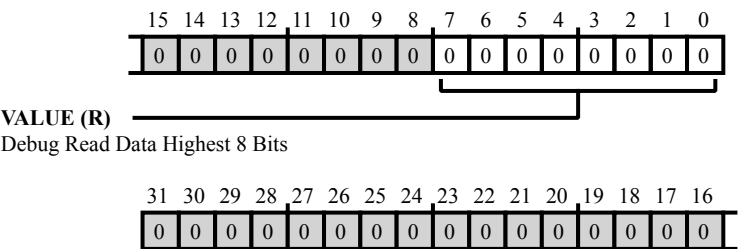


Figure 40-16: IIR_DBG_RDDAT_HI Register Diagram

Table 40-16: IIR_DBG_RDDAT_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	VALUE	Debug Read Data Highest 8 Bits. The <code>IIR_DBG_RDDAT_HI.VALUE</code> bit field holds the upper 8-bit read data.

IIR Debug Read Data Low Register

The `IIR_DBG_RDDAT_LO` register is part of the 40-bit wide debug mode read data register and holds the lower 32 bits.

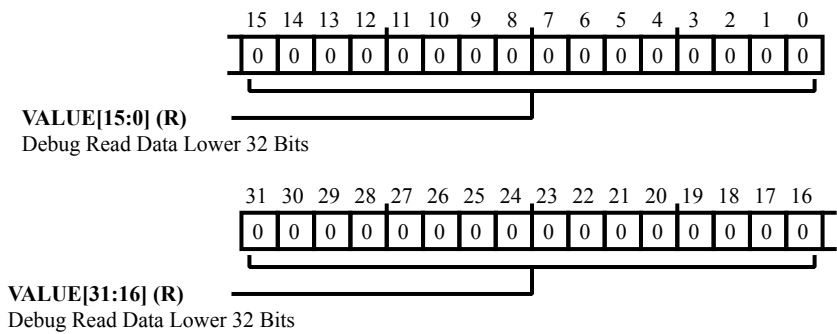


Figure 40-17: IIR_DBG_RDDAT_LO Register Diagram

Table 40-17: IIR_DBG_RDDAT_LO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	Debug Read Data Lower 32 Bits. The <code>IIR_DBG_RDDAT_LO.VALUE</code> bit field holds the lower 32-bit read data.

IIR Debug Write Data High Register

The `IIR_DBG_WRDAT_HI` register is part of the 40-bit wide debug mode write data register and holds the upper 8 bits.

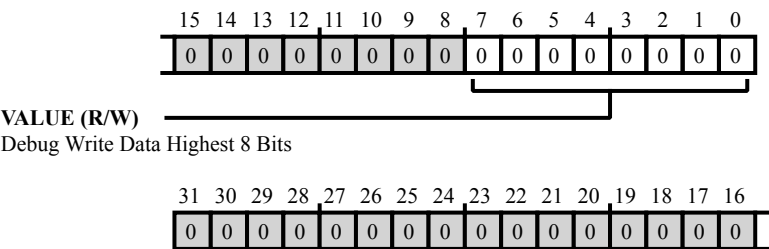


Figure 40-18: IIR_DBG_WRDAT_HI Register Diagram

Table 40-18: IIR_DBG_WRDAT_HI Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/W)	VALUE	Debug Write Data Highest 8 Bits. The <code>IIR_DBG_WRDAT_HI.VALUE</code> bit field holds the upper 8-bit write data.

IIR Debug Write Data Low Register

The `IIR_DBG_WRDAT_LO` register is part of the 40-bit wide debug mode write data register and holds the lower 32 bits.

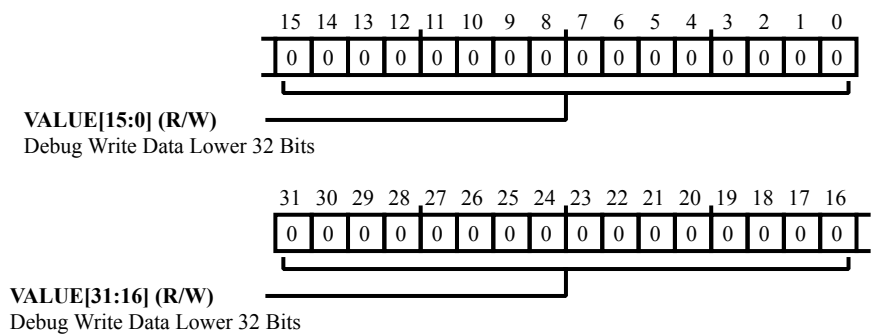


Figure 40-19: IIR_DBG_WRDAT_LO Register Diagram

Table 40-19: IIR_DBG_WRDAT_LO Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Debug Write Data Lower 32 Bits. The <code>IIR_DBG_WRDAT_LO.VALUE</code> bit field holds the lower 32-bit write data.

DMA Status Register

The `IIR_DMASTAT` registers indicate the status of DMA operations.

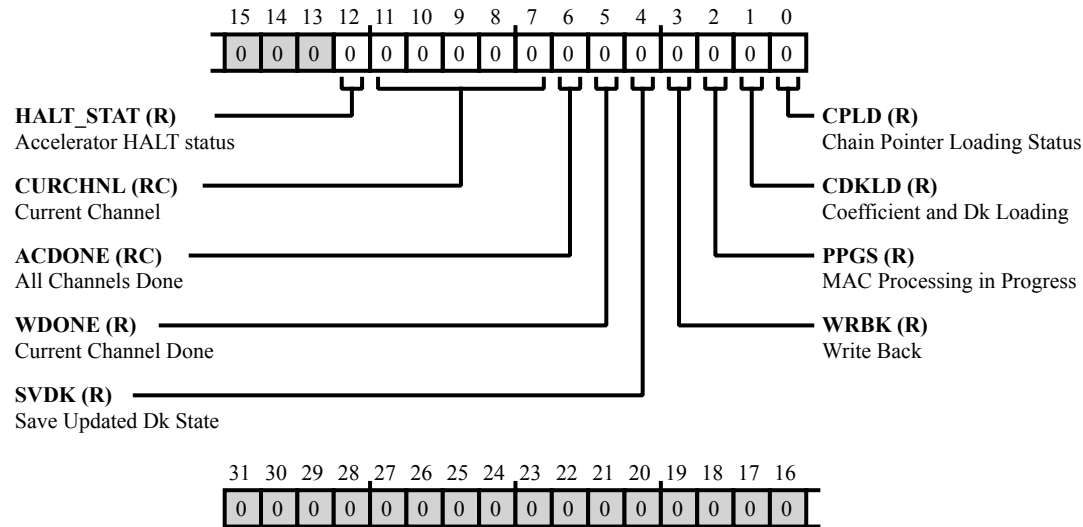


Figure 40-20: IIR_DMASTAT Register Diagram

Table 40-20: IIR_DMASTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/NW)	HALT_STAT	Accelerator HALT status. Acknowledge to core that accelerator is in halt state, if set.
11:7 (RC/NW)	CURCHNL	Current Channel. The <code>IIR_DMASTAT.CURCHNL</code> bit field indicates the channel that is being processed in the TDM slot. Zero indicates the last slot.
6 (RC/NW)	ACDONE	All Channels Done. The <code>IIR_DMASTAT.ACDONE</code> bit indicates all channels are done processing. Note that the <code>IIR_CTL1.CCINTR</code> bit does not affect this status bit. This bit is sticky and is cleared on register read.
5 (R/NW)	WDONE	Current Channel Done. The <code>IIR_DMASTAT.WDONE</code> bit indicates the processing of the current channel is complete. Note that the <code>IIR_CTL1.CCINTR</code> bit does not affect this status bit. This bit is sticky and is cleared on register read.
4 (R/NW)	SVDK	Save Updated Dk State. If there is more than one channel (<code>IIR_CTL1.CH>0</code>), the <code>IIR_DMASTAT.SVDK</code> bit toggles between 0 and 1 as it starts and completes the save state operation on one channel at a time.

Table 40-20: IIR_DMASTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/NW)	WRBK	Write Back. The IIR_DMASTAT.WRBK bit indicates the accelerator is writing back updated index registers.
2 (R/NW)	PPGS	MAC Processing in Progress. The IIR_DMASTAT.PPGS bit indicates MAC processing is in progress.
1 (R/NW)	CDKLD	Coefficient and Dk Loading. The IIR_DMASTAT.CDKLD bit indicates the coefficient and Dk are loading.
0 (R/NW)	CPLD	Chain Pointer Loading Status. The IIR_DMASTAT.CPLD bit indicates the IIR is in the chain pointer load state.
		0 State machine not in chain pointer load state
		1 State machine in chain pointer load state

Input Buffer Base Register

The `IIR_INBASE` register contains the word address with the lower 2 bits removed.

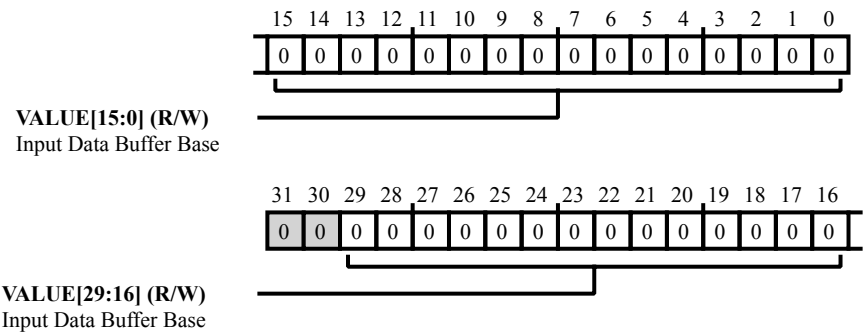


Figure 40-21: IIR_INBASE Register Diagram

Table 40-21: IIR_INBASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Input Data Buffer Base. The <code>IIR_INBASE.VALUE</code> bit field value is the input data buffer base address.

Input Data Index Register

The `IIR_INIDX` register contains a word address with the lower 2 bits removed.

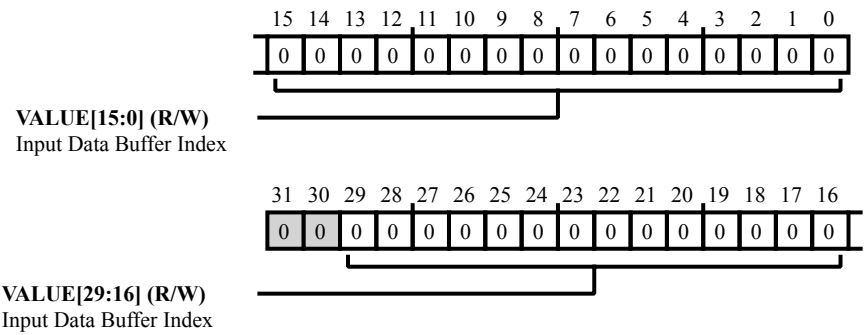


Figure 40-22: IIR_INIDX Register Diagram

Table 40-22: IIR_INIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Input Data Buffer Index. The <code>IIR_INIDX.VALUE</code> bit field value is the input data buffer index.

Input Data Buffer Length Register

The `IIR_INLEN` register provides the input data buffer length.

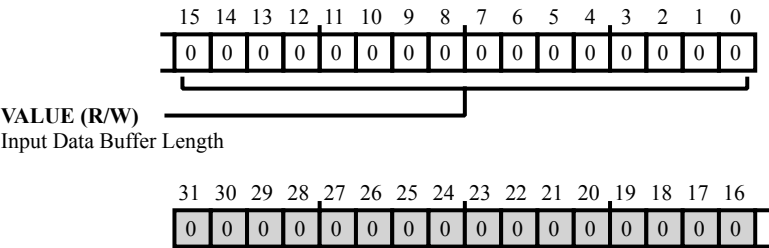


Figure 40-23: IIR_INLEN Register Diagram

Table 40-23: IIR_INLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Input Data Buffer Length. The <code>IIR_INLEN.VALUE</code> bit field value is the input data buffer length.

Input Data Index Modifier Register

The `IIR_INMOD` register provides the 16-bit input data buffer index modifier.

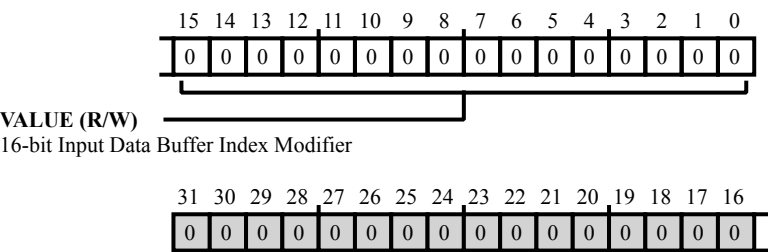


Figure 40-24: IIR_INMOD Register Diagram

Table 40-24: IIR_INMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Input Data Buffer Index Modifier. The <code>IIR_INMOD.VALUE</code> bit field value is the 16-bit input data buffer modifier.

MAC Status Register

The `IIR_MACSTAT` register indicates the status of MAC operations.

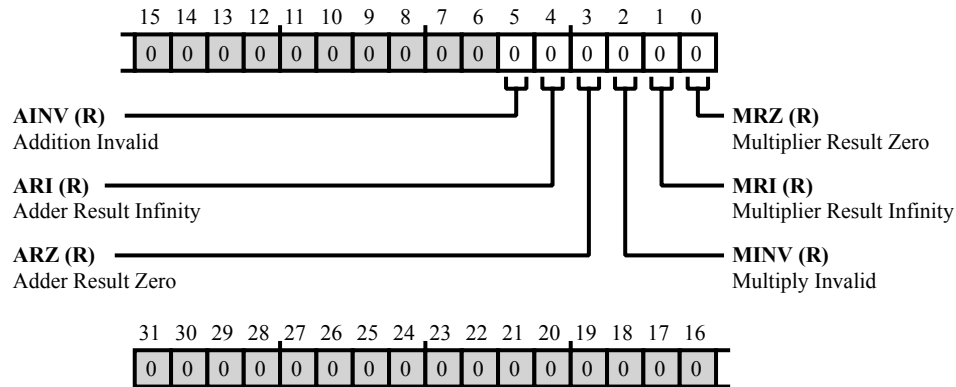


Figure 40-25: IIR_MACSTAT Register Diagram

Table 40-25: IIR_MACSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
5 (R/NW)	AINV	Addition Invalid. The <code>IIR_MACSTAT.AINV</code> bit indicates the addition is invalid.
4 (R/NW)	ARI	Adder Result Infinity. The <code>IIR_MACSTAT.ARI</code> bit indicates the adder result is infinity.
3 (R/NW)	ARZ	Adder Result Zero. The <code>IIR_MACSTAT.ARZ</code> bit indicates the adder result is zero.
2 (R/NW)	MINV	Multiply Invalid. The <code>IIR_MACSTAT.MINV</code> bit indicates the multiply operation is invalid.
1 (R/NW)	MRI	Multiplier Result Infinity. The <code>IIR_MACSTAT.MRI</code> bit indicates the multiplier result is infinity.
0 (R/NW)	MRZ	Multiplier Result Zero. The <code>IIR_MACSTAT.MRZ</code> bit indicates the multiplier result is zero.

Output Buffer Base Register

The `IIR_OUTBASE` register contains the word address with the lower 2 bits removed.

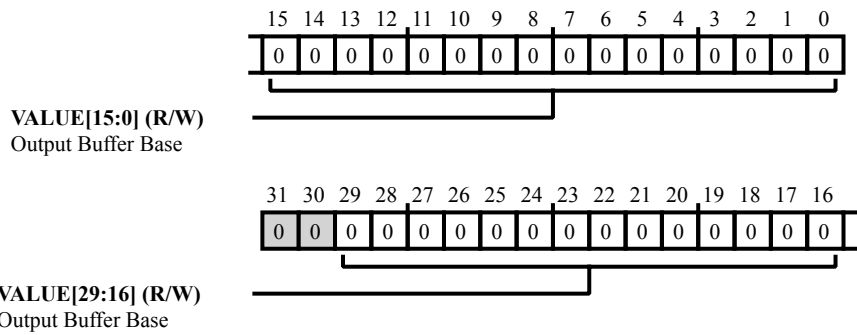


Figure 40-26: IIR_OUTBASE Register Diagram

Table 40-26: IIR_OUTBASE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Output Buffer Base. The <code>IIR_OUTBASE.VALUE</code> bit field provides the output buffer base address.

Output Data Buffer Index Register

The `IIR_OUTIDX` register should be written with word address without the lower 2 bits

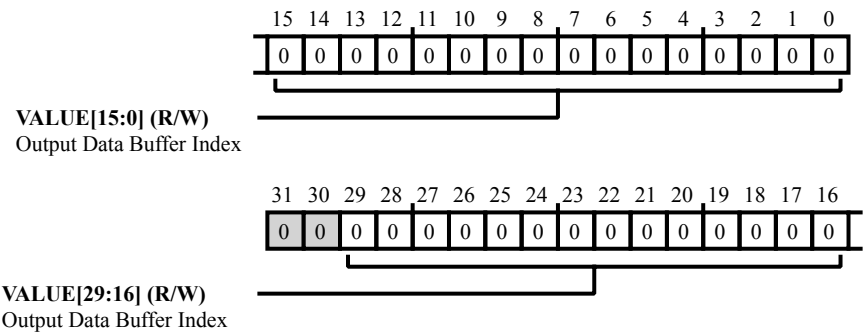


Figure 40-27: IIR_OUTIDX Register Diagram

Table 40-27: IIR_OUTIDX Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29:0 (R/W)	VALUE	Output Data Buffer Index. The <code>IIR_OUTIDX.VALUE</code> bit field provides the output data buffer index.

IIR Output Data Buffer Length Register

The `IIR_OUTLEN` register provides the output data buffer length.

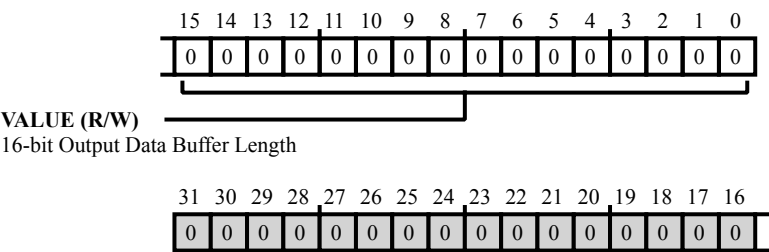


Figure 40-28: IIR_OUTLEN Register Diagram

Table 40-28: IIR_OUTLEN Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Output Data Buffer Length. The <code>IIR_OUTLEN.VALUE</code> bit field provides the output data buffer length.

IIR Output Data Index Modifier Register

The `IIR_OUTMOD` register provides the output data index modifier.

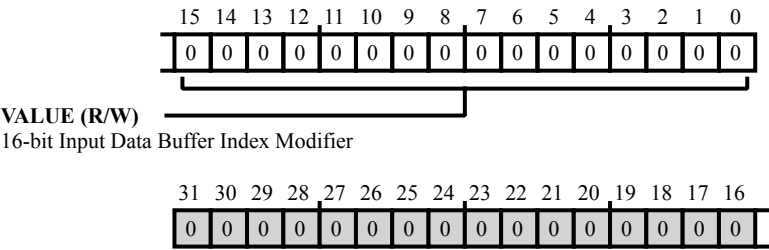


Figure 40-29: IIR_OUTMOD Register Diagram

Table 40-29: IIR_OUTMOD Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	16-bit Input Data Buffer Index Modifier. The <code>IIR_OUTMOD.VALUE</code> bit field provides the output data buffer index modifier.

Software Control Register1

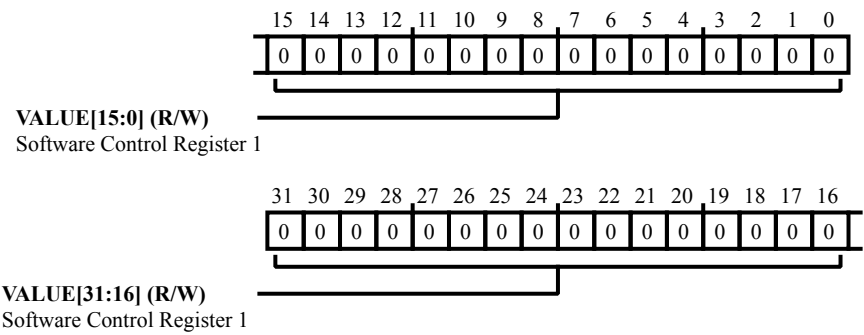


Figure 40-30: IIR_SCTL1 Register Diagram

Table 40-30: IIR_SCTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Software Control Register 1.

Software Control Register2

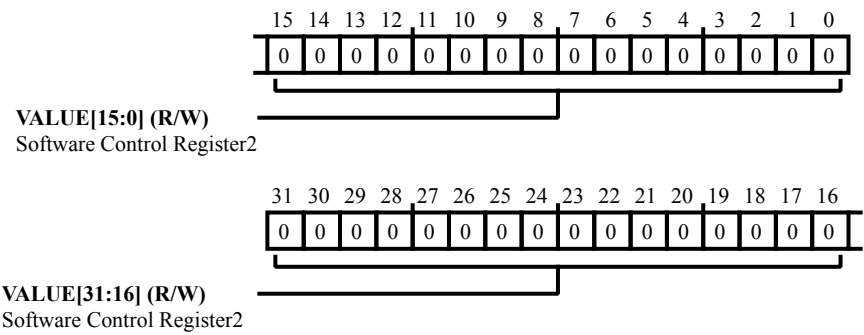


Figure 40-31: IIR_SCTL2 Register Diagram

Table 40-31: IIR_SCTL2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Software Control Register2.

Secondary Global Control Register

The `IIR_SGCTL` register configures the global parameters for the accelerator in ACM mode for loading CTL1 register as part of TCB.

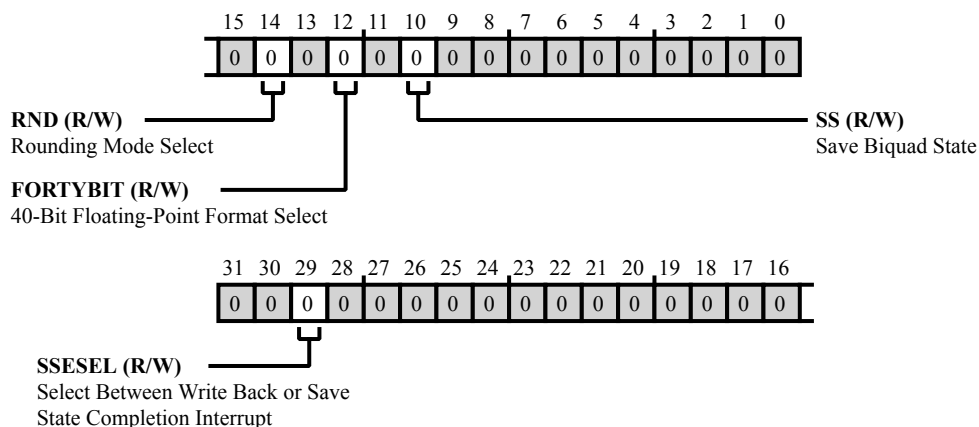


Figure 40-32: `IIR_SGCTL` Register Diagram

Table 40-32: `IIR_SGCTL` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	SSESEL	Select Between Write Back or Save State Completion Interrupt.
14 (R/W)	RND	Rounding Mode Select. The <code>IIR_SGCTL.RND</code> bit selects the rounding mode for a floating-point mode compatible with SHARC+.
		0 Round to nearest (even)
		1 Truncate (Round towards zero)
12 (R/W)	FORTYBIT	40-Bit Floating-Point Format Select. The <code>IIR_SGCTL.FORTYBIT</code> bit selects the floating point format.
		0 32-bit floating point
		1 40-bit floating point
10 (R/W)	SS	Save Biquad State. The <code>IIR_SGCTL.SS</code> bit configures the accelerator to store the Dk register settings into the internal memory. This can be used to save the biquad states before switching to another high priority accelerator task.

41 Boot ROM and Booting the Processor

Bootstrapping or booting is the series of events that occur when the system applies power to the processor or when the processor enters a hardware reset state. This section gives an in-depth description of these events and how to integrate an application effectively.

On reset, the processor begins fetching instruction from an internal ROM. The boot code contained within the ROM is designed to facilitate loading an application. The boot code can automatically initialize certain peripherals for communication based on a chosen boot mode, then load an application. For more information on what boot modes are available, see the [Boot Modes](#) section. The boot code can efficiently load an entire application, code, and data, into appropriate locations after the development tools repack the application into a boot stream.

A boot stream is an application or data that the boot-loader tool splits into blocks. A 16-byte header in each block provides instruction to the boot code for processing the associated data. The processor can perform several boot functions, depending on the flags set in the header. For more details on what options are available and a description of the stream format, refer to the [Boot Loader Stream](#) section.

The boot ROM provides a mechanism through available non-volatile programmable memory (OTP on this processor) to customize different aspects of the boot process. These customizations include: overriding default boot-peripheral instance, overriding default peripheral-timing parameters and disabling boot modes.

NOTE: In this chapter the term boot ROM describes the boot code that cannot be altered. The terms *program* and *application* are used to describe code that is used to customize the boot process.

Several utilities of the boot code are also available to the application. These utilities include features such as copying memory, comparing memory, or loading another boot stream at run time. The APIs may be used to help ensure that application code is more compatible with future products. The boot code also provides the ability to define a custom boot mode. This capability helps when support is not available for a desired boot mode. It allows second stage boot loaders for unsupported boot peripherals to leverage a significant amount of the existing boot ROM functionality.

SRAM Requirements

The boot process reserves 8K bytes of L2 ECC Protected SRAM for dedicated use. This topic describes how the reserved memory region is used during boot.

The boot process requires SRAM resources for stack use and to store various data items that require read/write access during the boot process. 8K bytes of L2 ECC protected SRAM is reserved for this purpose. The *Boot Process SRAM Requirements* table describes the various items stored in this memory region.

Table 41-1: Boot Process SRAM Requirements

Address	Size (Bytes)	Item	Description
0x201FE000	4	Reserved	
0x201FE004	4	Pointer to the struct ADI_ROM_BOOT_CONFIG object	Pointer to the boot configuration structure that is located on the stack. This location is used to find the location of the boot structure on the stack for debug.
0x201FE008	8	Reserved	
0x201FE010	1024	Internal Intermediate Buffer 0	The first of two internal buffers used for intermediate storage of boot content when using indirect and page mode accesses and for secure boot operations. Two buffers are used to allow SHA-224 and AES-128 operations to be performed on one buffer while simultaneously loading the other buffer.
0x201FE410	1024	Internal Intermediate Buffer 1	The second of two internal buffers used for intermediate storage of boot content when using indirect and page mode accesses and for secure boot operations. Two buffers are used to allow SHA-224 and AES-128 operations to be performed on one buffer while simultaneously loading the other buffer.
0x201FE810	16	struct ADI_ROM_BOOT_HEADER object	Storage location for all the block headers of the boot stream.
0x201FE820	2912	Storage for Secure Boot related descriptors	Contains a number of buffers for the various descriptors used by the Cryptographic Accelerators and provides storage for the secure header of a secure boot stream.
0x201FF400	3072	Stack for the boot process	The primary booting cores stack. The processor core should locate the stack in this region in order to preserve security in secure boot operations.

NOTE: To preserve the security of the product, the 8K byte region described here is not a bootable region of memory. If the boot process determines that a block of data in the boot stream is targeted towards this memory region, the boot process terminates and enters either the default error handler or, if applicable, a user-defined error handler. This reserved memory region is free for use after the boot process completes. In order to preserve security when using the boot API to boot a secure boot stream, the stack used during the execution of the boot API must be located at the default location in this reserved 8K bytes region of memory.

Preboot Operations

Preboot is responsible for configuration of all system resources prior to executing the required boot operation.

The steps performed by the preboot process are described here in the order of execution. Numerous stages of the preboot process are conditional based upon the content of the [RCU_BCODE](#) register.

NOTE: When a power on reset, hard reset, and software triggered system reset event completes, the processor operates in PLL Bypass mode (default). Partway through the preboot sequence the processor is brought into Full-On mode at the default settings unless the program has provisioned custom CGU settings in the OTP. The Oscillator Watchdog fault, enabled by default when reset completes, is disabled at this stage unless Oscillator Watchdogs settings are also supplied.

Start-up Sequence

This section describes the initial start-up sequence of all cores in the processor.

Upon completion of a power-on reset, hardware reset, or system reset event, only a single core is released from reset. It is responsible for managing the boot process. The following sections describe in detail the sequence of events that occur for each of the cores on the various product derivatives.

Core Reset Sequencing

System and hardware reset events result in the processor state being reset and the boot sequence is executed. Only a single core is initially released from reset and starts execution of the preboot software from the boot ROM. The sequencing between the RCU and the various cores is shown for each of the product derivatives.

For the ADSP-SC59x derivatives, core 0 is responsible for mastering the boot process. Cores 1 and 2 are held in reset until core 0 releases them from reset during the preboot phase of the boot process.

The initial sequence of events after the completion of the reset sequence are shown below.

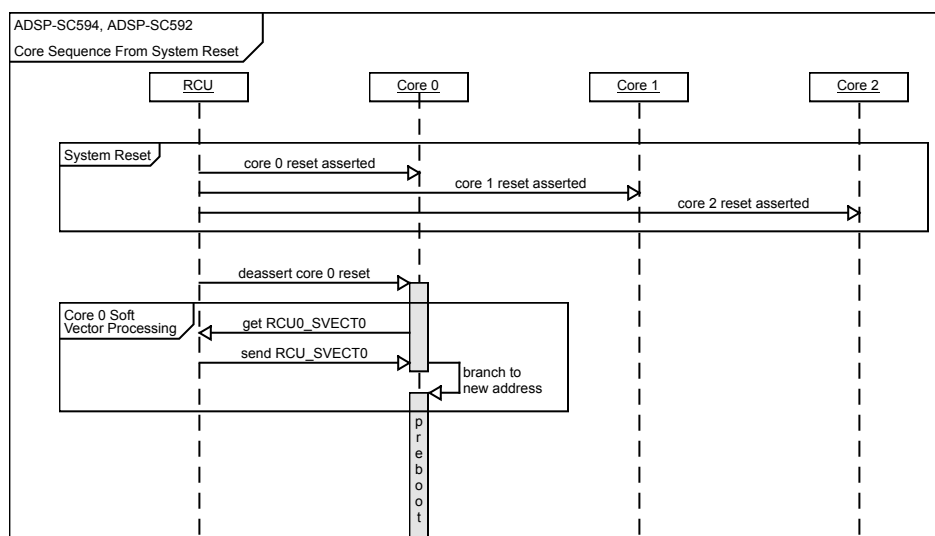


Figure 41-1: Core Reset Sequencing - ADSP-SC59x

The ADSP-2159x derivative contains two cores with IDs of 1 and 2. In this device, core 1 is responsible for mastering the boot process; core 2 is held in reset until released by core 1 during the preboot phase of the boot process. The initial sequence of events after the completion of the reset sequence are shown.

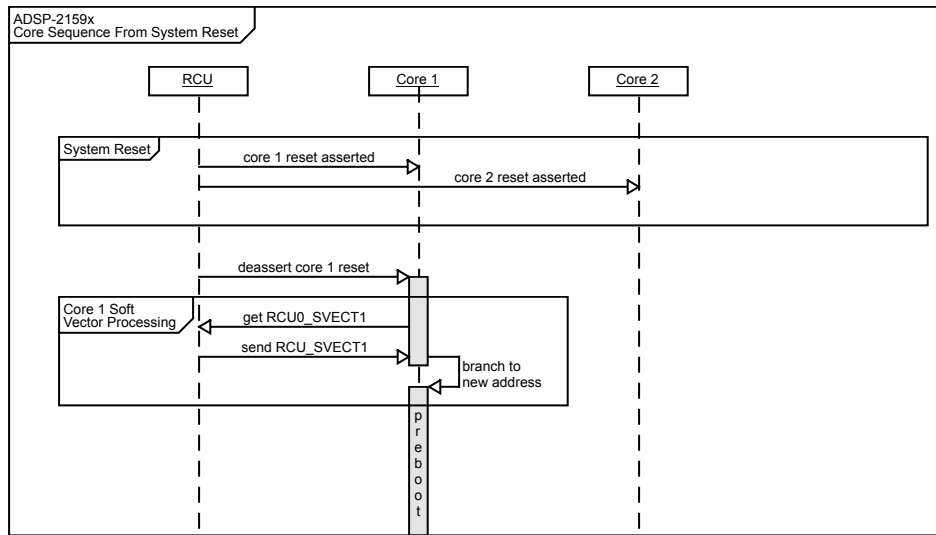


Figure 41-2: Core Reset Sequencing - ADSP-2159x

Core 1 Start-up

Describes the initial operations performed by the core 1 immediately
immediately after being released from reset and soft vector processing has completed

Upon initial release from reset the soft vectoring process results in core 1 executing from the boot ROM as long as the `RCU_SVECT1` register contained the default reset value during the soft vector processing. The operations the core performs varies depending on the product.

ADSP-SC59x Processors - Core 1 is not responsible for the booting of the processor from reset and thus enters a safe IDLE state allowing access to the cores L1 resources by any other core.

The core performs the following initial operations:

- Clear `RCU_MSG.C1IDLE`
- Check for OTPC boot completion.
- Disable Software Return feature in the BTB.
- Disable the BTB.
- Disable cache via `MODE2` register.
- Flush the cache.
- Set `IRPTL` to `0x00000000`.
- Set `RCU0_MSG.C1IDLE`
- Enter an endless IDLE loop

ADSP-2159x Processors - Core 1 is responsible for mastering the boot process and the following actions are performed.

- Clear `RCU_MSG.COIDLE`
- Check for OTPC boot completion.
- Disable Software Return feature in the BTB.
- Disable the BTB.
- Disable cache via `MODE2` register.
- Flush the cache.
- Set `IRPTL` to `0x00000000`.
- Configure primary and secondary DAG configurations to setup the C run-time environment.
- Clear the `CBUFEN`, `SRD1H`, `SRD1L`, `SRD2H`, `SRD2L`, `ALUSAT`, `TRUNCATE` bits in the `MODE1` register.
- Set the `IRPTEN`, `IPERREN`, `DPERREN`, `SPERREN` bits in the `MODE1` register.
- Read the `RCU0_STAT` register and store locally in the register set.
- Install the x.
- Configure L2 Controller
- Initiate hardware controlled L2 memory initialization sequence

Table 41-2: Primary/Secondary DAG Configuration for C Run-Time Setup

Primary/Secondary DAG Register	Value	Description
M7, M15	-1	Dedicated registers must always be set to -1
M6, M14	1	Dedicated registers must always be set to 1
M5, M13	0	Dedicated registers must always be set to 0
L0-L5	0	Preserved registers set initially to 0
L6, L7	0	Stack Length Register set initially to 0
L8-L15	0	Preserved registers set initially to 0
B6, B7	0x201FF400	Stack Base Registers
I7	0x201FFFFC	Stack Pointer
L6, L7	0x000002FF	Stack Length Registers
I6	0x201FFFFC	Frame Pointer

Core 2 Start-up

Describes the initial operations performed by core 2 immediately after being released from reset and soft vector processing has completed.

Upon initial release from reset the soft vectoring process results in core 2 executing from the boot ROM as long as the [RCU_SVECT2](#) register contained the default reset value during the soft vector processing. The operations the core performs varies depending on the product.

In the ADSP-SC59x derivatives, Core 1 is not responsible for the booting of the processor from reset and thus enters a safe IDLE state allowing access to the cores L1 resources by any other core. Core 2 has no entry point into the main boot rom to execute the preboot process or master boot process from reset of the device.

From reset, core 2 can perform the following operations:

- Clear `RCU_MSG.C2IDLE`
- Check for OTPC boot completion.
- Disable Software Return feature in the BTB.
- Disable the BTB.
- Disable cache via `MODE2` register.
- Flush the cache.
- Set `IRPTL` to `0x00000000`.
- Set `RCU_MSG.C2IDLE`
- Enter an endless IDLE loop

Fault Configuration

Describes the initial fault sources that are enabled allowing the processor to signal a fault to the system.

The following faults are enabled via the SEC. Only the faults are enabled, the boot process does not install any SEC interrupts.

Table 41-3: Initial Faults installed during Preboot

SEC Fault ID	SEC Fault Name	Description
0	<code>INTR_SEC0_ERR</code>	SEC Error
3	<code>INTR_WDOG0_EXP</code>	WDOG Expire
4	<code>INTR_WDOG1_EXP</code>	WDOG Expire
6	<code>INTR_OTPC0_ERR</code>	OTPC Expire
10	<code>INTR_L2CTL0_ECC_ERR</code>	L2 ECC Error
76	<code>INTR_SOFT3_INT</code>	Software Driven Interrupt 3. This is raised in the boot ROM error handler should it be entered.

Table 41-3: Initial Faults installed during Preboot (Continued)

SEC Fault ID	SEC Fault Name	Description
190	INTR_CRC0_ERR	CRC Error
191	INTR_CRC1_ERR	CRC Error
194	INTR_MDMA1_SRC_ERR	MDMA 1 Source DMA Channel Error
195	INTR_MDMA1_DST_ERR	MDMA 1 Destination DMA Channel Error

The SEC is configured for a fault delay of 0x100 via the SEC_FDLY.COUNT and SEC_FSRDLY.COUNT bit fields. This allows for a delay before the fault assertion is a custom error handler is installed and any SEC interrupts are enabled and handled for more advanced second stage boot scenarios.

The SYSFAULT pin is configured via the SEC to support both incoming and outgoing faults by enabling the SEC_FCTL.FIEN and SEC_FCTL.FOEN bits. The boot process captures an incoming fault if that fault is asserted by the external system on handover to the user application after boot.

NOTE: The installation of the fault sources is enabled by default and may be optionally bypassed using the RCU_BCODE.NOFAULTS bit.

NOTE: In the ADSP-SC594, ADSP-SC592 and ADSP-21594 processors the SYSFAULT pin is dedicated. However, in the ADSP-21593 and ADSP-21591 processors, the SYSFAULT appears on the PC_07 in pin mux. For specific boot modes in the ADSP-21593 and ADSP-21591 processors, the SYSFAULT pin is not available for fault indication due to this pin being used by the booting peripheral.

L2 Controller Configuration

The L2 controller is configured to ensure all L2 memory banks are ECC enabled and that the L2 memory scrub feature is disabled.

NOTE: The L2 Controller configuration is enabled by default and may be optionally bypassed by setting the RCU_BCODE.NOL2CONFIG bit.

L2 Memory Initialization

The L2 memory subsystem is ECC protected by default. The L2 memory must be initialized to ensure that no read from the L2 memory generates an ECC error. All L2 memory banks are initialized using the L2CTL_INIT bit. The boot process waits for each memory bank to signal initialization complete using the L2CTL_ISTAT bit. The RCU_MSG.L2INIT is set when L2 memory initialization is complete.

NOTE: The L2 memory initialization process is enabled by default and may be optionally bypassed by setting the RCU_BCODE.NOMEMINIT bit.

Idle On Entry

The Idle On Entry feature instructs the debugger to halt the boot code before continuing with any further preboot operations.

When this feature is enabled the processor executes a WFI/IDLE instruction and then continues once an event (such as an emulator exception) is serviced.

NOTE: Idle On Entry processing is disabled by default and is enabled by setting the `RCU_BCODE.IDLEONENTRY` bit prior to performing a system reset operation.

SPU Configuration

The SPU is configured differently depending upon the detected security state of the device. The first operation clears any existing security violations that may be indicated in the `SPU_STAT` register.

None of the peripherals are configured to ignore security signals and only the following masters are configured to generate secure transactions.

Table 41-4: SPU Secure Masters during Boot

SPU Endpoint ID	Master Name
146	MDMA0 Source DMA Channel
147	MDMA0 Destination DMA Channel
148	MDMA1 Source DMA Channel
149	MDMA1 Destination DMA channel
144	CRC0
145	CRC1
175	PKTE

SMPU Configuration

The SMPU is used to restrict access to various memory regions in the processor. The configuration applied during boot differs depending on the locked state of the processor.

By default the SMPU instances only allow secure read and write transactions. The *SMPU Configuration* table describes the configurations for the different security states.

Speculative reads are also disabled by setting the `SMPU_CTL.RSDIS` bit.

Table 41-5: SMPU Configuration

SMPU Instance	SMPU Instance	Open <code>SMPU_SECURECTL</code> Value	Locked <code>SMPU_SECURECTL</code> Value
SMC	0	<code>SMPU_SECURECTL.WNSEN</code> <code>SMPU_SECURECTL.RNSEN</code>	0x00000000 (Default Reset Value)
Core_L2_RAM_Boot_ROM0	2	<code>SMPU_SECURECTL.WNSEN</code> <code>SMPU_SECURECTL.RNSEN</code>	0x00000000 (Default Reset Value)
DMA_L2_RAM_Boot_ROM0	3	<code>SMPU_SECURECTL.WNSEN</code> <code>SMPU_SECURECTL.RNSEN</code>	0x00000000 (Default Reset Value)

Table 41-5: SMPU Configuration (Continued)

SMPU Instance	SMPU Instance	Open SMPU_SECURECTL Value	Locked SMPU_SECURECTL Value
DMC	9	SMPU_SECURECTL.WNSEN SMPU_SECURECTL.RNSEN	0x00000000 (Default Reset Value)
SPI	11	SMPU_SECURECTL.WNSEN SMPU_SECURECTL.RNSEN	0x00000000 (Default Reset Value)
Core_L2_ROM1_Boot_ROM1	4	SMPU_SECURECTL.WNSEN SMPU_SECURECTL.RNSEN	0x00000000 (Default Reset Value)
Core_L2_ROM2_Boot_ROM2	6	SMPU_SECURECTL.WNSEN SMPU_SECURECTL.RNSEN	0x00000000 (Default Reset Value)
DMA_L2_ROM1_Boot_ROM1	5	SMPU_SECURECTL.WNSEN SMPU_SECURECTL.RNSEN	0x00000000 (Default Reset Value)

Secure Debug Key Processing

In the event the processor is locked, the debug tools must submit a secure debug key that is matched with a key on the processor. A 128-bit Secure Debug Key must be provided the application program prior to locking the device.

The secure debug key is read from the OTP memory and then written to the corresponding register in the TAPC. After the key has been written the `TAPC_SDBGKEY_CTL.VALID` bit is set. Once the debug tools then submit their key a key compare operation is performed.

Debug tools must wait for the boot software to load the key before setting the `TAPC_SDBGKEY_CTL.VALID` bit before submitting the key for comparison.

The 128-bit Secure Debug Key is loaded as follows from the storage area in OTP.

Table 41-6: Secure Debug Key Load Procedure

Secure Debug Key[127:0]	Register
Secure Debug Key[31:0]	TAPC_SDBGKEY0
Secure Debug Key[63:32]	TAPC_SDBGKEY1
Secure Debug Key[95:64]	TAPC_SDBGKEY2
Secure Debug Key[127:96]	TAPC_SDBGKEY3

NOTE: In the ADSP-SC59x processor family, the boot ROM supports two 128-bit secure debug keys, `secure_emu_key0` and `secure_emu_key1`, where only one of them is used by the boot ROM at a time. Each of the secure debug keys is associated with a bypass key (`emu_key0_disable` or `emu_key1_disable`) which decides the validity of the corresponding key. Any non zero value written into the 16-bit bypass key will invalidate that particular secure debug key.

The validity of the secure debug keys can be determined as shown in the *Secure Debug Key Validation* table:

Table 41-7: Secure Debug Key Validation

Bypass Key 0 (emu_key0_disable) Value	Bypass Key 1 (emu_key1_disable) Value	Valid Secure Debug Key
Zero	Zero	secure_emu_key0
Zero	Non-zero	secure_emu_key0
Non-zero	Zero	secure_emu_key1
Non-zero	Non-zero	Both keys are valid

CAUTION: The boot code bypasses the key load operation entirely when a 16-bit non-zero value is programmed on both `emu_key0_disable` and `emu_key1_disable` fields in OTP *or* a key of 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF is programmed in both of the secure debug key fields provided in OTP. When debug access is subsequently needed, the program must load the key to the TAPC. If the processor fails to boot (due to corrupted firmware) then there is no debug access. The only way to gain access is to load an authenticated boot image that can then load the required keys prior to attempting to connect with a debugger.

CGU Configuration

This step in the preboot process reconfigures the internal clocks on the processor for improved boot performance.

The boot process can optionally configure the CGU in order to improve boot performance. The settings for the CGU are located within the `struct ADI_ROM_OTP_BOOT_CGU_INFO` structure that has storage allocated in the OTP as part of the `struct ADI_ROM_OTP_BOOT_INFO` structure.

Typically, CGU configuration is performed using an [Init Block](#) in the boot stream. This provides greatest flexibility. In situations where boot time must be kept to a minimum, settings in the OTP can be applied at this stage of preboot as opposed to during the boot process itself. When a processor is locked, the boot process does not support an [Init Block](#) in the boot stream. For a locked processor programs must use the OTP to reconfigure the clocks without adopting a multi-stage boot strategy.

On release from reset, the CGU is configured for PLL Bypass mode. To improve boot performance, the boot software reconfigures the CGU so that Full-On mode is entered with the default CGU settings.

If the program doesn't provide settings in OTP to configure the Oscillator Watchdog, then the Oscillator Watchdog fault, enabled by default after reset, is disabled prior to reconfiguring the CGU.

If settings are provided in the OTP to configure the Oscillator Watchdog, the fault is left enabled and, if applicable, the CGU is configured per the provided settings. The remainder of the boot process completes in Full-On mode.

The CGU configuration object (see `struct ADI_ROM_OTP_BOOT_CGU_INFO`) allows the boot software to configure the CGU for a more efficient boot process.

Table 41-8: ADI_ROM_OTP_BOOT_CGU_INFO Members

Type	Name	Description
	<code>ctl_WEN:1</code> (bitfield)	Enable write to the <code>CGU_CTL</code> register

Table 41-8: ADI_ROM_OTP_BOOT_CGU_INFO Members (Continued)

Type	Name	Description
uint32_t		
uint32_t	div_WEN:1 (bitfield)	Enable write to the CGU_DIV register
uint32_t	reserved0:1 (bitfield)	Reserved
uint32_t	div_DSEL:5 (bitfield)	CGU_DIV.DSEL value
uint32_t	div_CSEL:5 (bitfield)	CGU_DIV.CSEL value
uint32_t	div_S0SEL:3 (bitfield)	CGU_DIV.S0SEL value
uint32_t	div_SYSSEL:5 (bitfield)	CGU_DIV.SYSSEL value
uint32_t	div_S1SEL:3 (bitfield)	CGU_DIV.S1SEL value
uint32_t	div_OSEL:7 (bitfield)	CGU_DIV.OSEL value
uint32_t	ctl_DF:1 (bitfield)	CGU_CTL.DF value
uint32_t	ctl_MSEL:7 (bitfield)	CGU_CTL.MSEL value
uint32_t	auto_disable:1 (bitfield)	Disable polling on auto-alignment of clocks (not recommended)
uint32_t	Reserved1:6 (bitfield)	Reserved
uint32_t	clkoutsel_CLKOUTSEL:5 (bitfield)	CGU_CLKOUTSEL.CLKOUTSEL value
uint32_t	clkoutsel_WEN:1 (bitfield)	Enable write to the CGU_CLKOUTSEL register
uint32_t	Reserved2:12 (bitfield)	Reserved
uint32_t	oscwctl0_WEN:1 (bitfield)	Enable write to the CGU_OSCWDCTL instance 0 register
uint32_t	oscwctl0_HODF:6 (bitfield)	CGU_OSCWDCTL.HODF value
uint32_t	oscwctl0_HODEN:1 (bitfield)	CGU_OSCWDCTL.HODEN value
uint32_t	oscwctl0_CNGEN:1 (bitfield)	CGU_OSCWDCTL.CNGEN value
uint32_t	oscwctl0_BOUF:5 (bitfield)	CGU_OSCWDCTL.BOUF value
uint32_t	oscwctl0_BOUEN:1 (bitfield)	CGU_OSCWDCTL.BOUEN value
uint32_t	oscwctl0_FAULTEN:1 (bitfield)	CGU_OSCWDCTL.FAULTEN value
uint32_t	oscwctl0_MONDIS:1 (bitfield)	CGU_OSCWDCTL.MONDIS value
uint32_t	oscwctl0_FAULTPINDIS:1 (bitfield)	CGU_OSCWDCTL.FAULTPINDIS value
uint32_t	reserved3:14 (bitfield)	Reserved
uint32_t	oscwctl1_WEN:1 (bitfield)	Enable write to the CGU_OSCWDCTL instance 1 register
uint32_t	oscwctl1_HODF:6 (bitfield)	CGU_OSCWDCTL.HODF value
uint32_t	oscwctl1_HODEN:1 (bitfield)	CGU_OSCWDCTL.HODEN value
uint32_t	oscwctl1_CNGEN:1 (bitfield)	CGU_OSCWDCTL.CNGEN value

Table 41-8: ADI_ROM_OTP_BOOT_CGU_INFO Members (Continued)

Type	Name	Description
uint32_t	oscwctl1_BOUF:5 (bitfield)	CGU_OSCWDCTL.BOUF value
uint32_t	oscwctl1_BOUEN:1 (bitfield)	CGU_OSCWDCTL.BOUEN value
uint32_t	oscwctl1_FAULTEN:1 (bitfield)	CGU_OSCWDCTL.FAULTEN value
uint32_t	oscwctl1_MONDIS:1 (bitfield)	CGU_OSCWDCTL.MONDIS value
uint32_t	oscwctl1_FAULTPINDIS:1 (bitfield)	CGU_OSCWDCTL.FAULTPINDIS value
uint32_t	Reserved4:14 (bitfield)	Reserved

If a `CGU_STAT.WDIVERR`, `CGU_STAT.WDFMSERR`, `CGU_STAT.LWERR` or `CGU_STAT.ADDRERR` occurs at the entry to or completion of the configuration routine, the default error handler is called and the boot process terminates.

NOTE: Programs can bypass CGU configuration by setting the `RCU_BCODE.NOPREBOOT` bit when this part of the boot process is reached.

Releasing All Cores from Reset

The master booting core releases all other cores from reset allowing them to then run by default into a safe endless loop state. By releasing all other cores from reset, any dedicated core L1 memories become accessible to all cores via the system address space. In order for one core to load or read data from dedicated L1 memory of another core, the core must be released from the reset state. By default, the other cores in the processor execute a safe endless loop in the boot ROM.

L1 Memory Initialization

The processor initializes all parity and ECC protected memories to perform subsequent read operations without generating an ECC or parity error.

The *L1 Memory Initialization* table describes the methods used to initialize the various parity and ECC supported memories on the processor.

Table 41-9: L1 Memory Initialization

Resource To Fill Memory	Memory Type	Address	Count	Fill Value	Flag Set Upon Completion
Core 1	Core 1 L1 Bank 0	0x00048000	0x6000 (LW)	0x00000000	RCU_MSG.C1L1INIT
Core 1	Core 1 L1 Bank 1	0x00058000	0x6000 (LW)	0x00000000	
Core 1	Core 1 L1 Bank 2	0x00060000	0x4000 (LW)	0x00000000	
Core 1	Core 1 L1 Bank 3	0x00070000	0x4000 (LW)	0x00000000	

Table 41-9: L1 Memory Initialization (Continued)

Resource To Fill Memory	Memory Type	Address	Count	Fill Value	Flag Set Upon Completion
Core 2	Core 2 L1 Bank 0	0x00048000	0x6000 (LW)	0x00000000	RCU_MSG.C2L1INIT
Core 2	Core 2 L1 Bank 1	0x00058000	0x6000 (LW)	0x00000000	
Core 2	Core 2 L1 Bank 2	0x00060000	0x4000 (LW)	0x00000000	
Core 2	Core 2 L1 Bank 3	0x00070000	0x4000 (LW)	0x00000000	

NOTE: The Memory Initialization process is enabled by default and may be optionally bypassed by setting the `RCU_BCODE.NOMEMINIT` bit.

Default Entry Point

The first instruction executed for the core in the boot ROM is a read of the `RCU_SVECT0` register. The boot ROM then vectors to that location.

The *Default Entry Point* table defines the default entry point for each core in the processor

Table 41-10: Default Entry Point

Core ID	Register	Entry Point
0	<code>RCU_SVECT0</code>	0x00000000
1	<code>RCU_SVECT1</code>	0x00500004
2	<code>RCU_SVECT2</code>	0x00500004

The boot code does not set any initial default application entry points by writing the entry point to the `RCU_SVECT0` register in the processor. It will hold the default values of the power on reset case. The `RCU_SVECT0` register is updated at the end of the boot process. For the secure boot, the register is updated only after the successful authentication of the boot stream.

NO-BOOT Processing

No-Boot mode is executed when selected by the `SYS_BMODE[n]` pins. The boot mode is intended as a recovery boot mode or for debug purposes. The core simply executes in an endless loop in the boot ROM, terminating further execution of the boot process.

This boot mode is primarily intended for debug sessions when no boot source may be configured. It allows for a debugger to safely connect to the device and take control (assuming the debugger has been granted access rights as defined by the processor security implementation).

NOTE: NO-BOOT processing is usually only entered as a result of the boot mode pin sampling resulting in execution of the No-Boot boot mode. This processing can also be optionally enabled by setting `RCU_BCODE.HALT` field. The `RCU_BCODE.HALT` setting can be especially useful for debug sessions to force the execution of the NO-BOOT mode regardless of the `SYS_BMODE[n]` state allowing a user

application to be loaded via the debug tools without fear of the image being corrupted as a result of attempting to boot through another source.

SYS_RESOUT Signal

In order to signal to the external system that the processor is in a configured state and ready to start the boot process, the boot software de-asserts the `SYS_RESOUT` pin via the `RCU_CTL.RSTOUTDSRT` bit.

DMC Configuration

To boot to external DDR memories (to support booting to external memory when the processor is locked) the DMC must be configured.

Typically, DMC configuration is done using the [Init Block](#) in the boot stream. When the processor is locked the boot process does not support an [Init Block](#) in the boot stream. For a locked processor programs must use the OTP to configure the DMC without adopting a multi-stage boot strategy.

[Table 41-61 ADI_ROM_OTP_DMC_CONFIG Members](#) provides details of the OTP region that is used to store the DMC configuration. In the table, the `ADI_ROM_OTP_BOOT_CFG::dmcEn` must be set in order for the settings to apply.

Table 41-11: ADI_ROM_OTP_DMC_CONFIG Members

Type	Name	Description
uint32_t	ulDDR_DLLCTLCFG:16 (bit field)	Content of DMC CFG[15:0]
uint32_t	ulDDR_DLLCTLCFG:16 (bit field)	Content of DMC DLL CTL[15:0]
uint32_t	ulDDR_EMR2EMR3:16 (bit field)	Contents of DMC_EMR3 [15:0]
uint32_t	ulDDR_EMR2EMR3:16 (bit field)	Contents of DMC_EMR2 [15:0]
uint32_t	ulDDR_CTL	Content of DMC CTL[31:0]
uint32_t	ulDDR_MREMR1:16 (bit field)	Content of DMC EMR1[15:0]
uint32_t	ulDDR_MREMR1:16 (bit field)	Content of DMC MR[15:0]
uint32_t	ulDDR_TR0	Content of DMC_TR0[31:0]
uint32_t	ulDDR_TR1	Content of DMC_TR1[31:0]
uint32_t	ulDDR_TR2	Content of DMC_TR2[31:0]
uint32_t	ulDDR_ZQCTL0	Content of DMC PHY ZQCTL0[31:0]
uint32_t	ulDDR_ZQCTL1	Content of DMC PHY ZQCTL1[31:0]
uint32_t	ulDDR_ZQCTL2	Content of DMC PHY ZQCTL2[31:0]
uint32_t	ulDDRPHY_CACTL	Content of DMC PHY CA_CTL [31:0]
uint32_t	uBypassDelay_LANE0CTL1:6 (bit field)	Content of DMC LANE0_CTL1[15:10]
uint32_t	uBypassDelay_LANE1CTL1:6 (bit field)	Content of DMC LANE1_CTL1[15:10]

Table 41-11: ADI_ROM_OTP_DMC_CONFIG Members (Continued)

Type	Name	Description
uint32_t	uBypassDelay_LANE0CTL0:6 (bit field)	Content of DMC LANE0_CTL0[15:10]
uint32_t	uBypassDelay_LANE1CTL0:6 (bit field)	Content of DMC LANE1_CTL0[15:10]
uint32_t	reserved0:8 (bit field)	Reserved

There is an additional single bit located in OTP, `ADI_ROM_OTP_BOOT_CFG::dmcInv` allowing programs to invalidate the DMC settings stored in OTP.

NOTE: Once `ADI_ROM_OTP_BOOT_CFG::dmcInv` has been set in OTP there is no means to configure the DMC during the preboot phase.

The configuration of the DMC is bypassed if the `RCU_BCODE.NOPREBOOT` bit is set when this part of the boot process is reached.

Bypassing the Boot Process

The boot process can be bypassed allowing a program to start execution from the address stored in the core's soft vector register. This is useful when working in emulation sessions as it provides a mechanism to be able to execute programs directly from an accessible memory that already contains executable code.

To bypass the boot process, set the `RCU_BCODE.NOKERNEL` bit. The processor core vectors to the address stored in the core's corresponding `RCU_SVECT0` / `RCU_SVECT1` / `RCU_SVECT2` register instead of calling the required boot mode.

This feature is used along with other features such as disabling memory initialization.

Boot Mode Disable

Specific boot modes can be permanently disabled via OTP. If the disabled boot mode is enabled using the `SYS_BMODE[n]` pins, a boot error is generated.

A byte of storage is provided in the OTP to disable up to eight boot modes. The boot mode disable field can be programmed using the `adi_rom_otp_pgm()` routine. The `otp_data::bootModeDisable` member is used in the program operation to disable the various boot modes.

Table 41-12: Boot Mode Disable

<code>otp_data::bootModeDisable</code> Bit Position	Corresponding Boot Mode
0	SPI Master Boot Mode
1	SPI Slave Boot Mode
2	UART Slave Boot Mode
3	Linkport Slave Boot Mode

Table 41-12: Boot Mode Disable (Continued)

otp_data::bootModeDisable Bit Position	Corresponding Boot Mode
4	OSPI Master Boot Mode
7–5	Reserved

Boot Command Customization

Boot command customization allows permanent customization of a particular boot mode. For example, it is possible to change the peripheral instance used for boot operation.

Storage is provided in OTP for a command item for each supported boot mode.

Storage is provided in the `struct ADI_ROM_OTP_BOOT_CMD_INFO` member of `struct ADI_ROM_OTP_BOOT_INFO`. Refer to the corresponding boot modes boot command description for details on supported command options.

NOTE: Before programming boot command to the OTP, evaluate the boot command using the `adi_rom_Boot()` API and ensure that the boot command provides the desired functionality. Once the command is programmed to OTP, it cannot be reverted to original default settings.

Boot Mode Specific SPU Configuration

Prior to performing actual boot process, the processors SPU resources specific to the boot mode selected are configured. This is performed in the preboot phase as opposed to within the boot mode itself when calling the boot API, as it isolates the security functionality of the processor allowing it to be handled specifically by a separate process.

The following additional SPU resources are configured as secure masters according to the boot mode selected.

Table 41-13: Boot Mode Specific SPU Configuration

Boot Mode	SPU Endpoint ID	Master Name
SPI Master Boot (Memory-Mapped Mode) and OSPI Master Boot	146, 147	MDMA0 Source DMA Channel, MDMA0 Destination DMA Channel
SPI Master Boot (Peripheral Mode)	100, 98, 96, 94	SPI3 Receive DMA, SPI2 Receive DMA, SPI1 Receive DMA, SPI0 Receive DMA
SPI Slave Boot	100, 98, 96, 94	SPI3 Receive DMA, SPI2 Receive DMA, SPI1 Receive DMA, SPI0 Receive DMA
UART Slave Boot	79, 81, 83, 85	UART 0 Receive DMA, UART1 Receive DMA, UART2 Receive DMA, UART3 Receive DMA
LINKPORT Slave Boot	5, 6	LINKPORT 0 DMA, LINKPORT 1 DMA

NOTE: In the *Boot Mode Specific SPU Configuration* table, for a given boot mode, all the SPU resources are not configured. Only a single peripheral instance is enabled according the peripheral instance selected for boot. For example if the boot command for the boot mode indicates boot from UART0 only the UART0 Receive DMA is configured, the other UART Receive DMAs are not configured for secure access.

Executing the Boot Mode

The boot mode is called `adi_rom_Boot()` routine. The routine fetches and processes the boot stream from the configured boot source.

The table provides default parameters passed to each of the supported boot modes. For details on the API usage, refer to `adi_rom_Boot()`.

Table 41-14: Default Boot ROM API Parameters

Boot Mode	pAddress	flags	blockCount	pHook	Command
No Boot	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x00000000
SPI Master Boot	0x60000000	0x00040000	0x00000000	Points to an empty routine in ROM	0x0000207
SPI Slave Boot	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x0000212
UART Slave Boot	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x0000013
LINKPORT Slave Boot	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x0000014
OSPI Master Boot	0x60000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x0000008
Reserved	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x00000000
Reserved	0x00000000	0x00000000	0x00000000	Points to an empty routine in ROM	0x00000000

The hook function installed via pHook on this product does not perform any additional configuration.

Boot Modes

The boot implementation provides built-in support for booting from various peripherals.

The *Booting Modes* table describes the supported boot modes.

In slave boot mode, the processor functions as a slave to any host device. In these modes, the host device usually controls the processor `SYS_HWRST` input. Typically, the host applies the reset sequence and waits until the processor is ready to boot, depending on the peripheral in use, and transmits the boot stream data to the processor. Handshake signals are used to signal to the host that the processor is ready to accept more data.

In master boot mode, the processor controls the peripheral and requests data via the peripheral as and when required.

Individual boot modes can be disabled. For more information about disabling boot modes, see [Boot ROM OTP Customizations](#).

Table 41-15: Booting Modes

SYS_BMODE[2:0]	Boot Source	Description
000	No Boot	The processor does not boot. Rather the boot kernel executes some of the preboot operations then enters an endless WFI/IDLE state.
001	SPI Master Boot	Boot from integrated Flash memory through the SPI2 peripheral configured for memory-mapped mode.
010	SPI Slave Boot	Boot through the SPI2 peripheral configured as a slave.
011	UART Boot	Boots through UART0 configured as a slave receiver.
100	LINKPORT Boot	Boot through LINKPORT0 peripheral configured as a slave receiver. This boot mode is only applicable to derivatives supporting the LP0 instance.
101	OSPI Master Boot	Boot from integrated Flash memory through the OSPI peripheral configured for memory-mapped mode.
110, 111	Reserved	

No-Boot Mode

No-Boot mode is intended for device recovery purposes caused due to incorrect programming of the boot source memory allowing for target connection through an emulator. Emulation tools can also leverage the No-Boot functionality allowing for debug sessions to run the preboot software prior to loading an application while preventing the boot process from continuing and clobbering data loaded by the emulator.

This boot mode results in several preboot operations being performed before placing the core in a safe, endless loop located in the boot ROM.

For a complete list of operations performed when No-Boot is selected, see [Preboot Operations](#). The core terminates at the [NO-BOOT Processing](#) stage of the preboot process.

SPI Master Boot Mode

The SPI master boot routine provides support for booting the processor from SPI flash memories. The SPI boot mode uses a device auto-detection feature that is enabled by default. This lets the boot stream itself instruct updates to the SPI configuration and the read command used allowing for more efficient transactions.

NOTE: For default SPI master mode, the peripheral fault, SPI2_ERR with fault ID 132 is enabled.

Boot From External SPI Flash Devices

The SPI boot mode supports booting from 24-bit or 32-bit addressable flash devices. The boot mode uses the MDMA channels by default and configures the SPI flash for memory-mapped functionality. Peripheral DMA mode is also supported when calling the boot mode via `adi_rom_Boot()`.

When auto-device detection is enabled, the SPI memory is initially read using the standard 0x03 SPI read command with a reduced clocking frequency for maximum compatibility. The first nibble of the boot stream is then used to reconfigure the SPI interface and possible the SPI flash. Refer to [SPI Device Detection Routine](#).

NOTE: Support for automatic device detection via the first nibble of the boot stream is not supported when booting secure boot streams. Instead when signing the boot image an attribute can be set in the image header that specifies the configuration to use.

For booting, the SPI memory is connected as shown in the *SPI Memory Connections* figure.

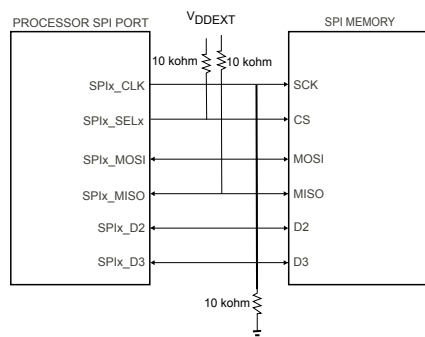


Figure 41-3: SPI Memory Connections

The pull-up resistor on the slave select signal ensures that the memory is deselected when the pin is in a high-impedance mode such as during reset.

Initialization codes are allowed to manipulate the `ADI_ROM_BOOT_CONFIG::dBootCommand` to extend boot mechanism to a second SPI memory connected to another slave select pin. Updating the field that specifies the slave select signal for use, allows the boot process to manage larger boot streams than fit into a single SPI device.

NOTE: If modifying the slave select signal used during the boot process, configure the pin multiplexing to enable the correct functionality for the pin. Once the boot process has proceeded past the configuration function and the boot process has actually started, the boot kernel will not perform any further pin multiplexing operations.

SPI Device Detection Routine

Since the boot mode supports booting from various SPI memories, the boot kernel automatically detects what type of memory is connected. To determine whether the SPI memory device requires a 24-bit or 32-bit addressing scheme, the boot kernel performs a device detection sequence prior to booting. The `SPI_MISO` signal requires a pull-up resistor. The routine relies on the fact that memories do not drive their data outputs unless the right number of address bytes are received.

Initially, the boot kernel transmits the read command on the `SPI_MOSI` line. Once the command has been sent, the boot kernel proceeds to transmit a single address byte and waits until the receive FIFO indicates that the buffer is no longer empty. The first received byte is discarded. The boot code then proceeds to issue another address byte while simultaneously receiving a byte. The process continues until a non 0xFF or 0x00 byte is received or until the full 4 address bytes is sent without any valid data being returned.

The receiving of a non 0x00 or 0xFF byte tells the boot code whether the memory device requires 24 or 32 address bits. The lower nibble of the received byte is then used to further customize the boot mode. This nibble is referred

to as the BCODE. The boot code applies settings to the SPI peripheral according to the *SPI Master Boot BCODE Descriptions*.

If the received value equals 0x00 or 0xFF, it is assumed that the memory device has not driven its data output thus, another zero byte is transmitted and the received data is tested again.

If the value still equals 0xFF, device detection continues. Device detection aborts immediately when a byte different than 0xFF is received. The boot process continues with normal boot operation and it reissues a command to again read from address 0. Two read sequences load the first block header. Separate read sequences load further block headers and block payload fields.

The *SPI Device Detection Principle* figure illustrates how individual devices behave.

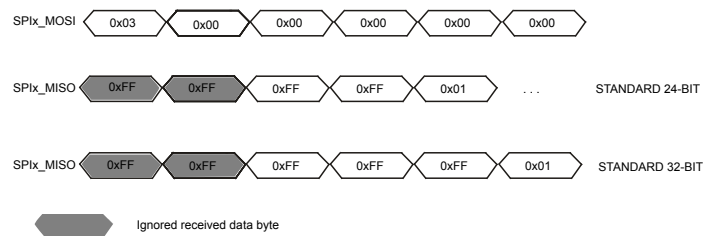


Figure 41-4: SPI Device Detection Principle

Table 41-16: SPI Master BCODE Configuration Lookup Table

Member	BCODE				
	0	1	2	3	4
Transfer Type	Single bit command, address and data	Single bit command, address and data	Single bit command, address and data	Single bit command, address and dual bit data	Single bit command, address and Quad data
ubDummyBytes	0x00	0x00	0x01	0x01	0x01
ubReadCommand	0x03	0x03	0x0B	0x3B	0x6B
ubDataBits	0x00	0x00	0x00	0x01	0x02
ubAddressBytes	0x03	0x03	0x03	0x03	0x03
uwClkLower	0x000F	0x000F	0x0001	0x0001	0x0001
uReserved0	0x0000	0x0000	0x0000	0x0000	0x0000
nTxCtl	0x00000003	0x00000003	0x00040003	0x00040033	0x00040033
nRxCtl	0x00000003	0x00000003	0x00040003	0x00140003	0x00240033
nCmdCtl	0x00000003	0x00000003	0x00000003	0x00000033	0x00000033
pMIOEnFunction	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
nDummy	0x00	0x00	0x00	0x00	0x00

Table 41-17: SPI Master BCODE Configuration Lookup Table

Member	BCODE					
	5	6	7	8	9	A
Transfer Type	Single bit command, Dual bit address and data	Single bit command, Dual bit address and data	Single bit command, Dual bit address and data	Single bit command, Quad bit address and data	Single bit command, Quad bit address and data	Single bit command, Quad bit address and data
ubDummyBytes	0x01	0x02	0x03	0x02	0x03	0x05
ubReadCommand	0xBB	0xBB	0xBB	0xEB	0xEB	0xEB
ubDataBits	0x01	0x01	0x01	0x02	0x02	0x02
ubAddressBytes	0x03	0x03	0x03	0x03	0x03	0x03
uwClkLower	0x0001	0x0001	0x0001	0x0001	0x0001	0x0001
uReserved0	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
nTxCtl	0x00140033	0x00140033	0x00140033	0x00240033	0x00240033	0x00240033
nRxCtl	0x00140033	0x00140033	0x00140033	0x00240033	0x00240033	0x00240033
nCmdCtl	0x00000033	0x00000033	0x00000033	0x00000033	0x00000033	0x00000033
pMIOEnFunction	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
nDummy	0x00	0x00	0x00	0x00	0x00	0x00

NOTE: For the above configurations, the addressing scheme can be 3-bytes or 4-bytes depending on the addressing of flash detected in auto-detection. The SPI mode byte issued for all the SPI master peripheral based configurations is 0x00. The mode byte is the first byte transmitted after the address cycles and is used to control the continuous read mode functionality in which the next read operation is not required to issue a command cycle. Continuous read mode is not supported during the boot process.

Supported Quad Mode Enable Methods

The boot ROM does not support enabling quad mode on the SPI flash device. To boot in quad mode, the flash device must be configured outside the boot ROM.

NOTE: It is more beneficial to boot initially in dualmode and use an initcode to enable Quad mode.

Run-Time API

The following table provides descriptions of the [adi_rom_Boot\(\)](#) command parameter.

Table 41-18: SPI Master Boot Command Descriptions

Bits	Name	Setting	Description
31:28	ROM_BCMD_SPIM_SPEED	0 to 1111	SPI clock divider. This value written to the SPI peripherals clock divider register
27	Reserved	Reserved	Reserved
26:22	ROM_BCMD_SPIM_DUMMY	00000	No dummy bytes required after the address
		00001	1 dummy byte required after the address
			
		11111	31 dummy bytes required after the address
21:20	ROM_BCMD_SPIM_ADDR	00	Flash device requires an 8-bit address
		01	Flash device requires a 16-bit address
		10	Flash device requires a 24-bit address
		11	Flash device requires a 32-bit address
19:16	ROM_BCMD_SPIM_BCODE	0000 to 1111	Boot mode-specific code. Specifies the boot mode-specific code that can further customize and control the boot process.
15	ROM_BCMD_SPIM_CMD	0	Issue the read command over the single bit bus
		1	Issue the read command over the multi-bit bus. Not recommended for use in ADSP-2159x processor.
14:12	ROM_BCMD_SPIM_SSEL	000	Use slave select 1 for SPI chip select
		001	Use slave select 2 for SPI chip select
		010	Use slave select 3 for SPI chip select
		011	Reserved
		100	Reserved

Table 41-18: SPI Master Boot Command Descriptions (Continued)

Bits	Name	Setting	Description
		101	Reserved
		110	Reserved
		111	Reserved
11:8	ROM_BCMD_SPIM_DEVENUM	0 to 16	Boot peripheral enumeration. So, for SPI3 it is set to 3, for SPI2 it is set to 2, for SPI0 it is set to 0. 0x4 to 0xF is reserved.
7	Reserved	Reserved	Reserved
6	ROM_BCMD_SPIM_NOAUTO	0	Automatic device detection disable. Perform automatic device detection and peripheral configuration based on the BCODE value (first nibble) of the boot streams block header
		1	Do not perform automatic device detection
5	ROM_BCMD_SPIM_NOCFG	0	Device configuration enable. Instructs the config routine to perform pinmuxing configuration and full peripheral configuration
		1	Device configuration disable.
4	ROM_BCMD_SPIM_HOST	0	Master boot mode enable.
		1	Slave boot mode enable.
3:0	ROM_BCMD_SPIM_DEVICE	0000	Reserved
		0001	Reserved
		0010	SPI Boot (Legacy peripheral DMA)
		0011	Reserved
		0100	Reserved
		0101	Reserved
		0110	Reserved
		0111	SPI Memory Mapped Boot
		1000 to 1111	Reserved

NOTE: All bits in the above table that are not defined must be set to zero. Supported features may be limited depending on peripheral instance.

SPI Slave Boot Mode

When using SPI slave mode boot, the processor consumes boot data from an external SPI host device. This mode supports single, dual, and quad-bit modes. The boot kernel always starts in single bit mode and can be changed using the appropriate command. The following figures show the hardware configuration for the modes. As in all slave boot modes, the host device controls the `SYS_HWRST` input of the processor.

NOTE: *Secure Boot Stream Padding*

For slave boot modes, the host must always send data in multiples of 1024 bytes. This requirement is due to the sizing of internal buffers used for DMA.

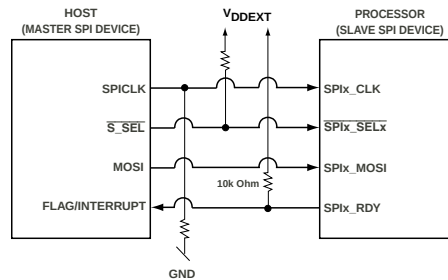


Figure 41-5: Connection between Host (SPI Master) and Processor (SPI Slave)

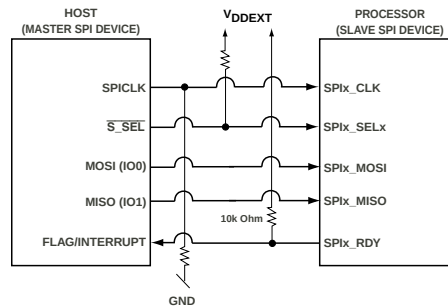


Figure 41-6: Connection between Host (SPI Master) and Processor (SPI Slave) DIOM

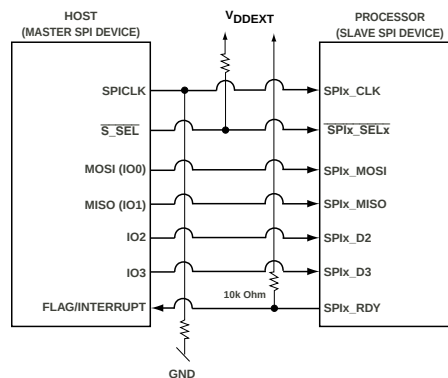


Figure 41-7: Connection between Host (SPI Master) and Processor (SPI Slave) QSPI

The host drives the SPI clock and is responsible for timing. The host must provide an active-low chip select signal that connects to the processor's `SPlx_SS` input signal with each byte transferred or remain low during the entire procedure. 8-bit data is expected and 16-bit mode is not supported.

In SPI slave boot mode, the boot kernel sets the `SPI_CTL.CPHA` bit and clears the `SPI_CTL.CPOL` bit. Therefore the `SPI_MISO` pin is latched on the falling edge of the `SPI_MOSI` pin.

The SPI slave processor detects the correct boot mode from the host SPI device by reading the first byte sent, defined as `SPICMD`. The *SPICMD Descriptions* table describes the available codes. These additional bytes must be sent prior to transmitting the data to configure the SPI device.

The *SPICMD Descriptions* table describes the following:

- Host starting in single bit mode
- Host starting in a mode other than single bit

Table 41-19: SPICMD Descriptions

SPICMD	Description
<i>If host starts in Single bit Mode</i>	
0x3	Keep single-bit mode
0x7	Switch to dual-bit mode
0xB	Switch to quad-bit mode
<i>If host device starts in DIOM or QSPI</i>	
0xAA,0xBF	Switch to dual-bit mode
0xEE,0xEE,0xFE,0xFF	Switch to quad-bit mode

In SPI slave boot mode, `SPIx_RDY` functionality is critical. The `SPIx_RDY` output is used for back pressure and requires a pulling resistor. The boot code requires the `SPIx_RDY` signal function as active-low. The host is only permitted to transfer data when `SPIx_RDY` is in the active state. This functionality allows the processor to hold off the host while the processor is in reset or executing the pre-boot and processor initialization sequences. The SPI is configured to deassert `SPIx_RDY` when the receive FIFO is filled to 75% or more.

NOTE: For default SPI slave mode, the peripheral fault, `SPI2_ERR` with Fault ID 132 is enabled.

The *SPI Program Flow on the Host Side* figure illustrates the required program flow on the host side.

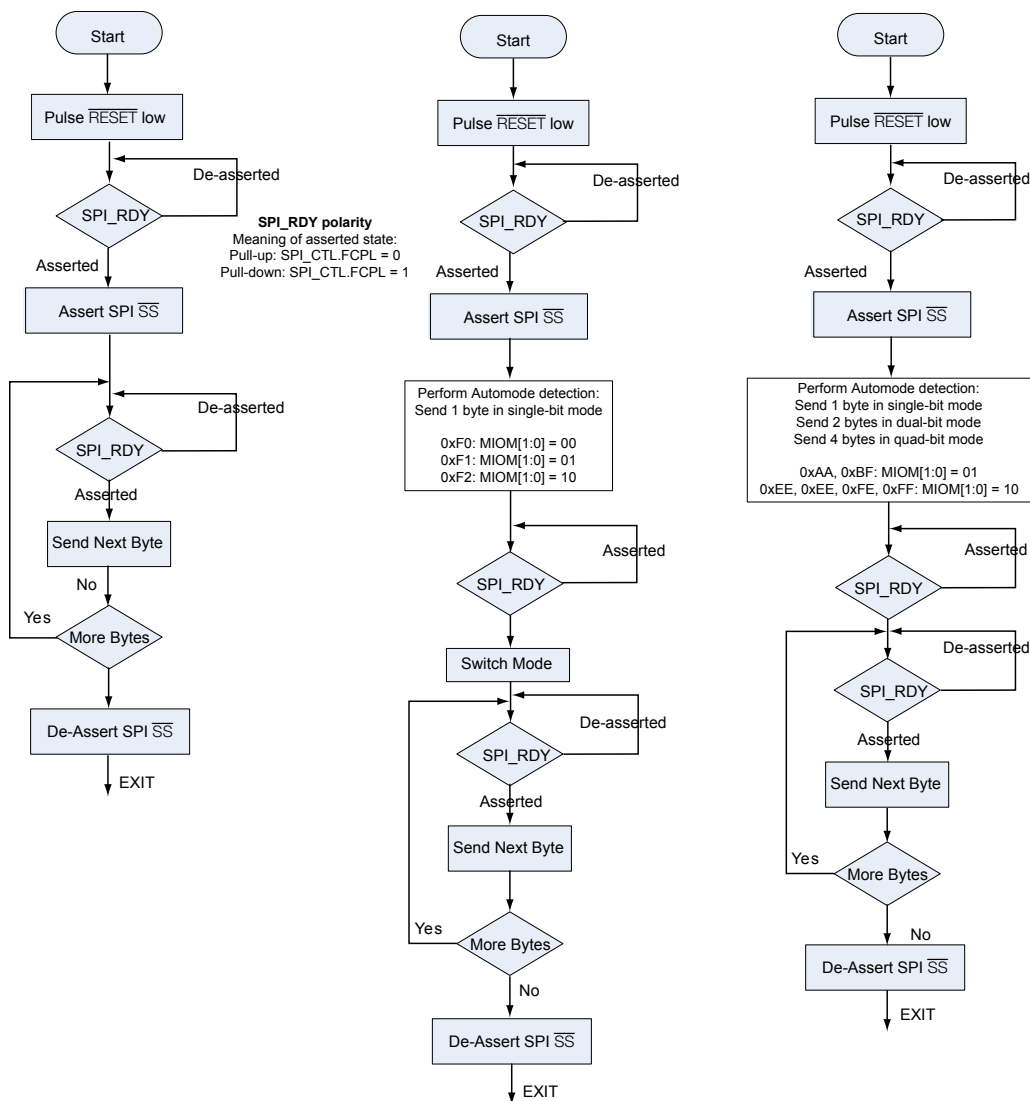


Figure 41-8: SPI Program Flow on the Host Side

Run-time API

The SPI slave boot mode can be called through the Boot Routine API function at run-time. Initiating a boot through the run-time API allows for extra customization such as disabling automatic device configuration or specifying a different SPI device other than the default.

When ROM_BCMD_NOCFG flag is specified, it is necessary to program pin multiplexing and other SPI configuration as required, while keeping the SPI_CTL.EN bit cleared.

The ROM_BCMD_NOAUTO flag can suppress auto mode detection. In that case, the desired configuration must be passed through the ROM_BCMD_SPIS_BCODE bit field, even if the ROM_BCMD_NOCFG flag is set.

The following table provides descriptions of the adi_rom_Boot parameter.

Table 41-20: SPI Slave Boot Command Bit Descriptions

Bit No. (Access)	Bit Name	Description/Enumeration
19:16	ROM_BCMD_S PIS_BCODE	Boot Mode Specific BCODE. Specifies the boot mode-specific code that can further customize and control the boot process.
		00xxb Single bit SPI bus
		01xxb Dual SPI bus
		10xxb Quad SPI bus
		11xxb Reserved
11:8	ROM_BCMD_ DEVENUM	Device enumeration. Specifies the SPI device to use.
		0x0 SPI0
		0x1 SPI1
		0x2 SPI2
		0x3 SPI3
		0x4 - 0xF Reserved
6	ROM_BCMD_ NOAUTO	Automatic device detection disable. When set, this bit disables automatic device detection and uses the setting provided in the other fields of this register to configure the boot mode.
5	ROM_BCMD_ NOCFG	Device configuration disable. When set, this bit disables device configuration. Device configuration includes reconfiguration of the peripherals MMR registers and device pin muxing.
4	ROM_BCMD_ HOST	Host boot mode enable. When set, enables SPI slave boot mode. Otherwise, use the master boot mode.
3:0	ROM_BCMD_ DEVICE	Boot source device. Specifies the device to boot from.
		0x2 SPI

NOTE: All bits in the table that are not defined must be set to zero. Supported features may be limited depending on peripheral instance.

Link Port Slave Boot Mode

This section describes booting from the link port with the processor as a slave.

Link port boot is a slave boot mode in which the processor receives boot data from an external link port master through link port 0. The link port is configured for receive mode and all transfers from the link port to memory are performed under the control of DMA. The maximum supported operating frequency of the link port is 66 MHz for

which the master boot source is responsible for deriving the clock frequency. The link port receiver operates at an asynchronous frequency up to the maximum supported operating frequency.

The link port protocol supports a way to generate link port transmit and receive service requests. The transmit service request is generated on the processor to transmit data when the transmitter is disabled. The receiver drives the LACK $\times$ signal high to initiate this activity. The receive service request is generated on a receiver when it is disabled. The transmitter drives the LCLK $\times$ signal high to initiate this activity.

Because the transmitter and receivers can be enabled at different times, external pull-down resistors are required on both the LCLK $\times$ and LACK $\times$ signals to eliminate any false service request assertions.

The link port slave boot mode initialization phase waits for the receive service request before passing control back to the main kernel. Once this initial receive service request has been detected, the receiving link port is enabled and the boot process completes. The receiving link port is not disabled again until after boot is complete. Once the link port is enabled, the receive DMA channel controls all transfers. The load function for the link port receive boot mode can then point to the peripheral DMA routine of the main kernel in a similar way to the SPI slave boot mode.

NOTE: For default Link Port slave mode, the peripheral DMA fault, LP0_DMA_ERR with fault ID 252 is enabled.

Run-time API

The linkport slave boot mode can be called through the boot routine API function at run time. The run-time API allows for more customization. Both device auto-detection and device configuration can be disabled, and a device other than the default LINKPORT0 can be specified.

If ROM_BCMD_NOCFG flag is specified, it is the programs responsibility to configure pin multiplexing as required.

The following table provides descriptions of the adi_rom_Boot parameter.

Table 41-21: LINKPORT Slave Boot command Bit Descriptions

Bit No. (Access)	Bit Name	Description/Enumeration
11:8	ROM_BCMD_DEVENUM	Device enumeration. Specifies the LINKPORT device to use.
		0x0 LP0
		0x1 LP1
		0x2 - 0xF Reserved
6	ROM_BCMD_NOAUTO	Automatic device detection disable. When set disables automatic device detection and uses the setting provided in the other fields of this register to configure the boot mode.
5	ROM_BCMD_NOCFG	Device configuration disable. When set, this bit disables device configuration. Device configuration includes reconfiguration of the peripherals MMR registers and device pin muxing.

Table 41-21: LINKPORT Slave Boot command Bit Descriptions (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4	ROM_BCMD_HOST	Host boot mode enable. When set, enables SPI slave boot. Otherwise, use the master boot mode.
3:0	ROM_BCMD_DEVICE	Boot source device. Specifies the device to boot from.
		0x4 LINKPORT

NOTE: All bits in the above table that are not defined must be set to zero. Supported features may be limited depending on peripheral instance.

UART Slave Boot Mode

When using UART slave mode boot, the processor receives boot data from a UART host device connected to the UART interface. The device connected to UART0 is initially detected using an autobaud detection sequence. After finishing the UART slave boot process, all control and status registers of the used resources are restored.

Further customization, such as disabling autobaud detection, and changing the device, use the boot routine API.

During boot operation, the host device usually relies on the RTS output of the UART device. At boot time, the processor does not evaluate RTS signals driven by the host. Since the RTS is in a high impedance state when the processor is in reset, or while executing a pre-boot, an external pull-up resistor to VDDEXT is recommended. The *Connection Between Host and Processor* figure shows the interconnection required for booting. The figure does not show physical line drivers and level shifters that are typically required to meet the individual UART-compatible standards.

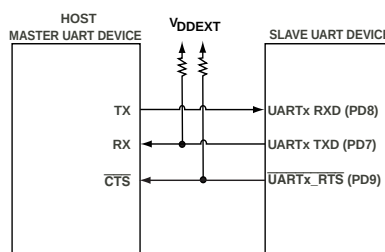


Figure 41-9: Connection Between Host and Processor

When the UART is enabled, the RTS immediately transitions low, encouraging the host to send the first boot stream data as shown in the *Host Relying on RTS* figure. For half-duplex UART connections, the host must avoid this action. The host must wait until it has received the 4 bytes from the slave processor before sending any data.

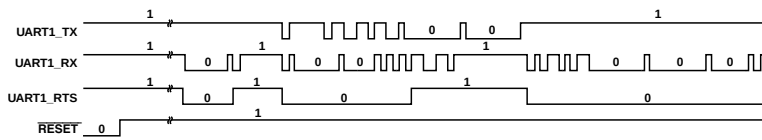


Figure 41-10: Host Relying on RTS

When the boot kernel is processing fill or Initcode blocks, it can require extra processing time and must delay the host from sending more data. This request is signaled using the RTS output.

The *Host Relying on RTS* figure shows RTS timing when an extended Initcode routine executes. Since code execution is distracting from the data loading, the host device must be prevented from sending more data. The timing of the RTS depends on the state of the `UART_CTL.RFRT` bit. This bit is cleared during UART slave boot mode when RTS is de-asserted, the UART receive FIFO contains 4 or more data words, and another start bit is detected.

NOTE: Secure Boot Stream Padding

For slave boot modes, the host must always send data in multiples of 1024 bytes. This requirement is due to the sizing of internal buffers used for DMA.

Autobaud Detection

The kernel supports autobaud detection using the '@' character as data. The host is expected to have its clock set to a rate supported in the UART.

To determine the bit rate when performing autobaud detection, use the following steps:

1. The boot kernel expects an '@' character (0x40, eight bits data, one start bit, one stop bit, no parity bit) on the UART RXD input.
2. The `UART_CLK.EDBO` and the `UART_CLK` register is cleared.
3. The boot kernel acknowledges, and the host then downloads the boot stream. The acknowledgment consists of 4 bytes: 0xBF, `UART_CLK` [15:8], `UART_CLK` [7:0], 0x00.
4. The host is requested to not send further bytes until it has received the complete acknowledge string.
5. Once the 0x00 byte is received, the host can send the entire boot stream.

The host knows the total byte count of the boot stream, but it is not required to know the content of the boot stream.

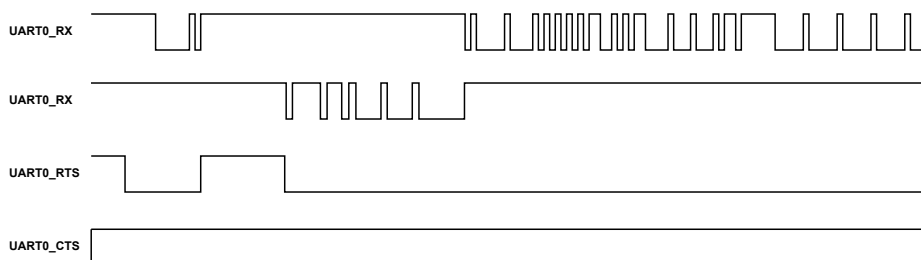


Figure 41-11: UART Autobaud Detection Waveform

The *UART Autobaud Detection Waveform* figure provides timing information for UART booting. After the bit rate is known, the UART is enabled and the kernel transmits the 4 acknowledge bytes.

NOTE: For default UART slave mode, the peripheral DMA fault, UART2_RXDMA_ERR with fault ID 252 is enabled.

Run-time API

The UART slave boot mode can be called through the boot routine API function at run time. The run-time API allows for more customization. Both autobaud detection and device configuration can be disabled, and a device other than the default UART0 can be specified.

If ROM_BCMD_NOCFG flag is specified, it is the programs responsibility to configure pin multiplexing as required.

Autobaud detection can be suppressed using the ROM_BCMD_NOAUTO flag. In this case, the desired configuration can be passed through the ROM_BCMD_UART_CLK bit field. If the ROM_BCMD_UART_CLK bit field is zero, UART_CLK is evaluated. If a value of 0xFFFF was present, the default error routine of the boot kernel is called and the booting process is aborted. Otherwise, the value in UART_CLK remains untouched.

The following table provides descriptions of the adi_rom_Boot parameter.

Table 41-22: UART Slave Boot command Bit Descriptions

Bit No. (Access)	Bit Name	Description/Enumeration
31:16	ROM_BCMD_UART_CLK	UART Clock Divider. When set to zero this field is ignored.
15	ROM_BCMD_UART_EDBO	UART Clock Divider Mode When set enables EDBO functionality.
11:8	ROM_BCMD_DEVENUM	Device enumeration. Specifies the UART device to use.
		0x0 UART0
		0x1 UART1
		0x2 UART2
		0x3 UART3
		0x4 - 0xF Reserved
6	ROM_BCMD_NOAUTO	Automatic device detection disable. When set disables automatic device detection and uses the setting provided in the other fields of this register to configure the boot mode.
5	ROM_BCMD_NOCFG	Device configuration disable. When set, this bit disables device configuration. Device configuration includes reconfiguration of the peripherals MMR registers and device pin muxing.

Table 41-22: UART Slave Boot command Bit Descriptions (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
4	ROM_BCMD_HOST	Host boot mode enable. When set, enables SPI slave boot. Otherwise, use the master boot mode.
3:0	ROM_BCMD_DEVICE	Boot source device. Specifies the device to boot from.
		0x3 UART

NOTE: All bits in the above table that are not defined must be set to zero. Supported features may be limited depending on peripheral instance.

OSPI Master Boot Mode

OSPI Master boot supports booting from flash devices via SPI3 controller. Similar to legacy SPI (SPI0/SPI1/SPI2), it supports boot from many different flash devices provided by different flash vendors further increasing the range of devices due to support for DDR modes of operation. This interface enables to reduce boot times by using faster modes of operation supported by the OSPI controller including DDR modes of operation to achieve higher transfer rates. Most flash vendors support atleast a basic and standard 0x03 read command allowing for a single bit SPI flash interface to be supported.

The OSPI boot mode also uses a device auto-detection feature that is enabled by default. This lets the boot stream itself instruct updates to the SPI configuration and the read command used allowing for more efficient transactions such as for enabling quad bus widths and DDR modes and faster SPI clocks.

Boot from External SPI Flash Devices

The OSPI boot mode supports booting from 24-bit or 32-bit addressable flash devices. The boot mode uses the MDMA channels, which works with OSPI controller to get data from flash memory in memory mapped mode..

When auto-device detection is enabled, the SPI memory is initially read using the standard 0x03 SPI read command with a reduced clocking frequency for maximum compatibility. The first nibble of the boot stream is then used to reconfigure the SPI interface.

NOTE: Support for automatic device detection via the first nibble of the boot stream is not supported when booting secure boot streams. Instead when signing the boot image an attribute can be set in the image header that specifies the configuration to use.

For booting, the SPI memory is connected as shown in the *SPI Memory Connections* figure.

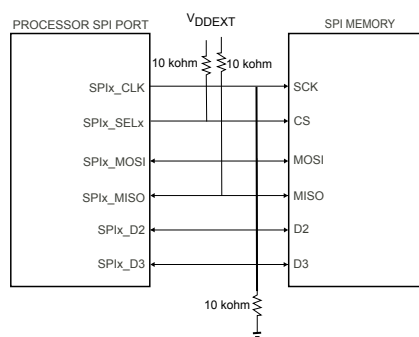


Figure 41-12: SPI Memory Connections

The pull-up resistor on the slave select signal ensures that the memory is deselected when the pin is in a high-impedance mode such as during reset.

Initialization codes are allowed to manipulate the `ADI_ROM_BOOT_CONFIG::dBootCommand` to extend boot mechanism to a second SPI memory connected to another slave select pin. Updating the field that specifies the slave select signal for use, allows the boot process to manage larger boot streams than fit into a single SPI device.

NOTE: If modifying the slave select signal used during the boot process, configure the pin multiplexing to enable the correct functionality for the pin. Once the boot process has proceeded past the configuration function and the boot process has actually started, the boot kernel will not perform any further pin multiplexing operations.

OSPI Device Detection Routine

As boot mode supports booting from various SPI memories, the boot kernel automatically detects what type of memory is connected. To determine whether the SPI memory device requires a 24-bit or 32-bit addressing scheme, the boot kernel performs a device detection sequence prior to booting. The `OSPI_MISO` signal requires a pull-up resistor. The routine relies on the fact that memories do not drive their data outputs unless the right number of address bytes are received.

The SPI flash autodetection routine determines the SPI configuration to be used by data in the boot stream. The autodetection first determines the number of address cycles needed to address the flash device.

This operation is performed by configuring the controller in STIG mode initially with the number of expected address bytes programmed to 3 and a basic read command (0x3). A read command from address 0x00000000 with a 3-byte address is issued. If the flash returns a value other than 0x00 or 0xFF, the flash is in 3-byte address mode. Otherwise, the same process is repeated with the STIG controller configured with four address bytes. Only three and four byte address cycles are supported.

The lower nibble of the received byte is then used to further customize the boot mode. This nibble is referred to as the **BCODE**. The boot code applies settings to the OSPI peripheral according to the *OSPI Master Boot BCODE Descriptions*.

The boot process continues with normal boot operation and it re-issues a command to re-read from address 0. Two read sequences load the first block header. Separate read sequences load further block headers and block payload fields.

The *SPI Device Detection Principle* figure illustrates how individual devices behave.



Figure 41-13: SPI Device Detection Principle

Table 41-23: OSPI Master BCODE Configuration Lookup Table

Member Name	BCODE				
	0	1	2	3	4
Transfer Type	Single bit command, address and data	Single bit command, address and data	Single bit command, address and data	Single bit command, dual address and data	Single bit command, dual address and data
ubDummyCycles	0x00	0x00	0x08	0x04	0x08
ubReadCommand	0x03	0x03	0x0B	0xBB	0xBB
ubDataBits	0x00	0x00	0x00	0x01	0x01
ubAddressBytes (This field is retained. Not used by boot kernel)	0x03	0x03	0x03	0x03	0x03
uwClkLower	0x000F	0x000F	0x0003	0x0003	0x0003
uReserved0	0x0000	0x0000	0x0000	0x0000	0x0000
nCfg	0x00000081	0x00000081	0x00000081	0x00000081	0x00000081
nDsr	0x00000002	0x00000002	0x00000002	0x00000002	0x00000002
nDrir	0x00000000	0x00000000	0x00000000	0x00011000	0x00011000
uReserved1	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
nDummy	0x00	0x00	0x00	0x00	0x00

Table 41-24: OSPI Master BCODE Configuration Lookup Table

Member Name	BCODE				
	5	6	7	8	9
Transfer Type	Single bit command, Quad bit address and data	Single bit command, Quad bit address and data	Single bit command, single bit DDR address and data	Single bit command, single bit DDR address and data	Single bit command, dual DTR address and data
ubDummyCycles	0x06	0x0A	0x06	0x08	0x02
ubReadCommand	0xEB	0xEB	0x0D	0x0D	0xBD
ubDataBits	0x02	0x02	0x00	0x00	0x01
ubAddressBytes (This field is retained. Not used by boot kernel)	0x03	0x03	0x03	0x03	0x03
uwClkLower	0x0003	0x0003	0x0007	0x0007	0x0007
uReserved0	0x0000	0x0000	0x0000	0x0000	0x0000
nCfg	0x00000081	0x00000081	0x00000081	0x00000081	0x00000081
nDsr	0x00000002	0x00000002	0x00000002	0x00000002	0x00000002
nDrir	0x00022000	0x00022000	0x00000400	0x00000400	0x00011400
uReserved1	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
nDummy	0x00	0x00	0x00	0x00	0x00

Table 41-25: OSPI Master BCODE Configuration Lookup Table

Member Name	BCODE					
	A	B	C	D	E	F
Transfer Type	Single bit command, dual DTR address and data	Single bit command, dual DTR address and data	Single bit command, quad DTR address and data	Single bit command, quad DTR address and data	Single bit command, quad DTR address and data	Reserved
ubDummyCycles	0x04	0x06	0x06	0x07	0x08	0x00
ubReadCommand	0xBD	0xBD	0xED	0xED	0xED	0x0B
ubDataBits	0x01	0x01	0x02	0x02	0x02	0x00
ubAddressBytes (This field is retained. Not used by boot kernel)	0x03	0x03	0x03	0x03	0x03	0x03
uwClkLower	0x0007	0x0007	0x0007	0x0007	0x0007	0x0003
uReserved0	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000

Table 41-25: OSPI Master BCODE Configuration Lookup Table (Continued)

Member Name	BCODE					
	A	B	C	D	E	F
nCfg	0x00000081	0x00000081	0x00000081	0x00000081	0x00000081	0x00000081
nDsr	0x00000002	0x00000002	0x00000002	0x00000002	0x00000002	0x00000002
nDrir	0x00011400	0x00011400	0x00022400	0x00022400	0x00022400	0x00000000
uReserved1	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
nDummy	0x00	0x00	0x00	0x00	0x00	0x00

Table 41-26: OSPI Master BCODE Configuration Lookup Table (Applicable to Secure Streams Only)

Member Name	BCODE					
	10	11	12	13	14	15
Transfer Type	Single bit command, dual address and data	Single bit command, quad address and data	Single bit command, single DTR address and data	Single bit command, single DTR address and data	Single bit command, quad DTR address and data	Single bit command, quad DTR address and data
ubDummyCycles	0x0C	0x04	0x04	0x05	0x03	0x09
ubReadCommand	0xBB	0xEB	0x0D	0x0D	0xED	0xED
ubDataBits	0x01	0x02	0x00	0x00	0x02	0x02
ubAddressBytes (This field is retained. Not used by boot kernel)	0x03	0x03	0x03	0x03	0x03	0x03
uwClkLower	0x0003	0x0003	0x0007	0x0007	0x0007	0x0003
uReserved0	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
nCfg	0x00000081	0x00000081	0x00000081	0x00000081	0x00000081	0x00000081
nDsr	0x00000002	0x00000002	0x00000002	0x00000002	0x00000002	0x00000002
nDrir	0x00011000	0x00022000	0x00000400	0x00000400	0x00022400	0x00022400
uReserved1	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
nDummy	0x00	0x00	0x00	0x00	0x00	0x00

NOTE: For the above configurations, the addressing scheme can be 3-bytes or 4-bytes depending on the addressing of flash detected in auto-detection. The SPI mode byte issued for all the SPI master peripheral based configurations is 0x00. The mode byte is the first byte transmitted after the address cycles and is used to control the continuous read mode functionality in which the next read operation is not required to issue a command cycle. Continuous read mode is not supported during the boot process.

Supported Quad Mode Enable Methods

The boot ROM does not support enabling quad mode on the SPI flash device. To boot in quad mode, the flash device must be configured outside the boot ROM.

To enable quad mode, initially boot in dualmode and use an initcode.

Run-Time API

The following table provides descriptions of the [adi_rom_Boot\(\)](#) command parameter.

Table 41-27: OSPI Master Boot Command Bit Descriptions

Bits	Name	Setting	Description
31:28	ROM_BCMD_SPIM_SPEED	0-1111b	SPI clock divider to be used. This value is written to the SPI peripherals clock divider register
27	ROM_BCMD_SPIM_CMDISK IP_EN	0	Issue a read command with every read operation
		1	Do not issue a read command with each read operation only issue the read command on the first read.
26:22	ROM_BCMD_SPIM_DUMMY	00000b	No dummy clock cycles required after the address
		00001b	1 dummy clock cycle required after the address
			
		11111b	31 dummy clock cycle required after the address
21:20	ROM_BCMD_SPIM_ADDR	00b	Flash device requires an 8-bit address
		01b	Flash device requires a 16-bit address
		10b	Flash device requires a 24-bit address
		11b	Flash device requires a 32-bit address
19:16	ROM_BCMD_SPIM_BCODE	0000b-1111b	Applies an initial configuration as described in the SPI Master Boot BCODE Configurations. Not all options may be available.

Table 41-27: OSPI Master Boot Command Bit Descriptions (Continued)

Bits	Name	Setting	Description
15	ROM_BCMD_SPIM_CMD	0	Issue the read command over the single bit bus
		1	Issue the read command over the multi-bit bus. Not recommended for use. Intended for use with products with on chip SPI flash only, and even then it can cause complications.
14:12	ROM_BCMD_SPIM_SSEL	000b	Use slave select 1 for SPI chip select
		001b	Use slave select 2 for SPI chip select
		010b	Use slave select 3 for SPI chip select
		011b	Reserved
		100b	Reserved
		101b	Reserved
		110b	Reserved
		111b	Reserved
11:8	ROM_BCMD_SPIM_DEVENUM	0 to 16	Specifies the boot peripheral enumeration. For OSPI Master boot, configure this field to 0.
7	Reserved	Reserved	Reserved
6	ROM_BCMD_SPIM_NOAUTO	0	Perform automatic device detection and peripheral configuration based on the BCODE value (first nibble) of the boot streams block header
		1	Do not perform automatic device detection

Table 41-27: OSPI Master Boot Command Bit Descriptions (Continued)

Bits	Name	Setting	Description
5	ROM_BCMD_SPIM_NOCFG	0	Instructs the config routine to perform pinmuxing configuration and full peripheral configuration
		1	The config routine will not change the configuration of the peripheral and it will not enable pinmuxing. This can be used if the boot software does not support specific features of a peripheral, users can configure the peripheral prior to calling the boot routine.
4	ROM_BCMD_SPIM_HOST	0	Master boot mode
		1	Not to be used for OSPI Master Boot
3:0	ROM_BCMD_SPIM_DEVICE	0000b	Not to be used for OSPI boot
		0001b	Not to be used for OSPI boot
		0010b	Not to be used for OSPI boot
		0011b	Not to be used for OSPI boot
		0100b	Not to be used for OSPI boot
		0101b	Not to be used for OSPI boot
		0110b	Not to be used for OSPI boot
		0111b	Not to be used for OSPI boot
		1000b	OSPI Memory Mapped Boot (Memory mapped mode and MDMA)
		1001b – 1111b	Not to be used for OSPI boot

NOTE: All bits in the above table that are not defined must be set to zero.

To support the above lookup table at varying SPICLK frequency range, boot kernel programs the Read data capture register such that sampling time does not go beyond one full cycle of SPI CLK for STR and half cycle of SPI CLK for DTR mode of operation. If this timing is not sufficient to support all flash vendors, there is a provision to update the Read data capture register from Customer OTP space with worst case value corresponding to different flash device used by user. This value has to be derived based on the min-max AC timing propagation delay from OSPI controller to pin.

Boot Loader Stream

A loader stream is a set of formatted blocks containing instructions for the boot kernel, as well as the application and data for loading to the chip. This section details the different aspects of the stream, its blocks, some common use cases, and the ROM function.

Each block begins with a block header that contains attributes of the block as well as flags to control its processing by the boot ROM. On power-up or reset, the processor begins executing the on-chip boot ROM. The boot stream is either read from memory or received from a peripheral, depending on the boot mode specified. Each block in the boot stream instructs the boot kernel to perform an action, most commonly to load data to a specified location. For a complete list of actions, refer to the [Block Types](#) section.

Some common actions include the following:

- running code that initializes a peripheral
- processing data then loading it to a location

As shown in the *Project Flow* figure, a utility is required to process the resulting output from the tool chain to create a valid boot stream. This utility can be in the form of a standalone application or script that parses an application image file, elf output file, or text-based file such as Intel hex. A flash programmer utility can format a boot stream internally.

The elfloader utility parses the application data, and creates a set of blocks representing the segmented application. When processing an actual image that must be loaded to a single contiguous memory block, this representation can contain as little as a single header block. The output of the standalone utility is stored in a loader file (.ldr). The loader file contains the boot stream and becomes available to hardware by:

- programming or burning it into non-volatile external memory, or
- sending it through a peripheral such as the UART during boot time

A loader stream must always begin with a first block and end with a final block. The loader file contains the boot stream and becomes available to hardware by:

- programming or burning it into non-volatile external memory, or
- sending it through a peripheral during boot time

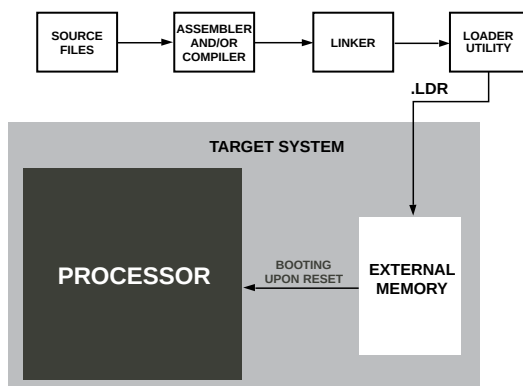


Figure 41-14: Project Flow

The *Booting Process* figure shows the parallel or serial boot stream contained in a flash memory device. In host boot scenarios, the non-volatile memory usually connects to the host processor rather than directly to the processor. After reset, the on-chip boot kernel reads and parses the headers and the loader stream is processed block-by-block. Finally, payload data is copied to destination addresses, either in on-chip L1 and L2 memory, or off-chip to SDRAM or SRAM.

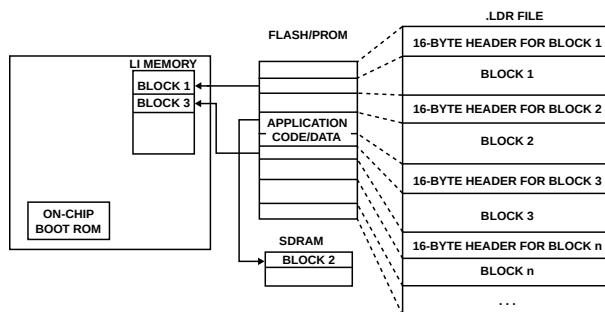


Figure 41-15: Booting Process

In some cases (for example, secure boot or when the `BFLAG_INDIRECT` flag for any block is set), the boot kernel uses another memory block for intermediate data storage. In order to preserve the security of the device processors do not allow these storage regions to be initialized with boot data. The boot stream is loaded to the intermediate storage then processed by the kernel and loaded to the final destination. The final destination cannot be the intermediate storage location otherwise the boot process terminates in an error.

Block Header

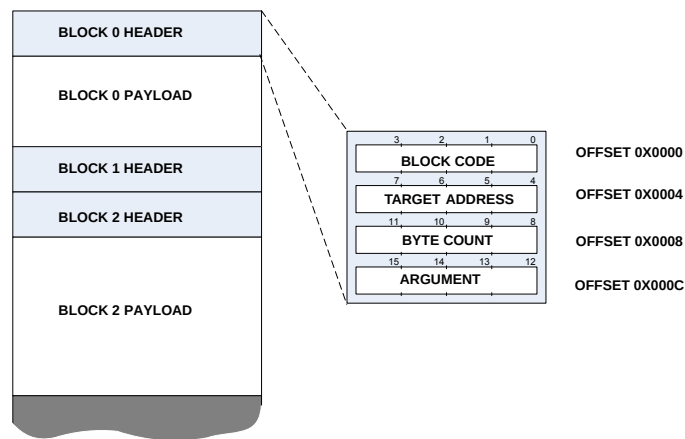


Figure 41-16: Loader Stream Block Structure

A boot stream consists of multiple boot blocks as shown in the figure. A 16-byte block header begins every block. The 16 bytes are functionally grouped into four 32-bit words:

- Block code
- Target address
- Byte count
- Argument field

This section provides a general description describes the fields in general. The use varies depending on the block type and boot mode.

Block Code

Table 41-28: Block Code Flags

Bit	Name	Description
0–3	BCODE	Specific to boot modes (see Boot Modes)
4	BFLAG_SAVE	Intended to allow for a user application to mark blocks for saving the memory of this block to off-chip memory in case of power failure. The on-chip boot kernel does not use this flag.
5	BFLAG_AUX	When set indicates that the byte address space translation for SHARC core boot blocks requires translation to the 48-bit PM address space. When cleared translation to the 32-bit address space is performed.
7–6	Reserved	
8	BFLAG_FILL	Fill the target location with a specified 32-bit value
9	Reserved	
10	BFLAG_CALLBACK	Calls the previously registered callback function

Table 41-28: Block Code Flags (Continued)

Bit	Name	Description
11	BFLAG_INIT	Calls function at target address. If the block contains a payload, the payload is loaded prior to the call.
12	BFLAG_IGNORE	Block payload is ignored
13	BFLAG_INDIRECT	Boots the payload to the intermediate storage location
14	BFLAG_FIRST	Indicates the block that is the beginning of a new application
15	BFLAG_FINAL	Indicates the last block of a loader stream. Booting will complete after processing the block. This flag does not denote the end of an application in a Multi-Application Boot Stream.
16–23	HDRCHK	A simple 8-bit XOR checksum of the other 24 bits in the boot block header.
24–31	HDRSIGN	0xAD, 0xAC or 0xAB. Indicates if the boot block is intended for core 0, core 1 or core 2 respectively.

TARGET_ADDRESS

The `TARGET_ADDRESS` holds the applicable address for the block, (where the code or data is loaded). However, the interpretation of the field differs depending on what specific flags are set in the block code. Refer to the documentation for each block type for details.

The following attributes must be true:

- The target address must be divisible by 4, as the boot kernel uses 32-bit DMA for certain operations.
- The target address must point to valid on-chip or off-chip memory locations.

BYTE_COUNT

The `BYTE_COUNT` must be divisible by 4, and can also be zero. This 32-bit field generally holds the size of the block. In some cases, it has a different use (such as when `BFLAG_FILL` is set). See the [Block Types](#) section for information on the variations.

ARGUMENT

The 32-bit field is a user-variable for most block types. The `Initcode` or `callback` routine can access this value and use it for optional instructions.

The different block types use the `ARGUMENT` field in various ways. See the [Block Types](#) descriptions for further information.

Block Types

A loader stream is a set of linked blocks and each block is responsible for performing a certain function dependent on the block type. The flags in the block header define a block type. Operations include functions such as loading

data, filling a memory region with data, and instructing the kernel to stop processing. This section describes each block type and its usage within a boot loader stream.

Normal Block

The primary function of a block is to load data into a specified location of memory. A normal block instructs the boot kernel to load the data contained in its payload to the location specified in the `TARGET_ADDRESS` field. The `BYTE_COUNT` defines the size of the payload. Once the correct amount of data has been loaded, the kernel moves on to process the next block in the stream.

Table 41-29: Flags

Flag	Required Value	Init
<code>TARGET_ADDRESS</code>	Y	Address where payload is loaded (must be valid)
<code>BYTE_COUNT</code>	Y	Size of block in bytes

First Block

A first block indicates the start of a boot stream and is always needed at the beginning of the boot stream. When a loader stream contains [Multi-Application Boot Streams](#), a first block that occurs within the loader stream indicates the beginning of a new application.

When the kernel processes the first block in a loader stream, the boot kernel uses the `TARGET_ADDRESS` to determine the application entry location. For more details, refer to [Boot Termination and Application Execution](#).

NOTE: A first block cannot be combined with a fill block.

Table 41-30: Flags

Flag	Required Value	Init
<code>BFLAG_FIRST</code>	Y	1
<code>ARGUMENT</code>	Y	Offset to the next application, or first address following loader stream. Commonly referred to as the <code>NEXTDXE</code> field.
<code>TARGET_ADDRESS</code>	Y	When the block is the first block in a loader stream it defines the start address for the application. If the block is not the first in a loader stream, use the target address as in normal operation.

Final Block

The final block marks the last block in a boot stream. After processing a final block, the boot kernel jumps to the start address of the application. For more information on the definition of the start address, refer to [Boot Termination and Application Execution](#).

There is further customization available to alter the kernel behavior. For example, the kernel can be instructed to return from the boot routine rather than jump to the application using initialization codes or the `adi_rom_Boot()` API.

Before the boot kernel passes program control to the application, it does some housekeeping. Most of the registers in use are set to their default state. However, some register values can differ depending on the boot mode. See [Boot Modes](#) for more information.

Table 41-31: Flags

Flag	Required Value	Init
BFLAG_FINAL	Y	1

Indirect Block

An indirect block is first loaded to a storage location before being copied to the destination. This block is used in the following situations:

- Some boot modes do not use DMA from the boot peripheral. The core is not always able to access some memory locations directly or efficiently. An intermediate load to a different location improves overall efficiency.
- In some booting situations, the data in the payload must be operated-on or analyzed before it is fully loaded (such as decryption or a checksum calculation). By using an intermediate location, such situations are simplified and are more efficient when loading to off-chip memories.

In some cases, a boot block does not fit into temporary storage memory. Having a larger buffer can improve boot performance. If an entire block cannot fit into the buffer, it is processed in pieces. Initialization code or callback functions can alter the temporary buffer region, including its location and size, by modifying the `ADI_ROM_BOOT_CONFIG::pTempBuffer` and `ADI_ROM_BOOT_CONFIG::dTempByteCount` variables in the [struct ADI_ROM_BOOT_CONFIG](#) structure.

Table 41-32: Flags

Flag	Required Value	Init
BFLAG_INDIRECT	Y	1
BFLAG_CALLBACK	N	Defines a callback function to operate on intermediate data. These 2 flags are often used together.

Ignore Block

It is a block that is ignored by the loader stream (in several cases). These blocks are useful when it is not possible to pass information in another block header. For example, if the first block contains data such as a firmware revision rather than application code, then `BFLAG_IGNORE` can be set with the correct application start address. This step ensures that the loader stream uses the correct start address. Since this block has no other function, identify it as an ignore block. Then, the kernel does not attempt to process any payload.

Ignore blocks clear the following flags, disabling the corresponding blocks from being processed if set along with BFLAG_IGNORE:

- BFLAG_INIT
- BFLAG_CALLBACK
- BFLAG_FINAL
- BFLAG_AUX

NOTE: When BFLAG_IGNORE is set along with BFLAG_FIRST, only the payload associated with the first block is ignored. The application entry point retrieved from the first block is always processed.

Table 41-33: Flags

Flag	Required Value	Init
BFLAG_IGNORE	Y	1
BYTE_COUNT	Y	Size of block to ignore; can be zero

Fill Block

A fill block instructs the boot kernel to perform a 32-bit memory fill of the memory region. Fill blocks help minimize the size of a boot stream when an application contains large arrays of data that need are initialized at startup. Zero fill is the most common fill block type. However, any 32-bit fill value is supported.

Table 41-34: Flags

Flag	Required Value	Init
BFLAG_FILL	Y	1
TARGET_ADDRESS	Y	Address where payload is loaded (must be valid)
BYTE_COUNT	Y	Size of block in bytes (must be multiple of 4)
ARGUMENT	Y	32-bit fill value

Init Block

An initialization block instructs the boot kernel to perform a function call to the target address after the entire block has loaded. The function called is referred to as the *initialization code (Initcode) routine*. Refer to the API reference [initcode\(\)](#) for full details.

If the Initcode routine has been previously loaded, the block can declare a zero-size and have no payload.

Initcode routines can be used to speed up and customize booting mechanisms exposed by the boot kernel. Traditionally, an Initcode routine is used to set up the system PLL, bit rates, wait states, and the external memory controllers. Boot time can be significantly reduced when an init block is executed early in the boot process.

Initcode routines are required to follow the C language calling conventions. The expected C prototype is:

```
void initcode(ADI_ROM_BOOT_CONFIG * pBootConfig)
```

NOTE: When programming in assembly, use a return from subroutine instruction for returns.

The structure provided to the Initcode routine by the boot kernel contains information about the block being processed. It includes header information, locations of temporary block data (for indirect blocks), target address, and byte count. See [struct ADI_ROM_BOOT_CONFIG](#) for a full list and details on the provided data.

In the simplest case, an Initcode routine consists of only a single block in which the BFLAG_INIT flag is set. For larger routines, a sequence of blocks can incrementally load the routine, and only the last block sets the BFLAG_INIT flag. In the latter case, the last block has no payload attached, and simply instructs the boot kernel to issue a call to subroutine instruction.

An Initcode routine can be overwritten by a successive block when it is no longer needed. Otherwise, the routine can be called at multiple points during the boot process, and even remain in memory after the application completes booting.

NOTE: The following list provides requirements for Initcode that is written in C or C++.

- Ensure that the Initcode routine does not contain calls to the run-time libraries
- Do not assume that parts of the run-time environment, such as the heap, are fully functional
- Ensure that all run-time components are loaded and initialized before the routine executes

The loader utility and tool set provide mechanisms to aid in implementing initialization codes and organizing them properly within the boot loader stream. A special project type is provided to allow the creation of Initcode routines as separate projects. Options are available to assign particular pieces of the application to be Initcode routines. For details and more information on the utility, see the *Loader and Utilities Manual*.

Table 41-35: Flags

Flag	Required Value	Init
BFLAG_INIT	Y	1
TARGET_ADDRESS	Y	Location to load payload data. Call to subroutine issued to the same location.
ARGUMENT	N	Can be used to supply block specific arguments
BYTE_COUNT	Y	Size of payload; can be zero

NOTE: Init blocks cause software not located in the boot ROM during the boot process to execute. In the case of a secure boot scenario, initcode routines are not supported. The secure boot authentication process is performed at the end of the boot process. Execution of any user software prior to the authentication process violates the secure boot requirements.

Callback Block

It instructs the boot kernel to call a pre-registered function when the payload of the block is loaded. The purpose of a callback routine is to apply standard processing to the block payload. The callback routine is registered through an Initcode routine prior to loading a block using the routine. Typically, callback routines contain checksum, decryption, decompression or hash algorithms.

To register a callback, create an [Init Block](#) whose Initcode modifies the `ADI_ROM_BOOT_CONFIG::pCallbackFunction` with the address of the callback function to execute. A callback function must be registered prior to processing a callback block.

As callback routines require access to the payload data of the boot blocks, load the block data before processing. Often an [Indirect Block](#) is used in combination with a callback block. Indirect blocks in combination with callback blocks allow for post processing of the loaded data before it is then transferred to the final destination.

Callback routines are expected to meet the C language calling conventions. The prototype is as follows:

```
ROM_BOOT_RESULT callback(
    ADI_ROM_BOOT_CONFIG * pBootConfig,
    ADI_ROM_BOOT_BUFFER * pBuffer,
    uint32_t nFlags
)
```

The `pBootConfig` argument contains a pointer to the [struct ADI_ROM_BOOT_CONFIG](#) information and `pBuffer` provides access to the address and size of the block (can vary when using indirect). The `nFlags` parameter is specifically used when the `BFLAG_INDIRECT` flag is also used. The `CBFLAG_DIRECT` flag indicates that the `BFLAG_INDIRECT` bit is not active so that the program only calls the callback routine once per block. When the `CBFLAG_DIRECT` flag is set, the `CBFLAG_FIRST` and `CBFLAG_FINAL` flags are also set.

NOTE: Callback blocks result in execution of software that is not located in the boot ROM during the boot process. In the case of a secure boot scenario callback routines are not supported because the secure boot authentication process is performed at the end of the boot process and execution of any program prior to the authentication process violates the secure boot requirements.

Callback Block in Conjunction with Indirect Block

A block using a callback routine is also loaded indirectly and there are slight behavioural differences. The procedure for loading is:

1. Load data into the temporary buffer defined by `ADI_ROM_BOOT_CONFIG::pTempBuffer`.
2. Issue a call to `ADI_ROM_BOOT_CONFIG::pCallbackFunction`.
3. After the callback routine returns and if the return value is zero, the memory DMA copies data to the destination.

When a block does not fit entirely into the temporary buffer, loading is performed similar to indirect blocks. The software calls the callback function after each chunk is loaded into the temporary storage. The `nFlags` parameter provides information on the specific iteration.

When a block does not fit entirely into the temporary storage area, `nFlags` tells the callback routine whether it is invoked for the first time (`CBFLAG_FIRST`) or called the last time (`CBFLAG_FINAL`) for a specific block.

When the software invokes DMA to copy the data, it relies on the supplied `pBuffer` data, not the `ADI_ROM_BOOT_CONFIG::pTempBuffer` and `ADI_ROM_BOOT_CONFIG::dTempByteCount` members of the boot structure.

The callback routine can control the source of the memory DMA by altering the content of the `pBuffer` structure. This alteration is necessary when the callback routine performs data manipulation such as decompression.

When the software uses an indirect block, the return value of the callback routine determines whether the DMA transfer occurs. If the value is non-zero, then the transfer does not occur.

Table 41-36: Flags

Flag	Required Value	Init
<code>BFLAG_CALLBACK</code>	Y	1

NOTE: Callback blocks cause software not located in the boot ROM during the boot process to execute. In the case of a secure boot scenario callback routines are not supported because the secure boot authentication process is performed at the end of the boot process and execution of any user software prior to the authentication process violates the secure boot requirements.

Save Block

A save block is intended to mark blocks in a boot stream for saving to off-chip memory. The on-chip boot kernel does not use this flag. A program can process the boot stream to find address of regions of memory for saving to external memory. On a reboot the program may then restore the previously saved contents. This block provides a means of doing a context restore after a reboot.

Table 41-37: Flags

Flag	Required Value	Init
<code>BFLAG_SAVE</code>	Y	1

Single-Block Boot Streams

The simplest boot stream consists of a single block header and one contiguous block of instructions and optionally data. When the appropriate flags are set in the block header, the kernel loads the block to the target address, terminates the boot process, and begins executing from the entry address of the application.

The *Initial Header for Single-Block Stream* table shows an example of a single-block boot stream header with settings that can be loaded using any boot mode. The `BFLAG_FIRST` and `BFLAG_FINAL` flags are both set at the same time. The desired location and size of the application determines the target address and byte count.

When using single block boot streams on products with multiple cores, the boot stream must be targeted towards the primary booting core that manages the boot process. If core 0 is the primary booting core, the boot stream must contain code that is intended for execution by that core. It is possible to boot a single block boot stream when using the API to load an application to the non-booting core. In this case, the BFLAG_RETURN flag must be set so the boot process returns to user application on the core mastering the boot process.

Table 41-38: Initial Header for Single-Block Stream

Field	Description of Value
BLOCK_CODE	0xAD000000 XORSUM BFLAG_FINAL BFLAG_FIRST
TARGET_ADDRESS	Start address of block
BYTE_COUNT	Number of bytes in the block
ARGUMENT	Functions as next-application pointer in multi-application boot streams.

Direct Code Execution

Applications can avoid long boot times and execute code from flash or SDRAM memory with minimal processing by the boot kernel. This feature is called direct code execution.

An initial boot block header is required for the processor to fetch and execute program code from the boot device as early as possible. The block uses safety mechanisms to avoid unpredictable processor behavior when boot memory is not yet programmed with valid data. Safety mechanisms include the header signature and the byte-wise XOR checksum. Rather than blindly executing code, the boot kernel first executes the pre-boot routine for system customization. It then loads the first block header and checks it for consistency. If the block header is corrupt, the boot kernel calls the error handler and does not start code execution.

If the initial block header check is good, the boot kernel interrogates the block flags. If the BFLAG_FINAL flag is set, the boot kernel terminates and executes the sequence as described in the [Boot Termination and Application Execution](#) section. If the application requires that the boot kernel customize the starting address in advance, the first block must also have the BFLAG_FIRST flag set. The target address field is then saved as the application start address.

When processing direct code execution blocks, the block instructs the processor executing the boot code to execute from the address specified. It is not possible to have core 0 boot the block and have it instruct core 1 to immediately start execution from the address provided.

For example, when the block header described in the *Direct Code Execution Header* table is placed at address 0x20000000, the boot kernel is instructed to issue a JUMP command to address 0x20000020.

Table 41-39: Example Direct Code Execution Header

Field	Value	Comments
BLOCK_CODE	0xAD7BD006	0xAD000000 XORSUM BFLAG_FINAL BFLAG_FIRST BFLAG_IGNORE BCODE

Table 41-39: Example Direct Code Execution Header (Continued)

Field	Value	Comments
TARGET_ADDRESS	0x20000020	Start address of application code. Provided as an example this would be dependent upon the start address required for a given product.
BYTE_COUNT	0x00000010	Ignores 16 bytes to provide space for control data such as version code and build data. This field is optional and can be zero. The payload is ignored due to the BFLAG_IGNORE flag being set.
ARGUMENT	0x00000010	Functions as next-application pointer in multi-application boot streams.

Multi-Application Boot Streams

This section describes loader streams that contain multiple applications.

A boot stream is typically generated from an application file. Loader streams with more than one application are commonly referred to as multi-application booting. A loader utility often accepts multiple application files as input parameters and generates a contiguous boot image. The subsequent applications are appended to the first.

The loader utility must also update the argument field of all BFLAG_FIRST blocks. The argument field of a BFLAG_FINAL block is called the *next-application* pointer.

The next-application pointer of the first application boot stream is a relative pointer to the start address of the second application boot stream. A multi-application boot image can be seen as a linked list of boot streams. The next-application pointer of the last application boot stream is a relative pointer to the next free address. The *Multi-Application Boot Stream Example* figure shows an example.

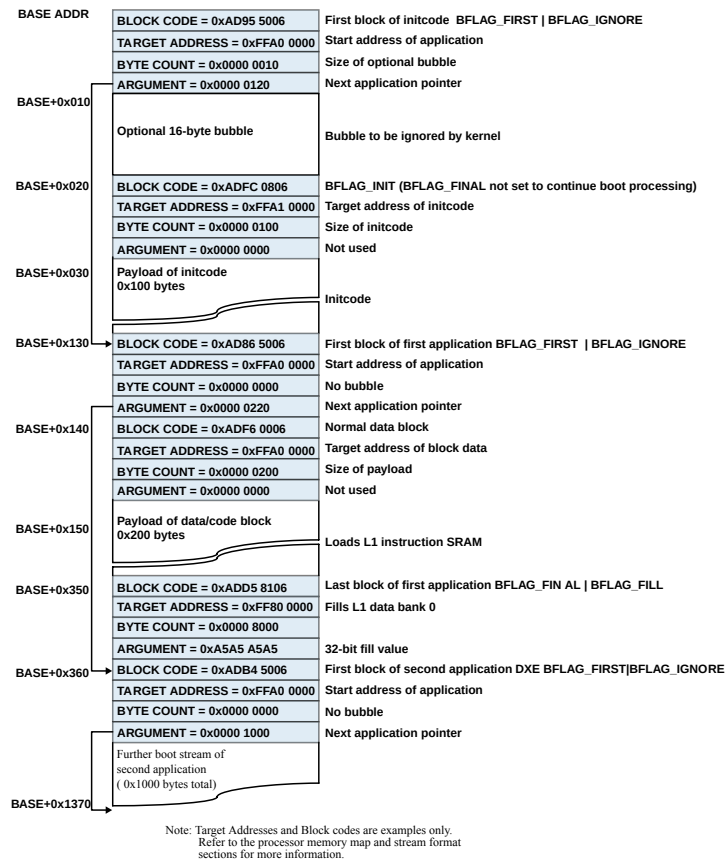


Figure 41-17: Multi-Application Boot Stream Example

The *Multi-Application Direct Code Execution Example* figure shows a linked list of initial block headers. The blocks instruct the boot kernel to terminate immediately and to start code execution at the address provided by the target address field of the individual blocks. There is nothing in the boot code that prevents multi-application streams from mixing regular boot streams and direct code execution blocks.

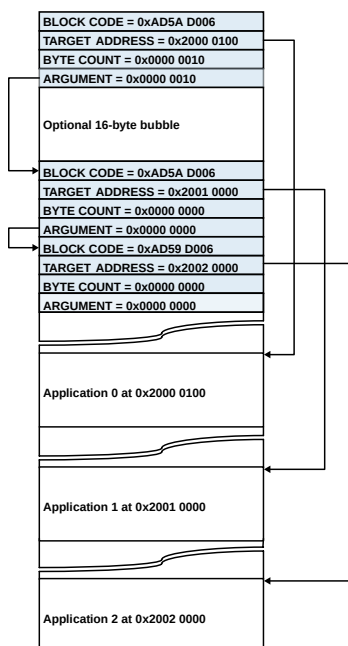


Figure 41-18: Multi-Application Direct Code Execution Example

CRC32 Protection

The boot kernel has mechanisms that allow verification of each block's payload using a 32-bit CRC. The boot ROM contains a function that can be called as an initcode to register the CRC callback and initialize the CRC peripheral with a specified polynomial. CRC32 protected loader streams are supported by the programming tools utilities.

To enable this feature an [Block Types](#) must be located in the boot stream with a `TARGET_ADDRESS` that points to the `adi_rom_Crc32Init()` function in the ROM. The `ARGUMENT` field contains the CRC32 checksum polynomial that is used to initialize the CRC lookup table. Once the CRC initcode function in the ROM has been executed, CRC verification is enabled for all subsequent blocks except:

- Ignore
- First

NOTE: CRC functionality is dependent upon the use of an Initblock and is not supported in secure booting.

Secure Boot

The secure boot process provides a way to integrate security in the processor boot sequence. A chain of trust is established within the system by ensuring the integrity and authenticity of the boot image. Confidentiality is also supported.

Secure boot increases protection against malicious, unsecured accesses to critical and confidential resources of the processor. The boot stream application code and data must be digitally signed in order to build up a chain of trust in

the system. This requirement allows the processor to distinguish between authentic and trusted code from non-authentic and untrusted code.

Secure boot also provides confidentiality support. The digitally signed boot image can be optionally encrypted. When loading an encrypted image, the ROM decrypts while loading, then authenticates, before any application code is executed.

Secure boot is an optional feature of the processor that is disabled by default. The feature is enabled using the OTP lock API; secure boot cannot be disabled after it has been enabled. When security is enabled, developers are not dependent on Analog Devices to provision the devices, sign code or provide security certificates. The required tools for signing and encrypting the boot images are provided with the processor development tools.

Integrity and Authenticity Protection

Integrity protection is based on the secure hash SHA-2 224/256-bit algorithm. Authenticity protection is based on the ECDSA algorithm.

ECDSA uses public key cryptography consisting of two keys: a private key and a public key. The public key is stored in OTP memory on the processor so that the secure boot process can verify the authenticity of the signed boot image. Only parties in possession of the private key are able to sign the images.

Confidentiality Protection

Confidentiality protection uses the AES algorithm. Two variants are supported: wrapped and unwrapped.

The wrapped variant uses a 128-bit Key Encryption Key (KEK) stored on the processor to decrypt the 128-bit AES decryption key embedded in the secure header. The unwrapped variant stores the AES description key on the processor and utilizes it to decrypt the entire image.

The privacy of the key stored on the device (whether AES or KEK) is paramount to the security of the system. Disclosure of this key compromises security of the entire system.

Anti-Cloning Protection

Anti-cloning protection is based on the confidentiality protection. If each processor in a system uses a unique private key for confidentiality protection, then cloning between these devices can be prevented. The boot image is incompatible with devices using a different private key for decryption.

Anti-Rollback Protection

The secure boot process supports anti-rollback protection using a 32-bit counter in OTP memory. A value of 0x00000000 in the OTP makes anti-rollback disabled by default. If anti-rollback protection is required, then the program can set the rollback ID when signing the boot image. When the boot image is authenticated, the secure boot software updates the counter in OTP (if the rollback ID in the boot image is greater than the value currently stored in the OTP counter).

The rollback ID stored in the secure boot image header is integrity-protected and cannot be altered.

NOTE: To enable anti-rollback protection for secure boot operations, write a non-zero value to the 32-bit counter in OTP memory. As long as the counter register remains at the default value of zero, anti-rollback protections are not enabled regardless of the rollback ID located in a secure boot stream.

CAUTION: There are a number of restrictions for the rollback ID in the OTP module. Programs should only use the OTP boot program ROM API to set the counter. Refer to the OTP counters section for information on the implementation strategy.

Terminology

ECDSA

Elliptical Curve Digital Signature Algorithm

BLp

Boot Loader plain text, Plaintext Format

BLx

Boot Loader without key, Keyless Format

BLw

Boot Loader wrapped, Wrapped Format

BLe

Boot Loader encrypted, Encrypted Format

SBLS

Secure Boot Loader Stream

SBH

Secure Boot Header

SBCR

Secure Boot Confidentiality Root

AES

Advanced Encryption Standard

Signing for Secure Boot Images

All boot images must be digitally signed to create secure boot images. The boot image is processed by the security utilities included with the development tools to sign and optionally encrypt the boot image. The security utilities operate with key-pairs consisting of a private and a public key. The private key is used for signing the images, and the public key is used to validate an image being loaded into the processor.

CAUTION: The private key generated from the signing utility, used for signing images, is never required by the processor for successful secure boot. The private key is only ever required by the signing utility and should be made available only within the system responsible for the image signing process.

The image signing utility provides the following functionality:

- Signing and encrypting of images
- Generation of ECDSA key pairs
- Generation of random encryption keys
- Extraction of the public key from an ECDSA key pair
- Setting [Secure Boot Image Attributes](#)

For more information on the use of the signing utility, refer to the Loader and Utilities manual.

Secure Boot Image Types

This section provides an overview of the different image types and how to use them.

Plaintext (BLp) Format

The BLp format provides integrity plus authentication protection of the boot image. The boot image is produced using 224-bit or 256-bit Elliptical Curve Digital Signature Algorithm (ECDSA) private key. To authenticate the image, program the corresponding public key into the OTP `public_key` field using the OTP boot program API.

SBH	Boot Loader Stream
-----	--------------------

Wrapped (BLw) Format

The BLw format provides the highest level of protection: integrity plus authentication, confidentiality, and anti-cloning protection. The image contains an AES wrapped image encryption key (denoted by [K]) within the secure header. The image data is encrypted with the wrapped key, preventing cloning. An extra key is required to unwrap the header; program this key into the OTP `pvt_128key` field using the OTP boot program API.

SBH [K]	Encrypted Boot Loader Stream
---------	------------------------------

Keyless (BLx) Format

The BLx format is similar to the BLw format except that the image does not contain the key at all. This format provides anti-cloning protection only if the secure key is unique per device. Program the decryption key for the data into OTP `pvt_128key` field using the OTP boot program API.

SBH	Encrypted Boot Loader Stream
-----	------------------------------

Secure Boot Image Format

Secure Boot images provide authenticity and integrity protection during the boot process. A secure boot image is comprised of a secure boot header and an optionally encrypted loader stream.

Signed images consist of the following sections to comprise a complete secure boot image:

- Secure Boot Header
- Image Attributes
- Image Section

The [Figure 41-19 Secure Boot Image](#) shows that the image attributes are encapsulated within the secure boot header. The image attributes are actually integrity protected along with the image section. The image section contains a standard boot loader stream. Some block types are not allowed as described in [Unsupported Boot Stream Blocks](#).

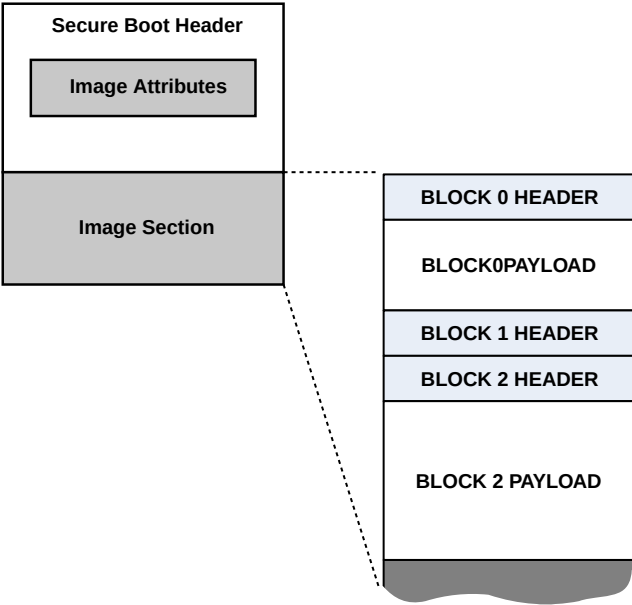


Figure 41-19: Secure Boot Image

Secure Boot Header

Table 41-40: Secure Boot Header

Bytes	Name	Description	Values		
			Keyless Format BLx	Wrapped Format BLw	Plain Text Format BLp
3:0	Type	Format and version of the image. Upper 24 bits are the image format and lower 8 bits are the image version	0x424c7802	0x424c7702	0x424c7002
67:4	Signature	The ECDSA signature of the image	Two 256-bit numbers		
99:68	Hash	Hash of the image	Hash size depends upon whether ECDSA 224 or ECDSA 256 is used. 1. For ECDSA 224 Hash Value: 95:68 bytes 2. For ECDSA 256 Hash Value: 99:68 bytes		
123:100	Key	Confidentiality (only applicable for certain formats)	Reserved	192-bit AES-WRAP data holding a 128-bit AES key	Reserved
139:124	IV	Initialization Vector (only applicable for certain formats)	16-bit IV generated during signing process		Reserved
143:140	Length	The length of the image section in bytes	Maximum supported byte count 0x10000000 bytes		
207:144	Attributes	Image attributes	Support for up to 8 image attributes		
211 :208	Reserved				

Secure Boot Processing Overview

The *Secure Boot Processing* figure illustrates how the block is processed using secure features (not all block header type details are shown). For details on the various block types and their function, see [Block Types](#). Some image types are decrypted. Decrypt indicates that the data is decrypted when applicable to the [Secure Boot Image Types](#) at that particular stage.

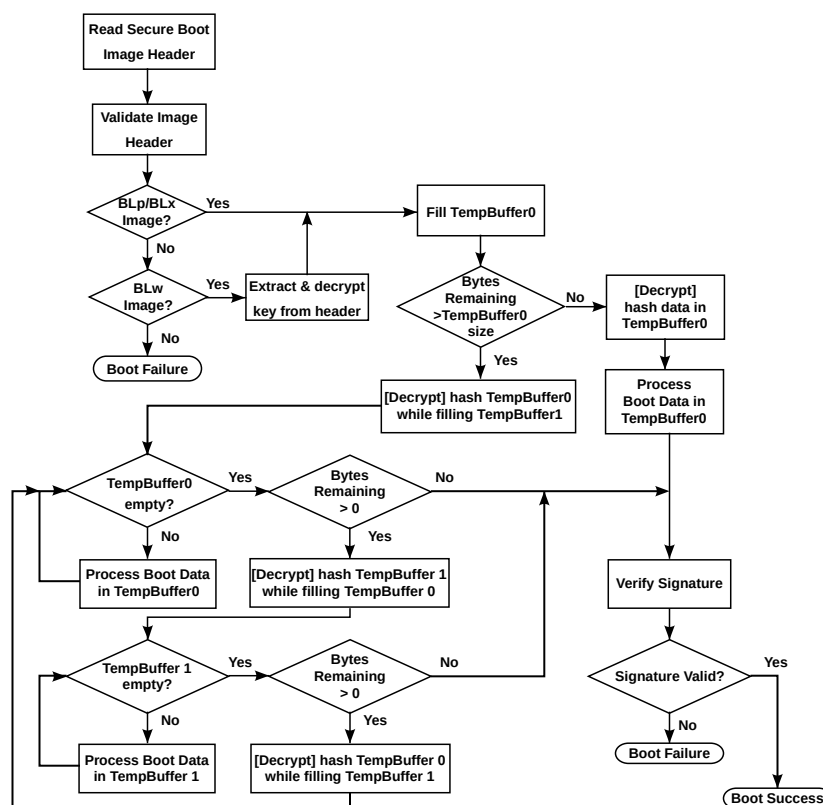


Figure 41-20: Secure Boot Processing

Unsupported Boot Stream Blocks

To ensure the security of the processor, the following block types are not supported in a secure boot image. If the boot kernel finds one of these block types, the boot process terminates.

- **Init Block:** It requires a call to the application code prior to the authentication of the boot image. If customizations or optimizations are necessary to improve the load performance, use a second stage loader style implementation. The first application contains only the custom code. Issue a call using the `api_boot()` routine to boot using the desired device.
- **Call Block:** It requires a call to a user-defined address prior to the authentication of the boot image, and therefore cannot be supported.

NOTE: Secure boot streams use double buffer [Page Mode](#) to optimize the boot process. This functionality allows for the performance of decrypt and hash operations on received data while new data is fetched from the boot source. This host in slave boot mode must ensure that more data is sent after the boot stream to ensure that the temp buffer is filled completely. The size of the secure boot stream minus the size of the secure boot header must be a multiple of the size of the temp buffer. The temp buffer default size is 1024 bytes.

Secure Boot Image Attributes

Secure boot image attributes form part of the secure boot header. The attributes provide more information about the content of the secure boot image.

All image attributes are integrity protected using the same algorithm as the image section. When the image authentication process completes and the image is successfully authenticated, the image attributes are known to be trustworthy.

Attributes are specified as type value pairs with both the type and value being a 32-bit value. The boot code supports the following image attributes.

Table 41-41: Secure Boot Image Attributes

ID	Name	Description	Values	
0x00000000	Unused	Unused attribute	0x00000000	
0x00000001	Version	Version of the attribute format	0x00000000	
0x00000002	Rollback ID	Current value of the rollback counter	0x00000000	Rollback disabled
			0x00000001 - 0x0000001F	Current firmware revision. It must be greater than or equal to the value retrieved from OTP.
0x80000001	NoRestore	Controls whether the boot process restored registers back to default values and clears sensitive security related information from the stack and dedicated structures.	0x1	Do not restore registers and clear security information from stack and dedicated structures.
			Other	Restore registers and clear security sensitive information from the stack and dedicated structures.
0x80000002	BCODE	Used by the boot mode drivers that support auto-detection to configure the device from a range of preconfigured settings	Values between 0x00000000 and 0x0000000F supported.	
0x80000003	ECDSA Type	Used by boot ROM to perform Authentication using ECDSA-224 or ECDSA-256 algorithm.	224 : Authentication using ECDSA-224 algorithm 256 : Authentication using ECDSA-256 algorithm	

Secure Boot Streams

The secure boot image sections contain the code and data for loading to the various memory regions of the processor. The content is in the form of a boot stream format consisting of block headers that provide a descriptor for the associated block payloads.

The image section within the secure boot stream consists of a standard [Boot Loader Stream](#) consisting of block headers and payloads as generated by the supporting tools and utilities.

The secure boot image section can contain application images for just a single core or for multiple cores allowing for a flexible booting strategy.

A single boot image containing program and data for multiple cores permits a full system boot initializing all internal memories without needing a second stage loader approach. If boot images are required to initialize external memories, then a multi-stage loader approach can be required to configure the memory interfaces. The advantage of a single boot image for multiple cores is that authentication and decryption of the boot image requires only one execution. However, it is over a larger boot image.

Multiple single-core boot images permit one core to boot then execute that core's boot image without booting the other cores. This booting strategy can be needed when a single processor must be brought up quickly to deal with some initial tasks before booting the rest of the system. Or, it can be used to initialize extra peripherals to use for external memory interfaces. The core that has previously booted can then control the boot process for the remaining cores.

Single Core Images

Single core boot images are the result of processing a single core's output from the linker and converting it to a compliant boot stream. The resulting boot stream has a single first block at the beginning of the boot stream and a final block at the end of the boot stream.

The [Block Header](#) HDRSIGN field of the resulting boot stream is used to identify which core the image is intended for. This identification is required so that the boot code can set the correct ([RCU_SVECT0](#) - [RCU_SVECT2](#)) register from the target address in the first block once the authentication is successful. When the boot image is loaded and authentication is successful, the boot code jumps to the location stored in the cores corresponding ([RCU_SVECT0](#) - [RCU_SVECT2](#)) register.

This boot stream must then be signed using the secure boot utilities resulting in the final secure boot image for a single core.

A second stage loader option is also available giving maximum flexibility both in terms of using ROM functionality, or creating a custom booting strategy. The simplest option is to have the application use `adi_rom_Boot()` to boot the main application image or indeed a third stage loader when one is required.

The boot stream is generated such that all image contents are in the system address space. Core specific L1 memory sections are converted during the boot stream generation process such that they are loaded through the multiprocessor memory space. The core executing the second stage loader can boot an application image intended for another core.

NOTE: The `ROM_BFLAG_RETURN` flag is set when calling the boot routine for the other cores than the core executing the second stage loader then cleared when loading the cores own image. Failure to do so would result in unintentional behavior.

When the core is used to boot a separately generated boot stream for another core, the `ROM_BFLAG_RETURN` flag should be used when calling `adi_rom_Boot()` to instruct the boot kernel to return to the calling application,

which in this case is back to the core that was running the second stage loader. Failure to set this flag results in the core vectoring to the location in its own (`RCU_SVECT0` - `RCU_SVECT2`) register which is not the intention.

By default when implementing a scheme such as previously described where one processor is responsible for booting images for another core, the other cores in the system will remain in their existing idle state that they should be placed in prior to the boot commencing. To allow the cores to execute new application, the core that is responsible for booting the other cores must reset the cores via the `RCU_CRCTL` then release them again from reset. If there is a requirement to release the core from the reset state, then this must be done within the application code of the running core.

Releasing other cores from reset to run application software while other processors are running a boot process requires careful system design. The drivers used by the boot kernel for the boot process assume the various peripheral and infrastructure resources such as MDMA channels and peripherals are available for use, and are not being used by another core. If the boot routine is being executed while other cores are running applications, then those applications must ensure that all the required boot resources are freed up and remain free in order for the boot process to complete on the remaining cores.

Multi-Core Boot Images

Multi-core boot images are generated as a result of processing the multiple linker output files for multiple cores together to create a single compliant boot stream. The resulting boot stream has a first header block at the beginning of each core boot stream and a single final block at the end of the boot stream. The boot stream must only have a single final block to allow the boot kernel to continue processing the entire boot stream. A final block results in the boot kernel terminating triggering the final public key authentication sequence.

The block headers block code `HDRSIGN` field of the resulting boot stream is used to identify the intended core for the image. This identification is required such that the boot code can set the correct (`RCU_SVECT0` - `RCU_SVECT2`) register when the first block is read to set the application start address for that core. When the boot image is loaded and authentication is successful, the core executing the boot sequence jumps to the location stored in the cores corresponding (`RCU_SVECT0` - `RCU_SVECT2`) register. However, since the boot stream has multiple first blocks present for each of the cores, the (`RCU_SVECT0` - `RCU_SVECT2`) register of the other cores is set for their applications. Upon the booting core running its application, it must release the other cores from reset in order for them to then start running their loaded applications.

If a product supports three or more cores, it is acceptable to create a dual core boot image and then load the other cores later using single core boot images stored elsewhere in the boot source. There are no restrictions on a multi-core boot stream that must contain an application for all the cores in a product.

The resulting multi-core boot stream must then be signed and optionally encrypted resulting in a compliant secure boot stream that can be used to boot a secure device.

Multiple core boot images are advantageous in the fact that applications can be loaded to all cores in the system in a single boot sequence resulting in the requirement to decrypt and authenticate only a single boot image.

NOTE:

If there is a requirement for a multi-core boot stream to load code to memory spaces that require a memory controller to be initialized that is not supported by the boot process a multi-stage booting approach is required that can initialize the peripheral. This boot stream is then authenticated, and executed prior to the loading of the boot stream.

Secure Debug Access

The TAPC controller provides a way to restrict access to secure resources of the processor. Secure access through the debug port is protected through a 128-bit security key that must match a key that has been loaded into OTP for access.

To access a locked processor, the TAPC must allow access to the part. The TAPC only allows access to the part if it is provided with a matching key to the data loaded into its `TAPC_USERKEYn` registers.

With the processor in a locked state, on initial boot the boot ROM reads either the 128-bit `secure_emu_key0` or the 128-bit `secure_emu_key1` from the OTP memory and programs the key into the `TAPC_SDBGKEY0` through the `TAPC_SDBGKEY3` registers before then setting the `TAPC_USERKEY_CTL.USERKEY_VALID` bit. The TAPC is then able to access a matching outside key to allow access. See [Secure Debug Key Processing](#) for more information about secure debug key validation.

CAUTION: A 16 bit non-zero value programmed on both `emu_key0_disable` and `emu_key1_disable` fields in OTP or a key of `0xFFFFFFFF`, `0xFFFFFFFF`, `0xFFFFFFFF`, `0xFFFFFFFF` programmed in both the secure debug key fields provided in OTP results in the boot code bypassing the key load operation entirely. If debug access is then ever required the key must be loaded to the TAPC by the program. If the processor fails to boot (due to corrupted firmware) then there is no debug access. The only way to gain access is to load an authenticated boot image that can then load the required keys prior to attempting to connect with a debugger.

The key is set in OTP using the OTP boot program API to program `secure_emu_key0/secure_emu_key1`. The key is read and loaded by the ROM in the following sequence:

1. Bits 31:0 of the key in OTP are stored to `TAPC_SDBGKEY0` bits 31:0
2. Bits 63:32 of the key in OTP are stored to `TAPC_SDBGKEY1` bits 31:0
3. Bits 95:64 of the key in OTP are stored to `TAPC_SDBGKEY2` bits 31:0
4. Bits 127:96 of the key in OTP are stored to `TAPC_SDBGKEY3` bits 31:0

Once the ROM has loaded the user key, a test key can be provided to the TAPC through JTAG. Refer to the Emulator manual for details for providing the key.

A key failure indication can be detected through the `TAPC_SDBGKEY_STAT` register. The boot code does not check the key status, nor does it enable any associated interrupts to signal key failure. The boot code continues to boot upon a key failure in a secure manner. The key failure status remains intact so that the application loaded can check for a failed challenge on the debug port.

The boot code can be configured to bypass the loading of the key during the boot sequence by setting the value of `secure_emu_key0` or `secure_emu_key1` in the OTP to all ones (see [struct otp_data](#)). In this case, the only way to gain access to the secure resources through the debug port is to load an alternate key using the application. The alternative key must always reside in a secure region of memory. Or, if sent remotely, it should be transmitted over a secure connection.

NOTE: In the ADSP-SC59x processor family, the boot ROM supports two 128-bit secure debug keys, `secure_emu_key0` and `secure_emu_key1`, where only one of them is used by the Boot ROM at a time. Each of the secure debug keys is associated with a bypass key (`emu_key0_disable` or `emu_key1_disable`) which decides the validity of the corresponding key. Any non zero value written into the 16-bit bypass key invalidates that secure debug key.

Failure Analysis

The boot ROM supports failure analysis on locked parts. Calling [adi_rom_otp_fa_enable\(\)](#) activates this feature. The user must activate this feature on a locked part before sending it for failure analysis. Without activation, failure analysis on a locked part is impossible due to security reasons. Do not enable this feature on open parts because it results in boot failure.

Boot ROM Errors and Failures

Any errors encountered while processing a secure boot image results in the ROM jumping to the [Error Handler](#) with a specific error code. The error can be of any boot specific error or product specific error. The below table shows different types of errors which are supported by the Boot ROM.

Warning: In secured booting, an incorrect key supplied during boot does not cause a boot failure and the processor continues to boot as normal. A program that supplies an incorrect key is unable to gain access to any secure resources of the processor.

Table 41-42: Secure Boot Header

Error Name	Error ID	Description
ROM_BOOT_ERR_GENERAL	0x0	General Boot Error
ROM_BOOT_ERR_UNSUPPORTED	0x1	Unsupported configuration (device/mode selection)
ROM_BOOT_ERR_BMODE_INIT	0x2	Initialization failure for the selected boot mode
ROM_BOOT_ERR_BMODE_CONFIG	0x3	Configuration failure for the selected boot mode
ROM_BOOT_ERR_BMODE_HOOK	0x4	Hook function failure
ROM_BOOT_ERR_BMODE_REG_FAILURE	0x5	Bmode registration failure
ROM_BOOT_ERR_SEC_AUTH_FAIL	0x6	Authentication failure
ROM_BOOT_ERR_SEC_DECRYPT	0x7	Decryption Failure
ROM_BOOT_ERR_ID	0x8	The image being loaded has a lower ID than was previously recorded, indicating image ID is out dated

Table 41-42: Secure Boot Header (Continued)

Error Name	Error ID	Description
ROM_BOOT_ERR_CLEANUP	0x9	Cleanup failure
ROM_BOOT_ERR_HDRSGN	0xA	Block header signature failure
ROM_BOOT_ERR_HOOK	0xB	Hook function failure
ROM_BOOT_ERR_HDRCHK	0xC	Header checksum failure
ROM_BOOT_SECURE_ERR_HDRCHK	0xD	Secure header failure
ROM_BOOT_ERR_NOFINAL	0xE	No final block
ROM_BOOT_ERR_BLKFILL	0xF	Failure to process fill block
ROM_BOOT_ERR_BLKCRC	0x10	Failed crc for a block
ROM_BOOT_ERR_DMA	0x12	DMA transfer failure
ROM_BOOT_ERR_CALLBACK	0x13	Callback failure
ROM_BOOT_ERR_NEXTDXE	0x14	Next dxe failure
ROM_BOOT_ERR_PLLCFG	0x15	PLL configuration failure
ROM_BOOT_ERR_INT	0x16	Internal ROM error
ROM_BOOT_ERR_NOKERNEL	0x17	Application returned to the bootrom
ROM_BOOT_ERR_OTPREAD	0x18	Failure to read OTP
ROM_BOOT_ERR_NODEVICE	0x19	No device selected in the bootCommand
ROM_BOOT_ERR_NOBMODE	0x1A	No boot mode selected
ROM_BOOT_ERR_BMODEDIS	0x1B	Boot mode is disabled
ROM_BOOT_ERR_NOPUBKEY	0x1C	No public key
ROM_BOOT_ERR_NOPVTKEY	0x1D	All private keys have been disabled
ROM_BOOT_ERR_SPICFG	0x1E	Failed SPI configuration
ROM_BOOT_ERR_UARTCFG	0x1F	failed UART configuration
ROM_BOOT_ERR_SECURITY_FAILURE	0x20	Failure to initialize secure features
ROM_BOOT_ERR_ECDSATYPE	0x21	Wrong ECDSA Type detected other than 224 and 256
ROM_BOOT_ERR_OTPLOCKBIT	0x22	Error in OTP area holding the Lock and Guard bits
ROM_BOOT_ERR_INVALID_FAEN	0x23	Error due to Failure Analysis Enable bit being set in open part
ROM_BOOT_ERR_SEC_HASH_COMPARE_FAIL	0x24	Error due to hash compare fail between preHash and Computed Hash
ROM_BOOT_ERR_A5_UNDEF_IRQ	0x25	Undefined instruction exception
ROM_BOOT_ERR_A5_SWI_IRQ	0x26	Software interrupt instruction
ROM_BOOT_ERR_A5_PREFETCH_IRQ	0x27	Prefetch abort exception

Table 41-42: Secure Boot Header (Continued)

Error Name	Error ID	Description
ROM_BOOT_ERR_A5_ABORT_IRQ	0x28	Data abort exception
ROM_BOOT_ERR_A5_NORM_IRQ	0x29	
ROM_BOOT_ERR_A5_FAST_IRQ	0x2A	
ROM_BOOT_ERR_SH_RCU_SVECT_IRQ	0x2B	
ROM_BOOT_ERR_SH_PARI_IRQ	0x2C	L1 Parity Error
ROM_BOOT_ERR_SH_ILOPI_IRQ	0x2D	Illegal opcode error
ROM_BOOT_ERR_SH_CB7I_IRQ	0x2E	Software stack (Circular Buffer 7) Overflow
ROM_BOOT_ERR_SH_IICDI_IRQ	0x2F	Unaligned LW/BW access + unintentional CMMR/ SMMR access
ROM_BOOT_ERR_SH_SOVFI_IRQ	0x30	Status loop or mode stack overflow; or PC stack full
ROM_BOOT_ERR_SH_ILADI_IRQ	0x31	Illegal Address Space detected
ROM_BOOT_ERR_SH_IIR2_IRQ	0x32	IIR 2 error
ROM_BOOT_ERR_SH_IIR3_IRQ	0x33	IIR 3 error
ROM_BOOT_ERR_SH_TMZHI_IRQ	0x34	Core Timer error
ROM_BOOT_ERR_SH_BKPI_IRQ	0x35	User Hardware Breakpoint
ROM_BOOT_ERR_SH_FIR_IRQ	0x36	FIR error
ROM_BOOT_ERR_SH_IIR0_IRQ	0x37	IIR 0 error
ROM_BOOT_ERR_SH_SECI_IRQ	0x38	System event controller interrupt
ROM_BOOT_ERR_SH_IIR1_IRQ	0x39	IIR 1 error
ROM_BOOT_ERR_SH_DAI0SPT1	0x3A	
ROM_BOOT_ERR_SH_DAI1SPT0	0x3B	
ROM_BOOT_ERR_SH_DAI1SPT1	0x3C	
ROM_BOOT_ERR_SH_RINSEQI_IRQ	0x3D	Restricted Instruction Sequence
ROM_BOOT_ERR_SH_CB15I_IRQ	0x3E	Circular Buffer 15 Overflow
ROM_BOOT_ERR_SH_TMZLI_IRQ	0x3F	Core Timer (Low Priority Option)
ROM_BOOT_ERR_SH_FIXI_IRQ	0x40	Fixed-point overflow exception
ROM_BOOT_ERR_SH_FLTOI_IRQ	0x41	Floating-point overflow exception
ROM_BOOT_ERR_SH_FLTUI_IRQ	0x42	Floating-point underflow exception
ROM_BOOT_ERR_SH_FLTII_IRQ	0x43	Floating-point invalid exception
ROM_BOOT_ERR_SH_EMULI_IRQ	0x44	Emulator low priority interrupt
ROM_BOOT_ERR_SH_SFT0I_IRQ	0x45	User software interrupt 0
ROM_BOOT_ERR_SH_SFT1I_IRQ	0x46	User software interrupt 1

Table 41-42: Secure Boot Header (Continued)

Error Name	Error ID	Description
ROM_BOOT_ERR_SH_SFT2I_IRQ	0x47	User software interrupt 2
ROM_BOOT_ERR_SH_SFT3I_IRQ	0x48	User software interrupt 3
ROM_BOOT_ERR_OTPBOOT_IRQ	0x49	

Boot ROM Programming Model

This section describes the programming model for booting the processor. The programming model includes booting functions, API calls, and data structures.

Boot Mode Driver API

The kernel provides a mechanism to provide a customization of supported boot modes or for implementation of completely new boot modes as second stage boot loaders. This allows programs to customize booting while still taking advantage of the rest of the booting framework. A custom boot mode may provide support for a peripheral that is not supported for boot by the ROM, or it could support one of the same peripherals but with a different configuration.

All the same security features can be supported when using a custom boot mode.

A full boot mode, as perceived by the boot implementation, is a collection of following functions:

1. Register – installs the driver functions listed below so they can be accessed by the boot process
2. Initialization – initialize the boot source
3. Configuration – configure the boot source
4. Load – read from the boot source
5. Cleanup – called after booting

The boot kernel is only aware of functions 2 through 5 and has a requirement to support the Load function. It is this function that is responsible for fetching the boot stream from the boot peripheral. The other functions are used prior to executing the kernel or for cleaning up after the kernel has completed processing the boot stream.

To install a custom boot mode:

1. Create a first stage boot application to define a Load function
2. Use the [adi_rom_BootKernel\(\)](#) API to call the boot kernel once the boot peripheral and pinmuxing has configured. Ensure all the fields of [struct ADI_ROM_BOOT_CONFIG](#) are configured accordingly prior to performing the call.

The boot mode can use the `pModeData` member of `ADI_ROM_BOOT_CONFIG` to preserve and access shared data across the different function calls if required.

All functions have the following prototype:

```
void apiFunction(ADI_ROM_BOOT_CONFIG* pBootStruct);
```

Another way to support custom boot mode is:

1. Create a first stage boot application to define all Init, Config, Load and Cleanup routines.
2. Use `adi_rom_Boot()` API with hook function installed to update the Init, Config, Load and Clean up functions after the preregister initialization is complete. This is checked by boot ROM to override the above functions inside the `ROM_HOOK_CALL_CAUSE: ROM_HOOK_REG_COMPLETE`

Load Function

The load function is required to read data from the source into the specified destination, according to the parameters given through the configuration `struct` parameter. The structure provides all of the required information read from the block header, or specified by the kernel to read the block header. The load function often makes use of the DMA APIs in order to simplify the load function implementation.

As the kernel processes the stream, it calls the load function to request data. Initially, the request is for the header, then the kernel makes requests according to the block flags it parses. The load function must only read from the device, and write where requested.

Relevant fields within the `ADI_ROM_BOOT_CONFIG` object for the load function are (but not limited to): `uwDataWidth`, `pSource`, `dByteCount`, `pDestination`, `loadType`.

Custom load functions must meet the following requirements.

- Protect against `dByteCount` values of zero
- Use multiple DMA units if `dByteCount` is greater than 65536 and the peripheral does not support byte count transfers greater than 65536
- The `pSource` and `pDestination` pointers must be properly updated after loading.

In slave boot modes, the boot kernel uses the address of the `dArgument` field in the `pHeader` block as the destination for the required dummy DMAs when payload data is consumed from `ROM_BFLAG_IGNORE` blocks. The load function must read the `ARGUMENT` word of the block early in the function (if required).

Initialization/Configuration Function

The Initialization and Configuration functions are called in sequence when calling a boot operation using an already supported boot peripheral via the `adi_rom_Boot()` API. These functions are used to configure the boot peripheral prior to calling the boot kernel. Both functions are called in sequence separated only by a call to a user-defined hook function. This hook function is useful when using built-in boot modes to further customize their function. The initialization and configuration functions are responsible for applying any required settings to any devices in use. For example, pin multiplexing may be required and data or pointers that are used by the load function must be initialized. The specific actions depend on the device and functionality used.

Cleanup Function

The cleanup function is called after the entire boot stream is read, and the kernel has completed its boot mode-specific function. This is only performed when using the `adi_rom_Boot()` API. Resetting any status registers, or device parameters is done to prepare the environment for the execution of the newly loaded application.

Error Handler

This section describes the default error handler for the ROM including information on how to customize the error handling.

The default error handler eventually puts the core into an idle state. This functionality can be overridden by using an Init Block (see [Block Types](#)) to modify the error function point in the `struct ADI_ROM_BOOT_CONFIG` structure. The error handler has access to the entire boot info structure and receives the instruction address that triggered the error.

When a part is locked, and the boot type has not disabled secure boot, only the default error handler is called.

The expected prototype is:

```
void ErrorFunction(ADI_ROM_BOOT_CONFIG* pBootStruct, void *pFailingAddress);
```

The error handler saves the failing address to the `ADI_ROM_BOOT_CONFIG` structure then raises the `INTR_SOFT3` fault signaling a fault condition to the system before then entering an endless loop in the boot rom.

NOTE: When using the `adi_rom_Boot()` function to perform a boot action, programs may need to manually configure the `INTR_SOFT3` fault signaling depending on the previous application software executed. Calling the boot process using `adi_rom_Boot()` does not result in a reset of the SEC and Fault installation and configuration as described in [Preboot Operations](#).

Page Mode

Page operations are beneficial for page oriented boot source devices, and to improve boot performance for secure boot operations. Page mode optimizes memory reads for block organized devices by always reading a page, rather than reading data on demand. Two 1024 byte buffers are used in page mode where the contents of one buffer are processed by the boot kernel while DMA is used to load the next data into the second buffer.

Blocking DMA is used to load the active buffer, forcing the process to pause until the DMA is complete. Non-blocking DMA is used to load the not active buffer, allowing the active buffer to process while loading the new data in parallel.

Page mode can be enabled when calling a boot mode using `adi_rom_Boot()`. Refer to the API documentation for the various modes supported by this API. Additionally programs can set the flag using the `dFlags` function (see `struct ADI_ROM_BOOT_CONFIG`) when using hook functions

NOTE: Do not customize page mode settings from the default installed by the boot process.

Boot Hook Function

The boot software allows installation of callback hooks through the use of the `adi_rom_Boot()` APIs hook function parameter. By using this feature, it is possible to alter the state of the processor, at different stages of the boot process and customize the boot structures to alter the behavior of the boot process.

The hook function must adhere to the following prototype:

```
nt32_t
hookFunction(ADI_ROM_BOOT_CONFIG* pBootconfig,
ROM_HOOK_CALL_CAUSE cause);
```

By modifying settings in the `ADI_ROM_BOOT_CONFIG` structure, many alterations of the boot process can be achieved. Much of the same functionality that is available in an Init Block can be provided through the hook function, with even more flexibility for customization. The hook function is called once after executing the boot modes Init routine then once again after executing the boot modes Config routine. A flag passed to the hook function allows software to determine at which point the call took place to allow for conditional processing to occur at different stages of the setup phase.

The hook function must return a zero value for normal booting to continue. A non-zero return value causes the ROM to omit loading any data and immediately transfer control according to [Boot Termination and Application Execution](#) .

When the hook function is called, a parameter is passed that indicates why the hook function was called. See [enum ROM_HOOK_CALL_CAUSE](#) for more details.

enum ROM_HOOK_CALL_CAUSE

Enumeration Type Declaration: `ROM_HOOK_CALL_CAUSE`

Passed to a user hook routine to indicate the reason of the call.

An optional hook routine is provided as a callback when calling a boot mode via `adi_rom_Boot`. This hook routine is called by the boot software first after the execution of the boot modes initialization routine then again after execution of the boot modes configuration routine. This parameter allows the users routine to identify at which point the call was made allowing the user to perform different actions for each call.

Table 41-43: `ROM_HOOK_CALL_CAUSE` Members

Enumerator	Description
<code>ROM_HOOK_CALL_INIT_COMPLETE</code>	Call was as a result of completion of the boot modes initialization function
<code>ROM_HOOK_CALL_CONFIG_COMPLETE</code>	Call was as a result of the completion of the boot modes configuration function
<code>ROM_HOOK_REG_COMPLETE</code>	Call was as a result of the completion of the boot modes pre-register initialization

Boot Return Feature

The [adi_rom_Boot\(\)](#) API provides a feature to bypass calling the loaded application when boot completes, and to return to the routine that made the call instead. The boot software returns the next address after the last loaded application block in the boot source when this feature is enabled.

To enable this feature, set the `ROM_BFLAG_RETURN` flag in the [adi_rom_Boot\(\)](#) `flags` argument when calling the API.

Boot Termination and Application Execution

When the boot kernel completes the processing of the boot stream, a sequence of events is required to then pass control to the loaded application.

When the boot process is complete, the core is required to vector to the application start address and start executing the newly loaded application. Typically, the first block of a boot stream, which is marked with the `BFLAG_FIRST` flag, contains the address of the application. In a multi-core system there may be multiple first blocks in the boot-stream indicating the start address of the application for each core. The application entry point for each core is loaded into the cores corresponding `RCU_SVECTn` register.

Upon boot completion only the core that performed the boot process will vector and start executing the loaded application. This core must then manage the process of resetting then releasing from the reset the other cores in the system in order to make them execute their newly loaded applications.

Execution of the loaded application can be bypassed when calling the boot mode using [adi_rom_Boot\(\)](#) and setting the `ROM_BFLAG_RETURN` flag.

Table 41-44: Application Entry Point Registers

Core ID	Corresponding RCU_SVECTn Register
0	RCU_SVECT0
1	RCU_SVECT1
2	RCU_SVECT2

Boot ROM OTP Customizations

The boot ROM provides a mechanism through available non-volatile programmable memory (OTP on this processor) to customize different aspects of the boot process. These customizations include: overriding default boot-peripheral instance, overriding default peripheral-timing parameters and disabling boot modes.

Data in OTP memory controls all ROM customization. The [struct ADI_ROM_OTP_BOOT_INFO](#) data structure accounts for most of the options.

CGU Initializations

Refer to [CGU Configuration](#)

Boot Command Customization

Refer to [Boot Command Customization](#)

DMC Configuration

Refer to [DMC Configuration](#)

Secure Boot Customization

All the public and private keys can be invalidated using the various key invalidation fields provided in the `ADI_ROM_OTP_BOOT_INFO` structure. This configuration is useful when a new key is required. The boot ROM always uses the lowest valid key enumeration. If key0 is valid, then it is used, if key0 is invalid and key1 is valid, then key1 is used. Refer to [Secure Boot](#) for details on the secure boot functionality.

Disabling Boot Modes

Refer to [Boot Mode Disable](#)

API Reference

The APIs defined in this section are exposed for general use.

adi_rom_Boot()

Provides access to boot an application at run-time through a supported peripheral.

API Details

```
void * adi_rom_Boot(
    void * pAddress,
    uint32_t flags,
    int32_t blockCount,
    ROM_BOOT_HOOK_FUNC * pHook,
    uint32_t command
)
```

pAddress

Pointer to source address of the boot stream.

flags

Global flags applied to the entire boot process

blockCount

Number of blocks to boot. Zero results in processing until a final block is reached.

pHook

Pointer to user implemented hook function for enabling callbacks during the registering of the boot mode with the boot kernel

command

The boot command defining the boot mode to use, the peripheral instance to boot from as well as some boot mode specific configuration

Returns

The 32-bit address of the next address in the boot source to process

Function Description

This function is used for any kind of second-stage boot for an already supported boot mode. It provides options to boot from any peripheral enumeration. In SPI Master boot the function supports using any SPI slave select signal.

Boot modes may support an auto-detection mechanism to identify the type of connected device. This function provides options to bypass such auto-detection and use custom configuration options. Options are also provided to bypass peripheral configurations such as pinmux settings or a peripheral configuration if an existing peripheral is more appropriately configured to allow communication with the boot source.

These features are all provided via the command parameter which is specific for each particular boot mode.

The source address of the boot stream is required for master boot modes that require that an address is issued in order to request data from the boot source. Slave boot modes are under full control of the host and use a handshake mechanism to indicate that the processor is ready to receive data. For boot modes such as UART Slave and SPI Slave this parameter is of little value in regards to the boot process itself. However it can prove useful in debug to see how far through the boot stream the boot process progressed in the event of a boot failure.

NOTE: The processor supports both SPI Memory-Mapped boot as well as Peripheral based SPI Boot. When the boot mode is called to boot from the memory-mapped boot mode via the command argument, the address must coincide with the processors memory-mapped SPI address space as defined by the processors internal memory map. When using the peripheral based boot mode use the absolute address of the boot stream in flash.

Flags passed using the flags argument are *global* flags and the operation is applied throughout the entire boot process. These must not be confused with the boot *block specific* flags which are part of the boot stream and indicate how a particular block in the boot stream is processed. Internally, the boot kernel takes the global flags supplied using this function call and combines them with a boot block's local flags to determine all the operations to perform on a given block. After processing the boot block the local flags are cleared and are ready to populate from the next boot block while the global flags remain.

Table 41-45: Global Flags

Bit Position	Flag Name	Description
18	ROM_BFLAG_HOOK	Calls the application supplied hook function after execution of bootmode init and config routines
19	ROM_BLAG_PAGEMODE	Enables page mode processing where blocks of data are fetched and processed from internal memory

Table 41-45: Global Flags (Continued)

Bit Position	Flag Name	Description
20	ROM_BFLAG_NOFIRSTHEADER	Set if calling the boot mode and the first block header has already been fetched and is present in the block header storage location
21	ROM_BFLAG_HEADER	Not set by the application, set by the boot code each time it is fetched by a block header
22–25	Reserved	
26	ROM_BFLAG_SLAVE	Slave boot mode. Causes different handling of ignore blocks by the kernel
27	ROM_BFLAG_WAKEUP	Enables conditional processing of boot blocks intended for wakeup events but not exclusively
28	ROM_BFLAG_NEXTDXE	Parses the stream using the next DXE pointer
29	ROM_BFLAG_RETURN	Returns the application after calling the API instead of running the new application
30	ROM_BFLAG_NORESET	No Reset
31	ROM_BFLAG_NORESTORE	Do not execute the boot peripherals cleanup routine to restore register contents

The `blockCount` argument specifies the number of blocks to process before terminating the boot process (default = 0x00000000). This value instructs the boot software to continue processing a boot stream until the `ROM_BFLAG_FINAL` flag is set. To load only a specified number of blocks, programs can use `blockCount` argument.

When `blockCount` is used in combination with the `ROM_BFLAG_NEXTDXE` flag then the block count is repurposed as a next application count. The boot kernel navigates the first blocks of multiple boot streams similar to a linked list. When the requested application count is reached, the program returns the pointer to this application in the boot source. This allows programs to use the boot kernel to find a specific application when multiple application boot streams are stored contiguously in the boot source.

The `pHook` argument is a function pointer to a hook routine. When used with the `ROM_BFLAG_HOOK` global flag the boot mode calls the hook routine after calling the boot modes `init` and functions in the boot mode's driver so that the boot software can be customized and the configuration changed.

The `pHook` argument when used with the `ROM_BFLAG_HOOK` global flag can enable new features not supported by the boot software and to install a custom load function or error handler for example.

The `command` argument describes the boot peripheral to boot from, the peripheral instance and contains additional boot mode specific configuration information and flags specific to the boot mode.

NOTE: When calling a boot mode via this API the program must first ensure that the boot peripheral is configured in the SPU as a secure master. The boot software does not configure the peripheral security via this API so that device security is fully controlled by a dedicated task.

adi_rom_BootKernel()

Calls the boot kernel allowing for implementation of custom boot modes.

API Details

```
void * adi_rom_BootKernel(ADI_ROM_BOOT_CONFIG * pBoot)
```

pBoot

Pointer to the `struct ADI_ROM_BOOT_CONFIG` boot structure containing the complete context of the boot configuration.

Returns

Pointer containing the address of the byte immediately following the end of the boot stream.

Function Description

The boot kernel performs the core processing of the boot stream. The boot kernel calls a load function to load data from the peripheral to the required destination. The boot kernel itself has no concept of what the boot peripheral is or how that peripheral is configured. The kernel calls the registered load function and the load function must then analyze the boot structure and provide the requested amount of data to the required destination.

The load function called by the kernel is provided via the `struct ADI_ROM_BOOT_REGISTRY` member of `struct ADI_ROM_BOOT_CONFIG`.

The boot kernel fetches a boot stream block header then a payload if one exists. The boot kernel takes care of the size of the data being requested and the destination address.

The load function that is registered with the kernel is required to update the `pSource`. Keeping this control under the load function as opposed to the boot kernel itself allows load functions to better control where the next block of data is fetched in the event the boot stream is fragmented or split into different areas of the boot source.

This function is used to implement a second stage boot loader for a peripheral in which there is no driver support in the boot ROM. The application is responsible for initializing the peripheral and the complete `struct ADI_ROM_BOOT_CONFIG` object before calling this function. The application is responsible for performing a vector to the newly loaded application on return from the function.

NOTE: Programs must ensure that when a new application is loaded it does not clobber the load function and the part of the software responsible for making the core jump to the start of the newly loaded application.

adi_rom_Crc32Init()

The CRC32 Initcode function in the boot rom that is called to enable CRC32 support of boot stream payloads.

API Details

```
ROM_BOOT_RESULT adi_rom_Crc32Init(ADI_ROM_BOOT_CONFIG * pBootConfig)
```

pBootConfig

Pointer to the `struct ADI_ROM_BOOT_CONFIG` object containing the complete boot configuration

Returns

Returns the following results

- `enum ROM_BOOT_RESULT` when `pBootConfig` or `pBootConfig->pHeader` are zero
- `enum ROM_BOOT_RESULT` when the callback is registered and lookup table initialized

Function Description

The boot process supports CRC32 protection of all boot block payloads. In order to enable this feature a global callback must be registered with the boot process using `struct ADI_ROM_BOOT_CONFIG` and the CRC peripherals look up table initialized from the application's polynomial.

An init block header where the `ROM_BFLAG_INIT` flag is set must be included in the boot stream to enable CRC functions. The `ADI_ROM_BOOT_HEADER::pTargetAddress` field must be set to the address of this function and the application polynomial is provided using the block's `struct ADI_ROM_BOOT_HEADER` member.

When the boot kernel processes the init block it calls this function in the boot ROM and registers the callback with the kernel and performs the look up table initialization.

CRC functionality is enabled on MDMA channel 1 interfaced to the CRC0 peripheral instance.

adi_rom_Crc32Poly()

Initializes the CRC peripheral for use with the user supplied polynomial.

API Details

```
ROM_BOOT_RESULT adi_rom_Crc32Poly(
    uint32_t CrcPoly,
    ROM_BOOT_MDMA_REGS const *const pDma
)
```

CrcPoly

None

pDma

None

Function Description

The CRC lookup table must be initialized for the CRC polynomial of choice to prepare the CRC peripheral for use.

adi_rom_GetAddress()

Used to find the location of various look-up tables and data objects used during the boot process.

API Details

```
int32_t adi_rom_GetAddress(ROM_GETADDR_VALUE value)
```

value

The `enum ROM_GETADDR_VALUE` enumeration specifying the object to retrieve the address of in the ROM memory

Returns

The byte address of the object in memory

Function Description

The function returns the address of the object specified by the enumerator provided as an argument to the function. Using this function can make software more code compatible with future products and silicon revisions.

adi_rom_MemCompare()

Verifies that a block of data is filled with a application supplied 32-bit value.

API Details

```
ROM_BOOT_RESULT adi_rom_MemCompare (
    ROM_DMA_MDMA_CONFIG * pDmaCfg,
    ROM_BOOT_MDMA_REGS const *const pDma
)
```

pDmaCfg

None

pDma

None

Function Description

The CRC peripheral is used in compare mode and a source MDMA channel is used to read data from a buffer and supply each 32-bit value to the CRC. The CRC peripheral checks that the incoming 32-bit value matches the 32-bit value to compare against.

adi_rom_MemCopy()

Performs a Memory-to-Memory DMA (MDMA) operation using a source and destination pair of DMA channels.

API Details

```
ROM_BOOT_RESULT adi_rom_MemCopy (
    ROM_DMA_MDMA_CONFIG * pDmaCfg,
```

```
ROM_BOOT_MDMA_REGS const *const pDma
)
```

pDmaCfg

Pointer to the `struct ROM_DMA_PDMA_CONFIG` object containing the peripheral DMA configuration

pDma

Pointer to the `struct ROM_BOOT_MDMA_REGS` objects that provides access to the DMA channel MMRs and associated CRC peripheral

Returns

Returns the following results

- `enum ROM_BOOT_RESULT` for a successful operation or when byte count is 0 as no operation is performed
- `enum ROM_BOOT_RESULT` if a configuration error was detected in the DMA channel prior to configuring the channels for the new operation
- if a configuration error occurred in the source MDMA channel
- if a configuration error occurred in the destination MDMA channel

Function Description

The memory copy routine performs transfers of blocks of data from one memory location to another. The routine takes a basic descriptor providing configuration details of the operation to perform using the `struct ROM_DMA_MDMA_CONFIG` object passed as the first argument. The second argument is a descriptor that provides access to the DMA channel's MMR registers and the associated CRC peripheral. When called from the higher level `adi_rom_MemDma()` routine this object is retrieved from the ROM.

NOTE: Applications are expected to make use the `adi_rom_MemDma()` routine for all MDMA operations as there is little additional optional configuration that is supported by using this routine.

adi_rom_MemCrc()

Performs CRC32 verification of a block of data by reading the contents and comparing them with an expected result.

API Details

```
ROM_BOOT_RESULT adi_rom_MemCrc (
    ROM_DMA_MDMA_CONFIG * pDmaCfg,
    ROM_BOOT_MDMA_REGS const *const pDma
)
```

pDmaCfg

Pointer to the `struct ROM_DMA_PDMA_CONFIG` object containing the peripheral DMA configuration

pDma

Pointer to the `struct ROM_BOOT_MDMA_REGS` objects that provides access to the DMA channel's MMRs and associated CRC peripheral

Function Description

The routine uses an MDMA channel pair source DMA channel and the CRC peripheral to calculate a CRC32 result of a data block using a previously supplied polynomial.

The polynomial is supplied through the `adi_rom_Crc32Poly()` routine to ensure consistent CRC peripheral configuration for the look up table initialization that uses the polynomial and the

NOTE: Applications are expected to make use the `adi_rom_MemDma()` routine for all MDMA operations, there is little additional optional configuration that is supported by using this routine.

adi_rom_MemDma()

Provides access to all MDMA operations supported by the boot ROM implementation.

API Details

```
ROM_BOOT_RESULT adi_rom_MemDma(ROM_DMA_MDMA_CONFIG * pDmaCfg)
```

pDmaCfg

Pointer to the `struct ROM_DMA_PDMA_CONFIG` object containing the MDMA configuration

Returns

Returns the following results using the `enum ROM_BOOT_RESULT` enumeration.

- `ROM_BOOT_SUCCESS` Success. General success can be used to indicate any general functional success for an operation during the boot process. This must be the return result for a boot mode drivers initialization, configuration, load and cleanup routines when overriding their functionality in second stage boot loaders to use custom functions. . Successful operation or when byte count is 0 as no operation is performed
- `ROM_BOOT_MDMA_ID_ERR` Illegal MDMA Channel ID. Returned by `adi_rom_MemDma()` if the MDMA channel ID is not supported. For supported channel IDs, see `ROM_DMA_MDMA_ID` . A MDMA channel ID is provided that is not supported
- `ROM_BOOT_CRC_SUPPORTED_ERR` CRC Not Supported Error. Returned by `adi_rom_MemDma()`, `adi_rom_MemFill()`, `adi_rom_MemCompare()` and `adi_rom_Crc32Poly()` if the supplied DMA configuration specified a MDMA channel that does not support CRC operations. . A CRC operation was attempted on a MDMA channel that does not support CRC

- `ROM_BOOT_MDMA_OPERATION_ERR` Illegal MDMA operation Specified. Returned by `adi_rom_MemDma()` if the MDMA operation is not supported. For supported operations, see `ROM_DMA_MDMA_OPERATION`. The operation is not supported
- `ROM_BOOT_DMA_FAILURE` DMA Failure. Returned by the DMA routines if an error was detected in the `DMA_STAT.IRQERR` prior to setting up a new DMA operation with the newly supplied configuration. . A configuration error is detected in the DMA channel prior to configuring the channel for the new operation
- `ROM_BOOT_DMA_FAILURE` DMA Failure. Returned by the DMA routines if an error was detected in the `DMA_STAT.IRQERR` prior to setting up a new DMA operation with the newly supplied configuration. . A configuration error was detected in the DMA channel and the operation requested involves only a single DMA channel
- A configuration error occurred in the source MDMA channel
- A configuration error occurred in the destination MDMA channel
- `ROM_BOOT_CRC_COUNT_ERR` CRC Byte Count was not a multiple 4. The CRC peripheral operates on 32-bit data only and as such all CRC operations must have a byte count that is a multiple of 4. This result is returned by the higher level `adi_rom_MemDma()` routine and the underlying `adi_rom_MemCompare()` and `adi_rom_MemFill()` routines if the byte count is not a multiple of 4 bytes. . A CRC operation is being requested and the byte count is not a multiple of 4
- `ROM_BOOT_CRC_FAILURE` MDMA CRC32 Failure. Returned by the higher level `adi_rom_MemDma()` routine and the underlying `adi_rom_MemCompare()` routine if the CRC32 result of the block of data did not match the expected result. . A CRC32 verification fails
- `ROM_BOOT_CRC_FAILURE` MDMA CRC32 Failure. Returned by the higher level `adi_rom_MemDma()` routine and the underlying `adi_rom_MemCompare()` routine if the CRC32 result of the block of data did not match the expected result. . A 32-bit memory compare fails

Function Description

The supported MDMA operations are listed in `enum ROM_DMA_MDMA_OPERATION`.

NOTE: The MDMA and CRC peripherals support on-the-fly CRC32 calculations during the transfer of data from one location to another—MDMA in the boot ROM software does not. For CRC calculations data is instead read back from its final destination and verified.

This function is the main entry point for all the MDMA functions supported by the boot ROM software. The individual functions that are called for each operation type are also exposed using the public API.

NOTE: Use this function for all operations. The lower level functions allow for some basic reconfiguration of default parameters but such modifications are not be required where these basic MDMA operations are required.

The boot ROM has an MDMA configuration data structure that is used to specify the overall MDMA configuration of the processor. It provides details on the DMA channel ID associated with each MDMA channel's source and destination DMA channel. It provides information about CRC support and the CRC peripheral instance that is used for a given MDMA channel. See `struct ROM_BOOT_MDMA` and `struct ROM_BOOT_MDMA_REGS` for full details of the content stored.

The configuration provided as the only argument to the function is provided in [Table 41-65 ROM_DMA_MDMA_CONFIG Members](#).

For a basic MDMA transfer from source to destination the program needs to configure:

- The `struct ROM_DMA_MDMA_CONFIG` type as `enum ROM_DMA_MDMA_OPERATION`
- The MDMA channel to use via `eId` MDMA Channel ID for example
- Set the address of the source data via `pSource` Source Pointer
- Set the destination address of the source data via `pDestination` Destination Pointer
- Set the byte count via `ByteCount` Byte Count
- Set `eDoneDetect` DMA Done Detection Method to `enum ROM_DMA_DONE_DETECT_METHOD` in order to poll for DMA completion

NOTE: The implementation of the MDMA operations does not support interrupt driven data transfers. The routines are implemented for polling DMA status during the boot process. The boot stream is restricted where byte counts and source and destination address alignment must all be a multiple of 4 bytes. MDMA routines must comply with these restrictions.

To use the CRC32 function of the MDMA routines, the application must first initialize the CRC lookup table from the application supplied polynomial. This operation can be performed by setting

`ROM_DMA_MDMA_CONFIG::eOperation` type as

`ROM_DMA_MDMA_OPERATION::ROM_DMA_CRC_LUT_INIT`. If an MDMA channel is specified that does not support CRC function, an error result is returned.

For further details of the individual operations supported see the following API references:

- `adi_rom_MemCopy()`
- `adi_rom_MemCrc()`
- `adi_rom_MemFill()`
- `adi_rom_MemCompare()`
- `adi_rom_Crc32Poly()`

adi_rom_MemFill()

Fills a block of memory with a 32-bit user supplied value.

API Details

```
ROM_BOOT_RESULT adi_rom_MemFill(
    ROM_DMA_MDMA_CONFIG * pDmaCfg,
    ROM_BOOT_MDMA_REGS const *const pDma
)
```

pDmaCfg

None

pDma

None

Returns

Returns the following results

- `enum ROM_BOOT_RESULT` for a successful operation or when byte count is 0 as no operation to be performed
- `enum ROM_BOOT_RESULT` if a configuration error was detected in the DMA channel prior to configuring the channel for the new operation
- A configuration error occurred in the source MDMA channel
- A configuration error occurred in the destination MDMA channel

Function Description

The CRC peripheral is configured for fill mode and the destination MDMA channel is configured to fill a block of memory with a fixed 32-bit value.

adi_rom_PeriphDma()

Provides access to any peripherals dedicated DMA channel for receive operations only.

API Details

```
ROM_BOOT_RESULT adi_rom_PeriphDma(ROM_DMA_PDMA_CONFIG * pDmaCfg)
```

pDmaCfg

Pointer to the `struct ROM_DMA_PDMA_CONFIG` object containing the peripheral DMA configuration

Returns

Returns the following results

- `ROM_BOOT_SUCCESS` Success. General success can be used to indicate any general functional success for an operation during the boot process. This must be the return result for a boot mode drivers initialization, configuration, load and cleanup routines when overriding their functionality in second stage boot loaders to use custom functions. for a successful operation or when byte count is 0 as no operation is performed
- `ROM_BOOT_DMA_ACTIVE` DMA Channel is Active Returned only by the peripheral DMA routine when an attempt to run another peripheral DMA operation is attempted and the DMA channel is already running. This is not currently implemented for MDMA operations. if the DMA channel is currently running
- `ROM_BOOT_DMA_FAILURE` DMA Failure. Returned by the DMA routines if an error was detected in the `DMA_STAT.IRQERR` prior to setting up a new DMA operation with the newly supplied configuration. if a configuration error was detected in the DMA channel after starting the DMA operation

Function Description

The peripheral DMA routine is used by the load routines of boot peripherals that have dedicated DMA channels and do not support MDMA channel pairs. Examples are the SPI when not configured for memory-mapped mode and UART peripherals.

In the boot implementation this routine is called from the peripheral load function to request data from the boot source. The routine supports both polling on DMA completion and non-blocking operation to allow for immediate return after starting the DMA operation and continuing with further processing.

NOTE: The function only supports read operations from the peripheral to memory. Transmit operations from memory to peripheral are not supported

adi_rom_otp_cfg()

Configures the OTPC to enable read and program operations.

API Details

```
bool adi_rom_otp_cfg(void)
```

Function Description

Programs may call this routine to ensure the OTPC is configured correctly for read and write access.

NOTE: The preboot process configures the OTPC for use and there is no direct requirement to call this function when using the OTP.

adi_rom_otp_get()

Reads the field from OTP as defined by the supplied [enum OTPCMD](#).

API Details

```
bool adi_rom_otp_get(  
    OTPCMD cmd,  
    uint32_t data[]  
)
```

cmd

The [enum OTPCMD](#) enumeration describes the OTP content to read

data[]

Pointer to storage area to store the read OTP contents

Function Description

Programs can read the various fields ([Table 41-70 OTPCMD Members](#)) of the OTP via this routine. The supplied [enum OTPCMD](#) object is used to specify the object to read.

adi_rom_otp_lock()

Locks the processor, enabling all security features.

API Details

```
bool adi_rom_otp_lock(void)
```

Function Description

This function is used to lock the processor from unauthorized access. Once this function is called the application must supply a secure debug key in order to gain access to the device with debug tools and the processor may only be booted using a secure boot stream.

WARNING: Programs must ensure that the OTP secure boot fields are all programmed. Secure boot can be verified prior to locking the processor. Programs must also provision a secure debug key.

adi_rom_otp_fa_enable()

Enables failure analysis feature for a locked part

API Details

```
bool adi_rom_otp_fa_enable(void)
```

Function Description

This function sets the bit in OTP and enables failure analysis on the locked part.

WARNING: This API should not be called or executed on an open part; it will result in boot failure.

adi_rom_otp_pgm()

Programs the OTP Memory with the contents of the `struct otp_data` object.

API Details

```
bool adi_rom_otp_pgm(otp_data * data)
```

data

Pointer to the `::otp_data` object containing the complete OTP contents to program

Function Description

The OTP memory is only programmed with values that are not 0. Any items that are 0 are ignored. Programs are expected to use this function for all OTP program operations.

callback()

Implements custom callbacks to previously loaded code during boot.

API Details

```
ROM_BOOT_RESULT callback(
    ADI_ROM_BOOT_CONFIG * pBootConfig,
    ADI_ROM_BOOT_BUFFER * pBuffer,
    uint32_t nFlags
)
```

pBootConfig

Pointer to the `struct ADI_ROM_BOOT_CONFIG` object containing the complete context of the boot procedure

pBuffer

Pointer to the `struct ADI_ROM_BOOT_BUFFER` object containing details of the payload associated with the callback

nFlags

The callback flags as set by the boot kernel

Function Description

A single callback function may be registered with the boot kernel using `pCallbackFunction` Pointer to the callback function that is called when processing boot blocks with the callback flag set. This function is then called whenever a block is processed with the `ROM_BFLAG_CALLBACK` flag set. Only a single callback function can be registered for the complete boot process.

Callbacks may be used alongside indirect blocks when post processing of the received boot data is required before sending the data to the final destination. An example of this is compression applied to block payloads. The compressed payload is loaded indirectly to the intermediate buffer where it is decompressed by the callback. The callback can modify the source address and byte count for the final MDMA transfer of the decompressed payload using the `pBuffer` parameter. The callback returns the boot kernel then handles the final transfer of the uncompressed data to its destination.

There are restrictions on the amount of data that can be loaded using indirect blocks depending on the size of the intermediate buffer. For this reason the `nFlags` parameter is used to indicate the status of the callback when handling larger blocks of indirect data. The table below defines the supported flags:

Bit Position	Flag Name	Description
0	<code>ROM_CBFLAG_DIRECT</code>	When set indicates the call was from the processing of a block header with the <code>ROM_BFLAG_CALLBACK</code> flag set
1	<code>ROM_CBFLAG_PAGESTART</code>	Indicates the callback was a result of a fetch of a page of data to the intermediate buffers
2	<code>ROM_CBFLAG_FIRST</code>	Set if the first fetch of payload data
3	<code>ROM_CBFLAG_FINAL</code>	Set if the final fetch of payload data
31:4	Reserved	Reserved

When a callback block header is received by the boot kernel, a call to the callback is performed with the `ROM_CBFLAG_DIRECT` flag set. If the `ROM_BFLAG_INDIRECT` flag or the `ROM_BFLAG_PAGEMODE` flags are set they indicate the use of indirect or page mode the `ROM_CBFLAG_FIRST` and `ROM_CBFLAG_FINAL` flags are cleared. If the transfer is a direct transfer straight to the final destination and not via the intermediate buffers, then the `ROM_CBFLAG_FIRST` and `ROM_CBFLAG_FINAL` flags are also set.

This allows software to identify a callback call based on the processing of a block header with the `ROM_BFLAG_CALLBACK` flag set.

Callbacks are also called when processing payloads indirectly or when page mode is enabled.

- The `ROM_CBFLAG_DIRECT` flag is cleared when the callback is a result of processing the payload data using the intermediate buffers.
- The `ROM_CBFLAG_FIRST` flag is set if the callback is a result of fetching the first block of data in the payload. This flag is also set if the complete block of data fits in the intermediate buffer.

- If the payload does not fit completely in the intermediate buffers multiple fetches must take place and thus multiple callbacks are generated.
- If no flags are set it indicates a callback on a payload transfer is neither the first nor the last block of data in the payload, so there is still further data in the payload to fetch.
- If only the ROM_CBFLAG_FINAL flag is set then it is the final block in a payload transfer.

The following table provides an overview of the flag states and their meaning for the processing of callbacks.

ROM_CBFLAG_DIRECT	ROM_CBFLAG_PAGESTART	ROM_CBFLAG_FIRST	ROM_CBFLAG_FINAL	Description
1	0	0	0	Callback as a result of processing a block header with indirect or page-mode enabled
1	0	1	1	Callback as a result of processing a block header with indirect and pagemode disabled
0	1	0	0	Callback as a result of fetching a page of data in pagemode
0	1	0	1	Callback as a result of fetching a page of data in pagemode and the final page in the block
0	0	1	0	Callback as a result of fetching the first part of payload in an indirect payload
0	0	0	0	Callback as a result of fetching an indirect payload, not first or last transfer in payload
0	0	0	1	Callback as a result of fetching the final part of payload in an indirect payload
0	0	1	1	Callback as a result of fetching the complete payload in an indirect payload

initcode()

Implements custom callbacks to previously loaded code during boot.

API Details

```
void initcode(ADI_ROM_BOOT_CONFIG * pBootConfig)
```

pBootConfig

Pointer to the `struct ADI_ROM_BOOT_CONFIG` object containing the complete context of the boot procedure

Function Description

Initcode functions are embedded into the boot stream to execute application code during the boot phase. Initcode functions help to optimally configure the CGU or any external memory interface that requires initialization to boot data to those memories.

A boot stream may have any number of initcodes present. The only requirement is that the code must be loaded prior to processing the `BFLAG_INIT` block.

The initcode routine is passed by the pointer to the complete boot context so that extensive boot customization tasks can be supported.

adi_rom_idle_loop()

Jumps to the IDLE loop inside the boot ROM space.

API Details

```
void adi_rom_idle_loop(void)
```

Function Description

This function is used to jump to the forever idle loop inside the boot ROM space.

adi_rom_CguInit()

Can be used for CGU initialization.

API Details

```
bool adi_rom_CguInit(const ADI_ROM_OTP_BOOT_CGU_INFO * const pSettings)
```

pSettings

Pointer to the struct `struct ADI_ROM_OTP_BOOT_CGU_INFO` object containing complete context of CGU initialization

Function Description

This function is used for CGU initialization.

adi_rom_DmcPhyCalibration()

Used for DMC PHY calibration.

API Details

```
void adi_rom_DmcPhyCalibration(ADI_ROM_OTP_DMC_CONFIG *pConfig, int csel_dsel_r)
```

pConfig

Pointer to the struct `struct ADI_ROM_OTP_DMC_CONFIG` object containing the complete context of the DMC PHY calibration

csel_dsel_r

Holds the CSEL to DSEL ratio

Function Description

This function is used for DMC PHY calibration.

adi_rom_DmcInit()

Used for DMC initialization.

API Details

```
void adi_rom_DmcInit(ADI_ROM_OTP_DMC_CONFIG *pConfig, uint32_t csel_dsel_r)
```

pConfig

Pointer to the struct `struct ADI_ROM_OTP_DMC_CONFIG` object containing the complete context of the DMC PHY calibration.

csel_dsel_r

Holds the CSEL to DSEL ratio.

Function Description

This function is used for DMC PHY calibration.

adi_rom_Shalnit()

Initializes the PKTE module for the SHA-224/256 operation.

API Details

```
void adi_rom_DmcInit(ADI_ROM_OTP_DMC_CONFIG *pConfig, uint32_t csel_dsel_r)
```

pCrypto

Pointer to Crypto descriptors

IV

Initialization Vector

Ecdsa_Word

ECDSA word count for 224/256 bit operation

Function Description

This API is used for the PKTE initialization for the SHA-224/256 bit operation

adi_rom_Sha()

Computes SHA-224/256 bit digest from the PKTE module.

API Details

```
adi_rom_Sha(CRYPTO_DESCRIPTOR * pCrypto, uint32_t Ecdsa_Word, void *output_p,
void *input_p, int size, bool finalFlag)
```

pCrypto

Pointer to Crypto descriptors

Ecdsa_Word

ECDSA word count for 224/256 bit operation

output_p

Pointer to the output buffer

input_p

Pointer to the input buffer

size

Size of the input buffer. It must be a multiple of 512

finalFlag

Final block or not

Function Description

This API is used to compute the message digest using SHA-224/256 using the PKTE module.

/ points to bootrom_jumtable_aes128_cbc_decrypt */*

```
typedef ROM_BOOT_RESULT (*_bootrom_jumtable_aes128_cbc_decrypt_t)
(CRYPTO_DESCRIPTOR
    * pCrypto, void *output_p, void *input_p, int size);
inline static ROM_BOOT_RESULT
adi_rom_Aes128CbcDecrypt(CRYPTO_DESCRIPTOR * pCrypto,
```



```

        void *output_p, void *input_p, int size){
    _bootrom_jumtable_aes128_cbc_decrypt_t pTmp =
        (_bootrom_jumtable_aes128_cbc_decrypt_t)
(FUNC_ROM_AES128_CBC_DECRYPT);
    return (*pTmp)(pCrypto, output_p, input_p, size);
}

```

/* points to bootrom_jumtable_aes128_key_unwrap */

```

        typedef ROM_BOOT_RESULT (*_bootrom_jumtable_aes128_key_unwrap_t)
(CRYPTO_DESCRIPTOR
    * pCrypto, void *WrappedIn_p, void *KeyOut_p, uint32_t KeyBits,
void *KWK_p);
    inline static ROM_BOOT_RESULT
adi_rom_Aes128KeyUnwrap(CRYPTO_DESCRIPTOR * pCrypto,
    void *WrappedIn_p, void *KeyOut_p, uint32_t KeyBits, void *KWK_p){
    _bootrom_jumtable_aes128_key_unwrap_t pTmp =
        (_bootrom_jumtable_aes128_key_unwrap_t)
(FUNC_ROM_AES128_KEY_UNWRAP);
    return (*pTmp)(pCrypto, WrappedIn_p, KeyOut_p, KeyBits, KWK_p);
}

```

/* points to bootrom_jumtable_aes128_cbc_loadkey */

```

        typedef void * (*_bootrom_jumtable_aes128_cbc_loadkey_t)(sa_t *
pSARRecord, uint32_t
    *key_p);
    inline static void * adi_rom_Aes128CbcLoadkey(sa_t * pSARRecord,
uint32_t *key_p)
    {
        _bootrom_jumtable_aes128_cbc_loadkey_t pTmp =
            (_bootrom_jumtable_aes128_cbc_loadkey_t)
(FUNC_ROM_AES128_CBC_LOADKEY);
        return (*pTmp)(pSARRecord, key_p);
    }

```

/* points to bootrom_jumtable_pka_init */

```

        typedef PKA_Status_t (*_bootrom_jumtable_pka_init_t)( PKA_IOArea_t *
const IOArea_p,
    Device_Handle_t Device, const uint32_t * const Firmware_p, const
uint32_t
    FirmwareWordCount);
    inline static PKA_Status_t adi_rom_PKA_Init( PKA_IOArea_t * const
IOArea_p,
    Device_Handle_t Device, const uint32_t * const Firmware_p, const
uint32_t
    FirmwareWordCount) {
        _bootrom_jumtable_pka_init_t pTmp =

```

```

        (_bootrom_jumtable_pka_init_t) (FUNC_ROM_PKA_INIT);
    return (*pTmp) (IOArea_p, Device, Firmware_p, FirmwareWordCount);
}

```

/* points to bootrom_jumtable_ecdsa_verify_init */

```

typedef ROM_BOOT_RESULT
    (*_bootrom_jumtable_ecdsa_verify_init_t) (SBHYBRID_EcdsaContext_t
* const Verify_p,
        const SBIF_ECDSA_PublicKey_t * const PublicKey_p, const
SBIF_ECDSA_Signature_t *
        const Signature_p, uint32_t EcdsaWord);
inline static ROM_BOOT_RESULT
adi_rom_EcdsaVerifyInit (SBHYBRID_EcdsaContext_t * const
        Verify_p, const SBIF_ECDSA_PublicKey_t * const PublicKey_p, const
        SBIF_ECDSA_Signature_t * const Signature_p, uint32_t EcdsaWord) {
    _bootrom_jumtable_ecdsa_verify_init_t pTmp =
        (_bootrom_jumtable_ecdsa_verify_init_t)
(FUNC_ROM_ECDSA_VERIFY_INIT);
    return (*pTmp) (Verify_p, PublicKey_p, Signature_p, EcdsaWord);
}

```

/* points to bootrom_jumtable_ecdsa_verify_set_digest */

```

typedef void *
    (*_bootrom_jumtable_ecdsa_verify_set_digest_t)
(SBHYBRID_EcdsaContext_t * const
        Verify_p, uint8_t * Digest_p);
inline static void *
adi_rom_EcdsaVerifySetDigest (SBHYBRID_EcdsaContext_t * const
        Verify_p, uint8_t * Digest_p) {
    _bootrom_jumtable_ecdsa_verify_set_digest_t pTmp =
        (_bootrom_jumtable_ecdsa_verify_set_digest_t)
(FUNC_ROM_ECDSA_VERIFY_SET_DIGEST);
    return (*pTmp) (Verify_p, Digest_p);
}

```

/* points to bootrom_jumtable_ecdsa_verify */

```

typedef SB_Result_t (*_bootrom_jumtable_ecdsa_verify_t)
(SBHYBRID_EcdsaContext_t *
        const Verify_p, uint32_t EcdsaWord);
inline static SB_Result_t adi_rom_EcdsaVerify (SBHYBRID_EcdsaContext_t *
const
        Verify_p, uint32_t EcdsaWord) {
    _bootrom_jumtable_ecdsa_verify_t pTmp =
        (_bootrom_jumtable_ecdsa_verify_t) (FUNC_ROM_ECDSA_VERIFY);
    return (*pTmp) (Verify_p, EcdsaWord);
}

```

Data Structures

The programming model for booting the processor uses the data structures defined in this section.

struct ADI_ROM_BOOT_BUFFER

Structure Type Declaration: ADI_ROM_BOOT_BUFFER

Boot Buffer.

A basic buffer type consisting of a pointer to the buffer and its size

Table 41-46: ADI_ROM_BOOT_BUFFER Members

Type	Name	Description
void *	pBuffer	Pointer to the buffer
int32_t	dByteCount	Size of the buffer

pBuffer

Pointer to the buffer

dByteCount

Size of the buffer

struct ADI_ROM_BOOT_CONFIG

Structure Type Declaration: ADI_ROM_BOOT_CONFIG

The Boot Configuration Object that contains all context for the boot process.

This structure contains the complete context for the boot process. A pointer to this object is passed through many routines and is presented to customizable routines such as initcodes, custom initialization, configuration, load and cleanup routines. The object is passed to error handlers and callbacks to customize and adapt the boot process for specific applications, especially in regards to multi-stage boot loader development.

Table 41-47: ADI_ROM_BOOT_CONFIG Members

Type	Name	Description
void *	pSource	Source address from where to fetch the next boot data.
void *	pDestination	Destination address to store the fetched data.
int32_t	dByteCount	Number of bytes to fetch from the boot source.
int32_t	dFlags	Control flags related to the boot kernel processing of blocks.

Table 41-47: ADI_ROM_BOOT_CONFIG Members (Continued)

Type	Name	Description
uint32_t	ulBlockCount	Limit of blocks processed during boot.
uint32_t	ulBlockCurrent	The number of blocks currently processed by the boot kernel
void *	pNextDxe	Pointer to the next application in the boot stream or the first free location after the boot stream.
uint32_t	uByteAddress	The destination address converted to the byte address space.
uint32_t volatile *	pControlRegister	Pointer to the boot peripherals control register.
int32_t	dControlValue	Storage for the boot peripheral main control value to enable that peripheral in a required configuration.
uint32_t volatile *	pPeripheralBase	Pointer to the boot peripherals base MMR address
uint32_t volatile *	pAuxControlRegister	Pointer to any register that may be used for auxiliary operations such as a timer control register for UART auto-baud detection
uint32_t volatile *	pAuxPeripheralBase	Pointer to the base address of any peripheral used for auxiliary operations such as the TIMER block
uint32_t volatile *	pSecControlRegister	Base MMR address of the SEC SSI instance associated with the boot peripheral if required for advanced second stage boot loader development
ADI_DMA_TypeDef *	pDmaBaseRegister	Base MMR address of the DMA channel associated with the boot peripheral.
ROM_DMA_DONE_DETECT_METHOD	loadType	Set by the kernel to specify to the boot peripherals load function if it is requesting a blocking or non-blocking DMA
ROM_DMA_MDMA_CONFIG	MdmaCfg	An MDMA descriptor that is used by the boot kernel for internal MDMA operations.
uint16_t	uwDataWidth	The maximum data width supported by the boot peripherals DMA channel. Set to 0 for 8-bit, 1 for 16-bit and 2 for 32-bit
uint16_t	uwSrcModifyMult	The source modify multiplier used to set DMA_XMOD for source MDMA operations or peripheral DMA transmit operations
uint16_t	uwDstModifyMult	The destination modify multiplier used to set DMA_XMOD for destination MDMA operations or peripheral DMA receive operations
uint16_t	uwUserShort	Free to use
int32_t	dUserLong	Free to use
int32_t	dReserved0	Reserved for future use

Table 41-47: ADI_ROM_BOOT_CONFIG Members (Continued)

Type	Name	Description
void *	pModeData	Pointer to the boot mode specific data structure.
int32_t	dBootCommand	The boot command value supplied during the call to the <code>adi_rom_boot()</code> routine.
ADI_ROM_BOOT_HEADER*	pHeader	Pointer to the boot header storage location where all boot stream block headers eventually reside for processing by the kernel.
void *	pTempBuffer	Pointer to the internal intermediate buffer. Used for processing of indirect blocks.
void	dReserved1	Reserved
int32_t	dTempByteCount	Size of the internal intermediate buffer in bytes.
void *	pTempSource	Current source address that is processed in the internal intermediate buffer.
int32_t	dPageByteCount	The page size used for page mode processing. On this product page size is fixed to 1024 bytes and this member is not used for the load requests.
ADI_ROM_BOOT_INTER_BUFFERS	bootBuffers	The internal intermediate buffer descriptors required when using indirect and page mode features.
ROM_BOOT_REGISTRY	bootRegistry	The registry object that is used to register a boot peripheral routines with the kernel.
ROM_BOOT_ERROR_FUNC *	pErrorFunction	Pointer to the error handler called in the event of an error.
ROM_BOOT_CALLBACK_FUNC *	pCallbackFunction	Pointer to the callback function that is called when processing boot blocks with the callback flag set.
ROM_BOOT_CALLBACK_FUNC *	pCrcFunction	Pointer to the CRC function that is used to perform CRC validation of the boot stream payload data.
ROM_BOOT_CALLBACK_FUNC *	pForwardFunction	Feature not supported on this product.
ADI_ROM_BOOT_MODES	bootModes	Access to all boot mode specific resources.
void *	pLogBuffer	Pointer to the log buffer. Logging is disabled by default on this product and must be configured from within initcodes or hook routines.
void *	pLogCurrent	The current position within the log buffer. Logging is disabled by default on this product and must be configured from within initcodes or hook routines.
int32_t	dLogByteCount	The size of the log buffer. Logging is disabled by default on this product and must be configured from within initcodes or hook routines.

Table 41-47: ADI_ROM_BOOT_CONFIG Members (Continued)

Type	Name	Description
ADI_ROM_OTP_BOOT_INFO*	pOtpBootInfo	Pointer to the ADI_ROM_OTP_BOOT_INFO boot information block that is read from OTP and contains boot customization options.
ADI_ROM_BOOT_KEY_TYPE	keyType	When set to a specific value allows keys not stored in OTP to perform secure boot evaluation on an open processor.
ADI_ROM_BOOT_TYPE	bootType	A key to indicate if the boot type is secure or non-secure for open parts.
ROM_SB_IMAGE_TYPE	secureBootImageType	The type of secure boot image.
SBIF_ECDSA_Header_t*	pSecureHeader	Pointer to the secure boot stream header that is loaded by the boot peripheral from the boot source during the configuration phase.
ADI_SBIF_ECDSA_PublicKey_t	publicKey	The public key used for secure boot image authentication.
CRYPTO_DESCRIPTOR	cryptoDescriptors	The descriptor items as required for PKTE operations.
SB_StorageArea_t*	pSB_Storage	Storage area reserved for some crypto operations.
int32_t	secureBytesRemaining	The number of bytes remaining to be processed in the secure boot stream.
uint32_t[4]	aesKey	The 128-bit AES decryption key.
uint32_t[6]	aesWrapKey	The key wrapped key from the BLW secure boot image.
uint32_t[4]	IV	The IV as read from the secure boot header, required to initialize the PKTE.
uint8_t *	pHash	Pointer to the output destination of the SHA-224 result that is required for authentication of the secure boot stream.
uint8_t *	esdsaType	Type of ECDSA algorithm to sign the image.
uint32_t	errorReturn	Storage location for the address of the instruction line following a call to the error handler.

pSource

Source address from where to fetch the next boot data.

The source address must be maintained by the boot peripheral's load function. The kernel does not update the source pointer automatically after requesting data. This lets load routines control and change the source address. It is useful when advanced second stage loaders need to change the source address (due to a fragmented boot stream) or to reset the address (if expanding into a second SPI flash device).

During debug it is useful in identifying the block in the boot stream that is currently being processed.

pDestination

Destination address to store the fetched data.

Used by the boot kernel to indicate the destination address for the fetched data. A boot peripheral's load function must transfer the data to this location before returning back to the kernel. The boot kernel updates this field depending on whether a block header or payload is being fetched. In normal operation mode the kernel loads this field with the storage area location of the block header. After processing the block header loads `pDestination` with the `pTargetAddress` Destination address of payload contents read from the fetched block header. When using page mode the destination points to the internal buffers and is then updated to transfer data to the final destination.

dByteCount

Number of bytes to fetch from the boot source.

The kernel sets this parameter to indicate to the load function the number of bytes requested by the kernel. The kernel is responsible for adjusting the byte count for page mode based accesses. The peripheral load function must return the required number of bytes to the destination address provided.

dFlags

Control flags related to the boot kernel block processing.

When calling a boot mode using the `adi_rom_Boot()` routine the `flags` supplied to that routine are used to initialize `dFlags`. These become global flags that remain set through the entire boot process. When a block header is received the lower 16 bits of the block header are OR'ed with the global flags. The boot software may clear some flags if it detects they are not compatible with other flags and then writes the resulting flags back to this member. The boot kernel then processes the block payload as instructed by the combination of global and boot block specific flags. Upon completion of the block processing original set of global flags are restored and the process repeated.

ulBlockCount

Limit of blocks processed during boot.

When calling the boot process the `adi_rom_Boot()` routine can accept a block limit for the number of blocks to process before terminating the boot process. If the block count is set to zero then the boot process will continue until a final block reached indicating end of the boot stream. This member holds the program's specified limit for the number of blocks to process and is used by the boot kernel after processing each block to compare it against the `ADI_ROM_BOOT_CONFIG::ulBlockCurrent` value. Boot process terminates when `ADI_ROM_BOOT_CONFIG::ulBlockCurrent` equals `ADI_ROM_BOOT_CONFIG::ulBlockCount`

ulBlockCurrent

The number of blocks currently processed by the boot kernel

pNextDxe

Pointer to the next application in the boot stream or the first free location after the boot stream.

This member is initialized when processing a first block in the boot stream. The `dArgument` `Argument` function varies depending on operation field of a first block contains the number of bytes left in the boot stream before we reach the end of that boot stream. This lets this pointer point to the next boot stream or to the first empty location after the boot stream. This allows for a feature when using the `adi_rom_Boot()` routine to find the address of an application in a linked list of boot streams or to find the first empty location after the boot stream.

uByteAddress

The destination address converted to the byte address space.

This member is used to store the byte address space equivalent of SHARC L1 memory addresses, allowing any core to load content to any SHARC cores L1 memory. The SHARC core in which the load is targeted is determined by the block header.

pControlRegister

Pointer to the boot peripherals control register.

This can be used by a boot mode peripherals driver in order to gain efficient access to a control MMR register in the boot peripheral.

NOTE: This is not used in this products boot implementation but may be leveraged by developers of second stage boot loaders if required

dControlValue

Storage for the boot peripheral main control value to enable that peripheral in a required configuration.

This can be used by a boot modes peripheral driver in order to store a control value that can be used to enable the peripheral for a required configuration.

pPeripheralBase

Pointer to the boot peripherals base MMR address

pAuxControlRegister

Pointer to any register that may be used for auxiliary operations such as a timer control register for UART autobaud detection

pAuxPeripheralBase

Pointer to the base address of any peripheral used for auxiliary operations such as the TIMER block

pSecControlRegister

Base MMR address of the SEC SSI instance associated with the boot peripheral should they be required for advanced second stage boot loader development

pDmaBaseRegister

Base MMR address of the DMA channel associated with the boot peripheral.

This is used by the boot kernel to gain access to the DMA channels status when using non blocking DMA operations when page mode or secure boot is required, so it must be set when implementing custom boot loaders in order for the kernel to get access to that peripherals DMA status.

NOTE: If a custom boot peripheral does not support the standard DMA instance then the custom driver will be required to set up a DMA instance in SRAM that this location points to and the load function would need to update the status accordingly to indicate when the DMA was running and when the DMA completed.

loadType

Set by the kernel to specify to the boot peripherals load function if it is requesting a blocking or non-blocking DMA

MdmaCfg

An MDMA descriptor that is used by the boot kernel for internal MDMA operations.

The boot kernel may be required to perform internal MDMA operations outside the control of the boot peripheral driver. Such operations include processing of fill blocks CRC callbacks for CRC verification and MDMA operations from the internal intermediate buffers for indirect block and page mode processing.

NOTE: Users must not use this item or reconfigure this item when developing custom boot drivers, it is intended purely for the internal use by the boot kernel

uwDataWidth

The maximum data width supported by the boot peripherals DMA channel. Set to 0 for 8-bit, 1 for 16-bit and 2 for 32-bit

uwSrcModifyMult

The source modify multiplier used to set [DMA_XMOD](#) for source MDMA operations or peripheral DMA transmit operations

uwDstModifyMult

The destination modify multiplier used to set [DMA_XMOD](#) for destination MDMA operations or peripheral DMA receive operations

uwUserShort

Free to use by the user

dUserLong

Free to use by the user

pModeData

Pointer to the boot mode specific data structure.

Can be set by a boot peripheral driver to allow for a single point of access to the boot mode specific object containing control and configuration information specific to that single boot mode.

dBootCommand

The boot command value supplied during the call to the `adi_rom_Boot()` routine

pHeader

Pointer to the boot header storage location where all boot stream block headers eventually reside for processing by the kernel

pTempBuffer

Pointer to the internal intermediate buffer. Used for processing of indirect blocks

dTempByteCount

Size of the internal intermediate buffer in bytes

pTempSource

Current source address that is being processed in the internal intermediate buffer

dPageByteCount

The page size used for page mode processing. On this product page size is fixed to 1024 bytes and this member is not used for the load requests

bootBuffers

The internal intermediate buffer descriptors required when using indirect and page mode features

bootRegistry

The registry object that is used to register a boot peripherals routines with the kernel.

When using the `adi_rom_Boot()` function the boot software calls a peripherals initialization, configuration, load and cleanup routines from the pointers stored in this object. The kernel itself only makes calls to the load function for the peripheral so when using the `adi_rom_BootKernel()` function the load function that is called by the boot kernel to fetch data from the boot source must be registered using `pLoadFunctionPointer` to the boot modes Load function .

pErrorFunction

Pointer to the error handler called in the event of an error

pCallbackFunction

Pointer to the callback function that is called when processing boot blocks with the callback flag set

pCrcFunction

Pointer to the CRC function that is used to perform CRC validation of the boot stream payload data

pForwardFunction

Feature not supported on this product

bootModes

Access to all boot mode specific resources

pLogBuffer

Pointer to the log buffer. Logging is disabled by default on this product and thus must be configured from within initcodes or hook routines

pLogCurrent

The current position within the log buffer. Logging is disabled by default on this product and thus must be configured from within initcodes or hook routines

dLogByteCount

The size of the log buffer. Logging is disabled by default on this product and thus must be configured from within initcodes or hook routines

pOtpBootInfo

Pointer to the `struct ADI_ROM_OTP_BOOT_INFO` boot information block that gets read from OTP and contains boot customization options

keyType

When set to a specific value allows keys not stored in OTP to be used for secure boot evaluation on an open processor.

By default when performing boot on an open part the decryption keys and the public key are fetched from OTP. By setting this field to `enum ADI_ROM_BOOT_KEY_TYPE` users can disable the fetching of the keys from OTP and instead provision the keys directly in the `ADI_ROM_BOOT_CONFIG::publicKey` and `ADI_ROM_BOOT_CONFIG::aesKey` members via hook routines when using the `adi_rom_Boot()` function.

bootType

A key to indicate if the boot type is secure or non-secure for open parts

secureBootImageType

The type of secure boot image

pSecureHeader

Pointer to the secure boot stream header that is loaded by the boot peripheral from the boot source during the configuration phase

publicKey

The public key used for secure boot image authentication

cryptoDescriptors

The descriptor items as required for the PKTE operations

pSB_Storage

Storage area reserved for some crypto operations

secureBytesRemaining

The number of bytes remaining to be processed in the secure boot stream

aesKey

The 128-bit AES decryption key

aesWrapKey

The key wrapped key from the BLW secure boot image

IV

The IV as read from the secure boot header as required to initialize the PKTE

pHash

Pointer to the output destination of the SHA-224 result that is required for authentication of the secure boot stream

ecdsaType

This holds the ecdsa type to be used for authentication. It can be 224-bit or 256-bit.

errorReturn

Storage location for the address of the instruction line following a call to the error handler

struct ADI_ROM_BOOT_HEADER

Structure Type Declaration: ADI_ROM_BOOT_HEADER

Boot Block Header.

Boot block headers control the loading process of the boot stream, For full details on the contents of the block header and supported flags see Boot Stream.

Table 41-48: ADI_ROM_BOOT_HEADER Members

Type	Name	Description
int32_t	dBlockCode	Instructs the boot kernel how to process the block.
void *	pTargetAddress	Destination Address of Payload
int32_t	dByteCount	Byte Count of the Payload
int32_t	dArgument	Argument functionality varies depending on operation

dBlockCode

Instructs the boot kernel how to process the block.

Contains a number of fields for verification of the block header and flags to indicate the type of block. This allows the kernel to process the block correctly.

pTargetAddress

Destination address of payload

dByteCount

Byte count of the payload

dArgument

Argument function varies depending on operation

struct ADI_ROM_BOOT_INTER_BUFFER

Structure Type Declaration: ADI_ROM_BOOT_INTER_BUFFER

The buffer object for the internal intermediate buffers used for indirect and page mode operations.

Table 41-49: ADI_ROM_BOOT_INTER_BUFFER Members

Type	Name	Description
uint8_t *	pBuffer	Pointer to the buffer
uint32_t	size	Size of the buffer
uint32_t	pageSize	Page size for block based devices.

pBuffer

Pointer to the buffer

size

Size of the buffer

pageSize

Page size for block based devices.

NOTE: This field is not used in this product. A fixed page size of 1024 bytes is used.

struct ADI_ROM_BOOT_INTER_BUFFERS

Structure Type Declaration: ADI_ROM_BOOT_INTER_BUFFERS

The boot kernels internal buffer object used to access the intermediate buffers and obtain buffer status.

Table 41-50: ADI_ROM_BOOT_INTER_BUFFERS Members

Type	Name	Description
ADI_ROM_BOOT_INTER_BUFFER[2]	buffer	The two buffer descriptors
ADI_ROM_BOOT_BUFFER_STATE	state	Buffer Status Information
void *	pSource	Original source address pointer of data loaded to active buffer. Not used on this product
ADI_DMA_TypeDef *	pDma	TBD

buffer

The two buffer descriptors

state

Buffer Status Information

pSource

Original source address pointer of data loaded to active buffer. Not used on this product

struct ADI_ROM_BOOT_LINKPORT

Structure Type Declaration: ADI_ROM_BOOT_LINKPORT

The linkport slave boot mode specific structure.

This structure contains all the boot context information that is specific to the linkport slave boot mode.

Table 41-51: ADI_ROM_BOOT_LINKPORT Members

Type	Name	Description
uint32_t	nFlags	Flags related to linkport boot mode
ADI_LP_TypeDef *	pRegisters	Pointer to the LP peripherals base MMR address, not used on this product

Table 41-51: ADI_ROM_BOOT_LINKPORT Members (Continued)

Type	Name	Description
ADI_DMA_TypeDef *	pRxDmaRegisters	Pointer to the DMA peripherals base MMR address,for receive operations, not used on this product
ADI_DMA_TypeDef *	pTxDmaRegisters	Pointer to the DMA peripherals base MMR address,for transmit operations, not used on this product

nFlags

Flags related to linkport boot mode

pRegisters

Pointer to the LP peripherals base MMR address, not used on this product

pRxDmaRegisters

Pointer to the DMA peripherals base MMR address,for receive operations, not used on this product

pTxDmaRegisters

Pointer to the DMA peripherals base MMR address,for transmit operations, not used on this product

struct ADI_ROM_BOOT_MODES

Structure Type Declaration: ADI_ROM_BOOT_MODES

Holds all boot mode specific configuration items.

A boot mode may have requirements for some dedicated storage. This object is used to collect all storage items for all the boot modes supported by the boot ROM.

Table 41-52: ADI_ROM_BOOT_MODES Members

Type	Name	Description
ADI_ROM_BOOT_SPI	spi	Access to all SPI boot mode resources
ADI_ROM_BOOT_UART	uart	Access to all UART boot mode resources
ADI_ROM_BOOT_LINKPORT	linkport	Access to all LINKPORT boot mode resources
ADI_ROM_BOOT_OSPI	ospi	Access to all OSPI boot mode resources
ADI_ROM_BOOT_CUSTOM	custom	Access to all custom boot mode resources

spi

Access to all SPI boot mode resources

uart

Access to all UART boot mode resources

linkport

Access to all LINKPORT boot mode resources

custom

Access to all custom boot mode resources

Spi3

Access to all OSPI boot mode resources

struct ADI_ROM_BOOT_REGISTRY

Structure Type Declaration: `ADI_ROM_BOOT_REGISTRY`

Boot Mode Registration.

Used to hold pointers for the boot mode initialization, configuration, load and cleanup functions. Can customize the registered content via hook routines or install load functions or cleanup functions from within init codes.

When using `adi_rom_Boot()` the boot process makes a call to the initialization function and the configuration function before calling the kernel. The kernel then runs and makes calls to the load function. The cleanup function is called when the kernel reaches the end of the boot stream.

When using the `adi_rom_BootKernel()` function only the load function is called during execution of the software in the boot ROM. All the functions here must return a `enum ROM_BOOT_RESULT` result in order for the boot process to continue. All functions expect a single argument that is the pointer to the boot structure object `struct ADI_ROM_BOOT_CONFIG`.

Table 41-53: ADI_ROM_BOOT_REGISTRY Members

Type	Name	Description
<code>ROM_BOOT_MODE_INIT_FUNC *</code>	<code>pInitFunction</code>	Pointer to the boot modes Initialization function
<code>ROM_BOOT_MODE_CONFIG_FUNC *</code>	<code>pConfigFunction</code>	Pointer to the boot modes Configuration function
<code>ROM_BOOT_MODE_LOAD_FUNC *</code>	<code>pLoadFunction</code>	Pointer to the boot modes Load function
<code>ROM_BOOT_MODE_CLEANUP_FUNC *</code>	<code>pCleanUpFunction</code>	Pointer to the boot modes Cleanup function
<code>void *</code>	<code>pReserved</code>	Reserved for future use
<code>int32_t</code>	<code>dReserved</code>	Reserved for future use

pInitFunction

Pointer to the boot modes Initialization function

pConfigFunction

Pointer to the boot modes Configuration function

pLoadFunction

Pointer to the boot modes Load function

pCleanUpFunction

Pointer to the boot modes Cleanup function

struct ADI_ROM_BOOT_SPI

Structure Type Declaration: ADI_ROM_BOOT_SPI

The SPI Master Boot Mode Specific Structure.

This structure contains all the boot context information that is specific for SPI master boot mode. During auto-detection information is copied from the required `: :ROM_SPI_LUTENTRY` item into this structure and used to configure the SPI peripheral for the mode of operation.

Table 41-54: ADI_ROM_BOOT_SPI Members

Type	Name	Description
uint8_t	ubReadCommand	Read command to read data from the SPI device
uint8_t	ubDummyBytes	Number of dummy bytes to issue after the read command
uint8_t	ubAddressBytes	Number of address bytes required to access the device
uint8_t	ubDataBits	Bus width when reading the data. 0 = single bit, 1 = dual bit, 2 = quad bit
uint16_t	uwClkLower	SPI clock divider value
uint16_t	uReserved0	Reserved
uint32_t	nTxCtl	The value written to the <code>SPI_TXCTL</code> register used for address transmit operations such as address cycles
uint32_t	nRxCtl	The value written to the <code>SPI_RXCTL</code> register that is used for all receive operations
uint32_t	nCmdCtl	The value written to the <code>SPI_TXCTL</code> register that is used for sending the read command to the SPI Flash
ROM_BOOT_SPIM_IO_ENABLE_FUNC *	pMIOEnFunction	Pointer to the function used to enable quad mode on the SPI flash
uint8_t	nDummy	The Dummy Byte value if dummy byte transfers are required and the bus is not three-stated
uint8_t	nFlags	Flags used for additional SPI configuration processing.
uint16_t	uReserved2	Reserved
void *	pXIPAddress	The memory-mapped SPI address to boot from
ADI_SPI_TypeDef *	pRegisters	Pointer to the SPI peripherals base MMR address, not used on this product

Table 41-54: ADI_ROM_BOOT_SPI Members (Continued)

Type	Name	Description
ADI_DMA_TypeDef *	pRxDmaRegisters	Pointer to the DMA peripherals base MMR address, for receive operations, not used on this product
ADI_DMA_TypeDef *	pTxDmaRegisters	Pointer to the DMA peripherals base MMR address,for transmit operations, not used on this product

ubReadCommand

Read command to use to read data from the SPI device

ubDummyBytes

Number of dummy bytes to issue after the read command

ubAddressBytes

Number of address bytes required to access the device

ubDataBits

The bus width used when reading the data. 0 for single bit, 1 for dual, 2 for quad

uwClkLower

The SPI clock divider value

nTxCtl

The value written to the [SPI_TXCTL](#) register that is used for the address transmit operations such as address cycles

nRxCtl

The value written to the [SPI_RXCTL](#) register that is used for all receive operations

nCmdCtl

The value written to the [SPI_TXCTL](#) register that is used for sending the read command to the SPI Flash

pMIOEnFunction

Pointer to the function used to enable quad mode on the SPI flash

nDummy

The Dummy Byte value if dummy byte transfers are required and the bus is not tri-stated

nFlags

Flags used for some additional SPI configuration processing. The flags supported are defined as follows:

Bit Position	Name	Description
0	ROM_SPI_FLAGS_CMDSKIP_EN	When set the configuration routine enables command skip mode where the SPI does not issue a read command for read operations
1	ROM_SPI_FLAGS_MULTICMD_EN	Configuration routine enables sending command cycles over dual or quad bit bus

pXIPAddress

The memory-mapped SPI address to boot from

pRegisters

Pointer to the SPI peripherals base MMR address, not used on this product

pRxDmaRegisters

Pointer to the DMA peripherals base MMR address, for receive operations, not used on this product

pTxDmaRegisters

Pointer to the DMA peripherals base MMR address, for transmit operations, not used on this product

struct ADI_ROM_BOOT_OSPI

Structure Type Declaration: ADI_ROM_BOOT_OSPI

The SPI Master Boot Mode Specific Structure.

This structure contains all the boot context information that is specific for OSPI master boot mode. During auto-detection information is copied from the required : :ROM_OSPI_LUTENTRY item into this structure and used to configure the OSPI peripheral for the mode of operation.

Table 41-55: ADI_ROM_BOOT_OSPI Members

Type	Name	Description
uint8_t	ubReadCommand	Read command to read data from the OSPI device
uint8_t	ubDummyCycles	Number of dummy cycles to issue after the read command
uint8_t	ubAddressBytes	Number of address bytes required to access the device
uint8_t	ubDataBits	Bus width when reading the data
uint16_t	uwClkLower	OSPI clock divider value
uint16_t	uReserved0	Reserved
uint32_t	nCfg	Value written to lower 16 bits of the OSPI_CFG register
uint32_t	nDsr	Value written to OSPI device size register
uint32_t	nDrir	Value written to OSPI device read instruction control register

Table 41-55: ADI_ROM_BOOT_OSPI Members (Continued)

Type	Name	Description
uint32_t	reserved1	Reserved
uint8_t	nDummy	The dummy byte value to be used
uint8_t	nFlags	Flags used for some additional SPI configuration processing
uint16_t	uReserved2	Reserved
void *	pXIPAddress	The memory-mapped OSPI address to boot
ADI_OSPI_TypeDef *	pRegisters	Pointer to the OSPI peripherals base MMR address, not used on this product
ADI_DMA_TypeDef *	pRxDmaRegisters	Pointer to the DMA peripherals base MMR address, for receive operations, not used on this product
ADI_DMA_TypeDef *	pTxDmaRegisters	Pointer to the DMA peripherals base MMR address, for transmit operations, not used on this product

ubReadCommand

Read command to use to read data from the SPI device

ubDummyCycles

Number of dummy cycles to issue after the read command

ubAddressBytes

Number of address bytes required to access the device

ubDataBits

The bus width used when reading the data. 0 for single bit, 1 for dual, 2 for quad

uwClkLower

The SPI clock divider value

nCfg

The value written to the Lower 16 bits of the OSPI Control register

nDsr

The value written to the OSPI Device Size register

nDrir

The value written to the OSPI Device Read Instruction Control register

nDummy

The Dummy Byte value if dummy byte transfers are required and the bus is not tri-stated

nFlags

Flags used for some additional SPI configuration processing. The flags supported are defined as follows:

- ROM_SPI_FLAGS_CMDSKIP_EN (Bit position 0): When set the configuration routine enables command skip mode where the SPI does not issue a read command for read operations
- ROM_SPI_FLAGS_MULTICMD_EN (Bit position 1): Configuration routine enables sending command cycles over dual or quad bit bus

pXIPAddress

The memory-mapped SPI address to boot

pRegisters

Pointer to the OSPI peripherals base MMR address (not used on this product)

pRxDmaRegisters

Pointer to the DMA peripherals base MMR address for receive operations (not used on this product)

pTxDmaRegisters

Pointer to the DMA peripherals base MMR address, for transmit operations (not used on this product)

struct ADI_ROM_BOOT_UART

Structure Type Declaration: ADI_ROM_BOOT_UART

The UART slave boot mode specific structure.

This structure contains all the boot context information that is specific for the UART slave boot mode.

Table 41-56: ADI_ROM_BOOT_UART Members

Type	Name	Description
uint32_t	nFlags	Flags related to UART Boot mode
ADI_UART_TypeDef*	pRegisters	Pointer to the UART peripherals base MMR address, not used on this product
ADI_DMA_TypeDef*	pRxDmaRegisters	Pointer to the DMA peripherals base MMR address for receive operations, not used on this product
ADI_DMA_TypeDef*	pTxDmaRegisters	Pointer to the DMA peripherals base MMR address for transmit operations, not used on this product

nFlags

Flags related to UART Boot mode

pRegisters

Pointer to the UART peripherals base MMR address, not used on this product

pRxDmaRegisters

Pointer to the DMA peripherals base MMR address for receive operations, not used on this product

pTxDmaRegisters

Pointer to the DMA peripherals base MMR address for transmit operations, not used on this product

struct ADI_ROM_OTP_BOOT_CFG

Structure Type Declaration: ADI_ROM_OTP_BOOT_CFG

The boot configuration object for storing further boot customization objects.

This is a 160-bit structure that is allocated to one contiguous region in the OTP memory array. The functionality allows for individual flags to enable or disable specific features of the boot process. Each flag is allocated in a separate 16-bit word so that each flag can be set at different times and the ECC information will not impact the setting of another flag.

Table 41-57: ADI_ROM_OTP_BOOT_CFG Members

Type	Name	Description
uint32_t	ctl_WEN:1 (bitfield)	Enable a write to the CGU_CTL register
uint32_t	div_WEN:1 (bitfield) (bitfield)	Enable a write to the CGU_DIV register
uint32_t	reserved0:1 (bitfield)	Reserved
uint32_t	div_DSEL:5 (bitfield)	CGU_DIV.DSEL value
uint32_t	div_CSEL:5 (bitfield) (bitfield)	CGU_DIV.CSEL value
uint32_t	div_S0SEL:3 (bitfield)	CGU_DIV.S0SEL value
uint32_t	div_SYSSSEL:5 (bitfield)	CGU_DIV.SYSSSEL value
uint32_t	div_S1SEL:3 (bitfield)	CGU_DIV.S1SEL value
uint32_t	div_OSEL:7 (bitfield)	CGU_DIV.OSEL value
uint32_t	div_DF:1 (bitfield)	CGU_CTL.DF value
uint32_t	div_MSEL:1 (bitfield) (bitfield)	CGU_CTL.MSEL value
uint32_t	auto_disable:1 (bitfield)	Disable polling on auto alignment of clocks (not recommended)

Table 41-57: ADI_ROM_OTP_BOOT_CFG Members (Continued)

Type	Name	Description
uint32_t	reserved1:16 (bitfield)	Reserved
uint32_t	clkoutsel_CLKOUTSEL:5 (bitfield)	CGU_CLKOUTSEL.CLKOUTSEL value
uint32_t	reserved5:15 (bitfield)	Reserved
uint32_t	clkoutsel_WEN:1 (bitfield) (bitfield)	Enable write to the CGU_CLKOUTSEL register
uint32_t	reserved2:12 (bitfield)	Reserved
uint32_t	oswctl0_WEN:1 (bitfield)	Enables write to the instance 0 register
uint32_t	oswctl0_HODF:6 (bitfield)	Value
uint32_t	oswctl0_HODEN:1 (bitfield)	Value
uint32_t	oswctl0_CNGEN:1 (bitfield)	Value
uint32_t	oswctl0_BOUF:1 (bitfield)	Value
uint32_t	oswctl0_BOUEN:5 (bitfield)	Value
uint32_t	oswctl0_FAULTEN:1 (bitfield)	Value
uint32_t	oswctl0_MONDIS:1 (bitfield)	Value
uint32_t	oswctl0_FAULTPINDIS:1 (bitfield)	Value
uint32_t	reserved3:15 (bitfield)	Reserved
uint32_t	oswctl1_WEN:1 (bitfield)	Enables write to the instance 1 register
uint32_t	oswctl1_HODF:6 (bitfield)	Value
uint32_t	oswctl1_HODEN:1 (bitfield)	Value
uint32_t	oswctl1_CNGEN:1 (bitfield)	Value

Table 41-57: ADI_ROM_OTP_BOOT_CFG Members (Continued)

Type	Name	Description
uint32_t	oswctl1_BOUF:1 (bitfield)	Value
uint32_t	oswctl1_BOUEN:5 (bitfield)	Value
uint32_t	oswctl1_FAULTEN:1 (bitfield)	Value
uint32_t	oswctl1_MONDIS:1 (bitfield)	Value
uint32_t	oswctl1_FAULTPINDIS:1 (bitfield)	Value
uint32_t	reserved4:14 (bitfield)	Reserved

lockMonitor

Flag to enable the status checking of ADI private OTP area.

pubkey0Inv

Invalidate Public Key 0, use next public key for secure boot

pubkey1Inv

Invalidate Public Key 1, Secure boot will no longer be operational as no further public keys

privkey0Inv

Invalidate Decryption Key 0, use next Decryption key for secure boot

privkey1Inv

Invalidate Decryption Key 1, use next Decryption key for secure boot

privkey2Inv

Invalidate Decryption Key 2, use next Decryption key for secure boot

privkey3Inv

Invalidate Decryption Key 3, once invalidated part will no longer be bootable

dmcEn

Enables configuration of the DMC from values in OTP stored in the format of the ADI_ROM_OTP_DMC_CONFIG object

dmcInv

Invalidates the DMC values in the OTP resulting in bypassing of DMC configuration

struct ADI_ROM_OTP_BOOT_CGU_INFO

Structure Type Declaration: ADI_ROM_OTP_BOOT_CGU_INFO

The CGU configuration object located in OTP for configuration of the CGU by the boot software.

This is a 128-bit structure that is allocated to one contiguous region in the OTP memory array. Allows the boot software to configure the CGU for a more efficient boot process.

Table 41-58: ADI_ROM_OTP_BOOT_CGU_INFO Members

Type	Name	Description
uint32_t	ctl_WEN:1 (bitfield)	Enable write to the CGU_CTL register
uint32_t	div_WEN:1 (bitfield)	Enable write to the CGU_DIV register
uint32_t	reserved0:1 (bitfield)	Reserved
uint32_t	div_DSEL:5 (bitfield)	CGU_DIV.DSEL value
uint32_t	div_CSEL:5 (bitfield)	CGU_DIV.CSEL value
uint32_t	div_S0SEL:3 (bitfield)	CGU_DIV.S0SEL value
uint32_t	div_SYSSEL:5 (bitfield)	CGU_DIV.SYSSEL value
uint32_t	div_S1SEL:3 (bitfield)	CGU_DIV.S1SEL value
uint32_t	div_OSEL:7 (bitfield)	CGU_DIV.OSEL value
uint32_t	ctl_DF:1 (bitfield)	CGU_CTL.DF value
uint32_t	ctl_MSEL:7 (bitfield)	CGU_CTL.MSEL value
uint32_t	auto_disable:1 (bitfield)	disable polling on auto-alignment of clocks (not recommended)
uint32_t	Reserved1:6 (bitfield)	Reserved
uint32_t	clkoutsel_CLKOUTSEL:5 (bitfield)	CGU_CLKOUTSEL.CLKOUTSEL value
uint32_t	clkoutsel_WEN:1 (bitfield)	Enable write to the CGU_CLKOUTSEL register
uint32_t	reserved2:12 (bitfield)	Reserved
uint32_t	oscwctl0_WEN:1 (bitfield)	Enable write to the CGU_OSCWDCTL instance 0 register
uint32_t	oscwctl0_HODF:6 (bitfield)	CGU_OSCWDCTL.HODF value
uint32_t	oscwctl0_HODEN:1 (bitfield)	CGU_OSCWDCTL.HODEN value
uint32_t	oscwctl0_CNGEN:1 (bitfield)	CGU_OSCWDCTL.CNGEN value
uint32_t	oscwctl0_BOUF:5 (bitfield)	CGU_OSCWDCTL.BOUF value
uint32_t	oscwctl0_BOUEN:1 (bitfield)	CGU_OSCWDCTL.BOUEN value
uint32_t	oscwctl0_FAULTEN:1 (bitfield)	CGU_OSCWDCTL.FAULTEN value
uint32_t	oscwctl0_MONDIS:1 (bitfield)	CGU_OSCWDCTL.MONDIS value
uint32_t	oscwctl0_FAULTPINDIS:1 (bitfield)	CGU_OSCWDCTL.FAULTPINDIS value

Table 41-58: ADI_ROM_OTP_BOOT_CGU_INFO Members (Continued)

Type	Name	Description
uint32_t	reserved3:14 (bitfield)	Reserved
uint32_t	oscwctl1_WEN:1 (bitfield)	Enable write to the CGU_OSCWDCTL instance 1 register
uint32_t	oscwctl1_HODF:6 (bitfield)	CGU_OSCWDCTL.HODF value
uint32_t	oscwctl1_HODEN:1 (bitfield)	CGU_OSCWDCTL.HODEN value
uint32_t	oscwctl1_CNGEN:1 (bitfield)	CGU_OSCWDCTL.CNGEN value
uint32_t	oscwctl1_BOUF:5 (bitfield)	CGU_OSCWDCTL.BOUF value
uint32_t	oscwctl1_BOUEN:1 (bitfield)	CGU_OSCWDCTL.BOUEN value
uint32_t	oscwctl1_FAULTEN:1 (bitfield)	CGU_OSCWDCTL.FAULTEN value
uint32_t	oscwctl1_MONDIS:1 (bitfield)	CGU_OSCWDCTL.MONDIS value
uint32_t	oscwctl1_FAULTPINDIS:1 (bitfield)	CGU_OSCWDCTL.FAULTPINDIS value
uint32_t	reserved4:14 (bitfield)	Reserved

ctl_WEN

Enable write to the [CGU_CTL](#) register

div_WEN

Enable write to the [CGU_DIV](#) register

div_DSEL

CGU_DIV.DSEL value

div_CSEL

CGU_DIV.CSEL value

div_S0SEL

CGU_DIV.S0SEL value

div_SYSSEL

CGU_DIV.SYSSEL value

div_S1SEL

CGU_DIV.S1SEL value

div_OSEL

CGU_DIV.OSEL value

ctl_DF

`CGU_CTL.DF` value

ctl_MSEL

`CGU_CTL.MSEL` value

auto_disable

disable polling on auto-alignment of clocks, NOT RECOMMENDED!

clkoutsel_CLKOUTSEL

`CGU_CLKOUTSEL.CLKOUTSEL` value

clkoutsel_WEN

Enable write to the `CGU_CLKOUTSEL` register

oswctl0_WEN

Enable write to the `CGU_OSCWDCTL` instance 0 register

oswctl0_HODF

`CGU_OSCWDCTL.HODF` value

oswctl0_HODEN

`CGU_OSCWDCTL.HODEN` value

oswctl0_CNGEN

`CGU_OSCWDCTL.CNGEN` value

oswctl0_BOUF

`CGU_OSCWDCTL.BOUF` value

oswctl0_BOUEN

`CGU_OSCWDCTL.BOUEN` value

oswctl0_FAULTEN

`CGU_OSCWDCTL.FAULTEN` value

oswctl0_MONDIS

`CGU_OSCWDCTL.MONDIS` value

oswctl0_FAULTPINDIS

`CGU_OSCWDCTL.FAULTPINDIS` value

oswctl1_WEN

Enable write to the `CGU_OSCWDCTL` instance 1 register

oswctl1_HODF

CGU_OSCWDCTL.HODF value

oswctl1_HODEN

CGU_OSCWDCTL.HODEN value

oswctl1_CNGEN

CGU_OSCWDCTL.CNGEN value

oswctl1_BOUF

CGU_OSCWDCTL.BOUF value

oswctl1_BOUEN

CGU_OSCWDCTL.BOUEN value

oswctl1_FAULTEN

CGU_OSCWDCTL.FAULTEN value

oswctl1_MONDIS

CGU_OSCWDCTL.MONDIS value

oswctl1_FAULTPINDIS

CGU_OSCWDCTL.FAULTPINDIS value

struct ADI_ROM_OTP_BOOT_CMD_INFO

Structure Type Declaration: ADI_ROM_OTP_BOOT_CMD_INFO

The boot command object for storing a customized boot command for each boot mode within the OTP memory array.

This is a 160-bit structure that is allocated to one contiguous region in the OTP memory array. Allows the boot ROM software to pass a custom boot command to a specific boot mode which changes the default boot behavior on startup. Can be used to change the default UART instance used for a UART boot operation (for example).

Table 41-59: ADI_ROM_OTP_BOOT_CMD_INFO Members

Type	Name	Description
uint32_t	spiMasterBootCmd	SPI Master Boot Mode
uint32_t	spiSlaveBootCmd	SPI Slave Boot Mode
uint32_t	lpBootCmd	Link Port Slave Boot Mode
uint32_t	uartBootCmd	UART Slave Boot Mode

Table 41-59: ADI_ROM_OTP_BOOT_CMD_INFO Members (Continued)

Type	Name	Description
uint32_t	ospiMasterBootCmd	OSPI Master Boot Mode

spiMasterBootCmd[SPI Master Boot Mode](#)**spiSlaveBootCmd**[SPI Slave Boot Mode](#)**lpBootCmd**[Link Port Slave Boot Mode](#)**uartBootCmd**[UART Slave Boot Mode](#)**ospiMasterBootCmd**[OSPI Master Boot Mode](#)**struct ADI_ROM_OTP_BOOT_INFO**

Structure Type Declaration: ADI_ROM_OTP_BOOT_INFO

The 576-bit boot info object located in OTP for boot customization.

This is a 576-bit structure that is allocated to one contiguous region in the OTP memory array. The content in OTP is stored in the format of this structure so boot can read the contents directly into this object.

Programs can read this object using the [adi_rom_otp_get\(\)](#) routine supplying the [enum OTPCMD](#) enumeration

Table 41-60: ADI_ROM_OTP_BOOT_INFO Members

Type	Name	Description
struct ADI_ROM_OTP_BOOT_CGU_INFO	cgu	CGU Configuration information
uint32_t	flashStartAddress	Flash Start address for the SPI/OSPI master boot mode
struct ADI_ROM_OTP_BOOT_CMD_INFO	bcmd	Boot Command customization for each boot mode to change default boot peripheral instance and configuration
struct ADI_ROM_OTP_BOOT_CMD_INFO	rdcr	Additional OSPI configuration used for OSPI boot mode

Table 41-60: ADI_ROM_OTP_BOOT_INFO Members (Continued)

Type	Name	Description
uint16_t	ospi_rdc_r_reserved0	Reserved
struct ADI_ROM_OTP_BOOT_CFG	bcfg	Additional boot configuration flags for key invalidation and to enable DMC configuration
uint32_t	val14	Reserved
uint16_t	otpReadTiming	Reserved, Must always be zero
uint16_t	reserved1	Reserved

cgu

CGU Configuration information

flashStartAddress

Flash start address for the SPI/OSPI master boot mode

bcmd

Boot Command customization for each boot mode to change the default boot peripheral instance and configuration

rdcr

Additional OSPI configuration info that is used for OSPI boot mode

bcfg

Additional boot configuration flags for key invalidation and for enabling DMC configuration

otpReadTiming

Reserved, Must always be zero

struct ADI_ROM_OTP_DMC_CONFIG

Structure Type Declaration: ADI_ROM_OTP_DMC_CONFIG

The 384-bit configuration object located in OTP for configuration of the DMC during preboot.

Makes use of the memories connected to the DMC peripheral during boot without the use of init codes in the boot stream. The settings can be applied to this object in the OTP memory. During preboot the boot software reads the object from OTP and configures the peripheral accordingly.

NOTE: When the device is open programs should avoid using OTP and instead use initcodes to initialize DMC because the initcode method is highly customizable. To lock the device to enable secure boot (to boot to

memories interfaced to the DMC) the configuration must be provisioned to OTP because initcodes are not supported in secure boot.

Table 41-61: ADI_ROM_OTP_DMC_CONFIG Members

Type	Name	Description
uint32_t	ulDDR_DLLCTLCFG	Content of DDR DLLCTL and DMC_CFG register
uint32_t	ulDDR_MR2MR3	Content of the DDR MR2 and MR3 register
uint32_t	ulDDR_CTL	Content of the DDR Control register
uint32_t	ulDDR_MRMR1	Content of the DDR MR and MR1 register
uint32_t	ulDDR_TR0	Content of the DDR Timing register
uint32_t	ulDDR_TR1	Content of the DDR Timing register
uint32_t	ulDDR_TR2	Content of the DDR Timing register
uint32_t	ulDDR_ZQCTL0	Content of ZQCTL0 register
uint32_t	ulDDR_ZQCTL1	Content of ZQCTL1 register
uint32_t	ulDDR_ZQCTL2	Content of ZQCTL2 register
uint32_t	ulDDRPHY_CACTL	Content of DDRPHY_DDR_CA_CTL register
uint32_t	uBypassDelay_LANE0CTL1:6	Content of DDR_LANE0_CTL1[15:10] register
uint32_t	uBypassDelay_LANE1CTL1:6	Content of DDR_LANE1_CTL1[15:10] register
uint32_t	uBypassDelay_LANE0CTL0:6	Content of DDR_LANE0_CTL0[15:10] register
uint32_t	uBypassDelay_LANE1CTL0:6	Content of DDR_LANE1_CTL0[15:10] register
uint32_t	reserved0:8	Reserved

struct ROM_BOOT_DMA_INSTANCE

Structure Type Declaration: ROM_BOOT_DMA_INSTANCE

DMA Channel Instance.

Specifies the base MMR address of the DMA channel as well as trigger and interrupt IDs

Table 41-62: ROM_BOOT_DMA_INSTANCE Members

Type	Name	Description
ADI_DMA_TypeDef *	pReg	Pointer to the base address of the DMA channel MMR registers
DMA_CHANn_TypeDef	eDmaChannelId	The actual DMA channel ID in the system
uint8_t	TriggerId	The trigger ID associated with the DMA channel
uint8_t	InterruptId	The interrupt ID associated with the DMA channel

pReg

Pointer to the base address of the DMA channel MMR registers

eDmaChannelId

The actual DMA channel ID in the system

TriggerId

The trigger ID associated with the DMA channel

InterruptId

The interrupt ID associated with the DMA channel

struct ROM_BOOT_MDMA

Structure Type Declaration: ROM_BOOT_MDMA

MDMA Channels.

Provides access to all the MDMA channels and CRC peripherals supported by the processor

Table 41-63: ROM_BOOT_MDMA Members

Type	Name	Description
ROM_BOOT_MDMA_REGS[PARAM_SYS0_NUM_MDMA_STREAMS]	Stream	Array of MDMA channel configurations supported by the processor

Stream

Array of MDMA channel configurations supported by the processor

struct ROM_BOOT_MDMA_REGS

Structure Type Declaration: ROM_BOOT_MDMA_REGS

MDMA Channel Registers.

Contains the Source and Destination MDMA channel instances for access to the MMRs and interrupt and trigger information. Information is also provided on the CRC support of the MDMA channel and access is provided to the corresponding CRC peripheral.

Table 41-64: ROM_BOOT_MDMA_REGS Members

Type	Name	Description
struct ROM_BOOT_DMA_INSTANCE	Src	The source DMA Channel in the MDMA pair
ROM_BOOT_DMA_INSTANCE	Dst	The destination DMA Channel in the MDMA pair
ADI_CRC_TypeDef *	pCrc	The base MMR address of the associated CRC peripheral if one exists

Table 41-64: ROM_BOOT_MDMA_REGS Members (Continued)

Type	Name	Description
enum ROM_BOOT_MDMA_CRC_SUPPORT	eCrcSupport	Indicates if the MDMA channel supports CRC or not

Src

The source DMA Channel in the MDMA pair

Dst

The destination DMA Channel in the MDMA pair

pCrc

The base MMR address of the associated CRC peripheral if one exists

eCrcSupport

Indicates if the MDMA channel supports CRC or not

struct ROM_DMA_MDMA_CONFIG

Structure Type Declaration: `ROM_DMA_MDMA_CONFIG`

MDMA Configuration Object.

The configurable structure for controlling the MDMA operation supplied to the `adi_rom_MemDma()` routine.

Table 41-65: ROM_DMA_MDMA_CONFIG Members

Type	Name	Description
enum ROM_DMA_MDMA_OPERATION	eOperation	Type of operation to perform
enum ROM_DMA_MDMA_ID	eId	MDMA Channel ID
void *	pSource	Source Pointer
void *	pDestination	Destination Pointer
uint32_t	ByteCount	Byte Count
enum ROM_DMA_DONE_DETECT_METHOD	eDoneDetect	DMA Done Detection Method
uint32_t	CrcCtl	CRC_CTL value when CRC operations are required
uint32_t	FillVal	Fill value for memory fill operations
uint32_t	CrcPoly	CRC Polynomial for CRC operations

Table 41-65: ROM_DMA_MDMA_CONFIG Members (Continued)

Type	Name	Description
uint32_t	CrcCompare	Value used for CRC compare operations or for a CRC32 result compare

eOperation

Type of operation to perform

eId

MDMA Channel ID

pSource

Source Pointer

pDestination

Destination Pointer

ByteCount

Byte Count

eDoneDetect

DMA Done Detection Method

CrcCtl

[CRC_CTL](#) value when CRC operations are required

FillVal

Fill value for memory fill operations

CrcPoly

CRC Polynomial for CRC operations

CrcCompare

Value used for CRC compare operations or for a CRC32 result compare

struct ROM_DMA_PDMA_CONFIG

Structure Type Declaration: ROM_DMA_PDMA_CONFIG

PDMA Configuration Object.

The user configurable structure for controlling the PDMA operation via the [adi_rom_PeriphDma\(\)](#) function.

Table 41-66: ROM_DMA_PDMA_CONFIG Members

Type	Name	Description
enum ROM_DMA_PDMA_OPERATION	eOperation	Type of operation to perform
ADI_DMA_TypeDef volatile *	pRegs	Pointer to the base address of the DMA channel MMR registers
uint16_t	dataWidth	The maximum supported data width of the DMA channel. Used to configure the DMA_CFG.PSIZE PSIZE field in DMA_CFG
uint16_t	dstModifyMult	The modify multiplier, usually set to 1
void *	pSource	Source Pointer used for transmit operations
void *	pDestination	Destination Pointer used for receive operations
uint32_t	byteCount	Number of bytes to transfer
enum ROM_DMA_DONE_DETECT_METHOD	eDoneDetect	DMA Done Detection method used for the transfer
enum ROM_DMA_DONE_DETECT_METHOD	loadType	Defines whether the load function DMA waits for the load to complete or returns to the kernel

eOperation

Type of operation to perform

pRegs

Pointer to the base address of the DMA channel MMR registers

dataWidth

The maximum supported data width of the DMA channel. Used to configure the DMA_CFG.PSIZE PSIZE field in DMA_CFG

dstModifyMult

The modify multiplier, usually set to 1

pSource

Source Pointer used for transmit operations

pDestination

Destination Pointer used for receive operations

byteCount

Number of bytes to transfer

eDoneDetect

DMA Done Detection method used for the transfer

loadType

Defines whether the load function DMA waits for the load to complete or returns to the kernel

struct otp_data

Structure Type Declaration: `otp_data`

Container for accessing data to be written to OTP via the `adi_rom_otp_pgm()` routine.

Any pointers that are NULL will result in the object not being written. Any data values of 0 will not be written.

Table 41-67: otp_data Members

Type	Name	Description
void (*)	reserved	
uint32_t (*) [ROM_OTP_SZ_huk]	huk	Pointer to 256-bit Hardware Unique Key
uint32_t (*) [ROM_OTP_SZ_rkek]	rkek	Pointer to 128-bit Root Key Encryption Key
uint 32_t (*) [ROM_OTP_SZ_dek]	dek	Pointer to 128-bit Local Encryption Key
uint32_t (*) [ROM_OTP_SZ_oem_public_key]	oem_public_key	Pointer to 512-bit OEM Public Key
uint32_t (*) [ROM_OTP_SZ_pvt_128key0]	pvt_128key0	Pointer to 128-bit AES Key
uint32_t (*) [ROM_OTP_SZ_pvt_128key1]	pvt_128key1	Pointer to 128-bit AES Key
uint32_t (*) [ROM_OTP_SZ_pvt_128key2]	pvt_128key2	Pointer to 128-bit AES Key
uint32_t (*) [ROM_OTP_SZ_pvt_128key3]	pvt_128key3	Pointer to 128-bit AES Key
uint32_t (*) [ROM_OTP_SZ_ek]	ek	Pointer to 256-bit endorsement key
uint32_t (*) [ROM_OTP_SZ_secure_emu_key0]	secure_emu_key0	Pointer to 128-bit Secure Debug Key

Table 41-67: otp_data Members (Continued)

Type	Name	Description
uint32_t(*) [ROM_OTP_SZ_secure_emu_key1]	secure_emu_key1	Pointer to 128-bit Secure Debug Key
uint32_t	emu_key0_disable :16 (bitfield)	Secure Debug key disable
uint32_t	emu_key1_disable :16 (bitfield)	Secure Debug key disable
uint32_t(*) [ROM_OTP_SZ_public_key0]	public_key0	Pointer to 512-bit public key used for boot stream authentication
uint32_t(*) [ROM_OTP_SZ_public_key1]	public_key1	Pointer to 512-bit public key used for boot stream authentication
uint32_t(*) [ROM_OTP_SZ_boot_info]	boot_info	Pointer to 608-bit boot customization structure, see also struct ADI_ROM_OTP_BOOT_INFO
uint8_t	antiroll_nv_cntr	Anti-rollback counter to prevent loading of older firmware during secure boot.
uint32_t(*) [ROM_OTP_SZ_gp1]	gp1	Pointer to 512-bit General purpose user space
uint32_t	bootModeDisable: 8 (bitfield)	Boot mode disable for permanently disabling specific boot modes
uint32_t(*) [ROM_OTP_SZ_preboot_ddr_cfg]	preboot_ddr_cfg	Pointer to 384-bit DMC configuration. See also struct ADI_ROM_OTP_BOOT_INFO
uint32_t(*) [ROM_OTP_SZ_stageID]	stageID	Pointer to 64-bit staging ID

huk

Pointer to 256-bit Hardware Unique Key

rkek

Pointer to 128-bit Root key Encryption Key

dek

Pointer to 128-bit Local Encryption Key

oem_public_key

Pointer to 512-bit OEM Public Key

pvt_128key0

Pointer to 128-bit AES Key

pvt_128key1

Pointer to 128-bit AES Key

pvt_128key2

Pointer to 128-bit AES Key

pvt_128key3

Pointer to 128-bit AES Key

ek

Pointer to 256-bit endorsement key

secure_emu_key0

Pointer to 128-bit Secure Debug Key

secure_emu_key1

Pointer to 128-bit Secure Debug Key

emu_key0_disable

Secure debug key disable

emu_key1_disable

Secure debug key disable

public_key0

Pointer to 512-bit public key used for boot stream authentication

public_key1

Pointer to 512-bit public key used for boot stream authentication

boot_info

Pointer to 608-bit boot customization structure, see also [struct ADI_ROM_OTP_BOOT_INFO](#)

antiroll_nv_cntr

Anti-rollback counter to prevent loading of older firmware during secure boot.

The counter supports values of 0 through 31. The counter feature is disabled as long as the counter is set initially to 0.

gp1

Pointer to 512-bit General purpose user space

bootModeDisable

Boot mode disable for permanently disabling specific boot modes

preboot_dds_cfg

Pointer to 384-bit DMC Configuration. See also [struct ADI_ROM_OTP_DMC_CONFIG](#)

stageID

Pointer to 64-bit staging ID

Enumerations

The programming model for booting the processor uses the enumerations defined in this section.

enum ADI_ROM_BOOT_KEY_TYPE

Enumeration Type Declaration: `ADI_ROM_BOOT_KEY_TYPE`

Indicates if custom security keys are used for evaluation of secure boot.

By default the boot process fetches all security keys from OTP for use during secure boot. Custom security lets the program set their custom keys in the [struct ADI_ROM_BOOT_CONFIG](#) item (not from OTP). Secure boot is evaluated using the [adi_rom_Boot\(\)](#) function without provisioning keys in OTP.

Table 41-68: ADI_ROM_BOOT_KEY_TYPE Members

Enumerator	Description
ADI_ROM_CUSTOM_SECURITY	Enable use of custom security keys for authentication and decryption

ADI_ROM_CUSTOM_SECURITY

Enable use of custom security keys for authentication and decryption

enum ADI_ROM_BOOT_TYPE

Enumeration Type Declaration: `ADI_ROM_BOOT_TYPE`

Indicate to the boot kernel (in an open processor) if secure or non secure boot is required.

The boot kernel defaults to a secure boot unless the boot structure has been configured to indicate Non-Secure Boot

Table 41-69: ADI_ROM_BOOT_TYPE Members

Enumerator	Description
ADI_ROM_SECURE_BOOT_DIS	Non-Secure Boot
ADI_ROM_SECURE_BOOT	Secure Boot

Table 41-69: ADI_ROM_BOOT_TYPE Members (Continued)

Enumerator	Description
ADI_ROM_SECURE_BOOT_HASH_ONLY	Boot kernel supports for hash compare only if this boot type selected using the ROM API. ECDSA verification is skipped.

ADI_ROM_SECURE_BOOT_DIS

Non-Secure Boot

ADI_ROM_SECURE_BOOT

Secure Boot

ADI_ROM_SECURE_BOOT_HASH_ONLY

Boot kernel supports for hash compare only if this boot type selected using ROM API. ECDSA verification is skipped.

enum OTPCMD

Enumeration Type Declaration: OTPCMD

Commands required by the `adi_rom_otp_get()` routine to retrieve specific fields from the OTP memory.

Table 41-70: OTPCMD Members

Enumerator	Description
otpcmd_huk	Hardware Unique Key
otpcmd_rkek	Root key Encryption Key
otpcmd_dek	Local Encryption Key
otpcmd_oem_public_key	OEM Public Key
otpcmd_pvt_128key0	Customer Private AES Key0
otpcmd_pvt_128key1	Customer Private AES Key1
otpcmd_pvt_128key2	Customer Private AES Key2
otpcmd_pvt_128key3	Customer Private AES Key3
otpcmd_ek	Endorsement Key
otpcmd_secure_emu_key0	Secure Emulation Key 0
otpcmd_secure_emu_key1	Secure Emulation Key 1
otpcmd_emu_key0_disable	Secure emulation key 0 disable
otpcmd_emu_key1_disable	Secure emulation key 1 disable
otpcmd_public_key0	Customer Public Key0
otpcmd_public_key1	Customer Public Key1

Table 41-70: OTPCMD Members (Continued)

Enumerator	Description
otpcmd_boot_info	Customer Programmable Boot Information
otpcmd_otpTiming	OTP Read timing override
otpcmd_antiroll_nv_cntr	AntiRollback NV Counter
otpcmd_gp1	General Purpose 1
otpcmd_bootModeDisable	Boot Mode Disable Bits
otpcmd_preboot_ddr_cfg	User DMC configuration
otpcmd_stageID	StageID

otpcmd_huk

Hardware Unique Key

otpcmd_rkek

Root Encryption Key

otpcmd_dek

Local Encryption Key

otpcmd_oem_public_key

OEM Public Key

otpcmd_pvt_128key0

Customer Private AES Key0

otpcmd_pvt_128key1

Customer Private AES Key1

otpcmd_pvt_128key2

Customer Private AES Key2

otpcmd_pvt_128key3

Customer Private AES Key3

otpcmd_ek

Endorsement Key

otpcmd_secure_emu_key0

Secure Emulation Key 0

otpcmd_secure_emu_key1

Secure Emulation Key 1

otpcmd_emu_key0_disable

Secure emulation key 0 disable

otpcmd_emu_key2_disable

Secure emulation key 2 disable

otpcmd_public_key0

Customer Public Key0

otpcmd_public_key1

Customer Public Key1

otpcmd_boot_info

Customer Programmable Boot Information

otpcmd_otpTiming

OTP Read timing override

otpcmd_antiroll_nv_cntr

AntiRollback NV Counter

otpcmd_gp1

General Purpose 1

otpcmd_bootModeDisable

Boot Mode Disable Bits

otpcmd_preboot_ddr_cfg

User DMC configuration

otpcmd_stageID

StageID

enum ROM_BOOT_MDMA_CRC_SUPPORT

Enumeration Type Declaration: ROM_BOOT_MDMA_CRC_SUPPORT

MDMA Channel CRC support.

Specifies whether the MDMA channel supports CRC operations or not

Table 41-71: ROM_BOOT_MDMA_CRC_SUPPORT Members

Enumerator	Description
ROM_BOOT_DMA_CRC_SUPPORTED	MDMA Channel supports CRC
ROM_BOOT_DMA_CRC_NOT_SUPPORTED	MDMA Channel does not support CRC

ROM_BOOT_DMA_CRC_SUPPORTED

MDMA Channel supports CRC

ROM_BOOT_DMA_CRC_NOT_SUPPORTED

MDMA Channel does not support CRC

enum ROM_BOOT_RESULT

Enumeration Type Declaration: ROM_BOOT_RESULT

General Boot ROM Return Types.

Used throughout the boot software to indicate various success and failure events that can occur during boot.

Table 41-72: ROM_BOOT_RESULT Members

Enumerator	Description
ROM_BOOT_FAILURE	Failure
ROM_BOOT_SUCCESS	Success
ROM_BOOT_HDR_CHKSUM_ERR	Boot Stream Block Header Checksum Error
ROM_BOOT_HDR_SIGN_ERR	Boot Stream Block Header Sign Failure
ROM_BOOT_HDR_DEST_ERR	Boot Stream Block Payload Destination Error
RESERVED0	Reserved
RESERVED1	Reserved
RESERVED2	Reserved
RESERVED3	Reserved
ROM_BOOT_CGU_WRITE_ERR	CGU Write Error
ROM_BOOT_DMA_SUCCESS	DMA operation was successful
ROM_BOOT_DMA_FAILURE	DMA Failure.
ROM_BOOT_DMA_ACTIVE	DMA Channel is Active
ROM_BOOT_DMA_CONFIG_ERR	DMA configuration error
ROM_BOOT_MDMA_ID_ERR	Illegal MDMA Channel ID
ROM_BOOT_MDMA_OPERATION_ERR	Illegal MDMA operation Specified

Table 41-72: ROM_BOOT_RESULT Members (Continued)

Enumerator	Description
ROM_BOOT_MDMA_CONFIG_ERR	Memory DMA configuration error
ROM_BOOT_MDMA_SRC_ERR	MDMA Source Channel Configuration Error
ROM_BOOT_MDMA_DST_ERR	MDMA Destination Channel Configuration Error
ROM_BOOT_MDMA_DONE_DETECT_ERR	Memory DMA Completed with errors
ROM_BOOT_MDMA_SUCCESS	Memory DMA Completed successfully
ROM_BOOT_PDMA_CONFIG_ERR	Peripheral DMA configuration invalid
ROM_BOOT_PDMA_SUCCESS	Peripheral DMA completed successfully
ROM_BOOT_CRC_FAILURE	MDMA CRC32 Failure
ROM_BOOT_CRC_CONFIG_ERR	MDMA CRC32 CONFIG error
ROM_BOOT_CRC_COUNT_ERR	CRC Byte Count was not a multiple 4
ROM_BOOT_CRC_SUPPORTED_ERR	CRC Not Supported Error.
ROM_BOOT_CRC_INITCODE_ERR	CRC32 Enable Failure During Boot
ROM_BOOT_CRC_CALLBACK_ERR	Error in Execution of the CRC Callback
ROM_BOOT_CRC_SUCCESS	MDMA CRC Success

ROM_BOOT_FAILURE

Failure.

General failure can be used to indicate any general failure throughout the boot process

ROM_BOOT_SUCCESS

Success.

General success can be used to indicate any general functional success for an operation during the boot process.

NOTE: This must be the return result for a boot mode drivers initialization, configuration, load and cleanup routines when overriding their functionality in second stage boot loaders to use custom functions.

ROM_BOOT_HDR_CHKSUM_ERR

Boot Stream Block Header Checksum Error.

Indicates that the 8-bit XOR checksum of the 16-byte block header failed to generated the expected result.

ROM_BOOT_HDR_SIGN_ERR

Boot Stream Block Header Sign Failure.

The 0xAD block required as byte 4 of the boot block header was not found.

ROM_BOOT_HDR_DEST_ERR

Boot Stream Block Payload Destination Error.

The target address field of the block header indicates that the payload for the block is destined towards an address that is not supported. Indicates an attempt to load data to the reserved 8 KB non-bootable region of memory reserved by the boot process.

ROM_BOOT_CGU_WRITE_ERR

CGU Write Error

Returned by the CGU configuration routine if a CGU error is set during its initialization from settings provisioned in the OTP.

ROM_BOOT_DMA_SUCCESS

DMA operation was successful

ROM_BOOT_DMA_FAILURE

DMA Failure.

Returned by the DMA routines if an error was detected in the `DMA_STAT.IRQERR` prior to setting up a new DMA operation with the newly supplied configuration.

ROM_BOOT_DMA_ACTIVE

DMA Channel is Active

Returned only by the peripheral DMA routine when an attempt to run another peripheral DMA operation is attempted and the DMA channel is already running.

NOTE: This is not currently implemented for MDMA operations.

ROM_BOOT_DMA_CONFIG_ERR

DMA configuration error

ROM_BOOT_MDMA_ID_ERR

Illegal MDMA Channel ID.

Returned by `adi_rom_MemDma()` if the MDMA channel ID is not supported. For supported channel IDs, see `enum ROM_DMA_MDMA_ID`

ROM_BOOT_MDMA_OPERATION_ERR

Illegal MDMA operation Specified.

Returned by `adi_rom_MemDma()` if the MDMA operation is not supported. For supported operations, see `enum ROM_DMA_MDMA_OPERATION`

ROM_BOOT_MDMA_CONFIG_ERR

Memory DMA configuration error

ROM_BOOT_MDMA_SRC_ERR

MDMA Source Channel Configuration Error.

Set by the MDMA routines if after configuring the MDMA source channel to start a DMA operation, an error is generated in the source channels `DMA_STAT.IRQERR`

ROM_BOOT_MDMA_DST_ERR

MDMA Destination Channel Configuration Error.

Set by the MDMA routines when an error is generated in the destination channels `DMA_STAT.IRQERR` bit (after configuring the MDMA source channel to start a DMA operation)

ROM_BOOT_MDMA_DONE_DETECT_ERR

Memory DMA Completed with errors

ROM_BOOT_MDMA_SUCCESS

Memory DMA Completed successfully

ROM_BOOT_PDMA_CONFIG_ERR

Peripheral DMA configuration invalid

ROM_BOOT_PDMA_SUCCESS

Peripheral DMA completed successfully

ROM_BOOT_CRC_FAILURE

MDMA CRC32 Failure.

Returned by the higher level `adi_rom_MemDma()` routine and the underlying `adi_rom_MemCompare()` routine if the CRC32 result of the block of data did not match the expected result.

ROM_BOOT_CRC_CONFIG_ERR

MDMA CRC32 CONFIG error

ROM_BOOT_CRC_COUNT_ERR

CRC Byte Count was not a multiple 4.

The CRC peripheral operates on 32-bit data only and as such all CRC operations must have a byte count that is a multiple of 4. This result is returned by the higher level `adi_rom_MemDma()` routine and the underlying `adi_rom_MemCompare()` and `adi_rom_MemFill()` routines if the byte count is not a multiple of 4 bytes.

ROM_BOOT_CRC_SUPPORTED_ERR

CRC Not Supported Error.

Returned by `adi_rom_MemDma()`, `adi_rom_MemFill()`, `adi_rom_MemCompare()` and `adi_rom_Crc32Poly()` if the supplied DMA configuration specified a MDMA channel that does not support CRC operations.

ROM_BOOT_CRC_INITCODE_ERR

CRC32 Enable Failure During Boot.

Returned by `adi_rom_Crc32Init()` if the boot process cannot enable the CRC32 functionality due to a NULL `struct ADI_ROM_BOOT_CONFIG` pointer or NULL `struct ADI_ROM_BOOT_HEADER` pointer located in `pHeader` Pointer to the boot header storage location where all boot stream block headers eventually reside for processing by the kernel

ROM_BOOT_CRC_CALLBACK_ERR

Error in Execution of the CRC Callback.

Returned by the default CRC callback function located in the boot ROM if any of the following conditions are met:

- The `struct ADI_ROM_BOOT_CONFIG` pointer passed to the callback is a NULL pointer
- The `struct ADI_ROM_BOOT_BUFFER` pointer to the buffer to run CRC validation is a NULL pointer
- The `ROM_CBFLAG_DIRECT` flag is not set in the supplied flags parameter indicating it was a direct callback
- The `struct ADI_ROM_BOOT_HEADER` pointer located in `pHeader` Pointer to the boot header storage location where all boot stream block headers eventually reside for processing by the kernel is a NULL pointer

ROM_BOOT_CRC_SUCCESS

MDMA CRC Success

enum ROM_CORE_ID

Enumeration Type Declaration: `ROM_CORE_ID`

Core ID.

An enumeration for referencing a particular core

Table 41-73: ROM_CORE_ID Members

Enumerator	Description
ROM_CORE_ID0	Core 0
ROM_CORE_NUM_CORES	Number of Cores
ROM_CORE_ID1	Core 1
ROM_CORE_ID2	Core 2

ROM_CORE_ID0

Core 0

ROM_CORE_NUM_CORES

Number of Cores

enum ROM_DMA_DONE_DETECT_METHOD

Enumeration Type Declaration: ROM_DMA_DONE_DETECT_METHOD

DMA Done Detection Method.

Specifies the method used to detect the completion of the requested DMA operation.

When a program requests a non-blocking DMA operation, separate software is required to check the status of the DMA channel. The boot ROM does not provide an API for this operation.

NOTE: Trigger mode is not supported on this product

Table 41-74: ROM_DMA_DONE_DETECT_METHOD Members

Enumerator	Description
ROM_DMA_DONE_NON_BLOCKING	Return without waiting for the DMA to complete
ROM_DMA_DONE_POLL_IRQDONE	Poll on the IRQDONE bit in the DMA Status register
ROM_DMA_DONE_WAKEUP_TRIGGER	Configure a trigger to wakeup the core on completion

ROM_DMA_DONE_NON_BLOCKING

Return without waiting for the DMA to complete

ROM_DMA_DONE_POLL_IRQDONE

Poll on the IRQDONE bit in the DMA Status register

ROM_DMA_DONE_WAKEUP_TRIGGER

Configure a trigger to wakeup the core on completion

enum ROM_DMA_MDMA_ID

Enumeration Type Declaration: ROM_DMA_MDMA_ID

MDMA Channel ID.

The ID of the Memory DMA channel. Used in the `struct ROM_DMA_MDMA_CONFIG` configuration to specify the Memory DMA channel to use for operations accessible using the `adi_rom_MemDma()` routine

Table 41-75: ROM_DMA_MDMA_ID Members

Enumerator	Description
ROM_DMA_MDMA0	Memory DMA Stream 0
ROM_DMA_MDMA1	Memory DMA Stream 1
ROM_DMA_MDMA2	Memory DMA Stream 2
ROM_DMA_MDMA3	Memory DMA Stream 3
ROM_DMA_MEMDMA_END_COUNT	Number of Memory DMA Streams

ROM_DMA_MDMA0

Memory DMA Stream 0

ROM_DMA_MDMA1

Memory DMA Stream 1

ROM_DMA_MDMA2

Memory DMA Stream 2

ROM_DMA_MDMA3

Memory DMA Stream 3

ROM_DMA_MEMDMA_END_COUNT

Number of Memory DMA Streams

enum ROM_DMA_MDMA_OPERATION

Enumeration Type Declaration: ROM_DMA_MDMA_OPERATION

MDMA Operation.

The operation determines if only an MDMA is required, or whether a CRC operation must be used in conjunction with the MDMA.

Table 41-76: ROM_DMA_MDMA_OPERATION Members

Enumerator	Description
ROM_DMA_MEM_COPY	Standard MDMA transfer from a source to a destination
ROM_DMA_MEM_CRC	Performs a CRC32 MDMA read operation and compares the result with an expected result
ROM_DMA_MEM_FILL	Uses the CRC peripheral to perform a fill operation with a 32-bit value
ROM_DMA_MEM_COMPARE	Uses the CRC peripheral to compare data with a constant 32-bit value
ROM_DMA_CRC_LUT_INIT	Initializes the CRC LUT from the supplied CRC Polynomial

ROM_DMA_MEM_COPY

Standard MDMA transfer from a source to a destination

ROM_DMA_MEM_CRC

Performs a CRC32 MDMA read operation and compares the result with an expected result

ROM_DMA_MEM_FILL

Uses the CRC peripheral to perform a fill operation with a 32-bit value

ROM_DMA_MEM_COMPARE

Uses the CRC peripheral to compare data with a constant 32-bit value

ROM_DMA_CRC_LUT_INIT

Initializes the CRC LUT from the supplied CRC Polynomial

enum ROM_DMA_PDMA_OPERATION

Enumeration Type Declaration: ROM_DMA_PDMA_OPERATION

Table 41-77: ROM_DMA_PDMA_OPERATION Members

Enumerator	Description
ROM_DMA_PERI_TX	Peripheral Transmit Operation
ROM_DMA_PERI_RX	Peripheral Receive Operation

ROM_DMA_PERI_TX

Peripheral Transmit Operation

ROM_DMA_PERI_RX

Peripheral Receive Operation

enum ROM_GETADDR_VALUE

Enumeration Type Declaration: ROM_GETADDR_VALUE

Parameter for `adi_rom_GetAddress()` function to retrieve the address of a data object stored in the boot ROM.

Table 41-78: ROM_GETADDR_VALUE Members

Enumerator	Description
ROM_GETADDR_CONSTANTS	Retrieve the address of the ROM_CONSTANTS_TYPE object
ROM_GETADDR_BMODE	Retrieve the address of the lookup table sorting the default <code>adi_rom_boot()</code> parameters for each boot mode
ROM_GETADDR_MDMAREGS	Retrieve the address of the <code>struct ROM_BOOT_MDMA_REGS</code> object
ROM_GETADDR_SPILUT	Retrieve the address of the lookup table in the ROM describing the various SPI master boot BCODE configurations
ROM_GETADDR_ECDSA_DOMAIN	Retrieve the address of the domain parameters used for ECDSA

ROM_GETADDR_CONSTANTS

Retrieve the address of the ROM_CONSTANTS_TYPE object

ROM_GETADDR_BMODE

Retrieve the address of the lookup table sorting the default `adi_rom_boot()` parameters for each boot mode

ROM_GETADDR_MDMAREGS

Retrieve the address of the `struct ROM_BOOT_MDMA_REGS` object

ROM_GETADDR_SPILUT

Retrieve the address of the lookup table in the rom describing the various SPI master boot BCODE configurations

ROM_GETADDR_ECDSA_DOMAIN

Retrieve the address of the domain parameters used for ECDSA

enum ROM_HOOK_CALL_CAUSE

Enumeration Type Declaration: ROM_HOOK_CALL_CAUSE

Passed to a user hook routine to indicate the reason of the call.

An optional hook routine is provided as a callback when calling a boot mode via `adi_rom_Boot`. This hook routine is called by the boot software first after the execution of the boot modes initialization routine then again after execution of the boot modes configuration routine. This parameter allows the users routine to identify at which point the call was made allowing the user to perform different actions for each call.

Table 41-79: ROM_HOOK_CALL_CAUSE Members

Enumerator	Description
ROM_HOOK_CALL_INIT_COMPLETE	Call was as a result of completion of the boot modes initialization function
ROM_HOOK_CALL_CONFIG_COMPLETE	Call was as a result of the completion of the boot modes configuration function
ROM_HOOK_REG_COMPLETE	Call was as a result of the completion of the boot modes pre-register initialization

enum ROM_SB_IMAGE_TYPE

Enumeration Type Declaration: ROM_SB_IMAGE_TYPE

Secure Boot Image Types.

The secure boot header contains a type field for the secure boot image type, this enumeration provides a complete list of all image types.

NOTE: The secure boot process does not necessarily support all image types defined.

Table 41-80: ROM_SB_IMAGE_TYPE Members

Enumerator	Description
ROM_SB_IMAGE_UNKNOWN	Unknown Secure Boot image type, used by software to initialize the type before detection of boot image type takes place
ROM_SB_IMAGE_BLP	Plain text BLP secure boot image supports authentication only with no decryption
ROM_SB_IMAGE_BLW	Keywrapped BLW secure boot image supports authentication and decryption, boot stream decryption key wrapped in the secure header
ROM_SB_IMAGE_BLE	Not supported by any Secure boot products. Secure boot image with key stored in plain text form in the secure header
ROM_SB_IMAGE_BLX	BLX Secure boot image supports authentication and decryption, boot stream decryption key wrapped located in OTP
ROM_SB_IMAGE_UNSUPPORTED	Used by software to indicate any other unsupported image type

ROM_SB_IMAGE_UNKNOWN

Unknown Secure Boot image type, used by software to initialize the type before detection of boot image type takes place

ROM_SB_IMAGE_BLP

Plain text BLP secure boot image supports authentication only with no decryption

ROM_SB_IMAGE_BLW

Keywrapped BLW secure boot image supports authentication and decryption, boot stream decryption key wrapped in the secure header

ROM_SB_IMAGE_BLE

Not supported by any Secure boot products. Secure boot image with key stored in plain text form in the secure header

ROM_SB_IMAGE_BLX

BLx Secure boot image supports authentication and decryption, boot stream decryption key wrapped located in OTP

ROM_SB_IMAGE_UNSUPPORTED

Used by software to indicate any other unsupported image type

enum ROM_SPI_PROTOCOL

Enumeration Type Declaration: ROM_SPI_PROTOCOL

The SPI Protocol to use.

SPI Flash devices can support multiple protocols to send commands to the SPI flash. Commands are usually sent over the single-bit bus, however a number of newer devices also support sending commands over the dual- or quad-bit bus.

WARNING: In system reset type events where the processor attempts to reboot and the flash may not have been reset, using the DUALIO or QUADIO protocols for the command cycles risks the boot process being unable to communicate with the SPI flash. Do not enable these features on SPI Flash devices if they are also the primary boot source used for booting from hardware reset and system reset events.

Table 41-81: ROM_SPI_PROTOCOL Members

Enumerator	Description
ROM_SPI_EXT_PROTOCOL	Extended protocol where the command cycle is sent on the single bit bus
ROM_SPI_DUALIO_PROTOCOL	DualIO protocol where the command cycle is sent on the dual bit bus
ROM_SPI_QUADIO_PROTOCOL	QuadIO protocol where the command cycle is sent on the quad bit bus

ROM_SPI_EXT_PROTOCOL

Extended protocol where the command cycle is sent on the single bit bus

ROM_SPI_DUALIO_PROTOCOL

DualIO protocol where the command cycle is sent on the dual bit bus

ROM_SPI_QUADIO_PROTOCOL

QuadIO protocol where the command cycle is sent on the quad bit bus

42 System Crossbars (SCB)

A modern system on a chip (SoCs) contains multi-cores, memory controllers, security blocks, and other high speed peripherals. As system integration increases, SoCs need to provide bus connectivity that allows better throughput to reduce performance bottlenecks. While traditional point-to-point connection buses have performed well in smaller systems, there is a need to use advanced switching based bus architectures for efficient handling of data transfer between multiplicity of data sources and sinks in the system. Additionally, mixing various traffic types in the same SoC (for example control, communication over peripherals and computing) while sharing the same bus resources, create different requirements from the Quality of Service (QoS) perspective.

The system crossbars (SCB) are the fundamental building blocks of a switch-fabric style for (on-chip) system bus interconnection. The SCBs connect system bus masters to system bus slaves, providing concurrent data transfer between multiple bus masters and multiple bus slaves. The SCB architecture addresses the challenges described above. The SCB provides sustainable throughput for simultaneous transactions in the system with configurable quality of service for each type of transaction (traffic) as required. A hierarchical model, built from multiple SCBs, provides a power and area efficient system interconnect, which satisfies the performance and flexibility requirements of a specific system.

SCB Features

The SCBs provide the following features:

- Efficient, pipelined bus transfer protocol for sustained throughput
- Full-duplex bus operation for flexibility and reduced latency
- Concurrent bus transfer support to allow multiple bus masters to access bus slaves simultaneously
- Protection model (privileged or secure) support for selective bus interconnect protection
- Simple priority-based QoS based arbitration model
- Programmable quality of service

SCB Functional Description

The following sections provide a functional description of the SCB.

- [SCB Architectural Concepts](#)

ADSP-2159x SCB0 Register List

Table 42-1: ADSP-2159x SCB0 Register List

Name	Description
SCB0_CRC0_CH0_READ_QOS	CRC0 Channel 0 Read Quality of Service Register
SCB0_CRC0_CH0_WRITE_QOS	CRC0 Channel 0 Write Quality of Service Register
SCB0_CRC0_CH1_READ_QOS	CRC0 Channel 1 Read Quality of Service Register
SCB0_CRC0_CH1_WRITE_QOS	CRC0 Channel 1 Write Quality of Service Register
SCB0_CRC1_CH0_READ_QOS	CRC1 Channel 0 Read Quality of Service Register
SCB0_CRC1_CH0_WRITE_QOS	CRC1 Channel 0 Write Quality of Service Register
SCB0_CRC1_CH1_READ_QOS	CRC1 Channel 1 Read Quality of Service Register
SCB0_CRC1_CH1_WRITE_QOS	CRC1 Channel 1 Write Quality of Service Register
SCB0_CRC2_CH0_READ_QOS	CRC2 Channel 0 Read Quality of Service Register
SCB0_CRC2_CH0_WRITE_QOS	CRC2 Channel 0 Write Quality of Service Register
SCB0_CRC2_CH1_READ_QOS	CRC2 Channel 1 Read Quality of Service Register
SCB0_CRC2_CH1_WRITE_QOS	CRC2 Channel 1 Write Quality of Service Register
SCB0_CRC3_CH0_READ_QOS	CRC3 Channel 0 Read Quality of Service Register
SCB0_CRC3_CH0_WRITE_QOS	CRC3 Channel 0 Write Quality of Service Register
SCB0_CRC3_CH1_READ_QOS	CRC3 Channel 1 Read Quality of Service Register
SCB0_CRC3_CH1_WRITE_QOS	CRC3 Channel 1 Write Quality of Service Register
SCB0_CRYPT0_READ_QOS	CRYPTO Read Quality of Service Register
SCB0_CRYPT0_WRITE_QOS	CRYPTO Write Quality of Service Register
SCB0_DBG_READ_QOS	DBG Read Quality of Service Register
SCB0_DBG_WRITE_QOS	DBG Write Quality of Service Register
SCB0_DLDMA0_CH0_READ_QOS	DLDMA0 Channel 0 Read Quality of Service Register
SCB0_DLDMA0_CH0_WRITE_QOS	DLDMA0 Channel 0 Write Quality of Service Register
SCB0_DLDMA0_CH1_READ_QOS	DLDMA0 Channel 1 Read Quality of Service Register
SCB0_DLDMA0_CH1_WRITE_QOS	DLDMA0 Channel 1 Write Quality of Service Register
SCB0_DLDMA1_CH0_READ_QOS	DLDMA1 Channel 0 Read Quality of Service Register
SCB0_DLDMA1_CH0_WRITE_QOS	DLDMA1 Channel 0 Write Quality of Service Register
SCB0_DLDMA1_CH1_READ_QOS	DLDMA1 Channel 1 Read Quality of Service Register
SCB0_DLDMA1_CH1_WRITE_QOS	DLDMA1 Channel 1 Write Quality of Service Register

Table 42-1: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_EMAC_READ_QOS	EMAC Read Quality of Service Register
SCB0_EMAC_WRITE_QOS	EMAC Write Quality of Service Register
SCB0_ETR_READ_QOS	ETR Read Quality of Service Register
SCB0_ETR_WRITE_QOS	ETR Write Quality of Service Register
SCB0_GIGE_READ_QOS	GIGE Read Quality of Service Register
SCB0_GIGE_WRITE_QOS	GIGE Write Quality of Service Register
SCB0_HSMDMA1_CH0_READ_QOS	HSMDMA1 Channel 0 Read Quality of Service Register
SCB0_HSMDMA1_CH0_WRITE_QOS	HSMDMA1 Channel 0 Write Quality of Service Register
SCB0_HSMDMA1_CH1_READ_QOS	HSMDMA1 Channel 1 Read Quality of Service Register
SCB0_HSMDMA1_CH1_WRITE_QOS	HSMDMA1 Channel 1 Write Quality of Service Register
SCB0_HSMDMA_CH0_READ_QOS	HSMDMA Channel 0 Read Quality of Service Register
SCB0_HSMDMA_CH0_WRITE_QOS	HSMDMA Channel 0 Write Quality of Service Register
SCB0_HSMDMA_CH1_READ_QOS	HSMDMA Channel 1 Read Quality of Service Register
SCB0_HSMDMA_CH1_WRITE_QOS	HSMDMA Channel 1 Write Quality of Service Register
SCB0_LP0_READ_QOS	LP0 Read Quality of Service Register
SCB0_LP0_WRITE_QOS	LP0 Write Quality of Service Register
SCB0_LP1_READ_QOS	LP1 Read Quality of Service Register
SCB0_LP1_WRITE_QOS	LP1 Write Quality of Service Register
SCB0_MLB_READ_QOS	MLB Read Quality of Service Register
SCB0_MLB_WRITE_QOS	MLB Write Quality of Service Register
SCB0_MSMDMA1_CH0_READ_QOS	MSMDMA1 Channel 0 Read Quality of Service Register
SCB0_MSMDMA1_CH0_WRITE_QOS	MSMDMA1 Channel 0 Write Quality of Service Register
SCB0_MSMDMA1_CH1_READ_QOS	MSMDMA1 Channel 1 Read Quality of Service Register
SCB0_MSMDMA1_CH1_WRITE_QOS	MSMDMA1 Channel 1 Write Quality of Service Register
SCB0_MSMDMA_CH0_READ_QOS	MSMDMA Channel 0 Read Quality of Service Register
SCB0_MSMDMA_CH0_WRITE_QOS	MSMDMA Channel 0 Write Quality of Service Register
SCB0_MSMDMA_CH1_READ_QOS	MSMDMA Channel 1 Read Quality of Service Register
SCB0_MSMDMA_CH1_WRITE_QOS	MSMDMA Channel 1 Write Quality of Service Registers
SCB0_PL310_M0_READ_QOS	ARM_L2CC M0 Read Quality of Service Register
SCB0_PL310_M0_WRITE_QOS	ARM_L2CC M0 Write Quality of Service Register
SCB0_PL310_M1_READ_QOS	ARM_L2CC M1 Read Quality of Service Register

Table 42-1: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_PL310_M1_WRITE_QOS	ARM_L2CC M1 Write Quality of Service Register
SCB0_PPI_F0_READ_QOS	PPI F0 Read Quality of Service Register
SCB0_PPI_F0_WRITE_QOS	PPI F0 Write Quality of Service Register
SCB0_PPI_F1_READ_QOS	PPI F1 Read Quality of Service Register
SCB0_PPI_F1_WRITE_QOS	PPI F1 Write Quality of Service Register
SCB0_LP_SYNC_MODE	LP Fabric (CLKO8) Synchronization Mode Register
SCB0_SH0_DPORT_READ_QOS	SH0 DPORT Read Quality of Service Register
SCB0_SH0_DPORT_WRITE_QOS	SH0 DPORT Write Quality of Service Register
SCB0_SH0_FIR_CH0_READ_QOS	SH0 FIR Channel 0 Read Quality of Service Register
SCB0_SH0_FIR_CH0_WRITE_QOS	SH0 FIR Channel 0 Write Quality of Service Register
SCB0_SH0_FIR_CH1_READ_QOS	SH0 FIR Channel 1 Read Quality of Service Register
SCB0_SH0_FIR_CH1_WRITE_QOS	SH0 FIR Channel 1 Write Quality of Service Register
SCB0_SH0_IIR_CH0_READ_QOS	SH0 IIR Channel 0 Read Quality of Service Register
SCB0_SH0_IIR_CH0_WRITE_QOS	SH0 IIR Channel 0 Write Quality of Service Register
SCB0_SH0_IIR_CH1_READ_QOS	SH0 IIR Channel 1 Read Quality of Service Register
SCB0_SH0_IIR_CH1_WRITE_QOS	SH0 IIR Channel 1 Write Quality of Service Register
SCB0_SH0_IPORT_READ_QOS	SH0 LPORT Read Quality of Service Register
SCB0_SH0_IPORT_WRITE_QOS	SH0 LPORT Write Quality of Service Register
SCB0_SH0_MMR_READ_QOS	SH0 MMR Read Quality of Service Register
SCB0_SH0_MMR_WRITE_QOS	SH0 MMR Write Quality of Service Register
SCB0_SH1_DPORT_READ_QOS	SH1 DPORT Read Quality of Service Register
SCB0_SH1_DPORT_WRITE_QOS	SH1 DPORT Write Quality of Service Register
SCB0_SH1_FIR_CH0_READ_QOS	SH1 FIR Channel 0 Read Quality of Service Register
SCB0_SH1_FIR_CH0_WRITE_QOS	SH1 FIR Channel 0 Write Quality of Service Register
SCB0_SH1_FIR_CH1_READ_QOS	SH1 FIR Channel 1 Read Quality of Service Register
SCB0_SH1_FIR_CH1_WRITE_QOS	SH1 FIR Channel 1 Write Quality of Service Register
SCB0_SH1_IIR_CH0_READ_QOS	SH1 IIR Channel 0 Read Quality of Service Register
SCB0_SH1_IIR_CH0_WRITE_QOS	SH1 IIR Channel 0 Write Quality of Service Register
SCB0_SH1_IIR_CH1_READ_QOS	SH1 IIR Channel 1 Read Quality of Service Register
SCB0_SH1_IIR_CH1_WRITE_QOS	SH1 IIR Channel 1 Write Quality of Service Register
SCB0_SH1_IPORT_READ_QOS	SH1 IPORT Read Quality of Service Register

Table 42-1: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_SH1_IPORT_WRITE_QOS	SH1 IPORT Write Quality of Service Register
SCB0_SH1_MMR_READ_QOS	SH1 MMR Read Quality of Service Register
SCB0_SH1_MMR_WRITE_QOS	SH1 MMR Write Quality of Service Register
SCB0_SP0A_READ_QOS	SP0A Read Quality of Service Register
SCB0_SP0A_WRITE_QOS	SP0A Write Quality of Service Register
SCB0_SP0B_READ_QOS	SP0B Read Quality of Service Register
SCB0_SP0B_WRITE_QOS	SP0B Write Quality of Service Register
SCB0_SP1A_READ_QOS	SP1A Read Quality of Service Register
SCB0_SP1A_WRITE_QOS	SP1A Write Quality of Service Register
SCB0_SP1B_READ_QOS	SP1B Read Quality of Service Register
SCB0_SP1B_WRITE_QOS	SP1B Write Quality of Service Register
SCB0_SP2A_READ_QOS	SP2A Read Quality of Service Register
SCB0_SP2A_WRITE_QOS	SP2A Write Quality of Service Register
SCB0_SP2B_READ_QOS	SP2B Read Quality of Service Register
SCB0_SP2B_WRITE_QOS	SP2B Write Quality of Service Register
SCB0_SP3A_READ_QOS	SP3A Read Quality of Service Register
SCB0_SP3A_WRITE_QOS	SP3A Write Quality of Service Register
SCB0_SP3B_READ_QOS	SP3B Read Quality of Service Register
SCB0_SP3B_WRITE_QOS	SP3B Write Quality of Service Register
SCB0_SP4A_READ_QOS	SP4A Read Quality of Service Register
SCB0_SP4A_WRITE_QOS	SP4A Write Quality of Service Register
SCB0_SP4B_READ_QOS	SP4B Read Quality of Service Register
SCB0_SP4B_WRITE_QOS	SP4B Write Quality of Service Register
SCB0_SP5A_READ_QOS	SP5A Read Quality of Service Register
SCB0_SP5A_WRITE_QOS	SP5A Write Quality of Service Register
SCB0_SP5B_READ_QOS	SP5B Read Quality of Service Register
SCB0_SP5B_WRITE_QOS	SP5B Write Quality of Service Register
SCB0_SP6A_READ_QOS	SP6A Read Quality of Service Register
SCB0_SP6A_WRITE_QOS	SP6A Write Quality of Service Registers
SCB0_SP6B_READ_QOS	SP6B Read Quality of Service Register
SCB0_SP6B_WRITE_QOS	SP6B Write Quality of Service Register

Table 42-1: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_SP7A_READ_QOS	SP7A Read Quality of Service Register
SCB0_SP7A_WRITE_QOS	SP7A Write Quality of Service Register
SCB0_SP7B_READ_QOS	SP7B Read Quality of Service Register
SCB0_SP7B_WRITE_QOS	SP7B Write Quality of Service Register
SCB0_SPI0RX_READ_QOS	SPI0 RX Read Quality of Service Register
SCB0_SPI0RX_WRITE_QOS	SPI0 RX Write Quality of Service Register
SCB0_SPI0TX_READ_QOS	SPI0 TX Read Quality of Service Register
SCB0_SPI0TX_WRITE_QOS	SPI0 TX Write Quality of Service Register
SCB0_SPI1RX_READ_QOS	SPI1 RX Read Quality of Service Register
SCB0_SPI1RX_WRITE_QOS	SPI1 RX Write Quality of Service Register
SCB0_SPI1TX_READ_QOS	SPI1 TX Read Quality of Service Register
SCB0_SPI1TX_WRITE_QOS	SPI1 TX Write Quality of Service Register
SCB0_SPI2RX_READ_QOS	SPI2 RX Read Quality of Service Register
SCB0_SPI2RX_WRITE_QOS	SPI2 RX Write Quality of Service Register
SCB0_SPI2TX_READ_QOS	SPI2 TX Read Quality of Service Register
SCB0_SPI2TX_WRITE_QOS	SPI2 TX Write Quality of Service Register
SCB0_SPI3RX_READ_QOS	SPI3 RX Read Quality of Service Register
SCB0_SPI3RX_WRITE_QOS	SPI3 RX Write Quality of Service Register
SCB0_SPI3TX_READ_QOS	SPI3 TX Read Quality of Service Register
SCB0_SPI3TX_WRITE_QOS	SPI3 TX Write Quality of Service Register
SCB0_UART0_RX_READ_QOS	UART0 RX Read Quality of Service Register
SCB0_UART0_RX_WRITE_QOS	UART0 RX Write Quality of Service Register
SCB0_UART0_TX_READ_QOS	UART0 TX Read Quality of Service Register
SCB0_UART0_TX_WRITE_QOS	UART0 TX Write Quality of Service Register
SCB0_UART1_RX_READ_QOS	UART1 RX Read Quality of Service Register
SCB0_UART1_RX_WRITE_QOS	UART1 RX Write Quality of Service Register
SCB0_UART1_TX_READ_QOS	UART1 TX Read Quality of Service Register
SCB0_UART1_TX_WRITE_QOS	UART1 TX Write Quality of Service Registers
SCB0_UART2_RX_READ_QOS	UART2 RX Read Quality of Service Register
SCB0_UART2_RX_WRITE_QOS	UART2 RX Write Quality of Service Register
SCB0_UART2_TX_READ_QOS	UART2 TX Read Quality of Service Register

Table 42-1: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_UART2_TX_WRITE_QOS	UART2 TX Write Quality of Service Register
SCB0_UART3_RX_READ_QOS	UART3 RX Read Quality of Service Register
SCB0_UART3_RX_WRITE_QOS	UART3 RX Write Quality of Service Register
SCB0_UART3_TX_READ_QOS	UART3 TX Read Quality of Service Register
SCB0_UART3_TX_WRITE_QOS	UART3 TX Write Quality of Service Register
SCB0_USB0_READ_QOS	USB0 Read Quality of Service Register
SCB0_USB0_WRITE_QOS	USB0 Write Quality of Service Register

ADSP-2159x SCB1 Register List

Table 42-2: ADSP-2159x SCB1 Register List

Name	Description
SCB1_DMC_IB_SYNC_MODE	DMC Fabric (CLK03) Synchronization Mode Register

ADSP-2159x SCB3 Register List

Table 42-3: ADSP-2159x SCB3 Register List

Name	Description
SCB3_DMC_MMR_IB_SYNC_MODE	DMC MMRG Fabric (CLK03) Synchronization Mode Register
SCB3_CAN_MMR_IB_SYNC_MODE	DMC MMRG Fabric (CLK04) Synchronization Mode Register
SCB3_LP_MMR_IB_SYNC_MODE	DMC MMRG Fabric (CLK08) Synchronization Mode Register

ADSP-2159x SCB4 Register List

Table 42-4: ADSP-2159x SCB4 Register List

Name	Description
SCB4_FABRIC_ACC_MMR_IB_READ_QOS	Fabric Acc Mmr Ib.read Qos
SCB4_FABRIC_ACC_MMR_IB_WRITE_QOS	Fabric Acc Mmr Ib.write Qos
SCB4_FABRIC_S1PORT_IB_READ_QOS	Fabric S1port Ib.read Qos
SCB4_FABRIC_S1PORT_IB_WRITE_QOS	Fabric S1port Ib.write Qos
SCB4_FABRIC_S2PORT_IB_READ_QOS	Fabric S2port Ib.read Qos
SCB4_FABRIC_S2PORT_IB_WRITE_QOS	Fabric S2port Ib.write Qos
SCB4_FIR_CH0_IB_READ_QOS	Fir Ch0 Ib.read Qos

Table 42-4: ADSP-2159x SCB4 Register List (Continued)

Name	Description
SCB4_FIR_CH0_IB_WRITE_QOS	Fir Ch0 Ib.write Qos
SCB4_FIR_CH1_IB_READ_QOS	Fir Ch1 Ib.read Qos
SCB4_FIR_CH1_IB_WRITE_QOS	Fir Ch1 Ib.write Qos
SCB4_IIR0_CH0_IB_READ_QOS	Iir0 Ch0 Ib.read Qos
SCB4_IIR0_CH0_IB_WRITE_QOS	Iir0 Ch0 Ib.write Qos
SCB4_IIR0_CH1_IB_READ_QOS	Iir0 Ch1 Ib.read Qos
SCB4_IIR0_CH1_IB_WRITE_QOS	Iir0 Ch1 Ib.write Qos
SCB4_IIR1_CH0_IB_READ_QOS	Iir1 Ch0 Ib.read Qos
SCB4_IIR1_CH0_IB_WRITE_QOS	Iir1 Ch0 Ib.write Qos
SCB4_IIR1_CH1_IB_READ_QOS	Iir1 Ch1 Ib.read Qos
SCB4_IIR1_CH1_IB_WRITE_QOS	Iir1 Ch1 Ib.write Qos
SCB4_IIR2_CH0_IB_READ_QOS	Iir2 Ch0 Ib.read Qos
SCB4_IIR2_CH0_IB_WRITE_QOS	Iir2 Ch0 Ib.write Qos
SCB4_IIR2_CH1_IB_READ_QOS	Iir2 Ch1 Ib.read Qos
SCB4_IIR2_CH1_IB_WRITE_QOS	Iir2 Ch1 Ib.write Qos
SCB4_IIR3_CH0_IB_READ_QOS	Iir3 Ch0 Ib.read Qos
SCB4_IIR3_CH0_IB_WRITE_QOS	Iir3 Ch0 Ib.write Qos
SCB4_IIR3_CH1_IB_READ_QOS	Iir3 Ch1 Ib.read Qos
SCB4_IIR3_CH1_IB_WRITE_QOS	Iir3 Ch1 Ib.write Qos
SCB4_SHARC_DPORT_READ_QOS	Sharc Dport.read Qos
SCB4_SHARC_DPORT_WRITE_QOS	Sharc Dport.write Qos

ADSP-2159x SCB5 Register List

Table 42-5: ADSP-2159x SCB5 Register List

Name	Description
SCB5_SPI2_OSPI_REMAP	SPI2/OSPI Memory Map Address Remap Register

SCB Architectural Concepts

This section describes the components of the SCB and the modules connected to it. The basic elements in the SCB are SCB masters, slaves master interfaces, and slave interfaces.

Masters

The controllers in the system that raise the data request in the form of a read/write transaction on the SCB are called masters. The system bus masters include peripheral Direct Memory Access (DMA) channels. These include the Serial Port (SPORT) DMA, and SPI DMAs, among others. Also included are the Memory-to-Memory DMA channels (MDMA), the L1 code fill block, and the processor cores.

Slaves

Slaves are SCB connections that are responding to transfer requests. Slaves include MMR registers, memory units, and various peripherals depending upon individual configurations. Each system slave has its own latencies and response times.

SCB Block Diagram

The SCB architectural model is shown in the *SCB Overview* figure. This figure shows a high level representation of a basic SCB connecting n Slaves to x Masters. A variable number of masters connect to a variable number of slaves in each SCB. In this example, all SIs connect to all MIs as indicated by the lines connecting them.

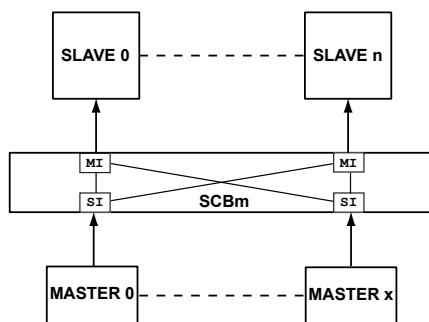


Figure 42-1: SCB Overview

Hierarchy Block Diagram

A system interconnect built from multiple SCBs in a hierarchical model is illustrated in the *SCB Hierarchy Overview* figure. The system master node level SCBs master connects multiple SIs to a single MI, which in turn connects to an SI of the system slave level node SCB.

As discussed above, all the masters in the system are distributed across different SCBs. A given SCB at system master node level connects directly to the system masters. These SCBs connect to SCB0 through its SIs forming a hierarchical structure. While a master has to access any slave, its first access goes through the SCB it is connected to, and then through SCB0, to reach its intended slave. This simplifies the connecting hardware in the basic SCB block by limiting the masters. Care must be taken when sharing masters to allow adequate throughput for their individual data transfer requirements.

In this example, all SIs are connected to all MIs.

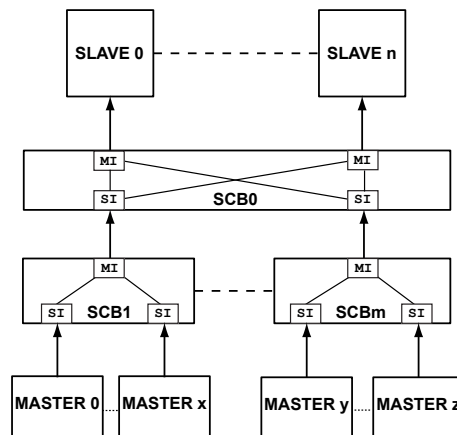


Figure 42-2: SCB Hierarchy Overview

NOTE: For an overall diagram of all SCB interconnections, see the SCB Block Diagram.

SCB Block Diagram

The *SCB Block Diagram* shows the functional blocks of the SCB module.

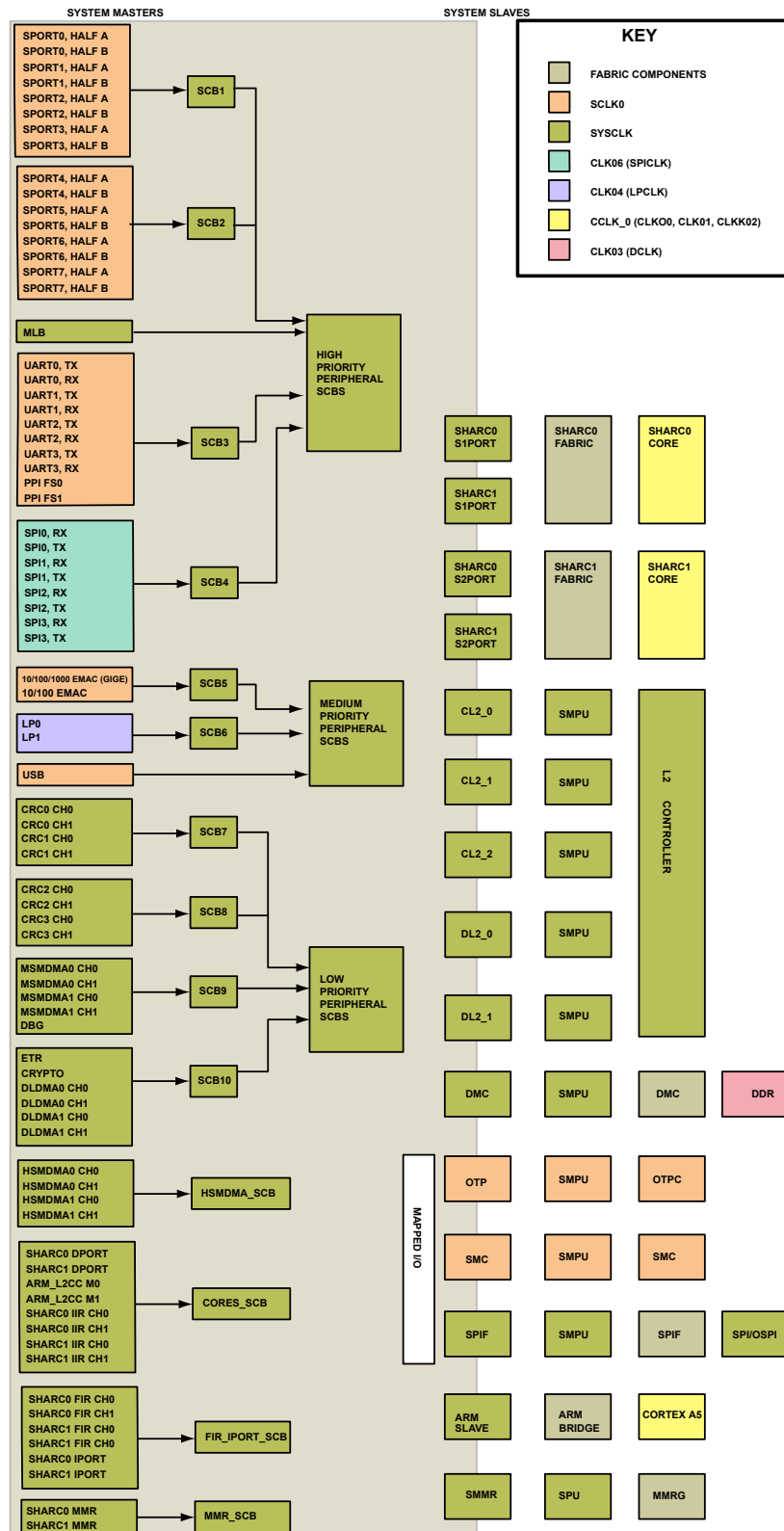


Figure 42-3: SCB Block Diagram

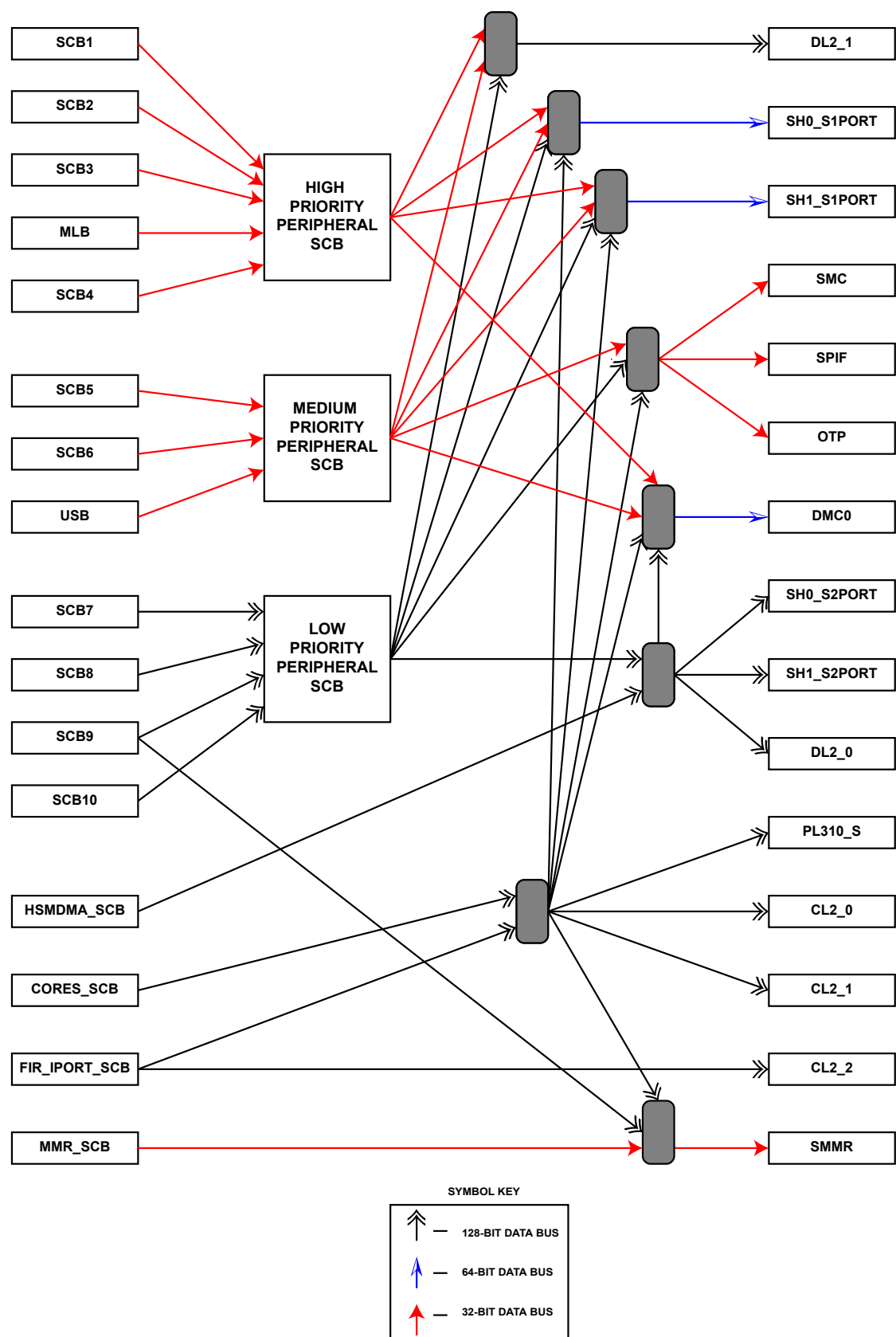


Figure 42-4: SCB Interconnections

The following are important points related to the system fabric on the ADSP-SC59x processors.

- The hierarchy of SCBs manages the system bus interconnections, multiplexing, and arbitration among the peripherals on the processor.
- The SCBs connections support DMA channels for some peripherals and support dedicated connections for others. The connections also support memory-mapped register access for internal memory (L1 and L2) and for external memory.

The slave interface of the crossbar (where masters such as DMA connect to performs two functions. The first function is arbitration. SCBx handles arbitration. The second function is clock conversion. The programmable QoS registers can be viewed as being associated with the SCBx.

- Most of the peripherals and their SCBs are in the *SCLK0* domain. MDMA0-7, EMDMA0/1, Crypto, FIR/IIR accelerators, MLB, DBG, ETR, SHARC+ and Arm cores and their SCBs are in the *SYSCLK* domain. The link port is in the CLK04 domain. SPI is in the CLK06 domain.
- Each peripheral has a latency for access across the SCB. The latency varies with the nature of the peripheral. Also, the number of active peripherals (especially for cases where multiple peripherals are active on a shared SCB) affects SCB performance. Refer to the [L2 System Memory](#) chapter for details on the L2 memory organization.

MDMA channels are unidirectional. For example, MDMA SRC is read-only and MDMA is write-only. Access to the MMR space of SCB0 is allowed only in secure mode. The MMR space of SCB0 has registers for programming the QoS of various masters and controlling the clock domain crossing.

- Access to the GPV (Global Programmers View) space is allowed only in secure mode. This space has registers for programming the QoS and CDC relationships and the remapping of various masters.
- The S2 slave port of SHARCs is intended primarily for MDMA between the L1 of the 2 SHARCs. Hence, only the HSMDMA0 (MDMA3) and HSMDMA1 (MDMA7) is given access to S2 ports while all others masters are given access to S1 port. This arrangement also means that S1 and S2 port share the same address space.
- SPI2/OSPI0 is low bandwidth slave – SYSCLK with a 32-bit interface. Only MDMA0/1/2/4/5/6 is used to perform MDMA to these slave nodes. HSMDMA0/1 (MDMA4 and 7) are intended for L1, L2 and DDR (these slaves have the same bandwidth capability as HSMDMA0/1).
- SPIF is a read-only flash. Peripherals are not able to read out of the SPI flash directly. Only MDMAs, crypto, cores, and debug tools are given access to it.
- LP to SPI2/OSPI0 access is allowed.
- The SPIF slave in the system fabric has remapping options in the SPIF fabric for SPI2 and OSPI0 in the same address space. The address map configuration is programmable using remap registers.

Interconnections Figure - TBD

The following acronyms are used in the figure.

SCB0-10

Indicate SCB interfaces, connecting the system bus masters and slaves

SCLK0, SYSCLK, CLK03, CLK04, CLK06

Indicate clock domains in which the specific SCBs operate. For more information on clock domains, see the Clock Generation Unit (CGU) chapter and the product data sheet.

CDC

Indicates the clock domain crossing

CL2_0, CL2_1, CL2_2

Indicates a 128-bit L2 slave

DL2_0, DL2_1

Indicates a 128-bit L2 slave

L1C1_S1

Indicates the S1 slave of the SHARC1 processor

L1C1_S2

Indicates the S2 slave of the SHARC1 processor

SHARC1_IPORT

Indicates the SHARC1 processor instruction master port

SHARC1_MMR

Indicates the SHARC1 MMR interface

A5_L2CC_M0/1

Indicates the Cortex A5 masters (0 and 1)

ARM Slave

Indicates the Cortex A5 slave

SHARC Fabric with FIR/IIR Accelerator

The accelerator block on each SHARC core of ADSP-SC59x processor has one FIR and four IIR accelerators. All instances of the FIR and IIR accelerators operate at the core clock (CCLK) frequency. Accelerator master ports can directly access the respective SHARC L1 memory with reduced latency. The access does not go through the system fabric. The SHARC core can also directly access the MMR registers of its respective accelerator. The accesses between one SHARC core and the accelerators that belong to the other SHARC core still go through the main system fabric.

The *SHARC Fabric Connectivity* figure shows a block diagram of the SHARC fabric with FIR/IIR accelerators integrated closely with the SHARC core.

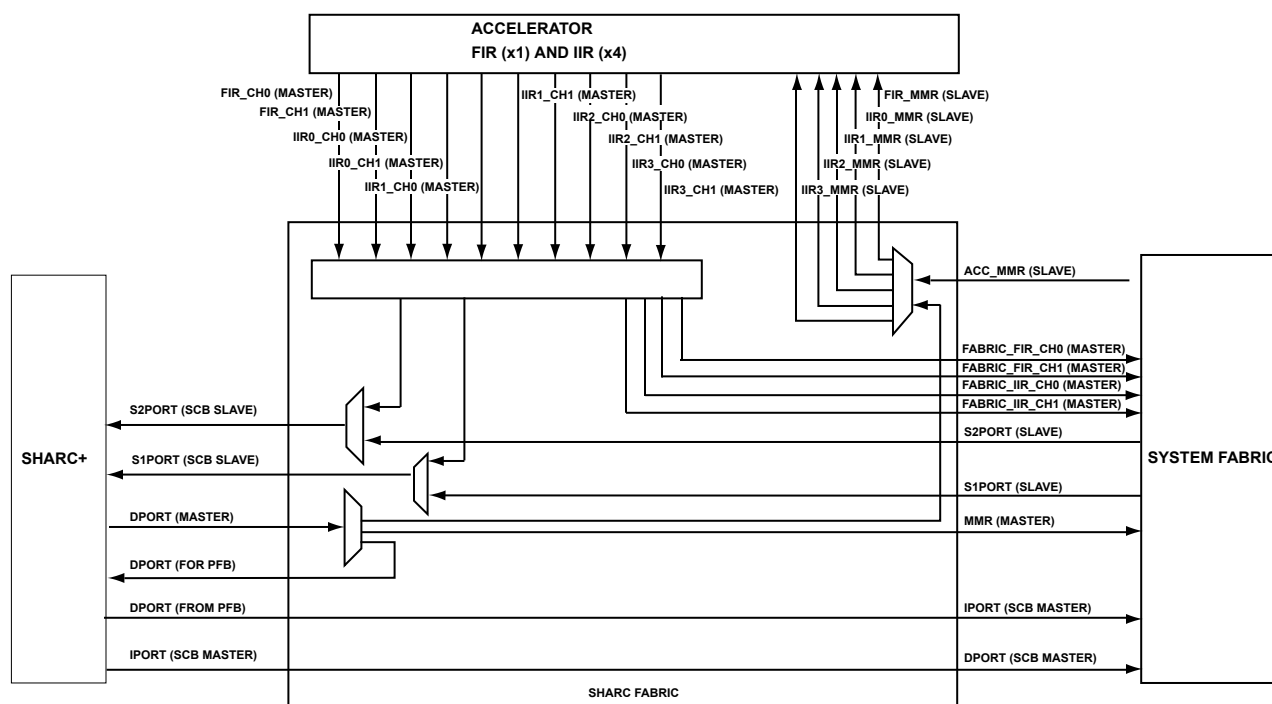


Figure 42-5: SHARC Fabric Connectivity

Note the following:

- The SHARC+ core can directly access the MMR space using its accelerators.
- The MMR port of the SHARC+ core is separate from the data port (DPORT). It is a separate master port for the main system fabric.
- PFB (prefetch buffer) — after the MMR split, the DPORT comes into the SHARC+ core for the PFB connection and returns to the SHARC+ fabric again for the clock domain crossing.
- Accelerators share the same security level as the respective core. The SHARC+ core associated with the accelerator can always access the MMR space of the accelerator. Accesses from the other masters such as other SHARC+ core/Arm Core/MSMDMA/DBG are allowed provided that the security requirements of the master are met.

To ensure complete bandwidth utilization and optimal performance of the L2 ports, the masters are allocated as shown in *L2 Port Master Allocation* table.

Table 42-6: L2Port Master Allocation

Masters	CL2_0	CL2_1	CL2_2	DL2_0	DL2_1
SHARC0 (Core 1)	✓	✓			
SHARC1 (Core 2)	✓	✓			
Cortex A5 (Core 0)	✓	✓			
SHARC0 IPORT			✓		
SHARC1 IPORT			✓		
SHARC0_FIR_CH0			✓		
SHARC0_FIR_CH1			✓		
SHARC0_IIR_CH0	✓	✓			
SHARC0_IIR_CH1	✓	✓			
SHARC1_FIR_CH0			✓		
SHARC1_FIR_CH1			✓		
SHARC1_IIR_CH1	✓	✓			
SHARC1_IIR_CH1	✓	✓			
HSMDMA0				✓	
HSMDMA0				✓	
Peripherals					✓

Note the following:

- To reduce the probability of port conflict in a multicore system, the cores are connected on the CL2_0 and CL2_1 ports of L2. CL2_0 has access only to the first 1MB of L2. CL2_1 has access to second 1MB of L2
- To avoid conflict during multi-core boot, the booting of the Arm core happens through the CL2_0 port while the booting of the SHARC cores happens through the CL2_1 port. The respective bootROM addresses are routed accordingly.

The addressable range for each of the ports is shown in the *L2 Port Address Allocation* table.

Table 42-7: L2 Port Address Allocation

	Memory Address Range			
SMPU Instance	Data		Instruction	
	Start Address	End Address	Start Address	End Address
CL2_0	0x2000_0000	0x200F_FFFF	0x2020_0000	0x2020_FFFF

Table 42-7: L2 Port Address Allocation (Continued)

SMPU Instance	Memory Address Range			
	Data		Instruction	
	Start Address	End Address	Start Address	End Address
DL2_0	0x2000_0000	0x2022_FFFF		
CL2_1	0x2010_0000	0x201F_FFFF	0x2021_0000	0x2022_FFFF
CL2_2	0x2000_0000	0x2022_FFFF		
DL2_1	0x2000_0000	0x2022_FFFF		

To support exclusive accesses to the 5 port L2 memory on the processor, the configuration of SMPUs as shown in the *SMPU Access Scheme for L2 Ports* figure is used.

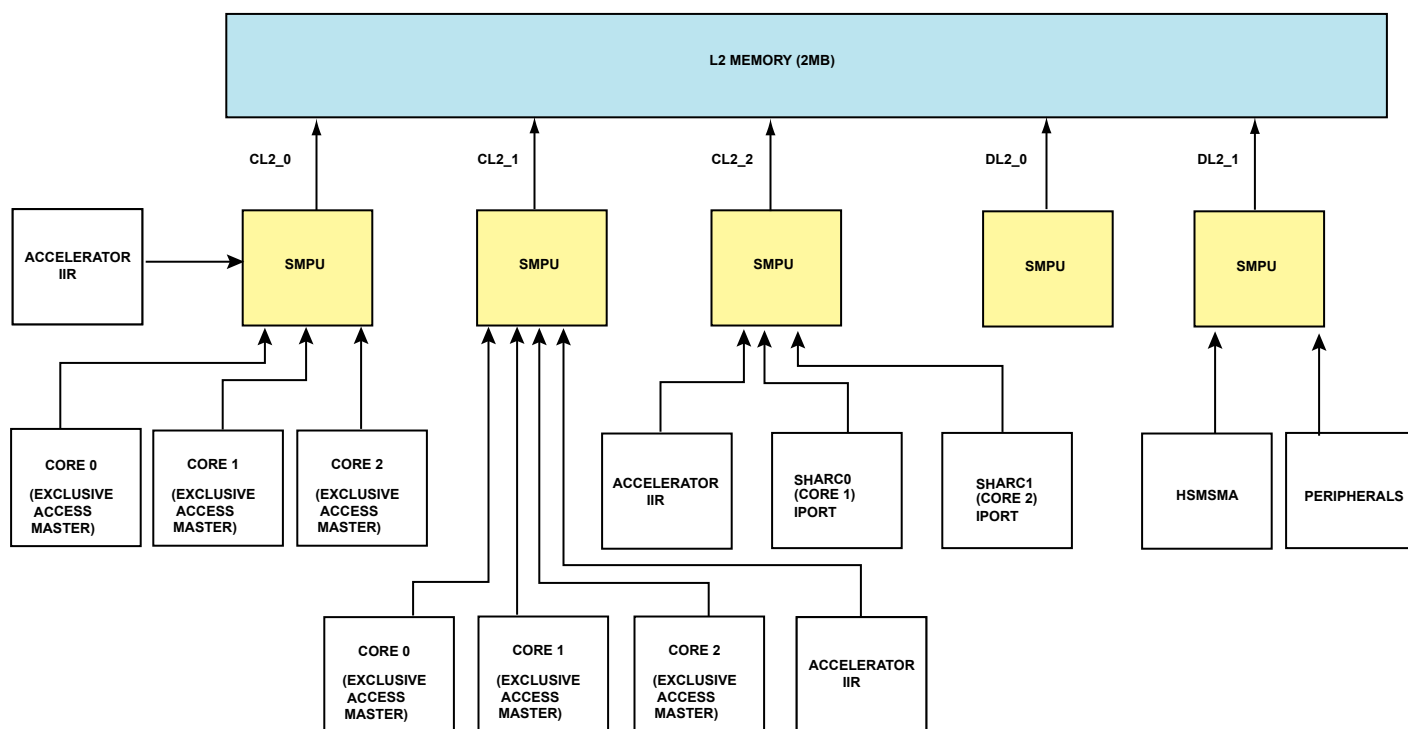


Figure 42-6: SMPU Access Scheme for L2 Ports

Table 42-8: SCB Controlled DMA Channel Peripherals

	Masters	DDE DMA Channels	Non DDE DMA Channels
SCB1	SPORT0, HALF A	DMA0	
	SPORT0, HALF B	DMA1	
	SPORT1, HALF A	DMA2	
	SPORT1, HALF B	DMA3	

Table 42-8: SCB Controlled DMA Channel Peripherals (Continued)

	Masters	DDE DMA Channels	Non DDE DMA Channels
	SPORT2, HALF A	DMA4	
	SPORT2, HALF B	DMA5	
	SPORT3, HALF A	DMA6	
	SPORT3, HALF B	DMA7	
SCB2	SPORT4, HALF A	DMA10	
	SPORT4, HALF B	DMA11	
	SPORT5, HALF A	DMA12	
	SPORT5, HALF B	DMA13	
	SPORT6, HALF A	DMA14	
	SPORT6, HALF B	DMA15	
	SPORT7, HALF A	DMA16	
	SPORT7, HALF B	DMA17	
SCB3	UART0, TX	DMA20	
	UART0, RX	DMA21	
	UART1, TX	DMA34	
	UART1, RX	DMA35	
	UART2, TX	DMA37	
	UART2, RX	DMA38	
	UART3, TX	DMA53	
	UART3, RX	DMA54	
SCB4	SPI0, TX	DMA22	
	SPI0, RX	DMA23	
	SPI1, TX	DMA24	
	SPI1, RX	DMA25	
	SPI2, TX	DMA26	
	SPI2, RX	DMA27	
	SPI3, TX	DMA55	
	SPI3, RX	DMA56	
SCB5	10/100/1000 EMAC (GIGE)	N/A	6 channels (3 receive and 3 transmit)
	10/100 EMAC	N/A	2 channels (1 receive and 1 transmit)

Table 42-8: SCB Controlled DMA Channel Peripherals (Continued)

	Masters	DDE DMA Channels	Non DDE DMA Channels
SCB6	LP0	DMA30	
	LP1	DMA36	
SCB7	CRC0/MDMA0	DMA8, DMA9	
	CRC1/MDMA1	DMA18, DMA19	
SCB8	CRC2/MDMA4	DMA45, DMA46	
	CRC3/MDMA5	DMA47, DMA48	
SCB9	MDMA2	DMA39, DMA40	
	MDMA6	DMA49, DMA50	
SCB10	ETR	N/A	N/A
	CRYPTO	N/A	2 channels (1 read, 1 write)
	EMDMA0	N/A	2 channels (1 read, 1 write)
	EMDMA1	N/A	2 channels (1 read, 1 write)
HSMDMA_SCB	MDMA3	DMA43, DMA44	
	MDMA7	DMA51, DMA52	
CORES_SCB	SHARC0 DPORT	N/A	N/A
	SHARC1 DPORT	N/A	N/A
	ARM_L2CC_M0	N/A	N/A
	ARM_L2CC_M1	N/A	N/A
	SHARC0 IIR	CH0, CH1	
	SHARC1 IIR	CH0, CH1	
FIR_IPORT_SCB	SHARC0 IIR	CH0, CH1	N/A
	SHARC1 IIR	CH0, CH1	N/A
	SHARC0 IPORT	N/A	N/A
	SHARC1 IPORT	N/A	N/A
MMR_SCB	SHARC0 MMR	N/A	N/A
	SHARC1 MMR	N/A	N/A

There are two types of peripherals that use DMA. The first have dedicated DMA channels controlled by the Dedicated DMA Engine (DDE) and have the same operating modes (see [DMA Operating Modes](#)) and use the same programming model ([DMA Channel Programming Model](#)). The second type is not controlled by the DDE module. These peripherals have their own operating modes and programming models (see the peripheral chapter for this information). The peripheral types are shown in the *SCB-Controlled DMA Channel Peripherals* table.

System Crossbars

The System Crossbars (SCB) are the fundamental building blocks of the system bus interconnect. The SCB (often referred to as the system interconnect fabric), is a collection of inter-connection units connecting system masters to slave memory spaces. The SCB connects one or more master devices to one or more memory-mapped slave devices. Each connected master can be a core that originates an SCB transaction, or a master interface of an upstream SCB cascaded interconnect. Each connected slave can be the final target of an SCB transaction or a slave interface of a downstream cascaded SCB interconnect (forming a hierarchy of SCBs).

Each SCB that has multiple masters and slaves share the total bandwidth of the SCB. (In a M:N configuration where M masters are connected to N slaves through the SCBx.)

The SCB provides separate channels for reads and writes. Read and write accesses through a given SCB do not share bandwidth. All the SCBs are 32 bits wide and run at SYSCLK speed, and can provide a bandwidth of up to 400 Mbytes per second for reads and writes separately (when SYSCLK = 100 MHz). Only SCB0, which is the major SCB in the SCB hierarchy, has the multiple paths between multiple master and slave interfaces.

See the [SCB Block Diagram](#).

All other SCBs in the chip connect to SCB0 through different slave interfaces. Other primary masters (DMAs, cores, and so on) in the system are distributed across these small SCBs. For a given SCB, all the master and slaves share the total bandwidth of the SCB. (Only SCB0 is the exception). Since different DMA channels are scattered across different SCBs (SCB1, SCB2 SCB3, and so on), they do not conflict for the bandwidth as long as they are in different SCB and are accessing different slaves. SCB0 allows for concurrent data transfer between multiple bus masters and multiple bus slaves, providing flexibility, and full-duplex operation.

If system accesses are carefully architected, SCB has a potential of providing sufficient sustained bandwidth in the end system.

Since the SCBs support burst transfers, it is important to configure the requesting master appropriately to make best use of available SCB bandwidth. For a DMA master, choosing the appropriate `DMA_CFG.MSIZE` value, is important from both a functional and a performance perspective. The value in the `DMA_CFG.PSIZE` bit field determines the width of the peripheral bus in use. It can be configured to 1-byte, 2-bytes, or 4-bytes. The `DMA_CFG.MSIZE` value determines the actual size of the SCB bus in use. It also determines the minimum number of bytes which are transferred from or to memory corresponding to a single DMA request or grant. The transfer can be 1-, 2-, 4-, 8-, 16-, or 32-bytes. If the `DMA_CFG.MSIZE` value is greater than the SCB bus width, the SCB performs burst transfers according to the width defined in `DMA_CFG.MSIZE`. When `DMA_CFG.MSIZE` is less than the SCB bus width, bursting is not supported and partial bus use results.

Each of the SCB unit in the fabric consists of N Slave interfaces (MSTn). Each of these interfaces has controls for read quality of service, write quality of service, and functional mode. A subset of these matrices includes controls for IB (Interface Block) sync mode, and bus functional mode. For more details on IB, see the clock domain synchronization section.

SCB Bus Master IDs

The SCB *Bus Master IDs* table indicates which masters are connected to each of the slave ports of SCB0. The tables also indicate the precise value of the ID as seen by the slave. These values are useful for SWU programming.

NOTE: For an overall diagram of all SCB interconnections, see the [SCB Block Diagram](#).

Table 42-9: Bus Master IDs

Master	Hex ID Values	Binary Values
DMA0 (SPORT0, HALF A)	0x0000, 0x0100	13'b0000x00000000
DMA1 (SPORT0, HALF B)	0x0010, 0x0110	13'b0000x00010000
DMA2 (SPORT1, HALF A)	0x0020, 0x0120	13'b0000x00100000
DMA3 (SPORT1, HALF B)	0x0030, 0x0130	13'b0000x00110000
DMA4 (SPORT2, HALF A)	0x0040, 0x0140	13'b0000x01000000
DMA5 (SPORT2, HALF B)	0x0050, 0x0150	13'b0000x01010000
DMA6 (SPORT3, HALF A)	0x0060, 0x0160	13'b0000x01100000
DMA7 (SPORT3, HALF B)	0x0070, 0x0170	13'b0000x01110000
DMA8 (Enh BW MDMA0 CRC, CH0)	0x0031, 0x0131	13'b0000x00110001
DMA9 (Enh BW MDMA0 CRC, CH1)	0x0021, 0x0121	13'b0000x00100001
MLB	0x0042	13'b0000001000010
DMA20 (UART0, TX)	0x0003, 0x0103	13'b0000x00000011
DMA21 (UART0, RX)	0x0043, 0x0143	13'b0000x01000011
DMA37 (UART2, TX)	0x0033, 0x0133	13'b0000x00110011
DMA22 (SPI0, TX)	0x0004, 0x0104	13'b0000x00000100
DMA23 (SPI0, RX)	0x0014, 0x0114	13'b0000x00010100
DMA24 (SPI1, TX)	0x0024, 0x0124	13'b0000x00100100
DMA25 (SPI1, RX)	0x0034, 0x0134	13'b0000x00110100
DMA26 (SPI2, TX)	0x0054, 0x0154	13'b0000x01010100
DMA27 (SPI2, RX)	0x0044, 0x0144	13'b0000x01000100
DMA30 (LP0)	0x0005, 0x005	13'b0000x00000101
DMA34 (UART1, TX)	0x0013, 0x0113	13'b0000x00010011
DMA35 (UART1, RX)	0x0023, 0x0123	13'b0000x00100011
DMA36 (LP1)	0x0015, 0x0115	13'b0000x00010101
CRYPTO	0x0056, 0x0156	13'b0000001010110
SH0_FIR_CH0	0x0X07 (X=don't care)	13'b0xxxx00000111
SH0_FIR_CH1	0x0X17 (X=don't care)	13'b0xxxx00010111

Table 42-9: Bus Master IDs (Continued)

Master	Hex ID Values	Binary Values
EMDMA0 (CH0)	0x0006	13'b0000000000110
EMDMA0 (CH1)	0x0016	13'b0000000010110
EMDMA1 (CH0)	0x0026	13'b0000000100110
EMDMA1 (CH1)	0x0036	13'b0000000110110
DMA39 (Enh BW MDMA2, CH0)	0x0008, 0x0108	13'b0000x00001000
DMA40 (Enh BW MDMA2, CH1)	0x0018, 0x0118	13'b0000x00011000
DBG	0x0048	13'b0000001001000
ETR	0x0046	13'b0000001000110
DMA18 (Enh BW MDMA1 CRC1, CH0)	0x0011, 0x0111	13'b0000x00010001
DMA19 (Enh BW MDMA1 CRC1, CH1)	0x0001, 0x0101	13'b0000x00000001
DMA38 (UART2, RX)	0x0053, 0x0153	13'b0000x01010011
SH0 (DPORT)	0x0X09(X=don't care)	13'b0xxxx00001001
SH0 (IPORT)	0x0087	13'b0000001000111
DMA43 (High Speed MDMA3, CH0)	0x000A, 0x010A	13'b0000x00001010
DMA44 (High Speed BW MDMA3, CH1)	0x001A, 0x011A	13'b0000x00011010
DMA10 (SPORT4, HALFA)	0x000B, 0x010B	13'b0000x00001011
DMA11 (SPORT4, HALFB)	0x001B, 0x011B	13'b0000x00011011
DMA12 (SPORT5, HALFA)	0x002B, 0x012B	13'b0000x00101011
DMA13 (SPORT5, HALFB)	0x003B, 0x013B	13'b0000x00111011
DMA14 (SPORT6, HALFA)	0x004B, 0x014B	13'b0000x01001011
DMA15 (SPORT6, HALFB)	0x005B, 0x015B	13'b0000x01011011
DMA16 (SPORT7, HALFA)	0x006B, 0x016B	13'b0000x01101011
DMA17 (SPORT7, HALFB)	0x007B, 0x017B	13'b0000x01111011
SH0_MMR	0x0X0C (X=don't care)	13'b0xxxx00001100
DMA53 (UART3, TX)	0x0063, 0x0163	13'b0000x01100011
DMA54 (UART3, RX)	0x0073, 0x0173	13'b0000x01110011
DMA55 (SPI3, TX)	0x0064, 0x0164	13'b0000x01100100
DMA56 (SPI3, RX)	0x0074, 0x0174	13'b0000x01110100
DMA45 (Enh BW MDMA4 CRC, CH0)	0x001D, 0x011D	13'b0000x00011101
DMA46 (Enh BW MDMA4 CRC, CH1)	0x000D, 0x010D	13'b0000x00001101
DMA47 (Enh BW MDMA5 CRC, CH0)	0x003D, 0x013D	13'b0000x00111101

Table 42-9: Bus Master IDs (Continued)

Master	Hex ID Values	Binary Values
DMA48 (Enh BW MDMA5 CRC, CH1)	0x002D, 0x012D	13'b0000x00101101
DMA49 (Enh BW MDMA6 CRC1, CH0)	0x0038, 0x0138	13'b0000x00111000
DMA50 (Enh BW MDMA6 CRC1, CH1)	0x0028, 0x0128	13'b0000x00101000
DMA51 (High Speed MDMA3, CH0)	0x002A, 0x012A	13'b0000x00101010
DMA52 (High Speed BW MDMA3, CH1)	0x003A, 0x013A	13'b0000x00111010
DMA28 (PPI F0)	0x0083, 0x0183	13'b0000x10000011
DMA29 (PPI F1)	0x0093, 0x0193	13'b0000x10010011
USB0	0x002E	13'b0000000101110
SH1 (IPORT)	0x0057	13'b0000001010111
SH1 (DPORT)	0x0X19(X=don't care)	13'b0xxxx00011001
SH1_MMR	0x0X2C(X=don't care)	13'b0xxxx00101100
ARM_L2CC_M0	0x0X29(X=don't care)	13'bxxxxx00101001
ARM_L2CC_M1	0x0X39(X=don't care)	13'bxxxxx00111001
GIGE	0x0X0F(X=don't care)	13'b0xxxx00001111
EMAC	0x0X1F(X=don't care)	13'b0xxxx00011111
SH1_FIR_CH0	0x0X27(X=don't care)	13'b0xxxx00100111
SH1_FIR_CH1	0x0X37(X=don't care)	13'b0xxxx00110111
SH0_IIR_CH0	0x0X49(X=don't care)	13'b0xxxx01001001
SH1_FIR_CH1	0x0X59(X=don't care)	13'b0xxxx01011001
SH1_IIR_CH0	0x0X69(X=don't care)	13'b0xxxx01101001
SH1_IIR_CH1	0x0X79(X=don't care)	13'b0xxxx01111001

SCB Programming Model

The following sections provide information for programming the SCB properly.

Programming SCB Arbitration

Each slave interface has a QoS value (priority) associated with both read and write channels. These values are 4 bits present in the SCB0_MSTx_RQOS and SCB0_MSTx_WQOS registers. At the entry point to the infrastructure, all transactions are allocated this programmable local QoS value. The arbitration of the transaction throughout the infrastructure uses this QoS. At any arbitration node, a fixed priority exists for transactions with a different QoS. The highest value has the highest priority.

If there are coincident transactions at an arbitration node with the same QoS that require arbitration, then the network uses a Least Recently Granted (LRG) algorithm. At each switch, the master with the highest QoS acquires

access and that switch output takes the QoS value of the winner for that transaction. At the next switch slave interface, the master uses the QoS value of the winner. QoS can have values from 0 (lowest priority) to 15 (highest priority).

For example in the following figure, *SCB Arbitration*:

1. At SCB1, masters (1, 2, 3) have RQOS values of (6, 4, 2)
2. At SCB2, masters (4, 5, 6) have RQOS values of (12, 13, 1)

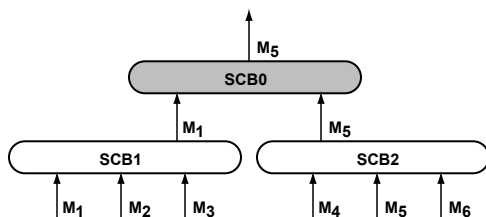


Figure 42-7: SCB Arbitration

In this case, master 1 wins at SCB1, and master 5 wins at SCB2. However, in a perfect competition at SCB0, masters 4 and 5 had the highest overall RQOS values. Masters 4 and 5 would have fought for arbitration directly at SCB0. However, because of the mini*SCBs, master 1, at a much lower RQOS value, is able to win against master 4 and make it all the way to SCB0.

Programming Clock Domain Crossing Registers

The *Clock Domian Crossing Options* table shows the various clock domain crossings that are available on the ADSP-SC59x processor. The clocking relationships are defined with respect to SYSCLK. The following clock domains are programmable: CLK03: SYSCLK, CLK04: SYSCLK and CLK08: SYSCLK CDC. The registers to configure the CDC mode are present within the GPV space of the respective fabric. For example: To program the CLK03: SYSCLK relationship, program the GPV registers within the DMC_CDC fabric and the MMRG fabric.

Table 42-10: Clock Domian Crossing Options

Functional Clocks	Possible Targets	Clocking Relationships Defined by					
		System Fabric	MMRG Fabric	DMC Fabric	SHARC Fabric)	Arm Fabric	SPIF Fabric
SYSCLK	System Fabric and Infrastructure Modules	1:1	1:1	*1	*	*	*
SCLK0	SCLK0 Clock Doamin	Synchronous (m:1)	Synchronous (m:1)	*	*	*	*
CLK00	SHARC0 and its Accelerators	*	*	*	Synchronous (m:1)	*	*
CLK01	SHARC1 and its Accelerators	*	*	*	Synchronous (m:1)	*	*

Table 42-10: Clock Domian Crossing Options (Continued)

Functional Clocks	Possible Tar-gets	Clocking Relationships Defined by					
		System Fabric	MMRG Fabric	DMC Fabric	SHARC Fab-ric)	Arm Fabric	SPIF Fabric
CLKO2	ARM	*	*	*	*	Synchronous (m:1)	*
CLKO3	DMC	*	Programmable	Programmable	*	*	*
CLKO4	CAN	*	Programmable	*	*	*	*
CLKO6	SPI	Synchronous (m:n) (except SPIF slave)	Synchronous (m:n)	*	*	*	Synchronous (m:n)
CLKO8	LP	Programmable	Programmable	*	*	*	*

*1 * Either the interface is not present or CDC is not required

Table 42-11: Sync Mode Bit Field Description

Sync Mode	Description
0	sync 1:1
1	sync n:1
2	sync 1:n
3	sync m:n
4	async

To change the clock domain crossing mode, follow the actions described in the *Changing Clock Domain Crossing Modes* table.

Table 42-12: Changing Clock Domain Crossing Modes

Original Mode	Required Mode	Action
ASYN	Any other mode	Change the clocks, then change the MMR register
Any mode	ASYN	Change the MMR register, then change clocks to ASYN.
m:n	1:1	Change the clocks, then change the register.
1:1	m:n	Change the register, then change the clocks.

SCB Programming Concepts

The SCB arbitration model among master or slave SCBs of the processor is fixed (not programmable). But, each slave does have a quality of service (QoS) programmable feature that affects arbitration.

The arbitration of transactions in SCB is based on the QoS value or the priority of the transaction. All masters with the same priority form a priority group. Arbitration is granted to the highest priority group from which a member is trying to win access, and within that group, to the highest master at that time. When a master wins arbitration, it is relegated to the bottom of its group to ensure that it cannot prevent other masters in its group from accessing the slave.

If you configure all master priorities to different levels, the arbiter implements a fixed priority scheme. This scheme occurs because each master is in a group of its own, and therefore, masters maintain their ordering.

The LRG and fixed priority modes concurrently exist when the master priority value registers are programmed with a combination of identical and unique values.

NOTE: The SCB arbitration hierarchy is fixed (for example, SCB1 master to SCB1 slave). However, multiple master inputs to the same slave permit QoS programming.

The *LRG Arbitration Example* figure shows three groups with different QoS values. Masters in the same group share a QoS value. The arbitration occurs using an LRG scheme.

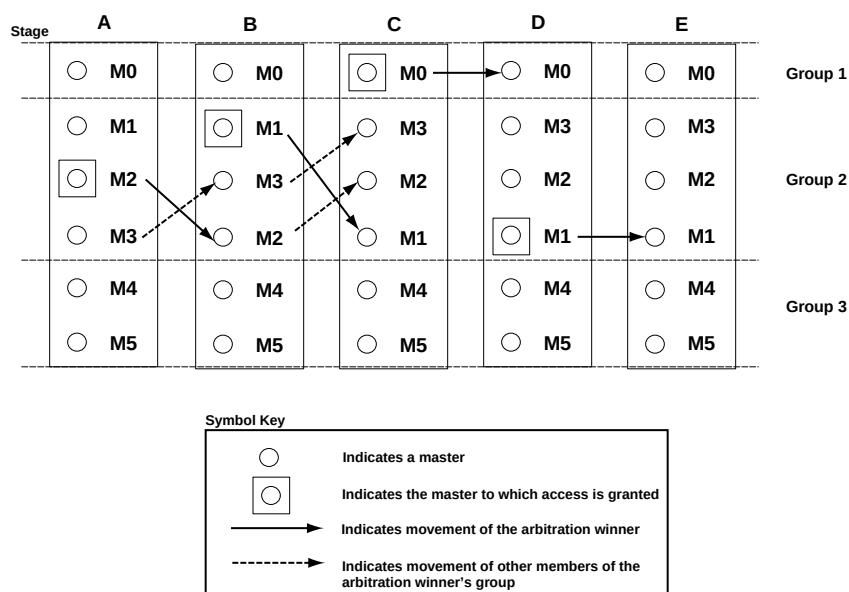


Figure 42-8: LRG Arbitration Example

The QoS value assigned to a transaction at entry point is carried forward by the transaction as it passes through all arbitration stages in the SCB. QoS for all masters is configured as programmable in the system fabric interconnect.

The priority of the masters fits into three groups:

- Group A - Peripherals with an external interface, without flow control and with real-time processing requirements

- Group B - Cores, peripherals with flow control, and offload engines
- Group C - MDMAs

Group A: Peripherals with external interface, without flow control and with real-time processing requirements

These masters are assigned the highest priority. They are latency-critical – a large increase in latency could potentially result in data corruption and catastrophic failure. This high priority SCB group includes masters connected to SCB1, SCB2, SCB3, and SCB4. Masters in this group are assigned the QoS reset value of 12.

Group B: Cores, peripherals with flow control, and offload engines

Core is latency-sensitive. Once a core requests data, it typically waits for the data without performing anything else in parallel. An increase in the latency has a direct impact on the performance of the core. This group is assigned medium priority and a QoS reset value of 7. Group B masters are connected to SCB5, SCB6, CORES_SCB, FIR_IPORT_SCB, and MMR_SCB.

Group C: MDMAs

These masters are assigned the lowest priority. MDMAs are data intensive and do not have any external interface. Masters in this group are assigned the QoS reset value of 1. The Group C masters are connected to SCB7, SCB8, SCB9, and SCB10.

Table 42-13: QoS Register Table

Master ID	Master	read_qos Reset Value	write_qos Reset Value
1	SPORT0_A_DMA	12	12
2	SPORT0_B_DMA	12	12
3	SPORT1_A_DMA	12	12
4	SPORT1_B_DMA	12	12
5	SPORT2_A_DMA	12	12
6	SPORT2_B_DMA	12	12
7	SPORT3_A_DMA	12	12
8	SPORT3_B_DMA	12	12
9	SPORT4_A_DMA	12	12
10	SPORT4_B_DMA	12	12
11	SPORT5_A_DMA	12	12
12	SPORT5_B_DMA	12	12
13	SPORT6_A_DMA	12	12
14	SPORT6_B_DMA	12	12

Table 42-13: QoS Register Table (Continued)

Master ID	Master	read_qos Reset Value	write_qos Reset Value
15	SPORT7_A_DMA	12	12
16	SPORT7_B_DMA	12	12
	UART0_TX	12	12
	UART0_RX	12	12
	UART1_TX	12	12
	UART1_RX	12	12
	UART2_TX	12	12
	UART2_RX	12	12
	UART3_TX	12	12
	UART3_RX	12	12
	PPI_F0	12	12
	PPI_F1	12	12
	SPI0RX	12	12
	SPI0TX	12	12
19	SPI1TX	12	12
20	SPI1RX	12	12
21	SPI2TX	12	12
22	SPI2RX	12	12
23	MLB	12	12
	GIGE	7	7
	EMAC	7	7
	LP0	7	7
	LP1	7	7
	USB	7	7
	SH0_DPORT	7	7
	SH0_IPORT	7	7
	SH1_IPORT	7	7
	ARM_L2CC_M0	7	7
	ARM_L2CC_M1	7	7
	SH0_IIR_CH0	7	7
	SH0_IIR_CH1	7	7

Table 42-13: QoS Register Table (Continued)

Master ID	Master	read_qos Reset Value	write_qos Reset Value
	SH1_IIR_CH0	7	7
	SH1_IIR_CH1	7	7
	SH0_FIR_CH0	7	7
	SH0_FIR_CH1	7	7
	SH0_MMR	7	7
	SH1_MMR	7	7
	CRC0_CH0	1	1
	CRC0_CH1	1	1
	CRC1_CH0	1	1
	CRC1_CH1	1	1
42	CRC2_CH0	1	1
43	CRC2_CH1	1	1
	CRC3_CH0	1	1
	CRC3_CH1	1	1
	MDMA2_CH0	1	1
	MDMA2_CH1	1	1
	MDMA3_CH0	1	1
	MDMA3_CH1	1	1
	MDMA6_CH0	1	1
	MDMA6_CH1	1	1
	MDMA7_CH0	1	1
	MDMA7_CH1	1	1
	EMDMA0_CH0	1	1
	EMDMA0_CH1	1	1
	EMDMA1_CH0	1	1
	EMDMA1_CH1	1	1
	CRYPTO	1	1
	ETR	1	1
	DBG	1	1

QoS Programming

Adhere to the following guidelines if software is modifying the reset value of QoS:

- The highest QoS of any master in a low priority SCB group must be less than the lowest QoS of any master in a medium priority SCB group
- The highest QoS of any master in medium priority SCB group must be less than the lowest QoS of any master in high priority SCB group

Reset values of QoS have been spaced out to allow software to lower or higher the priority of a few masters without having to modify the priority all the masters. For example, the QoS of the LP can be reduced from 12 to 11 without violating the guidelines.

ADSP-2159x SCB0 Register Descriptions

System Crossbar (SCB0) contains the following registers.

Table 42-14: ADSP-2159x SCB0 Register List

Name	Description
SCB0_CRC0_CH0_READ_QOS	CRC0 Channel 0 Read Quality of Service Register
SCB0_CRC0_CH0_WRITE_QOS	CRC0 Channel 0 Write Quality of Service Register
SCB0_CRC0_CH1_READ_QOS	CRC0 Channel 1 Read Quality of Service Register
SCB0_CRC0_CH1_WRITE_QOS	CRC0 Channel 1 Write Quality of Service Register
SCB0_CRC1_CH0_READ_QOS	CRC1 Channel 0 Read Quality of Service Register
SCB0_CRC1_CH0_WRITE_QOS	CRC1 Channel 0 Write Quality of Service Register
SCB0_CRC1_CH1_READ_QOS	CRC1 Channel 1 Read Quality of Service Register
SCB0_CRC1_CH1_WRITE_QOS	CRC1 Channel 1 Write Quality of Service Register
SCB0_CRC2_CH0_READ_QOS	CRC2 Channel 0 Read Quality of Service Register
SCB0_CRC2_CH0_WRITE_QOS	CRC2 Channel 0 Write Quality of Service Register
SCB0_CRC2_CH1_READ_QOS	CRC2 Channel 1 Read Quality of Service Register
SCB0_CRC2_CH1_WRITE_QOS	CRC2 Channel 1 Write Quality of Service Register
SCB0_CRC3_CH0_READ_QOS	CRC3 Channel 0 Read Quality of Service Register
SCB0_CRC3_CH0_WRITE_QOS	CRC3 Channel 0 Write Quality of Service Register
SCB0_CRC3_CH1_READ_QOS	CRC3 Channel 1 Read Quality of Service Register
SCB0_CRC3_CH1_WRITE_QOS	CRC3 Channel 1 Write Quality of Service Register
SCB0_CRYPT0_READ_QOS	CRYPTO Read Quality of Service Register
SCB0_CRYPT0_WRITE_QOS	CRYPTO Write Quality of Service Register
SCB0_DBG_READ_QOS	DBG Read Quality of Service Register
SCB0_DBG_WRITE_QOS	DBG Write Quality of Service Register
SCB0_DLDMA0_CH0_READ_QOS	DLDMA0 Channel 0 Read Quality of Service Register

Table 42-14: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_DLDMA0_CH0_WRITE_QOS	DLDMA0 Channel 0 Write Quality of Service Register
SCB0_DLDMA0_CH1_READ_QOS	DLDMA0 Channel 1 Read Quality of Service Register
SCB0_DLDMA0_CH1_WRITE_QOS	DLDMA0 Channel 1 Write Quality of Service Register
SCB0_DLDMA1_CH0_READ_QOS	DLDMA1 Channel 0 Read Quality of Service Register
SCB0_DLDMA1_CH0_WRITE_QOS	DLDMA1 Channel 0 Write Quality of Service Register
SCB0_DLDMA1_CH1_READ_QOS	DLDMA1 Channel 1 Read Quality of Service Register
SCB0_DLDMA1_CH1_WRITE_QOS	DLDMA1 Channel 1 Write Quality of Service Register
SCB0_EMAC_READ_QOS	EMAC Read Quality of Service Register
SCB0_EMAC_WRITE_QOS	EMAC Write Quality of Service Register
SCB0_ETR_READ_QOS	ETR Read Quality of Service Register
SCB0_ETR_WRITE_QOS	ETR Write Quality of Service Register
SCB0_GIGE_READ_QOS	GIGE Read Quality of Service Register
SCB0_GIGE_WRITE_QOS	GIGE Write Quality of Service Register
SCB0_HSMDMA1_CH0_READ_QOS	HSMDMA1 Channel 0 Read Quality of Service Register
SCB0_HSMDMA1_CH0_WRITE_QOS	HSMDMA1 Channel 0 Write Quality of Service Register
SCB0_HSMDMA1_CH1_READ_QOS	HSMDMA1 Channel 1 Read Quality of Service Register
SCB0_HSMDMA1_CH1_WRITE_QOS	HSMDMA1 Channel 1 Write Quality of Service Register
SCB0_HSMDMA_CH0_READ_QOS	HSMDMA Channel 0 Read Quality of Service Register
SCB0_HSMDMA_CH0_WRITE_QOS	HSMDMA Channel 0 Write Quality of Service Register
SCB0_HSMDMA_CH1_READ_QOS	HSMDMA Channel 1 Read Quality of Service Register
SCB0_HSMDMA_CH1_WRITE_QOS	HSMDMA Channel 1 Write Quality of Service Register
SCB0_LP0_READ_QOS	LP0 Read Quality of Service Register
SCB0_LP0_WRITE_QOS	LP0 Write Quality of Service Register
SCB0_LP1_READ_QOS	LP1 Read Quality of Service Register
SCB0_LP1_WRITE_QOS	LP1 Write Quality of Service Register
SCB0_MLB_READ_QOS	MLB Read Quality of Service Register
SCB0_MLB_WRITE_QOS	MLB Write Quality of Service Register
SCB0_MSMDMA1_CH0_READ_QOS	MSMDMA1 Channel 0 Read Quality of Service Register
SCB0_MSMDMA1_CH0_WRITE_QOS	MSMDMA1 Channel 0 Write Quality of Service Register
SCB0_MSMDMA1_CH1_READ_QOS	MSMDMA1 Channel 1 Read Quality of Service Register
SCB0_MSMDMA1_CH1_WRITE_QOS	MSMDMA1 Channel 1 Write Quality of Service Register

Table 42-14: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_MSMDMA_CH0_READ_QOS	MSMDMA Channel 0 Read Quality of Service Register
SCB0_MSMDMA_CH0_WRITE_QOS	MSMDMA Channel 0 Write Quality of Service Register
SCB0_MSMDMA_CH1_READ_QOS	MSMDMA Channel 1 Read Quality of Service Register
SCB0_MSMDMA_CH1_WRITE_QOS	MSMDMA Channel 1 Write Quality of Service Registers
SCB0_PL310_M0_READ_QOS	ARM_L2CC M0 Read Quality of Service Register
SCB0_PL310_M0_WRITE_QOS	ARM_L2CC M0 Write Quality of Service Register
SCB0_PL310_M1_READ_QOS	ARM_L2CC M1 Read Quality of Service Register
SCB0_PL310_M1_WRITE_QOS	ARM_L2CC M1 Write Quality of Service Register
SCB0_PPI_F0_READ_QOS	PPI F0 Read Quality of Service Register
SCB0_PPI_F0_WRITE_QOS	PPI F0 Write Quality of Service Register
SCB0_PPI_F1_READ_QOS	PPI F1 Read Quality of Service Register
SCB0_PPI_F1_WRITE_QOS	PPI F1 Write Quality of Service Register
SCB0_LP_SYNC_MODE	LP Fabric (CLKO8) Synchronization Mode Register
SCB0_SH0_DPORT_READ_QOS	SH0 DPORT Read Quality of Service Register
SCB0_SH0_DPORT_WRITE_QOS	SH0 DPORT Write Quality of Service Register
SCB0_SH0_FIR_CH0_READ_QOS	SH0 FIR Channel 0 Read Quality of Service Register
SCB0_SH0_FIR_CH0_WRITE_QOS	SH0 FIR Channel 0 Write Quality of Service Register
SCB0_SH0_FIR_CH1_READ_QOS	SH0 FIR Channel 1 Read Quality of Service Register
SCB0_SH0_FIR_CH1_WRITE_QOS	SH0 FIR Channel 1 Write Quality of Service Register
SCB0_SH0_IIR_CH0_READ_QOS	SH0 IIR Channel 0 Read Quality of Service Register
SCB0_SH0_IIR_CH0_WRITE_QOS	SH0 IIR Channel 0 Write Quality of Service Register
SCB0_SH0_IIR_CH1_READ_QOS	SH0 IIR Channel 1 Read Quality of Service Register
SCB0_SH0_IIR_CH1_WRITE_QOS	SH0 IIR Channel 1 Write Quality of Service Register
SCB0_SH0_IPORT_READ_QOS	SH0 LPORT Read Quality of Service Register
SCB0_SH0_IPORT_WRITE_QOS	SH0 LPORT Write Quality of Service Register
SCB0_SH0_MMR_READ_QOS	SH0 MMR Read Quality of Service Register
SCB0_SH0_MMR_WRITE_QOS	SH0 MMR Write Quality of Service Register
SCB0_SH1_DPORT_READ_QOS	SH1 DPORT Read Quality of Service Register
SCB0_SH1_DPORT_WRITE_QOS	SH1 DPORT Write Quality of Service Register
SCB0_SH1_FIR_CH0_READ_QOS	SH1 FIR Channel 0 Read Quality of Service Register
SCB0_SH1_FIR_CH0_WRITE_QOS	SH1 FIR Channel 0 Write Quality of Service Register

Table 42-14: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_SH1_FIR_CH1_READ_QOS	SH1 FIR Channel 1 Read Quality of Service Register
SCB0_SH1_FIR_CH1_WRITE_QOS	SH1 FIR Channel 1 Write Quality of Service Register
SCB0_SH1_IIR_CH0_READ_QOS	SH1 IIR Channel 0 Read Quality of Service Register
SCB0_SH1_IIR_CH0_WRITE_QOS	SH1 IIR Channel 0 Write Quality of Service Register
SCB0_SH1_IIR_CH1_READ_QOS	SH1 IIR Channel 1 Read Quality of Service Register
SCB0_SH1_IIR_CH1_WRITE_QOS	SH1 IIR Channel 1 Write Quality of Service Register
SCB0_SH1_IPORT_READ_QOS	SH1 IPORT Read Quality of Service Register
SCB0_SH1_IPORT_WRITE_QOS	SH1 IPORT Write Quality of Service Register
SCB0_SH1_MMR_READ_QOS	SH1 MMR Read Quality of Service Register
SCB0_SH1_MMR_WRITE_QOS	SH1 MMR Write Quality of Service Register
SCB0_SP0A_READ_QOS	SP0A Read Quality of Service Register
SCB0_SP0A_WRITE_QOS	SP0A Write Quality of Service Register
SCB0_SP0B_READ_QOS	SP0B Read Quality of Service Register
SCB0_SP0B_WRITE_QOS	SP0B Write Quality of Service Register
SCB0_SP1A_READ_QOS	SP1A Read Quality of Service Register
SCB0_SP1A_WRITE_QOS	SP1A Write Quality of Service Register
SCB0_SP1B_READ_QOS	SP1B Read Quality of Service Register
SCB0_SP1B_WRITE_QOS	SP1B Write Quality of Service Register
SCB0_SP2A_READ_QOS	SP2A Read Quality of Service Register
SCB0_SP2A_WRITE_QOS	SP2A Write Quality of Service Register
SCB0_SP2B_READ_QOS	SP2B Read Quality of Service Register
SCB0_SP2B_WRITE_QOS	SP2B Write Quality of Service Register
SCB0_SP3A_READ_QOS	SP3A Read Quality of Service Register
SCB0_SP3A_WRITE_QOS	SP3A Write Quality of Service Register
SCB0_SP3B_READ_QOS	SP3B Read Quality of Service Register
SCB0_SP3B_WRITE_QOS	SP3B Write Quality of Service Register
SCB0_SP4A_READ_QOS	SP4A Read Quality of Service Register
SCB0_SP4A_WRITE_QOS	SP4A Write Quality of Service Register
SCB0_SP4B_READ_QOS	SP4B Read Quality of Service Register
SCB0_SP4B_WRITE_QOS	SP4B Write Quality of Service Register
SCB0_SP5A_READ_QOS	SP5A Read Quality of Service Register

Table 42-14: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_SP5A_WRITE_QOS	SP5A Write Quality of Service Register
SCB0_SP5B_READ_QOS	SP5B Read Quality of Service Register
SCB0_SP5B_WRITE_QOS	SP5B Write Quality of Service Register
SCB0_SP6A_READ_QOS	SP6A Read Quality of Service Register
SCB0_SP6A_WRITE_QOS	SP6A Write Quality of Service Registers
SCB0_SP6B_READ_QOS	SP6B Read Quality of Service Register
SCB0_SP6B_WRITE_QOS	SP6B Write Quality of Service Register
SCB0_SP7A_READ_QOS	SP7A Read Quality of Service Register
SCB0_SP7A_WRITE_QOS	SP7A Write Quality of Service Register
SCB0_SP7B_READ_QOS	SP7B Read Quality of Service Register
SCB0_SP7B_WRITE_QOS	SP7B Write Quality of Service Register
SCB0_SPI0RX_READ_QOS	SPI0 RX Read Quality of Service Register
SCB0_SPI0RX_WRITE_QOS	SPI0 RX Write Quality of Service Register
SCB0_SPI0TX_READ_QOS	SPI0 TX Read Quality of Service Register
SCB0_SPI0TX_WRITE_QOS	SPI0 TX Write Quality of Service Register
SCB0_SPI1RX_READ_QOS	SPI1 RX Read Quality of Service Register
SCB0_SPI1RX_WRITE_QOS	SPI1 RX Write Quality of Service Register
SCB0_SPI1TX_READ_QOS	SPI1 TX Read Quality of Service Register
SCB0_SPI1TX_WRITE_QOS	SPI1 TX Write Quality of Service Register
SCB0_SPI2RX_READ_QOS	SPI2 RX Read Quality of Service Register
SCB0_SPI2RX_WRITE_QOS	SPI2 RX Write Quality of Service Register
SCB0_SPI2TX_READ_QOS	SPI2 TX Read Quality of Service Register
SCB0_SPI2TX_WRITE_QOS	SPI2 TX Write Quality of Service Register
SCB0_SPI3RX_READ_QOS	SPI3 RX Read Quality of Service Register
SCB0_SPI3RX_WRITE_QOS	SPI3 RX Write Quality of Service Register
SCB0_SPI3TX_READ_QOS	SPI3 TX Read Quality of Service Register
SCB0_SPI3TX_WRITE_QOS	SPI3 TX Write Quality of Service Register
SCB0_UART0_RX_READ_QOS	UART0 RX Read Quality of Service Register
SCB0_UART0_RX_WRITE_QOS	UART0 RX Write Quality of Service Register
SCB0_UART0_TX_READ_QOS	UART0 TX Read Quality of Service Register
SCB0_UART0_TX_WRITE_QOS	UART0 TX Write Quality of Service Register

Table 42-14: ADSP-2159x SCB0 Register List (Continued)

Name	Description
SCB0_UART1_RX_READ_QOS	UART1 RX Read Quality of Service Register
SCB0_UART1_RX_WRITE_QOS	UART1 RX Write Quality of Service Register
SCB0_UART1_TX_READ_QOS	UART1 TX Read Quality of Service Register
SCB0_UART1_TX_WRITE_QOS	UART1 TX Write Quality of Service Registers
SCB0_UART2_RX_READ_QOS	UART2 RX Read Quality of Service Register
SCB0_UART2_RX_WRITE_QOS	UART2 RX Write Quality of Service Register
SCB0_UART2_TX_READ_QOS	UART2 TX Read Quality of Service Register
SCB0_UART2_TX_WRITE_QOS	UART2 TX Write Quality of Service Register
SCB0_UART3_RX_READ_QOS	UART3 RX Read Quality of Service Register
SCB0_UART3_RX_WRITE_QOS	UART3 RX Write Quality of Service Register
SCB0_UART3_TX_READ_QOS	UART3 TX Read Quality of Service Register
SCB0_UART3_TX_WRITE_QOS	UART3 TX Write Quality of Service Register
SCB0_USB0_READ_QOS	USB0 Read Quality of Service Register
SCB0_USB0_WRITE_QOS	USB0 Write Quality of Service Register

CRC0 Channel 0 Read Quality of Service Register



Figure 42-9: SCB0_CRC0_CH0_READ_QOS Register Diagram

Table 42-15: SCB0_CRC0_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRC0 Channel 0 Write Quality of Service Register



Figure 42-10: SCB0_CRC0_CH0_WRITE_QOS Register Diagram

Table 42-16: SCB0_CRC0_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

CRC0 Channel 1 Read Quality of Service Register



Figure 42-11: SCB0_CRC0_CH1_READ_QOS Register Diagram

Table 42-17: SCB0_CRC0_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRC0 Channel 1 Write Quality of Service Register



Figure 42-12: SCB0_CRC0_CH1_WRITE_QOS Register Diagram

Table 42-18: SCB0_CRC0_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

CRC1 Channel 0 Read Quality of Service Register



Figure 42-13: SCB0_CRC1_CH0_READ_QOS Register Diagram

Table 42-19: SCB0_CRC1_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRC1 Channel 0 Write Quality of Service Register



Figure 42-14: SCB0_CRC1_CH0_WRITE_QOS Register Diagram

Table 42-20: SCB0_CRC1_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

CRC1 Channel 1 Read Quality of Service Register



Figure 42-15: SCB0_CRC1_CH1_READ_QOS Register Diagram

Table 42-21: SCB0_CRC1_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRC1 Channel 1 Write Quality of Service Register



Figure 42-16: SCB0_CRC1_CH1_WRITE_QOS Register Diagram

Table 42-22: SCB0_CRC1_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

CRC2 Channel 0 Read Quality of Service Register



Figure 42-17: SCB0_CRC2_CH0_READ_QOS Register Diagram

Table 42-23: SCB0_CRC2_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRC2 Channel 0 Write Quality of Service Register



Figure 42-18: SCB0_CRC2_CH0_WRITE_QOS Register Diagram

Table 42-24: SCB0_CRC2_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

CRC2 Channel 1 Read Quality of Service Register



Figure 42-19: SCB0_CRC2_CH1_READ_QOS Register Diagram

Table 42-25: SCB0_CRC2_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRC2 Channel 1 Write Quality of Service Register



Figure 42-20: SCB0_CRC2_CH1_WRITE_QOS Register Diagram

Table 42-26: SCB0_CRC2_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

CRC3 Channel 0 Read Quality of Service Register



Figure 42-21: SCB0_CRC3_CH0_READ_QOS Register Diagram

Table 42-27: SCB0_CRC3_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRC3 Channel 0 Write Quality of Service Register



Figure 42-22: SCB0_CRC3_CH0_WRITE_QOS Register Diagram

Table 42-28: SCB0_CRC3_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

CRC3 Channel 1 Read Quality of Service Register



Figure 42-23: SCB0_CRC3_CH1_READ_QOS Register Diagram

Table 42-29: SCB0_CRC3_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRC3 Channel 1 Write Quality of Service Register



Figure 42-24: SCB0_CRC3_CH1_WRITE_QOS Register Diagram

Table 42-30: SCB0_CRC3_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

CRYPTO Read Quality of Service Register



Figure 42-25: SCB0_CRYPT0_READ_QOS Register Diagram

Table 42-31: SCB0_CRYPT0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

CRYPTO Write Quality of Service Register



Figure 42-26: SCB0_CRYPTOWRITE_QOS Register Diagram

Table 42-32: SCB0_CRYPTOWRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

DBG Read Quality of Service Register



Figure 42-27: SCB0_DBG_READ_QOS Register Diagram

Table 42-33: SCB0_DBG_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

DBG Write Quality of Service Register



Figure 42-28: SCB0_DBG_WRITE_QOS Register Diagram

Table 42-34: SCB0_DBG_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

DLDMA0 Channel 0 Read Quality of Service Register



Figure 42-29: SCB0_DLDMA0_CH0_READ_QOS Register Diagram

Table 42-35: SCB0_DLDMA0_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

DLDMA0 Channel 0 Write Quality of Service Register



Figure 42-30: SCB0_DLDMA0_CH0_WRITE_QOS Register Diagram

Table 42-36: SCB0_DLDMA0_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

DLDMA0 Channel 1 Read Quality of Service Register



Figure 42-31: SCB0_DLDMA0_CH1_READ_QOS Register Diagram

Table 42-37: SCB0_DLDMA0_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

DLDMA0 Channel 1 Write Quality of Service Register



Figure 42-32: SCB0_DLDMA0_CH1_WRITE_QOS Register Diagram

Table 42-38: SCB0_DLDMA0_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

DLDMA1 Channel 0 Read Quality of Service Register



Figure 42-33: SCB0_DLDMA1_CH0_READ_QOS Register Diagram

Table 42-39: SCB0_DLDMA1_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

DLDMA1 Channel 0 Write Quality of Service Register



Figure 42-34: SCB0_DLDMA1_CH0_WRITE_QOS Register Diagram

Table 42-40: SCB0_DLDMA1_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

DLDMA1 Channel 1 Read Quality of Service Register



Figure 42-35: SCB0_DLDMA1_CH1_READ_QOS Register Diagram

Table 42-41: SCB0_DLDMA1_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

DLDMA1 Channel 1 Write Quality of Service Register



Figure 42-36: SCB0_DLDMA1_CH1_WRITE_QOS Register Diagram

Table 42-42: SCB0_DLDMA1_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

EMAC Read Quality of Service Register

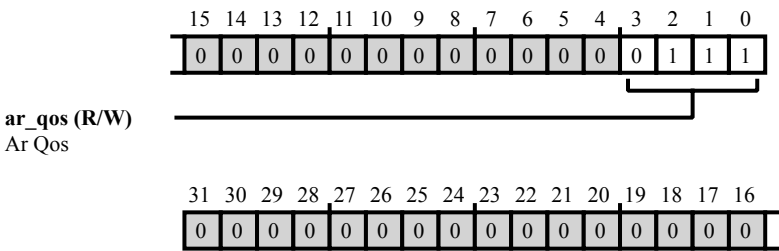


Figure 42-37: SCB0_EMAC_READ_QOS Register Diagram

Table 42-43: SCB0_EMAC_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

EMAC Write Quality of Service Register

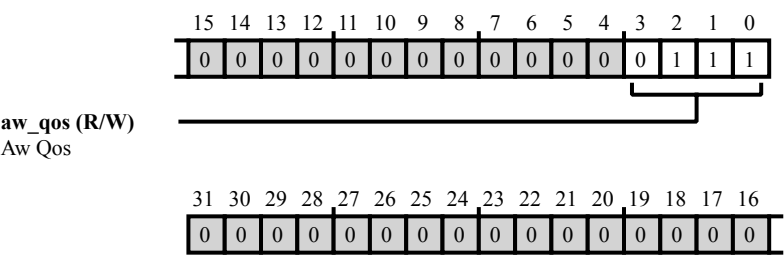


Figure 42-38: SCB0_EMAC_WRITE_QOS Register Diagram

Table 42-44: SCB0_EMAC_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

ETR Read Quality of Service Register



Figure 42-39: SCB0_ETR_READ_QOS Register Diagram

Table 42-45: SCB0_ETR_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

ETR Write Quality of Service Register



Figure 42-40: SCB0_ETR_WRITE_QOS Register Diagram

Table 42-46: SCB0_ETR_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

GIGE Read Quality of Service Register

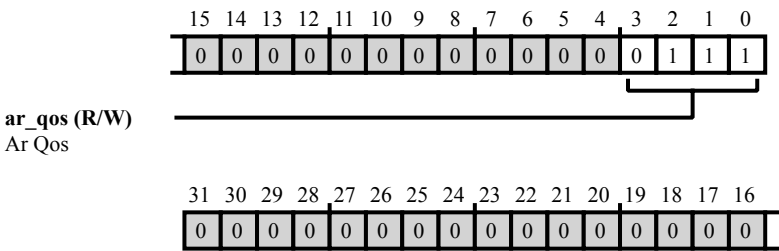


Figure 42-41: SCB0_GIGE_READ_QOS Register Diagram

Table 42-47: SCB0_GIGE_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

GIGE Write Quality of Service Register

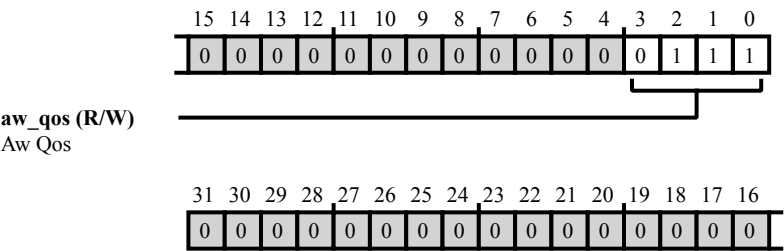


Figure 42-42: SCB0_GIGE_WRITE_QOS Register Diagram

Table 42-48: SCB0_GIGE_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

HSMDMA1 Channel 0 Read Quality of Service Register



Figure 42-43: SCB0_HSMDMA1_CH0_READ_QOS Register Diagram

Table 42-49: SCB0_HSMDMA1_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

HSMDMA1 Channel 0 Write Quality of Service Register



Figure 42-44: SCB0_HSMDMA1_CH0_WRITE_QOS Register Diagram

Table 42-50: SCB0_HSMDMA1_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

HSMDMA1 Channel 1 Read Quality of Service Register



Figure 42-45: SCB0_HSMDMA1_CH1_READ_QOS Register Diagram

Table 42-51: SCB0_HSMDMA1_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

HSMDMA1 Channel 1 Write Quality of Service Register



Figure 42-46: SCB0_HSMDMA1_CH1_WRITE_QOS Register Diagram

Table 42-52: SCB0_HSMDMA1_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

HSMDMA Channel 0 Read Quality of Service Register



Figure 42-47: SCB0_HSMDMA_CH0_READ_QOS Register Diagram

Table 42-53: SCB0_HSMDMA_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

HSMDMA Channel 0 Write Quality of Service Register



Figure 42-48: SCB0_HSMDMA_CH0_WRITE_QOS Register Diagram

Table 42-54: SCB0_HSMDMA_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

HSMDMA Channel 1 Read Quality of Service Register



Figure 42-49: SCB0_HSMDMA_CH1_READ_QOS Register Diagram

Table 42-55: SCB0_HSMDMA_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

HSMDMA Channel 1 Write Quality of Service Register



Figure 42-50: SCB0_HSMDMA_CH1_WRITE_QOS Register Diagram

Table 42-56: SCB0_HSMDMA_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

LP0 Read Quality of Service Register

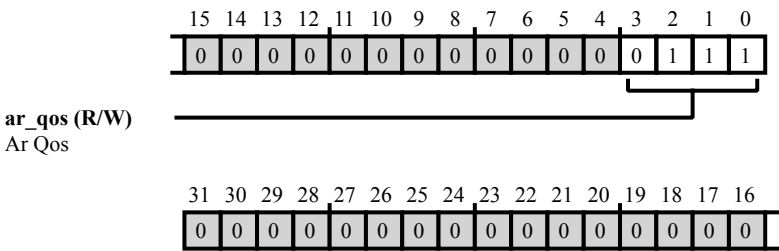


Figure 42-51: SCB0_LP0_READ_QOS Register Diagram

Table 42-57: SCB0_LP0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

LP0 Write Quality of Service Register

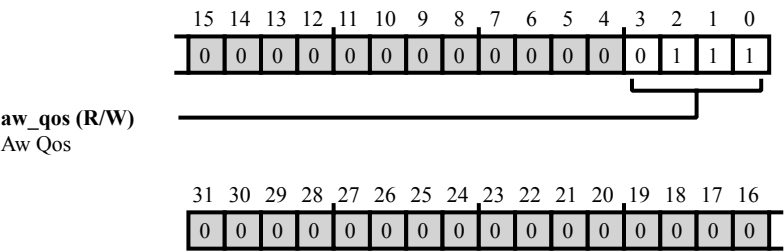


Figure 42-52: SCB0_LP0_WRITE_QOS Register Diagram

Table 42-58: SCB0_LP0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

LP1 Read Quality of Service Register

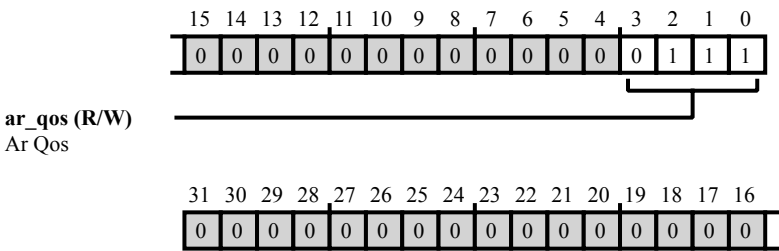


Figure 42-53: SCB0_LP1_READ_QOS Register Diagram

Table 42-59: SCB0_LP1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

LP1 Write Quality of Service Register

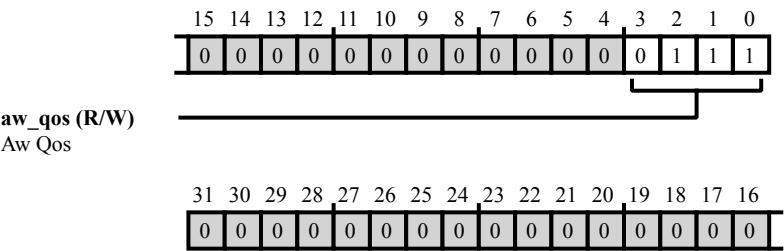


Figure 42-54: SCB0_LP1_WRITE_QOS Register Diagram

Table 42-60: SCB0_LP1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

MLB Read Quality of Service Register

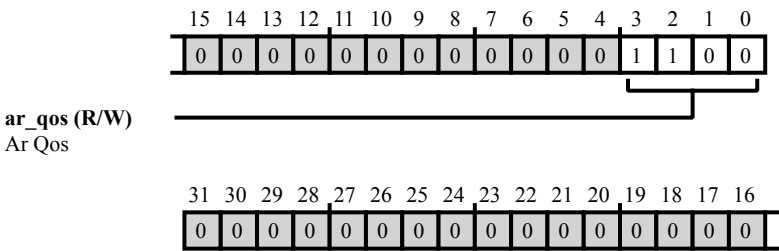


Figure 42-55: SCB0_MLB_READ_QOS Register Diagram

Table 42-61: SCB0_MLB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

MLB Write Quality of Service Register

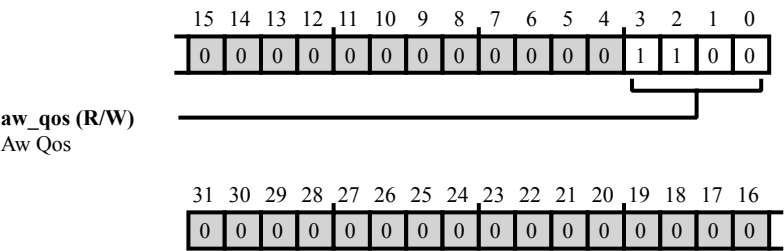


Figure 42-56: SCB0_MLB_WRITE_QOS Register Diagram

Table 42-62: SCB0_MLB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

MSMDMA1 Channel 0 Read Quality of Service Register



Figure 42-57: SCB0_MSMDMA1_CH0_READ_QOS Register Diagram

Table 42-63: SCB0_MSMDMA1_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

MSMDMA1 Channel 0 Write Quality of Service Register



Figure 42-58: SCB0_MSMDMA1_CH0_WRITE_QOS Register Diagram

Table 42-64: SCB0_MSMDMA1_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

MSMDMA1 Channel 1 Read Quality of Service Register



Figure 42-59: SCB0_MSMDMA1_CH1_READ_QOS Register Diagram

Table 42-65: SCB0_MSMDMA1_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

MSMDMA1 Channel 1 Write Quality of Service Register



Figure 42-60: SCB0_MSMDMA1_CH1_WRITE_QOS Register Diagram

Table 42-66: SCB0_MSMDMA1_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

MSMDMA Channel 0 Read Quality of Service Register



Figure 42-61: SCB0_MSMDMA_CH0_READ_QOS Register Diagram

Table 42-67: SCB0_MSMDMA_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

MSMDMA Channel 0 Write Quality of Service Register



Figure 42-62: SCB0_MSMDMA_CH0_WRITE_QOS Register Diagram

Table 42-68: SCB0_MSMDMA_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

MSMDMA Channel 1 Read Quality of Service Register



Figure 42-63: SCB0_MSMDMA_CH1_READ_QOS Register Diagram

Table 42-69: SCB0_MSMDMA_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

MSMDMA Channel 1 Write Quality of Service Registers



Figure 42-64: SCB0_MSMDMA_CH1_WRITE_QOS Register Diagram

Table 42-70: SCB0_MSMDMA_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

ARM_L2CC M0 Read Quality of Service Register

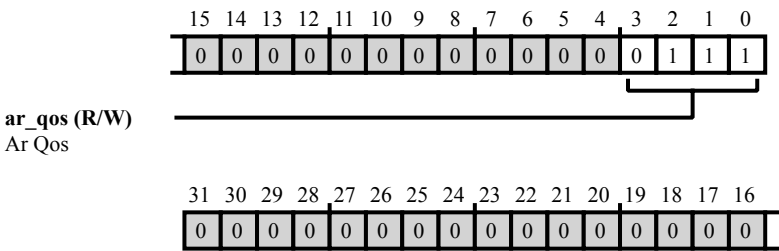


Figure 42-65: SCB0_PL310_M0_READ_QOS Register Diagram

Table 42-71: SCB0_PL310_M0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

ARM_L2CC M0 Write Quality of Service Register

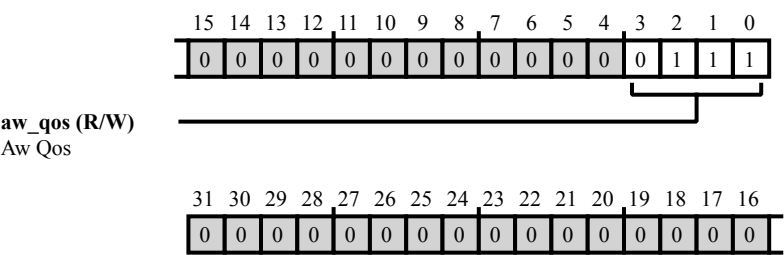


Figure 42-66: SCB0_PL310_M0_WRITE_QOS Register Diagram

Table 42-72: SCB0_PL310_M0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

ARM_L2CC M1 Read Quality of Service Register



Figure 42-67: SCB0_PL310_M1_READ_QOS Register Diagram

Table 42-73: SCB0_PL310_M1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

ARM_L2CC M1 Write Quality of Service Register



Figure 42-68: SCB0_PL310_M1_WRITE_QOS Register Diagram

Table 42-74: SCB0_PL310_M1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

PPI F0 Read Quality of Service Register

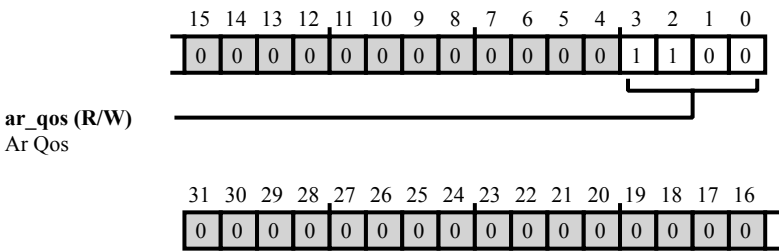


Figure 42-69: SCB0_PPI_F0_READ_QOS Register Diagram

Table 42-75: SCB0_PPI_F0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

PPI F0 Write Quality of Service Register

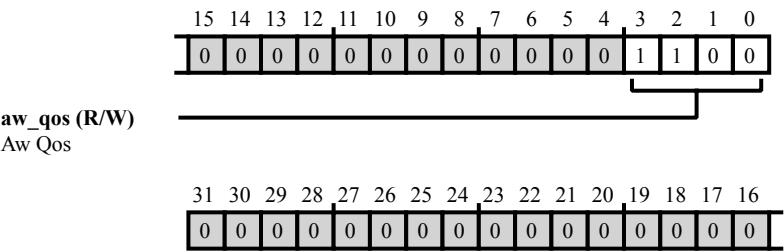


Figure 42-70: SCB0_PPI_F0_WRITE_QOS Register Diagram

Table 42-76: SCB0_PPI_F0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

PPI F1 Read Quality of Service Register

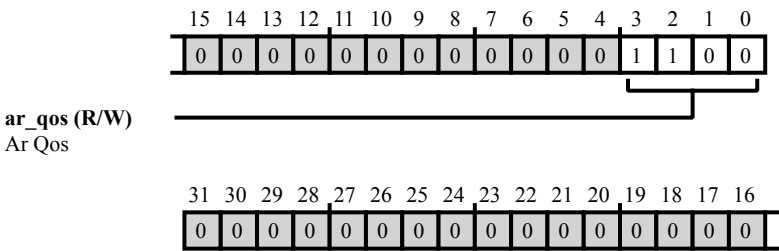


Figure 42-71: SCB0_PPI_F1_READ_QOS Register Diagram

Table 42-77: SCB0_PPI_F1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

PPI F1 Write Quality of Service Register

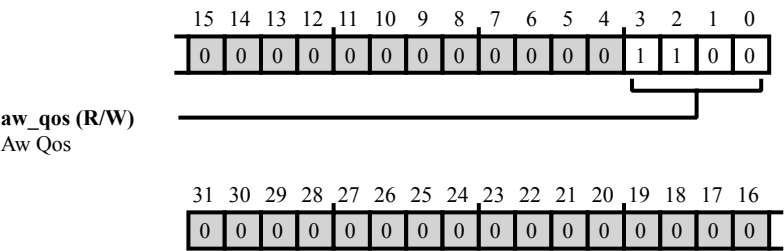


Figure 42-72: SCB0_PPI_F1_WRITE_QOS Register Diagram

Table 42-78: SCB0_PPI_F1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

LP Fabric (CLKO8) Synchronization Mode Register



Figure 42-73: SCB0_LP_SYNC_MODE Register Diagram

Table 42-79: SCB0_LP_SYNC_MODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	SYNC_MODE	Sync Mode. Default is ASYNC('h4) other valid option is M:N('h3)..

SH0 DPORT Read Quality of Service Register

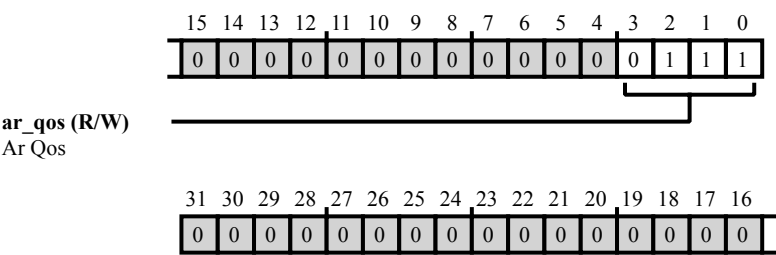


Figure 42-74: SCB0_SH0_DPORT_READ_QOS Register Diagram

Table 42-80: SCB0_SH0_DPORT_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH0 DPORT Write Quality of Service Register

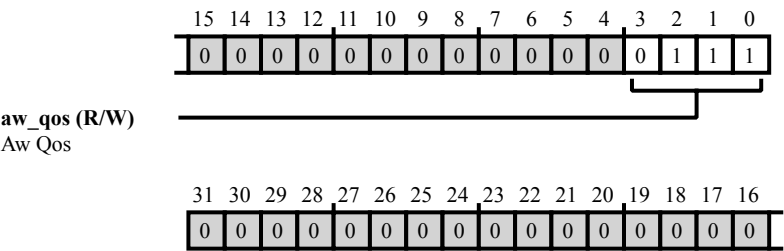


Figure 42-75: SCB0_SH0_DPORT_WRITE_QOS Register Diagram

Table 42-81: SCB0_SH0_DPORT_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH0 FIR Channel 0 Read Quality of Service Register

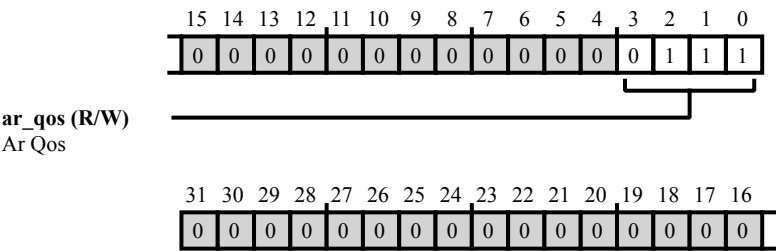


Figure 42-76: SCB0_SH0_FIR_CH0_READ_QOS Register Diagram

Table 42-82: SCB0_SH0_FIR_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH0 FIR Channel 0 Write Quality of Service Register

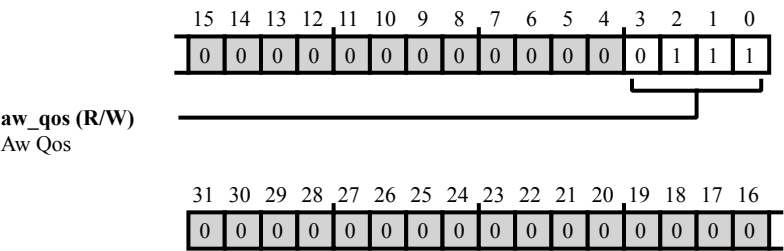


Figure 42-77: SCB0_SH0_FIR_CH0_WRITE_QOS Register Diagram

Table 42-83: SCB0_SH0_FIR_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH0 FIR Channel 1 Read Quality of Service Register

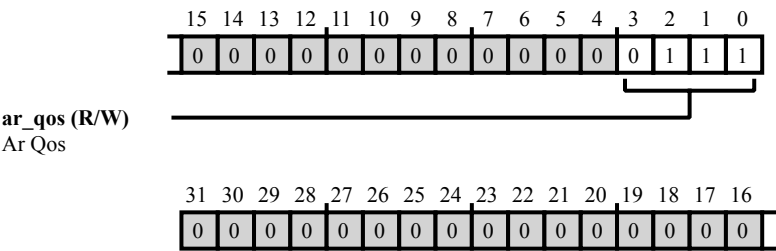


Figure 42-78: SCB0_SH0_FIR_CH1_READ_QOS Register Diagram

Table 42-84: SCB0_SH0_FIR_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH0 FIR Channel 1 Write Quality of Service Register

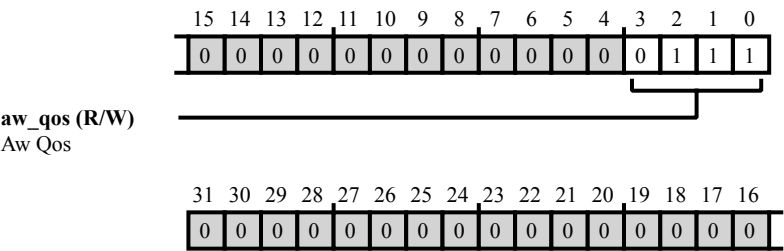


Figure 42-79: SCB0_SH0_FIR_CH1_WRITE_QOS Register Diagram

Table 42-85: SCB0_SH0_FIR_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH0 IIR Channel 0 Read Quality of Service Register

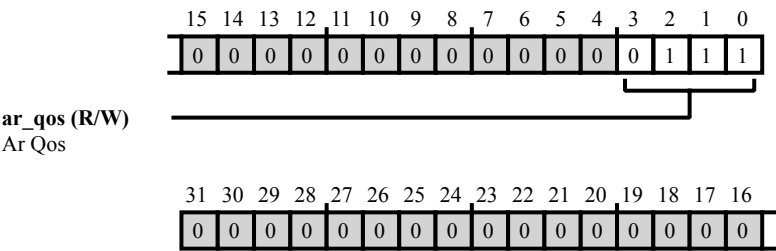


Figure 42-80: SCB0_SH0_IIR_CH0_READ_QOS Register Diagram

Table 42-86: SCB0_SH0_IIR_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH0 IIR Channel 0 Write Quality of Service Register

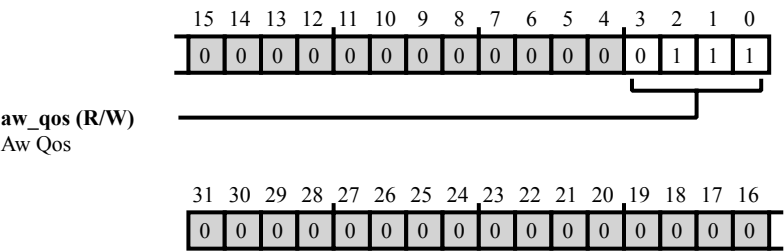


Figure 42-81: SCB0_SH0_IIR_CH0_WRITE_QOS Register Diagram

Table 42-87: SCB0_SH0_IIR_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH0 IIR Channel 1 Read Quality of Service Register

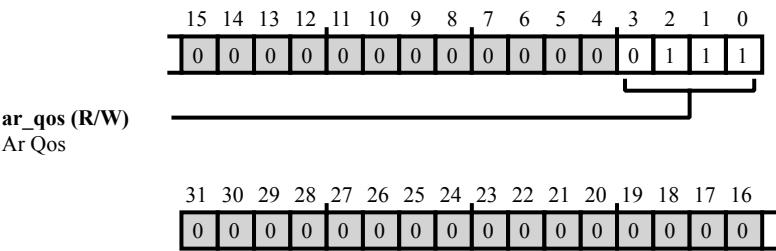


Figure 42-82: SCB0_SH0_IIR_CH1_READ_QOS Register Diagram

Table 42-88: SCB0_SH0_IIR_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH0 IIR Channel 1 Write Quality of Service Register

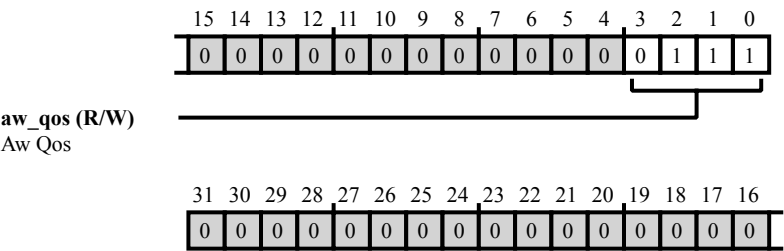


Figure 42-83: SCB0_SH0_IIR_CH1_WRITE_QOS Register Diagram

Table 42-89: SCB0_SH0_IIR_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH0 LPORT Read Quality of Service Register

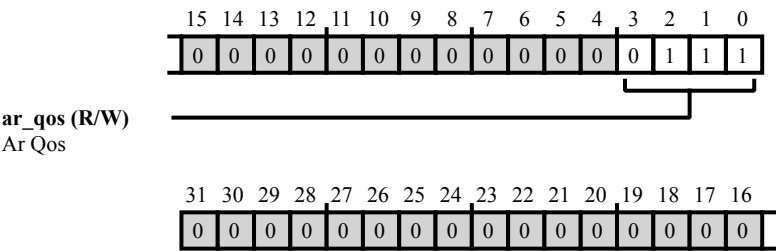


Figure 42-84: SCB0_SH0_IPORT_READ_QOS Register Diagram

Table 42-90: SCB0_SH0_IPORT_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH0 LPORT Write Quality of Service Register

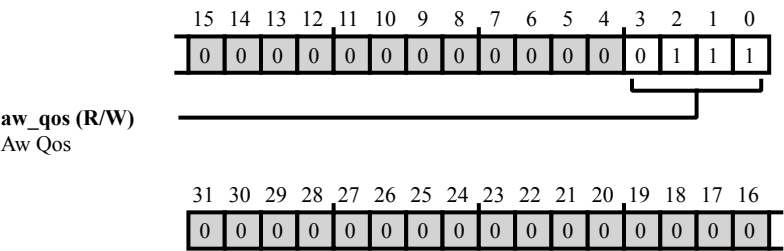


Figure 42-85: SCB0_SH0_IPORT_WRITE_QOS Register Diagram

Table 42-91: SCB0_SH0_IPORT_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH0 MMR Read Quality of Service Register



Figure 42-86: SCB0_SH0_MMR_READ_QOS Register Diagram

Table 42-92: SCB0_SH0_MMR_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH0 MMR Write Quality of Service Register



Figure 42-87: SCB0_SH0_MMR_WRITE_QOS Register Diagram

Table 42-93: SCB0_SH0_MMR_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH1 DPORT Read Quality of Service Register

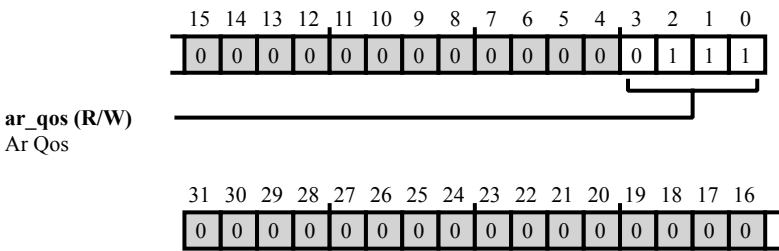


Figure 42-88: SCB0_SH1_DPORT_READ_QOS Register Diagram

Table 42-94: SCB0_SH1_DPORT_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH1 DPORT Write Quality of Service Register

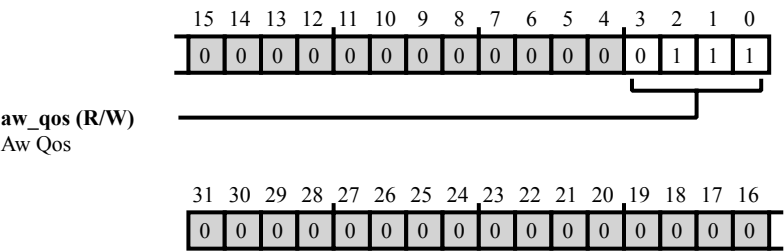


Figure 42-89: SCB0_SH1_DPORT_WRITE_QOS Register Diagram

Table 42-95: SCB0_SH1_DPORT_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH1 FIR Channel 0 Read Quality of Service Register

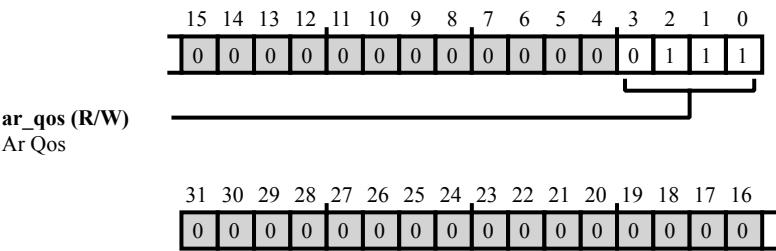


Figure 42-90: SCB0_SH1_FIR_CH0_READ_QOS Register Diagram

Table 42-96: SCB0_SH1_FIR_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH1 FIR Channel 0 Write Quality of Service Register

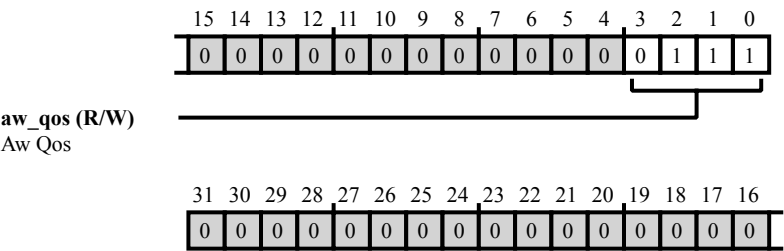


Figure 42-91: SCB0_SH1_FIR_CH0_WRITE_QOS Register Diagram

Table 42-97: SCB0_SH1_FIR_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH1 FIR Channel 1 Read Quality of Service Register

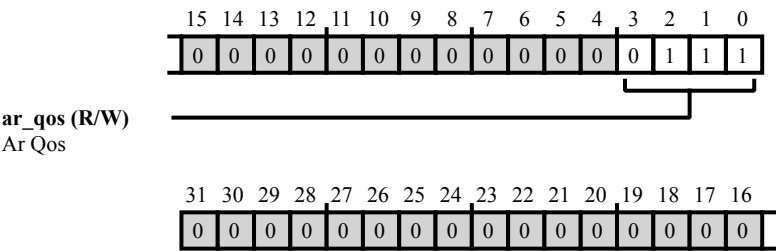


Figure 42-92: SCB0_SH1_FIR_CH1_READ_QOS Register Diagram

Table 42-98: SCB0_SH1_FIR_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH1 FIR Channel 1 Write Quality of Service Register

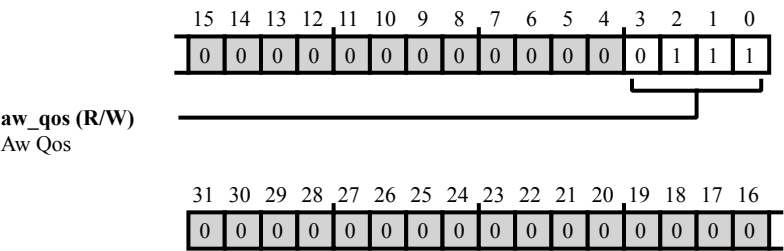


Figure 42-93: SCB0_SH1_FIR_CH1_WRITE_QOS Register Diagram

Table 42-99: SCB0_SH1_FIR_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH1 IIR Channel 0 Read Quality of Service Register

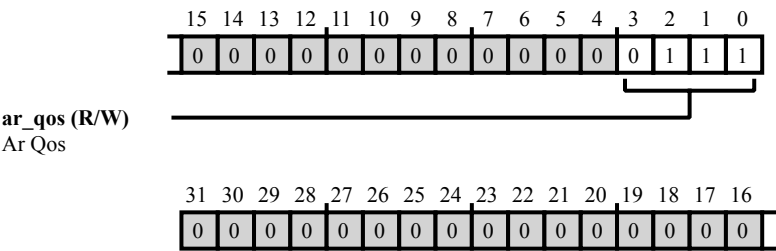


Figure 42-94: SCB0_SH1_IIR_CH0_READ_QOS Register Diagram

Table 42-100: SCB0_SH1_IIR_CH0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH1 IIR Channel 0 Write Quality of Service Register

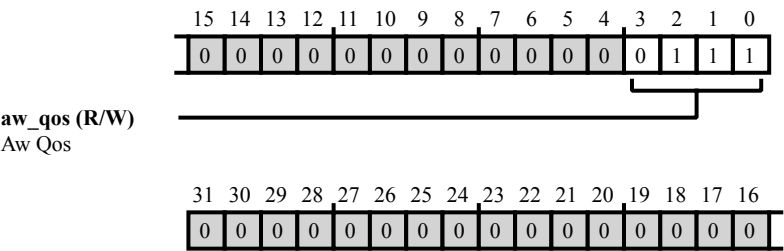


Figure 42-95: SCB0_SH1_IIR_CH0_WRITE_QOS Register Diagram

Table 42-101: SCB0_SH1_IIR_CH0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH1 IIR Channel 1 Read Quality of Service Register

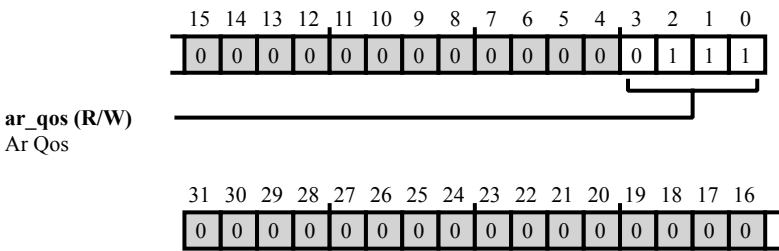


Figure 42-96: SCB0_SH1_IIR_CH1_READ_QOS Register Diagram

Table 42-102: SCB0_SH1_IIR_CH1_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH1 IIR Channel 1 Write Quality of Service Register

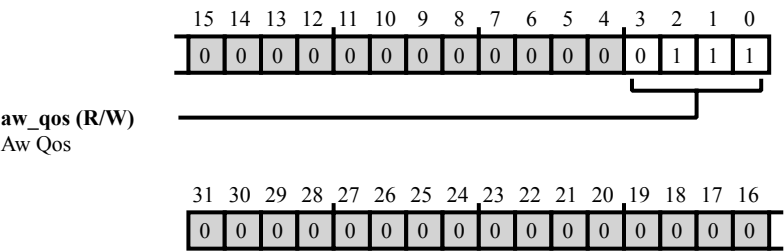


Figure 42-97: SCB0_SH1_IIR_CH1_WRITE_QOS Register Diagram

Table 42-103: SCB0_SH1_IIR_CH1_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH1 IPORT Read Quality of Service Register

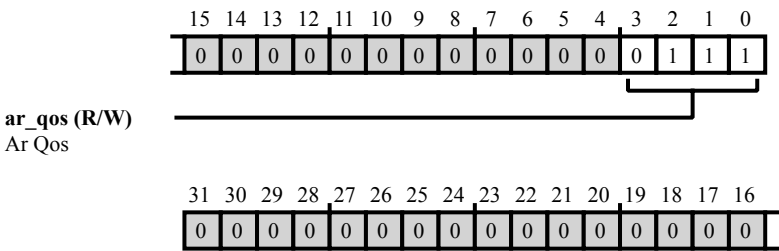


Figure 42-98: SCB0_SH1_IPORT_READ_QOS Register Diagram

Table 42-104: SCB0_SH1_IPORT_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH1 IPORT Write Quality of Service Register

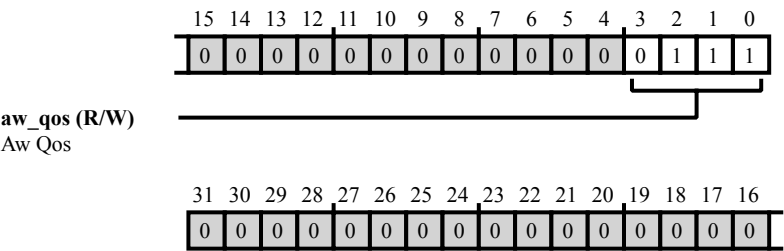


Figure 42-99: SCB0_SH1_IPORT_WRITE_QOS Register Diagram

Table 42-105: SCB0_SH1_IPORT_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SH1 MMR Read Quality of Service Register



Figure 42-100: SCB0_SH1_MMR_READ_QOS Register Diagram

Table 42-106: SCB0_SH1_MMR_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SH1 MMR Write Quality of Service Register



Figure 42-101: SCB0_SH1_MMR_WRITE_QOS Register Diagram

Table 42-107: SCB0_SH1_MMR_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP0A Read Quality of Service Register

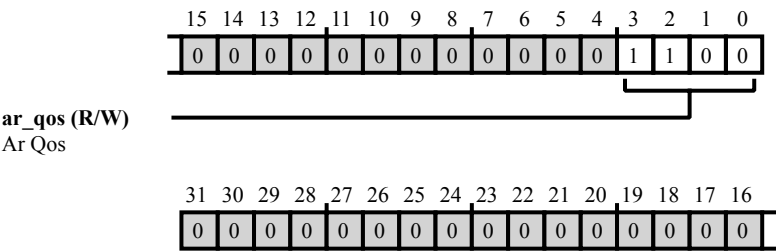


Figure 42-102: SCB0_SP0A_READ_QOS Register Diagram

Table 42-108: SCB0_SP0A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP0A Write Quality of Service Register

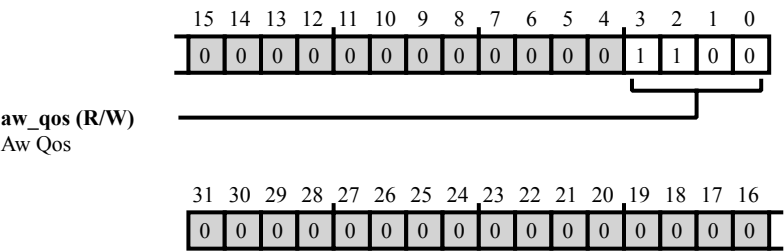


Figure 42-103: SCB0_SP0A_WRITE_QOS Register Diagram

Table 42-109: SCB0_SP0A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPOB Read Quality of Service Register

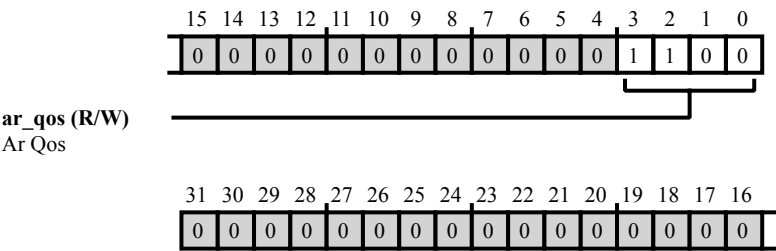


Figure 42-104: SCB0_SP0B_READ_QOS Register Diagram

Table 42-110: SCB0_SP0B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPOB Write Quality of Service Register

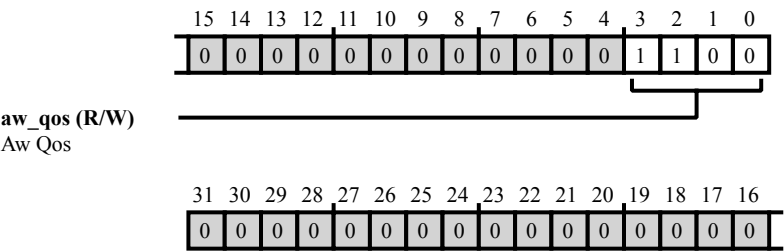


Figure 42-105: SCB0_SP0B_WRITE_QOS Register Diagram

Table 42-111: SCB0_SP0B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP1A Read Quality of Service Register

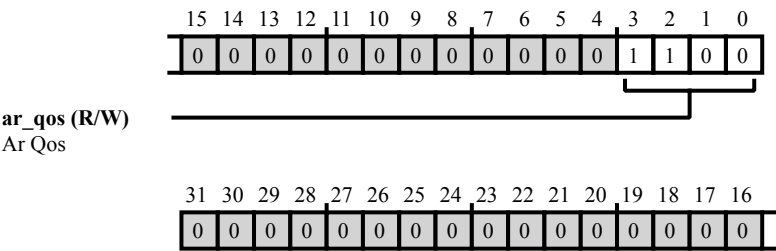


Figure 42-106: SCB0_SP1A_READ_QOS Register Diagram

Table 42-112: SCB0_SP1A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP1A Write Quality of Service Register

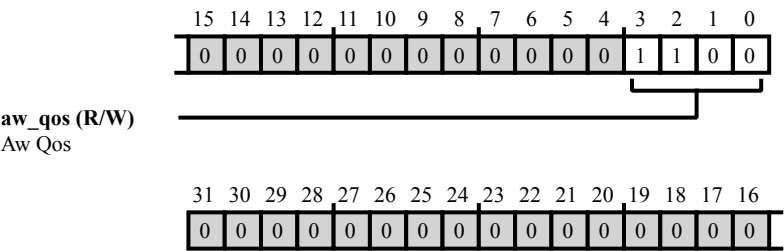


Figure 42-107: SCB0_SP1A_WRITE_QOS Register Diagram

Table 42-113: SCB0_SP1A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP1B Read Quality of Service Register

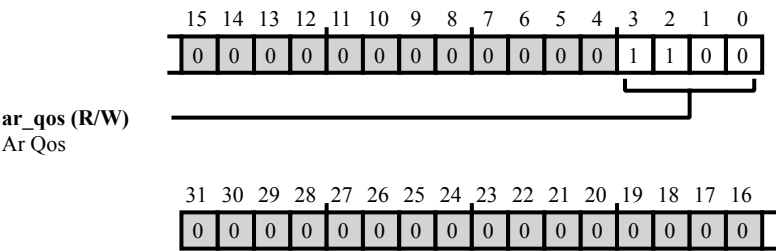


Figure 42-108: SCB0_SP1B_READ_QOS Register Diagram

Table 42-114: SCB0_SP1B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP1B Write Quality of Service Register

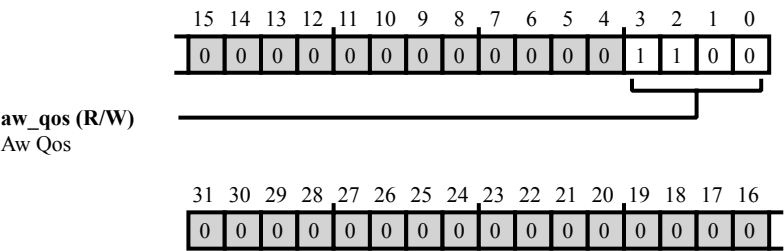


Figure 42-109: SCB0_SP1B_WRITE_QOS Register Diagram

Table 42-115: SCB0_SP1B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP2A Read Quality of Service Register

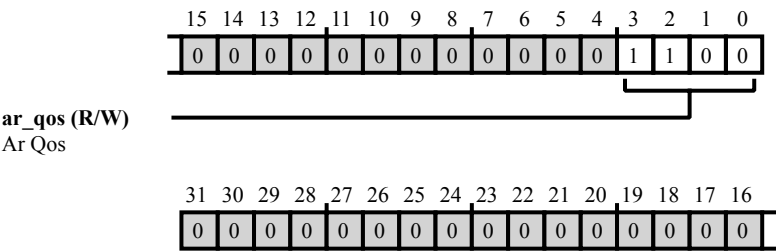


Figure 42-110: SCB0_SP2A_READ_QOS Register Diagram

Table 42-116: SCB0_SP2A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP2A Write Quality of Service Register

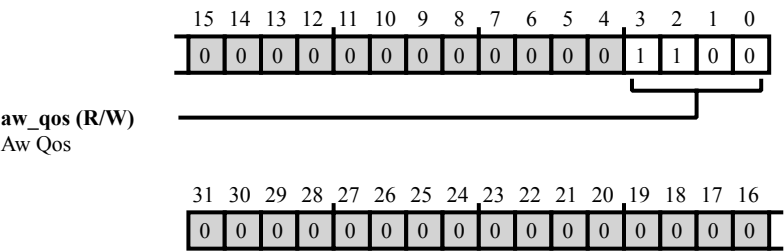


Figure 42-111: SCB0_SP2A_WRITE_QOS Register Diagram

Table 42-117: SCB0_SP2A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP2B Read Quality of Service Register

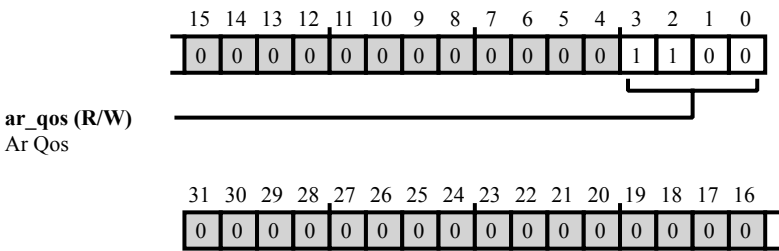


Figure 42-112: SCB0_SP2B_READ_QOS Register Diagram

Table 42-118: SCB0_SP2B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP2B Write Quality of Service Register

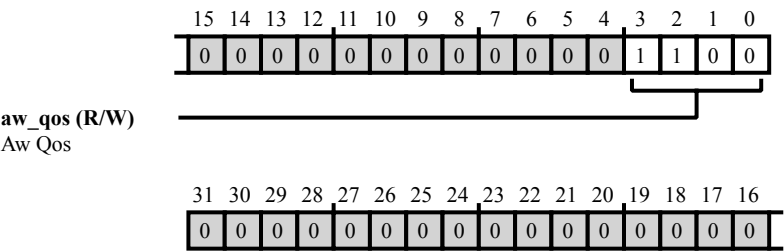


Figure 42-113: SCB0_SP2B_WRITE_QOS Register Diagram

Table 42-119: SCB0_SP2B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP3A Read Quality of Service Register

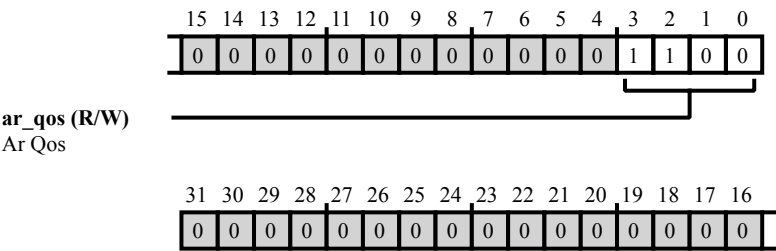


Figure 42-114: SCB0_SP3A_READ_QOS Register Diagram

Table 42-120: SCB0_SP3A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP3A Write Quality of Service Register

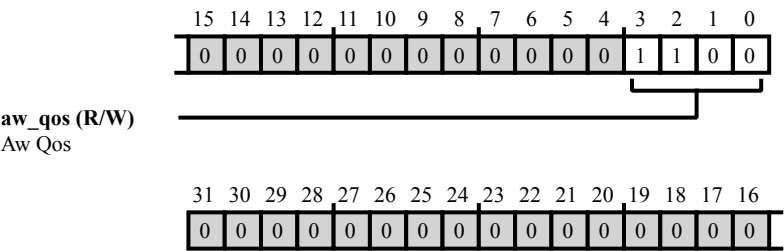


Figure 42-115: SCB0_SP3A_WRITE_QOS Register Diagram

Table 42-121: SCB0_SP3A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP3B Read Quality of Service Register

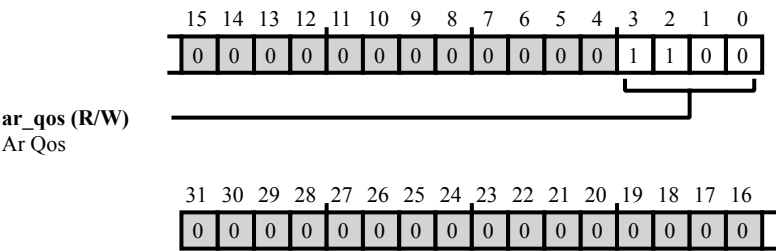


Figure 42-116: SCB0_SP3B_READ_QOS Register Diagram

Table 42-122: SCB0_SP3B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP3B Write Quality of Service Register

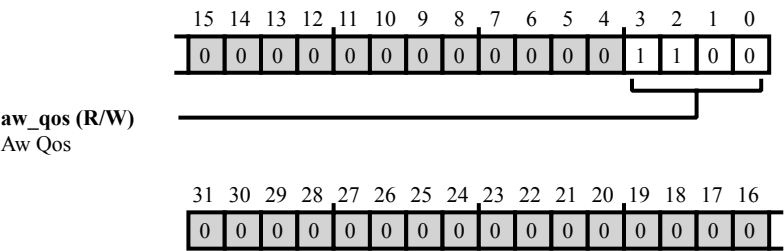


Figure 42-117: SCB0_SP3B_WRITE_QOS Register Diagram

Table 42-123: SCB0_SP3B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP4A Read Quality of Service Register

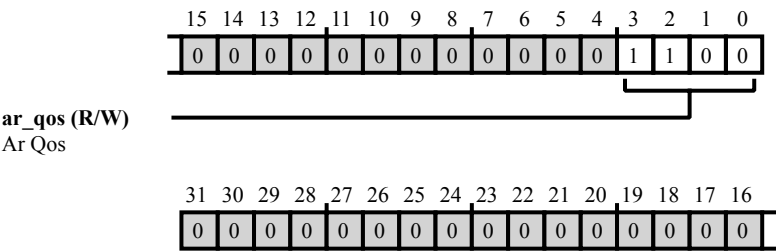


Figure 42-118: SCB0_SP4A_READ_QOS Register Diagram

Table 42-124: SCB0_SP4A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP4A Write Quality of Service Register

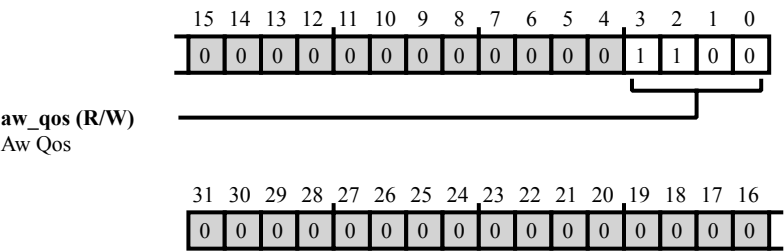


Figure 42-119: SCB0_SP4A_WRITE_QOS Register Diagram

Table 42-125: SCB0_SP4A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP4B Read Quality of Service Register

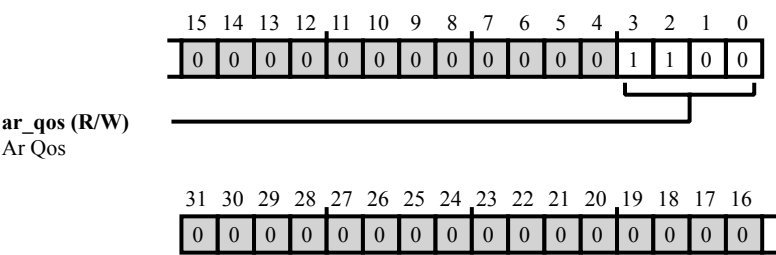


Figure 42-120: SCB0_SP4B_READ_QOS Register Diagram

Table 42-126: SCB0_SP4B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP4B Write Quality of Service Register

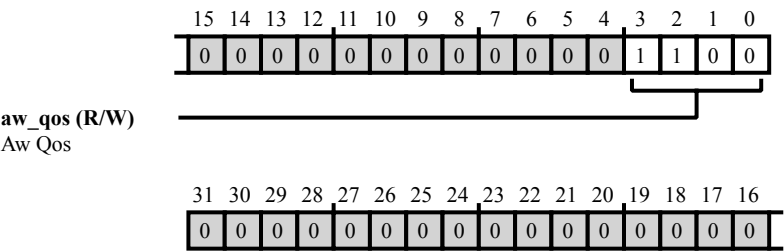


Figure 42-121: SCB0_SP4B_WRITE_QOS Register Diagram

Table 42-127: SCB0_SP4B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP5A Read Quality of Service Register

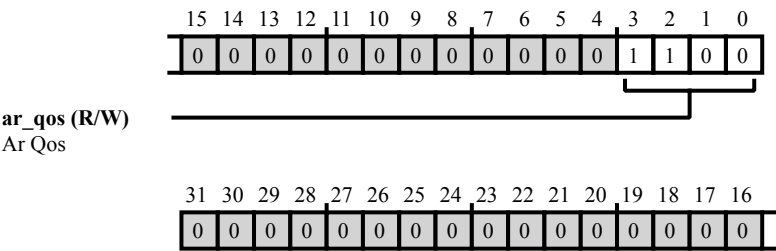


Figure 42-122: SCB0_SP5A_READ_QOS Register Diagram

Table 42-128: SCB0_SP5A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP5A Write Quality of Service Register

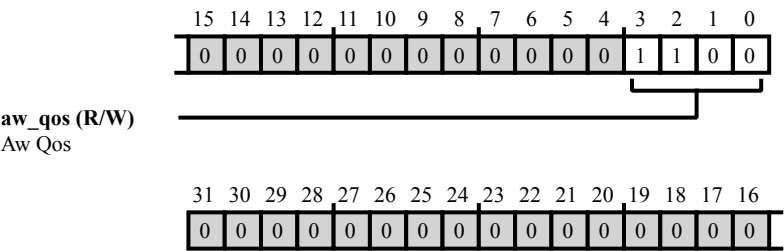


Figure 42-123: SCB0_SP5A_WRITE_QOS Register Diagram

Table 42-129: SCB0_SP5A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP5B Read Quality of Service Register

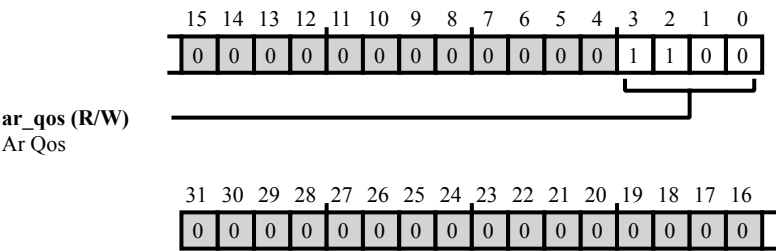


Figure 42-124: SCB0_SP5B_READ_QOS Register Diagram

Table 42-130: SCB0_SP5B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP5B Write Quality of Service Register

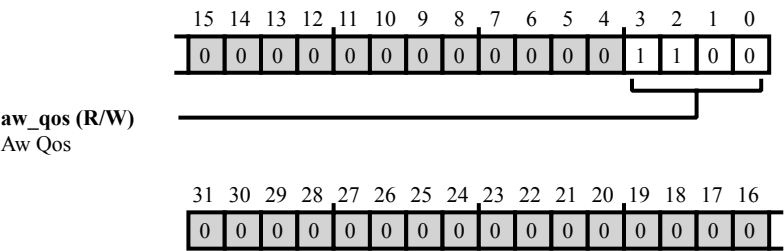


Figure 42-125: SCB0_SP5B_WRITE_QOS Register Diagram

Table 42-131: SCB0_SP5B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP6A Read Quality of Service Register

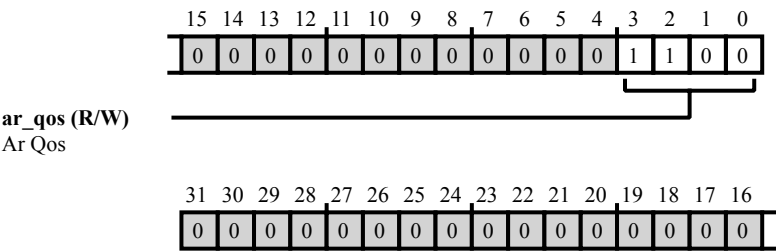


Figure 42-126: SCB0_SP6A_READ_QOS Register Diagram

Table 42-132: SCB0_SP6A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP6A Write Quality of Service Registers

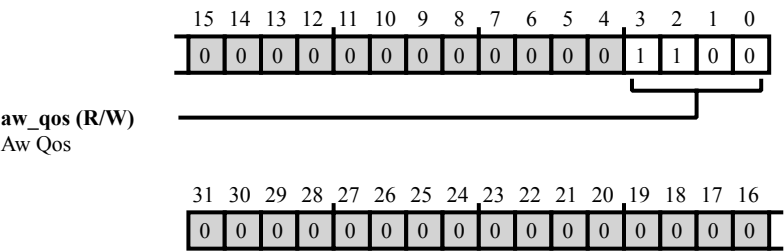


Figure 42-127: SCB0_SP6A_WRITE_QOS Register Diagram

Table 42-133: SCB0_SP6A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP6B Read Quality of Service Register

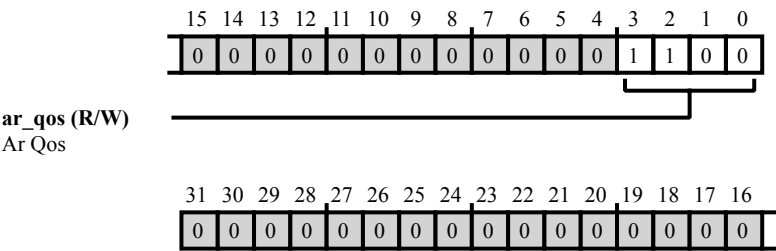


Figure 42-128: SCB0_SP6B_READ_QOS Register Diagram

Table 42-134: SCB0_SP6B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP6B Write Quality of Service Register

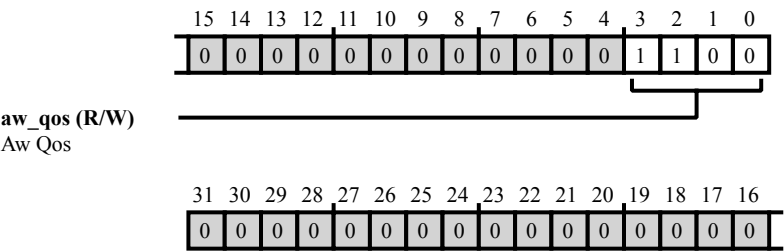


Figure 42-129: SCB0_SP6B_WRITE_QOS Register Diagram

Table 42-135: SCB0_SP6B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP7A Read Quality of Service Register

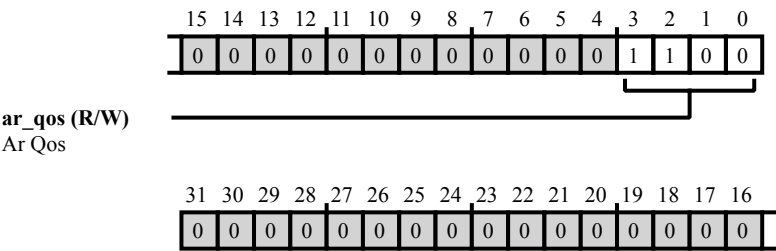


Figure 42-130: SCB0_SP7A_READ_QOS Register Diagram

Table 42-136: SCB0_SP7A_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP7A Write Quality of Service Register

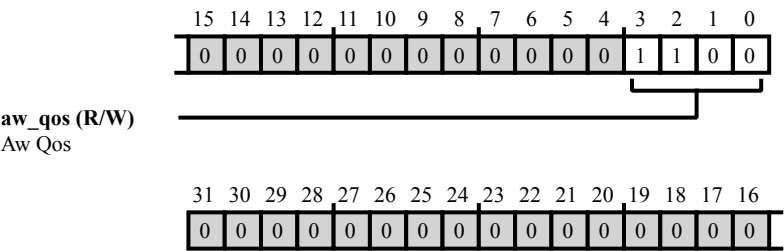


Figure 42-131: SCB0_SP7A_WRITE_QOS Register Diagram

Table 42-137: SCB0_SP7A_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SP7B Read Quality of Service Register

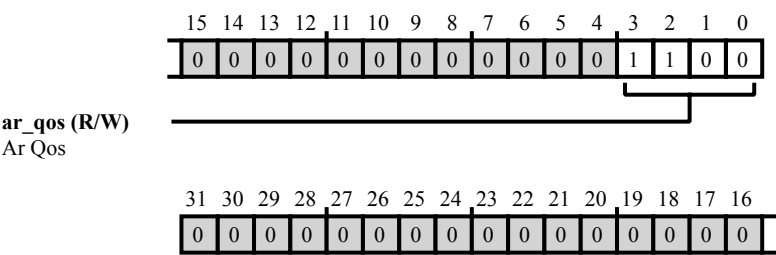


Figure 42-132: SCB0_SP7B_READ_QOS Register Diagram

Table 42-138: SCB0_SP7B_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SP7B Write Quality of Service Register

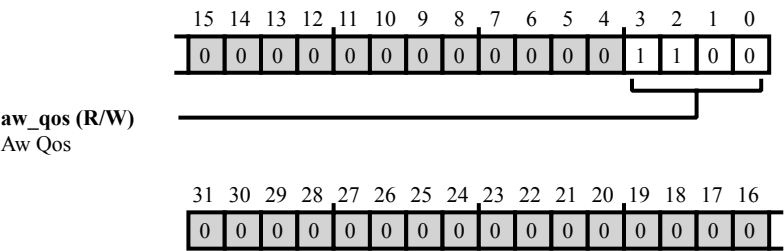


Figure 42-133: SCB0_SP7B_WRITE_QOS Register Diagram

Table 42-139: SCB0_SP7B_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPI0 RX Read Quality of Service Register

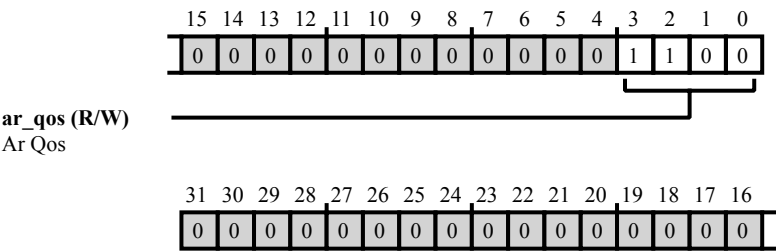


Figure 42-134: SCB0_SPI0RX_READ_QOS Register Diagram

Table 42-140: SCB0_SPI0RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPI0 RX Write Quality of Service Register

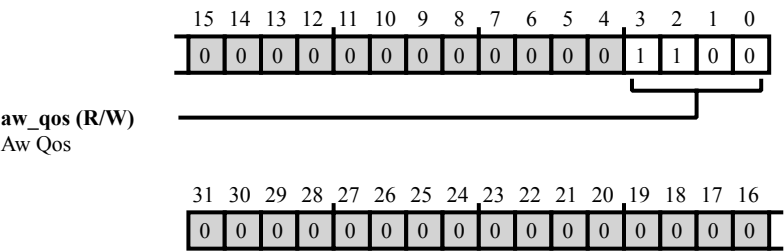


Figure 42-135: SCB0_SPI0RX_WRITE_QOS Register Diagram

Table 42-141: SCB0_SPI0RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPI0 TX Read Quality of Service Register

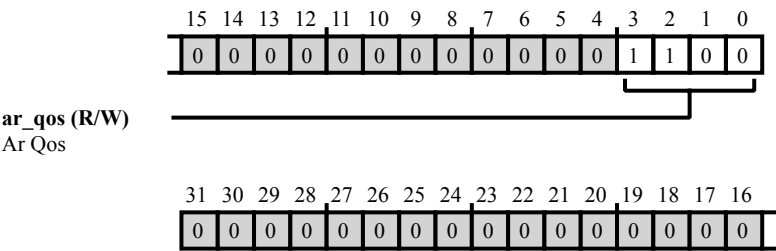


Figure 42-136: SCB0_SPI0TX_READ_QOS Register Diagram

Table 42-142: SCB0_SPI0TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPI0 TX Write Quality of Service Register

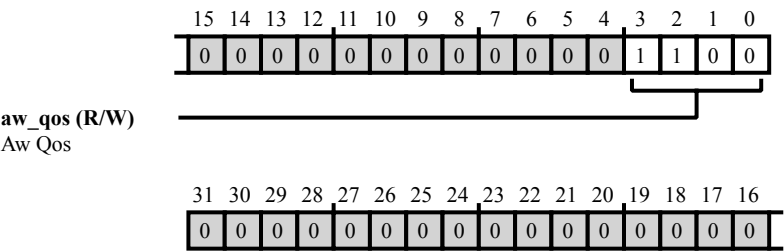


Figure 42-137: SCB0_SPI0TX_WRITE_QOS Register Diagram

Table 42-143: SCB0_SPI0TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPI1 RX Read Quality of Service Register

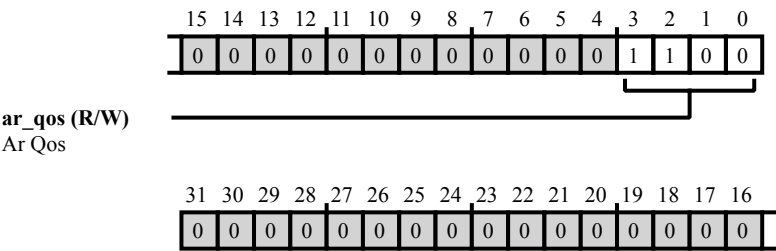


Figure 42-138: SCB0_SPI1RX_READ_QOS Register Diagram

Table 42-144: SCB0_SPI1RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPI1 RX Write Quality of Service Register

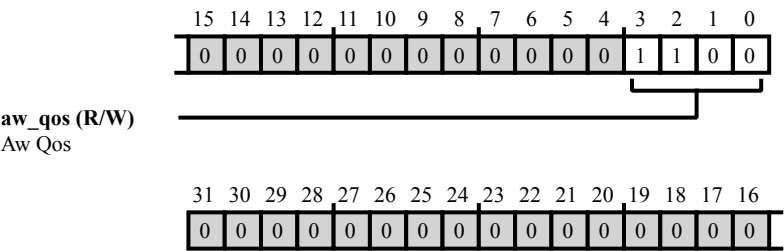


Figure 42-139: SCB0_SPI1RX_WRITE_QOS Register Diagram

Table 42-145: SCB0_SPI1RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPI1 TX Read Quality of Service Register

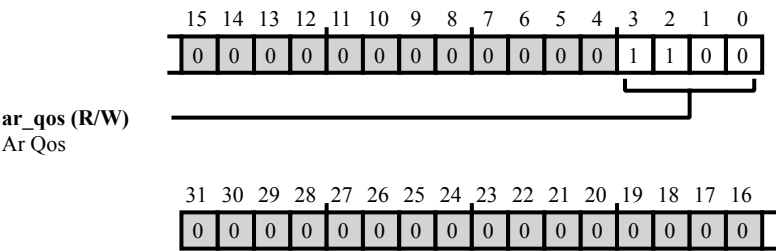


Figure 42-140: SCB0_SPI1TX_READ_QOS Register Diagram

Table 42-146: SCB0_SPI1TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPI1 TX Write Quality of Service Register

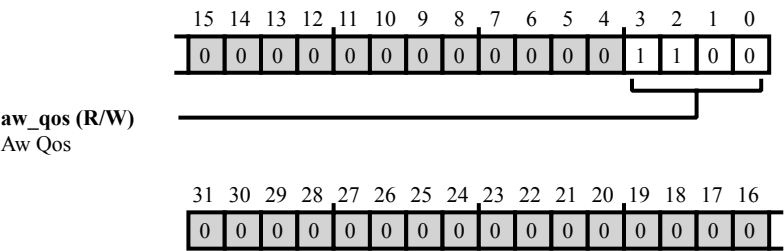


Figure 42-141: SCB0_SPI1TX_WRITE_QOS Register Diagram

Table 42-147: SCB0_SPI1TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPI2 RX Read Quality of Service Register

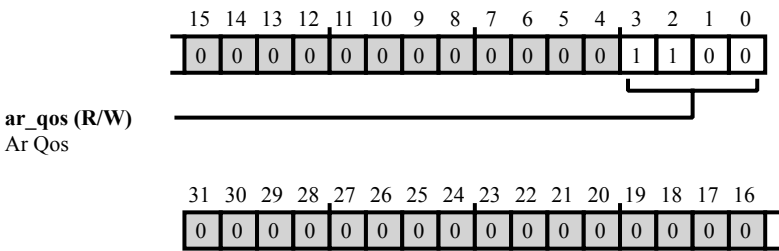


Figure 42-142: SCB0_SPI2RX_READ_QOS Register Diagram

Table 42-148: SCB0_SPI2RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPI2 RX Write Quality of Service Register

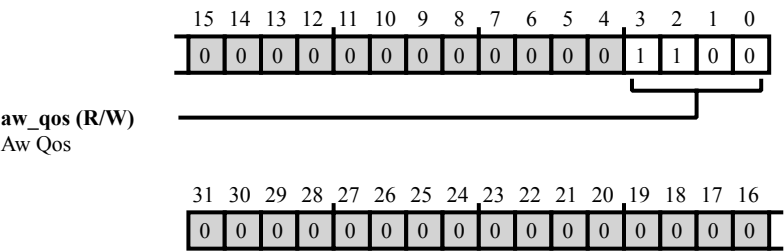


Figure 42-143: SCB0_SPI2RX_WRITE_QOS Register Diagram

Table 42-149: SCB0_SPI2RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPI2 TX Read Quality of Service Register

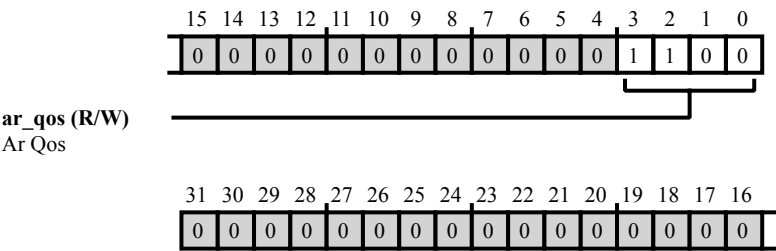


Figure 42-144: SCB0_SPI2TX_READ_QOS Register Diagram

Table 42-150: SCB0_SPI2TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPI2 TX Write Quality of Service Register

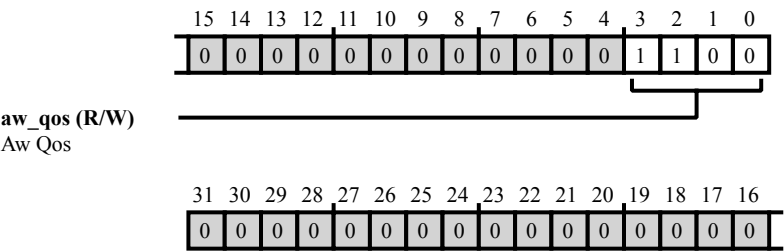


Figure 42-145: SCB0_SPI2TX_WRITE_QOS Register Diagram

Table 42-151: SCB0_SPI2TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPI3 RX Read Quality of Service Register

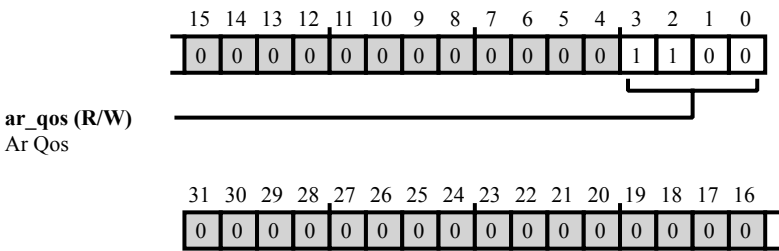


Figure 42-146: SCB0_SPI3RX_READ_QOS Register Diagram

Table 42-152: SCB0_SPI3RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPI3 RX Write Quality of Service Register

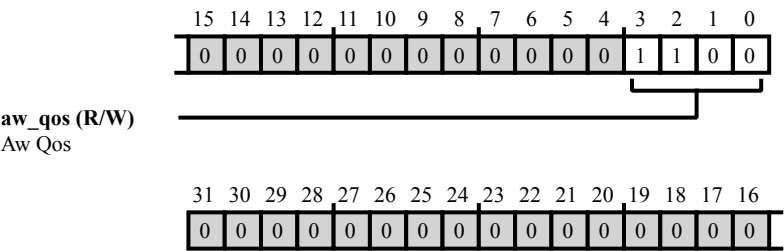


Figure 42-147: SCB0_SPI3RX_WRITE_QOS Register Diagram

Table 42-153: SCB0_SPI3RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

SPI3 TX Read Quality of Service Register

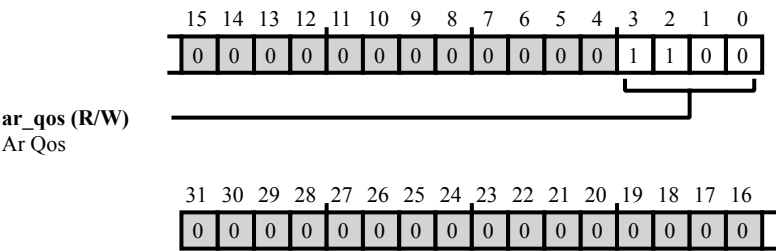


Figure 42-148: SCB0_SPI3TX_READ_QOS Register Diagram

Table 42-154: SCB0_SPI3TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

SPI3 TX Write Quality of Service Register

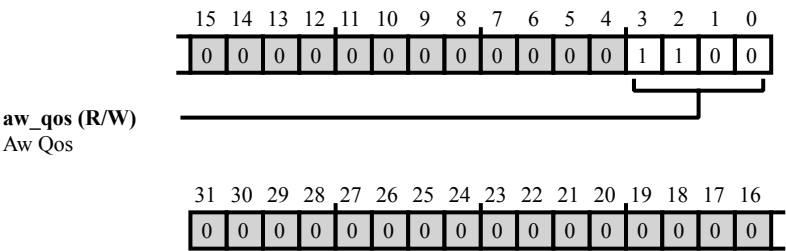


Figure 42-149: SCB0_SPI3TX_WRITE_QOS Register Diagram

Table 42-155: SCB0_SPI3TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

UART0 RX Read Quality of Service Register

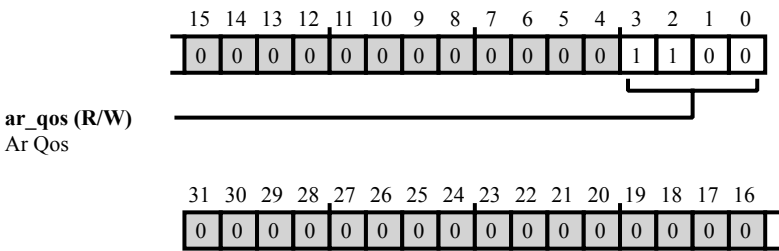


Figure 42-150: SCB0_UART0_RX_READ_QOS Register Diagram

Table 42-156: SCB0_UART0_RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

UART0 RX Write Quality of Service Register

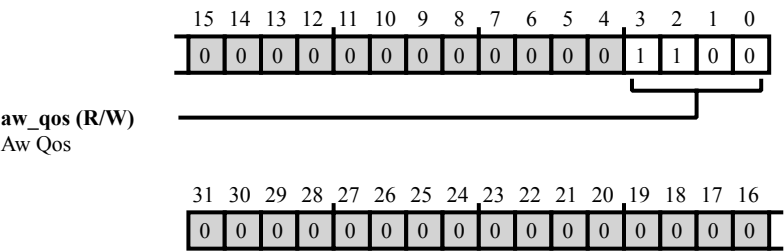


Figure 42-151: SCB0_UART0_RX_WRITE_QOS Register Diagram

Table 42-157: SCB0_UART0_RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

UART0 TX Read Quality of Service Register

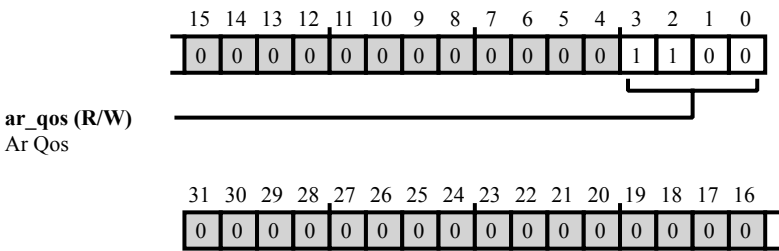


Figure 42-152: SCB0_UART0_TX_READ_QOS Register Diagram

Table 42-158: SCB0_UART0_TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

UART0 TX Write Quality of Service Register

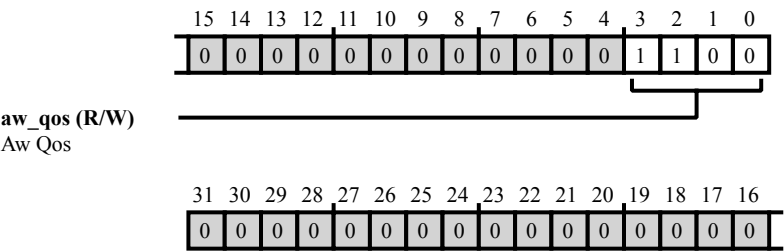


Figure 42-153: SCB0_UART0_TX_WRITE_QOS Register Diagram

Table 42-159: SCB0_UART0_TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

UART1 RX Read Quality of Service Register

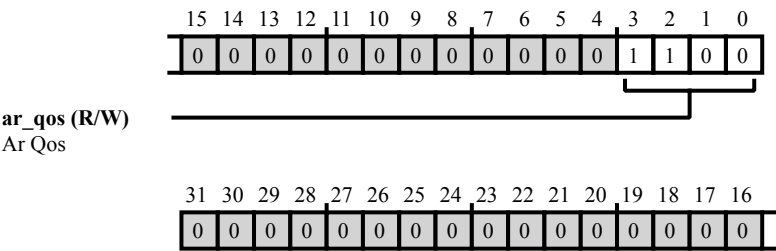


Figure 42-154: SCB0_UART1_RX_READ_QOS Register Diagram

Table 42-160: SCB0_UART1_RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

UART1 RX Write Quality of Service Register

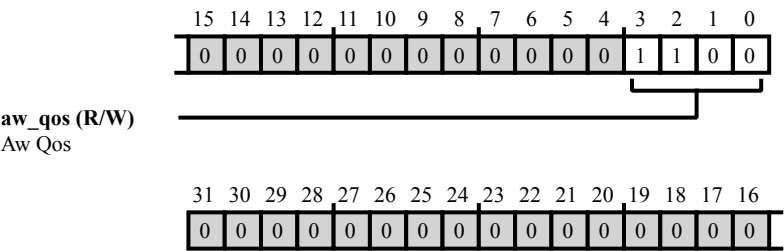


Figure 42-155: SCB0_UART1_RX_WRITE_QOS Register Diagram

Table 42-161: SCB0_UART1_RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

UART1 TX Read Quality of Service Register

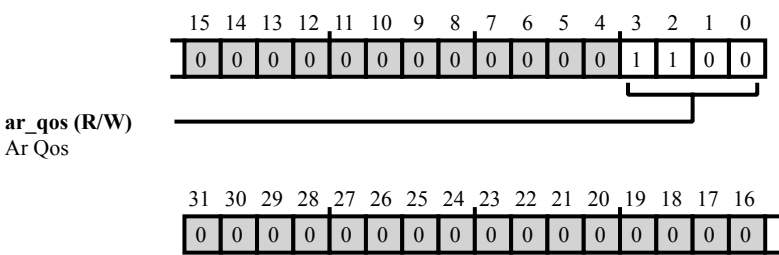


Figure 42-156: SCB0_UART1_TX_READ_QOS Register Diagram

Table 42-162: SCB0_UART1_TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

UART1 TX Write Quality of Service Registers

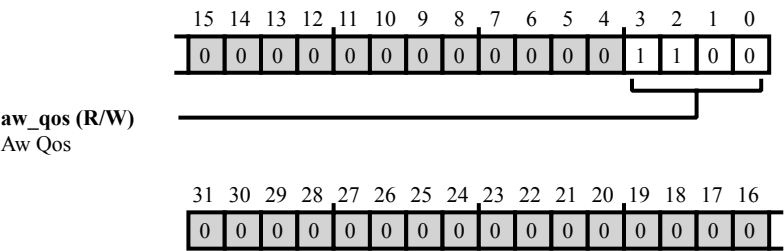


Figure 42-157: SCB0_UART1_TX_WRITE_QOS Register Diagram

Table 42-163: SCB0_UART1_TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

UART2 RX Read Quality of Service Register

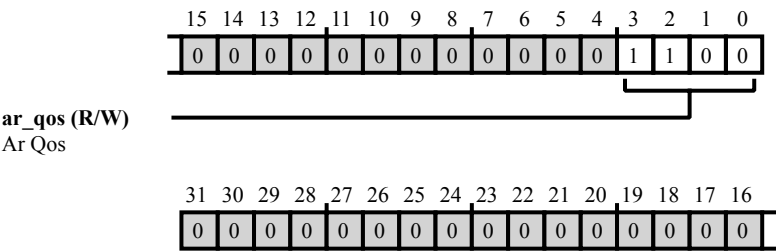


Figure 42-158: SCB0_UART2_RX_READ_QOS Register Diagram

Table 42-164: SCB0_UART2_RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

UART2 RX Write Quality of Service Register

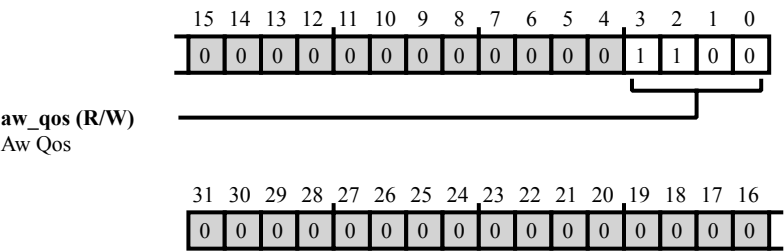


Figure 42-159: SCB0_UART2_RX_WRITE_QOS Register Diagram

Table 42-165: SCB0_UART2_RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

UART2 TX Read Quality of Service Register

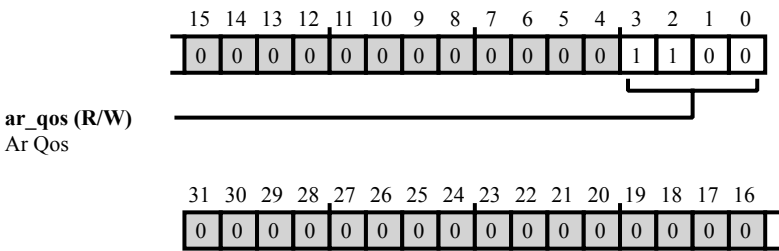


Figure 42-160: SCB0_UART2_TX_READ_QOS Register Diagram

Table 42-166: SCB0_UART2_TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

UART2 TX Write Quality of Service Register

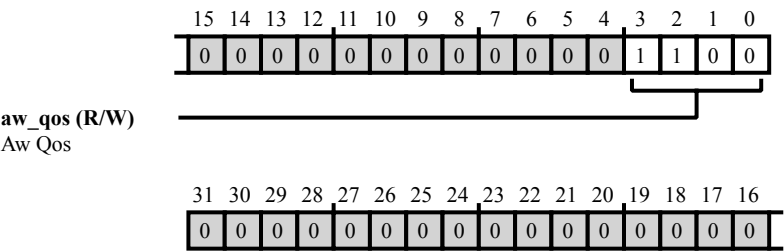


Figure 42-161: SCB0_UART2_TX_WRITE_QOS Register Diagram

Table 42-167: SCB0_UART2_TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

UART3 RX Read Quality of Service Register

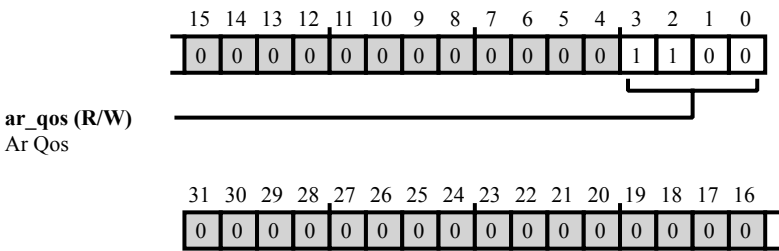


Figure 42-162: SCB0_UART3_RX_READ_QOS Register Diagram

Table 42-168: SCB0_UART3_RX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

UART3 RX Write Quality of Service Register

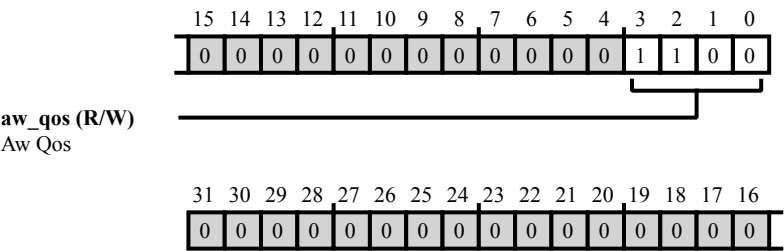


Figure 42-163: SCB0_UART3_RX_WRITE_QOS Register Diagram

Table 42-169: SCB0_UART3_RX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

UART3 TX Read Quality of Service Register

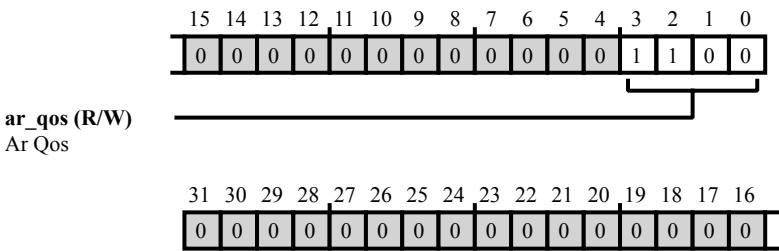


Figure 42-164: SCB0_UART3_TX_READ_QOS Register Diagram

Table 42-170: SCB0_UART3_TX_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

UART3 TX Write Quality of Service Register

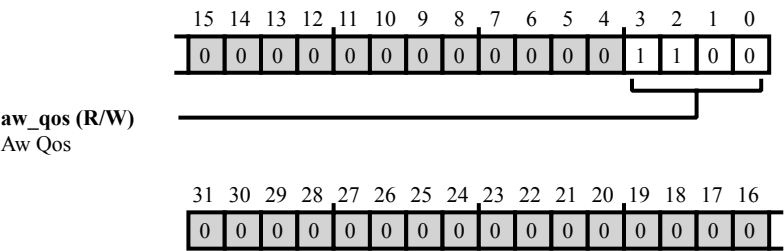


Figure 42-165: SCB0_UART3_TX_WRITE_QOS Register Diagram

Table 42-171: SCB0_UART3_TX_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

USB0 Read Quality of Service Register



Figure 42-166: SCB0_USB0_READ_QOS Register Diagram

Table 42-172: SCB0_USB0_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

USB0 Write Quality of Service Register



Figure 42-167: SCB0_USB0_WRITE_QOS Register Diagram

Table 42-173: SCB0_USB0_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

ADSP-2159x SCB1 Register Descriptions

(SCB1) contains the following registers.

Table 42-174: ADSP-2159x SCB1 Register List

Name	Description
SCB1_DMC_IB_SYNC_MODE	DMC Fabric (CLK03) Synchronization Mode Register

DMC Fabric (CLK03) Synchronization Mode Register



Figure 42-168: SCB1_DMC_IB_SYNC_MODE Register Diagram

Table 42-175: SCB1_DMC_IB_SYNC_MODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	SYNC_MODE	Sync Mode. Default is ASYNC('h4) other valid option is M:N('h3)..

ADSP-2159x SCB3 Register Descriptions

(SCB3) contains the following registers.

Table 42-176: ADSP-2159x SCB3 Register List

Name	Description
SCB3_DMC_MMR_IB_SYNC_MODE	DMC MMRG Fabric (CLK03) Synchronization Mode Register
SCB3_CAN_MMR_IB_SYNC_MODE	DMC MMRG Fabric (CLK04) Synchronization Mode Register
SCB3_LP_MMR_IB_SYNC_MODE	DMC MMRG Fabric (CLK08) Synchronization Mode Register

DMC MMRG Fabric (CLK03) Synchronization Mode Register



Figure 42-169: SCB3_DMC_MMR_IB_SYNC_MODE Register Diagram

Table 42-177: SCB3_DMC_MMR_IB_SYNC_MODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	SYNC_MODE	Sync Mode.

DMC MMRG Fabric (CLK04) Synchronization Mode Register



Figure 42-170: SCB3_CAN_MMR_IB_SYNC_MODE Register Diagram

Table 42-178: SCB3_CAN_MMR_IB_SYNC_MODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	SYNC_MODE	Sync Mode.

DMC MMRG Fabric (CLK08) Synchronization Mode Register



Figure 42-171: SCB3_LP_MMR_IB_SYNC_MODE Register Diagram

Table 42-179: SCB3_LP_MMR_IB_SYNC_MODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
2:0 (R/W)	SYNC_MODE	Sync Mode. Default is ASYNC('h4) other valid option is M:N('h3)..

ADSP-2159x SCB4 Register Descriptions

(SCB4) contains the following registers.

Table 42-180: ADSP-2159x SCB4 Register List

Name	Description
SCB4_FABRIC_ACC_MMR_IB_READ_QOS	Fabric Acc Mmr Ib.read Qos
SCB4_FABRIC_ACC_MMR_IB_WRITE_QOS	Fabric Acc Mmr Ib.write Qos
SCB4_FABRIC_S1PORT_IB_READ_QOS	Fabric S1port Ib.read Qos
SCB4_FABRIC_S1PORT_IB_WRITE_QOS	Fabric S1port Ib.write Qos
SCB4_FABRIC_S2PORT_IB_READ_QOS	Fabric S2port Ib.read Qos
SCB4_FABRIC_S2PORT_IB_WRITE_QOS	Fabric S2port Ib.write Qos
SCB4_FIR_CH0_IB_READ_QOS	Fir Ch0 Ib.read Qos
SCB4_FIR_CH0_IB_WRITE_QOS	Fir Ch0 Ib.write Qos
SCB4_FIR_CH1_IB_READ_QOS	Fir Ch1 Ib.read Qos
SCB4_FIR_CH1_IB_WRITE_QOS	Fir Ch1 Ib.write Qos
SCB4_IIR0_CH0_IB_READ_QOS	Iir0 Ch0 Ib.read Qos
SCB4_IIR0_CH0_IB_WRITE_QOS	Iir0 Ch0 Ib.write Qos
SCB4_IIR0_CH1_IB_READ_QOS	Iir0 Ch1 Ib.read Qos
SCB4_IIR0_CH1_IB_WRITE_QOS	Iir0 Ch1 Ib.write Qos
SCB4_IIR1_CH0_IB_READ_QOS	Iir1 Ch0 Ib.read Qos
SCB4_IIR1_CH0_IB_WRITE_QOS	Iir1 Ch0 Ib.write Qos
SCB4_IIR1_CH1_IB_READ_QOS	Iir1 Ch1 Ib.read Qos
SCB4_IIR1_CH1_IB_WRITE_QOS	Iir1 Ch1 Ib.write Qos

Table 42-180: ADSP-2159x SCB4 Register List (Continued)

Name	Description
SCB4_IIR2_CH0_IB_READ_QOS	Iir2 Ch0 Ib.read Qos
SCB4_IIR2_CH0_IB_WRITE_QOS	Iir2 Ch0 Ib.write Qos
SCB4_IIR2_CH1_IB_READ_QOS	Iir2 Ch1 Ib.read Qos
SCB4_IIR2_CH1_IB_WRITE_QOS	Iir2 Ch1 Ib.write Qos
SCB4_IIR3_CH0_IB_READ_QOS	Iir3 Ch0 Ib.read Qos
SCB4_IIR3_CH0_IB_WRITE_QOS	Iir3 Ch0 Ib.write Qos
SCB4_IIR3_CH1_IB_READ_QOS	Iir3 Ch1 Ib.read Qos
SCB4_IIR3_CH1_IB_WRITE_QOS	Iir3 Ch1 Ib.write Qos
SCB4_SHARC_DPORT_READ_QOS	Sharc Dport.read Qos
SCB4_SHARC_DPORT_WRITE_QOS	Sharc Dport.write Qos

Fabric Acc Mmr Ib.read Qos

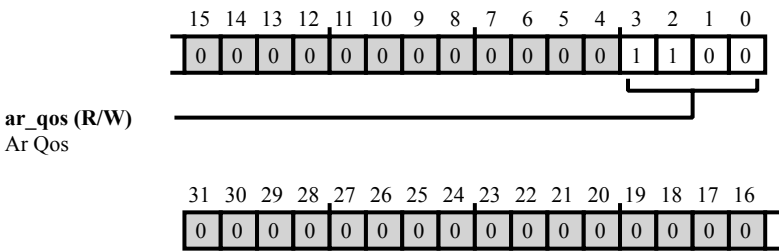


Figure 42-172: SCB4_FABRIC_ACC_MMR_IB_READ_QOS Register Diagram

Table 42-181: SCB4_FABRIC_ACC_MMR_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Fabric Acc Mmr Ib.write Qos

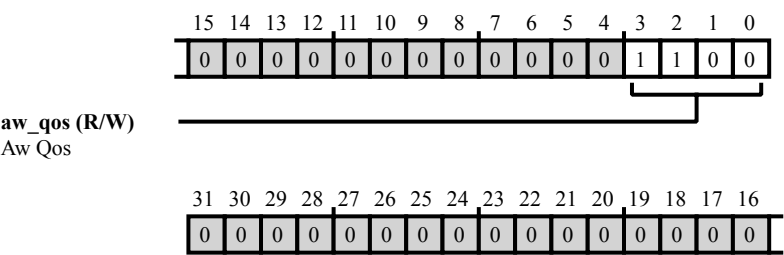


Figure 42-173: SCB4_FABRIC_ACC_MMR_IB_WRITE_QOS Register Diagram

Table 42-182: SCB4_FABRIC_ACC_MMR_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

Fabric S1port Ib.read Qos

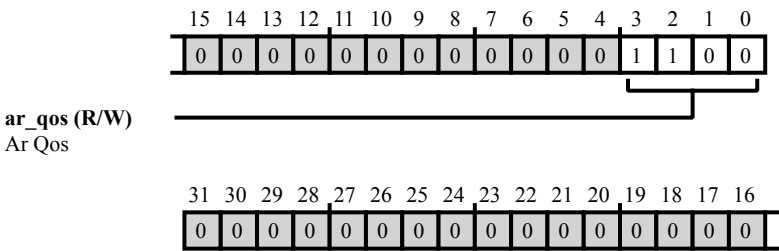


Figure 42-174: SCB4_FABRIC_S1PORT_IB_READ_QOS Register Diagram

Table 42-183: SCB4_FABRIC_S1PORT_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Fabric S1port Ib.write Qos

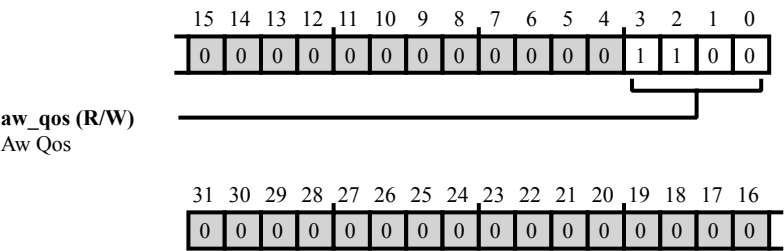


Figure 42-175: SCB4_FABRIC_S1PORT_IB_WRITE_QOS Register Diagram

Table 42-184: SCB4_FABRIC_S1PORT_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

Fabric S2port Ib.read Qos

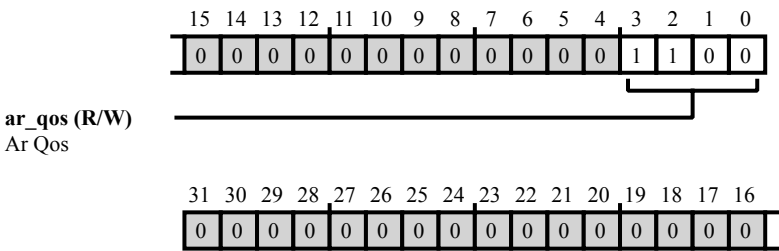


Figure 42-176: SCB4_FABRIC_S2PORT_IB_READ_QOS Register Diagram

Table 42-185: SCB4_FABRIC_S2PORT_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Fabric S2port Ib.write Qos

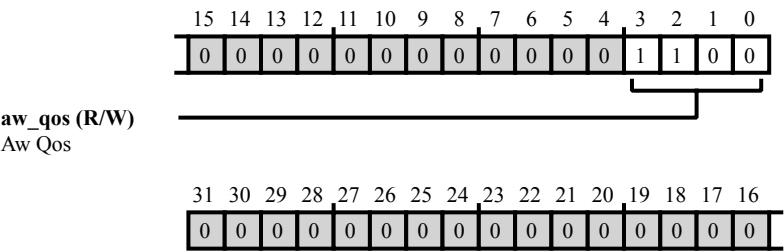


Figure 42-177: SCB4_FABRIC_S2PORT_IB_WRITE_QOS Register Diagram

Table 42-186: SCB4_FABRIC_S2PORT_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

Fir Ch0 Ib.read Qos

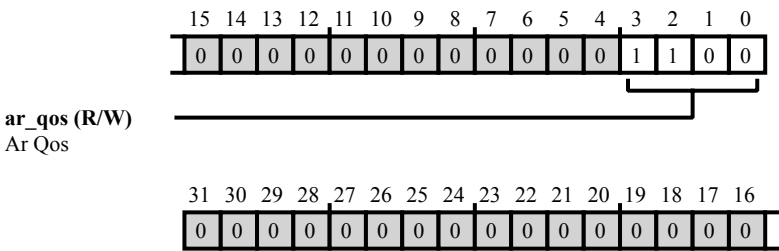


Figure 42-178: SCB4_FIR_CH0_IB_READ_QOS Register Diagram

Table 42-187: SCB4_FIR_CH0_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Fir Ch0 Ib.write Qos

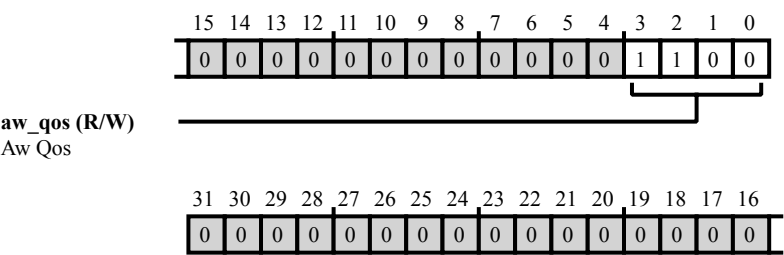


Figure 42-179: SCB4_FIR_CH0_IB_WRITE_QOS Register Diagram

Table 42-188: SCB4_FIR_CH0_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

Fir Ch1 Ib.read Qos

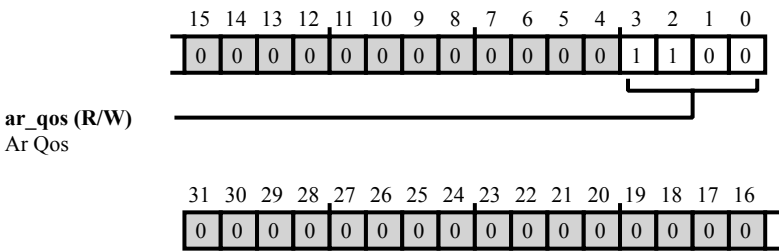


Figure 42-180: SCB4_FIR_CH1_IB_READ_QOS Register Diagram

Table 42-189: SCB4_FIR_CH1_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Fir Ch1 Ib.write Qos

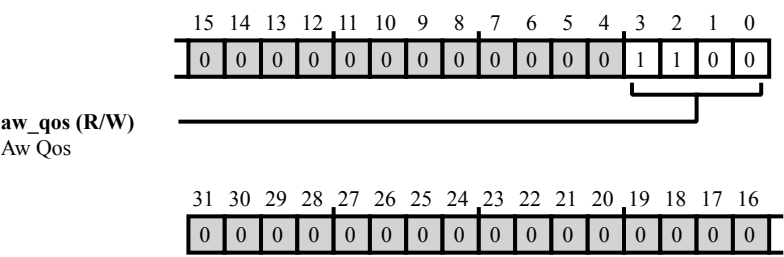


Figure 42-181: SCB4_FIR_CH1_IB_WRITE_QOS Register Diagram

Table 42-190: SCB4_FIR_CH1_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir0 Ch0 Ib.read Qos

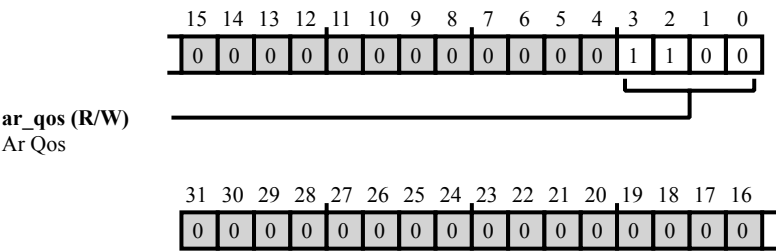


Figure 42-182: SCB4_IIR0_CH0_IB_READ_QOS Register Diagram

Table 42-191: SCB4_IIR0_CH0_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir0 Ch0 Ib.write Qos

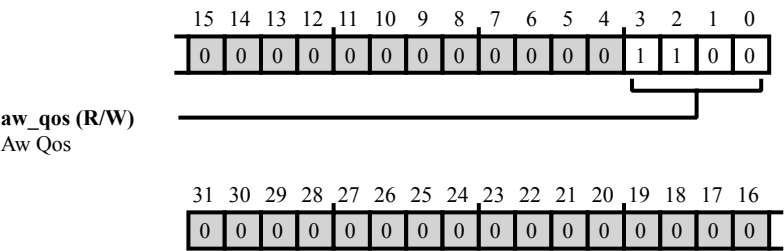


Figure 42-183: SCB4_IIR0_CH0_IB_WRITE_QOS Register Diagram

Table 42-192: SCB4_IIR0_CH0_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir0 Ch1 Ib.read Qos

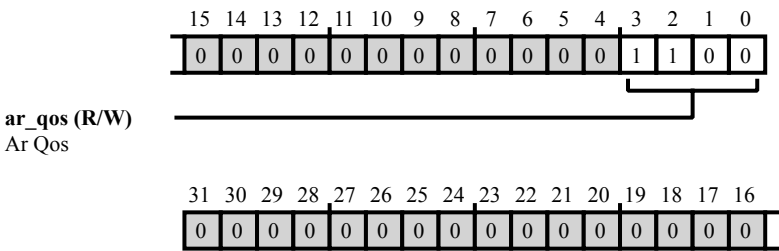


Figure 42-184: SCB4_IIR0_CH1_IB_READ_QOS Register Diagram

Table 42-193: SCB4_IIR0_CH1_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir0 Ch1 Ib.write Qos

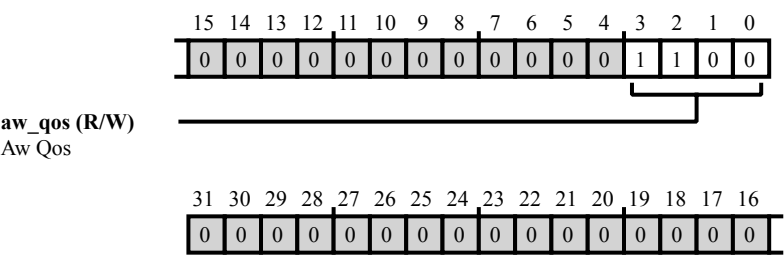


Figure 42-185: SCB4_IIR0_CH1_IB_WRITE_QOS Register Diagram

Table 42-194: SCB4_IIR0_CH1_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir1 Ch0 Ib.read Qos

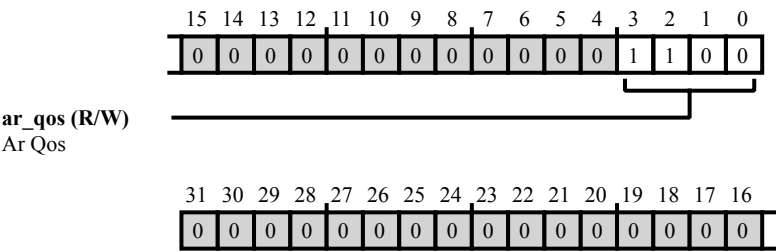


Figure 42-186: SCB4_IIR1_CH0_IB_READ_QOS Register Diagram

Table 42-195: SCB4_IIR1_CH0_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir1 Ch0 Ib.write Qos

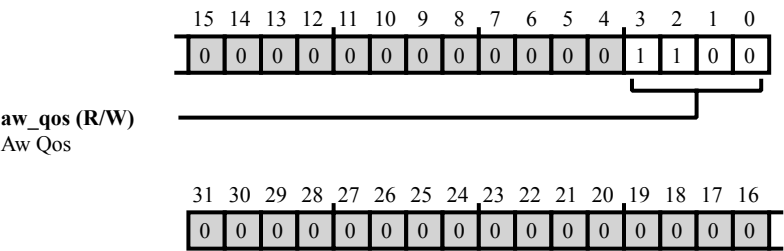


Figure 42-187: SCB4_IIR1_CH0_IB_WRITE_QOS Register Diagram

Table 42-196: SCB4_IIR1_CH0_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir1 Ch1 Ib.read Qos

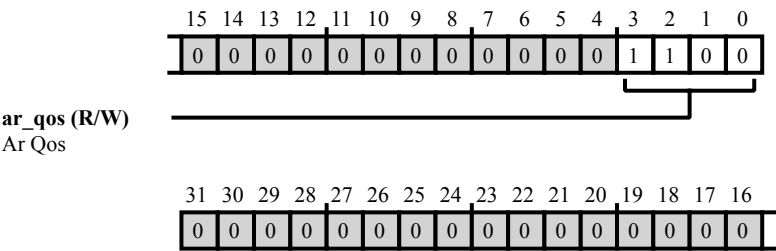


Figure 42-188: SCB4_IIR1_CH1_IB_READ_QOS Register Diagram

Table 42-197: SCB4_IIR1_CH1_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir1 Ch1 Ib.write Qos

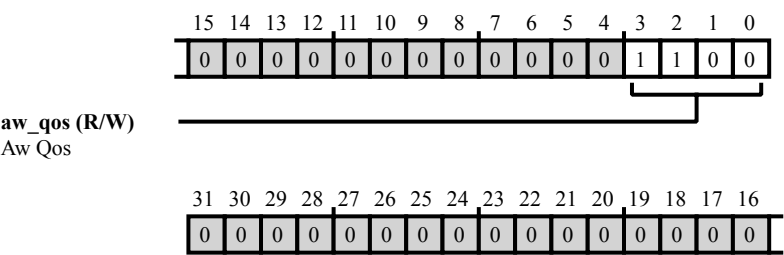


Figure 42-189: SCB4_IIR1_CH1_IB_WRITE_QOS Register Diagram

Table 42-198: SCB4_IIR1_CH1_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir2 Ch0 Ib.read Qos

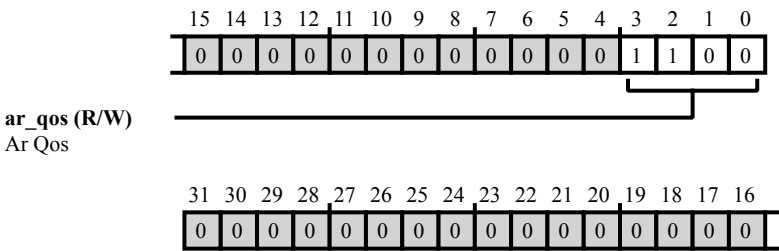


Figure 42-190: SCB4_IIR2_CH0_IB_READ_QOS Register Diagram

Table 42-199: SCB4_IIR2_CH0_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir2 Ch0 Ib.write Qos

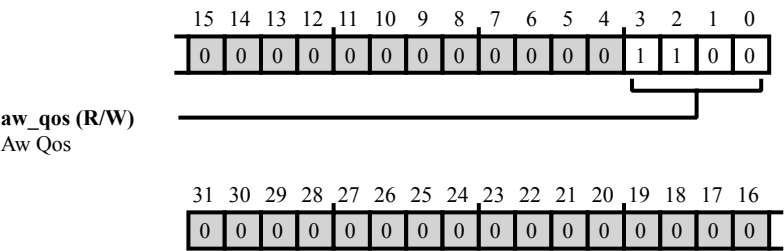


Figure 42-191: SCB4_IIR2_CH0_IB_WRITE_QOS Register Diagram

Table 42-200: SCB4_IIR2_CH0_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir2 Ch1 Ib.read Qos

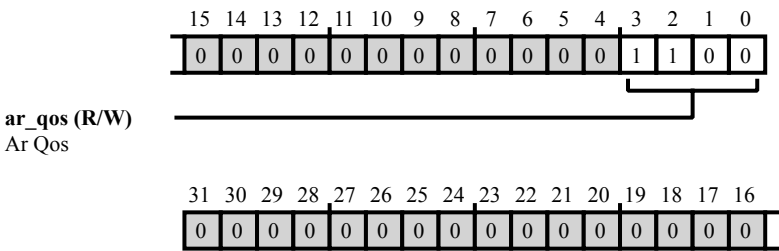


Figure 42-192: SCB4_IIR2_CH1_IB_READ_QOS Register Diagram

Table 42-201: SCB4_IIR2_CH1_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir2 Ch1 Ib.write Qos

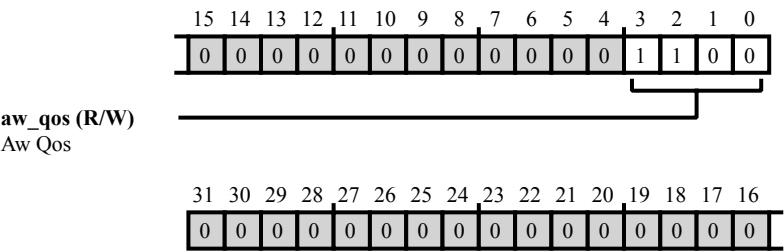


Figure 42-193: SCB4_IIR2_CH1_IB_WRITE_QOS Register Diagram

Table 42-202: SCB4_IIR2_CH1_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir3 Ch0 Ib.read Qos

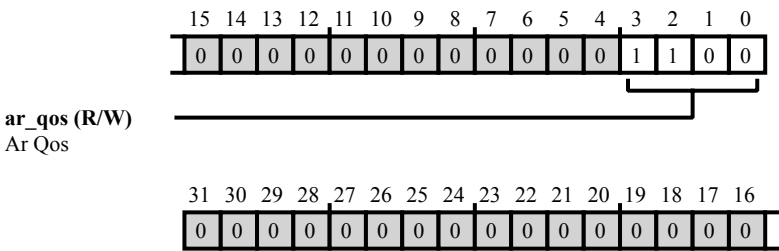


Figure 42-194: SCB4_IIR3_CH0_IB_READ_QOS Register Diagram

Table 42-203: SCB4_IIR3_CH0_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir3 Ch0 Ib.write Qos

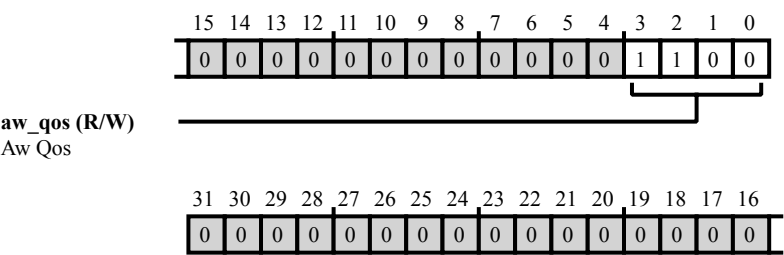


Figure 42-195: SCB4_IIR3_CH0_IB_WRITE_QOS Register Diagram

Table 42-204: SCB4_IIR3_CH0_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

lir3 Ch1 Ib.read Qos

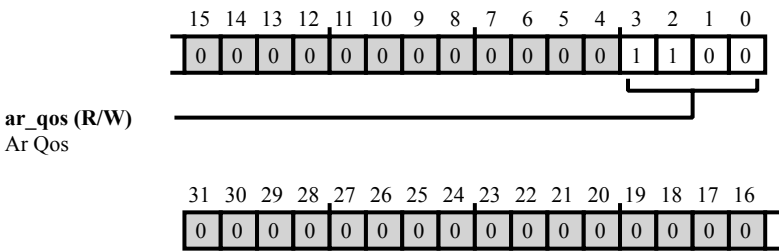


Figure 42-196: SCB4_IIR3_CH1_IB_READ_QOS Register Diagram

Table 42-205: SCB4_IIR3_CH1_IB_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

lir3 Ch1 Ib.write Qos

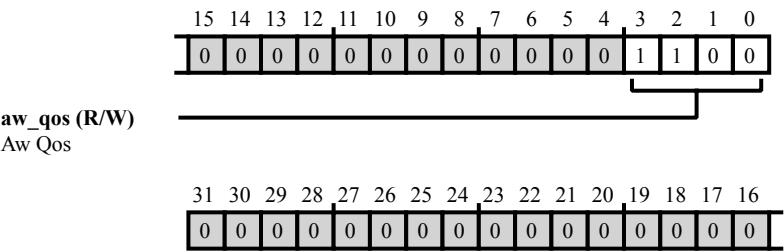


Figure 42-197: SCB4_IIR3_CH1_IB_WRITE_QOS Register Diagram

Table 42-206: SCB4_IIR3_CH1_IB_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

Sharc Dport.read Qos

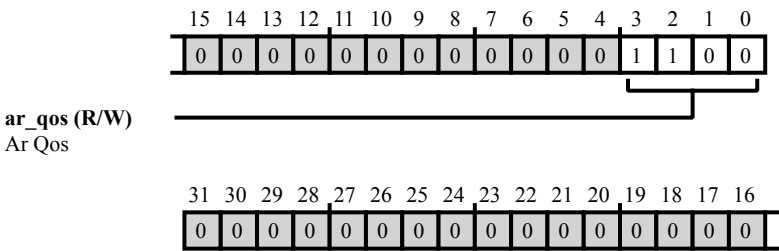


Figure 42-198: SCB4_SHARC_DPORT_READ_QOS Register Diagram

Table 42-207: SCB4_SHARC_DPORT_READ_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AR_QOS	Ar Qos.

Sharc Dport.write Qos

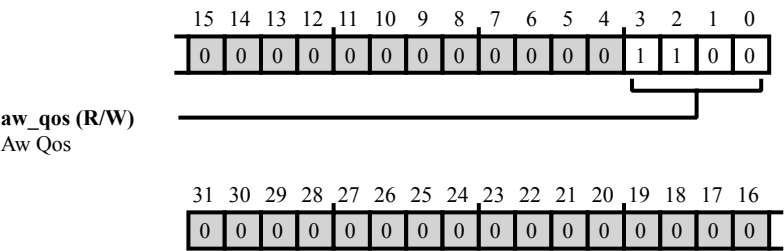


Figure 42-199: SCB4_SHARC_DPORT_WRITE_QOS Register Diagram

Table 42-208: SCB4_SHARC_DPORT_WRITE_QOS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W)	AW_QOS	Aw Qos.

ADSP-2159x SCB5 Register Descriptions

System Crossbars (SCB5) contains the following registers.

Table 42-209: ADSP-2159x SCB5 Register List

Name	Description
SCB5_SPI2_OSPI_REMAP	SPI2/OSPI Memory Map Address Remap Register

SPI2/OSPI Memory Map Address Remap Register



Figure 42-200: SCB5_SPI2_OSPI_REMAP Register Diagram

Table 42-210: SCB5_SPI2_OSPI_REMAP Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
1:0 (R/W)	REMAP	.	
		0	Selects map for SPI2 only 0x6000_0000 - 0x7FFF_FFFF - SPIF
		1	Selects map for OSPI only 0x6000_0000 - 0x7FFF_FFFF - OSPI
		2	Selects map for SPI2/OSPI split sharing 0x6000_0000 - 602F_FFFF - SPIF 0x6030_0000 - 0x7FFF_FFFF - OSPI

43 System Watchpoint Unit (SWU)

The system watchpoint unit (SWU) is a single module used for transaction monitoring. The SWU is attached to each system slave through the system crossbar interface and provides ports for all address channel signals for the system crossbar. The SWU does not have ports for the read/write data channel signals or the low-power interface signals.

Each SWU contains four match groups of registers with associated hardware. These four SWU match groups operate independently, but share common event (interrupt and trigger) outputs. Each match group can monitor either the write or read address channel and can operate in either watchpoint mode or bandwidth mode.

In this chapter, the naming convention for registers omit the numeric reference to any of the SWUs or it's groups. For example, SWU_GCTL represents registers SWU1_GCTL, SWU2_GCTL, and so on. Similarly, SWU_CUR[n] represents SWU1_CUR0/CUR1/CUR2/CUR3, SWU2_CUR0/CUR1/CUR2/CUR3 and so on.

SWU Features

The system watchpoint unit has the following features.

- Four independent match groups for each SWU
- Each match group can operate in either bandwidth mode or watchpoint mode

SWU Functional Description

This section describes the function of the SWU match block, interface block, and MMR block.

NOTE: In the *SWU Interrupt List* table, there are three SWUs, SWU1 (CL2_0), SWU3 (CL2_1) and SWU5 (CL2_2) for L2 Memory Core access and two SWUs, SWU2 (DL2_0) and SWU4 (DL2_1) for L2 Memory DMA access. CL2_2, DL2_0 and DL2_1 have access to the entire L2 range. CL2_0 has access to first 1MB and CL2_1 has access to second 1MB of L2.

ADSP-2159x SWU Register List

The System Watchpoint Unit (SWU) provides debug and development support through flexible transaction level and bandwidth monitoring and associated event triggering. The SWU can generate events based on monitoring transactions at the system slaves through watchpoint-match groups. The SWU also provides watchpoint event status

reporting, a global lock, and processor reset capability. A set of registers governs SWU operations. For more information on SWU functionality, see the SWU register descriptions.

Table 43-1: ADSP-2159x SWU Register List

Name	Description
SWU_CNT[n]	Count Register n
SWU_CTL[n]	Control Register n
SWU_CUR[n]	Current Register n
SWU_GCTL	Global Control Register
SWU_GSTAT	Global Status Register
SWU_HIST[n]	Bandwidth History Register n
SWU_ID[n]	ID Register n
SWU_LA[n]	Lower Address Register n
SWU_TARG[n]	Target Register n
SWU_UA[n]	Upper Address Register n

ADSP-2159x SWU Interrupt List

Table 43-2: ADSP-2159x SWU Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
313	SWU0_EVT	SWU0 Event SMC	None	
314	SWU1_EVT	SWU1 Event L2 Memory Core Port 0	None	
315	SWU2_EVT	SWU2 Event L2 Memory DMA Port 0	None	
316	SWU7_EVT	SWU7 Event SHARC0 Slave Port 1	None	
317	SWU8_EVT	SWU8 Event SHARC0 Slave Port 2	None	
318	SWU9_EVT	SWU9 Event SHARC1 Slave Port 1	None	
319	SWU10_EVT	SWU10 Event SHARC1 Slave Port 2	None	
320	SWU11_EVT	SWU11 Event SMMR	None	
321	SWU12_EVT	SWU12 Event SPI2/OSPI	None	
322	SWU13_EVT	SWU13 Event DMC0	None	
323	SWU3_EVT	SWU3 Event CL2_1	None	
324	SWU4_EVT	SWU4 Event DL2_1	None	
325	SWU5_EVT	SWU5 Event CL2_2	None	

ADSP-2159x SWU Trigger List

Table 43-3: ADSP-2159x SWU Trigger List Masters

Trigger ID	Name	Description	Sensitivity
110	SWU0_EVT	SWU0 Event	None
111	SWU1_EVT	SWU1 Event	None
112	SWU2_EVT	SWU2 Event	None
113	SWU7_EVT	SWU7 Event	None
114	SWU8_EVT	SWU8 Event	None
115	SWU9_EVT	SWU9 Event	None
116	SWU10_EVT	SWU10 Event	None
117	SWU11_EVT	SWU11 Event	None
118	SWU12_EVT	SWU12 Event	None
119	SWU13_EVT	SWU13 Event	None
120	SWU3_EVT	SWU3 Event	None
121	SWU4_EVT	SWU4 Event	None
122	SWU5_EVT	SWU5 Event	None
123	SWU0_DBG	SWU0 Debug	Edge
124	SWU1_DBG	SWU1 Debug	Edge
125	SWU2_DBG	SWU2 Debug	Edge
126	SWU7_DBG	SWU7 Debug	Edge
127	SWU8_DBG	SWU8 Debug	Edge
128	SWU9_DBG	SWU9 Debug	Edge
129	SWU10_DBG	SWU10 Debug	Edge
130	SWU11_DBG	SWU11 Debug	Edge
131	SWU12_DBG	SWU12 Debug	Edge
132	SWU13_DBG	SWU13 Debug	Edge
133	SWU3_DBG	SWU3 Debug	Edge
134	SWU4_DBG	SWU4 Debug	Edge
135	SWU5_DBG	SWU5 Debug	Edge

Table 43-4: ADSP-2159x SWU Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
112	SWU0_EN	SWU0 Enable	Pulse

Table 43-4: ADSP-2159x SWU Trigger List Slaves (Continued)

Trigger ID	Name	Description	Sensitivity
113	SWU1_EN	SWU1 Enable	Pulse
114	SWU2_EN	SWU2 Enable	Pulse
115	SWU7_EN	SWU7 Enable	Pulse
116	SWU8_EN	SWU8 Enable	Pulse
117	SWU9_EN	SWU9 Enable	Pulse
118	SWU10_EN	SWU10 Enable	Pulse
119	SWU11_EN	SWU11 Enable	Pulse
120	SWU12_EN	SWU12 Enable	Pulse
121	SWU13_EN	SWU13 Enable	Pulse
122	SWU3_EN	SWU3 Enable Event CL2_1	Pulse
123	SWU4_EN	SWU4 Enable Event DL2_1	Pulse
124	SWU5_EN	SWU5 Enable Event CL2_2	Pulse

SWU Definitions

The following definitions are helpful when using the SWU module.

Watchpoint Mode

Mode in which transactions are recognized on an exact match. Actions can be configured to be taken after a specified number of matches have occurred.

Bandwidth Mode

Mode in which transactions are recognized and counted inside sampling window.

SWU Architectural Concepts

The information in this section provides basic module design concepts.

SWU-to-SCB Interface

The SWU system crossbar interface block latches all transactions on the system crossbar read and write address channels when the `SWU_GCTL.EN` register enable bit is set.

SWU Block Diagram

The *System Watchpoint Unit Top-Level Block Diagram* figure shows the SWU block diagram.

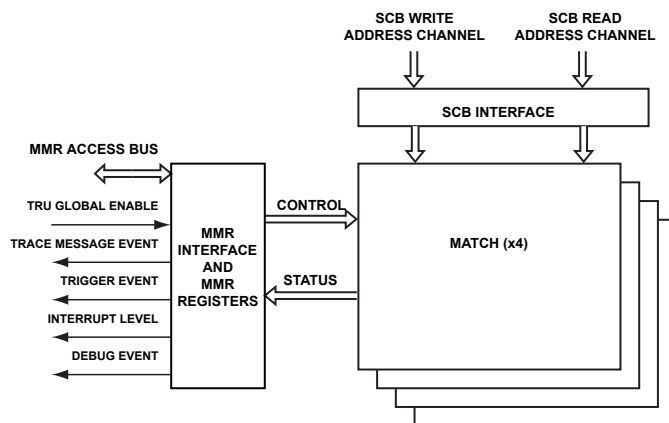


Figure 43-1: System Watchpoint Unit Top-Level Block Diagram

SCB Interface Block

The SWU system crossbar (SCB) latches all transactions on the SCB read and write address channels when the `SWU_GCTL.EN` bit is set.

MMR Interface Block

The SWU MMR block contains the peripheral bus interface and the SWU MMR registers. It also merges all interrupt requests and events from each match block into common outputs.

SWU Operating Modes

There are two operating modes supported by the SWU: bandwidth mode and watchpoint mode.

Bandwidth Mode

In bandwidth mode, the SWU module counts transactions which match the properties specified in the `SWU_CTL[n]` register during a sampling window determined by the respective `SWU_CNT[n]` register. At the end of the sampling window, the SWU stores results in the `SWU_HIST[n]` register. If the sampled bandwidth falls outside a programmed range, then the programmed action occurs.

Watchpoint Mode

In watchpoint mode, if the `SWU_CTL[n].CNTEN` bit is set, the SWU module decrements the `SWU_CUR[n]` register for each match, until it equals zero, at which point any programmed actions occur. The `SWU_CUR[n]` register is then reloaded from the `SWU_CNT[n]` register (if the `SWU_CTL[n].CNTRPTEN` bit is set), and the cycle repeats. If the `SWU_CTL[n].CNTRPTEN` bit is not set, any programmed actions happen on every match.

Match Block

There are four match blocks for each SWU. Each SWU match block can monitor either the read or write address channel, selected by the `SWU_CTL[n].DIR` bit. The SWU match block can operate in either watchpoint or bandwidth mode, as selected by the `SWU_CTL[n].BWEN` bit.

In either mode, the SWU match block can be programmed to match based on address (exact, inclusive or exclusive range), ID (with masking), security, and lock type. All enabled matches are AND'ed together to determine a match.

Scaling

Scaling allows the SWU to count more transactions by scaling the number of transactions and the number of clock cycles in bandwidth window (CNTn register) by 10,100 or 1000. This functionality is applicable only in bandwidth mode (SWU_CTL[n].BWEN==1).

Consider a case where the SWU_TARG[n].BWMAX bit field is programmed to 2. In the absence of any scaling, bandwidth overflow occurs when the SWU_CUR[n].CURBW bit field value > the SWU_TARG[n].BWMAX bit field value (2). With scaling set to 1:100 and after 275 transactions, the SWU_CUR[n].CURBW value is still 2 and equal to SWU_TARG[n].BWMAX. This event triggers a bandwidth overflow as the actual number of transactions is greater than 200 (2×100). The code can be rerun with a smaller scaling selected to analyze the cause for the overflow.

The counter increments after every group of N transactions with scaling enabled, where N is either 10,100 or 1000. (For N = 10, 0–9 transactions == 0 scaled transaction, 10–19 transactions == 1 scaled transaction, 20–29 transactions == 2 scaled transactions, and so on).

Fractional counts with scaling enabled are discarded and not rolled over from one bandwidth window to the next. For example, consider a case where scaling by 1000 is configured and the first window has 1200 transactions. The second window has 2800 transactions. The bandwidth for the first window is read as 1 and the second as 2. The 200 transactions from the first window do not get carried forward to the next window.

Do not use scaling by 10 with the SWU_CTL[n].BLENINC bit enabled if any of the masters accessing the slave can launch a transaction of burst length 16.

SWU Event Control

The SWU can generate the following events when a match occurs and when the event is enabled by configuring the proper bits in the control register.

1. Trace Message
2. Trigger
3. Interrupt request
4. Debug

SWU Interrupts

All interrupt requests and events from each match block are merged into common outputs.

SWU Status and Errors

SWU status and errors are reported in the [SWU_GSTAT](#) register. The SWU records an address error when a write or read attempt is made to the MMR address space of the SWU and the register does not exist. This error is the only one the SWU records. The register contains bits that perform the following functions.

- Indicate whether a particular match group sampled a transaction that is below a minimum target or above a maximum target in bandwidth mode.
- Indicate whether a watchpoint match occurred for each match group.
- Indicate whether an interrupt request was triggered due to a match event from one of the match groups.

Triggers

The SWU can be either a trigger master or a trigger slave depending on the trigger routing unit (TRU) configuration. As a trigger master, programs must set the [SWU_CTL\[n\]](#).TRGEN bit so that when a match condition is met, a trigger event is generated. Each SWU in the system can also be a trigger slave when mapped as one in the TRU.

When the SWU is a slave, a trigger event activates the SWU by automatically setting the [SWU_GCTL.EN](#) bit. Since the SWU can be automatically enabled through a trigger event, programs must pre-configure the SWU before enabling the TRU. Furthermore, although a trigger event can enable the SWU as a slave, to disable the SWU, programs must manually clear the [SWU_GCTL.EN](#) bit.

SWU Programming Model

Program the appropriate registers to use the SWU. Each control register configures aspects such as:

- The direction of monitoring (reads or writes)
- Whether SWU uses bandwidth mode or watchpoint mode
- The setup of events that are triggered when a condition is met while monitoring using the SWU

Configure supplemental registers such as the lower ([SWU_LA\[n\]](#)) and upper ([SWU_UA\[n\]](#)) address boundaries before enabling the SWU.

Once the SWU has been enabled and the monitoring conditions are met, events are generated when configured.

The global status register ([SWU_GSTAT](#)) can be read to observe the status of the units.

The *SWU Logical Flow* diagram shows the logical program flow of the SWU.

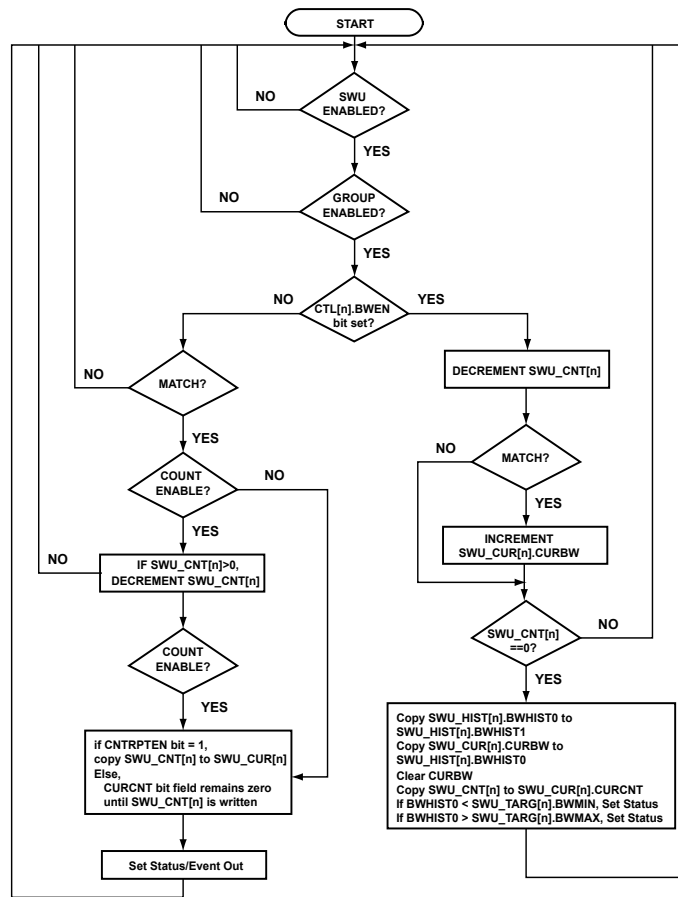


Figure 43-2: SWU Logical Flow

SWU Mode Configuration

The following sections show the steps for configuring SWU bandwidth mode and watchpoint mode.

Configuring the SWU for Bandwidth Mode

In bandwidth mode, the SWU counts transactions which match during a sampling window. At the end of the sampling window, the SWU stores the results. An action can be taken if the sampled bandwidth goes above or falls below a programmed range.

1. Configure the `SWU_CTL[n].DIR` bit to test the match on writes or reads.
2. Configure the `SWU_CTL[n].ACMPM` bits to address comparisons, exact match, matches inside a range or matches outside a range.
3. If ID comparison is desired, set the `SWU_CTL[n].IDCMPEN` bit.
4. Set the `SWU_CTL[n].BLENINC` bit to increment by burst length or clear it to increment by 1.
5. Configure the `SWU_CTL[n].MAXACT` and `SWU_CTL[n].MINACT` bits to enable actions taken when the bandwidth goes above the maximum, or falls below the minimum, respectively.

6. Set the `SWU_CTL[n].BWEN=1` to enable bandwidth mode.
7. Program the lower address register, `SWU_LA[n]`, and upper address register, `SWU_UA[n]`, to define the memory range for comparison.
8. If ID comparison is enabled, program the ID register, `SWU_ID[n]`.
9. Program the count register, `SWU_CNT[n]`, with the number of clock cycles for which the SWU counts the number of matches.
10. If the SWU is set to respond when the bandwidth measurement underflows or overflows, program the min and max values into the `SWU_TARG[n]` register.
11. Enable the SWU

The SWU counts the number of matches in a pre-defined number of clock cycles as programmed. As an option, it can define lower and upper limits. If the matches fall outside the limits, an action can be taken.

Configuring the SWU for Watchpoint Mode

In watchpoint mode, the SWU can trigger a programmed action after every match or after a number of matches. This sequence can be automatically reset.

1. Set the `SWU_CTL[n].DIR` bit to test the match on writes or reads.
2. Configure the `SWU_CTL[n].ACMPM` bits for address comparisons, exact match, matches inside a range or matches outside a range.
3. If ID comparison is desired, set the `SWU_CTL[n].IDCMPEN`.
4. Set the `SWU_CTL[n].CNTEN` bit to enable the events to be triggered when the count decrements to zero.
5. If needed, set the `SWU_CTL[n].CNTRPTEN` bit to automatically reload the counter after it has decremented to zero to start another match sequence.
6. Clear the `SWU_CTL[n].BWEN = 0` to configure watchpoint mode.
7. Configure the lower address register, `SWU_LA[n]`, and upper address register, `SWU_UA[n]`, to define the memory range for comparison.
8. If ID comparison is enabled, configure the ID register, `SWU_ID[n]`.
9. Configure the count register, `SWU_CNT[n]`, to determine how many matches occur before the watchpoint group responds.
10. Enable the SWU.

The SWU detects and counts down the number of match occurrences. When the counter expires, an action is taken.

ADSP-2159x SWU Register Descriptions

System Watchpoint Unit (SWU) contains the following registers.

Table 43-5: ADSP-2159x SWU Register List

Name	Description
SWU_CNT[n]	Count Register n
SWU_CTL[n]	Control Register n
SWU_CUR[n]	Current Register n
SWU_GCTL	Global Control Register
SWU_GSTAT	Global Status Register
SWU_HIST[n]	Bandwidth History Register n
SWU_ID[n]	ID Register n
SWU_LA[n]	Lower Address Register n
SWU_TARG[n]	Target Register n
SWU_UA[n]	Upper Address Register n

Count Register n

The SWU count registers (`SWU_CNT[n]`) contain a 16-bit count field (`SWU_CNT[n].COUNT`) whose usage differs depending on the mode of the watchpoint group. In bandwidth mode, the `SWU_CNT[n].COUNT` field value defines the number of clock cycles in a bandwidth period. In watchpoint mode, when the cycle count is enabled, the `SWU_CNT[n].COUNT` field value determines how many matches occur before the watchpoint group takes action.

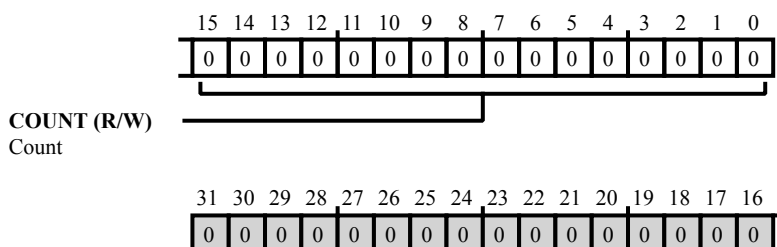


Figure 43-3: `SWU_CNT[n]` Register Diagram

Table 43-6: `SWU_CNT[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	COUNT	Count. The <code>SWU_CNT[n].COUNT</code> field value defines the number of clock cycles in a bandwidth period. In watchpoint mode, when the cycle count is enabled, the <code>SWU_CNT[n].COUNT</code> field value determines how many matches occur before the watchpoint group takes action.

Control Register n

The SWU control registers (`SWU_CTL[n]`) contain watchpoint attribute controls for all four watchpoint groups. These controls include enabling watchpoints, selecting the transaction direction for match, selecting address comparison mode, enabling ID comparison, enabling security comparison, enabling cycle count, enabling count repeat, enabling debug events, enabling interrupts, enabling triggers, enabling trace messages, enabling bandwidth mode, selecting the burst length increment, and enabling bandwidth underflow and overflow detection.

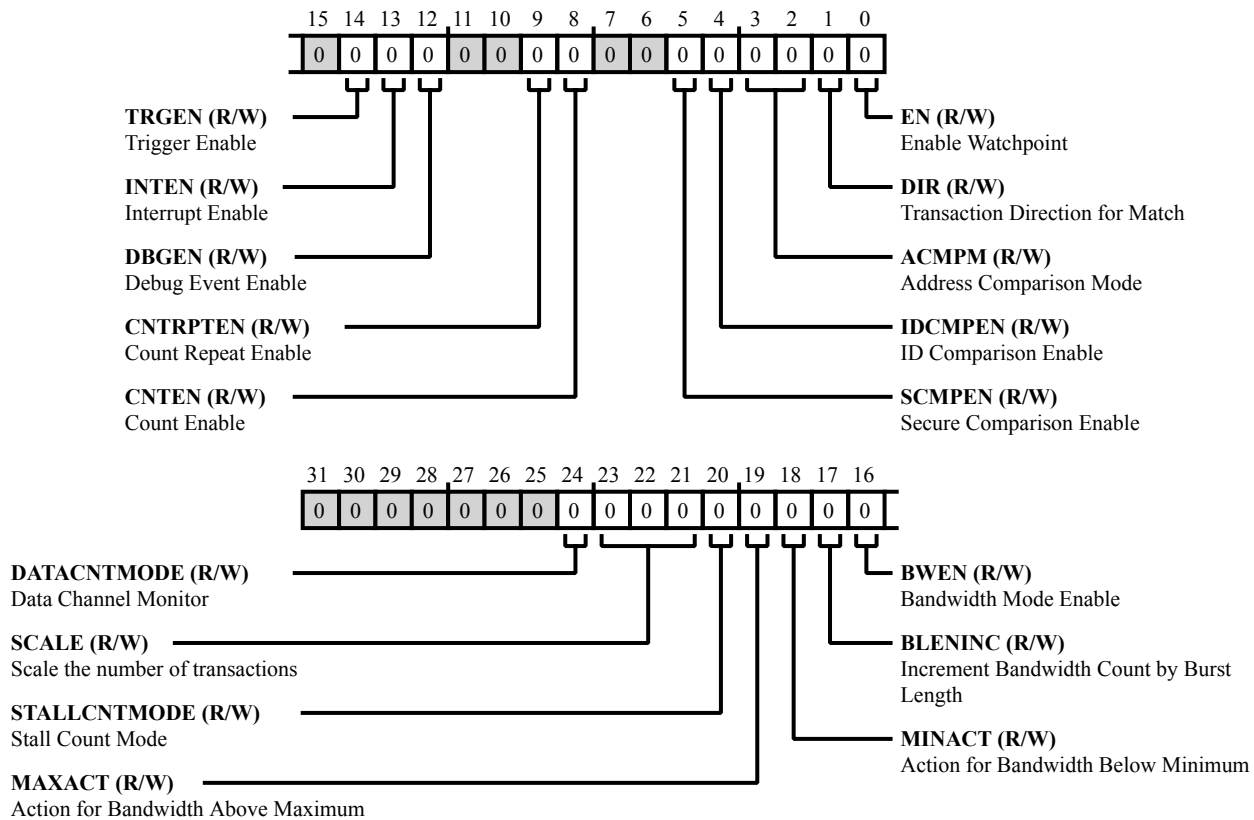


Figure 43-4: `SWU_CTL[n]` Register Diagram

Table 43-7: SWU_CTL[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
24 (R/W)	DATAcntMODE	Data Channel Monitor. The SWU_CTL[n].DATAcntMODE bit determines whether an address channel or a data channel is monitored. Note that in data channel only ID,READY and VALID signals are monitored and hence other comparisons (Address, Lock, Secure) will be ignored even if enabled. SWU_CTL[n].STALLcntMODE and SWU_CTL[n].DIR can be used in conjunction with this bit
		0 Monitor address channel
		1 Monitor data channel
23:21 (R/W)	SCALE	Scale the number of transactions. The SWU_CTL[n].SCALE bit field allows a program to count more transactions by scaling the number of transactions and also the number of clock cycles in the bandwidth window (SWU_CNT[n] register) by 10,100 or 1000. This is applicable only in bandwidth mode (SWU_CTL[n].BWEN==1).
		0 No scaling
		1 1:10
		2 1:100
		3 Reserved
		4 1:1000
		5-7 Reserved
20 (R/W)	STALLcntMODE	Stall Count Mode. The SWU_CTL[n].STALLcntMODE bit determines whether the number of stalls are counted or whether the number of transactions are counted. This feature is only valid in bandwidth mode.
		0 Count number of transactions
		1 Count number of stalls
19 (R/W)	MAXACT	Action for Bandwidth Above Maximum. Each SWU_CTL[n].MAXACT bit determines whether a watchpoint group takes action on bandwidth overflow. This feature is only valid in bandwidth mode.
		0 No Action
		1 Take Action
18 (R/W)	MINACT	Action for Bandwidth Below Minimum. Each SWU_CTL[n].MINACT bit determines whether a watchpoint group takes action on bandwidth underflow. This feature is only valid in bandwidth mode.
		0 No Action
		1 Take Action

Table 43-7: SWU_CTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
17 (R/W)	BLENINC	Increment Bandwidth Count by Burst Length. Each SWU_CTL[n] . BLENINC bit controls how a watchpoint group's bandwidth count is incremented in the SWU_CUR[n] register's SWU_CUR[n] . CURBW field. If the SWU_CTL[n] . BLENINC bit is cleared (= 0), the SWU increments the bandwidth count by 1 for each matching transaction. If the SWU_CTL[n] . BLENINC bit is set (=1), the SWU increments the bandwidth count by the burst length of the transaction for each matching transaction. This feature is only valid for bandwidth mode (SWU_CTL[n] . BWEN bit == 1). Note that if the address range match is enabled (SWU_CTL[n] . ACMPPM bits) and if any address of a burst falls within the address range, the SWU_CUR[n] . CURBW field is incremented by the burst length even if some of the burst address fall outside of the range. Also, note that the burst size of the transaction is not included in the increment, only the burst length of the transaction. This increment operation provides an approximate (not exact) number of bus cycles consumed during the bandwidth.
		0 Increment by 1
		1 Burst Length Increment for Bandwidth Count
16 (R/W)	BWEN	Bandwidth Mode Enable. Each SWU_CTL[n] . BWEN bit controls whether a watchpoint group operates in watchpoint mode or bandwidth mode. In watchpoint mode, the SWU_CTL[n] . CNTEN and (optionally) SWU_CTL[n] . CNTRPTEN registers control usage of the cycle count for watchpoint group operations. In bandwidth mode, the SWU_CTL[n] . BLENINC, SWU_TARG[n], and SWU_HIST[n] registers control usage of watchpoint matches for watchpoint group operations.
		0 Watchpoint Mode
		1 Bandwidth Mode
14 (R/W)	TRGEN	Trigger Enable. Each SWU_CTL[n] . TRGEN bit controls whether a match for a watchpoint group generates a trigger event. This feature is valid in both bandwidth and watchpoint modes.
		0 Disable
		1 Enable
13 (R/W)	INTEN	Interrupt Enable. Each SWU_CTL[n] . INTEN bit controls whether a match for a watchpoint group generates an interrupt. This feature is valid in both bandwidth and watchpoint modes.
		0 Disable
		1 Enable

Table 43-7: SWU_CTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
12 (R/W)	DBGEN	Debug Event Enable. Each SWU_CTL[n].DBGEN bit controls debug event comparison for a watchpoint group, permitting matches based on debug status.
		0 Disable
		1 Enable
9 (R/W)	CNTRPTEN	Count Repeat Enable. Each SWU_CTL[n].CNTRPTEN bit controls whether the watchpoint group's cycle count is reloaded and repeated after cycle countdown. If the SWU_CTL[n] register's SWU_CTL[n].CNTRPTEN bit is set, the SWU_CUR[n] register's SWU_CUR[n].CURCNT field is reloaded from SWU_CNT[n] register's SWU_CNT[n].COUNT field, and the countdown starts again. If SWU_CTL[n].CNTRPTEN bit is cleared, the expired count remains zero, and no further events are signalled. (See the SWU_CTL[n].CNTEN bit description for information regarding the countdown setup.)
		0 Disable
		1 Enable
8 (R/W)	CNTEN	Count Enable. Each SWU_CTL[n].CNTEN bit controls whether the cycle count in the watchpoint group's SWU_CNT[n] register is decremented each cycle until it reaches zero. This feature is only valid in watchpoint mode (SWU_CTL[n].BWEN bit == 0). When the count reaches zero, any enabled watchpoint events are triggered. (See the SWU_CTL[n].CNTRPTEN bit description for optional actions at that may occur at the end of the countdown.)
		0 Disable
		1 Enable
5 (R/W)	SCMPEN	Secure Comparison Enable. Each SWU_CTL[n].SCMPEN bit controls secure transaction comparison operation of an SWU watchpoint group, permitting matches based on transaction security.
		0 Match on all transaction
		1 Match only secure transactions
4 (R/W)	IDCMPEN	ID Comparison Enable. Each SWU_CTL[n].IDCMPEN bit controls the ID comparison operation of an SWU watchpoint group. The ID match is based on comparison with the value in the SWU_ID[n] register.

Table 43-7: SWU_CTL[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
3:2 (R/W)	ACMPM	Address Comparison Mode. Each set of SWU_CTL[n] . ACMPM bits control the address comparison operation of an SWU watchpoint group. The address within range for comparison is defined as (SWU_LA[n] register <= address < SWU_UA[n] register). The address outside range for comparison is defined as (address < SWU_LA[n]) or (SWU_UA[n] <= address).
		0 No address comparison
		1 Exact match on LAn
		2 Match on address within range
		3 Match on address outside range
1 (R/W)	DIR	Transaction Direction for Match. Each SWU_CTL[n] . DIR bit determines whether the SWU check reads or writes for watchpoint matches.
		0 Match on reads only
		1 Match on writes only
0 (R/W)	EN	Enable Watchpoint. Each SWU_CTL[n] . EN bit controls the operation of one SWU watchpoint group. Clearing the SWU_CTL[n] . EN bit halts the execution of watchpoint or bandwidth tracking operations in the watchpoint group without resetting status or configuration registers. Setting the SWU_CTL[n] . EN bit enables the SWU watchpoint group to begin or resume operation with the current configuration and status.
		0 Disable
		1 Enable

Current Register n

The SWU current register ($SWU_CUR[n]$) operation varies depending whether the watchpoint group is in bandwidth mode or watchpoint mode. In both modes, the watchpoint count begins when the SWU loads the register's $SWU_CUR[n].CURCNT$ field from the $SWU_CNT[n]$ register's $SWU_CNT[n].COUNT$ field when the watchpoint count is enabled ($SWU_CTL[n]$ register, $SWU_CTL[n].CNTEN$ bit =1).

In bandwidth mode, the current count field ($SWU_CUR[n].CURCNT$) contains the cycle count remaining within the current watchpoint period. The SWU decrements this value every cycle until the count reaches zero. At that point, the SWU reloads the $SWU_CUR[n].CURCNT$ field from $SWU_CNT[n]$ register's $SWU_CNT[n].COUNT$ field. In bandwidth mode, the current bandwidth field ($SWU_CUR[n].CURBW$) contains the count of watchpoint matches (bandwidth) accumulated in the current watchpoint period.

In watchpoint mode, the current count field ($SWU_CUR[n].CURCNT$) contains the watchpoint match count remaining within the current watchpoint period. The SWU decrements this value with every watchpoint match until the count reaches zero. At that point, the SWU reloads the $SWU_CUR[n].CURCNT$ field from $SWU_CNT[n]$ register's $SWU_CNT[n].COUNT$ field if the $SWU_CTL[n]$ register's $SWU_CTL[n].CNTRPTEN$ bit is set (=1). In watchpoint mode, the current bandwidth field ($SWU_CUR[n].CURBW$) is undefined.

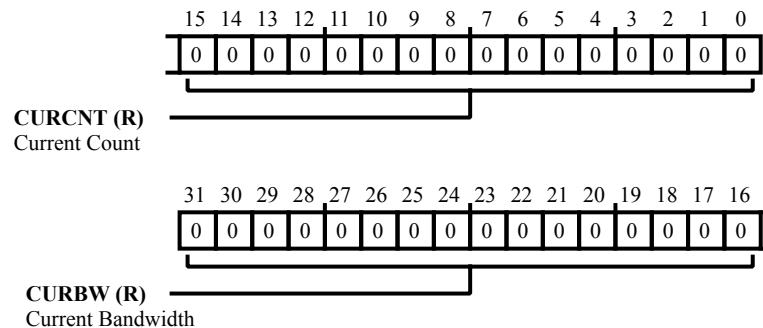


Figure 43-5: $SWU_CUR[n]$ Register Diagram

Table 43-8: $SWU_CUR[n]$ Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/NW)	CURBW	Current Bandwidth.
15:0 (R/NW)	CURCNT	Current Count.

Global Control Register

The SWU global control register (`SWU_GCTL`) provides SWU reset and enable.

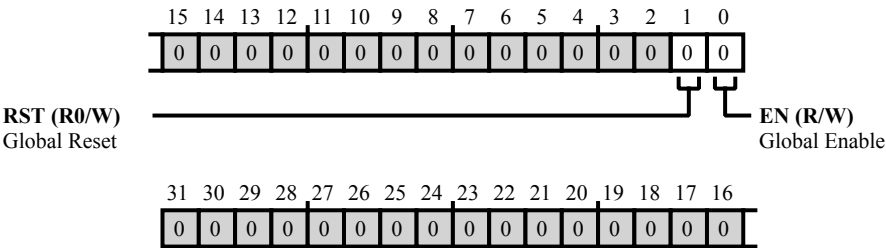


Figure 43-6: SWU_GCTL Register Diagram

Table 43-9: SWU_GCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration	
1 (R0/W)	RST	Global Reset. The <code>SWU_GCTL.RST</code> is write-1-action/read zero and controls the SWU operational state. Setting <code>SWU_GCTL.RST</code> resets all SWU registers to their default values and halts all SWU operations.	
		0	No Action
		1	Reset
0 (R/W)	EN	Global Enable. The <code>SWU_GCTL.EN</code> controls the SWU operational state. Clearing <code>SWU_GCTL.EN</code> halts the execution of all watchpoint and bandwidth tracking operations without resetting status registers or associated signals. Setting <code>SWU_GCTL.EN</code> enables the SWU to begin/resume operation with the current configuration and status.	
		0	Disable
		1	Enable

Global Status Register

The SWU global status register (`SWU_GSTAT`) contains status bits for all four watchpoint groups.

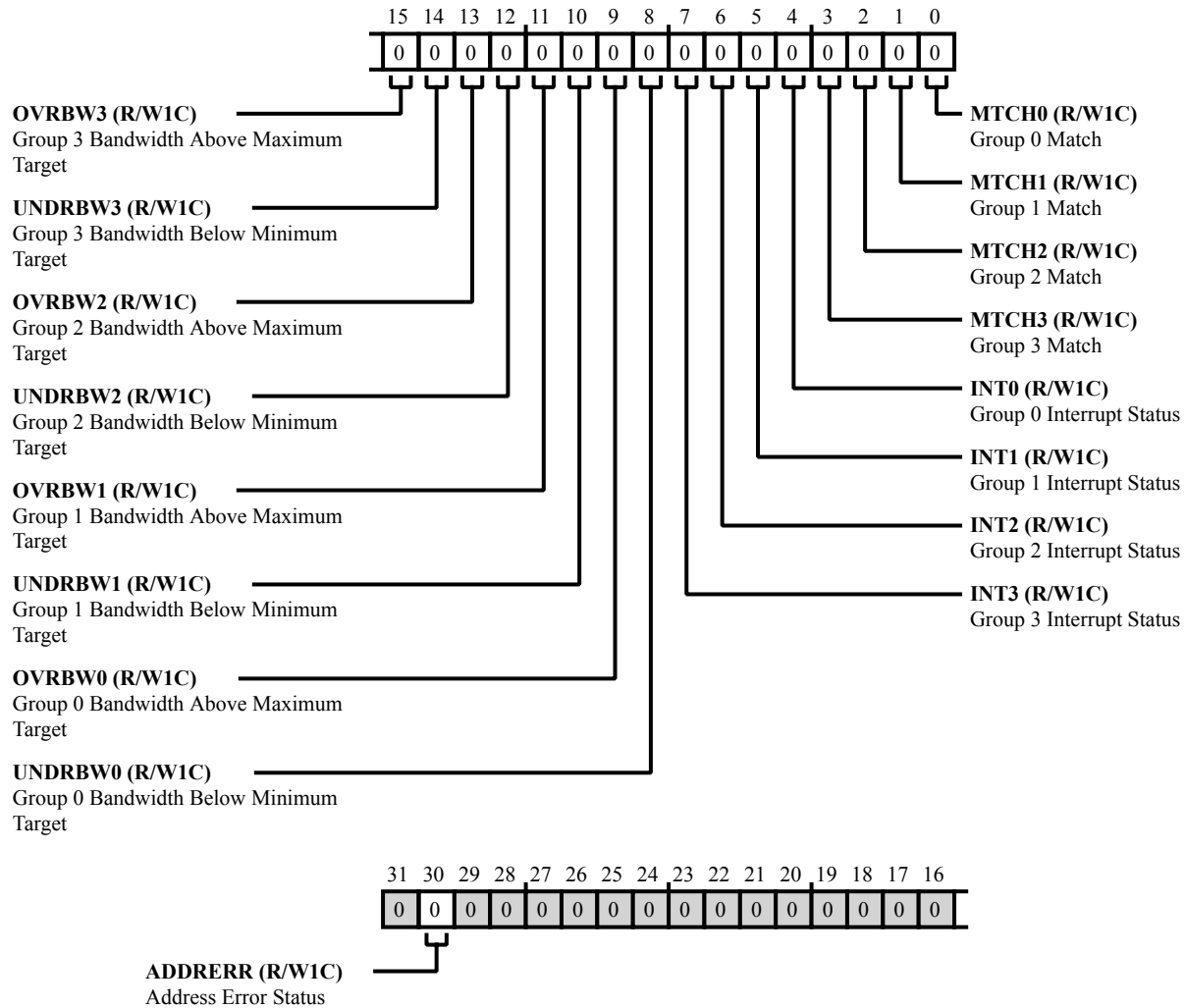


Figure 43-7: SWU_GSTAT Register Diagram

Table 43-10: SWU_GSTAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
30 (R/W1C)	ADDRERR	Address Error Status. The <code>SWU_GSTAT.ADDRERR</code> indicates that the SWU generated an address error. This status bit is sticky; write-1-to-clear it.
		0 Inactive
		1 Active

Table 43-10: SWU_GSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W1C)	OVRBW3	Group 3 Bandwidth Above Maximum Target. See SWU_GSTAT.OVRBW0 description.
		0 Group 3 was not above maximum bandwidth
		1 Group 3 was above maximum bandwidth
14 (R/W1C)	UNDRBW3	Group 3 Bandwidth Below Minimum Target. See SWU_GSTAT.UNDRBW0 description.
		0 Group 3 was not below minimum bandwidth
		1 Group 3 was below minimum bandwidth
13 (R/W1C)	OVRBW2	Group 2 Bandwidth Above Maximum Target. See SWU_GSTAT.OVRBW0 description.
		0 Group 2 was not above maximum bandwidth
		1 Group 2 was above maximum bandwidth
12 (R/W1C)	UNDRBW2	Group 2 Bandwidth Below Minimum Target. See SWU_GSTAT.UNDRBW0 description.
		0 Group 2 was not below minimum bandwidth
		1 Group 2 was below minimum bandwidth
11 (R/W1C)	OVRBW1	Group 1 Bandwidth Above Maximum Target. See SWU_GSTAT.OVRBW0 description.
		0 Group 1 was not above maximum bandwidth
		1 Group 1 was above maximum bandwidth
10 (R/W1C)	UNDRBW1	Group 1 Bandwidth Below Minimum Target. See SWU_GSTAT.UNDRBW0 description.
		0 Group 1 was not below minimum bandwidth
		1 Group 1 was below minimum bandwidth
9 (R/W1C)	OVRBW0	Group 0 Bandwidth Above Maximum Target. The SWU_GSTAT.OVRBW0 - SWU_GSTAT.OVRBW3 -- Group 0 through 3 watch-point bandwidth over maximum target bits. Each maximum bandwidth bit indicate (for each group)s that the measured bandwidth over the period defined by the SWU_CNT[n] register was over the maximum target. This status bit is sticky; write-1-to-clear it.
		0 Group 0 was not above maximum bandwidth
		1 Group 0 was above maximum bandwidth

Table 43-10: SWU_GSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
8 (R/W1C)	UNDRBW0	Group 0 Bandwidth Below Minimum Target. The SWU_GSTAT.UNDRBW0 - SWU_GSTAT.UNDRBW3 -- Group 0 through 3 watchpoint bandwidth below minimum target bits. Each minimum bandwidth bit indicates (for each group) that the measured bandwidth over the period defined by the SWU_CNT[n] register was below the minimum target. This status bit is sticky; write-1-to-clear it.
		0 Group 0 was not below minimum bandwidth
		1 Group 0 was below minimum bandwidth
7 (R/W1C)	INT3	Group 3 Interrupt Status. See SWU_GSTAT.INT0 description.
		0 No Interrupt
		1 Interrupt Occurred
6 (R/W1C)	INT2	Group 2 Interrupt Status. See SWU_GSTAT.INT0 description.
		0 No Interrupt
		1 Interrupt Occurred
5 (R/W1C)	INT1	Group 1 Interrupt Status. See SWU_GSTAT.INT0 description.
		0 No Interrupt
		1 Interrupt Occurred
4 (R/W1C)	INT0	Group 0 Interrupt Status. The SWU_GSTAT.INT0 - SWU_GSTAT.INT3 -- Group 0 through 3 interrupt bits. Each interrupt bit indicates (for each group) whether a watchpoint group is contributing to the SWU's interrupt output. This status bit is sticky; write-1-to-clear it.
		0 No interrupt
		1 Interrupt Occurred
3 (R/W1C)	MTCH3	Group 3 Match. See SWU_GSTAT.MTCH0 description.
		0 No Match
		1 Group 3 Watchpoint Match

Table 43-10: SWU_GSTAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
2 (R/W1C)	MTCH2	Group 2 Match. See SWU_GSTAT.MTCH0 description.
		0 No match
		1 Group 2 Watchpoint Match
1 (R/W1C)	MTCH1	Group 1 Match. See SWU_GSTAT.MTCH0 description.
		0 No match
		1 Group 1 Watchpoint Match
0 (R/W1C)	MTCH0	Group 0 Match. The SWU_GSTAT.MTCH0 - SWU_GSTAT.MTCH3 -- Group 0 through 3 match bits. Each match bit indicates (for each group) whether a watchpoint match has occurred in a SWU watchpoint group, as controlled by the group's related watchpoint control register (SWU_CTL[n]). This status bit is sticky; write-1-to-clear it.
		0 No match
		1 Group 0 Watchpoint Match

Bandwidth History Register n

The SWU bandwidth history registers (`SWU_HIST[n]`) contain data copied from a watchpoint group's current bandwidth value (`SWU_CUR[n]` register, `SWU_CUR[n].CURBW` bits) at the end of the last two watchpoint periods. At the end of each watchpoint period, the SWU copies the previous bandwidth value from the `SWU_HIST[n].BWHIST0` field to the `SWU_HIST[n].BWHIST1` field and copies the new bandwidth value from the `SWU_CUR[n].CURBW` field to the `SWU_HIST[n].BWHIST0` field.

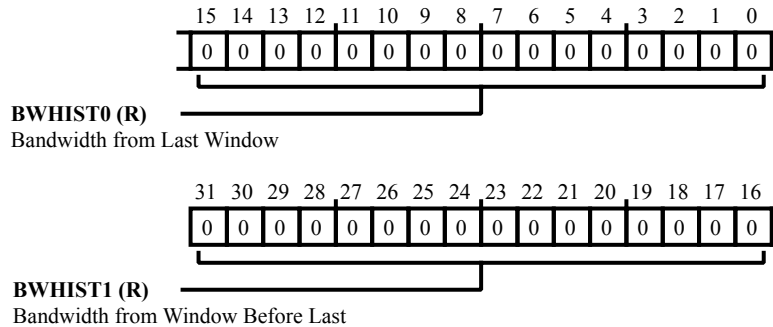


Figure 43-8: `SWU_HIST[n]` Register Diagram

Table 43-11: `SWU_HIST[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/NW)	BWHIST1	Bandwidth from Window Before Last.
15:0 (R/NW)	BWHIST0	Bandwidth from Last Window.

ID Register n

The SWU ID registers (`SWU_ID[n]`) contain a 16-bit ID field (`SWU_ID[n].ID`) and a 16-bit ID mask field (`SWU_ID[n].IDMASK`) that watchpoint groups use for ID comparison. The ID on the bus is AND'ed with the `SWU_ID[n].IDMASK` field, then the watchpoint group compares the result against the `SWU_ID[n].ID` field.

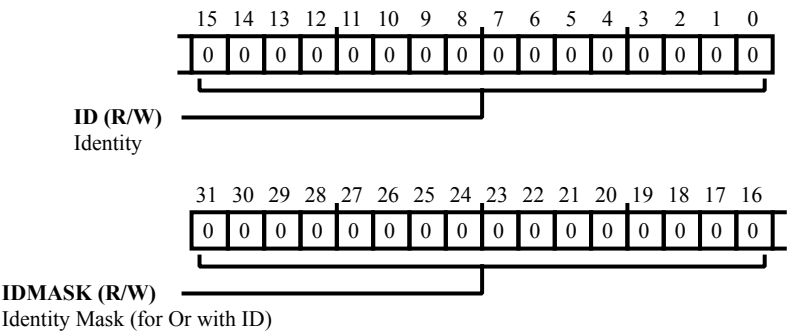


Figure 43-9: `SWU_ID[n]` Register Diagram

Table 43-12: `SWU_ID[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	IDMASK	Identity Mask (for Or with ID).
15:0 (R/W)	ID	Identity.

Lower Address Register n

The SWU lower address registers (`SWU_LA[n]`) contain each watchpoint group's lower address for address match comparison. In exact match on `SWU_LA[n]` address mode (`SWU_CTL[n].ACMPM` bits = 01), the watchpoint group uses only this address for match comparison.

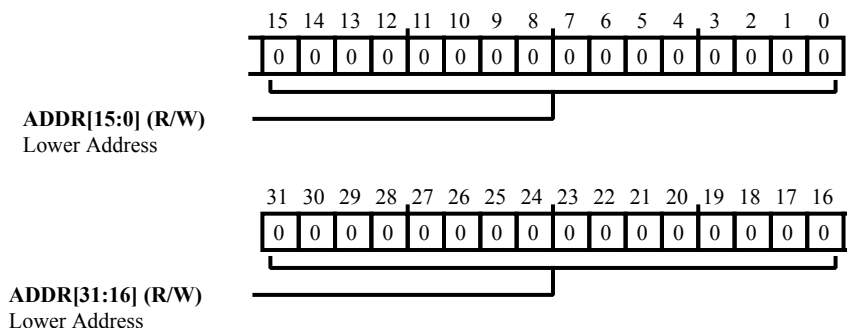


Figure 43-10: `SWU_LA[n]` Register Diagram

Table 43-13: `SWU_LA[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ADDR	Lower Address.

Target Register n

The SWU target registers (`SWU_TARG[n]`) contain a minimum value field (`SWU_TARG[n].BWMIN`) and maximum value field (`SWU_TARG[n].BWMAX`) of bandwidth targets used by watchpoint groups in bandwidth mode. When the bandwidth period expires, if the current bandwidth value (`SWU_CUR[n]` register, `SWU_CUR[n].CURBW` bits) is below the minimum target or above the maximum target, the watchpoint group takes action as enabled by the `SWU_CTL[n]` register's `SWU_CTL[n].MINACT` or `SWU_CTL[n].MAXACT` bits.

In bandwidth mode, note that the watchpoint group increments its count of either data bus transactions or address bus transactions (bursts) as selected by the `SWU_CTL[n].BLENINC` bit. Keep this mode selection in mind when programming the bandwidth target values.

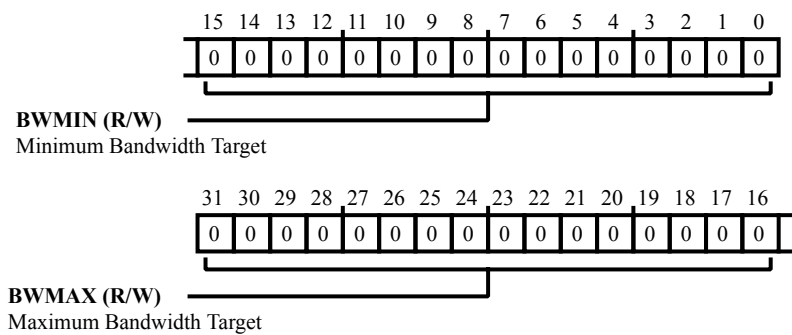


Figure 43-11: `SWU_TARG[n]` Register Diagram

Table 43-14: `SWU_TARG[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	BWMAX	Maximum Bandwidth Target.
15:0 (R/W)	BWMIN	Minimum Bandwidth Target.

Upper Address Register n

The SWU upper address registers (*SWU_UA[n]*) contain each watchpoint group's upper address for address match comparison. In exact match on *SWU_LA[n]* address mode (*SWU_CTL[n].ACMPM* bits =01), the *SWU_UA[n]* is not used for match comparison.

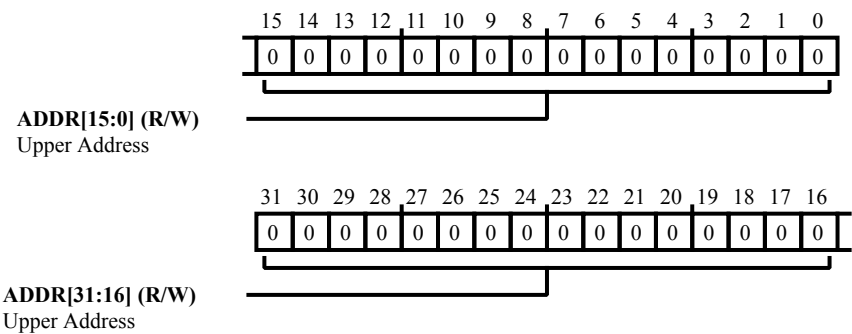


Figure 43-12: SWU_UA[n] Register Diagram

Table 43-15: SWU_UA[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ADDR	Upper Address.

44 Memory Error Protection Unit (MEPU)

Memory Error Protection Unit (MEPU) handles single-bit and double-bit memory error detection and correction across the memories in the chip and controls routing of their interrupts/triggers.

It consists of following blocks:

- Memory Error Controller (MEC)
- Parity Controller (PCTL)

Memory Error Controller (MEC)

The Memory Error Controller (MEC) manages memory parity, ECC errors and warning inputs from various cores and peripherals and sends out interrupt and trigger outputs. It is a generic N x M multiplexer and interrupt controller for error or warning inputs to interrupt and trigger outputs. It has control features such as enable and disable and interrupt masking, and status signaling for input/outputs. For more information on MEC functionality, see the MEC registers description.

MEC Features

The MEC unit has the following features:

- Interrupt controller for memory parity, ECC errors and warnings
- N x M multiplexer for error or warning inputs to interrupt and trigger outputs
- Peripheral bus slave interface for register read and write
- Control and status registers
 - Control for enabling or disabling each input error status and masking its interrupt
 - Control for enabling or disabling each output error interrupt
 - Input error status bits are sticky and write-1-to-clear
 - Output error interrupt status bits are read only
 - Lock feature for control registers write. Status registers capture lock write error.

- Peripheral and component identification registers
- Capability to retain parity, ECC error and warning status even after any reset for fault source debug
- Fully-configurable design

Parity Controller (PCTL)

The Parity Controller (PCTL) handles parity encoding and decoding to and from memory data for single bit memory error detection.

PCTL Features

The PCTL unit has the following features:

- Generic memory parity controller
 - Generation of write parity bits from write data bits
 - Concatenation of parity bits with data bits for memory write data
 - Generation of write enable mask for parity bits to memory
 - Extraction of read data bits from memory read data
 - Detection of read parity error from memory read data
- Even parity logic for parity generation and error detection
- Provision for parity bit interleaving
- Single parity error output per memory instance
- Memory initialization control logic
- Fully-configurable design

MEC Functional Description

The MEC works with the PCTL to detect and correct memory error across the memories in the chip. It also controls the routing of the generated interrupts and triggers.

ADSP-2159x MEC Register List

The Memory Error Controller (MEC) manages memory parity/ecc errors/warning inputs from various cores and peripherals and sends out interrupt/trigger outputs. It is generic N x M multiplexer and interrupt controller for error/warning inputs to interrupt/trigger outputs. It has control features such as enable/disable and interrupt masking, and status signaling for inputs/outputs. For more information on MEC functionality, see the MEC register descriptions.

Table 44-1: ADSP-2159x MEC Register List

Name	Description
MEC_CID0	Component ID0 Register
MEC_CID1	Component ID1 Register
MEC_CID2	Component ID2 Register
MEC_CID3	Component ID3 Register
MEC_CLR	Clear Register
MEC_ECCERR_CTL[y]	ECC Error Control Register
MEC_ECCERR_IMASK[y]	ECC Error Interrupt Mask Register
MEC_ECCERR_STAT[y]	ECC Error Status Register
MEC_EEIRQ_GCTL[q]	ECC Error Interrupt Request Global Control Register
MEC_EEIRQ_GSTAT[q]	ECC Error Interrupt Request Global Status Register
MEC_PEIRQ_GCTL[p]	Parity Error Interrupt Request Global Control Register
MEC_PEIRQ_GSTAT[p]	Parity Error Interrupt Request Global Status Register
MEC_PERR_CTL0	Parity Error Control Register
MEC_PERR_CTL1	Parity Error Control Register
MEC_PERR_IMASK0	Parity Error Interrupt Mask Register
MEC_PERR_IMASK1	Parity Error Interrupt Mask Register
MEC_PERR_STAT0	Parity Error Status Register
MEC_PERR_STAT1	Parity Error Status Register
MEC_PID0	Peripheral ID0 Register
MEC_PID1	Peripheral ID1 Register
MEC_PID2	Peripheral ID2 Register
MEC_PID3	Peripheral ID3 Register
MEC_PID4	Peripheral ID4 Register
MEC_PID5	Peripheral ID5 Register
MEC_PID6	Peripheral ID6 Register
MEC_PID7	Peripheral ID7 Register

ADSP-2159x MEC Interrupt List

Table 44-2: ADSP-2159x MEC Interrupt List

Interrupt ID	Name	Description	Sensitivity	DMA Channel
13	MEC1_EEIRQ0	MEC1 ECC Error Interrupt Request		
14	Reserved	Reserved	Reserved	Reserved
15	MEC1_PEIRQ0	MEC1 Parity Error Interrupt Request	Level	
16	MEC1_PEIRQ1	MEC1 Parity Error Interrupt Request	Level	
17	MEC1_PEIRQ2	MEC1 Parity Error Interrupt Request	Level	
18	MEC1_PEIRQ3	MEC1 Parity Error Interrupt Request	Level	
19	MEC0_EEIRQ0	MEC0 ECC Error Interrupt Request		
20	Reserved	Reserved	Reserved	Reserved
21	MEC0_PEIRQ0	MEC0 Parity Error Interrupt Request	Level	
22	MEC0_PEIRQ1	MEC0 Parity Error Interrupt Request	Level	
23	MEC0_PEIRQ2	MEC0 Parity Error Interrupt Request	Level	
24	MEC0_PEIRQ3	MEC0 Parity Error Interrupt Request	Level	
25	MEC2_EEIRQ0	MEC2 ECC Error Interrupt Request		
26	Reserved	Reserved	Reserved	Reserved
27	MEC2_PEIRQ0	MEC2 Parity Error Interrupt Request	Level	
28	MEC2_PEIRQ1	MEC2 Parity Error Interrupt Request	Level	
29	MEC2_PEIRQ2	MEC2 Parity Error Interrupt Request	Level	
30	MEC2_PEIRQ3	MEC2 Parity Error Interrupt Request	Level	

ADSP-2159x MEC Trigger List

Table 44-3: ADSP-2159x MEC Trigger List Masters

Trigger ID	Name	Description	Sensitivity
58	MEC0_EEIRQ0	MEC0 ECC Error Interrupt Request	
59	MEC0_PEIRQ0	MEC0 Parity Error Interrupt Request	Level
60	MEC0_PEIRQ1	MEC0 Parity Error Interrupt Request	Level
61	MEC0_PEIRQ2	MEC0 Parity Error Interrupt Request	Level
62	MEC0_PEIRQ3	MEC0 Parity Error Interrupt Request	Level
63	MEC1_EEIRQ0	MEC1 ECC Error Interrupt Request	
64	MEC1_PEIRQ0	MEC1 Parity Error Interrupt Request	Level
65	MEC1_PEIRQ1	MEC1 Parity Error Interrupt Request	Level

Table 44-3: ADSP-2159x MEC Trigger List Masters (Continued)

Trigger ID	Name	Description	Sensitivity
66	MEC1_PEIRQ2	MEC1 Parity Error Interrupt Request	Level
67	MEC1_PEIRQ3	MEC1 Parity Error Interrupt Request	Level
68	MEC2_EEIRQ0	MEC2 ECC Error Interrupt Request	
69	MEC2_PEIRQ0	MEC2 Parity Error Interrupt Request	Level
70	MEC2_PEIRQ1	MEC2 Parity Error Interrupt Request	Level
71	MEC2_PEIRQ2	MEC2 Parity Error Interrupt Request	Level
72	MEC2_PEIRQ3	MEC2 Parity Error Interrupt Request	Level

Table 44-4: ADSP-2159x MEC Trigger List Slaves

Trigger ID	Name	Description	Sensitivity
None			

MEPU Block Diagram

The *MEPU Block Diagram* shows the functional blocks within the Memory Error Protection Unit (MEPU) unit.

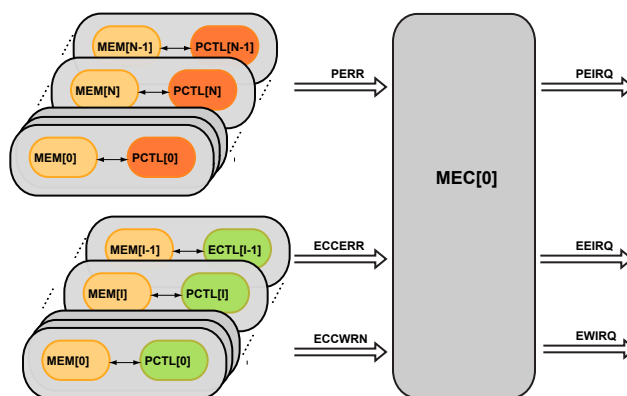


Figure 44-1: MEPU Block Diagram

MEC Block Diagram

The *MEC Interface Block Diagram* shows the functional blocks within the MEC module.

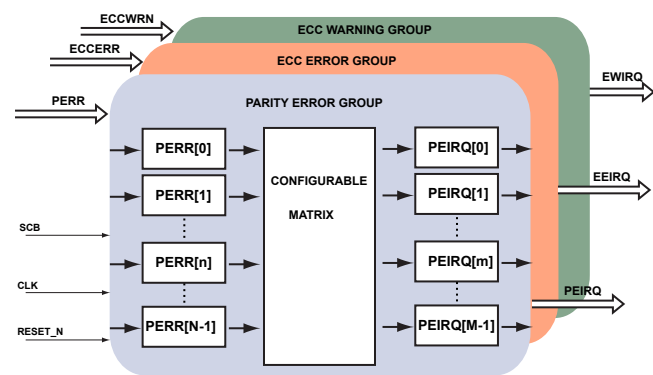


Figure 44-2: MEC Unit Block Diagram

PCTL Block Diagram

The *PCTL Block Diagram* shows the functional blocks within the PCTL module.

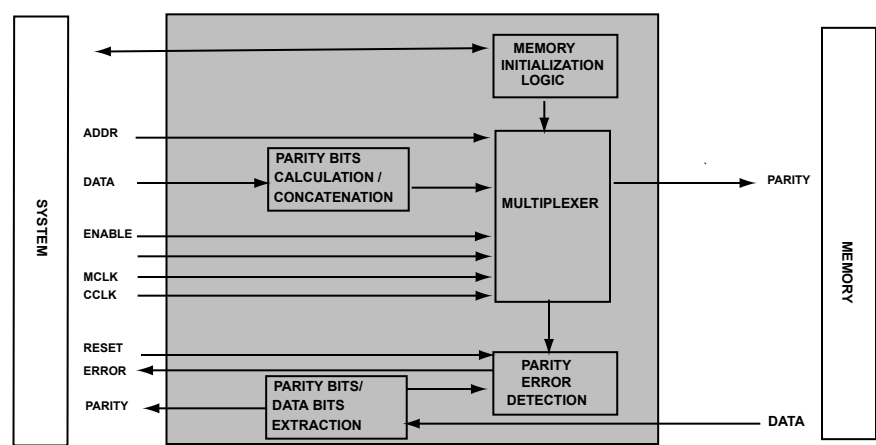


Figure 44-3: PCTL Block Diagram

MECx Input and Output Block Diagram

The *MECx Input and Output Block Diagram* shows the input and output routing to various cores.

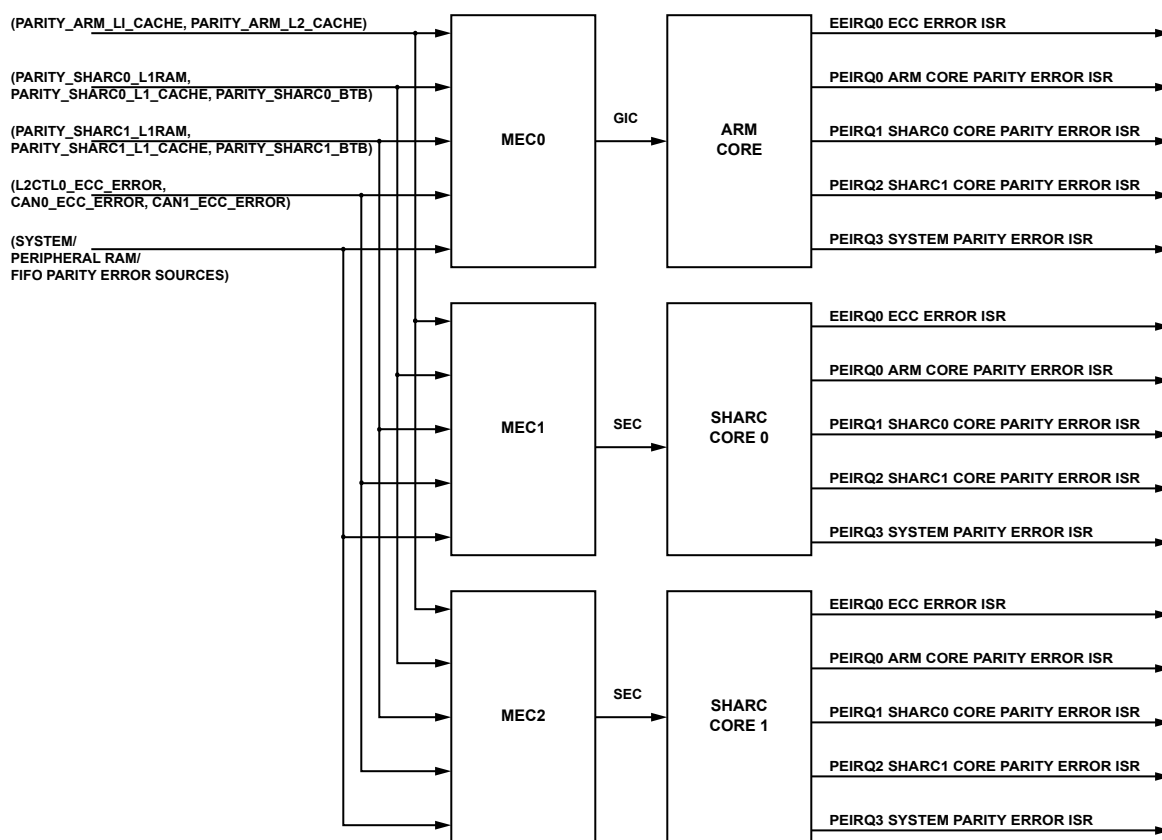


Figure 44-4: MECx Input and Output Block Diagram

MEC Architectural Concepts

The MEC unit communicates with the Cortex-A5 directly through the subsystem's SYS bus crossbar. It services MMR addresses associated with the MEC function unit and translates the addresses into specific transcendental MEC functions. The results are read from common result registers shared by all the functions in single-precision floating-point format. The block stalls all read-accesses until the result is ready.

MEC Configuration

The following tables describe the range of values for configuration and the mapping of parity errors.

Table 44-5: Mapping of Input Parity Errors (PERR)

PERR ID	Name	Description
0	PERR_C0_L1CR	Core0 (ARM) L1 Cache RAM Parity Error
1	PERR_C0_L2CR	Core0 (ARM) L2 Cache RAM Parity Error
2	PERR_C1_L1R	Core1 (SHARC0) L1 RAM Parity Error
3	PERR_C1_L1CR	Core1 (SHARC0) L1 Cache RAM Parity Error
4	PERR_C1_BPR	Core1 (SHARC0) Branch Predictor RAM Parity Error
5	PERR_C2_L1R	Core2 (SHARC1) L1 RAM Parity Error

Table 44-5: Mapping of Input Parity Errors (PERR) (Continued)

PERR ID	Name	Description
6	PERR_C2_L1CR	Core2 (SHARC1) L1 Cache RAM Parity Error
7	PERR_C2_BPR	Core2 (SHARC1) Branch Predictor RAM Parity Error
8	PERR_ASRC0_FR	ASRC0 FIFO RAM Parity Error
9	PERR_ASRC1_FR	ASRC1 FIFO RAM Parity Error
10	PERR_ASRC2_FR	ASRC2 FIFO RAM Parity Error
11	PERR_ASRC3_FR	ASRC3 FIFO RAM Parity Error
12	PERR_IIR0_R	IIR0 RAM Parity Error
13	PERR_FIR0_R	FIR0 RAM Parity Error
14	PERR_USB0_FR	USB0 FIFO RAM Parity Error
15	PERR_CAN0_MBR	CAN0 Mailbox RAM Parity Error
16	PERR_CAN0_AMR	CAN0 Acceptance Mask RAM Parity Error
17	PERR_CAN1_MBR	CAN1 Mailbox RAM Parity Error
18	PERR_CAN1_AMR	CAN1 Acceptance Mask RAM Parity Error
19	PERR_TRNG0_DBR	TRNG0 Data Buffer RAM Parity Error
20	PERR_PKA0_DR	PKA0 Data RAM Parity Error
21	PERR_SPE0_BR	SPE0 Buffer RAM Parity Error
22	PERR_SPE0_AR	SPE0 ARC4 RAM Parity Error
23	PERR_EMAC0_TFR	EMAC0 Transmit FIFO RAM Parity Error
24	PERR_EMAC0_RFR	EMAC0 Receive FIFO RAM Parity Error
25	PERR_MSI0_FR	MSI0 FIFO RAM Parity Error
26	PERR_MLB0_DBR	MLB0 Data Buffer RAM Parity Error
27	PERR_MLB0_CTR	MLB0 Channel Table RAM Parity Error
28	PERR_TMC0_TDR	TMC0 Trace Data RAM Parity Error

Table 44-6: Mapping of Output Parity Errors (PERR)

PERR ID	Name	Description
0	PEIRQ_C0	Core0 (ARM) Parity Error Interrupt
1	PEIRQ_C1	Core1 (SHARC0) Parity Error Interrupt
2	PEIRQ_C2	Core2 (SHARC1) Parity Error Interrupt
3	PEIRQ_SYS	System Peripheral Parity Error Interrupt

Table 44-7: Mapping of Input ECC Errors (ECCERR)

PERR ID	Name	Description
0	ECCERR_L2CTL0	L2CTL0 ECC Error

Table 44-8: Mapping of Output ECC Errors (EEIRQ)

PERR ID	Name	Description
0	EWIRQ_L2CTL0	L2CTL0 ECC Error Interrupt

PCTL Integration

There is one PCTL instance per port per memory instance (for example, one PCTL instance for single port memory and two PCTL instances for dual port memories).

Memory initialization control logic supported by PCTL is used only for instances attached to Arm L1 cache memories.

An additional software trigger master and initialization trigger slave is provided in TRU for starting memory initialization. A pulse from this trigger slave will start initialization of Arm L1 cache memories. When all the locations of a particular memory instance get initialized, PCTL generates a memory initialization done signal for corresponding memory instance. These signals from all PCTL instances corresponding to Arm L1 cache memories are AND'ed to generate a single memory initialization done output which is connected as an interrupt to SEC/GIC and a trigger master to TRU.

ADSP-2159x MEC Register Descriptions

Memory Error Controller (MEC) contains the following registers.

Table 44-9: ADSP-2159x MEC Register List

Name	Description
MEC_CID0	Component ID0 Register
MEC_CID1	Component ID1 Register
MEC_CID2	Component ID2 Register
MEC_CID3	Component ID3 Register
MEC_CLR	Clear Register
MEC_ECCERR_CTL[y]	ECC Error Control Register
MEC_ECCERR_IMASK[y]	ECC Error Interrupt Mask Register
MEC_ECCERR_STAT[y]	ECC Error Status Register
MEC_EEIRQ_GCTL[q]	ECC Error Interrupt Request Global Control Register
MEC_EEIRQ_GSTAT[q]	ECC Error Interrupt Request Global Status Register

Table 44-9: ADSP-2159x MEC Register List (Continued)

Name	Description
MEC_PEIRQ_GCTL[p]	Parity Error Interrupt Request Global Control Register
MEC_PEIRQ_GSTAT[p]	Parity Error Interrupt Request Global Status Register
MEC_PERR_CTL0	Parity Error Control Register
MEC_PERR_CTL1	Parity Error Control Register
MEC_PERR_IMASK0	Parity Error Interrupt Mask Register
MEC_PERR_IMASK1	Parity Error Interrupt Mask Register
MEC_PERR_STAT0	Parity Error Status Register
MEC_PERR_STAT1	Parity Error Status Register
MEC_PID0	Peripheral ID0 Register
MEC_PID1	Peripheral ID1 Register
MEC_PID2	Peripheral ID2 Register
MEC_PID3	Peripheral ID3 Register
MEC_PID4	Peripheral ID4 Register
MEC_PID5	Peripheral ID5 Register
MEC_PID6	Peripheral ID6 Register
MEC_PID7	Peripheral ID7 Register

Component ID0 Register

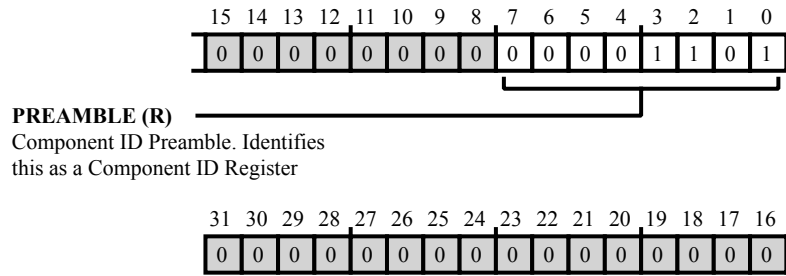


Figure 44-5: MEC_CID0 Register Diagram

Table 44-10: MEC_CID0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PREAMBLE	Component ID Preamble. Identifies this as a Component ID Register.

Component ID1 Register

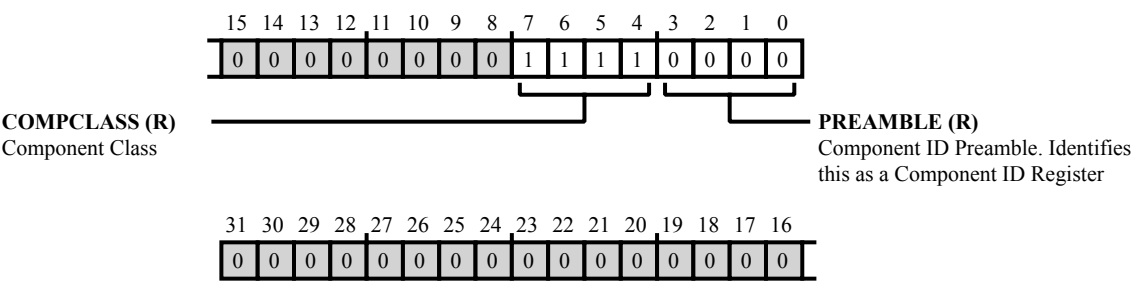


Figure 44-6: MEC_CID1 Register Diagram

Table 44-11: MEC_CID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	COMPCLASS	Component Class. Identifies the Component Class. Dedicated debug blocks (core debug access port, Program Trace, etc.) should identify as CoreSight (i.e. 0x9) and implement the full complement of CoreSight registers including DEVTYPE. All other ADI components should identify as System (i.e. 0xF) components. See CoreSight Architecture Specification for more detail.
		9 CoreSight
		15 System
3:0 (R/NW)	PREAMBLE	Component ID Preamble. Identifies this as a Component ID Register.

Component ID2 Register

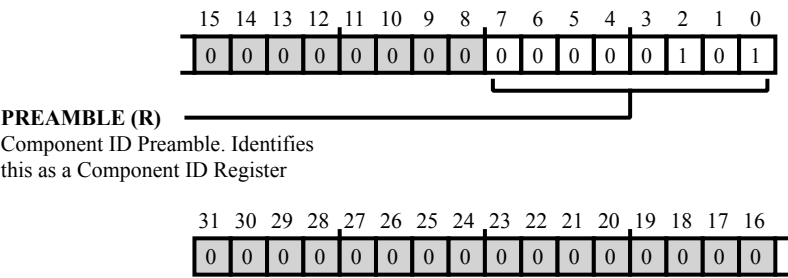


Figure 44-7: MEC_CID2 Register Diagram

Table 44-12: MEC_CID2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PREAMBLE	Component ID Preamble. Identifies this as a Component ID Register.

Component ID3 Register

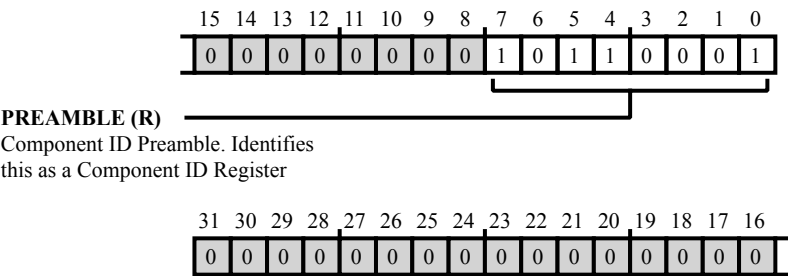


Figure 44-8: MEC_CID3 Register Diagram

Table 44-13: MEC_CID3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PREAMBLE	Component ID Preamble. Identifies this as a Component ID Register.

Clear Register

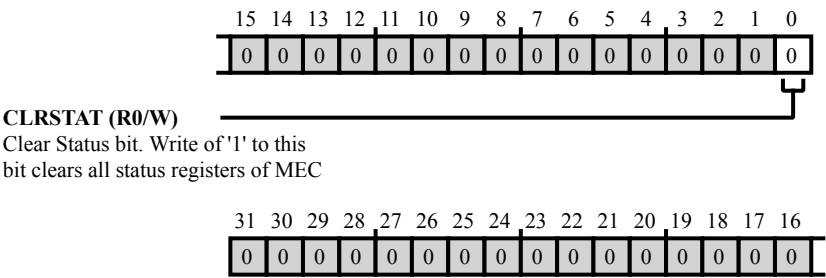


Figure 44-9: MEC_CLR Register Diagram

Table 44-14: MEC_CLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R0/W)	CLRSTAT	Clear Status bit. Write of '1' to this bit clears all status registers of MEC.

ECC Error Control Register

`MEC_ECCERR_CTL[y]` register bits control enable/disable for ECC error inputs from various cores/peripherals and decide whether their status will be reflected in ECC error status register bits.

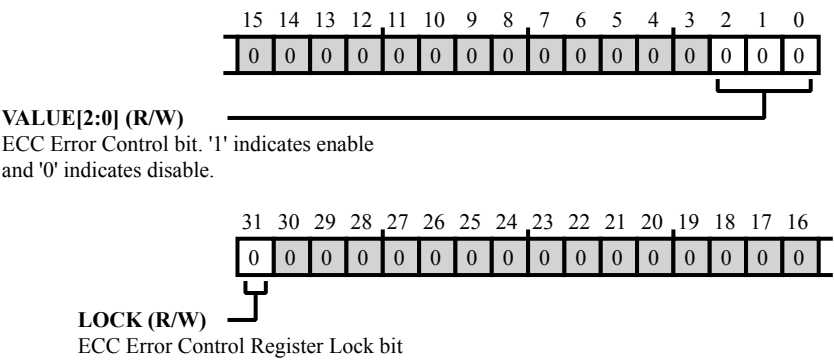


Figure 44-10: MEC_ECCERR_CTL[y] Register Diagram

Table 44-15: MEC_ECCERR_CTL[y] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	ECC Error Control Register Lock bit. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>MEC_ECCERR_CTL[y].LOCK</code> bit is enabled, the <code>MEC_ECCERR_CTL[y]</code> register is read only.
		0 Unlock
		1 Lock
2:0 (R/W)	VALUE	ECC Error Control bit. '1' indicates enable and '0' indicates disable..
		1 L2 CTL ECC Error
		2 CAN0 ECC Error
		4 CAN1 ECC Error

ECC Error Interrupt Mask Register

`MEC_ECCERR_IMASK[y]` register bits control interrupt masks for ECC error inputs from various cores/peripherals.

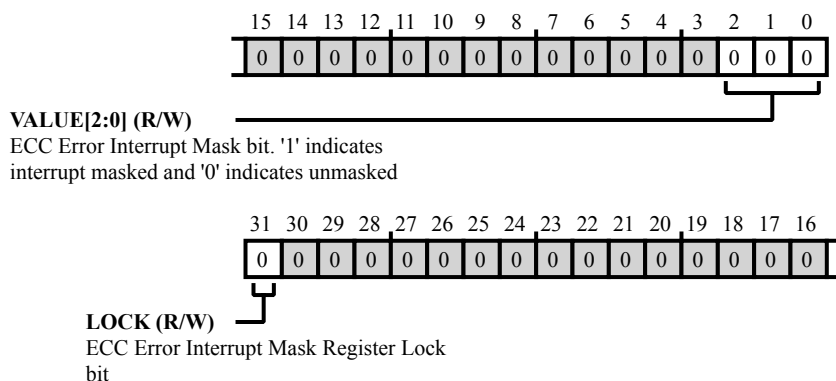


Figure 44-11: MEC_ECCERR_IMASK[y] Register Diagram

Table 44-16: MEC_ECCERR_IMASK[y] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	ECC Error Interrupt Mask Register Lock bit. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>MEC_ECCERR_IMASK[y].LOCK</code> bit is enabled, the <code>MEC_ECCERR_IMASK[y]</code> register is read only.
		0 Unlock
		1 Lock
2:0 (R/W)	VALUE	ECC Error Interrupt Mask bit. '1' indicates interrupt masked and '0' indicates unmasked.
		1 L2 CTL ECC Error
		2 CAN0 ECC Error
		4 CAN1 ECC Error

ECC Error Status Register

`MEC_ECCERR_STAT[y]` register bits reflect status for ECC error inputs from various cores/peripherals. Writing '1' to these bits clear corresponding ECC error status.

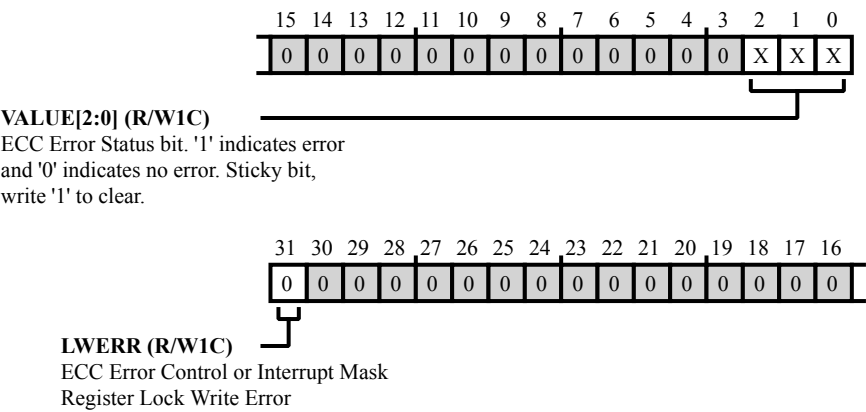


Figure 44-12: MEC_ECCERR_STAT[y] Register Diagram

Table 44-17: MEC_ECCERR_STAT[y] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	ECC Error Control or Interrupt Mask Register Lock Write Error. The <code>MEC_ECCERR_STAT[y].LWERR</code> bit indicates (when set) there was an attempted write to an MEC register while the <code>MEC_ECCERR_CTL[y].LOCK</code> bit was set and while the global lock bit was enabled (<code>SPU_CTL.GLCK</code> bit =1). This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
2:0 (R/W1C)	VALUE	ECC Error Status bit. '1' indicates error and '0' indicates no error. Sticky bit, write '1' to clear..
		1 L2 CTL ECC Error
		2 CAN0 ECC Error
		4 CAN1 ECC Error

ECC Error Interrupt Request Global Control Register

`MEC_EEIRQ_GCTL[q]` register bits control enable/disable of ECC error interrupt/trigger outputs.

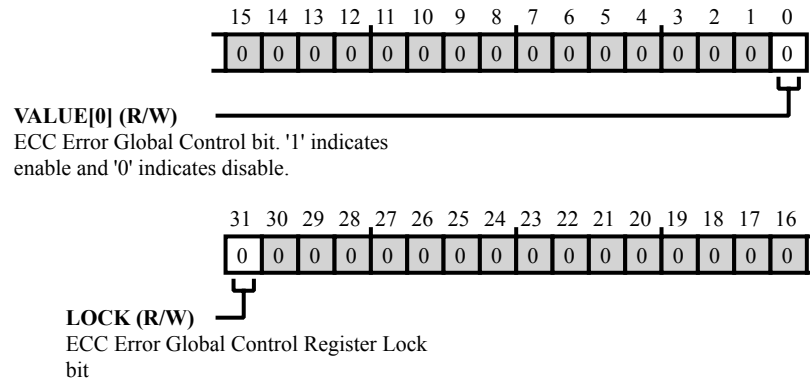


Figure 44-13: MEC_EEIRQ_GCTL[q] Register Diagram

Table 44-18: MEC_EEIRQ_GCTL[q] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	ECC Error Global Control Register Lock bit. If the global lock is enabled (SPU_CTL . GLCK bit =1) and the MEC_EEIRQ_GCTL[q] . LOCK bit is enabled, the MEC_EEIRQ_GCTL[q] register is read only.
		0 Unlock
		1 Lock
0 (R/W)	VALUE	ECC Error Global Control bit. '1' indicates enable and '0' indicates disable..

ECC Error Interrupt Request Global Status Register

MEC_EEIRQ_GSTAT[q] register bits reflect status of ECC error interrupt/trigger outputs.

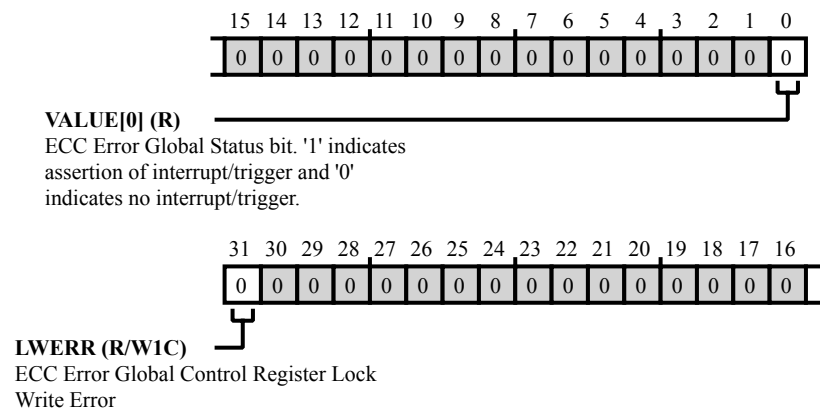


Figure 44-14: MEC_EEIRQ_GSTAT[q] Register Diagram

Table 44-19: MEC_EEIRQ_GSTAT[q] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	ECC Error Global Control Register Lock Write Error. The MEC_EEIRQ_GSTAT[q].LWERR bit indicates (when set) there was an attempted write to an MEC register while the MEC_EEIRQ_GCTL[q].LOCK bit was set and while the global lock bit was enabled (SPU_CTL.GLCK bit =1). This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
0 (R/NW)	VALUE	ECC Error Global Status bit. '1' indicates assertion of interrupt/trigger and '0' indicates no interrupt/trigger..

Parity Error Interrupt Request Global Control Register

`MEC_PEIRQ_GCTL[p]` register bits control enable/disable of parity error interrupt/trigger outputs.

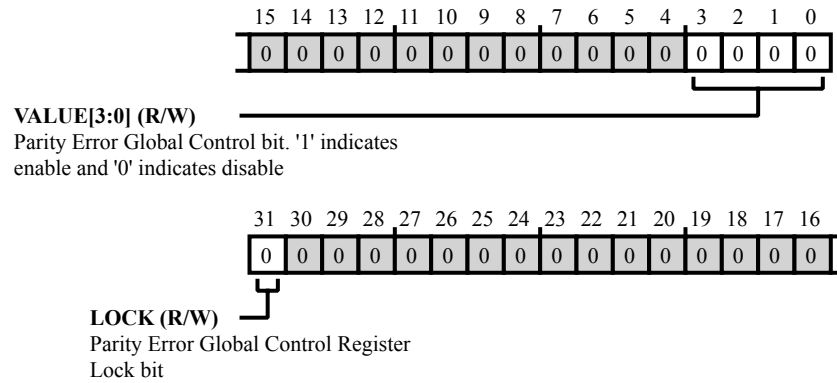


Figure 44-15: MEC_PEIRQ_GCTL[p] Register Diagram

Table 44-20: MEC_PEIRQ_GCTL[p] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Parity Error Global Control Register Lock bit. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>MEC_PEIRQ_GCTL[p].LOCK</code> bit is enabled, the <code>MEC_PEIRQ_GCTL[p]</code> register is read only.
		0 Unlock
		1 Lock
3:0 (R/W)	VALUE	Parity Error Global Control bit. '1' indicates enable and '0' indicates disable.

Parity Error Interrupt Request Global Status Register

MEC_PEIRQ_GSTAT[p] register bits reflect status of parity error interrupt/trigger outputs.

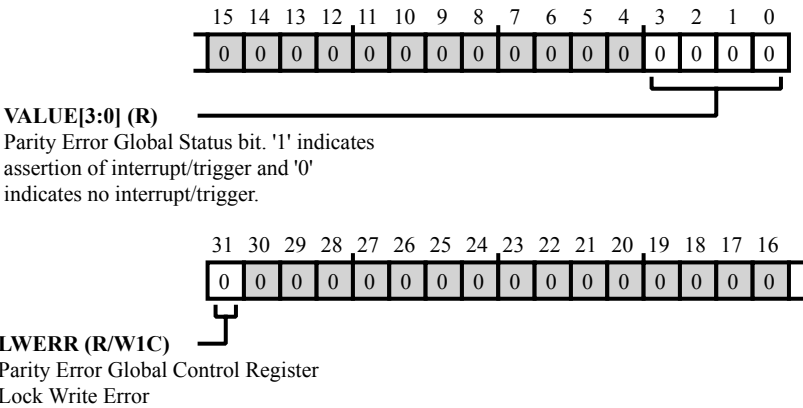


Figure 44-16: MEC_PEIRQ_GSTAT[p] Register Diagram

Table 44-21: MEC_PEIRQ_GSTAT[p] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Parity Error Global Control Register Lock Write Error. The MEC_PEIRQ_GSTAT[p] . LWERR bit indicates (when set) there was an attempted write to an MEC register while the MEC_PEIRQ_GCTL[p] . LOCK bit was set and while the global lock bit was enabled (SPU_CTL . GLCK bit =1). This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
3:0 (R/NW)	VALUE	Parity Error Global Status bit. '1' indicates assertion of interrupt/trigger and '0' indicates no interrupt/trigger..

Parity Error Control Register

[MEC_PERR_CTL0](#) register bits control enable/disable for parity error inputs from various cores/peripherals and decide whether their status will be reflected in parity error status register bits.

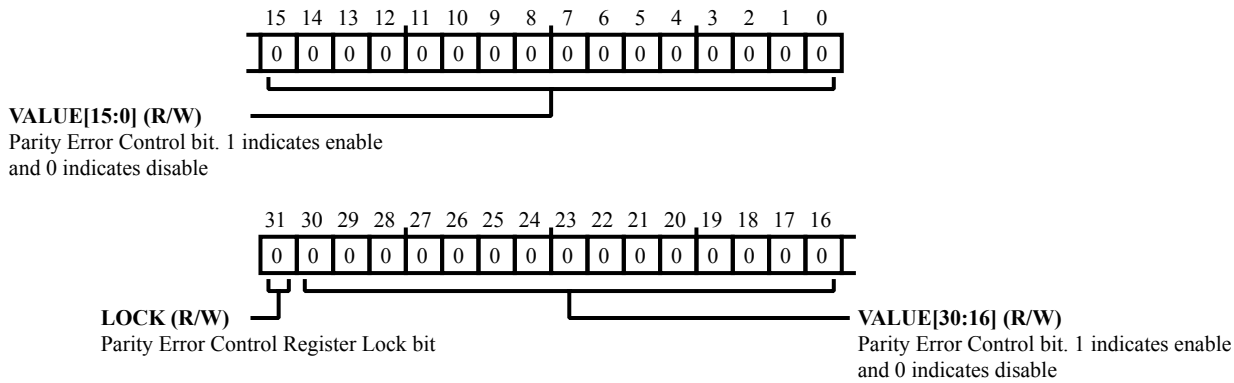


Figure 44-17: MEC_PERR_CTL0 Register Diagram

Table 44-22: MEC_PERR_CTL0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Parity Error Control Register Lock bit. If the global lock is enabled (SPU_CTL . GLCK bit =1) and the MEC_PERR_CTL0 . LOCK bit is enabled, the MEC_PERR_CTL0 register is read only.
		0 Unlock
		1 Lock
30:0 (R/W)	VALUE	Parity Error Control bit. 1 indicates enable and 0 indicates disable.
		1 Core 0 L1 Cache RAM Parity Error
		2 Core 0 L2 Cache RAM Parity Error
		4 Core 1 L1 RAM Parity Error
		8 Core 1 L1 Cache Parity Error
		16 Core 1 Branch Predictor Parity Error
		32 Core 2 L1 RAM Parity Error
		64 Core 2 L1 Cache Parity Error
		128 Core 2 Branch Predictor Parity Error
		256 ASRC0 FIFO Parity Error
		512 ASRC1 FIFO Parity Error
		1024 ASRC2 FIFO Parity Error

Table 44-22: MEC_PERR_CTL0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		2048 ASRC3 FIFO Parity Error
		4096 ASRC4 FIFO Parity Error
		8192 ASRC5 FIFO Parity Error
		16384 ASRC6 FIFO Parity Error
		32768 ASRC7 FIFO Parity Error
		65536 SHARC 0 IIR0 RAM Parity Error
		131072 SHARC 0 IIR1 RAM Parity Error
		262144 SHARC 0 IIR2 RAM Parity Error
		524288 SHARC 0 IIR3 RAM Parity Error
		1048576 SHARC 0 FIR0 RAM Parity Error
		2097152 SHARC 1 IIR0 RAM Parity Error
		4194304 SHARC 1 IIR1 RAM Parity Error
		8388608 SHARC 1 IIR2 RAM Parity Error
		16777216 SHARC 1 IIR3 RAM Parity Error
		33554432 SHARC 1 FIR0 RAM Parity Error
		67108864 USB0 FIFO RAM Parity Error
		134217728 TRNG0 Data Buffer Parity Error
		268435456 PKA0 Data RAM Parity Error
		536870912 SPE0 Buffer RAM Parity Error
		1073741824 SPE0 ARC4 RAM Parity Error

Parity Error Control Register

[MEC_PERR_CTL1](#) register bits control enable/disable for parity error inputs from various cores/peripherals and decide whether their status will be reflected in parity error status register bits.

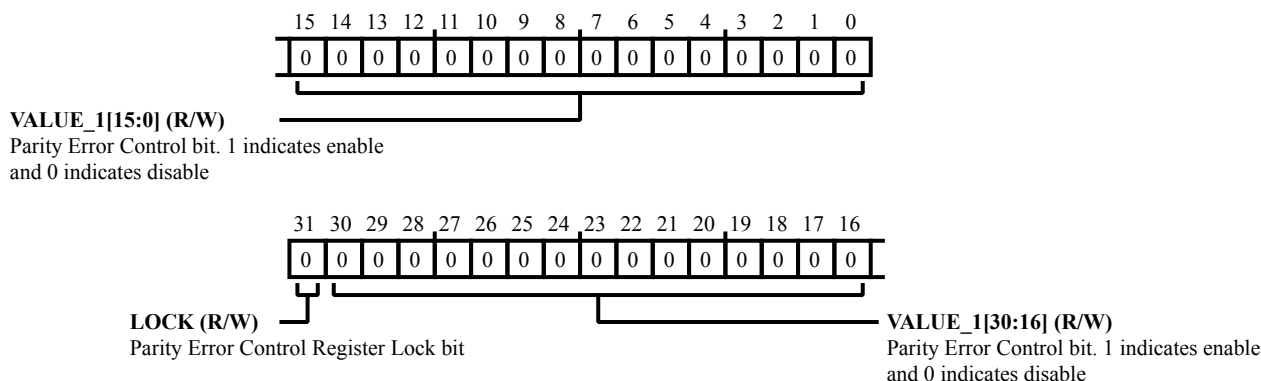


Figure 44-18: MEC_PERR_CTL1 Register Diagram

Table 44-23: MEC_PERR_CTL1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Parity Error Control Register Lock bit.
		If the global lock is enabled (SPU_CTL . GLCK bit =1) and the MEC_PERR_CTL1 . LOCK bit is enabled, the MEC_PERR_CTL1 register is read only.
		0 Unlock 1 Lock
30:0 (R/W)	VALUE_1	Parity Error Control bit. 1 indicates enable and 0 indicates disable.
		1 EMAC0 Transmit FIFO RAM Parity Error
		2 EMAC0 Receive FIFO RAM Parity Error
		4 MLB0 Data Buffer RAM Parity Error
		8 MLB0 Channel Table RAM Parity Error
		16 TMC0 Trace Data RAM Parity Error
		32 EMAC1 Transmit FIFO RAM Parity Error 64 EMAC1 Receive FIFO RAM Parity Error

Parity Error Interrupt Mask Register

MEC_PERR_IMASK0 register bits control interrupt masks for parity error inputs from various cores/peripherals.

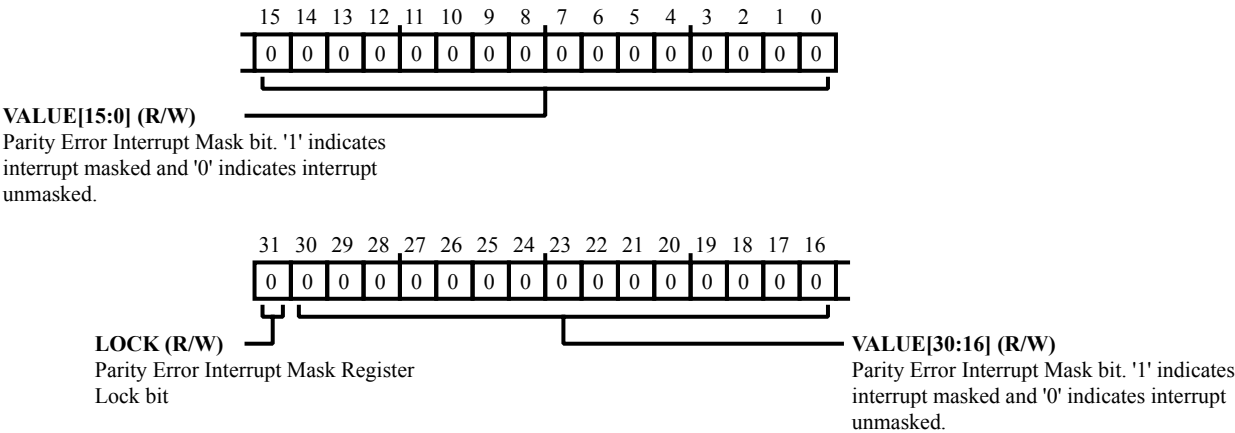


Figure 44-19: MEC_PERR_IMASK0 Register Diagram

Table 44-24: MEC_PERR_IMASK0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Parity Error Interrupt Mask Register Lock bit.
		If the global lock is enabled (SPU_CTL . GLCK bit =1) and the MEC_PERR_IMASK0 . LOCK bit is enabled, the MEC_PERR_IMASK0 register is read only.
		0 Unlock 1 Lock
30:0 (R/W)	VALUE	Parity Error Interrupt Mask bit. '1' indicates interrupt masked and '0' indicates interrupt unmasked..
		1 Core 0 L1 Cache RAM Parity Error
		2 Core 0 L2 Cache RAM Parity Error
		4 Core 1 L1 RAM Parity Error
		8 Core 1 L1 Cache Parity Error
		16 Core 1 Branch Predictor Parity Error
		32 Core 2 L1 RAM Parity Error
		64 Core 2 L1 Cache Parity Error
		128 Core 2 Branch Predictor Parity Error
		256 ASRC0 FIFO Parity Error
		512 ASRC1 FIFO Parity Error

Table 44-24: MEC_PERR_IMASK0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		1024 ASRC2 FIFO Parity Error
		2048 ASRC3 FIFO Parity Error
		4096 ASRC4 FIFO Parity Error
		8192 ASRC5 FIFO Parity Error
		16384 ASRC6 FIFO Parity Error
		32768 ASRC7 FIFO Parity Error
		65536 SHARC 0 IIR0 RAM Parity Error
		131072 SHARC 0 IIR1 RAM Parity Error
		262144 SHARC 0 IIR2 RAM Parity Error
		524288 SHARC 0 IIR3 RAM Parity Error
		1048576 SHARC 0 FIR0 RAM Parity Error
		2097152 SHARC 1 IIR0 RAM Parity Error
		4194304 SHARC 1 IIR1 RAM Parity Error
		8388608 SHARC 1 IIR2 RAM Parity Error
		16777216 SHARC 1 IIR3 RAM Parity Error
		33554432 SHARC 1 FIR0 RAM Parity Error
		67108864 USB0 FIFO RAM Parity Error
		134217728 TRNG0 Data Buffer Parity Error
		268435456 PKA0 Data RAM Parity Error
		536870912 SPE0 Buffer RAM Parity Error
		1073741824 SPE0 ARC4 RAM Parity Error

Parity Error Interrupt Mask Register

`MEC_PERR_IMASK1` register bits control interrupt masks for parity error inputs from various cores/peripherals.

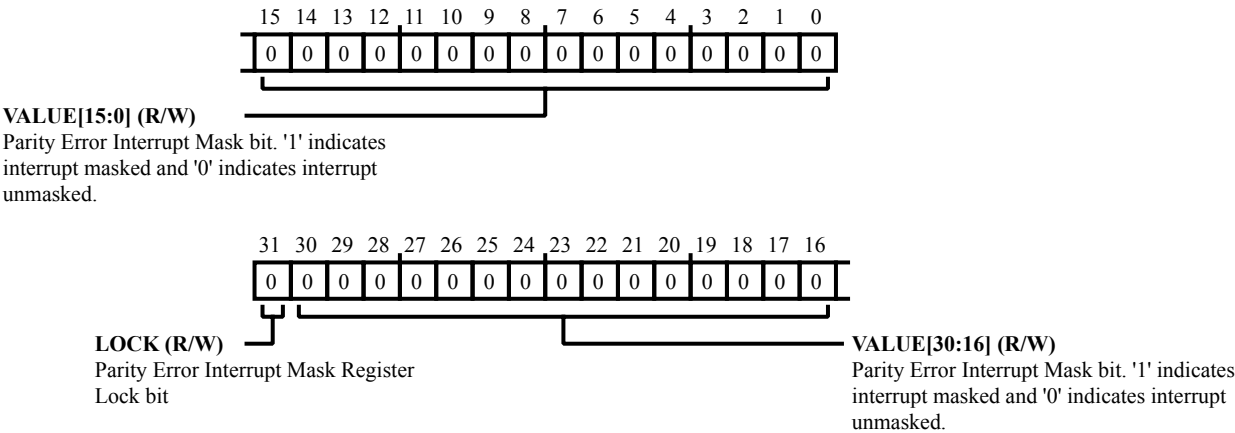


Figure 44-20: MEC_PERR_IMASK1 Register Diagram

Table 44-25: MEC_PERR_IMASK1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W)	LOCK	Parity Error Interrupt Mask Register Lock bit. If the global lock is enabled (<code>SPU_CTL.GLCK</code> bit =1) and the <code>MEC_PERR_IMASK1.LOCK</code> bit is enabled, the <code>MEC_PERR_IMASK1</code> register is read only.
		0 Unlock
		1 Lock
30:0 (R/W)	VALUE	Parity Error Interrupt Mask bit. '1' indicates interrupt masked and '0' indicates interrupt unmasked..
		1 EMAC0 Transmit FIFO RAM Parity Error
		2 EMAC0 Receive FIFO RAM Parity Error
		4 MLB0 Data Buffer RAM Parity Error
		8 MLB0 Channel Table RAM Parity Error
		16 TMC0 Trace Data RAM Parity Error
		32 EMAC1 Transmit FIFO RAM Parity Error
		64 EMAC1 Receive FIFO RAM Parity Error

Parity Error Status Register

MEC_PERR_STAT0 register bits reflect status for parity error inputs from various cores/peripherals. Writing '1' to these bits clear corresponding parity error status.

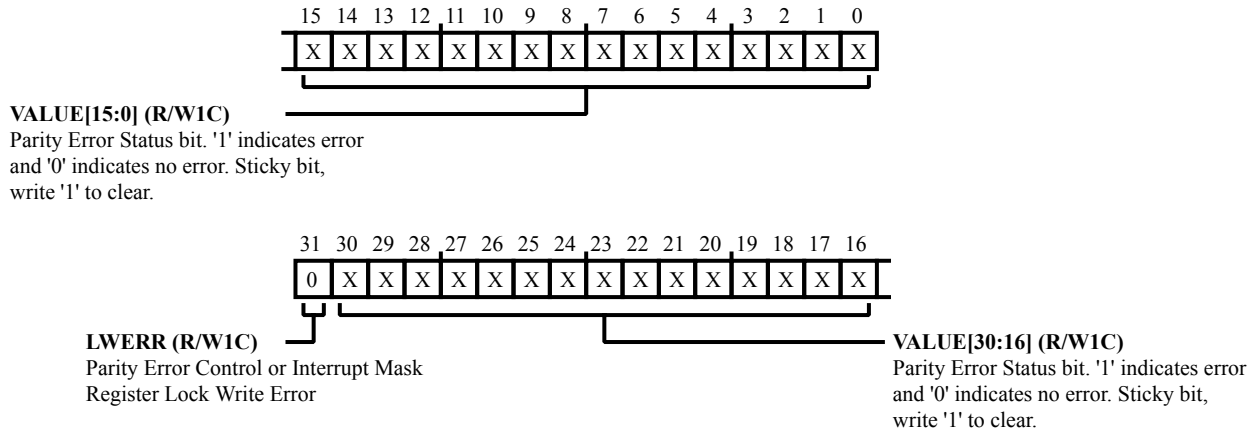


Figure 44-21: MEC_PERR_STAT0 Register Diagram

Table 44-26: MEC_PERR_STAT0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Parity Error Control or Interrupt Mask Register Lock Write Error. The <code>MEC_PERR_STAT0.LWERR</code> bit indicates (when set) there was an attempted write to an MEC register while the <code>MEC_PERR_CTL0.LOCK</code> bit was set and while the global lock bit was enabled (<code>SPU_CTL.GLCK</code> bit =1). This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
30:0 (R/W1C)	VALUE	Parity Error Status bit. '1' indicates error and '0' indicates no error. Sticky bit, write '1' to clear..
		1 Core 0 L1 Cache RAM Parity Error
		2 Core 0 L2 Cache RAM Parity Error
		4 Core 1 L1 RAM Parity Error
		8 Core 1 L1 Cache Parity Error
		16 Core 1 Branch Predictor Parity Error
		32 Core 2 L1 RAM Parity Error
		64 Core 2 L1 Cache Parity Error
		128 Core 2 Branch Predictor Parity Error

Table 44-26: MEC_PERR_STAT0 Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		256 ASRC0 FIFO Parity Error
		512 ASRC1 FIFO Parity Error
		1024 ASRC2 FIFO Parity Error
		2048 ASRC3 FIFO Parity Error
		4096 ASRC4 FIFO Parity Error
		8192 ASRC5 FIFO Parity Error
		16384 ASRC6 FIFO Parity Error
		32768 ASRC7 FIFO Parity Error
		65536 SHARC 0 IIR0 RAM Parity Error
		131072 SHARC 0 IIR1 RAM Parity Error
		262144 SHARC 0 IIR2 RAM Parity Error
		524288 SHARC 0 IIR3 RAM Parity Error
		1048576 SHARC 0 FIR0 RAM Parity Error
		2097152 SHARC 1 IIR0 RAM Parity Error
		4194304 SHARC 1 IIR1 RAM Parity Error
		8388608 SHARC 1 IIR2 RAM Parity Error
		16777216 SHARC 1 IIR3 RAM Parity Error
		33554432 SHARC 1 FIR0 RAM Parity Error
		67108864 USB0 FIFO RAM Parity Error
		134217728 TRNG0 Data Buffer Parity Error
		268435456 PKA0 Data RAM Parity Error
		536870912 SPE0 Buffer RAM Parity Error
		1073741824 SPE0 ARC4 RAM Parity Error

Parity Error Status Register

MEC_PERR_STAT1 register bits reflect status for parity error inputs from various cores/peripherals. Writing '1' to these bits clear corresponding parity error status.

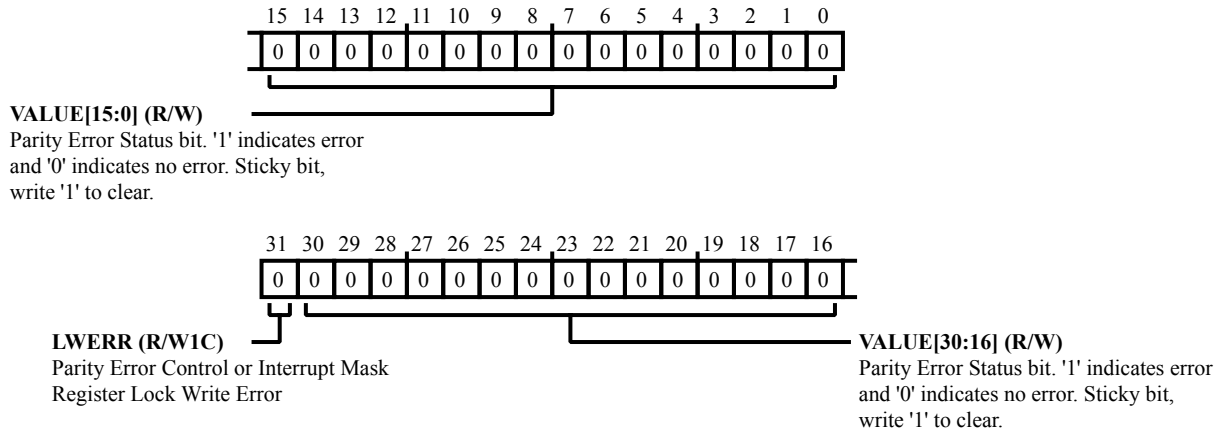


Figure 44-22: MEC_PERR_STAT1 Register Diagram

Table 44-27: MEC_PERR_STAT1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31 (R/W1C)	LWERR	Parity Error Control or Interrupt Mask Register Lock Write Error. The MEC_PERR_STAT1.LWERR bit indicates (when set) there was an attempted write to an MEC register while the MEC_PERR_CTL1.LOCK bit was set and while the global lock bit was enabled (SPU_CTL.GLCK bit =1). This status bit is sticky; write-1-to-clear it.
		0 No Error
		1 Error Occurred
30:0 (R/W)	VALUE	Parity Error Status bit. '1' indicates error and '0' indicates no error. Sticky bit, write '1' to clear..
		1 EMAC0 Transmit FIFO RAM Parity Error
		2 EMAC0 Receive FIFO RAM Parity Error
		4 MLB0 Data Buffer RAM Parity Error
		8 MLB0 Channel Table RAM Parity Error
		16 TMC0 Trace Data RAM Parity Error
		32 EMAC1 Transmit FIFO RAM Parity Error
		64 EMAC1 Receive FIFO RAM Parity Error

Peripheral ID0 Register

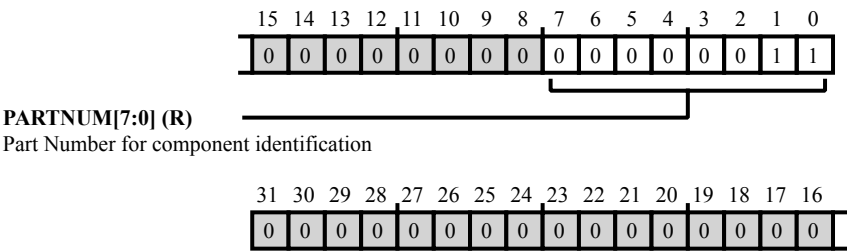


Figure 44-23: MEC_PID0 Register Diagram

Table 44-28: MEC_PID0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PARTNUM	Part Number for component identification.

Peripheral ID1 Register

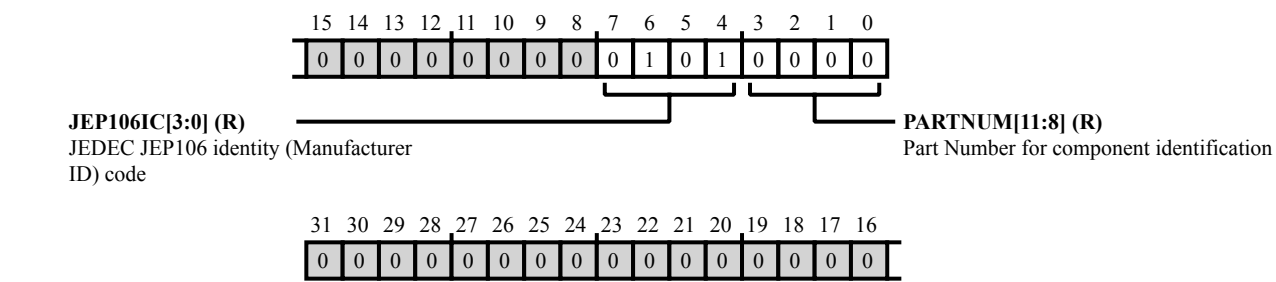


Figure 44-24: MEC_PID1 Register Diagram

Table 44-29: MEC_PID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	JEP106IC	JEDEC JEP106 identity (Manufacturer ID) code.
3:0 (R/NW)	PARTNUM	Part Number for component identification.

Peripheral ID2 Register

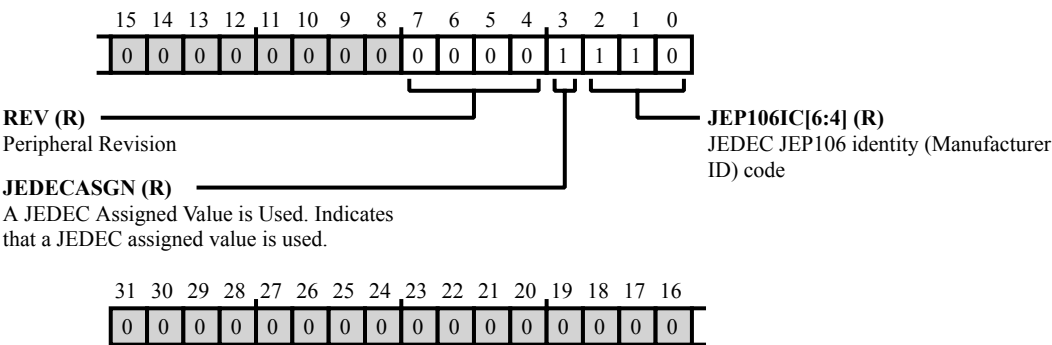


Figure 44-25: MEC_PID2 Register Diagram

Table 44-30: MEC_PID2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	REV	Peripheral Revision.
3 (R/NW)	JEDECASGN	A JEDEC Assigned Value is Used. Indicates that a JEDEC assigned value is used..
2:0 (R/NW)	JEP106IC	JEDEC JEP106 identity (Manufacturer ID) code.

Peripheral ID3 Register

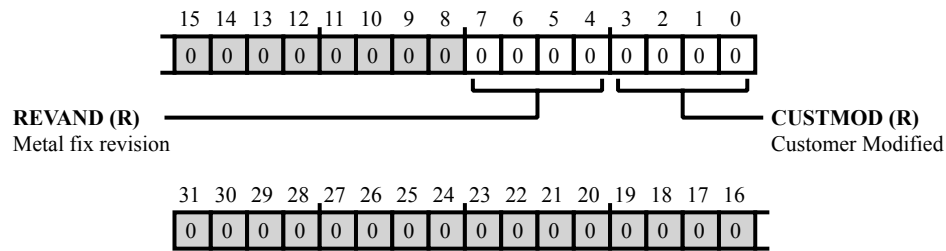


Figure 44-26: MEC_PID3 Register Diagram

Table 44-31: MEC_PID3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	REVAND	Metal fix revision.
3:0 (R/NW)	CUSTMOD	Customer Modified.

Peripheral ID4 Register

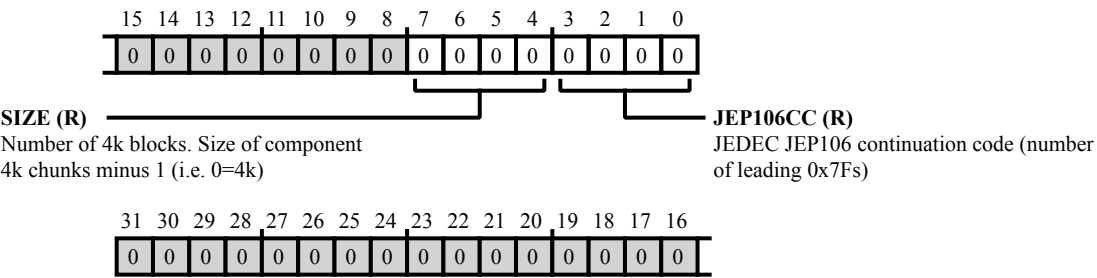


Figure 44-27: MEC_PID4 Register Diagram

Table 44-32: MEC_PID4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	SIZE	Number of 4k blocks. Size of component 4k chunks minus 1 (i.e. 0=4k).
3:0 (R/NW)	JEP106CC	JEDEC JEP106 continuation code (number of leading 0x7Fs).

Peripheral ID5 Register

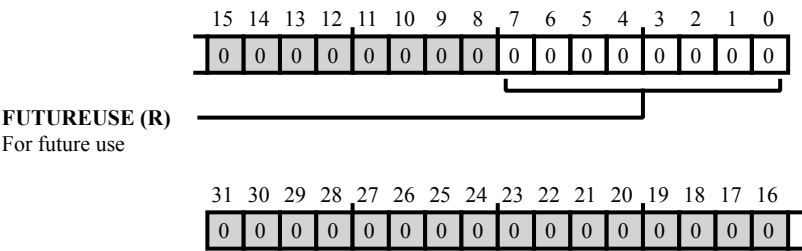


Figure 44-28: MEC_PID5 Register Diagram

Table 44-33: MEC_PID5 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	FUTUREUSE	For future use.

Peripheral ID6 Register

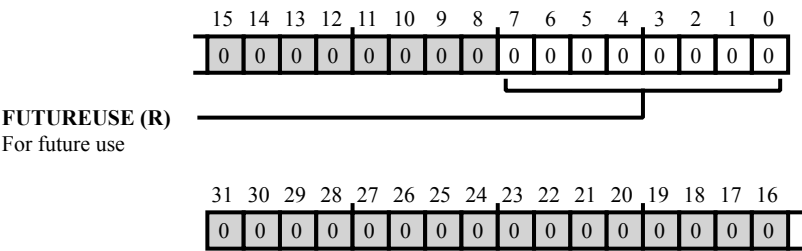


Figure 44-29: MEC_PID6 Register Diagram

Table 44-34: MEC_PID6 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	FUTUREUSE	For future use.

Peripheral ID7 Register

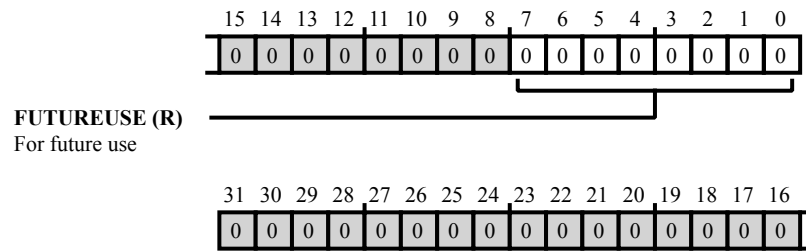


Figure 44-30: MEC_PID7 Register Diagram

Table 44-35: MEC_PID7 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	FUTUREUSE	For future use.

45 System Debug and Trace Unit (DBG)

The system debug and trace unit is based on Arm CoreSight technology. CoreSight is a set of architecture specifications defining debug and trace architecture. The processor uses CoreSight infrastructure to provide industry standard debug and trace capabilities.

<http://infocenter.arm.com/help/>

The applicable documentation for more details about the ArmCoreSight feature includes:

- CoreSight PFT Architecture Specification , ARM IHI 0035B (PFT)
- System Trace Macrocell, Programmers' Model Architecture Specification, ARM IHI 0054A (STM)
- CoreSight Trace Memory Controller, ARM DDI0461B (TMC)
- CoreSight Components Technical Reference Manual, ARM DDI 0314H (TPIU)
- Embedded Cross Trigger Technical Reference Manual, ARM DDI 0291A
- ETM for A5 Technical Reference Manual, DDI0435C

DBG Features

The system debug and trace unit contains the following features.

- System JTAG TAP controller for system debug features, boundary scan, and public JTAG features
- A debug interface to core, and other system resources
- Direct and run-time access to the memory system and system MMRs
- Direct control over system reset
- Support for debug immediately after reset (boot debug)
- Group halt (debug event immediately halts all specified endpoints)
- Real-time on-chip visibility is made available to all developers, including software developers

DBG Functional Description

The following sections provide functional descriptions of the DBG unit.

ADSP-2159x CSPFT Register List

Table 45-1: ADSP-2159x CSPFT Register List

Name	Description
CSPFT_ACTR[n]	Address Comparator Access Type Register
CSPFT_ACVR[n]	Address Comparator Value Register
CSPFT_AUTHSTATUS	Authentication Status Register
CSPFT_CCER	Configuration Code Extension Register
CSPFT_CID0	Component ID0 Register
CSPFT_CID1	Component ID1 Register
CSPFT_CID2	Component ID2 Register
CSPFT_CID3	Component ID3 Register
CSPFT_CIDCMR	Context ID Comparator Mask Register
CSPFT_CIDCVR[n]	Context ID Comparator Value
CSPFT_CLAIMCLR	Claim Tag Clear Register
CSPFT_CLAIMSET	Claim Tag Set Register
CSPFT_CNTENR[n]	Counter Enable Event Register
CSPFT_CNTRLDEVR[n]	Counter Reload Event Register
CSPFT_CNTRLDVR[n]	Counter Reload Value Register
CSPFT_CNTVR[n]	Counter Value Register
CSPFT_CTL	Main Control Register
CSPFT_DEVTYPE	Device Type Identifier Register
CSPFT_EXTOUTEVR[n]	External Output Event Register
CSPFT_HWFEBT	Hardware Feature Register
CSPFT_LAR	Lock Access Register
CSPFT_LSR	Lock Status Register
CSPFT_PID0	Peripheral ID0 Register
CSPFT_PID1	Peripheral ID1 Register
CSPFT_PID2	Peripheral ID2 Register
CSPFT_PID3	Peripheral ID3 Register
CSPFT_PID4	Peripheral ID4 Register

Table 45-1: ADSP-2159x CSPFT Register List (Continued)

Name	Description
CSPFT_STAT	Status Register
CSPFT_SYNCFR	Synchronization Frequency Register
CSPFT_TECTL	TraceEnable Control Register
CSPFT_TEEVENT	TraceEnable Event Register
CSPFT_TRACEIDR	CoreSight Trace ID Register
CSPFT_TRIGGER	Trigger Event Register
CSPFT_TSSCTL	TraceEnable Start/Stop Control Register

ADSP-2159x TAPC Register List

The Test Access Port Controller (TAPC) provides access to debug features. A set of registers governs TAPC operations. For more information on TAPC functionality, see the TAPC register descriptions.

Table 45-2: ADSP-2159x TAPC Register List

Name	Description
TAPC_DBGCTL	Debug Control Register
TAPC_IDCODE	IDCODE Register
TAPC_SDBGKEY0	Secure Debug Key 0 Register
TAPC_SDBGKEY1	Secure Debug Key 1 Register
TAPC_SDBGKEY2	Secure Debug Key 2 Register
TAPC_SDBGKEY3	Secure Debug Key 3 Register
TAPC_SDBGKEY_CTL	Secure Debug Key Control Register
TAPC_SDBGKEY_STAT	Secure Debug Key Status Register
TAPC_USERCODE	USERCODE Register

DBG Block Diagram

The block diagram is shown below.

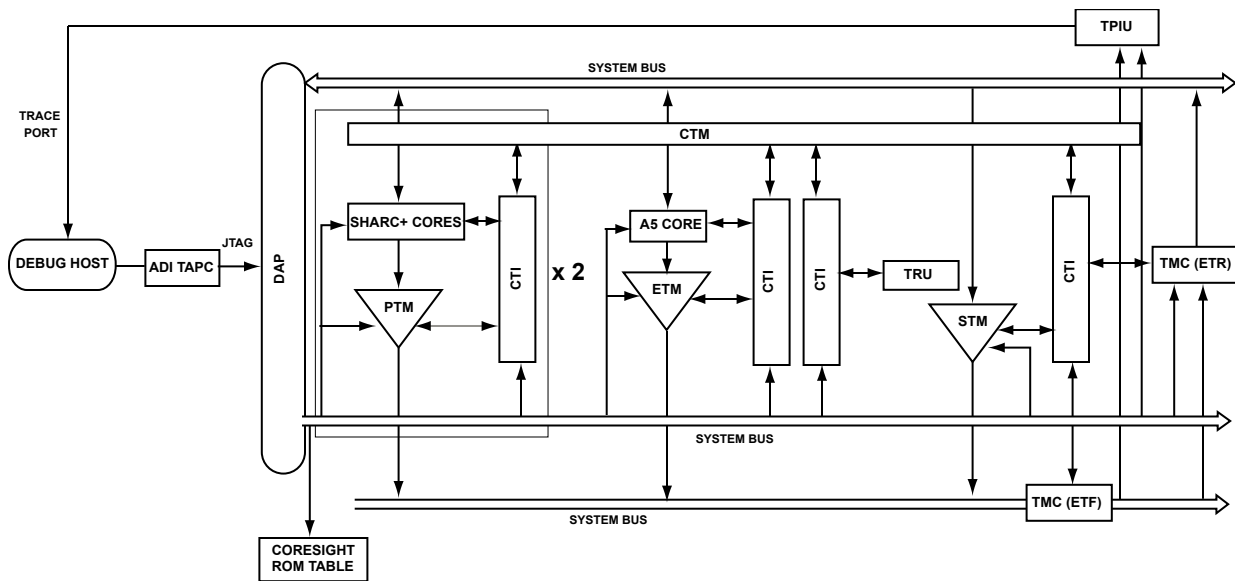


Figure 45-1: ADSP-SC5xx Block Diagram

DBG Definitions

The following terms are useful when working with the debug features of the processor and programming tools.

Test Access Port Controller (TAPC)

Provides IDCODE and SDBGKEY features.

Debug Access Port (DAP)

Core Sight Interface providing a single port for two debug options: JTAG-DP (JTAG Debug Port), SW-DP (Serial Wire Debug Port)

Embedded Trace Macrocell (ETM)

Provides Arm Core Trace.

Program Trace Macrocell (PTM)

Provides DSP (SHARC +) Core Trace.

Standard Trace Macrocell (STM)

Provides capability to trace up to 32 hardware events and supports 32 software stimulus for data transfer between the user and emulator.

Embedded Cross Trigger (ECT)

The ECTs are responsible managing events and triggers as follows.

- CTI (Cross Trigger Interface) is a CoreSight component for enabling cross triggering of events across a system. From the view of the ECT, it is responsible for combining and mapping trigger requests.
- CTM (Cross Trigger Matrix) is a CoreSight component for connecting multiple CTIs. From the view of the ECT, it is responsible for connecting CTIs and distribution of events.

NOTE: An embedded cross trigger is not the same as the master and slave trigger in the trigger routing unit.

Trace Capture Devices

The trace capture devices capture and format the trace data. There are two trace capture devices in the system:

- ETF - Embedded Trace Funnel - Provides a buffer for burst trace data.
- ETR - Embedded Trace Router - Provides interface for trace data to be stored in system memories.

Trace Port Interface Unit (TPIU)

The TPIU acts as a bridge between the on-chip trace data, with separate IDs, to a data stream. It encapsulates IDs, when required, that the Trace Port Analyzer (TPA) captures.

Serial Wire Output (SWO)

SWO is a trace data drain that acts as a bridge between the on-chip trace data and a data stream that the Trace Port Analyzer (TPA) captures.

Test Access Port Controller (TAPC)

TAPC is an independent component daisy-chained to the DAP in the JTAG scan path. The TAPC component provides the product IDCODE (Chip ID) and a security feature.

The TAPC component provides security features to the chip using a debug key match features. This feature permits only those components which have the SDBGKEY to connect and debug the chip. Initially, a 128-bits user key is programmed in the SDBGKEY registers ([TAPC_SDBGKEY0](#), [TAPC_SDBGKEY1](#), [TAPC_SDBGKEY2](#), [TAPC_SDBGKEY3](#)) in TAPC by a secure master. The `TAPC_SDBGKEY_CTL.VALID` bit is set.

Then, a user key is entered through the emulator for a match to the initially programmed-entered user key. On a successful match, the chip connects to the emulator. It can be debugged on a failed user key match. Further attempts at keys matching are disabled. JTAG reset or system reset is required to reenables the user key match logic.

Embedded Trace Macrocell (ETM)

ETM is the standard trace support provided for the A5 processor. For details of programming and functionality, refer to the Arm documentation.

Debug Access Ports

The Debug Access Port (DAP) is an implementation of an Arm Debug Interface version 5 (ADIV5) comprising a number of components supplied in a single configuration. All the supplied components fit into the various architectural components for Debug Ports (DPs). The components are used to access the DAP from an external debugger and APs, to access on-chip system resources.

The DAP provides access to all debug and trace capabilities through a single external interface, a JTAG Debug Port (JTAG-DP). The JTAG-DP is based on the IEEE 1149.1 Test Access Port (TAP) and Boundary Scan Architecture. DAP has some additions to support BSCA and IDCODE features that Core Sight DAP does not support.

Trace Unit

The trace module provides instruction, data tracing, and system activity tracing for the processor. The program trace module on the processor is similar to the embedded trace macrocell module provided by the Arm processor. System trace is provided using the system trace macrocell as part of the CoreSight debug and trace interface. The trace module uses an interface based on the AMBA Trace Bus (ATB) standard to output its trace data. The trace data can be either exported to an off-chip trace port analyzer or captured on an on-chip buffer. The PFT and STM modules capture information on the processor both before and after a specific event. The modules add no burden to the processor performance when it runs at full speed.

- [Programmable Flow Trace \(CSPFT\)](#)
- [System Trace Module \(STM\)](#)
- [Embedded Trace Macrocell \(ETM\)](#)

Programmable Flow Trace (CSPFT)

When tracing processor execution, trace information can be generated for every instruction the processor executes. This information would be easy to interpret, but would require a prohibitively high trace bandwidth to get the trace data off the chip. With program flow tracing, only branch points are traced. The debugger uses the source code to infer the rest of the executed code.

Certain instructions in the program and events are identified as waypoints. A waypoint is a point where instruction execution involves a change of program flow. The CSPFT only traces those waypoints. These waypoints are:

- All indirect branches
- All direct branches
- Exceptions or interrupts
- Emulator debug entry and exit

When a waypoint occurs, trace data is generated to describe it. From this data and the source code, a trace decompressor can determine what instructions were executed and recreate the instruction flow. To allow the decompressor to calculate where it is in the source code, conditional instructions are marked as waypoints, regardless of whether they pass or fail their condition test. Events like interrupts or debug entry and exit can be promoted from non-waypoint instructions to waypoints to trace the interrupted program flow.

Tracing a waypoint implies the execution of all instructions from the target address of the previous waypoint up to the current waypoint. Non-waypoint instructions are not explicitly traced but the debugger must infer them using the source code. The concept of an instruction block is used throughout this manual and refers to the contiguous block of instructions between two waypoints.

The programming model and function is a subset of the PTM (Programmable Trace Module) of Arm.

System Trace Module (STM)

The STM is a trace source that is integrated into a CoreSight system, and is designed primarily for high-bandwidth trace of instrumentation embedded into software. The STM enables tracing of system activity from various sources:

- Instrumented software, using memory-mapped stimulus ports
- Hardware events

The STM supports the following features:

- Multiple software masters writing software instrumentation independently. Each master can use multiple stimulus ports.
- Time stamping of the system activity. The time stamp is a global time stamp which can be shared with other trace sources in the system to enable correlation of activity from multiple trace sources.
- Indication that specific events have occurred, such as a particular hardware event or a piece of software instrumentation. These events are known as triggers and can be indicated in the trace stream, or through signals to other system components.

Thirty-two hardware event resources are connected as output from the TRU which allows monitoring all of the hardware events that can generate a trigger.

Embedded Cross Trigger (ECT)

ECT provides an interface to the CoreSight debug system enabling the subsystems to interact (cross trigger) with each other. ECT provides a mechanism to forward debug events from one connected subsystem to another connected subsystem. The different subsystems connected to the ECT depend on the processor design. For example, in a multiprocessor system, the interface can be connected to each of the cores and one to the trace subsystem. For a uniprocessor system, the interface can include just the core and trace subsystem connection.

- CTI Cross trigger interface. A CoreSight component for enabling cross triggering of events across a system.
- CTM Cross trigger matrix. A CoreSight component for connecting multiple cross trigger interfaces.

The main function of the ECT (CTI and CTM) is to pass debug events from one connected subsystem to another connected subsystem.

For example, the ECT can communicate debug state information from the core to trace subsystem for a single processor system or to another core in a multiprocessor-based system. Program execution on both the subsystem can be stopped at the same time.

The *Trigger Flow* figure shows a simple debug trigger flow sequence. On each CTI, there are four channel, eight input, and eight output debug triggers. All the eight inputs and outputs can be mapped to a single channel or different channels based on the debug trigger to channel mapping. When a trigger input occurs, it creates a channel event. The channel event causes all the output debug triggers to be triggered. The embedded cross trigger depends on the debug trigger it connects to.

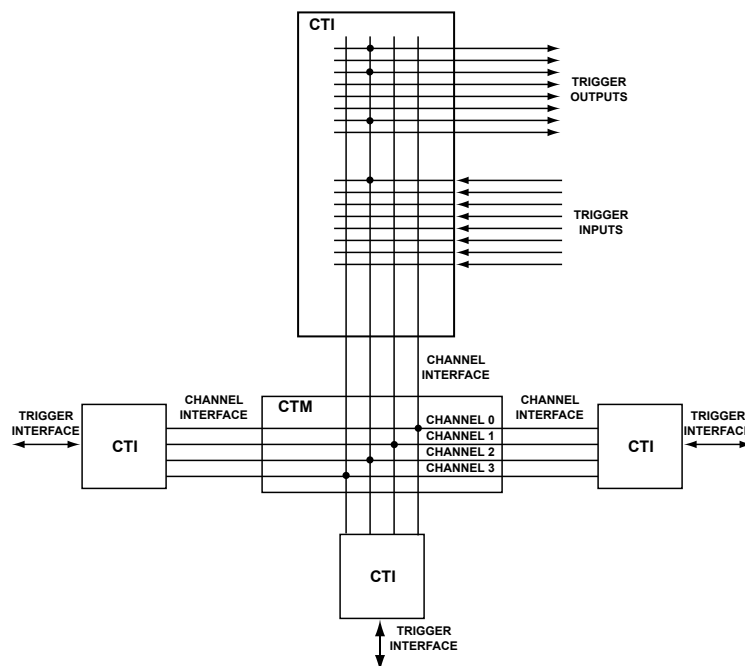


Figure 45-2: Trigger Flow

Refer to the *ECT Integration* figure. There are 5 CTIs in the system that connect to the core, trace, and system module, respectively. The CTIs all interconnect through the CTM. This configuration allows the core to trigger debug events on the trace, on the system and on the core itself. CTI0 handles all the ETM and core0 debug triggers. CTI1 handles all the PFT and core1 debug triggers. CTI2 handles all the PFT and core2 debug triggers. CTI3 handles all the system debug triggers. CTI4 handles all the trace components debug triggers.

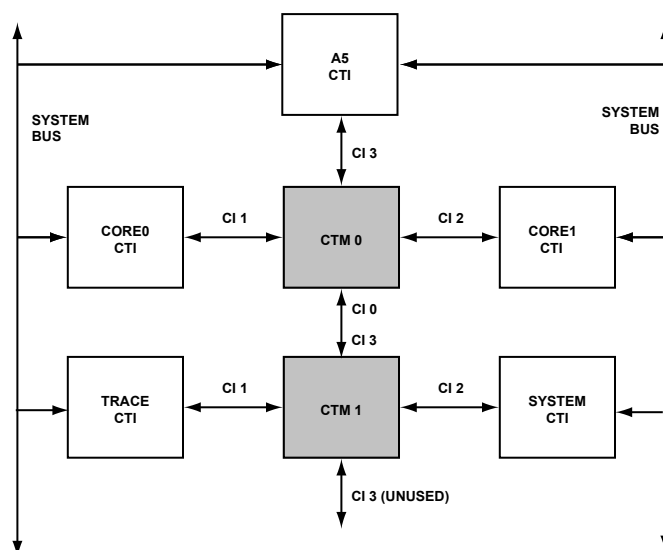


Figure 45-3: ECT Integration

CTI Debug Trigger Tables

The *System CTI Trigger Connection* tables show the debug trigger that connects to each CTI.

Table 45-3: System CTI Trigger Connection

CTI Port	Input	CTI Port	Output
CTITRIGIN[7]	From STM ASYNCOUT	CTITRIGOUT[7:6]	Unused
CTITRIGIN [6]	From STM TRIGOUTHETE		
CTITRIGIN [5]	From STM TRIGOUTSW	CTITRIGOUT[5]	To TPIU FLUSHIN
CTITRIGIN [4]	From STM TRIGOUTSPTE	CTITRIGOUT[4]	To ETR TRIGIN
CTITRIGIN [3]	From ETR	CTITRIGOUT[3]	To ETR FLUSHIN
CTITRIGIN [2]	From ETR ACQCOMP	CTITRIGOUT[2]	To ETR TRIGIN
CTITRIGIN [1]	From ETR	CTITRIGOUT[1]	To ETR FLUSHIN
CTITRIGIN [0]	From ACQCOMP	CTITRIGOUT[0]	To ETR TRIGIN

Table 45-4: ADSP-SC5xx-Core 1/2 CTI Trace Connection

CTI Port	Input	CTI Port	Output
CTITRIGIN[7]	Tied Low (Unused)	CTITRIGOUT[7]	DBGRESTART
CTITRIGIN[6]	PTM TRIGGER	CTITRIGOUT[6]	SEC

Table 45-4: ADSP-SC5xx-Core 1/2 CTI Trace Connection (Continued)

CTI Port	Input	CTI Port	Output
CTITRIGIN[5:2]	PTM EXTOUT[3:0]	CTITRIGOUT[4:1]	PTM EXTIN[3:0]
CTITRIGIN[1]	Tied Low (Unused)	CTITRIGOUT[5]	Unused
CTITRIGIN[0]	DBGTRIGGER	CTITRIGOUT[0]	EDBGRQ

Table 45-5: ADSP-2159x CTI Trace Connection

CTI Port	Input	CTI Port	Output
CTITRIGIN[7]	Tied Low (Unused)	CTITRIGOUT[7]	DBGRESTART
CTITRIGIN[6]	PTM TRIGGER	CTITRIGOUT[6]	SEC
CTITRIGIN[5:2]	PTM EXTOUT[3:0]	CTITRIGOUT[5]	Unused
CTITRIGIN[1]	Tied Low (Unused)	CTITRIGOUT[4:1]	PTM EXTIN[3:0]
CTITRIGIN[0]	DBGTRIGGER	CTITRIGOUT[0]	EDBGRQ

Table 45-6: SC5xx ARM Core CTI Connection

CTI Port	Input	CTI Port	Output
CTITRIGIN[7]	Tied Low	CTITRIGOUT[7]	DBGRESTART
CTITRIGIN[6]	ETM TRIGGER	CTITRIGOUT[6]	GEC
CTITRIGIN[5]	COMMRX	CTITRIGOUT[5]	Unused
CTITRIGIN[4]	COMMTX	CTITRIGOUT[4]	ETM EXTIN[3]
CTITRIGIN[3:2]	ETMEXTOUT[1:0]	CTITRIGOUT[3:2]	ETM EXTIN[2:1]
CTITRIGIN[1]	PMU IRQ	CTITRIGOUT[1]	ETM EXTIN[0]
CTITRIGIN[0]	DBGTRIGGER	CTITRIGOUT[0]	EDBGRQ

Table 45-7: Trigger Descriptions

Signal Name	Description
STM ASYNCOUT	Alignment synchronization output. This signal is asserted for one clock cycle when an ASYNC-VERSION-FREQ sequence is completely output on the ATB, and can be used for cross-triggering.
STM TRIGOUTHETE	Trigger output. This signal is asserted for one clock cycle when a trigger event is detected on match using STMHETER.
STM TRIGOUTSW	Trigger output. This signal is asserted for one clock cycle when a trigger event is generated on writes to a TRIG location in the extended stimulus port registers.
STM TRIGOUTSPTE	Trigger output. This signal is asserted for one clock cycle when a trigger event is detected on match using STMSPTER.

Table 45-7: Trigger Descriptions (Continued)

Signal Name	Description
ETR / ETF FULL	This output indicates the value of the full bit in the ETR/ETF status register. A full bit indicates the amount of data in ETF/ETR. An output signal indicating when the circular buffer or FIFO is full, or within a program-mable amount of being full.
ETR / ETF ACQCOMP	This output indicates the value of the FtEmpty bit in the ETR/ETF status register. The bit it is set when trace capture has stopped. An output signal indicating when trace capture has stopped, usually following a trigger condition.
ETR/ETF/ TPIU TRIGIN	This input can cause a trigger event (Start /Stop Trigger). An input signal indicating when a trigger condition has occurred.
ETR/ETF /TPIU FLUSHIN	This input can cause a Trace flush. An input signal indicating a flush request
PTM/ETM TRIGGER	Trigger Input. Trigger event specifies the conditions that must be met to generate a trigger.
PTM/ETM EXTOUT[3:0]	PTM Output.
PTM/ETM EXTIN[3:0]	PTM Input
SEC	CTI Interrupt
DBGRESTART	This is an output from the CTI to a processor core or to system to return from debug mode.
DBGTRIGGER	This is a processor core output signal indicating that the core has moved to debug mode. If the CTIs are setup for synchronous halt, it will generate EDBGQR to everyone else.
EDBGRQ	This is an output from CTI and an input (as debug halt) to a processor core and to the system in general. It can assert as a result of another core going to emulation space (DBGTRIGGER) or by setting the corresponding bit in the CTI.
SYS_DBGRESTART	This is an output from the CTI to system to return from debug mode.
To TRU	TRU Master Event
From TRU	TRU Slave Event

ADSP-2159x CSPFT Register Descriptions

Program Flow Trace (CSPFT) contains the following registers.

Table 45-8: ADSP-2159x CSPFT Register List

Name	Description
CSPFT_ACTR[n]	Address Comparator Access Type Register
CSPFT_ACVR[n]	Address Comparator Value Register
CSPFT_AUTHSTATUS	Authentication Status Register

Table 45-8: ADSP-2159x CSPFT Register List (Continued)

Name	Description
CSPFT_CCER	Configuration Code Extension Register
CSPFT_CID0	Component ID0 Register
CSPFT_CID1	Component ID1 Register
CSPFT_CID2	Component ID2 Register
CSPFT_CID3	Component ID3 Register
CSPFT_CIDCMR	Context ID Comparator Mask Register
CSPFT_CIDCVR[n]	Context ID Comparator Value
CSPFT_CLAIMCLR	Claim Tag Clear Register
CSPFT_CLAIMSET	Claim Tag Set Register
CSPFT_CNTENR[n]	Counter Enable Event Register
CSPFT_CNTRLDEVR[n]	Counter Reload Event Register
CSPFT_CNTRLDVR[n]	Counter Reload Value Register
CSPFT_CNTVR[n]	Counter Value Register
CSPFT_CTL	Main Control Register
CSPFT_DEVTYPE	Device Type Identifier Register
CSPFT_EXTOUTEVR[n]	External Output Event Register
CSPFT_HWFEAT	Hardware Feature Register
CSPFT_LAR	Lock Access Register
CSPFT_LSR	Lock Status Register
CSPFT_PID0	Peripheral ID0 Register
CSPFT_PID1	Peripheral ID1 Register
CSPFT_PID2	Peripheral ID2 Register
CSPFT_PID3	Peripheral ID3 Register
CSPFT_PID4	Peripheral ID4 Register
CSPFT_STAT	Status Register
CSPFT_SYNCFR	Synchronization Frequency Register
CSPFT_TECTL	TraceEnable Control Register
CSPFT_TEEVENT	TraceEnable Event Register
CSPFT_TRACEIDR	CoreSight Trace ID Register
CSPFT_TRIGGER	Trigger Event Register
CSPFT_TSSCTL	TraceEnable Start/Stop Control Register

Address Comparator Access Type Register

The `CSPFT_ACTR[n]` register specifies whether the context ID needs to match.

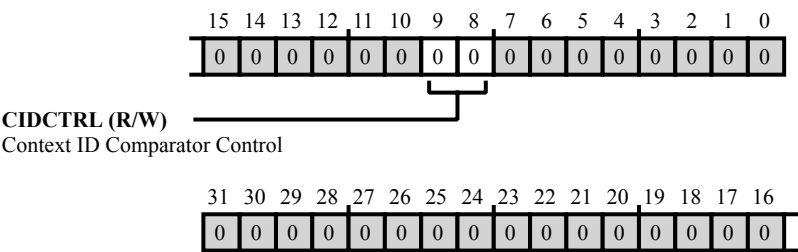


Figure 45-4: CSPFT_ACTR[n] Register Diagram

Table 45-9: CSPFT_ACTR[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
9:8 (R/W)	CIDCTRL	Context ID Comparator Control. The <code>CSPFT_ACTR[n].CIDCTRL</code> contains the ID comparator control value.
		0 Ignore Context ID
		1 Match if Context ID Comparator 0 Matches
		2 Match if Context ID Comparator 1 Matches
		3 Match if Context ID Comparator 2 Matches

Address Comparator Value Register

The `CSPFT_ACVR[n]` register holds an address for comparison.

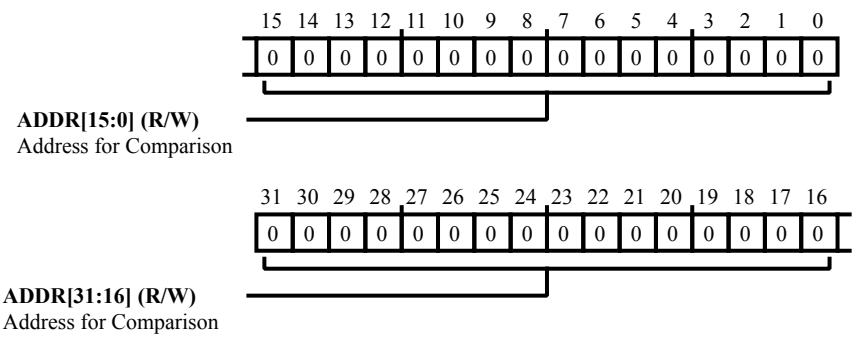


Figure 45-5: CSPFT_ACVR[n] Register Diagram

Table 45-10: CSPFT_ACVR[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	ADDR	Address for Comparison. The <code>CSPFT_ACVR[n].ADDR</code> bit field contains the address used for comparison.

Authentication Status Register

The `CSPFT_AUTHSTATUS` register reports the level of tracing currently permitted based on the DBGEN signal.

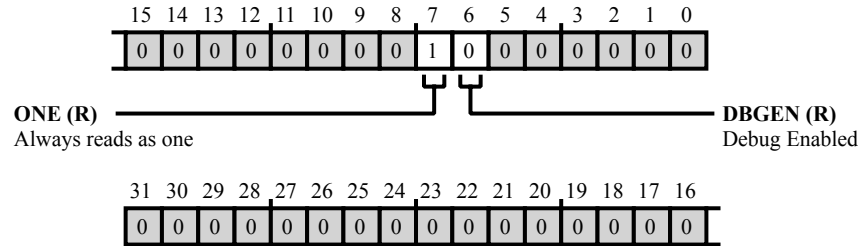


Figure 45-6: CSPFT_AUTHSTATUS Register Diagram

Table 45-11: CSPFT_AUTHSTATUS Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7 (R/NW)	ONE	Always reads as one.
6 (R/NW)	DBGEN	Debug Enabled. The <code>CSPFT_AUTHSTATUS.DBGEN</code> bit indicates that invasive debug is enabled. Normally, NIDEN is used in conjunction with a signal that enables invasive debug, DBGEN. Non-invasive debug is disabled only if both NIDEN and DBGEN signals are LOW. In a PTM, typically these signals are ORed together and the result is used to determine whether non-invasive debug is enabled.

Configuration Code Extension Register

The `CSPFT_CCER` register holds extra feature information. (See `CSPFT_HWFEAT`.)

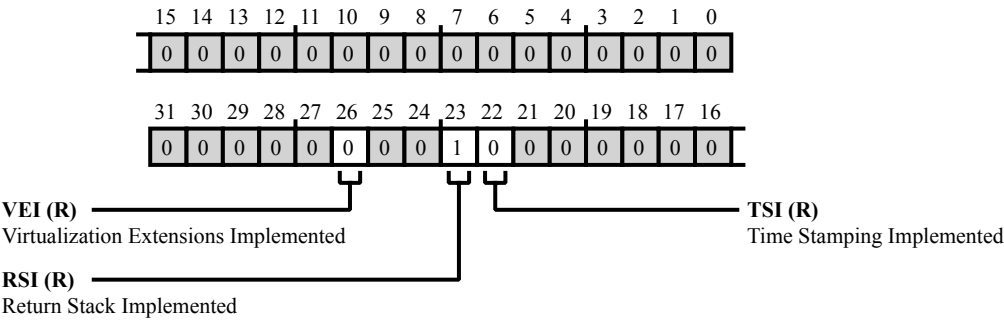


Figure 45-7: CSPFT_CCER Register Diagram

Table 45-12: CSPFT_CCER Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
26 (R/NW)	VEI	Virtualization Extensions Implemented. The <code>CSPFT_CCER.VEI</code> bit indicates if the virtualization extensions are implemented
		0 Not Implemented
		1 Implemented
23 (R/NW)	RSI	Return Stack Implemented. The <code>CSPFT_CCER.RSI</code> bit indicates if a return stack is implemented.
		0 Not Implemented
		1 Implemented
22 (R/NW)	TSI	Time Stamping Implemented. The <code>CSPFT_CCER.TSI</code> bit indicates if time stamping is implemented.
		0 Disabled
		1 Enabled

Component ID0 Register

The `CSPFT_CID0` register holds sections of the CoreSight Component ID for CSPFT.

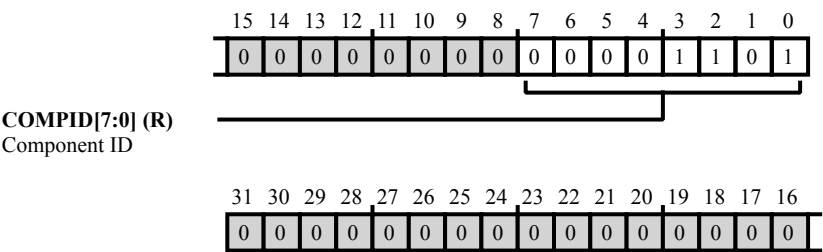


Figure 45-8: CSPFT_CID0 Register Diagram

Table 45-13: CSPFT_CID0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMPID	Component ID. The <code>CSPFT_CID0.COMPID</code> bit field identifies this component as a CoreSight component.

Component ID1 Register

The `CSPFT_CID1` register holds sections of the CoreSight Component ID for CSPFT.

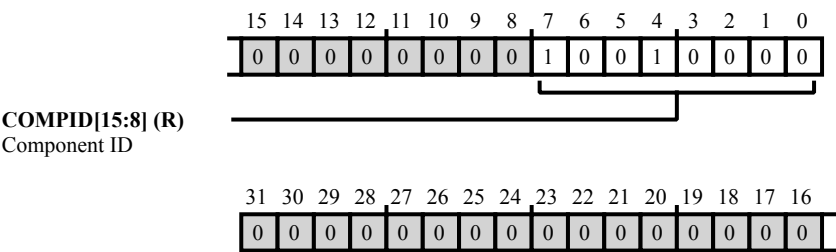


Figure 45-9: CSPFT_CID1 Register Diagram

Table 45-14: CSPFT_CID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMPID	Component ID. The <code>CSPFT_CID1.COMPID</code> bit field identifies this component as a CoreSight component.

Component ID2 Register

The `CSPFT_CID2` register holds sections of the CoreSight Component ID for CSPFT.

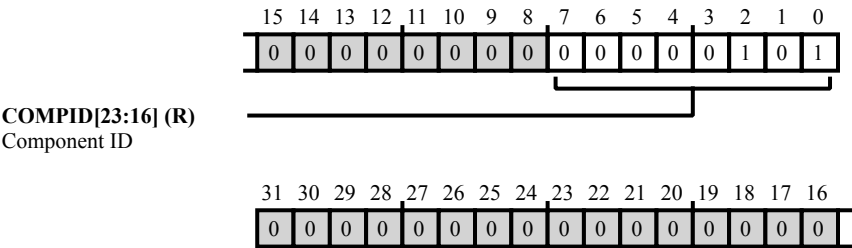


Figure 45-10: CSPFT_CID2 Register Diagram

Table 45-15: CSPFT_CID2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMPID	Component ID. The <code>CSPFT_CID2.COMPID</code> bit field identifies this component as a CoreSight component.

Component ID3 Register

The `CSPFT_CID3` register holds sections of the CoreSight Component ID for CSPFT.

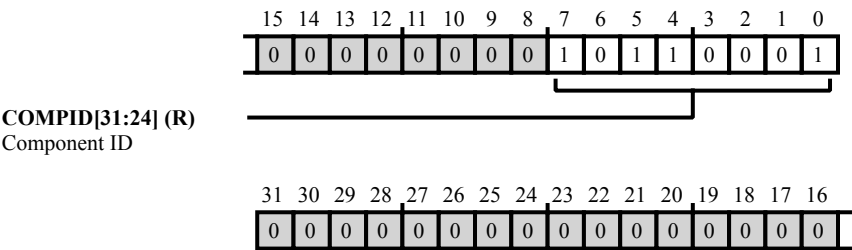


Figure 45-11: CSPFT_CID3 Register Diagram

Table 45-16: CSPFT_CID3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	COMPID	Component ID. The <code>CSPFT_CID3.COMPID</code> bit field identifies this component as a CoreSight component.

Context ID Comparator Mask Register

The `CSPFT_CIDCMR` register holds a 32-bit mask for use for all context ID comparisons.

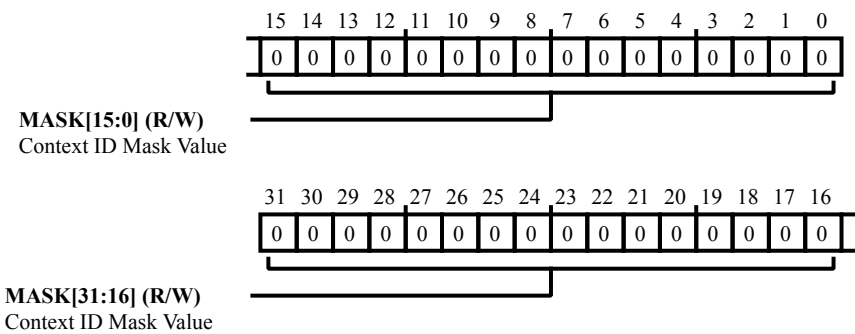


Figure 45-12: CSPFT_CIDCMR Register Diagram

Table 45-17: CSPFT_CIDCMR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	MASK	Context ID Mask Value. The <code>CSPFT_CIDCMR.MASK</code> bit field holds a 32-bit mask for use in all context ID comparisons.

Context ID Comparator Value

The `CSPFT_CIDCVR[n]` register holds a context ID value for comparison.

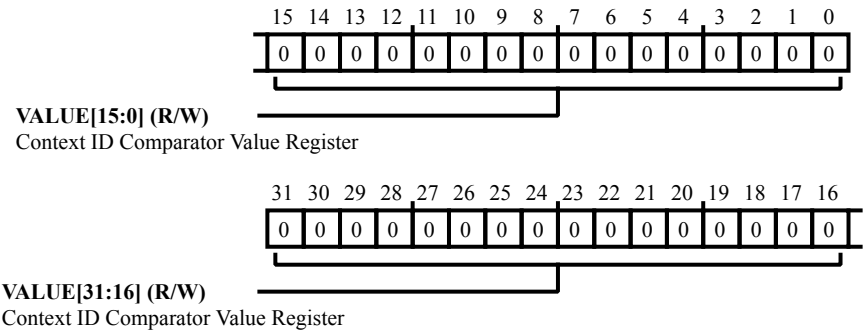


Figure 45-13: `CSPFT_CIDCVR[n]` Register Diagram

Table 45-18: `CSPFT_CIDCVR[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	Context ID Comparator Value Register. The <code>CSPFT_CIDCVR[n].VALUE</code> bit field holds a context ID value for comparison.

Claim Tag Clear Register

The `CSPFT_CLAIMCLR` register is used to clear bits in the claim tag or get the current value of the claim tag.

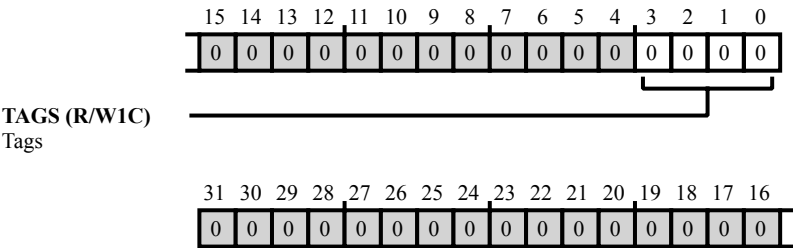


Figure 45-14: CSPFT_CLAIMCLR Register Diagram

Table 45-19: CSPFT_CLAIMCLR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R/W1C)	TAGS	Tags. A read of the <code>CSPFT_CLAIMCLR.TAGS</code> bit field returns the current value, a write clears bits.

Claim Tag Set Register

The `CSPFT_CLAIMSET` register is used to set bits in the claim tag and find the number of bits supported by the claim tag.

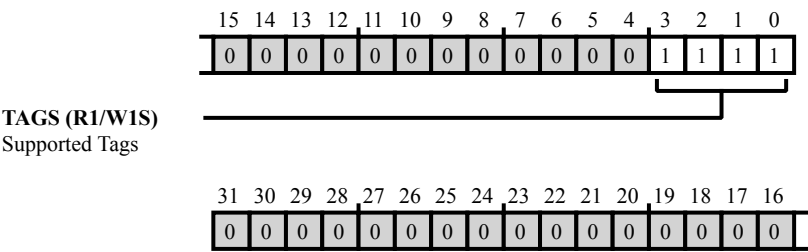


Figure 45-15: CSPFT_CLAIMSET Register Diagram

Table 45-20: CSPFT_CLAIMSET Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3:0 (R1/W1S)	TAGS	Supported Tags. The <code>CSPFT_CLAIMSET.TAGS</code> bit field sets bits in the claim tag and finds the number of bits supported by the claim tag.

Counter Enable Event Register

The `CSPFT_CNTENR[n]` register describes the event that enables the corresponding counter.

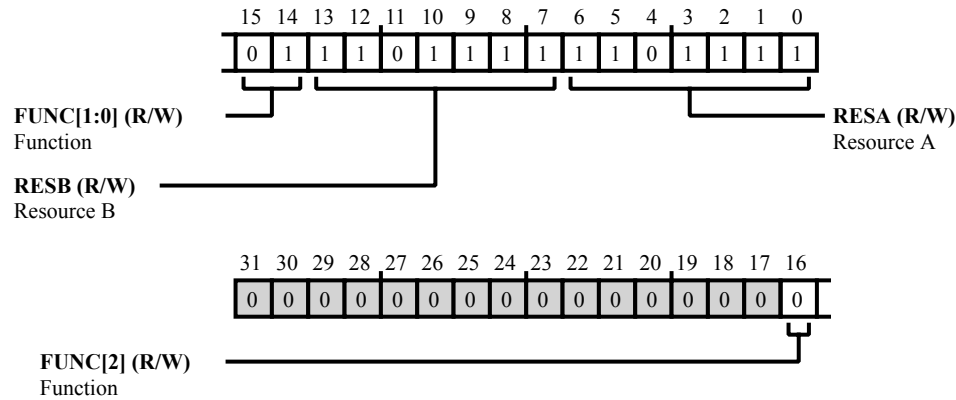


Figure 45-16: `CSPFT_CNTENR[n]` Register Diagram

Table 45-21: `CSPFT_CNTENR[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:14 (R/W)	FUNC	Function. The <code>CSPFT_CNTENR[n]</code> . FUNC bit field specifies the logical operation that combines the two resources that define the event.
		0 A
		1 NOT(A)
		2 A AND B
		3 NOT(A) AND B
		4 NOT(A) AND NOT(B)
		5 A OR B
		6 NOT(A) OR B
		7 NOT(A) OR NOT(B)
13:7 (R/W)	RESB	Resource B. The <code>CSPFT_CNTENR[n]</code> . RESB bit field specifies one of the two resources that can be combined by the logical operation specified in the <code>CSPFT_CNTENR[n]</code> . FUNC field (See <code>CSPFT_CNTENR[n]</code> . RESA for list of possible values).
6:0 (R/W)	RESA	Resource A. The <code>CSPFT_CNTENR[n]</code> . RESA bit field specifies one of the two resources that can be combined by the logical operation specified in the <code>CSPFT_CNTENR[n]</code> . FUNC field.

Table 45-21: CSPFT_CNTENR[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		0 Single Addr Comparator 0
		1 Single Addr Comparator 1
		2 Single Addr Comparator 2
		3 Single Addr Comparator 3
		4 Single Addr Comparator 4
		5 Single Addr Comparator 5
		6 Single Addr Comparator 6
		7 Single Addr Comparator 7
		8 Single Addr Comparator 8
		9 Single Addr Comparator 9
		10 Single Addr Comparator 10
		11 Single Addr Comparator 11
		12 Single Addr Comparator 12
		13 Single Addr Comparator 13
		14 Single Addr Comparator 14
		15 Single Addr Comparator 15
		16 Addr Range Comparator 0
		17 Addr Range Comparator 1
		18 Addr Range Comparator 2
		19 Addr Range Comparator 3
		20 Addr Range Comparator 4
		21 Addr Range Comparator 5
		22 Addr Range Comparator 6
		23 Addr Range Comparator 7
		64 Counter 0 at Zero
		65 Counter 1 at Zero
		66 Counter 2 at Zero
		67 Counter 3 at Zero
		88 Context ID Comparator 0
		89 Context ID Comparator 1

Table 45-21: CSPFT_CNTENR[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		90	Context ID Comparator 2
		95	TraceEnable Start/Stop Resource 0 or 1
		96	External Inputs 0
		97	External Inputs 1
		98	External Inputs 2
		99	External Inputs 3
		110	Trace Prohibited
		111	Always TRUE

Counter Reload Event Register

The `CSPFT_CNTRLDEVR[n]` register defines the event that causes the corresponding counter to be reloaded with the value held in the corresponding `CSPFT_CNTRLDVR[n]` register.

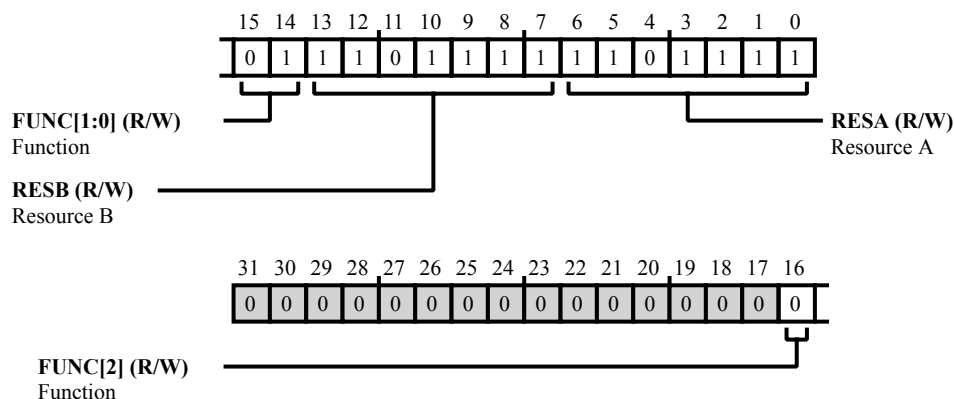


Figure 45-17: CSPFT_CNTRLDEVR[n] Register Diagram

Table 45-22: CSPFT_CNTRLDEVR[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:14 (R/W)	FUNC	Function. The <code>CSPFT_CNTRLDEVR[n]</code> .FUNC bit field specifies the logical operation that combines the two resources that define the event.
		0 A
		1 NOT(A)
		2 A AND B
		3 NOT(A) AND B
		4 NOT(A) AND NOT(B)
		5 A OR B
		6 NOT(A) OR B
		7 NOT(A) OR NOT(B)
13:7 (R/W)	RESB	Resource B. The <code>CSPFT_CNTRLDEVR[n]</code> .RESB bit field specifies one of the two resources that can be combined by the logical operation specified in the <code>CSPFT_CNTRLDEVR[n]</code> .FUNC field (See <code>CSPFT_CNTRLDEVR[n]</code> .RESA for list of possible values).

Table 45-22: CSPFT_CNTRLDEV[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	RESA	Resource A. The CSPFT_CNTRLDEV[n] . RESA bit field specifies one of the two resources that can be combined by the logical operation specified in the CSPFT_CNTRLDEV[n] . FUNC field.
		0 Single Addr Comparator 0
		1 Single Addr Comparator 1
		2 Single Addr Comparator 2
		3 Single Addr Comparator 3
		4 Single Addr Comparator 4
		5 Single Addr Comparator 5
		6 Single Addr Comparator 6
		7 Single Addr Comparator 7
		8 Single Addr Comparator 8
		9 Single Addr Comparator 9
		10 Single Addr Comparator 10
		11 Single Addr Comparator 11
		12 Single Addr Comparator 12
		13 Single Addr Comparator 13
		14 Single Addr Comparator 14
		15 Single Addr Comparator 15
		16 Addr Range Comparator 0
		17 Addr Range Comparator 1
		18 Addr Range Comparator 2
		19 Addr Range Comparator 3
		20 Addr Range Comparator 4
		21 Addr Range Comparator 5
		22 Addr Range Comparator 6
		23 Addr Range Comparator 7
		64 Counter 0 at zero
		65 Counter 1 at zero
		66 Counter 2 at zero

Table 45-22: CSPFT_CNTRLDEVR[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		67	Counter 3 at zero
		88	Context ID comparator 0
		89	Context ID comparator 1
		90	Context ID comparator 2
		95	TraceEnable start/stop resource 0 or 1
		96	External Inputs 0
		97	External Inputs 1
		98	External Inputs 2
		99	External Inputs 3
		110	Trace prohibited
		111	Always TRUE

Counter Reload Value Register

The `CSPFT_CNTRLDVR[n]` register specifies the starting value of the corresponding counter.

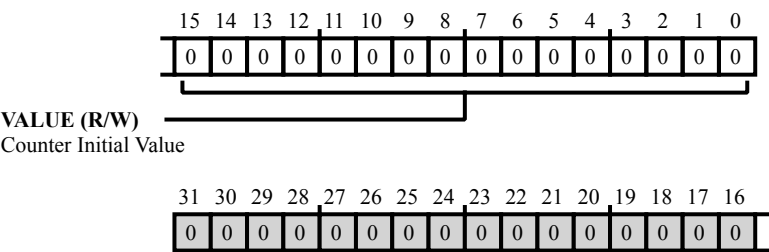


Figure 45-18: `CSPFT_CNTRLDVR[n]` Register Diagram

Table 45-23: `CSPFT_CNTRLDVR[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Counter Initial Value. The <code>CSPFT_CNTRLDVR[n].VALUE</code> bit field specifies the starting value of the corresponding counter.

Counter Value Register

The `CSPFT_CNTVR[n]` register holds the current value of the corresponding counter.

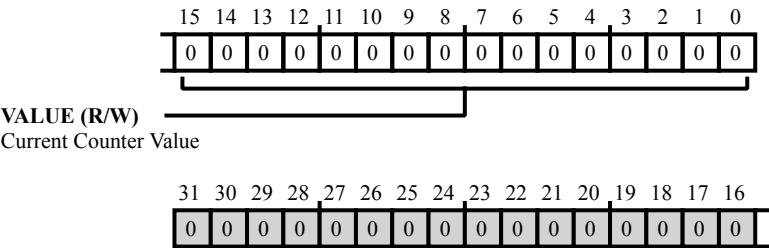


Figure 45-19: CSPFT_CNTVR[n] Register Diagram

Table 45-24: CSPFT_CNTVR[n] Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15:0 (R/W)	VALUE	Current Counter Value. The <code>CSPFT_CNTVR[n].VALUE</code> bit field specifies the current value of the corresponding counter.

Main Control Register

The `CSPFT_CTL` register controls general operation of the PTM, such as whether tracing is enabled.

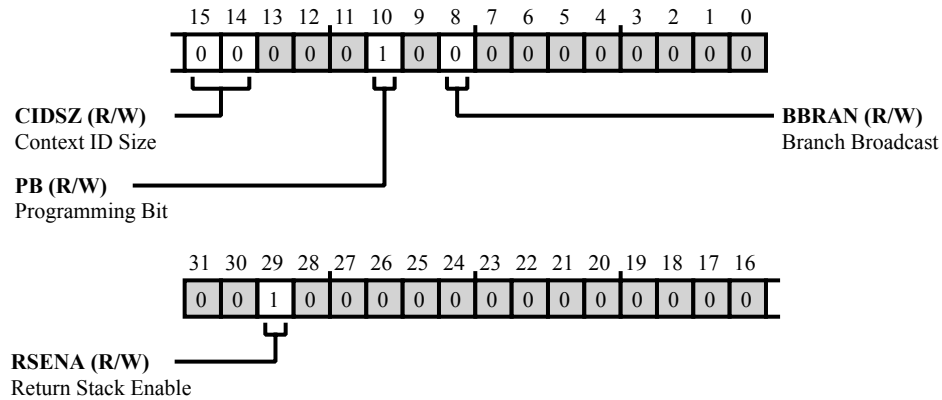


Figure 45-20: CSPFT_CTL Register Diagram

Table 45-25: CSPFT_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
29 (R/W)	RSENA	Return Stack Enable. When the <code>CSPFT_CTL.RSENA</code> bit is set, the first indirect branch back to an address generates a branch without exception packet, and subsequent branches back to the same address generate E Atoms. This compresses inner loops of HW loops and code that indirectly branches back.
15:14 (R/W)	CIDSZ	Context ID Size. The <code>CSPFT_CTL.CIDSZ</code> bit field specifies the byte size to trace. Only the bytes specified are traced, even if the new Context ID value is larger than this.
		0 No Context ID Tracing
		1 One byte Traced
		2 Two Bytes Traced
		3 Three Bytes Traced

Table 45-25: CSPFT_CTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
10 (R/W)	PB	Programming Bit. To program the CSPFT, use the following procedure. 1. Set the CSPFT_CTL . PB bit to disable all trace functionality. 2. Poll the CSPFT_STAT . PB bit waiting for it to be 1 (FIFO drained, trace halted). 3. Program the trace registers, counter and other registers, as required. 4. Set this bit to 0. 5. Poll the CSPFT_STAT . PB bit until it reads 0 (trace status reset, trace restarted). When the CSPFT_CTL . PB bit is set, the FIFO is drained and no more trace is produced. All counters are held in their present state and the external outputs are forced low. After the FIFO is drained, the CSPFT_STAT . PB is set to reflect that the part is ready to program. When this bit is cleared, the trace status is cleared and trace is restarted.
		0 Trace Enabled
		1 Trace Disabled
8 (R/W)	BBRAN	Branch Broadcast. Set the CSPFT_CTL . BBRAN bit to 1 to enable branch broadcasting. Branch broadcasting traces the address of direct branch instructions rather than producing E atoms.

Device Type Identifier Register

The `CSPFT_DEVTYPE` register is read-only. It provides a debugger with information about the component when the part number field is not recognized. The debugger can then report this information.

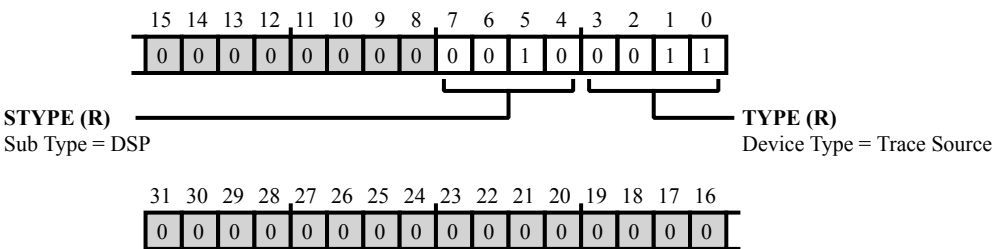


Figure 45-21: CSPFT_DEVTYPE Register Diagram

Table 45-26: CSPFT_DEVTYPE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	STYPE	Sub Type = DSP.
3:0 (R/NW)	TYPE	Device Type = Trace Source.

External Output Event Register

The `CSPFT_EXTOUTEVR[n]` register defines the event that controls the corresponding EXTOUT external output signal.

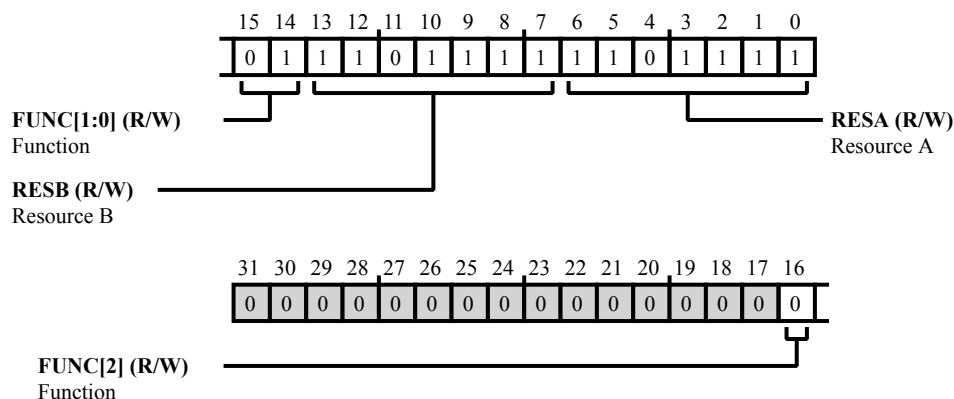


Figure 45-22: `CSPFT_EXTOUTEVR[n]` Register Diagram

Table 45-27: `CSPFT_EXTOUTEVR[n]` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:14 (R/W)	FUNC	Function. The <code>CSPFT_EXTOUTEVR[n].FUNC</code> bit field specifies the logical operation that combines the two resources that define the event.
		0 A
		1 NOT(A)
		2 A AND B
		3 NOT(A) AND B
		4 NOT(A) AND NOT(B)
		5 A OR B
		6 NOT(A) OR B
		7 NOT(A) OR NOT(B)
13:7 (R/W)	RESB	Resource B. The <code>CSPFT_EXTOUTEVR[n].RESB</code> bit field specifies one of the two resources that can be combined by the logical operation specified in the <code>CSPFT_EXTOUTEVR[n].FUNC</code> field (See <code>CSPFT_EXTOUTEVR[n].RESA</code> for list of possible values).

Table 45-27: CSPFT_EXTOUTEVR[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	RESA	Resource A. The CSPFT_EXTOUTEVR[n] . RESA bit field specifies one of the two resources that can be combined by the logical operation specified in the CSPFT_EXTOUTEVR[n] . FUNC field.
		0 Single Addr Comparator 0
		1 Single Addr Comparator 1
		2 Single Addr Comparator 2
		3 Single Addr Comparator 3
		4 Single Addr Comparator 4
		5 Single Addr Comparator 5
		6 Single Addr Comparator 6
		7 Single Addr Comparator 7
		8 Single Addr Comparator 8
		9 Single Addr Comparator 9
		10 Single Addr Comparator 10
		11 Single Addr Comparator 11
		12 Single Addr Comparator 12
		13 Single Addr Comparator 13
		14 Single Addr Comparator 14
		15 Single Addr Comparator 15
		16 Addr Range Comparator 0
		17 Addr Range Comparator 1
		18 Addr Range Comparator 2
		19 Addr Range Comparator 3
		20 Addr Range Comparator 4
		21 Addr Range Comparator 5
		22 Addr Range Comparator 6
		23 Addr Range Comparator 7
		64 Counter 0 at Zero
		65 Counter 1 at Zero
		66 Counter 2 at Zero

Table 45-27: CSPFT_EXTOUTEVR[n] Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		67	Counter 3 at Zero
		88	Context ID Comparator 0
		89	Context ID Comparator 1
		90	Context ID Comparator 2
		95	TraceEnable Start/Stop Resource 0 or 1
		96	External Inputs 0
		97	External Inputs 1
		98	External Inputs 2
		99	External Inputs 3
		110	Trace Prohibited
		111	Always TRUE

Hardware Feature Register

The `CSPFT_HWFEAT` register enables software to read the implementation defined configuration of the PTM, giving the number of each type of hardware resource. Each field indicates the number of instances of a particular resource, zero indicates that there are no implemented resources of that type.

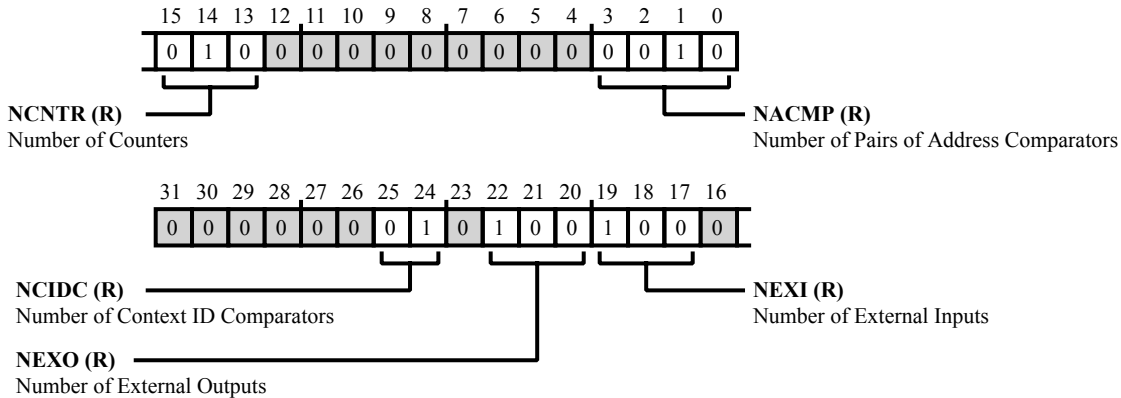


Figure 45-23: CSPFT_HWFEAT Register Diagram

Table 45-28: CSPFT_HWFEAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25:24 (R/NW)	NCIDC	Number of Context ID Comparators. The <code>CSPFT_HWFEAT.NCIDC</code> bit field identifies the number of context ID comparators.
22:20 (R/NW)	NEXO	Number of External Outputs. The <code>CSPFT_HWFEAT.NEXO</code> bit field identifies the number of external outputs (up to four).
19:17 (R/NW)	NEXI	Number of External Inputs. The <code>CSPFT_HWFEAT.NEXI</code> bit field identifies the number of external inputs (up to four).
15:13 (R/NW)	NCNTR	Number of Counters. The <code>CSPFT_HWFEAT.NCNTR</code> bit field identifies the number of counters (up to four) that are configured using the counter registers.
3:0 (R/NW)	NACMP	Number of Pairs of Address Comparators. The <code>CSPFT_HWFEAT.NACMP</code> bit field identifies the number of pairs of address comparators as address range comparators (ARCs). In this case, two adjacent address comparators form the ARC, so you can use address comparators 1 and 2 to define the first ARC. An ARC matches when any instruction in the specified range is committed for execution, regardless of whether the instruction passes its condition code test.

Table 45-28: CSPFT_HWFEAT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		0	No Address Comparators
		1	1 Pair of Address Comparators
		2	2 Pairs of Address Comparators
		3	3 Pairs of Address Comparators
		4	4 Pairs of Address Comparators
		5	5 Pairs of Address Comparators
		6	6 Pairs of Address Comparators
		7	7 Pairs of Address Comparators
		8	8 Pairs of Address Comparators

Lock Access Register

The `CSPFT_LAR` register is used to provide lock and unlock access to all other CSPFT registers.

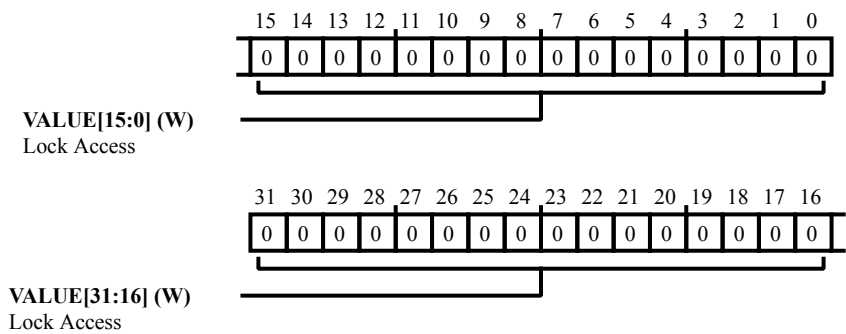


Figure 45-24: CSPFT_LAR Register Diagram

Table 45-29: CSPFT_LAR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (RX/W)	VALUE	Lock Access. Write 0xC5ACCE55 to the <code>CSPFT_LAR.VALUE</code> bit field to unlock. Write any other value to lock.

Lock Status Register

The `CSPFT_LSR` register is used to detect if the lock registers are implemented and if they are currently locked.

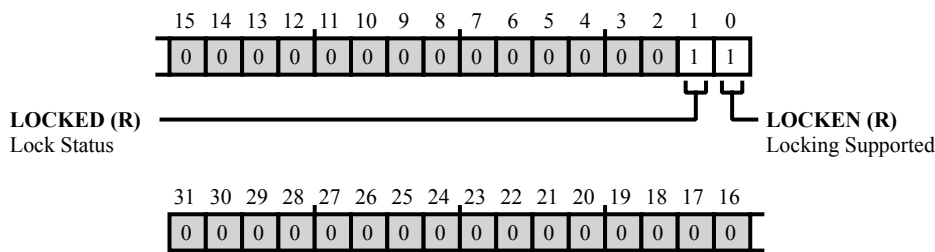


Figure 45-25: CSPFT_LSR Register Diagram

Table 45-30: CSPFT_LSR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	LOCKED	Lock Status. The <code>CSPFT_LSR.LOCKED</code> bit indicates whether the PFT is locked.
		0 Writes are permitted
		1 Locked. Writes are ignored
0 (R/NW)	LOCKEN	Locking Supported. The <code>CSPFT_LSR.LOCKEN</code> bit indicates whether the lock registers are implemented for this interface.
		0 Locking is Not Required. This access is from an interface that ignores the lock registers.
		1 Locking is Required. This access is from an interface that requires the PFT to be unlocked.

Peripheral ID0 Register

The `CSPFT_PID0` register holds peripheral identification information.

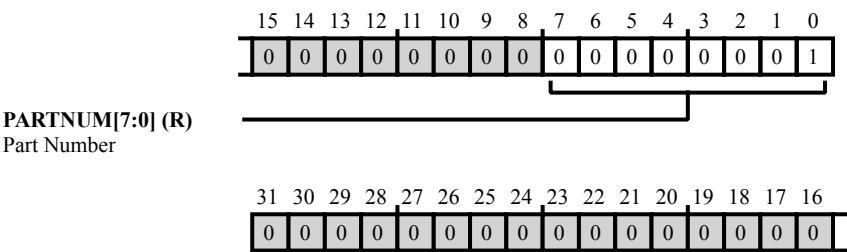


Figure 45-26: CSPFT_PID0 Register Diagram

Table 45-31: CSPFT_PID0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:0 (R/NW)	PARTNUM	Part Number. The <code>CSPFT_PID0 . PARTNUM</code> bit field holds the peripheral identification number.

Peripheral ID1 Register

The `CSPFT_PID1` register holds peripheral identification information.

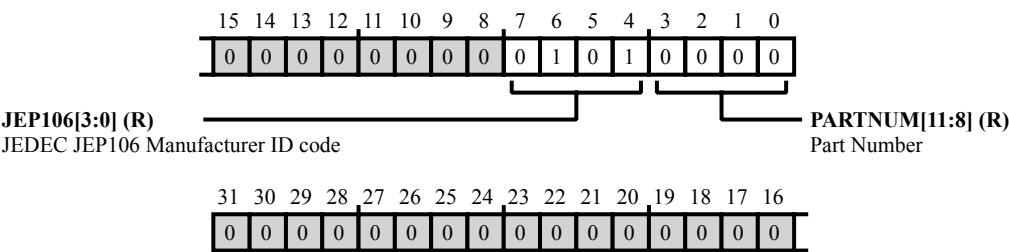


Figure 45-27: CSPFT_PID1 Register Diagram

Table 45-32: CSPFT_PID1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	JEP106	JEDEC JEP106 Manufacturer ID code.
3:0 (R/NW)	PARTNUM	Part Number. The <code>CSPFT_PID1</code> . <code>PARTNUM</code> bit field holds the peripheral identification number.

Peripheral ID2 Register

The CSPFT_PID2 register holds peripheral identification information.

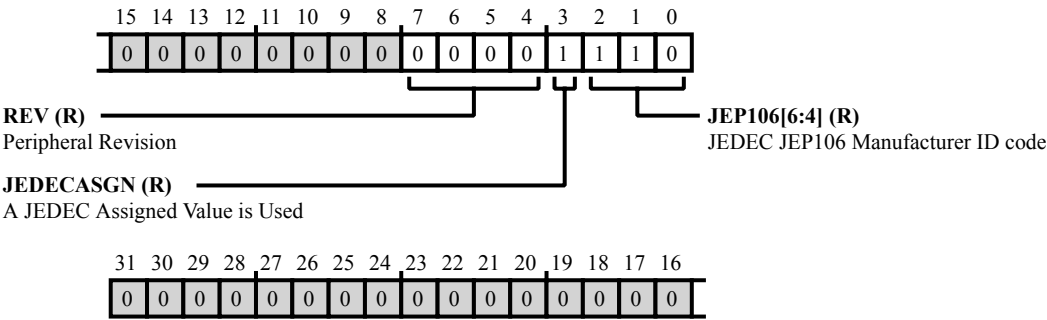


Figure 45-28: CSPFT_PID2 Register Diagram

Table 45-33: CSPFT_PID2 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	REV	Peripheral Revision.
3 (R/NW)	JEDECASGN	A JEDEC Assigned Value is Used. The CSPFT_PID2 . JEDECASGN bit indicates that a JEDEC assigned value is used.
2:0 (R/NW)	JEP106	JEDEC JEP106 Manufacturer ID code.

Peripheral ID3 Register

The `CSPFT_PID3` register holds peripheral identification information.

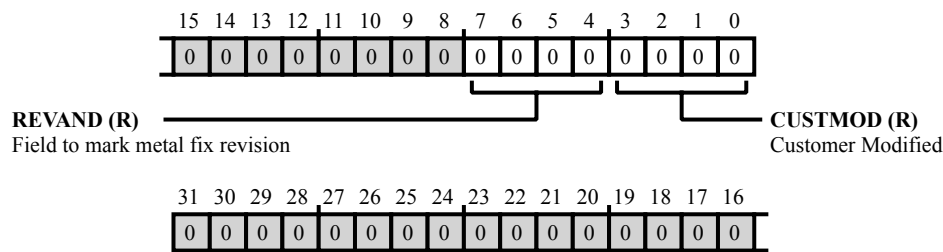


Figure 45-29: CSPFT_PID3 Register Diagram

Table 45-34: CSPFT_PID3 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	REVAND	Field to mark metal fix revision.
3:0 (R/NW)	CUSTMOD	Customer Modified.

Peripheral ID4 Register

The `CSPFT_PID4` register holds peripheral identification information.

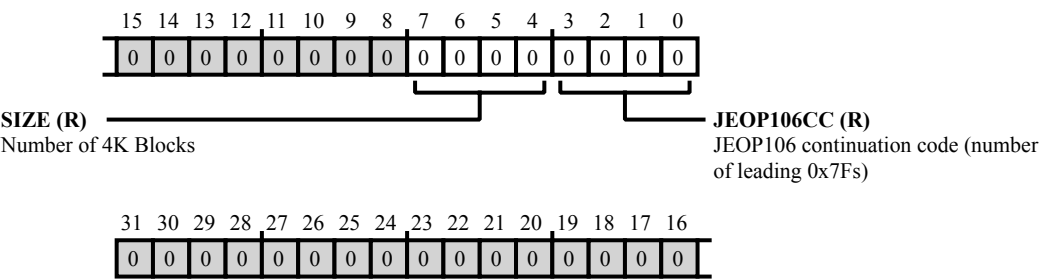


Figure 45-30: CSPFT_PID4 Register Diagram

Table 45-35: CSPFT_PID4 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
7:4 (R/NW)	SIZE	Number of 4K Blocks. The <code>CSPFT_PID4.SIZE</code> bit field contains the size of the component in 4K chunks minus 1 (for example 0=4K).
3:0 (R/NW)	JEOP106CC	JEOP106 continuation code (number of leading 0x7Fs).

Status Register

The `CSPFT_STAT` register provides information about the current status of the trace and trigger logic.

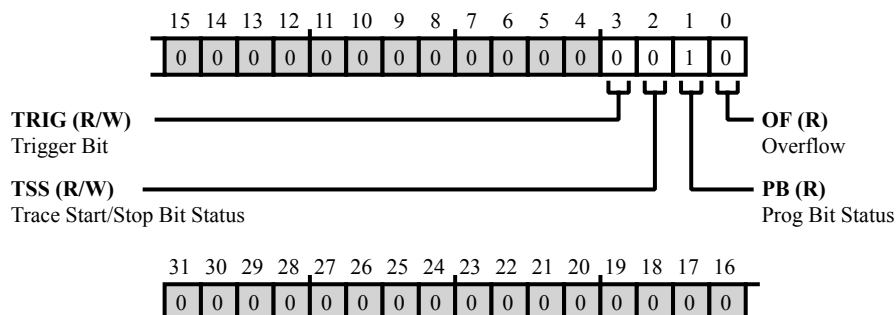


Figure 45-31: CSPFT_STAT Register Diagram

Table 45-36: CSPFT_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
3 (R/W)	TRIG	Trigger Bit. The <code>CSPFT_STAT.TRIG</code> bit is set when the trigger occurs, and prevents the trigger from being output until the CSPFT is next programmed. This bit is reset when the <code>CSPFT_CTL.PB</code> bit transitions from 1 to 0.
2 (R/W)	TSS	Trace Start/Stop Bit Status. The <code>CSPFT_STAT.TSS</code> bit holds the current status of the trace start/stop resource. If = 1, it indicates that a trace start address has been matched, without a corresponding trace stop address match. This bit =0 when trace is restarted (the <code>CSPFT_CTL.PB</code> bit transitions from 1 to 0).
1 (R/NW)	PB	Prog Bit Status. The <code>CSPFT_STAT.PB</code> bit indicates the current effective value of the <code>CSPFT_CTL.PB</code> bit. The program must wait for this bit to =1 before programming the CSPFT. (See the <code>CSPFT_CTL.PB</code> bit description).
0 (R/NW)	OF	Overflow. If the <code>CSPFT_STAT.OF</code> bit is =1, there is an overflow. This bit is cleared =0 when the trace is restarted (<code>CSPFT_CTL.PB</code> transitions from 1 to 0)

Synchronization Frequency Register

The `CSPFT_SYNCFR` register holds the trace synchronization frequency value.

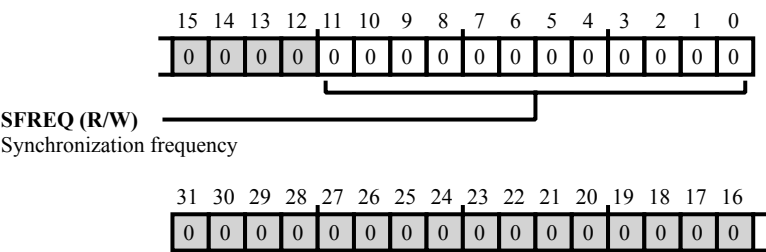


Figure 45-32: CSPFT_SYNCFR Register Diagram

Table 45-37: CSPFT_SYNCFR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
11:0 (R/W)	SFREQ	Synchronization frequency. The <code>CSPFT_SYNCFR.SFREQ</code> bit field is the number of 128-byte blocks of trace data after which you want to drop an address synchronization packet. If the circular buffer size is 16k, ensure that there are a few A-syncs in the buffer, so setting this to 16 means that every 2k there is an A-Sync packet.

TraceEnable Control Register

The `CSPFT_TECTL` register controls the start stop logic, whether resources specified are used for include or exclude, and specifies the address range comparators to use.

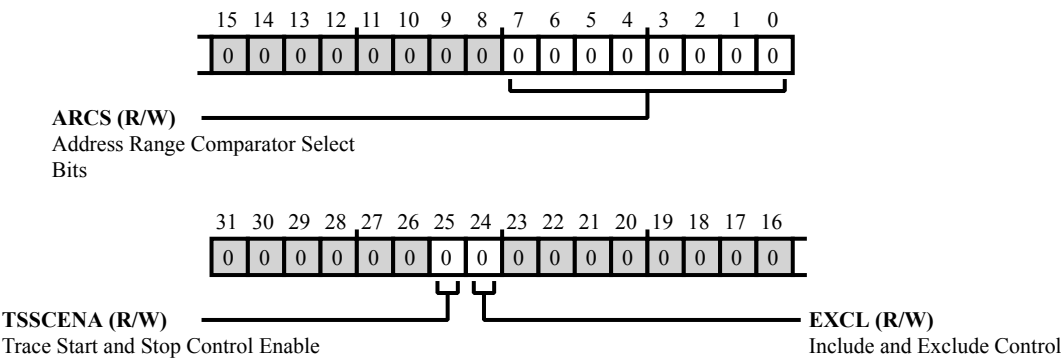


Figure 45-33: CSPFT_TECTL Register Diagram

Table 45-38: CSPFT_TECTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
25 (R/W)	TSSCENA	Trace Start and Stop Control Enable.
		0 Tracing is not affected by the trace start/stop logic
		1 Tracing is controlled by the trace on and off address comparators
24 (R/W)	EXCL	Include and Exclude Control.
		0 Include. The specified address range comparators indicate the regions where tracing can occur. When outside the region trace is prevented.
		1 Exclude. The specified address range comparators indicate regions to be excluded from the trace. When outside the range tracing is enabled.
7:0 (R/W)	ARCS	Address Range Comparator Select Bits. When a bit in the <code>CSPFT_TECTL.ARCS</code> bit field is set to 1, it selects an address range comparator, for include/exclude control.

TraceEnable Event Register

The `CSPFT_TEEVENT` register defines the TraceEnable enabling event.

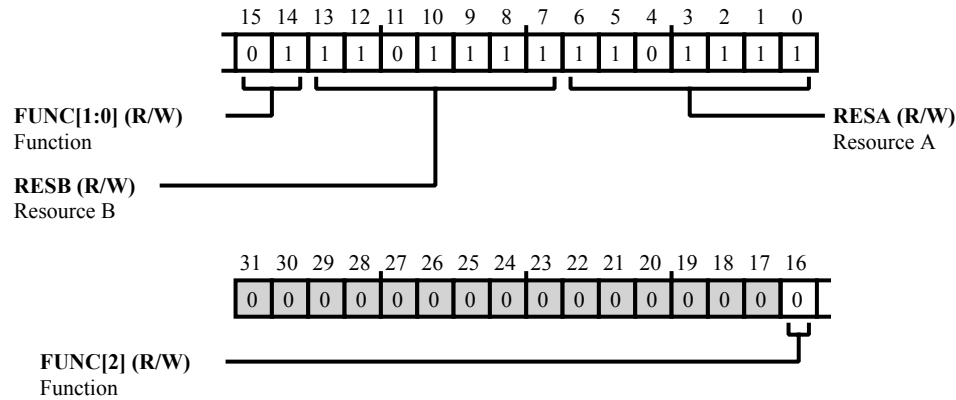


Figure 45-34: `CSPFT_TEEVENT` Register Diagram

Table 45-39: `CSPFT_TEEVENT` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:14 (R/W)	FUNC	Function. The <code>CSPFT_TEEVENT.FUNC</code> bit field specifies the logical operation that combines the two resources that define the event.
		0 A
		1 NOT(A)
		2 A AND B
		3 NOT(A) AND B
		4 NOT(A) AND NOT(B)
		5 A OR B
		6 NOT(A) OR B
		7 NOT(A) OR NOT(B)
13:7 (R/W)	RESB	Resource B. The <code>CSPFT_TEEVENT.RESB</code> bit field specifies one of the two resources that can be combined by the logical operation specified in the <code>CSPFT_TEEVENT.FUNC</code> field. (See <code>CSPFT_TEEVENT.RESA</code> for list of possible values).
6:0 (R/W)	RESA	Resource A. The <code>CSPFT_TEEVENT.RESA</code> bit field specifies one of the two resources that can be combined by the logical operation specified in the <code>CSPFT_TEEVENT.FUNC</code> field.
		0 Single Addr Comparator 0

Table 45-39: CSPFT_TEEVENT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		1 Single Addr Comparator 1
		2 Single Addr Comparator 2
		3 Single Addr Comparator 3
		4 Single Addr Comparator 4
		5 Single Addr Comparator 5
		6 Single Addr Comparator 6
		7 Single Addr Comparator 7
		8 Single Addr Comparator 8
		9 Single Addr Comparator 9
		10 Single Addr Comparator 10
		11 Single Addr Comparator 11
		12 Single Addr Comparator 12
		13 Single Addr Comparator 13
		14 Single Addr Comparator 14
		15 Single Addr Comparator 15
		16 Addr Range Comparator 0
		17 Addr Range Comparator 1
		18 Addr Range Comparator 2
		19 Addr Range Comparator 3
		20 Addr Range Comparator 4
		21 Addr Range Comparator 5
		22 Addr Range Comparator 6
		23 Addr Range Comparator 7
		64 Counter 0 at Zero
		65 Counter 1 at Zero
		66 Counter 2 at Zero
		67 Counter 3 at Zero
		88 Context ID Comparator 0
		89 Context ID Comparator 1
		90 Context ID Comparator 2

Table 45-39: CSPFT_TEEVENT Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		95	TraceEnable Start/Stop Resource 0 or 1
		96	External Inputs 0
		97	External Inputs 1
		98	External Inputs 2
		99	External Inputs 3
		110	Trace Prohibited
		111	Always TRUE

CoreSight Trace ID Register

The `CSPFT_TRACEIDR` register defines the 7-bit trace ID, for output to the trace bus.

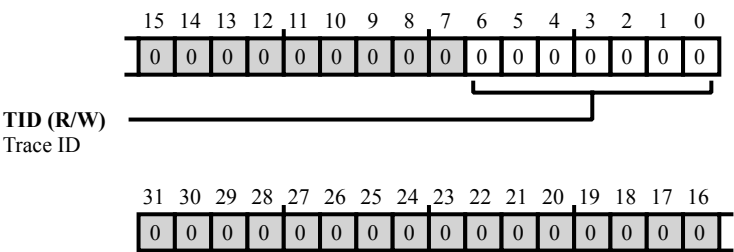


Figure 45-35: CSPFT_TRACEIDR Register Diagram

Table 45-40: CSPFT_TRACEIDR Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
6:0 (R/W)	TID	Trace ID. Defines the 7-bit trace ID, for output to the trace bus. Used in systems where multiple trace sources are present and tracing simultaneously. For example, when outputs trace onto the AMBA 3 Advanced Trace Bus, a unique ID is required for each trace source.

Trigger Event Register

The `CSPFT_TRIGGER` register defines the event that controls the trigger. This event creates the trigger output signal that is in the ATCLK domain.

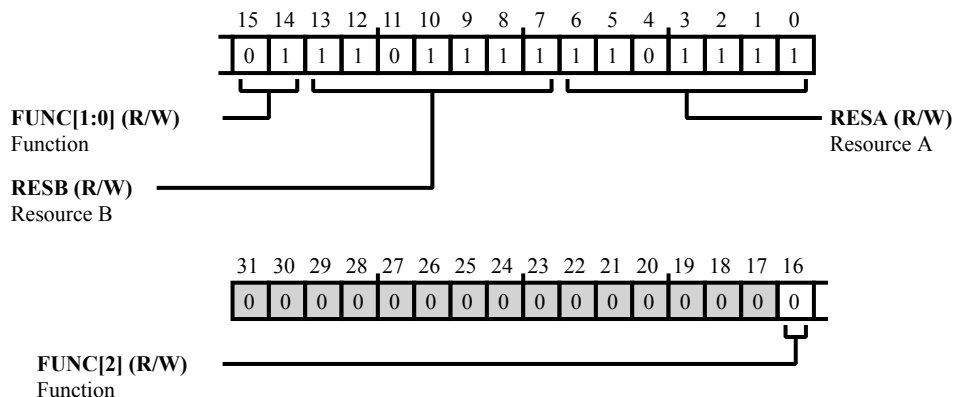


Figure 45-36: CSPFT_TRIGGER Register Diagram

Table 45-41: CSPFT_TRIGGER Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
16:14 (R/W)	FUNC	Function. Specifies the logical operation that combines the two resources that define the event
		0 A
		1 NOT(A)
		2 A AND B
		3 NOT(A) AND B
		4 NOT(A) AND NOT(B)
		5 A OR B
		6 NOT(A) OR B
		7 NOT(A) OR NOT(B)
13:7 (R/W)	RESB	Resource B. Specifies one of the two resources that can be combined by the logical operation specified in the <code>CSPFT_TRIGGER.FUNC</code> field. (See <code>CSPFT_TRIGGER.RESA</code> for list of possible values.)
6:0 (R/W)	RESA	Resource A. Specifies one of the two resources that can be combined by the logical operation specified in the <code>CSPFT_TRIGGER.FUNC</code> field
		0 Single Addr Comparator 0

Table 45-41: CSPFT_TRIGGER Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
		1 Single Addr Comparator 1
		2 Single Addr Comparator 2
		3 Single Addr Comparator 3
		4 Single Addr Comparator 4
		5 Single Addr Comparator 5
		6 Single Addr Comparator 6
		7 Single Addr Comparator 7
		8 Single Addr Comparator 8
		9 Single Addr Comparator 9
		10 Single Addr Comparator 10
		11 Single Addr Comparator 11
		12 Single Addr Comparator 12
		13 Single Addr Comparator 13
		14 Single Addr Comparator 14
		15 Single Addr Comparator 15
		16 Addr Range Comparator 0
		17 Addr Range Comparator 1
		18 Addr Range Comparator 2
		19 Addr Range Comparator 3
		20 Addr Range Comparator 4
		21 Addr Range Comparator 5
		22 Addr Range Comparator 6
		23 Addr Range Comparator 7
		64 Counter 0 at Zero
		65 Counter 1 at Zero
		66 Counter 2 at Zero
		67 Counter 3 at Zero
		88 Context ID Comparator 0
		89 Context ID Comparator 1
		90 Context ID Comparator 2

Table 45-41: CSPFT_TRIGGER Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		95	TraceEnable Start/Stop Resource 0 or 1
		96	External Inputs 0
		97	External Inputs 1
		98	External Inputs 2
		99	External Inputs 3
		110	Trace Prohibited
		111	Always TRUE

TraceEnable Start/Stop Control Register

The `CSPFT_TSSCTL` register specifies the single address comparators that hold the trace start and stop addresses.

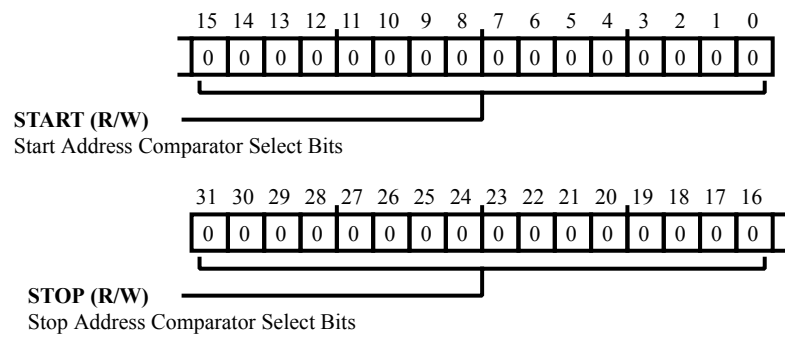


Figure 45-37: CSPFT_TSSCTL Register Diagram

Table 45-42: CSPFT_TSSCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:16 (R/W)	STOP	Stop Address Comparator Select Bits. When a bit is set to 1, it selects a single address comparator as a stop address for the TraceEnable start/stop block.
		0 disabled
		1 Address Comparator 1
		2 Address Comparator 2
		4 Address Comparator 3
		8 Address Comparator 4
		16 Address Comparator 5
		32 Address Comparator 6
		64 Address Comparator 7
		128 Address Comparator 8
		256 Address Comparator 9
		512 Address Comparator 10
		1024 Address Comparator 11
		2048 Address Comparator 12
		4096 Address Comparator 13
		8192 Address Comparator 14
		16384 Address Comparator 15

Table 45-42: CSPFT_TSSCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration	
		32768	Address Comparator 16
15:0 (R/W)	START	Start Address Comparator Select Bits. When a bit is set to 1, it selects a single address comparator as a start address for the TraceEnable start/stop block.	
		0	Disabled
		1	Address Comparator 1
		2	Address Comparator 2
		4	Address Comparator 3
		8	Address Comparator 4
		16	Address Comparator 5
		32	Address Comparator 6
		64	Address Comparator 7
		128	Address Comparator 8
		256	Address Comparator 9
		512	Address Comparator 10
		1024	Address Comparator 11
		2048	Address Comparator 12
		4096	Address Comparator 13
		8192	Address Comparator 14
		16384	Address Comparator 15
		32768	Address Comparator 16

ADSP-2159x TAPC Register Descriptions

TAPC (TAPC) contains the following registers.

Table 45-43: ADSP-2159x TAPC Register List

Name	Description
TAPC_DBGCTL	Debug Control Register
TAPC_IDCODE	IDCODE Register
TAPC_SDBGKEY0	Secure Debug Key 0 Register
TAPC_SDBGKEY1	Secure Debug Key 1 Register

Table 45-43: ADSP-2159x TAPC Register List (Continued)

Name	Description
TAPC_SDBGKEY2	Secure Debug Key 2 Register
TAPC_SDBGKEY3	Secure Debug Key 3 Register
TAPC_SDBGKEY_CTL	Secure Debug Key Control Register
TAPC_SDBGKEY_STAT	Secure Debug Key Status Register
TAPC_USERCODE	USERCODE Register

Debug Control Register

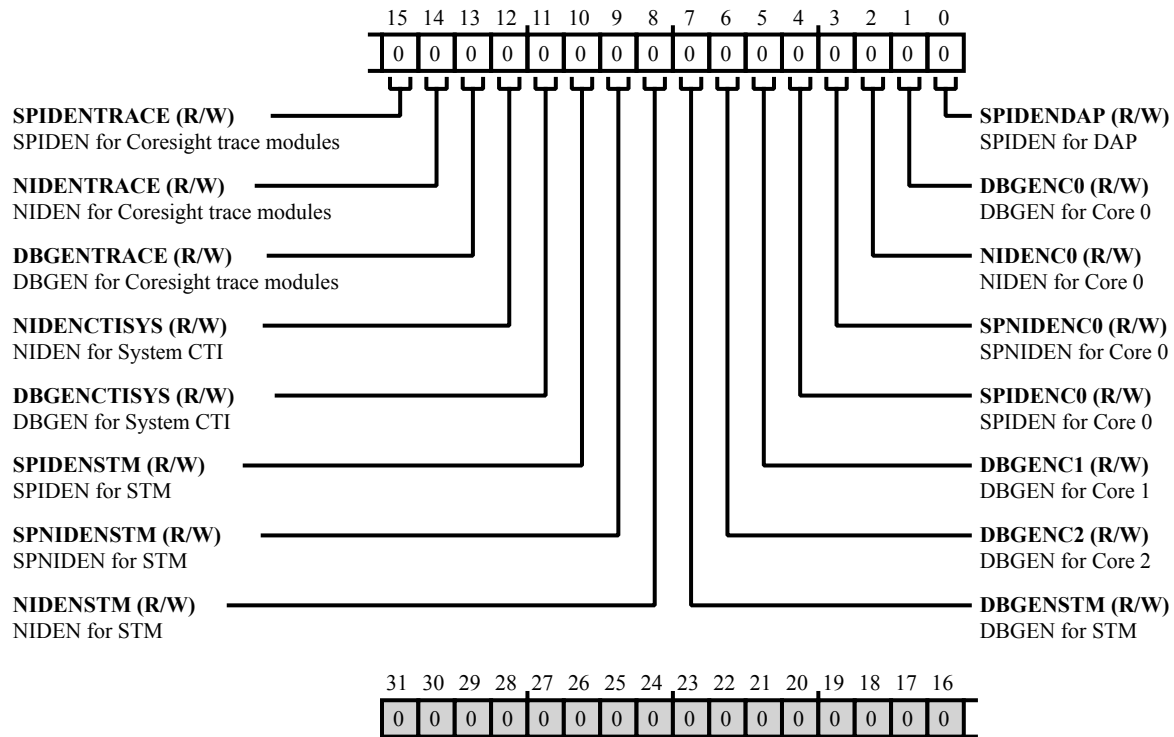


Figure 45-38: TAPC_DBGCTL Register Diagram

Table 45-44: TAPC_DBGCTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
15 (R/W)	SPIDENTRACE	SPIDEN for Coresight trace modules.
14 (R/W)	NIDENTRACE	NIDEN for Coresight trace modules.
13 (R/W)	DBGENTRACE	DBGEN for Coresight trace modules.
12 (R/W)	NIDENCTISYS	NIDEN for System CTI.
11 (R/W)	DBGENCTISYS	DBGEN for System CTI.
10 (R/W)	SPIDENSTM	SPIDEN for STM.

Table 45-44: TAPC_DBGCTL Register Fields (Continued)

Bit No. (Access)	Bit Name	Description/Enumeration
9 (R/W)	SPNIDENSTM	SPNIDEN for STM.
8 (R/W)	NIDENSTM	NIDEN for STM.
7 (R/W)	DBGENSTM	DBGEN for STM.
6 (R/W)	DBGENC2	DBGEN for Core 2.
5 (R/W)	DBGENC1	DBGEN for Core 1.
4 (R/W)	SPIDENC0	SPIDEN for Core 0.
3 (R/W)	SPNIDENC0	SPNIDEN for Core 0.
2 (R/W)	NIDENC0	NIDEN for Core 0.
1 (R/W)	DBGENC0	DBGEN for Core 0.
0 (R/W)	SPIDENDAP	SPIDEN for DAP.

IDCODE Register

The `TAPC_IDCODE` register holds the IDCODE. The bit field is defined as follows.

$\text{IDCODE}[31:28] = 0x1^* - \text{REVID}$

$\text{IDCODE}[27:12] = 0x280B - \text{JTAG ID}$

$\text{IDCODE}[11:1] = 0x65 - \text{Manufacturer ID}$

$\text{IDCODE}[0] = 0x1$

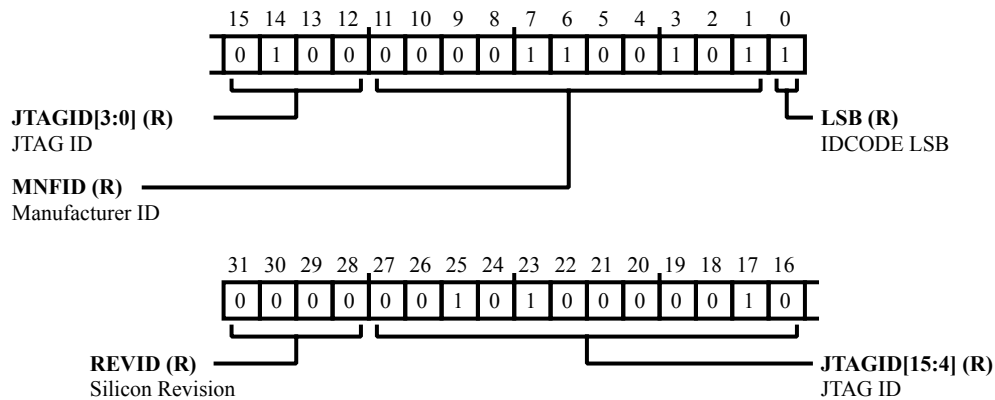


Figure 45-39: TAPC_IDCODE Register Diagram

Table 45-45: TAPC_IDCODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:28 (R/NW)	REVID	Silicon Revision. The <code>TAPC_IDCODE.REVID</code> bit field holds the silicon revision. See the processor anomaly list for details.
27:12 (R/NW)	JTAGID	JTAG ID.
11:1 (R/NW)	MNFID	Manufacturer ID.
0 (R/NW)	LSB	IDCODE LSB.

Secure Debug Key 0 Register

The `TAPC_SDBGKEY0` register allows a locked part to unlock debug access through the JTAG or SWD interfaces. A debug key of 128 bits needs to be written into the Secure Debug Key registers (`TAPC_SDBGKEY0`, `TAPC_SDBGKEY1`, `TAPC_SDBGKEY2`, `TAPC_SDBGKEY3`) in the TAPC through the peripheral bus interface.

These registers hold the value of the key against which a matching key provided by the debug user is compared to enable a debug session. The task of writing these registers is performed (initially) by boot ROM code which copies a customer-selected key from the Flash memory info block to these registers.

An SDBGKEY value of all 0's is always an invalid key, a value of all 1's match the default value of the Secure Debug Key Compare registers and requires no entry in these registers. It is recommended programs have a significant number of 0's and 1's in a pseudo-random pattern throughout the 128-bit code for maximum protection.

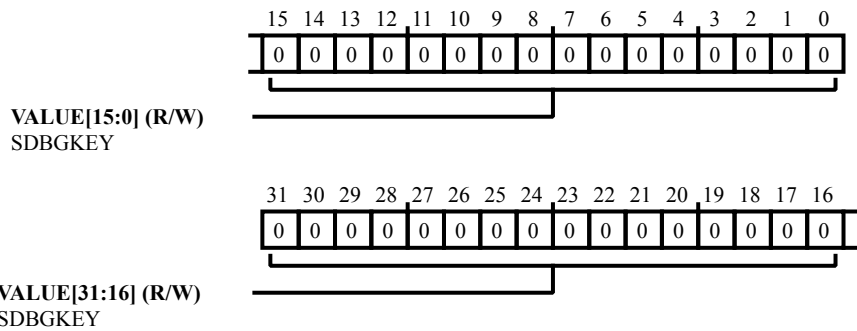


Figure 45-40: TAPC_SDBGKEY0 Register Diagram

Table 45-46: TAPC_SDBGKEY0 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SDBGKEY. The <code>TAPC_SDBGKEY0.VALUE</code> bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 1 Register

The `TAPC_SDBGKEY1` register allows a locked part to unlock debug access through the JTAG or SWD interfaces. See the `TAPC_SDBGKEY0` register description for more information.

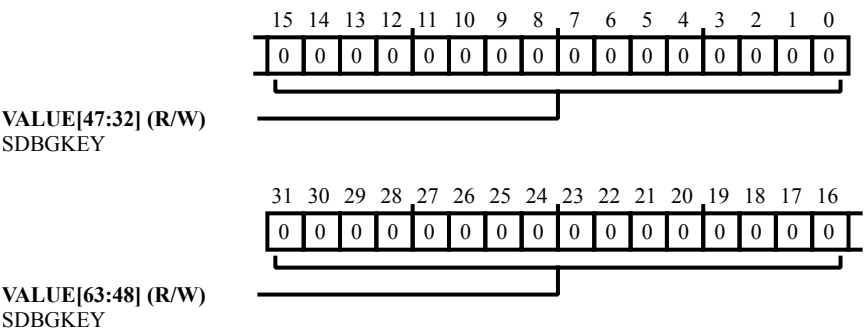


Figure 45-41: TAPC_SDBGKEY1 Register Diagram

Table 45-47: TAPC_SDBGKEY1 Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SDBGKEY. The <code>TAPC_SDBGKEY1.VALUE</code> bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 2 Register

The `TAPC_SDBGKEY2` register allows a locked part to unlock debug access through the JTAG or SWD interfaces. See the `TAPC_SDBGKEY0` register description for more information.

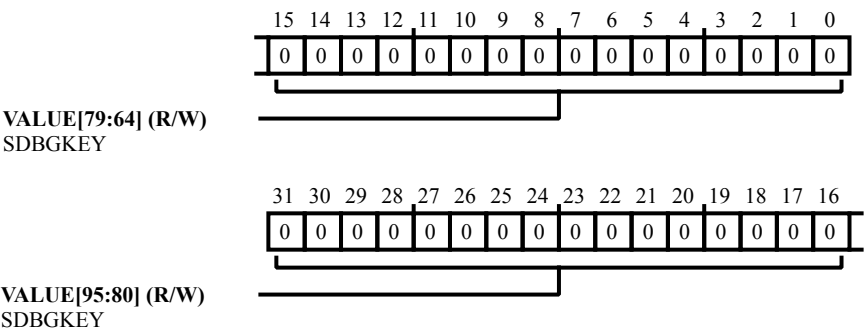


Figure 45-42: `TAPC_SDBGKEY2` Register Diagram

Table 45-48: `TAPC_SDBGKEY2` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SDBGKEY. The <code>TAPC_SDBGKEY2.VALUE</code> bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key 3 Register

The `TAPC_SDBGKEY3` register allows a locked part to unlock debug access through the JTAG or SWD interfaces. See the `TAPC_SDBGKEY0` register description for more information.

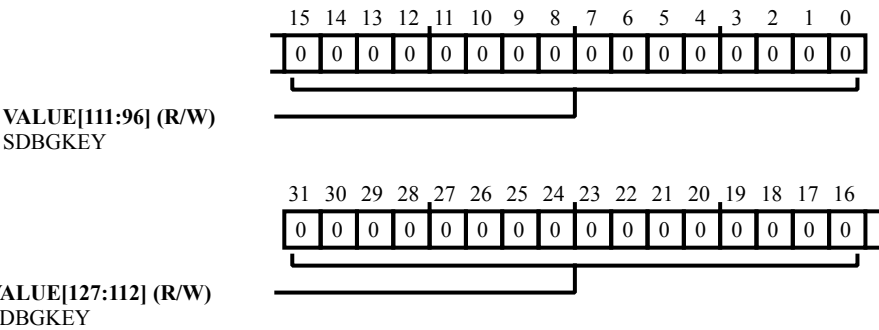


Figure 45-43: `TAPC_SDBGKEY3` Register Diagram

Table 45-49: `TAPC_SDBGKEY3` Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/W)	VALUE	SDBGKEY. The <code>TAPC_SDBGKEY3.VALUE</code> bit field holds the value of the key against which a matching key provided by the debug user is compared to enable a debug session.

Secure Debug Key Control Register

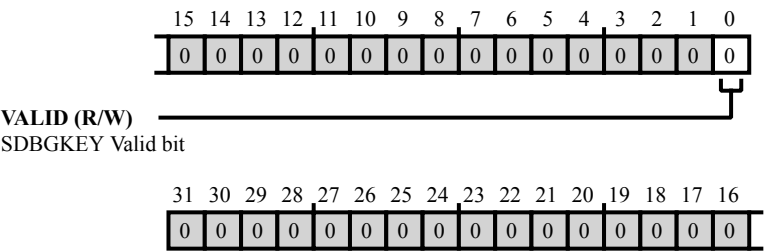


Figure 45-44: TAPC_SDBGKEY_CTL Register Diagram

Table 45-50: TAPC_SDBGKEY_CTL Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
0 (R/W)	VALID	SDBGKEY Valid bit.

Secure Debug Key Status Register

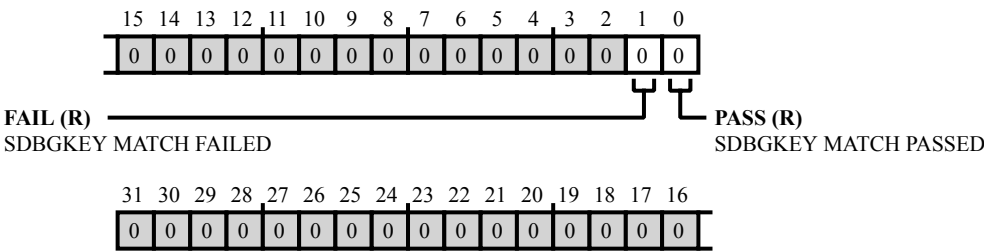


Figure 45-45: TAPC_SDBGKEY_STAT Register Diagram

Table 45-51: TAPC_SDBGKEY_STAT Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
1 (R/NW)	FAIL	SDBGKEY MATCH FAILED.
0 (R/NW)	PASS	SDBGKEY MATCH PASSED.

USERCODE Register

The `TAPC_USERCODE` register

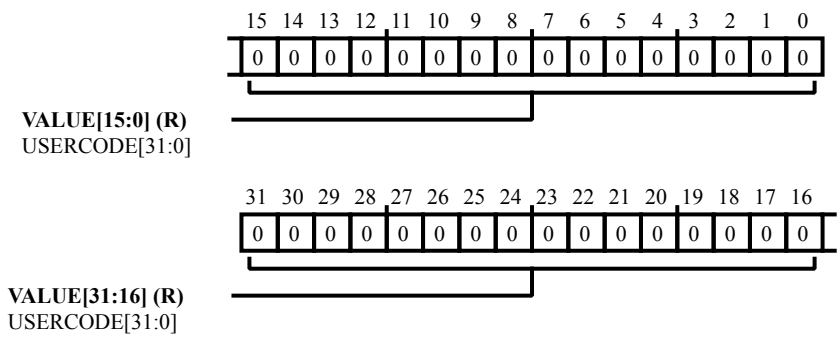


Figure 45-46: TAPC_USERCODE Register Diagram

Table 45-52: TAPC_USERCODE Register Fields

Bit No. (Access)	Bit Name	Description/Enumeration
31:0 (R/NW)	VALUE	USERCODE[31:0].

Appendix A ADSP-2159x Register List

This appendix lists Memory-Mapped Register address and register names. The modules are presented in alphabetical order.

Table A-1: ADSP-2159x DMC0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31071000	DMC0_DDR_LANE0_CTL0	DMC0 Data Lane 0 Control Register 0	0x00000000
0x31071004	DMC0_DDR_LANE0_CTL1	DMC0 Data Lane 0 Control Register 1	0x00000000
0x3107100C	DMC0_DDR_LANE1_CTL0	DMC0 Data Lane 1 Control Register 0	0x00000000
0x31071010	DMC0_DDR_LANE1_CTL1	DMC0 Data Lane 1 Control Register 1	0x00000000
0x31071018	DMC0_DDR_ROOT_CTL	DMC0 DDR ROOT Module Control Register	0x00000800
0x31071034	DMC0_DDR_ZQ_CTL0	DMC0 DDR Calibration Control Register 0	0xE0000000
0x31071038	DMC0_DDR_ZQ_CTL1	DMC0 DDR Calibration Control Register 1	0x00000000
0x3107103C	DMC0_DDR_ZQ_CTL2	DMC0 DDR Calibration Control Register 2	0x00000000
0x31071068	DMC0_DDR_CA_CTL	DMC0 DDR CA Lane Control Register	0x00000000

Table A-2: ADSP-2159x ARMDBG0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31120000	ARMDBG0_DIDR	ARMDBG0 Debug ID Register.	0x1203F001
0x31120018	ARMDBG0_WFAR	ARMDBG0 The Watchpoint Fault Address Register.	0x00000000
0x3112001C	ARMDBG0_VCR	ARMDBG0 Vector Catch Register.	0x00000000
0x31120024	ARMDBG0_ECR	ARMDBG0 Event Catch Register.	0x00000000
0x31120028	ARMDBG0_DSCCR	ARMDBG0 Debug State Cache Control Register.	0x00000000
0x3112002C	ARMDBG0_DSMCR	ARMDBG0 Debug State MMU Control Register.	0x00000000
0x31120080	ARMDBG0_DTRRX	ARMDBG0 Read Data Transfer Register.	0x00000000
0x31120084	ARMDBG0_ITR	ARMDBG0 Instruction Transfer Register.	0xFFFFFFFF
0x31120088	ARMDBG0_DSCR	ARMDBG0 Debug Status and Control Register.	0x02030002
0x3112008C	ARMDBG0_DTRTX	ARMDBG0 Write Data Transfer Register.	0x00000000
0x31120090	ARMDBG0_DRCR	ARMDBG0 Debug Run Control Register.	0x00000000
0x311200A0	ARMDBG0_PCSR	ARMDBG0 Program Counter Sampling Register.	0xFFFFFFFF

Table A-2: ADSP-2159x ARMDBG0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x311200A4	ARMDBG0_CIDSR	ARMDBG0 Context ID Sampling Register.	0x00000000
0x31120100	ARMDBG0_BVR0	ARMDBG0 Breakpoint Value Register 0.	0x00000000
0x31120104	ARMDBG0_BVR1	ARMDBG0 Breakpoint Value Register 1.	0x00000000
0x31120108	ARMDBG0_BVR2	ARMDBG0 Breakpoint Value Register 2.	0x00000000
0x31120140	ARMDBG0_BCR0	ARMDBG0 Breakpoint Control Register 0.	0x00000000
0x31120144	ARMDBG0_BCR1	ARMDBG0 Breakpoint Control Register 1.	0x00000000
0x31120148	ARMDBG0_BCR2	ARMDBG0 Breakpoint Control Register 2.	0x00000000
0x31120180	ARMDBG0_WVR0	ARMDBG0 Watchpoint Value Register 0.	0x00000000
0x31120184	ARMDBG0_WVR1	ARMDBG0 Watchpoint Value Register 1.	0x00000000
0x311201C0	ARMDBG0_WCR0	ARMDBG0 Watchpoint Control Register 0.	0x00000000
0x311201C4	ARMDBG0_WCR1	ARMDBG0 Watchpoint Control Register 1.	0x00000000
0x31120300	ARMDBG0_OSLAR	ARMDBG0 Operating System Lock Access Register.	0x00000000
0x31120304	ARMDBG0_OSLSR	ARMDBG0 Operating System Lock Status Register.	0x00000000
0x31120310	ARMDBG0_PRCR	ARMDBG0 Device Powerdown and Reset Control Register.	0x00000000
0x31120314	ARMDBG0_PRSR	ARMDBG0 Device Powerdown and Reset Status Register.	0x00000009
0x31120D00	ARMDBG0_MIDR	ARMDBG0 Main ID Register.	0x410FC051
0x31120D04	ARMDBG0_CTR	ARMDBG0 Cache Type Register.	0x83338003
0x31120D08	ARMDBG0_TCMTR	ARMDBG0 TCM Type Register.	0x00000000
0x31120D0C	ARMDBG0_TLBTR	ARMDBG0 TCM Type Register.	0x00000000
0x31120D10	ARMDBG0_MPUIR	ARMDBG0 MPU Type Register (alias of MIDR in VMSA).	0x410FC051
0x31120D14	ARMDBG0_MPIDR	ARMDBG0 Multiprocessor Affinity Register.	0xC0000000
0x31120D18	ARMDBG0_MIDRALIAS1	ARMDBG0 Main ID Register (alias at 0xd18).	0x410FC051
0x31120D1C	ARMDBG0_MIDRALIAS2	ARMDBG0 Main ID Register (alias at 0xd1c).	0x410FC051
0x31120D20	ARMDBG0_ID_PFR0	ARMDBG0 Processor Feature Register 0.	0x00001231
0x31120D24	ARMDBG0_ID_PFR1	ARMDBG0 Processor Feature Register 1.	0x00000011
0x31120D28	ARMDBG0_ID_DFR0	ARMDBG0 Debug Feature Register 0.	0x02010444
0x31120D2C	ARMDBG0_ID_AFR0	ARMDBG0 Auxiliary Feature Register 0.	0x00000000
0x31120D30	ARMDBG0_ID_MMFR0	ARMDBG0 Memory Model Feature Register 0.	0x00100003
0x31120D34	ARMDBG0_ID_MMFR1	ARMDBG0 Memory Model Feature Register 1.	0x40000000

Table A-2: ADSP-2159x ARMDBG0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31120D38	ARMDBG0_ID_MMFR2	ARMDBG0 Memory Model Feature Register 2.	0x01230000
0x31120D3C	ARMDBG0_ID_MMFR3	ARMDBG0 Memory Model Feature Register 3.	0x00102211
0x31120D40	ARMDBG0_ID_ISAR0	ARMDBG0 ISA Feature Register 0.	0x00101111
0x31120D44	ARMDBG0_ID_ISAR1	ARMDBG0 ISA Feature Register 1.	0x13112111
0x31120D48	ARMDBG0_ID_ISAR2	ARMDBG0 ISA Feature Register 2.	0x21232041
0x31120D4C	ARMDBG0_ID_ISAR3	ARMDBG0 ISA Feature Register 3.	0x11112131
0x31120D50	ARMDBG0_ID_ISAR4	ARMDBG0 ISA Feature Register 4.	0x00011142
0x31120D54	ARMDBG0_ID_ISAR5	ARMDBG0 ISA Feature Register 5.	0x00000000
0x31120EF8	ARMDBG0_ITMISCOUT	ARMDBG0 Miscellaneous Outputs Integration Register.	0x00000000
0x31120EFC	ARMDBG0_ITMISCIN	ARMDBG0 Miscellaneous Inputs Integration Register.	0x00000006
0x31120F00	ARMDBG0_ITCTRL	ARMDBG0 Integration Mode Control Register.	0x00000000
0x31120FA0	ARMDBG0_CLAIMSET	ARMDBG0 Claim Tag Set Register.	0x000000FF
0x31120FA4	ARMDBG0_CLAIMCLR	ARMDBG0 Claim Tag Clear Register.	0x00000000
0x31120FB0	ARMDBG0_LOCKACCESS	ARMDBG0 Lock Access Register.	0x00000000
0x31120FB4	ARMDBG0_LOCKSTATUS	ARMDBG0 Lock Status Register.	0x00000003
0x31120FB8	ARMDBG0_AUTHSTATUS	ARMDBG0 Authentication Status Register.	0x000000AA
0x31120FC8	ARMDBG0_DEVID	ARMDBG0 Device Identifier.	0x00000F13
0x31120FCC	ARMDBG0_DEVTYPE	ARMDBG0 Device Type Register.	0x00000015
0x31120FD0	ARMDBG0_PIR4	ARMDBG0 Peripheral ID Register 4.	0x00000004
0x31120FE0	ARMDBG0_PIR0	ARMDBG0 Peripheral ID Register 0.	0x00000005
0x31120FE4	ARMDBG0_PIR1	ARMDBG0 Peripheral ID Register 1.	0x000000BC
0x31120FE8	ARMDBG0_PIR2	ARMDBG0 Peripheral ID Register 2.	0x0000001B
0x31120FEC	ARMDBG0_PIR3	ARMDBG0 Peripheral ID Register 3.	0x00000000
0x31120FF0	ARMDBG0_CIR0	ARMDBG0 Component ID Register 0.	0x0000000D
0x31120FF4	ARMDBG0_CIR1	ARMDBG0 Component ID Register 1.	0x00000090
0x31120FF8	ARMDBG0_CIR2	ARMDBG0 Component ID Register 2.	0x00000005
0x31120FFC	ARMDBG0_CIR3	ARMDBG0 Component ID Register 3.	0x000000B1

Table A-3: ADSP-2159x ARMETM0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3112C000	ARMETM0_CR	ARMETM0 Main Control Register	0x00000441
0x3112C004	ARMETM0_CCR	ARMETM0 Configuration Code Register	0x8D294024
0x3112C008	ARMETM0_TRIGGER	ARMETM0 Trigger Event Register	0x00000000
0x3112C00C	ARMETM0_ASICCTLR	ARMETM0 ASIC Control Register	0x00000000
0x3112C010	ARMETM0_SR	ARMETM0 Status Register	0x00000002
0x3112C014	ARMETM0_SCR	ARMETM0 System Configuration Register	0x00020C0C
0x3112C018	ARMETM0_TSSCR	ARMETM0 TraceEnable Start/Stop Control Register	0x00000000
0x3112C01C	ARMETM0_TECR2	ARMETM0 TraceEnable Control Register 2	0x00000000
0x3112C020	ARMETM0_TEEVR	ARMETM0 TraceEnable Event Register	0x00000000
0x3112C024	ARMETM0_TECR1	ARMETM0 TraceEnable Control Register 1	0x00000000
0x3112C02C	ARMETM0_FFLR	ARMETM0 FIFOFULL Level Register	0x00000000
0x3112C030	ARMETM0_VDEVR	ARMETM0 ViewData Event Register	0x00000000
0x3112C034	ARMETM0_VDCR1	ARMETM0 ViewData Control Register 1	0x00000000
0x3112C03C	ARMETM0_VDCR3	ARMETM0 ViewData Control Register 3	0x00000000
0x3112C040	ARMETM0_ACVR1	ARMETM0 Address Comparator Value Register 1	0x00000000
0x3112C044	ARMETM0_ACVR2	ARMETM0 Address Comparator Value Register 2	0x00000000
0x3112C048	ARMETM0_ACVR3	ARMETM0 Address Comparator Value Register 3	0x00000000
0x3112C04C	ARMETM0_ACVR4	ARMETM0 Address Comparator Value Register 4	0x00000000
0x3112C050	ARMETM0_ACVR5	ARMETM0 Address Comparator Value Register 5	0x00000000
0x3112C054	ARMETM0_ACVR6	ARMETM0 Address Comparator Value Register 6	0x00000000
0x3112C058	ARMETM0_ACVR7	ARMETM0 Address Comparator Value Register 7	0x00000000
0x3112C05C	ARMETM0_ACVR8	ARMETM0 Address Comparator Value Register 8	0x00000000
0x3112C080	ARMETM0_ACTR1	ARMETM0 Address Comparator Access Type Register 1	0x00000000
0x3112C084	ARMETM0_ACTR2	ARMETM0 Address Comparator Access Type Register 2	0x00000000
0x3112C088	ARMETM0_ACTR3	ARMETM0 Address Comparator Access Type Register 3	0x00000000
0x3112C08C	ARMETM0_ACTR4	ARMETM0 Address Comparator Access Type Registers 4	0x00000000
0x3112C090	ARMETM0_ACTR5	ARMETM0 Address Comparator Access Type Register 5	0x00000000
0x3112C094	ARMETM0_ACTR6	ARMETM0 Address Comparator Access Type Register 6	0x00000000
0x3112C098	ARMETM0_ACTR7	ARMETM0 Address Comparator Access Type Register 7	0x00000000
0x3112C09C	ARMETM0_ACTR8	ARMETM0 Address Comparator Access Type Register 8	0x00000000
0x3112C0C0	ARMETM0_DCVR1	ARMETM0 Data Comparator Value Register 1	0x00000000

Table A-3: ADSP-2159x ARMETM0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3112C0C8	ARMETM0_DCVR3	ARMETM0 Data Comparator Value Register 3	0x00000000
0x3112C100	ARMETM0_DCMR1	ARMETM0 Data Comparator Mask Register 1	0x00000000
0x3112C108	ARMETM0_DCMR3	ARMETM0 Data Comparator Mask Register 3	0x00000000
0x3112C140	ARMETM0_CNTRL DVR1	ARMETM0 Counter Reload Value Register 1	0x00000000
0x3112C144	ARMETM0_CNTRL DVR2	ARMETM0 Counter Reload Value Register 2	0x00000000
0x3112C150	ARMETM0_CNTENR1	ARMETM0 Counter Enable Register 1	0x00000000
0x3112C154	ARMETM0_CNTENR2	ARMETM0 Counter Enable Register 2	0x00000000
0x3112C160	ARMETM0_CNTRL DEVR1	ARMETM0 Counter Reload Event Register 1	0x00000000
0x3112C164	ARMETM0_CNTRL DEVR2	ARMETM0 Counter Reload Event Register 2	0x00000000
0x3112C170	ARMETM0_CNTVR1	ARMETM0 Counter Value Register 1	0x00000000
0x3112C174	ARMETM0_CNTVR2	ARMETM0 Counter Value Register 2	0x00000000
0x3112C180	ARMETM0_SEQ12EVR	ARMETM0 Sequencer State Transition 12 Event Register	0x00000000
0x3112C184	ARMETM0_SEQ21EVR	ARMETM0 Sequencer State Transition 21 Event Register	0x00000000
0x3112C188	ARMETM0_SEQ23EVR	ARMETM0 Sequencer State Transition 23 Event Register	0x00000000
0x3112C18C	ARMETM0_SEQ31EVR	ARMETM0 Sequencer State Transition 31 Event Register	0x00000000
0x3112C190	ARMETM0_SEQ32EVR	ARMETM0 Sequencer State Transition 32 Event Register	0x00000000
0x3112C194	ARMETM0_SEQ13EVR	ARMETM0 Sequencer State Transition 13 Event Register	0x00000000
0x3112C19C	ARMETM0_SQR	ARMETM0 Current Sequencer State Register	0x00000000
0x3112C1A0	ARMETM0_EXTOUTEVR1	ARMETM0 External Output Event Register 1	0x00000000
0x3112C1A4	ARMETM0_EXTOUTEVR2	ARMETM0 External Output Event Register 2	0x00000000
0x3112C1B0	ARMETM0_CIDCVR	ARMETM0 Context ID Comparator Value Register	0x00000000
0x3112C1BC	ARMETM0_CIDCMR	ARMETM0 Context ID Comparator Mask Register	0x00000000
0x3112C1E0	ARMETM0_SYNCFR	ARMETM0 Synchronization Frequency Register	0x00000400
0x3112C1E4	ARMETM0_IDR	ARMETM0 ETM ID Register	0x410CF252
0x3112C1E8	ARMETM0_CCER	ARMETM0 Configuration Code Extension Register	0x304008F2
0x3112C1EC	ARMETM0_EXTINSELR	ARMETM0 Extended External Input Selection Register	0x00000000
0x3112C1F8	ARMETM0_TSEVR	ARMETM0 Timestamp Event Register	0x00000000
0x3112C1FC	ARMETM0_AUXCR	ARMETM0 Auxiliary Control Register	0x00000000
0x3112C200	ARMETM0_TRACEIDR	ARMETM0 CoreSight Trace ID Register	0x00000000
0x3112C208	ARMETM0_ETMAUXIDR	ARMETM0 Auxiliary ID Register	0x00000003
0x3112C314	ARMETM0_PDSR	ARMETM0 Power-Down Status Register	0x00000001

Table A-3: ADSP-2159x ARMETM0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3112CEDC	ARMETM0_ITMISCOUT	ARMETM0 Miscellaneous Outputs Register	0x00000000
0x3112CEE0	ARMETM0_ITMISCIN	ARMETM0 Miscellaneous Inputs Register	0x00000000
0x3112CEE8	ARMETM0_ITTRIGGER- REQ	ARMETM0 Trigger Request Register	0x00000000
0x3112CEEC	ARMETM0_ITATBDATA0	ARMETM0 ATB Data Register 0	0x00000000
0x3112CEF0	ARMETM0_ITATBCTR2	ARMETM0 ATB Control Register 2	0x00000000
0x3112CEF4	ARMETM0_ITATBCTR1	ARMETM0 ATB Control Register 1	0x00000000
0x3112CEF8	ARMETM0_ITATBCTR0	ARMETM0 ATB Control Register 0	0x00000000
0x3112CF00	ARMETM0_ITCTRL	ARMETM0 Integration Mode Control Register	0x00000000
0x3112CFA0	ARMETM0_CLAIMSET	ARMETM0 Claim Tag Set Register	0x000000FF
0x3112CFA4	ARMETM0_CLAIMCLR	ARMETM0 Claim Tag Clear Register	0x00000000
0x3112CFB0	ARMETM0_LAR	ARMETM0 Lock Access Register	0x00000000
0x3112CFB4	ARMETM0_LSR	ARMETM0 Lock Status Register	0x00000003
0x3112CFB8	ARMETM0_AUTHSTATUS	ARMETM0 Authentication Status Register	0x00000008
0x3112CFC8	ARMETM0_DEVID	ARMETM0 Device Identifier	0x00000000
0x3112CFCC	ARMETM0_DEVTYPE	ARMETM0 Device Type Register	0x00000013
0x3112CFD0	ARMETM0_PIR4	ARMETM0 Peripheral ID Register 4	0x00000004
0x3112CFD4	ARMETM0_PIR5	ARMETM0 Peripheral ID Register 5	0x00000000
0x3112CFD8	ARMETM0_PIR6	ARMETM0 Peripheral ID Register 6	0x00000000
0x3112CFDC	ARMETM0_PIR7	ARMETM0 Peripheral ID Register 7	0x00000000
0x3112CFE0	ARMETM0_PIR0	ARMETM0 Peripheral ID Register 0	0x00000055
0x3112CFE4	ARMETM0_PIR1	ARMETM0 Peripheral ID Register 1	0x000000B9
0x3112CFE8	ARMETM0_PIR2	ARMETM0 Peripheral ID Register 2	0x0000002B
0x3112CFEC	ARMETM0_PIR3	ARMETM0 Peripheral ID Register 3	0x00000000
0x3112CFF0	ARMETM0_CIR0	ARMETM0 Component ID Register 0	0x0000000D
0x3112CFF4	ARMETM0_CIR1	ARMETM0 Component ID Register 1	0x00000090
0x3112CFF8	ARMETM0_CIR2	ARMETM0 Component ID Register 2	0x00000005
0x3112CFFC	ARMETM0_CIR3	ARMETM0 Component ID Register 3	0x000000B1

Table A-4: ADSP-2159x ARMPMU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31121000	ARMP- MU0_PMXEVCNTR0	ARMPMU0 PM0 Counter Register.	0x00000000
0x31121004	ARMP- MU0_PMXEVCNTR1	ARMPMU0 PM1 Counter Register.	0x00000000
0x3112107C	ARMPMU0_PMCCNTR	ARMPMU0 Cycle Count Register.	0x00000000
0x31121400	ARMPMU0_PMXEVTYP- ER0	ARMPMU0 PM0 Event Type Register.	0x00000000
0x31121404	ARMPMU0_PMXEVTYP- ER1	ARMPMU0 PM0 Event Type Register.	0x00000000
0x3112147C	ARMPMU0_PMCCFILTR	ARMPMU0 Cycle Count Filter Control Register.	0x00000000
0x31121C00	ARMPMU0_PMCNTEN- SET	ARMPMU0 Count Enable Set Register.	0x00000000
0x31121C20	ARMP- MU0_PMCNTENCLR	ARMPMU0 Count Enable Clear Register.	0x00000000
0x31121C40	ARMPMU0_PMINTENSET	ARMPMU0 Interrupt Enable Set Register.	0x00000000
0x31121C60	ARMPMU0_PMINTENCLR	ARMPMU0 Interrupt Enable Clear Register.	0x00000000
0x31121C80	ARMPMU0_PMOVSR	ARMPMU0 Overflow Flag Status Register.	0x00000000
0x31121CA0	ARMPMU0_PMSWINC	ARMPMU0 Software Increment Register.	0x00000000
0x31121E00	ARMPMU0_PMCFGR	ARMPMU0 Configuration Register.	0x0009DF02
0x31121E04	ARMPMU0_PMCR	ARMPMU0 Control Register.	0x41051000
0x31121E08	ARMPMU0_PMUSERENR	ARMPMU0 User Enable Register.	0x00000000
0x31121E20	ARMPMU0_PMCEID0	ARMPMU0 Common Event Identification Register 0.	0x003FFFFFFF
0x31121E24	ARMPMU0_PMCEID1	ARMPMU0 Common Event Identification Register 1.	0x00000000
0x31121FB0	ARMPMU0_PMLAR	ARMPMU0 Lock Access Register.	0x00000000
0x31121FB4	ARMPMU0_PMLSR	ARMPMU0 Lock Status Register.	0x00000003
0x31121FB8	ARMPMU0_PMAUTHSTA- TUS	ARMPMU0 Authentication Status Register.	0x00000088
0x31121FCC	ARMPMU0_PMDEVTYPE	ARMPMU0 Device Type Register.	0x00000016
0x31121FD0	ARMPMU0_PIR4	ARMPMU0 Peripheral ID Register 4.	0x00000004
0x31121FE0	ARMPMU0_PIR0	ARMPMU0 Peripheral ID Register 0.	0x000000A5
0x31121FE4	ARMPMU0_PIR1	ARMPMU0 Peripheral ID Register 1.	0x000000B9
0x31121FE8	ARMPMU0_PIR2	ARMPMU0 Peripheral ID Register 2.	0x0000001B
0x31121FEC	ARMPMU0_PIR3	ARMPMU0 Peripheral ID Register 3.	0x00000000

Table A-4: ADSP-2159x ARMPMU0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31121FF0	ARMPMU0_CIR0	ARMPMU0 Component ID Register 0.	0x0000000D
0x31121FF4	ARMPMU0_CIR1	ARMPMU0 Component ID Register 1.	0x00000090
0x31121FF8	ARMPMU0_CIR2	ARMPMU0 Component ID Register 2.	0x00000005
0x31121FFC	ARMPMU0_CIR3	ARMPMU0 Component ID Register 3.	0x000000B1

Table A-5: ADSP-2159x ARMROM0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31110000	ARMROM0_ROMEN-TRY00	ARMROM0 ROM Entry 00	0x00010003
0x31110004	ARMROM0_ROMEN-TRY01	ARMROM0 ROM Entry 01	0x00011003
0x31110008	ARMROM0_ROMEN-TRY02	ARMROM0 ROM Entry 02	0x00012002
0x3111000C	ARMROM0_ROMEN-TRY03	ARMROM0 ROM Entry 03	0x00013002
0x31110010	ARMROM0_ROMEN-TRY04	ARMROM0 ROM Entry 04	0x00014002
0x31110014	ARMROM0_ROMEN-TRY05	ARMROM0 ROM Entry 05	0x00015002
0x31110018	ARMROM0_ROMEN-TRY06	ARMROM0 ROM Entry 06	0x00016002
0x3111001C	ARMROM0_ROMEN-TRY07	ARMROM0 ROM Entry 07	0x00017002
0x31110020	ARMROM0_ROMEN-TRY08	ARMROM0 ROM Entry 08	0x00018003
0x31110024	ARMROM0_ROMEN-TRY09	ARMROM0 ROM Entry 09	0x00019002
0x31110028	ARMROM0_ROMEN-TRY10	ARMROM0 ROM Entry 10	0x0001A002
0x3111002C	ARMROM0_ROMEN-TRY11	ARMROM0 ROM Entry 11	0x0001B002
0x31110030	ARMROM0_ROMEN-TRY12	ARMROM0 ROM Entry 12	0x0001C003
0x31110034	ARMROM0_ROMEN-TRY13	ARMROM0 ROM Entry 13	0x0001D002

Table A-5: ADSP-2159x ARMROM0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31110038	ARMROM0_ROMEN- TRY14	ARMROM0 ROM Entry 14	0x0001E002
0x3111003C	ARMROM0_ROMEN- TRY15	ARMROM0 ROM Entry 15	0x0001F002

Table A-6: ADSP-2159x ASRC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C9240	ASRC0_CTL01	ASRC0 Control Register for ASRC 0 and 1	0x00000000
0x310C9244	ASRC0_CTL23	ASRC0 Control Register for ASRC 2 and 3	0x00000000
0x310C9248	ASRC0_MUTE	ASRC0 Mute Register	0x00000000
0x310C9260	ASRC0_RAT01	ASRC0 Ratio Register for ASRC 0 and 1	0x80008000
0x310C9264	ASRC0_RAT23	ASRC0 Ratio Register for ASRC 2 and 3	0x80008000

Table A-7: ADSP-2159x ASRC1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA240	ASRC1_CTL01	ASRC1 Control Register for ASRC 0 and 1	0x00000000
0x310CA244	ASRC1_CTL23	ASRC1 Control Register for ASRC 2 and 3	0x00000000
0x310CA248	ASRC1_MUTE	ASRC1 Mute Register	0x00000000
0x310CA260	ASRC1_RAT01	ASRC1 Ratio Register for ASRC 0 and 1	0x80008000
0x310CA264	ASRC1_RAT23	ASRC1 Ratio Register for ASRC 2 and 3	0x80008000

Table A-8: ADSP-2159x CANFD0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31046000	CANFD0_CFG	CANFD0 Configuration Register	0x5980000F
0x31046004	CANFD0_CTL1	CANFD0 Control1 Register	0x00000000
0x31046008	CANFD0_TMR	CANFD0 Timer Register	0x00000000
0x31046010	CANFD0_RX_MB_GMSK	CANFD0 Receive Mailbox Global Mask Register	0x00000000
0x31046014	CANFD0_RX_14_MSK	CANFD0 Receive Mailbox14 Mask Register	0x00000000
0x31046018	CANFD0_RX_15_MSK	CANFD0 Receive Mailbox15 Mask Register	0x00000000
0x3104601C	CANFD0_ECR	CANFD0 Error Count Register	0x00000000

Table A-8: ADSP-2159x CANFD0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31046020	CANFD0_ESR1	CANFD0 Error and Status1 Register	0x00000000
0x31046024	CANFD0_IMSK2	CANFD0 Mailbox Interrupt Mask2 Register	0x00000000
0x31046028	CANFD0_IMSK1	CANFD0 Mailbox Interrupt Mask1 Register	0x00000000
0x3104602C	CANFD0_IFLG2	CANFD0 Mailbox Interrupt Flag2 Register	0x00000000
0x31046030	CANFD0_IFLG1	CANFD0 Mailbox Interrupt Flag1 Register	0x00000000
0x31046034	CANFD0_CTL2	CANFD0 Control2 Register	0x00800000
0x31046038	CANFD0_ESR2	CANFD0 Error and Status2 Register	0x00000000
0x31046044	CANFD0_CRC	CANFD0 CRC Register	0x00000000
0x31046048	CANFD0_RX_FIFO_GMSK	CANFD0 Receive FIFO Global Mask Register	0x00000000
0x3104604C	CANFD0_RX_FIFO	CANFD0 Receive FIFO Information Register	0x00000000
0x31046050	CANFD0_TIMING	CANFD0 Can Bit Timing Register	0x00000000
0x31046880	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046884	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046888	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104688C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046890	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046894	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046898	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104689C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468A0	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468A4	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468A8	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468AC	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468B0	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468B4	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468B8	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468BC	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468C0	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468C4	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468C8	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468CC	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000

Table A-8: ADSP-2159x CANFD0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310468D0	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468D4	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468D8	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468DC	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468E0	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468E4	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468E8	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468EC	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468F0	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468F4	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468F8	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x310468FC	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046900	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046904	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046908	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104690C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046910	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046914	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046918	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104691C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046920	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046924	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046928	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104692C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046930	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046934	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046938	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104693C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046940	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046944	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046948	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000

Table A-8: ADSP-2159x CANFD0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3104694C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046950	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046954	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046958	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104695C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046960	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046964	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046968	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104696C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046970	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046974	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046978	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x3104697C	CANFD0_RX_IMSK[n]	CANFD0 Receive Individual Mask Register	0x00000000
0x31046AE0	CANFD0_MEC	CANFD0 Memory Error Control Register	0x800C0080
0x31046AE4	CANFD0_ERR_IADDR	CANFD0 Error Injection Address Register	0x00000000
0x31046AE8	CANFD0_ERR_IDP	CANFD0 Error Injection Data Pattern Register	0x00000000
0x31046AEC	CANFD0_ERR_IPP	CANFD0 Error Injection Parity Pattern Register	0x00000000
0x31046AF0	CANFD0_ERR_RADDR	CANFD0 Error Report Address Register	0x00000000
0x31046AF4	CANFD0_ERR_RDAT	CANFD0 Error Report Data Register	0x00000000
0x31046AF8	CANFD0_ERR_RSYN	CANFD0 Error Report Syndrome Register	0x00000000
0x31046AFC	CANFD0_ERR_STAT	CANFD0 Error Status Register	0x00000000
0x31046B00	CANFD0_PN_CTL1	CANFD0 Pretended Networking Control1 Register	0x00000100
0x31046B04	CANFD0_PN_CTL2	CANFD0 Pretended Networking Control2 Register	0x00000000
0x31046B08	CANFD0_WUM	CANFD0 Pretended Networking Wakeup Match Register	0x00000000
0x31046B0C	CANFD0_FLTR_ID1	CANFD0 Pretended Networking ID Filter1 Register	0x00000000
0x31046B10	CANFD0_FLTR_DLC	CANFD0 Pretended Networking DLC Filter Register	0x00000008
0x31046B14	CANFD0_FLTR_DATA1_LO	CANFD0 Pretended Networking Payload Low Filter1 Register	0x00000000
0x31046B18	CANFD0_FLTR_DATA1_HI	CANFD0 Pretended Networking Payload Low Filter2 Register	0x00000000
0x31046B1C	CANFD0_FLTR_ID2_IDMSK	CANFD0 Pretended Networking ID Filter2 / IDMask Register	0x00000000

Table A-8: ADSP-2159x CANFD0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31046B20	CANFD0_FLTR_DA- TA2_DMSK_LO	CANFD0 Pretended Networking Payload Low Filter2 / Payload Low Mask Register	0x00000000
0x31046B24	CANFD0_FLTR_DA- TA2_DMSK_HI	CANFD0 Pretended Networking Payload High Filter2 High Order Bits / Payload High Mask Register	0x00000000
0x31046B40	CANFD0_WMB[n]_STAT	CANFD0 Wakeup Message Buffer Control/Status Register	0x00000000
0x31046B44	CANFD0_WMB[n]_ID	CANFD0 Wakeup Message ID Buffer Register	0x00000000
0x31046B48	CANFD0_WMB[n]_DA- TA_LO	CANFD0 Wakeup Message Buffer Data 0-3 Register	0x00000000
0x31046B4C	CANFD0_WMB[n]_DA- TA_HI	CANFD0 Wakeup Message Buffer Data 4-7 Register	0x00000000
0x31046B50	CANFD0_WMB[n]_STAT	CANFD0 Wakeup Message Buffer Control/Status Register	0x00000000
0x31046B54	CANFD0_WMB[n]_ID	CANFD0 Wakeup Message ID Buffer Register	0x00000000
0x31046B58	CANFD0_WMB[n]_DA- TA_LO	CANFD0 Wakeup Message Buffer Data 0-3 Register	0x00000000
0x31046B5C	CANFD0_WMB[n]_DA- TA_HI	CANFD0 Wakeup Message Buffer Data 4-7 Register	0x00000000
0x31046B60	CANFD0_WMB[n]_STAT	CANFD0 Wakeup Message Buffer Control/Status Register	0x00000000
0x31046B64	CANFD0_WMB[n]_ID	CANFD0 Wakeup Message ID Buffer Register	0x00000000
0x31046B68	CANFD0_WMB[n]_DA- TA_LO	CANFD0 Wakeup Message Buffer Data 0-3 Register	0x00000000
0x31046B6C	CANFD0_WMB[n]_DA- TA_HI	CANFD0 Wakeup Message Buffer Data 4-7 Register	0x00000000
0x31046B70	CANFD0_WMB[n]_STAT	CANFD0 Wakeup Message Buffer Control/Status Register	0x00000000
0x31046B74	CANFD0_WMB[n]_ID	CANFD0 Wakeup Message ID Buffer Register	0x00000000
0x31046B78	CANFD0_WMB[n]_DA- TA_LO	CANFD0 Wakeup Message Buffer Data 0-3 Register	0x00000000
0x31046B7C	CANFD0_WMB[n]_DA- TA_HI	CANFD0 Wakeup Message Buffer Data 4-7 Register	0x00000000
0x31046C00	CANFD0_FD_CTL	CANFD0 CANFD Control Register	0x80000100
0x31046C04	CANFD0_FD_TIMING	CANFD0 CANFD Bit Timing Register	0x00000000
0x31046C08	CANFD0_FD_CRC	CANFD0 CANFD CRC Register	0x00000000

Table A-9: ADSP-2159x CANFD1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31047000	CANFD1_CFG	CANFD1 Configuration Register	0x5980000F
0x31047004	CANFD1_CTL1	CANFD1 Control1 Register	0x00000000
0x31047008	CANFD1_TMR	CANFD1 Timer Register	0x00000000
0x31047010	CANFD1_RX_MB_GMSK	CANFD1 Receive Mailbox Global Mask Register	0x00000000
0x31047014	CANFD1_RX_14_MSK	CANFD1 Receive Mailbox14 Mask Register	0x00000000
0x31047018	CANFD1_RX_15_MSK	CANFD1 Receive Mailbox15 Mask Register	0x00000000
0x3104701C	CANFD1_ECR	CANFD1 Error Count Register	0x00000000
0x31047020	CANFD1_ESR1	CANFD1 Error and Status1 Register	0x00000000
0x31047024	CANFD1_IMSK2	CANFD1 Mailbox Interrupt Mask2 Register	0x00000000
0x31047028	CANFD1_IMSK1	CANFD1 Mailbox Interrupt Mask1 Register	0x00000000
0x3104702C	CANFD1_IFLG2	CANFD1 Mailbox Interrupt Flag2 Register	0x00000000
0x31047030	CANFD1_IFLG1	CANFD1 Mailbox Interrupt Flag1 Register	0x00000000
0x31047034	CANFD1_CTL2	CANFD1 Control2 Register	0x00800000
0x31047038	CANFD1_ESR2	CANFD1 Error and Status2 Register	0x00000000
0x31047044	CANFD1_CRC	CANFD1 CRC Register	0x00000000
0x31047048	CANFD1_RX_FIFO_GMSK	CANFD1 Receive FIFO Global Mask Register	0x00000000
0x3104704C	CANFD1_RX_FIFO	CANFD1 Receive FIFO Information Register	0x00000000
0x31047050	CANFD1_TIMING	CANFD1 Can Bit Timing Register	0x00000000
0x31047880	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047884	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047888	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104788C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047890	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047894	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047898	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104789C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478A0	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478A4	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478A8	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478AC	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478B0	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000

Table A-9: ADSP-2159x CANFD1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310478B4	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478B8	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478BC	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478C0	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478C4	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478C8	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478CC	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478D0	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478D4	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478D8	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478DC	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478E0	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478E4	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478E8	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478EC	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478F0	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478F4	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478F8	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x310478FC	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047900	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047904	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047908	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104790C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047910	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047914	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047918	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104791C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047920	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047924	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047928	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104792C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000

Table A-9: ADSP-2159x CANFD1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31047930	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047934	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047938	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104793C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047940	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047944	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047948	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104794C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047950	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047954	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047958	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104795C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047960	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047964	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047968	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104796C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047970	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047974	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047978	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x3104797C	CANFD1_RX_IMSK[n]	CANFD1 Receive Individual Mask Register	0x00000000
0x31047AE0	CANFD1_MEC	CANFD1 Memory Error Control Register	0x800C0080
0x31047AE4	CANFD1_ERR_IADDR	CANFD1 Error Injection Address Register	0x00000000
0x31047AE8	CANFD1_ERR_IDP	CANFD1 Error Injection Data Pattern Register	0x00000000
0x31047AEC	CANFD1_ERR_IPP	CANFD1 Error Injection Parity Pattern Register	0x00000000
0x31047AF0	CANFD1_ERR_RADDR	CANFD1 Error Report Address Register	0x00000000
0x31047AF4	CANFD1_ERR_RDAT	CANFD1 Error Report Data Register	0x00000000
0x31047AF8	CANFD1_ERR_RSYN	CANFD1 Error Report Syndrome Register	0x00000000
0x31047AFC	CANFD1_ERR_STAT	CANFD1 Error Status Register	0x00000000
0x31047B00	CANFD1_PN_CTL1	CANFD1 Pretended Networking Control1 Register	0x00000100
0x31047B04	CANFD1_PN_CTL2	CANFD1 Pretended Networking Control2 Register	0x00000000
0x31047B08	CANFD1_WUM	CANFD1 Pretended Networking Wakeup Match Register	0x00000000

Table A-9: ADSP-2159x CANFD1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31047B0C	CANFD1_FLTR_ID1	CANFD1 Pretended Networking ID Filter1 Register	0x00000000
0x31047B10	CANFD1_FLTR_DLC	CANFD1 Pretended Networking DLC Filter Register	0x00000008
0x31047B14	CANFD1_FLTR_DATA1_LO	CANFD1 Pretended Networking Payload Low Filter1 Register	0x00000000
0x31047B18	CANFD1_FLTR_DATA1_HI	CANFD1 Pretended Networking Payload Low Filter2 Register	0x00000000
0x31047B1C	CANFD1_FLTR_ID2_IDMSK	CANFD1 Pretended Networking ID Filter2 / IDMask Register	0x00000000
0x31047B20	CANFD1_FLTR_DATA2_DMSK_LO	CANFD1 Pretended Networking Payload Low Filter2 / Payload Low Mask Register	0x00000000
0x31047B24	CANFD1_FLTR_DATA2_DMSK_HI	CANFD1 Pretended Networking Payload High Filter2 High Order Bits / Payload High Mask Register	0x00000000
0x31047B40	CANFD1_WMB[n]_STAT	CANFD1 Wakeup Message Buffer Control/Status Register	0x00000000
0x31047B44	CANFD1_WMB[n]_ID	CANFD1 Wakeup Message ID Buffer Register	0x00000000
0x31047B48	CANFD1_WMB[n]_DATA_LO	CANFD1 Wakeup Message Buffer Data 0-3 Register	0x00000000
0x31047B4C	CANFD1_WMB[n]_DATA_HI	CANFD1 Wakeup Message Buffer Data 4-7 Register	0x00000000
0x31047B50	CANFD1_WMB[n]_STAT	CANFD1 Wakeup Message Buffer Control/Status Register	0x00000000
0x31047B54	CANFD1_WMB[n]_ID	CANFD1 Wakeup Message ID Buffer Register	0x00000000
0x31047B58	CANFD1_WMB[n]_DATA_LO	CANFD1 Wakeup Message Buffer Data 0-3 Register	0x00000000
0x31047B5C	CANFD1_WMB[n]_DATA_HI	CANFD1 Wakeup Message Buffer Data 4-7 Register	0x00000000
0x31047B60	CANFD1_WMB[n]_STAT	CANFD1 Wakeup Message Buffer Control/Status Register	0x00000000
0x31047B64	CANFD1_WMB[n]_ID	CANFD1 Wakeup Message ID Buffer Register	0x00000000
0x31047B68	CANFD1_WMB[n]_DATA_LO	CANFD1 Wakeup Message Buffer Data 0-3 Register	0x00000000
0x31047B6C	CANFD1_WMB[n]_DATA_HI	CANFD1 Wakeup Message Buffer Data 4-7 Register	0x00000000
0x31047B70	CANFD1_WMB[n]_STAT	CANFD1 Wakeup Message Buffer Control/Status Register	0x00000000
0x31047B74	CANFD1_WMB[n]_ID	CANFD1 Wakeup Message ID Buffer Register	0x00000000
0x31047B78	CANFD1_WMB[n]_DATA_LO	CANFD1 Wakeup Message Buffer Data 0-3 Register	0x00000000

Table A-9: ADSP-2159x CANFD1 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31047B7C	CANFD1_WMB[n]_DA-TA_HI	CANFD1 Wakeup Message Buffer Data 4-7 Register	0x00000000
0x31047C00	CANFD1_FD_CTL	CANFD1 CANFD Control Register	0x80000100
0x31047C04	CANFD1_FD_TIMING	CANFD1 CANFD Bit Timing Register	0x00000000
0x31047C08	CANFD1_FD_CRC	CANFD1 CANFD CRC Register	0x00000000

Table A-10: ADSP-2159x CDU0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3108F000	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F004	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F008	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F00C	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F010	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F014	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F018	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F01C	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F020	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F024	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F028	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F02C	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F030	CDU0_CFG[n]	CDU0 CDU Configuration	0x00000001
0x3108F040	CDU0_STAT	CDU0 CDU Status	0x00000000
0x3108F044	CDU0_CLKINSEL	CDU0 CLKIN Select	0x00000000
0x3108F048	CDU0_REVID	CDU0 CDU Revision ID	0x00000011

Table A-11: ADSP-2159x CGU0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3108D000	CGU0_CTL	CGU0 Control Register	0x00003C00
0x3108D004	CGU0_PLLCTL	CGU0 PLL Control Register	0x00000000
0x3108D008	CGU0_STAT	CGU0 Status Register	0x0000000F

Table A-11: ADSP-2159x CGU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108D00C	CGU0_DIV	CGU0 Clocks Divisor Register	0x0A034482
0x3108D010	CGU0_CLKOUTSEL	CGU0 CLKOUT Select Register	0x00000000
0x3108D014	CGU0_OSCWDCTL	CGU0 Oscillator Watchdog Register	0x00007F00
0x3108D018	CGU0_TSCTL	CGU0 Time Stamp Control Register	0x00000000
0x3108D01C	CGU0_TSVALUE0	CGU0 Time Stamp Counter Initial 32 LSB Value Register	0x00000000
0x3108D020	CGU0_TSVALUE1	CGU0 Time Stamp Counter Initial MSB Value Register	0x00000000
0x3108D024	CGU0_TSCOUNT0	CGU0 Time Stamp Counter 32 LSB Register	0x00000000
0x3108D028	CGU0_TSCOUNT1	CGU0 Time Stamp Counter 32 MSB Register	0x00000000
0x3108D02C	CGU0_CCBF_DIS	CGU0 Core Clock Buffer Disable Register	0x00000000
0x3108D030	CGU0_CCBF_STAT	CGU0 Core Clock Buffer Status Register	0x00000000
0x3108D038	CGU0_SCBF_DIS	CGU0 System Clock Buffer Disable Register	0x00000000
0x3108D03C	CGU0_SCBF_STAT	CGU0 System Clock Buffer Status Register	0x00000000
0x3108D040	CGU0_DIVEX	CGU0 DIV Register Extension	0x00200030
0x3108D048	CGU0_REVID	CGU0 Revision ID Register	0x00000030

Table A-12: ADSP-2159x CGU1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108E000	CGU1_CTL	CGU1 Control Register	0x00003C00
0x3108E004	CGU1_PLLCTL	CGU1 PLL Control Register	0x00000000
0x3108E008	CGU1_STAT	CGU1 Status Register	0x0000000F
0x3108E00C	CGU1_DIV	CGU1 Clocks Divisor Register	0x0A034482
0x3108E014	CGU1_OSCWDCTL	CGU1 Oscillator Watchdog Register	0x00007F00
0x3108E02C	CGU1_CCBF_DIS	CGU1 Core Clock Buffer Disable Register	0x00000000
0x3108E030	CGU1_CCBF_STAT	CGU1 Core Clock Buffer Status Register	0x00000000
0x3108E038	CGU1_SCBF_DIS	CGU1 System Clock Buffer Disable Register	0x00000000
0x3108E03C	CGU1_SCBF_STAT	CGU1 System Clock Buffer Status Register	0x00000000
0x3108E040	CGU1_DIVEX	CGU1 DIV Register Extension	0x00200030
0x3108E048	CGU1_REVID	CGU1 Revision ID Register	0x00000030

Table A-13: ADSP-2159x CNT0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3100B000	CNT0_CFG	CNT0 Configuration Register	0x00000000
0x3100B004	CNT0_IMSK	CNT0 Interrupt Mask Register	0x00000000
0x3100B008	CNT0_STAT	CNT0 Status Register	0x00000000
0x3100B00C	CNT0_CMD	CNT0 Command Register	0x00000000
0x3100B010	CNT0_DEBNCE	CNT0 Debounce Register	0x00000000
0x3100B014	CNT0_CNTR	CNT0 Counter Register	0x00000000
0x3100B018	CNT0_MAX	CNT0 Maximum Count Register	0x00000000
0x3100B01C	CNT0_MIN	CNT0 Minimum Count Register	0x00000000

Table A-14: ADSP-2159x CRC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A5000	CRC0_CTL	CRC0 Control Register	0x00000000
0x310A5004	CRC0_DCNT	CRC0 Data Word Count Register	0x00000000
0x310A5008	CRC0_DCNTRL	CRC0 Data Word Count Reload Register	0x00000000
0x310A5014	CRC0_COMP	CRC0 Data Compare Register	0x00000000
0x310A5018	CRC0_FILLVAL	CRC0 Fill Value Register	0x00000000
0x310A501C	CRC0_DFIFO	CRC0 Data FIFO Register	0x00000000
0x310A5020	CRC0_INEN	CRC0 Interrupt Enable Register	0x00000000
0x310A5024	CRC0_INEN_SET	CRC0 Interrupt Enable Set Register	0x00000000
0x310A5028	CRC0_INEN_CLR	CRC0 Interrupt Enable Clear Register	0x00000000
0x310A502C	CRC0_POLY	CRC0 Polynomial Register	0x00000000
0x310A5040	CRC0_STAT	CRC0 Status Register	0x00000000
0x310A5044	CRC0_DCNTCAP	CRC0 Data Count Capture Register	0x00000000
0x310A504C	CRC0_RESULT_FIN	CRC0 CRC Final Result Register	0x00000000
0x310A5050	CRC0_RESULT_CUR	CRC0 CRC Current Result Register	0x00000000

Table A-15: ADSP-2159x CRC1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A6000	CRC1_CTL	CRC1 Control Register	0x00000000
0x310A6004	CRC1_DCNT	CRC1 Data Word Count Register	0x00000000

Table A-15: ADSP-2159x CRC1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A6008	CRC1_DCNTRLD	CRC1 Data Word Count Reload Register	0x00000000
0x310A6014	CRC1_COMP	CRC1 Data Compare Register	0x00000000
0x310A6018	CRC1_FILLVAL	CRC1 Fill Value Register	0x00000000
0x310A601C	CRC1_DFIFO	CRC1 Data FIFO Register	0x00000000
0x310A6020	CRC1_INEN	CRC1 Interrupt Enable Register	0x00000000
0x310A6024	CRC1_INEN_SET	CRC1 Interrupt Enable Set Register	0x00000000
0x310A6028	CRC1_INEN_CLR	CRC1 Interrupt Enable Clear Register	0x00000000
0x310A602C	CRC1_POLY	CRC1 Polynomial Register	0x00000000
0x310A6040	CRC1_STAT	CRC1 Status Register	0x00000000
0x310A6044	CRC1_DCNTCAP	CRC1 Data Count Capture Register	0x00000000
0x310A604C	CRC1_RESULT_FIN	CRC1 CRC Final Result Register	0x00000000
0x310A6050	CRC1_RESULT_CUR	CRC1 CRC Current Result Register	0x00000000

Table A-16: ADSP-2159x CRC2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310AA000	CRC2_CTL	CRC2 Control Register	0x00000000
0x310AA004	CRC2_DCNT	CRC2 Data Word Count Register	0x00000000
0x310AA008	CRC2_DCNTRLD	CRC2 Data Word Count Reload Register	0x00000000
0x310AA014	CRC2_COMP	CRC2 Data Compare Register	0x00000000
0x310AA018	CRC2_FILLVAL	CRC2 Fill Value Register	0x00000000
0x310AA01C	CRC2_DFIFO	CRC2 Data FIFO Register	0x00000000
0x310AA020	CRC2_INEN	CRC2 Interrupt Enable Register	0x00000000
0x310AA024	CRC2_INEN_SET	CRC2 Interrupt Enable Set Register	0x00000000
0x310AA028	CRC2_INEN_CLR	CRC2 Interrupt Enable Clear Register	0x00000000
0x310AA02C	CRC2_POLY	CRC2 Polynomial Register	0x00000000
0x310AA040	CRC2_STAT	CRC2 Status Register	0x00000000
0x310AA044	CRC2_DCNTCAP	CRC2 Data Count Capture Register	0x00000000
0x310AA04C	CRC2_RESULT_FIN	CRC2 CRC Final Result Register	0x00000000
0x310AA050	CRC2_RESULT_CUR	CRC2 CRC Current Result Register	0x00000000

Table A-17: ADSP-2159x CRC3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310AB000	CRC3_CTL	CRC3 Control Register	0x00000000
0x310AB004	CRC3_DCNT	CRC3 Data Word Count Register	0x00000000
0x310AB008	CRC3_DCNTRLD	CRC3 Data Word Count Reload Register	0x00000000
0x310AB014	CRC3_COMP	CRC3 Data Compare Register	0x00000000
0x310AB018	CRC3_FILLVAL	CRC3 Fill Value Register	0x00000000
0x310AB01C	CRC3_DFIFO	CRC3 Data FIFO Register	0x00000000
0x310AB020	CRC3_INEN	CRC3 Interrupt Enable Register	0x00000000
0x310AB024	CRC3_INEN_SET	CRC3 Interrupt Enable Set Register	0x00000000
0x310AB028	CRC3_INEN_CLR	CRC3 Interrupt Enable Clear Register	0x00000000
0x310AB02C	CRC3_POLY	CRC3 Polynomial Register	0x00000000
0x310AB040	CRC3_STAT	CRC3 Status Register	0x00000000
0x310AB044	CRC3_DCNTCAP	CRC3 Data Count Capture Register	0x00000000
0x310AB04C	CRC3_RESULT_FIN	CRC3 CRC Final Result Register	0x00000000
0x310AB050	CRC3_RESULT_CUR	CRC3 CRC Current Result Register	0x00000000

Table A-18: ADSP-2159x CTI0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31128000	CTI0_CTICONTROL	CTI0 CTI Control Register	0x00000000
0x31128010	CTI0_CTIINTACK	CTI0 CTI Interrupt Acknowledge Register	0x00000000
0x31128014	CTI0_CTIAPPSET	CTI0 CTI Application Trigger Set Register	0x00000000
0x31128018	CTI0_CTIAPPCLEAR	CTI0 CTI Application Trigger Clear Register	0x00000000
0x3112801C	CTI0_CTIAPPULSE	CTI0 CTI Application Pulse Register	0x00000000
0x31128020	CTI0_CTIINEN0	CTI0 CTI Trigger 0 to Channel Enable Register	0x00000000
0x31128024	CTI0_CTIINEN1	CTI0 CTI Trigger 1 to Channel Enable Register	0x00000000
0x31128028	CTI0_CTIINEN2	CTI0 CTI Trigger 2 to Channel Enable Register	0x00000000
0x3112802C	CTI0_CTIINEN3	CTI0 CTI Trigger 3 to Channel Enable Register	0x00000000
0x31128030	CTI0_CTIINEN4	CTI0 CTI Trigger 4 to Channel Enable Register	0x00000000
0x31128034	CTI0_CTIINEN5	CTI0 CTI Trigger 5 to Channel Enable Register	0x00000000
0x31128038	CTI0_CTIINEN6	CTI0 CTI Trigger 6 to Channel Enable Register	0x00000000
0x3112803C	CTI0_CTIINEN7	CTI0 CTI Trigger 7 to Channel Enable Register	0x00000000

Table A-18: ADSP-2159x CTI0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x311280A0	CTI0_CTIOUTEN0	CTI0 CTI Channel to Trigger 0 Enable Register	0x00000000
0x311280A4	CTI0_CTIOUTEN1	CTI0 CTI Channel to Trigger 1 Enable Register	0x00000000
0x311280A8	CTI0_CTIOUTEN2	CTI0 CTI Channel to Trigger 2 Enable Register	0x00000000
0x311280AC	CTI0_CTIOUTEN3	CTI0 CTI Channel to Trigger 3 Enable Register	0x00000000
0x311280B0	CTI0_CTIOUTEN4	CTI0 CTI Channel to Trigger 4 Enable Register	0x00000000
0x311280B4	CTI0_CTIOUTEN5	CTI0 CTI Channel to Trigger 5 Enable Register	0x00000000
0x311280B8	CTI0_CTIOUTEN6	CTI0 CTI Channel to Trigger 6 Enable Register	0x00000000
0x311280BC	CTI0_CTIOUTEN7	CTI0 CTI Channel to Trigger 7 Enable Register	0x00000000
0x31128130	CTI0_CTITRIGINSTATUS	CTI0 CTI Trigger In Status Register	0x00000000
0x31128134	CTI0_CTITRIGOUTSTA- TUS	CTI0 CTI Trigger Out Status Register	0x00000000
0x31128138	CTI0_CTICHINSTATUS	CTI0 CTI Channel In Status Register	0x00000000
0x3112813C	CTI0_CTICHOUTSTATUS	CTI0 CTI Channel Out Status Register	0x00000000
0x31128140	CTI0_CTIGATE	CTI0 Enable CTI Channel Gate Register	0x0000000F
0x31128144	CTI0_ASICCTL	CTI0 External Multiplexor Control Register	0x00000000
0x31128EDC	CTI0_ITCHINACK	CTI0 ITCHINACK	0x00000000
0x31128EE0	CTI0_ITTRIGINACK	CTI0 ITTRIGINACK	0x00000000
0x31128EE4	CTI0_ITCHOUT	CTI0 ITCHOUT	0x00000000
0x31128EE8	CTI0_ITTRIGOUT	CTI0 ITTRIGOUT	0x00000000
0x31128EEC	CTI0_ITCHOUTACK	CTI0 ITCHOUTACK	0x00000000
0x31128EF0	CTI0_ITTRIGOUTACK	CTI0 ITTRIGOUTACK	0x00000000
0x31128EF4	CTI0_ITCHIN	CTI0 ITCHIN	0x00000000
0x31128EF8	CTI0_ITTRIGIN	CTI0 ITTRIGIN	0x00000000
0x31128F00	CTI0_ITCTRL	CTI0 Integration Mode Control Register	0x00000000
0x31128FA0	CTI0_CLAIMSET	CTI0 Claim Tag Set Register	0x0000000F
0x31128FA4	CTI0_CLAIMCLR	CTI0 Claim Tag Clear Register	0x00000000
0x31128FB0	CTI0_LAR	CTI0 Lock Access Register	0x00000000
0x31128FB4	CTI0_LSR	CTI0 Lock Status Register	0x00000003
0x31128FB8	CTI0_AUTHSTATUS	CTI0 Authentication Status	0x00000005
0x31128FC8	CTI0_DEVID	CTI0 Device ID	0x00040800
0x31128FCC	CTI0_DEVTYPE	CTI0 Device Type	0x00000014

Table A-18: ADSP-2159x CTI0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31128FD0	CTI0_PERIPHID4	CTI0 Peripheral ID4	0x00000004
0x31128FD4	CTI0_PERIPHID5	CTI0 Peripheral ID5	0x00000000
0x31128FD8	CTI0_PERIPHID6	CTI0 Peripheral ID6	0x00000000
0x31128FDC	CTI0_PERIPHID7	CTI0 Peripheral ID7	0x00000000
0x31128FE0	CTI0_PERIPHID0	CTI0 Peripheral ID0	0x00000006
0x31128FE4	CTI0_PERIPHID1	CTI0 Peripheral ID1	0x000000B9
0x31128FE8	CTI0_PERIPHID2	CTI0 Peripheral ID2	0x0000003B
0x31128FEC	CTI0_PERIPHID3	CTI0 Peripheral ID3	0x00000000
0x31128FF0	CTI0_COMPID0	CTI0 Component ID0	0x0000000D
0x31128FF4	CTI0_COMPID1	CTI0 Component ID1	0x00000090
0x31128FF8	CTI0_COMPID2	CTI0 Component ID2	0x00000005
0x31128FFC	CTI0_COMPID3	CTI0 Component ID3	0x000000B1

Table A-19: ADSP-2159x CTI1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31102000	CTI1_CTICONTROL	CTI1 CTI Control Register	0x00000000
0x31102010	CTI1_CTIINTACK	CTI1 CTI Interrupt Acknowledge Register	0x00000000
0x31102014	CTI1_CTIAPPSET	CTI1 CTI Application Trigger Set Register	0x00000000
0x31102018	CTI1_CTIAPPCLEAR	CTI1 CTI Application Trigger Clear Register	0x00000000
0x3110201C	CTI1_CTIAPPPULSE	CTI1 CTI Application Pulse Register	0x00000000
0x31102020	CTI1_CTIINEN0	CTI1 CTI Trigger 0 to Channel Enable Register	0x00000000
0x31102024	CTI1_CTIINEN1	CTI1 CTI Trigger 1 to Channel Enable Register	0x00000000
0x31102028	CTI1_CTIINEN2	CTI1 CTI Trigger 2 to Channel Enable Register	0x00000000
0x3110202C	CTI1_CTIINEN3	CTI1 CTI Trigger 3 to Channel Enable Register	0x00000000
0x31102030	CTI1_CTIINEN4	CTI1 CTI Trigger 4 to Channel Enable Register	0x00000000
0x31102034	CTI1_CTIINEN5	CTI1 CTI Trigger 5 to Channel Enable Register	0x00000000
0x31102038	CTI1_CTIINEN6	CTI1 CTI Trigger 6 to Channel Enable Register	0x00000000
0x3110203C	CTI1_CTIINEN7	CTI1 CTI Trigger 7 to Channel Enable Register	0x00000000
0x311020A0	CTI1_CTIOUTEN0	CTI1 CTI Channel to Trigger 0 Enable Register	0x00000000
0x311020A4	CTI1_CTIOUTEN1	CTI1 CTI Channel to Trigger 1 Enable Register	0x00000000

Table A-19: ADSP-2159x CTI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x311020A8	CTI1_CTIOUTEN2	CTI1 CTI Channel to Trigger 2 Enable Register	0x00000000
0x311020AC	CTI1_CTIOUTEN3	CTI1 CTI Channel to Trigger 3 Enable Register	0x00000000
0x311020B0	CTI1_CTIOUTEN4	CTI1 CTI Channel to Trigger 4 Enable Register	0x00000000
0x311020B4	CTI1_CTIOUTEN5	CTI1 CTI Channel to Trigger 5 Enable Register	0x00000000
0x311020B8	CTI1_CTIOUTEN6	CTI1 CTI Channel to Trigger 6 Enable Register	0x00000000
0x311020BC	CTI1_CTIOUTEN7	CTI1 CTI Channel to Trigger 7 Enable Register	0x00000000
0x31102130	CTI1_CTITRIGINSTATUS	CTI1 CTI Trigger In Status Register	0x00000000
0x31102134	CTI1_CTITRIGOUTSTA- TUS	CTI1 CTI Trigger Out Status Register	0x00000000
0x31102138	CTI1_CTICHINSTATUS	CTI1 CTI Channel In Status Register	0x00000000
0x3110213C	CTI1_CTICHOUTSTATUS	CTI1 CTI Channel Out Status Register	0x00000000
0x31102140	CTI1_CTIGATE	CTI1 Enable CTI Channel Gate Register	0x0000000F
0x31102144	CTI1_ASICCTL	CTI1 External Multiplexor Control Register	0x00000000
0x31102EDC	CTI1_ITCHINACK	CTI1 ITCHINACK	0x00000000
0x31102EE0	CTI1_ITTRIGINACK	CTI1 ITTRIGINACK	0x00000000
0x31102EE4	CTI1_ITCHOUT	CTI1 ITCHOUT	0x00000000
0x31102EE8	CTI1_ITTRIGOUT	CTI1 ITTRIGOUT	0x00000000
0x31102EEC	CTI1_ITCHOUTACK	CTI1 ITCHOUTACK	0x00000000
0x31102EF0	CTI1_ITTRIGOUTACK	CTI1 ITTRIGOUTACK	0x00000000
0x31102EF4	CTI1_ITCHIN	CTI1 ITCHIN	0x00000000
0x31102EF8	CTI1_ITTRIGIN	CTI1 ITTRIGIN	0x00000000
0x31102F00	CTI1_ITCTRL	CTI1 Integration Mode Control Register	0x00000000
0x31102FA0	CTI1_CLAIMSET	CTI1 Claim Tag Set Register	0x0000000F
0x31102FA4	CTI1_CLAIMCLR	CTI1 Claim Tag Clear Register	0x00000000
0x31102FB0	CTI1_LAR	CTI1 Lock Access Register	0x00000000
0x31102FB4	CTI1_LSR	CTI1 Lock Status Register	0x00000003
0x31102FB8	CTI1_AUTHSTATUS	CTI1 Authentication Status	0x00000005
0x31102FC8	CTI1_DEVID	CTI1 Device ID	0x00040800
0x31102FCC	CTI1_DEVTYPE	CTI1 Device Type	0x00000014
0x31102FD0	CTI1_PERIPHID4	CTI1 Peripheral ID4	0x00000004
0x31102FD4	CTI1_PERIPHID5	CTI1 Peripheral ID5	0x00000000

Table A-19: ADSP-2159x CTI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31102FD8	CTI1_PERIPHID6	CTI1 Peripheral ID6	0x00000000
0x31102FDC	CTI1_PERIPHID7	CTI1 Peripheral ID7	0x00000000
0x31102FE0	CTI1_PERIPHID0	CTI1 Peripheral ID0	0x00000006
0x31102FE4	CTI1_PERIPHID1	CTI1 Peripheral ID1	0x000000B9
0x31102FE8	CTI1_PERIPHID2	CTI1 Peripheral ID2	0x0000003B
0x31102FEC	CTI1_PERIPHID3	CTI1 Peripheral ID3	0x00000000
0x31102FF0	CTI1_COMPID0	CTI1 Component ID0	0x0000000D
0x31102FF4	CTI1_COMPID1	CTI1 Component ID1	0x00000090
0x31102FF8	CTI1_COMPID2	CTI1 Component ID2	0x00000005
0x31102FFC	CTI1_COMPID3	CTI1 Component ID3	0x000000B1

Table A-20: ADSP-2159x CTI2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31106000	CTI2_CTICONTROL	CTI2 CTI Control Register	0x00000000
0x31106010	CTI2_CTIINTACK	CTI2 CTI Interrupt Acknowledge Register	0x00000000
0x31106014	CTI2_CTIAPPSET	CTI2 CTI Application Trigger Set Register	0x00000000
0x31106018	CTI2_CTIAPPCLEAR	CTI2 CTI Application Trigger Clear Register	0x00000000
0x3110601C	CTI2_CTIAPPPULSE	CTI2 CTI Application Pulse Register	0x00000000
0x31106020	CTI2_CTIINEN0	CTI2 CTI Trigger 0 to Channel Enable Register	0x00000000
0x31106024	CTI2_CTIINEN1	CTI2 CTI Trigger 1 to Channel Enable Register	0x00000000
0x31106028	CTI2_CTIINEN2	CTI2 CTI Trigger 2 to Channel Enable Register	0x00000000
0x3110602C	CTI2_CTIINEN3	CTI2 CTI Trigger 3 to Channel Enable Register	0x00000000
0x31106030	CTI2_CTIINEN4	CTI2 CTI Trigger 4 to Channel Enable Register	0x00000000
0x31106034	CTI2_CTIINEN5	CTI2 CTI Trigger 5 to Channel Enable Register	0x00000000
0x31106038	CTI2_CTIINEN6	CTI2 CTI Trigger 6 to Channel Enable Register	0x00000000
0x3110603C	CTI2_CTIINEN7	CTI2 CTI Trigger 7 to Channel Enable Register	0x00000000
0x311060A0	CTI2_CTIOUTEN0	CTI2 CTI Channel to Trigger 0 Enable Register	0x00000000
0x311060A4	CTI2_CTIOUTEN1	CTI2 CTI Channel to Trigger 1 Enable Register	0x00000000
0x311060A8	CTI2_CTIOUTEN2	CTI2 CTI Channel to Trigger 2 Enable Register	0x00000000
0x311060AC	CTI2_CTIOUTEN3	CTI2 CTI Channel to Trigger 3 Enable Register	0x00000000

Table A-20: ADSP-2159x CTI2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x311060B0	CTI2_CTIOUTEN4	CTI2 CTI Channel to Trigger 4 Enable Register	0x00000000
0x311060B4	CTI2_CTIOUTEN5	CTI2 CTI Channel to Trigger 5 Enable Register	0x00000000
0x311060B8	CTI2_CTIOUTEN6	CTI2 CTI Channel to Trigger 6 Enable Register	0x00000000
0x311060BC	CTI2_CTIOUTEN7	CTI2 CTI Channel to Trigger 7 Enable Register	0x00000000
0x31106130	CTI2_CTITRIGINSTATUS	CTI2 CTI Trigger In Status Register	0x00000000
0x31106134	CTI2_CTITRIGOUTSTA- TUS	CTI2 CTI Trigger Out Status Register	0x00000000
0x31106138	CTI2_CTICHINSTATUS	CTI2 CTI Channel In Status Register	0x00000000
0x3110613C	CTI2_CTICHOUTSTATUS	CTI2 CTI Channel Out Status Register	0x00000000
0x31106140	CTI2_CTIGATE	CTI2 Enable CTI Channel Gate Register	0x0000000F
0x31106144	CTI2_ASICCTL	CTI2 External Multiplexor Control Register	0x00000000
0x31106EDC	CTI2_ITCHINACK	CTI2 ITCHINACK	0x00000000
0x31106EE0	CTI2_ITTRIGINACK	CTI2 ITTRIGINACK	0x00000000
0x31106EE4	CTI2_ITCHOUT	CTI2 ITCHOUT	0x00000000
0x31106EE8	CTI2_ITTRIGOUT	CTI2 ITTRIGOUT	0x00000000
0x31106EEC	CTI2_ITCHOUTACK	CTI2 ITCHOUTACK	0x00000000
0x31106EF0	CTI2_ITTRIGOUTACK	CTI2 ITTRIGOUTACK	0x00000000
0x31106EF4	CTI2_ITCHIN	CTI2 ITCHIN	0x00000000
0x31106EF8	CTI2_ITTRIGIN	CTI2 ITTRIGIN	0x00000000
0x31106F00	CTI2_ITCTRL	CTI2 Integration Mode Control Register	0x00000000
0x31106FA0	CTI2_CLAIMSET	CTI2 Claim Tag Set Register	0x0000000F
0x31106FA4	CTI2_CLAIMCLR	CTI2 Claim Tag Clear Register	0x00000000
0x31106FB0	CTI2_LAR	CTI2 Lock Access Register	0x00000000
0x31106FB4	CTI2_LSR	CTI2 Lock Status Register	0x00000003
0x31106FB8	CTI2_AUTHSTATUS	CTI2 Authentication Status	0x00000005
0x31106FC8	CTI2_DEVID	CTI2 Device ID	0x00040800
0x31106FCC	CTI2_DEVTYPE	CTI2 Device Type	0x00000014
0x31106FD0	CTI2_PERIPHID4	CTI2 Peripheral ID4	0x00000004
0x31106FD4	CTI2_PERIPHID5	CTI2 Peripheral ID5	0x00000000
0x31106FD8	CTI2_PERIPHID6	CTI2 Peripheral ID6	0x00000000
0x31106FDC	CTI2_PERIPHID7	CTI2 Peripheral ID7	0x00000000

Table A-20: ADSP-2159x CTI2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31106FE0	CTI2_PERIPHID0	CTI2 Peripheral ID0	0x00000006
0x31106FE4	CTI2_PERIPHID1	CTI2 Peripheral ID1	0x000000B9
0x31106FE8	CTI2_PERIPHID2	CTI2 Peripheral ID2	0x0000003B
0x31106FEC	CTI2_PERIPHID3	CTI2 Peripheral ID3	0x00000000
0x31106FF0	CTI2_COMPID0	CTI2 Component ID0	0x0000000D
0x31106FF4	CTI2_COMPID1	CTI2 Component ID1	0x00000090
0x31106FF8	CTI2_COMPID2	CTI2 Component ID2	0x00000005
0x31106FFC	CTI2_COMPID3	CTI2 Component ID3	0x000000B1

Table A-21: ADSP-2159x CTI3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110D000	CTI3_CTICONTROL	CTI3 CTI Control Register	0x00000000
0x3110D010	CTI3_CTIINTACK	CTI3 CTI Interrupt Acknowledge Register	0x00000000
0x3110D014	CTI3_CTIAPPSET	CTI3 CTI Application Trigger Set Register	0x00000000
0x3110D018	CTI3_CTIAPPCLEAR	CTI3 CTI Application Trigger Clear Register	0x00000000
0x3110D01C	CTI3_CTIAPPPULSE	CTI3 CTI Application Pulse Register	0x00000000
0x3110D020	CTI3_CTIINEN0	CTI3 CTI Trigger 0 to Channel Enable Register	0x00000000
0x3110D024	CTI3_CTIINEN1	CTI3 CTI Trigger 1 to Channel Enable Register	0x00000000
0x3110D028	CTI3_CTIINEN2	CTI3 CTI Trigger 2 to Channel Enable Register	0x00000000
0x3110D02C	CTI3_CTIINEN3	CTI3 CTI Trigger 3 to Channel Enable Register	0x00000000
0x3110D030	CTI3_CTIINEN4	CTI3 CTI Trigger 4 to Channel Enable Register	0x00000000
0x3110D034	CTI3_CTIINEN5	CTI3 CTI Trigger 5 to Channel Enable Register	0x00000000
0x3110D038	CTI3_CTIINEN6	CTI3 CTI Trigger 6 to Channel Enable Register	0x00000000
0x3110D03C	CTI3_CTIINEN7	CTI3 CTI Trigger 7 to Channel Enable Register	0x00000000
0x3110D0A0	CTI3_CTIOUTEN0	CTI3 CTI Channel to Trigger 0 Enable Register	0x00000000
0x3110D0A4	CTI3_CTIOUTEN1	CTI3 CTI Channel to Trigger 1 Enable Register	0x00000000
0x3110D0A8	CTI3_CTIOUTEN2	CTI3 CTI Channel to Trigger 2 Enable Register	0x00000000
0x3110D0AC	CTI3_CTIOUTEN3	CTI3 CTI Channel to Trigger 3 Enable Register	0x00000000
0x3110D0B0	CTI3_CTIOUTEN4	CTI3 CTI Channel to Trigger 4 Enable Register	0x00000000
0x3110D0B4	CTI3_CTIOUTEN5	CTI3 CTI Channel to Trigger 5 Enable Register	0x00000000

Table A-21: ADSP-2159x CTI3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110D0B8	CTI3_CTIOUTEN6	CTI3 CTI Channel to Trigger 6 Enable Register	0x00000000
0x3110D0BC	CTI3_CTIOUTEN7	CTI3 CTI Channel to Trigger 7 Enable Register	0x00000000
0x3110D130	CTI3_CTITRIGINSTATUS	CTI3 CTI Trigger In Status Register	0x00000000
0x3110D134	CTI3_CTITRIGOUTSTA- TUS	CTI3 CTI Trigger Out Status Register	0x00000000
0x3110D138	CTI3_CTICHINSTATUS	CTI3 CTI Channel In Status Register	0x00000000
0x3110D13C	CTI3_CTICHOUTSTATUS	CTI3 CTI Channel Out Status Register	0x00000000
0x3110D140	CTI3_CTIGATE	CTI3 Enable CTI Channel Gate Register	0x0000000F
0x3110D144	CTI3_ASICCTL	CTI3 External Multiplexor Control Register	0x00000000
0x3110DEDC	CTI3_ITCHINACK	CTI3 ITCHINACK	0x00000000
0x3110DEE0	CTI3_ITTRIGINACK	CTI3 ITTRIGINACK	0x00000000
0x3110DEE4	CTI3_ITCHOUT	CTI3 ITCHOUT	0x00000000
0x3110DEE8	CTI3_ITTRIGOUT	CTI3 ITTRIGOUT	0x00000000
0x3110DEEC	CTI3_ITCHOUTACK	CTI3 ITCHOUTACK	0x00000000
0x3110DEF0	CTI3_ITTRIGOUTACK	CTI3 ITTRIGOUTACK	0x00000000
0x3110DEF4	CTI3_ITCHIN	CTI3 ITCHIN	0x00000000
0x3110DEF8	CTI3_ITTRIGIN	CTI3 ITTRIGIN	0x00000000
0x3110DF00	CTI3_ITCTRL	CTI3 Integration Mode Control Register	0x00000000
0x3110DFA0	CTI3_CLAIMSET	CTI3 Claim Tag Set Register	0x0000000F
0x3110DFA4	CTI3_CLAIMCLR	CTI3 Claim Tag Clear Register	0x00000000
0x3110DFB0	CTI3_LAR	CTI3 Lock Access Register	0x00000000
0x3110DFB4	CTI3_LSR	CTI3 Lock Status Register	0x00000003
0x3110DFB8	CTI3_AUTHSTATUS	CTI3 Authentication Status	0x00000005
0x3110DFC8	CTI3_DEVID	CTI3 Device ID	0x00040800
0x3110DFCC	CTI3_DEVTYPE	CTI3 Device Type	0x00000014
0x3110DFD0	CTI3_PERIPHID4	CTI3 Peripheral ID4	0x00000004
0x3110DFD4	CTI3_PERIPHID5	CTI3 Peripheral ID5	0x00000000
0x3110DFD8	CTI3_PERIPHID6	CTI3 Peripheral ID6	0x00000000
0x3110DFDC	CTI3_PERIPHID7	CTI3 Peripheral ID7	0x00000000
0x3110DFE0	CTI3_PERIPHID0	CTI3 Peripheral ID0	0x00000006
0x3110DFE4	CTI3_PERIPHID1	CTI3 Peripheral ID1	0x000000B9

Table A-21: ADSP-2159x CTI3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110DFE8	CTI3_PERIPHID2	CTI3 Peripheral ID2	0x0000003B
0x3110DFEC	CTI3_PERIPHID3	CTI3 Peripheral ID3	0x00000000
0x3110DFF0	CTI3_COMPID0	CTI3 Component ID0	0x0000000D
0x3110DFF4	CTI3_COMPID1	CTI3 Component ID1	0x00000090
0x3110DFF8	CTI3_COMPID2	CTI3 Component ID2	0x00000005
0x3110DFFC	CTI3_COMPID3	CTI3 Component ID3	0x000000B1

Table A-22: ADSP-2159x CTI4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110C000	CTI4_CTICONTROL	CTI4 CTI Control Register	0x00000000
0x3110C010	CTI4_CTIINTACK	CTI4 CTI Interrupt Acknowledge Register	0x00000000
0x3110C014	CTI4_CTIAPPSET	CTI4 CTI Application Trigger Set Register	0x00000000
0x3110C018	CTI4_CTIAPPCLEAR	CTI4 CTI Application Trigger Clear Register	0x00000000
0x3110C01C	CTI4_CTIAPPPULSE	CTI4 CTI Application Pulse Register	0x00000000
0x3110C020	CTI4_CTIINEN0	CTI4 CTI Trigger 0 to Channel Enable Register	0x00000000
0x3110C024	CTI4_CTIINEN1	CTI4 CTI Trigger 1 to Channel Enable Register	0x00000000
0x3110C028	CTI4_CTIINEN2	CTI4 CTI Trigger 2 to Channel Enable Register	0x00000000
0x3110C02C	CTI4_CTIINEN3	CTI4 CTI Trigger 3 to Channel Enable Register	0x00000000
0x3110C030	CTI4_CTIINEN4	CTI4 CTI Trigger 4 to Channel Enable Register	0x00000000
0x3110C034	CTI4_CTIINEN5	CTI4 CTI Trigger 5 to Channel Enable Register	0x00000000
0x3110C038	CTI4_CTIINEN6	CTI4 CTI Trigger 6 to Channel Enable Register	0x00000000
0x3110C03C	CTI4_CTIINEN7	CTI4 CTI Trigger 7 to Channel Enable Register	0x00000000
0x3110C0A0	CTI4_CTIOUTEN0	CTI4 CTI Channel to Trigger 0 Enable Register	0x00000000
0x3110C0A4	CTI4_CTIOUTEN1	CTI4 CTI Channel to Trigger 1 Enable Register	0x00000000
0x3110C0A8	CTI4_CTIOUTEN2	CTI4 CTI Channel to Trigger 2 Enable Register	0x00000000
0x3110C0AC	CTI4_CTIOUTEN3	CTI4 CTI Channel to Trigger 3 Enable Register	0x00000000
0x3110C0B0	CTI4_CTIOUTEN4	CTI4 CTI Channel to Trigger 4 Enable Register	0x00000000
0x3110C0B4	CTI4_CTIOUTEN5	CTI4 CTI Channel to Trigger 5 Enable Register	0x00000000
0x3110C0B8	CTI4_CTIOUTEN6	CTI4 CTI Channel to Trigger 6 Enable Register	0x00000000
0x3110C0BC	CTI4_CTIOUTEN7	CTI4 CTI Channel to Trigger 7 Enable Register	0x00000000

Table A-22: ADSP-2159x CTI4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3110C130	CTI4_CTITRIGINSTATUS	CTI4 CTI Trigger In Status Register	0x00000000
0x3110C134	CTI4_CTITRIGOUTSTA- TUS	CTI4 CTI Trigger Out Status Register	0x00000000
0x3110C138	CTI4_CTICHINSTATUS	CTI4 CTI Channel In Status Register	0x00000000
0x3110C13C	CTI4_CTICHOUTSTATUS	CTI4 CTI Channel Out Status Register	0x00000000
0x3110C140	CTI4_CTIGATE	CTI4 Enable CTI Channel Gate Register	0x0000000F
0x3110C144	CTI4_ASICCTL	CTI4 External Multiplexor Control Register	0x00000000
0x3110CEDC	CTI4_ITCHINACK	CTI4 ITCHINACK	0x00000000
0x3110CEE0	CTI4_ITTRIGINACK	CTI4 ITTRIGINACK	0x00000000
0x3110CEE4	CTI4_ITCHOUT	CTI4 ITCHOUT	0x00000000
0x3110CEE8	CTI4_ITTRIGOUT	CTI4 ITTRIGOUT	0x00000000
0x3110CEEC	CTI4_ITCHOUTACK	CTI4 ITCHOUTACK	0x00000000
0x3110CEF0	CTI4_ITTRIGOUTACK	CTI4 ITTRIGOUTACK	0x00000000
0x3110CEF4	CTI4_ITCHIN	CTI4 ITCHIN	0x00000000
0x3110CEF8	CTI4_ITTRIGIN	CTI4 ITTRIGIN	0x00000000
0x3110CF00	CTI4_ITCTRL	CTI4 Integration Mode Control Register	0x00000000
0x3110CFA0	CTI4_CLAIMSET	CTI4 Claim Tag Set Register	0x0000000F
0x3110CFA4	CTI4_CLAIMCLR	CTI4 Claim Tag Clear Register	0x00000000
0x3110CFB0	CTI4_LAR	CTI4 Lock Access Register	0x00000000
0x3110CFB4	CTI4_LSR	CTI4 Lock Status Register	0x00000003
0x3110CFB8	CTI4_AUTHSTATUS	CTI4 Authentication Status	0x00000005
0x3110CFC8	CTI4_DEVID	CTI4 Device ID	0x00040800
0x3110CFCC	CTI4_DEVTYPE	CTI4 Device Type	0x00000014
0x3110CFD0	CTI4_PERIPHID4	CTI4 Peripheral ID4	0x00000004
0x3110CFD4	CTI4_PERIPHID5	CTI4 Peripheral ID5	0x00000000
0x3110CFD8	CTI4_PERIPHID6	CTI4 Peripheral ID6	0x00000000
0x3110CFDC	CTI4_PERIPHID7	CTI4 Peripheral ID7	0x00000000
0x3110CFE0	CTI4_PERIPHID0	CTI4 Peripheral ID0	0x00000006
0x3110CFE4	CTI4_PERIPHID1	CTI4 Peripheral ID1	0x000000B9
0x3110CFE8	CTI4_PERIPHID2	CTI4 Peripheral ID2	0x0000003B
0x3110CFEC	CTI4_PERIPHID3	CTI4 Peripheral ID3	0x00000000

Table A-22: ADSP-2159x CTI4 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3110CFF0	CTI4_COMPID0	CTI4 Component ID0	0x0000000D
0x3110CFF4	CTI4_COMPID1	CTI4 Component ID1	0x00000090
0x3110CFF8	CTI4_COMPID2	CTI4 Component ID2	0x00000005
0x3110CFFC	CTI4_COMPID3	CTI4 Component ID3	0x000000B1

Table A-23: ADSP-2159x CSPFT0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31103000	CSPFT0_CTL	CSPFT0 Main Control Register	0x20000400
0x31103004	CSPFT0_HWFEAT	CSPFT0 Hardware Feature Register	0x01484002
0x31103008	CSPFT0_TRIGGER	CSPFT0 Trigger Event Register	0x000077EF
0x31103010	CSPFT0_STAT	CSPFT0 Status Register	0x00000002
0x31103018	CSPFT0_TSSCTL	CSPFT0 TraceEnable Start/Stop Control Register	0x00000000
0x31103020	CSPFT0_TEEVENT	CSPFT0 TraceEnable Event Register	0x000077EF
0x31103024	CSPFT0_TECTL	CSPFT0 TraceEnable Control Register	0x00000000
0x31103040	CSPFT0_ACVR[n]	CSPFT0 Address Comparator Value Register	0x00000000
0x31103044	CSPFT0_ACVR[n]	CSPFT0 Address Comparator Value Register	0x00000000
0x31103048	CSPFT0_ACVR[n]	CSPFT0 Address Comparator Value Register	0x00000000
0x3110304C	CSPFT0_ACVR[n]	CSPFT0 Address Comparator Value Register	0x00000000
0x31103080	CSPFT0_ACTR[n]	CSPFT0 Address Comparator Access Type Register	0x00000000
0x31103084	CSPFT0_ACTR[n]	CSPFT0 Address Comparator Access Type Register	0x00000000
0x31103088	CSPFT0_ACTR[n]	CSPFT0 Address Comparator Access Type Register	0x00000000
0x3110308C	CSPFT0_ACTR[n]	CSPFT0 Address Comparator Access Type Register	0x00000000
0x31103140	CSPFT0_CNTRL DVR[n]	CSPFT0 Counter Reload Value Register	0x00000000
0x31103144	CSPFT0_CNTRL DVR[n]	CSPFT0 Counter Reload Value Register	0x00000000
0x31103150	CSPFT0_CNTENR[n]	CSPFT0 Counter Enable Event Register	0x000077EF
0x31103154	CSPFT0_CNTENR[n]	CSPFT0 Counter Enable Event Register	0x000077EF
0x31103160	CSPFT0_CNTRL DEVR[n]	CSPFT0 Counter Reload Event Register	0x000077EF
0x31103164	CSPFT0_CNTRL DEVR[n]	CSPFT0 Counter Reload Event Register	0x000077EF
0x31103170	CSPFT0_CNTVR[n]	CSPFT0 Counter Value Register	0x00000000
0x31103174	CSPFT0_CNTVR[n]	CSPFT0 Counter Value Register	0x00000000

Table A-23: ADSP-2159x CSPFT0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x311031A0	CSPFT0_EXTOUTEVR[n]	CSPFT0 External Output Event Register	0x000077EF
0x311031A4	CSPFT0_EXTOUTEVR[n]	CSPFT0 External Output Event Register	0x000077EF
0x311031A8	CSPFT0_EXTOUTEVR[n]	CSPFT0 External Output Event Register	0x000077EF
0x311031AC	CSPFT0_EXTOUTEVR[n]	CSPFT0 External Output Event Register	0x000077EF
0x311031B0	CSPFT0_CIDCVR[n]	CSPFT0 Context ID Comparator Value	0x00000000
0x311031BC	CSPFT0_CIDCMR	CSPFT0 Context ID Comparator Mask Register	0x00000000
0x311031E0	CSPFT0_SYNCFR	CSPFT0 Synchronization Frequency Register	0x00000000
0x311031E8	CSPFT0_CCER	CSPFT0 Configuration Code Extension Register	0x00800000
0x31103200	CSPFT0_TRACEIDR	CSPFT0 CoreSight Trace ID Register	0x00000000
0x31103FA0	CSPFT0_CLAIMSET	CSPFT0 Claim Tag Set Register	0x0000000F
0x31103FA4	CSPFT0_CLAIMCLR	CSPFT0 Claim Tag Clear Register	0x00000000
0x31103FB0	CSPFT0_LAR	CSPFT0 Lock Access Register	0x00000000
0x31103FB4	CSPFT0_LSR	CSPFT0 Lock Status Register	0x00000003
0x31103FB8	CSPFT0_AUTHSTATUS	CSPFT0 Authentication Status Register	0x00000080
0x31103FCC	CSPFT0_DEVTTYPE	CSPFT0 Device Type Identifier Register	0x00000023
0x31103FD0	CSPFT0_PID4	CSPFT0 Peripheral ID4 Register	0x00000000
0x31103FE0	CSPFT0_PID0	CSPFT0 Peripheral ID0 Register	0x00000001
0x31103FE4	CSPFT0_PID1	CSPFT0 Peripheral ID1 Register	0x00000050
0x31103FE8	CSPFT0_PID2	CSPFT0 Peripheral ID2 Register	0x0000000E
0x31103FEC	CSPFT0_PID3	CSPFT0 Peripheral ID3 Register	0x00000000
0x31103FF0	CSPFT0_CID0	CSPFT0 Component ID0 Register	0x0000000D
0x31103FF4	CSPFT0_CID1	CSPFT0 Component ID1 Register	0x00000090
0x31103FF8	CSPFT0_CID2	CSPFT0 Component ID2 Register	0x00000005
0x31103FFC	CSPFT0_CID3	CSPFT0 Component ID3 Register	0x000000B1

Table A-24: ADSP-2159x CSPFT1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31107000	CSPFT1_CTL	CSPFT1 Main Control Register	0x20000400
0x31107004	CSPFT1_HWFEAT	CSPFT1 Hardware Feature Register	0x01484002
0x31107008	CSPFT1_TRIGGER	CSPFT1 Trigger Event Register	0x000077EF

Table A-24: ADSP-2159x CSPFT1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31107010	CSPFT1_STAT	CSPFT1 Status Register	0x00000002
0x31107018	CSPFT1_TSSCTL	CSPFT1 TraceEnable Start/Stop Control Register	0x00000000
0x31107020	CSPFT1_TEEVENT	CSPFT1 TraceEnable Event Register	0x000077EF
0x31107024	CSPFT1_TECTL	CSPFT1 TraceEnable Control Register	0x00000000
0x31107040	CSPFT1_ACVR[n]	CSPFT1 Address Comparator Value Register	0x00000000
0x31107044	CSPFT1_ACVR[n]	CSPFT1 Address Comparator Value Register	0x00000000
0x31107048	CSPFT1_ACVR[n]	CSPFT1 Address Comparator Value Register	0x00000000
0x3110704C	CSPFT1_ACVR[n]	CSPFT1 Address Comparator Value Register	0x00000000
0x31107080	CSPFT1_ACTR[n]	CSPFT1 Address Comparator Access Type Register	0x00000000
0x31107084	CSPFT1_ACTR[n]	CSPFT1 Address Comparator Access Type Register	0x00000000
0x31107088	CSPFT1_ACTR[n]	CSPFT1 Address Comparator Access Type Register	0x00000000
0x3110708C	CSPFT1_ACTR[n]	CSPFT1 Address Comparator Access Type Register	0x00000000
0x31107140	CSPFT1_CNTRLDVR[n]	CSPFT1 Counter Reload Value Register	0x00000000
0x31107144	CSPFT1_CNTRLDVR[n]	CSPFT1 Counter Reload Value Register	0x00000000
0x31107150	CSPFT1_CNTENR[n]	CSPFT1 Counter Enable Event Register	0x000077EF
0x31107154	CSPFT1_CNTENR[n]	CSPFT1 Counter Enable Event Register	0x000077EF
0x31107160	CSPFT1_CNTRLDEVR[n]	CSPFT1 Counter Reload Event Register	0x000077EF
0x31107164	CSPFT1_CNTRLDEVR[n]	CSPFT1 Counter Reload Event Register	0x000077EF
0x31107170	CSPFT1_CNTVR[n]	CSPFT1 Counter Value Register	0x00000000
0x31107174	CSPFT1_CNTVR[n]	CSPFT1 Counter Value Register	0x00000000
0x311071A0	CSPFT1_EXTOUTEVR[n]	CSPFT1 External Output Event Register	0x000077EF
0x311071A4	CSPFT1_EXTOUTEVR[n]	CSPFT1 External Output Event Register	0x000077EF
0x311071A8	CSPFT1_EXTOUTEVR[n]	CSPFT1 External Output Event Register	0x000077EF
0x311071AC	CSPFT1_EXTOUTEVR[n]	CSPFT1 External Output Event Register	0x000077EF
0x311071B0	CSPFT1_CIDCVR[n]	CSPFT1 Context ID Comparator Value	0x00000000
0x311071BC	CSPFT1_CIDCMR	CSPFT1 Context ID Comparator Mask Register	0x00000000
0x311071E0	CSPFT1_SYNCFR	CSPFT1 Synchronization Frequency Register	0x00000000
0x311071E8	CSPFT1_CCER	CSPFT1 Configuration Code Extension Register	0x00800000
0x31107200	CSPFT1_TRACEIDR	CSPFT1 CoreSight Trace ID Register	0x00000000
0x31107FA0	CSPFT1_CLAIMSET	CSPFT1 Claim Tag Set Register	0x0000000F
0x31107FA4	CSPFT1_CLAIMCLR	CSPFT1 Claim Tag Clear Register	0x00000000

Table A-24: ADSP-2159x CSPFT1 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31107FB0	CSPFT1_LAR	CSPFT1 Lock Access Register	0x00000000
0x31107FB4	CSPFT1_LSR	CSPFT1 Lock Status Register	0x00000003
0x31107FB8	CSPFT1_AUTHSTATUS	CSPFT1 Authentication Status Register	0x00000080
0x31107FCC	CSPFT1_DEVTYPE	CSPFT1 Device Type Identifier Register	0x00000023
0x31107FD0	CSPFT1_PID4	CSPFT1 Peripheral ID4 Register	0x00000000
0x31107FE0	CSPFT1_PID0	CSPFT1 Peripheral ID0 Register	0x00000001
0x31107FE4	CSPFT1_PID1	CSPFT1 Peripheral ID1 Register	0x00000050
0x31107FE8	CSPFT1_PID2	CSPFT1 Peripheral ID2 Register	0x0000000E
0x31107FEC	CSPFT1_PID3	CSPFT1 Peripheral ID3 Register	0x00000000
0x31107FF0	CSPFT1_CID0	CSPFT1 Component ID0 Register	0x0000000D
0x31107FF4	CSPFT1_CID1	CSPFT1 Component ID1 Register	0x00000090
0x31107FF8	CSPFT1_CID2	CSPFT1 Component ID2 Register	0x00000005
0x31107FFC	CSPFT1_CID3	CSPFT1 Component ID3 Register	0x000000B1

Table A-25: ADSP-2159x DAI0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310C9000	DAI0_EXTD_CLK0	DAI0 Extended Clock Routing Control Register 0	0x00000000
0x310C9004	DAI0_EXTD_CLK1	DAI0 Extended Clock Routing Control Register 1	0x00000000
0x310C9008	DAI0_EXTD_CLK2	DAI0 Extended Clock Routing Control Register 2	0x00000000
0x310C900C	DAI0_EXTD_CLK3	DAI0 Extended Clock Routing Control Register 3	0x00000000
0x310C9010	DAI0_EXTD_CLK4	DAI0 Extended Clock Routing Control Register 4	0x00000000
0x310C9014	DAI0_EXTD_CLK5	DAI0 Extended Clock Routing Control Register 5	0x00000000
0x310C9018	DAI0_EXTD_DAT0	DAI0 Extended Serial Data Routing Control Register 0	0x00000000
0x310C901C	DAI0_EXTD_DAT1	DAI0 Extended Serial Data Routing Control Register 1	0x00000000
0x310C9020	DAI0_EXTD_DAT2	DAI0 Extended Serial Data Routing Control Register 2	0x00000000
0x310C9024	DAI0_EXTD_DAT3	DAI0 Extended Serial Data Routing Control Register 3	0x00000000
0x310C9028	DAI0_EXTD_DAT4	DAI0 Extended Serial Data Routing Control Register 4	0x00000000
0x310C902C	DAI0_EXTD_DAT5	DAI0 Extended Serial Data Routing Control Register 5	0x00000000
0x310C9030	DAI0_EXTD_DAT6	DAI0 Extended Serial Data Routing Control Register 6	0x00000000
0x310C9034	DAI0_EXTD_FS0	DAI0 Extended Frame Sync Routing Control Register 0	0x00000000

Table A-25: ADSP-2159x DAI0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C9038	DAI0_EXTD_FS1	DAI0 Extended Frame Sync Routing Control Register 1	0x00000000
0x310C903C	DAI0_EXTD_FS2	DAI0 Extended Frame Sync Routing Control Register 2	0x00000000
0x310C9044	DAI0_EXTD_FS4	DAI0 Extended Frame Sync Routing Control Register 4	0x00000000
0x310C9048	DAI0_EXTD_PIN0	DAI0 Extended Pin Buffer Assignment Register 0	0x00000000
0x310C904C	DAI0_EXTD_PIN1	DAI0 Extended Pin Buffer Assignment Register 1	0x00000000
0x310C9050	DAI0_EXTD_PIN2	DAI0 Extended Pin Buffer Assignment Register 2	0x00000000
0x310C9054	DAI0_EXTD_PIN3	DAI0 Extended Pin Buffer Assignment Register 3	0x00000000
0x310C9058	DAI0_EXTD_PIN4	DAI0 Extended Pin Buffer Assignment Register 4	0x00000000
0x310C905C	DAI0_EXTD_MISC0	DAI0 Extended Miscellaneous Control Register 0	0x00000000
0x310C9060	DAI0_EXTD_MISC1	DAI0 Extended Miscellaneous Control Register 1	0x00000000
0x310C9064	DAI0_EXTD_MISC2	DAI0 Extended Miscellaneous Control Register 2	0x00000000
0x310C9068	DAI0_EXTD_PBEN0	DAI0 Extended Pin Buffer Enable Register 0	0x00000000
0x310C906C	DAI0_EXTD_PBEN1	DAI0 Extended Pin Buffer Enable Register 1	0x00000000
0x310C9070	DAI0_EXTD_PBEN2	DAI0 Extended Pin Buffer Enable Register 2	0x00000000
0x310C9074	DAI0_EXTD_PBEN3	DAI0 Extended Pin Buffer Enable Register 3	0x00000000
0x310C90C0	DAI0_CLK0	DAI0 Clock Routing Control Register 0	0x252630C2
0x310C90C4	DAI0_CLK1	DAI0 Clock Routing Control Register 1	0x3DEF7BDE
0x310C90C8	DAI0_CLK2	DAI0 Clock Routing Control Register 2	0x01EF7BDE
0x310C90CC	DAI0_CLK3	DAI0 Clock Routing Control Register 3	0x3C000000
0x310C90D0	DAI0_CLK4	DAI0 Clock Routing Control Register 4	0x3DEF03DE
0x310C90D4	DAI0_CLK5	DAI0 Clock Routing Control Register 5	0x3DEF7BDE
0x310C9100	DAI0_DAT0	DAI0 Serial Data Routing Control Register 0	0x08144040
0x310C9104	DAI0_DAT1	DAI0 Serial Data Routing Control Register 1	0x0F38B289
0x310C9108	DAI0_DAT2	DAI0 Serial Data Routing Control Register 2	0x00000450
0x310C910C	DAI0_DAT3	DAI0 Serial Data Routing Control Register 3	0x00000000
0x310C9110	DAI0_DAT4	DAI0 Serial Data Routing Control Register 4	0x00000000
0x310C9114	DAI0_DAT5	DAI0 Serial Data Routing Control Register 5	0x00000000
0x310C9118	DAI0_DAT6	DAI0 Serial Data Routing Control Register 6	0x00FBFEBE
0x310C9140	DAI0_FS0	DAI0 Frame Sync Routing Control Register 0	0x2736B4E3
0x310C9144	DAI0_FS1	DAI0 Frame Sync Routing Control Register 1	0x3DEF7BDE
0x310C9148	DAI0_FS2	DAI0 Frame Sync Routing Control Register 2	0x00007BDE

Table A-25: ADSP-2159x DAI0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C9150	DAI0_FS4	DAI0 Frame Sync Routing Control Register 4	0x000F7BDE
0x310C9180	DAI0_PIN0	DAI0 Pin Buffer Assignment Register 0	0x04C80A94
0x310C9184	DAI0_PIN1	DAI0 Pin Buffer Assignment Register 1	0x04E84B96
0x310C9188	DAI0_PIN2	DAI0 Pin Buffer Assignment Register 2	0x03668C98
0x310C918C	DAI0_PIN3	DAI0 Pin Buffer Assignment Register 3	0x03A714A3
0x310C9190	DAI0_PIN4	DAI0 Pin Buffer Assignment Register 4	0x05694F9E
0x310C91C0	DAI0_MISC0	DAI0 Miscellaneous Control Register 0	0x3DEF7BDE
0x310C91C4	DAI0_MISC1	DAI0 Miscellaneous Control Register 1	0x3DEF7BDE
0x310C91C8	DAI0_MISC2	DAI0 Miscellaneous Control Register 1	0x000F7BDE
0x310C91E0	DAI0_PBEN0	DAI0 Pin Buffer Enable Register 0	0x0E2482CA
0x310C91E4	DAI0_PBEN1	DAI0 Pin Buffer Enable Register 1	0x1348D30F
0x310C91E8	DAI0_PBEN2	DAI0 Pin Buffer Enable Register 2	0x1A5545D6
0x310C91EC	DAI0_PBEN3	DAI0 Pin Buffer Enable Register 3	0x1D71F79B
0x310C9200	DAI0_IMSK_FE	DAI0 Falling-Edge Interrupt Mask Register	0x00000000
0x310C9204	DAI0_IMSK_RE	DAI0 Rising-Edge Interrupt Mask Register	0x00000000
0x310C9210	DAI0_IMSK_PRI	DAI0 Core Interrupt Priority Assignment Register	0x00000000
0x310C9220	DAI0_IRPTL_H	DAI0 High Priority Interrupt Latch Register	0x00000000
0x310C9224	DAI0_IRPTL_L	DAI0 Low Priority Interrupt Latch Register	0x00000000
0x310C9230	DAI0_IRPTL_HS	DAI0 Shadow High Priority Interrupt Latch Register	0x00000000
0x310C9234	DAI0_IRPTL_LS	DAI0 Shadow Low Priority Interrupt Latch Register	0x00000000
0x310C92E4	DAI0_PIN_STAT	DAI0 Pin Status Register	0x00000000
0x310C92E8	DAI0_GBL_SP_EN	DAI0 Global SPORT Enable Register	0x00000000
0x310C92EC	DAI0_GBL_INT_EN	DAI0 Global SPORT Interrupt Grouping Register	0x00000000
0x310C92F0	DAI0_GBL_PCG_EN	DAI0 Global PCG Enable Control Register	0x00000000

Table A-26: ADSP-2159x DAI1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA000	DAI1_EXTD_CLK0	DAI1 Extended Clock Routing Control Register 0	0x00000000
0x310CA004	DAI1_EXTD_CLK1	DAI1 Extended Clock Routing Control Register 1	0x00000000
0x310CA008	DAI1_EXTD_CLK2	DAI1 Extended Clock Routing Control Register 2	0x00000000

Table A-26: ADSP-2159x DAI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA00C	DAI1_EXTD_CLK3	DAI1 Extended Clock Routing Control Register 3	0x00000000
0x310CA010	DAI1_EXTD_CLK4	DAI1 Extended Clock Routing Control Register 4	0x00000000
0x310CA014	DAI1_EXTD_CLK5	DAI1 Extended Clock Routing Control Register 5	0x00000000
0x310CA018	DAI1_EXTD_DAT0	DAI1 Extended Serial Data Routing Control Register 0	0x00000000
0x310CA01C	DAI1_EXTD_DAT1	DAI1 Extended Serial Data Routing Control Register 1	0x00000000
0x310CA020	DAI1_EXTD_DAT2	DAI1 Extended Serial Data Routing Control Register 2	0x00000000
0x310CA024	DAI1_EXTD_DAT3	DAI1 Extended Serial Data Routing Control Register 3	0x00000000
0x310CA028	DAI1_EXTD_DAT4	DAI1 Extended Serial Data Routing Control Register 4	0x00000000
0x310CA02C	DAI1_EXTD_DAT5	DAI1 Extended Serial Data Routing Control Register 5	0x00000000
0x310CA030	DAI1_EXTD_DAT6	DAI1 Extended Serial Data Routing Control Register 6	0x00000000
0x310CA034	DAI1_EXTD_FS0	DAI1 Extended Frame Sync Routing Control Register 0	0x00000000
0x310CA038	DAI1_EXTD_FS1	DAI1 Extended Frame Sync Routing Control Register 1	0x00000000
0x310CA03C	DAI1_EXTD_FS2	DAI1 Extended Frame Sync Routing Control Register 2	0x00000000
0x310CA044	DAI1_EXTD_FS4	DAI1 Extended Frame Sync Routing Control Register 4	0x00000000
0x310CA048	DAI1_EXTD_PIN0	DAI1 Extended Pin Buffer Assignment Register 0	0x00000000
0x310CA04C	DAI1_EXTD_PIN1	DAI1 Extended Pin Buffer Assignment Register 1	0x00000000
0x310CA050	DAI1_EXTD_PIN2	DAI1 Extended Pin Buffer Assignment Register 2	0x00000000
0x310CA054	DAI1_EXTD_PIN3	DAI1 Extended Pin Buffer Assignment Register 3	0x00000000
0x310CA058	DAI1_EXTD_PIN4	DAI1 Extended Pin Buffer Assignment Register 4	0x00000000
0x310CA05C	DAI1_EXTD_MISC0	DAI1 Extended Miscellaneous Control Register 0	0x00000000
0x310CA060	DAI1_EXTD_MISC1	DAI1 Extended Miscellaneous Control Register 1	0x00000000
0x310CA064	DAI1_EXTD_MISC2	DAI1 Extended Miscellaneous Control Register 2	0x00000000
0x310CA068	DAI1_EXTD_PBEN0	DAI1 Extended Pin Buffer Enable Register 0	0x00000000
0x310CA06C	DAI1_EXTD_PBEN1	DAI1 Extended Pin Buffer Enable Register 1	0x00000000
0x310CA070	DAI1_EXTD_PBEN2	DAI1 Extended Pin Buffer Enable Register 2	0x00000000
0x310CA074	DAI1_EXTD_PBEN3	DAI1 Extended Pin Buffer Enable Register 3	0x00000000
0x310CA0C0	DAI1_CLK0	DAI1 Clock Routing Control Register 0	0x252630C2
0x310CA0C4	DAI1_CLK1	DAI1 Clock Routing Control Register 1	0x3DEF7BDE
0x310CA0C8	DAI1_CLK2	DAI1 Clock Routing Control Register 2	0x01EF7BDE
0x310CA0CC	DAI1_CLK3	DAI1 Clock Routing Control Register 3	0x3C000000
0x310CA0D0	DAI1_CLK4	DAI1 Clock Routing Control Register 4	0x3DEF03DE

Table A-26: ADSP-2159x DAI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA0D4	DAI1_CLK5	DAI1 Clock Routing Control Register 5	0x3DEF7BDE
0x310CA100	DAI1_DAT0	DAI1 Serial Data Routing Control Register 0	0x08144040
0x310CA104	DAI1_DAT1	DAI1 Serial Data Routing Control Register 1	0x0F38B289
0x310CA108	DAI1_DAT2	DAI1 Serial Data Routing Control Register 2	0x00000450
0x310CA10C	DAI1_DAT3	DAI1 Serial Data Routing Control Register 3	0x00000000
0x310CA110	DAI1_DAT4	DAI1 Serial Data Routing Control Register 4	0x00000000
0x310CA114	DAI1_DAT5	DAI1 Serial Data Routing Control Register 5	0x00000000
0x310CA118	DAI1_DAT6	DAI1 Serial Data Routing Control Register 6	0x00FBFEBE
0x310CA140	DAI1_FS0	DAI1 Frame Sync Routing Control Register 0	0x2736B4E3
0x310CA144	DAI1_FS1	DAI1 Frame Sync Routing Control Register 1	0x3DEF7BDE
0x310CA148	DAI1_FS2	DAI1 Frame Sync Routing Control Register 2	0x00007BDE
0x310CA150	DAI1_FS4	DAI1 Frame Sync Routing Control Register 4	0x000F7BDE
0x310CA180	DAI1_PIN0	DAI1 Pin Buffer Assignment Register 0	0x04C80A94
0x310CA184	DAI1_PIN1	DAI1 Pin Buffer Assignment Register 1	0x04E84B96
0x310CA188	DAI1_PIN2	DAI1 Pin Buffer Assignment Register 2	0x03668C98
0x310CA18C	DAI1_PIN3	DAI1 Pin Buffer Assignment Register 3	0x03A714A3
0x310CA190	DAI1_PIN4	DAI1 Pin Buffer Assignment Register 4	0x05694F9E
0x310CA1C0	DAI1_MISC0	DAI1 Miscellaneous Control Register 0	0x3DEF7BDE
0x310CA1C4	DAI1_MISC1	DAI1 Miscellaneous Control Register 1	0x3DEF7BDE
0x310CA1C8	DAI1_MISC2	DAI1 Miscellaneous Control Register 1	0x000F7BDE
0x310CA1E0	DAI1_PBEN0	DAI1 Pin Buffer Enable Register 0	0x0E2482CA
0x310CA1E4	DAI1_PBEN1	DAI1 Pin Buffer Enable Register 1	0x1348D30F
0x310CA1E8	DAI1_PBEN2	DAI1 Pin Buffer Enable Register 2	0x1A5545D6
0x310CA1EC	DAI1_PBEN3	DAI1 Pin Buffer Enable Register 3	0x1D71F79B
0x310CA200	DAI1_IMSK_FE	DAI1 Falling-Edge Interrupt Mask Register	0x00000000
0x310CA204	DAI1_IMSK_RE	DAI1 Rising-Edge Interrupt Mask Register	0x00000000
0x310CA210	DAI1_IMSK_PRI	DAI1 Core Interrupt Priority Assignment Register	0x00000000
0x310CA220	DAI1_IRPTL_H	DAI1 High Priority Interrupt Latch Register	0x00000000
0x310CA224	DAI1_IRPTL_L	DAI1 Low Priority Interrupt Latch Register	0x00000000
0x310CA230	DAI1_IRPTL_HS	DAI1 Shadow High Priority Interrupt Latch Register	0x00000000
0x310CA234	DAI1_IRPTL_LS	DAI1 Shadow Low Priority Interrupt Latch Register	0x00000000

Table A-26: ADSP-2159x DAI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA2E4	DAI1_PIN_STAT	DAI1 Pin Status Register	0x00000000
0x310CA2E8	DAI1_GBL_SP_EN	DAI1 Global SPORT Enable Register	0x00000000
0x310CA2EC	DAI1_GBL_INT_EN	DAI1 Global SPORT Interrupt Grouping Register	0x00000000

Table A-27: ADSP-2159x DAPROM0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31100000	DAPROM0_ROMENTRY00	DAPROM0 ROM Entry 00	0x00001003
0x31100004	DAPROM0_ROMENTRY01	DAPROM0 ROM Entry 01	0x00002003
0x31100008	DAPROM0_ROMENTRY02	DAPROM0 ROM Entry 02	0x00003003
0x3110000C	DAPROM0_ROMENTRY03	DAPROM0 ROM Entry 03	0x00004003
0x31100010	DAPROM0_ROMENTRY04	DAPROM0 ROM Entry 04	0x00005003
0x31100014	DAPROM0_ROMENTRY05	DAPROM0 ROM Entry 05	0x00006003
0x31100018	DAPROM0_ROMENTRY06	DAPROM0 ROM Entry 06	0x00007003
0x3110001C	DAPROM0_ROMENTRY07	DAPROM0 ROM Entry 07	0x00008003
0x31100020	DAPROM0_ROMENTRY08	DAPROM0 ROM Entry 08	0x00009003
0x31100024	DAPROM0_ROMENTRY09	DAPROM0 ROM Entry 09	0x0000A003
0x31100028	DAPROM0_ROMENTRY10	DAPROM0 ROM Entry 10	0x0000B003
0x3110002C	DAPROM0_ROMENTRY11	DAPROM0 ROM Entry 11	0x0000C003
0x31100030	DAPROM0_ROMENTRY12	DAPROM0 ROM Entry 12	0x0000D003
0x31100034	DAPROM0_ROMENTRY13	DAPROM0 ROM Entry 13	0x00010003

Table A-28: ADSP-2159x DMA0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022000	DMA0_DSCPTR_NXT	DMA0 Pointer to Next Initial Descriptor Register	0x00000000
0x31022004	DMA0_ADDRSTART	DMA0 Start Address of Current Buffer Register	0x00000000
0x31022008	DMA0_CFG	DMA0 Configuration Register	0x00000000
0x3102200C	DMA0_XCNT	DMA0 Inner Loop Count Start Value Register	0x00000000
0x31022010	DMA0_XMOD	DMA0 Inner Loop Address Increment Register	0x00000000
0x31022014	DMA0_YCNT	DMA0 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022018	DMA0_YMOD	DMA0 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-28: ADSP-2159x DMA0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022024	DMA0_DSCPTR_CUR	DMA0 Current Descriptor Pointer Register	0x00000000
0x31022028	DMA0_DSCPTR_PRV	DMA0 Previous Initial Descriptor Pointer Register	0x00000000
0x3102202C	DMA0_ADDR_CUR	DMA0 Current Address Register	0x00000000
0x31022030	DMA0_STAT	DMA0 Status Register	0x00006000
0x31022034	DMA0_XCNT_CUR	DMA0 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31022038	DMA0_YCNT_CUR	DMA0 Current Row Count (2D only) Register	0x00000000
0x31022040	DMA0_BWLCNT	DMA0 Bandwidth Limit Count Register	0x00000000
0x31022044	DMA0_BWLCNT_CUR	DMA0 Bandwidth Limit Count Current Register	0x00000000
0x31022048	DMA0_BWMCNT	DMA0 Bandwidth Monitor Count Register	0x00000000
0x3102204C	DMA0_BWMCNT_CUR	DMA0 Bandwidth Monitor Count Current Register	0x00000000

Table A-29: ADSP-2159x DMA1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022080	DMA1_DSCPTR_NXT	DMA1 Pointer to Next Initial Descriptor Register	0x00000000
0x31022084	DMA1_ADDRSTART	DMA1 Start Address of Current Buffer Register	0x00000000
0x31022088	DMA1_CFG	DMA1 Configuration Register	0x00000000
0x3102208C	DMA1_XCNT	DMA1 Inner Loop Count Start Value Register	0x00000000
0x31022090	DMA1_XMOD	DMA1 Inner Loop Address Increment Register	0x00000000
0x31022094	DMA1_YCNT	DMA1 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022098	DMA1_YMOD	DMA1 Outer Loop Address Increment (2D only) Register	0x00000000
0x310220A4	DMA1_DSCPTR_CUR	DMA1 Current Descriptor Pointer Register	0x00000000
0x310220A8	DMA1_DSCPTR_PRV	DMA1 Previous Initial Descriptor Pointer Register	0x00000000
0x310220AC	DMA1_ADDR_CUR	DMA1 Current Address Register	0x00000000
0x310220B0	DMA1_STAT	DMA1 Status Register	0x00006000
0x310220B4	DMA1_XCNT_CUR	DMA1 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310220B8	DMA1_YCNT_CUR	DMA1 Current Row Count (2D only) Register	0x00000000
0x310220C0	DMA1_BWLCNT	DMA1 Bandwidth Limit Count Register	0x00000000
0x310220C4	DMA1_BWLCNT_CUR	DMA1 Bandwidth Limit Count Current Register	0x00000000
0x310220C8	DMA1_BWMCNT	DMA1 Bandwidth Monitor Count Register	0x00000000

Table A-29: ADSP-2159x DMA1 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310220CC	DMA1_BWMCNT_CUR	DMA1 Bandwidth Monitor Count Current Register	0x00000000

Table A-30: ADSP-2159x DMA10 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023000	DMA10_DSCPTR_NXT	DMA10 Pointer to Next Initial Descriptor Register	0x00000000
0x31023004	DMA10_ADDRSTART	DMA10 Start Address of Current Buffer Register	0x00000000
0x31023008	DMA10_CFG	DMA10 Configuration Register	0x00000000
0x3102300C	DMA10_XCNT	DMA10 Inner Loop Count Start Value Register	0x00000000
0x31023010	DMA10_XMOD	DMA10 Inner Loop Address Increment Register	0x00000000
0x31023014	DMA10_YCNT	DMA10 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023018	DMA10_YMOD	DMA10 Outer Loop Address Increment (2D only) Register	0x00000000
0x31023024	DMA10_DSCPTR_CUR	DMA10 Current Descriptor Pointer Register	0x00000000
0x31023028	DMA10_DSCPTR_PRV	DMA10 Previous Initial Descriptor Pointer Register	0x00000000
0x3102302C	DMA10_ADDR_CUR	DMA10 Current Address Register	0x00000000
0x31023030	DMA10_STAT	DMA10 Status Register	0x00006000
0x31023034	DMA10_XCNT_CUR	DMA10 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31023038	DMA10_YCNT_CUR	DMA10 Current Row Count (2D only) Register	0x00000000
0x31023040	DMA10_BWLCNT	DMA10 Bandwidth Limit Count Register	0x00000000
0x31023044	DMA10_BWLCNT_CUR	DMA10 Bandwidth Limit Count Current Register	0x00000000
0x31023048	DMA10_BWMCNT	DMA10 Bandwidth Monitor Count Register	0x00000000
0x3102304C	DMA10_BWMCNT_CUR	DMA10 Bandwidth Monitor Count Current Register	0x00000000

Table A-31: ADSP-2159x DMA11 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023080	DMA11_DSCPTR_NXT	DMA11 Pointer to Next Initial Descriptor Register	0x00000000
0x31023084	DMA11_ADDRSTART	DMA11 Start Address of Current Buffer Register	0x00000000
0x31023088	DMA11_CFG	DMA11 Configuration Register	0x00000000
0x3102308C	DMA11_XCNT	DMA11 Inner Loop Count Start Value Register	0x00000000

Table A-31: ADSP-2159x DMA11 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31023090	DMA11_XMOD	DMA11 Inner Loop Address Increment Register	0x00000000
0x31023094	DMA11_YCNT	DMA11 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023098	DMA11_YMOD	DMA11 Outer Loop Address Increment (2D only) Register	0x00000000
0x310230A4	DMA11_DSCPTR_CUR	DMA11 Current Descriptor Pointer Register	0x00000000
0x310230A8	DMA11_DSCPTR_PRV	DMA11 Previous Initial Descriptor Pointer Register	0x00000000
0x310230AC	DMA11_ADDR_CUR	DMA11 Current Address Register	0x00000000
0x310230B0	DMA11_STAT	DMA11 Status Register	0x00006000
0x310230B4	DMA11_XCNT_CUR	DMA11 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310230B8	DMA11_YCNT_CUR	DMA11 Current Row Count (2D only) Register	0x00000000
0x310230C0	DMA11_BWLCNT	DMA11 Bandwidth Limit Count Register	0x00000000
0x310230C4	DMA11_BWLCNT_CUR	DMA11 Bandwidth Limit Count Current Register	0x00000000
0x310230C8	DMA11_BWMCNT	DMA11 Bandwidth Monitor Count Register	0x00000000
0x310230CC	DMA11_BWMCNT_CUR	DMA11 Bandwidth Monitor Count Current Register	0x00000000

Table A-32: ADSP-2159x DMA12 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31023100	DMA12_DSCPTR_NXT	DMA12 Pointer to Next Initial Descriptor Register	0x00000000
0x31023104	DMA12_ADDRSTART	DMA12 Start Address of Current Buffer Register	0x00000000
0x31023108	DMA12_CFG	DMA12 Configuration Register	0x00000000
0x3102310C	DMA12_XCNT	DMA12 Inner Loop Count Start Value Register	0x00000000
0x31023110	DMA12_XMOD	DMA12 Inner Loop Address Increment Register	0x00000000
0x31023114	DMA12_YCNT	DMA12 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023118	DMA12_YMOD	DMA12 Outer Loop Address Increment (2D only) Register	0x00000000
0x31023124	DMA12_DSCPTR_CUR	DMA12 Current Descriptor Pointer Register	0x00000000
0x31023128	DMA12_DSCPTR_PRV	DMA12 Previous Initial Descriptor Pointer Register	0x00000000
0x3102312C	DMA12_ADDR_CUR	DMA12 Current Address Register	0x00000000
0x31023130	DMA12_STAT	DMA12 Status Register	0x00006000

Table A-32: ADSP-2159x DMA12 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31023134	DMA12_XCNT_CUR	DMA12 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31023138	DMA12_YCNT_CUR	DMA12 Current Row Count (2D only) Register	0x00000000
0x31023140	DMA12_BWLCNT	DMA12 Bandwidth Limit Count Register	0x00000000
0x31023144	DMA12_BWLCNT_CUR	DMA12 Bandwidth Limit Count Current Register	0x00000000
0x31023148	DMA12_BWMCNT	DMA12 Bandwidth Monitor Count Register	0x00000000
0x3102314C	DMA12_BWMCNT_CUR	DMA12 Bandwidth Monitor Count Current Register	0x00000000

Table A-33: ADSP-2159x DMA13 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31023180	DMA13_DSCPTR_NXT	DMA13 Pointer to Next Initial Descriptor Register	0x00000000
0x31023184	DMA13_ADDRSTART	DMA13 Start Address of Current Buffer Register	0x00000000
0x31023188	DMA13_CFG	DMA13 Configuration Register	0x00000000
0x3102318C	DMA13_XCNT	DMA13 Inner Loop Count Start Value Register	0x00000000
0x31023190	DMA13_XMOD	DMA13 Inner Loop Address Increment Register	0x00000000
0x31023194	DMA13_YCNT	DMA13 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023198	DMA13_YMOD	DMA13 Outer Loop Address Increment (2D only) Register	0x00000000
0x310231A4	DMA13_DSCPTR_CUR	DMA13 Current Descriptor Pointer Register	0x00000000
0x310231A8	DMA13_DSCPTR_PRV	DMA13 Previous Initial Descriptor Pointer Register	0x00000000
0x310231AC	DMA13_ADDR_CUR	DMA13 Current Address Register	0x00000000
0x310231B0	DMA13_STAT	DMA13 Status Register	0x00006000
0x310231B4	DMA13_XCNT_CUR	DMA13 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310231B8	DMA13_YCNT_CUR	DMA13 Current Row Count (2D only) Register	0x00000000
0x310231C0	DMA13_BWLCNT	DMA13 Bandwidth Limit Count Register	0x00000000
0x310231C4	DMA13_BWLCNT_CUR	DMA13 Bandwidth Limit Count Current Register	0x00000000
0x310231C8	DMA13_BWMCNT	DMA13 Bandwidth Monitor Count Register	0x00000000
0x310231CC	DMA13_BWMCNT_CUR	DMA13 Bandwidth Monitor Count Current Register	0x00000000

Table A-34: ADSP-2159x DMA14 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31023200	DMA14_DSCPTR_NXT	DMA14 Pointer to Next Initial Descriptor Register	0x00000000
0x31023204	DMA14_ADDRSTART	DMA14 Start Address of Current Buffer Register	0x00000000
0x31023208	DMA14_CFG	DMA14 Configuration Register	0x00000000
0x3102320C	DMA14_XCNT	DMA14 Inner Loop Count Start Value Register	0x00000000
0x31023210	DMA14_XMOD	DMA14 Inner Loop Address Increment Register	0x00000000
0x31023214	DMA14_YCNT	DMA14 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023218	DMA14_YMOD	DMA14 Outer Loop Address Increment (2D only) Register	0x00000000
0x31023224	DMA14_DSCPTR_CUR	DMA14 Current Descriptor Pointer Register	0x00000000
0x31023228	DMA14_DSCPTR_PRV	DMA14 Previous Initial Descriptor Pointer Register	0x00000000
0x3102322C	DMA14_ADDR_CUR	DMA14 Current Address Register	0x00000000
0x31023230	DMA14_STAT	DMA14 Status Register	0x00006000
0x31023234	DMA14_XCNT_CUR	DMA14 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31023238	DMA14_YCNT_CUR	DMA14 Current Row Count (2D only) Register	0x00000000
0x31023240	DMA14_BWLCNT	DMA14 Bandwidth Limit Count Register	0x00000000
0x31023244	DMA14_BWLCNT_CUR	DMA14 Bandwidth Limit Count Current Register	0x00000000
0x31023248	DMA14_BWMCNT	DMA14 Bandwidth Monitor Count Register	0x00000000
0x3102324C	DMA14_BWMCNT_CUR	DMA14 Bandwidth Monitor Count Current Register	0x00000000

Table A-35: ADSP-2159x DMA15 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31023280	DMA15_DSCPTR_NXT	DMA15 Pointer to Next Initial Descriptor Register	0x00000000
0x31023284	DMA15_ADDRSTART	DMA15 Start Address of Current Buffer Register	0x00000000
0x31023288	DMA15_CFG	DMA15 Configuration Register	0x00000000
0x3102328C	DMA15_XCNT	DMA15 Inner Loop Count Start Value Register	0x00000000
0x31023290	DMA15_XMOD	DMA15 Inner Loop Address Increment Register	0x00000000
0x31023294	DMA15_YCNT	DMA15 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023298	DMA15_YMOD	DMA15 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-35: ADSP-2159x DMA15 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310232A4	DMA15_DSCPTR_CUR	DMA15 Current Descriptor Pointer Register	0x00000000
0x310232A8	DMA15_DSCPTR_PRV	DMA15 Previous Initial Descriptor Pointer Register	0x00000000
0x310232AC	DMA15_ADDR_CUR	DMA15 Current Address Register	0x00000000
0x310232B0	DMA15_STAT	DMA15 Status Register	0x00006000
0x310232B4	DMA15_XCNT_CUR	DMA15 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310232B8	DMA15_YCNT_CUR	DMA15 Current Row Count (2D only) Register	0x00000000
0x310232C0	DMA15_BWLCNT	DMA15 Bandwidth Limit Count Register	0x00000000
0x310232C4	DMA15_BWLCNT_CUR	DMA15 Bandwidth Limit Count Current Register	0x00000000
0x310232C8	DMA15_BWMCNT	DMA15 Bandwidth Monitor Count Register	0x00000000
0x310232CC	DMA15_BWMCNT_CUR	DMA15 Bandwidth Monitor Count Current Register	0x00000000

Table A-36: ADSP-2159x DMA16 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023300	DMA16_DSCPTR_NXT	DMA16 Pointer to Next Initial Descriptor Register	0x00000000
0x31023304	DMA16_ADDRSTART	DMA16 Start Address of Current Buffer Register	0x00000000
0x31023308	DMA16_CFG	DMA16 Configuration Register	0x00000000
0x3102330C	DMA16_XCNT	DMA16 Inner Loop Count Start Value Register	0x00000000
0x31023310	DMA16_XMOD	DMA16 Inner Loop Address Increment Register	0x00000000
0x31023314	DMA16_YCNT	DMA16 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023318	DMA16_YMOD	DMA16 Outer Loop Address Increment (2D only) Register	0x00000000
0x31023324	DMA16_DSCPTR_CUR	DMA16 Current Descriptor Pointer Register	0x00000000
0x31023328	DMA16_DSCPTR_PRV	DMA16 Previous Initial Descriptor Pointer Register	0x00000000
0x3102332C	DMA16_ADDR_CUR	DMA16 Current Address Register	0x00000000
0x31023330	DMA16_STAT	DMA16 Status Register	0x00006000
0x31023334	DMA16_XCNT_CUR	DMA16 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31023338	DMA16_YCNT_CUR	DMA16 Current Row Count (2D only) Register	0x00000000
0x31023340	DMA16_BWLCNT	DMA16 Bandwidth Limit Count Register	0x00000000
0x31023344	DMA16_BWLCNT_CUR	DMA16 Bandwidth Limit Count Current Register	0x00000000

Table A-36: ADSP-2159x DMA16 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023348	DMA16_BWMCNT	DMA16 Bandwidth Monitor Count Register	0x00000000
0x3102334C	DMA16_BWMCNT_CUR	DMA16 Bandwidth Monitor Count Current Register	0x00000000

Table A-37: ADSP-2159x DMA17 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31023380	DMA17_DSCPTR_NXT	DMA17 Pointer to Next Initial Descriptor Register	0x00000000
0x31023384	DMA17_ADDRSTART	DMA17 Start Address of Current Buffer Register	0x00000000
0x31023388	DMA17_CFG	DMA17 Configuration Register	0x00000000
0x3102338C	DMA17_XCNT	DMA17 Inner Loop Count Start Value Register	0x00000000
0x31023390	DMA17_XMOD	DMA17 Inner Loop Address Increment Register	0x00000000
0x31023394	DMA17_YCNT	DMA17 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31023398	DMA17_YMOD	DMA17 Outer Loop Address Increment (2D only) Register	0x00000000
0x310233A4	DMA17_DSCPTR_CUR	DMA17 Current Descriptor Pointer Register	0x00000000
0x310233A8	DMA17_DSCPTR_PRV	DMA17 Previous Initial Descriptor Pointer Register	0x00000000
0x310233AC	DMA17_ADDR_CUR	DMA17 Current Address Register	0x00000000
0x310233B0	DMA17_STAT	DMA17 Status Register	0x00006000
0x310233B4	DMA17_XCNT_CUR	DMA17 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310233B8	DMA17_YCNT_CUR	DMA17 Current Row Count (2D only) Register	0x00000000
0x310233C0	DMA17_BWLCNT	DMA17 Bandwidth Limit Count Register	0x00000000
0x310233C4	DMA17_BWLCNT_CUR	DMA17 Bandwidth Limit Count Current Register	0x00000000
0x310233C8	DMA17_BWMCNT	DMA17 Bandwidth Monitor Count Register	0x00000000
0x310233CC	DMA17_BWMCNT_CUR	DMA17 Bandwidth Monitor Count Current Register	0x00000000

Table A-38: ADSP-2159x DMA18 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A7100	DMA18_DSCPTR_NXT	DMA18 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7104	DMA18_ADDRSTART	DMA18 Start Address of Current Buffer Register	0x00000000
0x310A7108	DMA18_CFG	DMA18 Configuration Register	0x00000000

Table A-38: ADSP-2159x DMA18 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A710C	DMA18_XCNT	DMA18 Inner Loop Count Start Value Register	0x00000000
0x310A7110	DMA18_XMOD	DMA18 Inner Loop Address Increment Register	0x00000000
0x310A7114	DMA18_YCNT	DMA18 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7118	DMA18_YMOD	DMA18 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A7124	DMA18_DSCPTR_CUR	DMA18 Current Descriptor Pointer Register	0x00000000
0x310A7128	DMA18_DSCPTR_PRV	DMA18 Previous Initial Descriptor Pointer Register	0x00000000
0x310A712C	DMA18_ADDR_CUR	DMA18 Current Address Register	0x00000000
0x310A7130	DMA18_STAT	DMA18 Status Register	0x00006000
0x310A7134	DMA18_XCNT_CUR	DMA18 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A7138	DMA18_YCNT_CUR	DMA18 Current Row Count (2D only) Register	0x00000000
0x310A7140	DMA18_BWLCNT	DMA18 Bandwidth Limit Count Register	0x00000000
0x310A7144	DMA18_BWLCNT_CUR	DMA18 Bandwidth Limit Count Current Register	0x00000000
0x310A7148	DMA18_BWMCNT	DMA18 Bandwidth Monitor Count Register	0x00000000
0x310A714C	DMA18_BWMCNT_CUR	DMA18 Bandwidth Monitor Count Current Register	0x00000000

Table A-39: ADSP-2159x DMA19 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7180	DMA19_DSCPTR_NXT	DMA19 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7184	DMA19_ADDRSTART	DMA19 Start Address of Current Buffer Register	0x00000000
0x310A7188	DMA19_CFG	DMA19 Configuration Register	0x00000000
0x310A718C	DMA19_XCNT	DMA19 Inner Loop Count Start Value Register	0x00000000
0x310A7190	DMA19_XMOD	DMA19 Inner Loop Address Increment Register	0x00000000
0x310A7194	DMA19_YCNT	DMA19 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7198	DMA19_YMOD	DMA19 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A71A4	DMA19_DSCPTR_CUR	DMA19 Current Descriptor Pointer Register	0x00000000
0x310A71A8	DMA19_DSCPTR_PRV	DMA19 Previous Initial Descriptor Pointer Register	0x00000000
0x310A71AC	DMA19_ADDR_CUR	DMA19 Current Address Register	0x00000000

Table A-39: ADSP-2159x DMA19 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A71B0	DMA19_STAT	DMA19 Status Register	0x00006000
0x310A71B4	DMA19_XCNT_CUR	DMA19 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A71B8	DMA19_YCNT_CUR	DMA19 Current Row Count (2D only) Register	0x00000000
0x310A71C0	DMA19_BWLCNT	DMA19 Bandwidth Limit Count Register	0x00000000
0x310A71C4	DMA19_BWLCNT_CUR	DMA19 Bandwidth Limit Count Current Register	0x00000000
0x310A71C8	DMA19_BWMCNT	DMA19 Bandwidth Monitor Count Register	0x00000000
0x310A71CC	DMA19_BWMCNT_CUR	DMA19 Bandwidth Monitor Count Current Register	0x00000000

Table A-40: ADSP-2159x DMA2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022100	DMA2_DSCPTR_NXT	DMA2 Pointer to Next Initial Descriptor Register	0x00000000
0x31022104	DMA2_ADDRSTART	DMA2 Start Address of Current Buffer Register	0x00000000
0x31022108	DMA2_CFG	DMA2 Configuration Register	0x00000000
0x3102210C	DMA2_XCNT	DMA2 Inner Loop Count Start Value Register	0x00000000
0x31022110	DMA2_XMOD	DMA2 Inner Loop Address Increment Register	0x00000000
0x31022114	DMA2_YCNT	DMA2 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022118	DMA2_YMOD	DMA2 Outer Loop Address Increment (2D only) Register	0x00000000
0x31022124	DMA2_DSCPTR_CUR	DMA2 Current Descriptor Pointer Register	0x00000000
0x31022128	DMA2_DSCPTR_PRV	DMA2 Previous Initial Descriptor Pointer Register	0x00000000
0x3102212C	DMA2_ADDR_CUR	DMA2 Current Address Register	0x00000000
0x31022130	DMA2_STAT	DMA2 Status Register	0x00006000
0x31022134	DMA2_XCNT_CUR	DMA2 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31022138	DMA2_YCNT_CUR	DMA2 Current Row Count (2D only) Register	0x00000000
0x31022140	DMA2_BWLCNT	DMA2 Bandwidth Limit Count Register	0x00000000
0x31022144	DMA2_BWLCNT_CUR	DMA2 Bandwidth Limit Count Current Register	0x00000000
0x31022148	DMA2_BWMCNT	DMA2 Bandwidth Monitor Count Register	0x00000000
0x3102214C	DMA2_BWMCNT_CUR	DMA2 Bandwidth Monitor Count Current Register	0x00000000

Table A-41: ADSP-2159x DMA20 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026080	DMA20_DSCPTR_NXT	DMA20 Pointer to Next Initial Descriptor Register	0x00000000
0x31026084	DMA20_ADDRSTART	DMA20 Start Address of Current Buffer Register	0x00000000
0x31026088	DMA20_CFG	DMA20 Configuration Register	0x00000000
0x3102608C	DMA20_XCNT	DMA20 Inner Loop Count Start Value Register	0x00000000
0x31026090	DMA20_XMOD	DMA20 Inner Loop Address Increment Register	0x00000000
0x31026094	DMA20_YCNT	DMA20 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026098	DMA20_YMOD	DMA20 Outer Loop Address Increment (2D only) Register	0x00000000
0x310260A4	DMA20_DSCPTR_CUR	DMA20 Current Descriptor Pointer Register	0x00000000
0x310260A8	DMA20_DSCPTR_PRV	DMA20 Previous Initial Descriptor Pointer Register	0x00000000
0x310260AC	DMA20_ADDR_CUR	DMA20 Current Address Register	0x00000000
0x310260B0	DMA20_STAT	DMA20 Status Register	0x00006000
0x310260B4	DMA20_XCNT_CUR	DMA20 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310260B8	DMA20_YCNT_CUR	DMA20 Current Row Count (2D only) Register	0x00000000
0x310260C0	DMA20_BWLCNT	DMA20 Bandwidth Limit Count Register	0x00000000
0x310260C4	DMA20_BWLCNT_CUR	DMA20 Bandwidth Limit Count Current Register	0x00000000
0x310260C8	DMA20_BWMCNT	DMA20 Bandwidth Monitor Count Register	0x00000000
0x310260CC	DMA20_BWMCNT_CUR	DMA20 Bandwidth Monitor Count Current Register	0x00000000

Table A-42: ADSP-2159x DMA21 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026000	DMA21_DSCPTR_NXT	DMA21 Pointer to Next Initial Descriptor Register	0x00000000
0x31026004	DMA21_ADDRSTART	DMA21 Start Address of Current Buffer Register	0x00000000
0x31026008	DMA21_CFG	DMA21 Configuration Register	0x00000000
0x3102600C	DMA21_XCNT	DMA21 Inner Loop Count Start Value Register	0x00000000
0x31026010	DMA21_XMOD	DMA21 Inner Loop Address Increment Register	0x00000000
0x31026014	DMA21_YCNT	DMA21 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026018	DMA21_YMOD	DMA21 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-42: ADSP-2159x DMA21 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31026024	DMA21_DSCPTR_CUR	DMA21 Current Descriptor Pointer Register	0x00000000
0x31026028	DMA21_DSCPTR_PRV	DMA21 Previous Initial Descriptor Pointer Register	0x00000000
0x3102602C	DMA21_ADDR_CUR	DMA21 Current Address Register	0x00000000
0x31026030	DMA21_STAT	DMA21 Status Register	0x00006000
0x31026034	DMA21_XCNT_CUR	DMA21 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31026038	DMA21_YCNT_CUR	DMA21 Current Row Count (2D only) Register	0x00000000
0x31026040	DMA21_BWLCNT	DMA21 Bandwidth Limit Count Register	0x00000000
0x31026044	DMA21_BWLCNT_CUR	DMA21 Bandwidth Limit Count Current Register	0x00000000
0x31026048	DMA21_BWMCNT	DMA21 Bandwidth Monitor Count Register	0x00000000
0x3102604C	DMA21_BWMCNT_CUR	DMA21 Bandwidth Monitor Count Current Register	0x00000000

Table A-43: ADSP-2159x DMA22 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102D000	DMA22_DSCPTR_NXT	DMA22 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D004	DMA22_ADDRSTART	DMA22 Start Address of Current Buffer Register	0x00000000
0x3102D008	DMA22_CFG	DMA22 Configuration Register	0x00000000
0x3102D00C	DMA22_XCNT	DMA22 Inner Loop Count Start Value Register	0x00000000
0x3102D010	DMA22_XMOD	DMA22 Inner Loop Address Increment Register	0x00000000
0x3102D014	DMA22_YCNT	DMA22 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D018	DMA22_YMOD	DMA22 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D024	DMA22_DSCPTR_CUR	DMA22 Current Descriptor Pointer Register	0x00000000
0x3102D028	DMA22_DSCPTR_PRV	DMA22 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D02C	DMA22_ADDR_CUR	DMA22 Current Address Register	0x00000000
0x3102D030	DMA22_STAT	DMA22 Status Register	0x00006000
0x3102D034	DMA22_XCNT_CUR	DMA22 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D038	DMA22_YCNT_CUR	DMA22 Current Row Count (2D only) Register	0x00000000
0x3102D040	DMA22_BWLCNT	DMA22 Bandwidth Limit Count Register	0x00000000
0x3102D044	DMA22_BWLCNT_CUR	DMA22 Bandwidth Limit Count Current Register	0x00000000

Table A-43: ADSP-2159x DMA22 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D048	DMA22_BWMCNT	DMA22 Bandwidth Monitor Count Register	0x00000000
0x3102D04C	DMA22_BWMCNT_CUR	DMA22 Bandwidth Monitor Count Current Register	0x00000000

Table A-44: ADSP-2159x DMA23 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D080	DMA23_DSCPTR_NXT	DMA23 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D084	DMA23_ADDRSTART	DMA23 Start Address of Current Buffer Register	0x00000000
0x3102D088	DMA23_CFG	DMA23 Configuration Register	0x00000000
0x3102D08C	DMA23_XCNT	DMA23 Inner Loop Count Start Value Register	0x00000000
0x3102D090	DMA23_XMOD	DMA23 Inner Loop Address Increment Register	0x00000000
0x3102D094	DMA23_YCNT	DMA23 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D098	DMA23_YMOD	DMA23 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D0A4	DMA23_DSCPTR_CUR	DMA23 Current Descriptor Pointer Register	0x00000000
0x3102D0A8	DMA23_DSCPTR_PRV	DMA23 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D0AC	DMA23_ADDR_CUR	DMA23 Current Address Register	0x00000000
0x3102D0B0	DMA23_STAT	DMA23 Status Register	0x00006000
0x3102D0B4	DMA23_XCNT_CUR	DMA23 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D0B8	DMA23_YCNT_CUR	DMA23 Current Row Count (2D only) Register	0x00000000
0x3102D0C0	DMA23_BWLCNT	DMA23 Bandwidth Limit Count Register	0x00000000
0x3102D0C4	DMA23_BWLCNT_CUR	DMA23 Bandwidth Limit Count Current Register	0x00000000
0x3102D0C8	DMA23_BWMCNT	DMA23 Bandwidth Monitor Count Register	0x00000000
0x3102D0CC	DMA23_BWMCNT_CUR	DMA23 Bandwidth Monitor Count Current Register	0x00000000

Table A-45: ADSP-2159x DMA24 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D100	DMA24_DSCPTR_NXT	DMA24 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D104	DMA24_ADDRSTART	DMA24 Start Address of Current Buffer Register	0x00000000
0x3102D108	DMA24_CFG	DMA24 Configuration Register	0x00000000

Table A-45: ADSP-2159x DMA24 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102D10C	DMA24_XCNT	DMA24 Inner Loop Count Start Value Register	0x00000000
0x3102D110	DMA24_XMOD	DMA24 Inner Loop Address Increment Register	0x00000000
0x3102D114	DMA24_YCNT	DMA24 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D118	DMA24_YMOD	DMA24 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D124	DMA24_DSCPTR_CUR	DMA24 Current Descriptor Pointer Register	0x00000000
0x3102D128	DMA24_DSCPTR_PRV	DMA24 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D12C	DMA24_ADDR_CUR	DMA24 Current Address Register	0x00000000
0x3102D130	DMA24_STAT	DMA24 Status Register	0x00006000
0x3102D134	DMA24_XCNT_CUR	DMA24 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D138	DMA24_YCNT_CUR	DMA24 Current Row Count (2D only) Register	0x00000000
0x3102D140	DMA24_BWLCNT	DMA24 Bandwidth Limit Count Register	0x00000000
0x3102D144	DMA24_BWLCNT_CUR	DMA24 Bandwidth Limit Count Current Register	0x00000000
0x3102D148	DMA24_BWMCNT	DMA24 Bandwidth Monitor Count Register	0x00000000
0x3102D14C	DMA24_BWMCNT_CUR	DMA24 Bandwidth Monitor Count Current Register	0x00000000

Table A-46: ADSP-2159x DMA25 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102D180	DMA25_DSCPTR_NXT	DMA25 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D184	DMA25_ADDRSTART	DMA25 Start Address of Current Buffer Register	0x00000000
0x3102D188	DMA25_CFG	DMA25 Configuration Register	0x00000000
0x3102D18C	DMA25_XCNT	DMA25 Inner Loop Count Start Value Register	0x00000000
0x3102D190	DMA25_XMOD	DMA25 Inner Loop Address Increment Register	0x00000000
0x3102D194	DMA25_YCNT	DMA25 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D198	DMA25_YMOD	DMA25 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D1A4	DMA25_DSCPTR_CUR	DMA25 Current Descriptor Pointer Register	0x00000000
0x3102D1A8	DMA25_DSCPTR_PRV	DMA25 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D1AC	DMA25_ADDR_CUR	DMA25 Current Address Register	0x00000000

Table A-46: ADSP-2159x DMA25 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D1B0	DMA25_STAT	DMA25 Status Register	0x00006000
0x3102D1B4	DMA25_XCNT_CUR	DMA25 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D1B8	DMA25_YCNT_CUR	DMA25 Current Row Count (2D only) Register	0x00000000
0x3102D1C0	DMA25_BWLCNT	DMA25 Bandwidth Limit Count Register	0x00000000
0x3102D1C4	DMA25_BWLCNT_CUR	DMA25 Bandwidth Limit Count Current Register	0x00000000
0x3102D1C8	DMA25_BWMCNT	DMA25 Bandwidth Monitor Count Register	0x00000000
0x3102D1CC	DMA25_BWMCNT_CUR	DMA25 Bandwidth Monitor Count Current Register	0x00000000

Table A-47: ADSP-2159x DMA26 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D200	DMA26_DSCPTR_NXT	DMA26 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D204	DMA26_ADDRSTART	DMA26 Start Address of Current Buffer Register	0x00000000
0x3102D208	DMA26_CFG	DMA26 Configuration Register	0x00000000
0x3102D20C	DMA26_XCNT	DMA26 Inner Loop Count Start Value Register	0x00000000
0x3102D210	DMA26_XMOD	DMA26 Inner Loop Address Increment Register	0x00000000
0x3102D214	DMA26_YCNT	DMA26 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D218	DMA26_YMOD	DMA26 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D224	DMA26_DSCPTR_CUR	DMA26 Current Descriptor Pointer Register	0x00000000
0x3102D228	DMA26_DSCPTR_PRV	DMA26 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D22C	DMA26_ADDR_CUR	DMA26 Current Address Register	0x00000000
0x3102D230	DMA26_STAT	DMA26 Status Register	0x00006000
0x3102D234	DMA26_XCNT_CUR	DMA26 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D238	DMA26_YCNT_CUR	DMA26 Current Row Count (2D only) Register	0x00000000
0x3102D240	DMA26_BWLCNT	DMA26 Bandwidth Limit Count Register	0x00000000
0x3102D244	DMA26_BWLCNT_CUR	DMA26 Bandwidth Limit Count Current Register	0x00000000
0x3102D248	DMA26_BWMCNT	DMA26 Bandwidth Monitor Count Register	0x00000000
0x3102D24C	DMA26_BWMCNT_CUR	DMA26 Bandwidth Monitor Count Current Register	0x00000000

Table A-48: ADSP-2159x DMA27 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102D280	DMA27_DSCPTR_NXT	DMA27 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D284	DMA27_ADDRSTART	DMA27 Start Address of Current Buffer Register	0x00000000
0x3102D288	DMA27_CFG	DMA27 Configuration Register	0x00000000
0x3102D28C	DMA27_XCNT	DMA27 Inner Loop Count Start Value Register	0x00000000
0x3102D290	DMA27_XMOD	DMA27 Inner Loop Address Increment Register	0x00000000
0x3102D294	DMA27_YCNT	DMA27 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D298	DMA27_YMOD	DMA27 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D2A4	DMA27_DSCPTR_CUR	DMA27 Current Descriptor Pointer Register	0x00000000
0x3102D2A8	DMA27_DSCPTR_PRV	DMA27 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D2AC	DMA27_ADDR_CUR	DMA27 Current Address Register	0x00000000
0x3102D2B0	DMA27_STAT	DMA27 Status Register	0x00006000
0x3102D2B4	DMA27_XCNT_CUR	DMA27 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D2B8	DMA27_YCNT_CUR	DMA27 Current Row Count (2D only) Register	0x00000000
0x3102D2C0	DMA27_BWLCNT	DMA27 Bandwidth Limit Count Register	0x00000000
0x3102D2C4	DMA27_BWLCNT_CUR	DMA27 Bandwidth Limit Count Current Register	0x00000000
0x3102D2C8	DMA27_BWMCNT	DMA27 Bandwidth Monitor Count Register	0x00000000
0x3102D2CC	DMA27_BWMCNT_CUR	DMA27 Bandwidth Monitor Count Current Register	0x00000000

Table A-49: ADSP-2159x DMA28 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026400	DMA28_DSCPTR_NXT	DMA28 Pointer to Next Initial Descriptor Register	0x00000000
0x31026404	DMA28_ADDRSTART	DMA28 Start Address of Current Buffer Register	0x00000000
0x31026408	DMA28_CFG	DMA28 Configuration Register	0x00000000
0x3102640C	DMA28_XCNT	DMA28 Inner Loop Count Start Value Register	0x00000000
0x31026410	DMA28_XMOD	DMA28 Inner Loop Address Increment Register	0x00000000
0x31026414	DMA28_YCNT	DMA28 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026418	DMA28_YMOD	DMA28 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-49: ADSP-2159x DMA28 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31026424	DMA28_DSCPTR_CUR	DMA28 Current Descriptor Pointer Register	0x00000000
0x31026428	DMA28_DSCPTR_PRV	DMA28 Previous Initial Descriptor Pointer Register	0x00000000
0x3102642C	DMA28_ADDR_CUR	DMA28 Current Address Register	0x00000000
0x31026430	DMA28_STAT	DMA28 Status Register	0x00006000
0x31026434	DMA28_XCNT_CUR	DMA28 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31026438	DMA28_YCNT_CUR	DMA28 Current Row Count (2D only) Register	0x00000000
0x31026440	DMA28_BWLCNT	DMA28 Bandwidth Limit Count Register	0x00000000
0x31026444	DMA28_BWLCNT_CUR	DMA28 Bandwidth Limit Count Current Register	0x00000000
0x31026448	DMA28_BWMCNT	DMA28 Bandwidth Monitor Count Register	0x00000000
0x3102644C	DMA28_BWMCNT_CUR	DMA28 Bandwidth Monitor Count Current Register	0x00000000

Table A-50: ADSP-2159x DMA29 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31026480	DMA29_DSCPTR_NXT	DMA29 Pointer to Next Initial Descriptor Register	0x00000000
0x31026484	DMA29_ADDRSTART	DMA29 Start Address of Current Buffer Register	0x00000000
0x31026488	DMA29_CFG	DMA29 Configuration Register	0x00000000
0x3102648C	DMA29_XCNT	DMA29 Inner Loop Count Start Value Register	0x00000000
0x31026490	DMA29_XMOD	DMA29 Inner Loop Address Increment Register	0x00000000
0x31026494	DMA29_YCNT	DMA29 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026498	DMA29_YMOD	DMA29 Outer Loop Address Increment (2D only) Register	0x00000000
0x310264A4	DMA29_DSCPTR_CUR	DMA29 Current Descriptor Pointer Register	0x00000000
0x310264A8	DMA29_DSCPTR_PRV	DMA29 Previous Initial Descriptor Pointer Register	0x00000000
0x310264AC	DMA29_ADDR_CUR	DMA29 Current Address Register	0x00000000
0x310264B0	DMA29_STAT	DMA29 Status Register	0x00006000
0x310264B4	DMA29_XCNT_CUR	DMA29 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310264B8	DMA29_YCNT_CUR	DMA29 Current Row Count (2D only) Register	0x00000000
0x310264C0	DMA29_BWLCNT	DMA29 Bandwidth Limit Count Register	0x00000000
0x310264C4	DMA29_BWLCNT_CUR	DMA29 Bandwidth Limit Count Current Register	0x00000000

Table A-50: ADSP-2159x DMA29 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310264C8	DMA29_BWMCNT	DMA29 Bandwidth Monitor Count Register	0x00000000
0x310264CC	DMA29_BWMCNT_CUR	DMA29 Bandwidth Monitor Count Current Register	0x00000000

Table A-51: ADSP-2159x DMA3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022180	DMA3_DSCPTR_NXT	DMA3 Pointer to Next Initial Descriptor Register	0x00000000
0x31022184	DMA3_ADDRSTART	DMA3 Start Address of Current Buffer Register	0x00000000
0x31022188	DMA3_CFG	DMA3 Configuration Register	0x00000000
0x3102218C	DMA3_XCNT	DMA3 Inner Loop Count Start Value Register	0x00000000
0x31022190	DMA3_XMOD	DMA3 Inner Loop Address Increment Register	0x00000000
0x31022194	DMA3_YCNT	DMA3 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022198	DMA3_YMOD	DMA3 Outer Loop Address Increment (2D only) Register	0x00000000
0x310221A4	DMA3_DSCPTR_CUR	DMA3 Current Descriptor Pointer Register	0x00000000
0x310221A8	DMA3_DSCPTR_PRV	DMA3 Previous Initial Descriptor Pointer Register	0x00000000
0x310221AC	DMA3_ADDR_CUR	DMA3 Current Address Register	0x00000000
0x310221B0	DMA3_STAT	DMA3 Status Register	0x00006000
0x310221B4	DMA3_XCNT_CUR	DMA3 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310221B8	DMA3_YCNT_CUR	DMA3 Current Row Count (2D only) Register	0x00000000
0x310221C0	DMA3_BWLCNT	DMA3 Bandwidth Limit Count Register	0x00000000
0x310221C4	DMA3_BWLCNT_CUR	DMA3 Bandwidth Limit Count Current Register	0x00000000
0x310221C8	DMA3_BWMCNT	DMA3 Bandwidth Monitor Count Register	0x00000000
0x310221CC	DMA3_BWMCNT_CUR	DMA3 Bandwidth Monitor Count Current Register	0x00000000

Table A-52: ADSP-2159x DMA30 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x30FFF000	DMA30_DSCPTR_NXT	DMA30 Pointer to Next Initial Descriptor Register	0x00000000
0x30FFF004	DMA30_ADDRSTART	DMA30 Start Address of Current Buffer Register	0x00000000
0x30FFF008	DMA30_CFG	DMA30 Configuration Register	0x00000000
0x30FFF00C	DMA30_XCNT	DMA30 Inner Loop Count Start Value Register	0x00000000

Table A-52: ADSP-2159x DMA30 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x30FFF010	DMA30_XMOD	DMA30 Inner Loop Address Increment Register	0x00000000
0x30FFF014	DMA30_YCNT	DMA30 Outer Loop Count Start Value (2D only) Register	0x00000000
0x30FFF018	DMA30_YMOD	DMA30 Outer Loop Address Increment (2D only) Register	0x00000000
0x30FFF024	DMA30_DSCPTR_CUR	DMA30 Current Descriptor Pointer Register	0x00000000
0x30FFF028	DMA30_DSCPTR_PRV	DMA30 Previous Initial Descriptor Pointer Register	0x00000000
0x30FFF02C	DMA30_ADDR_CUR	DMA30 Current Address Register	0x00000000
0x30FFF030	DMA30_STAT	DMA30 Status Register	0x00006000
0x30FFF034	DMA30_XCNT_CUR	DMA30 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x30FFF038	DMA30_YCNT_CUR	DMA30 Current Row Count (2D only) Register	0x00000000
0x30FFF040	DMA30_BWLCNT	DMA30 Bandwidth Limit Count Register	0x00000000
0x30FFF044	DMA30_BWLCNT_CUR	DMA30 Bandwidth Limit Count Current Register	0x00000000
0x30FFF048	DMA30_BWMCNT	DMA30 Bandwidth Monitor Count Register	0x00000000
0x30FFF04C	DMA30_BWMCNT_CUR	DMA30 Bandwidth Monitor Count Current Register	0x00000000

Table A-53: ADSP-2159x DMA34 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026180	DMA34_DSCPTR_NXT	DMA34 Pointer to Next Initial Descriptor Register	0x00000000
0x31026184	DMA34_ADDRSTART	DMA34 Start Address of Current Buffer Register	0x00000000
0x31026188	DMA34_CFG	DMA34 Configuration Register	0x00000000
0x3102618C	DMA34_XCNT	DMA34 Inner Loop Count Start Value Register	0x00000000
0x31026190	DMA34_XMOD	DMA34 Inner Loop Address Increment Register	0x00000000
0x31026194	DMA34_YCNT	DMA34 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026198	DMA34_YMOD	DMA34 Outer Loop Address Increment (2D only) Register	0x00000000
0x310261A4	DMA34_DSCPTR_CUR	DMA34 Current Descriptor Pointer Register	0x00000000
0x310261A8	DMA34_DSCPTR_PRV	DMA34 Previous Initial Descriptor Pointer Register	0x00000000
0x310261AC	DMA34_ADDR_CUR	DMA34 Current Address Register	0x00000000
0x310261B0	DMA34_STAT	DMA34 Status Register	0x00006000

Table A-53: ADSP-2159x DMA34 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310261B4	DMA34_XCNT_CUR	DMA34 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310261B8	DMA34_YCNT_CUR	DMA34 Current Row Count (2D only) Register	0x00000000
0x310261C0	DMA34_BWLCNT	DMA34 Bandwidth Limit Count Register	0x00000000
0x310261C4	DMA34_BWLCNT_CUR	DMA34 Bandwidth Limit Count Current Register	0x00000000
0x310261C8	DMA34_BWMCNT	DMA34 Bandwidth Monitor Count Register	0x00000000
0x310261CC	DMA34_BWMCNT_CUR	DMA34 Bandwidth Monitor Count Current Register	0x00000000

Table A-54: ADSP-2159x DMA35 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31026100	DMA35_DSCPTR_NXT	DMA35 Pointer to Next Initial Descriptor Register	0x00000000
0x31026104	DMA35_ADDRSTART	DMA35 Start Address of Current Buffer Register	0x00000000
0x31026108	DMA35_CFG	DMA35 Configuration Register	0x00000000
0x3102610C	DMA35_XCNT	DMA35 Inner Loop Count Start Value Register	0x00000000
0x31026110	DMA35_XMOD	DMA35 Inner Loop Address Increment Register	0x00000000
0x31026114	DMA35_YCNT	DMA35 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026118	DMA35_YMOD	DMA35 Outer Loop Address Increment (2D only) Register	0x00000000
0x31026124	DMA35_DSCPTR_CUR	DMA35 Current Descriptor Pointer Register	0x00000000
0x31026128	DMA35_DSCPTR_PRV	DMA35 Previous Initial Descriptor Pointer Register	0x00000000
0x3102612C	DMA35_ADDR_CUR	DMA35 Current Address Register	0x00000000
0x31026130	DMA35_STAT	DMA35 Status Register	0x00006000
0x31026134	DMA35_XCNT_CUR	DMA35 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31026138	DMA35_YCNT_CUR	DMA35 Current Row Count (2D only) Register	0x00000000
0x31026140	DMA35_BWLCNT	DMA35 Bandwidth Limit Count Register	0x00000000
0x31026144	DMA35_BWLCNT_CUR	DMA35 Bandwidth Limit Count Current Register	0x00000000
0x31026148	DMA35_BWMCNT	DMA35 Bandwidth Monitor Count Register	0x00000000
0x3102614C	DMA35_BWMCNT_CUR	DMA35 Bandwidth Monitor Count Current Register	0x00000000

Table A-55: ADSP-2159x DMA36 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x30FFF080	DMA36_DSCPTR_NXT	DMA36 Pointer to Next Initial Descriptor Register	0x00000000
0x30FFF084	DMA36_ADDRSTART	DMA36 Start Address of Current Buffer Register	0x00000000
0x30FFF088	DMA36_CFG	DMA36 Configuration Register	0x00000000
0x30FFF08C	DMA36_XCNT	DMA36 Inner Loop Count Start Value Register	0x00000000
0x30FFF090	DMA36_XMOD	DMA36 Inner Loop Address Increment Register	0x00000000
0x30FFF094	DMA36_YCNT	DMA36 Outer Loop Count Start Value (2D only) Register	0x00000000
0x30FFF098	DMA36_YMOD	DMA36 Outer Loop Address Increment (2D only) Register	0x00000000
0x30FFF0A4	DMA36_DSCPTR_CUR	DMA36 Current Descriptor Pointer Register	0x00000000
0x30FFF0A8	DMA36_DSCPTR_PRV	DMA36 Previous Initial Descriptor Pointer Register	0x00000000
0x30FFF0AC	DMA36_ADDR_CUR	DMA36 Current Address Register	0x00000000
0x30FFF0B0	DMA36_STAT	DMA36 Status Register	0x00006000
0x30FFF0B4	DMA36_XCNT_CUR	DMA36 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x30FFF0B8	DMA36_YCNT_CUR	DMA36 Current Row Count (2D only) Register	0x00000000
0x30FFF0C0	DMA36_BWLCNT	DMA36 Bandwidth Limit Count Register	0x00000000
0x30FFF0C4	DMA36_BWLCNT_CUR	DMA36 Bandwidth Limit Count Current Register	0x00000000
0x30FFF0C8	DMA36_BWMCNT	DMA36 Bandwidth Monitor Count Register	0x00000000
0x30FFF0CC	DMA36_BWMCNT_CUR	DMA36 Bandwidth Monitor Count Current Register	0x00000000

Table A-56: ADSP-2159x DMA37 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026280	DMA37_DSCPTR_NXT	DMA37 Pointer to Next Initial Descriptor Register	0x00000000
0x31026284	DMA37_ADDRSTART	DMA37 Start Address of Current Buffer Register	0x00000000
0x31026288	DMA37_CFG	DMA37 Configuration Register	0x00000000
0x3102628C	DMA37_XCNT	DMA37 Inner Loop Count Start Value Register	0x00000000
0x31026290	DMA37_XMOD	DMA37 Inner Loop Address Increment Register	0x00000000
0x31026294	DMA37_YCNT	DMA37 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026298	DMA37_YMOD	DMA37 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-56: ADSP-2159x DMA37 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310262A4	DMA37_DSCPTR_CUR	DMA37 Current Descriptor Pointer Register	0x00000000
0x310262A8	DMA37_DSCPTR_PRV	DMA37 Previous Initial Descriptor Pointer Register	0x00000000
0x310262AC	DMA37_ADDR_CUR	DMA37 Current Address Register	0x00000000
0x310262B0	DMA37_STAT	DMA37 Status Register	0x00006000
0x310262B4	DMA37_XCNT_CUR	DMA37 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310262B8	DMA37_YCNT_CUR	DMA37 Current Row Count (2D only) Register	0x00000000
0x310262C0	DMA37_BWLCNT	DMA37 Bandwidth Limit Count Register	0x00000000
0x310262C4	DMA37_BWLCNT_CUR	DMA37 Bandwidth Limit Count Current Register	0x00000000
0x310262C8	DMA37_BWMCNT	DMA37 Bandwidth Monitor Count Register	0x00000000
0x310262CC	DMA37_BWMCNT_CUR	DMA37 Bandwidth Monitor Count Current Register	0x00000000

Table A-57: ADSP-2159x DMA38 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31026200	DMA38_DSCPTR_NXT	DMA38 Pointer to Next Initial Descriptor Register	0x00000000
0x31026204	DMA38_ADDRSTART	DMA38 Start Address of Current Buffer Register	0x00000000
0x31026208	DMA38_CFG	DMA38 Configuration Register	0x00000000
0x3102620C	DMA38_XCNT	DMA38 Inner Loop Count Start Value Register	0x00000000
0x31026210	DMA38_XMOD	DMA38 Inner Loop Address Increment Register	0x00000000
0x31026214	DMA38_YCNT	DMA38 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026218	DMA38_YMOD	DMA38 Outer Loop Address Increment (2D only) Register	0x00000000
0x31026224	DMA38_DSCPTR_CUR	DMA38 Current Descriptor Pointer Register	0x00000000
0x31026228	DMA38_DSCPTR_PRV	DMA38 Previous Initial Descriptor Pointer Register	0x00000000
0x3102622C	DMA38_ADDR_CUR	DMA38 Current Address Register	0x00000000
0x31026230	DMA38_STAT	DMA38 Status Register	0x00006000
0x31026234	DMA38_XCNT_CUR	DMA38 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31026238	DMA38_YCNT_CUR	DMA38 Current Row Count (2D only) Register	0x00000000
0x31026240	DMA38_BWLCNT	DMA38 Bandwidth Limit Count Register	0x00000000
0x31026244	DMA38_BWLCNT_CUR	DMA38 Bandwidth Limit Count Current Register	0x00000000

Table A-57: ADSP-2159x DMA38 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31026248	DMA38_BWMCNT	DMA38 Bandwidth Monitor Count Register	0x00000000
0x3102624C	DMA38_BWMCNT_CUR	DMA38 Bandwidth Monitor Count Current Register	0x00000000

Table A-58: ADSP-2159x DMA39 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109A000	DMA39_DSCPTR_NXT	DMA39 Pointer to Next Initial Descriptor Register	0x00000000
0x3109A004	DMA39_ADDRSTART	DMA39 Start Address of Current Buffer Register	0x00000000
0x3109A008	DMA39_CFG	DMA39 Configuration Register	0x00000220
0x3109A00C	DMA39_XCNT	DMA39 Inner Loop Count Start Value Register	0x00000000
0x3109A010	DMA39_XMOD	DMA39 Inner Loop Address Increment Register	0x00000000
0x3109A014	DMA39_YCNT	DMA39 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109A018	DMA39_YMOD	DMA39 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109A024	DMA39_DSCPTR_CUR	DMA39 Current Descriptor Pointer Register	0x00000000
0x3109A028	DMA39_DSCPTR_PRV	DMA39 Previous Initial Descriptor Pointer Register	0x00000000
0x3109A02C	DMA39_ADDR_CUR	DMA39 Current Address Register	0x00000000
0x3109A030	DMA39_STAT	DMA39 Status Register	0x00006000
0x3109A034	DMA39_XCNT_CUR	DMA39 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109A038	DMA39_YCNT_CUR	DMA39 Current Row Count (2D only) Register	0x00000000
0x3109A040	DMA39_BWLCNT	DMA39 Bandwidth Limit Count Register	0x00000000
0x3109A044	DMA39_BWLCNT_CUR	DMA39 Bandwidth Limit Count Current Register	0x00000000
0x3109A048	DMA39_BWMCNT	DMA39 Bandwidth Monitor Count Register	0x00000000
0x3109A04C	DMA39_BWMCNT_CUR	DMA39 Bandwidth Monitor Count Current Register	0x00000000

Table A-59: ADSP-2159x DMA4 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31022200	DMA4_DSCPTR_NXT	DMA4 Pointer to Next Initial Descriptor Register	0x00000000
0x31022204	DMA4_ADDRSTART	DMA4 Start Address of Current Buffer Register	0x00000000
0x31022208	DMA4_CFG	DMA4 Configuration Register	0x00000000

Table A-59: ADSP-2159x DMA4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102220C	DMA4_XCNT	DMA4 Inner Loop Count Start Value Register	0x00000000
0x31022210	DMA4_XMOD	DMA4 Inner Loop Address Increment Register	0x00000000
0x31022214	DMA4_YCNT	DMA4 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022218	DMA4_YMOD	DMA4 Outer Loop Address Increment (2D only) Register	0x00000000
0x31022224	DMA4_DSCPTR_CUR	DMA4 Current Descriptor Pointer Register	0x00000000
0x31022228	DMA4_DSCPTR_PRV	DMA4 Previous Initial Descriptor Pointer Register	0x00000000
0x3102222C	DMA4_ADDR_CUR	DMA4 Current Address Register	0x00000000
0x31022230	DMA4_STAT	DMA4 Status Register	0x00006000
0x31022234	DMA4_XCNT_CUR	DMA4 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31022238	DMA4_YCNT_CUR	DMA4 Current Row Count (2D only) Register	0x00000000
0x31022240	DMA4_BWLCNT	DMA4 Bandwidth Limit Count Register	0x00000000
0x31022244	DMA4_BWLCNT_CUR	DMA4 Bandwidth Limit Count Current Register	0x00000000
0x31022248	DMA4_BWMCNT	DMA4 Bandwidth Monitor Count Register	0x00000000
0x3102224C	DMA4_BWMCNT_CUR	DMA4 Bandwidth Monitor Count Current Register	0x00000000

Table A-60: ADSP-2159x DMA40 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109A080	DMA40_DSCPTR_NXT	DMA40 Pointer to Next Initial Descriptor Register	0x00000000
0x3109A084	DMA40_ADDRSTART	DMA40 Start Address of Current Buffer Register	0x00000000
0x3109A088	DMA40_CFG	DMA40 Configuration Register	0x00000222
0x3109A08C	DMA40_XCNT	DMA40 Inner Loop Count Start Value Register	0x00000000
0x3109A090	DMA40_XMOD	DMA40 Inner Loop Address Increment Register	0x00000000
0x3109A094	DMA40_YCNT	DMA40 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109A098	DMA40_YMOD	DMA40 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109A0A4	DMA40_DSCPTR_CUR	DMA40 Current Descriptor Pointer Register	0x00000000
0x3109A0A8	DMA40_DSCPTR_PRV	DMA40 Previous Initial Descriptor Pointer Register	0x00000000
0x3109A0AC	DMA40_ADDR_CUR	DMA40 Current Address Register	0x00000000
0x3109A0B0	DMA40_STAT	DMA40 Status Register	0x00006000

Table A-60: ADSP-2159x DMA40 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109A0B4	DMA40_XCNT_CUR	DMA40 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109A0B8	DMA40_YCNT_CUR	DMA40 Current Row Count (2D only) Register	0x00000000
0x3109A0C0	DMA40_BWLCNT	DMA40 Bandwidth Limit Count Register	0x00000000
0x3109A0C4	DMA40_BWLCNT_CUR	DMA40 Bandwidth Limit Count Current Register	0x00000000
0x3109A0C8	DMA40_BWMCNT	DMA40 Bandwidth Monitor Count Register	0x00000000
0x3109A0CC	DMA40_BWMCNT_CUR	DMA40 Bandwidth Monitor Count Current Register	0x00000000

Table A-61: ADSP-2159x DMA43 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109B000	DMA43_DSCPTR_NXT	DMA43 Pointer to Next Initial Descriptor Register	0x00000000
0x3109B004	DMA43_ADDRSTART	DMA43 Start Address of Current Buffer Register	0x00000000
0x3109B008	DMA43_CFG	DMA43 Configuration Register	0x00000220
0x3109B00C	DMA43_XCNT	DMA43 Inner Loop Count Start Value Register	0x00000000
0x3109B010	DMA43_XMOD	DMA43 Inner Loop Address Increment Register	0x00000000
0x3109B014	DMA43_YCNT	DMA43 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109B018	DMA43_YMOD	DMA43 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109B024	DMA43_DSCPTR_CUR	DMA43 Current Descriptor Pointer Register	0x00000000
0x3109B028	DMA43_DSCPTR_PRV	DMA43 Previous Initial Descriptor Pointer Register	0x00000000
0x3109B02C	DMA43_ADDR_CUR	DMA43 Current Address Register	0x00000000
0x3109B030	DMA43_STAT	DMA43 Status Register	0x0000B000
0x3109B034	DMA43_XCNT_CUR	DMA43 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109B038	DMA43_YCNT_CUR	DMA43 Current Row Count (2D only) Register	0x00000000
0x3109B040	DMA43_BWLCNT	DMA43 Bandwidth Limit Count Register	0x00000000
0x3109B044	DMA43_BWLCNT_CUR	DMA43 Bandwidth Limit Count Current Register	0x00000000
0x3109B048	DMA43_BWMCNT	DMA43 Bandwidth Monitor Count Register	0x00000000
0x3109B04C	DMA43_BWMCNT_CUR	DMA43 Bandwidth Monitor Count Current Register	0x00000000

Table A-62: ADSP-2159x DMA44 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109B080	DMA44_DSCPTR_NXT	DMA44 Pointer to Next Initial Descriptor Register	0x00000000
0x3109B084	DMA44_ADDRSTART	DMA44 Start Address of Current Buffer Register	0x00000000
0x3109B088	DMA44_CFG	DMA44 Configuration Register	0x00000222
0x3109B08C	DMA44_XCNT	DMA44 Inner Loop Count Start Value Register	0x00000000
0x3109B090	DMA44_XMOD	DMA44 Inner Loop Address Increment Register	0x00000000
0x3109B094	DMA44_YCNT	DMA44 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109B098	DMA44_YMOD	DMA44 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109B0A4	DMA44_DSCPTR_CUR	DMA44 Current Descriptor Pointer Register	0x00000000
0x3109B0A8	DMA44_DSCPTR_PRV	DMA44 Previous Initial Descriptor Pointer Register	0x00000000
0x3109B0AC	DMA44_ADDR_CUR	DMA44 Current Address Register	0x00000000
0x3109B0B0	DMA44_STAT	DMA44 Status Register	0x0000B000
0x3109B0B4	DMA44_XCNT_CUR	DMA44 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109B0B8	DMA44_YCNT_CUR	DMA44 Current Row Count (2D only) Register	0x00000000
0x3109B0C0	DMA44_BWLCNT	DMA44 Bandwidth Limit Count Register	0x00000000
0x3109B0C4	DMA44_BWLCNT_CUR	DMA44 Bandwidth Limit Count Current Register	0x00000000
0x3109B0C8	DMA44_BWMCNT	DMA44 Bandwidth Monitor Count Register	0x00000000
0x3109B0CC	DMA44_BWMCNT_CUR	DMA44 Bandwidth Monitor Count Current Register	0x00000000

Table A-63: ADSP-2159x DMA45 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7200	DMA45_DSCPTR_NXT	DMA45 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7204	DMA45_ADDRSTART	DMA45 Start Address of Current Buffer Register	0x00000000
0x310A7208	DMA45_CFG	DMA45 Configuration Register	0x00000000
0x310A720C	DMA45_XCNT	DMA45 Inner Loop Count Start Value Register	0x00000000
0x310A7210	DMA45_XMOD	DMA45 Inner Loop Address Increment Register	0x00000000
0x310A7214	DMA45_YCNT	DMA45 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7218	DMA45_YMOD	DMA45 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-63: ADSP-2159x DMA45 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A7224	DMA45_DSCPTR_CUR	DMA45 Current Descriptor Pointer Register	0x00000000
0x310A7228	DMA45_DSCPTR_PRV	DMA45 Previous Initial Descriptor Pointer Register	0x00000000
0x310A722C	DMA45_ADDR_CUR	DMA45 Current Address Register	0x00000000
0x310A7230	DMA45_STAT	DMA45 Status Register	0x00006000
0x310A7234	DMA45_XCNT_CUR	DMA45 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A7238	DMA45_YCNT_CUR	DMA45 Current Row Count (2D only) Register	0x00000000
0x310A7240	DMA45_BWLCNT	DMA45 Bandwidth Limit Count Register	0x00000000
0x310A7244	DMA45_BWLCNT_CUR	DMA45 Bandwidth Limit Count Current Register	0x00000000
0x310A7248	DMA45_BWMCNT	DMA45 Bandwidth Monitor Count Register	0x00000000
0x310A724C	DMA45_BWMCNT_CUR	DMA45 Bandwidth Monitor Count Current Register	0x00000000

Table A-64: ADSP-2159x DMA46 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310A7280	DMA46_DSCPTR_NXT	DMA46 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7284	DMA46_ADDRSTART	DMA46 Start Address of Current Buffer Register	0x00000000
0x310A7288	DMA46_CFG	DMA46 Configuration Register	0x00000000
0x310A728C	DMA46_XCNT	DMA46 Inner Loop Count Start Value Register	0x00000000
0x310A7290	DMA46_XMOD	DMA46 Inner Loop Address Increment Register	0x00000000
0x310A7294	DMA46_YCNT	DMA46 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7298	DMA46_YMOD	DMA46 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A72A4	DMA46_DSCPTR_CUR	DMA46 Current Descriptor Pointer Register	0x00000000
0x310A72A8	DMA46_DSCPTR_PRV	DMA46 Previous Initial Descriptor Pointer Register	0x00000000
0x310A72AC	DMA46_ADDR_CUR	DMA46 Current Address Register	0x00000000
0x310A72B0	DMA46_STAT	DMA46 Status Register	0x00006000
0x310A72B4	DMA46_XCNT_CUR	DMA46 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A72B8	DMA46_YCNT_CUR	DMA46 Current Row Count (2D only) Register	0x00000000
0x310A72C0	DMA46_BWLCNT	DMA46 Bandwidth Limit Count Register	0x00000000
0x310A72C4	DMA46_BWLCNT_CUR	DMA46 Bandwidth Limit Count Current Register	0x00000000

Table A-64: ADSP-2159x DMA46 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A72C8	DMA46_BWMCNT	DMA46 Bandwidth Monitor Count Register	0x00000000
0x310A72CC	DMA46_BWMCNT_CUR	DMA46 Bandwidth Monitor Count Current Register	0x00000000

Table A-65: ADSP-2159x DMA47 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7300	DMA47_DSCPTR_NXT	DMA47 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7304	DMA47_ADDRSTART	DMA47 Start Address of Current Buffer Register	0x00000000
0x310A7308	DMA47_CFG	DMA47 Configuration Register	0x00000000
0x310A730C	DMA47_XCNT	DMA47 Inner Loop Count Start Value Register	0x00000000
0x310A7310	DMA47_XMOD	DMA47 Inner Loop Address Increment Register	0x00000000
0x310A7314	DMA47_YCNT	DMA47 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7318	DMA47_YMOD	DMA47 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A7324	DMA47_DSCPTR_CUR	DMA47 Current Descriptor Pointer Register	0x00000000
0x310A7328	DMA47_DSCPTR_PRV	DMA47 Previous Initial Descriptor Pointer Register	0x00000000
0x310A732C	DMA47_ADDR_CUR	DMA47 Current Address Register	0x00000000
0x310A7330	DMA47_STAT	DMA47 Status Register	0x00006000
0x310A7334	DMA47_XCNT_CUR	DMA47 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A7338	DMA47_YCNT_CUR	DMA47 Current Row Count (2D only) Register	0x00000000
0x310A7340	DMA47_BWLCNT	DMA47 Bandwidth Limit Count Register	0x00000000
0x310A7344	DMA47_BWLCNT_CUR	DMA47 Bandwidth Limit Count Current Register	0x00000000
0x310A7348	DMA47_BWMCNT	DMA47 Bandwidth Monitor Count Register	0x00000000
0x310A734C	DMA47_BWMCNT_CUR	DMA47 Bandwidth Monitor Count Current Register	0x00000000

Table A-66: ADSP-2159x DMA48 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7380	DMA48_DSCPTR_NXT	DMA48 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7384	DMA48_ADDRSTART	DMA48 Start Address of Current Buffer Register	0x00000000
0x310A7388	DMA48_CFG	DMA48 Configuration Register	0x00000000

Table A-66: ADSP-2159x DMA48 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A738C	DMA48_XCNT	DMA48 Inner Loop Count Start Value Register	0x00000000
0x310A7390	DMA48_XMOD	DMA48 Inner Loop Address Increment Register	0x00000000
0x310A7394	DMA48_YCNT	DMA48 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7398	DMA48_YMOD	DMA48 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A73A4	DMA48_DSCPTR_CUR	DMA48 Current Descriptor Pointer Register	0x00000000
0x310A73A8	DMA48_DSCPTR_PRV	DMA48 Previous Initial Descriptor Pointer Register	0x00000000
0x310A73AC	DMA48_ADDR_CUR	DMA48 Current Address Register	0x00000000
0x310A73B0	DMA48_STAT	DMA48 Status Register	0x00006000
0x310A73B4	DMA48_XCNT_CUR	DMA48 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A73B8	DMA48_YCNT_CUR	DMA48 Current Row Count (2D only) Register	0x00000000
0x310A73C0	DMA48_BWLCNT	DMA48 Bandwidth Limit Count Register	0x00000000
0x310A73C4	DMA48_BWLCNT_CUR	DMA48 Bandwidth Limit Count Current Register	0x00000000
0x310A73C8	DMA48_BWMCNT	DMA48 Bandwidth Monitor Count Register	0x00000000
0x310A73CC	DMA48_BWMCNT_CUR	DMA48 Bandwidth Monitor Count Current Register	0x00000000

Table A-67: ADSP-2159x DMA49 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310AC000	DMA49_DSCPTR_NXT	DMA49 Pointer to Next Initial Descriptor Register	0x00000000
0x310AC004	DMA49_ADDRSTART	DMA49 Start Address of Current Buffer Register	0x00000000
0x310AC008	DMA49_CFG	DMA49 Configuration Register	0x00000220
0x310AC00C	DMA49_XCNT	DMA49 Inner Loop Count Start Value Register	0x00000000
0x310AC010	DMA49_XMOD	DMA49 Inner Loop Address Increment Register	0x00000000
0x310AC014	DMA49_YCNT	DMA49 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310AC018	DMA49_YMOD	DMA49 Outer Loop Address Increment (2D only) Register	0x00000000
0x310AC024	DMA49_DSCPTR_CUR	DMA49 Current Descriptor Pointer Register	0x00000000
0x310AC028	DMA49_DSCPTR_PRV	DMA49 Previous Initial Descriptor Pointer Register	0x00000000
0x310AC02C	DMA49_ADDR_CUR	DMA49 Current Address Register	0x00000000

Table A-67: ADSP-2159x DMA49 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310AC030	DMA49_STAT	DMA49 Status Register	0x00006000
0x310AC034	DMA49_XCNT_CUR	DMA49 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310AC038	DMA49_YCNT_CUR	DMA49 Current Row Count (2D only) Register	0x00000000
0x310AC040	DMA49_BWLCNT	DMA49 Bandwidth Limit Count Register	0x00000000
0x310AC044	DMA49_BWLCNT_CUR	DMA49 Bandwidth Limit Count Current Register	0x00000000
0x310AC048	DMA49_BWMCNT	DMA49 Bandwidth Monitor Count Register	0x00000000
0x310AC04C	DMA49_BWMCNT_CUR	DMA49 Bandwidth Monitor Count Current Register	0x00000000

Table A-68: ADSP-2159x DMA5 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022280	DMA5_DSCPTR_NXT	DMA5 Pointer to Next Initial Descriptor Register	0x00000000
0x31022284	DMA5_ADDRSTART	DMA5 Start Address of Current Buffer Register	0x00000000
0x31022288	DMA5_CFG	DMA5 Configuration Register	0x00000000
0x3102228C	DMA5_XCNT	DMA5 Inner Loop Count Start Value Register	0x00000000
0x31022290	DMA5_XMOD	DMA5 Inner Loop Address Increment Register	0x00000000
0x31022294	DMA5_YCNT	DMA5 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022298	DMA5_YMOD	DMA5 Outer Loop Address Increment (2D only) Register	0x00000000
0x310222A4	DMA5_DSCPTR_CUR	DMA5 Current Descriptor Pointer Register	0x00000000
0x310222A8	DMA5_DSCPTR_PRV	DMA5 Previous Initial Descriptor Pointer Register	0x00000000
0x310222AC	DMA5_ADDR_CUR	DMA5 Current Address Register	0x00000000
0x310222B0	DMA5_STAT	DMA5 Status Register	0x00006000
0x310222B4	DMA5_XCNT_CUR	DMA5 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310222B8	DMA5_YCNT_CUR	DMA5 Current Row Count (2D only) Register	0x00000000
0x310222C0	DMA5_BWLCNT	DMA5 Bandwidth Limit Count Register	0x00000000
0x310222C4	DMA5_BWLCNT_CUR	DMA5 Bandwidth Limit Count Current Register	0x00000000
0x310222C8	DMA5_BWMCNT	DMA5 Bandwidth Monitor Count Register	0x00000000
0x310222CC	DMA5_BWMCNT_CUR	DMA5 Bandwidth Monitor Count Current Register	0x00000000

Table A-69: ADSP-2159x DMA50 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310AC080	DMA50_DSCPTR_NXT	DMA50 Pointer to Next Initial Descriptor Register	0x00000000
0x310AC084	DMA50_ADDRSTART	DMA50 Start Address of Current Buffer Register	0x00000000
0x310AC088	DMA50_CFG	DMA50 Configuration Register	0x00000222
0x310AC08C	DMA50_XCNT	DMA50 Inner Loop Count Start Value Register	0x00000000
0x310AC090	DMA50_XMOD	DMA50 Inner Loop Address Increment Register	0x00000000
0x310AC094	DMA50_YCNT	DMA50 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310AC098	DMA50_YMOD	DMA50 Outer Loop Address Increment (2D only) Register	0x00000000
0x310AC0A4	DMA50_DSCPTR_CUR	DMA50 Current Descriptor Pointer Register	0x00000000
0x310AC0A8	DMA50_DSCPTR_PRV	DMA50 Previous Initial Descriptor Pointer Register	0x00000000
0x310AC0AC	DMA50_ADDR_CUR	DMA50 Current Address Register	0x00000000
0x310AC0B0	DMA50_STAT	DMA50 Status Register	0x00006000
0x310AC0B4	DMA50_XCNT_CUR	DMA50 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310AC0B8	DMA50_YCNT_CUR	DMA50 Current Row Count (2D only) Register	0x00000000
0x310AC0C0	DMA50_BWLCNT	DMA50 Bandwidth Limit Count Register	0x00000000
0x310AC0C4	DMA50_BWLCNT_CUR	DMA50 Bandwidth Limit Count Current Register	0x00000000
0x310AC0C8	DMA50_BWMCNT	DMA50 Bandwidth Monitor Count Register	0x00000000
0x310AC0CC	DMA50_BWMCNT_CUR	DMA50 Bandwidth Monitor Count Current Register	0x00000000

Table A-70: ADSP-2159x DMA51 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109C000	DMA51_DSCPTR_NXT	DMA51 Pointer to Next Initial Descriptor Register	0x00000000
0x3109C004	DMA51_ADDRSTART	DMA51 Start Address of Current Buffer Register	0x00000000
0x3109C008	DMA51_CFG	DMA51 Configuration Register	0x00000220
0x3109C00C	DMA51_XCNT	DMA51 Inner Loop Count Start Value Register	0x00000000
0x3109C010	DMA51_XMOD	DMA51 Inner Loop Address Increment Register	0x00000000
0x3109C014	DMA51_YCNT	DMA51 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109C018	DMA51_YMOD	DMA51 Outer Loop Address Increment (2D only) Register	0x00000000

Table A-70: ADSP-2159x DMA51 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109C024	DMA51_DSCPTR_CUR	DMA51 Current Descriptor Pointer Register	0x00000000
0x3109C028	DMA51_DSCPTR_PRV	DMA51 Previous Initial Descriptor Pointer Register	0x00000000
0x3109C02C	DMA51_ADDR_CUR	DMA51 Current Address Register	0x00000000
0x3109C030	DMA51_STAT	DMA51 Status Register	0x0000B000
0x3109C034	DMA51_XCNT_CUR	DMA51 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109C038	DMA51_YCNT_CUR	DMA51 Current Row Count (2D only) Register	0x00000000
0x3109C040	DMA51_BWLCNT	DMA51 Bandwidth Limit Count Register	0x00000000
0x3109C044	DMA51_BWLCNT_CUR	DMA51 Bandwidth Limit Count Current Register	0x00000000
0x3109C048	DMA51_BWMCNT	DMA51 Bandwidth Monitor Count Register	0x00000000
0x3109C04C	DMA51_BWMCNT_CUR	DMA51 Bandwidth Monitor Count Current Register	0x00000000

Table A-71: ADSP-2159x DMA52 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109C080	DMA52_DSCPTR_NXT	DMA52 Pointer to Next Initial Descriptor Register	0x00000000
0x3109C084	DMA52_ADDRSTART	DMA52 Start Address of Current Buffer Register	0x00000000
0x3109C088	DMA52_CFG	DMA52 Configuration Register	0x00000222
0x3109C08C	DMA52_XCNT	DMA52 Inner Loop Count Start Value Register	0x00000000
0x3109C090	DMA52_XMOD	DMA52 Inner Loop Address Increment Register	0x00000000
0x3109C094	DMA52_YCNT	DMA52 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3109C098	DMA52_YMOD	DMA52 Outer Loop Address Increment (2D only) Register	0x00000000
0x3109C0A4	DMA52_DSCPTR_CUR	DMA52 Current Descriptor Pointer Register	0x00000000
0x3109C0A8	DMA52_DSCPTR_PRV	DMA52 Previous Initial Descriptor Pointer Register	0x00000000
0x3109C0AC	DMA52_ADDR_CUR	DMA52 Current Address Register	0x00000000
0x3109C0B0	DMA52_STAT	DMA52 Status Register	0x0000B000
0x3109C0B4	DMA52_XCNT_CUR	DMA52 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3109C0B8	DMA52_YCNT_CUR	DMA52 Current Row Count (2D only) Register	0x00000000
0x3109C0C0	DMA52_BWLCNT	DMA52 Bandwidth Limit Count Register	0x00000000
0x3109C0C4	DMA52_BWLCNT_CUR	DMA52 Bandwidth Limit Count Current Register	0x00000000

Table A-71: ADSP-2159x DMA52 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109C0C8	DMA52_BWMCNT	DMA52 Bandwidth Monitor Count Register	0x00000000
0x3109C0CC	DMA52_BWMCNT_CUR	DMA52 Bandwidth Monitor Count Current Register	0x00000000

Table A-72: ADSP-2159x DMA53 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31026380	DMA53_DSCPTR_NXT	DMA53 Pointer to Next Initial Descriptor Register	0x00000000
0x31026384	DMA53_ADDRSTART	DMA53 Start Address of Current Buffer Register	0x00000000
0x31026388	DMA53_CFG	DMA53 Configuration Register	0x00000000
0x3102638C	DMA53_XCNT	DMA53 Inner Loop Count Start Value Register	0x00000000
0x31026390	DMA53_XMOD	DMA53 Inner Loop Address Increment Register	0x00000000
0x31026394	DMA53_YCNT	DMA53 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026398	DMA53_YMOD	DMA53 Outer Loop Address Increment (2D only) Register	0x00000000
0x310263A4	DMA53_DSCPTR_CUR	DMA53 Current Descriptor Pointer Register	0x00000000
0x310263A8	DMA53_DSCPTR_PRV	DMA53 Previous Initial Descriptor Pointer Register	0x00000000
0x310263AC	DMA53_ADDR_CUR	DMA53 Current Address Register	0x00000000
0x310263B0	DMA53_STAT	DMA53 Status Register	0x00006000
0x310263B4	DMA53_XCNT_CUR	DMA53 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310263B8	DMA53_YCNT_CUR	DMA53 Current Row Count (2D only) Register	0x00000000
0x310263C0	DMA53_BWLCNT	DMA53 Bandwidth Limit Count Register	0x00000000
0x310263C4	DMA53_BWLCNT_CUR	DMA53 Bandwidth Limit Count Current Register	0x00000000
0x310263C8	DMA53_BWMCNT	DMA53 Bandwidth Monitor Count Register	0x00000000
0x310263CC	DMA53_BWMCNT_CUR	DMA53 Bandwidth Monitor Count Current Register	0x00000000

Table A-73: ADSP-2159x DMA54 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31026300	DMA54_DSCPTR_NXT	DMA54 Pointer to Next Initial Descriptor Register	0x00000000
0x31026304	DMA54_ADDRSTART	DMA54 Start Address of Current Buffer Register	0x00000000
0x31026308	DMA54_CFG	DMA54 Configuration Register	0x00000000

Table A-73: ADSP-2159x DMA54 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102630C	DMA54_XCNT	DMA54 Inner Loop Count Start Value Register	0x00000000
0x31026310	DMA54_XMOD	DMA54 Inner Loop Address Increment Register	0x00000000
0x31026314	DMA54_YCNT	DMA54 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31026318	DMA54_YMOD	DMA54 Outer Loop Address Increment (2D only) Register	0x00000000
0x31026324	DMA54_DSCPTR_CUR	DMA54 Current Descriptor Pointer Register	0x00000000
0x31026328	DMA54_DSCPTR_PRV	DMA54 Previous Initial Descriptor Pointer Register	0x00000000
0x3102632C	DMA54_ADDR_CUR	DMA54 Current Address Register	0x00000000
0x31026330	DMA54_STAT	DMA54 Status Register	0x00006000
0x31026334	DMA54_XCNT_CUR	DMA54 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31026338	DMA54_YCNT_CUR	DMA54 Current Row Count (2D only) Register	0x00000000
0x31026340	DMA54_BWLCNT	DMA54 Bandwidth Limit Count Register	0x00000000
0x31026344	DMA54_BWLCNT_CUR	DMA54 Bandwidth Limit Count Current Register	0x00000000
0x31026348	DMA54_BWMCNT	DMA54 Bandwidth Monitor Count Register	0x00000000
0x3102634C	DMA54_BWMCNT_CUR	DMA54 Bandwidth Monitor Count Current Register	0x00000000

Table A-74: ADSP-2159x DMA55 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102D300	DMA55_DSCPTR_NXT	DMA55 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D304	DMA55_ADDRSTART	DMA55 Start Address of Current Buffer Register	0x00000000
0x3102D308	DMA55_CFG	DMA55 Configuration Register	0x00000000
0x3102D30C	DMA55_XCNT	DMA55 Inner Loop Count Start Value Register	0x00000000
0x3102D310	DMA55_XMOD	DMA55 Inner Loop Address Increment Register	0x00000000
0x3102D314	DMA55_YCNT	DMA55 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D318	DMA55_YMOD	DMA55 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D324	DMA55_DSCPTR_CUR	DMA55 Current Descriptor Pointer Register	0x00000000
0x3102D328	DMA55_DSCPTR_PRV	DMA55 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D32C	DMA55_ADDR_CUR	DMA55 Current Address Register	0x00000000

Table A-74: ADSP-2159x DMA55 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102D330	DMA55_STAT	DMA55 Status Register	0x00006000
0x3102D334	DMA55_XCNT_CUR	DMA55 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D338	DMA55_YCNT_CUR	DMA55 Current Row Count (2D only) Register	0x00000000
0x3102D340	DMA55_BWLCNT	DMA55 Bandwidth Limit Count Register	0x00000000
0x3102D344	DMA55_BWLCNT_CUR	DMA55 Bandwidth Limit Count Current Register	0x00000000
0x3102D348	DMA55_BWMCNT	DMA55 Bandwidth Monitor Count Register	0x00000000
0x3102D34C	DMA55_BWMCNT_CUR	DMA55 Bandwidth Monitor Count Current Register	0x00000000

Table A-75: ADSP-2159x DMA56 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102D380	DMA56_DSCPTR_NXT	DMA56 Pointer to Next Initial Descriptor Register	0x00000000
0x3102D384	DMA56_ADDRSTART	DMA56 Start Address of Current Buffer Register	0x00000000
0x3102D388	DMA56_CFG	DMA56 Configuration Register	0x00000000
0x3102D38C	DMA56_XCNT	DMA56 Inner Loop Count Start Value Register	0x00000000
0x3102D390	DMA56_XMOD	DMA56 Inner Loop Address Increment Register	0x00000000
0x3102D394	DMA56_YCNT	DMA56 Outer Loop Count Start Value (2D only) Register	0x00000000
0x3102D398	DMA56_YMOD	DMA56 Outer Loop Address Increment (2D only) Register	0x00000000
0x3102D3A4	DMA56_DSCPTR_CUR	DMA56 Current Descriptor Pointer Register	0x00000000
0x3102D3A8	DMA56_DSCPTR_PRV	DMA56 Previous Initial Descriptor Pointer Register	0x00000000
0x3102D3AC	DMA56_ADDR_CUR	DMA56 Current Address Register	0x00000000
0x3102D3B0	DMA56_STAT	DMA56 Status Register	0x00006000
0x3102D3B4	DMA56_XCNT_CUR	DMA56 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x3102D3B8	DMA56_YCNT_CUR	DMA56 Current Row Count (2D only) Register	0x00000000
0x3102D3C0	DMA56_BWLCNT	DMA56 Bandwidth Limit Count Register	0x00000000
0x3102D3C4	DMA56_BWLCNT_CUR	DMA56 Bandwidth Limit Count Current Register	0x00000000
0x3102D3C8	DMA56_BWMCNT	DMA56 Bandwidth Monitor Count Register	0x00000000
0x3102D3CC	DMA56_BWMCNT_CUR	DMA56 Bandwidth Monitor Count Current Register	0x00000000

Table A-76: ADSP-2159x DMA6 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022300	DMA6_DSCPTR_NXT	DMA6 Pointer to Next Initial Descriptor Register	0x00000000
0x31022304	DMA6_ADDRSTART	DMA6 Start Address of Current Buffer Register	0x00000000
0x31022308	DMA6_CFG	DMA6 Configuration Register	0x00000000
0x3102230C	DMA6_XCNT	DMA6 Inner Loop Count Start Value Register	0x00000000
0x31022310	DMA6_XMOD	DMA6 Inner Loop Address Increment Register	0x00000000
0x31022314	DMA6_YCNT	DMA6 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022318	DMA6_YMOD	DMA6 Outer Loop Address Increment (2D only) Register	0x00000000
0x31022324	DMA6_DSCPTR_CUR	DMA6 Current Descriptor Pointer Register	0x00000000
0x31022328	DMA6_DSCPTR_PRV	DMA6 Previous Initial Descriptor Pointer Register	0x00000000
0x3102232C	DMA6_ADDR_CUR	DMA6 Current Address Register	0x00000000
0x31022330	DMA6_STAT	DMA6 Status Register	0x00006000
0x31022334	DMA6_XCNT_CUR	DMA6 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x31022338	DMA6_YCNT_CUR	DMA6 Current Row Count (2D only) Register	0x00000000
0x31022340	DMA6_BWLCNT	DMA6 Bandwidth Limit Count Register	0x00000000
0x31022344	DMA6_BWLCNT_CUR	DMA6 Bandwidth Limit Count Current Register	0x00000000
0x31022348	DMA6_BWMCNT	DMA6 Bandwidth Monitor Count Register	0x00000000
0x3102234C	DMA6_BWMCNT_CUR	DMA6 Bandwidth Monitor Count Current Register	0x00000000

Table A-77: ADSP-2159x DMA7 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31022380	DMA7_DSCPTR_NXT	DMA7 Pointer to Next Initial Descriptor Register	0x00000000
0x31022384	DMA7_ADDRSTART	DMA7 Start Address of Current Buffer Register	0x00000000
0x31022388	DMA7_CFG	DMA7 Configuration Register	0x00000000
0x3102238C	DMA7_XCNT	DMA7 Inner Loop Count Start Value Register	0x00000000
0x31022390	DMA7_XMOD	DMA7 Inner Loop Address Increment Register	0x00000000
0x31022394	DMA7_YCNT	DMA7 Outer Loop Count Start Value (2D only) Register	0x00000000
0x31022398	DMA7_YMOD	DMA7 Outer Loop Address Increment (2D only) Register	0x00000000
0x310223A4	DMA7_DSCPTR_CUR	DMA7 Current Descriptor Pointer Register	0x00000000
0x310223A8	DMA7_DSCPTR_PRV	DMA7 Previous Initial Descriptor Pointer Register	0x00000000
0x310223AC	DMA7_ADDR_CUR	DMA7 Current Address Register	0x00000000

Table A-77: ADSP-2159x DMA7 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310223B0	DMA7_STAT	DMA7 Status Register	0x00006000
0x310223B4	DMA7_XCNT_CUR	DMA7 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310223B8	DMA7_YCNT_CUR	DMA7 Current Row Count (2D only) Register	0x00000000
0x310223C0	DMA7_BWLCNT	DMA7 Bandwidth Limit Count Register	0x00000000
0x310223C4	DMA7_BWLCNT_CUR	DMA7 Bandwidth Limit Count Current Register	0x00000000
0x310223C8	DMA7_BWMCNT	DMA7 Bandwidth Monitor Count Register	0x00000000
0x310223CC	DMA7_BWMCNT_CUR	DMA7 Bandwidth Monitor Count Current Register	0x00000000

Table A-78: ADSP-2159x DMA8 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7000	DMA8_DSCPTR_NXT	DMA8 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7004	DMA8_ADDRSTART	DMA8 Start Address of Current Buffer Register	0x00000000
0x310A7008	DMA8_CFG	DMA8 Configuration Register	0x00000000
0x310A700C	DMA8_XCNT	DMA8 Inner Loop Count Start Value Register	0x00000000
0x310A7010	DMA8_XMOD	DMA8 Inner Loop Address Increment Register	0x00000000
0x310A7014	DMA8_YCNT	DMA8 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7018	DMA8_YMOD	DMA8 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A7024	DMA8_DSCPTR_CUR	DMA8 Current Descriptor Pointer Register	0x00000000
0x310A7028	DMA8_DSCPTR_PRV	DMA8 Previous Initial Descriptor Pointer Register	0x00000000
0x310A702C	DMA8_ADDR_CUR	DMA8 Current Address Register	0x00000000
0x310A7030	DMA8_STAT	DMA8 Status Register	0x00006000
0x310A7034	DMA8_XCNT_CUR	DMA8 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A7038	DMA8_YCNT_CUR	DMA8 Current Row Count (2D only) Register	0x00000000
0x310A7040	DMA8_BWLCNT	DMA8 Bandwidth Limit Count Register	0x00000000
0x310A7044	DMA8_BWLCNT_CUR	DMA8 Bandwidth Limit Count Current Register	0x00000000
0x310A7048	DMA8_BWMCNT	DMA8 Bandwidth Monitor Count Register	0x00000000
0x310A704C	DMA8_BWMCNT_CUR	DMA8 Bandwidth Monitor Count Current Register	0x00000000

Table A-79: ADSP-2159x DMA9 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A7080	DMA9_DSCPTR_NXT	DMA9 Pointer to Next Initial Descriptor Register	0x00000000
0x310A7084	DMA9_ADDRSTART	DMA9 Start Address of Current Buffer Register	0x00000000
0x310A7088	DMA9_CFG	DMA9 Configuration Register	0x00000000
0x310A708C	DMA9_XCNT	DMA9 Inner Loop Count Start Value Register	0x00000000
0x310A7090	DMA9_XMOD	DMA9 Inner Loop Address Increment Register	0x00000000
0x310A7094	DMA9_YCNT	DMA9 Outer Loop Count Start Value (2D only) Register	0x00000000
0x310A7098	DMA9_YMOD	DMA9 Outer Loop Address Increment (2D only) Register	0x00000000
0x310A70A4	DMA9_DSCPTR_CUR	DMA9 Current Descriptor Pointer Register	0x00000000
0x310A70A8	DMA9_DSCPTR_PRV	DMA9 Previous Initial Descriptor Pointer Register	0x00000000
0x310A70AC	DMA9_ADDR_CUR	DMA9 Current Address Register	0x00000000
0x310A70B0	DMA9_STAT	DMA9 Status Register	0x00006000
0x310A70B4	DMA9_XCNT_CUR	DMA9 Current Count (1D) or Intra-row XCNT (2D) Register	0x00000000
0x310A70B8	DMA9_YCNT_CUR	DMA9 Current Row Count (2D only) Register	0x00000000
0x310A70C0	DMA9_BWLCNT	DMA9 Bandwidth Limit Count Register	0x00000000
0x310A70C4	DMA9_BWLCNT_CUR	DMA9 Bandwidth Limit Count Current Register	0x00000000
0x310A70C8	DMA9_BWMCNT	DMA9 Bandwidth Monitor Count Register	0x00000000
0x310A70CC	DMA9_BWMCNT_CUR	DMA9 Bandwidth Monitor Count Current Register	0x00000000

Table A-80: ADSP-2159x DMC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31070004	DMC0_CTL	DMC0 Control Register	0x00000000
0x31070008	DMC0_STAT	DMC0 Status Register	0x00000001
0x3107000C	DMC0_EFFCTL	DMC0 Efficiency Control Register	0x00440000
0x31070010	DMC0_PRIO	DMC0 Priority ID Register 1	0x00000000
0x31070014	DMC0_PRIOMSK	DMC0 Priority ID Mask Register 1	0x00000000
0x31070018	DMC0_PRIO2	DMC0 Priority ID Register 2	0x00000000
0x3107001C	DMC0_PRIOMSK2	DMC0 Priority ID Mask Register 2	0x00000000
0x31070040	DMC0_CFG	DMC0 Configuration Register	0x00000000
0x31070044	DMC0_TR0	DMC0 Timing 0 Register	0x00000000
0x31070048	DMC0_TR1	DMC0 Timing 1 Register	0x00000000

Table A-80: ADSP-2159x DMC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3107004C	DMC0_TR2	DMC0 Timing 2 Register	0x00000000
0x3107005C	DMC0_MSK	DMC0 Mask (Mode Register Shadow) Register	0x00000000
0x31070060	DMC0_MR	DMC0 Shadow MR0 Register (DDR3)	0x00000000
0x31070064	DMC0_MR1	DMC0 Shadow MR1 Register (DDR3)	0x00000000
0x31070068	DMC0_MR2	DMC0 Shadow MR2 Register (DDR3)	0x00000000
0x3107006C	DMC0_EMRS	DMC0 Shadow EMRS Register	0x00000000
0x31070080	DMC0_DLLCTL	DMC0 DLL Control Register	0x0000054B
0x31070090	DMC0_DT_CALIB_ADDR	DMC0 Data Calibration Address Register	0x00000000
0x31070094	DMC0_DT_DATA_CAL- IB_DATA0	DMC0 Data Calibration Data 0 Register	0x00000000
0x31070098	DMC0_DT_DATA_CAL- IB_DATA1	DMC0 Data Calibration Data 1 Register	0xFFFFFFFF
0x31070100	DMC0_RDDATABUFID1	DMC0 DMC Read Data Buffer ID Register 1	0x00000000
0x31070104	DMC0_RDDATABUFMSK1	DMC0 DMC Read Data Buffer Mask Register 1	0x00000000
0x31070108	DMC0_RDDATABUFID2	DMC0 DMC Read Data Buffer ID Register 2	0x00000000
0x3107010C	DMC0_RDDATABUFMSK2	DMC0 DMC Read Data Buffer Mask Register 2	0x00000000

Table A-81: ADSP-2159x DPM0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31090000	DPM0_CTL	DPM0 Control Register	0x00000000
0x31090004	DPM0_STAT	DPM0 Status Register	0x00000001
0x31090070	DPM0_PER_DIS0	DPM0 Peripherals Disable Register 0	0x00000000
0x31090074	DPM0_PER_DIS1	DPM0 Peripherals Disable Register 1	0x00000000
0x31090084	DPM0_REVID	DPM0 Revision ID	0x00000020

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31040000	EMAC0_MACCFG	EMAC0 MAC Configuration Register	0x00008000
0x31040004	EMAC0_MACFRMFILT	EMAC0 MAC Rx Frame Filter Register	0x00000000
0x31040008	EMAC0_HASHTBL_HI	EMAC0 Hash Table High Register	0x00000000
0x3104000C	EMAC0_HASHTBL_LO	EMAC0 Hash Table Low Register	0x00000000

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31040010	EMAC0_SMI_ADDR	EMAC0 SMI Address Register	0x00000000
0x31040014	EMAC0_SMI_DATA	EMAC0 SMI Data Register	0x00000000
0x31040018	EMAC0_FLOWCTL	EMAC0 FLOW Control Register	0x00000000
0x3104001C	EMAC0_VLANTAG	EMAC0 VLAN Tag Register	0x00000000
0x31040024	EMAC0_DBG	EMAC0 Debug Register	0x00000000
0x31040030	EMAC0_LPI_CTLSTAT	EMAC0 Low Power Idle Control and Status Register	0x00000000
0x31040034	EMAC0_LPI_TMRSCCTL	EMAC0 Low Power Idle Timeout Register	0x00000000
0x31040038	EMAC0_ISTAT	EMAC0 Interrupt Status Register	0x00000000
0x3104003C	EMAC0_IMSK	EMAC0 Interrupt Mask Register	0x00000000
0x31040040	EMAC0_ADDR0_HI	EMAC0 MAC Address 0 High Register	0x8000FFFF
0x31040044	EMAC0_ADDR0_LO	EMAC0 MAC Address 0 Low Register	0xFFFFFFFF
0x31040048	EMAC0_ADDR1_HI	EMAC0 MAC Address 1 High Register	0x00000000
0x3104004C	EMAC0_ADDR1_LO	EMAC0 MAC Address 1 Low Register	0x00000000
0x310400D8	EMAC0_GIGE_CTLSTAT	EMAC0 RGMII Control and Status Register	0x00000000
0x310400DC	EMAC0_WDOG_TIMEOUT	EMAC0 Watchdog Timeout Register	0x00000000
0x31040100	EMAC0_MMC_CTL	EMAC0 MMC Control Register	0x00000000
0x31040104	EMAC0_MMC_RXINT	EMAC0 MMC Rx Interrupt Register	0x00000000
0x31040108	EMAC0_MMC_TXINT	EMAC0 MMC Tx Interrupt Register	0x00000000
0x3104010C	EMAC0_MMC_RXIMSK	EMAC0 MMC Rx Interrupt Mask Register	0x00000000
0x31040110	EMAC0_MMC_TXIMSK	EMAC0 MMC TX Interrupt Mask Register	0x00000000
0x31040114	EMAC0_TXOCTCNT_GB	EMAC0 Tx OCT Count (Good/Bad) Register	0x00000000
0x31040118	EMAC0_TXFRMCNT_GB	EMAC0 Tx Frame Count (Good/Bad) Register	0x00000000
0x3104011C	EMAC0_TXBCASTFRM_G	EMAC0 Tx Broadcast Frames (Good) Register	0x00000000
0x31040120	EMAC0_TXMCASTFRM_G	EMAC0 Tx Multicast Frames (Good) Register	0x00000000
0x31040124	EMAC0_TX64_GB	EMAC0 Tx 64-Byte Frames (Good/Bad) Register	0x00000000
0x31040128	EMAC0_TX65TO127_GB	EMAC0 Tx 65- to 127-Byte Frames (Good/Bad) Register	0x00000000
0x3104012C	EMAC0_TX128TO255_GB	EMAC0 Tx 128- to 255-Byte Frames (Good/Bad) Register	0x00000000
0x31040130	EMAC0_TX256TO511_GB	EMAC0 Tx 256- to 511-Byte Frames (Good/Bad) Register	0x00000000
0x31040134	EMAC0_TX512TO1023_GB	EMAC0 Tx 512- to 1023-Byte Frames (Good/Bad) Register	0x00000000

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31040138	EMAC0_TX1024TOM- AX_GB	EMAC0 Tx 1024- to Max-Byte Frames (Good/Bad) Reg- ister	0x00000000
0x3104013C	EMAC0_TXU- CASTFRM_GB	EMAC0 Tx Unicast Frames (Good/Bad) Register	0x00000000
0x31040140	EMAC0_TXMCASTFRM_G B	EMAC0 Tx Multicast Frames (Good/Bad) Register	0x00000000
0x31040144	EMAC0_TXBCASTFRM_G B	EMAC0 Tx Broadcast Frames (Good/Bad) Register	0x00000000
0x31040148	EMAC0_TXUNDR_ERR	EMAC0 Tx Underflow Error Register	0x00000000
0x3104014C	EMAC0_TXSNGCOL_G	EMAC0 Tx Single Collision (Good) Register	0x00000000
0x31040150	EMAC0_TXMULTCOL_G	EMAC0 Tx Multiple Collision (Good) Register	0x00000000
0x31040154	EMAC0_TXDEFERRED	EMAC0 Tx Deferred Register	0x00000000
0x31040158	EMAC0_TXLATECOL	EMAC0 Tx Late Collision Register	0x00000000
0x3104015C	EMAC0_TXEXCESSCOL	EMAC0 Tx Excess Collision Register	0x00000000
0x31040160	EMAC0_TXCARR_ERR	EMAC0 Tx Carrier Error Register	0x00000000
0x31040164	EMAC0_TXOCTCNT_G	EMAC0 Tx Octet Count (Good) Register	0x00000000
0x31040168	EMAC0_TXFRMCNT_G	EMAC0 Tx Frame Count (Good) Register	0x00000000
0x3104016C	EMAC0_TXEXCESSDEF	EMAC0 Tx Excess Deferral Register	0x00000000
0x31040170	EMAC0_TXPAUSEFRM	EMAC0 Tx Pause Frame Register	0x00000000
0x31040174	EMAC0_TXVLANFRM_G	EMAC0 Tx VLAN Frames (Good) Register	0x00000000
0x31040178	EMAC0_TXOVRSIZE_G	EMAC0 Number of Tx Frames (Good) greater than max- size	0x00000000
0x31040180	EMAC0_RXFRMCNT_GB	EMAC0 Rx Frame Count (Good/Bad) Register	0x00000000
0x31040184	EMAC0_RXOCTCNT_GB	EMAC0 Rx Octet Count (Good/Bad) Register	0x00000000
0x31040188	EMAC0_RXOCTCNT_G	EMAC0 Rx Octet Count (Good) Register	0x00000000
0x3104018C	EMAC0_RXBCASTFRM_G	EMAC0 Rx Broadcast Frames (Good) Register	0x00000000
0x31040190	EMAC0_RXMCASTFRM_G	EMAC0 Rx Multicast Frames (Good) Register	0x00000000
0x31040194	EMAC0_RXCRC_ERR	EMAC0 Rx CRC Error Register	0x00000000
0x31040198	EMAC0_RXALIGN_ERR	EMAC0 Rx alignment Error Register	0x00000000
0x3104019C	EMAC0_RXRUNT_ERR	EMAC0 Rx Runt Error Register	0x00000000
0x310401A0	EMAC0_RXJAB_ERR	EMAC0 Rx Jab Error Register	0x00000000
0x310401A4	EMAC0_RXUSIZE_G	EMAC0 Rx Undersize (Good) Register	0x00000000

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310401A8	EMAC0_RXOSIZE_G	EMAC0 Rx Oversize (Good) Register	0x00000000
0x310401AC	EMAC0_RX64_GB	EMAC0 Rx 64-Byte Frames (Good/Bad) Register	0x00000000
0x310401B0	EMAC0_RX65TO127_GB	EMAC0 Rx 65- to 127-Byte Frames (Good/Bad) Register	0x00000000
0x310401B4	EMAC0_RX128TO255_GB	EMAC0 Rx 128- to 255-Byte Frames (Good/Bad) Register	0x00000000
0x310401B8	EMAC0_RX256TO511_GB	EMAC0 Rx 256- to 511-Byte Frames (Good/Bad) Register	0x00000000
0x310401BC	EMAC0_RX512TO1023_GB	EMAC0 Rx 512- to 1023-Byte Frames (Good/Bad) Register	0x00000000
0x310401C0	EMAC0_RX1024TOMAX_GB	EMAC0 Rx 1024- to Max-Byte Frames (Good/Bad) Register	0x00000000
0x310401C4	EMAC0_RXUCASTFRM_G	EMAC0 Rx Unicast Frames (Good) Register	0x00000000
0x310401C8	EMAC0_RXLEN_ERR	EMAC0 Rx Length Error Register	0x00000000
0x310401CC	EMAC0_RXOORTYPE	EMAC0 Rx Out Of Range Type Register	0x00000000
0x310401D0	EMAC0_RXPAUSEFRM	EMAC0 Rx Pause Frames Register	0x00000000
0x310401D4	EMAC0_RXFIFO_OVF	EMAC0 Rx FIFO Overflow Register	0x00000000
0x310401D8	EMAC0_RXVLANFRM_GB	EMAC0 Rx VLAN Frames (Good/Bad) Register	0x00000000
0x310401DC	EMAC0_RXWDOG_ERR	EMAC0 Rx Watch Dog Error Register	0x00000000
0x310401E0	EMAC0_RXRCV_ERR	EMAC0 Rx Error Frames Received Register	0x00000000
0x310401E4	EMAC0_RXCTLFrm_G	EMAC0 Rx Good Control Frames Register	0x00000000
0x31040200	EMAC0_IPC_RXIMSK	EMAC0 MMC IPC Rx Interrupt Mask Register	0x00000000
0x31040208	EMAC0_IPC_RXINT	EMAC0 MMC IPC Rx Interrupt Register	0x00000000
0x31040210	EMAC0_RXIPV4_GD_FRM	EMAC0 Rx IPv4 Datagrams (Good) Register	0x00000000
0x31040214	EMAC0_RXIPV4_HDR_ERR_FRM	EMAC0 Rx IPv4 Datagrams Header Errors Register	0x00000000
0x31040218	EMAC0_RXIPV4_NO-PAY_FRM	EMAC0 Rx IPv4 Datagrams No Payload Frame Register	0x00000000
0x3104021C	EMAC0_RXIPV4_FRAG_FRM	EMAC0 Rx IPv4 Datagrams Fragmented Frames Register	0x00000000
0x31040220	EMAC0_RXIPV4_UDSBL_FRM	EMAC0 Rx IPv4 UDP Disabled Frames Register	0x00000000
0x31040224	EMAC0_RXIPV6_GD_FRM	EMAC0 Rx IPv6 Datagrams Good Frames Register	0x00000000
0x31040228	EMAC0_RXIPV6_HDR_ERR_FRM	EMAC0 Rx IPv6 Datagrams Header Error Frames Register	0x00000000

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3104022C	EMAC0_RXIPV6_NO- PAY_FRM	EMAC0 Rx IPv6 Datagrams No Payload Frames Register	0x00000000
0x31040230	EMAC0_RXUDP_GD_FRM	EMAC0 Rx UDP Good Frames Register	0x00000000
0x31040234	EMAC0_RXUDP_ERR_FR M	EMAC0 Rx UDP Error Frames Register	0x00000000
0x31040238	EMAC0_RXTCP_GD_FRM	EMAC0 Rx TCP Good Frames Register	0x00000000
0x3104023C	EMAC0_RXTCP_ERR_FRM	EMAC0 Rx TCP Error Frames Register	0x00000000
0x31040240	EMAC0_RXICMP_GD_FR M	EMAC0 Rx ICMP Good Frames Register	0x00000000
0x31040244	EMAC0_RXICMP_ERR_FR M	EMAC0 Rx ICMP Error Frames Register	0x00000000
0x31040250	EMAC0_RXIPV4_GD_OCT	EMAC0 Rx IPv4 Datagrams Good Octets Register	0x00000000
0x31040254	EMAC0_RXIPV4_HDR_ER R_OCT	EMAC0 Rx IPv4 Datagrams Header Errors Register	0x00000000
0x31040258	EMAC0_RXIPV4_NO- PAY_OCT	EMAC0 Rx IPv4 Datagrams No Payload Octets Register	0x00000000
0x3104025C	EMAC0_RXIPV4_FRAG_O CT	EMAC0 Rx IPv4 Datagrams Fragmented Octets Register	0x00000000
0x31040260	EMAC0_RXIPV4_UDSBL_ OCT	EMAC0 Rx IPv4 UDP Disabled Octets Register	0x00000000
0x31040264	EMAC0_RXIPV6_GD_OCT	EMAC0 Rx IPv6 Good Octets Register	0x00000000
0x31040268	EMAC0_RXIPV6_HDR_ER R_OCT	EMAC0 Rx IPv6 Header Errors Register	0x00000000
0x3104026C	EMAC0_RXIPV6_NO- PAY_OCT	EMAC0 Rx IPv6 No Payload Octets Register	0x00000000
0x31040270	EMAC0_RXUDP_GD_OCT	EMAC0 Rx UDP Good Octets Register	0x00000000
0x31040274	EMAC0_RXUDP_ERR_OC T	EMAC0 Rx UDP Error Octets Register	0x00000000
0x31040278	EMAC0_RXTCP_GD_OCT	EMAC0 Rx TCP Good Octets Register	0x00000000
0x3104027C	EMAC0_RXTCP_ERR_OC T	EMAC0 Rx TCP Error Octets Register	0x00000000
0x31040280	EMAC0_RXICMP_GD_OC T	EMAC0 Rx ICMP Good Octets Register	0x00000000
0x31040284	EMAC0_RXICMP_ERR_OC T	EMAC0 Rx ICMP Error Octets Register	0x00000000

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31040400	EMAC0_L3L4_CTL	EMAC0 Layer3 and Layer4 Control Register	0x00000000
0x31040404	EMAC0_L4_ADDR	EMAC0 Layer 4 Address Register	0x00000000
0x31040410	EMAC0_L3_ADDR0	EMAC0 Layer 3 Address0 Register	0x00000000
0x31040414	EMAC0_L3_ADDR1	EMAC0 Layer 3 Address1 Register	0x00000000
0x31040418	EMAC0_L3_ADDR2	EMAC0 Layer 3 Address2 Register	0x00000000
0x3104041C	EMAC0_L3_ADDR3	EMAC0 Layer 3 Address3 Register	0x00000000
0x31040584	EMAC0_VLAN_INCL	EMAC0 VLAN Tag Inclusion or Replacement Register	0x00000000
0x31040588	EMAC0_VLAN_HSHTBL	EMAC0 VLAN Hash Table Register	0x00000000
0x31040700	EMAC0_TM_CTL	EMAC0 Time Stamp Control Register	0x00002000
0x31040704	EMAC0_TM_SUBSEC	EMAC0 Time Stamp Sub Second Increment Register	0x00000000
0x31040708	EMAC0_TM_SEC	EMAC0 Time Stamp Low Seconds Register	0x00000000
0x3104070C	EMAC0_TM_NSEC	EMAC0 Time Stamp Nanoseconds Register	0x00000000
0x31040710	EMAC0_TM_SECUPDT	EMAC0 Time Stamp Seconds Update Register	0x00000000
0x31040714	EMAC0_TM_NSECUPDT	EMAC0 Time Stamp Nanoseconds Update Register	0x00000000
0x31040718	EMAC0_TM_ADDEND	EMAC0 Time Stamp Addend Register	0x00000000
0x3104071C	EMAC0_TM_PPS0TGTM	EMAC0 Time Stamp Target Time Seconds Register	0x00000000
0x31040720	EMAC0_TM_PPS0NTGTM	EMAC0 Time Stamp Target Time Nanoseconds Register	0x00000000
0x31040724	EMAC0_TM_HISEC	EMAC0 Time Stamp High Second Register	0x00000000
0x31040728	EMAC0_TM_STMPSTAT	EMAC0 Time Stamp Status Register	0x00000000
0x3104072C	EMAC0_TM_PPSCtl	EMAC0 PPS Control Register	0x00000000
0x31040730	EMAC0_TM_AUXSTMP_N SEC	EMAC0 Time Stamp Auxiliary TS Nano Seconds Register	0x00000000
0x31040734	EMAC0_TM_AUXSTMP_S EC	EMAC0 Time Stamp Auxiliary TM Seconds Register	0x00000000
0x31040738	EMAC0_MAC_AVCTL	EMAC0 AV MAC Control Register	0x00000000
0x31040760	EMAC0_TM_PPS0INTVL	EMAC0 Time Stamp PPS Interval Register	0x00000000
0x31040764	EMAC0_TM_PPS0WIDTH	EMAC0 PPS Width Register	0x00000000
0x31040780	EMAC0_TM_PPS1TGTM	EMAC0 PPS 1 Target Time Seconds Register	0x00000000
0x31040784	EMAC0_TM_PPS1NTGTM	EMAC0 PPS 1 Target Time Nanoseconds Register	0x00000000
0x31040788	EMAC0_TM_PPS1INTVL	EMAC0 PPS 1 Interval Register	0x00000000
0x3104078C	EMAC0_TM_PPS1WIDTH	EMAC0 PPS 1 Width Register	0x00000000

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310407A0	EMAC0_TM_PPS2TGTM	EMAC0 PPS 2 Target Time Seconds Register	0x00000000
0x310407A4	EMAC0_TM_PPS2NTGTM	EMAC0 PPS 2 Target Time Nanoseconds Register	0x00000000
0x310407A8	EMAC0_TM_PPS2INTVL	EMAC0 PPS 2 Interval Register	0x00000000
0x310407AC	EMAC0_TM_PPS2WIDTH	EMAC0 PPS 2 Width Register	0x00000000
0x310407C0	EMAC0_TM_PPS3TGTM	EMAC0 PPS 3 Target Time Seconds Register	0x00000000
0x310407C4	EMAC0_TM_PPS3NTGTM	EMAC0 PPS 3 Target Time Nanoseconds Register	0x00000000
0x310407C8	EMAC0_TM_PPS3INTVL	EMAC0 PPS 3 Interval Register	0x00000000
0x310407CC	EMAC0_TM_PPS3WIDTH	EMAC0 PPS 3 Width Register	0x00000000
0x31041000	EMAC0_DMA0_BUS- MODE	EMAC0 DMA Bus Mode Register	0x00020101
0x31041004	EMAC0_DMA0_TXPOLL	EMAC0 DMA Tx Poll Demand Register	0x00000000
0x31041008	EMAC0_DMA0_RXPOLL	EMAC0 DMA Rx Poll Demand register	0x00000000
0x3104100C	EMAC0_DMA0_RXDSC_A DDR	EMAC0 DMA Rx Descriptor List Address Register	0x00000000
0x31041010	EMAC0_DMA0_TXDSC_A DDR	EMAC0 DMA Tx Descriptor List Address Register	0x00000000
0x31041014	EMAC0_DMA0_STAT	EMAC0 DMA Status Register	0x00000000
0x31041018	EMAC0_DMA0_OPMODE	EMAC0 DMA Operation Mode Register	0x00000000
0x3104101C	EMAC0_DMA0_IEN	EMAC0 DMA Interrupt Enable Register	0x00000000
0x31041020	EMAC0_DMA0_MISS_FRM	EMAC0 DMA Missed Frame Register	0x00000000
0x31041024	EMAC0_DMA0_RXIW- DOG	EMAC0 DMA Rx Interrupt Watch Dog Register	0x00000000
0x31041028	EMAC0_DMA0_BMMODE	EMAC0 DMA SCB Bus Mode Register	0xC0110001
0x3104102C	EMAC0_DMA0_BMSTAT	EMAC0 DMA SCB Status Register	0x00000000
0x31041048	EMAC0_DMA0_TXDSC_C UR	EMAC0 DMA Tx Descriptor Current Register	0x00000000
0x3104104C	EMAC0_DMA0_RXDSC_C UR	EMAC0 DMA Rx Descriptor Current Register	0x00000000
0x31041050	EMAC0_DMA0_TXBUF_C UR	EMAC0 DMA Tx Buffer Current Register	0x00000000
0x31041054	EMAC0_DMA0_RXBUF_C UR	EMAC0 DMA Rx Buffer Current Register	0x00000000
0x31041100	EMAC0_DMA1_BUS- MODE	EMAC0 DMA Bus Mode Register	0x00020100

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31041104	EMAC0_DMA1_TXPOLL	EMAC0 DMA Tx Poll Demand Register	0x00000000
0x31041108	EMAC0_DMA1_RXPOLL	EMAC0 DMA Rx Poll Demand Register	0x00000000
0x3104110C	EMAC0_DMA1_RXDSC_A DDR	EMAC0 DMA Rx Descriptor List Address Register	0x00000000
0x31041110	EMAC0_DMA1_TXDSC_A DDR	EMAC0 DMA Tx Descriptor List Address Register	0x00000000
0x31041114	EMAC0_DMA1_STAT	EMAC0 DMA Status Register	0x00000000
0x31041118	EMAC0_DMA1_OPMODE	EMAC0 DMA Operation Mode Register	0x00000000
0x3104111C	EMAC0_DMA1_IEN	EMAC0 DMA Interrupt Enable Register	0x00000000
0x31041120	EMAC0_DMA1_MISS_FRM	EMAC0 DMA Missed Frame Register	0x00000000
0x31041124	EMAC0_DMA1_RXIW- DOG	EMAC0 DMA Rx Interrupt Watch Dog Register	0x00000000
0x31041130	EMAC0_DMA1_CHSFCS	EMAC0 Channel 1 Control Bits for Slot Function Regis- ter	0x00000000
0x31041148	EMAC0_DMA1_TXDSC_C UR	EMAC0 DMA Tx Descriptor Current Register	0x00000000
0x3104114C	EMAC0_DMA1_RXDSC_C UR	EMAC0 DMA Rx Descriptor Current Register	0x00000000
0x31041150	EMAC0_DMA1_TXBUF_C UR	EMAC0 DMA Tx Buffer Current Register	0x00000000
0x31041154	EMAC0_DMA1_RXBUF_C UR	EMAC0 DMA Rx Buffer Current Register	0x00000000
0x31041160	EMAC0_DMA1_CHCBSCT L	EMAC0 Channel 1 Credit Shaping Control Register	0x00000000
0x31041164	EMAC0_DMA1_CHCBSST AT	EMAC0 Channel 1 Average Traffic Transmitted Register	0x00000000
0x31041168	EMAC0_DMA1_CHISC	EMAC0 Channel 1 Idle Slope Credit Value Register	0x00000000
0x3104116C	EMAC0_DMA1_CHSSC	EMAC0 Channel 1 Send Slope Credit Value Register	0x00000000
0x31041170	EMAC0_DMA1_CHHIC	EMAC0 Channel 1 High Credit Value Register	0x00000000
0x31041174	EMAC0_DMA1_CHLOC	EMAC0 Channel 1 Low Credit Value Register	0x00000000
0x31041200	EMAC0_DMA2_BUS- MODE	EMAC0 DMA Bus Mode Register	0x00020101
0x31041204	EMAC0_DMA2_TXPOLL	EMAC0 DMA Tx Poll Demand Register	0x00000000
0x31041208	EMAC0_DMA2_RXPOLL	EMAC0 DMA Rx Poll Demand register	0x00000000

Table A-82: ADSP-2159x EMAC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3104120C	EMAC0_DMA2_RXDSC_A DDR	EMAC0 DMA Rx Descriptor List Address Register	0x00000000
0x31041210	EMAC0_DMA2_TXDSC_A DDR	EMAC0 DMA Tx Descriptor List Address Register	0x00000000
0x31041214	EMAC0_DMA2_STAT	EMAC0 DMA Status Register	0x00000000
0x31041218	EMAC0_DMA2_OPMODE	EMAC0 DMA Operation Mode Register	0x00000000
0x3104121C	EMAC0_DMA2_IEN	EMAC0 DMA Interrupt Enable Register	0x00000000
0x31041220	EMAC0_DMA2_MISS_FRM	EMAC0 DMA Missed Frame Register	0x00000000
0x31041224	EMAC0_DMA2_RXIW- DOG	EMAC0 DMA Rx Interrupt Watch Dog Register	0x00000000
0x31041230	EMAC0_DMA2_CHSFC	EMAC0 Channel 2 Control Bits for Slot Function Register	0x00000000
0x31041248	EMAC0_DMA2_TXDSC_C UR	EMAC0 DMA Tx Descriptor Current Register	0x00000000
0x3104124C	EMAC0_DMA2_RXDSC_C UR	EMAC0 DMA Rx Descriptor Current Register	0x00000000
0x31041250	EMAC0_DMA2_TXBUF_C UR	EMAC0 DMA Tx Buffer Current Register	0x00000000
0x31041254	EMAC0_DMA2_RXBUF_C UR	EMAC0 DMA Rx Buffer Current Register	0x00000000
0x31041260	EMAC0_DMA2_CHCBSCT L	EMAC0 Channel 2 Credit Shaping Control Register	0x00000000
0x31041264	EMAC0_DMA2_CHCBSST AT	EMAC0 Channel 2 Avg Traffic Transmitted Status Register	0x00000000
0x31041268	EMAC0_DMA2_CHISC	EMAC0 Channel 2 Idle Slope Credit Value Register	0x00000000
0x3104126C	EMAC0_DMA2_CHSSC	EMAC0 Channel 2 Send Slope Credit Value Register	0x00000000
0x31041270	EMAC0_DMA2_CHHIC	EMAC0 Channel 2 High Credit Value Register	0x00000000
0x31041274	EMAC0_DMA2_CHLOC	EMAC0 Channel 2 Low Credit Value Register	0x00000000

Table A-83: ADSP-2159x EMAC1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31042000	EMAC1_MACCFG	EMAC1 MAC Configuration Register	0x00000000
0x31042004	EMAC1_MACFRMFILT	EMAC1 MAC Rx Frame Filter Register	0x00000000
0x31042008	EMAC1_HASHTBL_HI	EMAC1 Hash Table High Register	0x00000000

Table A-83: ADSP-2159x EMAC1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3104200C	EMAC1_HASHTBL_LO	EMAC1 Hash Table Low Register	0x00000000
0x31042010	EMAC1_SMI_ADDR	EMAC1 SMI Address Register	0x00000000
0x31042014	EMAC1_SMI_DATA	EMAC1 SMI Data Register	0x00000000
0x31042018	EMAC1_FLOWCTL	EMAC1 FLOW Control Register	0x00000000
0x3104201C	EMAC1_VLANTAG	EMAC1 VLAN Tag Register	0x00000000
0x31042024	EMAC1_DBG	EMAC1 Debug Register	0x00000000
0x31042038	EMAC1_ISTAT	EMAC1 Interrupt Status Register	0x00000000
0x3104203C	EMAC1_IMSK	EMAC1 Interrupt Mask Register	0x00000000
0x31042040	EMAC1_ADDR0_HI	EMAC1 MAC Address 0 High Register	0x8000FFFF
0x31042044	EMAC1_ADDR0_LO	EMAC1 MAC Address 0 Low Register	0xFFFFFFFF
0x31042048	EMAC1_ADDR1_HI	EMAC1 MAC Address 1 High Register	0x00000000
0x3104204C	EMAC1_ADDR1_LO	EMAC1 MAC Address 1 Low Register	0x00000000
0x310420DC	EMAC1_WDOG_TIMEOUT	EMAC1 Watchdog Timeout Register	0x00000000
0x31042100	EMAC1_MMC_CTL	EMAC1 MMC Control Register	0x00000000
0x31042104	EMAC1_MMC_RXINT	EMAC1 MMC Rx Interrupt Register	0x00000000
0x31042108	EMAC1_MMC_TXINT	EMAC1 MMC Tx Interrupt Register	0x00000000
0x3104210C	EMAC1_MMC_RXIMSK	EMAC1 MMC Rx Interrupt Mask Register	0x00000000
0x31042110	EMAC1_MMC_TXIMSK	EMAC1 MMC TX Interrupt Mask Register	0x00000000
0x31042114	EMAC1_TXOCTCNT_GB	EMAC1 Tx OCT Count (Good/Bad) Register	0x00000000
0x31042118	EMAC1_TXFRMCNT_GB	EMAC1 Tx Frame Count (Good/Bad) Register	0x00000000
0x3104211C	EMAC1_TXBCASTFRM_G	EMAC1 Tx Broadcast Frames (Good) Register	0x00000000
0x31042120	EMAC1_TXMCASTFRM_G	EMAC1 Tx Multicast Frames (Good) Register	0x00000000
0x31042124	EMAC1_TX64_GB	EMAC1 Tx 64-Byte Frames (Good/Bad) Register	0x00000000
0x31042128	EMAC1_TX65TO127_GB	EMAC1 Tx 65- to 127-Byte Frames (Good/Bad) Register	0x00000000
0x3104212C	EMAC1_TX128TO255_GB	EMAC1 Tx 128- to 255-Byte Frames (Good/Bad) Register	0x00000000
0x31042130	EMAC1_TX256TO511_GB	EMAC1 Tx 256- to 511-Byte Frames (Good/Bad) Register	0x00000000
0x31042134	EMAC1_TX512TO1023_GB	EMAC1 Tx 512- to 1023-Byte Frames (Good/Bad) Register	0x00000000
0x31042138	EMAC1_TX1024TOMAX_GB	EMAC1 Tx 1024- to Max-Byte Frames (Good/Bad) Register	0x00000000

Table A-83: ADSP-2159x EMAC1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3104213C	EMAC1_TXU- CASTFRM_GB	EMAC1 Tx Unicast Frames (Good/Bad) Register	0x00000000
0x31042140	EMAC1_TXMCASTFRM_G B	EMAC1 Tx Multicast Frames (Good/Bad) Register	0x00000000
0x31042144	EMAC1_TXBCASTFRM_G B	EMAC1 Tx Broadcast Frames (Good/Bad) Register	0x00000000
0x31042148	EMAC1_TXUNDR_ERR	EMAC1 Tx Underflow Error Register	0x00000000
0x3104214C	EMAC1_TXSNGCOL_G	EMAC1 Tx Single Collision (Good) Register	0x00000000
0x31042150	EMAC1_TXMULTCOL_G	EMAC1 Tx Multiple Collision (Good) Register	0x00000000
0x31042154	EMAC1_TXDEFERRED	EMAC1 Tx Deferred Register	0x00000000
0x31042158	EMAC1_TXLATECOL	EMAC1 Tx Late Collision Register	0x00000000
0x3104215C	EMAC1_TXEXCESSCOL	EMAC1 Tx Excess Collision Register	0x00000000
0x31042160	EMAC1_TXCARR_ERR	EMAC1 Tx Carrier Error Register	0x00000000
0x31042164	EMAC1_TXOCTCNT_G	EMAC1 Tx Octet Count (Good) Register	0x00000000
0x31042168	EMAC1_TXFRMCNT_G	EMAC1 Tx Frame Count (Good) Register	0x00000000
0x3104216C	EMAC1_TXEXCESSDEF	EMAC1 Tx Excess Deferral Register	0x00000000
0x31042170	EMAC1_TXPAUSEFRM	EMAC1 Tx Pause Frame Register	0x00000000
0x31042174	EMAC1_TXVLANFRM_G	EMAC1 Tx VLAN Frames (Good) Register	0x00000000
0x31042178	EMAC1_TXOVRSIZE_G	EMAC1 Number of Tx Frames (Good) greater than max-size	0x00000000
0x31042180	EMAC1_RXFRMCNT_GB	EMAC1 Rx Frame Count (Good/Bad) Register	0x00000000
0x31042184	EMAC1_RXOCTCNT_GB	EMAC1 Rx Octet Count (Good/Bad) Register	0x00000000
0x31042188	EMAC1_RXOCTCNT_G	EMAC1 Rx Octet Count (Good) Register	0x00000000
0x3104218C	EMAC1_RXBCASTFRM_G	EMAC1 Rx Broadcast Frames (Good) Register	0x00000000
0x31042190	EMAC1_RXMCASTFRM_G	EMAC1 Rx Multicast Frames (Good) Register	0x00000000
0x31042194	EMAC1_RXCRC_ERR	EMAC1 Rx CRC Error Register	0x00000000
0x31042198	EMAC1_RXALIGN_ERR	EMAC1 Rx alignment Error Register	0x00000000
0x3104219C	EMAC1_RXRUNT_ERR	EMAC1 Rx Runt Error Register	0x00000000
0x310421A0	EMAC1_RXJAB_ERR	EMAC1 Rx Jab Error Register	0x00000000
0x310421A4	EMAC1_RXUSIZE_G	EMAC1 Rx Undersize (Good) Register	0x00000000
0x310421A8	EMAC1_RXOSIZE_G	EMAC1 Rx Oversize (Good) Register	0x00000000
0x310421AC	EMAC1_RX64_GB	EMAC1 Rx 64-Byte Frames (Good/Bad) Register	0x00000000

Table A-83: ADSP-2159x EMAC1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310421B0	EMAC1_RX65TO127_GB	EMAC1 Rx 65- to 127-Byte Frames (Good/Bad) Register	0x00000000
0x310421B4	EMAC1_RX128TO255_GB	EMAC1 Rx 128- to 255-Byte Frames (Good/Bad) Register	0x00000000
0x310421B8	EMAC1_RX256TO511_GB	EMAC1 Rx 256- to 511-Byte Frames (Good/Bad) Register	0x00000000
0x310421BC	EMAC1_RX512TO1023_GB	EMAC1 Rx 512- to 1023-Byte Frames (Good/Bad) Register	0x00000000
0x310421C0	EMAC1_RX1024TOM- AX_GB	EMAC1 Rx 1024- to Max-Byte Frames (Good/Bad) Register	0x00000000
0x310421C4	EMAC1_RXUCASTFRM_G	EMAC1 Rx Unicast Frames (Good) Register	0x00000000
0x310421C8	EMAC1_RXLEN_ERR	EMAC1 Rx Length Error Register	0x00000000
0x310421CC	EMAC1_RXOORTYPE	EMAC1 Rx Out Of Range Type Register	0x00000000
0x310421D0	EMAC1_RXPAUSEFRM	EMAC1 Rx Pause Frames Register	0x00000000
0x310421D4	EMAC1_RXFIFO_OVF	EMAC1 Rx FIFO Overflow Register	0x00000000
0x310421D8	EMAC1_RXVLANFRM_GB	EMAC1 Rx VLAN Frames (Good/Bad) Register	0x00000000
0x310421DC	EMAC1_RXWDOG_ERR	EMAC1 Rx Watch Dog Error Register	0x00000000
0x310421E0	EMAC1_RXRCV_ERR	EMAC1 Rx Error Frames Received Register	0x00000000
0x310421E4	EMAC1_RXCTLFRM_G	EMAC1 Rx Good Control Frames Register	0x00000000
0x31042200	EMAC1_IPC_RXIMSK	EMAC1 MMC IPC Rx Interrupt Mask Register	0x00000000
0x31042208	EMAC1_IPC_RXINT	EMAC1 MMC IPC Rx Interrupt Register	0x00000000
0x31042210	EMAC1_RXIPV4_GD_FRM	EMAC1 Rx IPv4 Datagrams (Good) Register	0x00000000
0x31042214	EMAC1_RXIPV4_HDR_ER R_FRM	EMAC1 Rx IPv4 Datagrams Header Errors Register	0x00000000
0x31042218	EMAC1_RXIPV4_NO- PAY_FRM	EMAC1 Rx IPv4 Datagrams No Payload Frame Register	0x00000000
0x3104221C	EMAC1_RXIPV4_FRAG_FR M	EMAC1 Rx IPv4 Datagrams Fragmented Frames Register	0x00000000
0x31042220	EMAC1_RXIPV4_UDSBL_F RM	EMAC1 Rx IPv4 UDP Disabled Frames Register	0x00000000
0x31042224	EMAC1_RXIPV6_GD_FRM	EMAC1 Rx IPv6 Datagrams Good Frames Register	0x00000000
0x31042228	EMAC1_RXIPV6_HDR_ER R_FRM	EMAC1 Rx IPv6 Datagrams Header Error Frames Register	0x00000000
0x3104222C	EMAC1_RXIPV6_NO- PAY_FRM	EMAC1 Rx IPv6 Datagrams No Payload Frames Register	0x00000000

Table A-83: ADSP-2159x EMAC1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31042230	EMAC1_RXUDP_GD_FRM	EMAC1 Rx UDP Good Frames Register	0x00000000
0x31042234	EMAC1_RXUDP_ERR_FRM	EMAC1 Rx UDP Error Frames Register	0x00000000
0x31042238	EMAC1_RXTCP_GD_FRM	EMAC1 Rx TCP Good Frames Register	0x00000000
0x3104223C	EMAC1_RXTCP_ERR_FRM	EMAC1 Rx TCP Error Frames Register	0x00000000
0x31042240	EMAC1_RXICMP_GD_FRM	EMAC1 Rx ICMP Good Frames Register	0x00000000
0x31042244	EMAC1_RXICMP_ERR_FRM	EMAC1 Rx ICMP Error Frames Register	0x00000000
0x31042250	EMAC1_RXIPV4_GD_OCT	EMAC1 Rx IPv4 Datagrams Good Octets Register	0x00000000
0x31042254	EMAC1_RXIPV4_HDR_ERR_OCT	EMAC1 Rx IPv4 Datagrams Header Errors Register	0x00000000
0x31042258	EMAC1_RXIPV4_NO-PAY_OCT	EMAC1 Rx IPv4 Datagrams No Payload Octets Register	0x00000000
0x3104225C	EMAC1_RXIPV4_FRAG_OCT	EMAC1 Rx IPv4 Datagrams Fragmented Octets Register	0x00000000
0x31042260	EMAC1_RXIPV4_UDSBL_OCT	EMAC1 Rx IPv4 UDP Disabled Octets Register	0x00000000
0x31042264	EMAC1_RXIPV6_GD_OCT	EMAC1 Rx IPv6 Good Octets Register	0x00000000
0x31042268	EMAC1_RXIPV6_HDR_ERR_OCT	EMAC1 Rx IPv6 Header Errors Register	0x00000000
0x3104226C	EMAC1_RXIPV6_NO-PAY_OCT	EMAC1 Rx IPv6 No Payload Octets Register	0x00000000
0x31042270	EMAC1_RXUDP_GD_OCT	EMAC1 Rx UDP Good Octets Register	0x00000000
0x31042274	EMAC1_RXUDP_ERR_OCT	EMAC1 Rx UDP Error Octets Register	0x00000000
0x31042278	EMAC1_RXTCP_GD_OCT	EMAC1 Rx TCP Good Octets Register	0x00000000
0x3104227C	EMAC1_RXTCP_ERR_OCT	EMAC1 Rx TCP Error Octets Register	0x00000000
0x31042280	EMAC1_RXICMP_GD_OCT	EMAC1 Rx ICMP Good Octets Register	0x00000000
0x31042284	EMAC1_RXICMP_ERR_OCT	EMAC1 Rx ICMP Error Octets Register	0x00000000
0x31042400	EMAC1_L3L4_CTL	EMAC1 Layer3 and Layer4 Control Register	0x00000000
0x31042404	EMAC1_L4_ADDR	EMAC1 Layer 4 Address Register	0x00000000

Table A-83: ADSP-2159x EMAC1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31042410	EMAC1_L3_ADDR0	EMAC1 Layer 3 Address0 Register	0x00000000
0x31042414	EMAC1_L3_ADDR1	EMAC1 Layer 3 Address1 Register	0x00000000
0x31042418	EMAC1_L3_ADDR2	EMAC1 Layer 3 Address2 Register	0x00000000
0x3104241C	EMAC1_L3_ADDR3	EMAC1 Layer 3 Address3 Register	0x00000000
0x31042584	EMAC1_VLAN_INCL	EMAC1 VLAN Tag Inclusion or Replacement Register	0x00000000
0x31042588	EMAC1_VLAN_HSHTBL	EMAC1 VLAN Hash Table Register	0x00000000
0x31042760	EMAC1_TM_PPS0INTVL	EMAC1 Time Stamp PPS Interval Register	0x00000000
0x31042764	EMAC1_TM_PPS0WIDTH	EMAC1 PPS Width Register	0x00000000
0x31043000	EMAC1_DMA0_BUS- MODE	EMAC1 DMA Bus Mode Register	0x00020101
0x31043004	EMAC1_DMA0_TXPOLL	EMAC1 DMA Tx Poll Demand Register	0x00000000
0x31043008	EMAC1_DMA0_RXPOLL	EMAC1 DMA Rx Poll Demand register	0x00000000
0x3104300C	EMAC1_DMA0_RXDSC_A DDR	EMAC1 DMA Rx Descriptor List Address Register	0x00000000
0x31043010	EMAC1_DMA0_TXDSC_A DDR	EMAC1 DMA Tx Descriptor List Address Register	0x00000000
0x31043014	EMAC1_DMA0_STAT	EMAC1 DMA Status Register	0x00000000
0x31043018	EMAC1_DMA0_OPMODE	EMAC1 DMA Operation Mode Register	0x00000000
0x3104301C	EMAC1_DMA0_IEN	EMAC1 DMA Interrupt Enable Register	0x00000000
0x31043020	EMAC1_DMA0_MISS_FRM	EMAC1 DMA Missed Frame Register	0x00000000
0x31043024	EMAC1_DMA0_RXIW- DOG	EMAC1 DMA Rx Interrupt Watch Dog Register	0x00000000
0x31043028	EMAC1_DMA0_BMMODE	EMAC1 DMA SCB Bus Mode Register	0x40110001
0x3104302C	EMAC1_DMA0_BMSTAT	EMAC1 DMA SCB Status Register	0x00000000
0x31043048	EMAC1_DMA0_TXDSC_C UR	EMAC1 DMA Tx Descriptor Current Register	0x00000000
0x3104304C	EMAC1_DMA0_RXDSC_C UR	EMAC1 DMA Rx Descriptor Current Register	0x00000000
0x31043050	EMAC1_DMA0_TXBUF_C UR	EMAC1 DMA Tx Buffer Current Register	0x00000000
0x31043054	EMAC1_DMA0_RXBUF_C UR	EMAC1 DMA Rx Buffer Current Register	0x00000000

Table A-84: ADSP-2159x EMDMA0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310E002C	EMDMA0_CTL	EMDMA0 External Memory DMA Control Register	0x00000000
0x310E0080	EMDMA0_INDX1	EMDMA0 External Index Register	0x00000000
0x310E0084	EMDMA0_MOD1	EMDMA0 External Modifier Register	0x00000000
0x310E0088	EMDMA0_CNT1	EMDMA0 External Count Register	0x00000000
0x310E008C	EMDMA0_INDX0	EMDMA0 Internal Index Register	0x00000000
0x310E0090	EMDMA0_MOD0	EMDMA0 Internal Modifier Register	0x00000000
0x310E0094	EMDMA0_CNT0	EMDMA0 Internal Count Register	0x00000000
0x310E0098	EMDMA0_CHNPTR	EMDMA0 Chain Pointer Register	0x00000000
0x310E009C	EMDMA0_BASE	EMDMA0 External Base Address Register	0x00000000
0x310E00A0	EMDMA0_TPTR	EMDMA0 Tap List Pointer Register	0x00000000
0x310E00A4	EMDMA0_BUFLEN	EMDMA0 Circular Buffer Length Register	0x00000000
0x310E00AC	EMDMA0_TCNT	EMDMA0 Delay Line Tap Count Register	0x00000000

Table A-85: ADSP-2159x EMDMA1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310E0030	EMDMA1_CTL	EMDMA1 External Memory DMA Control Register	0x00000000
0x310E00C0	EMDMA1_INDX1	EMDMA1 External Index Register	0x00000000
0x310E00C4	EMDMA1_MOD1	EMDMA1 External Modifier Register	0x00000000
0x310E00C8	EMDMA1_CNT1	EMDMA1 External Count Register	0x00000000
0x310E00CC	EMDMA1_INDX0	EMDMA1 Internal Index Register	0x00000000
0x310E00D0	EMDMA1_MOD0	EMDMA1 Internal Modifier Register	0x00000000
0x310E00D4	EMDMA1_CNT0	EMDMA1 Internal Count Register	0x00000000
0x310E00D8	EMDMA1_CHNPTR	EMDMA1 Chain Pointer Register	0x00000000
0x310E00DC	EMDMA1_BASE	EMDMA1 External Base Address Register	0x00000000
0x310E00E0	EMDMA1_TPTR	EMDMA1 Tap List Pointer Register	0x00000000
0x310E00E4	EMDMA1_BUFLEN	EMDMA1 Circular Buffer Length Register	0x00000000
0x310E00EC	EMDMA1_TCNT	EMDMA1 Delay Line Tap Count Register	0x00000000

Table A-86: ADSP-2159x EPPI0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31029000	EPPI0_STAT	EPPI0 Status Register	0x00000000
0x31029004	EPPI0_HCNT	EPPI0 Horizontal Transfer Count Register	0x00000000
0x31029008	EPPI0_HDLY	EPPI0 Horizontal Delay Count Register	0x00000000
0x3102900C	EPPI0_VCNT	EPPI0 Vertical Transfer Count Register	0x00000000
0x31029010	EPPI0_VDLY	EPPI0 Vertical Delay Count Register	0x00000000
0x31029014	EPPI0_FRAME	EPPI0 Lines Per Frame Register	0x00000000
0x31029018	EPPI0_LINE	EPPI0 Samples Per Line Register	0x00000000
0x3102901C	EPPI0_CLKDIV	EPPI0 Clock Divide Register	0x00000000
0x31029020	EPPI0_CTL	EPPI0 Control Register	0x00000000
0x31029024	EPPI0_FS1_WLHB	EPPI0 FS1 Width Register / EPPI Horizontal Blanking Samples Per Line Register	0x00000000
0x31029028	EPPI0_FS1_PASPL	EPPI0 FS1 Period Register / EPPI Active Samples Per Line Register	0x00000000
0x3102902C	EPPI0_FS2_WLVB	EPPI0 FS2 Width Register / EPPI Lines Of Vertical Blanking Register	0x00000000
0x31029030	EPPI0_FS2_PALPF	EPPI0 FS2 Period Register / EPPI Active Lines Per Field Register	0x00000000
0x31029034	EPPI0_IMSK	EPPI0 Interrupt Mask Register	0x00000000
0x3102903C	EPPI0_ODDCLIP	EPPI0 Clipping Register for ODD (Chroma) Data Register	0xFFFF0000
0x31029040	EPPI0_EVENCLIP	EPPI0 Clipping Register for EVEN (Luma) Data Register	0xFFFF0000
0x31029044	EPPI0_FS1_DLY	EPPI0 Frame Sync 1 Delay Value Register	0x00000000
0x31029048	EPPI0_FS2_DLY	EPPI0 Frame Sync 2 Delay Value Register	0x00000000
0x3102904C	EPPI0_CTL2	EPPI0 Control Register 2 Register	0x00000000

Table A-87: ADSP-2159x FIR0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C3000	FIR0_CTL1	FIR0 FIR Global Control Register	0x00000000
0x310C3004	FIR0_DMASTAT	FIR0 FIR DMA Status Register	0x00000000
0x310C3008	FIR0_MACSTAT	FIR0 FIR MAC Status Register	0x00000000
0x310C3010	FIR0_DBG_CTL	FIR0 FIR Debug Control Register	0x00000000
0x310C3014	FIR0_DBG_ADDR	FIR0 Debug Address Register	0x00000000

Table A-87: ADSP-2159x FIR0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C3018	FIR0_DBG_WRDAT	FIR0 FIR Debug Data Write Register	0x00000000
0x310C301C	FIR0_DBG_RDDAT	FIR0 FIR Debug Data Read Register	0x00000000
0x310C3040	FIR0_CTL2	FIR0 FIR Channel Control Register	0x00000000
0x310C3044	FIR0_INIDX	FIR0 FIR Input Data Index Register	0x00000000
0x310C3048	FIR0_INMOD	FIR0 FIR Input Data Modifier Register	0x00000000
0x310C304C	FIR0_INCNT	FIR0 FIR Input Data Count Register	0x00000000
0x310C3050	FIR0_INBASE	FIR0 FIR Input Data Base Register	0x00000000
0x310C3054	FIR0_OUTIDX	FIR0 FIR Output Data Index Register	0x00000000
0x310C3058	FIR0_OUTMOD	FIR0 FIR Output Data Modifier Register	0x00000000
0x310C305C	FIR0_OUTCNT	FIR0 FIR Output Data Count Register	0x00000000
0x310C3060	FIR0_OUTBASE	FIR0 FIR Output Data Base Register	0x00000000
0x310C3064	FIR0_COEFIDX	FIR0 FIR Coefficient Index Register	0x00000000
0x310C3068	FIR0_COEFMOD	FIR0 FIR Coefficient Modifier Register	0x00000000
0x310C306C	FIR0_COEFCNT	FIR0 FIR Coefficient Count Register	0x00000000
0x310C3070	FIR0_CHNPTR	FIR0 FIR Chain Pointer Register	0x00000000
0x310C3074	FIR0_SCTL1	FIR0 software control register 1	0x00000000
0x310C3078	FIR0_SCTL2	FIR0 software control register 2	0x00000000
0x310C307C	FIR0_SGCTL	FIR0 Secondary global control register	0x00000000

Table A-88: ADSP-2159x FIR1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310BD000	FIR1_CTL1	FIR1 FIR Global Control Register	0x00000000
0x310BD004	FIR1_DMASTAT	FIR1 FIR DMA Status Register	0x00000000
0x310BD008	FIR1_MACSTAT	FIR1 FIR MAC Status Register	0x00000000
0x310BD010	FIR1_DBG_CTL	FIR1 FIR Debug Control Register	0x00000000
0x310BD014	FIR1_DBG_ADDR	FIR1 Debug Address Register	0x00000000
0x310BD018	FIR1_DBG_WRDAT	FIR1 FIR Debug Data Write Register	0x00000000
0x310BD01C	FIR1_DBG_RDDAT	FIR1 FIR Debug Data Read Register	0x00000000
0x310BD040	FIR1_CTL2	FIR1 FIR Channel Control Register	0x00000000
0x310BD044	FIR1_INIDX	FIR1 FIR Input Data Index Register	0x00000000

Table A-88: ADSP-2159x FIR1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310BD048	FIR1_INMOD	FIR1 FIR Input Data Modifier Register	0x00000000
0x310BD04C	FIR1_INCNT	FIR1 FIR Input Data Count Register	0x00000000
0x310BD050	FIR1_INBASE	FIR1 FIR Input Data Base Register	0x00000000
0x310BD054	FIR1_OUTIDX	FIR1 FIR Output Data Index Register	0x00000000
0x310BD058	FIR1_OUTMOD	FIR1 FIR Output Data Modifier Register	0x00000000
0x310BD05C	FIR1_OUTCNT	FIR1 FIR Output Data Count Register	0x00000000
0x310BD060	FIR1_OUTBASE	FIR1 FIR Output Data Base Register	0x00000000
0x310BD064	FIR1_COEFIDX	FIR1 FIR Coefficient Index Register	0x00000000
0x310BD068	FIR1_COEFMOD	FIR1 FIR Coefficient Modifier Register	0x00000000
0x310BD06C	FIR1_COEFCNT	FIR1 FIR Coefficient Count Register	0x00000000
0x310BD070	FIR1_CHNPTR	FIR1 FIR Chain Pointer Register	0x00000000
0x310BD074	FIR1_SCTL1	FIR1 software control register 1	0x00000000
0x310BD078	FIR1_SCTL2	FIR1 software control register 2	0x00000000
0x310BD07C	FIR1_SGCTL	FIR1 Secondary global control register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2000	GICDST0_EN	GICDST0 GIC Port 0 Enable	0x00000000
0x310B2080	GICDST0_SGI_SECURITY	GICDST0 Software Generated Interrupt Security Register	0x00000000
0x310B2084	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B2088	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B208C	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B2090	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B2094	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B2098	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B209C	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B20A0	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B20A4	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B20A8	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B20AC	GICDST0_SPI_SECURITY[n]	GICDST0 Shared Peripheral Interrupt Security Register	0x00000000
0x310B2104	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B2108	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B210C	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B2110	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B2114	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B2118	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B211C	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B2120	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B2124	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B2128	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B212C	GICDST0_SPI_EN_SET[n]	GICDST0 Shared Peripheral Interrupt Enable Set Register	0x00000000
0x310B2184	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B2188	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B218C	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B2190	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B2194	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B2198	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B219C	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B21A0	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B21A4	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B21A8	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B21AC	GICDST0_SPI_EN_CLR[n]	GICDST0 Shared Peripheral Interrupt Enable Clear Register	0x00000000
0x310B2200	GICDST0_SGI_PND_SET	GICDST0 Software Generated Interrupt Pending Set Register	0x00000000
0x310B2204	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B2208	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B220C	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B2210	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B2214	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B2218	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B221C	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B2220	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B2224	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B2228	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B222C	GICDST0_SPI_PND_SET[n]]	GICDST0 Shared Peripheral Interrupt Pending Set Register	0x00000000
0x310B2280	GICDST0_SGI_PND_CLR	GICDST0 Software Generated Interrupt Clear-Pending Register	0x00000000
0x310B2284	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B2288	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B228C	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B2290	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B2294	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B2298	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B229C	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B22A0	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B22A4	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B22A8	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B22AC	GICDST0_SPI_PND_CLR[n]]	GICDST0 Shared Peripheral Interrupt Pending Clear Register	0x00000000
0x310B2300	GICDST0_SGI_ACTIVE	GICDST0 Software Generated Interrupt Active Register	0x00000000
0x310B2304	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B2308	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B230C	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B2310	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B2314	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B2318	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B231C	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B2320	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B2324	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B2328	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B232C	GICDST0_SPI_ACTIVE[n]	GICDST0 Shared Peripheral Interrupt Active Register	0x00000000
0x310B2400	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2401	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2402	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2403	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2404	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2405	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2406	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2407	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2408	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2409	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B240A	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B240B	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B240C	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B240D	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B240E	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B240F	GICDST0_SGI_PRIO[n]	GICDST0 Software Generated Interrupt Priority Register	0x00000000
0x310B2420	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2421	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2422	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2423	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2424	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2425	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2426	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2427	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2428	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2429	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B242A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B242B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B242C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B242D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B242E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B242F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2430	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2431	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2432	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2433	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2434	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2435	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2436	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2437	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2438	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2439	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B243A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B243B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B243C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B243D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B243E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B243F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2440	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2441	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2442	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2443	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2444	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2445	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2446	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2447	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2448	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2449	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B244A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B244B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B244C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B244D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B244E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B244F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2450	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2451	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2452	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2453	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2454	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2455	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2456	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2457	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2458	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2459	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B245A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B245B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B245C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B245D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B245E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B245F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2460	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2461	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2462	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2463	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2464	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2465	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2466	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2467	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2468	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2469	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B246A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B246B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B246C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B246D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B246E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B246F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2470	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2471	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2472	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2473	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2474	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2475	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2476	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2477	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2478	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2479	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B247A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B247B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B247C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B247D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B247E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B247F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2480	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2481	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2482	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2483	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2484	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2485	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2486	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2487	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2488	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2489	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B248A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B248B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B248C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B248D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B248E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B248F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2490	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2491	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2492	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2493	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2494	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2495	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2496	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2497	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2498	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2499	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B249A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B249B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B249C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B249D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B249E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B249F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A0	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A1	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A2	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A3	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A4	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A5	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A6	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A7	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A8	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24A9	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24AA	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24AB	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24AC	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24AD	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24AE	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B24AF	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B0	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B1	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B2	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B3	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B4	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B5	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B6	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B7	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B8	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24B9	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24BA	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24BB	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24BC	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24BD	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24BE	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24BF	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C0	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C1	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C2	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C3	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C4	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C5	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C6	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C7	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C8	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24C9	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24CA	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24CB	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24CC	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24CD	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B24CE	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24CF	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D0	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D1	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D2	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D3	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D4	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D5	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D6	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D7	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D8	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24D9	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24DA	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24DB	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24DC	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24DD	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24DE	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24DF	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E0	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E1	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E2	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E3	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E4	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E5	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E6	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E7	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E8	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24E9	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24EA	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24EB	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24EC	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B24ED	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24EE	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24EF	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F0	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F1	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F2	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F3	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F4	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F5	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F6	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F7	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F8	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24F9	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24FA	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24FB	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24FC	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24FD	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24FE	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B24FF	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2500	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2501	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2502	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2503	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2504	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2505	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2506	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2507	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2508	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2509	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B250A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B250B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B250C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B250D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B250E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B250F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2510	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2511	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2512	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2513	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2514	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2515	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2516	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2517	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2518	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2519	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B251A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B251B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B251C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B251D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B251E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B251F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2520	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2521	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2522	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2523	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2524	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2525	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2526	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2527	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2528	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2529	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B252A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B252B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B252C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B252D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B252E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B252F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2530	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2531	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2532	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2533	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2534	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2535	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2536	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2537	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2538	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2539	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B253A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B253B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B253C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B253D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B253E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B253F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2540	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2541	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2542	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2543	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2544	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2545	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2546	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2547	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2548	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2549	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B254A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B254B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B254C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B254D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B254E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B254F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2550	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2551	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2552	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2553	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2554	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2555	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2556	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2557	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2558	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2559	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B255A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B255B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B255C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B255D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B255E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B255F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2560	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2561	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2562	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2563	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2564	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2565	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2566	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2567	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2568	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2569	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B256A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B256B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B256C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B256D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B256E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B256F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2570	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2571	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2572	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2573	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2574	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2575	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2576	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2577	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2578	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2579	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B257A	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B257B	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B257C	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B257D	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B257E	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B257F	GICDST0_SPI_PRIO[n]	GICDST0 Shared Peripheral Interrupt Priority Register	0x00000000
0x310B2820	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2821	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2822	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2823	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2824	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2825	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2826	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2827	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2828	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2829	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B282A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B282B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B282C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B282D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B282E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B282F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2830	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2831	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2832	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2833	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2834	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2835	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2836	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2837	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2838	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2839	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B283A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B283B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B283C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B283D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B283E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B283F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2840	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2841	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2842	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2843	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2844	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2845	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2846	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2847	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2848	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2849	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B284A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B284B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B284C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B284D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B284E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B284F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2850	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2851	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2852	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2853	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2854	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2855	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2856	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2857	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2858	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2859	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B285A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B285B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B285C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B285D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B285E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B285F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2860	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2861	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2862	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2863	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2864	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2865	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2866	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2867	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2868	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2869	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B286A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B286B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B286C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B286D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B286E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B286F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2870	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2871	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2872	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2873	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2874	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2875	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2876	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2877	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2878	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2879	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B287A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B287B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B287C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B287D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B287E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B287F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2880	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2881	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2882	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2883	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2884	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2885	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2886	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2887	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2888	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2889	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B288A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B288B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B288C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B288D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B288E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B288F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2890	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2891	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2892	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2893	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2894	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2895	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2896	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2897	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2898	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2899	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B289A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B289B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B289C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B289D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B289E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B289F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A0	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A1	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B28A2	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A3	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A4	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A5	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A6	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A7	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A8	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28A9	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28AA	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28AB	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28AC	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28AD	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28AE	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28AF	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B0	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B1	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B2	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B3	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B28B4	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B5	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B6	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B7	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B8	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28B9	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28BA	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28BB	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28BC	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28BD	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28BE	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28BF	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C0	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C1	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C2	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C3	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C4	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C5	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B28C6	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C7	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C8	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28C9	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28CA	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28CB	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28CC	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28CD	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28CE	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28CF	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D0	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D1	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D2	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D3	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D4	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D5	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D6	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D7	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B28D8	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28D9	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28DA	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28DB	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28DC	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28DD	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28DE	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28DF	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E0	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E1	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E2	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E3	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E4	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E5	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E6	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E7	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E8	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28E9	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B28EA	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28EB	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28EC	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28ED	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28EE	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28EF	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F0	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F1	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F2	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F3	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F4	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F5	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F6	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F7	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F8	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28F9	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28FA	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28FB	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B28FC	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28FD	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28FE	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B28FF	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2900	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2901	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2902	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2903	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2904	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2905	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2906	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2907	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2908	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2909	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B290A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B290B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B290C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B290D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B290E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B290F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2910	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2911	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2912	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2913	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2914	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2915	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2916	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2917	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2918	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2919	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B291A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B291B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B291C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B291D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B291E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B291F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2920	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2921	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2922	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2923	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2924	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2925	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2926	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2927	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2928	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2929	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B292A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B292B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B292C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B292D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B292E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B292F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2930	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2931	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2932	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2933	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2934	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2935	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2936	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2937	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2938	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2939	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B293A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B293B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B293C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B293D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B293E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B293F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2940	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2941	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2942	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2943	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2944	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2945	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2946	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2947	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2948	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2949	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B294A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B294B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B294C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B294D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B294E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B294F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2950	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2951	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2952	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2953	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2954	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2955	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2956	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2957	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2958	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2959	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B295A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B295B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B295C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B295D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B295E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B295F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2960	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2961	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2962	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2963	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2964	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2965	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2966	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2967	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2968	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2969	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B296A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B296B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B296C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B296D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B296E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B296F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2970	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2971	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2972	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2973	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2974	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2975	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2976	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2977	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2978	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2979	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B297A	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B297B	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B297C	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B297D	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B297E	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B297F	GICDST0_SPI_TRGT[n]	GICDST0 Shared Peripheral Interrupt Processor Targets Register	0x00000000
0x310B2C08	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C0C	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C10	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C14	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C18	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C1C	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C20	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C24	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C28	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C2C	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C30	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C34	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555

Table A-89: ADSP-2159x GICDST0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B2C38	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C3C	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C40	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2C44	GICDST0_SPI_CFG[n]	GICDST0 Shared Peripheral Interrupt Configuration Register	0x55555555
0x310B2D04	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D08	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D0C	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D10	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D14	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D18	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D1C	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D20	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D24	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D28	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2D2C	GICDST0_SPI[n]	GICDST0 Shared Peripheral Interrupt Register	0x00000000
0x310B2F00	GICDST0_SGI_CTL	GICDST0 Software Generated Interrupt Control Register	0x00000000

Table A-90: ADSP-2159x GICCPU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310B4000	GICCPU0_CTL	GICCPU0 CPU Interface Control Register (ICCICR)	0x00000000
0x310B4004	GICCPU0_PRIO_MSK	GICCPU0 Priority Mask Register (ICCIPMR)	0x00000000
0x310B4008	GICCPU0_BIN_PT	GICCPU0 Binary Point Register (ICCBPR)	0x00000000
0x310B400C	GICCPU0_INT_ACK	GICCPU0 Interrupt Acknowledge Register (ICCIAR)	0x00000000
0x310B4010	GICCPU0_EOI	GICCPU0 End of Interrupt Register (ICCEOIR)	0x00000000
0x310B4014	GICCPU0_RUN_PRIO	GICCPU0 Running Priority Register (ICCRPR)	0x00000000
0x310B4018	GICCPU0_PND_HI	GICCPU0 Highest Pending Interrupt Register (ICCH-PIR)	0x00000000
0x310B401C	GICCPU0_BIN_PT_ALIAS	GICCPU0 Aliased Binary Point Register (ICCABPR)	0x00000000

Table A-91: ADSP-2159x HADC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31016000	HADC0_CTL	HADC0 Control Register	0x000001AA
0x31016004	HADC0_CHAN_MSK	HADC0 Channel Mask Register	0x0000FF00
0x31016008	HADC0_IMSK	HADC0 Interrupt Mask Register	0x00000000
0x3101600C	HADC0_STAT	HADC0 Status Register	0x00000000
0x31016010	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016014	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016018	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x3101601C	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016020	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016024	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016028	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x3101602C	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016030	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016034	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016038	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x3101603C	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016040	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016044	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x31016048	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000
0x3101604C	HADC0_DATA[nn]	HADC0 Channel Data Registers	0x00000000

Table A-92: ADSP-2159x IIR0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C4000	IIR0_CTL1	IIR0 Global Control Register	0x00000000
0x310C4004	IIR0_DMASTAT	IIR0 DMA Status Register	0x00000000
0x310C4008	IIR0_MACSTAT	IIR0 MAC Status Register	0x00000000
0x310C400C	IIR0_DBG_CTL	IIR0 IIR Debug Control Register	0x00000000
0x310C4010	IIR0_DBG_ADDR	IIR0 IIR Debug Address Register	0x00000000
0x310C4014	IIR0_DBG_WRDAT_LO	IIR0 IIR Debug Write Data Low Register	0x00000000
0x310C4018	IIR0_DBG_WRDAT_HI	IIR0 IIR Debug Write Data High Register	0x00000000

Table A-92: ADSP-2159x IIR0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C401C	IIR0_DBG_RDDAT_LO	IIR0 IIR Debug Read Data Low Register	0x00000000
0x310C4020	IIR0_DBG_RDDAT_HI	IIR0 IIR Debug Read Data High Register	0x00000000
0x310C4040	IIR0_CTL2	IIR0 Channel Control Register	0x00000000
0x310C4044	IIR0_INIDX	IIR0 Input Data Index Register	0x00000000
0x310C4048	IIR0_INMOD	IIR0 Input Data Index Modifier Register	0x00000000
0x310C404C	IIR0_INLEN	IIR0 Input Data Buffer Length Register	0x00000000
0x310C4050	IIR0_INBASE	IIR0 Input Buffer Base Register	0x00000000
0x310C4054	IIR0_OUTIDX	IIR0 Output Data Buffer Index Register	0x00000000
0x310C4058	IIR0_OUTMOD	IIR0 IIR Output Data Index Modifier Register	0x00000000
0x310C405C	IIR0_OUTLEN	IIR0 IIR Output Data Buffer Length Register	0x00000000
0x310C4060	IIR0_OUTBASE	IIR0 Output Buffer Base Register	0x00000000
0x310C4064	IIR0_COEFIDX	IIR0 Coefficient Buffer Index Register	0x00000000
0x310C4068	IIR0_COEFMOD	IIR0 Coefficient Index Modifier Register	0x00000000
0x310C406C	IIR0_COEFLEN	IIR0 Coefficient Buffer Length Register	0x00000000
0x310C4070	IIR0_CHNPTR	IIR0 Chain Pointer Register	0x00000000
0x310C4074	IIR0_SCTL1	IIR0 Software Control Register1	0x00000000
0x310C4078	IIR0_SCTL2	IIR0 Software Control Register2	0x00000000
0x310C407C	IIR0_SGCTL	IIR0 Secondary Global Control Register	0x00000000

Table A-93: ADSP-2159x IIR1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C0000	IIR1_CTL1	IIR1 Global Control Register	0x00000000
0x310C0004	IIR1_DMASTAT	IIR1 DMA Status Register	0x00000000
0x310C0008	IIR1_MACSTAT	IIR1 MAC Status Register	0x00000000
0x310C000C	IIR1_DBG_CTL	IIR1 IIR Debug Control Register	0x00000000
0x310C0010	IIR1_DBG_ADDR	IIR1 IIR Debug Address Register	0x00000000
0x310C0014	IIR1_DBG_WRDAT_LO	IIR1 IIR Debug Write Data Low Register	0x00000000
0x310C0018	IIR1_DBG_WRDAT_HI	IIR1 IIR Debug Write Data High Register	0x00000000
0x310C001C	IIR1_DBG_RDDAT_LO	IIR1 IIR Debug Read Data Low Register	0x00000000
0x310C0020	IIR1_DBG_RDDAT_HI	IIR1 IIR Debug Read Data High Register	0x00000000

Table A-93: ADSP-2159x IIR1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C0040	IIR1_CTL2	IIR1 Channel Control Register	0x00000000
0x310C0044	IIR1_INIDX	IIR1 Input Data Index Register	0x00000000
0x310C0048	IIR1_INMOD	IIR1 Input Data Index Modifier Register	0x00000000
0x310C004C	IIR1_INLEN	IIR1 Input Data Buffer Length Register	0x00000000
0x310C0050	IIR1_INBASE	IIR1 Input Buffer Base Register	0x00000000
0x310C0054	IIR1_OUTIDX	IIR1 Output Data Buffer Index Register	0x00000000
0x310C0058	IIR1_OUTMOD	IIR1 IIR Output Data Index Modifier Register	0x00000000
0x310C005C	IIR1_OUTLEN	IIR1 IIR Output Data Buffer Length Register	0x00000000
0x310C0060	IIR1_OUTBASE	IIR1 Output Buffer Base Register	0x00000000
0x310C0064	IIR1_COEFIDX	IIR1 Coefficient Buffer Index Register	0x00000000
0x310C0068	IIR1_COEFMOD	IIR1 Coefficient Index Modifier Register	0x00000000
0x310C006C	IIR1_COEFLEN	IIR1 Coefficient Buffer Length Register	0x00000000
0x310C0070	IIR1_CHNPTR	IIR1 Chain Pointer Register	0x00000000
0x310C0074	IIR1_SCTL1	IIR1 Software Control Register1	0x00000000
0x310C0078	IIR1_SCTL2	IIR1 Software Control Register2	0x00000000
0x310C007C	IIR1_SGCTL	IIR1 Secondary Global Control Register	0x00000000

Table A-94: ADSP-2159x IIR2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C1000	IIR2_CTL1	IIR2 Global Control Register	0x00000000
0x310C1004	IIR2_DMASTAT	IIR2 DMA Status Register	0x00000000
0x310C1008	IIR2_MACSTAT	IIR2 MAC Status Register	0x00000000
0x310C100C	IIR2_DBG_CTL	IIR2 IIR Debug Control Register	0x00000000
0x310C1010	IIR2_DBG_ADDR	IIR2 IIR Debug Address Register	0x00000000
0x310C1014	IIR2_DBG_WRDAT_LO	IIR2 IIR Debug Write Data Low Register	0x00000000
0x310C1018	IIR2_DBG_WRDAT_HI	IIR2 IIR Debug Write Data High Register	0x00000000
0x310C101C	IIR2_DBG_RDDAT_LO	IIR2 IIR Debug Read Data Low Register	0x00000000
0x310C1020	IIR2_DBG_RDDAT_HI	IIR2 IIR Debug Read Data High Register	0x00000000
0x310C1040	IIR2_CTL2	IIR2 Channel Control Register	0x00000000
0x310C1044	IIR2_INIDX	IIR2 Input Data Index Register	0x00000000

Table A-94: ADSP-2159x IIR2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C1048	IIR2_INMOD	IIR2 Input Data Index Modifier Register	0x00000000
0x310C104C	IIR2_INLEN	IIR2 Input Data Buffer Length Register	0x00000000
0x310C1050	IIR2_INBASE	IIR2 Input Buffer Base Register	0x00000000
0x310C1054	IIR2_OUTIDX	IIR2 Output Data Buffer Index Register	0x00000000
0x310C1058	IIR2_OUTMOD	IIR2 IIR Output Data Index Modifier Register	0x00000000
0x310C105C	IIR2_OUTLEN	IIR2 IIR Output Data Buffer Length Register	0x00000000
0x310C1060	IIR2_OUTBASE	IIR2 Output Buffer Base Register	0x00000000
0x310C1064	IIR2_COEFIDX	IIR2 Coefficient Buffer Index Register	0x00000000
0x310C1068	IIR2_COEFMOD	IIR2 Coefficient Index Modifier Register	0x00000000
0x310C106C	IIR2_COEFLEN	IIR2 Coefficient Buffer Length Register	0x00000000
0x310C1070	IIR2_CHNPTR	IIR2 Chain Pointer Register	0x00000000
0x310C1074	IIR2_SCTL1	IIR2 Software Control Register1	0x00000000
0x310C1078	IIR2_SCTL2	IIR2 Software Control Register2	0x00000000
0x310C107C	IIR2_SGCTL	IIR2 Secondary Global Control Register	0x00000000

Table A-95: ADSP-2159x IIR3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C2000	IIR3_CTL1	IIR3 Global Control Register	0x00000000
0x310C2004	IIR3_DMASTAT	IIR3 DMA Status Register	0x00000000
0x310C2008	IIR3_MACSTAT	IIR3 MAC Status Register	0x00000000
0x310C200C	IIR3_DBG_CTL	IIR3 IIR Debug Control Register	0x00000000
0x310C2010	IIR3_DBG_ADDR	IIR3 IIR Debug Address Register	0x00000000
0x310C2014	IIR3_DBG_WRDAT_LO	IIR3 IIR Debug Write Data Low Register	0x00000000
0x310C2018	IIR3_DBG_WRDAT_HI	IIR3 IIR Debug Write Data High Register	0x00000000
0x310C201C	IIR3_DBG_RDDAT_LO	IIR3 IIR Debug Read Data Low Register	0x00000000
0x310C2020	IIR3_DBG_RDDAT_HI	IIR3 IIR Debug Read Data High Register	0x00000000
0x310C2040	IIR3_CTL2	IIR3 Channel Control Register	0x00000000
0x310C2044	IIR3_INIDX	IIR3 Input Data Index Register	0x00000000
0x310C2048	IIR3_INMOD	IIR3 Input Data Index Modifier Register	0x00000000
0x310C204C	IIR3_INLEN	IIR3 Input Data Buffer Length Register	0x00000000

Table A-95: ADSP-2159x IIR3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C2050	IIR3_INBASE	IIR3 Input Buffer Base Register	0x00000000
0x310C2054	IIR3_OUTIDX	IIR3 Output Data Buffer Index Register	0x00000000
0x310C2058	IIR3_OUTMOD	IIR3 IIR Output Data Index Modifier Register	0x00000000
0x310C205C	IIR3_OUTLEN	IIR3 IIR Output Data Buffer Length Register	0x00000000
0x310C2060	IIR3_OUTBASE	IIR3 Output Buffer Base Register	0x00000000
0x310C2064	IIR3_COEFIDX	IIR3 Coefficient Buffer Index Register	0x00000000
0x310C2068	IIR3_COEFMOD	IIR3 Coefficient Index Modifier Register	0x00000000
0x310C206C	IIR3_COEFLEN	IIR3 Coefficient Buffer Length Register	0x00000000
0x310C2070	IIR3_CHNPTR	IIR3 Chain Pointer Register	0x00000000
0x310C2074	IIR3_SCTL1	IIR3 Software Control Register1	0x00000000
0x310C2078	IIR3_SCTL2	IIR3 Software Control Register2	0x00000000
0x310C207C	IIR3_SGCTL	IIR3 Secondary Global Control Register	0x00000000

Table A-96: ADSP-2159x IIR4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310BE000	IIR4_CTL1	IIR4 Global Control Register	0x00000000
0x310BE004	IIR4_DMASTAT	IIR4 DMA Status Register	0x00000000
0x310BE008	IIR4_MACSTAT	IIR4 MAC Status Register	0x00000000
0x310BE00C	IIR4_DBG_CTL	IIR4 IIR Debug Control Register	0x00000000
0x310BE010	IIR4_DBG_ADDR	IIR4 IIR Debug Address Register	0x00000000
0x310BE014	IIR4_DBG_WRDAT_LO	IIR4 IIR Debug Write Data Low Register	0x00000000
0x310BE018	IIR4_DBG_WRDAT_HI	IIR4 IIR Debug Write Data High Register	0x00000000
0x310BE01C	IIR4_DBG_RDDAT_LO	IIR4 IIR Debug Read Data Low Register	0x00000000
0x310BE020	IIR4_DBG_RDDAT_HI	IIR4 IIR Debug Read Data High Register	0x00000000
0x310BE040	IIR4_CTL2	IIR4 Channel Control Register	0x00000000
0x310BE044	IIR4_INIDX	IIR4 Input Data Index Register	0x00000000
0x310BE048	IIR4_INMOD	IIR4 Input Data Index Modifier Register	0x00000000
0x310BE04C	IIR4_INLEN	IIR4 Input Data Buffer Length Register	0x00000000
0x310BE050	IIR4_INBASE	IIR4 Input Buffer Base Register	0x00000000
0x310BE054	IIR4_OUTIDX	IIR4 Output Data Buffer Index Register	0x00000000

Table A-96: ADSP-2159x IIR4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310BE058	IIR4_OUTMOD	IIR4 IIR Output Data Index Modifier Register	0x00000000
0x310BE05C	IIR4_OUTLEN	IIR4 IIR Output Data Buffer Length Register	0x00000000
0x310BE060	IIR4_OUTBASE	IIR4 Output Buffer Base Register	0x00000000
0x310BE064	IIR4_COEFIDX	IIR4 Coefficient Buffer Index Register	0x00000000
0x310BE068	IIR4_COEFMOD	IIR4 Coefficient Index Modifier Register	0x00000000
0x310BE06C	IIR4_COEFLEN	IIR4 Coefficient Buffer Length Register	0x00000000
0x310BE070	IIR4_CHNPTR	IIR4 Chain Pointer Register	0x00000000
0x310BE074	IIR4_SCTL1	IIR4 Software Control Register1	0x00000000
0x310BE078	IIR4_SCTL2	IIR4 Software Control Register2	0x00000000
0x310BE07C	IIR4_SGCTL	IIR4 Secondary Global Control Register	0x00000000

Table A-97: ADSP-2159x IIR5 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310BA000	IIR5_CTL1	IIR5 Global Control Register	0x00000000
0x310BA004	IIR5_DMASTAT	IIR5 DMA Status Register	0x00000000
0x310BA008	IIR5_MACSTAT	IIR5 MAC Status Register	0x00000000
0x310BA00C	IIR5_DBG_CTL	IIR5 IIR Debug Control Register	0x00000000
0x310BA010	IIR5_DBG_ADDR	IIR5 IIR Debug Address Register	0x00000000
0x310BA014	IIR5_DBG_WRDAT_LO	IIR5 IIR Debug Write Data Low Register	0x00000000
0x310BA018	IIR5_DBG_WRDAT_HI	IIR5 IIR Debug Write Data High Register	0x00000000
0x310BA01C	IIR5_DBG_RDDAT_LO	IIR5 IIR Debug Read Data Low Register	0x00000000
0x310BA020	IIR5_DBG_RDDAT_HI	IIR5 IIR Debug Read Data High Register	0x00000000
0x310BA040	IIR5_CTL2	IIR5 Channel Control Register	0x00000000
0x310BA044	IIR5_INIDX	IIR5 Input Data Index Register	0x00000000
0x310BA048	IIR5_INMOD	IIR5 Input Data Index Modifier Register	0x00000000
0x310BA04C	IIR5_INLEN	IIR5 Input Data Buffer Length Register	0x00000000
0x310BA050	IIR5_INBASE	IIR5 Input Buffer Base Register	0x00000000
0x310BA054	IIR5_OUTIDX	IIR5 Output Data Buffer Index Register	0x00000000
0x310BA058	IIR5_OUTMOD	IIR5 IIR Output Data Index Modifier Register	0x00000000
0x310BA05C	IIR5_OUTLEN	IIR5 IIR Output Data Buffer Length Register	0x00000000

Table A-97: ADSP-2159x IIR5 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310BA060	IIR5_OUTBASE	IIR5 Output Buffer Base Register	0x00000000
0x310BA064	IIR5_COEFIDX	IIR5 Coefficient Buffer Index Register	0x00000000
0x310BA068	IIR5_COEFMOD	IIR5 Coefficient Index Modifier Register	0x00000000
0x310BA06C	IIR5_COEFLEN	IIR5 Coefficient Buffer Length Register	0x00000000
0x310BA070	IIR5_CHNPTR	IIR5 Chain Pointer Register	0x00000000
0x310BA074	IIR5_SCTL1	IIR5 Software Control Register1	0x00000000
0x310BA078	IIR5_SCTL2	IIR5 Software Control Register2	0x00000000
0x310BA07C	IIR5_SGCTL	IIR5 Secondary Global Control Register	0x00000000

Table A-98: ADSP-2159x IIR6 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310BB000	IIR6_CTL1	IIR6 Global Control Register	0x00000000
0x310BB004	IIR6_DMASTAT	IIR6 DMA Status Register	0x00000000
0x310BB008	IIR6_MACSTAT	IIR6 MAC Status Register	0x00000000
0x310BB00C	IIR6_DBG_CTL	IIR6 IIR Debug Control Register	0x00000000
0x310BB010	IIR6_DBG_ADDR	IIR6 IIR Debug Address Register	0x00000000
0x310BB014	IIR6_DBG_WRDAT_LO	IIR6 IIR Debug Write Data Low Register	0x00000000
0x310BB018	IIR6_DBG_WRDAT_HI	IIR6 IIR Debug Write Data High Register	0x00000000
0x310BB01C	IIR6_DBG_RDDAT_LO	IIR6 IIR Debug Read Data Low Register	0x00000000
0x310BB020	IIR6_DBG_RDDAT_HI	IIR6 IIR Debug Read Data High Register	0x00000000
0x310BB040	IIR6_CTL2	IIR6 Channel Control Register	0x00000000
0x310BB044	IIR6_INIDX	IIR6 Input Data Index Register	0x00000000
0x310BB048	IIR6_INMOD	IIR6 Input Data Index Modifier Register	0x00000000
0x310BB04C	IIR6_INLEN	IIR6 Input Data Buffer Length Register	0x00000000
0x310BB050	IIR6_INBASE	IIR6 Input Buffer Base Register	0x00000000
0x310BB054	IIR6_OUTIDX	IIR6 Output Data Buffer Index Register	0x00000000
0x310BB058	IIR6_OUTMOD	IIR6 IIR Output Data Index Modifier Register	0x00000000
0x310BB05C	IIR6_OUTLEN	IIR6 IIR Output Data Buffer Length Register	0x00000000
0x310BB060	IIR6_OUTBASE	IIR6 Output Buffer Base Register	0x00000000
0x310BB064	IIR6_COEFIDX	IIR6 Coefficient Buffer Index Register	0x00000000

Table A-98: ADSP-2159x IIR6 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310BB068	IIR6_COEFMOD	IIR6 Coefficient Index Modifier Register	0x00000000
0x310BB06C	IIR6_COEFLEN	IIR6 Coefficient Buffer Length Register	0x00000000
0x310BB070	IIR6_CHNPTR	IIR6 Chain Pointer Register	0x00000000
0x310BB074	IIR6_SCTL1	IIR6 Software Control Register1	0x00000000
0x310BB078	IIR6_SCTL2	IIR6 Software Control Register2	0x00000000
0x310BB07C	IIR6_SGCTL	IIR6 Secondary Global Control Register	0x00000000

Table A-99: ADSP-2159x IIR7 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310BC000	IIR7_CTL1	IIR7 Global Control Register	0x00000000
0x310BC004	IIR7_DMASTAT	IIR7 DMA Status Register	0x00000000
0x310BC008	IIR7_MACSTAT	IIR7 MAC Status Register	0x00000000
0x310BC00C	IIR7_DBG_CTL	IIR7 IIR Debug Control Register	0x00000000
0x310BC010	IIR7_DBG_ADDR	IIR7 IIR Debug Address Register	0x00000000
0x310BC014	IIR7_DBG_WRDAT_LO	IIR7 IIR Debug Write Data Low Register	0x00000000
0x310BC018	IIR7_DBG_WRDAT_HI	IIR7 IIR Debug Write Data High Register	0x00000000
0x310BC01C	IIR7_DBG_RDDAT_LO	IIR7 IIR Debug Read Data Low Register	0x00000000
0x310BC020	IIR7_DBG_RDDAT_HI	IIR7 IIR Debug Read Data High Register	0x00000000
0x310BC040	IIR7_CTL2	IIR7 Channel Control Register	0x00000000
0x310BC044	IIR7_INIDX	IIR7 Input Data Index Register	0x00000000
0x310BC048	IIR7_INMOD	IIR7 Input Data Index Modifier Register	0x00000000
0x310BC04C	IIR7_INLEN	IIR7 Input Data Buffer Length Register	0x00000000
0x310BC050	IIR7_INBASE	IIR7 Input Buffer Base Register	0x00000000
0x310BC054	IIR7_OUTIDX	IIR7 Output Data Buffer Index Register	0x00000000
0x310BC058	IIR7_OUTMOD	IIR7 IIR Output Data Index Modifier Register	0x00000000
0x310BC05C	IIR7_OUTLEN	IIR7 IIR Output Data Buffer Length Register	0x00000000
0x310BC060	IIR7_OUTBASE	IIR7 Output Buffer Base Register	0x00000000
0x310BC064	IIR7_COEFIDX	IIR7 Coefficient Buffer Index Register	0x00000000
0x310BC068	IIR7_COEFMOD	IIR7 Coefficient Index Modifier Register	0x00000000
0x310BC06C	IIR7_COEFLEN	IIR7 Coefficient Buffer Length Register	0x00000000

Table A-99: ADSP-2159x IIR7 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310BC070	IIR7_CHNPTR	IIR7 Chain Pointer Register	0x00000000
0x310BC074	IIR7_SCTL1	IIR7 Software Control Register1	0x00000000
0x310BC078	IIR7_SCTL2	IIR7 Software Control Register2	0x00000000
0x310BC07C	IIR7_SGCTL	IIR7 Secondary Global Control Register	0x00000000

Table A-100: ADSP-2159x L2CTL0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31080000	L2CTL0_CTL	L2CTL0 Control Register	0x00000000
0x31080010	L2CTL0_STAT	L2CTL0 Status Register	0x00000000
0x31080014	L2CTL0_RPCR0	L2CTL0 Read Priority Count Register	0x0F0F0F0F
0x31080018	L2CTL0_WPCR0	L2CTL0 Write Priority Count Register	0x0F0F0F0F
0x31080024	L2CTL0_INIT	L2CTL0 Initialization Register	0x00000000
0x31080038	L2CTL0_ISTAT	L2CTL0 Initialization Status Register	0x00000000
0x3108003C	L2CTL0_PCTL	L2CTL0 Power Control Register	0x00000000
0x31080040	L2CTL0_ERRADDR0	L2CTL0 ECC Error Address 0 Register	0x20000000
0x31080044	L2CTL0_ERRADDR1	L2CTL0 ECC Error Address 1 Register	0x20040000
0x31080048	L2CTL0_ERRADDR2	L2CTL0 ECC Error Address 2 Register	0x20080000
0x3108004C	L2CTL0_ERRADDR3	L2CTL0 ECC Error Address 3 Register	0x200C0000
0x31080050	L2CTL0_ERRADDR4	L2CTL0 ECC Error Address 4 Register	0x20100000
0x31080054	L2CTL0_ERRADDR5	L2CTL0 ECC Error Address 5 Register	0x20140000
0x31080058	L2CTL0_ERRADDR6	L2CTL0 ECC Error Address 6 Register	0x20180000
0x3108005C	L2CTL0_ERRADDR7	L2CTL0 ECC Error Address 7 Register	0x201C0000
0x31080060	L2CTL0_ERRADDR8	L2CTL0 ECC Error Address 8 Register	0x20200000
0x31080080	L2CTL0_ET0	L2CTL0 Error Type 0 Register	0x00000000
0x31080084	L2CTL0_EADDR0	L2CTL0 Error Type 0 Address Register	0x00000000
0x31080088	L2CTL0_ET1	L2CTL0 Error Type 1 Register	0x00000000
0x3108008C	L2CTL0_EADDR1	L2CTL0 Error Type 1 Address Register	0x00000000
0x310800EC	L2CTL0_SCTL	L2CTL0 Scrub Control Register	0x00000000
0x310800F0	L2CTL0_SADR	L2CTL0 Scrub Start Address Register	0x00000000
0x310800F4	L2CTL0_SCNT	L2CTL0 Scrub Count Register	0x00000000

Table A-100: ADSP-2159x L2CTL0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310800FC	L2CTL0_REVID	L2CTL0 Revision ID Register	0x00000003
0x31080100	L2CTL0_STAT_1	L2CTL0 L2 Port Error Status Register	0x00000000
0x31080104	L2CTL0_ET2	L2CTL0 Error Type 2 Register	0x00000000
0x31080108	L2CTL0_EADDR2	L2CTL0 Error Type 2 Address Register	0x00000000
0x3108010C	L2CTL0_ET3	L2CTL0 Error Type 3 Register	0x00000000
0x31080110	L2CTL0_EADDR3	L2CTL0 Error Type 3 Address Register	0x00000000
0x31080114	L2CTL0_ET4	L2CTL0 Error Type 4 Register	0x00000000
0x31080118	L2CTL0_EADDR4	L2CTL0 Error Type 4 Address Register	0x00000000
0x31080130	L2CTL0_RPCR1	L2CTL0 Read Priority Count Register	0x0000000F
0x31080134	L2CTL0_WPCR1	L2CTL0 Write Priority Count Register	0x0000000F

Table A-101: ADSP-2159x LP0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x30FFE000	LP0_CTL	LP0 Control Register	0x00000000
0x30FFE004	LP0_STAT	LP0 Status Register	0x00000000
0x30FFE008	LP0_DIV	LP0 Clock Divider Value Register	0x00000000
0x30FFE010	LP0_TX	LP0 Transmit Buffer Register	0x00000000
0x30FFE014	LP0_RX	LP0 Receive Buffer Register	0x00000000
0x30FFE018	LP0_TXIN_SHDW	LP0 Shadow Input Transmit Buffer Register	0x00000000
0x30FFE01C	LP0_TXOUT_SHDW	LP0 Shadow Output Transmit Buffer Register	0x00000000

Table A-102: ADSP-2159x LP1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x30FFE100	LP1_CTL	LP1 Control Register	0x00000000
0x30FFE104	LP1_STAT	LP1 Status Register	0x00000000
0x30FFE108	LP1_DIV	LP1 Clock Divider Value Register	0x00000000
0x30FFE110	LP1_TX	LP1 Transmit Buffer Register	0x00000000
0x30FFE114	LP1_RX	LP1 Receive Buffer Register	0x00000000
0x30FFE118	LP1_TXIN_SHDW	LP1 Shadow Input Transmit Buffer Register	0x00000000
0x30FFE11C	LP1_TXOUT_SHDW	LP1 Shadow Output Transmit Buffer Register	0x00000000

Table A-103: ADSP-2159x MEC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A2000	MEC0_PEIRQ_GCTL[p]	MEC0 Parity Error Interrupt Request Global Control Register	0x00000000
0x310A2010	MEC0_PEIRQ_GSTAT[p]	MEC0 Parity Error Interrupt Request Global Status Register	0x00000000
0x310A2040	MEC0_PERR_CTL0	MEC0 Parity Error Control Register	0x00000000
0x310A2044	MEC0_PERR_CTL1	MEC0 Parity Error Control Register	0x00000000
0x310A2080	MEC0_PERR_STAT0	MEC0 Parity Error Status Register	0x00000000
0x310A2084	MEC0_PERR_STAT1	MEC0 Parity Error Status Register	0x00000000
0x310A20C0	MEC0_PERR_IMASK0	MEC0 Parity Error Interrupt Mask Register	0x00000000
0x310A20C4	MEC0_PERR_IMASK1	MEC0 Parity Error Interrupt Mask Register	0x00000000
0x310A2100	MEC0_EEIRQ_GCTL[q]	MEC0 ECC Error Interrupt Request Global Control Register	0x00000000
0x310A2110	MEC0_EEIRQ_GSTAT[q]	MEC0 ECC Error Interrupt Request Global Status Register	0x00000000
0x310A2140	MEC0_ECCERR_CTL[y]	MEC0 ECC Error Control Register	0x00000000
0x310A2180	MEC0_ECCERR_STAT[y]	MEC0 ECC Error Status Register	0x00000000
0x310A21C0	MEC0_ECCERR_IMASK[y]	MEC0 ECC Error Interrupt Mask Register	0x00000000
0x310A2F00	MEC0_CLR	MEC0 Clear Register	0x00000000
0x310A2FD0	MEC0_PID4	MEC0 Peripheral ID4 Register	0x00000000
0x310A2FD4	MEC0_PID5	MEC0 Peripheral ID5 Register	0x00000000
0x310A2FD8	MEC0_PID6	MEC0 Peripheral ID6 Register	0x00000000
0x310A2FDC	MEC0_PID7	MEC0 Peripheral ID7 Register	0x00000000
0x310A2FE0	MEC0_PID0	MEC0 Peripheral ID0 Register	0x00000003
0x310A2FE4	MEC0_PID1	MEC0 Peripheral ID1 Register	0x00000050
0x310A2FE8	MEC0_PID2	MEC0 Peripheral ID2 Register	0x0000000E
0x310A2FEC	MEC0_PID3	MEC0 Peripheral ID3 Register	0x00000000
0x310A2FF0	MEC0_CID0	MEC0 Component ID0 Register	0x0000000D
0x310A2FF4	MEC0_CID1	MEC0 Component ID1 Register	0x000000F0
0x310A2FF8	MEC0_CID2	MEC0 Component ID2 Register	0x00000005
0x310A2FFC	MEC0_CID3	MEC0 Component ID3 Register	0x000000B1

Table A-104: ADSP-2159x MEC1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A3000	MEC1_PEIRQ_GCTL[p]	MEC1 Parity Error Interrupt Request Global Control Register	0x00000000
0x310A3010	MEC1_PEIRQ_GSTAT[p]	MEC1 Parity Error Interrupt Request Global Status Register	0x00000000
0x310A3040	MEC1_PERR_CTL0	MEC1 Parity Error Control Register	0x00000000
0x310A3044	MEC1_PERR_CTL1	MEC1 Parity Error Control Register	0x00000000
0x310A3080	MEC1_PERR_STAT0	MEC1 Parity Error Status Register	0x00000000
0x310A3084	MEC1_PERR_STAT1	MEC1 Parity Error Status Register	0x00000000
0x310A30C0	MEC1_PERR_IMASK0	MEC1 Parity Error Interrupt Mask Register	0x00000000
0x310A30C4	MEC1_PERR_IMASK1	MEC1 Parity Error Interrupt Mask Register	0x00000000
0x310A3100	MEC1_EEIRQ_GCTL[q]	MEC1 ECC Error Interrupt Request Global Control Register	0x00000000
0x310A3110	MEC1_EEIRQ_GSTAT[q]	MEC1 ECC Error Interrupt Request Global Status Register	0x00000000
0x310A3140	MEC1_ECCERR_CTL[y]	MEC1 ECC Error Control Register	0x00000000
0x310A3180	MEC1_ECCERR_STAT[y]	MEC1 ECC Error Status Register	0x00000000
0x310A31C0	MEC1_ECCERR_IMASK[y]	MEC1 ECC Error Interrupt Mask Register	0x00000000
0x310A3F00	MEC1_CLR	MEC1 Clear Register	0x00000000
0x310A3FD0	MEC1_PID4	MEC1 Peripheral ID4 Register	0x00000000
0x310A3FD4	MEC1_PID5	MEC1 Peripheral ID5 Register	0x00000000
0x310A3FD8	MEC1_PID6	MEC1 Peripheral ID6 Register	0x00000000
0x310A3FDC	MEC1_PID7	MEC1 Peripheral ID7 Register	0x00000000
0x310A3FE0	MEC1_PID0	MEC1 Peripheral ID0 Register	0x00000003
0x310A3FE4	MEC1_PID1	MEC1 Peripheral ID1 Register	0x00000050
0x310A3FE8	MEC1_PID2	MEC1 Peripheral ID2 Register	0x0000000E
0x310A3FEC	MEC1_PID3	MEC1 Peripheral ID3 Register	0x00000000
0x310A3FF0	MEC1_CID0	MEC1 Component ID0 Register	0x0000000D
0x310A3FF4	MEC1_CID1	MEC1 Component ID1 Register	0x000000F0
0x310A3FF8	MEC1_CID2	MEC1 Component ID2 Register	0x00000005
0x310A3FFC	MEC1_CID3	MEC1 Component ID3 Register	0x000000B1

Table A-105: ADSP-2159x MEC2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A4000	MEC2_PEIRQ_GCTL[p]	MEC2 Parity Error Interrupt Request Global Control Register	0x00000000
0x310A4010	MEC2_PEIRQ_GSTAT[p]	MEC2 Parity Error Interrupt Request Global Status Register	0x00000000
0x310A4040	MEC2_PERR_CTL0	MEC2 Parity Error Control Register	0x00000000
0x310A4044	MEC2_PERR_CTL1	MEC2 Parity Error Control Register	0x00000000
0x310A4080	MEC2_PERR_STAT0	MEC2 Parity Error Status Register	0x00000000
0x310A4084	MEC2_PERR_STAT1	MEC2 Parity Error Status Register	0x00000000
0x310A40C0	MEC2_PERR_IMASK0	MEC2 Parity Error Interrupt Mask Register	0x00000000
0x310A40C4	MEC2_PERR_IMASK1	MEC2 Parity Error Interrupt Mask Register	0x00000000
0x310A4100	MEC2_EEIRQ_GCTL[q]	MEC2 ECC Error Interrupt Request Global Control Register	0x00000000
0x310A4110	MEC2_EEIRQ_GSTAT[q]	MEC2 ECC Error Interrupt Request Global Status Register	0x00000000
0x310A4140	MEC2_ECCERR_CTL[y]	MEC2 ECC Error Control Register	0x00000000
0x310A4180	MEC2_ECCERR_STAT[y]	MEC2 ECC Error Status Register	0x00000000
0x310A41C0	MEC2_ECCERR_IMASK[y]	MEC2 ECC Error Interrupt Mask Register	0x00000000
0x310A4F00	MEC2_CLR	MEC2 Clear Register	0x00000000
0x310A4FD0	MEC2_PID4	MEC2 Peripheral ID4 Register	0x00000000
0x310A4FD4	MEC2_PID5	MEC2 Peripheral ID5 Register	0x00000000
0x310A4FD8	MEC2_PID6	MEC2 Peripheral ID6 Register	0x00000000
0x310A4FDC	MEC2_PID7	MEC2 Peripheral ID7 Register	0x00000000
0x310A4FE0	MEC2_PID0	MEC2 Peripheral ID0 Register	0x00000003
0x310A4FE4	MEC2_PID1	MEC2 Peripheral ID1 Register	0x00000050
0x310A4FE8	MEC2_PID2	MEC2 Peripheral ID2 Register	0x0000000E
0x310A4FEC	MEC2_PID3	MEC2 Peripheral ID3 Register	0x00000000
0x310A4FF0	MEC2_CID0	MEC2 Component ID0 Register	0x0000000D
0x310A4FF4	MEC2_CID1	MEC2 Component ID1 Register	0x000000F0
0x310A4FF8	MEC2_CID2	MEC2 Component ID2 Register	0x00000005
0x310A4FFC	MEC2_CID3	MEC2 Component ID3 Register	0x000000B1

Table A-106: ADSP-2159x SCB1 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x30200020	SCB1_DMC_IB_SYNC_MODE	SCB1 DMC Fabric (CLK03) Synchronization Mode Register	0x00000004

Table A-107: ADSP-2159x SCB5 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x30400000	SCB5_SPI2_OSPI_REMAP	SCB5 SPI2/OSPI Memory Map Address Remap Register	0x00000000

Table A-108: ADSP-2159x SCB0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x30042100	SCB0_SP0A_READ_QOS	SCB0 SP0A Read Quality of Service Register	0x0000000C
0x30042104	SCB0_SP0A_WRITE_QOS	SCB0 SP0A Write Quality of Service Register	0x0000000C
0x30043100	SCB0_SP0B_READ_QOS	SCB0 SP0B Read Quality of Service Register	0x0000000C
0x30043104	SCB0_SP0B_WRITE_QOS	SCB0 SP0B Write Quality of Service Register	0x0000000C
0x30044100	SCB0_SP1A_READ_QOS	SCB0 SP1A Read Quality of Service Register	0x0000000C
0x30044104	SCB0_SP1A_WRITE_QOS	SCB0 SP1A Write Quality of Service Register	0x0000000C
0x30045100	SCB0_SP1B_READ_QOS	SCB0 SP1B Read Quality of Service Register	0x0000000C
0x30045104	SCB0_SP1B_WRITE_QOS	SCB0 SP1B Write Quality of Service Register	0x0000000C
0x30046100	SCB0_SP2A_READ_QOS	SCB0 SP2A Read Quality of Service Register	0x0000000C
0x30046104	SCB0_SP2A_WRITE_QOS	SCB0 SP2A Write Quality of Service Register	0x0000000C
0x30047100	SCB0_SP2B_READ_QOS	SCB0 SP2B Read Quality of Service Register	0x0000000C
0x30047104	SCB0_SP2B_WRITE_QOS	SCB0 SP2B Write Quality of Service Register	0x0000000C
0x30048100	SCB0_SP3A_READ_QOS	SCB0 SP3A Read Quality of Service Register	0x0000000C
0x30048104	SCB0_SP3A_WRITE_QOS	SCB0 SP3A Write Quality of Service Register	0x0000000C
0x30049100	SCB0_SP3B_READ_QOS	SCB0 SP3B Read Quality of Service Register	0x0000000C
0x30049104	SCB0_SP3B_WRITE_QOS	SCB0 SP3B Write Quality of Service Register	0x0000000C
0x3004A100	SCB0_CRC0_CH0_READ_QOS	SCB0 CRC0 Channel 0 Read Quality of Service Register	0x00000001
0x3004A104	SCB0_CRC0_CH0_WRITE_QOS	SCB0 CRC0 Channel 0 Write Quality of Service Register	0x00000001
0x3004B100	SCB0_CRC0_CH1_READ_QOS	SCB0 CRC0 Channel 1 Read Quality of Service Register	0x00000001

Table A-108: ADSP-2159x SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3004B104	SCB0_CRC0_CH1_WRITE_QOS	SCB0 CRC0 Channel 1 Write Quality of Service Register	0x00000001
0x3004C100	SCB0_CRC1_CH0_READ_QOS	SCB0 CRC1 Channel 0 Read Quality of Service Register	0x00000001
0x3004C104	SCB0_CRC1_CH0_WRITE_QOS	SCB0 CRC1 Channel 0 Write Quality of Service Register	0x00000001
0x3004D100	SCB0_CRC1_CH1_READ_QOS	SCB0 CRC1 Channel 1 Read Quality of Service Register	0x00000001
0x3004D104	SCB0_CRC1_CH1_WRITE_QOS	SCB0 CRC1 Channel 1 Write Quality of Service Register	0x00000001
0x3004E100	SCB0_UART2_RX_READ_QOS	SCB0 UART2 RX Read Quality of Service Register	0x0000000C
0x3004E104	SCB0_UART2_RX_WRITE_QOS	SCB0 UART2 RX Write Quality of Service Register	0x0000000C
0x3004F100	SCB0_SH0_DPORT_READ_QOS	SCB0 SH0 DPORT Read Quality of Service Register	0x00000007
0x3004F104	SCB0_SH0_DPORT_WRITE_QOS	SCB0 SH0 DPORT Write Quality of Service Register	0x00000007
0x30050100	SCB0_SH0_LPORT_READ_QOS	SCB0 SH0 LPORT Read Quality of Service Register	0x00000007
0x30050104	SCB0_SH0_LPORT_WRITE_QOS	SCB0 SH0 LPORT Write Quality of Service Register	0x00000007
0x30051100	SCB0_SP4A_READ_QOS	SCB0 SP4A Read Quality of Service Register	0x0000000C
0x30051104	SCB0_SP4A_WRITE_QOS	SCB0 SP4A Write Quality of Service Register	0x0000000C
0x30052100	SCB0_SP4B_READ_QOS	SCB0 SP4B Read Quality of Service Register	0x0000000C
0x30052104	SCB0_SP4B_WRITE_QOS	SCB0 SP4B Write Quality of Service Register	0x0000000C
0x30053100	SCB0_UART3_TX_READ_QOS	SCB0 UART3 TX Read Quality of Service Register	0x0000000C
0x30053104	SCB0_UART3_TX_WRITE_QOS	SCB0 UART3 TX Write Quality of Service Register	0x0000000C
0x30054100	SCB0_UART3_RX_READ_QOS	SCB0 UART3 RX Read Quality of Service Register	0x0000000C
0x30054104	SCB0_UART3_RX_WRITE_QOS	SCB0 UART3 RX Write Quality of Service Register	0x0000000C
0x30055100	SCB0_HSMDMA_CH0_READ_QOS	SCB0 HSMDMA Channel 0 Read Quality of Service Register	0x00000001

Table A-108: ADSP-2159x SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x30055104	SCB0_HSMDMA_CH0_WRITE_QOS	SCB0 HSMDMA Channel 0 Write Quality of Service Register	0x00000001
0x30056100	SCB0_SP5A_READ_QOS	SCB0 SP5A Read Quality of Service Register	0x0000000C
0x30056104	SCB0_SP5A_WRITE_QOS	SCB0 SP5A Write Quality of Service Register	0x0000000C
0x30057100	SCB0_MLB_READ_QOS	SCB0 MLB Read Quality of Service Register	0x0000000C
0x30057104	SCB0_MLB_WRITE_QOS	SCB0 MLB Write Quality of Service Register	0x0000000C
0x30058100	SCB0_UART0_TX_READ_QOS	SCB0 UART0 TX Read Quality of Service Register	0x0000000C
0x30058104	SCB0_UART0_TX_WRITE_QOS	SCB0 UART0 TX Write Quality of Service Register	0x0000000C
0x30059100	SCB0_UART0_RX_READ_QOS	SCB0 UART0 RX Read Quality of Service Register	0x0000000C
0x30059104	SCB0_UART0_RX_WRITE_QOS	SCB0 UART0 RX Write Quality of Service Register	0x0000000C
0x3005A100	SCB0_SP5B_READ_QOS	SCB0 SP5B Read Quality of Service Register	0x0000000C
0x3005A104	SCB0_SP5B_WRITE_QOS	SCB0 SP5B Write Quality of Service Register	0x0000000C
0x3005B100	SCB0_HSMDMA_CH1_READ_QOS	SCB0 HSMDMA Channel 1 Read Quality of Service Register	0x00000001
0x3005B104	SCB0_HSMDMA_CH1_WRITE_QOS	SCB0 HSMDMA Channel 1 Write Quality of Service Register	0x00000001
0x3005C100	SCB0_UART2_TX_READ_QOS	SCB0 UART2 TX Read Quality of Service Register	0x0000000C
0x3005C104	SCB0_UART2_TX_WRITE_QOS	SCB0 UART2 TX Write Quality of Service Register	0x0000000C
0x3005D100	SCB0_SPI0TX_READ_QOS	SCB0 SPI0 TX Read Quality of Service Register	0x0000000C
0x3005D104	SCB0_SPI0TX_WRITE_QOS	SCB0 SPI0 TX Write Quality of Service Register	0x0000000C
0x3005E100	SCB0_SPI0RX_READ_QOS	SCB0 SPI0 RX Read Quality of Service Register	0x0000000C
0x3005E104	SCB0_SPI0RX_WRITE_QOS	SCB0 SPI0 RX Write Quality of Service Register	0x0000000C
0x3005F100	SCB0_SPI1TX_READ_QOS	SCB0 SPI1 TX Read Quality of Service Register	0x0000000C
0x3005F104	SCB0_SPI1TX_WRITE_QOS	SCB0 SPI1 TX Write Quality of Service Register	0x0000000C
0x30060100	SCB0_SPI1RX_READ_QOS	SCB0 SPI1 RX Read Quality of Service Register	0x0000000C

Table A-108: ADSP-2159x SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x30060104	SCB0_SPI1RX_WRITE_QOS	SCB0 SPI1 RX Write Quality of Service Register	0x0000000C
0x30061100	SCB0_SPI2TX_READ_QOS	SCB0 SPI2 TX Read Quality of Service Register	0x0000000C
0x30061104	SCB0_SPI2TX_WRITE_QOS	SCB0 SPI2 TX Write Quality of Service Register	0x0000000C
0x30062100	SCB0_SPI2RX_READ_QOS	SCB0 SPI2 RX Read Quality of Service Register	0x0000000C
0x30062104	SCB0_SPI2RX_WRITE_QOS	SCB0 SPI2 RX Write Quality of Service Register	0x0000000C
0x30063100	SCB0_SP6A_READ_QOS	SCB0 SP6A Read Quality of Service Register	0x0000000C
0x30063104	SCB0_SP6A_WRITE_QOS	SCB0 SP6A Write Quality of Service Registers	0x0000000C
0x30064100	SCB0_SP6B_READ_QOS	SCB0 SP6B Read Quality of Service Register	0x0000000C
0x30064104	SCB0_SP6B_WRITE_QOS	SCB0 SP6B Write Quality of Service Register	0x0000000C
0x30065100	SCB0_LP0_READ_QOS	SCB0 LP0 Read Quality of Service Register	0x00000007
0x30065104	SCB0_LP0_WRITE_QOS	SCB0 LP0 Write Quality of Service Register	0x00000007
0x30066100	SCB0_SP7A_READ_QOS	SCB0 SP7A Read Quality of Service Register	0x0000000C
0x30066104	SCB0_SP7A_WRITE_QOS	SCB0 SP7A Write Quality of Service Register	0x0000000C
0x30067100	SCB0_SP7B_READ_QOS	SCB0 SP7B Read Quality of Service Register	0x0000000C
0x30067104	SCB0_SP7B_WRITE_QOS	SCB0 SP7B Write Quality of Service Register	0x0000000C
0x30068100	SCB0_SH0_MMR_READ_QOS	SCB0 SH0 MMR Read Quality of Service Register	0x00000007
0x30068104	SCB0_SH0_MMR_WRITE_QOS	SCB0 SH0 MMR Write Quality of Service Register	0x00000007
0x30069100	SCB0_SPI3RX_READ_QOS	SCB0 SPI3 RX Read Quality of Service Register	0x0000000C
0x30069104	SCB0_SPI3RX_WRITE_QOS	SCB0 SPI3 RX Write Quality of Service Register	0x0000000C
0x3006A100	SCB0_UART1_TX_READ_QOS	SCB0 UART1 TX Read Quality of Service Register	0x0000000C
0x3006A104	SCB0_UART1_TX_WRITE_QOS	SCB0 UART1 TX Write Quality of Service Registers	0x0000000C
0x3006B100	SCB0_UART1_RX_READ_QOS	SCB0 UART1 RX Read Quality of Service Register	0x0000000C
0x3006B104	SCB0_UART1_RX_WRITE_QOS	SCB0 UART1 RX Write Quality of Service Register	0x0000000C
0x3006C100	SCB0_SPI3TX_READ_QOS	SCB0 SPI3 TX Read Quality of Service Register	0x0000000C

Table A-108: ADSP-2159x SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3006C104	SCB0_SPI3TX_WRITE_QOS	SCB0 SPI3 TX Write Quality of Service Register	0x0000000C
0x3006D100	SCB0_LP1_READ_QOS	SCB0 LP1 Read Quality of Service Register	0x00000007
0x3006D104	SCB0_LP1_WRITE_QOS	SCB0 LP1 Write Quality of Service Register	0x00000007
0x3006E100	SCB0_CRC2_CH1_READ_QOS	SCB0 CRC2 Channel 1 Read Quality of Service Register	0x00000001
0x3006E104	SCB0_CRC2_CH1_WRITE_QOS	SCB0 CRC2 Channel 1 Write Quality of Service Register	0x00000001
0x3006F100	SCB0_CRC2_CH0_READ_QOS	SCB0 CRC2 Channel 0 Read Quality of Service Register	0x00000001
0x3006F104	SCB0_CRC2_CH0_WRITE_QOS	SCB0 CRC2 Channel 0 Write Quality of Service Register	0x00000001
0x30070100	SCB0_CRC3_CH1_READ_QOS	SCB0 CRC3 Channel 1 Read Quality of Service Register	0x00000001
0x30070104	SCB0_CRC3_CH1_WRITE_QOS	SCB0 CRC3 Channel 1 Write Quality of Service Register	0x00000001
0x30071100	SCB0_CRYPT-TO_READ_QOS	SCB0 CRYPTO Read Quality of Service Register	0x00000001
0x30071104	SCB0_CRYPT-TO_WRITE_QOS	SCB0 CRYPTO Write Quality of Service Register	0x00000001
0x30072100	SCB0_CRC3_CH0_READ_QOS	SCB0 CRC3 Channel 0 Read Quality of Service Register	0x00000001
0x30072104	SCB0_CRC3_CH0_WRITE_QOS	SCB0 CRC3 Channel 0 Write Quality of Service Register	0x00000001
0x30073100	SCB0_MSMDMA1_CH1_READ_QOS	SCB0 MSMDMA1 Channel 1 Read Quality of Service Register	0x00000001
0x30073104	SCB0_MSMDMA1_CH1_WRITE_QOS	SCB0 MSMDMA1 Channel 1 Write Quality of Service Register	0x00000001
0x30074100	SCB0_SH0_FIR_CH0_READ_QOS	SCB0 SH0 FIR Channel 0 Read Quality of Service Register	0x00000007
0x30074104	SCB0_SH0_FIR_CH0_WRITE_QOS	SCB0 SH0 FIR Channel 0 Write Quality of Service Register	0x00000007
0x30075100	SCB0_SH0_FIR_CH1_READ_QOS	SCB0 SH0 FIR Channel 1 Read Quality of Service Register	0x00000007
0x30075104	SCB0_SH0_FIR_CH1_WRITE_QOS	SCB0 SH0 FIR Channel 1 Write Quality of Service Register	0x00000007

Table A-108: ADSP-2159x SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x30076100	SCB0_MSMDMA1_CH0_READ_QOS	SCB0 MSMDMA1 Channel 0 Read Quality of Service Register	0x00000001
0x30076104	SCB0_MSMDMA1_CH0_WRITE_QOS	SCB0 MSMDMA1 Channel 0 Write Quality of Service Register	0x00000001
0x30077100	SCB0_HSMDMA1_CH1_READ_QOS	SCB0 HSMDMA1 Channel 1 Read Quality of Service Register	0x00000001
0x30077104	SCB0_HSMDMA1_CH1_WRITE_QOS	SCB0 HSMDMA1 Channel 1 Write Quality of Service Register	0x00000001
0x30078100	SCB0_DLDMA0_CH0_READ_QOS	SCB0 DLDMA0 Channel 0 Read Quality of Service Register	0x00000001
0x30078104	SCB0_DLDMA0_CH0_WRITE_QOS	SCB0 DLDMA0 Channel 0 Write Quality of Service Register	0x00000001
0x30079100	SCB0_DLDMA0_CH1_READ_QOS	SCB0 DLDMA0 Channel 1 Read Quality of Service Register	0x00000001
0x30079104	SCB0_DLDMA0_CH1_WRITE_QOS	SCB0 DLDMA0 Channel 1 Write Quality of Service Register	0x00000001
0x3007A100	SCB0_DLDMA1_CH0_READ_QOS	SCB0 DLDMA1 Channel 0 Read Quality of Service Register	0x00000001
0x3007A104	SCB0_DLDMA1_CH0_WRITE_QOS	SCB0 DLDMA1 Channel 0 Write Quality of Service Register	0x00000001
0x3007B100	SCB0_DLDMA1_CH1_READ_QOS	SCB0 DLDMA1 Channel 1 Read Quality of Service Register	0x00000001
0x3007B104	SCB0_DLDMA1_CH1_WRITE_QOS	SCB0 DLDMA1 Channel 1 Write Quality of Service Register	0x00000001
0x3007C100	SCB0_MSMDMA_CH0_READ_QOS	SCB0 MSMDMA Channel 0 Read Quality of Service Register	0x00000001
0x3007C104	SCB0_MSMDMA_CH0_WRITE_QOS	SCB0 MSMDMA Channel 0 Write Quality of Service Register	0x00000001
0x3007D100	SCB0_MSMDMA_CH1_READ_QOS	SCB0 MSMDMA Channel 1 Read Quality of Service Register	0x00000001
0x3007D104	SCB0_MSMDMA_CH1_WRITE_QOS	SCB0 MSMDMA Channel 1 Write Quality of Service Registers	0x00000001
0x3007E100	SCB0_HSMDMA1_CH0_READ_QOS	SCB0 HSMDMA1 Channel 0 Read Quality of Service Register	0x00000001
0x3007E104	SCB0_HSMDMA1_CH0_WRITE_QOS	SCB0 HSMDMA1 Channel 0 Write Quality of Service Register	0x00000001
0x3007F100	SCB0_PPI_F0_READ_QOS	SCB0 PPI F0 Read Quality of Service Register	0x0000000C

Table A-108: ADSP-2159x SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3007F104	SCB0_PPI_F0_WRITE_QOS	SCB0 PPI F0 Write Quality of Service Register	0x0000000C
0x30080100	SCB0_PPI_F1_READ_QOS	SCB0 PPI F1 Read Quality of Service Register	0x0000000C
0x30080104	SCB0_PPI_F1_WRITE_QOS	SCB0 PPI F1 Write Quality of Service Register	0x0000000C
0x30081100	SCB0_USB0_READ_QOS	SCB0 USB0 Read Quality of Service Register	0x00000007
0x30081104	SCB0_USB0_WRITE_QOS	SCB0 USB0 Write Quality of Service Register	0x00000007
0x30082100	SCB0_SH1_IPORT_READ_QOS	SCB0 SH1 IPORT Read Quality of Service Register	0x00000007
0x30082104	SCB0_SH1_IPORT_WRITE_QOS	SCB0 SH1 IPORT Write Quality of Service Register	0x00000007
0x30083100	SCB0_SH1_DPORT_READ_QOS	SCB0 SH1 DPORT Read Quality of Service Register	0x00000007
0x30083104	SCB0_SH1_DPORT_WRITE_QOS	SCB0 SH1 DPORT Write Quality of Service Register	0x00000007
0x30084100	SCB0_SH1_MMR_READ_QOS	SCB0 SH1 MMR Read Quality of Service Register	0x00000007
0x30084104	SCB0_SH1_MMR_WRITE_QOS	SCB0 SH1 MMR Write Quality of Service Register	0x00000007
0x30085100	SCB0_PL310_M0_READ_QOS	SCB0 ARM_L2CC M0 Read Quality of Service Register	0x00000007
0x30085104	SCB0_PL310_M0_WRITE_QOS	SCB0 ARM_L2CC M0 Write Quality of Service Register	0x00000007
0x30086100	SCB0_PL310_M1_READ_QOS	SCB0 ARM_L2CC M1 Read Quality of Service Register	0x00000007
0x30086104	SCB0_PL310_M1_WRITE_QOS	SCB0 ARM_L2CC M1 Write Quality of Service Register	0x00000007
0x30087100	SCB0_GIGE_READ_QOS	SCB0 GIGE Read Quality of Service Register	0x00000007
0x30087104	SCB0_GIGE_WRITE_QOS	SCB0 GIGE Write Quality of Service Register	0x00000007
0x30088100	SCB0_DBG_READ_QOS	SCB0 DBG Read Quality of Service Register	0x00000001
0x30088104	SCB0_DBG_WRITE_QOS	SCB0 DBG Write Quality of Service Register	0x00000001
0x30089100	SCB0_ETR_READ_QOS	SCB0 ETR Read Quality of Service Register	0x00000001
0x30089104	SCB0_ETR_WRITE_QOS	SCB0 ETR Write Quality of Service Register	0x00000001
0x3008A100	SCB0_EMAC_READ_QOS	SCB0 EMAC Read Quality of Service Register	0x00000007
0x3008A104	SCB0_EMAC_WRITE_QOS	SCB0 EMAC Write Quality of Service Register	0x00000007

Table A-108: ADSP-2159x SCB0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3008B100	SCB0_SH1_FIR_CH1_READ_QOS	SCB0 SH1 FIR Channel 1 Read Quality of Service Register	0x00000007
0x3008B104	SCB0_SH1_FIR_CH1_WRITE_QOS	SCB0 SH1 FIR Channel 1 Write Quality of Service Register	0x00000007
0x3008C100	SCB0_SH1_FIR_CH0_READ_QOS	SCB0 SH1 FIR Channel 0 Read Quality of Service Register	0x00000007
0x3008C104	SCB0_SH1_FIR_CH0_WRITE_QOS	SCB0 SH1 FIR Channel 0 Write Quality of Service Register	0x00000007
0x3008D100	SCB0_SH0_IIR_CH0_READ_QOS	SCB0 SH0 IIR Channel 0 Read Quality of Service Register	0x00000007
0x3008D104	SCB0_SH0_IIR_CH0_WRITE_QOS	SCB0 SH0 IIR Channel 0 Write Quality of Service Register	0x00000007
0x3008E100	SCB0_SH0_IIR_CH1_READ_QOS	SCB0 SH0 IIR Channel 1 Read Quality of Service Register	0x00000007
0x3008E104	SCB0_SH0_IIR_CH1_WRITE_QOS	SCB0 SH0 IIR Channel 1 Write Quality of Service Register	0x00000007
0x3008F100	SCB0_SH1_IIR_CH0_READ_QOS	SCB0 SH1 IIR Channel 0 Read Quality of Service Register	0x00000007
0x3008F104	SCB0_SH1_IIR_CH0_WRITE_QOS	SCB0 SH1 IIR Channel 0 Write Quality of Service Register	0x00000007
0x30090100	SCB0_SH1_IIR_CH1_READ_QOS	SCB0 SH1 IIR Channel 1 Read Quality of Service Register	0x00000007
0x30090104	SCB0_SH1_IIR_CH1_WRITE_QOS	SCB0 SH1 IIR Channel 1 Write Quality of Service Register	0x00000007
0x300C2020	SCB0_LP_SYNC_MODE	SCB0 LP Fabric (CLK08) Synchronization Mode Register	0x00000004

Table A-109: ADSP-2159x SCB4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x30342100	SCB4_IIR2_CH0_IB_READ_QOS	SCB4 Iir2 Ch0 Ib.read Qos	0x0000000C
0x30342104	SCB4_IIR2_CH0_IB_WRITE_QOS	SCB4 Iir2 Ch0 Ib.write Qos	0x0000000C
0x30343100	SCB4_SHARC_DPORT_READ_QOS	SCB4 Sharc Dport.read Qos	0x0000000C
0x30343104	SCB4_SHARC_DPORT_WRITE_QOS	SCB4 Sharc Dport.write Qos	0x0000000C

Table A-109: ADSP-2159x SCB4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x30344100	SCB4_FAB- RIC_S2PORT_IB_READ_Q OS	SCB4 Fabric S2port Ib.read Qos	0x0000000C
0x30344104	SCB4_FAB- RIC_S2PORT_IB_WRITE_ QOS	SCB4 Fabric S2port Ib.write Qos	0x0000000C
0x30345100	SCB4_FIR_CH0_IB_READ_ QOS	SCB4 Fir Ch0 Ib.read Qos	0x0000000C
0x30345104	SCB4_FIR_CH0_IB_WRITE_ _QOS	SCB4 Fir Ch0 Ib.write Qos	0x0000000C
0x30346100	SCB4_FIR_CH1_IB_READ_ QOS	SCB4 Fir Ch1 Ib.read Qos	0x0000000C
0x30346104	SCB4_FIR_CH1_IB_WRITE_ _QOS	SCB4 Fir Ch1 Ib.write Qos	0x0000000C
0x30347100	SCB4_IIR0_CH0_IB_READ_ _QOS	SCB4 Iir0 Ch0 Ib.read Qos	0x0000000C
0x30347104	SCB4_IIR0_CH0_IB_WRIT E_QOS	SCB4 Iir0 Ch0 Ib.write Qos	0x0000000C
0x30348100	SCB4_IIR0_CH1_IB_READ_ _QOS	SCB4 Iir0 Ch1 Ib.read Qos	0x0000000C
0x30348104	SCB4_IIR0_CH1_IB_WRIT E_QOS	SCB4 Iir0 Ch1 Ib.write Qos	0x0000000C
0x30349100	SCB4_IIR2_CH1_IB_READ_ _QOS	SCB4 Iir2 Ch1 Ib.read Qos	0x0000000C
0x30349104	SCB4_IIR2_CH1_IB_WRIT E_QOS	SCB4 Iir2 Ch1 Ib.write Qos	0x0000000C
0x3034A100	SCB4_FAB- RIC_S1PORT_IB_READ_Q OS	SCB4 Fabric S1port Ib.read Qos	0x0000000C
0x3034A104	SCB4_FAB- RIC_S1PORT_IB_WRITE_ QOS	SCB4 Fabric S1port Ib.write Qos	0x0000000C
0x3034B100	SCB4_IIR3_CH0_IB_READ_ _QOS	SCB4 Iir3 Ch0 Ib.read Qos	0x0000000C
0x3034B104	SCB4_IIR3_CH0_IB_WRIT E_QOS	SCB4 Iir3 Ch0 Ib.write Qos	0x0000000C
0x3034C100	SCB4_IIR3_CH1_IB_READ_ _QOS	SCB4 Iir3 Ch1 Ib.read Qos	0x0000000C

Table A-109: ADSP-2159x SCB4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3034C104	SCB4_IIR3_CH1_IB_WRIT E_QOS	SCB4 Iir3 Ch1 Ib.write Qos	0x0000000C
0x3034D100	SCB4_FAB- RIC_ACC_MMR_IB_READ _QOS	SCB4 Fabric Acc Mmr Ib.read Qos	0x0000000C
0x3034D104	SCB4_FAB- RIC_ACC_MMR_IB_WRIT E_QOS	SCB4 Fabric Acc Mmr Ib.write Qos	0x0000000C
0x3034E100	SCB4_IIR1_CH0_IB_READ _QOS	SCB4 Iir1 Ch0 Ib.read Qos	0x0000000C
0x3034E104	SCB4_IIR1_CH0_IB_WRIT E_QOS	SCB4 Iir1 Ch0 Ib.write Qos	0x0000000C
0x3034F100	SCB4_IIR1_CH1_IB_READ _QOS	SCB4 Iir1 Ch1 Ib.read Qos	0x0000000C
0x3034F104	SCB4_IIR1_CH1_IB_WRIT E_QOS	SCB4 Iir1 Ch1 Ib.write Qos	0x0000000C

Table A-110: ADSP-2159x SCB3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x30102020	SCB3_LP_MMR_IB_SYNC_ MODE	SCB3 DMC MMRG Fabric (CLK08) Synchronization Mode Register	0x00000004
0x30105020	SCB3_DMC_MMR_IB_SYN C_MODE	SCB3 DMC MMRG Fabric (CLK03) Synchronization Mode Register	0x00000004
0x3010B020	SCB3_CAN_MMR_IB_SYN C_MODE	SCB3 DMC MMRG Fabric (CLK04) Synchronization Mode Register	0x00000004

Table A-111: ADSP-2159x MISCREG MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A9000	MIS- CREG_SH0_PFB_RANGE_ SELECT	MISCREG Prefetch Range Selection Register	0xFFFFFFFF
0x310A9010	MISCREG_CAN_SYSCTL	MISCREG	0x00000000
0x310A9014	MIS- CREG_SHARC_BRIDGE_R EMAP	MISCREG	0x00000000

Table A-111: ADSP-2159x MISCREG MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A901C	MIS- CREG_PFB_L2CC_EXCL_C TL	MISCREG	0x00000000
0x310A9044	MIS- CREG_SH1_PFB_RANGE_ SELECT	MISCREG Prefetch Range Selection Register	0xFFFFFFFF

Table A-112: ADSP-2159x MLB0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109D000	MLB0_CTL0	MLB0 MediaLB Control 0 Register	0x00000000
0x3109D008	MLB0_PCTL0	MLB0 MediaLB 6-pin Control 0 Register	0x00000000
0x3109D00C	MLB0_MS0	MLB0 Channel Status 0 Register	0x00000000
0x3109D014	MLB0_MS1	MLB0 Channel Status 1 Register	0x00000000
0x3109D020	MLB0_MSS	MLB0 System Status Register	0x00000000
0x3109D024	MLB0_MSD	MLB0 System Data Register	0x00000000
0x3109D02C	MLB0_MIEN	MLB0 Interrupt Enable Register	0x00000000
0x3109D034	MLB0_GCTL	MLB0 MLB Global Control Register	0x00000000
0x3109D03C	MLB0_CTL1	MLB0 Control 1 Register	0x00000000
0x3109D080	MLB0_HCTL	MLB0 HBI Control Register	0x00000000
0x3109D088	MLB0_HCMR0	MLB0 HBI Channel Mask 0 Register	0x00000000
0x3109D08C	MLB0_HCMR1	MLB0 HBI Channel Mask 1 Register	0x00000000
0x3109D090	MLB0_HCER0	MLB0 HBI Channel Error 0 Register	0x00000000
0x3109D094	MLB0_HCER1	MLB0 HBI Channel Error 1 Register	0x00000000
0x3109D098	MLB0_HCBR0	MLB0 HBI Channel Busy 0 Register	0x00000000
0x3109D09C	MLB0_HCBR1	MLB0 HBI Channel Busy 1 Register	0x00000000
0x3109D0C0	MLB0_MDAT0	MLB0 Memory Interface Control Data 0 Register	0x00000000
0x3109D0C4	MLB0_MDAT1	MLB0 Memory Interface Control Data 1 Register	0x00000000
0x3109D0C8	MLB0_MDAT2	MLB0 Memory Interface Control Data 2 Register	0x00000000
0x3109D0CC	MLB0_MDAT3	MLB0 Memory Interface Control Data 3 Register	0x00000000
0x3109D0D0	MLB0_MDWE0	MLB0 Memory Interface Control Data Write Enable 0 Register	0x00000000

Table A-112: ADSP-2159x MLB0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109D0D4	MLB0_MDWE1	MLB0 Memory Interface Control Data Write Enable 1 Register	0x00000000
0x3109D0D8	MLB0_MDWE2	MLB0 Memory Interface Control Data Write Enable 2 Register	0x00000000
0x3109D0DC	MLB0_MDWE3	MLB0 Memory Interface Control Data Write Enable 3 Register	0x00000000
0x3109D0E0	MLB0_MCTL	MLB0 Memory Interface Control Register	0x00000000
0x3109D0E4	MLB0_MADR	MLB0 Memory Interface Address Register	0x00000000
0x3109D3C0	MLB0_ACTL	MLB0 Bus Control Register	0x00000000
0x3109D3D0	MLB0_ACSR0	MLB0 Peripheral Channel Status 0 Register	0x00000000
0x3109D3D4	MLB0_ACSR1	MLB0 Peripheral Channel Status 1 Register	0x00000000
0x3109D3D8	MLB0_ACMR0	MLB0 Peripheral Channel Mask 0 Register	0x00000000
0x3109D3DC	MLB0_ACMR1	MLB0 Peripheral Channel Mask 1 Register	0x00000000

Table A-113: ADSP-2159x OSPI0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31027000	OSPI0_CTL	OSPI0 Octal SPI Control Register	0x80780081
0x31027004	OSPI0_DRICTL	OSPI0 Device Read Instruction Control Register	0x00000003
0x31027008	OSPI0_DWICTL	OSPI0 Device Write Instruction Control Register	0x00000002
0x3102700C	OSPI0_DLY	OSPI0 Device Delay Register	0x00000000
0x31027010	OSPI0_RDC	OSPI0 Read Data Capture Register	0x00000001
0x31027014	OSPI0_DSCTL	OSPI0 Device Size Control Register	0x00101002
0x31027024	OSPI0_REMAPADDR	OSPI0 Remap Address Register	0x00000000
0x31027028	OSPI0_MBCTL	OSPI0 Mode Bit Control Register	0x00000000
0x31027038	OSPI0_WCCTL	OSPI0 Write Completion Control Register	0x00010005
0x3102703C	OSPI0_POLLEXP	OSPI0 Polling Expiration Register	0xFFFFFFFF
0x31027040	OSPI0_ISTAT	OSPI0 Interrupt Status Register	0x00000000
0x31027044	OSPI0_IMSK	OSPI0 Interrupt Mask Register	0x00000000
0x31027050	OSPI0_WRPROT_LWR	OSPI0 Lower Write Protection Register	0x00000000
0x31027054	OSPI0_WRPROT_UP	OSPI0 Upper Write Protection Register	0x00000000
0x31027058	OSPI0_WRPROT_CTL	OSPI0 Write Protection Control Register	0x00000000

Table A-113: ADSP-2159x OSPI0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3102708C	OSPI0_FCMCTL	OSPI0 Flash Command Control Memory Register	0x00000000
0x31027090	OSPI0_FCCTL	OSPI0 Flash Command Control Register	0x00000000
0x31027094	OSPI0_FCA	OSPI0 Flash Command Address Register	0x00000000
0x310270A0	OSPI0_FCRD_LWR	OSPI0 Flash Command Read Data Register (Lower)	0x00000000
0x310270A4	OSPI0_FCRD_UP	OSPI0 Flash Command Read Data Register (Upper)	0x00000000
0x310270A8	OSPI0_FCWD_LWR	OSPI0 Flash Command Write Data Register (Lower)	0x00000000
0x310270AC	OSPI0_FCWD_UP	OSPI0 Flash Command Write Data Register (Upper)	0x00000000
0x310270B0	OSPI0_POLSTAT	OSPI0 Polling Flash Status Register	0x00000000
0x310270B4	OSPI0_PHYCTL	OSPI0 PHY Control Register	0x40000000
0x310270B8	OSPI0_PHYMCTL	OSPI0 PHY DLL Master Control Register	0x00800000
0x310270BC	OSPI0_DLLOB_LWR	OSPI0 DLL Observable Register (Lower)	0x00000000
0x310270C0	OSPI0_DLLOB_UP	OSPI0 DLL Observable Register (Upper)	0x00000000
0x310270E0	OSPI0_OE_LWR	OSPI0 Opcode Extension Register (Lower)	0x13EDFA00
0x310270E4	OSPI0_OE_UP	OSPI0 Opcode Extension Register (Upper)	0x06F90000
0x310270FC	OSPI0_MODID	OSPI0 Module ID Register	0x04000300

Table A-114: ADSP-2159x OTPC0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31011004	OTPC0_STAT	OTPC0 OTP Status Register	0x00000000
0x3101102C	OTPC0_SECU_STATE	OTPC0 OTP Security State Register	0x00000001

Table A-115: ADSP-2159x PADS0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004604	PADS0_PCFG0	PADS0 Peripheral PAD Configuration0 Register	0x00050009
0x31004608	PADS0_PCFG1	PADS0 Peripheral Configuration1 Register	0x00000000
0x3100460C	PADS0_PORTA0_DS	PADS0 PORTA 0 to 7 pins DS control	0x00249249
0x31004610	PADS0_PORTA1_DS	PADS0 PORTA 8 - 15 pins DS control	0x00249249
0x31004614	PADS0_PORTB0_DS	PADS0 PORTB 0 to 7 pins DS control	0x00249249
0x31004618	PADS0_PORTB1_DS	PADS0 PORTB 8 - 15 pins DS control	0x00249249
0x3100461C	PADS0_PORTC0_DS	PADS0 PORTC 0 to 7 pins DS control	0x00249249

Table A-115: ADSP-2159x PADS0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004620	PADS0_PORTC1_DS	PADS0 PORTC 8 - 15 pins DS control	0x00249249
0x31004624	PADS0_PORTD0_DS	PADS0 PORTD 0 to 7 pins DS control	0x00249249
0x31004628	PADS0_PORTD1_DS	PADS0 PORTD 8 to 15 pins DS control	0x00249249
0x3100462C	PADS0_PORTE0_DS	PADS0 PORTE 0 to 7 pins DS control	0x00249249
0x31004630	PADS0_PORTE1_DS	PADS0 PORTE 8 to 15 pins DS control	0x00249249
0x31004634	PADS0_PORTF0_DS	PADS0 PORTF 0 to 7 pins DS control	0x00249249
0x31004638	PADS0_PORTF1_DS	PADS0 PORTF 8 to 15 pins DS control	0x00249249
0x3100463C	PADS0_PORTG0_DS	PADS0 PORTG 0 to 7 pins DS control	0x00249249
0x31004640	PADS0_PORTG1_DS	PADS0 PORTG 8 to 15 pins DS control	0x00249249
0x31004644	PADS0_PORTH0_DS	PADS0 PORTH 0 to 7 pins DS control	0x00249249
0x31004648	PADS0_PORTH1_DS	PADS0 PORTH 8 to 15 pins DS control	0x00249249
0x3100464C	PADS0_PORTI0_DS	PADS0 PORTI 0 to 7 pins DS control	0x00249249
0x31004650	PADS0_NONPORTS_DS	PADS0 Non-GPIO Drive Strength Register	0x0045244A
0x31004678	PADS0_DAI0_0_DS	PADS0 DAI0 1 to 10 pins DS control	0x09249249
0x3100467C	PADS0_DAI0_1_DS	PADS0 DAI0 11 to 20 pins DS control	0x09249249
0x31004680	PADS0_DAI1_0_DS	PADS0 DAI1 1 to 10 pins DS control	0x09249249
0x31004684	PADS0_DAI1_1_DS	PADS0 DAI1 11 to 20 pins DS control	0x09249249
0x31004690	PADS0_DAI0_IE	PADS0 DAI0 Port Input Enable Control Register	0x00000000
0x31004694	PADS0_DAI1_IE	PADS0 DAI1 Port Input Enable Control Register	0x00000000
0x31004698	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x3100469C	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x310046A0	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x310046A4	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x310046A8	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x310046AC	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x310046B0	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x310046B4	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x310046B8	PADS0_PORT[n]_PUE	PADS0 PORTx Pull-Up Enable	0x00000000
0x310046BC	PADS0_DAI[n]_PUE	PADS0 DAIx Pull-Up Enable	0x00000000
0x310046C0	PADS0_DAI[n]_PUE	PADS0 DAIx Pull-Up Enable	0x00000000
0x310046C4	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000

Table A-115: ADSP-2159x PADS0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310046C8	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000
0x310046CC	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000
0x310046D0	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000
0x310046D4	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000
0x310046D8	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000
0x310046DC	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000
0x310046E0	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000
0x310046E4	PADS0_PORT[n]_PDE	PADS0 PORTx Pull-Down Enable	0x00000000
0x310046FC	PADS0_DAI[n]_PDE	PADS0 DAIx Pull-Down Enable	0x00000000
0x31004700	PADS0_DAI[n]_PDE	PADS0 DAIx Pull-Down Enable	0x00000000

Table A-116: ADSP-2159x PCG0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C9300	PCG0_CTLA0	PCG0 Precision Clock A Control 0 Register	0x00000000
0x310C9304	PCG0_CTLA1	PCG0 Precision Clock A Control 1 Register	0x00000000
0x310C9308	PCG0_CTLB0	PCG0 Precision Clock B Control 0 Register	0x00000000
0x310C930C	PCG0_CTLB1	PCG0 Precision Clock B Control 1 Register	0x00000000
0x310C9310	PCG0_PW1	PCG0 Precision Clock Pulse Width Control 1 Register	0x00000000
0x310C9314	PCG0_SYNC1	PCG0 Precision Clock Frame Sync Synchronization 1 Register	0x00000000
0x310C9318	PCG0_CTLE0	PCG0 Precision Clock E Control 0 Register	0x00000000
0x310C931C	PCG0_CTLE1	PCG0 Precision Clock E Control 1 Register	0x00000000
0x310C9320	PCG0_CTLF0	PCG0 Precision Clock F Control 0 Register	0x00000000
0x310C9324	PCG0_CTLF1	PCG0 Precision Clock F Control 1 Register	0x00000000
0x310C9328	PCG0_PW3	PCG0 Precision Clock Pulse Width Control 3 Register	0x00000000
0x310C932C	PCG0_SYNC3	PCG0 Precision Clock Frame Sync Synchronization 3 Register	0x00000000
0x310CA300	PCG0_CTLC0	PCG0 Precision Clock C Control 0 Register	0x00000000
0x310CA304	PCG0_CTLC1	PCG0 Precision Clock C Control 1 Register	0x00000000
0x310CA308	PCG0_CTLD0	PCG0 Precision Clock D Control 0 Register	0x00000000
0x310CA30C	PCG0_CTLD1	PCG0 Precision Clock D Control 1 Register	0x00000000

Table A-116: ADSP-2159x PCG0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x310CA310	PCG0_PW2	PCG0 Precision Clock Pulse Width Control 2 Register	0x00000000
0x310CA314	PCG0_SYNC2	PCG0 Precision Clock Frame Sync Synchronization 2 Register	0x00000000
0x310CA318	PCG0_CTLG0	PCG0 Precision Clock G Control 0 Register	0x00000000
0x310CA31C	PCG0_CTLG1	PCG0 Precision Clock G Control 1 Register	0x00000000
0x310CA320	PCG0_CTLH0	PCG0 Precision Clock H Control 0 Register	0x00000000
0x310CA324	PCG0_CTLH1	PCG0 Precision Clock H Control 1 Register	0x00000000
0x310CA328	PCG0_PW4	PCG0 Precision Clock Pulse Width Control 4 Register	0x00000000
0x310CA32C	PCG0_SYNC4	PCG0 Precision Clock Frame Sync Synchronization 4 Register	0x00000000

Table A-117: ADSP-2159x PINT0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31005000	PINT0_MSK_SET	PINT0 PINT Mask Set Register	0x00000000
0x31005004	PINT0_MSK_CLR	PINT0 PINT Mask Clear Register	0x00000000
0x31005008	PINT0_REQ	PINT0 PINT Request Register	0x00000000
0x3100500C	PINT0_ASSIGN	PINT0 PINT Assign Register	0x00000101
0x31005010	PINT0_EDGE_SET	PINT0 PINT Edge Set Register	0x00000000
0x31005014	PINT0_EDGE_CLR	PINT0 PINT Edge Clear Register	0x00000000
0x31005018	PINT0_INV_SET	PINT0 PINT Invert Set Register	0x00000000
0x3100501C	PINT0_INV_CLR	PINT0 PINT Invert Clear Register	0x00000000
0x31005020	PINT0_PINSTATE	PINT0 PINT Pin State Register	0x00000000
0x31005024	PINT0_LATCH	PINT0 PINT Latch Register	0x00000000

Table A-118: ADSP-2159x PINT1 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31005100	PINT1_MSK_SET	PINT1 PINT Mask Set Register	0x00000000
0x31005104	PINT1_MSK_CLR	PINT1 PINT Mask Clear Register	0x00000000
0x31005108	PINT1_REQ	PINT1 PINT Request Register	0x00000000
0x3100510C	PINT1_ASSIGN	PINT1 PINT Assign Register	0x00000101
0x31005110	PINT1_EDGE_SET	PINT1 PINT Edge Set Register	0x00000000

Table A-118: ADSP-2159x PINT1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31005114	PINT1_EDGE_CLR	PINT1 PINT Edge Clear Register	0x00000000
0x31005118	PINT1_INV_SET	PINT1 PINT Invert Set Register	0x00000000
0x3100511C	PINT1_INV_CLR	PINT1 PINT Invert Clear Register	0x00000000
0x31005120	PINT1_PINSTATE	PINT1 PINT Pin State Register	0x00000000
0x31005124	PINT1_LATCH	PINT1 PINT Latch Register	0x00000000

Table A-119: ADSP-2159x PINT2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31005200	PINT2_MSK_SET	PINT2 PINT Mask Set Register	0x00000000
0x31005204	PINT2_MSK_CLR	PINT2 PINT Mask Clear Register	0x00000000
0x31005208	PINT2_REQ	PINT2 PINT Request Register	0x00000000
0x3100520C	PINT2_ASSIGN	PINT2 PINT Assign Register	0x00000101
0x31005210	PINT2_EDGE_SET	PINT2 PINT Edge Set Register	0x00000000
0x31005214	PINT2_EDGE_CLR	PINT2 PINT Edge Clear Register	0x00000000
0x31005218	PINT2_INV_SET	PINT2 PINT Invert Set Register	0x00000000
0x3100521C	PINT2_INV_CLR	PINT2 PINT Invert Clear Register	0x00000000
0x31005220	PINT2_PINSTATE	PINT2 PINT Pin State Register	0x00000000
0x31005224	PINT2_LATCH	PINT2 PINT Latch Register	0x00000000

Table A-120: ADSP-2159x PINT3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31005300	PINT3_MSK_SET	PINT3 PINT Mask Set Register	0x00000000
0x31005304	PINT3_MSK_CLR	PINT3 PINT Mask Clear Register	0x00000000
0x31005308	PINT3_REQ	PINT3 PINT Request Register	0x00000000
0x3100530C	PINT3_ASSIGN	PINT3 PINT Assign Register	0x00000101
0x31005310	PINT3_EDGE_SET	PINT3 PINT Edge Set Register	0x00000000
0x31005314	PINT3_EDGE_CLR	PINT3 PINT Edge Clear Register	0x00000000
0x31005318	PINT3_INV_SET	PINT3 PINT Invert Set Register	0x00000000
0x3100531C	PINT3_INV_CLR	PINT3 PINT Invert Clear Register	0x00000000
0x31005320	PINT3_PINSTATE	PINT3 PINT Pin State Register	0x00000000

Table A-120: ADSP-2159x PINT3 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31005324	PINT3_LATCH	PINT3 PINT Latch Register	0x00000000

Table A-121: ADSP-2159x PINT4 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31005400	PINT4_MSK_SET	PINT4 PINT Mask Set Register	0x00000000
0x31005404	PINT4_MSK_CLR	PINT4 PINT Mask Clear Register	0x00000000
0x31005408	PINT4_REQ	PINT4 PINT Request Register	0x00000000
0x3100540C	PINT4_ASSIGN	PINT4 PINT Assign Register	0x00000101
0x31005410	PINT4_EDGE_SET	PINT4 PINT Edge Set Register	0x00000000
0x31005414	PINT4_EDGE_CLR	PINT4 PINT Edge Clear Register	0x00000000
0x31005418	PINT4_INV_SET	PINT4 PINT Invert Set Register	0x00000000
0x3100541C	PINT4_INV_CLR	PINT4 PINT Invert Clear Register	0x00000000
0x31005420	PINT4_PINSTATE	PINT4 PINT Pin State Register	0x00000000
0x31005424	PINT4_LATCH	PINT4 PINT Latch Register	0x00000000

Table A-122: ADSP-2159x PINT5 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31005500	PINT5_MSK_SET	PINT5 PINT Mask Set Register	0x00000000
0x31005504	PINT5_MSK_CLR	PINT5 PINT Mask Clear Register	0x00000000
0x31005508	PINT5_REQ	PINT5 PINT Request Register	0x00000000
0x3100550C	PINT5_ASSIGN	PINT5 PINT Assign Register	0x00000101
0x31005510	PINT5_EDGE_SET	PINT5 PINT Edge Set Register	0x00000000
0x31005514	PINT5_EDGE_CLR	PINT5 PINT Edge Clear Register	0x00000000
0x31005518	PINT5_INV_SET	PINT5 PINT Invert Set Register	0x00000000
0x3100551C	PINT5_INV_CLR	PINT5 PINT Invert Clear Register	0x00000000
0x31005520	PINT5_PINSTATE	PINT5 PINT Pin State Register	0x00000000
0x31005524	PINT5_LATCH	PINT5 PINT Latch Register	0x00000000

Table A-123: ADSP-2159x PINT6 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31005600	PINT6_MSK_SET	PINT6 PINT Mask Set Register	0x00000000
0x31005604	PINT6_MSK_CLR	PINT6 PINT Mask Clear Register	0x00000000
0x31005608	PINT6_REQ	PINT6 PINT Request Register	0x00000000
0x3100560C	PINT6_ASSIGN	PINT6 PINT Assign Register	0x00000101
0x31005610	PINT6_EDGE_SET	PINT6 PINT Edge Set Register	0x00000000
0x31005614	PINT6_EDGE_CLR	PINT6 PINT Edge Clear Register	0x00000000
0x31005618	PINT6_INV_SET	PINT6 PINT Invert Set Register	0x00000000
0x3100561C	PINT6_INV_CLR	PINT6 PINT Invert Clear Register	0x00000000
0x31005620	PINT6_PINSTATE	PINT6 PINT Pin State Register	0x00000000
0x31005624	PINT6_LATCH	PINT6 PINT Latch Register	0x00000000

Table A-124: ADSP-2159x PINT7 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31005700	PINT7_MSK_SET	PINT7 PINT Mask Set Register	0x00000000
0x31005704	PINT7_MSK_CLR	PINT7 PINT Mask Clear Register	0x00000000
0x31005708	PINT7_REQ	PINT7 PINT Request Register	0x00000000
0x3100570C	PINT7_ASSIGN	PINT7 PINT Assign Register	0x00000101
0x31005710	PINT7_EDGE_SET	PINT7 PINT Edge Set Register	0x00000000
0x31005714	PINT7_EDGE_CLR	PINT7 PINT Edge Clear Register	0x00000000
0x31005718	PINT7_INV_SET	PINT7 PINT Invert Set Register	0x00000000
0x3100571C	PINT7_INV_CLR	PINT7 PINT Invert Clear Register	0x00000000
0x31005720	PINT7_PINSTATE	PINT7 PINT Pin State Register	0x00000000
0x31005724	PINT7_LATCH	PINT7 PINT Latch Register	0x00000000

Table A-125: ADSP-2159x PKA0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D4000	PKA0_APTR	PKA0 PKA Vector_A Address	0x00000000
0x310D4004	PKA0_BPTR	PKA0 PKA Vector_B Address	0x00000000
0x310D4008	PKA0_CPTR	PKA0 PKA Vector_C Address	0x00000000
0x310D400C	PKA0_DPTR	PKA0 PKA Vector_D Address	0x00000000

Table A-125: ADSP-2159x PKA0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D4010	PKA0_ALEN	PKA0 PKA Vector_A Length	0x00000000
0x310D4014	PKA0_BLEN	PKA0 PKA Vector_B Length	0x00000000
0x310D4018	PKA0_SHIFT	PKA0 PKA Bit Shift Value	0x00000000
0x310D401C	PKA0_FUNC	PKA0 PKA Function	0x00000000
0x310D4020	PKA0_COMPARE	PKA0 PKA Compare Result	0x00000001
0x310D4024	PKA0_RESULTMSW	PKA0 PKA Most-Significant-Word of Result Vector	0x00008000
0x310D4028	PKA0_DIVMSW	PKA0 PKA Most-Significant-Word of Divide Remainder	0x00008000
0x310D6000	PKA0_RAM	PKA0 Start of PKA RAM space	0x00000000

Table A-126: ADSP-2159x PKIC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D8000	PKIC0_POL_CTL	PKIC0 Polarity Control Register	0x00000000
0x310D8004	PKIC0_TYPE_CTL	PKIC0 Type Control Register	0x00000000
0x310D8008	PKIC0_EN_CTL	PKIC0 Enable Control Register	0x00000000
0x310D800C	PKIC0_RAW_STAT	PKIC0 Raw Status Register	0x00000000
0x310D800C	PKIC0_EN_SET	PKIC0 Enable Set Register	0x00000000
0x310D8010	PKIC0_ACK	PKIC0 Acknowledge Register	0x00000000
0x310D8010	PKIC0_EN_STAT	PKIC0 Enabled Status Register	0x00000000
0x310D8014	PKIC0_EN_CLR	PKIC0 Enable Clear Register	0x00000000

Table A-127: ADSP-2159x PKTE0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CD000	PKTE0_CTL_STAT	PKTE0 Packet Engine Control Register	0x00000002
0x310CD004	PKTE0_SRC_ADDR	PKTE0 Packet Engine Source Address	0x00000000
0x310CD008	PKTE0_DEST_ADDR	PKTE0 Packet Engine Destination Address	0x00000000
0x310CD00C	PKTE0_SA_ADDR	PKTE0 Packet Engine SA Address	0x00000000
0x310CD010	PKTE0_STATE_ADDR	PKTE0 Packet Engine State Record Address	0x00000000
0x310CD014	PKTE0_ARC4STATE_ADD R	PKTE0 Packet Engine ARC4 State Record Address	0x00000000
0x310CD018	PKTE0_USERID	PKTE0 Packet Engine User ID	0x00000000

Table A-127: ADSP-2159x PKTE0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CD01C	PKTE0_LEN	PKTE0 Packet Engine Length Register	0x00800000
0x310CD080	PKTE0_CDRBASE_ADDR	PKTE0 Packet Engine Command Descriptor Ring Base Address	0x00000000
0x310CD084	PKTE0_RDRBASE_ADDR	PKTE0 Packet Engine Result Descriptor Ring Base Address	0x00000000
0x310CD088	PKTE0_RING_CFG	PKTE0 Packet Engine Ring Configuration	0x00000000
0x310CD08C	PKTE0_RING_THRESH	PKTE0 Packet Engine Ring Threshold Registers	0x00000000
0x310CD090	PKTE0_CDSC_INCR	PKTE0 Packet Engine Command Descriptor Count Increment Register	0x00000000
0x310CD090	PKTE0_CDSC_CNT	PKTE0 Packet Engine Command Descriptor Count Register	0x00000000
0x310CD094	PKTE0_RDSC_CNT	PKTE0 Packet Engine Result Descriptor Count Registers	0x00000000
0x310CD094	PKTE0_RDSC_DECR	PKTE0 Packet Engine Result Descriptor Count Decrement Registers	0x00000000
0x310CD098	PKTE0_RING_PTR	PKTE0 Packet Engine Ring Pointer Status	0x00000000
0x310CD09C	PKTE0_RING_STAT	PKTE0 Packet Engine Ring Status	0x00000000
0x310CD100	PKTE0_CFG	PKTE0 Packet Engine Configuration Register	0x00000000
0x310CD104	PKTE0_STAT	PKTE0 Packet Engine Status Register	0x00040402
0x310CD10C	PKTE0_BUF_THRESH	PKTE0 Packet Engine Buffer Threshold Register	0x00800080
0x310CD110	PKTE0_INBUF_INCR	PKTE0 Packet Engine Input Buffer Count Increment Register	0x00000000
0x310CD110	PKTE0_INBUF_CNT	PKTE0 Packet Engine Input Buffer Count Register	0x00000000
0x310CD114	PKTE0_OUTBUF_CNT	PKTE0 Packet Engine Output Buffer Count Register	0x00000000
0x310CD114	PKTE0_OUTBUF_DECR	PKTE0 Packet Engine Output Buffer Count Decrement Register	0x00000000
0x310CD118	PKTE0_BUF_PTR	PKTE0 Packet Engine Buffer Pointer Register	0x00000000
0x310CD120	PKTE0_DMA_CFG	PKTE0 Packet Engine DMA Configuration Register	0x00180006
0x310CD1D0	PKTE0_ENDIAN_CFG	PKTE0 Packet Engine Endian Configuration Register	0x00E400E4
0x310CD1E0	PKTE0_HLT_CTL	PKTE0 Packet Engine Halt Control Register	0x00000000
0x310CD1E0	PKTE0_HLT_STAT	PKTE0 Packet Engine Halt Status Register	0x00000000
0x310CD1E4	PKTE0_CONT	PKTE0 PKTE Continue Register	0x00000000
0x310CD1E8	PKTE0_CLK_CTL	PKTE0 PE Clock Control Register	0x0000001F
0x310CD200	PKTE0_IUMSK_STAT	PKTE0 Interrupt Unmasked Status Register	0x00000000

Table A-127: ADSP-2159x PKTE0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CD204	PKTE0_IMSK_STAT	PKTE0 Interrupt Masked Status Register	0x00000000
0x310CD204	PKTE0_INT_CLR	PKTE0 Interrupt Clear Register	0x00000000
0x310CD208	PKTE0_INT_EN	PKTE0 Interrupt Enable Register	0x00000000
0x310CD20C	PKTE0_INT_CFG	PKTE0 Interrupt Configuration Register	0x00000000
0x310CD210	PKTE0_IMSK_EN	PKTE0 Interrupt Mask Enable Register	0x00000000
0x310CD214	PKTE0_IMSK_DIS	PKTE0 Interrupt Mask Disable Register	0x00000000
0x310CD400	PKTE0_SA_CMD0	PKTE0 SA Command 0	0x00000000
0x310CD404	PKTE0_SA_CMD1	PKTE0 SA Command 1	0x00000000
0x310CD408	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD40C	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD410	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD414	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD418	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD41C	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD420	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD424	PKTE0_SA_KEY[n]	PKTE0 SA Key Registers	0x00000000
0x310CD428	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD42C	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD430	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD434	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD438	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD43C	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD440	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD444	PKTE0_SA_IDIGEST[n]	PKTE0 SA Inner Hash Digest Registers	0x00000000
0x310CD448	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD44C	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD450	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD454	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD458	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD45C	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD460	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000

Table A-127: ADSP-2159x PKTE0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CD464	PKTE0_SA_ODIGEST[n]	PKTE0 SA Outer Hash Digest Registers	0x00000000
0x310CD468	PKTE0_SA_SPI	PKTE0 SA SPI Register	0x00000000
0x310CD46C	PKTE0_SA_SEQNUM[n]	PKTE0 SA Sequence Number Register	0x00000000
0x310CD470	PKTE0_SA_SEQNUM[n]	PKTE0 SA Sequence Number Register	0x00000000
0x310CD474	PKTE0_SA_SEQ- NUM_MSK[n]	PKTE0 SA Sequence Number Mask Registers	0x00000000
0x310CD478	PKTE0_SA_SEQ- NUM_MSK[n]	PKTE0 SA Sequence Number Mask Registers	0x00000000
0x310CD47C	PKTE0_SA_ARC4IJPTR	PKTE0 ARC4 i and j Pointer Register	0x00000000
0x310CD47C	PKTE0_SA_RDY	PKTE0 SA Ready Indicator	0x00000000
0x310CD47C	PKTE0_SA_NONCE	PKTE0 SA Initialization Vector Register	0x00000000
0x310CD500	PKTE0_STATE_IV[n]	PKTE0 State Initialization Vector Registers	0x00000000
0x310CD504	PKTE0_STATE_IV[n]	PKTE0 State Initialization Vector Registers	0x00000000
0x310CD508	PKTE0_STATE_IV[n]	PKTE0 State Initialization Vector Registers	0x00000000
0x310CD50C	PKTE0_STATE_IV[n]	PKTE0 State Initialization Vector Registers	0x00000000
0x310CD510	PKTE0_STATE_BYTE_CN T[n]	PKTE0 State Hash Byte Count Registers	0x00000000
0x310CD514	PKTE0_STATE_BYTE_CN T[n]	PKTE0 State Hash Byte Count Registers	0x00000000
0x310CD518	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD51C	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD520	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD524	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD528	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD52C	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD530	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD534	PKTE0_STATE_IDIGEST[n]	PKTE0 State Inner Digest Registers	0x00000000
0x310CD700	PKTE0_ARC4STATE_BUF	PKTE0 Starting Entry of 256-byte ARC4 State Buffer	0x00000000
0x310CD800	PKTE0_DATAIO_BUF	PKTE0 Starting Entry of 256-byte Data Input/Output Buffer	0x00000000

Table A-128: ADSP-2159x PORTA MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004000	PORTA_FER	PORTA Port x Function Enable Register	0x00000000
0x31004004	PORTA_FER_SET	PORTA Port x Function Enable Set Register	0x00000000
0x31004008	PORTA_FER_CLR	PORTA Port x Function Enable Clear Register	0x00000000
0x3100400C	PORTA_DATA	PORTA Port x GPIO Data Register	0x00000000
0x31004010	PORTA_DATA_SET	PORTA Port x GPIO Data Set Register	0x00000000
0x31004014	PORTA_DATA_CLR	PORTA Port x GPIO Data Clear Register	0x00000000
0x31004018	PORTA_DIR	PORTA Port x GPIO Direction Register	0x00000000
0x3100401C	PORTA_DIR_SET	PORTA Port x GPIO Direction Set Register	0x00000000
0x31004020	PORTA_DIR_CLR	PORTA Port x GPIO Direction Clear Register	0x00000000
0x31004024	PORTA_INEN	PORTA Port x GPIO Input Enable Register	0x00000000
0x31004028	PORTA_INEN_SET	PORTA Port x GPIO Input Enable Set Register	0x00000000
0x3100402C	PORTA_INEN_CLR	PORTA Port x GPIO Input Enable Clear Register	0x00000000
0x31004030	PORTA_MUX	PORTA Port x Multiplexer Control Register	0x00000000
0x31004034	PORTA_DATA_TGL	PORTA Port x GPIO Output Toggle Register	0x00000000
0x31004038	PORTA_POL	PORTA Port x GPIO Polarity Invert Register	0x00000000
0x3100403C	PORTA_POL_SET	PORTA Port x GPIO Polarity Invert Set Register	0x00000000
0x31004040	PORTA_POL_CLR	PORTA Port x GPIO Polarity Invert Clear Register	0x00000000
0x31004044	PORTA_LOCK	PORTA Port x GPIO Lock Register	0x00000000
0x31004048	PORTA_TRIG_TGL	PORTA Port x GPIO Trigger Toggle Register	0x00000000

Table A-129: ADSP-2159x PORTB MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004080	PORTB_FER	PORTB Port x Function Enable Register	0x00000000
0x31004084	PORTB_FER_SET	PORTB Port x Function Enable Set Register	0x00000000
0x31004088	PORTB_FER_CLR	PORTB Port x Function Enable Clear Register	0x00000000
0x3100408C	PORTB_DATA	PORTB Port x GPIO Data Register	0x00000000
0x31004090	PORTB_DATA_SET	PORTB Port x GPIO Data Set Register	0x00000000
0x31004094	PORTB_DATA_CLR	PORTB Port x GPIO Data Clear Register	0x00000000
0x31004098	PORTB_DIR	PORTB Port x GPIO Direction Register	0x00000000
0x3100409C	PORTB_DIR_SET	PORTB Port x GPIO Direction Set Register	0x00000000

Table A-129: ADSP-2159x PORTB MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310040A0	PORTB_DIR_CLR	PORTB Port x GPIO Direction Clear Register	0x00000000
0x310040A4	PORTB_INEN	PORTB Port x GPIO Input Enable Register	0x00000000
0x310040A8	PORTB_INEN_SET	PORTB Port x GPIO Input Enable Set Register	0x00000000
0x310040AC	PORTB_INEN_CLR	PORTB Port x GPIO Input Enable Clear Register	0x00000000
0x310040B0	PORTB_MUX	PORTB Port x Multiplexer Control Register	0x00000000
0x310040B4	PORTB_DATA_TGL	PORTB Port x GPIO Output Toggle Register	0x00000000
0x310040B8	PORTB_POL	PORTB Port x GPIO Polarity Invert Register	0x00000000
0x310040BC	PORTB_POL_SET	PORTB Port x GPIO Polarity Invert Set Register	0x00000000
0x310040C0	PORTB_POL_CLR	PORTB Port x GPIO Polarity Invert Clear Register	0x00000000
0x310040C4	PORTB_LOCK	PORTB Port x GPIO Lock Register	0x00000000
0x310040C8	PORTB_TRIG_TGL	PORTB Port x GPIO Trigger Toggle Register	0x00000000

Table A-130: ADSP-2159x PORTC MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004100	PORTC_FER	PORTC Port x Function Enable Register	0x00000000
0x31004104	PORTC_FER_SET	PORTC Port x Function Enable Set Register	0x00000000
0x31004108	PORTC_FER_CLR	PORTC Port x Function Enable Clear Register	0x00000000
0x3100410C	PORTC_DATA	PORTC Port x GPIO Data Register	0x00000000
0x31004110	PORTC_DATA_SET	PORTC Port x GPIO Data Set Register	0x00000000
0x31004114	PORTC_DATA_CLR	PORTC Port x GPIO Data Clear Register	0x00000000
0x31004118	PORTC_DIR	PORTC Port x GPIO Direction Register	0x00000000
0x3100411C	PORTC_DIR_SET	PORTC Port x GPIO Direction Set Register	0x00000000
0x31004120	PORTC_DIR_CLR	PORTC Port x GPIO Direction Clear Register	0x00000000
0x31004124	PORTC_INEN	PORTC Port x GPIO Input Enable Register	0x00000000
0x31004128	PORTC_INEN_SET	PORTC Port x GPIO Input Enable Set Register	0x00000000
0x3100412C	PORTC_INEN_CLR	PORTC Port x GPIO Input Enable Clear Register	0x00000000
0x31004130	PORTC_MUX	PORTC Port x Multiplexer Control Register	0x00000000
0x31004134	PORTC_DATA_TGL	PORTC Port x GPIO Output Toggle Register	0x00000000
0x31004138	PORTC_POL	PORTC Port x GPIO Polarity Invert Register	0x00000000
0x3100413C	PORTC_POL_SET	PORTC Port x GPIO Polarity Invert Set Register	0x00000000

Table A-130: ADSP-2159x PORTC MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004140	PORTC_POL_CLR	PORTC Port x GPIO Polarity Invert Clear Register	0x00000000
0x31004144	PORTC_LOCK	PORTC Port x GPIO Lock Register	0x00000000
0x31004148	PORTC_TRIG_TGL	PORTC Port x GPIO Trigger Toggle Register	0x00000000

Table A-131: ADSP-2159x PORTD MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004180	PORTD_FER	PORTD Port x Function Enable Register	0x00000000
0x31004184	PORTD_FER_SET	PORTD Port x Function Enable Set Register	0x00000000
0x31004188	PORTD_FER_CLR	PORTD Port x Function Enable Clear Register	0x00000000
0x3100418C	PORTD_DATA	PORTD Port x GPIO Data Register	0x00000000
0x31004190	PORTD_DATA_SET	PORTD Port x GPIO Data Set Register	0x00000000
0x31004194	PORTD_DATA_CLR	PORTD Port x GPIO Data Clear Register	0x00000000
0x31004198	PORTD_DIR	PORTD Port x GPIO Direction Register	0x00000000
0x3100419C	PORTD_DIR_SET	PORTD Port x GPIO Direction Set Register	0x00000000
0x310041A0	PORTD_DIR_CLR	PORTD Port x GPIO Direction Clear Register	0x00000000
0x310041A4	PORTD_INEN	PORTD Port x GPIO Input Enable Register	0x00000000
0x310041A8	PORTD_INEN_SET	PORTD Port x GPIO Input Enable Set Register	0x00000000
0x310041AC	PORTD_INEN_CLR	PORTD Port x GPIO Input Enable Clear Register	0x00000000
0x310041B0	PORTD_MUX	PORTD Port x Multiplexer Control Register	0x00000000
0x310041B4	PORTD_DATA_TGL	PORTD Port x GPIO Output Toggle Register	0x00000000
0x310041B8	PORTD_POL	PORTD Port x GPIO Polarity Invert Register	0x00000000
0x310041BC	PORTD_POL_SET	PORTD Port x GPIO Polarity Invert Set Register	0x00000000
0x310041C0	PORTD_POL_CLR	PORTD Port x GPIO Polarity Invert Clear Register	0x00000000
0x310041C4	PORTD_LOCK	PORTD Port x GPIO Lock Register	0x00000000
0x310041C8	PORTD_TRIG_TGL	PORTD Port x GPIO Trigger Toggle Register	0x00000000

Table A-132: ADSP-2159x PORTE MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004200	PORTE_FER	PORTE Port x Function Enable Register	0x00000000
0x31004204	PORTE_FER_SET	PORTE Port x Function Enable Set Register	0x00000000

Table A-132: ADSP-2159x PORTE MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004208	PORTE_FER_CLR	PORTE Port x Function Enable Clear Register	0x00000000
0x3100420C	PORTE_DATA	PORTE Port x GPIO Data Register	0x00000000
0x31004210	PORTE_DATA_SET	PORTE Port x GPIO Data Set Register	0x00000000
0x31004214	PORTE_DATA_CLR	PORTE Port x GPIO Data Clear Register	0x00000000
0x31004218	PORTE_DIR	PORTE Port x GPIO Direction Register	0x00000000
0x3100421C	PORTE_DIR_SET	PORTE Port x GPIO Direction Set Register	0x00000000
0x31004220	PORTE_DIR_CLR	PORTE Port x GPIO Direction Clear Register	0x00000000
0x31004224	PORTE_INEN	PORTE Port x GPIO Input Enable Register	0x00000000
0x31004228	PORTE_INEN_SET	PORTE Port x GPIO Input Enable Set Register	0x00000000
0x3100422C	PORTE_INEN_CLR	PORTE Port x GPIO Input Enable Clear Register	0x00000000
0x31004230	PORTE_MUX	PORTE Port x Multiplexer Control Register	0x00000000
0x31004234	PORTE_DATA_TGL	PORTE Port x GPIO Output Toggle Register	0x00000000
0x31004238	PORTE_POL	PORTE Port x GPIO Polarity Invert Register	0x00000000
0x3100423C	PORTE_POL_SET	PORTE Port x GPIO Polarity Invert Set Register	0x00000000
0x31004240	PORTE_POL_CLR	PORTE Port x GPIO Polarity Invert Clear Register	0x00000000
0x31004244	PORTE_LOCK	PORTE Port x GPIO Lock Register	0x00000000
0x31004248	PORTE_TRIG_TGL	PORTE Port x GPIO Trigger Toggle Register	0x00000000

Table A-133: ADSP-2159x PORTF MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004280	PORTF_FER	PORTF Port x Function Enable Register	0x00000000
0x31004284	PORTF_FER_SET	PORTF Port x Function Enable Set Register	0x00000000
0x31004288	PORTF_FER_CLR	PORTF Port x Function Enable Clear Register	0x00000000
0x3100428C	PORTF_DATA	PORTF Port x GPIO Data Register	0x00000000
0x31004290	PORTF_DATA_SET	PORTF Port x GPIO Data Set Register	0x00000000
0x31004294	PORTF_DATA_CLR	PORTF Port x GPIO Data Clear Register	0x00000000
0x31004298	PORTF_DIR	PORTF Port x GPIO Direction Register	0x00000000
0x3100429C	PORTF_DIR_SET	PORTF Port x GPIO Direction Set Register	0x00000000
0x310042A0	PORTF_DIR_CLR	PORTF Port x GPIO Direction Clear Register	0x00000000
0x310042A4	PORTF_INEN	PORTF Port x GPIO Input Enable Register	0x00000000

Table A-133: ADSP-2159x PORTF MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310042A8	PORTF_INEN_SET	PORTF Port x GPIO Input Enable Set Register	0x00000000
0x310042AC	PORTF_INEN_CLR	PORTF Port x GPIO Input Enable Clear Register	0x00000000
0x310042B0	PORTF_MUX	PORTF Port x Multiplexer Control Register	0x00000000
0x310042B4	PORTF_DATA_TGL	PORTF Port x GPIO Output Toggle Register	0x00000000
0x310042B8	PORTF_POL	PORTF Port x GPIO Polarity Invert Register	0x00000000
0x310042BC	PORTF_POL_SET	PORTF Port x GPIO Polarity Invert Set Register	0x00000000
0x310042C0	PORTF_POL_CLR	PORTF Port x GPIO Polarity Invert Clear Register	0x00000000
0x310042C4	PORTF_LOCK	PORTF Port x GPIO Lock Register	0x00000000
0x310042C8	PORTF_TRIG_TGL	PORTF Port x GPIO Trigger Toggle Register	0x00000000

Table A-134: ADSP-2159x PORTG MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004300	PORTG_FER	PORTG Port x Function Enable Register	0x00000000
0x31004304	PORTG_FER_SET	PORTG Port x Function Enable Set Register	0x00000000
0x31004308	PORTG_FER_CLR	PORTG Port x Function Enable Clear Register	0x00000000
0x3100430C	PORTG_DATA	PORTG Port x GPIO Data Register	0x00000000
0x31004310	PORTG_DATA_SET	PORTG Port x GPIO Data Set Register	0x00000000
0x31004314	PORTG_DATA_CLR	PORTG Port x GPIO Data Clear Register	0x00000000
0x31004318	PORTG_DIR	PORTG Port x GPIO Direction Register	0x00000000
0x3100431C	PORTG_DIR_SET	PORTG Port x GPIO Direction Set Register	0x00000000
0x31004320	PORTG_DIR_CLR	PORTG Port x GPIO Direction Clear Register	0x00000000
0x31004324	PORTG_INEN	PORTG Port x GPIO Input Enable Register	0x00000000
0x31004328	PORTG_INEN_SET	PORTG Port x GPIO Input Enable Set Register	0x00000000
0x3100432C	PORTG_INEN_CLR	PORTG Port x GPIO Input Enable Clear Register	0x00000000
0x31004330	PORTG_MUX	PORTG Port x Multiplexer Control Register	0x00000000
0x31004334	PORTG_DATA_TGL	PORTG Port x GPIO Output Toggle Register	0x00000000
0x31004338	PORTG_POL	PORTG Port x GPIO Polarity Invert Register	0x00000000
0x3100433C	PORTG_POL_SET	PORTG Port x GPIO Polarity Invert Set Register	0x00000000
0x31004340	PORTG_POL_CLR	PORTG Port x GPIO Polarity Invert Clear Register	0x00000000
0x31004344	PORTG_LOCK	PORTG Port x GPIO Lock Register	0x00000000

Table A-134: ADSP-2159x PORTG MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004348	PORTG_TRIG_TGL	PORTG Port x GPIO Trigger Toggle Register	0x00000000

Table A-135: ADSP-2159x PORTH MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004380	PORTH_FER	PORTH Port x Function Enable Register	0x00000000
0x31004384	PORTH_FER_SET	PORTH Port x Function Enable Set Register	0x00000000
0x31004388	PORTH_FER_CLR	PORTH Port x Function Enable Clear Register	0x00000000
0x3100438C	PORTH_DATA	PORTH Port x GPIO Data Register	0x00000000
0x31004390	PORTH_DATA_SET	PORTH Port x GPIO Data Set Register	0x00000000
0x31004394	PORTH_DATA_CLR	PORTH Port x GPIO Data Clear Register	0x00000000
0x31004398	PORTH_DIR	PORTH Port x GPIO Direction Register	0x00000000
0x3100439C	PORTH_DIR_SET	PORTH Port x GPIO Direction Set Register	0x00000000
0x310043A0	PORTH_DIR_CLR	PORTH Port x GPIO Direction Clear Register	0x00000000
0x310043A4	PORTH_INEN	PORTH Port x GPIO Input Enable Register	0x00000000
0x310043A8	PORTH_INEN_SET	PORTH Port x GPIO Input Enable Set Register	0x00000000
0x310043AC	PORTH_INEN_CLR	PORTH Port x GPIO Input Enable Clear Register	0x00000000
0x310043B0	PORTH_MUX	PORTH Port x Multiplexer Control Register	0x00000000
0x310043B4	PORTH_DATA_TGL	PORTH Port x GPIO Output Toggle Register	0x00000000
0x310043B8	PORTH_POL	PORTH Port x GPIO Polarity Invert Register	0x00000000
0x310043BC	PORTH_POL_SET	PORTH Port x GPIO Polarity Invert Set Register	0x00000000
0x310043C0	PORTH_POL_CLR	PORTH Port x GPIO Polarity Invert Clear Register	0x00000000
0x310043C4	PORTH_LOCK	PORTH Port x GPIO Lock Register	0x00000000
0x310043C8	PORTH_TRIG_TGL	PORTH Port x GPIO Trigger Toggle Register	0x00000000

Table A-136: ADSP-2159x PORTI MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31004400	PORTI_FER	PORTI Port x Function Enable Register	0x00000000
0x31004404	PORTI_FER_SET	PORTI Port x Function Enable Set Register	0x00000000
0x31004408	PORTI_FER_CLR	PORTI Port x Function Enable Clear Register	0x00000000
0x3100440C	PORTI_DATA	PORTI Port x GPIO Data Register	0x00000000

Table A-136: ADSP-2159x PORTI MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31004410	PORTI_DATA_SET	PORTI Port x GPIO Data Set Register	0x00000000
0x31004414	PORTI_DATA_CLR	PORTI Port x GPIO Data Clear Register	0x00000000
0x31004418	PORTI_DIR	PORTI Port x GPIO Direction Register	0x00000000
0x3100441C	PORTI_DIR_SET	PORTI Port x GPIO Direction Set Register	0x00000000
0x31004420	PORTI_DIR_CLR	PORTI Port x GPIO Direction Clear Register	0x00000000
0x31004424	PORTI_INEN	PORTI Port x GPIO Input Enable Register	0x00000000
0x31004428	PORTI_INEN_SET	PORTI Port x GPIO Input Enable Set Register	0x00000000
0x3100442C	PORTI_INEN_CLR	PORTI Port x GPIO Input Enable Clear Register	0x00000000
0x31004430	PORTI_MUX	PORTI Port x Multiplexer Control Register	0x00000000
0x31004434	PORTI_DATA_TGL	PORTI Port x GPIO Output Toggle Register	0x00000000
0x31004438	PORTI_POL	PORTI Port x GPIO Polarity Invert Register	0x00000000
0x3100443C	PORTI_POL_SET	PORTI Port x GPIO Polarity Invert Set Register	0x00000000
0x31004440	PORTI_POL_CLR	PORTI Port x GPIO Polarity Invert Clear Register	0x00000000
0x31004444	PORTI_LOCK	PORTI Port x GPIO Lock Register	0x00000000
0x31004448	PORTI_TRIG_TGL	PORTI Port x GPIO Trigger Toggle Register	0x00000000

Table A-137: ADSP-2159x RCU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108C000	RCU0_CTL	RCU0 Control Register	0x00000700
0x3108C004	RCU0_STAT	RCU0 Status Register	0x00000021
0x3108C008	RCU0_CRCTL	RCU0 Core Reset Outputs Control Register	0x00000006
0x3108C00C	RCU0_CRSTAT	RCU0 Core Reset Outputs Status Register	0x00000006
0x3108C018	RCU0_SRRQSTAT	RCU0 System Reset Request Status Register	0x00000000
0x3108C01C	RCU0_SIDIS	RCU0 System Interface Disable Register	0x00000000
0x3108C020	RCU0_SISTAT	RCU0 System Interface Status Register	0x00000000
0x3108C024	RCU0_SVECT_LCK	RCU0 SVECT Lock Register	0x00000000
0x3108C028	RCU0_BCODE	RCU0 Boot Code Register	0x00000000
0x3108C02C	RCU0_SVECT0	RCU0 Software Vector Register 0	0x00000040
0x3108C030	RCU0_SVECT1	RCU0 Software Vector Register 1	0x00500004
0x3108C034	RCU0_SVECT2	RCU0 Software Vector Register 2	0x00500004

Table A-137: ADSP-2159x RCU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108C06C	RCU0_MSG	RCU0 Message Register	0x00000000
0x3108C070	RCU0_MSG_SET	RCU0 Message Set Bits Register	0x00000000
0x3108C074	RCU0_MSG_CLR	RCU0 Message Clear Bits Register	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089000	SEC0_GCTL	SEC0 Global Control Register	0x00000000
0x31089004	SEC0_GSTAT	SEC0 Global Status Register	0x00000000
0x31089008	SEC0_RAISE	SEC0 Global Raise Register	0x00000000
0x3108900C	SEC0_END	SEC0 Global End Register	0x00000000
0x31089010	SEC0_FCTL	SEC0 Fault Control Register	0x00000000
0x31089014	SEC0_FSTAT	SEC0 Fault Status Register	0x00000000
0x31089018	SEC0_FSID	SEC0 Fault Source ID Register	0x00000000
0x3108901C	SEC0_FEND	SEC0 Fault End Register	0x00000000
0x31089020	SEC0_FDLY	SEC0 Fault Delay Register	0x00000000
0x31089024	SEC0_FDLY_CUR	SEC0 Fault Delay Current Register	0x00000000
0x31089028	SEC0_FSRDLY	SEC0 Fault System Reset Delay Register	0x00000000
0x3108902C	SEC0_FSRDLY_CUR	SEC0 Fault System Reset Delay Current Register	0x00000000
0x31089030	SEC0_FCOPP	SEC0 Fault COP Period Register	0x00000000
0x31089034	SEC0_FCOPP_CUR	SEC0 Fault COP Period Current Register	0x00000000
0x31089440	SEC0_CCTL[n]	SEC0 SCI Control Register n	0x00000000
0x31089444	SEC0_CSTAT[n]	SEC0 SCI Status Register n	0x00000000
0x31089448	SEC0_CPND[n]	SEC0 Core Pending Register n	0x00000000
0x3108944C	SEC0_CACT[n]	SEC0 SCI Active Register n	0x00000000
0x31089450	SEC0_CPMASK[n]	SEC0 SCI Priority Mask Register n	0x000000FF
0x31089454	SEC0_CGMSK[n]	SEC0 SCI Group Mask Register n	0x00000000
0x31089458	SEC0_CPLVL[n]	SEC0 SCI Priority Level Register n	0x00000007
0x3108945C	SEC0_CSID[n]	SEC0 SCI Source ID Register n	0x00000000
0x31089480	SEC0_CCTL[n]	SEC0 SCI Control Register n	0x00000000
0x31089484	SEC0_CSTAT[n]	SEC0 SCI Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089488	SEC0_CPND[n]	SEC0 Core Pending Register n	0x00000000
0x3108948C	SEC0_CACT[n]	SEC0 SCI Active Register n	0x00000000
0x31089490	SEC0_CPMASK[n]	SEC0 SCI Priority Mask Register n	0x000000FF
0x31089494	SEC0_CGMSK[n]	SEC0 SCI Group Mask Register n	0x00000000
0x31089498	SEC0_CPLVL[n]	SEC0 SCI Priority Level Register n	0x00000007
0x3108949C	SEC0_CSID[n]	SEC0 SCI Source ID Register n	0x00000000
0x31089800	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089804	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089808	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108980C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089810	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089814	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089818	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108981C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089820	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089824	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089828	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108982C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089830	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089834	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089838	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108983C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089840	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089844	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089848	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108984C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089850	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089854	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089858	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108985C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089860	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089864	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089868	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108986C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089870	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089874	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089878	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108987C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089880	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089884	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089888	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108988C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089890	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089894	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089898	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108989C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898A0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898A4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898A8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898AC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898B0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898B4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898B8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898BC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898C0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898C4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898C8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898CC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898D0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898D4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898D8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898DC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310898E0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898E4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898E8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898EC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898F0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898F4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310898F8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310898FC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089900	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089904	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089908	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108990C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089910	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089914	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089918	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108991C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089920	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089924	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089928	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108992C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089930	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089934	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089938	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108993C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089940	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089944	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089948	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108994C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089950	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089954	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089958	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108995C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089960	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089964	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089968	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108996C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089970	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089974	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089978	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108997C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089980	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089984	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089988	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108998C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089990	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089994	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089998	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x3108999C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899A0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899A4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899A8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899AC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899B0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899B4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899B8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899BC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899C0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899C4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899C8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899CC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899D0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899D4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310899D8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899DC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899E0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899E4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899E8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899EC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899F0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899F4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x310899F8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x310899FC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089A54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089A98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089A9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089ABC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089ACC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089AD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089ADC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089AF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089AFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089B4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089B98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089B9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BBC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089BC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BCC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BDC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089BF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089BFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089C44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089C98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089C9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CBC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089CC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CCC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CDC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089CF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089CFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089D3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089D98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089D9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089DB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DBC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DCC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DDC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089DF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089DFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089E34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089E98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089E9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089EB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EBC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089ECC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089ED0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089ED4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089ED8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EDC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089EF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089EFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F00	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F04	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F08	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F0C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F10	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F14	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F18	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F1C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F20	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F24	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F28	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089F2C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F30	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F34	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F38	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F3C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F40	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F44	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F48	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F4C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F50	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F54	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F58	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F5C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F60	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F64	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F68	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F6C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F70	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F74	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F78	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F7C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F80	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F84	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F88	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F8C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F90	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F94	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089F98	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089F9C	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FA0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FA4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-138: ADSP-2159x SEC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31089FA8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FAC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FB0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FB4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FB8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FBC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FC0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FC4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FC8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FCC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FD0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FD4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FD8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FDC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FE0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FE4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FE8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FEC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FF0	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FF4	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000
0x31089FF8	SEC0_SCTL[n]	SEC0 Source Control Register n	0x00000000
0x31089FFC	SEC0_SSTAT[n]	SEC0 Source Status Register n	0x00000000

Table A-139: ADSP-2159x SMC0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3100600C	SMC0_B0CTL	SMC0 Bank 0 Control Register	0x01000000
0x31006010	SMC0_B0TIM	SMC0 Bank 0 Timing Register	0x01010101
0x31006014	SMC0_B0ETIM	SMC0 Bank 0 Extended Timing Register	0x00020200
0x3100601C	SMC0_B1CTL	SMC0 Bank 1 Control Register	0x01000000
0x31006020	SMC0_B1TIM	SMC0 Bank 1 Timing Register	0x01010101

Table A-139: ADSP-2159x SMC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31006024	SMC0_B1ETIM	SMC0 Bank 1 Extended Timing Register	0x00020200
0x3100602C	SMC0_B2CTL	SMC0 Bank 2 Control Register	0x01000000
0x31006030	SMC0_B2TIM	SMC0 Bank 2 Timing Register	0x01010101
0x31006034	SMC0_B2ETIM	SMC0 Bank 2 Extended Timing Register	0x00020200
0x3100603C	SMC0_B3CTL	SMC0 Bank 3 Control Register	0x01000000
0x31006040	SMC0_B3TIM	SMC0 Bank 3 Timing Register	0x01010101
0x31006044	SMC0_B3ETIM	SMC0 Bank 3 Extended Timing Register	0x00020200

Table A-140: ADSP-2159x SMPU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31007000	SMPU0_CTL	SMPU0 SMPU Control Register	0x00000000
0x31007004	SMPU0_STAT	SMPU0 SMPU Status Register	0x00000000
0x31007008	SMPU0_IADDR	SMPU0 Interrupt Address Register	0x00000000
0x3100700C	SMPU0_IDTLS	SMPU0 Interrupt Details Register	0x00000000
0x31007010	SMPU0_BADDR	SMPU0 Bus Error Address Register	0x00000000
0x31007014	SMPU0_BDTLS	SMPU0 Bus Error Details Register	0x00000000
0x31007020	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x31007024	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007028	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x3100702C	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007030	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x31007034	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007038	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x3100703C	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007040	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x31007044	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007048	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x3100704C	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007050	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x31007054	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000

Table A-140: ADSP-2159x SMPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31007058	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x3100705C	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007060	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x31007064	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007068	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x3100706C	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007070	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x31007074	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007078	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x3100707C	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007080	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x31007084	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x31007088	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x3100708C	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x31007090	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x31007094	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007098	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x3100709C	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x310070A0	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x310070A4	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x310070A8	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x310070AC	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x310070B0	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x310070B4	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x310070B8	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000
0x310070BC	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x310070C0	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x310070C4	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x310070C8	SMPU0_RCTL[n]	SMPU0 Region n Control Register	0x00000000
0x310070CC	SMPU0_RADDR[n]	SMPU0 Region n Address Register	0x00000000
0x310070D0	SMPU0_RIDA[n]	SMPU0 Region n ID A Register	0x00000000

Table A-140: ADSP-2159x SMPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310070D4	SMPU0_RIDMSKA[n]	SMPU0 Region n ID Mask A Register	0x00000000
0x310070D8	SMPU0_RIDB[n]	SMPU0 Region n ID B Register	0x00000000
0x310070DC	SMPU0_RIDMSKB[n]	SMPU0 Region n ID Mask B Register	0x00000000
0x31007220	SMPU0_REVID	SMPU0 SMPU Revision ID Register	0x00000010
0x31007800	SMPU0_SECURECTL	SMPU0 SMPU Control Secure Accesses Register	0x00000000
0x31007820	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007824	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007828	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x3100782C	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007830	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007834	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x31007838	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000
0x3100783C	SMPU0_SECURERCTL[n]	SMPU0 Region n Control Secure Accesses Register	0x00000000

Table A-141: ADSP-2159x SMPU11 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A1000	SMPU11_CTL	SMPU11 SMPU Control Register	0x00000000
0x310A1004	SMPU11_STAT	SMPU11 SMPU Status Register	0x00000000
0x310A1008	SMPU11_IADDR	SMPU11 Interrupt Address Register	0x00000000
0x310A100C	SMPU11_IDTLS	SMPU11 Interrupt Details Register	0x00000000
0x310A1010	SMPU11_BADDR	SMPU11 Bus Error Address Register	0x00000000
0x310A1014	SMPU11_BDTLS	SMPU11 Bus Error Details Register	0x00000000
0x310A1020	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A1024	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A1028	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A102C	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1030	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A1034	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1038	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A103C	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000

Table A-141: ADSP-2159x SMPU11 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A1040	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A1044	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1048	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A104C	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1050	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A1054	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A1058	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A105C	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1060	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A1064	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1068	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A106C	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A1070	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A1074	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1078	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A107C	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1080	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A1084	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A1088	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A108C	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A1090	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A1094	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1098	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A109C	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A10A0	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A10A4	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A10A8	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A10AC	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A10B0	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A10B4	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A10B8	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000

Table A-141: ADSP-2159x SMPU11 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A10BC	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A10C0	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A10C4	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A10C8	SMPU11_RCTL[n]	SMPU11 Region n Control Register	0x00000000
0x310A10CC	SMPU11_RADDR[n]	SMPU11 Region n Address Register	0x00000000
0x310A10D0	SMPU11_RIDA[n]	SMPU11 Region n ID A Register	0x00000000
0x310A10D4	SMPU11_RIDMSKA[n]	SMPU11 Region n ID Mask A Register	0x00000000
0x310A10D8	SMPU11_RIDB[n]	SMPU11 Region n ID B Register	0x00000000
0x310A10DC	SMPU11_RIDMSKB[n]	SMPU11 Region n ID Mask B Register	0x00000000
0x310A1220	SMPU11_REVID	SMPU11 SMPU Revision ID Register	0x00000010
0x310A1800	SMPU11_SECURECTL	SMPU11 SMPU Control Secure Accesses Register	0x00000000
0x310A1820	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1824	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1828	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A182C	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1830	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1834	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A1838	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000
0x310A183C	SMPU11_SECURERCTL[n]	SMPU11 Region n Control Secure Accesses Register	0x00000000

Table A-142: ADSP-2159x SMPU12 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31012000	SMPU12_CTL	SMPU12 SMPU Control Register	0x00000000
0x31012004	SMPU12_STAT	SMPU12 SMPU Status Register	0x00000000
0x31012008	SMPU12_IADDR	SMPU12 Interrupt Address Register	0x00000000
0x3101200C	SMPU12_IDTLS	SMPU12 Interrupt Details Register	0x00000000
0x31012010	SMPU12_BADDR	SMPU12 Bus Error Address Register	0x00000000
0x31012014	SMPU12_BDTLS	SMPU12 Bus Error Details Register	0x00000000
0x31012020	SMPU12_RCTL[n]	SMPU12 Region n Control Register	0x00000000
0x31012024	SMPU12_RADDR[n]	SMPU12 Region n Address Register	0x00000000

Table A-142: ADSP-2159x SMPU12 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31012028	SMPU12_RIDA[n]	SMPU12 Region n ID A Register	0x00000000
0x3101202C	SMPU12_RIDMSKA[n]	SMPU12 Region n ID Mask A Register	0x00000000
0x31012030	SMPU12_RIDB[n]	SMPU12 Region n ID B Register	0x00000000
0x31012034	SMPU12_RIDMSKB[n]	SMPU12 Region n ID Mask B Register	0x00000000
0x31012038	SMPU12_RCTL[n]	SMPU12 Region n Control Register	0x00000000
0x3101203C	SMPU12_RADDR[n]	SMPU12 Region n Address Register	0x00000000
0x31012040	SMPU12_RIDA[n]	SMPU12 Region n ID A Register	0x00000000
0x31012044	SMPU12_RIDMSKA[n]	SMPU12 Region n ID Mask A Register	0x00000000
0x31012048	SMPU12_RIDB[n]	SMPU12 Region n ID B Register	0x00000000
0x3101204C	SMPU12_RIDMSKB[n]	SMPU12 Region n ID Mask B Register	0x00000000
0x31012050	SMPU12_RCTL[n]	SMPU12 Region n Control Register	0x00000000
0x31012054	SMPU12_RADDR[n]	SMPU12 Region n Address Register	0x00000000
0x31012058	SMPU12_RIDA[n]	SMPU12 Region n ID A Register	0x00000000
0x3101205C	SMPU12_RIDMSKA[n]	SMPU12 Region n ID Mask A Register	0x00000000
0x31012060	SMPU12_RIDB[n]	SMPU12 Region n ID B Register	0x00000000
0x31012064	SMPU12_RIDMSKB[n]	SMPU12 Region n ID Mask B Register	0x00000000
0x31012068	SMPU12_RCTL[n]	SMPU12 Region n Control Register	0x00000000
0x3101206C	SMPU12_RADDR[n]	SMPU12 Region n Address Register	0x00000000
0x31012070	SMPU12_RIDA[n]	SMPU12 Region n ID A Register	0x00000000
0x31012074	SMPU12_RIDMSKA[n]	SMPU12 Region n ID Mask A Register	0x00000000
0x31012078	SMPU12_RIDB[n]	SMPU12 Region n ID B Register	0x00000000
0x3101207C	SMPU12_RIDMSKB[n]	SMPU12 Region n ID Mask B Register	0x00000000
0x31012080	SMPU12_RCTL[n]	SMPU12 Region n Control Register	0x00000000
0x31012084	SMPU12_RADDR[n]	SMPU12 Region n Address Register	0x00000000
0x31012088	SMPU12_RIDA[n]	SMPU12 Region n ID A Register	0x00000000
0x3101208C	SMPU12_RIDMSKA[n]	SMPU12 Region n ID Mask A Register	0x00000000
0x31012090	SMPU12_RIDB[n]	SMPU12 Region n ID B Register	0x00000000
0x31012094	SMPU12_RIDMSKB[n]	SMPU12 Region n ID Mask B Register	0x00000000
0x31012098	SMPU12_RCTL[n]	SMPU12 Region n Control Register	0x00000000
0x3101209C	SMPU12_RADDR[n]	SMPU12 Region n Address Register	0x00000000
0x310120A0	SMPU12_RIDA[n]	SMPU12 Region n ID A Register	0x00000000

Table A-142: ADSP-2159x SMPU12 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310120A4	SMPU12_RIDMSKA[n]	SMPU12 Region n ID Mask A Register	0x00000000
0x310120A8	SMPU12_RIDB[n]	SMPU12 Region n ID B Register	0x00000000
0x310120AC	SMPU12_RIDMSKB[n]	SMPU12 Region n ID Mask B Register	0x00000000
0x310120B0	SMPU12_RCTL[n]	SMPU12 Region n Control Register	0x00000000
0x310120B4	SMPU12_RADDR[n]	SMPU12 Region n Address Register	0x00000000
0x310120B8	SMPU12_RIDA[n]	SMPU12 Region n ID A Register	0x00000000
0x310120BC	SMPU12_RIDMSKA[n]	SMPU12 Region n ID Mask A Register	0x00000000
0x310120C0	SMPU12_RIDB[n]	SMPU12 Region n ID B Register	0x00000000
0x310120C4	SMPU12_RIDMSKB[n]	SMPU12 Region n ID Mask B Register	0x00000000
0x310120C8	SMPU12_RCTL[n]	SMPU12 Region n Control Register	0x00000000
0x310120CC	SMPU12_RADDR[n]	SMPU12 Region n Address Register	0x00000000
0x310120D0	SMPU12_RIDA[n]	SMPU12 Region n ID A Register	0x00000000
0x310120D4	SMPU12_RIDMSKA[n]	SMPU12 Region n ID Mask A Register	0x00000000
0x310120D8	SMPU12_RIDB[n]	SMPU12 Region n ID B Register	0x00000000
0x310120DC	SMPU12_RIDMSKB[n]	SMPU12 Region n ID Mask B Register	0x00000000
0x31012220	SMPU12_REVID	SMPU12 SMPU Revision ID Register	0x00000010
0x31012800	SMPU12_SECURECTL	SMPU12 SMPU Control Secure Accesses Register	0x00000000
0x31012820	SMPU12_SECURERCTL[n]	SMPU12 Region n Control Secure Accesses Register	0x00000000
0x31012824	SMPU12_SECURERCTL[n]	SMPU12 Region n Control Secure Accesses Register	0x00000000
0x31012828	SMPU12_SECURERCTL[n]	SMPU12 Region n Control Secure Accesses Register	0x00000000
0x3101282C	SMPU12_SECURERCTL[n]	SMPU12 Region n Control Secure Accesses Register	0x00000000
0x31012830	SMPU12_SECURERCTL[n]	SMPU12 Region n Control Secure Accesses Register	0x00000000
0x31012834	SMPU12_SECURERCTL[n]	SMPU12 Region n Control Secure Accesses Register	0x00000000
0x31012838	SMPU12_SECURERCTL[n]	SMPU12 Region n Control Secure Accesses Register	0x00000000
0x3101283C	SMPU12_SECURERCTL[n]	SMPU12 Region n Control Secure Accesses Register	0x00000000

Table A-143: ADSP-2159x SMPU2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31083000	SMPU2_CTL	SMPU2 SMPU Control Register	0x00000000
0x31083004	SMPU2_STAT	SMPU2 SMPU Status Register	0x00000000

Table A-143: ADSP-2159x SMPU2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31083008	SMPU2_IADDR	SMPU2 Interrupt Address Register	0x00000000
0x3108300C	SMPU2_IDTLS	SMPU2 Interrupt Details Register	0x00000000
0x31083010	SMPU2_BADDR	SMPU2 Bus Error Address Register	0x00000000
0x31083014	SMPU2_BDTLS	SMPU2 Bus Error Details Register	0x00000000
0x31083020	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x31083024	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083028	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x3108302C	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083030	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x31083034	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083038	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x3108303C	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083040	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x31083044	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083048	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x3108304C	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083050	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x31083054	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083058	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x3108305C	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083060	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x31083064	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083068	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x3108306C	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083070	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x31083074	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083078	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x3108307C	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083080	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x31083084	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x31083088	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000

Table A-143: ADSP-2159x SMPU2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108308C	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x31083090	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x31083094	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x31083098	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x3108309C	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x310830A0	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x310830A4	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x310830A8	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x310830AC	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x310830B0	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x310830B4	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x310830B8	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x310830BC	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x310830C0	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x310830C4	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x310830C8	SMPU2_RCTL[n]	SMPU2 Region n Control Register	0x00000000
0x310830CC	SMPU2_RADDR[n]	SMPU2 Region n Address Register	0x00000000
0x310830D0	SMPU2_RIDA[n]	SMPU2 Region n ID A Register	0x00000000
0x310830D4	SMPU2_RIDMSKA[n]	SMPU2 Region n ID Mask A Register	0x00000000
0x310830D8	SMPU2_RIDB[n]	SMPU2 Region n ID B Register	0x00000000
0x310830DC	SMPU2_RIDMSKB[n]	SMPU2 Region n ID Mask B Register	0x00000000
0x310831A0	SMPU2_EXACADD[n]	SMPU2 Exclusive Access IDn Address	0x00000000
0x310831A4	SMPU2_EXACSTAT[n]	SMPU2 Exclusive Access Status	0x00000000
0x310831A8	SMPU2_EXACADD[n]	SMPU2 Exclusive Access IDn Address	0x00000000
0x310831AC	SMPU2_EXACSTAT[n]	SMPU2 Exclusive Access Status	0x00000000
0x310831B0	SMPU2_EXACADD[n]	SMPU2 Exclusive Access IDn Address	0x00000000
0x310831B4	SMPU2_EXACSTAT[n]	SMPU2 Exclusive Access Status	0x00000000
0x31083220	SMPU2_REVID	SMPU2 SMPU Revision ID Register	0x00000010
0x31083800	SMPU2_SECURECTL	SMPU2 SMPU Control Secure Accesses Register	0x00000000
0x31083820	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083824	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000

Table A-143: ADSP-2159x SMPU2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31083828	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x3108382C	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083830	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083834	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x31083838	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000
0x3108383C	SMPU2_SECURERCTL[n]	SMPU2 Region n Control Secure Accesses Register	0x00000000

Table A-144: ADSP-2159x SMPU3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31084000	SMPU3_CTL	SMPU3 SMPU Control Register	0x00000000
0x31084004	SMPU3_STAT	SMPU3 SMPU Status Register	0x00000000
0x31084008	SMPU3_IADDR	SMPU3 Interrupt Address Register	0x00000000
0x3108400C	SMPU3_IDTLS	SMPU3 Interrupt Details Register	0x00000000
0x31084010	SMPU3_BADDR	SMPU3 Bus Error Address Register	0x00000000
0x31084014	SMPU3_BDTLS	SMPU3 Bus Error Details Register	0x00000000
0x31084020	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x31084024	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084028	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x3108402C	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084030	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x31084034	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084038	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x3108403C	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084040	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x31084044	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084048	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x3108404C	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084050	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x31084054	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084058	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000

Table A-144: ADSP-2159x SMPU3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108405C	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084060	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x31084064	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084068	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x3108406C	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084070	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x31084074	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084078	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x3108407C	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084080	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x31084084	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x31084088	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x3108408C	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x31084090	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x31084094	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084098	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x3108409C	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x310840A0	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x310840A4	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x310840A8	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x310840AC	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x310840B0	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x310840B4	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x310840B8	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x310840BC	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000
0x310840C0	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x310840C4	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x310840C8	SMPU3_RCTL[n]	SMPU3 Region n Control Register	0x00000000
0x310840CC	SMPU3_RADDR[n]	SMPU3 Region n Address Register	0x00000000
0x310840D0	SMPU3_RIDA[n]	SMPU3 Region n ID A Register	0x00000000
0x310840D4	SMPU3_RIDMSKA[n]	SMPU3 Region n ID Mask A Register	0x00000000

Table A-144: ADSP-2159x SMPU3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310840D8	SMPU3_RIDB[n]	SMPU3 Region n ID B Register	0x00000000
0x310840DC	SMPU3_RIDMSKB[n]	SMPU3 Region n ID Mask B Register	0x00000000
0x31084220	SMPU3_REVID	SMPU3 SMPU Revision ID Register	0x00000010
0x31084800	SMPU3_SECURECTL	SMPU3 SMPU Control Secure Accesses Register	0x00000000
0x31084820	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084824	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084828	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x3108482C	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084830	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084834	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x31084838	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000
0x3108483C	SMPU3_SECURERCTL[n]	SMPU3 Region n Control Secure Accesses Register	0x00000000

Table A-145: ADSP-2159x SMPU4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31085000	SMPU4_CTL	SMPU4 SMPU Control Register	0x00000000
0x31085004	SMPU4_STAT	SMPU4 SMPU Status Register	0x00000000
0x31085008	SMPU4_IADDR	SMPU4 Interrupt Address Register	0x00000000
0x3108500C	SMPU4_IDTLS	SMPU4 Interrupt Details Register	0x00000000
0x31085010	SMPU4_BADDR	SMPU4 Bus Error Address Register	0x00000000
0x31085014	SMPU4_BDTLS	SMPU4 Bus Error Details Register	0x00000000
0x31085020	SMPU4_RCTL[n]	SMPU4 Region n Control Register	0x00000000
0x31085024	SMPU4_RADDR[n]	SMPU4 Region n Address Register	0x00000000
0x31085028	SMPU4_RIDA[n]	SMPU4 Region n ID A Register	0x00000000
0x3108502C	SMPU4_RIDMSKA[n]	SMPU4 Region n ID Mask A Register	0x00000000
0x31085030	SMPU4_RIDB[n]	SMPU4 Region n ID B Register	0x00000000
0x31085034	SMPU4_RIDMSKB[n]	SMPU4 Region n ID Mask B Register	0x00000000
0x31085038	SMPU4_RCTL[n]	SMPU4 Region n Control Register	0x00000000
0x3108503C	SMPU4_RADDR[n]	SMPU4 Region n Address Register	0x00000000
0x31085040	SMPU4_RIDA[n]	SMPU4 Region n ID A Register	0x00000000

Table A-145: ADSP-2159x SMPU4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31085044	SMPU4_RIDMSKA[n]	SMPU4 Region n ID Mask A Register	0x00000000
0x31085048	SMPU4_RIDB[n]	SMPU4 Region n ID B Register	0x00000000
0x3108504C	SMPU4_RIDMSKB[n]	SMPU4 Region n ID Mask B Register	0x00000000
0x31085050	SMPU4_RCTL[n]	SMPU4 Region n Control Register	0x00000000
0x31085054	SMPU4_RADDR[n]	SMPU4 Region n Address Register	0x00000000
0x31085058	SMPU4_RIDA[n]	SMPU4 Region n ID A Register	0x00000000
0x3108505C	SMPU4_RIDMSKA[n]	SMPU4 Region n ID Mask A Register	0x00000000
0x31085060	SMPU4_RIDB[n]	SMPU4 Region n ID B Register	0x00000000
0x31085064	SMPU4_RIDMSKB[n]	SMPU4 Region n ID Mask B Register	0x00000000
0x31085068	SMPU4_RCTL[n]	SMPU4 Region n Control Register	0x00000000
0x3108506C	SMPU4_RADDR[n]	SMPU4 Region n Address Register	0x00000000
0x31085070	SMPU4_RIDA[n]	SMPU4 Region n ID A Register	0x00000000
0x31085074	SMPU4_RIDMSKA[n]	SMPU4 Region n ID Mask A Register	0x00000000
0x31085078	SMPU4_RIDB[n]	SMPU4 Region n ID B Register	0x00000000
0x3108507C	SMPU4_RIDMSKB[n]	SMPU4 Region n ID Mask B Register	0x00000000
0x31085080	SMPU4_RCTL[n]	SMPU4 Region n Control Register	0x00000000
0x31085084	SMPU4_RADDR[n]	SMPU4 Region n Address Register	0x00000000
0x31085088	SMPU4_RIDA[n]	SMPU4 Region n ID A Register	0x00000000
0x3108508C	SMPU4_RIDMSKA[n]	SMPU4 Region n ID Mask A Register	0x00000000
0x31085090	SMPU4_RIDB[n]	SMPU4 Region n ID B Register	0x00000000
0x31085094	SMPU4_RIDMSKB[n]	SMPU4 Region n ID Mask B Register	0x00000000
0x31085098	SMPU4_RCTL[n]	SMPU4 Region n Control Register	0x00000000
0x3108509C	SMPU4_RADDR[n]	SMPU4 Region n Address Register	0x00000000
0x310850A0	SMPU4_RIDA[n]	SMPU4 Region n ID A Register	0x00000000
0x310850A4	SMPU4_RIDMSKA[n]	SMPU4 Region n ID Mask A Register	0x00000000
0x310850A8	SMPU4_RIDB[n]	SMPU4 Region n ID B Register	0x00000000
0x310850AC	SMPU4_RIDMSKB[n]	SMPU4 Region n ID Mask B Register	0x00000000
0x310850B0	SMPU4_RCTL[n]	SMPU4 Region n Control Register	0x00000000
0x310850B4	SMPU4_RADDR[n]	SMPU4 Region n Address Register	0x00000000
0x310850B8	SMPU4_RIDA[n]	SMPU4 Region n ID A Register	0x00000000
0x310850BC	SMPU4_RIDMSKA[n]	SMPU4 Region n ID Mask A Register	0x00000000

Table A-145: ADSP-2159x SMPU4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310850C0	SMPU4_RIDB[n]	SMPU4 Region n ID B Register	0x00000000
0x310850C4	SMPU4_RIDMSKB[n]	SMPU4 Region n ID Mask B Register	0x00000000
0x310850C8	SMPU4_RCTL[n]	SMPU4 Region n Control Register	0x00000000
0x310850CC	SMPU4_RADDR[n]	SMPU4 Region n Address Register	0x00000000
0x310850D0	SMPU4_RIDA[n]	SMPU4 Region n ID A Register	0x00000000
0x310850D4	SMPU4_RIDMSKA[n]	SMPU4 Region n ID Mask A Register	0x00000000
0x310850D8	SMPU4_RIDB[n]	SMPU4 Region n ID B Register	0x00000000
0x310850DC	SMPU4_RIDMSKB[n]	SMPU4 Region n ID Mask B Register	0x00000000
0x310851A0	SMPU4_EXACADD[n]	SMPU4 Exclusive Access IDn Address	0x00000000
0x310851A4	SMPU4_EXACSTAT[n]	SMPU4 Exclusive Access Status	0x00000000
0x310851A8	SMPU4_EXACADD[n]	SMPU4 Exclusive Access IDn Address	0x00000000
0x310851AC	SMPU4_EXACSTAT[n]	SMPU4 Exclusive Access Status	0x00000000
0x310851B0	SMPU4_EXACADD[n]	SMPU4 Exclusive Access IDn Address	0x00000000
0x310851B4	SMPU4_EXACSTAT[n]	SMPU4 Exclusive Access Status	0x00000000
0x31085220	SMPU4_REVID	SMPU4 SMPU Revision ID Register	0x00000010
0x31085800	SMPU4_SECURECTL	SMPU4 SMPU Control Secure Accesses Register	0x00000000
0x31085820	SMPU4_SECURERCTL[n]	SMPU4 Region n Control Secure Accesses Register	0x00000000
0x31085824	SMPU4_SECURERCTL[n]	SMPU4 Region n Control Secure Accesses Register	0x00000000
0x31085828	SMPU4_SECURERCTL[n]	SMPU4 Region n Control Secure Accesses Register	0x00000000
0x3108582C	SMPU4_SECURERCTL[n]	SMPU4 Region n Control Secure Accesses Register	0x00000000
0x31085830	SMPU4_SECURERCTL[n]	SMPU4 Region n Control Secure Accesses Register	0x00000000
0x31085834	SMPU4_SECURERCTL[n]	SMPU4 Region n Control Secure Accesses Register	0x00000000
0x31085838	SMPU4_SECURERCTL[n]	SMPU4 Region n Control Secure Accesses Register	0x00000000
0x3108583C	SMPU4_SECURERCTL[n]	SMPU4 Region n Control Secure Accesses Register	0x00000000

Table A-146: ADSP-2159x SMPU5 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31086000	SMPU5_CTL	SMPU5 SMPU Control Register	0x00000000
0x31086004	SMPU5_STAT	SMPU5 SMPU Status Register	0x00000000
0x31086008	SMPU5_IADDR	SMPU5 Interrupt Address Register	0x00000000

Table A-146: ADSP-2159x SMPU5 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108600C	SMPU5_IDTLS	SMPU5 Interrupt Details Register	0x00000000
0x31086010	SMPU5_BADDR	SMPU5 Bus Error Address Register	0x00000000
0x31086014	SMPU5_BDTLS	SMPU5 Bus Error Details Register	0x00000000
0x31086020	SMPU5_RCTL[n]	SMPU5 Region n Control Register	0x00000000
0x31086024	SMPU5_RADDR[n]	SMPU5 Region n Address Register	0x00000000
0x31086028	SMPU5_RIDA[n]	SMPU5 Region n ID A Register	0x00000000
0x3108602C	SMPU5_RIDMSKA[n]	SMPU5 Region n ID Mask A Register	0x00000000
0x31086030	SMPU5_RIDB[n]	SMPU5 Region n ID B Register	0x00000000
0x31086034	SMPU5_RIDMSKB[n]	SMPU5 Region n ID Mask B Register	0x00000000
0x31086038	SMPU5_RCTL[n]	SMPU5 Region n Control Register	0x00000000
0x3108603C	SMPU5_RADDR[n]	SMPU5 Region n Address Register	0x00000000
0x31086040	SMPU5_RIDA[n]	SMPU5 Region n ID A Register	0x00000000
0x31086044	SMPU5_RIDMSKA[n]	SMPU5 Region n ID Mask A Register	0x00000000
0x31086048	SMPU5_RIDB[n]	SMPU5 Region n ID B Register	0x00000000
0x3108604C	SMPU5_RIDMSKB[n]	SMPU5 Region n ID Mask B Register	0x00000000
0x31086050	SMPU5_RCTL[n]	SMPU5 Region n Control Register	0x00000000
0x31086054	SMPU5_RADDR[n]	SMPU5 Region n Address Register	0x00000000
0x31086058	SMPU5_RIDA[n]	SMPU5 Region n ID A Register	0x00000000
0x3108605C	SMPU5_RIDMSKA[n]	SMPU5 Region n ID Mask A Register	0x00000000
0x31086060	SMPU5_RIDB[n]	SMPU5 Region n ID B Register	0x00000000
0x31086064	SMPU5_RIDMSKB[n]	SMPU5 Region n ID Mask B Register	0x00000000
0x31086068	SMPU5_RCTL[n]	SMPU5 Region n Control Register	0x00000000
0x3108606C	SMPU5_RADDR[n]	SMPU5 Region n Address Register	0x00000000
0x31086070	SMPU5_RIDA[n]	SMPU5 Region n ID A Register	0x00000000
0x31086074	SMPU5_RIDMSKA[n]	SMPU5 Region n ID Mask A Register	0x00000000
0x31086078	SMPU5_RIDB[n]	SMPU5 Region n ID B Register	0x00000000
0x3108607C	SMPU5_RIDMSKB[n]	SMPU5 Region n ID Mask B Register	0x00000000
0x31086080	SMPU5_RCTL[n]	SMPU5 Region n Control Register	0x00000000
0x31086084	SMPU5_RADDR[n]	SMPU5 Region n Address Register	0x00000000
0x31086088	SMPU5_RIDA[n]	SMPU5 Region n ID A Register	0x00000000
0x3108608C	SMPU5_RIDMSKA[n]	SMPU5 Region n ID Mask A Register	0x00000000

Table A-146: ADSP-2159x SMPU5 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31086090	SMPU5_RIDB[n]	SMPU5 Region n ID B Register	0x00000000
0x31086094	SMPU5_RIDMSKB[n]	SMPU5 Region n ID Mask B Register	0x00000000
0x31086098	SMPU5_RCTL[n]	SMPU5 Region n Control Register	0x00000000
0x3108609C	SMPU5_RADDR[n]	SMPU5 Region n Address Register	0x00000000
0x310860A0	SMPU5_RIDA[n]	SMPU5 Region n ID A Register	0x00000000
0x310860A4	SMPU5_RIDMSKA[n]	SMPU5 Region n ID Mask A Register	0x00000000
0x310860A8	SMPU5_RIDB[n]	SMPU5 Region n ID B Register	0x00000000
0x310860AC	SMPU5_RIDMSKB[n]	SMPU5 Region n ID Mask B Register	0x00000000
0x310860B0	SMPU5_RCTL[n]	SMPU5 Region n Control Register	0x00000000
0x310860B4	SMPU5_RADDR[n]	SMPU5 Region n Address Register	0x00000000
0x310860B8	SMPU5_RIDA[n]	SMPU5 Region n ID A Register	0x00000000
0x310860BC	SMPU5_RIDMSKA[n]	SMPU5 Region n ID Mask A Register	0x00000000
0x310860C0	SMPU5_RIDB[n]	SMPU5 Region n ID B Register	0x00000000
0x310860C4	SMPU5_RIDMSKB[n]	SMPU5 Region n ID Mask B Register	0x00000000
0x310860C8	SMPU5_RCTL[n]	SMPU5 Region n Control Register	0x00000000
0x310860CC	SMPU5_RADDR[n]	SMPU5 Region n Address Register	0x00000000
0x310860D0	SMPU5_RIDA[n]	SMPU5 Region n ID A Register	0x00000000
0x310860D4	SMPU5_RIDMSKA[n]	SMPU5 Region n ID Mask A Register	0x00000000
0x310860D8	SMPU5_RIDB[n]	SMPU5 Region n ID B Register	0x00000000
0x310860DC	SMPU5_RIDMSKB[n]	SMPU5 Region n ID Mask B Register	0x00000000
0x31086220	SMPU5_REVID	SMPU5 SMPU Revision ID Register	0x00000010
0x31086800	SMPU5_SECURECTL	SMPU5 SMPU Control Secure Accesses Register	0x00000000
0x31086820	SMPU5_SECURERCTL[n]	SMPU5 Region n Control Secure Accesses Register	0x00000000
0x31086824	SMPU5_SECURERCTL[n]	SMPU5 Region n Control Secure Accesses Register	0x00000000
0x31086828	SMPU5_SECURERCTL[n]	SMPU5 Region n Control Secure Accesses Register	0x00000000
0x3108682C	SMPU5_SECURERCTL[n]	SMPU5 Region n Control Secure Accesses Register	0x00000000
0x31086830	SMPU5_SECURERCTL[n]	SMPU5 Region n Control Secure Accesses Register	0x00000000
0x31086834	SMPU5_SECURERCTL[n]	SMPU5 Region n Control Secure Accesses Register	0x00000000
0x31086838	SMPU5_SECURERCTL[n]	SMPU5 Region n Control Secure Accesses Register	0x00000000
0x3108683C	SMPU5_SECURERCTL[n]	SMPU5 Region n Control Secure Accesses Register	0x00000000

Table A-147: ADSP-2159x SMPU6 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31087000	SMPU6_CTL	SMPU6 SMPU Control Register	0x00000000
0x31087004	SMPU6_STAT	SMPU6 SMPU Status Register	0x00000000
0x31087008	SMPU6_IADDR	SMPU6 Interrupt Address Register	0x00000000
0x3108700C	SMPU6_IDTLS	SMPU6 Interrupt Details Register	0x00000000
0x31087010	SMPU6_BADDR	SMPU6 Bus Error Address Register	0x00000000
0x31087014	SMPU6_BDTLS	SMPU6 Bus Error Details Register	0x00000000
0x31087020	SMPU6_RCTL[n]	SMPU6 Region n Control Register	0x00000000
0x31087024	SMPU6_RADDR[n]	SMPU6 Region n Address Register	0x00000000
0x31087028	SMPU6_RIDA[n]	SMPU6 Region n ID A Register	0x00000000
0x3108702C	SMPU6_RIDMSKA[n]	SMPU6 Region n ID Mask A Register	0x00000000
0x31087030	SMPU6_RIDB[n]	SMPU6 Region n ID B Register	0x00000000
0x31087034	SMPU6_RIDMSKB[n]	SMPU6 Region n ID Mask B Register	0x00000000
0x31087038	SMPU6_RCTL[n]	SMPU6 Region n Control Register	0x00000000
0x3108703C	SMPU6_RADDR[n]	SMPU6 Region n Address Register	0x00000000
0x31087040	SMPU6_RIDA[n]	SMPU6 Region n ID A Register	0x00000000
0x31087044	SMPU6_RIDMSKA[n]	SMPU6 Region n ID Mask A Register	0x00000000
0x31087048	SMPU6_RIDB[n]	SMPU6 Region n ID B Register	0x00000000
0x3108704C	SMPU6_RIDMSKB[n]	SMPU6 Region n ID Mask B Register	0x00000000
0x31087050	SMPU6_RCTL[n]	SMPU6 Region n Control Register	0x00000000
0x31087054	SMPU6_RADDR[n]	SMPU6 Region n Address Register	0x00000000
0x31087058	SMPU6_RIDA[n]	SMPU6 Region n ID A Register	0x00000000
0x3108705C	SMPU6_RIDMSKA[n]	SMPU6 Region n ID Mask A Register	0x00000000
0x31087060	SMPU6_RIDB[n]	SMPU6 Region n ID B Register	0x00000000
0x31087064	SMPU6_RIDMSKB[n]	SMPU6 Region n ID Mask B Register	0x00000000
0x31087068	SMPU6_RCTL[n]	SMPU6 Region n Control Register	0x00000000
0x3108706C	SMPU6_RADDR[n]	SMPU6 Region n Address Register	0x00000000
0x31087070	SMPU6_RIDA[n]	SMPU6 Region n ID A Register	0x00000000
0x31087074	SMPU6_RIDMSKA[n]	SMPU6 Region n ID Mask A Register	0x00000000
0x31087078	SMPU6_RIDB[n]	SMPU6 Region n ID B Register	0x00000000
0x3108707C	SMPU6_RIDMSKB[n]	SMPU6 Region n ID Mask B Register	0x00000000
0x31087080	SMPU6_RCTL[n]	SMPU6 Region n Control Register	0x00000000

Table A-147: ADSP-2159x SMPU6 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31087084	SMPU6_RADDR[n]	SMPU6 Region n Address Register	0x00000000
0x31087088	SMPU6_RIDA[n]	SMPU6 Region n ID A Register	0x00000000
0x3108708C	SMPU6_RIDMSKA[n]	SMPU6 Region n ID Mask A Register	0x00000000
0x31087090	SMPU6_RIDB[n]	SMPU6 Region n ID B Register	0x00000000
0x31087094	SMPU6_RIDMSKB[n]	SMPU6 Region n ID Mask B Register	0x00000000
0x31087098	SMPU6_RCTL[n]	SMPU6 Region n Control Register	0x00000000
0x3108709C	SMPU6_RADDR[n]	SMPU6 Region n Address Register	0x00000000
0x310870A0	SMPU6_RIDA[n]	SMPU6 Region n ID A Register	0x00000000
0x310870A4	SMPU6_RIDMSKA[n]	SMPU6 Region n ID Mask A Register	0x00000000
0x310870A8	SMPU6_RIDB[n]	SMPU6 Region n ID B Register	0x00000000
0x310870AC	SMPU6_RIDMSKB[n]	SMPU6 Region n ID Mask B Register	0x00000000
0x310870B0	SMPU6_RCTL[n]	SMPU6 Region n Control Register	0x00000000
0x310870B4	SMPU6_RADDR[n]	SMPU6 Region n Address Register	0x00000000
0x310870B8	SMPU6_RIDA[n]	SMPU6 Region n ID A Register	0x00000000
0x310870BC	SMPU6_RIDMSKA[n]	SMPU6 Region n ID Mask A Register	0x00000000
0x310870C0	SMPU6_RIDB[n]	SMPU6 Region n ID B Register	0x00000000
0x310870C4	SMPU6_RIDMSKB[n]	SMPU6 Region n ID Mask B Register	0x00000000
0x310870C8	SMPU6_RCTL[n]	SMPU6 Region n Control Register	0x00000000
0x310870CC	SMPU6_RADDR[n]	SMPU6 Region n Address Register	0x00000000
0x310870D0	SMPU6_RIDA[n]	SMPU6 Region n ID A Register	0x00000000
0x310870D4	SMPU6_RIDMSKA[n]	SMPU6 Region n ID Mask A Register	0x00000000
0x310870D8	SMPU6_RIDB[n]	SMPU6 Region n ID B Register	0x00000000
0x310870DC	SMPU6_RIDMSKB[n]	SMPU6 Region n ID Mask B Register	0x00000000
0x31087220	SMPU6_REVID	SMPU6 SMPU Revision ID Register	0x00000010
0x31087800	SMPU6_SECURECTL	SMPU6 SMPU Control Secure Accesses Register	0x00000000
0x31087820	SMPU6_SECURERCTL[n]	SMPU6 Region n Control Secure Accesses Register	0x00000000
0x31087824	SMPU6_SECURERCTL[n]	SMPU6 Region n Control Secure Accesses Register	0x00000000
0x31087828	SMPU6_SECURERCTL[n]	SMPU6 Region n Control Secure Accesses Register	0x00000000
0x3108782C	SMPU6_SECURERCTL[n]	SMPU6 Region n Control Secure Accesses Register	0x00000000
0x31087830	SMPU6_SECURERCTL[n]	SMPU6 Region n Control Secure Accesses Register	0x00000000
0x31087834	SMPU6_SECURERCTL[n]	SMPU6 Region n Control Secure Accesses Register	0x00000000

Table A-147: ADSP-2159x SMPU6 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31087838	SMPU6_SECURERCTL[n]	SMPU6 Region n Control Secure Accesses Register	0x00000000
0x3108783C	SMPU6_SECURERCTL[n]	SMPU6 Region n Control Secure Accesses Register	0x00000000

Table A-148: ADSP-2159x SMPU9 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A0000	SMPU9_CTL	SMPU9 SMPU Control Register	0x00000000
0x310A0004	SMPU9_STAT	SMPU9 SMPU Status Register	0x00000000
0x310A0008	SMPU9_IADDR	SMPU9 Interrupt Address Register	0x00000000
0x310A000C	SMPU9_IDTLS	SMPU9 Interrupt Details Register	0x00000000
0x310A0010	SMPU9_BADDR	SMPU9 Bus Error Address Register	0x00000000
0x310A0014	SMPU9_BDTLS	SMPU9 Bus Error Details Register	0x00000000
0x310A0020	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A0024	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0028	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A002C	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0030	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A0034	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0038	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A003C	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0040	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A0044	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0048	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A004C	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0050	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A0054	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0058	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A005C	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0060	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A0064	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0068	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000

Table A-148: ADSP-2159x SMPU9 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A006C	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0070	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A0074	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0078	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A007C	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0080	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A0084	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A0088	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A008C	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A0090	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A0094	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A0098	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A009C	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A00A0	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A00A4	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A00A8	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A00AC	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A00B0	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A00B4	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A00B8	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A00BC	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A00C0	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A00C4	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A00C8	SMPU9_RCTL[n]	SMPU9 Region n Control Register	0x00000000
0x310A00CC	SMPU9_RADDR[n]	SMPU9 Region n Address Register	0x00000000
0x310A00D0	SMPU9_RIDA[n]	SMPU9 Region n ID A Register	0x00000000
0x310A00D4	SMPU9_RIDMSKA[n]	SMPU9 Region n ID Mask A Register	0x00000000
0x310A00D8	SMPU9_RIDB[n]	SMPU9 Region n ID B Register	0x00000000
0x310A00DC	SMPU9_RIDMSKB[n]	SMPU9 Region n ID Mask B Register	0x00000000
0x310A01A0	SMPU9_EXACADD[n]	SMPU9 Exclusive Access IDn Address	0x00000000
0x310A01A4	SMPU9_EXACSTAT[n]	SMPU9 Exclusive Access Status	0x00000000

Table A-148: ADSP-2159x SMPU9 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A01A8	SMPU9_EXACADD[n]	SMPU9 Exclusive Access IDn Address	0x00000000
0x310A01AC	SMPU9_EXACSTAT[n]	SMPU9 Exclusive Access Status	0x00000000
0x310A01B0	SMPU9_EXACADD[n]	SMPU9 Exclusive Access IDn Address	0x00000000
0x310A01B4	SMPU9_EXACSTAT[n]	SMPU9 Exclusive Access Status	0x00000000
0x310A0220	SMPU9_REVID	SMPU9 SMPU Revision ID Register	0x00000010
0x310A0800	SMPU9_SECURECTL	SMPU9 SMPU Control Secure Accesses Register	0x00000000
0x310A0820	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0824	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0828	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A082C	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0830	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0834	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A0838	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000
0x310A083C	SMPU9_SECURERCTL[n]	SMPU9 Region n Control Secure Accesses Register	0x00000000

Table A-149: ADSP-2159x SPDIF0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C9280	SPDIF0_TX_CTL	SPDIF0 Transmit Control Register	0x00000000
0x310C9284	SPDIF0_TX_STAT_A0	SPDIF0 Transmit Status A0 Register	0x00000000
0x310C9288	SPDIF0_TX_STAT_B0	SPDIF0 Transmit Status B0 Register	0x00000000
0x310C92A0	SPDIF0_RX_CTL	SPDIF0 Receive Control	0x00000018
0x310C92A4	SPDIF0_RX_STAT	SPDIF0 Receive Status Register	0x00000000
0x310C92A8	SPDIF0_RX_STAT0_A	SPDIF0 Receive Status A0 Register	0x00000000
0x310C92AC	SPDIF0_RX_STAT0_B	SPDIF0 Receive Status B0 Register	0x00000000
0x310C92B0	SPDIF0_RX_STAT1_A	SPDIF0 Receive Status A1 Register	0x00000000
0x310C92B4	SPDIF0_RX_STAT1_B	SPDIF0 Receive Status B1 Register	0x00000000
0x310C9350	SPDIF0_TX_STAT_A1	SPDIF0 Transmit Status A1 Register	0x00000000
0x310C9354	SPDIF0_TX_STAT_A2	SPDIF0 Transmit Status A2 Register	0x00000000
0x310C9358	SPDIF0_TX_STAT_A3	SPDIF0 Transmit Status A3 Register	0x00000000
0x310C935C	SPDIF0_TX_STAT_A4	SPDIF0 Transmit Status A4 Register	0x00000000

Table A-149: ADSP-2159x SPDIF0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C9360	SPDIF0_TX_STAT_A5	SPDIF0 Transmit Status A5 Register	0x00000000
0x310C9368	SPDIF0_TX_STAT_B1	SPDIF0 Transmit Status B1 Register	0x00000000
0x310C936C	SPDIF0_TX_STAT_B2	SPDIF0 Transmit Status B2 Register	0x00000000
0x310C9370	SPDIF0_TX_STAT_B3	SPDIF0 Transmit Status B3 Register	0x00000000
0x310C9374	SPDIF0_TX_STAT_B4	SPDIF0 Transmit Status B4 Register	0x00000000
0x310C9378	SPDIF0_TX_STAT_B5	SPDIF0 Transmit Status B5 Register	0x00000000
0x310C9380	SPDIF0_TX_UBUFF_A0	SPDIF0 Transmit User Buffer A0 Register	0x00000000
0x310C9384	SPDIF0_TX_UBUFF_A1	SPDIF0 Transmit User Buffer A1 Register	0x00000000
0x310C9388	SPDIF0_TX_UBUFF_A2	SPDIF0 Transmit User Buffer A2 Register	0x00000000
0x310C938C	SPDIF0_TX_UBUFF_A3	SPDIF0 Transmit User Buffer A3 Register	0x00000000
0x310C9390	SPDIF0_TX_UBUFF_A4	SPDIF0 Transmit User Buffer A4 Register	0x00000000
0x310C9394	SPDIF0_TX_UBUFF_A5	SPDIF0 Transmit User Buffer A5 Register	0x00000000
0x310C93A0	SPDIF0_TX_UBUFF_B0	SPDIF0 Transmit User Buffer B0 Register	0x00000000
0x310C93A4	SPDIF0_TX_UBUFF_B1	SPDIF0 Transmit User Buffer B1 Register	0x00000000
0x310C93A8	SPDIF0_TX_UBUFF_B2	SPDIF0 Transmit User Buffer B2 Register	0x00000000
0x310C93AC	SPDIF0_TX_UBUFF_B3	SPDIF0 Transmit User Buffer B3 Register	0x00000000
0x310C93B0	SPDIF0_TX_UBUFF_B4	SPDIF0 Transmit User Buffer B4 Register	0x00000000
0x310C93B4	SPDIF0_TX_UBUFF_B5	SPDIF0 Transmit User Buffer B5 Register	0x00000000
0x310C93BC	SPDIF0_TX_USRUPDT	SPDIF0 User Bit Update Register	0x00000000

Table A-150: ADSP-2159x SPDIF1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA280	SPDIF1_TX_CTL	SPDIF1 Transmit Control Register	0x00000000
0x310CA284	SPDIF1_TX_STAT_A0	SPDIF1 Transmit Status A0 Register	0x00000000
0x310CA288	SPDIF1_TX_STAT_B0	SPDIF1 Transmit Status B0 Register	0x00000000
0x310CA2A0	SPDIF1_RX_CTL	SPDIF1 Receive Control	0x00000018
0x310CA2A4	SPDIF1_RX_STAT	SPDIF1 Receive Status Register	0x00000000
0x310CA2A8	SPDIF1_RX_STAT0_A	SPDIF1 Receive Status A0 Register	0x00000000
0x310CA2AC	SPDIF1_RX_STAT0_B	SPDIF1 Receive Status B0 Register	0x00000000
0x310CA2B0	SPDIF1_RX_STAT1_A	SPDIF1 Receive Status A1 Register	0x00000000

Table A-150: ADSP-2159x SPDIF1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310CA2B4	SPDIF1_RX_STAT1_B	SPDIF1 Receive Status B1 Register	0x00000000
0x310CA350	SPDIF1_TX_STAT_A1	SPDIF1 Transmit Status A1 Register	0x00000000
0x310CA354	SPDIF1_TX_STAT_A2	SPDIF1 Transmit Status A2 Register	0x00000000
0x310CA358	SPDIF1_TX_STAT_A3	SPDIF1 Transmit Status A3 Register	0x00000000
0x310CA35C	SPDIF1_TX_STAT_A4	SPDIF1 Transmit Status A4 Register	0x00000000
0x310CA360	SPDIF1_TX_STAT_A5	SPDIF1 Transmit Status A5 Register	0x00000000
0x310CA368	SPDIF1_TX_STAT_B1	SPDIF1 Transmit Status B1 Register	0x00000000
0x310CA36C	SPDIF1_TX_STAT_B2	SPDIF1 Transmit Status B2 Register	0x00000000
0x310CA370	SPDIF1_TX_STAT_B3	SPDIF1 Transmit Status B3 Register	0x00000000
0x310CA374	SPDIF1_TX_STAT_B4	SPDIF1 Transmit Status B4 Register	0x00000000
0x310CA378	SPDIF1_TX_STAT_B5	SPDIF1 Transmit Status B5 Register	0x00000000
0x310CA380	SPDIF1_TX_UBUFF_A0	SPDIF1 Transmit User Buffer A0 Register	0x00000000
0x310CA384	SPDIF1_TX_UBUFF_A1	SPDIF1 Transmit User Buffer A1 Register	0x00000000
0x310CA388	SPDIF1_TX_UBUFF_A2	SPDIF1 Transmit User Buffer A2 Register	0x00000000
0x310CA38C	SPDIF1_TX_UBUFF_A3	SPDIF1 Transmit User Buffer A3 Register	0x00000000
0x310CA390	SPDIF1_TX_UBUFF_A4	SPDIF1 Transmit User Buffer A4 Register	0x00000000
0x310CA394	SPDIF1_TX_UBUFF_A5	SPDIF1 Transmit User Buffer A5 Register	0x00000000
0x310CA3A0	SPDIF1_TX_UBUFF_B0	SPDIF1 Transmit User Buffer B0 Register	0x00000000
0x310CA3A4	SPDIF1_TX_UBUFF_B1	SPDIF1 Transmit User Buffer B1 Register	0x00000000
0x310CA3A8	SPDIF1_TX_UBUFF_B2	SPDIF1 Transmit User Buffer B2 Register	0x00000000
0x310CA3AC	SPDIF1_TX_UBUFF_B3	SPDIF1 Transmit User Buffer B3 Register	0x00000000
0x310CA3B0	SPDIF1_TX_UBUFF_B4	SPDIF1 Transmit User Buffer B4 Register	0x00000000
0x310CA3B4	SPDIF1_TX_UBUFF_B5	SPDIF1 Transmit User Buffer B5 Register	0x00000000
0x310CA3BC	SPDIF1_TX_USRUPDT	SPDIF1 User Bit Update Register	0x00000000

Table A-151: ADSP-2159x SPI0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102E004	SPI0_CTL	SPI0 Control Register	0x00000050
0x3102E008	SPI0_RXCTL	SPI0 Receive Control Register	0x00000000
0x3102E00C	SPI0_TXCTL	SPI0 Transmit Control Register	0x00000000

Table A-151: ADSP-2159x SPI0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102E010	SPI0_CLK	SPI0 Clock Rate Register	0x00000000
0x3102E014	SPI0_DLY	SPI0 Delay Register	0x00000301
0x3102E018	SPI0_SLVSEL	SPI0 Slave Select Register	0x0000FE00
0x3102E01C	SPI0_RWC	SPI0 Received Word Count Register	0x00000000
0x3102E020	SPI0_RWCR	SPI0 Received Word Count Reload Register	0x00000000
0x3102E024	SPI0_TWC	SPI0 Transmitted Word Count Register	0x00000000
0x3102E028	SPI0_TWCR	SPI0 Transmitted Word Count Reload Register	0x00000000
0x3102E030	SPI0_IMSK	SPI0 Interrupt Mask Register	0x00000000
0x3102E034	SPI0_IMSK_CLR	SPI0 Interrupt Mask Clear Register	0x00000000
0x3102E038	SPI0_IMSK_SET	SPI0 Interrupt Mask Set Register	0x00000000
0x3102E040	SPI0_STAT	SPI0 Status Register	0x00440001
0x3102E044	SPI0_ILAT	SPI0 Masked Interrupt Condition Register	0x00000000
0x3102E048	SPI0_ILAT_CLR	SPI0 Masked Interrupt Clear Register	0x00000000
0x3102E050	SPI0_RFIFO	SPI0 Receive FIFO Data Register	0x00000000
0x3102E058	SPI0_TFIFO	SPI0 Transmit FIFO Data Register	0x00000000

Table A-152: ADSP-2159x SPI1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102F004	SPI1_CTL	SPI1 Control Register	0x00000050
0x3102F008	SPI1_RXCTL	SPI1 Receive Control Register	0x00000000
0x3102F00C	SPI1_TXCTL	SPI1 Transmit Control Register	0x00000000
0x3102F010	SPI1_CLK	SPI1 Clock Rate Register	0x00000000
0x3102F014	SPI1_DLY	SPI1 Delay Register	0x00000301
0x3102F018	SPI1_SLVSEL	SPI1 Slave Select Register	0x0000FE00
0x3102F01C	SPI1_RWC	SPI1 Received Word Count Register	0x00000000
0x3102F020	SPI1_RWCR	SPI1 Received Word Count Reload Register	0x00000000
0x3102F024	SPI1_TWC	SPI1 Transmitted Word Count Register	0x00000000
0x3102F028	SPI1_TWCR	SPI1 Transmitted Word Count Reload Register	0x00000000
0x3102F030	SPI1_IMSK	SPI1 Interrupt Mask Register	0x00000000
0x3102F034	SPI1_IMSK_CLR	SPI1 Interrupt Mask Clear Register	0x00000000

Table A-152: ADSP-2159x SPI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3102F038	SPI1_IMSK_SET	SPI1 Interrupt Mask Set Register	0x00000000
0x3102F040	SPI1_STAT	SPI1 Status Register	0x00440001
0x3102F044	SPI1_ILAT	SPI1 Masked Interrupt Condition Register	0x00000000
0x3102F048	SPI1_ILAT_CLR	SPI1 Masked Interrupt Clear Register	0x00000000
0x3102F050	SPI1_RFIFO	SPI1 Receive FIFO Data Register	0x00000000
0x3102F058	SPI1_TFIFO	SPI1 Transmit FIFO Data Register	0x00000000

Table A-153: ADSP-2159x SPI2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31030004	SPI2_CTL	SPI2 Control Register	0x00000050
0x31030008	SPI2_RXCTL	SPI2 Receive Control Register	0x00000000
0x3103000C	SPI2_TXCTL	SPI2 Transmit Control Register	0x00000000
0x31030010	SPI2_CLK	SPI2 Clock Rate Register	0x00000000
0x31030014	SPI2_DLY	SPI2 Delay Register	0x00000301
0x31030018	SPI2_SLVSEL	SPI2 Slave Select Register	0x0000FE00
0x3103001C	SPI2_RWC	SPI2 Received Word Count Register	0x00000000
0x31030020	SPI2_RWCR	SPI2 Received Word Count Reload Register	0x00000000
0x31030024	SPI2_TWC	SPI2 Transmitted Word Count Register	0x00000000
0x31030028	SPI2_TWCR	SPI2 Transmitted Word Count Reload Register	0x00000000
0x31030030	SPI2_IMSK	SPI2 Interrupt Mask Register	0x00000000
0x31030034	SPI2_IMSK_CLR	SPI2 Interrupt Mask Clear Register	0x00000000
0x31030038	SPI2_IMSK_SET	SPI2 Interrupt Mask Set Register	0x00000000
0x31030040	SPI2_STAT	SPI2 Status Register	0x00440001
0x31030044	SPI2_ILAT	SPI2 Masked Interrupt Condition Register	0x00000000
0x31030048	SPI2_ILAT_CLR	SPI2 Masked Interrupt Clear Register	0x00000000
0x31030050	SPI2_RFIFO	SPI2 Receive FIFO Data Register	0x00000000
0x31030058	SPI2_TFIFO	SPI2 Transmit FIFO Data Register	0x00000000
0x31030060	SPI2_MMRDH	SPI2 Memory Mapped Read Header	0x00000000
0x31030064	SPI2_MMTOP	SPI2 SPI Memory Top Address	0x00000000

Table A-154: ADSP-2159x SPI3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31031004	SPI3_CTL	SPI3 Control Register	0x00000050
0x31031008	SPI3_RXCTL	SPI3 Receive Control Register	0x00000000
0x3103100C	SPI3_TXCTL	SPI3 Transmit Control Register	0x00000000
0x31031010	SPI3_CLK	SPI3 Clock Rate Register	0x00000000
0x31031014	SPI3_DLY	SPI3 Delay Register	0x00000301
0x31031018	SPI3_SLVSEL	SPI3 Slave Select Register	0x0000FE00
0x3103101C	SPI3_RWC	SPI3 Received Word Count Register	0x00000000
0x31031020	SPI3_RWCR	SPI3 Received Word Count Reload Register	0x00000000
0x31031024	SPI3_TWC	SPI3 Transmitted Word Count Register	0x00000000
0x31031028	SPI3_TWCR	SPI3 Transmitted Word Count Reload Register	0x00000000
0x31031030	SPI3_IMSK	SPI3 Interrupt Mask Register	0x00000000
0x31031034	SPI3_IMSK_CLR	SPI3 Interrupt Mask Clear Register	0x00000000
0x31031038	SPI3_IMSK_SET	SPI3 Interrupt Mask Set Register	0x00000000
0x31031040	SPI3_STAT	SPI3 Status Register	0x00440001
0x31031044	SPI3_ILAT	SPI3 Masked Interrupt Condition Register	0x00000000
0x31031048	SPI3_ILAT_CLR	SPI3 Masked Interrupt Clear Register	0x00000000
0x31031050	SPI3_RFIFO	SPI3 Receive FIFO Data Register	0x00000000
0x31031058	SPI3_TFIFO	SPI3 Transmit FIFO Data Register	0x00000000

Table A-155: ADSP-2159x SPORT0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002000	SPORT0_CTL_A	SPORT0 Half SPORT 'A' Control Register	0x00000000
0x31002004	SPORT0_DIV_A	SPORT0 Half SPORT 'A' Divisor Register	0x00000000
0x31002008	SPORT0_MCTL_A	SPORT0 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100200C	SPORT0_CS0_A	SPORT0 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002010	SPORT0_CS1_A	SPORT0 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002014	SPORT0_CS2_A	SPORT0 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002018	SPORT0_CS3_A	SPORT0 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000

Table A-155: ADSP-2159x SPORT0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002020	SPORT0_ERR_A	SPORT0 Half SPORT 'A' Error Register	0x00000000
0x31002024	SPORT0_MSTAT_A	SPORT0 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002028	SPORT0_CTL2_A	SPORT0 Half SPORT 'A' Control 2 Register	0x00000000
0x31002040	SPORT0_TXPRI_A	SPORT0 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002044	SPORT0_RXPRI_A	SPORT0 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002048	SPORT0_TXSEC_A	SPORT0 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100204C	SPORT0_RXSEC_A	SPORT0 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002080	SPORT0_CTL_B	SPORT0 Half SPORT 'B' Control Register	0x00000000
0x31002084	SPORT0_DIV_B	SPORT0 Half SPORT 'B' Divisor Register	0x00000000
0x31002088	SPORT0_MCTL_B	SPORT0 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100208C	SPORT0_CS0_B	SPORT0 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002090	SPORT0_CS1_B	SPORT0 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002094	SPORT0_CS2_B	SPORT0 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002098	SPORT0_CS3_B	SPORT0 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310020A0	SPORT0_ERR_B	SPORT0 Half SPORT 'B' Error Register	0x00000000
0x310020A4	SPORT0_MSTAT_B	SPORT0 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310020A8	SPORT0_CTL2_B	SPORT0 Half SPORT 'B' Control 2 Register	0x00000000
0x310020C0	SPORT0_TXPRI_B	SPORT0 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310020C4	SPORT0_RXPRI_B	SPORT0 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310020C8	SPORT0_TXSEC_B	SPORT0 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310020CC	SPORT0_RXSEC_B	SPORT0 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-156: ADSP-2159x SPORT1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002100	SPORT1_CTL_A	SPORT1 Half SPORT 'A' Control Register	0x00000000
0x31002104	SPORT1_DIV_A	SPORT1 Half SPORT 'A' Divisor Register	0x00000000
0x31002108	SPORT1_MCTL_A	SPORT1 Half SPORT 'A' Multichannel Control Register	0x00000000

Table A-156: ADSP-2159x SPORT1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3100210C	SPORT1_CS0_A	SPORT1 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002110	SPORT1_CS1_A	SPORT1 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002114	SPORT1_CS2_A	SPORT1 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002118	SPORT1_CS3_A	SPORT1 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002120	SPORT1_ERR_A	SPORT1 Half SPORT 'A' Error Register	0x00000000
0x31002124	SPORT1_MSTAT_A	SPORT1 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002128	SPORT1_CTL2_A	SPORT1 Half SPORT 'A' Control 2 Register	0x00000000
0x31002140	SPORT1_TXPRI_A	SPORT1 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002144	SPORT1_RXPRI_A	SPORT1 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002148	SPORT1_TXSEC_A	SPORT1 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100214C	SPORT1_RXSEC_A	SPORT1 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002180	SPORT1_CTL_B	SPORT1 Half SPORT 'B' Control Register	0x00000000
0x31002184	SPORT1_DIV_B	SPORT1 Half SPORT 'B' Divisor Register	0x00000000
0x31002188	SPORT1_MCTL_B	SPORT1 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100218C	SPORT1_CS0_B	SPORT1 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002190	SPORT1_CS1_B	SPORT1 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002194	SPORT1_CS2_B	SPORT1 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002198	SPORT1_CS3_B	SPORT1 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310021A0	SPORT1_ERR_B	SPORT1 Half SPORT 'B' Error Register	0x00000000
0x310021A4	SPORT1_MSTAT_B	SPORT1 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310021A8	SPORT1_CTL2_B	SPORT1 Half SPORT 'B' Control 2 Register	0x00000000
0x310021C0	SPORT1_TXPRI_B	SPORT1 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310021C4	SPORT1_RXPRI_B	SPORT1 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310021C8	SPORT1_TXSEC_B	SPORT1 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310021CC	SPORT1_RXSEC_B	SPORT1 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-157: ADSP-2159x SPORT2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002200	SPORT2_CTL_A	SPORT2 Half SPORT 'A' Control Register	0x00000000
0x31002204	SPORT2_DIV_A	SPORT2 Half SPORT 'A' Divisor Register	0x00000000
0x31002208	SPORT2_MCTL_A	SPORT2 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100220C	SPORT2_CS0_A	SPORT2 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002210	SPORT2_CS1_A	SPORT2 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002214	SPORT2_CS2_A	SPORT2 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002218	SPORT2_CS3_A	SPORT2 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002220	SPORT2_ERR_A	SPORT2 Half SPORT 'A' Error Register	0x00000000
0x31002224	SPORT2_MSTAT_A	SPORT2 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002228	SPORT2_CTL2_A	SPORT2 Half SPORT 'A' Control 2 Register	0x00000000
0x31002240	SPORT2_TXPRI_A	SPORT2 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002244	SPORT2_RXPRI_A	SPORT2 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002248	SPORT2_TXSEC_A	SPORT2 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100224C	SPORT2_RXSEC_A	SPORT2 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002280	SPORT2_CTL_B	SPORT2 Half SPORT 'B' Control Register	0x00000000
0x31002284	SPORT2_DIV_B	SPORT2 Half SPORT 'B' Divisor Register	0x00000000
0x31002288	SPORT2_MCTL_B	SPORT2 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100228C	SPORT2_CS0_B	SPORT2 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002290	SPORT2_CS1_B	SPORT2 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002294	SPORT2_CS2_B	SPORT2 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002298	SPORT2_CS3_B	SPORT2 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310022A0	SPORT2_ERR_B	SPORT2 Half SPORT 'B' Error Register	0x00000000
0x310022A4	SPORT2_MSTAT_B	SPORT2 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310022A8	SPORT2_CTL2_B	SPORT2 Half SPORT 'B' Control 2 Register	0x00000000
0x310022C0	SPORT2_TXPRI_B	SPORT2 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000

Table A-157: ADSP-2159x SPORT2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310022C4	SPORT2_RXPRI_B	SPORT2 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310022C8	SPORT2_TXSEC_B	SPORT2 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310022CC	SPORT2_RXSEC_B	SPORT2 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-158: ADSP-2159x SPORT3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002300	SPORT3_CTL_A	SPORT3 Half SPORT 'A' Control Register	0x00000000
0x31002304	SPORT3_DIV_A	SPORT3 Half SPORT 'A' Divisor Register	0x00000000
0x31002308	SPORT3_MCTL_A	SPORT3 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100230C	SPORT3_CS0_A	SPORT3 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002310	SPORT3_CS1_A	SPORT3 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002314	SPORT3_CS2_A	SPORT3 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002318	SPORT3_CS3_A	SPORT3 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002320	SPORT3_ERR_A	SPORT3 Half SPORT 'A' Error Register	0x00000000
0x31002324	SPORT3_MSTAT_A	SPORT3 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002328	SPORT3_CTL2_A	SPORT3 Half SPORT 'A' Control 2 Register	0x00000000
0x31002340	SPORT3_TXPRI_A	SPORT3 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002344	SPORT3_RXPRI_A	SPORT3 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002348	SPORT3_TXSEC_A	SPORT3 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100234C	SPORT3_RXSEC_A	SPORT3 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002380	SPORT3_CTL_B	SPORT3 Half SPORT 'B' Control Register	0x00000000
0x31002384	SPORT3_DIV_B	SPORT3 Half SPORT 'B' Divisor Register	0x00000000
0x31002388	SPORT3_MCTL_B	SPORT3 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100238C	SPORT3_CS0_B	SPORT3 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002390	SPORT3_CS1_B	SPORT3 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002394	SPORT3_CS2_B	SPORT3 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000

Table A-158: ADSP-2159x SPORT3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002398	SPORT3_CS3_B	SPORT3 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310023A0	SPORT3_ERR_B	SPORT3 Half SPORT 'B' Error Register	0x00000000
0x310023A4	SPORT3_MSTAT_B	SPORT3 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310023A8	SPORT3_CTL2_B	SPORT3 Half SPORT 'B' Control 2 Register	0x00000000
0x310023C0	SPORT3_TXPRI_B	SPORT3 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310023C4	SPORT3_RXPRI_B	SPORT3 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310023C8	SPORT3_TXSEC_B	SPORT3 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310023CC	SPORT3_RXSEC_B	SPORT3 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-159: ADSP-2159x SPORT4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002400	SPORT4_CTL_A	SPORT4 Half SPORT 'A' Control Register	0x00000000
0x31002404	SPORT4_DIV_A	SPORT4 Half SPORT 'A' Divisor Register	0x00000000
0x31002408	SPORT4_MCTL_A	SPORT4 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100240C	SPORT4_CS0_A	SPORT4 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002410	SPORT4_CS1_A	SPORT4 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002414	SPORT4_CS2_A	SPORT4 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002418	SPORT4_CS3_A	SPORT4 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002420	SPORT4_ERR_A	SPORT4 Half SPORT 'A' Error Register	0x00000000
0x31002424	SPORT4_MSTAT_A	SPORT4 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002428	SPORT4_CTL2_A	SPORT4 Half SPORT 'A' Control 2 Register	0x00000000
0x31002440	SPORT4_TXPRI_A	SPORT4 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002444	SPORT4_RXPRI_A	SPORT4 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002448	SPORT4_TXSEC_A	SPORT4 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100244C	SPORT4_RXSEC_A	SPORT4 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002480	SPORT4_CTL_B	SPORT4 Half SPORT 'B' Control Register	0x00000000
0x31002484	SPORT4_DIV_B	SPORT4 Half SPORT 'B' Divisor Register	0x00000000

Table A-159: ADSP-2159x SPORT4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002488	SPORT4_MCTL_B	SPORT4 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100248C	SPORT4_CS0_B	SPORT4 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002490	SPORT4_CS1_B	SPORT4 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002494	SPORT4_CS2_B	SPORT4 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002498	SPORT4_CS3_B	SPORT4 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310024A0	SPORT4_ERR_B	SPORT4 Half SPORT 'B' Error Register	0x00000000
0x310024A4	SPORT4_MSTAT_B	SPORT4 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310024A8	SPORT4_CTL2_B	SPORT4 Half SPORT 'B' Control 2 Register	0x00000000
0x310024C0	SPORT4_TXPRI_B	SPORT4 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310024C4	SPORT4_RXPRI_B	SPORT4 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310024C8	SPORT4_TXSEC_B	SPORT4 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310024CC	SPORT4_RXSEC_B	SPORT4 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-160: ADSP-2159x SPORT5 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002500	SPORT5_CTL_A	SPORT5 Half SPORT 'A' Control Register	0x00000000
0x31002504	SPORT5_DIV_A	SPORT5 Half SPORT 'A' Divisor Register	0x00000000
0x31002508	SPORT5_MCTL_A	SPORT5 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100250C	SPORT5_CS0_A	SPORT5 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002510	SPORT5_CS1_A	SPORT5 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002514	SPORT5_CS2_A	SPORT5 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002518	SPORT5_CS3_A	SPORT5 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002520	SPORT5_ERR_A	SPORT5 Half SPORT 'A' Error Register	0x00000000
0x31002524	SPORT5_MSTAT_A	SPORT5 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002528	SPORT5_CTL2_A	SPORT5 Half SPORT 'A' Control 2 Register	0x00000000

Table A-160: ADSP-2159x SPORT5 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002540	SPORT5_TXPRI_A	SPORT5 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002544	SPORT5_RXPRI_A	SPORT5 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002548	SPORT5_TXSEC_A	SPORT5 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100254C	SPORT5_RXSEC_A	SPORT5 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002580	SPORT5_CTL_B	SPORT5 Half SPORT 'B' Control Register	0x00000000
0x31002584	SPORT5_DIV_B	SPORT5 Half SPORT 'B' Divisor Register	0x00000000
0x31002588	SPORT5_MCTL_B	SPORT5 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100258C	SPORT5_CS0_B	SPORT5 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002590	SPORT5_CS1_B	SPORT5 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002594	SPORT5_CS2_B	SPORT5 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002598	SPORT5_CS3_B	SPORT5 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310025A0	SPORT5_ERR_B	SPORT5 Half SPORT 'B' Error Register	0x00000000
0x310025A4	SPORT5_MSTAT_B	SPORT5 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310025A8	SPORT5_CTL2_B	SPORT5 Half SPORT 'B' Control 2 Register	0x00000000
0x310025C0	SPORT5_TXPRI_B	SPORT5 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310025C4	SPORT5_RXPRI_B	SPORT5 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310025C8	SPORT5_TXSEC_B	SPORT5 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310025CC	SPORT5_RXSEC_B	SPORT5 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-161: ADSP-2159x SPORT6 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002600	SPORT6_CTL_A	SPORT6 Half SPORT 'A' Control Register	0x00000000
0x31002604	SPORT6_DIV_A	SPORT6 Half SPORT 'A' Divisor Register	0x00000000
0x31002608	SPORT6_MCTL_A	SPORT6 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100260C	SPORT6_CS0_A	SPORT6 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002610	SPORT6_CS1_A	SPORT6 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000

Table A-161: ADSP-2159x SPORT6 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002614	SPORT6_CS2_A	SPORT6 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002618	SPORT6_CS3_A	SPORT6 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002620	SPORT6_ERR_A	SPORT6 Half SPORT 'A' Error Register	0x00000000
0x31002624	SPORT6_MSTAT_A	SPORT6 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002628	SPORT6_CTL2_A	SPORT6 Half SPORT 'A' Control 2 Register	0x00000000
0x31002640	SPORT6_TXPRI_A	SPORT6 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002644	SPORT6_RXPRI_A	SPORT6 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002648	SPORT6_TXSEC_A	SPORT6 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100264C	SPORT6_RXSEC_A	SPORT6 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002680	SPORT6_CTL_B	SPORT6 Half SPORT 'B' Control Register	0x00000000
0x31002684	SPORT6_DIV_B	SPORT6 Half SPORT 'B' Divisor Register	0x00000000
0x31002688	SPORT6_MCTL_B	SPORT6 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100268C	SPORT6_CS0_B	SPORT6 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002690	SPORT6_CS1_B	SPORT6 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002694	SPORT6_CS2_B	SPORT6 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002698	SPORT6_CS3_B	SPORT6 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310026A0	SPORT6_ERR_B	SPORT6 Half SPORT 'B' Error Register	0x00000000
0x310026A4	SPORT6_MSTAT_B	SPORT6 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310026A8	SPORT6_CTL2_B	SPORT6 Half SPORT 'B' Control 2 Register	0x00000000
0x310026C0	SPORT6_TXPRI_B	SPORT6 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000
0x310026C4	SPORT6_RXPRI_B	SPORT6 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310026C8	SPORT6_TXSEC_B	SPORT6 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310026CC	SPORT6_RXSEC_B	SPORT6 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-162: ADSP-2159x SPORT7 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31002700	SPORT7_CTL_A	SPORT7 Half SPORT 'A' Control Register	0x00000000
0x31002704	SPORT7_DIV_A	SPORT7 Half SPORT 'A' Divisor Register	0x00000000
0x31002708	SPORT7_MCTL_A	SPORT7 Half SPORT 'A' Multichannel Control Register	0x00000000
0x3100270C	SPORT7_CS0_A	SPORT7 Half SPORT 'A' Multichannel 0-31 Select Register	0x00000000
0x31002710	SPORT7_CS1_A	SPORT7 Half SPORT 'A' Multichannel 32-63 Select Register	0x00000000
0x31002714	SPORT7_CS2_A	SPORT7 Half SPORT 'A' Multichannel 64-95 Select Register	0x00000000
0x31002718	SPORT7_CS3_A	SPORT7 Half SPORT 'A' Multichannel 96-127 Select Register	0x00000000
0x31002720	SPORT7_ERR_A	SPORT7 Half SPORT 'A' Error Register	0x00000000
0x31002724	SPORT7_MSTAT_A	SPORT7 Half SPORT 'A' Multichannel Status Register	0x00000000
0x31002728	SPORT7_CTL2_A	SPORT7 Half SPORT 'A' Control 2 Register	0x00000000
0x31002740	SPORT7_TXPRI_A	SPORT7 Half SPORT 'A' Tx Buffer (Primary) Register	0x00000000
0x31002744	SPORT7_RXPRI_A	SPORT7 Half SPORT 'A' Rx Buffer (Primary) Register	0x00000000
0x31002748	SPORT7_TXSEC_A	SPORT7 Half SPORT 'A' Tx Buffer (Secondary) Register	0x00000000
0x3100274C	SPORT7_RXSEC_A	SPORT7 Half SPORT 'A' Rx Buffer (Secondary) Register	0x00000000
0x31002780	SPORT7_CTL_B	SPORT7 Half SPORT 'B' Control Register	0x00000000
0x31002784	SPORT7_DIV_B	SPORT7 Half SPORT 'B' Divisor Register	0x00000000
0x31002788	SPORT7_MCTL_B	SPORT7 Half SPORT 'B' Multichannel Control Register	0x00000000
0x3100278C	SPORT7_CS0_B	SPORT7 Half SPORT 'B' Multichannel 0-31 Select Register	0x00000000
0x31002790	SPORT7_CS1_B	SPORT7 Half SPORT 'B' Multichannel 32-63 Select Register	0x00000000
0x31002794	SPORT7_CS2_B	SPORT7 Half SPORT 'B' Multichannel 64-95 Select Register	0x00000000
0x31002798	SPORT7_CS3_B	SPORT7 Half SPORT 'B' Multichannel 96-127 Select Register	0x00000000
0x310027A0	SPORT7_ERR_B	SPORT7 Half SPORT 'B' Error Register	0x00000000
0x310027A4	SPORT7_MSTAT_B	SPORT7 Half SPORT 'B' Multichannel Status Register	0x00000000
0x310027A8	SPORT7_CTL2_B	SPORT7 Half SPORT 'B' Control 2 Register	0x00000000
0x310027C0	SPORT7_TXPRI_B	SPORT7 Half SPORT 'B' Tx Buffer (Primary) Register	0x00000000

Table A-162: ADSP-2159x SPORT7 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310027C4	SPORT7_RXPRI_B	SPORT7 Half SPORT 'B' Rx Buffer (Primary) Register	0x00000000
0x310027C8	SPORT7_TXSEC_B	SPORT7 Half SPORT 'B' Tx Buffer (Secondary) Register	0x00000000
0x310027CC	SPORT7_RXSEC_B	SPORT7 Half SPORT 'B' Rx Buffer (Secondary) Register	0x00000000

Table A-163: ADSP-2159x SPU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B000	SPU0_CTL	SPU0 Control Register	0x000000AD
0x3108B004	SPU0_STAT	SPU0 Status Register	0x00000000
0x3108B400	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B404	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B408	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B40C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B410	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B414	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B418	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B41C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B420	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B424	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B428	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B42C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B430	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B434	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B438	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B43C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B440	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B444	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B448	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B44C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B450	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B454	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B458	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B45C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B460	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B464	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B468	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B46C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B470	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B474	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B478	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B47C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B480	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B484	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B488	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B48C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B490	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B494	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B498	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B49C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4A0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4A4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4A8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4AC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4B0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4B4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4B8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4BC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4C0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4C4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4C8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4CC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4D0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B4D4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4D8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4DC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4E0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4E4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4E8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4EC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4F0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4F4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4F8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B4FC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B500	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B504	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B508	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B50C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B510	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B514	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B518	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B51C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B520	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B524	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B528	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B52C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B530	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B534	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B538	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B53C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B540	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B544	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B548	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B54C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B550	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B554	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B558	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B55C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B560	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B564	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B568	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B56C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B570	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B574	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B578	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B57C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B580	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B584	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B588	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B58C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B590	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B594	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B598	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B59C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5A0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5A4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5A8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5AC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5B0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5B4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5B8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5BC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5C0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5C4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5C8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B5CC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5D0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5D4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5D8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5DC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5E0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5E4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5E8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5EC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5F0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5F4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5F8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B5FC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B600	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B604	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B608	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B60C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B610	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B614	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B618	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B61C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B620	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B624	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B628	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B62C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B630	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B634	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B638	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B63C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B640	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B644	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B648	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B64C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B650	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B654	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B658	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B65C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B660	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B664	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B668	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B66C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B670	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B674	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B678	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B67C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B680	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B684	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B688	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B68C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B690	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B694	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B698	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B69C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6A0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6A4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6A8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6AC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6B0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6B4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6B8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6BC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6C0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108B6C4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6C8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6CC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6D0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6D4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6D8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6DC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6E0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6E4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6E8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6EC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6F0	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6F4	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6F8	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B6FC	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B700	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B704	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B708	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B70C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B710	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B714	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B718	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B71C	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B720	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B724	SPU0_WP[n]	SPU0 Write Protect Register n	0x00000000
0x3108B840	SPU0_SECURECTL	SPU0 Secure Control Register	0x00000000
0x3108B84C	SPU0_SECURECHK	SPU0 Secure Check Register	0xFFFFFFFF
0x3108B980	SPU0_SECUREC[n]	SPU0 Secure Core Registers	0x00000001
0x3108B984	SPU0_SECUREC[n]	SPU0 Secure Core Registers	0x00000001
0x3108B988	SPU0_SECUREC[n]	SPU0 Secure Core Registers	0x00000001
0x3108BA00	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BA04	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA08	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA0C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA10	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA14	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA18	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA1C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA20	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA24	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA28	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA2C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA30	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA34	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA38	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA3C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA40	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA44	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA48	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA4C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA50	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA54	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA58	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA5C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA60	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA64	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA68	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA6C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA70	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA74	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA78	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA7C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BA80	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA84	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA88	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA8C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA90	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA94	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA98	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BA9C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAA0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAA4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAA8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAAC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAB0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAB4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAB8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BABC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAC0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAC4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAC8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BACC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAD0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAD4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAD8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BADC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAE0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAE4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAE8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAEC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAF0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAF4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BAF8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BAFC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB00	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB04	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB08	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB0C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB10	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB14	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB18	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB1C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB20	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB24	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB28	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB2C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB30	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB34	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB38	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB3C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB40	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB44	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB48	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB4C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB50	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB54	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB58	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB5C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB60	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB64	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB68	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB6C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB70	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB74	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BB78	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB7C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB80	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB84	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB88	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB8C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB90	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB94	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB98	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BB9C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBA0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBA4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBA8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBAC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBB0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBB4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBB8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBBC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBC0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBC4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBC8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBCC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBD0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBD4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBD8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBDC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBE0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBE4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBE8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBEC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBF0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BBF4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBF8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BBFC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC00	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC04	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC08	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC0C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC10	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC14	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC18	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC1C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC20	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC24	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC28	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC2C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC30	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC34	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC38	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC3C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC40	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC44	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC48	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC4C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC50	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC54	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC58	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC5C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC60	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC64	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC68	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC6C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BC70	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC74	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC78	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC7C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC80	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC84	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC88	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC8C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC90	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC94	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC98	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BC9C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCA0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCA4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCA8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCAC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCB0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCB4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCB8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCBC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCC0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCC4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCC8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCCC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCD0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCD4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCD8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCDC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCE0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCE4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCE8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-163: ADSP-2159x SPU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108BCEC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCF0	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCF4	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCF8	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BCFC	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD00	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD04	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD08	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD0C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD10	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD14	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD18	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD1C	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD20	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001
0x3108BD24	SPU0_SECUREP[n]	SPU0 Secure Peripheral Register	0x00000001

Table A-164: ADSP-2159x SWU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31015000	SWU0_GCTL	SWU0 Global Control Register	0x00000000
0x31015004	SWU0_GSTAT	SWU0 Global Status Register	0x00000000
0x31015010	SWU0_CTL[n]	SWU0 Control Register n	0x00000000
0x31015014	SWU0_LA[n]	SWU0 Lower Address Register n	0x00000000
0x31015018	SWU0_UA[n]	SWU0 Upper Address Register n	0x00000000
0x3101501C	SWU0_ID[n]	SWU0 ID Register n	0x00000000
0x31015020	SWU0_CNT[n]	SWU0 Count Register n	0x00000000
0x31015024	SWU0_TARG[n]	SWU0 Target Register n	0x00000000
0x31015028	SWU0_HIST[n]	SWU0 Bandwidth History Register n	0x00000000
0x3101502C	SWU0_CUR[n]	SWU0 Current Register n	0x00000000
0x31015030	SWU0_CTL[n]	SWU0 Control Register n	0x00000000
0x31015034	SWU0_LA[n]	SWU0 Lower Address Register n	0x00000000

Table A-164: ADSP-2159x SWU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31015038	SWU0_UA[n]	SWU0 Upper Address Register n	0x00000000
0x3101503C	SWU0_ID[n]	SWU0 ID Register n	0x00000000
0x31015040	SWU0_CNT[n]	SWU0 Count Register n	0x00000000
0x31015044	SWU0_TARG[n]	SWU0 Target Register n	0x00000000
0x31015048	SWU0_HIST[n]	SWU0 Bandwidth History Register n	0x00000000
0x3101504C	SWU0_CUR[n]	SWU0 Current Register n	0x00000000
0x31015050	SWU0_CTL[n]	SWU0 Control Register n	0x00000000
0x31015054	SWU0_LA[n]	SWU0 Lower Address Register n	0x00000000
0x31015058	SWU0_UA[n]	SWU0 Upper Address Register n	0x00000000
0x3101505C	SWU0_ID[n]	SWU0 ID Register n	0x00000000
0x31015060	SWU0_CNT[n]	SWU0 Count Register n	0x00000000
0x31015064	SWU0_TARG[n]	SWU0 Target Register n	0x00000000
0x31015068	SWU0_HIST[n]	SWU0 Bandwidth History Register n	0x00000000
0x3101506C	SWU0_CUR[n]	SWU0 Current Register n	0x00000000
0x31015070	SWU0_CTL[n]	SWU0 Control Register n	0x00000000
0x31015074	SWU0_LA[n]	SWU0 Lower Address Register n	0x00000000
0x31015078	SWU0_UA[n]	SWU0 Upper Address Register n	0x00000000
0x3101507C	SWU0_ID[n]	SWU0 ID Register n	0x00000000
0x31015080	SWU0_CNT[n]	SWU0 Count Register n	0x00000000
0x31015084	SWU0_TARG[n]	SWU0 Target Register n	0x00000000
0x31015088	SWU0_HIST[n]	SWU0 Bandwidth History Register n	0x00000000
0x3101508C	SWU0_CUR[n]	SWU0 Current Register n	0x00000000

Table A-165: ADSP-2159x SWU1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31093000	SWU1_GCTL	SWU1 Global Control Register	0x00000000
0x31093004	SWU1_GSTAT	SWU1 Global Status Register	0x00000000
0x31093010	SWU1_CTL[n]	SWU1 Control Register n	0x00000000
0x31093014	SWU1_LA[n]	SWU1 Lower Address Register n	0x00000000
0x31093018	SWU1_UA[n]	SWU1 Upper Address Register n	0x00000000

Table A-165: ADSP-2159x SWU1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109301C	SWU1_ID[n]	SWU1 ID Register n	0x00000000
0x31093020	SWU1_CNT[n]	SWU1 Count Register n	0x00000000
0x31093024	SWU1_TARG[n]	SWU1 Target Register n	0x00000000
0x31093028	SWU1_HIST[n]	SWU1 Bandwidth History Register n	0x00000000
0x3109302C	SWU1_CUR[n]	SWU1 Current Register n	0x00000000
0x31093030	SWU1_CTL[n]	SWU1 Control Register n	0x00000000
0x31093034	SWU1_LA[n]	SWU1 Lower Address Register n	0x00000000
0x31093038	SWU1_UA[n]	SWU1 Upper Address Register n	0x00000000
0x3109303C	SWU1_ID[n]	SWU1 ID Register n	0x00000000
0x31093040	SWU1_CNT[n]	SWU1 Count Register n	0x00000000
0x31093044	SWU1_TARG[n]	SWU1 Target Register n	0x00000000
0x31093048	SWU1_HIST[n]	SWU1 Bandwidth History Register n	0x00000000
0x3109304C	SWU1_CUR[n]	SWU1 Current Register n	0x00000000
0x31093050	SWU1_CTL[n]	SWU1 Control Register n	0x00000000
0x31093054	SWU1_LA[n]	SWU1 Lower Address Register n	0x00000000
0x31093058	SWU1_UA[n]	SWU1 Upper Address Register n	0x00000000
0x3109305C	SWU1_ID[n]	SWU1 ID Register n	0x00000000
0x31093060	SWU1_CNT[n]	SWU1 Count Register n	0x00000000
0x31093064	SWU1_TARG[n]	SWU1 Target Register n	0x00000000
0x31093068	SWU1_HIST[n]	SWU1 Bandwidth History Register n	0x00000000
0x3109306C	SWU1_CUR[n]	SWU1 Current Register n	0x00000000
0x31093070	SWU1_CTL[n]	SWU1 Control Register n	0x00000000
0x31093074	SWU1_LA[n]	SWU1 Lower Address Register n	0x00000000
0x31093078	SWU1_UA[n]	SWU1 Upper Address Register n	0x00000000
0x3109307C	SWU1_ID[n]	SWU1 ID Register n	0x00000000
0x31093080	SWU1_CNT[n]	SWU1 Count Register n	0x00000000
0x31093084	SWU1_TARG[n]	SWU1 Target Register n	0x00000000
0x31093088	SWU1_HIST[n]	SWU1 Bandwidth History Register n	0x00000000
0x3109308C	SWU1_CUR[n]	SWU1 Current Register n	0x00000000

Table A-166: ADSP-2159x SWU10 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31143000	SWU10_GCTL	SWU10 Global Control Register	0x00000000
0x31143004	SWU10_GSTAT	SWU10 Global Status Register	0x00000000
0x31143010	SWU10_CTL[n]	SWU10 Control Register n	0x00000000
0x31143014	SWU10_LA[n]	SWU10 Lower Address Register n	0x00000000
0x31143018	SWU10_UA[n]	SWU10 Upper Address Register n	0x00000000
0x3114301C	SWU10_ID[n]	SWU10 ID Register n	0x00000000
0x31143020	SWU10_CNT[n]	SWU10 Count Register n	0x00000000
0x31143024	SWU10_TARG[n]	SWU10 Target Register n	0x00000000
0x31143028	SWU10_HIST[n]	SWU10 Bandwidth History Register n	0x00000000
0x3114302C	SWU10_CUR[n]	SWU10 Current Register n	0x00000000
0x31143030	SWU10_CTL[n]	SWU10 Control Register n	0x00000000
0x31143034	SWU10_LA[n]	SWU10 Lower Address Register n	0x00000000
0x31143038	SWU10_UA[n]	SWU10 Upper Address Register n	0x00000000
0x3114303C	SWU10_ID[n]	SWU10 ID Register n	0x00000000
0x31143040	SWU10_CNT[n]	SWU10 Count Register n	0x00000000
0x31143044	SWU10_TARG[n]	SWU10 Target Register n	0x00000000
0x31143048	SWU10_HIST[n]	SWU10 Bandwidth History Register n	0x00000000
0x3114304C	SWU10_CUR[n]	SWU10 Current Register n	0x00000000
0x31143050	SWU10_CTL[n]	SWU10 Control Register n	0x00000000
0x31143054	SWU10_LA[n]	SWU10 Lower Address Register n	0x00000000
0x31143058	SWU10_UA[n]	SWU10 Upper Address Register n	0x00000000
0x3114305C	SWU10_ID[n]	SWU10 ID Register n	0x00000000
0x31143060	SWU10_CNT[n]	SWU10 Count Register n	0x00000000
0x31143064	SWU10_TARG[n]	SWU10 Target Register n	0x00000000
0x31143068	SWU10_HIST[n]	SWU10 Bandwidth History Register n	0x00000000
0x3114306C	SWU10_CUR[n]	SWU10 Current Register n	0x00000000
0x31143070	SWU10_CTL[n]	SWU10 Control Register n	0x00000000
0x31143074	SWU10_LA[n]	SWU10 Lower Address Register n	0x00000000
0x31143078	SWU10_UA[n]	SWU10 Upper Address Register n	0x00000000
0x3114307C	SWU10_ID[n]	SWU10 ID Register n	0x00000000
0x31143080	SWU10_CNT[n]	SWU10 Count Register n	0x00000000

Table A-166: ADSP-2159x SWU10 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31143084	SWU10_TARG[n]	SWU10 Target Register n	0x00000000
0x31143088	SWU10_HIST[n]	SWU10 Bandwidth History Register n	0x00000000
0x3114308C	SWU10_CUR[n]	SWU10 Current Register n	0x00000000

Table A-167: ADSP-2159x SWU11 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31097000	SWU11_GCTL	SWU11 Global Control Register	0x00000000
0x31097004	SWU11_GSTAT	SWU11 Global Status Register	0x00000000
0x31097010	SWU11_CTL[n]	SWU11 Control Register n	0x00000000
0x31097014	SWU11_LA[n]	SWU11 Lower Address Register n	0x00000000
0x31097018	SWU11_UA[n]	SWU11 Upper Address Register n	0x00000000
0x3109701C	SWU11_ID[n]	SWU11 ID Register n	0x00000000
0x31097020	SWU11_CNT[n]	SWU11 Count Register n	0x00000000
0x31097024	SWU11_TARG[n]	SWU11 Target Register n	0x00000000
0x31097028	SWU11_HIST[n]	SWU11 Bandwidth History Register n	0x00000000
0x3109702C	SWU11_CUR[n]	SWU11 Current Register n	0x00000000
0x31097030	SWU11_CTL[n]	SWU11 Control Register n	0x00000000
0x31097034	SWU11_LA[n]	SWU11 Lower Address Register n	0x00000000
0x31097038	SWU11_UA[n]	SWU11 Upper Address Register n	0x00000000
0x3109703C	SWU11_ID[n]	SWU11 ID Register n	0x00000000
0x31097040	SWU11_CNT[n]	SWU11 Count Register n	0x00000000
0x31097044	SWU11_TARG[n]	SWU11 Target Register n	0x00000000
0x31097048	SWU11_HIST[n]	SWU11 Bandwidth History Register n	0x00000000
0x3109704C	SWU11_CUR[n]	SWU11 Current Register n	0x00000000
0x31097050	SWU11_CTL[n]	SWU11 Control Register n	0x00000000
0x31097054	SWU11_LA[n]	SWU11 Lower Address Register n	0x00000000
0x31097058	SWU11_UA[n]	SWU11 Upper Address Register n	0x00000000
0x3109705C	SWU11_ID[n]	SWU11 ID Register n	0x00000000
0x31097060	SWU11_CNT[n]	SWU11 Count Register n	0x00000000
0x31097064	SWU11_TARG[n]	SWU11 Target Register n	0x00000000

Table A-167: ADSP-2159x SWU11 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31097068	SWU11_HIST[n]	SWU11 Bandwidth History Register n	0x00000000
0x3109706C	SWU11_CUR[n]	SWU11 Current Register n	0x00000000
0x31097070	SWU11_CTL[n]	SWU11 Control Register n	0x00000000
0x31097074	SWU11_LA[n]	SWU11 Lower Address Register n	0x00000000
0x31097078	SWU11_UA[n]	SWU11 Upper Address Register n	0x00000000
0x3109707C	SWU11_ID[n]	SWU11 ID Register n	0x00000000
0x31097080	SWU11_CNT[n]	SWU11 Count Register n	0x00000000
0x31097084	SWU11_TARG[n]	SWU11 Target Register n	0x00000000
0x31097088	SWU11_HIST[n]	SWU11 Bandwidth History Register n	0x00000000
0x3109708C	SWU11_CUR[n]	SWU11 Current Register n	0x00000000

Table A-168: ADSP-2159x SWU12 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A8000	SWU12_GCTL	SWU12 Global Control Register	0x00000000
0x310A8004	SWU12_GSTAT	SWU12 Global Status Register	0x00000000
0x310A8010	SWU12_CTL[n]	SWU12 Control Register n	0x00000000
0x310A8014	SWU12_LA[n]	SWU12 Lower Address Register n	0x00000000
0x310A8018	SWU12_UA[n]	SWU12 Upper Address Register n	0x00000000
0x310A801C	SWU12_ID[n]	SWU12 ID Register n	0x00000000
0x310A8020	SWU12_CNT[n]	SWU12 Count Register n	0x00000000
0x310A8024	SWU12_TARG[n]	SWU12 Target Register n	0x00000000
0x310A8028	SWU12_HIST[n]	SWU12 Bandwidth History Register n	0x00000000
0x310A802C	SWU12_CUR[n]	SWU12 Current Register n	0x00000000
0x310A8030	SWU12_CTL[n]	SWU12 Control Register n	0x00000000
0x310A8034	SWU12_LA[n]	SWU12 Lower Address Register n	0x00000000
0x310A8038	SWU12_UA[n]	SWU12 Upper Address Register n	0x00000000
0x310A803C	SWU12_ID[n]	SWU12 ID Register n	0x00000000
0x310A8040	SWU12_CNT[n]	SWU12 Count Register n	0x00000000
0x310A8044	SWU12_TARG[n]	SWU12 Target Register n	0x00000000
0x310A8048	SWU12_HIST[n]	SWU12 Bandwidth History Register n	0x00000000

Table A-168: ADSP-2159x SWU12 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310A804C	SWU12_CUR[n]	SWU12 Current Register n	0x00000000
0x310A8050	SWU12_CTL[n]	SWU12 Control Register n	0x00000000
0x310A8054	SWU12_LA[n]	SWU12 Lower Address Register n	0x00000000
0x310A8058	SWU12_UA[n]	SWU12 Upper Address Register n	0x00000000
0x310A805C	SWU12_ID[n]	SWU12 ID Register n	0x00000000
0x310A8060	SWU12_CNT[n]	SWU12 Count Register n	0x00000000
0x310A8064	SWU12_TARG[n]	SWU12 Target Register n	0x00000000
0x310A8068	SWU12_HIST[n]	SWU12 Bandwidth History Register n	0x00000000
0x310A806C	SWU12_CUR[n]	SWU12 Current Register n	0x00000000
0x310A8070	SWU12_CTL[n]	SWU12 Control Register n	0x00000000
0x310A8074	SWU12_LA[n]	SWU12 Lower Address Register n	0x00000000
0x310A8078	SWU12_UA[n]	SWU12 Upper Address Register n	0x00000000
0x310A807C	SWU12_ID[n]	SWU12 ID Register n	0x00000000
0x310A8080	SWU12_CNT[n]	SWU12 Count Register n	0x00000000
0x310A8084	SWU12_TARG[n]	SWU12 Target Register n	0x00000000
0x310A8088	SWU12_HIST[n]	SWU12 Bandwidth History Register n	0x00000000
0x310A808C	SWU12_CUR[n]	SWU12 Current Register n	0x00000000

Table A-169: ADSP-2159x SWU13 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109E000	SWU13_GCTL	SWU13 Global Control Register	0x00000000
0x3109E004	SWU13_GSTAT	SWU13 Global Status Register	0x00000000
0x3109E010	SWU13_CTL[n]	SWU13 Control Register n	0x00000000
0x3109E014	SWU13_LA[n]	SWU13 Lower Address Register n	0x00000000
0x3109E018	SWU13_UA[n]	SWU13 Upper Address Register n	0x00000000
0x3109E01C	SWU13_ID[n]	SWU13 ID Register n	0x00000000
0x3109E020	SWU13_CNT[n]	SWU13 Count Register n	0x00000000
0x3109E024	SWU13_TARG[n]	SWU13 Target Register n	0x00000000
0x3109E028	SWU13_HIST[n]	SWU13 Bandwidth History Register n	0x00000000
0x3109E02C	SWU13_CUR[n]	SWU13 Current Register n	0x00000000

Table A-169: ADSP-2159x SWU13 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3109E030	SWU13_CTL[n]	SWU13 Control Register n	0x00000000
0x3109E034	SWU13_LA[n]	SWU13 Lower Address Register n	0x00000000
0x3109E038	SWU13_UA[n]	SWU13 Upper Address Register n	0x00000000
0x3109E03C	SWU13_ID[n]	SWU13 ID Register n	0x00000000
0x3109E040	SWU13_CNT[n]	SWU13 Count Register n	0x00000000
0x3109E044	SWU13_TARG[n]	SWU13 Target Register n	0x00000000
0x3109E048	SWU13_HIST[n]	SWU13 Bandwidth History Register n	0x00000000
0x3109E04C	SWU13_CUR[n]	SWU13 Current Register n	0x00000000
0x3109E050	SWU13_CTL[n]	SWU13 Control Register n	0x00000000
0x3109E054	SWU13_LA[n]	SWU13 Lower Address Register n	0x00000000
0x3109E058	SWU13_UA[n]	SWU13 Upper Address Register n	0x00000000
0x3109E05C	SWU13_ID[n]	SWU13 ID Register n	0x00000000
0x3109E060	SWU13_CNT[n]	SWU13 Count Register n	0x00000000
0x3109E064	SWU13_TARG[n]	SWU13 Target Register n	0x00000000
0x3109E068	SWU13_HIST[n]	SWU13 Bandwidth History Register n	0x00000000
0x3109E06C	SWU13_CUR[n]	SWU13 Current Register n	0x00000000
0x3109E070	SWU13_CTL[n]	SWU13 Control Register n	0x00000000
0x3109E074	SWU13_LA[n]	SWU13 Lower Address Register n	0x00000000
0x3109E078	SWU13_UA[n]	SWU13 Upper Address Register n	0x00000000
0x3109E07C	SWU13_ID[n]	SWU13 ID Register n	0x00000000
0x3109E080	SWU13_CNT[n]	SWU13 Count Register n	0x00000000
0x3109E084	SWU13_TARG[n]	SWU13 Target Register n	0x00000000
0x3109E088	SWU13_HIST[n]	SWU13 Bandwidth History Register n	0x00000000
0x3109E08C	SWU13_CUR[n]	SWU13 Current Register n	0x00000000

Table A-170: ADSP-2159x SWU2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31094000	SWU2_GCTL	SWU2 Global Control Register	0x00000000
0x31094004	SWU2_GSTAT	SWU2 Global Status Register	0x00000000
0x31094010	SWU2_CTL[n]	SWU2 Control Register n	0x00000000

Table A-170: ADSP-2159x SWU2 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31094014	SWU2_LA[n]	SWU2 Lower Address Register n	0x00000000
0x31094018	SWU2_UA[n]	SWU2 Upper Address Register n	0x00000000
0x3109401C	SWU2_ID[n]	SWU2 ID Register n	0x00000000
0x31094020	SWU2_CNT[n]	SWU2 Count Register n	0x00000000
0x31094024	SWU2_TARG[n]	SWU2 Target Register n	0x00000000
0x31094028	SWU2_HIST[n]	SWU2 Bandwidth History Register n	0x00000000
0x3109402C	SWU2_CUR[n]	SWU2 Current Register n	0x00000000
0x31094030	SWU2_CTL[n]	SWU2 Control Register n	0x00000000
0x31094034	SWU2_LA[n]	SWU2 Lower Address Register n	0x00000000
0x31094038	SWU2_UA[n]	SWU2 Upper Address Register n	0x00000000
0x3109403C	SWU2_ID[n]	SWU2 ID Register n	0x00000000
0x31094040	SWU2_CNT[n]	SWU2 Count Register n	0x00000000
0x31094044	SWU2_TARG[n]	SWU2 Target Register n	0x00000000
0x31094048	SWU2_HIST[n]	SWU2 Bandwidth History Register n	0x00000000
0x3109404C	SWU2_CUR[n]	SWU2 Current Register n	0x00000000
0x31094050	SWU2_CTL[n]	SWU2 Control Register n	0x00000000
0x31094054	SWU2_LA[n]	SWU2 Lower Address Register n	0x00000000
0x31094058	SWU2_UA[n]	SWU2 Upper Address Register n	0x00000000
0x3109405C	SWU2_ID[n]	SWU2 ID Register n	0x00000000
0x31094060	SWU2_CNT[n]	SWU2 Count Register n	0x00000000
0x31094064	SWU2_TARG[n]	SWU2 Target Register n	0x00000000
0x31094068	SWU2_HIST[n]	SWU2 Bandwidth History Register n	0x00000000
0x3109406C	SWU2_CUR[n]	SWU2 Current Register n	0x00000000
0x31094070	SWU2_CTL[n]	SWU2 Control Register n	0x00000000
0x31094074	SWU2_LA[n]	SWU2 Lower Address Register n	0x00000000
0x31094078	SWU2_UA[n]	SWU2 Upper Address Register n	0x00000000
0x3109407C	SWU2_ID[n]	SWU2 ID Register n	0x00000000
0x31094080	SWU2_CNT[n]	SWU2 Count Register n	0x00000000
0x31094084	SWU2_TARG[n]	SWU2 Target Register n	0x00000000
0x31094088	SWU2_HIST[n]	SWU2 Bandwidth History Register n	0x00000000
0x3109408C	SWU2_CUR[n]	SWU2 Current Register n	0x00000000

Table A-171: ADSP-2159x SWU3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31095000	SWU3_GCTL	SWU3 Global Control Register	0x00000000
0x31095004	SWU3_GSTAT	SWU3 Global Status Register	0x00000000
0x31095010	SWU3_CTL[n]	SWU3 Control Register n	0x00000000
0x31095014	SWU3_LA[n]	SWU3 Lower Address Register n	0x00000000
0x31095018	SWU3_UA[n]	SWU3 Upper Address Register n	0x00000000
0x3109501C	SWU3_ID[n]	SWU3 ID Register n	0x00000000
0x31095020	SWU3_CNT[n]	SWU3 Count Register n	0x00000000
0x31095024	SWU3_TARG[n]	SWU3 Target Register n	0x00000000
0x31095028	SWU3_HIST[n]	SWU3 Bandwidth History Register n	0x00000000
0x3109502C	SWU3_CUR[n]	SWU3 Current Register n	0x00000000
0x31095030	SWU3_CTL[n]	SWU3 Control Register n	0x00000000
0x31095034	SWU3_LA[n]	SWU3 Lower Address Register n	0x00000000
0x31095038	SWU3_UA[n]	SWU3 Upper Address Register n	0x00000000
0x3109503C	SWU3_ID[n]	SWU3 ID Register n	0x00000000
0x31095040	SWU3_CNT[n]	SWU3 Count Register n	0x00000000
0x31095044	SWU3_TARG[n]	SWU3 Target Register n	0x00000000
0x31095048	SWU3_HIST[n]	SWU3 Bandwidth History Register n	0x00000000
0x3109504C	SWU3_CUR[n]	SWU3 Current Register n	0x00000000
0x31095050	SWU3_CTL[n]	SWU3 Control Register n	0x00000000
0x31095054	SWU3_LA[n]	SWU3 Lower Address Register n	0x00000000
0x31095058	SWU3_UA[n]	SWU3 Upper Address Register n	0x00000000
0x3109505C	SWU3_ID[n]	SWU3 ID Register n	0x00000000
0x31095060	SWU3_CNT[n]	SWU3 Count Register n	0x00000000
0x31095064	SWU3_TARG[n]	SWU3 Target Register n	0x00000000
0x31095068	SWU3_HIST[n]	SWU3 Bandwidth History Register n	0x00000000
0x3109506C	SWU3_CUR[n]	SWU3 Current Register n	0x00000000
0x31095070	SWU3_CTL[n]	SWU3 Control Register n	0x00000000
0x31095074	SWU3_LA[n]	SWU3 Lower Address Register n	0x00000000
0x31095078	SWU3_UA[n]	SWU3 Upper Address Register n	0x00000000
0x3109507C	SWU3_ID[n]	SWU3 ID Register n	0x00000000
0x31095080	SWU3_CNT[n]	SWU3 Count Register n	0x00000000

Table A-171: ADSP-2159x SWU3 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31095084	SWU3_TARG[n]	SWU3 Target Register n	0x00000000
0x31095088	SWU3_HIST[n]	SWU3 Bandwidth History Register n	0x00000000
0x3109508C	SWU3_CUR[n]	SWU3 Current Register n	0x00000000

Table A-172: ADSP-2159x SWU4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31096000	SWU4_GCTL	SWU4 Global Control Register	0x00000000
0x31096004	SWU4_GSTAT	SWU4 Global Status Register	0x00000000
0x31096010	SWU4_CTL[n]	SWU4 Control Register n	0x00000000
0x31096014	SWU4_LA[n]	SWU4 Lower Address Register n	0x00000000
0x31096018	SWU4_UA[n]	SWU4 Upper Address Register n	0x00000000
0x3109601C	SWU4_ID[n]	SWU4 ID Register n	0x00000000
0x31096020	SWU4_CNT[n]	SWU4 Count Register n	0x00000000
0x31096024	SWU4_TARG[n]	SWU4 Target Register n	0x00000000
0x31096028	SWU4_HIST[n]	SWU4 Bandwidth History Register n	0x00000000
0x3109602C	SWU4_CUR[n]	SWU4 Current Register n	0x00000000
0x31096030	SWU4_CTL[n]	SWU4 Control Register n	0x00000000
0x31096034	SWU4_LA[n]	SWU4 Lower Address Register n	0x00000000
0x31096038	SWU4_UA[n]	SWU4 Upper Address Register n	0x00000000
0x3109603C	SWU4_ID[n]	SWU4 ID Register n	0x00000000
0x31096040	SWU4_CNT[n]	SWU4 Count Register n	0x00000000
0x31096044	SWU4_TARG[n]	SWU4 Target Register n	0x00000000
0x31096048	SWU4_HIST[n]	SWU4 Bandwidth History Register n	0x00000000
0x3109604C	SWU4_CUR[n]	SWU4 Current Register n	0x00000000
0x31096050	SWU4_CTL[n]	SWU4 Control Register n	0x00000000
0x31096054	SWU4_LA[n]	SWU4 Lower Address Register n	0x00000000
0x31096058	SWU4_UA[n]	SWU4 Upper Address Register n	0x00000000
0x3109605C	SWU4_ID[n]	SWU4 ID Register n	0x00000000
0x31096060	SWU4_CNT[n]	SWU4 Count Register n	0x00000000
0x31096064	SWU4_TARG[n]	SWU4 Target Register n	0x00000000

Table A-172: ADSP-2159x SWU4 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31096068	SWU4_HIST[n]	SWU4 Bandwidth History Register n	0x00000000
0x3109606C	SWU4_CUR[n]	SWU4 Current Register n	0x00000000
0x31096070	SWU4_CTL[n]	SWU4 Control Register n	0x00000000
0x31096074	SWU4_LA[n]	SWU4 Lower Address Register n	0x00000000
0x31096078	SWU4_UA[n]	SWU4 Upper Address Register n	0x00000000
0x3109607C	SWU4_ID[n]	SWU4 ID Register n	0x00000000
0x31096080	SWU4_CNT[n]	SWU4 Count Register n	0x00000000
0x31096084	SWU4_TARG[n]	SWU4 Target Register n	0x00000000
0x31096088	SWU4_HIST[n]	SWU4 Bandwidth History Register n	0x00000000
0x3109608C	SWU4_CUR[n]	SWU4 Current Register n	0x00000000

Table A-173: ADSP-2159x SWU5 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31098000	SWU5_GCTL	SWU5 Global Control Register	0x00000000
0x31098004	SWU5_GSTAT	SWU5 Global Status Register	0x00000000
0x31098010	SWU5_CTL[n]	SWU5 Control Register n	0x00000000
0x31098014	SWU5_LA[n]	SWU5 Lower Address Register n	0x00000000
0x31098018	SWU5_UA[n]	SWU5 Upper Address Register n	0x00000000
0x3109801C	SWU5_ID[n]	SWU5 ID Register n	0x00000000
0x31098020	SWU5_CNT[n]	SWU5 Count Register n	0x00000000
0x31098024	SWU5_TARG[n]	SWU5 Target Register n	0x00000000
0x31098028	SWU5_HIST[n]	SWU5 Bandwidth History Register n	0x00000000
0x3109802C	SWU5_CUR[n]	SWU5 Current Register n	0x00000000
0x31098030	SWU5_CTL[n]	SWU5 Control Register n	0x00000000
0x31098034	SWU5_LA[n]	SWU5 Lower Address Register n	0x00000000
0x31098038	SWU5_UA[n]	SWU5 Upper Address Register n	0x00000000
0x3109803C	SWU5_ID[n]	SWU5 ID Register n	0x00000000
0x31098040	SWU5_CNT[n]	SWU5 Count Register n	0x00000000
0x31098044	SWU5_TARG[n]	SWU5 Target Register n	0x00000000
0x31098048	SWU5_HIST[n]	SWU5 Bandwidth History Register n	0x00000000

Table A-173: ADSP-2159x SWU5 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3109804C	SWU5_CUR[n]	SWU5 Current Register n	0x00000000
0x31098050	SWU5_CTL[n]	SWU5 Control Register n	0x00000000
0x31098054	SWU5_LA[n]	SWU5 Lower Address Register n	0x00000000
0x31098058	SWU5_UA[n]	SWU5 Upper Address Register n	0x00000000
0x3109805C	SWU5_ID[n]	SWU5 ID Register n	0x00000000
0x31098060	SWU5_CNT[n]	SWU5 Count Register n	0x00000000
0x31098064	SWU5_TARG[n]	SWU5 Target Register n	0x00000000
0x31098068	SWU5_HIST[n]	SWU5 Bandwidth History Register n	0x00000000
0x3109806C	SWU5_CUR[n]	SWU5 Current Register n	0x00000000
0x31098070	SWU5_CTL[n]	SWU5 Control Register n	0x00000000
0x31098074	SWU5_LA[n]	SWU5 Lower Address Register n	0x00000000
0x31098078	SWU5_UA[n]	SWU5 Upper Address Register n	0x00000000
0x3109807C	SWU5_ID[n]	SWU5 ID Register n	0x00000000
0x31098080	SWU5_CNT[n]	SWU5 Count Register n	0x00000000
0x31098084	SWU5_TARG[n]	SWU5 Target Register n	0x00000000
0x31098088	SWU5_HIST[n]	SWU5 Bandwidth History Register n	0x00000000
0x3109808C	SWU5_CUR[n]	SWU5 Current Register n	0x00000000

Table A-174: ADSP-2159x SWU7 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31140000	SWU7_GCTL	SWU7 Global Control Register	0x00000000
0x31140004	SWU7_GSTAT	SWU7 Global Status Register	0x00000000
0x31140010	SWU7_CTL[n]	SWU7 Control Register n	0x00000000
0x31140014	SWU7_LA[n]	SWU7 Lower Address Register n	0x00000000
0x31140018	SWU7_UA[n]	SWU7 Upper Address Register n	0x00000000
0x3114001C	SWU7_ID[n]	SWU7 ID Register n	0x00000000
0x31140020	SWU7_CNT[n]	SWU7 Count Register n	0x00000000
0x31140024	SWU7_TARG[n]	SWU7 Target Register n	0x00000000
0x31140028	SWU7_HIST[n]	SWU7 Bandwidth History Register n	0x00000000
0x3114002C	SWU7_CUR[n]	SWU7 Current Register n	0x00000000

Table A-174: ADSP-2159x SWU7 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31140030	SWU7_CTL[n]	SWU7 Control Register n	0x00000000
0x31140034	SWU7_LA[n]	SWU7 Lower Address Register n	0x00000000
0x31140038	SWU7_UA[n]	SWU7 Upper Address Register n	0x00000000
0x3114003C	SWU7_ID[n]	SWU7 ID Register n	0x00000000
0x31140040	SWU7_CNT[n]	SWU7 Count Register n	0x00000000
0x31140044	SWU7_TARG[n]	SWU7 Target Register n	0x00000000
0x31140048	SWU7_HIST[n]	SWU7 Bandwidth History Register n	0x00000000
0x3114004C	SWU7_CUR[n]	SWU7 Current Register n	0x00000000
0x31140050	SWU7_CTL[n]	SWU7 Control Register n	0x00000000
0x31140054	SWU7_LA[n]	SWU7 Lower Address Register n	0x00000000
0x31140058	SWU7_UA[n]	SWU7 Upper Address Register n	0x00000000
0x3114005C	SWU7_ID[n]	SWU7 ID Register n	0x00000000
0x31140060	SWU7_CNT[n]	SWU7 Count Register n	0x00000000
0x31140064	SWU7_TARG[n]	SWU7 Target Register n	0x00000000
0x31140068	SWU7_HIST[n]	SWU7 Bandwidth History Register n	0x00000000
0x3114006C	SWU7_CUR[n]	SWU7 Current Register n	0x00000000
0x31140070	SWU7_CTL[n]	SWU7 Control Register n	0x00000000
0x31140074	SWU7_LA[n]	SWU7 Lower Address Register n	0x00000000
0x31140078	SWU7_UA[n]	SWU7 Upper Address Register n	0x00000000
0x3114007C	SWU7_ID[n]	SWU7 ID Register n	0x00000000
0x31140080	SWU7_CNT[n]	SWU7 Count Register n	0x00000000
0x31140084	SWU7_TARG[n]	SWU7 Target Register n	0x00000000
0x31140088	SWU7_HIST[n]	SWU7 Bandwidth History Register n	0x00000000
0x3114008C	SWU7_CUR[n]	SWU7 Current Register n	0x00000000

Table A-175: ADSP-2159x SWU8 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31141000	SWU8_GCTL	SWU8 Global Control Register	0x00000000
0x31141004	SWU8_GSTAT	SWU8 Global Status Register	0x00000000
0x31141010	SWU8_CTL[n]	SWU8 Control Register n	0x00000000

Table A-175: ADSP-2159x SWU8 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31141014	SWU8_LA[n]	SWU8 Lower Address Register n	0x00000000
0x31141018	SWU8_UA[n]	SWU8 Upper Address Register n	0x00000000
0x3114101C	SWU8_ID[n]	SWU8 ID Register n	0x00000000
0x31141020	SWU8_CNT[n]	SWU8 Count Register n	0x00000000
0x31141024	SWU8_TARG[n]	SWU8 Target Register n	0x00000000
0x31141028	SWU8_HIST[n]	SWU8 Bandwidth History Register n	0x00000000
0x3114102C	SWU8_CUR[n]	SWU8 Current Register n	0x00000000
0x31141030	SWU8_CTL[n]	SWU8 Control Register n	0x00000000
0x31141034	SWU8_LA[n]	SWU8 Lower Address Register n	0x00000000
0x31141038	SWU8_UA[n]	SWU8 Upper Address Register n	0x00000000
0x3114103C	SWU8_ID[n]	SWU8 ID Register n	0x00000000
0x31141040	SWU8_CNT[n]	SWU8 Count Register n	0x00000000
0x31141044	SWU8_TARG[n]	SWU8 Target Register n	0x00000000
0x31141048	SWU8_HIST[n]	SWU8 Bandwidth History Register n	0x00000000
0x3114104C	SWU8_CUR[n]	SWU8 Current Register n	0x00000000
0x31141050	SWU8_CTL[n]	SWU8 Control Register n	0x00000000
0x31141054	SWU8_LA[n]	SWU8 Lower Address Register n	0x00000000
0x31141058	SWU8_UA[n]	SWU8 Upper Address Register n	0x00000000
0x3114105C	SWU8_ID[n]	SWU8 ID Register n	0x00000000
0x31141060	SWU8_CNT[n]	SWU8 Count Register n	0x00000000
0x31141064	SWU8_TARG[n]	SWU8 Target Register n	0x00000000
0x31141068	SWU8_HIST[n]	SWU8 Bandwidth History Register n	0x00000000
0x3114106C	SWU8_CUR[n]	SWU8 Current Register n	0x00000000
0x31141070	SWU8_CTL[n]	SWU8 Control Register n	0x00000000
0x31141074	SWU8_LA[n]	SWU8 Lower Address Register n	0x00000000
0x31141078	SWU8_UA[n]	SWU8 Upper Address Register n	0x00000000
0x3114107C	SWU8_ID[n]	SWU8 ID Register n	0x00000000
0x31141080	SWU8_CNT[n]	SWU8 Count Register n	0x00000000
0x31141084	SWU8_TARG[n]	SWU8 Target Register n	0x00000000
0x31141088	SWU8_HIST[n]	SWU8 Bandwidth History Register n	0x00000000
0x3114108C	SWU8_CUR[n]	SWU8 Current Register n	0x00000000

Table A-176: ADSP-2159x SWU9 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31142000	SWU9_GCTL	SWU9 Global Control Register	0x00000000
0x31142004	SWU9_GSTAT	SWU9 Global Status Register	0x00000000
0x31142010	SWU9_CTL[n]	SWU9 Control Register n	0x00000000
0x31142014	SWU9_LA[n]	SWU9 Lower Address Register n	0x00000000
0x31142018	SWU9_UA[n]	SWU9 Upper Address Register n	0x00000000
0x3114201C	SWU9_ID[n]	SWU9 ID Register n	0x00000000
0x31142020	SWU9_CNT[n]	SWU9 Count Register n	0x00000000
0x31142024	SWU9_TARG[n]	SWU9 Target Register n	0x00000000
0x31142028	SWU9_HIST[n]	SWU9 Bandwidth History Register n	0x00000000
0x3114202C	SWU9_CUR[n]	SWU9 Current Register n	0x00000000
0x31142030	SWU9_CTL[n]	SWU9 Control Register n	0x00000000
0x31142034	SWU9_LA[n]	SWU9 Lower Address Register n	0x00000000
0x31142038	SWU9_UA[n]	SWU9 Upper Address Register n	0x00000000
0x3114203C	SWU9_ID[n]	SWU9 ID Register n	0x00000000
0x31142040	SWU9_CNT[n]	SWU9 Count Register n	0x00000000
0x31142044	SWU9_TARG[n]	SWU9 Target Register n	0x00000000
0x31142048	SWU9_HIST[n]	SWU9 Bandwidth History Register n	0x00000000
0x3114204C	SWU9_CUR[n]	SWU9 Current Register n	0x00000000
0x31142050	SWU9_CTL[n]	SWU9 Control Register n	0x00000000
0x31142054	SWU9_LA[n]	SWU9 Lower Address Register n	0x00000000
0x31142058	SWU9_UA[n]	SWU9 Upper Address Register n	0x00000000
0x3114205C	SWU9_ID[n]	SWU9 ID Register n	0x00000000
0x31142060	SWU9_CNT[n]	SWU9 Count Register n	0x00000000
0x31142064	SWU9_TARG[n]	SWU9 Target Register n	0x00000000
0x31142068	SWU9_HIST[n]	SWU9 Bandwidth History Register n	0x00000000
0x3114206C	SWU9_CUR[n]	SWU9 Current Register n	0x00000000
0x31142070	SWU9_CTL[n]	SWU9 Control Register n	0x00000000
0x31142074	SWU9_LA[n]	SWU9 Lower Address Register n	0x00000000
0x31142078	SWU9_UA[n]	SWU9 Upper Address Register n	0x00000000
0x3114207C	SWU9_ID[n]	SWU9 ID Register n	0x00000000
0x31142080	SWU9_CNT[n]	SWU9 Count Register n	0x00000000

Table A-176: ADSP-2159x SWU9 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31142084	SWU9_TARG[n]	SWU9 Target Register n	0x00000000
0x31142088	SWU9_HIST[n]	SWU9 Bandwidth History Register n	0x00000000
0x3114208C	SWU9_CUR[n]	SWU9 Current Register n	0x00000000

Table A-177: ADSP-2159x TAPC MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31130000	TAPC_IDCODE	TAPC IDCODE Register	0x028240CB
0x31130004	TAPC_USERCODE	TAPC USERCODE Register	0x00000000
0x31130008	TAPC_SDBGKEY_CTL	TAPC Secure Debug Key Control Register	0x00000000
0x3113000C	TAPC_SDBGKEY_STAT	TAPC Secure Debug Key Status Register	0x00000000
0x31130010	TAPC_SDBGKEY0	TAPC Secure Debug Key 0 Register	0x00000000
0x31130014	TAPC_SDBGKEY1	TAPC Secure Debug Key 1 Register	0x00000000
0x31130018	TAPC_SDBGKEY2	TAPC Secure Debug Key 2 Register	0x00000000
0x3113001C	TAPC_SDBGKEY3	TAPC Secure Debug Key 3 Register	0x00000000
0x31131000	TAPC_DBGCTL	TAPC Debug Control Register	0x00000000

Table A-178: ADSP-2159x TIMER0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31018004	TIMER0_RUN	TIMER0 Run Register	0x00000000
0x31018008	TIMER0_RUN_SET	TIMER0 Run Set Register	0x00000000
0x3101800C	TIMER0_RUN_CLR	TIMER0 Run Clear Register	0x00000000
0x31018010	TIMER0_STOP_CFG	TIMER0 Stop Configuration Register	0x00000000
0x31018014	TIMER0_STOP_CFG_SET	TIMER0 Stop Configuration Set Register	0x00000000
0x31018018	TIMER0_STOP_CFG_CLR	TIMER0 Stop Configuration Clear Register	0x00000000
0x3101801C	TIMER0_DATA_IMSK	TIMER0 Data Interrupt Mask Register	0x0000FFFF
0x31018020	TIMER0_STAT_IMSK	TIMER0 Status Interrupt Mask Register	0x0000FFFF
0x31018024	TIMER0_TRG_MSK	TIMER0 Trigger Master Mask Register	0x0000FFFF
0x31018028	TIMER0_TRG_IE	TIMER0 Trigger Slave Enable Register	0x00000000
0x3101802C	TIMER0_DATA_ILAT	TIMER0 Data Interrupt Latch Register	0x00000000
0x31018030	TIMER0_STAT_ILAT	TIMER0 Status Interrupt Latch Register	0x00000000

Table A-178: ADSP-2159x TIMER0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31018034	TIMER0_ERR_TYPE	TIMER0 Error Type Status Register	0x00000000
0x31018038	TIMER0_BCAST_PER	TIMER0 Broadcast Period Register	0x00000000
0x3101803C	TIMER0_BCAST_WID	TIMER0 Broadcast Width Register	0x00000000
0x31018040	TIMER0_BCAST_DLY	TIMER0 Broadcast Delay Register	0x00000000
0x31018060	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018064	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018068	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101806C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018070	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018080	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018084	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018088	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101808C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018090	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310180A0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310180A4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310180A8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x310180AC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310180B0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310180C0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310180C4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310180C8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x310180CC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310180D0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310180E0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310180E4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310180E8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x310180EC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310180F0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018100	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018104	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001

Table A-178: ADSP-2159x TIMER0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31018108	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101810C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018110	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018120	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018124	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018128	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101812C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018130	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018140	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018144	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018148	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101814C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018150	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018160	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018164	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018168	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101816C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018170	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018180	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018184	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018188	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101818C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018190	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310181A0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310181A4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310181A8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x310181AC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310181B0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310181C0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310181C4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310181C8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000

Table A-178: ADSP-2159x TIMER0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310181CC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310181D0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x310181E0	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x310181E4	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x310181E8	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x310181EC	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x310181F0	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018200	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018204	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018208	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101820C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018210	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018220	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018224	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018228	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101822C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018230	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000
0x31018240	TIMER0_TMR[n]_CFG	TIMER0 Timer n Configuration Register	0x00000000
0x31018244	TIMER0_TMR[n]_CNT	TIMER0 Timer n Counter Register	0x00000001
0x31018248	TIMER0_TMR[n]_PER	TIMER0 Timer n Period Register	0x00000000
0x3101824C	TIMER0_TMR[n]_WID	TIMER0 Timer n Width Register	0x00000000
0x31018250	TIMER0_TMR[n]_DLY	TIMER0 Timer n Delay Register	0x00000000

Table A-179: ADSP-2159x TMU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31016800	TMU0_CTL	TMU0 TMU Control Register	0x00000000
0x31016804	TMU0_TEMP	TMU0 Temperature Value Register	0x00000000
0x31016808	TMU0_AVG	TMU0 Averaging Register	0x00000000
0x3101680C	TMU0_FLT_LIM_HI	TMU0 Fault High Limit Register	0x000000FF
0x31016810	TMU0_ALRT_LIM_HI	TMU0 Alert High Limit Register	0x000000FF

Table A-179: ADSP-2159x TMU0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31016814	TMU0_FLT_LIM_LO	TMU0 Fault Low Limit Register	0x00000100
0x31016818	TMU0_ALRT_LIM_LO	TMU0 Alert Low Limit Register	0x00000100
0x3101681C	TMU0_STAT	TMU0 Status Register	0x00000000
0x31016824	TMU0_GAIN	TMU0 Gain Value Register	0x00000000
0x31016828	TMU0_IMSK	TMU0 Interrupt Mask Register	0x0000000F
0x3101682C	TMU0_OFFSET	TMU0 Offset Register	0x00000000
0x31016834	TMU0_CNV_BLANK	TMU0 Temperature conversion blank register	0x03100000
0x31016838	TMU0_REFR_CNTR	TMU0 Temperature Refresh Counter	0x00000000

Table A-180: ADSP-2159x TRNG0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310D0000	TRNG0_INPUT[n]	TRNG0 TRNG Input Registers	0x00000000
0x310D0000	TRNG0_OUTPUT[n]	TRNG0 TRNG Output Registers	0x00000000
0x310D0004	TRNG0_INPUT[n]	TRNG0 TRNG Input Registers	0x00000000
0x310D0004	TRNG0_OUTPUT[n]	TRNG0 TRNG Output Registers	0x00000000
0x310D0008	TRNG0_OUTPUT[n]	TRNG0 TRNG Output Registers	0x00000000
0x310D000C	TRNG0_OUTPUT[n]	TRNG0 TRNG Output Registers	0x00000000
0x310D0010	TRNG0_STAT	TRNG0 TRNG Status Register	0x00000000
0x310D0010	TRNG0_INTACK	TRNG0 TRNG Interrupt Acknowledge Register	0x00000000
0x310D0014	TRNG0_CTL	TRNG0 TRNG Control Register	0x00000000
0x310D0018	TRNG0_CFG	TRNG0 TRNG Configuration Register	0x00000000
0x310D001C	TRNG0_ALMCNT	TRNG0 TRNG Alarm Counter Register	0x000000FF
0x310D0020	TRNG0_FROEN	TRNG0 TRNG FRO Enable Register	0x000000FF
0x310D0024	TRNG0_FRODETUNE	TRNG0 TRNG FRO De-tune Register	0x00000000
0x310D0028	TRNG0_ALMMSK	TRNG0 TRNG Alarm Mask Register	0x00000000
0x310D002C	TRNG0_ALMSTP	TRNG0 TRNG Alarm Stop Register	0x00000000
0x310D0030	TRNG0_LFSR_L	TRNG0 TRNG LFSR Access Register	0x00000000
0x310D0034	TRNG0_LFSR_M	TRNG0 TRNG LFSR Access Register	0x00000000
0x310D0038	TRNG0_LFSR_H	TRNG0 TRNG LFSR Access Register	0x00000000
0x310D003C	TRNG0_CNT	TRNG0 Counter Register	0x00000000

Table A-180: ADSP-2159x TRNG0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310D0040	TRNG0_RUNCNT	TRNG0 TRNG Run Count Registers	0x00000000
0x310D0040	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0044	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D0044	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0048	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D0048	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D004C	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D004C	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0050	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D0050	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0054	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D0054	TRNG0_KEY[n]	TRNG0 Post-Process Key Registers	0x00000000
0x310D0058	TRNG0_RUN[n]	TRNG0 TRNG Run Test State and Result Registers	0x00000000
0x310D005C	TRNG0_MONOBITCNT	TRNG0 TRNG Monobit Test Result Register	0x00002710
0x310D0060	TRNG0_POKER[n]	TRNG0 TRNG Poker Test Result Registers	0x00000000
0x310D0060	TRNG0_V[n]	TRNG0 TRNG Post-Process "V" Value Registers	0x00000000
0x310D0064	TRNG0_POKER[n]	TRNG0 TRNG Poker Test Result Registers	0x00000000
0x310D0064	TRNG0_V[n]	TRNG0 TRNG Post-Process "V" Value Registers	0x00000000
0x310D0068	TRNG0_POKER[n]	TRNG0 TRNG Poker Test Result Registers	0x00000000
0x310D006C	TRNG0_POKER[n]	TRNG0 TRNG Poker Test Result Registers	0x00000000
0x310D0070	TRNG0_TEST	TRNG0 TRNG Test Register	0x00000000
0x310D0074	TRNG0_BLKCNT	TRNG0 TRNG Block Count Register	0x00000000

Table A-181: ADSP-2159x TRU0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A000	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A004	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A008	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A00C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A010	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000

Table A-181: ADSP-2159x TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A014	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A018	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A01C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A020	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A024	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A028	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A02C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A030	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A034	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A038	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A03C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A040	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A044	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A048	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A04C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A050	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A054	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A058	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A05C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A060	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A064	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A068	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A06C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A070	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A074	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A078	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A07C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A080	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A084	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A088	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A08C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000

Table A-181: ADSP-2159x TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A090	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A094	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A098	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A09C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0A0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0A4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0A8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0AC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0B0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0B4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0B8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0BC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0C0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0C4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0C8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0CC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0D0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0D4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0D8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0DC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0E0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0E4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0E8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0EC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0F0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0F4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0F8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A0FC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A100	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A104	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A108	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000

Table A-181: ADSP-2159x TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A10C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A110	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A114	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A118	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A11C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A120	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A124	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A128	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A12C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A130	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A134	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A138	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A13C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A140	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A144	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A148	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A14C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A150	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A154	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A158	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A15C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A160	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A164	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A168	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A16C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A170	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A174	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A178	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A17C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A180	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A184	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000

Table A-181: ADSP-2159x TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A188	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A18C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A190	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A194	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A198	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A19C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1A0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1A4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1A8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1AC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1B0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1B4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1B8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1BC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1C0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1C4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1C8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1CC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1D0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1D4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1D8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1DC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1E0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1E4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1E8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1EC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1F0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1F4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1F8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A1FC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A200	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000

Table A-181: ADSP-2159x TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A204	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A208	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A20C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A210	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A214	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A218	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A21C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A220	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A224	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A228	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A22C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A230	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A234	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A238	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A23C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A240	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A244	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A248	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A24C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A250	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A254	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A258	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A25C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A260	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A264	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A268	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A26C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A270	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A274	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A278	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A27C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000

Table A-181: ADSP-2159x TRU0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3108A280	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A284	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A288	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A28C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A290	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A294	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A298	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A29C	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2A0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2A4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2A8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2AC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2B0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2B4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2B8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2BC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2C0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2C4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2C8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2CC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2D0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2D4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2D8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2DC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2E0	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2E4	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2E8	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A2EC	TRU0_SSR[n]	TRU0 Slave Select Register	0x00000000
0x3108A7E0	TRU0_MTR	TRU0 Master Trigger Register	0x00000000
0x3108A7E8	TRU0_ERRADDR	TRU0 Error Address Register	0x00000000
0x3108A7EC	TRU0_STAT	TRU0 Status Information Register	0x00000000

Table A-181: ADSP-2159x TRU0 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3108A7F4	TRU0_GCTL	TRU0 Global Control Register	0x00000000

Table A-182: ADSP-2159x TWI0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31001400	TWI0_CLKDIV	TWI0 SCL Clock Divider Register	0x00000000
0x31001404	TWI0_CTL	TWI0 Control Register	0x00000000
0x31001408	TWI0_SLVCTL	TWI0 Slave Mode Control Register	0x00000000
0x3100140C	TWI0_SLVSTAT	TWI0 Slave Mode Status Register	0x00000000
0x31001410	TWI0_SLVADDR	TWI0 Slave Mode Address Register	0x00000000
0x31001414	TWI0_MSTRCTL	TWI0 Master Mode Control Registers	0x00000000
0x31001418	TWI0_MSTRSTAT	TWI0 Master Mode Status Register	0x00000000
0x3100141C	TWI0_MSTRADDR	TWI0 Master Mode Address Register	0x00000000
0x31001420	TWI0_ISTAT	TWI0 Interrupt Status Register	0x00000000
0x31001424	TWI0_IMSK	TWI0 Interrupt Mask Register	0x00000000
0x31001428	TWI0_FIFCTL	TWI0 FIFO Control Register	0x00000000
0x3100142C	TWI0_FIFOSTAT	TWI0 FIFO Status Register	0x00000000
0x31001480	TWI0_TXDATA8	TWI0 Tx Data Single-Byte Register	0x00000000
0x31001484	TWI0_TXDATA16	TWI0 Tx Data Double-Byte Register	0x00000000
0x31001488	TWI0_RXDATA8	TWI0 Rx Data Single-Byte Register	0x00000000
0x3100148C	TWI0_RXDATA16	TWI0 Rx Data Double-Byte Register	0x00000000

Table A-183: ADSP-2159x TWI1 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31001500	TWI1_CLKDIV	TWI1 SCL Clock Divider Register	0x00000000
0x31001504	TWI1_CTL	TWI1 Control Register	0x00000000
0x31001508	TWI1_SLVCTL	TWI1 Slave Mode Control Register	0x00000000
0x3100150C	TWI1_SLVSTAT	TWI1 Slave Mode Status Register	0x00000000
0x31001510	TWI1_SLVADDR	TWI1 Slave Mode Address Register	0x00000000
0x31001514	TWI1_MSTRCTL	TWI1 Master Mode Control Registers	0x00000000
0x31001518	TWI1_MSTRSTAT	TWI1 Master Mode Status Register	0x00000000

Table A-183: ADSP-2159x TWI1 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x3100151C	TWI1_MSTRADDR	TWI1 Master Mode Address Register	0x00000000
0x31001520	TWI1_ISTAT	TWI1 Interrupt Status Register	0x00000000
0x31001524	TWI1_IMSK	TWI1 Interrupt Mask Register	0x00000000
0x31001528	TWI1_FIFCTL	TWI1 FIFO Control Register	0x00000000
0x3100152C	TWI1_FIFOSTAT	TWI1 FIFO Status Register	0x00000000
0x31001580	TWI1_TXDATA8	TWI1 Tx Data Single-Byte Register	0x00000000
0x31001584	TWI1_TXDATA16	TWI1 Tx Data Double-Byte Register	0x00000000
0x31001588	TWI1_RXDATA8	TWI1 Rx Data Single-Byte Register	0x00000000
0x3100158C	TWI1_RXDATA16	TWI1 Rx Data Double-Byte Register	0x00000000

Table A-184: ADSP-2159x TWI2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31001600	TWI2_CLKDIV	TWI2 SCL Clock Divider Register	0x00000000
0x31001604	TWI2_CTL	TWI2 Control Register	0x00000000
0x31001608	TWI2_SLVCTL	TWI2 Slave Mode Control Register	0x00000000
0x3100160C	TWI2_SLVSTAT	TWI2 Slave Mode Status Register	0x00000000
0x31001610	TWI2_SLVADDR	TWI2 Slave Mode Address Register	0x00000000
0x31001614	TWI2_MSTRCTL	TWI2 Master Mode Control Registers	0x00000000
0x31001618	TWI2_MSTRSTAT	TWI2 Master Mode Status Register	0x00000000
0x3100161C	TWI2_MSTRADDR	TWI2 Master Mode Address Register	0x00000000
0x31001620	TWI2_ISTAT	TWI2 Interrupt Status Register	0x00000000
0x31001624	TWI2_IMSK	TWI2 Interrupt Mask Register	0x00000000
0x31001628	TWI2_FIFCTL	TWI2 FIFO Control Register	0x00000000
0x3100162C	TWI2_FIFOSTAT	TWI2 FIFO Status Register	0x00000000
0x31001680	TWI2_TXDATA8	TWI2 Tx Data Single-Byte Register	0x00000000
0x31001684	TWI2_TXDATA16	TWI2 Tx Data Double-Byte Register	0x00000000
0x31001688	TWI2_RXDATA8	TWI2 Rx Data Single-Byte Register	0x00000000
0x3100168C	TWI2_RXDATA16	TWI2 Rx Data Double-Byte Register	0x00000000

Table A-185: ADSP-2159x TWI3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31001000	TWI3_CLKDIV	TWI3 SCL Clock Divider Register	0x00000000
0x31001004	TWI3_CTL	TWI3 Control Register	0x00000000
0x31001008	TWI3_SLVCTL	TWI3 Slave Mode Control Register	0x00000000
0x3100100C	TWI3_SLVSTAT	TWI3 Slave Mode Status Register	0x00000000
0x31001010	TWI3_SLVADDR	TWI3 Slave Mode Address Register	0x00000000
0x31001014	TWI3_MSTRCTL	TWI3 Master Mode Control Registers	0x00000000
0x31001018	TWI3_MSTRSTAT	TWI3 Master Mode Status Register	0x00000000
0x3100101C	TWI3_MSTRADDR	TWI3 Master Mode Address Register	0x00000000
0x31001020	TWI3_ISTAT	TWI3 Interrupt Status Register	0x00000000
0x31001024	TWI3_IMSK	TWI3 Interrupt Mask Register	0x00000000
0x31001028	TWI3_FIFCTL	TWI3 FIFO Control Register	0x00000000
0x3100102C	TWI3_FIFOSTAT	TWI3 FIFO Status Register	0x00000000
0x31001080	TWI3_TXDATA8	TWI3 Tx Data Single-Byte Register	0x00000000
0x31001084	TWI3_TXDATA16	TWI3 Tx Data Double-Byte Register	0x00000000
0x31001088	TWI3_RXDATA8	TWI3 Rx Data Single-Byte Register	0x00000000
0x3100108C	TWI3_RXDATA16	TWI3 Rx Data Double-Byte Register	0x00000000

Table A-186: ADSP-2159x TWI4 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31001100	TWI4_CLKDIV	TWI4 SCL Clock Divider Register	0x00000000
0x31001104	TWI4_CTL	TWI4 Control Register	0x00000000
0x31001108	TWI4_SLVCTL	TWI4 Slave Mode Control Register	0x00000000
0x3100110C	TWI4_SLVSTAT	TWI4 Slave Mode Status Register	0x00000000
0x31001110	TWI4_SLVADDR	TWI4 Slave Mode Address Register	0x00000000
0x31001114	TWI4_MSTRCTL	TWI4 Master Mode Control Registers	0x00000000
0x31001118	TWI4_MSTRSTAT	TWI4 Master Mode Status Register	0x00000000
0x3100111C	TWI4_MSTRADDR	TWI4 Master Mode Address Register	0x00000000
0x31001120	TWI4_ISTAT	TWI4 Interrupt Status Register	0x00000000
0x31001124	TWI4_IMSK	TWI4 Interrupt Mask Register	0x00000000
0x31001128	TWI4_FIFCTL	TWI4 FIFO Control Register	0x00000000

Table A-186: ADSP-2159x TWI4 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x3100112C	TWI4_FIFOSTAT	TWI4 FIFO Status Register	0x00000000
0x31001180	TWI4_TXDATA8	TWI4 Tx Data Single-Byte Register	0x00000000
0x31001184	TWI4_TXDATA16	TWI4 Tx Data Double-Byte Register	0x00000000
0x31001188	TWI4_RXDATA8	TWI4 Rx Data Single-Byte Register	0x00000000
0x3100118C	TWI4_RXDATA16	TWI4 Rx Data Double-Byte Register	0x00000000

Table A-187: ADSP-2159x TWI5 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31001200	TWI5_CLKDIV	TWI5 SCL Clock Divider Register	0x00000000
0x31001204	TWI5_CTL	TWI5 Control Register	0x00000000
0x31001208	TWI5_SLVCTL	TWI5 Slave Mode Control Register	0x00000000
0x3100120C	TWI5_SLVSTAT	TWI5 Slave Mode Status Register	0x00000000
0x31001210	TWI5_SLVADDR	TWI5 Slave Mode Address Register	0x00000000
0x31001214	TWI5_MSTRCTL	TWI5 Master Mode Control Registers	0x00000000
0x31001218	TWI5_MSTRSTAT	TWI5 Master Mode Status Register	0x00000000
0x3100121C	TWI5_MSTRADDR	TWI5 Master Mode Address Register	0x00000000
0x31001220	TWI5_ISTAT	TWI5 Interrupt Status Register	0x00000000
0x31001224	TWI5_IMSK	TWI5 Interrupt Mask Register	0x00000000
0x31001228	TWI5_FIFCTL	TWI5 FIFO Control Register	0x00000000
0x3100122C	TWI5_FIFOSTAT	TWI5 FIFO Status Register	0x00000000
0x31001280	TWI5_TXDATA8	TWI5 Tx Data Single-Byte Register	0x00000000
0x31001284	TWI5_TXDATA16	TWI5 Tx Data Double-Byte Register	0x00000000
0x31001288	TWI5_RXDATA8	TWI5 Rx Data Single-Byte Register	0x00000000
0x3100128C	TWI5_RXDATA16	TWI5 Rx Data Double-Byte Register	0x00000000

Table A-188: ADSP-2159x UART0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x31003004	UART0_CTL	UART0 Control Register	0x00000000
0x31003008	UART0_STAT	UART0 Status Register	0x000000A0
0x3100300C	UART0_SCR	UART0 Scratch Register	0x00000000

Table A-188: ADSP-2159x UART0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x31003010	UART0_CLK	UART0 Clock Rate Register	0x0000FFFF
0x31003014	UART0_IMSK	UART0 Interrupt Mask Register	0x00000000
0x31003018	UART0_IMSK_SET	UART0 Interrupt Mask Set Register	0x00000000
0x3100301C	UART0_IMSK_CLR	UART0 Interrupt Mask Clear Register	0x00000000
0x31003020	UART0_RBR	UART0 Receive Buffer Register	0x00000000
0x31003024	UART0_THR	UART0 Transmit Hold Register	0x00000000
0x31003028	UART0_TAIP	UART0 Transmit Address/Insert Pulse Register	0x00000000
0x3100302C	UART0_TSR	UART0 Transmit Shift Register	0x000007FF
0x31003030	UART0_RSR	UART0 Receive Shift Register	0x00000000
0x31003034	UART0_TXCNT	UART0 Transmit Counter Register	0x00000000
0x31003038	UART0_RXCNT	UART0 Receive Counter Register	0x00000000

Table A-189: ADSP-2159x UART1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31003404	UART1_CTL	UART1 Control Register	0x00000000
0x31003408	UART1_STAT	UART1 Status Register	0x000000A0
0x3100340C	UART1_SCR	UART1 Scratch Register	0x00000000
0x31003410	UART1_CLK	UART1 Clock Rate Register	0x0000FFFF
0x31003414	UART1_IMSK	UART1 Interrupt Mask Register	0x00000000
0x31003418	UART1_IMSK_SET	UART1 Interrupt Mask Set Register	0x00000000
0x3100341C	UART1_IMSK_CLR	UART1 Interrupt Mask Clear Register	0x00000000
0x31003420	UART1_RBR	UART1 Receive Buffer Register	0x00000000
0x31003424	UART1_THR	UART1 Transmit Hold Register	0x00000000
0x31003428	UART1_TAIP	UART1 Transmit Address/Insert Pulse Register	0x00000000
0x3100342C	UART1_TSR	UART1 Transmit Shift Register	0x000007FF
0x31003430	UART1_RSR	UART1 Receive Shift Register	0x00000000
0x31003434	UART1_TXCNT	UART1 Transmit Counter Register	0x00000000
0x31003438	UART1_RXCNT	UART1 Receive Counter Register	0x00000000

Table A-190: ADSP-2159x UART2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31003804	UART2_CTL	UART2 Control Register	0x00000000
0x31003808	UART2_STAT	UART2 Status Register	0x000000A0
0x3100380C	UART2_SCR	UART2 Scratch Register	0x00000000
0x31003810	UART2_CLK	UART2 Clock Rate Register	0x0000FFFF
0x31003814	UART2_IMSK	UART2 Interrupt Mask Register	0x00000000
0x31003818	UART2_IMSK_SET	UART2 Interrupt Mask Set Register	0x00000000
0x3100381C	UART2_IMSK_CLR	UART2 Interrupt Mask Clear Register	0x00000000
0x31003820	UART2_RBR	UART2 Receive Buffer Register	0x00000000
0x31003824	UART2_THR	UART2 Transmit Hold Register	0x00000000
0x31003828	UART2_TAIP	UART2 Transmit Address/Insert Pulse Register	0x00000000
0x3100382C	UART2_TSR	UART2 Transmit Shift Register	0x000007FF
0x31003830	UART2_RSR	UART2 Receive Shift Register	0x00000000
0x31003834	UART2_TXCNT	UART2 Transmit Counter Register	0x00000000
0x31003838	UART2_RXCNT	UART2 Receive Counter Register	0x00000000

Table A-191: ADSP-2159x UART3 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31003C04	UART3_CTL	UART3 Control Register	0x00000000
0x31003C08	UART3_STAT	UART3 Status Register	0x000000A0
0x31003C0C	UART3_SCR	UART3 Scratch Register	0x00000000
0x31003C10	UART3_CLK	UART3 Clock Rate Register	0x0000FFFF
0x31003C14	UART3_IMSK	UART3 Interrupt Mask Register	0x00000000
0x31003C18	UART3_IMSK_SET	UART3 Interrupt Mask Set Register	0x00000000
0x31003C1C	UART3_IMSK_CLR	UART3 Interrupt Mask Clear Register	0x00000000
0x31003C20	UART3_RBR	UART3 Receive Buffer Register	0x00000000
0x31003C24	UART3_THR	UART3 Transmit Hold Register	0x00000000
0x31003C28	UART3_TAIP	UART3 Transmit Address/Insert Pulse Register	0x00000000
0x31003C2C	UART3_TSR	UART3 Transmit Shift Register	0x000007FF
0x31003C30	UART3_RSR	UART3 Receive Shift Register	0x00000000
0x31003C34	UART3_TXCNT	UART3 Transmit Counter Register	0x00000000

Table A-191: ADSP-2159x UART3 MMR Register Addresses (Continued)

Memory Map-ped Address	Register Name	Description	Reset Value
0x31003C38	UART3_RXCNT	UART3 Receive Counter Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses

Memory Map-ped Address	Register Name	Description	Reset Value
0x310C5000	USBC0_OTG_CTL	USBC0 OTG Control and Status Register	0x00010000
0x310C5004	USBC0_OTG_IRQ	USBC0 OTG Interrupt Register	0x00000000
0x310C5008	USBC0_AHB_CFG	USBC0 AHB Configuration Register	0x00000000
0x310C500C	USBC0_CFG	USBC0 USB Configuration Register	0x00001410
0x310C5010	USBC0_RST_CTL	USBC0 Reset Register	0x80000000
0x310C5014	USBC0_ISTAT	USBC0 Interrupt Status Register	0x04000020
0x310C5018	USBC0_IMSK	USBC0 Interrupt Mask Register	0x00000000
0x310C501C	USBC0_RXDBG_STAT	USBC0 Receive Status Debug Read Register	0x00000000
0x310C5020	USBC0_RXDATA_STAT	USBC0 Receive Status Read/Pop Register	0x00000000
0x310C5024	USBC0_RXFIFOSZ	USBC0 Receive FIFO Size Register	0x00000800
0x310C5028	USBC0_RXFIFOSZ_NP	USBC0 Non-periodic Transmit FIFO Size Register	0x08000800
0x310C502C	USBC0_TXFIFO_STAT_NP	USBC0 Non-periodic Transmit FIFO/Queue Status Register	0x00080800
0x310C5034	USBC0_PHYIF_CTL	USBC0 PHY Interface Control Register	0x00000000
0x310C5040	USBC0_MODID	USBC0 Module ID Register	0x4F54400A
0x310C5044	USBC0_HWCFG1	USBC0 User Hardware Configuration 1 Register	0x00000000
0x310C5048	USBC0_HWCFG2	USBC0 User Hardware Configuration 2 Register	0x228BEC90
0x310C504C	USBC0_HWCFG3	USBC0 User Hardware Configuration 3 Register	0x0FA006E8
0x310C5050	USBC0_HWCFG4	USBC0 User Hardware Configuration 4 Register	0xCE002020
0x310C505C	USBC0_DFIFO_CFG	USBC0 DFIFO Configuration Register	0x0FA01000
0x310C5100	USBC0_TXFIFOSZ_PER_H	USBC0 Host Periodic Transmit FIFO Size Register	0x04001000
0x310C5104	USBC0_TXFIFOSZ1_IEP_H	USBC0 Device IN Endpoint 1 Transmit FIFO Size Register	0x01000900
0x310C5108	USBC0_TXFIFOSZ2_IEP_H	USBC0 Device IN Endpoint 2 Transmit FIFO Size Register	0x01000A00
0x310C510C	USBC0_TXFIFOSZ3_IEP_H	USBC0 Device IN Endpoint 3 Transmit FIFO Size Register	0x01000B00
0x310C5400	USBC0_CFG_H	USBC0 Host Configuration Register	0x00000200

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C5404	USBC0_FIR_H	USBC0 Host Frame Interval Register	0x0000EA60
0x310C5408	USBC0_FNUM_H	USBC0 Host Frame number/Frame time remaining Register	0x00003FFF
0x310C5414	USBC0_ISTAT_H	USBC0 Host All Channels Interrupt Register	0x00000000
0x310C5418	USBC0_IMSK_H	USBC0 Host All Channels Interrupt Mask Register	0x00000000
0x310C541C	USBC0_FL_BADDR_H	USBC0 Host Frame List Base Address Register	0x00000000
0x310C5440	USBC0_PORT_CTL_H	USBC0 Host Port Control and Status Register	0x00000000
0x310C5500	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C5504	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5508	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C550C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5510	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5514	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C551C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5520	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C5524	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5528	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C552C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5530	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5534	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C553C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5540	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C5544	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5548	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C554C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5550	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5554	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C555C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5560	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C5564	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5568	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C556C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5570	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5574	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C557C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5580	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C5584	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5588	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C558C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5590	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5594	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C559C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C55A0	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C55A4	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C55A8	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C55AC	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C55B0	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C55B4	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C55BC	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C55C0	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C55C4	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C55C8	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C55CC	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C55D0	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C55D4	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C55DC	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C55E0	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C55E4	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C55E8	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C55EC	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C55F0	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C55F4	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C55FC	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5600	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C5604	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5608	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C560C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5610	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5614	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C561C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5620	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C5624	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5628	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C562C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5630	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5634	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C563C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5640	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C5644	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5648	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C564C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5650	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5654	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C565C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5660	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C5664	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5668	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C566C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5670	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5674	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C567C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5680	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C5684	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C5688	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C568C	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C5690	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C5694	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C569C	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C56A0	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C56A4	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C56A8	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C56AC	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C56B0	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C56B4	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C56BC	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C56C0	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000
0x310C56C4	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C56C8	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C56CC	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C56D0	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C56D4	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C56DC	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C56E0	USBC0_CHAR[n]_H	USBC0 Host Channel n Characteristics Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C56E4	USBC0_SPLT_CTL[n]_H	USBC0 Host Channel n split control Register	0x00000000
0x310C56E8	USBC0_ISTAT[n]_H	USBC0 Host Channel n Interrupt Status Register	0x00000000
0x310C56EC	USBC0_IMSK[n]_H	USBC0 Host Channel n Interrupt Mask Register	0x00000000
0x310C56F0	USBC0_TSIZ[n]_H	USBC0 Host Channel n Transfer Size Register	0x00000000
0x310C56F4	USBC0_DMA_ADDR[n]_H	USBC0 Host Channel n DMA Address Register	0x00000000
0x310C56FC	USBC0_DMA_BADDR[n]_H	USBC0 Host Channel n DMA Buffer Address Register	0x00000000
0x310C5800	USBC0_CFG_D	USBC0 Device Configuration Register	0x08100000
0x310C5804	USBC0_CTL_D	USBC0 Device Control Register	0x00000002
0x310C5808	USBC0_STAT_D	USBC0 Device Status Register	0x00000002
0x310C5810	USBC0_IMASK_IEP_D	USBC0 Device IN Endpoint common Interrupt mask Register	0x00000000
0x310C5814	USBC0_IMASK_OEP_D	USBC0 Device OUT Endpoint common Interrupt mask Register	0x00000000
0x310C5818	USBC0_ISTAT_D	USBC0 Device All Endpoint Interrupt Status Register	0x00000000
0x310C581C	USBC0_IMSK_D	USBC0 Device All Endpoint Interrupt Mask Register	0x00000000
0x310C5828	USBC0_VBUSDIS_D	USBC0 Device VBUS Discharge time Register	0x000017D7
0x310C582C	USBC0_VBUSPULSE_D	USBC0 Device VBUS pulsing time Register	0x000005B8
0x310C5830	USBC0_THR_CTL_D	USBC0 Device Threshold Control Register	0x0C100020
0x310C5834	USBC0_IMSK_IEP_FEMPT_D	USBC0 Device IN Endpoint FIFO Empty Interrupt Mask Register	0x00000000
0x310C5920	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C5928	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C5930	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C5934	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C5938	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C593C	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C5940	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C5948	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C5950	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C5954	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C5958	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C595C	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C5960	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C5968	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C5970	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C5974	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C5978	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C597C	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C5980	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C5988	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C5990	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C5994	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C5998	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C599C	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C59A0	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C59A8	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C59B0	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C59B4	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C59B8	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C59BC	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C59C0	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C59C8	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C59D0	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C59D4	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C59D8	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C59DC	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C59E0	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C59E8	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C59F0	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C59F4	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C59F8	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C59FC	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C5A00	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C5A08	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C5A10	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C5A14	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C5A18	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C5A1C	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C5A20	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C5A28	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C5A30	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C5A34	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C5A38	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C5A3C	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C5A40	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C5A48	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C5A50	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C5A54	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000
0x310C5A58	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C5A5C	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C5A60	USBC0_CTL_IEP[n]_D	USBC0 Device Control IN Endpoint n Control Register	0x00000000
0x310C5A68	USBC0_ISTAT_IEP[n]_D	USBC0 Device Control IN Endpoint n Interrupt Control Register	0x00000080
0x310C5A70	USBC0_TSIZ_IEP[n]_D	USBC0 Device Control IN Endpoint n Transfer size Register	0x00000000
0x310C5A74	USBC0_DMA_ADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Address Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C5A78	USBC0_TXFSTAT_IEP[n]_D	USBC0 Device Control IN Endpoint Transmit FIFO Status Register	0x00000800
0x310C5A7C	USBC0_DMA_BADDR_IEP[n]_D	USBC0 Device Control IN Endpoint n DMA Buffer Address Register	0x00000000
0x310C5B20	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5B28	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5B30	USBC0_TSI_Z_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5B34	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5B3C	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5B40	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5B48	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5B50	USBC0_TSI_Z_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5B54	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5B5C	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5B60	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5B68	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5B70	USBC0_TSI_Z_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5B74	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5B7C	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5B80	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5B88	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5B90	USBC0_TSI_Z_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5B94	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5B9C	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5BA0	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5BA8	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C5BB0	USBC0_TSIz_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5BB4	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5BBC	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5BC0	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5BC8	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5BD0	USBC0_TSIz_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5BD4	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5BDC	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5BE0	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5BE8	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5BF0	USBC0_TSIz_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5BF4	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5BFC	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5C00	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5C08	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5C10	USBC0_TSIz_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5C14	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5C1C	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5C20	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5C28	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5C30	USBC0_TSIz_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5C34	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5C3C	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5C40	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000

Table A-192: ADSP-2159x USBC0 MMR Register Addresses (Continued)

Memory Map- ped Address	Register Name	Description	Reset Value
0x310C5C48	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5C50	USBC0_TSIz_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5C54	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5C5C	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5C60	USBC0_CTL_OEP[n]_D	USBC0 Device OUT Endpoint n Control Register	0x00000000
0x310C5C68	USBC0_ISTAT_OEP[n]_D	USBC0 Device OUT Endpoint n Interrupt Register	0x00000000
0x310C5C70	USBC0_TSIz_OEP[n]_D	USBC0 Device OUT Endpoint n Transfer Size Register	0x00000000
0x310C5C74	USBC0_DMA_ADDR_OEP[n]_D	USBC0 Device OUT Endpoint n DMA Address Register	0x00000000
0x310C5C7C	USBC0_DMA_BADDR_OEP[n]_D	USBC0 Device OUT Endpoint n Buffer Address Register	0x00000000
0x310C5E00	USBC0_PWR_CTL	USBC0 Power and clock gating control Register	0x00000000

Table A-193: ADSP-2159x WDOG0 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31008000	WDOG0_CTL	WDOG0 Control Register	0x00000AD0
0x31008004	WDOG0_CNT	WDOG0 Count Register	0x00000000
0x31008008	WDOG0_STAT	WDOG0 Watchdog Timer Status Register	0x00000000
0x3100800C	WDOG0_WIN	WDOG0 Watchdog Timer Window Register	0xFFFFFFFF

Table A-194: ADSP-2159x WDOG1 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31008800	WDOG1_CTL	WDOG1 Control Register	0x00000AD0
0x31008804	WDOG1_CNT	WDOG1 Count Register	0x00000000
0x31008808	WDOG1_STAT	WDOG1 Watchdog Timer Status Register	0x00000000
0x3100880C	WDOG1_WIN	WDOG1 Watchdog Timer Window Register	0xFFFFFFFF

Table A-195: ADSP-2159x WDOG2 MMR Register Addresses

Memory Map- ped Address	Register Name	Description	Reset Value
0x31009000	WDOG2_CTL	WDOG2 Control Register	0x00000AD0
0x31009004	WDOG2_CNT	WDOG2 Count Register	0x00000000
0x31009008	WDOG2_STAT	WDOG2 Watchdog Timer Status Register	0x00000000
0x3100900C	WDOG2_WIN	WDOG2 Watchdog Timer Window Register	0xFFFFFFFF